The Complete System Design Knowledge Base

From first principles to expert practice

Built from the manuscript sources · 38 chapter(s) in this edition

Part I — Foundations of System Design Thinking

Chapter 1 — What Is System Design?

1.1 Learning objectives
1.2 Prerequisites
1.3 Definition of system design
1.4 Software architecture
1.5 High-level vs low-level design
1.6 Functional requirements
1.7 Non-functional requirements (the "-ilities" catalog)
1.8 Architectural decision-making and ADRs
1.9 Why system design matters (cost of wrong decisions)
1.10 How system design is practiced in industry
1.11 How system design interviews work
1.12 Common beginner mistakes
1.13 Summary and revision notes
1.14 Knowledge checkpoint
1.15 Interview questions
1.16 Practical exercises
1.17 Design challenges
1.18 References
1.19 Open verification items

Chapter 2 — How to Think Like a System Designer

2.1 Learning objectives
2.2 Prerequisites
2.3 Decomposition into components
2.4 System boundaries and context diagrams
2.5 Understanding and negotiating requirements
2.6 Identifying bottlenecks
2.7 Making and stating assumptions
2.8 Evaluating trade-offs
2.9 Designing for failure
2.10 Reasoning about scale
2.11 Back-of-the-envelope capacity estimation
2.12 Architecture documentation
2.13 Summary and revision notes
2.14 Knowledge checkpoint
2.15 Interview questions
2.16 Practical exercises
2.17 Design challenges
2.18 References
2.19 Open verification items

Chapter 3 — The System Design Learning Method

3.1 Learning objectives
3.2 Prerequisites
3.3 The ten-step concept-mastery loop

3.4 Active recall, spaced repetition, the Feynman technique, interleaving and deliberate practice
3.5 Reading papers and engineering blogs
3.6 Drawing architecture from memory
3.7 Building simplified implementations
3.8 Beginner methodology: a 12-month path
3.9 Advanced methodology for experienced engineers
3.10 Tracking progress
3.11 Summary and revision notes
3.12 Knowledge checkpoint
3.13 Interview questions
3.14 Practical exercises
3.15 Design challenges
3.16 References
3.17 Open verification items

Chapter 4 — Quantitative Foundations for System Design

4.1 Learning objectives
4.2 Prerequisites
4.3 Exponents, powers of two and powers of ten
4.4 Logarithms
4.5 Units: bits, bytes, rates and prefixes
4.6 Probability essentials
4.7 Statistics: describing measurements
4.8 Percentiles and tail latency
4.9 Basic linear algebra
4.10 Queueing theory
4.11 Little’s Law
4.12 Amdahl’s Law and Gustafson’s Law
4.13 The Universal Scalability Law
4.14 Availability arithmetic
4.15 Reliability: MTBF, MTTR and failure rates
4.16 Durability and replication
4.17 Latency numbers every engineer should know
4.18 Quantitative relationships that recur in system design
4.19 Summary and revision notes
4.20 Knowledge checkpoint
4.21 Interview questions
4.22 Practical exercises
4.23 Design challenges
4.24 References
4.25 Open verification items

Part II — Computer Science Foundations

Chapter 5 — Computer Architecture

5.1 Learning objectives
5.2 Prerequisites
5.3 The CPU: cores, registers, clock and the instruction pipeline
5.4 CPU caches

5.5 RAM and the memory hierarchy
5.6 Storage devices
5.7 I/O operations
5.8 CPU-bound versus I/O-bound workloads
5.9 Memory latency and bandwidth
5.10 NUMA
5.11 SIMD and accelerators
5.12 Throughput versus latency at the hardware level
5.13 Summary and revision notes
5.14 Knowledge checkpoint
5.15 Interview questions
5.16 Practical exercises
5.17 Design challenges
5.18 References
5.19 Open verification items

Chapter 6 — Operating Systems

6.1 Learning objectives
6.2 Prerequisites
6.3 Processes and threads
6.4 Context switching and its cost
6.5 CPU scheduling
6.6 Virtual memory, paging and the page cache
6.7 System calls and the kernel/user boundary
6.8 Concurrency versus parallelism
6.9 Race conditions and critical sections
6.10 Synchronization primitives
6.11 Deadlock, livelock, starvation and priority inversion
6.12 Inter-process communication
6.13 I/O models: from blocking to io_uring
6.14 File systems, journaling and fsync
6.15 Concurrency models in practice
6.16 Summary and revision notes
6.17 Knowledge checkpoint
6.18 Interview questions
6.19 Practical exercises
6.20 Design challenges
6.21 References
6.22 Open verification items

Chapter 7 — Networking Fundamentals

7.1 Learning objectives
7.2 Prerequisites
7.3 OSI and TCP/IP models
7.4 IP addressing, IPv4/IPv6, subnets and CIDR
7.5 Routing basics, BGP and anycast
7.6 DNS: the Internet’s directory
7.7 TCP: reliable byte streams over an unreliable network

7.8 UDP and when to use it
7.9 HTTP/1.1, HTTP/2, HTTP/3 and QUIC
7.10 TLS 1.3 and HTTPS
7.11 WebSockets (introduction)
7.12 Forward proxies and reverse proxies
7.13 NAT
7.14 Firewalls and security groups
7.15 Latency, bandwidth, the bandwidth-delay product and packet loss
7.16 Connection pooling and keep-alive
7.17 Datacenter networking
7.18 What happens when you type a URL
7.19 Summary and revision notes
7.20 Knowledge checkpoint
7.21 Interview questions
7.22 Practical exercises
7.23 Design challenges
7.24 References
7.25 Open verification items

Chapter 8 — Web Application Fundamentals

8.1 Learning objectives
8.2 Prerequisites
8.3 Client–server architecture
8.4 Frontend and backend; SPA, SSR and mobile clients
8.5 The request–response lifecycle end to end
8.6 REST API design
8.7 GraphQL
8.8 gRPC and RPC
8.9 Serialization: JSON, Protocol Buffers, Avro and Thrift
8.10 Cookies and sessions
8.11 Authentication
8.12 Authorization: RBAC, ABAC and ReBAC
8.13 JSON Web Tokens (JWT)
8.14 OAuth 2.0 and OpenID Connect
8.15 API versioning and compatibility
8.16 API documentation: OpenAPI
8.17 Summary and revision notes
8.18 Knowledge checkpoint
8.19 Interview questions
8.20 Practical exercises
8.21 Design challenges
8.22 References
8.23 Open verification items

Chapter 9 — Real-Time Communication

9.1 Learning objectives
9.2 Prerequisites
9.3 Short polling

9.4 Long polling
9.5 Server-Sent Events
9.6 WebSockets in depth
9.7 WebRTC basics
9.8 Mobile push: APNs and FCM
9.9 Connection management at scale
9.10 Choosing a real-time technique
9.11 Summary and revision notes
9.12 Knowledge checkpoint
9.13 Interview questions
9.14 Practical exercises
9.15 Design challenges
9.16 References
9.17 Open verification items

Chapter 10 — Security Fundamentals for System Designers

10.1 Learning objectives
10.2 Prerequisites
10.3 Threat modeling
10.4 Cryptography essentials
10.5 PKI and certificates
10.6 Password storage
10.7 The OWASP Top 10 in architectural terms
10.8 Secrets management
10.9 Encryption at rest and in transit, KMS and envelope encryption
10.10 Network security, zero trust and mTLS
10.11 DDoS basics
10.12 Audit logging and tamper evidence
10.13 A security checklist for every design
10.14 Summary and revision notes
10.15 Knowledge checkpoint
10.16 Interview questions
10.17 Practical exercises
10.18 Design challenges
10.19 References
10.20 Open verification items

Part III — Data Systems

Chapter 11 — Relational Databases

11.1 Learning objectives
11.2 Prerequisites
11.3 The relational model: tables, rows and columns
11.4 Keys: primary, foreign, natural and surrogate
11.5 Normalization and denormalization
11.6 SQL and joins
11.7 Indexes
11.8 B-trees and B+ trees
11.9 Query execution and plans

11.10 Query optimization
11.11 Transactions
11.12 ACID
11.13 Isolation levels and anomalies
11.14 Multi-version concurrency control (MVCC)
11.15 Locking
11.16 PostgreSQL versus MySQL
11.17 Data modeling for a real application
11.18 Summary and revision notes
11.19 Knowledge checkpoint
11.20 Interview questions
11.21 Practical exercises
11.22 Design challenges
11.23 References
11.24 Open verification items

Chapter 12 — NoSQL Databases

12.1 Learning objectives
12.2 Prerequisites
12.3 Why NoSQL emerged
12.4 Key-value stores
12.5 Document databases
12.6 Wide-column stores
12.7 Graph databases
12.8 Time-series databases
12.9 Search engines as databases
12.10 Vector databases
12.11 NewSQL
12.12 Selection framework and SQL versus NoSQL comparison
12.13 Polyglot persistence
12.14 Summary and revision notes
12.15 Knowledge checkpoint
12.16 Interview questions
12.17 Practical exercises
12.18 Design challenges
12.19 References
12.20 Open verification items

Chapter 13 — Storage Engine Internals

13.1 Learning objectives
13.2 Prerequisites
13.3 Pages and blocks
13.4 Buffer pools
13.5 Write-ahead logging and recovery
13.6 B-tree storage engines
13.7 Log-structured storage
13.8 LSM trees: memtables and SSTables
13.9 Compaction strategies

13.10 Bloom filters in storage
13.11 Read, write and space amplification
13.12 Durability: fsync, checksums and torn writes
13.13 Columnar storage
13.14 Case studies: InnoDB, RocksDB and the PostgreSQL heap
13.15 Summary and revision notes
13.16 Knowledge checkpoint
13.17 Interview questions
13.18 Practical exercises
13.19 Design challenges
13.20 References
13.21 Open verification items

Chapter 14 — Database Scaling: Replication, Partitioning and Sharding

14.1 Learning objectives
14.2 Prerequisites
14.3 Read replicas and primary–replica architecture
14.4 Synchronous, asynchronous and semi-synchronous replication
14.5 Multi-primary replication
14.6 Partitioning: range, hash, list and directory-based
14.7 Sharding strategies and choosing shard keys
14.8 Consistent hashing and virtual nodes
14.9 Rebalancing
14.10 Hot partitions and skew
14.11 Secondary indexes in partitioned databases
14.12 Cross-shard queries and distributed transactions
14.13 Connection pooling and proxies
14.14 Summary and revision notes
14.15 Knowledge checkpoint
14.16 Interview questions
14.17 Practical exercises
14.18 Design challenges
14.19 References
14.20 Open verification items

Chapter 15 — Object, Block and File Storage

15.1 Learning objectives
15.2 Prerequisites
15.3 Block, file and object storage compared
15.4 Object storage architecture (the S3 model)
15.5 Distributed file systems: GFS, HDFS, Colossus and Tectonic
15.6 Replication versus erasure coding
15.7 Metadata services
15.8 Multipart upload, chunking and deduplication
15.9 Storage tiers and lifecycle policies
15.10 Summary and revision notes
15.11 Knowledge checkpoint
15.12 Interview questions

15.13 Practical exercises
15.14 Design challenges
15.15 References
15.16 Open verification items

Chapter 16 — Search Systems

16.1 Learning objectives
16.2 Prerequisites
16.3 Inverted indexes
16.4 Tokenization and analysis
16.5 Ranking: TF-IDF and BM25
16.6 Index construction and segment merging
16.7 Distributed search
16.8 Elasticsearch and OpenSearch architecture
16.9 Typeahead and autocomplete
16.10 Near-real-time indexing
16.11 Beyond lexical matching: a bridge to semantic and hybrid search
16.12 Summary and revision notes
16.13 Knowledge checkpoint
16.14 Interview questions
16.15 Practical exercises
16.16 Design challenges
16.17 References
16.18 Open verification items

Chapter 17 — Data Structures and Algorithms for Scale

17.1 Learning objectives
17.2 Prerequisites
17.3 Hash functions at scale
17.4 Bloom filters and cuckoo filters
17.5 HyperLogLog
17.6 Count-Min sketch and heavy hitters
17.7 Skip lists
17.8 Merkle trees
17.9 Geospatial indexing
17.10 Unique ID generation
17.11 Tries
17.12 Reservoir sampling and streaming algorithms
17.13 Summary and revision notes
17.14 Knowledge checkpoint
17.15 Interview questions
17.16 Practical exercises
17.17 Design challenges
17.18 References
17.19 Open verification items

Part IV — Scalability and Performance

Chapter 18 — Scalability

18.1 Learning objectives
18.2 Prerequisites
18.3 Vertical scaling
18.4 Horizontal scaling
18.5 Stateless vs stateful services
18.6 Load distribution
18.7 Bottleneck identification
18.8 Scaling application servers
18.9 Scaling databases (overview)
18.10 The scale cube
18.11 Capacity planning
18.12 Summary and revision notes
18.13 Knowledge checkpoint
18.14 Interview questions
18.15 Practical exercises
18.16 Design challenges
18.17 References
18.18 Open verification items

Chapter 19 — Load Balancing

19.1 Learning objectives
19.2 Prerequisites
19.3 Layer 4 versus layer 7 load balancing
19.4 Load-balancing algorithms
19.5 Health checks
19.6 Session affinity
19.7 Active–passive and active–active balancer pairs
19.8 Balancer failure and redundancy
19.9 DNS and global server load balancing
19.10 Client-side load balancing
19.11 Choosing a load balancer: products compared
19.12 Summary and revision notes
19.13 Knowledge checkpoint
19.14 Interview questions
19.15 Practical exercises
19.16 Design challenges
19.17 References
19.18 Open verification items

Chapter 20 — Caching

20.1 Learning objectives
20.2 Prerequisites
20.3 Cache fundamentals and hit ratio
20.4 CPU cache versus application cache
20.5 Where to cache
20.6 Redis and Memcached internals
20.7 Caching patterns: cache-aside, read-through, write-through, write-back, write-around
20.8 Time to live (TTL)

20.9 Eviction: LRU, LFU, ARC and TinyLFU
20.10 Cache invalidation
20.11 Cache stampede
20.12 Cache penetration
20.13 Cache avalanche
20.14 Distributed caching and consistent hashing
20.15 Hot keys
20.16 Redis versus Memcached
20.17 Summary and revision notes
20.18 Knowledge checkpoint
20.19 Interview questions
20.20 Practical exercises
20.21 Design challenges
20.22 References
20.23 Open verification items

Chapter 21 — Content Delivery Networks

21.1 Learning objectives
21.2 Prerequisites
21.3 CDN architecture
21.4 Edge, mid-tier and origin
21.5 Cache hierarchy and origin shielding
21.6 Cache keys, HTTP caching headers and invalidation
21.7 Static versus dynamic content
21.8 Video delivery
21.9 Edge computing on CDNs
21.10 Netflix Open Connect
21.11 CDN security
21.12 Summary and revision notes
21.13 Knowledge checkpoint
21.14 Interview questions
21.15 Practical exercises
21.16 Design challenges
21.17 References
21.18 Open verification items

Chapter 22 — API and Traffic Management

22.1 Learning objectives
22.2 Prerequisites
22.3 Rate limiting
22.4 Token bucket
22.5 Leaky bucket
22.6 Fixed window
22.7 Sliding window log and sliding window counter
22.8 Distributed rate limiting
22.9 Throttling and load shedding
22.10 Backpressure
22.11 Timeouts and deadlines

22.12 Retries with exponential backoff and jitter
22.13 Circuit breakers
22.14 Bulkheads
22.15 Idempotency and idempotency keys
22.16 Summary and revision notes
22.17 Knowledge checkpoint
22.18 Interview questions
22.19 Practical exercises
22.20 Design challenges
22.21 References
22.22 Open verification items

Chapter 23 — Performance Engineering

23.1 Learning objectives
23.2 Prerequisites
23.3 Performance methodology
23.4 Profiling CPU and memory
23.5 Load testing and benchmarking pitfalls
23.6 Tail latency at scale
23.7 Hedged and tied requests
23.8 Batching, pipelining and compression
23.9 Garbage collection and runtime effects
23.10 Capacity models from measurements
23.11 Summary and revision notes
23.12 Knowledge checkpoint
23.13 Interview questions
23.14 Practical exercises
23.15 Design challenges
23.16 References
23.17 Open verification items

Part V — Distributed Systems

Chapter 24 — Distributed Systems Fundamentals

24.1 Learning objectives
24.2 Prerequisites
24.3 What is a distributed system?
24.4 Why distributed systems exist
24.5 The eight fallacies of distributed computing
24.6 System models: timing and failures
24.7 Network partitions
24.8 Partial failures, gray failures and timeouts
24.9 Physical clocks: drift, NTP, PTP and skew
24.10 Logical clocks: Lamport clocks
24.11 Vector clocks and version vectors
24.12 Hybrid logical clocks
24.13 The FLP impossibility result
24.14 The end-to-end argument
24.15 Summary and revision notes

24.16 Knowledge checkpoint
24.17 Interview questions
24.18 Practical exercises
24.19 Design challenges
24.20 References
24.21 Open verification items

Chapter 25 — Consistency and Availability

25.1 Learning objectives
25.2 Prerequisites
25.3 Linearizability
25.4 Sequential consistency
25.5 Causal consistency
25.6 Session guarantees
25.7 Eventual consistency
25.8 The CAP theorem
25.9 PACELC
25.10 Consistency trade-offs in real databases
25.11 Jepsen-style testing
25.12 Summary and revision notes
25.13 Knowledge checkpoint
25.14 Interview questions
25.15 Practical exercises
25.16 Design challenges
25.17 References
25.18 Open verification items

Chapter 26 — Consensus and Coordination

26.1 Learning objectives
26.2 Prerequisites
26.3 The consensus problem
26.4 Quorums
26.5 Leader election
26.6 Paxos
26.7 Raft
26.8 ZooKeeper and ZAB
26.9 etcd
26.10 Distributed locks
26.11 Leases
26.12 Split brain
26.13 Fencing tokens
26.14 Coordination services in practice
26.15 Summary and revision notes
26.16 Knowledge checkpoint
26.17 Interview questions
26.18 Practical exercises
26.19 Design challenges
26.20 References

26.21 Open verification items

Chapter 27 — Replication and Fault Tolerance

27.1 Learning objectives
27.2 Prerequisites
27.3 Replication strategies overview
27.4 Primary–backup replication
27.5 Chain replication
27.6 Multi-leader replication
27.7 Leaderless replication
27.8 Read and write quorums: R + W > N
27.9 Sloppy quorums and hinted handoff
27.10 Failure detection
27.11 Failover
27.12 Recovery
27.13 Anti-entropy and read repair
27.14 Conflict resolution: LWW, version vectors and CRDTs
27.15 Summary and revision notes
27.16 Knowledge checkpoint
27.17 Interview questions
27.18 Practical exercises
27.19 Design challenges
27.20 References
27.21 Open verification items

Chapter 28 — Distributed Transactions

28.1 Learning objectives
28.2 Prerequisites
28.3 Why distributed transactions are hard
28.4 Two-phase commit
28.5 Three-phase commit
28.6 Sagas: a preview
28.7 Percolator-style transactions
28.8 Spanner and TrueTime
28.9 Calvin and deterministic databases
28.10 Distributed SQL: CockroachDB, YugabyteDB and TiDB
28.11 Choosing a transaction strategy
28.12 Summary and revision notes
28.13 Knowledge checkpoint
28.14 Interview questions
28.15 Practical exercises
28.16 Design challenges
28.17 References
28.18 Open verification items

Chapter 29 — Distributed Data Processing

29.1 Learning objectives
29.2 Prerequisites
29.3 Batch processing concepts

29.4 MapReduce
29.5 Apache Spark
29.6 Partitioning and shuffle
29.7 Data locality
29.8 Stream processing concepts
29.9 Apache Flink
29.10 Lambda and Kappa architectures
29.11 Batch versus stream; Spark versus Flink
29.12 Summary and revision notes
29.13 Knowledge checkpoint
29.14 Interview questions
29.15 Practical exercises
29.16 Design challenges
29.17 References
29.18 Open verification items

Part VI — Messaging and Event-Driven Systems

Chapter 30 — Messaging Fundamentals

30.1 Learning objectives
30.2 Prerequisites
30.3 Synchronous vs asynchronous communication
30.4 Message queues
30.5 Publish/subscribe
30.6 Event-driven architecture
30.7 Event brokers
30.8 Delivery guarantees
30.9 Ordering
30.10 Retries and poison messages
30.11 Dead-letter queues
30.12 Kafka vs RabbitMQ (and SQS, Pulsar)
30.13 Summary and revision notes
30.14 Knowledge checkpoint
30.15 Interview questions
30.16 Practical exercises
30.17 Design challenges
30.18 References
30.19 Open verification items

Chapter 31 — Kafka Deep Dive

31.1 Learning objectives
31.2 Prerequisites
31.3 The log abstraction
31.4 Brokers and clusters: ZooKeeper mode versus KRaft
31.5 Topics and partitions
31.6 Producers
31.7 Consumers and consumer groups
31.8 Offsets
31.9 Replication, ISR and leader election

31.10 Retention and log compaction
31.11 Delivery semantics and transactions
31.12 Rebalancing protocols
31.13 Performance
31.14 Kafka Streams and Kafka Connect
31.15 Operating Kafka
31.16 Summary and revision notes
31.17 Knowledge checkpoint
31.18 Interview questions
31.19 Practical exercises
31.20 Design challenges
31.21 References
31.22 Open verification items

Chapter 32 — Event-Driven Architecture Patterns

32.1 Learning objectives
32.2 Prerequisites
32.3 Event sourcing
32.4 CQRS
32.5 Transactional outbox
32.6 Change data capture
32.7 The saga pattern
32.8 Choreography versus orchestration
32.9 Exactly-once processing and its limits
32.10 Idempotent consumers
32.11 Workflow engines
32.12 Summary and revision notes
32.13 Knowledge checkpoint
32.14 Interview questions
32.15 Practical exercises
32.16 Design challenges
32.17 References
32.18 Open verification items

Part VII — Software Architecture and Design

Chapter 33 — Architecture Patterns

33.1 Learning objectives
33.2 Prerequisites
33.3 Layered architecture
33.4 Monolithic architecture
33.5 Modular monolith
33.6 Service-oriented architecture
33.7 Microservices (overview)
33.8 Event-driven architecture (overview)
33.9 Serverless architecture
33.10 Hexagonal architecture (ports and adapters)
33.11 Clean architecture
33.12 Cell-based architecture

33.13 Monolith versus microservices
33.14 Summary and revision notes
33.15 Knowledge checkpoint
33.16 Interview questions
33.17 Practical exercises
33.18 Design challenges
33.19 References
33.20 Open verification items

Chapter 34 — Microservices

34.1 Learning objectives
34.2 Prerequisites
34.3 Service boundaries
34.4 Domain-driven design
34.5 Bounded contexts and context maps
34.6 API gateways and backends for frontends
34.7 Service discovery
34.8 Inter-service communication
34.9 Distributed tracing (introduction)
34.10 Service mesh
34.11 Data ownership and database-per-service
34.12 Deployment strategies
34.13 Migration from a monolith
34.14 Anti-patterns
34.15 Summary and revision notes
34.16 Knowledge checkpoint
34.17 Interview questions
34.18 Practical exercises
34.19 Design challenges
34.20 References
34.21 Open verification items

Chapter 35 — Object-Oriented Design

35.1 Learning objectives
35.2 Prerequisites
35.3 Classes and objects
35.4 Encapsulation
35.5 Inheritance
35.6 Polymorphism
35.7 Abstraction
35.8 Composition over inheritance
35.9 Interfaces and contracts
35.10 Cohesion and coupling
35.11 Common mistakes
35.12 Summary and revision notes
35.13 Knowledge checkpoint
35.14 Interview questions
35.15 Practical exercises

35.16 Design challenges
35.17 References
35.18 Open verification items

Chapter 36 — SOLID Principles

36.1 Learning objectives
36.2 Prerequisites
36.3 Single Responsibility Principle
36.4 Open/Closed Principle
36.5 Liskov Substitution Principle
36.6 Interface Segregation Principle
36.7 Dependency Inversion Principle
36.8 SOLID at the architecture level
36.9 Summary and revision notes
36.10 Knowledge checkpoint
36.11 Interview questions
36.12 Practical exercises
36.13 Design challenges
36.14 References
36.15 Open verification items

Chapter 37 — Design Patterns

37.1 Learning objectives
37.2 Prerequisites
37.3 Factory
37.4 Abstract factory
37.5 Builder
37.6 Singleton
37.7 Strategy
37.8 Observer
37.9 Adapter
37.10 Decorator
37.11 Proxy
37.12 Command
37.13 Repository
37.14 Dependency injection
37.15 State, Template Method and Chain of Responsibility
37.16 Concurrency patterns
37.17 Choosing a pattern
37.18 Summary and revision notes
37.19 Knowledge checkpoint
37.20 Interview questions
37.21 Practical exercises
37.22 Design challenges
37.23 References
37.24 Open verification items

Chapter 38 — UML and Low-Level Design

38.1 Learning objectives

38.2 Prerequisites
38.3 Class diagrams
38.4 Sequence diagrams
38.5 Component diagrams
38.6 State diagrams
38.7 Activity diagrams
38.8 Object relationships in depth
38.9 The LLD interview method
38.10 LLD exercises
38.11 Summary and revision notes
38.12 Knowledge checkpoint
38.13 Interview questions
38.14 Practical exercises
38.15 Design challenges
38.16 References
38.17 Open verification items

Appendix E — Learning Roadmap and Proficiency Levels

E.1 How to use this roadmap
E.2 The five proficiency levels
E.3 Self-assessment
E.4 Track 1 — Full 12-month program (beginner to Level 4)
E.5 Track 2 — 16-week interview track (Level 2–3 engineer → senior interviews)
E.6 Track 3 — 24-week ML/AI infrastructure track (software engineer → ML/AI systems)
E.7 Monthly milestone review template

Appendix A — Glossary

A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
P
Q
R
S
T
U

V
W
X
Z

Part I — Foundations of System Design Thinking

Chapter 1 — What Is System Design?

> **Level:** 1 · **Track:** Shared · **Estimated reading time:** 3.5 h

Every application you have ever used — a photo app, a bank, a search engine — is two things at once. It is *code*, which tells a computer what to do, and it is a *system*, which decides where that code runs, where its data lives, how many copies of each exist, what happens when a machine dies at 3 a.m., and how the whole thing keeps working when ten times more people show up next month. This chapter is about the second thing.

We will follow one fictional product through the whole chapter: **PhotoShare**, a simple photo-sharing app in which people upload photos, follow friends and scroll a feed. PhotoShare starts life on a single rented server with a thousand users. By the end of the chapter it serves ten million daily users across multiple machines and data centers. At every stage you will see *why* the design had to change, *which requirement* forced the change, and *what it cost*. That progression — requirements forcing decisions, decisions carrying costs — is system design.

1.1 Learning objectives

After this chapter you should be able to:

- Define *system design*, *software architecture*, *high-level design* and *low-level design*, and explain how they differ and overlap.
- Distinguish functional requirements from non-functional requirements, and write each in a testable form.
- Name at least fifteen quality attributes (“-ilities”), state a measurable metric for each, and identify common tensions between them.
- Convert an availability target (for example 99.9%) into an allowed-downtime budget, and compute the availability of components in series.
- Estimate average and peak request rates, storage growth and network egress from a user count, using explicit assumptions.
- Write an Architecture Decision Record (ADR) in Nygard’s format and use a weighted decision matrix to compare options.
- Explain why early architectural decisions are expensive to reverse, using a concrete migration-cost calculation.
- Describe how design documents, design reviews and RFC processes work in industry.

- Describe the structure of a 45–60-minute system design interview and what interviewers evaluate.
- Recognize and avoid the twelve most common beginner mistakes in system design.

1.2 Prerequisites

This chapter assumes no prior system design knowledge. Helpful but not required:

- Basic programming in any language — so that “a function”, “a class” and “a web request” are familiar words.
- A rough idea of what happens when a browser loads a web page — treated fully in §8.5 (The request–response lifecycle end to end); here we only need “a client sends a request, a server sends a response”.
- A rough idea of what a database is — treated fully in Ch 11 (Relational Databases); here we only need “a program that stores data durably and answers queries about it”.

Every other term is defined when it first appears.

1.3 Definition of system design

1.3.1 What a “system” is

DEFINITION

Definition — System. A set of interacting **components** (programs, machines, data stores, networks and people) that together deliver a capability no single component delivers alone. Components interact only through **interfaces**: agreed formats and protocols for exchanging requests, responses and data.

DEFINITION

Definition — Component. A unit of a system with a clear responsibility and an interface — for example a web service, a database, a cache, a message queue, a load balancer or a third-party payment provider.

First-principles explanation

Think of a busy restaurant. A single recipe is like a single function in code: it turns inputs (ingredients) into an output (a dish). But a restaurant is not a recipe. It is a *system*: a front-of-house that takes orders, a ticket rail that queues them, stations for grill and salad, a walk-in fridge (storage), a dishwasher (cleanup), a manager who notices when

the grill cook is overwhelmed, and a supplier who delivers flour every Tuesday. The food can be excellent and the restaurant can still fail — because orders get lost, the fridge breaks, or Saturday night brings three times more guests than the kitchen can handle.

Software is the same. The code that resizes a photo can be perfect, and PhotoShare can still fail because the disk filled up, the only server rebooted during a deploy, or a celebrity's post caused a thousand times the normal traffic in ten seconds.

1.3.2 System design

> DEFINITION
>
> **Definition — System design.** The process of deciding the components of a software system, their responsibilities, their interfaces, how data flows and is stored between them, and how they are deployed — so that the system meets its functional requirements *and* its non-functional requirements (performance, scalability, availability, durability, security, cost and others) under stated assumptions about load and failure. The output of the process is also called the *system design*: a set of decisions, diagrams and documents.

Two parts of the definition deserve emphasis.

1. **"So that it meets its requirements."** A design is never good or bad in the abstract — only good or bad *for a stated set of requirements*. A single server is an excellent design for 1,000 users and a terrible one for 10 million.
2. **"Under stated assumptions about load and failure."** Designers must say out loud how much traffic they expect and what kinds of failure they expect to survive. A design that is silent about these is not finished.

Why does this exist?

For most of computing history, a program ran on one machine. Today almost any useful service must handle more data, more users and more failure than one machine can. A modern commodity server fails on the order of once every few years; a fleet of thousands therefore sees failures every day Barroso et al. 2018. Users are spread across continents, and light in optical fiber needs roughly 5 ms per 1,000 km one way, so a single location cannot be fast for everyone (see §7.15 (Latency, bandwidth, BDP, packet loss)). Regulations require some data to stay in certain countries. Each of these forces pushes the system apart into multiple components, and the moment you have multiple components you must decide how they divide the work, how they talk, and what happens when one of them stops. System design exists because those decisions are hard to get right, expensive to get wrong and impossible to avoid.

How it works internally

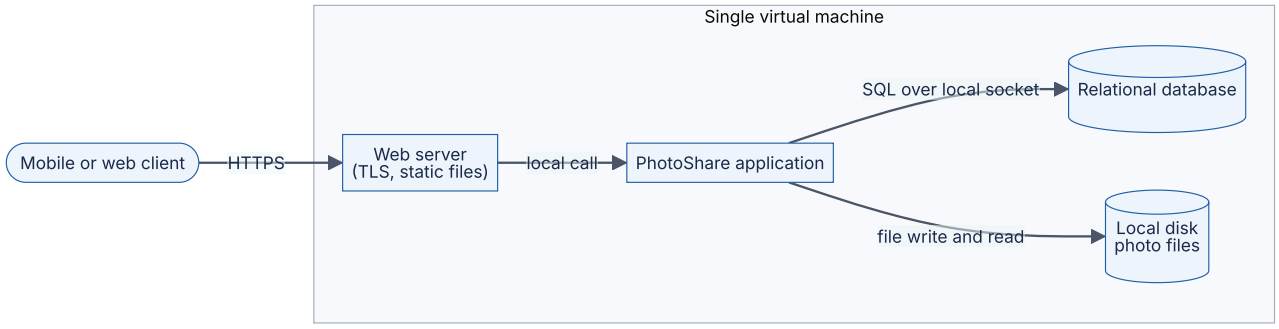
System design is iterative, but in practice most designers follow a loop like this:

1. **Understand the problem.** Who are the users? What must the system do (functional requirements, §1.6)? How well must it do it (non-functional requirements, §1.7)?
2. **Quantify.** Estimate users, request rates, data sizes and growth (back-of-the-envelope estimation; see §2.11 and Ch 89).
3. **Propose a simple design.** Start with the smallest design that could work: a client, a service, a database.
4. **Stress the design against the requirements.** Where does it break first — CPU, memory, disk, network, a single point of failure, a consistency rule? (See §2.6 (Identifying bottlenecks).)
5. **Evolve the design.** Add or change components to fix the weakest point, noting what each change costs.
6. **Record decisions** and their rationale (ADRs, §1.8), and communicate the design (design docs, §1.10).
7. **Validate in production** with monitoring, load tests and incidents, and feed what you learn back into step 1.

Architecture diagram

Here is PhotoShare on launch day. Everything runs on one rented virtual machine.

Figure 1.1 — PhotoShare v0: a single-server design.



Reading the diagram.

- **Components.** A *client* (the mobile app or browser), a *web server* that terminates HTTPS and serves static files, the *PhotoShare application* (business logic: sign-up, upload, feed), a *relational database* holding users, follows and photo metadata, and the *local disk* holding the photo files themselves.
- **Protocols.** The client speaks HTTPS to the web server. Inside the machine, the application talks to the database over a local socket using SQL and reads and writes photo files through the operating system's file system.

- **Direction of data flow.** Uploads flow left to right: client → web server → application → database row plus file on disk. Feed requests flow the same way and responses return along the same path.
- **Synchronous vs asynchronous.** Every arrow is solid: every step is synchronous. The client waits while the application stores the photo and generates any thumbnails.
- **Storage boundaries.** All state — rows and files — lives on one machine's disk. The *subgraph* is the deployment boundary: everything inside shares one fate.
- **Failure points.** The virtual machine is a **single point of failure (SPOF)**: if it reboots, crashes, or its disk fails, PhotoShare is completely down, and photos may be lost if there is no backup.
- **Scaling options.** The only option without redesign is **vertical scaling** — renting a bigger machine (see §18.3). That buys time but has a ceiling and does not remove the SPOF.

This design is not a mistake. For a launch with 1,000 users it is the *right* design: cheap, simple, and easy to change. The rest of the chapter shows which requirements eventually break it.

Mathematical foundations

The first quantitative question in any design is "how much load?" The simplest useful formula converts daily activity into a request rate.

FORMULA

Average and peak request rate

$$\text{RPS}_{\text{avg}} = \frac{\text{DAU} \times r}{86,400} \qquad \text{RPS}_{\text{peak}} = k \times \text{RPS}_{\text{avg}}$$

Symbol	Meaning	Units
RPS_{avg}	Average request rate over a day	requests/second
DAU	Daily active users	users
r	Requests (of the type being estimated) per user per day	requests/user/day
$86,400$	Seconds in a day	seconds/day
k	Peak factor: ratio of the busiest period's rate to the daily average (dimensionless; 2–5 is a common assumption for consumer apps)	—
RPS_{peak}	Request rate during the busiest period	requests/second

RPS (requests per second) is the canonical unit in this book; you will also see **QPS** (queries per second) used as a synonym.

> WORKED EXAMPLE
>
> **Worked example — PhotoShare on launch day.**
>
> Assumptions (stated, as always): 1,000 DAU; each user views 50 photos per day; peak factor $k = 3$.
>
> 1. Daily photo views: $1,000 \times 50 = 50,000$ views/day.
> 2. Average rate: $50,000/86,400 \approx 0.58$ RPS.
> 3. Peak rate: $3 \times 0.58 \approx 1.7$ RPS.
>
> **Interpretation.** Under two requests per second at peak. A single small virtual machine running an ordinary web framework typically handles hundreds of such requests per second (an order-of-magnitude assumption; measure your own stack, see §23.10).
>
> **Engineering implication.** Any design more complex than Figure 1.1 would be *over-engineering* at this scale: more cost, more moving parts, more ways to fail, and no benefit users can feel.

Code example

Estimation is arithmetic, but writing it as a tiny program makes your assumptions explicit and easy to change. The same script is reused in §1.9 when PhotoShare grows.

```
        """Back-of-the-envelope estimator for the PhotoShare running example.

        Uses SI units: 1 MB = 10**6 bytes, 1 TB = 10**12 bytes.
        """
        from dataclasses import dataclass

        SECONDS_PER_DAY = 86_400


        @dataclass
        class Workload:
            dau: int                  # daily active users
            views_per_user: float     # photo views per user per day
            uploads_per_user: float   # photo uploads per user per day
            stored_bytes_per_upload: int  # original + thumbnails, bytes
            served_bytes_per_view: int    # bytes sent to the client per view
            peak_factor: float = 3.0      # peak-hour rate / daily average rate


        def estimate(w: Workload) -> dict:
            views_per_day = w.dau * w.views_per_user
            uploads_per_day = w.dau * w.uploads_per_user
            read_rps = views_per_day / SECONDS_PER_DAY
            write_rps = uploads_per_day / SECONDS_PER_DAY
            storage_per_day = uploads_per_day * w.stored_bytes_per_upload
            egress_bits_per_s = views_per_day * w.served_bytes_per_view * 8 /
SECONDS_PER_DAY
            return {
                "read_rps_avg": read_rps,
                "read_rps_peak": read_rps * w.peak_factor,
                "write_rps_avg": write_rps,
                "write_rps_peak": write_rps * w.peak_factor,
                "storage_TB_per_day": storage_per_day / 1e12,
                "storage_PB_per_year": storage_per_day * 365 / 1e15,
                "egress_Gbps_avg": egress_bits_per_s / 1e9,
                "egress_Gbps_peak": egress_bits_per_s * w.peak_factor / 1e9,
            }


        if __name__ == "__main__":
            for label, dau in [("v0 launch", 1_000), ("v3 scale", 10_000_000)]:
                w = Workload(dau=dau, views_per_user=50, uploads_per_user=0.2,
                             stored_bytes_per_upload=2_500_000,
                             served_bytes_per_view=200_000)
                print(label)
                for key, value in estimate(w).items():
                    print(f"  {key:>20}: {value:,.3f}")
```

What to notice:

- Every assumption is a named field with a unit in the comment. When a reviewer disagrees with “50 views per user”, they change one number, not your reasoning.
- The peak factor is explicit. Systems are sized for the peak, not the average.

- Egress is converted from bytes to **bits** (`* 8`) because network links are rated in bits per second.
- Running it prints about 0.58 average read RPS for v0 and about 5,787 average (17,361 peak) read RPS, 5 TB/day of new storage and about 9.3 Gbit/s average egress for v3. Those v3 numbers drive most of §1.9.

Real-world industry applications

> INDUSTRY EXAMPLE
>
> **Instagram (2011).** About a year after launch, Instagram's engineering team published a description of its stack: a load balancer in front of Django application servers, PostgreSQL for data (partitioned across multiple database instances, with replicas in a different availability zone), Redis and Memcached for feeds and caching, photos stored in Amazon S3 and served through a CDN, and a task queue for asynchronous work. Their stated principles were to keep things simple, avoid reinventing the wheel, and use proven technologies. The problem (rapid growth with a tiny team) drove an architecture built mostly from well-understood, managed components rather than custom infrastructure. The trade-off: less control and some vendor dependence, in exchange for a very small operations burden. *Evidence: Primary* — Instagram Engineering 2011

Advantages and disadvantages

Treating design as an explicit activity — rather than letting structure emerge from whatever code is written first — has a clear profile. **Advantages:** problems are found on a whiteboard instead of in production; trade-offs are chosen deliberately; teams share a mental model; costs are predictable. **Disadvantages:** design takes time; designs made with poor information can be confidently wrong; and heavy upfront design can delay learning from real users. The practical answer is *proportionate* design: spend design effort in proportion to how expensive a decision would be to reverse (§1.8, §2.8).

Failure scenarios

Table 1.1 — Failure scenarios for the design process itself.

What goes wrong	Why	Detection	Recovery	Prevention
System collapses under launch traffic	Load never estimated; peak factor ignored	Latency and error rate spike; saturated CPU or connections	Emergency vertical scaling; shed load	Estimate peak load before launch (§1.3.2); load-test
Total outage when one machine fails	Single point of failure nobody named	Health checks fail; users report outage	Restore from backup onto a new machine	List SPOFs in every design review (§1.10)
Months spent building infrastructure for users who never arrive	Design sized for imagined scale	Low utilization; slipped launch	Simplify; delete unused components	Design for 10× current load, not 1,000×
Nobody remembers why a component exists	Decisions not recorded	Repeated debates; fear of changing anything	Reconstruct history from commits and people	ADRs (§1.8)

Common misconceptions

COMMON MISTAKE

"System design means drawing boxes and arrows." The diagram is the *output*. The substance is the reasoning: which requirements forced each box, what each arrow carries, what happens when each box fails, and what the design costs. A beautiful diagram with no requirements and no numbers is decoration.

COMMON MISTAKE

"System design only matters at Google scale." A three-person startup still chooses where its data lives, how it backs up, and what happens during a deploy. The decisions are smaller, but the cost of getting the *irreversible* ones wrong (data model, identifiers, storage location of user files) is felt at every scale.

1.4 Software architecture

1.4.1 What architecture is

DEFINITION

Definition — Software architecture. "The set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both" [Bass et al. 2021]. The international standard for architecture description defines it as the fundamental concepts or properties of a system in its environment, embodied in its elements, relationships, and in the principles of its design and evolution [ISO/IEC/IEEE 42010].

First-principles explanation

Return to the restaurant. Its *architecture* is not the color of the napkins; it is the set of decisions that would be painful to change once the doors open: where the kitchen sits relative to the dining room, how many burners there are, whether orders go through a single ticket rail or directly to stations, whether there is a walk-in fridge or daily deliveries. You can change the menu weekly; moving the kitchen means closing for months.

Martin Fowler summarized the same idea for software: architecture is about "the important stuff — whatever that is", meaning the decisions that are hard to change and that the team must share an understanding of [Fowler 2003]. The parenthetical "whatever that is" is honest: what counts as architectural depends on the system. For PhotoShare, *where photo files live* is architectural (changing it means migrating terabytes); *which image-resizing library we call* is usually not.

Why does this exist?

Large systems are built by many people over many years. Without a shared structure, each person optimizes locally and the whole becomes inconsistent: three ways to authenticate, four copies of the user table, no one able to say what happens if the cache is down. Architecture exists to (1) make the system understandable enough to reason about, (2) make the most important qualities — performance, availability, security, modifiability — achievable by design rather than by luck, and (3) let teams work in parallel along stable boundaries.

How it works internally

An architecture is described by several **views**, because no single picture answers every question. Kruchten's influential "4+1" model proposes four views plus scenarios that tie them together [Kruchten 1995]:

1. **Logical view** — the main abstractions and their responsibilities (users, photos, feeds, follows).
2. **Process view** — what runs concurrently, and how runtime components communicate (requests, queues, threads).
3. **Development view** — how the code is organized into modules, packages and repositories, and which teams own them.
4. **Physical view** — which machines, data centers and networks the software is deployed on.
5. **+1: Scenarios** — a few important use cases ("a user uploads a photo") traced through all four views to validate them.

The C4 model (context, containers, components, code) is a more recent, lightweight alternative used widely in industry; it is covered in §2.12 (Architecture documentation).

Architectures also follow recognizable **styles** or **patterns** — layered, monolithic, modular monolith, microservices, event-driven, serverless — each a reusable answer to a recurring set of forces. They are the subject of Ch 33 (Architecture Patterns) and Ch 34 (Microservices).

KEY CONCEPT

Conway's law. Melvin Conway observed in 1968 that organizations which design systems are constrained to produce designs that copy the organization's communication structure [Conway 1968]. If three teams build a compiler, you tend to get a three-pass compiler. Architecture and team structure therefore co-evolve: choosing service boundaries is partly choosing team boundaries, and vice versa (see §34.3 (Service boundaries)).

1.4.2 Architecture, system design and design: how the words relate

The terms overlap heavily, and different companies use them differently. The working distinction used in this book is in Table 1.2.

Table 1.2 — Overlapping disciplines: architecture, system design and low-level design.

Aspect	Software architecture	System design	Low-level design
What it is	The significant, hard-to-change structures of a system	Choosing components, data flow and deployment to meet requirements at scale	Internal structure of one component
Typical questions	Monolith or services? Which quality attributes dominate?	How many servers? Which database? Where to cache? What if a node fails?	Which classes, interfaces, algorithms, schemas?
Unit of reasoning	Elements, relations, properties	Services, stores, networks, traffic, failures	Classes, functions, tables, state machines
Time horizon	Years	Months to years	Weeks to months
Typical artifacts	Architecture views, principles, ADRs	Design docs, capacity estimates, deployment diagrams	Class/sequence diagrams, API specs, schema DDL
Where in this book	Ch 33–34	Most of Book I; case studies Ch 52–56	Ch 35–38

In everyday industry usage, "system design" usually means the distributed-systems end of architecture: servers, databases, caches, queues, networks and their failure modes. "Software architecture" is the broader discipline that also includes code-level structure

and organizational concerns. This book treats system design as the large-scale, runtime-focused part of architecture.

Real-world industry applications

INDUSTRY EXAMPLE

Architecture as a role. Many companies do not have a job titled "architect"; architectural decisions are made by senior and staff engineers through design documents and reviews (§1.10). Others have dedicated architects who own cross-team standards. In both models the activity is the same — identifying the hard-to-change decisions and making them deliberately — but the accountability differs. *Evidence: Illustrative* — a general description of common industry practice, not a claim about any specific company.

Advantages and disadvantages

An explicit architecture makes systems understandable, supports parallel work, and lets you predict quality attributes before building. Its costs are upfront effort, the risk of an "ivory-tower" architecture that ignores implementation realities, and rigidity if the architecture is enforced long after the requirements that justified it have changed.

Failure scenarios

Table 1.3 — Architecture-level failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
"Big ball of mud" — every module depends on every other	No agreed structure; boundaries eroded under deadline pressure	Small changes break distant features; slow builds	Incremental refactoring toward modules (§33.5)	Explicit module boundaries; dependency checks in CI
Architecture on paper differs from reality	Diagrams never updated	New engineers' mental model is wrong; incidents surprise everyone	Re-derive diagrams from deployment config and traces	Keep diagrams next to code; review in design reviews
Distributed monolith	Split into services without splitting data or ownership	Every release requires coordinating many services	Merge services or re-cut boundaries	Draw boundaries around data ownership (§34.11, §34.14)

Common misconceptions

COMMON MISTAKE

"Architecture is decided once, at the start." Requirements change, traffic grows, teams reorganize. Good architecture is *evolved* deliberately, with decisions revisited when their context changes — which is exactly why ADRs record the context (§1.8).

1.5 High-level vs low-level design

1.5.1 Definitions

DEFINITION

Definition — High-level design (HLD). A description of a system as a set of major components (services, data stores, queues, caches, external dependencies), their responsibilities, the interfaces and protocols between them, data flows, and deployment topology — at a level of detail where each component is a box whose internals are not yet specified.

DEFINITION

Definition — Low-level design (LLD). A description of the internals of one component: its modules, classes or functions, data structures, algorithms, database schemas, API request and response formats, error handling and state transitions — at a level of detail from which an engineer can write code.

First-principles explanation

A city planner decides where the roads, the water mains and the power substations go, and how much traffic and water each must carry. That is high-level design. An architect then draws the plumbing for one particular building: pipe diameters, which tap connects to which riser, where the shut-off valves are. That is low-level design. Both are design; they answer different questions at different zoom levels, and a good HLD makes the LLD of each part *possible* without dictating it.

Why does this exist?

Splitting design by zoom level lets the most consequential decisions (where data lives, how components communicate, how the system survives failures) be made and reviewed first, by people looking at the whole. It then lets each team design its component in detail independently, as long as it respects the interfaces the HLD fixed.

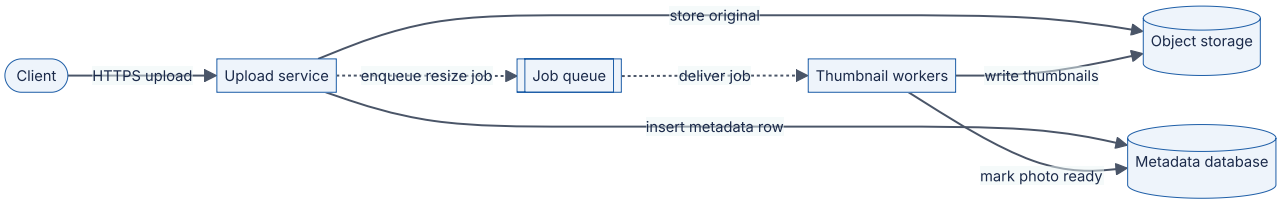
How it works internally

1. The HLD fixes **boundaries and contracts**: which component owns which data and exposes which API.
2. Each component's LLD fills in **internals**: classes, schemas, algorithms, concurrency.
3. LLD work frequently discovers problems the HLD missed (an operation that needs data from two owners; an API that cannot be made idempotent). These flow back to revise the HLD — design is a conversation between zoom levels, not a waterfall.

Architecture diagram

The HLD of PhotoShare's upload path at a later stage (detailed in §1.9) looks like this.

Figure 1.2 — High-level design of the PhotoShare upload path.

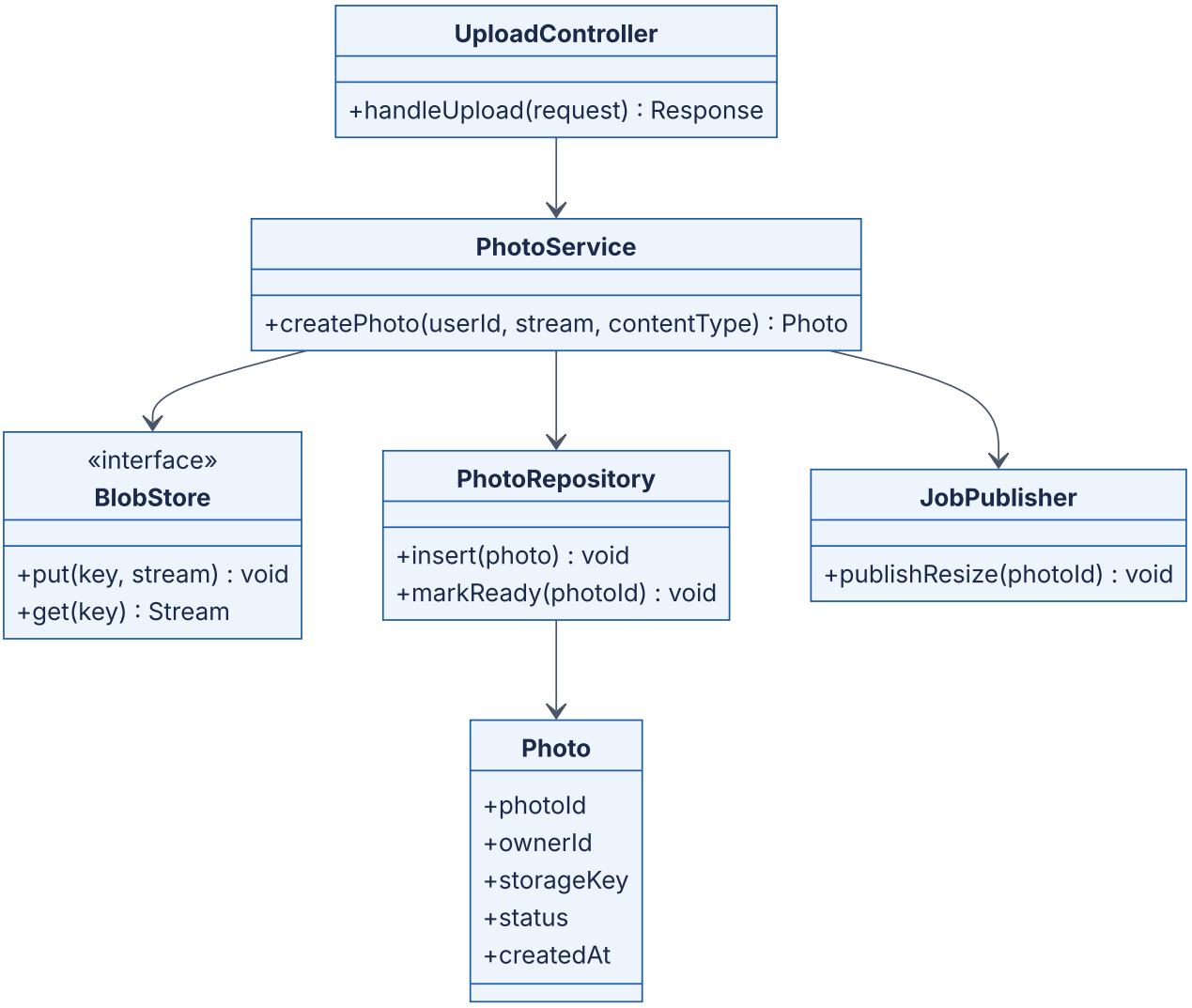


Reading the diagram.

- **Components.** The *upload service* accepts photos; *object storage* holds the image bytes; the *metadata database* holds rows describing each photo (owner, time, status, storage key); the *job queue* buffers work; *thumbnail workers* produce smaller versions of each photo.
- **Protocols.** HTTPS from the client; the object store is typically accessed over an HTTP API; the database over its wire protocol (SQL); the queue over the broker's protocol.
- **Direction and sync/async.** Solid arrows are synchronous: the client waits while the original is stored and the metadata row inserted. Dotted arrows are asynchronous: the resize job is enqueued and processed later, so the client gets a response without waiting for thumbnails.
- **Storage boundaries.** Two stores with different jobs: large immutable blobs in object storage, small mutable rows in the database.
- **Failure points.** If the queue is unavailable, uploads can still succeed but thumbnails are delayed; if workers crash, jobs remain in the queue and are retried. A crash *between* storing the original and inserting the row leaves an orphaned object — a consistency gap the LLD must handle (for example, by a periodic cleanup job).
- **Scaling options.** Upload service and workers scale horizontally by adding instances; the queue absorbs bursts so workers can be sized for average rather than peak load.

Notice what the HLD does *not* say: how the upload service validates file types, which class generates storage keys, or what the table columns are. That is LLD.

Figure 1.3 — Low-level design of the upload component (class view).



Reading the diagram.

- **Components.** `UploadController` handles HTTP concerns (parsing, authentication result, response codes). `PhotoService` holds the business rule "store bytes, record metadata, request thumbnails". `BlobStore` is an *interface*, so the same code can target local disk in tests and object storage in production. `PhotoRepository` wraps database access. `JobPublisher` wraps the queue. `Photo` is the data record.
- **Direction.** Arrows show dependency: the controller depends on the service, which depends on three abstractions. Nothing depends on the controller.
- **Sync/async and storage boundaries.** All calls in this diagram are synchronous in-process calls; the asynchrony from Figure 1.2 is hidden behind `JobPublisher`.
- **Failure points.** `createPhoto` performs three external operations that can each fail; the LLD must define ordering (store bytes first, then the row, then the job) and com-

pensation (a cleanup for orphaned objects). A `status` field (`PENDING` → `READY`) makes partial progress visible.

- **Scaling options.** Scaling is not decided here; this component is stateless, which is what allows the HLD to run many copies of it. Object-oriented principles behind this structure are covered in Ch 35 (Object-Oriented Design) and Ch 36 (SOLID Principles).

Alternatives and trade-offs

Table 1.4 — High-level design vs low-level design.

Aspect	High-level design	Low-level design
What it is	Components, data flows, deployment	Internals of one component
How it works	Boxes, arrows, contracts, capacity numbers	Classes, schemas, algorithms, state machines
Main concern	Quality attributes: scale, availability, latency, cost	Correctness, clarity, testability, extensibility
Typical diagrams	Flowchart/deployment, sequence across services	Class, sequence within a service, state, ER
Cost of a wrong decision	High — often needs data migration	Lower — usually a refactor within one team
Who reviews	Senior engineers across teams	The owning team
Interview format	"Design Instagram" (Ch 87–88, 91)	"Design a parking lot" (Ch 38)
Choose this zoom when	Deciding what to build and where it runs	Deciding how one part is written

Common misconceptions

COMMON MISTAKE

"HLD is for architects, LLD is for junior engineers." Both require skill. A clumsy LLD (no idempotency, no error handling, a schema that cannot be indexed) can sink a sound HLD. Senior engineers move fluidly between levels — and recognizing *when* an LLD detail is actually an HLD concern (for example, a schema choice that blocks partitioning, see §14.7) is a hallmark of seniority.

COMMON MISTAKE

"Once the HLD is approved, it is fixed." LLD routinely uncovers HLD flaws. Treat the HLD as a hypothesis that implementation tests.

1.6 Functional requirements

1.6.1 What functional requirements are and how to write them

> DEFINITION
>
> **Definition — Functional requirement (FR).** A statement of a behavior the system must provide: an input, action or event, and the observable result. Functional requirements describe *what* the system does — for example, "a signed-in user can upload a JPEG or PNG photo up to 20 MB, and it appears in their followers' feeds".

First-principles explanation

A restaurant's functional requirements are its menu and services: it serves pasta, takes reservations, offers takeaway. They say nothing about how fast the food arrives or how many guests fit — those are non-functional (§1.7). If you forget a functional requirement ("we also do delivery"), you build the wrong kitchen.

Why does this exist?

Every design decision is justified by some requirement. If you do not write the functional requirements down, two things happen: the team builds features nobody needs (and designs infrastructure for them), and it misses features that change the design fundamentally. "Users can delete their account and all their photos" sounds minor, but it forces you to be able to *find and erase every copy* of a user's data — in replicas, caches, backups, CDN edges and analytics pipelines — which is an architectural property, not a button.

How it works internally

1. **Identify actors.** Who interacts with the system? For PhotoShare: viewers, posters, moderators, advertisers, internal services.
2. **List use cases** per actor, as verb phrases: *upload photo*, *follow user*, *view feed*, *like photo*, *report photo*.
3. **Make each one testable**: specify inputs, limits and the observable outcome. "Upload photos" becomes "upload a JPEG/PNG/HEIC up to 20 MB; on success, return a photo ID within the latency target".
4. **Prioritize** — for example with **MoSCoW**: *Must*, *Should*, *Could*, *Won't (this time)*.
5. **Write the non-goals.** An explicit "out of scope" list prevents the design from silently growing.
6. **Sketch the API.** A rough interface for each must-have use case (detailed API design is §8.6) catches ambiguities early.

Table 1.5 — PhotoShare functional requirements (v1 scope).

ID	Requirement (testable form)	Priority
FR-1	A user can sign up and sign in with email and password	Must
FR-2	A signed-in user can upload a JPEG, PNG or HEIC photo up to 20 MB with an optional caption up to 2,200 characters	Must
FR-3	A user can follow and unfollow another user	Must
FR-4	A user can view a home feed of photos from followed users, newest first, 20 per page	Must
FR-5	A user can like a photo and see the like count	Should
FR-6	A user can delete a photo; it disappears from all feeds	Must
FR-7	A user can delete their account and all associated data	Must (legal)
FR-8	A user can comment on a photo	Could
—	Video, direct messages, stories, search, ads	Won't (v1 non-goals)

An API sketch for the must-haves might read:

```
POST   /v1/photos                (multipart: image, caption)  -> 201 {photoId}
DELETE /v1/photos/{photoId}                                   -> 204
PUT    /v1/users/{userId}/followers/me                        -> 204
DELETE /v1/users/{userId}/followers/me                        -> 204
GET    /v1/feed?cursor={cursor}&limit=20                      -> 200 {items,
nextCursor}
```

What to notice:

- The feed uses a **cursor** rather than a page number, because new photos arrive constantly and page numbers would skip or repeat items (pagination is detailed in §8.6).
- Follow is modeled as an idempotent `PUT`: repeating it has no further effect, which makes client retries safe (see §22.15 (Idempotency)).

Failure scenarios

Table 1.6 — Functional-requirement failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
A late "small" feature forces redesign	Requirement with architectural impact discovered late (e.g., account deletion, data export)	Feature estimate is months, not days	Retrofit data inventory and deletion pipeline	Ask about lifecycle, legal and admin use cases up front
Scope creep	No non-goals list	Design keeps growing; launch slips	Re-prioritize with MoSCoW	Write non-goals explicitly
Ambiguous requirement built two ways	"Feed" never defined (chronological? ranked?)	Teams' implementations disagree	Agree a definition; rework one side	Testable phrasing with examples

Common misconceptions

COMMON MISTAKE

"Functional requirements are the product manager's job, not the designer's." Product managers own *which* features matter; designers must interrogate each feature for architectural consequences. "Newest first" and "ranked by relevance" are one word apart in a requirements document and an entire ML system apart in architecture (see Ch 68 (Recommendation and Ranking Systems Architecture)).

1.7 Non-functional requirements (the "-ilities" catalog)

1.7.1 What non-functional requirements are

DEFINITION

Definition — Non-functional requirement (NFR). A requirement on *how well* the system performs its functions, or on constraints under which it operates, expressed as a measurable target — for example "p99 feed latency ≤ 300 ms at 2,000 RPS" or "monthly availability ≥ 99.9%". NFRs are also called **quality attributes** or, informally, the **"-ilities"** (scalability, availability, durability, …).

First-principles explanation

Two restaurants can serve the identical menu. One seats 20 and serves each dish in 40 minutes; the other seats 400 and serves each dish in 12 minutes, never runs out of bread, and passes every health inspection. Same functional requirements, completely different buildings, staff and suppliers. The difference is non-functional — and it is the non-functional requirements that determine the *architecture*.

Why does this exist?

Functional requirements can usually be met by a single program on a single machine. It is the non-functional requirements — serve 17,000 requests per second, never lose a photo, survive a data-center outage, cost less than $0.01 per user per month — that force systems to be distributed, replicated, cached and partitioned. Bass, Clements and Kazman make this the central thesis of software architecture: architecture is driven primarily by quality attributes, not by functionality [Bass et al. 2021]. Kleppmann similarly frames reliability, scalability and maintainability as the three central concerns when building data-intensive applications [Kleppmann 2017].

How it works internally

An NFR is only useful if it is **measurable**. Vague words — “fast”, “scalable”, “highly available” — cannot be tested and therefore cannot guide decisions. A reliable technique is the **quality attribute scenario** [Bass et al. 2021], which has six parts:

1. **Source** — who or what generates the stimulus (users, an attacker, a failing disk).
2. **Stimulus** — the event (a feed request, a node crash, a login attempt).
3. **Environment** — the conditions (normal load, peak load, during a deploy).
4. **Artifact** — the part of the system affected (feed service, metadata database).
5. **Response** — what the system does.
6. **Response measure** — the number that decides pass or fail.

Example: “**When** an availability zone fails (source/stimulus) **during** peak traffic (environment), the PhotoShare feed service (artifact) continues serving requests from the remaining zones (response) with **no more than 60 seconds** of elevated errors and **p99 latency ≤ 500 ms** after failover (response measure).”

In production, measurable NFRs become **service level indicators (SLIs)** — the measured quantity, such as the fraction of successful requests — and **service level objectives (SLOs)** — the target value for an SLI over a window [Beyer et al. 2016]. A **service level agreement (SLA)** is a contract with consequences (often financial) if an SLO-like target is missed. These are developed fully in §39.6 (SLI, SLO, SLA) and §39.7 (Error budgets).

> DEFINITION
>
> **Definition — Percentile latency (p50, p95, p99, p99.9).** The latency value below which the given percentage of requests fall. If p99 latency is 300 ms, 99% of requests completed in 300 ms or less and 1% took longer. p50 is the median.

Percentiles matter more than averages because users experience the slow requests, and a single page that fans out to many backend calls is slowed by the *slowest* of them [Dean

and Barroso 2013] (fan-out amplification is quantified in §4.8).

1.7.2 The catalog

The catalog is split into qualities observable while the system runs (Table 1.7) and qualities that matter as the system is built, changed and paid for (Table 1.8). Each row gives a measurable metric and a PhotoShare target.

Table 1.7 — Runtime quality attributes.

Attribute	What it means	Typical metric	PhotoShare example target	Deep dive
Latency (performance)	Time to complete one operation	p50/p99 in ms	Feed API p99 ≤ 300 ms	§4.8, Ch 23
Throughput	Work completed per unit time	RPS; MB/s	2,000 feed RPS at peak	Ch 23
Scalability	Ability to handle more load by adding resources	Cost or nodes per 1,000 RPS as load grows	10× users with ≤ 10× cost	Ch 18
Availability	Fraction of time the system serves correctly	% successful requests per 30 days	≥ 99.9% monthly	§4.14, Ch 39
Reliability	Probability of correct operation over time	Error rate; MTBF	< 0.1% failed uploads	Ch 39
Durability	Probability stored data is not lost	Annual object-loss probability	No acknowledged photo lost	§4.16, Ch 15
Consistency	Guarantees about what reads see after writes	Model name; staleness bound in s	Own uploads visible to self immediately; to followers within 5 s	Ch 25
Fault tolerance / resilience	Continued service despite component failure	Failures tolerated; recovery time	Survive loss of one zone	Ch 27, Ch 39
Recoverability (disaster recovery)	Ability to restore after disaster	RTO (time to restore); RPO (data loss window)	RTO 4 h; RPO 15 min	§39.8
Security	Protection of confidentiality, integrity, availability	Vulnerability count; time to patch; auth coverage	All traffic TLS; private photos never served to non-followers	Ch 10, Ch 50
Privacy	Appropriate handling of personal data	Deletion completed within N days; data minimized	Account deletion propagates in ≤ 30 days	Ch 50
Usability / accessibility	Ease and inclusiveness of use	Task success rate; accessibility conformance level	Screen-reader alt text on all photos	—
Elasticity	Speed of adapting capacity to load	Time to add capacity	Scale out in ≤ 5 min	§42.12

Table 1.8 — Development-time and business quality attributes.

Attribute	What it means	Typical metric	PhotoShare example target	Deep dive
Maintainability	Ease of understanding and fixing	Time to fix a bug; onboarding time	New engineer ships in week 1	Ch 35–36
Modifiability / extensibility	Ease of adding features	Number of modules touched per feature	Adding video touches ≤ 3 services	Ch 33
Testability	Ease of verifying behavior	Test coverage of critical paths; test run time	Full test suite ≤ 10 min	Ch 36
Deployability	Ease and safety of releasing	Deploy frequency; change-failure rate	Daily deploys; rollback ≤ 5 min	Ch 43
Observability	Ability to understand internal state from outputs	% of requests traced; time to diagnose	Every request has a trace ID	Ch 40
Operability	Ease of running in production	Pages per week; manual steps per release	≤ 2 pages per on-call week	Ch 39
Interoperability	Ability to work with other systems	Supported standards and API versions	Public API with versioning	§8.15
Portability	Ability to run in other environments	Effort to move cloud or region	Containerized services	Ch 41–42
Compliance	Conformance to laws and standards	Audit findings	EU users' data stored in the EU	Ch 50
Cost efficiency	Resources consumed per unit of value	$ per 1,000 requests; $ per user per month	≤ $0.01 per MAU per month	Ch 51
Sustainability	Energy and carbon per unit of work	kWh or gCO_2e per request	Track and report	Ch 51

The international quality-model standard ISO/IEC 25010 organizes many of these attributes into a formal hierarchy of characteristics and sub-characteristics; it is a useful checklist when you suspect you have forgotten a category [ISO/IEC 25010].

Mathematical foundations

Availability is the NFR most often stated as a number, and the number hides a surprisingly strict budget.

FORMULA

Availability and downtime budget

$$A = \frac{\text{MTBF}}{\text{MTBF}+\text{MTTR}} \qquad D = (1 - A) \times T$$

Symbol	Meaning	Units
A	Availability: fraction of time the service is up (dimensionless, 0–1)	—
MTBF	Mean time between failures	hours
MTTR	Mean time to repair (restore service)	hours
D	Allowed downtime in the period	minutes (or hours)
T	Length of the period	minutes (or hours)

Table 1.9 — Downtime budgets for common availability targets.

Availability	Downtime per 30-day month	Downtime per 365-day year
99% (“two nines”)	7.2 h	3.65 days
99.9% (“three nines”)	43.2 min	8.76 h
99.95%	21.6 min	4.38 h
99.99% (“four nines”)	4.32 min	52.6 min
99.999% (“five nines”)	25.9 s	5.26 min

WORKED EXAMPLE

Worked example — What does 99.9% allow, and can PhotoShare v0 meet it?

1. Minutes in a 30-day month: $30 \times 24 \times 60 = 43,200$ min.
2. Allowed downtime at 99.9%: $(1 - 0.999) \times 43,200 = 0.001 \times 43,200 = 43.2$ min.
3. PhotoShare v0 (Figure 1.1) is deployed by stopping the application, copying new code and restarting: assume 3 minutes per deploy, and 5 deploys per week ≈ 21.7 deploys per month, giving $21.7 \times 3 \approx 65$ min of planned downtime.
4. Add one unplanned VM failure per quarter with an MTTR of 2 hours, averaging 40 min per month.
5. Total expected downtime $\approx 65 + 40 = 105$ min per month, so $A \approx 1 - 105/43,200 \approx 0.9976$, i.e. 99.76%.

Interpretation. v0 misses 99.9% by more than a factor of two, *before any traffic growth*.

Engineering implication. Meeting 99.9% requires removing deploy downtime (multiple instances behind a load balancer, rolling deploys — §43.4) and shortening MTTR (automatic failover). An availability target, not request volume, is what first forces PhotoShare off a single server.

When a request must pass through several components in sequence, *all* must be up, so availabilities multiply.

FORMULA

Availability of components in series (independent failures)

$$A_{\text{series}} = \prod_{i=1}^{n} A_i$$

Symbol	Meaning	Units
A_{series}	Availability of the whole request path	—
A_i	Availability of component i	—
n	Number of components the request depends on	count

WORKED EXAMPLE

Worked example — A three-tier request path.

A feed request passes through a load balancer ($A = 0.9999$), the feed service ($A = 0.9995$) and the database ($A = 0.9995$).

1. $A_{\text{series}} = 0.9999 \times 0.9995 \times 0.9995$.
2. $0.9995 \times 0.9995 = 0.99900025$.
3. $0.99900025 \times 0.9999 \approx 0.99890$.

Interpretation. About 99.89% — *below* 99.9%, even though every component individually exceeds it.

Engineering implication. Every synchronous dependency you add lowers availability. Designers counter this with redundancy (parallel components; $A_{\text{parallel}} = 1 - \prod (1 - A_i)$ under independence), caching, and making non-critical dependencies asynchronous or optional. The full arithmetic, including the danger of assuming independence, is in §4.14.

Code example

SLOs are increasingly written as configuration that monitoring systems evaluate automatically. The following is illustrative, not the syntax of any specific tool.

```
# illustrative SLO specification for the PhotoShare feed service
service: feed-api
slos:
  - name: feed-availability
    sli: "fraction of GET /v1/feed requests returning non-5xx"
    objective: 0.999          # 99.9 %
    window: 30d               # rolling 30-day window
  - name: feed-latency
    sli: "fraction of GET /v1/feed requests completing in <= 300 ms"
    objective: 0.99           # i.e. p99 <= 300 ms
    window: 30d
```

What to notice:

- A latency SLO is expressed as “the fraction of requests faster than a threshold”, which is equivalent to a percentile target and easier to aggregate across servers.
- The window matters: 99.9% over 30 days allows a single 43-minute outage; 99.9% over every 5-minute window allows almost none.

Real-world industry applications

INDUSTRY EXAMPLE

Google SRE and error budgets. Google's Site Reliability Engineering book argues that 100% is the wrong reliability target for almost everything: users cannot tell the difference beyond a point (their own networks and devices are less reliable), and each additional nine costs disproportionately more. Instead, teams agree an SLO, and the gap between the SLO and 100% becomes an **error budget** that can be "spent" on risky launches; when the budget is exhausted, releases slow down in favor of reliability work. *Evidence: Primary* — [Beyer et al. 2016]

1.7.3 NFRs pull against each other

Quality attributes conflict. Improving one usually costs another, and *naming* the conflict is the heart of design reasoning.

Table 1.10 — Common tensions between quality attributes.

Tension	Why they conflict	Typical resolution	Deep dive
Consistency vs availability (under network partition)	Replicas that cannot communicate must either refuse requests or risk divergent answers	Choose per operation: strong for payments, eventual for like counts	Ch 25
Latency vs consistency (even without partitions)	Coordinating replicas adds round trips	Read from nearby replica; accept bounded staleness	§25.9 (PACELC)
Durability vs write latency	Waiting for multiple disks or zones to confirm takes time	Synchronous replication to a quorum; async to remote region	Ch 27
Availability vs cost	Redundancy means paying for idle capacity	Match nines to business value	Ch 39, Ch 51
Security vs usability	Extra checks add friction	Risk-based authentication	Ch 10
Simplicity (operability) vs scalability	Each scaling technique adds components	Scale only the measured bottleneck	Ch 18
Performance vs modifiability	Optimized code is specialized and harder to change	Optimize hot paths only, after profiling	Ch 23

Failure scenarios

Table 1.11 — Non-functional-requirement failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
System is “fast” in tests, slow for users	Averages measured instead of tail percentiles	p99 dashboards; user complaints	Find tail causes (GC, slow dependencies)	Specify and test p99, not mean
Gold-plated availability	99.999% demanded without business case	Cost far above budget	Renegotiate SLO	Tie each nine to business impact
Data loss despite “replication”	Durability never specified; replicas asynchronous; backups untested	Restore fails during incident	Partial recovery from logs	State RPO; test restores regularly
Privacy breach via forgotten copy	Deletion requirement ignored caches, logs, backups	Audit or regulator finding	Purge pipelines	Data inventory; deletion as an NFR

Common misconceptions

COMMON MISTAKE

“The system must be highly available, scalable and fast.” This is not a requirement; it is a wish. Every system wants these. A requirement has a number, a percentile, a window and a load level: “99.9% of feed requests per 30 days return successfully; p99 ≤ 300 ms at 2,000 RPS”.

COMMON MISTAKE

“Availability and reliability are the same thing.” Availability is the fraction of time the system is *up*; reliability is the probability it operates *correctly* over a period. A system that is up but returns wrong feeds is available and unreliable. See §39.3.

COMMON MISTAKE

“Aim for 100%.” A 100% target is unachievable, makes every change frighteningly risky, and leaves no error budget for shipping features [Beyer et al. 2016].

1.8 Architectural decision-making and ADRs

1.8.1 Architectural decisions

DEFINITION

Definition — Architectural decision. A design choice that addresses an architecturally significant requirement — one that affects structure, quality attributes, interfaces or technology in a way that is costly to change later. Examples: choosing a relational database over a key-value store; storing files in object storage; making photo processing asynchronous.

DEFINITION

Definition — Architecture Decision Record (ADR). A short document that captures one architectural decision: its context, the decision itself, its status, and its consequences. ADRs are numbered, kept in sequence (often in the code repository), and never edited to reverse a decision — a new ADR *supersedes* the old one [Nygard 2011].

First-principles explanation

A court does not just announce “guilty” or “not guilty”; it publishes the reasoning, so later judges can tell whether a new case is similar enough for the ruling to apply. An ADR is a judgment with its reasoning attached. Two years from now, an engineer wondering “why on earth are thumbnails generated asynchronously?” can read the ADR, see that the context was “synchronous resizing pushed upload p99 to 4 seconds”, and decide whether that context still holds.

Why does this exist?

Without records, architectural knowledge lives in people’s heads and leaves when they do. Nygard described the resulting dilemma for a newcomer facing an unexplained decision: either *blindly accept* it (and keep something that may no longer make sense) or *blindly change* it (and possibly break a quality attribute the decision was protecting) [Nygard 2011]. ADRs make a third option possible: read the context, then decide.

How it works internally

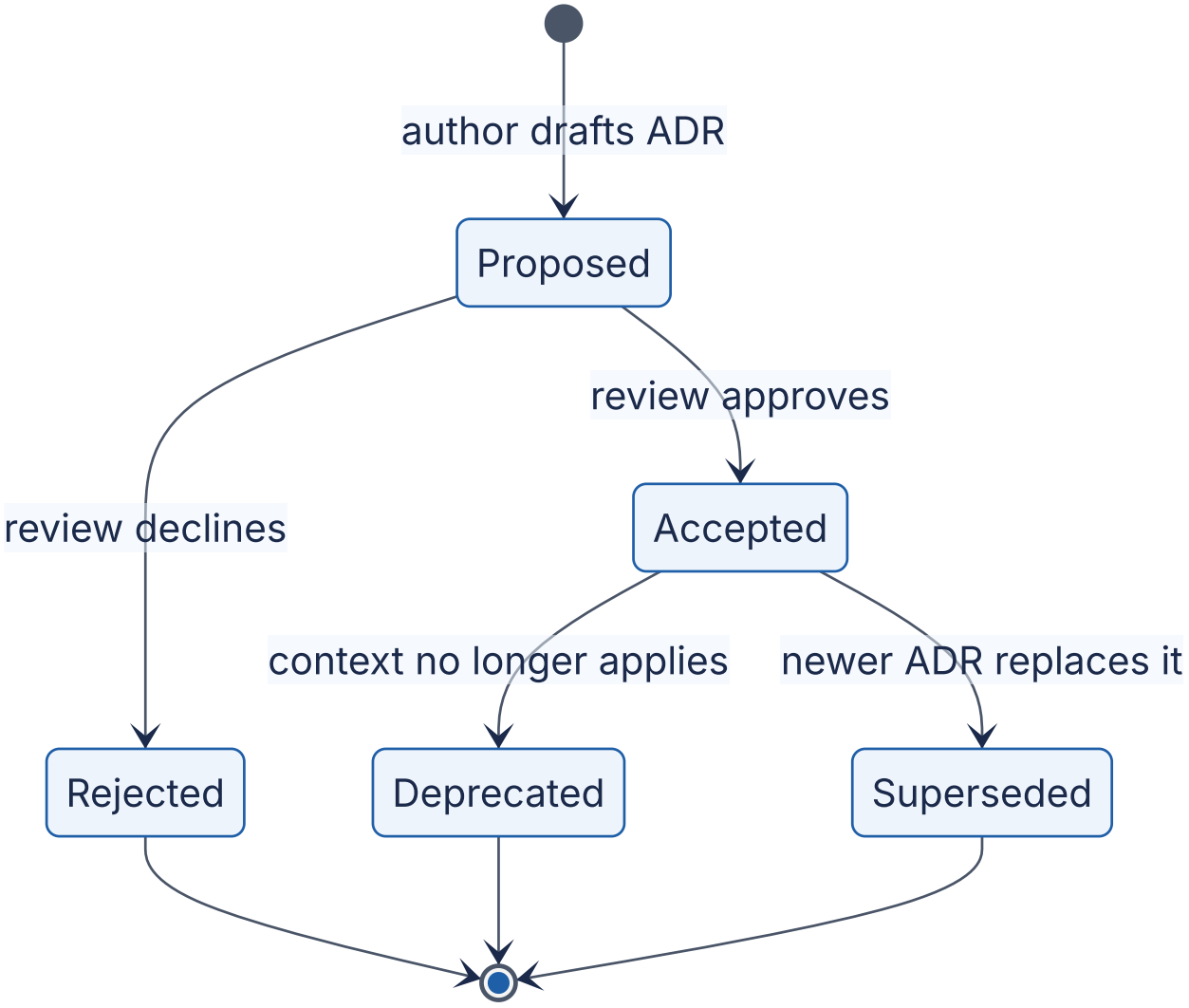
A disciplined decision process looks like this:

1. **Identify the driver.** Which requirement or problem forces a decision now? (For PhotoShare: “the local disk is 80% full and v1 must reach 99.9% availability.”)

2. **Assess reversibility.** Is this a cheap-to-reverse ("two-way door") or expensive-to-reverse ("one-way door") decision? Spend effort proportionally (developed in §2.8).
3. **Enumerate options** — at least two plus "do nothing".
4. **Choose criteria** from the NFRs (§1.7) and weight them.
5. **Evaluate** options against criteria — qualitatively, with a decision matrix, or with a prototype or benchmark when the numbers are uncertain.
6. **Decide and record** the decision as an ADR with status *Proposed*, circulate it, and move it to *Accepted* after review.
7. **Revisit** when the context changes, writing a new ADR that *supersedes* the old one.

Architecture diagram

Figure 1.4 — ADR lifecycle.



Reading the diagram.

- **Components (states).** Each box is the *status* field of an ADR. Nygard's original format lists proposed, accepted, deprecated and superseded; *Rejected* is a common ex-

tension that preserves the record of options the team declined.

- **Transitions.** An ADR starts as *Proposed* and becomes *Accepted* or *Rejected* through review. An accepted ADR is never edited to say the opposite; it becomes *Deprecated* (no longer relevant) or *Superseded* (with a link to the replacing ADR).
- **Direction and sync/async.** This is a state machine, not a data flow; all transitions are human decisions, made asynchronously over days.
- **Storage boundaries.** ADRs typically live as Markdown files in the same repository as the code they govern, so they are versioned and reviewed like code.
- **Failure points.** The common failure is an ADR stuck in *Proposed* forever, or an accepted ADR silently contradicted by code. Design reviews (§1.10) check for both.
- **Scaling options.** At organization scale, teams keep per-repository ADR logs plus a shared log for cross-cutting decisions.

Mathematical foundations

When several criteria matter, a **weighted decision matrix** makes the trade-off explicit and arguable.

FORMULA

Weighted decision score

$$S_j = \sum_{i=1}^{m} w_i \, s_{ij}, \qquad \sum_{i=1}^{m} w_i = 1$$

Symbol	Meaning	Units
S_j	Total score of option j	points (same scale as s_{ij})
w_i	Weight of criterion i (importance; sums to 1)	—
s_{ij}	Score of option j on criterion i (here 1 = poor, 5 = excellent)	points
m	Number of criteria	count

WORKED EXAMPLE

Worked example — Where should PhotoShare store photo files?

Options: (A) keep files on each application server's local disk; (B) a shared network file server (NFS); (C) managed object storage.

Table 1.12 — Decision matrix for photo storage.

Criterion	Weight	A: Local disk	B: NFS server	C: Object storage
Durability	0.30	1	3	5
Scalability	0.25	1	2	5
Operability	0.20	3	2	5
Cost	0.15	5	3	3
Latency	0.10	5	4	3

1. $S_A = 0.30(1) + 0.25(1) + 0.20(3) + 0.15(5) + 0.10(5) = 0.30 + 0.25 + 0.60 + 0.75 + 0.50 = 2.40$.
2. $S_B = 0.30(3) + 0.25(2) + 0.20(2) + 0.15(3) + 0.10(4) = 0.90 + 0.50 + 0.40 + 0.45 + 0.40 = 2.65$.
3. $S_C = 0.30(5) + 0.25(5) + 0.20(5) + 0.15(3) + 0.10(3) = 1.50 + 1.25 + 1.00 + 0.45 + 0.30 = 4.50$.

Interpretation. Object storage wins clearly. A *sensitivity check* matters more than the totals. Suppose a cost-conscious reviewer doubles cost's weight to 0.30 and halves durability's to 0.15:

4. $S_A = 0.15(1) + 0.25(1) + 0.20(3) + 0.30(5) + 0.10(5) = 0.15 + 0.25 + 0.60 + 1.50 + 0.50 = 3.00$.
5. $S_B = 0.15(3) + 0.25(2) + 0.20(2) + 0.30(3) + 0.10(4) = 0.45 + 0.50 + 0.40 + 0.90 + 0.40 = 2.65$.
6. $S_C = 0.15(5) + 0.25(5) + 0.20(5) + 0.30(3) + 0.10(3) = 0.75 + 1.25 + 1.00 + 0.90 + 0.30 = 4.20$.

C still wins by more than a point, so the ranking is robust to this disagreement.

Engineering implication. The matrix does not make the decision; it exposes *which weights* would have to change for the decision to flip, so that reviewers argue about priorities rather than conclusions. Scores of 1–5 are judgments; where a number can be measured (latency, cost per TB), measure it.

Code example

Here is the ADR that records the decision. ADRs are plain Markdown; the block below uses the `text` tag so its content is shown verbatim.

```
# ADR-0003: Store photo binaries in managed object storage

Status: Accepted (2026-03-14). Supersedes ADR-0001 "Store photos on local disk".
Deciders: PhotoShare backend team. Reviewed in design review DR-12.

## Context
- v0 stores photo files on the single VM's local disk (ADR-0001).
- The disk is 80% full; uploads add ~50 GB/day at 100,000 DAU (§1.9 estimate).
- v1 must meet 99.9% monthly availability, which requires at least two
  stateless application servers behind a load balancer. Files on one
  server's local disk are invisible to the other servers.
- There is no off-machine copy of photos; a disk failure loses all photos.

## Decision
Store every original and thumbnail in managed object storage under the key
photos/{photoId}/{variant}. Application servers keep no photo files locally.
Clients download images through the CDN, never directly from app servers.

## Options considered
A. Local disk per server - rejected: blocks horizontal scaling; no durability.
B. Shared NFS server - rejected: new single point of failure; we would
   operate replication and backups ourselves.
C. Managed object storage - chosen (decision matrix score 4.50 vs 2.65, 2.40).

## Consequences
+ Application servers become stateless and can be added or replaced freely.
+ Durability and capacity are the provider's responsibility.
- Upload path gains a network hop (~tens of ms to the object store).
- Storage cost becomes a per-GB monthly bill that grows with users; add a
  lifecycle policy to move old originals to a colder tier (see Ch 15).
- One-time migration of existing files (~2 TB); plan a dual-read period.
- Vendor-specific API; wrap it behind the BlobStore interface (Figure 1.3).
```

What to notice:

- **Context is factual and dated**; it includes the numbers that made the decision necessary. When those numbers change, the ADR tells you the decision may need revisiting.
- **Consequences list negatives**, not only benefits. An ADR with no downsides is advocacy, not a record.
- **Supersession** is explicit in both directions: ADR-0001 should be updated only to change its status to “Superseded by ADR-0003”.
- Nygard’s original format is Title, Status, Context, Decision, Consequences [Nygard 2011]; “Options considered” and “Deciders” are widely used extensions.

Real-world industry applications

INDUSTRY EXAMPLE

Lightweight ADRs in open-source and public-sector projects. After Nygard's 2011 post, the format spread widely; many public repositories contain an ADR numbered 0001 titled "Record architecture decisions", whose decision is simply to adopt ADRs for the project. The appeal is low cost: a page of Markdown reviewed in the same pull-request workflow as code. *Evidence: Secondary* — [Nygard 2011] and the many public repositories that adopt its format; the specific spread is an observation, not a measured statistic.

Advantages and disadvantages

Advantages: cheap to write; versioned with the code; preserves the *why*; speeds onboarding; prevents re-litigating settled questions; forces explicit consequences.
Disadvantages: only as good as team discipline; can become bureaucratic if required for trivial choices; a large ADR log needs an index to remain navigable.

Alternatives and trade-offs

ADRs complement rather than replace design documents (§1.10): a design doc explores a whole problem and may produce several decisions; each significant decision is then distilled into an ADR. Wikis are easier to browse but drift from the code. Commit messages capture *what* changed but rarely the alternatives rejected.

Failure scenarios

Table 1.13 — Decision-making failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Decision re-litigated every quarter	Rationale not recorded	Same debate in multiple meetings	Write the ADR retroactively	ADR for every one-way-door decision
Analysis paralysis	Treating a reversible choice as irreversible	Weeks of debate on a library choice	Time-box; pick and move on	Classify reversibility first (§2.8)
Decision by loudest voice	Criteria never made explicit	Dissenters disengage; later regret	Re-open with an explicit matrix	Agree criteria and weights before scoring
Stale ADR misleads	Context changed, ADR not superseded	Code contradicts accepted ADR	Write a superseding ADR	Review ADR log in design reviews

Common misconceptions

COMMON MISTAKE

“The decision matrix decides.” It structures judgment; it does not replace it. Weights encode values, and a team can make any option win by adjusting them. Its value is making those values visible.

1.9 Why system design matters (cost of wrong decisions)

1.9.1 Decisions differ in their cost of reversal

KEY CONCEPT

The asymmetry of design decisions. Most code-level decisions are cheap to reverse: rename a function, swap a library, redeploy. A few decisions are very expensive to reverse because *data* has been written in their shape: where files live, the primary key format, the partitioning key, the consistency model clients rely on, the public API contract. The job of system design is to spot the expensive few early and decide them deliberately, while keeping everything else cheap to change.

First-principles explanation

Moving furniture is easy; moving a load-bearing wall after the house is built is not. Data is software’s load-bearing wall: once billions of rows or petabytes of files exist in a particular shape, changing that shape means a migration that runs for days or months while the system stays live.

Why does this exist?

Bad design shows up as cost in four currencies: **money** (over-provisioned or emergency hardware), **time** (migrations that consume a team for a quarter), **user trust** (outages, data loss) and **opportunity** (features not built because engineers are fighting fires). Boehm and Basili summarized decades of industry data with the widely quoted observation that fixing a problem after delivery is often far more expensive than fixing it during requirements and design [Boehm and Basili 2001]; the exact multiplier varies greatly between studies and projects, but the direction is consistent.

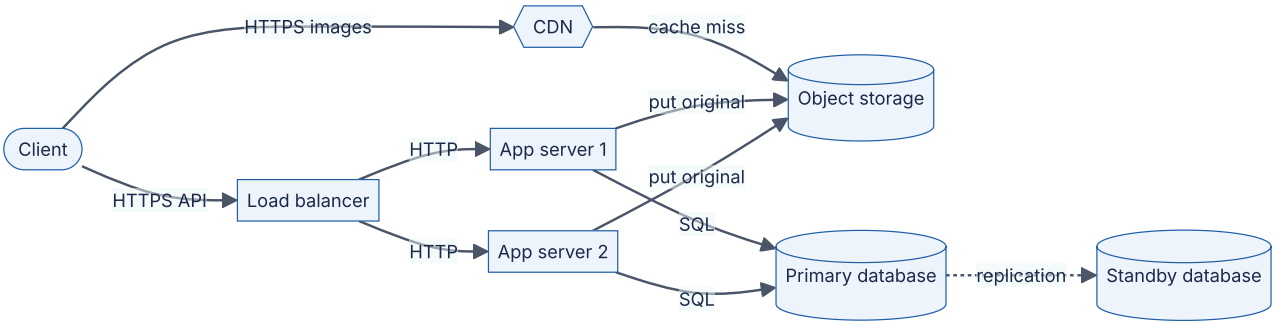
The opposite error — **over-engineering** — has the same currencies. Building a 50-service architecture for 1,000 users spends money and time on problems that do not yet exist, and makes the system harder to change when you learn what users actually want.

How it works internally — PhotoShare grows up

We now evolve PhotoShare from Figure 1.1 through three more stages. Each stage is triggered by a specific requirement that the previous design cannot meet. All estimates come from the script in §1.3.2 with the same assumptions (50 photo views and 0.2 uploads per user per day; 2.5 MB stored per upload; 200 KB served per view; peak factor 3), plus one more: each user loads the feed about 5 times per day, so feed API requests are one-tenth of photo views.

Stage 1 — 100,000 DAU: separate state from compute. Two requirements break v0. The 99.9% availability target cannot be met with deploy downtime (§1.7.2 worked example), and uploads now add $100,000 \times 0.2 \times 2.5$ MB = 50 GB per day to a single local disk. ADR-0003 (§1.8) moves files to object storage; a managed database with a standby replica replaces the local one; two stateless application servers sit behind a load balancer; images are served through a CDN.

Figure 1.5 — PhotoShare v1: stateless application tier and managed storage.



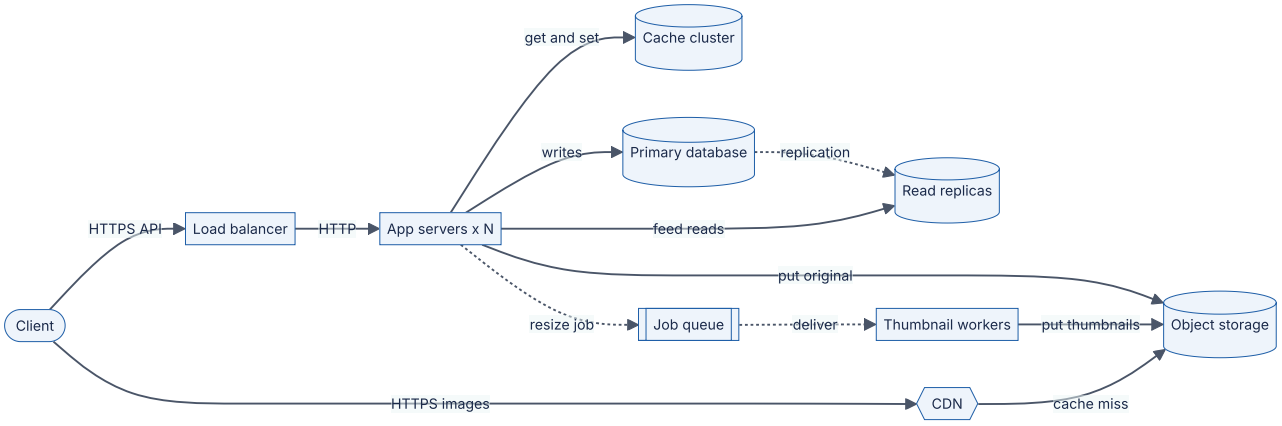
Reading the diagram.

- **Components.** A *load balancer* spreads API requests across two identical *app servers* (see Ch 19 (Load Balancing)). A *primary database* holds metadata, with a *standby* ready to take over. *Object storage* holds image bytes; a *CDN* (an external network of edge caches, drawn as a hexagon) serves images close to users (see Ch 21).
- **Protocols.** HTTPS from clients; plain HTTP inside the private network (or HTTPS, depending on the security model, §10.10); SQL to the database; the object store's HTTP API.
- **Direction and sync/async.** API calls and image fetches are synchronous. Replication from primary to standby is drawn dotted: it is commonly asynchronous, so the standby may lag slightly (replication modes are covered in §14.4).
- **Storage boundaries.** App servers are now **stateless**: they hold no data that cannot be lost. All state lives in the database and object storage. This is what allows any app server to serve any request.

- **Failure points.** Losing one app server halves capacity but keeps the service up. The primary database is still critical: failover to the standby takes time and, with asynchronous replication, may lose the last few seconds of writes.
- **Scaling options.** Add app servers horizontally; scale the database vertically for now.

Stage 2 — 1,000,000 DAU: protect the database. Feed API traffic is now $1,000,000 \times 5 / 86,400 \approx 58$ RPS average and about 174 RPS at peak — modest — but each feed request runs a join across follows and photos, and the database CPU is the bottleneck. Synchronous thumbnail generation pushes upload p99 above 3 seconds. Two additions fix this: a **cache** for feeds, sessions and hot metadata (Ch 20), plus **read replicas** for queries that tolerate slight staleness (§14.3); and an **asynchronous job queue** so uploads return as soon as the original is stored (Figure 1.2; Ch 30).

Figure 1.6 — PhotoShare v2: cache, read replicas and asynchronous processing.



Reading the diagram.

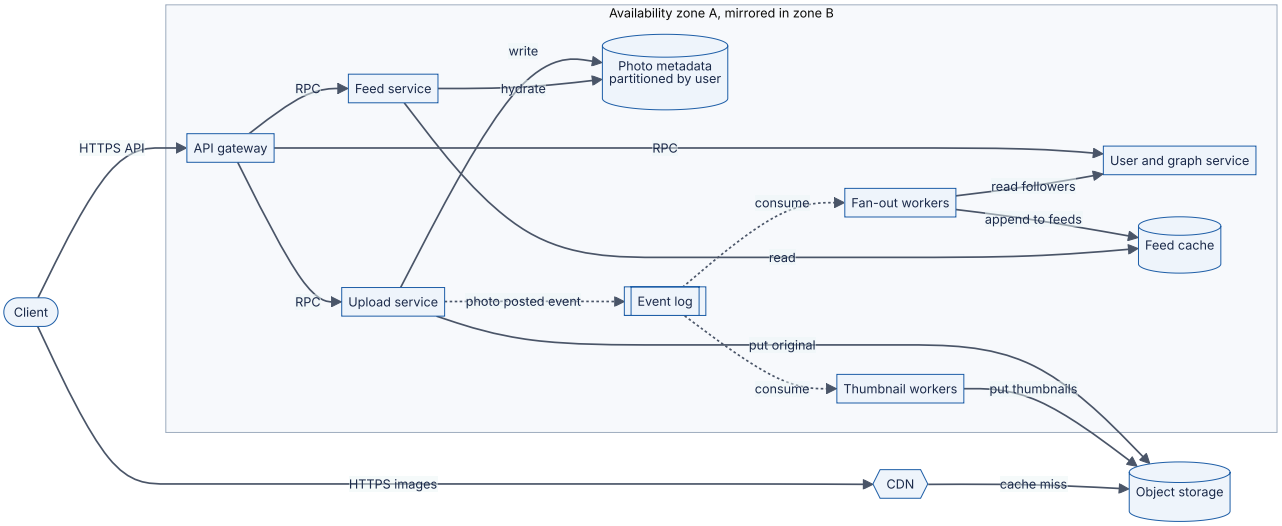
- **Components.** New: a *cache cluster* (in-memory key-value store), *read replicas* of the database, a *job queue* and *thumbnail workers*.
- **Protocols.** The cache uses its own lightweight protocol over TCP; the queue uses the broker's protocol.
- **Direction and sync/async.** Feed reads check the cache first (cache-aside, §20.7) and fall back to a replica on a miss. Uploads are synchronous only until the original is stored and the row written; resizing happens asynchronously (dotted arrows).
- **Storage boundaries.** Data now exists in up to four places — primary, replicas, cache, object storage — so **consistency** becomes a design question: a user who just uploaded may not see the photo if their next read hits a lagging replica. The consistency target in Table 1.7 ("own uploads visible to self immediately") is met by routing a user's reads of their own profile to the primary for a short time after a write (read-your-writes, §25.6).
- **Failure points.** A cache outage sends all reads to the replicas at once and can overload them (a *cache avalanche*, §20.13). A poisoned job that always crashes a worker

must be moved aside to a dead-letter queue (§30.11).

- **Scaling options.** App servers and workers scale horizontally; replicas scale reads; the primary still takes every write.

Stage 3 — 10,000,000 DAU: scale every tier and survive a zone failure. The estimator now gives about 5,800 average and 17,400 peak photo views per second, about 1,700 peak feed API RPS, 5 TB of new photo data per day (1.8 PB per year before replication) and about 9.3 Gbit/s of average image egress (28 Gbit/s at peak). The availability target rises to "survive loss of one availability zone". The design splits into a few services along clear data-ownership lines, partitions the metadata database by user ID (Ch 14), precomputes feeds asynchronously when a photo is posted (*fan-out on write*, examined in §53.3), and runs in at least two zones.

Figure 1.7 — PhotoShare v3: services, partitioned data and multi-zone deployment.



Reading the diagram.

- **Components.** An *API gateway* authenticates and routes requests (§34.6). The *upload*, *feed* and *user and graph* services each own their data. An *event log* (Ch 31) records "photo posted" events; *fan-out workers* insert the new photo ID into each follower's precomputed feed in the *feed cache*; *thumbnail workers* consume the same events. Photo metadata is **partitioned** by user ID across many database nodes.
- **Protocols.** HTTPS at the edge; internal RPC (for example gRPC, §8.8) between gateway and services; the event log's protocol for producers and consumers.
- **Direction and sync/async.** The read path (gateway → feed service → feed cache → metadata) is synchronous and short. The write path returns once the original and metadata are stored; everything else — thumbnails, fan-out — happens asynchronously from the event log, so a user with 100,000 followers does not make the upload slow.

- **Storage boundaries.** Object storage and the CDN are regional or global managed services outside the zone boundary; metadata, the event log and the feed cache are replicated across zones.
- **Failure points.** Loss of a zone: traffic shifts to the mirror zone, which must have spare capacity (≥ 50% headroom with two zones). Fan-out lag: followers see new photos seconds late — acceptable under the "within 5 s" consistency NFR, and visible as event-log consumer lag in monitoring (Ch 40). Celebrity accounts with tens of millions of followers make fan-out on write too expensive; a hybrid with *fan-out on read* for those accounts is the standard fix (§53.3).
- **Scaling options.** Every service is stateless and scales horizontally; metadata scales by adding partitions; the event log by adding partitions and consumers; the CDN absorbs image egress.

Table 1.14 — PhotoShare stages, triggers and costs.

Stage	DAU	Trigger (broken requirement)	Key changes	New cost or complexity
v0 (Fig. 1.1)	1,000	—	One VM	SPOF; no backups
v1 (Fig. 1.5)	100,000	99.9% availability; disk filling	LB, stateless app servers, managed DB plus standby, object storage, CDN	Network hops; managed-service bills; failover semantics
v2 (Fig. 1.6)	1,000,000	DB CPU saturated; upload p99 > 3 s	Cache, read replicas, job queue, workers	Stale reads; cache invalidation; queue operations
v3 (Fig. 1.7)	10,000,000	Zone-failure tolerance; write volume; 28 Gbit/s egress	Services, partitioned metadata, event log, precomputed feeds, multi-zone	Distributed-systems failure modes; more teams and on-call

Mathematical foundations

WORKED EXAMPLE

Worked example — The cost of one wrong early decision.

Suppose PhotoShare had *not* written ADR-0003 and kept files on application-server disks until 1,000,000 DAU, by which time roughly 100 days at 0.5 TB/day had accumulated about 50 TB.

1. Data to move: $50\ \text{TB} = 50 \times 10^{12}$ bytes $= 4 \times 10^{14}$ bits.
2. Available copy bandwidth, without starving production traffic: assume 1 Gbit/s $= 10^{9}$ bit/s.
3. Copy time: $4 \times 10^{14} / 10^{9} = 4 \times 10^{5}$ s ≈ 111 h ≈ 4.6 days of continuous copying.
4. Meanwhile 0.5 TB/day of new uploads keeps arriving, so the migration also needs dual-writes, verification checksums, a cut-over plan and a rollback plan — realistically several engineer-weeks, plus risk of data loss.

Interpretation. Making the decision at v1 with 2 TB took a few hours of copying; deferring it made it 25× larger and turned it into a project.

Engineering implication. Decisions about *where data lives* should be made early and on purpose, even when the system is small. Decisions about *how much compute you run* can safely wait.

Real-world industry applications

INDUSTRY EXAMPLE

Knight Capital (2012). On 1 August 2012, Knight Capital deployed new trading code to its order-routing servers using a manual process; one of eight servers did not receive the new code, and a repurposed configuration flag activated long-unused legacy logic on that server. In about 45 minutes the firm executed millions of unintended trades and lost more than $460 million. The U.S. Securities and Exchange Commission's subsequent order cites, among other failures, the lack of adequate technical controls and written procedures for deployment. The lesson for designers: deployability and operability (Table 1.8) are architectural qualities, not afterthoughts — dead code, reused flags and manual deploys are design decisions with consequences. *Evidence: Primary* — [SEC 2013]

INDUSTRY EXAMPLE

A counter that outgrew its integer. In December 2014 YouTube reported that the view count of the music video “Gangnam Style” had exceeded the maximum value of a signed 32-bit integer (2,147,483,647), and that the counter had been upgraded to a 64-bit integer. Choosing the width of an identifier or counter is a tiny decision on day one and a data migration years later. *Evidence: Secondary* — widely reported at the time; primary source to be confirmed (see §1.19).

Failure scenarios

Table 1.15 — Costly design failure patterns.

What goes wrong	Why	Detection	Recovery	Prevention
Stateful app servers block scaling	Files or sessions stored locally	New servers serve 404s or log users out	Move state to shared stores; migration	Stateless services by default (§18.5)
Identifier overflow	32-bit IDs or counters	Inserts fail at about 2.1 billion	Emergency schema migration	64-bit or globally unique IDs (§17.10)
Premature microservices	Splitting before boundaries are understood	Every feature touches many services	Merge into a modular monolith	Start with a modular monolith (§33.5)
Unplanned celebrity load	Design assumed uniform users	Hot partitions; fan-out lag	Special-case heavy users	Ask about skew in requirements (§14.10)

Common misconceptions

COMMON MISTAKE

“We can always fix the architecture later.” You can always fix *code* later. Fixing data shape, storage location and public contracts later costs migrations that grow with the data. Decide those deliberately; defer the rest.

1.10 How system design is practiced in industry

1.10.1 Design documents

> DEFINITION
>
> **Definition — Design document (design doc).** A written proposal, produced before significant implementation, that states a problem, its goals and non-goals, a proposed design, the alternatives considered, and cross-cutting concerns (security, privacy, operations, cost), so that others can review it while changes are still cheap.

First-principles explanation

Before a building is constructed, its drawings are circulated to the structural engineer, the fire inspector and the client. Each reads for different risks, and each objection costs an eraser rather than a demolition crew. A design doc is the software equivalent: the cheapest place to find a design flaw is in a document.

How it works internally

Practice varies, but a widely cited public description of Google's design-doc culture describes informal documents, written in a collaborative editor, typically containing [Ubl 2020]:

1. **Context and scope** — the landscape the system lives in and what is being built.
2. **Goals and non-goals** — including explicit things the system will *not* do.
3. **The actual design** — often with a system-context diagram, APIs and data storage, emphasizing trade-offs rather than implementation detail.
4. **Alternatives considered** — and why they were rejected.
5. **Cross-cutting concerns** — security, privacy, observability and similar.

The same source describes larger design docs of roughly 10–20 pages and shorter "mini design docs" for smaller changes. *Software Engineering at Google* likewise treats design docs as a standard documentation type [Winters et al. 2020]. Amazon is publicly known for narrative documents instead of slide decks, including the "PR/FAQ" (a mock press release and FAQ written before building a product) described by two former Amazon executives [Bryar and Carr 2021].

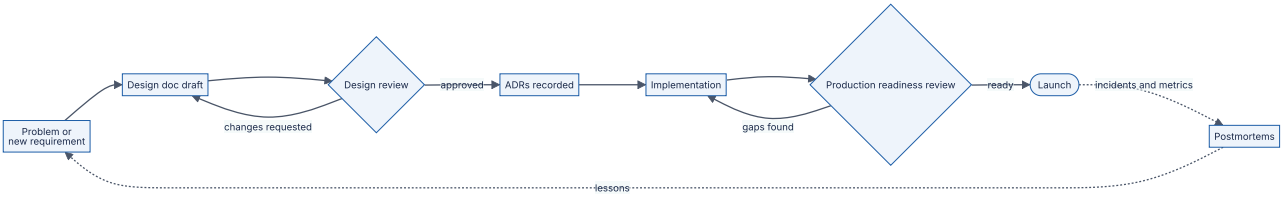
1.10.2 Design reviews and production readiness

A **design review** is a meeting or asynchronous comment thread in which experienced engineers outside the authoring team challenge a design doc. Reviewers typically probe: Are the requirements and numbers right? What are the single points of failure? What hap-

pens under 10× load? How is it deployed and rolled back? How is it monitored? What does it cost? Is there a simpler design?

Close to launch, many organizations run a **production readiness review (PRR)**: a checklist-based assessment that a service is operable — monitoring, alerting, capacity, runbooks, on-call — before it receives production traffic. Google's SRE book describes PRRs as part of how SRE teams engage with services [Beyer et al. 2016]. After incidents, **blameless postmortems** (§39.12) feed lessons back into future designs.

Figure 1.8 — The life of a design in a typical engineering organization.



Reading the diagram.

- **Components.** Activities (rectangles), gates where a decision is made (diamonds) and the launch milestone (stadium).
- **Direction.** Work flows left to right from problem to launch, with two loops back: review feedback to the doc, and readiness gaps to implementation.
- **Sync/async.** Solid arrows are the planned path. Dotted arrows are the slower, asynchronous feedback from production — incidents and metrics — that eventually create new problems to design for.
- **Storage boundaries.** The durable artifacts are the design doc, the ADRs and the postmortems; together they form the organization's architectural memory.
- **Failure points.** Skipping the review gate (designs built unreviewed), a review that becomes a rubber stamp, or postmortem lessons that never feed back into designs.
- **Scaling options.** Large organizations scale review capacity with tiered reviews: lightweight team review for reversible changes, cross-org architecture review for one-way-door decisions.

1.10.3 RFC processes

The term **RFC** ("Request for Comments") comes from the Internet's standards process; RFC 1, "Host Software", was written by Steve Crocker in 1969 [Crocker 1969], and the IETF's current standards process is itself defined in an RFC [Bradner 1996]. Many open-source projects run public proposal processes in the same spirit: Python Enhancement Proposals (PEPs) [Python PEP 1], Rust RFCs [Rust RFCs] and Kubernetes Enhancement Proposals (KEPs) [Kubernetes KEPs]. Inside companies, "RFC" usually means a design doc circulated widely for comment before a decision, often across teams.

Table 1.16 — Design artifacts compared.

Aspect	Design doc	ADR	RFC (company or open source)	PRR
What it is	Proposal for a whole design	Record of one decision	Proposal circulated for broad comment	Operability checklist
When	Before implementation	At decision time	Before cross-team or public change	Before launch
Length	1–20 pages	~1 page	2–20 pages	Checklist
Audience	Team plus reviewers	Future maintainers	Whole organization or community	Operators and SRE
Output	Approved design	Durable rationale	Consensus decision	Launch go/no-go
Choose when	Non-trivial new system or change	Any one-way-door decision	Change affects many teams or users	New service or major change

Real-world industry applications

INDUSTRY EXAMPLE

Open-source RFCs as public design reviews. In the Rust and Python projects, any significant language change is proposed as a document in a public repository, discussed in the open, and accepted or rejected by a designated team, leaving a permanent record of the rationale and alternatives. They are among the best freely available corpora of real design reasoning — and good reading practice (see §3.5). *Evidence: Primary* — [Python PEP 1]; [Rust RFCs]

Failure scenarios

Table 1.17 — Design-process failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Review as rubber stamp	Reviewers lack time or context	Incidents in "reviewed" systems	Postmortem identifies missed risk	Named reviewers with explicit checklists
Design by committee	Every reviewer's preference absorbed	Bloated design; slipped dates	Author re-asserts goals and non-goals	Clear decision owner
Doc written after the fact	Process seen as paperwork	Doc describes code, lists no alternatives	Treat as documentation, not review	Review before implementation starts

1.11 How system design interviews work

1.11.1 Format

DEFINITION

Definition — System design interview. A 45–60-minute interview in which a candidate designs a system from an open-ended prompt (for example, “design a photo-sharing service”) in conversation with an interviewer, who evaluates the candidate’s process, technical judgment and communication rather than a single correct answer.

First-principles explanation

The interview is a compressed design review in which you are the author and the interviewer is a friendly but probing reviewer. There is no single correct answer because real design has none; what is being assessed is whether you would be a good person to have in a real design review.

How it works internally

Formats differ between companies and levels (Ch 90 separates verified public information from general advice), but most follow a shape like Figure 1.9. Popular preparation guides describe a similar sequence — understand the problem and scope, propose a high-level design, dive deep, wrap up [Xu 2020].

Figure 1.9 — A typical 45-minute system design interview.



Reading the diagram.

- **Components.** Six phases, each with a suggested time budget for a 45-minute slot.
- **Direction.** Left to right. The dotted arrow shows that deep dives often send you back to revise the high-level design — that is expected, not a failure.
- **What each phase produces.** A short list of FRs and numeric NFRs; a few key numbers (peak RPS, storage per year); a handful of API calls and core tables; a diagram like Figure 1.6 or 1.7; a deep analysis of one or two components (feed generation, partitioning, consistency); a summary of bottlenecks, failure modes and what you would do next.
- **Failure points.** Spending 20 minutes on requirements, or jumping to the diagram with no requirements at all; going silent; designing for scale nobody asked for.
- **Scaling to longer formats.** In a 60-minute slot, the extra time usually goes to deep dives.

What interviewers evaluate

Table 1.18 — Interview signals: weak vs strong.

Signal	Weak	Strong
Requirements	Starts drawing immediately	Asks targeted questions; states FRs, numeric NFRs and non-goals
Estimation	Skips, or computes to six digits	Order-of-magnitude numbers that drive decisions
Design	Lists technologies by name	Explains what each component is for and why it is needed
Trade-offs	"X is the best"	"X gives A at the cost of B; given our NFRs, that's acceptable"
Depth	Shallow everywhere	Deep on the hardest part of *this* problem
Failure thinking	Assumes nothing breaks	Names SPOFs, failure modes and recovery
Communication	Monologue or silence	Thinks aloud, checks in, adapts to hints

Expectations rise with level: a junior candidate is expected to produce a working design with guidance; a senior candidate to drive the conversation and identify the hard parts unprompted; a staff-level candidate to reason about evolution, organizational and cost concerns, and multiple viable designs (rubrics by level are in Ch 88).

> INTERVIEW TIP
>
> **Interview tip — the first two minutes.** For "design PhotoShare", a strong opening sounds like: *"Let me confirm scope: upload, follow, and a home feed of followed users' photos — chronological, not ranked? I'll leave out video, search and messaging. For scale, shall I assume around 10 million daily users, read-heavy — maybe 50 views per upload? And for quality targets: feed p99 under a few hundred milliseconds, uploads must never be lost, and a few seconds of delay before followers see a new photo is fine?"* In thirty seconds you have stated FRs, non-goals, scale, read/write ratio, latency, durability and consistency — and given the interviewer a chance to redirect.

Variants include **low-level/object-oriented design** interviews (Ch 38), **ML system design** interviews (Ch 92), and product-flavored versus infrastructure-flavored prompts. Full preparation is in Book III (Ch 87–92).

1.12 Common beginner mistakes

Most beginner mistakes are versions of one error: *designing without stating requirements and numbers*. Table 1.19 lists the twelve most common.

Table 1.19 — Common beginner mistakes and their fixes.

#	Mistake	Symptom	Fix	See
1	Jumping to a solution	Diagram before requirements	Spend the first minutes on FRs and NFRs	§1.6–1.7
2	Vague NFRs	“Fast and scalable”	Numbers, percentiles, windows	§1.7
3	No estimation	Arbitrary component counts	Back-of-the-envelope RPS, storage, bandwidth	§1.3.2, Ch 89
4	Over-engineering	Microservices, Kafka and sharding for 1,000 users	Design for ~10× current load; evolve	§1.9
5	Buzzword architecture	Technologies named, purpose unexplained	Justify every box by a requirement	§1.3
6	Ignoring failure	No answer to “what if this node dies?”	Name SPOFs; add redundancy where the NFR demands it	Ch 39
7	Averages instead of tails	p99 never mentioned	Specify and reason in percentiles	§1.7.1, §4.8
8	Stateful app servers	Sessions or files on local disk	Push state into shared stores	§18.5
9	One database for everything, forever	DB is every bottleneck	Cache, replicas, partitioning — when measured	Ch 14, Ch 20
10	Silent trade-offs	Only advantages stated	Always name the cost of each choice	§1.7.3
11	Forgetting the data lifecycle	No deletion, retention or backup plan	Treat deletion, backup and retention as requirements	§1.6, §39.9
12	Forgetting operations	No monitoring, deploy or rollback story	Include observability and deployability	Ch 40, Ch 43

COMMON MISTAKE

The two-sided trap. Mistakes 4 and 9 are opposites, and beginners often swing from one to the other. The cure for both is the same: let *measured or estimated* load and *stated* requirements decide when each component is added. Figure 1.1 was right for v0; Figure 1.7 was right for v3; neither is right for the other.

RECOMMENDATION

A five-question self-check before any design discussion. (1) What must it do? (2) How well — in numbers? (3) How big — in RPS, bytes and growth? (4) What fails, and what happens then? (5) What did I give up, and why is that acceptable?

1.13 Summary and revision notes

SUMMARY

- **System design** is deciding components, responsibilities, interfaces, data flow, storage and deployment so a system meets functional *and* non-functional requirements under stated load and failure assumptions. A design is only good relative to its requirements.
- **Software architecture** is the set of significant, hard-to-change structures of a system; system design is its large-scale, runtime-focused part. Describe architecture with multiple views (4+1, C4).
- **HLD** fixes components, boundaries and contracts; **LLD** fixes the internals of one component. Design moves between the two levels.
- **Functional requirements** say what the system does; write them testably, prioritize them, and list non-goals. Some “small” FRs (account deletion) have large architectural consequences.
- **Non-functional requirements** say how well; they drive the architecture. Make them measurable (number, percentile, window, load) using quality attribute scenarios, SLIs and SLOs.
- 99.9% availability allows 43.2 minutes of downtime per 30 days; components in series multiply availabilities, so each synchronous dependency lowers the total.
- Quality attributes conflict (consistency vs availability, durability vs latency, availability vs cost). Name the tension and choose per operation.
- Record one-way-door decisions as **ADRs** (title, status, context, decision, consequences); supersede, never rewrite. Use weighted matrices to make priorities explicit.
- Decisions about **where data lives and in what shape** are expensive to reverse; decisions about compute quantity are cheap. Avoid both under- and over-engineering.
- Industry practice: **design docs** → **design review** → ADRs → implementation → **production readiness review** → launch → **postmortems**; RFC processes broaden review.
- Interviews compress a design review into 45–60 minutes: requirements, estimation, API and data, high-level design, deep dives, trade-offs.

1.14 Knowledge checkpoint

1. In one sentence each, define *system design* and *software architecture*.
2. Why is “a single server” a good design for PhotoShare v0 but a bad one for v3?
3. Give two things that belong in an HLD and two that belong in an LLD.
4. Rewrite “uploads should be fast” as a measurable NFR.

5. What are the six parts of a quality attribute scenario?
6. How much downtime does 99.95% availability allow in a 30-day month?
7. Three components with availabilities 0.999, 0.999 and 0.9999 are in series. What is the path's availability?
8. Distinguish availability from reliability with an example.
9. Distinguish an SLI, an SLO and an SLA.
10. What are the five sections of Nygard's ADR format?
11. Why is an accepted ADR superseded rather than edited?
12. A product has 2 million DAU, each making 30 API requests per day, with peak factor 3. Estimate average and peak RPS.
13. Which PhotoShare requirement first forced the move off a single server — load or availability? Why?
14. Name the five typical sections of a design doc as described in public accounts of Google's practice.
15. Name four of the twelve beginner mistakes in Table 1.19 and the fix for each.

Answers

1. System design: choosing components, interfaces, data flow, storage and deployment so a system meets its functional and non-functional requirements under stated load and failure. Software architecture: the set of structures needed to reason about a system — its elements, their relations and the properties of both [Bass et al. 2021].
2. At 1,000 DAU the peak load is about 1.7 RPS and simplicity dominates. At 10 million DAU the load (about 17,400 photo views per second at peak, 5 TB/day of new data), the zone-failure requirement and the 99.9% availability target all exceed what one machine — itself a single point of failure — can deliver.
3. HLD: the list of services and data stores; the protocol and sync/async nature of each interaction (also deployment topology, data ownership). LLD: class structure and interfaces; table schemas and indexes (also algorithms, error handling, state machines).
4. For example: "For photos up to 20 MB under normal load at up to 70 uploads per second, 99% of uploads return a photo ID within 2 s, measured server-side over a 30-day window."
5. Source, stimulus, environment, artifact, response, response measure.
6. $(1 - 0.9995) \times 43,200 = 21.6$ minutes.
7. $0.999 \times 0.999 \times 0.9999 = 0.998001 \times 0.9999 \approx 0.99790$, about 99.79%.
8. Availability is the fraction of time the system is up and serving; reliability is the probability it operates correctly over a period. A feed service that responds quickly but sometimes shows other users' private photos is available but unreliable (and insecure).
9. SLI: the measured quantity (fraction of successful requests). SLO: the target for that SLI over a window (≥ 99.9% per 30 days). SLA: a contract with consequences if a stated target is missed.

10. Title, Status, Context, Decision, Consequences.
11. So that the historical reasoning remains intact: future readers can see what was decided, under what context, and what replaced it. Editing would erase the record of why the earlier choice was once right.
12. Requests/day $= 2 \times 10^6 \times 30 = 6 \times 10^7$; average $= 6 \times 10^7 / 86,400 \approx 694$ RPS; peak $\approx 3 \times 694 \approx 2,083$ RPS.
13. Availability. At 100,000 DAU the load was still small, but deploy downtime plus occasional VM failures gave roughly 99.76%, below the 99.9% target; only multiple stateless servers behind a load balancer remove deploy downtime and the single point of failure.
14. Context and scope; goals and non-goals; the actual design; alternatives considered; cross-cutting concerns [Ubl 2020].
15. Any four, e.g. jumping to a solution → clarify FRs and NFRs first; vague NFRs → add numbers, percentiles and windows; over-engineering → design for about 10× current load and evolve; ignoring failure → name SPOFs and the recovery path.

1.15 Interview questions

Basic

Q1. What is the difference between functional and non-functional requirements? Give an example of each for a URL shortener.

Answer. Functional requirements describe behavior — what the system does: “given a long URL, return a short code; visiting the short URL redirects to the long one”. Non-functional requirements describe quality — how well, under what constraints, with numbers: “redirects complete with p99 ≤ 50 ms server-side at 10,000 RPS; the service is available 99.95% per month; a created mapping is never lost”. The NFRs drive the design: the latency and read volume suggest caching (Ch 20), and the durability requirement suggests a replicated store (Ch 27). The full design is in §52.1.

Q2. What is a single point of failure, and how would you find them in a design?

Answer. A component whose failure alone makes the whole system (or a critical function) unavailable. Trace each critical request path end to end and, for every component on it, ask “if this one instance disappears now, what happens?”. Typical SPOFs are a lone database primary, a single load balancer, one availability zone, a shared configuration service, and a person who is the only one able to deploy. Removing a SPOF usually means redundancy plus automatic failover, but each removal costs money and complexity, so remove the ones the availability NFR requires.

Intermediate

Q3. Your product manager asks for “five nines”. How do you respond?

Answer. First translate: 99.999% allows about 26 seconds of downtime per 30-day month — less than a typical database failover and less than most deploy rollbacks. Then ask what the business impact of downtime actually is and which user journeys need the target (checkout, perhaps; profile editing, probably not). Explain the cost: multi-zone or multi-region active–active deployment, automated failover, rigorous change management, and every synchronous dependency must itself exceed five nines because availabilities multiply in series. Propose per-journey SLOs (for example 99.95% for checkout, 99.9% elsewhere) with an error budget, and revisit with data [Beyer et al. 2016].

Q4. When would you introduce a cache, and what new problems does it bring?

Answer. When measurements or estimates show a read-heavy workload where a backing store is the bottleneck or too slow for the latency NFR, and where data can tolerate some staleness or be invalidated reliably — as at PhotoShare v2 (Figure 1.6). New problems: stale reads and invalidation (§20.10); cold-start and stampedes when popular keys expire together (§20.11); a cache outage overwhelming the database (§20.13); memory cost; and a new component to operate. State the expected hit ratio and what happens on a miss and on cache failure.

Advanced

Q5. Walk through how you would evolve a single-server photo-sharing app to 10 million DAU. What triggers each change?

Answer. Estimate first: at 10 million DAU with 50 views and 0.2 uploads per user per day, about 5,800 average and 17,400 peak photo views per second, about 1,700 peak feed API RPS, 5 TB/day of new data and about 28 Gbit/s peak egress. Then evolve by trigger, not by fashion. (1) Availability target and disk growth → stateless app servers behind a load balancer, managed database with standby, object storage and CDN; data location is decided early because it is expensive to change later. (2) Database CPU and upload latency → cache with cache-aside, read replicas with read-your-writes routing, async thumbnail jobs. (3) Zone-failure tolerance and write volume → services split by data ownership, metadata partitioned by user ID, an event log driving fan-out-on-write feed precomputation with a fan-out-on-read path for celebrity accounts, and multi-zone deployment with capacity headroom. At each stage, name the new failure modes (stale reads, cache avalanche, consumer lag, hot partitions) and how you would observe them.

Q6. Two senior engineers disagree about whether to adopt a new distributed database. How would you drive the decision?

Answer. Classify reversibility first: a primary datastore holding production data is a one-way door, so it deserves a design doc and review. Agree the decision drivers (specific NFRs with numbers), criteria and weights *before* scoring options, including "do nothing" and an incremental option (for example partitioning the existing database). Replace opinion with evidence where possible: a time-boxed prototype under representative load, failure-injection tests, and an estimate of migration and operational cost (team skills, on-call burden). Score with a weighted matrix, run a sensitivity check to see which weight changes would flip the result, and record the outcome as an ADR with explicit negative consequences and the conditions under which it should be revisited. The goal is a decision both engineers can commit to even if one disagrees.

1.16 Practical exercises

1. **Run and extend the estimator.** Run the §1.3.2 script. Add fields for feed API requests (5 per user per day) and replication factor (3), and print API RPS and replicated storage per year for 1 million and 10 million DAU.
2. **Write NFRs.** Pick an app you use daily. Write five measurable NFRs, one of them as a full six-part quality attribute scenario.
3. **Availability arithmetic.** A request path has a CDN (99.99%), API gateway (99.95%), service (99.9%) and database (99.95%). Compute the path availability and the monthly downtime it implies. Which component would you make redundant first, and why?
4. **Write an ADR.** Write ADR-0004 for PhotoShare: "Generate thumbnails asynchronously". Include context with numbers, at least two options considered, and at least two negative consequences.
5. **Read a real design record.** Choose one accepted Python PEP or Rust RFC. Identify its context, decision, alternatives and consequences, and compare its structure with Nygard's ADR format.

1.17 Design challenges

1. **PhotoShare goes private.** Add the requirement "accounts can be private; only approved followers can see their photos". Which components in Figure 1.7 change? How do you stop the CDN from serving private photos to anyone with the URL? (Hint: signed URLs with short TTLs; see Ch 21.)
2. **PhotoShare in two regions.** EU regulators require EU users' photos and metadata to stay in the EU. Sketch the changes to Figure 1.7, list the new failure modes, and state which NFR tensions from Table 1.10 become sharper. (See Ch 44.)
3. **Cost ceiling.** The business caps infrastructure cost at $0.01 per monthly active user per month. Using the v3 numbers, identify the two largest cost drivers (con-

sider storage growth and egress) and propose design changes — storage tiers, image formats, CDN cache hit ratio — with estimated savings. (See Ch 51.)

1.18 References

1. Barroso, L. A., Hölzle, U., Ranganathan, P. *The Datacenter as a Computer: Designing Warehouse-Scale Machines*, 3rd ed. Morgan & Claypool, 2018.
2. Bass, L., Clements, P., Kazman, R. *Software Architecture in Practice*, 4th ed. Addison-Wesley, 2021.
3. Beyer, B., Jones, C., Petoff, J., Murphy, N. R. (eds.). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016. https://sre.google/sre-book/table-of-contents/
4. Boehm, B., Basili, V. R. "Software Defect Reduction Top 10 List." *IEEE Computer*, 34(1), 2001.
5. Bradner, S. *The Internet Standards Process — Revision 3*. RFC 2026, IETF, 1996. https://www.rfc-editor.org/rfc/rfc2026
6. Bryar, C., Carr, B. *Working Backwards: Insights, Stories, and Secrets from Inside Amazon*. St. Martin's Press, 2021.
7. Conway, M. E. "How Do Committees Invent?" *Datamation*, 14(4), 1968.
8. Crocker, S. *Host Software*. RFC 1, 1969. https://www.rfc-editor.org/rfc/rfc1
9. Dean, J., Barroso, L. A. "The Tail at Scale." *Communications of the ACM*, 56(2), 2013.
10. Fowler, M. "Who Needs an Architect?" *IEEE Software*, 20(5), 2003.
11. Instagram Engineering. "What Powers Instagram: Hundreds of Instances, Dozens of Technologies." Instagram Engineering blog, 2011. https://instagram-engineering.com/what-powers-instagram-hundreds-of-instances-dozens-of-technologies-adf2e22da2ad
12. ISO/IEC 25010. *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. ISO/IEC, 2011; revised 2023.
13. ISO/IEC/IEEE 42010. *Software, systems and enterprise — Architecture description*. ISO/IEC/IEEE, 2011; revised 2022.
14. Kleppmann, M. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
15. Kruchten, P. "Architectural Blueprints — The '4+1' View Model of Software Architecture." *IEEE Software*, 12(6), 1995.
16. Kubernetes Authors. *Kubernetes Enhancement Proposals (KEPs)*. GitHub. https://github.com/kubernetes/enhancements
17. Nygard, M. "Documenting Architecture Decisions." Cognitect blog, 15 November 2011. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions
18. Warsaw, B., Hylton, J., Goodger, D., Coghlan, A. *PEP 1 — PEP Purpose and Guidelines*. Python Software Foundation. https://peps.python.org/pep-0001/

19. The Rust Project. *Rust RFCs*. GitHub. https://github.com/rust-lang/rfcs
20. U.S. Securities and Exchange Commission. *In the Matter of Knight Capital Americas LLC* (administrative proceeding order). SEC, 2013.
21. Ubl, M. “Design Docs at Google.” *Industrial Empathy*, 2020. https://www.industrialempathy.com/posts/design-docs-at-google/
22. Winters, T., Manshreck, T., Wright, H. *Software Engineering at Google: Lessons Learned from Programming Over Time*. O’Reilly Media, 2020.
23. Xu, A. *System Design Interview – An Insider’s Guide*. Independently published, 2020.

1.19 Open verification items

- [Nygard 2011] and [Ubl 2020]: URLs could not be fetched from the authoring environment (network-restricted); existence and dates confirmed only via search results. Re-check URLs and the Ubl page-length claim (“roughly 10–20 pages”; “mini design docs”).
- [SEC 2013]: confirm release number and URL of the Knight Capital order, and the figures “one of eight servers”, “about 45 minutes” and “more than $460 million”.
- YouTube “Gangnam Style” 32-bit counter (December 2014): locate a primary source (YouTube/Google post) or keep as Secondary.
-
- [Boehm and Basili 2001]: confirm wording of the after-delivery cost-of-fix item; the commonly quoted “100×” multiplier is deliberately not stated.
- [ISO/IEC 25010] and [ISO/IEC/IEEE 42010]: confirm exact current titles of the 2023 and 2022 revisions.
-
-

Chapter 2 — How to Think Like a System Designer

> **Level:** 1–2 · **Track:** Shared · **Estimated reading time:** 3 h

Chapter 1 told you *what* system design is. This chapter is about *how* experienced designers think while they do it. It does not cover specific technologies (databases, caches, message brokers). Those come in Parts II–VI. What you get here is a set of thinking tools that apply to every design problem in this book, whether it is a URL shortener, a payment system or a large-language-model serving platform.

To keep the tools concrete, the whole chapter follows one running problem from a vague request to a documented architecture:

> WORKED EXAMPLE
>
> **Running problem — QuickBite.** A start-up asks you to “build the backend for our food-delivery app.” Customers browse nearby restaurants, put dishes in a cart, pay, and watch a courier bring the food. Restaurants receive orders on a tablet and mark them ready. Couriers receive delivery jobs on a phone and share their location while driving. QuickBite runs in one country today and wants to grow.
>
> QuickBite is a hypothetical company invented for teaching. Every number attached to it in this chapter is an assumption we make and state openly. None of them is a measurement of any real food-delivery company.

2.1 Learning objectives

After this chapter you should be able to:

- **Decompose** a product description into components with clear responsibilities, using cohesion, coupling and data ownership as criteria.
- **Draw** a system-context diagram that separates what you are building from the people and external systems it depends on.
- **Elicit, quantify and negotiate** functional and non-functional requirements, turning vague wishes (“fast”, “real-time”, “never down”) into testable numbers.
- **Identify the bottleneck** of a request path using the resource model (CPU, memory, disk, network, locks and other software resources), the utilization law and Little’s

Law.
- **State assumptions** explicitly, rank them by how much the design depends on them, and plan how to check them.
- **Evaluate trade-offs** with a weighted decision matrix, and judge how much process a decision needs by asking whether it is reversible (a “two-way door”) or not (a “one-way door”).
- **Design for failure** by listing failure modes, limiting blast radius and computing the availability of a dependency chain.
- **Reason about scale** in orders of magnitude, and predict which parts of a design break at 10×, 100× and 1000× the load.
- **Perform** a complete back-of-the-envelope capacity estimate with explicit arithmetic.
- **Document** an architecture with the C4 model, a design document and Architecture Decision Records (ADRs).

2.2 Prerequisites

- Functional vs non-functional requirements — §1.6 and §1.7. This chapter turns them into numbers and negotiates them.
- High-level vs low-level design — §1.5. Decomposition here stays at the high level.
- Powers of ten and units (KB vs KiB) — §4.3 and §4.5. Every estimate in §2.10 and §2.11 depends on them. This chapter uses SI prefixes throughout (1 KB = 10^3 bytes, 1 GB = 10^9 bytes).
- A rough idea of what a server, a database and a network request are. No deeper knowledge is assumed. Every other term is defined when first used.

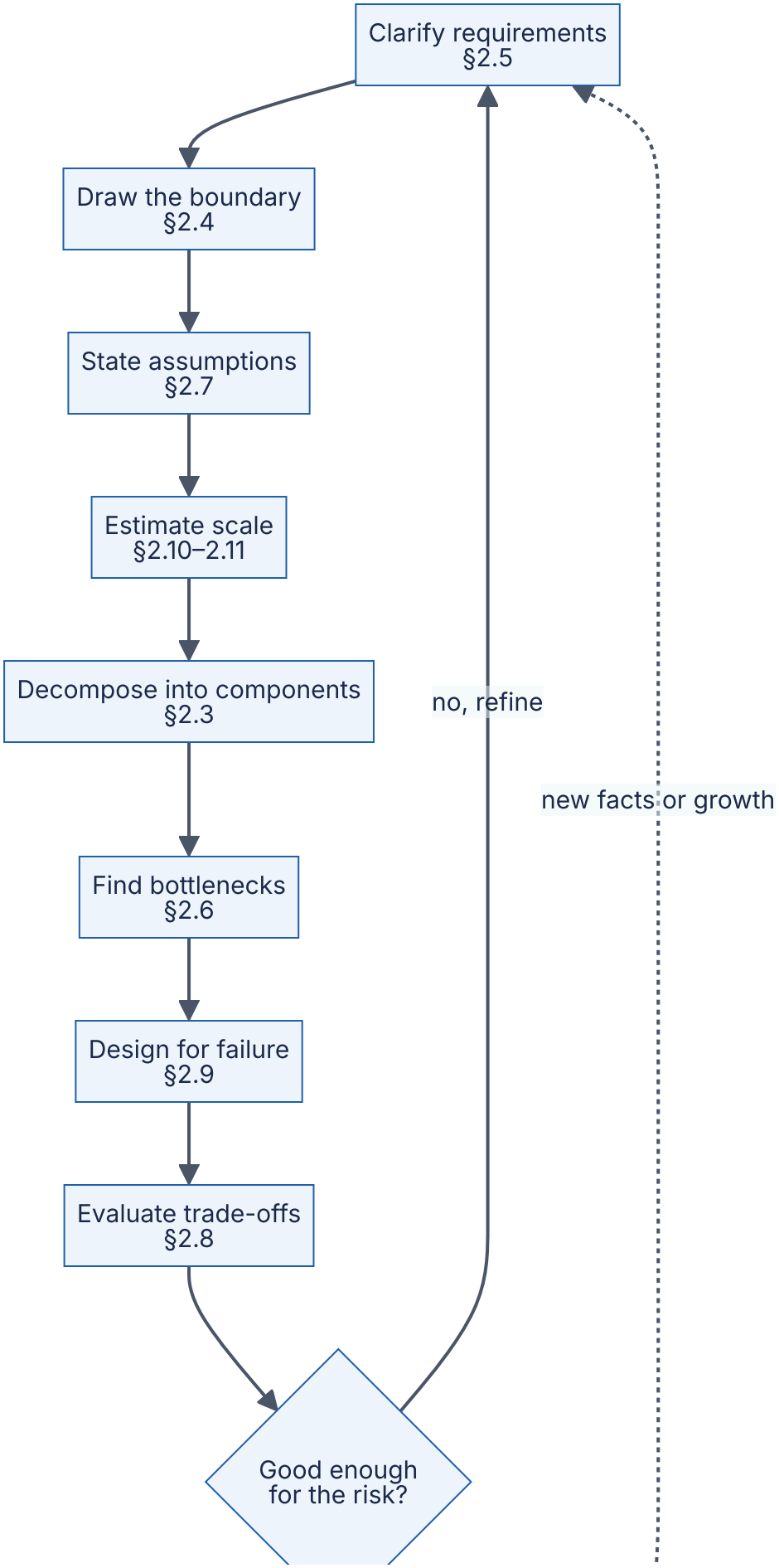
Helpful but optional: Little’s Law (§4.11) and availability arithmetic (§4.14). This chapter introduces both informally and points forward to the full treatment.

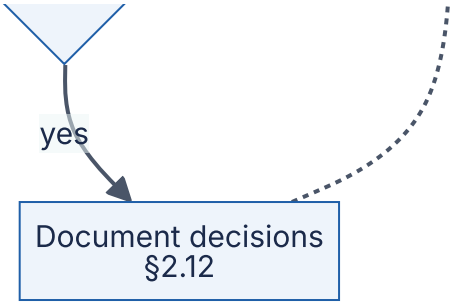
2.3 Decomposition into components

2.3.1 The thinking loop and where decomposition fits

Before the first tool, look at the whole process. Experienced designers do not move in a straight line from requirements to boxes and arrows. They work in a loop. They make a rough pass through every concern, find the riskiest part, and go round again at higher resolution. The sections of this chapter are the tools used at each step of that loop. They are presented one at a time for teaching, but in practice you switch between them constantly.

Figure 2.1 — The system designer’s thinking loop.


Clarify requirements §2.5
Draw the boundary §2.4
State assumptions §2.7
Estimate scale §2.10–2.11
Decompose into components §2.3
Find bottlenecks §2.6
Design for failure §2.9
Evaluate trade-offs §2.8
Good enough for the risk?
no, refine
new facts or growth



Reading the diagram. Each rectangle is a thinking activity, labeled with the section that teaches it. It is not a software component. Solid arrows show the usual order within one pass. The diamond is the decision every designer makes at the end of a pass: is the design good enough *for the risk involved*? A prototype for ten users clears that bar after one rough pass. A payment system may need many passes. The "no, refine" edge returns to requirements because a weak design is very often caused by a misunderstood requirement. The dotted arrow from documentation back to requirements is asynchronous: weeks or months later, production data or business growth disproves an assumption and starts the loop again. Nothing in this loop "fails" in the software sense. The failure to watch for is getting stuck, for example by polishing component diagrams (§2.3) before estimating scale (§2.11). The way to avoid that is to time-box each pass.

2.3.2 What decomposition is

DEFINITION

Definition — Decomposition. Dividing a system into smaller parts (**components**) so that each part has one clear responsibility, a well-defined interface, and as few dependencies on other parts as possible. Each part can then be understood, built, scaled and changed mostly on its own.

First-principles explanation

Think of a restaurant kitchen. Nobody "does the cooking." There is a grill station, a fry station, a pastry station and a pass where plates are checked before they leave. Each station has its own tools and ingredients and gets work through a narrow interface: the ticket. When the fryer breaks, the grill keeps working. When dessert orders double, you add a pastry cook without touching the grill.

Software decomposition works the same way. You split a big, fuzzy problem ("the food-delivery backend") into parts. Each part answers one question well ("what is on the menu?", "where is the courier?"). The parts interact through narrow, explicit interfaces, usually an API or a message.

Why does this exist?

A system nobody can hold in their head cannot be reasoned about. Decomposition gives you four concrete benefits:

1. **Comprehension.** One component fits in one engineer's head.
2. **Independent scaling.** In QuickBite, courier location updates arrive thousands of times per second, but restaurants edit menus a few times per day. Separating them lets each be scaled on its own terms.
3. **Failure isolation.** A bug in the recommendation feature should not stop customers from paying.
4. **Team autonomy.** Different teams can own different components. Conway's observation that organizations produce designs that copy their own communication structures [Conway 1968] means team boundaries and component boundaries end up shaping each other whether you plan it or not.

David Parnas made the classic argument for *how* to choose the parts. Divide a system so that each module hides a design decision that is likely to change, rather than by the steps of a processing flowchart [Parnas 1972]. That principle, **information hiding**, is still the best single test of a good boundary.

How it works internally

A practical decomposition procedure:

1. **List the nouns and verbs.** From the product description: *customer, restaurant, menu, dish, cart, order, payment, courier, delivery, location, notification*. Verbs: *browse, search, add to cart, place order, pay, accept, prepare, assign, pick up, track, deliver, rate*.
2. **Group by data ownership.** Ask "who is the single source of truth for this data?" Menus belong with restaurants. Orders and their states belong together. Courier locations are a separate, high-volume stream.
3. **Group by rate of change and load profile.** Things that change at very different speeds, or see very different traffic, usually belong apart. Menus are read-heavy and rarely written. Location is write-heavy and short-lived.
4. **Group by consistency need.** Money movements need strong guarantees: no double charges. The display of a courier's position can tolerate being a few seconds stale.
5. **Name each component with one sentence of responsibility.** If the sentence needs "and", consider splitting it.
6. **Draw the interactions.** Mark each one as synchronous (the caller waits) or asynchronous (the caller hands off a message and moves on).
7. **Check against scenarios.** Walk the top three user journeys through the diagram. Every step must have exactly one owner.

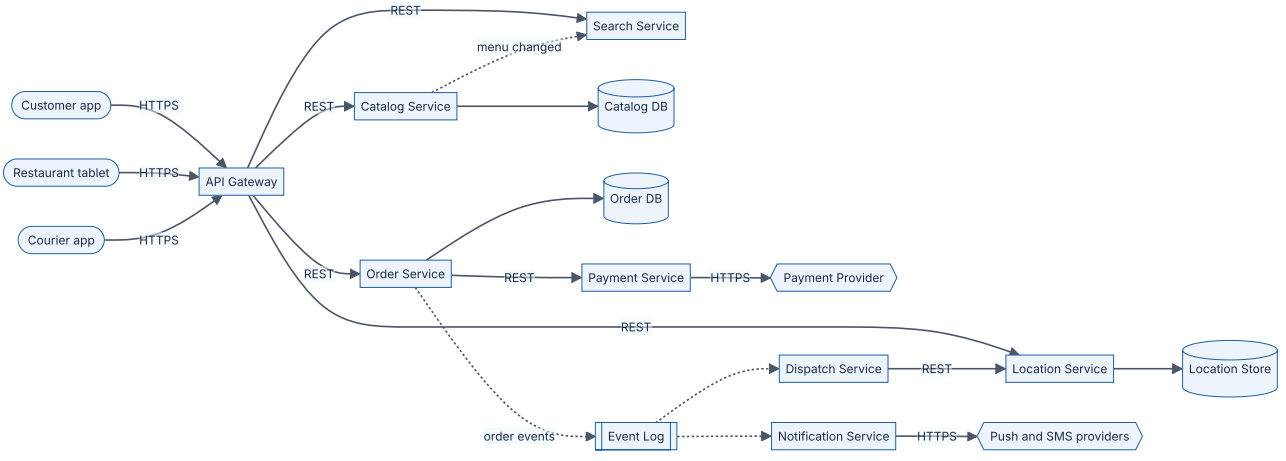
Applying this to QuickBite gives the components in Table 2.1.

Table 2.1 — QuickBite components and their single responsibilities.

Component	Responsibility (one sentence)	Owns data	Load profile	Consistency need
API gateway	Authenticates clients and routes requests	None (stateless)	All traffic	n/a
Catalog service	Serves restaurants, menus and prices	Restaurants, menus	Read-heavy	Minutes of staleness OK
Search service	Finds restaurants by location, cuisine, text	Search index (derived)	Read-heavy	Derived, can lag
Order service	Owns the life cycle of an order	Orders, order items	Moderate writes	Strong
Payment service	Charges and re-funds via a payment provider	Payment records	Low volume	Strong, idempotent
Dispatch service	Assigns couriers to ready orders	Assignments	Bursty	Strong per assignment
Location service	Ingests and serves courier positions	Recent positions	Write-heavy	Seconds of stale-ness OK
Notification service	Sends push, SMS and email	Delivery log	Bursty, async	At-least-once

Architecture diagram

Figure 2.2 — QuickBite first-pass decomposition.



Reading the diagram. Three client types (stadiums) reach the system only through the **API gateway** over HTTPS. The gateway is stateless, so it scales horizontally behind a load balancer (Chapter 19). Solid arrows are synchronous request/response calls. Placing an order calls the order service, which calls the payment service, which calls the external

payment provider (hexagon). That chain is the critical path, and every link in it can fail (§2.9). Dotted arrows are asynchronous. Once an order is committed, the order service publishes an event to the **event log** (subroutine shape; see Chapter 30). The dispatch and notification services consume it at their own pace, so a slow SMS provider cannot slow down checkout. Each cylinder is a storage boundary owned by exactly one service. No service reads another service's database, which keeps data ownership clear (§34.11). The catalog service pushes menu changes asynchronously to the search service, whose index is a *derived* copy that may lag by seconds. The location store is the heaviest write path, as §2.11 will show, so it scales independently, for example with an in-memory store partitioned by city. Likely failure points: the payment provider (external, outside your control), the order database (single source of truth) and the event log (if it stops, dispatch stops). Each needs a mitigation in §2.9.

> NOTE
>
> **Components are logical, not deployment units.** Figure 2.2 shows *logical* components. On day one, QuickBite might deploy all of them as one process: a **modular monolith** (§33.5). Decomposition decides where the *boundaries* are. Deployment decides how many *processes* there are. §2.8 makes that second decision with a decision matrix.

Real-world industry applications

> INDUSTRY EXAMPLE
>
> **Amazon's move to services.** In a 2006 interview, Amazon's CTO Werner Vogels described how Amazon's early monolithic application was restructured into services. Each service owned its data and could only be reached through a published interface. Direct database access between teams was ruled out. He linked this to team ownership ("you build it, you run it"). The design choice was data ownership behind interfaces, the same rule used in Figure 2.2. The trade-off he acknowledged was added operational and coordination complexity in exchange for team independence and scalability. *Evidence: Primary* — [Gray 2006]

Advantages and disadvantages

Good decomposition makes reasoning, scaling, failure isolation and team ownership easier. It also has costs. Every boundary that crosses a network turns a function call (~nanoseconds) into a remote call (~0.1–0.5 ms or more within a datacenter, see §4.17), and that call can fail independently. Data split across components loses cross-component transactions (Chapter 28). Too many components multiply operational load: more deployments, dashboards and on-call rotations.

Alternatives and trade-offs

Table 2.2 — Ways to choose component boundaries.

Approach	Splits by	Works well when	Main risk
By technical layer	UI / logic / data	Small systems, one team	Every feature change touches every layer
By business capability	Orders, catalog, payments	Most product systems	Needs domain understanding up front
By volatility (Parnas)	Decisions likely to change	Long-lived systems	Requires predicting change
By load profile	Read-heavy vs write-heavy	Very uneven traffic	Can fragment one domain concept
By domain-driven bounded context	Consistent language and model	Large organizations	Heavy analysis cost (§34.4–34.5)

In practice you combine them. QuickBite splits by business capability first, then splits *location* out of *dispatch* because of its very different load profile.

Failure scenarios

Table 2.3 — Decomposition failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Distributed monolith	Services share a database or must deploy together	Every release touches many services; cross-service schema locks	Merge tightly coupled services	Enforce data ownership; review coupling
Chatty interfaces	Boundary cuts through a hot loop	One user request causes dozens of internal calls; high p99	Batch calls or move the boundary	Walk scenarios through the diagram before building
Orphan responsibility	No component owns a step, e.g. refunds	Bugs bounced between teams	Assign an owner explicitly	Step 7 of the procedure: one owner per step
Premature split	Boundaries drawn before the domain is understood	Frequent cross-boundary changes	Re-merge into a modular monolith	Start coarse; split when evidence appears

Common misconceptions

COMMON MISTAKE

“More components means a better design.” The number of boxes is not a quality measure. A good decomposition has *few, cohesive* components with *narrow* interfaces. A useful test: if two components change together in most pull requests, they are probably one component.

COMMON MISTAKE

“Decompose by technology.” “The Redis service” or “the Kafka service” are not responsibilities. Components are named after what they do for the business (“location service”). The technology they use is an internal detail that may change.

2.4 System boundaries and context diagrams

2.4.1 Boundaries and the context diagram

DEFINITION

Definition — System boundary. The line that separates what you are designing and are responsible for from everything outside it: users, other teams’ systems and third-party services. You control what is inside. You can only *depend on* what is outside.

DEFINITION

Definition — Context diagram. A diagram that shows the whole system as a single box, surrounded by the people (actors) and external systems that interact with it, with each interaction labeled. Context diagrams come from structured analysis [DeMarco 1979] and are Level 1 of the C4 model [Brown C4].

First-principles explanation

Before renovating a house, a builder walks the property line. Fences, the street, the neighbor’s tree and the water main are not part of the job, but every one of them constrains it. A context diagram is that walk around the property line. It answers three questions before any internal design starts: *who uses this?*, *what does it depend on?* and *what is not our job?*

Why does this exist?

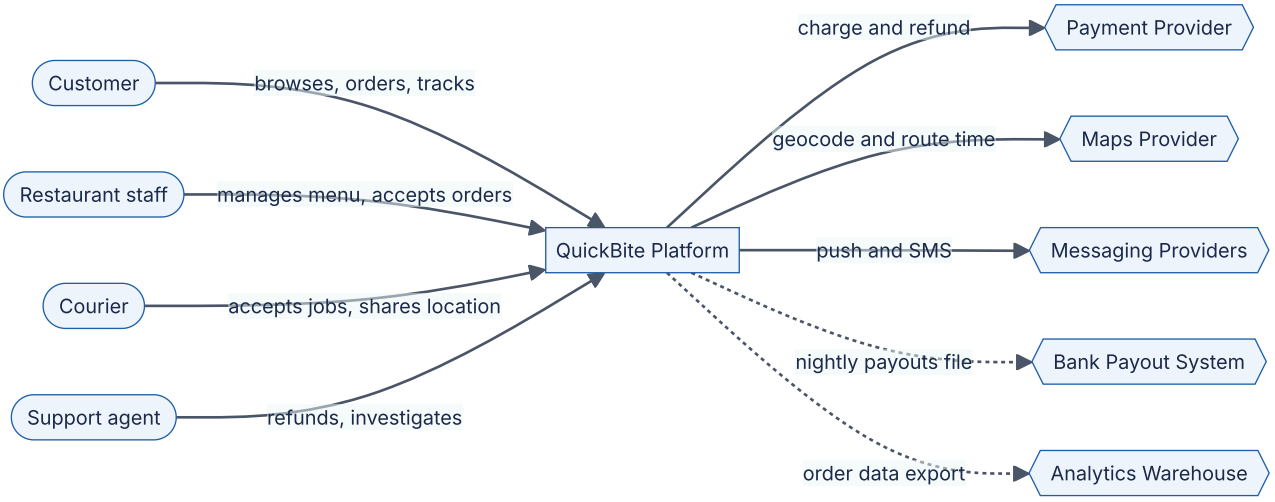
Many expensive design failures are scope failures, not technical ones. A team builds a payment system and finds out late that tax calculation was expected. Or it assumes an external maps API can take 10,000 requests per second when the contract allows 100. Drawing the boundary early brings these surprises to the surface while they are still cheap to fix. It also gives non-engineers, such as product managers, legal and finance, a diagram they can actually check.

How it works internally

1. Draw your system as one box. Resist the urge to draw its insides.
2. Around it, place every **actor** (human role) and every **external system**.
3. Label each interaction with *what* flows (not *how*): “places orders”, “sends card charge”.
4. For every external system, note three facts: its **availability promise** (its SLA, see §39.6), its **rate limits** and its **latency**. These numbers are inherited constraints you do not control.
5. Write down what is explicitly **out of scope**.

Architecture diagram

Figure 2.3 — QuickBite system-context diagram.



Reading the diagram. The single rectangle is the whole QuickBite platform, deliberately shown with no internal detail. Four human actors (stadiums) interact with it. Note the **support agent**, an actor often forgotten in first drafts, who needs refund and investigation tools. Five external systems (hexagons) sit outside the boundary. Solid arrows are synchronous, user-facing interactions. A customer waits while a card charge happens, so the payment provider’s latency and availability become part of QuickBite’s own checkout latency and availability. Dotted arrows are asynchronous batch flows. Payouts to restau-

rants and data exports run nightly and can be retried without users noticing. Each hexagon is a failure point outside your control. The maps provider is especially dangerous because both search (delivery-time estimates) and dispatch depend on it. Hexagons also bring rate limits and per-call costs, so scaling QuickBite may be limited by *contracts* rather than by servers.

WORKED EXAMPLE

Worked example — Inherited constraints from the boundary. Suppose the maps provider's contract allows 500 requests per second and charges $0.005 per route-time request. (These figures are assumptions for QuickBite, not any real provider's price list.)

1. If every restaurant listing in search results needs a live route-time estimate, and search peaks at 2,000 requests per second showing 20 restaurants each, QuickBite would need 2,000 × 20 = 40,000 route requests per second.
2. That is 40,000 / 500 = 80 times the contract limit.
3. Cost at that rate for one peak hour: 40,000 × 3,600 × $0.005 = $720,000.

Interpretation. Calling the maps provider per listing is impossible, regardless of how good the internal architecture is. **Engineering implication:** estimate delivery times internally, for example from precomputed travel-time grids per city zone, and call the provider only once per *placed order*. The boundary diagram exposed this constraint before any code was written.

Real-world industry applications

INDUSTRY EXAMPLE

C4 system-context diagrams. Simon Brown's C4 model makes the system-context diagram its first and most abstract level. It recommends showing the software system as one box with its users and the other software systems it depends on, and states that the diagram should be understandable by non-technical stakeholders. *Evidence: Primary* — [Brown C4]

Failure scenarios and misconceptions

Table 2.4 — Boundary failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Hidden dependency outage	An external system was never listed	Incident traced to an unlisted vendor	Add fallback; add it to the diagram	Inventory every outbound call
Rate-limit wall	Vendor limits not read	HTTP 429 errors at peak	Cache, batch, negotiate a higher tier	Step 4: record limits per external system
Scope creep	Boundary never agreed	Late "but we assumed you'd do X"	Re-plan	Explicit out-of-scope list signed off

COMMON MISTAKE

"External systems are always available." A third-party SaaS API typically promises somewhere between 99.9% and 99.99% availability, as of 2026. At 99.9% that is up to about 43 minutes of downtime per 30-day month. Your design must decide what QuickBite does during those minutes (§2.9).

2.5 Understanding and negotiating requirements

2.5.1 Eliciting and negotiating requirements

DEFINITION

Definition — Requirement. A statement of what the system must do (a **functional requirement**, e.g. "a customer can cancel an order before the restaurant accepts it") or how well it must do it (a **non-functional requirement**, e.g. "99% of order placements complete within 800 ms"). A requirement is **testable** if you could write an automated check or a measurement that proves it is met.

First-principles explanation

A customer asks a tailor for "a nice suit." A bad tailor starts cutting. A good tailor asks: for what occasion? Which climate? What budget? Ready by when? The good tailor is not being difficult. "Nice" can mean a hundred different suits, and only one of them is what the customer actually wants. Requirements gathering is the tailor's questions. **Negotiation** is the conversation that follows: "a hand-stitched suit by Friday for that budget is not possible, but a made-to-measure one is."

Why does this exist?

Vague requirements cause two opposite failures. **Under-engineering** builds a system that falls over on launch day because nobody asked how many users would arrive. **Over-engineering** spends six months building multi-region active–active replication for a prod-

uct with 200 users, because someone said “it must never go down.” Both come from the same root cause: a requirement nobody turned into a number.

Requirements also conflict. “Show the courier’s exact position in real time” pushes against “keep infrastructure costs under budget” and “protect courier privacy.” Someone has to decide how those conflicts are resolved, and the designer’s job is to make that decision *visible and deliberate*.

How it works internally

A reliable elicitation and negotiation procedure:

1. **Identify users and their top journeys.** For QuickBite: *browse and order*, *track delivery*, *restaurant accepts and prepares*, *courier delivers*.
2. **Write functional requirements as user-observable behaviors**, and rank them with **MoSCoW** priorities: Must, Should, Could, Won’t (this time).
3. **For each journey, ask the “-ility” questions** (§1.7): How many users? How fast? How available? How consistent? How durable? How secure? What does it cost?
4. **Quantify every non-functional requirement** as a metric, a threshold and a measurement window. “Fast” becomes “p99 < 800 ms measured over 5-minute windows”. The notation p99 means the 99th percentile: 99% of requests are faster than this value (§4.8).
5. **Differentiate by journey.** Availability and latency targets rarely need to be uniform. Checkout deserves a stricter target than the restaurant’s sales-history report.
6. **Surface conflicts and negotiate.** For each expensive requirement, show its cost and offer a cheaper alternative that still meets the underlying need.
7. **Record what was agreed, and what was explicitly rejected.**

Table 2.5 — QuickBite requirements after negotiation.

ID	Requirement (as first stated)	Negotiated, testable version	Priority
F1	“Customers can order food”	Customer can place an order of 1–30 items from one restaurant and pay by card or wallet	Must
F2	“Real-time tracking”	Courier position shown on a map, refreshed every 5 s; position at most 10 s old at p95	Must
F3	“Restaurants get orders instantly”	New order appears on the restaurant tablet within 5 s at p99	Must
F4	“Smart recommendations”	Deferred to phase 2; phase 1 shows “popular near you”	Won’t (now)
N1	“Never goes down”	Order placement available 99.95% per 30-day month; browsing 99.9%	Must
N2	“Fast”	Place-order p99 < 800 ms excluding payment-provider time; browse p99 < 300 ms	Must
N3	“Never lose an order”	Once the customer sees “order confirmed”, the order survives the loss of any single node or zone (RPO = 0 for orders)	Must
N4	“Never double-charge”	Each order is charged at most once, even if the client retries (idempotency, §22.15)	Must
N5	“Menus always correct”	Menu or price changes visible to customers within 60 s; price is re-checked at order time	Should

Notice what negotiation did to N1. “Never goes down” is impossible: no system reaches 100%. It was also unnecessary for most features. The negotiated version gives each journey its own target and makes the target measurable. RPO in N3 is the *recovery point objective,* the amount of recent data you may lose in a disaster (§39.8).

Mathematical foundations

The most useful number in a negotiation is the downtime a target actually allows.

FORMULA

Allowed downtime for an availability target

$$T_{down} = (1 - A) \times T_{period}$$

Symbol	Meaning	Units
T_{down}	Maximum downtime allowed in the period	minutes
A	Availability target, as a fraction	dimensionless (e.g. 0.9995)
T_{period}	Length of the measurement period	minutes

WORKED EXAMPLE

Worked example — What “three and a half nines” buys QuickBite. A 30-day month has 30 × 24 × 60 = 43,200 minutes.

1. Browsing at 99.9%: T_{down} = (1 − 0.999) × 43,200 = 0.001 × 43,200 = **43.2 minutes per month**.
2. Order placement at 99.95%: (1 − 0.9995) × 43,200 = 0.0005 × 43,200 = **21.6 minutes per month**.
3. A naive “five nines” (99.999%): 0.00001 × 43,200 = **0.43 minutes**, about 26 seconds per month.

Interpretation. Twenty-six seconds a month is less than the time a person takes to notice an alert, open a laptop and log in. Five nines requires fully automated detection and failover across independent zones or regions, which multiplies cost and complexity. **Engineering implication:** for a regional food-delivery start-up, 99.95% on checkout is ambitious but reachable with redundancy inside one region (§39.4). Five nines would be over-engineering, and the payment provider’s own availability probably caps the achievable number anyway (§2.9).

Real-world industry applications

INDUSTRY EXAMPLE

Service level objectives at Google. Google's Site Reliability Engineering book argues that 100% is the wrong reliability target for almost everything. Users cannot tell the difference between very high availability and 100%, because their own devices and networks are less reliable than that. Each extra nine costs a lot. Google's practice is to agree an explicit **service level objective (SLO)** with product owners and treat the remaining unreliability as an **error budget** to be spent on releasing features. This is requirement negotiation turned into an ongoing process (§39.6–39.7). *Evidence: Primary* — [Beyer et al. 2016]

Advantages, alternatives and trade-offs

Up-front negotiation costs meeting time, and stakeholders may not know the answers yet. When they do not, do not block. Make an assumption (§2.7), label it, and design so that it is cheap to change if wrong. In interviews, spend roughly the first five to ten minutes of a 45-minute session on requirements. In industry, requirements live in a design document (§2.12) and get re-negotiated as data arrives.

Failure scenarios

Table 2.6 — Requirements failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Launch-day overload	Peak load never quantified	Errors and latency spike at launch	Emergency scaling, load shedding (§22.9)	Ask "how many, at peak?"
Gold-plating	"Never down" accepted literally	Cost far exceeds the budget; delivery late	Simplify	Translate every adjective into a number
Silent conflict	Two stakeholders want contradictory things	Rework after review	Escalate with options and costs	Step 6: surface conflicts explicitly
Uniform targets	One SLO applied to all features	Expensive redundancy on unimportant paths	Tier the features	Step 5: differentiate by journey

Common misconceptions

COMMON MISTAKE

“Requirements are given to the designer.” In practice they are *discovered and negotiated* by the designer together with stakeholders. A designer who only takes orders will build exactly the wrong thing, very efficiently.

INTERVIEW TIP

Interview tip. Say your negotiated requirements out loud and write them on the board as numbers: “I’ll assume 2 million daily active users, checkout at 99.95%, tracking refreshed every 5 seconds — does that match what you have in mind?” Interviewers reward this because it shows you design *to a target*, not by habit.

2.6 Identifying bottlenecks

2.6.1 The resource model and the bottleneck

DEFINITION

Definition — Bottleneck. The resource whose capacity limits the throughput of the whole system for a given workload. Throughput cannot rise above the bottleneck’s capacity no matter how much the other resources are improved. Formally, it is the resource with the highest **service demand** per request (defined below).

First-principles explanation

Picture a highway that narrows from four lanes to one for a bridge. Widening the highway before or after the bridge to eight lanes changes nothing: cars still cross the bridge one lane at a time. The bridge is the bottleneck. Worse, traffic does not slow down evenly. It piles up in a queue *in front of* the bridge. That is why a system near its bottleneck shows rising latency long before it shows errors.

Two consequences follow, and they explain most performance work:

1. **Improving a non-bottleneck resource gives zero improvement in throughput.** Faster CPUs do not help if the database disk is the limit.
2. **Fixing the bottleneck moves it.** Once the bridge is widened, the next narrowest point becomes the new bottleneck. Performance work is a sequence of “find the bottleneck, relieve it, find the next one.”

Why does this exist?

Systems are made of finite resources. When a QuickBite lunch rush arrives, the first resource to run out decides whether customers see a spinner or an error. If you cannot name that resource, you will add capacity in the wrong place, and pay for servers that make no difference.

How it works internally: the resource model

Every request consumes a small amount of several **resources**. The classical hardware four plus one software category cover almost every case:

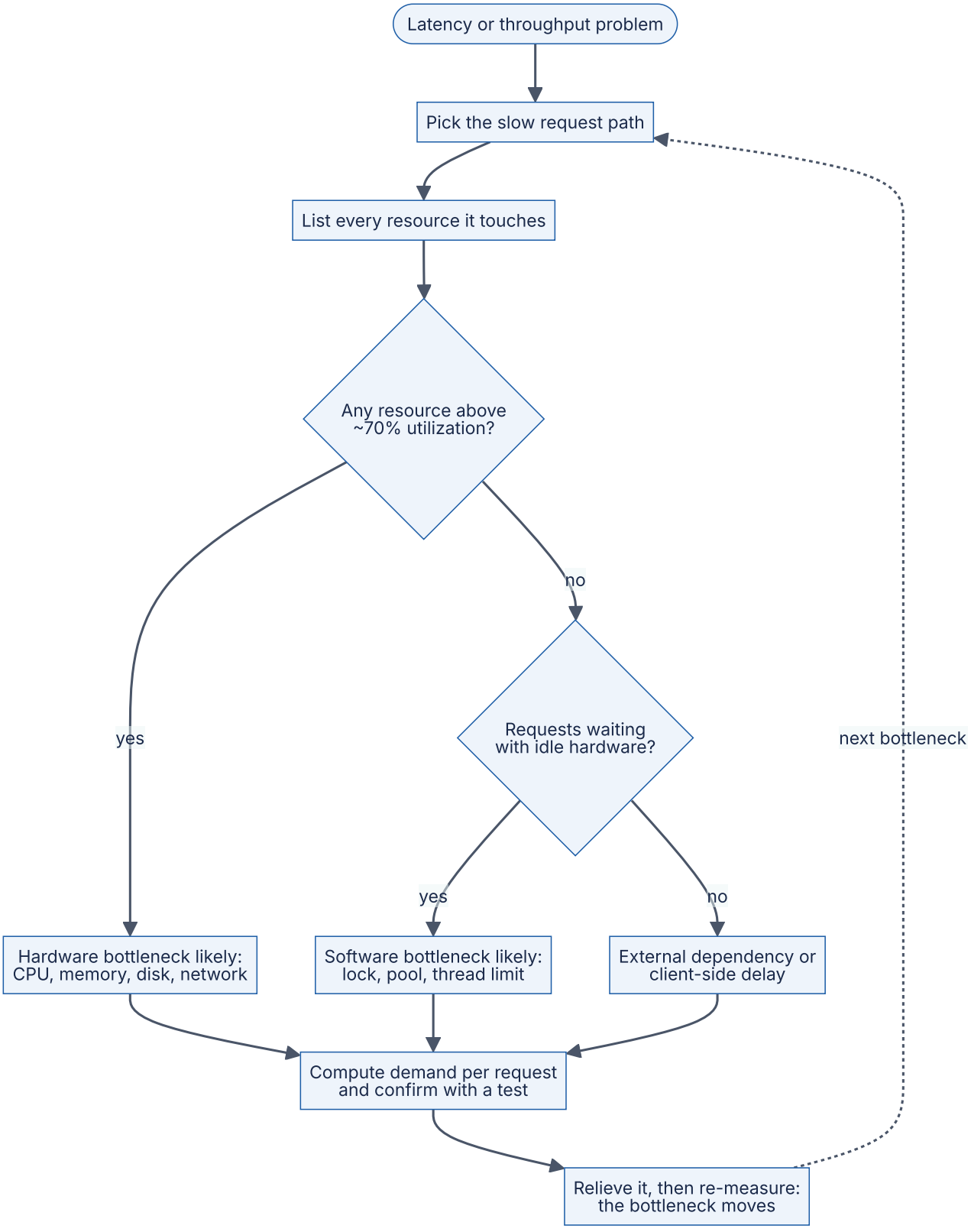
Table 2.7 — The resource model.

Resource	What a request consumes	Utilization signal	Saturation signal	Typical relief
CPU	Cycles for parsing, serialization, business logic, encryption	% busy per core	Run-queue length > core count	Optimize hot code, cache results, add cores or nodes
Memory	Working set, buffers, caches, per-connection state	% used	Swapping, OOM kills, GC pauses	Reduce footprint, add RAM, partition data
Disk	Reads and writes (IOPS), bytes per second, fsync latency	% time busy, IOPS used vs limit	Long I/O queue, rising await time	Cache, batch writes, faster media, partition
Network	Bytes in and out, packets, connections	Bandwidth used vs link capacity	Dropped packets, retransmits	Compress, CDN, bigger links, keep traffic local
Locks and other software resources	Exclusive access to a row, mutex, thread, connection-pool slot, file handle	Fraction of time held	Waiters queued; pool exhausted	Shorten hold time, finer-grained locks, bigger pools, partition the hot item

The last row is the one beginners miss. A **software resource**, such as a database row lock, a thread pool or a connection pool, is just as finite as a CPU. It can be the bottleneck while every hardware dashboard shows idle machines.

The standard diagnostic method is Brendan Gregg's **USE method**: for every resource, check **U**tilization, **S**aturation and **E**rrors [Gregg USE]. Figure 2.4 turns it into a procedure.

Figure 2.4 — Finding the bottleneck with the USE method.



Reading the diagram. This is a reasoning procedure, not a running system. You start from a symptom (stadium) and pick one slow request path, because bottlenecks are workload-specific: browsing and ordering have different ones. The first decision checks hardware utilization. Around 70% is a common warning threshold, because queueing delay grows steeply as utilization approaches 100% (§4.10). The second decision is the key di-

agnostic insight: *requests are waiting while the hardware is idle*. This almost always means a software resource such as a lock or a pool is saturated. If neither applies, the time is being spent outside your servers, for example in a third-party API or on a slow mobile network. Every branch leads to the same discipline: compute the demand per request and confirm the hypothesis with a measurement or load test (Chapter 23). The dotted edge back to the start is the loop that never ends: relieving one bottleneck exposes the next.

Mathematical foundations

Two laws from operational analysis of computer systems [Denning and Buzen 1978] are enough to find most bottlenecks with arithmetic alone.

FORMULA

Service demand, the utilization law and the bottleneck bound

$$D_i = V_i \times S_i \qquad U_i = X \times D_i \qquad X_{max} = \frac{c_i}{D_{max}}$$

Symbol	Meaning	Units
D_i	Service demand: total busy time one request places on resource i	seconds per request
V_i	Visits: how many times one request uses resource i	visits per request
S_i	Service time per visit to resource i	seconds per visit
U_i	Utilization of resource i, per unit of capacity	fraction (0–1)
X	System throughput	requests per second
c_i	Number of parallel units of resource i (e.g. cores)	units
X_{max}	Maximum throughput, set by the resource with the largest demand per unit	requests per second

The utilization law for a resource with c_i parallel units is $U_i = X \times D_i / c_i$. The bottleneck is the resource with the largest D_i / c_i.

FORMULA

Little’s Law (concurrency)

$$L = \lambda \times W$$

Symbol	Meaning	Units
L	Average number of requests in the system (or holding a resource) at once	requests
λ	Average arrival rate	requests per second
W	Average time each request spends in the system (or holds the resource)	seconds

Little’s Law holds for any stable system, regardless of the arrival pattern [Little 1961]. Full treatment in §4.11.

WORKED EXAMPLE

Worked example — Finding QuickBite's checkout bottleneck.

Situation. At dinner peak QuickBite receives 60 order placements per second (this number is derived in §2.11). Customers complain that checkout sometimes spins for 5 seconds and then fails, yet CPU dashboards show the database at only 25% utilization. The order transaction, as first written, does this:

1. Begin a database transaction.
2. Insert the order and its items.
3. Update the restaurant's row in `restaurant_load` to increment `open_orders` (used to stop accepting orders when a kitchen is overwhelmed). This takes a **row lock** until commit.
4. Call the payment service, which calls the payment provider. Average 300 ms.
5. Commit.

Measured per-order demands (assumed values for the exercise):

Table 2.8 — Per-order resource demand in the checkout path.

Resource	Demand per order D_i	Parallel units c_i	D_i/c_i	Max orders/s = $1/(D_i/c_i)$	Utilization at 60/s
API tier CPU	8 ms	32 cores	0.25 ms	4,000	60 × 0.25 ms = 1.5%
Database CPU	5 ms	8 cores	0.625 ms	1,600	3.8%
Database WAL disk (fsync)	2 ms	1 device	2 ms	500	12%
Network (25 KB per order, 10 Gbps link)	0.02 ms	1 link	0.02 ms	50,000	0.1%
DB connection pool (20 connections)	320 ms held	20	16 ms	62.5	96%
Row lock on one popular restaurant	320 ms held	1	320 ms	3.1	see below

Step 1 — Hardware. Every hardware resource is far below 70% utilization. The largest is the WAL disk at 12%, so hardware is not the bottleneck. Following Figure 2.4, "requests waiting with idle hardware" points to a software resource.

Step 2 — Connection pool, by Little's Law. Each order holds a database connection from step 1 to step 5: about 300 ms of payment time plus about 20 ms of database work, so $W = 0.32$ s. Then $L = \lambda \times W = 60 \times 0.32 =$ **19.2 connections busy on av-**

erage, out of a pool of 20. Utilization is 19.2 / 20 = 96%. Any burst above 62.5 orders/s (20 / 0.32) makes requests queue for a connection, and queueing at 96% utilization is severe (§4.10).

Step 3 — The row lock. A celebrity chef's restaurant runs a promotion and receives 8 orders per second. Each order holds that restaurant's row lock for 0.32 s, so the row can serve at most 1 / 0.32 = **3.1 orders per second**. Arrivals (8/s) exceed capacity (3.1/s), so the queue grows without bound until requests time out at 5 s. That matches the complaint exactly. This is the bottleneck for that workload: a single **hot row**.

Step 4 — Relieve it. Move the slow external call *out of* the database transaction. First authorize the payment, keyed by an idempotency key so retries cannot double-charge (§22.15). Then run a short transaction (insert order, update counter, commit) of about 15 ms. Then capture the payment, or publish an event to do so asynchronously.

Step 5 — Re-compute. Hold time W drops from 0.32 s to 0.015 s.

- Pool: $L = 60 \times 0.015 =$ **0.9 connections busy**, which is 4.5% of the pool.
- Hot row: maximum 1 / 0.015 ≈ **66 orders/s per restaurant**, comfortably above 8.
- The bottleneck has moved. It is now the WAL disk at 500 orders/s, about 8× headroom over today's peak. That is the next thing to watch as QuickBite grows (group commit and partitioning address it, §13.5 and Chapter 14).

Interpretation. Adding database servers or CPUs would have cost money and fixed nothing: the limit was a lock held during a network call. **Engineering implication:** never hold a lock or a pooled connection across a call to a slow or external system. In general, *hold time multiplied by arrival rate* (Little's Law) tells you how many units of any software resource you need.

Code example

A short, runnable script that turns the table above into a bottleneck report. Use it to check your own estimates.

```
        # Bottleneck finder using the utilization law (U = X * D / c).
        resources = {
            # name: (demand per request in seconds, parallel units)
            "api_cpu":   (0.008, 32),
            "db_cpu":    (0.005, 8),
            "wal_disk":  (0.002, 1),
            "network":   (0.00002, 1),
            "db_pool":   (0.320, 20),
            "hot_row":   (0.320, 1),
        }
        arrival_rate = 60.0  # requests per second

        for name, (demand, units) in sorted(
                resources.items(), key=lambda kv: kv[1][0] / kv[1][1], reverse=True):
            max_x = units / demand
            util = arrival_rate * demand / units
            flag = "SATURATED" if util >= 1 else ("HOT" if util >= 0.7 else "ok")
            print(f"{name:9s} max={max_x:9.1f}/s  util@{arrival_rate:.0f}/s={util:7.1%}
{flag}")
```

What to notice:

- Resources are sorted by D_i/c_i, so the first line printed is the bottleneck.
- `hot_row` shows a utilization above 100% (1920%) at 60/s, because the script applies the *whole* arrival rate to one row. That is the worst case where every order goes to a single restaurant. In the worked example only 8/s hit the hot row, which still exceeds its 3.1/s capacity. Edit the script to model per-key arrival rates.
- Changing `0.320` to `0.015` for the pool and the row reproduces Step 5 and shows the bottleneck moving to `wal_disk`.

Real-world industry applications

> INDUSTRY EXAMPLE
>
> **The USE method at Netflix and beyond.** Brendan Gregg developed the USE method while working on performance analysis and has published it, together with per-operating-system checklists, as a way to find resource bottlenecks quickly without guessing. It explicitly includes software resources such as mutex locks, thread pools and file descriptors alongside CPUs, memory, disks and network interfaces. *Evidence: Primary* — [Gregg USE], [Gregg 2020]

Alternatives and trade-offs

The USE method is resource-centric. Its complement is the **RED method** (Rate, Errors, Duration per service), which is request-centric and better for spotting *which* service is slow in a microservice graph. The two work well together (§23.3). Profiling (§23.4) finds

which code consumes a resource. Load testing (§23.5) confirms a bottleneck hypothesis empirically. The arithmetic above tells you where to point those tools.

Failure scenarios

Table 2.9 — Bottleneck-analysis failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Scaling the wrong tier	Added servers to a non-bottleneck	Throughput unchanged after scaling	Re-run USE analysis	Compute demand per resource first
Hot key or hot row	Skewed popularity, e.g. a viral restaurant	One key's latency far above the others	Shorten hold time, shard the counter	Model per-key rates, not just totals
Pool exhaustion	Long hold times, often across remote calls	"Timeout acquiring connection" errors with idle DB	Move remote calls out of transactions	Little's Law sizing
Retry amplification	Clients retry into a saturated resource	Load rises as success rate falls	Retry budgets, backoff with jitter (§22.12)	Cap retries; shed load

Common misconceptions

COMMON MISTAKE

"Low CPU means the system has spare capacity." CPU is one resource of five. QuickBite's checkout failed with every machine mostly idle, because the bottleneck was a lock. Always check software resources when latency is high and utilization is low.

COMMON MISTAKE

"Averages are enough." A system can be fine on average and saturated on one hot key, one shard or one minute of the day. Bottleneck analysis must use *peak* rates and *per-partition* rates (§14.10).

2.7 Making and stating assumptions

2.7.1 The assumption register

> DEFINITION
>
> **Definition — Design assumption.** A belief about the workload, the environment or the business that the design depends on but that has not yet been verified, for example “80% of requests are reads” or “the payment provider responds within 1 s at p99”. An assumption is **stated** when it is written down with its value, its source, your confidence in it and how it will be checked.

First-principles explanation

A weather forecaster does not refuse to forecast because tomorrow is unknown. She says “70% chance of rain,” which is an explicit, checkable statement of uncertainty. System designers face the same situation: you never have all the facts before you must design. The professional response is not to wait. It is to assume *out loud*, so that everyone can see which parts of the design rest on which beliefs.

Why does this exist?

Every design rests on assumptions anyway. The only choice is whether they are visible. Hidden assumptions are dangerous because nobody checks them, and when one turns out wrong, nobody knows which parts of the design must change. Stated assumptions turn surprises into planned decision points.

How it works internally

1. **Write each assumption as a number with a source.** “2 M daily active users (marketing forecast for year 1).”
2. **Rate confidence**: high (measured), medium (comparable data exists), low (guess).
3. **Do a sensitivity check.** Ask: *if this is wrong by 10×, does the design change?* Assumptions where the answer is “yes” are **load-bearing**.
4. **Attach a check and a tripwire** to every load-bearing assumption: how you will verify it, and the observed value that will trigger a redesign.
5. **Prefer designs that are robust to the assumptions you are least sure of.**

Table 2.10 — QuickBite assumption register (excerpt).

ID	Assumption	Value	Source	Confidence	Load-bearing?	Check and tripwire
A1	Daily active users	2 M	Business forecast	Low	Yes: sizes every tier	Weekly DAU dashboard; redesign review at 5 M
A2	Orders per DAU per day	0.25	Industry intuition	Low	Yes: order DB and payments	Measure in beta
A3	Busiest hour share of daily orders	20%	Assumed dinner peak	Medium	Yes: peak sizing	Hourly order histogram
A4	Courier location update interval	4 s	Product decision	High	Yes: location write load	Config value, reviewed with product
A5	Payment provider p99 latency	1 s	Vendor documentation	Medium	Moderate: timeouts, pools	Client-side latency histogram
A6	Average menu size	100 items × 1 KB	Sample of 50 menus	Medium	No: menus fit in memory even at 10×	Catalog size metric

The sensitivity column is the payoff. A6 is not load-bearing: all menus total about 5 GB (§2.11), and 50 GB would still fit in a small cache cluster, so being 10× wrong changes nothing structural. A4 *is* load-bearing: halving the interval to 2 s doubles the heaviest write path in the system.

WORKED EXAMPLE

Worked example — Sensitivity of the location tier to assumption A4. Peak active couriers are assumed to be 20,000.

1. Update interval 4 s: 20,000 / 4 = **5,000 writes/s**.
2. Product asks for 1 s “for smoother maps”: 20,000 / 1 = **20,000 writes/s**, 4× the load.
3. At an assumed 200 bytes per update including protocol overhead, ingress rises from 5,000 × 200 = 1 MB/s to 20,000 × 200 = 4 MB/s. That is modest for the network, but 4× the write operations for the store.

Interpretation. Network bandwidth is not the issue. The *write rate* to the location store is. **Engineering implication:** keep the 4 s interval on the server and have the client app *interpolate* the marker’s movement between updates. Customers see smooth motion at a quarter of the cost. The assumption register made this negotiation possible because the cost of the request was visible immediately.

INDUSTRY EXAMPLE

Assumptions in published design-review practice. Google's internal design-doc culture, as described by a Google engineer, asks authors to state context, goals and *non-goals*, and to discuss alternatives and trade-offs, so that reviewers can challenge the premises behind a design and not only its mechanics. *Evidence: Secondary* — [Ubl 2020]

Failure scenarios and misconceptions

Table 2.11 — Assumption failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Stale assumption	The world changed, e.g. a new market launched	Metrics drift past the tripwire	Run the redesign review	Tripwires on load-bearing assumptions
Assumption mistaken for fact	Repeated until nobody remembers it was a guess	Nobody can name its source	Re-label and re-check	"Source" and "confidence" columns
Average-case assumption	Peak and skew ignored	Outages at peaks or on hot keys	Add peak and skew factors	Always state a peak factor

COMMON MISTAKE

"Stating assumptions shows you don't know the answer." The opposite is true. In interviews and design reviews, explicit assumptions signal seniority. They show you know *which* unknowns matter and have a plan for each one.

2.8 Evaluating trade-offs

2.8.1 Weighted decision matrices

DEFINITION

Definition — Trade-off. A situation where improving one desired property of a system makes another worse. Examples: lower latency at higher cost, stronger consistency at lower availability during network partitions, faster delivery at higher operational risk. Every design decision is a trade-off. If you cannot name what an option makes worse, you have not understood it yet.

First-principles explanation

Buying a car means trading price against comfort, fuel economy against power, cargo space against parking ease. No car wins on everything. A sensible buyer decides *which properties matter most to them*, scores the candidates, and also notices how hard the choice is to undo: leasing for a year is easy to reverse, buying a car and selling it a month later loses money. System design decisions have exactly the same two dimensions, **weighted fit** and **reversibility**.

Why does this exist?

Without a method, design decisions are made by whoever argues loudest or by habit ("we always use X"). A structured method makes the criteria and their importance explicit. It lets people disagree about *weights* (a business question) separately from *scores* (an engineering question), and it leaves a record for the ADR (§2.12).

How it works internally: the weighted decision matrix

1. List the **options**, including "do nothing" or "the simplest thing" where it makes sense.
2. List the **criteria** that come from the requirements (§2.5), not from personal taste.
3. Assign **weights** that sum to 1 (or 100%), agreed with stakeholders.
4. **Score** each option against each criterion on a fixed scale, e.g. 1–5, with a one-line justification per score.
5. Compute weighted totals.
6. Run a **sensitivity check**: does the winner change if a weight moves by ±10 percentage points? If it does, the decision is close and the deciding factor should be discussed explicitly.
7. Check **reversibility** before committing (below).

This is a lightweight form of the concept-selection matrices used in engineering design, such as Pugh's method [Pugh 1991].

FORMULA

Weighted decision score

$$S_j = \sum_{i=1}^{n} w_i \times s_{ij}, \qquad \sum_{i=1}^{n} w_i = 1$$

Symbol	Meaning	Units
S_j	Total weighted score of option j	score points (same scale as s_{ij})
w_i	Weight of criterion i	fraction (0–1)
s_{ij}	Score of option j on criterion i	score points, e.g. 1–5
n	Number of criteria	count

WORKED EXAMPLE

Worked example — Modular monolith or microservices for QuickBite v1? The team is eight engineers and must launch in four months. Criteria and weights come from Table 2.5 and from business constraints.

Table 2.12 — Weighted decision matrix for QuickBite's deployment architecture.

Criterion	Weight	Modular monolith	Microservices (8 services)
Time to launch with 8 engineers	0.30	5	2
Operational simplicity	0.20	5	2
Independent scaling of hot paths	0.20	3	5
Failure isolation	0.15	3	4
Team autonomy as headcount grows	0.15	3	5
Weighted total	**1.00**	**3.90**	**3.35**

Arithmetic for the monolith: 0.30×5 + 0.20×5 + 0.20×3 + 0.15×3 + 0.15×3 = 1.50 + 1.00 + 0.60 + 0.45 + 0.45 = **3.90**. For microservices: 0.30×2 + 0.20×2 + 0.20×5 + 0.15×4 + 0.15×5 = 0.60 + 0.40 + 1.00 + 0.60 + 0.75 = **3.35**.

Sensitivity. Move 10 points of weight from time-to-launch to independent scaling (0.20 and 0.30). Monolith: 0.20×5 + 0.20×5 + 0.30×3 + 0.45 + 0.45 = 1.00 + 1.00 + 0.90 + 0.90 = **3.80**. Microservices: 0.20×2 + 0.20×2 + 0.30×5 + 0.60 + 0.75 = 0.40 + 0.40 + 1.50 + 1.35 = **3.65**. The monolith still wins, so the result is robust to a reasonable disagreement about weights.

A hybrid surfaces. The one criterion where the monolith loses badly is independent scaling, and §2.11 shows that the location path dominates write load. Deploying the **location service separately** and everything else as a modular monolith captures most of the scaling benefit at a fraction of the operational cost.

Interpretation. The matrix did not "decide". It made the reasoning inspectable and revealed a better third option. **Engineering implication:** record the matrix and the sensitivity result in the ADR, and name the trigger for revisiting it, for example "more than 30 engineers or more than 3 teams blocked on shared releases."

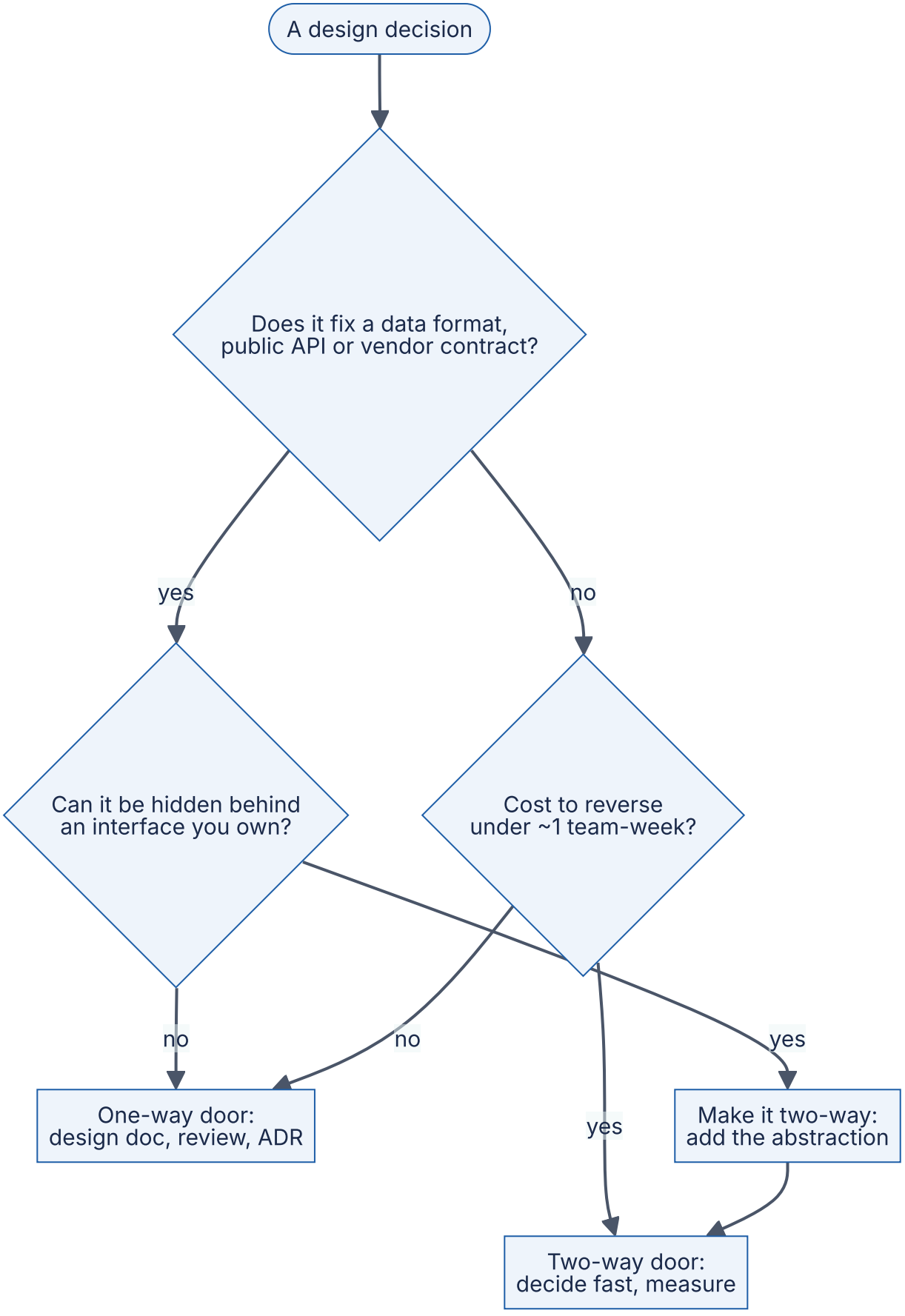
2.8.2 Reversibility: one-way and two-way doors

The matrix measures *fit*. It says nothing about the *cost of being wrong*. In his 2015 letter to shareholders, Jeff Bezos distinguished **Type 1** decisions, which are consequential and irreversible or nearly so ("one-way doors") and deserve slow, careful deliberation, from

Type 2 decisions, which are reversible (“two-way doors”) and should be made quickly by individuals or small groups with good judgment. He warned that large organizations tend to apply the heavyweight Type 1 process to Type 2 decisions, and that this causes slowness and too little experimentation [Bezos 2016].

In architecture, reversibility is mostly about **data and contracts**. Code behind a stable interface is cheap to change. Data formats that have accumulated years of records, and public APIs that thousands of clients depend on, are not.

Figure 2.5 — Choosing the decision process by reversibility.


A design decision
Does it fix a data format, public API or vendor contract?
yes
no
Can it be hidden behind an interface you own?
Cost to reverse under ~1 team-week?
no
no
yes
yes
One-way door: design doc, review, ADR
Make it two-way: add the abstraction
Two-way door: decide fast, measure

Reading the diagram. A decision (stadium) passes through up to three questions (diamonds). The first asks whether the decision fixes something that is costly to change later: stored data formats, public APIs, multi-year vendor contracts. If it does, the second asks whether an interface you own can hide it. A repository layer in front of the database, or an internal payment service in front of the payment provider, turns a one-way door into a two-way one (box T2). That is often the best move available. Decisions that fix nothing durable, and are cheap to reverse, are two-way doors: decide quickly and let measurement correct you. Everything else is a one-way door and earns the full process: a design document, a review and an ADR (§2.12). The "~1 team-week" threshold is a heuristic. Calibrate it to your organization. There is no software data flow here. The failure mode this procedure prevents is organizational: spending weeks debating a two-way door, or rushing a one-way door.

Table 2.13 — QuickBite decisions classified by reversibility.

Decision	Door	Why
Web framework for the catalog service	Two-way	Internal; can be rewritten behind the same API
Cache eviction policy	Two-way	A configuration change
Primary key and ID format for orders	One-way	Embedded in receipts, URLs, partner systems, years of data
Public order-status API shape for restaurant POS partners	One-way	External clients depend on it; versioning needed (§8.15)
Payment provider	One-way unless abstracted	Stored payment tokens are provider-specific
Monolith vs separate location service	Mostly two-way	Modular boundaries make extraction later feasible

INDUSTRY EXAMPLE

Amazon's Type 1 / Type 2 decisions. The two-door model comes from Amazon's 2015 shareholder letter. Bezos presented it as a way for a growing company to keep moving fast: reserve heavyweight deliberation for irreversible choices and push reversible ones down to small groups. *Evidence: Primary* — [Bezos 2016]

Alternatives, failure scenarios and misconceptions

Alternatives to a weighted matrix include simple **pros/cons lists** (fine for two-way doors), **Pugh matrices** (score relative to a baseline option, [Pugh 1991]) and formal methods such as the **Architecture Tradeoff Analysis Method (ATAM)** for large systems, which ties quality-attribute scenarios to architectural decisions [Bass et al. 2021].

Table 2.14 — Trade-off evaluation failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Matrix theater	Scores reverse-engineered to justify a choice already made	Justifications are vague	Re-score independently	Have two people score blind
Missing option	The best option was never listed	A reviewer proposes it late	Add and re-score	Always include "simplest thing" and a hybrid
Analysis paralysis on a two-way door	One process for every decision	Weeks of debate on reversible items	Time-box; pick and measure	Classify reversibility first
Rushed one-way door	Treated like a quick choice	Expensive migration later	Migration project	Figure 2.5 before deciding

COMMON MISTAKE

"There is a best architecture." There is only a best architecture *for a set of weighted requirements and constraints*. Change the team size, the deadline or the load, and the winner changes. This is why the requirements and assumptions behind a decision must be recorded alongside it.

2.9 Designing for failure

2.9.1 Failure as a normal event

DEFINITION

Definition — Designing for failure. Treating component failure as a *normal, expected event* rather than an exception. For every component and dependency, the design states what happens when it is slow, returns errors or disappears, and makes sure the **blast radius** (the set of users and features affected) is as small as possible.

First-principles explanation

Aircraft are not designed on the assumption that engines never fail. They are designed to fly on one engine. Buildings have fire doors that confine a fire to one section. Designing for failure applies the same idea to software: accept that every part *will* fail at some point, and decide in advance how the rest of the system keeps working.

Why does this exist?

At scale, failure is a matter of arithmetic, not bad luck. With 1,000 servers that each fail on average once every three years, the fleet sees roughly one failure a day ($1{,}000 / (3 \times 365) \approx 0.9$). Networks drop packets, disks wear out, deploys ship bugs and third parties

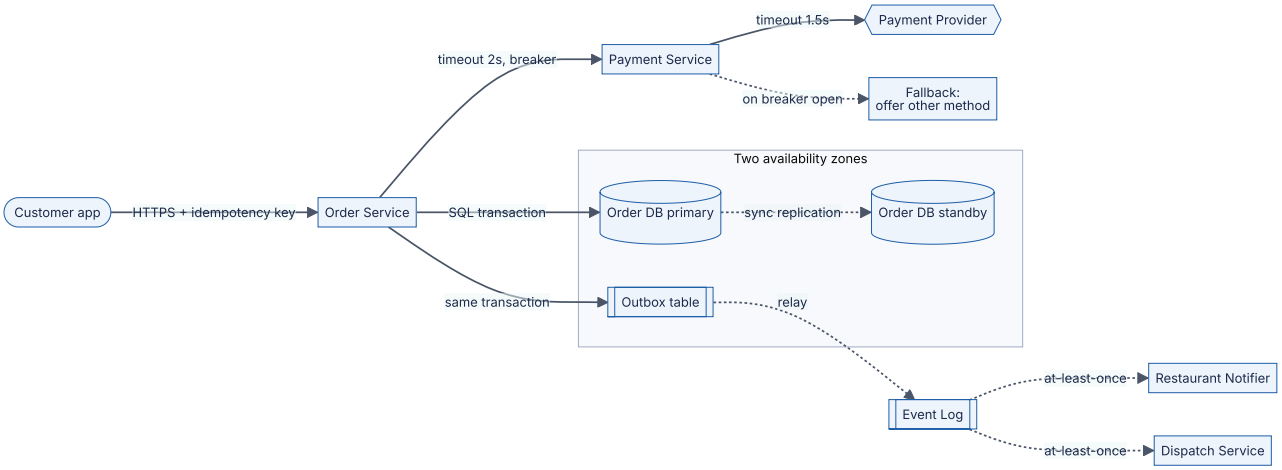
have outages. James Hamilton's widely cited guidance for internet-scale services starts from this premise: expect failures, and design every component so that failures are handled automatically and without drama [Hamilton 2007]. The "fallacies of distributed computing," such as "the network is reliable," catalog the assumptions that make systems brittle (§24.5).

How it works internally

A failure-mode walk-through, sometimes called a lightweight **failure mode and effects analysis (FMEA)**:

1. For each component and each arrow in the decomposition diagram, ask: *what if it is down? Slow? Returning wrong data?*
2. Decide the **desired behavior**: fail fast, retry, degrade gracefully or queue for later.
3. Apply standard mechanisms: **timeouts** on every remote call, **retries with backoff and jitter** only for idempotent operations, **circuit breakers** to stop calling a failing dependency, **bulkheads** to isolate resource pools (all in Chapter 22), **redundancy** across zones (§39.4) and **asynchrony** to take non-essential work off the critical path.
4. Remove things from the **critical path**, the chain of synchronous calls a user must wait for. Each item removed from it raises availability (formula below).

Figure 2.6 — QuickBite checkout designed for failure.



Reading the diagram. The customer's request carries an **idempotency key**, so a retry after a network error cannot create a second order or a second charge (requirement N4). The synchronous critical path is customer → order service → payment service → payment provider. Each hop has an explicit **timeout**, and the payment call sits behind a **circuit breaker**. If the provider is failing, the breaker opens and the customer is offered another payment method within milliseconds, instead of waiting for timeouts (dotted fallback). The order database runs inside a subgraph spanning two **availability zones**: a primary with a synchronously replicated standby, which meets N3 (RPO = 0). Instead of calling

the restaurant notifier and dispatch directly, the order service writes an event to an **outbox table** in the same transaction as the order (§32.5). A relay then publishes it to the event log. Restaurant notification and dispatch are therefore *off the critical path*: if either is down, orders still succeed, and the events are delivered when they recover (at-least-once, so consumers must be idempotent, §32.10). Remaining single points of failure to watch: the payment provider (mitigated by the fallback, or by a second provider later) and the event-log relay (mitigated by running two relays).

FORMULA

Availability of components in series

$$A_{series} = \prod_{i=1}^{n} A_i$$

Symbol	Meaning	Units
A_{series}	Availability of a request path that needs *all* n components	fraction (0–1)
A_i	Availability of component i	fraction (0–1)
n	Number of components on the synchronous path	count

This assumes independent failures. Parallel (redundant) components and correlated failures are treated in §4.14.

WORKED EXAMPLE

Worked example — Taking the restaurant notifier off the critical path. Assumed monthly availabilities: order service 99.95%, order DB 99.95%, payment provider 99.9%, restaurant notifier 99.9%.

1. Notifier called synchronously during checkout: 0.9995 × 0.9995 × 0.999 × 0.999 = 0.9990 × 0.9980 ≈ **0.9970**, i.e. 99.70%. That allows 0.003 × 43,200 ≈ **130 minutes** of checkout failure per month.
2. Notifier asynchronous via the outbox: 0.9995 × 0.9995 × 0.999 ≈ 0.9990 × 0.999 ≈ **0.9980**, i.e. 99.80%, about **86 minutes** per month.
3. Requirement N1 is 99.95%, or 21.6 minutes. Even the improved path misses it, because the payment provider alone allows 43 minutes.

Interpretation. A series chain can never be more available than its weakest link, and every extra synchronous dependency lowers the total. **Engineering implication:** either renegotiate N1 to exclude provider outages (a common SLO definition), or add a second payment provider in parallel. With two independent providers at 99.9% each, the pair is unavailable only when both fail: 1 − (0.001 × 0.001) = 99.9999%. This is a one-way-door-sized decision for a start-up (contracts, token migration), so it goes through Figure 2.5.

INDUSTRY EXAMPLE

Netflix's Chaos Monkey. Netflix publicly described the "Simian Army," starting with Chaos Monkey, a tool that randomly terminates production instances during business hours. Because engineers know instances will disappear, they must build services that tolerate it, and weaknesses are found while engineers are at work rather than at 3 a.m. This is designing for failure enforced by continuous practice (§39.11). *Evidence: Primary* — [Izrailevsky and Tseitlin 2011]

Table 2.15 — Failure scenarios for QuickBite checkout.

What goes wrong	Why	Detection	Recovery	Prevention
Payment provider outage	External dependency	Error rate on provider calls; breaker opens	Fallback method; queue payment capture	Second provider; breaker
Duplicate order on retry	Client retried after a timeout	Two orders with the same cart and customer	Refund the duplicate	Idempotency key (N4)
Order DB primary lost	Hardware or zone failure	Health checks, replication monitor	Automatic failover to standby	Synchronous standby in another zone
Event relay stalls	Bug or crash	Outbox backlog age metric	Restart; events retained in outbox	Two relays; alert on backlog age
Retry storm	Many clients retry at once	Load spikes as success falls	Shed load, open breakers	Exponential backoff with jitter, retry budgets

COMMON MISTAKE

"Redundancy alone makes a system reliable." Two replicas that share a power supply, a configuration push or a software bug fail together. Redundancy only helps against *independent* failures, so look for correlated ones: same zone, same deploy, same dependency.

2.10 Reasoning about scale

2.10.1 Orders of magnitude

DEFINITION

Definition — Order of magnitude. A factor of ten. Two quantities differ by k orders of magnitude when their ratio is about 10^k. Reasoning in orders of magnitude means estimating only the *power of ten* of a quantity (is it 10, 1,000 or 1,000,000 requests per second?) because that, not the exact value, decides the architecture.

First-principles explanation

Cooking dinner for 4 people, for 40 and for 4,000 are not the same task done more times. For 4 you use your home kitchen. For 40 you need a rented hall, borrowed pans and helpers. For 4,000 you need an industrial kitchen, refrigerated trucks, food-safety inspections and shift managers. Each 10× step changes the *kind* of solution, not just its size. Software behaves the same way, and this is why experienced designers first ask "what order of magnitude?"

Why does this exist?

Most design choices are insensitive to a factor of 2 but sensitive to a factor of 10. A single PostgreSQL primary comfortably handles QuickBite's 60 orders per second. At 600 it needs careful tuning and read replicas. At 6,000 it likely needs partitioning (Chapter 14). Knowing the order of magnitude tells you which of these architectures you are in, and how far you are from the next one.

How it works internally

1. Estimate the key rates and sizes to within a factor of about 3 (§2.11).
2. Compare them with reference capacities (Table 2.17 and §4.17) to find which tier is closest to a limit.
3. For each 10× step of growth, ask which components hit a *qualitatively* new problem.

Table 2.16 — What changes for QuickBite at each order of magnitude.

Scale	Orders/day	Peak orders/s	What breaks or changes	Typical response
1× (one city)	5,000	~0.6	Nothing; one server and one DB suffice	Modular monolith, managed DB with standby
10× (region)	50,000	~6	Browse reads dominate; menu queries load the DB	Cache menus, CDN for images (Chapters 20–21)
100× (country)	500,000	~60	Location writes (~5,000/s); hot restaurants; payment pool sizing	Separate location store; short transactions; async events
1000× (multi-country)	5,000,000	~600	Single order-DB write limits; data residency laws; multi-region latency	Partition orders by city or region; regional cells (§33.12, Chapter 44)

WORKED EXAMPLE

Worked example — Where does QuickBite's single order database run out? From §2.6 the WAL disk caps the order DB at about 500 orders/s, assuming 2 ms of fsync demand per order.

1. At 100× (today's design point) peak is 60 orders/s: utilization = 60 / 500 = 12%.
2. At 1000× peak is 600 orders/s: 600 / 500 = 120%, so the database saturates.
3. Group commit, which flushes many transactions in one fsync, might cut per-order disk demand 4× (an assumed factor), giving 2,000 orders/s capacity and 30% utilization at 1000×. That buys roughly one more order of magnitude.

Interpretation. The current design has about 8× headroom on its most constrained resource. **Engineering implication:** no partitioning work is needed now (that would be over-engineering), but partitioning by city should be *designed* before order volume reaches about 5× today's level, giving time to build it.

INDUSTRY EXAMPLE

"Design for 10× growth, plan to rewrite before 100×." Jeff Dean of Google has given talks on building large distributed systems in which he advises designing for about ten times the current scale while expecting a significant redesign before about a hundred times. The point is that one architecture rarely spans more than one or two orders of magnitude gracefully. *Evidence: Secondary* — talk slides; see Open verification items. [Dean 2009]

COMMON MISTAKE

"Design for Google scale from day one." Designing QuickBite v1 for 1000× its launch load would cost months of extra work, and the assumptions it relied on would likely be wrong by the time that scale arrived. Design for the next order of magnitude, keep the one after that *possible*, and use tripwires (§2.7) to know when to start.

2.11 Back-of-the-envelope capacity estimation

2.11.1 The estimation method

DEFINITION

Definition — Back-of-the-envelope estimation. A quick calculation, done with round numbers and stated assumptions, that gives the order of magnitude of a system's load and resource needs (requests per second, storage, bandwidth, memory, number of nodes). The aim is to be within a factor of about 2–3 of the truth with a few minutes of arithmetic. This section introduces the method. Chapter 89 is the full handbook.

First-principles explanation

Enrico Fermi famously estimated quantities such as the number of piano tuners in Chicago by multiplying a chain of reasonable guesses: population, households with pianos, tunings per year, tunings a tuner can do. The individual errors partly cancel out, and the answer lands near the right power of ten. Capacity estimation is the same technique applied to users, requests and bytes.

Why does this exist?

Estimation tells you *which architecture you are in* (§2.10) before you spend money. It also exposes surprises early. In QuickBite's case, the largest write load turns out to be courier GPS updates, not orders, a fact that reshapes the decomposition.

How it works internally

1. Write down the assumptions (they come from the register in §2.7).
2. Convert daily volumes to an **average rate**, then to a **peak rate**.
3. Multiply by **per-request sizes** to get bandwidth, and by **retention** and **copies** to get storage.
4. Divide peak load by **per-node capacity at a target utilization** to get node counts, then add spare capacity for failures.
5. Round sensibly, compare against reference capacities, and name the dominant cost or bottleneck.

FORMULA

Average and peak request rate

$$R_{avg} = \frac{U \times a}{86,400} \qquad R_{peak} = \frac{U \times a \times h}{3,600} \times b$$

Symbol	Meaning	Units
R_{avg}	Average request rate over a day	requests/s
R_{peak}	Design peak request rate	requests/s
U	Daily active users	users
a	Actions of this type per user per day	requests/user/day
h	Share of daily volume in the busiest hour	fraction
b	Burst factor above the busiest-hour average	dimensionless
86,400 / 3,600	Seconds per day / per hour	s

FORMULA

Storage and node count

$$S = n_{day} \times s \times T \times k \qquad N = \lceil \frac{R_{peak}}{c \times u} \rceil + N_{spare}$$

Symbol	Meaning	Units
S	Total storage needed	bytes
n_{day}	Records created per day	records/day
s	Size per record, including indexes and overhead	bytes
T	Retention period	days
k	Number of stored copies (replication)	dimensionless
N	Nodes needed	nodes
c	Throughput one node sustains	requests/s
u	Target utilization, leaving head-room for queueing (§4.10)	fraction
N_{spare}	Extra nodes so a failure or deploy does not overload the rest	nodes

Table 2.17 — Rough reference capacities for estimation (order of magnitude only, as of 2026).

Resource	Rough capacity	Note
Round trip within one datacenter	~0.1–0.5 ms	See §4.17 and Appendix F for dated values
Round trip across a continent	~50–100 ms	Bounded by the speed of light in fiber
Stateless API node, simple JSON requests	~10^3 RPS	Varies by 10× with language and work per request
Relational DB primary, simple indexed writes	~10^3–10^4 writes/s	Depends on fsync, schema, hardware
In-memory key-value store node	~10^5 ops/s	Small values, pipelining helps
1 Gbps network link	125 MB/s	1 Gbps ÷ 8 bits per byte

These numbers are deliberately rough. Measure your own system before relying on any of them.

WORKED EXAMPLE

Worked example — Full capacity estimate for QuickBite (the 100× design point). SI units throughout (1 KB = 10^3 bytes).

Assumptions (from Table 2.10, plus new ones marked "new"). U = 2,000,000 DAU. Orders a = 0.25 per DAU per day. Browse reads 30 per DAU per day (new). Busiest-hour share h = 0.20 for both. Burst factor b = 2 (new; covers promotions). 20,000 couriers active at peak, updating every 4 s. Order record 2 KB including items and status history (new). Average browse response 20 KB with images served separately by a CDN (new).

Step 1 — Orders. - Daily: 2,000,000 × 0.25 = **500,000 orders/day**. - Average: 500,000 / 86,400 ≈ **5.8 orders/s**. - Busiest hour: 500,000 × 0.20 = 100,000 orders in one hour; 100,000 / 3,600 ≈ 27.8 orders/s. - Design peak: 27.8 × 2 ≈ 55.6, rounded up to **60 orders/s**. This is the figure used in §2.6.

Step 2 — Browse reads. - Daily: 2,000,000 × 30 = 60,000,000 reads/day. Average: 60,000,000 / 86,400 ≈ **694 RPS**. - Busiest hour: 60,000,000 × 0.20 / 3,600 ≈ 3,333 RPS. Design peak: × 2 ≈ **7,000 RPS** (rounded up). - Read-to-order ratio at peak: 7,000 / 60 ≈ 117:1. Browsing is read-heavy and highly cacheable.

Step 3 — Courier locations and tracking. - Writes at peak: 20,000 / 4 s = **5,000 writes/s**. That is about 83× the order write rate, so this is the dominant write path. - Orders in the delivery leg at once, by Little's Law (§2.6): arrivals 27.8 orders/s × 900 s average delivery leg (15 min, new assumption) ≈ **25,000 orders**. - Tracking reads: if half of those customers have the app open and it refreshes every 5 s, 25,000 × 0.5 / 5 = **2,500 reads/s**.

Step 4 — Storage. - Orders: 500,000 × 2 KB = 1 GB/day. Over 5 years: 1 GB × 365 × 5 ≈ 1.8 TB. Add 50% for indexes: ≈ 2.7 TB. Two copies (primary + standby): **≈ 5.5 TB**. One database server can hold this. - Locations: assume 6,000 couriers active on average across the day, giving 6,000 / 4 = 1,500 writes/s. Per day: 1,500 × 86,400 = 129.6 million points. At 100 bytes each: ≈ 13 GB/day. Keep 30 days hot: 13 × 30 ≈ **390 GB**, then downsample to an archive. - Menus: 50,000 restaurants × 100 items × 1 KB = **5 GB**, small enough to cache entirely in memory (§20.3).

Step 5 — Bandwidth. - Browse egress at peak: 7,000 × 20 KB = 140 MB/s = 140 × 8 = **1.12 Gbps**, which needs more than one 1 Gbps link or a load balancer tier with 10 Gbps links. - Location ingress: 5,000 × 200 bytes = 1 MB/s = **8 Mbps**, negligible.

Step 6 — API nodes for browsing. With c = 1,000 RPS per node, u = 0.6 and N_spare = 2 (one zone or one deploy batch down): 7,000 / (1,000 × 0.6) = 11.7, rounded up to 12, plus 2 = **14 nodes**.

Step 7 — Interpretation.

Table 2.18 — QuickBite estimate summary (100× design point).

Quantity	Value	Dominant consequence
Peak orders	60/s	One relational primary is enough (§2.6: ~500/s limit)
Peak browse reads	7,000 RPS	Cache menus; ~14 API nodes
Peak location writes	5,000/s	Separate, in-memory, partitioned location store
Tracking reads	2,500/s	Serve from the location store, not the order DB
Order storage (5 years, 2 copies)	~5.5 TB	Single DB cluster fine; plan partitioning at ~5×
Location storage (30 days hot)	~390 GB	Time-partitioned; expire old data with a TTL
Peak egress (API, excluding images)	~1.1 Gbps	CDN for images; 10 Gbps load balancer links

Engineering implication. The estimate confirms the decomposition in Figure 2.2. It also justifies the one deviation from the monolith in §2.8: location must scale independently because it carries about 83× the write rate of orders. Nothing here needs partitioning yet, and that is as important to know as what does.

COMMON MISTAKE

"Estimation needs precise numbers." Round aggressively: 86,400 ≈ 10^5 seconds per day is fine for a first pass. What matters is stating assumptions, keeping units consistent and reaching the right power of ten. Spending ten minutes on the third significant digit while never computing peak load is the classic mistake.

2.12 Architecture documentation

2.12.1 Structure and rationale

DEFINITION

Definition — Architecture documentation. The written and diagrammed record of a system's structure and of the decisions that shaped it. It is written so that people who were not in the room (new team members, reviewers, your future self) can understand *what* the system is and *why* it is that way.

First-principles explanation

A map of a city at one scale cannot answer every question. You need a country map to see where the city is, a city map to see districts, and a street map to find a door. Architecture documentation works the same way: diagrams at several zoom levels for *structure*, and a logbook of decisions for *reasons*. Without the logbook, the next team sees a strange design choice and "fixes" it, reintroducing the problem it solved.

Why does this exist?

Knowledge in heads leaves when people do. Design decisions also tend to be revisited endlessly when their reasons are not written down. Three lightweight artifacts cover most needs:

Table 2.19 — Three architecture documentation artifacts.

Artifact	Answers	Written when	Lifetime
C4 diagrams	What are the parts and how do they connect, at each zoom level?	Before building; updated as the system changes	Living
Design document	What problem, what design, what alternatives, what risks?	Before a significant piece of work	Snapshot, then archived
Architecture Decision Record (ADR)	What did we decide, in what context, with what consequences?	At each significant decision	Immutable; superseded, never edited

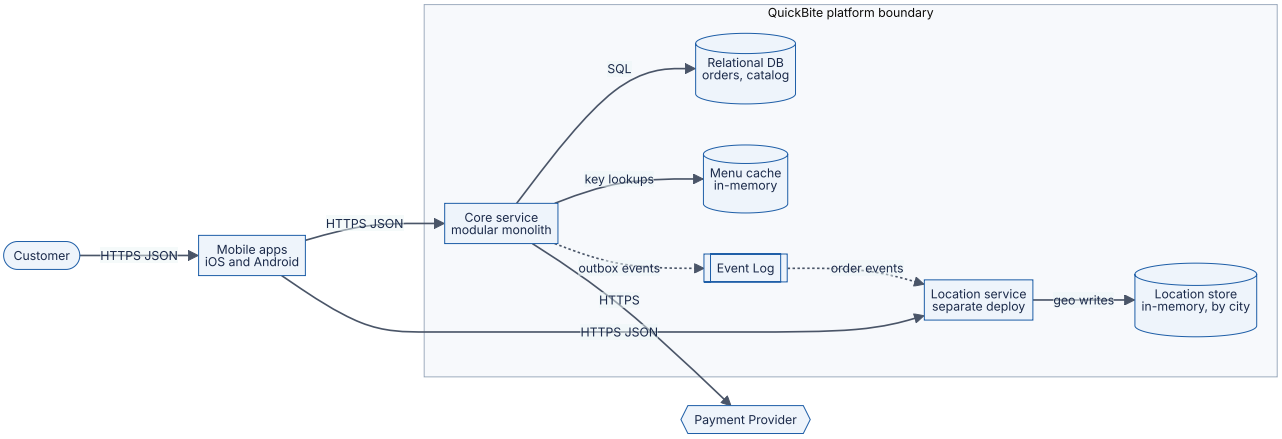
2.12.2 The C4 model

The **C4 model**, created by Simon Brown, describes a software system at four zoom levels [Brown C4]:

1. **Context.** The system as one box, with its users and external systems (Figure 2.3).
2. **Containers.** The separately deployable or runnable units (applications, databases, queues) and the technologies and protocols between them. In C4, "container" means a runtime unit, not specifically a Docker container.
3. **Components.** The major building blocks inside one container.
4. **Code.** Classes and functions. This level is usually generated from code or skipped.

This book draws C4 views with standard flowchart diagrams rather than an experimental C4 diagram syntax (see the style guide).

Figure 2.7 — QuickBite C4 container view, after the §2.8 decision.



Reading the diagram. This is C4 level 2 for QuickBite after the trade-off decision in §2.8. The **mobile apps** are containers too, because they are separately deployed runtime units. The subgraph is the platform's deployment boundary, matching the single box in Figure 2.3. The **core service** is the modular monolith: catalog, orders, payments, dispatch and notifications run in one process with internal module boundaries from Table 2.1. It talks SQL to the relational database (storage boundary for orders and catalog) and does key lookups against an in-memory **menu cache**, which keeps 7,000 browse RPS off the database (§2.11). The **location service** is deployed separately because of its 5,000 writes/s. It stores positions in an in-memory store partitioned by city and learns which courier is on which order from asynchronous order events (dotted). Synchronous paths (solid) are user-facing. Asynchronous paths (dotted) tolerate delay. Failure points match §2.9: the payment provider (external), the relational DB (standby in a second zone) and the event log. Scaling options: add core-service nodes horizontally, add location-store partitions per city, and later extract further modules into services if the tripwires from §2.7 fire.

2.12.3 Design documents

A **design document** (also called an RFC or a tech spec) is written *before* significant work, so that reviewers can find problems while changes are still cheap. Google engineers describe a widely copied structure: context and scope, goals and non-goals, the design itself, alternatives considered, and cross-cutting concerns such as security, privacy and observability [Ubl 2020]. A minimal skeleton:

```
Title, author(s), reviewers, status, date
1. Context and scope        - what exists today, what problem we solve
2. Goals and non-goals      - testable requirements (Table 2.5 style)
3. Assumptions              - register with confidence and tripwires (Table 2.10)
4. Estimates                - capacity numbers (Table 2.18)
5. Design                   - C4 context and container diagrams; APIs; data model
6. Alternatives considered  - decision matrix, reversibility (Table 2.12, Figure 2.5)
7. Failure modes            - FMEA table (Table 2.15)
8. Cross-cutting concerns   - security, privacy, observability, cost
9. Rollout and migration    - phases, flags, rollback plan
10. Open questions
```

What to notice:

- Sections 2–7 map directly onto the thinking tools of this chapter. A design document is the written output of the thinking loop in Figure 2.1.
- “Non-goals” and “Alternatives considered” are the sections reviewers value most, because they show what was rejected and why.

2.12.4 Architecture Decision Records

An **Architecture Decision Record (ADR)** is a short text file recording *one* significant decision. Michael Nygard’s original format has five parts: title, status, context, decision and consequences [Nygard 2011]. ADRs are numbered, stored in the code repository next to the code they affect, and never edited after acceptance. A changed decision gets a *new* ADR that **supersedes** the old one, so the history of reasoning is preserved.

```
ADR-007: Deploy the location service separately from the core monolith

Status: Accepted (2026-10-30). Supersedes: none.

Context: Peak location writes are estimated at 5,000/s, about 83x peak order
writes (design doc section 4). The core service is a modular monolith chosen
by weighted matrix (score 3.90 vs 3.35 for microservices; robust to +/-10 pt
weight changes). Location is the only module whose load profile differs by
more than 10x from the rest.

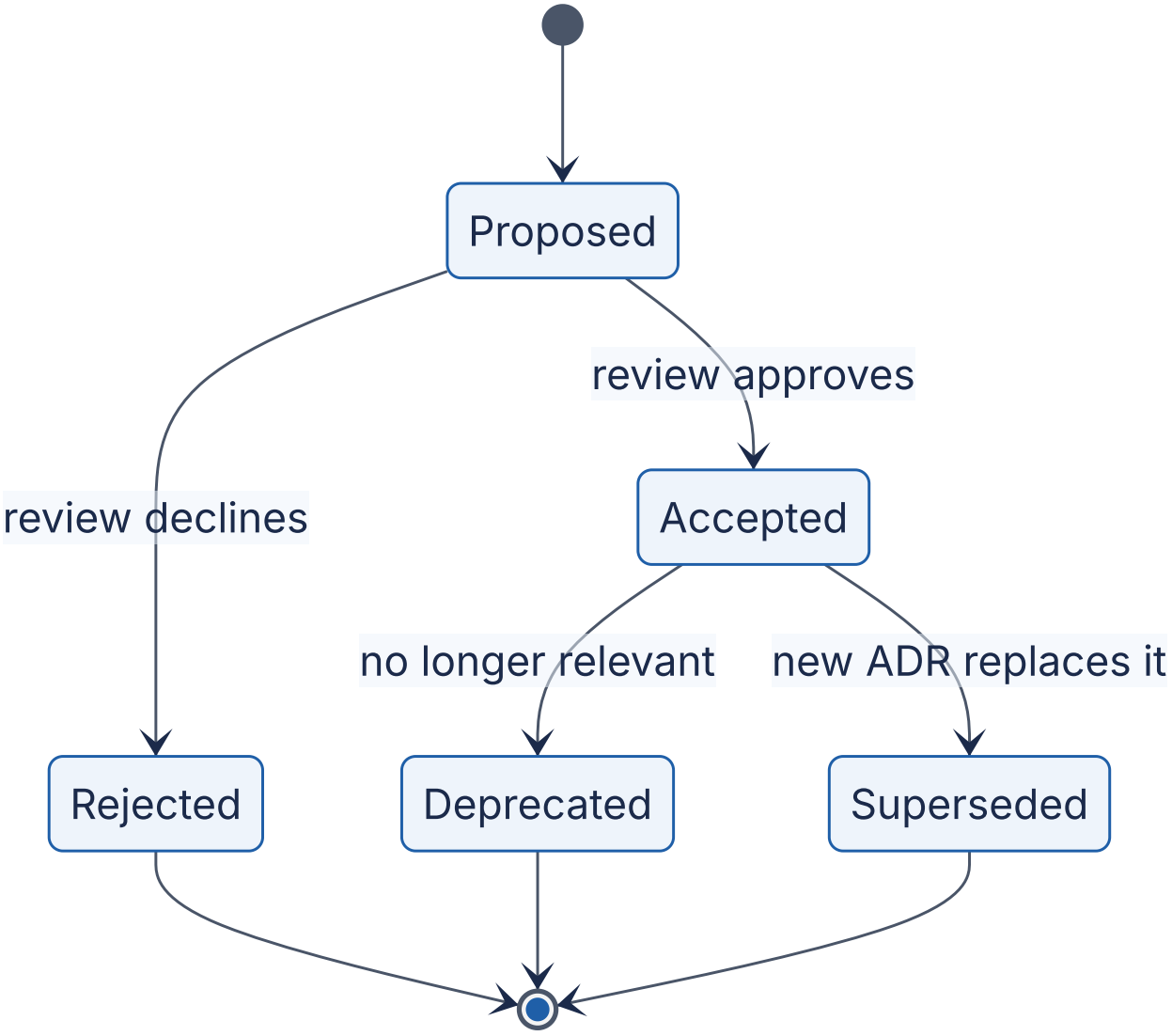
Decision: Run the location module as its own deployable service with its own
in-memory, city-partitioned store. All other modules remain in the monolith.

Consequences: (+) location scales independently; a location outage cannot
block checkout. (-) one more deployment pipeline and on-call surface; courier
assignment data reaches it asynchronously, with seconds of lag.
Revisit if: engineering headcount exceeds 30, or another module's peak load
exceeds 1,000 writes/s.
```

What to notice:

- The **context** carries the numbers and assumptions. Without them, a future reader cannot tell whether the decision is still valid.
- **Consequences** list the downsides honestly. An ADR with only positives is a sales pitch, not a record.
- “Revisit if” turns the assumption tripwires of §2.7 into an explicit trigger.

Figure 2.8 — ADR life cycle.



Reading the diagram. Each state is the *status* field of an ADR, not a running software component. An ADR starts as **Proposed**, often in a pull request, so that review happens with the usual code-review tooling. Review moves it to **Accepted** or **Rejected**. Rejected ADRs are kept, because knowing an option was considered and declined saves future debate. An accepted ADR is never rewritten. When circumstances change it becomes **Superseded**, with a link to the new ADR, or **Deprecated** if the thing it governed no longer exists. All transitions are human, asynchronous actions. The failure mode is decay:

ADRs that are never updated to Superseded make the record misleading, so check ADR status during design reviews.

INDUSTRY EXAMPLE

ADRs and C4 in practice. Nygard's 2011 post proposed ADRs as a lightweight alternative to large architecture documents that nobody reads, and the format has since spread widely, with open-source templates and tooling. Simon Brown's C4 model is likewise published openly with notation guidance and is used to standardize diagrams across teams. *Evidence: Primary* — [Nygard 2011], [Brown C4]

Table 2.20 — Documentation failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Diagram drift	Diagrams not updated with the code	New joiners find components missing	Redraw from the running system	Diagrams in the repo; review with code changes
Lost rationale	Decisions made in chat or meetings	"Why is it like this?" has no answer	Write retroactive ADRs	ADR required for one-way doors
Unreadable tome	80-page document nobody reads	No review comments	Split into C4 views plus ADRs	Keep docs short; link out to details

COMMON MISTAKE

"Documentation slows us down." Writing an ADR takes about half an hour. Re-arguing an undocumented decision, or reversing it by accident, costs days. Document one-way doors always and two-way doors only when they are surprising.

2.13 Summary and revision notes

SUMMARY

- **Design is a loop, not a line** (Figure 2.1): requirements → boundary → assumptions → estimates → decomposition → bottlenecks → failure → trade-offs → documentation, repeated at higher resolution until the design is good enough *for the risk*.
- **Decompose** by business capability, data ownership, load profile and likely change (information hiding). One sentence of responsibility per component. One owner per data store.
- **Draw the boundary first.** External systems bring availability, latency, rate limits and costs you do not control. QuickBite's maps contract ruled out per-listing route calls before any code existed.
- **Turn adjectives into numbers.** "Never down" became 99.95% for checkout (21.6 min/month). "Real-time" became a 5 s refresh with at most 10 s staleness.
- **The bottleneck is the resource with the largest demand per unit of capacity.** Check CPU, memory, disk, network *and* software resources (locks, pools). Utilization law: $U = X \times D/c$. Little's Law: $L = \lambda W$. QuickBite's checkout was limited by a row lock held across a payment call, not by hardware.
- **State assumptions** with source, confidence, sensitivity and a tripwire. A load-bearing assumption is one where a 10× error changes the design.
- **Evaluate trade-offs** with a weighted matrix plus a sensitivity check. Then classify reversibility: one-way doors get deliberation and an ADR, two-way doors get speed.
- **Design for failure.** Timeouts, idempotent retries, circuit breakers, redundancy across zones, and moving work off the critical path. A series path's availability is the product of its parts.
- **Think in orders of magnitude.** Each 10× brings qualitatively new problems. Design for the next 10×, keep the following one possible.
- **Estimate on the back of an envelope.** Average → peak → bandwidth → storage → nodes. QuickBite's heaviest write path is courier location (5,000/s), about 83× its order writes.
- **Document** structure with C4 (context, containers, components, code), plans with design docs, and decisions with immutable, superseding ADRs.

2.14 Knowledge checkpoint

1. Name four criteria for choosing component boundaries.
2. What does a context diagram deliberately leave out, and why?
3. Convert "the site must be fast" into a testable requirement.

4. How many minutes of downtime per 30-day month does 99.95% allow?
5. Which five resource categories make up the resource model?
6. A database is at 20% CPU but requests time out waiting. What class of bottleneck is most likely?
7. Each request holds a pooled connection for 250 ms and 100 requests/s arrive. How many connections are busy on average?
8. What makes an assumption “load-bearing”?
9. Why run a sensitivity check on a weighted decision matrix?
10. Give one example each of a one-way door and a two-way door in system design.
11. Three components in series each have 99.9% availability. What is the path’s availability?
12. 1 million DAU each perform 50 reads per day, 15% of them in the busiest hour. What is the busiest-hour read rate?
13. What are the four levels of the C4 model?
14. Why is an accepted ADR never edited?

Answers

1. Business capability, data ownership, load profile or rate of change, consistency need, and likely future change (information hiding). Any four.
2. The system’s internal structure. It focuses attention on actors, external dependencies and scope, and stays readable for non-technical stakeholders.
3. For example: “p99 latency of the search endpoint below 300 ms, measured over 5-minute windows at up to 7,000 RPS.”
4. $(1 - 0.9995) \times 43{,}200 = 21.6$ minutes.
5. CPU, memory, disk, network, and locks or other software resources (pools, threads, file handles).
6. A software resource: a lock, a connection pool or a thread pool. Requests are waiting while hardware is idle.
7. $L = \lambda W = 100 \times 0.25 = 25$ connections.
8. If it were wrong by about 10×, the design would have to change.
9. To learn whether the winner depends on a contested weight. If small weight changes flip the result, the decision is close and the deciding factor must be discussed explicitly.
10. One-way: the order ID format, a public partner API, the stored data model. Two-way: a cache eviction policy, an internal framework behind a stable API.
11. $0.999^3 \approx 0.997$, i.e. 99.7%.
12. $1{,}000{,}000 \times 50 \times 0.15 / 3{,}600 \approx 2{,}083$ RPS (before any burst factor).
13. Context, Containers, Components, Code.
14. To preserve the history of reasoning. A changed decision gets a new ADR that supersedes the old one, so readers can see what was believed when.

2.15 Interview questions

Basic

Q1. You are asked to “design a food-delivery app.” What do you do in the first five minutes?

Answer. Clarify before designing. Name the users (customers, restaurants, couriers, support) and the top journeys. Agree the functional scope and say explicitly what is out of scope. Then quantify: daily active users, orders per day, peak factor, latency targets per journey, availability for checkout, and consistency needs (“no double charges”). State the assumptions you are making and confirm them with the interviewer. Finish with a quick estimate (orders/s, reads/s, location writes/s) so that the architecture you draw is sized to a target.

Q2. What is a bottleneck, and why does adding servers sometimes not help?

Answer. It is the resource with the largest demand per unit of capacity. It caps throughput for the whole path. Adding capacity anywhere else changes nothing, as with widening a highway before a one-lane bridge. Servers add CPU and memory, but if the limit is a database disk, a hot row lock or a connection pool, throughput stays the same. Find it with the USE method and the utilization law before scaling anything.

Intermediate

Q3. Checkout latency is high at peak but every server’s CPU is below 30%. Walk through your diagnosis.

Answer. Low utilization with high latency points to waiting on a software resource or an external dependency. Look at per-hop latencies in traces. Check pool wait times, lock waits in the database and the latency of external calls. Apply Little’s Law to pools: busy connections = arrival rate × hold time. A common cause is holding a transaction (and therefore a row lock and a connection) across a remote payment call. That gives long hold times, pool exhaustion and hot-row queues. The fix is to shorten the critical section: do the external call outside the transaction, use an idempotency key, and keep transactions to milliseconds. Re-measure afterwards, because the bottleneck moves.

Q4. How do you decide between a modular monolith and microservices for a new product?

Answer. Derive criteria from requirements and constraints: time to market, team size, operational maturity, independent scaling needs, failure isolation. Weight and score them, and run a sensitivity check. For a small team with a deadline, a modular monolith usually wins. Then look for components whose load profile differs by an order of magnitude or

more (QuickBite's location service) and extract only those. Note reversibility: well-kept module boundaries make later extraction a two-way door. Record the decision and its re-visit triggers in an ADR.

Advanced

Q5. Your checkout SLO is 99.95%, but the payment provider guarantees only 99.9%. What are your options?

Answer. In series, the path cannot beat its weakest link, so 99.95% is unreachable as stated. Options: (1) redefine the SLO to exclude declared third-party outages, which is common but must be agreed with the business; (2) add a second provider in parallel, giving roughly $1 - 0.001^2$ for independent failures, at the cost of contracts, token handling and reconciliation, a one-way door; (3) accept the order and defer payment capture (authorize later, queue retries) for trusted customers, trading fraud risk for availability; (4) graceful degradation, such as offering alternative payment methods when the breaker opens. Present the costs and reversibility of each and let the business choose.

Q6. QuickBite expands from one country to ten. What changes, and what would you do first?

Answer. That is roughly 10×, so expect qualitative change. Order writes reach ~600/s and saturate the single primary's WAL disk (§2.10). Data residency laws may require in-country storage. Cross-continent round trips of ~100 ms hurt latency. Payment providers differ by country. First, re-run the estimate and assumption register. Then partition by geography into regional cells, each with its own order database and location store, which addresses scale, residency, blast radius and latency together. Keep a thin global layer for identity and routing only. Treat the partition key (region or city) as a one-way door with a full design doc. Migrate incrementally, one country per cell, with rollback plans.

2.16 Practical exercises

1. **Context diagram.** Draw a system-context diagram for a ride-hailing app. List every external system with an assumed SLA, rate limit and latency, and write three out-of-scope statements.
2. **Requirements negotiation.** Rewrite these as testable requirements and propose a cheaper acceptable alternative for each: "instant search", "zero data loss", "global availability".
3. **Bottleneck arithmetic.** Edit the script in §2.6 so that 10% of orders go to the single most popular restaurant. At what total order rate does the hot row saturate with a 320 ms hold time? With 15 ms?
4. **Estimation.** Repeat the §2.11 estimate for 20 M DAU and a 2 s location update interval. Which tier changes order of magnitude first?

5. **ADR.** Write an ADR for QuickBite's choice of an idempotency-key scheme for payments, including consequences and revisit triggers.

2.17 Design challenges

1. **Grocery delivery variant.** QuickBite adds grocery delivery: orders of 40–100 items, with stock levels that change constantly and substitutions. Rework Table 2.1, the assumption register and the bottleneck analysis. Where does inventory consistency create new locks or hot rows, and how do you keep them short?
2. **Holiday surge.** On one public holiday per year, orders spike to 8× the normal dinner peak for three hours. Compare three strategies: over-provision permanently, autoscale, or queue orders with a "your order is being placed" experience. Use a weighted decision matrix and classify each strategy's reversibility. Estimate the cost difference, stating your price assumptions.

2.18 References

1. Bass, L., Clements, P., Kazman, R. *Software Architecture in Practice*, 4th ed. Addison-Wesley, 2021.
2. Beyer, B., Jones, C., Petoff, J., Murphy, N. R. (eds.). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016. https://sre.google/sre-book/table-of-contents/
3. Bezos, J. *2015 Letter to Shareholders*. Amazon.com, Inc., 2016 (published with the 2015 annual report).
4. Brown, S. *The C4 model for visualising software architecture*. c4model.com, accessed 2026. https://c4model.com
5. Conway, M. E. *How Do Committees Invent?* Datamation 14(4), 1968.
6. Dean, J. *Designs, Lessons and Advice from Building Large Distributed Systems*. Keynote, LADIS Workshop, 2009.
7. DeMarco, T. *Structured Analysis and System Specification*. Yourdon Press, 1979.
8. Denning, P. J., Buzen, J. P. *The Operational Analysis of Queueing Network Models*. ACM Computing Surveys 10(3), 1978.
9. Gray, J. *A Conversation with Werner Vogels: Learning from the Amazon Technology Platform*. ACM Queue 4(4), 2006.
10. Gregg, B. *The USE Method*. brendangregg.com. https://www.brendangregg.com/usemethod.html
11. Gregg, B. *Systems Performance: Enterprise and the Cloud*, 2nd ed. Addison-Wesley, 2020.
12. Hamilton, J. *On Designing and Deploying Internet-Scale Services*. Proceedings of the 21st Large Installation System Administration Conference (LISA '07), USENIX, 2007.

13. Izrailevsky, Y., Tseitlin, A. *The Netflix Simian Army*. Netflix Technology Blog, 2011.
14. Little, J. D. C. *A Proof for the Queuing Formula: $L = \lambda W$*. Operations Research 9(3), 1961.
15. Nygard, M. *Documenting Architecture Decisions*. Cognitect blog, 2011. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions
16. Parnas, D. L. *On the Criteria To Be Used in Decomposing Systems into Modules*. Communications of the ACM 15(12), 1972.
17. Pugh, S. *Total Design: Integrated Methods for Successful Product Engineering*. Addison-Wesley, 1991.
18. Ubl, M. *Design Docs at Google*. Industrial Empathy (blog), 2020. https://www.industrialempathy.com/posts/design-docs-at-google/

2.19 Open verification items

- **[Bezos 2016]** The exact wording and canonical URL (Amazon IR site or SEC exhibit) of the Type 1 / Type 2, “one-way / two-way door” passage. The chapter paraphrases it. sec.gov could not be reached from the authoring session.
- **[Dean 2009]** Confirm that the “design for ~10× growth, plan to rewrite before ~100×” advice appears in the LADIS 2009 keynote slides and not only in other Dean talks (e.g. Stanford, 2007). Hence the label is *Secondary*.
- **[Ubl 2020]** Confirm the publication year and URL. Only secondary summaries were checked.
- **[Gray 2006]** Confirm that “you build it, you run it” and the no-direct-database-access rule appear in this interview.
- **[DeMarco 1979]** The first edition is dated 1978 or 1979 depending on the source.
- **[Izrailevsky and Tseitlin 2011]** Confirm authors and date of the Netflix blog post.
- Table 2.17 capacities and the “99.9–99.99% third-party SLA” range are rough, as of 2026. Reconcile them with Appendix F and §4.17.
- The style guide asks for longer runnable programs in `code/ch02/`. The §2.6 script is short and was not copied there.

Chapter 3 — The System Design Learning Method

> **Level:** 1 · **Track:** Shared · **Estimated reading time:** 3.5 h

System design is a large field. This book covers it in 93 chapters, and the field keeps changing while you read. No one masters it by reading straight through once. Engineers who become strong designers usually have two things: a deep stock of well-understood building blocks, and a reliable way to add new blocks. This chapter gives you that way. It is a *method*, so it applies to every later chapter, from B-trees (Chapter 11) to LLM inference (Chapter 73).

3.1 Learning objectives

After this chapter you should be able to:

- Apply the ten-step concept-mastery loop to any system-design concept and produce the artifact each step requires.
- Explain, with evidence, why retrieval practice and spaced practice beat rereading and highlighting, and schedule reviews to match.
- Read a systems paper using Keshav's three-pass method and judge an engineering blog post's claims by the strength of its evidence.
- Draw a reference architecture from memory, score it against a checklist, and repair the gaps.
- Scope a simplified implementation that keeps the core mechanism and leaves out production concerns.
- Follow a 12-month beginner path through this book, or build a gap-driven plan if you are already experienced.
- Track your progress with a self-assessment rubric, flashcards and a design journal.

3.2 Prerequisites

- **What system design is** — §1.3–§1.5. You need to know what counts as a "concept" or "building block" before you can learn concepts methodically.
- **Trade-off reasoning** — §2.8. Step 6 of the loop asks you to analyze trade-offs. Chapter 2 gives you the vocabulary.

- **Basic Python** — enough to read a 60-line program with a class, a list and a dictionary. Sections 3.3 and 3.4 contain short runnable programs.
- **Hashing (informal)** — you should know that a hash function maps any input to a fixed-size number. Chapter 17 (§17.3) covers hashing in depth; you do not need it yet.

3.3 The ten-step concept-mastery loop

3.3.1 Why a loop rather than a reading list

> DEFINITION
>
> **Definition — Concept-mastery loop.** A fixed sequence of ten learning activities applied to one system-design concept. Each activity produces a concrete artifact (a definition, a diagram, a program, a design). The concept counts as mastered only when you can rebuild every artifact without notes.

First-principles explanation

Think about learning to cook one dish well. Reading the recipe tells you what the dish is. Knowing why each ingredient is there tells you what goes wrong if you leave one out. Watching a chef shows you technique. But you only really know the dish after you have cooked it yourself, burned it once, fixed it, and then cooked it for someone else without the recipe in front of you. The loop applies the same idea to a concept such as caching or consistent hashing.

Why does this exist?

Most engineers learn system design by *exposure*: they read blog posts, watch videos and skim chapters. Exposure feels productive because the material seems familiar on the second reading. Familiarity is not the same as the ability to *use* a concept. Learning researchers call this mismatch an illusion of competence. Rereading and highlighting are among the least effective study techniques measured [Dunlosky et al. 2013]. A design interview or a design review asks you to *produce*: choose a mechanism, draw it, estimate it and defend it. The loop makes production the main activity.

How it works internally

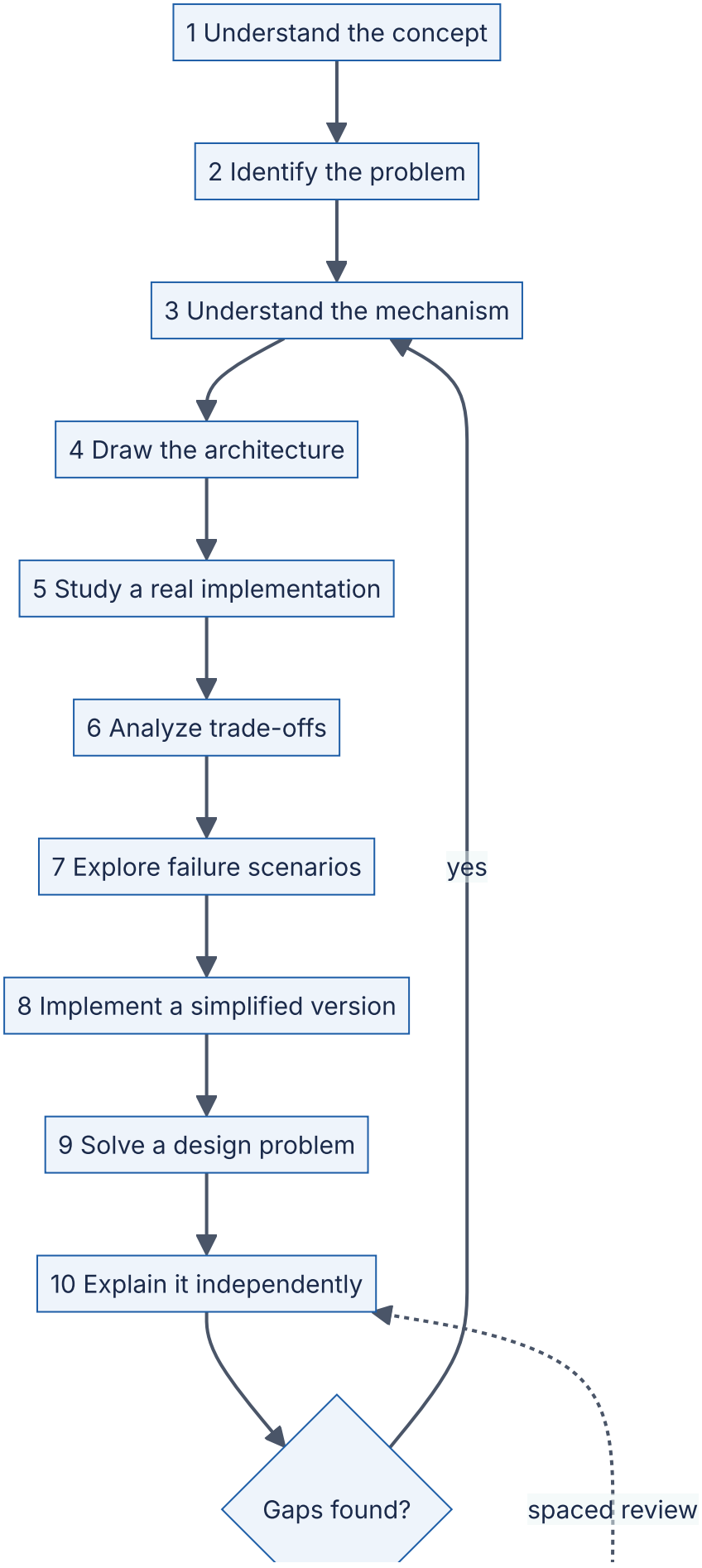
Table 3.1 — The ten steps, the question each answers, and the artifact each produces.

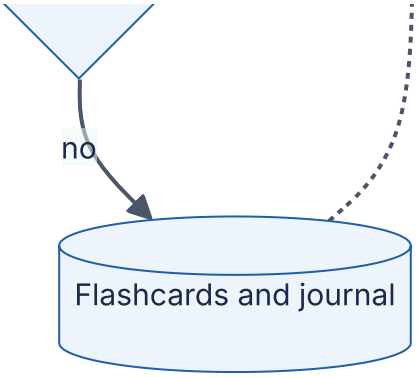
#	Step	Question it answers	Artifact	Typical time (L1–L2)
1	Understand the concept	What is it, in one precise sentence?	Definition in your own words	15 min
2	Identify the problem it solves	What breaks without it?	A “before” scenario with numbers	20 min
3	Understand the mechanism	How does it work, step by step?	Numbered mechanism description	45 min
4	Draw the architecture	Where does it sit and what talks to it?	Diagram drawn from memory	20 min
5	Study an actual implementation	How did a real system build it?	Notes on one paper, docs page or codebase	60–120 min
6	Analyze trade-offs	What does it cost, and versus what?	Comparison table against alternatives	30 min
7	Explore failure scenarios	How does it break?	Failure table (what, why, detect, recover, prevent)	30 min
8	Implement a simplified version	Can I build the core?	Runnable program of 50–300 lines with a measurement	2–4 h
9	Solve a design problem	Can I use it in context?	A small design that uses the concept	45–60 min
10	Explain it independently	Can I teach it without notes?	2–5 minute spoken or written explanation	15 min

A full pass takes about 6–10 hours for one substantial concept. Plan for that. A concept such as “HTTP status codes” does not need all ten steps. A concept such as consistent hashing, Raft or the KV cache does.

The steps are ordered so that each one gives the next one something to work with. You cannot weigh trade-offs (6) until you know the mechanism (3). You should not implement (8) before you know the failure modes (7), because the failures tell you what your simulation must be able to show. Step 10 then checks all the earlier steps. Wherever your explanation falters, you return to that step. That return is why it is a loop and not a line.

Figure 3.1 — The ten-step concept-mastery loop.


1 Understand the concept
2 Identify the problem
3 Understand the mechanism
4 Draw the architecture
5 Study a real implementation
6 Analyze trade-offs
7 Explore failure scenarios
8 Implement a simplified version
9 Solve a design problem
10 Explain it independently
Gaps found?
yes
spaced review



Reading the diagram. The ten rectangles are learning activities, done in order. The solid arrows mean “finish this before starting the next”. The decision node after step 10 is your self-test. If you cannot explain some part without notes, you go back to step 3 (mechanism), because most gaps turn out to be gaps in the mechanism. If you can, the concept goes into long-term storage: the cylinder represents your flashcard deck and design journal (§3.10). The dotted arrow is asynchronous. Days or weeks later, a scheduled review asks you to repeat step 10 from memory. The weak point in this process is skipping steps 7 and 8, which feel optional but are where most understanding is built. To scale the method across a whole book, run several concepts’ loops interleaved, not one after another (§3.4.5).

Common misconceptions

COMMON MISTAKE

Misconception — “I understood it when I read it, so I know it.” Understanding while reading is recognition: the text is in front of you and gives you the cues. Using a concept in a design is recall: you have to bring it up with no cues. Only steps 4, 8, 9 and 10 test recall. If you skip them, you will not know what you are unable to do.

3.3.2 The loop demonstrated: consistent hashing

The rest of §3.3 runs one concept through all ten steps. It shows the artifacts at the depth we expect from you. We use **consistent hashing** because it is compact enough for one section and still has a real mechanism, real failure modes and real implementations. Chapter 14 (§14.8) and Chapter 20 (§20.14) treat it in full. Treat what follows as a sample of how to study, not the canonical treatment.

Step 1 — Understand the concept

DEFINITION

Definition — Consistent hashing. A way of assigning keys to nodes so that, when a node is added or removed, only about K/N of the K keys change owner, where N is the number of nodes. Both keys and nodes are hashed onto the same circular number space (the **ring**). Each key belongs to the first node found by moving clockwise from the key's position [Karger et al. 1997].

Self-check artifact: one sentence in your own words, with no jargon you cannot define. For example: "It spreads keys over servers so that adding a server moves only that server's fair share of keys."

Step 2 — Identify the problem it solves

Suppose you run four cache nodes and route each key with `node = hash(key) mod 4`. Traffic grows, so you add a fifth node and the rule becomes `hash(key) mod 5`. For most keys, `hash mod 4` and `hash mod 5` point to different nodes. Those keys now live on the wrong node, so nearly every request misses the cache and falls through to the database at the very moment you were trying to add capacity.

FORMULA

Fraction of keys remapped when growing from N to $N+1$ nodes

$$f_{\text{mod}} = 1 - \frac{1}{N+1} = \frac{N}{N+1} \qquad f_{\text{ring}} \approx \frac{1}{N+1}$$

Symbol	Meaning	Units
N	Number of nodes before the change	nodes
f_{mod}	Expected fraction of keys whose owner changes under `hash mod N` (uniform hashing)	dimensionless (0–1)
f_{ring}	Expected fraction of keys whose owner changes under consistent hashing	dimensionless (0–1)

The f_{mod} result comes from a simple observation. A key stays on the same node only if `h mod N` equals `h mod (N+1)`. For uniformly distributed hashes, that happens for about $1/(N+1)$ of keys. Under consistent hashing, the new node takes over only the keys that fall in its arcs of the ring. On average that is its fair share, $1/(N+1)$.

WORKED EXAMPLE

Worked example — Adding a fifth cache node.

1. Setup: $N = 4$, $K = 100,000,000$ cached keys, cache hit ratio 95 %, and 50,000 reads per second (RPS) reach the cache tier.
2. Naive scheme: $f_{\text{mod}} = 4/5 = 0.80$. About $0.80 \times 10^8 = 8 \times 10^7$ keys are now on the wrong node.
3. Ring scheme: $f_{\text{ring}} = 1/5 = 0.20$. About 2×10^7 keys move. They move only *to the new node*, which starts out empty.
4. Database impact right after the change, before the cache refills. Under mod-N, about 80 % of reads miss, plus the normal 5 % on the rest. The database sees about $0.80 \times 50,000 + 0.05 \times 0.20 \times 50,000 = 40,000 + 500 = 40,500$ RPS instead of the usual $0.05 \times 50,000 = 2,500$ RPS. That is a 16× surge. Under the ring, about 20 % of reads miss, plus the normal 5 % on the remaining traffic. The database sees about $0.20 \times 50,000 + 0.05 \times 0.80 \times 50,000 = 10,000 + 2,000 = 12,000$ RPS, a 4.8× surge.

Interpretation. Both schemes cause a miss spike. Consistent hashing makes it about 3.4× smaller ($40,500/12,000$) and limits it to one node's share of keys.
Engineering implication. If your database can absorb about 12,000 RPS, consistent hashing lets you add cache nodes without an outage. With mod-N you would need to warm the new layout ahead of time or add nodes only during low traffic. Chapter 20 (§20.11, cache stampede) covers the remaining spike.

Self-check artifact: a "before" scenario with numbers, like the one above.

Step 3 — Understand the mechanism

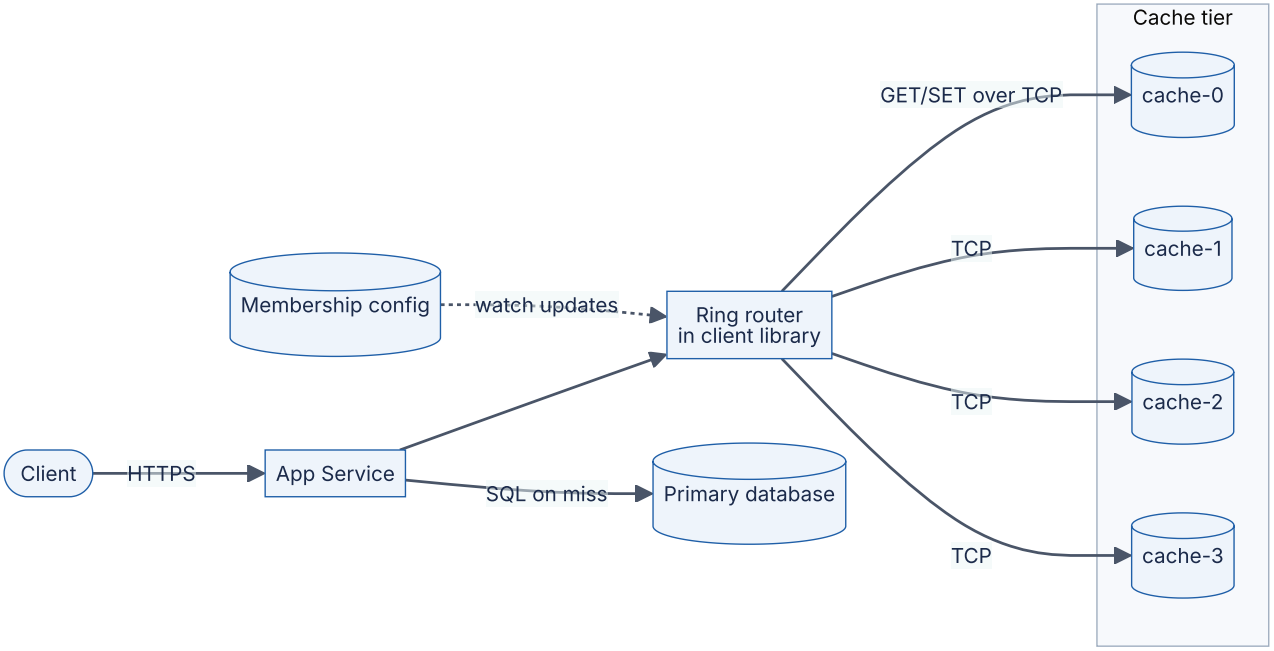
1. Pick a hash function that maps strings to integers in $[0, 2^m)$. Treat that range as a circle in which $2^m - 1$ is next to 0.
2. For each node, compute V positions by hashing distinct labels such as `"cache-2#vn17"`. Each position is a **virtual node** (vnode). Store all $N \times V$ positions in a sorted array, each mapped to its physical node.
3. To look up a key, hash it and binary-search the sorted array for the first position greater than the key's hash. If there is none, wrap around to index 0. The node that owns that position owns the key. A lookup costs $O(\log(NV))$.
4. To add a node, insert its V positions. Each one takes over the arc between its predecessor and itself. Every other key keeps its owner.
5. To remove a node, delete its positions. Each of its arcs goes to the next position clockwise. Because the vnodes are scattered around the ring, the removed node's load is spread over many survivors instead of all landing on one neighbor.
6. For replication, walk clockwise and collect the first R *distinct physical* nodes. Dynamo calls this list the **preference list** [DeCandia et al. 2007].

Why do virtual nodes matter? With $V = 1$, each node owns one arc, and arc lengths from N random points vary widely. One node can easily own two or three times its fair share. With V vnodes, each node's load is the sum of V arcs, and sums of many random lengths stay close to the average. The relative spread shrinks roughly in proportion to $1/\sqrt{V}$. Step 8 measures this.

Step 4 — Draw the architecture

Close the book and draw where consistent hashing sits in a distributed cache. Then compare your drawing with Figure 3.2.

Figure 3.2 — Consistent hashing in a client-routed distributed cache.



Reading the diagram. A client calls the app service over HTTPS. Inside the app service, a client library holds the ring: the sorted vnode positions and their owners. It is in-process, so a lookup is a binary search in memory, typically microseconds. The ring router sends each `GET` / `SET` to exactly one cache node over a TCP connection, such as the Memcached or Redis protocol. All of these are synchronous calls. On a miss, the app service reads the primary database over SQL and fills the cache (cache-aside, §20.7). The dotted arrow from the membership config is asynchronous. A coordination service or config file tells every app instance when nodes join or leave (§26.8–§26.9). There are two storage boundaries: the in-memory cache tier, which is disposable, and the database, which is durable. Failure points: a cache node that dies sends its keys to clockwise successors and causes a miss spike, and app instances that see *different* membership send the same key to different nodes. To scale, add cache nodes (each takes about $1/(N+1)$ of the keys) or raise V to even out the load.

Self-check artifact: your own drawing, annotated with which calls are synchronous and where state lives. If your drawing left out the membership channel, you have found your first gap. Section 3.6 explains how to score drawings like this.

Step 5 — Study an actual implementation

Now read how real systems built it. Pick *one* primary source and read it with the three-pass method (§3.5). For consistent hashing, three sources are worth knowing:

INDUSTRY EXAMPLE

Akamai and the original paper. Karger and colleagues introduced consistent hashing to spread web-cache load without hot spots, in a setting where different clients may see different sets of caches. Their paper defines the properties a good scheme needs. Two of them are *balance*, meaning keys spread evenly, and *monotonicity*, meaning that adding a cache moves keys only onto the new cache, never between old caches. The paper also uses multiple copies of each cache point on the circle to improve balance, which is the idea behind virtual nodes. *Evidence: Primary* — [Karger et al. 1997]

INDUSTRY EXAMPLE

Amazon Dynamo. Dynamo partitions its key space with consistent hashing and assigns each physical node several virtual nodes. This lets a more powerful machine take more vnodes, and it spreads a failed node's load over many survivors. Each key is replicated on the first R distinct physical nodes clockwise, its preference list (the paper calls this count N). The paper compares three strategies for placing tokens. It reports that dividing the ring into equal-sized partitions, decoupled from node placement, gave the best load balance and the smallest membership metadata. *Evidence: Primary* — [DeCandia et al. 2007]

INDUSTRY EXAMPLE

Apache Cassandra. Cassandra's original design hashes keys onto a ring with an order-preserving hash function and assigns each node one position. The paper acknowledges the load imbalance this causes. It handles the imbalance by analyzing load on the ring and moving lightly loaded nodes so they take load from heavily loaded ones. *Evidence: Primary* — [Lakshman and Malik 2010]

When you read an implementation, write down where it *departs* from the textbook mechanism and why. Departures are usually the most instructive part. Dynamo moved from random tokens to fixed equal-sized partitions because random tokens made bootstrapping

and archiving slow. That is an operational reason, not an algorithmic one, and only a real implementation would show it to you.

Step 6 — Analyze trade-offs

Table 3.2 — Key-to-node assignment schemes compared.

Dimension	`hash mod N`	Ring + virtual nodes	Rendezvous (HRW) hashing	Jump consistent hash
What it is	Modulo of key hash	Keys and vnodes on a circle	Each key picks the node with the highest score `hash(key, node)`	Arithmetic function from key to bucket number
Keys moved on add	~$N/(N+1)$	~$1/(N+1)$	~$1/(N+1)$	~$1/(N+1)$
Lookup cost	$O(1)$	$O(\log NV)$	$O(N)$ (scores every node)	$O(\log N)$, no table
Memory	None	$O(NV)$ positions	$O(N)$ node list	None
Load balance	Excellent	Good; improves with V	Excellent	Excellent
Arbitrary node removal	Remaps almost everything	Yes	Yes	No; buckets must be numbered $0..N-1$ and removed from the end
Weighted nodes	Awkward	Natural (more vnodes)	Possible with weighted scores	No
Choose when	N never changes	General-purpose caches and stores	Small N, simple code	Storage shards that only grow

Sources: [Karger et al. 1997], [Thaler and Ravishankar 1998], [Lamping and Veach 2014]. Two further variants are worth knowing by name. **Maglev hashing** builds a fixed-size lookup table for $O(1)$ lookups in load balancers and accepts a little extra disruption in exchange [Eisenbud et al. 2016]. **Consistent hashing with bounded loads** caps every node at $(1+\varepsilon)$ times the average load and sends overflow keys clockwise [Mirrokni et al. 2018].

Self-check artifact: a table like Table 3.2 written from memory, plus one sentence saying which scheme you would pick for a 6-node Redis cache and why.

Step 7 — Explore failure scenarios

Table 3.3 — Failure scenarios for consistent hashing.

What goes wrong	Why	Detection	Recovery	Prevention
One node overloaded	Too few vnodes, so arcs are uneven	Per-node RPS and memory skew	Add vnodes to the others; rebalance	$V \geq 100$; bounded-load variant
Hot key	One key gets a large share of traffic; hashing cannot split one key	Top-K key metrics (§17.6)	Replicate the hot key; add a local cache tier	Key splitting, request coalescing (§20.15)
Successor overload after node death	With $V = 1$, the clockwise neighbor inherits *all* of the dead node's keys	Neighbor CPU and miss-ratio spike	Bring up a replacement node	Virtual nodes spread the load over many successors
Split views of membership	Clients update membership at different times	Same key read from two nodes; inconsistent reads	Converge config; flush the affected range	Single source of truth with watches; versioned ring
Miss storm on resize	New node starts cold	Database RPS spike	Rate-limit fills; serve stale data	Pre-warm; add nodes off-peak; stampede protection (§20.11)
Poor hash function	Clustering of positions	Skewed arc lengths at build time	Change the hash, which remaps everything	Use a well-mixed hash (e.g., MurmurHash, xxHash, MD5 truncated)

The split-view row is the one beginners miss. The algorithm assumes every client has the same ring. Keeping that true is a distributed-systems problem in its own right, and you will meet it again in Chapter 26.

Step 8 — Implement a simplified version

Keep the mechanism and leave out everything else: no networking, no replication, no persistence. The goal is a program that lets you *see* the two properties you studied, minimal movement and load balance.

```
    """Simplified consistent-hashing ring with virtual nodes (teaching version)."""
    import bisect
    import hashlib
    from collections import Counter


    def h(key: str) -> int:
        """Map a string to a 64-bit integer position on the ring."""
        digest = hashlib.md5(key.encode("utf-8")).digest()
        return int.from_bytes(digest[:8], "big")


    class HashRing:
        def __init__(self, nodes=(), vnodes: int = 100):
            self.vnodes = vnodes
            self._positions = []   # sorted ring positions
            self._owner = {}       # position -> physical node name
            for node in nodes:
                self.add_node(node)

        def add_node(self, node: str) -> None:
            for i in range(self.vnodes):
                pos = h(f"{node}#vn{i}")
                bisect.insort(self._positions, pos)
                self._owner[pos] = node

        def remove_node(self, node: str) -> None:
            for i in range(self.vnodes):
                pos = h(f"{node}#vn{i}")
                self._positions.remove(pos)
                del self._owner[pos]

        def get_node(self, key: str) -> str:
            """Return the first node clockwise from the key's position."""
            if not self._positions:
                raise LookupError("ring is empty")
            idx = bisect.bisect_right(self._positions, h(key))
            if idx == len(self._positions):  # wrap around past the top
                idx = 0
            return self._owner[self._positions[idx]]


    def mod_n_node(key: str, nodes: list) -> str:
        """The naive scheme: hash(key) mod N."""
        return nodes[h(key) % len(nodes)]


    if __name__ == "__main__":
        keys = [f"user:{i}" for i in range(100_000)]
        before = [f"cache-{i}" for i in range(4)]
        after = before + ["cache-4"]

        moved = sum(mod_n_node(k, before) != mod_n_node(k, after) for k in keys)
        print(f"mod-N : {moved / len(keys):.1%} of keys moved (4 -> 5 nodes)")

        for v in (1, 100):
```

```
ring = HashRing(before, vnodes=v)
old = {k: ring.get_node(k) for k in keys}
ring.add_node("cache-4")
moved = sum(old[k] != ring.get_node(k) for k in keys)
load = Counter(ring.get_node(k) for k in keys)
spread = max(load.values()) / (len(keys) / len(after))
print(f"ring vnodes={v:<3}: {moved / len(keys):.1%} moved, "
      f"busiest node at {spread:.2f}x the mean load")
```

Running it with Python 3.8 or later prints:

```
mod-N : 79.9% of keys moved (4 -> 5 nodes)
ring vnodes=1  : 24.4% moved, busiest node at 2.60x the mean load
ring vnodes=100: 21.4% moved, busiest node at 1.08x the mean load
```

What to notice:

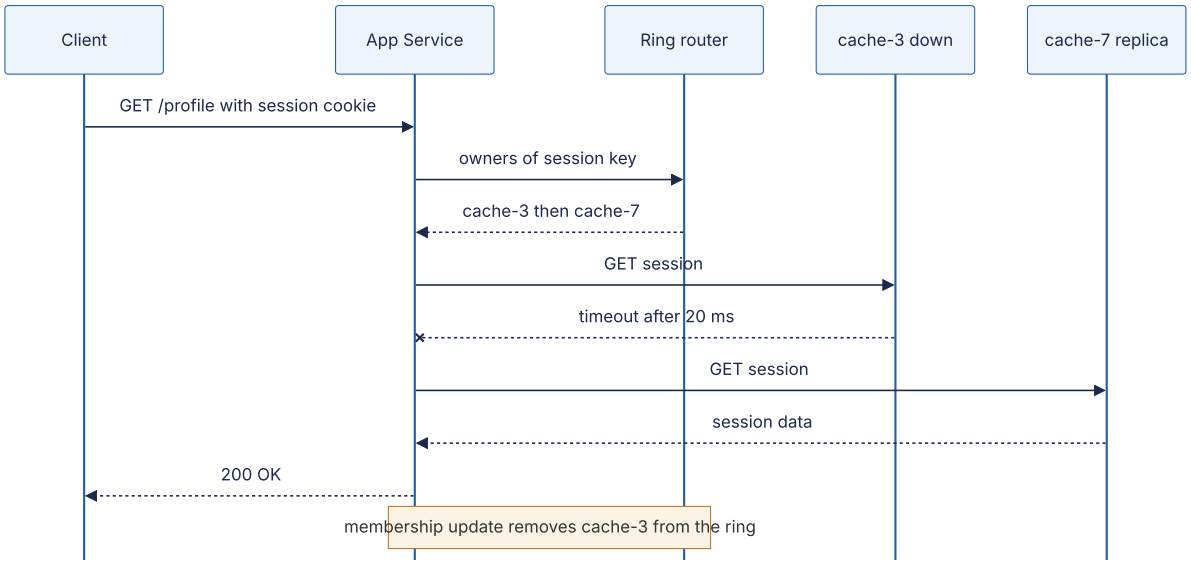
- `h()` uses MD5 only as a *mixing* function, which is fine here because nothing depends on it being cryptographically secure. Python's built-in `hash()` is randomized per process for strings, so two app instances would build different rings. That is exactly the split-view failure from Table 3.3, reproduced by accident.
- `bisect_right` plus the wrap-around check *is* the clockwise walk. Those two lines are the whole algorithm.
- The measurement agrees with the formula. Mod-N moved 79.9 % against a predicted 80 %. The ring moved about 20 % (21.4 % with 100 vnodes).
- With one vnode per node, the busiest node carries 2.6× its fair share. With 100 vnodes it carries 1.08×. You have now *seen* why virtual nodes exist, which is worth more than reading it.
- `bisect.insort` costs $O(NV)$ per insert. That is fine at this scale. A production ring would rebuild a sorted array once per membership change.

Step 9 — Solve a design problem

Use the concept inside a small design. **Problem:** a session store for 20 million concurrently active users. Each session is about 2 KB. Reads run at 200,000 RPS and writes at 20,000 RPS. A cache node may fail without logging out more than about 10 % of users.

A sketch of the solution. Data volume is $2 \times 10^7 \times 2\,\text{KB} = 40\,\text{GB}$ (SI units). With replication factor 2 that is 80 GB. Ten nodes with 16 GB of usable memory each hold it with headroom. Assign keys with a ring of 10 nodes × 150 vnodes. Write each session to the first two distinct physical nodes clockwise. If one node dies, its share (~10 %) has a replica on many different successors thanks to the vnodes, so no user is logged out and no single successor doubles its load. Figure 3.3 shows the read path during that failure.

Figure 3.3 — Session read when the primary owner has failed.



Reading the diagram. The participants are the client, the app service, the in-process ring router, the failed owner (cache-3) and the second replica on the preference list (cache-7). All arrows are synchronous request/response calls. The client uses HTTPS, and the app talks to the cache nodes over TCP. The crossed arrow is the failure: the call to cache-3 times out after a short, explicit deadline (§22.11). The app then tries the next node on the preference list. The cost to the user is one timeout of added latency, here 20 ms. The note shows the asynchronous repair: once failure detection (§27.10) removes cache-3, the router stops trying it and the extra latency goes away. The storage boundary is the cache tier, and each session lives on two nodes. To scale, add nodes. Each one takes about $1/(N+1)$ of the sessions.

Step 10 — Explain it independently

Close everything. Set a timer for three minutes and explain consistent hashing aloud, or in writing, to an imagined colleague who knows what a hash function is but nothing else. A good explanation covers five beats:

1. **The problem**: mod-N remaps about $N/(N+1)$ of keys when N changes.
2. **The trick**: put nodes and keys on the same ring and walk clockwise.
3. **The consequence**: only about $1/(N+1)$ of keys move, and only to or from the changed node.
4. **The refinement**: virtual nodes fix uneven arcs and spread failure load.
5. **The limits**: it does nothing for hot keys, and every client must share the same membership view.

Record yourself if you can. Then mark each beat as “fluent”, “hesitant” or “missing”. Each beat you marked “hesitant” or “missing” sends you back to the matching step: beats 1–2 to steps 2–3, beat 4 to step 8, beat 5 to step 7. Then turn the shaky beats into flash-cards (§3.10.2).

RECOMMENDATION

Recommendation — Keep the artifacts. Store all ten artifacts for each concept in one folder or journal entry (§3.10.3). Months later, rebuilding a concept from your own artifacts is far quicker than the first loop was. That folder becomes your personal edition of this book.

3.4 Active recall, spaced repetition, the Feynman technique, interleaving and deliberate practice

The loop tells you *what* to do with a concept. This section explains *how to practice* so that what you learn lasts. The techniques come from cognitive psychology. They were tested mostly on facts, vocabulary, science texts and mathematics, not on architecture, so applying them to system design is a reasonable extrapolation rather than a measured result.

3.4.1 What the evidence says, in one table

Dunlosky and colleagues reviewed ten widely used study techniques. They rated each one's *utility* by how well it generalized across learners, materials and test conditions [Dunlosky et al. 2013].

Table 3.4 — Study techniques by rated utility, and their system-design form.

Technique	Rated utility	System-design form
Practice testing (retrieval practice)	High	Flashcards; drawing from memory; mock design sessions
Distributed practice (spacing)	High	Scheduled reviews days and weeks apart (§3.4.3)
Elaborative interrogation	Moderate	Asking "why does Dynamo use vnodes rather than one token?"
Self-explanation	Moderate	Step 10 of the loop; the Feynman technique (§3.4.4)
Interleaved practice	Moderate	Mixing caching, queuing and replication problems in one session
Summarization	Low	Writing chapter summaries (useful only with good training)
Highlighting / underlining	Low	Marking up PDFs
Keyword mnemonic	Low	Acronyms for lists of terms
Imagery for text	Low	Picturing a text's content
Rereading	Low	Reading a chapter a second time

The ratings of the top five are the ones that matter. The two techniques that feel most like studying, highlighting and rereading, rated low. The two that feel most like being tested, practice testing and spacing, rated high.

KEY CONCEPT

Key insight — Learning versus performance. How well you do *during* practice is a poor guide to how much you are learning. Conditions that make practice feel harder, such as spacing, interleaving and testing yourself before you feel ready, often produce better long-term retention. Bjork called these **desirable difficulties** [Bjork 1994]; see [Soderstrom and Bjork 2015] for a review. If a study session felt smooth and easy, be suspicious of it.

3.4.2 Active recall (retrieval practice)

DEFINITION

Definition — Active recall. Deliberately retrieving information from memory without looking at the source, for example by answering a question, drawing a diagram or explaining aloud. Its measured benefit for later retention is called the **testing effect**.

In a well-known experiment, students read short prose passages under one of three schedules. They either studied four times (SSSS), studied three times and then took one free-recall test (SSST), or studied once and took three tests (STTT). On a test five minutes later, more studying gave better recall. On a test one week later, the order reversed. The repeated-testing group recalled about 61 % of the idea units, against about 40 % for the repeated-study group, even though the testing group had seen the text only once [Roediger and Karpicke 2006]. A later study found that retrieval practice also beat building concept maps, an “active” elaborative technique, on a test of meaningful learning [Karpicke and Blunt 2011].

Applied to system design. Every reading session should end with retrieval: close the book, and write or draw what you just learned. The checkpoints at the end of each chapter in this book exist for this reason. Answer them before you open the `<details>` block.

3.4.3 Spaced repetition

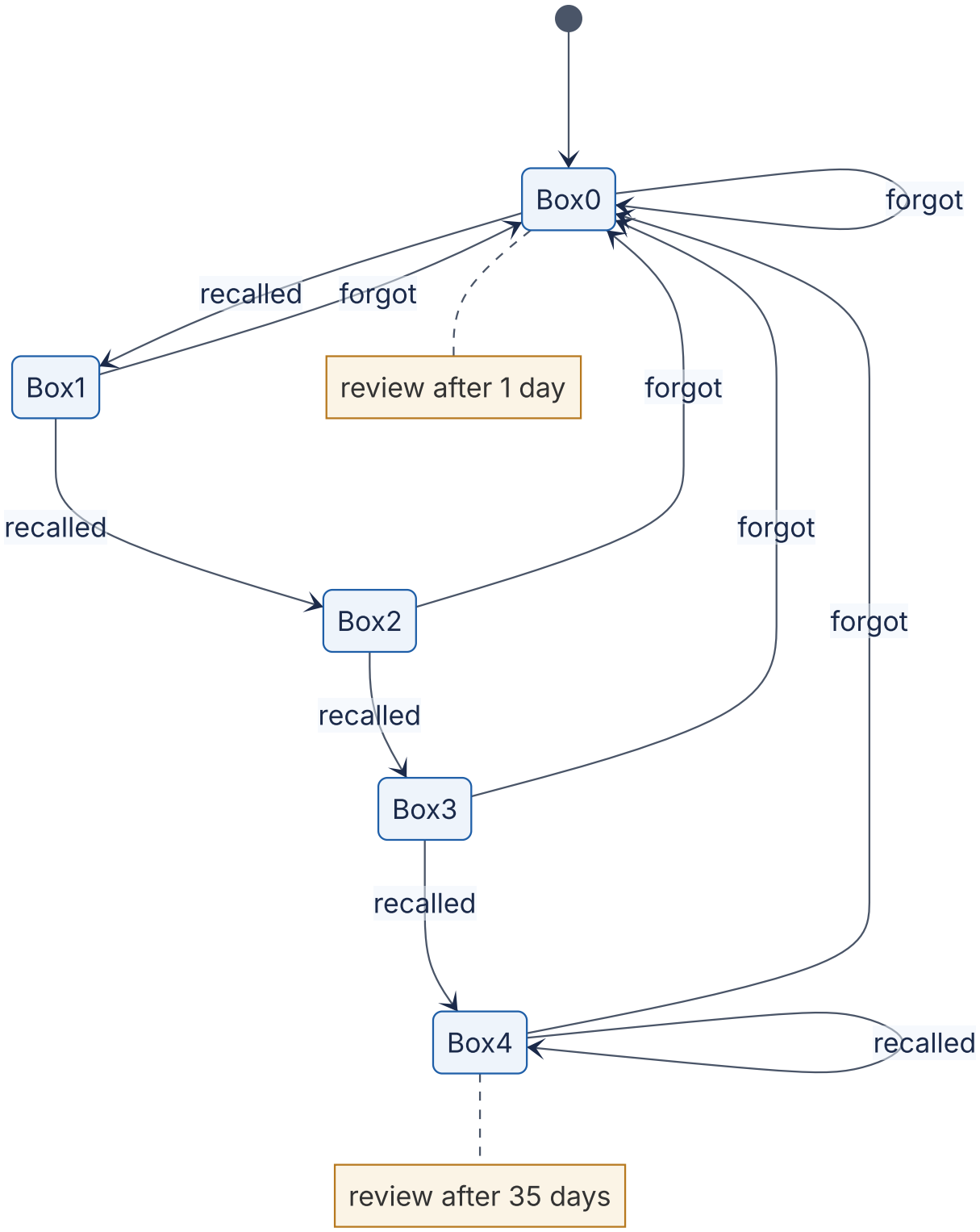
DEFINITION

Definition — Spaced repetition. Reviewing material at increasing intervals, so that each review happens around the time you would otherwise begin to forget it. The underlying finding, that spacing study sessions apart beats massing them together, is the **spacing effect**.

The spacing effect is one of the oldest results in experimental psychology. It goes back to Ebbinghaus's experiments on himself in the 1880s [Ebbinghaus 1885/1913]. A meta-analysis of hundreds of comparisons confirmed that spaced practice beats massed practice for verbal recall [Cepeda et al. 2006]. A follow-up study asked *how far apart* reviews should be. The best gap between study sessions depended on how long you need to remember the material. Expressed as a fraction of that retention interval, the best gap was about 20–40 % for a one-week retention interval and fell to about 5–10 % for a one-year interval [Cepeda et al. 2008].

The practical form of spaced repetition is the **Leitner system**: flashcards move through numbered boxes. A correct answer promotes a card to the next box, which is reviewed less often. A wrong answer sends it back to box 0. Software such as Anki automates the same idea with adaptive intervals.

Figure 3.4 — A five-box Leitner schedule for flashcards.



Reading the diagram. Each state is a box with its own review interval: 1, 3, 7, 16 and 35 days for boxes 0 to 4. The intervals are a reasonable starting point, not an empirically optimal set. Each transition is one review. “Recalled” moves a card one box up, and the top box keeps it there. “Forgot” sends it back to box 0. Data flows one card at a time. All of the state is the box number and due date per card, so a spreadsheet is enough storage.

The failure point is a backlog: if you skip a week, the reviews that were due pile up. The fix is to cap new cards per day, not to skip reviews. The schedule scales because mastered cards cost one review every five weeks or so.

The scheduler itself is small enough to build in a few minutes, which makes it a good first "simplified implementation" (§3.7):

```
        """Minimal Leitner-box scheduler for system-design flashcards."""
        from dataclasses import dataclass, field
        from datetime import date, timedelta

        INTERVALS = [1, 3, 7, 16, 35]  # days until next review, per box (box 0..4)


        @dataclass
        class Card:
            prompt: str
            box: int = 0
            due: date = field(default_factory=date.today)

            def review(self, recalled: bool, today: date) -> None:
                # Success promotes one box; failure sends the card back to box 0.
                self.box = min(self.box + 1, len(INTERVALS) - 1) if recalled else 0
                self.due = today + timedelta(days=INTERVALS[self.box])


        def due_cards(cards, today):
            return [c for c in cards if c.due <= today]


        if __name__ == "__main__":
            start = date(2026, 1, 5)
            card = Card("Why do virtual nodes reduce load imbalance?", due=start)
            today = start
            for ok in [True, True, False, True, True, True]:
                card.review(ok, today)
                print(f"{today}  recalled={ok!s:<5} -> box {card.box}, next {card.due}")
                today = card.due
```

Output:

```
2026-01-05  recalled=True  -> box 1, next 2026-01-08
2026-01-08  recalled=True  -> box 2, next 2026-01-15
2026-01-15  recalled=False -> box 0, next 2026-01-16
2026-01-16  recalled=True  -> box 1, next 2026-01-19
2026-01-19  recalled=True  -> box 2, next 2026-01-26
2026-01-26  recalled=True  -> box 3, next 2026-02-11
```

What to notice:

- `field(default_factory=date.today)` matters. A plain `date.today()` default would be evaluated once, when the class is defined, and every card would share that date. This is the kind of bug you only find by building the thing.
- One failure on 15 January wipes out two promotions. That is harsh by design: a card you forgot needs to be relearned, not just reviewed.
- After six reviews spread over five weeks, the card is due in 16 days. Spaced repetition costs little per item, and the cost keeps falling.

> WORKED EXAMPLE
>
> **Worked example — Daily review load.** You add 5 new cards per study day, 5 days a week, for a year: about $5 \times 5 \times 52 = 1,300$ cards. In the steady state, assume 70 % of cards sit in boxes 3–4 (reviewed about every 16–35 days, call it every 25 days), 20 % in box 2 (every 7 days) and 10 % in boxes 0–1 (every 2 days on average). Daily reviews $\approx 1,300 \times (0.70/25 + 0.20/7 + 0.10/2) = 1,300 \times (0.028 + 0.029 + 0.050) \approx 139$ cards. At about 8 seconds per card that is roughly 19 minutes a day. **Interpretation.** A year of system-design vocabulary stays fresh for under 20 minutes a day, *if* most cards are mastered. **Implication.** Keep cards atomic so they move up the boxes quickly. Cards that bounce between boxes 0 and 1 dominate the load, so rewrite or split them.

3.4.4 The Feynman technique (self-explanation)

The “Feynman technique” is a popular name for a four-step routine. You pick a concept, explain it in plain language as if to a newcomer, find the places where your explanation breaks down or falls back on jargon, and go back to the source to fix those places. Richard Feynman did not publish this method under that name. The label was attached later. The research behind it is the work on **self-explanation**: learners who explain material to themselves while studying understand it better than those who only read it [Chi et al. 1994]. Dunlosky and colleagues rated self-explanation and elaborative interrogation as having moderate utility [Dunlosky et al. 2013].

In system design, the routine is step 10 of the loop, plus one rule: **every “it just handles that” is a gap.** If your explanation of Kafka says “the broker handles durability”, you have not explained durability. You have named it. Push until you reach a mechanism, such as “the leader waits for in-sync replicas to append the record before acknowledging when `acks=all`” (§31.6, §31.9).

3.4.5 Interleaving

DEFINITION

Definition — Interleaving. Mixing different kinds of problems or concepts within one practice session (A B C A C B), instead of finishing all practice on one kind before moving to the next (AAA BBB CCC, called **blocking**).

In one study, students who practiced mixed (shuffled) mathematics problems did worse during practice but much better on a delayed test than students who practiced in blocks [Rohrer and Taylor 2007]. Interleaving also helped people learn to tell categories apart, such as painters' styles, even though most participants *believed* blocking had worked better for them [Kornell and Bjork 2008].

A design problem never tells you which chapter it comes from. Interleaving trains the skill that matters most in a real design: noticing *which* tool a situation calls for. In practice, once you have studied caching, queues and replication, practice with mixed questions such as "this read path is slow; is it a cache problem, a fan-out problem, or a replication-lag problem?" instead of ten caching questions in a row.

3.4.6 Deliberate practice

DEFINITION

Definition — Deliberate practice. Effortful practice designed to improve performance. It targets specific weaknesses, has clear goals, gives immediate feedback and is usually guided by a teacher or coach [Ericsson et al. 1993].

Ericsson and colleagues argued that differences in expert performance largely reflect accumulated deliberate practice. Later work found the picture more mixed. A meta-analysis found that deliberate practice explained about 12 % of the variance in performance across domains, and less in education and professions than in games, music and sports [Macnamara et al. 2014]. The honest conclusion is this: practice that targets weaknesses and comes with feedback helps, but hours alone do not guarantee expertise.

For a system designer, the scarce ingredient is **feedback**. You can get it from mock interviews with a scoring rubric (Chapter 88), from design reviews by senior engineers (§3.9.2), from comparing your design with published ones (Chapters 52–56), and from running your own simplified implementation to see whether it behaves as you predicted.

COMMON MISTAKE

Misconception — “I’m a visual learner, so I should only watch videos.” Matching instruction to self-reported “learning styles” has not been shown to improve outcomes in properly designed experiments [Pashler et al. 2008]. Diagrams do help with system design, but because the *subject* is spatial and structural, not because of your learning style. Everyone should draw.

3.5 Reading papers and engineering blogs

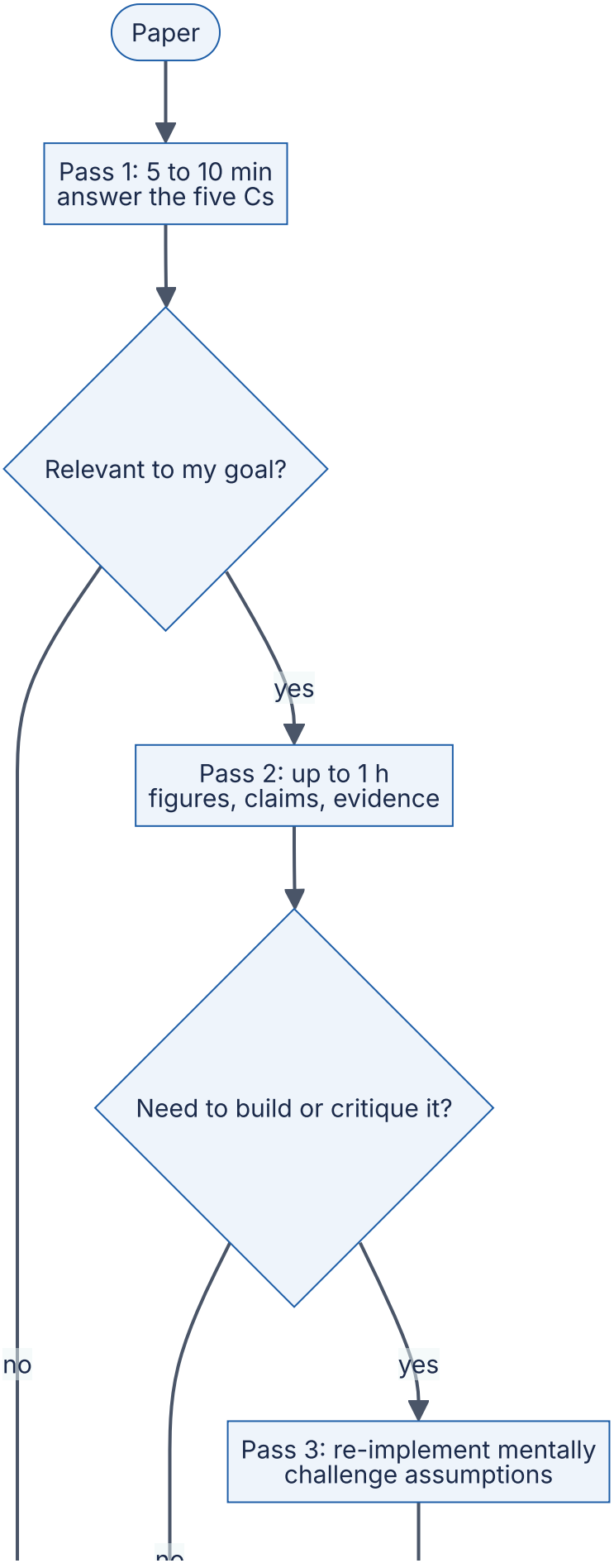
Step 5 of the loop sends you to primary sources. In system design, those are mostly conference papers (SOSP, OSDI, NSDI, USENIX ATC, VLDB, SIGMOD), official documentation and company engineering blogs. Each needs a different way of reading.

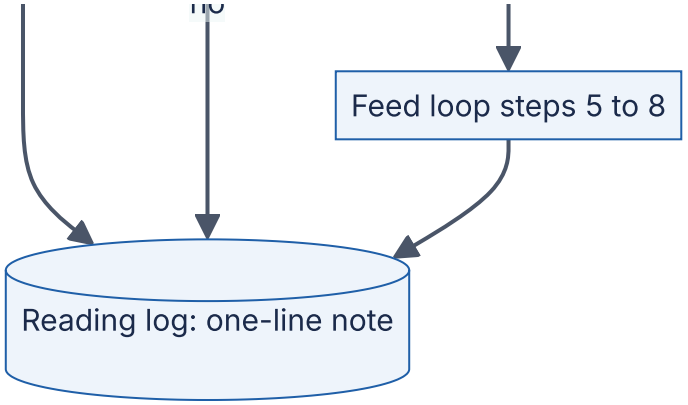
3.5.1 Keshav’s three-pass method

Keshav’s two-page note is the standard advice for reading systems papers. Read a paper in up to three passes, and decide after each pass whether the next one is worth the time [Keshav 2007].

1. **First pass (about 5–10 minutes).** Read the title, abstract and introduction. Read the section headings and nothing under them. Glance at any mathematics, then read the conclusions and scan the references. Afterwards, you should be able to answer the **five Cs**: *Category* (what type of paper is it?), *Context* (which other work does it build on?), *Correctness* (do the assumptions look valid?), *Contributions* (what is new?) and *Clarity* (is it well written?).
2. **Second pass (up to about an hour).** Read carefully but skip proofs. Study the figures, graphs and axes. Mark the references you have not read. Afterwards, you should be able to summarize the paper’s main thrust, with evidence, to someone else.
3. **Third pass (about 4–5 hours for a beginner, about an hour for an experienced reader).** Virtually re-implement the paper. Make the same assumptions, recreate the work, and compare your version with the authors’. This pass exposes hidden assumptions and missing citations.

Figure 3.5 — The three-pass reading decision flow.


Paper
Pass 1: 5 to 10 min
answer the five Cs
Relevant to my goal?
yes
Pass 2: up to 1 h
figures, claims, evidence
Need to build or critique it?
no
yes
Pass 3: re-implement mentally
challenge assumptions



Reading the diagram. A paper enters at the top. Each rectangle is a pass with a time budget, and each diamond is a decision to stop or go deeper. Every path ends at the reading log, the cylinder. Even a paper you rejected after five minutes gets a one-line note, so you will not reread it by accident. Only the third pass feeds steps 5–8 of the loop. The usual failure is starting at the third pass: reading page one in detail and giving up on page four. To scale this across many papers, do many first passes and only a few third passes. For most engineers, about one paper in five deserves the third.

When you apply the method to a *systems* paper, add two questions to the five Cs. **What was the workload?** Dynamo's design follows from Amazon's shopping-cart writes, which had to be "always writeable". **What did they choose not to provide?** GFS gave up a POSIX-compatible interface and offered relaxed consistency [Ghemawat et al. 2003]. A paper's non-goals tell you as much as its goals.

Table 3.5 — A starter reading list mapped to this book.

Paper	Read with	Why this one first
Lamport, *Time, Clocks, and the Ordering of Events* [Lamport 1978]	§24.10	Short; defines "happened-before"
The Google File System [Ghemawat et al. 2003]	§15.5	Clear workload-driven trade-offs
MapReduce [Dean and Ghemawat 2004]	§29.4	Simple model; good first third-pass paper
Dynamo [DeCandia et al. 2007]	§14.8, §27.7	Assembles many chapter concepts in one design
The Chubby Lock Service [Burrows 2006]	§26.14	Engineering lessons, not just algorithms
ZooKeeper [Hunt et al. 2010]	§26.8	Coordination as a service
Kafka [Kreps et al. 2011]	§31.3	The log as a central abstraction
Raft [Ongaro and Ousterhout 2014]	§26.7	Written explicitly to be understandable
The Tail at Scale [Dean and Barroso 2013]	§23.6–§23.7	Magazine-length; shapes how you think about latency
Spanner [Corbett et al. 2012]	§28.8	Advanced; read after Chapters 25–26

3.5.2 Reading engineering blogs critically

Company engineering blogs are where the field's current practice gets written down. They are also written to recruit engineers and to promote the company, and they rarely go through peer review. Read them as **evidence of what one team did under its own constraints**, not as recommendations for you.

Table 3.6 — Critical-reading checklist for engineering posts.

Ask	Why	Red flag
What was the workload and scale?	Solutions depend on the workload	No numbers at all, or only "massive scale"
What was the baseline?	A speed-up needs something to compare against	"10× faster" with no stated "than what"
What did they try that failed?	Shows what the real constraint was	Only a success story
What did it cost?	Every choice costs something	No downsides mentioned
When was it written?	Systems and versions change	No date; describes a deprecated version
Can it be checked?	Separates evidence from narrative	No benchmark method; no code; no follow-up
Who benefits from you believing it?	Vendor posts sell products	"Our product solved it" with no alternatives considered

This book uses the same discipline in its **evidence labels** (§9 of the style guide): Primary, Secondary or Illustrative. Label your own notes the same way. A claim you read in a conference talk summary is Secondary, even if it sounds authoritative.

3.6 Drawing architecture from memory

3.6.1 Why drawing from memory works

Drawing a system from memory is retrieval practice (§3.4.2) on structured material. It also builds what Chase and Simon called **chunks**. In their studies, chess masters reconstructed real game positions from a brief glance far better than novices. On random piece placements, the masters' advantage largely disappeared [Chase and Simon 1973]. Their expertise was stored as meaningful patterns, not as better raw memory. Reference architectures are the chunks of system design: "CDN → load balancer → stateless service → cache → primary/replica database", or "producer → partitioned log → consumer group". Once each of these is a single chunk, your working memory is free to reason about what is *different* in the problem in front of you.

3.6.2 The protocol

1. **Choose a target**: a figure from a chapter you finished at least two days ago.
2. **Draw blind for 10 minutes** on paper or a whiteboard. Use the book's shapes (§6 of the style guide): stadiums for clients, rectangles for services, cylinders for stores, subroutine shapes for queues.
3. **Annotate edges**: protocol, synchronous or asynchronous, direction.
4. **Compare** with the source and score your drawing with Table 3.7.
5. **Write the diff**: every missing component and every wrong edge becomes a flashcard.
6. **Redraw** after the next spacing interval (3 days, then 7, then 16).

Table 3.7 — Scoring rubric for a drawing from memory (0–2 points each, 12 maximum).

Criterion	0	1	2
Components	Major pieces missing	All major pieces present	Also includes supporting pieces (config, discovery, monitoring)
Data flow	Arrows missing or wrong	Correct direction	Read and write paths distinguished
Sync vs async	Not marked	Partly marked	Every edge correct
Protocols	None	Some	Every edge labeled
Storage boundaries	Unclear where state lives	Stores shown	Durability of each store stated
Failure points	None marked	One marked	Each single point of failure plus its mitigation

A score of 10–12 on a figure means you can use that architecture fluently. Below 7 means you should go back to loop step 3 for the components you missed.

> INTERVIEW TIP
>
> **Interview tip.** Interviewers do not score how pretty your diagram is. They notice whether you mark the read and write paths separately and whether you can point at the single points of failure without being asked. Those are the two highest-value rows of Table 3.7, and both can be trained.

3.7 Building simplified implementations

3.7.1 Why build at all

Feynman's blackboard at the time of his death carried the line "What I cannot create, I do not understand." Building a small version of a mechanism forces you to make every decision the prose glossed over: what happens on wrap-around, what a timeout returns, which component owns a piece of state. The consistent-hashing ring in §3.3 and the Leitner scheduler in §3.4.3 each surfaced a bug that reading would not have, a process-randomized `hash()` and a shared default date.

3.7.2 Scoping rules

1. **Keep the mechanism; fake the environment.** Simulate the network with function calls and the clock with a counter. Simulate disks with dictionaries.
2. **One file, under 300 lines.** If it grows past that, you are building a product, not a learning model.

3. **Build in a measurement.** Every simplified implementation should print a number that tests a claim from the chapter: keys moved, hit ratio, p99 latency, messages lost.
4. **Inject one failure.** Kill a node, drop a message or skew a clock, then check whether the behavior matches Step 7's failure table.
5. **Compare with a real system afterwards.** Read the corresponding part of a real codebase or its documentation and list three things it does that yours does not. That list is your map to production concerns.

Table 3.8 — Simplified implementations that pay off, by chapter.

Build	Chapter	The measurement that proves it works
LRU cache with TTL	§20.8–§20.9, §38.10	Hit ratio against cache size on a Zipf-like key stream
Token-bucket rate limiter	§22.4	Accepted rate converges to the refill rate
Consistent-hash ring	§14.8	Keys moved ≈ $1/(N+1)$; max/mean load against vnode count
Bloom filter	§17.4	Measured false-positive rate matches the formula
Append-only log with offsets	§31.3, §31.8	Consumer restarts from its committed offset
Leader election with leases	§26.5, §26.11	At most one leader per lease epoch under clock skew
Raft log replication (simulated network)	§26.7	No committed entry lost when a minority of nodes crash
Retry with exponential backoff and jitter	§22.12	Load on a recovering server, with and without jitter

Chapter 93 turns several of these into full projects with deployment, testing and monitoring. The simplified versions here are the step before that.

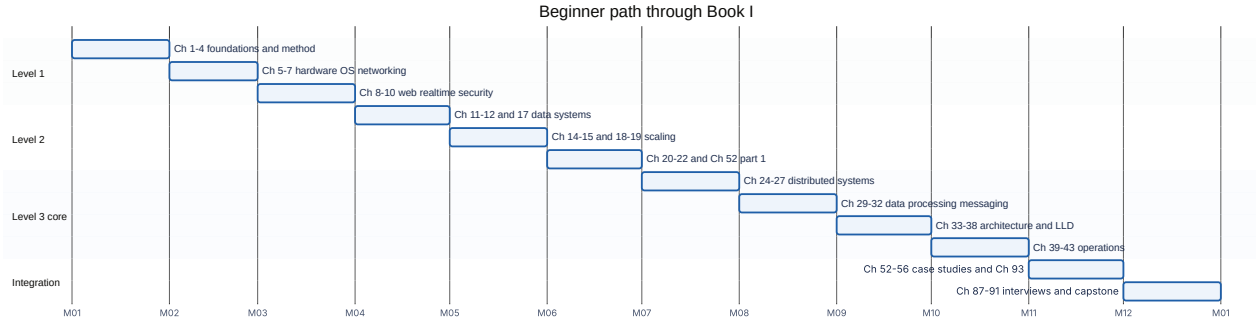
WARNING / TRADE-OFF

Warning — The toy-to-production gap. A 100-line Raft simulation that passes your tests is not a correct Raft. Consensus implementations fail in rare interleavings of events that small tests seldom reach, which is why Jepsen-style testing exists (§25.11). Treat your build as a model of the mechanism, never as evidence that the real problem is easy.

3.8 Beginner methodology: a 12-month path

This path is for readers starting at Level 1: you can program, but you have not designed distributed systems. It assumes 8–10 hours a week. That is enough for about four full concept loops a month plus lighter reading. The target is **Level 2 by month 6 and Level 3 on core Book I material by month 12**. Levels 4 and 5 (senior/staff, principal) depend on production experience that no reading plan can replace. Part IX and §3.9 are where you go next.

Figure 3.6 — The 12-month beginner path (months shown as M01–M12).



Reading the diagram. Each bar is one month of study, and bars run one after another. The `after` dependencies mean that each block assumes you have finished the one before it. The calendar dates are placeholders, so read the axis as month numbers. The three sections match the proficiency levels you should reach by the end of each phase. The fourth section, Integration, combines everything rather than adding new material. The common failure is slipping in months 7–8. Distributed systems is the steepest part of the path, so plan for those two months to take three. There are two ways to scale the plan. At 15+ hours a week, the whole path fits in about eight months. An ML-track reader can swap month 9 for Chapters 57–60 and month 10 for Chapters 66 and 72–75.

Table 3.9 — Month-by-month plan with loop targets and milestones.

Month	Chapters	Full-loop concepts (examples)	Milestone artifact
1	1–4	Latency vs throughput; Little's Law; availability arithmetic	Flashcard deck started; first design journal entry
2	5–7	Memory hierarchy; event loops; TCP handshake; DNS	Request lifecycle drawn from memory (§8.5 preview)
3	8–10	REST design; OAuth 2.0 flow; TLS; threat model	API spec for a URL shortener (§52.1)
4	11–12, 17	B+ tree index; isolation levels; Bloom filter	Bloom filter build (Table 3.8)
5	14–15, 18–19	Replication lag; consistent hashing; L4 vs L7 balancing	Hash ring build (§3.3)
6	20–22; 52.1–52.3	Cache-aside; cache stampede; token bucket	**L2 checkpoint**: rate limiter design (§52.3), timed at 45 min
7	24–27	Lamport clocks; linearizability; Raft; quorums	Raft election simulation
8	29–32	Partitioned log; consumer groups; outbox; saga	Log with offsets build
9	33–38	Modular monolith; bounded contexts; LLD of an LRU cache	LLD exercise from §38.10
10	39–43	SLOs and error budgets; tracing; canary releases	One postmortem study (§3.9.3)
11	52–56; 93.1–93.2	Five case studies through all 16 template sections	One beginner project from Ch 93 deployed
12	87–89, 91	Estimation drills; interview pacing	**L3 checkpoint**: three mock interviews scored with the Ch 88 rubric

Chapters 13, 16, 23 and 28 and all of Part IX are **deferred to year two**. They are L3–L5 material that builds on everything above. Read their introductions when a case study points to them, but do not run full loops on them yet.

RECOMMENDATION

Recommendation — The weekly rhythm. Use two weekday sessions of 2 hours for loop steps 1–7 on one concept, and one weekend session of 3 hours for steps 8–9. Do 15–20 minutes of flashcards on five days. Close the week with a 30-minute review: one drawing from memory of a concept from last month and one journal entry. This rhythm builds in spacing, interleaving and retrieval without any extra planning.

3.9 Advanced methodology for experienced engineers

If you already build production systems, you do not need a path from Chapter 1. You need to know **where your gaps are**. Experience tends to be deep in the systems you have run and thin everywhere else. A payments engineer may know idempotency keys (§22.15) and sagas very well and know little about CDNs or GPU serving.

3.9.1 Gap analysis

Rate yourself from 0 to 4 on each of the book's 19 parts, where 0 means "cannot define the main terms" and 4 means "could teach it and have operated it in production". Set a target for each part from your role or the job you want. Then rank the parts by weighted gap.

FORMULA

Gap priority

$$P_i = (T_i - C_i) \times W_i$$

Symbol	Meaning	Units
P_i	Priority score for part i	points (0–12)
T_i	Target proficiency for part i	0–4 scale
C_i	Current self-rated proficiency (use 0 if $C_i > T_i$ would make P_i negative)	0–4 scale
W_i	Relevance weight: 1 = nice to have, 2 = useful, 3 = essential for role	dimensionless (1–3)

WORKED EXAMPLE

Worked example — A backend engineer moving to an infrastructure role.

Table 3.10 — Gap-analysis scores for one engineer.

Part	T	C	W	$P = (T - C) \times W$
V Distributed systems	4	2	3	$2 \times 3 = 6$
VIII Operations	4	3	3	$1 \times 3 = 3$
IV Scalability and performance	3	2	2	$1 \times 2 = 2$
XIII Training infrastructure	2	0	1	$2 \times 1 = 2$

Interpretation. Part V comes first with a score of 6, three times any other part. Parts IV and XIII tie at 2. Their gaps differ, but weighting by relevance brings them level. **Implication.** Spend the next quarter on Chapters 24–28 with full loops, and keep the others to first-pass reading. Re-rate yourself each quarter, because a self-rating is only a guess until a design review or a mock interview tests it.

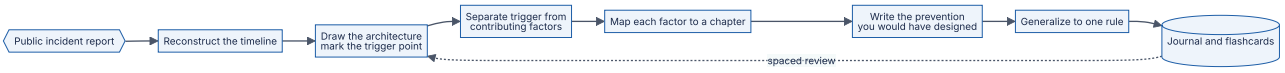
3.9.2 Design reviews as a learning instrument

Design reviews (§1.10) are deliberate practice with expert feedback already built in. Use them on purpose. **As an author**, write a design document for a real change and ask reviewers to challenge specific sections, such as failure modes or capacity numbers. **As a reviewer**, read others' designs with Table 3.7 and Table 3.3 in mind, and write at least one question about failure handling and one about cost. **As an observer**, sit in on reviews outside your team. Hearing how senior engineers question a design teaches you what to ask before you can answer it yourself. After each review, write down in your journal one thing you would have missed.

3.9.3 Postmortem study

Production incidents are the most honest record of how systems fail. Many organizations publish blameless postmortems. The SRE book describes postmortem culture as a way to learn from failure without assigning blame [Beyer et al. 2016]. Curated collections, such as Dan Luu's public list of postmortems, make it easy to find many of them [Luu, postmortems]. Research that analyzes failures across many systems shows patterns that single reports cannot. One study of failures in distributed data-intensive systems found that most catastrophic failures came from incorrect handling of non-fatal errors that the software had already signaled [Yuan et al. 2014]. A study of hundreds of public cloud outage reports cataloged their root causes and how far their impact spread [Gunawi et al. 2016].

Figure 3.7 — The postmortem study loop.



Reading the diagram. The hexagon is an external source, a published report. Each rectangle is an analysis step, done in order. The two most valuable steps are drawing the architecture yourself, since reports rarely include a complete diagram, and separating the *trigger* (what started it) from the *contributing factors* (why it spread and why recovery was slow). Mapping each factor to a chapter turns an anecdote into a lesson you can review. The rule you generalize goes into durable storage, your journal and flashcards. The dotted arrow is a spaced review in which you redraw the architecture from memory. The failure point in this process is stopping at the trigger. “Someone typed the wrong command” is never the whole story.

INDUSTRY EXAMPLE

Amazon S3, US-EAST-1, February 2017. While debugging a billing-system issue, an engineer ran an established playbook command meant to remove a small number of servers. An input was entered incorrectly, and a much larger set of servers was removed, including servers supporting two S3 subsystems: the index subsystem and the placement subsystem. Both needed a full restart, which took longer than expected because those subsystems had not been fully restarted in years while S3 had grown substantially. In response, AWS changed the tool to remove capacity more slowly and to refuse to take any subsystem below its minimum required capacity. It also prioritized splitting the index subsystem into smaller cells. **Lessons mapped to chapters:** guard rails on operational tools (§43.3), blast-radius reduction with cells (§33.12, Chapter 44), and recovery time as a design property (§39.8). *Evidence: Primary* — [AWS 2017]

3.9.4 Reading production incident reports critically

Published reports are edited for a public audience and tend to leave out organizational causes and near misses. Read them with Table 3.6 in mind. Separate **what the report states** from **what you infer**. Look for the **detection gap**, the time from trigger to first alert, which is often more instructive than the bug. Ask what the incident shows about the **design**, not the operator: why could one command remove so much capacity? One report a week gives you about 50 failure case studies a year, and that breadth is a large part of what distinguishes a Level 4 engineer.

3.10 Tracking progress

3.10.1 Self-assessment rubric

Rate yourself each quarter on the seven dimensions below, and ask a peer or mentor to rate you as well. A large gap between their rating and yours is itself useful information.

Table 3.11 — Self-assessment rubric mapped to the five proficiency levels.

Dimension	L1 Beginner	L2 Intermediate	L3 Advanced	L4 Senior/Staff	L5 Principal
Concept knowledge	Defines core terms	Explains building blocks	Explains internals	Knows edge cases and variants	Shapes how the field uses the concepts
Mechanism depth	Describes what it does	Describes how, step by step	Implements a simplified version	Debugs production behavior	Designs new mechanisms
Trade-off reasoning	Names one pro and one con	Compares two options	Compares options with numbers	Weighs cost, organization and time	Sets direction for many teams
Estimation	Needs a template	Estimates RPS and storage	Estimates the bottleneck resource	Builds a capacity model	Owns fleet-level planning
Failure reasoning	Rarely considers failure	Lists failure modes	Designs detection and recovery	Limits the blast radius	Sets reliability strategy
Communication	Describes components	Draws a clear diagram	Leads a 45-minute design discussion	Runs design reviews	Writes standards others adopt
Implementation	Follows tutorials	Builds small services	Builds simplified distributed mechanisms	Ships and operates such systems	Builds platforms others build on

Appendix E gives the full definitions of the levels. Your overall level is roughly the level reached by your *weakest* three dimensions, not your strongest, because a design is only as strong as its weakest reasoning.

3.10.2 Flashcard templates

Make each card **atomic** (one fact or one relationship) and prefer **why** and **when** cards over **what** cards. Six card types cover most of system design:

```
[DEFINITION]  Q: Define linearizability in one sentence.
[MECHANISM]   Q: In a hash ring, which node owns key k?  (answer in two steps)
[NUMBER]      Q: Rough round-trip time within one datacenter? (cite Appendix F, with
year)
[TRADE-OFF]   Q: When would you choose jump hash over a ring with vnodes?
[FAILURE]     Q: A cache node dies under mod-N hashing. What happens to the database,
and why?
[DIAGRAM]     Q: Draw cache-aside read and write paths. (check against Fig. 20.x)
```

Each card should also record its source section (for example `src: §3.3 step 6`), so a failed card leads you straight to the material it came from.

3.10.3 Design journal template

```
Date:              Concept / problem:              Loop steps completed: 1-10
1. One-sentence definition (from memory):
2. Problem it solves, with a number:
3. Mechanism in 5 steps or fewer:
4. Drawing score (Table 3.7): __ / 12   Missing:
5. Source studied + evidence label:
6. Best alternative and when I would pick it:
7. Scariest failure mode and its mitigation:
8. Build: repo/file + the measurement it printed:
9. Design problem used in + what I would change next time:
10. Explanation beats fluent / hesitant / missing:
Cards created:                    Next review date:
```

3.10.4 The concept tracker

Table 3.12 — Concept tracker (example rows).

Concept	Section	Steps done	Drawing score	Last review	Next review	Confidence (1–5)
Consistent hashing	§14.8	1–10	11/12	2027-05-03	2027-05-19	4
Raft leader election	§26.7	1–7	8/12	2027-07-10	2027-07-13	2
Cache stampede	§20.11	1–4, 6–7	9/12	2027-06-02	2027-06-09	3

Look at the tracker every week. Three numbers tell you whether the method is working: **flashcard retention**, meaning the fraction recalled on review, where around 85–90 % is a healthy target; **average drawing score**, which should rise over a quarter; and **loops completed per month**, about four on the beginner path. If retention falls below about 80 %, you are adding cards faster than you are learning them. Slow down. If drawing scores stall, put more time into loop step 3.

3.11 Summary and revision notes

SUMMARY

- Mastery means you can **produce** the concept without notes: define it, draw it, build it, use it and explain it. Recognizing it when you read it does not count.
- The **ten-step loop** turns each concept into ten artifacts. Step 10 (explain independently) checks the other nine, and any gap sends you back to step 3.
- Consistent hashing shows the loop's value. The formula predicts that ~80 % of keys move under mod-N and ~20 % under a ring when going from 4 to 5 nodes. A 60-line build confirmed it and showed that virtual nodes cut the busiest node's load from 2.6× to 1.08× the mean.
- **Retrieval practice** and **spacing** have the strongest evidence. Rereading and highlighting have the weakest [Dunlosky et al. 2013]. Self-explanation and interleaving have moderate support. Deliberate practice helps, but it explains only part of the differences in expertise.
- Read papers in **three passes** [Keshav 2007]. Read blogs as evidence of one team's choices under their own constraints, and label every claim Primary, Secondary or Illustrative.
- **Drawing from memory** builds chunks. Score your drawings out of 12 on components, flows, sync/async, protocols, storage and failure points.
- **Simplified implementations** keep the mechanism, fake the environment, print a measurement and inject a failure.
- Beginners: follow the 12-month path to reach L2 by month 6 and L3 on core Book I material by month 12. Experienced engineers: rank gaps with $P = (T - C) \times W$, use design reviews as deliberate practice and study one incident report a week.
- Track flashcard retention (~85–90 %), drawing scores and loops per month. Judge your level by your weakest dimensions.

3.12 Knowledge checkpoint

1. What artifact does each of loop steps 2, 4, 8 and 10 produce?
2. Why does the loop return to step 3, not step 1, when step 10 reveals a gap?
3. Using the formula, what fraction of keys move when a mod-N cluster grows from 9 to 10 nodes? And under consistent hashing?
4. Why does Python's built-in `hash()` make a poor ring hash for a multi-process service?
5. Name two techniques Dunlosky et al. rated high utility and two they rated low.
6. In Roediger and Karpicke's experiment, which condition did best at five minutes and which at one week?

7. According to Cepeda et al. (2008), does the best review gap, as a fraction of the retention interval, grow or shrink as the retention interval gets longer?
8. What is the difference between interleaved and blocked practice, and why does interleaving suit system design?
9. List Keshav's five Cs.
10. Which pass of the three-pass method feeds loop steps 5–8, and roughly how long does it take a beginner?
11. Give three questions from the blog-reading checklist.
12. What did Chase and Simon's random-position condition show about expertise?
13. State four scoping rules for a simplified implementation.
14. Compute the gap priority for a part with target 3, current 1 and weight 2.
15. In a postmortem study, why separate the trigger from the contributing factors?
16. Your flashcard retention has fallen to 72 %. What should you change?

Answers

1. Step 2: a "before" scenario with numbers. Step 4: a diagram drawn from memory. Step 8: a runnable program that prints a measurement. Step 10: a 2–5 minute explanation given without notes.
2. Most gaps in an explanation are gaps in the mechanism. The definition (step 1) is rarely what is missing.
3. Mod-N: $N/(N+1) = 9/10 = 90\%$. Consistent hashing: about $1/(N+1) = 10\%$.
4. Python randomizes `hash()` for strings in each process, so different processes would build different rings. That is a split-view failure.
5. High: practice testing and distributed practice. Low: any two of rereading, highlighting, summarization, keyword mnemonic and imagery for text.
6. At five minutes, repeated study (SSSS) did best. At one week, repeated testing (STTT) did best, about 61 % against 40 %.
7. It shrinks: from about 20–40 % for a one-week interval to about 5–10 % for a one-year interval.
8. Interleaving mixes problem types within a session, while blocking finishes one type before starting the next. Real design problems do not say which tool they need, and interleaving trains you to choose.
9. Category, Context, Correctness, Contributions, Clarity.
10. The third pass, which takes about 4–5 hours for a beginner.
11. Any three of: What was the workload and scale? What was the baseline? What failed? What did it cost? When was it written? Can it be checked? Who benefits?
12. On random positions, the masters' advantage largely disappeared. Their expertise was stored as meaningful patterns (chunks), not as better general memory.
13. Keep the mechanism and fake the environment; one file under 300 lines; build in a measurement; inject one failure; compare with a real system afterwards.
14. $P = (3 - 1) \times 2 = 4$.

15. The trigger explains how the incident started. The contributing factors explain why it spread and why recovery was slow, and those are what design can change.
16. Add fewer new cards, and rewrite or split the cards that keep falling back to box 0. Cards that are too big or vague cause most of the failures.

3.13 Interview questions

Interviewers sometimes ask how you learn, especially for senior roles. The concept used to demonstrate the loop, consistent hashing, also comes up often in interviews.

Basic

Q1. How do you get up to speed on a technology you have never used? Answer. Describe a method, not a list of resources: find the problem it solves and what came before it; first-pass the official architecture docs or paper; draw the architecture from memory and check it; build a toy that measures something you care about; read its known failure modes or incident reports; and write a one-page summary for your team, which exposes what you do not yet understand.

Q2. Why does `hash(key) mod N` cause problems when you add a cache node? Answer. With uniform hashing, about $N/(N+1)$ of keys change owner, so most of the cache misses at once and the load falls on the database. Consistent hashing moves only about $1/(N+1)$ of keys, all of them to the new node.

Intermediate

Q3. What do virtual nodes fix in consistent hashing, and what do they cost? Answer. With one position per node, arc lengths vary widely, so one node can carry two or three times its share. A dead node's whole load also lands on a single successor. With V vnodes per node, load evens out, the spread shrinking roughly as $1/\sqrt{V}$, and failure load is spread over many nodes. The costs are $O(NV)$ memory for the ring, larger membership metadata and more key ranges to manage during rebalancing.

Q4. Tell me about a postmortem you learned from. Answer. Choose a public report, such as the 2017 S3 disruption. Describe the trigger (a mistyped input to a capacity-removal command), the contributing factors (no minimum-capacity safeguard, and subsystems that had not been fully restarted in years at their current scale), and the design changes (slower removal, capacity floors, cell partitioning). Then say what you changed in your own work because of it.

Advanced

Q5. Your organization's design reviews keep missing failure modes that later cause incidents. How would you change the process? Answer. Require a failure table in the design template (what goes wrong, why, detection, recovery, prevention); give one reviewer the explicit failure-hunting role; link each design to postmortems of similar architectures; and track whether later incidents were or were not anticipated in the design. That is deliberate practice at team scale, with feedback from real outcomes.

Q6. A ring-based cache shows one node at 3× average CPU even with 200 vnodes per node. Diagnose it. Answer. With 200 vnodes, the ring itself should keep the busiest node within roughly 10 % of the mean, so look elsewhere. The likely causes are a hot key, which hashing cannot split; clients with inconsistent membership sending one range to the wrong node; or a heterogeneous node, such as a smaller instance or a noisy neighbor. Check top-K key metrics and compare ring versions across clients. Fixes include replicating hot keys, adding a local cache tier, bounded-load hashing and versioned membership.

3.14 Practical exercises

1. **Run the loop.** Take the cache-aside pattern (§20.7) through all ten steps. Turn in the ten artifacts, and use the journal template (§3.10.3) for at least one of them.
2. **Extend the ring.** Add `get_nodes(key, r)` to the §3.3 `HashRing` so that it returns r distinct physical nodes. Measure the fraction of keys whose replica set changes when you remove a node.
3. **Measure the vnode curve.** For $V \in \{1, 10, 50, 100, 500\}$, plot max/mean load across 10 nodes. Check whether the excess over 1.0 shrinks roughly as $1/\sqrt{V}$.
4. **Three passes.** Read the Dynamo paper with Keshav's method. Turn in the five Cs after pass 1, a one-paragraph summary after pass 2, and, after pass 3, a list of three assumptions that would not hold for a bank ledger.
5. **Critique a post.** Choose a recent engineering blog post about a migration. Score it against Table 3.6 and rewrite its main claim with an evidence label.

3.15 Design challenges

1. **Design your own learning system.** Treat your next six months of study as a system. State requirements (hours per week, target level), components (deck, journal, tracker, build repository), data flows (reading → artifacts → cards → reviews) and failure modes (backlog, burnout, illusion of competence). Add detection and recovery for each failure mode.
2. **A team incident library.** Design a lightweight internal system in which engineers log public and internal postmortems, tag them with this book's chapters and receive

spaced “redraw this architecture” prompts. Estimate storage and request rates for an organization of 2,000 engineers.

3. **Gap-driven onboarding.** Design a 90-day onboarding plan for an experienced engineer joining an ML-infrastructure team from a web-backend background. Use the gap formula, Table 3.11 and specific chapters from Book II.

3.16 References

1. Beyer, B., Jones, C., Petoff, J., Murphy, N. R. (eds.). *Site Reliability Engineering: How Google Runs Production Systems* (Ch. 15, “Postmortem Culture: Learning from Failure”). O’Reilly, 2016. https://sre.google/sre-book/postmortem-culture/
2. Bjork, R. A. *Memory and metamemory considerations in the training of human beings*. In J. Metcalfe and A. P. Shimamura (eds.), *Metacognition: Knowing about Knowing*, MIT Press, 1994.
3. Burrows, M. *The Chubby Lock Service for Loosely-Coupled Distributed Systems*. OSDI, 2006.
4. Cepeda, N. J., Pashler, H., Vul, E., Wixted, J. T., Rohrer, D. *Distributed Practice in Verbal Recall Tasks: A Review and Quantitative Synthesis*. Psychological Bulletin 132(3), 2006.
5. Cepeda, N. J., Vul, E., Rohrer, D., Wixted, J. T., Pashler, H. *Spacing Effects in Learning: A Temporal Ridgeline of Optimal Retention*. Psychological Science 19(11), 2008. https://pubmed.ncbi.nlm.nih.gov/19076480/
6. Chase, W. G., Simon, H. A. *Perception in Chess*. Cognitive Psychology 4(1), 1973.
7. Chi, M. T. H., de Leeuw, N., Chiu, M.-H., LaVancher, C. *Eliciting Self-Explanations Improves Understanding*. Cognitive Science 18(3), 1994.
8. Corbett, J. C., et al. *Spanner: Google’s Globally-Distributed Database*. OSDI, 2012.
9. Dean, J., Barroso, L. A. *The Tail at Scale*. Communications of the ACM 56(2), 2013.
10. Dean, J., Ghemawat, S. *MapReduce: Simplified Data Processing on Large Clusters*. OSDI, 2004.
11. DeCandia, G., et al. *Dynamo: Amazon’s Highly Available Key-value Store*. SOSP, 2007.
12. Dunlosky, J., Rawson, K. A., Marsh, E. J., Nathan, M. J., Willingham, D. T. *Improving Students’ Learning With Effective Learning Techniques: Promising Directions From Cognitive and Educational Psychology*. Psychological Science in the Public Interest 14(1), 2013.
13. Ebbinghaus, H. *Memory: A Contribution to Experimental Psychology* (trans. H. A. Ruger and C. E. Bussenius). Teachers College, Columbia University, 1913 (German original 1885).
14. Eisenbud, D. E., et al. *Maglev: A Fast and Reliable Software Network Load Balancer*. NSDI, 2016.

15. Ericsson, K. A., Krampe, R. T., Tesch-Römer, C. *The Role of Deliberate Practice in the Acquisition of Expert Performance*. Psychological Review 100(3), 1993.
16. Ghemawat, S., Gobioff, H., Leung, S.-T. *The Google File System*. SOSP, 2003.
17. Gunawi, H. S., et al. *Why Does the Cloud Stop Computing? Lessons from Hundreds of Service Outages*. ACM SoCC, 2016.
18. Hunt, P., Konar, M., Junqueira, F. P., Reed, B. *ZooKeeper: Wait-free Coordination for Internet-scale Systems*. USENIX ATC, 2010.
19. Karger, D., Lehman, E., Leighton, T., Panigrahy, R., Levine, M., Lewin, D. *Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web*. ACM STOC, 1997.
20. Karpicke, J. D., Blunt, J. R. *Retrieval Practice Produces More Learning than Elaborative Studying with Concept Mapping*. Science 331(6018), 2011.
21. Keshav, S. *How to Read a Paper*. ACM SIGCOMM Computer Communication Review 37(3), 2007.
22. Kornell, N., Bjork, R. A. *Learning Concepts and Categories: Is Spacing the "Enemy of Induction"?* Psychological Science 19(6), 2008.
23. Kreps, J., Narkhede, N., Rao, J. *Kafka: a Distributed Messaging System for Log Processing*. NetDB Workshop, 2011.
24. Lakshman, A., Malik, P. *Cassandra: A Decentralized Structured Storage System*. ACM SIGOPS Operating Systems Review 44(2), 2010.
25. Lamping, J., Veach, E. *A Fast, Minimal Memory, Consistent Hash Algorithm*. arXiv:1406.2294, 2014. https://arxiv.org/abs/1406.2294
26. Lamport, L. *Time, Clocks, and the Ordering of Events in a Distributed System*. Communications of the ACM 21(7), 1978.
27. Luu, D. *A collection of postmortems* (GitHub repository). https://github.com/danluu/post-mortems
28. Macnamara, B. N., Hambrick, D. Z., Oswald, F. L. *Deliberate Practice and Performance in Music, Games, Sports, Education, and Professions: A Meta-Analysis*. Psychological Science 25(8), 2014.
29. Mirrokni, V., Thorup, M., Zadimoghaddam, M. *Consistent Hashing with Bounded Loads*. ACM-SIAM SODA, 2018. https://arxiv.org/abs/1608.01350
30. Ongaro, D., Ousterhout, J. *In Search of an Understandable Consensus Algorithm*. USENIX ATC, 2014.
31. Pashler, H., McDaniel, M., Rohrer, D., Bjork, R. *Learning Styles: Concepts and Evidence*. Psychological Science in the Public Interest 9(3), 2008.
32. Roediger, H. L., Karpicke, J. D. *Test-Enhanced Learning: Taking Memory Tests Improves Long-Term Retention*. Psychological Science 17(3), 2006.
33. Rohrer, D., Taylor, K. *The Shuffling of Mathematics Problems Improves Learning*. Instructional Science 35, 2007.
34. Soderstrom, N. C., Bjork, R. A. *Learning Versus Performance: An Integrative Review*. Perspectives on Psychological Science 10(2), 2015.

35. Thaler, D. G., Ravishankar, C. V. *Using Name-Based Mappings to Increase Hit Rates*. IEEE/ACM Transactions on Networking 6(1), 1998.
36. Yuan, D., et al. *Simple Testing Can Prevent Most Critical Failures: An Analysis of Production Failures in Distributed Data-Intensive Systems*. OSDI, 2014.
37. Amazon Web Services [AWS 2017]. *Summary of the Amazon S3 Service Disruption in the Northern Virginia (US-EAST-1) Region*. 2017. https://aws.amazon.com/message/41926/

3.17 Open verification items

- [AWS 2017] URL and wording of the S3 post-event summary could not be fetched from this environment (network restriction); the summary in §3.9.3 is from the author's knowledge of the report and should be checked against the primary text.
- Roediger and Karpicke (2006) Experiment 2 one-week figures (~61 % STTT vs ~40 % SSSS) were confirmed only through secondary summaries; verify against the paper's Figure/Results.
- Macnamara et al. (2014): headline 12 % of variance confirmed; the domain ordering (lower in education and professions than in games, music, sports) is from memory and should be checked.
- Attribution of "What I cannot create, I do not understand" to Feynman's blackboard (Caltech Archives, 1988): widely documented; add an archival citation if one is required.
- Leitner intervals (1, 3, 7, 16, 35 days) are a teaching choice, not an empirically derived schedule; the text says so.
- The "~1/√V" load-spread scaling is stated as an approximation; it is now derived in §14.8.4 (UQ-012, resolved).

Chapter 4 — Quantitative Foundations for System Design

> **Level:** 1–3 · **Track:** Shared · **Estimated reading time:** 6 h

System design is often presented as a drawing exercise: boxes, arrows, a cache here, a queue there. The drawing is the easy part. What separates a design that works from one that merely looks plausible is *arithmetic*. How many machines? How much memory? What happens to latency when traffic doubles? How many nines does the whole system get when every component has three? How likely is it that all three replicas of a file die before you notice the first one is gone?

This chapter gives you the mathematical toolkit to answer those questions, assuming only high-school algebra. Each formula comes with an analogy, its units, a fully worked example and its engineering implication. Short Python programs check formulas against simulation — a habit worth keeping: never trust a formula you have not seen agree with reality at least once.

The tools are grouped into four clusters:

- **Number sense** (§4.3–§4.5): exponents, logarithms and units, the arithmetic of "how big" and "how fast".
- **Uncertainty** (§4.6–§4.9): probability, statistics, percentiles and the small amount of linear algebra you need before the ML chapters.
- **Performance laws** (§4.10–§4.13): queueing theory, Little's Law, Amdahl's and Gustafson's laws and the Universal Scalability Law.
- **Dependability** (§4.14–§4.16): availability, reliability and durability arithmetic.

The chapter closes with the latency numbers that anchor estimation (§4.17) and with the six quantitative relationships that recur throughout the rest of the book (§4.18).

4.1 Learning objectives

After this chapter you should be able to:

- Convert fluently between powers of two and powers of ten, and between bits, bytes, SI prefixes and binary prefixes, stating which convention you use.
- Use logarithms to explain why binary search and B-trees need so few steps, and compute the height of a B-tree for a given fan-out and data size.

- Calculate the probability that at least one of N components fails, the expected number of failures, and the binomial probability of exactly k failures.
- Compute mean, median, variance, standard deviation and percentiles from a sample, and explain why the mean is misleading for heavy-tailed latency distributions.
- Quantify tail-latency amplification under fan-out using $P(\text{all fast}) = p^N$.
- Compute a dot product, a cosine similarity and the floating-point cost of a matrix multiplication.
- Apply the M/M/1 formulas to predict how latency grows with utilization, and explain why a shared queue beats separate queues.
- Size thread pools and database connection pools with Little's Law.
- Predict the limits of parallel speedup with Amdahl's Law, Gustafson's Law and the Universal Scalability Law.
- Compute the availability of components in series, in parallel and in k-of-n configurations, and convert availability targets to downtime budgets.
- Relate MTBF, MTTR, failure rate and availability, and estimate the annual probability of data loss for a replicated store.
- Use dated latency reference numbers to sanity-check designs, and reason quantitatively about six recurring trade-offs.

4.2 Prerequisites

- High-school algebra: fractions, exponents, solving a linear equation — every formula in the chapter is built from these.
- The vocabulary of latency, throughput and availability from §1.7 (Non-functional requirements) — this chapter makes those words measurable.
- Orders-of-magnitude reasoning from §2.10 (Reasoning about scale) — helpful but not required; this chapter supplies the underlying mathematics.
- A Python 3 interpreter (3.10 or later) if you want to run the code examples. All examples use only the standard library.

4.3 Exponents, powers of two and powers of ten

4.3.1 What an exponent is

DEFINITION

Definition — Exponent. In b^n, the **base** b is multiplied by itself n times; n is the **exponent** (or power). By convention $b^0 = 1$ and $b^{-n} = 1/b^n$.

First-principles explanation

Imagine folding a sheet of paper in half repeatedly. After one fold it is 2 layers thick, after two folds 4 layers, after three folds 8. Each fold *doubles* the count, so after n folds there are 2^n layers. After 10 folds you have 1,024 layers; after 20 folds, about a million. Exponential growth is the arithmetic of repeated multiplication, and computers are full of it, because every extra bit in an address or identifier doubles the number of values it can represent.

Three rules do almost all the work in estimation:

FORMULA

Laws of exponents

$$b^m \times b^n = b^{m+n} \qquad \frac{b^m}{b^n} = b^{m-n} \qquad (b^m)^n = b^{mn}$$

Symbol	Meaning	Units
b	Base (2 for binary quantities, 10 for decimal)	dimensionless
m, n	Exponents	dimensionless

WORKED EXAMPLE

Worked example — Multiplying large numbers by adding exponents. A service handles 2×10^4 requests per second (RPS). How many requests is that per day?

1. Seconds per day: $86,400 \approx 8.64 \times 10^4$.
2. Multiply the mantissas: $2 \times 8.64 = 17.28$.
3. Add the exponents: $10^4 \times 10^4 = 10^8$.
4. Result: $17.28 \times 10^8 = 1.728 \times 10^9$ requests per day.

Interpretation. About 1.7 billion requests per day. **Engineering implication.** If each request writes a 1 KB log line, that is $1.7 \times 10^9 \times 10^3 = 1.7 \times 10^{12}$ bytes, roughly 1.7 TB of logs per day — a number that immediately tells you logs need their own retention and sampling plan.

4.3.2 Powers of two

Memory addresses, identifiers, hash spaces and page sizes are all powers of two because hardware is built from binary switches. The key coincidence that makes mental arithmetic possible is

$$2^{10} = 1,024 \approx 10^3.$$

Every additional factor of 2^{10} is therefore “about another factor of a thousand”.

Table 4.1 — Powers of two that every engineer should recognize.

Power	Exact value	Approximately	Binary prefix	Typical meaning
2^{10}	1,024	one thousand (10^3)	kibi (Ki)	1 KiB of memory
2^{16}	65,536	65 thousand	—	Number of TCP/UDP port numbers; max of a 16-bit unsigned integer + 1
2^{20}	1,048,576	one million (10^6)	mebi (Mi)	1 MiB
2^{30}	1,073,741,824	one billion (10^9)	gibi (Gi)	1 GiB
2^{32}	4,294,967,296	4.3 billion	—	IPv4 address space; 32-bit ID space
2^{40}	1,099,511,627,776	one trillion (10^{12})	tebi (Ti)	1 TiB
2^{50}	1,125,899,906,842,624	one quadrillion (10^{15})	pebi (Pi)	1 PiB
2^{60}	1,152,921,504,606,846,976	one quintillion (10^{18})	exbi (Ei)	1 EiB
2^{64}	18,446,744,073,709,551,616	1.8×10^{19}	—	64-bit ID or hash space

Notice that the approximation drifts: 2^{10} is 2.4% larger than 10^3, but 2^{40} is 10% larger than 10^{12} and 2^{60} is 15% larger than 10^{18}. For estimation this is fine; for billing or capacity contracts it is not (see §4.5.2).

WORKED EXAMPLE

Worked example — How long until a 32-bit ID runs out? A table uses a 32-bit unsigned integer as its auto-increment primary key and receives 10,000 inserts per second.

1. ID space: $2^{32} = 4,294,967,296$ values.
2. Seconds to exhaust: $4,294,967,296 / 10,000 = 429,497$ s.
3. Convert to days: $429,497 / 86,400 \approx 4.97$ days.

Interpretation. Under five days. Even at 100 inserts per second the space lasts only about 497 days. **Engineering implication.** Use 64-bit identifiers for any table that grows without bound; at 10,000 inserts per second a 64-bit space lasts $1.8 \times 10^{19} / 10^4 \approx 1.8 \times 10^{15}$ s, roughly 58 million years. Unique ID generation is covered in §17.10.

4.3.3 Powers of ten and scientific notation

Scientific notation writes a number as a mantissa between 1 and 10 times a power of ten: 86,400 becomes 8.64×10^4. In estimation you usually round the mantissa to one significant figure and keep only the exponent honest. Being off by a factor of 2 rarely changes a design decision; being off by a factor of 1,000 always does. The **order of magnitude** of a number is its power of ten, and most back-of-the-envelope reasoning (§2.11, Chapter 89) is order-of-magnitude reasoning.

4.4 Logarithms

4.4.1 What a logarithm is

DEFINITION

Definition — Logarithm. The logarithm base b of x, written $\log_b x$, is the exponent to which b must be raised to give x: if $b^y = x$ then $y = \log_b x$.

First-principles explanation

A logarithm answers the question “how many times do I have to multiply (or divide) to get there?”. Go back to the folded paper: if you want 1,024 layers, how many folds? $\log_2 1024 = 10$. Turn it around: if you have a phone book of 1,024 pages and open it in the middle, then in the middle of the correct half, and so on, how many openings until you reach one page? Also 10. A logarithm counts *halvings*. That is the reason logarithms appear wherever a data structure repeatedly cuts the search space: binary search, balanced trees, B-trees, skip lists, consistent-hash rings with binary-searched tokens.

Why does this exist?

Data sets grow multiplicatively (thousands, millions, billions), and the cost of searching them should not. Logarithms convert the multiplicative growth of n into the additive growth of the step count: multiplying n by 1,000 adds only about 10 steps to a binary search.

4.4.2 Rules and change of base

FORMULA

Logarithm rules and change of base

$$\log_b(xy) = \log_b x + \log_b y \qquad \log_b(x^k) = k \log_b x \qquad \log_b x = \frac{\ln x}{\ln b} = \frac{\log_{10} x}{\log_{10} b}$$

Symbol	Meaning	Units
b	Base of the logarithm ($b > 0$, $b \neq 1$)	dimensionless
x, y	Positive numbers	dimensionless
k	Any real exponent	dimensionless
$\ln$	Natural logarithm, base $e \approx 2.71828$	—

The change-of-base rule means you only need one logarithm on your calculator. Two handy constants: $\log_2 10 \approx 3.32$ and $\log_{10} 2 \approx 0.301$.

WORKED EXAMPLE

Worked example — Binary search over one billion keys. How many comparisons does binary search need to find a key among $n = 10^9$ sorted keys?

1. We need $\log_2(10^9)$.
2. Change of base: $\log_2(10^9) = 9 \times \log_2 10 = 9 \times 3.32 = 29.9$.
3. Round up: 30 comparisons.

Interpretation. Thirty comparisons to search a billion items; a linear scan needs on average 500 million. **Engineering implication.** Each comparison in RAM costs on the order of 100 ns if it misses the CPU cache (§4.17), so 30 comparisons take roughly 3 µs. A sorted array or balanced tree in memory is fast enough for almost any lookup workload; the problem only becomes hard when each step requires a disk read, which is exactly the problem B-trees solve.

4.4.3 B-tree fan-out: why databases need only three or four disk reads

A binary tree halves the search space at each node, so a billion keys need about 30 levels. If each level is a random disk read, 30 reads is far too slow. A **B-tree** (detailed in §11.8 and §13.6) instead stores hundreds of keys per node, sized to fill one disk page, so each node splits the search space into hundreds of ranges [Bayer and McCreight 1972]. The number of children per node is the **fan-out** f.

FORMULA

Height of a B-tree

$$h = \lceil \log_f n \rceil = \lceil \frac{\log n}{\log f} \rceil$$

Symbol	Meaning	Units
h	Number of levels from root to leaf (page reads per lookup if nothing is cached)	levels
n	Number of keys indexed	keys
f	Fan-out: child pointers per internal node	children/node
$\lceil \cdot \rceil$	Ceiling: round up to the next integer	—

The derivation is the same counting argument as the paper folds: one root reaches f children, which reach f^2 grandchildren, so h levels reach f^h leaves. We need $f^h \geq n$, so $h \geq \log_f n$.

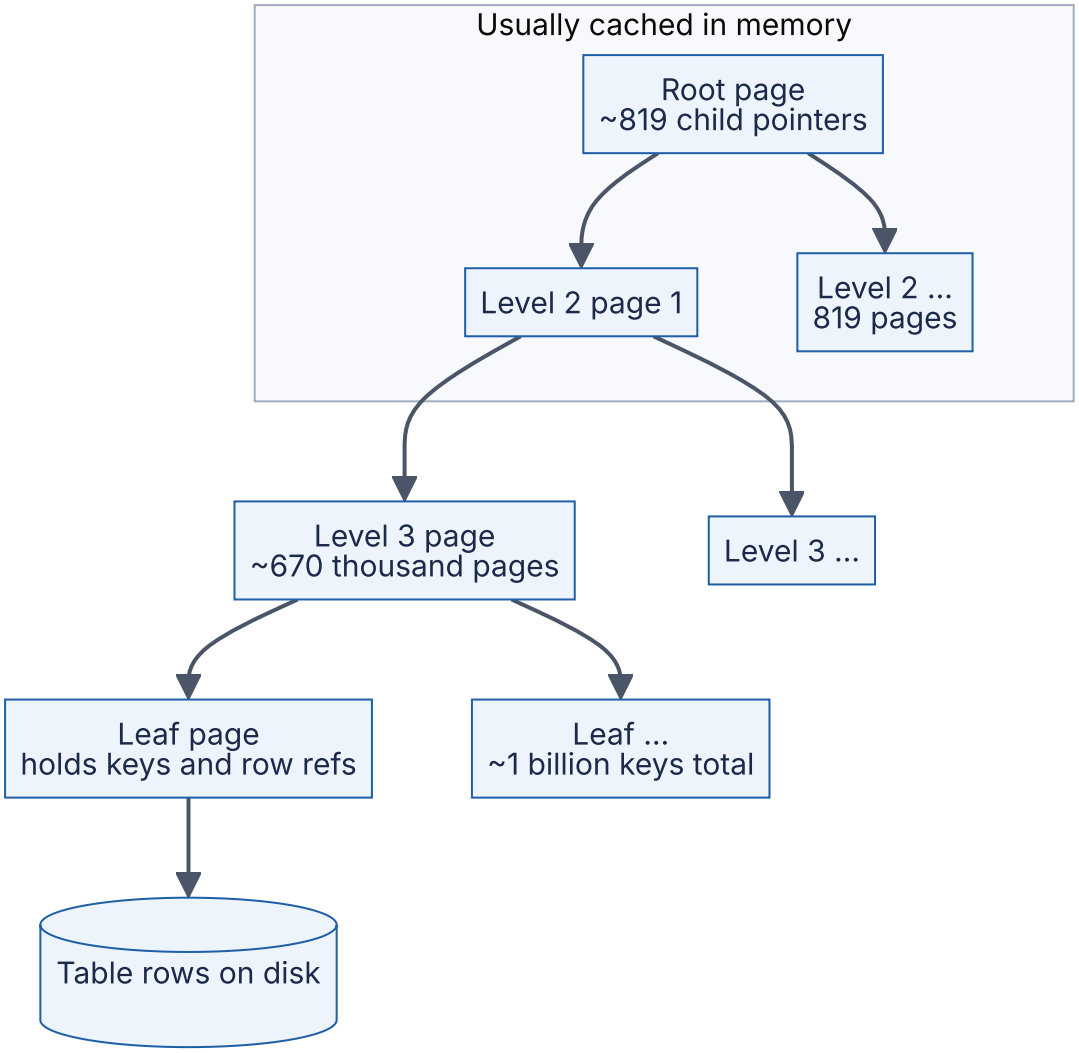
WORKED EXAMPLE

Worked example — Fan-out of a 16 KiB page. A database uses 16 KiB pages (16,384 bytes). Each index entry in an internal node is an 8-byte key plus an 8-byte child pointer = 16 bytes. Assume 20% of each page goes to headers and free space. How tall is the index on 10^9 keys?

1. Usable bytes per page: $16,384 \times 0.8 = 13,107$ bytes.
2. Fan-out: $f = 13,107/16 \approx 819$ children per node.
3. Height: $\log_{819}(10^9) = \frac{\log_{10} 10^9}{\log_{10} 819} = \frac{9}{2.913} = 3.09$.
4. Round up: $h = 4$ levels.
5. Compare: with $f = 1,000$ the height would be exactly $9/3 = 3$; a binary tree needs 30.

Interpretation. Four page reads locate any one of a billion keys. **Engineering implication.** The top two levels contain only $1 + 819 = 820$ pages, about 13 MiB, which fits easily in the buffer pool (§13.4). In practice a point lookup therefore costs one or two real I/Os, around 0.1–0.2 ms on an NVMe SSD (§4.17). Larger keys shrink fan-out: a 100-byte string key gives $f \approx 13,107/108 \approx 121$ and $h = \lceil 9/2.083 \rceil = 5$. Short keys are an index-design decision with measurable latency consequences.

Figure 4.1 — B-tree fan-out turns a billion keys into four levels.



Reading the diagram. Each rectangle is one fixed-size page. A lookup flows top-down synchronously: read the root, binary-search its keys in memory, follow one child pointer, repeat. The number of pages per level multiplies by the fan-out (1 → 819 → about 670,000 → leaves), so the count of *levels* grows only logarithmically. The `Cached` subgraph marks the storage boundary that matters: pages above it are in RAM and cost nanoseconds to microseconds, pages below it may require a disk read. The failure point is a cold cache after a restart, when every level costs an I/O. Scaling the data by another factor of 819 adds only one level.

COMMON MISTAKE

Mistake — "Logarithmic means constant, so the base doesn't matter." Thirty dependent SSD reads at ~100 µs is 3 ms; four is 0.4 ms. The fan-out changes the constant by an order of magnitude.

4.5 Units: bits, bytes, rates and prefixes

4.5.1 Bits versus bytes

DEFINITION

Definition — Bit and byte. A **bit** (symbol b) is a single binary digit, 0 or 1. A **byte** (symbol B) is 8 bits. Storage and memory are conventionally measured in bytes; network link speeds are conventionally measured in bits per second (bps).

The lowercase/uppercase distinction is load-bearing: 1 Gbps (gigabit per second) is one-eighth of 1 GB/s (gigabyte per second). Mixing them up produces an error of exactly 8×, which is large enough to invalidate a design and small enough to escape notice.

FORMULA

Converting link speed to byte throughput

$$R_{\text{B/s}} = \frac{R_{\text{bps}}}{8} \qquad t_{\text{transfer}} = \frac{8 \times S}{R_{\text{bps}}}$$

Symbol	Meaning	Units
R_{bps}	Link rate	bits/second
$R_{\text{B/s}}$	The same rate expressed in bytes	bytes/second
S	Amount of data to transfer	bytes
t_{transfer}	Ideal transfer time, ignoring protocol overhead and congestion	seconds

WORKED EXAMPLE

Worked example — Copying 1 TB across a 1 Gbps link.

1. Convert data to bits: $1\ \text{TB} = 10^{12}$ bytes $= 8 \times 10^{12}$ bits.
2. Divide by the rate: $8 \times 10^{12} / 10^{9} = 8,000$ s.
3. Convert to hours: $8,000 / 3,600 \approx 2.2$ h.
4. At 10 Gbps: 800 s ≈ 13 min.

Interpretation. Even a "fast" link takes hours to move a terabyte; real transfers are slower because TCP/IP headers, TLS and congestion reduce goodput, commonly by 5–10% or more. **Engineering implication.** Rebuilding a failed 20 TB storage node over a single 10 Gbps link takes at least $8 \times 2 \times 10^{13} / 10^{10} = 16,000$ s, about 4.4 hours. That repair window feeds directly into the durability calculations of §4.16, and it is why distributed stores spread re-replication across many source nodes in parallel.

4.5.2 SI versus binary prefixes

Two systems of prefixes coexist. The **SI (decimal) prefixes** k, M, G, T, P, E mean $10^3, 10^6, 10^9, \ldots$ exactly. The **binary prefixes** Ki, Mi, Gi, Ti, Pi, Ei, standardized by the IEC (now in IEC 80000-13) [IEC 80000-13], mean $2^{10}, 2^{20}, 2^{30}, \ldots$ exactly. Operating systems, memory vendors and disk vendors have historically used "GB" for both, which causes the classic "missing" capacity on a new drive.

Table 4.2 — SI versus binary prefixes.

SI name	SI value	Binary name	Binary value	Binary/SI ratio
kilobyte (KB or kB)	10^3	kibibyte (KiB)	2^{10}	1.024
megabyte (MB)	10^6	mebibyte (MiB)	2^{20}	1.049
gigabyte (GB)	10^9	gibibyte (GiB)	2^{30}	1.074
terabyte (TB)	10^{12}	tebibyte (TiB)	2^{40}	1.100
petabyte (PB)	10^{15}	pebibyte (PiB)	2^{50}	1.126
exabyte (EB)	10^{18}	exbibyte (EiB)	2^{60}	1.153

Book convention. Following `terminology.md`, this book uses KB/MB/GB/TB $= 10^3/10^6/10^9/10^{12}$ bytes for estimation and KiB/MiB/GiB when power-of-two precision matters (page sizes, memory allocation). Network rates are always decimal: 1 Gbps $= 10^9$ bits/s.

WORKED EXAMPLE

Worked example — The "missing" 7% on a new drive. A drive sold as 1 TB is shown by an operating system as "931 GB".

1. The vendor means SI: 10^{12} bytes.
2. The operating system divides by 2^{30} but labels the result "GB": $10^{12} / 1,073,741,824 = 931.3$.
3. So the drive holds 931.3 GiB, and nothing is missing.

Interpretation. A 6.9% labeling discrepancy at the tera scale, growing to 12.6% at peta and 15.3% at exa. **Engineering implication.** In a capacity plan for 5 PB of data, confusing PB with PiB means under-provisioning by $5 \times 0.126 = 0.63$ PB. State your units in every capacity document.

4.5.3 Rates and time conversions

Most capacity estimates start from a daily or monthly volume and must become a per-second rate, because servers are sized per second.

Table 4.3 — Time conversions used in estimation.

Period	Seconds (exact)	Handy approximation
1 minute	60	—
1 hour	3,600	3.6×10^3
1 day	86,400	$\approx 10^5$ (overestimates seconds by 16%)
1 week	604,800	$\approx 6 \times 10^5$
1 month (30 days)	2,592,000	$\approx 2.6 \times 10^6$
1 year (365 days)	31,536,000	$\approx \pi \times 10^7$

FORMULA

Average and peak rate from a daily volume

$$\bar{\lambda} = \frac{V_{\text{day}}}{86,400} \qquad \lambda_{\text{peak}} = k \times \bar{\lambda}$$

Symbol	Meaning	Units
V_{day}	Events (requests, writes, messages) per day	events/day
$\bar{\lambda}$	Average rate	events/second
k	Peak-to-average ratio, measured or assumed (often 2–10 for consumer traffic)	dimensionless
λ_{peak}	Rate the system must be sized for	events/second

WORKED EXAMPLE

Worked example — 100 million requests per day.

1. Average: $10^8 / 86,400 = 1,157$ RPS.
2. Shortcut check: $10^8 / 10^5 = 1,000$ RPS (16% low, acceptable for a first pass).
3. Assume a peak-to-average ratio $k = 3$: $\lambda_{\text{peak}} \approx 3,500$ RPS.

Interpretation. “100 million a day” sounds enormous but is a few thousand requests per second at peak. **Engineering implication.** A few application servers can carry this request rate; the harder questions are usually storage growth and tail latency, not raw request rate. The peak factor k is an assumption that must be stated and, once the system is live, measured.

4.6 Probability essentials

4.6.1 Probability, complement and independence

DEFINITION

Definition — Probability. A number between 0 and 1 that measures how likely an event is: 0 means impossible, 1 means certain. If an experiment is repeated many times, the probability of event A, written $P(A)$, is the long-run fraction of repetitions in which A happens.

First-principles explanation

Think of a fleet as a bag of marbles: if 1 in 1,000 is red, a blind draw is red with probability 0.001. Probability in system design is about the *fleet*: a disk with a 2% annual failure probability is rare on its own and a daily event among 100,000 disks.

Two rules carry most of the weight. The **complement rule**: $P(\text{not } A) = 1 - P(A)$; computing "nothing goes wrong" is often far easier than enumerating what can. **Independence**: A and B are independent if knowing one happened tells you nothing about the other; then, and *only* then, $P(A \text{ and } B) = P(A) \times P(B)$. Disks in different datacenters fail nearly independently; disks sharing a power supply do not.

FORMULA

Probability that at least one of N independent components fails

$$P(\text{at least one fails}) = 1 - (1 - p)^N$$

Symbol	Meaning	Units
p	Probability that a single component fails during the period of interest	dimensionless (0–1)
N	Number of independent components	count
$(1 - p)^N$	Probability that *none* fails (complement, multiplied N times by independence)	dimensionless

Derivation. One component survives with probability $1 - p$. By independence, all N survive with probability $(1 - p) \times (1 - p) \times \cdots = (1 - p)^N$. "At least one fails" is the complement of "all survive". A useful approximation for small p: $(1 - p)^N \approx e^{-Np}$, so $P(\text{at least one}) \approx 1 - e^{-Np}$, and when $Np \ll 1$ this is simply $\approx Np$.

WORKED EXAMPLE

Worked example — At least one of 1,000 servers fails today. Each server independently fails on a given day with probability $p = 0.001$ (one failure per thousand server-days). The fleet has $N = 1,000$ servers.

1. Probability one server survives the day: $1 - 0.001 = 0.999$.
2. Probability all survive: 0.999^{1000}. Using the approximation: $e^{-1000 \times 0.001} = e^{-1} = 0.368$. (Exact: 0.3677.)
3. Probability at least one fails: $1 - 0.368 = 0.632$.

Interpretation. A component that almost never fails individually produces a failure on 63% of days at fleet scale. At $N = 10,000$ the probability is $1 - e^{-10} > 0.9999$: a failure *every* day. **Engineering implication.** At scale, failure is not an exception to handle but a steady background rate to design for. This is the quantitative core of "design for failure" (§2.9): automation must detect and replace failed nodes without human intervention, because humans cannot be paged every day for routine events.

4.6.2 Union and intersection

FORMULA

Union (inclusion-exclusion) and intersection

$$P(A \cup B) = P(A) + P(B) - P(A \cap B) \qquad P(A \cap B) = P(A) \times P(B \mid A)$$

Symbol	Meaning	Units
$P(A \cup B)$	Probability that A or B (or both) happens	dimensionless
$P(A \cap B)$	Probability that both happen	dimensionless
$P(B \mid A)$	Conditional probability of B given that A happened; equals $P(B)$ when independent	dimensionless

The subtraction in the union rule avoids counting "both" twice. The conditional form of the intersection rule is how correlation enters: if two replicas share a rack, $P(B \mid A)$ — the probability that the second fails *given* that the first did — can be many times larger than $P(B)$.

WORKED EXAMPLE

Worked example — Two dependencies, independent versus correlated. A request needs both a cache and a database. On a given day, the cache has an outage with probability 0.01 and the database with probability 0.02.

1. If independent: $P(\text{both}) = 0.01 \times 0.02 = 0.0002$.
2. $P(\text{either}) = 0.01 + 0.02 - 0.0002 = 0.0298$.
3. Now suppose they share a network switch, so that given a cache outage the database is also out with probability 0.5: $P(\text{both}) = 0.01 \times 0.5 = 0.005$ — 25 times the independent value.

Interpretation. The probability of "something breaks" barely changes, but the probability of "both break together" rises 25×. **Engineering implication.** Redundancy arithmetic (§4.14, §4.16) assumes independence. Every shared dependency — a rack, a power feed, a software version, a configuration push — breaks that assumption, which is why real systems spread replicas across failure domains.

4.6.3 Expected value

DEFINITION

Definition — Expected value. The probability-weighted average outcome of a random quantity X: $E[X] = \sum_i x_i \, P(X = x_i)$. It is the long-run average over many repetitions.

FORMULA

Expected value and linearity of expectation

$$E[X] = \sum_i x_i \, P(X = x_i) \qquad E[X_1 + X_2 + \cdots + X_N] = E[X_1] + \cdots + E[X_N]$$

Symbol	Meaning	Units
X	A random quantity (failures, latency, cost)	units of the quantity
x_i	A possible value of X	units of the quantity
$P(X = x_i)$	Probability of that value	dimensionless

Linearity is powerful because it holds *even when the components are not independent*. The expected number of failed servers is $N \times p$ whatever the correlation.

WORKED EXAMPLE

Worked example — Expected cost of a retry policy. A call fails with probability 0.05 and is retried once on failure. What is the expected number of calls per request?

1. Outcomes: 1 call with probability 0.95; 2 calls with probability 0.05.
2. $E[\text{calls}] = 1 \times 0.95 + 2 \times 0.05 = 0.95 + 0.10 = 1.05$.

Interpretation. Retries add 5% load in normal operation. But if a dependency is degraded so that 50% of calls fail, the same policy gives $1 \times 0.5 + 2 \times 0.5 = 1.5$ calls per request — a 50% load increase on the struggling dependency. **Engineering implication.** Retries amplify load precisely when the system is weakest. Retry budgets (§22.12) cap this expected amplification.

4.6.4 The binomial distribution

When there are N independent trials, each "failing" with probability p, the number of failures K follows a **binomial distribution**.

FORMULA

Binomial probability of exactly k failures

$$P(K = k) = \binom{N}{k} p^k (1-p)^{N-k}, \qquad \binom{N}{k} = \frac{N!}{k!\,(N-k)!}, \qquad E[K] = Np$$

Symbol	Meaning	Units
N	Number of independent components (trials)	count
k	Number of failures we ask about	count
p	Per-component failure probability in the period	dimensionless
$\binom{N}{k}$	Number of ways to choose which k components fail ("N choose k")	count
$E[K]$	Expected number of failures	count

Derivation. One specific set of k failures and $N-k$ survivals has probability $p^k(1-p)^{N-k}$ by independence. There are $\binom{N}{k}$ such sets, and they are mutually exclusive, so we add them.

WORKED EXAMPLE

Worked example — How many spare servers are enough? Continuing the fleet of $N = 1,000$ servers with daily failure probability $p = 0.001$:

1. Expected failures per day: $E[K] = 1000 \times 0.001 = 1$.
2. $P(K = 0) = 0.999^{1000} = 0.3677$.
3. $P(K = 1) = 1000 \times 0.001 \times 0.999^{999} = 0.3681$.
4. $P(K = 2) = \binom{1000}{2} \times 0.001^2 \times 0.999^{998} = 499,500 \times 10^{-6} \times 0.3685 = 0.1840$.
5. $P(K \geq 3) = 1 - 0.3677 - 0.3681 - 0.1840 = 0.0802$.

Interpretation. On about 8% of days — roughly 29 days per year — three or more servers fail. **Engineering implication.** Provisioning spare capacity for "the average day" (one failure) leaves you short on one day in twelve. Capacity headroom should be chosen against a percentile of the failure distribution, not its mean.

4.7 Statistics: describing measurements

Probability reasons forward from a model to predictions. **Statistics** reasons backward from measurements (latency samples, failure logs) to a description of what is happening.

4.7.1 Mean and median

DEFINITION

Definition — Mean and median. The **mean** (arithmetic average) of n values is their sum divided by n. The **median** is the middle value after sorting (the average of the two middle values when n is even); half of the values are at or below it.

First-principles explanation

If nine people in a café each earn 50,000 a year and a billionaire walks in, the mean income in the room jumps to over 100 million while the median does not move. The mean is sensitive to every value, including extreme ones; the median is sensitive only to rank. Latency data behave like the café: most requests are quick, a few are extremely slow, and the slow ones drag the mean.

FORMULA

Mean, variance and standard deviation

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \sigma^2 = \frac{1}{n}\sum_{i=1}^{n} (x_i - \bar{x})^2 \qquad \sigma = \sqrt{\sigma^2}$$

Symbol	Meaning	Units
x_i	The i-th measurement	e.g. ms
n	Number of measurements	count
$\bar{x}$	Mean	same as x (ms)
σ^2	Variance: average squared distance from the mean	ms²
σ	Standard deviation: typical distance from the mean	ms

This is the *population* variance (divide by n). When estimating the variance of a larger population from a sample, statisticians divide by $n - 1$ instead (the *sample* variance); for the large samples typical of latency monitoring the difference is negligible.

WORKED EXAMPLE

Worked example — Ten latency measurements with one outlier. Measured latencies in ms: 12, 15, 11, 14, 13, 16, 12, 18, 14, 250.

1. Sum: $12 + 15 + 11 + 14 + 13 + 16 + 12 + 18 + 14 + 250 = 375$. Mean: $375/10 = 37.5$ ms.
2. Sorted: 11, 12, 12, 13, 14, 14, 15, 16, 18, 250. Median: average of 5th and 6th values $= (14 + 14)/2 = 14$ ms.
3. Squared deviations from 37.5: $(12 - 37.5)^2 = 650.25$; similarly 506.25, 702.25, 552.25, 600.25, 462.25, 650.25, 380.25, 552.25, and $(250 - 37.5)^2 = 45,156.25$. Sum $= 50,212.5$.
4. Variance: $50,212.5/10 = 5,021.25$ ms². Standard deviation: $\sqrt{5021.25} = 70.9$ ms.
5. Without the outlier: mean 13.9 ms, median 14 ms, standard deviation 2.1 ms.

Interpretation. One slow request in ten makes the mean 2.7 times the median and inflates the standard deviation 34-fold. The mean (37.5 ms) describes *no actual request*: nine were under 20 ms and one was 250 ms. **Engineering implication.** Report latency as percentiles (§4.8), not as mean ± standard deviation. Mean and standard deviation summarize a distribution well only when it is roughly symmetric and thin-tailed, which latency almost never is.

4.7.2 Distributions: normal, exponential, log-normal and Pareto

A **probability distribution** describes how likely each value is. Four shapes cover most of what you will meet.

The **normal (Gaussian) distribution** is the symmetric bell curve. It arises when many small independent effects add up. About 68% of values lie within 1σ of the mean, 95% within 2σ and 99.7% within 3σ. Extreme values are vanishingly rare: a 6σ event has probability around 10^{-9}.

The **exponential distribution** describes the time between independent random events occurring at a constant average rate λ, such as request arrivals from many independent users, or failures of a component that does not age. Its mean is $1/\lambda$ and $P(T > t) = e^{-\lambda t}$. It is **memoryless**: having waited 5 seconds tells you nothing about how much longer you will wait. It underpins queueing theory (§4.10) and reliability theory (§4.15).

The **log-normal distribution** describes a quantity whose *logarithm* is normal — the result of many small *multiplicative* effects. Request latency often looks roughly log-normal: a long right tail, with the mean above the median. If $\ln X$ has mean μ and standard deviation s, then the median of X is e^{μ} and the mean is $e^{\mu + s^2/2}$.

The **Pareto (power-law) distribution** has $P(X > x) = (x_m/x)^{\alpha}$ for $x \geq x_m$. Its tail decays polynomially, not exponentially, so extreme values are far more common than a normal model predicts. File sizes, object popularity, and per-user activity are frequently described as heavy-tailed (Harchol-Balter 2013 discusses heavy-tailed job sizes in depth). With $\alpha \approx 1.16$ you get the “80/20 rule”: the top 20% of items account for 80% of the total. For $\alpha \leq 1$ the mean is infinite; for $\alpha \leq 2$ the variance is infinite, so sample averages never settle down.

FORMULA

Tail formulas for three skewed distributions

Exponential: $P(T > t) = e^{-\lambda t}$, $E[T] = \frac{1}{\lambda}$ Log-normal: median = e^{μ}, mean = $e^{\mu + s^2/2}$ Pareto: $P(X >$

Symbol	Meaning	Units
λ	Event rate of the exponential distribution	events/second
t	Elapsed time	seconds
μ, s	Mean and standard deviation of $\ln X$ for a log-normal X	ln of the unit of X
x_m	Pareto minimum (scale) value	unit of X (e.g. bytes)
α	Pareto tail index: smaller means heavier tail	dimensionless

Table 4.4 — Four distributions and where they appear in systems.

Distribution	Shape	Tail	Typical system quantity	Mean vs median
Normal	Symmetric bell	Very thin (e^{-x^2})	Aggregates of many small effects; measurement noise	Equal
Exponential	Decaying from zero	Thin (e^{-x})	Inter-arrival times; failures of non-aging parts	Mean ≈ 1.44 × median
Log-normal	Right-skewed	Moderately heavy	Request latency, service times	Mean > median
Pareto	Right-skewed, power law	Heavy (polynomial)	File sizes, key popularity, user activity	Mean ≫ median (may be infinite)

WORKED EXAMPLE

Worked example — Mean versus median for a log-normal latency. Suppose $\ln(\text{latency in ms})$ has mean $\mu = \ln 20 = 2.996$ and standard deviation $s = 1.0$.

1. Median: $e^{\mu} = 20$ ms.
2. Mean: $e^{\mu + s^2/2} = e^{2.996 + 0.5} = 20 \times e^{0.5} = 20 \times 1.649 = 33.0$ ms.
3. The 99th percentile of a normal variable is $\mu + 2.326\,s$, so p99 $= e^{2.996 + 2.326} = 20 \times e^{2.326} = 20 \times 10.24 = 205$ ms.

Interpretation. The p99 is about 10 times the median, and the mean sits 65% above the median. **Engineering implication.** If you set a timeout at the mean (33 ms) you will time out nearly one request in three: $\ln 33 - \ln 20 = 0.50$, which is $0.5\,s$ above μ, and a normal variable exceeds $\mu + 0.5\,s$ with probability 0.31. The Python example in §4.8.2 samples this exact distribution and confirms these values empirically.

4.7.3 Why averages lie

Averages mislead in four recurring ways: **skew** (the mean is pulled toward the tail, as above); **time aggregation** (a day-long average of 50 ms can hide a 10-minute window at 2 s); **averaging percentiles** (a fleet p99 is not the mean of per-host p99s — merge histograms instead, §4.8.3); and **averaging over users** (if 1% of users send 50% of requests, per-request and per-user averages can tell opposite stories).

COMMON MISTAKE

Mistake — "Average latency is 40 ms, so we meet our 100 ms SLO." An SLO is about the fraction of requests that are fast enough. A mean of 40 ms is compatible with 10% of requests taking over a second. SLOs are written in percentiles (§39.6) for exactly this reason.

4.8 Percentiles and tail latency

4.8.1 Definition

DEFINITION

Definition — Percentile. The q-th percentile of a set of measurements is the value at or below which q% of the measurements fall. Latency percentiles are written **p50** (the median), **p95**, **p99** and **p99.9** (also written p999): the 50th, 95th, 99th and 99.9th percentiles.

First-principles explanation

Line up 1,000 runners by finishing time: the p50 is the 500th runner's time, the p99 the 990th's, the p99.9 the 999th's. Percentiles describe what a *fraction of requests* experiences — if your p99 is 2 s, one request in a hundred is painfully slow, and a page making 50 requests will often hit one (§4.8.4).

The region above roughly p99 is called the **tail**, and **tail latency** is the study of why those requests are slow and how to make them faster. Causes include garbage-collection pauses, queueing behind a large request, background compaction, cache misses, retransmitted packets and noisy neighbors [Dean and Barroso 2013].

4.8.2 Computing percentiles from samples

The simplest definition is the **nearest-rank method**.

FORMULA

Nearest-rank percentile

$$r = \lceil \frac{q}{100} \times n \rceil, \qquad \mathrm{p}q = x_{(r)}$$

Symbol	Meaning	Units
q	Desired percentile, $0 < q \le 100$	percent
n	Number of samples	count
r	1-based rank in the sorted list	position
$x_{(r)}$	The r-th smallest sample	e.g. ms

Other definitions interpolate between neighboring samples (NumPy's default does); they differ noticeably only when n is small. With 10 samples, a "p99" is just the maximum, which is why a p99 needs at least several hundred samples and a p99.9 several thousand to mean anything.

WORKED EXAMPLE

Worked example — Percentiles of the ten measurements from §4.7.1. Sorted: 11, 12, 12, 13, 14, 14, 15, 16, 18, 250.

1. p50: $r = \lceil 0.50 \times 10 \rceil = 5 \rightarrow 14$ ms.
2. p90: $r = \lceil 0.90 \times 10 \rceil = 9 \rightarrow 18$ ms.
3. p95: $r = \lceil 0.95 \times 10 \rceil = \lceil 9.5 \rceil = 10 \rightarrow 250$ ms.

Interpretation. p90 and p95 differ by a factor of 14 because with ten samples they land on different single observations. **Engineering implication.** Always report the sample count alongside a high percentile, and alert on percentiles computed over windows large enough to contain hundreds of tail samples.

The following program samples the log-normal distribution of §4.7.2, computes percentiles, and then checks the fan-out formula of §4.8.4 by simulation.

```
        import math
        import random
        import statistics

        def percentile_nearest_rank(samples, p):
            """Return the p-th percentile (0 < p <= 100) using the nearest-rank
method."""
            if not samples:
                raise ValueError("no samples")
            ordered = sorted(samples)
            rank = math.ceil(p / 100 * len(ordered))   # 1-based rank
            return ordered[max(rank, 1) - 1]

        random.seed(42)
        # Log-normal latencies: median ~20 ms, heavy right tail (sigma = 1.0)
        latencies_ms = [random.lognormvariate(math.log(20), 1.0) for _ in
range(100_000)]

        print(f"mean   = {statistics.fmean(latencies_ms):7.1f} ms")
        for p in (50, 90, 99, 99.9):
            print(f"p{p:<5} = {percentile_nearest_rank(latencies_ms, p):7.1f} ms")

        # Fan-out: a request waits for the slowest of N parallel calls
        p99 = percentile_nearest_rank(latencies_ms, 99)
        for n in (1, 10, 100):
            trials = 20_000
            slow = sum(
                max(random.choice(latencies_ms) for _ in range(n)) > p99
                for _ in range(trials)
            )
            print(f"N={n:>3}: simulated P(slow) = {slow/trials:.3f}, "
                  f"formula 1 - 0.99^N = {1 - 0.99**n:.3f}")
```

Output (Python 3.13):

```
mean   =    32.9 ms
p50    =    20.0 ms
p90    =    72.2 ms
p99    =   201.3 ms
p99.9  =   428.4 ms
N=  1: simulated P(slow) = 0.010, formula 1 - 0.99^N = 0.010
N= 10: simulated P(slow) = 0.093, formula 1 - 0.99^N = 0.096
N=100: simulated P(slow) = 0.637, formula 1 - 0.99^N = 0.634
```

What to notice:

- The simulated mean (32.9 ms), median (20.0 ms) and p99 (201 ms) match the closed-form values of §4.7.2 (33.0, 20 and 205 ms) within sampling error.
- `max(...)` models a request that must wait for *all* of its parallel sub-requests; the simulated probability of exceeding the single-server p99 matches $1 - 0.99^N$.
- Sorting 100,000 samples is fine offline, but production systems cannot keep every sample; they use histograms.

4.8.3 Histograms

A **histogram** counts how many samples fall into each of a fixed set of buckets. It uses constant memory regardless of request volume, and two histograms with identical buckets can be merged by adding counts, which makes fleet-wide percentiles possible. Percentiles are read from the cumulative counts, with an error no larger than the bucket width. Libraries such as HdrHistogram use buckets whose width grows with the value (logarithmic or log-linear buckets) so that *relative* error stays bounded across many orders of magnitude [Tene, HdrHistogram].

WORKED EXAMPLE

Worked example — p99 and p99.9 from a histogram. 10,000 requests are counted into buckets (ms): [0,10): 6,000; [10,20): 2,500; [20,50): 1,000; [50,100): 350; [100,200): 120; [200,500): 25; [500,1000): 5.

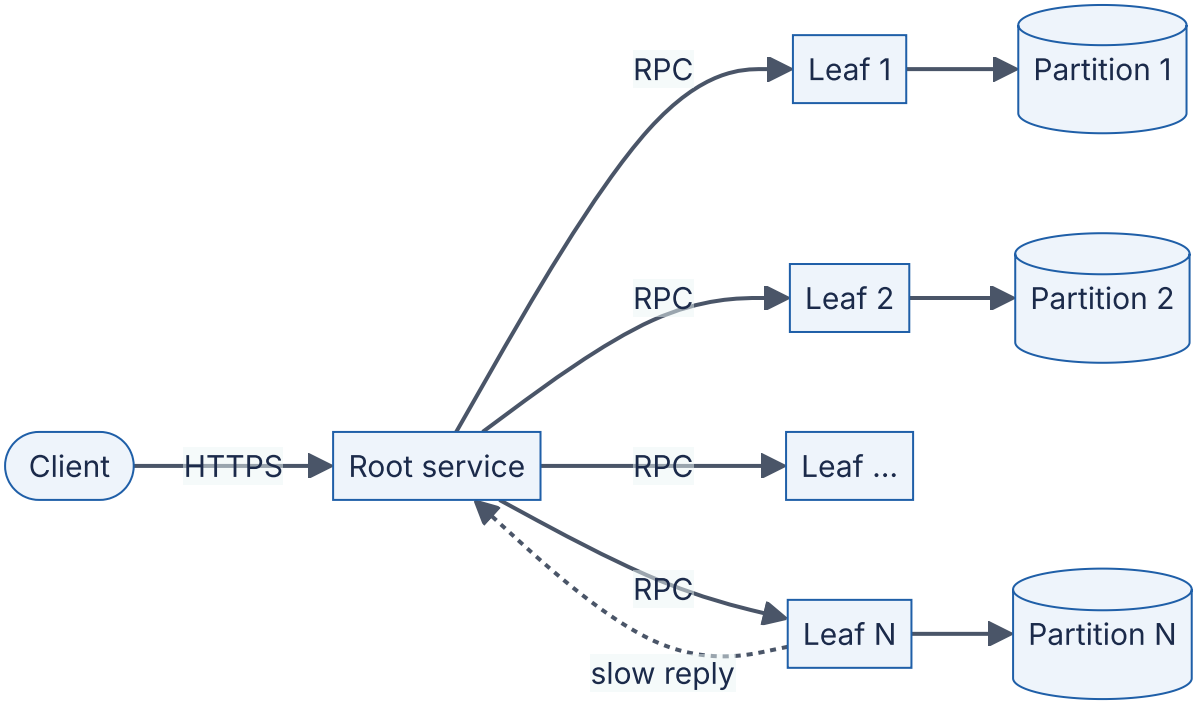
1. Cumulative counts: 6,000; 8,500; 9,500; 9,850; 9,970; 9,995; 10,000.
2. p99 is rank $0.99 \times 10,000 = 9,900$. The cumulative count first reaches 9,900 in the [100,200) bucket (9,850 → 9,970).
3. Interpolate linearly inside the bucket: $(9,900 - 9,850)/120 = 0.417$ of the way through, so p99 $\approx 100 + 0.417 \times 100 = 142$ ms.
4. p99.9 is rank 9,990, inside [200,500) (9,970 → 9,995): $(9,990 - 9,970)/25 = 0.8$, so p99.9 $\approx 200 + 0.8 \times 300 = 440$ ms.

Interpretation. The true p99 is somewhere in [100, 200) ms; interpolation assumes samples are spread evenly inside the bucket. **Engineering implication.** Bucket boundaries should be densest where your SLO thresholds lie. If the SLO is “p99 < 150 ms”, a bucket edge exactly at 150 ms lets you count SLO violations exactly rather than estimate them.

4.8.4 Fan-out amplification

Modern requests are rarely served by one machine. A search query may be scattered to hundreds of index partitions (§16.7); a page render may call dozens of microservices. If a request must wait for *all* of its *N* sub-requests, then it is slow whenever *any* of them is slow.

Figure 4.2 — Fan-out: the slowest leaf determines the response time.



Reading the diagram. The client sends one HTTPS request to a root service, which issues N parallel RPCs to leaf services, each owning one data partition (the cylinders mark storage boundaries). All arrows are synchronous request/response except the dotted arrow, which highlights the one leaf whose reply arrives late. The root cannot answer until every leaf replies (or a deadline fires), so the response time is the *maximum* of N leaf latencies. Every leaf is a failure point for the whole request. Scaling out by adding partitions increases N and therefore makes the tail *worse* unless tail-tolerant techniques are used.

FORMULA

Probability that a fan-out request is fast

$$P(\text{all } N \text{ fast}) = p^N \qquad P(\text{at least one slow}) = 1 - p^N \qquad p_{\text{required}} = P_{\text{target}}^{1/N}$$

Symbol	Meaning	Units
p	Probability that one leaf responds within the threshold (e.g. 0.99 at the leaf's p99)	dimensionless
N	Number of parallel sub-requests the root must wait for	count
P_{target}	Desired probability that the whole request is fast	dimensionless
p_{required}	Per-leaf probability needed to meet P_{target}	dimensionless

This is the complement rule of §4.6.1 again, assuming leaf latencies are independent.

WORKED EXAMPLE

Worked example — 100-way fan-out. Each leaf answers within 10 ms with probability 0.99 (its p99 is 10 ms). The root fans out to $N = 100$ leaves.

1. $P(\text{all fast}) = 0.99^{100} = 0.366$.
2. $P(\text{at least one slow}) = 1 - 0.366 = 0.634$.
3. For 99% of root requests to be fast we need $p = 0.99^{1/100} = 0.99990$: every leaf must meet the 10 ms threshold at its p99.99.
4. With $N = 10$: $0.99^{10} = 0.904$, so 9.6% of requests are slow.

Interpretation. A leaf p99 becomes the root's p37 or so: 63% of user requests see at least one slow leaf. This is the example Dean and Barroso use to open *The Tail at Scale*: with a one-in-a-hundred chance of a slow leaf, 63% of 100-way fan-out requests are slow [Dean and Barroso 2013]. **Engineering implication.** In fan-out architectures you must engineer the leaf p99.9 or p99.99, not the leaf p99, or use tail-tolerant techniques that break the p^N law.

Real-world industry applications

INDUSTRY EXAMPLE

Google — tail-tolerant techniques. Dean and Barroso describe how large fan-out services at Google tolerate latency variability instead of trying to eliminate it. **Hedged requests** send a second copy of a request to another replica if the first has not answered within, for example, the 95th-percentile expected latency, and use whichever reply arrives first; **tied requests** enqueue the request on two servers and let the first to start cancel the other. They report a benchmark reading 1,000 keys from 100 BigTable servers in which hedging after 10 ms cut the p99.9 latency from 1,800 ms to 74 ms while sending only about 2% more requests. Trade-off: hedging spends a little extra load to cut the tail, and must be disabled or rate-limited under overload so it does not amplify it. Chapter 23 (§23.6–§23.7) covers implementation. *Evidence: Primary* — [Dean and Barroso 2013]

Two further traps are covered in Chapter 23: computing a fleet p99 by averaging per-host p99s (merge histograms instead, §4.8.3), and **coordinated omission**, where a load generator that waits for slow replies stops sending during slow periods and so under-samples the tail (§23.5).

COMMON MISTAKE

Mistake — "Our p99 is fine, so only 1% of users suffer." If a user session issues 100 requests, the probability that the session contains at least one request slower than p99 is $1 - 0.99^{100} = 63\%$. Percentiles are per request; user experience is per session.

4.9 Basic linear algebra

This section is a preparation for the ML chapters (Part XI onward), where embeddings, similarity search and matrix multiplication dominate cost. You need only three ideas: vectors, the dot product and matrix multiplication.

4.9.1 Vectors

DEFINITION

Definition — Vector. An ordered list of d numbers, $\mathbf{a} = (a_1, a_2, \ldots, a_d)$, where d is the **dimension**. Geometrically, a vector is an arrow from the origin to a point in d-dimensional space; its **length** (or norm) is $\| \mathbf{a} \| = \sqrt{a_1^2 + \cdots + a_d^2}$.

First-principles explanation

A map location is a 2-dimensional vector (latitude, longitude); a song might be a 3-dimensional vector (tempo, loudness, acousticness). An ML **embedding** is the same idea with hundreds or thousands of learned dimensions, arranged so that similar items point in similar directions. "Find similar items" becomes "find nearby arrows" (Chapter 74).

4.9.2 Dot product and cosine similarity

FORMULA

Dot product and cosine similarity

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{d} a_i b_i \qquad \cos\theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$$

Symbol	Meaning	Units
$\mathbf{a}, \mathbf{b}$	Two vectors of the same dimension	(embedding units; dimensionless)
d	Dimension	count
a_i, b_i	i-th components	dimensionless
$\|\mathbf{a}\|$	Length of $\mathbf{a}$	dimensionless
θ	Angle between the vectors	radians or degrees
$\cos\theta$	Cosine similarity: 1 = same direction, 0 = unrelated (perpendicular), −1 = opposite	dimensionless

The dot product multiplies matching components and adds the results. Dividing by both lengths removes the effect of magnitude, so cosine similarity measures *direction only*. If vectors are normalized to length 1 in advance (common for embeddings), cosine similarity *is* the dot product, which is cheaper to compute.

WORKED EXAMPLE

Worked example — Which document is closest to the query? Query $\mathbf{q} = (1, 2, 2)$; document $\mathbf{a} = (2, 1, 2)$; document $\mathbf{b} = (2, 4, 4)$.

1. $\mathbf{q} \cdot \mathbf{a} = 1 \times 2 + 2 \times 1 + 2 \times 2 = 2 + 2 + 4 = 8$.
2. $\|\mathbf{q}\| = \sqrt{1 + 4 + 4} = 3$; $\|\mathbf{a}\| = \sqrt{4 + 1 + 4} = 3$.
3. $\cos\theta_{qa} = 8/(3 \times 3) = 0.889$ (an angle of about 27°).
4. $\mathbf{q} \cdot \mathbf{b} = 2 + 8 + 8 = 18$; $\|\mathbf{b}\| = \sqrt{4 + 16 + 16} = 6$; $\cos\theta_{qb} = 18/(3 \times 6) = 1.0$.

Interpretation. $\mathbf{b}$ is exactly twice $\mathbf{q}$, so it points the same way: cosine 1.0. The raw dot product (18 vs 8) also prefers $\mathbf{b}$ but would be fooled by long vectors in general. **Engineering implication.** Choose the similarity metric the embedding model was trained with; normalize vectors once at ingestion so that serving-time similarity is a plain dot product.

4.9.3 Matrices and the cost of matrix multiplication

DEFINITION

Definition — Matrix. A rectangular grid of numbers with m rows and n columns (an $m \times n$ matrix). A set of n embeddings of dimension d is naturally stored as an $n \times d$ matrix, one embedding per row.

Multiplying an $m \times k$ matrix A by a $k \times n$ matrix B produces an $m \times n$ matrix C in which each entry C_{ij} is the dot product of row i of A with column j of B. Each of the $m \times n$ entries is a dot product of length k.

FORMULA

Cost of matrix multiplication

$$\text{FLOPs} = 2\,m\,n\,k \qquad \text{arithmetic intensity} = \frac{\text{FLOPs}}{\text{bytes moved}}$$

Symbol	Meaning	Units
m, k	Rows and columns of A	count
k, n	Rows and columns of B (inner dimension k must match)	count
FLOPs	Floating-point operations: one multiply and one add per term	operations
bytes moved	Data read from and written to memory	bytes
arithmetic intensity	FLOPs per byte; low values mean memory-bandwidth-bound, high values compute-bound	FLOP/byte

WORKED EXAMPLE

Worked example — Brute-force similarity search and the value of batching.

Part A: one query against one million embeddings. $n = 10^6$ embeddings of dimension $d = 768$ stored as 4-byte floats.

1. Storage: $10^6 \times 768 \times 4 = 3.07 \times 10^9$ bytes ≈ 3.1 GB.
2. Compute: one dot product per embedding, $10^6 \times 768 = 7.68 \times 10^8$ multiply-adds = 1.54×10^9 FLOPs.
3. Intensity: $1.54 \times 10^9 / 3.07 \times 10^9 = 0.5$ FLOP/byte — every float is loaded once and used once.
4. If memory delivers 20 GB/s to this computation (an illustrative figure), scanning takes $3.07/20 = 0.15$ s per query, regardless of how fast the arithmetic is.

Part B: a neural-network layer. Multiply a batch of m inputs ($m \times 4096$) by a 4096×4096 weight matrix stored in 2-byte (16-bit) floats.

1. Weights: $4096 \times 4096 \times 2 = 33.5$ MB.
2. With $m = 1$: FLOPs $= 2 \times 1 \times 4096 \times 4096 = 3.36 \times 10^7$, intensity ≈ $3.36 \times 10^7 / 3.36 \times 10^7 = 1$ FLOP/byte.
3. With $m = 32$: FLOPs $= 2 \times 32 \times 4096 \times 4096 = 1.07 \times 10^9$; the weights are read once for the whole batch, so intensity ≈ 32 FLOP/byte.

Interpretation. Brute-force search and single-request inference are *memory-bound*: the processor waits for data. Batching reuses each loaded weight for many inputs and raises arithmetic intensity proportionally. **Engineering implication.** At a million vectors and many queries per second, brute force is too slow, which motivates approximate nearest neighbor (ANN) indexes (Chapter 74). For model serving, batching is the main lever for throughput — at a latency cost quantified in §4.18.6.

```
        import math

        def dot(a, b):
            return sum(x * y for x, y in zip(a, b, strict=True))

        def norm(a):
            return math.sqrt(dot(a, a))

        def cosine_similarity(a, b):
            return dot(a, b) / (norm(a) * norm(b))

        def matmul(A, B):
            """Naive matrix multiply: (m x k) times (k x n) -> (m x n)."""
            m, k, n = len(A), len(B), len(B[0])
            assert all(len(row) == k for row in A), "inner dimensions must match"
            return [[sum(A[i][t] * B[t][j] for t in range(k)) for j in range(n)]
                    for i in range(m)]

        query = [1, 2, 2]
        docs = {"doc_a": [2, 1, 2], "doc_b": [2, 4, 4], "doc_c": [-2, 1, 0]}
        for name, vec in docs.items():
            print(f"{name}: dot = {dot(query, vec):3d}, cosine =
{cosine_similarity(query, vec):.3f}")

        # Score all documents at once: (1 x 3) query times (3 x 3) matrix of doc columns
        D = [list(col) for col in zip(*docs.values())]   # 3 x 3, one document per col-
umn
        print("scores via matmul:", matmul([query], D)[0])
```

Output:

```
doc_a: dot =   8, cosine = 0.889
doc_b: dot =  18, cosine = 1.000
doc_c: dot =   0, cosine = 0.000
scores via matmul: [8, 18, 0]
```

What to notice:

- `zip(..., strict=True)` (Python 3.10+) raises an error if dimensions differ, a cheap guard against mixing embeddings from different models.
- `doc_c` is perpendicular to the query (dot product 0): unrelated, not opposite.
- Scoring many documents is one matrix multiplication. The triple loop in `matmul` performs exactly $m \times n \times k$ multiply-adds; production libraries (BLAS, GPU kernels) do the same arithmetic but reorganize it to reuse data in caches.

4.10 Queueing theory

4.10.1 Vocabulary

DEFINITION

Definition — Queueing system. A model in which **jobs** (requests, messages, packets) arrive at a **service center**, wait in a **queue** if all **servers** are busy, receive service, and depart. The **arrival rate** λ is the average number of jobs arriving per second; the **service rate** μ is the average number of jobs one server can complete per second when it is never idle, so the average **service time** is $S = 1/\mu$.

First-principles explanation

Picture a supermarket checkout. Customers arrive at random and take a random time to serve. When the cashier is idle half the time, a queue rarely forms; when the cashier is busy 95% of the time, a long queue is routine, although the cashier is not technically overloaded. **Waiting is caused by variability, and its effect grows without bound as utilization approaches 100%.** With perfectly regular arrivals and service, a cashier at 99% would never have a queue. Every CPU core, disk, thread, connection and network link in a design is a checkout.

Why does this exist?

Capacity plans that ignore queueing assume a server at 90% utilization performs like one at 50%. It does not. Queueing theory, begun by Erlang for telephone exchanges and developed for computer systems from the 1960s onward [Kleinrock 1975; Harchol-Balter 2013], quantifies how much worse it is.

Kendall notation names queue models as A/S/c: arrival process / service-time distribution / number of servers. “M” stands for *Markovian* (memoryless): Poisson arrivals, meaning exponentially distributed gaps between arrivals (§4.7.2), or exponentially distributed service times. **M/M/1** is the simplest model: random arrivals, random service, one server, first-come-first-served, unlimited queue.

4.10.2 The M/M/1 queue

Figure 4.3 — The M/M/1 queue.



Reading the diagram. Jobs flow left to right. Arrivals come from many independent clients at average rate λ (requests/s); the queue (subroutine shape) is a buffer, typically in

memory, with no limit in this model; the single server processes one job at a time at average rate μ. In steady state, departures equal arrivals (λ), so throughput is set by demand, not by the server, as long as $\lambda < \mu$. The failure point is the queue itself: if $\lambda \geq \mu$ it grows without bound until memory or timeouts intervene. Scaling options are a faster server (raise μ), more servers (M/M/c, §4.10.4) or less demand (caching, load shedding).

FORMULA

M/M/1 results

$$\rho = \frac{\lambda}{\mu} \qquad W = \frac{1}{\mu - \lambda} = \frac{S}{1 - \rho} \qquad W_q = \frac{\rho}{\mu - \lambda} \qquad L = \frac{\rho}{1 - \rho} \qquad L_q = \frac{\rho^2}{1 - \rho}$$

Symbol	Meaning	Units
λ	Average arrival rate	jobs/second
μ	Average service rate of the server	jobs/second
$S = 1/\mu$	Average service time	seconds
ρ	Utilization: fraction of time the server is busy (must be < 1)	dimensionless
W	Average time in system (waiting + service), i.e. response time	seconds
W_q	Average time waiting in the queue before service starts	seconds
L	Average number of jobs in the system	jobs
L_q	Average number of jobs waiting in the queue	jobs

Derivation sketch. Because arrivals are memoryless, the number of jobs in the system behaves like a random walk that steps up at rate λ and down at rate μ. Balancing the flow between neighboring states gives $P(n \text{ jobs}) = (1 - \rho)\rho^n$, a geometric distribution. Its mean is $L = \rho/(1 - \rho)$. Little's Law (§4.11) then gives $W = L/\lambda = 1/(\mu - \lambda)$. A further useful fact: in M/M/1 the response time itself is exponentially distributed with rate $\mu - \lambda$, so its q-th percentile is $W \times \ln \frac{1}{1-q}$; the p99 is $W \times \ln 100 = 4.6W$ [Harchol-Balter 2013].

WORKED EXAMPLE

Worked example — One server at increasing load. A service handles requests in 10 ms on average ($\mu = 100$/s).

1. At $\lambda = 50$/s: $\rho = 0.5$; $W = 1/(100 - 50) = 0.020$ s $= 20$ ms; $L = 0.5/0.5 = 1$ request.
2. At $\lambda = 80$/s: $\rho = 0.8$; $W = 1/20 = 50$ ms; $L = 0.8/0.2 = 4$.
3. At $\lambda = 90$/s: $\rho = 0.9$; $W = 1/10 = 100$ ms; $L = 9$; $L_q = 0.81/0.1 = 8.1$; p99 $= 4.605 \times 100 \approx 461$ ms.
4. At $\lambda = 99$/s: $\rho = 0.99$; $W = 1/1 = 1,000$ ms.

Interpretation. Going from 50% to 90% utilization (1.8× the traffic) multiplies mean latency by 5, and the last 9% of traffic (90 → 99/s) multiplies it by another 10.
Engineering implication. Plan latency-sensitive services to run at moderate utilization at peak — often 50–70% for interactive work — and reserve high utilization for batch work whose latency nobody waits for.

Table 4.5 — M/M/1 response time as a multiple of service time.

Utilization ρ	$W/S = 1/(1-\rho)$	W when S = 10 ms	Jobs in system L	p99 when S = 10 ms
0.50	2	20 ms	1	92 ms
0.70	3.3	33 ms	2.3	154 ms
0.80	5	50 ms	4	230 ms
0.90	10	100 ms	9	461 ms
0.95	20	200 ms	19	921 ms
0.99	100	1,000 ms	99	4,605 ms

4.10.3 Why latency explodes near 100% utilization

The factor $1/(1-\rho)$ is the whole story. The distance to saturation, $1-\rho$, is in the denominator, so each halving of the remaining headroom doubles the latency: 0.8 → 0.9 doubles W, 0.9 → 0.95 doubles it again, and so on. The curve is the familiar “hockey stick”. Two practical consequences follow:

- **Small traffic increases near saturation cause large latency increases.** A service at 90% that sees a 5% traffic spike moves to 94.5%, and its mean latency nearly doubles.
- **Overload is not graceful.** At $\rho \geq 1$ there is no steady state: the queue grows for as long as the overload lasts, and every request that joins it waits for all earlier ones. Bounded queues, timeouts and load shedding (§22.9) exist to convert unbounded latency into fast, explicit rejection.

The M/M/1 model assumes exponential service times. Real service times can be more or less variable; the general **Pollaczek–Khinchine** formula for the M/G/1 queue shows that mean queueing delay is proportional to $(1 + C_S^2)/2$, where C_S is the coefficient of variation (standard deviation divided by mean) of the service time [Harchol-Balter 2013]. Exponential service has $C_S = 1$; perfectly constant service ($C_S = 0$) halves the queueing delay; heavy-tailed service ($C_S \gg 1$) multiplies it. The $1/(1 - \rho)$ explosion is present in all of them.

The following simulation checks the M/M/1 formula. It uses the recurrence "a job starts when it arrives or when the previous job finishes, whichever is later".

```
        import random

        def simulate_mm1(lam, mu, n_customers=200_000, seed=1):
            """Simulate a FIFO single-server queue with Poisson arrivals and
            exponential service times. Returns mean time in system (seconds)."""
            rng = random.Random(seed)
            arrival = 0.0          # arrival time of current customer
            server_free_at = 0.0   # time at which the server finishes its current job
            total_time = 0.0
            for _ in range(n_customers):
                arrival += rng.expovariate(lam)            # next arrival
                start = max(arrival, server_free_at)       # wait if server busy
                server_free_at = start + rng.expovariate(mu)
                total_time += server_free_at - arrival     # queueing + service
            return total_time / n_customers

        mu = 100.0  # server completes 100 requests/s on average (10 ms each)
        print(" rho   formula W (ms)   simulated W (ms)")
        for lam in (50, 80, 90, 95, 99):
            formula = 1 / (mu - lam)
            sim = simulate_mm1(lam, mu)
            print(f"{lam/mu:4.2f}   {formula*1000:12.1f}   {sim*1000:14.1f}")
```

Output:

```
 rho   formula W (ms)   simulated W (ms)
0.50           20.0             20.0
0.80           50.0             49.7
0.90          100.0             98.6
0.95          200.0            220.4
0.99         1000.0           1059.1
```

What to notice:

- At moderate utilization 200,000 simulated requests match the formula to within about 1%.
- Near saturation the simulation is noisy (±6–10% here) even with 200,000 requests: the queue drifts through long busy periods, so averages converge slowly. Load tests

near saturation need long runs for the same reason.
- `max(arrival, server_free_at)` is the entire queueing mechanism: waiting happens only when a job arrives before the previous one finishes.

4.10.4 Multiple servers: M/M/c intuition

With c identical servers sharing **one queue** (M/M/c), a job waits only if all c servers are busy. The probability of waiting is given by the **Erlang C** formula; utilization is $\rho = \lambda/(c\mu)$.

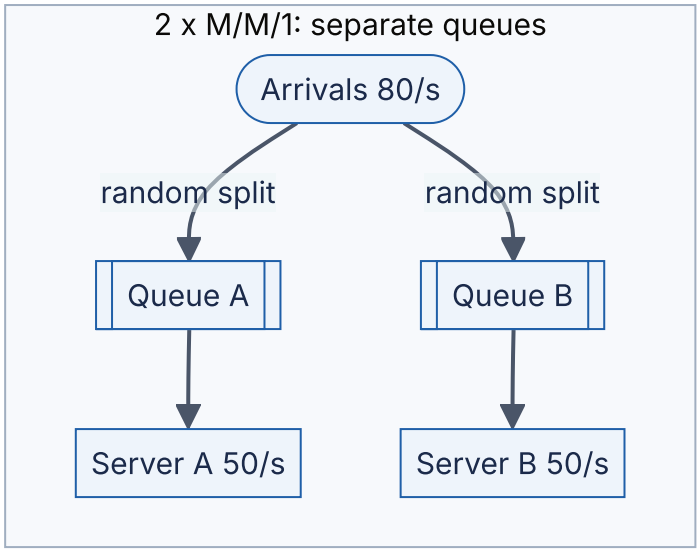
FORMULA

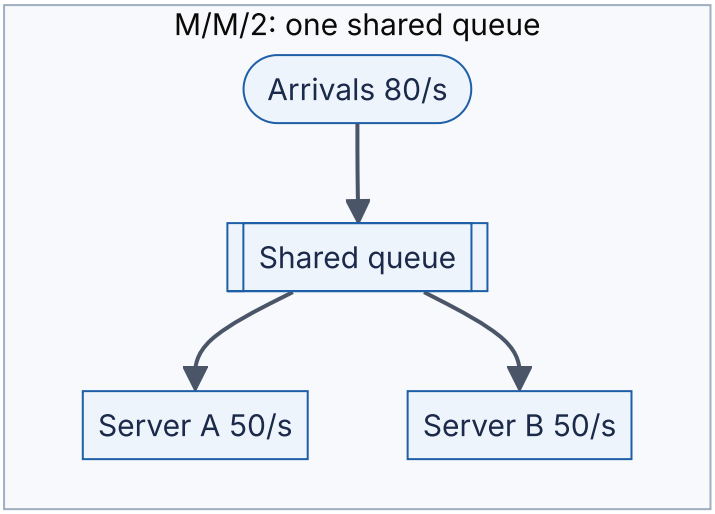
M/M/c (Erlang C)

$$a = \frac{\lambda}{\mu}, \quad \rho = \frac{a}{c}, \quad P_{\text{wait}} = \frac{\dfrac{a^c}{c!\,(1-\rho)}}{\sum_{k=0}^{c-1} \dfrac{a^k}{k!} + \dfrac{a^c}{c!\,(1-\rho)}}, \quad W = \frac{P_{\text{wait}}}{c\mu - \lambda} + \frac{1}{\mu}$$

Symbol	Meaning	Units
c	Number of servers sharing one queue	count
μ	Service rate of *each* server	jobs/second
a	Offered load: average number of busy servers	servers (Erlangs)
ρ	Per-server utilization (< 1)	dimensionless
P_{wait}	Probability an arriving job finds all servers busy	dimensionless
W	Mean response time	seconds

Figure 4.4 — One shared queue versus separate queues.





Reading the diagram. Both subgraphs have the same hardware (two servers at 50 jobs/s) and the same demand (80 jobs/s, $\rho = 0.8$). On the left, any idle server takes the next job from a shared queue. On the right, each job is assigned to a server's private queue at arrival, so one queue can hold waiting jobs while the other server sits idle. All paths are synchronous hand-offs. The failure point on the right is *imbalance*; the scaling lesson is that pooling beats partitioning for latency.

WORKED EXAMPLE

Worked example — Pooling. $\lambda = 80$/s; compare three designs with 100 jobs/s of total capacity.

1. *One fast server* (M/M/1, $\mu = 100$): $W = 1/(100 - 80) = 50$ ms.
2. *Two slow servers, separate queues* (each M/M/1 with $\lambda = 40$, $\mu = 50$): $W = 1/(50 - 40) = 100$ ms.
3. *Two slow servers, shared queue* (M/M/2, $\mu = 50$): $a = 80/50 = 1.6$, $\rho = 0.8$. Numerator: $a^2/(2!\,(1 - 0.8)) = 2.56/0.4 = 6.4$. Denominator: $1 + 1.6 + 6.4 = 9$. $P_{\text{wait}} = 6.4/9 = 0.711$. $W = 0.711/(100 - 80) + 1/50 = 0.0356 + 0.020 = 55.6$ ms.

Interpretation. The shared queue (55.6 ms) is nearly as good as one server twice as fast (50 ms) and almost twice as good as separate queues (100 ms) on identical hardware. **Engineering implication.** Prefer a shared work queue (one listen socket feeding a pool of workers, one broker topic with competing consumers) over static assignment. Load-balancing algorithms such as least-connections and power-of-two-choices (§19.4) approximate a shared queue across machines.

```
        import math

        def erlang_c(c, lam, mu):
            """Probability that an arriving request must wait (M/M/c, Erlang C)."""
            a = lam / mu                # offered load in "servers' worth" of work
            rho = a / c                 # per-server utilization
            if rho >= 1:
                return 1.0
            top = a**c / (math.factorial(c) * (1 - rho))
            bottom = sum(a**k / math.factorial(k) for k in range(c)) + top
            return top / bottom

        def mmc_response_time(c, lam, mu):
            """Mean time in system W for an M/M/c queue (seconds)."""
            wait = erlang_c(c, lam, mu) / (c * mu - lam)
            return wait + 1 / mu

        lam = 80.0
        print(f"One fast server  (M/M/1, mu=100): W = {mmc_response_time(1, lam,
100)*1000:.1f} ms")
        print(f"Two slow servers, shared queue (M/M/2, mu=50 each): "
              f"W = {mmc_response_time(2, lam, 50)*1000:.1f} ms")
        print(f"Two slow servers, separate queues (2 x M/M/1, mu=50, lam=40 each): "
              f"W = {mmc_response_time(1, lam/2, 50)*1000:.1f} ms")
```

Output:

```
One fast server  (M/M/1, mu=100): W = 50.0 ms
Two slow servers, shared queue (M/M/2, mu=50 each): W = 55.6 ms
Two slow servers, separate queues (2 x M/M/1, mu=50, lam=40 each): W = 100.0 ms
```

What to notice: with $c = 1$ the Erlang C function reduces to $P_{\text{wait}} = \rho$ and $W = 1/(\mu - \lambda)$, so the same code covers M/M/1. The `rho >= 1` guard reflects that no steady state exists at or above saturation.

> COMMON MISTAKE
>
> **Mistake — "We are at 85% CPU, so we have 15% headroom."** Headroom in *throughput* is 15%, but latency at 85% is already $1/(1 - 0.85) = 6.7$ times the service time in an M/M/1 model, and every further point of load costs more latency than the last. Express headroom as the latency you will have at the forecast peak, not as spare percentage.

4.11 Little's Law

4.11.1 Statement and intuition

> DEFINITION
>
> **Definition — Little's Law.** For any stable system observed over a long period, the average number of items in the system equals the average arrival rate multiplied by the average time each item spends in the system: $L = \lambda W$ [Little 1961].

First-principles explanation

A café serves 20 customers per hour, and each customer stays for 30 minutes on average. How many customers are inside at a typical moment? Every hour 20 people come in, each contributing half an hour of "presence", so the café accumulates $20 \times 0.5 = 10$ customer-hours of presence per hour: on average 10 people are inside. That is all Little's Law says. Remarkably, it does not depend on the arrival pattern, the service-time distribution, the number of servers or the scheduling order. It needs only that the system is stable (what goes in eventually comes out) and that the three quantities are long-run averages measured over the same boundary.

FORMULA

Little's Law

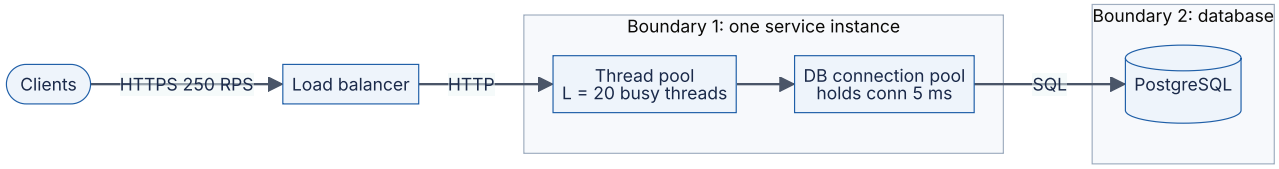
$$L = \lambda \times W$$

Symbol	Meaning	Units
L	Average number of items in the system (concurrency, occupancy, in-flight requests)	items
λ	Average arrival rate (equals throughput in a stable system)	items/second
W	Average time an item spends inside the system boundary	seconds

Derivation intuition. Plot the number of items inside the system, $N(t)$, over a long interval of length T. The area under that curve can be computed two ways. Summed vertically, it is (average height) × (width) $= L \times T$. Summed horizontally, each item contributes a horizontal strip as long as its stay, so the area is (number of items) × (average stay) $= \lambda T \times W$. Equating the two gives $LT = \lambda TW$, i.e. $L = \lambda W$. Little's 1961 paper made this argument rigorous.

Because the law holds for *any* boundary, you can draw the boundary around a thread pool, a connection pool, a broker topic, a whole datacenter or a single CPU, and solve for whichever of the three quantities you do not know.

Figure 4.5 — Choosing the boundary for Little's Law.



Reading the diagram. Clients send HTTPS requests to a load balancer, which forwards plain HTTP to a service instance. Inside the instance (boundary 1) each request occupies a worker thread for its whole duration, and briefly borrows a database connection to issue SQL to PostgreSQL (boundary 2, the storage boundary). All calls are synchronous. Little's Law applies separately to each boundary: threads in use = request rate × request duration; connections in use = query rate × connection hold time. The failure point is any pool whose occupancy L approaches its size, because requests then queue for a thread or connection. Scaling options are more instances (spreading λ), shorter hold times (reducing W) or larger pools (raising the limit on L).

4.11.2 Capacity planning examples

WORKED EXAMPLE

Worked example 1 — Sizing a thread pool. A service receives 2,000 RPS at peak, spread across 8 instances. Each request holds a worker thread for 80 ms on average (mostly waiting on downstream calls).

1. Per-instance arrival rate: $\lambda = 2,000/8 = 250$ RPS.
2. Average busy threads: $L = \lambda W = 250 \times 0.080 = 20$ threads.
3. A pool of exactly 20 would run at 100% utilization and queue without bound (§4.10). Treat the pool as an M/M/c system with c threads, $\mu = 1/0.08 = 12.5$/s per thread and $a = 20$.
4. Using the Erlang C code of §4.10.4: with $c = 25$ ($\rho = 0.80$), 20.9% of requests wait for a thread and mean latency is 83.4 ms; with $c = 30$ ($\rho = 0.67$), 2.5% wait and latency is 80.2 ms; with $c = 35$, 0.16% wait.

Interpretation. Little's Law gives the *average* concurrency (20); queueing theory tells you how much headroom above it buys acceptable waiting. **Engineering implication.** Size the pool around 30 threads per instance, and re-derive it whenever W changes: if a downstream dependency slows from 80 ms to 400 ms, required concurrency jumps to $250 \times 0.4 = 100$ threads, and a 30-thread pool saturates. That is why per-dependency timeouts and bulkheads (§22.11, §22.14) are capacity mechanisms, not just error handling.

WORKED EXAMPLE

Worked example 2 — Sizing a database connection pool. 30 application instances together issue 1,200 queries per second. Each query holds a connection for 5 ms (network round trip plus execution).

1. Connections in use, fleet-wide: $L = 1,200 \times 0.005 = 6$ connections on average.
2. Per instance: $\lambda = 1,200/30 = 40$ queries/s, so $L = 40 \times 0.005 = 0.2$ connections.
3. A common default of 10 connections per instance opens $30 \times 10 = 300$ connections to serve an average demand of 6.
4. Incident case: a missing index makes queries take 500 ms. Now $L = 1,200 \times 0.5 = 600$ connections are needed — more than the 300 that exist, so requests queue for connections, threads block, and the outage spreads upstream.

Interpretation. The pool is 50× oversized in normal operation and still too small during the incident. **Engineering implication.** Oversized pools are not free: in PostgreSQL each connection is a separate server process with its own memory, so hundreds of mostly idle connections waste database resources and let a slow-query incident push far more concurrent work onto an already struggling database. A small pool (here perhaps 2–3 per instance) or a central pooler such as PgBouncer (§14.13), combined with query timeouts, caps concurrency at the database where it matters.

WORKED EXAMPLE

Worked example 3 — Inferring latency from a backlog. A message broker topic holds a backlog of 5,000 messages, and consumers process 1,000 messages per second while producers add messages at the same rate.

1. Rearranging Little's Law: $W = L/\lambda = 5,000/1,000 = 5$ s.

Interpretation. Each message spends about 5 s waiting in the topic before processing. **Engineering implication.** Consumer lag measured in *messages* (the metric most brokers expose) converts to lag in *seconds* — the metric users care about — by dividing by throughput. Alert on the latter.

COMMON MISTAKE

Mistake — Mixing boundaries. Little's Law requires that L, λ and W are measured on the *same* boundary. Using end-to-end request latency (which includes time spent in the database) with the occupancy of the application thread pool is correct only if threads are held for the entire request. If an asynchronous framework releases the thread while waiting for I/O, the thread-holding time W is much smaller than request latency.

4.12 Amdahl's Law and Gustafson's Law

4.12.1 Amdahl's Law

DEFINITION

Definition — Speedup. The ratio of the time a job takes on one processor (or node) to the time it takes on N: $S(N) = T(1)/T(N)$. Perfect, **linear** speedup is $S(N) = N$.

First-principles explanation

Eight friends cook a dinner. Chopping vegetables parallelizes beautifully: eight people chop eight times faster. But the oven takes 40 minutes no matter how many cooks there are. However many friends you add, dinner cannot take less than 40 minutes. Gene Amdahl made this argument in 1967 about parallel computers [Amdahl 1967]: the part of a job that cannot be parallelized sets a hard ceiling on speedup.

FORMULA

Amdahl's Law (fixed problem size)

$$S(N) = \frac{1}{(1 - p) + \frac{p}{N}} \qquad \lim_{N \to \infty} S(N) = \frac{1}{1 - p}$$

Symbol	Meaning	Units
p	Fraction of the single-processor run time that can be parallelized	dimensionless (0–1)
$1 - p$	Serial fraction	dimensionless
N	Number of processors, cores or nodes	count
$S(N)$	Speedup relative to one processor	dimensionless (×)

Derivation. Normalize the one-processor time to 1. The serial part still takes $1 - p$; the parallel part takes p/N on N processors. Speedup is the old time (1) divided by the new time.

WORKED EXAMPLE

Worked example — A 95% parallel job. A batch job spends 5% of its time in a single-threaded setup phase and 95% in work that splits perfectly across cores.

1. On 8 cores: $S = 1/(0.05 + 0.95/8) = 1/(0.05 + 0.11875) = 1/0.16875 = 5.93$.
2. On 64 cores: $S = 1/(0.05 + 0.01484) = 1/0.06484 = 15.4$.
3. Ceiling: $1/0.05 = 20$, however many cores.
4. Efficiency on 64 cores: $15.4/64 = 24\%$.

Interpretation. Moving from 8 to 64 cores (8× the hardware) improves speed only 2.6×. **Engineering implication.** Before buying more parallelism, measure and shrink the serial fraction: a single-threaded coordinator, a global lock, a sequential merge step. Cutting the serial fraction from 5% to 1% raises the ceiling from 20 to 100.

4.12.2 Gustafson's Law

Amdahl assumes a *fixed* problem. In practice, when engineers get more machines they usually tackle *bigger* problems — more data, higher resolution, larger models — in the same wall-clock time. John Gustafson observed in 1988 that under this assumption the serial part stays roughly constant while the parallel part grows with N [Gustafson 1988].

FORMULA

Gustafson's Law (scaled problem size)

$$S_{\text{scaled}}(N) = (1 - p) + pN = N - (1 - p)(N - 1)$$

Symbol	Meaning	Units
p	Fraction of the *parallel run's* time spent in parallel work	dimensionless
$1 - p$	Fraction of the parallel run's time that is serial	dimensionless
N	Number of processors	count
S_{scaled}	How much more work is completed in the same time than one processor could do	dimensionless (×)

WORKED EXAMPLE

Worked example — The same 5% serial fraction, scaled. On 64 processors a job spends 5% of its wall-clock time in serial work.

1. $S_{\text{scaled}} = 0.05 + 0.95 \times 64 = 0.05 + 60.8 = 60.85$.

Interpretation. 64 processors complete about 61 times as much work as one would in the same time, versus Amdahl's 15.4 for a fixed problem. The laws do not contradict each other; they answer different questions ("how much faster?" vs "how much bigger?"). **Engineering implication.** Horizontal scaling of data-parallel systems (MapReduce, Spark, distributed training) mostly lives in Gustafson's world: more nodes handle more data. Latency-sensitive single requests live in Amdahl's world: more nodes do not make one request much faster.

4.13 The Universal Scalability Law

4.13.1 Beyond Amdahl: coordination costs

Amdahl's Law predicts that adding nodes never *hurts*: throughput keeps rising toward a ceiling. Real systems often get *slower* past some size. Neil Gunther's **Universal Scalability Law** (USL) explains why by adding a second penalty: **coherency** (or crosstalk), the cost of nodes keeping each other consistent [Gunther 2007].

First-principles explanation

Return to the kitchen. Contention is the queue for the single oven (Amdahl's serial part). Coherency is the time cooks spend *talking to each other* — "did you already salt this?". If every cook must check with every other cook, the number of conversations grows like $N(N-1)$, roughly N^2. With few cooks this is negligible; with many it dominates, and adding a cook makes dinner later.

FORMULA

Universal Scalability Law

$$C(N) = \frac{N}{1 + \alpha(N - 1) + \beta N(N - 1)} \qquad N^* = \sqrt{\frac{1 - \alpha}{\beta}}$$

Symbol	Meaning	Units
N	Number of nodes, cores, threads or concurrent users	count
$C(N)$	Relative capacity: throughput at N divided by throughput at 1	dimensionless (×)
α	Contention coefficient: serialization on shared resources (queues, locks)	dimensionless (0–1)
β	Coherency coefficient: pairwise coordination cost (cache coherence, consensus, data exchange)	dimensionless (≥ 0)
N^*	Concurrency at which throughput peaks	count

Gunther's later work writes the coefficients as σ (contention) and κ (coherency); the formula is the same. With $\beta = 0$ the USL reduces exactly to Amdahl's Law with serial fraction α. N^* comes from setting the derivative of $C(N)$ to zero.

WORKED EXAMPLE

Worked example — Where adding nodes stops helping. A clustered service is measured and fitted with $\alpha = 0.05$ and $\beta = 0.001$.

1. $C(8) = 8/(1 + 0.05 \times 7 + 0.001 \times 8 \times 7) = 8/(1 + 0.35 + 0.056) = 8/1.406 = 5.69$.
2. $C(32) = 32/(1 + 1.55 + 0.992) = 32/3.542 = 9.03$.
3. $C(64) = 64/(1 + 3.15 + 4.032) = 64/8.182 = 7.82$.
4. Peak: $N^* = \sqrt{(1 - 0.05)/0.001} = \sqrt{950} = 30.8$ nodes.

Interpretation. Throughput peaks around 31 nodes at about 9× a single node; doubling to 64 nodes *loses* 13% of throughput. Amdahl with the same α predicts 15.4× at 64 nodes. **Engineering implication.** If load tests show throughput flattening and then falling as you add nodes or threads, look for coherency costs: all-to-all communication, cache-line ping-pong, distributed locks, broadcast invalidation. The cure is architectural (partitioning so that nodes rarely coordinate), not more hardware.

```
        import math

        def amdahl(p, n):
            """Speedup with n processors when fraction p of the work is parallel."""
            return 1 / ((1 - p) + p / n)

        def gustafson(p, n):
            """Scaled speedup when the parallel part grows with n."""
            return (1 - p) + p * n

        def usl(n, alpha, beta):
            """Relative capacity C(n) under the Universal Scalability Law."""
            return n / (1 + alpha * (n - 1) + beta * n * (n - 1))

        p, alpha, beta = 0.95, 0.05, 0.001
        print("   N  Amdahl  Gustafson   USL")
        for n in (1, 2, 4, 8, 16, 32, 64, 128):
            print(f"{n:4d}  {amdahl(p, n):6.2f}  {gustafson(p, n):9.2f}  {usl(n, alpha,
beta):5.2f}")
        n_star = math.sqrt((1 - alpha) / beta)
        print(f"USL peak at N* = {n_star:.1f}; Amdahl ceiling = {1/(1-p):.0f}")
```

Table 4.6 — Output: three scaling models with a 5% serial fraction.

N	Amdahl	Gustafson	USL (α=0.05, β=0.001)
1	1.00	1.00	1.00
2	1.90	1.95	1.90
4	3.48	3.85	3.44
8	5.93	7.65	5.69
16	9.14	15.25	8.04
32	12.55	30.45	9.03
64	15.42	60.85	7.82
128	17.41	121.65	5.42

The program also prints `USL peak at N* = 30.8; Amdahl ceiling = 20`. What to notice: the three columns agree for small N, which is why a few measurements at low concurrency cannot distinguish the models. Measure out to the concurrency you intend to run. In practice the USL coefficients are fitted by regression to throughput measured at several values of N [Gunther 2007]; Chapter 23 (§23.10) shows the workflow.

COMMON MISTAKE

Mistake — Extrapolating linear scaling from a small test. "Two nodes gave 1.9×, so 100 nodes will give 95×." Every model in Table 4.6 is nearly linear at $N = 2$. Contention and coherency only show up at scale, so linear extrapolation is the most common way capacity plans fail.

4.14 Availability arithmetic

4.14.1 Nines and downtime budgets

DEFINITION

Definition — Availability. The fraction of time (or of requests) for which a system serves correctly: $A = \text{uptime}/(\text{uptime} + \text{downtime})$, or, request-based, $A = \text{successful requests}/\text{valid requests}$. It is commonly quoted in "nines": 99.9% is "three nines".

Table 4.7 — Nines and the downtime they allow (365-day year, 30-day month, 7-day week).

Availability	Nines	Downtime per year	Downtime per month	Downtime per week
90%	1	36.5 days	72 h	16.8 h
99%	2	3.65 days (87.6 h)	7.2 h	1.68 h
99.9%	3	8.76 h	43.2 min	10.1 min
99.95%	3.5	4.38 h	21.6 min	5.04 min
99.99%	4	52.6 min	4.32 min	60.5 s
99.999%	5	5.26 min	25.9 s	6.05 s
99.9999%	6	31.5 s	2.59 s	0.60 s

FORMULA

Downtime budget

$$D = (1 - A) \times T$$

Symbol	Meaning	Units
A	Availability target	dimensionless
T	Length of the period	minutes (or any time unit)
D	Allowed downtime in the period (the error budget, in time)	same unit as T

WORKED EXAMPLE

Worked example — What four nines allows. Target $A = 99.99\%$ over a 30-day month.

1. $T = 30 \times 24 \times 60 = 43,200$ min.
2. $D = 0.0001 \times 43,200 = 4.32$ min.

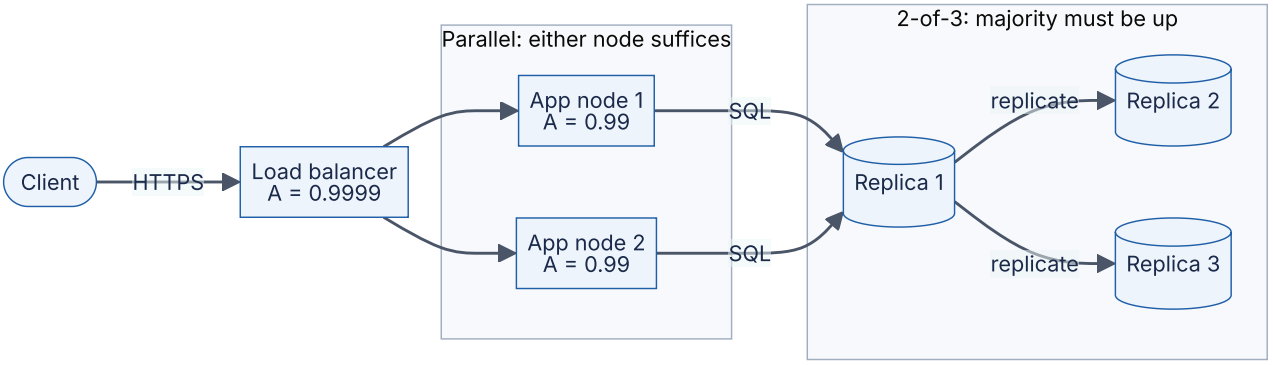
Interpretation. Four minutes and 19 seconds per month, total. **Engineering implication.** A human cannot be paged, wake up, log in and diagnose in four minutes. Four nines therefore implies *automated* detection and failover; five nines (26 s/month) implies the failover itself must be nearly invisible. Each additional nine changes the architecture, not just the effort.

INDUSTRY EXAMPLE

Google SRE — error budgets. Google's SRE practice turns an availability SLO into an **error budget** equal to $1 - \text{SLO}$: a 99.9% SLO allows 0.1% of requests (or time) to fail. While budget remains, teams may ship risky changes; when it is exhausted, releases slow until reliability recovers. The SRE book also argues that 100% is the wrong target, because users cannot distinguish the last nines from failures in their own devices and networks, while each extra nine costs disproportionately more. Error budgets are covered in §39.7. *Evidence: Primary* — [Beyer et al. 2016]

4.14.2 Series and parallel composition

Figure 4.6 — Series, parallel and k-of-n availability.



Reading the diagram. A request flows synchronously left to right: HTTPS from the client to a single load balancer, then to *either* application node, then over SQL to the current leader replica (Replica 1), which replicates each write to the other two and needs a majority (2 of 3) of replicas up to serve, as consensus-based stores do (§26.4); if the leader fails, a surviving replica is elected in its place. The three stages are in **series**: the request fails if any stage fails. Within the middle stage the two nodes are in **parallel**: the stage fails only if both fail. The cylinders are the storage boundary. The single load balancer is the obvious single point of failure; the scaling and availability option is to make it redundant too (§19.8).

FORMULA

Series, parallel and k-of-n availability (independent failures)

$$A_{\text{series}} = \prod_{i=1}^{n} A_i \qquad A_{\text{parallel}} = 1 - \prod_{i=1}^{n} (1 - A_i) \qquad A_{k\text{-of-}n} = \sum_{j=k}^{n} \binom{n}{j} A^j (1 - A)^{n-j}$$

Symbol	Meaning	Units
A_i	Availability of component i	dimensionless
$\prod$	Product over all components	—
n	Number of components	count
k	Minimum number of identical components that must be up	count
A	Availability of each identical component (k-of-n case)	dimensionless

Derivation. *Series:* the system is up only if every component is up; with independence, multiply. *Parallel:* the system is down only if every component is down; the probability all are down is $\prod (1 - A_i)$, and availability is its complement. *k-of-n:* the number of up components is binomial (§4.6.4); add the probabilities of $k, k+1, \ldots, n$ components up.

WORKED EXAMPLE

Worked example — Composing the system of Figure 4.6.

1. *Series of three single components* first, for intuition: LB 99.99%, one app node 99.95%, one database 99.9%: $0.9999 \times 0.9995 \times 0.999 = 0.99840$, i.e. 99.84%, about 14.0 h of downtime per year — worse than the weakest component.
2. *Parallel app tier:* two nodes at 99%: $1 - (0.01)^2 = 1 - 0.0001 = 0.9999$.
3. *2-of-3 database:* $A = 0.99$: $\binom{3}{2}(0.99)^2(0.01) + (0.99)^3 = 3 \times 0.9801 \times 0.01 + 0.970299 = 0.029403 + 0.970299 = 0.999702$.
4. *Whole system in series:* $0.9999 \times 0.9999 \times 0.999702 = 0.999502$, about 4.4 h of downtime per year.

Interpretation. Series composition always lowers availability; redundancy raises it dramatically — two 99% nodes give 99.99%. **Engineering implication.** A request path through 10 services, each 99.9% available, is at most $0.999^{10} = 99.0\%$ available. Long synchronous call chains are an availability liability, which is one argument for asynchronous decoupling (Part VI) and for keeping critical paths short.

```
        from math import comb, prod

        def series(*avail):
            """All components must be up (assumes independent failures)."""
            return prod(avail)

        def parallel(*avail):
            """At least one component must be up (independent failures)."""
            return 1 - prod(1 - a for a in avail)

        def k_of_n(k, n, a):
            """At least k of n identical, independent components must be up."""
            return sum(comb(n, i) * a**i * (1 - a)**(n - i) for i in range(k, n + 1))

        def downtime_per_year(a, hours_per_year=365 * 24):
            return (1 - a) * hours_per_year

        web_tier = parallel(0.99, 0.99)          # two app nodes behind a load balancer
        db_quorum = k_of_n(2, 3, 0.99)           # 2-of-3 replicated database
        system = series(0.9999, web_tier, db_quorum)   # LB -> web tier -> DB

        for name, a in [("web tier", web_tier), ("db 2-of-3", db_quorum), ("system",
system)]:
            print(f"{name:10s} A = {a:.6f}  downtime = {downtime_per_year(a):6.2f}
h/year")
```

Output:

```
web tier   A = 0.999900  downtime =   0.88 h/year
db 2-of-3  A = 0.999702  downtime =   2.61 h/year
system     A = 0.999502  downtime =   4.36 h/year
```

What to notice: `series`, `parallel` and `k_of_n` compose — the result of one can be fed into another to model any tree of components. Every function assumes independent failures, so the outputs are *upper bounds* on what a real system with shared dependencies achieves.

WARNING / TRADE-OFF

Warning — Redundancy arithmetic is only as good as its independence assumption. Two app nodes deployed by the same bad configuration push fail together; three replicas on one power feed fail together. Also, parallel redundancy requires *failover to work*: if detecting a failure and rerouting takes 30 s, each failure still costs 30 s of errors. Model failover time explicitly, and place replicas in separate failure domains (racks, zones, regions; §39.4).

4.15 Reliability: MTBF, MTTR and failure rates

4.15.1 Definitions

DEFINITION

Definition — Reliability, MTBF, MTTR and failure rate. Reliability $R(t)$ is the probability that a component operates without failure for a period of length t. The **mean time between failures (MTBF)** is the average operating time from the end of one repair to the next failure (for non-repairable parts the same quantity is called **MTTF**, mean time to failure). The **mean time to repair (MTTR)** is the average time from failure to restored service, including detection, diagnosis and repair. The **failure rate** λ is the average number of failures per unit of operating time.

First-principles explanation

Availability asks “what fraction of the time is it working?”. Reliability asks “how likely is it to keep working for the next hour (or day, or job)?”. A car that breaks down every week but is fixed in five minutes by the roadside is highly *available* and poorly *reliable*; a car that never breaks down for a year but then spends a month in the garage is the opposite. Both properties matter, for different users (§4.18.2).

FORMULA

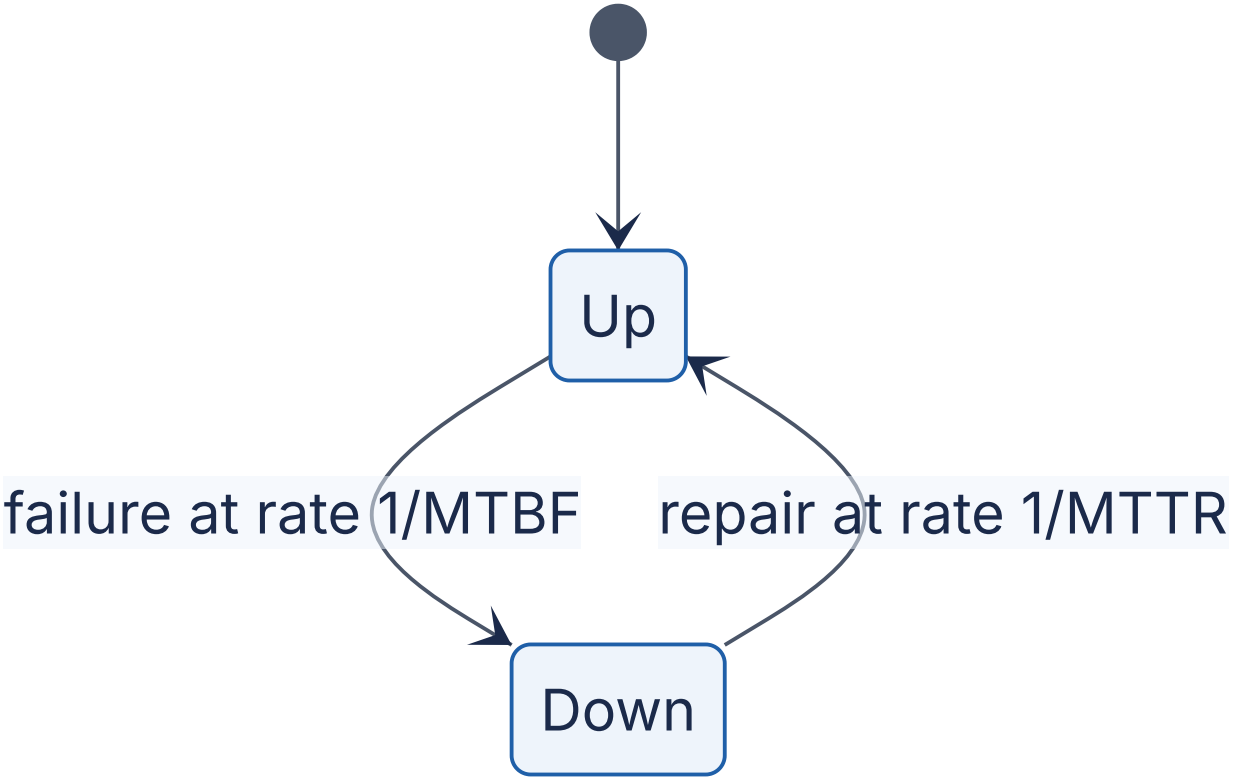
Reliability and availability from MTBF and MTTR

$$\lambda = \frac{1}{\text{MTBF}} \qquad R(t) = e^{-\lambda t} \qquad A = \frac{\text{MTBF}}{\text{MTBF}+\text{MTTR}} \qquad \text{AFR} = 1 - e^{-8760/\text{MTTF}}$$

Symbol	Meaning	Units
MTBF	Mean operating time between failures	hours
MTTR	Mean time to repair (detect + diagnose + fix + restore)	hours
MTTF	Mean time to failure of a non-repairable part	hours
λ	Constant failure rate	failures/hour
t	Mission time: the period we need it to survive	hours
$R(t)$	Probability of no failure during t	dimensionless
A	Steady-state availability	dimensionless
AFR	Annualized failure rate: probability a part fails within a year (8,760 h)	dimensionless (often %)

Derivation. $R(t) = e^{-\lambda t}$ follows from assuming a *constant* failure rate: failures are random events with no memory of age, so the time to failure is exponentially distributed (§4.7.2) with mean $1/\lambda = \text{MTBF}$. Availability is the long-run fraction of each up–down cycle spent up: MTBF out of MTBF + MTTR. The constant-rate assumption is a simplification: real hardware follows a “bathtub curve” with higher failure rates in early life (infant mortality) and at wear-out, and is roughly constant in between.

Figure 4.7 — The up-down cycle of a repairable component.



Reading the diagram. The component alternates between two states. It stays `Up` for an exponentially distributed time with mean MTBF, then moves to `Down`, where it stays for a repair time with mean MTTR. There is no data flow or protocol; the arrows are state transitions. Availability is the fraction of time spent in `Up`. The two levers are visible: make failures rarer (longer MTBF) or make repair faster (shorter MTTR). In distributed systems the second is usually cheaper — automatic failover turns an MTTR of hours into seconds.

WORKED EXAMPLE

Worked example — Two ways to get a nine. A server has MTBF = 1,000 h and MTTR = 1 h.

1. $A = 1000/(1000 + 1) = 0.99900$ (three nines).
2. Option 1, double MTBF to 2,000 h (better hardware): $A = 2000/2001 = 0.99950$.
3. Option 2, cut MTTR to 6 min = 0.1 h (automated failover): $A = 1000/1000.1 = 0.99990$ (four nines).
4. Reliability over a 100-hour batch job: $R(100) = e^{-100/1000} = e^{-0.1} = 0.905$. Over one MTBF: $R(1000) = e^{-1} = 0.368$.

Interpretation. A 10× reduction in MTTR buys a full nine; a 2× increase in MTBF buys half of one. Note that only 37% of components survive to their MTBF: “mean time” is not “typical time”. **Engineering implication.** Invest in detection and automated recovery (health checks, failover, restarts) before investing in more expensive hardware. For long jobs, reliability matters more than availability: a 100-hour job on this server fails about 10% of the time, so it needs checkpointing.

WORKED EXAMPLE

Worked example — Disk failures in a fleet. A disk datasheet quotes MTTF = 1,000,000 h. The fleet has 10,000 disks.

1. $\text{AFR} = 1 - e^{-8760/1,000,000} = 1 - e^{-0.00876} = 0.0087 = 0.87\%$.
2. Expected failures per year: $10,000 \times 0.0087 = 87$, about 1.7 per week.

Interpretation. A “million-hour” disk still fails nearly twice a week in a 10,000-disk fleet. **Engineering implication.** Field studies have found real-world replacement rates substantially above datasheet AFRs — Schroeder and Gibson report annual replacement rates typically exceeding 1% and commonly 2–4% across the large populations they studied [Schroeder and Gibson 2007]. Plan with measured fleet data, and use a pessimistic AFR (for example 2–4%) when you have none.

4.16 Durability and replication

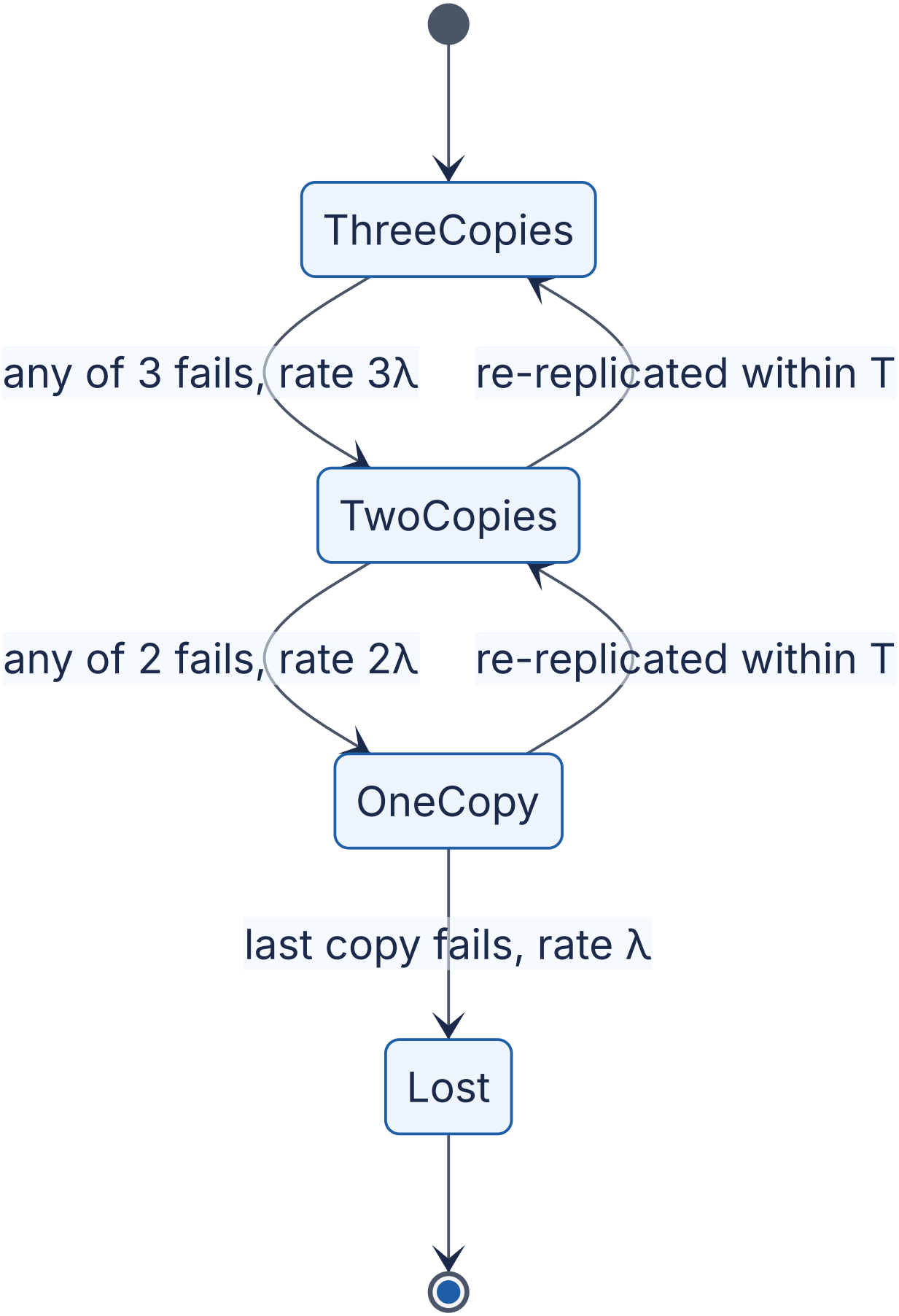
4.16.1 Durability as a probability

DEFINITION

Definition — Durability. The probability that data, once acknowledged as written, is not lost over a given period (usually a year). It is quoted in nines like availability: “eleven nines of durability” means an annual loss probability of 10^{-11} per object.

Durability and availability are different: a storage system can be unavailable for an hour (you cannot read your data) without losing anything, and it can be available while silently having lost an object. Data is lost when *every* copy is destroyed before the system notices and repairs the first loss. Durability is therefore a race between failure and repair.

Figure 4.8 — The replica-loss race for three-way replication.


ThreeCopies
any of 3 fails, rate 3λ
re-replicated within T
TwoCopies
any of 2 fails, rate 2λ
re-replicated within T
OneCopy
last copy fails, rate λ
Lost

Reading the diagram. States count the surviving copies of one piece of data. Each copy fails independently at rate λ, so the rate of leaving a state is (copies) × λ. A failed copy is re-created from a survivor in an average repair time T, which is dominated by detection time and by copy bandwidth (§4.5.1). `Lost` is absorbing: there is no way back. Data is lost only if two more failures occur while the group is still degraded. The failure point is a long repair window; the scaling lever is parallel re-replication, which shrinks T.

FORMULA

Approximate annual probability of losing an n-way replicated item

$$P_{\text{loss/yr}} \approx n!\ \lambda^{n}\ T^{n-1} \times H \qquad \text{valid when } \lambda T \ll 1$$

Symbol	Meaning	Units
n	Number of replicas	count
λ	Per-replica failure rate (AFR / 8,760 for small AFR)	failures/hour
T	Mean time to detect and re-replicate a lost copy	hours
H	Hours per year (8,760)	hours/year
$P_{\text{loss/yr}}$	Annual probability that all n replicas are lost	dimensionless

Derivation (three replicas). First failures in the group occur at rate 3λ. Given a first failure, one of the two survivors fails before repair with probability about $2\lambda T$, and then the last survivor fails before *its* repair completes with probability about λT. Multiply: loss rate $\approx 3\lambda \times 2\lambda T \times \lambda T = 6\lambda^{3}T^{2}$ per hour, and multiply by 8,760 for a year. The same chain for n replicas gives $n!\,\lambda^{n}T^{n-1}$. This is the standard approximation from a Markov model with one repair at a time and independent failures.

WORKED EXAMPLE

Worked example — How many replicas, and how fast must repair be? Disks have AFR = 2%, so $\lambda = 0.02/8760 = 2.28 \times 10^{-6}$ per hour. Repair takes $T = 24$ h, so $\lambda T = 5.48 \times 10^{-5}$.

1. One replica: $P \approx \text{AFR} = 2 \times 10^{-2}$.
2. Two replicas: $2\lambda^2 T \times 8760 = 2 \times 0.02 \times \lambda T = 0.04 \times 5.48 \times 10^{-5} = 2.2 \times 10^{-6}$.
3. Three replicas: $6\lambda^3 T^2 \times 8760 = 3 \times 0.02 \times 2(\lambda T)^2 = 0.06 \times 2 \times 3.00 \times 10^{-9} = 3.6 \times 10^{-10}$.
4. Fleet view: with 10^6 independent three-replica groups, expected losses are $10^6 \times 3.6 \times 10^{-10} = 3.6 \times 10^{-4}$ groups per year.
5. Slower repair, $T = 168$ h (one week): $\lambda T = 3.84 \times 10^{-4}$; three-replica loss $= 0.06 \times 2 \times (3.84 \times 10^{-4})^2 = 1.8 \times 10^{-8}$, which is $7^2 = 49$ times worse.

Interpretation. Each extra replica multiplies durability by roughly $1/(\lambda T)$, here about four orders of magnitude, and for three replicas loss probability grows with the *square* of the repair time. **Engineering implication.** Fast, parallel re-replication is as important as the replica count. These numbers are, however, optimistic upper bounds on durability: they assume independence. In practice correlated failures — a rack losing power, a bad firmware batch, a datacenter-wide event, a software bug that deletes all copies — dominate data loss, which is why durable stores spread replicas across failure domains, keep backups outside the replication system, and checksum data to catch silent corruption (§13.12, §39.9).

Table 4.8 — Annual loss probability per item (AFR 2%, independent failures).

Replicas	T = 1 h	T = 24 h	T = 168 h
1	2×10^{-2}	2×10^{-2}	2×10^{-2}
2	9.1×10^{-8}	2.2×10^{-6}	1.5×10^{-5}
3	6.3×10^{-13}	3.6×10^{-10}	1.8×10^{-8}

INDUSTRY EXAMPLE

Correlated failures in real storage fleets. Ford et al. analyzed a year of availability data from Google's distributed storage and found that failures are often correlated — many nodes failing together in bursts, for example after power or network events — and that models assuming independence can overestimate durability by orders of magnitude; their models incorporate correlated failures and placement across failure domains [Ford et al. 2010]. Cidon et al. showed that with random replica placement across a large cluster, almost *any* simultaneous failure of three nodes destroys some chunk, and proposed limiting the number of distinct replica sets ("copysets") to reduce the frequency of loss events [Cidon et al. 2013]. Amazon S3 documentation states that S3 is designed for 99.999999999% (eleven nines) annual durability of objects, achieved by storing data redundantly across multiple devices and facilities [AWS S3 documentation]. Replication and erasure coding are compared in §15.6. *Evidence: Primary* — [Ford et al. 2010; Cidon et al. 2013; AWS S3 documentation]

4.17 Latency numbers every engineer should know

4.17.1 The table and its history

A table of hardware and network latencies popularized by Jeff Dean around 2009–2010, building on a similar table by Peter Norvig, is the standard estimation reference [Dean 2009; Norvig]. Its value lies in the *ratios* between rows, which change far more slowly than the absolute numbers. Since then SSDs have replaced disks for latency-sensitive data and datacenter networks have moved to 25–100 Gbps and beyond, while memory latency has barely moved. Treat every number as an order-of-magnitude anchor and measure your own hardware.

Table 4.9 — Latency reference numbers (approximate; era-dependent).

Operation	Classic value (as circulated c. 2010)	Approximate order of magnitude, as of 2025	Notes
L1 cache reference	0.5 ns	~1 ns	A few CPU cycles
Branch mispredict	5 ns	~5 ns	Pipeline flush
L2 cache reference	7 ns	~3–5 ns	
Mutex lock/unlock (uncontended)	25 ns	~10–25 ns	Contended locks cost far more
Main memory reference	100 ns	~80–120 ns	Has changed little in 15 years
Compress 1 KB with a fast compressor (Zippy/Snappy)	3 µs	~1–3 µs	
Send 1 KB over a network link	10 µs (1 Gbps)	~1 µs (10 Gbps)	Serialization time only
Read 4 KB randomly from SSD	150 µs (SATA era)	~50–100 µs (NVMe)	
Read 1 MB sequentially from memory	250 µs	~50–100 µs	One thread; depends on bandwidth per core
Round trip within one datacenter	500 µs	~50–500 µs	Kernel bypass can be much lower
Read 1 MB sequentially from SSD	1 ms	~0.15–0.3 ms (NVMe)	
Disk (HDD) seek	10 ms	~4–10 ms	Mechanical; unchanged
Read 1 MB sequentially from HDD	20 ms	~5 ms	~200 MB/s sequential
Packet round trip California ↔ Netherlands	150 ms	~140–150 ms	Bounded by the speed of light in fiber

The modern column is this book's order-of-magnitude estimate for commodity server hardware as of 2025, not a benchmark result; ranges reflect variation across hardware generations. Colin Scott's interactive version of the table extrapolates the values year by year [Scott].

Physics sets the floor

Light in optical fiber travels at about two-thirds of its vacuum speed, roughly 200,000 km/s, or 5 µs per km. The great-circle distance from California to the Netherlands is about 8,800 km, so the round trip cannot take less than $2 \times 8{,}800 / 200{,}000 = 0.088$ s $= 88$ ms, even with perfect routing. No engineering removes that floor; only moving data closer (caching, CDNs, replicas in more regions; Chapters 21 and 44) does.

WORKED EXAMPLE

Worked example — Reading 1 GB from four places, using the classic column.

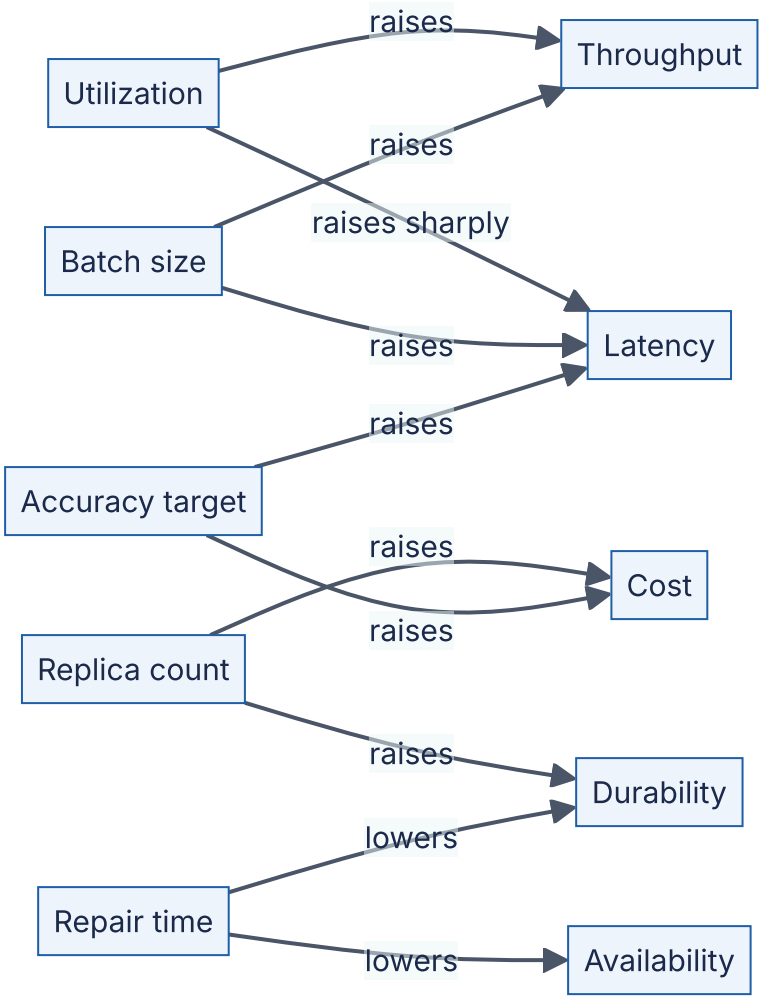
1. From memory: $1,000 \times 250\ \mu s\ = 0.25$ s.
2. From SSD (sequential): $1,000 \times 1$ ms $= 1$ s.
3. From HDD (sequential): $1,000 \times 20$ ms $= 20$ s.
4. Over a 1 Gbps network: $8 \times 10^9 / 10^9 = 8$ s.
5. From HDD with a seek per 4 KB block (random): $250,000$ blocks $\times$ 10 ms $= 2,500$ s ≈ 42 min.

Interpretation. Memory, SSD, network and disk span two orders of magnitude for sequential access, and random disk access is another two orders slower than sequential. **Engineering implication.** Design data layouts for sequential access (log-structured storage, §13.7; columnar files, §13.13), keep hot working sets in memory (Chapter 20), and count cross-region round trips on every user-facing path: five sequential 150 ms round trips is already 750 ms.

4.18 Quantitative relationships that recur in system design

The formulas of this chapter are not independent facts. They describe six trade-offs you will meet in almost every later chapter.

Figure 4.9 — Six recurring quantitative relationships.



Reading the diagram. This is a conceptual map, not a data flow: there are no protocols or storage boundaries. Left-hand nodes are design levers, right-hand nodes are outcomes, and each arrow gives the direction of the effect. Most levers improve one outcome while worsening another — that is what makes them trade-offs. The subsections below quantify each arrow.

4.18.1 Latency ↔ throughput

In a *closed* system, where a fixed number of clients each wait for a reply before sending the next request, Little's Law gives throughput $X = N/R$: concurrency divided by latency. In an *open* system, where requests arrive regardless of how fast earlier ones finish, raising throughput toward capacity raises latency by $1/(1-\rho)$ (§4.10).

WORKED EXAMPLE

Worked example — One connection versus many. A client sends requests one at a time over one connection. Each request spends 1.9 ms on the network round trip and 0.1 ms of server processing, so the server can complete at most 10,000 requests/s, and an unloaded request takes $R = 2$ ms.

1. One connection: $X = N/R = 1/0.002 = 500$ requests/s, no matter how powerful the server is. The server is 5% utilized.
2. Sixteen concurrent connections: ignoring queueing, $X = 16/0.002 = 8,000$ requests/s. But the server would then be 80% utilized, and by the M/M/1 approximation its 0.1 ms becomes $0.1/(1 - 0.8) = 0.5$ ms. Solving $X = 16/R$ and $R = 1.9 + 0.1/(1 - X/10,000)$ ms together gives $X \approx 7,100$ requests/s and $R \approx 2.25$ ms.
3. Sixty-four connections: the naive $64/0.002 = 32,000$ requests/s exceeds capacity, so X approaches 10,000 requests/s and Little's Law forces $R = N/X = 64/10,000 = 6.4$ ms. The extra 4.4 ms is pure queueing.

Interpretation. Throughput is limited by concurrency × (1 / latency) and by capacity; latency rises as throughput approaches capacity. **Engineering implication.** If throughput is low while servers are idle, the bottleneck is client concurrency or round trips (fix with pooling, pipelining, batching); if latency is high while servers are busy, the bottleneck is capacity.

4.18.2 Availability ↔ reliability

WORKED EXAMPLE

Worked example — More available, less reliable. System X fails once a day (MTBF = 24 h) but recovers in 1 s. System Y fails once a year (MTBF = 8,760 h) but takes a day to recover.

1. $A_X = 86,400/86,401 = 0.999988$ (almost five nines).
2. $A_Y = 8,760/8,784 = 0.99727$ (under three nines).
3. Probability of completing a 6-hour job without interruption: $R_X(6) = e^{-6/24} = 0.78$; $R_Y(6) = e^{-6/8760} = 0.9993$.

Interpretation. X is far more available; Y is far more reliable. **Engineering implication.** Interactive services with retries care about availability; long-running jobs, transactions and stateful sessions care about reliability. Choose the metric that matches what the user experiences, and design accordingly (checkpointing for jobs, fast failover for services).

4.18.3 Replication ↔ durability (and cost and write latency)

From §4.16, each extra replica improves durability by about $1/(\lambda T)$ but costs a full copy of storage and, for synchronous replication, adds the slowest acknowledgment to every write.

WORKED EXAMPLE

Worked example — Paying for nines. With AFR 2% and $T = 24$ h (Table 4.8):

1. Two replicas: 2.2×10^{-6} annual loss probability, 2× storage.
2. Three replicas: 3.6×10^{-10}, 3× storage: 50% more storage for about 6,000× better durability.
3. Synchronous write to 3 replicas waiting for all: if each replica acknowledges within 5 ms with probability 0.99, all three do with probability $0.99^3 = 0.970$ — the fan-out law of §4.8.4 applied to writes. Waiting for 2 of 3 (a quorum) instead: $3 \times 0.99^2 \times 0.01 + 0.99^3 = 0.9997$.

Interpretation. Durability is cheap in storage relative to what it buys, but synchronous replication taxes the write tail. **Engineering implication.** Quorum writes (§27.8) capture most of the durability while hiding the slowest replica; erasure coding (§15.6) reaches comparable durability with less storage at the cost of more expensive repair.

4.18.4 Accuracy ↔ latency

Many computations can trade correctness for speed: sampling, approximate algorithms, smaller models, fewer candidates. The statistical baseline is the **standard error** of a mean estimated from n samples, $\sigma/\sqrt{n}$.

FORMULA

Standard error of a sample mean

$$\mathrm{SE} = \frac{\sigma}{\sqrt{n}}$$

Symbol	Meaning	Units
σ	Standard deviation of the quantity being measured	units of the quantity
n	Number of independent samples	count
SE	Typical error of the estimated mean	units of the quantity

WORKED EXAMPLE

Worked example — The square-root tax. Latency has $\sigma = 50$ ms. A monitoring job estimates mean latency from samples.

1. $n = 100$: SE $= 50/\sqrt{100} = 5$ ms.
2. $n = 10,000$: SE $= 50/100 = 0.5$ ms.

Interpretation. Ten times the accuracy costs a hundred times the samples, and so a hundred times the processing time. **Engineering implication.** The same diminishing returns appear elsewhere: an ANN index searched more exhaustively gains recall slowly while latency grows roughly in proportion to the work done (Chapter 74); a ranking stage that scores 5,000 candidates at, say, 20 µs each spends 100 ms versus 10 ms for 500 (an illustrative cost). Decide the accuracy the product needs, then buy exactly that much.

4.18.5 GPU utilization ↔ throughput

For ML workloads, throughput is governed by how much of the accelerator's peak arithmetic is actually used. The **model FLOPs utilization (MFU)** is the achieved model FLOP/s divided by peak FLOP/s. A dense transformer forward pass costs roughly $2P$ FLOPs per token for a model with P parameters [Kaplan et al. 2020].

FORMULA

Compute-bound token throughput

$$X_{\text{tokens}} = \frac{\text{MFU} \times F_{\text{peak}}}{2P}$$

Symbol	Meaning	Units
F_{peak}	Peak accelerator throughput at the precision used	FLOP/s
MFU	Model FLOPs utilization	dimensionless (0–1)
P	Model parameters	count
$2P$	Forward-pass FLOPs per token (approximation)	FLOP/token
X_{tokens}	Tokens processed per second per accelerator	tokens/s

WORKED EXAMPLE

Worked example — Utilization is money. A hypothetical accelerator has $F_{\text{peak}} = 10^{15}$ FLOP/s (1,000 TFLOP/s, the order of magnitude of data-center GPUs at 16-bit precision as of 2024–2025). The model has $P = 7 \times 10^{9}$ parameters, so $2P = 1.4 \times 10^{10}$ FLOP/token.

1. At MFU = 40%: $X = 0.4 \times 10^{15} / 1.4 \times 10^{10} = 28,571$ tokens/s.
2. At MFU = 20%: $X = 14,286$ tokens/s.

Interpretation. Halving utilization halves throughput and doubles the cost per token on the same hardware. **Engineering implication.** Training clusters push MFU as high as possible because they are batch work. Online inference cannot run accelerators at 100% busy for the queueing reasons of §4.10 — latency explodes — so serving systems balance utilization against latency SLOs (Chapters 64, 66 and 73).

4.18.6 Batch size ↔ latency

Batching amortizes a fixed per-batch cost (kernel launch, loading weights from memory, a network round trip, a disk flush) across many items. A simple model: a batch of B items takes $T(B) = t_0 + B\,t_1$.

FORMULA

Batching model

$$T(B) = t_0 + B\,t_1 \qquad X(B) = \frac{B}{t_0 + B\,t_1} \qquad W_{\text{fill}} \approx \frac{B-1}{2\lambda}$$

Symbol	Meaning	Units
B	Batch size	items
t_0	Fixed cost per batch	seconds
t_1	Marginal cost per item	seconds/item
$T(B)$	Time to process one batch	seconds
$X(B)$	Throughput	items/second
λ	Arrival rate of items	items/second
W_{fill}	Average time an item waits for its batch to fill	seconds

WORKED EXAMPLE

Worked example — Choosing a batch size. An inference server has $t_0 = 8$ ms and $t_1 = 0.5$ ms per request; requests arrive at $\lambda = 1,000$/s.

1. $B = 1$: $T = 8.5$ ms; $X = 1/0.0085 = 118$/s — far below demand.
2. $B = 8$: $T = 12$ ms; $X = 8/0.012 = 667$/s — still below demand.
3. $B = 32$: $T = 24$ ms; $X = 32/0.024 = 1,333$/s — meets demand; average fill wait $\approx 31/2000 = 15.5$ ms, so latency $\approx 15.5 + 24 \approx 40$ ms.
4. $B = 64$: $T = 40$ ms; $X = 1,600$/s; fill wait $\approx 63/2000 = 31.5$ ms; latency ≈ 72 ms.

Interpretation. Throughput rises toward the ceiling $1/t_1 = 2,000$/s with diminishing returns, while latency rises roughly linearly with B. **Engineering implication.** Choose the *smallest* batch that meets throughput with headroom, and cap the fill wait with a timeout (dynamic batching) so that low-traffic periods do not wait for full batches. In memory-bound LLM decoding the effect is even stronger: if each decoding step must stream all $7 \times 10^9 \times 2$ bytes = 14 GB of weights from memory at a hypothetical 3 TB/s, a step takes about 4.7 ms whether it serves 1 or 32 sequences, so batch 32 yields about $32/0.0047 \approx 6,800$ tokens/s instead of about 210 (ignoring attention-cache traffic; Chapter 73).

Table 4.10 — The six relationships at a glance.

Relationship	Governing formula	One-line rule
Latency ↔ throughput	$X = N/R$; $W = S/(1-\rho)$	More concurrency raises throughput until queueing raises latency
Availability ↔ reliability	$A = \text{MTBF}/(\text{MTBF} + \text{MTTR})$; $R(t) = e^{-t/\text{MTBF}}$	Fast repair buys availability; only rare failure buys reliability
Replication ↔ durability	$P_{\text{loss}} \approx n!\lambda^n T^{n-1} H$	Each replica adds ~$1/(\lambda T)$; repair speed matters as much
Accuracy ↔ latency	$\text{SE} = \sigma/\sqrt{n}$	10× accuracy often costs 100× work
GPU utilization ↔ throughput	$X = \text{MFU} \cdot F_{\text{peak}}/(2P)$	Throughput and cost per token scale with utilization
Batch size ↔ latency	$X = B/(t_0 + Bt_1)$	Batching buys throughput with latency

4.19 Summary and revision notes

SUMMARY

- **Powers and logs.** $2^{10} \approx 10^3$; carry exponents exactly and mantissas roughly. Logarithms count halvings: binary search over 10^9 keys takes 30 steps; a B-tree with fan-out ~800 needs 4 levels, and the top levels are cached.
- **Units.** Network rates are bits/s, storage is bytes: divide by 8. KB/MB/GB are decimal, KiB/MiB/GiB binary; the gap is 7% at tera and 13% at peta. One day ≈ 86,400 s ≈ 10^5 s.
- **Probability.** $P(\text{at least one of } N \text{ fails}) = 1 - (1-p)^N \approx 1 - e^{-Np}$. Expected values add even without independence. The binomial distribution gives the chance of exactly k failures — size spares against its tail, not its mean.
- **Statistics.** Latency is right-skewed (log-normal or heavier); the mean is pulled up by the tail and describes no real request. Report percentiles with sample counts; never average percentiles — merge histograms.
- **Tail at scale.** Fan-out to N leaves: $P(\text{all fast}) = p^N$. With $N = 100$ and leaf p99, 63% of requests are slow; leaves must be engineered at p99.99 or use hedged/tied requests.
- **Linear algebra.** Cosine similarity compares directions; a matrix multiply costs $2mnk$ FLOPs; brute-force vector search and batch-1 inference are memory-bound; batching raises arithmetic intensity.
- **Queueing.** M/M/1: $W = 1/(\mu - \lambda) = S/(1-\rho)$. Latency doubles each time the remaining headroom halves. Shared queues (M/M/c) beat separate queues on identical hardware.
- **Little's Law.** $L = \lambda W$ on any stable boundary: size thread pools and connection pools from rate × hold time, then add queueing headroom; convert backlog to delay with $W = L/\lambda$.
- **Scaling laws.** Amdahl: ceiling $1/(1-p)$ for fixed problems. Gustafson: near-linear for growing problems. USL adds coherency β, producing a throughput peak at $N^* = \sqrt{(1-\alpha)/\beta}$ and retrograde scaling beyond it.
- **Availability.** Series multiplies availabilities (always worse than the weakest); parallel multiplies unavailabilities; k-of-n is binomial. Four nines = 4.3 min/month and demands automated recovery. All of it assumes independence.
- **Reliability.** $A = \text{MTBF}/(\text{MTBF} + \text{MTTR})$; $R(t) = e^{-t/\text{MTBF}}$. Cutting MTTR is usually the cheapest nine. Only 37% of parts survive to their MTBF.
- **Durability.** Annual loss $\approx n!\lambda^n T^{n-1} \times 8760$: each replica buys ~$1/(\lambda T)$, and repair time enters squared for three replicas. Correlated failures dominate real loss.
- **Latency numbers.** Memory ~100 ns, NVMe random read ~tens of µs, data-center round trip ~0.1–0.5 ms, cross-continent round trip ~100+ ms (physics floor ~5 µs/km of fiber). Use ratios; date your numbers.

- **Trade-offs.** Throughput costs latency near saturation; fast repair buys availability, not reliability; replicas buy durability with storage and write tail; accuracy costs work at a square-root rate; utilization sets cost per token; batching buys throughput with latency.

4.20 Knowledge checkpoint

1. Roughly how many bytes is 2^{40} bytes, and what is its binary-prefix name?
2. How many levels does a B-tree with fan-out 500 need to index 10^9 keys?
3. A 25 Gbps link must copy 50 TB. What is the minimum transfer time?
4. Each of 500 independent servers fails on a given day with probability 0.002. What is the probability that at least one fails today, and the expected number of failures?
5. Why is the median often a better summary of latency than the mean?
6. Using the nearest-rank method, what is the p90 of 1, 2, 3, 4, 5, 6, 7, 8, 9, 100?
7. A root service fans out to 20 leaves, each fast with probability 0.995. What fraction of root requests are fast?
8. Compute the cosine similarity of $(3, 4)$ and $(4, 3)$.
9. An M/M/1 server has a service time of 5 ms. What is the mean response time at 60% and at 95% utilization?
10. A service processes 400 RPS and each request holds a thread for 150 ms. What is the average number of busy threads?
11. A job is 90% parallelizable. What is its maximum speedup under Amdahl's Law, and its speedup on 10 cores?
12. In the USL, what physical effect does β model, and what happens to throughput when $\beta > 0$ and N grows large?
13. A request path calls five services in series, each 99.95% available. What is the path availability, and how much downtime per 30-day month does it allow?
14. A component has MTBF = 500 h and MTTR = 30 min. What is its availability?
15. Why does halving the re-replication time improve the durability of a three-replica system by about 4×?
16. Why is a round trip between California and the Netherlands never faster than about 88 ms?

Answers

1. $2^{40} = 1,099,511,627,776 \approx 1.1 \times 10^{12}$ bytes; one tebibyte (TiB).
2. $\log_{500} 10^9 = 9/\log_{10} 500 = 9/2.699 = 3.33$, so 4 levels.
3. $50 \times 10^{12} \times 8/(25 \times 10^9) = 16,000$ s ≈ 4.4 h (before protocol overhead).
4. $1 - 0.998^{500} \approx 1 - e^{-1} = 0.632$; expected failures $500 \times 0.002 = 1$.
5. Latency is right-skewed; a few very slow requests pull the mean up so that it describes no actual request, while the median reflects the typical request and is ro-

bust to outliers.

6. Rank $\lceil 0.9 \times 10 \rceil = 9$; the 9th value is 9.
7. $0.995^{20} = 0.905$, so about 90.5% fast and 9.5% slow.
8. Dot $= 12 + 12 = 24$; both lengths are 5; cosine $= 24/25 = 0.96$.
9. $W = S/(1 - \rho)$: $5/0.4 = 12.5$ ms at 60%; $5/0.05 = 100$ ms at 95%.
10. $L = 400 \times 0.15 = 60$ threads (size the pool above this for queueing headroom).
11. Ceiling $1/0.1 = 10$; on 10 cores $1/(0.1 + 0.09) = 5.26$.
12. Coherency: pairwise coordination such as cache coherence, consensus or data exchange, growing like N^2. Throughput peaks at N^* and then declines (retrograde scaling).
13. $0.9995^5 = 0.99750$; downtime $0.0025 \times 43,200 = 108$ min per month.
14. $500/500.5 = 0.999$ (three nines).
15. For three replicas annual loss probability is proportional to T^2, so halving T divides it by $2^2 = 4$.
16. The great-circle distance is about 8,800 km; light in fiber travels about 200,000 km/s; $2 \times 8,800/200,000 = 88$ ms. Real routes are longer, so measured RTTs are higher.

4.21 Interview questions

Basic

Q1. Your service averages 50 ms latency. Is that enough to say it meets a "p99 under 200 ms" SLO? Answer. No. The mean says nothing about the tail: a right-skewed distribution can have a 50 ms mean with several percent of requests above 200 ms. You need the p99 itself, computed from a histogram over a window with enough samples (hundreds of requests in the tail), and you should check it at fine time resolution so that bad minutes are not averaged away.

Q2. Estimate the average and peak RPS for 500 million requests per day. Answer. $5 \times 10^8/86,400 \approx 5,800$ RPS on average (or ~5,000 using 10^5 s/day). With an assumed peak-to-average ratio of 2–3, plan for roughly 12,000–17,000 RPS, and say the ratio is an assumption to be replaced by measurement.

Q3. Two independent replicas each have 99% availability. What is the availability of the pair, and what assumptions are you making? Answer. $1 - 0.01^2 = 99.99\%$. Assumptions: failures are independent (no shared rack, power, network, deployment or software bug), failover is instantaneous and automatic, and either replica alone can carry the full load. Each assumption, if false, lowers the real number.

Intermediate

Q4. A database server is at 70% CPU and the team wants to consolidate to fewer servers to reach 90%. What would you tell them? Answer. In an M/M/1 approximation, response time scales as $1/(1-\rho)$: going from 0.7 to 0.9 triples queueing-inclusive latency ($3.3\times$ to $10\times$ the service time), and the tail grows proportionally (M/M/1 p99 $\approx 4.6\times$ mean). For a latency-sensitive OLTP database that is usually unacceptable; for batch analytics it may be fine. I would model the latency at the forecast *peak* after consolidation, and keep enough headroom to absorb a replica failure as well.

Q5. How would you size the connection pool for 40 application instances that together run 3,000 queries/s with a mean connection hold time of 4 ms? Answer. Little's Law: $L = 3,000 \times 0.004 = 12$ connections busy on average across the fleet, 0.3 per instance. A small per-instance pool (2–4) or a shared pooler with on the order of 20–40 server connections gives queueing headroom. The important parts are a query timeout and a cap at the database, because if hold time rises to 200 ms during an incident, demand becomes 600 connections and an uncapped fleet would overload the database.

Q6. A search service fans out to 50 partitions. Leaf p99 is 20 ms. What will the root p99 roughly look like, and what would you do? Answer.
$P(\text{all under 20 ms}) = 0.99^{50} = 0.605$, so 40% of root requests exceed 20 ms; the root p99 will be set by roughly the leaf p99.98 ($0.99^{1/50} \approx 0.9998$), which may be many times 20 ms. Remedies: reduce leaf variability (GC tuning, isolation from background work), hedged or tied requests, per-leaf deadlines with partial results, and fewer, larger partitions if that is feasible.

Advanced

Q7. Load tests show throughput rising to 16 nodes, flat at 32, and falling at 64. Explain using a model, and say what you would investigate. Answer. This is retrograde scaling, which Amdahl's Law cannot produce but the USL can: a non-zero coherency term $\beta N(N-1)$ grows quadratically and eventually dominates. Fitting α and β to the measurements gives $N^{*} = \sqrt{(1-\alpha)/\beta}$ near 32. I would look for all-to-all interactions: distributed locks, cache invalidation broadcasts, gossip or membership traffic, cross-partition transactions, hot shared rows. The remedy is to partition so that most work needs no coordination, not to add nodes.

Q8. Design the replication policy for a storage system targeting 10^{-9} annual loss probability per object with disks at AFR 3%. Answer. $\lambda = 0.03/8760 = 3.4 \times 10^{-6}$/h. Two replicas with $T = 1$ h give $2 \times 0.03 \times \lambda T \approx 2 \times 10^{-7}$ — not enough. Three replicas with $T = 24$ h give $3 \times 0.03 \times 2(\lambda T)^2 = 0.09 \times 2 \times (8.2 \times 10^{-5})^2 \approx 1.2 \times 10^{-9}$ — borderline, so I would cut repair time (parallel re-replication across many nodes, fast detection) to a few hours, which gives roughly 10^{-11}–10^{-10}. Then I would state the caveat: the independent-failure model is opti-

mistic, so I would place replicas in distinct failure domains, consider erasure coding across zones for cost, add end-to-end checksums and scrubbing for silent corruption, and keep backups against logical deletion.

Q9. An LLM serving team reports 25% MFU and asks whether to raise batch size. How do you reason about it? Answer. Throughput scales with MFU, so raising utilization lowers cost per token. In the decode phase, each step is memory-bandwidth-bound (all weights are streamed per step), so larger batches amortize that fixed cost and raise throughput nearly linearly until compute or KV-cache memory becomes the limit. But per-request latency rises: batch formation time grows with B/λ, and step time grows once compute-bound. I would plot throughput and p99 time per output token against batch size, pick the largest batch that keeps p99 within the SLO, use dynamic batching with a maximum wait, and keep utilization below the knee of the queueing curve for bursty traffic.

4.22 Practical exercises

1. **Percentiles versus means.** Modify the §4.8.2 program to use $\sigma = 0.5$ and $\sigma = 1.5$ for the log-normal. Record mean, p50, p99 and p99.9 for each, and explain how the ratio p99/p50 changes. Check against $e^{2.326\sigma}$.
2. **Coordinated omission.** Write a load generator that sends a request, waits for the reply, then sends the next (closed loop), against a simulated server that pauses for 1 s every 10 s. Compare the p99 it records with that of an open-loop generator that sends at a fixed rate and measures latency from the *intended* send time.
3. **M/M/1 versus M/D/1.** Change `simulate_mm1` so that service times are constant (always $1/\mu$). Verify that mean waiting time in the queue halves, as the Pollaczek–Khinchine formula predicts.
4. **Pool sizing.** Using `erlang_c`, find the smallest thread pool for $\lambda = 500$ RPS and mean hold time 40 ms such that fewer than 1% of requests wait for a thread. Then repeat for a hold time of 200 ms.
5. **Availability of your own system.** List every synchronous dependency on the critical path of a service you know, estimate each one's availability, and compute the series availability with the calculator. Which single change raises it most?
6. **USL fitting.** Measure throughput of a local service (or a multithreaded program) at 1, 2, 4, 8, 16 and 32 threads. Fit α and β by minimizing squared error over a grid, and predict N^*.

4.23 Design challenges

1. **Fan-out search tier.** You must build a search tier over 2 billion documents with a root p99 of 150 ms. Each leaf can hold 20 million documents and has a p50 of 15 ms

and a p99 of 60 ms. Determine the number of leaves, the probability that a root request waits on at least one leaf slower than 60 ms, and a design (replication, hedging, deadlines, partial results) that meets the SLO. Show the arithmetic for every step.

2. **Four-nines checkout.** A checkout path calls an API gateway, an order service, a payment service and a database. The payment provider offers 99.95%. Can the path meet 99.99%? If not, what architectural changes (asynchronous capture, retries with idempotency keys, a second provider in parallel) change the arithmetic, and what new failure modes do they introduce?
3. **Durable object store on a budget.** Design the redundancy scheme for 10 PB of user data with a target of 10^{-11} annual loss probability per object, using drives with an assumed 2–4% AFR and a 10 Gbps network per node. Compute repair times, compare three-way replication with an erasure code of your choice (see §15.6 for the arithmetic), and identify which correlated failures your model ignores.

4.24 References

1. Amdahl, G. M. “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities.” *AFIPS Spring Joint Computer Conference Proceedings*, vol. 30, pp. 483–485, 1967.
2. Amazon Web Services. *Amazon S3 User Guide — Data protection in Amazon S3*. AWS documentation, accessed 2026.
3. Bayer, R., McCreight, E. “Organization and Maintenance of Large Ordered Indices.” *Acta Informatica* 1(3):173–189, 1972.
4. Beyer, B., Jones, C., Petoff, J., Murphy, N. R. (eds.). *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, 2016. https://sre.google/sre-book/table-of-contents/
5. Cidon, A., Rumble, S., Stutsman, R., Katti, S., Ousterhout, J., Rosenblum, M. “Copysets: Reducing the Frequency of Data Loss in Cloud Storage.” *USENIX Annual Technical Conference (ATC)*, 2013.
6. Dean, J. “Designs, Lessons and Advice from Building Large Distributed Systems.” Keynote, *LADIS Workshop*, 2009.
7. Dean, J., Barroso, L. A. “The Tail at Scale.” *Communications of the ACM* 56(2):74–80, 2013.
8. Ford, D., Labelle, F., Popovici, F. I., Stokely, M., Truong, V.-A., Barroso, L., Grimes, C., Quinlan, S. “Availability in Globally Distributed Storage Systems.” *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2010.
9. Gunther, N. J. *Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable Applications and Services*. Springer, 2007.
10. Gustafson, J. L. “Reevaluating Amdahl’s Law.” *Communications of the ACM* 31(5):532–533, 1988.

11. Harchol-Balter, M. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press, 2013.
12. International Electrotechnical Commission. *IEC 80000-13: Quantities and units — Part 13: Information science and technology*. IEC, 2008.
13. Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., Amodei, D. “Scaling Laws for Neural Language Models.” arXiv:2001.08361, 2020. https://arxiv.org/abs/2001.08361
14. Kleinrock, L. *Queueing Systems, Volume 1: Theory*. Wiley, 1975.
15. Little, J. D. C. “A Proof for the Queuing Formula: L = λW.” *Operations Research* 9(3):383–387, 1961.
16. Norvig, P. “Teach Yourself Programming in Ten Years.” Essay with table of approximate timings. https://norvig.com/21-days.html
17. Schroeder, B., Gibson, G. A. “Disk Failures in the Real World: What Does an MTTF of 1,000,000 Hours Mean to You?” *USENIX Conference on File and Storage Technologies (FAST)*, 2007.
18. Scott, C. “Latency Numbers Every Programmer Should Know” (interactive, extrapolated by year). https://colin-scott.github.io/personal_website/research/interactive_latency.html
19. Tene, G. *HdrHistogram: A High Dynamic Range Histogram*. http://hdrhistogram.org/

4.25 Open verification items

- Dean and Barroso (2013) hedged-request benchmark figures (p99.9 from 1,800 ms to 74 ms with ~2% extra requests, 1,000 keys over 100 BigTable servers): confirm exact numbers and wording against the CACM article.
- Schroeder and Gibson (2007) field replacement rates (“typically exceeding 1%, 2–4% common”): confirm against the paper’s abstract.
- Table 4.9 “classic” column: the circulated versions differ (e.g., 1 KB vs 2 KB network send; 20 ms vs 30 ms for 1 MB sequential disk read; SSD rows were added later than Dean’s 2009 talk). Verify each value against Dean’s LADIS 2009 slides and Norvig’s essay, and attribute row by row.
- Table 4.9 “as of 2025” column is the author’s order-of-magnitude estimate; validate against current vendor specifications or measurements before publication.
- AWS S3 durability statement: confirm current wording and add the documentation URL.
- Gunther’s notation (σ, κ in later work vs α, β in *Guerrilla Capacity Planning*): confirm which notation the 2007 book uses.
- IEC 80000-13: confirm the current edition year.
- Copy runnable programs to `code/ch04/` per style guide §8 (not done by this author; out of scope for this file).

Part II — Computer Science Foundations

Chapter 5 — Computer Architecture

> **Level:** 1 · **Track:** Shared · **Estimated reading time:** 3 h

Every distributed system, however large, is built out of single machines. When a database is “slow”, a queue “backs up” or a service “needs more threads”, the real cause almost always comes down to one of a small number of physical facts: it takes about 100 ns to fetch a value from main memory, a spinning disk can perform only about 100 random reads per second, a network card can only move so many bytes per second, and a CPU core can do useful work only when its data is close at hand. This chapter teaches those facts from the ground up. You need no prior knowledge of hardware. By the end you will be able to explain *why* system designers favor sequential I/O, batching, caching and data locality, and you will be able to back those preferences with numbers.

Throughout the chapter we use decimal units for estimation (KB, MB, GB = 10^3, 10^6, 10^9 bytes) and binary units where hardware is built in powers of two (KiB, MiB, GiB = 2^{10}, 2^{20}, 2^{30} bytes), for example cache sizes and page sizes. Hardware numbers are labeled “as of , approximate”. They change every product generation, so learn the **orders of magnitude and the ratios**, which are far more stable than any single figure.

5.1 Learning objectives

After this chapter you should be able to:

- Describe what a CPU core does on every clock cycle, and explain pipelining, branch prediction, superscalar and out-of-order execution, and simultaneous multithreading (hyperthreading) in plain language.
- Calculate CPU execution time from instruction count, cycles per instruction and clock rate, and estimate the cost of branch mispredictions.
- Explain how L1, L2 and L3 caches, cache lines and associativity work, and calculate average memory access time for a multi-level cache.
- Explain cache coherence using the MESI protocol, recognize false sharing in code, and fix it.
- Draw the memory hierarchy from registers to archival storage with dated latency, capacity and cost figures.
- Calculate hard disk access time from seek time, rotational latency and transfer time, and explain why random I/O on a disk is hundreds of times slower than sequential I/O.

- Explain how flash SSDs store data (pages, blocks, erase-before-write), what write amplification is, and why NVMe outperforms SATA.
- Compare programmed I/O, interrupts, DMA, polling and kernel bypass.
- Diagnose whether a workload is CPU-bound or I/O-bound and size a thread pool accordingly.
- Estimate how long it takes to scan 1 GB from RAM, SSD, HDD or the network, and explain the difference between latency and bandwidth.
- Explain NUMA and its effect on multi-socket servers.
- Explain what SIMD and hardware accelerators do and when they matter.
- Map hardware facts to system design decisions: sequential I/O, batching, caching, data locality and queue depth.

5.2 Prerequisites

- Powers of two and powers of ten — §4.3 (cache sizes, addresses and capacities are powers of two).
- Units: bits, bytes, rates and prefixes — §4.5 (we convert between ns, µs, ms and between MB/s and Gbit/s constantly).
- Little's Law — §4.11 (we use it to relate queue depth, latency and throughput for memory and SSDs).
- Latency numbers every engineer should know — §4.17 (this chapter explains the hardware *behind* those numbers).
- Basic programming in any language; the code examples use C and Python, and every line that matters is explained.

5.3 The CPU: cores, registers, clock and the instruction pipeline

5.3.1 What a CPU does

> DEFINITION
>
> **Definition — Central processing unit (CPU).** The hardware that executes a program's **instructions**: simple operations such as "add these two numbers", "load a value from memory address X", or "if this value is zero, jump to instruction Y". A modern CPU package contains several independent **cores**, each of which can execute its own stream of instructions.

First-principles explanation

Imagine a restaurant kitchen. The **chef** is a CPU core. The **recipe** is the program: a numbered list of small steps. The chef's **two hands and the cutting board** are the **regis-**

ters, a tiny number of slots holding the values being worked on right now. The **counter beside the stove** is the cache, the **walk-in fridge** at the back of the kitchen is main memory (RAM), and the **supplier's warehouse across town** is the disk. The chef works at astonishing speed with whatever is in hand, but any trip to the fridge or the warehouse stops the cooking. Most of the techniques in this chapter are tricks for keeping the chef's hands full.

The analogy also explains cores: a kitchen with eight chefs, each following their own recipe, can prepare eight dishes at once — provided they do not keep reaching for the same pan.

Why does this exist?

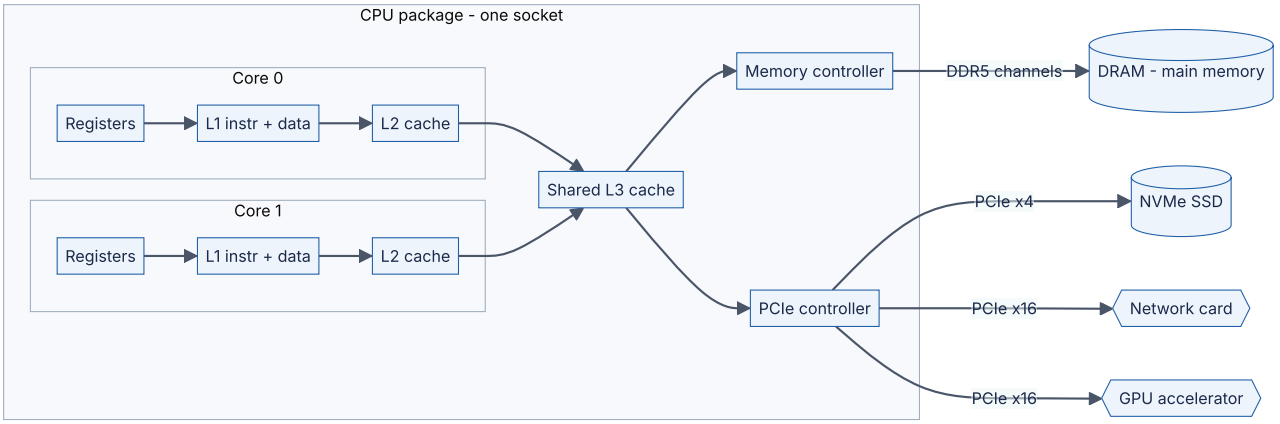
Early computers ran one instruction at a time and memory was about as fast as the processor. Since the 1980s processor speed has grown much faster than memory speed — the so-called **memory wall** [Hennessy & Patterson 2017, ch. 2]. Every feature in a modern core (caches, pipelines, prediction, out-of-order execution) exists to keep a very fast execution engine busy despite slow memory and unpredictable control flow.

How it works internally

At its simplest a core repeats the **fetch–decode–execute cycle**:

1. **Fetch.** Read the next instruction from memory at the address held in the **program counter** (PC) register.
2. **Decode.** Work out what the instruction means: which operation, which registers, which memory address.
3. **Execute.** Perform the operation in an execution unit — the **arithmetic logic unit** (ALU) for integer maths, the floating-point unit for decimals, the load/store unit for memory.
4. **Memory access.** If the instruction reads or writes memory, do so (via the caches, §5.4).
5. **Write back.** Store the result in the destination register and advance the program counter (or jump, if the instruction was a branch).

Figure 5.1 — Inside a single-socket server.



Reading the diagram. The package (the chip you hold in your hand) contains several **cores**; we show two, but server parts in 2025 carry anywhere from 8 to well over 100. Each core has private **registers**, a private **L1 cache** split into an instruction half and a data half, and usually a private **L2 cache**. All cores share an **L3 cache**, which connects to the **memory controller** and the **PCIe controller**. The memory controller drives several **DDR5 channels** to the DRAM modules (main memory). The PCIe controller connects storage (NVMe SSDs), the network card and accelerators such as GPUs over **PCIe lanes** (x4 = four lanes, x16 = sixteen). Arrows point outward from the core because the core initiates requests; data flows back along the same path. All of these paths are synchronous from the core's point of view (an instruction waits for its load), except device I/O, which is asynchronous via DMA and interrupts (§5.7). Every arrow is a potential bottleneck: the further right you go, the higher the latency and the lower the bandwidth per byte of capacity. The storage boundary that matters most is the one between DRAM (volatile, lost on power failure) and the SSD (persistent). The machine scales up by adding cores, memory channels, sockets (§5.10) and PCIe devices; beyond that you scale out across machines, which is the subject of most of this book.

5.3.2 Registers, the clock and the "iron law" of performance

DEFINITION

Definition — Register. A tiny, extremely fast storage slot inside a core, typically 64 bits wide for general-purpose registers. x86-64 exposes 16 general-purpose integer registers to programs; ARMv8-A (AArch64) exposes 31. Reading a register costs essentially nothing beyond the instruction that uses it.

DEFINITION

Definition — Clock rate. The frequency at which a core's internal circuitry steps from one state to the next, measured in hertz (cycles per second). A 4 GHz core has a **cycle time** of $1 / (4 \times 10^9)$ s = 0.25 ns.

The clock is like a metronome in the kitchen: on every tick each part of the core can advance its work by one step. For decades chip makers raised clock rates every year. Around 2005 this stopped: higher frequencies needed more voltage, and power (and therefore heat) grows faster than linearly with voltage and frequency. Clock rates have hovered around 2–6 GHz since (as of 2025, approximate), so the industry turned to doing **more work per cycle** and putting **more cores** on each chip [Hennessy & Patterson 2017, ch. 1]. This “end of free speed-ups” is why concurrency and horizontal scaling dominate modern system design.

How long a program takes is governed by three numbers multiplied together.

FORMULA

CPU execution time (the “iron law”)

$$T_{CPU} = N_{instr} \times CPI \times t_{cycle} = \frac{N_{instr} \times CPI}{f}$$

Symbol	Meaning	Units
T_{CPU}	CPU time to run the program	seconds
N_{instr}	Number of instructions executed	instructions
CPI	Average clock cycles per instruction (its inverse is IPC, instructions per cycle)	cycles/instruction
t_{cycle}	Duration of one clock cycle	seconds/cycle
f	Clock frequency, $1/t_{cycle}$	cycles/second (Hz)

WORKED EXAMPLE

Worked example — Running a program and paying for mispredicted branches.

A program executes $N_{instr} = 4 \times 10^9$ instructions on a 4 GHz core ($t_{cycle} = 0.25$ ns) at an average CPI of 0.5 (the core completes two instructions per cycle on average).

1. $T_{CPU} = 4 \times 10^9 \times 0.5 \times 0.25 \times 10^{-9}$ s $= 0.5$ s.

Now suppose 20% of instructions are branches, 5% of those branches are mispredicted (§5.3.4), and each misprediction costs 15 cycles.

2. Extra CPI from mispredictions $= 0.20 \times 0.05 \times 15 = 0.15$ cycles per instruction.
3. New CPI $= 0.5 + 0.15 = 0.65$.
4. New $T_{CPU} = 4 \times 10^9 \times 0.65 \times 0.25 \times 10^{-9} = 0.65$ s.

Cross-check: $4 \times 10^9 \times 0.2 \times 0.05 = 4 \times 10^7$ mispredictions $\times$ 15 cycles $= 6 \times 10^8$ cycles $= 6 \times 10^8 \times 0.25$ ns $= 0.15$ s extra. ✓

Interpretation. One branch in a hundred instructions going the “wrong way” costs 30% of the run time. **Engineering implication.** Raw clock speed is only one of three factors; CPI — which depends on caches, branches and dependencies — usually varies far more between two versions of the same code than clock rate varies between two CPUs.

5.3.3 The instruction pipeline

DEFINITION

Definition — Instruction pipeline. A technique in which the core splits instruction execution into stages and works on different stages of several consecutive instructions at the same time, like an assembly line.

First-principles explanation

A laundry has a washer, a dryer and a folding table. Doing one load start-to-finish before starting the next wastes the washer while the dryer runs. Instead you start load 2 in the washer the moment load 1 moves to the dryer. Each load still takes as long as before (its **latency** is unchanged), but a load *finishes* far more often (its **throughput** rises). A CPU pipeline is the same idea applied to instructions. This distinction between latency and throughput recurs at every level of system design (§5.12).

How it works internally

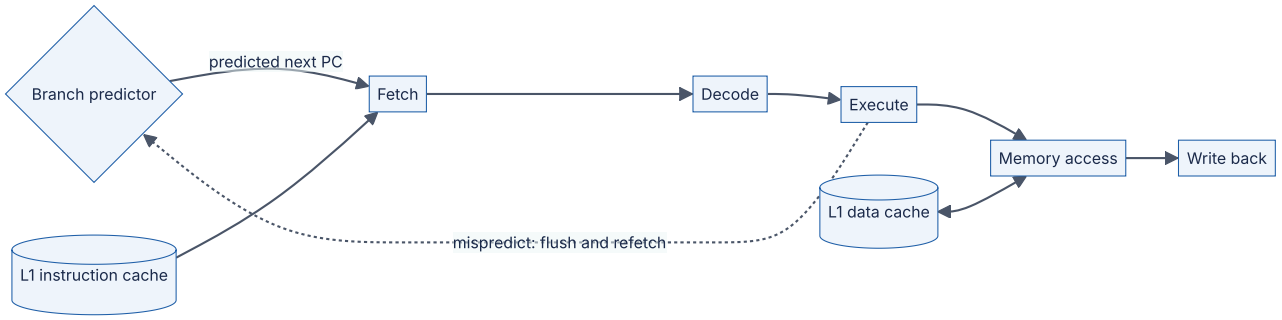
The classic teaching pipeline has five stages: instruction fetch (IF), instruction decode (ID), execute (EX), memory access (MEM) and write back (WB) [Hennessy & Patterson 2017, appendix C]. Real cores have between roughly 10 and 20 stages (as of 2025, approximate), but the principle is identical.

Table 5.1 — Five instructions flowing through a five-stage pipeline.

Cycle →	1	2	3	4	5	6	7	8	9
Instruction 1	IF	ID	EX	MEM	WB				
Instruction 2		IF	ID	EX	MEM	WB			
Instruction 3			IF	ID	EX	MEM	WB		
Instruction 4				IF	ID	EX	MEM	WB	
Instruction 5					IF	ID	EX	MEM	WB

Without pipelining, five instructions would take 5 × 5 = 25 cycles. With it they take 9 cycles, and once the pipeline is full one instruction completes **every** cycle. Each individual instruction still takes 5 cycles.

Figure 5.2 — A pipelined core with a branch predictor.



Reading the diagram. Instructions move left to right through five stages; each arrow is a hand-off on a clock edge, so the whole chain is synchronous. The **fetch** stage reads instructions from the **L1 instruction cache**; the **memory** stage reads and writes the **L1 data cache** (bidirectional arrow). The **branch predictor** tells fetch which instruction comes next before the core actually knows. The dotted arrow is the failure path: when the execute stage discovers that a branch went the other way, the work in all earlier stages is wrong and is thrown away (flushed), and fetching restarts from the correct address. Pipelines fail to deliver one instruction per cycle when an instruction needs a result not yet produced (a **data hazard**), when two instructions need the same unit (a **structural hazard**), or when the next instruction is unknown (a **control hazard**). Designers scale this structure by making it wider (§5.3.5) rather than much deeper, because deeper pipelines make each misprediction more expensive.

5.3.4 Branch prediction

DEFINITION

Definition — Branch prediction. Hardware that guesses the outcome of a conditional jump (a **branch**, e.g. an `if` or a loop condition) before it is computed, so that the pipeline can keep fetching instructions. A wrong guess is a **misprediction** and forces a pipeline flush.

A long pipeline needs to know which instruction to fetch next on every cycle, but a branch's outcome is only known several stages later. Waiting would leave the pipeline empty for 10–20 cycles at every `if`, and typical code has a branch every five to ten instructions. So the core *guesses*. Modern predictors record the recent history of each branch and of the branches before it, and are right well over 95% of the time on typical code (approximate; it depends heavily on the program) [Hennessy & Patterson 2017, ch. 3]. A misprediction costs roughly 15–20 cycles on recent high-performance x86 cores (as of 2025, approximate).

The practical lesson: **predictable code runs faster**. Loops over sorted data, or code whose `if` almost always goes the same way, mispredict rarely. A filter over random data where each element passes with 50% probability mispredicts about half the time. Compilers can sometimes replace such branches with branch-free "conditional move" instructions, and data-processing engines often sort or partition data first precisely to make branches predictable.

5.3.5 Superscalar and out-of-order execution (intuition)

DEFINITION

Definition — Superscalar execution. A core that can issue and complete more than one instruction per cycle by having several execution units working in parallel.

DEFINITION

Definition — Out-of-order (OoO) execution. A technique in which the core executes instructions as soon as their inputs are ready rather than strictly in program order, while still making the results *appear* in program order (it **retires** them in order).

Back to the kitchen. A good chef reading a recipe does not stand idle while the water boils; they skip ahead to chopping onions, which does not depend on the water, and come back when the pot is ready. That is out-of-order execution. And a chef with two hands and

several burners can sear, stir and plate in the same moment; that is superscalar execution.

Internally, the core decodes a window of upcoming instructions — several hundred on recent high-end cores (as of 2025, approximate) — into a buffer called the **reorder buffer**. Each instruction waits in a **scheduler** until its inputs are ready, then dispatches to any free execution unit. Results are held until every earlier instruction has completed and are then retired in order, which preserves the illusion of sequential execution for the programmer. The key algorithmic idea dates to Tomasulo's work at IBM in 1967 [Tomasulo 1967].

Why does this matter to a system designer? Because the main thing OoO hides is **memory latency**. If a loop issues ten independent loads, an OoO core can have all ten outstanding at once and pay roughly one memory latency instead of ten. If each load depends on the previous one — walking a linked list, chasing pointers through a tree — nothing can overlap and the core pays the full latency every time. This is why arrays usually beat linked lists by a wide margin in practice, even when asymptotic complexity is identical, and why B-trees with wide nodes (§11.8) beat binary trees on real hardware.

5.3.6 Multiple cores and simultaneous multithreading (hyperthreading)

DEFINITION

Definition — Multi-core processor. A single chip containing several complete cores, each with its own pipeline and private caches, sharing the last-level cache and memory.

DEFINITION

Definition — Simultaneous multithreading (SMT). A technique in which one physical core runs two (occasionally more) instruction streams at once, interleaving them through the same execution units. Intel markets its implementation as **Hyper-Threading Technology**. Each stream appears to the operating system as a separate **logical CPU** (or **hardware thread**).

Even with out-of-order execution, a single thread often leaves execution units idle — for example while waiting for a cache miss. SMT gives the core a second recipe to work from: when thread A stalls, the core issues instructions from thread B. The core duplicates only the per-thread state (registers, program counter) and shares nearly everything else, so the extra silicon is small. The benefit depends heavily on the workload. Intel's original description reported gains of up to about 30% on some server benchmarks [Marr et al. 2002]; workloads that already saturate the execution units or the caches may gain nothing or even slow down, because two threads now compete for the same L1 and L2.

Three consequences for system design:

- **A "vCPU" in the cloud is usually a hardware thread, not a core.** On most x86 instance types at the major cloud providers, two vCPUs share one physical core (as of 2025; check your provider's documentation per instance family). Do not assume 16 vCPUs give 16 cores' worth of throughput.
- **Benchmarks must state whether SMT is on.** Doubling thread count from 8 to 16 on an 8-core, 16-thread machine will not double CPU-bound throughput.
- **Security.** Because SMT siblings share caches and execution units, they can leak information to each other through timing side channels. Some operators disable SMT for multi-tenant isolation, and some recent client CPUs ship without it (as of 2024).

Failure scenarios

Table 5.2 — CPU-level failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Service throughput flat after doubling vCPUs	vCPUs were SMT siblings; the workload saturates execution units	Per-core utilization near 100%; IPC drops as threads are added	Size by physical cores; scale out	Benchmark with physical-core counts in mind
Hot loop far slower on production data than in tests	Test data was sorted or uniform; production data makes branches unpredictable	`perf stat` shows high branch-miss rate	Sort/partition input; use branch-free code	Benchmark on realistic data distributions
Throttled CPU under sustained load	Thermal or power limits lower the clock rate	Clock frequency drops in telemetry	Improve cooling; spread load	Capacity-plan at sustained, not turbo, frequency

Common misconceptions

COMMON MISTAKE

"A higher GHz number means a faster CPU." Clock rate is only one factor in the iron law. A 3 GHz core with an IPC of 4 completes more work than a 4 GHz core with an IPC of 2. Compare CPUs on benchmarks of your own workload.

COMMON MISTAKE

"The CPU runs my code one line at a time, in order." It overlaps dozens to hundreds of instructions and executes them out of order; only the *visible* results are in order. That is why independent operations can be nearly free while dependent chains are slow.

5.4 CPU caches

5.4.1 Why caches exist

> DEFINITION
>
> **Definition — CPU cache.** A small, fast memory built from SRAM (static RAM) on the processor chip that holds copies of recently used main-memory data, so that most loads and stores do not have to wait for DRAM.

First-principles explanation

Picture a researcher in a library. The book on the **desk** is instantly available. The **shelf behind the chair** holds a few dozen books and takes a few seconds to reach. The **library stacks** on the same floor take a minute. The **off-site depository** takes a day. The researcher keeps the books they are using right now on the desk and the ones they will probably need soon on the shelf. When they fetch a book from the stacks, they bring back its neighbors too, because research tends to move along the shelf.

A CPU cache is that desk and shelf. It works because programs exhibit **locality of reference**:

- **Temporal locality** — data used recently is likely to be used again soon (a loop counter, a hot configuration object).
- **Spatial locality** — data near recently used data is likely to be used soon (the next element of an array, the next field of a struct).

Why does this exist?

A DRAM access costs about 80–120 ns (as of 2025, approximate), which at 4 GHz is 320–480 cycles. A core that could issue several instructions per cycle would spend almost all of its time waiting. Making all of main memory as fast as a register is physically impossible: fast memory (SRAM) needs about six transistors per bit, sits on the expensive processor die, and the longer the wires to it, the slower it gets. So designers build a **hierarchy**: tiny and very fast near the core, larger and slower further away.

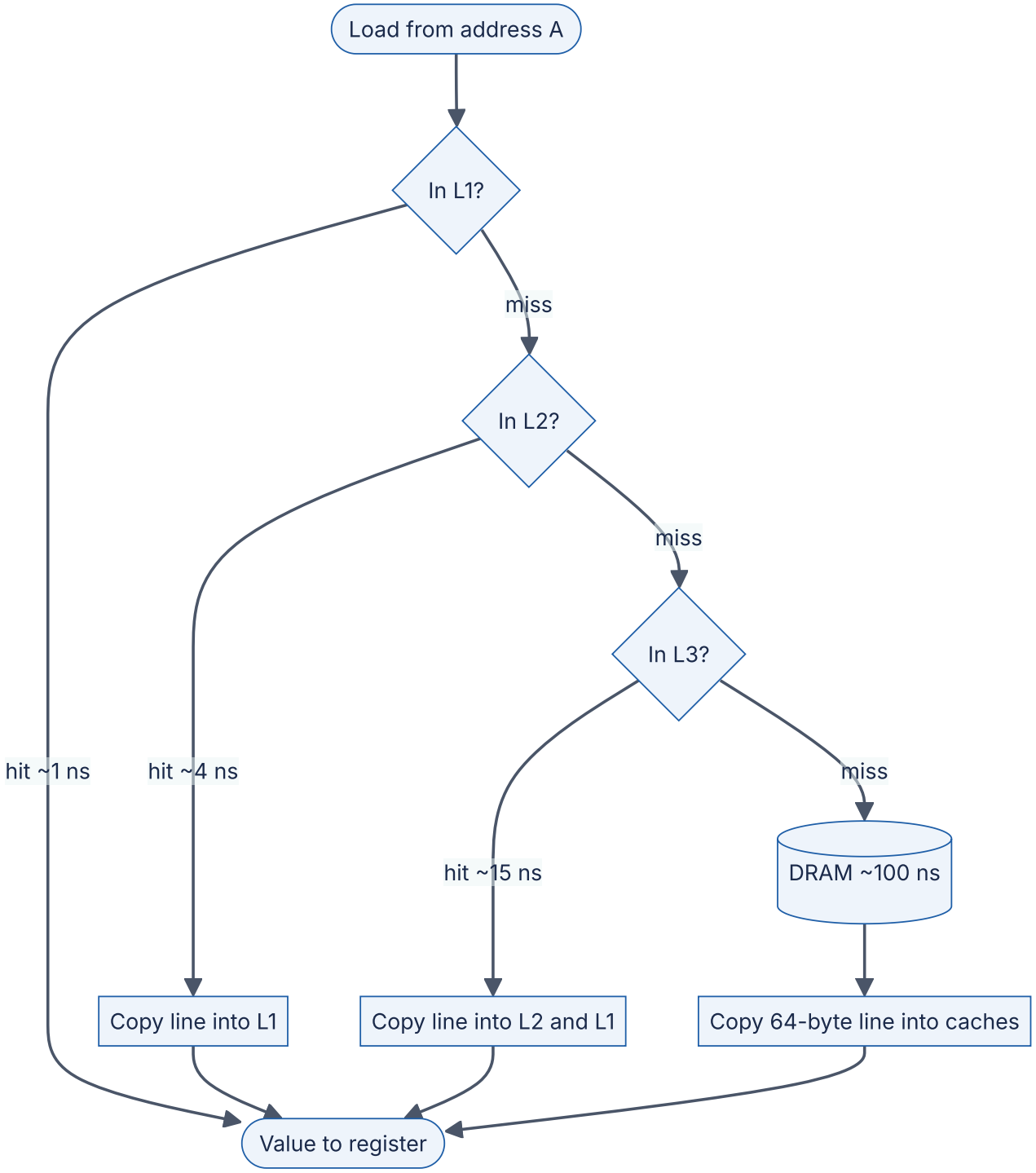
5.4.2 Cache levels

Table 5.3 — Typical CPU cache levels (as of 2024–2025, approximate; sizes and latencies vary by product).

Level	Typical size	Shared by	Latency (cycles)	Latency at ~4 GHz	Analogy
Registers	Tens of 64-bit values visible per thread	One hardware thread	0–1	< 0.3 ns	Hands
L1 data	32–64 KiB	One core	~4–6	~1–1.5 ns	Desk
L2	0.5–3 MiB	One core (usually)	~12–18	~3–5 ns	Shelf behind you
L3 (last-level cache)	8 MiB to several hundred MiB	All cores on the chip or chiplet	~40–80	~10–20 ns	Stacks on your floor
DRAM	GBs to TBs	Whole socket (or machine)	~300–500	~80–120 ns	Off-site depository

Sources for the ranges: vendor optimization manuals and independent measurements summarized in [Hennessy & Patterson 2017, ch. 2] and [Drepper 2007]; check `lscpu` on your own machine. The machine on which this chapter's examples were tested reports 48 KiB L1d and 2 MiB L2 per core.

Figure 5.3 — What happens on a load instruction.



Reading the diagram. A load instruction checks each level in turn. A **hit** returns the data at that level's latency; a **miss** passes the request to the next level. When the data finally arrives it is copied into the faster levels so the next access hits (this is temporal locality at work), possibly **evicting** some other line to make room. Each step is synchronous for the instruction that needs the value, but an out-of-order core keeps running independent instructions while it waits, and several misses can be outstanding at once (§5.9.2). The storage boundary is between L3 (on chip, SRAM) and DRAM (off chip). The main failure mode is a **miss storm**: a working set larger than the caches turns most loads into

100 ns trips. The way to scale is not a bigger cache — you cannot buy one — but a smaller, denser working set and access patterns the hardware can predict.

5.4.3 Cache lines and locality

DEFINITION

Definition — Cache line. The fixed-size unit in which data moves between memory and caches. On x86-64 and most ARM server cores a line is 64 bytes; some designs (for example Apple's M-series cores) use 128 bytes (as of 2024). A load of a single byte brings its whole line into the cache.

Because a whole line arrives at once, reading the first element of an array also brings in the next seven 8-byte elements "for free". Hardware **prefetchers** go further: when they detect that you are walking through memory in a regular pattern, they start fetching the next lines before you ask. Sequential access is therefore the best case in the entire memory system, and random access the worst.

Code example — row-major vs column-major traversal

```c
/* traverse.c — build: gcc -O2 -std=c11 traverse.c -o traverse */
#define _POSIX_C_SOURCE 199309L
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 8192                /* 8192 x 8192 ints = 256 MiB */

static double now(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec / 1e9;
}

int main(void) {
    int *m = malloc((size_t)N * N * sizeof *m);
    if (!m) return 1;
    for (size_t i = 0; i < (size_t)N * N; i++) m[i] = 1;

    long long sum = 0;
    double t0 = now();
    for (size_t r = 0; r < N; r++)          /* row-major: consecutive addresses
*/
        for (size_t c = 0; c < N; c++)
            sum += m[r * N + c];
    double t1 = now();
    for (size_t c = 0; c < N; c++)          /* column-major: jumps of N*4 bytes
*/
        for (size_t r = 0; r < N; r++)
            sum += m[r * N + c];
    double t2 = now();

    printf("row-major:    %.3f s\n", t1 - t0);
    printf("column-major: %.3f s\n", t2 - t1);
    printf("checksum: %lld\n", sum);         /* stops the compiler deleting
loops */
    free(m);
    return 0;
}
```

What to notice:

- Both loops add exactly the same 67,108,864 integers; only the **order** differs.
- The row-major loop touches consecutive 4-byte integers, so each 64-byte line serves 16 additions and the prefetcher streams lines ahead of time.
- The column-major loop jumps 8192 × 4 = 32,768 bytes between consecutive accesses, so every access lands on a different line and, because the matrix is 256 MiB, most of those lines have been evicted before the loop comes back for their neighbors.

- On the test machine (a cloud x86-64 VM, 2026) the row-major loop took 0.038 s and the column-major loop 0.667 s — about **17× slower** for identical arithmetic. Your ratio will differ, but the direction will not.
- `sum` is printed so that the optimizing compiler cannot prove the loops are useless and delete them — a classic benchmarking pitfall (§23.5).

5.4.4 Associativity: where a line may live

A cache must find a line quickly, so it cannot search every slot. Instead the memory address decides which **set** (group of slots) a line may occupy.

DEFINITION

Definition — Set-associative cache. A cache divided into S sets of W slots (**ways**). A line at a given address may be stored in any of the W ways of exactly one set. $W = 1$ is a **direct-mapped** cache; a cache with a single set holding every line is **fully associative**.

The analogy is a coat check with numbered racks. Your ticket number decides the rack (the set); within the rack your coat can go on any free hook (any way). If many guests' tickets map to the same rack, that rack overflows even though others are empty — a **conflict miss**.

FORMULA

Address decomposition in a set-associative cache

$$S = \frac{C}{B \times W}, \qquad \text{offset bits} = \log_2 B, \qquad \text{index bits} = \log_2 S$$

$$\text{set index} = \left\lfloor \frac{A}{B} \right\rfloor \bmod S, \qquad \text{tag} = \left\lfloor \frac{A}{B \times S} \right\rfloor$$

Symbol	Meaning	Units
C	Cache capacity	bytes
B	Line (block) size	bytes
W	Associativity (ways per set)	lines
S	Number of sets	sets
A	Memory address	byte address
tag	High-order address bits stored with the line to identify it	—

WORKED EXAMPLE

Worked example — Where does address 0x12345 go in a 32 KiB, 8-way cache with 64-byte lines?

1. Sets: $S = 32\,768/(64 \times 8) = 32\,768/512 = 64$ sets.
2. Offset bits $= \log_2 64 = 6$; index bits $= \log_2 64 = 6$.
3. Address $A = 0\text{x}12345 = 74\,565$.
4. Offset within the line $= 74\,565 \bmod 64 = 74\,565 - 1165 \times 64 = 74\,565 - 74\,560 = 5$.
5. Line number $= \lfloor 74\,565/64 \rfloor = 1165$; set index $= 1165 \bmod 64 = 1165 - 18 \times 64 = 13$.
6. Tag $= \lfloor 1165/64 \rfloor = 18$ (0x12).

Check in binary: 0x12345 = `0001 0010 0011 0100 0101`; the low 6 bits `000101` = 5 ✓, the next 6 bits `001101` = 13 ✓, the remaining bits `10010` = 18 ✓.

Interpretation. Addresses that differ by a multiple of $B \times S = 64 \times 64 = 4096$ bytes land in the **same set**. **Engineering implication.** A loop that strides through memory in steps of exactly 4 KiB (for example, reading one field from each of many page-aligned 4 KiB records) can use only one set — 8 lines, 512 bytes — of a 32 KiB cache. Power-of-two strides are a known performance trap; padding a row or record by one cache line often removes it [Drepper 2007, §6.2].

5.4.5 Average memory access time

FORMULA

Average memory access time (AMAT), applied recursively to each level

$$AMAT = t_{L1} + m_{L1} \times (t_{L2} + m_{L2} \times (t_{L3} + m_{L3} \times t_{mem}))$$

Symbol	Meaning	Units
t_{L1}, t_{L2}, t_{L3}	Hit time at each cache level	ns
t_{mem}	DRAM access time	ns
m_{L1}	Fraction of all accesses that miss L1	dimensionless (0–1)
m_{L2}, m_{L3}	**Local** miss rate: fraction of accesses *reaching* that level that miss it	dimensionless (0–1)

The same formula, with two levels, is the **effective access time** of any cache in front of a slower store: $EAT = h \times t_{hit} + (1 - h) \times t_{miss}$, where h is the hit ratio. You will use it again for application caches (§20.3).

WORKED EXAMPLE

Worked example — Good locality versus poor locality.

Assume t_{L1} = 1 ns, t_{L2} = 4 ns, t_{L3} = 15 ns, t_{mem} = 90 ns.

Good locality (array scans, small hot data): m_{L1} = 0.05, m_{L2} = 0.20, m_{L3} = 0.30.

1. L3 term: $15 + 0.30 \times 90 = 15 + 27 = 42$ ns.
2. L2 term: $4 + 0.20 \times 42 = 4 + 8.4 = 12.4$ ns.
3. AMAT $= 1 + 0.05 \times 12.4 = 1 + 0.62 = 1.62$ ns.

Poor locality (random lookups in a large hash table): m_{L1} = 0.30, m_{L2} = 0.60, m_{L3} = 0.70.

1. L3 term: $15 + 0.70 \times 90 = 15 + 63 = 78$ ns.
2. L2 term: $4 + 0.60 \times 78 = 4 + 46.8 = 50.8$ ns.
3. AMAT $= 1 + 0.30 \times 50.8 = 1 + 15.24 = 16.24$ ns.

Interpretation. The same hardware delivers memory ten times slower on average for the second workload. **Engineering implication.** Data layout (compact structs, arrays instead of pointer-linked nodes, hot fields grouped together) can change performance by an order of magnitude without changing the algorithm. The application-level twin of this example: raising a cache hit ratio from 90% to 99% in front of a 100 µs SSD read, with a 1 µs hit, cuts effective access time from $0.9 \times 1 + 0.1 \times 100 = 10.9$ µs to $0.99 \times 1 + 0.01 \times 100 = 1.99$ µs, about 5.5× faster.

5.4.6 Cache coherence and the MESI protocol

DEFINITION

Definition — Cache coherence. The guarantee that all cores observe a single, consistent sequence of values for each memory location, even though each core keeps private copies in its own caches.

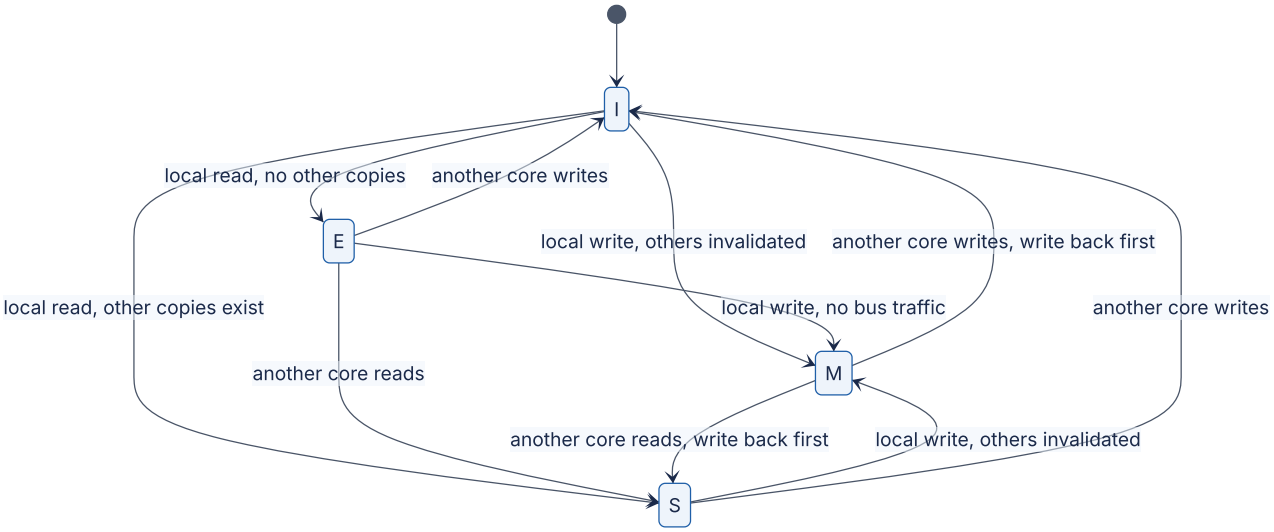
Suppose cores 0 and 1 both cache the variable `x`. Core 0 writes `x = 5`. If core 1 keeps reading its stale copy, the program breaks. Coherence protocols prevent this by tracking, for every cached line, which cores hold it and whether it has been changed. The textbook protocol is **MESI**, named after the four states a line can be in within one core's cache [Papamarcos & Patel 1984; Sorin, Hill & Wood 2011]:

- **Modified (M)** — this core has the only copy and has changed it; memory is stale. The core must write the line back before anyone else may use it.
- **Exclusive (E)** — this core has the only copy, and it matches memory. The core may write to it without telling anyone.

- **Shared (S)** — several cores may hold read-only copies that match memory.
- **Invalid (I)** — this cache does not hold a valid copy.

The analogy is a shared document with a single “edit pen”. Anyone may hold a read-only photocopy (Shared). To write, you must collect every photocopy and shred it (invalidate them), then you hold the pen (Modified). Real processors use extensions — Intel describes MESIF, AMD MOESI — that add states to reduce memory traffic, but the core idea is the same.

Figure 5.4 — MESI state transitions for one cache line in one core’s cache.



Reading the diagram. Each box is the state of *one* 64-byte line in *one* core’s cache; every core runs this machine independently for every line. “Local” events come from this core’s own instructions; “another core” events arrive as coherence messages over the on-chip interconnect (a ring or mesh linking cores, caches and memory controllers) or, between sockets, over the socket-to-socket link. Transitions caused by our own reads and writes are synchronous with the instruction; invalidations and write-backs triggered by other cores arrive asynchronously. Note the two expensive arcs: moving from Shared to Modified requires invalidating every other copy and waiting for acknowledgments, and a remote read of a Modified line forces a write-back. When two cores repeatedly write the same line, it ping-pongs between their caches, each transfer costing tens of nanoseconds on one chip and more across sockets. Coherence traffic is the reason “just add cores” stops scaling for shared, frequently written data.

5.4.7 False sharing

DEFINITION

Definition — False sharing. A performance problem in which threads on different cores write to *different* variables that happen to sit in the *same* cache line, so the coherence protocol bounces the line between cores as if they were sharing data.

Coherence works at the granularity of a whole line, not a variable. Two counters 8 bytes apart, each updated by its own thread, look to the hardware exactly like one heavily contended variable. The program is logically correct and has no locks, yet it scales *negatively*.

Code example — demonstrating and fixing false sharing in C

```c
/* false_sharing.c — build: gcc -O2 -std=c11 -pthread false_sharing.c -o fs */
#define _POSIX_C_SOURCE 199309L
#include <pthread.h>
#include <stdalign.h>
#include <stdatomic.h>
#include <stdint.h>
#include <stdio.h>
#include <time.h>

#define ITERATIONS 50000000ULL   /* 5 x 10^7 increments per thread */
#define CACHE_LINE 64            /* x86-64 line size; check yours */

/* Variant A: two counters 8 bytes apart -> same cache line. */
struct packed {
    _Atomic uint64_t a;
    _Atomic uint64_t b;
};

/* Variant B: each counter starts its own 64-byte cache line. */
struct padded {
    alignas(CACHE_LINE) _Atomic uint64_t a;
    alignas(CACHE_LINE) _Atomic uint64_t b;
};

static struct packed P;
static struct padded Q;

static void *worker(void *arg) {
    _Atomic uint64_t *counter = arg;
    for (uint64_t i = 0; i < ITERATIONS; i++)
        atomic_fetch_add_explicit(counter, 1, memory_order_relaxed);
    return NULL;
}

static double run(_Atomic uint64_t *x, _Atomic uint64_t *y) {
    struct timespec t0, t1;
    pthread_t ta, tb;
    clock_gettime(CLOCK_MONOTONIC, &t0);
    pthread_create(&ta, NULL, worker, (void *)x);
    pthread_create(&tb, NULL, worker, (void *)y);
    pthread_join(ta, NULL);
    pthread_join(tb, NULL);
    clock_gettime(CLOCK_MONOTONIC, &t1);
    return (double)(t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec) / 1e9;
}

int main(void) {
    printf("gap between counters: packed=%td B, padded=%td B\n",
           (char *)&P.b - (char *)&P.a, (char *)&Q.b - (char *)&Q.a);
    printf("packed (false sharing): %.2f s\n", run(&P.a, &P.b));
    printf("padded (separate lines): %.2f s\n", run(&Q.a, &Q.b));
    return 0;
}
```

What to notice:

- Each thread touches **only its own counter**; there is no logical sharing and no lock.
- In `struct packed` the counters are 8 bytes apart, so they share a 64-byte line. In `struct padded`, `alignas(64)` forces each counter onto its own line (the struct grows from 16 to 128 bytes).
- `atomic_fetch_add_explicit` compiles to a locked read-modify-write instruction, which requires the core to hold the line in the Modified state — every increment by one thread therefore steals the line from the other.
- On the 4-vCPU test VM (2026), three runs gave 1.57–1.69 s for the packed layout versus 0.31–0.34 s for the padded layout: about **5× slower purely because of memory layout**. With plain (non-atomic) increments, the effect is smaller and noisier because store buffers and compiler optimizations hide part of it — another reason to measure rather than assume.
- The fix costs memory (112 wasted bytes here). Pad only data that is genuinely written by different threads at high frequency: per-thread counters, queue head and tail indices, per-core statistics.

Real-world industry applications

INDUSTRY EXAMPLE

LMAX Disruptor (low-latency financial exchange). LMAX built an inter-thread messaging library for its trading platform and found that the head and tail sequence counters of a queue, written by different threads, suffered false sharing. The Disruptor pads its sequence counters to occupy their own cache lines and avoids locks on the hot path; the authors attribute a large part of its throughput advantage over conventional bounded queues to this “mechanical sympathy” with the cache hierarchy. Trade-off: padded structures waste memory and the design assumes a known line size. *Evidence: Primary* — [Thompson et al. 2011]

INDUSTRY EXAMPLE

Netflix (JVM service on larger cloud instances). When Netflix moved a stateless Java service to an instance type with three times the vCPUs, throughput rose only about 25% instead of the expected ~3×. Using hardware performance counters, engineers traced the problem to cache-line contention inside the JVM’s type-checking code — first false sharing, then, after a workaround, true sharing of a frequently written cache field. Fixing it brought throughput close to the expected scaling. Trade-off: diagnosing it required low-level counters that are not exposed on every cloud instance type. *Evidence: Primary* — [Filanovsky & Sane 2022]

Runtimes offer help: Java's internal `@Contended` annotation, C++17's `std::hardware_destructive_interference_size`, and explicit padding fields in Go and Rust.

Advantages and disadvantages of caches

Caches make the common case (good locality) almost as fast as registers, with no effort from the programmer. Their disadvantages are that performance becomes **data-dependent and hard to predict**, that coherence traffic limits how well shared, written data scales across cores, and that shared caches create interference between unrelated workloads — one tenant's large scan can evict another tenant's hot data (the **noisy neighbor** problem, Ch 47).

Failure scenarios

Table 5.4 — Cache-related failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Throughput falls as threads are added	False sharing or a hot shared counter	`perf c2c` on Linux; high cross-core cache-to-cache transfers	Pad or shard per-thread state	Per-thread or per-core data; aggregate occasionally
Latency spikes when a batch job runs on the same host	Batch job's scan evicts the service's working set from shared L3	LLC miss rate rises in sync with the batch job	Move the job; pin to other cores	Cache partitioning (e.g., Intel RDT) or separate hosts
Mysterious slowdown at certain array sizes	Power-of-two stride causes conflict misses in one set	Performance cliff at sizes like 4096 or 8192	Pad rows/records	Avoid power-of-two strides in hot loops
Pointer-heavy structure slow despite good big-O	Each hop is a dependent cache miss	Low IPC, high cache-miss counts	Switch to arrays / B-tree-like wide nodes	Design data structures for locality

Common misconceptions

> COMMON MISTAKE
>
> **"Lock-free code always scales."** Lock-free code still writes to shared cache lines. If many cores update the same atomic counter, the line ping-pongs between them and throughput can be *lower* than with one thread. Scalable designs avoid shared writes altogether (per-core counters, sharded state).

COMMON MISTAKE

“Caches only matter to C programmers.” Every language runs on the same hardware. Java object headers, Python object pointers and Go slices all have cache footprints; the gap between “array of values” and “array of pointers to objects” is often several-fold in any language.

5.5 RAM and the memory hierarchy

5.5.1 Main memory (DRAM)

DEFINITION

Definition — DRAM (dynamic random-access memory). The technology used for main memory. Each bit is stored as charge in a tiny capacitor that leaks and must be **refreshed** every few tens of milliseconds. DRAM is **volatile**: its contents are lost when power is removed.

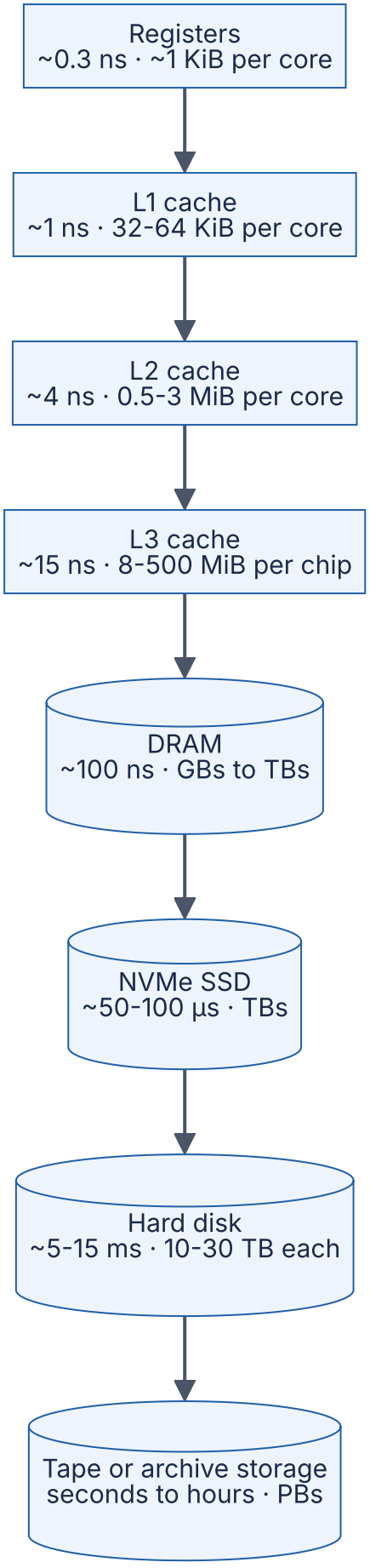
DRAM uses one transistor and one capacitor per bit, versus about six transistors for the SRAM in caches, so it is far denser and cheaper — but slower. Memory chips sit on modules (DIMMs) that plug into **channels**, each an independent 64-bit data path to the CPU’s memory controller (DDR5 splits each module’s path into two 32-bit subchannels, still 64 bits in total). Inside a chip, bits are arranged in banks of rows and columns; reading a row that is already “open” is faster than opening a new one, which is one more reason sequential access is cheaper than random access [Drepper 2007, §2]. Servers use **ECC** (error-correcting code) memory, which stores extra check bits to correct single-bit errors.

5.5.2 The memory hierarchy

DEFINITION

Definition — Memory hierarchy. The layered arrangement of storage technologies in a computer, from small, fast and expensive per byte (registers, caches) to large, slow and cheap per byte (disks, tape), in which each level acts as a cache for the level below it.

Figure 5.5 — The memory and storage hierarchy (as of 2024–2025, approximate).


Registers
~0.3 ns · ~1 KiB per core
L1 cache
~1 ns · 32-64 KiB per core
L2 cache
~4 ns · 0.5-3 MiB per core
L3 cache
~15 ns · 8-500 MiB per chip
DRAM
~100 ns · GBs to TBs
NVMe SSD
~50-100 µs · TBs
Hard disk
~5-15 ms · 10-30 TB each
Tape or archive storage
seconds to hours · PBs

Reading the diagram. Read it top to bottom as “faster, smaller, costlier per byte” turning into “slower, larger, cheaper per byte”. Each arrow means “misses at this level are served by the next level”. The levels above DRAM are managed by hardware automatically; the boundary between DRAM and SSD is managed by software — the operating system’s page cache (§6.6) or your database’s buffer pool (§13.4). The most important storage boundary is **volatility**: everything from DRAM upward vanishes on power loss, so durability requires reaching the SSD, disk or a remote replica (§6.14). Each step down costs roughly 3–1000× in latency. Systems “scale” this pyramid by keeping the working set as high up as possible: more RAM to cache hot data, SSDs instead of disks for random access, and disks or object storage only for cold, sequentially read data.

Table 5.5 — Memory hierarchy numbers (as of 2024–2025, approximate; prices are volatile).

Level	Typical capacity	Random-access latency	Sequential bandwidth	Approx. cost per GB	Persistent?
L1 / L2 / L3 cache	KiB to hundreds of MiB	~1 / ~4 / ~15 ns	Hundreds of GB/s per core (L1)	Not sold separately	No
DRAM (DDR5)	16 GB – several TB per server	~80–120 ns	~38 GB/s per DDR5-4800 channel; ~300–600 GB/s per socket	~$2–5	No
NVMe SSD (TLC flash)	1–30+ TB	~50–100 µs read	~3–7 GB/s (PCIe 4.0 x4); ~10–14 GB/s (PCIe 5.0 x4)	~$0.05–0.15	Yes
SATA SSD	0.25–8 TB	~80–150 µs read	~0.55 GB/s	~$0.05–0.10	Yes
HDD (7200 RPM)	4–30+ TB	~5–15 ms	~0.15–0.28 GB/s	~$0.015–0.03	Yes
LTO-9 tape	18 TB native per cartridge	Seconds to minutes (mount, wind)	~0.4 GB/s native	< $0.01	Yes

Sources: interface limits from the DDR5, PCIe and SATA specifications (bandwidth arithmetic in §5.9); device ranges from vendor datasheets of the period; LTO-9 figures from the LTO Program; general structure from [Hennessy & Patterson 2017, ch. 2] and [Bryant & O’Hallaron 2015, ch. 6]. Prices are retail/street approximations for 2024 and changed sharply in 2025–2026 for DRAM and flash; treat them as ratios, not quotes.

The ratios are the lesson: DRAM costs roughly 30–100× more per GB than flash, and flash roughly 3–5× more than disk, while latency falls by about 1,000× from DRAM to flash and 100× again from flash to disk. Choosing where data lives is an economic decision as much as a technical one.

5.6 Storage devices

5.6.1 Hard disk drives (HDDs)

DEFINITION

Definition — Hard disk drive (HDD). A storage device that records data magnetically on spinning platters, read and written by heads mounted on a moving arm (the **actuator**). Data is organized in concentric **tracks** divided into **sectors** of 512 or 4096 bytes.

First-principles explanation

Think of a record player. To play a particular song, you lift the arm and move it to the right groove (the **seek**), then wait for the start of the song to come round under the needle (**rotational latency**), then listen to the song (**transfer**). Moving the arm and waiting for the platter take milliseconds — millions of CPU cycles — whereas reading the data once the head is in place is fast. An HDD rewards you for reading long runs of adjacent data and punishes you for jumping around.

FORMULA

HDD access time

$$T_{access} = T_{seek} + T_{rot} + T_{transfer}, \qquad T_{rot} = \frac{1}{2} \times \frac{60}{RPM}, \qquad T_{transfer} = \frac{S}{R}$$

$$IOPS_{random} \approx \frac{1}{T_{access}}$$

Symbol	Meaning	Units
T_{seek}	Average time to move the head to the target track	ms
T_{rot}	Average rotational latency: half a revolution	ms
RPM	Spindle speed	revolutions per minute
S	Size of the request	bytes
R	Sustained media transfer rate	bytes/second
$IOPS_{random}$	Random I/O operations per second for one outstanding request	operations/second

WORKED EXAMPLE

Worked example — Random versus sequential reads on a 7200 RPM disk.

Assume T_{seek} = 8.5 ms (a typical desktop-class average), R = 200 MB/s (as of 2024, approximate; outer tracks are faster than inner tracks).

1. One revolution: $60/7200 = 0.008\,33$ s $= 8.33$ ms, so $T_{rot} = 8.33/2 = 4.17$ ms.
2. Transfer of a 4 KiB block: $4096/(200 \times 10^6) = 2.05 \times 10^{-5}$ s $= 0.02$ ms.
3. Random 4 KiB read: $T_{access} = 8.5 + 4.17 + 0.02 = 12.69$ ms.
4. Random IOPS $\approx 1/0.012\,69 \approx 79$ operations/s, so random throughput $= 79 \times 4096 \approx 0.32$ MB/s.
5. Sequential throughput ≈ 200 MB/s, which is $200/0.32 \approx 620\times$ the random throughput.
6. Reading 100 MB contiguously after one positioning: $12.67 + (10^8/2 \times 10^8) \times 1000 = 12.67 + 500 = 512.67$ ms, an effective $10^8/0.512\,67 \approx 195$ MB/s — 97.5% of the maximum.

Interpretation. For small random reads, 99.8% of the time is mechanical positioning (12.67/12.69). **Engineering implication.** On disks, design for large sequential transfers: append-only logs, LSM trees (§13.8), large blocks in distributed file systems (§15.5) and batch scans. Even a 15,000 RPM enterprise disk (T_{rot} = 2 ms, seek ≈ 3.5 ms) reaches only about $1/0.0055 \approx 180$ random IOPS.

HDDs remain the cheapest online storage per byte (Table 5.5), so they still hold most of the world's bulk data in object stores and backup systems — where access is overwhelmingly sequential [Ruemmler & Wilkes 1994].

5.6.2 Solid-state drives (SSDs) and flash memory

DEFINITION

Definition — Solid-state drive (SSD). A storage device with no moving parts that stores data in **NAND flash** memory cells, managed by an on-board controller running a **flash translation layer** (FTL).

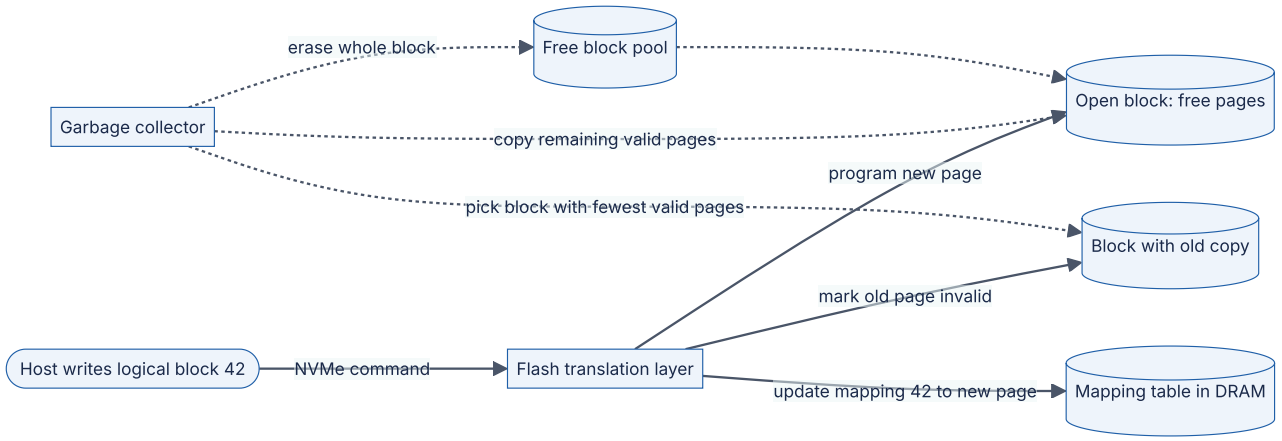
Flash is fast to read and has no seek time, but it has three peculiar rules that shape everything about SSD behavior [Agrawal et al. 2008]:

1. **Read and write in pages.** The smallest unit you can read or program (write) is a **page**, typically 4–16 KiB (as of 2024, approximate).
2. **Erase in blocks.** A page cannot be overwritten in place. It must first be erased, and erasure works only on a whole **erase block** of hundreds of pages (several MiB in modern chips).

3. **Cells wear out.** Each block survives a limited number of program/erase (P/E) cycles — on the order of 1,000–3,000 for TLC (three bits per cell) flash and fewer for QLC (four bits per cell) (as of 2024, approximate; vendors rarely publish exact figures).

The analogy is a notebook written in pen. You cannot rub out one line; to reuse a page you must tear out a whole chapter, and the binding can only take so many tear-outs. So when you "update" a line, you write the new version on a fresh page and cross out the old one. Periodically you copy the still-valid lines from a messy chapter to a fresh one and tear the old chapter out. That is exactly what an SSD controller does.

Figure 5.6 — How an SSD handles an overwrite and garbage collection.



Reading the diagram. The host addresses data by **logical block address** (LBA); the FTL translates LBAs to physical flash pages using a mapping table held in the SSD's own DRAM (backed up to flash). An overwrite from the host (solid arrows, synchronous with the write command) becomes "program a fresh page, then repoint the mapping", and the old page is merely marked invalid. In the background (dotted arrows, asynchronous), the **garbage collector** chooses a block with few valid pages, copies the survivors elsewhere and erases the block, returning it to the free pool. The failure point is garbage collection under sustained random writes: when free blocks run out, the host's writes wait behind GC, producing latency spikes. Drives mitigate this with **over-provisioning** (spare capacity invisible to the host) and the host helps with **TRIM**/deallocate commands that tell the drive which pages no longer hold useful data. The mapping table is also why SSDs need power-loss protection capacitors in enterprise models.

5.6.3 Write amplification and endurance

FORMULA

Write amplification factor and drive lifetime

$$WAF = \frac{\text{bytes written to flash}}{\text{bytes written by the host}}, \qquad \text{Lifetime (days)} \approx \frac{C \times PE}{W_{host} \times WAF}$$

Symbol	Meaning	Units
WAF	Write amplification factor (≥ 1 for a drive without compression)	dimensionless
C	Raw flash capacity	bytes
PE	Rated program/erase cycles per block	cycles
W_{host}	Host write volume per day	bytes/day

WORKED EXAMPLE

Worked example — How long will this SSD last?

A 1 TB TLC SSD, assumed rating $PE = 3000$ cycles, so total flash write budget $= 10^{12} \times 3000 = 3 \times 10^{15}$ bytes $= 3$ PB. A database writes $W_{host} = 500$ GB/day (0.5 drive writes per day).

1. With sequential, log-structured writes, $WAF \approx 2.5$: flash writes $= 500 \times 2.5 = 1250$ GB/day $= 1.25$ TB/day. Lifetime $= 3000$ TB/1.25 TB/day $= 2400$ days ≈ 6.6 years.
2. With small random in-place updates on a nearly full drive, $WAF \approx 5$: flash writes $= 2.5$ TB/day. Lifetime $= 3000/2.5 = 1200$ days ≈ 3.3 years.

Interpretation. The same host workload wears the drive out twice as fast purely because of the write *pattern*. **Engineering implication.** Write amplification compounds: an LSM tree's own compaction (§13.11) multiplies with the SSD's internal WAF. Prefer large, sequential, aligned writes; leave free space (or over-provisioning) on write-heavy drives; and monitor the drive's wear indicators (SMART / NVMe health log).

5.6.4 SATA versus NVMe

SSDs first plugged into the same **SATA** interface designed for hard disks. SATA III signals at 6 Gbit/s, and after its 8b/10b line encoding (10 bits on the wire per data byte) delivers at most 600 MB/s; real drives reach about 550 MB/s. Its command protocol, AHCI, offers

one command queue of 32 entries — ample for a disk that can do 100 operations per second, but a straitjacket for flash that can do hundreds of thousands.

NVMe (Non-Volatile Memory Express) is a protocol designed for flash that attaches drives directly to **PCIe** lanes. It supports up to about 65,535 I/O queues, each up to 65,536 commands deep [NVM Express Base Specification], so every CPU core can have its own submission and completion queue pair with no locking between cores. A PCIe 4.0 x4 link carries roughly 7.9 GB/s of payload after encoding (16 GT/s × 4 lanes × 128/130 ÷ 8 ≈ 7.9 GB/s), and fast drives deliver about 7 GB/s sequential and around a million random 4 KiB reads per second at high queue depth (as of 2024, approximate).

5.6.5 IOPS, throughput and queue depth

DEFINITION

Definition — IOPS. Input/output operations per second: the number of independent read or write requests a device completes per second, quoted for a stated request size (usually 4 KiB) and **queue depth** (number of requests outstanding at once).

FORMULA

Little's Law applied to a storage device

$$IOPS = \frac{QD}{L}, \qquad \text{Throughput} = IOPS \times S$$

Symbol	Meaning	Units
QD	Average number of requests outstanding (queue depth)	requests
L	Average latency per request	seconds
S	Request size	bytes

WORKED EXAMPLE

Worked example — Why a single-threaded reader cannot exploit an NVMe SSD.

An NVMe SSD has a random-read latency of $L = 80$ µs.

1. One request at a time ($QD = 1$): $IOPS = 1/(80 \times 10^{-6}) = 12\,500$; throughput $= 12\,500 \times 4096 = 51.2$ MB/s.
2. To reach the drive's rated 10^6 IOPS: $QD = IOPS \times L = 10^6 \times 80 \times 10^{-6} = 80$ requests outstanding.

Interpretation. A program that issues one blocking read, waits, then issues the next will use about 1% of a fast SSD. **Engineering implication.** To use modern storage you need **concurrency in the I/O path**: many threads, asynchronous I/O (io_uring, §6.13), or batched requests. The same law governs memory (§5.9.2) and network connections (§7.15).

5.6.6 Persistent memory and newer tiers

Persistent memory (PMem) is byte-addressable like DRAM — the CPU reads it with ordinary load instructions — but keeps its contents without power. The best-known product, Intel Optane persistent memory, offered larger capacities than DRAM at read latencies of a few hundred nanoseconds (approximate); Intel announced in 2022 that it was winding the business down. The ideas survive in capacitor-backed DRAM inside storage controllers and in **CXL** (Compute Express Link), which attaches extra memory over the PCIe physical layer at somewhat higher latency than local DRAM. Programming persistent memory requires explicit cache-line flushes and fences, because data still in a volatile CPU cache is not durable.

Table 5.6 — Storage device comparison (as of 2024–2025, approximate).

	HDD	SATA SSD	NVMe SSD	Persistent memory / CXL memory
What it is	Spinning magnetic platters	Flash behind a disk-era interface	Flash on PCIe with a flash-native protocol	Byte-addressable memory on the memory bus or CXL
How it works	Mechanical seek + rotation	FTL, 1 queue × 32 commands	FTL, up to ~64K queues	CPU loads/stores
Advantages	Cheapest per TB; good sequential speed	Drop-in replacement for disks	Very high IOPS and bandwidth	Nanosecond-scale access, large capacity
Disadvantages	~100 random IOPS; mechanical failure	Interface caps at ~550 MB/s	Wear-out; GC latency spikes	Niche, costly; Optane discontinued
Performance	~80–200 IOPS; ~0.2 GB/s	~50–100K IOPS; 0.55 GB/s	~0.5–1.5M IOPS; 3–14 GB/s	~100s ns latency
Cost per GB	~$0.02	~$0.05–0.10	~$0.05–0.15	Above DRAM-adjacent tiers
Choose when	Cold, bulk, sequential data	Legacy servers, budget	Databases, logs, anything random-access	Very large in-memory datasets; specialized logging

Real-world industry applications

INDUSTRY EXAMPLE

Apache Kafka (append-only log on ordinary disks). Kafka's designers observed that sequential disk throughput is hundreds of times higher than random-access throughput and that the operating system's page cache already keeps recently written data in RAM. Kafka therefore stores each partition as an append-only file, reads and writes it sequentially, and relies on the page cache rather than an in-process cache, which lets brokers sustain high throughput even on hard disks. Trade-off: the design favors sequential consumers; random lookups by key are not what it is for. *Evidence: Primary* — [Apache Kafka Documentation, Design]

Failure scenarios

Table 5.7 — Storage failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
p99 write latency spikes on a busy SSD	Garbage collection when free blocks are exhausted	Latency spikes correlate with sustained writes	Throttle writes; free space	Over-provision; TRIM; sequential writes
SSD wears out early	High write amplification from small random writes	SMART "percentage used" climbs fast	Replace drive; rebuild replica	Log-structured writes; endurance-rated drives
Database slow after moving from SSD to cheaper HDD tier	Random-access workload on ~100 IOPS devices	Queue depth high, low throughput, high await	Move back; add cache	Match the device to the access pattern
Data lost after power failure despite successful write	Data was still in volatile cache (OS, drive)	Corruption on restart	Restore from replica/backup	`fsync` and drives with power-loss protection (§6.14)

COMMON MISTAKE

"SSDs make access patterns irrelevant." Sequential I/O on an SSD is still faster than random I/O, writes are amplified by small random updates, and a single blocking thread cannot exploit the device's parallelism. SSDs shrink the penalty for randomness from ~600× to a few times; they do not remove it.

5.7 I/O operations

DEFINITION

Definition — I/O (input/output). Any transfer of data between the CPU/memory and a device outside it: storage, network cards, GPUs, keyboards. A **device controller** on each device does the physical work; the CPU communicates with it through **device registers** and shared memory.

The central question of I/O design is: *what does the CPU do while a slow device works?* Back in the kitchen, ordering from the supplier can be handled four ways. You can stand at the phone repeating "is it ready yet?" (**polling**). You can do something else and let the supplier ring a bell when the delivery arrives (**interrupts**). You can have the supplier's driver carry the goods straight into the fridge without you lifting a box (**DMA**). Or, for the busiest restaurants, you can bypass the front office entirely and deal with the driver directly (**kernel bypass**).

5.7.1 Programmed I/O

In **programmed I/O** (PIO) the CPU itself moves every word between the device and memory, reading and writing device registers in a loop. It is simple and still used for tiny,

infrequent transfers (configuring a device), but for bulk data it wastes the most expensive resource in the machine — CPU cycles — on copying.

5.7.2 Interrupts

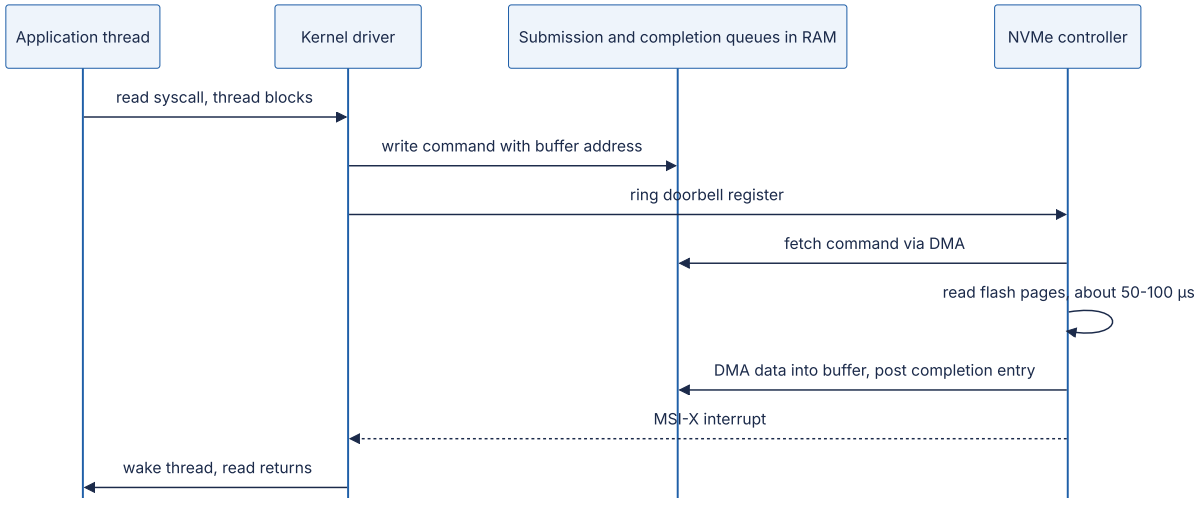
An **interrupt** is a hardware signal from a device that makes the CPU suspend its current work, run a short **interrupt handler** in the kernel, and then resume. The CPU starts an operation, goes off to run other threads, and is notified on completion. Modern PCIe devices raise **message-signaled interrupts** (MSI-X), which can be steered to a specific core. Interrupts are efficient when events are rare, but each one costs on the order of a microsecond or more of CPU time (handler entry, cache pollution, possible context switch; approximate). At millions of events per second — a single 10 Gbit/s link can deliver up to about 14.9 million minimum-size Ethernet frames per second (10^{10} bit/s ÷ (84 bytes × 8) on the wire, counting preamble and inter-frame gap) — one interrupt per packet would overwhelm the CPU.

5.7.3 Direct memory access (DMA)

DEFINITION

Definition — Direct memory access (DMA). A mechanism that lets a device controller read from or write to main memory directly, without the CPU copying each word; the CPU only sets up the transfer and is told when it finishes.

Figure 5.7 — Reading a block from an NVMe SSD with DMA and an interrupt.



Reading the diagram. Four participants: the application thread, the kernel's NVMe driver, the command queues living in ordinary RAM, and the SSD's controller. The application's `read` system call is synchronous *for the thread* — it blocks — but the CPU core is free to run other threads in the meantime. The kernel writes a command describing the request and the destination buffer into a submission queue in memory, then writes

to a **doorbell** register on the device over PCIe. From then on the device does the work: it fetches the command, reads flash, and **DMAs** the data straight into the buffer — no CPU copying. Finally it posts a completion entry and raises an interrupt (dashed arrow: asynchronous notification). The storage boundary is crossed only by the device. Failure points: the device may time out (the driver resets it and fails the I/O), and a buggy device with unrestricted DMA could overwrite arbitrary memory, which is why servers use an **IOMMU** to restrict which addresses each device may access. Scaling comes from many queue pairs (one per core) and many outstanding commands (§5.6.5).

5.7.4 Polling and interrupt coalescing

When events are very frequent, **polling** — a core repeatedly checking a completion queue — is cheaper than taking an interrupt per event, at the cost of burning a core even when idle. Practical systems mix the two. Network cards **coalesce** interrupts (one interrupt per batch of packets or per few microseconds), and Linux's NAPI network subsystem switches from interrupts to polling under load, then back again when traffic subsides. This is the batching principle of §5.12 appearing inside the kernel. How programs see all this — blocking calls versus non-blocking interfaces such as `epoll` and `io_uring` — is covered in §6.13; underneath, the device always works in parallel with the CPU, and software's job is to keep enough requests in flight.

5.7.5 Kernel bypass (introduction)

For each I/O, the conventional path crosses the user/kernel boundary, runs generic driver code, and often copies data between kernel and user buffers. At 10–100 Gbit/s or a million IOPS, those per-operation costs (each a few hundred nanoseconds to microseconds) become the bottleneck. **Kernel bypass** lets an application drive the device directly from user space:

- **DPDK** (Data Plane Development Kit) — user-space network drivers that poll NIC queues; used in software routers, load balancers and packet processors [DPDK Documentation].
- **SPDK** (Storage Performance Development Kit) — the same approach for NVMe storage [SPDK Documentation].
- **RDMA** (remote direct memory access) — network cards that read and write a *remote* machine's memory without involving the remote CPU, used in HPC and some databases and ML clusters (Ch 49, Ch 65).
- **io_uring** (Linux 5.1, 2019) — not a full bypass, but shared submission/completion rings between application and kernel that remove most per-operation system calls and can poll [Axboe 2019].

Bypass trades generality, security isolation and operational simplicity for raw speed: you dedicate cores to polling, give up the kernel's TCP stack or file system, and take on more

of the driver's responsibilities. Most services never need it; network appliances, exchanges and storage engines sometimes do.

Table 5.8 — I/O mechanisms compared.

Mechanism	CPU cost per transfer	Latency	Best for	Drawback
Programmed I/O	Very high (CPU copies every word)	Low for tiny transfers	Device configuration	Wastes CPU on bulk data
Interrupt-driven + DMA	Low per byte; ~µs per interrupt	Adds interrupt + wake-up time	General-purpose I/O	Interrupt storms at high event rates
Polling	A dedicated core, even when idle	Lowest	Very high event rates	Burns CPU and power
Coalescing / hybrid (NAPI)	Low	Slightly higher for single events	Network stacks under variable load	Tuning trade-off between latency and CPU
Kernel bypass (DPDK/SPDK/RDMA)	Lowest per operation	Lowest	Packet processing, storage engines, HPC	Complexity; loses kernel features

5.8 CPU-bound versus I/O-bound workloads

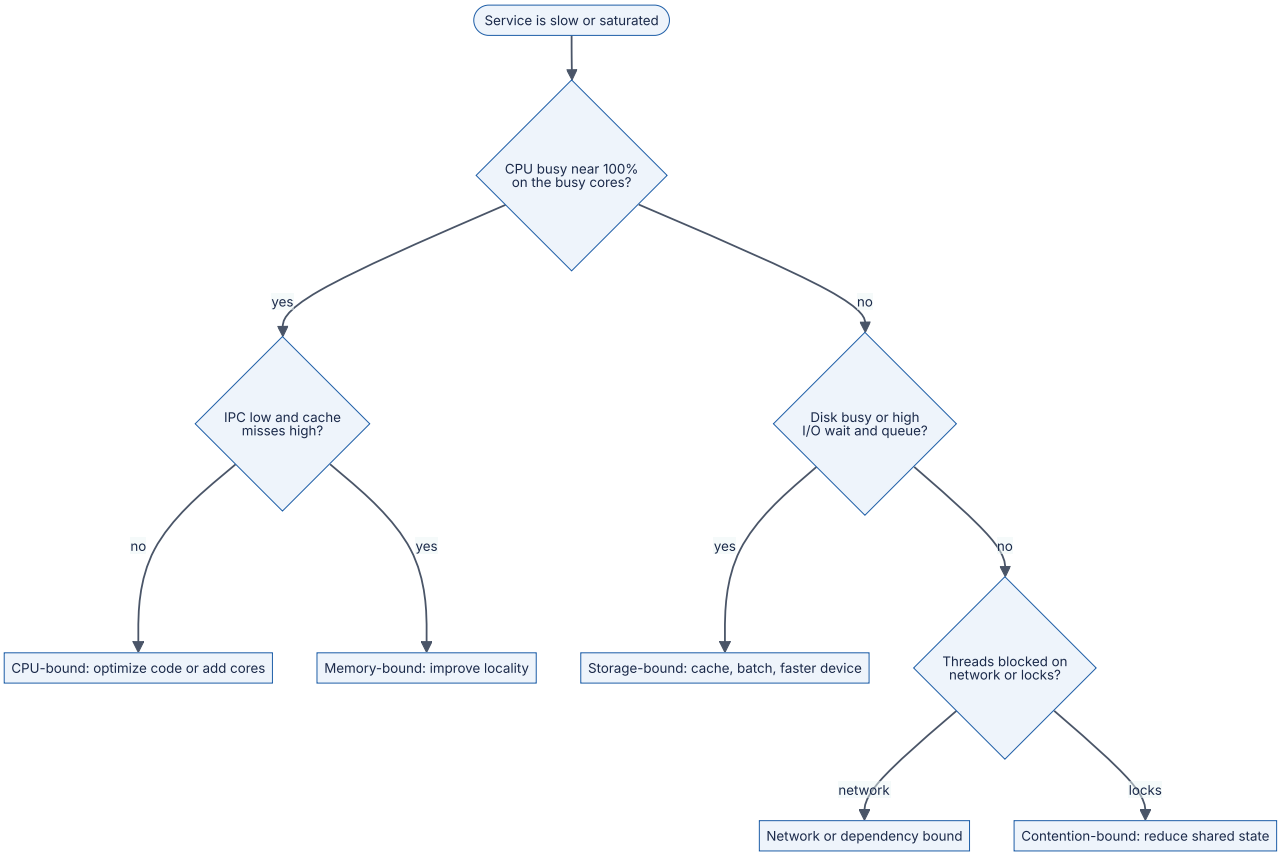
DEFINITION

Definition — CPU-bound and I/O-bound. A workload is **CPU-bound** when its speed is limited by how fast the processor can execute instructions; giving it a faster CPU or more cores speeds it up. It is **I/O-bound** when it spends most of its time waiting for storage or the network; a faster CPU barely helps, but more concurrency or faster devices do. A related category, **memory-bound**, is limited by memory latency or bandwidth (§5.9) — the CPU looks "busy" but is mostly stalled.

A cook who spends all day chopping is CPU-bound: hire more cooks. A cook who spends all day waiting for deliveries is I/O-bound: hiring more cooks only adds more people waiting — what helps is ordering many deliveries at once, or keeping stock closer to hand (caching).

5.8.1 How to tell

Figure 5.8 — Diagnosing what limits a workload.



Reading the diagram. Start at the top with a symptom and walk the decisions using measurements, not guesses. CPU utilization per core (not the machine average, which hides one saturated core) separates "computing" from "waiting". Among busy CPUs, instructions per cycle and cache-miss counters (from `perf stat` on Linux) separate genuine computation from stalls on memory. Among idle CPUs, disk utilization and queue length (`iostat -x`) reveal storage limits, and thread dumps or off-CPU profiles reveal whether threads wait on network calls to dependencies or on locks. Each leaf names a different remedy; applying the wrong one (more threads for a CPU-bound service, a faster CPU for an I/O-bound one) wastes money. This is the resource model of §2.6 applied to one machine; the systematic version is Brendan Gregg's USE method (§23.3) [Gregg 2020].

Table 5.9 — Signals that distinguish CPU-bound from I/O-bound workloads.

Signal	CPU-bound	I/O-bound
Per-core CPU utilization	Near 100% in user time	Low or moderate
`time` output for a command	user + sys ≈ real (× cores used)	real ≫ user + sys
Effect of a faster CPU	Proportional speed-up	Little effect
Effect of more threads	Helps up to the core count, then hurts	Helps until the device or dependency saturates
Typical examples	Compression, encryption, image resizing, JSON parsing at scale, model inference on CPU	Database queries over the network, file servers, API gateways, web crawlers

A caution about the Linux `%iowait` metric: it reports idle time during which some I/O was outstanding. It is a hint, not a measure of I/O capacity; a fully busy CPU can hide I/O waits, and on multi-core machines the number is easy to misread [Gregg 2020].

5.8.2 Why it matters for thread pools

FORMULA

Thread pool sizing heuristic [Goetz et al. 2006, §8.2]

$$N_{threads} = N_{cores} \times U_{target} \times (1 + \frac{W}{C})$$

Symbol	Meaning	Units
$N_{threads}$	Threads in the pool	threads
N_{cores}	Cores available to the pool	cores
U_{target}	Target CPU utilization (0–1)	dimensionless
W	Average time a task spends waiting (I/O, remote calls)	seconds
C	Average CPU time per task	seconds

WORKED EXAMPLE

Worked example — Sizing pools for two services on an 8-core machine.

1. *Image-thumbnail service* (CPU-bound): $W \approx 0$, so $N = 8 \times 1 \times (1 + 0) = 8$ threads. Goetz et al. suggest $N_{cores} + 1$ to cover the occasional page fault. Using 64 threads would add context switches and cache thrashing, not throughput.
2. *Order API* (I/O-bound): each request uses $C = 5$ ms of CPU and waits $W = 50$ ms on the database. At $U_{target} = 1$: $N = 8 \times (1 + 50/5) = 8 \times 11 = 88$ threads. At $U_{target} = 0.8$, leaving headroom: $8 \times 0.8 \times 11 = 70.4 \approx 70$ threads.
3. Maximum throughput of the order API is set by CPU: 8 cores$/5$ ms $= 1600$ requests/s at 100% CPU. Little's Law check: 1600 req/s $\times 55$ ms $= 88$ requests in flight ✓.

Interpretation. The I/O-bound service needs about 11× more threads than cores, because each thread spends 10/11 of its life waiting. **Engineering implication.** The formula assumes the dependency can absorb the load: 88 concurrent requests need about 88 database connections (§14.13). If the database saturates first, more threads only lengthen its queue. Event loops and async runtimes (§6.15) achieve the same concurrency with far fewer threads.

COMMON MISTAKE

“Python threads speed up CPU-bound work.” In CPython the global interpreter lock (GIL) lets only one thread execute Python bytecode at a time, so threads help I/O-bound Python code but not pure-Python CPU-bound code; use processes or native extensions (such as numpy) that release the lock. Free-threaded CPython builds exist experimentally since Python 3.13 (as of 2025), which may change this advice over time.

5.9 Memory latency and bandwidth

5.9.1 Two different numbers

DEFINITION

Definition — Latency and bandwidth (hardware). Latency is the time from issuing one request to receiving its first byte (for example, ~100 ns for a DRAM load). **Bandwidth** is the maximum rate at which data can be moved once transfers are flowing (for example, ~38 GB/s for one DDR5-4800 channel). Achieved bandwidth is **throughput**.

A motorway is the usual picture: latency is how long one car takes to drive from A to B; bandwidth is how many cars per hour the road can carry. Adding lanes raises bandwidth but does not make any single car arrive sooner. For hardware, latency improves slowly (DRAM latency has changed little in two decades), while bandwidth improves steadily with more channels and faster signaling [Hennessy & Patterson 2017, ch. 2]. Designs that need bandwidth can buy it; designs that need low latency must avoid the trip altogether — by caching, prefetching or co-locating data.

FORMULA

Peak DRAM bandwidth

$$BW_{peak} = R_{transfer} \times W_{bus} \times N_{channels}$$

Symbol	Meaning	Units
$R_{transfer}$	Data transfers per second per channel (the "4800" in DDR5-4800)	transfers/second (MT/s)
W_{bus}	Width of a channel's data path: 8 bytes (64 bits)	bytes/transfer
$N_{channels}$	Populated memory channels	channels

WORKED EXAMPLE

Worked example — How much bandwidth does a server socket have?

1. One DDR5-4800 channel: $4800 \times 10^6 \times 8 = 38.4 \times 10^9$ B/s $= 38.4$ GB/s.
2. A socket with 8 channels: $8 \times 38.4 = 307.2$ GB/s. With 12 channels (as on some 2023–2024 server CPUs): $12 \times 38.4 = 460.8$ GB/s.

Interpretation. These are theoretical peaks; sustained throughput on streaming benchmarks is typically 70–90% of peak (approximate). **Engineering implication.** Leaving channels unpopulated to save money (four DIMMs in an eight-channel server) halves memory bandwidth — a common, invisible misconfiguration for analytics and ML workloads.

5.9.2 Why one core cannot use all the bandwidth: Little's Law in memory

Bandwidth is only achieved if enough requests are in flight to cover the latency — the same law as for SSDs (§5.6.5).

WORKED EXAMPLE

Worked example — Memory-level parallelism.

Each core can track a limited number of outstanding L1 misses — on the order of 10–20 on recent x86 cores (as of 2024, approximate; hardware prefetchers into L2 add more). Assume 12 outstanding 64-byte misses and 100 ns latency.

1. Bytes in flight $= 12 \times 64 = 768$ B.
2. Throughput $= 768$ B/100 ns $= 7.68$ B/ns $= 7.68$ GB/s.
3. To saturate an 8-channel socket at 307.2 GB/s with a loaded latency of ~110 ns: bytes in flight $= 307.2 \times 10^{9} \times 110 \times 10^{-9} = 33\,792$ B $= 528$ cache lines.

Interpretation. One thread doing demand loads tops out well below the socket's bandwidth; it takes many cores — or very effective prefetching — to keep 500+ lines in flight. **Engineering implication.** Memory-bound jobs (scans, aggregations, embedding lookups) scale with cores *until* the memory channels saturate, then stop, however many more cores you add. Pointer chasing is worst of all: with one outstanding miss, throughput is 64 B/100 ns = 0.64 GB/s.

5.9.3 Worked examples: scanning 1 GB

The single most useful exercise in this chapter is estimating how long it takes to read 1 GB (10^9 bytes) from each place it might live.

WORKED EXAMPLE

Worked example — Time to read 1 GB (as of 2024–2025, approximate rates).

Sequential reads: $T = 10^9$ B/throughput.

1. DRAM, one thread streaming at ~10 GB/s: $10^9/10^{10} = 0.1$ s $= 100$ ms. Whole socket at ~200 GB/s achieved: 5 ms.
2. NVMe SSD (PCIe 4.0) at ~7 GB/s: $10^9/(7\times10^9) \approx 0.143$ s $= 143$ ms.
3. SATA SSD at ~550 MB/s: $10^9/(5.5\times10^8) \approx 1.82$ s.
4. HDD at ~200 MB/s: $10^9/(2\times10^8) = 5$ s.
5. Network at 1 / 10 / 100 Gbit/s ($= 0.125$ / 1.25 / 12.5 GB/s): 8 s / 0.8 s / 0.08 s, before protocol overhead of a few percent.

Random reads: the same gigabyte read as many small, dependent requests.

6. DRAM, random 64-byte lines, one at a time: $10^9/64 = 15\,625\,000$ lines $\times$ 100 ns $= 1.5625$ s — about 16× slower than streaming the same memory.
7. NVMe, random 4 KiB reads at QD 1: $10^9/4096 \approx 244\,141$ reads $\times$ 80 µs ≈ 19.5 s. At QD 80 (~10^6 IOPS): $244\,141/10^6 \approx 0.24$ s.
8. HDD, random 4 KiB reads: $244\,141 \times 12.69$ ms ≈ 3098 s ≈ 51.6 minutes.

Interpretation. Between the best case (5 ms) and the worst (52 minutes) lies a factor of about 600,000 — for the *same* gigabyte. **Engineering implication.** The access pattern matters as much as the device: random HDD access is 620× slower than sequential HDD access, and random DRAM access is slower than sequential NVMe access. This is why databases build indexes that turn scans into a few targeted reads, why analytics systems store columns contiguously, and why caches exist at every level.

Table 5.10 — Time to read 1 GB, summary (as of 2024–2025, approximate).

Source and pattern	Rate or latency assumed	Time
DRAM, all cores streaming	~200 GB/s	~5 ms
DRAM, one thread streaming	~10 GB/s	~0.1 s
NVMe SSD, sequential	~7 GB/s	~0.14 s
10 Gbit/s network	1.25 GB/s	~0.8 s
DRAM, dependent random 64 B reads	100 ns each	~1.6 s
SATA SSD, sequential	0.55 GB/s	~1.8 s
HDD, sequential	0.2 GB/s	~5 s
NVMe SSD, random 4 KiB, QD 1	80 µs each	~20 s
HDD, random 4 KiB	12.7 ms each	~52 min

5.9.4 Latency numbers, then and now

Jeff Dean's list of "numbers everyone should know", presented in 2009 and building on a similar table by Peter Norvig, is the best-known summary of these facts [Dean 2009; Norvig 2001]. §4.17 discusses it in full; Table 5.11 shows how the hardware-specific entries compare with this chapter's figures.

Table 5.11 — Classic latency numbers versus approximate 2024–2025 values.

Operation	Dean 2009	Approx. 2024–2025 (this chapter)	What changed
L1 cache reference	0.5 ns	~1 ns	About the same in cycles; clocks stalled near 3–5 GHz
Branch mispredict	5 ns	~4–5 ns (15–20 cycles)	Little change
L2 cache reference	7 ns	~3–5 ns	Somewhat faster
Main memory reference	100 ns	~80–120 ns	Essentially unchanged
Read 1 MB sequentially from memory	250 µs	~50–100 µs per thread	Bandwidth grew ~3–5×
Disk seek	10 ms	~4–10 ms	Mechanics barely changed
Read 1 MB sequentially from disk	20 ms	~5 ms	Higher areal density
Random read from SSD	Not in 2009 list (150 µs in later versions)	~50–100 µs	SSDs became mainstream
Read 1 MB sequentially from NVMe SSD	Not in 2009 list (1 ms for SSD in later versions)	~0.15–0.3 ms	New tier

Versions of Dean's list differ slightly in individual entries; the durable lesson is the *shape*: memory latency stuck, bandwidth up, disks still mechanical, flash filling the gap between them.

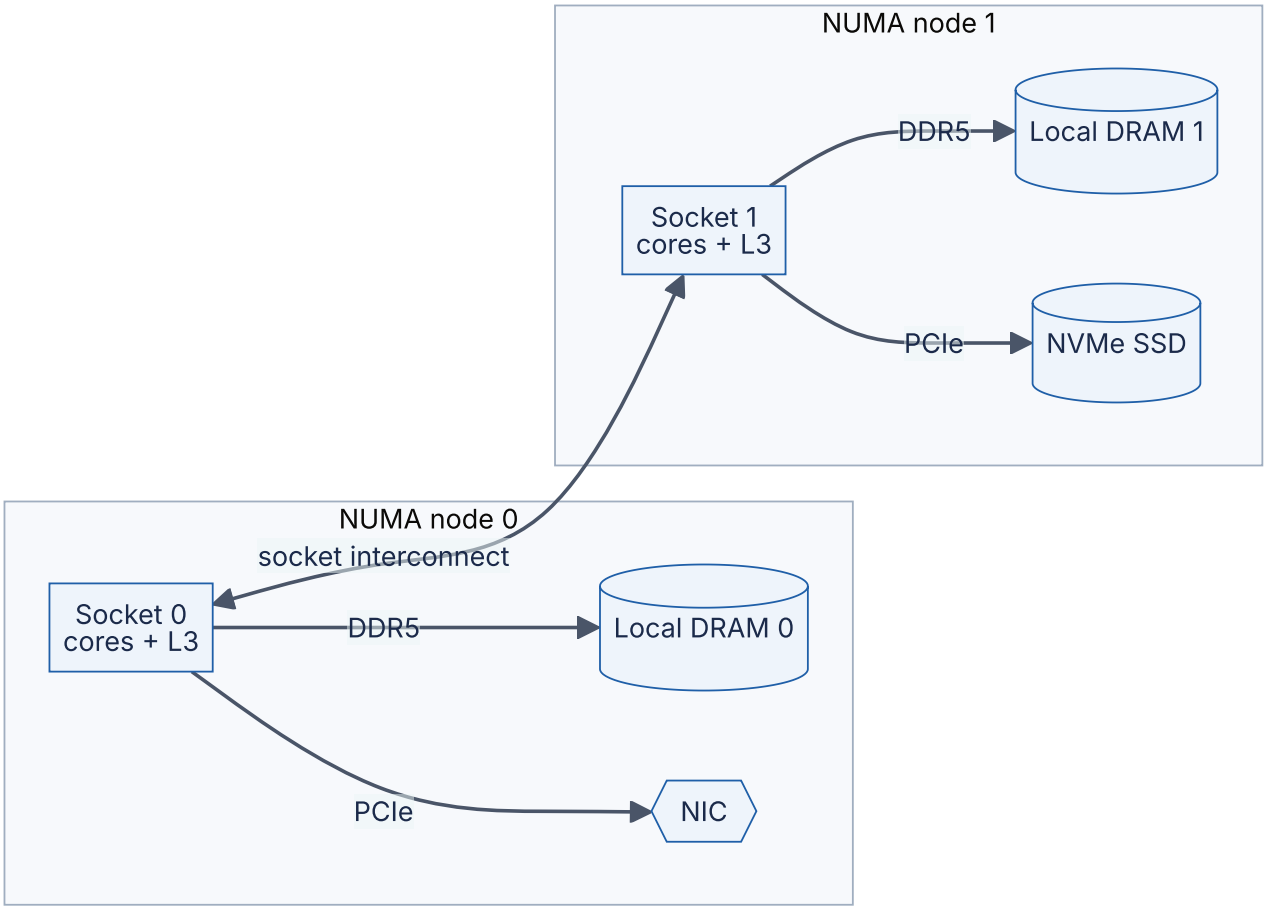
5.10 NUMA

DEFINITION

Definition — NUMA (non-uniform memory access). A memory architecture in which each processor socket (or each chiplet group within a socket) has its own directly attached memory. A core can access all memory in the machine, but accessing its **local** memory is faster than accessing **remote** memory attached to another socket.

Picture two kitchens joined by a corridor, each with its own fridge. A chef can fetch from either fridge, but the far one takes noticeably longer and the corridor can get crowded. NUMA exists because a single shared memory controller cannot scale to dozens of cores and hundreds of GB/s; attaching memory to each socket multiplies total bandwidth, at the price of uneven latency [Lameter 2013].

Figure 5.9 — A two-socket NUMA server.



Reading the diagram. Each socket, with its cores, its L3 cache, its memory channels and its PCIe devices, forms a **NUMA node**. A core on socket 0 reaches DRAM 0 directly; to reach DRAM 1 its request travels over the **socket-to-socket interconnect** (Intel UPI, AMD Infinity Fabric), adding latency — remote accesses commonly cost about 1.5–2× local ones (as of 2024, approximate; Linux reports relative distances via `numactl --hardware`, typically 10 for local and around 20 or more for remote). Devices are local to one node as well: packets arriving on a NIC attached to socket 0 are cheapest to process on socket 0's cores. The interconnect is shared and carries coherence traffic too, so it is both a latency penalty and a bandwidth bottleneck. Failure points are performance failures — a process whose threads run on node 1 while its memory sits on node 0. Scaling within the machine means keeping threads, their memory and their devices on the same node.

WORKED EXAMPLE

Worked example — The cost of ignoring NUMA.

Assume local DRAM latency 80 ns and remote 140 ns.

1. Two sockets, memory placed without regard to which node will use it: about half of accesses are remote. Average $= 0.5 \times 80 + 0.5 \times 140 = 40 + 70 = 110$ ns, 37.5% worse than all-local.
2. Four sockets, memory interleaved evenly: 75% remote. Average $= 0.25 \times 80 + 0.75 \times 140 = 20 + 105 = 125$ ns, 56% worse.

Interpretation. For memory-bound work, NUMA-unaware placement directly inflates latency and congests the interconnect. **Engineering implication.** On large servers, pin latency-sensitive processes to one node (`numactl --cpunodebind=0 --membind=0`), run one process per node rather than one huge process, or prefer smaller single-socket instances. Cloud VMs that span sockets expose NUMA topology too.

INDUSTRY EXAMPLE

MySQL “swap insanity” on two-socket servers. Jeremy Cole described MySQL servers with a buffer pool sized for the whole machine’s RAM that nevertheless swapped to disk. Linux’s default policy allocated memory from the local node of the allocating thread, so one NUMA node filled up while the other had free memory, and the kernel swapped instead of using the remote node. His recommended remedy was to start MySQL with memory interleaved across nodes (`numactl --interleave=all`), accepting uniformly higher latency in exchange for predictable behavior; later MySQL versions added an option to interleave the buffer pool. *Evidence: Primary* — [Cole 2010]

COMMON MISTAKE

“All RAM in a server is equally fast.” On multi-socket and many-chiplet servers it is not. Check `lscpu` and `numactl --hardware` before benchmarking, and keep the benchmark’s threads and memory on the same node or document that you did not.

5.11 SIMD and accelerators

5.11.1 SIMD

DEFINITION

Definition — SIMD (single instruction, multiple data). CPU instructions that apply the same operation to several data elements packed into one wide **vector register** at once — for example, adding eight pairs of 32-bit floats in a single instruction.

Instead of a cook slicing one carrot at a time, SIMD is a mandoline that slices eight at once with a single stroke. It pays off when the same operation runs over large arrays of uniform data: image and audio processing, compression, checksums, string search, database column scans and machine-learning arithmetic.

Table 5.12 — Common SIMD instruction sets (as of 2025).

Instruction set	Platform	Register width	32-bit floats per instruction
SSE family	x86	128 bits	4
AVX / AVX2	x86	256 bits	8
AVX-512	x86 (many server parts)	512 bits	16
NEON (Advanced SIMD)	ARM	128 bits	4
SVE / SVE2	ARM (some server parts)	128–2048 bits, implementation-defined	4–64

You rarely write SIMD instructions by hand. Compilers **auto-vectorize** simple loops, and libraries such as numpy, BLAS and database engines ship vectorized kernels. Your job is to give them data they can vectorize: contiguous arrays of one type, not lists of objects. The database research system MonetDB/X100 showed that processing data in cache-sized *vectors* of column values, rather than one row at a time, cut interpretation overhead and enabled SIMD, which shaped modern vectorized analytics engines [Boncz et al. 2005].

```python
"""simd.py — interpreted loop vs vectorized (SIMD-backed) numpy."""
import time
import numpy as np

n = 10_000_000
a = np.random.rand(n).astype(np.float32)
b = np.random.rand(n).astype(np.float32)

t0 = time.perf_counter()
out = [0.0] * n
for i in range(n):                  # one element at a time, interpreted
    out[i] = a[i] * b[i] + 1.0
t1 = time.perf_counter()
c = a * b + 1.0                     # whole-array ops in compiled, SIMD loops
t2 = time.perf_counter()

print(f"python loop: {t1 - t0:.2f} s")
print(f"numpy:       {t2 - t1:.4f} s  ({(t1 - t0) / (t2 - t1):.0f}x faster)")
```

What to notice:

- Both versions compute the same 10 million results.
- The speed-up (about 20× on the test VM in 2026; often larger) comes from two sources: removing the interpreter's per-element overhead, and SIMD in numpy's compiled loops. Do not attribute it all to SIMD.
- `a * b + 1.0` makes two passes over memory and allocates a temporary array; at this size the vectorized version is limited by memory bandwidth (§5.9), not arithmetic.

5.11.2 Accelerators

When one kind of computation dominates a workload, it can pay to build or buy hardware specialized for it:

- **GPUs** — thousands of simple arithmetic lanes and very high-bandwidth memory (HBM), designed for massively parallel, regular computation. They dominate ML training and inference. Chapter 64 covers their architecture, the roofline model [Williams et al. 2009] and the compute-bound versus memory-bound distinction in depth.
- **TPUs and other ML ASICs** — chips built around large matrix-multiply units [Jouppi et al. 2017].
- **Fixed-function units inside the CPU** — AES and SHA instructions make TLS encryption cheap; compression and cryptography offload engines exist on some server CPUs and NICs.
- **SmartNICs / DPUs and FPGAs** — network cards with programmable processors that offload packet processing, storage virtualization or encryption from the host CPU.

The design question is always whether the workload is large and regular enough to justify specialized hardware, and whether data can reach it fast enough — an accelerator starved by PCIe or by CPU-side data preparation delivers a fraction of its rated speed.

5.12 Throughput versus latency at the hardware level

5.12.1 The pattern underneath everything

Every component in this chapter shows the same trade-off. A **pipeline** raises instruction throughput without shortening any instruction. **Out-of-order** cores and **memory-level parallelism** raise memory throughput by keeping many misses in flight. **NVMe queues** raise IOPS by keeping dozens of requests outstanding. **Interrupt coalescing** raises packet throughput by delaying notifications. In each case Little's Law ($L = \lambda W$, §4.11) holds: to get throughput λ from a resource with latency W, you need $L = \lambda W$ operations in flight. Latency is mostly fixed by physics; throughput is bought with parallelism and batching.

FORMULA

Throughput with batching (fixed plus per-item cost)

$$X(B) = \frac{B}{F + B \times v}, \qquad T_{batch}(B) = F + B \times v, \qquad X_{max} = \frac{1}{v}$$

Symbol	Meaning	Units
B	Items per batch	items
F	Fixed cost per operation (system call, device round trip, interrupt, network RTT)	seconds
v	Variable cost per item	seconds/item
$X(B)$	Throughput at batch size B	items/second
$T_{batch}(B)$	Time to complete one batch	seconds

WORKED EXAMPLE

Worked example — Batching small writes.

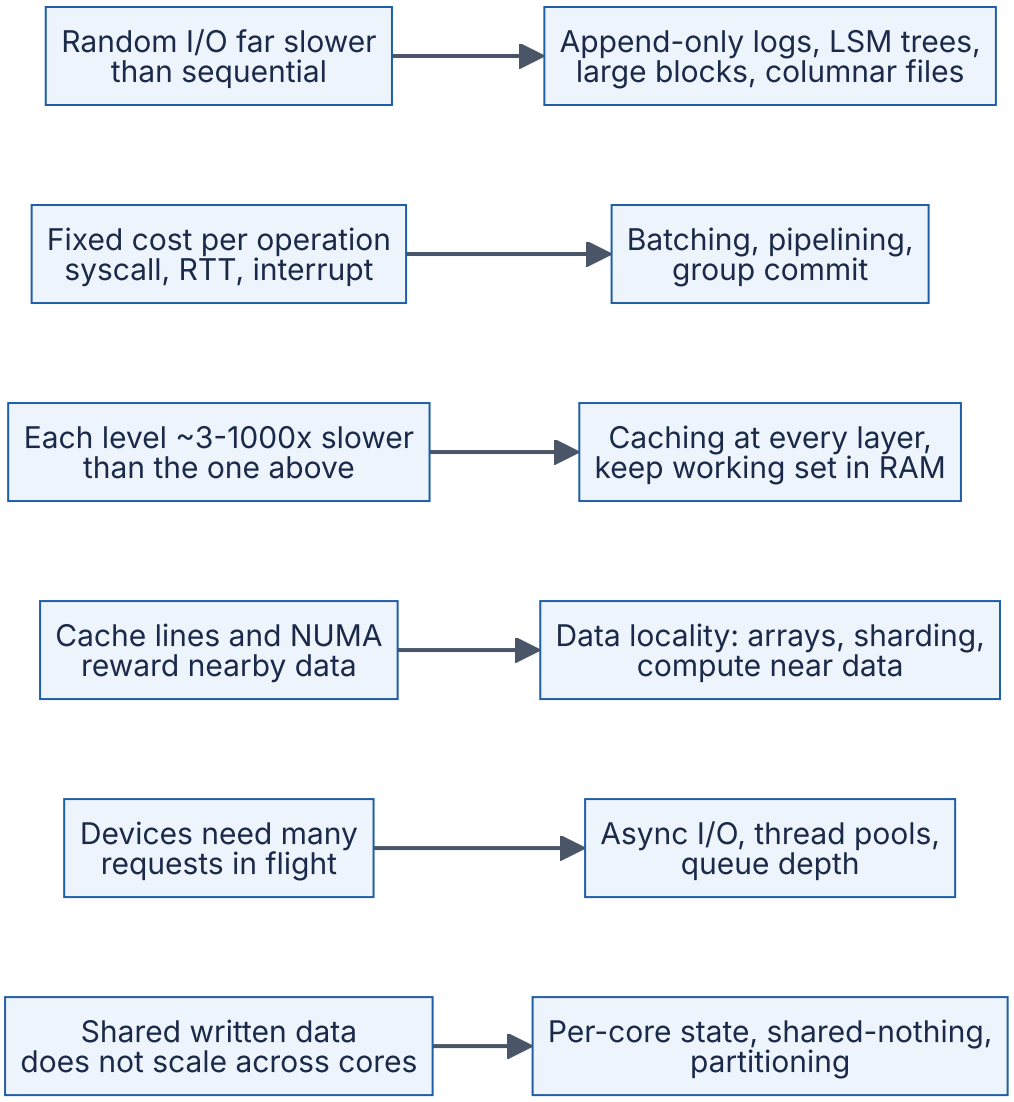
Writing 100-byte records to a log, each write call has fixed cost $F = 50$ µs and per-record cost $v = 0.1$ µs.

1. $B = 1$: $X = 1/50.1\ \mu s \approx 19\,960$ records/s; each write completes in 50.1 µs.
2. $B = 100$: $T = 50 + 100 \times 0.1 = 60$ µs; $X = 100/60\ \mu s \approx 1.67$ million records/s.
3. $B = 1000$: $T = 50 + 100 = 150$ µs; $X = 1000/150\ \mu s \approx 6.67$ million records/s.
4. Ceiling: $X_{max} = 1/0.1\ \mu s = 10$ million records/s.

Interpretation. Batching 100 records multiplies throughput by about 84× while raising per-batch latency only from 50 µs to 60 µs; going to 1000 gains another 4× in throughput but per-batch latency rises 2.5×, plus the time spent *waiting* for the batch to fill. **Engineering implication.** Batch until the fixed cost is amortized (a few times F/v items), then stop; bound the wait with a timer. Kafka producers (`linger.ms` , §31.6), group commit in databases (§13.5) and dynamic batching in model serving (Ch 66) all implement this curve.

5.12.2 How hardware facts drive system design decisions

Figure 5.10 — From hardware facts to design decisions.



Reading the diagram. Each row pairs a physical fact (left) with the design principle it forces (right); arrows mean “therefore”. None of these is a fashion — each follows from numbers you calculated in this chapter. The same principles reappear at datacenter scale: a cross-region network round trip is to a datacenter what a DRAM miss is to a core, so caching, batching, locality and parallelism return in Part IV (scalability, caching, CDNs) and Part V (distributed systems). When a design violates one of these pairs — chatty per-item network calls, random writes to disks, a global counter updated by every core — it usually fails at scale for precisely the reason on the left.

Table 5.13 — Hardware facts and the design decisions they drive.

Hardware fact (§)	Design decision	Examples in this book
Sequential ≫ random on HDD (~620×) and still better on SSD (§5.6)	Write sequentially; read in large chunks	Write-ahead logs (§13.5), LSM trees (§13.8), Kafka (§31.13)
Fixed cost per operation (§5.7, §5.12)	Batch and pipeline	Kafka batching (§31.6), HTTP/2 multiplexing (§7.9), dynamic batching (Ch 66)
Memory hierarchy steps of 3–1000× (§5.5)	Cache hot data close to its consumer	Buffer pools (§13.4), Redis (§20.6), CDNs (Ch 21)
Cache lines, prefetching, NUMA (§5.4, §5.10)	Keep related data together and near the compute	Columnar storage (§13.13), data locality in Spark (§29.7)
Devices need queue depth (§5.6.5)	Concurrency in the I/O path	io_uring and event loops (§6.13), connection pools (§7.16)
Coherence makes shared writes expensive (§5.4.6)	Avoid shared mutable state; partition it	Shard-per-core designs, partitioned databases (Ch 14)
DRAM is volatile (§5.5)	Durability needs persistent media or replicas	fsync (§6.14), replication (Ch 27)

INDUSTRY EXAMPLE

ScyllaDB and the Seastar framework (shard-per-core). ScyllaDB, a Cassandra-compatible database, is built on the Seastar framework, which runs one thread per core, partitions data so that each core owns its shard, and passes messages between cores instead of sharing locked data structures. The design avoids cache-line contention and lock overhead and keeps each core's data in its own caches and local NUMA memory. Trade-off: everything must be asynchronous, and skewed keys can overload a single core. *Evidence: Primary* — [Seastar Documentation]

RECOMMENDATION

Mechanical sympathy in practice. (1) Estimate before you build: write down the latency and bandwidth of each hop. (2) Prefer arrays and contiguous layouts for hot data. (3) Turn many small operations into fewer large ones. (4) Keep enough requests in flight to use your devices. (5) Do not share frequently written data across cores or machines. (6) Measure with hardware counters (`perf`) before and after; intuition about hardware is often wrong.

INTERVIEW TIP

Interview tip. When an interviewer asks "why is X fast?" (Kafka, Redis, a columnar database), the answer almost always maps onto one row of Table 5.13. Say the hardware fact and its number first, then the design choice.

5.13 Summary and revision notes

SUMMARY

- A CPU core repeatedly fetches, decodes and executes instructions. Execution time = instructions × CPI × cycle time; clock rates stalled near 3–5 GHz around 2005, so performance now comes from more work per cycle and more cores.
- Pipelining, superscalar issue and out-of-order execution raise throughput by overlapping instructions; branch prediction keeps the pipeline fed. Mispredictions (~15–20 cycles) and dependent memory accesses are what slow cores down.
- SMT (hyperthreading) shares one core between two hardware threads; a cloud vCPU is usually one such thread, not a full core.
- Caches (L1 ~1 ns, L2 ~4 ns, L3 ~15 ns, DRAM ~100 ns; as of 2024–2025, approximate) exploit temporal and spatial locality. Data moves in 64-byte cache lines; sequential access is the best case and pointer chasing the worst.
- AMAT = hit time + miss rate × miss penalty, applied level by level; layout alone can change it 10×.
- MESI keeps caches coherent per line; writes to one line from several cores cause ping-pong. False sharing is unrelated variables in one line; fix it with padding or per-thread data.
- The memory hierarchy spans ~1 ns to ~10 ms and about 1000× in cost per GB; DRAM and above are volatile.
- HDD access = seek + rotational latency + transfer; ~80–200 random IOPS versus ~200 MB/s sequential (~620× gap for 4 KiB reads).
- Flash writes pages, erases blocks and wears out; the FTL remaps writes, garbage-collects and causes write amplification. NVMe's many deep queues unlock flash parallelism that SATA's single 32-entry queue cannot.
- IOPS = queue depth / latency: fast devices need many requests in flight.
- I/O evolves from programmed I/O to interrupts + DMA to polling and kernel bypass as event rates rise.
- CPU-bound work needs ~one thread per core; I/O-bound work needs about N_cores × (1 + W/C) threads, limited by downstream capacity.
- Latency barely improves; bandwidth does. Peak DRAM bandwidth = transfer rate × 8 bytes × channels; achieving it needs hundreds of misses in flight.
- NUMA makes remote memory ~1.5–2× slower; keep threads, memory and devices on one node.
- SIMD and accelerators speed up regular, data-parallel work if data is contiguous and can be fed fast enough.

- Hardware facts drive design: sequential I/O, batching, caching, data locality, concurrency in the I/O path, and no shared mutable hot spots.

5.14 Knowledge checkpoint

1. Using the iron law, what happens to execution time if CPI rises from 0.5 to 0.75 at a fixed clock rate and instruction count?
2. Why does a deeper pipeline make branch mispredictions more expensive?
3. Why can an out-of-order core hide the latency of ten independent loads but not ten loads in a linked list?
4. What is the difference between temporal and spatial locality? Give one example of each.
5. In a 32 KiB, 8-way cache with 64-byte lines, how many sets are there, and why are addresses 4 KiB apart a problem?
6. Compute AMAT for t_{L1} = 1 ns, m_{L1} = 0.1, and a flat 50 ns penalty for every L1 miss.
7. Name the four MESI states and say which one allows a write without any coherence traffic.
8. Two threads increment separate counters and get slower when you add the second thread. What is the likely cause and the fix?
9. A 5400 RPM disk has an average seek of 10 ms. What is its average rotational latency and approximate random-read IOPS for small blocks?
10. Why can a flash SSD not overwrite a page in place, and what does the FTL do instead?
11. A device has 100 µs latency. What queue depth is needed for 400,000 IOPS?
12. What is DMA and what problem does it solve?
13. A service's `time` output shows real = 10 s, user + sys = 0.5 s. Is it CPU-bound or I/O-bound? What would you change first?
14. What is the peak bandwidth of six DDR5-5600 channels?
15. On a two-socket server, local latency is 90 ns and remote 150 ns. What is the average latency if 30% of accesses are remote?

Answers

1. Time rises by 50% (0.75 / 0.5 = 1.5), because time is proportional to CPI.
2. The branch outcome is known later in a deeper pipeline, so more stages hold wrongly fetched instructions that must be flushed; more cycles are lost per misprediction.
3. Independent loads can all be issued and outstanding at once, overlapping their latency. In a linked list each address is known only after the previous load returns, so the latencies add up serially.

4. Temporal: reusing the same data soon (a loop counter, a hot config value). Spatial: using nearby data soon (the next array element, adjacent struct fields).
5. $32\,768/(64 \times 8) = 64$ sets. Addresses $64 \times 64 = 4096$ bytes apart map to the same set, so a 4 KiB stride can use only 8 lines (one set) and suffers conflict misses.
6. $1 + 0.1 \times 50 = 6$ ns.
7. Modified, Exclusive, Shared, Invalid. Writes to an Exclusive line (moving it to Modified) need no bus/interconnect traffic; a line already in Modified also needs none.
8. False sharing: both counters are in one cache line, which ping-pongs between cores. Pad each counter to its own 64-byte line or keep per-thread counters and combine them at the end.
9. One revolution $= 60/5400 = 11.1$ ms; average rotational latency $= 5.56$ ms. Access $\approx 10 + 5.56 \approx 15.6$ ms, so $\approx 1/0.0156 \approx 64$ IOPS.
10. Flash must be erased before being reprogrammed, and erase works only on whole blocks of many pages. The FTL writes the new data to a free page, updates its logical-to-physical map, marks the old page invalid and later garbage-collects blocks.
11. $QD = 400\,000 \times 100 \times 10^{-6} = 40$ outstanding requests.
12. Direct memory access lets a device copy data to or from RAM itself; the CPU only sets up the transfer and handles the completion, so CPU cycles are not spent copying bytes.
13. I/O-bound (or waiting on something): the CPU was busy only 5% of the time. Find what it waits on — disk, network or locks — and add concurrency, caching or batching rather than a faster CPU.
14. $5600 \times 10^6 \times 8 \times 6 = 268.8 \times 10^9$ B/s $= 268.8$ GB/s.
15. $0.7 \times 90 + 0.3 \times 150 = 63 + 45 = 108$ ns.

5.15 Interview questions

Basic

Q1. Why is reading data sequentially faster than reading it randomly, on both disks and memory? Answer. On an HDD a random read pays a seek (~4–10 ms) and half a rotation (~4 ms at 7200 RPM) for every request, while a sequential read pays them once and then streams at ~200 MB/s — about 620× faster for 4 KiB requests. In memory, data arrives in 64-byte cache lines and hardware prefetchers detect sequential patterns and fetch ahead, so sequential reads hit in cache while random reads each pay ~100 ns of DRAM latency. SSDs narrow the gap but still favor large sequential I/O.

Q2. What is a cache line and why should a programmer care? Answer. The 64-byte (on most x86 and ARM servers) unit in which data moves between memory and caches. It means neighbors of anything you read are nearly free (so arrays and compact structs are

fast), and that coherence works per line, so threads writing different variables in the same line interfere (false sharing).

Intermediate

Q3. You moved a service from an 8-vCPU to a 32-vCPU instance and throughput rose only 30%. What could explain this, and how would you investigate? Answer. Candidates: (1) the service is limited by something other than CPU — a database, a lock, a single-threaded component (check per-thread CPU, thread dumps); (2) contention on shared cache lines (atomic counters, false sharing, a JVM internal structure) — check with `perf c2c` or cache-miss counters; (3) NUMA effects if the larger instance spans sockets; (4) vCPUs are hyperthreads, so 32 vCPUs are 16 cores. Measure per-core utilization and IPC before and after, then test hypotheses one by one. Netflix's published 2022 case was exactly this pattern, caused by cache-line sharing in the JVM.

Q4. How would you size a thread pool for a service that calls a database? Answer. Measure CPU time per request C and wait time W. Start with $N = N_{cores} \times U \times (1 + W/C)$; e.g., 8 cores, $C = 5$ ms, $W = 50$ ms gives ~88 threads at full utilization. Then cap it by the downstream: the database connection pool and the database's own capacity. Validate with a load test, watching p99 latency and CPU; if latency rises without throughput gain, the pool is too large. For very high concurrency, consider async I/O instead.

Advanced

Q5. Explain why Kafka can sustain high throughput even on hard disks. Answer. Each partition is an append-only log, so writes are sequential (no seeks). Reads by consumers are also mostly sequential and usually served from the OS page cache because recently written data is still in RAM. Producers and brokers batch many messages per request, amortizing per-request fixed costs (Table 5.13). Data is sent from page cache to socket without copying through user space (zero-copy, §31.13). Each design choice maps to a hardware fact: sequential ≫ random, fixed per-operation costs, and DRAM ≫ disk.

Q6. Design the counters for a metrics library that must record 50 million increments per second across 64 cores. Answer. A single shared atomic counter would ping-pong one cache line between 64 cores and cap at a few tens of millions of updates per second or less, with heavy cross-core traffic. Instead, give each core (or thread) its own counter, each padded to its own cache line (or grouped per core in a per-core region), and increment it with plain or relaxed atomic operations. A reader sums all slots when scraping — reads are rare, so their cost is irrelevant. This trades a little memory (64 lines × 64 B = 4 KiB per counter) and slightly stale reads for near-linear scaling. Keep each core's slots on its own NUMA node. This is the same "partition, don't share" principle used by shard-per-core databases.

5.16 Practical exercises

1. **Cache-line stride benchmark (Python + numpy).** Run the program below. Plot nanoseconds per element against stride. Explain why the cost per element rises until the stride reaches about 64–128 bytes and then levels off, and why total time stays roughly constant for strides up to 64 bytes.

```
"""stride.py — how access stride affects time per element touched.

Run: python3 stride.py   (needs numpy)
"""
import time
import numpy as np

N_BYTES = 256 * 1024 * 1024            # 256 MiB buffer, far larger than any L3
data = np.ones(N_BYTES // 8, dtype=np.int64)   # 8-byte elements

def ns_per_element(stride_elems: int, repeats: int = 3) -> float:
    view = data[::stride_elems]          # strided view, no copy
    best = float("inf")
    for _ in range(repeats):
        t0 = time.perf_counter()
        view.sum()                       # touches every element of the view
        best = min(best, time.perf_counter() - t0)
    return best / view.size * 1e9

print(f"{'stride (B)':>10} {'elements':>12} {'ns/element':>11}")
for stride in [1, 2, 4, 8, 16, 32, 64, 128, 512]:
    n = data[::stride].size
    print(f"{stride * 8:>10} {n:>12,} {ns_per_element(stride):>11.2f}")
```

What to notice: up to a 64-byte stride every cache line is still fetched, so total time is dominated by moving 256 MiB from DRAM and the time *per element* grows as fewer elements share each line; beyond 64 bytes each element costs one line fetch (plus adjacent-line prefetch effects up to ~128 bytes). On the test VM the cost rose from ~0.7 ns/element at an 8-byte stride to ~7 ns/element at 256 bytes and above.

2. **False sharing.** Compile and run `false_sharing.c` from §5.4.7. Then change `CACHE_LINE` to 32 and 128 and explain the results. Finally, run the packed version with both threads pinned to the same core (`taskset -c 0 ./fs`) and explain why the penalty disappears.
3. **Disk and SSD.** Using `fio`, measure random 4 KiB read IOPS at queue depths 1, 4, 16 and 64 on a scratch file on your SSD. Check that IOPS ≈ QD / latency until the device saturates.
4. **CPU-bound or I/O-bound?** Run a compression job (`gzip` on a large file) and a script that downloads 100 URLs sequentially, each under `time`. Compare real versus user + sys and watch `vmstat 1` during each.

5. **Topology tour.** On a Linux machine, record the output of `lscpu`, `lscpu -C` (cache sizes and line size) and `numactl --hardware`. Draw your own version of Figure 5.1 with real numbers.

5.17 Design challenges

1. **Log-structured key-value store on HDDs.** You must store 20 TB of 1 KB values on HDDs with 50,000 writes/s and 5,000 random reads/s. Estimate how many disks a random-write design would need versus an append-only log with an in-memory index, and what the memory cost of the index would be.
2. **Analytics scan budget.** Analysts want to scan a 2 TB table in under 10 seconds. Using Table 5.10, decide whether this is feasible from NVMe on one machine, from RAM on one machine, or only across a cluster, and how many nodes and what storage format (row or column) you would choose.
3. **Low-latency order gateway.** A trading gateway must handle 2 million messages/s with p99 under 20 µs on one server. Choose between interrupts, polling and kernel bypass for the NIC; decide thread-to-core and NUMA placement; and explain how you would avoid false sharing between the receive and send threads.

5.18 References

1. Agrawal, N., Prabhakaran, V., Wobber, T., Davis, J. D., Manasse, M., Panigrahy, R. *Design Tradeoffs for SSD Performance*. USENIX Annual Technical Conference, 2008.
2. Apache Software Foundation. *Apache Kafka Documentation — Design*. https://kafka.apache.org/documentation/#design
3. Axboe, J. *Efficient IO with io_uring*. 2019. https://kernel.dk/io_uring.pdf
4. Boncz, P., Zukowski, M., Nes, N. *MonetDB/X100: Hyper-Pipelining Query Execution*. CIDR, 2005.
5. Bryant, R. E., O'Hallaron, D. R. *Computer Systems: A Programmer's Perspective*, 3rd ed. Pearson, 2015.
6. Cole, J. *The MySQL "swap insanity" problem and the effects of the NUMA architecture*. Blog post, 2010.
7. Dean, J. *Designs, Lessons and Advice from Building Large Distributed Systems*. Keynote, LADIS Workshop, 2009.
8. DPDK Project. *Data Plane Development Kit Documentation*. https://doc.dpdk.org/
9. Drepper, U. *What Every Programmer Should Know About Memory*. Red Hat, 2007. https://www.akkadia.org/drepper/cpumemory.pdf
10. Filanovsky, V., Sane, H. *Seeing through hardware counters: a journey to threefold performance increase*. Netflix Technology Blog, 2022.

11. Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., Lea, D. *Java Concurrency in Practice*. Addison-Wesley, 2006.
12. Gregg, B. *Systems Performance: Enterprise and the Cloud*, 2nd ed. Addison-Wesley, 2020.
13. Hennessy, J. L., Patterson, D. A. *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.
14. Jouppi, N. P., et al. *In-Datacenter Performance Analysis of a Tensor Processing Unit*. ISCA, 2017.
15. Lameter, C. *NUMA (Non-Uniform Memory Access): An Overview*. ACM Queue 11(7), 2013.
16. LTO Program. *LTO technology specifications (LTO-9)*. https://www.lto.org/
17. Marr, D. T., Binns, F., Hill, D. L., Hinton, G., Koufaty, D. A., Miller, J. A., Upton, M. *Hyper-Threading Technology Architecture and Microarchitecture*. Intel Technology Journal 6(1), 2002.
18. Norvig, P. *Teach Yourself Programming in Ten Years*. 2001. https://norvig.com/21-days.html
19. NVM Express, Inc. *NVM Express Base Specification*. https://nvmexpress.org/specifications/
20. Papamarcos, M. S., Patel, J. H. *A Low-Overhead Coherence Solution for Multiprocessors with Private Cache Memories*. ISCA, 1984.
21. Ruemmler, C., Wilkes, J. *An Introduction to Disk Drive Modeling*. IEEE Computer 27(3), 1994.
22. Seastar Project. *Seastar Documentation*. https://seastar.io/
23. Sorin, D. J., Hill, M. D., Wood, D. A. *A Primer on Memory Consistency and Cache Coherence*. Morgan & Claypool, 2011.
24. SPDK Project. *Storage Performance Development Kit Documentation*. https://spdk.io/
25. Thompson, M., Farley, D., Barker, M., Gee, P., Stewart, A. *Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads*. LMAX, 2011. https://lmax-exchange.github.io/disruptor/
26. Tomasulo, R. M. *An Efficient Algorithm for Exploiting Multiple Arithmetic Units*. IBM Journal of Research and Development 11(1), 1967.
27. Williams, S., Waterman, A., Patterson, D. *Roofline: An Insightful Visual Performance Model for Multicore Architectures*. Communications of the ACM 52(4), 2009.

5.19 Open verification items

- Cost per GB in Table 5.5 and Table 5.6 reflects approximate 2024 street prices; DRAM and NAND prices moved sharply in 2025–2026. Re-check before publication.
- Exact URLs for [Dean 2009] (LADIS keynote slides), [Cole 2010] and [Filanovsky & Sane 2022] are omitted pending verification; confirm the specific entries of Dean’s

list (versions differ, e.g. 20 ms vs 30 ms for 1 MB from disk).
- Confirm the “up to ~30%” Hyper-Threading gain against [Marr et al. 2002] and the MySQL release that added buffer-pool NUMA interleaving.
- Cite a primary source (Intel investor communication, 2022) for the Optane wind-down.
- Microarchitectural ranges (branch-mispredict penalty 15–20 cycles; 10–20 outstanding L1 misses per core; reorder buffer “several hundred” entries) should be checked against current Intel/AMD/Arm optimization manuals.
- TLC endurance of 1,000–3,000 P/E cycles is a commonly quoted range, not a vendor specification.
- Status of free-threaded CPython (experimental from 3.13) may have changed.

Chapter 6 — Operating Systems

> **Level:** 1–2 · **Track:** Shared · **Estimated reading time:** 3 h

Every service you will design in this book, whether a URL shortener, a Kafka broker or an LLM inference server, is in the end a set of processes running on an operating system. The operating system decides which of them runs, how much memory each one sees, how its bytes reach the network card and the disk, and what happens when two of them touch the same data at once. When a service is "slow for no reason", the reason is often here: a context-switch storm, a page-fault spike, a thread pool sized for 200 that is serving 20,000 connections, or an `fsync` that took 400 ms. This chapter gives you the vocabulary and the mental models to see those problems, estimate them with numbers, and choose designs that avoid them.

6.1 Learning objectives

After this chapter you should be able to:

- Explain what a process and a thread are, draw the process state diagram, and list what the kernel stores in a process control block.
- Describe the steps of a context switch and estimate both its direct cost and its indirect cache cost.
- Compare FCFS, round robin, priority, MLFQ and Linux's CFS/EEVDF schedulers, and calculate turnaround and response times for a small workload.
- Trace a memory access through the TLB, page table and page-fault handler, and calculate effective access time and TLB reach.
- Explain what a system call is, why it costs more than a function call, and how batching and vDSO reduce that cost.
- Distinguish concurrency from parallelism and identify a race condition and its critical section in code.
- Choose between a mutex, semaphore, condition variable, atomic/CAS operation, read-write lock and lock-free structure, and explain the ABA problem.
- State the four Coffman conditions, apply prevention, avoidance and detection to deadlock, and explain priority inversion using the Mars Pathfinder incident.
- Compare blocking, non-blocking, multiplexed (`select` / `poll` / `epoll` / `kqueue`) and completion-based (`io_uring`) I/O, and explain the C10K problem.
- Write a crash-safe file update using `fsync` correctly and name the common durability pitfalls.

- Compare thread-per-request, event loop, actor, goroutine and async/await concurrency models and size a server's memory for a target number of concurrent connections.

6.2 Prerequisites

- CPU cores, registers and caches — §5.3–§5.4 (context-switch cost is mostly a cache story).
- RAM and the memory hierarchy — §5.5 and §5.9 (virtual memory sits on top of it).
- Storage devices and I/O with DMA and interrupts — §5.6–§5.7 (I/O models and `fsync` depend on them).
- Little's Law — §4.11 (used to size thread pools and connection counts).
- Amdahl's Law — §4.12 (limits on parallel speed-up).
- Powers of two and binary prefixes — §4.3 and §4.5 (page sizes and address spaces are powers of two). In this chapter, **KiB/MiB/GiB** (powers of two) are used for page sizes, stacks and address spaces; **KB/MB/GB** (powers of ten) for rough capacity estimates.

6.3 Processes and threads

6.3.1 What an operating system is for

> DEFINITION
>
> **Definition — Operating system (OS).** The software layer that manages a computer's hardware (CPU, memory, storage, network) and provides running programs with safe, shared, abstract access to it. The privileged core of the OS that runs with full hardware access is the **kernel**.

First-principles explanation

Think of a busy shared kitchen. Many cooks (programs) want the same stoves (CPU cores), counter space (memory) and fridge (disk). The kitchen manager (the kernel) gives each cook a private-looking workstation, rotates stove time fairly, and is the only person allowed to touch the gas valves (the hardware).

The OS provides three big illusions, which is how *Operating Systems: Three Easy Pieces* organizes the subject [Arpaci-Dusseau 2018]:

1. **Virtualization** — each program believes it has its own CPU and its own large, private memory.
2. **Concurrency** — many things appear to happen at once, safely.

3. **Persistence** — data survives power loss, through file systems.

Why does this exist?

Running several programs at once (**multiprogramming**) keeps the CPU busy while one program waits for I/O, but creates three problems: programs could overwrite each other's memory, one could hog the CPU forever, and each would have to drive the hardware itself. The kernel solves all three by being the single trusted intermediary. The CPU enforces this with **privilege levels**: user code runs in **user mode** and cannot execute privileged instructions or touch kernel memory; the kernel runs in **kernel mode** (on x86-64, ring 0 versus ring 3).

6.3.2 The process and its address space

DEFINITION

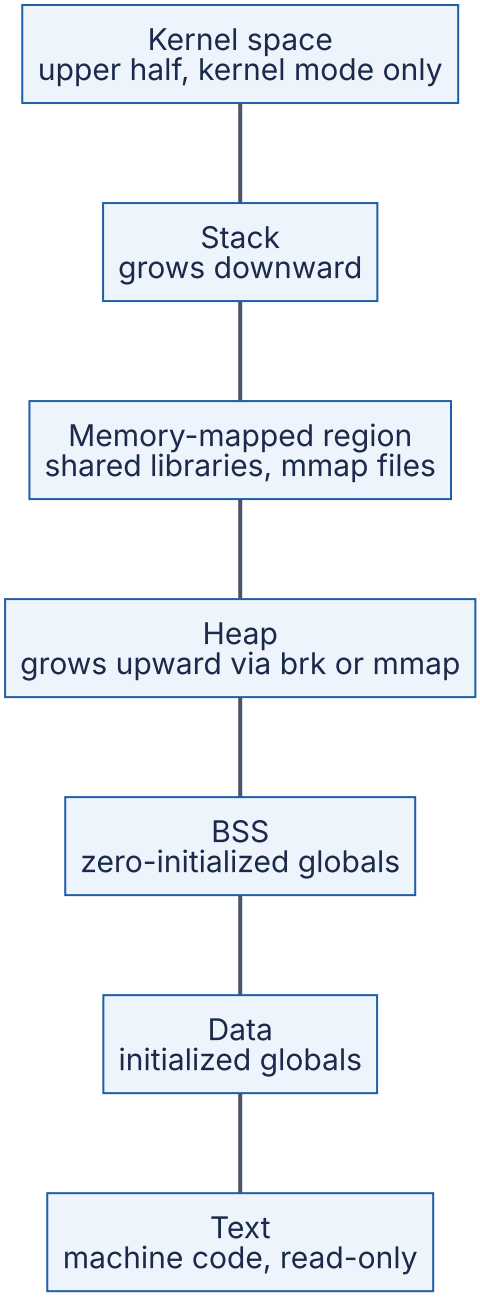
Definition — Process. A running instance of a program: its code, its private **virtual address space**, one or more threads of execution, and the OS resources it holds (open files, sockets, signal handlers, credentials).

DEFINITION

Definition — Address space. The range of memory addresses a process can use. On Linux x86-64 with 4-level page tables, user space spans 2^{47} bytes = 128 TiB of *virtual* addresses; the kernel occupies the upper half and is not accessible from user mode.

A program is a recipe on disk; a process is a cook following it, with their own bench and progress marker. Every process sees the same layout of its private memory (Figure 6.1).

Figure 6.1 — Layout of a process's virtual address space on Linux x86-64.



Reading the diagram. High addresses are at the top. **Text** holds the program's instructions, mapped read-only so a stray write crashes rather than corrupts. **Data** and **BSS** hold global variables. The **heap** (`malloc` / `new`) grows upward; the **stack** (call frames, locals; one per thread) grows downward. The **memory-mapped region** holds shared libraries and `mmap`ed files (§6.6.6). The gap between heap and stack is *unbacked*: no physical memory is used until a page is touched. The **kernel** region is mapped into every process but protected by the CPU's privilege check. Failure points: a stack overflowing into its guard page (a crash) and a heap growing until memory runs out (§6.6.4). There are no network edges or storage boundaries: this is one process's view of memory.

6.3.3 The process control block

DEFINITION

Definition — Process control block (PCB). The kernel data structure that records everything needed to stop a process and later resume it exactly where it left off. In Linux it is `struct task_struct`.

The PCB is the cook's clipboard. It typically contains [Silberschatz et al. 2018]:

- **Identity:** process ID (PID), parent PID, user and group IDs.
- **State:** new, ready, running, blocked, terminated (§6.3.4).
- **CPU context:** saved program counter, stack pointer and general-purpose registers (on Linux these are saved on the task's kernel stack and in `thread_struct`).
- **Scheduling information:** priority, nice value, accumulated CPU time, which run queue it is on.
- **Memory-management information:** a pointer to the page tables and the list of mapped regions (Linux `mm_struct`).
- **I/O and file information:** the table of open file descriptors, current directory.
- **Signal information:** pending signals and installed handlers.
- **Accounting:** CPU time used, start time, resource limits.

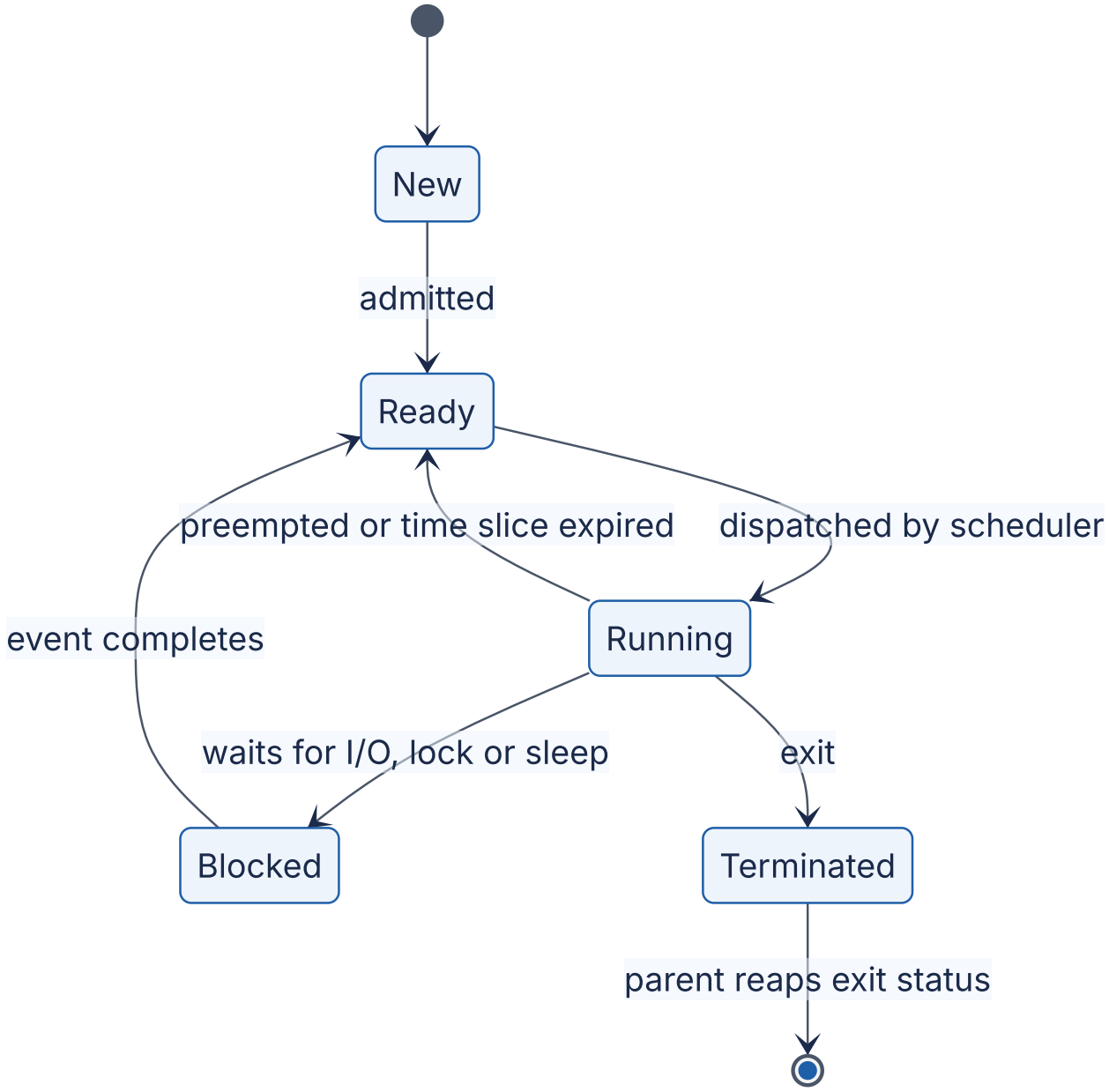
On Linux, each thread is also a `task_struct`; threads of one process point at the same `mm_struct` and file table. A `task_struct` is several kilobytes, and each task also has a 16 KiB kernel stack on x86-64 (as of Linux 6.x), so 10,000 threads cost roughly 200 MB of kernel memory before their user-space stacks are counted.

6.3.4 Process states

DEFINITION

Definition — Process state. The scheduling status of a process (or thread): whether it is running on a CPU, ready and waiting for a CPU, or blocked waiting for an event.

Figure 6.2 — Process state diagram.



Reading the diagram. A process is **New** while the kernel builds its PCB and address space, then **Ready**: runnable but waiting for a CPU. The scheduler (§6.5) **dispatches** one ready task per core to **Running**. A running process leaves the CPU in three ways: it is **preempted** (slice expired or a higher-priority task woke) and returns to Ready; it waits for something not yet available (disk, network packet, lock, timer) and becomes **Blocked**; or it **exits**. When the awaited event occurs, usually signaled by a hardware interrupt (§5.7), the process returns to Ready, *not* straight to Running. A **Terminated** process whose parent has not collected its exit status is a **zombie**. Every transition is made by the kernel.

Linux reports these states in `ps` and `top` with single letters: **R** (running *or* ready, Linux does not distinguish them), **S** (interruptible sleep, the normal blocked state), **D** (uninterruptible sleep, usually waiting on disk or network file system I/O, cannot be killed until

the I/O completes), **T** (stopped by a signal or debugger) and **Z** (zombie). On Linux, tasks in **D** count toward the **load average** along with R tasks, which is why a server with a hung NFS mount can show a load average of 200 while its CPUs are idle [Gregg 2020].

6.3.5 Threads

> DEFINITION
>
> **Definition — Thread.** An independent sequence of execution within a process, with its own program counter, registers and stack, sharing the process's address space, heap, globals and open files with the process's other threads.

If a process is a cook with a private workstation, threads are several cooks *sharing one workstation*: they pass ingredients instantly (shared memory) but can reach for the same knife at once (a race condition, §6.9).

How it works internally

1. A process starts with one thread (the **main thread**).
2. Creating a thread (`pthread_create`, `new Thread()`, `threading.Thread`) asks the kernel, on Linux via the `clone()` system call, to create a new schedulable task that *shares* the address space and file table instead of copying them.
3. The runtime allocates a new **user-space stack** for the thread (by default 8 MiB of *virtual* address space for glibc threads on Linux, taken from `ulimit -s`; 1 MiB for Java platform threads on 64-bit Linux, as of JDK 21) plus a **guard page** below it to catch overflow.
4. The kernel scheduler sees each thread as an independent task. Two threads of one process can run truly simultaneously on two cores.

Linux uses a **1:1 threading model**: every user thread is one kernel task. Go goroutines, Erlang processes and Java virtual threads multiplex many lightweight user-space threads onto a few kernel threads (an **M:N model**, §6.15). New processes are created with `fork()`, a near-copy of the caller made cheap by copy-on-write (§6.6.5), usually followed by `exec()` to load a new program.

Table 6.1 — Processes versus threads.

Aspect	Process	Thread
Address space	Private	Shared with sibling threads
Creation cost (Linux, order of magnitude)	Tens to hundreds of µs (`fork` + `exec` more)	~10 µs or less
Communication	IPC: pipes, sockets, shared memory (§6.12)	Direct reads/writes to shared memory
Isolation	Strong: a crash or wild pointer affects only itself	Weak: one bad thread can crash all
Context-switch cost	Higher: may change page tables	Lower: same page tables
Memory overhead	Full set of page tables and mappings	Stack + kernel task (~16 KiB kernel stack)
Typical use	Isolation between services, browser tabs, worker pools (e.g. PostgreSQL backends)	Parallelism inside one service (e.g. Java/Go servers)
Choose when	Faults must be contained, or code is not thread-safe	Tasks share large data and need low-latency communication

The creation-cost figures are orders of magnitude, not benchmarks; measure on your own hardware.

Real-world industry applications

INDUSTRY EXAMPLE

PostgreSQL versus MySQL. PostgreSQL forks one backend *process* per client connection, containing faults at the cost of per-connection memory, which is why large deployments put a pooler such as PgBouncer in front (§14.13). MySQL uses one *thread* per connection inside a single process: cheaper connections, shared fate. *Evidence: Primary* — [PostgreSQL Docs: Architectural Fundamentals]; [MySQL Docs: Connection Interfaces]

Failure scenarios

What goes wrong	Why	Detection	Recovery	Prevention
Zombie processes accumulate until no PIDs are left	Parent never calls `wait()` ; in containers, the app runs as PID 1 and does not reap orphans	`ps` shows many `Z` entries; `fork` fails with `EAGAIN`	Restart the parent	Reap children; run a minimal init (e.g. `tini`) as PID 1 in containers
Load average 200 but CPU idle	Many tasks in **D** state on hung storage or NFS	`ps -eo stat` shows `D` ; `/proc/<pid>/stack`	Fix storage; reboot if stuck	Timeouts on network file systems; monitor I/O latency

Common misconceptions

COMMON MISTAKE

“Threads are free.” Each costs a kernel task, kernel stack, user stack and scheduling work. Ten thousand threads is feasible; a million kernel threads on one machine is not (§6.15.6).

6.4 Context switching and its cost

DEFINITION

Definition — Context switch. The kernel operation that stops one thread running on a CPU core, saves its CPU state into its PCB, and loads the saved state of another thread so that it resumes where it left off.

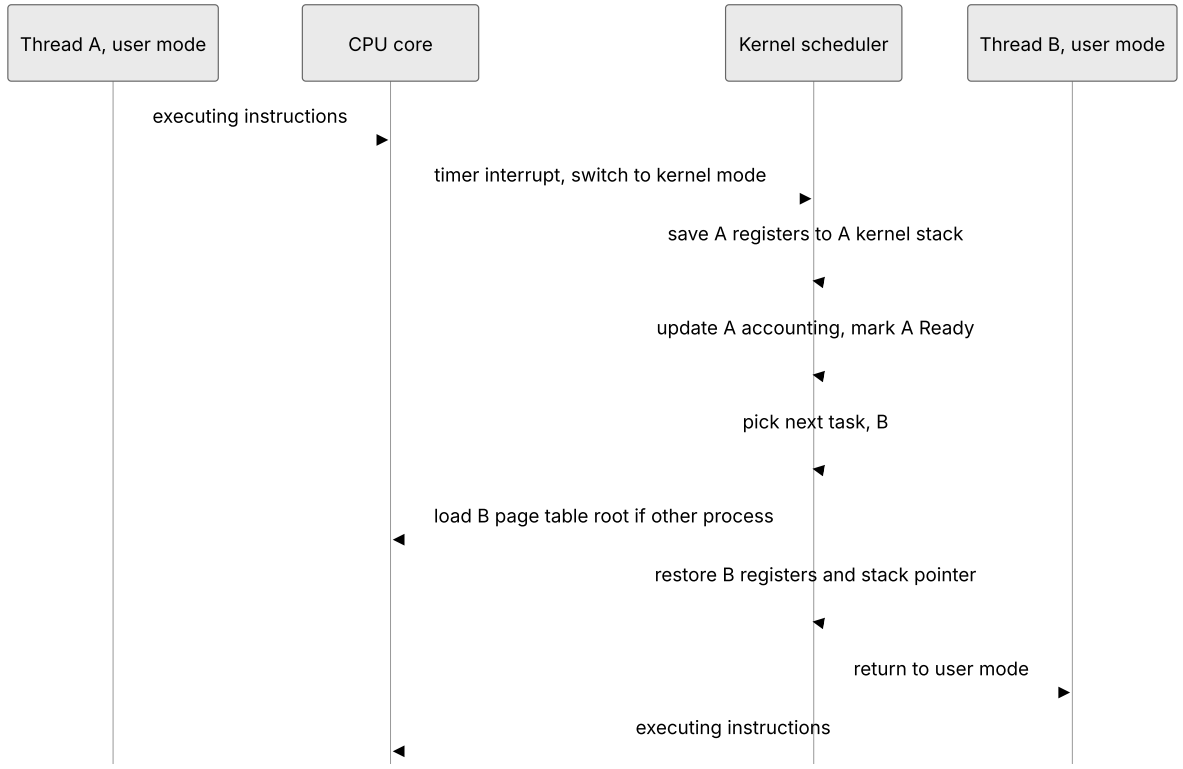
First-principles explanation

Switching between two books, noting your page is quick; reloading the plot into your head is what costs. Likewise, saving and loading registers is quick; the expensive part is that the new thread finds the CPU’s caches full of the *old* thread’s data. Switching exists because a core runs one thread at a time: pausing a thread that waits for a 10 ms disk read lets the core do useful work meanwhile.

How it works internally

A switch is triggered in one of two ways. A **voluntary** switch happens when the running thread blocks (reads from an empty socket, waits for a lock, sleeps). An **involuntary** switch happens when the kernel preempts the thread, usually because a periodic **timer interrupt** found its time slice used up or a higher-priority thread became ready.

Figure 6.3 — A context switch between two threads triggered by a timer interrupt.



Reading the diagram. The dotted arrow is asynchronous: the timer interrupt arrives regardless of what A is doing and forces the CPU into kernel mode, saving A's instruction pointer and stack pointer. Solid arrows are synchronous steps on the same core. The kernel saves A's remaining registers, returns A to the Ready queue and asks the scheduling policy (§6.5) for the next task. If B belongs to a *different process*, the kernel loads B's page-table root (on x86-64, the `CR3` register); **process-context identifiers (PCIDs)** let TLB entries of different processes coexist, avoiding a full TLB flush. B's registers are restored and the CPU returns to user mode. No storage or network is involved; the failure point is performance: too-frequent switches leave the core in the kernel and refilling caches.

Mathematical foundations

There are two costs.

- **Direct cost** — the kernel instructions above: entering the kernel, saving and restoring registers, running the scheduler, switching page tables. On modern x86-64 Linux this is roughly **1–5 µs** per switch (as of the 2020s; it varies with CPU, kernel version and security mitigations such as those for Spectre and Meltdown).
- **Indirect cost** — after the switch the incoming thread's data and instructions are no longer in the L1/L2 caches (§5.4) and its address translations are no longer in the TLB (§6.6.3), so it runs slowly until they are reloaded. This can exceed the direct cost by one or two orders of magnitude for threads with large working sets [Li et al. 2007].

FORMULA

Context-switch overhead fraction

$$f = \frac{C}{Q + C}$$

Symbol	Meaning	Units
f	Fraction of CPU time lost to switching	dimensionless (0–1)
C	Effective cost of one switch, direct + indirect	µs
Q	Average time a thread runs between switches	µs

FORMULA

Cache refill estimate (indirect cost)

$$C_{\text{refill}} \approx \frac{W}{L} \times \frac{t_{\text{miss}}}{M}$$

Symbol	Meaning	Units
C_{refill}	Time spent reloading the working set after a switch	ns
W	Working set that was evicted	bytes
L	Cache line size (64 on x86-64 and most ARM servers)	bytes/line
t_{miss}	Latency of one miss to the next memory level	ns
M	Memory-level parallelism: misses overlapped in flight	dimensionless (≥ 1)

WORKED EXAMPLE

Worked example — When does switching hurt?

Case 1: CPU-bound batch job. A thread runs a full 4 ms slice ($Q = 4000$ µs) and the direct switch cost is $C = 3$ µs. $f = 3/(4000 + 3) = 0.00075$, i.e. **0.075 %** of CPU time. Negligible.

Case 2: the same switch, but the thread's 1 MiB working set was evicted to DRAM. $W = 1,048,576$ bytes, $L = 64$ bytes, so $W/L = 16,384$ lines. Take $t_{\text{miss}} = 90$ ns to DRAM and $M = 8$ overlapping misses: $C_{\text{refill}} = 16,384 \times 90/8 = 184,320$ ns $\approx$ **184 µs**, about 60× the direct cost.

Case 3: a chatty server. Threads handle tiny requests and block on the network every 20 µs ($Q = 20$ µs). With a modest 10 µs effective switch cost (direct plus a small refill), $f = 10/(20 + 10) = 0.33$: **a third of the CPU goes to switching**.

Interpretation. Context switches are cheap when threads run for milliseconds and expensive when they run for microseconds. **Engineering implication:** for high-rate, short-request workloads, prefer designs that do many small units of work per switch: event loops (§6.13), batching (§23.8), fewer threads than cores-times-a-small-factor for CPU-bound pools, and pinning hot threads to cores.

How to observe it

- `vmstat 1` — the `cs` column is context switches per second for the whole machine. Tens of thousands per second per core is normal for a busy server; hundreds of thousands per core deserves investigation.
- `pidstat -w 1` or `/proc/<pid>/status` — per-task voluntary switches (blocking: lock contention, small I/Os) versus involuntary ones (preemption: CPU contention). If thread migrations between NUMA sockets are frequent, pin threads (§5.10).
- `perf sched` and `perf stat -e context-switches,cpu-migrations` for detailed analysis [Gregg 2020].

COMMON MISTAKE

"A context switch costs a few microseconds, so it never matters." The few microseconds is only the direct cost. The cache and TLB refill afterwards can cost far more, and it appears in your profile as "slow application code", not as kernel time.

6.5 CPU scheduling

DEFINITION

Definition — CPU scheduler. The part of the kernel that decides which ready thread runs next on each core and for how long. The rule it follows is the **scheduling policy**; the mechanism that performs the switch is the dispatcher (§6.4).

First-principles explanation

A doctor with a waiting room is a scheduler. Strict arrival order leaves a 2-minute renewal stuck behind a 60-minute consultation; quickest-first minimizes average waiting but may leave the long case waiting all day; 10 minutes each in rotation means nobody waits long to be *seen*, but long cases finish later. Every CPU scheduling policy is one of these trade-offs. It optimizes some mix of:

- **Turnaround time** — arrival to completion; matters for batch work.
- **Response time** — arrival to first run; matters for interactive work and request handlers.
- **Throughput**, **fairness** (each thread, user or container gets its share) and **utilization**.

These conflict: shortest-job-first minimizes turnaround but is unfair to long jobs; rapid rotation minimizes response time but adds switching overhead.

6.5.1 First-come, first-served (FCFS)

Run jobs to completion in arrival order. Simple, but suffers the **convoy effect**: short jobs queue behind a long one. **Shortest job first (SJF)** and its preemptive form **STCF** are optimal for average turnaround but need job lengths in advance, which a general-purpose OS does not have [Arpaci-Dusseau 2018].

6.5.2 Round robin (RR)

Run each ready thread for a fixed **time slice** (**quantum**), then move it to the back of the queue. RR gives good response time and cannot starve anyone, but too short a slice wastes time switching (§6.4) and too long a slice degenerates into FCFS. General-purpose systems use roughly 1–10 ms.

FORMULA

Average turnaround and response time

$$\bar{T}_{\text{turn}} = \frac{1}{n}\sum_{i=1}^{n}\left(T^{i}_{\text{completion}} - T^{i}_{\text{arrival}}\right) \qquad \bar{T}_{\text{resp}} = \frac{1}{n}\sum_{i=1}^{n}\left(T^{i}_{\text{first run}} - T^{i}_{\text{arrival}}\right)$$

Symbol	Meaning	Units
n	Number of jobs	jobs
T^{i}_{arrival}	Time job i became ready	ms
$T^{i}_{\text{first run}}$	Time job i first got a CPU	ms
$T^{i}_{\text{completion}}$	Time job i finished	ms

WORKED EXAMPLE

Worked example — FCFS vs SJF vs RR on one core. Three jobs arrive at $t = 0$: A needs 100 ms of CPU, B needs 10 ms, C needs 10 ms. Ignore switching cost.

FCFS (order A, B, C). Completions: A = 100, B = 110, C = 120. Average turnaround = (100 + 110 + 120)/3 = **110 ms**. First runs: A = 0, B = 100, C = 110; average response = **70 ms**.

SJF (order B, C, A). Completions: B = 10, C = 20, A = 120. Average turnaround = (10 + 20 + 120)/3 = **50 ms**. Average response = (0 + 10 + 20)/3 = **10 ms**.

RR, 10 ms slice. A runs 0–10, B 10–20 (done), C 20–30 (done), then A alone 30–120. Average turnaround = (120 + 20 + 30)/3 ≈ **56.7 ms**. Average response = (0 + 10 + 20)/3 = **10 ms**.

Interpretation. RR matches SJF's response time and comes close to its turnaround *without knowing job lengths*. FCFS is more than twice as bad on turnaround because two short jobs waited behind a long one. **Engineering implication:** the same convoy effect appears in application queues: one slow request type in a shared FIFO worker pool inflates latency for every fast request. Separate queues or pools per request class (bulkheads, §22.14) apply this lesson at the service level.

6.5.3 Priority scheduling

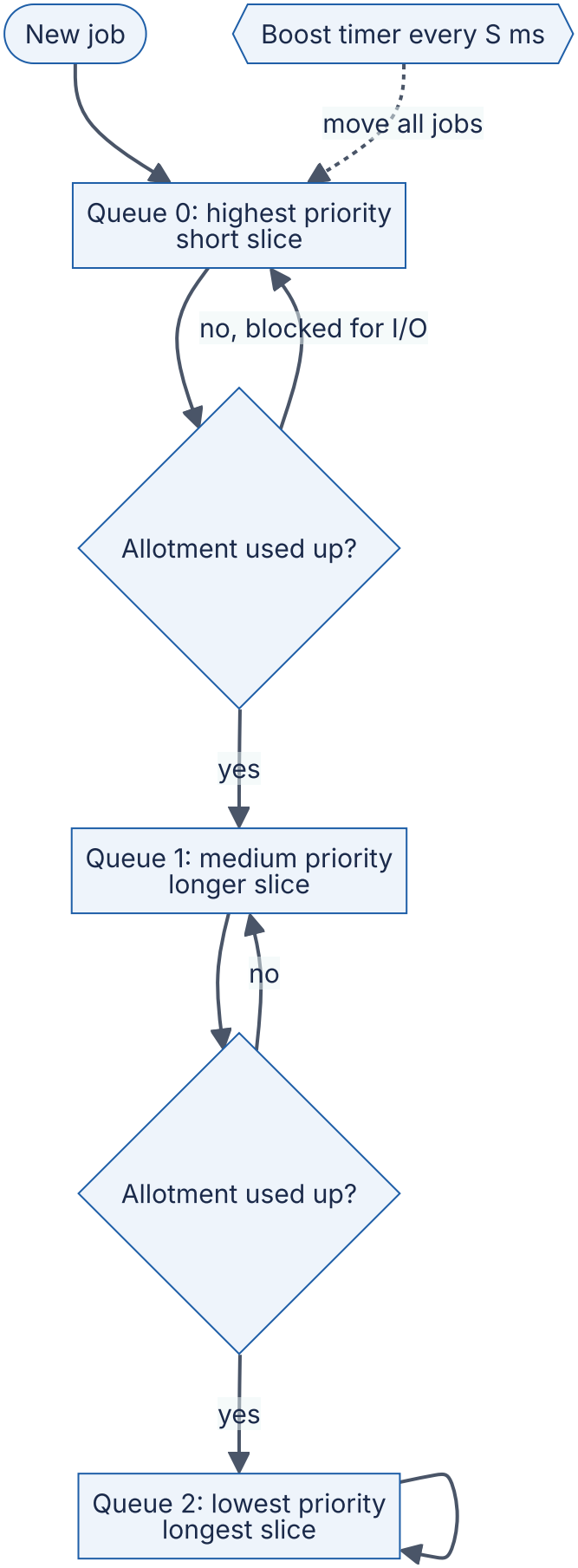
Each thread has a priority; the highest-priority ready thread runs. This lets important work (an audio driver, a heartbeat thread) preempt bulk work, but low-priority threads can **starve**: wait forever because higher-priority work keeps arriving. The classic remedy is **aging**: gradually raise the priority of threads that have waited a long time. Priorities also create **priority inversion** with locks (§6.11.5).

6.5.4 Multi-level feedback queue (MLFQ)

MLFQ, first described for the CTSS time-sharing system in 1962 by Corbató and colleagues, learns a thread’s behavior instead of requiring it to be declared. It keeps several RR queues at different priority levels and applies five rules, as formulated in OSTEP [Arpaci-Dusseau 2018]:

1. If priority(A) > priority(B), A runs.
2. If priority(A) = priority(B), A and B run round robin.
3. A new job starts at the highest priority.
4. Once a job uses up its time allotment at a level, however many times it gave up the CPU, its priority drops one level.
5. After some period S, move every job back to the top queue (a **priority boost**).

Figure 6.4 — Multi-level feedback queue decisions.


New job
Boost timer every S ms
move all jobs
Queue 0: highest priority short slice
no, blocked for I/O
Allotment used up?
yes
Queue 1: medium priority longer slice
no
Allotment used up?
yes
Queue 2: lowest priority longest slice

Reading the diagram. Rectangles are run queues; the highest non-empty queue is served (rule 1), round robin within it (rule 2). New jobs enter Queue 0 (rule 3). Interactive jobs block for I/O before exhausting their allotment and stay high, so they respond fast; CPU-bound jobs sink to lower queues with longer slices (rule 4). The dotted arrow is the asynchronous priority boost (rule 5), which prevents starvation of the bottom queue. Counting *total* allotment used stops a program gaming the scheduler by yielding just before its slice expires. Variants of MLFQ underlie the schedulers of BSD derivatives, Solaris and Windows [Arpaci-Dusseau 2018].

6.5.5 Linux: CFS and EEVDF

Linux's normal scheduling class (`SCHED_NORMAL`, also called `SCHED_OTHER`) aims at **proportional-share fairness**: each runnable thread should get a share of CPU proportional to its **weight**, which is derived from its **nice value** (−20 = highest weight, +19 = lowest).

CFS (Completely Fair Scheduler) was the default from Linux 2.6.23 (2007) until Linux 6.6 (2023). It tracks for each thread a **virtual runtime** (`vruntime`): actual CPU time scaled inversely by weight. It keeps runnable threads in a red-black tree ordered by `vruntime` and always runs the thread that has had the least weighted CPU, so picking the next thread costs $O(\log n)$ [Linux Docs: CFS].

EEVDF (Earliest Eligible Virtual Deadline First) replaced CFS's selection logic in Linux 6.6 (October 2023), based on an algorithm by Stoica and Abdel-Wahab [Stoica & Abdel-Wahab 1995]. Each thread has a **lag**: CPU time owed minus time received. Threads with non-negative lag are **eligible**; among them EEVDF runs the one with the earliest **virtual deadline** (roughly eligible time plus requested slice divided by weight). Fairness is kept, and latency-sensitive threads that request shorter slices get a principled advantage instead of hand-tuned heuristics [Linux Docs: EEVDF; Corbet 2023]. The default base slice is about 0.75 ms (as of Linux 6.6–6.x, scaled up on many-CPU machines).

Linux also has **real-time** classes (`SCHED_FIFO`, `SCHED_RR`, priorities 1–99) that always preempt normal threads, `SCHED_DEADLINE` for tasks with explicit runtime and period guarantees, and, from Linux 6.12 (2024), **sched_ext**, which lets a scheduling policy be loaded as a BPF program [Linux man-pages: sched(7)].

FORMULA

Proportional CPU share

$$s_i = \frac{w_i}{\sum_{j \in R} w_j}$$

Symbol	Meaning	Units
s_i	Fraction of one core's time thread i receives when all threads in R compete	dimensionless (0–1)
w_i	Weight of thread i, from its nice value (nice 0 = 1024; each nice step changes weight by about 1.25×)	dimensionless
R	Set of runnable threads competing on that core	—

WORKED EXAMPLE

Worked example — Nice values. Two CPU-bound threads share one core. Thread X has nice 0 (weight 1024); thread Y has nice 5 (weight 335, from the kernel's weight table). $s_X = 1024/(1024 + 335) = 1024/1359 \approx 0.753$; $s_Y \approx 0.247$. **Interpretation.** Five nice levels give roughly a 3:1 split, consistent with the rule of thumb that each nice step shifts about 10 % of CPU between two competing threads. **Engineering implication:** to keep a log-compaction or backup job from hurting request latency on the same machine, running it at nice 10–19 is a cheap first step; cgroup CPU weights do the same for whole containers.

Table 6.2 — CPU scheduling policies compared.

Policy	How it works	Advantages	Disadvantages	Choose when
FCFS	Run in arrival order to completion	Simple, no switching	Convoy effect; poor response time	Batch queues of similar-sized jobs
SJF / STCF	Shortest (remaining) job first	Optimal average turnaround	Needs job lengths; starves long jobs	Job sizes known (some batch systems)
Round robin	Fixed slice, rotate	Good response time; no starvation	Switching overhead; poor turnaround for equal jobs	Time-sharing with similar priorities
Priority	Highest priority runs	Important work goes first	Starvation; priority inversion	Real-time and control systems
MLFQ	Priority queues; demote CPU hogs; periodic boost	Learns behavior; good interactivity	Many tuning knobs	General-purpose desktops and servers
CFS / EEVDF (Linux)	Weighted fair share by virtual runtime; EEVDF adds lag and virtual deadlines	Fair, scalable, tunable by nice and cgroups	Fairness is not latency; complex	Default on Linux servers (as of 2026)

Real-world industry applications

INDUSTRY EXAMPLE

CPU limits in containers cause throttling. Kubernetes CPU limits use CFS **bandwidth control**: a cgroup gets a CPU-time quota per period (default 100 ms, `cpu.max` in cgroup v2). A service limited to 1 CPU with 4 busy threads uses its quota in 25 ms and is **throttled** for the remaining 75 ms, delaying in-flight requests even on an idle node. `nr_throttled` in `cpu.stat` reveals it. Mitigations: raise or remove limits for latency-critical services, or size thread pools to the limit rather than the host's cores (§42.13). *Evidence: Primary* — [Linux Docs: CFS Bandwidth Control]

Common misconceptions

COMMON MISTAKE

"Nice -20 makes my latency-sensitive service fast." Nice changes the *share* of CPU under contention; on a machine that is not CPU-saturated it has almost no effect. For latency, keep runnable threads below core count and avoid CPU throttling.

6.6 Virtual memory, paging and the page cache

6.6.1 The idea of virtual memory

DEFINITION

Definition — Virtual memory. A mechanism in which every address a program uses is a **virtual address** that the hardware's **memory-management unit (MMU)** translates, using tables maintained by the kernel, into a **physical address** in RAM, or into a fault if no valid mapping exists.

First-principles explanation

A hotel guest is told "you are in room 12"; the front desk's private sheet for that guest says room 12 is physically room 407. Two guests can both have "room 12", luggage can be moved without telling the guest, and a guest asking for a room not on their sheet is stopped at the desk. Virtual memory gives each process exactly this, and so delivers **isolation** (separate tables), **convenience** (the same layout for everyone, Figure 6.1) and **flexibility** (memory allocated lazily, shared, mapped to files or moved to disk).

6.6.2 Paging and page tables

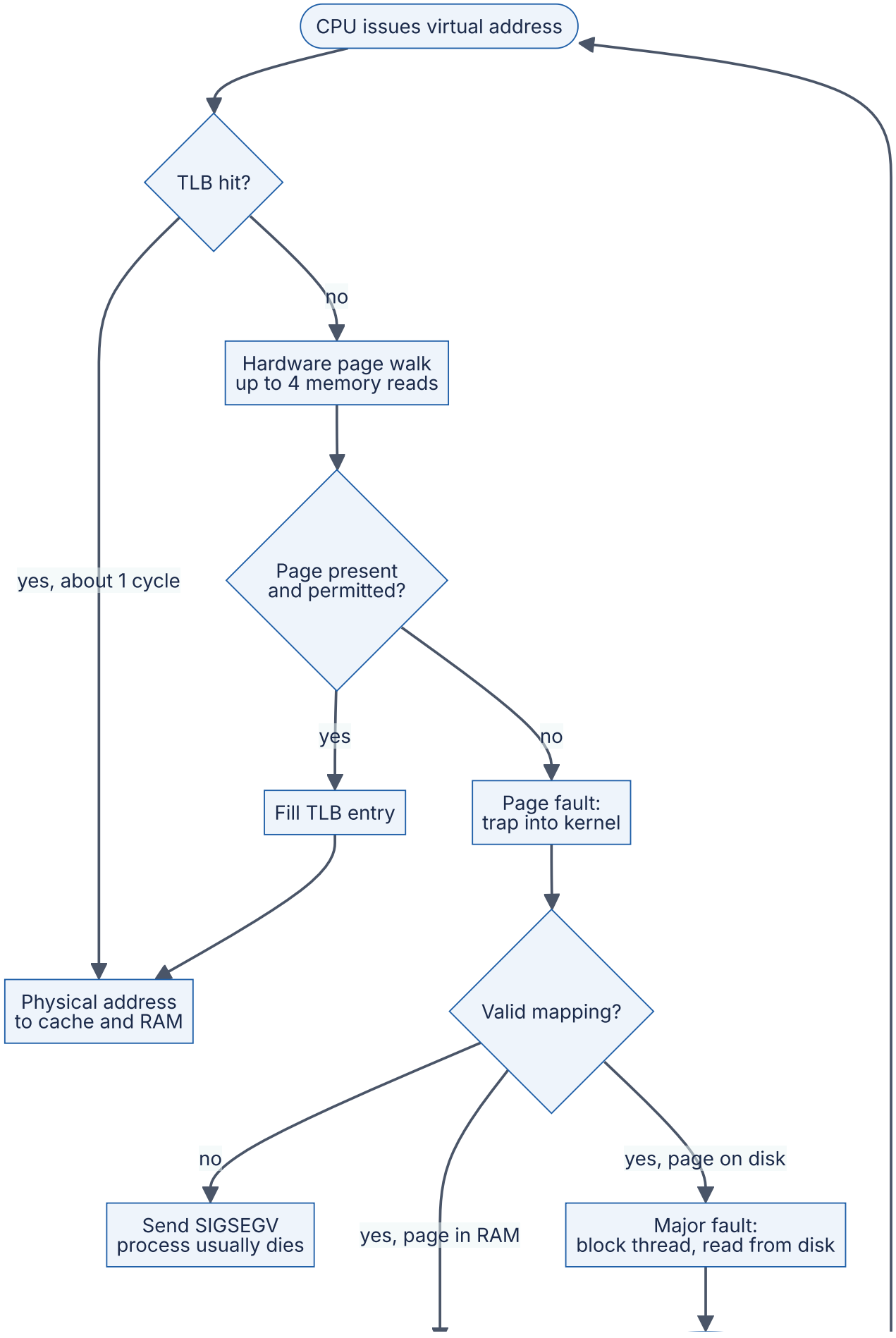
Memory is divided into fixed-size blocks: **pages** in virtual memory and **frames** in physical memory, typically **4 KiB** on x86-64 and most ARM servers. A virtual address is split into a **virtual page number** and an **offset** within the page. The kernel keeps a **page table** per process that maps virtual page numbers to physical frame numbers, plus permission bits: *present*, *writable*, *user-accessible*, *no-execute*, and *accessed*/*dirty* flags that the hardware sets so the kernel knows which pages were used or modified.

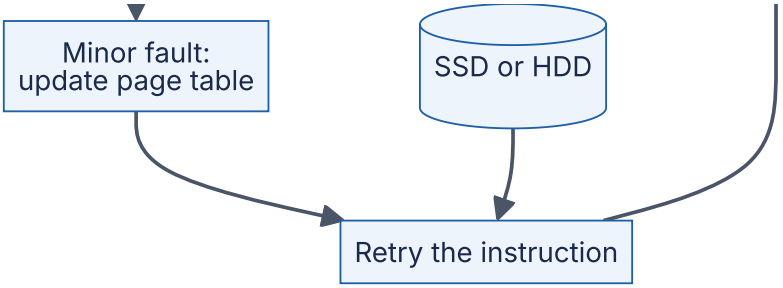
A flat table for 128 TiB of virtual space would need 2^{35} entries, far too large. So the table is a **multi-level tree**: on x86-64 with 4-level paging, the 48-bit address is split into four 9-bit indices (one per level) and a 12-bit offset (9 + 9 + 9 + 9 + 12 = 48). Only the branches actually used are allocated. Newer CPUs support 5-level paging (57-bit addresses) for very large-memory machines.

6.6.3 The TLB

Walking a four-level tree means up to four extra memory reads for every load or store, which would make every memory access several times slower. The **translation lookaside buffer (TLB)** is a small cache inside the CPU core that holds recent virtual-to-physical translations. Typical server cores have tens of entries in the first-level TLB and roughly 1,000–3,000 entries in a second-level TLB (as of 2020s x86-64 and ARM designs; consult your CPU's documentation).

Figure 6.5 — Translating a virtual address: TLB, page walk and page fault.


CPU issues virtual address
TLB hit?
yes, about 1 cycle
no
Hardware page walk
up to 4 memory reads
Page present
and permitted?
yes
no
Fill TLB entry
Page fault:
trap into kernel
Physical address
to cache and RAM
Valid mapping?
no
Send SIGSEGV
process usually dies
yes, page in RAM
yes, page on disk
Major fault:
block thread, read from disk



Reading the diagram. Every load, store and instruction fetch starts at the top. A **TLB hit** makes translation essentially free. On a **miss**, the MMU hardware walks the page table; if the entry is present and permitted, the TLB is filled. Otherwise the CPU raises a **page fault**, a synchronous trap into the kernel. An invalid address (null pointer, write to read-only code) gets **SIGSEGV**. A valid page already in RAM (a shared library page, a fresh zero page on first touch) is a cheap **minor fault**: the kernel just fills in the entry. A page that must come from storage (a file page or a swapped-out page) is a **major fault**: the thread blocks while the disk is read, the storage boundary in this figure. Then the instruction is retried. The failure point that matters is the major-fault path, which turns a 100 ns access into a 100 µs (SSD) to 10 ms (HDD) wait.

FORMULA

TLB reach

$$R_{\text{TLB}} = E \times P$$

Symbol	Meaning	Units
R_{TLB}	Amount of memory addressable without a TLB miss	bytes
E	Number of TLB entries	entries
P	Page size per entry	bytes

FORMULA

Effective memory access time with page faults

$$\mathrm{EAT} = (1 - p) \times t_{\mathrm{mem}} + p \times t_{\mathrm{fault}}$$

Symbol	Meaning	Units
EAT	Average time per memory access	ns
p	Probability that an access causes a major page fault	dimensionless (0–1)
t_{mem}	Time for an access with no fault	ns
t_{fault}	Time to service a major fault, including the disk read	ns

WORKED EXAMPLE

Worked example — Why huge pages and why swapping hurts.

TLB reach. A core with $E = 1,536$ second-level entries and 4 KiB pages has $R = 1,536 \times 4 = 6,144$ KiB **= 6 MiB**. A database with a 64 GiB in-memory buffer pool touching memory randomly will miss in the TLB on most accesses. With 2 MiB **huge pages**, the same entries cover $1,536 \times 2$ MiB **= 3 GiB**, 512× more. This is why databases and JVMs often use explicit huge pages.

Page faults. Let $t_{\mathrm{mem}} = 100$ ns and a major fault from an NVMe SSD take $t_{\mathrm{fault}} = 100$ µs = 100,000 ns. If one access in 10,000 faults ($p = 10^{-4}$): EAT $= 0.9999 \times 100 + 0.0001 \times 100,000 = 99.99 + 10 \approx$ **110 ns**, a 10 % slowdown. To keep the slowdown under 10 % with a 10 ms HDD ($t_{\mathrm{fault}} = 10^7$ ns), we need $p \times 10^7 < 10$, i.e. $p < 10^{-6}$: **fewer than one fault per million accesses**.

Interpretation. Even rare major faults dominate the average. **Engineering implication:** a latency-sensitive service must keep its working set in RAM. When the working set exceeds RAM and the system swaps heavily (**thrashing**), throughput collapses by orders of magnitude rather than degrading gracefully.

6.6.4 Demand paging and overcommit

Linux allocates physical memory lazily: `malloc(1 GiB)` reserves only virtual addresses, and frames are assigned on first touch. It also **overcommits** by default, promising more virtual memory than exists. When the bet fails, the **OOM killer** kills a process, typically the largest, with SIGKILL; a container exceeding its cgroup limit (`memory.max`) is killed the same way (§42.3). Watch **resident set size** (RSS, physical pages in use), not virtual size.

6.6.5 Copy-on-write

When `fork()` creates a child, the kernel shares all the parent's pages with the child and marks them read-only in both. The first write by either side triggers a fault, and only then is that one page copied. This makes `fork` fast and allows a process to take a consistent **snapshot** of its memory cheaply.

INDUSTRY EXAMPLE

Redis background snapshots. Redis calls `fork()` to snapshot; the child writes its frozen copy-on-write view to disk while the parent keeps serving writes. Overhead is proportional to pages modified meanwhile, up to twice the dataset for write-heavy instances. Redis recommends disabling **Transparent Huge Pages**, because then one small write copies a 2 MiB page instead of 4 KiB, causing latency spikes. *Evidence: Primary* — [Redis Docs: Persistence]; [Redis Docs: Latency diagnosis]

6.6.6 The page cache and mmap

DEFINITION

Definition — Page cache. The kernel's in-RAM cache of file contents, organized in pages. Reads are served from it when possible; writes normally modify pages in it (marking them **dirty**) and return immediately, with the kernel writing them back to storage later.

Linux uses almost all otherwise-free RAM as page cache, so `free` reports little "free" memory on a healthy server; watch "available" instead. By default dirty pages are written back by kernel threads after they have been dirty for about 30 s, checked every 5 s (`vm.dirty_expire_centisecs = 3000`, `vm.dirty_writeback_centisecs = 500`, as of Linux 6.x) or sooner under memory pressure. The consequence for durability is in §6.14.

`mmap` maps a file directly into a process's address space: reading the memory reads the file through the page cache, with no `read()` system call and no copy into a user buffer. It is elegant for read-mostly data (shared libraries are loaded this way), but it hides I/O inside ordinary memory accesses: any pointer dereference may block on a major fault, I/O errors arrive as a `SIGBUS` signal instead of an error code, and the kernel, not the application, decides what to evict. Crotty, Leis and Pavlo argue that for these reasons database systems should usually manage their own buffer pool instead of relying on `mmap` [Crotty et al. 2022] (see §13.4).

INDUSTRY EXAMPLE

Apache Kafka relies on the page cache. Kafka brokers append to log files and let the OS page cache hold recent data instead of caching messages in the JVM, avoiding double caching, surviving process restarts and keeping heaps small. Consumers of recent data are served from the page cache via `sendfile`, without copying through user space (§31.13). *Evidence: Primary* — [Apache Kafka Docs: Design, Persistence]

6.6.7 Swapping

Swapping moves rarely used **anonymous** pages (heap and stack, with no file behind them) to disk so their frames can be reused; clean file-backed pages are simply dropped and re-read later. Modest swap moves idle memory out of the way; swapping a working set is thrashing. Many latency-sensitive deployments disable swap or set `vm.swappiness` low.

Failure scenarios

What goes wrong	Why	Detection	Recovery	Prevention
Process killed with exit code 137, no error log	OOM killer or cgroup limit sent SIGKILL	`dmesg` "Out of memory: Killed process"; container `OOMKilled` status	Restart with more memory	Set memory limits above peak RSS; bound caches and queues
Latency rises 100× under load	Working set exceeds RAM; major faults or swap thrashing	`vmstat` `si` / `so` columns; major-fault counters (`sar -B`)	Add RAM or shed load	Capacity plan working set; avoid swap for latency tiers
"Free memory is 1 %" alert fires constantly	Page cache is counted as used	Compare with "available"	None needed	Alert on available memory and major faults instead

6.7 System calls and the kernel/user boundary

DEFINITION

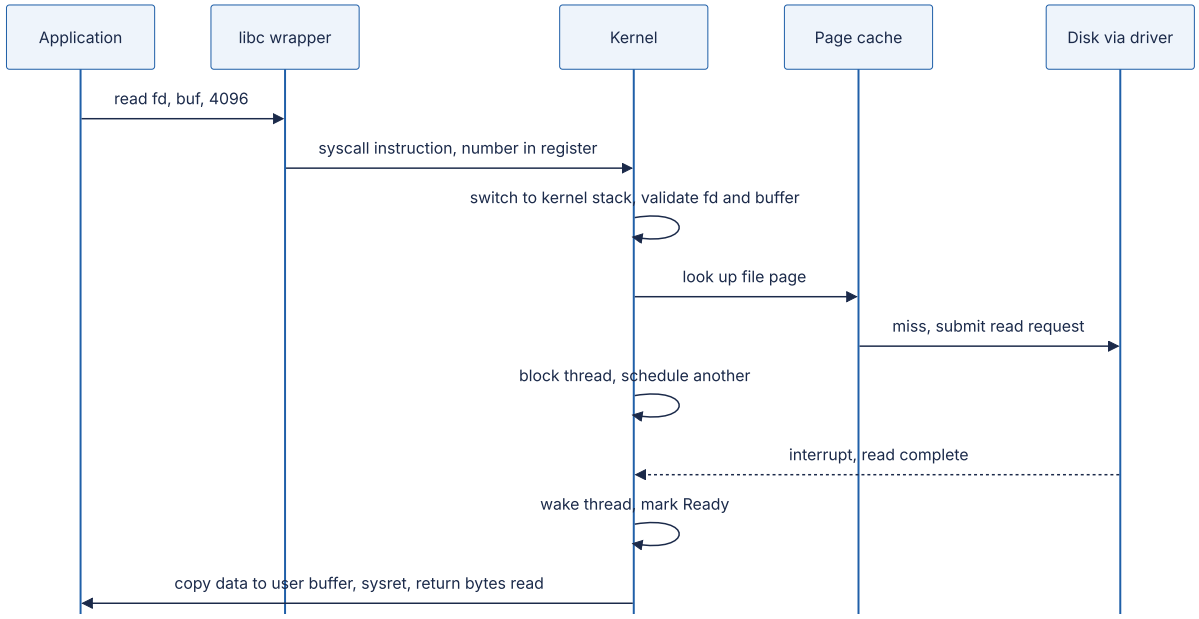
Definition — System call (syscall). A controlled entry from user mode into the kernel to request a privileged service, such as reading a file, sending on a socket, allocating memory or creating a process. Linux x86-64 has a few hundred system calls, each identified by a number [Linux man-pages: syscalls(2)].

First-principles explanation

A bank customer cannot walk into the vault; they hand a slip through one narrow window where every request is checked. A system call is that window, and library functions such as `printf` or Python's `open()` are wrappers that fill in the slip.

How it works internally

Figure 6.6 — A `read()` system call that must wait for the disk.



Reading the diagram. The library wrapper puts the system-call number and arguments in registers and executes the `syscall` instruction, entering the kernel at a fixed entry point, the only way in. The kernel *validates* everything, since user code can pass any value: is the descriptor open, does the buffer lie in the caller's memory? On a page-cache hit (§6.6.6) it copies the data and returns within about a microsecond. On a miss it submits a request across the storage boundary and the thread **blocks** (a voluntary context switch). The dotted arrow is the device's asynchronous completion interrupt; the thread becomes Ready, runs again, and the kernel copies the data out and returns. Failure points: bad arguments return `EBADF` or `EFAULT`, device errors `EIO`, and a signal can interrupt a slow call with `EINTR`, which callers must retry.

What it costs

A trivial system call costs on the order of **100 ns to a few hundred ns** on modern x86-64 Linux, against about 1–5 ns for a function call (as of the 2020s). The cost rose in 2018 when **kernel page-table isolation (KPTI)**, the Meltdown mitigation, began switching page tables on every kernel entry on affected CPUs. A rough Python measurement on this

chapter's test VM found `os.getppid()` at about 190 ns per call against about 50 ns for a built-in function call.

Three techniques reduce syscall cost:

1. **Batching** — do more per call: `readv`/`writev` for scatter/gather I/O, `sendmmsg` for many UDP datagrams, larger buffers.
2. **vDSO (virtual dynamic shared object)** — the kernel maps a small read-only library into every process so that some calls, notably `clock_gettime` and `gettimeofday`, run entirely in user mode by reading kernel-maintained data.
3. **Shared-memory rings** — `io_uring` (§6.13.5) lets the application queue many I/O requests in memory shared with the kernel, submitting them with one system call or, in polled mode, none.

To observe system calls, `strace -c -p <pid>` counts and times them (with heavy overhead); `perf trace` and eBPF tools such as `syscount` are cheaper [Gregg 2020].

> COMMON MISTAKE
>
> **"Calling `write()` means my data is on disk."** `write()` returns when the data is in the page cache. It is durable only after a successful `fsync()` (§6.14.3).

6.8 Concurrency versus parallelism

> DEFINITION
>
> **Definition — Concurrency and parallelism. Concurrency** is the *structuring* of a program as multiple tasks whose lifetimes overlap, so that progress on one does not wait for the others to finish. **Parallelism** is the *simultaneous execution* of multiple computations on multiple hardware units (cores, CPUs, GPUs).

One cook juggling three dishes, stirring one while another simmers, is concurrent but not parallel. Three cooks each making one dish are parallel. Rob Pike's summary: concurrency is about *dealing with* many things at once; parallelism is about *doing* many things at once [Pike 2012].

I/O-bound work (a web server waiting on databases) needs concurrency: thousands of requests in flight, mostly waiting, which one core can serve with an event loop (§6.13). **CPU-bound** work (image encoding) needs parallelism, with speed-up limited by the serial fraction (Amdahl's Law, §4.12). Concurrency alone gives no CPU speed-up.

CPython's **global interpreter lock (GIL)** is the standard illustration: Python threads are concurrent, and overlap well while waiting on I/O, but only one executes Python bytecode

at a time, so CPU-bound Python code gains nothing from threads. Use processes (`multiprocessing`), native extensions that release the GIL, or the optional free-threaded build of CPython (experimental in 3.13, officially supported but not the default in 3.14, as of 2026) for CPU parallelism.

6.9 Race conditions and critical sections

DEFINITION

Definition — Race condition. A bug in which the result of a program depends on the relative timing or interleaving of concurrent operations on shared state. A **data race** is the specific case where two threads access the same memory location concurrently, at least one access is a write, and nothing orders them.

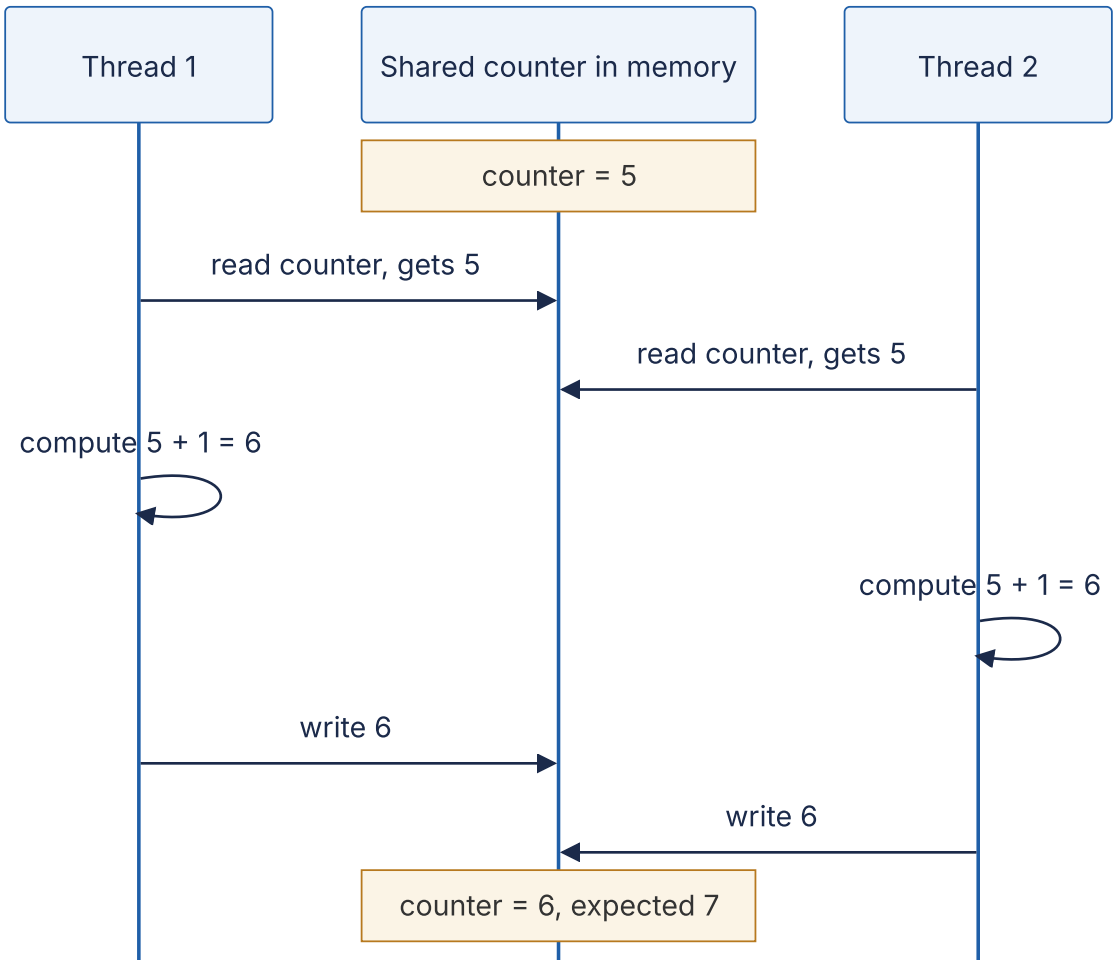
DEFINITION

Definition — Critical section. A region of code that accesses shared state and must not be executed by more than one thread at a time. Guaranteeing this is **mutual exclusion**.

First-principles explanation

Two people deposit $50 at the same moment into a shared account holding $100, at different cash machines. Each machine reads $100, adds $50 and writes $150; the correct balance is $200. Neither machine erred alone: the bug is the *interleaving* of read, modify and write, the **lost update**.

Figure 6.7 — Two threads lose an update to a shared counter.



Reading the diagram. Each arrow is a synchronous step, but the two threads' steps interleave in an order chosen by the scheduler and hardware. Both read before either writes, so one increment is lost. The critical section is *read, compute, write*; making it indivisible, with a lock or one atomic instruction, removes the bug. The same failure occurs across machines when two service instances read-modify-write a database row (§11.13, §26.13).

Code example

The Python program below makes the race visible. `counter += 1` in CPython is not atomic, but the GIL makes the bad interleaving rare, so `time.sleep(0)` deliberately yields the CPU between the read and the write, exactly where a preemption could occur.

```
"""Lost updates: a read-modify-write race, then the fix with a mutex."""
import threading
import time

N_THREADS = 4
N_INCREMENTS = 10_000

counter = 0
lock = threading.Lock()


def unsafe_increment() -> None:
    global counter
    for _ in range(N_INCREMENTS):
        current = counter      # 1. read
        time.sleep(0)          # widen the window: yield the CPU (see note)
        counter = current + 1  # 2. modify + 3. write


def safe_increment() -> None:
    global counter
    for _ in range(N_INCREMENTS):
        with lock:             # critical section starts
            current = counter
            time.sleep(0)
            counter = current + 1
                               # critical section ends (lock released)


def run(worker) -> int:
    global counter
    counter = 0
    threads = [threading.Thread(target=worker) for _ in range(N_THREADS)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    return counter


if __name__ == "__main__":
    expected = N_THREADS * N_INCREMENTS
    print(f"expected       = {expected}")
    print(f"without a lock = {run(unsafe_increment)}")
    print(f"with a lock    = {run(safe_increment)}")
```

A run on CPython 3.13 printed `expected = 40000`, `without a lock = 10000` and `with a lock = 40000`; the unsafe count varies between runs.

What to notice:

- The unsafe version loses about three quarters of the increments: writes overwrite each other.

- `with lock:` makes read-compute-write a critical section: correct, but threads now take turns.
- Removing `time.sleep(0)` makes the race *rarer*, not absent. Tests that pass “most of the time” are the hallmark of races.

The Go version needs no artificial delay, because goroutines run in parallel on multiple cores:

```go
// Three counters incremented by many goroutines: racy, mutex, atomic.
package main

import (
    "fmt"
    "sync"
    "sync/atomic"
)

const goroutines = 8
const increments = 100_000

func main() {
    var wg sync.WaitGroup

    // 1. Racy: counter++ is a read-modify-write, not one indivisible step.
    racy := 0
    for g := 0; g < goroutines; g++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for i := 0; i < increments; i++ {
                racy++
            }
        }()
    }
    wg.Wait()

    // 2. Mutex: only one goroutine at a time inside the critical section.
    var mu sync.Mutex
    locked := 0
    for g := 0; g < goroutines; g++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for i := 0; i < increments; i++ {
                mu.Lock()
                locked++
                mu.Unlock()
            }
        }()
    }
    wg.Wait()

    // 3. Atomic: the hardware performs the increment indivisibly.
    var atomicCount atomic.Int64
    for g := 0; g < goroutines; g++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for i := 0; i < increments; i++ {
                atomicCount.Add(1)
            }
        }()
    }
    wg.Wait()
```

```
        fmt.Println("expected:", goroutines*increments)
        fmt.Println("racy:    ", racy)
        fmt.Println("mutex:   ", locked)
        fmt.Println("atomic:  ", atomicCount.Load())
    }
```

On a 4-core VM with Go 1.24, `go run .` printed `racy: 493557` against `expected: 800000`; the mutex and atomic counters were exact.

What to notice:

- `go run -race .` runs Go's **race detector**, which reports `WARNING: DATA RACE` with both stack traces; use `-race` in CI [Go Docs: Data Race Detector].
- `atomic.Int64` (Go 1.19+) is one locked hardware instruction, usually faster than a mutex for a single counter; for two or more variables, use a mutex.
- In Go, Java and C/C++, racy programs have weak or undefined semantics: compilers and CPUs may reorder or cache values [Go Docs: Memory Model].

COMMON MISTAKE

"It is only a counter; a small error is fine." The same pattern guards bank balances, inventory counts, rate-limit tokens and "is this seat booked?" flags. Treat every read-modify-write of shared state as a critical section, or replace it with a single atomic operation.

6.10 Synchronization primitives

Each primitive below answers a specific coordination problem. Table 6.3 at the end of the section compares them.

6.10.1 Mutexes and spinlocks

DEFINITION

Definition — Mutex (mutual-exclusion lock). An object with two operations, `lock` and `unlock`, guaranteeing that at most one thread holds it at a time. A thread that calls `lock` while another holds it waits.

A mutex is the key to a single-occupancy bathroom. A **spinlock** waits by looping, burning CPU but reacting within nanoseconds; it suits tiny kernel critical sections. A **blocking mutex** puts the waiter to sleep (§6.4) and wakes it on unlock. Linux builds mutexes on **futexes**: an uncontended lock is one atomic instruction in user space; only under contention does the thread call `futex()` to sleep in the kernel [Franke et al. 2002]. Most mutexes

spin briefly, then sleep. Uncontended locks cost tens of nanoseconds; contended ones cost microseconds of context switching.

6.10.2 Semaphores

DEFINITION

Definition — Semaphore. An integer counter with two atomic operations introduced by Dijkstra: **wait** (P, or *acquire*) decrements it and blocks if the result would be negative; **signal** (V, or *release*) increments it and wakes a waiter [Dijkstra 1965].

A semaphore initialized to N is a car park with N spaces: the natural tool for **limiting concurrency**, such as at most 20 database connections or 4 concurrent inferences on one GPU. Unlike a mutex, any thread may signal it, so it has no owner and cannot support priority inheritance (§6.11.5).

6.10.3 Condition variables

DEFINITION

Definition — Condition variable. A queue of threads waiting for some condition on shared state to become true, always used together with a mutex. `wait` atomically releases the mutex and sleeps; `notify` (`signal`) wakes one waiter and `notify_all` (`broadcast`) wakes all.

A mutex answers “only one at a time”; a condition variable answers “wait until something is true”, such as “the queue is not empty”. The bounded buffer below is the canonical example: producers wait while the buffer is full, consumers while it is empty.

```
        """A bounded buffer (producer/consumer) built from one mutex and two condition
variables."""
        import threading
        from collections import deque


        class BoundedBuffer:
            def __init__(self, capacity: int) -> None:
                self.capacity = capacity
                self.items = deque()
                self.lock = threading.Lock()
                self.not_full = threading.Condition(self.lock)   # producers wait here
                self.not_empty = threading.Condition(self.lock)  # consumers wait here

            def put(self, item) -> None:
                with self.lock:
                    while len(self.items) == self.capacity:      # while, not if
                        self.not_full.wait()                     # releases lock while
asleep
                    self.items.append(item)
                    self.not_empty.notify()                      # wake one consumer

            def get(self):
                with self.lock:
                    while not self.items:
                        self.not_empty.wait()
                    item = self.items.popleft()
                    self.not_full.notify()                       # wake one producer
                    return item


        if __name__ == "__main__":
            buf = BoundedBuffer(capacity=8)
            N_PRODUCERS, PER_PRODUCER = 3, 10_000
            results = []

            def producer(pid: int) -> None:
                for i in range(PER_PRODUCER):
                    buf.put((pid, i))

            def consumer() -> None:
                for _ in range(N_PRODUCERS * PER_PRODUCER):
                    results.append(buf.get())

            threads = [threading.Thread(target=producer, args=(p,)) for p in
range(N_PRODUCERS)]
            threads.append(threading.Thread(target=consumer))
            for t in threads:
                t.start()
            for t in threads:
                t.join()
            print("consumed:", len(results), "unique:", len(set(results)))
```

Output: `consumed: 30000 unique: 30000`.

What to notice:

- The condition is re-checked in a `while` **loop**: under **Mesa semantics**, used by almost every real system, a woken thread only becomes runnable, and another thread may take the item first; **spurious wake-ups** are also allowed [Lampson & Redell 1980].
- `wait()` releases the lock while sleeping and reacquires it before returning.
- The bounded capacity is **backpressure** (§22.10): producers slow to the consumer's pace instead of exhausting memory. Python's `queue.Queue` works this way.

6.10.4 Atomics and compare-and-swap

DEFINITION

Definition — Compare-and-swap (CAS). An atomic CPU instruction that, in one indivisible step, compares a memory location with an expected value and, only if they are equal, replaces it with a new value, reporting whether it succeeded. On x86-64 it is `LOCK CMPXCHG`; ARMv8.1 and later provide `CAS`, and earlier ARM uses load-linked/store-conditional pairs.

Atomic instructions (atomic add, exchange, CAS) make a single read-modify-write indivisible in hardware, with no lock and no sleeping. CAS is the building block for everything else: to apply any update, read the current value, compute the new one, and CAS it in; if another thread changed the value meanwhile, the CAS fails and you retry.

```
        import java.util.concurrent.atomic.AtomicLong;

        /** A lock-free "max seen so far" tracker built from a compare-and-swap retry
loop. */
        public class CasCounter {
            private final AtomicLong max = new AtomicLong(Long.MIN_VALUE);
            private final AtomicLong retries = new AtomicLong();

            void offer(long candidate) {
                while (true) {
                    long current = max.get();                       // 1. read
                    if (candidate <= current) return;               // nothing to do
                    if (max.compareAndSet(current, candidate)) {    // 2. swap only if
unchanged
                        return;
                    }
                    retries.incrementAndGet();                      // 3. someone beat
us: retry
                }
            }

            public static void main(String[] args) throws InterruptedException {
                CasCounter tracker = new CasCounter();
                Thread[] workers = new Thread[4];
                for (int t = 0; t < workers.length; t++) {
                    final int id = t;
                    workers[t] = new Thread(() -> {
                        for (long i = 0; i < 1_000_000; i++) {
                            tracker.offer(i * workers.length + id);
                        }
                    });
                    workers[t].start();
                }
                for (Thread w : workers) w.join();
                System.out.println("max     = " + tracker.max.get());   // 3999999
                System.out.println("retries = " + tracker.retries.get());
            }
        }
```

On JDK 21 with 4 threads this printed `max = 3999999` and several hundred thousand retries.

What to notice:

- No thread blocks; a failed CAS means another thread made progress. This is **lock-free** progress.
- Each retry bounced a cache line between cores (§5.4); under heavy contention a CAS loop can be slower than a mutex.
- Atomics also impose **memory ordering**: writes before an atomic store become visible to threads that observe it. Plain variables have no such guarantee.

6.10.5 Read-write locks

A **read-write lock** allows many concurrent readers or one writer. It helps when reads dominate *and* critical sections are long, such as a routing table. Caveats: every reader updates the lock's shared counter, so for very short reads on many cores it can be slower than a mutex; and readers can starve writers unless the lock is **writer-preferring**. For read-mostly data, **copy-on-write snapshots** (readers load an atomic pointer to an immutable version; writers swap in a new one) and Linux's **read-copy-update (RCU)** avoid reader locking entirely.

6.10.6 Lock-free data structures and the ABA problem

A **lock-free** data structure guarantees that some thread always makes progress, even if others are paused mid-operation; a **wait-free** one guarantees that *every* thread finishes in a bounded number of steps [Herlihy & Shavit 2012]. The classic example is the **Treiber stack** [Treiber 1986]: `push` reads the current top, sets the new node's `next` to it, and CASes the top from old to new.

The **ABA problem** breaks naive CAS designs. CAS checks only that the value *looks* the same, not that it *is* unchanged:

1. Thread 1 reads top = A (with A.next = B) and is preempted before its CAS(top, A → B).
2. Thread 2 pops A, pops B, frees B, then pushes A back (or a new node reusing A's memory). Top is A again, but A.next is now something else.
3. Thread 1 resumes; CAS(top, A → B) *succeeds*, because top equals A, and installs the freed node B as the top. The stack is corrupted.

Remedies: pair the pointer with a **version counter** and CAS both together (a tagged pointer); use **hazard pointers** [Michael 2004] or **epoch-based reclamation** so memory is not reused while any thread may hold a reference; or rely on garbage collection, which prevents node reuse while referenced. In practice, use tested concurrent collections (`java.util.concurrent`, Go channels, Rust's `crossbeam`) rather than writing your own.

Table 6.3 — Synchronization primitives compared.

Primitive	Problem it solves	Waiting behavior	Typical cost (uncontended)	Pitfalls	Choose when
Spinlock	Mutual exclusion for tiny sections	Busy-waits	~10–20 ns	Wastes CPU; terrible if holder is preempted	Kernel code, sections of a few instructions
Mutex	Mutual exclusion	Spin briefly, then sleep	~20–50 ns	Deadlock, contention, priority inversion	Default for protecting shared state
Semaphore	Limit concurrency to N	Sleeps	Similar to mutex	No ownership; easy to miscount	Connection or resource pools
Condition variable	Wait until a predicate holds	Sleeps, releases mutex	Similar to mutex	Must loop on predicate; lost wake-ups	Queues, producer/consumer
Atomic / CAS	Single-variable read-modify-write	Never sleeps; may retry	~5–20 ns; more when contended	Only one variable; ABA; ordering subtleties	Counters, flags, pointers
Read-write lock	Many readers, rare writers	Sleeps	Higher than mutex	Writer starvation; reader-counter contention	Long read-mostly sections
Lock-free structure	Progress without locks	Never sleeps	Varies	ABA; memory reclamation; very hard to verify	Proven library implementations only

Costs are orders of magnitude on current x86-64 servers (as of the 2020s) and grow sharply under contention, when cache lines move between cores.

6.11 Deadlock, livelock, starvation and priority inversion

6.11.1 Deadlock and the Coffman conditions

DEFINITION

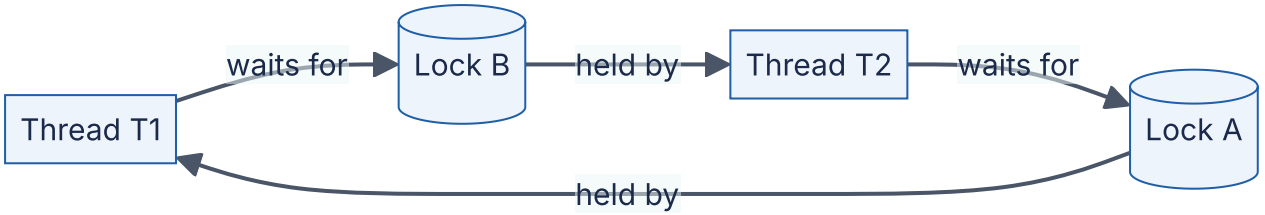
Definition — Deadlock. A state in which each thread in a set is waiting for a resource held by another thread in the same set, so none can ever proceed.

Four cars at a four-way junction, each waiting for the car on its right, will never move. Deadlock requires all four of these conditions at once [Coffman et al. 1971; Tanenbaum & Bos 2014]:

1. **Mutual exclusion** — a resource can be held by only one thread at a time.
2. **Hold and wait** — a thread holds at least one resource while waiting for another.

3. **No preemption** — a resource cannot be forcibly taken away; it is released only voluntarily.
4. **Circular wait** — there is a cycle of threads, each waiting for a resource held by the next.

Figure 6.8 — Resource-allocation graph with a deadlock cycle.



Reading the diagram. Rectangles are threads, cylinders are locks. Thread → lock means "waits for"; lock → thread means "is held by". Following the arrows from T1 returns to T1: a **cycle**. With single-instance resources such as locks, a cycle is necessary and sufficient for deadlock, which is how deadlock detectors, including those in databases (§11.15), work. Every edge is a synchronous blocking wait; breaking any one edge removes the deadlock.

The code below produces exactly this cycle. Because a real deadlock would hang forever, it uses `acquire(timeout=...)` to detect the situation:

```
        """Two threads take two locks in opposite order -> deadlock (detected via time-
out)."""
        import threading
        import time

        lock_a = threading.Lock()
        lock_b = threading.Lock()


        def worker(name: str, first: threading.Lock, second: threading.Lock) -> None:
            with first:
                time.sleep(0.1)                      # make the bad interleaving very
likely
                if second.acquire(timeout=1.0):      # a real program would block for-
ever
                    second.release()
                    print(f"{name}: finished")
                else:
                    print(f"{name}: gave up after 1 s -- probable deadlock")


        t1 = threading.Thread(target=worker, args=("T1", lock_a, lock_b))
        t2 = threading.Thread(target=worker, args=("T2", lock_b, lock_a))  # opposite
order!
        t1.start(); t2.start(); t1.join(); t2.join()

        # Fix: impose a global lock order (always A before B) and the cycle cannot form.
        t1 = threading.Thread(target=worker, args=("T1", lock_a, lock_b))
        t2 = threading.Thread(target=worker, args=("T2", lock_a, lock_b))
        t1.start(); t2.start(); t1.join(); t2.join()
```

Typical output: `T1: gave up after 1 s -- probable deadlock`, `T2: finished`, `T1: finished`, `T2: finished`.

What to notice:

- In the first run each thread holds one lock and wants the other. When T1 gives up and releases A, T2 can finish: the timeout broke **hold and wait**, which is how databases resolve deadlocks by aborting a victim transaction.
- In the second run both threads take A before B. No cycle can form, so no timeout is needed.

6.11.2 Handling deadlock: prevention, avoidance, detection

Prevention makes one Coffman condition impossible by design:

- *Circular wait:* define a global **lock ordering** (by address, by ID, by level) and always acquire locks in that order. This is the most practical technique.
- *Hold and wait:* acquire all needed locks at once, or use `tryLock` and release everything on failure.

- *No preemption:* use timeouts so a waiting thread gives up what it holds.
- *Mutual exclusion:* avoid locks entirely: immutable data, lock-free structures, or one owner thread per piece of state (actors, §6.15.3).

Avoidance refuses requests that *could* lead to deadlock. Dijkstra's **banker's algorithm** grants a resource only if some order still exists in which every thread can obtain its declared maximum and finish (a **safe state**) [Dijkstra 1965]; needing maximum demands in advance makes it rare in practice.

Detection and recovery lets deadlocks happen, finds cycles in the wait-for graph and aborts a victim, as databases do. For applications, a **thread dump** (`jstack`, Go's `SIGQUIT` dump) shows each thread's held and awaited locks.

6.11.3 Livelock

In a **livelock**, threads keep reacting to each other without progress, like two people in a corridor stepping aside the same way again and again; threads that `tryLock`, fail, release and retry in lock-step do the same. The fix is **randomized backoff**, as with retry jitter (§22.12).

6.11.4 Starvation

In **starvation**, a thread waits indefinitely because others are always preferred: low priority under strict priority scheduling, writers under a reader-preferring lock, a thread that always loses an unfair lock. Remedies are fairness policies: FIFO locks (Java's `new ReentrantLock(true)`), aging, priority boosts. Fair locks cost throughput, since each handoff to a sleeping thread is a context switch, so most locks default to unfair.

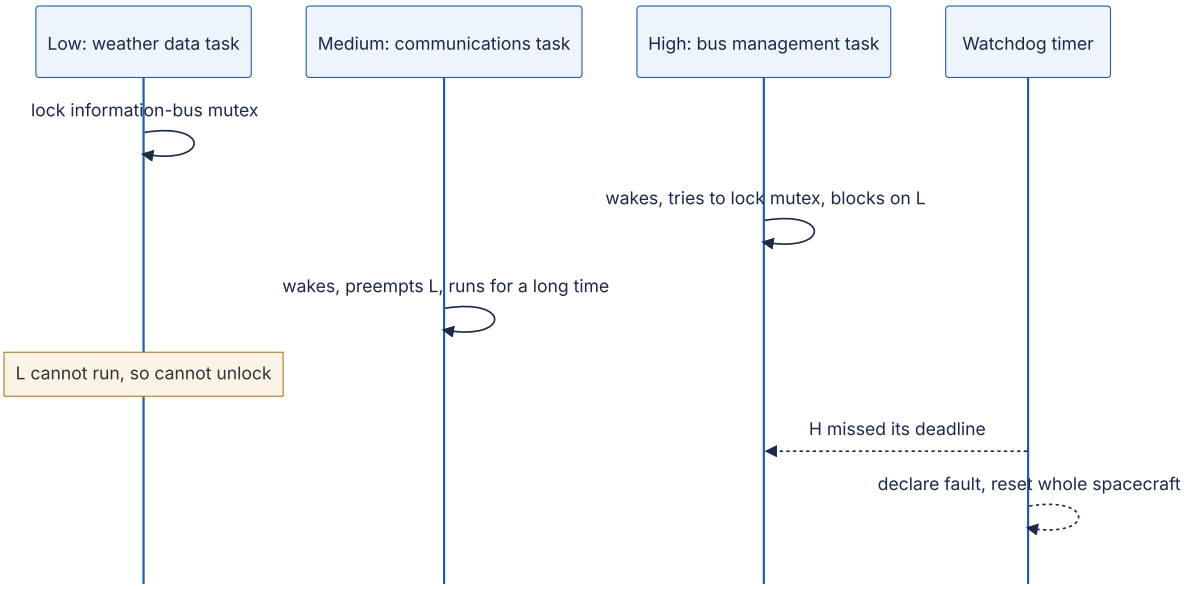
6.11.5 Priority inversion

DEFINITION

Definition — Priority inversion. A situation in which a high-priority thread is blocked waiting for a lock held by a low-priority thread, while a medium-priority thread, which needs no lock, preempts the low-priority one, so the high-priority thread effectively runs at the lowest priority.

6.11.6 Case study: Mars Pathfinder, 1997

Figure 6.9 — Priority inversion on Mars Pathfinder.



Reading the diagram. Three tasks share one CPU under strict priorities, and one mutex guards a shared memory area, the information bus. The low-priority weather task takes the mutex; the high-priority bus task blocks on it, normally briefly. The medium-priority communications task, which needs no mutex, preempts the low task, so the mutex is never released and the high task waits as long as the medium task runs. The dotted arrows are asynchronous: a watchdog saw that the high-priority work had not completed in time and reset the computer. Failure point: the mutex. Fix: priority inheritance.

INDUSTRY EXAMPLE

Mars Pathfinder, July 1997. Days after landing on 4 July 1997, the Pathfinder lander began suffering total system resets, losing data each time. JPL engineers reproduced the problem on an identical ground system and traced it to priority inversion on a mutex in the VxWorks real-time operating system. The mutex had been created with **priority inheritance** disabled. Priority inheritance temporarily raises a lock holder to the priority of the highest thread waiting for it, so the low task would have run at high priority, released the lock quickly, and not been preempted by the medium task [Sha et al. 1990]. Because the team had left debugging facilities enabled in the flight software, they could change the mutex's initialization parameter remotely; after the patch was uploaded the resets stopped. The lessons: real-time systems need priority-inheritance (or priority-ceiling) locks, and the ability to diagnose and patch in production is a design requirement, not a luxury. *Evidence: Primary* — account by Glenn Reeves, JPL software team lead [Reeves 1997], as summarized by Mike Jones [Jones 1997]

Priority inversion also appears in servers, when a background job holds a lock, pool slot or database row lock that user-facing requests need.

Table 6.4 — Liveness failures compared.

Failure	What it looks like	Threads are	Root cause	Detection	Fix
Deadlock	Requests hang forever; CPU idle	Blocked	Cycle of waits (all four Coffman conditions)	Thread dump shows lock cycle	Lock ordering, timeouts, victim abort
Livelock	CPU busy, no progress	Running	Symmetric retry reactions	High CPU with zero throughput	Randomized backoff
Starvation	Some requests never finish; others are fine	Ready or blocked	Unfair scheduling or locking	Per-class latency; wait-time histograms	Fair locks, aging, quotas
Priority inversion	High-priority work delayed by low-priority work	High blocked; medium running	Lock held by low-priority thread	Latency spikes correlated with background jobs	Priority inheritance or ceiling; avoid sharing locks across priorities

6.12 Inter-process communication

Processes have private address spaces (§6.3.2), so they cooperate through kernel-provided channels, trading speed, structure and reach.

- **Pipes** — a one-way byte stream through a kernel buffer (64 KiB by default on Linux), as in `grep error log | wc -l`; *named pipes* (FIFOs) let unrelated processes connect. A writer to a full pipe blocks: built-in backpressure.
- **Unix domain sockets** — the socket API on one machine, addressed by a file path; bidirectional, and able to pass file descriptors. Docker's API and local database connections use them; they skip the TCP/IP stack.
- **TCP/UDP sockets** — slower locally, but the same code works across machines (§7.7).
- **Shared memory** — processes map the same physical pages (`shm_open` + `mmap`). Fastest, with no system calls after setup, but the processes must synchronize themselves. PostgreSQL's shared buffer pool uses it.
- **Message queues** — POSIX `mq_open` keeps discrete, prioritized messages in the kernel, locally; distributed brokers (Chapter 30) generalize the idea.
- **Signals** — asynchronous notifications carrying only a number: `SIGTERM` (please exit), `SIGKILL` (exit now, uncatchable), `SIGINT` (Ctrl-C), `SIGHUP` (often "reload"), `SIGCHLD` (child exited). Handlers interrupt the program anywhere, so they may call only **async-signal-safe** functions; set a flag and act in the main loop [Linux man-pages: signal(7)]. Kubernetes sends `SIGTERM`, waits a grace period (30 s by default), then `SIGKILL`, so services must handle `SIGTERM` to drain gracefully (§43.4).

Table 6.5 — IPC mechanisms compared.

Mechanism	Direction	Data model	Speed (local)	Scope	Synchronization	Choose when
Pipe / FIFO	One-way	Byte stream	Fast	Same machine (pipe: related processes)	Built in (blocking)	Streaming between parent and child, shell pipelines
Unix domain socket	Two-way	Stream or datagram	Fast	Same machine	Built in	Local client–server, sidecars, passing file descriptors
TCP socket	Two-way	Stream	Moderate	Network	Built in	Anything that may later run on another machine
Shared memory	Any	Raw bytes	Fastest	Same machine	Do it yourself	Very high throughput, low latency, large shared data
POSIX message queue	One-way	Discrete messages	Fast	Same machine	Built in	Small prioritized messages between local processes
Signal	One-way	A number	Fast	Same machine	Asynchronous	Lifecycle control: stop, reload, child exit

6.13 I/O models: from blocking to io_uring

A network server spends most of its life waiting. The **I/O model** decides what a thread does while it waits, and so how many connections one machine can hold.

6.13.1 Blocking and non-blocking I/O

With **blocking I/O**, `read()` on a socket with no data puts the thread to sleep until data arrives: simple code, but one thread per waiting connection. With **non-blocking I/O** (`O_NONBLOCK`), `read()` returns `EAGAIN` immediately when there is no data, so one thread can juggle many sockets, but it needs a way to sleep until *any* of them is ready rather than spinning.

6.13.2 I/O multiplexing: select, poll, epoll, kqueue

I/O multiplexing lets one thread ask the kernel: “here are my sockets; sleep until at least one can be read or written without blocking, and tell me which.” These are **readiness** notifications.

- `select` (4.2BSD) passes descriptor bitmaps on every call, is limited to descriptors below `FD_SETSIZE` (1,024 on Linux/glibc), and scans all of them: $O(n)$ work per wake-up.
- `poll` uses an array, removing the limit, but still scans all n.
- `epoll` (Linux 2.6) registers interest once (`epoll_ctl`); the kernel maintains a ready list, and `epoll_wait` returns only ready descriptors, so cost scales with *ready*, not watched, descriptors [Linux man-pages: epoll(7)].
- `kqueue` (FreeBSD 4.1, 2000; also macOS) scales the same way and also covers files, signals, timers and processes [Lemon 2001].

Level-triggered notification keeps reporting a socket while data remains; **edge-triggered** reports only new arrivals, so the program must read until `EAGAIN`. Readiness has a blind spot: regular files are always “ready” yet reads can block on disk, which is why Node.js’s libuv runs file operations on a thread pool (4 threads by default) [Node.js Docs: Don’t Block the Event Loop].

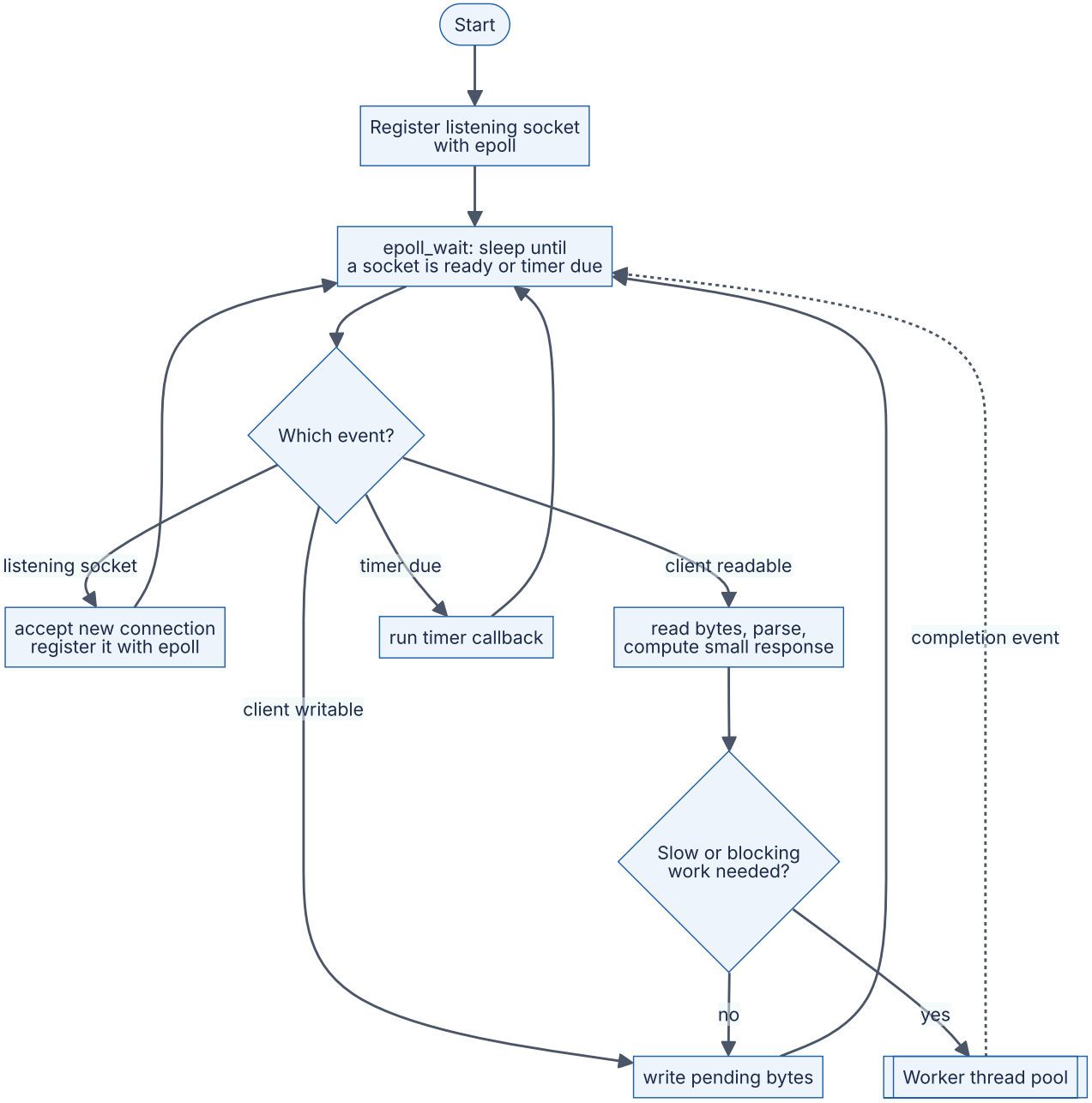
6.13.3 The C10K problem

In 1999 Dan Kegel’s web page *The C10K problem* asked how one server could handle ten thousand simultaneous clients, when the hardware clearly could but the software could not [Kegel 1999]. Thread-per-connection hit memory and scheduling limits, and `select` / `poll` spent their time rescanning idle sockets. Kegel surveyed the strategies (thread per client, readiness notification, asynchronous completion); the eventual answer was event-driven servers on `epoll` / `kqueue`, the architecture of nginx, HAProxy, Node.js and Redis. Today a tuned server holds millions of mostly idle connections (see §9.9).

6.13.4 The event loop

> DEFINITION
>
> **Definition — Event loop.** A single thread that repeatedly waits for events (ready sockets, expired timers, completed tasks) using an I/O multiplexing call, and dispatches each event to a short, non-blocking handler.

Figure 6.10 — An event loop on epoll.



Reading the diagram. One thread sleeps in `epoll_wait`, the only place it may block, and dispatches each ready event: accept a connection, read and parse a request, flush output, run a timer. Handlers must be short. Blocking or CPU-heavy work (disk reads, password hashing, a synchronous database driver) goes to a **worker pool**, whose completion returns asynchronously as a new event (dotted arrow). The failure point is a slow handler: while one callback runs for 200 ms, *every* connection on the loop waits 200 ms. To use more cores, run one loop per core (nginx workers, `SO_REUSEPORT`, Node.js cluster).

The Python program below is a complete event-loop echo server. On Linux, `selectors.DefaultSelector` uses `epoll`; on macOS, `kqueue`.

```
        """A single-threaded echo server: one event loop, many connections.

        On Linux, selectors.DefaultSelector is backed by epoll; on macOS/BSD by kqueue.
        """
        import selectors
        import socket

        sel = selectors.DefaultSelector()


        def accept(server: socket.socket) -> None:
            conn, _addr = server.accept()              # ready, so this will not block
            conn.setblocking(False)
            sel.register(conn, selectors.EVENT_READ, handle)


        def handle(conn: socket.socket) -> None:
            data = conn.recv(4096)                     # ready, so this will not block
            if data:
                conn.sendall(data)                     # simplification: assumes the
send buffer has room
            else:                                      # empty read = peer closed
                sel.unregister(conn)
                conn.close()


        def serve(host: str = "127.0.0.1", port: int = 9000) -> None:
            server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            server.bind((host, port))
            server.listen(1024)
            server.setblocking(False)
            sel.register(server, selectors.EVENT_READ, accept)
            print(f"selector = {type(sel).__name__}; listening on {host}:{port}")
            while True:                                # THE event loop
                for key, _mask in sel.select():        # block until >= 1 socket is
ready
                    callback = key.data
                    callback(key.fileobj)              # run the handler; it must not
block


        if __name__ == "__main__":
            serve()
```

Connect with `nc 127.0.0.1 9000` from several terminals; in testing it printed `selector = EpollSelector` and served 200 concurrent clients correctly.

What to notice:

- There is exactly one blocking call, `sel.select()`. Everything else runs only when the kernel has said it will not block.

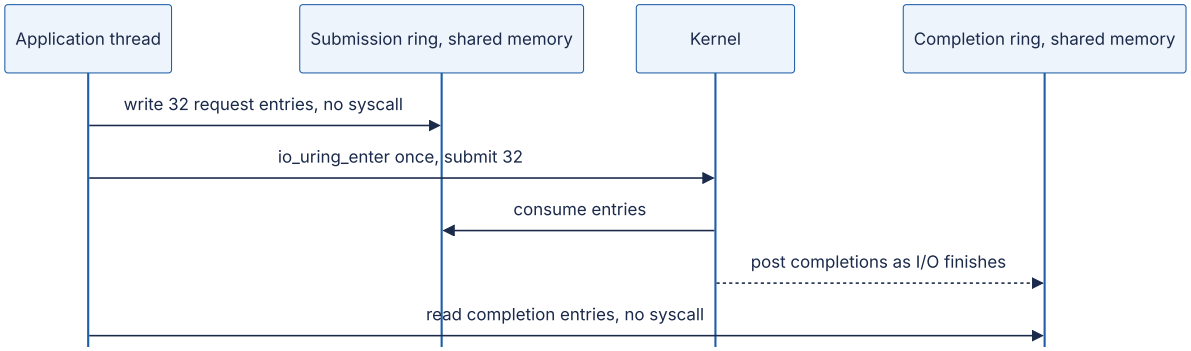
- `sendall` on a non-blocking socket can still fail if the client is slow to read; a production server keeps a per-connection output buffer and registers for `EVENT_WRITE`. Frameworks handle this for you.

6.13.5 Completion-based I/O and io_uring

Readiness APIs need one system call to learn a socket is ready and another to read it. **Completion-based** APIs instead let the application *submit* an operation and later be told it is *complete* (Windows I/O completion ports; Linux's older AIO worked well only for `O_DIRECT` files).

io_uring, added in Linux 5.1 (2019) by Jens Axboe, provides two ring buffers in memory shared between application and kernel: a **submission queue (SQ)** and a **completion queue (CQ)** [Axboe 2019].

Figure 6.11 — io_uring submission and completion rings.



Reading the diagram. The application writes request entries (opcode, descriptor, buffer, user tag) into the shared submission ring with plain memory writes, then submits any number with *one* `io_uring_enter` call. The kernel performs them, files and sockets alike, and posts results asynchronously to the completion ring (dotted arrow), possibly out of order; the tag matches results to requests. Reaping completions needs no system call, and in **SQPOLL** mode a kernel thread polls the submission ring so submission needs none either. Costs: a more complex programming model and a large new kernel attack surface. Google reported in 2023 that most kernel exploits submitted to its bug-bounty program involved io_uring and that it had restricted io_uring on ChromeOS, Android apps and its production servers [Google Security Blog 2023]; some container runtimes block it by default.

Table 6.6 — I/O models compared.

Model	How it works	Advantages	Disadvantages	Scales to	Choose when
Blocking, thread per connection	Each thread blocks in `read` / `write`	Simplest code; natural stack traces	Memory and switching per connection	Thousands of connections	Moderate concurrency; blocking libraries
Non-blocking + busy polling	Loop calling `read` until not `EAGAIN`	Lowest latency on a dedicated core	Burns a full core	Few connections	Kernel-bypass and trading systems
`select` / `poll`	Readiness for a set passed each call	Portable	$O(n)$ per call; `select` limited to 1,024	Hundreds	Small tools, portability
`epoll` / `kqueue`	Register once; wait for ready list	$O(\text{ready})$; mature	Readiness only; files still block	Millions of sockets	Network servers on Linux/BSD
`io_uring`	Shared submission/completion rings	Batching; async files and sockets; few syscalls	Newer; security restrictions; complex	Very high IOPS	Storage engines, proxies on modern Linux

6.14 File systems, journaling and fsync

6.14.1 What a file system does

A **file system** turns an array of numbered disk blocks into named, hierarchical files. Each file has an **inode** holding metadata (size, owner, permissions, timestamps) and the locations of its data blocks; a **directory** maps names to inode numbers. Creating `/data/log.txt` touches several structures: a new inode, the free-block bitmap, data blocks and the parent directory.

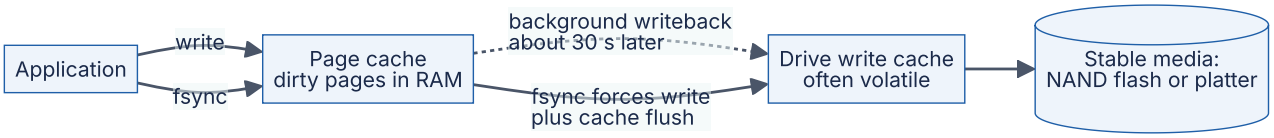
6.14.2 Crash consistency and journaling

A crash midway through those updates can leave a directory entry pointing at an uninitialized inode; older file systems needed a full-disk `fsck`, taking hours on large disks. A **journaling** file system uses a **write-ahead log** (§13.5): record the intended changes in a journal, commit, then apply them in place; after a crash, replay committed entries in seconds. ext4 offers three modes: **`data=journal`** (data and metadata both journaled: safest, slowest), **`data=ordered`** (the default: only metadata journaled, but data blocks are written before the metadata that points at them) and **`data=writeback`** (metadata only, no ordering: a crash can expose stale data). XFS journals metadata. **Copy-on-write** file systems such as ZFS and Btrfs never overwrite in place: they write new blocks and atomically switch a root pointer.

Journaling protects the file system's *own* structures. It does **not** make your application's data durable at any particular moment.

6.14.3 fsync and durability

Figure 6.12 — The write path from application to stable storage.



Reading the diagram. `write()` copies data into the page cache and returns; nothing is on disk yet. The dotted arrow is asynchronous background writeback (§6.6.6), roughly 30 s later by default. `fsync(fd)` is the synchronous path: it writes the file's dirty pages and metadata, waits for the device, and issues a **cache flush** so the drive's volatile write cache reaches stable media, the storage boundary that matters. Enterprise SSDs with **power-loss protection** capacitors can treat their cache as stable. Failure points: a crash loses dirty pages, a drive that ignores flushes loses its cache, and a writeback I/O error can lose data silently.

Durability pitfalls, each of which has caused real data loss:

1. **`write()` is not durable.** Without `fsync` (or `fdatasync`, which skips non-essential metadata), a crash loses acknowledged writes.
2. **The directory needs `fsync` too.** Creating or renaming a file changes the *directory*, which must also be synced [Linux man-pages: fsync(2); Pillai et al. 2014].
3. **"Write temp, then rename" needs `fsync` before the rename**, or a crash can leave the new name pointing at an empty file.
4. **`fsync` errors are not retryable.** On Linux a failed writeback may mark pages clean, so a second `fsync` can succeed although the data was lost. After the 2018 "fsyncgate" investigation, PostgreSQL crashes and recovers from its write-ahead log on any `fsync` failure [PostgreSQL Wiki: Fsync Errors; Rebello et al. 2020].
5. **macOS `fsync` does not flush the drive cache**; use `fcntl(fd, F_FULLFSYNC)` [Apple: fsync(2) man page].
6. **`O_DIRECT` is not `O_SYNC`**: bypassing the page cache does not make writes durable.
7. **Torn writes**: a crash mid-write can leave half-old, half-new sectors; databases detect this with checksums (§13.12).

The program below replaces a file atomically and durably on POSIX systems:

```
        """Atomically and durably replace a file on a POSIX system (Linux, macOS)."""
        import os


        def durable_replace(path: str, data: bytes) -> None:
            directory = os.path.dirname(os.path.abspath(path))
            tmp_path = path + ".tmp"

            fd = os.open(tmp_path, os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o644)
            try:
                view = memoryview(data)
                while view:                           # write() may write fewer bytes
than asked
                    written = os.write(fd, view)
                    view = view[written:]
                os.fsync(fd)                          # 1. file contents reach stable
storage
            finally:
                os.close(fd)

            os.replace(tmp_path, path)                # 2. atomic rename over the old
file

            dir_fd = os.open(directory, os.O_RDONLY)
            try:
                os.fsync(dir_fd)                      # 3. the rename itself (a directory
entry) is durable
            finally:
                os.close(dir_fd)


        if __name__ == "__main__":
            durable_replace("config.json", b'{"version": 2}\n')
            with open("config.json", "rb") as f:
                print(f.read())
```

What to notice:

- Readers see the complete old or complete new file, never a mixture: `os.replace` is an atomic rename within one file system.
- Skipping step 1 risks an empty file under the new name; skipping step 3 risks the old file reappearing after a crash.
- If `os.fsync` raises, the function propagates the exception rather than retrying (pitfall 4). On macOS, add `fcntl.fcntl(fd, fcntl.F_FULLFSYNC)` for power-loss durability.

FORMULA

Group-commit throughput

$$X = \frac{B}{t_{\text{fsync}}}$$

Symbol	Meaning	Units
X	Maximum durable commits per second on one log	commits/s
B	Commits made durable by one `fsync` (batch size)	commits
t_{fsync}	Latency of one `fsync`	s

WORKED EXAMPLE

Worked example — Why databases batch fsyncs. An SSD without power-loss protection takes $t_{\text{fsync}} = 2$ ms (0.002 s) per `fsync`. One commit per `fsync`: $X = 1/0.002 =$ **500 commits/s**, no matter how fast the CPU is. **Group commit** waits briefly to collect concurrent commits into one `fsync`. With $B = 50$: $X = 50/0.002 =$ **25,000 commits/s**, at the price of up to a couple of milliseconds of extra latency per commit. **Interpretation.** Durable write throughput is set by fsync latency, not bandwidth. **Engineering implication:** databases batch commits into one log flush, and drives with power-loss protection raise the ceiling further. When a vendor quotes “100,000 writes per second”, ask how many `fsync`s that included.

Table 6.7 — Durability options for a write.

Option	What is guaranteed at return	Relative cost	Choose when
`write()` only	Data in page cache; lost on power failure or kernel crash	Lowest	Caches, temporary data, data you can regenerate
`write()` + `fdatasync()`	Data and size metadata on stable storage	High (one device flush)	Write-ahead logs and append-only files
`write()` + `fsync()`	Data and all metadata on stable storage	High	General durable files
+ `fsync()` on directory	The file’s name and existence are durable	Additional flush	Newly created or renamed files
`O_DSYNC` / `O_SYNC` on open	Every `write` behaves as if followed by `fdatasync` / `fsync`	Highest per write	Small, critical, infrequent writes
Replication instead of local fsync	Data in RAM on several machines	Network round trip	Distributed logs that accept correlated-failure risk (§27.3)

6.15 Concurrency models in practice

The five models below answer one question differently: *what represents a waiting request, and what does it cost?*

6.15.1 Thread per request

Each request gets an OS thread (usually from a pool) running blocking code start to finish: classic Tomcat servlets, Apache httpd's worker model, threaded Python WSGI servers. **Strengths:** sequential code, readable stack traces, parallelism across cores. **Weaknesses:** each waiting request costs a whole thread, capping concurrency by memory and switching overhead, and a slow dependency can tie up every pool thread.

6.15.2 Event loop (Node.js, nginx, Redis)

One thread runs an event loop (§6.13.4); a waiting request costs a small state object and a callback. Node.js runs JavaScript on one libuv loop; nginx runs one loop per worker process; Redis executes commands on one main thread, making each command trivially atomic. **Strengths:** tiny per-connection cost, no locks for loop-owned state. **Weaknesses:** one core per loop, and any blocking callback stalls every connection [Node.js Docs: Don't Block the Event Loop].

6.15.3 Actors (Erlang/OTP, Akka)

An **actor** is an isolated unit of state with a **mailbox**; actors share no memory, communicate only by asynchronous messages and process one message at a time, so their state has no data races [Hewitt et al. 1973]. Erlang processes, scheduled by the BEAM virtual machine with one scheduler thread per core, take a few hundred machine words each at creation [Erlang Docs: Efficiency Guide]. Erlang/OTP adds **supervision trees** that restart crashed actors ("let it crash") [Armstrong 2003]; Akka brings the model to the JVM. **Weaknesses:** unbounded mailboxes, extra plumbing for request/response, and hard reasoning about ordering across actors.

> INDUSTRY EXAMPLE
>
> **WhatsApp on Erlang.** WhatsApp built its messaging servers on Erlang/FreeBSD and reported in a 2012 engineering blog post more than two million simultaneous TCP connections on a single server, attributing it to lightweight Erlang processes and kernel tuning. *Evidence: Primary* — [WhatsApp Blog 2012]

6.15.4 Goroutines (Go)

Go multiplexes **goroutines** onto `GOMAXPROCS` OS threads (by default the number of usable CPUs). A goroutine starts with a 2 KiB stack that grows on demand. On network I/O the runtime parks the goroutine and registers the socket with its **netpoller** (built on `epoll` / `kqueue`), so blocking-style code gets event-loop efficiency; since Go 1.14 long-running goroutines can also be preempted. **Weaknesses:** shared memory still allows races and deadlocks (§6.9), and leaked goroutines accumulate silently. Java's **virtual threads** (JDK 21, 2023) apply the same idea to the JVM [OpenJDK JEP 444].

6.15.5 async/await (Python asyncio, JavaScript, C#, Rust)

async/await is syntax for event-loop code that reads sequentially. An `async` function is a **coroutine**; each `await` is a point where it may suspend and return control to the loop. Scheduling is **cooperative**: coroutines yield only at `await`.

```
        """10,000 concurrent 'requests', one thread, about one second."""
        import asyncio
        import time


        async def handle_request(i: int) -> int:
            await asyncio.sleep(1.0)        # stands in for a 1 s network call; yields
to the loop
            return i


        async def main() -> None:
            start = time.perf_counter()
            results = await asyncio.gather(*(handle_request(i) for i in range(10_000)))
            elapsed = time.perf_counter() - start
            print(f"{len(results)} requests finished in {elapsed:.2f} s on one thread")


        asyncio.run(main())
```

Output in testing: `10000 requests finished in 1.09 s on one thread`.

What to notice:

- 10,000 one-second waits overlap on one thread: this is concurrency without parallelism (§6.8).
- Replace `await asyncio.sleep(1.0)` with `time.sleep(1.0)` and the program takes about 10,000 seconds: one blocking call inside a coroutine blocks the whole loop.
- async is "contagious": a function that awaits must itself be `async`, so libraries split into synchronous and asynchronous versions (the "function color" problem).

6.15.6 Comparison and capacity

Table 6.8 — Concurrency models compared.

Aspect	Thread per request	Event loop	Actors	Goroutines / virtual threads	async/await
Unit of concurrency	OS thread	Callback + state	Actor with mailbox	Runtime-managed thread	Coroutine
Memory per waiting request (order of magnitude)	64 KiB–1 MiB touched stack + kernel task	~1 KiB or less	~1–3 KB	~2–8 KiB	~1–2 KiB
Scheduling	Preemptive, kernel	Cooperative, one thread	Runtime, per message	Runtime, mostly preemptive	Cooperative at `await`
Uses multiple cores	Yes	No (run one loop per core)	Yes	Yes	Only with multiple loops or executors
Shared-state safety	Locks needed	Safe within the loop	Safe by isolation	Locks or channels needed	Safe within one loop
Main failure mode	Thread exhaustion; context switching	Blocking the loop	Mailbox overflow	Goroutine leaks; races	Accidental blocking call
Code style	Sequential, blocking	Callbacks or promises	Message handlers	Sequential, blocking	Sequential with `await`
Examples	Tomcat, Apache httpd, Django with threads	nginx, Node.js, Redis	Erlang/OTP, Akka	Go servers, Java 21 virtual threads	FastAPI, aiohttp, Rust Tokio
Choose when	Moderate concurrency, blocking libraries	Many connections, light per-request CPU	Fault isolation, many stateful entities	High concurrency with simple code	High I/O concurrency in Python/JS/Rust

Memory figures are orders of magnitude for planning (as of 2026 runtimes), not measurements; measure your own service’s resident memory per connection.

FORMULA

Memory for concurrent connections

$$M_{\text{total}} = N \times (m_{\text{unit}} + m_{\text{conn}})$$

Symbol	Meaning	Units
M_{total}	Memory needed for concurrency overhead	bytes
N	Concurrent connections (or in-flight requests)	connections
m_{unit}	Resident memory per concurrency unit (thread stack + kernel task, or coroutine)	bytes
m_{conn}	Per-connection buffers (socket buffers, TLS state, request object)	bytes

WORKED EXAMPLE

Worked example — 10,000 and 100,000 connections. A chat gateway must hold $N = 10,000$ mostly idle WebSocket connections. Assume $m_{\text{conn}} = 20$ KB of socket buffers and TLS state per connection in every model.

Thread per connection (Java platform threads). Each thread reserves 1 MiB of virtual stack, but only touched pages are resident; assume 64 KiB touched + 16 KiB kernel stack + ~8 KiB other kernel state ≈ 88 KB. $M = 10,000 \times (88 + 20)$ KB **= 1.08 GB**. Workable, though with 10,000 schedulable threads. *Goroutines.* Assume 8 KiB per goroutine (≈ 8 KB). $M = 10,000 \times (8 + 20)$ KB **= 280 MB**.

Now scale to $N = 100,000$: threads need **10.8 GB** plus 100,000 kernel tasks (beyond typical `pids.max` and thread limits, and every wake-up is a context switch); goroutines need **2.8 GB**; an event loop with ~1 KB per connection needs about **2.1 GB**, dominated by socket buffers.

Interpretation. Thread-per-connection works at thousands of connections and becomes the bottleneck at hundreds of thousands, where per-connection buffers also start to dominate. **Engineering implication:** for connection-heavy services (gateways, push, chat; §9.9) choose event loops, goroutines or virtual threads; for CPU-heavy work, core count matters more than the model.

FORMULA

Thread-pool sizing

$$N_{\text{threads}} = \lambda \times W \qquad \text{and, for mixed CPU/I/O work,} \qquad N_{\text{threads}} \approx N_{\text{cpu}} \times U \times (1 + \frac{T_{\text{wait}}}{T_{\text{cpu}}})$$

Symbol	Meaning	Units
N_{threads}	Threads needed to sustain the load	threads
λ	Request arrival rate	requests/s
W	Time a request occupies a thread	s
N_{cpu}	Available cores	cores
U	Target CPU utilization	dimensionless (0–1)
T_{wait}, T_{cpu}	Per-request time spent waiting and computing	ms

WORKED EXAMPLE

Worked example — Sizing a blocking service. A service receives $\lambda = 2,000$ requests/s; each request spends 10 ms on CPU and 90 ms waiting for a database ($W = 0.1$ s). Little's Law (§4.11): $N_{\text{threads}} = 2,000 \times 0.1 =$ **200 threads** in use on average. CPU check: $2,000 \times 0.010$ s = 20 core-seconds per second, so at least **20 cores**. With $N_{\text{cpu}} = 32$, $U = 0.7$: $N_{\text{threads}} \approx 32 \times 0.7 \times (1 + 90/10) = 224$, consistent with 200 [Goetz et al. 2006]. **Interpretation.** 200–250 threads is reasonable here. If the database slows to 900 ms, the same load needs $2,000 \times 0.91 \approx 1,820$ threads: a fixed pool of 250 saturates and requests queue. **Engineering implication:** bound the pool and the queue, set timeouts on downstream calls (§22.11), and shed load rather than letting a slow dependency consume every thread.

COMMON MISTAKE

"Async is faster." async/await and event loops do not make individual operations faster; they make *waiting* cheaper, so more requests can wait at once. For CPU-bound work they are no better than threads, and a single misplaced blocking call makes them much worse.

6.16 Summary and revision notes

SUMMARY

- A **process** is a running program with a private address space; a **thread** is an execution context sharing its process's memory. The PCB (`task_struct`) stores everything needed to resume. States: New → Ready ⇄ Running → Blocked → Ready → … → Terminated.
- A **context switch** costs ~1–5 µs directly, but cache and TLB refill can cost 10–100× more. Overhead fraction $f = C/(Q + C)$: it matters when threads run only microseconds between switches.
- **Scheduling** trades turnaround, response time and fairness. RR fixes FCFS's convoy effect; MLFQ learns interactivity; Linux uses weighted fair sharing (CFS from 2.6.23, EEVDF from 6.6). CPU quotas in containers can throttle a service even when the host is idle.
- **Virtual memory** translates addresses through page tables cached in the TLB. TLB reach = entries × page size; huge pages extend it. Major page faults cost 100 µs–10 ms, so even rare faults dominate average access time. The **page cache** holds file data; `mmap` maps it into memory; copy-on-write makes `fork` cheap.
- A **system call** costs ~100 ns or more versus ~1 ns for a function call; batch, use vDSO, or use shared rings (`io_uring`).
- **Concurrency** is about structure and overlapping waits; **parallelism** is simultaneous execution. Neither implies the other.
- A **race condition** is a timing-dependent bug; protect every read-modify-write of shared state as a **critical section**, and run race detectors in CI.
- Mutex for exclusion; semaphore to limit concurrency; condition variable (in a `while` loop) to wait for a predicate; atomics/CAS for single-word updates; read-write locks for long read-mostly sections; lock-free structures only from proven libraries, minding ABA.
- **Deadlock** needs all four Coffman conditions; break circular wait with lock ordering. Livelock → randomized backoff; starvation → fairness; priority inversion → priority inheritance (Mars Pathfinder, 1997).
- IPC: pipes, sockets, shared memory (fastest), message queues, signals (handle `SIGTERM`).
- I/O: blocking needs a thread per connection; `epoll` / `kqueue` readiness notification solved C10K; `io_uring` adds batched completion-based I/O. An event loop must never block.
- `write()` is not durable. Use `fsync` on the file *and* its directory, treat `fsync` errors as fatal, and batch commits (group commit $X = B/t_{\text{fsync}}$).
- Thread-per-request is simple but costs ~100 KB per connection; event loops, goroutines, virtual threads and async/await make waiting cheap. Size pools

> with Little’s Law and bound them.

6.17 Knowledge checkpoint

1. What is the difference between a program, a process and a thread?
2. Name five pieces of information stored in a PCB.
3. Why does a process go from Blocked to Ready rather than directly to Running?
4. Distinguish the direct and indirect costs of a context switch.
5. Three jobs of 100, 10 and 10 ms arrive together. What is the average turnaround time under FCFS (in that order) and under SJF?
6. How does a CPU limit in a container cause latency spikes on an idle host?
7. Calculate the TLB reach of 2,048 entries with 4 KiB pages and with 2 MiB pages.
8. What is the difference between a minor and a major page fault?
9. Why must a condition-variable wait be inside a `while` loop?
10. Describe the ABA problem in one or two sentences and give one remedy.
11. List the four Coffman conditions and the most practical way to break one.
12. Why does `epoll` not help with reading regular files?
13. List the three steps to durably replace a file and the consequence of skipping each.
14. A service handles 1,000 requests/s, each holding a thread for 250 ms. How many threads are busy on average?

Answers

1. A program is an executable file; a process is a running instance with its own address space and resources; a thread is one execution context (registers, stack) inside a process, sharing its memory.
2. Any five of: PID and parent PID, state, saved registers and program counter, scheduling priority and accounting, page-table pointer and memory map, open file table, signal handlers, credentials, resource limits.
3. The event that unblocked it does not mean a CPU is free; it must be chosen by the scheduler like any other ready task.
4. Direct: kernel work to save and restore state and choose the next task (~1–5 µs). Indirect: slower execution afterwards because caches and TLB hold the previous thread’s state.
5. FCFS: completions 100, 110, 120 → average 110 ms. SJF: 10, 20, 120 → average 50 ms.
6. CFS bandwidth control gives the cgroup a quota per period (default 100 ms). Several threads exhaust it early and the cgroup is throttled for the rest of the period, delaying requests regardless of host load.
7. 2,048 × 4 KiB = 8 MiB; 2,048 × 2 MiB = 4 GiB.

8. A minor fault needs no I/O (page already in RAM, or a fresh zero page); a major fault reads from storage and blocks the thread for 100 µs–10 ms.
9. Under Mesa semantics another thread may change the state before the woken waiter reacquires the lock, and spurious wake-ups are allowed.
10. A CAS succeeds because a location again holds value A, although it changed to B and back in between, so the CAS acts on stale assumptions. Remedies: version-tagged pointers, hazard pointers, epoch-based reclamation.
11. Mutual exclusion, hold and wait, no preemption, circular wait. Break circular wait with a global lock order.
12. Regular files are always reported ready, yet reads can block on disk. Use a thread pool or `io_uring`.
13. `fsync` the temporary file (else the new name may point at empty data), rename atomically (else readers may see a partial file), `fsync` the directory (else the old file may reappear after a crash).
14. Little's Law: 1,000 × 0.25 = 250 threads.

6.18 Interview questions

Basic

Q1. What is the difference between a process and a thread, and when would you choose processes? Answer. A process has its own address space; threads share their process's memory and file descriptors but have their own stacks and registers. Threads communicate cheaply and switch faster; processes isolate faults. Choose processes for isolation (untrusted code, non-thread-safe libraries, PostgreSQL backends) or to bypass a GIL; threads for low-latency sharing of large in-memory state.

Q2. What is a race condition? Give an example and two fixes. Answer. A bug whose outcome depends on the interleaving of concurrent operations; e.g. two threads both read 5 and both write 6. Fixes: a mutex around the read-modify-write, or one atomic operation (atomic add, CAS loop); across services, transactions or conditional writes. Tests rarely reveal races, so use race detectors.

Q3. What is a deadlock and how do you prevent it in application code? Answer. A cycle of threads each waiting for a resource held by the next, requiring all four Coffman conditions. Acquire locks in one global order, never hold locks across I/O or unknown call-backs, use `tryLock` with timeouts, and prefer single-owner state or message passing. Detect with thread dumps.

Intermediate

Q4. Your service's p99 latency doubled after you increased the worker pool from 64 to 2,000 threads on a 16-core machine. Why might that be? Answer. If the work is CPU-bound, 2,000 runnable threads on 16 cores each wait behind ~125 others; involuntary switches multiply and evict caches, so throughput falls while queueing grows, and lock contention rises. Check `vmstat` (cs, run queue), voluntary versus involuntary switches and lock profiles. Size the pool with Little's Law and the CPU/wait ratio, with a bounded queue and load shedding. If the work is I/O-bound, async or virtual threads are cheaper than a huge pool.

Q5. Explain how an event loop serves 10,000 connections on one thread, and what breaks it. Answer. Non-blocking sockets are registered with `epoll` / `kqueue`; the loop sleeps in one wait call and runs short handlers for the ready sockets, so a waiting connection costs a small state object, not a thread. It breaks when a handler blocks or burns CPU (synchronous DB driver, large JSON parse, file I/O), stalling every connection. Mitigations: worker pools, one loop per core, event-loop-lag monitoring.

Q6. A database acknowledges writes after `write()` but before `fsync()`. What can go wrong, and how do real databases make commits durable but fast? Answer. A crash loses data still in the page cache, so acknowledged commits vanish. Durable systems append to a write-ahead log and `fdatasync` before acknowledging, using group commit to amortize flushes ($X = B/t_{\text{fsync}}$); they `fsync` directories after creating files, treat `fsync` errors as fatal, and use drives with power-loss protection. Alternatively, replicate to other machines before acknowledging, trading flush latency for a network round trip and correlated-failure risk.

Advanced

Q7. Explain priority inversion with a real example and how to prevent it, including outside real-time systems. Answer. A high-priority task blocks on a lock held by a low-priority task, which a medium-priority task preempts, so the high task waits indefinitely. Mars Pathfinder (1997) reset repeatedly because a VxWorks mutex shared by a low-priority weather task and a high-priority bus task lacked priority inheritance; JPL enabled it remotely. Prevention: priority inheritance or ceiling locks, no locks shared across priority levels, short critical sections. In servers, isolate pools and resources so background jobs cannot hold what user traffic needs.

Q8. Compare epoll and io_uring. When would you adopt io_uring? Answer. `epoll` is readiness-based: it says a socket can be read, then a separate call reads it, and file reads can still block. `io_uring` is completion-based, with shared submission and completion rings, giving batching, asynchronous file I/O and, with SQPOLL, near-zero system calls. Adopt it for high-IOPS storage engines or proxies where syscall overhead is a mea-

sured bottleneck, accepting complexity, kernel-version dependence and security restrictions. For typical web services, `epoll` via a mature runtime is enough.

Q9. Design the concurrency model for a WebSocket gateway holding 1 million mostly idle connections across a fleet. Justify it with numbers. Answer. Thread-per-connection at ~100 KB each needs ~100 GB and a million kernel tasks: impractical. An event loop or goroutines at ~1–8 KB plus ~20 KB of socket and TLS buffers needs ~25–30 GB in total; spread it over, say, 10 nodes × 100,000 connections so one node loss affects 10 %. Run one loop per core (or Go with GOMAXPROCS = cores), raise file-descriptor limits, size socket buffers deliberately, push heavy work to other services via queues, and drain connections gradually on `SIGTERM` to avoid reconnect storms (§9.9).

6.19 Practical exercises

1. **Watch process states.** Run `sleep 100 &`, a CPU loop (`yes > /dev/null &`) and `cat` reading from a terminal. Use `ps -o pid,stat,wchan:20,cmd` to identify R and S states. Kill the parent of a child that has exited without being reaped and observe the zombie.
2. **Measure a context switch.** Run the following ping-pong program, once normally and once pinned to one core with `taskset -c 0`. Explain the difference. On one test VM for this chapter, it gave about 24 µs per round trip unpinned and about 4 µs pinned, because cross-core wake-ups add inter-processor interrupts and idle-state exits.

```
    """Two processes bounce one byte over a pair of pipes (POSIX only: uses fork).

    Each round trip forces at least two blocking reads and wake-ups, so the
    time per round trip upper-bounds the cost of two context switches plus
    four system calls and Python interpreter overhead.
    """
    import os
    import time

    ROUNDS = 100_000

    parent_to_child_r, parent_to_child_w = os.pipe()
    child_to_parent_r, child_to_parent_w = os.pipe()

    pid = os.fork()
    if pid == 0:                                    # child process
        for _ in range(ROUNDS):
            os.read(parent_to_child_r, 1)           # block until parent writes
            os.write(child_to_parent_w, b"x")       # wake the parent
        os._exit(0)

    start = time.perf_counter()
    for _ in range(ROUNDS):
        os.write(parent_to_child_w, b"x")           # wake the child
        os.read(child_to_parent_r, 1)               # block until child answers
    elapsed = time.perf_counter() - start
    os.waitpid(pid, 0)
    print(f"{elapsed / ROUNDS * 1e6:.1f} us per round trip")
```

3. **Find the race.** Run the Go program in §6.9 with `go run -race .`. Then write a version that uses a channel owned by one goroutine instead of a mutex, and confirm the race detector is silent.
4. **Break the event loop.** Add `time.sleep(2)` inside `handle()` in the echo server of §6.13.4, connect three clients, and measure how long the third waits. Fix it by moving the slow work to a `concurrent.futures.ThreadPoolExecutor`.
5. **Count system calls.** Run `strace -c python3 -c "open('/etc/hostname').read()"` and then the same for a program that copies a 100 MB file using 1-byte reads versus 64 KiB reads. Relate the time difference to §6.7.
6. **Durability check.** Write a program that appends 10,000 small records with and without `fsync` after each, and with one `fsync` per 100 records. Compute the commits per second and compare with the group-commit formula in §6.14.3.

6.20 Design challenges

1. **Log-ingestion agent.** Design a host agent that tails 500 log files, batches lines, and ships them to a remote collector at 50 MB/s, surviving agent restarts without losing or duplicating more than one batch. Decide: threads versus event loop versus `io_uring`; how offsets are made durable (`fsync` strategy and cost); how backpres-

sure works when the collector is slow; how the agent behaves when the disk fills. Justify each choice with numbers from this chapter.

2. **Multi-tenant job runner.** Design a single-machine runner for untrusted user jobs that guarantees each tenant a fair CPU share and a memory cap, and kills runaway jobs without affecting others. Discuss processes versus threads, cgroup CPU weights versus quotas (and throttling), OOM behavior, signals for graceful termination, and how to prevent a tenant from starving others through a shared lock or file system. Relate to §47.
3. **In-memory rate limiter.** Design a per-API-key rate limiter inside a service handling 200,000 requests/s on 32 cores. Compare a global mutex-protected map, a sharded map with per-shard locks, per-core counters merged periodically, and atomic CAS on token buckets. Estimate contention, explain where false sharing appears (§5.4), and state which you would ship and why (see §22.4).

6.21 References

1. Arpaci-Dusseau, R. H., and Arpaci-Dusseau, A. C. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018 (v1.00; free online). https://ostep.org
2. Silberschatz, A., Galvin, P. B., and Gagne, G. *Operating System Concepts*, 10th ed. Wiley, 2018.
3. Tanenbaum, A. S., and Bos, H. *Modern Operating Systems*, 4th ed. Pearson, 2014.
4. Kegel, D. *The C10K problem*. Web page, first published 1999. http://www.kegel.com/c10k.html
5. Axboe, J. *Efficient IO with io_uring*. 2019. https://kernel.dk/io_uring.pdf
6. Linux man-pages project. `syscalls(2)`, `sched(7)`, `signal(7)`, `epoll(7)`, `fsync(2)`, `mmap(2)`, `futex(2)`, `io_uring(7)`, `pipe(7)`. https://man7.org/linux/man-pages/
7. Linux kernel documentation. *CFS Scheduler*; *EEVDF Scheduler*; *CFS Bandwidth Control*. https://docs.kernel.org/scheduler/
8. Stoica, I., and Abdel-Wahab, H. *Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation*. Technical Report TR-95-22, Old Dominion University, 1995.
9. Corbet, J. “An EEVDF CPU scheduler for Linux.” LWN.net, 2023. https://lwn.net/Articles/925371/
10. Gregg, B. *Systems Performance: Enterprise and the Cloud*, 2nd ed. Addison-Wesley, 2020.
11. Li, C., Ding, C., and Shen, K. “Quantifying the Cost of Context Switch.” *Workshop on Experimental Computer Science (ExpCS)*, 2007.
12. Franke, H., Russell, R., and Kirkwood, M. “Fuss, Futexes and Furwocks: Fast Userlevel Locking in Linux.” *Ottawa Linux Symposium*, 2002.
13. Dijkstra, E. W. *Cooperating Sequential Processes* (EWD 123). Technological University, Eindhoven, 1965.

14. Lampson, B. W., and Redell, D. D. “Experience with Processes and Monitors in Mesa.” *Communications of the ACM* 23(2), 1980.
15. Herlihy, M., and Shavit, N. *The Art of Multiprocessor Programming*, revised 1st ed. Morgan Kaufmann, 2012.
16. Treiber, R. K. *Systems Programming: Coping with Parallelism*. IBM Research Report RJ 5118, 1986.
17. Michael, M. M. “Hazard Pointers: Safe Memory Reclamation for Lock-Free Objects.” *IEEE Transactions on Parallel and Distributed Systems* 15(6), 2004.
18. Coffman, E. G., Elphick, M. J., and Shoshani, A. “System Deadlocks.” *ACM Computing Surveys* 3(2), 1971.
19. Sha, L., Rajkumar, R., and Lehoczky, J. P. “Priority Inheritance Protocols: An Approach to Real-Time Synchronization.” *IEEE Transactions on Computers* 39(9), 1990.
20. Reeves, G. E. “What really happened on Mars?” Email account, December 1997; reproduced at https://www.cs.cornell.edu/courses/cs614/1999sp/papers/pathfinder.html
21. Jones, M. B. “What really happened on Mars Rover Pathfinder.” *The RISKS Digest* 19(49), 1997.
22. Lemon, J. “Kqueue: A Generic and Scalable Event Notification Facility.” *USENIX Annual Technical Conference, FREENIX Track*, 2001.
23. Pillai, T. S., et al. “All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications.” *OSDI*, 2014.
24. Rebello, A., et al. “Can Applications Recover from fsync Failures?” *USENIX Annual Technical Conference*, 2020.
25. PostgreSQL Wiki. *Fsync Errors*. https://wiki.postgresql.org/wiki/Fsync_Errors
26. Crotty, A., Leis, V., and Pavlo, A. “Are You Sure You Want to Use MMAP in Your Database Management System?” *CIDR*, 2022.
27. Apache Kafka. *Documentation: Design — Persistence*. https://kafka.apache.org/documentation/#persistence
28. Redis. *Redis persistence* and *Diagnosing latency issues*. https://redis.io/docs/
29. PostgreSQL. *Documentation: Architectural Fundamentals*. https://www.postgresql.org/docs/current/tutorial-arch.html
30. MySQL. *MySQL 8.0 Reference Manual: Connection Interfaces*. https://dev.mysql.com/doc/
31. Pike, R. “Concurrency is not Parallelism.” Talk, Heroku Waza conference, 2012.
32. The Go Authors. *Data Race Detector* and *The Go Memory Model*. https://go.dev/doc/articles/race_detector, https://go.dev/ref/mem
33. Node.js. *Don’t Block the Event Loop (or the Worker Pool)*. https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop
34. Google Security Blog. “Learnings from kCTF VRP’s 42 Linux kernel exploits submissions.” 2023. https://security.googleblog.com/
35. Apple. `fsync(2)` macOS manual page.

36. Hewitt, C., Bishop, P., and Steiger, R. “A Universal Modular ACTOR Formalism for Artificial Intelligence.” *IJCAI*, 1973.
37. Armstrong, J. *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, Royal Institute of Technology (KTH), Stockholm, 2003.
38. Erlang/OTP. *Efficiency Guide: Processes*. https://www.erlang.org/doc/
39. WhatsApp. “1 million is so 2011.” WhatsApp Blog, January 2012.
40. OpenJDK. *JEP 444: Virtual Threads*. 2023. https://openjdk.org/jeps/444
41. Goetz, B., et al. *Java Concurrency in Practice*. Addison-Wesley, 2006.

6.22 Open verification items

- Exact LWN article URL and date for [Corbet 2023] (lwn.net was unreachable from the authoring environment). EEVDF merge in 6.6 and the 0.75 ms × (1 + log2 ncpus) base slice were confirmed against the v6.6 scheduler pull request and commit e4ec3318; whether later 6.x releases changed the default is not verified.
- Magnitude of indirect context-switch cost reported by [Li et al. 2007]; direct-cost range 1–5 µs on current hardware.
- Mars Pathfinder details (task names, watchdog mechanism, remote patch via onboard interpreter) against Reeves’s original text; Cornell mirror URL.
- Google Security Blog 2023 io_uring statistics and the exact scope of the restriction; full article URL.
- WhatsApp 2012 “2 million connections” blog post URL and wording.
- Erlang process initial size (“a few hundred words”) for the current OTP release; Python 3.14 free-threading status.
- Whether Dijkstra’s banker’s algorithm should be cited to EWD 123 or EWD 108.
- MySQL “Connection Interfaces” page title for thread-per-connection.
- TLB entry counts (1,000–3,000 second-level entries) for current server CPUs.

Chapter 7 — Networking Fundamentals

> **Level:** 1–2 · **Track:** Shared · **Estimated reading time:** 5 h

Every system in this book is a set of computers that talk to each other. A database replica talks to its primary, a browser talks to a load balancer, and a model server streams tokens to a phone. Whenever two computers talk, the network sets hard limits on how fast they can do it, how often it fails, and what it costs. Those limits come from physics (the speed of light), from economics (the cost of bandwidth), and from decades-old protocol decisions that you cannot change. You can design around them only if you understand them.

This chapter builds the network up from nothing. We start with the layered model that lets thousands of independently built protocols cooperate, then follow a single request down through addressing, routing, name resolution, transport, security and application protocols. We finish by tracing everything that happens when you type a URL into a browser. Throughout, the goal is quantitative intuition: by the end you should be able to estimate, with arithmetic, how long a request will take and why.

7.1 Learning objectives

After this chapter you should be able to:

- Explain the OSI and TCP/IP layer models and describe how **encapsulation** wraps application data in transport, network and link headers.
- Read and compute IPv4 and IPv6 CIDR blocks: network address, broadcast address, usable host count and whether an address falls in a subnet.
- Describe how packets are routed inside a network and between autonomous systems with BGP, and explain how anycast steers users to a nearby server.
- Draw the DNS resolution sequence (stub, recursive resolver, root, TLD, authoritative), name the common record types, and reason about TTLs and the limits of DNS-based load balancing.
- Explain TCP’s three-way handshake, four-way close, TIME_WAIT, sequence numbers, retransmission, flow control and congestion control (slow start, AIMD, CUBIC, BBR), and identify head-of-line blocking.
- Decide when UDP is the right transport and what you must rebuild on top of it.
- Compare HTTP/1.1, HTTP/2 and HTTP/3 (over QUIC) and justify a choice for a given workload.

- Sequence the TLS 1.3 handshake, explain certificate validation, and state the security caveats of 0-RTT.
- Distinguish forward proxies from reverse proxies, and explain NAT, firewalls and cloud security groups.
- Decompose latency into propagation, transmission, queuing and processing delay; compute the bandwidth-delay product; and apply the Mathis formula to estimate TCP throughput under loss.
- Quantify the cost of connection setup and size a connection pool.
- Describe leaf–spine (Clos) datacenter fabrics and why east–west traffic dominates modern datacenters.

7.2 Prerequisites

- Units: bits vs bytes, SI prefixes, rates — §4.5 (every calculation here converts between bits per second and bytes).
- Percentiles and tail latency — §4.8 (network latency is heavy-tailed; averages mislead).
- Little's Law — §4.11 (used to size connection pools in §7.16).
- Latency numbers every engineer should know — §4.17 (this chapter derives several of them).
- Processes, sockets as file descriptors, and I/O models (blocking, epoll) — §6.7 and §6.13 (a TCP connection is a kernel object your process reads and writes).

No prior networking knowledge is assumed.

7.3 OSI and TCP/IP models

7.3.1 The postal analogy

Imagine you want to send a letter to a friend in another country. You write the letter (the content). You put it in an envelope and write your friend's name on it. The post office puts your envelope into a mail sack addressed to a sorting center in the destination city. The mail sack goes into a truck, then a plane, then another truck. At each sorting center, a worker reads only the sack label, not your letter, and decides where it goes next. At the far end, the sack is opened, the envelope is delivered to the right house, and your friend opens it and reads the letter.

Three ideas in this story are the foundation of all computer networking:

1. **Layering.** Each participant does one job and trusts the others to do theirs. The letter writer does not care whether the mail flies or sails. The truck driver does not read letters.

2. **Encapsulation.** Each layer wraps the thing it was given in its own envelope with its own address, and the matching layer at the far end unwraps it.
3. **Hop-by-hop forwarding.** Intermediate sorting centers look only at the outer address and forward toward the destination. They do not need the whole route in advance.

Computer networks work the same way. Your application writes bytes. The operating system wraps them in a **segment** with port numbers (which program on the machine), then in a **packet** with IP addresses (which machine on the planet), then in a **frame** with hardware addresses (which device on this particular wire). Routers along the way read the packet header and forward it, and the receiving machine unwraps each layer in reverse.

DEFINITION

Definition — Protocol. A set of rules two parties agree on for exchanging messages: message formats, the meaning of each field, and the actions each side takes on sending, receiving or timing out. A **protocol stack** is a set of protocols layered so that each uses the service provided by the layer below and offers a service to the layer above.

DEFINITION

Definition — Encapsulation. The process by which each layer of the stack prepends its own header (and sometimes appends a trailer) to the data unit handed down by the layer above, treating that data unit as an opaque payload. **Decapsulation** is the reverse at the receiver.

Why does this exist?

In the 1970s, every computer vendor had its own networking scheme, and machines from different vendors could not talk. Layering solved two problems. First, it made **interoperability** possible: if two vendors implement the same protocol at a layer, their machines can communicate regardless of what is above or below. Second, it made **independent evolution** possible: Wi-Fi replaced Ethernet cables at the bottom and HTTP/3 replaced HTTP/1.1 at the top, while IP in the middle stayed the same. This “narrow waist” design, with many link technologies below and many applications above one common Internet Protocol, is the main reason the Internet scaled from a few dozen hosts in 1970 to billions of devices.

7.3.2 The two models

There are two reference models you will hear about. The **OSI model** (Open Systems Interconnection), standardized by ISO in the early 1980s, has seven layers. It is a teach-

ing vocabulary; the OSI protocol suite itself lost to TCP/IP in the market. The **TCP/IP model** (also called the Internet model) has four or five layers and describes the protocols the Internet actually runs. Engineers mix the vocabularies freely: “an L4 load balancer” uses the OSI number for the transport layer, and “an L7 proxy” uses the OSI number for the application layer.

Table 7.1 — OSI and TCP/IP layers compared.

OSI layer	TCP/IP layer	Job	Example protocols	Data unit (PDU)	Address used
7 Application	Application	Meaning of messages for a specific application	HTTP, DNS, SMTP, gRPC	Message	URL, hostname
6 Presentation	(folded into application)	Encoding, compression, encryption	TLS*, JSON, Protobuf	—	—
5 Session	(folded into application)	Dialogue management, resumption	TLS sessions*, RPC sessions	—	—
4 Transport	Transport	Process-to-process delivery; reliability; flow and congestion control	TCP, UDP, QUIC**	Segment (TCP), datagram (UDP)	Port number
3 Network	Internet	Host-to-host delivery across many networks	IPv4, IPv6, ICMP	Packet	IP address
2 Data link	Link	Delivery across one physical network segment	Ethernet, Wi-Fi (802.11)	Frame	MAC address
1 Physical	Link (or separate physical layer)	Bits as voltages, light or radio	Copper, fiber, radio	Bit/symbol	—

*TLS does not fit cleanly: it runs on top of TCP but below HTTP, so people place it at 5, 6 or “4.5”. **QUIC runs on top of UDP yet provides transport services (reliability, streams, congestion control), so it is a transport protocol implemented in user space.

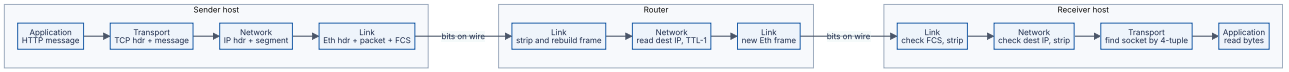
Do not memorize the OSI layers 5 and 6 as separate things that exist in real stacks; they do not. What matters in practice is the five-layer view: physical, link, network, transport, application. This is the organizing structure of the standard textbook treatment [Kurose and Ross 2021], which is the recommended companion for this chapter.

7.3.3 How encapsulation works internally

Follow one HTTP request from a browser to a web server.

1. **Application.** The browser produces the bytes `GET /index.html HTTP/1.1\r\nHost: example.com\r\n\r\n` (about 40 bytes) and calls `write()` on a socket.
2. **Transport.** The kernel's TCP implementation copies the bytes into the socket's send buffer, splits them into segments no larger than the **maximum segment size (MSS)**, and prepends a 20-byte TCP header (up to 60 bytes with options) containing source port (say 52,814), destination port 443, a sequence number, an acknowledgment number, flags and a window size.
3. **Network.** IP prepends a 20-byte IPv4 header (or a 40-byte IPv6 header) containing source and destination IP addresses, a time-to-live (TTL, renamed "hop limit" in IPv6) counter, and the protocol number of the payload (6 for TCP, 17 for UDP).
4. **Link.** The network interface driver looks up the MAC address of the next hop (the local router) using ARP (IPv4) or Neighbor Discovery (IPv6), and prepends a 14-byte Ethernet header (destination MAC, source MAC, EtherType) and appends a 4-byte frame check sequence (a CRC for error detection).
5. **Physical.** The network card serializes the frame onto the wire as electrical or optical symbols.
6. **Each router** receives the frame, checks the CRC, strips the Ethernet header, reads the IP destination, decrements the TTL, looks up the next hop, and wraps the packet in a *new* link-layer frame for the next link. The IP packet travels end to end; link-layer frames live for one hop only.
7. **The server** decapsulates in reverse: the NIC checks the frame, IP checks the destination address, TCP uses the (source IP, source port, destination IP, destination port) **4-tuple** to find the right socket, and the application reads the original bytes.

Figure 7.1 — Encapsulation down the sender's stack and decapsulation up the receiver's.



Reading the diagram. The left group is the sending host; data flows down its stack, gaining one header per layer. The middle group is a single router: it operates only up to the network layer, stripping the incoming Ethernet frame, reading and modifying the IP header (decrementing TTL), and building a fresh frame for the outgoing link. The right group is the receiving host, where data flows up and each layer removes its own header. All arrows are synchronous hand-offs within the same packet's journey; nothing here is queued asynchronously except inside each device's transmit buffer, which is a failure point (buffers overflow and drop packets, see §7.15). Storage boundaries are absent: the network stores nothing durably. To scale, you add links and routers; the layered design means hosts need not change.

Mathematical foundations: header overhead

FORMULA

Protocol efficiency (goodput fraction)

$$\eta = \frac{P}{P + H}$$

Symbol	Meaning	Units
η	Fraction of transmitted bytes that are application payload	dimensionless (0–1)
P	Application payload bytes per frame	bytes
H	Total per-frame overhead from all layers (headers, trailers, and on Ethernet the preamble and inter-frame gap)	bytes

WORKED EXAMPLE

Worked example — How much of a gigabit link carries your data?

A standard Ethernet **maximum transmission unit (MTU)** is 1,500 bytes: the largest IP packet a frame can carry. Inside that, IPv4 uses 20 bytes and TCP 20 bytes, plus 12 bytes for the commonly enabled TCP timestamp option.

1. TCP payload per packet: $1500 - 20 - 20 - 12 = 1448$ bytes.
2. On-the-wire cost per frame: 1,500 (packet) + 14 (Ethernet header) + 4 (FCS) + 8 (preamble) + 12 (inter-frame gap) = 1,538 bytes.
3. Efficiency: $\eta = 1448/1538 \approx 0.941$.
4. Maximum TCP goodput on 1 Gbps Ethernet: $0.941 \times 1000 \approx 941$ Mbps.

Interpretation: about 6% of a full-size frame is overhead. Now consider a tiny 40-byte request carried in one packet: the wire cost is $40 + 20 + 32 + 38 = 130$ bytes, so efficiency is $40/130 \approx 31\%$.

Engineering implication: small messages are dominated by header cost (and, more importantly, by per-packet processing cost in CPUs and switches). This is one reason systems **batch** small writes (§23.8) and why the 941 Mbps "real" ceiling of gigabit Ethernet is a well-known measurement in `iperf` tests.

Common misconceptions

COMMON MISTAKE

“Layer 7 means HTTP and Layer 4 means TCP, so a Layer 4 device cannot see anything about HTTP.” Roughly right, but the numbers are labels for *what a device inspects*, not for physical boxes. A “Layer 4 load balancer” makes decisions using IP addresses and ports; a “Layer 7 load balancer” parses HTTP and can route on path or header. Both receive the full packet (§19.3).

7.4 IP addressing, IPv4/IPv6, subnets and CIDR

7.4.1 What an IP address is

DEFINITION

Definition — IP address. A numeric identifier assigned to a network interface that the Internet Protocol uses to deliver packets. An **IPv4** address is 32 bits, written as four decimal octets (`203.0.113.25`) [RFC 791]. An **IPv6** address is 128 bits, written as eight groups of four hexadecimal digits separated by colons (`2001:db8:0:1::25`) [RFC 8200].

First-principles explanation

A postal address is hierarchical: country, city, street, house number. A sorting center in another country only needs to read the country to forward the letter; only the local post office reads the house number. IP addresses are hierarchical in the same way, but the split point is flexible. The leading bits form the **network prefix** (which network), and the trailing bits identify the **host** within that network. Routers far away only need the prefix.

Why does this exist?

Routers must decide where to send every packet in nanoseconds. If they had to store a route for every one of billions of devices, their tables would not fit in fast memory. Grouping addresses into prefixes lets a router store one entry for millions of hosts. The original 1981 scheme fixed the split at 8, 16 or 24 bits (“classes A, B, C”), which wasted addresses: an organization needing 300 addresses had to take a class B with 65,536. **Classless Inter-Domain Routing (CIDR)**, introduced in 1993 and specified today in RFC 4632, allows the split at any bit.

7.4.2 CIDR notation and subnet arithmetic

DEFINITION

Definition — CIDR block. An address range written as `address/prefix-length`, for example `192.168.10.64/26`. The prefix length n says that the first n bits identify the network; the remaining $32 - n$ bits (IPv4) or $128 - n$ bits (IPv6) identify hosts. The **subnet mask** is the same information as a dotted number with n one-bits followed by zeros (`/26` = `255.255.255.192`).

FORMULA

Addresses in a CIDR block

$$N_{\text{addr}} = 2^{W - n} \qquad N_{\text{hosts,IPv4}} = 2^{32 - n} - 2$$

Symbol	Meaning	Units
N_{addr}	Total addresses in the block	addresses
W	Address width: 32 for IPv4, 128 for IPv6	bits
n	Prefix length	bits
$N_{\text{hosts,IPv4}}$	Assignable IPv4 hosts on a traditional subnet: subtract the **network address** (all host bits 0) and the **broadcast address** (all host bits 1)	addresses

The "−2" applies to ordinary IPv4 LAN subnets. Point-to-point `/31` links (RFC 3021) and single-host `/32` routes are exceptions, and cloud providers reserve more: AWS reserves five addresses in every VPC subnet (network, VPC router, DNS, one reserved for future use, broadcast), as of 2025 [AWS Docs: VPC subnets].

WORKED EXAMPLE

Worked example 1 — Which subnet is 192.168.10.77/26 in?

1. A `/26` leaves $32 - 26 = 6$ host bits, so each subnet holds $2^6 = 64$ addresses.
2. The mask is 26 one-bits: `11111111.11111111.11111111.11000000` = `255.255.255.192`.
3. Only the last octet is split. Subnets start at multiples of 64 in that octet: .0, .64, .128, .192.
4. 77 lies between 64 and 127, so the network address is **192.168.10.64**.
5. Broadcast address: last address of the block = 64 + 64 − 1 = **192.168.10.127**.
6. Usable hosts: **192.168.10.65 – 192.168.10.126**, which is $64 - 2 = 62$ hosts.

Bitwise check: 77 = `01001101`; AND with mask `11000000` gives `01000000` = 64. Correct.

WORKED EXAMPLE

Worked example 2 — Carving a cloud VPC into subnets.

You are given the VPC range `10.0.0.0/16` and want equal subnets across three availability zones, each with public and private tiers (six subnets), leaving room to grow.

1. The VPC has $2^{32-16} = 65,536$ addresses.
2. Choose `/20` subnets: $2^{20-16} = 16$ subnets of $2^{12} = 4,096$ addresses each. Six are used now; ten remain for growth.
3. With `/20`, the third octet advances in steps of $4096/256 = 16$: `10.0.0.0/20`, `10.0.16.0/20`, `10.0.32.0/20`, … , `10.0.240.0/20`.
4. Usable per subnet on AWS: $4096 - 5 = 4,091$.
5. Membership test: is `10.0.37.200` in `10.0.32.0/20`? The block covers third-octet values 32–47 (32 + 16 − 1). 37 is in that range, so **yes**; its range is `10.0.32.0` – `10.0.47.255`.

Interpretation: a `/20` per subnet sounds generous, but container platforms that give every pod its own VPC address (for example, the AWS VPC CNI for Kubernetes) can consume thousands of addresses per subnet. Running out of IPs in a subnet is a common production incident; size for pods, not for nodes (Ch 42).

WORKED EXAMPLE

Worked example 3 — Do two networks overlap?

Team A uses `10.0.0.0/16` (10.0.0.0–10.0.255.255). Team B uses `10.0.128.0/17` (10.0.128.0–10.0.255.255). B's range lies entirely inside A's, so the two VPCs **overlap** and cannot be peered or connected with simple routing: a packet to 10.0.200.1 would be ambiguous. Engineering implication: allocate non-overlapping RFC 1918 ranges centrally (an IP address management plan) *before* teams create networks; renumbering later is painful.

Table 7.2 — CIDR quick reference (IPv4).

Prefix	Mask	Addresses	Typical use
/8	255.0.0.0	16,777,216	Large private network (10.0.0.0/8)
/16	255.255.0.0	65,536	A cloud VPC
/20	255.255.240.0	4,096	A large subnet per availability zone
/24	255.255.255.0	256	Classic LAN; smallest prefix widely accepted in global BGP
/26	255.255.255.192	64	Small subnet
/30	255.255.255.252	4	Legacy point-to-point link (2 usable)
/31	255.255.255.254	2	Point-to-point link (RFC 3021, both usable)
/32	255.255.255.255	1	A single host route

7.4.3 Private and special-purpose ranges

Some ranges are reserved and never routed on the public Internet. The most important are the **private** ranges of RFC 1918, which any organization may reuse internally; they reach the Internet only through NAT (§7.13).

Table 7.3 — Special-purpose address ranges.

Range	Size	Purpose
10.0.0.0/8	16,777,216	Private (RFC 1918)
172.16.0.0/12	1,048,576	Private (RFC 1918): 172.16.0.0–172.31.255.255
192.168.0.0/16	65,536	Private (RFC 1918), common in homes
100.64.0.0/10	4,194,304	Shared address space for carrier-grade NAT (RFC 6598)
127.0.0.0/8	16,777,216	Loopback (`127.0.0.1` = this machine)
169.254.0.0/16	65,536	Link-local; cloud instance metadata services commonly live at `169.254.169.254`
192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24	256 each	Documentation examples (RFC 5737); used in this book
`::1/128`	1	IPv6 loopback
`fe80::/10`	—	IPv6 link-local
`fc00::/7`	—	IPv6 unique local addresses (RFC 4193), the IPv6 analog of private space
`2000::/3`	—	IPv6 global unicast
`2001:db8::/32`	—	IPv6 documentation (RFC 3849)

7.4.4 IPv4 exhaustion and IPv6

IPv4 offers $2^{32} \approx 4.29$ billion addresses, fewer than the number of connected devices. The regional Internet registries ran out of freely allocatable IPv4 space during the 2010s, and IPv4 addresses are now bought and sold. Two responses coexist: NAT (§7.13), which lets many devices share one public address, and IPv6, which offers $2^{128} \approx 3.4 \times 10^{38}$ addresses.

IPv6 makes several other changes [RFC 8200]:

- **Fixed 40-byte header** with optional extension headers, simpler for hardware to parse.
- **No fragmentation by routers.** Only the sender fragments; routers send an ICMPv6 “Packet Too Big” message instead, so path MTU discovery is mandatory.
- **No header checksum** (the link and transport layers already check).
- **Standard /64 subnets.** A single LAN gets 2^{64} addresses, enough for hosts to self-assign addresses (SLAAC). A typical site receives a `/48`, which holds $2^{64-48} = 65,536$ `/64` subnets.
- **Notation shortcuts.** Leading zeros in a group may be dropped, and one run of all-zero groups may be replaced by `::`. So `2001:0db8:0000:0000:0000:0000:0000:0025` is

`2001:db8::25`.

Clients with both IPv4 and IPv6 use **Happy Eyeballs** (RFC 8305): they start connecting over IPv6 and, if that has not succeeded within a short delay (around 250 ms is recommended), race an IPv4 attempt, using whichever connects first. This prevents broken IPv6 paths from adding seconds of delay.

COMMON MISTAKE

"Private addresses are a security feature." RFC 1918 ranges are not routed on the Internet, which incidentally makes hosts unreachable from outside without NAT rules. But reachability is not access control: a compromised host inside the network reaches them freely. Use firewalls and authentication (§7.14, §10.10), not address choice, for security.

7.5 Routing basics, BGP and anycast

7.5.1 Forwarding and longest-prefix match

DEFINITION

Definition — Routing and forwarding. Forwarding is the per-packet act of choosing an outgoing interface for a packet based on its destination address and the **forwarding table**. **Routing** is the control-plane process of building those tables by exchanging reachability information with other routers.

Each router holds a table of prefixes and next hops. When a packet arrives, the router selects the **most specific** (longest) prefix that contains the destination. This rule is called **longest-prefix match**.

WORKED EXAMPLE

Worked example — Longest-prefix match.

A router has these entries:

Prefix	Next hop
0.0.0.0/0 (default route)	ISP uplink
10.0.0.0/8	Core router A
10.1.0.0/16	Core router B
10.1.2.0/24	Access switch C

- Destination `10.1.2.3` matches /0, /8, /16 and /24. Longest is /24, so it goes to **C**.
- `10.1.9.9` matches /0, /8, /16: goes to **B**.
- `10.200.0.1` matches /0 and /8: goes to **A**.
- `8.8.8.8` matches only /0: goes to the **ISP uplink**.

Engineering implication: longest-prefix match lets operators override broad routes with specific ones. It also lets an attacker or a misconfiguration hijack traffic by announcing a *more specific* prefix than the legitimate owner, as the industry example below shows.

When several equally good next hops exist, routers use **equal-cost multi-path (ECMP)** routing: they hash each packet's flow identifiers (the 5-tuple of protocol, source and destination address and port) to pick one path. All packets of one flow take the same path, which avoids reordering, while different flows spread across paths. ECMP is the foundation of datacenter fabrics (§7.17) and of horizontally scaled load balancers (§19.8).

7.5.2 Interior and exterior routing

The Internet is a network of networks. Each independently operated network, such as an ISP, a cloud provider or a large enterprise, is an **autonomous system (AS)** identified by an AS number (ASN). Routing happens at two levels:

- **Interior gateway protocols** (OSPF, IS-IS) run *inside* an AS. They compute shortest paths using link costs, converge in seconds or less, and assume everyone is cooperative.
- **The Border Gateway Protocol (BGP)**, version 4 (RFC 4271), runs *between* ASes. It is the protocol that holds the Internet together.

DEFINITION

Definition — BGP. A **path-vector** routing protocol in which neighboring routers (peers) exchange, over a TCP session on port 179, announcements of the form "prefix P is reachable via AS path [X, Y, Z]". Each AS chooses among the paths it hears according to its own **policy**, then re-announces its chosen path (with its own ASN prepended) to the peers its policy allows.

How BGP works internally

1. Two routers in different ASes open a TCP connection on port 179 and exchange OPEN messages.
2. Each sends UPDATE messages announcing prefixes it can reach, with attributes including the **AS path** (the list of ASes traversed), the next hop, and preference hints.
3. Each router stores all received paths, then chooses one best path per prefix. The decision is policy-first: a network prefers routes learned from paying customers over routes from peers, and peers over upstream providers it pays. Only after policy does it prefer the shortest AS path, and then a series of tie-breakers.
4. The router installs the best path in its forwarding table and announces it onward, prepending its own ASN. Loops are prevented because a router rejects any path that already contains its own ASN.
5. When a route disappears, a WITHDRAW propagates. Convergence across the Internet can take seconds to minutes.

Two consequences matter for system designers. First, **BGP picks paths by business policy, not latency**: traffic between two cities may detour through a third. Second, **BGP trusts announcements by default**. The Resource Public Key Infrastructure (RPKI) lets prefix owners publish cryptographically signed statements of which ASN may originate their prefixes, and route-origin validation is increasingly deployed, but it is not universal.

7.5.3 Anycast

DEFINITION

Definition — Anycast. An addressing scheme in which the same IP prefix is announced via BGP from many locations at once. Each client's packets are routed to whichever announcing location is "closest" according to BGP path selection.

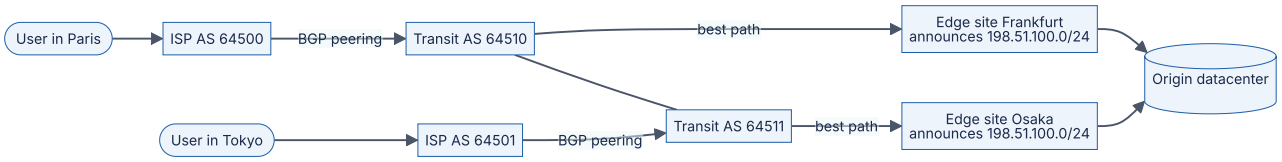
Analogy: a national emergency phone number connects you to your nearest dispatch center, not to a single central office. You dial the same number everywhere.

Anycast is how the DNS root servers, public resolvers such as `8.8.8.8` and `1.1.1.1`, and most CDNs (§21.3) put one address in front of hundreds of sites. Its strengths are auto-

matic proximity, natural absorption of distributed denial-of-service (DDoS) traffic across many sites (§10.11), and failover by **withdrawal**: if a site fails, it stops announcing the prefix, and BGP reroutes clients to the next-nearest site.

Its weaknesses follow from BGP. “Closest” means best by BGP policy, which usually but not always correlates with lowest latency. And because routing can change mid-connection, a long-lived TCP connection can suddenly have its packets delivered to a different site that has no state for it. For this reason anycast was first popular for short UDP exchanges such as DNS; operators that use it for TCP rely on route stability in practice and design for the occasional reset.

Figure 7.2 — BGP routing between autonomous systems, with an anycast prefix announced from two sites.



Reading the diagram. Two users (stadiums) reach their ISPs, which are separate autonomous systems connected to transit providers by BGP sessions. Two edge sites announce the *same* prefix, `198.51.100.0/24`. The Paris user’s ISP hears the shortest policy-preferred path via Frankfurt, and the Tokyo user’s ISP hears a path via Osaka, so the same destination address is served from different continents. Traffic flows synchronously left to right for requests (responses return along a path that may differ, because Internet routing is asymmetric). The origin datacenter is the storage boundary; the edges may cache (§21.4). Failure points: if Frankfurt withdraws its announcement, Paris traffic reconverges toward Osaka (or another site) within seconds to minutes, and in-flight TCP connections break. Scaling: add more announcing sites.

Real-world industry applications

INDUSTRY EXAMPLE

Facebook, 4 October 2021. During routine maintenance, a command intended to assess backbone capacity unintentionally disconnected Facebook’s backbone, and a bug in an audit tool failed to stop it. Facebook’s DNS servers are designed to withdraw their BGP anycast announcements when they cannot reach the datacenters, as a health safeguard. With the backbone gone, every DNS site withdrew at once, so `facebook.com` and related names could not be resolved anywhere. The outage lasted roughly six hours, and recovery was slowed because internal tools depended on the same network. Lesson: a health-driven withdrawal that is correct for one site can become a global outage when all sites share a failure; build in “never withdraw the last route” guards and out-of-band access. *Evidence: Primary* — [Janardhan 2021]

INDUSTRY EXAMPLE

YouTube prefix hijack, February 2008. Pakistan Telecom, intending to block YouTube domestically, announced a more-specific `/24` covering YouTube's address space. Its upstream provider propagated the announcement globally, and by longest-prefix match much of the Internet sent YouTube traffic to Pakistan for about two hours. The incident is a standard case study for BGP's trust model and motivates prefix filtering and RPKI. *Evidence: Secondary* — [RIPE NCC 2008]

Failure scenarios

Table 7.4 — Routing failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Route leak or hijack	A network announces prefixes it does not own, or re-announces routes against policy	External BGP monitors; sudden traffic shifts; latency jumps	Contact upstreams; announce more-specific prefixes	RPKI origin validation, prefix filters, max-prefix limits
Self-inflicted withdrawal	Health checks or automation withdraw all announcements simultaneously	Global reachability probes fail	Out-of-band access to re-announce	Guard rails preventing withdrawal of the last route; staged changes
Slow convergence	BGP path exploration after a failure	Elevated loss for seconds to minutes	Wait; fail over at a higher layer (DNS, client retry)	Multiple upstreams; BFD for fast failure detection
Anycast connection resets	Route change moves an in-flight TCP flow to another site	Spikes in TCP resets during route changes	Client retries	Short-lived connections; stable announcements

7.6 DNS: the Internet's directory

7.6.1 What DNS is and why it exists

DEFINITION

Definition — Domain Name System (DNS). A hierarchical, distributed database that maps human-readable **domain names** (such as `www.example.com`) to data, most often IP addresses, and the protocol used to query it [RFC 1034; RFC 1035].

First-principles explanation

People remember names; routers need numbers. In the early ARPANET, every host downloaded a single file, `HOSTS.TXT`, listing every name and address. As the network grew, that file became too large, changed too often, and depended on one central maintainer. DNS replaced it with a **delegated hierarchy**, much like a library catalog split across buildings: the front desk (the **root**) does not know where every book is, but it knows which building holds each subject (the **top-level domains** such as `.com`), and each building knows which room holds each shelf (the **authoritative servers** for `example.com`). Responsibility for each zone is delegated to whoever owns it, so no one party maintains everything.

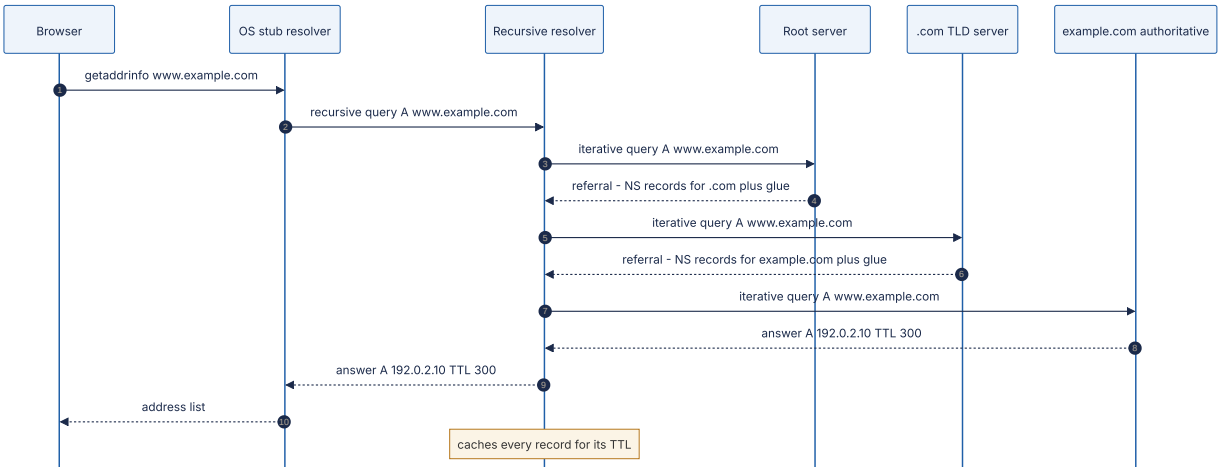
The cast of characters

- **Stub resolver.** A small library in your operating system. It does not do the work itself; it asks a recursive resolver and waits.
- **Recursive resolver** (also called a caching or full-service resolver). Run by your ISP, your company, or a public service such as `8.8.8.8` or `1.1.1.1`. It chases the answer through the hierarchy on your behalf and caches the results.
- **Root name servers.** Thirteen named server identities (`a.root-servers.net` through `m.root-servers.net`), each replicated via anycast (§7.5.3) to many physical instances worldwide. They know where each top-level domain's servers are.
- **TLD name servers.** Authoritative for a top-level domain such as `.com` or `.uk`. They know the name servers for each second-level domain.
- **Authoritative name servers.** Hold the actual records for a zone such as `example.com`. Managed DNS providers and cloud DNS services operate these for most companies.

7.6.2 Recursive versus iterative resolution

There are two query styles. In a **recursive query**, the client asks "give me the final answer", and the server must either return it or an error. Your stub resolver sends recursive queries to the recursive resolver. In an **iterative query**, the server returns the best information it has, often a **referral** ("I don't know, but ask these servers"), and the asker continues. Recursive resolvers send iterative queries to root, TLD and authoritative servers.

Figure 7.3 — DNS resolution of www.example.com with an empty cache.



Reading the diagram. Six participants appear from left to right: the application, the operating system's stub resolver, the recursive resolver, and three levels of authoritative servers. Steps 1–2 are the recursive part: the stub asks once and waits. Steps 3–8 are iterative: the resolver walks down the hierarchy, receiving referrals (dashed return arrows) from the root and the TLD, each containing the name-server (NS) records for the next level and **glue** records giving those servers' IP addresses. Step 8 is the final answer, here an A record with a 300-second TTL. All exchanges are synchronous request/response, normally over UDP port 53. The resolver's cache is the storage boundary: once populated, steps 3–8 are skipped for every client of that resolver until the TTL expires, and the root and TLD referrals are cached for much longer (typically days). Failure points: any server not responding forces a timeout and retry against another listed server (each zone must have at least two). Scaling comes from caching and from anycast replication of the root and TLD servers. (The address 192.0.2.10 is a documentation address used for illustration.)

Mathematical foundations: what a cold lookup costs

WORKED EXAMPLE

Worked example — Cold versus warm DNS lookup.

Assume the client is 5 ms from its recursive resolver, and the resolver is 20 ms from the root instance, 30 ms from the TLD server and 40 ms from the authoritative server (all round-trip times).

1. Cold cache: $5 + 20 + 30 + 40 = 95$ ms before the browser can even open a connection.
2. Root and TLD referrals cached (typical, since they have long TTLs) but the name itself expired: $5 + 40 = 45$ ms.
3. Fully warm at the resolver: 5 ms.
4. Cached in the operating system or browser: effectively 0 ms.

Interpretation: almost all real lookups hit case 3 or 4, which is why DNS feels free. The cold path matters for rarely visited domains, for very low TTLs, and for every new third-party domain a page references. Engineering implication: minimize the number of distinct hostnames on a critical path, and use `<link rel="dns-prefetch">` or `preconnect` hints for domains you know you will need.

7.6.3 Record types

Table 7.5 — Common DNS record types.

Type	Maps a name to	Example	Typical use
A	An IPv4 address	`www.example.com. 300 IN A 192.0.2.10`	Host address
AAAA	An IPv6 address	`www.example.com. 300 IN AAAA 2001:db8::10`	Host address over IPv6
CNAME	Another name (an alias)	`shop.example.com. 3600 IN CNAME shops.saas-provider.example.`	Point your name at a provider's hostname
MX	A mail server name, with priority	`example.com. 3600 IN MX 10 mail1.example.com.`	Email delivery (lower number = preferred)
NS	The authoritative name servers of a zone	`example.com. 86400 IN NS ns1.dnsprovider.example.`	Delegation
TXT	Arbitrary text	`example.com. 300 IN TXT "v=spf1 include:_spf.mailer.example -all"`	Domain ownership proofs, SPF, DKIM, DMARC
SRV	Host and port for a service, with priority and weight	`_sip._tcp.example.com. 300 IN SRV 10 60 5060 sip1.example.com.`	Service discovery (RFC 2782); used by SIP, XMPP, some service meshes
SOA	Zone metadata: primary server, serial, timers, negative-caching TTL	—	Zone administration
PTR	A name, for reverse lookup of an address	`10.2.0.192.in-addr.arpa. PTR www.example.com.`	Logs, mail server checks
CAA	Which certificate authorities may issue for the domain	`example.com. CAA 0 issue "letsencrypt.org"`	Restricting certificate issuance (§7.10)
HTTPS / SVCB	Connection parameters: ALPN protocols, ports, address hints	`example.com. HTTPS 1 . alpn="h3,h2"`	Advertising HTTP/3 before the first connection (RFC 9460)

Three rules trip people up. First, a **CNAME cannot coexist with other records at the same name**, so you cannot put a CNAME at the zone apex (`example.com` itself, which must hold SOA and NS records). DNS providers offer non-standard "ALIAS" or "CNAME flattening" features to work around this. Second, every record in a response carries its own TTL. Third, MX and SRV records point to *names*, never directly to addresses.

7.6.4 TTL and caching

DEFINITION

Definition — DNS time to live (TTL). A per-record value, in seconds, that tells any cache how long it may reuse the record before asking again. **Negative caching** stores "this name does not exist" (NXDOMAIN) answers for a duration derived from the zone's SOA record [RFC 2308].

Caches exist at several levels: the browser, the operating system, the recursive resolver, and sometimes in application runtimes. (Some runtimes have historically cached DNS answers far longer than the TTL, or forever, by default; the Java Virtual Machine's behavior depends on configuration and security manager settings, so check your runtime.)

TTL is a trade-off between **agility** (how quickly a change takes effect) and **load and latency** (how often resolvers must re-query). Short TTLs (30–60 s) let you fail over quickly but raise query volume and the fraction of slow cold lookups. Long TTLs (hours to a day) are efficient but make changes slow.

WORKED EXAMPLE

Worked example — How long does a DNS failover really take?

Your record has TTL 300 s, and your health checker takes 30 s (three failed probes, 10 s apart) to declare the primary site dead and update DNS.

1. Detection: 30 s.
2. Worst-case cache staleness: a resolver that refreshed one second before the update keeps serving the old answer for up to 300 s.
3. Worst-case total for well-behaved resolvers: $30 + 300 = 330$ s, about 5.5 minutes. The average client re-queries halfway through the TTL, so the mean is about $30 + 150 = 180$ s.
4. Clients holding open connections are not affected by DNS at all: they keep sending to the dead address until their connections fail and they reconnect.
5. Misbehaving caches that enforce a minimum TTL or ignore it extend the tail indefinitely.

Engineering implication: DNS failover is a *minutes-scale* mechanism. If you need seconds, put a stable address in front (an anycast virtual IP or a load balancer, §19.8) and fail over behind it.

Before a planned migration, lower the TTL well in advance. If the old TTL is 86,400 s (one day), you must lower it at least one day before the cut-over, because resolvers that cached the record just before you changed it will not re-query for a day.

7.6.5 DNS-based load balancing and its limits

Because a DNS answer can differ per query, DNS is the coarsest and cheapest load balancer:

- **Round-robin DNS.** Return several A records, rotating their order. Clients typically try the first.
- **Weighted answers.** Return each address in proportion to configured weights.
- **Geo-DNS / latency-based DNS.** Answer based on the resolver's (or, with the EDNS Client Subnet extension, the client's) location, pointing users to a nearby region (Ch 44, §19.9).
- **Health-checked DNS.** Remove addresses whose health checks fail.

The limits are fundamental, not implementation details:

1. **Caching defeats fine control.** A large resolver (an ISP serving millions of users) caches one answer and hands it to all its clients for the TTL, so "1/N of queries" is not "1/N of users".
2. **Resolver location is not client location.** Geo-DNS sees the recursive resolver's address. A user in Singapore using a resolver in the US may be sent to the US region, unless EDNS Client Subnet is supported.
3. **Failover speed is bounded by TTL** and by stale clients, as computed above.
4. **No load awareness.** DNS does not see connection counts or server latency; it picks before the connection exists.
5. **Clients behave inconsistently.** Some try only the first address; some try all; some pin forever.

DNS is therefore used to pick a **region or site**, and real load balancers distribute connections *within* the site (Chapter 19).

Transport details

DNS normally uses UDP port 53 because a query and a response each fit in one datagram, avoiding a handshake. Classic DNS limited UDP responses to 512 bytes; the EDNS(0) extension (RFC 6891) lets clients advertise larger buffers, and current operational guidance favors about 1,232 bytes to avoid IP fragmentation. If a response is too large, the server sets the truncation (TC) bit and the client retries over TCP. Zone transfers between servers use TCP. Encrypted variants exist: DNS over TLS (port 853, RFC 7858) and DNS over HTTPS (RFC 8484), which hide queries from on-path observers.

Failure scenarios

Table 7.6 — DNS failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Authoritative provider outage	DDoS or provider failure; all NS records at one provider	Resolution failures from external probes	Wait, or change NS at the registrar (slow: TLD TTLs are long)	Two independent DNS providers serving the same zone
Stale records after migration	TTL was long when you changed the record	Traffic still reaching old addresses	Keep old endpoint alive until TTL × 2 has passed	Lower TTL in advance of planned changes
Resolver overload / latency	Too many lookups per request; no local caching	High p99 on connection setup; resolver timeouts	Add local caching resolver	Cache in-process with TTL respect; reuse connections
Negative-cache trap	Name queried before it was created; NXDOMAIN cached	"Works for me, not for them"	Wait out SOA negative TTL	Create records before announcing names
Split-horizon mismatch	Internal and external views of a zone differ	Errors only inside or only outside the network	Fix the view	Test both views in deployment checks

COMMON MISTAKE

"Changing a DNS record takes 24–48 hours to propagate." DNS does not propagate; caches *expire*. A change is visible to a given resolver as soon as its cached copy's TTL runs out. The folklore "48 hours" comes from long TTLs, especially on NS records at the TLD.

7.7 TCP: reliable byte streams over an unreliable network

7.7.1 What TCP provides

IP is a **best-effort** service: packets may be lost, duplicated, delayed or delivered out of order, and IP will not tell you. Most applications want the opposite: a reliable, ordered pipe of bytes. The **Transmission Control Protocol** builds that pipe on top of IP [RFC 9293].

DEFINITION

Definition — TCP. A connection-oriented transport protocol that provides a reliable, ordered, full-duplex **byte stream** between two endpoints identified by (IP address, port) pairs. It achieves reliability with sequence numbers, acknowledgments and retransmission; it protects a slow receiver with **flow control**; and it protects the network with **congestion control**.

First-principles explanation

Imagine mailing a 300-page manuscript one page per envelope through an unreliable postal service. To make sure your editor gets every page in order, you would number each page, ask the editor to send back postcards saying “I have everything up to page 57”, and resend any page not acknowledged within a reasonable time. You would also avoid sending all 300 pages at once if the editor’s desk is small (flow control) or if the post office is visibly overwhelmed (congestion control). That is TCP. The one twist is that TCP numbers **bytes**, not pages: the stream has no message boundaries, so an application that writes “hello” and then “world” may receive “hellowor” and “ld”. Application protocols must add their own framing (a length prefix or a delimiter).

7.7.2 The segment

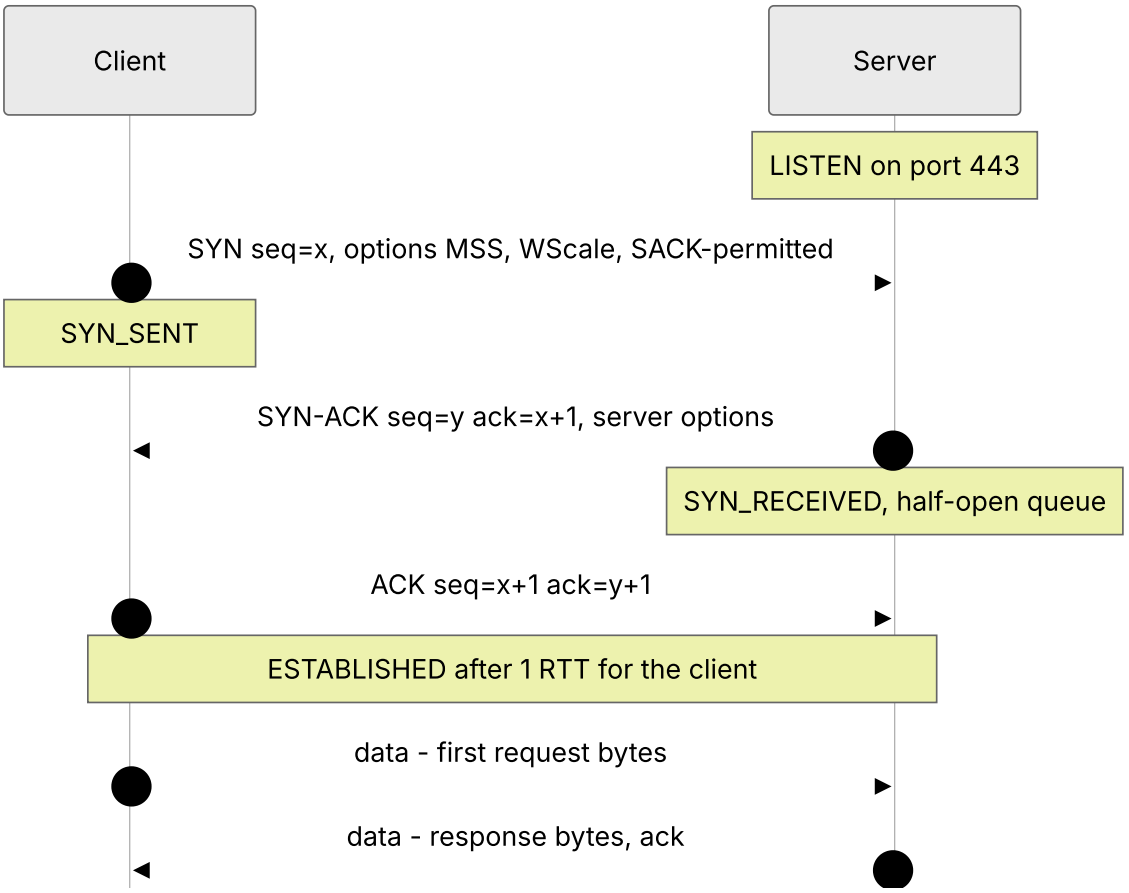
Every TCP segment carries a header of 20 bytes plus up to 40 bytes of options. The fields you will reason about most are:

- **Source and destination ports** (16 bits each). Together with the IP addresses they form the connection’s 4-tuple.
- **Sequence number** (32 bits). The stream offset of the first payload byte in this segment, relative to a random **initial sequence number (ISN)**.
- **Acknowledgment number** (32 bits). The next byte the sender of this segment expects to receive: “I have everything before this”.
- **Flags.** SYN (synchronize, opens a connection), ACK (acknowledgment field valid), FIN (no more data from me), RST (abort), PSH, URG, plus ECE/CWR for explicit congestion notification.
- **Window** (16 bits). How many more bytes the sender of this segment is willing to receive (flow control, §7.7.6).
- **Options.** MSS, window scale, selective acknowledgments (SACK) and timestamps, mostly negotiated in the handshake.

7.7.3 The three-way handshake

Before data flows, both sides must agree that the connection exists and learn each other’s initial sequence numbers. Each ISN is chosen randomly so that stray segments from an old connection, or an off-path attacker, are unlikely to fall into the valid window.

Figure 7.4 — TCP three-way handshake followed by the first request.



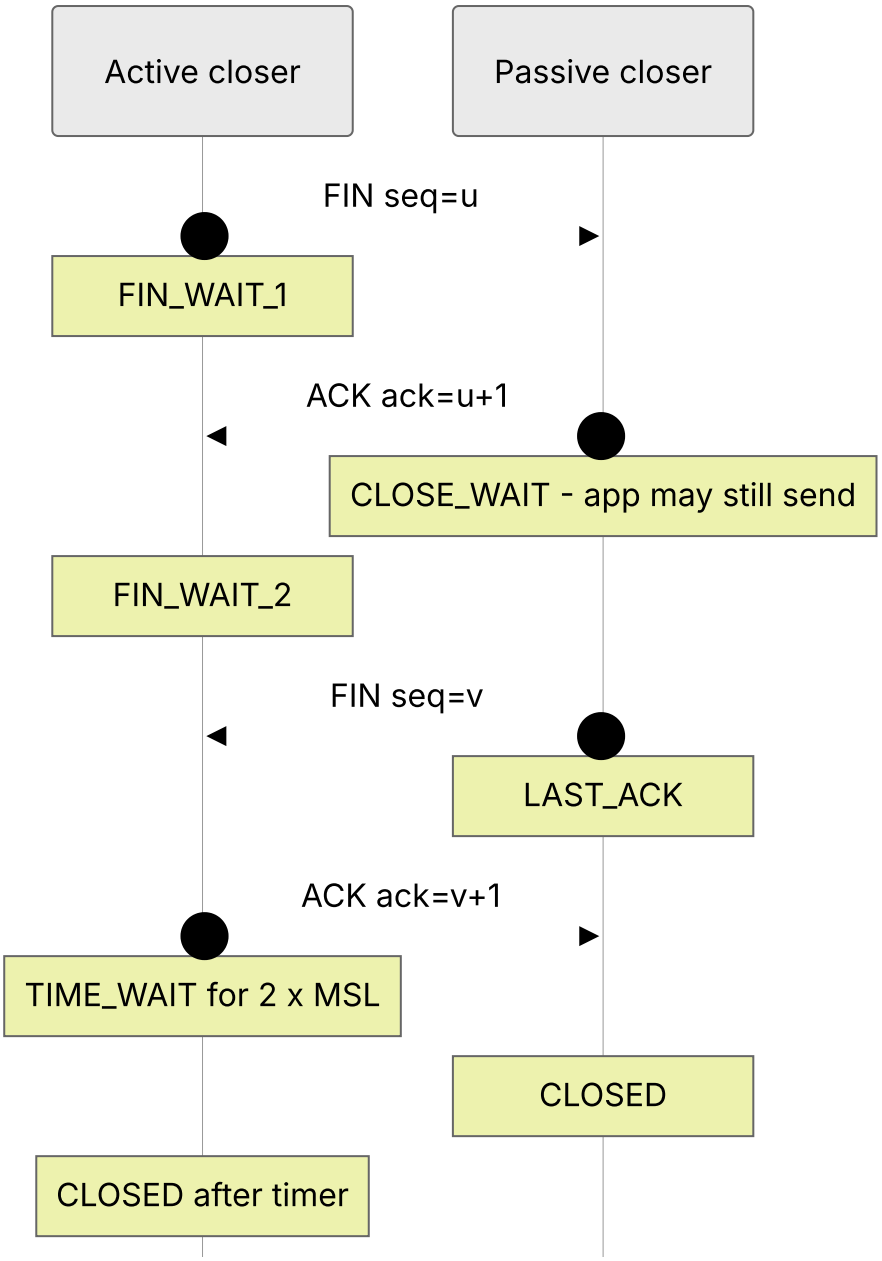
Reading the diagram. Two participants: the client, which actively opens, and the server, which is listening. Step 1 is the SYN, carrying the client's random ISN x and its option offers. Step 2, the SYN-ACK, acknowledges $x + 1$ (a SYN consumes one sequence number) and carries the server's own ISN y. Step 3 completes the handshake. The client may send data immediately after step 3, so the handshake costs the client **one round-trip time (RTT)** before its request can leave; the server sees the request 1.5 RTTs after the client began. All exchanges are synchronous. The server's half-open queue (connections between steps 2 and 3) is a failure point: a **SYN flood** attack fills it with spoofed SYNs, which servers counter with **SYN cookies**, encoding the connection state in the ISN instead of storing it. Scaling: the server's accept queue (`listen` backlog) must be sized for connection bursts.

TCP Fast Open (RFC 7413) lets a returning client put data in the SYN using a cookie from a previous connection, saving one RTT, but middlebox interference and replay concerns have limited its deployment. QUIC (§7.9.4) achieves the same goal more robustly.

7.7.4 The four-way close and TIME_WAIT

TCP is full-duplex, so each direction closes independently. Each side sends a FIN when it has no more data, and the other acknowledges it.

Figure 7.5 — TCP four-way close and the TIME_WAIT state.



Reading the diagram. The **active closer** is whichever side calls `close()` first; the other is the **passive closer**. Steps 1–2 close the active side's direction; the passive side enters CLOSE_WAIT and may keep sending until its application also closes (a pile-up of CLOSE_WAIT sockets is the classic symptom of an application that forgets to close connections). Steps 3–4 close the other direction. Steps 2 and 3 are often combined into one segment, making the close effectively three segments. The active closer then waits in **TIME_WAIT** for twice the **maximum segment lifetime (MSL)**. All steps are synchronous; RST is the abrupt alternative that skips all of this and discards unsent data.

TIME_WAIT exists for two reasons. First, if the final ACK (step 4) is lost, the passive side will retransmit its FIN, and the active side must still be around to re-acknowledge it. Second, it ensures that delayed segments from this connection have died out before a

new connection with the same 4-tuple can be created and misinterpret them. RFC 793 set MSL to two minutes; Linux hard-codes TIME_WAIT at 60 seconds.

WORKED EXAMPLE

Worked example — Ephemeral port exhaustion from TIME_WAIT.

A service opens a new TCP connection for every request to one backend (one destination IP and port) and closes it afterward, so the service is the active closer.

1. Each connection needs a unique source port. Linux's default ephemeral port range is 32,768–60,999: $60,999 - 32,768 + 1 = 28,232$ ports.
2. Each closed connection holds its port in TIME_WAIT for 60 s.
3. Sustainable new-connection rate to that single destination: $28,232/60 \approx 470$ connections per second.
4. At 1,000 requests per second with one connection per request, ports run out after about $28,232/1,000 \approx 28$ s, and `connect()` starts failing with `EADDRNOTAVAIL`.

Interpretation: TIME_WAIT is not a bug, but opening a connection per request is. Fixes, in order of preference: reuse connections with keep-alive and pooling (§7.16); add destination IPs or ports (each distinct 4-tuple has its own port space); widen the ephemeral range; enable `net.ipv4.tcp_tw_reuse` for outgoing connections. Never use the long-removed `tcp_tw_recycle`, which broke clients behind NAT.

7.7.5 Sequence numbers, acknowledgments and retransmission

TCP uses **cumulative acknowledgments**: ACK number *N* means "I have all bytes before *N*". The sender keeps every unacknowledged byte in its send buffer and retransmits on one of two signals:

1. **Retransmission timeout (RTO).** If no ACK arrives within a timeout computed from measured RTTs, the sender retransmits the oldest unacknowledged segment, doubles the timeout (**exponential backoff**), and treats the event as severe congestion.
2. **Fast retransmit.** If the receiver gets a segment out of order, it immediately resends its last ACK. Three **duplicate ACKs** strongly suggest a single lost segment with later ones still arriving, so the sender retransmits at once without waiting for the timer [RFC 5681].

Selective acknowledgments (SACK) let the receiver report exactly which blocks beyond the gap it holds, so the sender retransmits only what is missing.

FORMULA

Retransmission timeout (RFC 6298)

On each new RTT sample R (after the first):

$$\text{RTTVAR} \leftarrow (1-\beta)\,\text{RTTVAR} + \beta\,|\text{SRTT} - R|$$

$$\text{SRTT} \leftarrow (1-\alpha)\,\text{SRTT} + \alpha R$$

$$\text{RTO} = \text{SRTT} + \max(G,\ 4\,\text{RTTVAR})$$

Symbol	Meaning	Units
R	Latest measured round-trip time	ms
SRTT	Smoothed (exponentially averaged) RTT	ms
RTTVAR	Smoothed RTT variation	ms
α, β	Smoothing gains, $1/8$ and $1/4$	dimensionless
G	Clock granularity	ms
RTO	Retransmission timeout; RFC 6298 says round up to at least 1 s	ms

WORKED EXAMPLE

Worked example — Computing an RTO.

Current state: SRTT = 100 ms, RTTVAR = 10 ms. A new sample arrives: $R = 140$ ms. Assume G is negligible.

1. RTTVAR $= 0.75 \times 10 + 0.25 \times |100 - 140| = 7.5 + 10 = 17.5$ ms (computed with the *old* SRTT, as the RFC requires).
2. SRTT $= 0.875 \times 100 + 0.125 \times 140 = 87.5 + 17.5 = 105$ ms.
3. RTO $= 105 + 4 \times 17.5 = 175$ ms.
4. RFC 6298 rounds this up to its 1 s minimum; Linux uses a 200 ms minimum, so Linux would wait 200 ms.

Interpretation: inside a datacenter, where RTTs are around 0.1–0.5 ms, a single timeout-driven retransmission costs 200 ms, hundreds of times the normal latency. This is why one lost packet can dominate p99 latency for small RPCs (§23.6), and why datacenter operators tune minimum RTOs down and work hard to avoid drops.

7.7.6 Flow control and the receive window

Flow control prevents a fast sender from overflowing a slow receiver. Each ACK advertises the **receive window** (`rwnd`): the free space in the receiver's socket buffer. The sender never has more than `rwnd` unacknowledged bytes outstanding. If the application stops reading, the buffer fills, the advertised window drops to zero, and the sender pauses, probing periodically until the window reopens. Flow control is how TCP provides **backpressure** end to end (§22.10).

The header's window field is 16 bits, so it can express at most 65,535 bytes. That is far too small for fast long-distance links (see the bandwidth-delay product in §7.15), so the **window scale** option (RFC 7323), negotiated in the SYN, multiplies the window by 2^s for $s \le 14$, allowing windows up to about 1 GiB ($65,535 \times 2^{14} = 1,073,725,440$ bytes).

7.7.7 Congestion control

Flow control protects the receiver. **Congestion control** protects the network. In October 1986, the early Internet suffered **congestion collapse**: throughput between Lawrence Berkeley Laboratory and UC Berkeley fell from 32 kbps to 40 bps [Jacobson 1988] because senders retransmitted aggressively into already-full queues, so most bandwidth carried duplicate packets that were then dropped. Van Jacobson's 1988 algorithms, refined into today's RFC 5681, made each sender infer congestion and back off.

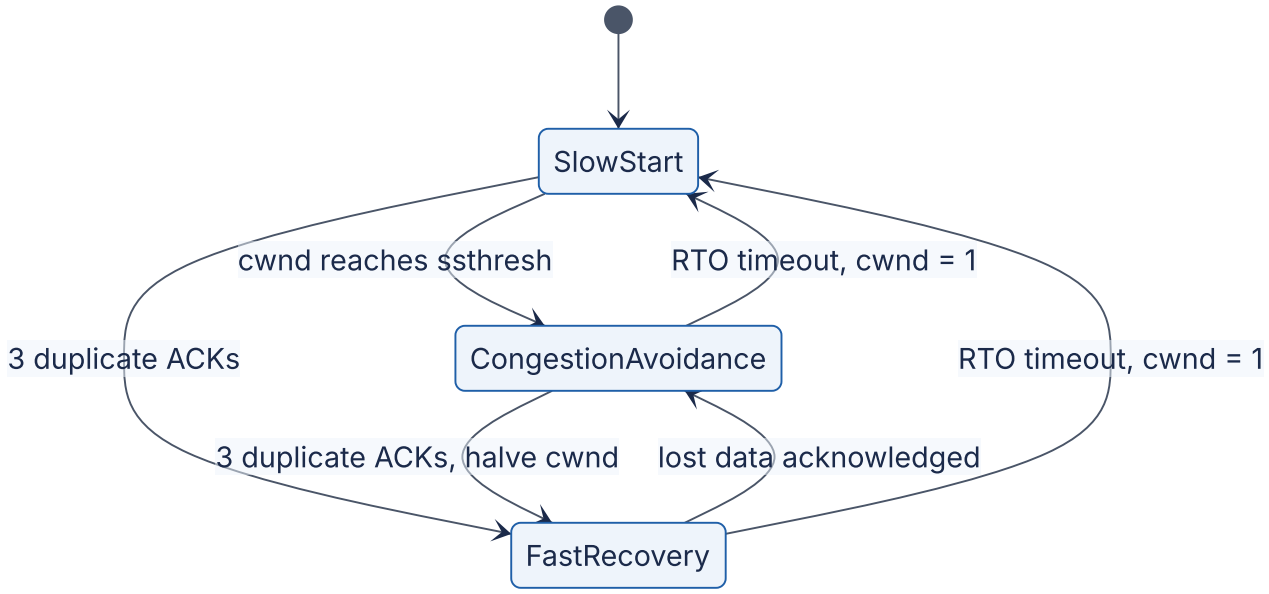
> DEFINITION
>
> **Definition — Congestion window (cwnd).** A sender-side limit, in bytes or segments, on unacknowledged data in flight, based on the sender's estimate of what the network path can absorb. The sender may have at most $\min(\text{cwnd}, \text{rwnd})$ bytes outstanding.

How classic congestion control works

1. **Slow start.** A new connection starts with a small initial window (10 segments, about 14.6 KB, per RFC 6928) and increases cwnd by one segment for every segment acknowledged. That **doubles cwnd every RTT**: "slow" only in comparison with sending everything at once.
2. When cwnd reaches the **slow-start threshold** (`ssthresh`), the sender switches to **congestion avoidance**: it adds about one segment per RTT (**additive increase**).
3. On a loss detected by three duplicate ACKs, the sender halves cwnd (**multiplicative decrease**), sets ssthresh to the new value, retransmits, and continues in congestion avoidance via **fast recovery**.
4. On a retransmission timeout, the sender assumes something severe happened, sets ssthresh to half the flight size, resets cwnd to one segment, and re-enters slow start.

Steps 2 and 3 together are **AIMD (additive increase, multiplicative decrease)**. It produces the familiar sawtooth of cwnd over time, and it has a beautiful property: multiple AIMD flows sharing a bottleneck converge toward an equal share.

Figure 7.6 — Loss-based TCP congestion control states (Reno-style).



Reading the diagram. Each box is a sender state. A connection begins in slow start, where cwnd doubles each RTT. Reaching ssthresh moves it to congestion avoidance (linear growth). Three duplicate ACKs, a mild loss signal, move it to fast recovery with cwnd halved; once the lost data is acknowledged, it returns to congestion avoidance. A timeout, the severe signal, sends it all the way back to slow start with cwnd of one segment. All transitions are driven by asynchronous events (ACK arrivals and timers) on the sender. Failure point: repeated timeouts on a lossy path keep the connection in slow start with exponentially growing RTOs, which users experience as a “hung” transfer.

WORKED EXAMPLE

Worked example — How many RTTs does slow start need for 1 MB?

Assume MSS = 1,460 bytes, initial cwnd = 10 segments, no loss, a large receive window, and ignore delayed ACKs.

1. Segments needed: $1,000,000/1,460 \approx 685$.
2. Segments sent per round (RTT): 10, 20, 40, 80, 160, 320, 640.
3. Cumulative: 10, 30, 70, 150, 310, 630, 1,270.
4. After 6 rounds, 630 < 685; the 7th round finishes the transfer. So **7 RTTs** of data transfer.
5. At an 80 ms RTT that is $7 \times 80 = 560$ ms, plus 1 RTT of TCP handshake and 1 RTT of TLS 1.3 handshake (§7.10): $9 \times 80 = 720$ ms in total, regardless of whether the link is 100 Mbps or 10 Gbps.

Interpretation: for small and medium transfers on long paths, **latency, not bandwidth, determines download time**. A 10× faster link barely helps; reducing RTT (a CDN edge close to the user, §21.3) or reusing a connection whose cwnd is already large (§7.16) helps enormously.

CUBIC

On fast long-distance paths, Reno's "add one segment per RTT" is painfully slow to regain a large window after a loss: after a loss halves a 10,000-segment window, regaining the missing 5,000 segments at one segment per 100 ms RTT takes $5,000 \times 0.1 = 500$ s. **CUBIC** [Ha et al. 2008; RFC 9438] replaces linear growth with a cubic function of *time since the last loss*, independent of RTT. It has been Linux's default since 2006 and is the default in most operating systems today.

FORMULA

CUBIC window growth

$$W(t) = C\,(t - K)^3 + W_{\max}, \qquad K = \sqrt[3]{\frac{W_{\max}\,(1-\beta)}{C}}$$

Symbol	Meaning	Units
$W(t)$	Congestion window t seconds after the last loss	segments
$W_{\max}$	Window just before the last loss	segments
β	Multiplicative decrease factor; window becomes $\beta W_{\max}$ after loss (0.7)	dimensionless
C	Scaling constant (0.4)	segments/s³
K	Time to climb back to $W_{\max}$	s

WORKED EXAMPLE

Worked example — CUBIC recovery time.

Suppose $W_{\max} = 100$ segments when a loss occurs.

1. After the loss: $W(0) = 0.7 \times 100 = 70$ segments. Check with the formula: $K^3 = 100 \times 0.3/0.4 = 75$, so $K = \sqrt[3]{75} \approx 4.22$ s, and $W(0) = 0.4 \times (-4.22)^3 + 100 = 0.4 \times (-75) + 100 = 70$. Consistent.
2. Growth is fast at first, flattens as $t \to K$ (the curve is cautious near the window where loss last occurred), and reaches 100 segments at $t \approx 4.22$ s.
3. Beyond K the window accelerates again, probing for newly available bandwidth.

Interpretation: recovery time depends on $W_{\max}$ and constants, not on RTT, so long-RTT flows are not penalized as much as under Reno. The plateau around $W_{\max}$ keeps CUBIC near the last known-good operating point.

BBR

Loss-based algorithms (Reno, CUBIC) treat packet loss as *the* congestion signal. That has two problems. On paths with large buffers, they keep filling the queue until it overflows, adding hundreds of milliseconds of queuing delay (**bufferbloat**, §7.15). On paths with random loss unrelated to congestion (wireless, long-haul links), they back off when they should not.

BBR (Bottleneck Bandwidth and Round-trip propagation time) [Cardwell et al. 2016] instead builds an explicit model of the path. It continuously estimates the **bottleneck bandwidth** (the maximum recent delivery rate) and the **minimum RTT** (the propagation delay with empty queues). It then paces packets at the estimated bandwidth and keeps roughly one bandwidth-delay product in flight, periodically probing slightly above that rate to discover more bandwidth and periodically draining the queue to re-measure minimum RTT.

Table 7.7 — Congestion control algorithms compared.

Aspect	Reno (AIMD)	CUBIC	BBR (v1)
Congestion signal	Packet loss	Packet loss	Measured bandwidth and RTT model
Growth function	+1 MSS per RTT	Cubic in time since last loss	Pacing at estimated bottleneck rate, with probing
Response to loss	Halve cwnd	Multiply by 0.7	Largely ignores isolated loss
Queue occupancy	Fills buffers until loss	Fills buffers until loss	Aims for little standing queue
Strength	Simple, fair among equals	Good on high-BDP paths; default almost everywhere	Low latency; high throughput with random loss
Weakness	Slow on high-BDP paths	Bufferbloat; collapses with random loss	Fairness issues versus loss-based flows; retransmissions with shallow buffers
Where used	Teaching baseline	Linux, Windows, macOS defaults	Google services, available in Linux since 4.9

INDUSTRY EXAMPLE

BBR at Google. Google reported deploying BBR on its B4 inter-datacenter backbone, where it delivered 2–25× higher throughput than CUBIC, and on YouTube and google.com traffic, where it reduced median RTT (by reducing queuing) while improving throughput. Later work (BBRv2/v3) addresses fairness and loss-sensitivity issues found in BBRv1. *Evidence: Primary* — [Cardwell et al. 2016]

7.7.8 Nagle's algorithm and delayed ACKs

In 1984, John Nagle observed that interactive programs sending one keystroke per write produced 41-byte packets (40 bytes of headers, 1 byte of data), wasting bandwidth. **Nagle's algorithm** (RFC 896) says: if there is unacknowledged data in flight, buffer small writes until either the ACK arrives or a full segment accumulates.

Separately, receivers use **delayed ACKs**: they wait briefly (up to 500 ms by RFC 1122; Linux uses about 40 ms or more) hoping to piggyback the ACK on response data or to acknowledge two segments at once.

Each idea is reasonable; together they cause a notorious stall. A client writes a request header (small write 1), then the body (small write 2), then waits for a response. Write 1 is sent. Nagle holds write 2 until write 1 is acknowledged. The server cannot respond before it has the whole request, so it has nothing to piggyback on and delays its ACK. The pair sit idle for the delayed-ACK timer, adding tens of milliseconds to every request. The fixes are to assemble the full message and write it once, or to set the `TCP_NODELAY` socket option, which disables Nagle. Almost every RPC framework and database driver sets `TCP_NODELAY` by default.

7.7.9 Head-of-line blocking

Head-of-line (HOL) blocking occurs when an item at the front of a queue holds up everything behind it. TCP guarantees in-order delivery of one byte stream, so if segment 5 is lost and segments 6–20 arrive, the receiving kernel holds 6–20 in its buffer and the application can read nothing until segment 5's retransmission arrives, at least one RTT later. If the application multiplexed several independent requests over that connection (HTTP/2, §7.9.3), all of them stall because of one packet that belonged to only one of them. This is the central motivation for QUIC (§7.9.4), which provides independent streams with no cross-stream HOL blocking.

7.7.10 Code example: a TCP echo server and client

```
        # echo_server.py — a minimal, runnable TCP echo server (Python 3.8+).
        import socket

        HOST, PORT = "127.0.0.1", 9000

        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
            srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)  # rebind despite
TIME_WAIT
            srv.bind((HOST, PORT))
            srv.listen(128)                          # backlog of completed handshakes
            print(f"listening on {HOST}:{PORT}")
            while True:
                conn, addr = srv.accept()            # handshake already done by the ker-
nel
                with conn:
                    conn.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
                    while True:
                        data = conn.recv(4096)       # may return fewer bytes than were
sent
                        if not data:                 # empty read: peer sent FIN
                            break
                        conn.sendall(data)           # loops until every byte is queued
```

```
        # echo_client.py — measures connect time and one request/response round trip.
        import socket, time

        t0 = time.perf_counter()
        s = socket.create_connection(("127.0.0.1", 9000))   # SYN, SYN-ACK, ACK
        t1 = time.perf_counter()
        s.sendall(b"hello")
        reply = s.recv(4096)
        t2 = time.perf_counter()
        s.close()                                              # client is active closer:
TIME_WAIT here
        print(reply, f"connect={1e3*(t1-t0):.3f} ms  rtt={1e3*(t2-t1):.3f} ms")
```

What to notice:

- `accept()` returns only after the kernel has completed the three-way handshake; your application never sees SYNs.
- `recv(4096)` returns *up to* 4,096 bytes. TCP is a byte stream, so a real protocol must loop until a complete message has arrived, using a length prefix or delimiter.
- `recv` returning `b""` is how the application learns that the peer sent FIN.
- `SO_REUSEADDR` lets a restarted server bind its port while old connections linger in TIME_WAIT.
- The server handles one client at a time; production servers use threads, an event loop (§6.13) or `asyncio`.

- Over loopback, both timings will be tens of microseconds; the same client against a server 80 ms away would show about 80 ms for each.

Failure scenarios

Table 7.8 — TCP failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Ephemeral port exhaustion	Connection per request; TIME_WAIT holds ports	`EADDRNOTAVAIL`; many TIME_WAIT sockets in `ss -s`	Reuse connections; `tcp_tw_reuse`	Pooling and keep-alive (§7.16)
CLOSE_WAIT leak	Application never closes sockets after peer FIN	Growing CLOSE_WAIT count; file-descriptor exhaustion	Restart process; fix code	Close in `finally`/context managers; timeouts
Half-open connection	Peer crashed or a NAT/firewall dropped state; no FIN sent	Writes hang or eventually reset; reads block forever	Application-level timeouts; reconnect	TCP keep-alive or application heartbeats; read deadlines
Retransmission-driven tail latency	Packet loss; minimum RTO of 200 ms+	p99 spikes; `nstat` retransmission counters	—	Avoid drops; hedged requests (§23.7)
Nagle/delayed-ACK stall	Multiple small writes per request	Latency floor of about 40 ms on small requests	Set `TCP_NODELAY`	Write whole messages; framework defaults
SYN flood	Spoofed SYNs fill the half-open queue	Many SYN_RECV sockets	SYN cookies	SYN cookies on by default; DDoS protection

COMMON MISTAKE

"TCP guarantees my message was processed." An ACK means the peer's *kernel* received the bytes into a buffer, not that the application read them, let alone acted on them or wrote them durably. If the server crashes after ACKing, the data is gone. End-to-end confirmation must come from the application (§24.14, the end-to-end argument).

COMMON MISTAKE

"A TCP connection notices immediately when the other side dies." If the peer vanishes without sending FIN or RST (power loss, cable cut, a firewall silently dropping state), an idle connection looks healthy indefinitely. TCP keep-alive probes default to starting after two hours of idleness on Linux. Always set application-level timeouts.

7.8 UDP and when to use it

DEFINITION

Definition — UDP. The User Datagram Protocol [RFC 768] is a connectionless transport protocol that adds only port numbers, a length and a checksum to IP. Its 8-byte header provides process-to-process delivery of individual **datagrams** with no handshake, no acknowledgments, no retransmission, no ordering, and no flow or congestion control.

If TCP is registered mail with numbered pages and receipts, UDP is a postcard: cheap, immediate, and with no guarantee it arrives or that two postcards arrive in order. UDP does preserve message boundaries: one `sendto()` produces one datagram, and the receiver gets exactly that datagram or nothing.

Why does this exist?

Some applications are hurt more by TCP's guarantees than helped:

- **Request/response with tiny messages.** A DNS query fits in one packet. A TCP handshake would triple the latency; if the reply is lost, the client simply asks again.
- **Real-time media and games.** A voice packet that arrives 300 ms late is useless; it is better to skip it than to stall the stream waiting for a retransmission (TCP's head-of-line blocking).
- **Custom transports.** QUIC (§7.9.4) implements its own reliability, streams and congestion control *on top of* UDP, because UDP passes through existing firewalls and NATs and can be implemented in user space without kernel changes.
- **Multicast and broadcast**, such as DHCP address assignment and market-data feeds, which TCP's one-to-one connections cannot express.
- **High-volume, loss-tolerant telemetry**, such as StatsD metrics, where an occasional lost sample does not matter.

Table 7.9 — TCP versus UDP.

Aspect	TCP	UDP
What it is	Reliable, ordered byte stream	Unreliable datagrams
Connection setup	3-way handshake (1 RTT)	None
Header	20–60 bytes	8 bytes
Reliability and ordering	Yes, via sequence numbers and retransmission	No
Message boundaries	No (byte stream)	Yes (one datagram = one message)
Flow and congestion control	Yes	No; application's responsibility
Head-of-line blocking	Yes, within the stream	No
State on server	Per connection	None required
Typical uses	HTTP/1.1, HTTP/2, databases, RPC	DNS, VoIP, games, QUIC/HTTP/3, DHCP, telemetry
Choose when	You need every byte, in order, and can tolerate setup cost	Timeliness beats completeness, or you are building your own transport

What you must rebuild on UDP

Choosing UDP moves responsibilities into your application: detecting loss (sequence numbers), deciding whether to retransmit, reordering, splitting messages to fit the path MTU (a UDP datagram larger than the MTU gets fragmented by IP, and losing any fragment loses the whole datagram), and above all **congestion control**. A UDP application that sends at a fixed high rate regardless of loss harms every other flow on the path; RFC 8085 gives guidelines for responsible UDP use.

> WARNING / TRADE-OFF
>
> **UDP amplification attacks.** Because UDP has no handshake, an attacker can forge the source address of a small request so that a server sends a much larger response to the victim. Open DNS resolvers, NTP and memcached servers exposed over UDP have been abused for very large DDoS attacks. Never expose UDP services that send responses much larger than requests to the Internet without rate limiting and source validation (§10.11).

7.9 HTTP/1.1, HTTP/2, HTTP/3 and QUIC

7.9.1 HTTP semantics

The **Hypertext Transfer Protocol** is a request/response application protocol. A client sends a request with a **method** (GET, POST, PUT, DELETE, …), a **target** (path), **header**

fields (metadata such as `Host`, `Content-Type`, `Cache-Control`) and an optional body. The server answers with a **status code** (200 OK, 301 redirect, 404 not found, 503 unavailable, …), headers and an optional body. RFC 9110 defines these **semantics** separately from how they are encoded on the wire, and three wire versions coexist: HTTP/1.1 [RFC 9112], HTTP/2 [RFC 9113] and HTTP/3 [RFC 9114]. Your application code sees the same methods, headers and status codes whichever version carries them. API design on top of HTTP is covered in §8.6. For a performance-oriented treatment of everything in this section, see [Grigorik 2013].

7.9.2 HTTP/1.1: text, keep-alive and pipelining

HTTP/1.1 messages are human-readable text:

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: curl/8.5.0
Accept: */*
```

The response begins `HTTP/1.1 200 OK`, followed by headers, a blank line and the body. The body's end is marked by `Content-Length` or by **chunked transfer encoding**.

HTTP/1.0 opened a new TCP connection per request. HTTP/1.1 made **persistent connections** (keep-alive) the default: after a response, the connection stays open for the next request, saving a TCP (and TLS) handshake and keeping the warmed-up congestion window.

The remaining problem is that one connection carries **one request at a time**. HTTP/1.1 defined **pipelining**, sending several requests without waiting, but responses must return in request order, so one slow response blocks all those behind it (application-level head-of-line blocking). Buggy intermediaries and that HOL problem meant browsers never enabled pipelining by default. Browsers instead open **multiple parallel connections**, typically six per host, and sites used to "shard" assets across hostnames to get more. Each extra connection costs handshakes, server memory and separate slow starts, and connections compete with each other for bandwidth.

7.9.3 HTTP/2: binary framing and multiplexing

HTTP/2 (originally RFC 7540 in 2015, now RFC 9113) keeps HTTP semantics but changes the wire format.

How it works internally

1. **Binary framing.** Each message is split into **frames** with a 9-byte header giving length, type (HEADERS, DATA, SETTINGS, WINDOW_UPDATE, RST_STREAM,

GOAWAY, PING, …), flags and a **stream identifier**.

2. **Streams and multiplexing.** Each request/response pair is a **stream** with its own ID (odd numbers for client-initiated streams). Frames from many streams interleave on one TCP connection, so a slow response no longer blocks others at the HTTP layer. One connection per origin replaces six.
3. **Header compression with HPACK** [RFC 7541]. Requests repeat nearly identical headers (cookies, user agent). HPACK keeps a **static table** of 61 common header fields and a **dynamic table** of recently sent ones on both ends, so repeated headers are sent as small integer indexes; new values may be Huffman-coded. HPACK avoids the compression side-channel attacks (CRIME) that made general-purpose compression of headers unsafe.
4. **Per-stream and connection flow control** with WINDOW_UPDATE frames, so one large download cannot starve other streams of buffer space.
5. **Stream cancellation** with RST_STREAM, without closing the connection.

Two features have faded. The original priority tree was complex and inconsistently implemented, and RFC 9113 deprecates it in favor of the simpler scheme in RFC 9218. **Server push**, which let servers send resources before they were requested, proved hard to use well; major browsers have removed support.

The TCP head-of-line problem remains

HTTP/2 removed head-of-line blocking at the HTTP layer, but all streams still share one TCP byte stream. A single lost packet stalls *every* stream until it is retransmitted (§7.7.9). On a clean network HTTP/2's single connection beats six HTTP/1.1 connections; at loss rates of a few percent, typical of poor mobile networks, it can do worse, because six independent connections each suffer only their own losses.

7.9.4 QUIC and HTTP/3

QUIC [RFC 9000] is a general-purpose transport protocol, developed first at Google and standardized by the IETF in 2021, that runs over UDP. **HTTP/3** [RFC 9114] is HTTP mapped onto QUIC.

Why does this exist?

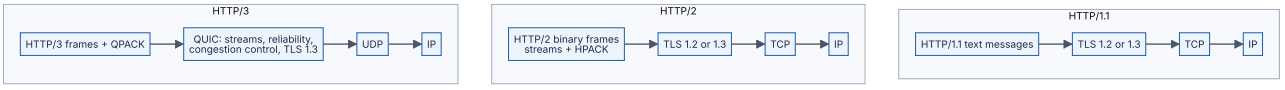
Three problems could not be fixed inside TCP. First, TCP's head-of-line blocking is inherent in its single ordered stream. Second, a secure connection costs two sequential handshakes (TCP, then TLS). Third, TCP lives in operating-system kernels and is inspected and sometimes rewritten by **middleboxes** (firewalls, NATs, load balancers), so new TCP features take a decade to deploy, a problem called **ossification**. Building on UDP, implementing the transport in user space, and encrypting nearly all transport headers solves all three.

How it works internally

1. **Integrated handshake.** QUIC carries the TLS 1.3 handshake inside its own packets [RFC 9001]. Transport and cryptographic setup happen together in **1 RTT**; a resumed connection can send application data in its first flight (**0-RTT**, with the caveats in §7.10.4).
2. **Independent streams.** Reliability and ordering are per stream. A lost packet delays only the streams whose data it carried.
3. **Connection IDs.** A QUIC connection is identified by connection IDs chosen by the endpoints, not by the IP/port 4-tuple. When a phone moves from Wi-Fi to cellular and its IP address changes, the connection **migrates** without a new handshake.
4. **Encrypted headers.** Packet numbers and most transport metadata are encrypted, so middleboxes cannot depend on (and ossify) them.
5. **Better loss recovery.** Packet numbers are never reused (a retransmission gets a new number), which removes TCP's ambiguity about whether an ACK refers to the original or the retransmission [RFC 9002].
6. **QPACK** [RFC 9204] replaces HPACK, because HPACK assumes in-order delivery of headers, which independent streams no longer guarantee.

Clients learn that a server supports HTTP/3 from an `Alt-Svc` response header on an earlier HTTP/2 connection, or from a DNS HTTPS record (§7.6.3), and fall back to HTTP/2 over TCP when UDP is blocked, as it is on some enterprise networks.

Figure 7.7 — Protocol stacks for the three HTTP versions.



Reading the diagram. Each column is one protocol stack, top (application) to bottom (network). In HTTP/1.1 and HTTP/2, the security layer (TLS) sits between HTTP and TCP, and reliability lives in the kernel's TCP. HTTP/2 changes only the top box: binary framing and multiplexed streams over a single TCP connection. In HTTP/3, QUIC absorbs the jobs of both TCP and TLS and runs over UDP, so the stream abstraction moves into the transport and loss on one stream no longer blocks others. Arrows represent encapsulation (synchronous hand-off of each message down the stack). Failure point specific to HTTP/3: networks that block or throttle UDP, handled by falling back to HTTP/2. Scaling consideration: QUIC runs in user space and encrypts every packet individually, so it has historically cost more server CPU per byte than kernel TCP with TLS; implementations close the gap with UDP segmentation offload and batching.

Table 7.10 — HTTP/1.1, HTTP/2 and HTTP/3 compared.

Aspect	HTTP/1.1	HTTP/2	HTTP/3
Specification	RFC 9112	RFC 9113	RFC 9114 (over QUIC, RFC 9000)
Encoding	Text	Binary frames	Binary frames
Transport	TCP	TCP	QUIC over UDP
Concurrency per connection	One request at a time	Many multiplexed streams	Many independent streams
Head-of-line blocking	HTTP layer and TCP	TCP layer only	None across streams
Header compression	None	HPACK	QPACK
Encryption	Optional (TLS)	Effectively required by browsers	Always (TLS 1.3 built in)
New-connection setup with TLS 1.3	2 RTT (TCP + TLS)	2 RTT	1 RTT; 0 RTT on resumption
Connection migration	No	No	Yes, via connection IDs
Debuggability	Plain text with telnet or curl	Needs tools	Needs tools and decrypting keys
Choose when	Simple clients, internal tooling, legacy	Default for APIs and gRPC; most traffic	Mobile and lossy networks, latency-sensitive web traffic

RECOMMENDATION

Choosing an HTTP version. Terminate HTTP/2 (and HTTP/3 where your load balancer or CDN supports it) at the edge for browsers and mobile clients. Inside a datacenter, HTTP/2 over long-lived connections (for example, gRPC, §8.8) is the common default: loss is rare, so TCP HOL blocking matters little, while kernel TCP is cheap and mature. Do not hand-roll HTTP/3 between internal services without a measured reason.

COMMON MISTAKE

“HTTP/2 is always faster than HTTP/1.1.” HTTP/2 wins in most browser workloads because it removes connection limits and compresses headers. But one connection means one congestion window and one point of HOL blocking: under heavy loss, or when a single connection’s cwnd is the bottleneck for bulk transfer, several HTTP/1.1 connections can outperform it. Measure on your traffic.

7.10 TLS 1.3 and HTTPS

7.10.1 What TLS provides

DEFINITION

Definition — Transport Layer Security (TLS). A protocol that gives two communicating applications **confidentiality** (eavesdroppers cannot read the data), **integrity** (tampering is detected) and **authentication** (at least the server proves its identity, usually with an X.509 certificate). **HTTPS** is HTTP carried over TLS, on TCP port 443 by default. TLS 1.3 is specified in RFC 8446 (2018).

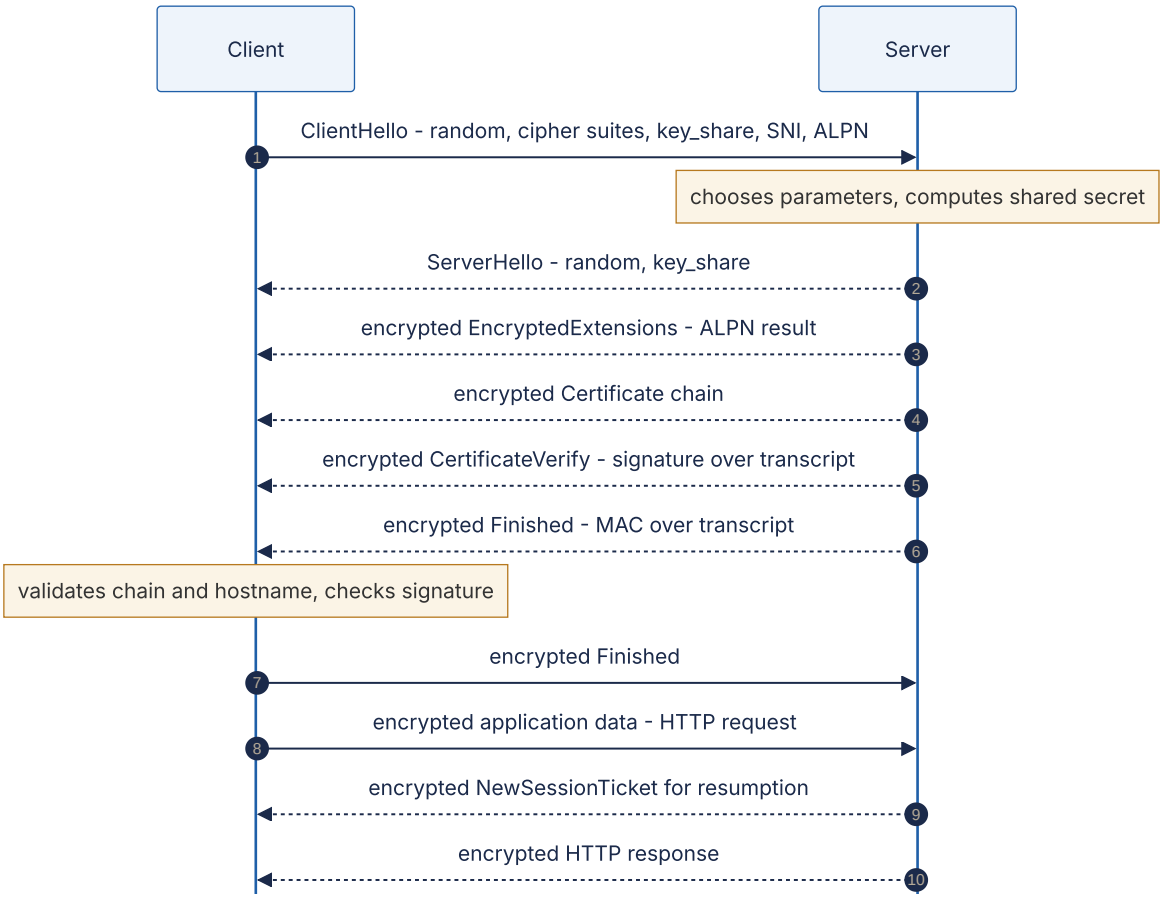
Analogy: before discussing secrets with a stranger on the phone, you check their government-issued ID (the certificate, vouched for by a trusted issuer), then the two of you agree on a code that only you two know, without ever saying it aloud on the line (a **key exchange**), and from then on you speak only in that code and seal every sentence so changes would be noticed (authenticated encryption).

The cryptography itself (Diffie–Hellman, signatures, AEAD ciphers) is covered in §10.4. Here we focus on the round trips and their operational consequences.

7.10.2 The TLS 1.3 handshake

TLS 1.2 needed **two** round trips before application data could flow. TLS 1.3 needs **one**, because the client guesses the key-exchange group the server will accept and sends its key share in the very first message.

Figure 7.8 — TLS 1.3 full handshake (after the TCP handshake).



Reading the diagram. Step 1, the **ClientHello**, carries a random nonce, the cipher suites the client supports, its ephemeral Diffie–Hellman **key share**, the **Server Name Indication (SNI)** naming the host it wants (so one IP address can serve many sites), and the **ALPN** list of application protocols it speaks (`h2`, `http/1.1`). Steps 2–6 are the server's single flight. From the ServerHello onward, both sides can derive handshake keys, so everything after it, including the certificate, is encrypted (in TLS 1.2 the certificate traveled in clear text). CertificateVerify proves the server holds the private key for the certificate; Finished proves nobody tampered with the handshake. In step 7 the client sends its Finished and, in step 8, its first request, all after **one RTT**. Steps 9–10 arrive later: a session ticket for faster resumption, and the response. Everything is synchronous request/response over the already-established TCP connection. Failure points: certificate validation (expired, wrong name, untrusted issuer) aborts the handshake; mismatched protocol versions or cipher suites fail at step 2. Scaling consideration: the server's signature in step 5 is the main CPU cost of a handshake, which is a reason to reuse connections (§7.16) and resume sessions.

7.10.3 Certificates and the chain of trust

A **certificate** binds a public key to one or more names (in the Subject Alternative Name field), with a validity period, signed by a **certificate authority (CA)**. Browsers and operating systems ship a **root store** of trusted root CAs. Servers send their own (**leaf**) certifi-

cate plus **intermediate** certificates; the client builds a chain from leaf to a trusted root and checks:

1. each signature in the chain verifies with the issuer's public key;
2. the current time is within every certificate's validity period;
3. the requested hostname matches a name in the leaf;
4. the certificate has not been revoked (via OCSP or CRLs, imperfectly in practice); and
5. publicly trusted certificates appear in Certificate Transparency logs, as major browsers require.

Operationally, the most common TLS outage is mundane: **an expired certificate**. Publicly trusted certificates have a maximum lifetime of 398 days as of 2025, and the CA/Browser Forum has adopted a schedule to shorten it further over the coming years; ACME-based automation (as popularized by Let's Encrypt) is the only sane response. Monitor expiry dates of every certificate, including internal ones.

Mutual TLS (mTLS), in which the client also presents a certificate, is the standard way to authenticate services to each other inside a zero-trust network (§10.10). PKI design — issuance, revocation and private CAs — is covered in §10.5.

Table 7.11 — TLS 1.2 versus TLS 1.3.

Aspect	TLS 1.2 (RFC 5246)	TLS 1.3 (RFC 8446)
Full handshake	2 RTT	1 RTT
Resumption	1 RTT	1 RTT, or 0-RTT with early data
Key exchange	RSA key transport or (EC)DHE	(EC)DHE only, so forward secrecy is mandatory
Cipher suites	Many, including weak legacy options	Five AEAD suites
Certificate visibility	Sent in clear text	Encrypted
Removed features	—	RSA key transport, static DH, CBC-mode ciphers, compression, renegotiation

Forward secrecy means that stealing a server's long-term private key later does not let an attacker decrypt previously recorded sessions, because each session's keys came from ephemeral Diffie–Hellman values that were discarded.

7.10.4 0-RTT and its caveats

With a session ticket from a previous connection, a TLS 1.3 client can send **early data** in its first flight, encrypted under a key derived from the resumption pre-shared key.

Combined with QUIC, a returning client can deliver a request with zero round trips of setup. Two caveats make 0-RTT dangerous if used carelessly [RFC 8446 §8]:

1. **Replay.** An attacker who records the first flight can resend it, and the server may process the early data twice. TLS cannot fully prevent this across a distributed server fleet.
2. **Weaker forward secrecy for early data**, which is protected only by the resumption key, not by fresh ephemeral key exchange.

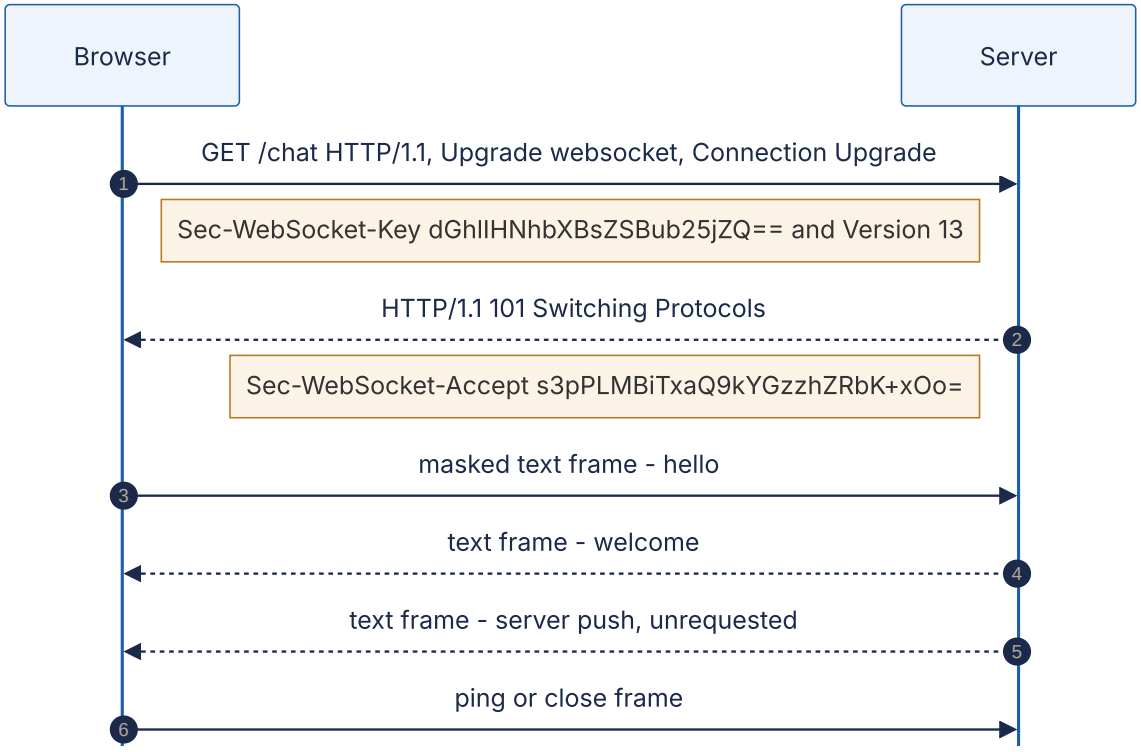
The rule: accept 0-RTT only for **idempotent, safe requests** (GET without side effects). Servers and CDNs that support it typically restrict it to such methods, and HTTP defines the `Early-Data` header and the 425 (Too Early) status code (RFC 8470) so origins can reject replay-unsafe early requests. Never let a payment or any non-idempotent POST ride in 0-RTT (see idempotency keys, §22.15).

> COMMON MISTAKE
>
> **"TLS is too slow to use internally."** With TLS 1.3, a handshake costs one RTT (sub-millisecond inside a datacenter) and some CPU, and modern CPUs encrypt with AES-GCM at several GB/s per core using hardware instructions. With connection reuse, the steady-state cost is small. The exceptions are extreme-throughput paths, where kernel TLS and NIC offload exist for exactly this reason.

7.11 WebSockets (introduction)

HTTP is client-initiated: the server cannot speak until asked. Chat, live dashboards, collaborative editors and multiplayer games need the server to push data whenever it has some. **WebSocket** [RFC 6455] provides a persistent, full-duplex, message-oriented channel between browser and server, created by **upgrading** an HTTP/1.1 connection.

Figure 7.9 — WebSocket opening handshake and framing.



Reading the diagram. Steps 1–2 are an ordinary HTTP/1.1 exchange over an existing TCP (usually TLS) connection. The client sends a random base64 key; the server proves it understood the WebSocket protocol by returning `Sec-WebSocket-Accept`, the base64-encoded SHA-1 hash of the key concatenated with the fixed GUID `258EAFA5-E914-47DA-95CA-C5AB0DC85B11` (the values shown are the example from RFC 6455). After the `101` response the same TCP connection carries WebSocket **frames** in both directions, at any time (steps 3–6): either side may send without being asked, so the arrows are asynchronous in the application sense. Frames from client to server are **masked** with a random 4-byte key to stop malicious pages from crafting bytes that confuse caching proxies. Control frames (ping, pong, close) keep the connection alive and close it cleanly. Failure points: intermediaries with idle timeouts (load balancers commonly default to around 60 s) silently kill quiet connections, so applications send periodic pings; and every connection pins state to one server, which complicates deploys and load balancing.

The full design space (long polling, Server-Sent Events, WebSockets at scale, sticky routing, presence) is Chapter 9. WebSockets can also run over HTTP/2 (RFC 8441) and HTTP/3 (RFC 9220) using an extended CONNECT method.

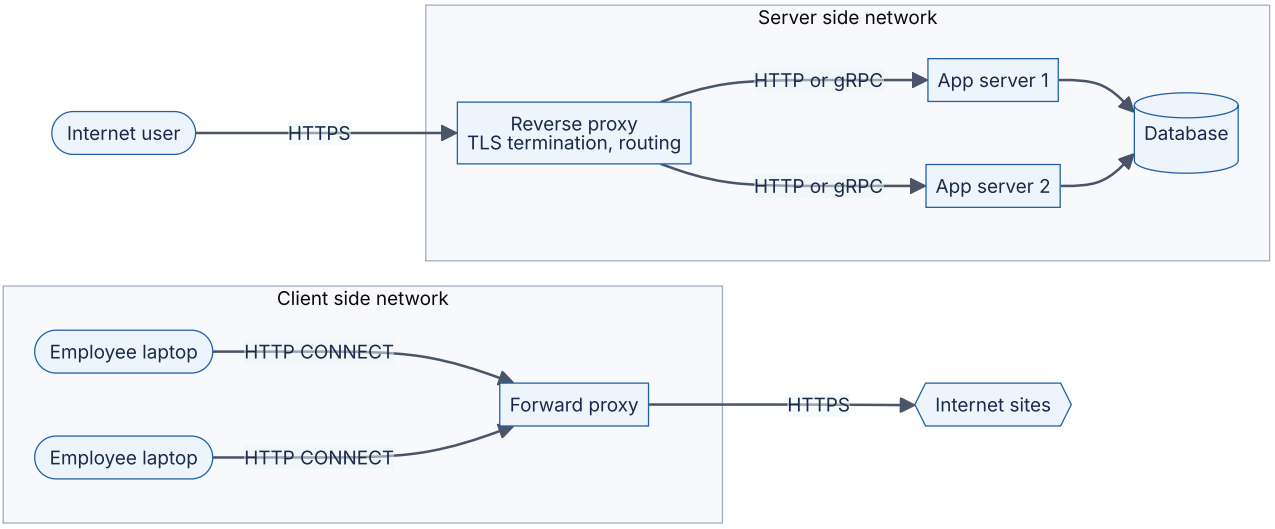
7.12 Forward proxies and reverse proxies

DEFINITION

Definition — Proxy. An intermediary that terminates a client's connection and opens its own connection onward, relaying requests and responses. A **forward proxy** acts on behalf of **clients** and is chosen by them (or their network administrator). A **reverse proxy** acts on behalf of **servers** and is chosen by the site operator; clients usually do not know it exists.

Analogy: a forward proxy is your personal assistant who places all your calls, so callees see the assistant's number. A reverse proxy is a company switchboard: callers dial one number and the receptionist routes them to the right employee, who never has to expose a direct line.

Figure 7.10 — Forward proxy versus reverse proxy.



Reading the diagram. On the left, client machines inside an organization send all outbound traffic through a forward proxy, using the HTTP `CONNECT` method to tunnel TLS connections; destination sites see the proxy's address. On the right, Internet users connect to a reverse proxy, which terminates TLS, inspects HTTP and forwards each request to one of several application servers over internal HTTP or gRPC; users see only the proxy. Both paths are synchronous request/response. The database behind the app servers is the storage boundary. Failure points: each proxy is a single point of failure unless replicated behind anycast, ECMP or DNS (§19.8). Scaling: reverse proxies scale horizontally and absorb connection-handling work (TLS, slow clients) so app servers do not have to.

Table 7.12 — Forward and reverse proxies compared.

Aspect	Forward proxy	Reverse proxy
Acts for	Clients	Servers
Configured by	Client or client's network	Service operator
Hides	Client identity from servers	Server topology from clients
Typical functions	Egress filtering, caching, auditing, access control, anonymity	TLS termination, load balancing, routing by path or host, caching, compression, rate limiting, WAF, request buffering
Examples	Squid, corporate secure web gateways, egress proxies in VPCs	NGINX, HAProxy, Envoy, cloud load balancers, CDNs, API gateways
System-design relevance	Controlled egress from private networks	Nearly every web architecture's front door

A reverse proxy that distributes load is a **load balancer** (Chapter 19); one that adds authentication, quotas and API management is an **API gateway** (§34.6); one deployed beside every service instance is a **sidecar** in a **service mesh**. Proxies that terminate TCP also shield back-ends from slow clients: the proxy buffers a slowly uploaded request and hands the application a complete one, so the application's limited workers are not tied up for seconds.

WARNING / TRADE-OFF

The client IP problem. Behind a reverse proxy, the application sees the proxy's IP as the source. Proxies pass the original address in the `X-Forwarded-For` or standard `Forwarded` header (RFC 7239), or at L4 via the PROXY protocol. Trust these headers only when they were set by your own proxy; otherwise clients can forge them to evade rate limits or IP allow-lists.

7.13 NAT

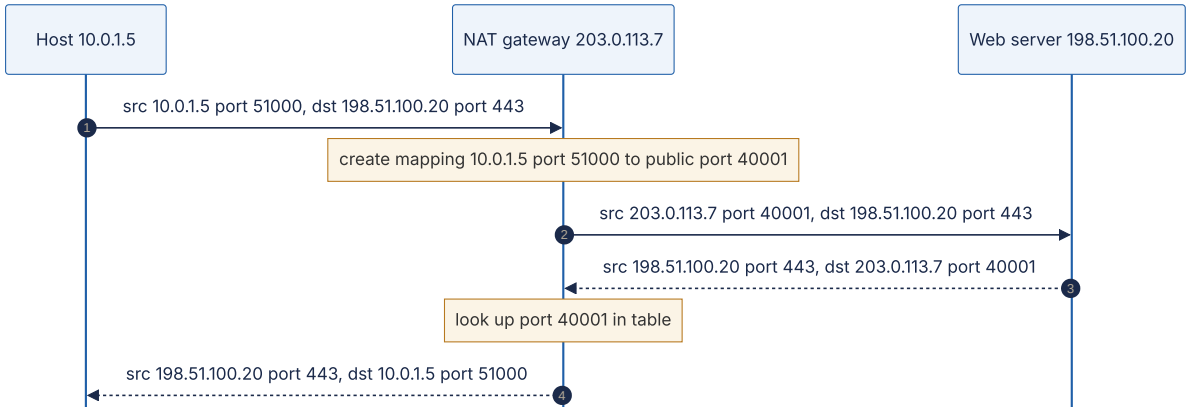
DEFINITION

Definition — Network Address Translation (NAT). A function, usually in a router or gateway, that rewrites the source and/or destination IP addresses (and usually ports) of packets as they pass, keeping a **translation table** so that replies can be rewritten back. The most common form, **source NAT with port translation** (NAPT, often just "NAT"), lets many hosts with private addresses share one public address [RFC 3022].

Analogy: an office building with one street address and a mailroom. Outgoing letters carry the building's return address with an internal "room code" written on them; when

replies come back with that code, the mailroom forwards them to the right room. A letter that arrives with no room code the mailroom has issued has nowhere to go.

Figure 7.11 — Source NAT for an outbound connection.



Reading the diagram. A private host (left) opens a connection to a public server (right) through a NAT gateway (middle). In step 1 the packet carries the host's private address. The NAT rewrites the source to its own public address and a free public port, recording the mapping (steps 2–3). The server replies to the public address (step 4), and the NAT reverses the translation using the table (steps 5–6). All traffic is synchronous and flows through the same NAT in both directions: the NAT is **stateful**, which makes it a failure point (if it restarts or the mapping times out, existing connections die) and a scaling limit (ports per destination).

Consequences for system design

- **Inbound connections are impossible by default**, because no mapping exists until the inside host sends first. Peer-to-peer systems (WebRTC, §9.7) use STUN to discover their public mapping and fall back to TURN relays when NAT behavior prevents direct connections.
- **Mappings time out.** Idle UDP mappings may expire in tens of seconds and TCP mappings in minutes to hours, depending on the device. Long-lived idle connections need keep-alives shorter than the NAT timeout, or they become half-open (§7.7).
- **Port exhaustion.** A single public IP has about 64,000 usable ports per destination (IP, port, protocol). AWS documents a NAT gateway limit of 55,000 simultaneous connections per assigned IP address to each unique destination, raisable by assigning more IPs (as of 2025) [AWS Docs: NAT gateways]. A fleet of services calling one external API through one NAT IP can exhaust this; connection pooling and more NAT IPs fix it.
- **Carrier-grade NAT (CGNAT)**: mobile and some fixed ISPs place many subscribers behind shared addresses from `100.64.0.0/10`. Thousands of users can share one public IP, so **IP-based rate limiting and blocking punish innocent users**.

- **Cost.** Managed cloud NAT gateways usually charge per gigabyte processed; routing bulk traffic to cloud storage through them instead of private endpoints is a common surprise on the bill.

7.14 Firewalls and security groups

DEFINITION

Definition — Firewall. A device or software function that permits or denies traffic according to rules. A **stateless** (packet-filter) firewall evaluates each packet independently against rules on addresses, ports and protocol. A **stateful** firewall tracks connections, so a reply to an allowed outbound connection is automatically permitted. **Application-layer firewalls**, including web application firewalls (WAFs), parse protocols such as HTTP to block malicious requests.

Cloud providers expose two common forms. A **security group** (AWS, similar constructs elsewhere) is a stateful, allow-only rule set attached to a network interface or instance. A **network ACL** is a stateless, ordered allow-and-deny rule list attached to a subnet. Because NACLs are stateless, you must explicitly allow return traffic, typically to the client's ephemeral port range, which is a frequent source of "connection hangs" misconfigurations.

Table 7.13 — Firewall mechanisms compared.

Aspect	Stateless ACL	Stateful firewall / security group	Web application firewall
Inspects	Packet headers	Headers plus connection state	HTTP requests (L7)
Return traffic	Must be explicitly allowed	Allowed automatically	N/A (proxy)
Rule style	Ordered allow/deny	Usually allow-only	Signatures, rate rules, custom logic
Cost of state	None	Connection table (can be exhausted)	Full proxying
Typical placement	Subnet boundary	Each instance or interface	In front of web apps, often at the CDN
Choose when	Coarse subnet guardrails	Default micro-segmentation between tiers	Blocking injection, bots, abusive patterns

Practical rules

1. **Default deny.** Allow only the ports and sources each tier needs: the load balancer may reach the app tier on port 8080; only the app tier may reach the database on

5432.

2. **Reference groups, not addresses.** Security groups can name other groups as sources (“allow from sg-app”), which keeps rules correct as instances scale in and out.
3. **Firewalls are one layer of defense in depth.** Combine them with authentication and encryption (mTLS, §10.10); a network position is not an identity.
4. **Watch the state table.** Stateful devices track every connection; a flood of connections (legitimate or attack) can exhaust their tables and drop everything. Linux’s `conntrack` table is a common culprit on busy proxies.
5. **Silent drops cause timeouts, rejections cause fast failure.** A firewall that drops packets makes clients wait for a timeout; one that sends RST or ICMP unreachable fails fast. Choose deliberately.

7.15 Latency, bandwidth, the bandwidth-delay product and packet loss

7.15.1 The four components of latency

Think of a highway. **Bandwidth** is the number of lanes: how many cars per second can pass a point. **Latency** is how long one car takes to travel from entrance to exit. Adding lanes does not make the drive shorter. A packet’s one-way delay through each hop is the sum of four parts.

> DEFINITION
>
> **Definition — Components of network delay. Propagation delay**: time for a signal to travel the physical distance. **Transmission (serialization) delay**: time to push all of a packet’s bits onto the link. **Queuing delay**: time spent waiting in buffers behind other packets. **Processing delay**: time for a device to examine headers, look up routes and copy the packet.

FORMULA

One-hop delay

$$d_{\text{hop}} = \underbrace{\frac{D}{v}}_{\text{propagation}} + \underbrace{\frac{L}{R}}_{\text{transmission}} + d_{\text{queue}} + d_{\text{proc}}$$

Symbol	Meaning	Units
D	Physical length of the link	m (or km)
v	Signal speed in the medium: about 2×10^8 m/s in optical fiber (roughly two-thirds of c)	m/s
L	Packet size	bits
R	Link rate (bandwidth)	bits/s
d_{queue}	Queuing delay: bytes waiting ahead ÷ link rate	s
d_{proc}	Processing delay: microseconds in software, sub-microsecond in switch hardware	s

Table 7.14 — Delay components at a glance.

Component	Depends on	Typical magnitude	How to reduce it
Propagation	Distance, medium	~5 µs per km of fiber; tens of ms across oceans	Move closer: CDNs, edge, regional replicas
Transmission	Packet size, link rate	12 µs for 1,500 B at 1 Gbps; 1.2 µs at 10 Gbps	Faster links, smaller packets
Queuing	Load, buffer size	0 to hundreds of ms	Lower utilization, AQM, BBR
Processing	Device and software path	~µs (hardware) to tens of µs (software)	Kernel bypass, offloads, fewer hops

WORKED EXAMPLE

Worked example 1 — The speed-of-light floor from New York to London.

The great-circle distance between New York and London is about 5,570 km.

1. One-way propagation in fiber: $5,570 \text{ km} / 200,000 \text{ km/s} = 0.02785 \text{ s} = 27.85 \text{ ms}$.
2. Round-trip time: $2 \times 27.85 = 55.7 \text{ ms}$.
3. Even in a vacuum at $c \approx 299,792$ km/s, the RTT would be $2 \times 5,570 / 299,792 \approx 37.2 \text{ ms}$.
4. Real cables do not follow the great circle, and routers, transmission and queuing add more, so measured RTTs are noticeably higher than 55.7 ms (often in the 70 ms range).

Interpretation: no engineering can make a New York–London round trip faster than about 56 ms over fiber. A user in London calling a single-region service in New York pays at least that on every round trip, and a page needing five sequential round trips pays at least 280 ms. Engineering implication: for global users, reduce round trips (connection reuse, TLS 1.3, QUIC) and move content and computation closer (CDNs, Ch 21; multi-region, Ch 44).

WORKED EXAMPLE

Worked example 2 — Transmission versus propagation.

Send one 1,500-byte packet ($L = 12,000$ bits).

- On a 10 Mbps link: $12,000 / 10^7 = 1.2$ ms.
- On a 1 Gbps link: $12,000 / 10^9 = 12$ µs.
- On a 100 Gbps link: $12,000 / 10^{11} = 120$ ns.

Compare with propagation across a 100 m datacenter cable: $100 / (2 \times 10^8) = 0.5$ µs. Interpretation: on slow access links, serialization dominates; on fast datacenter links, serialization becomes negligible and per-hop processing and software stacks dominate.

WORKED EXAMPLE

Worked example 3 — Bufferbloat.

A home router has a 10 MB buffer in front of a 1 Gbps uplink. When a large upload fills that buffer, a newly arriving packet waits behind $10 \times 10^6 \times 8 = 8 \times 10^7$ bits, so its queuing delay is $8 \times 10^7 / 10^9 = 0.08$ s = 80 ms. On a 20 Mbps uplink the same buffer would add $8 \times 10^7 / (2 \times 10^7) = 4$ s. Loss-based congestion control (CUBIC) keeps such buffers full, so a video call alongside a big upload suffers seconds of lag. Fixes: active queue management (CoDel, FQ-CoDel) on the bottleneck, and delay-aware congestion control such as BBR.

7.15.2 Throughput and the bandwidth-delay product

A reliable sender can only have a window's worth of unacknowledged data in flight; it then waits for ACKs, which take one RTT. So:

FORMULA

Window-limited throughput and the bandwidth-delay product (BDP)

$$\text{Throughput} \le \frac{W}{\text{RTT}} \qquad \text{BDP} = R \times \text{RTT}$$

Symbol	Meaning	Units
W	Effective window, $\min(\text{cwnd}, \text{rwnd})$	bytes
RTT	Round-trip time	s
R	Bottleneck link rate	bits/s (divide by 8 for bytes/s)
BDP	Data that must be in flight to keep the link full	bits or bytes

WORKED EXAMPLE

Worked example — BDP for a 10 Gbps, 80 ms path.

1. BDP $= 10 \times 10^9$ bits/s $\times\ 0.080$ s $= 8 \times 10^8$ bits.
2. In bytes: $8 \times 10^8 / 8 = 10^8$ bytes $=$ **100 MB**.
3. Without window scaling, W is at most 65,535 bytes, giving throughput $\le 65,535 \times 8 / 0.08 \approx 6.55$ Mbps, about 0.07% of the link.
4. To fill the link, the receiver must advertise (and the sender buffer) about 100 MB per connection: window scaling must be on and kernel buffer limits (`net.ipv4.tcp_rmem` and `tcp_wmem` maximums on Linux) must be at least that large.

Interpretation: on “long fat networks”, a single TCP connection is limited by buffers and window, not by the link. Engineering implication: bulk cross-region transfers (backups, replication, dataset copies) need tuned buffers, parallel streams, or both.

7.15.3 Packet loss and the Mathis formula

Loss hurts TCP twice: each lost packet must be retransmitted, and each loss signal halves (Reno) or cuts (CUBIC) the congestion window. For loss-based congestion avoidance with random loss probability p, Mathis, Semke, Mahdavi and Ott derived a simple upper bound on steady-state throughput [Mathis et al. 1997].

FORMULA

Mathis throughput formula

$$\text{Throughput} \le \frac{\text{MSS}}{\text{RTT}} \times \frac{C}{\sqrt{p}}$$

Symbol	Meaning	Units
MSS	Maximum segment size	bits (or bytes, for bytes/s)
RTT	Round-trip time	s
p	Packet loss probability	dimensionless
C	Constant, $\sqrt{3/2} \approx 1.22$ for periodic loss with every packet ACKed	dimensionless

WORKED EXAMPLE

Worked example — Loss caps a long-distance connection.

MSS = 1,460 bytes = 11,680 bits; RTT = 80 ms.

1. MSS/RTT $= 11,680/0.08 = 146,000$ bits/s.
2. With $p = 10^{-4}$ (0.01% loss): $C/\sqrt{p} = 1.22/0.01 = 122$, so throughput $\leq 146,000 \times 122 \approx 17.8$ Mbps.
3. With $p = 10^{-6}$: $C/\sqrt{p} = 1,220$, so throughput ≤ 178 Mbps.
4. To reach 10 Gbps: $\sqrt{p} = 1.22 \times 146,000/10^{10} \approx 1.79 \times 10^{-5}$, so $p \approx 3.2 \times 10^{-10}$, about one loss in three billion packets.

Interpretation: throughput falls with $1/\text{RTT}$ and $1/\sqrt{p}$, independent of link speed. A tiny 0.01% loss caps a single transcontinental CUBIC- or Reno-style flow at under 20 Mbps on a 10 Gbps link. Engineering implications: (a) find and fix lossy links, which are often a dirty optic or a bad cable; (b) reduce RTT; (c) for bulk transfers, use parallel connections or a model-based algorithm such as BBR that does not treat random loss as congestion. The formula is an approximation for loss-based algorithms in congestion avoidance; it does not describe BBR, short flows still in slow start, or timeout-dominated regimes.

7.16 Connection pooling and keep-alive

7.16.1 The cost of a new connection

Before the first byte of a response can arrive over a new HTTPS connection, the client pays for DNS, the TCP handshake, the TLS handshake, and the request itself.

Table 7.15 — Round trips before the first response byte (DNS cached).

Stack	Setup RTTs	Request RTT	Total	At RTT 80 ms	At RTT 0.5 ms (same datacenter)
HTTP over TCP, no TLS	1	1	2	160 ms	1.0 ms
HTTPS, TCP + TLS 1.2	1 + 2	1	4	320 ms	2.0 ms
HTTPS, TCP + TLS 1.3	1 + 1	1	3	240 ms	1.5 ms
HTTP/3, QUIC first connection	1	1	2	160 ms	1.0 ms
HTTP/3, QUIC 0-RTT resumption	0	1	1	80 ms	0.5 ms
Any stack, reused connection	0	1	1	80 ms	0.5 ms

Round trips are only part of the cost. Each new connection also burns CPU (the TLS asymmetric cryptography is the expensive part), allocates kernel and proxy memory, starts in slow start with a small congestion window (§7.7.7), and leaves a TIME_WAIT entry when closed (§7.7.4).

WORKED EXAMPLE

Worked example — Reusing connections to a cross-region dependency.

Service A in Virginia calls service B in Oregon (RTT about 70 ms) 200 times per second. Each call takes 10 ms of server time.

1. New TCP + TLS 1.3 connection per call: latency $\approx 3 \times 70 + 10 = 220$ ms per call.
2. Reused connection: $1 \times 70 + 10 = 80$ ms per call, a 2.75× reduction.
3. Handshake work avoided: 200 TLS handshakes per second, each involving a signature on B and verification on A.
4. Ports: with per-call connections and A as active closer, $200 \times 60 = 12,000$ sockets sit in TIME_WAIT at any time, about 43% of the 28,232 default ephemeral ports.

Interpretation: connection reuse is not a micro-optimization; it changes latency by a factor of nearly three and removes a scaling cliff.

7.16.2 Sizing a pool with Little's Law

A **connection pool** keeps a set of open connections that callers borrow and return. For protocols that carry one request at a time per connection (HTTP/1.1, most database protocols), the number of connections in use equals the number of requests in flight, which Little's Law gives directly (§4.11).

FORMULA

Connections needed for a one-request-per-connection protocol

$$N = \lambda \times W$$

Symbol	Meaning	Units
N	Average connections busy	connections
λ	Request rate	requests/s
W	Time a request holds a connection (network RTT plus server time)	s

WORKED EXAMPLE

Worked example — Database pool sizing.

An application server issues 2,000 queries per second; each holds a connection for 5 ms on average (0.5 ms RTT plus 4.5 ms of query execution).

1. $N = 2,000 \times 0.005 = 10$ busy connections on average.
2. Add headroom for bursts and tail latency: if p99 hold time is 25 ms, bursts can need $2,000 \times 0.025 = 50$. A pool of 20–50 is reasonable.
3. With 40 application servers each holding a pool of 50, the database must accept 2,000 connections. Many databases degrade with thousands of connections (PostgreSQL uses a process per connection), which is why connection proxies such as PgBouncer sit in between (Chapter 14).

Interpretation: size pools from rate × hold time, not by guessing; and remember pools multiply across instances.

7.16.3 Keep-alive pitfalls

- **Idle-timeout races.** If the server (or a load balancer) closes idle connections after 60 s and the client's pool keeps them for 90 s, the client will occasionally send a request on a connection the server has just closed, getting a reset. Set client idle

timeouts **shorter** than every server-side timeout along the path, and retry idempotent requests on this specific failure.

- **Stale connections after failover.** Long-lived connections keep talking to an old backend after DNS changes, so set a maximum connection lifetime.
- **Load imbalance with multiplexed protocols.** An HTTP/2 or gRPC client may send all its traffic over one long-lived connection to one backend; an L4 load balancer balances connections, not requests, so new backends receive nothing. Use L7 (request-level) load balancing or client-side balancing, and cap connection age so connections periodically redistribute (§19.10).
- **TCP keep-alive is not HTTP keep-alive.** HTTP keep-alive means "reuse this connection for more requests". TCP keep-alive sends empty probes on an idle connection to detect dead peers and refresh NAT and firewall state.

7.17 Datacenter networking

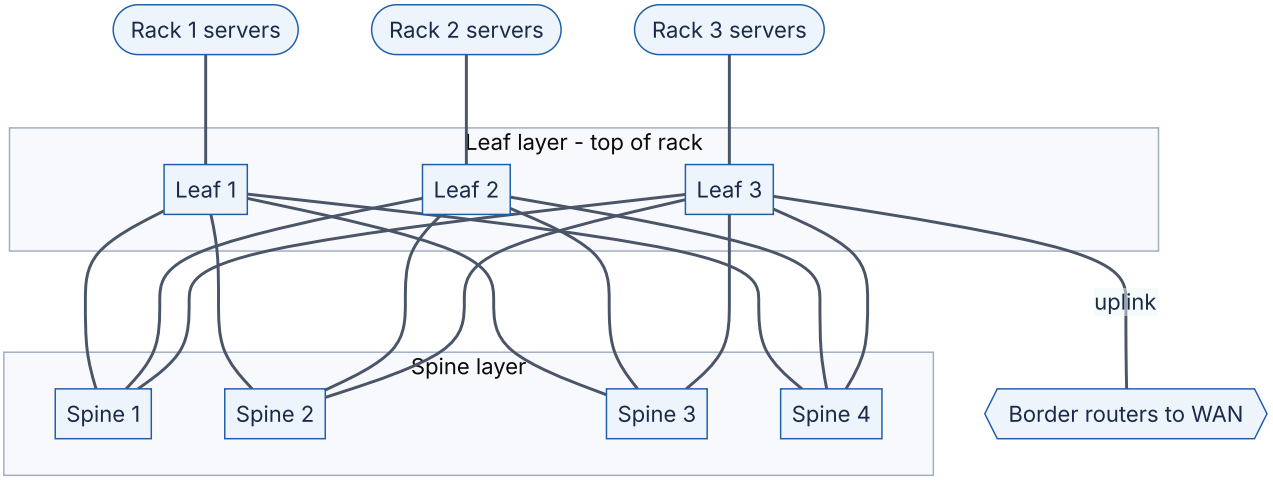
7.17.1 From trees to leaf–spine

Traditional enterprise networks were **trees**: servers connect to access switches, which connect to aggregation switches, which connect to a pair of large core routers. Trees suit **north–south** traffic (between users outside and servers inside). Modern applications are different: a single user request fans out to dozens of internal services, caches and databases, and batch jobs shuffle terabytes between servers. Most bytes now flow **east–west**, server to server inside the datacenter. In a tree, east–west traffic converges on the few links near the top, which become the bottleneck, and the expensive core routers become both a cost and a failure concentration.

DEFINITION

Definition — Leaf-spine (folded Clos) network. A two-tier topology in which every **leaf** (top-of-rack) switch connects to every **spine** switch, and servers attach only to leaves. Any two servers on different leaves are exactly two switch hops apart (leaf → spine → leaf), and traffic is spread across all spines with ECMP. It is a form of the multistage **Clos network**, first described by Charles Clos in 1953 for telephone switching. Larger fabrics add a third tier.

Figure 7.12 — A leaf-spine fabric with ECMP.



Reading the diagram. Servers in each rack connect to their leaf switch. Every leaf has one uplink to every spine (a full bipartite mesh), so a flow from rack 1 to rack 3 has four equal-cost two-hop paths, and leaves pick among them by hashing each flow's 5-tuple (ECMP). Spines never connect to each other or to servers. One leaf (or a dedicated pair) connects to border routers for north–south traffic. Links are undirected because traffic flows both ways; all forwarding is synchronous hardware switching. Failure points: losing a spine removes one quarter of cross-rack capacity rather than isolating anything; losing a leaf isolates its rack, which is why critical services spread replicas across racks. Scaling: add spines for more bandwidth, add leaves (up to the spine port count) for more racks.

FORMULA

Oversubscription ratio at a leaf

$$\text{Oversubscription} = \frac{\text{total server-facing bandwidth}}{\text{total uplink bandwidth}}$$

Symbol	Meaning	Units
Server-facing bandwidth	Sum of the leaf's downlink port speeds	Gbps
Uplink bandwidth	Sum of the leaf's spine-facing port speeds	Gbps

WORKED EXAMPLE

Worked example — Sizing a leaf-spine fabric.

Each leaf has 48 × 25 Gbps server ports and 6 × 100 Gbps uplinks; each spine has 64 × 100 Gbps ports.

1. Downlink: $48 \times 25 = 1,200$ Gbps. Uplink: $6 \times 100 = 600$ Gbps.
2. Oversubscription: $1,200/600 = 2{:}1$. If every server sends cross-rack at full rate, each gets at most 12.5 Gbps.
3. Each leaf uses one port on each of 6 spines, so there are 6 spines. Each spine has 64 ports, so at most 64 leaves.
4. Maximum servers: $64 \times 48 = 3,072$.
5. Total leaf-to-spine capacity: $64 \text{ leaves} \times 600 \text{ Gbps} = 38.4$ Tbps. The **bisection bandwidth** (capacity between any half of the leaves and the other half) is $32 \times 600 = 19.2$ Tbps.

Interpretation: oversubscription is a cost lever. 1:1 (“non-blocking”) fabrics cost more uplinks and spines; 3:1 is acceptable for workloads that are mostly rack-local. Larger fabrics add a third tier of switches to go beyond the port count of one spine.

INDUSTRY EXAMPLE

Google Jupiter. Google’s SIGCOMM 2015 paper describes a decade of five datacenter network generations, culminating in Jupiter. Instead of buying ever-larger proprietary routers, Google built multistage Clos topologies from many small switches using commodity merchant silicon, managed them with centralized, software-defined control rather than traditional distributed routing protocols, and scaled capacity about 100× over ten years to more than 1 Pbps of bisection bandwidth. The paper’s argument (that bandwidth demand from east–west traffic grows faster than any single box can, so the network must scale out like the servers) is now standard practice across large datacenter operators. *Evidence: Primary* — [Singh et al. 2015]

7.17.2 Datacenter-specific problems

- **Incast.** When a client fans a request out to 100 servers that all reply at the same instant, the replies converge on one leaf port, overflow its shallow buffer, and drop packets. The dropped flows wait for a 200 ms minimum RTO in a network whose RTT is about 100 µs, producing extreme tail latency (§23.6). Mitigations: limit fan-out concurrency, jittered responses, smaller minimum RTOs, and ECN-based congestion control such as DCTCP.
- **ECMP hash collisions.** Two elephant flows hashed onto the same spine link share it while other links sit idle. Flowlet switching, packet spraying and adaptive routing

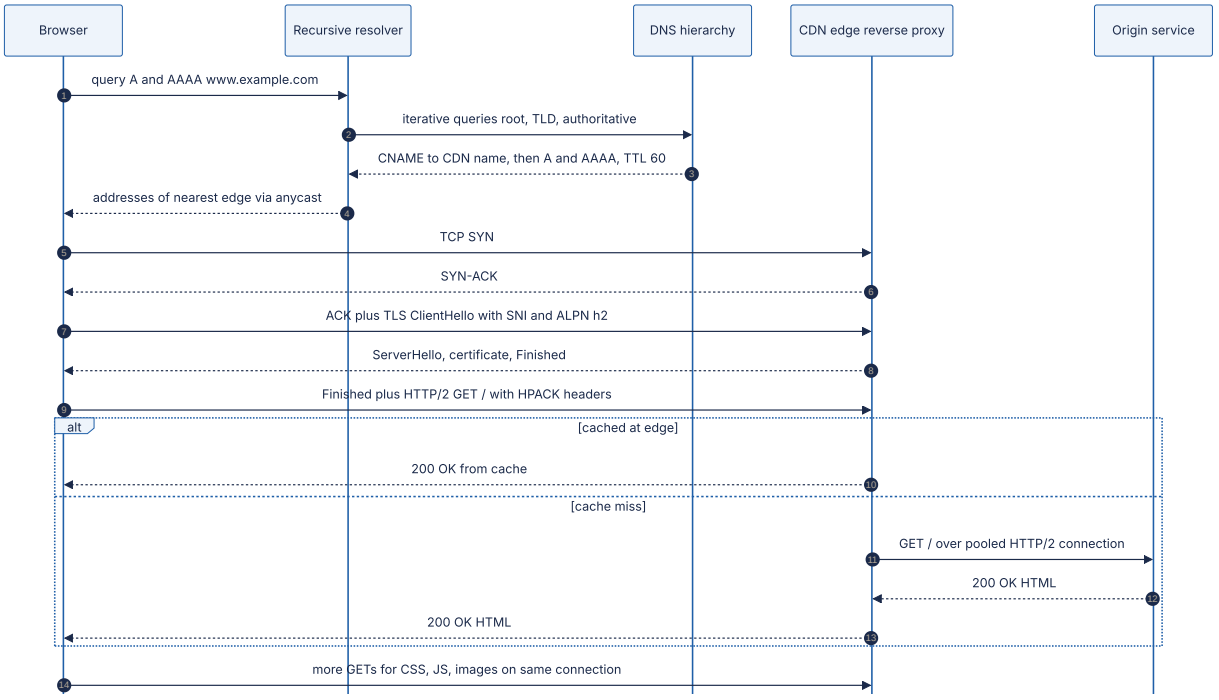
reduce this.

- **Gray failures.** A link that drops 1% of packets is worse than a dead one, because routing still uses it while TCP throughput collapses (recall the Mathis formula). Operators monitor per-link error counters and drain suspect links automatically.

7.18 What happens when you type a URL

Everything in this chapter appears in the life of one request. Suppose you type `https://www.example.com/` into a browser on a laptop connected by Wi-Fi at home, with a cold DNS cache, and the site is served by a CDN edge 20 ms away (RTT) with HTTP/2.

Figure 7.13 — Loading https://www.example.com/ end to end.



Reading the diagram. Steps 1–4: the browser (via the operating system stub) asks a recursive resolver, which walks the DNS hierarchy (§7.6); the authoritative server returns a CNAME pointing into the CDN, whose own authoritative servers answer with an anycast address (§7.5.3). Steps 5–7: the TCP handshake (§7.7.3), with the TLS ClientHello piggybacked on the final ACK. Step 8: the server's TLS 1.3 flight (§7.10.2), including the certificate the browser validates. Step 9: the client's Finished and the first HTTP/2 request, with ALPN having selected `h2` (§7.9.3). Steps 10–13: the edge serves from cache or fetches from the origin over a pre-warmed, pooled connection (§7.16), so the user does not pay the origin's handshakes. Step 14: subresources reuse the same multiplexed connection. All steps are synchronous; the edge cache and origin are the storage boundaries. Failure points include DNS (every step depends on it), certificate validity, and the origin path on cache misses.

7.18.1 Steps the diagram leaves out

Figure 7.13 shows the protocol messages. Several steps happen around them:

1. **Before any network traffic**, the browser parses the URL into scheme (`https`, so port 443), host and path, applies its HSTS list (which may force HTTPS), and checks its HTTP cache and any existing connection to the origin. A reused connection skips steps 1–9 of the figure entirely.
2. **Choosing an address (§7.4).** With both AAAA and A answers, Happy Eyeballs tries IPv6 first and races IPv4 if needed.
3. **The first hop (§7.3).** The destination is off-subnet, so the laptop frames the packet for its default gateway, learning the gateway's MAC address with ARP or Neighbor Discovery.
4. **NAT (§7.13).** For IPv4, the home router rewrites the private source address (for example `192.168.1.23`) to its public address and records the mapping; the ISP may add carrier-grade NAT.
5. **Routing (§7.5).** ISP routers forward by longest-prefix match, and BGP delivers the anycast destination to a nearby edge site.
6. **Edge admission (§7.14, Chapter 19).** DDoS filtering and an L4 load balancer (ECMP plus consistent hashing) pick the server running the reverse proxy.
7. **Behind the edge (§8.5, Chapter 21).** On a cache miss, the origin's reverse proxy routes the request to an application server, which may call caches and databases across a leaf–spine fabric (§7.17).
8. **After the response.** The browser discovers CSS, JavaScript and images and fetches them as concurrent streams on the same connection, ramping up under slow start (§7.7.7). An `Alt-Svc` header may advertise HTTP/3 for later connections.

WORKED EXAMPLE

Worked example — Time to first byte budget.

Assume: 5 ms RTT to the resolver; resolver needs root, TLD and authoritative lookups costing 15, 20 and 30 ms, plus 25 ms for the CDN’s authoritative servers; 20 ms RTT to the edge; 2 ms edge processing on a cache hit.

1. DNS: $5 + 15 + 20 + 30 + 25 = 95$ ms (cold); 5 ms warm.
2. TCP handshake: 20 ms.
3. TLS 1.3 handshake: 20 ms.
4. Request and first response byte: $20 + 2 = 22$ ms.
5. Total cold: $95 + 20 + 20 + 22 = 157$ ms. Warm DNS: $5 + 62 = 67$ ms. Reused connection: 22 ms.
6. On a cache miss with the origin 70 ms from the edge over a pooled connection, add $70 + $ origin processing (say 30 ms) = 100 ms.

Interpretation: on a cold visit, connection setup (DNS, TCP, TLS) is most of the time to first byte, and the server’s own work is a small slice. This is why the biggest wins for global latency are fewer round trips and shorter round trips, not faster servers.

7.19 Summary and revision notes

SUMMARY

- **Layering and encapsulation.** Each layer adds its own header: application message → TCP segment or UDP datagram (ports) → IP packet (addresses) → link frame (MAC). Routers rewrite link frames at every hop; IP packets travel end to end.
- **Addressing.** IPv4 is 32 bits, IPv6 is 128. CIDR `/n` leaves 2^{32-n} IPv4 addresses (minus 2 on ordinary subnets; minus 5 on AWS). Private ranges: 10/8, 172.16/12, 192.168/16. Plan non-overlapping ranges before networks need to connect.
- **Routing.** Forwarding uses longest-prefix match. BGP connects autonomous systems by policy, not latency, and trusts announcements by default. Anycast announces one prefix from many sites to steer users to a nearby one.
- **DNS.** Stub → recursive resolver (recursive query) → root → TLD → authoritative (iterative queries). Caches expire by TTL; nothing "propagates". DNS load balancing picks sites, not servers, and fails over in minutes.
- **TCP.** 1-RTT handshake; four-way close; the active closer holds TIME_WAIT (60 s on Linux), which can exhaust ephemeral ports. Reliability via sequence numbers, cumulative ACKs, fast retransmit and RTO (200 ms minimum on Linux). Flow control uses the receive window; congestion control uses cwnd: slow start doubles per RTT, AIMD halves on loss, CUBIC grows by a cubic of time, BBR paces at measured bandwidth. One lost segment blocks the whole stream (head-of-line blocking). Set `TCP_NODELAY` for RPC.
- **UDP.** 8-byte header, no setup, no reliability. Use it when timeliness beats completeness or to build your own transport; you then own congestion control.
- **HTTP.** 1.1: one request at a time per connection, about six connections per host. 2: binary frames, multiplexed streams, HPACK, but TCP HOL blocking. 3: QUIC over UDP with independent streams, built-in TLS 1.3, 1-RTT (0-RTT on resumption) setup and connection migration.
- **TLS 1.3.** 1-RTT handshake, encrypted certificates, mandatory forward secrecy. 0-RTT early data is replayable: use only for idempotent requests. Expired certificates are the most common TLS outage; automate renewal.
- **Middle of the network.** Forward proxies act for clients; reverse proxies act for servers (TLS termination, routing, load balancing). NAT shares public IPs, blocks unsolicited inbound traffic, and has finite ports per destination. Security groups are stateful; network ACLs are stateless.
- **Numbers.** Fiber propagation is about 5 µs/km; NY–London RTT is at least 56 ms. BDP $= R \times \text{RTT}$: 10 Gbps × 80 ms = 100 MB in flight. Mathis: throughput $\leq (\text{MSS}/\text{RTT}) \times 1.22/\sqrt{p}$, so 0.01% loss caps an 80 ms flow near 18 Mbps.

- **Connections.** A new HTTPS connection with TLS 1.3 costs 3 RTTs to first byte, versus 1 on a reused connection. Pool connections; size pools with $N = \lambda W$; set client idle timeouts shorter than server timeouts.
- **Datacenters.** East–west traffic dominates; leaf–spine Clos fabrics with ECMP give every rack equal-cost two-hop paths and scale out with commodity switches (Google Jupiter: over 1 Pbps of bisection bandwidth).

7.20 Knowledge checkpoint

1. Which headers does a router remove and rebuild at each hop, and which header fields does it modify?
2. What are the network address, broadcast address and usable host range of `172.20.14.200/22`?
3. A router has routes for `10.0.0.0/8`, `10.4.0.0/14` and `10.4.4.0/24`. Which is used for `10.5.1.1`?
4. Why can anycast break long-lived TCP connections, and why is it nevertheless ideal for DNS?
5. Distinguish recursive and iterative DNS queries, and say which participants send each kind.
6. Why can’t you place a CNAME at the zone apex?
7. You need to move a service to new IP addresses; the A record’s TTL is 86,400 s. What must you do and when?
8. What does an acknowledgment number of 5,001 mean?
9. Why does the active closer of a TCP connection enter TIME_WAIT, and for how long on Linux?
10. Explain the difference between flow control and congestion control, naming the window each uses.
11. Starting from an initial window of 10 segments, how many RTTs of slow start are needed to send 300 segments with no loss?
12. Explain how Nagle’s algorithm and delayed ACKs together add latency, and give the standard fix.
13. HTTP/2 multiplexes streams over one connection. Why does it still suffer head-of-line blocking, and how does HTTP/3 avoid it?
14. Why must 0-RTT early data be limited to idempotent requests?
15. Compute the BDP of a 1 Gbps link with a 40 ms RTT, in megabytes.
16. Using the Mathis formula, what is the maximum throughput with MSS 1,460 bytes, RTT 20 ms and 1% loss?
17. Why are network ACLs a frequent cause of “connection hangs” while security groups are not?

18. What is the oversubscription ratio of a leaf with 32 × 100 Gbps server ports and 8 × 400 Gbps uplinks?

Answers

1. The router strips the incoming link-layer (Ethernet) header and trailer and builds a new frame with new source and destination MACs for the next link. In the IP header it decrements TTL (hop limit) and, for IPv4, recomputes the header checksum. NAT devices also rewrite addresses and ports.
2. A `/22` has $2^{10} = 1,024$ addresses; the third octet advances in steps of 4. 14 lies in 12–15, so network `172.20.12.0`, broadcast `172.20.15.255`, usable `172.20.12.1` – `172.20.15.254` (1,022 hosts).
3. `10.4.0.0/14` covers 10.4.0.0–10.7.255.255, which includes 10.5.1.1; the /24 does not. Longest match is the /14.
4. A BGP route change can deliver a flow's later packets to a different site that has no state for the connection, which resets it. DNS queries are typically single UDP request/response exchanges with no connection state, so a route change between queries is harmless.
5. A recursive query asks the server for a final answer; the stub resolver sends it to the recursive resolver. An iterative query accepts referrals; the recursive resolver sends these to root, TLD and authoritative servers.
6. A CNAME may not coexist with any other record at the same name, and the apex must hold SOA and NS records.
7. Lower the TTL (for example to 60 s) at least 86,400 s before the change, so all caches have refreshed with the short TTL; make the change; keep the old addresses serving until at least the short TTL (plus margin for misbehaving caches) has passed; then raise the TTL again.
8. The receiver has received every byte up to and including byte 5,000 (relative to the ISN) and expects byte 5,001 next.
9. To retransmit the final ACK if the peer resends its FIN, and to let delayed segments of the old connection expire before the same 4-tuple can be reused. Linux uses 60 s (2 × MSL).
10. Flow control protects the receiver's buffer using the advertised receive window (rwnd). Congestion control protects the network using the sender's congestion window (cwnd). The sender is limited by $\min(\text{cwnd}, \text{rwnd})$.
11. Cumulative segments: 10, 30, 70, 150, 310. 300 segments complete in the 5th RTT.
12. With unacknowledged data in flight, Nagle holds a second small write; the receiver delays its ACK because it has no response ready until the full request arrives; both wait for the delayed-ACK timer. Fix: write whole messages at once and set `TCP_NODELAY`.
13. All streams share one ordered TCP byte stream, so a lost segment blocks delivery of later bytes for every stream. QUIC orders data per stream, so loss stalls only the affected stream.

14. An attacker can capture and replay the first flight, causing the server to process the early request again; idempotent requests are safe to repeat.
15. $10^9 \times 0.04 = 4 \times 10^7$ bits $= 5 \times 10^6$ bytes $= 5$ MB.
16. MSS/RTT $= 11,680/0.02 = 584,000$ bits/s; $1.22/\sqrt{0.01} = 12.2$; throughput ≤ about 7.1 Mbps.
17. NACLs are stateless, so return traffic to the client's ephemeral ports must be explicitly allowed; if it is not, SYN-ACKs or responses are silently dropped and clients wait for timeouts. Security groups are stateful and allow return traffic automatically.
18. Downlink $32 \times 100 = 3,200$ Gbps; uplink $8 \times 400 = 3,200$ Gbps; ratio 1:1 (non-blocking).

7.21 Interview questions

Basic

Q1. What is the difference between TCP and UDP, and when would you choose UDP?

Answer. TCP provides a connection-oriented, reliable, ordered byte stream with flow and congestion control, at the cost of a 1-RTT handshake, head-of-line blocking and per-connection state. UDP provides unreliable, unordered datagrams with an 8-byte header and no setup. Choose UDP when timeliness matters more than completeness (voice, video, games), when exchanges are a single small request and response (DNS), or when you need a custom transport (QUIC). Mention the cost: the application must handle loss, ordering, message sizing to the MTU and, critically, congestion control.

Q2. Walk me through what happens during a DNS lookup.

Answer. The application calls the OS stub resolver, which checks local caches and then sends a recursive query to a recursive resolver. If the resolver's cache is cold, it queries a root server, which refers it to the TLD servers; the TLD server refers it to the domain's authoritative servers; the authoritative server returns the record with a TTL. The resolver caches every record for its TTL and returns the answer. Mention UDP port 53 with TCP fallback, TTL trade-offs, and that DNS changes do not "propagate"; caches expire.

Q3. What happens during a TCP three-way handshake, and why three messages?

Answer. The client sends SYN with its random ISN; the server replies SYN-ACK with its own ISN and acknowledges the client's; the client ACKs the server's ISN. Three messages are the minimum for each side to announce its ISN and confirm the other's has been received, in both directions. It costs the client one RTT before sending data. Strong answers mention option negotiation (MSS, window scaling, SACK), SYN floods and SYN cookies.

Intermediate

Q4. Your service's p99 latency to a dependency in another region is far higher than its p50. What network causes would you investigate?

Answer. First, connection setup: if connections are not reused, some requests pay DNS plus TCP plus TLS handshakes (3+ RTTs instead of 1); check pool hit rates and idle-timeout races that produce resets and retries. Second, packet loss: a single retransmission timeout costs at least 200 ms on Linux, and loss also shrinks cwnd; check retransmission counters (`nstat`, `ss -ti`). Third, slow start after idle: large responses on cold or idle connections need several RTTs to ramp. Fourth, queuing (bufferbloat) on congested links, Nagle/delayed-ACK stalls on small writes, DNS resolution timeouts, and NAT port exhaustion. Measure with connection-phase timings (e.g., `curl -w`) and distributed traces before changing anything.

Q5. Compare HTTP/1.1, HTTP/2 and HTTP/3 for a mobile application.

Answer. HTTP/1.1 serializes requests per connection, so clients open several connections, each paying handshakes and slow start. HTTP/2 multiplexes streams on one connection with HPACK header compression, a big improvement, but a lost TCP segment stalls all streams, which hurts on lossy mobile links. HTTP/3 over QUIC gives per-stream reliability (no cross-stream HOL blocking), a 1-RTT combined transport and TLS handshake with 0-RTT resumption, and connection migration when the device switches networks. Costs: UDP may be blocked (keep HTTP/2 fallback), and QUIC uses more server CPU. For a mobile app, enable HTTP/3 at the CDN or edge with HTTP/2 fallback, and use 0-RTT only for idempotent requests.

Q6. How would you use DNS for global traffic distribution, and what are its limitations?

Answer. Use latency-based or geo-DNS with health checks to return the nearest healthy region's address, typically pointing at a regional load balancer or anycast address. Limitations: resolvers cache answers for the TTL, so control is coarse and failover takes the detection time plus up to one TTL (and longer for misbehaving caches); geo decisions see the resolver, not the client, unless EDNS Client Subnet is used; DNS has no knowledge of load; clients with open connections ignore DNS changes. So use DNS for region selection with moderate TTLs (30–300 s), anycast or load balancers for fast failover within and across sites, and client retries across regions for resilience.

Advanced

Q7. A nightly replication job copies data between regions 80 ms apart over a 10 Gbps link but achieves only about 200 Mbps. Diagnose and fix.

Answer. Compute the BDP: $10\ \text{Gbps} \times 80\ \text{ms} = 100$ MB must be in flight. Check whether the window is the limit: 200 Mbps × 0.08 s = 2 MB, so the effective window is about 2 MB, possibly because of `tcp_rmem` / `tcp_wmem` maximums or application socket buffer settings. Check loss: by Mathis, 200 Mbps at 80 ms RTT corresponds to $p \approx (1.22 \times 146{,}000/2 \times 10^8)^2 \approx 8 \times 10^{-7}$, so even tiny loss could explain it with CUBIC. Fixes: raise buffer limits and ensure window scaling; use several parallel streams (each has its own cwnd); consider BBR for the bulk sender, since it does not treat random loss as congestion; fix lossy links; compress data. Verify with `ss -ti` (cwnd, rtt, retransmits) and `iperf3`.

Q8. Design the network path for a global API with low latency, high availability and DDoS resilience.

Answer. Publish the API hostname with DNS from two independent providers, pointing at an anycast address announced from many edge sites, so users reach a nearby edge and attacks spread across sites. At each edge, L4 load balancers using ECMP and consistent hashing feed L7 reverse proxies that terminate TLS 1.3 and HTTP/2/HTTP/3, apply WAF rules and rate limits, and cache what they can. Edges forward to the nearest healthy regional origin over pre-warmed, pooled, long-lived connections (mTLS), avoiding per-request handshakes on long paths. Regions are active–active behind health-checked routing, with DNS TTLs of about 60 s as a slower backstop. Inside regions, leaf–spine fabrics and security groups segment tiers. Discuss trade-offs: anycast and TCP stability, 0-RTT replay restrictions, certificate automation, and cost of egress.

Q9. Explain BBR to someone who knows CUBIC, including when you would and would not deploy it.

Answer. CUBIC infers congestion from loss: it grows cwnd as a cubic function of time since the last loss and multiplies cwnd by 0.7 on loss, so it fills buffers until they overflow and reacts badly to random loss. BBR builds a model: it measures the bottleneck bandwidth (maximum delivery rate) and the minimum RTT, paces at the bandwidth estimate, and keeps about one BDP in flight, cycling its pacing gain to probe for more bandwidth and periodically draining queues to re-measure min RTT. Benefits: much higher throughput on lossy long paths (Google reported 2–25× on its B4 backbone) and lower queuing delay. Concerns: BBRv1 can be unfair to loss-based flows sharing a bottleneck and can cause high retransmission rates with shallow buffers; later versions address this. Deploy it on senders serving users over the WAN or doing bulk transfers on lossy paths; be cautious in shared, shallow-buffer datacenter fabrics, and A/B test.

7.22 Practical exercises

1. **Trace DNS by hand.** Run `dig +trace www.wikipedia.org` and identify the root, TLD and authoritative steps. Then run `dig www.wikipedia.org` twice against your resolver

and compare the TTL values in the two answers; explain the difference. Query `dig example.com MX`, `dig example.com TXT`, and `dig _xmpp-server._tcp.jabber.org SRV`, and read each record using Table 7.5.

2. **Time a request phase by phase.** Run:

```
        curl -o /dev/null -s -w 'dns=%{time_namelookup} tcp=%{time_connect} tls=%
{time_appconnect} ttfb=%{time_starttransfer} total=%{time_total}\n'
https://www.example.com/
```

   All values are cumulative seconds from the start. Compute the TCP handshake (`tcp − dns`), TLS handshake (`tls − tcp`) and server time (`ttfb − tls`). Repeat with `--http1.1`, `--http2` and (if your curl supports it) `--http3`, and against a nearby and a distant site. Estimate RTT from the TCP phase and check it against `ping`.

3. **Watch a handshake.** Start the echo server from §7.7.10, then in another terminal run `sudo tcpdump -i lo -nn 'tcp port 9000'` and run the client. Identify the SYN, SYN-ACK, ACK, data segments, FIN/ACK exchanges and their sequence and acknowledgment numbers. Run `ss -tan state time-wait` afterward and find the TIME_WAIT socket on the client side. Repeat the capture in Wireshark and use "Follow TCP Stream".

4. **Exhaust and fix.** Write a client that opens and closes a new connection to the echo server in a tight loop, and count TIME_WAIT sockets with `ss -s`. Then rewrite it to reuse one connection, and compare throughput.

5. **Feel the Mathis formula.** On a Linux test machine, add delay and loss to loopback with `sudo tc qdisc add dev lo root netem delay 40ms loss 0.1%` (giving roughly an 80 ms RTT), measure throughput with `iperf3`, then remove it with `sudo tc qdisc del dev lo root`. Compare your result with the formula, and repeat with `-C bbr` if the BBR module is available.

6. **Subnet calculator.** Verify the worked examples in §7.4 with Python's standard library:

```
        import ipaddress
        n = ipaddress.ip_network("192.168.10.77/26", strict=False)
        print(n, n.netmask, n.broadcast_address, n.num_addresses - 2)   #
192.168.10.64/26 255.255.255.192 192.168.10.127 62
        print(ipaddress.ip_address("10.0.37.200") in
ipaddress.ip_network("10.0.32.0/20"))  # True
        print(list(ipaddress.ip_network("10.0.0.0/16").subnets(new_prefix=20))
[:3])
        print(ipaddress.ip_network("10.0.0.0/16").overlaps(ipaddress.ip_network("1
0.0.128.0/17")))  # True
```

7. **Inspect a certificate.** Run `openssl s_client -connect www.example.com:443 -servername www.example.com -tls1_3 </dev/null` and identify the chain, the negotiated protocol and cipher, and the leaf's validity period.

7.23 Design challenges

1. **Global login service.** Users on five continents must log in with p95 under 300 ms. The credential database must live in one region. Design the network path (DNS, anycast or regional endpoints, TLS termination location, connection reuse between edge and origin) and compute the round-trip budget for a user in Sydney with the database in Virginia (great-circle distance about 15,700 km). Which steps can 0-RTT safely accelerate, and which cannot?
2. **Egress at scale.** 3,000 containers in private subnets call a single third-party payments API through cloud NAT. Calls take 300 ms, peak load is 20,000 requests per second, and each call currently opens a new TLS connection. Estimate concurrent connections with Little's Law, identify the NAT and TIME_WAIT limits you will hit, and redesign (pooling, egress proxies, multiple NAT IPs, keep-alive settings).
3. **Datacenter fabric for an analytics cluster.** 2,000 servers with 25 Gbps NICs run shuffle-heavy batch jobs. Choose leaf and spine port counts and an oversubscription ratio, compute the bisection bandwidth, and explain how you would detect and mitigate incast and ECMP hot spots.

7.24 References

1. Postel, J. *RFC 791: Internet Protocol*. IETF, 1981. https://www.rfc-editor.org/rfc/rfc791
2. Deering, S., Hinden, R. *RFC 8200: Internet Protocol, Version 6 (IPv6) Specification*. IETF, 2017. https://www.rfc-editor.org/rfc/rfc8200
3. Mockapetris, P. *RFC 1034: Domain Names — Concepts and Facilities*. IETF, 1987. https://www.rfc-editor.org/rfc/rfc1034
4. Mockapetris, P. *RFC 1035: Domain Names — Implementation and Specification*. IETF, 1987. https://www.rfc-editor.org/rfc/rfc1035
5. Eddy, W. (Ed.). *RFC 9293: Transmission Control Protocol (TCP)*. IETF, 2022. https://www.rfc-editor.org/rfc/rfc9293
6. Postel, J. *RFC 768: User Datagram Protocol*. IETF, 1980. https://www.rfc-editor.org/rfc/rfc768
7. Fielding, R., Nottingham, M., Reschke, J. (Eds.). *RFC 9110: HTTP Semantics*. IETF, 2022. https://www.rfc-editor.org/rfc/rfc9110
8. Fielding, R., Nottingham, M., Reschke, J. (Eds.). *RFC 9112: HTTP/1.1*. IETF, 2022. https://www.rfc-editor.org/rfc/rfc9112

9. Thomson, M., Benfield, C. (Eds.). *RFC 9113: HTTP/2*. IETF, 2022. https://www.rfc-editor.org/rfc/rfc9113
10. Bishop, M. (Ed.). *RFC 9114: HTTP/3*. IETF, 2022. https://www.rfc-editor.org/rfc/rfc9114
11. Iyengar, J., Thomson, M. (Eds.). *RFC 9000: QUIC: A UDP-Based Multiplexed and Secure Transport*. IETF, 2021. https://www.rfc-editor.org/rfc/rfc9000
12. Rescorla, E. *RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3*. IETF, 2018. https://www.rfc-editor.org/rfc/rfc8446
13. Fette, I., Melnikov, A. *RFC 6455: The WebSocket Protocol*. IETF, 2011. https://www.rfc-editor.org/rfc/rfc6455
14. Allman, M., Paxson, V., Blanton, E. *RFC 5681: TCP Congestion Control*. IETF, 2009. https://www.rfc-editor.org/rfc/rfc5681
15. Other IETF RFCs cited by number in this chapter, each available at `https://www.rfc-editor.org/rfc/rfcNNNN`: 896 (Nagle's algorithm), 1122 (host requirements), 1918 (private addresses), 2308 (negative caching), 2782 (SRV), 3021 (/31 links), 3022 (NAT), 3849 and 5737 (documentation prefixes), 4193 (IPv6 ULA), 4271 (BGP-4), 4632 (CIDR), 5246 (TLS 1.2), 6298 (RTO), 6598 (CGNAT space), 6891 (EDNS(0)), 6928 (initial window), 7239 (Forwarded), 7323 (window scaling), 7413 (TCP Fast Open), 7541 (HPACK), 7858 (DNS over TLS), 8085 (UDP guidelines), 8305 (Happy Eyeballs v2), 8441 and 9220 (WebSockets over HTTP/2 and HTTP/3), 8470 (early data in HTTP), 8484 (DNS over HTTPS), 9001 (QUIC-TLS), 9002 (QUIC loss recovery), 9204 (QPACK), 9218 (HTTP priorities), 9438 (CUBIC), 9460 (SVCB/HTTPS records).
16. Kurose, J. F., Ross, K. W. *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.
17. Grigorik, I. *High Performance Browser Networking*. O'Reilly Media, 2013. https://hpbn.co
18. Mathis, M., Semke, J., Mahdavi, J., Ott, T. *The Macroscopic Behavior of the TCP Congestion Avoidance Algorithm*. ACM SIGCOMM Computer Communication Review 27(3), 1997.
19. Cardwell, N., Cheng, Y., Gunn, C. S., Hassas Yeganeh, S., Jacobson, V. *BBR: Congestion-Based Congestion Control*. ACM Queue 14(5), 2016. https://queue.acm.org/detail.cfm?id=3022184
20. Singh, A., Ong, J., Agarwal, A., et al. *Jupiter Rising: A Decade of Clos Topologies and Centralized Control in Google's Datacenter Network*. ACM SIGCOMM, 2015. https://research.google/pubs/pub43837/
21. Jacobson, V. *Congestion Avoidance and Control*. ACM SIGCOMM, 1988.
22. Ha, S., Rhee, I., Xu, L. *CUBIC: A New TCP-Friendly High-Speed TCP Variant*. ACM SIGOPS Operating Systems Review 42(5), 2008.
23. Clos, C. *A Study of Non-Blocking Switching Networks*. Bell System Technical Journal 32(2), 1953.

24. Janardhan, S. *More details about the October 4 outage*. Engineering at Meta, 2021. https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/
25. RIPE NCC. *YouTube Hijacking: A RIPE NCC RIS case study*. RIPE NCC, 2008.
26. Amazon Web Services. *NAT gateways* (Amazon VPC User Guide). AWS Documentation, accessed 2025. https://docs.aws.amazon.com/vpc/latest/userguide/nat-gateway-working-with.html
27. Amazon Web Services. *Subnet CIDR blocks* (Amazon VPC User Guide). AWS Documentation, accessed 2025.

7.25 Open verification items

- [RIPE NCC 2008]: confirm the exact title and URL of the YouTube hijacking case study and the “about two hours” duration.
- [AWS Docs: VPC subnets]: confirm the URL and that five addresses per subnet are still reserved (stated “as of 2025”).
- The current CA/Browser Forum schedule for reducing maximum public certificate lifetime below 398 days (deliberately not quantified in §7.10.3).
- “Measured NY–London RTTs are often in the 70 ms range” (§7.15.1) is an order-of-magnitude statement; replace with a dated measurement if one is wanted.
- Jacobson’s 1986 collapse figure (32 kbps to 40 bps) and Google’s BBR results (2–25× on B4) should be checked against the primary texts.
- Linux defaults quoted (TIME_WAIT 60 s, minimum RTO 200 ms, delayed-ACK minimum about 40 ms, ephemeral range 32,768–60,999, keep-alive after two hours) are version- and distribution-dependent.
- Runnable code is inline only; per style guide §8 it should also be copied to `code/ch07/` (not done, since this task was limited to the chapter file).

Chapter 8 — Web Application Fundamentals

> **Level:** 1–2 · **Track:** A (Traditional) · **Estimated reading time:** 5 h

8.1 Learning objectives

After this chapter you should be able to:

- Explain the client–server model and why stateless application servers are the basis of horizontal scaling.
- Compare server-side rendering, single-page applications, static generation and native mobile clients, and say which constraints each puts on the backend.
- Trace one HTTPS request from a browser through DNS, the load balancer, the web server, the application and the database and back, and estimate its latency hop by hop.
- Design a resource-oriented REST API: choose URIs, methods, status codes, a pagination scheme (offset or cursor, with the SQL behind each), filtering, sorting and an RFC 9457 error format.
- Explain how GraphQL executes a query, why it suffers from the N+1 problem and how DataLoader fixes it.
- Define a gRPC service in Protocol Buffers, choose among its four streaming modes and propagate deadlines.
- Encode a Protocol Buffers message by hand, explain varints, and state the schema-evolution rules for Protobuf, Avro, Thrift and JSON.
- Configure cookies safely (`Secure`, `HttpOnly`, `SameSite`) and choose between server-side sessions and signed cookies.
- Distinguish authentication from authorization; compare RBAC, ABAC and ReBAC, and describe Google’s Zanzibar.
- Build and validate a JWT, explain HS256 versus RS256, and describe the revocation problem and its mitigations.
- Draw the OAuth 2.0 authorization code flow with PKCE, and say what OpenID Connect adds.
- Version and deprecate an API without breaking clients, and describe it in OpenAPI.

8.2 Prerequisites

- DNS resolution and TTLs — §7.6 (the first hop of every web request is a DNS lookup).
- TCP handshake and connection reuse — §7.7, §7.16 (connection setup dominates the latency of short requests).
- HTTP/1.1, HTTP/2 and HTTP/3 — §7.9 (REST and gRPC are built on HTTP semantics and HTTP/2 framing).
- TLS 1.3 — §7.10 (cookies marked `Secure`, bearer tokens and OAuth all assume an encrypted channel).
- Reverse proxies — §7.12 (load balancers and web servers in front of the application are reverse proxies).
- Basic SQL `SELECT … WHERE … ORDER BY … LIMIT` — §11.6 (needed for the pagination queries; you can read this chapter first and come back).
- Hashes, MACs and digital signatures at the level of "a MAC proves a shared-key holder made this; a signature proves a private-key holder made this" — §10.4 (needed for signed cookies, JWTs and OAuth).

8.3 Client–server architecture

8.3.1 The client–server model

> DEFINITION
>
> **Definition — Client-server architecture.** A way of dividing a system into two roles: a **client**, which initiates requests on behalf of a user or another program, and a **server**, which waits for requests, performs work on shared resources (data, computation) and returns responses. The two communicate over a network using an agreed **protocol**.

First-principles explanation

Think of a restaurant. You, the diner, are the **client**. You do not walk into the kitchen; you tell a waiter what you want, in a format the kitchen understands (the menu is the **protocol**). The kitchen is the **server**: it holds the shared, expensive resources (stoves, ingredients, the chef's skill), serves many diners at once, and sends back a plate (the **response**). Three properties of this arrangement carry directly into software:

1. **The client initiates.** The kitchen never comes to your table unasked. In classic HTTP the server cannot send anything until the client asks; Chapter 9 covers the techniques that work around this.

2. **The server is shared and centralized.** One kitchen serves hundreds of tables, so the restaurant can keep its ingredients (data) consistent in one place.
3. **The menu decouples the two.** You can change tables, or the kitchen can hire a new chef, without either side noticing, as long as the menu stays the same. In software, that contract is the **API (application programming interface)**.

Why does this exist?

Before networks, a program and its data lived on one computer. Once many people needed the *same* data — a bank's balances, an airline's seat map — something had to own the single authoritative copy and arbitrate concurrent changes. Putting that copy on a server, and letting many thin clients talk to it, solves three problems at once: **shared state** (one source of truth), **security** (clients never touch the database directly, so they cannot bypass business rules) and **independent evolution** (you can ship a new server without reinstalling every client, and vice versa, provided the contract holds).

How it works internally

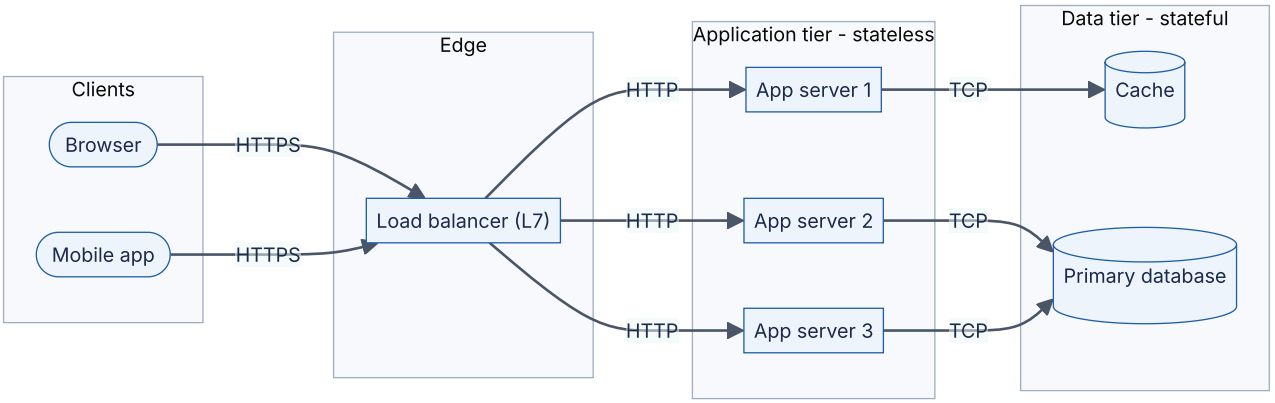
A modern web system usually has three **tiers**:

1. **Presentation tier** — the client: a browser running HTML/JavaScript, or a native mobile app.
2. **Application tier** — one or more **services** that implement business logic. They are normally **stateless**: they keep nothing about a user between requests in their own memory.
3. **Data tier** — databases, caches and object stores that hold durable state.

Each request flows inwards (client → application → data) and the response flows back out. Because the application tier is stateless, any copy of it can handle any request, which is what makes the tier easy to scale horizontally behind a load balancer (see §18.5 and Chapter 19).

Architecture diagram

Figure 8.1 — Three-tier client-server architecture.



Reading the diagram. Two kinds of client (stadium shapes) send HTTPS requests to a layer-7 load balancer, which terminates TLS and forwards plain HTTP (or re-encrypted HTTPS) inside the private network to one of three identical application servers. All arrows are synchronous request/response calls. The application servers are stateless, so the only storage boundary is the data-tier subgraph: the cache and the primary database. Failure points: the load balancer (mitigated by running a redundant pair, §19.8), any single app server (the load balancer's health checks remove it, §19.5) and the database (the hardest to make redundant; Chapter 14). Scaling: add app servers freely; the data tier scales with caching (Chapter 20), replicas and partitioning (Chapter 14).

8.3.2 Statelessness

DEFINITION

Definition — Stateless service. A service that keeps no client-specific data in its own memory between requests. Every request carries, or lets the service look up from a shared store, everything needed to process it.

A waiter who memorizes your order is **stateful**: if he goes home sick, your order is lost and only he can serve your table. A waiter who writes every order on a ticket and pins it to a shared board is **stateless**: any waiter can pick up any table. Stateless servers can be added, removed, restarted and replaced without losing anything, which is why every scaling, deployment and fault-tolerance technique later in this book assumes them.

State does not disappear; it moves. Login status moves into a cookie or token the client sends on every request (§8.10, §8.13), or into a shared session store (§8.10.2). Shopping carts move into a database. The price is an extra lookup per request (typically ~0.5–1 ms to an in-datacenter cache), which is almost always worth paying.

Failure scenarios

Table 8.1 — Client-server failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Users logged out after a deploy	Sessions stored in app-server memory	Spike in 401s and login traffic after each rollout	Users log in again	Move sessions to a shared store or signed cookie
One server overloaded, others idle	Sticky sessions pin heavy users to one server	Uneven per-server CPU and p99	Drain and rebalance	Make servers stateless; avoid affinity (§19.6)
Old mobile clients break	Server changed the API contract	Error rate segmented by app version	Hotfix server to accept old format	Compatibility rules and versioning (§8.15)
Database saturated	Every tier's requests ultimately land on one primary	DB CPU, connection count, lock waits	Shed load, add read replicas	Caching, connection pooling, partitioning

Common misconceptions

COMMON MISTAKE

"Stateless means the system has no state." It means the *compute* tier holds no per-client state between requests. The state lives in databases, caches and the client. Designing a stateless tier is really deciding *where* state should live.

COMMON MISTAKE

"Client-server is the opposite of peer-to-peer, so it is obsolete." Almost every web, mobile and inter-service interaction today is client–server; even "peer" microservices act as client in one call and server in the next. Peer-to-peer appears in specific niches (WebRTC media, §9.7; BitTorrent-style distribution).

8.4 Frontend and backend; SPA, SSR and mobile clients

8.4.1 Frontend and backend

The **frontend** is the code that runs on the user's device and draws the interface: HTML, CSS and JavaScript in a browser, or Swift/Kotlin in a mobile app. The **backend** is the code that runs on servers you control: APIs, business logic, databases, background jobs. The split matters for design because the two sides have opposite trust and update properties:

- **Trust.** Anything on the client can be read and modified by the user. Validation in the frontend is a usability feature; validation and authorization in the backend are security features. Never trust a price, a user ID or a role sent by a client.

- **Update speed.** You can redeploy the backend in minutes. A web frontend updates on the next page load. A mobile app updates only when users install the new version, so versions that are months or years old stay in use and your API must keep serving them (§8.15).

8.4.2 Rendering strategies

Where the HTML is produced determines how many round trips the user waits for, how much work the server does and how search engines see the page.

DEFINITION

Definition — Server-side rendering (SSR). The server produces complete HTML for each request; the browser displays it immediately and then loads JavaScript to make it interactive (**hydration**).

DEFINITION

Definition — Single-page application (SPA) / client-side rendering (CSR). The server sends a nearly empty HTML shell plus a JavaScript bundle; the JavaScript calls JSON APIs and builds the page in the browser. Navigation afterwards happens without full page loads.

DEFINITION

Definition — Static site generation (SSG). HTML is produced at build time and served as static files, usually from a CDN (Chapter 21). Pages that change per user cannot be fully static.

In restaurant terms: SSR is a plated dish (ready to eat, but the kitchen does all the work for every order); SPA is a meal kit (one trip to fetch the kit, then the diner assembles everything, quick to re-assemble but slow to the first bite); SSG is the pre-made sandwich in the display fridge (instant, but everyone gets the same sandwich).

Table 8.2 — Rendering strategies compared.

Aspect	SSR	SPA (CSR)	SSG	Native mobile
Where HTML/UI is built	Server, per request	Browser	Build server, once	Device (compiled app)
Round trips before content	1 (HTML)	≥ 2–3 (shell, JS bundle, API)	1, from a CDN edge	API calls only; UI is installed
Server CPU per page view	High	Low (JSON APIs only)	~0	Low (JSON/Protobuf APIs only)
SEO (crawler sees content)	Good	Weaker; depends on crawler running JS	Good	N/A
Personalized content	Yes	Yes	No (or via client-side calls)	Yes
Backend shape	Server renders templates and calls services	Pure API (REST/GraphQL)	Build pipeline + CDN	Pure API, must tolerate old versions
Typical use	Content and e-commerce pages	Dashboards, editors, apps behind login	Docs, marketing, blogs	Consumer apps

Modern frameworks blend these (SSR for the first page, SPA navigation afterwards; static pages with per-request “islands”). For a system designer the key consequence is the **backend shape**: an SPA or mobile app needs a well-designed JSON/Protobuf API, and SSR needs a rendering tier that itself is a client of your APIs.

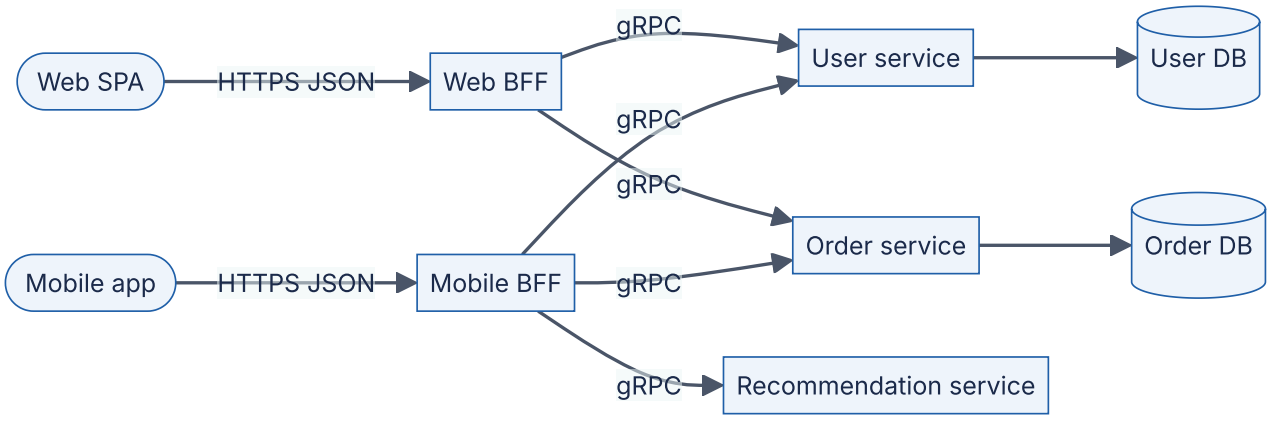
8.4.3 Mobile clients and the backend for frontend

Mobile clients add three constraints:

1. **Old versions never die quickly.** Expect several app versions live at once; any field you remove from a response may crash an old build.
2. **Networks are slow and flaky.** Cellular round-trip times of 50–150 ms are common (as of 2026; varies widely by network). Chatty APIs that need ten sequential calls to draw one screen add up to a second or more of waiting. Mobile clients therefore favor fewer, coarser calls, compact encodings and offline caches.
3. **Different screens need different data.** A phone’s feed card shows 3 fields; the desktop card shows 12.

One common answer is the **backend for frontend (BFF)** pattern: a thin server per client type that aggregates downstream services into exactly the payload that client needs (covered in §34.6). GraphQL (§8.7) is another answer to the same problem.

Figure 8.2 — Backend-for-frontend aggregation.



Reading the diagram. Each client type talks only to its own BFF over HTTPS with JSON. The BFFs call internal services over gRPC (binary, HTTP/2; §8.8) and merge results, so a phone makes one round trip over the slow cellular link instead of three. All calls are synchronous. Each service owns its own database (storage boundaries at the cylinders). Failure points: a slow recommendation service delays the mobile screen unless the BFF uses a deadline and degrades gracefully (§8.8.4). Scaling: BFFs are stateless and scale horizontally; the cost is one more service per client type to own and deploy.

8.5 The request–response lifecycle end to end

8.5.1 What happens when you press Enter

DEFINITION

Definition — Request-response lifecycle. The complete sequence of network and processing steps between a client issuing a request and receiving the full response, including name resolution, connection establishment, routing through intermediaries, application processing, data access and the return path.

First-principles explanation

Before your restaurant order is cooked, someone must find the restaurant's address (DNS), drive there (TCP connection), show ID at the door (TLS handshake), be seated by a host who picks a free waiter (load balancer), have the waiter take the order to the kitchen (web server to application), and wait while the kitchen fetches ingredients from the pantry (database). Only the last two steps are "real work". The others are setup, and for short requests setup often dominates.

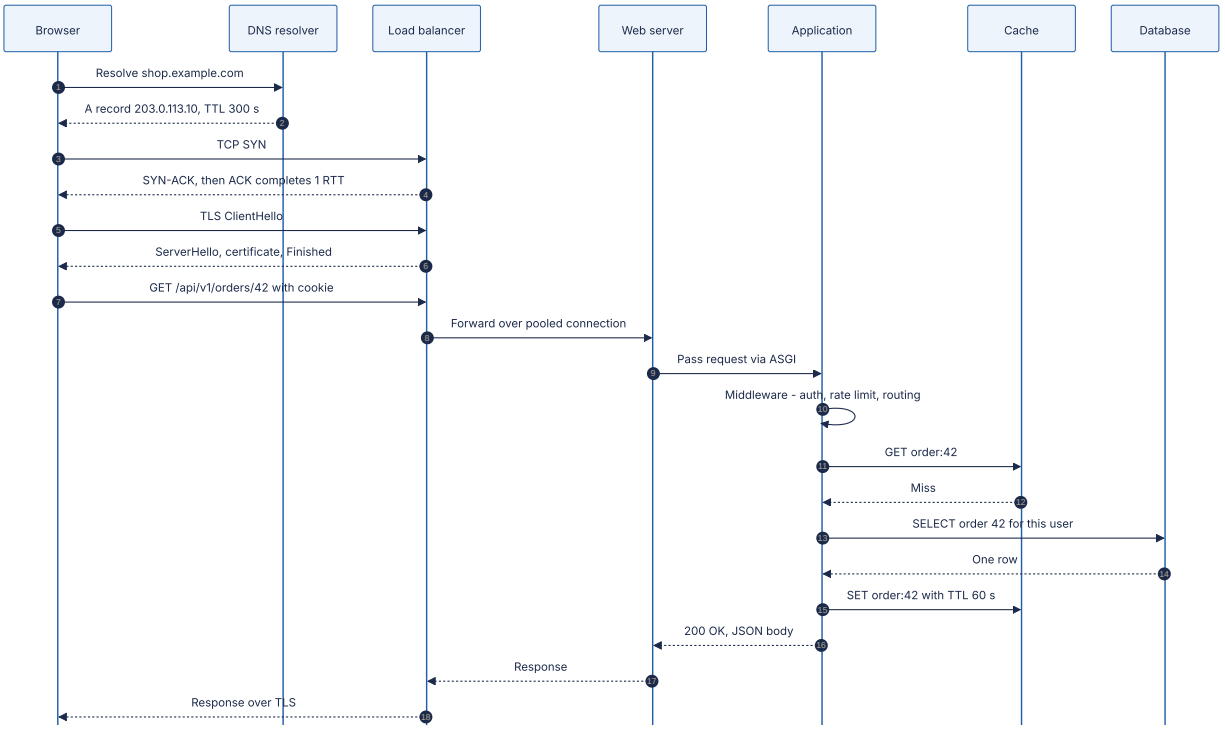
How it works internally

Take `GET https://shop.example.com/api/v1/orders/42` from a browser. (§7.18 follows a similar journey at the packet level; here the focus is the server side.)

1. **URL parsing and cache check.** The browser splits the URL into scheme (`https`), host (`shop.example.com`), path and query. If a fresh copy is in the HTTP cache (`Cache-Control: max-age`), it stops here (§20.5).
2. **DNS resolution** (§7.6). The browser asks the OS stub resolver, which asks a recursive resolver, which (on a cache miss) walks root → `.com` → `example.com` authoritative servers. The answer is an IP address, often of a CDN edge or a regional load balancer, cached for its TTL.
3. **TCP handshake** (§7.7). SYN, SYN-ACK, ACK: one round trip (RTT) before data can flow. With HTTP/3 this is replaced by QUIC, which combines transport and TLS setup (§7.9).
4. **TLS 1.3 handshake** (§7.10). One more RTT to agree keys and verify the server certificate (zero RTT on resumption, with replay caveats).
5. **HTTP request sent.** Method, path, headers (`Host`, `Cookie` or `Authorization`, `Accept`), and possibly a body. Over HTTP/2 this is a set of compressed frames on a stream within the connection.
6. **Load balancer** (Chapter 19). A layer-7 load balancer terminates TLS, reads the HTTP request, applies routing rules (path `/api/` → API pool), picks a healthy backend and forwards the request, often over a pooled keep-alive connection. It adds headers such as `X-Forwarded-For` so the backend knows the client IP.
7. **Web server / reverse proxy.** Something like Nginx or Envoy (or the application server's own HTTP layer) parses the request, enforces limits (body size, timeouts), may serve static files, and passes the request to the application process via HTTP, a socket or an in-process call (for Python, via the ASGI or WSGI interface).
8. **Application framework.** Middleware runs in order: request ID and tracing, authentication (cookie or token → user identity, §8.10–§8.13), rate limiting (Chapter 22), then the **router** maps `GET /api/v1/orders/{id}` to a handler function.
9. **Business logic and authorization.** The handler checks that this user may read order 42 (§8.12), then fetches data.
10. **Data access.** The handler borrows a connection from a pool (§7.16), sends SQL to the database (or reads the cache first), and waits. The database parses, plans, reads pages from its buffer pool or disk and returns rows (Chapter 11).
11. **Response built.** The handler serializes the result (JSON, §8.9), sets status (`200 OK`), headers (`Content-Type`, `Cache-Control`, `ETag`) and returns it through the middleware stack, web server and load balancer.
12. **Browser processes the response.** It decompresses, parses, updates its cache and cookie jar, and renders or hands the JSON to JavaScript. The TCP/TLS connection stays open (keep-alive) for later requests.

Architecture diagram

Figure 8.3 — End-to-end lifecycle of one HTTPS API request.



Reading the diagram. Time flows downward. Messages 1–2 are DNS over UDP (or DNS over HTTPS); 3–4 are the TCP handshake; 5–6 are the TLS 1.3 handshake. Message 7 is the actual HTTP request, carried over the encrypted connection. The load balancer forwards it (8) over a connection it already holds open to the web server, which hands it to the application process (9). The application runs middleware (10), tries the cache (11–12), falls through to the database (13–14) and populates the cache on the way out (15) in the cache-aside pattern (§20.7). Everything is synchronous: the browser waits for the whole chain. Storage boundaries are the cache and the database. Failure points: DNS (stale or failing resolvers), the load balancer, a slow database query that holds a pooled connection, and timeouts that are not aligned along the chain. Scaling options: CDN and HTTP caching remove steps 3–15 entirely for cacheable responses; connection reuse removes steps 3–6 for repeat requests; app servers scale horizontally; the database scales with replicas and partitions.

Mathematical foundations

FORMULA

End-to-end latency of a request on a new connection

$$T_{\text{total}} = T_{\text{DNS}} + T_{\text{TCP}} + T_{\text{TLS}} + T_{\text{req}} + T_{\text{server}} + T_{\text{transfer}}$$

with $T_{\text{TCP}} \approx 1 \times RTT$, $T_{\text{TLS 1.3}} \approx 1 \times RTT$, $T_{\text{req}} \approx 1 \times RTT$ and $T_{\text{transfer}} \approx \frac{S}{B}$ for a response that fits in the congestion window.

Symbol	Meaning	Units
T_{DNS}	DNS lookup time (≈ 0 if cached)	ms
RTT	Client-to-load-balancer round-trip time	ms
T_{server}	Time from the load balancer receiving the request to sending the response (app + DB)	ms
S	Response size	bytes
B	Bottleneck bandwidth	bytes/ms

WORKED EXAMPLE

Worked example — Cold versus warm request. A user is 40 ms RTT from the region. Uncached DNS takes 30 ms. The server spends 2 ms in the load balancer and web server, 8 ms in application code and one 3 ms database query: $T_{\text{server}} = 13$ ms. The 4 KB response transfers in well under 1 ms on a broadband link, so ignore it.

1. Cold (new DNS, new connection): $30 + 40 + 40 + 40 + 13 = 163$ ms.
2. Warm (DNS cached, connection reused): $0 + 0 + 0 + 40 + 13 = 53$ ms.
3. Share of the cold request spent on actual server work: $13/163 \approx 8\%$.

Interpretation. For short API calls, the network setup costs about ten times as much as the work. **Engineering implication:** before optimizing a 3 ms query, make sure clients reuse connections (HTTP keep-alive, HTTP/2 multiplexing), keep DNS TTLs reasonable, terminate TLS close to users (CDN edges, §21.3) and avoid designs that need several *sequential* round trips per screen.

Table 8.3 — Typical latency budget per hop (order-of-magnitude, as of 2026).

Hop	Typical cost	Dominated by
DNS lookup (cached in OS/browser)	~0 ms	—
DNS lookup (recursive resolver cache miss)	10–100 ms	RTTs to authoritative servers
TCP + TLS 1.3 setup	2 × RTT (e.g. 20–200 ms on the public internet)	Distance to the TLS terminator
Load balancer and reverse proxy	~0.1–1 ms	Parsing, TLS termination CPU
Application code (simple handler)	1–20 ms	Serialization, framework overhead, language runtime
Cache read in the same datacenter	~0.2–1 ms	Network RTT inside the datacenter
Indexed database point read	~0.5–5 ms	Network RTT + buffer-pool hit or disk read
Unindexed scan of a large table	100 ms–seconds	Rows read from disk (Chapter 11)

These figures are illustrative ranges for estimation; measure your own system (Chapter 23). See §4.17 and Appendix F for dated reference numbers.

Failure scenarios

Table 8.4 — Lifecycle failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Users hit old servers after a migration	DNS answers cached for the old TTL	Traffic still arriving at old IPs	Keep old endpoints serving until TTL expires	Lower TTL days before a planned change
502/504 from the load balancer	Backend crashed or slower than the LB timeout	LB 5xx metrics by backend	Remove unhealthy backends	Health checks; timeouts that shrink inward along the chain
Requests pile up, then everything times out	DB connection pool exhausted by slow queries	Pool wait time, active connections	Kill slow queries, shed load	Query timeouts, indexes, pool sizing (§14.13)
Client retries make an outage worse	Timeouts plus immediate retries multiply load	Request rate rises while success rate falls	Shed load, open circuit breakers	Exponential backoff with jitter, retry budgets (§22.12)

COMMON MISTAKE

"Latency is mostly the database." For a typical small API call, connection setup and round trips often cost more than the query. Count round trips first, then optimize server time.

8.6 REST API design

8.6.1 What REST is: Fielding's constraints

DEFINITION

Definition — REST (Representational State Transfer). An architectural style for distributed hypermedia systems, defined by Roy Fielding in his 2000 doctoral dissertation as a set of constraints on how components interact [Fielding 2000]. In everyday usage, a "REST API" is an HTTP API that models its domain as **resources** identified by URIs and manipulated with the standard HTTP methods.

First-principles explanation

A library does not give each patron a custom set of instructions. It has a uniform system: every book has a call number (a **resource identifier**), and there are a few standard actions — look up, borrow, return — that work the same for every book. You can walk into any library and use it. REST applies the same idea to networked software: instead of inventing a new verb per operation (`getOrder`, `cancelOrderV2`, `listAllOrdersForUser`), you name *things* (`/orders/42`) and use a small, fixed set of verbs that every client, proxy and cache already understands.

Why does this exist?

Fielding was describing why the web had scaled to millions of independently developed servers and clients. Its uniform interface lets intermediaries — caches, proxies, CDNs, load balancers — understand and optimize traffic without knowing anything about the application. A CDN can cache any `GET` response marked cacheable; a proxy can retry any idempotent request. Custom RPC protocols of the 1990s could not offer that.

Table 8.5 — Fielding's REST constraints.

Constraint	Meaning	What it buys you
Client–server	Separate the user interface from data storage	Independent evolution of client and server
Stateless	Each request contains all information needed to understand it; no session state on the server	Visibility, reliability, horizontal scalability
Cache	Responses label themselves cacheable or not	Fewer interactions; lower latency
Uniform interface	Resource identification, manipulation through representations, self-descriptive messages, hypermedia as the engine of application state (HATEOAS)	Generic intermediaries and clients
Layered system	A client cannot tell whether it talks to the origin or an intermediary	Load balancers, caches, gateways can be inserted transparently
Code on demand (optional)	Servers may send executable code (e.g. JavaScript)	Extensible clients

NOTE

Note — "RESTful" in practice. Most public "REST" APIs implement the first five constraints partially and skip HATEOAS. Fielding has written that an API which is not hypertext-driven is not REST in his sense. This book uses "REST API" in the common, looser sense — resource-oriented HTTP + JSON — and flags HATEOAS separately (§8.6.8).

8.6.2 Resources and URIs

A **resource** is any named thing the API exposes: a user, an order, a collection of orders, the current exchange rate. A **representation** is a serialized snapshot of it (JSON, usually). Design rules that hold up in practice:

1. **Nouns, not verbs, in paths.** `POST /orders`, not `POST /createOrder`. The method is the verb.
2. **Plural collections, item by identifier.** `/orders` (collection) and `/orders/42` (item).
3. **Nest only for true containment, at most one level.** `/orders/42/items` is clear; `/users/7/orders/42/items/3/discounts` couples clients to your data model. Prefer `/order-items/3` once items have their own identity.
4. **Opaque, stable identifiers.** Prefer IDs that do not leak row counts or allow enumeration (UUIDs or Snowflake-style IDs, §17.10).
5. **Actions that do not fit CRUD** become sub-resources or state changes: `POST /orders/42/cancellation` or `PATCH /orders/42 {"status":"cancelled"}`, rather than `POST /cancelOrder?id=42`.

6. **Consistent casing** (`snake_case` or `camelCase` in JSON, kebab-case in paths) across the whole API.

8.6.3 HTTP methods: safety and idempotency

Two properties of methods matter enormously for distributed systems, because networks lose responses and clients retry.

DEFINITION

Definition — Safe method. A method whose semantics are read-only: the client does not request, and does not expect, any state change on the server [RFC 9110 §9.2.1]. Logging or metrics side effects are allowed.

DEFINITION

Definition — Idempotent method. A method for which the intended effect on the server of sending the same request $N > 0$ times is the same as sending it once [RFC 9110 §9.2.2]. The *response* may differ (a second `DELETE` may return 404); the *state* does not.

Analogy: pressing a lift's call button is idempotent (pressing five times summons one lift); dropping a coin in a vending machine is not (five coins, five charges). If a client sends `POST /payments` and the connection drops before the response arrives, it cannot know whether the payment happened. Retrying a non-idempotent request risks a double charge. Retrying an idempotent one is always safe.

Table 8.6 — HTTP method properties [RFC 9110].

Method	Typical use	Safe	Idempotent	Request body	Cacheable
`GET`	Read a resource or collection	Yes	Yes	No (no defined semantics)	Yes
`HEAD`	`GET` without the body (check existence, size)	Yes	Yes	No	Yes
`OPTIONS`	Discover allowed methods; CORS preflight	Yes	Yes	Optional	No
`POST`	Create in a collection; any non-idempotent action	No	**No**	Yes	Only with explicit freshness; rare
`PUT`	Create or fully replace at a known URI	No	Yes	Yes	No
`PATCH`	Partial update [RFC 5789]	No	Not guaranteed	Yes	No
`DELETE`	Remove a resource	No	Yes	Rarely	No

What to notice:

- `PUT /users/7/avatar` with the full new representation is idempotent: sending it twice leaves the same avatar. `PATCH {"op":"increment","path":"/balance"}` is not.
- To make `POST` safe to retry, clients send an **idempotency key** header (for example a UUID per logical operation); the server stores the key with the result and returns the stored result for duplicates. Chapter 22 (§22.15) covers the full design.

8.6.4 Status codes

Status codes are part of the uniform interface: a load balancer, a retry library or a monitoring system can act on `503` without understanding your JSON.

Table 8.7 — Status codes every API designer uses [RFC 9110 unless noted].

Code	Meaning	Use it when	Client should
200 OK	Success with body	Successful `GET`, `PUT`, `PATCH`	Use the body
201 Created	New resource created	`POST` created something; include `Location` header	Follow `Location`
202 Accepted	Queued for asynchronous processing	Long jobs; return a status URL	Poll the status resource
204 No Content	Success, no body	`DELETE`, or update returning nothing	Nothing
301 / 308	Moved permanently	Resource has a new URI (308 preserves the method)	Update bookmarks
304 Not Modified	Cached copy still valid	Conditional `GET` with `If-None-Match` matched the `ETag`	Use cached copy
400 Bad Request	Malformed request	Unparseable JSON, invalid parameter syntax	Fix the request; do not retry as is
401 Unauthorized	Not authenticated	Missing, expired or invalid credentials; send `WWW-Authenticate`	Authenticate, then retry
403 Forbidden	Authenticated but not allowed	Lacks permission or scope	Do not retry
404 Not Found	No such resource (or hidden from you)	Unknown ID; also to avoid revealing existence	Do not retry
409 Conflict	Conflicts with current state	Duplicate unique key; version conflict	Re-read, resolve, retry
412 Precondition Failed	`If-Match` ETag did not match	Optimistic concurrency failure	Re-read and retry
422 Unprocessable Content	Well-formed but semantically invalid	Validation errors (end date before start date)	Fix input
429 Too Many Requests [RFC 6585]	Rate limited	Client exceeded quota; send `Retry-After`	Back off
500 Internal Server Error	Unexpected server bug	Unhandled exception	Retry only if idempotent, with backoff
502 / 504	Bad gateway / gateway timeout	An upstream failed or timed out	Retry idempotent requests with backoff
503 Service Unavailable	Temporarily overloaded or in maintenance	Load shedding; send `Retry-After`	Back off and retry

COMMON MISTAKE

Returning `200 OK` with `{"error": "..."}` in the body. Every intermediary and monitoring tool then counts failures as successes, retries never happen, and error-rate SLOs (§39.6) silently read 100%. Use the status code for the class of outcome and the body for detail.

8.6.5 Pagination: offset versus cursor

A collection such as `/orders` may hold millions of items. Returning all of them would take seconds, megabytes and a lot of database work, so APIs return **pages**.

Offset pagination

The client asks for `?limit=20&offset=40` (or `?page=3&per_page=20`). The SQL is straightforward:

```
-- Page 3 of 20 items, newest first (PostgreSQL)
SELECT id, title, created_at
FROM articles
ORDER BY created_at DESC, id DESC
LIMIT 20 OFFSET 40;
```

It is simple and supports "jump to page 37". It has two serious problems:

1. **Cost grows with depth.** The database cannot jump to row 40; it must produce and discard the first 40 rows in sorted order, then return 20. Deep pages read offset+limit rows.
2. **Results shift under concurrent writes.** If a new article is inserted while the user reads page 1, every item moves down one place: the last item of page 1 appears again at the top of page 2. Deletions cause items to be skipped silently.

Cursor (keyset) pagination

Instead of "skip N rows", the client says "give me rows *after this one*". The server returns an opaque **cursor** encoding the sort key of the last row it sent; the next query uses it in a `WHERE` clause that an index can seek to directly.

```sql
        -- Supporting index: the sort order, with id as a unique tie-breaker
        CREATE INDEX idx_articles_created_id ON articles (created_at DESC, id DESC);

        -- First page
        SELECT id, title, created_at
        FROM articles
        ORDER BY created_at DESC, id DESC
        LIMIT 21;                       -- one extra row tells us whether a next page
exists

        -- Next page: $1, $2 = created_at and id of the last row of the previous page
        SELECT id, title, created_at
        FROM articles
        WHERE (created_at, id) < ($1, $2)    -- row-value comparison: seek, don't skip
        ORDER BY created_at DESC, id DESC
        LIMIT 21;
```

What to notice:

- The `id` tie-breaker makes the sort order **total**. Without it, two rows with the same `created_at` could be skipped or duplicated at a page boundary.
- The row-value comparison `(created_at, id) < ($1, $2)` means “earlier timestamp, or same timestamp with smaller id”. PostgreSQL, MySQL 8 and SQLite support it; on databases that do not, expand it to `created_at < $1 OR (created_at = $1 AND id < $2)`.
- Fetching `limit + 1` rows answers “is there a next page?” without a separate `COUNT(*)`, which is itself expensive on large tables.

FORMULA

Rows touched per page

$$R_{\text{offset}} = o + \ell \qquad R_{\text{keyset}} \approx \ell + \log_b n$$

Symbol	Meaning	Units
o	Offset (rows skipped)	rows
ℓ	Page size (limit)	rows
n	Rows in the table or index	rows
b	B-tree fan-out (entries per index page, typically hundreds)	entries

WORKED EXAMPLE

Worked example — Page 50,000 of an activity feed. Table of $n = 10^7$ rows, page size $\ell = 20$, user (or a scraper) requests page 50,000, so $o = 49,999 \times 20 = 999,980$.

1. Offset: $R = 999,980 + 20 = 1,000,000$ index entries walked, 999,980 of them discarded.
2. Keyset: one B-tree descent of $\log_{200} 10^7 \approx 3$ levels, then 21 entries: about 24 entries.
3. Ratio: $10^6/24 \approx 40,000\times$ more work for the offset query.

Interpretation. Offset pagination is fine for shallow pages (an admin table with a few thousand rows) but degrades linearly with depth and is an easy denial-of-service vector. **Engineering implication:** use cursors for feeds, timelines, logs, sync and any public API; cap `offset` if you must offer it.

Table 8.8 — Offset versus cursor pagination.

Aspect	Offset / page number	Cursor (keyset)
Request	`?limit=20&offset=40`	`?limit=20&cursor=eyJ…`
Cost of deep pages	Grows linearly with offset	Constant (index seek)
Stable under inserts/deletes	No: duplicates and skips	Yes, relative to the last seen key
Jump to arbitrary page	Yes	No (only next/previous)
Total count	Often shown (costly `COUNT(*)`)	Usually omitted
Sort flexibility	Any `ORDER BY`	Needs an index matching the sort key plus a unique tie-breaker
Choose when	Small, mostly static datasets; UI needs page numbers	Large or fast-changing collections; infinite scroll; API sync

Code example

The following FastAPI service implements cursor pagination with an opaque, base64url-encoded cursor, optional filtering, and RFC 9457 error responses (§8.6.7). It runs as written with `pip install fastapi uvicorn` and `uvicorn articles_api:app`.

```
        # articles_api.py — run with:  pip install fastapi uvicorn  &&  uvicorn
articles_api:app
        import base64
        import json
        import sqlite3

        from fastapi import FastAPI, Query, Request
        from fastapi.responses import JSONResponse

        app = FastAPI(title="Articles API", version="1.0.0")

        # In-memory database so the example is self-contained.
        db = sqlite3.connect(":memory:", check_same_thread=False)
        db.row_factory = sqlite3.Row
        db.executescript("""
        CREATE TABLE articles (
            id         INTEGER PRIMARY KEY,
            title      TEXT    NOT NULL,
            status     TEXT    NOT NULL,
            created_at TEXT    NOT NULL           -- ISO-8601 UTC timestamp
        );
        CREATE INDEX idx_articles_created_id ON articles (created_at DESC, id DESC);
        """)
        db.executemany(
            "INSERT INTO articles (id, title, status, created_at) VALUES (?, ?, ?, ?)",
            [(i, f"Article {i}", "draft" if i % 3 == 0 else "published",
              f"2026-01-{i % 28 + 1:02d}T10:00:00Z") for i in range(1, 101)],
        )


        class Problem(Exception):
            """An error that is rendered as RFC 9457 problem details."""
            def __init__(self, status: int, title: str, detail: str,
                         type_: str = "about:blank"):
                self.status, self.title, self.detail, self.type_ = status, title, de-
tail, type_


        @app.exception_handler(Problem)
        async def problem_handler(request: Request, exc: Problem) -> JSONResponse:
            return JSONResponse(
                status_code=exc.status,
                media_type="application/problem+json",
                content={"type": exc.type_, "title": exc.title, "status": exc.status,
                         "detail": exc.detail, "instance": request.url.path},
            )


        def encode_cursor(created_at: str, row_id: int) -> str:
            raw = json.dumps([created_at, row_id]).encode()
            return base64.urlsafe_b64encode(raw).decode().rstrip("=")


        def decode_cursor(cursor: str) -> tuple[str, int]:
            try:
                padded = cursor + "=" * (-len(cursor) % 4)
```

```
            created_at, row_id = json.loads(base64.urlsafe_b64decode(padded))
            return str(created_at), int(row_id)
        except Exception:
            raise Problem(400, "Invalid cursor",
                          "The cursor parameter is malformed or expired.",
                          "https://api.example.com/problems/invalid-cursor")


    @app.get("/v1/articles")
    def list_articles(
        limit: int = Query(20, ge=1, le=100),
        cursor: str | None = None,
        status: str | None = Query(None, pattern="^(draft|published)$"),
    ):
        where, params = [], []
        if status is not None:
            where.append("status = ?")
            params.append(status)
        if cursor is not None:
            created_at, last_id = decode_cursor(cursor)
            where.append("(created_at, id) < (?, ?)")      # keyset predicate
            params += [created_at, last_id]

        sql = "SELECT id, title, status, created_at FROM articles"
        if where:
            sql += " WHERE " + " AND ".join(where)
        sql += " ORDER BY created_at DESC, id DESC LIMIT ?"
        params.append(limit + 1)                            # fetch one extra row

        rows = db.execute(sql, params).fetchall()
        has_more = len(rows) > limit
        rows = rows[:limit]
        next_cursor = (encode_cursor(rows[-1]["created_at"], rows[-1]["id"])
                       if has_more else None)
        return {"data": [dict(r) for r in rows], "next_cursor": next_cursor}


    @app.get("/v1/articles/{article_id}")
    def get_article(article_id: int):
        row = db.execute("SELECT id, title, status, created_at FROM articles WHERE
id = ?",
                         (article_id,)).fetchone()
        if row is None:
            raise Problem(404, "Not Found", f"No article with id {article_id} ex-
ists.")
        return dict(row)
```

What to notice:

- **The cursor is opaque.** Clients must treat it as a token, not parse it. That leaves you free to change its contents (add a sort field, sign or encrypt it, add an expiry) without a breaking change. In production, sign the cursor with an HMAC (as in §8.10.2) so clients cannot forge one that seeks to arbitrary positions.

- **`limit` is bounded** (`le=100`). An unbounded page size is a denial-of-service hole. FastAPI rejects `limit=500` with `422` automatically.
- **SQL parameters are bound with `?`**, never formatted into the string, which prevents SQL injection (§10.7).
- **Filters and cursors compose**: the `status` predicate and the keyset predicate are both in the `WHERE` clause. A production version would also bind the cursor to the filter that produced it, so a cursor from `status=draft` cannot be replayed against `status=published`.

Exercising it with curl:

```
        curl -s 'http://localhost:8000/v1/articles?limit=2'
        # {"data":[{"id":83,"title":"Article
83","status":"published","created_at":"2026-01-28T10:00:00Z"},
        #
{"id":55,...}],"next_cursor":"WyIyMDI2LTAxLTI4VDEwOjAwOjAwWiIsIDU1XQ"}

        curl -s 'http://localhost:8000/v1/articles?
limit=2&cursor=WyIyMDI2LTAxLTI4VDEwOjAwOjAwWiIsIDU1XQ'

        curl -si 'http://localhost:8000/v1/articles/999'
        # HTTP/1.1 404 Not Found
        # content-type: application/problem+json
        # {"type":"about:blank","title":"Not Found","status":404,
        #  "detail":"No article with id 999 exists.","instance":"/v1/articles/999"}
```

8.6.6 Filtering and sorting

Common conventions, any of which is fine if applied consistently:

- **Filtering by field:** `GET /orders?status=shipped&customer_id=c_17`. For ranges: `created_after=2026-01-01T00:00:00Z` or `created_at[gte]=…`.
- **Sorting:** `?sort=-created_at,id` (minus for descending).
- **Sparse fieldsets:** `?fields=id,title` to reduce payload size, a partial answer to the over-fetching that motivates GraphQL.
- **Search** is a different operation from filtering. Free-text search belongs on a search endpoint backed by a search engine (Chapter 16), not on `LIKE '%term%'` against the primary database.

Every filter and sort combination you allow is a query shape your database must serve efficiently. Allow-list the fields, and make sure each permitted combination has a supporting index; otherwise one creative client can trigger a full table scan per request. With cursor pagination, the cursor must encode the values of *every* sort column plus the tie-breaker.

8.6.7 Error format: RFC 9457 problem details

Ad-hoc error bodies (`{"err": 1}`, `{"message": "..."}`, `{"errors": [...]}`) force every client to write custom parsing per API. RFC 9457 (which obsoletes RFC 7807) defines a standard JSON shape served with media type `application/problem+json` [RFC 9457]:

```
{
  "type": "https://api.example.com/problems/insufficient-funds",
  "title": "Insufficient funds",
  "status": 403,
  "detail": "Your balance is 30.00 EUR; the transfer requires 50.00 EUR.",
  "instance": "/v1/transfers/7f9c2e",
  "balance_cents": 3000,
  "required_cents": 5000
}
```

What to notice:

- **`type`** is a URI that identifies the *kind* of problem; clients branch on it. It should ideally resolve to human-readable documentation. When omitted it defaults to `about:blank`, in which case `title` should be the HTTP status phrase.
- **`title`** is a short, stable summary of the type; **`detail`** explains *this* occurrence; **`instance`** identifies this occurrence (a request path or an error ID you can grep logs for).
- **Extension members** (`balance_cents`) carry machine-readable specifics. Never put stack traces, SQL or internal hostnames in error bodies.

8.6.8 HATEOAS, briefly

HATEOAS (Hypermedia as the Engine of Application State) means responses include links telling the client what it can do next, so the client follows links instead of constructing URLs:

```
{
  "id": "ord_42",
  "status": "placed",
  "_links": {
    "self":   { "href": "/v1/orders/ord_42" },
    "cancel": { "href": "/v1/orders/ord_42/cancellation", "method": "POST" },
    "items":  { "href": "/v1/orders/ord_42/items" }
  }
}
```

The `cancel` link disappears once the order ships, so the server, not the client, encodes the workflow rules. Full HATEOAS is rare in public JSON APIs because most clients are written against documentation and hard-code URL templates anyway. A lightweight and

widely used piece of it is pagination links in the HTTP `Link` header (`Link: <…?cursor=abc>; rel="next"`).

Real-world industry applications

INDUSTRY EXAMPLE

Stripe and GitHub list endpoints. Stripe's list endpoints use cursor-style pagination: the client passes the ID of the last object it saw in `starting_after` (or `ending_before`) together with `limit`, and the response carries `has_more` [Stripe API Docs: Pagination]. GitHub's REST API returns pagination URLs in the HTTP `Link` header with `rel="next"`, `rel="last"` and similar relations, and some endpoints use cursor-based `before` / `after` parameters [GitHub Docs: Using pagination in the REST API]. Both choices follow from the same problem: collections that are large and constantly changing, accessed by automated clients that page to the end. *Evidence: Primary* — [Stripe API Docs: Pagination]; [GitHub Docs: Using pagination in the REST API]

Failure scenarios

Table 8.9 — REST API failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Duplicate orders or charges	Client retried a timed-out `POST`	Duplicate rows with the same payload seconds apart	Refund / merge duplicates	Idempotency keys on all non-idempotent writes (§22.15)
Database CPU pinned by one client	Deep `offset` pages or unindexed sort	Slow-query log; per-client cost metrics	Rate-limit the client	Cursor pagination; allow-listed, indexed sorts
Lost update	Two clients `PUT` the same resource concurrently	Users report reverted changes	Restore from audit log	`ETag` + `If-Match`, return `412` on mismatch
Clients crash on new error	Error body shape changed	Client error reports by version	Roll back	One stable error format (RFC 9457) from day one

Common misconceptions

COMMON MISTAKE

"`PUT` is for updates, `POST` is for creates." The real distinction is idempotency and who chooses the URI. `PUT /articles/my-slug` creates *or* replaces at a client-chosen URI, idempotently. `POST /articles` lets the server choose the URI and is not idempotent. Partial updates are `PATCH`.

COMMON MISTAKE

“`401` means the user lacks permission.” `401 Unauthorized` (misleadingly named) means *not authenticated*. Lacking permission while authenticated is `403 Forbidden`. Many APIs return `404` instead of `403` for resources the caller should not even know exist.

8.7 GraphQL

8.7.1 Schema, operations and resolvers

DEFINITION

Definition — GraphQL. A query language for APIs and a runtime for executing those queries against a strongly typed **schema**. The client sends a document describing exactly the fields it wants; the server returns JSON of the same shape [GraphQL Spec].

First-principles explanation

A REST API is a set menu: `/users/7` returns whatever the kitchen decided a “user” contains. If you need the user, their last five posts and each post’s comment count, you either order three dishes (three round trips — **under-fetching**) or receive a platter of fields you never eat (**over-fetching**). GraphQL is ordering à la carte from one waiter: you write down precisely what you want, nested as deep as you like, and get exactly that back in one trip.

Why does this exist?

Facebook built GraphQL in 2012 for its native mobile apps, where many REST round trips over cellular networks made news-feed screens slow and every new screen needed a new endpoint; it was open-sourced in 2015 and is now governed by the GraphQL Foundation [Byron 2015]. The specification’s most recent edition at the time of writing is September 2025 [GraphQL Spec].

How it works internally

1. **Schema.** The server publishes types in the Schema Definition Language (SDL). Three root types are entry points: `Query` (reads), `Mutation` (writes) and `Subscription` (event streams).
2. **Parse and validate.** The server parses the query document into an abstract syntax tree and validates every field and argument against the schema *before* running any-

thing. Typos fail fast with a precise error.

3. **Execute.** The executor walks the query tree. For every field it calls a **resolver**: a function `(parent, args, context) → value`. The resolver for `Query.posts` returns a list of post objects; then, for *each* post, the resolver for `Post.author` runs with that post as `parent`.
4. **Respond.** The result is JSON shaped like the query, plus an `errors` array if some fields failed. Over HTTP, GraphQL servers usually expose one endpoint (`POST /graphql`) and often return `200 OK` even when `errors` is non-empty, which matters for monitoring.

```
"""A blog: users write posts; posts have comments."""
type Query {
  user(id: ID!): User
  posts(first: Int = 10, after: String): PostConnection!
}

type Mutation {
  createPost(input: CreatePostInput!): CreatePostPayload!
}

type User {
  id: ID!
  name: String!
  posts(first: Int = 10): [Post!]!
}

type Post {
  id: ID!
  title: String!
  body: String!
  author: User!
  comments(first: Int = 10): [Comment!]!
  publishedAt: String
}

type Comment {
  id: ID!
  text: String!
  author: User!
}

# Relay-style cursor pagination
type PostConnection {
  edges: [PostEdge!]!
  pageInfo: PageInfo!
}

type PostEdge {
  cursor: String!
  node: Post!
}

type PageInfo {
  hasNextPage: Boolean!
  endCursor: String
}

input CreatePostInput {
  title: String!
  body: String!
}

type CreatePostPayload {
  post: Post
```

```
  errors: [String!]!
}
```

A client query and the matching curl call:

```
curl -s https://api.example.com/graphql \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer eyJhbGciOi...' \
  -d '{"query":"query RecentPosts { posts(first: 5) { edges { node { title au-
thor { name } } } pageInfo { hasNextPage endCursor } } }"}'
```

What to notice:

- `!` means non-null; `[Post!]!` is a non-null list of non-null posts. Nullability is part of the contract: making a nullable field non-null later is safe for clients, the reverse is not.
- Pagination uses the cursor model of §8.6.5 (`first`, `after`, `endCursor`), in the "connection" shape popularized by Facebook's Relay client.
- Mutations return a payload type with a domain `errors` list, so business-rule failures are typed data rather than transport errors.

8.7.2 The N+1 problem and DataLoader

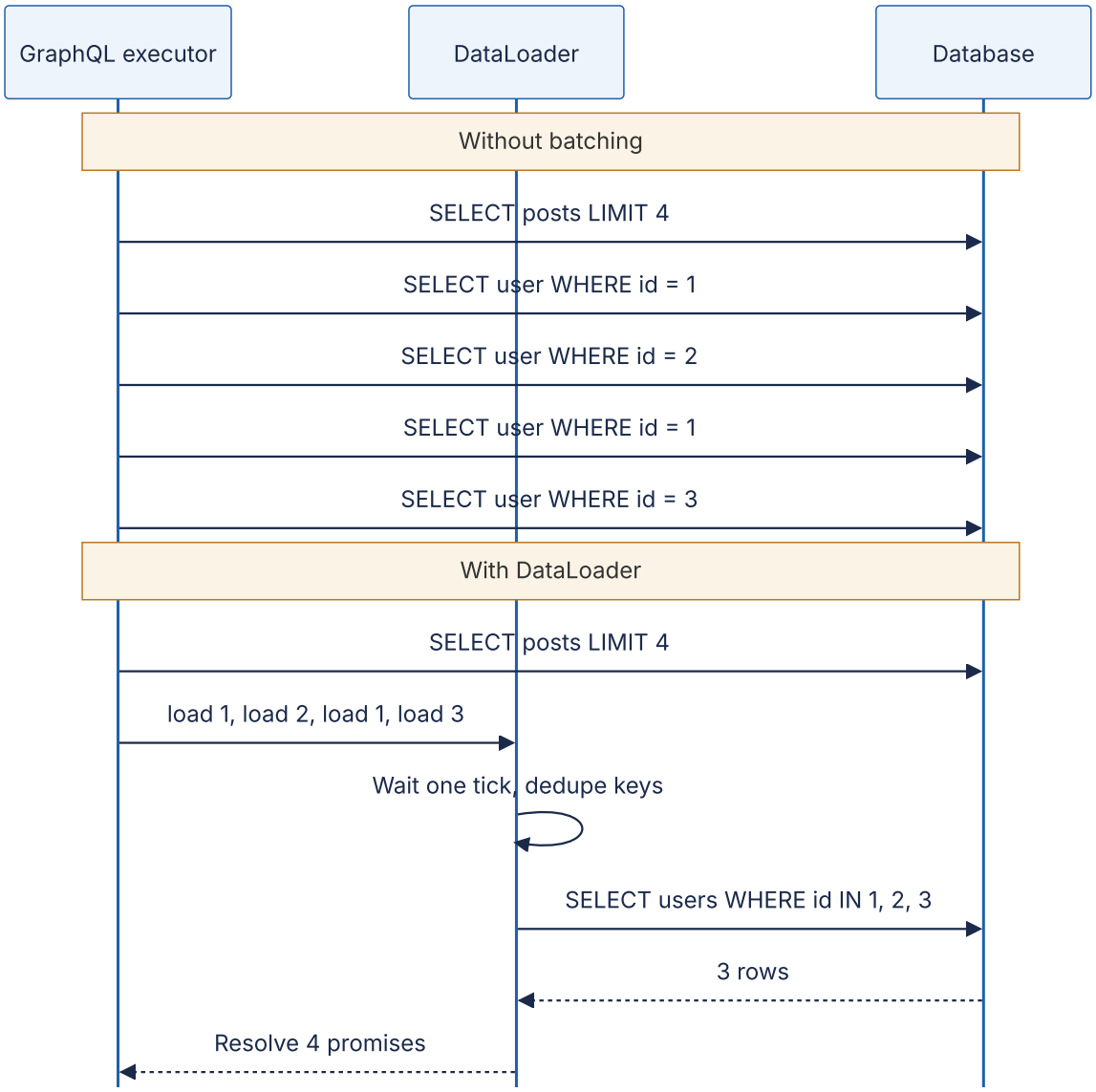
The resolver model is elegant but naive. For the query above, `posts` runs one SQL query that returns 5 posts; then `Post.author` runs **once per post**, each issuing `SELECT … FROM users WHERE id = ?`. That is $1 + N$ queries — the **N+1 problem**. Nesting multiplies it.

> WORKED EXAMPLE
>
> **Worked example — Query count for a nested feed.** A screen asks for 50 posts, the first 10 comments on each, and each comment's author. Each database round trip costs 1 ms.
>
> 1. Naive resolvers: 1 (posts) +50 (comments per post) $+50 \times 10 = 500$ (comment authors) $= 551$ queries, about 551 ms if executed one after another.
> 2. With batching per tree level: 1 query for posts, 1 for all comments `WHERE post_id IN (…50 ids…)`, 1 for all authors `WHERE id IN (…)`: 3 queries, about 3 ms.
>
> **Interpretation.** The difference is two orders of magnitude, and it grows with page size. **Engineering implication:** a GraphQL server without batching is a load generator pointed at your database.

DataLoader is the standard fix, published by Facebook as a small JavaScript library and re-implemented in every major language [DataLoader]. Resolvers do not query directly; they call `loader.load(id)`, which returns a promise. DataLoader collects every `load` is-

sued during one tick of the event loop, then calls a **batch function** once with all the keys, and resolves each promise from the result. It also memoizes keys for the duration of the request, so the same author requested 30 times is fetched once.

Figure 8.4 — N+1 resolution versus DataLoader batching.



Reading the diagram. In the first half, the executor issues one database query per post for the author field: five synchronous round trips, one of them a duplicate. In the second half, the four author resolvers each call `load` on a per-request DataLoader; it waits until the current tick ends, deduplicates the keys to `{1, 2, 3}`, makes one `IN` query, and fans the rows back out to the four waiting resolvers. The storage boundary is the database. Failure point: if the batch query fails, every waiting resolver must receive the error. Scaling: batching turns $O(N)$ round trips into $O(\text{depth})$.

A minimal DataLoader in Python's `asyncio` shows the mechanism:

```
        import asyncio


        class DataLoader:
            """Coalesce every load(key) issued in the same event-loop tick into one
batch call."""

            def __init__(self, batch_fn):
                self.batch_fn = batch_fn          # async fn: list[key] -> list[value],
same order
                self.cache = {}                   # key -> Future (per-request memoiza-
tion)
                self.queue = []                   # keys waiting for the next dispatch

            def load(self, key):
                if key in self.cache:
                    return self.cache[key]
                fut = asyncio.get_running_loop().create_future()
                self.cache[key] = fut
                self.queue.append(key)
                if len(self.queue) == 1:          # first key in this tick: schedule a
dispatch
                    asyncio.get_running_loop().call_soon(
                        lambda: asyncio.ensure_future(self._dispatch()))
                return fut

            async def _dispatch(self):
                keys, self.queue = self.queue, []
                try:
                    values = await self.batch_fn(keys)
                    for k, v in zip(keys, values):
                        self.cache[k].set_result(v)
                except Exception as exc:           # fail every waiter, not just one
                    for k in keys:
                        self.cache[k].set_exception(exc)


        queries = []
        AUTHORS = {1: "Ada", 2: "Linus", 3: "Grace"}


        async def fetch_authors(ids):
            queries.append(f"SELECT id, name FROM authors WHERE id IN {tuple(ids)}")
            return [AUTHORS.get(i) for i in ids]


        async def main():
            loader = DataLoader(fetch_authors)    # create one loader PER REQUEST
            posts = [{"id": 10, "author_id": 1}, {"id": 11, "author_id": 2},
                     {"id": 12, "author_id": 1}, {"id": 13, "author_id": 3}]
            names = await asyncio.gather(*(loader.load(p["author_id"]) for p in posts))
            print(names)     # ['Ada', 'Linus', 'Ada', 'Grace']
            print(queries)   # ['SELECT id, name FROM authors WHERE id IN (1, 2, 3)']
```

```
        asyncio.run(main())
```

What to notice:

- `call_soon` defers the dispatch until every resolver running in the current tick has called `load`; that is the whole trick.
- The batch function must return values **in the same order as the keys**, with `None` (or an error) for missing keys.
- The loader is created **per request**. A process-wide loader would leak one user's cached data to another user and would never notice updates.

8.7.3 Caching and security challenges

REST gets HTTP caching almost for free: `GET /posts/7` has a URL, so browsers and CDNs can cache it by URL with `Cache-Control` and `ETag`. GraphQL makes this harder:

- **One URL, usually `POST`.** CDNs and browsers do not cache `POST` responses by default, and the cache key would have to include the query body.
- **Every query is unique.** Two clients requesting slightly different fields produce different responses, so shared-cache hit ratios are low.
- **Mixed freshness.** One response may combine a user's name (cacheable for hours) and their unread count (seconds); the whole response gets the most restrictive policy.

Mitigations: **persisted queries** (clients send a hash of a pre-registered query via `GET /graphql?id=<sha256>`, which is cacheable by URL and also blocks arbitrary queries); **normalized client caches** (Apollo Client and Relay store objects by type and ID and reuse them across queries); and **server-side caching in resolvers or DataLoader batch functions** using the techniques of Chapter 20.

The same flexibility creates a security problem: one small request can ask for an enormous result (`friends { friends { friends { … } } }`). Production servers enforce **depth limits**, **query cost analysis** (estimate cost from the schema and arguments before executing; reject or rate-limit by cost, not by request count), pagination maximums on every list field, and timeouts. GitHub's public GraphQL API, for example, rate-limits by a computed per-query cost rather than by request count [GitHub Docs: GraphQL rate limits].

> COMMON MISTAKE
>
> **"GraphQL is faster than REST."** GraphQL removes client round trips and overfetching; it does not make the server faster. Without batching and cost limits it is frequently slower and harder to protect. It moves complexity from the client to the server.

8.8 gRPC and RPC

8.8.1 Remote procedure calls

DEFINITION

Definition — Remote procedure call (RPC). A mechanism that lets a program call a function in another process, usually on another machine, with syntax that resembles a local function call. Generated **stubs** serialize the arguments, send them over the network, and deserialize the result [Birrell and Nelson 1984].

Instead of resources and verbs, RPC exposes **operations**: `GetOrder(order_id)`, `ReserveInventory(sku, qty)`. That maps naturally to how programmers think inside one codebase, which is why internal service-to-service APIs often use RPC. The danger is the illusion of locality: a remote call can be a million times slower than a local one, can fail halfway, and can succeed on the server while the response is lost (see the fallacies of distributed computing, §24.5). Good RPC frameworks make these differences explicit through deadlines, status codes and retries.

8.8.2 gRPC: Protocol Buffers over HTTP/2

DEFINITION

Definition — gRPC. An open-source RPC framework, originally developed at Google and released in 2015, that uses Protocol Buffers as its interface definition language and message format and HTTP/2 as its transport [gRPC Docs].

You describe the service once in a `.proto` file; the `protoc` compiler generates client stubs and server interfaces in a dozen or so languages.

```
syntax = "proto3";

package shop.v1;

import "google/protobuf/timestamp.proto";

// The service contract: four methods, one per gRPC streaming mode.
service OrderService {
  // Unary: one request, one response.
  rpc GetOrder(GetOrderRequest) returns (Order);
  // Server streaming: one request, a stream of responses.
  rpc WatchOrderStatus(GetOrderRequest) returns (stream OrderStatusEvent);
  // Client streaming: a stream of requests, one response.
  rpc UploadLineItems(stream LineItem) returns (UploadSummary);
  // Bidirectional streaming: both sides stream independently.
  rpc Chat(stream ChatMessage) returns (stream ChatMessage);
}

message GetOrderRequest {
  string order_id = 1;
}

enum OrderStatus {
  ORDER_STATUS_UNSPECIFIED = 0;   // proto3 enums must start at zero
  ORDER_STATUS_PLACED = 1;
  ORDER_STATUS_SHIPPED = 2;
  ORDER_STATUS_DELIVERED = 3;
}

message Order {
  string order_id = 1;
  string customer_id = 2;
  repeated LineItem items = 3;
  OrderStatus status = 4;
  google.protobuf.Timestamp created_at = 5;
  reserved 6;                     // field 6 was deleted; never reuse the number
  reserved "coupon_code";
}

message LineItem {
  string sku = 1;
  int32 quantity = 2;
  int64 unit_price_cents = 3;    // money as integer cents, never float
}

message OrderStatusEvent {
  string order_id = 1;
  OrderStatus status = 2;
}

message UploadSummary {
  int32 items_received = 1;
}

message ChatMessage {
```

```
  string text = 1;
}
```

What to notice:

- The numbers (`= 1`, `= 2`) are **field numbers**, not default values. They, not the names, are written on the wire (§8.9.2), so they must never change or be reused; `reserved` enforces that for deleted fields.
- The version lives in the package name (`shop.v1`), so a breaking redesign becomes `shop.v2` alongside it (§8.15).
- `stream` on the request, the response or both selects the streaming mode.

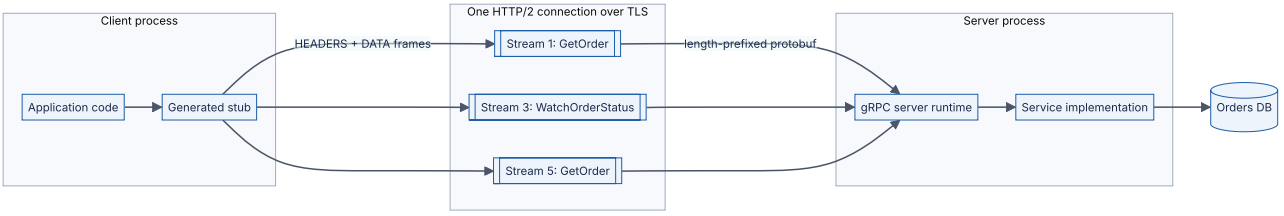
How it works internally

Each call maps onto HTTP/2 (§7.9):

1. The client opens (or reuses) one long-lived HTTP/2 connection, called a **channel**, to the server.
2. Each RPC becomes an HTTP/2 **stream**: a `POST` to the path `/shop.v1.OrderService/GetOrder` with `content-type: application/grpc` and, if the client set one, a `grpc-timeout` header carrying the deadline.
3. Each message is **length-prefixed**: 1 byte compression flag + 4-byte big-endian length + the Protobuf bytes, carried in HTTP/2 DATA frames.
4. The server replies with headers, zero or more length-prefixed messages, and **trailers** containing `grpc-status` (0 = OK, 4 = DEADLINE_EXCEEDED, 14 = UNAVAILABLE, and so on) and an optional `grpc-message`.

Because HTTP/2 multiplexes many streams over one TCP connection, hundreds of concurrent RPCs share a connection without head-of-line blocking at the HTTP layer (TCP-level head-of-line blocking remains; §7.7).

Figure 8.5 — gRPC call mapped onto an HTTP/2 connection.



Reading the diagram. Application code calls a generated stub as if it were local. The stub serializes the request to Protobuf and sends it as HEADERS and DATA frames on a new stream of a single shared HTTP/2 connection; three concurrent calls occupy streams 1, 3 and 5 (client-initiated HTTP/2 streams use odd numbers). The server runtime reassembles each stream, deserializes the message, invokes the implementation and streams the response back with a trailing `grpc-status`. Unary calls are synchronous

request/response; stream 3 stays open while the server pushes events. Storage boundary: the database behind the implementation. Failure points: a single connection pins all of a client's traffic to one server, which defeats layer-4 load balancers; use layer-7 (HTTP/2-aware) proxies or client-side load balancing (§19.10). Scaling: add servers behind an L7 balancer that balances per stream, not per connection.

8.8.3 Streaming modes

Table 8.10 — The four gRPC call types.

Mode	Shape	Example use
Unary	1 request → 1 response	Fetch an order; most calls
Server streaming	1 request → stream of responses	Order status updates; large result sets in chunks; log tailing
Client streaming	Stream of requests → 1 response	Uploading telemetry or a file in chunks, then a summary
Bidirectional streaming	Two independent streams	Chat, collaborative editing, interactive speech recognition

Streams are flow-controlled by HTTP/2, so a slow receiver applies backpressure to a fast sender (§22.10) instead of buffering without bound.

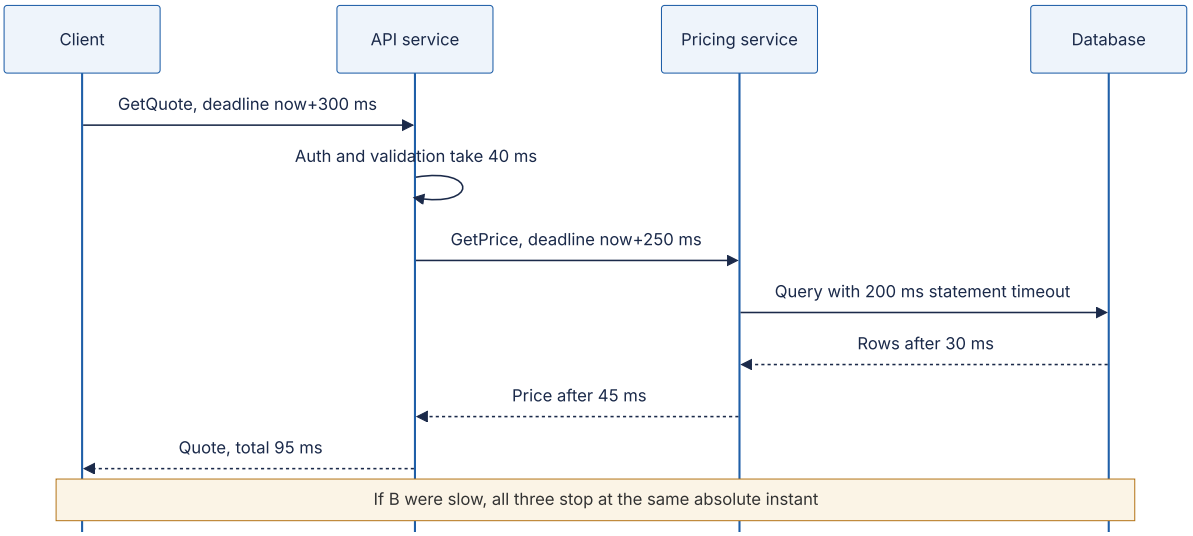
8.8.4 Deadlines and their propagation

DEFINITION

Definition — Deadline. An absolute point in time by which a client needs an RPC to complete. gRPC transmits it as a remaining timeout; when it passes, the client stops waiting with `DEADLINE_EXCEEDED` and the server can see that the call was canceled [gRPC Docs: Deadlines].

Without deadlines, a slow dependency makes callers wait indefinitely, holding threads, memory and connections until the whole call graph is exhausted (Chapter 22). gRPC's default is **no deadline**, so you must set one on every call. A deadline, unlike a per-hop timeout, should be **propagated**: if the front end has 300 ms left and has already spent 40 ms, it passes the remaining ~260 ms downstream, minus a margin for its own post-processing, so no service keeps working on an answer nobody is waiting for.

Figure 8.6 — Deadline propagation through a call chain.



Reading the diagram. The client sets an absolute deadline of 300 ms. The API service spends 40 ms, then calls the pricing service with the *remaining* budget minus a 10 ms reserve for building its own response. The pricing service in turn bounds its database query below its own remaining time. All calls are synchronous. Failure point: if any hop sets a longer timeout than its caller’s remaining budget, that hop does work whose result will be discarded. Scaling: deadlines plus cancellation free resources quickly under overload, which keeps a slow dependency from cascading.

Code example

```
        # pip install grpcio grpcio-tools
        # python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=.
orders.proto
        import grpc
        import orders_pb2
        import orders_pb2_grpc

        with grpc.insecure_channel("localhost:50051") as channel:   # use TLS creden-
tials in production
            stub = orders_pb2_grpc.OrderServiceStub(channel)

            # Unary call with a 200 ms deadline.
            try:
                order = stub.GetOrder(orders_pb2.GetOrderRequest(order_id="o-42"),
timeout=0.2)
                print(order.order_id, orders_pb2.OrderStatus.Name(order.status))
            except grpc.RpcError as err:
                if err.code() == grpc.StatusCode.DEADLINE_EXCEEDED:
                    print("gave up after 200 ms; serve a fallback")
                else:
                    raise

            # Server streaming: iterate over responses as they arrive.
            for event in stub.WatchOrderStatus(orders_pb2.GetOrderRequest(order_id="o-
42"),
                                               timeout=30):
                print(orders_pb2.OrderStatus.Name(event.status))
```

What to notice:

- `timeout=0.2` is converted by the library into the `grpc-timeout` header; the server can read the remaining time with `context.time_remaining()` and pass it on.
- Errors are typed status codes, not free-text. Retry `UNAVAILABLE` with backoff; do not retry `INVALID_ARGUMENT`; retry `DEADLINE_EXCEEDED` only if the operation is idempotent and budget remains.
- Browsers cannot speak raw gRPC because they do not expose HTTP/2 trailers and framing to JavaScript; **gRPC-Web** with a translating proxy, or a REST/JSON gateway, is used for browser clients.

Industry application

INDUSTRY EXAMPLE

Google Stubby → gRPC. gRPC's documentation describes it as the open-source successor to Stubby, the general-purpose RPC infrastructure Google used internally to connect its services, and it is now a Cloud Native Computing Foundation project [gRPC Docs]. The design goals that carried over — a compact binary IDL-based format, generated stubs in many languages, deadlines and cancellation as first-class concepts — come from operating very large internal call graphs where a slow or hung call would otherwise cascade. *Evidence: Primary* — [gRPC Docs]

8.8.5 Choosing REST, GraphQL or gRPC

Table 8.11 — REST versus GraphQL versus gRPC.

Row	REST	GraphQL	gRPC
What it is	Resource-oriented HTTP API style	Typed query language + runtime over one endpoint	RPC framework: Protobuf IDL over HTTP/2
How it works	URIs + standard methods + status codes; usually JSON	Client sends a query; resolvers assemble the response tree	Generated stubs; length-prefixed binary messages on HTTP/2 streams
Advantages	Universal tooling; HTTP caching and CDNs; easy to debug with curl	Exactly the data a screen needs in one round trip; strong typing; introspection	Compact and fast; streaming; deadlines; strict contracts and codegen
Disadvantages	Over/under-fetching; many round trips for complex screens; loose contracts unless documented	Hard to cache; N+1 risk; query-cost security; errors hidden in 200s	Not native in browsers; binary is harder to inspect; needs L7-aware load balancing
Performance	Good; JSON parsing cost; benefits from HTTP caching	Fewer client round trips; server work per request is higher and variable	Smallest payloads and lowest serialization cost of the three
Operational complexity	Low	Medium–high (cost limits, batching, schema governance)	Medium (proto management, L7 proxies, gRPC-Web for browsers)
Cost	Low; bandwidth higher than binary	Server CPU for resolution; engineering time	Low bandwidth and CPU; tooling investment
Use cases	Public APIs, CRUD services, webhooks, cacheable content	Product frontends aggregating many entities; mobile apps with diverse screens	Internal service-to-service calls; low-latency or streaming backends; polyglot microservices
Choose when	Third parties consume your API or responses are cacheable	Many client types need different shapes of a rich graph of data	You control both ends and need efficiency, streaming or strict typing

A common production pattern uses all three: gRPC between internal services, GraphQL or a BFF for first-party frontends, and REST for public and partner APIs.

8.9 Serialization: JSON, Protocol Buffers, Avro and Thrift

8.9.1 Why encoding matters

DEFINITION

Definition — Serialization (encoding). Translating an in-memory data structure into a sequence of bytes for storage or transmission; **deserialization (decoding)** is the reverse [Kleppmann 2017, ch. 4].

Objects in memory are full of pointers that mean nothing to another process. To send an order across the network you must flatten it into bytes that the receiver, possibly written in another language and deployed at a different time, can rebuild. Three questions decide the format: **How big and how fast?** (bandwidth and CPU at millions of messages per second), **Is it self-describing or schema-driven?**, and **How does it survive change?** — the last is the most important in practice, because in any system with rolling deployments, old and new code run side by side and read each other's data.

8.9.2 The formats

JSON is text, human-readable, universally supported and self-describing: field names travel with every message. Its costs are size (names repeated in every record), parsing CPU, and a weak type system: no distinction between integers and floats, no binary type (bytes must be base64-encoded, +33%), and numbers above 2^{53} lose precision in JavaScript, which is why many APIs send 64-bit IDs as strings.

Protocol Buffers (Protobuf) is Google's schema-driven binary format [Protocol Buffers Docs]. Each field is written as a **tag** followed by a value. The tag is a varint holding `(field_number << 3) | wire_type`, where the wire type tells the reader how to find the end of the value even if it does not recognize the field: 0 = VARINT, 1 = I64 (8 fixed bytes), 2 = LEN (length-delimited: strings, bytes, embedded messages, packed repeated fields), 5 = I32 (4 fixed bytes); 3 and 4 are deprecated group markers.

DEFINITION

Definition — Varint. A variable-length encoding of an unsigned integer in base 128: each byte carries 7 bits of the number, least-significant group first, and the most significant bit (MSB) of each byte is a continuation flag — 1 means "another byte follows" [Protocol Buffers Docs: Encoding].

Small numbers, which dominate real data (counts, enum values, field tags), take one byte instead of four or eight. Negative numbers in an `int32` / `int64` field always take 10 bytes, so signed fields that are often negative use `sint32` / `sint64`, which first apply **ZigZag** encoding, mapping $0, -1, 1, -2, \ldots$ to $0, 1, 2, 3, \ldots$.

FORMULA

Varint length

$$\text{bytes}(n) = \max\left(1, \left\lceil \frac{\lfloor \log_2 n \rfloor + 1}{7} \right\rceil\right)$$

Symbol	Meaning	Units
n	Non-negative integer being encoded	—
$\lfloor \log_2 n \rfloor + 1$	Number of significant bits in n (for $n \geq 1$)	bits
7	Payload bits per varint byte	bits/byte

WORKED EXAMPLE

Worked example — Encoding a record by hand. The record (the example used in [Kleppmann 2017, ch. 4]) is `user_name = "Martin"` (field 1, string), `favorite_number = 1337` (field 2, int64), `interests = ["daydreaming", "hacking"]` (field 3, repeated string).

1. Field 1: tag $(1 \ll 3) \mid 2 = 10 =$ `0a`; length $6 =$ `06`; then the 6 UTF-8 bytes `4d 61 72 74 69 6e`. Subtotal 8 bytes.
2. Field 2: tag $(2 \ll 3) \mid 0 = 16 =$ `10`. Value 1337 = binary `101 0011 1001` (11 bits → 2 bytes). Low 7 bits `0111001` = 0x39, set the continuation bit → `b9`; remaining bits `1010` = `0a`. Subtotal 3 bytes: `10 b9 0a`.
3. Field 3, twice: tag $(3 \ll 3) \mid 2 = 26 =$ `1a`, length `0b`, 11 bytes of “daydreaming” (13 bytes); then `1a 07` + 7 bytes of “hacking” (9 bytes).
4. Total: $8 + 3 + 13 + 9 = 33$ bytes. The same record as compact JSON, `{"userName":"Martin","favoriteNumber":1337,"interests":["daydreaming","hacking"]}`, is 81 bytes.

Interpretation. Protobuf is 2.5× smaller here because field names are replaced by one-byte tags and the number by two bytes instead of four ASCII digits.
Engineering implication: field numbers 1–15 fit in a one-byte tag; reserve them for the most frequent fields.

```
        def encode_varint(n: int) -> bytes:
            """Protocol Buffers base-128 varint for a non-negative integer."""
            out = bytearray()
            while True:
                byte = n & 0x7F               # take the low 7 bits
                n >>= 7
                if n:
                    out.append(byte | 0x80)   # more bytes follow: set the MSB
                else:
                    out.append(byte)          # last byte: MSB clear
                    return bytes(out)


        def encode_tag(field_number: int, wire_type: int) -> bytes:
            return encode_varint((field_number << 3) | wire_type)


        def encode_string(field_number: int, s: str) -> bytes:
            data = s.encode("utf-8")
            return encode_tag(field_number, 2) + encode_varint(len(data)) + data


        msg = (encode_string(1, "Martin")
               + encode_tag(2, 0) + encode_varint(1337)
               + encode_string(3, "daydreaming")
               + encode_string(3, "hacking"))
        print(msg.hex(" "), len(msg), "bytes")
        # 0a 06 4d 61 72 74 69 6e 10 b9 0a 1a 0b 64 61 79 64 72 65 61 6d 69 6e 67 1a 07
68 61 63 6b 69 6e 67 33 bytes
```

What to notice: this output is byte-for-byte identical to what the official Protobuf library produces for the equivalent `.proto` message; and because every field carries its own tag and wire type, a reader that does not know field 3 can still skip exactly the right number of bytes.

Apache Avro takes the opposite approach: the binary data contains **no field tags or names at all**, just values concatenated in schema order (integers as ZigZag varints, strings as length + bytes). The same record encodes to 32 bytes [Kleppmann 2017, ch. 4]. The reader therefore needs the exact **writer's schema** to parse the bytes; it then **resolves** that schema against its own **reader's schema** by field *name*, filling defaults for fields the writer lacked and ignoring fields the reader does not know [Apache Avro Spec]. The writer's schema travels in the file header (Avro object container files), or a short schema ID is prefixed to each message and looked up in a **schema registry**, the usual arrangement with Kafka (Chapter 31). Because Avro has no hand-assigned tags, schemas can be generated from database tables, which suits data pipelines (Chapter 61).

Apache Thrift, created at Facebook [Slee et al. 2007], is an IDL plus RPC framework like gRPC. Fields carry numeric tags as in Protobuf; it offers several encodings, including

BinaryProtocol (fixed-width numbers; 59 bytes for the example record) and CompactProtocol (varints and packed field headers; 34 bytes) [Kleppmann 2017, ch. 4].

Table 8.12 — Serialization formats compared (sizes for the 3-field example record).

Format	Schema	Field identity on wire	Example size	Human-readable	Typical use
JSON	Optional (JSON Schema)	Names, every record	81 B	Yes	Public APIs, browsers, config
Thrift Binary	Required (IDL)	Numeric tags	59 B	No	Thrift RPC
Thrift Compact	Required (IDL)	Numeric tags, packed	34 B	No	Thrift RPC, storage
Protobuf	Required (`.proto`)	Numeric tags	33 B	No	gRPC, internal APIs, storage
Avro	Required, writer's schema needed to read	None (schema order)	32 B	No	Kafka with schema registry, data lakes

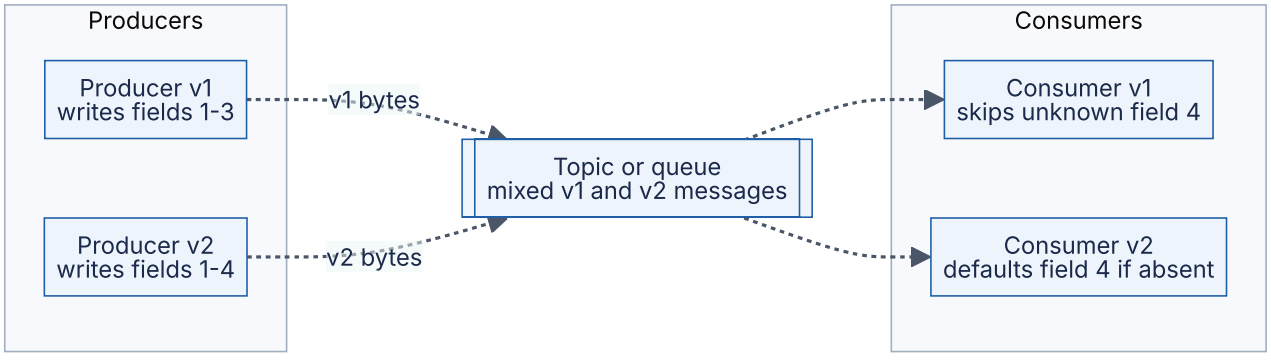
8.9.3 Schema evolution and compatibility

DEFINITION

Definition — Backward and forward compatibility. Backward compatibility: newer code can read data written by older code. **Forward compatibility**: older code can read data written by newer code [Kleppmann 2017, ch. 4].

During a rolling deployment both hold at once: new servers read messages from old servers (backward) and old servers, not yet upgraded, read messages from new ones (forward). Mobile apps and long-lived data on disk or in Kafka stretch the window from minutes to years.

Figure 8.7 — Mixed versions during a rolling deployment.



Reading the diagram. Two producer versions write to the same queue during a deployment, so the queue (the storage boundary) holds both message versions; arrows are dotted because the path is asynchronous. The old consumer must ignore field 4, which it has never heard of (forward compatibility); the new consumer must cope with messages lacking field 4 (backward compatibility). Failure point: if either assumption breaks, a deployment that looked fine in testing fails only when versions mix. Scaling: the same rules let independent teams deploy producers and consumers on separate schedules.

Table 8.13 — Schema-evolution rules by format.

Change	Protobuf (proto3)	Avro	Thrift	JSON API
Add a field	Safe: old readers skip unknown tags; new readers see the default	Safe only if the new field has a default	Safe if `optional`	Safe if clients ignore unknown fields
Remove a field	Safe; mark its number and name `reserved`	Safe only if the field had a default	Safe for `optional`; never reuse the tag	Breaking for clients that read it
Rename a field	Wire-safe (names not encoded); breaks the JSON mapping and code	Use `aliases` in the reader's schema	Wire-safe	Breaking
Change a field number / tag	Breaking	n/a	Breaking	n/a
Change type	Only between compatible types (e.g. `int32` ↔ `int64`, with truncation risk)	Only defined promotions (`int` → `long` → `float` → `double`)	Limited	Usually breaking
Add a required field	Not possible in proto3 (no `required`)	Breaking unless defaulted	Breaking	Breaking for requests

RECOMMENDATION

Practical rules. Only add optional fields with sensible defaults. Never reuse a deleted field number or name. Never change a field's meaning while keeping its name. Treat `required` as permanent. Run an automated compatibility check (schema-registry compatibility modes, `buf breaking` for Protobuf, OpenAPI diff tools) in CI, so incompatible changes fail the build instead of production.

COMMON MISTAKE

“Binary formats are always better.” For a public API consumed by browsers and third parties, JSON’s readability, tooling and zero-setup debugging usually outweigh a 2–3× size difference, especially after HTTP compression (gzip/Brotli) shrinks repeated field names. Binary formats win for high-volume internal traffic and stored data.

8.10 Cookies and sessions

8.10.1 Cookies

DEFINITION

Definition — HTTP cookie. A small name–value pair that a server asks the browser to store, via the `Set-Cookie` response header, and that the browser automatically returns in the `Cookie` request header on later requests to matching URLs [RFC 6265].

HTTP is stateless: each request arrives without memory of the previous one. A cookie is the coat-check ticket: the server hands you a numbered ticket once, you show it every time you come back, and the cloakroom (the server) knows which coat is yours. Cookies power sessions, preferences and tracking. Because browsers attach them **automatically**, including to requests triggered by other websites, their attributes are security-critical.

```
Set-Cookie: __Host-sid=8f2c1e9a7b; Path=/; Secure; HttpOnly; SameSite=Lax; Max-Age=3600
```

Table 8.14 — Cookie attributes.

Attribute	Effect	Why it matters
`Expires` / `Max-Age`	When the browser deletes the cookie; absent → deleted when the browser session ends	Bounds the lifetime of a stolen cookie
`Domain`	Send to this domain *and its subdomains*; absent → exact host only	Setting `Domain=example.com` exposes the cookie to every subdomain
`Path`	Send only for URLs under this path	Organization, not a security boundary
`Secure`	Send only over HTTPS	Prevents leakage over plaintext networks
`HttpOnly`	Not readable by JavaScript (`document.cookie`)	Limits theft via cross-site scripting (XSS)
`SameSite=Strict`	Never sent on cross-site requests	Strongest CSRF protection; breaks inbound links that need login
`SameSite=Lax`	Sent on top-level cross-site navigations using safe methods (`GET`), not on cross-site `POST`, iframes or fetches	Good default for session cookies
`SameSite=None`	Sent on all requests; must be combined with `Secure`	Needed only for genuine cross-site use (embedded widgets, some SSO)
`__Host-` name prefix	Browser enforces `Secure`, `Path=/` and no `Domain`	Stops subdomains from overwriting the cookie

Cross-site request forgery (CSRF) is the attack that `SameSite` targets: a malicious page makes your browser submit `POST https://bank.example/transfer`, and the browser helpfully attaches your bank session cookie. `SameSite=Lax` or `Strict` stops the cookie being sent; anti-CSRF tokens (a per-session random value the server embeds in forms and checks on submission) are the traditional complement. `SameSite` was specified after RFC 6265 and is defined in its revision draft (“RFC 6265bis”). As of 2026, Chromium-based browsers treat cookies without a `SameSite` attribute as `Lax`, but not every browser does, so always set it explicitly.

RFC 6265 requires browsers to support at least 4,096 bytes per cookie and 50 cookies per domain [RFC 6265 §6.1]. Every cookie is sent on every matching request, so 4 KB of cookies on a page with 80 asset requests to the same domain uploads 320 KB of headers. Serve static assets from a separate cookie-less domain or CDN.

8.10.2 Sessions

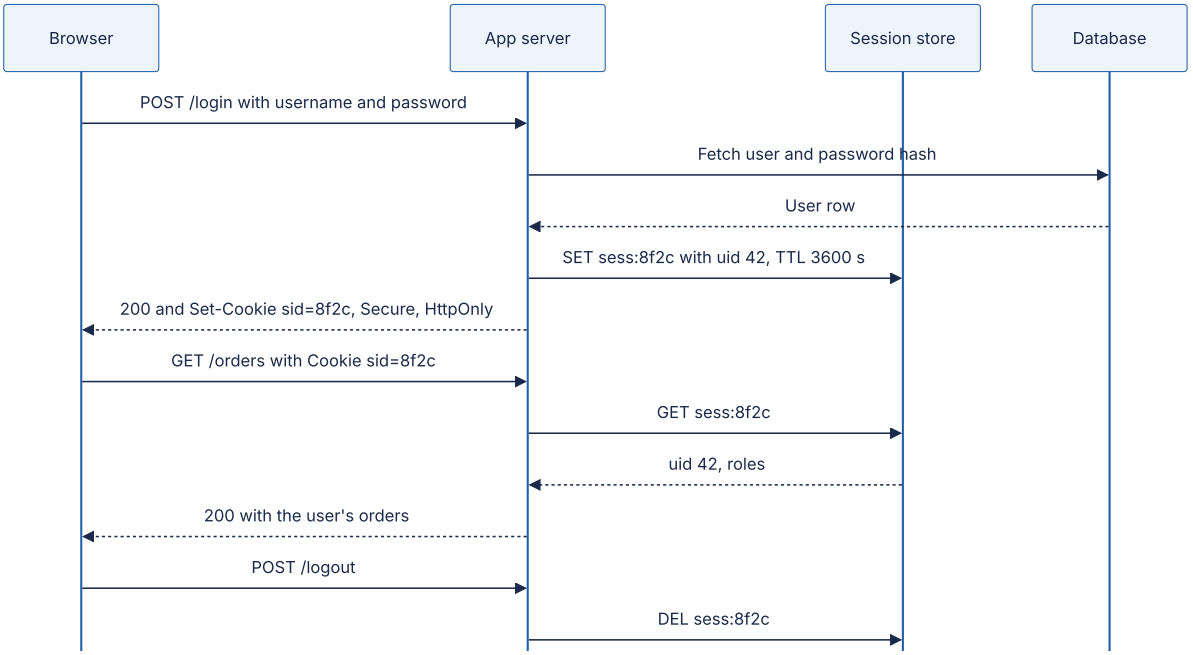
DEFINITION

Definition — Session. Server-recognized state that links a series of requests to one authenticated user (or anonymous visitor) over time, typically identified by a cookie.

There are two ways to store it.

Server-side session store. The cookie holds only a long random **session ID** (at least 128 bits of entropy). Session data — user ID, roles, CSRF token, cart — lives in a shared store such as Redis, keyed by that ID. Every request costs one lookup; logging out or revoking a session is a single `DELETE`.

Figure 8.8 — Server-side session lookup.



Reading the diagram. Login verifies the password against the database, creates a session record in the store with a TTL, and sends only the opaque ID to the browser. Every later request costs one synchronous session-store read (about 0.5 ms in the same datacenter). Logout deletes the record, so a copied cookie stops working immediately. Storage boundaries: the user database and the session store. Failure point: the session store is on the critical path of every authenticated request, so it must be replicated (Chapter 20); if it fails, everyone is logged out. Scaling: 10 million active sessions × ~500 bytes ≈ 5 GB, which fits in a single replicated Redis primary; partition by session ID beyond that.

Signed cookie (client-side session). The session data itself is in the cookie, with a **MAC** (message authentication code) computed with a server-side secret key. The server needs no store: it recomputes the MAC and rejects any cookie whose contents were al-

tered. The data is *not* encrypted — the user can read it — only protected against tampering.

```
        import base64, hashlib, hmac, json, time

        SECRET_KEY = b"load-me-from-a-secrets-manager-32+bytes"   # never hard-code in
production


        def _b64(data: bytes) -> str:
            return base64.urlsafe_b64encode(data).rstrip(b"=").decode()


        def _unb64(text: str) -> bytes:
            return base64.urlsafe_b64decode(text + "=" * (-len(text) % 4))


        def sign_session(data: dict, ttl_s: int = 3600) -> str:
            body = _b64(json.dumps({**data, "exp": int(time.time()) + ttl_s}).encode())
            mac = hmac.new(SECRET_KEY, body.encode(), hashlib.sha256).digest()
            return f"{body}.{_b64(mac)}"


        def verify_session(cookie: str) -> dict | None:
            try:
                body, mac = cookie.split(".")
            except ValueError:
                return None
            expected = hmac.new(SECRET_KEY, body.encode(), hashlib.sha256).digest()
            if not hmac.compare_digest(expected, _unb64(mac)):    # constant-time com-
pare
                return None
            data = json.loads(_unb64(body))
            return data if data["exp"] >= time.time() else None
```

What to notice: `hmac.compare_digest` compares in constant time, so an attacker cannot learn the correct MAC byte by byte from response timing; the expiry is *inside* the signed body, so it cannot be extended by the client; and there is no way to revoke one cookie before `exp` except rotating `SECRET_KEY`, which logs out everyone. Web frameworks implement exactly this; Flask's default session is a signed cookie, configured as follows:

```
        from flask import Flask
        import os

        app = Flask(__name__)
        app.config.update(
            SECRET_KEY=os.environ["SECRET_KEY"],
            SESSION_COOKIE_SECURE=True,      # only sent over HTTPS
            SESSION_COOKIE_HTTPONLY=True,    # invisible to JavaScript
            SESSION_COOKIE_SAMESITE="Lax",   # not sent on cross-site POSTs
            PERMANENT_SESSION_LIFETIME=3600, # seconds
        )
        # After login, Flask emits for example:
        # Set-Cookie: session=eyJ1aWQiOiJhZGEifQ.auQy5w.2G2C9...; Expires=...; Secure;
HttpOnly; Path=/; SameSite=Lax
```

Table 8.15 — Server-side sessions versus signed cookies.

Aspect	Server-side session store	Signed cookie
Cookie contents	Random ID only	Data + MAC (readable, tamper-evident)
Per-request cost	One store lookup (~0.5–1 ms)	One HMAC computation (microseconds)
Revocation / logout everywhere	Immediate (`DEL`)	Not possible before expiry without key rotation or a deny-list
Size limit	Unlimited server-side	~4 KB, sent on every request
Infrastructure	Replicated store on the critical path	None beyond key management
Choose when	You need instant revocation, large session state, or admin “log out this device”	Small, short-lived state; minimal infrastructure

COMMON MISTAKE

Keeping the same session ID across login. If an attacker can plant a known session ID in a victim’s browser before login and the server keeps it afterwards, the attacker shares the authenticated session (**session fixation**). Always issue a fresh session ID (or clear the signed session) at login and at privilege changes.

8.11 Authentication

DEFINITION

Definition — Authentication (AuthN). Verifying *who* a client is. **Authorization (AuthZ)** is deciding *what* that verified identity may do (§8.12). AuthN failures return `401` ; AuthZ failures return `403` .

At a hotel, the front desk checks your passport (authentication) and gives you a key card; the card then opens only your room and the gym (authorization). Keep the two separate in code too: authentication middleware turns credentials into an identity once per request; authorization checks run in every handler that touches a resource.

Authentication factors fall into three classes: something you **know** (password, PIN), something you **have** (phone, hardware security key), something you **are** (fingerprint, face). **Multi-factor authentication (MFA)** requires factors from at least two classes, so a phished or leaked password alone is not enough.

Passwords. Never store passwords, or fast hashes of them. Store the output of a deliberately slow, salted **password-hashing function** (Argon2id, scrypt, bcrypt or PBKDF2), tuned so one guess costs tens to hundreds of milliseconds; §10.6 covers parameters and the arithmetic of offline cracking. Compare in constant time, rate-limit login attempts per account and per IP, and check new passwords against lists of breached passwords, as NIST's guidance recommends [NIST SP 800-63B-4].

```
        import hashlib, hmac, os

        def hash_password(password: str) -> str:
            salt = os.urandom(16)                                    # unique per user
            dk = hashlib.scrypt(password.encode(), salt=salt, n=2**17, r=8, p=1,
                                maxmem=256 * 1024 * 1024)            # ~128 MiB of mem-
ory per guess
            return f"scrypt$131072$8$1${salt.hex()}${dk.hex()}"

        def verify_password(password: str, stored: str) -> bool:
            _, n, r, p, salt_hex, dk_hex = stored.split("$")
            dk = hashlib.scrypt(password.encode(), salt=bytes.fromhex(salt_hex),
                                n=int(n), r=int(r), p=int(p), maxmem=256 * 1024 * 1024)
            return hmac.compare_digest(dk, bytes.fromhex(dk_hex))
```

What to notice: the parameters ($N = 2^{17}$, $r = 8$, $p = 1$, the OWASP minimum for scrypt listed in Table 10.8) are stored with the hash so they can be raised later; on a typical server core this takes a few hundred milliseconds per call, which is negligible for one login and crippling for an attacker trying billions of guesses.

Second factors, from weakest to strongest:

- **SMS or email one-time codes.** Better than nothing; vulnerable to SIM-swap, interception and phishing.
- **TOTP** (time-based one-time passwords, RFC 6238): an authenticator app and the server share a secret and each compute a 6-digit code from the current 30-second time window. Not phishing-resistant: a fake site can relay the code within its window.
- **WebAuthn / passkeys** (W3C WebAuthn with FIDO2): the device holds a private key per site and signs a server challenge; the browser binds the signature to the real

site's origin, so a look-alike domain cannot obtain a usable response. This is **phishing-resistant** and can replace passwords entirely.

8.11.1 Sessions versus tokens

After login, the client needs something to present on each request. There are two families:

- **Session (reference) credentials:** an opaque random ID that means nothing by itself; the server looks it up (§8.10.2).
- **Self-contained (value) tokens:** a signed token, typically a JWT (§8.13), carrying the identity and claims; any server with the key can verify it without a lookup.

Table 8.16 — Session IDs versus self-contained tokens.

Aspect	Opaque session ID	Self-contained token (JWT)
Validation	Lookup in a shared store	Signature check, local, no network call
Revocation	Immediate	Hard: valid until `exp` (§8.13.4)
Size on each request	~32–64 bytes	Hundreds of bytes to over 1 KB
Works across services and organizations	Only those that can reach the store	Any party with the verification key
Typical transport	`Cookie` header (browsers)	`Authorization: Bearer` header (APIs, mobile, service-to-service)
Choose when	First-party web apps, especially with server rendering	Distributed APIs, third-party access (OAuth), short-lived service credentials

A common hybrid: the browser holds an opaque session cookie with the BFF or web tier; behind it, services exchange short-lived JWT access tokens.

8.12 Authorization: RBAC, ABAC and ReBAC

8.12.1 Three models

DEFINITION

Definition — Access-control model. A rule system that answers "may subject S perform action A on resource R (in context C)?" The three models in common use differ in what the rules are written in terms of: **roles** (RBAC), **attributes** (ABAC) or **relationships** (ReBAC).

Role-based access control (RBAC). Permissions attach to roles (`viewer`, `editor`, `admin`); users are assigned roles. It is what most company org charts look like: anyone with the "nurse" badge can open the medicine cabinet. RBAC is simple to audit ("who has admin?") but coarse: "editor" usually means editor of *everything*. Scoping roles per tenant or per project (`editor@project-7`) helps until the number of role assignments explodes.

Attribute-based access control (ABAC). A policy is a predicate over attributes of the subject, the resource, the action and the environment: "a user may read a document if their department equals the document's department, their clearance ≥ its classification, and it is business hours". ABAC is expressive and handles context (time, location, device posture), but policies become hard to reason about and audit, and every check needs all the attributes at hand. Policy engines such as Open Policy Agent evaluate such rules outside application code.

```
        ROLE_PERMISSIONS = {
            "viewer": {"doc:read"},
            "editor": {"doc:read", "doc:write"},
            "admin":  {"doc:read", "doc:write", "doc:delete", "user:manage"},
        }

        def rbac_allows(user_roles: set[str], permission: str) -> bool:
            return any(permission in ROLE_PERMISSIONS.get(r, set()) for r in user_roles)

        def abac_allows(sub: dict, action: str, res: dict, ctx: dict) -> bool:
            if action == "read":
                return (sub["department"] == res["department"]
                        and sub["clearance"] >= res["classification"]
                        and 8 <= ctx["hour_utc"] < 20)          # business hours only
            if action == "write":
                return sub["id"] == res["owner_id"]
            return False
```

Relationship-based access control (ReBAC). Permissions follow from a graph of relationships: Alice can view this document because she is a member of a group that is a viewer of the folder that contains it. This is how sharing works in collaboration products ("anyone in Engineering can comment"), and neither roles nor flat attributes express it naturally.

8.12.2 Google Zanzibar

INDUSTRY EXAMPLE

Zanzibar: Google's consistent, global authorization system. Google built one authorization service to serve the access-control checks of many products (the paper lists Calendar, Cloud, Drive, Maps, Photos and YouTube among its clients). It stores permissions as **relation tuples** of the form `object#relation@user`, where the user may itself be a set of users such as `group:eng#member`. A per-namespace configuration of **userset rewrite rules** derives relations from others — for example, every `editor` is also a `viewer`, and a document's viewers include the viewers of its parent folder. A **check** is a graph search over tuples and rewrites. The paper reports trillions of stored ACL tuples (more than two trillion), millions of authorization checks per second, 95th-percentile latency under 10 ms and availability above 99.999% over three years of production. Data is stored in Spanner; aggressive caching, request deduplication and a specialized index (Leopard) for deeply nested groups keep latency low. To avoid the **"new enemy" problem** — checking an old ACL against new content, for example after removing a user and then adding sensitive content — clients store an opaque consistency token, the **zookie**, with content updates, and later checks are evaluated at a snapshot at least as fresh as that zookie [Pang et al. 2019]. Open-source systems inspired by the paper, such as OpenFGA and SpiceDB, exist as of 2026. *Evidence: Primary* — [Pang et al. 2019]

A toy check function shows the core idea of tuples plus rewrite rules:

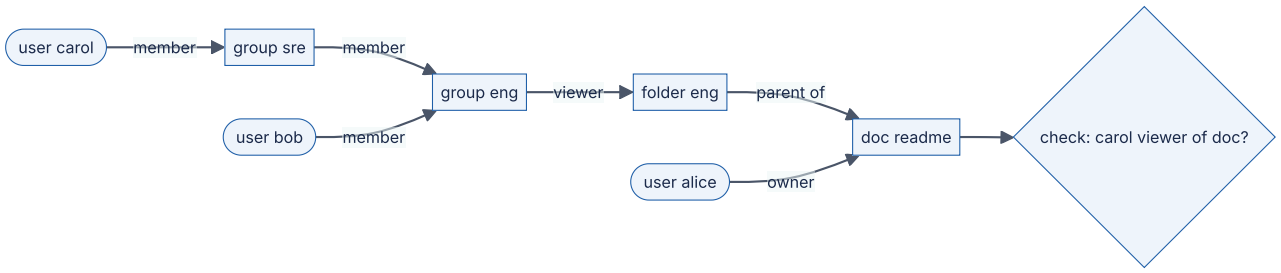
```
        # Relation tuples: (object, relation, subject). A subject is a user or a userset
"obj#rel".
        TUPLES = {
            ("doc:readme", "owner",  "user:alice"),
            ("doc:readme", "parent", "folder:eng"),          # doc lives in a folder
            ("folder:eng", "viewer", "group:eng#member"),    # eng members can view the
folder
            ("group:eng",  "member", "user:bob"),
            ("group:eng",  "member", "group:sre#member"),    # nested group
            ("group:sre",  "member", "user:carol"),
        }
        # Rewrite rules: an editor is an owner or editor; a viewer is an editor,
        # or a viewer of the parent folder.
        REWRITES = {"editor": ["owner"], "viewer": ["editor", ("parent", "viewer")]}


        def check(obj: str, relation: str, user: str, depth: int = 0) -> bool:
            if depth > 25:                                    # guard against cycles
                return False
            for (o, r, s) in TUPLES:
                if o == obj and r == relation:
                    if s == user:                             # direct tuple
                        return True
                    if "#" in s and check(*s.split("#"), user, depth + 1):   # userset
                        return True
            for rule in REWRITES.get(relation, []):
                if isinstance(rule, str):                     # same object, implied re-
lation
                    if check(obj, rule, user, depth + 1):
                        return True
                else:                                         # follow a pointer, e.g.
parent
                    via, target_rel = rule
                    if any(o == obj and r == via and check(s, target_rel, user, depth +
1)
                           for (o, r, s) in TUPLES):
                        return True
            return False


        print(check("doc:readme", "viewer", "user:alice"))   # True: owner -> editor ->
viewer
        print(check("doc:readme", "viewer", "user:carol"))   # True: sre in eng; eng
views folder
        print(check("doc:readme", "editor", "user:bob"))     # False: bob can only view
```

What to notice: the answer for Carol required walking five edges across three objects, which is why a real system needs indexing, caching and a depth limit; and sharing a folder with a group is one tuple write, not millions of per-document ACL updates.

Figure 8.9 — Relationship graph for a Zanzibar-style check.



Reading the diagram. Users (stadium shapes) are linked by `member` edges to groups, groups to a folder by a `viewer` edge, and the folder to the document by a `parent` edge. Answering "is Carol a viewer of the document?" is a search from the document back to Carol: viewer of doc ⇐ viewer of parent folder ⇐ member of group eng ⇐ member of group sre ⇐ Carol. All edges are stored tuples in one logical database (the storage boundary). Failure points: very deep or wide group nesting makes checks slow, and stale caches can grant access that was just revoked (hence zookies). Scaling: partition tuples by object, cache sub-results, and precompute group membership closure for hot nested groups.

Table 8.17 — RBAC, ABAC and ReBAC compared.

Aspect	RBAC	ABAC	ReBAC
Rules written in terms of	Roles	Attributes of subject, resource, environment	Relationships between objects and subjects
Example	"Editors can publish"	"Same department and business hours"	"Viewers of the parent folder can view"
Strength	Simple, auditable	Context-aware, fine-grained	Natural sharing and hierarchy; per-object
Weakness	Coarse; role explosion	Hard to audit; needs attributes at check time	Needs a graph store; check latency
Choose when	Internal tools, admin consoles, B2B tenants	Regulatory or contextual rules	Collaboration and sharing products

Real systems combine them: ReBAC for per-object sharing, RBAC for administrative roles, ABAC conditions (IP range, MFA freshness) on top.

COMMON MISTAKE

Checking authorization only in the frontend, or only on the list page. Hiding a button does not stop a direct API call, and checking permission when listing orders does not protect `GET /orders/43`. Every handler that touches a resource must check it; **broken object-level authorization** — returning object 43 because the caller asked for it — is one of the most common API vulnerabilities (§10.7).

8.13 JSON Web Tokens (JWT)

8.13.1 Structure

DEFINITION

Definition — JSON Web Token (JWT). A compact, URL-safe string representing a set of **claims** (statements about a subject) as a JSON object, digitally signed (JWS) or encrypted (JWE) so its integrity can be verified [RFC 7519]. This section covers the signed form, which is by far the most common.

A signed JWT is like a festival wristband: the organizer prints your ticket type and the dates on it and seals it with a tamper-evident clasp. Any steward can check it at any gate without phoning the box office. The flip side: if you are banned mid-festival, every steward will still honor the wristband until it expires.

A JWT has three base64url-encoded parts separated by dots: `header.payload.signature`.

```
{"alg": "RS256", "typ": "JWT", "kid": "2026-10-key-1"}
```

```
{
  "iss": "https://auth.example.com",
  "sub": "user-123",
  "aud": "https://api.example.com",
  "iat": 1793340000,
  "nbf": 1793340000,
  "exp": 1793340900,
  "jti": "7d1e4c1a-3b9e-4f0a-9b8e-2f6c0f1e5a77",
  "scope": "articles:read articles:write"
}
```

The registered claims are **`iss`** (issuer), **`sub`** (subject, the user or client), **`aud`** (audience: which API the token is for), **`exp`** (expiry, seconds since the Unix epoch), **`nbf`** (not before), **`iat`** (issued at) and **`jti`** (unique token ID) [RFC 7519 §4.1]. The **signature** is computed over `base64url(header) + "." + base64url(payload)`.

WARNING / TRADE-OFF

A JWT is signed, not encrypted. Anyone holding the token can base64-decode and read every claim. Never put secrets or sensitive personal data in a JWT payload.

8.13.2 Signing: HS256 versus RS256

Table 8.18 — HS256 versus RS256.

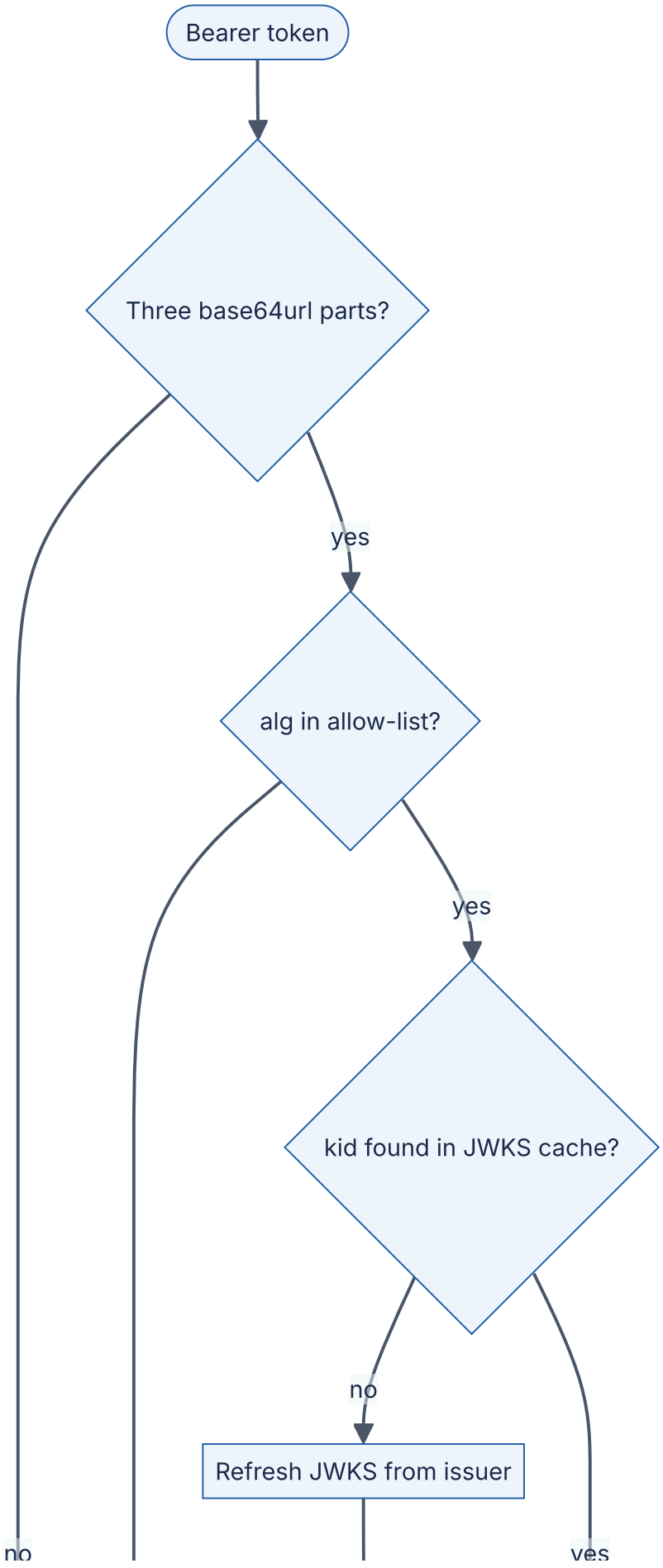
Aspect	HS256 (HMAC with SHA-256)	RS256 (RSA PKCS#1 v1.5 signature with SHA-256)
Key type	One shared secret	Private key signs; public key verifies
Who can create tokens	Anyone who can verify them	Only the private-key holder
Signature size	32 bytes (43 base64url chars)	256 bytes with a 2048-bit key (342 chars)
Speed	Very fast	Signing slower; verification fast
Key distribution	Secret must reach every verifier	Public keys published at a JWKS endpoint; rotated by `kid`
Choose when	One service issues and verifies its own tokens	An identity provider issues tokens many services verify

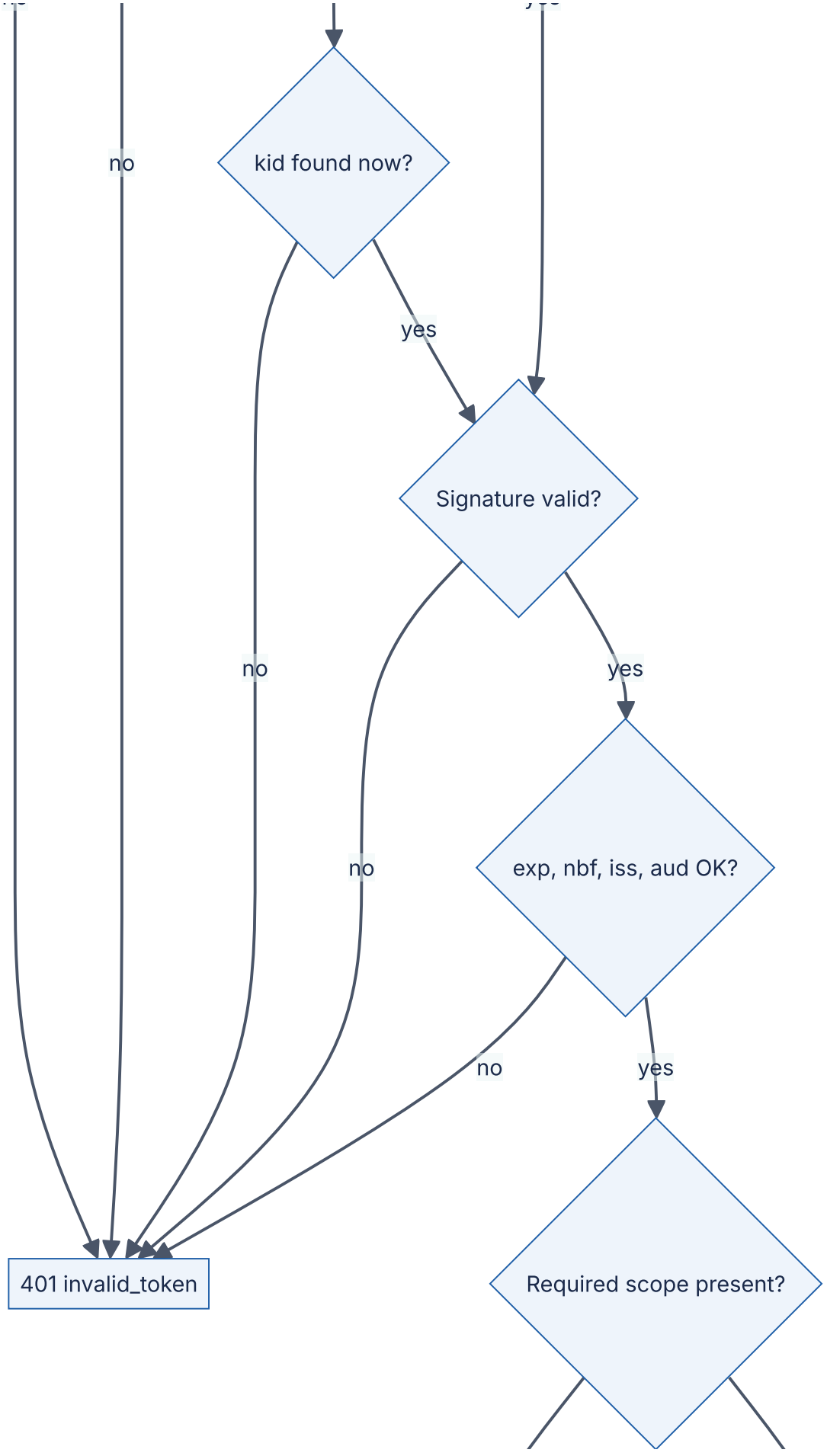
Elliptic-curve signatures (ES256, EdDSA) offer the asymmetric property of RS256 with much smaller keys and signatures; check that your libraries and partners support them.

8.13.3 Validation

Validation must check, in order: the token is well-formed; the `alg` is one you **expect** (taken from configuration, never from the token's header); the signature verifies with the right key (selected by `kid` from a trusted JWKS); `exp` is in the future and `nbf` is in the past, with a small clock-skew leeway; `iss` is your trusted issuer; `aud` contains *this* API; and only then do you read scopes and claims.

Figure 8.10 — JWT validation pipeline.


Bearer token
Three base64url parts?
yes
alg in allow-list?
yes
kid found in JWKS cache?
no
Refresh JWKS from issuer
no
yes


no
kid found now?
yes
Signature valid?
no
yes
no
exp, nbf, iss, aud OK?
no
yes
401 invalid_token
Required scope present?



Reading the diagram. A bearer token enters at the top and passes a series of local checks; only the JWKS refresh involves a network call (to the issuer's key endpoint), and it happens only when an unknown `kid` appears — typically just after the issuer rotates keys. Failures before the scope check produce `401` with `WWW-Authenticate: Bearer error="invalid_token"`; a valid token lacking the scope produces `403 insufficient_scope` [RFC 6750 §3.1]. Failure points: an attacker sending random `kid` values can force repeated JWKS fetches, so rate-limit refreshes. Scaling: validation is CPU-only and stateless, which is the whole appeal.

```
        import time, uuid
        import jwt                                      # pip install "pyjwt[crypto]"
        from cryptography.hazmat.primitives import serialization
        from cryptography.hazmat.primitives.asymmetric import rsa

        ISSUER, AUDIENCE = "https://auth.example.com", "https://api.example.com"

        # Authorization server: holds the private key.
        private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
        public_pem = private_key.public_key().public_bytes(
            serialization.Encoding.PEM, serialization.PublicFormat.SubjectPublicKeyInfo)

        def issue_access_token(user_id: str, scopes: list[str], ttl_s: int = 900) ->
str:
            now = int(time.time())
            claims = {"iss": ISSUER, "aud": AUDIENCE, "sub": user_id, "iat": now, "nbf":
now,
                      "exp": now + ttl_s, "jti": str(uuid.uuid4()), "scope": "
".join(scopes)}
            return jwt.encode(claims, private_key, algorithm="RS256",
                              headers={"kid": "2026-10-key-1"})

        # Resource server: holds only the public key.
        def validate_access_token(token: str) -> dict:
            return jwt.decode(
                token, public_pem,
                algorithms=["RS256"],               # pin the algorithm; never trust the
header
                audience=AUDIENCE, issuer=ISSUER,
                leeway=30,                          # tolerate 30 s of clock skew
                options={"require": ["exp", "iat", "sub", "aud", "iss"]},
            )

        token = issue_access_token("user-123", ["articles:read", "articles:write"])
        print(len(token), validate_access_token(token)["scope"])   # ~700 articles:read
articles:write
```

What to notice:

- `algorithms=["RS256"]` defends against two classic attacks: tokens with `"alg": "none"` (no signature at all), and **algorithm confusion**, where an attacker signs an HS256 token using the server's *public* RSA key as the HMAC secret and a careless library verifies it [RFC 8725].
- The token is about 700 characters; at 10,000 requests per second that is about 7 MB/s of header bytes, which HTTP/2 header compression reduces on reused connections.
- In a FastAPI service, the same validation sits in a dependency that returns `401` or `403` with RFC 6750 `WWW-Authenticate` errors:

```
        from fastapi import Depends, FastAPI, HTTPException
        from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer

        app = FastAPI()
        bearer = HTTPBearer(auto_error=False)

        def current_claims(creds: HTTPAuthorizationCredentials | None = Depends(bearer))
-> dict:
            if creds is None:
                raise HTTPException(401, "Missing bearer token",
                                    headers={"WWW-Authenticate": 'Bearer realm="api"'})
            try:
                return validate_access_token(creds.credentials)
            except jwt.InvalidTokenError:
                raise HTTPException(401, "Invalid or expired token",
                                    headers={"WWW-Authenticate": 'Bearer
error="invalid_token"'})

        def require_scope(scope: str):
            def checker(claims: dict = Depends(current_claims)) -> dict:
                if scope not in claims.get("scope", "").split():
                    raise HTTPException(403, f"Requires scope {scope}", headers={
                        "WWW-Authenticate": f'Bearer error="insufficient_scope", scope="
{scope}"'})
                return claims
            return checker

        @app.delete("/v1/articles/{article_id}", status_code=204)
        def delete_article(article_id: int, claims: dict =
Depends(require_scope("articles:delete"))):
            ...   # delete, recording claims["sub"] in the audit log
```

8.13.4 The revocation problem

Because verification is local, a server cannot know that a token was revoked — after logout, a password change, or a stolen laptop — until it expires. Mitigations, usually combined:

1. **Short access-token lifetimes** (5–15 minutes) plus refresh tokens (§8.14.4), which *are* checked against a server-side store on each refresh.
2. **Deny-list by `jti`** (or by `sub` + "tokens issued before time T") in a fast replicated store, checked on each request. This reintroduces a lookup, but the list holds only revoked, not-yet-expired tokens.
3. **Token introspection** (RFC 7662): the resource server asks the authorization server whether a token is active, which trades the local-verification advantage for immediacy; often used only for high-risk operations.

WORKED EXAMPLE

Worked example — Revocation window and deny-list size. Access tokens live $T = 15$ min. Users revoke sessions at $r = 100$ per minute.

1. Worst-case exposure after revocation without a deny-list: up to 15 min; if revocation times are uniformly distributed relative to issuance, on average 7.5 min.
2. Deny-list size: entries need be kept only until the token would have expired anyway, so $N = r \times T = 100 \times 15 = 1,500$ entries — about 1,500 × ~50 bytes ≈ 75 KB.

Interpretation. A deny-list is tiny and can be replicated into every service's memory and refreshed every few seconds. **Engineering implication:** "JWTs cannot be revoked" is false; the cost is a small, bounded amount of shared state.

COMMON MISTAKE

Storing JWTs in `localStorage` because "cookies are old-fashioned". Any XSS bug can read `localStorage` and exfiltrate the token. For browser apps, an `HttpOnly`, `Secure`, `SameSite` session cookie to your own backend (which holds the tokens server-side, the BFF pattern) is usually safer.

8.14 OAuth 2.0 and OpenID Connect

8.14.1 The delegation problem

DEFINITION

Definition — OAuth 2.0. An authorization framework that lets a third-party application obtain *limited* access to an HTTP service on behalf of a resource owner, by obtaining an **access token** from an authorization server instead of using the owner's credentials [RFC 6749].

Many cars come with a **valet key**: it starts the engine but cannot open the boot or the glovebox. You hand the valet a limited key instead of your full key ring. Before OAuth, a photo-printing site that wanted your photos from another service asked for your password — your full key ring — and could then do anything, forever, until you changed it. OAuth gives the printing site a valet key: a token scoped to “read photos”, time-limited, revocable, and issued without the site ever seeing your password.

8.14.2 Roles and grants

OAuth defines four roles [RFC 6749 §1.1]:

- **Resource owner** — the user who owns the data.
- **Client** — the application requesting access (the photo printer). A **confidential client** (a server-side app) can keep a client secret; a **public client** (SPA, mobile app) cannot, because its code is in users’ hands.
- **Authorization server (AS)** — authenticates the owner, asks for consent and issues tokens.
- **Resource server (RS)** — the API that accepts access tokens.

Tokens are presented as **bearer tokens** — “whoever bears this token may use it” — in the `Authorization: Bearer <token>` header, so they must only travel over TLS [RFC 6750].

Table 8.19 — OAuth 2.0 grant types and their status.

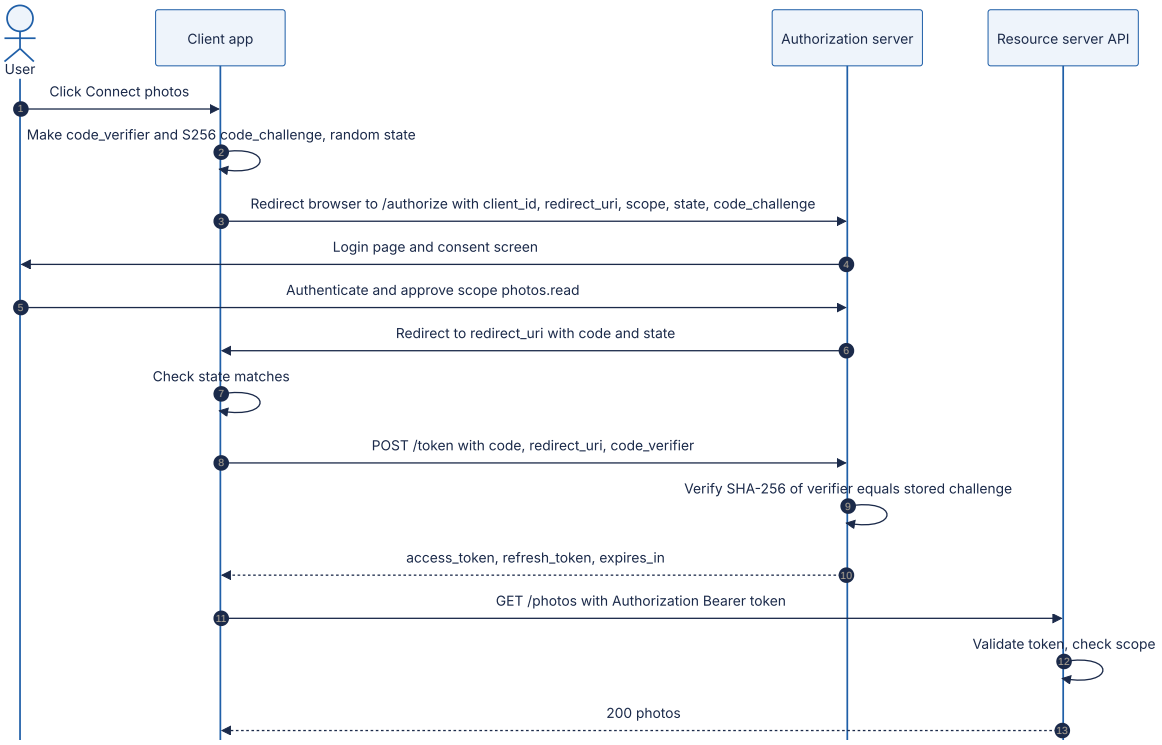
Grant	Who uses it	Status (as of 2026)
Authorization code + PKCE	Web apps, SPAs, mobile and desktop apps acting for a user	Recommended for all user-facing clients
Client credentials	A service acting for itself (machine-to-machine)	Recommended
Refresh token	Any client renewing an access token	Recommended, with rotation for public clients
Device authorization (RFC 8628)	TVs and CLIs without a convenient browser	Recommended for those devices
Implicit	Old SPAs (token returned in the URL fragment)	Deprecated; omitted from OAuth 2.1
Resource owner password credentials	Client collects the user’s password	Deprecated; omitted from OAuth 2.1

8.14.3 Authorization code flow with PKCE

DEFINITION

Definition — PKCE (Proof Key for Code Exchange, “pixy”). An extension in which the client creates a random secret, the **code verifier**, sends only its SHA-256 hash, the **code challenge**, with the authorization request, and must present the verifier when redeeming the authorization code. An attacker who intercepts the code cannot redeem it without the verifier [RFC 7636].

Figure 8.11 — OAuth 2.0 authorization code flow with PKCE.



Reading the diagram. Steps 3–6 happen through the user’s browser via redirects (the **front channel**), where URLs can leak through history, logs and malicious apps registered for the same redirect scheme on mobile devices. The authorization server only ever sees the *hash* of the verifier there. Step 8 is a direct HTTPS call from client to authorization server (the **back channel**), where the client proves possession of the verifier; an attacker who stole the code in step 6 lacks it. The `state` value checked in step 7 stops an attacker from injecting their own authorization response into the user’s session (CSRF against the redirect endpoint). Steps 11–13 are ordinary API calls with a bearer token, validated as in §8.13.3 or by introspection. All calls are synchronous. Storage boundaries: the authorization server stores codes (single use, short-lived, typically under a minute), challenges, grants and refresh tokens. Failure points: the authorization server is on the login path for every user and must be highly available; resource servers that validate JWTs locally keep working during a brief AS outage until tokens expire. Scaling: token validation scales

with the number of resource servers; the token endpoint scales like any stateless service backed by a replicated store.

```
        import base64, hashlib, secrets

        def make_pkce_pair() -> tuple[str, str]:
            # 32 random bytes -> 43-character base64url verifier (RFC 7636 allows 43-128
chars)
            verifier =
base64.urlsafe_b64encode(secrets.token_bytes(32)).rstrip(b"=").decode()
            return verifier, challenge_for(verifier)

        def challenge_for(verifier: str) -> str:
            digest = hashlib.sha256(verifier.encode("ascii")).digest()
            return base64.urlsafe_b64encode(digest).rstrip(b"=").decode()

        # Test vector from RFC 7636 Appendix B:
        assert challenge_for("dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk") == \
            "E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM"
```

The token exchange (step 8) on the wire:

```
        curl -s https://auth.example.com/oauth/token \
          -d grant_type=authorization_code \
          -d code=SplxlOBeZQQYbYS6WxSbIA \
          -d redirect_uri=https://app.example.com/callback \
          -d client_id=photo-printer \
          -d code_verifier=dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
        # {"access_token":"eyJhbGciOi...","token_type":"Bearer","expires_in":900,
        #  "refresh_token":"8xLOxBtZp8...","scope":"photos.read"}
```

What to notice: PKCE was designed for public clients, which have no secret; current guidance recommends it for confidential clients too [RFC 9700]. Redirect URIs must be registered in advance and matched exactly, since a loose match lets an attacker redirect codes to their own site.

8.14.4 Client credentials and refresh tokens

Client credentials grant. No user is involved: a backend service authenticates as itself (client ID plus secret, a signed JWT assertion, or mutual TLS) and receives a token scoped to the APIs it may call. This is the standard pattern for machine-to-machine calls between organizations or between internal services when not using mTLS identities (§10.10).

```
        curl -s -u billing-service:$CLIENT_SECRET https://auth.example.com/oauth/token \
          -d grant_type=client_credentials -d scope=invoices.write
```

Refresh tokens let a client obtain new short-lived access tokens without user interaction. Because the refresh token is long-lived and is checked by the authorization server on

each use, it is where revocation actually happens: revoke the refresh token and the client loses access within one access-token lifetime. For public clients, **refresh-token rotation** issues a new refresh token on every use and invalidates the old one; if an old one is ever presented again, the server assumes theft and revokes the whole token family [RFC 9700].

8.14.5 OAuth 2.1

OAuth 2.1 is an IETF Internet-Draft that consolidates RFC 6749, RFC 6750, PKCE and the security best current practice into one document: PKCE required for the authorization code grant, implicit and password grants removed, exact redirect-URI matching, and no bearer tokens in query strings. As of October 2026 it is still a draft (revision -16, September 2026), not an RFC [OAuth 2.1 draft]; in practice, following RFC 9700 gets you the same behavior today.

8.14.6 OpenID Connect

OAuth answers “may this app access these photos?” It deliberately says nothing about *who the user is*; an access token is meant for the resource server and may be opaque to the client. Applications nonetheless abused OAuth for “Sign in with …”, which caused vulnerabilities. **OpenID Connect (OIDC)** is a thin identity layer on top of OAuth 2.0 that standardizes login [OpenID Connect Core 1.0]:

- The client requests `scope=openid` (plus `profile`, `email` as needed).
- The token response additionally contains an **ID token**: a JWT *for the client* (its `aud` is the client ID) asserting who logged in — `iss`, `sub` (stable user identifier at that issuer), `aud`, `exp`, `iat`, `auth_time`, and the `nonce` the client sent, which prevents replay of an ID token into a different login.
- A **UserInfo endpoint** returns profile claims for the access token.
- **Discovery** (`/.well-known/openid-configuration`) publishes the issuer’s endpoints and its JWKS URL, so clients configure themselves from one URL.

> COMMON MISTAKE
>
> **Sending the ID token to your API as an access token, or accepting any provider’s access token as proof of login.** The ID token’s audience is the *client*; the access token’s audience is the *API*. Validate each only for its intended audience. Identify users by the pair (`iss`, `sub`), never by email address alone, which can change or be reassigned.

8.15 API versioning and compatibility

8.15.1 What breaks clients

An API is a promise to every client that ever integrated with it. You cannot redeploy those clients; you can only stop breaking them. Applying the rules of §8.9.3 to HTTP APIs:

Table 8.20 — Breaking and non-breaking API changes.

Non-breaking (additive)	Breaking
Add a new endpoint	Remove or rename an endpoint or field
Add an optional request parameter	Add a required request parameter
Add a field to a response	Change a field's type, format or meaning
Add a new enum value *if* clients were told to tolerate unknown values	Add an enum value clients switch on exhaustively
Relax a validation rule	Tighten validation; change defaults; change error codes
Add a new error `type` within an existing status	Change pagination or authentication scheme

Clients share responsibility: a **tolerant reader** ignores unknown fields and unknown enum values instead of failing. Document that obligation explicitly.

8.15.2 Versioning strategies

Table 8.21 — API versioning strategies.

Strategy	Example	Advantages	Disadvantages
URI path	`GET /v2/orders/42`	Obvious; easy to route, cache and test in a browser	A version is a whole parallel API; encourages big-bang versions
Custom header	`Api-Version: 2`	Clean URIs	Invisible in logs and links; caches must `Vary` on the header
Media type	`Accept: application/vnd.example.v2+json`	Most aligned with HTTP content negotiation	Awkward tooling; rarely understood by clients
Date-based, pinned per client	`Stripe-Version: 2024-06-20`	Small, frequent changes; each client upgrades on its own schedule	Server must maintain transformations for every live version

Two practices that work regardless of the mechanism: use **major versions only** for breaking changes (additive changes need no new version), and keep the number of live major versions small (one or two).

INDUSTRY EXAMPLE

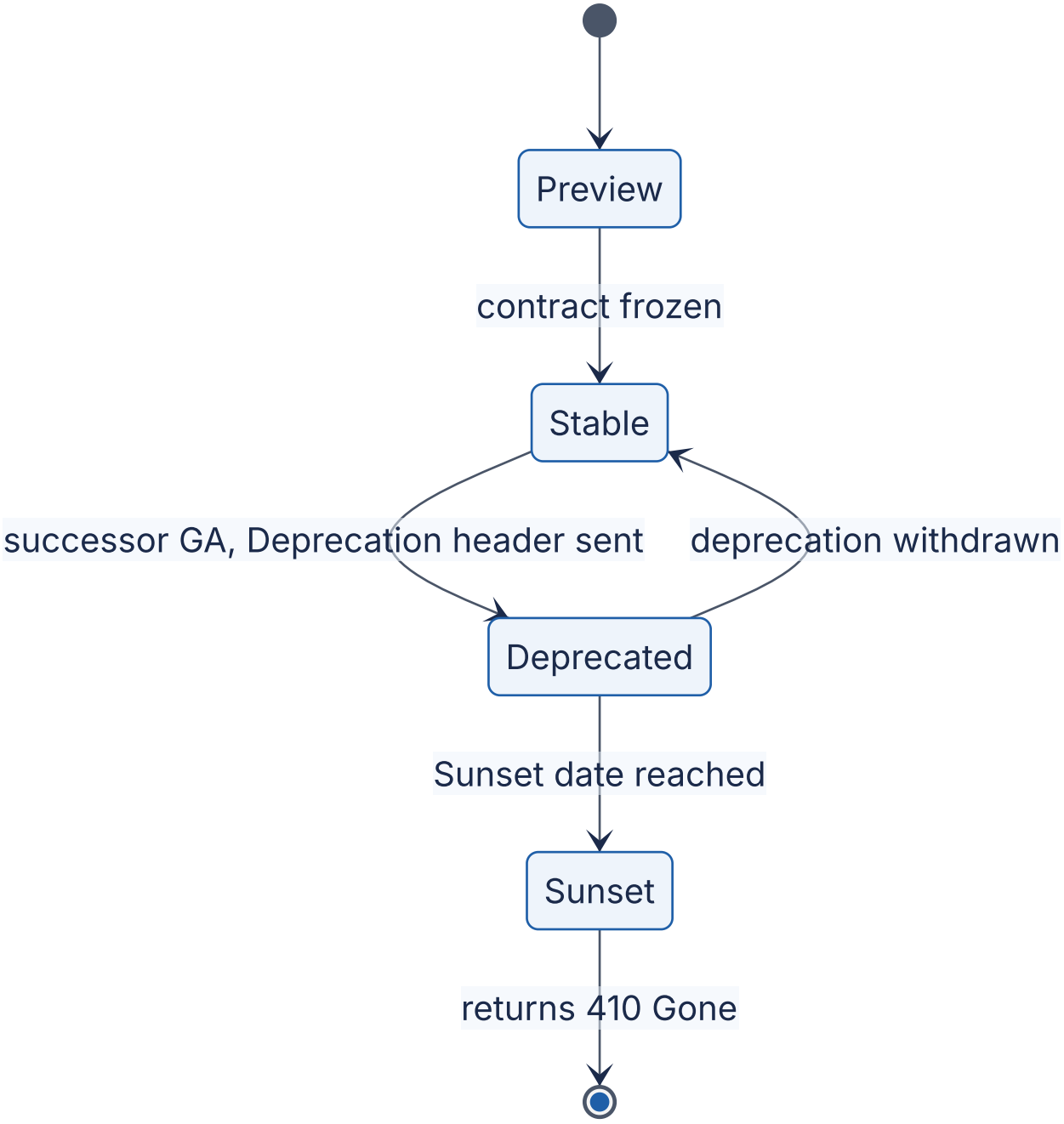
Stripe's date-based versions. Stripe pins each account to the API version current when it first made a request, and lets requests override it with a `Stripe-Version` header. Internally the latest code produces the latest response shape, and a chain of small "version change" modules transforms responses backwards, one step per version, to whatever version the client is pinned to. This lets Stripe ship breaking improvements frequently without forcing integrations to change [Leach 2017]. GitHub's REST API similarly uses date-named versions selected by the `X-GitHub-Api-Version` header [GitHub Docs: API versions]. *Evidence: Primary* — [Leach 2017]; [GitHub Docs: API versions]

8.15.3 Deprecation

Deprecation is a process, not an event:

1. **Announce** in the changelog and documentation, with a date.
2. **Signal in-band.** Send the `Deprecation` response header [RFC 9745] on deprecated endpoints, a `Link` with `rel="deprecation"` pointing to migration docs, and the `Sunset` header [RFC 8594] with the date the endpoint will stop working.
3. **Measure.** Log usage of deprecated endpoints per client ID; contact the remaining heavy users directly.
4. **Brown-outs.** Before the sunset date, fail a small, increasing fraction of calls for short scheduled windows, so forgotten integrations surface while there is time to fix them.
5. **Remove**, returning `410 Gone` with a problem-details body that links to the migration guide.

Figure 8.12 — Lifecycle of an API version.



Reading the diagram. A version starts as a preview (no compatibility promise), becomes stable when its contract is frozen, and is deprecated once a successor is generally available; from then on responses carry `Deprecation` and `Sunset` headers. At the sunset date it is removed and answers `410 Gone`. The transitions are driven by policy and calendar, not by traffic. Failure point: removing a version that still carries significant traffic; usage metrics per client must gate the final transition.

8.16 API documentation: OpenAPI

DEFINITION

Definition — OpenAPI Specification (OAS). A vendor-neutral, machine-readable format (YAML or JSON) for describing HTTP APIs: paths, operations, parameters, request and response bodies (as JSON Schema), status codes and security schemes [OpenAPI Specification]. It grew out of Swagger and is maintained by the OpenAPI Initiative; version 3.1 aligned its schemas with JSON Schema, and version 3.2.0 was released in September 2025.

An OpenAPI document is the API's contract in a form that tools can use: generating interactive documentation, client SDKs and server stubs, mock servers, request validation at gateways, and **contract tests** and **breaking-change detection** in CI (diffing the spec against the previous release flags the changes in Table 8.20 automatically).

```yaml
openapi: 3.1.0
info:
  title: Articles API
  version: 1.0.0
paths:
  /v1/articles:
    get:
      summary: List articles, newest first
      parameters:
        - name: limit
          in: query
          schema: { type: integer, minimum: 1, maximum: 100, default: 20 }
        - name: cursor
          in: query
          description: Opaque cursor from a previous response's next_cursor.
          schema: { type: string }
      security:
        - bearerAuth: []
      responses:
        "200":
          description: A page of articles.
          content:
            application/json:
              schema:
                type: object
                required: [data, next_cursor]
                properties:
                  data:
                    type: array
                    items: { $ref: "#/components/schemas/Article" }
                  next_cursor: { type: [string, "null"] }
        "400":
          description: Invalid cursor.
          content:
            application/problem+json:
              schema: { $ref: "#/components/schemas/Problem" }
components:
  securitySchemes:
    bearerAuth: { type: http, scheme: bearer, bearerFormat: JWT }
  schemas:
    Article:
      type: object
      required: [id, title, status, created_at]
      properties:
        id: { type: integer }
        title: { type: string }
        status: { type: string, enum: [draft, published] }
        created_at: { type: string, format: date-time }
    Problem:
      type: object
      properties:
        type: { type: string, format: uri }
        title: { type: string }
        status: { type: integer }
```

```
                detail: { type: string }
                instance: { type: string }
```

What to notice: `next_cursor` is typed `[string, "null"]`, OpenAPI 3.1's JSON Schema way of saying "nullable"; errors are documented with the `application/problem+json` media type of §8.6.7; and the bearer scheme declares that operations need a JWT.

There are two workflows. In **design-first**, teams write and review the OpenAPI document before code, then generate stubs; this suits public APIs where the contract matters most. In **code-first**, the framework derives the document from code: FastAPI, for example, serves the document generated from the `articles_api.py` example of §8.6.5 at `/openapi.json` and interactive documentation at `/docs`, with no extra work. Either way, keep the spec in version control and treat changes to it as API changes that require review. GraphQL APIs are self-describing through **introspection**, and gRPC APIs through their `.proto` files, which play the same role.

8.17 Summary and revision notes

SUMMARY

- **Client–server**: clients initiate, servers own shared state. Keep the application tier **stateless**; state lives in databases, caches, cookies or tokens.
- **Rendering**: SSR (server builds HTML), SPA (browser builds UI from JSON APIs), SSG (built once, served from a CDN), native mobile (old versions live for years). SPAs and mobile apps need a well-designed API; BFFs tailor it per client.
- **Lifecycle**: DNS → TCP → TLS → load balancer → web server → app middleware → handler → cache/DB → response. On a cold connection, setup round trips often cost more than server work; reuse connections.
- **REST**: resources named by URIs, standard methods, status codes. Know which methods are **safe** (GET, HEAD, OPTIONS) and **idempotent** (those plus PUT, DELETE). Make POST retryable with idempotency keys.
- **Pagination**: offset costs $O(\text{offset} + \text{limit})$ and shifts under writes; **cursor/keyset** pagination with `WHERE (sort_key, id) < (…)` and a matching index is constant-cost and stable.
- **Errors**: use status codes for the class of outcome and RFC 9457 `application/problem+json` for detail.
- **GraphQL**: typed schema, client-chosen fields, one round trip. Fix **N+1** with per-request **DataLoader** batching; HTTP caching is hard (use persisted queries); enforce depth and cost limits.
- **gRPC**: Protobuf IDL, generated stubs, HTTP/2 streams; unary, server-, client- and bidirectional streaming. Always set and propagate **deadlines**; use L7 load balancing.
- **Serialization**: JSON is readable and universal; Protobuf and Thrift use numeric tags and varints; Avro has no tags and resolves writer's against reader's schema. Evolve by adding optional fields with defaults; never reuse field numbers.
- **Cookies**: `Secure`, `HttpOnly`, `SameSite=Lax` (or Strict), `__Host-` prefix. Sessions are server-side (instant revocation, one lookup) or signed cookies (no store, no early revocation).
- **AuthN vs AuthZ**: who you are (401) vs what you may do (403). Hash passwords with slow salted functions (§10.6); prefer phishing-resistant MFA (WebAuthn/passkeys).
- **Authorization models**: RBAC (roles), ABAC (attribute predicates), ReBAC (relationship graph; Zanzibar's tuples, rewrites and zookies).
- **JWT**: header.payload.signature, signed not encrypted. Pin the algorithm; validate `exp`, `iss`, `aud`. HS256 = shared secret; RS256/ES256 = private signs, public verifies. Revocation needs short TTLs, refresh tokens and/or deny-lists.

- **OAuth 2.0**: delegated, scoped access via tokens. Use authorization code + PKCE for user-facing apps, client credentials for machines, rotating refresh tokens. OAuth 2.1 is still a draft (as of 2026). **OIDC** adds the ID token and standard login.
- **Versioning**: additive changes need no new version; breaking changes do. URI, header, media-type or date-based pinning; deprecate with `Deprecation` / `Sunset` headers, usage metrics and brown-outs.
- **OpenAPI** is the machine-readable contract: docs, SDKs, validation and breaking-change detection in CI.

8.18 Knowledge checkpoint

1. Why does making the application tier stateless make horizontal scaling easy, and where does the state go?
2. Name two backend consequences of supporting a native mobile client that an SSR website does not impose.
3. A user 50 ms RTT away makes a first request needing an uncached DNS lookup (20 ms), TCP and TLS 1.3 setup, and 15 ms of server time. Estimate the latency, and the latency of the next request.
4. Which HTTP methods are safe? Which are idempotent? Why does the distinction matter to a retry library?
5. When should an API return 401, 403, 404, 409, 422 and 429?
6. Write the keyset-pagination `WHERE` clause for sorting by `created_at DESC, id DESC`, and explain why `id` is included.
7. Page size 50, page 10,001 with offset pagination: how many rows does the database walk? Roughly how many with keyset pagination?
8. What are the five standard members of an RFC 9457 problem-details object?
9. A GraphQL query fetches 20 posts and each post's author. How many database queries run with naive resolvers and with DataLoader?
10. Why must a DataLoader instance be created per request?
11. What are the four gRPC call types? Give one use for each.
12. What happens to a gRPC call with no deadline when the server hangs?
13. Encode the integer 300 as a Protobuf varint in hex.
14. In Protobuf, why must you never reuse a deleted field number? What does Avro need instead of field numbers?
15. What does each of `Secure`, `HttpOnly` and `SameSite=Lax` protect against?
16. Give one advantage and one disadvantage of a signed-cookie session versus a server-side session store.
17. In a JWT, why must the server not use the `alg` value from the token header to choose the verification algorithm?

18. Why does PKCE stop an attacker who intercepts the authorization code?
19. What does OpenID Connect add to OAuth 2.0?
20. Classify as breaking or non-breaking: adding an optional query parameter; renaming a response field; adding a required request field.

Answers

1. Any instance can serve any request, so instances can be added, removed or replaced behind a load balancer without losing anything. State moves to databases, caches, a session store, or the client (cookies, tokens).
2. Old app versions remain in use for months or years, so the API must stay backward compatible; slow cellular links make round trips expensive, favoring coarse-grained or aggregated endpoints (BFF, GraphQL) and compact payloads.
3. Cold: $20 + 50 + 50 + 50 + 15 = 185$ ms. Next request on the same connection with cached DNS: $50 + 15 = 65$ ms.
4. Safe: GET, HEAD, OPTIONS (and TRACE). Idempotent: the safe methods plus PUT and DELETE. A retry library can automatically retry idempotent requests after a timeout; retrying POST may duplicate effects unless an idempotency key is used.
5. 401: not authenticated. 403: authenticated but not permitted. 404: no such resource (or hidden). 409: conflicts with current state, such as a duplicate key. 422: well-formed but semantically invalid input. 429: rate limit exceeded, with `Retry-After`.
6. `WHERE (created_at, id) < (:last_created_at, :last_id) ORDER BY created_at DESC, id DESC LIMIT :n`. `id` makes the order total, so rows sharing a timestamp are neither skipped nor repeated across pages.
7. Offset $= 10,000 \times 50 = 500,000$, so about 500,050 rows. Keyset: an index descent (~3–4 levels) plus 50–51 rows.
8. `type`, `title`, `status`, `detail`, `instance`.
9. Naive: $1 + 20 = 21$. DataLoader: 2 (one for posts, one batched `IN` query for distinct authors).
10. Its cache would otherwise serve one user's data (possibly unauthorized) to another user and return stale data across requests.
11. Unary (fetch an order); server streaming (status updates, tailing logs); client streaming (uploading telemetry in chunks); bidirectional (chat, live transcription).
12. The client waits indefinitely, holding a thread or connection and memory; under load such calls accumulate and can exhaust the caller, cascading up the call chain.
13. 300 = `1 0010 1100`. Low 7 bits `010 1100` = 0x2C, plus continuation bit → `ac`; remaining bits `10` → `02`. Result: `ac 02`.
14. Old data or old clients would interpret the new field's bytes as the old field, causing silent corruption. Avro needs the writer's schema (from a file header or schema registry) and resolves fields by name, using defaults.
15. `Secure`: leakage over unencrypted HTTP. `HttpOnly`: theft of the cookie by JavaScript injected through XSS. `SameSite=Lax`: the cookie being sent on cross-site POSTs and subresource requests, that is, CSRF.

16. Advantage: no store lookup and no shared infrastructure. Disadvantage: cannot be revoked individually before expiry; limited to ~4 KB sent on every request; contents readable by the user.
17. An attacker controls the header. They could set `alg: none`, or `HS256` and sign with the server's public RSA key as the HMAC secret (algorithm confusion). The server must use its configured algorithm allow-list.
18. The code is useless without the code verifier, which never travels through the browser; only its SHA-256 hash does, and the hash cannot be inverted.
19. A standard identity layer: the `openid` scope, the ID token (a JWT about the authenticated user with audience = client), the UserInfo endpoint, standard claims, nonce-based replay protection and discovery metadata.
20. Non-breaking; breaking; breaking.

8.19 Interview questions

Basic

Q1. Walk me through what happens when a user types a URL and presses Enter. Answer. The browser parses the URL and checks its HTTP cache. It resolves the host name through DNS (OS cache, then recursive resolver, then authoritative servers), opens a TCP connection (one RTT) and performs a TLS 1.3 handshake (one RTT), or a combined QUIC handshake on HTTP/3. It sends the HTTP request with headers and cookies. A load balancer terminates TLS, routes on host and path, and forwards to a healthy backend over a pooled connection. The web server passes it to the application, where middleware authenticates, rate-limits and routes to a handler; the handler authorizes, reads cache and database, serializes a response and returns it back through the chain. The browser parses and renders, fetching subresources, often over the same connection. Strong answers attach rough numbers to each hop and note which steps connection reuse and CDNs remove.

Q2. What is the difference between PUT, PATCH and POST? Answer. POST asks the target collection or resource to process the body, commonly creating a resource whose URI the server chooses; it is not idempotent. PUT creates or completely replaces the resource at the URI the client names; it is idempotent. PATCH applies a partial modification; it is not guaranteed idempotent (an "increment" patch is not). Retries are therefore safe for PUT but need idempotency keys for POST.

Q3. Cookies or tokens for authentication? Answer. For a first-party browser application, an opaque session ID in a `Secure; HttpOnly; SameSite` cookie is simple and secure and allows instant logout. For APIs consumed by mobile apps, other services or third parties, short-lived bearer tokens (often JWTs) obtained through OAuth fit better, because any service can verify them without a shared store. Many systems combine them: the

browser holds a session cookie with a BFF, which holds tokens and calls services with them.

Intermediate

Q4. Design pagination for an activity feed API with 500 million rows and constant inserts. Answer. Use cursor (keyset) pagination ordered by `(created_at DESC, id DESC)`, backed by a composite index (per user, if the feed is per user: `(user_id, created_at DESC, id DESC)`). The response returns `data` and an opaque `next_cursor` encoding the last row's sort key, base64url-encoded and HMAC-signed, optionally with an expiry and a hash of the filter parameters. Query with a row-value comparison and `LIMIT n+1` to detect more pages. Cap `limit` at, say, 100. Omit total counts or serve approximate ones. Explain why offset fails: cost grows linearly with depth (deep pages walk millions of rows) and inserts cause duplicates. Mention that cursors survive deletes of the boundary row because the comparison is on values, not on row existence.

Q5. Your GraphQL API's database load tripled after a new mobile screen launched. Diagnose and fix. Answer. Likely N+1 resolution on nested lists, possibly compounded by a query requesting large page sizes or deep nesting. Confirm with per-request query counts and resolver tracing. Fix with per-request DataLoaders for every relation field (batch by parent IDs, dedupe), enforce maximum `first` values, add query-cost analysis with per-client budgets, and use persisted queries so the expensive query can be reviewed and cached. Consider caching hot entities in the batch functions.

Q6. How do you revoke access for a user whose laptop was stolen, if your APIs use JWT access tokens? Answer. Revoke their refresh tokens at the authorization server (so no new access tokens are minted) and end their server-side sessions. For outstanding access tokens, either accept exposure until expiry (keep TTLs at 5–15 minutes) or push a deny-list entry — by `jti`, or "all tokens for `sub` issued before T" — to a replicated store or in-memory cache in every resource server, checked during validation. Entries can expire with the tokens, so the list stays small. High-risk operations can additionally require introspection or step-up authentication.

Advanced

Q7. Design authorization for a document-collaboration product with folders, nested groups and link sharing, serving 100,000 checks per second at p99 under 20 ms. Answer. Use ReBAC in the Zanzibar style. Store tuples `object#relation@subject` (subjects can be usersets like `group:eng#member`) in a partitioned, replicated store, partitioned by object ID. Define namespace rewrite rules: owner ⊂ editor ⊂ commenter ⊂ viewer, and viewer inherited from parent folder; link sharing is a tuple with a special subject such as `anyone-with-link`. A check service evaluates the rewrite graph with memoization of sub-results, request deduplication, depth limits and a precomputed membership

index for large nested groups. For freshness, return a consistency token with each ACL or content write and evaluate later checks at a snapshot at least that recent, avoiding the new-enemy problem; otherwise allow slightly stale cached results for latency. Provide list/filter APIs ("which of these 50 docs can Alice see?") as batched checks. Discuss failure modes: the authorization service on every read path must be multi-region and degrade by failing closed; audit logs of tuple changes; and the cost of deep nesting.

Q8. Your company runs 300 microservices with a mix of REST/JSON and gRPC. Propose an inter-service API strategy, including versioning and schema evolution. Answer. Default to gRPC with Protobuf for internal synchronous calls: compact, typed, streaming, deadline propagation. Keep all `.proto` files in a central schema repository with linting and breaking-change detection in CI; version by package (`orders.v1`), additive changes only within a version. Expose public and partner APIs as REST/JSON with OpenAPI, generated or transcoded from the same definitions where possible, with RFC 9457 errors and date-based or major-version URIs. Mandate deadlines on every call, propagation of deadline and trace context, and status-code-based retry policies with budgets. Use L7 or client-side load balancing for HTTP/2. For asynchronous events, use Avro or Protobuf with a schema registry enforcing backward/forward compatibility. Measure the usage of every deprecated field and method before removal.

Q9. Walk through the security of a SPA that signs users in with an external identity provider and calls your API. Answer. The SPA is a public client, so use the authorization code flow with PKCE (S256), a random `state`, and an OIDC `nonce`; register exact redirect URIs. Preferably run a BFF: the browser holds only an `HttpOnly; Secure; SameSite=Lax` (or Strict), `__Host-` session cookie with the BFF, which performs the code exchange as a confidential client and stores access and refresh tokens server-side, so XSS cannot steal tokens. Validate the ID token (signature via JWKS, `iss`, `aud` = client ID, `exp`, `nonce`) and identify users by (`iss`, `sub`). The API validates access tokens with pinned algorithms, `aud` = API, scopes, and short TTLs; refresh tokens rotate with reuse detection. Add CSRF protection for state-changing BFF endpoints, a strict Content Security Policy, and an authorization check on every object access.

8.20 Practical exercises

1. **Build a small REST API (2–4 h).** Extend `articles_api.py` (§8.6.5) into a FastAPI or Flask service backed by SQLite or PostgreSQL with `POST /v1/articles` (201 + `Location`), `GET` with cursor pagination and filtering by `status` and `author_id`, `PATCH` with `If-Match` ETags (412 on mismatch), and `DELETE` (204). Protect writes with the JWT dependency of §8.13.3 and scopes `articles:write` and `articles:delete`. Return RFC 9457 errors throughout. Write tests that page through 1,000 rows while inserting new ones, and assert that no row is returned twice.

2. **Measure offset versus keyset.** Load 5 million rows into PostgreSQL. Time `OFFSET` queries at offsets 0, 10^4, 10^5 and 10^6, and the equivalent keyset queries, and read `EXPLAIN (ANALYZE, BUFFERS)` for each. Plot latency against page depth.
3. **Encode by hand.** Write a decoder for the varint encoder of §8.9.2 that turns the 33-byte message back into fields, skipping unknown field numbers using the wire type. Add `sint64` ZigZag support and verify against the official `protobuf` library.
4. **Fix an N+1.** Implement the blog schema of §8.7.1 with Strawberry or Ariadne (Python) over SQLite, log every SQL statement, and count queries for a request of 50 posts with comments and authors. Add DataLoaders and compare.
5. **OAuth end to end.** Run an open-source identity provider (for example Keycloak) locally, register a public client, and implement the authorization code + PKCE flow from a small Python web app, including `state`, `nonce` and ID-token validation. Then demonstrate that replaying a stolen code without the verifier fails.

8.21 Design challenges

1. **Public API for a payments startup.** Design the external API for creating charges, refunds and listing transactions: resource model, idempotency keys, pagination, error types, webhooks, authentication for merchants (API keys vs OAuth for platforms), versioning and deprecation policy. Justify every status code.
2. **One backend, three frontends.** A retailer has an SSR website, an iOS/Android app with five live versions, and partner integrations. Choose between REST, GraphQL, BFFs and gRPC for each layer, draw the architecture, and explain how a schema change to “product price” rolls out without breaking any client.
3. **Authorization at scale.** A B2B SaaS product needs per-tenant roles, per-project sharing and conditional rules (“contractors only from the corporate network”). Combine RBAC, ReBAC and ABAC; specify the data model, the check API, caching and consistency, and how you would audit “who can see project X?”.

8.22 References

1. Fielding, R. T. *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine, 2000. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm
2. Fielding, R., Nottingham, M., Reschke, J. (Eds.). *HTTP Semantics*. RFC 9110, IETF, 2022. https://www.rfc-editor.org/rfc/rfc9110
3. Dusseault, L., Snell, J. *PATCH Method for HTTP*. RFC 5789, IETF, 2010. https://www.rfc-editor.org/rfc/rfc5789
4. Nottingham, M., Fielding, R. *Additional HTTP Status Codes*. RFC 6585, IETF, 2012. https://www.rfc-editor.org/rfc/rfc6585

5. Nottingham, M., Wilde, E., Dalal, S. *Problem Details for HTTP APIs*. RFC 9457, IETF, 2023. https://www.rfc-editor.org/rfc/rfc9457
6. Barth, A. *HTTP State Management Mechanism*. RFC 6265, IETF, 2011. https://www.rfc-editor.org/rfc/rfc6265
7. Hardt, D. (Ed.). *The OAuth 2.0 Authorization Framework*. RFC 6749, IETF, 2012. https://www.rfc-editor.org/rfc/rfc6749
8. Jones, M., Hardt, D. *The OAuth 2.0 Authorization Framework: Bearer Token Usage*. RFC 6750, IETF, 2012. https://www.rfc-editor.org/rfc/rfc6750
9. Jones, M., Bradley, J., Sakimura, N. *JSON Web Token (JWT)*. RFC 7519, IETF, 2015. https://www.rfc-editor.org/rfc/rfc7519
10. Sakimura, N., Bradley, J., Agarwal, N. *Proof Key for Code Exchange by OAuth Public Clients*. RFC 7636, IETF, 2015. https://www.rfc-editor.org/rfc/rfc7636
11. Richer, J. (Ed.). *OAuth 2.0 Token Introspection*. RFC 7662, IETF, 2015. https://www.rfc-editor.org/rfc/rfc7662
12. Sheffer, Y., Hardt, D., Jones, M. *JSON Web Token Best Current Practices*. RFC 8725, IETF, 2020. https://www.rfc-editor.org/rfc/rfc8725
13. Lodderstedt, T., Bradley, J., Labunets, A., Fett, D. *Best Current Practice for OAuth 2.0 Security*. RFC 9700, IETF, 2025. https://www.rfc-editor.org/rfc/rfc9700
14. Hardt, D., Parecki, A., Lodderstedt, T. *The OAuth 2.1 Authorization Framework*. Internet-Draft draft-ietf-oauth-v2-1-16, IETF, 2026 (work in progress). https://datatracker.ietf.org/doc/draft-ietf-oauth-v2-1/
15. Denniss, W., Bradley, J., Jones, M., Tschofenig, H. *OAuth 2.0 Device Authorization Grant*. RFC 8628, IETF, 2019. https://www.rfc-editor.org/rfc/rfc8628
16. Sakimura, N., Bradley, J., Jones, M., de Medeiros, B., Mortimore, C. *OpenID Connect Core 1.0*. OpenID Foundation, 2014. https://openid.net/specs/openid-connect-core-1_0.html
17. M’Raihi, D., Machani, S., Pei, M., Rydell, J. *TOTP: Time-Based One-Time Password Algorithm*. RFC 6238, IETF, 2011. https://www.rfc-editor.org/rfc/rfc6238
18. NIST. *SP 800-63B-4: Digital Identity Guidelines — Authentication and Authenticator Management*. National Institute of Standards and Technology, 2025. https://pages.nist.gov/800-63-4/sp800-63b.html
19. Wilde, E. *The Sunset HTTP Header Field*. RFC 8594, IETF, 2019. https://www.rfc-editor.org/rfc/rfc8594
20. Dalal, S., Wilde, E. *The Deprecation HTTP Response Header Field*. RFC 9745, IETF, 2025. https://www.rfc-editor.org/rfc/rfc9745
21. GraphQL Foundation. *GraphQL Specification*, September 2025 Edition. 2025. https://spec.graphql.org/September2025/
22. Byron, L. *GraphQL: A data query language*. Facebook Engineering Blog, 2015.
23. GraphQL Foundation. *DataLoader* (source repository and documentation). 2015. https://github.com/graphql/dataloader
24. gRPC Authors. *gRPC Documentation* (Introduction, Core concepts, Deadlines). Accessed 2026. https://grpc.io/docs/

25. Google. *Protocol Buffers Documentation: Encoding*. Accessed 2026. https://protobuf.dev/programming-guides/encoding/
26. Apache Software Foundation. *Apache Avro Specification*. Accessed 2026. https://avro.apache.org/docs/
27. Slee, M., Agarwal, A., Kwiatkowski, M. *Thrift: Scalable Cross-Language Services Implementation*. Facebook white paper, 2007.
28. Kleppmann, M. *Designing Data-Intensive Applications*, Chapter 4: Encoding and Evolution. O'Reilly Media, 2017.
29. Birrell, A. D., Nelson, B. J. *Implementing Remote Procedure Calls*. ACM Transactions on Computer Systems 2(1), 1984.
30. Pang, R., Cáceres, R., Burrows, M., Chen, Z., Dave, P., Germer, N., Golynski, A., Graney, K., Kang, N., Kissner, L., Korn, J. L., Parmar, A., Richards, C. D., Wang, M. *Zanzibar: Google's Consistent, Global Authorization System*. USENIX Annual Technical Conference (ATC), 2019. https://www.usenix.org/conference/atc19/presentation/pang
31. OpenAPI Initiative. *OpenAPI Specification* (v3.1.0, 2021; v3.2.0, 2025). https://spec.openapis.org/oas/
32. Leach, B. *APIs as infrastructure: future-proofing Stripe with versioning*. Stripe Blog, 2017. https://stripe.com/blog/api-versioning
33. Stripe. *API Reference: Pagination*. Accessed 2026. https://docs.stripe.com/api/pagination
34. GitHub. *Using pagination in the REST API*. GitHub Docs, accessed 2026. https://docs.github.com/en/rest/using-the-rest-api/using-pagination-in-the-rest-api
35. GitHub. *API Versions*. GitHub Docs, accessed 2026. https://docs.github.com/en/rest/about-the-rest-api/api-versions
36. GitHub. *Rate limits and query limits for the GraphQL API*. GitHub Docs, accessed 2026. https://docs.github.com/en/graphql/overview/rate-limits-and-query-limits-for-the-graphql-api

8.23 Open verification items

- Zanzibar throughput: the chapter says "millions of authorization checks per second"; confirm the exact figure in the paper's abstract (secondary sources quote "more than 10 million client queries per second").
- `SameSite` default: confirm current (2026) behavior of Chromium, Firefox and Safari for cookies without a `SameSite` attribute, and the publication status of RFC 6265bis.
- OAuth 2.1: re-check the draft revision (or RFC number, if published) at the time of book publication; -16 was current in September 2026.
- NIST SP 800-63B-4: reference aligned with Chapter 10 (§10.6); confirm final wording at publication time.

- Exact URLs and titles of [Byron 2015], the GitHub GraphQL rate-limit page, and the Stripe pagination page.
- Table 8.3 latency ranges are illustrative estimation figures; cross-check against Appendix F when it is written.
- The “few hundred milliseconds per scrypt hash” figure was measured once (≈0.4 s at $N = 2^{17}$, $r = 8$, $p = 1$) on a single cloud vCPU in 2026; Chapter 10 (§10.6.2) owns the authoritative parameter guidance.

Chapter 9 — Real-Time Communication

> **Level:** 2 · **Track:** A (Traditional) · **Estimated reading time:** 3 h

The web was built as a conversation in which only one side may speak first. A browser asks; a server answers; the exchange ends. That model, described in §8.5 (The request–response lifecycle), works well for loading a page or submitting a form. It works badly when the *server* is the first to know something: a new chat message has arrived, a stock price has moved, a driver is two minutes away, a colleague has started typing. This chapter is about the family of techniques that let a server tell a client about something *now*, without waiting to be asked.

Picture a restaurant kitchen and a diner. **Short polling** is the diner walking to the kitchen every five minutes to ask “is my food ready?”. **Long polling** is the diner asking once and the chef saying “wait here, I’ll tell you as soon as it is”. **Server-Sent Events** is a waiter assigned to the diner’s table who walks over every time a course is ready. **WebSockets** is a telephone line between table and kitchen that either side can talk into at any moment. **WebRTC** lets two diners talk to each other directly across the room instead of passing notes through the kitchen. **Mobile push** is the restaurant sending a text message to a diner who has gone home: the phone company decides when it arrives, and it might not arrive at all. Every technique in this chapter is a trade between how quickly the news arrives, how much it costs to keep the line open, and how much infrastructure must understand the line.

9.1 Learning objectives

After this chapter you should be able to:

- Explain the mechanism, wire format and failure modes of short polling, long polling, Server-Sent Events (SSE), WebSockets, WebRTC and mobile push.
- Calculate the request rate, wasted requests and added latency of a polling design from the number of clients and the polling interval.
- Walk through the RFC 6455 WebSocket opening handshake, compute `Sec-WebSocket-Accept`, and decode a WebSocket frame byte by byte, including masking.
- Describe how ICE, STUN and TURN establish a peer-to-peer WebRTC session, and choose between mesh, SFU and MCU topologies with bandwidth arithmetic.
- Design a mobile push pipeline using APNs and FCM that tolerates dropped notifications, stale device tokens and collapsed messages.

- Size a connection gateway fleet for 10 million concurrent connections, including memory, server count, heartbeat load and reconnect-storm behavior.
- Design a pub/sub backplane, a presence system and a graceful-drain deployment procedure for long-lived connections.
- Choose the right real-time technique for a given product requirement and defend the choice with a comparison of cost, latency and operational complexity.

9.2 Prerequisites

- TCP connections, the three-way handshake and head-of-line blocking — §7.7 (needed because every technique here rides on long-lived TCP or UDP flows).
- HTTP/1.1, HTTP/2, HTTP/3 and QUIC — §7.9 (SSE and WebSockets interact with HTTP versions in non-obvious ways).
- TLS 1.3 — §7.10 (handshake cost dominates reconnect storms).
- WebSockets introduction — §7.11 (this chapter is the deep dive).
- Proxies, reverse proxies and NAT — §7.12, §7.13 (they break long-lived connections and motivate STUN/TURN).
- I/O models and event loops (epoll, kqueue, async/await) — §6.13, §6.15 (how one server holds hundreds of thousands of idle connections).
- Little's Law — §4.11 (concurrent connections = arrival rate × connection lifetime).
- Load balancing and session affinity — §19.3, §19.6 (sticky routing for connections).
- Retries with exponential backoff and jitter — §22.12 (reconnect storms).

9.3 Short polling

9.3.1 What short polling is

DEFINITION

Definition — Short polling. A technique in which the client sends an ordinary HTTP request at a fixed interval T to ask whether new data exists; the server answers immediately, either with the new data or with an empty response, and the connection's request ends.

First-principles explanation

A child in the back seat asking "are we there yet?" every minute is short polling. The parent answers right away, almost always with "no". The child learns of arrival at most one minute late, and the parent spends most of the trip answering a question whose answer has not changed. Nothing about the car or the road had to change to support this protocol, which is precisely why it is popular.

Why does this exist?

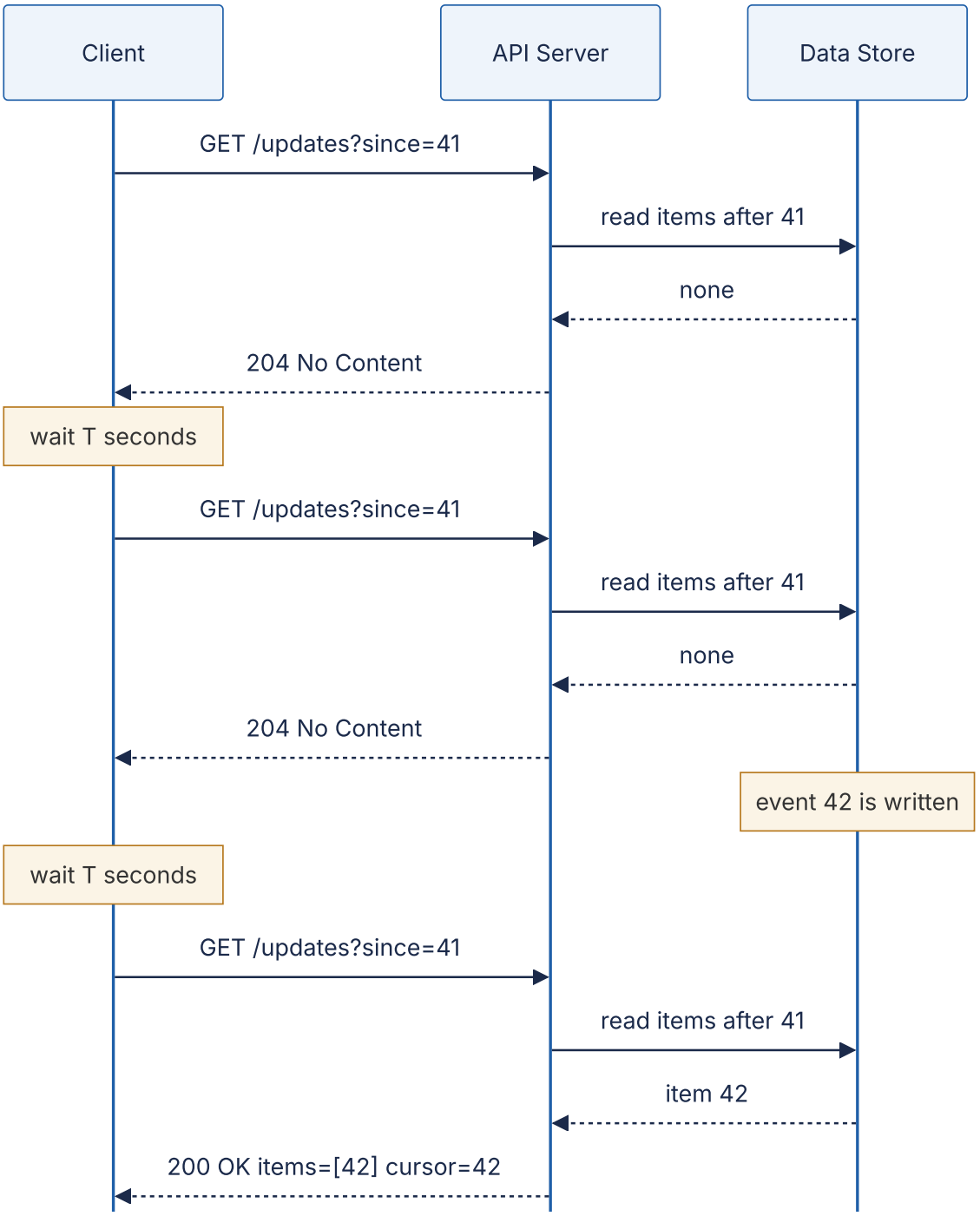
Short polling needs nothing beyond what every HTTP client, proxy, CDN, firewall and load balancer already supports. It was the first way web pages showed “live” data, and it remains the right answer when updates are infrequent, a few seconds of staleness is acceptable and clients are few — a build-status page polled every 30 seconds by a few hundred engineers, for example.

How it works internally

1. The client starts a timer with period T (for example, 5 seconds).
2. When the timer fires, the client sends `GET /updates?since=<cursor>`. The **cursor** is an opaque value (a sequence number or timestamp) identifying the last item the client has seen.
3. The server looks up anything newer than the cursor and responds immediately: `200 OK` with items and a new cursor, or `204 No Content` (or `200` with an empty list, or `304 Not Modified` with an `ETag`) when nothing changed.
4. The client applies the items, stores the new cursor and waits for the next tick.
5. With HTTP keep-alive (§7.16), the TCP and TLS connection is reused across polls, so each poll costs one request/response round trip rather than a full handshake.

Architecture diagram

Figure 9.1 — Short polling.



Reading the diagram. The client, a stateless API server and the data store exchange only synchronous HTTP over a reused keep-alive connection (solid = request, dashed = response); the server never initiates. Event 42 is written between polls, so the client learns of it up to T seconds late — inherent, not a bug. The data store receives one read per poll whether or not anything changed, which is the failure point at scale. Scaling is easy: stateless servers behind a load balancer, with a cache in front of the “anything new?” query.

Mathematical foundations

FORMULA

Short-polling request rate

$$R = \frac{N}{T}$$

Symbol	Meaning	Units
R	Request rate arriving at the servers	requests/second
N	Number of concurrently active (polling) clients	clients
T	Polling interval	seconds

FORMULA

Fraction of wasted (empty) polls and added latency

If updates for a client arrive as a Poisson process with rate λ, the probability that a given interval contains no update is

$$P_{\text{empty}} = e^{-\lambda T} \approx 1 - \lambda T \quad (\text{when } \lambda T \ll 1)$$

and the average delay between an event occurring and the client seeing it is

$$D_{\text{avg}} = \frac{T}{2} + t_{\text{rtt}}$$

Symbol	Meaning	Units
P_{empty}	Fraction of polls that return nothing	dimensionless (0–1)
λ	Average update rate per client	events/second
T	Polling interval	seconds
D_{avg}	Average notification delay	seconds
t_{rtt}	Network round-trip time for one poll	seconds

WORKED EXAMPLE

Worked example — A notification badge polled every 5 seconds.

A web app has $N = 1,000,000$ users online at peak. Each tab polls `/notifications` every $T = 5$ s. A typical user receives one notification every 10 minutes, so $\lambda = 1/600 \approx 0.00167$ events/s.

1. Request rate: $R = 1,000,000/5 = 200,000$ requests/s.
2. Empty fraction: $\lambda T = 5/600 = 0.00833$; $P_{\text{empty}} = e^{-0.00833} \approx 0.9917$. So **99.2 %** of requests — about 198,300 per second — return nothing.
3. Useful requests: $200,000 \times 0.0083 \approx 1,700$ per second actually carry a notification.
4. Header overhead: assume about 800 bytes of request headers (cookies, user-agent, auth token) and 200 bytes of response headers on HTTP/1.1 — about 1 KB per poll (SI units, 1 KB = 1,000 bytes). $200,000 \times 1$ KB $= 200$ MB/s ≈ 1.6 Gbit/s of traffic that mostly says “nothing new”. HTTP/2 header compression (HPACK) shrinks repeated headers considerably, but the request count and server work remain.
5. Average delay: $D_{\text{avg}} \approx 2.5$ s plus one round trip.

Interpretation. You are paying for 200,000 requests per second of capacity — load balancers, TLS termination, application threads and a database or cache lookup each — to deliver about 1,700 useful updates per second. Halving the interval to 2.5 s to improve freshness *doubles* the bill.

Engineering implication. Short polling’s cost scales with N/T, not with the rate of real events. When λT is small (rare events, short interval), switch to a push technique; when events are frequent relative to the interval (for example, a dashboard that changes every second and is polled every 10 s), polling’s waste is low and its simplicity wins.

Table 9.1 — Short-polling cost for 1 million clients as the interval shrinks (λ = 1 event per 10 minutes per client).

Interval T	Requests/s	Empty polls	Avg. delay	Header traffic (≈1 KB/poll)
60 s	16,667	90.5 %	30 s	16.7 MB/s
30 s	33,333	95.1 %	15 s	33.3 MB/s
10 s	100,000	98.3 %	5 s	100 MB/s
5 s	200,000	99.2 %	2.5 s	200 MB/s
1 s	1,000,000	99.8 %	0.5 s	1 GB/s

The table shows the central tension: every halving of delay doubles the cost, while the useful work stays constant at about 1,667 notifications per second.

Real-world industry applications

> INDUSTRY EXAMPLE
>
> **Status dashboards and job progress pages.** A CI system's build page or a cloud console's "creating instance…" spinner typically polls a status endpoint every few seconds. The number of watchers is small, staleness of a few seconds is invisible to users, and the endpoint can be served from a cache, so the waste is cheap and the operational simplicity (no long-lived connections, works through every proxy) dominates. *Evidence: Illustrative* — a common pattern used for teaching; not a claim about any specific product.

Advantages and disadvantages

- **Advantages:** works everywhere (every proxy, CDN, firewall, HTTP version); servers stay stateless; trivially cacheable (`ETag`, `Cache-Control: max-age`); easy to rate-limit and to reason about; failures are self-healing because the next poll simply retries.
- **Disadvantages:** latency is bounded below by $T/2$ on average; cost grows with clients and polling frequency rather than with real events; mobile radios are woken on every poll, draining battery; synchronized clients (for example, all tabs opened at 09:00) create load spikes unless the timer is jittered.

Alternatives and trade-offs

Long polling (§9.4) removes empty responses and the $T/2$ delay; SSE (§9.5) and WebSockets (§9.6) remove per-update request overhead. **Adaptive polling** — lengthening T while nothing changes, or honoring a server-supplied `Retry-After` hint — is a cheap middle ground.

Failure scenarios

Table 9.2 — Short-polling failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Synchronized poll spikes	Clients start timers at the same moment (deploy, page load at shift start)	Request-rate graph shows sawtooth at multiples of T	Shed load with `429` + `Retry-After`	Add ±20 % random jitter to every timer
Database overload	Every empty poll hits the database	DB CPU tracks request rate, not write rate	Cache "latest cursor per user" in memory	Serve "anything new?" from a cache or version counter
Missed updates	Client uses wall-clock timestamps as cursor; clock skew or same-millisecond writes	Users report gaps	Re-fetch from an earlier cursor	Use monotonically increasing sequence numbers

Common misconceptions

COMMON MISTAKE

"Polling every second is fine; requests are cheap." A request that returns nothing still costs a TLS record, a load-balancer decision, an application thread slot, an authentication check and usually a cache or database lookup. Multiplied by N/T, "cheap" becomes the dominant load on the system. Always compute $R = N/T$ before choosing an interval.

9.4 Long polling

9.4.1 What long polling is

DEFINITION

Definition — Long polling. A technique in which the client sends an HTTP request that the server deliberately does not answer until either new data is available or a server-side timeout T_{hold} expires; on receiving any response, the client immediately sends the next request.

First-principles explanation

Now the child asks once and the parent says "I'll tell you when we arrive — and if nothing happens in half an hour I'll say so anyway". The news arrives the instant it happens, but the parent must *remember* that a question is pending — that memory is the new cost.

Why does this exist?

Mid-2000s webmail and chat needed sub-second updates, but browsers could only make JavaScript-initiated HTTP requests. Nothing in HTTP forbids a server from answering slowly, so holding the request open approximates push using only standard HTTP. Alex Russell named this family of techniques **Comet** in 2006 [Russell 2006]; the CometD project's **Bayeux** protocol standardized a message format, and RFC 6202 documented the pitfalls [Loreto et al. 2011]. Long polling survives as the universal fallback transport in libraries such as Socket.IO.

How it works internally

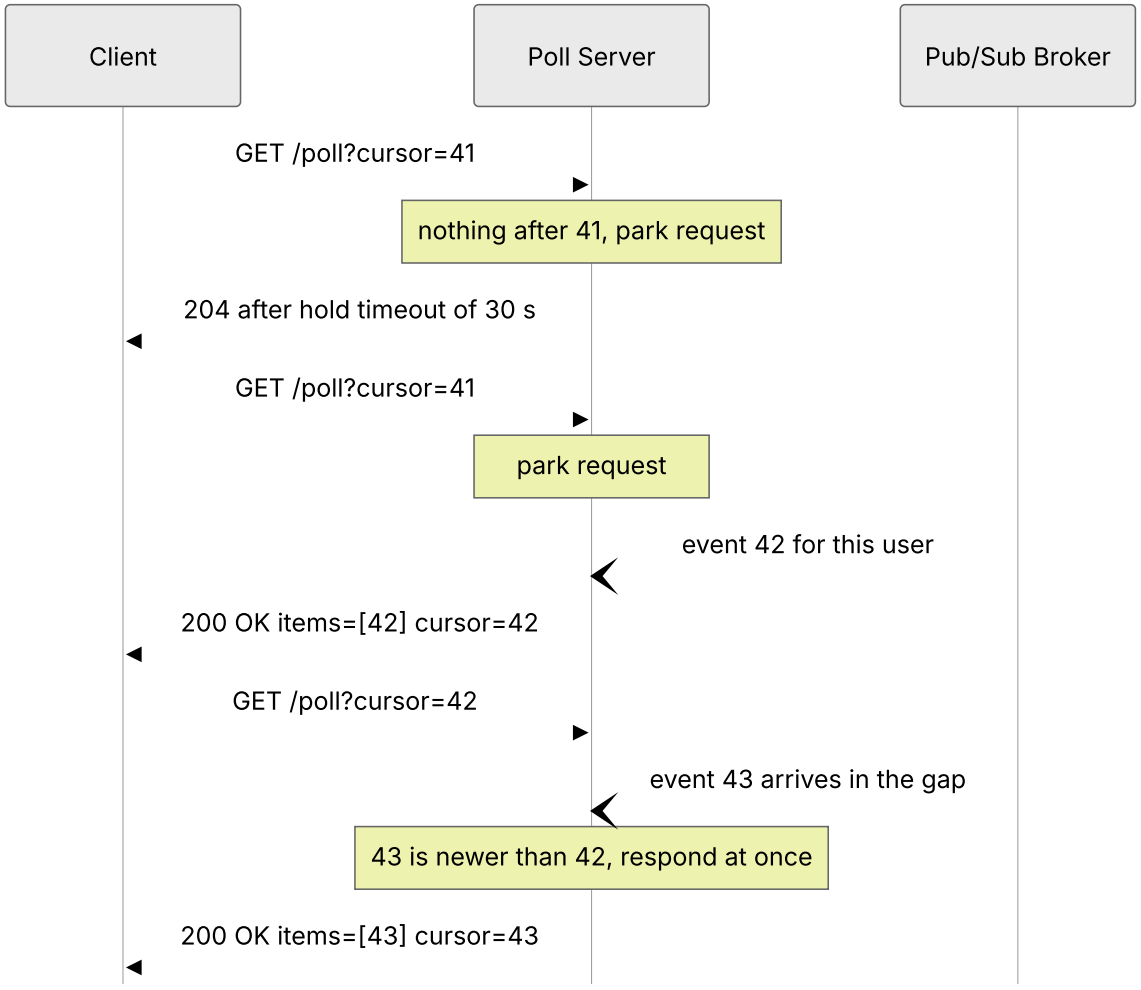
1. The client sends `GET /poll?cursor=41`.
2. The server checks for items after cursor 41. If any exist, it responds immediately (this is exactly short polling's fast path).
3. If none exist, the server *parks* the request in an in-memory waiter list keyed by user or topic. On an event-loop server (§6.13) a parked request costs only memory, not a thread.
4. When an event for that user arrives from a broker, the server wakes the waiter and responds with the new items and cursor.
5. If no event arrives within T_{hold} (commonly 20–55 seconds), the server responds `204 No Content` or an empty list with the same cursor.
6. On any response the client immediately re-polls with the latest cursor (after a back-off delay if the response was an error).

Timeouts. T_{hold} must be shorter than every idle timeout on the path. The AWS Application Load Balancer's default idle timeout is 60 s and NGINX's `proxy_read_timeout` defaults to 60 s (as of 2024); corporate proxies may be shorter, so 25–30 s is a common safe choice. If an intermediary cuts the request first, the client sees an error instead of a clean empty response — and if it retries without backoff, an error storm.

Ordering and loss. Between response k and request k+1 there is a one-round-trip gap with no outstanding request. The cursor closes it: the server keeps a per-user buffer of recent events with monotonically increasing sequence numbers, and each request asks for "everything after c". Clients apply items in sequence order and discard duplicates (an event racing a timeout may be sent twice). Never run two concurrent long polls per client: responses can arrive out of order. A parked request also occupies one of the browser's roughly six HTTP/1.1 connections per origin [Loreto et al. 2011].

Architecture diagram

Figure 9.2 — Long polling with cursors and a hold timeout.



Reading the diagram. The poll server receives events asynchronously (open arrowheads) from a pub/sub broker. The first request parks and ends empty after the 30 s hold timeout; the second is answered the moment event 42 arrives — one network hop instead of $T/2$. Event 43 arrives while the client is re-polling; the cursor and the server-side buffer ensure nothing is lost. If the client reconnects to a *different* server, that server must answer "after 42", so the buffer must be shared or routing sticky (§9.9.4). Failure points: intermediaries that time out first, and in-memory buffers lost on restart. Capacity is planned in concurrent held connections.

Mathematical foundations

FORMULA

Long-polling request rate

With Poisson events at rate λ per client and hold timeout T_h, each request lasts $\min(T_h, X)$ where X is the time to the next event. Its mean is

$$E[\text{duration}] = \frac{1 - e^{-\lambda T_h}}{\lambda}, \qquad R = \frac{N}{E[\text{duration}]}, \qquad C_{\text{open}} \approx N$$

Symbol	Meaning	Units
λ	Average update rate per client	events/second
T_h	Server hold timeout	seconds
$E[\text{duration}]$	Mean lifetime of one long-poll request	seconds
N	Number of connected clients	clients
R	Request rate at the servers	requests/second
C_{open}	Concurrently open requests (Little's Law: $R \times E[\text{duration}]$)	requests

WORKED EXAMPLE

Worked example — The same 1 million users, now with long polling.

Use the numbers from §9.3: $N = 1,000,000$, $\lambda = 1/600$ events/s, and a hold timeout $T_h = 30$ s.

1. $\lambda T_h = 30/600 = 0.05$; $e^{-0.05} = 0.9512$.
2. $E[\text{duration}] = (1 - 0.9512)/(1/600) = 0.0488 \times 600 \approx 29.3$ s.
3. $R = 1,000,000/29.3 \approx 34,000$ requests/s, versus 200,000 requests/s for 5-second short polling — about **5.9× fewer** requests.
4. Average notification delay: about one network round trip (tens of milliseconds) instead of 2.5 s.
5. Concurrent open requests: $34,000 \times 29.3 \approx 1,000,000$ — every client holds one open request at all times.

Interpretation. Long polling improves both cost and latency, but it shifts the bottleneck from request throughput to *concurrent held connections*. 95 % of responses are still empty timeouts ($e^{-0.05}$), each carrying full HTTP headers.

Engineering implication. Long polling needs an asynchronous server that can park a million requests and an event-routing layer that wakes the right one — the same infrastructure as a WebSocket gateway (§9.9). Once you have built that, SSE or WebSockets usually deliver more for the same effort.

Real-world industry applications

INDUSTRY EXAMPLE

Universal fallback transports. Real-time libraries such as Socket.IO and SockJS offer long polling as a transport for networks where WebSockets are blocked by proxies or firewalls. Socket.IO's documentation notes that when HTTP long-polling is enabled with several server nodes, the load balancer must use sticky sessions, because successive polls of one session must reach the node that holds its state. *Evidence: Primary* — Socket.IO documentation, "Using multiple nodes" [Socket.IO Docs].

Advantages and disadvantages

- **Advantages:** near-instant delivery over plain HTTP; works through almost every proxy; client logic is simple; degrades to polling when events are frequent.
- **Disadvantages:** every message still carries full HTTP headers and costs a new request; a round-trip gap after each response (mitigated by cursors); holds a connection per client, so servers must be asynchronous; intermediary timeouts must be un-

derstood; occupies one of the browser's HTTP/1.1 connections per origin; strictly server-to-client (client-to-server messages need separate requests).

Alternatives and trade-offs

SSE (§9.5) gives the same reach for server-to-client data with one request for many messages and built-in resumption; WebSockets (§9.6) remove per-message HTTP overhead for bidirectional traffic. Long polling wins where streaming responses are broken — some antivirus products and corporate proxies buffer responses until they complete, which ruins SSE but leaves long polling intact.

Failure scenarios

Table 9.3 — Long-polling failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Requests cut off at 60 s with 504	Hold timeout exceeds a proxy or LB idle timeout	Spike in 504/502 at exactly the intermediary's timeout	Lower hold timeout	Keep hold timeout below the smallest idle timeout on the path
Lost events	Server buffers in memory and restarts; client lands on another node	Sequence gaps detected by client	Client resyncs with a full fetch	Shared event store or sticky routing plus resync-on-gap
Retry storm	Clients re-poll instantly after errors	Request rate jumps far above N/T_h	Shed with 503 + `Retry-After`	Exponential backoff with jitter on errors (§22.12)

Common misconceptions

> COMMON MISTAKE
>
> **"Long polling is just polling with a longer interval."** No — the interval is not a client timer at all. The *server* decides when to answer, so delivery latency is one hop, not $T/2$. What it shares with polling is the per-message HTTP cost and the reconnect after every response.

9.5 Server-Sent Events

9.5.1 What SSE is

> DEFINITION
>
> **Definition — Server-Sent Events (SSE).** A W3C/WHATWG-standardized mechanism in which a client opens one long-lived HTTP request and the server streams a sequence of UTF-8 text events down the response body using the `text/event-stream` format; browsers expose it through the `EventSource` API, which parses events and reconnects automatically, resuming from the last received event ID [WHATWG HTML §9.2].

First-principles explanation

Long polling is like ordering a newspaper one issue at a time: each issue arrives, and then you must place the next order. SSE is a newspaper subscription: you sign up once and issues keep arriving at your door. If you move house (the connection drops), you tell the newspaper "the last issue I received was number 1,042", and they send you everything from 1,043 onward.

Why does this exist?

Long polling wastes a full request per message. Many features need only server-to-client data: feeds, notifications, live scores, progress bars, and the token-by-token output of large language models (Chapter 73). SSE standardizes "keep the response open and keep writing", adds a tiny text framing, and builds reconnection and resumption into the browser.

How it works internally

1. The client creates `new EventSource("/events")` (optionally `{ withCredentials: true }` for cross-origin cookies). The browser sends `GET /events` with `Accept: text/event-stream`.
2. The server responds `200 OK` with `Content-Type: text/event-stream` and `Cache-Control: no-cache`, then **does not end the response**. It writes events as they occur.
3. Each event is a block of `field: value` lines terminated by a blank line. The fields are:
   - `data:` — the payload; several `data:` lines are joined with newline characters.
   - `event:` — an event type; the browser dispatches it to listeners registered with `addEventListener("<type>", …)`; without it the event fires `onmessage`.
   - `id:` — sets the stream's **last event ID**.
   - `retry:` — an integer in milliseconds; sets the reconnection delay.

    - A line beginning with `:` is a **comment**, ignored by the parser — servers send one every 15–30 seconds as a heartbeat to stop intermediaries from closing an idle connection.
4. If the connection drops because of a network error, the browser waits the reconnection time (initially an implementation-defined value of a few seconds, or whatever `retry:` set) and re-sends the request with a `Last-Event-ID` header containing the last `id:` it received. The server replays events after that ID from a buffer and then continues live.
5. The server can stop reconnection permanently by responding `204 No Content`. A response with a status other than 200, or a wrong `Content-Type`, makes the browser *fail* the connection rather than retry — so a load balancer returning `503` during an outage silently turns off auto-reconnect, and your application code must notice (`readyState === EventSource.CLOSED`) and recreate the `EventSource` with backoff.

A stream on the wire looks like this:

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
Cache-Control: no-cache

retry: 5000

: heartbeat

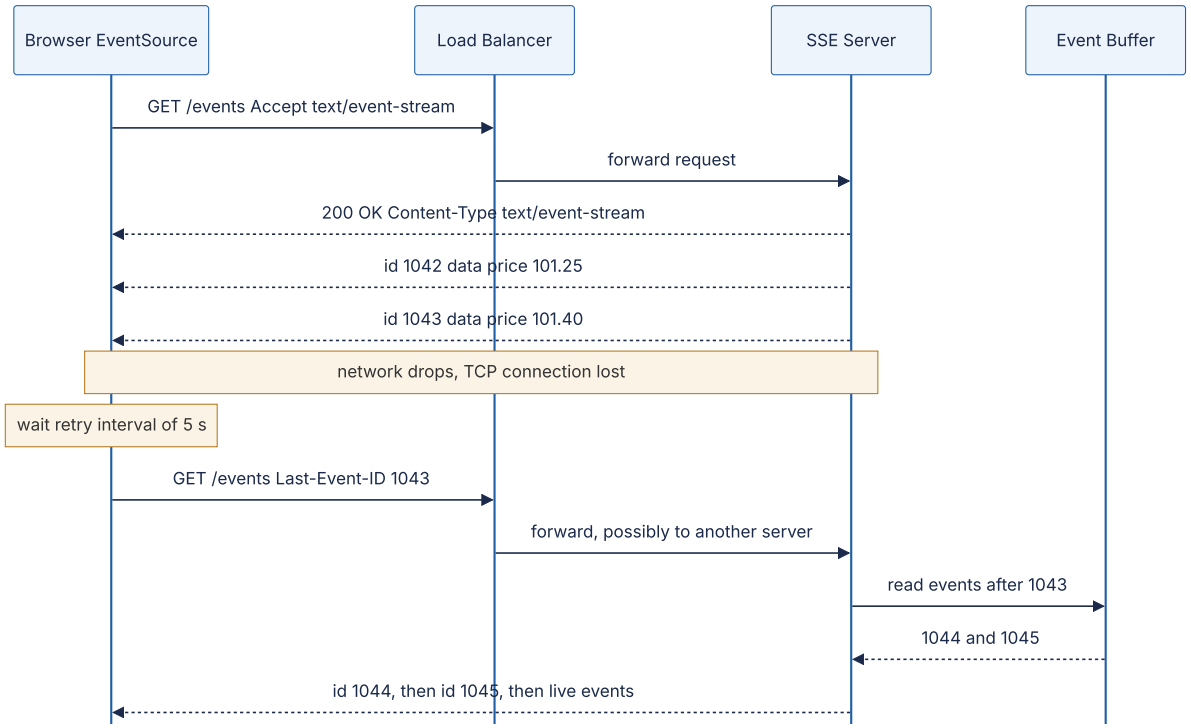
id: 1042
event: price
data: {"symbol":"ACME","price":101.25}

id: 1043
event: price
data: {"symbol":"ACME","price":101.40}
```

Interaction with HTTP versions. Over HTTP/1.1 each `EventSource` holds a whole TCP connection, and browsers allow only about six HTTP/1.1 connections per origin *across all tabs*, so a seventh tab can stall. Over HTTP/2 (§7.9) each stream is multiplexed on one shared connection, limited by the server's `SETTINGS_MAX_CONCURRENT_STREAMS` (commonly 100 or more), and a new stream needs no TCP or TLS handshake; the price is TCP head-of-line blocking across all streams, which HTTP/3 removes. Finally, reverse proxies that **buffer** responses (NGINX does by default) hold events back: disable buffering on stream routes (`proxy_buffering off;` or the header `X-Accel-Buffering: no`) and keep compression middleware off them.

Architecture diagram

Figure 9.3 — SSE streaming, disconnection and resumption with `Last-Event-ID`.



Reading the diagram. Every dashed arrow from the server is a chunk of the *same* streamed response. After a network failure the browser reconnects by itself with `Last-Event-ID`; the load balancer may pick a different server, which is why the event buffer (Redis Streams, Kafka, a replicated ring) lives outside the SSE server. Data flows only server-to-client; client actions use ordinary HTTP requests. Failure points: buffering proxies, idle timeouts on silent streams (fixed by heartbeat comments), and a buffer that no longer reaches back to the client's ID (force a full resync). SSE servers scale like the gateways in §9.9.

Code example

The following SSE endpoint uses only the Python standard library and has been run and tested with `curl -N`. It keeps a bounded history so that reconnecting clients resume from `Last-Event-ID`.

```
        """Minimal Server-Sent Events endpoint using only the standard library.

        Run:   python3 sse_server.py
        Test:  curl -N http://127.0.0.1:8080/events
               curl -N -H "Last-Event-ID: 3" http://127.0.0.1:8080/events
        """
        import asyncio
        import json
        import time

        HISTORY: list[tuple[int, str]] = []    # (event id, JSON payload) ring buffer
        HISTORY_LIMIT = 1000
        new_event = asyncio.Condition()

        async def producer() -> None:
            """Pretend to be a business event source: one price tick per second."""
            event_id = 0
            while True:
                await asyncio.sleep(1)
                event_id += 1
                payload = json.dumps({"symbol": "ACME", "price": 100 + event_id, "ts":
time.time()})
                async with new_event:
                    HISTORY.append((event_id, payload))
                    del HISTORY[:-HISTORY_LIMIT]          # keep the buffer bounded
                    new_event.notify_all()

        def format_event(event_id: int, data: str) -> bytes:
            return f"id: {event_id}\nevent: price\ndata: {data}\n\n".encode()

        async def handle(reader: asyncio.StreamReader, writer: asyncio.StreamWriter) ->
None:
            # 1. Read the request line and headers (no body for GET).
            request_line = await reader.readline()
            headers = {}
            while (line := await reader.readline()) not in (b"\r\n", b"\n", b""):
                name, _, value = line.decode().partition(":")
                headers[name.strip().lower()] = value.strip()
            if not request_line.startswith(b"GET /events"):
                writer.write(b"HTTP/1.1 404 Not Found\r\nContent-Length: 0\r\n\r\n")
                await writer.drain(); writer.close(); return

            # 2. Resume point: the browser sends Last-Event-ID automatically on recon-
nect.
            try:
                last_seen = int(headers.get("last-event-id", "0"))
            except ValueError:
                last_seen = 0

            writer.write(b"HTTP/1.1 200 OK\r\n"
                         b"Content-Type: text/event-stream\r\n"
                         b"Cache-Control: no-cache\r\n"
                         b"Connection: keep-alive\r\n\r\n"
                         b"retry: 5000\n\n")          # ask client to wait 5 s before
reconnecting
            try:
```

```
                while True:
                    # 3. Send everything newer than last_seen (replay, then live).
                    pending = [(i, d) for i, d in HISTORY if i > last_seen]
                    for event_id, data in pending:
                        writer.write(format_event(event_id, data))
                        last_seen = event_id
                    if not pending:
                        writer.write(b": keep-alive\n\n")   # comment line; defeats idle
timeouts
                    await writer.drain()                   # backpressure: wait if
socket buffer full
                    async with new_event:
                        try:
                            await asyncio.wait_for(new_event.wait(), timeout=15)
                        except asyncio.TimeoutError:
                            pass
            except (ConnectionResetError, BrokenPipeError):
                pass
            finally:
                writer.close()

        async def main() -> None:
            asyncio.create_task(producer())
            server = await asyncio.start_server(handle, "127.0.0.1", 8080)
            async with server:
                await server.serve_forever()

        if __name__ == "__main__":
            asyncio.run(main())
```

On the browser side, consuming this stream takes four lines:

```
const es = new EventSource("/events");
es.addEventListener("price", (e) => render(JSON.parse(e.data)));
es.onerror = () => { if (es.readyState === EventSource.CLOSED) scheduleRecreate(); };
// the browser reconnects by itself after network errors and sends Last-Event-ID
```

What to notice:

- The response never ends; the server writes `id:` / `event:` / `data:` blocks separated by blank lines. Curl started with `-H "Last-Event-ID: 2"` received events 3, 4, 5… — the replay-then-live behavior the browser gets for free.
- `await writer.drain()` is the backpressure point: if a client reads slowly, the coroutine pauses instead of letting the process buffer unbounded data (§9.6.7).
- The `: keep-alive` comment is invisible to `EventSource` but keeps proxies and load balancers from closing an idle connection.
- The in-memory `HISTORY` would be lost on restart and is not shared between servers; production systems put it in Redis Streams, Kafka or a database so any server can replay.

Real-world industry applications

INDUSTRY EXAMPLE

Streaming responses from large-language-model APIs. Several widely used LLM HTTP APIs stream generated tokens to clients as `text/event-stream` responses, because generation is strictly one-directional (server to client), each response is short-lived (seconds to minutes), and SSE passes through standard HTTP infrastructure, authentication and rate limiting unchanged. Chapter 73 discusses time to first token (TTFT) and inter-token latency (ITL), which is what this streaming exposes to users. *Evidence: Illustrative* — a widely observed pattern; consult each provider's API documentation for specifics.

Advantages and disadvantages

- **Advantages:** plain HTTP (works with existing auth, cookies, CORS, load balancers, HTTP/2 and HTTP/3); automatic reconnection and resumption built into browsers; tiny per-event framing (tens of bytes); simple to implement on any server that can flush a response; easy to debug with `curl -N`.
- **Disadvantages:** server-to-client only; UTF-8 text only (binary must be base64-encoded, +33 %); the standard `EventSource` cannot set custom request headers (use cookies, a short-lived token in the URL, or a `fetch`-based stream reader instead); HTTP/1.1 connection-per-origin limits; buffering proxies and compression can delay events; non-200 responses stop automatic reconnection.

Failure scenarios

Table 9.4 — SSE failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Events arrive in bursts minutes late	Reverse proxy or compression buffers the response	Server log timestamps vs. client receipt times diverge	Disable buffering on the route	`proxy_buffering off` / `X-Accel-Buffering: no`; no gzip on streams
Streams die every 60 s	Idle timeout on LB or proxy	Reconnect rate ≈ connections / 60 s	Send heartbeats	Comment line every 15–30 s, below every idle timeout
Clients never reconnect after outage	LB returned 502/503; `EventSource` fails permanently on non-200	Connected-client gauge drops and stays low	Application recreates `EventSource` with jittered backoff	Handle `readyState === CLOSED` in `onerror`
Seventh tab hangs	HTTP/1.1 six-connection limit per origin	Requests pending in browser dev tools	Close tabs	Serve SSE over HTTP/2; share one stream across tabs (`BroadcastChannel`, `SharedWorker`)

Common misconceptions

COMMON MISTAKE

"SSE is obsolete because WebSockets exist." SSE is the better tool for one-directional streams: it rides on ordinary HTTP (so HTTP/2 multiplexing, standard authentication and observability all apply), it resumes automatically, and it needs no special proxy configuration. Choose WebSockets when the *client* also sends frequent low-latency messages, or when you need binary frames.

9.6 WebSockets in depth

9.6.1 What a WebSocket is

DEFINITION

Definition — WebSocket. A protocol, standardized in RFC 6455 [Fette & Melnikov 2011], that starts as an HTTP/1.1 request, *upgrades* the underlying TCP connection, and thereafter carries a full-duplex sequence of framed messages in both directions with 2–14 bytes of framing overhead per frame. URLs use the schemes `ws://` (plain TCP, default port 80) and `wss://` (over TLS, default port 443).

First-principles explanation

HTTP is a postal system: every letter needs a full envelope, and the recipient can only reply, never write first. A WebSocket is a telephone call that begins with a short postal exchange — "may we switch to the telephone?", "yes" — after which either party speaks whenever it likes for a few bytes of overhead. Because the call starts as a letter, it passes through the same post offices (ports 80/443, HTTP proxies).

Why does this exist?

Before RFC 6455 (2011), bidirectional browser communication meant gluing a long poll (downstream) to separate POSTs (upstream), each carrying hundreds of bytes of headers. Games, collaborative editors, trading screens and chat needed many small messages per second both ways. RFC 6455 reuses HTTP's ports for the initial connection, keeps framing minimal, keeps the browser's origin-based security model, and avoids giving web pages a raw TCP socket that could attack other protocols.

9.6.2 The opening handshake

How it works internally

1. The client opens a TCP connection (and, for `wss://`, a TLS session) to the server.
2. It sends an HTTP/1.1 `GET` with these headers:
   - `Upgrade: websocket` and `Connection: Upgrade` — ask to switch protocols.
   - `Sec-WebSocket-Key:` — 16 random bytes, base64-encoded, fresh for each connection.
   - `Sec-WebSocket-Version: 13` — the only version in RFC 6455.
   - `Origin:` — set by browsers; the server must check it (see the security note below).
   - Optionally `Sec-WebSocket-Protocol:` (application subprotocols, for example `graphql-transport-ws` or `mqtt`) and `Sec-WebSocket-Extensions:` (for example `permessage-deflate`, RFC 7692).
3. The server validates the request and responds `HTTP/1.1 101 Switching Protocols` with `Upgrade: websocket`, `Connection: Upgrade` and **`Sec-WebSocket-Accept`**, computed as base64(SHA-1(key + `258EAFA5-E914-47DA-95CA-C5AB0DC85B11`)). That fixed GUID appears in RFC 6455; the computation proves that the server actually understood a WebSocket handshake (and was not, say, an unsuspecting HTTP server or cache replaying a response). It is not an authentication mechanism.
4. From the next byte onward, both sides speak the WebSocket framing protocol on the same TCP connection. HTTP is gone.

```
GET /chat HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Origin: https://example.com
Sec-WebSocket-Version: 13

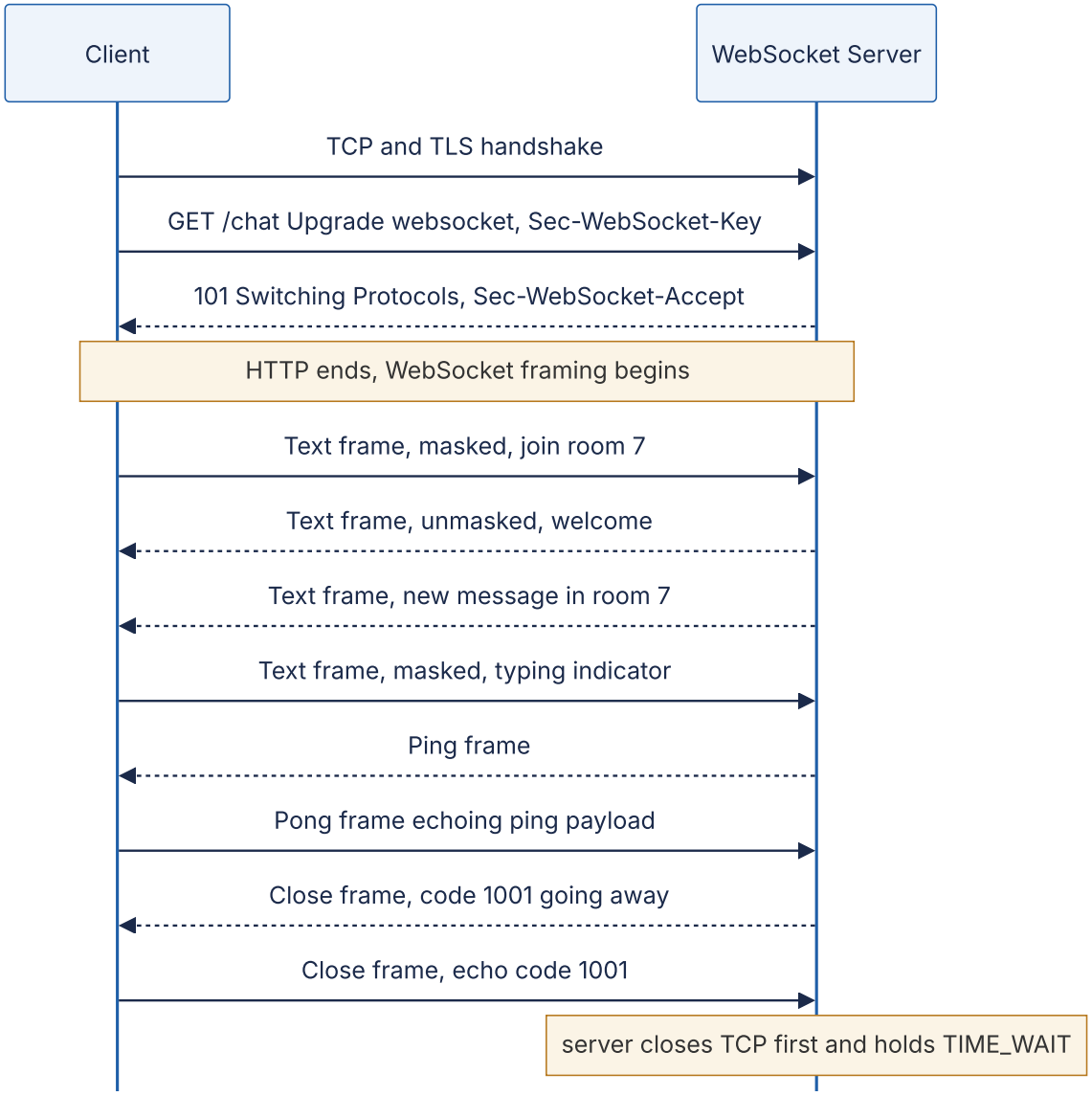
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

WARNING / TRADE-OFF

Cross-Site WebSocket Hijacking. Browsers attach cookies to the WebSocket handshake but WebSockets are not subject to CORS. If your server authenticates the handshake by cookie and does not check `Origin` against an allow-list, any website the user visits can open an authenticated WebSocket to your service. Check `Origin`, or authenticate with a short-lived token sent in the first message or in the URL rather than relying on cookies alone.

HTTP/2 and HTTP/3. RFC 6455's `Upgrade` mechanism exists only in HTTP/1.1. RFC 8441 defines an "extended CONNECT" method that carries a WebSocket over a single HTTP/2 stream, and RFC 9220 does the same for HTTP/3; support across browsers, proxies and servers is uneven (as of 2024), so most deployments still run WebSockets over a dedicated HTTP/1.1-upgraded TCP connection.

Figure 9.4 — WebSocket opening handshake, message exchange, heartbeat and closing handshake.



Reading the diagram. One TCP connection carries the whole session. Only the upgrade request and the `101` response are HTTP; afterward frames flow independently in both directions — the server pushes a chat message unasked, the client sends a typing indicator without waiting. Client frames are masked, server frames are not (§9.6.4). Ping/Pong is a protocol-level liveness check. In the closing handshake each side sends one Close frame and the server closes TCP first, so `TIME_WAIT` (§7.7) accumulates on the server. Failure points: proxies that do not understand `Upgrade`, idle timeouts, and abrupt TCP loss with no Close frame (reported locally as 1006). The server is stateful for the connection's lifetime, which drives §9.9.

Code example

```
        import base64
        import hashlib

        GUID = "258EAFA5-E914-47DA-95CA-C5AB0DC85B11"  # fixed by RFC 6455

        def accept_key(sec_websocket_key: str) -> str:
            """Compute Sec-WebSocket-Accept from the client's Sec-WebSocket-Key."""
            digest = hashlib.sha1((sec_websocket_key + GUID).encode("ascii")).digest()
            return base64.b64encode(digest).decode("ascii")

        print(accept_key("dGhlIHNhbXBsZSBub25jZQ=="))   # s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

What to notice: the output matches the example in RFC 6455 exactly. SHA-1's known collision weaknesses do not matter here because the value is a protocol-confirmation nonce, not a security signature.

9.6.3 Frames

How it works internally

After the handshake, every message is sent as one or more **frames**. The header is at least 2 bytes:

Table 9.5 — WebSocket frame header fields (RFC 6455 §5.2).

Field	Size	Meaning
FIN	1 bit	1 = this is the final fragment of a message
RSV1–3	3 bits	Reserved for extensions (RSV1 marks a compressed message under `permessage-deflate`)
Opcode	4 bits	Frame type (Table 9.6)
MASK	1 bit	1 = payload is masked; must be 1 for client→server, 0 for server→client
Payload length	7 bits	0–125 = actual length; 126 = next 2 bytes hold length; 127 = next 8 bytes hold length
Extended length	0, 2 or 8 bytes	Present when length field is 126 or 127
Masking key	0 or 4 bytes	Present when MASK = 1
Payload	variable	Application data (after unmasking)

Table 9.6 — WebSocket opcodes.

Opcode	Type	Notes
0x0	Continuation	Subsequent fragment of a fragmented message
0x1	Text	Payload must be valid UTF-8
0x2	Binary	Arbitrary bytes (Protocol Buffers, MessagePack, media)
0x8	Close	Optional 2-byte status code + UTF-8 reason
0x9	Ping	Receiver must answer with a Pong carrying the same payload
0xA	Pong	Reply to Ping, or unsolicited one-way heartbeat

Control frames (Close, Ping, Pong) carry at most 125 bytes of payload, must not be fragmented, and may be injected *between* the fragments of a large data message — so a 100 MB upload does not delay a heartbeat. **Fragmentation** lets a sender stream a message of unknown length: the first frame has opcode Text or Binary with FIN = 0, middle frames use opcode Continuation, and the last frame sets FIN = 1. Note that WebSocket gives you *messages*, not a byte stream: the receiver's API delivers whole messages, which is one of its key conveniences over raw TCP.

Mathematical foundations

FORMULA

WebSocket framing overhead per frame

$$H = 2 + L_{\text{ext}} + 4m, \qquad L_{\text{ext}} = \begin{cases} 0 & n \le 125 \\ 2 & 126 \le n \le 65,535 \\ 8 & n \ge 65,536 \end{cases}$$

Symbol	Meaning	Units
H	Header bytes per frame	bytes
n	Payload length	bytes
L_{ext}	Extended-length bytes	bytes
m	1 if masked (client→server), 0 otherwise	dimensionless

WORKED EXAMPLE

Worked example — Sending a 60-byte chat message.

1. Server→client: $n = 60 \le 125$, unmasked → $H = 2$ bytes. Total 62 bytes, 3 % overhead.
2. Client→server: masked → $H = 2 + 4 = 6$ bytes. Total 66 bytes.
3. The same message as an HTTP/1.1 long-poll response would carry roughly 200–800 bytes of headers — 3× to 13× the payload.
4. TLS 1.3 with AES-GCM adds about 22 bytes per record (5-byte record header, 1-byte inner content type, 16-byte authentication tag), and TCP/IP adds 40+ bytes per packet, so for tiny messages the transport, not WebSocket, dominates.

Engineering implication. For high-frequency small messages (cursor positions, game state, typing indicators) WebSocket framing is close to free, which is why chat and multiplayer systems prefer it. To save more, batch several small updates into one frame every 50–100 ms.

Code example

The following encoder/decoder is runnable with the standard library and reproduces the byte layout from Table 9.5.

```
        import os
        import struct

        def encode_frame(payload: bytes, opcode: int = 0x1, mask: bool = True) -> bytes:
            """Build a single, unfragmented frame (FIN=1)."""
            first = 0x80 | opcode                       # FIN bit + opcode
            n = len(payload)
            mask_bit = 0x80 if mask else 0x00
            if n <= 125:
                header = struct.pack("!BB", first, mask_bit | n)
            elif n <= 0xFFFF:
                header = struct.pack("!BBH", first, mask_bit | 126, n)
            else:
                header = struct.pack("!BBQ", first, mask_bit | 127, n)
            if not mask:
                return header + payload
            key = os.urandom(4)                         # fresh masking key per frame
            masked = bytes(b ^ key[i % 4] for i, b in enumerate(payload))
            return header + key + masked

        def decode_frame(frame: bytes):
            """Parse one complete frame; return (fin, opcode, payload)."""
            b0, b1 = frame[0], frame[1]
            fin, opcode = bool(b0 & 0x80), b0 & 0x0F
            masked, n = bool(b1 & 0x80), b1 & 0x7F
            pos = 2
            if n == 126:
                (n,) = struct.unpack("!H", frame[pos:pos + 2]); pos += 2
            elif n == 127:
                (n,) = struct.unpack("!Q", frame[pos:pos + 8]); pos += 8
            key = b""
            if masked:
                key = frame[pos:pos + 4]; pos += 4
            data = frame[pos:pos + n]
            if masked:
                data = bytes(b ^ key[i % 4] for i, b in enumerate(data))
            return fin, opcode, data

        print(encode_frame(b"Hello", mask=False).hex(" "))  # 81 05 48 65 6c 6c 6f
        f = encode_frame(b"Hello")                          # 11 bytes: 2 header + 4
key + 5
        print(len(f), decode_frame(f))                      # 11 (True, 1, b'Hello')
```

What to notice:

- `0x81` = FIN (0x80) + Text opcode (0x1); `0x05` = unmasked, length 5. This is the unmasked “Hello” example given in RFC 6455 §5.7.
- A masked frame is 4 bytes longer, and the masked payload bytes look random on the wire even though the plaintext is “Hello”.
- A 70,000-byte binary frame uses the 8-byte extended length: 2 + 8 + 4 = 14 header bytes (the maximum).

- Production libraries also validate UTF-8 in text frames, enforce maximum message sizes, reassemble fragments and reject unmasked client frames — never hand-roll this in production.

9.6.4 Masking: why the client scrambles its own data

Every client-to-server frame is XORed with a fresh, unpredictable 4-byte **masking key** chosen by the browser. Masking is *not* encryption — the key travels in the frame header in clear. It exists to protect **intermediaries**. Researchers showed that if a malicious script could choose the exact bytes a browser puts on the wire, it could craft WebSocket payloads that a confused transparent proxy would parse as a separate HTTP request, poisoning that proxy's cache for other users [Huang et al. 2011]. Because the script cannot predict the masking key, it cannot control the bytes that appear on the wire. RFC 6455 §10.3 explains this rationale. Server-to-client frames are not masked because the threat is script-controlled *upstream* bytes. Servers must close the connection (code 1002) if a client sends an unmasked frame.

9.6.5 Heartbeats (ping/pong) and the closing handshake

A TCP connection can die silently: a phone moves into a tunnel, a NAT device forgets its mapping after a few minutes of idleness, a load balancer reaps an idle flow. Neither endpoint is told. **Heartbeats** solve two problems at once: they *detect* dead peers (no pong within a deadline ⇒ close and clean up), and they *prevent* idle-timeout disconnections by keeping traffic flowing.

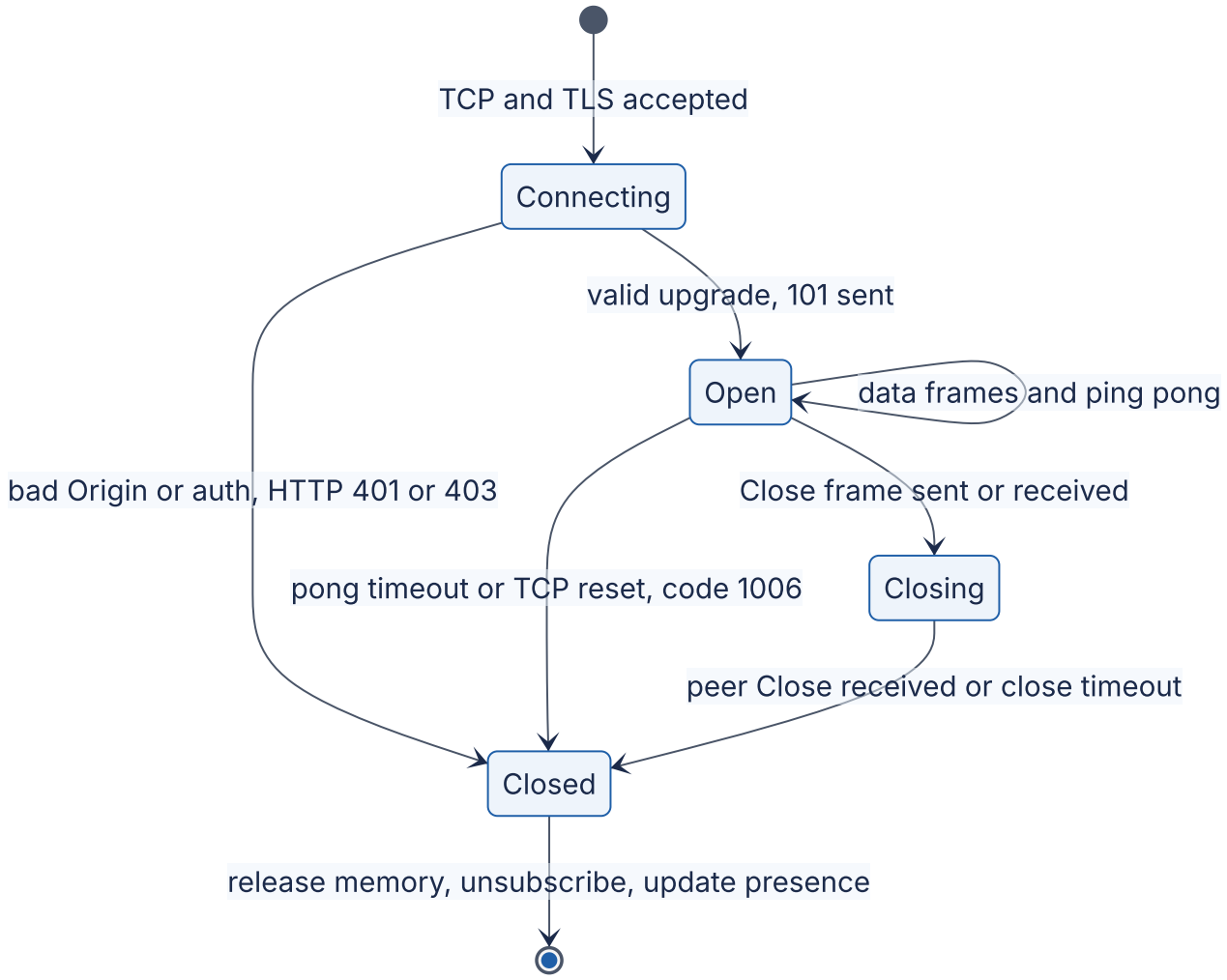
- The server sends a Ping every H seconds (commonly 20–30 s) and closes the connection if no Pong arrives within a timeout. Browsers answer Pings automatically.
- The browser `WebSocket` API does **not** let JavaScript send Pings, so client-side liveness detection needs an application-level heartbeat (for example, a `{"type":"ping"}` text message every 25 s and a "reconnect if nothing received for 60 s" rule).
- H must be smaller than the smallest idle timeout on the path (NAT, mobile carrier, load balancer).

Closing. Either side sends a Close frame with a status code; the other replies with a Close frame; then the TCP connection closes, ideally server first. The codes matter operationally because clients decide whether and how fast to reconnect based on them.

Table 9.7 — WebSocket close codes you will meet in practice.

Code	Name	Meaning and typical client reaction
1000	Normal closure	Purpose fulfilled; do not reconnect automatically
1001	Going away	Server shutting down or browser navigating away; reconnect with jitter
1002	Protocol error	Malformed frame; reconnect, report bug
1003	Unsupported data	For example binary sent to a text-only endpoint
1005	No status received	Reserved: reported locally when Close had no code; never sent
1006	Abnormal closure	Reserved: reported locally when TCP dropped without a Close frame; reconnect with backoff
1007	Invalid payload data	For example invalid UTF-8 in a text frame
1008	Policy violation	Generic rejection (auth failure, slow consumer); reconnect only after fixing cause
1009	Message too big	Exceeded server's maximum message size
1011	Internal error	Server-side failure; reconnect with backoff
1012	Service restart	IANA-registered (not in RFC 6455 text); reconnect with jitter
1013	Try again later	IANA-registered; overload — reconnect with longer backoff
4000–4999	Private use	Application-defined, for example 4001 "token expired"

Figure 9.5 — Lifecycle of a WebSocket connection as seen by a gateway.



Reading the diagram. Authentication and `Origin` checks belong in *Connecting*, before `101` — rejecting there is cheapest. *Open* loops on data and heartbeat frames. The orderly exit passes through *Closing*; heartbeat failure or TCP reset jumps straight to *Closed*. The exit action from *Closed* — unsubscribe from the backplane, update presence (§9.9) — is where leaks and "ghost online" bugs live. A close timeout of a few seconds stops an unresponsive peer from holding resources.

9.6.6 Proxies and load balancers

Long-lived, upgraded connections stress infrastructure built for short HTTP exchanges:

- **Hop-by-hop headers.** Proxies strip `Upgrade` and `Connection` unless configured otherwise; NGINX needs `proxy_http_version 1.1;`, `proxy_set_header Upgrade $http_upgrade;` and `proxy_set_header Connection "upgrade";`. Managed L7 load balancers generally support WebSockets natively.
- **Idle timeouts.** The 60 s defaults from §9.4 apply; keep heartbeats more frequent than the smallest timeout.

- **Use `wss://`.** Old transparent proxies and carrier middleboxes mangle unrecognized traffic on port 80 but cannot see inside TLS [Grigorik 2013].
- **L4 vs L7.** An L4 load balancer passes TCP through and scales to millions of flows; an L7 proxy terminates TLS and HTTP (path routing, central certificates) but holds two sockets per connection, and its upstream connections from one IP to one gateway IP:port are limited by the ephemeral port range (Linux default 32768–60999, about 28,000 ports) — add upstream IPs or ports beyond that.
- **Balancing happens only at connect time.** A newly added gateway gets no load until clients reconnect, so least-connections beats round robin (§19.4); deploys need draining (§9.9.7).

9.6.7 Backpressure

Backpressure is a slow receiver pushing back on a fast sender (§22.10). TCP provides it at the transport layer: when a receiver stops reading, the sender's kernel buffer fills and `await send()` stops completing. The danger is the layer above: a gateway fanning a room message out to 10,000 connections must queue data for the one client on a congested mobile link. Unbounded, a few hundred slow clients can exhaust memory and kill every healthy connection on the gateway. Browsers have the mirror problem: `WebSocket.send()` never blocks; it just grows `bufferedAmount`. Policies for a full per-connection queue:

1. **Disconnect the slow consumer** (close 1008 or an application code) and let it reconnect and resync from durable storage — simplest and safest; protects the gateway.
2. **Drop or coalesce** — for "latest value wins" data (prices, cursor positions, presence), keep only the newest value per key.
3. **Degrade** — switch the client to a lower update rate or a summary feed.

The chat server in Exercise 9.2 (§9.14) implements policy 1 with a bounded `asyncio.Queue` per connection.

Real-world industry applications

INDUSTRY EXAMPLE

Slack's real-time messaging. Slack's engineering blog describes clients holding WebSocket connections to stateful **Gateway Servers** (user information and channel subscriptions in memory), which subscribe to **Channel Servers** that own channels via consistent hashing; separate Presence Servers track who is online. A message goes to the owning Channel Server, which pushes it to every subscribed Gateway Server, which writes it to local sockets. A separate post describes migrating millions of concurrent WebSockets at the edge from HAProxy to Envoy, motivated largely by operational problems such as configuration reloads. *Evidence: Primary* — [Slack Engineering 2023]; [Slack Engineering 2021].

Advantages and disadvantages

- **Advantages:** full duplex with minimal per-message overhead (2–14 bytes); binary and text messages; message boundaries preserved; universal browser support; passes through ports 80/443.
- **Disadvantages:** stateful connections complicate load balancing, autoscaling and deploys; you must build heartbeats, reconnection, resumption, ordering and back-pressure yourself (none are in the protocol); bypasses HTTP-level caching, compression defaults and much of the HTTP observability tooling; security is easy to get wrong (Origin checks, message-size limits, authentication refresh on long sessions).

Failure scenarios

Table 9.8 — WebSocket failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Handshake fails with 400/426 behind proxy	Proxy strips `Upgrade` / `Connection`	101 rate ≈ 0 for one route	Fix proxy config	Configure WebSocket routes explicitly; integration test through the real proxy
"Ghost" connections	TCP peer vanished; no FIN received	Connection gauge grows, traffic per connection falls	Close on pong timeout	Server pings every 20–30 s with timeout
Gateway OOM	Unbounded per-connection send queues to slow clients	Memory rises with fan-out volume	Kill slow consumers	Bounded queues + slow-consumer policy
Reconnect storm after deploy	All connections on a node closed at once; clients retry immediately	Spike of handshakes and TLS CPU	Shed with 1013	Gradual drain + jittered backoff (§9.9.6–9.9.7)

Common misconceptions

COMMON MISTAKE

"TCP keepalive makes WebSocket heartbeats unnecessary." Linux TCP keepalive defaults to the first probe after 2 hours of idleness — far longer than NAT and load-balancer idle timeouts — and it is not visible to the application. Use WebSocket Ping/Pong (server side) and an application heartbeat (browser side).

9.7 WebRTC basics

9.7.1 What WebRTC is

DEFINITION

Definition — WebRTC. A set of IETF protocols (overview in RFC 8825 [Alvestrand 2021]) and a W3C browser API [W3C WebRTC] for real-time audio, video and arbitrary data directly between peers. Media travels over SRTP, data over SCTP data channels, both secured by DTLS, typically over UDP; peers discover a working network path using ICE with the help of STUN and TURN servers.

First-principles explanation

Everything so far routes through your servers. For a video call that is wasteful: two people in one city would send video to a data center and back. WebRTC lets browsers talk directly — but most devices sit behind **NAT** (§7.13), which drops unsolicited inbound packets. Think of two employees in different buildings whose receptionists only admit callers they have already phoned. They need a mutual friend to pass numbers (**signaling**), a way to learn the number the outside world sees (**STUN**), a trick of both calling at once so each receptionist expects the call (**ICE connectivity checks**, or hole punching), and as a last resort a switchboard that relays the call (**TURN**).

Why does this exist?

Conversation needs mouth-to-ear latency below roughly 150–200 ms and tolerates a little loss better than delay. TCP-based techniques retransmit lost packets and suffer head-of-line blocking — the wrong trade for live media. WebRTC gives browsers encrypted, congestion-controlled UDP media and data channels with NAT traversal built in.

How it works internally

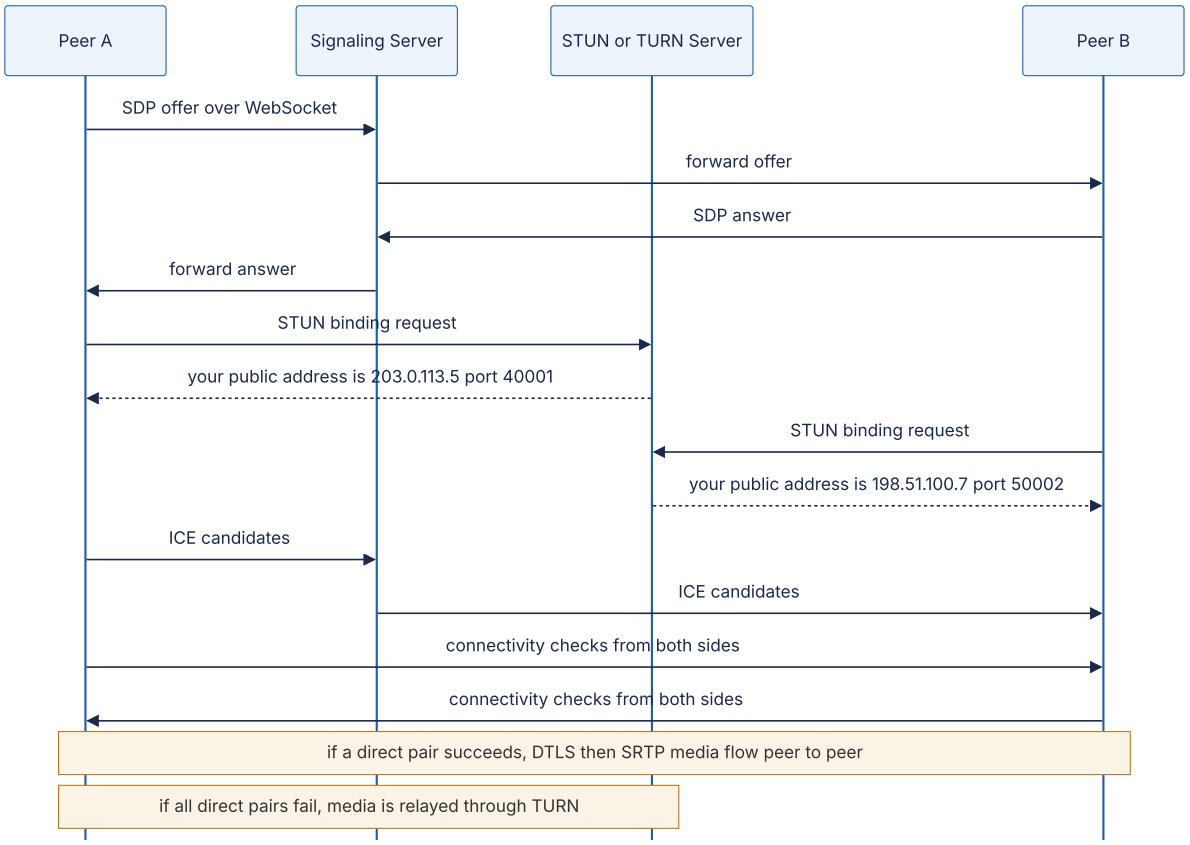
1. **Signaling (not specified by WebRTC).** Peers exchange session descriptions — **SDP offer** and **answer** listing codecs, encryption fingerprints and media streams —

through any channel you choose, commonly a WebSocket to your signaling server. The browser API that produces offers and answers is defined by JSEP [RFC 8829].

2. **Candidate gathering.** Each peer's ICE agent collects **candidates** (address:port pairs): *host* candidates (local interface addresses), *server-reflexive* candidates (the public address:port a **STUN** server reports seeing the request come from [RFC 8489]) and *relayed* candidates (an address allocated on a **TURN** relay server [RFC 8656]). Candidates are trickled to the other peer through signaling as they are found.
3. **Connectivity checks.** ICE [RFC 8445] pairs local and remote candidates, sends STUN binding requests on each pair simultaneously from both sides (punching holes through both NATs), and nominates the best pair that works, preferring direct paths over relayed ones.
4. **Security.** A DTLS handshake on the chosen path authenticates peers using the certificate fingerprints exchanged in SDP and derives keys for **SRTP** (media). Data channels run SCTP over DTLS. Encryption is mandatory.
5. **Media flow and adaptation.** Media flows peer to peer (or via TURN); congestion control and the codec adapt bitrate continuously; RTCP feedback reports loss and jitter.

When NATs allocate a new external port per destination (often called symmetric NAT) or firewalls block UDP, direct paths fail and traffic relays through TURN (over UDP, TCP or TLS on 443). TURN carries every media byte, so it is the most expensive component of a WebRTC deployment.

Figure 9.6 — WebRTC session establishment: signaling, STUN, ICE and TURN fallback.



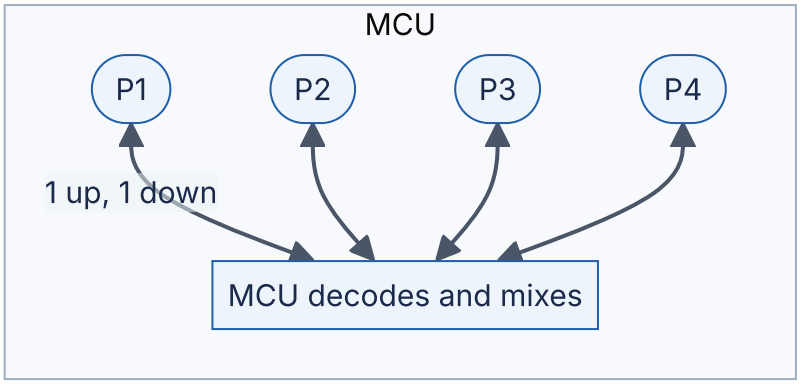
Reading the diagram. The signaling server is *your* service (often a WebSocket gateway) and relays only offers, answers and candidates over TCP — never media. The STUN server answers "what address do you appear to come from?" with one small UDP exchange. The peers then run simultaneous connectivity checks; if any direct pair works, encrypted media flows peer to peer over UDP and your infrastructure carries nothing. Otherwise TURN relays every media packet — the fallback path and the main cost driver, best deployed in several regions close to users.

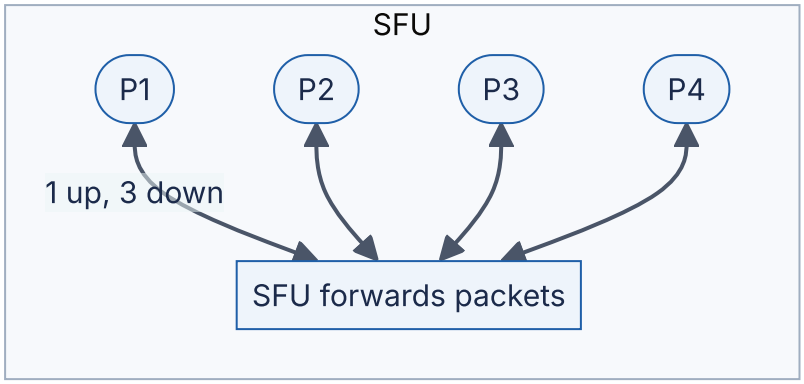
9.7.2 Group calls: mesh, SFU and MCU

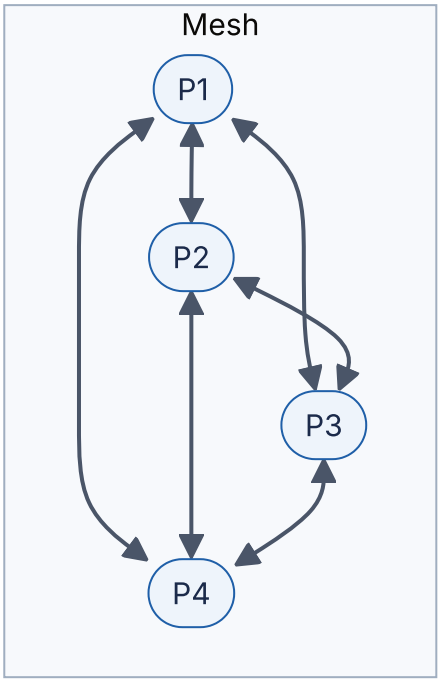
Peer-to-peer works for two people. For N participants there are three topologies:

- **Mesh** — every participant sends its stream to every other participant directly.
- **SFU (Selective Forwarding Unit)** — every participant sends one stream (often several resolutions at once, called **simulcast**) to a server, which *forwards* packets to each receiver without decoding them, choosing a resolution per receiver.
- **MCU (Multipoint Control Unit)** — the server *decodes* all streams, mixes them into one composite (a grid video, a mixed audio track), re-encodes and sends one stream to each participant.

Figure 9.7 — Mesh, SFU and MCU topologies for a four-person call.







Reading the diagram. In the mesh, each participant uploads three copies of its video, so home upstream bandwidth is the bottleneck. With an SFU, each participant uploads once and downloads three streams; the server only copies packets (cheap CPU, high bandwidth). With an MCU, each participant uploads and downloads one stream, but the server

decodes, mixes and re-encodes everything (expensive CPU, added latency, no end-to-end encryption). All arrows are asynchronous UDP media; signaling (not shown) runs through your WebSocket service.

FORMULA

Per-participant bandwidth by topology

Topology	Upload per participant	Download per participant	Server work
Mesh	$(N-1)\,b$	$(N-1)\,b$	none
SFU	b (or $\approx 1.5b$–$2b$ with simulcast)	$(N-1)\,b$ (less if receivers get lower layers)	forward $N(N-1)$ streams
MCU	b	b	decode N, mix, encode up to N

Symbol	Meaning	Units
N	Participants in the call	participants
b	Bitrate of one participant's stream	Mbit/s

WORKED EXAMPLE

Worked example — Six-person video call at 1.5 Mbit/s per stream.

1. Mesh upload per participant: $(6-1)\times 1.5 = 7.5$ Mbit/s. Many residential connections upload less than this, so the call degrades — mesh is practical for about 2–4 participants.
2. SFU upload: 1.5 Mbit/s (say 2.5 Mbit/s with three simulcast layers). Download: $5\times 1.5 = 7.5$ Mbit/s at full quality; with the SFU sending low layers for thumbnails and full quality only for the active speaker, perhaps $1.5 + 4\times 0.3 = 2.7$ Mbit/s.
3. SFU server egress for this one call: $6\times 7.5 = 45$ Mbit/s at full quality. A server with 10 Gbit/s of egress at 70 % target utilization carries about $7,000/45 \approx 155$ such calls — bandwidth, not CPU, is the SFU's limit.
4. MCU: each participant sends and receives 1.5 Mbit/s, but the server must decode six and encode up to six video streams per call — CPU-bound, typically far fewer calls per server than an SFU.

Engineering implication. Most modern group-video services use SFUs (often cascaded across regions) because they scale better per dollar and preserve per-receiver adaptation; MCUs survive for interop with legacy room systems and very weak clients.

Table 9.9 — Group-call topologies compared.

Aspect	Mesh	SFU	MCU
How it works	Every peer sends to every peer	Server forwards packets selectively	Server decodes, mixes, re-encodes
Advantages	No media servers; lowest latency for 2–3 people	Scales to large calls; per-receiver quality; cheap CPU	Minimal client bandwidth and CPU
Disadvantages	Upload grows with *N*	Download grows with *N*; server bandwidth heavy	Very high server CPU; extra latency; breaks end-to-end encryption
Choose when	1:1 or ≤ 4 people	Default for group calls and webinars	Legacy endpoints, recording composites, very weak clients

Failure scenarios and misconceptions

Table 9.10 — WebRTC failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Call never connects for some users	Symmetric NAT or UDP-blocking firewall; no TURN configured	ICE state `failed`; failures cluster by network	Retry with TURN over TCP/TLS 443	Always deploy TURN with TCP/TLS fallback
TURN bill explodes	Many sessions relay media	Relay bandwidth per session metric	—	Regional TURN, short-lived credentials, monitor relay ratio
Media freezes on network change	Wi-Fi → cellular changes the address	ICE disconnected	ICE restart via signaling	Handle `iceconnectionstatechange`; trigger ICE restart
Open TURN relay abused	Static TURN credentials leaked	Unknown traffic on relays	Rotate credentials	Time-limited, HMAC-derived TURN credentials per session

COMMON MISTAKE

“WebRTC means I don’t need servers.” You always need a signaling service, STUN, and — for reliable connectivity — TURN; group calls need SFUs. WebRTC removes servers from the *media path* only in the best case.

9.8 Mobile push: APNs and FCM

9.8.1 What mobile push is

> DEFINITION
>
> **Definition — Mobile push notification.** A message sent by an application's server to a platform-operated push service — Apple Push Notification service (**APNs**) for Apple platforms, Firebase Cloud Messaging (**FCM**) for Android (and optionally iOS and web) — which delivers it, on a best-effort basis, over a single persistent connection that the operating system maintains to that service on behalf of all apps.

Browsers have an analogous, standardized mechanism, Web Push [Thomson et al. 2016], in which a service worker receives messages relayed by the browser vendor's push service; the design principles below apply to it as well.

First-principles explanation

If every app kept its own background WebSocket, heartbeats would wake the cellular radio constantly and drain the battery in hours. So mobile operating systems suspend background apps and keep **one** connection to the platform push service, multiplexing every app's notifications over it — one mailroom for the whole apartment building. You hand your letter to the mailroom; it decides when to deliver, may combine or discard letters, and never sends a receipt.

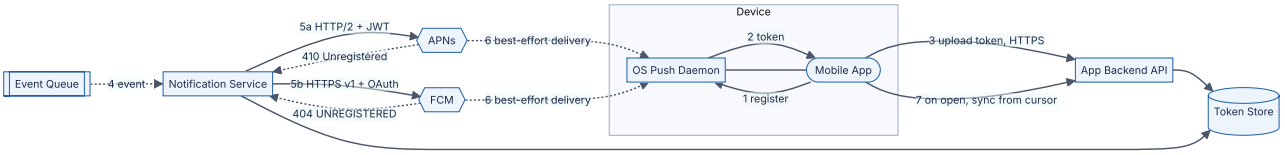
How it works internally

1. **Registration.** On launch, the app asks the OS for a push **token** (APNs device token, FCM registration token). The token identifies *this app on this device* to the push service.
2. **Token upload.** The app sends the token, with the user ID and a timestamp, to your backend, which stores it (a user typically has several tokens: phone, tablet, reinstalls).
3. **Send.** When an event occurs, your **notification service** (§52.4 designs one) looks up the user's tokens and calls the provider:
   - **APNs:** an HTTP/2 `POST /3/device/<token>` to `api.push.apple.com` (or `api.sandbox.push.apple.com` for development), authenticated by a JWT signed with your `.p8` key (token-based) or by a TLS client certificate. Headers include `apns-topic` (the app bundle ID), `apns-push-type` (`alert`, `background`, `voip`, …), `apns-priority` (10 = send immediately, 5 = send at a power-efficient time), `apns-expiration` and `apns-collapse-id` [Apple: Sending notification requests to APNs].

- **FCM:** an HTTPS `POST` `https://fcm.googleapis.com/v1/projects/<project>/messages:send` (the HTTP v1 API) with an OAuth 2.0 access token from a service account. The JSON body names the token (or a topic) and carries `notification` and/or `data` payloads plus platform options such as `android.collapse_key`, `android.ttl` and `android.priority` [FCM Docs].

4. **Store and forward.** If the device is offline, the provider stores the message for a limited time and delivers it when the device reconnects — with strict limits (below).
5. **Feedback.** The provider's response tells you whether the token is still valid. APNs returns HTTP 410 (`Unregistered`) for a token that is no longer active [Apple: Handling notification responses from APNs]; FCM returns `UNREGISTERED` (HTTP 404) or `INVALID_ARGUMENT` (HTTP 400). Delete or deactivate such tokens.

Figure 9.8 — Mobile push architecture.



Reading the diagram. The OS push daemon owns the device's single connection to the provider. Steps 1–3 obtain and upload the token over HTTPS into your token store. Events arrive asynchronously (dotted) at the notification service, which calls APNs or FCM synchronously. Provider-to-device delivery is asynchronous and best-effort — the dotted arrows mark where your guarantees end. Error responses (410/404) let the service prune dead tokens. Step 7 is the most important arrow: on open, the app syncs authoritative state with a cursor. Failure points: throttling, expired tokens, offline devices, OS suppression (Focus, battery saver). The notification service is a stateless worker pool scaled by send rate.

9.8.2 Delivery is not guaranteed — design for it

Push is a **hint**, never the source of truth:

- **Limited storage while offline.** APNs stores only the most recent notification per app for an offline device; sending a new one discards the stored one [Apple: Sending notification requests to APNs]. FCM stores up to 100 non-collapsible messages per device; past that it discards all of them and tells the app (on Android via a "deleted messages" callback) that it must sync with your server [FCM Docs: collapsible messages].
- **Expiry.** Messages older than their TTL (`apns-expiration`; FCM `ttl`, default and maximum 4 weeks) are dropped.
- **OS policy.** Low-priority and background ("silent") pushes may be delayed, batched or throttled by the OS to save power; Apple's guidance is to treat background notifi-

cations as low priority and to avoid sending more than a few per hour (as of 2024). Users can disable notifications entirely.
- **No delivery receipt.** A `200` from APNs or FCM means "accepted", not "displayed".

The robust pattern is **tickle, then fetch**: the push says "something changed" (optionally with display text), and the app, when opened or woken, fetches the authoritative state from your API since its last cursor — the same cursor idea as §9.4.

Collapse keys

When notifications about the same thing pile up — "3 new messages", "4 new messages" — only the latest matters. A **collapse key** tells the provider that a newer message *replaces* an older pending one. APNs uses the `apns-collapse-id` header (at most 64 bytes), and notifications sharing an ID are shown as one. FCM uses `collapse_key` and keeps at most **four** distinct collapse keys per device, with no guarantee which four survive [FCM Docs: collapsible messages]. Use collapse keys for state ("order status: shipped"), not for messages that each carry unique content.

Token lifecycle

Tokens change when users reinstall, restore a backup to a new device, clear app data, or when the platform rotates them. Rules of thumb (as of 2024–2025):

1. Fetch the token on every app start and upload it if changed, with a `last_seen` timestamp.
2. Delete tokens on `410` / `404 UNREGISTERED` responses immediately.
3. Expire tokens not refreshed for a period you choose; Google recommends treating tokens older than about one month as stale, and FCM itself treats Android tokens inactive for 270 days as expired [FCM Docs: token management].
4. Keep one row per (user, device, app) so that a user's ten old reinstall tokens do not each receive a copy.
5. Never put sensitive content in the payload when the lock screen could show it; payloads are limited to 4 KB (4,096 bytes) for regular notifications on both APNs and FCM.

Mathematical foundations

WORKED EXAMPLE

Worked example — Breaking-news push to 50 million devices.

Assume 50 million registered tokens, of which 8 % are stale (uninstalled apps), and you want delivery started within 5 minutes.

1. Required send rate: $50,000,000/300\ \text{s} \approx 167,000$ requests/s to APNs and FCM combined.
2. With HTTP/2 multiplexing, assume one worker connection sustains about 1,000 sends/s (an Illustrative figure; measure your own): $167,000/1,000 = 167$ concurrent provider connections across the worker fleet.
3. Wasted sends to stale tokens: $0.08 \times 50\ \text{M} = 4$ million requests, each returning 410/404 — prune them so that the next campaign is 8 % cheaper.
4. Return traffic: if 10 % of recipients open the app in the first minute and each open triggers a sync, your API sees $5,000,000/60 \approx 83,000$ extra requests/s. Stagger large sends, or put enough content in the payload that opening needs no immediate fetch.

Engineering implication. The bottleneck of a mass push is often *your own API* absorbing the return traffic, not the push provider.

Table 9.11 — APNs and FCM compared (as of 2024–2025).

Aspect	APNs	FCM
Platforms	iOS, iPadOS, macOS, watchOS, tvOS, Safari web push	Android; also iOS (via APNs) and web
Server API	HTTP/2, `POST /3/device/<token>`	HTTPS HTTP v1 `messages:send`
Authentication	JWT from `.p8` key, or TLS client certificate	OAuth 2.0 service-account access token
Max payload	4 KB (VoIP: 5 KB)	4 KB
Offline storage	Latest notification per app only	Up to 100 non-collapsible; 4 collapse keys
Collapse mechanism	`apns-collapse-id` (≤ 64 bytes)	`collapse_key`
Invalid-token signal	HTTP 410 `Unregistered`	`UNREGISTERED` (404)
Fan-out helpers	None (one request per token)	Topics and device groups

COMMON MISTAKE

"A 200 OK from APNs means the user saw the notification." It means APNs accepted the request. The device may be offline beyond the expiration, the notification may be replaced by a newer one, or the user may have disabled alerts. Track engagement through the app itself (for example, an "opened from notification" event), not through provider responses.

9.9 Connection management at scale

Sections 9.4–9.6 each ended with the same observation: once a server holds long-lived connections, it becomes *stateful*, and the problems shift from "how many requests per second?" to "how many connections, where are they, and what happens when they all move at once?". This section assembles the pieces into a production architecture.

9.9.1 Connection gateways

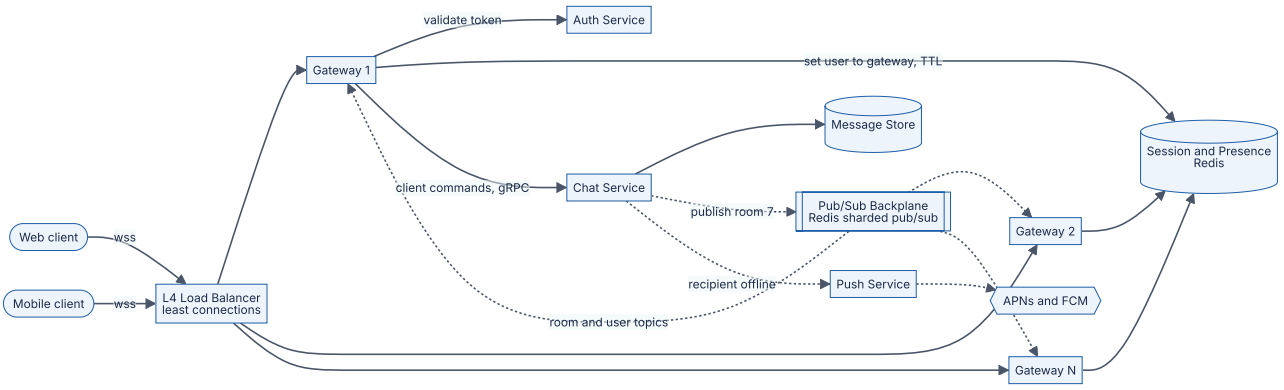
DEFINITION

Definition — Connection gateway. A dedicated, thin service tier whose only job is to terminate long-lived client connections (WebSocket, SSE, long polling), authenticate them, track which user and topics each connection belongs to, and move messages between those connections and back-end services through a messaging layer. It contains no business logic.

Why does this exist?

Put chat logic, feed ranking and the WebSocket server in one service, and every deploy of chat logic disconnects every user, every memory leak kills millions of connections, and CPU-based autoscaling adds instances that receive no connections until clients reconnect. Separating a **gateway tier** (stateful, memory-bound, simple, rarely deployed) from **business services** (stateless, CPU-bound, deployed many times a day) is the central design move of large real-time systems — an airport terminal that airlines can reschedule around without rebuilding it.

Figure 9.9 — Architecture of a connection gateway fleet.



Reading the diagram. Clients connect over `wss` through an L4 load balancer (least connections) and stay on the gateway they reach. On connect, the gateway validates the token and records "user → gateway" with a TTL in a Redis registry. Gateways subscribe to backplane topics for rooms with local members. A client message goes synchronously (gRPC) to the chat service, which persists it in the message store — the durable boundary — and *then* publishes it asynchronously (dotted); the backplane delivers only to subscribed gateways, which write to local sockets. Offline recipients go to the push service (§9.8). Failure points: a gateway crash drops its connections (clients reconnect and resync from the store); a backplane outage pauses live fan-out but loses no data; a registry outage breaks presence and targeted delivery. Gateways scale by connections, the chat service by request rate, the backplane by fan-out.

9.9.2 Memory per connection and capacity estimation

An idle connection consumes almost no CPU — on an event-loop server (§6.13), it is a file descriptor registered with `epoll` — so gateways are usually **memory-bound** and **blast-radius-bound** rather than CPU-bound. The C10K problem of the late 1990s [Kegel 1999] asked how to hold 10,000 connections on one server; modern kernels and runtimes routinely hold hundreds of thousands to millions.

Table 9.12 — Per-connection memory budget for a TLS WebSocket gateway (Illustrative assumptions; measure your own stack).

Component	Typical range	Assumed
Kernel socket structures, idle (buffers grow only while data is in flight)	2–10 KB	4 KB
TLS session state and record buffers (depends heavily on library and buffer-release settings)	5–40 KB	16 KB
Runtime task (goroutine stack, Erlang process, async task)	2–8 KB	4 KB
Application read/write buffers	4–16 KB	8 KB
Session state: user, auth claims, subscriptions, outbound queue	1–20 KB	8 KB
Total	**14–94 KB**	**40 KB**

Note the caveat in the first row: Linux autotunes socket buffers up to `tcp_rmem` / `tcp_wmem` maximums of several megabytes per socket. A burst of traffic to many slow clients can multiply kernel memory far above the idle figure; cap buffer maximums with `sysctl` and bound application queues (§9.6.7).

FORMULA

Gateway fleet sizing

$$n_{\text{conn,max}} = \min\left(\frac{f \cdot M}{m},\ c_{\text{blast}}\right) \qquad S = Z \cdot \left\lceil \frac{C}{(Z-1)\, n_{\text{conn,max}}} \right\rceil$$

Symbol	Meaning	Units
M	RAM per gateway server	bytes
f	Fraction of RAM budgeted for connections	dimensionless
m	Memory per connection	bytes/connection
c_{blast}	Maximum connections you accept losing in one server failure	connections
$n_{\text{conn,max}}$	Design ceiling of connections per server	connections
C	Peak concurrent connections	connections
Z	Availability zones; the fleet must survive loss of one	zones
S	Total gateway servers	servers

WORKED EXAMPLE

Worked example — 10 million concurrent connections.

Requirements: $C = 10,000,000$ concurrent WebSocket connections at peak; three availability zones ($Z = 3$); survive the loss of one zone; per-connection memory $m = 40$ KB from Table 9.12 (SI units: 1 KB = 1,000 bytes); servers with $M = 64$ GB RAM, of which $f = 50\,\%$ is budgeted for connections (the rest for the OS, runtime heap, garbage-collection headroom and traffic bursts).

1. **Memory limit per server:** 0.5×64 GB/40 KB $= 32 \times 10^9 / 40 \times 10^3 = 800,000$ connections.
2. **Blast-radius limit:** losing one server should disconnect at most about 5 % of users and produce a reconnect wave the rest of the fleet can absorb. Choose $c_{\text{blast}} = 500,000$. So $n_{\text{conn,max}} = \min(800,000, 500,000) = 500,000$.
3. **Zone-failure sizing:** after losing one zone, the remaining two must hold all 10 M: servers per zone $= \lceil 10,000,000/(2 \times 500,000) \rceil = 10$. Total $S = 3 \times 10 = 30$.
4. **Deploy and surge headroom:** during a rolling deploy one server per zone is draining, and traffic may exceed forecast by 10–20 %. Add two servers per zone: **36 servers (12 per zone)**.
5. **Normal load per server:** $10,000,000/36 \approx 278,000$ connections → $278,000 \times 40$ KB ≈ 11.1 GB of connection memory.
6. **After losing a zone:** $10,000,000/24 \approx 417,000$ connections per server → 16.7 GB — inside the 32 GB budget.
7. **Total connection memory:** $10,000,000 \times 40$ KB $= 400$ GB across the fleet.
8. **File descriptors:** each process needs more than 500,000 open files; set the `nofile` limit to about 1,000,000.
9. **Heartbeats:** with a 25 s ping interval, $10,000,000/25 = 400,000$ pings/s fleet-wide, about 11,000 per server — trivial CPU, but visible in packet-rate and NAT-table metrics.
10. **Message CPU:** if each connection receives on average one message every 10 s, a server at 278,000 connections sends about 27,800 frames/s plus 11,000 pongs/s to process — well within one multi-core server (Illustrative; benchmark with your payload size and TLS library).

Interpretation. Memory alone would allow about 13 servers (10 M/800 k); fault tolerance and blast radius nearly triple that. That is typical: connection fleets are sized by failure behavior, not by steady-state efficiency.

Engineering implication. Measure m on your real stack with a load generator holding idle TLS connections (resident memory ÷ connections), re-measure under message load, and state the blast-radius limit explicitly in the design document.

Real-world industry applications

INDUSTRY EXAMPLE

WhatsApp's per-server connection counts. In January 2012 WhatsApp's blog reported pushing a single server past 2 million concurrent TCP connections, following an earlier 1 million milestone; a contemporaneous Erlang Factory talk by Rick Reed described the Erlang/FreeBSD stack and the kernel and VM tuning involved. These figures show what a tuned stack *can* hold; production fleets generally run below such peaks to keep headroom for failures and deploys. *Evidence: Primary* — [WhatsApp 2012]; [Reed 2012].

INDUSTRY EXAMPLE

Discord's Elixir gateway. Discord's engineering blog describes scaling its Elixir/Erlang real-time system to nearly 5 million concurrent users, including fan-out problems in very large communities and bursts of reconnecting sessions overwhelming a few coordinating processes, plus libraries the team built in response (for example, Manifold to distribute message sending across nodes). *Evidence: Primary* — [Discord 2017].

9.9.3 Routing messages: registries and the pub/sub backplane

A message for room 7 is created by a chat service instance; its recipients' connections are scattered across 36 gateways. Two routing designs exist, and large systems use both:

1. **Topic-based pub/sub backplane.** Each gateway subscribes to `room:7` when its first local member of room 7 connects and unsubscribes when the last leaves; publishers send to `room:7` without knowing which gateways care. Redis Pub/Sub is the usual starting point: simple and sub-millisecond, but **fire-and-forget (at-most-once)** — a gateway whose subscription blips misses messages. In Redis Cluster, classic `PUBLISH` is broadcast to every node; **sharded pub/sub** (`SPUBLISH`/`SSUBSCRIBE`, Redis 7.0+) keeps a channel on the shard owning its hash slot [Redis Docs: Pub/Sub]. Kafka or NATS fit when you need replay or more throughput.
2. **Connection registry with direct routing.** A registry maps `user → (gateway, connection ID)`. For one-to-one messages the sender looks up the user's gateway(s) and calls them directly (gRPC). This avoids creating millions of per-user topics but needs a registry that is kept accurate as connections churn.

Because the live path is lossy, correctness comes from the durable path: every message gets a per-conversation **sequence number** in the message store; clients track the last sequence they have seen; on reconnect, or when they notice a gap, they fetch the missing

range over HTTPS. Live delivery is an optimization of latency; the store guarantees completeness. This is the same at-least-once-plus-deduplication pattern as §30.8.

FORMULA

Fan-out load

$$D = \mu \times \bar{r}$$

Symbol	Meaning	Units
D	Message deliveries written to sockets	deliveries/second
μ	Messages published	messages/second
$\bar{r}$	Average number of *online* recipients per message	recipients/message

WORKED EXAMPLE

Worked example — Fan-out for a team chat product.

At peak, users send $\mu = 50,000$ messages/s; the average channel has 40 members, 25 % online, so $\bar{r} = 10$ and $D = 500,000$ deliveries/s — about 13,900 socket writes/s per gateway across 36 gateways. But one announcement channel with 200,000 online members turns a single message into 200,000 deliveries. Large channels need posting rate limits and delivery once per *gateway* (each gateway then fans out locally), so the backplane carries 36 copies rather than 200,000.

9.9.4 Sticky routing and session resumption

A WebSocket is sticky by construction for its lifetime. The question is what happens on *reconnect*:

- **Stateless reconnect (preferred).** Any gateway can accept any user; everything needed to resume — subscriptions, last-seen sequence numbers — is rebuilt from the client's handshake and the shared registry. Load balancing is then simple least-connections.
- **Sticky reconnect.** The load balancer returns a user to the same gateway (consistent hashing on user ID or an affinity cookie, §19.6) so a per-connection buffer can be resumed without a store query — cheaper in the common case, but it breaks whenever the fleet resizes and concentrates hot users.

Long-polling and SSE fallbacks *require* stickiness, or shared state, because consecutive HTTP requests of one logical session must find the same buffer (§9.4).

9.9.5 Presence with heartbeats

Presence — the green dot — answers "is this user online right now?". The naïve design writes `online` on connect and `offline` on disconnect; it fails because disconnections are often silent (§9.6.5) and gateways crash without cleaning up. Robust presence uses **leases**:

1. On connect and on each heartbeat (every h = 30 s), the gateway writes `presence:user42 = gateway-7` with a TTL of about $3h$ = 90 s.
2. If heartbeats stop, the key expires and the user becomes offline within at most 90 s — no explicit cleanup needed after a crash.
3. A user with several devices is online if any device's lease is alive.
4. Status changes (online → offline, active → idle) are published to interested watchers through the backplane.

To cut write load, gateways refresh *batched* leases (one pipeline per second per gateway) or refresh a per-gateway liveness key and record users per gateway, so a dead gateway expires all its users at once.

> WORKED EXAMPLE
>
> **Worked example — Presence load for 10 million online users.**
>
> 1. Lease refresh writes with $h = 30$ s: $10,000,000/30 \approx 333,000$ writes/s — a sharded Redis cluster is required (several shards at an assumed 100,000+ simple operations/s each), or the per-gateway lease design above, which needs only 36 writes/s plus membership changes.
> 2. Status-change fan-out: if each user changes status every 10 minutes on average and 50 online contacts watch each user, $10,000,000 \times (1/600) \times 50 \approx 833,000$ presence deliveries/s — more than the chat messages themselves.
>
> **Engineering implication.** Presence is often the most expensive real-time feature. Common mitigations: deliver presence only for users currently visible on the watcher's screen (subscribe on view), coalesce rapid flaps (offline-then-online within a few seconds), and do not show live presence for very large groups.

9.9.6 Reconnect storms and jittered backoff

When a gateway dies, a load balancer fails over or a region recovers, hundreds of thousands of clients lose their connections at once. If all reconnect immediately, surviving gateways face a spike of TLS handshakes, the auth service a spike of token checks, and the message store a spike of resync queries; they time out, clients retry, and the storm sustains itself — the **thundering herd**. The fix is exponential backoff with **jitter** (§22.12, [Brooker 2015]).

FORMULA

Reconnect arrival rate with uniform jitter

$$\lambda_{\text{reconnect}} \approx \frac{n_{\text{lost}}}{W}$$

Symbol	Meaning	Units
$\lambda_{\text{reconnect}}$	Peak reconnect (handshake) rate	connections/second
n_{lost}	Connections dropped by the failure	connections
W	Width of the random delay window for the first reconnect attempt	seconds

WORKED EXAMPLE

Worked example — One gateway of 36 fails.

1. $n_{\text{lost}} = 278,000$ connections (from §9.9.2).
2. No jitter, everyone retries within about 1 s: $\lambda \approx 278,000$ handshakes/s, about 7,900 per surviving gateway, plus 278,000 auth checks/s and 278,000 resync queries/s — likely to overload auth and the store.
3. Uniform jitter over $W = 30$ s: $\lambda \approx 9,300$/s fleet-wide, about 265 per gateway — comfortably absorbed.
4. A simulation of 500,000 clients confirms the arithmetic: all in one second without jitter; a peak of about 16,900 per one-second bucket with a 30 s uniform window (expected $500,000/30 \approx 16,700$).

Engineering implication. Jitter turns a spike into a plateau. Add server-side protection too: a token-bucket admission limit on new handshakes per gateway, close code 1013 (“try again later”) with a server-suggested delay, and resumable sessions so a reconnect costs a delta fetch rather than a full resync.

```
        import random

        def full_jitter_delay(attempt: int, base: float = 1.0, cap: float = 60.0) ->
float:
            """'Full jitter': uniform in [0, min(cap, base * 2**attempt)] seconds."""
            return random.uniform(0, min(cap, base * (2 ** attempt)))

        # illustrative client loop
        # attempt = 0
        # while not connected:
        #     await asyncio.sleep(full_jitter_delay(attempt)); attempt += 1
        #     connected = await try_connect()      # reset attempt = 0 after a stable
minute
```

What to notice: clients that failed together spread out immediately and back off further if the outage persists; `cap` bounds the wait; reset the attempt counter only after a connection has stayed healthy for a while, or a server that accepts and immediately drops connections causes a tight retry loop.

9.9.7 Graceful drain on deploy

Deploying a gateway restarts the process that owns its connections. Doing it abruptly is a self-inflicted reconnect storm. A graceful **drain**:

1. **Stop new connections.** Fail the gateway's readiness check or deregister it from the load balancer; existing connections stay open.
2. **Close gradually.** Over a drain window D (for example, 5 minutes), close connections at a steady rate n/D — 278,000 / 300 s ≈ 930 per second — using close code 1012 (service restart) or 1001 (going away), or send an application "please reconnect" message with a randomized delay hint.
3. **Clients reconnect with jitter** and land on other gateways, because the load balancer no longer routes to this one.
4. **Clean up and exit.** Unsubscribe from the backplane, release presence leases, flush metrics, exit.
5. **Orchestrator settings.** Kubernetes gives a pod 30 seconds by default (`terminationGracePeriodSeconds`) between `SIGTERM` and `SIGKILL`; gateways need this raised to cover D, plus a `preStop` hook that starts the drain (§42.7).

Rolling 36 servers in batches of three (one per zone) with a 5-minute drain takes $12 \times 5 = 60$ minutes — another reason to keep business logic out of the gateway and deploy it rarely.

Failure scenarios

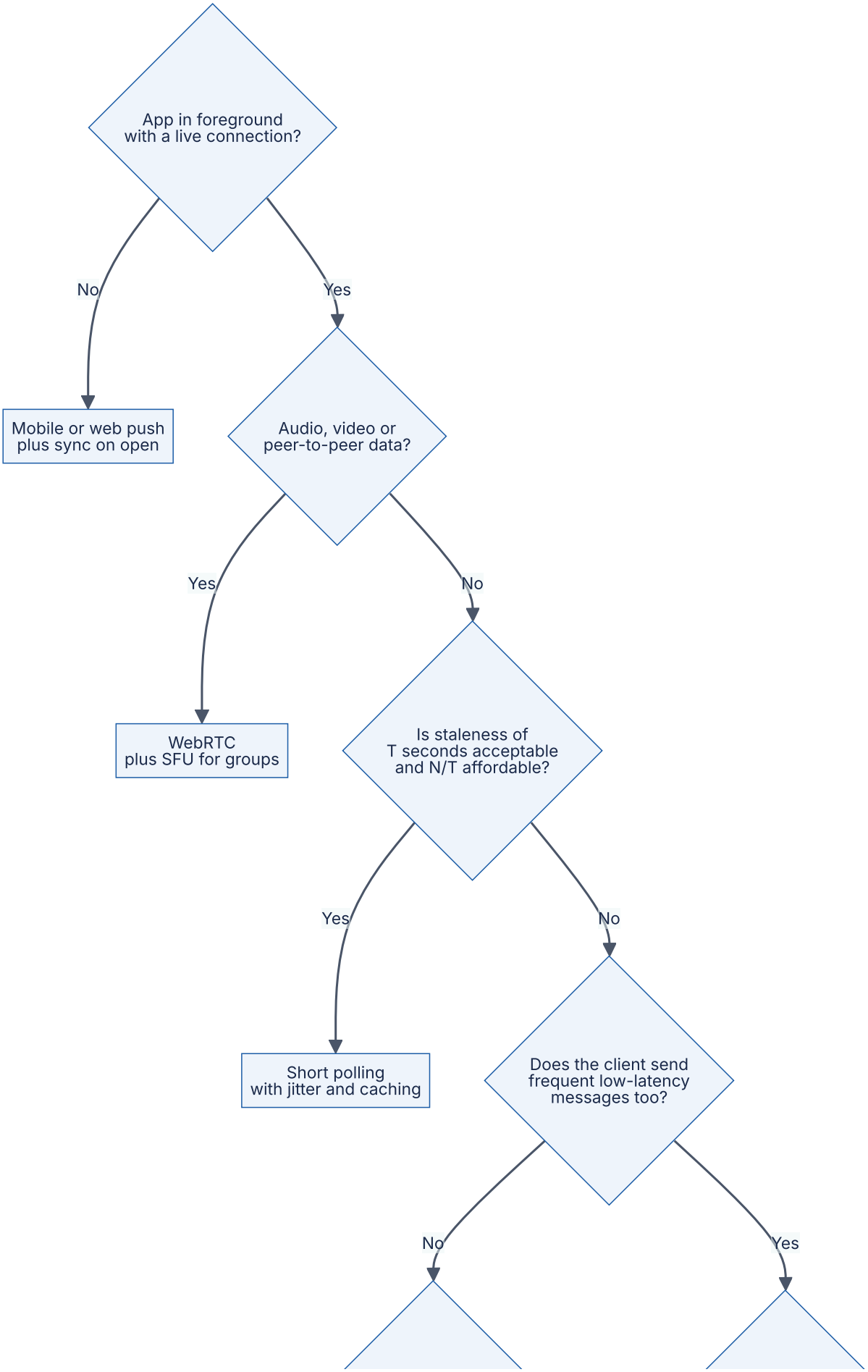
Table 9.13 — Connection-fleet failure scenarios.

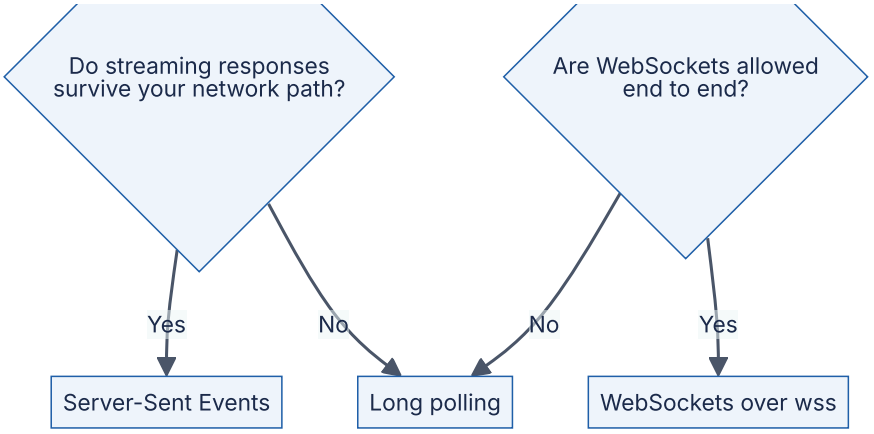
What goes wrong	Why	Detection	Recovery	Prevention
Reconnect storm takes down auth	Mass disconnect, synchronized retries	Handshake rate and auth latency spike together	Admission control, 1013 with delay	Jittered backoff; cached token validation at gateway
New gateways sit idle after scale-out	Connections only rebalance on reconnect	Connection-count skew across gateways	Shed a few % of connections from hot gateways	Least-connections LB; periodic gentle rebalancing
Messages lost during backplane blip	Redis Pub/Sub is at-most-once	Clients detect sequence gaps	Client fetches missing range	Sequence numbers + durable store as source of truth
Users shown online for hours	Presence written on connect, never expired	Online count exceeds connection count	Expire stale entries	Lease with TTL refreshed by heartbeat

9.10 Choosing a real-time technique

No technique is best; each answers a different question. Work through the requirements in this order: *Is the app even running? Which direction does data flow? How fresh must it be? Is it media? What does the network allow?*

Figure 9.10 — Decision flowchart for choosing a real-time technique.


App in foreground with a live connection?
No
Yes
Mobile or web push plus sync on open
Audio, video or peer-to-peer data?
Yes
No
WebRTC plus SFU for groups
Is staleness of T seconds acceptable and N/T affordable?
Yes
No
Short polling with jitter and caching
Does the client send frequent low-latency messages too?
No
Yes



Reading the diagram. Diamonds are requirement questions; rectangles are recommended techniques. Only platform push can reach a suspended mobile app, so that split comes first; media goes to WebRTC regardless. For ordinary data, the cheapest technique that meets the freshness requirement wins; long polling is the fallback for hostile networks. Real systems combine branches: a chat app uses WebSockets in the foreground, push in the background, HTTPS for history and WebRTC for calls.

Table 9.14 — Real-time techniques compared.

Aspect	Short polling	Long polling	SSE	WebSockets	WebRTC	Mobile push
What it is	Periodic HTTP requests	Held HTTP request	Streamed HTTP response	Upgraded full-duplex TCP	P2P UDP media/data	OS-level provider delivery
Direction	Client pulls	Server → client	Server → client	Both	Both, peer to peer	Server → device
Latency	≈ T/2	≈ 1 RTT	≈ ½ RTT	≈ ½ RTT	Lowest; UDP	Seconds to never
Overhead per message	Full HTTP request	Full HTTP request	~10–50 bytes	2–14 bytes	RTP/SRTP headers	Provider API call
Server state	None	Parked request	Open stream	Open connection	Signaling + TURN/SFU	Token store
Infrastructure friendliness	Excellent	Good (timeouts)	Good (buffering)	Needs Upgrade + timeouts	NAT traversal, TURN	Provider-dependent
Resumption	Cursor	Cursor	Built in (`Last-Event-ID`)	Build your own	ICE restart	Collapse + sync
Operational complexity	Low	Medium	Medium	High	Very high	Medium
Cost driver	*N*/*T* requests	Concurrent connections	Concurrent connections	Concurrent connections + fan-out	TURN/SFU bandwidth	Send volume + return traffic
Use cases	Dashboards, job status	Fallback transport	Feeds, notifications, LLM token streams	Chat, collaboration, games, trading	Calls, screen share, P2P files	Background alerts
Choose when	Rare updates, few clients	Streaming blocked	One-way, HTTP-native	Two-way, high frequency	Real-time media	App not running

RECOMMENDATION

Default choices for common products. Notifications badge on a website: SSE (or short polling at 30–60 s if traffic is small). Chat and collaborative editing: WebSockets in the foreground, mobile push in the background, HTTPS sync for history. Live sports scores or stock tickers for millions of viewers: SSE or WebSockets fed from a CDN or edge fan-out tier, with “latest value wins” coalescing. Video meetings: WebRTC with an SFU, WebSockets for signaling. Ride-hailing driver location to a rider: WebSockets or SSE while the app is open, push for “driver arrived” when it is not.

INTERVIEW TIP

In an interview, state the requirement that drives the choice (“updates are server-to-client only and must survive reconnects, so SSE with `Last-Event-ID`”), then immediately address the scale consequence: how many concurrent connections, how many gateways, how messages reach the right gateway, and what happens on reconnect. Interviewers care far more about the connection-management design (§9.9) than about the name of the protocol.

9.11 Summary and revision notes

SUMMARY

- HTTP is client-initiated; every technique in this chapter is a way for the server to speak first, trading freshness against connection cost and infrastructure complexity.
- **Short polling:** $R = N/T$; average delay $T/2$; with rare events nearly every request is wasted ($P_{\text{empty}} = e^{-\lambda T}$). Fine for few clients or slow-changing data; jitter the timers.
- **Long polling:** the server holds the request until data or a hold timeout (keep it below every intermediary's idle timeout, e.g. 25–30 s); cursors prevent loss in the gap between polls; cost moves from requests to concurrent connections. Comet (2006) and RFC 6202 document it.
- **SSE:** one streamed `text/event-stream` response; fields `data`, `event`, `id`, `retry`, comments for heartbeats; browsers reconnect and send `Last-Event-ID`; server-to-client only; text only; prefer HTTP/2 to escape the ~6 connections-per-origin limit; disable proxy buffering.
- **WebSockets (RFC 6455):** HTTP/1.1 `Upgrade` handshake with `Sec-WebSocket-Accept` = base64(SHA-1(key + GUID)); frames with 2–14 byte headers; opcodes text, binary, continuation, close, ping, pong; client frames masked to protect intermediaries; heartbeats below idle timeouts; close codes guide reconnection; check `Origin`; bound send queues and disconnect slow consumers.
- **WebRTC:** signaling (yours) + ICE with STUN (discover public address) and TURN (relay fallback) + DTLS-SRTP media; mesh for ≤ 4, SFU for most group calls, MCU for weak or legacy clients.
- **Mobile push:** one OS-managed connection to APNs/FCM; best effort; APNs keeps only the latest notification per app when offline, FCM up to 100 messages and 4 collapse keys; 4 KB payloads; prune tokens on 410/404; push is a tickle — sync from your API with a cursor.
- **At scale:** thin, rarely deployed connection gateways; size by memory *and* blast radius *and* zone failure (10 M connections × 40 KB ≈ 400 GB; 36 servers of 64 GB in the worked example); route through a pub/sub backplane or connection registry; make the durable store with sequence numbers the source of truth; presence as TTL leases refreshed by heartbeats; jittered reconnects ($\lambda \approx n/W$); drain gradually on deploy.

9.12 Knowledge checkpoint

1. A site has 400,000 tabs open, each polling every 4 s. What request rate reaches the servers?

2. Why does long polling need a cursor, and what goes wrong without one?
3. Why should a long-poll hold timeout be 25–30 s rather than 120 s?
4. Name the four SSE fields and the purpose of a line that starts with a colon.
5. What header does a browser send when an `EventSource` reconnects, and what must the server do with it?
6. Why does SSE over HTTP/1.1 cause problems for users with many tabs, and how does HTTP/2 change this?
7. Compute `Sec-WebSocket-Accept` conceptually: what inputs and which functions?
8. What are the header sizes of an unmasked 100-byte frame and a masked 1,000-byte frame?
9. Why are client-to-server WebSocket frames masked and server-to-client frames not?
10. What is the difference between close codes 1000, 1001, 1006 and 1008?
11. Why can't browser JavaScript rely on WebSocket Ping frames for its own liveness detection?
12. What role do STUN and TURN each play in ICE, and which one carries media?
13. In a 5-person SFU call at 2 Mbit/s per stream, what is each participant's upload and full-quality download?
14. What does APNs do with notifications sent while a device is offline? What does FCM do?
15. Using the formula in §9.9.2, how many gateways are needed for 3 M connections with a 300 k per-server ceiling across 3 zones that must survive one zone failure?
16. Why are presence systems built on TTL leases rather than explicit "offline" writes?

Answers

1. $R = 400,000/4 = 100,000$ requests/s.
2. After each response there is a gap of about one round trip with no outstanding request; events in that gap would be lost unless the next request asks for "everything after cursor c" from a server-side buffer. The cursor also lets the client discard duplicates.
3. Proxies and load balancers commonly close idle requests at 60 s (for example, ALB and NGINX defaults); a hold longer than the smallest idle timeout produces errors instead of clean empty responses.
4. `data`, `event`, `id`, `retry`. A line starting with `:` is a comment, ignored by the client and used as a heartbeat to keep intermediaries from timing out the stream.
5. `Last-Event-ID`, containing the last `id:` received; the server replays events after it from a buffer (or signals a full resync if the buffer no longer reaches back that far).
6. Each SSE stream occupies one of roughly six HTTP/1.1 connections per origin shared across all tabs, so further tabs' requests stall. Over HTTP/2 streams are multiplexed on one connection, limited only by the negotiated maximum concurrent streams (commonly 100+).
7. Concatenate the client's `Sec-WebSocket-Key` with the fixed GUID `258EAFA5-E914-47DA-95CA-C5AB0DC85B11`, take SHA-1, base64-encode the 20-byte digest.

8. Unmasked 100 bytes: 2-byte header. Masked 1,000 bytes: 2 + 2 (extended length) + 4 (key) = 8 bytes.
9. Masking with an unpredictable key prevents a malicious script from choosing the exact bytes on the wire and thereby confusing intermediaries (cache poisoning). The attacker controls only browser (upstream) data, so only client frames are masked.
10. 1000 normal closure (done; do not auto-reconnect); 1001 going away (server shutdown or navigation; reconnect with jitter); 1006 abnormal closure, never sent, reported locally when TCP died without a Close frame; 1008 policy violation (for example auth failure or slow consumer).
11. The browser `WebSocket` API exposes no method to send Pings or observe Pongs; browsers answer server Pings automatically. Client-side liveness requires an application-level heartbeat message.
12. STUN tells a peer the public address and port its NAT presents (server-reflexive candidate); TURN allocates a relay address and forwards traffic when no direct path works. Only TURN carries media.
13. Upload 2 Mbit/s (more with simulcast); download $(5-1)\times 2 = 8$ Mbit/s.
14. APNs stores only the most recent notification per app and discards older ones. FCM stores up to 100 non-collapsible messages (beyond that it drops them all and signals the app to sync) and at most four collapse keys; both drop messages past their expiry/TTL.
15. Per zone $\lceil 3,000,000/(2\times 300,000)\rceil = 5$; total $3\times 5 = 15$ servers, plus deploy/surge headroom (for example 18).
16. Disconnections are often silent and gateways can crash; a lease that must be refreshed by heartbeats expires automatically, so presence converges to correct without relying on cleanup code that may never run.

9.13 Interview questions

Basic

Q1. What is the difference between short polling, long polling and WebSockets?

Answer. Short polling sends a request every T seconds and gets an immediate (often empty) answer: simple, stateless, average delay $T/2$, cost N/T requests/s. Long polling sends a request that the server holds until data arrives or a timeout fires, then the client re-requests: near-instant delivery over plain HTTP, but a full HTTP exchange per message and one held connection per client. WebSockets upgrade one HTTP/1.1 connection into a persistent full-duplex channel with 2–14 bytes of framing per message: lowest overhead and bidirectional, but stateful, requiring heartbeats, reconnection logic and proxy/load-balancer configuration.

Q2. When would you choose Server-Sent Events over WebSockets?

Answer. When data flows only from server to client (notifications, feeds, live scores, LLM token streams) and you want to stay on plain HTTP. SSE gets HTTP/2 multiplexing, standard authentication, cookies and observability for free; browsers reconnect automatically and resume with `Last-Event-ID`. Choose WebSockets when the client also sends frequent low-latency messages or when you need binary frames.

Intermediate

Q3. Users report that WebSocket connections drop every 60 seconds in production but not locally. Diagnose.

Answer. Exactly-periodic drops point to an idle timeout on an intermediary — commonly a load balancer or reverse proxy with a 60 s default — closing connections that carried no traffic. Confirm by correlating close times with connection-open times and checking close codes (1006 on the client indicates TCP loss without a Close frame). Fix by sending server Pings (or application heartbeats) every 20–30 s, below the smallest idle timeout, and optionally raising the timeout on WebSocket routes. Also verify that the proxy forwards `Upgrade` / `Connection` correctly and that heartbeats are not suppressed by client backgrounding on mobile.

Q4. Design delivery of a chat message to recipients connected to different gateway servers.

Answer. The sender’s gateway forwards the message to a chat service, which assigns a per-conversation sequence number and persists it (source of truth). The service then publishes to the conversation’s topic on a pub/sub backplane (for example, Redis sharded pub/sub); every gateway with at least one local member is subscribed and writes the message to its local sockets. Recipients without a live connection get an APNs/FCM push with a collapse key. Because the backplane is at-most-once, clients track the last sequence number seen and fetch missing ranges over HTTPS on reconnect or gap detection; deduplicate by sequence number. Discuss large-room fan-out (deliver once per gateway, rate limits) and backpressure (bounded per-connection queues).

Advanced

Q5. Size a gateway fleet for 10 million concurrent connections and explain how you would deploy it without disrupting users.

Answer. Measure memory per idle TLS WebSocket (assume 40 KB) → 400 GB total. On 64 GB servers with 50 % budgeted, memory allows 800 k per server, but cap at a blast-radius limit such as 500 k. To survive losing one of three zones, each zone needs $\lceil 10\text{M}/(2\times 500\text{k})\rceil = 10$ servers → 30, plus deploy/surge headroom → 36 (≈ 278 k each, ≈ 11

GB). Raise file-descriptor limits to ~1 M. Heartbeats at 25 s → 400 k pings/s fleet-wide. For deploys: keep the gateway thin so it rarely changes; drain one server per zone at a time — remove from the load balancer, close connections gradually over ~5 minutes with code 1012, clients reconnect with full-jitter backoff, then exit; raise `terminationGracePeriodSeconds` accordingly. Protect dependencies with admission control on handshakes and cached token validation.

Q6. A gateway crash causes a cascading outage of your auth service. What happened and how do you prevent it?

Answer. A reconnect storm: hundreds of thousands of clients reconnected within a second, each requiring a TLS handshake and a token validation; the auth service saturated, handshakes timed out, clients retried immediately, and the load became self-sustaining — possibly spreading to other gateways and the message store via resync queries. Prevention: client-side exponential backoff with full jitter (spreading n reconnects over a window W gives $\lambda \approx n/W$); server-side admission control (token bucket on new handshakes, close 1013 with a retry hint); validate signed tokens locally at the gateway instead of calling auth on every connect; resumable sessions so a reconnect costs a small delta fetch; smaller per-gateway connection counts to limit blast radius; load-test failure of a gateway at peak.

9.14 Practical exercises

Exercise 9.1 — SSE endpoint with resumption. Run the standard-library SSE server from §9.5. (a) Connect with `curl -N http://127.0.0.1:8080/events`, disconnect after five events, and reconnect with `-H "Last-Event-ID: 5"`; confirm no events are lost or duplicated. (b) Write a small HTML page using `EventSource` and watch the browser reconnect automatically when you restart the server. (c) Replace the in-memory `HISTORY` with a Redis Stream (`XADD` / `XRANGE`) so two server processes behind a proxy can both resume a client. (d) Put NGINX in front with default settings and observe buffering; then fix it with `proxy_buffering off`.

Exercise 9.2 — WebSocket chat server with heartbeats and backpressure. The following server uses the `websockets` library (`pip install "websockets>=13"`; tested with version 17.3 on Python 3.13). It supports rooms, server Pings every 20 s, a maximum message size, and a bounded outbox per client that disconnects slow consumers with close code 1008.

```
        """Room-based WebSocket chat server with heartbeats and slow-consumer protec-
tion."""
        import asyncio
        import json

        from websockets.asyncio.server import serve, ServerConnection
        from websockets.exceptions import ConnectionClosed

        OUTBOX_LIMIT = 100                          # max queued messages per client
        rooms: dict[str, set["Client"]] = {}

        class Client:
            def __init__(self, ws: ServerConnection, room: str, name: str):
                self.ws, self.room, self.name = ws, room, name
                self.outbox: asyncio.Queue[str] = asyncio.Queue(maxsize=OUTBOX_LIMIT)

            def offer(self, message: str) -> None:
                """Non-blocking enqueue; a full outbox means a slow consumer."""
                try:
                    self.outbox.put_nowait(message)
                except asyncio.QueueFull:
                    # Policy: disconnect slow consumers rather than buffer without
limit.
                    asyncio.ensure_future(self.ws.close(code=1008, reason="slow con-
sumer"))

            async def writer(self) -> None:
                while True:
                    message = await self.outbox.get()
                    await self.ws.send(message)    # awaits if the TCP send buffer is
full

        def publish(room: str, sender: str, text: str) -> None:
            message = json.dumps({"room": room, "from": sender, "text": text})
            for client in list(rooms.get(room, ())):
                client.offer(message)

        async def handler(ws: ServerConnection) -> None:
            # First message must be a join: {"room": "...", "name": "..."}
            try:
                hello = json.loads(await asyncio.wait_for(ws.recv(), timeout=10))
                room, name = str(hello["room"]), str(hello["name"])
            except (asyncio.TimeoutError, ValueError, KeyError, TypeError):
                await ws.close(code=1008, reason="expected join message")
                return

            client = Client(ws, room, name)
            rooms.setdefault(room, set()).add(client)
            writer_task = asyncio.create_task(client.writer())
            publish(room, "system", f"{name} joined")
            try:
                async for raw in ws:                        # reader loop
                    if isinstance(raw, str) and len(raw) <= 4096:
                        publish(room, name, raw)
            except ConnectionClosed:
                pass
```

```
            finally:
                writer_task.cancel()
                rooms[room].discard(client)
                if not rooms[room]:
                    del rooms[room]
                publish(room, "system", f"{name} left")

        async def main() -> None:
            # Ping every 20 s; close if no Pong within 20 s (detects dead TCP peers).
            async with serve(handler, "127.0.0.1", 8765,
                             ping_interval=20, ping_timeout=20,
                             max_size=64 * 1024) as server:
                await server.serve_forever()

        if __name__ == "__main__":
            asyncio.run(main())
```

What to notice:

- Each connection has two tasks: the handler's reader loop and a writer draining a bounded queue. Fan-out (`publish`) never awaits, so one slow socket cannot stall delivery to the rest of the room.
- When a client's outbox fills, the server closes it with 1008; in testing, a client that stopped reading while another flooded the room was disconnected with `1008 slow consumer` while the server stayed healthy.
- The `finally` block is the cleanup path from Figure 9.5: unsubscribe and announce departure regardless of how the connection ended.
- `max_size` bounds memory per inbound message (oversized messages are closed with 1009).

Tasks: (a) write a client with `websockets.asyncio.client.connect` that joins a room and prints messages; (b) add per-room sequence numbers and a `{"type":"resume","after":N}` message backed by a bounded history; (c) run two server processes and connect them with Redis Pub/Sub so rooms span processes; (d) add a client reconnect loop using `full_jitter_delay` from §9.9.6 and kill the server to watch the reconnect spread.

Exercise 9.3 — Measure memory per connection. Using Exercise 9.2's server, open 10,000 idle connections from a load generator (raise `ulimit -n` first), record the server's resident memory before and after, and compute KB per connection. Repeat with `wss://` (TLS) and compare against Table 9.12.

Exercise 9.4 — Capacity spreadsheet. Implement the formulas from §9.3, §9.4 and §9.9 in a script that takes N, T, λ, m, M, f, Z and c_{blast} and prints request rates, gateway counts, memory, heartbeat rate and reconnect-storm peak for a given jitter window.

9.15 Design challenges

1. **Live sports scores for 20 million viewers.** Scores change a few times per minute; viewers must see updates within 2 seconds. Compare short polling through a CDN (cacheable responses, $T = 2$ s) with SSE through an edge fan-out tier. Estimate request rate, connection count and egress for each, and decide.
2. **Collaborative document editor.** Up to 50 concurrent editors per document, 10 keystroke operations per second per editor, cursors and presence shown live. Design the gateway routing (all editors of a document on one node, or a backplane?), ordering (per-document sequence numbers), backpressure for cursor updates (coalesce), and what happens when the node holding a document fails.
3. **Ride-hailing driver tracking.** 1 million active drivers send GPS every 4 s; riders watch their assigned driver while the app is open and receive “driver arrived” when it is not. Choose techniques for each direction, size the ingest and fan-out, and design behavior when the rider’s phone loses connectivity for 30 s.

9.16 References

1. Fette, I., Melnikov, A. *The WebSocket Protocol* (RFC 6455). IETF, 2011. https://www.rfc-editor.org/rfc/rfc6455
2. WHATWG. *HTML Living Standard — §9.2 Server-sent events*. WHATWG, living standard. https://html.spec.whatwg.org/multipage/server-sent-events.html
3. Loreto, S., Saint-Andre, P., Salsano, S., Wilkins, G. *Known Issues and Best Practices for the Use of Long Polling and Streaming in Bidirectional HTTP* (RFC 6202). IETF, 2011. https://www.rfc-editor.org/rfc/rfc6202
4. Russell, A. *Comet: Low Latency Data for the Browser*. Infrequently Noted (blog), 2006. https://infrequently.org/2006/03/comet-low-latency-data-for-the-browser/
5. Alvestrand, H. *Overview: Real-Time Protocols for Browser-Based Applications* (RFC 8825). IETF, 2021. https://www.rfc-editor.org/rfc/rfc8825
6. W3C. *WebRTC: Real-Time Communication in Browsers*. W3C Recommendation. https://www.w3.org/TR/webrtc/
7. Uberti, J., Jennings, C., Rescorla, E. *JavaScript Session Establishment Protocol (JSEP)* (RFC 8829). IETF, 2021. https://www.rfc-editor.org/rfc/rfc8829
8. Keränen, A., Holmberg, C., Rosenberg, J. *Interactive Connectivity Establishment (ICE)* (RFC 8445). IETF, 2018. https://www.rfc-editor.org/rfc/rfc8445
9. Petit-Huguenin, M., Salgueiro, G., Rosenberg, J., Wing, D., Mahy, R., Matthews, P. *Session Traversal Utilities for NAT (STUN)* (RFC 8489). IETF, 2020. https://www.rfc-editor.org/rfc/rfc8489
10. Reddy, T., Johnston, A., Matthews, P., Rosenberg, J. *Traversal Using Relays around NAT (TURN)* (RFC 8656). IETF, 2020. https://www.rfc-editor.org/rfc/rfc8656

11. McManus, P. *Bootstrapping WebSockets with HTTP/2* (RFC 8441). IETF, 2018. https://www.rfc-editor.org/rfc/rfc8441
12. Hamilton, R. *Bootstrapping WebSockets with HTTP/3* (RFC 9220). IETF, 2022. https://www.rfc-editor.org/rfc/rfc9220
13. Yoshino, T. *Compression Extensions for WebSocket* (RFC 7692). IETF, 2015. https://www.rfc-editor.org/rfc/rfc7692
14. Huang, L.-S., Chen, E. Y., Barth, A., Rescorla, E., Jackson, C. *Talking to Yourself for Fun and Profit*. Web 2.0 Security and Privacy Workshop (W2SP), 2011.
15. Grigorik, I. *High Performance Browser Networking*. O'Reilly Media, 2013. https://hpbn.co/
16. Apple. *Sending notification requests to APNs*. Apple Developer Documentation. https://developer.apple.com/documentation/usernotifications/sending-notification-requests-to-apns
17. Apple. *Handling notification responses from APNs*. Apple Developer Documentation. https://developer.apple.com/documentation/usernotifications/handling-notification-responses-from-apns
18. Google. *Non-collapsible and collapsible messages*. Firebase Cloud Messaging documentation. https://firebase.google.com/docs/cloud-messaging/customize-messages/collapsible-message-types
19. Google. *Best practices for FCM registration token management*. Firebase Cloud Messaging documentation. https://firebase.google.com/docs/cloud-messaging/manage-tokens
20. Google. *Firebase Cloud Messaging HTTP v1 API*. Firebase documentation. https://firebase.google.com/docs/cloud-messaging
21. Slack Engineering. *Real-time Messaging*. Slack Engineering blog, 2023. https://slack.engineering/real-time-messaging/
22. Slack Engineering. *Migrating Millions of Concurrent Websockets to Envoy*. Slack Engineering blog, 2021. https://slack.engineering/migrating-millions-of-concurrent-websockets-to-envoy/
23. Discord. *How Discord Scaled Elixir to 5,000,000 Concurrent Users*. Discord blog, 2017. https://discord.com/blog/how-discord-scaled-elixir-to-5-000-000-concurrent-users
24. WhatsApp. *1 million is so 2011*. WhatsApp blog, 2012. https://blog.whatsapp.com/1-million-is-so-2011
25. Reed, R. *Scaling to Millions of Simultaneous Connections*. Erlang Factory SF Bay Area, 2012. https://www.erlang-factory.com/upload/presentations/558/efsf2012-whatsapp-scaling.pdf
26. Brooker, M. *Exponential Backoff and Jitter*. AWS Architecture Blog, 2015. https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/
27. Redis. *Redis Pub/Sub* (including sharded Pub/Sub). Redis documentation. https://redis.io/docs/latest/develop/interact/pubsub/

28. Socket.IO. *Using multiple nodes*. Socket.IO documentation. https://socket.io/docs/v4/using-multiple-nodes/
29. Kegel, D. *The C10K problem*. 1999 (updated later). http://www.kegel.com/c10k.html
30. Thomson, M., Damaggio, E., Raymor, B. *Generic Event Delivery Using HTTP Push* (RFC 8030). IETF, 2016. https://www.rfc-editor.org/rfc/rfc8030

9.17 Open verification items

- APNs: exact current wording that only the latest notification per app is stored for offline devices; `apns-collapse-id` 64-byte limit; VoIP payload 5 KB; guidance of “two or three” background pushes per hour (Apple docs not fetched directly in this session).
- FCM: default/maximum TTL of 4 weeks (28 days) and 4 KB payload limit (firebase.google.com was unreachable from the authoring environment; limits on 100 stored messages, four collapse keys and 270-day token expiry were confirmed via search snippets of the official pages only).
- SSE: confirm current WHATWG text that non-200 responses “fail the connection” (no auto-reconnect) and the default reconnection time wording.
- Default idle timeouts (AWS ALB 60 s, NGINX `proxy_read_timeout` 60 s) and Kubernetes `terminationGracePeriodSeconds` 30 s — stable for years but flagged “as of 2024”.
- Authors of RFC 8489 and RFC 8656 reference entries; exact URL of Russell 2006 post.
- Per-connection memory figures in Table 9.12 are Illustrative and should be replaced with measured values (Exercise 9.3).
- Runnable programs should also be copied to `code/ch09/` per style guide §8 (not done; out of this author’s file scope).

Chapter 10 — Security Fundamentals for System Designers

> **Level:** 2 · **Track:** Shared · **Estimated reading time:** 5 h

Every system you design will be attacked. Not “might be”: a service with a public IP address receives automated scanning traffic within minutes of going online, and any system that stores money, personal data or compute capacity is worth an attacker’s time. Security is therefore not a feature you add at the end. It is a property of the architecture, in the same way that availability and latency are, and the cheapest time to get it right is when the boxes and arrows are still on the whiteboard.

This chapter gives you the vocabulary and mental models a system designer needs: finding threats systematically, choosing cryptographic building blocks, storing passwords and secrets, encrypting data with managed keys, limiting the blast radius of a breach, surviving denial-of-service attacks and keeping records an intruder cannot quietly rewrite. The focus is defensive and architectural. Chapter 50 builds on it with identity architecture, supply-chain security and privacy engineering.

10.1 Learning objectives

After this chapter you should be able to:

- Draw a data-flow diagram for a system, mark its trust boundaries and enumerate threats with STRIDE.
- Choose the correct cryptographic primitive (hash, MAC, symmetric encryption, asymmetric encryption, signature, key exchange) for a stated security goal and explain what it does *not* guarantee.
- Explain how a browser decides to trust a TLS certificate, and design certificate issuance and rotation for internal services.
- Store passwords with Argon2id, scrypt or bcrypt, choose cost parameters, and compute an attacker’s expected brute-force cost.
- Map each OWASP Top 10 (2021) category to an architectural control.
- Design secrets management with a vault, short-lived credentials and automated rotation.
- Explain envelope encryption with a key management service (KMS) and draw its sequence diagram.

- Contrast perimeter security with zero trust, and explain mTLS, least privilege and network segmentation.
- Classify DDoS attacks as volumetric, protocol or application-layer and place mitigations at the right layer.
- Design a tamper-evident audit log.
- Apply a security checklist to any system design.

10.2 Prerequisites

- TCP, UDP and the OSI layers — §7.3, §7.7, §7.8 (DDoS attacks target specific layers).
- TLS 1.3 and HTTPS — §7.10 (this chapter explains the cryptography and certificates underneath it).
- Firewalls and security groups — §7.14 (the starting point for network segmentation).
- Authentication, authorization, sessions, JWT and OAuth 2.0 — §8.10–§8.14 (we assume you know what a session token is).
- Powers of two and logarithms — §4.3, §4.4 (key sizes and brute-force estimates are exponential).
- Probability essentials — §4.6 (collision and risk estimates).

10.3 Threat modeling

10.3.1 What “secure” means: security goals

DEFINITION

Definition — Security property. A statement about what must remain true about a system even when an adversary is actively trying to violate it. The classic triad is **confidentiality** (only authorized parties can read data), **integrity** (data and behavior cannot be changed without detection) and **availability** (authorized parties can use the system when they need it), often abbreviated **CIA**. System designers add **authentication** (knowing who is acting), **authorization** (deciding what that actor may do) and **non-repudiation** (an actor cannot credibly deny an action they took).

First-principles explanation

Think about a house. Confidentiality is curtains: people outside cannot see your belongings. Integrity is a tamper seal on the medicine cabinet: you may not stop someone opening it, but you will know if they did. Availability is being able to get into your own home: a burglar who glues your lock shut has harmed you without stealing anything.
Authentication is the doorman recognizing your face; authorization is the rule that your

face opens the front door but not the neighbor's flat. Non-repudiation is the signed visitor's book: a visitor cannot later claim they were never there.

"Is this system secure?" has no answer on its own. The only answerable question is "secure against *whom*, protecting *what*, with respect to *which* property?" A public weather website needs integrity and availability but almost no confidentiality. A password database needs all three, and confidentiality failures there are catastrophic. Threat modeling is the discipline of asking these questions systematically instead of by intuition.

10.3.2 Assets, adversaries and trust boundaries

DEFINITION

Definition — Threat model. A structured description of a system, the **assets** in it that are worth protecting, the **adversaries** who might attack it and their capabilities, the **threats** (ways a security property could be violated), and the **mitigations** chosen for each threat [Shostack 2014].

DEFINITION

Definition — Trust boundary. A line in a system where the level of trust changes: data or requests crossing it come from a party that the receiving side does not fully control or trust. Examples: the internet-to-load-balancer edge, the browser-to-server connection, a service calling a third-party API, a tenant's code running on shared infrastructure.

Assets are the things an attacker wants or the things you cannot afford to lose: user credentials, payment data, personal data, signing keys, the ability to send email from your domain, compute capacity (attackers mine cryptocurrency on stolen cloud accounts), and your reputation. Rank them: a leaked public product catalog is embarrassing; leaked password hashes or private keys are a crisis.

Adversaries range from automated opportunists (bots scanning the whole internet for default passwords and known vulnerabilities), through targeted external attackers who study *your* system and phish *your* employees, malicious or compromised **insiders** who already hold legitimate credentials, and supply-chain attackers who compromise a library or vendor you depend on, up to well-resourced state actors. Every internet-facing design must handle the first three. The insider is the one beginners forget, and the reason zero trust (§10.10) and audit logging (§10.12) exist.

Trust boundaries are where attacks happen. Inside a single process, data you created yourself can be trusted; the moment data crosses a boundary, it must be **authenticated**

(who sent it?), **authorized** (are they allowed?) and **validated** (is it well-formed?). Almost every vulnerability class in §10.7 is a failure to do one of these three things at a boundary.

10.3.3 Data-flow diagrams

How it works internally

A **data-flow diagram (DFD)** is the standard picture for threat modeling. It uses only four element types, plus boundaries:

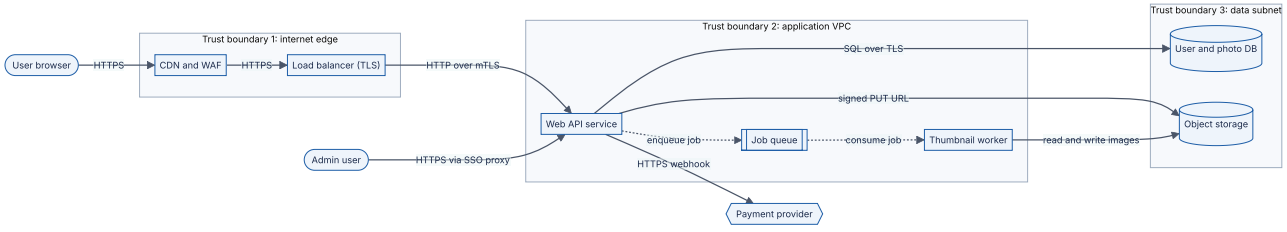
1. **External entity** — something outside your control that sends or receives data (a user, a partner API).
2. **Process** — code that transforms data (a web server, a worker).
3. **Data store** — where data rests (a database, a bucket, a cache, a log).
4. **Data flow** — an arrow showing data moving between the above, labeled with what moves and over which protocol.
5. **Trust boundary** — a dashed box or line where trust changes.

The rules are deliberately simple so that every engineer on a team can draw one in ten minutes. The value is not the drawing; it is that each arrow crossing a boundary becomes a question: *how is this authenticated, authorized, validated and protected in transit?*

Shostack frames the whole exercise as four questions [Shostack 2014]: **What are we working on?** (draw the DFD) · **What can go wrong?** (apply STRIDE) · **What are we going to do about it?** (choose mitigations) · **Did we do a good job?** (review and test).

Architecture diagram

Figure 10.1 — Data-flow diagram of a photo-sharing web application with trust boundaries.



Reading the diagram. Two external entities (the user's browser and an administrator) and one external system (a payment provider, hexagon) sit outside every boundary. Trust boundary 1 is the internet edge: the CDN with a web application firewall (WAF) and the TLS-terminating load balancer are the first components that see untrusted traffic, so they carry rate limiting and coarse filtering. Trust boundary 2 is the application network: the web API is the only process that authenticates users and authorizes actions; it talks to the load balancer over mutual TLS. Solid arrows are synchronous request/response flows; the

dotted arrows to and from the job queue are asynchronous — the thumbnail worker processes uploads later, so a failure there delays thumbnails but does not break uploads. Trust boundary 3 contains the storage: the database and object store accept connections only from the application subnet. Notice where the risky crossings are: every arrow from `U`, `ADM` and `PAY` crosses at least one boundary, and the worker parses user-uploaded image files, which is a classic place for parser vulnerabilities. Scaling options (more web replicas, more workers) do not change the boundaries, but each new data store or third-party integration adds a new arrow that must be modeled.

10.3.4 STRIDE

DEFINITION

Definition — STRIDE. A mnemonic for six threat categories, developed at Microsoft in 1999 by Loren Kohnfelder and Praerit Garg and popularized by Shostack [Shostack 2014]: **S**poofing, **T**ampering, **R**epudiation, **I**nformation disclosure, **D**enial of service and **E**levation of privilege. Each category is the violation of one security property.

Table 10.1 — STRIDE categories, the property each violates, and typical mitigations.

Threat	Violates	Plain-language question	Typical mitigations
Spoofing	Authentication	Can someone pretend to be another user or service?	Strong authentication, MFA, mTLS, signed tokens
Tampering	Integrity	Can someone change data in transit or at rest?	TLS, MACs, signatures, AEAD encryption, DB permissions
Repudiation	Non-repudiation	Can someone deny doing something?	Audit logs, signatures, tamper-evident storage
Information disclosure	Confidentiality	Can someone read what they should not?	Encryption, access control, data minimization
Denial of service	Availability	Can someone stop legitimate use?	Rate limits, quotas, autoscaling, DDoS protection
Elevation of privilege	Authorization	Can someone gain rights they were not granted?	Least privilege, authorization checks on every request, sandboxing

You apply STRIDE **per element** of the DFD: for each process, data store and flow, walk through the six letters and ask whether the threat applies. Not every letter applies to every element type: an external entity can be spoofed but cannot be “tampered with” by you; a data flow can be tampered with, disclosed or flooded. This mechanical walk-

through is what makes STRIDE useful — it stops you from fixating on the one threat you happened to think of first.

10.3.5 Worked example: STRIDE on the photo-sharing application

WORKED EXAMPLE

Worked example — STRIDE pass over Figure 10.1.

Step 1 — Assets. User credentials and session tokens; private photos; payment tokens returned by the provider; admin capabilities (delete any account); the database administrator credentials; the TLS private keys.

Step 2 — Walk each boundary-crossing element. The table below shows a representative (not exhaustive) result.

Step 3 — Rank. Score each threat for likelihood and impact on a 1–3 scale (see the risk-score formula below) and fix high scores first.

Step 4 — Record. The result is a short table in the design document, owned by the team, revisited whenever the DFD changes.

Table 10.2 — STRIDE threats for the photo-sharing application (excerpt).

#	Element	STRIDE	Threat	Mitigation	L × I
T1	User → Web API	S	Credential stuffing with passwords leaked from other sites	Rate limit per account and IP; breached-password check; MFA; Argon2id storage	3 × 3 = 9
T2	Web API	E	User changes `photo_id` in URL and reads another user's private photo (IDOR)	Object-level authorization check on every read, enforced in a shared data-access layer	3 × 3 = 9
T3	Web API → Object store	I	Bucket accidentally made public	Block public access at account level; serve via short-lived signed URLs	2 × 3 = 6
T4	Thumbnail worker	E	Malicious image exploits image-parsing library, attacker runs code in worker	Sandbox worker (no network, minimal IAM role), patch libraries, separate account	2 × 3 = 6
T5	Payment webhook	S	Attacker forges "payment succeeded" webhook	Verify provider's HMAC signature and timestamp (§10.4.6)	2 × 3 = 6
T6	Admin → Web API	R	Admin deletes an account and later denies it	Append-only, tamper-evident audit log of admin actions (§10.12)	1 × 2 = 2
T7	Upload endpoint	D	Attacker uploads thousands of 1 GB files	Size limits, per-user quotas, rate limits	2 × 2 = 4
T8	Web API → DB	T	SQL injection modifies rows	Parameterized queries; DB user without DDL rights	2 × 3 = 6

Look at what the exercise produced: eight concrete, testable requirements, most of which are architectural (where the authorization check lives, which account the worker runs in, how the bucket is exposed) rather than "write secure code". That is the point of threat modeling for a system designer.

Mathematical foundations

FORMULA

Qualitative risk score

$$R = L \times I$$

Symbol	Meaning	Units
R	Risk score used to rank threats	dimensionless (1–9 on a 3-point scale)
L	Likelihood that the threat is exploited within the planning horizon (1 = unlikely, 2 = plausible, 3 = expected)	ordinal score
I	Impact if exploited (1 = minor, 2 = serious, 3 = severe: data breach, financial loss, regulatory)	ordinal score

WORKED EXAMPLE

Worked example — Ranking T1 against T6. Credential stuffing (T1) is *expected* for any consumer login page, because attackers replay billions of leaked username–password pairs automatically: $L = 3$. Its impact is account takeover and private-photo exposure: $I = 3$. So $R = 3 \times 3 = 9$. Admin repudiation (T6) requires a malicious administrator ($L = 1$) and its impact is a dispute rather than a breach ($I = 2$): $R = 2$. *Interpretation:* fix T1 in the first release; T6 can follow. *Engineering implication:* the score is ordinal, not a probability — do not multiply it by dollar figures or average it across threats. Its only job is to order the work and make the reasoning visible to reviewers.

Advantages, disadvantages and alternatives

STRIDE is easy to teach, systematic and tied to properties, which makes it ideal for design reviews. Its weaknesses: it can produce long lists of low-value threats, it does not rank risk by itself, and it focuses on technical threats more than abuse of legitimate features (spam, fraud, harassment). Alternatives include **attack trees** (a root goal such as “read another user’s photo” decomposed into AND/OR subgoals — good for deep analysis of one critical asset), **PASTA** (a seven-stage risk-centric process), **LINDDUN** (privacy threats) and **kill-chain** analysis (focused on detection and response). Many teams use STRIDE for breadth and attack trees for the top two or three risks.

Failure scenarios

Table 10.3 — Threat-modeling failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Threat model is stale	Architecture changed, DFD did not	New data store or vendor absent from DFD in review	Re-run STRIDE on changed elements	Make DFD update a checklist item for design changes
Threat model is a document nobody reads	Produced for compliance, not engineering	Mitigations not traced to tickets/tests	Convert each mitigation to a tracked task	Keep it short; review it in design review
Insider threats ignored	"Everything inside the VPC is trusted" assumption	Red-team exercise with internal credentials	Add authorization and audit inside the network	Include insider row in adversary list

Common misconceptions

COMMON MISTAKE

"We use HTTPS, so we are secure." TLS protects data *in transit* between two endpoints. It does nothing about a logged-in user requesting someone else's data (T2), a forged webhook (T5) or SQL injection (T8). Most real breaches happen through authenticated, encrypted connections.

10.4 Cryptography essentials

Cryptography is the toolbox that turns security goals into mathematics. As a system designer you will almost never implement a cryptographic algorithm, but you will constantly *choose* one, and choosing the wrong primitive for a goal (encrypting when you needed to authenticate, hashing when you needed a secret key) is one of the most common design errors. This section explains each primitive by analogy, then by mechanism, then with the numbers that tell you how strong it is.

Table 10.4 — Which primitive achieves which goal.

Goal	Primitive	Needs a key?	Typical algorithm (as of 2026)	Analogy
Detect accidental or malicious change, given a trusted reference value	Cryptographic hash	No	SHA-256, SHA-3, BLAKE2/3	Fingerprint
Prove a message came from someone holding a shared secret, unchanged	MAC	Shared secret	HMAC-SHA-256	Wax seal pressed with a ring both parties own
Keep data secret *and* detect tampering, between parties sharing a key	Authenticated (AEAD) symmetric encryption	Shared secret	AES-256-GCM, ChaCha20-Poly1305	Locked box with a tamper seal
Let anyone send you a secret without a pre-shared key	Asymmetric encryption / KEM	Public + private key	RSA-OAEP, ECIES, ML-KEM	Padlock you hand out open; only you hold its key
Agree on a shared secret over a public channel	Key exchange	Ephemeral key pairs	ECDHE (X25519, P-256)	Mixing paint colors (see §10.4.4)
Prove to *anyone* that a specific key holder approved a message	Digital signature	Private key signs, public key verifies	Ed25519, ECDSA P-256, RSA-PSS	Notarized handwritten signature
Produce unpredictable keys, nonces, tokens	CSPRNG	Seeded by OS	`/dev/urandom`, `getrandom()`, `secrets`	Fair dice nobody can watch

10.4.1 Hash functions

DEFINITION

Definition — Cryptographic hash function. A deterministic function H that maps an input of any length to a fixed-length output (the **digest**), such that it is computationally infeasible to (1) find an input that hashes to a given digest (**preimage resistance**), (2) given one input, find a different input with the same digest (**second-preimage resistance**), and (3) find any two distinct inputs with the same digest (**collision resistance**).

First-principles explanation

A hash is a fingerprint. You cannot reconstruct a person from a fingerprint, two people practically never share one, and the tiniest change in the person (a cut on the finger) changes the print. SHA-256 takes a 4 GB disk image or the single letter “a” and produces 256 bits; flip one input bit and roughly half of the output bits flip (the **avalanche effect**).

Hashes give you integrity *only if the reference digest itself is trustworthy*. If an attacker can change a download *and* the digest published next to it, the hash proves nothing. That is why hashes are combined with keys (MACs) or signatures, and why a plain hash is never a password-storage scheme on its own (§10.6).

How it works internally

SHA-2 hashes pad the input, split it into 512-bit (SHA-256) blocks and mix each block into a running internal state with a **compression function** (the **Merkle–Damgård construction**); the final state is the digest. A side effect is the **length-extension property**: given $H(m)$ and the length of m, anyone can compute $H(m \parallel \text{padding} \parallel x)$ without knowing m. That is harmless for fingerprints but fatal if you build a MAC as $H(\text{key} \parallel \text{message})$ — the reason HMAC exists (§10.4.6). SHA-3 uses a different "sponge" construction without this property.

MD5 and SHA-1 are broken for collision resistance: practical collisions were published for MD5 in 2004 and for SHA-1 in 2017 (the "SHAttered" attack). Do not use them for any security purpose.

Mathematical foundations

FORMULA

Birthday bound for hash collisions

$$P_{\text{collision}} \approx \frac{k^2}{2^{n+1}}$$

Symbol	Meaning	Units
k	Number of distinct inputs hashed	count
n	Digest length	bits
$P_{\text{collision}}$	Probability that at least two of the k digests are equal (valid while $P \ll 1$)	probability

Consequence: an n-bit hash offers about $n/2$ bits of collision resistance, because P approaches $1/2$ when $k \approx 2^{n/2}$.

WORKED EXAMPLE

Worked example — Is a 64-bit truncated hash safe as a content ID? Suppose a storage service names objects by the first 64 bits of their SHA-256 digest and stores $k = 10^{10}$ objects (ten billion).

1. $k^2 = 10^{20}$.
2. $2^{65} \approx 3.69 \times 10^{19}$.
3. $P \approx 10^{20}/3.69 \times 10^{19} \approx 2.7$ — the approximation has broken down because P is not small; in reality a collision is all but certain.

With the full 256-bit digest: $P \approx 10^{20}/2^{257} \approx 10^{20}/2.3 \times 10^{77} \approx 4 \times 10^{-58}$.

Interpretation: 64 bits feels large, but birthday collisions arrive at around $2^{32} \approx 4.3$ billion items. *Engineering implication:* if an adversary benefits from a collision (deduplication, content addressing, signatures), use at least 256-bit digests; truncate only for non-adversarial uses such as cache sharding.

10.4.2 Symmetric encryption and AES-GCM

DEFINITION

Definition — Symmetric encryption. Encryption in which the same secret key is used to encrypt and decrypt. A **block cipher** such as AES (Advanced Encryption Standard) encrypts fixed 128-bit blocks under a 128-, 192- or 256-bit key; a **mode of operation** turns the block cipher into a scheme for messages of any length.

DEFINITION

Definition — Authenticated encryption with associated data (AEAD). A symmetric scheme that provides confidentiality for the plaintext *and* integrity for both the ciphertext and optional unencrypted **associated data** (AAD), producing an authentication **tag** that decryption checks before releasing any plaintext. AES-GCM and ChaCha20-Poly1305 are AEAD schemes.

First-principles explanation

Symmetric encryption is a lockbox where sender and receiver hold copies of the same key. It is extremely fast: modern CPUs have AES instructions (AES-NI on x86, equivalent extensions on ARM) that encrypt at multiple gigabytes per second per core, which is why every bulk-data protocol — TLS records, disk encryption, database encryption — uses symmetric ciphers for the data itself. The hard part is not encrypting; it is getting the

same key to both parties securely and keeping it secret, which the rest of this chapter addresses.

An *unauthenticated* mode such as AES-CBC or AES-CTR hides the content but does not detect changes. An attacker who flips bit 7 of a CTR ciphertext flips bit 7 of the plaintext — turning “pay $100” into “pay $900” without knowing the key. AEAD adds the tamper seal: any change to ciphertext, tag, nonce or associated data makes decryption fail.

How it works internally

AES-GCM (Galois/Counter Mode, NIST SP 800-38D):

1. Take a key K and a 96-bit **nonce** (number used once) N.
2. Generate a keystream by encrypting the counter values $N \parallel 2, N \parallel 3, \ldots$ with AES; XOR it with the plaintext to get ciphertext (counter mode).
3. Compute a 128-bit tag over the AAD and ciphertext using GHASH, a polynomial hash keyed by a value derived from K, and mask it with the encryption of $N \parallel 1$.
4. Output (nonce, ciphertext, tag). Decryption recomputes the tag and rejects mismatches before returning plaintext.

The single most important rule: **never reuse a nonce with the same key.** A repeated nonce produces the same keystream, so XOR of two ciphertexts reveals the XOR of the two plaintexts, and it also leaks enough to forge tags. Use a 96-bit random nonce per message, or a counter that is guaranteed never to repeat, and rotate keys before the nonce collision probability becomes meaningful.

FORMULA

Random-nonce collision probability for AES-GCM

$$P_{\text{reuse}} \approx \frac{q^2}{2^{97}}$$

Symbol	Meaning	Units
q	Number of messages encrypted under one key with random 96-bit nonces	messages
P_{reuse}	Probability that any two of them share a nonce (birthday bound with $n = 96$)	probability

WORKED EXAMPLE

Worked example — How many messages per key? NIST SP 800-38D caps random-nonce GCM at 2^{32} invocations per key. Check the risk: $q = 2^{32}$ gives $q^2 = 2^{64}$, so $P \approx 2^{64}/2^{97} = 2^{-33} \approx 1.2 \times 10^{-10}$. A service encrypting 50,000 messages per second reaches $2^{32} \approx 4.3 \times 10^9$ messages in $4.3 \times 10^9 / 5 \times 10^4 \approx 86,000$ s, about one day. *Engineering implication:* a high-throughput service must rotate data keys at least daily, or use per-object keys (envelope encryption, §10.9), or a nonce-misuse-resistant mode such as AES-GCM-SIV, or a scheme with larger nonces (XChaCha20-Poly1305 uses 192-bit nonces).

The associated data is a powerful design tool. If you encrypt a user's tax ID in row 42 of the `users` table and put `table=users;id=42` in the AAD, an attacker with database write access cannot copy that ciphertext into row 43 — decryption would fail because the context differs. The runnable example in §10.4.8 demonstrates exactly this.

10.4.3 Asymmetric cryptography: RSA and elliptic curves

DEFINITION

Definition — Asymmetric (public-key) cryptography. Cryptography using a mathematically linked **key pair**: a **public key** that can be shared with anyone and a **private key** kept secret by its owner. What one key does, only the other can undo or verify. Security rests on problems believed hard to invert: factoring large integers (RSA) or computing discrete logarithms on elliptic curves (ECC).

First-principles explanation

Picture a padlock factory. You manufacture thousands of open padlocks, all of which only your key opens, and leave them in a bowl on the street. Anyone can put a message in a box, snap one of your padlocks shut, and mail it to you; nobody, including the sender, can reopen it — only you. The open padlocks are your public key; your key is the private key. Asymmetric cryptography solves the problem symmetric encryption cannot: two parties who have never met can communicate securely, provided the sender knows the padlock really is yours (that is what certificates in §10.5 establish).

The price is speed and size. An RSA-2048 private-key operation costs on the order of a millisecond of CPU, thousands of times more than encrypting a short message with AES. So real systems use asymmetric cryptography only to establish or protect a symmetric key and to sign things, never for bulk data. This combination is called **hybrid encryption**.

How it works internally

- **RSA**: pick two large random primes p and q; the public key is $(n = pq, e)$, the private key is d with $ed \equiv 1 \pmod{(p-1)(q-1)}$. Encryption is $c = m^e \bmod n$, decryption $m = c^d \bmod n$. Breaking it requires factoring n. RSA must always be used with a padding scheme (OAEP for encryption, PSS for signatures); "textbook RSA" is insecure.
- **Elliptic-curve cryptography (ECC)**: the private key is a random integer a; the public key is the point $A = a \cdot G$ obtained by "adding" a fixed base point G on a curve to itself a times. Recovering a from A (the elliptic-curve discrete logarithm problem) is believed to be infeasible. ECC achieves the same security as RSA with much smaller keys and faster operations, which is why TLS 1.3 deployments overwhelmingly use X25519 and P-256.

Table 10.5 — Approximate equivalent security strengths (NIST SP 800-57 Part 1).

Security strength (bits)	Symmetric	RSA modulus	Elliptic curve key	Hash for collision resistance	Status
112	3-key 3DES (legacy)	2048	224	SHA-224	Minimum acceptable today
128	AES-128	3072	256 (P-256, X25519, Ed25519)	SHA-256	Recommended default
192	AES-192	7680	384 (P-384)	SHA-384	High assurance
256	AES-256	15360	512+ (P-521)	SHA-512	Long-term / post-quantum margin for symmetric

"128-bit security" means the best known attack takes about 2^{128} operations. At 10^{18} operations per second (far beyond all computers on Earth combined) that is $3.4 \times 10^{38} / 10^{18} = 3.4 \times 10^{20}$ s, about 10^{13} years.

WARNING / TRADE-OFF

Quantum computers and "harvest now, decrypt later." A large fault-tolerant quantum computer running Shor's algorithm would break RSA and ECC; Grover's algorithm only roughly halves symmetric strength, leaving AES-256 a large margin. No such machine exists as of 2026, but recorded traffic could be decrypted later. NIST published post-quantum standards in August 2024 — FIPS 203 (ML-KEM), FIPS 204 (ML-DSA) and FIPS 205 (SLH-DSA) — and TLS libraries have begun deploying hybrid X25519 + ML-KEM key exchange. Design for **crypto agility** (configurable algorithms) and prefer AES-256 for long-lived data.

10.4.4 Key exchange and forward secrecy

First-principles explanation

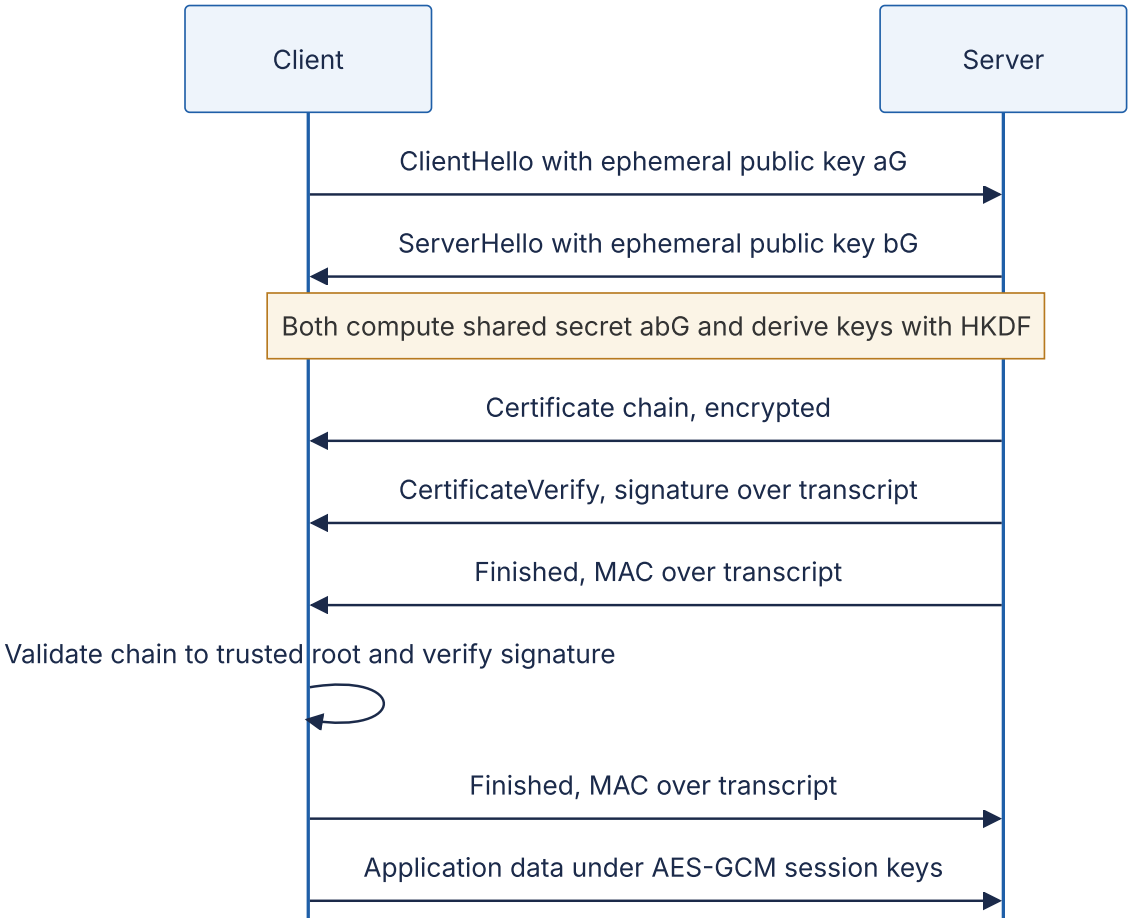
The classic analogy for **Diffie–Hellman key exchange** uses paint. Alice and Bob publicly agree on a common color, yellow. Alice secretly picks red, Bob secretly picks blue. Alice mixes yellow + red and sends the result (orange) across the open channel; Bob mixes yellow + blue and sends green. Alice adds her secret red to Bob's green; Bob adds his secret blue to Alice's orange. Both end up with the same yellow + red + blue brown. An eavesdropper saw yellow, orange and green but cannot "unmix" paint to recover red or blue.

In the elliptic-curve version (ECDH), Alice's secret is a, her public value aG; Bob's are b and bG; both compute the shared point abG. The shared secret is then fed through a **key-derivation function** such as HKDF to produce symmetric session keys.

> DEFINITION
>
> **Definition — Forward secrecy.** A property of a key-exchange protocol whereby compromise of a party's long-term private key does not reveal session keys from past sessions. It is achieved by using fresh **ephemeral** key pairs for every session (the "E" in ECDHE) and deleting them afterward.

Why it matters: before TLS 1.3, many servers used RSA key transport, where the client encrypted the session secret with the server's long-term RSA public key. Anyone who recorded traffic and later stole that one private key could decrypt *years* of recorded sessions. TLS 1.3 removed RSA key transport entirely; every full handshake uses ephemeral (EC)DHE, and the server's long-term key is used only to *sign* the handshake, proving its identity.

Figure 10.2 — Simplified TLS 1.3 handshake: ephemeral key exchange plus signature.



Reading the diagram. All messages flow synchronously over one TCP (or QUIC) connection, and the full handshake completes in one round trip before application data flows (round-trip costs and 0-RTT resumption are covered in §7.10). The ephemeral public values travel in the clear; the shared secret never crosses the network. From the ServerHello onward, everything is encrypted with keys derived from that secret, including the certificate (hiding which certificate the server uses from passive observers). The server proves it owns the certificate's private key by **signing** the handshake transcript (CertificateVerify); the client validates the certificate chain (§10.5). The Finished messages are MACs over everything exchanged so far, so a man-in-the-middle who altered any earlier message is detected. Failure points: an expired or untrusted certificate (handshake aborts), a client and server with no common cipher suite, or clock skew on the client breaking validity checks.

10.4.5 Digital signatures

DEFINITION

Definition — Digital signature. A scheme with three algorithms: key generation, **sign** (using the private key, produces a signature over a message) and **verify** (using the public key, checks a signature against a message). A valid signature shows that the message was approved by the holder of the private key and has not changed since. Because verification needs only the public key, *anyone* can verify, which provides non-repudiation.

Signatures are the notary of cryptography. They secure software updates (the update client holds the vendor's public key and refuses unsigned binaries), container images, JWTs signed with RS256 or ES256 (§8.13), certificates themselves, Git commits and the TLS handshake. In practice you sign a *hash* of the message: the signing algorithm hashes internally, so signing a 2 GB file costs one SHA-256 pass plus one fast private-key operation.

Prefer **Ed25519** (fast, deterministic, small 64-byte signatures, few ways to misuse) or **ECDSA P-256** where compliance requires NIST curves. ECDSA is fragile when the per-signature random value repeats or is biased: in 2010 researchers extracted Sony's PlayStation 3 code-signing key because the same random value was reused for every signature. Ed25519 derives this value deterministically, eliminating the failure mode.

10.4.6 Message authentication codes and HMAC

DEFINITION

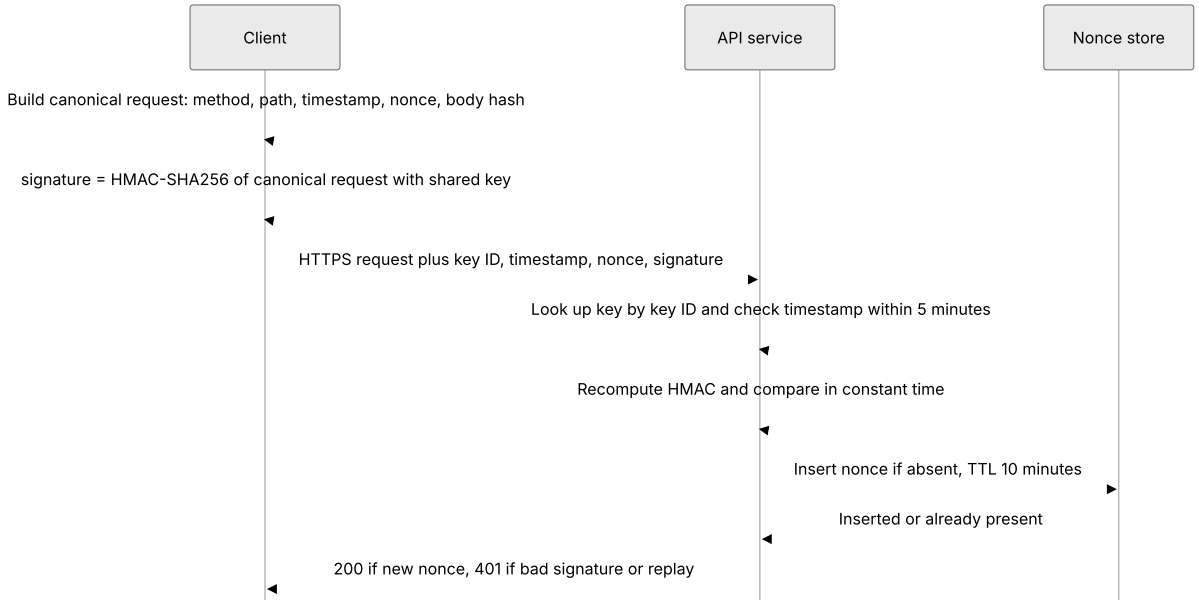
Definition — Message authentication code (MAC). A short tag computed from a message and a secret key, $t = \mathrm{MAC}_K(m)$, such that without K it is infeasible to produce a valid tag for any new message. **HMAC** is the standard construction from a hash function: $\mathrm{HMAC}_K(m) = H((K \oplus \text{opad}) \,\|\, H((K \oplus \text{ipad}) \,\|\, m))$ [Krawczyk et al. 1997].

A MAC is a wax seal pressed with a ring that both parties own. The receiver, holding the same ring, can tell the letter is genuine and unopened. But because both own the ring, a third party cannot tell *which* of them sealed it — so MACs provide integrity and authenticity between the two key holders, but **not** non-repudiation. That is the essential difference from a signature, and the reason payment providers sign webhooks with an HMAC (they only need you to know it came from them) while software vendors sign releases with asymmetric signatures (millions of verifiers who must not be able to forge).

HMAC's nested structure defeats the length-extension attack described in §10.4.1. It is also fast (two hash computations) and needs only a shared key, which makes it the stan-

dard tool for **request signing**: AWS Signature Version 4, Stripe and GitHub webhook signatures and many internal service-to-service schemes use HMAC-SHA-256.

Figure 10.3 — HMAC request signing with replay protection.



Reading the diagram. The client and API service share a secret key identified by a key ID (so keys can be rotated: during rotation the server accepts both the old and new key IDs). The client signs a **canonical** representation — a byte-exact serialization of every field the server will act on — because if a field is not covered (say, the HTTP method), an attacker can change it without invalidating the signature. The timestamp bounds how long a captured request remains useful; the nonce store (in production a shared cache such as Redis with a TTL slightly longer than twice the allowed skew) rejects exact replays inside that window. The signature comparison must be constant-time to avoid leaking, byte by byte, how much of a forged signature was correct. All calls are synchronous; the nonce store is the only shared state and a single point of failure — if it is unavailable, the service must choose between failing closed (reject requests) and failing open (accept possible replays), a decision that belongs in the design document.

Code example: HMAC request signing

```
        """HMAC-SHA256 request signing and verification with replay protection."""
        import hashlib
        import hmac
        import secrets
        import time

        MAX_SKEW_SECONDS = 300          # accept requests at most 5 minutes old/new

        def canonical_request(method: str, path: str, timestamp: str,
                              nonce: str, body: bytes) -> bytes:
            body_hash = hashlib.sha256(body).hexdigest()
            # Every field the server relies on must be covered by the signature.
            return "\n".join([method.upper(), path, timestamp, nonce,
body_hash]).encode()

        def sign(key: bytes, method: str, path: str, body: bytes) -> dict:
            ts = str(int(time.time()))
            nonce = secrets.token_hex(16)
            mac = hmac.new(key, canonical_request(method, path, ts, nonce, body),
                           hashlib.sha256).hexdigest()
            return {"X-Key-Id": "client-42", "X-Timestamp": ts,
                    "X-Nonce": nonce, "X-Signature": mac}

        class Verifier:
            def __init__(self, keys: dict[str, bytes]):
                self.keys = keys
                self.seen_nonces: dict[str, int] = {}    # production: shared store with
TTL

            def verify(self, method: str, path: str, body: bytes, headers: dict) ->
bool:
                key = self.keys.get(headers.get("X-Key-Id", ""))
                if key is None:
                    return False
                ts = int(headers["X-Timestamp"])
                now = int(time.time())
                if abs(now - ts) > MAX_SKEW_SECONDS:
                    return False                                    # stale or future
                expected = hmac.new(key, canonical_request(method, path, headers["X-
Timestamp"],
                                                           headers["X-Nonce"], body),
                                    hashlib.sha256).hexdigest()
                if not hmac.compare_digest(expected, headers["X-Signature"]):
                    return False                                    # forged or tampered
                # Expire old nonces, then reject a nonce we have already seen (replay).
                self.seen_nonces = {n: t for n, t in self.seen_nonces.items()
                                    if now - t <= 2 * MAX_SKEW_SECONDS}
                if headers["X-Nonce"] in self.seen_nonces:
                    return False
                self.seen_nonces[headers["X-Nonce"]] = ts
                return True
```

Tested with Python 3.13: a correctly signed request verifies; replaying it, altering one byte of the body, or signing with a different key all return `False`.

What to notice:

- `hmac.compare_digest` compares in constant time; `==` returns early at the first differing byte and can leak timing information.
- The body is covered via its SHA-256 hash, and the method and path are included, so neither can be swapped.
- The order of checks matters: the nonce is recorded only *after* the signature verifies, so an attacker cannot fill the nonce store with garbage that blocks legitimate nonces.
- The in-memory `seen_nonces` dictionary works for one process only; with several API replicas, use a shared store with an atomic "set if absent" operation (for example Redis `SET key value NX EX 600`).

10.4.7 Randomness

Every key, nonce, salt, session token and password-reset link depends on unpredictable random numbers. General-purpose generators (`random` in Python, `Math.random()` in JavaScript, `rand()` in C) are designed for simulations: they are fast and statistically uniform but **predictable** — observing a few hundred outputs of Python's Mersenne Twister lets an attacker compute all future outputs. Security code must use a **cryptographically secure pseudo-random number generator (CSPRNG)** seeded from the operating system's entropy pool: `secrets` or `os.urandom` in Python, `crypto.getRandomValues` in browsers, `SecureRandom` in Java, `crypto/rand` in Go.

FORMULA

Entropy of a uniformly random secret

$$H = L \cdot \log_2 N$$

Symbol	Meaning	Units
H	Entropy: the number of bits an attacker must effectively guess	bits
L	Length of the secret	symbols
N	Number of equally likely choices per symbol	symbols

Valid only when every symbol is chosen independently and uniformly at random — true for machine-generated tokens, false for human-chosen passwords.

WORKED EXAMPLE

Worked example — How long should a session token be?
`secrets.token_urlsafe(32)` draws 32 random bytes ($N = 256$, $L = 32$): $H = 32 \times 8 = 256$ bits. An attacker making 10^6 guesses per second for a year (3.15×10^{13} guesses) against 10^8 live sessions succeeds with probability about $3.15 \times 10^{13} \times 10^8 / 2^{256} \approx 3 \times 10^{-56}$.
Engineering implication: 128 bits of CSPRNG output is already ample; the realistic risk is tokens leaking through logs, URLs and referrer headers, not guessing.

10.4.8 Don’t roll your own crypto

“Don’t roll your own crypto” does not only mean “don’t invent a cipher”. It means: do not assemble primitives into your own protocol or storage format when a vetted, high-level construction exists. The canonical failures are all compositions of *correct* primitives: encrypting without authenticating (bit-flipping attacks), MAC-then-encrypt with distinguishable padding errors (padding-oracle attacks against CBC), `SHA256(key || msg)` as a MAC (length extension), reused GCM nonces, comparing MACs with `==` (timing attacks), and ECB mode, which encrypts identical blocks to identical ciphertext so that patterns in an image remain visible. Ferguson, Schneier and Kohno make the case that most cryptographic failures in practice are engineering failures in how primitives are combined and keys are handled, not breaks of the primitives themselves [Ferguson et al. 2010].

RECOMMENDATION

Practical rule set. (1) Use TLS 1.3 for data in transit — do not design a transport protocol. (2) Use a high-level library API (libsodium/NaCl `secretbox`, Google Tink, or the `cryptography` package’s AEAD classes) that chooses nonces, modes and tag checks for you. (3) Use a managed KMS for key storage and envelope encryption (§10.9). (4) Use a password-hashing function, not a hash (§10.6). (5) Use your platform’s JWT/OAuth libraries, pinned to an explicit algorithm allow-list. (6) Have a cryptographer review any design where you still feel you need to combine primitives.

Code example: AEAD, signatures and key agreement

```
        """AES-256-GCM with associated data, Ed25519 signatures and X25519 key agree-
ment."""
        import os
        from cryptography.exceptions import InvalidSignature, InvalidTag
        from cryptography.hazmat.primitives import hashes
        from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey
        from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
        from cryptography.hazmat.primitives.ciphers.aead import AESGCM
        from cryptography.hazmat.primitives.kdf.hkdf import HKDF

        # --- 1. Authenticated symmetric encryption: AES-256-GCM ---
        key = AESGCM.generate_key(bit_length=256)
        aead = AESGCM(key)
        nonce = os.urandom(12)                        # 96-bit nonce; NEVER reuse with
same key
        aad = b"user_id=42;table=payments"            # authenticated but not encrypted
        ct = aead.encrypt(nonce, b"card ending 4242", aad)
        print("ciphertext+tag bytes:", len(ct))       # plaintext length + 16-byte tag
        print("decrypted:", aead.decrypt(nonce, ct, aad))
        try:
            aead.decrypt(nonce, ct, b"user_id=43;table=payments")   # wrong context
        except InvalidTag:
            print("AAD mismatch -> InvalidTag (ciphertext cannot be moved to another
row)")
        tampered = bytearray(ct); tampered[0] ^= 1
        try:
            aead.decrypt(nonce, bytes(tampered), aad)
        except InvalidTag:
            print("bit flip -> InvalidTag")

        # --- 2. Digital signatures: Ed25519 ---
        signing_key = Ed25519PrivateKey.generate()
        verify_key = signing_key.public_key()
        release = b"release v2.4.1 sha256=9f86d08..."
        sig = signing_key.sign(release)
        print("signature bytes:", len(sig))
        verify_key.verify(sig, release)               # raises if invalid
        try:
            verify_key.verify(sig, release + b"!")
        except InvalidSignature:
            print("modified message -> InvalidSignature")

        # --- 3. Key agreement: X25519 + HKDF ---
        alice, bob = X25519PrivateKey.generate(), X25519PrivateKey.generate()
        shared_a = alice.exchange(bob.public_key())  # only public keys cross the net-
work
        shared_b = bob.exchange(alice.public_key())
        assert shared_a == shared_b
        session_key = HKDF(algorithm=hashes.SHA256(), length=32, salt=None,
                           info=b"example-protocol v1 session key").derive(shared_a)
        print("both sides derived the same 256-bit session key:", session_key.hex()
[:16], "...")
```

Tested with `cryptography` 50.0.1: decryption succeeds with the right AAD; a changed AAD or a single flipped ciphertext bit raises `InvalidTag`; a modified release string raises `InvalidSignature`; both X25519 parties derive the same key.

What to notice:

- The 16-byte plaintext became 32 bytes: GCM ciphertext is the same length as the plaintext plus a 16-byte tag. The 12-byte nonce must be stored alongside it (it is not secret).
- Decryption fails *loudly* on any tampering or context mismatch; there is no way to obtain partially decrypted garbage.
- The raw X25519 output is never used directly as a key; HKDF with a protocol-specific `info` string turns it into a uniformly random key bound to its purpose.

Table 10.6 — Hash vs MAC vs signature vs encryption.

	Hash	MAC (HMAC)	Digital signature	AEAD encryption
Confidentiality	No	No	No	Yes
Integrity	Only against a trusted digest	Yes	Yes	Yes
Authenticity	No	Yes, among key holders	Yes, publicly verifiable	Yes, among key holders
Non-repudiation	No	No	Yes	No
Key	None	Shared secret	Private / public pair	Shared secret
Speed	GB/s	GB/s	~10^4–10^5 signatures/s per core (Ed25519)	GB/s with AES-NI
Typical use	Checksums, content addressing, Merkle trees	Webhooks, API request signing, cookies	Software updates, certificates, JWT (RS/ES), audit anchors	Data at rest and in transit

Common misconceptions

COMMON MISTAKE

"Encoding, hashing and encryption are interchangeable." Base64 is an *encoding*: reversible by anyone, provides no security. A hash is one-way and keyless: anyone can compute it, so it cannot keep a low-entropy value (a phone number, a password) secret — an attacker hashes every candidate. Encryption is reversible *with the key*. Kubernetes Secrets stored as base64 in etcd without encryption at rest are, from a security standpoint, plaintext.

10.5 PKI and certificates

DEFINITION

Definition — Public key infrastructure (PKI). The set of roles, policies and systems that bind public keys to identities. A **certificate** (X.509) is a document stating "this public key belongs to this name (for example `api.example.com`), valid from date A to date B", signed by a **certificate authority (CA)**. A client trusts a certificate if it can build a **chain** of valid signatures from it to a **root CA** certificate in its local **trust store**.

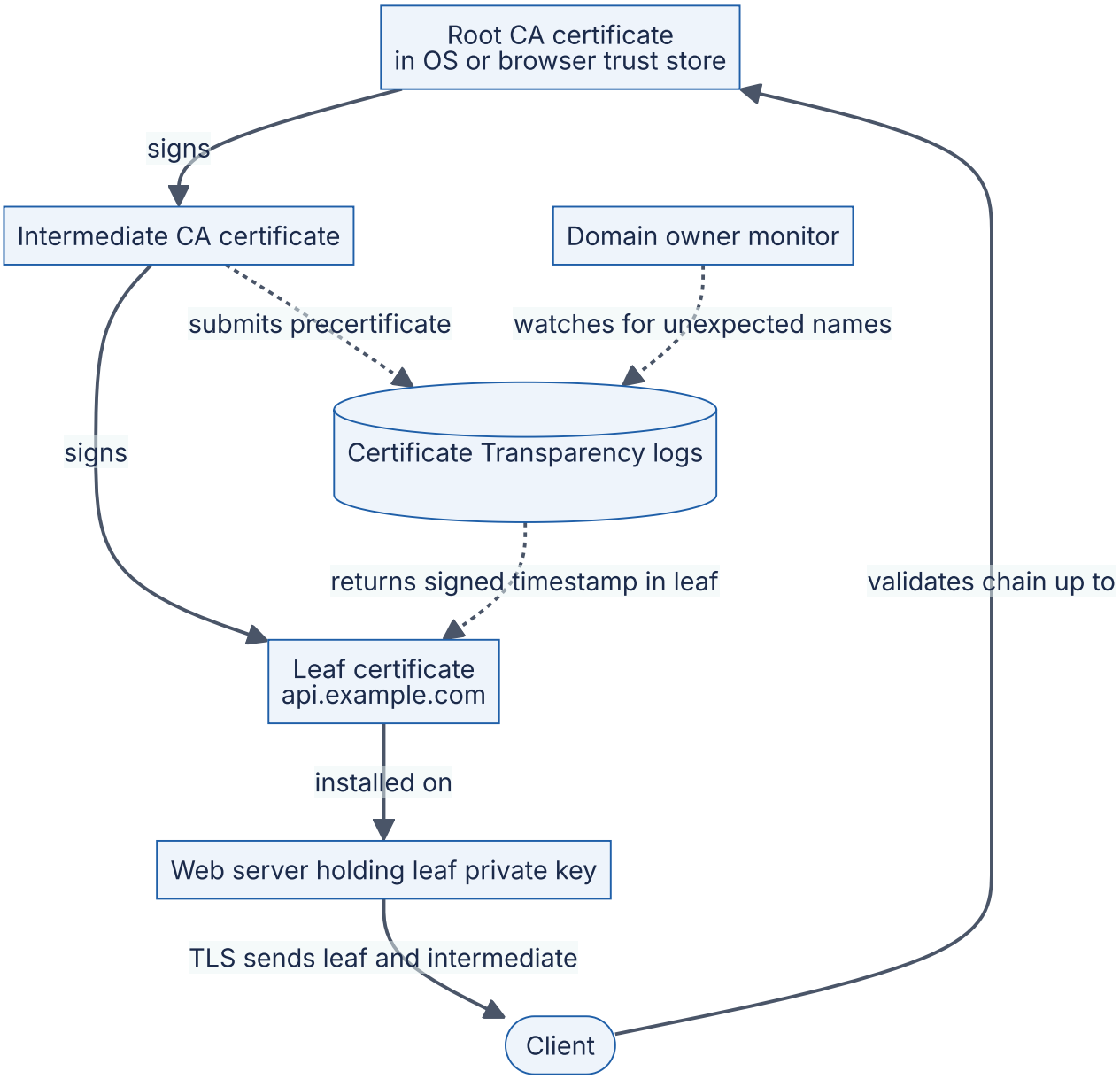
First-principles explanation

Asymmetric cryptography lets anyone lock a box with your padlock — but how does a stranger know the padlock in the bowl is *yours* and not one an impostor swapped in? A passport solves the same problem for people: a government you already trust vouches for the link between a photo and a name. A CA is that government for public keys. Your browser or operating system ships with roughly a hundred-odd root CA certificates (the exact number varies by vendor), and it trusts any certificate that chains back to one of them.

How it works internally

1. The server operator generates a key pair and sends the CA a **certificate signing request** containing the public key and the domain name.
2. The CA verifies control of the domain — for automated CAs, via the **ACME** protocol (RFC 8555): placing a token at a well-known HTTP path or in a DNS TXT record.
3. The CA signs a certificate using an **intermediate CA** key. Root keys are kept offline in hardware security modules and only sign intermediates, so a root key is used a few times per decade.
4. In the TLS handshake the server sends its **leaf** certificate plus intermediates. The client checks each signature up to a trusted root, the validity dates, the domain name, key-usage constraints and revocation status.

Figure 10.4 — Certificate chain of trust and Certificate Transparency.



Reading the diagram. Solid arrows show the signing hierarchy and the synchronous TLS path: the root signs the intermediate, the intermediate signs the leaf, and the server presents leaf plus intermediate to the client, which already holds the root. Dotted arrows show the asynchronous transparency path: before issuance the CA submits a precertificate to public append-only logs and embeds the logs' signed timestamps in the final certificate; browsers that enforce CT reject publicly trusted certificates without them. A domain owner's monitor watches the logs and alerts on any certificate for its names it did not request. Failure points: an expired leaf or intermediate (the most common certificate outage), a server that forgets to send the intermediate (works in some clients, fails in others), and a compromised or misbehaving CA — the reason CT exists.

Revocation — withdrawing a certificate before expiry because its key leaked — works poorly in practice: revocation lists grow large, online status checks (OCSP) add latency and leak browsing history, and clients usually "soft-fail" when the responder is unreach-

able. The industry's answer is shorter lifetimes. Let's Encrypt has issued 90-day certificates since 2015 to force automation, and in 2025 the CA/Browser Forum voted to shrink the maximum lifetime of publicly trusted TLS certificates in stages, reaching 47 days by 2029 (as of 2026; check the current Baseline Requirements).

DEFINITION

Definition — Certificate Transparency (CT). A system of public, append-only, Merkle-tree-based logs into which CAs record every certificate they issue, so that mis-issued certificates become publicly detectable [RFC 6962].

INDUSTRY EXAMPLE

DigiNotar (2011). Attackers compromised the Dutch CA DigiNotar and issued hundreds of fraudulent certificates, including one for `*.google.com` that was used to intercept traffic of users in Iran. Browsers removed DigiNotar from their trust stores and the company went bankrupt within weeks. The incident directly motivated Certificate Transparency: had CT existed, Google's monitors would have seen the rogue certificate within hours of issuance. *Evidence: Secondary* — [Fox-IT 2012]

Certificate rotation for internal services

For service-to-service mTLS (§10.10) you run a private CA (cloud private CA services, HashiCorp Vault's PKI engine, or a service mesh's built-in CA). The design rules: issue **short-lived** certificates (hours to days); rotate automatically well before expiry (renew at about two-thirds of lifetime); have services reload certificates without restarting; alert on any certificate with less than, say, 25% of its lifetime remaining; and treat the CA's private key as the most sensitive key in the company (store it in an HSM or KMS).

Table 10.7 — Certificate failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Expired certificate outage	Manual renewal forgotten; long lifetime meant the process was never exercised	Synthetic TLS probes; expiry metrics	Emergency renewal and deploy	Automated issuance (ACME), short lifetimes, expiry alerts
Private key leak	Key in a repo, image or backup	Secret scanning; CT monitor shows unexpected certs	Revoke, reissue, rotate	Keys generated on host or in KMS/HSM, never copied
Rogue certificate for your domain	Compromised or careless CA	CT log monitoring	Request revocation; report to browser vendors	CAA DNS records restricting which CAs may issue

10.6 Password storage

10.6.1 Why passwords need special treatment

First-principles explanation

Suppose you run a coat check. You could keep each customer's ticket number in a ledger, but if the ledger is stolen, the thief can claim every coat. Better: keep only a "fingerprint" of each ticket, so a stolen ledger is useless — *unless* the tickets are so predictable that the thief can fingerprint every possible ticket and match them. Human passwords are predictable tickets: "Summer2024!" and its millions of cousins appear in every attacker's wordlist.

The design goal therefore is: **assume the password database will one day be stolen** (via SQL injection, a leaked backup or an insider), and make each password as expensive as possible to recover from what was stolen. Three techniques combine to achieve this.

1. **Hash, don't encrypt.** The server never needs the password back; it only needs to check a candidate. A one-way function means there is no decryption key to steal.
2. **Salt.** A **salt** is a unique random value (at least 16 bytes) stored next to each hash and mixed into it. Without salts, identical passwords produce identical hashes, so one computation cracks every user with "123456", and attackers can use precomputed **rainbow tables**. With salts, every guess must be computed separately per user.
3. **Slow, memory-hard hashing.** SHA-256 is designed to be fast; a single high-end GPU computes on the order of 10^{10} SHA-256 hashes per second (order-of-magnitude figure as of 2024). A **password-hashing function** (bcrypt, scrypt, Argon2id) is deliberately slow and, for scrypt and Argon2, **memory-hard**: each guess needs tens of megabytes of RAM, which defeats the massive parallelism of GPUs and custom chips.

Optionally add a **pepper**: a secret key, stored outside the database (in a KMS or HSM), applied as an HMAC before or after hashing. A database-only leak then yields hashes that cannot be attacked at all without also stealing the pepper. NIST SP 800-63B recommends this additional keyed step.

INDUSTRY EXAMPLE

LinkedIn (2012). About 6.5 million LinkedIn password hashes were posted publicly in 2012; they were unsalted SHA-1 hashes, and a large fraction were cracked within days. In 2016 a much larger dataset from the same breach (reported at over 100 million accounts) surfaced. The lesson is both technical (fast, unsalted hashes) and architectural (credential reuse turns one site's breach into credential stuffing against every other site — threat T1 in Table 10.2). *Evidence: Secondary* — widely reported in the technology press, 2012 and 2016.

10.6.2 Algorithms and cost parameters

Table 10.8 — Password-hashing functions compared (parameters per [OWASP Password Storage Cheat Sheet], as of 2026).

	Argon2id	scrypt	bcrypt	PBKDF2-HMAC-SHA-256
What it is	Winner of the 2015 Password Hashing Competition, RFC 9106	Memory-hard KDF, RFC 7914	Blowfish-based adaptive hash (1999)	Iterated HMAC, NIST-approved
Cost knobs	memory m, iterations t, parallelism p	N (CPU/memory), r (block size), p	work factor (cost c ⇒ 2^c rounds)	iteration count
Memory-hard	Yes	Yes	No (4 KiB)	No
OWASP minimum	m = 19 MiB, t = 2, p = 1	$N = 2^{17}$, r = 8, p = 1	cost ≥ 10	≥ 600,000 iterations
Gotchas	Library support varies	Memory = $128 \cdot N \cdot r$ bytes (128 MiB at OWASP minimum)	Truncates input at 72 bytes	GPU-friendly; use only when FIPS-140 is required
Choose when	Default for new systems	Argon2id unavailable	Legacy systems, wide library support	Compliance mandates FIPS

RFC 9106 also gives two general-purpose Argon2id recommendations: $t = 1$, $p = 4$, $m = 2$ GiB where memory allows, or $t = 3$, $p = 4$, $m = 64$ MiB otherwise [RFC 9106]. The OWASP minimums are lower because login servers must handle many concurrent hashes.

Choosing cost parameters is a capacity-planning decision. Pick the most expensive setting that keeps a single hash at roughly 50–500 ms on your production hardware and that your login fleet can afford at peak. If peak login rate is 200 per second and one hash costs 100 ms of one core and 64 MiB, you need about $200 \times 0.1 = 20$ cores and 20×64 MiB ≈ 1.3 GiB of RAM dedicated to hashing — plus headroom, because an attacker can make login attempts to burn your CPU (rate-limit the endpoint). Store the algorithm and parameters with each hash so you can raise them later and **rehash on next successful login**.

Mathematical foundations

FORMULA

Expected offline brute-force time

$$T = \frac{S}{2 \cdot R \cdot G}$$

Symbol	Meaning	Units
T	Expected time to find one password (average: half the space)	seconds
S	Size of the search space: N^L for random passwords, or the size of the attacker's ranked wordlist up to the target	candidates
R	Guesses per second per attacking device for this hash function and cost	guesses/s/device
G	Number of attacking devices (GPUs)	devices

Salting means T is paid **per user**; without salts, one pass covers all users.

WORKED EXAMPLE

Worked example — Fast hash vs slow hash. A user picks a random 8-character password from the 95 printable ASCII characters.

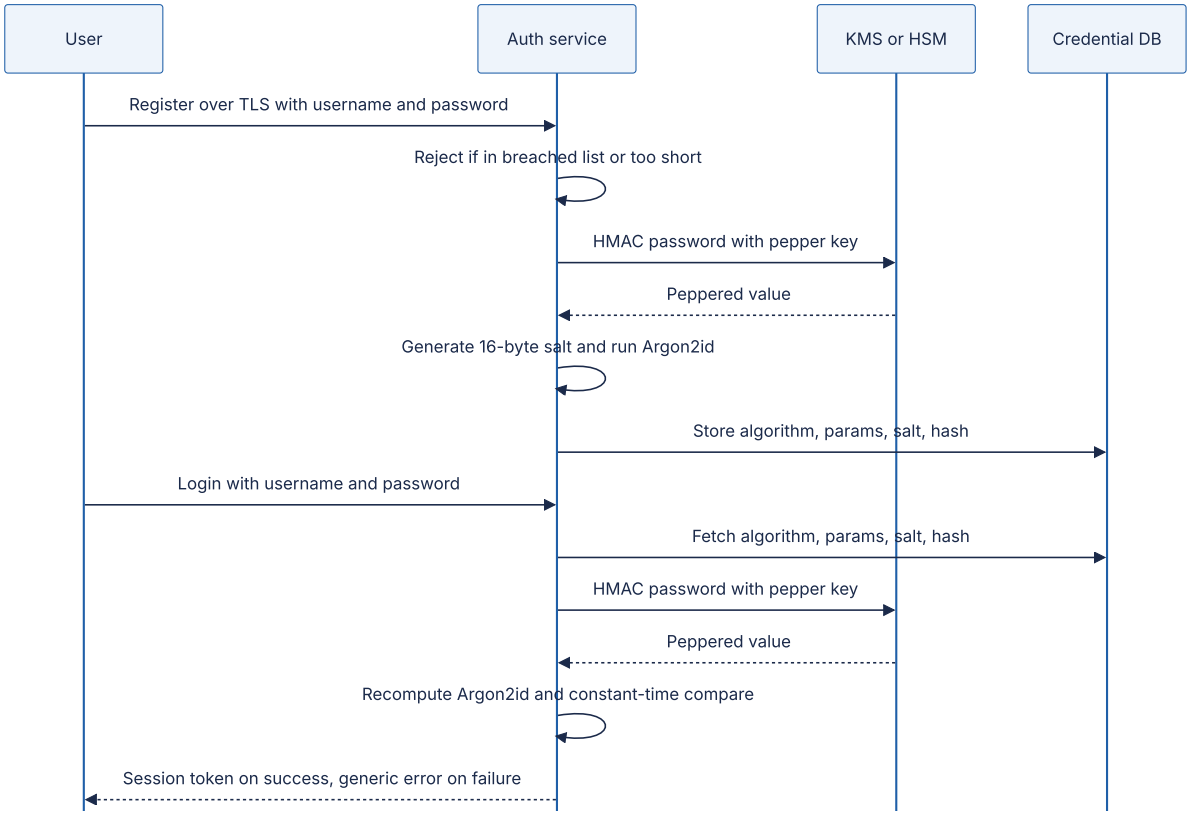
1. Search space: $S = 95^8 \approx 6.63 \times 10^{15}$ (entropy $8 \log_2 95 \approx 52.6$ bits).
2. *Salted SHA-256,* $R \approx 10^{10}$ /s, $G = 10$ GPUs: $T = 6.63 \times 10^{15} / (2 \times 10^{11}) \approx 3.3 \times 10^4$ s $\approx$ **9 hours**.
3. *Argon2id or bcrypt tuned to ~100 ms on a server core*. An attacker's GPU is far less advantaged on memory-hard functions; assume an optimistic $R \approx 10^4$ /s per device: $T = 6.63 \times 10^{15} / (2 \times 10^5) \approx 3.3 \times 10^{10}$ s $\approx$ **1,050 years** per password.
4. A 12-character random password ($S = 95^{12} \approx 5.4 \times 10^{23}$) against the same slow hash: $\approx 2.7 \times 10^{18}$ s, about 8.6×10^{10} years.

Interpretation: the slow hash multiplies the attacker's cost by 10^6; length multiplies it exponentially. *But* human passwords are not random: "Summer2024!" appears near the top of real wordlists, so S for it is perhaps 10^6, and even at 10^4 guesses/s it falls in under a minute. *Engineering implication:* slow hashing protects the middle of the distribution; protecting the weakest passwords requires refusing them at registration (breached-password lists) and adding MFA.

NIST guidance. NIST SP 800-63B, revised as SP 800-63B-4 (final in 2025), reverses several folk rules: do **not** impose composition rules (“one uppercase, one symbol”), do **not** force periodic password changes without evidence of compromise, **do** check new passwords against lists of breached and commonly used passwords, allow at least 64 characters and all printable characters including spaces, and require a minimum length — 15 characters when the password is the only authentication factor, 8 when it is part of multi-factor authentication [NIST SP 800-63B-4].

Architecture diagram

Figure 10.5 — Registration and login with salted Argon2id and a pepper.



Reading the diagram. All calls are synchronous. The password exists in plaintext only in memory of the auth service, briefly. The pepper key never leaves the KMS or HSM; the service sends it data to MAC (some designs instead load the pepper into service memory at startup, trading isolation for one fewer network call per login). The credential database stores only the self-describing record — for example `argon2id$m=19456,t=2,p=1$<salt>$<hash>` — so a stolen database alone is not enough to start guessing. The failure response is deliberately generic (“invalid username or password”) and takes the same time for unknown users to avoid revealing which accounts exist. Scaling: the Argon2id step dominates CPU and memory, so login capacity scales with auth-service replicas; rate limiting per account and per IP sits in front of it.

Code example

```
        """Password hashing with scrypt (standard library) and Argon2id (cryptography >=
44)."""
        import base64
        import hashlib
        import hmac
        import os
        import time

        # ---------- scrypt (stdlib) ----------
        SCRYPT_N, SCRYPT_R, SCRYPT_P = 2**17, 8, 1   # OWASP minimum: N=2^17, r=8, p=1
        SCRYPT_MAXMEM = 256 * 1024 * 1024            # memory needed is ~128*N*r = 128
MiB

        def b64(b: bytes) -> str:
            return base64.b64encode(b).decode().rstrip("=")

        def unb64(s: str) -> bytes:
            return base64.b64decode(s + "=" * (-len(s) % 4))

        def scrypt_hash(password: str) -> str:
            salt = os.urandom(16)                                  # unique per pass-
word
            dk = hashlib.scrypt(password.encode("utf-8"), salt=salt,
                                n=SCRYPT_N, r=SCRYPT_R, p=SCRYPT_P,
                                maxmem=SCRYPT_MAXMEM, dklen=32)
            # Store algorithm, parameters, salt and hash together so they can evolve.
            return f"scrypt${SCRYPT_N}${SCRYPT_R}${SCRYPT_P}${b64(salt)}${b64(dk)}"

        def scrypt_verify(password: str, stored: str) -> bool:
            alg, n, r, p, salt_s, dk_s = stored.split("$")
            if alg != "scrypt":
                raise ValueError("unknown algorithm")
            n, r, p = int(n), int(r), int(p)
            candidate = hashlib.scrypt(password.encode("utf-8"), salt=unb64(salt_s),
                                       n=n, r=r, p=p,
                                       maxmem=max(SCRYPT_MAXMEM, 256 * n * r), dklen=32)
            return hmac.compare_digest(candidate, unb64(dk_s))      # constant-time com-
pare

        def needs_rehash(stored: str) -> bool:
            _, n, r, p, *_ = stored.split("$")
            return (int(n), int(r), int(p)) != (SCRYPT_N, SCRYPT_R, SCRYPT_P)

        # ---------- Argon2id (cryptography library) ----------
        from cryptography.hazmat.primitives.kdf.argon2 import Argon2id
        from cryptography.exceptions import InvalidKey

        ARGON_T, ARGON_M_KIB, ARGON_P = 2, 19 * 1024, 1   # OWASP minimum: m=19 MiB,
t=2, p=1

        def argon2id_hash(password: str) -> str:
            salt = os.urandom(16)
            kdf = Argon2id(salt=salt, length=32, iterations=ARGON_T,
                           lanes=ARGON_P, memory_cost=ARGON_M_KIB)
```

```
            tag = kdf.derive(password.encode("utf-8"))
            return f"argon2id${ARGON_M_KIB}${ARGON_T}${ARGON_P}${b64(salt)}${b64(tag)}"

        def argon2id_verify(password: str, stored: str) -> bool:
            alg, m, t, p, salt_s, tag_s = stored.split("$")
            if alg != "argon2id":
                raise ValueError("unknown algorithm")
            kdf = Argon2id(salt=unb64(salt_s), length=32, iterations=int(t),
                           lanes=int(p), memory_cost=int(m))
            try:
                kdf.verify(password.encode("utf-8"), unb64(tag_s))  # constant-time in-
side
                return True
            except InvalidKey:
                return False
```

Tested with Python 3.13 and `cryptography` 50.0.1 on a cloud VM: one scrypt hash at OWASP-minimum settings took about 400 ms, one Argon2id hash about 33 ms; correct passwords verify, a one-letter change fails, and hashing the same password twice gives different strings.

What to notice:

- `hashlib.scrypt` refuses to use more than 32 MiB by default; `maxmem` must be raised for $N = 2^{17}$, $r = 8$ (128 MiB).
- The stored string is self-describing (algorithm, parameters, salt, hash), which is what makes `needs_rehash` and gradual parameter upgrades possible. Production code should prefer the standard PHC string format produced by maintained libraries such as `argon2-cffi`.
- ~33 ms for Argon2id at OWASP-minimum settings is a floor; raise m or t until the cost reaches your latency and capacity budget.

> COMMON MISTAKE
>
> **“Encrypting passwords is safer than hashing them.”** Encryption is reversible, so whoever obtains the key — an attacker with application-server access, a rogue administrator — recovers every password at once. Systems should never need to recover a user’s password; if a product requirement says “email users their password”, the requirement is wrong.

10.7 The OWASP Top 10 in architectural terms

DEFINITION

Definition — OWASP Top 10. A consensus ranking of the most critical web application security risk categories, published by the Open Worldwide Application Security Project from vulnerability data contributed by testing organizations plus a community survey. This section uses the **2021 edition** [OWASP 2021]. A 2025 edition has since been published; it keeps broken access control at #1, folds SSRF into it, and adds categories for software supply-chain failures and mishandling of exceptional conditions, so the architectural controls below remain applicable.

The Top 10 is often taught as a list of coding bugs. For a system designer, each category is better read as a *missing architectural control*: something that should exist once, centrally, rather than being re-implemented correctly by every developer on every endpoint.

Table 10.9 — OWASP Top 10 (2021) mapped to architectural controls.

ID	Category	What goes wrong	Architectural control
A01	Broken Access Control	Users act outside their permissions: IDOR, missing function-level checks, CORS misconfiguration	Deny by default; central policy engine or authorization middleware; object-level checks in the data-access layer; tenant ID derived from the token, never from the request body
A02	Cryptographic Failures	Sensitive data unencrypted or protected with weak/misused crypto	TLS everywhere; KMS-managed keys; AEAD; data classification deciding what is encrypted (§10.9)
A03	Injection	Untrusted data interpreted as code: SQL, NoSQL, OS command, LDAP; cross-site scripting is included here in 2021	Parameterized queries/ORMs; no shell invocation with user input; context-aware output encoding; Content Security Policy
A04	Insecure Design	Missing or ineffective security controls by design, not by implementation	Threat modeling (§10.3); abuse-case requirements; rate limits and business-logic limits designed in
A05	Security Misconfiguration	Default credentials, verbose errors, open buckets, unnecessary features enabled	Infrastructure as code with policy checks; hardened base images; configuration drift detection
A06	Vulnerable and Outdated Components	Known-vulnerable libraries, frameworks, OS packages	Software bill of materials (SBOM); dependency scanning in CI; patch SLAs; automated update PRs
A07	Identification and Authentication Failures	Credential stuffing, weak session handling, missing MFA	Central identity provider; MFA; rate limiting; secure session tokens (§10.4.7)
A08	Software and Data Integrity Failures	Unsigned updates, compromised CI/CD, insecure deserialization	Signed artifacts and provenance; protected build pipeline; no deserialization of untrusted native objects
A09	Security Logging and Monitoring Failures	Breaches go undetected for months	Central, tamper-evident audit logs (§10.12); alerting on security events
A10	Server-Side Request Forgery (SSRF)	Server fetches an attacker-chosen URL, reaching internal services	Egress allow-lists via a dedicated fetch proxy; block metadata and private ranges; IMDSv2-style token-protected metadata

10.7.1 Broken access control (A01)

Broken access control is #1 because authorization is the one control that cannot be bolted on by infrastructure: only the application knows that photo 9001 belongs to user 42. The classic form is the **insecure direct object reference (IDOR)**: `GET /api/photos/9001` returns the photo to anyone authenticated, because the handler checks *that* the user is logged in but not *whether* they own photo 9001.

Architectural patterns that make the correct behavior the default:

- **Deny by default.** A new endpoint without an explicit policy rejects all requests.
- **Centralize decisions.** Route every authorization question through one library or service (a policy engine such as Open Policy Agent, or a relationship-based system modeled on Google's Zanzibar — §8.12), so the rules are written and reviewed once.
- **Push checks to the data layer.** Repository methods take the caller's identity and add `WHERE owner_id = :caller` (or tenant filters) automatically; database row-level security can enforce the same rule underneath.
- **Use unguessable identifiers as defense in depth, not as the control.** Random UUIDs make enumeration harder but are not authorization; IDs leak through URLs, logs and shares.

10.7.2 Injection (A03)

Injection happens when data and code share a channel. A query built as `"SELECT * FROM users WHERE name = '" + name + "'"` lets an attacker supply `' OR '1'='1` and change the query's *structure*. **Parameterized queries** separate the channels: the query text is sent to the database with placeholders, and values are sent separately, so a value can never become syntax.

```
        import sqlite3
        db = sqlite3.connect(":memory:")
        db.execute("CREATE TABLE users (name TEXT, email TEXT)")
        db.execute("INSERT INTO users VALUES ('alice', 'a@example.com')")
        attacker_input = "x' OR '1'='1"
        # Vulnerable (illustrative, never do this): f"... WHERE name =
'{attacker_input}'"
        rows = db.execute("SELECT email FROM users WHERE name = ?",
(attacker_input,)).fetchall()
        print(rows)   # [] -- the input is treated purely as a value
```

What to notice: the `?` placeholder is bound by the driver; no escaping function is needed or involved. The architectural layer of defense is to give the application's database user only the rights it needs (no `DROP`, no access to other schemas), so a missed injection cannot become a full compromise.

10.7.3 Server-side request forgery (A10)

Many features make the server fetch a URL the user supplied: link previews, webhooks, “import from URL”, PDF rendering. If the server can reach internal addresses, the attacker can make it fetch `http://169.254.169.254/` (the cloud instance-metadata endpoint that hands out the instance’s IAM credentials), `http://localhost:8080/admin` or internal databases’ HTTP interfaces.

> INDUSTRY EXAMPLE
>
> **Capital One (2019).** An attacker exploited a misconfigured web application firewall to make requests to the AWS instance-metadata service, obtained temporary credentials for the instance’s IAM role, and used that role’s overly broad permissions to copy data on roughly 100 million credit-card applicants from S3. Two architectural failures compounded: an SSRF path to metadata, and a role with far more S3 access than the WAF needed. Later that year AWS introduced IMDSv2, which requires a session token obtained via an HTTP `PUT`, defeating most simple SSRF requests. *Evidence: Secondary* — public court filings and widespread reporting; [AWS Docs: IMDSv2].

Defense in depth for SSRF: (1) route all user-driven outbound fetches through a dedicated **egress proxy** in its own network segment with no route to internal ranges; (2) validate URLs (scheme, port, resolved IP) and **pin** the validated IP for the connection, otherwise **DNS rebinding** — a hostname that resolves to a public IP at check time and a private IP at fetch time — bypasses the check; (3) disable redirects or re-validate each hop; (4) require IMDSv2 and give instance roles least privilege.

```
        """Defense-in-depth check for user-supplied URLs fetched by a server (SSRF)."""
        import ipaddress
        import socket
        from urllib.parse import urlsplit

        ALLOWED_SCHEMES = {"https"}
        ALLOWED_PORTS = {443}

        def resolve_and_check(url: str) -> list[str]:
            parts = urlsplit(url)
            if parts.scheme not in ALLOWED_SCHEMES:
                raise ValueError(f"scheme {parts.scheme!r} not allowed")
            port = parts.port or 443
            if port not in ALLOWED_PORTS or not parts.hostname:
                raise ValueError("port or host not allowed")
            infos = socket.getaddrinfo(parts.hostname, port, proto=socket.IPPROTO_TCP)
            addrs = sorted({info[4][0] for info in infos})
            for a in addrs:
                ip = ipaddress.ip_address(a)
                if not ip.is_global:   # rejects private, loopback, link-local
(169.254.169.254), etc.
                    raise ValueError(f"{parts.hostname} resolves to non-public {ip}")
            # Caller must connect to one of THESE addresses (pin them) to avoid DNS re-
binding.
            return addrs
```

Tested results: `http://example.com/` is denied (scheme), `https://127.0.0.1/admin`, `https://169.254.169.254/latest/meta-data/`, `https://10.0.0.5:443/` and `https://[::1]/` are denied (non-public address), and `https://localhost:8443/` is denied (port). What to notice: the check uses *every* resolved address, not just the first, and `ip.is_global` covers private, loopback, link-local and other reserved ranges for both IPv4 and IPv6. Application-level validation is the inner layer; the network-level egress proxy is the layer that still holds when a developer forgets to call this function.

> COMMON MISTAKE
>
> **“A WAF fixes the OWASP Top 10.”** A web application firewall blocks known attack *patterns* at the edge and buys time for patching. It cannot know who owns photo 9001 (A01), cannot fix insecure design (A04), and can itself be a component with an SSRF flaw, as the Capital One case shows. Treat the WAF as one layer, not the control.

10.8 Secrets management

DEFINITION

Definition — Secret. Any value that grants access and must be kept confidential: database passwords, API keys, OAuth client secrets, TLS and signing private keys, encryption keys, webhook HMAC keys. **Secrets management** is the practice of storing, distributing, rotating, auditing and revoking secrets so that they are available to the workloads that need them and to nobody else.

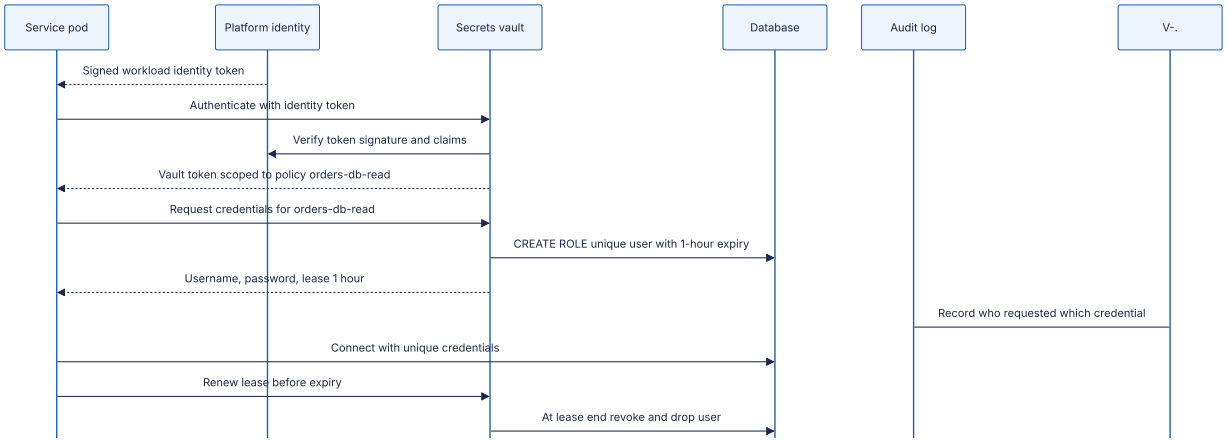
First-principles explanation

In an office, physical keys drift: copies are cut for contractors, left in drawers, never returned. Eventually nobody knows who can open the server room. Software secrets drift the same way and faster: a database password gets pasted into a configuration file, committed to Git, baked into a container image, printed in a debug log, copied into a CI variable and a teammate's laptop. Each copy is a place it can leak from, and a secret that has existed unchanged for three years has had three years to leak.

The progression of maturity is:

1. **Hard-coded** in source — anyone with repository access has production access, forever (Git history keeps it).
2. **Environment variables / config files** injected at deploy time — out of source, but static, widely copied, visible in crash dumps.
3. **Central vault** — a secrets manager (HashiCorp Vault, AWS Secrets Manager, Google Secret Manager, Azure Key Vault) stores secrets encrypted, releases them only to authenticated workloads, logs every read and supports rotation.
4. **Dynamic, short-lived credentials** — the vault *creates* a unique credential per workload on demand with a lease (say one hour) and revokes it at expiry.
5. **Identity-based ("secretless")** — workloads authenticate with a platform-attested identity (cloud IAM role, Kubernetes service-account token, SPIFFE ID) and receive short-lived tokens; there is no long-lived secret to leak.

Figure 10.6 — A workload obtaining dynamic database credentials from a vault.



Reading the diagram. The pod starts with no secret at all: the platform (for example Kubernetes or a cloud provider) gives it a signed identity token describing which service it is. The vault verifies that token and maps the identity to a policy. Instead of returning a shared password, the vault connects to the database with its own administrative credential and creates a unique, time-limited database user. The workload receives credentials it alone holds; the vault records the issuance asynchronously in an audit log (dotted arrow). Leases are renewed while the pod lives; when the pod dies the lease expires and the vault drops the user. Failure points: the vault is now a critical dependency — if it is unavailable, new pods cannot get credentials, so vaults are run as replicated clusters (Vault uses Raft consensus, §26.7) and clients cache credentials for their lease. Scaling: one credential per pod means the database must tolerate many distinct users; for very large fleets, issue per-service rather than per-pod credentials.

FORMULA

Credential misuse window

$$W = \begin{cases} T_{\text{detect}} + T_{\text{rotate}} & \text{long-lived credential} \\ \min\left(T_{\text{TTL}},\ T_{\text{detect}} + T_{\text{revoke}}\right) & \text{short-lived credential} \end{cases}$$

Symbol	Meaning	Units
W	Time during which a leaked credential remains usable	hours
T_{detect}	Time from leak to detection	hours
$T_{\text{rotate}}, T_{\text{revoke}}$	Time to rotate or revoke after detection	hours
T_{TTL}	Lifetime of a short-lived credential	hours

WORKED EXAMPLE

Worked example — Static key vs one-hour credential. Industry breach studies commonly report detection times measured in months; assume $T_{\text{detect}} = 200$ days $= 4,800$ h and $T_{\text{rotate}} = 24$ h. A static API key leaked to a public repository is usable for $W = 4,824$ h. A one-hour dynamic credential leaked in the same way is usable for at most $W = \min(1, 4,824) = 1$ h. *Interpretation:* short lifetimes shrink the window by more than three orders of magnitude *without* requiring detection to improve. *Engineering implication:* invest first in making credentials short-lived and automatically renewed; detection and response remain necessary for the vault and identity provider themselves.

Rotation must be designed, not improvised. A safe rotation for a static secret uses **two valid versions**: (1) create the new secret and register it with the receiving side, which now accepts both; (2) roll out the new secret to all callers; (3) verify no traffic uses the old one (metrics per key ID); (4) revoke the old one. This is why key IDs appear in the HMAC design of Figure 10.3. Complement rotation with **secret scanning** in repositories and CI, pre-commit hooks, and an incident runbook that answers "this key leaked — what do we do in the next 15 minutes?"

WARNING / TRADE-OFF

Kubernetes Secrets are not encrypted by default. They are base64-encoded objects in etcd; anyone with etcd access or broad RBAC `get secrets` permission can read them. Enable encryption at rest for etcd with a KMS provider, restrict RBAC, and prefer mounting secrets from an external vault (§42.11).

10.9 Encryption at rest and in transit, KMS and envelope encryption

10.9.1 In transit and at rest

Encryption in transit protects data moving over a network from eavesdropping and tampering. The rule is simple: TLS 1.2 or later (prefer 1.3) on *every* hop, including internal ones — between load balancer and service, service and database, service and cache. "The internal network is trusted" is exactly the perimeter assumption that zero trust (§10.10) rejects.

Encryption at rest protects stored data from someone who obtains the storage medium or a copy of it. It is applied at different layers, and the layer determines *which* attacker it stops.

Table 10.10 — Layers of encryption at rest and what each protects against.

Layer	Example	Protects against	Does **not** protect against	Cost
Full-disk / volume	Cloud volume encryption, LUKS	Stolen disks, discarded hardware	Anyone who can query the running system	Near-zero; usually default
Storage service	S3 SSE-KMS, managed DB storage encryption	Above, plus snapshots/backups copied without key permission	Application or DB credentials compromise	Low; KMS call costs
Database (TDE)	Transparent data encryption	Stolen DB files and backups	SQL injection, compromised DB user	Low
Application / field-level	App encrypts SSN column with a KMS-managed key before writing	DB admins, SQL injection reading raw columns, backup leaks	Compromise of the application with decrypt permission	Higher: no querying on ciphertext, key plumbing
Client-side / end-to-end	Messaging apps, client-encrypted backups	The service operator itself	Compromised client device	Highest: server cannot read or index data

The key insight: lower layers are cheap and defeat *physical* and *backup* threats; higher layers are expensive and defeat *logical* threats from people and processes with access to the running system. Most designs use volume or service encryption everywhere, and field-level encryption for a small set of highly sensitive fields (government IDs, health data, payment details).

10.9.2 Key management services and envelope encryption

DEFINITION

Definition — Key management service (KMS). A managed service, backed by hardware security modules (HSMs), that generates and stores **key-encryption keys (KEKs)** that never leave it in plaintext, performs cryptographic operations with them on request, enforces access policies on every use and logs every call. Examples: AWS KMS, Google Cloud KMS, Azure Key Vault, HashiCorp Vault's Transit engine.

DEFINITION

Definition — Envelope encryption. Encrypting data with a **data-encryption key (DEK)** generated for that data, then encrypting ("wrapping") the DEK with a KEK held in a KMS, and storing the wrapped DEK alongside the ciphertext [AWS Docs: KMS concepts].

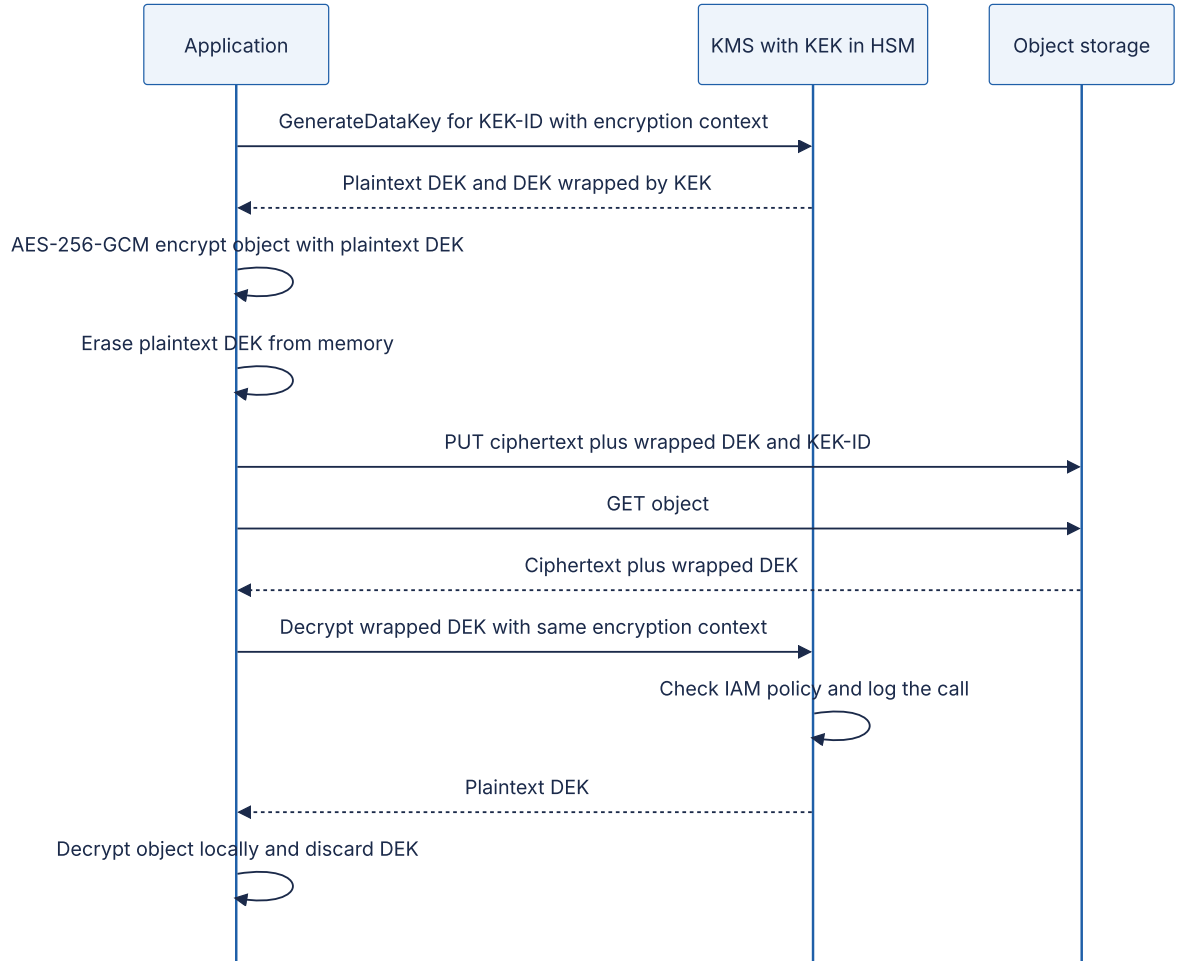
First-principles explanation

A hotel does not give each guest the master key. Each room has its own key (the DEK); the room keys are kept in a safe at the front desk, and only the manager has the key to the safe (the KEK in the KMS). Losing one room key affects one room; changing the safe's lock does not require changing every room's lock.

Why does this exist?

Three problems with sending all data to the KMS directly: (1) **size** — KMS encrypt APIs accept only small payloads (AWS KMS `Encrypt` takes at most 4,096 bytes of plaintext); (2) **latency and throughput** — every encryption would cost a network round trip and count against request quotas; (3) **blast radius** — one key encrypting everything means one leak exposes everything. Envelope encryption sends only 32-byte keys to the KMS and encrypts the bulk data locally at memory speed.

Figure 10.7 — Envelope encryption: write and read paths.



Reading the diagram. All calls are synchronous. On the write path the application asks the KMS for a fresh DEK; the KMS returns it twice — in plaintext for immediate use, and wrapped under the KEK for storage. The object is encrypted locally and stored with its

wrapped DEK, so the storage system holds only ciphertext. On the read path the application sends the small wrapped DEK to the KMS; the KMS enforces the access policy *at decrypt time* and writes an audit record, which means “who read this data” becomes visible in KMS logs even if storage access logging is off. The **encryption context** (non-secret key–value pairs such as tenant and object name) is bound to the wrapped DEK as associated data; a wrapped DEK copied onto another tenant’s object fails to unwrap. Failure points: KMS unavailability blocks all reads and writes of new objects (mitigate with regional KMS keys, retries and, carefully, short-lived DEK caching); a deleted KEK makes all data under it unrecoverable (KMS services enforce deletion waiting periods for this reason). Scaling: KMS calls grow with the number of objects, not their size.

Code example: envelope encryption with a simulated KMS

```
        """Envelope encryption with a simulated KMS (the KEK never leaves the 'KMS')."""
        import json
        import os
        from dataclasses import dataclass
        from cryptography.hazmat.primitives.ciphers.aead import AESGCM

        class FakeKMS:
            """Stands in for AWS KMS / Cloud KMS / Vault Transit. Holds KEKs inter-
nally."""
            def __init__(self):
                self._keks = {"kek-2026-10": AESGCM.generate_key(bit_length=256)}
                self.current = "kek-2026-10"

            def generate_data_key(self, context: dict) -> tuple[bytes, dict]:
                dek = AESGCM.generate_key(bit_length=256)            # fresh DEK per ob-
ject
                nonce = os.urandom(12)
                aad = json.dumps(context, sort_keys=True).encode()   # "encryption con-
text"
                wrapped = AESGCM(self._keks[self.current]).encrypt(nonce, dek, aad)
                return dek, {"kek_id": self.current, "nonce": nonce.hex(), "wrapped":
wrapped.hex()}

            def decrypt_data_key(self, blob: dict, context: dict) -> bytes:
                aad = json.dumps(context, sort_keys=True).encode()
                return AESGCM(self._keks[blob["kek_id"]]).decrypt(
                    bytes.fromhex(blob["nonce"]), bytes.fromhex(blob["wrapped"]), aad)

            def rotate(self) -> None:
                new_id = f"kek-{len(self._keks) + 1}"
                self._keks[new_id] = AESGCM.generate_key(bit_length=256)
                self.current = new_id                                 # old KEKs kept
for decrypt

            def rewrap(self, blob: dict, context: dict) -> dict:
                """Re-encrypt a DEK under the current KEK without touching the data."""
                dek = self.decrypt_data_key(blob, context)
                nonce = os.urandom(12)
                aad = json.dumps(context, sort_keys=True).encode()
                return {"kek_id": self.current, "nonce": nonce.hex(),
                        "wrapped": AESGCM(self._keks[self.current]).encrypt(nonce, dek,
aad).hex()}

        @dataclass
        class EncryptedObject:
            wrapped_dek: dict
            nonce: bytes
            ciphertext: bytes

        def encrypt_object(kms: FakeKMS, plaintext: bytes, context: dict) ->
EncryptedObject:
            dek, wrapped = kms.generate_data_key(context)             # 1 KMS call per
object
            nonce = os.urandom(12)
```

```
            ct = AESGCM(dek).encrypt(nonce, plaintext, None)          # bulk data en-
crypted locally
            del dek                                                   # drop plaintext
DEK ASAP
            return EncryptedObject(wrapped, nonce, ct)

        def decrypt_object(kms: FakeKMS, obj: EncryptedObject, context: dict) -> bytes:
            dek = kms.decrypt_data_key(obj.wrapped_dek, context)
            return AESGCM(dek).decrypt(obj.nonce, obj.ciphertext, None)

        if __name__ == "__main__":
            kms = FakeKMS()
            ctx = {"tenant": "acme", "object": "invoices/2026-10.pdf"}
            data = os.urandom(5_000_000)                              # 5 MB document
            obj = encrypt_object(kms, data, ctx)
            assert decrypt_object(kms, obj, ctx) == data
            print("round trip OK; wrapped DEK is",
len(bytes.fromhex(obj.wrapped_dek["wrapped"])), "bytes")

            kms.rotate()
            obj.wrapped_dek = kms.rewrap(obj.wrapped_dek, ctx)        # rotate KEK: re-
wrap ~60 bytes,
            assert decrypt_object(kms, obj, ctx) == data              # not re-encrypt 5
MB
            print("after KEK rotation, DEK now wrapped by", obj.wrapped_dek["kek_id"])

            try:
                decrypt_object(kms, obj, {"tenant": "evil", "object": "invoices/2026-
10.pdf"})
            except Exception as e:
                print("wrong encryption context ->", type(e).__name__)
```

Output (tested):

```
round trip OK; wrapped DEK is 48 bytes
after KEK rotation, DEK now wrapped by kek-2
wrong encryption context -> InvalidTag
```

What to notice:

- **Rotation is cheap.** Rotating the KEK means rewrapping a 48-byte wrapped DEK per object, not re-encrypting 5 MB of data. Managed KMS services go further: automatic rotation keeps old key versions for decryption, so existing wrapped DEKs remain valid without any rewrap.
- **Crypto-shredding.** Deleting a tenant's KEK (or an object's DEK) renders the data permanently unreadable, including every backup copy — a practical way to honor deletion requests in systems with immutable backups.
- **Context binding.** The encryption context is authenticated, so a confused-deputy bug that passes the wrong tenant fails closed.

- `del dek` only removes the Python reference; Python cannot guarantee the bytes are wiped from memory. Production envelope encryption should use a maintained SDK (for example the AWS Encryption SDK or Google Tink), which also defines a standard on-disk message format.

FORMULA

KMS request rate under envelope encryption

$$Q_{\text{KMS}} = \frac{\lambda_{\text{obj}}}{B}$$

Symbol	Meaning	Units
Q_{KMS}	KMS requests generated	requests/s
λ_{obj}	Rate of objects (or records) encrypted or decrypted	objects/s
B	Objects protected per KMS call (1 for a DEK per object; larger with DEK caching or per-batch keys)	objects/request

WORKED EXAMPLE

Worked example — Sizing KMS usage. An ingestion service writes $\lambda = 20,000$ records per second. With one DEK per record ($B = 1$), it would send 20,000 KMS requests per second — likely above a default per-account quota (AWS KMS cryptographic-operation quotas are in the thousands to tens of thousands of requests per second depending on region and key type, as of 2026) and costly at roughly $0.03 per 10,000 requests: $20,000 \times 86,400/10,000 \times \$0.03 \approx \$5,184$ per day. If each of 40 ingestion nodes instead generates a DEK once per minute and uses it for a batch of records (storing the wrapped DEK once per batch), then $Q = 40/60 \approx 0.67$ requests per second, about $0.17 per day. *Interpretation:* DEK granularity is a trade-off between cost/throughput and blast radius: a per-minute DEK exposes at most one minute of one node's records if leaked from memory. *Engineering implication:* choose per-object DEKs for large or individually deletable objects, and time- or count-bounded cached DEKs for high-rate small records — and keep under the GCM per-key message limit from §10.4.2.

10.10 Network security, zero trust and mTLS

10.10.1 From the castle to zero trust

The traditional **perimeter model** is a castle with a moat: a firewall at the edge, a VPN for remote workers, and implicit trust for anything inside. Its failure mode is total. Once an attacker is inside — through a phished laptop, a vulnerable VPN appliance or a compromised internal service — they can reach everything, a technique called **lateral movement**. Most large breaches are a small initial foothold followed by weeks of lateral movement.

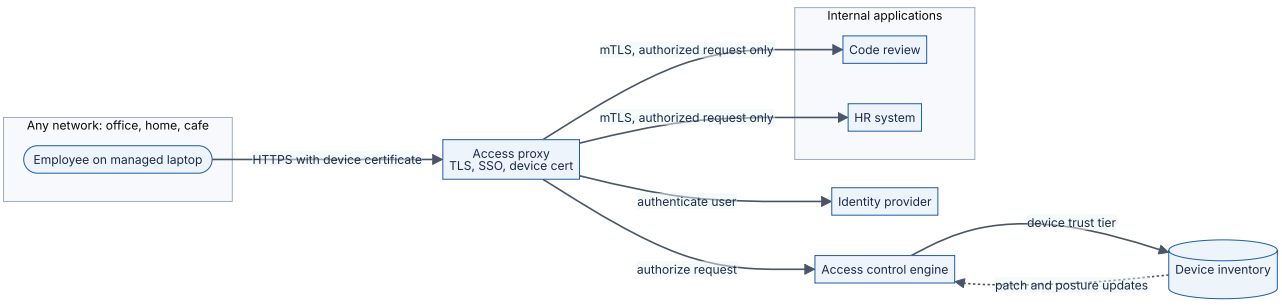
DEFINITION

Definition — Zero trust. A security model in which network location confers no trust: every request is authenticated, authorized and encrypted based on the identity of the user, the device and the workload, plus contextual signals, regardless of where it originates. NIST SP 800-207 describes it as moving defenses "from static, network-based perimeters to focus on users, assets, and resources" [NIST SP 800-207].

INDUSTRY EXAMPLE

Google BeyondCorp. After the 2009 "Operation Aurora" intrusions, Google began moving its corporate applications off the privileged internal network. In BeyondCorp, as described by Ward and Beyer, employee devices sit on an *unprivileged* network even inside Google offices; every request to an internal application goes through an internet-facing **access proxy**; access decisions are made per request by an **access control engine** using the user's identity and group membership, a **device inventory** with per-device certificates, and a computed **trust tier** for the device (patch level, management state). Remote and in-office access become identical, and the VPN is retired. Follow-up papers describe the multi-year migration and the access-proxy design. *Evidence: Primary* — [Ward & Beyer 2014]; [Osborn et al. 2016]

Figure 10.8 — Zero-trust access in the style of BeyondCorp.



Reading the diagram. The employee's location is irrelevant: the office network has no special privileges. Every request reaches the access proxy over HTTPS, presenting a device certificate (identifying the machine) and a single-sign-on session (identifying the person). The proxy asks the identity provider to authenticate the user and the access control engine to authorize *this request to this application*, using group membership and the device's current trust tier from the inventory (fed asynchronously by device-management data). Only authorized requests are forwarded, over mTLS, to applications that accept connections from the proxy alone. Failure points: the proxy and access engine become critical, highly available infrastructure; an over-permissive policy reproduces the old flat network. Scaling: the proxy is a stateless, horizontally scaled L7 tier (§19.3).

10.10.2 Mutual TLS and workload identity

Ordinary TLS authenticates only the server. **Mutual TLS (mTLS)** has *both* sides present certificates and prove possession of the private key, so a service knows cryptographically which workload is calling it — not merely which IP address, which can be spoofed or reused. In practice mTLS between services is provided by a **service mesh** sidecar or proxy (§34.10) that obtains short-lived certificates automatically. The **SPIFFE** standard defines a portable workload identity such as `spiffe://example.org/ns/payments/sa/api` carried in an X.509 certificate (an SVID), so authorization policies can be written as "only `orders` may call `payments` `POST /charge`", independent of IP addresses.

10.10.3 Least privilege and segmentation

Least privilege grants every user, service and credential the minimum permissions needed for its task, for the minimum time [Adkins et al. 2020]. In practice: one IAM role per service, not a shared "app" role; read-only database users for read-only services; no standing production admin access (engineers request time-boxed, approved, logged elevation — "just-in-time access"); and periodic review that removes unused permissions (cloud providers report "last used" data for this).

Segmentation limits blast radius at the network layer: separate VPCs or accounts for production and non-production; subnets for edge, application and data tiers with security groups allowing only the required flows (the boundaries in Figure 10.1); Kubernetes NetworkPolicies that default-deny pod-to-pod traffic; and dedicated egress paths. **Microsegmentation** pushes this down to per-workload policies. Zero trust does not abolish network controls — it stops *relying* on them. Defense in depth means an attacker must defeat identity, authorization, network policy and monitoring, not just one.

Table 10.11 — Perimeter model vs zero trust.

	Perimeter (“castle and moat”)	Zero trust
What it is	Trust based on network location	Trust based on verified identity and context per request
How it works	Edge firewall, VPN, flat internal network	Identity-aware proxies, mTLS, per-request policy
Advantages	Simple; works with legacy apps	Limits lateral movement; uniform remote access
Disadvantages	One breach exposes everything	Requires strong identity, device inventory, mature tooling
Operational complexity	Low initially, high after incidents	High upfront; policy management is ongoing
Choose when	Never as the only control	Default target architecture for new systems

10.11 DDoS basics

DEFINITION

Definition — Distributed denial of service (DDoS). An attack that makes a service unavailable by overwhelming some finite resource — network bandwidth, packet-processing capacity, connection-state tables, or application CPU, memory and database capacity — with traffic from many sources at once, typically a **botnet** of compromised devices or abused open servers.

First-principles explanation

Imagine a restaurant. A volumetric attack is a crowd so large it blocks the street, so no customer reaches the door. A protocol attack is a crowd that walks in, occupies every table, and orders nothing. An application-layer attack is a crowd of well-dressed customers each ordering the single dish that takes the kitchen an hour to cook. Each needs a different defense, applied at a different place.

Table 10.12 — DDoS attack classes.

	Volumetric	Protocol (state exhaustion)	Application layer (L7)
Target resource	Link bandwidth	Connection tables in fire-walls, load balancers, servers	Application CPU, DB, cache, downstream APIs
Examples	UDP floods; reflection/amplification via DNS, NTP, memcached	SYN floods, fragmented-packet floods	HTTP floods on expensive endpoints; HTTP/2 Rapid Reset
Measured in	Bits per second (Gbps–Tbps)	Packets per second	Requests per second
Looks like	Obviously abnormal traffic	Half-open connections	Legitimate-looking requests
Primary defense	Upstream capacity: any-cast scrubbing networks, CDN	SYN cookies, stateless edge proxies, scrubbing	WAF rules, rate limits, bot detection, caching, au-toscaling, load shedding

Reflection and amplification are what make volumetric attacks cheap. The attacker sends small UDP requests to thousands of open servers with the *victim's* IP address forged as the source; each server sends its (much larger) reply to the victim.

FORMULA

Amplified attack bandwidth

$$B_{\text{victim}} = B_{\text{attacker}} \times A$$

Symbol	Meaning	Units
B_{victim}	Traffic arriving at the victim	bits/s
B_{attacker}	Spoofed request traffic the at-tacker sends	bits/s
A	Bandwidth amplification factor of the abused protocol (response bytes / request bytes)	dimensionless

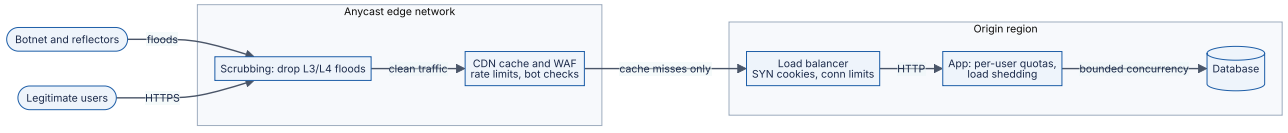
WORKED EXAMPLE

Worked example — DNS amplification. US-CERT lists DNS amplification factors of roughly 28–54 [US-CERT TA14-017A]. An attacker with 2 Gbps of spoofing capacity and $A = 50$ produces $B = 2 \times 50 = 100$ Gbps at the victim. A single origin data center with a 10 Gbps uplink is saturated tenfold; nothing the origin does locally helps, because the link is full before packets reach any server. *Engineering implication:* volumetric defense must happen **upstream**, in networks whose capacity exceeds the attack — anycast CDNs and scrubbing services that advertise your IP prefixes from many locations and absorb the flood across their global capacity.

INDUSTRY EXAMPLE

GitHub memcached attack (2018) and HTTP/2 Rapid Reset (2023). On 28 February 2018 GitHub received a 1.35 Tbps memcached-reflection attack; memcached servers exposed on UDP can amplify requests by factors in the tens of thousands. GitHub routed traffic to its DDoS-mitigation provider and the attack was mitigated within about ten minutes. In 2023 the “HTTP/2 Rapid Reset” technique (CVE-2023-44487) — opening and immediately cancelling streams on a single connection — produced application-layer floods that Google reported peaking at about 398 million requests per second. *Evidence: Primary* — [GitHub 2018]; [Google Cloud 2023]

Figure 10.9 — DDoS mitigation in layers.



Reading the diagram. Traffic flows left to right through successively narrower and more expensive layers. The anycast edge, spread across many locations, absorbs volumetric and protocol floods (the scrubbing stage) because its aggregate capacity is far larger than any single origin’s. The CDN serves cacheable content without touching the origin and applies L7 controls: WAF rules, per-IP and per-session rate limits, and bot challenges. The origin should accept traffic only from the edge (allow-list the CDN’s ranges or use authenticated origin pulls), otherwise attackers bypass the edge by hitting the origin IP directly. Inside, the load balancer defends connection state, and the application enforces per-user quotas and sheds load (§22.9) so that the database — the scarcest resource — sees bounded concurrency. Scaling: autoscaling helps against L7 floods only up to your budget; an attacker’s marginal cost per request is usually lower than yours, so the main lever is making requests cheap (caching) or refusing them early.

RECOMMENDATION

Design rules. Put every public endpoint behind an anycast CDN or DDoS-protection service; hide origin IPs; cache aggressively; make expensive endpoints (search, report generation, login) authenticated and rate-limited; set timeouts and bounded queues everywhere (§22.11); know in advance who you call and what you toggle during an attack (a runbook, rehearsed).

10.12 Audit logging and tamper evidence

DEFINITION

Definition — Audit log. An append-only record of security-relevant events — who (actor), did what (action), to which resource, when, from where, and with what result — retained so that actions can be investigated and attributed. A log is **tamper-evident** if any modification, deletion or reordering of past entries can be detected.

First-principles explanation

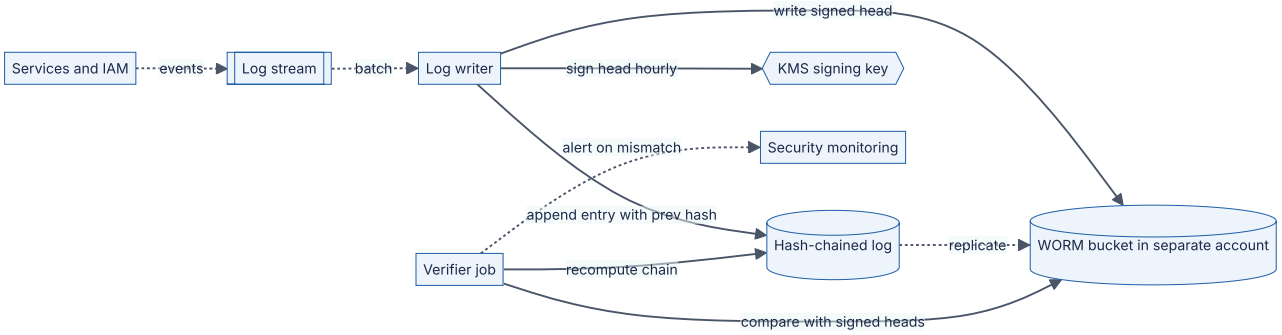
An audit log is a bank's bound, page-numbered ledger written in ink: you can add lines but cannot tear out a page or erase a line without it being obvious. Application debug logs are the opposite — a whiteboard. Attackers who gain privileged access routinely delete or edit logs to hide their tracks, and malicious insiders are precisely the people with access to logs. So audit logs must be designed so that the people being audited cannot rewrite them.

What to log. Authentication events (success, failure, MFA changes), authorization failures, privilege and role changes, access to sensitive data (reads, exports), administrative and configuration changes, key and secret usage (KMS logs this for you), and security-control changes (disabling logging is itself an event). Each entry: timestamp (UTC, synchronized clock), actor identity (user and workload), source (IP, device), action, resource, result, and a request or trace ID. **What not to log:** passwords, session tokens, full card numbers, secret values, or more personal data than necessary — the log becomes a sensitive data store itself.

How tamper evidence works. Each entry includes the hash of the previous entry, forming a **hash chain**: $h_i = H(h_{i-1} \parallel e_i)$. Changing any past entry changes its hash and breaks every link after it. A chain alone is not enough — whoever can write the log can recompute the whole chain — so the latest hash (the **head**) is periodically **anchored** somewhere the writer cannot alter: signed by a key in a KMS, written to a separate account's write-once (WORM) storage, or published to an external transparency log. Schneier and Kelsey described keyed hash-chain audit logs in 1999; Crosby and Wallach later showed

how Merkle-tree structures make such logs efficiently verifiable — the same idea behind Certificate Transparency (§10.5) [Schneier & Kelsey 1999; Crosby & Wallach 2009].

Figure 10.10 — Tamper-evident audit logging pipeline.



Reading the diagram. Services emit events asynchronously to a stream (dotted arrows), so logging never blocks user requests — at the cost that events can be delayed; the stream must be durable so events are not lost. A single writer appends each event with the previous entry's hash. Every hour it asks the KMS to sign the current head (the signing key is usable only by the writer's role and never leaves the KMS) and stores signed heads and log replicas in a WORM bucket in a separate account with object lock, so even production administrators cannot delete them before retention expires. An independent verifier recomputes the chain and compares it with the signed heads, alerting the security team on any mismatch. Failure points: the writer is a bottleneck and single point of ordering (partition the chain per service or per tenant to scale); clock skew complicates ordering (use the chain order, not timestamps, as the authority).

```
        """Tamper-evident audit log: each entry commits to the previous one (hash
chain)."""
        import hashlib
        import hmac
        import json

        GENESIS = "0" * 64

        def entry_hash(prev_hash: str, event: dict) -> str:
            payload = json.dumps(event, sort_keys=True, separators=(",", ":")).encode()
            return hashlib.sha256(prev_hash.encode() + payload).hexdigest()

        class AuditLog:
            def __init__(self):
                self.entries: list[dict] = []

            def append(self, event: dict) -> str:
                prev = self.entries[-1]["hash"] if self.entries else GENESIS
                h = entry_hash(prev, event)
                self.entries.append({"event": event, "prev": prev, "hash": h})
                return h                        # publish/anchor this "head" externally

            def verify(self, anchored_head: str | None = None) -> int | None:
                """Return index of first bad entry, or None if the chain is intact."""
                prev = GENESIS
                for i, e in enumerate(self.entries):
                    if e["prev"] != prev or not hmac.compare_digest(e["hash"],
entry_hash(prev, e["event"])):
                        return i
                    prev = e["hash"]
                if anchored_head is not None and prev != anchored_head:
                    return len(self.entries)   # tail truncated or rewritten
                return None
```

Tested: an intact chain verifies; editing the `resource` field of entry 1 makes `verify` return index 1; deleting the last entry is caught only when the verifier is given the previously anchored head.

What to notice: canonical JSON (`sort_keys`, fixed separators) makes the hash reproducible; truncation is detectable *only* because the verifier holds an anchored head from outside the log; and in production the anchor is signed and stored where the writer cannot overwrite it.

INDUSTRY EXAMPLE

AWS CloudTrail log file integrity validation. CloudTrail delivers hourly digest files that contain the SHA-256 hashes of the log files delivered in that hour, are signed with an RSA key held by AWS, and include the signature of the previous digest — a signed hash chain. Customers verify with a CLI command and are advised to store logs in a separate account with S3 Object Lock. *Evidence: Primary* — [AWS Docs: CloudTrail log file integrity validation]

10.13 A security checklist for every design

Use this table in every design document and design review — and in interviews, where spending two minutes on it distinguishes senior candidates.

Table 10.13 — Security checklist for every design.

Area	Question to answer in the design	Default good answer	Section
Threat model	What are the assets, adversaries and trust boundaries? Top 3 threats?	DFD with boundaries; STRIDE table; owners for mitigations	§10.3
Authentication	How is every user and every service identified?	Central IdP, MFA for humans; mTLS or signed tokens for services	§10.10, §8.11
Authorization	Where is each access decision made? Default?	Deny by default; central policy; object-level checks in data layer	§10.7.1
Input handling	What untrusted input crosses boundaries, and how is it validated?	Schema validation; parameterized queries; output encoding; SSRF egress proxy	§10.7
Data in transit	Is every hop encrypted, including internal ones?	TLS 1.3 everywhere; mTLS between services	§10.4.4, §10.10
Data at rest	What is sensitive? At which layer is it encrypted? Who can decrypt?	Volume encryption everywhere; KMS envelope/field encryption for sensitive fields	§10.9
Keys and secrets	Where do secrets live? Lifetime? Rotation?	Vault or KMS; short-lived dynamic credentials; no secrets in code or images	§10.8
Passwords	If you store them, how?	Argon2id (or scrypt/bcrypt), unique salt, optional pepper, breached-list check	§10.6
Least privilege	What is the minimum each component can do? Blast radius if compromised?	Per-service roles; segmented networks; no standing admin access	§10.10.3
Availability under attack	What happens under a 100× traffic flood?	CDN/anycast, rate limits, quotas, load shedding, hidden origin	§10.11
Logging and detection	Can you reconstruct who did what? Can logs be altered?	Structured audit log; tamper-evident; separate account; alerts	§10.12
Supply chain	How do dependencies and builds stay trustworthy?	Pinned dependencies, scanning, signed artifacts, protected CI	§10.7, Ch 50
Incident response	Who is paged, what is revoked, how do you rotate everything?	Runbooks; tested key rotation; backups that attackers cannot delete	§39.12
Privacy and retention	Do you need this data? For how long?	Collect minimum; retention limits; crypto-shredding for deletion	§10.9, Ch 50

10.14 Summary and revision notes

SUMMARY

- Security is a property of the architecture. Ask “against whom, protecting what, for which property (CIA, authentication, authorization, non-repudiation)?”
- Threat modeling: draw a DFD, mark trust boundaries, walk STRIDE per element, rank with likelihood × impact, and turn mitigations into tracked requirements. Most mitigations are architectural.
- Pick primitives by goal: hash (fingerprint, no key), MAC (shared-key authenticity), AEAD such as AES-GCM (secrecy + integrity), asymmetric encryption/KEM (no pre-shared key), signatures (publicly verifiable, non-repudiation), ECDHE (key agreement with forward secrecy), CSPRNG (all randomness).
- Never reuse a GCM nonce with the same key; never use `H(key‖msg)` as a MAC; never compare MACs with `==`; never use MD5/SHA-1 for security; use high-level libraries.
- PKI binds keys to names through CA signatures; Certificate Transparency makes mis-issuance detectable; short-lived, automatically rotated certificates beat revocation.
- Passwords: Argon2id (OWASP minimum m = 19 MiB, t = 2, p = 1), scrypt or bcrypt; unique salt; optional pepper in a KMS; store parameters; rehash on login; check breached lists; MFA. Expected cracking time $T = S/(2RG)$.
- OWASP Top 10 categories are missing central controls: deny-by-default authorization, parameterized queries, egress proxies against SSRF, signed artifacts, audit logging.
- Secrets: vault, short-lived dynamic credentials, workload identity, two-version rotation, secret scanning. Misuse window shrinks from months to the TTL.
- Encrypt in transit everywhere (TLS 1.3, mTLS). At rest, the layer decides which attacker you stop. Envelope encryption: DEK encrypts data locally, KMS-held KEK wraps the DEK; cheap rotation, auditable decrypts, crypto-shredding.
- Zero trust: no trust from network location; per-request authentication and authorization of user, device and workload (BeyondCorp, NIST SP 800-207), plus least privilege and segmentation for defense in depth.
- DDoS: volumetric (bandwidth, absorb upstream), protocol (state, SYN cookies/stateless edge), application (requests, rate limits, caching, shedding). Amplification $B_{\text{victim}} = B_{\text{attacker}} \times A$.
- Audit logs: who/what/which/when/where/result, no secrets; hash-chained, anchored heads signed and stored in WORM storage outside the writer’s control.

10.15 Knowledge checkpoint

1. Which STRIDE category does each of the following violate: (a) a forged webhook, (b) an admin denying a deletion, (c) reading another user's invoice by changing an ID?
2. Why is SHA-256(key ‖ message) not a safe MAC, and what does HMAC change?
3. What happens if AES-GCM encrypts two messages with the same key and nonce?
4. What is the purpose of associated data (AAD) in an AEAD scheme? Give a database example.
5. Why does TLS 1.3 use ephemeral Diffie–Hellman rather than RSA key transport?
6. A MAC and a signature both detect tampering. Which one provides non-repudiation, and why?
7. What problem does Certificate Transparency solve that the CA chain alone does not?
8. Name three reasons salts are required in password storage.
9. Compute the entropy of a 10-character password chosen uniformly from 62 alphanumeric characters.
10. Give two architectural controls for SSRF beyond URL validation.
11. Classify: SYN flood, DNS amplification, HTTP flood on `/search`.
12. Why must the head of a hash-chained audit log be anchored outside the log?

Answers

1. a. Spoofing; (b) Repudiation; (c) Elevation of privilege (an authorization failure — an IDOR; it also causes information disclosure).
2. SHA-256 is vulnerable to length extension: from the tag of `key‖m`, an attacker can compute a valid tag for `key‖m‖padding‖x` without the key. HMAC's nested two-pass construction with inner and outer padded keys prevents this.
3. The same keystream is used twice, so XORing the ciphertexts reveals the XOR of the plaintexts, and the authentication key can be recovered, enabling forgeries. Confidentiality and integrity both fail.
4. AAD is authenticated but not encrypted; it binds ciphertext to its context. Putting `table=users;id=42` in the AAD prevents an attacker with DB write access from moving that ciphertext to row 43.
5. Forward secrecy: ephemeral keys are discarded after each session, so stealing the server's long-term key later does not decrypt recorded traffic. The long-term key only signs.
6. The signature. A MAC key is shared by both parties, so either could have produced the tag; only the private-key holder can produce a signature, and anyone can verify it.
7. It detects a trusted CA issuing a certificate for your domain to someone else (compromise or error). The chain only shows *some* trusted CA signed it; CT logs make every issuance public so domain owners can notice.

8. Identical passwords no longer have identical hashes; precomputed (rainbow) tables become useless; attackers must attack each user separately, multiplying cost by the number of users.
9. $H = 10 \times \log_2 62 \approx 10 \times 5.954 = 59.5$ bits.
10. A dedicated egress proxy in a network segment without routes to internal ranges; blocking or token-protecting the metadata service (IMDSv2); least-privilege instance roles; pinning resolved IPs and disabling redirects.
11. SYN flood: protocol (state exhaustion). DNS amplification: volumetric. HTTP flood on `/search`: application layer.
12. Anyone who can write the log can recompute the entire chain after editing it. A head signed by a separate key and stored in WORM storage the writer cannot modify fixes a point the rewritten chain will not match.

10.16 Interview questions

Basic

Q1. How should a web application store user passwords? Answer. Never in plaintext or reversibly encrypted. Hash each password with a slow, memory-hard password-hashing function — Argon2id by default (at least m = 19 MiB, t = 2, p = 1 per OWASP), otherwise scrypt or bcrypt — with a unique random salt of at least 16 bytes. Store algorithm and parameters with the hash so you can raise cost later and rehash on next login. Optionally apply a pepper (a secret key in a KMS/HSM) so a database-only leak is not attackable. Compare in constant time, rate-limit login, check new passwords against breached lists, and offer MFA. Size the login fleet for the hashing cost.

Q2. What is the difference between encryption, hashing and signing? Answer. Encryption provides confidentiality and is reversible with a key. Hashing is a keyless one-way fingerprint for integrity against a trusted reference. Signing uses a private key to produce a tag anyone can verify with the public key, providing integrity, authenticity and non-repudiation. A MAC is the shared-key analog of a signature without non-repudiation. AEAD encryption combines confidentiality with integrity.

Intermediate

Q3. Design encryption for a multi-tenant document store where each tenant's data must be deletable on request, including from backups. Answer. Use envelope encryption with a per-tenant KEK in a KMS and a per-document DEK. Each document is encrypted locally with AES-256-GCM under its DEK; the wrapped DEK and KEK ID are stored with it, and the encryption context includes tenant ID and document ID. IAM policy on each KEK allows decrypt only by the service's role, and KMS logs every decrypt for audit. To delete a tenant, schedule deletion of its KEK: all its DEKs, and therefore all copies

of its documents including immutable backups, become unreadable (crypto-shredding). Discuss KMS quotas (DEK per document is fine for large documents; batch keys for tiny records), KMS availability as a dependency, regional keys for residency, and the waiting period that protects against accidental KEK deletion.

Q4. A partner sends you webhooks. How do you make sure they are genuine and not replayed? Answer. Share a high-entropy secret per partner (with a key ID for rotation). The partner computes HMAC-SHA-256 over a canonical string of timestamp, a unique event ID or nonce, and the raw body. You verify by recomputing over the raw bytes, comparing in constant time, rejecting timestamps outside a ±5-minute window and rejecting event IDs already seen (a shared store with TTL just longer than the window). Process idempotently by event ID (§22.15), since retries are legitimate. Support two active keys during rotation. Use TLS for confidentiality; HMAC provides authenticity.

Advanced

Q5. Your company runs 400 microservices on a flat internal network with shared database passwords in environment variables. Propose a two-year security roadmap. Answer. Start with visibility: inventory services and data flows, threat-model the most sensitive data stores, centralize tamper-evident audit logging. Phase 1 (0–6 months): move secrets into a vault and rotate all of them (assume compromise); add secret scanning; one IAM role and one database user per service. Phase 2 (6–12 months): issue short-lived SPIFFE-style workload certificates via a mesh or library; enable mTLS in permissive then strict mode; move databases to dynamic credentials. Phase 3 (12–24 months): default-deny network policies, per-service authorization policies, an identity-aware proxy for internal tools, just-in-time admin access. Measure progress (share of traffic on mTLS, secrets older than 30 days, wildcard permissions), roll out with dashboards and per-phase rollback switches. Trade-offs: mesh latency and operational load, database connection limits with per-workload users, and the need for owned, reviewed policies.

Q6. How would you design an audit logging system for a bank that regulators and internal investigators must be able to trust, even against a malicious administrator? Answer. Define the event schema (actor, workload, action, resource, result, source, trace ID) and emit from every service through a durable stream so logging does not block requests. A writer service builds per-partition hash chains; hourly it signs each chain head using a KMS key restricted to the writer role and writes signed heads plus log segments to WORM storage (object lock in compliance mode) in a separate account administered by a different team. An independent verifier recomputes chains and compares with signed heads; mismatches alert security operations. Separate duties: no single administrator controls the production account, the log account and the KMS key policy. Index a copy in a SIEM for search, treating the SIEM as derived and untrusted. Address clock sync, retention (years, per regulation), personal data minimization, and Merkle-tree structures if third parties need efficient inclusion proofs.

10.17 Practical exercises

1. **Password hashing benchmark.** Run the code in §10.6.2. Increase Argon2id memory to 64 MiB and t to 3 and measure latency. Compute how many cores you would need for 500 logins per second at peak with 50% headroom.
2. **HMAC request signing.** Extend the code in §10.4.6 to support two active keys per client, and write a test proving that removing the HTTP method from the canonical string lets an attacker turn a signed `GET` into a `DELETE`.
3. **Envelope encryption.** Extend the `FakeKMS` example in §10.9.2 with a per-tenant KEK and a `delete_tenant` operation. Show that encrypted objects for a deleted tenant can no longer be decrypted while other tenants are unaffected. Then replace `FakeKMS` with a real KMS client in a sandbox account if you have one.
4. **Brute-force calculator.** Write a function that takes alphabet size, length, guesses per second and number of GPUs and prints expected time. Use it to compare a 4-word and a 6-word Diceware passphrase (7,776 words) against bcrypt and against unsalted SHA-256.
5. **Tamper-evident log.** Modify the audit log in §10.12 so that every 100 entries the head is signed with an Ed25519 key (§10.4.8). Write a verifier that accepts only chains whose signed heads verify.
6. **Threat model.** Draw a DFD for a URL shortener (§52.1) with trust boundaries and produce a STRIDE table with at least eight threats ranked by $L \times I$.

10.18 Design challenges

1. **Secure file-sharing link service.** Users share files by link; links can expire and be revoked; recipients may be anonymous. Design storage encryption, link-token generation (entropy and length), revocation, download rate limiting and audit logging. Identify where signed URLs, HMAC and envelope encryption each fit.
2. **Zero-trust migration for an admin console.** A company's internal admin console is reachable over VPN by 300 support agents and offers "view any customer" functionality. Redesign access with an identity-aware proxy, device posture, just-in-time elevation for sensitive actions and an audit trail that customers can be shown. State the failure mode if the identity provider is down.
3. **DDoS-resilient public API.** A public API serves 20,000 RPS normally, with an expensive search endpoint costing 50 ms of database time per request. Design layered defenses for a 1 Tbps volumetric attack and for a 2 million RPS application-layer flood from residential IPs. Quantify how much the cache hit ratio and per-key rate limits reduce database load.

10.19 References

1. Shostack, A. *Threat Modeling: Designing for Security*. Wiley, 2014.
2. Kohnfelder, L., Garg, P. *The Threats to Our Products*. Microsoft internal paper, 1999.
3. OWASP Foundation. *OWASP Top 10:2021*. 2021. https://owasp.org/Top10/
4. OWASP Foundation. *OWASP Top 10:2025*. 2025–2026. https://owasp.org/Top10/2025/
5. OWASP Cheat Sheet Series. *Password Storage Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
6. NIST. *SP 800-63B-4: Digital Identity Guidelines — Authentication and Authenticator Management*. National Institute of Standards and Technology, 2025. https://pages.nist.gov/800-63-4/sp800-63b.html
7. Biryukov, A., Dinu, D., Khovratovich, D., Josefsson, S. *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. RFC 9106, IETF, 2021. https://www.rfc-editor.org/rfc/rfc9106
8. Percival, C., Josefsson, S. *The scrypt Password-Based Key Derivation Function*. RFC 7914, IETF, 2016. https://www.rfc-editor.org/rfc/rfc7914
9. Ward, R., Beyer, B. “BeyondCorp: A New Approach to Enterprise Security.” *;login:* 39(6), USENIX, 2014.
10. Osborn, B., McWilliams, J., Beyer, B., Saltonstall, M. “BeyondCorp: Design to Deployment at Google.” *;login:* 41(1), USENIX, 2016.
11. Rose, S., Borchert, O., Mitchell, S., Connelly, S. *Zero Trust Architecture*. NIST SP 800-207, 2020. https://doi.org/10.6028/NIST.SP.800-207
12. Ferguson, N., Schneier, B., Kohno, T. *Cryptography Engineering: Design Principles and Practical Applications*. Wiley, 2010.
13. Adkins, H., Beyer, B., Blankinship, P., Lewandowski, P., Oprea, A., Stubblefield, A. *Building Secure and Reliable Systems*. O’Reilly, 2020.
14. Amazon Web Services. *AWS Key Management Service Developer Guide — AWS KMS concepts (envelope encryption, data keys, encryption context)*. https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html
15. Amazon Web Services. *Amazon EC2 User Guide — Use the Instance Metadata Service Version 2 (IMDSv2)*. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-instance-metadata-service.html
16. Amazon Web Services. *AWS CloudTrail User Guide — Validating CloudTrail log file integrity*. https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-log-file-validation-intro.html
17. Krawczyk, H., Bellare, M., Canetti, R. *HMAC: Keyed-Hashing for Message Authentication*. RFC 2104, IETF, 1997. https://www.rfc-editor.org/rfc/rfc2104
18. Rescorla, E. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, IETF, 2018. https://www.rfc-editor.org/rfc/rfc8446

19. Laurie, B., Langley, A., Kasper, E. *Certificate Transparency*. RFC 6962, IETF, 2013. https://www.rfc-editor.org/rfc/rfc6962
20. Barnes, R., Hoffman-Andrews, J., McCarney, D., Kasten, J. *Automatic Certificate Management Environment (ACME)*. RFC 8555, IETF, 2019. https://www.rfc-editor.org/rfc/rfc8555
21. Dworkin, M. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST SP 800-38D, 2007.
22. Barker, E. *Recommendation for Key Management: Part 1 — General*. NIST SP 800-57 Part 1 Rev. 5, 2020.
23. NIST. *FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA)*. August 2024.
24. US-CERT (CISA). *Alert TA14-017A: UDP-Based Amplification Attacks*. 2014 (updated).
25. GitHub Engineering. “February 28th DDoS Incident Report.” GitHub Blog, 2018.
26. Google Cloud. “How it works: The novel HTTP/2 ‘Rapid Reset’ DDoS attack.” Google Cloud Blog, 2023.
27. Fox-IT. *Black Tulip: Report of the Investigation into the DigiNotar Certificate Authority Breach*. 2012.
28. Schneier, B., Kelsey, J. “Secure Audit Logs to Support Computer Forensics.” *ACM Transactions on Information and System Security* 2(2), 1999.
29. Crosby, S. A., Wallach, D. S. “Efficient Data Structures for Tamper-Evident Logging.” *USENIX Security Symposium*, 2009.

10.20 Open verification items

- OWASP Top 10:2025 final category names, numbering and release status (reported as unveiled November 2025, finalized early 2026); owasp.org was not reachable from the authoring environment.
- NIST SP 800-63B-4 exact final wording on 15-character single-factor minimum and on peppers/keyed hashing (confirmed only through secondary sources).
- CA/Browser Forum ballot SC-081 lifetime schedule (200 → 100 → 47 days, 2026–2029).
- GPU guessing rates (~10^{10} SHA-256/s per GPU; ~10^{4}/s for tuned memory-hard hashes) are order-of-magnitude estimates.
- AWS KMS request quotas and pricing ($0.03 per 10,000 requests) as of 2026.
- LinkedIn 2012/2016 and Capital One 2019 figures (secondary reporting).
- Exact URLs for BeyondCorp ;login: papers and the AWS KMS envelope-encryption page anchor.

Part III — Data Systems

Chapter 11 — Relational Databases

> **Level:** 1–2 · **Track:** A (Traditional) · **Estimated reading time:** 5 h

Almost every system you will design has a relational database somewhere in it. The orders in a shop, the balances in a bank, the users of a social network and the configuration of a cloud platform usually live in PostgreSQL, MySQL or a database that speaks their SQL dialect. Relational databases have been the default for fifty years because they solve three hard problems at once: they let you describe *what* data you want without saying *how* to fetch it, they keep data correct while thousands of clients change it concurrently, and they keep committed data safe through crashes.

This chapter explains how they do it, using one running example: the order system of an online shop. You will learn to model its data, index it, read query plans, choose isolation levels and avoid costly concurrency bugs. On-disk internals (write-ahead logging, buffer pools, LSM trees) are covered in Chapter 13, replication and sharding in Chapter 14, and distributed transactions in Chapter 28.

11.1 Learning objectives

After this chapter you should be able to:

- Explain the relational model and choose primary keys (natural or surrogate) and foreign keys.
- Normalize a schema to BCNF using functional dependencies, and justify each deliberate denormalization.
- Write inner, outer, semi- and anti-joins, and estimate the I/O cost of nested-loop, hash and sort-merge joins.
- Design clustered, secondary, composite, covering and partial indexes for a stated query workload.
- Compute B+ tree height and I/Os per lookup from page size, key size and row count.
- Read a PostgreSQL `EXPLAIN (ANALYZE, BUFFERS)` plan and diagnose a cardinality misestimate.
- Define ACID precisely and distinguish ACID consistency from consistency in distributed systems.
- Reproduce dirty reads, non-repeatable reads, phantoms, lost updates and write skew, and state which isolation level prevents each in PostgreSQL and MySQL.

- Explain MVCC in PostgreSQL (tuple versions, `xmin`/`xmax`, VACUUM) and InnoDB (undo logs, purge).
- Explain two-phase locking, gap locks and deadlock detection, and avoid deadlocks and hot-row contention.
- Produce an end-to-end schema, DDL and queries for an e-commerce order system.

11.2 Prerequisites

- Logarithms and B-tree fan-out — §4.4 (index height is a logarithm with a large base).
- Storage devices and latency numbers — §5.6, §4.17 (a random SSD read costs ~100 µs; RAM ~100 ns).
- Page cache and `fsync` — §6.6, §6.14 (durability depends on forcing the log to disk).
- Race conditions, mutexes and deadlock — §6.9–§6.11 (database locking applies the same ideas to rows).
- Keyset pagination — §8.6.5 (we reuse it in the order-history query).
- Idempotency keys — §8.6.3 and §22.15 (the order API must be safe to retry).

11.3 The relational model: tables, rows and columns

11.3.1 Relations, tuples and attributes

> DEFINITION
>
> **Definition — Relation.** A **relation** is a named set of **tuples** that share the same **attributes**. Each attribute has a name and a **domain** (the set of allowed values, such as 64-bit integers). In SQL a relation is a **table**, a tuple is a **row** and an attribute is a **column**. The relation's name and attributes together form its **schema**; the set of rows at a moment is its **instance**.

First-principles explanation

A table looks like a spreadsheet, but three differences matter. A relation is a *set*: rows have no inherent order and are identified by their values (a key), never by "row 17". Every column has a fixed type, enforced on every write. And relationships are expressed by *values*: an order row stores the customer's ID, not a pointer. These properties let the query optimizer rearrange *how* a question is answered without changing the answer.

Why does this exist?

Before 1970, hierarchical (IBM IMS) and network (CODASYL) databases linked records by physical pointers, and programs *navigated* them: "go to customer 42, follow the first-

order pointer…". Reorganizing storage broke the programs. Codd's paper "A Relational Model of Data for Large Shared Data Banks" proposed that users should see only relations, and that access paths should be the database's private business [Codd 1970]. This **physical data independence** is why you can add an index to a production table today and every existing query keeps working, only faster.

How it works internally

1. You state *what* you want in SQL, a declarative language first built at IBM for System R.
2. The database translates it into **relational algebra**: selection σ (filter rows), projection π (choose columns), join ⋈, union, difference and aggregation.
3. Algebraic equivalences ("filter before joining", "join A with B before C") let the optimizer generate equivalent plans and pick the cheapest (§11.10).
4. The executor runs that plan against physical structures (heap files, B+ trees) that the query never mentions (§11.9).

NULL and three-valued logic

SQL adds **NULL**, meaning "unknown or not applicable". Any comparison with NULL yields UNKNOWN, not TRUE or FALSE, and `WHERE` keeps only rows whose condition is TRUE.

> COMMON MISTAKE
>
> **Mistake — `WHERE coupon_code <> 'SPRING'` returns orders without a coupon.** It does not: for rows where `coupon_code` is NULL, the comparison is UNKNOWN and the row is dropped. Write `WHERE coupon_code IS DISTINCT FROM 'SPRING'` (PostgreSQL, SQLite) or add `OR coupon_code IS NULL`. Likewise `NOT IN (subquery)` returns no rows if the subquery yields a single NULL; prefer `NOT EXISTS`.

11.4 Keys: primary, foreign, natural and surrogate

11.4.1 Superkeys, candidate keys and primary keys

DEFINITION

Definition — Keys. A **superkey** is a set of columns whose values are unique across all rows. A **candidate key** is a minimal superkey: remove any column and it stops being unique. The **primary key** is the candidate key chosen as the row's main identifier; it must be unique and NOT NULL. Other candidate keys are enforced with `UNIQUE` constraints. A **foreign key** is a column (or set) in one table whose values must match a key of another table, a rule called **referential integrity**.

In the shop, `customers` has two candidate keys: `customer_id` and `email`. We choose `customer_id` as the primary key and declare `email UNIQUE`. `orders.customer_id` is a foreign key to `customers.customer_id`: the database rejects an order for a customer that does not exist, and rejects deleting a customer who still has orders unless you specify otherwise.

The **referential action** says what happens to child rows when a parent is deleted: `NO ACTION` / `RESTRICT` (reject; the default), `CASCADE`, `SET NULL` or `SET DEFAULT`. Use `CASCADE` for true ownership (an order owns its items) and the default for independent entities (deactivate products rather than delete them).

Code example

```
        import sqlite3

        db = sqlite3.connect(":memory:")
        db.executescript("""
        PRAGMA foreign_keys = ON;            -- SQLite enforces FKs only when this is
set
        CREATE TABLE customers (customer_id INTEGER PRIMARY KEY, email TEXT NOT NULL
UNIQUE);
        CREATE TABLE orders (
          order_id    INTEGER PRIMARY KEY,
          customer_id INTEGER NOT NULL REFERENCES customers (customer_id)
        );
        INSERT INTO customers VALUES (1, 'ana@example.com');
        INSERT INTO orders VALUES (100, 1);
        """)
        for sql in ["INSERT INTO orders VALUES (101, 99)",                    # no cus-
tomer 99
                    "INSERT INTO customers VALUES (2, 'ana@example.com')",   # duplicate
candidate key
                    "DELETE FROM customers WHERE customer_id = 1"]:          # still has
an order
            try:
                db.execute(sql)
            except sqlite3.IntegrityError as e:
                print(f"{sql[:40]:<42} -> {e}")
```

What to notice:

- All three statements fail with `IntegrityError`: the database, not application code, guards the invariants. Output: `FOREIGN KEY constraint failed`, `UNIQUE constraint failed: customers.email`, `FOREIGN KEY constraint failed`.
- SQLite enforces foreign keys only after `PRAGMA foreign_keys = ON`, per connection. PostgreSQL and MySQL (InnoDB) always enforce them.
- Foreign-key checks cost an index lookup on the parent for every child insert. PostgreSQL does **not** automatically index the *child* column (`orders.customer_id`); without that index, deleting a customer scans the whole `orders` table. Index foreign-key columns you join or delete through.

11.4.2 Natural versus surrogate keys

DEFINITION

Definition — Natural and surrogate keys. A **natural key** is a candidate key with business meaning (email address, ISBN, country code). A **surrogate key** is a meaningless value generated by the system (an auto-increment integer, a UUID) whose only job is identification.

Business identifiers change (people change email addresses; product codes are reissued), and every foreign key that copied the old value must change too. Surrogate keys never change, are compact and make joins cheap. The common rule: a surrogate primary key, plus a `UNIQUE` constraint on the natural key. Natural keys suit small, stable code tables (ISO country or currency codes).

The *kind* of surrogate matters for B+ tree indexes (§11.8). Sequential integers always insert at the right-most leaf, so pages fill completely and stay hot in cache. Random UUIDv4 values insert into a random leaf, causing page splits, half-empty pages and a working set as large as the whole index. Time-ordered identifiers (UUIDv7, Snowflake-style IDs; §17.10) keep the insert locality of integers while being globally unique without a central counter.

Table 11.1 — Choosing a primary key.

Key type	Size	Insert locality in B+ tree	Globally unique	Leaks information	Choose when
Natural key (email, SKU)	Variable	Random	Within domain	Yes (it is data)	Small stable code tables
`bigint` identity / auto-increment	8 bytes	Sequential (excellent)	Per table only	Row count, growth rate	Single database, default choice
UUIDv4	16 bytes	Random (poor)	Yes	No	IDs generated offline by clients; small tables
UUIDv7 / Snowflake-style	16 / 8 bytes	Roughly sequential	Yes	Creation time	Many writers or shards (§17.10)

INDUSTRY EXAMPLE

Instagram — sharded PostgreSQL with time-ordered 64-bit IDs. Instagram described generating primary keys inside PostgreSQL with a function that packs 41 bits of milliseconds since a custom epoch, 13 bits of logical shard ID and 10 bits of a per-shard sequence. IDs sort by time (good B+ tree locality, cheap “newest first” queries), fit in a `bigint`, and need no central ID service. The trade-off is that IDs reveal creation time and that the bit budget caps per-shard throughput at 1,024 IDs per millisecond. *Evidence: Primary* — [Instagram Engineering 2012]

COMMON MISTAKE

Mistake — “UUIDs are slow, never use them.” The cost is randomness, not size. A UUIDv4 primary key in InnoDB, where the primary key *is* the storage order (§11.7), scatters inserts across the whole table; a UUIDv7 key does not.

11.5 Normalization and denormalization

11.5.1 Redundancy and the three anomalies

First-principles explanation

Imagine the shop's first version kept one spreadsheet row per order line, with the customer's email and the product's name copied into every row. When Ana changes her email, 40 rows must change; miss one and the database holds two "truths". Normalization's goal is that **every fact is stored exactly once**.

Redundant designs suffer three **anomalies**:

1. **Update anomaly** — changing one fact requires changing many rows; a partial update leaves contradictions.
2. **Insert anomaly** — you cannot record a fact without an unrelated one: a new product cannot be stored until somebody orders it.
3. **Delete anomaly** — deleting one fact destroys another: canceling and purging the only order of a product loses the product's name and price.

DEFINITION

Definition — Functional dependency. In a relation R, the functional dependency $X \to Y$ holds if any two rows that agree on the columns X also agree on the columns Y. X is the **determinant**. Example: `product_id → product_name` because one product ID always has one name.

11.5.2 The normal forms

DEFINITION

Definition — Normal forms (1NF to BCNF). 1NF: every column holds a single atomic value (no lists or repeating groups). **2NF:** 1NF, and no non-key column depends on only *part* of a composite candidate key. **3NF:** 2NF, and no non-key column depends on another non-key column (no transitive dependency); formally, for every non-trivial $X \to A$, either X is a superkey or A is part of some candidate key. **BCNF (Boyce–Codd normal form):** for every non-trivial $X \to A$, X is a superkey. BCNF removes 3NF's exception for key columns.

Kent's one-line summary of 2NF and 3NF is that every non-key column must provide a fact about "the key, the whole key, and nothing but the key" [Kent 1983]. Higher forms (4NF, 5NF) deal with multi-valued and join dependencies and rarely matter in application design.

Table 11.2 — Normal forms at a glance.

Form	Forbids	Typical symptom	Fix
1NF	Lists or repeating groups in a column	`items = 'MUG-1x2; TEE-1x1'`	One row per item
2NF	Dependency on part of a composite key	Product name repeated on every order line	Move to a table keyed by the part
3NF	Non-key → non-key dependency	Customer email repeated on every order	Move to a table keyed by the determinant
BCNF	Any determinant that is not a superkey	Overlapping candidate keys (see below)	Split on the offending dependency

11.5.3 Worked decomposition: from a flat export to BCNF

WORKED EXAMPLE

Worked example — Normalizing the shop's order export. The legacy export has one row per order line:

`order_lines_flat(order_id, order_date, customer_id, customer_email, customer_city, product_id, product_name, list_price_cents, quantity, unit_price_cents)`

Step 1 — 1NF. An even older format stored all items of an order in one text cell. Splitting that cell into one row per product makes every value atomic, giving the table above.

Step 2 — List the functional dependencies. From the business rules: - FD1: `order_id → order_date, customer_id` - FD2: `customer_id → customer_email, customer_city` - FD3: `customer_email → customer_id` (emails are unique) - FD4: `product_id → product_name, list_price_cents` - FD5: `order_id, product_id → quantity, unit_price_cents` (one line per product per order)

The only candidate key is `{order_id, product_id}`: together they determine every column, and neither does alone.

Step 3 — 2NF. FD1 and FD4 have determinants that are *part* of the key, so `order_date`, the customer columns and the product columns depend on part of the key. Split each partial dependency into its own table: `- orders_wide(order_id, order_date, customer_id, customer_email, customer_city) - products(product_id, product_name, list_price_cents) - order_lines(order_id, product_id, quantity, unit_price_cents)`

Step 4 — 3NF. In `orders_wide`, `order_id → customer_id → customer_email, customer_city` is a transitive dependency. Split on FD2: `- orders(order_id, order_date, customer_id) - customers(customer_id, customer_email, customer_city)`

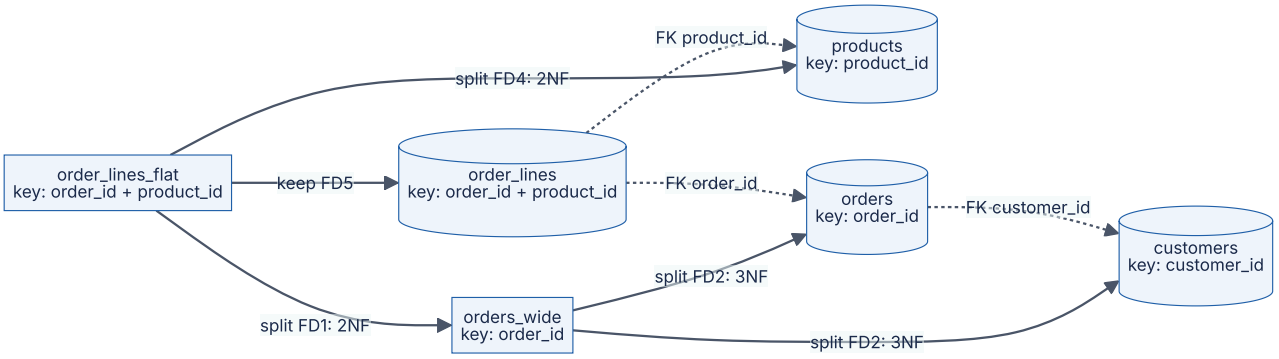
Step 5 — BCNF check. In `customers`, the determinants are `customer_id` and `customer_email`, both candidate keys. In every other table the only determinant is the primary key. All five tables are in BCNF.

Step 6 — Lossless-join check. A split of R into R_1 and R_2 is **lossless** (joining them back reproduces exactly the original rows, no more, no fewer) if the shared columns form a key of at least one side. Each split above shares a column that is a key of one side (`order_id`, `product_id`, `customer_id`), so joining the five tables reproduces the export exactly.

Interpretation. Ana's email is now stored once, products exist without orders, and deleting an order never loses product data. **Engineering implication.** Notice that

> `unit_price_cents` stayed in `order_lines` while `list_price_cents` moved to `products`. They are different facts: the price *charged* on that day versus the price *today*. Copying the price into the order line is not redundancy; it is history, and dropping it would make every past invoice change whenever marketing changes a price.

Figure 11.1 — Decomposing the flat order export into BCNF tables.



Reading the diagram. The rectangle on the left is the unnormalized input, middle rectangles are intermediate tables and cylinders are the final BCNF tables. Solid arrows are decomposition steps labeled with the dependency that justified them; dotted arrows are the foreign keys that make the decomposition lossless. These are join paths, not runtime flows, so there is no synchronous or asynchronous traffic here. The risk a decomposition introduces is a lost dependency (§11.5.4); the scaling cost is one more join per extra table, which denormalization later buys back.

11.5.4 When BCNF and 3NF disagree

Most 3NF schemas are already in BCNF; the exception needs overlapping candidate keys. Suppose each delivery depot is run by one carrier (`depot → carrier`) and each carrier serves a postal code from one depot (`postal_code, carrier → depot`). In `coverage(postal_code, carrier, depot)` the candidate keys are `{postal_code, carrier}` and `{postal_code, depot}`, so `depot → carrier` violates BCNF but not 3NF (`carrier` is part of a key). Splitting into `depots(depot, carrier)` and `coverage(postal_code, depot)` reaches BCNF but `postal_code, carrier → depot` can no longer be enforced by a unique constraint. BCNF decomposition is always lossless but not always **dependency-preserving**; 3NF can always achieve both [Ramakrishnan and Gehrke 2003].

11.5.5 Denormalization

DEFINITION

Definition — Denormalization. Deliberately storing a fact more than once, or storing a derived value, to make important reads cheaper, while accepting the cost of keeping the copies consistent.

Normalization optimizes for correct, cheap writes; some read paths (a product page with its review count, a revenue dashboard) would then need large joins or aggregations on every request. Common forms of denormalization:

1. **Stored aggregates** — `orders.total_cents`, `products.review_count`, updated in the same transaction as the rows they summarize.
2. **Copied columns** — `order_lines.product_name` kept so invoices render without a join (and, like the unit price, it is also a historical fact).
3. **Materialized views** — a stored query result, refreshed periodically (`REFRESH MATERIALIZED VIEW CONCURRENTLY` in PostgreSQL).
4. **Read models outside the database** — caches (Chapter 20), search indexes (Chapter 16) or CQRS projections fed by change data capture (§32.4, §32.6). These are updated asynchronously and are therefore eventually consistent.

RECOMMENDATION

Recommendation — Normalize the source of truth; denormalize with an owner. Add a redundant copy only for a measured read problem, and name what keeps it consistent: the same transaction, a trigger, or an asynchronous pipeline (eventually consistent, needs a reconciliation job). A copy with no owner will drift.

Table 11.3 — Failure scenarios in schema design.

What goes wrong	Why	Detection	Recovery	Prevention
Two customers rows with the same email	Natural key not declared `UNIQUE`	Duplicate-detection query; login bugs	Merge rows, repoint FKs	`UNIQUE` on every candidate key
Denormalized total disagrees with items	Update path forgot the aggregate	Nightly reconciliation query	Recompute from source rows	Update in the same transaction; reconciliation job
Old invoices change when prices change	Price stored only in `products`	Customer complaints, audit	Restore from backups or logs	Store price-at-purchase in `order_lines`
Orphaned order lines	No FK constraint	Anti-join `NOT EXISTS` check	Delete or repair orphans	Declare FKs; `ON DELETE CASCADE` for owned rows

11.6 SQL and joins

11.6.1 SQL as a declarative language

> DEFINITION
>
> **Definition — SQL.** The Structured Query Language, standardized by ANSI/ISO, with sublanguages for defining schema (DDL: `CREATE`, `ALTER`, `DROP`), manipulating data (DML: `SELECT`, `INSERT`, `UPDATE`, `DELETE`) and controlling transactions (`BEGIN`, `COMMIT`, `ROLLBACK`). Every major database implements the core standard plus its own dialect.

A `SELECT` is evaluated, logically, in the order `FROM`/`JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT`, which is why a `SELECT` alias cannot be used in `WHERE`. The physical execution order may differ (a filter pushed below a join) as long as the result is identical.

11.6.2 Join types

A **join** combines rows from two tables that satisfy a condition. Using the shop tables:

```
        -- Inner join: orders with their customer (orders without a customer cannot ex-
ist: FK)
        SELECT o.order_id, c.email, o.total_cents
        FROM orders o
        JOIN customers c ON c.customer_id = o.customer_id;

        -- Left outer join: every customer, with order count (0 for customers who never
ordered)
        SELECT c.customer_id, c.email, COUNT(o.order_id) AS orders
        FROM customers c
        LEFT JOIN orders o ON o.customer_id = c.customer_id
        GROUP BY c.customer_id, c.email;

        -- Semi-join: customers who bought product 42 (each customer once, however many
orders)
        SELECT c.email FROM customers c
        WHERE EXISTS (SELECT 1 FROM orders o JOIN order_items i ON i.order_id =
o.order_id
                      WHERE o.customer_id = c.customer_id AND i.product_id = 42);

        -- Anti-join: customers with no orders at all
        SELECT c.email FROM customers c
        WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

What to notice:

- `COUNT(o.order_id)` counts non-NULL values, so customers with no orders get 0; `COUNT(*)` would wrongly give 1, because the outer join produces one row padded

with NULLs.
- `EXISTS` expresses "at least one match" without duplicating the customer per matching row; a plain join followed by `DISTINCT` gives the same answer with more work.
- A `RIGHT JOIN` is a left join with the tables swapped; a `FULL OUTER JOIN` keeps unmatched rows from both sides. All four statements run unchanged in SQLite and PostgreSQL.

11.6.3 Join algorithms

The SQL says *what* to join; the database picks *how*. There are three families, and every relational engine uses some of them [Ramakrishnan and Gehrke 2003; Graefe 1993].

First-principles explanation

To match 1,000 wedding guests to their table assignments, you could read the whole assignment list once per guest (**nested loop**), file the assignments on cards by surname and look each guest up (**hash join**), or sort both lists and walk them together (**sort-merge join**). The fastest choice depends on list sizes, existing order and desk space (memory).

How it works internally

1. **Nested-loop join (NLJ).** For each row of the *outer* input, scan the *inner* input and emit matching pairs. **Block NLJ** reads the outer side in memory-sized chunks so the inner side is scanned once per chunk instead of once per row. **Index NLJ** replaces the inner scan with an index lookup: for each outer row, descend the inner table's B+ tree on the join column. Index NLJ is the best algorithm when the outer side is small, which is the case for most OLTP queries ("this customer's orders").
2. **Hash join.** *Build* phase: read the smaller input and insert each row into an in-memory hash table keyed on the join column. *Probe* phase: stream the larger input and look up each row's key. It needs an equality condition. If the build side does not fit in memory, both inputs are first **partitioned** by a hash of the key into spill files on disk so that each partition pair fits (the **Grace hash join**).
3. **Sort-merge join.** Sort both inputs on the join key (or read them in order from an index), then advance two cursors in step, emitting matches. It handles equality and also benefits range conditions; it is attractive when inputs are already sorted or when the output must be sorted anyway.

```
        from collections import defaultdict

        customers = [(1, "Berlin"), (2, "Austin"), (3, "Pune")]           #
(customer_id, city)
        orders = [(10, 1, 2500), (11, 3, 900), (12, 1, 4100), (13, 4, 50)]  # (order_id,
customer_id, total)

        def nested_loop_join(outer, inner, okey, ikey):
            for o in outer:                       # for every outer row ...
                for i in inner:                   # ... scan the whole inner input
                    if o[okey] == i[ikey]:
                        yield o + i

        def hash_join(build, probe, bkey, pkey):
            table = defaultdict(list)
            for b in build:                       # build phase: hash the smaller input
                table[b[bkey]].append(b)
            for p in probe:                       # probe phase: one lookup per probe row
                for b in table.get(p[pkey], ()):
                    yield b + p

        def sort_merge_join(left, right, lkey, rkey):
            left, right = sorted(left, key=lambda r: r[lkey]), sorted(right, key=lambda
r: r[rkey])
            i = j = 0
            while i < len(left) and j < len(right):
                lv, rv = left[i][lkey], right[j][rkey]
                if lv < rv:
                    i += 1
                elif lv > rv:
                    j += 1
                else:                             # emit every pair in the matching
groups
                    j_end = j
                    while j_end < len(right) and right[j_end][rkey] == lv:
                        j_end += 1
                    while i < len(left) and left[i][lkey] == lv:
                        for r in right[j:j_end]:
                            yield left[i] + r
                        i += 1
                    j = j_end

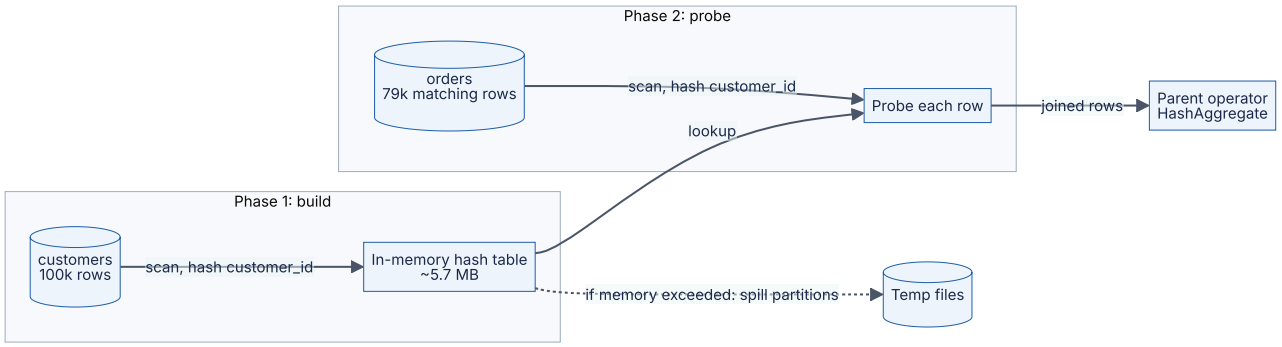
        for name, rows in [("nested loop", nested_loop_join(customers, orders, 0, 1)),
                           ("hash", hash_join(customers, orders, 0, 1)),
                           ("sort-merge", sort_merge_join(customers, orders, 0, 1))]:
            print(f"{name:<12}", sorted(rows, key=lambda r: r[2]))
```

What to notice:

- All three print the same three rows: Berlin with orders 10 and 12, Pune with order 11. Order 13 (customer 4) has no match and disappears, as an inner join requires.
- The nested loop performs $3 \times 4 = 12$ comparisons; the hash join performs 3 inserts and 4 probes. With a million rows on each side, that difference is 10^{12} versus 2×10^{6}

operations.

Figure 11.2 — Hash join: build on the smaller input, probe with the larger.



Reading the diagram. Cylinders are tables or temporary files; rectangles are in-memory operator state; every solid arrow is a synchronous in-process data flow. The build phase must finish before probing starts (blocking), while the probe phase passes each `orders` row upward immediately (pipelined). The dotted arrow is the degradation path: if the hash table exceeds the memory budget (`work_mem` in PostgreSQL), both inputs are partitioned to disk and re-read. Sizes come from the real plan in §11.10. Scaling: more memory, an earlier filter on the build side, or parallel probe workers.

Mathematical foundations

The classic way to compare join algorithms counts **page I/Os**, because on a disk-resident database they dominate [Ramakrishnan and Gehrke 2003]. Let the outer relation R have M pages and the inner relation S have N pages.

FORMULA

I/O cost of join algorithms

$$C_{\text{tuple NLJ}} = M + p_R \cdot M \cdot N \qquad C_{\text{block NLJ}} = M + \lceil \frac{M}{B-2} \rceil N$$

$$C_{\text{index NLJ}} = M + p_R \cdot M \cdot c_{\text{idx}} \qquad C_{\text{Grace hash}} \approx 3(M+N) \qquad C_{\text{sort-merge}} \approx 3(M+N) \text{ to } 5(M+N)$$

Symbol	Meaning	Units
M, N	Pages in the outer relation R and inner relation S	pages
p_R	Rows per page of R	rows/page
B	Buffer pages available to the join (one for output, one for the inner scan)	pages
c_{idx}	Page reads to find matching inner rows through an index (descent plus row fetch)	pages/row
C	Total page reads and writes	page I/Os

The hash and sort-merge figures assume enough memory to finish in two passes (roughly $B > \sqrt{\min(M,N)}$ for hash, $B > \sqrt{\max(M,N)}$ for sort); with fewer buffers, more passes are needed. If the build side fits entirely in memory, a hash join reads each input once: $M + N$.

WORKED EXAMPLE

Worked example — Joining `orders` with `order_items`. The PostgreSQL tables of §11.10 have $M = 15,385$ pages for `orders` (1,000,000 rows, so $p_R = 65$) and $N = 18,383$ pages for `order_items` (2,500,000 rows), with 8 KiB pages. Assume $B = 1,002$ buffer pages (about 8 MiB).

1. Tuple NLJ: $15,385 + 65 \times 15,385 \times 18,383 = 15,385 + 1.84 \times 10^{10} \approx 1.8 \times 10^{10}$ I/Os. At 100 µs per random read that is 1.8×10^{6} s, about 21 days.
2. Block NLJ: $\lceil 15,385/1,000 \rceil = 16$ chunks; $15,385 + 16 \times 18,383 = 309,513$ I/Os, about 31 s at 100 µs each (much less with sequential reads).
3. Grace hash join: $3 \times (15,385 + 18,383) = 101,304$ I/Os, about 10 s; if `orders` fits in memory, $33,768$ I/Os.
4. Index NLJ for **one customer's** 10 orders: the outer side is 10 rows; each needs about 4 page reads in the `order_items` primary-key index plus heap: roughly $10 \times 4 = 40$ I/Os, about 4 ms cold and well under 1 ms when cached (the real plan in §11.10 took 0.43 ms).

Interpretation. For a whole-table join, the naive nested loop is impossible, block NLJ is tolerable and the hash join wins. For a selective OLTP query, the index nested loop wins by four orders of magnitude, because it touches only the rows it needs.
Engineering implication. There is no universally best join algorithm; the optimizer must know *how many rows* each side produces, which is why cardinality estimation (§11.10) decides performance.

Table 11.4 — Join algorithms compared.

Algorithm	Join conditions	Memory	Best when	Weakness
Nested loop (block)	Any	Small	Tiny inputs; non-equality conditions	Quadratic on large inputs
Index nested loop	Equality or range on indexed inner column	Small	Small outer side, indexed inner (OLTP)	Random I/O per outer row if outer is large
Hash join	Equality only	Build side (spills if needed)	Large unsorted inputs, analytics	Blocking build; spills when memory is short
Sort-merge join	Equality (and some ranges)	Sort buffers	Inputs already sorted or output needs sorting	Sorting cost if inputs are unsorted

As of 2026, PostgreSQL implements all three. MySQL implements nested-loop variants (including batched key access) and, since 8.0.18, hash joins; it has no sort-merge join [MySQL Docs: Hash Join].

11.7 Indexes

11.7.1 What an index is

DEFINITION

Definition — Index. A secondary data structure, kept in sync with a table by the database, that maps values of one or more **key columns** to the locations of the rows holding them, so that rows can be found without scanning the whole table. Most relational indexes are B+ trees (§11.8); others include hash, GiST, GIN and BRIN indexes in PostgreSQL and full-text and spatial indexes in MySQL.

First-principles explanation

A book's index finds "isolation levels" without reading 900 pages because it is sorted, small and points to page numbers. It must be updated when the book changes, takes paper, and is useless for a word on every page ("the"). Database indexes behave the same way: fast for selective conditions, extra work on every write, extra storage, no help when a query needs much of the table.

Why does this exist?

Without an index, `SELECT * FROM orders WHERE customer_id = 4242` read all 15,385 pages of the table and took 70 ms to find 10 rows (§11.10). With an index it read 16 pages and took 0.14 ms.

11.7.2 Clustered and secondary indexes

DEFINITION

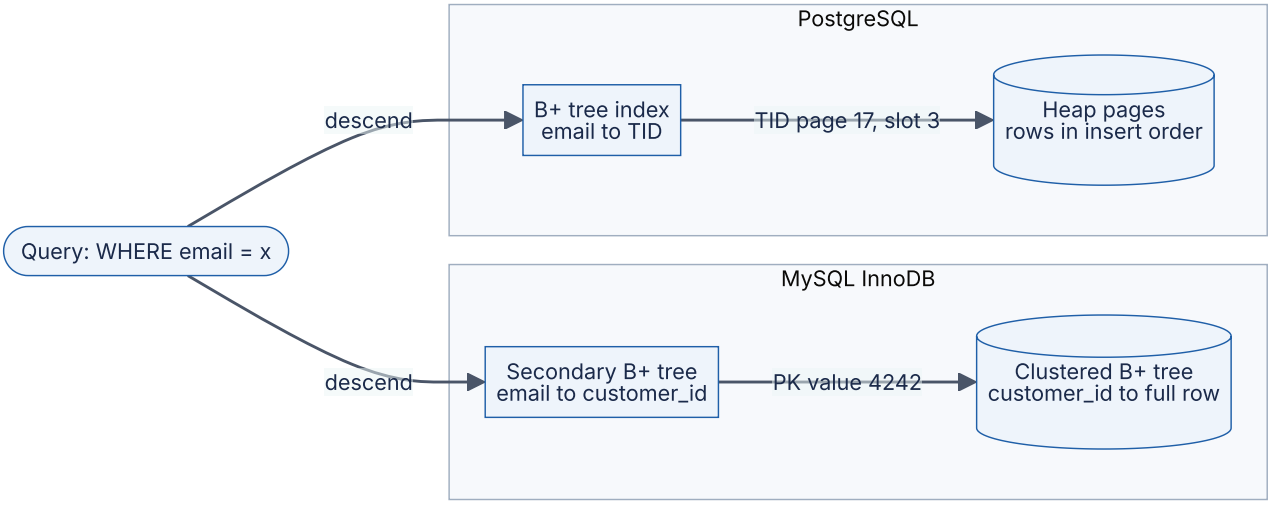
Definition — Clustered and secondary index. In a **clustered index** (an index-organized table), the leaf level of the index *is* the table: rows are stored in key order inside the index's leaf pages. A table can have at most one. A **secondary index** stores, for each key, a pointer to the row; the table can have many.

The two most popular open-source databases make opposite choices:

- **InnoDB (MySQL)** stores every table as a B+ tree clustered on the primary key. If you declare none, it uses the first `UNIQUE NOT NULL` index, or else generates a hidden 6-byte row ID. Secondary index leaves store the indexed columns plus the *primary-key value*. A secondary lookup therefore descends two trees: the secondary index to find the primary key, then the clustered index to find the row [MySQL Docs: Clustered and Secondary Indexes].

- **PostgreSQL** stores rows in an unordered **heap** file. Every index, including the primary key, is secondary; its leaves store a **TID** (tuple identifier: page number and slot), so a lookup descends one tree and then reads one heap page. The `CLUSTER` command reorders the heap once, but later inserts are not kept in order.

Figure 11.3 — Secondary-index lookup paths in InnoDB and PostgreSQL.



Reading the diagram. The same query descends two storage designs; all arrows are synchronous reads inside one database process. InnoDB's secondary index yields a primary-key *value*, which drives a second descent of the clustered index; PostgreSQL's index yields a physical TID, followed by one heap-page read. The logical pointer survives row moves, so clustered-index splits never touch secondary indexes; the physical pointer is cheaper to follow but must be updated in every index when an update moves a row (§11.14). Failure point: a long InnoDB primary key is copied into every secondary index. Scaling: keep InnoDB keys short and monotonic; leave free space in PostgreSQL heap pages (`fillfactor`).

11.7.3 Composite indexes and the leftmost-prefix rule

A **composite index** on `(customer_id, created_at)` sorts entries by `customer_id`, and within equal `customer_id` by `created_at`, like a phone book sorted by surname and then first name. You can use it to find everyone named "Rao" (leftmost column), or "Rao, Priya" (both columns), but not everyone named "Priya": those entries are scattered across the whole book. This is the **leftmost-prefix rule**: a B+ tree index can seek on a prefix of its columns, with equality on all but possibly the last column used, which may be a range.

The index on `(customer_id, created_at)` serves:

- `WHERE customer_id = ?`
- `WHERE customer_id = ? AND created_at >= ?`

- `WHERE customer_id = ? ORDER BY created_at DESC LIMIT 20` (no sort needed: entries are already in order)

and does *not* efficiently serve `WHERE created_at >= ?`. Some engines can **skip-scan** (seek once per distinct leading value), which helps only when the leading column has few distinct values: MySQL since 8.0.13, and PostgreSQL from version 18 (as of 2026).

> RECOMMENDATION
>
> **Recommendation — Column order in composite indexes.** Put columns compared with equality first, then the column used for a range or `ORDER BY`. Selectivity of the leading column matters less than whether queries can use the prefix at all. One well-ordered composite index often replaces two or three single-column indexes.

11.7.4 Covering, partial and expression indexes

- A **covering index** contains every column a query needs, so it can be answered from the index alone (an **index-only scan**). PostgreSQL 11+ adds payload columns with `INCLUDE`; InnoDB secondary indexes always cover the primary key. PostgreSQL still visits heap pages not marked all-visible in the **visibility map**, which VACUUM maintains (§11.14).
- A **partial index** indexes only rows matching a predicate: `CREATE INDEX ... ON orders (created_at) WHERE status = 'pending'`. If 1% of orders are pending, the index is 1% of the size. PostgreSQL and SQLite support partial indexes; MySQL does not (as of 8.4).
- An **expression index** indexes a computed value such as `lower(email)` (MySQL 8.0.13+: functional key parts).

```
        import sqlite3

        db = sqlite3.connect(":memory:")
        db.executescript("""
        CREATE TABLE orders (
          order_id    INTEGER PRIMARY KEY,          -- rowid alias: the table's clus-
tered key
          customer_id INTEGER NOT NULL,
          status      TEXT    NOT NULL,
          total_cents INTEGER NOT NULL,
          created_at  TEXT    NOT NULL
        );
        CREATE INDEX orders_cust_created ON orders (customer_id, created_at);
        CREATE INDEX orders_pending ON orders (created_at) WHERE status = 'pending';
        """)
        db.executemany(
            "INSERT INTO orders (customer_id, status, total_cents, created_at) VALUES
(?,?,?,?)",
            [(i % 1000, "pending" if i % 100 == 0 else "shipped", i, f"2026-10-{1 + i %
28:02d}")
             for i in range(50_000)])

        queries = {
            "both columns":        "SELECT * FROM orders WHERE customer_id = 7 AND
created_at >= '2026-10-15'",
            "leftmost column only": "SELECT * FROM orders WHERE customer_id = 7",
            "second column only":  "SELECT * FROM orders WHERE created_at >= '2026-10-
15'",
            "covering":            "SELECT created_at FROM orders WHERE customer_id =
7",
            "partial index":       "SELECT order_id FROM orders WHERE status = 'pending'
AND created_at < '2026-10-03'",
        }
        for label, sql in queries.items():
            plan = db.execute("EXPLAIN QUERY PLAN " + sql).fetchall()
            print(f"{label:<21} -> {plan[0][3]}")
```

Output (SQLite 3.45):

```
both columns          -> SEARCH orders USING INDEX orders_cust_created (customer_id=?
AND created_at>?)
leftmost column only  -> SEARCH orders USING INDEX orders_cust_created (customer_id=?)
second column only    -> SCAN orders
covering              -> SEARCH orders USING COVERING INDEX orders_cust_created
(customer_id=?)
partial index         -> SEARCH orders USING COVERING INDEX orders_pending
(created_at<?)
```

What to notice:

- `SEARCH` means an index seek; `SCAN` means reading the whole table. Filtering on the second column alone falls back to a scan: the leftmost-prefix rule in action.

- The covering query asks only for `created_at`, which is in the index, so the table is never read.
- The partial-index query is covering too, because SQLite stores the rowid (`order_id`) in every index entry, just as InnoDB stores the primary key.

11.7.5 When an index does not help

Mathematical foundations

An index lookup through a secondary index costs roughly one *random* page read per matching row (when the rows are scattered across the table), while a full scan costs one *sequential* page read per table page. Above some fraction of matching rows, the scan wins.

FORMULA

Break-even selectivity for a secondary-index scan

$$s^{*} = \frac{P \cdot c_{\text{seq}}}{n \cdot c_{\text{rand}}}$$

Symbol	Meaning	Units
s^{*}	Fraction of rows matched above which a full scan is cheaper	dimensionless
P	Pages in the table	pages
n	Rows in the table	rows
c_{seq}	Cost of a sequential page read	cost units/page
c_{rand}	Cost of a random page read	cost units/page

WORKED EXAMPLE

Worked example — When does PostgreSQL stop using the index on `orders`? $P = 15,385$ pages, $n = 1,000,000$ rows, and PostgreSQL's default cost parameters are `seq_page_cost` $= 1$ and `random_page_cost` $= 4$.

1. Full scan cost: $15,385 \times 1 = 15,385$ units.
2. Index cost for fraction s: $s \times 10^6 \times 4$ units (ignoring the small index-page cost).
3. Break-even: $s^* = 15,385/(4 \times 10^6) = 0.0038$, about 0.4% of rows, or 3,800 rows.

Interpretation. A query matching 10 rows uses the index; one matching 50% of rows (`customer_id > 50000` in §11.10) is planned as a sequential scan, which is exactly what PostgreSQL chose. **Engineering implication.** Indexes serve *selective* predicates. PostgreSQL softens the cliff with **bitmap scans**, which collect matching TIDs, sort them by page and read each heap page once; and correlation statistics lower the estimated cost when the table is physically ordered by the indexed column. On SSDs, many teams lower `random_page_cost` to about 1.1–2 because random and sequential reads cost nearly the same.

Table 11.5 — Index variants.

Variant	What it stores	Speeds up	Costs	Engine support
Clustered (primary)	Full rows in key order	PK lookups and PK range scans	Wide secondary indexes (InnoDB)	InnoDB, SQL Server, SQLite rowid tables
Secondary	Key + row pointer (TID or PK)	Selective lookups on other columns	One extra structure to update per write	All
Composite	Several columns, lexicographic order	Prefix equality + range/sort	Useless without the leading column	All
Covering / `INCLUDE`	Extra payload columns	Index-only scans	Larger index	PostgreSQL 11+, SQL Server; InnoDB via PK
Partial	Only rows matching a predicate	Queries on a small, hot subset	Query must imply the predicate	PostgreSQL, SQLite (not MySQL)
Expression / functional	Computed values	Queries on the same expression	Expression evaluated on write	PostgreSQL, MySQL 8.0.13+, SQLite

COMMON MISTAKE

Mistake — "Add an index on every column to be safe." A table with eight indexes turns one row insert into nine B+ tree insertions, inflates storage and the buffer pool's working set, and in PostgreSQL prevents heap-only-tuple updates of indexed columns (§11.14). Index the queries you run; drop indexes whose `pg_stat_user_indexes.idx_scan` stays at zero.

COMMON MISTAKE

Mistake — Wrapping the indexed column in a function. `WHERE date(created_at) = '2026-10-30'` cannot use an index on `created_at`, because the index is sorted by `created_at`, not by `date(created_at)`. Rewrite as a range: `WHERE created_at >= '2026-10-30' AND created_at < '2026-10-31'`. The same applies to implicit type casts, such as comparing a text column with a number in MySQL.

11.8 B-trees and B+ trees

11.8.1 Structure and operations

DEFINITION

Definition — B-tree and B+ tree. A **B-tree** is a balanced search tree whose nodes are disk pages holding many sorted keys; every leaf is at the same depth, and nodes split and merge to stay between half-full and full [Bayer and McCreight 1972]. A **B+ tree** is the variant used by almost every relational database: internal nodes hold only separator keys and child pointers, all keys and values (or row pointers) live in the leaves, and leaves are linked to their siblings so range scans can walk sideways [Comer 1979; Petrov 2019].

Why does this exist?

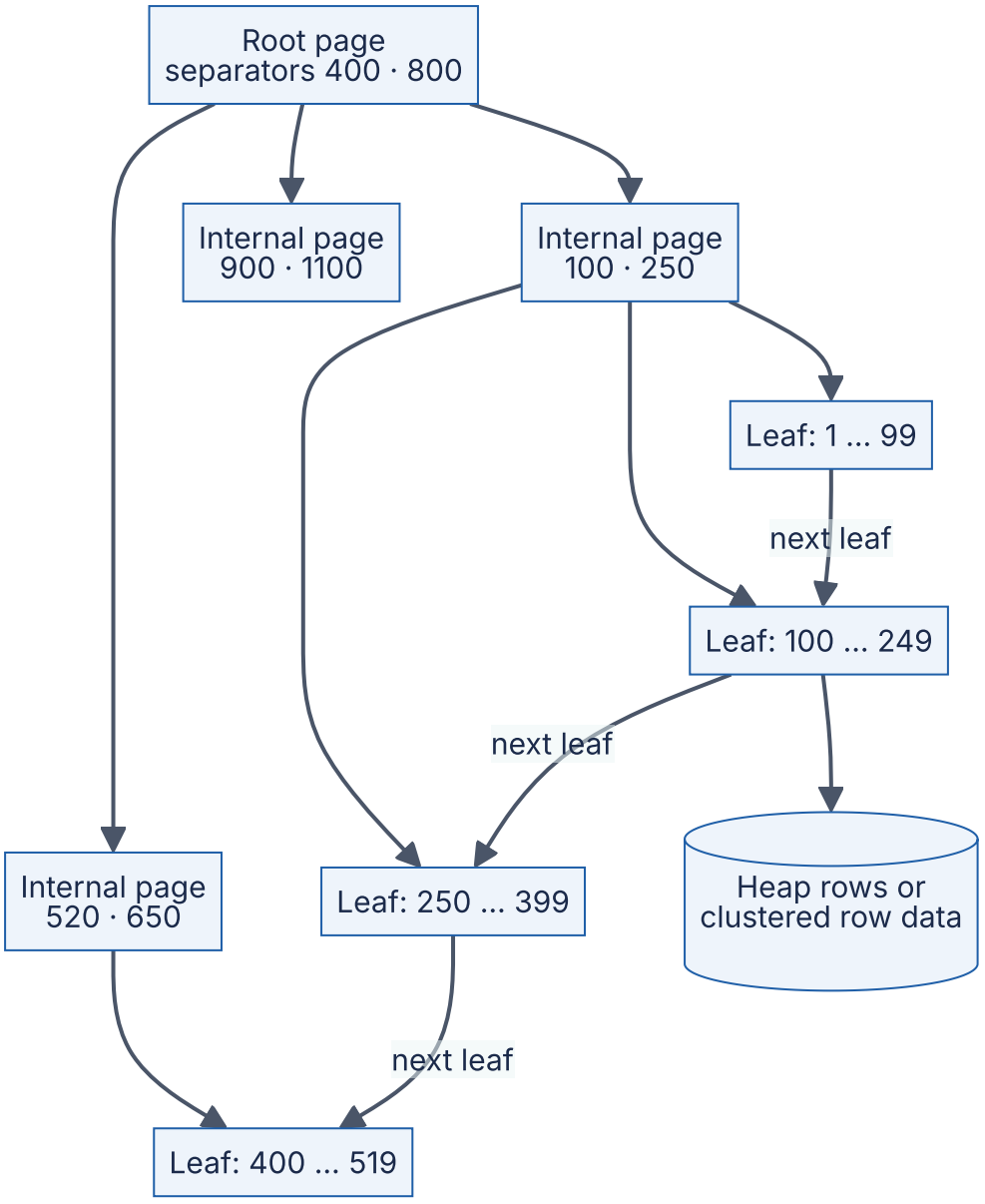
A binary search tree needs about 30 levels for a billion keys (§4.4.2): 30 dependent disk reads. The B-tree makes each node one page holding hundreds of keys, so the tree is wide and shallow. Keeping values only in leaves (the B+ variant) makes internal nodes denser, so more of the tree fits in memory, and sibling links turn range predicates into a sequential walk.

How it works internally

1. **Search.** Start at the root page. Binary-search its separator keys in memory to choose a child pointer. Repeat until a leaf; binary-search the leaf.
2. **Range scan.** Search for the lower bound, then follow leaf sibling links until the upper bound.
3. **Insert.** Search for the target leaf and insert the key in order. If the leaf overflows, **split** it into two half-full leaves and insert a separator key into the parent; if the parent overflows, split it too. A split of the root creates a new root, which is the only way the tree grows taller, so all leaves stay at the same depth.
4. **Delete.** Remove the key; if a node becomes too empty, borrow from or merge with a sibling (many engines merge lazily or not at all, leaving reclaimable space).

5. **Concurrency and crash safety.** Splits modify several pages, so engines latch pages briefly and log every change in the write-ahead log; PostgreSQL uses Lehman and Yao's right-links so readers never block on a concurrent split [Lehman and Yao 1981]. Details are in §13.5–§13.6.

Figure 11.4 — A B+ tree on `order_id` with three levels.



Reading the diagram. Each rectangle is one fixed-size page (8 KiB in PostgreSQL, 16 KiB in InnoDB). A search follows child pointers synchronously, one page per level: for `order_id` 180 the root goes left (180 < 400) and the internal page picks the second leaf. The "next leaf" arrows are sibling links that range scans walk. The cylinder is the storage boundary below the leaves: a heap row (PostgreSQL) or the row itself (InnoDB clustered index). Failure points: a split changes several pages atomically, hence WAL logging

(§13.5), and a cold cache makes every level an I/O. Scaling: multiplying the data by the fan-out adds one level.

11.8.2 Fan-out, height and I/Os per lookup

§4.4.3 derived $h = \lceil \log_f n \rceil$ for a tree whose leaves hold one key each. In a real B+ tree each leaf holds L entries, so only the internal levels need the logarithm.

FORMULA

Height of a B+ tree with L entries per leaf

$$h = 1 + \left\lceil \log_f \left\lceil \frac{n}{L} \right\rceil \right\rceil, \qquad f = \left\lfloor \frac{u \cdot P}{e_{\text{int}}} \right\rfloor, \qquad L = \left\lfloor \frac{u \cdot P}{e_{\text{leaf}}} \right\rfloor$$

Symbol	Meaning	Units
h	Levels from root to leaf (page reads per lookup with a cold cache)	levels
n	Entries (rows) indexed	rows
P	Page size	bytes
u	Average fraction of each page holding entries (fill factor minus headers)	dimensionless
e_{int}	Bytes per internal entry (separator key + child pointer + overhead)	bytes
e_{leaf}	Bytes per leaf entry (full row for a clustered index; key + pointer for a secondary index)	bytes
f	Fan-out of internal pages	children/page
L	Entries per leaf page	entries/page

WORKED EXAMPLE

Worked example — A billion-row `orders` table in InnoDB with 16 KiB pages. Assume $u = 0.8$, a `bigint` primary key with 16-byte internal entries (as in §4.4.3), and 200-byte rows.

1. Usable bytes per page: $0.8 \times 16,384 = 13,107$ bytes.
2. Fan-out: $f = \lfloor 13,107/16 \rfloor = 819$. Rows per leaf: $L = \lfloor 13,107/200 \rfloor = 65$.
3. Leaf pages: $\lceil 10^9/65 \rceil = 15,384,616$ (about 235 GiB of row data).
4. Internal levels: $\lceil 15,384,616/819 \rceil = 18,785$ pages, then $\lceil 18,785/819 \rceil = 23$, then 1 root. Check with the formula: $\log_{819}(15,384,616) = 16.55/6.71 = 2.47$, rounded up to 3, plus the leaf level: $h = 4$.
5. Memory for all internal pages: $(1 + 23 + 18,785) \times 16$ KiB $= 18,809 \times 16$ KiB ≈ 294 MiB.
6. A secondary index on `customer_id` (8-byte key + 8-byte primary key per leaf entry, $L = 819$) has $\lceil 10^9/819 \rceil = 1,221,002$ leaves (about 18.6 GiB), then 1,491, 2 and 1 pages: also 4 levels, with only 23 MiB of internal pages.

I/Os per lookup:

- Cold cache: 4 page reads for a primary-key lookup.
- Internal pages cached (294 MiB is trivial for a database server): **1 random read** per primary-key lookup, about 0.1 ms on NVMe (§4.17).
- Secondary-index lookup in InnoDB: 1 read for the secondary leaf + 1 for the clustered leaf = **2 reads**. In PostgreSQL: index leaf + heap page = 2 reads, similarly.

Interpretation. A billion rows cost only one more level than a million rows (a million rows needs $1 + \lceil \log_{819} 15,385 \rceil = 3$ levels). **Engineering implication.** Index lookups stay fast at any table size you will meet in one database; what grows is the *leaf* working set. If the hot rows' leaves (here up to 235 GiB) do not fit in the buffer pool, every lookup pays a real I/O, and that, not tree height, is the scaling limit. Wide keys shrink f and L; random keys spread the hot set across all leaves.

```
        import math

        def bplus_tree(n_rows, page_bytes=16 * 1024, fill=0.8, internal_entry=16,
leaf_entry=200):
            """Return (fan-out, entries per leaf, pages per level from root to
leaves)."""
            usable = page_bytes * fill
            fanout = int(usable // internal_entry)          # child pointers per inter-
nal page
            per_leaf = int(usable // leaf_entry)            # entries per leaf page
            levels = [math.ceil(n_rows / per_leaf)]         # leaf level first
            while levels[-1] > 1:
                levels.append(math.ceil(levels[-1] / fanout))
            return fanout, per_leaf, levels[::-1]

        for label, leaf_entry in [("clustered, 200-byte rows", 200), ("secondary, 8-byte
key + 8-byte PK", 16)]:
            f, L, pages = bplus_tree(10**9, leaf_entry=leaf_entry)
            internal_mib = sum(pages[:-1]) * 16 / 1024
            leaf_gib = pages[-1] * 16 / 1024**2
            print(f"{label}: fan-out={f}, entries/leaf={L}, height={len(pages)}")
            print(f"  pages per level (root..leaf) = {pages}")
            print(f"  internal levels = {internal_mib:,.0f} MiB, leaf level =
{leaf_gib:,.1f} GiB")
```

What to notice:

- Output: the clustered tree has pages per level `[1, 23, 18785, 15384616]` and the secondary tree `[1, 2, 1491, 1221002]`; both have height 4, matching the hand calculation.
- Change `leaf_entry` to 1,000 (wide rows) and the leaf level grows fivefold while the height stays 4: width costs memory before it costs levels.

> COMMON MISTAKE
>
> **Mistake — "B-tree and B+ tree are the same thing."** In a classic B-tree, values can live in internal nodes, so a range scan must move up and down the tree. Database "B-tree" indexes are B+ trees (PostgreSQL's `btree`, InnoDB's indexes): values only in leaves, leaves linked. In interviews, say "B+ tree" and explain the leaf links.

11.9 Query execution and plans

11.9.1 From SQL text to rows

Every statement passes through the same pipeline:

1. **Parse** — check syntax and build a parse tree.

2. **Analyze and rewrite** — resolve table and column names against the catalog, check permissions, expand views, and apply rewrite rules (for example, flattening a sub-query into a join).
3. **Plan (optimize)** — enumerate alternative physical plans, estimate the cost of each using table statistics, and choose the cheapest (§11.10).
4. **Execute** — run the plan's operators against the storage engine, which reads pages through the buffer pool (§13.4).

11.9.2 The Volcano (iterator) model

> DEFINITION
>
> **Definition — Volcano (iterator) model.** An execution model in which a query plan is a tree of operators, each implementing the same interface — `open()`, `next()` and `close()` — and each `next()` call pulls one row from the operator's children on demand [Graefe 1994].

First-principles explanation

In a kitchen line, the waiter asks plating for the next dish, plating asks the grill, the grill asks the fridge; nobody cooks a thousand steaks in advance. This **pull model** lets operators compose freely, keeps memory small for streaming operators, and lets a `LIMIT 20` stop the scans below after 20 rows.

Sort, hash aggregation and the build side of a hash join cannot emit a row until they have consumed all input. These **blocking** operators need memory proportional to their input and spill to disk when it runs out; in `EXPLAIN`, their startup cost is close to their total cost.

```
        class Operator:
            """Volcano iterator: every operator exposes open(), next() and close()."""
            def open(self): pass
            def next(self): raise NotImplementedError   # returns one row, or None when
exhausted
            def close(self): pass

        class SeqScan(Operator):
            def __init__(self, rows): self.rows = rows
            def open(self): self.pos = 0
            def next(self):
                if self.pos == len(self.rows): return None
                self.pos += 1
                return self.rows[self.pos - 1]

        class Filter(Operator):
            def __init__(self, child, pred): self.child, self.pred = child, pred
            def open(self): self.child.open()
            def next(self):
                while (row := self.child.next()) is not None:
                    if self.pred(row): return row
                return None
            def close(self): self.child.close()

        class HashJoin(Operator):
            def __init__(self, build, probe, bkey, pkey):
                self.build, self.probe, self.bkey, self.pkey = build, probe, bkey, pkey
            def open(self):                       # blocking: consumes the whole build
side
                self.table, self.pending = {}, []
                self.build.open()
                while (b := self.build.next()) is not None:
                    self.table.setdefault(b[self.bkey], []).append(b)
                self.build.close()
                self.probe.open()
            def next(self):                       # pipelined: one probe row at a time
                while not self.pending:
                    p = self.probe.next()
                    if p is None: return None
                    self.pending = [b + p for b in self.table.get(p[self.pkey], [])]
                return self.pending.pop()
            def close(self): self.probe.close()

        class Project(Operator):
            def __init__(self, child, cols): self.child, self.cols = child, cols
            def open(self): self.child.open()
            def next(self):
                row = self.child.next()
                return None if row is None else tuple(row[c] for c in self.cols)
            def close(self): self.child.close()

        class Limit(Operator):
            def __init__(self, child, n): self.child, self.n = child, n
            def open(self): self.child.open(); self.seen = 0
            def next(self):
                if self.seen == self.n: return None   # stop pulling: the scan below
```

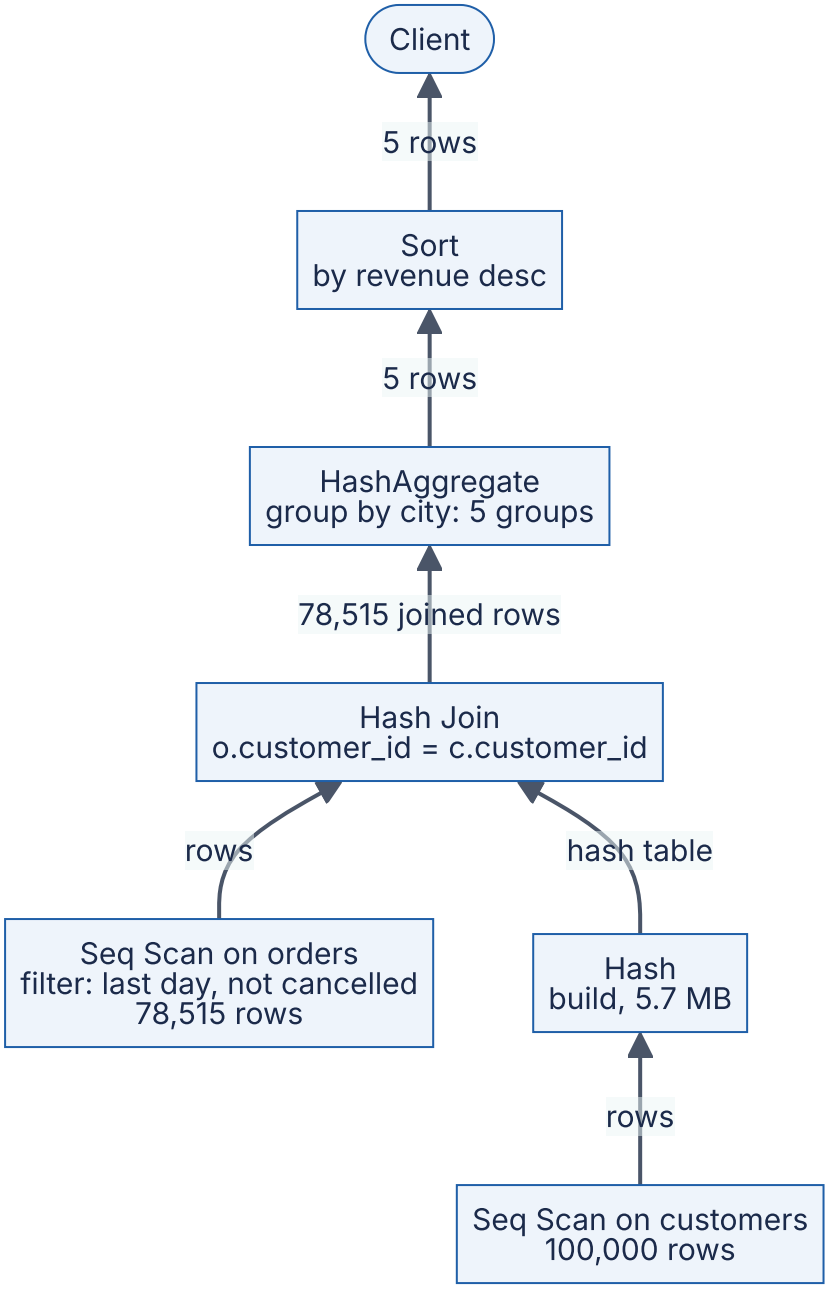
```
never finishes
                self.seen += 1
                return self.child.next()
            def close(self): self.child.close()

        # SELECT c.city, o.total FROM customers c JOIN orders o ON c.id = o.customer_id
        # WHERE o.total > 1000 LIMIT 2
        customers = [(1, "Berlin"), (2, "Austin"), (3, "Pune")]
        orders = [(10, 1, 2500), (11, 3, 900), (12, 1, 4100), (13, 2, 1800)]
        plan = Limit(Project(HashJoin(SeqScan(customers),
                                      Filter(SeqScan(orders), lambda r: r[2] > 1000), 0,
1),
                             cols=[1, 4]), 2)
        plan.open()
        while (row := plan.next()) is not None:
            print(row)
        plan.close()
```

What to notice:

- The output is `('Berlin', 2500)` and `('Berlin', 4100)`. Order 13 (Austin, 1800) also qualifies but is never produced: `Limit` stopped pulling after two rows, so the orders scan stopped early.
- `HashJoin.open()` drains its build child completely (blocking); `next()` handles one probe row at a time (pipelined).
- Every row costs several virtual function calls. Modern analytic engines reduce that overhead with **vectorized** execution (each `next()` returns a batch of about a thousand values) [Boncz et al. 2005] or by **compiling** the plan into machine code [Neumann 2011]. PostgreSQL remains a Volcano-style executor, with optional JIT compilation of expressions.

Figure 11.5 — The executed plan tree for “revenue by city over the last day”.



Reading the diagram. This is the real plan PostgreSQL 16 chose for the revenue query of §11.10, drawn bottom-up: rows flow upward while `next()` calls flow down, all synchronously inside one backend process. The scans are the storage boundary; `Hash`, `HashAggregate` and `Sort` are blocking and spill to temporary files if they exceed `work_mem`. The main failure point is a misestimated row count, which can make this plan win when an index-driven plan would be better. Scaling: an index on `orders(created_at)` would replace the million-row scan, and parallel workers could split it.

11.10 Query optimization

11.10.1 The cost-based optimizer

DEFINITION

Definition — Cost-based optimizer. The component that enumerates equivalent physical plans for a query (access paths, join orders, join algorithms), estimates each plan's cost from **statistics** about the data using a **cost model**, and returns the plan with the lowest estimated cost.

The design dates from IBM's System R. Selinger and colleagues introduced the three ideas every optimizer still uses: estimate the **selectivity** of each predicate from catalog statistics; cost plans in a weighted mix of page I/O and CPU; and find the best join order by **dynamic programming**, building the best plan for every subset of tables from the best plans of smaller subsets, while remembering plans that produce **interesting orders** (sorted output that a later merge join or `ORDER BY` can reuse) [Selinger et al. 1979].

Mathematical foundations

FORMULA

Size of the join-order search space

$$N_{\text{left-deep}} = n! \qquad N_{\text{bushy}} = \frac{(2(n-1))!}{(n-1)!} \qquad W_{\text{DP}} \approx n \cdot 2^{n-1}$$

Symbol	Meaning	Units
n	Number of tables joined	tables
$N_{\text{left-deep}}$	Join orders where every join's inner input is a base table	plans
N_{bushy}	All binary join trees, including joins of two intermediate results	plans
W_{DP}	Approximate subproblems examined by Selinger-style dynamic programming over left-deep plans	subproblems

WORKED EXAMPLE

Worked example — Ten tables. For $n = 10$: left-deep orders $10! = 3,628,800$; bushy trees $18!/9! \approx 1.76 \times 10^{10}$; dynamic programming examines about $10 \times 2^9 = 5,120$ subset extensions. **Interpretation.** Dynamic programming makes ten-way joins tractable, but 2^n still explodes: at $n = 20$ it is about 10 million. **Engineering implication.** PostgreSQL switches to a genetic (randomized) search at 12 `FROM` items by default (`geqo_threshold`), and stops reordering explicit joins beyond 8 (`join_collapse_limit`); very wide joins generated by ORMs or reporting tools can therefore get poor plans.

11.10.2 Cost model and statistics in PostgreSQL

PostgreSQL's costs are in arbitrary units anchored on one sequential page read. The defaults (verified on PostgreSQL 16) are `seq_page_cost` = 1, `random_page_cost` = 4, `cpu_tuple_cost` = 0.01, `cpu_index_tuple_cost` = 0.005 and `cpu_operator_cost` = 0.0025 [PostgreSQL Docs: Using EXPLAIN].

FORMULA

Cost of a sequential scan with one filter (PostgreSQL)

$$C_{\text{seq}} = P \cdot c_{\text{seq}} + n \cdot c_{\text{tuple}} + n \cdot k \cdot c_{\text{op}}$$

Symbol	Meaning	Units
P	Heap pages (`pg_class.relpages`)	pages
n	Rows (`pg_class.reltuples`)	rows
k	Operators evaluated per row in the filter	operators
c_{seq}, c_{tuple}, c_{op}	`seq_page_cost`, `cpu_tuple_cost`, `cpu_operator_cost`	cost units

To estimate how many rows survive a filter, the planner uses per-column statistics collected by `ANALYZE` from a random sample (visible in `pg_stats`): the number of distinct values, the **most common values** with their frequencies, a histogram of the rest, the null fraction and the physical **correlation** between column order and heap order.

FORMULA

Cardinality estimate under the independence assumption

$$\hat{r} = n \cdot \prod_{i=1}^{k} s_i, \qquad s_i \approx \text{freq}(v_i) \text{ or } \frac{1}{d_i}$$

Symbol	Meaning	Units
$\hat{r}$	Estimated rows satisfying all k predicates	rows
n	Rows in the table	rows
s_i	Selectivity of predicate i (fraction of rows passing it)	dimensionless
$\text{freq}(v_i)$	Sampled frequency of value v_i if it is a most-common value	dimensionless
d_i	Number of distinct values in the column	values

WORKED EXAMPLE

Worked example — Checking the planner's arithmetic on real plans. On the 1,000,000-row `orders` table (15,385 pages):

1. `EXPLAIN SELECT * FROM orders WHERE customer_id = 4242` reported `cost=0.00..27885.00`. By the formula: $15,385 \times 1 + 10^6 \times 0.01 + 10^6 \times 1 \times 0.0025 = 15,385 + 10,000 + 2,500 = 27,885$. Exact match.
2. Its row estimate was 10: with 100,000 distinct customers, $s = 1/100,000$ and $\hat{r} = 10^6/10^5 = 10$. Actual: 10.
3. Now a table `addr` of 1,000,000 rows where `city` determines `country` (every Pune row is in India). Each column has 5 values at about 20% each. For `WHERE city = 'Pune' AND country = 'IN'` the independence assumption gives $\hat{r} = 10^6 \times 0.2 \times 0.2 = 40,000$. PostgreSQL estimated 39,960; the actual count was 200,000, an underestimate of 5×.

Interpretation. The formulas are simple and exact for single predicates on well-sampled columns; the error comes from *correlation*, which the independence assumption ignores. **Engineering implication.** Errors multiply through joins: an estimate that is 5× too low feeding a join whose own estimate is 5× too low is 25× off, which is enough to choose a nested loop over a million rows. For correlated columns, PostgreSQL 10+ supports extended statistics: after `CREATE STATISTICS addr_city_country (dependencies) ON city, country FROM addr; ANALYZE addr;` the estimate became 198,500.

INDUSTRY EXAMPLE

Research evidence — How good are query optimizers? Leis and colleagues built the Join Order Benchmark on the real IMDB dataset (queries with many joins over correlated data) and measured PostgreSQL and several commercial systems. Cardinality estimates were routinely wrong by orders of magnitude, errors grew with the number of joins, and estimation errors mattered far more for plan quality than the cost model did [Leis et al. 2015]. *Evidence: Primary* — [Leis et al. 2015]

11.10.3 Reading EXPLAIN and EXPLAIN ANALYZE

`EXPLAIN` shows the chosen plan with estimates; `EXPLAIN ANALYZE` actually runs the query and adds measured times and row counts; `BUFFERS` adds page counts. (MySQL 8.0.18+ has `EXPLAIN ANALYZE` with a different format.) The following is real PostgreSQL 16 output for the order-history query after creating an index on `orders (customer_id, created_at DESC)`, with parallel query disabled for readability:

```
Limit  (cost=44.03..44.05 rows=10 width=32) (actual time=0.110..0.112 rows=10 loops=1)
  Buffers: shared hit=3 read=13
  ->  Sort  (cost=44.03..44.05 rows=10 width=32) (actual time=0.109..0.110 rows=10
loops=1)
        Sort Key: created_at DESC
        Sort Method: quicksort  Memory: 25kB
        ->  Bitmap Heap Scan on orders  (cost=4.50..43.86 rows=10 width=32) (actual
time=0.042..0.099 rows=10 loops=1)
              Recheck Cond: (customer_id = 4242)
              Heap Blocks: exact=10
              ->  Bitmap Index Scan on orders_customer_created_idx  (cost=0.00..4.50
rows=10 width=0) (actual time=0.028..0.028 rows=10 loops=1)
                    Index Cond: (customer_id = 4242)
Planning Time: 0.219 ms
Execution Time: 0.135 ms
```

How to read it:

1. Read from the most indented node outward: the bitmap index scan finds 10 TIDs, the heap scan fetches 10 pages, the sort orders them, the limit returns them.
2. `cost=startup..total` and `rows` are estimates; `actual time=first..last` is in milliseconds *per loop* (multiply by `loops`).
3. **Compare estimated and actual `rows` at every node.** A gap above about 10× points to stale statistics (`ANALYZE`), correlated columns (`CREATE STATISTICS`) or an unestimatable predicate (functions, casts).
4. `Buffers: shared hit=3 read=13`: 3 pages from the buffer pool, 13 from the OS or disk.

With only 10 orders per customer, sorting 10 rows was estimated cheaper than walking the index in `created_at` order; for a customer with thousands of orders the plan switches to an index scan that needs no sort.

WARNING / TRADE-OFF

Warning — `EXPLAIN ANALYZE` executes the statement. For `INSERT`, `UPDATE` or `DELETE`, wrap it: `BEGIN; EXPLAIN ANALYZE UPDATE ...; ROLLBACK;`. Also remember that the first run may read from disk and the second from cache; compare `Buffers` before comparing times.

Table 11.6 — Failure scenarios in query planning.

What goes wrong	Why	Detection	Recovery	Prevention
Query suddenly 100× slower after a data load	Stale statistics; plan flipped	`pg_stat_statements` latency jump; estimate vs actual gap	`ANALYZE` the table	Autovacuum/analyze tuning; analyze after bulk loads
Nested loop over millions of rows	Underestimated cardinality from correlated predicates	`rows=` estimate far below actual	Extended statistics; rewrite query	`CREATE STATISTICS` on correlated columns
Index ignored	Function or cast on column; low selectivity	`Seq Scan` with selective filter	Expression index or rewritten predicate	Code review of predicates
Hash join or sort spills to disk	Memory budget too small for build side	`Batches: >1`, `Sort Method: external merge`	Raise `work_mem` for that session	Filter earlier; size memory per query

11.11 Transactions

11.11.1 Commit, rollback and savepoints

DEFINITION

Definition — Transaction. A sequence of reads and writes that the database treats as one logical unit: either all of its effects become permanent together (**commit**) or none of them do (**abort** or **rollback**), and concurrently running transactions are isolated from each other's intermediate states.

First-principles explanation

Moving money between two accounts is two writes: debit one, credit the other. If the server crashes between them, or a constraint fails on the second, money is created or destroyed. Without transactions, every application would need its own crash-recovery and

concurrency logic for every multi-step operation, and most would get it wrong. A transaction lets you write the steps as if nothing else were happening and nothing could fail, and makes the database responsible for the rest [Gray and Reuter 1993].

How it works internally

1. `BEGIN` starts a transaction (many drivers start one implicitly; with **autocommit**, every statement is its own transaction).
2. Each write is recorded in the write-ahead log and applied to pages in memory; concurrency control (MVCC and locks, §11.14–§11.15) decides what other transactions may see and do.
3. `COMMIT` forces the log records to durable storage, then makes the changes visible. `ROLLBACK`, an error or a crash before the commit record is durable undoes them.
4. `SAVEPOINT name` / `ROLLBACK TO name` undo part of a transaction without abandoning it.

```
        import sqlite3

        db = sqlite3.connect(":memory:")
        db.executescript("""
        CREATE TABLE accounts (
          account_id    INTEGER PRIMARY KEY,
          balance_cents INTEGER NOT NULL CHECK (balance_cents >= 0)   -- invariant: no
overdraft
        );
        INSERT INTO accounts VALUES (1, 10000), (2, 0);
        """)

        def transfer(src, dst, cents):
            try:
                with db:                                    # BEGIN; ... COMMIT (or
ROLLBACK on error)
                    db.execute("UPDATE accounts SET balance_cents = balance_cents + ? "
                               "WHERE account_id = ?", (cents, dst))          # credit
first, on purpose
                    db.execute("UPDATE accounts SET balance_cents = balance_cents - ? "
                               "WHERE account_id = ?", (cents, src))          # may vio-
late the CHECK
                return "committed"
            except sqlite3.IntegrityError as e:
                return f"rolled back ({e})"

        print(transfer(1, 2, 7000), db.execute("SELECT * FROM accounts").fetchall())
        print(transfer(1, 2, 7000), db.execute("SELECT * FROM accounts").fetchall())
```

What to notice:

- Output: `committed [(1, 3000), (2, 7000)]`, then `rolled back (CHECK constraint failed: balance_cents >= 0) [(1, 3000), (2, 7000)]`. The second transfer's credit had already been applied when the debit failed, yet it vanished: atomicity.

- The invariant lives in the schema (`CHECK`), so no code path can violate it, including ones written next year.

WARNING / TRADE-OFF

Warning — Keep transactions short, and never wait on the network inside one. An open transaction holds row locks and pins old row versions (§11.14). Do slow external work, such as payment calls, before or after the transaction, and make it idempotent (§22.15).

11.12 ACID

DEFINITION

Definition — ACID. The four guarantees of a database transaction, named by Härder and Reuter [Härder and Reuter 1983]: **Atomicity** — all of a transaction's writes take effect, or none do. **Consistency** — a transaction takes the database from one state satisfying all declared invariants (constraints) to another. **Isolation** — concurrently executing transactions do not see each other's partial effects; at the strongest level, the outcome equals some serial execution. **Durability** — once a commit is acknowledged, its effects survive crashes.

How each is implemented:

- **Atomicity** — undo information (InnoDB undo records; in PostgreSQL, uncommitted versions are never marked visible).
- **Consistency** — constraints (`PRIMARY KEY`, `UNIQUE`, `FOREIGN KEY`, `CHECK`, `NOT NULL`). Invariants the schema cannot express ("at least one doctor on call") are the application's job, helped by isolation.
- **Isolation** — MVCC and locking (§11.13–§11.15).
- **Durability** — **write-ahead logging**: the commit record is flushed with `fsync` before "committed" is returned, with many commits sharing one flush (**group commit**). `synchronous_commit = off` (PostgreSQL) or `innodb_flush_log_at_trx_commit = 2` trade recent commits for throughput (§13.5). Surviving machine loss needs replication (Chapter 14).

11.12.1 The "C" in ACID is not the "C" in CAP

The word *consistency* is used for at least three unrelated ideas. Mixing them up is a classic interview mistake.

Table 11.7 — Three meanings of "consistency".

Term	Question it answers	Scope	Where in this book
ACID consistency	Do committed states satisfy the declared invariants?	One database, constraints	§11.12
Strong consistency (linearizability), the "C" in CAP	Does every read see the latest completed write, as if there were one copy?	Replicated data	§25.3, §25.8
Eventual consistency	Will replicas converge if writes stop?	Replicated data	§25.7

A single PostgreSQL server is ACID-consistent and, for reads on that server, linearizable. Add an asynchronous read replica and a client can commit a write on the primary and then read the old value from the replica: the system is still ACID at the primary, but reads are no longer linearizable (§14.4). Kleppmann notes that the C in ACID is really a property of the application's use of transactions, not something the database provides by itself [Kleppmann 2017].

11.13 Isolation levels and anomalies

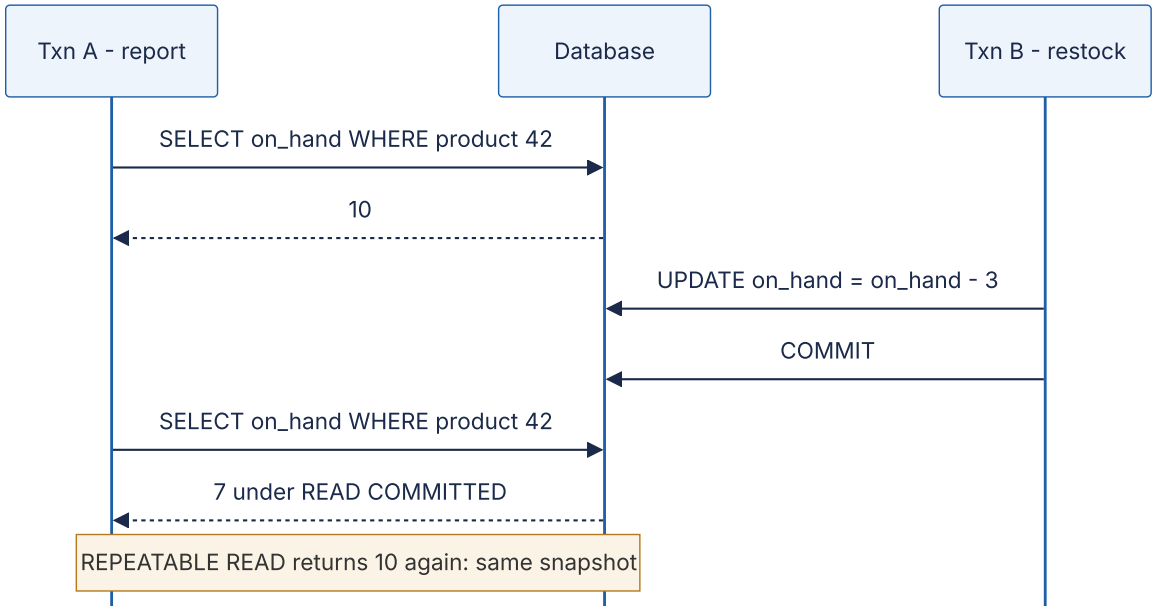
11.13.1 The anomalies

Perfect isolation, called **serializability**, means the result of running transactions concurrently equals the result of running them one at a time in *some* order. It costs throughput, so databases offer weaker **isolation levels** that permit specific **anomalies**. Berenson and colleagues showed that the SQL standard's definitions were ambiguous and cataloged the anomalies precisely [Berenson et al. 1995]:

DEFINITION

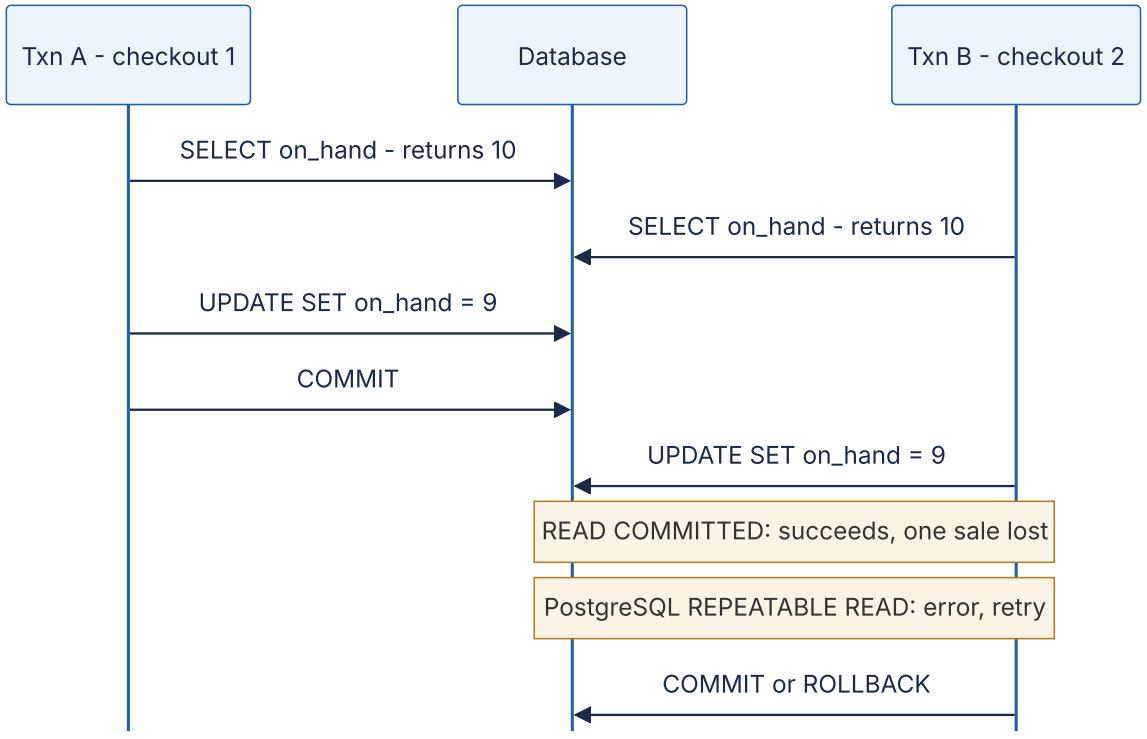
Definition — Isolation anomalies. Dirty write: a transaction overwrites another's uncommitted write. **Dirty read:** a transaction reads another's uncommitted write (which may later roll back). **Non-repeatable (fuzzy) read:** a transaction reads a row twice and sees different committed values because another transaction updated it in between. **Phantom:** a transaction runs the same *predicate* query twice (`WHERE status = 'pending'`) and gets a different set of rows because another transaction inserted or deleted matching rows. **Lost update:** two transactions read the same value, both compute a new value from it and write; the second write silently overwrites the first. **Read skew:** a transaction sees part of another transaction's effects (reads account A before a transfer commits and account B after it). **Write skew:** two transactions read an overlapping set of rows, each checks an invariant, then each writes a *different* row; each write is valid alone but together they violate the invariant.

Figure 11.6 — Non-repeatable read under READ COMMITTED.



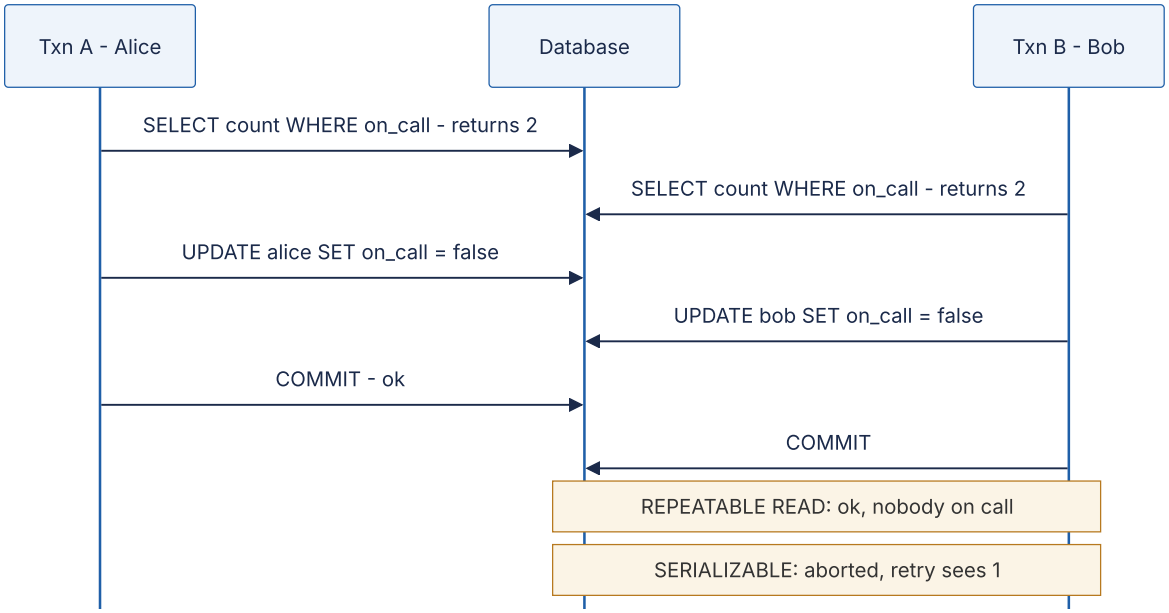
Reading the diagram. Two clients on separate connections send synchronous SQL; time runs downward. Under READ COMMITTED each *statement* takes a new snapshot, so A's second read sees B's committed change; under PostgreSQL REPEATABLE READ the *transaction* keeps its first snapshot and sees 10 twice. Both outcomes were reproduced on PostgreSQL 16. Both row versions exist inside the database at once (§11.14); the failure is any logic in A that combines the two reads, such as a report whose totals do not add up.

Figure 11.7 — Lost update from an application-side read-modify-write.



Reading the diagram. Each checkout reads 10 and writes the literal 9 computed *in the application*. Under READ COMMITTED (and MySQL REPEATABLE READ) B silently overwrites A: two units sold, one decremented. Under PostgreSQL REPEATABLE READ, B's `UPDATE` fails with `could not serialize access due to concurrent update` and B must retry. The fix at any level is to let the database do the arithmetic, `UPDATE inventory SET on_hand = on_hand - 1 WHERE product_id = 42 AND on_hand >= 1`, or to lock first with `SELECT ... FOR UPDATE`, or to check a version (§11.15.3).

Figure 11.8 — Write skew: both doctors go off call.



Reading the diagram. The invariant "at least one doctor on call" is checked by a read and protected by nothing: the transactions write *different* rows, so neither snapshot isolation nor row locks see a conflict, and both commit at REPEATABLE READ. PostgreSQL SERIALIZABLE tracks what each transaction *read*, detects the dangerous dependency pattern and aborts one with `could not serialize access due to read/write dependencies among transactions`; its retry sees one doctor on call. In the shop the same pattern double-redeems a limited coupon. Below SERIALIZABLE, materialize the conflict by locking a shared row with `FOR UPDATE`.

11.13.2 The standard levels and what databases actually do

Table 11.8 — ANSI SQL isolation levels as defined by the three phenomena.

Level	Dirty read	Non-repeatable read	Phantom
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SERIALIZABLE	Prevented	Prevented	Prevented

Berenson and colleagues pointed out that this table misses lost update, read skew and write skew, and that **snapshot isolation** (each transaction reads from a consistent snapshot taken at its start, and a transaction aborts if it tries to write a row that another transaction changed after its snapshot) fits nowhere: it prevents all three standard phenomena yet permits write skew, so it is not serializable [Berenson et al. 1995]. Real databases implement the levels differently, so the level *name* tells you little; check the behavior.

Table 11.9 — Anomalies permitted in practice (PostgreSQL 16, MySQL 8.0 InnoDB).

Anomaly	PG READ COMMITTED (default)	PG REPEATABLE READ (snapshot isolation)	PG SERIALIZABLE (SSI)	MySQL READ COMMITTED	MySQL REPEATABLE READ (default)	MySQL SERIALIZABLE
Dirty read	No	No	No	No	No	No
Non-repeatable read	Yes	No	No	Yes	No	No
Phantom	Yes	No	No	Yes	No for plain reads	No
Lost update	Yes	No (error)	No	Yes	**Yes**	No
Read skew	Yes	No	No	Yes	No	No
Write skew	Yes	Yes	No	Yes	Yes	No

Key facts behind the table:

- **PostgreSQL** accepts `READ UNCOMMITTED` but runs it as READ COMMITTED. REPEATABLE READ is snapshot isolation. SERIALIZABLE (since 9.1) is **Serializable Snapshot Isolation (SSI)**: snapshot isolation plus tracking of read–write dependencies, aborting a transaction when a cycle is possible [Cahill et al. 2008; Ports and Grittner 2012]. It takes no extra blocking locks, but applications must retry on serialization failures (SQLSTATE `40001`).
- **MySQL InnoDB** defaults to REPEATABLE READ. Plain `SELECT`s read a snapshot, but `UPDATE`, `DELETE` and locking reads (`SELECT ... FOR UPDATE`) read the *latest committed* row and do not check whether it changed since the snapshot, which is why

lost updates and write skew get through. Locking reads take next-key locks that block phantom inserts (§11.15). SERIALIZABLE converts plain `SELECT`s into `SELECT ... FOR SHARE` when autocommit is off, i.e., two-phase locking [MySQL Docs: Transaction Isolation Levels].

- Kleppmann's Hermitage project collects independent tests of these behaviors [Kleppmann 2014].

The following program reproduced two rows of Table 11.9 against PostgreSQL 16:

```
        # PostgreSQL-specific: needs a running PostgreSQL and `pip install psycopg2-
binary`.
        import psycopg2

        DSN = "dbname=shop user=postgres host=localhost"   # adjust to your server

        def session(level):
            conn = psycopg2.connect(DSN)
            conn.set_session(isolation_level=level)
            return conn, conn.cursor()

        def reset():
            conn = psycopg2.connect(DSN); conn.autocommit = True
            conn.cursor().execute("""
                DROP TABLE IF EXISTS stock, doctors;
                CREATE TABLE stock (product_id int PRIMARY KEY, on_hand int NOT NULL);
                INSERT INTO stock VALUES (42, 10);
                CREATE TABLE doctors (name text PRIMARY KEY, on_call bool NOT NULL);
                INSERT INTO doctors VALUES ('alice', true), ('bob', true);""")
            conn.close()

        def try_commit(conn):
            try:
                conn.commit(); return "commit"
            except psycopg2.Error as e:
                conn.rollback(); return "abort: " + e.pgerror.splitlines()[0]

        def lost_update(level):           # read-modify-write computed in the applica-
tion
            reset(); (a, ca), (b, cb) = session(level), session(level)
            ca.execute("SELECT on_hand FROM stock WHERE product_id = 42"); va =
ca.fetchone()[0]
            cb.execute("SELECT on_hand FROM stock WHERE product_id = 42"); vb =
cb.fetchone()[0]
            ca.execute("UPDATE stock SET on_hand = %s WHERE product_id = 42", (va -
1,)); a.commit()
            try:
                cb.execute("UPDATE stock SET on_hand = %s WHERE product_id = 42", (vb -
1,))
                result = try_commit(b)
            except psycopg2.Error as e:
                b.rollback(); result = "abort: " + e.pgerror.splitlines()[0]
            print(f"lost update, {level}: B {result}")

        def write_skew(level):            # invariant: at least one doctor on call
            reset(); (a, ca), (b, cb) = session(level), session(level)
            for cur, me in ((ca, "alice"), (cb, "bob")):
                cur.execute("SELECT count(*) FROM doctors WHERE on_call")
                if cur.fetchone()[0] >= 2:
                    cur.execute("UPDATE doctors SET on_call = false WHERE name = %s",
(me,))
            print(f"write skew, {level}: A {try_commit(a)}; B {try_commit(b)}")

        for level in ("READ COMMITTED", "REPEATABLE READ"):
            lost_update(level)
```

```
        for level in ("REPEATABLE READ", "SERIALIZABLE"):
            write_skew(level)
```

Output on PostgreSQL 16.15:

```
lost update, READ COMMITTED: B commit
lost update, REPEATABLE READ: B abort: ERROR:  could not serialize access due to concur-
rent update
write skew, REPEATABLE READ: A commit; B commit
write skew, SERIALIZABLE: A commit; B abort: ERROR:  could not serialize access due to
read/write dependencies among transactions
```

What to notice:

- Two sessions are interleaved by hand in one thread; nothing blocks, because PostgreSQL readers never wait for writers.
- At REPEATABLE READ and SERIALIZABLE, an abort is *normal* behavior, not a bug. Production code wraps such transactions in a retry loop with a bounded number of attempts.

RECOMMENDATION

Recommendation — Choosing an isolation level. Default to READ COMMITTED (PostgreSQL) and make every read-modify-write atomic in SQL (`SET x = x - 1 WHERE ... AND x >= 1`) or explicitly locked (`FOR UPDATE`). Use SERIALIZABLE, with retries, for transactions that check an invariant across several rows and then write. In MySQL, do not assume REPEATABLE READ protects read-modify-write cycles; use locking reads.

11.14 Multi-version concurrency control (MVCC)

11.14.1 The idea

DEFINITION

Definition — MVCC. A concurrency-control technique in which a write creates a new **version** of a row instead of overwriting it in place, and each transaction reads the versions visible in its **snapshot**: the set of transactions that had committed when the snapshot was taken. Readers therefore never block writers and writers never block readers; only writers of the same row wait for each other.

First-principles explanation

A library keeps every edition of a reference book; a reader arriving at 10:00 gets the edition current at 10:00, even if a new one is shelved at 10:05. Editors never wait for readers. The costs are shelf space and a librarian who discards editions nobody can still be reading. PostgreSQL and InnoDB differ in *where* they keep old editions and *who* discards them.

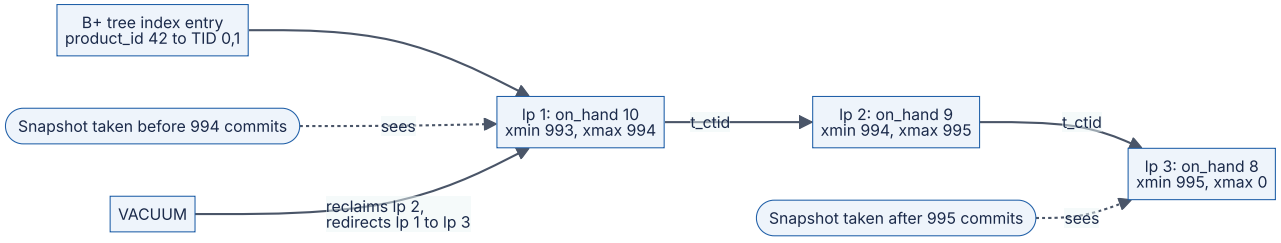
11.14.2 PostgreSQL: versions in the heap, cleaned by VACUUM

Each heap tuple carries a header with **`xmin`** (the ID of the transaction that created this version), **`xmax`** (the ID of the transaction that deleted or replaced it, or 0) and **`ctid`** (the location of this version, or of its successor). An `UPDATE` sets `xmax` on the old version and inserts a complete new version. A snapshot records the oldest still-running transaction ID, the next unassigned ID and the list of IDs in progress; a version is visible if its `xmin` committed before the snapshot and its `xmax` is empty, aborted or not yet visible to the snapshot [PostgreSQL Docs: Concurrency Control].

Real output from PostgreSQL 16 after inserting a stock row and updating it twice (the `pageinspect` extension shows raw page contents):

```
        INSERT INTO stock VALUES (42, 10);                          -- transaction 993
        UPDATE stock SET on_hand = 9 WHERE product_id = 42;         -- transaction 994
        UPDATE stock SET on_hand = 8 WHERE product_id = 42;         -- transaction 995
        SELECT xmin, xmax, ctid, * FROM stock;                      -- only the visible
version
        --  xmin | xmax | ctid  | product_id | on_hand
        --   995 |    0 | (0,3) |         42 |       8
        SELECT lp, t_xmin, t_xmax, t_ctid FROM heap_page_items(get_raw_page('stock',
0));
        --  lp | t_xmin | t_xmax | t_ctid
        --   1 |    993 |    994 | (0,2)
        --   2 |    994 |    995 | (0,3)
        --   3 |    995 |      0 | (0,3)
```

Figure 11.9 — PostgreSQL version chain for product 42.



Reading the diagram. All three versions sit in heap page 0. The updates changed no indexed column and the page had room, so they were **heap-only tuple (HOT)** updates: the index still points to line pointer 1 and readers follow `t_ctid` links to the version their

snapshot can see (dotted arrows show visibility, not messages). On the real page, VACUUM turned line pointer 1 into a redirect to 3 and freed 2. Failure point: a long-running transaction pins an old snapshot, so VACUUM cannot clean dead versions in any table and tables bloat. Scaling: an update that changes an indexed column or finds the page full is not HOT, and every index gets a new entry.

VACUUM (run automatically by **autovacuum**) removes dead versions, records free space for reuse, updates the **visibility map** that index-only scans rely on (§11.7.4), and **freezes** old tuples. Freezing matters because transaction IDs are 32-bit counters compared with wrap-around arithmetic: only about 2^{31} IDs are "in the past" at any moment. A tuple must be frozen (marked as visible to everyone) before its `xmin` becomes 2 billion transactions old, or it would appear to be in the future [PostgreSQL Docs: Routine Vacuuming].

FORMULA

Time until transaction-ID wraparound

$$T_{\text{wrap}} = \frac{2^{31} - A}{\lambda_{\text{xid}}}$$

Symbol	Meaning	Units
T_{wrap}	Time until PostgreSQL must stop assigning transaction IDs	seconds
A	Current age of the oldest unfrozen transaction ID in the database	transactions
λ_{xid}	Rate of transactions that write (read-only transactions consume no ID)	transactions/second

WORKED EXAMPLE

Worked example — How long can freezing be blocked? A shop database commits $\lambda_{xid} = 5,000$ writing transactions per second. Autovacuum normally forces an anti-wraparound vacuum once a table's oldest ID reaches `autovacuum_freeze_max_age` = 200 million (the default).

1. Time to reach the forced-vacuum threshold from zero: $2 \times 10^8 / 5,000 = 40,000$ s, about 11 hours.
2. Suppose a forgotten idle transaction blocks freezing at age $A = 2 \times 10^8$. Remaining headroom: $2^{31} - 2 \times 10^8 = 2,147,483,648 - 200,000,000 \approx 1.95 \times 10^9$.
3. $T_{wrap} = 1.95 \times 10^9 / 5,000 = 389,497$ s, about 4.5 days.

Interpretation. A busy database burns through its safety margin in days, not years. As it approaches the limit PostgreSQL emits warnings and eventually refuses new writing transactions until a vacuum completes. **Engineering implication.** Alert on `age(datfrozenxid)` and on transactions open longer than a few minutes (`pg_stat_activity.xact_start`), and never leave sessions "idle in transaction" (`idle_in_transaction_session_timeout`).

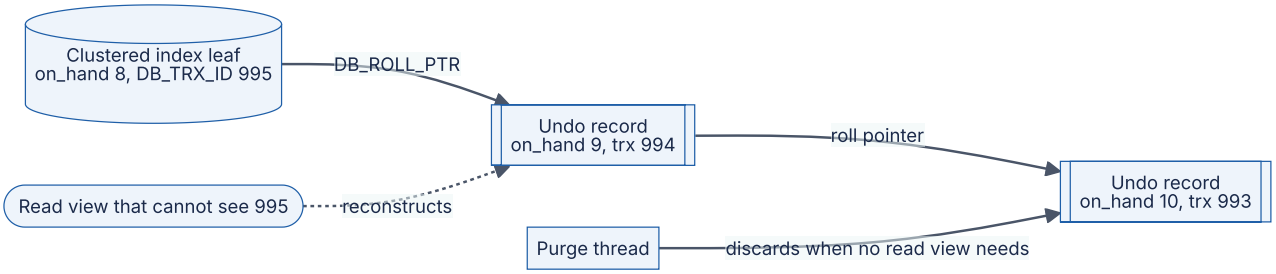
INDUSTRY EXAMPLE

Sentry — transaction ID wraparound outage. Sentry published a postmortem of an outage in which its PostgreSQL database stopped accepting writes to prevent transaction ID wraparound, and service returned only after vacuuming. The lesson: in PostgreSQL, vacuum is part of the write path's capacity budget, not optional maintenance. *Evidence: Primary* — [Sentry 2015]

11.14.3 InnoDB: update in place, old versions in the undo log

InnoDB updates the row *in place* in the clustered index and writes the *previous* values to an **undo log** record. Each clustered-index record has hidden columns `DB_TRX_ID` (the last transaction to modify it, 6 bytes) and `DB_ROLL_PTR` (a pointer to the undo record holding the previous version, 7 bytes). A consistent read creates a **read view** (the set of transactions it must not see); if the current row is too new, InnoDB follows the roll pointers backwards, reconstructing older versions until it reaches a visible one. A background **purge** thread discards undo records no read view needs; the backlog appears as the **history list length** in `SHOW ENGINE INNODB STATUS` [MySQL Docs: InnoDB Multi-Versioning].

Figure 11.10 — InnoDB current row and undo chain for product 42.



Reading the diagram. The cylinder is the one current version in the clustered index; the subroutine shapes are undo records, linked newest to oldest. A reader whose read view excludes transaction 995 applies undo backwards until it reaches a visible version (dotted arrow: a visibility decision). Purge is asynchronous background work. Failure point: a long transaction blocks purge, the history list grows and hot-row reads walk ever-longer chains. Scaling benefit: updates do not bloat the table, and secondary indexes change only when their columns change.

Table 11.10 — MVCC in PostgreSQL and InnoDB.

Aspect	PostgreSQL	MySQL InnoDB
Where old versions live	In the table heap, next to current ones	In undo logs, outside the table
What an update writes	A complete new tuple (and index entries unless HOT)	In-place change plus an undo record
Version metadata	`xmin`, `xmax`, `ctid` in each tuple header	`DB_TRX_ID`, `DB_ROLL_PTR` hidden columns
Reading an old version	Scan to the visible tuple; cheap	Reconstruct by applying undo; cost grows with chain length
Garbage collection	VACUUM / autovacuum; must also freeze XIDs	Purge threads
Rollback cost	Nearly free (versions never become visible)	Must apply undo to restore rows
Symptom of a long transaction	Table and index bloat; wraparound risk	Growing history list; slower reads

11.15 Locking

11.15.1 Two-phase locking and lock granularity

DEFINITION

Definition — Two-phase locking (2PL). A protocol in which each transaction acquires locks (shared **S** for reading, exclusive **X** for writing) before accessing data and, once it releases any lock, acquires no new ones: a *growing* phase followed by a *shrinking* phase. **Strict 2PL** holds all X locks until commit or abort. Any schedule produced by 2PL is serializable [Eswaran et al. 1976].

S locks are compatible with each other; X locks conflict with everything. MVCC databases lock mainly for *writes*: two updates of one row serialize on its row lock, while plain reads take no row locks. Granularities:

- **Row locks** — taken by `UPDATE`, `DELETE` and explicit `SELECT ... FOR UPDATE` / `FOR SHARE`. PostgreSQL also has weaker `FOR NO KEY UPDATE` and `FOR KEY SHARE` modes so that foreign-key checks do not block updates of non-key columns.
- **Table locks** — PostgreSQL has eight table-lock modes from `ACCESS SHARE` (taken by every `SELECT`) to `ACCESS EXCLUSIVE` (taken by most `ALTER TABLE` forms) [PostgreSQL Docs: Explicit Locking]. InnoDB uses **intention locks** (IS, IX) on the table to signal row locks below.
- **Gap and next-key locks (InnoDB)** — at REPEATABLE READ, a locking read or update over a range locks the index records it scans *and the gaps between them* (a **next-key lock** is a record lock plus the gap before it). This blocks inserts into the range, preventing phantoms, at the price of extra blocking [MySQL Docs: InnoDB Locking]. READ COMMITTED disables most gap locking.

`SELECT ... FOR UPDATE SKIP LOCKED` (PostgreSQL 9.5+, MySQL 8.0+) lets many workers pull different rows from a job table without blocking on each other, a common way to build a simple, transactional work queue.

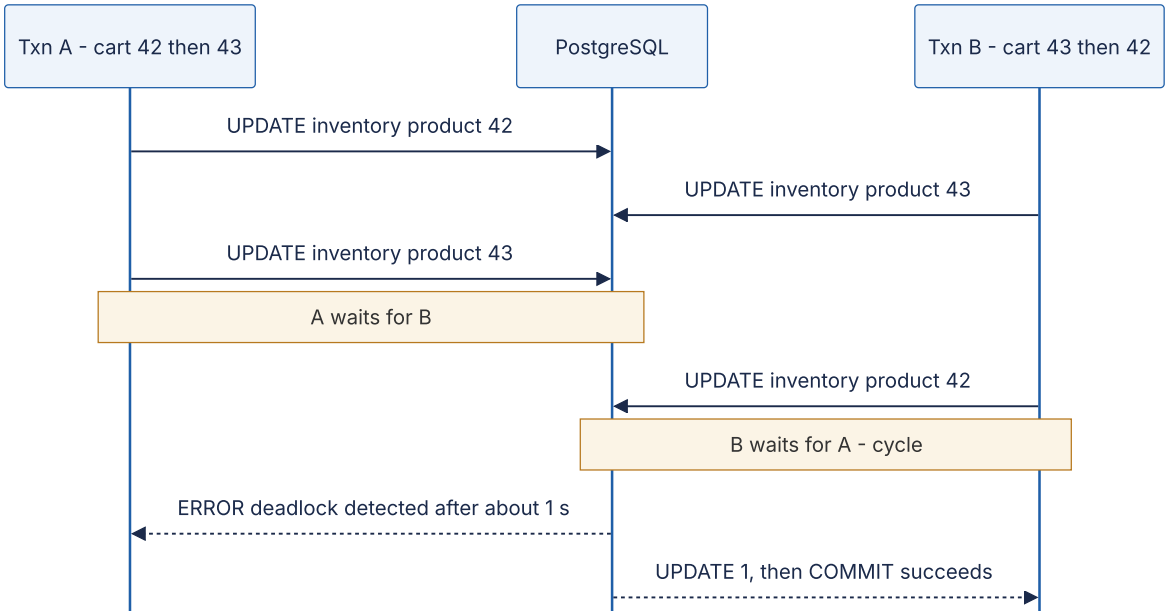
WARNING / TRADE-OFF

Warning — A schema change can stop all traffic. `ALTER TABLE orders ADD COLUMN ...` needs PostgreSQL's `ACCESS EXCLUSIVE` lock for at least a moment. If a long report query holds `ACCESS SHARE`, the `ALTER` waits, and every new query on `orders` queues *behind the waiting* `ALTER`. Set `lock_timeout` (for example 2 s) on migrations and retry; use online schema-change tools for MySQL (such as GitHub's gh-ost [GitHub gh-ost]).

11.15.2 Deadlocks

A **deadlock** occurs when transactions wait for each other's locks in a cycle (§6.11). Databases detect cycles in the **wait-for graph** and abort one transaction, the **victim**. PostgreSQL checks for a cycle after a waiter has been blocked for `deadlock_timeout` (default 1 s); InnoDB detects them immediately by default (`innodb_deadlock_detect`), and otherwise relies on `innodb_lock_wait_timeout` (default 50 s) [MySQL Docs: Deadlock Detection].

Figure 11.11 — A deadlock between two checkouts, as reproduced on PostgreSQL 16.



Reading the diagram. Each checkout updates inventory rows in cart order with synchronous, blocking statements. A holds 42 and waits for 43; B holds 43 and waits for 42: a cycle in the wait-for graph. After A had waited `deadlock_timeout`, PostgreSQL aborted it with `deadlock detected` (SQLSTATE `40P01`) and B's update proceeded. The cost is a retried transaction and up to a second of latency, and deadlocks become more frequent as concurrency grows. **Prevention:** acquire locks in a canonical order, for example by sorting cart lines by `product_id`, as the code in §11.17 does.

11.15.3 Hot rows and optimistic concurrency

FORMULA

Throughput ceiling of a single hot row

$$X_{\max} = \frac{1}{t_{\text{hold}}}$$

Symbol	Meaning	Units
$X_{\max}$	Maximum transactions per second that can update the same row	transactions/second
t_{hold}	Time each transaction holds the row's X lock (from its update to commit)	seconds

WORKED EXAMPLE

Worked example — A flash sale on one product. Each checkout updates the product's `inventory` row, then (wrongly) calls a payment provider inside the transaction before committing.

1. With a 200 ms payment call inside the transaction: $X_{\max} = 1/0.2 = 5$ checkouts per second, however many servers you add.
2. Payment moved outside the transaction; update and commit take 2 ms including a commit flush and one network round trip: $X_{\max} = 1/0.002 = 500$ per second.
3. Stock split into 10 bucket rows (each holding a tenth of the units, chosen at random per checkout): about $10 \times 500 = 5,000$ per second.

Interpretation. Lock hold time, not CPU, bounds a hot row: this is Little's Law (§4.11) applied to a lock with capacity one. **Engineering implication.** Keep slow work out of transactions, update hot rows last (just before commit) so the lock is held briefly, and split hot counters when one row cannot keep up.

When conflicts are rare, **optimistic concurrency control** avoids holding locks across user think time: write only if the row's version is unchanged since you read it.

```
        import sqlite3

        db = sqlite3.connect(":memory:")
        db.executescript("""
        CREATE TABLE products (
          product_id  INTEGER PRIMARY KEY,
          description TEXT    NOT NULL,
          version     INTEGER NOT NULL DEFAULT 1
        );
        INSERT INTO products (product_id, description) VALUES (7, 'Blue mug');
        """)

        def save(product_id, new_description, version_read):
            """Optimistic concurrency control: the write succeeds only if nobody wrote
since we read."""
            with db:
                cur = db.execute(
                    "UPDATE products SET description = ?, version = version + 1 "
                    "WHERE product_id = ? AND version = ?",
                    (new_description, product_id, version_read))
            return cur.rowcount == 1          # 0 rows updated means a concurrent edit
won

        # Two editors load the same row (version 1), then both try to save.
        _, _, v_alice = db.execute("SELECT * FROM products WHERE product_id =
7").fetchone()
        _, _, v_bob = db.execute("SELECT * FROM products WHERE product_id =
7").fetchone()
        print("alice saved:", save(7, "Blue ceramic mug", v_alice))
        print("bob saved:  ", save(7, "Navy mug", v_bob))       # rejected: must re-read
and retry
        print(db.execute("SELECT * FROM products").fetchall())
```

What to notice:

- Output: Alice's save returns `True`, Bob's `False`, and the row is `(7, 'Blue ceramic mug', 2)`: Bob's stale edit is rejected instead of silently overwriting Alice's (a lost update prevented without locks).
- The check-and-write is one atomic `UPDATE`; HTTP APIs expose the same idea with `ETag` and `If-Match` headers.

11.16 PostgreSQL versus MySQL

11.16.1 Comparison

Both are mature, open-source ACID databases that handle most transactional workloads well; the differences decide edge cases, operations and ecosystem fit.

Table 11.11 — PostgreSQL and MySQL (InnoDB) compared, as of PostgreSQL 16–18 and MySQL 8.0–8.4.

Row	PostgreSQL	MySQL (InnoDB)
What it is	Extensible object-relational database; PostgreSQL License	Relational database; GPLv2 community edition plus commercial editions (Oracle)
How it works	Heap tables, all indexes secondary (TID pointers); MVCC versions in heap; VACUUM; process per connection; 8 KiB pages	Index-organized tables clustered on PK; secondary indexes store PK; MVCC via undo logs; purge; thread per connection; 16 KiB pages
Advantages	Rich SQL (CTEs, window functions, `JSONB`, partial/expression indexes, GIN/GiST/BRIN), transactional DDL, SSI serializable, extensions (PostGIS, pgvector)	Fast PK lookups and PK range scans, compact secondary-index updates, mature replication tooling, very wide hosting and operator familiarity
Disadvantages	Vacuum tuning and XID wraparound; non-HOT updates touch every index; connections are costly (needs a pooler, §14.13)	No partial indexes; DDL not transactional (atomic since 8.0, but commits implicitly); REPEATABLE READ allows lost updates; gap-lock surprises
Performance	Strong on complex queries, joins and analytics-style SQL; good write throughput when HOT applies	Strong on simple PK-centric OLTP and high connection counts; secondary lookups cost two tree descents
Operational complexity	Autovacuum, bloat and freeze monitoring; major-version upgrades via `pg_upgrade` or logical replication	Purge lag and undo growth; online schema change tools for large tables
Cost	Free; managed offerings on all major clouds	Free community edition; managed offerings on all major clouds; commercial support from Oracle
Use cases	Complex domains, geospatial, JSON-heavy data, mixed OLTP and reporting	High-volume web OLTP with simple access patterns; large existing MySQL ecosystems
Choose when	You need advanced SQL, data types, extensions or serializable isolation	Your team and tooling know MySQL, or the workload is dominated by PK access and you rely on its replication ecosystem

Table 11.12 — Defaults and behaviors that change application code.

Behavior	PostgreSQL	MySQL InnoDB
Default isolation level	READ COMMITTED	REPEATABLE READ
Serializable implementation	SSI (optimistic, aborts)	2PL with shared locks (blocking)
Phantom protection at REPEATABLE READ	Snapshot	Snapshot for reads; next-key locks for locking reads
Join algorithms	Nested loop, hash, merge	Nested loop, hash (8.0.18+)
Partial indexes	Yes	No
`INCLUDE` covering columns	Yes (11+)	Implicit PK only
Deadlock detection	After `deadlock_timeout` (1 s)	Immediate by default

INDUSTRY EXAMPLE

Uber — moving a core datastore from PostgreSQL to MySQL (2016). Uber's engineering blog attributed the move to PostgreSQL's physical design at its scale: non-HOT updates write new entries in every index (write amplification), physical WAL replication shipped those amplified changes across data centers, and MVCC on replicas interacted badly with long queries. InnoDB's PK-pointer secondary indexes and row-based replication fit its write pattern better. PostgreSQL practitioners responded with mitigations (HOT-friendly schemas, `fillfactor`, logical replication). The lesson is to match storage design to workload. *Evidence: Primary* — [Klitzke 2016]

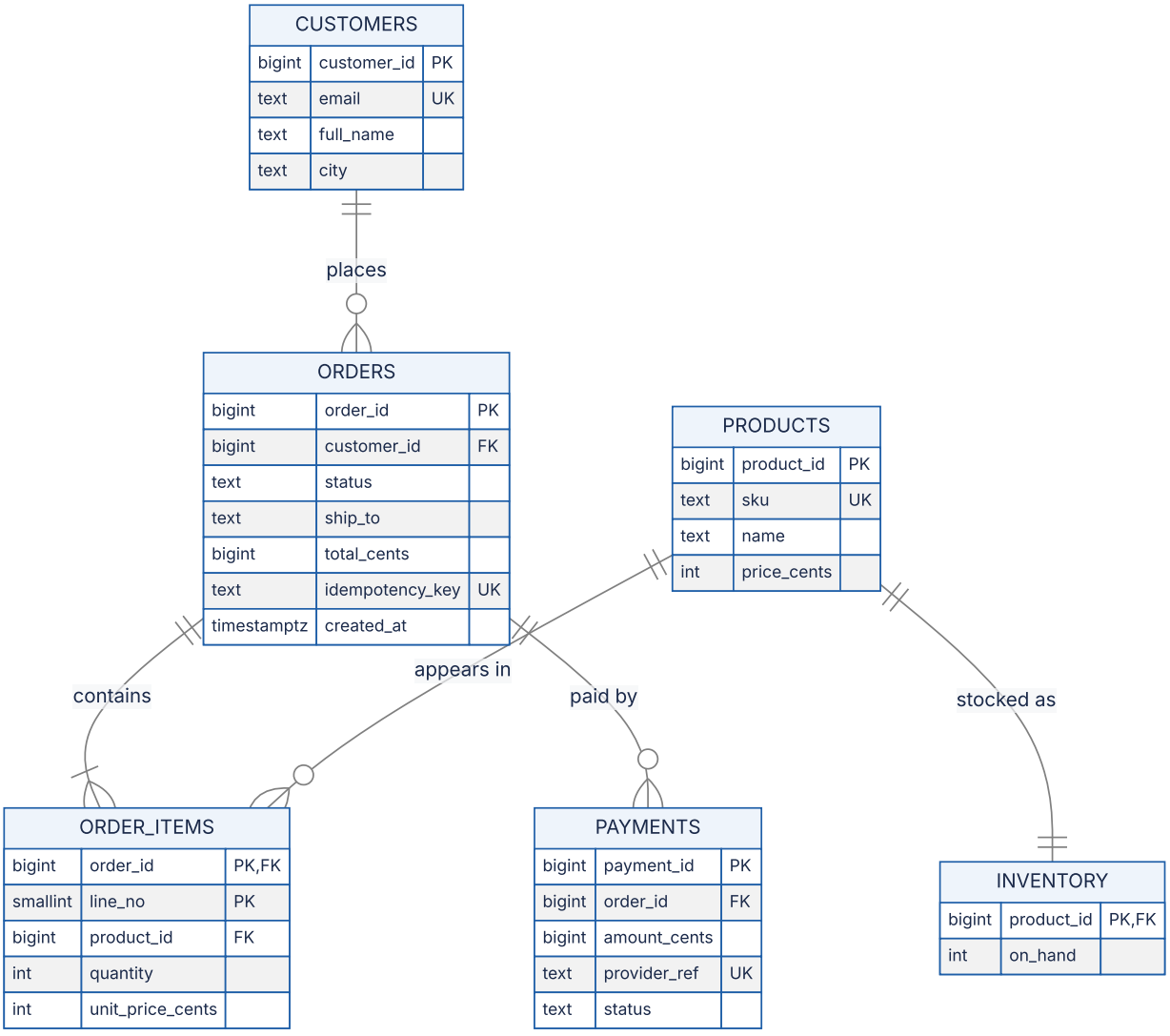
11.17 Data modeling for a real application

11.17.1 Requirements to schema

The order service must reserve stock and create an order atomically, never oversell, accept retries without duplicates, record payments, show order history newest first and report best sellers. We model it in five steps:

1. **Entities and relationships** from nouns and verbs: customers *place* orders; orders *contain* items; items *reference* products; products *have* inventory; orders are *paid by* payments.
2. **Keys**: `bigint` identity surrogates; natural keys (`email`, `sku`, `provider_ref`, `idempotency_key`) as `UNIQUE`.
3. **Normalize** to BCNF (§11.5), keeping historical facts (`unit_price_cents`, `ship_to`) on the order.
4. **Invariants as constraints**: `CHECK (on_hand >= 0)` makes overselling impossible.
5. **Indexes from queries**: one per access path, and no others.

Figure 11.12 — Entity-relationship diagram of the order system.



Reading the diagram. Each box is a table with key markers (PK, FK, UK); crow's-foot lines give cardinality: a customer places zero or more orders, an order has one or more items, a product has exactly one inventory row, an order has zero or more payment attempts. Every line is a foreign key enforced synchronously inside one database, the single storage boundary. Failure points are rows many transactions touch: the `inventory` rows of popular products (§11.15.3). Scaling: read replicas for history and reports (§14.3), then partitioning orders and items by `customer_id` (§14.7).

11.17.2 DDL (PostgreSQL)

The following DDL was run on PostgreSQL 16:

```
        CREATE TABLE customers (
          customer_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
          email        text NOT NULL UNIQUE,
          full_name    text NOT NULL,
          city         text NOT NULL,
          created_at   timestamptz NOT NULL DEFAULT now()
        );
        CREATE TABLE products (
          product_id   bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
          sku          text NOT NULL UNIQUE,
          name         text NOT NULL,
          price_cents  integer NOT NULL CHECK (price_cents >= 0)
        );
        CREATE TABLE inventory (
          product_id   bigint PRIMARY KEY REFERENCES products,
          on_hand      integer NOT NULL CHECK (on_hand >= 0)
        );
        CREATE TABLE orders (
          order_id         bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
          customer_id      bigint NOT NULL REFERENCES customers,
          status           text NOT NULL DEFAULT 'pending'
                           CHECK (status IN ('pending','paid','shipped','cancelled')),
          ship_to          text NOT NULL,
          total_cents      bigint NOT NULL CHECK (total_cents >= 0),
          idempotency_key  text NOT NULL UNIQUE,
          created_at       timestamptz NOT NULL DEFAULT now()
        );
        CREATE TABLE order_items (
          order_id          bigint NOT NULL REFERENCES orders ON DELETE CASCADE,
          line_no           smallint NOT NULL,
          product_id        bigint NOT NULL REFERENCES products,
          quantity          integer NOT NULL CHECK (quantity > 0),
          unit_price_cents  integer NOT NULL CHECK (unit_price_cents >= 0),
          PRIMARY KEY (order_id, line_no)
        );
        CREATE TABLE payments (
          payment_id    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
          order_id      bigint NOT NULL REFERENCES orders,
          amount_cents  bigint NOT NULL CHECK (amount_cents > 0),
          provider_ref  text UNIQUE,
          status        text NOT NULL CHECK (status IN
('authorized','captured','failed','refunded')),
          created_at    timestamptz NOT NULL DEFAULT now()
        );
        -- One index per access path
        CREATE INDEX orders_customer_created_idx ON orders (customer_id, created_at
DESC, order_id DESC); -- history
        CREATE INDEX order_items_product_idx     ON order_items (product_id);       --
FK checks, sales by product
        CREATE INDEX payments_order_idx          ON payments (order_id);            --
payments of an order
        CREATE INDEX orders_pending_created_idx  ON orders (created_at) WHERE status =
'pending'; -- sweeper
```

What to notice:

- Money is stored as integer cents (or `numeric`), never floating point. Timestamps use `timestamptz`.
- `order_items` has a composite primary key `(order_id, line_no)`: its leading column serves "items of this order" and the foreign-key check, so no separate index on `order_items.order_id` is needed.
- `total_cents` is a stored aggregate (a denormalization, §11.5.5) written in the same transaction as the items; it is also the amount charged, a fact worth keeping.
- `status` uses `CHECK` rather than a separate lookup table or PostgreSQL `ENUM` type; adding a status later is a one-line constraint change.

11.17.3 The place-order transaction and core queries

The order service runs this transaction (shown with literal values; the application binds parameters). The decrements are atomic conditional updates, sorted by `product_id` to avoid deadlocks; the idempotency key makes retries safe (§22.15):

```
        BEGIN;
        INSERT INTO orders (customer_id, ship_to, total_cents, idempotency_key)
        VALUES (1, '1 Main St', 0, '7f1c9a52-key')
        ON CONFLICT (idempotency_key) DO NOTHING
        RETURNING order_id;                      -- no row returned: a retry; fetch and
return the existing order
        UPDATE inventory SET on_hand = on_hand - 2 WHERE product_id = 1 AND on_hand >=
2;  -- expect 1 row
        UPDATE inventory SET on_hand = on_hand - 1 WHERE product_id = 2 AND on_hand >=
1;  -- 0 rows: out of stock, ROLLBACK
        INSERT INTO order_items (order_id, line_no, product_id, quantity,
unit_price_cents)
        SELECT 1, row_number() OVER (ORDER BY v.product_id), v.product_id, v.qty,
p.price_cents
        FROM (VALUES (1, 2), (2, 1)) AS v(product_id, qty) JOIN products p USING
(product_id);
        UPDATE orders SET total_cents = (SELECT sum(quantity * unit_price_cents)
                                         FROM order_items WHERE order_id = 1)
        WHERE order_id = 1;
        COMMIT;
```

The complete model also runs on SQLite through Python's standard library; this script creates the schema, places an order, retries it with the same idempotency key, attempts an order that would oversell, and runs the history, best-seller and anti-join queries:

```
        """E-commerce order schema and queries (Section 11.17), runnable with Python's
sqlite3.
        The PostgreSQL DDL in the chapter differs only in types (bigint identity, times-
tamptz)."""
        import sqlite3

        SCHEMA = """
        PRAGMA foreign_keys = ON;
        CREATE TABLE customers (
          customer_id INTEGER PRIMARY KEY,
          email       TEXT NOT NULL UNIQUE,
          full_name   TEXT NOT NULL,
          city        TEXT NOT NULL,
          created_at  TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
        );
        CREATE TABLE products (
          product_id  INTEGER PRIMARY KEY,
          sku         TEXT NOT NULL UNIQUE,
          name        TEXT NOT NULL,
          price_cents INTEGER NOT NULL CHECK (price_cents >= 0)
        );
        CREATE TABLE inventory (
          product_id INTEGER PRIMARY KEY REFERENCES products,
          on_hand    INTEGER NOT NULL CHECK (on_hand >= 0)
        );
        CREATE TABLE orders (
          order_id        INTEGER PRIMARY KEY,
          customer_id     INTEGER NOT NULL REFERENCES customers,
          status          TEXT NOT NULL DEFAULT 'pending'
                          CHECK (status IN ('pending','paid','shipped','cancelled')),
          ship_to         TEXT NOT NULL,
          total_cents     INTEGER NOT NULL DEFAULT 0 CHECK (total_cents >= 0),
          idempotency_key TEXT NOT NULL UNIQUE,
          created_at      TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
        );
        CREATE TABLE order_items (
          order_id         INTEGER NOT NULL REFERENCES orders ON DELETE CASCADE,
          line_no          INTEGER NOT NULL,
          product_id       INTEGER NOT NULL REFERENCES products,
          quantity         INTEGER NOT NULL CHECK (quantity > 0),
          unit_price_cents INTEGER NOT NULL CHECK (unit_price_cents >= 0),
          PRIMARY KEY (order_id, line_no)
        );
        CREATE TABLE payments (
          payment_id   INTEGER PRIMARY KEY,
          order_id     INTEGER NOT NULL REFERENCES orders,
          amount_cents INTEGER NOT NULL CHECK (amount_cents > 0),
          provider_ref TEXT UNIQUE,
          status       TEXT NOT NULL CHECK (status IN
('authorized','captured','failed','refunded')),
          created_at   TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
        );
        CREATE INDEX orders_customer_created ON orders (customer_id, created_at DESC,
order_id DESC);
        CREATE INDEX order_items_product     ON order_items (product_id);
        CREATE INDEX payments_order          ON payments (order_id);
```

```
        CREATE INDEX orders_pending_created  ON orders (created_at) WHERE status =
'pending';
        """

        class OutOfStock(Exception):
            pass

        def place_order(db, customer_id, ship_to, items, idempotency_key):
            """items: list of (product_id, quantity). Returns order_id. All-or-noth-
ing."""
            with db:  # BEGIN ... COMMIT, or ROLLBACK if an exception escapes
                row = db.execute(
                    "INSERT INTO orders (customer_id, ship_to, idempotency_key) VALUES
(?, ?, ?) "
                    "ON CONFLICT (idempotency_key) DO NOTHING RETURNING order_id",
                    (customer_id, ship_to, idempotency_key)).fetchone()
                if row is None:  # a retry of a request we already processed
                    return db.execute("SELECT order_id FROM orders WHERE idempotency_key
= ?",
                                      (idempotency_key,)).fetchone()[0]
                order_id, total = row[0], 0
                for line_no, (product_id, qty) in enumerate(sorted(items), start=1):  #
fixed lock order
                    cur = db.execute(
                        "UPDATE inventory SET on_hand = on_hand - ? WHERE product_id = ?
AND on_hand >= ?",
                        (qty, product_id, qty))
                    if cur.rowcount != 1:
                        raise OutOfStock(product_id)  # aborts the whole transaction
                    price = db.execute("SELECT price_cents FROM products WHERE
product_id = ?",
                                       (product_id,)).fetchone()[0]
                    db.execute("INSERT INTO order_items VALUES (?, ?, ?, ?, ?)",
                               (order_id, line_no, product_id, qty, price))
                    total += qty * price
                db.execute("UPDATE orders SET total_cents = ? WHERE order_id = ?", (to-
tal, order_id))
                return order_id

        ORDER_HISTORY = """
        SELECT order_id, status, total_cents, created_at
        FROM orders
        WHERE customer_id = :cid
          AND (created_at, order_id) < (:after_created, :after_id)   -- keyset cursor
        ORDER BY created_at DESC, order_id DESC
        LIMIT 20"""

        TOP_PRODUCTS = """
        SELECT p.sku, SUM(i.quantity) AS units, SUM(i.quantity * i.unit_price_cents) AS
revenue_cents
        FROM order_items i
        JOIN orders   o ON o.order_id   = i.order_id
        JOIN products p ON p.product_id = i.product_id
        WHERE o.status <> 'cancelled'
        GROUP BY p.sku
        ORDER BY revenue_cents DESC
        LIMIT 3"""
```

```
        NO_ORDERS = """
        SELECT c.email FROM customers c
        WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)"""

        db = sqlite3.connect(":memory:")  # default mode: BEGIN issued implicitly before
DML
        db.executescript(SCHEMA)
        db.executemany("INSERT INTO customers (email, full_name, city) VALUES (?, ?,
?)",
                       [("ana@example.com", "Ana", "Pune"), ("bo@example.com", "Bo",
"Lima")])
        db.executemany("INSERT INTO products (sku, name, price_cents) VALUES (?, ?, ?)",
                       [("MUG-1", "Mug", 1200), ("TEE-1", "T-shirt", 2500), ("CAP-1",
"Cap", 1800)])
        db.execute("INSERT INTO inventory SELECT product_id, 5 FROM products")

        o1 = place_order(db, 1, "1 Main St", [(2, 1), (1, 2)], "key-001")
        o1_retry = place_order(db, 1, "1 Main St", [(2, 1), (1, 2)], "key-001")
        print("order", o1, "retry returns", o1_retry)
        try:
            place_order(db, 1, "1 Main St", [(3, 99), (1, 1)], "key-002")
        except OutOfStock as e:
            print("out of stock: product", e.args[0])
        print("inventory:", db.execute("SELECT product_id, on_hand FROM
inventory").fetchall())
        print("orders:", db.execute("SELECT order_id, total_cents FROM
orders").fetchall())
        print("history:", db.execute(ORDER_HISTORY, {"cid": 1, "after_created": "9999",
"after_id": 0}).fetchall())
        print("top products:", db.execute(TOP_PRODUCTS).fetchall())
        print("no orders:", db.execute(NO_ORDERS).fetchall())
        print("plan:", db.execute("EXPLAIN QUERY PLAN " + ORDER_HISTORY,
                                  {"cid": 1, "after_created": "9999", "after_id":
0}).fetchall()[0][3])
        try:
            db.execute("INSERT INTO orders (customer_id, ship_to, idempotency_key)
VALUES (99, 'x', 'k')")
        except sqlite3.IntegrityError as e:
            print("foreign key enforced:", e)
```

What to notice:

- Output: the retry returns the same order (1); the oversell attempt raises `OutOfStock` for product 3 and leaves inventory at `[(1, 3), (2, 4), (3, 5)]`. Product 1 had already been decremented (lines are processed in `product_id` order) when product 3 failed, and the rollback restored it. Only one order exists, with total 4,900 cents.
- The history query uses a keyset cursor (§8.6.5); the first page passes a cursor greater than any real value. Its plan is `SEARCH orders USING INDEX orders_customer_created (customer_id=? AND created_at<?)`: the composite index serves both the filter and the sort.

- Best sellers: `TEE-1` (2,500 cents) then `MUG-1` (2,400 cents); the anti-join finds `bo@example.com`, who never ordered.

Table 11.13 — Failure scenarios in the order system.

What goes wrong	Why	Detection	Recovery	Prevention
Overselling	Read-then-write stock check in the application	Negative-stock or oversold reports	Cancel and refund	Conditional `UPDATE ... AND on_hand >= q` plus `CHECK (on_hand >= 0)`
Duplicate orders on client retry	Timeout after commit; client retries	Two orders with identical items seconds apart	Refund duplicate	`UNIQUE` idempotency key; `ON CONFLICT DO NOTHING`
Deadlocks at peak	Carts lock inventory rows in different orders	`deadlock detected` errors in logs	Automatic retry	Sort lines by `product_id` before updating
Checkout stalls during a sale	Payment call inside the transaction holds a hot row lock	Lock waits in `pg_stat_activity`; p99 latency	Kill long transactions	Pay outside the transaction; short transactions; split hot counters
Order history slows over time	Missing or wrong composite index; offset pagination	`EXPLAIN` shows sort or scan	Add index	Index `(customer_id, created_at DESC, order_id DESC)`; keyset pagination

11.18 Summary and revision notes

SUMMARY

- Relations are typed sets related by values; physical data independence lets the database choose access paths [Codd 1970].
- Use surrogate primary keys, `UNIQUE` natural keys and foreign keys; index the child columns you join or delete through.
- Normalize the source of truth to 3NF/BCNF on functional dependencies, with lossless splits; denormalize only for measured reads, with a named consistency mechanism. Prices at purchase time are history, not redundancy.
- Join algorithms: index nested loop for small outer inputs (OLTP), hash join for large unsorted equality joins, sort-merge when inputs are sorted. The right choice depends on row counts, so cardinality estimation drives performance.
- Composite indexes follow the leftmost-prefix rule (equality columns first, then range/sort). Covering indexes avoid table reads; partial indexes index only hot subsets. A secondary index pays off only below a break-even selectivity, about 0.4% of rows in our example.
- A B+ tree with 16 KiB pages indexes 10^9 rows in 4 levels; with internal pages cached, a lookup costs about one random I/O (two for an InnoDB secondary index). The leaf working set, not height, limits scale.
- Plans are trees of Volcano iterators; blocking operators need memory and can spill. In `EXPLAIN ANALYZE`, compare estimated and actual rows at each node.
- ACID: atomicity (undo), consistency (constraints), isolation (concurrency control), durability (WAL + fsync, §13.5). ACID consistency is unrelated to linearizability or CAP consistency.
- Isolation-level names differ by database. PostgreSQL: READ COMMITTED default, REPEATABLE READ = snapshot isolation (prevents lost update by aborting, allows write skew), SERIALIZABLE = SSI. MySQL: REPEATABLE READ default, allows lost updates and write skew for read-modify-write cycles.
- MVCC keeps old versions so readers never block writers: PostgreSQL in the heap (VACUUM, freezing, wraparound), InnoDB in undo logs (purge). Long-running transactions hurt both.
- Writers still lock rows (strict 2PL for writes); InnoDB adds gap/next-key locks. Avoid deadlocks with a canonical lock order; a hot row's throughput is at most $1/t_{\text{hold}}$, so keep slow work out of transactions.

11.19 Knowledge checkpoint

1. Why does `WHERE coupon_code <> 'SPRING'` not return rows where `coupon_code` is NULL?

2. A table `enrollment(student_id, course_id, course_title, grade)` has key `(student_id, course_id)`. Which normal form does it violate, and how do you fix it?
3. Why is storing `unit_price_cents` in `order_items` not a normalization violation?
4. Estimate the I/O cost of a block nested-loop join with $M = 1,000$, $N = 5,000$ and $B = 102$ buffer pages.
5. Which of these queries can use an index on `(customer_id, created_at)`: (a) `customer_id = 5`, (b) `created_at > X`, (c) `customer_id = 5 ORDER BY created_at`?
6. How many levels does a B+ tree need for 10^8 rows with $f = 500$ and $L = 100$?
7. What makes an operator "blocking" in the Volcano model? Name three.
8. Distinguish the C in ACID from the C in CAP.
9. Which anomaly does PostgreSQL REPEATABLE READ allow that SERIALIZABLE prevents? Give an example.
10. Does MySQL InnoDB REPEATABLE READ prevent lost updates from `SELECT` then `UPDATE ... SET x = <computed value>`?
11. Why does a long-running idle transaction cause table bloat in PostgreSQL?
12. A transaction holds a hot row's lock for 10 ms. What is the maximum update rate on that row?

Answers

1. Comparisons with NULL evaluate to UNKNOWN, and `WHERE` keeps only TRUE rows. Use `IS DISTINCT FROM` or add `OR coupon_code IS NULL`.
2. 2NF: `course_title` depends only on `course_id`, part of the key. Move it to `courses(course_id, course_title)`.
3. It records the price charged at purchase time, a different fact from the product's current price; it is determined by the order line, not by `product_id`.
4. $1,000 + \lceil 1,000/100 \rceil \times 5,000 = 1,000 + 50,000 = 51,000$ I/Os.
5. a. and (c). (b) cannot seek because `created_at` is not a leftmost prefix (only a full index scan or skip scan is possible).
6. Leaves: $10^8/100 = 10^6$. $\lceil \log_{500} 10^6 \rceil = \lceil 2.22 \rceil = 3$. Height $= 1 + 3 = 4$.
7. It must consume all input before producing its first output row: sort, hash aggregate, hash-join build side (also materialize).
8. ACID consistency: transactions preserve declared invariants in one database. CAP consistency: linearizability, i.e., every read of replicated data sees the latest completed write.
9. Write skew. Two doctors each check that two are on call and each sets themselves off call; both commit under REPEATABLE READ.
10. No. The `UPDATE` reads the latest committed row without checking it against the snapshot. Use `SELECT ... FOR UPDATE`, an atomic `SET x = x - 1`, or a version check.
11. Its snapshot may still need old row versions, so VACUUM cannot remove any version that became dead after the snapshot was taken, in any table.
12. $1/0.010 = 100$ updates per second.

11.20 Interview questions

Basic

Q1. What is a database index, and what does it cost? Answer. A sorted structure, usually a B+ tree, mapping key values to row locations, so a selective lookup costs a few page reads instead of a full scan (about 4 levels at 10^9 rows, roughly 1 I/O with internal pages cached). Costs: every write to indexed columns updates the index, extra storage and memory, and no benefit for low-selectivity predicates. Index the real workload: composite indexes with the leftmost-prefix rule, covering indexes for hot paths, drop unused ones.

Q2. Explain ACID with an example. Answer. Transferring $70 from A to B: atomicity, both debit and credit happen or neither; consistency, declared invariants (no negative balance, valid references) hold after commit; isolation, a concurrent report never sees money in flight; durability, once "committed" is returned a crash cannot lose it, because the WAL was flushed. Note that ACID consistency differs from distributed "strong consistency".

Intermediate

Q3. Two users buy the last unit of a product at the same moment. How do you guarantee you do not oversell? Answer. Make check and decrement one atomic statement, `UPDATE inventory SET on_hand = on_hand - 1 WHERE product_id = ? AND on_hand >= 1`, treat 0 rows updated as out of stock, and back it with `CHECK (on_hand >= 0)`. The row lock serializes the updates; the second finds 0. Alternatives: `SELECT ... FOR UPDATE`, or SERIALIZABLE with retries. Application read-modify-write loses updates at READ COMMITTED and at MySQL REPEATABLE READ. For a flash sale, discuss hot-row throughput ($1/t_{\text{hold}}$), payment outside the transaction and stock buckets.

Q4. A query that took 5 ms now takes 4 s after a large data import. How do you investigate? Answer. Capture `EXPLAIN (ANALYZE, BUFFERS)` and find nodes where estimated and actual rows differ by an order of magnitude; after an import, stale statistics are likely, so run `ANALYZE`. If correlated predicates keep estimates wrong, add extended statistics. Check for unusable indexes (functions, casts), spills (`external merge`, `Batches > 1`) and cold caches (`Buffers read`). Then guard with analyze settings and `pg_stat_statements` monitoring.

Advanced

Q5. Compare PostgreSQL's and InnoDB's MVCC, and explain how each fails under a long-running analytics query on the primary. Answer. PostgreSQL writes a new heap tuple per update (`xmin` / `xmax`), adds index entries for non-HOT updates, and relies

on VACUUM to remove dead versions and freeze XIDs. InnoDB updates in place, keeps prior versions in undo logs and purges them in the background. A multi-hour query holds an old snapshot: in PostgreSQL, VACUUM cannot remove newer dead versions, so hot tables bloat, index-only scans degrade and freezing stalls; in InnoDB, purge stalls, the history list grows and hot-row reads walk longer undo chains. Mitigations: run analytics on a replica or warehouse, set statement and idle-in-transaction timeouts, monitor `age(datfrozenxid)` and history list length.

Q6. Design the concurrency control for an "at least one admin per organization" rule. Answer. This is write skew: each demotion checks a count, then writes a different row. Options: (1) SERIALIZABLE (PostgreSQL SSI) with a retry loop; simplest, with occasional aborts. (2) Materialize the conflict: `SELECT ... FROM organizations WHERE org_id = ? FOR UPDATE` before counting, so changes in one organization serialize; works at READ COMMITTED and in MySQL. (3) A stored `admin_count` with `CHECK (admin_count >= 1)` updated in the same transaction. REPEATABLE READ alone is insufficient in both databases; across shards the invariant needs a single owner (Chapter 28).

11.21 Practical exercises

1. Load the §11.17 schema into PostgreSQL with one million orders (use `generate_series`). Run the order-history query before and after creating `orders_customer_created_idx` with `EXPLAIN (ANALYZE, BUFFERS)`; record pages read and time.
2. Reproduce the correlated-columns misestimate of §11.10 and fix it with `CREATE STATISTICS`. Then build a three-table join where the misestimate changes the join algorithm.
3. Run the isolation program of §11.13 at all four PostgreSQL levels and, if you have MySQL, adapt it to MySQL 8; fill in Table 11.9 from your own results.
4. Install `pageinspect`, update one row 10 times with and without an index on the updated column, and observe HOT chains, `n_tup_hot_upd` and the effect of `VACUUM`.
5. Write a load test with 50 concurrent clients buying from 5 products in random cart order. Count deadlocks, then sort cart lines by `product_id` and count again.

11.22 Design challenges

1. **Seat reservation.** Model a cinema with seats, showings and bookings so that a seat can never be double-booked, even under concurrent requests and client retries. Choose constraints, isolation level and indexes, and justify each.
2. **Ledger.** Design a double-entry ledger (accounts, journal entries, postings) in which every entry balances to zero and balances can be read in O(1). Decide what is normalized, what is a stored aggregate, and how the aggregate stays correct.

3. **Multi-tenant SaaS schema.** One PostgreSQL cluster hosts 5,000 tenants of very different sizes. Compare a shared schema with a `tenant_id` column, schema-per-tenant and database-per-tenant on index design, isolation, migrations and noisy neighbors; prepare the move of large tenants to shards (Chapter 14).

11.23 References

1. Codd, E. F. “A Relational Model of Data for Large Shared Data Banks.” *Communications of the ACM* 13(6), 1970.
2. Ramakrishnan, R., Gehrke, J. *Database Management Systems*, 3rd ed. McGraw-Hill, 2003.
3. Bayer, R., McCreight, E. “Organization and Maintenance of Large Ordered Indexes.” *Acta Informatica* 1(3), 1972.
4. Comer, D. “The Ubiquitous B-Tree.” *ACM Computing Surveys* 11(2), 1979.
5. Lehman, P. L., Yao, S. B. “Efficient Locking for Concurrent Operations on B-Trees.” *ACM Transactions on Database Systems* 6(4), 1981.
6. Kent, W. “A Simple Guide to Five Normal Forms in Relational Database Theory.” *Communications of the ACM* 26(2), 1983.
7. Selinger, P. G., Astrahan, M. M., Chamberlin, D. D., Lorie, R. A., Price, T. G. “Access Path Selection in a Relational Database Management System.” *Proceedings of ACM SIGMOD*, 1979.
8. Graefe, G. “Volcano — An Extensible and Parallel Query Evaluation System.” *IEEE Transactions on Knowledge and Data Engineering* 6(1), 1994.
9. Graefe, G. “Query Evaluation Techniques for Large Databases.” *ACM Computing Surveys* 25(2), 1993.
10. Boncz, P., Zukowski, M., Nes, N. “MonetDB/X100: Hyper-Pipelining Query Execution.” *CIDR*, 2005.
11. Neumann, T. “Efficiently Compiling Efficient Query Plans for Modern Hardware.” *Proceedings of the VLDB Endowment* 4(9), 2011.
12. Leis, V., Gubichev, A., Mirchev, A., Boncz, P., Kemper, A., Neumann, T. “How Good Are Query Optimizers, Really?” *Proceedings of the VLDB Endowment* 9(3), 2015.
13. Eswaran, K. P., Gray, J. N., Lorie, R. A., Traiger, I. L. “The Notions of Consistency and Predicate Locks in a Database System.” *Communications of the ACM* 19(11), 1976.
14. Härder, T., Reuter, A. “Principles of Transaction-Oriented Database Recovery.” *ACM Computing Surveys* 15(4), 1983.
15. Gray, J., Reuter, A. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.
16. Berenson, H., Bernstein, P., Gray, J., Melton, J., O’Neil, E., O’Neil, P. “A Critique of ANSI SQL Isolation Levels.” *Proceedings of ACM SIGMOD*, 1995.

17. Cahill, M. J., Röhm, U., Fekete, A. D. “Serializable Isolation for Snapshot Databases.” *Proceedings of ACM SIGMOD*, 2008.
18. Ports, D. R. K., Grittner, K. “Serializable Snapshot Isolation in PostgreSQL.” *Proceedings of the VLDB Endowment* 5(12), 2012.
19. Kleppmann, M. *Designing Data-Intensive Applications*. O’Reilly, 2017 (chapters 3 and 7).
20. Kleppmann, M. *Hermitage: Testing Transaction Isolation Levels*. 2014. https://github.com/ept/hermitage
21. Petrov, A. *Database Internals*. O’Reilly, 2019.
22. PostgreSQL Global Development Group. *PostgreSQL 16 Documentation: Concurrency Control*. https://www.postgresql.org/docs/16/mvcc.html
23. PostgreSQL Global Development Group. *PostgreSQL 16 Documentation: Using EXPLAIN*. https://www.postgresql.org/docs/16/using-explain.html
24. PostgreSQL Global Development Group. *PostgreSQL 16 Documentation: Routine Vacuuming*. https://www.postgresql.org/docs/16/routine-vacuuming.html
25. PostgreSQL Global Development Group. *PostgreSQL 16 Documentation: Explicit Locking*. https://www.postgresql.org/docs/16/explicit-locking.html
26. Oracle. *MySQL 8.0 Reference Manual: Transaction Isolation Levels*. https://dev.mysql.com/doc/refman/8.0/en/innodb-transaction-isolation-levels.html
27. Oracle. *MySQL 8.0 Reference Manual: InnoDB Locking*. https://dev.mysql.com/doc/refman/8.0/en/innodb-locking.html
28. Oracle. *MySQL 8.0 Reference Manual: InnoDB Multi-Versioning*. https://dev.mysql.com/doc/refman/8.0/en/innodb-multi-versioning.html
29. Oracle. *MySQL 8.0 Reference Manual: Clustered and Secondary Indexes*. https://dev.mysql.com/doc/refman/8.0/en/innodb-index-types.html
30. Oracle. *MySQL 8.0 Reference Manual: Deadlock Detection*. https://dev.mysql.com/doc/refman/8.0/en/innodb-deadlock-detection.html
31. Oracle. *MySQL 8.0 Reference Manual: Hash Join Optimization*. https://dev.mysql.com/doc/refman/8.0/en/hash-joins.html
32. Instagram Engineering. “Sharding & IDs at Instagram.” Instagram Engineering Blog, 2012.
33. Klitzke, E. “Why Uber Engineering Switched from Postgres to MySQL.” Uber Engineering Blog, 2016.
34. Sentry. “Transaction ID Wraparound in Postgres.” Sentry Blog, 2015.
35. GitHub. *gh-ost: GitHub’s Online Schema-migration Tool for MySQL*. https://github.com/github/gh-ost

11.24 Open verification items

- PostgreSQL 18 B-tree skip scan (stated as available from version 18, released 2025) and MySQL 8.4 still lacking partial indexes: confirm against current release notes.

- Sentry 2015 wraparound postmortem: title, date and details are from memory; confirm URL and wording.
- Uber 2016 blog post URL (moved from eng.uber.com); Instagram 2012 post URL.
- Exact PostgreSQL 16 thresholds for wraparound warnings and write refusal (stated qualitatively here).
- Claim that MySQL 8.0/8.4 has no sort-merge join.

Chapter 12 — NoSQL Databases

> **Level:** 2 · **Track:** A (Traditional) · **Estimated reading time:** 3.5 h

Chapter 11 described the relational database: one general-purpose model that can answer almost any question you think of later. That generality assumes the data fits on one primary node, the schema is known up front, and joins are cheap because both tables share a disk. In the mid-2000s several companies hit all three limits at once and built databases that dropped some relational guarantees in exchange for the property they needed most. The industry later grouped them under one awkward label, **NoSQL**.

Think of a city's storage options. A relational database is a well-run library: everything cataloged, cross-referenced and lendable under strict rules, but there is one building and one front desk. A **key-value store** is a wall of numbered lockers: you can find locker 4,711 instantly, but you cannot ask "which lockers contain umbrellas?". A **document database** is a filing cabinet in which each folder holds everything about one customer. A **wide-column store** is a warehouse whose aisles are sorted by a label you choose, so a forklift can sweep one aisle end to end. A **graph database** is a map of who knows whom, in which walking from one person to a friend is a single step. A **time-series database** is a weather station's logbook, written strictly in time order and summarized as it ages. Each is better than the library at one kind of question. This chapter teaches which question each family answers, how, and how to choose.

12.1 Learning objectives

After this chapter you should be able to:

- Explain the pressures that produced NoSQL and the 2006–2010 papers that defined it.
- Describe the data model, internals and failure modes of key-value, document, wide-column, graph and time-series databases.
- Design a DynamoDB single-table schema from access patterns, and compute its capacity units and billing break-even.
- Model a Cassandra table query-first and size its partitions with the documented formula.
- Choose between embedding and referencing in MongoDB and order compound indexes by the equality–sort–range rule.

- Explain when index-free adjacency beats joins, and size a time-series store using Gorilla compression and downsampling.
- Apply a selection flowchart and design a polyglot-persistence architecture, accounting for its operational cost.

12.2 Prerequisites

- Relational model, joins, B-tree indexes and ACID — §11.3–§11.12 (NoSQL is defined by what it removes).
- Range and hash partitioning — §14.6 (every family here is partitioned).
- Latency numbers — §4.17 (why single-partition reads are fast and scatter-gather is not).
- Serialization formats — §8.9 (documents are serialized objects).
- Back-of-the-envelope estimation — §2.11.

Several topics appear here only in summary because another chapter owns them: log-structured merge trees (Chapter 13), replication and consistent hashing (Chapter 14, §14.8), quorums (§27.8), and consistency models (Chapter 25).

12.3 Why NoSQL emerged

12.3.1 Three pressures on the relational database

> DEFINITION
>
> **Definition — NoSQL database.** A database that does not use the relational table-and-join model as its primary data model and typically trades some combination of cross-record transactions, ad-hoc querying and fixed schemas for horizontal scale-out, flexible schemas or availability during failures. The name is usually read as “not only SQL”; it describes a movement, not a specification.

First-principles explanation

A single relational server is a kitchen with one head chef: orders are consistent because one person sees everything. A stadium caterer serving 100,000 meals an hour must open many kitchens, and then “consistent at every instant” requires the kitchens to phone each other before every plate leaves — over phone lines that sometimes go down. NoSQL systems are the designs that accept many kitchens as the starting point.

Why does this exist?

Three pressures arrived together between roughly 2004 and 2010:

1. **Scale-out.** Data and traffic outgrew the largest single server; commodity servers were cheap, very large ones were not. A relational database can be sharded by hand (§14.7), but cross-partition joins and transactions become slow or unavailable and the application inherits the routing logic. If most requests touch one entity, why not partition by that entity natively?
2. **Schema flexibility.** Product teams changed their data shape weekly, and `ALTER TABLE` on a multi-terabyte table in 2008-era MySQL could lock or rewrite it for hours. Self-describing records let each record carry its own shape.
3. **Availability.** During a network partition a replicated system must either reject some requests or accept writes that may conflict — the trade-off formalized as the CAP theorem [Brewer 2000; Gilbert & Lynch 2002] and treated in §25.8. Several NoSQL systems chose to stay writable, because a failed "add to cart" loses money.

A fourth argument came from researchers: Stonebraker and Çetintemel argued that "one size fits all" was over and that specialized engines would beat a general row store by an order of magnitude on their own workloads [Stonebraker & Çetintemel 2005; Stonebraker et al. 2007]. NoSQL is one branch of that specialization.

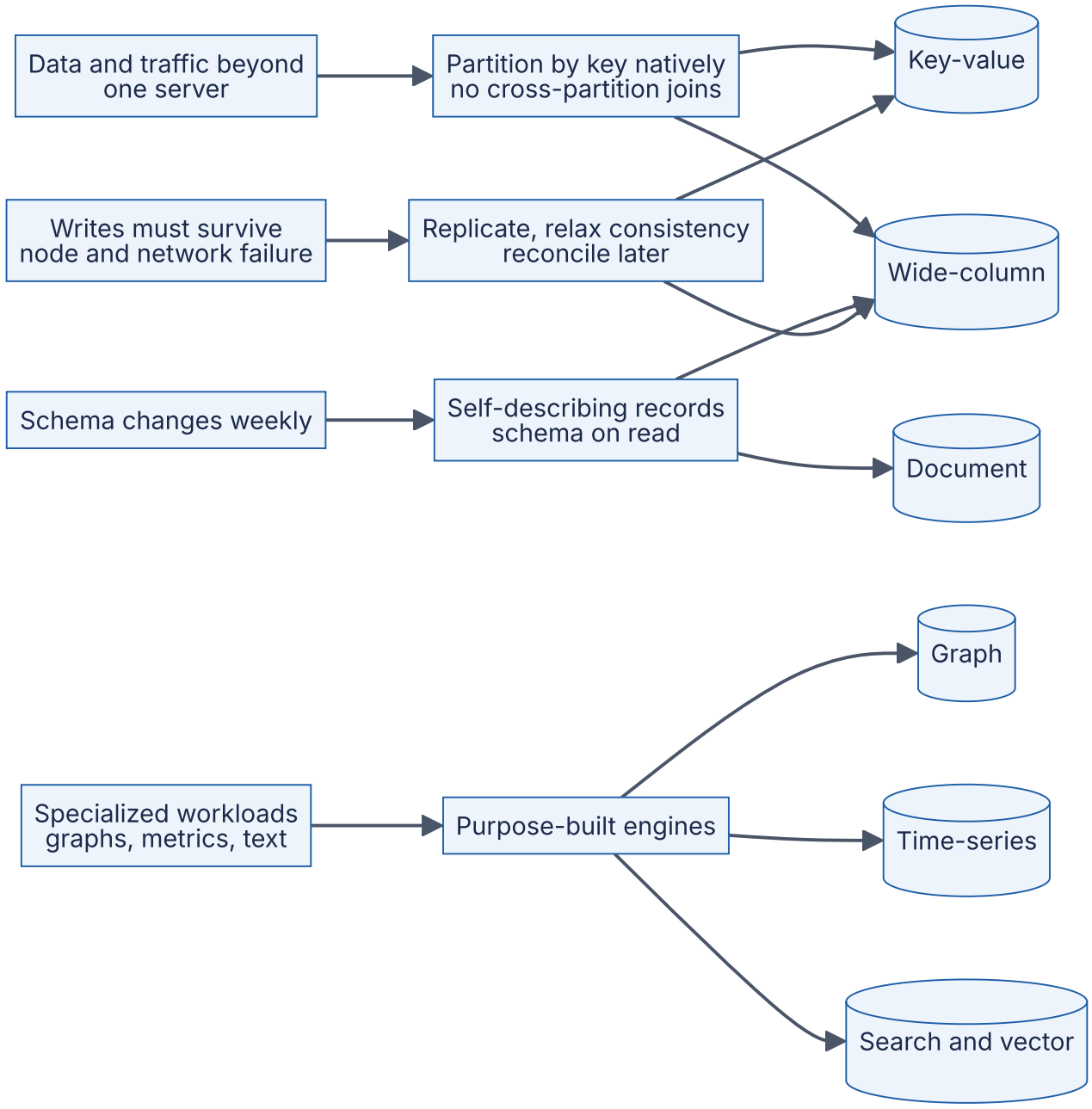
12.3.2 The papers that defined the movement

Four publications supplied most of the ideas that later products copied:

- **Bigtable** (Google, OSDI 2006) — "a sparse, distributed, persistent multidimensional sorted map" indexed by row key, column key and timestamp, stored as immutable files on GFS [Chang et al. 2006]. It is the ancestor of HBase, Cassandra's storage model and Google Cloud Bigtable.
- **Dynamo** (Amazon, SOSP 2007) — an "always writeable" key-value store for the shopping cart, combining consistent hashing, replication across N nodes, sloppy quorums, hinted handoff, vector clocks and Merkle-tree anti-entropy [DeCandia et al. 2007]. Its ideas live on in Cassandra, Riak and Voldemort; Amazon's managed DynamoDB service shares the name and lessons but a different, leader-based design [Elhemali et al. 2022].
- **PNUTS** (Yahoo!, VLDB 2008) — geographically replicated tables with per-record timeline consistency [Cooper et al. 2008].
- **Cassandra** (Facebook, LADIS 2009; SIGOPS OSR 2010) — Dynamo's distribution design married to Bigtable's data model, built for Facebook's inbox search [Lakshman & Malik 2010].

Open-source projects followed: CouchDB (2005), HBase (2008), MongoDB and Redis (2009). The label came from a 2009 San Francisco meetup on distributed, non-relational databases organized by Johan Oskarsson, who needed a Twitter hashtag [Sadalage & Fowler 2012].

Figure 12.1 — Forces that produced NoSQL and the design responses.



Reading the diagram. Pressures (left) led to design responses (middle), embodied by database families (cylinders); arrows mean “led to”, not data flow. One family can answer several pressures: wide-column stores partition natively, tolerate sparse schemas and, in Cassandra’s case, stay writable during failures. The bottom row is the “one size does not fit all” argument: graphs, metrics and text differ so much from row lookups that a specialized layout wins. There are no storage boundaries or failure paths here; each family’s failure modes appear in its own section.

12.3.3 What was given up, and what came back

Early systems typically gave up **joins** (the application stores data pre-joined), **multi-record transactions** (atomicity only within one key, row or document) and **ad-hoc**

queries (you query by designed keys, or scan). Many defaulted to **eventual consistency** — replicas converge if writes stop (§25.7).

The pendulum then swung back: MongoDB 4.0 and DynamoDB added transactions in 2018, Cassandra added Paxos-based lightweight transactions, and the **NewSQL** systems of §12.11 showed that scalable, strongly consistent SQL is possible at a latency cost. The useful question today is "which data model and consistency guarantees fit this access pattern?" — answered in §12.12.

KEY CONCEPT

Key insight — Aggregate orientation. Key-value, document and wide-column databases store an **aggregate** — related data read and written as a unit, such as an order with its line items — under one key, and atomicity, partitioning and replication happen per aggregate [Sadalage & Fowler 2012]. When access patterns follow aggregates these stores scale linearly; slicing across aggregates in unforeseen ways calls for a relational model. Graph databases are relationship-oriented instead.

COMMON MISTAKE

"NoSQL means no schema." The schema still exists, in the code that reads the data: **schema-on-read** rather than the relational **schema-on-write** [Kleppmann 2017, ch. 2]. Evolving records become easy, but every reader must handle every historical shape, so mature deployments add validation (MongoDB's `$jsonSchema`, typed CQL tables).

COMMON MISTAKE

"NoSQL is faster than SQL." A cached PostgreSQL key lookup takes tens of microseconds, like a NoSQL lookup. NoSQL systems keep latency *flat* as data grows, by partitioning and by making expensive operations (joins, scans) impossible or explicit.

12.4 Key-value stores

12.4.1 The key-value model

DEFINITION

Definition — Key-value store. A database whose primary interface is `get(key)`, `put(key, value)` and `delete(key)`, where the value is opaque to the database or has a small set of built-in types. The key alone determines where the value lives.

First-principles explanation

A coat check is a key-value store: hand over a coat, get ticket 217, later exchange 217 for the coat. The attendant never looks in the pockets, which is why it is fast and why you cannot ask “which coats have gloves?”. Because the ticket alone says which rack holds the coat, you can add racks and attendants who never talk to each other.

Why does this exist?

Many hot paths are pure lookups by identifier: session by ID, cart by user, rate-limit counter by API key. For these, a query planner and joins are overhead. Removing them makes partitioning trivial — `hash(key) mod P`, or consistent hashing (§14.8) — and keeps latency flat at any scale. **In-memory stores** such as Redis and Memcached give sub-millisecond latency and usually hold caches or ephemeral state (§20.6); **durable distributed key-value databases** such as DynamoDB, Riak and FoundationDB persist across replicas and act as a system of record.

12.4.2 Redis: a data-structure server

Redis values are typed data structures, and commands execute one at a time on a main thread, so each command — `INCR`, `ZADD` — is atomic without locks.

Table 12.1 — Redis value types and what they are for.

Type	Example command	Typical use
String	`SET session:9f3 "{...}" EX 1800`	Cached objects, sessions with TTL
Hash	`HSET user:42 name Ana plan pro`	Small objects with fields
Set	`SADD online:room7 u42`	Membership, deduplication
Sorted set	`ZINCRBY leaderboard 15 u42`	Leaderboards, rate-limit windows, priority queues
Stream	`XADD events * type click`	Append-only event log with consumer groups

Durability is optional: **RDB** snapshots write point-in-time files; the **AOF** (append-only file) logs every write and can `fsync` every second, bounding loss to about one second. Redis Cluster splits keys into 16,384 hash slots across primaries with asynchronous replicas, so an acknowledged write can be lost on failover. Licensing has shifted (as of 2026): Redis moved to source-available licenses in 2024, the Linux Foundation’s **Valkey** fork followed, and Redis 8 added an AGPLv3 option [Redis Docs].

WARNING / TRADE-OFF

Redis as a primary database. Using Redis as the only copy of business-critical data means accepting asynchronous-replication loss windows, RAM-bounded capacity (tens of times more per GB than SSD) and restore time proportional to dataset size. Decide deliberately.

12.4.3 DynamoDB: the key-value and sort-key model

DynamoDB, Amazon's managed service, is the clearest example of a durable key-value model designed for predictable latency at any scale [Elhemali et al. 2022; AWS Docs: DynamoDB].

DEFINITION

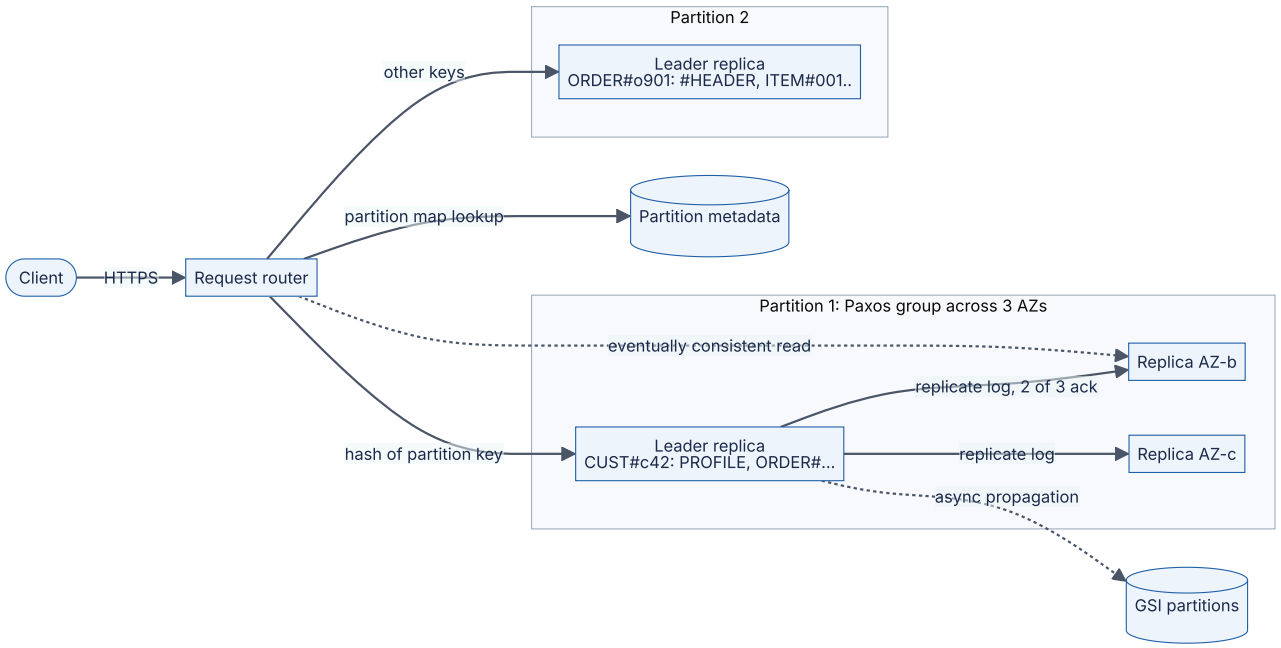
Definition — Partition key, sort key and item collection. In DynamoDB every **item** (a record of up to 400 KB, as of 2026) has a primary key made of a **partition key** and, optionally, a **sort key**. DynamoDB hashes the partition key to choose the storage partition. All items sharing a partition key form an **item collection**, stored together and ordered by sort key, so a single **Query** can return a contiguous, sorted range of them.

How it works internally

1. A client sends `PutItem`, `GetItem` or `Query` over HTTPS to stateless **request routers**, which authenticate it and hash the partition key against a cached partition map.
2. Each partition is replicated across three Availability Zones as a **Multi-Paxos** group; only the leader accepts writes and serves strongly consistent reads [Elhemali et al. 2022].
3. The leader appends a write to its write-ahead log and acknowledges once two of three replicas have persisted it. Each replica stores a B-tree plus the log.
4. An **eventually consistent read** (the default) may go to any replica and miss a just-acknowledged write; a **strongly consistent read** goes to the leader and costs twice as much.
5. Partitions that grow too large or hot are split; **adaptive capacity** shifts throughput toward hot partitions.

Every request touches one partition (or one item collection), so latency does not grow with table size.

Figure 12.2 — DynamoDB data model and request path.



Reading the diagram. Stateless request routers receive HTTPS requests and route by the hash of the partition key — never the sort key; inside a partition, item collections are kept in sort-key order. Writes are acknowledged synchronously once two of three replicas in different Availability Zones have logged them; eventually consistent reads may go to any replica (dotted). Global secondary indexes are separate partitions fed asynchronously, so GSI reads are always eventually consistent. Failure points: losing a replica is invisible; losing a leader triggers a seconds-long election for that partition only. Scaling is by partition splits, invisible to clients.

Secondary indexes: LSI and GSI

To query by anything other than the primary key you declare an index, which DynamoDB maintains on every write.

Table 12.2 — Local versus global secondary indexes (limits as of 2026).

Property	Local secondary index (LSI)	Global secondary index (GSI)
Key	Same partition key, different sort key	Any attribute(s) as new partition and sort key
When created	Only at table creation	Any time
Limit per table	5	20 by default (adjustable quota)
Consistency	Strong or eventual	Eventual only
Throughput	Shares the table's	Its own; throttling can back-pressure table writes
Size constraint	Item collection ≤ 10 GB	None beyond table limits

A **sparse index** includes only items that have the index key attribute; **index overloading** stores several entity types in one GSI under generic attribute names such as `GSI1PK`.

12.4.4 Single-table design: a worked access-pattern example

Relational design normalizes first and writes queries later. In DynamoDB you list every **access pattern** first, then design keys so each pattern is one `GetItem` or `Query`. With no joins, entities read together share an item collection, often in one table per service — **single-table design** [DeBrie 2020; AWS Docs: DynamoDB]. Consider an order service:

Table 12.3 — Access patterns and the operation that serves each.

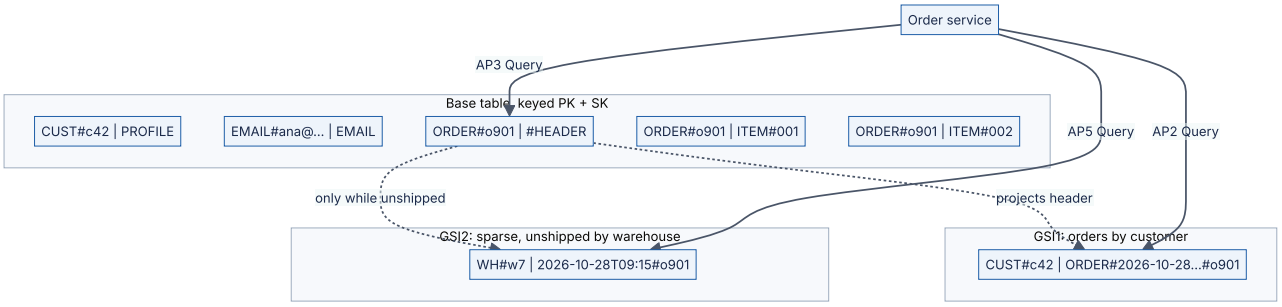
#	Access pattern	Operation
AP1	Get a customer's profile	`GetItem PK=CUST#c42, SK=PROFILE`
AP2	List a customer's orders, newest first	`Query GSI1 PK=CUST#c42, SK begins_with ORDER#`, descending
AP3	Get one order with all its line items	`Query PK=ORDER#o901` (strongly consistent)
AP4	Find a customer by e-mail; keep e-mail unique	`GetItem PK=EMAIL#ana@example.com`
AP5	List a warehouse's unshipped orders, oldest first	`Query GSI2 PK=WH#w7` (sparse)

Table 12.4 — Item layout in the single table.

PK	SK	GSI1PK / GSI1SK	GSI2PK / GSI2SK	Other attributes
`CUST#c42`	`PROFILE`	—	—	name, e-mail, address
`EMAIL#ana@example.com`	`EMAIL`	—	—	customerId
`ORDER#o901`	`#HEADER`	`CUST#c42` / `ORDER#2026-10-28T09:15#o901`	`WH#w7` / `2026-10-28T09:15#o901`	status, total
`ORDER#o901`	`ITEM#001`	—	—	sku, qty, price
`ORDER#o901`	`ITEM#002`	—	—	sku, qty, price

In this design AP3 is one Query (the header sorts first because `#` precedes letters in byte order); AP2's ISO-8601 sort keys make lexical order equal time order; AP4 doubles as a uniqueness constraint, because sign-up writes the profile and `EMAIL#` item in one `TransactWriteItems` call conditioned on `attribute_not_exists(PK)`; and AP5 is sparse — shipping an order removes `GSI2PK`, dropping it from GSI2.

Figure 12.3 — Single-table design with two global secondary indexes.



Reading the diagram. The order service issues synchronous Queries (solid) against the base table or an index, each touching one partition key. Dotted arrows are asynchronous index propagation: a header write is copied into GSI1, and into GSI2 only while it carries the warehouse attribute. Each GSI is a separate storage boundary with its own partitions and capacity. Failure points: a throttled GSI slows base-table writes, and GSI reads can briefly miss a new order. Scaling: every pattern stays one partition-local request regardless of customer count.

Code example

This runnable program imitates the key model so you can experiment without an AWS account.

```
        """A toy single-table store that mimics DynamoDB's key model:
        hash(partition key) -> partition; items sorted by sort key inside it;
        global secondary indexes maintained from attributes on each item."""
        import bisect
        import hashlib
        from collections import defaultdict

        class Table:
            def __init__(self, partitions=4, gsis=()):
                self.partitions = [defaultdict(list) for _ in range(partitions)]
                self.gsis = {name: Table(partitions) for name in gsis}   # name -> index
table

            def _part(self, pk):
                h = int(hashlib.md5(pk.encode()).hexdigest(), 16)
                return self.partitions[h % len(self.partitions)]

            def put(self, item):
                rows = self._part(item["PK"])[item["PK"]]          # the item collection
                keys = [r["SK"] for r in rows]
                i = bisect.bisect_left(keys, item["SK"])
                if i < len(rows) and rows[i]["SK"] == item["SK"]:
                    rows[i] = item                                   # overwrite same
key
                else:
                    rows.insert(i, item)                             # keep sorted by SK
                for name, index in self.gsis.items():                # sparse: only if
keys present
                    if f"{name}PK" in item:
                        index.put({**item, "PK": item[f"{name}PK"], "SK": item[f"
{name}SK"]})

            def query(self, pk, sk_prefix="", newest_first=False):
                rows = [r for r in self._part(pk).get(pk, []) if
r["SK"].startswith(sk_prefix)]
                return rows[::-1] if newest_first else rows

        t = Table(gsis=("GSI1", "GSI2"))
        t.put({"PK": "CUST#c42", "SK": "PROFILE", "name": "Ana", "email":
"ana@example.com"})
        t.put({"PK": "EMAIL#ana@example.com", "SK": "EMAIL", "customer": "c42"})
        for oid, ts, wh in [("o900", "2026-10-21T08:02", None), ("o901", "2026-10-
28T09:15", "WH#w7")]:
            header = {"PK": f"ORDER#{oid}", "SK": "#HEADER", "total": 59.0,
                      "GSI1PK": "CUST#c42", "GSI1SK": f"ORDER#{ts}#{oid}"}
            if wh:                                       # only unshipped orders enter GSI2
                header |= {"GSI2PK": wh, "GSI2SK": f"{ts}#{oid}"}
            t.put(header)
            for n in (1, 2):
                t.put({"PK": f"ORDER#{oid}", "SK": f"ITEM#{n:03d}", "sku": f"SKU-{n}",
"qty": n})

        print("AP1 profile:  ", t.query("CUST#c42", "PROFILE")[0]["name"])
        print("AP2 orders:   ", [r["SK"] for r in t.gsis["GSI1"].query("CUST#c42",
"ORDER#", newest_first=True)])
        print("AP3 order:    ", [r["SK"] for r in t.query("ORDER#o901")])
```

```
        print("AP4 by email: ", t.query("EMAIL#ana@example.com")[0]["customer"])
        print("AP5 to ship:  ", [r["SK"] for r in t.gsis["GSI2"].query("WH#w7")])
```

Output (Python 3.10+):

```
AP1 profile:   Ana
AP2 orders:    ['ORDER#2026-10-28T09:15#o901', 'ORDER#2026-10-21T08:02#o900']
AP3 order:     ['#HEADER', 'ITEM#001', 'ITEM#002']
AP4 by email:  c42
AP5 to ship:   ['2026-10-28T09:15#o901']
```

What to notice:

- `_part` hashes only the partition key, which is why a Query must name exactly one.
- `bisect` keeps item collections sorted, so sort-key prefix and range conditions are contiguous reads.
- `put` skips an index when its key is absent — that is all a sparse index is; order `o900` never enters GSI2.
- Real GSIs update asynchronously and allow duplicate keys; the toy does neither.

12.4.5 Capacity: read and write units

DynamoDB meters throughput in units. One **write capacity unit (WCU)** is one write per second of an item up to 1 KB; one **read capacity unit (RCU)** is one strongly consistent read per second of up to 4 KB, or two eventually consistent reads (as of 2026) [AWS Docs: DynamoDB]. Sizes are rounded *up*: a GetItem rounds each item, but a Query sums the sizes of all returned items before rounding.

FORMULA

DynamoDB capacity units

$$\text{WCU} = \sum_{\text{write types}} r_w \times \lceil \frac{s_w}{1\,\text{KB}} \rceil \times t_w \qquad \text{RCU} = \sum_{\text{read types}} r_r \times \lceil \frac{s_r}{4\,\text{KB}} \rceil \times c_r$$

Symbol	Meaning	Units
r_w, r_r	Rate of a given write or read type, including index writes	operations/second
s_w	Size of the item written (for a GSI, the projected size)	KB
s_r	Item size for GetItem; total returned size for Query	KB
t_w	1 for standard writes, 2 for transactional writes	dimensionless
c_r	0.5 eventually consistent, 1 strongly consistent, 2 transactional	dimensionless

WORKED EXAMPLE

Worked example — Sizing the order table at peak.

Peak load: 1,000 orders/s placed. Each order writes a 1.5 KB header and three 0.6 KB line items. GSI1 projects the whole header (1.5 KB); GSI2 projects keys only (under 1 KB). Reads: 10,000 order-detail lookups/s (AP3), eventually consistent, each returning 1.5 + 3 × 0.6 = 3.3 KB.

1. Header $1,000 \times \lceil 1.5 \rceil = 2,000$ WCU; line items $3,000 \times \lceil 0.6 \rceil = 3,000$ WCU.
2. GSI1 header copy $1,000 \times 2 = 2,000$ WCU; GSI2 copy $1,000 \times 1 = 1,000$ WCU.
3. Total $8,000$ WCU, of which 3,000 (37.5 %) is index maintenance.
4. Query reads: $\lceil 3.3/4 \rceil = 1$ unit × 0.5 → $10,000 \times 0.5 = 5,000$ RCU.
5. Four GetItem calls instead round each item: $10,000 \times 4 \times 0.5 = 20,000$ RCU.

Interpretation. Co-locating items cut read cost by 75 %; index maintenance is over a third of the write bill.

Engineering implication. Project only needed attributes into indexes, keep items small, read related items with one Query — and check the per-partition ceiling of 3,000 RCU and 1,000 WCU (as of 2026). A warehouse receiving over about 1,000 orders/s would throttle its single `WH#w7` GSI2 key whatever the table capacity, so shard it as `WH#w7#0` … `WH#w7#9` and query all ten.

Billing is **provisioned** (per WCU-hour and RCU-hour) or **on-demand** (per request unit used); the choice reduces to one ratio.

FORMULA

On-demand versus provisioned break-even utilization

$$u^{*} = \frac{p_{\text{prov}}/3600}{p_{\text{od}}}$$

Symbol	Meaning	Units
u^{*}	Average utilization of provisioned capacity above which provisioned is cheaper	fraction (0–1)
p_{prov}	Price of one provisioned capacity unit for one hour	USD per unit-hour
p_{od}	Price of one on-demand request unit	USD per request unit

WORKED EXAMPLE

Worked example — Which billing mode for the order table?

Assume us-east-1 list prices as of 2026 (verify before use): provisioned \$0.00065 per WCU-hour and \$0.00013 per RCU-hour; on-demand \$0.625 per million write request units and \$0.125 per million read request units.

1. Writes: $u^{*} = (0.00065/3600)/(0.625 \times 10^{-6}) = 1.806 \times 10^{-7}/6.25 \times 10^{-7} = 0.289$; reads give the same 0.289.
2. The table averages 30 % of peak. Provisioned at peak for 720 hours: $8,000 \times 0.00065 \times 720 = \$3,744$ plus $5,000 \times 0.00013 \times 720 = \$468$, total **\$4,212**.
3. On-demand over 2,592,000 s: $2,400 \times 2,592,000 = 6.22 \times 10^{9}$ write units → $\$3,888$; $1,500 \times 2,592,000 = 3.89 \times 10^{9}$ read units → \$486; total **\$4,374**.

Interpretation. At 30 % utilization the modes cost within 4 %, as the break-even predicts.

Engineering implication. Spiky traffic (average far below 29 % of peak) favors on-demand; steady traffic with auto scaling near 70 % utilization costs about 40 % of on-demand when provisioned. Storage and replication are billed separately.

Real-world industry applications

INDUSTRY EXAMPLE

Amazon DynamoDB at Prime Day. Amazon needed a store whose latency did not degrade with scale or noisy neighbors. DynamoDB combines hash partitioning, three-way Multi-Paxos replication, global admission control and splitting for heat; the ATC 2022 paper reports a peak of 89.2 million requests per second during Prime Day 2021 at single-digit-millisecond latency. The trade-off is a restrictive query model. *Evidence: Primary* — [Elhemali et al. 2022]

INDUSTRY EXAMPLE

Dynamo's shopping cart. The cart must accept "add to cart" even during partitions; divergent versions are tracked with vector clocks and merged by union, which can resurrect deleted items — availability bought with correctness. *Evidence: Primary* — [DeCandia et al. 2007]

Advantages and disadvantages

- **Advantages:** predictable latency at any size; near-linear scale-out; fully managed options; per-item TTL deletes without consuming write capacity.
- **Disadvantages:** only designed access patterns are cheap; new patterns need indexes or backfills; no joins or aggregations; hot keys cap throughput; single-table schemas are hard to read.

Failure scenarios

Table 12.5 — Key-value store failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Throttling with spare table capacity	One partition key exceeds the per-partition limit	Throttle metrics with low table utilization; key-level contributor insights	Write-shard the key with a suffix	Choose high-cardinality partition keys (§14.10)
Base-table writes throttled	GSI under-provisioned or hot	GSI throttle events	Raise GSI capacity	Size GSIs like tables; sparse projections
Stale read after write	Eventually consistent read or GSI read	Users see missing items briefly	Retry or read from base table strongly	Strong reads where read-your-writes matters (§25.6)

Common misconceptions

COMMON MISTAKE

“Scan is fine for small tables.” A Scan reads and bills every item, and tables grow. Design a key or index for each online pattern; keep Scan for offline jobs.

12.5 Document databases

12.5.1 The document model

DEFINITION

Definition — Document database. A database that stores self-describing, hierarchical records — **documents**, usually JSON or a binary encoding of it such as BSON — grouped into **collections**, and that can index and query fields inside the documents, including nested fields and array elements.

First-principles explanation

A doctor’s paper chart is a document: allergies, visits and each visit’s prescriptions sit in one folder, and a new kind of test result is just a new page. The cost appears with questions that cut across folders — “who was prescribed drug X last month?” — which need an index or opening every folder.

Why does this exist?

Application objects contain lists and nested objects; normalized tables (§11.5) split them apart and reassemble them with joins — the **object-relational impedance mismatch**. A document database stores the object as the application sees it, so one aggregate is one read and a new field needs no migration. MongoDB (2009) made the model mainstream; PostgreSQL’s `jsonb` brings much of it into a relational engine.

How it works internally

Using MongoDB as the reference [MongoDB Docs]:

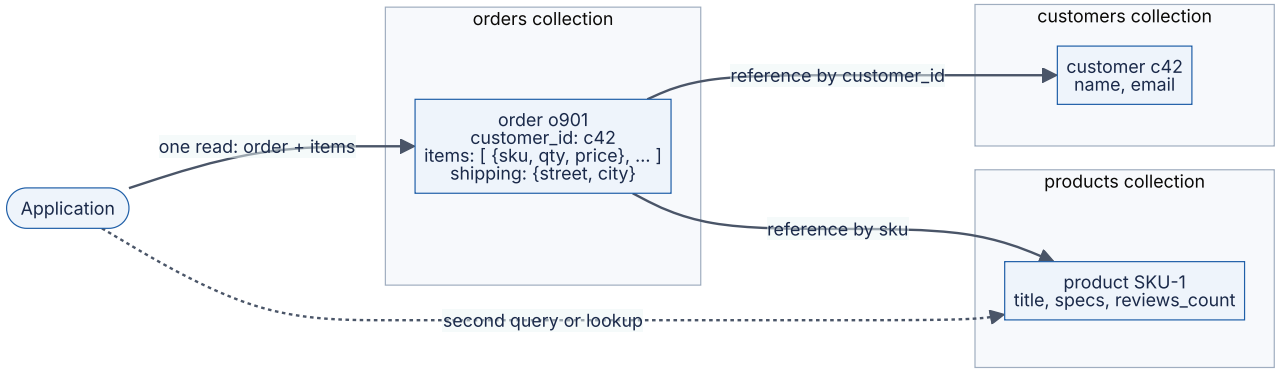
1. Documents are stored as **BSON**, a typed binary JSON (dates, decimals, binary), up to 16 MB each, in WiredTiger B-trees with document-level concurrency control and compression.
2. Every collection has a unique `_id` index; secondary indexes are separate B-trees, and an index on an array field is **multikey** — one entry per element.

3. A **replica set** (typically three members) elects a primary with a Raft-like protocol; secondaries replicate its operation log (**oplog**) asynchronously. With **write concern** `majority` — the default since MongoDB 5.0 — an acknowledged write survives failover.
4. **Sharding** splits a collection by a **shard key** (ranged or hashed) into chunks across replica sets; a `mongos` router targets one shard when the query includes the shard key and scatter-gathers otherwise (§14.11).

12.5.2 Embedding versus referencing

The central modeling decision is whether related data goes *inside* a document or *in another document* that you point to.

Figure 12.4 — Embedding versus referencing in a document model.



Reading the diagram. The application reads an order in one synchronous request because line items and the address are **embedded**: owned by the order, bounded and always read with it. Customers and products are **referenced** by ID because they are shared and change independently; fetching them needs a second query or a `$lookup` (dotted, optional). Each collection is a storage boundary that may sit on a different shard, so a reference can cost a network hop. Failure point: an embedded price is a deliberate snapshot, but an embedded copy expected to stay current will drift. Scaling: shard orders by customer or order ID; one order stays single-shard.

Table 12.6 — Embed or reference?

Embed when	Reference when
The child is read with the parent nearly every time	The child is often read on its own
The relationship is one-to-few and bounded	One-to-many is large or unbounded (comments, events)
The child has no identity outside the parent	Many parents share the child (many-to-many)
You need atomic updates of parent and child together	The child changes frequently and independently
Snapshot semantics are desired (price at purchase)	You need a single current value everywhere

> WARNING / TRADE-OFF
>
> **The unbounded array.** Embedding every comment or reading makes a document grow forever until the 16 MB limit rejects writes. Use the **bucket pattern** (one document per entity per hour or per 100 events) or a separate collection.

12.5.3 Indexes and queries

MongoDB offers compound, multikey, text, geospatial, hashed, wildcard, partial, unique and TTL indexes, up to 64 per collection; for compound indexes its documentation recommends the **ESR rule** — **E**quality fields first, then **S**ort, then **R**ange [MongoDB Docs].

```
        // Orders of one customer, newest first, above a value threshold
        db.orders.createIndex({ customer_id: 1, placed_at: -1, total: 1 });   // E, S, R

        db.orders.find({ customer_id: "c42", total: { $gte: 50 } })
                 .sort({ placed_at: -1 })
                 .limit(20);

        // A transaction across two documents (replica set or sharded cluster)
        const session = db.getMongo().startSession();
        session.withTransaction(() => {
          const s = session.getDatabase("shop");
          s.inventory.updateOne({ sku: "SKU-1", qty: { $gte: 2 } }, { $inc: { qty: -2 }
});
          s.orders.insertOne({ _id: "o902", customer_id: "c42", items: [{ sku: "SKU-1",
qty: 2 }] });
        }, { writeConcern: { w: "majority" } });
```

What to notice:

- The index jumps to one customer, finds entries already in `placed_at` order (no in-memory sort) and filters `total` while walking. With `total` before `placed_at`, the sort could not use the index.
- Multi-document ACID transactions exist since 4.0 (2018) on replica sets and 4.2 (2019) on sharded clusters; the default lifetime limit is 60 seconds, so keep them small.
- If stock is insufficient, `updateOne` matches nothing but does not throw; real code must check `modifiedCount` and abort.

Real-world industry applications

INDUSTRY EXAMPLE

Product catalog with heterogeneous attributes. Shoes have sizes, laptops CPUs, books ISBNs. Relationally this becomes an entity-attribute-value table or dozens of sparse columns; as documents each product carries its own attributes, a wildcard index covers searchable ones, and a catalog page is one read. *Evidence: Illustrative* — a common teaching pattern; not a claim about any company.

Advantages and disadvantages

- **Advantages:** natural object mapping; one-read aggregates; flexible schema; rich indexes and aggregation; built-in replication, sharding and transactions.
- **Disadvantages:** many-to-many needs application joins or `$lookup`; denormalized copies drift; poor shard keys create hot shards; schema drift without validation; queries lacking the shard key fan out.

Failure scenarios

Table 12.7 — Document database failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Acknowledged writes vanish after failover	Write concern below `majority`; primary failed before replication	Rollback files on old primary	Reconcile from rollback data	Use `w: "majority"` for important writes
Hot shard	Monotonic shard key (timestamps, ObjectIds) routes all inserts to one chunk	Per-shard insert rate skew	Reshard (supported since 5.0)	Hashed or compound shard key
Slow queries after growth	Missing or misordered compound index; collection scan	`explain()` shows `COLLSCAN`; slow-query log	Add ESR-ordered index	Review query shapes in code review

Common misconceptions

COMMON MISTAKE

“Documents mean you never need transactions.” Single-document writes are atomic, but an invariant spanning two documents — stock and order, debit and credit — needs a transaction or an idempotent workflow (§32.7). Keep invariants inside one document where you can; use transactions where you cannot.

12.6 Wide-column stores

12.6.1 The Bigtable model

> DEFINITION
>
> **Definition — Wide-column store.** A database whose rows are identified by a key, sorted (globally or within a partition) by that key, and contain a sparse, variable set of columns grouped into **column families**; each row may have millions of columns and each cell may hold several timestamped versions. Bigtable, HBase, Cassandra and ScyllaDB belong to this family.

First-principles explanation

Picture a spreadsheet with rows sorted by key, so wide that most cells are empty — and empty cells cost nothing. Reading one row, or sweeping every row starting with `com.cnn.`, is cheap because neighbors are stored together; finding rows by a column's value is not.

Why does this exist?

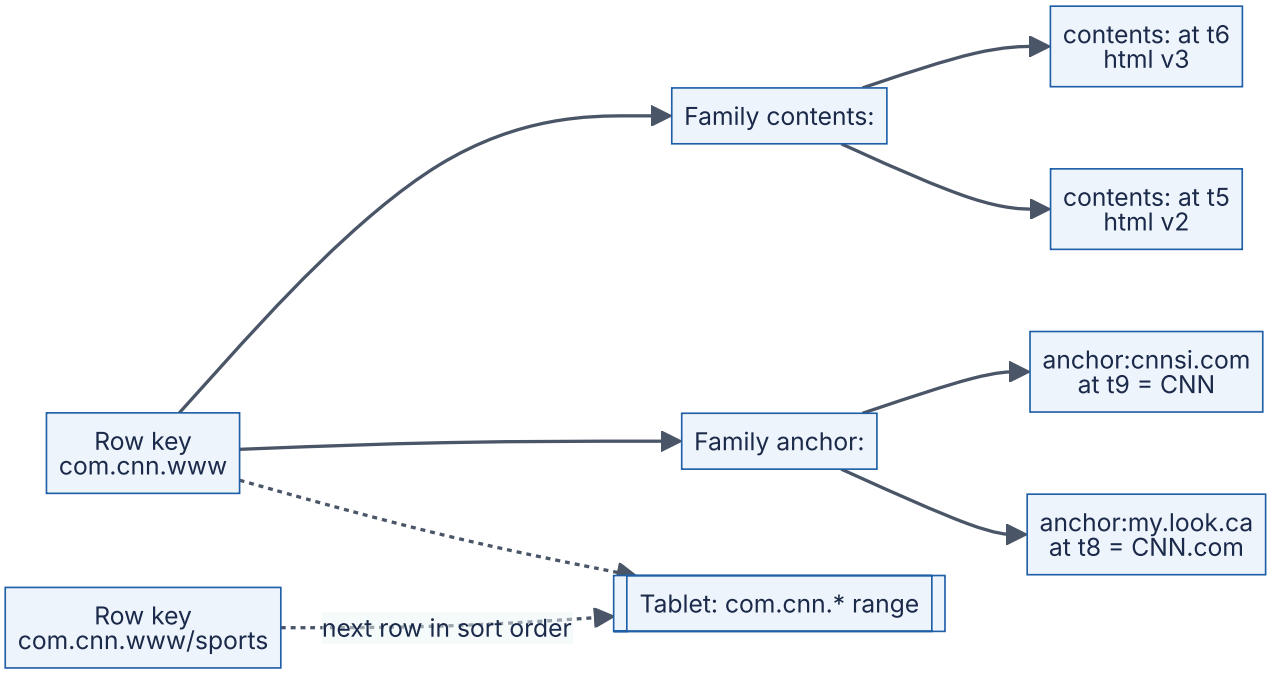
Google needed one store for the web crawl, Earth imagery and analytics across over sixty products, serving both batch scans and low-latency reads on thousands of commodity machines [Chang et al. 2006]. Relational schemas were too rigid for pages with arbitrary inbound links, and key-value stores could not scan ranges.

How it works internally

The Bigtable paper defines the model precisely: `(row: string, column: string, time: int64) → string` [Chang et al. 2006].

1. **Rows** are sorted by row key; single-row reads and writes are atomic.
2. **Column families** are declared in the schema and tuned together; **qualifiers** within a family are created freely at write time (`anchor:cnnsi.com`).
3. **Timestamps** version each cell; garbage collection keeps the last *n* versions or recent ones.
4. Contiguous row ranges called **tablets** (about 100–200 MB in the paper) are the unit of distribution; they split as they grow.
5. Writes go to a commit log and a memtable, flushed to immutable SSTables on GFS and merged by compaction — the log-structured merge design of §13.8.

Figure 12.5 — Bigtable data model: the Webtable example.



Reading the diagram. A Webtable row is keyed by the reversed URL `com.cnn.www`, so a domain's pages sort together into one tablet. Family `contents:` holds timestamped page versions; family `anchor:` has one qualifier per referring site, valued with the link text. Solid arrows show containment; dotted arrows show adjacent rows sharing a tablet, the storage and distribution unit. Failure point: a crashed tablet server's tablets are briefly unavailable until reassigned and the commit log replayed. Scaling: tablets split and move between servers.

HBase reproduces this design on Hadoop (regions, RegionServers, HDFS, ZooKeeper) with strongly consistent single-row operations; Facebook built its 2010 Messages product on it [Borthakur et al. 2011].

12.6.2 Cassandra: partition key and clustering columns

Cassandra keeps Bigtable's sorted layout within a partition but distributes data the Dynamo way: no master, and a Murmur3 hash of the partition key gives a **token** that decides the owning nodes on a ring (consistent hashing, §14.8) [Lakshman & Malik 2010; Apache Cassandra Docs].

DEFINITION

Definition — Primary key in CQL. In Cassandra Query Language, `PRIMARY KEY ((a, b), c, d)` declares `(a, b)` as the **partition key** — hashed to choose the replicas and defining what is stored together — and `c, d` as **clustering columns**, which order rows *within* the partition on disk.

1. Any node can **coordinate** a request, forwarding it to the RF replicas (replication factor, typically 3, across racks or zones), each of which writes its commit log and memtable (§13.8).
2. The client picks a **consistency level** per request — `ONE`, `QUORUM`, `LOCAL_QUORUM`, `ALL`; with RF = 3, `QUORUM` reads and writes overlap ($R + W > N$, §27.8).
3. Missed writes are repaired by hinted handoff, read repair and Merkle-tree anti-entropy (§27.9, §27.13); concurrent writes to a cell resolve by **last-write-wins** on timestamps (§27.14).

12.6.3 Query-first modeling with CQL

Because a query can efficiently read only one partition (or a contiguous slice of it), Cassandra modeling starts from the queries and creates **one table per query**, duplicating data [Chebotko et al. 2015; Apache Cassandra Docs]. Suppose an IoT platform must answer:

- Q1: readings of one sensor for one day, newest first.
- Q2: the latest reading of every sensor at one site.

```
-- Q1: one partition per sensor per day (the "bucket")
CREATE TABLE readings_by_sensor_day (
    sensor_id   uuid,
    day         date,
    ts          timestamp,
    temperature float,
    humidity    float,
    battery     float,
    PRIMARY KEY ((sensor_id, day), ts)
) WITH CLUSTERING ORDER BY (ts DESC)
  AND default_time_to_live = 7776000;      -- 90 days

-- Q2: one partition per site; one row per sensor, overwritten on each reading
CREATE TABLE latest_by_site (
    site_id     text,
    sensor_id   uuid,
    ts          timestamp,
    temperature float,
    PRIMARY KEY ((site_id), sensor_id)
);

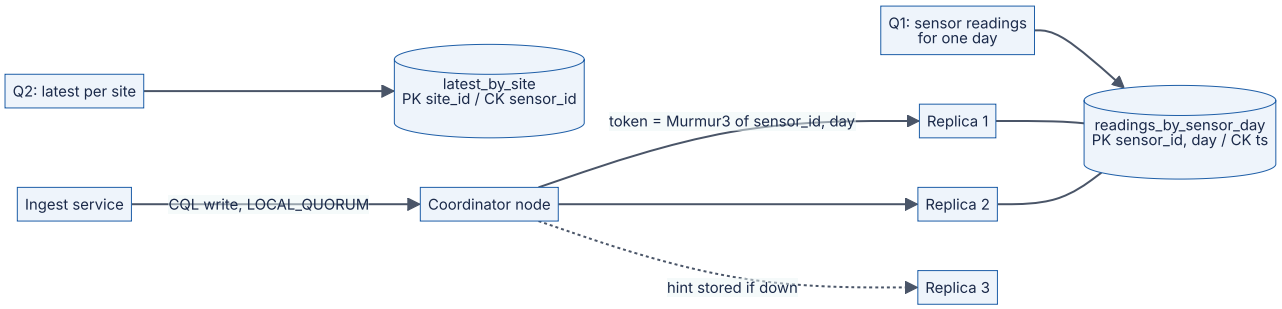
SELECT ts, temperature FROM readings_by_sensor_day
 WHERE sensor_id = 5b6e0f6a-1c1e-4a51-9a52-0c2f7f0b7d11 AND day = '2026-10-30'
 LIMIT 100;
```

What to notice:

- `(sensor_id, day)` bounds each partition to one sensor-day; without `day` it would grow forever.

- `CLUSTERING ORDER BY (ts DESC)` makes “latest 100” a sequential read of the partition’s head.
- Every reading is written to both tables — duplication is the price of single-partition reads.
- `WHERE temperature > 30` is rejected without `ALLOW FILTERING`, which scans: a sign the model lacks a table.
- TTL-expired cells become **tombstones** until compaction removes them.

Figure 12.6 — Query-first modeling and partition placement in Cassandra.



Reading the diagram. Left: each query gets a table whose partition key matches its equality conditions and whose clustering column gives its order. Right: the runtime write path — the ingest service sends a CQL write to any node, which hashes the partition key to a token and forwards to the three owning replicas. With `LOCAL_QUORUM` the write succeeds synchronously after two local acknowledgments; a down replica gets it later via a stored hint (dotted, asynchronous). Each table is a separate storage structure, so ingest writes twice. Failure points: one replica down is tolerated; two down fails `QUORUM` for those partitions. Scaling: add nodes and token ranges redistribute.

12.6.4 Sizing partitions

Oversized partitions slow compaction, repair and reads and concentrate load on *RF* nodes. The Cassandra documentation gives formulas for a partition’s cell count and size [Apache Cassandra Docs].

FORMULA

Cassandra partition cells and size

$$N_v = N_r \, (N_c - N_{pk} - N_s) + N_s$$

$$S_t = \sum_I \text{sizeOf}(c_{k_i}) + \sum_J \text{sizeOf}(c_{s_j}) + N_r \times (\sum_K \text{sizeOf}(c_{r_k}) + \sum_L \text{sizeOf}(c_{c_l})) + N_v \times \text{sizeOf}(t_{\text{avg}})$$

Symbol	Meaning	Units
N_v	Number of cells (values) in the partition	cells
N_r	Number of rows in the partition	rows
N_c	Number of columns in the table	columns
N_{pk}	Number of partition-key columns	columns
N_s	Number of static columns	columns
S_t	Estimated partition size on disk, one replica, before compression	bytes
c_k, c_s, c_r, c_c	Partition-key, static, regular and clustering columns	bytes each
t_{avg}	Average per-cell metadata (the docs use the 8-byte timestamp)	bytes

WORKED EXAMPLE

Worked example — Sizing `readings_by_sensor_day`.

Each sensor reports every 10 s. Column sizes: `sensor_id` uuid 16 B, `day` date 4 B, `ts` timestamp 8 B, three floats 4 B each. $N_c = 6$, $N_{pk} = 2$, $N_s = 0$, $t_{\text{avg}} = 8$ B.

1. Rows per day: $86,400/10 = 8,640$.
2. Cells: $N_v = 8,640 \times (6 - 2 - 0) + 0 = 34,560$.
3. Size: partition key $16 + 4 = 20$ B; per row regular + clustering $= 12 + 8 = 20$ B; so $S_t = 20 + 8,640 \times 20 + 34,560 \times 8 = 20 + 172,800 + 276,480 = 449,300$ B $\approx$ **0.45 MB**.
4. Counterfactual — no `day` bucket, one year of data: $N_r = 3,153,600$, $N_v = 12,614,400$, $S_t \approx 63.1 + 100.9 = 164$ MB, and still growing.
5. Cluster: 10^6 sensors × 0.45 MB/day × RF 3 ≈ **1.35 TB/day** uncompressed; about 121 TB at the 90-day TTL.

Interpretation. Daily buckets keep partitions near 0.45 MB, far below the widely cited guidance of about 100 MB (and the hard limit of two billion cells); the unbucketed design passes 100 MB within a year and keeps growing. Per-cell metadata is 61 % of the bucketed size.

Engineering implication. Size buckets to what one query reads: “last 7 days” reads 7 partitions in parallel, which is fine. Use the uncompressed estimate as the safe bound.

Real-world industry applications

INDUSTRY EXAMPLE

Discord’s message store. Discord stored messages in Cassandra with primary key `((channel_id, bucket), message_id)` — a fixed time bucket and a time-ordered Snowflake ID — so recent messages are one partition slice. Its 2017 post covers bucketing, tombstones and hot partitions; a 2023 post covers migrating trillions of messages to ScyllaDB, a C++ Cassandra-compatible database, to cut tail latency. *Evidence: Primary* — [Discord 2017; Discord 2023]

Advantages and disadvantages

- **Advantages:** linear write scaling; masterless, no single point of failure (Cassandra); built-in multi-datacenter replication; per-request consistency; fast time-ordered slices.
- **Disadvantages:** one table per query multiplies writes; no joins; secondary indexes fan out (Cassandra 5.0’s Storage-Attached Indexes help); tombstones; last-write-wins drops concurrent updates; repair and compaction need skill.

Failure scenarios

Table 12.8 — Wide-column store failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Read timeouts on one partition	Large or hot partition; one key gets most traffic	Partition-size histograms; per-node latency skew	Re-model with buckets; migrate data	Size partitions with S_t; bounded buckets
Queries fail on tombstones	Delete-heavy or queue-like table; scans pass many tombstones	Tombstone warnings (default warn at 1,000, fail at 100,000 per query)	Compact; change model	Avoid queue patterns; use TTL with time-window compaction
Deleted data reappears	Node down longer than `gc_grace_seconds` (default 10 days) missed the tombstone	Zombie rows after node return	Rebuild the node rather than rejoining it	Run repair within the grace period
Lost concurrent update	Last-write-wins on cell timestamps; client clock skew	Application-level audits	Re-apply from source	Lightweight transactions (Paxos) for conditional writes; synchronized clocks

Common misconceptions

COMMON MISTAKE

"Cassandra is eventually consistent, so it cannot hold anything important." Consistency is per request: `QUORUM` reads and writes with RF = 3 return the latest acknowledged value in normal operation, and lightweight transactions (`IF NOT EXISTS`) give linearizable compare-and-set within a partition. What Cassandra lacks is multi-partition transactions (§25.10).

12.7 Graph databases

12.7.1 Property graphs and RDF

DEFINITION

Definition — Graph database. A database whose data model is a graph of **vertices** (nodes) and **edges** (relationships), optimized for traversing edges. In the **property graph** model, nodes carry labels, edges carry a type and a direction, and both carry key-value properties. In the **RDF** model, all data is a set of subject–predicate–object **triples**.

First-principles explanation

Asked “who knows someone who knows a cardiologist?”, you think of your friends and theirs — you never consult a city-wide list of friendships. A graph database stores each node with direct links to its neighbors, so a neighborhood question costs time proportional to the neighborhood, not the population.

Why does this exist?

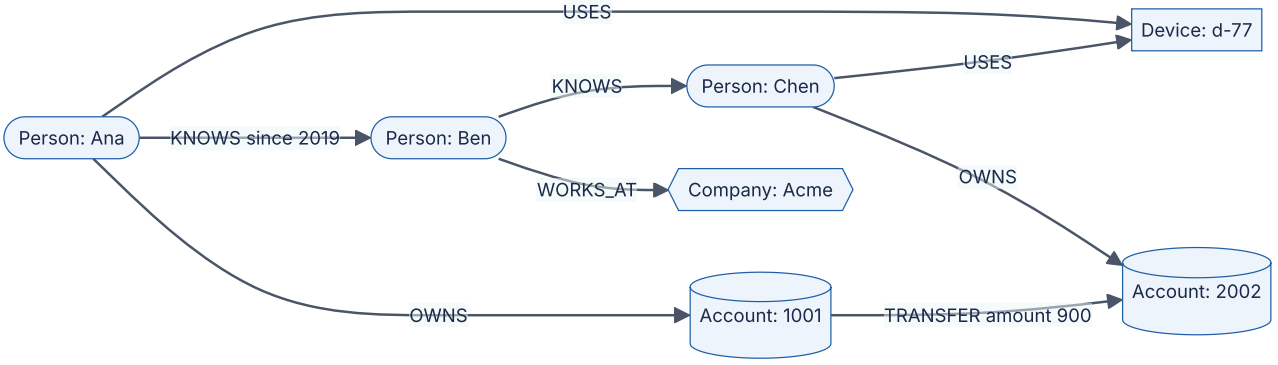
Some domains are mostly relationships: social networks, fraud rings, access-control hierarchies, knowledge graphs. In SQL each hop is a join, and a question of unknown depth needs a recursive common table expression with hard-to-predict cost. Graph databases make traversal the primitive.

Table 12.9 — Property graph versus RDF.

Aspect	Property graph	RDF
Unit of data	Nodes and edges with properties	Triples (subject, predicate, object)
Edge properties	Native	Need reification or RDF-star
Identifiers	Database-internal IDs	Global IRIs, mergeable across datasets
Query languages	Cypher / openCypher, Gremlin, GQL (ISO/IEC 39075:2024)	SPARQL (W3C)
Schema and reasoning	Optional constraints	Ontologies (RDFS, OWL) and inference
Typical systems	Neo4j, Memgraph, TigerGraph, Amazon Neptune	Amazon Neptune, GraphDB, Apache Jena, Virtuoso

[W3C 2014; ISO 2024; Robinson et al. 2015]

Figure 12.7 — A property graph for fraud and social queries.



Reading the diagram. Shapes here distinguish node labels (people, device, accounts, company), not the book’s infrastructure legend. Each arrow is a typed, directed relation-

ship that may carry properties (`since`, `amount`). The fraud signal is a short cycle: Ana and Chen share device `d-77` and money flows between their accounts — a pattern found by walking edges from Ana. This is a logical view with no network boundaries; deployments usually keep the whole graph on one primary with read replicas, because cutting a graph across machines turns hops into network calls.

12.7.2 Index-free adjacency and Cypher

How it works internally

Neo4j's native store illustrates **index-free adjacency** [Robinson et al. 2015; Neo4j Docs]:

1. Nodes, relationships and properties live in stores of fixed-size records, so node *n* is found by arithmetic on its ID, not an index search.
2. A node record points to its first relationship; each relationship points to its two nodes and to the next and previous relationships of both — a linked list per node.
3. A traversal follows these pointers; no global index is consulted per hop. Indexes are used once, to find the starting nodes.
4. Transactions are ACID on the primary; secondaries serve reads and failover.

Two Cypher queries — the fraud pattern and a friends-of-friends recommendation:

```
// Accounts of people within 2 KNOWS hops of Ana who use the same device as Ana
MATCH (a:Person {name: 'Ana'})-[:KNOWS*1..2]-(p:Person)-[:USES]->(d:Device)<-[:USES]-(a)
MATCH (p)-[:OWNS]->(acct:Account)
RETURN DISTINCT p.name, d.id, acct.id;

// People Ana's friends know, whom Ana does not know yet, ranked by mutual friends
MATCH (me:Person {name: 'Ana'})-[:KNOWS]-(f)-[:KNOWS]-(fof:Person)
WHERE fof <> me AND NOT (me)-[:KNOWS]-(fof)
RETURN fof.name, count(DISTINCT f) AS mutual
ORDER BY mutual DESC LIMIT 10;
```

What to notice:

- Patterns are ASCII art, `(node)-[:TYPE]->(node)`; `*1..2` is a variable-length path (a recursive CTE in SQL).
- Each query anchors on one indexed node and expands outward, so cost depends on Ana's neighborhood, not the population.
- `NOT (me)-[:KNOWS]-(fof)` is an anti-join written as a pattern.

12.7.3 When graphs beat joins

FORMULA

Cost of a k-hop traversal

$$E_k = \sum_{i=1}^{k} d^i \qquad T_{\text{graph}} \approx E_k \times t_{\text{ptr}} \qquad T_{\text{join}} \approx E_k \times t_{\text{idx}}$$

Symbol	Meaning	Units
E_k	Edges expanded by a k-hop traversal without pruning	edges
d	Average out-degree (neighbors per node)	edges/node
k	Number of hops	hops
t_{ptr}	Cost of following one adjacency pointer	µs
t_{idx}	Cost of one index lookup on the edge table, growing with $\log N$	µs

WORKED EXAMPLE

Worked example — Friends-of-friends-of-friends.

A social graph has $N = 100$ million people with average degree $d = 150$. Assume, for illustration only, $t_{\text{ptr}} \approx 0.2$ µs (one or two memory accesses) and $t_{\text{idx}} \approx 1$ µs (a cached B-tree descent of 4 levels, §11.8).

1. $k = 2$: $E_2 = 150 + 150^2 = 22,650$ expansions → graph ≈ 4.5 ms; join ≈ 23 ms.
2. $k = 3$: $E_3 = 22,650 + 3,375,000 = 3,397,650$ → graph ≈ 0.68 s; join ≈ 3.4 s.
3. Neither cost contains N except through the logarithm in t_{idx}: doubling the population barely changes either.

Interpretation. Index-free adjacency wins a constant factor per hop (here 5×, illustrative) and avoids large intermediate join results, but the d^k term defeats *both* systems by three hops.

Engineering implication. Graphs beat joins for variable-depth, pattern-shaped queries over bounded neighborhoods (fraud rings, permission inheritance). Prune or precompute deep traversals; for one or two fixed hops, a well-indexed relational database is usually as good.

Real-world industry applications

INDUSTRY EXAMPLE

Facebook TAO. Facebook serves its social graph of objects and typed associations through TAO, a geographically distributed, read-optimized graph cache over sharded MySQL. The API is graph-shaped, the storage relational: a graph *data model* does not require a graph *database* when queries are mostly one hop and scale is planetary. *Evidence: Primary* — [Bronson et al. 2013]

Advantages and disadvantages

- **Advantages:** expressive path queries; cost tied to the touched subgraph; easy to add relationship types; graph algorithm libraries.
- **Disadvantages:** hard to partition; writes and size usually bounded by one primary; scarcer expertise; whole-graph analytics belong in batch processing.

Failure scenarios

Table 12.10 — Graph database failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Query never returns	Unbounded variable-length path explodes as d^k	Slow-query log; memory alarms	Kill query; add bounds	Always bound `*1..k`; set query timeouts
Supernode slows everything	A node with millions of edges (celebrity, default device ID)	Degree distribution metrics	Split by relationship type or time	Model high-degree nodes specially
Primary saturates	Writes cannot scale beyond one node	CPU and commit latency on primary	Scale up; offload reads to replicas	Keep only the relationship-heavy subset in the graph

Common misconceptions

COMMON MISTAKE

"If my data has relationships, I need a graph database." Relational databases were built for relationships. Choose a graph database when questions are about *paths and patterns of unknown depth*.

12.8 Time-series databases

12.8.1 The time-series data model

DEFINITION

Definition — Time-series database (TSDB). A database optimized for sequences of timestamped measurements. Data is organized into **series**, each identified by a metric name plus a set of **labels** (tags) such as `{host="web-3", region="eu"}`, and each series is an append-only list of **samples** `(timestamp, value)`.

First-principles explanation

A ship's log records the position every hour, in order, and nobody edits last Tuesday's entry; old pages are summarized ("averaged 12 knots in March"). Because the next entry is usually "same as before, one hour later", the log can be written very compactly.

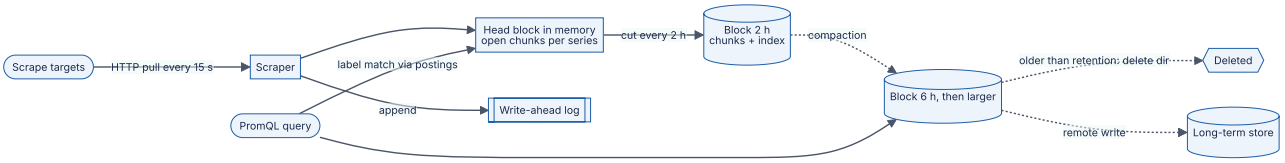
Why does this exist?

10,000 servers exporting 1,000 metrics every 15 s produce about 667,000 samples per second. A general-purpose database spends more on row headers and index entries than on each sample's 16 bytes (8-byte timestamp, 8-byte float). TSDBs exploit four properties: writes append near "now", reads scan recent ranges, adjacent samples are similar, and old data matters only in aggregate.

How it works internally

1. An **inverted index** maps each `label=value` pair to the series that have it (a postings list, §16.3), so `{region="eu", job="api"}` is a list intersection.
2. Each series' samples are buffered in memory and logged to a **write-ahead log**.
3. Buffers are cut into **chunks** (up to 120 samples in Prometheus) compressed Gorilla-style, grouped into time-partitioned **blocks** (2 hours in Prometheus) and compacted into larger blocks.
4. **Retention** deletes whole expired blocks — a directory removal, not row deletes — and **downsampling** replaces old samples with aggregates (min, max, sum, count per 5 minutes or hour).

Figure 12.8 — Prometheus TSDB write and storage path.



Reading the diagram. Prometheus pulls metrics over HTTP and appends each sample to the in-memory head block and the on-disk write-ahead log. Every two hours the head becomes an immutable block of compressed chunks plus an index; compaction (dotted, asynchronous) merges blocks, and retention (15 days by default) deletes old block directories. Queries use the postings index and read both head and blocks. The local disk is a single-node storage boundary; long retention and durability come from remote-write to systems such as Thanos, Cortex or Mimir. Failure points: a crash costs a WAL replay; a lost disk loses local history.

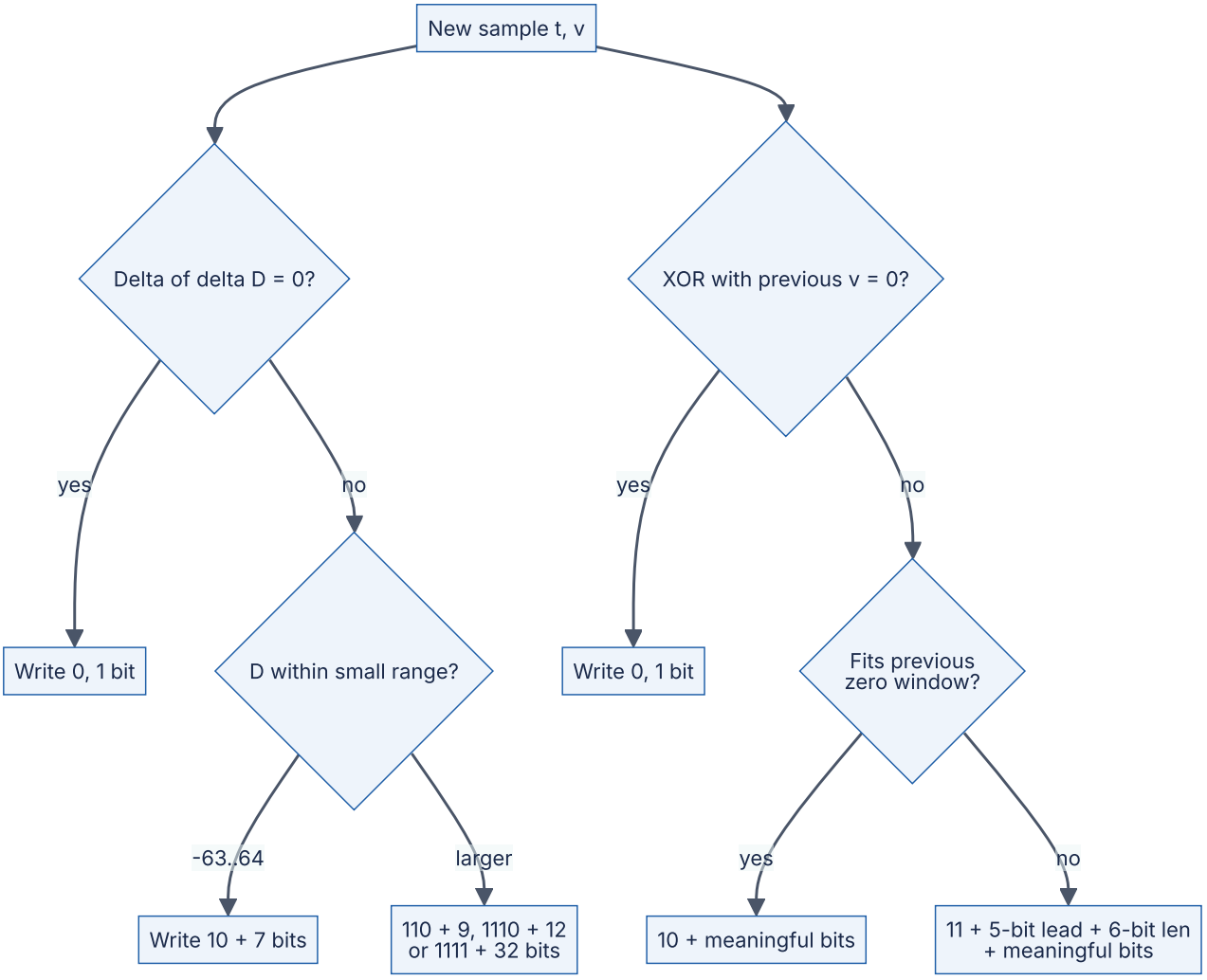
12.8.2 Gorilla compression: delta-of-delta and XOR

Facebook's Gorilla paper [Pelkonen et al. 2015] introduced the compression most modern TSDBs adapt, encoding timestamps and values separately against the previous sample.

Timestamps — delta-of-delta. With a fixed scrape interval, consecutive deltas are equal, so the encoder stores $D = (t_n - t_{n-1}) - (t_{n-1} - t_{n-2})$ with a prefix code: `0` if $D = 0$; `10` + 7 bits for $D \in [-63, 64]$; `110` + 9 bits for $[-255, 256]$; `1110` + 12 bits for $[-2047, 2048]$; else `1111` + 32 bits.

Values — XOR. Consecutive values share sign, exponent and leading mantissa bits, so XOR with the previous value is mostly zeros: store `0` if it is zero; otherwise `1`, then either `0` plus the meaningful bits within the previous leading/trailing-zero window, or `1`, a 5-bit leading-zero count, a 6-bit length and the meaningful bits.

Figure 12.9 — Gorilla's per-sample encoding decisions.



Reading the diagram. Each sample takes two independent paths, timestamp and value, both depending only on the previous sample — so chunks decode sequentially from the start. The common case on each path costs one bit. There are no network or storage boundaries; the failure point is data that defeats the assumptions (jittery timestamps, noisy values). Series encode independently, so compression parallelizes across cores.

This program counts the bits used for three kinds of series:

```
        """Gorilla-style compression (Pelkonen et al., VLDB 2015): delta-of-delta
        timestamps and XOR-encoded float values. Counts bits; returns bits per point."""
        import random
        import struct

        def float_bits(x: float) -> int:
            return struct.unpack(">Q", struct.pack(">d", x))[0]

        def encode(points):
            """points: list of (unix_seconds, float). Returns total bits used."""
            t0, v0 = points[0]
            bits = 64 + 64                          # first timestamp and value stored
raw
            prev_t, prev_delta, prev_v = t0, 0, float_bits(v0)
            prev_lead, prev_trail = None, None
            for t, v in points[1:]:
                delta = t - prev_t
                dod = delta - prev_delta            # delta of delta
                if dod == 0:
                    bits += 1                                   # '0'
                elif -63 <= dod <= 64:
                    bits += 2 + 7                               # '10' + 7 bits
                elif -255 <= dod <= 256:
                    bits += 3 + 9                               # '110' + 9 bits
                elif -2047 <= dod <= 2048:
                    bits += 4 + 12                              # '1110' + 12 bits
                else:
                    bits += 4 + 32                              # '1111' + 32 bits
                prev_t, prev_delta = t, delta

                x = float_bits(v) ^ prev_v
                if x == 0:
                    bits += 1                                   # '0': value repeated
                else:
                    lead = min(64 - x.bit_length(), 31)         # 5-bit field caps at 31
                    trail = (x & -x).bit_length() - 1
                    if prev_lead is not None and lead >= prev_lead and trail >=
prev_trail:
                        bits += 2 + (64 - prev_lead - prev_trail)   # '10' + reuse win-
dow
                    else:
                        bits += 2 + 5 + 6 + (64 - lead - trail)     # '11' + new window
                        prev_lead, prev_trail = lead, trail
                prev_v = float_bits(v)
            return bits

        random.seed(7)
        start = 1_793_318_400                       # 2026-10-30 00:00:00 UTC
        series = {
            "constant gauge":   [(start + 15 * i, 42.0) for i in range(120)],
            "counter (+1/scr)": [(start + 15 * i, float(1000 + i)) for i in range(120)],
            "noisy CPU %":      [(start + 15 * i + random.choice([0, 0, 0, 1]),
                                  round(random.uniform(20, 80), 1)) for i in
range(120)],
        }
        for name, pts in series.items():
```

```
            b = encode(pts)
            print(f"{name:18s} {b / len(pts) / 8:5.2f} bytes/point  ({128 / (b /
len(pts)):4.1f}x vs 16 B)")
```

Output:

```
constant gauge      0.39 bytes/point  (41.1x vs 16 B)
counter (+1/scr)    1.33 bytes/point  (12.0x vs 16 B)
noisy CPU %         7.90 bytes/point  ( 2.0x vs 16 B)
```

What to notice:

- A 120-sample chunk of a constant gauge costs 2 bits per sample once the interval is established (the second sample pays 9 bits to record the first delta); the 128-bit uncompressed header dominates, which is why chunks should not be too short.
- The steadily increasing counter lands at 1.33 bytes per point — close to the 1.37 bytes per point the Gorilla paper reports for Facebook's production data, a 12× reduction from 16 bytes [Pelkonen et al. 2015].
- Random one-decimal values compress only 2× because their mantissas differ in almost every bit; storing only the precision you need is a strong lever.
- The real format stores the first timestamp as a 14-bit offset from a block header.

12.8.3 Retention and downsampling

Prometheus documents a simple disk-sizing rule: needed disk ≈ retention time × ingested samples per second × bytes per sample, with 1–2 bytes per sample typical [Prometheus Docs]. Adding downsampled tiers generalizes it.

FORMULA

Tiered time-series storage

$$S = N_{\text{series}} \times \sum_{i} \frac{R_i}{\Delta_i} \times b_i$$

Symbol	Meaning	Units
S	Total storage	bytes
N_{series}	Number of active series	series
R_i	Retention period of tier i	seconds
Δ_i	Sample (or rollup) interval of tier i	seconds
b_i	Bytes per stored point in tier i after compression	bytes/point

WORKED EXAMPLE

Worked example — Metrics for a 1-million-series platform.

1 million active series scraped every 15 s ($66,667$ samples/s); raw points compress to 1.5 B. Rollups store five aggregates (min, max, sum, count, last) at about 2 B each, so 10 B per rollup point.

1. Raw for 15 days: $R/\Delta = 1,296,000/15 = 86,400$ points per series; $10^6 \times 86,400 \times 1.5 = 1.296 \times 10^{11}$ B $\approx$ **130 GB**.
2. Raw for one year instead: $31,536,000/15 = 2,102,400$ points; $10^6 \times 2,102,400 \times 1.5 \approx$ **3.15 TB**.
3. One-hour rollups for one year: $8,760$ points; $10^6 \times 8,760 \times 10 = 87.6$ GB.
4. Tiered total: $130 + 87.6 \approx$ **218 GB**, about 14× less than a year of raw data. Without compression (16 B/point) the 15-day raw tier alone would be 1.38 TB.

Interpretation. Compression buys about 10×; downsampling another order of magnitude, at the cost of detail older than 15 days.

Engineering implication. You cannot compute a correct p99 from averaged rollups, so keep histograms or sketches if old percentiles matter. Series count, not sample rate, is usually the first limit (Table 12.12).

12.8.4 Prometheus, InfluxDB and TimescaleDB

Table 12.11 — Three time-series databases compared (as of 2026).

Aspect	Prometheus TSDB	InfluxDB	TimescaleDB
What it is	Pull-based monitoring system with embedded TSDB	Purpose-built TSDB with push ingestion	PostgreSQL extension
Data model	Metric + labels → float samples	Measurement + tags + multiple typed fields	SQL tables turned into **hypertables**
Storage	2-hour blocks, Gorilla-style chunks, postings index	Version 3 rewritten on Apache Arrow, DataFusion and Parquet	Time-partitioned chunks (default 7 days); columnar compression of old chunks
Query	PromQL	SQL and InfluxQL (v3)	Full PostgreSQL SQL, joins with relational tables
Downsampling	Recording rules; long-term stores	Downsampling tasks or queries	Continuous aggregates; retention policies drop chunks
Choose when	Operational metrics and alerting	High-ingest metrics or IoT with tags and fields	You need SQL, joins and PostgreSQL tooling

[Prometheus Docs; InfluxData Docs; Timescale Docs]

Real-world industry applications

INDUSTRY EXAMPLE

Facebook Gorilla. Facebook's HBase-backed monitoring store was too slow for queries over recent data. Gorilla keeps the latest 26 hours in memory at 1.37 bytes per point, writes through to HBase, and reported 73× lower query latency and 14× higher throughput. *Evidence: Primary* — [Pelkonen et al. 2015]

Failure scenarios

Table 12.12 — Time-series database failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Memory exhaustion, ingestion stalls	**Cardinality explosion**: a label with unbounded values (user ID, request ID) creates millions of series	Active-series count jumps; head memory growth	Drop the label via relabeling; restart	Review labels; per-target series limits
Out-of-order samples rejected	Clock skew or delayed batches outside the accepted window	Ingestion error counters	Backfill as blocks	Synchronized clocks (§24.9); bounded delays
Wrong long-term percentiles	Averaged rollups cannot reproduce percentiles	Mismatch with raw data	Recompute from raw if still retained	Store histograms or sketches in rollups
Local history lost	Single-node TSDB disk failure	Gaps in dashboards	Restore from remote store	Remote-write or replicated pairs

Common misconceptions

COMMON MISTAKE

"A TSDB is just a table with a timestamp column." At small volume, yes. At scale the differences are physical: 1–2 bytes per sample, retention as file deletion, and an inverted label index. TimescaleDB keeps SQL but changes the storage layout underneath.

12.9 Search engines as databases

DEFINITION

Definition — Search engine (as a data store). A system such as Elasticsearch, OpenSearch or Apache Solr that stores JSON documents and builds an **inverted index** — a map from each term to the list of documents containing it — so it can answer full-text, relevance-ranked, faceted and fuzzy queries.

A relational `LIKE '%running shoes%'` scans every row and cannot rank; an inverted index intersects two postings lists and scores matches with BM25, and also serves faceted counts and log analytics (Chapter 16). As a *database* it has limits: indexing is **near-real-time** (Elasticsearch refreshes every 1 second by default), there are no multi-document transactions, and mapping changes often require reindexing [Elastic Docs]. The standard architecture keeps the system of record in a transactional database and feeds the index through change data capture (§32.6).

RECOMMENDATION

Do not make the search index your only copy. Treat it as a derived, rebuildable view. If you cannot regenerate it from a source of truth in hours, you have created a primary database without the durability features of one.

12.10 Vector databases

DEFINITION

Definition — Vector database. A data store that indexes high-dimensional numeric vectors (**embeddings** produced by machine-learning models) and answers **approximate nearest neighbor (ANN) search** queries: "the k stored vectors most similar to this query vector", by cosine similarity, dot product or Euclidean distance.

Embeddings place similar items — a beach photo and the text "sunny coast" — near each other in hundreds of dimensions. Exact search over 100 million 768-dimension vectors costs about 7.7×10^{10} multiply-adds per query; ANN indexes such as **HNSW** (a layered proximity graph [Malkov & Yashunin 2020]) and **IVF** (search only nearby clusters) trade a little recall for orders of magnitude less work. As of 2026 you meet vector search as dedicated databases (Milvus, Pinecone, Qdrant, Weaviate), as extensions (pgvector, MongoDB Atlas, Cassandra 5.0, Redis, Elasticsearch) and as libraries (FAISS). Treat it as a specialized secondary index, fed from a system of record and rebuilt when the embedding model changes; Chapter 74 covers the design in depth.

12.11 NewSQL

DEFINITION

Definition — NewSQL (distributed SQL). Relational databases that keep SQL and ACID transactions with serializable or strong isolation while scaling horizontally across nodes and regions, by partitioning data into ranges replicated with a consensus protocol. The term was coined by analyst Matthew Aslett in 2011 [Pavlo & Aslett 2016].

NewSQL scales out *without* giving up transactions and joins: split tables into key ranges, replicate each with Paxos or Raft (§26.6, §26.7), run cross-range transactions with a two-phase-commit variant (§28.4), and order transactions with timestamps.

Table 12.13 — Three distributed SQL systems (deep treatment in Chapter 28).

System	Origin	Replication	Clock and ordering	Notable feature
Spanner	Google, OSDI 2012	Paxos per split	TrueTime (GPS and atomic clocks) with commit wait	External consistency across the globe [Corbett et al. 2012]
CockroachDB	Cockroach Labs, SIGMOD 2020 paper	Raft per range	Hybrid logical clocks, bounded uncertainty	PostgreSQL wire compatibility; serializable by default [Taft et al. 2020]
TiDB	PingCAP, VLDB 2020 paper	Raft in TiKV	Timestamp oracle in the Placement Driver	MySQL compatibility; columnar TiFlash replicas for analytics [Huang et al. 2020]

The price is latency: a write must reach a majority of replicas (milliseconds within a region, tens of milliseconds across regions), cross-range transactions add coordination, and Spanner's commit wait pauses for clock uncertainty. Choose NewSQL when you have outgrown one PostgreSQL or MySQL primary *and* need multi-row transactions (§28.8, §28.10).

12.12 Selection framework and SQL versus NoSQL comparison

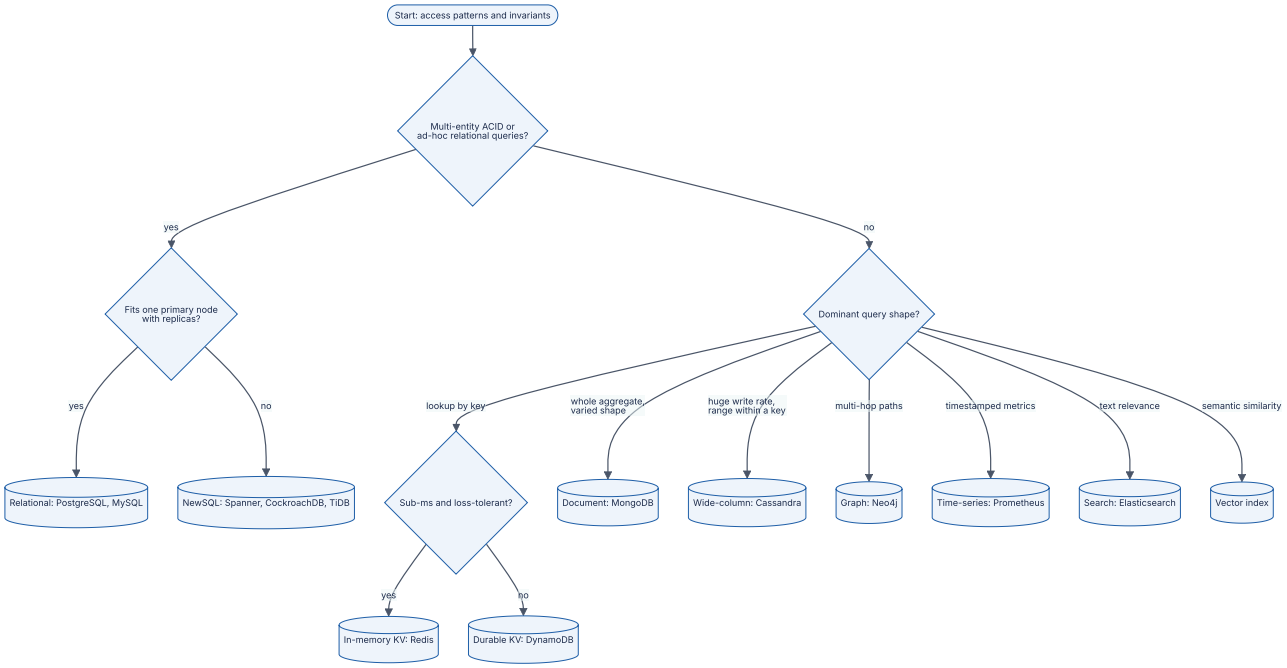
12.12.1 A decision procedure

Choosing a database is choosing which questions will be cheap and which impossible:

1. **List access patterns** with rates and latency targets ("get order by ID, 10,000/s, p99 < 20 ms").

2. **List invariants** and how many entities each spans (“stock never negative”: one; “debits equal credits”: two).
3. **Estimate size and growth** (§2.11).
4. **Pick the simplest store that serves the critical patterns**, adding specialized stores only for patterns it cannot serve.

Figure 12.10 — Database selection flowchart.



Reading the diagram. Diamonds ask about requirements, not technologies; cylinders are families with one example each. The first question routes toward relational storage: if invariants span entities or queries are not yet known, a relational database with replicas (§14.3) is the default, and NewSQL applies only when one primary cannot carry the load. Otherwise the dominant query shape picks the family. There are no protocols or failure paths; in practice one system serves several patterns and specialized stores are added as *derived* views (§12.13). Revisit the chart when access patterns change.

> RECOMMENDATION
>
> **Default to PostgreSQL until a requirement rules it out.** One PostgreSQL primary with replicas handles tens of thousands of transactions per second and terabytes of data, with `jsonb`, full-text search, TimescaleDB and pgvector covering many specialized needs. Move a workload when you can name and measure the requirement it fails.

12.12.2 Comparison table

Table 12.14 — Relational and NoSQL families compared.

Row	Relational (SQL)	Key-value	Document	Wide-column	Graph	Time-series
What it is	Tables, rows, joins, ACID	Opaque values by key	Nested JSON-like documents	Sorted sparse rows in partitions	Nodes and typed edges	Labeled series of samples
How it works	B-tree pages, WAL, cost-based planner	Hash partitioning, replicas	B-tree per collection, shard key	LSM storage, hash ring, clustering order	Linked records, pointer chasing	Compressed chunks, time blocks, label index
Advantages	Ad-hoc queries, integrity, transactions	Flat latency at any scale	Matches objects; flexible schema	Massive writes; multi-region	Multi-hop queries	1–2 B/sample; cheap retention
Disadvantages	Hard to scale writes past one node	Only designed queries	Cross-document joins weak	One table per query; tombstones	Hard to partition	Narrow query model; cardinality limits
Performance	~ms point reads; joins vary	~ms durable, sub-ms in memory	~ms by indexed field	~ms partition slices; fast writes	Cost tied to neighborhood	Millions of samples/s per node
Operational complexity	Low single-node; high when sharded	Low if managed	Medium	High (repair, compaction)	Medium	Low single-node; high clustered
Cost	Scale-up hardware, licenses	Pay per request or RAM	Moderate	Many commodity nodes	Large-memory nodes	Cheap disk per sample
Use cases	Orders, payments, inventory	Sessions, carts, counters	Catalogs, profiles, content	Messages, IoT, event logs	Fraud, social, permissions	Metrics, IoT telemetry
Choose when	Invariants span entities	Access is by key at scale	Aggregates read whole	Write volume dominates	Questions are paths	Data is timestamped samples

The boundaries are blurring — PostgreSQL stores documents, MongoDB runs transactions, DynamoDB speaks PartiQL (a SQL-compatible language) — so compare specific products against specific requirements.

INTERVIEW TIP

Interview tip — justify, don't recite. "We'll use Cassandra because it scales" earns little credit. "Messages are written 50,000/s and always read as the latest page of one channel, so I'll partition by `(channel_id, bucket)` with message ID as clustering column; there are no cross-channel transactions, so we don't need a relational store here" shows the access-pattern reasoning interviewers look for.

12.13 Polyglot persistence

12.13.1 One service, several stores

DEFINITION

Definition — Polyglot persistence. Using several different data-storage technologies within one system, each chosen for the access pattern it serves best, rather than forcing all data into one database [Fowler 2011; Sadalage & Fowler 2012].

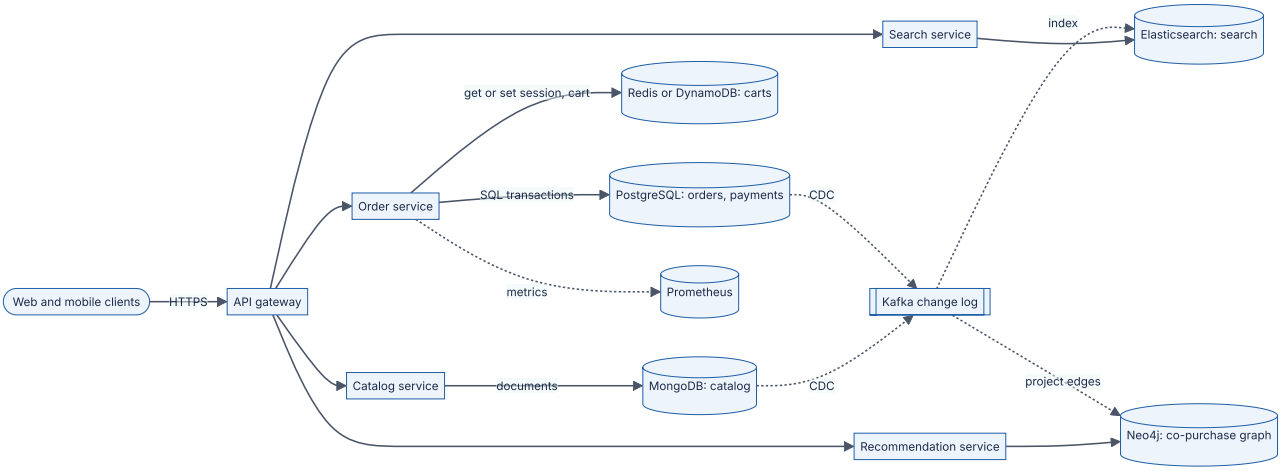
First-principles explanation

A household keeps money in a bank, documents in a cabinet and food in a fridge; nobody stores food in the cabinet for simplicity's sake. But every extra place needs upkeep, and each must be packed and checked when you move.

How it works internally

1. One store is the **system of record** for each piece of data — usually a relational or durable key-value database that enforces the invariants.
2. Other stores hold **derived views**: a search index, a cache, a graph projection, an analytics copy.
3. Changes flow from the system of record to derived stores **asynchronously** through change data capture or a transactional outbox (§32.5, §32.6) and an event log such as Kafka (Chapter 31) — never through application "dual writes" that can half-fail.
4. Each derived store can be rebuilt by replaying the log or re-exporting the source.

Figure 12.11 — Polyglot persistence in an e-commerce platform.



Reading the diagram. Clients reach services via an API gateway over HTTPS. Solid arrows are synchronous: the order service owns orders and payments in PostgreSQL and carts in a key-value store; the catalog service owns product documents in MongoDB.

Dotted arrows are asynchronous: CDC streams committed changes into Kafka, whose consumers build the Elasticsearch index and the Neo4j co-purchase graph; services emit metrics to Prometheus. Each database is a separate failure domain with its own backups. If the CDC pipeline stalls, search and recommendations go stale but orders keep working — the intended degradation. Each store scales independently, and derived stores can be rebuilt from Kafka.

12.13.2 The operational costs

Table 12.15 — What each additional database costs you.

Cost	What it involves
Expertise	On-call engineers who know its failure modes and tuning
Operations	Patching, upgrades, capacity planning, dashboards, alerts
Backup and recovery	Separate backups, restore drills and recovery objectives
Consistency	Lagging derived stores; reconciliation jobs
Security and compliance	Another access model and audit trail; deletions must reach every copy
Money	Baseline cluster cost even when idle; data stored several times

INDUSTRY EXAMPLE

Database-per-service. Microservices commonly give each service its own database (§34.11), so polyglot persistence follows naturally — and cross-service queries and consistency become pipeline problems. *Evidence: Illustrative* — a widely described architectural pattern; not a claim about any specific company.

WARNING / TRADE-OFF

Dual writes. Writing to PostgreSQL and then Elasticsearch in one handler diverges silently if the process crashes between them. Derive the second write from the committed first via an outbox or CDC.

RECOMMENDATION

Budget the number of stores. Add a technology only when its requirement justifies a permanent operational cost; many organizations keep an approved list and review exceptions.

Common misconceptions

COMMON MISTAKE

“Polyglot persistence means every service should use a different database.” The goal is fit, not variety; services with similar patterns should share technology, tooling and runbooks.

12.14 Summary and revision notes

SUMMARY

- NoSQL arose from scale-out, schema-flexibility and availability pressures plus “one size does not fit all”; Bigtable, Dynamo, PNUTS and Cassandra defined the designs.
- Key-value, document and wide-column stores are **aggregate-oriented**: data read together is stored together under one key.
- **DynamoDB:** partition key (hashed) + sort key (ordered item collection); GSIs are asynchronous, eventually consistent copies; LSIs share the partition key. Design keys from access patterns; use sparse and overloaded indexes. WCU = 1 KB writes/s, RCU = 4 KB strong reads/s (½ for eventual); per-partition limits 3,000 RCU / 1,000 WCU; on-demand versus provisioned break-even ≈ 29 % utilization (as of 2026 prices).
- **MongoDB:** embed what is owned, bounded and read together; reference what is shared or unbounded; order compound indexes Equality–Sort–Range; multi-document transactions since 4.0; use `w: "majority"`.
- **Bigtable/Cassandra:** sorted sparse rows; CQL `PRIMARY KEY ((partition), clustering)`; one table per query; bucket partitions and size them with $N_v = N_r(N_c - N_{pk} - N_s) + N_s$; beware tombstones and last-write-wins.
- **Graphs:** property graph versus RDF; index-free adjacency makes traversal cost depend on the neighborhood, but d^k growth still dominates deep queries.
- **Time-series:** labeled series, inverted label index, delta-of-delta and XOR compression (~1.37 B/sample in Gorilla), time blocks, retention by deletion, downsampling tiers; cardinality is the first limit.
- Search engines and vector databases are derived indexes fed by CDC; NewSQL adds scale-out SQL transactions at a latency cost (Chapter 28).
- Default to relational; add a specialized store for a named, measured requirement; feed derived stores asynchronously; budget each store’s operational cost.

12.15 Knowledge checkpoint

1. Name the three pressures that produced NoSQL and one paper associated with each design response.
2. Why must a DynamoDB Query name exactly one partition key?
3. Give two differences between an LSI and a GSI.
4. A Query returns three items of 1.2 KB each, eventually consistent. How many RCUs does it consume? How many would three GetItem calls consume?
5. What makes a GSI sparse, and what is it useful for?
6. State the ESR rule and explain why sort fields precede range fields.
7. In `PRIMARY KEY ((sensor_id, day), ts)`, which columns determine placement and which determine order?
8. How many cells does a Cassandra partition hold with 10,000 rows, 5 columns, 1 partition-key column and 1 static column?
9. Why can deleted data reappear in Cassandra?
10. What is index-free adjacency, and what does it not fix?
11. Which Gorilla encoding costs one bit for a timestamp and one bit for a value?
12. Why should a search index not be the only copy of data?

Answers

1. Scale-out (Bigtable, Dynamo: partition by key), schema flexibility (Bigtable's sparse columns; document stores), availability (Dynamo: sloppy quorums, always-writable).
2. Only the partition key is hashed to locate data; without it the request would have to visit every partition.
3. An LSI shares the table's partition key, must be created with the table, supports strong reads and limits item collections to 10 GB; a GSI has any key, can be added later, is eventually consistent and has its own throughput.
4. Query sums sizes: 3.6 KB → 1 unit × 0.5 = 0.5 RCU. GetItems round each item: 3 × 1 × 0.5 = 1.5 RCU.
5. Only items carrying the index key attribute appear in it; it is a cheap, self-maintaining list such as "orders awaiting shipment".
6. Equality, Sort, Range. Once a range condition is applied, matching entries are no longer in sort-field order, so the sort would need an in-memory step.
7. `(sensor_id, day)` is hashed for placement; `ts` orders rows within the partition.
8. $10,000 \times (5 - 1 - 1) + 1 = 30,001$ cells.
9. A replica that was down longer than `gc_grace_seconds` missed the tombstone; after the tombstone is purged elsewhere, the old value on that replica wins on repair.
10. Each node points directly to its relationships, so a hop is a pointer chase rather than an index lookup. It does not fix exponential growth of neighborhoods (d^k).
11. Delta-of-delta equal to 0 and XOR with the previous value equal to 0 (same interval, repeated value).

12. It is near-real-time, non-transactional and often needs reindexing after mapping changes; it must be rebuildable from a system of record.

12.16 Interview questions

Basic

Q1. When would you choose a NoSQL database over a relational one?

Answer. When a named requirement exceeds what a relational primary with replicas can deliver or fits another model better: write volume or data size beyond one node with simple per-key access (key-value, wide-column); aggregates of varying shape read whole (document); multi-hop path queries (graph); high-rate timestamped samples (time-series). If invariants span entities or queries are still unknown, start relational.

Q2. What is the difference between a partition key and a sort key?

Answer. The partition key is hashed to choose the partition, so it should have high cardinality and even load. The sort key orders items within a partition, enabling range, prefix and newest-first queries over one item collection.

Intermediate

Q3. Design a DynamoDB schema for a chat app: send a message, list a conversation's recent messages, list a user's conversations by latest activity.

Answer. Messages: `PK=CONV#id, SK=MSG#<time-ordered ID>`, read newest first with a descending Query. Membership: `PK=USER#u, SK=CONV#id` carrying `lastActivity`, plus a GSI keyed `USER#u` / `<lastActivity>#CONV#id` for recency order (update or batch on each message). A conversation above about 1,000 writes/s needs write sharding; use TTL for retention.

Q4. A Cassandra cluster shows read timeouts on a few nodes only. Diagnose.

Answer. Suspect a large or hot partition or tombstones: check partition-size histograms, tombstone warnings and key-level traffic. Typical causes are partition keys without a time bucket and queue-like deletes. Fix the model (bucket, avoid deletes, TTL with time-window compaction), migrate, and repair within `gc_grace_seconds`.

Advanced

Q5. Design storage for a social network: profiles, posts, follow graph, feed, search and metrics.

Answer. Profiles and posts in a sharded relational or document store keyed by user (system of record). The follow graph as adjacency lists in a key-value or wide-column store, since one-hop queries dominate (the TAO pattern), with a batch graph job for multi-hop recommendations. Feeds as precomputed lists in wide-column or Redis; search in Elasticsearch via CDC; metrics in Prometheus. Discuss read-your-writes for own posts, fan-out cost and the cost of operating many stores.

Q6. Your team proposes moving from PostgreSQL to MongoDB "for scale". How do you evaluate it?

Answer. Demand the measured bottleneck (write rate, size, latency) and check whether indexes, replicas, partitioning, `jsonb` or a cache fix it. MongoDB fits if the workload is aggregate-shaped and a shard key matches access patterns; otherwise you trade joins and constraints for application code. Price the migration (backfill, CDC cut-over), skills, and invariants needing transactions.

12.17 Practical exercises

Exercise 12.1 — Key design. Extend the toy store of §12.4.4 with AP6, "all orders containing a given SKU in the last 30 days". Propose a GSI, add it, and compute its WCU cost at 1,000 orders/s.

Exercise 12.2 — Capacity. Recompute the §12.4.5 example with strongly consistent reads and transactional header writes. At what average utilization does provisioned capacity become cheaper?

Exercise 12.3 — Cassandra modeling. In a Cassandra 5 container, create the §12.6.3 tables, load one day for 100 sensors, and compare `nodetool tablestats` partition sizes with S_t.

Exercise 12.4 — Compression. Modify the Gorilla program to round the CPU values to integers and to remove timestamp jitter. Report the bytes per point for each change.

12.18 Design challenges

1. **Ride-sharing trip store.** 2 million trips per day, GPS points every 4 s per active trip, queries by trip, by driver over 90 days, and live positions. Choose stores, keys and buckets; size storage for one year.
2. **Fraud detection platform.** Card transactions at 20,000/s must be scored in 50 ms using shared-device and shared-address rings up to 3 hops. Decide what lives in a graph, what is precomputed, and how the graph is fed.
3. **Migrating a monolith's database.** A 4 TB PostgreSQL database serves orders, catalog, search and analytics. Propose a polyglot target, the CDC pipeline, the cut-

over sequence and the operational budget.

12.19 References

1. DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., Vogels, W. *Dynamo: Amazon's Highly Available Key-value Store*. SOSP, 2007. https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf
2. Chang, F., Dean, J., Ghemawat, S., Hsieh, W. C., Wallach, D. A., Burrows, M., Chandra, T., Fikes, A., Gruber, R. E. *Bigtable: A Distributed Storage System for Structured Data*. OSDI, 2006.
3. Lakshman, A., Malik, P. *Cassandra: A Decentralized Structured Storage System*. ACM SIGOPS Operating Systems Review 44(2), 2010.
4. Pelkonen, T., Franklin, S., Teller, J., Cavallaro, P., Huang, Q., Meza, J., Veeraraghavan, K. *Gorilla: A Fast, Scalable, In-Memory Time Series Database*. PVLDB 8(12), 2015. https://www.vldb.org/pvldb/vol8/p1816-teller.pdf
5. Elhemali, M., et al. *Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service*. USENIX ATC, 2022. https://www.usenix.org/conference/atc22/presentation/elhemali
6. Cooper, B. F., et al. *PNUTS: Yahoo!'s Hosted Data Serving Platform*. PVLDB 1(2), 2008.
7. Kleppmann, M. *Designing Data-Intensive Applications*, ch. 2. O'Reilly, 2017.
8. Sadalage, P. J., Fowler, M. *NoSQL Distilled*. Addison-Wesley, 2012.
9. Stonebraker, M., Çetintemel, U. *"One Size Fits All": An Idea Whose Time Has Come and Gone*. ICDE, 2005.
10. Stonebraker, M., et al. *The End of an Architectural Era (It's Time for a Complete Rewrite)*. VLDB, 2007.
11. Brewer, E. *Towards Robust Distributed Systems*. PODC keynote, 2000.
12. Gilbert, S., Lynch, N. *Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services*. ACM SIGACT News 33(2), 2002.
13. Amazon Web Services. *Amazon DynamoDB Developer Guide*. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/
14. Amazon Web Services. *Amazon DynamoDB pricing*. https://aws.amazon.com/dynamodb/pricing/
15. DeBrie, A. *The DynamoDB Book*. Self-published, 2020.
16. MongoDB. *MongoDB Manual* (data modeling, indexes, transactions, limits). https://www.mongodb.com/docs/manual/
17. Apache Cassandra. *Cassandra Documentation: Data Modeling*. https://cassandra.apache.org/doc/latest/cassandra/data_modeling/
18. Chebotko, A., Kashlev, A., Lu, S. *A Big Data Modeling Methodology for Apache Cassandra*. IEEE BigData Congress, 2015.

19. Borthakur, D., et al. *Apache Hadoop Goes Realtime at Facebook*. SIGMOD, 2011.
20. Discord. *How Discord Stores Billions of Messages*. Discord blog, 2017. https://discord.com/blog/how-discord-stores-billions-of-messages
21. Discord. *How Discord Stores Trillions of Messages*. Discord blog, 2023. https://discord.com/blog/how-discord-stores-trillions-of-messages
22. Neo4j. *Neo4j Documentation* (Cypher Manual, Operations Manual). https://neo4j.com/docs/
23. Robinson, I., Webber, J., Eifrem, E. *Graph Databases*, 2nd ed. O'Reilly, 2015.
24. ISO/IEC 39075:2024. *Information technology — Database languages — GQL*. ISO, 2024.
25. W3C. *RDF 1.1 Concepts and Abstract Syntax*. W3C Recommendation, 2014. https://www.w3.org/TR/rdf11-concepts/
26. Bronson, N., et al. *TAO: Facebook's Distributed Data Store for the Social Graph*. USENIX ATC, 2013.
27. Prometheus. *Storage*. Prometheus documentation. https://prometheus.io/docs/prometheus/latest/storage/
28. InfluxData. *InfluxDB documentation*. https://docs.influxdata.com/
29. Timescale. *TimescaleDB documentation*. https://docs.timescale.com/
30. Elastic. *Elasticsearch Guide* (near real-time search, refresh). https://www.elastic.co/guide/
31. Malkov, Y. A., Yashunin, D. A. *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs*. IEEE TPAMI 42(4), 2020.
32. Corbett, J. C., et al. *Spanner: Google's Globally-Distributed Database*. OSDI, 2012.
33. Taft, R., et al. *CockroachDB: The Resilient Geo-Distributed SQL Database*. SIGMOD, 2020.
34. Huang, D., et al. *TiDB: A Raft-based HTAP Database*. PVLDB 13(12), 2020.
35. Pavlo, A., Aslett, M. *What's Really New with NewSQL?* SIGMOD Record 45(2), 2016.
36. Fowler, M. *PolyglotPersistence*. martinfowler.com, 2011. https://martinfowler.com/bliki/PolyglotPersistence.html
37. Redis. *Redis documentation*. https://redis.io/docs/latest/

12.20 Open verification items

- DynamoDB prices (on-demand $0.625/M write and $0.125/M read request units after the November 2024 reduction; provisioned $0.00065/WCU-h, $0.00013/RCU-h, us-east-1) confirmed only via third-party pages; aws.amazon.com was unreachable.
- DynamoDB limits (400 KB items, per-partition 3,000 RCU/1,000 WCU, 20 GSIs, 5 LSIs, 10 GB item collections) and the 89.2 M requests/s figure (ATC 2022).
- Cassandra "~100 MB per partition" is community guidance, not official docs; tombstone thresholds and `gc_grace_seconds` defaults per version.
- Redis 8 AGPLv3 option and InfluxDB 3 architecture (as of 2026).

- Pointer-chase and index-lookup costs in §12.7.3 are illustrative assumptions.

Chapter 13 — Storage Engine Internals

> **Level:** 3 · **Track:** A (Traditional) · **Estimated reading time:** 3.5 h

When you call `INSERT` on PostgreSQL, `Put` on RocksDB or `save()` on an ORM, a piece of software called the **storage engine** decides where those bytes live on disk, how they are found again, and what happens if the power fails halfway through. Two databases with the same SQL dialect can differ tenfold in write volume, disk space or tail latency purely because of their storage engines. This chapter opens the box: pages, buffer pools, write-ahead logs, B-trees and log-structured merge (LSM) trees, and the numbers that let you choose an engine for a workload instead of by reputation. For book-length treatments, see [Petrov 2019] and chapter 3 of [Kleppmann 2017].

13.1 Learning objectives

After this chapter you should be able to:

- Draw a slotted page and explain why records get stable identifiers.
- Explain a buffer pool's page table, pin counts and dirty flags; compare LRU, clock and LRU-K; and justify `O_DIRECT`.
- Describe WAL, LSNs, fuzzy checkpoints and the three ARIES passes, and estimate recovery time.
- Trace a B-tree page split, explain latch crabbing, and contrast in-place with copy-on-write B-trees.
- Build and run a toy LSM tree (memtable, SSTables, WAL replay, compaction), starting from Bitcask.
- Calculate write, read and space amplification for size-tiered, leveled and hybrid compaction, and apply the RUM conjecture.
- Explain torn writes, checksums, the doublewrite buffer and full-page writes.
- Explain why columnar layout, encoding and vectorized execution make analytical scans 10–100× cheaper.
- Choose between InnoDB-, PostgreSQL-heap- and RocksDB-style engines for a workload, with numbers.

13.2 Prerequisites

- Storage devices, IOPS and SSD write amplification — §5.6 (engine designs respond to device behavior).
- Page cache, `mmap`, copy-on-write — §6.6.5–§6.6.6 (buffer pools replace or sit beside them).
- `fsync`, journaling and group commit — §6.14 (the group-commit formula in §6.14.3 is not repeated).
- B+ trees, transactions and MVCC — §11.8, §11.11, §11.14 (here: how a B-tree is stored and made crash-safe).
- Logarithms — §4.4. Units: **KiB/MiB** for page and block sizes; **KB/MB/GB/TB** (powers of ten) for data volumes.

13.3 Pages and blocks

13.3.1 Why storage engines think in pages

> DEFINITION
>
> **Definition — Page.** The fixed-size unit (typically 4–16 KiB) in which a storage engine reads, writes, caches and logs data. A page on disk is sometimes called a **block**; the engine keeps a copy of a page in memory in a **frame** of its buffer pool.

First-principles explanation

Imagine a library that lends books only by the whole shelf: to read one sentence you carry a shelf. Storage devices behave like that library. An SSD reads flash in pages of 4–16 KiB; an HDD pays milliseconds of seek before transferring anything (§5.6.1), so reading 100 bytes costs almost the same as reading 8 KiB. A database therefore never asks the disk for “a row”; it asks for the page containing it, and organizes tables, indexes and logs in page-sized units.

Why does this exist?

Pages give **I/O efficiency** (one request moves many rows), **a unit of caching and recovery** (the buffer pool caches pages; the log describes changes to pages, §13.5) and **addressability** (a row is named by `(page, slot)`). Larger pages mean fewer I/Os for scans and shallower B-trees, but more bytes written when one small row changes and a larger unit that a crash can tear (§13.12).

Table 13.1 — Default page sizes (as of 2025).

Engine	Default page size	Configurable?
PostgreSQL heap and indexes	8 KiB	Only at compile time
MySQL InnoDB	16 KiB	4–64 KiB, at initialization (`innodb_page_size`)
SQLite	4 KiB	Yes, per database (`PRAGMA page_size`)
RocksDB data block (inside an SSTable)	4 KiB before compression	Yes (`block_size`)
LMDB	Operating-system page size (usually 4 KiB)	No

13.3.2 The slotted page

DEFINITION

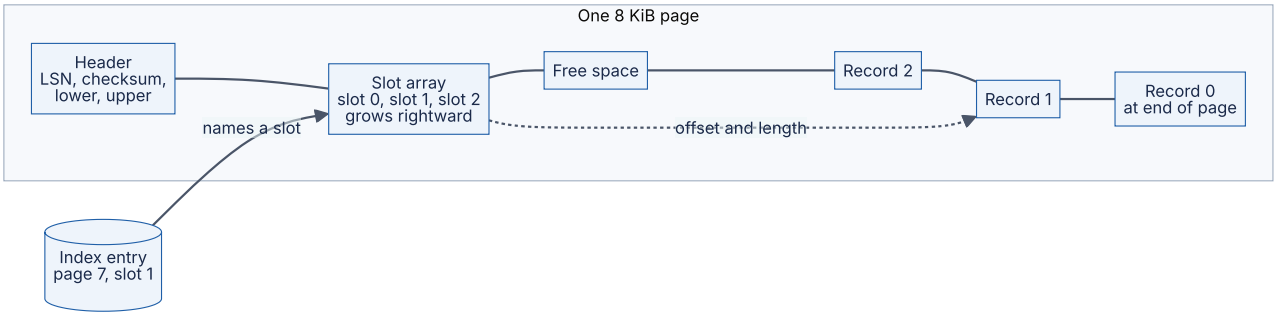
Definition — Slotted page. A page layout in which a **header** is followed by an array of **slots** (also called line pointers or item IDs) that grows from the front, while variable-length **records** (tuples) are packed from the back. Each slot stores the offset and length of one record; free space sits in the middle.

How it works internally

1. **Header:** page LSN (§13.5), checksum (§13.12), slot count and free-space pointers. PostgreSQL's header is 24 bytes, with `pd_lower` and `pd_upper` bounding the free-space hole [PostgreSQL Docs: Database Page Layout].
2. **Insert:** copy the record to the end of free space, then append a slot pointing to it. The record's identifier is `(page, slot)`.
3. **Read:** look up the slot and follow its offset.
4. **Delete:** mark the slot unused; the bytes become a hole.
5. **Grow or compact:** move records within the page and rewrite their slot offsets. Because indexes name *slots*, not byte offsets, these moves are invisible to them — the whole point of the indirection.

Architecture diagram

Figure 13.1 — A slotted page.



Reading the diagram. One contiguous 8 KiB buffer, left to right as on disk. The slot array grows rightward and records grow leftward from the end; the page is full when they meet in the free space. An index entry (a separate storage structure) names `(page 7, slot 1)`; the lookup is synchronous, and the dotted arrow marks the slot-to-record indirection, not an asynchronous path. Failure point: a crash that writes only part of the page leaves header, slots and records inconsistent, hence checksums and torn-write protection (§13.12). Tables scale by adding pages, never by growing one.

Mathematical foundations

FORMULA

Records per slotted page

$$n = \left\lfloor \frac{P - H}{r + s} \right\rfloor$$

Symbol	Meaning	Units
n	Records that fit in one page	records
P	Page size	bytes
H	Page header size	bytes
r	Stored record size including per-record header and alignment padding	bytes
s	Slot (line pointer) size	bytes

WORKED EXAMPLE

Worked example — How big is a 1-billion-row PostgreSQL table?

Each row holds 76 bytes of column data. PostgreSQL adds a 23-byte tuple header, padded to 24 bytes, and aligns the tuple to 8 bytes; each line pointer is 4 bytes; the page header is 24 bytes [PostgreSQL Docs: Database Page Layout].

1. Stored record: $24 + 76 = 100$ bytes, rounded up to a multiple of 8: $r = 104$ bytes.
2. Per record including its slot: $104 + 4 = 108$ bytes.
3. Records per page: $n = \lfloor (8192 - 24)/108 \rfloor = \lfloor 75.6 \rfloor = 75$.
4. Pages: $10^9/75 = 13\,333\,334$ pages; size $= 13\,333\,334 \times 8192 \approx 109.2$ GB.
5. Raw column data: $10^9 \times 76 = 76$ GB. Overhead factor: $109.2/76 \approx 1.44$.

Interpretation. Before any index or dead tuple, the table is 44% larger than its data, because of MVCC tuple headers (§11.14), slots and alignment. **Engineering implication.** Narrow rows pay proportionally more; ordering 8-byte columns first reduces padding, and a deliberate `fillfactor` below 100% trades space for cheaper updates (§13.14.3). This 1.44 is part of the **space amplification** of §13.11.

Code example

The following program implements a 4 KiB slotted page with insert, get, delete and compaction. It is standalone; run it with `python3`.

```
        """A minimal slotted page: header, slot array growing forward, records growing
        backward from the end of the page. Record IDs (slot numbers) never change."""
        import struct

        PAGE_SIZE = 4096
        HEADER = struct.Struct("<HH")      # slot_count, free_end (offset where record
space begins)
        SLOT = struct.Struct("<HH")        # record offset, record length (offset 0 =
deleted slot)


        class SlottedPage:
            def __init__(self):
                self.buf = bytearray(PAGE_SIZE)
                HEADER.pack_into(self.buf, 0, 0, PAGE_SIZE)

            def _header(self):
                return HEADER.unpack_from(self.buf, 0)

            def _slot_pos(self, slot):
                return HEADER.size + slot * SLOT.size

            def free_space(self):
                count, free_end = self._header()
                return free_end - (HEADER.size + count * SLOT.size)

            def insert(self, record: bytes) -> int:
                count, free_end = self._header()
                if len(record) + SLOT.size > self.free_space():
                    self.compact()                                    # reclaim holes,
then retry
                    count, free_end = self._header()
                    if len(record) + SLOT.size > self.free_space():
                        raise ValueError("page full")
                offset = free_end - len(record)
                self.buf[offset:free_end] = record                    # record grows
downward
                SLOT.pack_into(self.buf, self._slot_pos(count), offset, len(record))
                HEADER.pack_into(self.buf, 0, count + 1, offset)      # slot array grows
upward
                return count                                          # slot number =
stable record ID

            def get(self, slot: int) -> bytes:
                offset, length = SLOT.unpack_from(self.buf, self._slot_pos(slot))
                if offset == 0:
                    raise KeyError(f"slot {slot} deleted")
                return bytes(self.buf[offset:offset + length])

            def delete(self, slot: int) -> None:
                SLOT.pack_into(self.buf, self._slot_pos(slot), 0, 0) # leaves a hole un-
til compact()

            def compact(self) -> None:
                """Slide live records to the end of the page; slot numbers stay the
same."""
```

```
                count, _ = self._header()
                live = []
                for s in range(count):
                    off, length = SLOT.unpack_from(self.buf, self._slot_pos(s))
                    if off:
                        live.append((s, bytes(self.buf[off:off + length])))
                free_end = PAGE_SIZE
                for s, rec in live:
                    free_end -= len(rec)
                    self.buf[free_end:free_end + len(rec)] = rec
                    SLOT.pack_into(self.buf, self._slot_pos(s), free_end, len(rec))
                HEADER.pack_into(self.buf, 0, count, free_end)


        if __name__ == "__main__":
            page = SlottedPage()
            ids = [page.insert(b"row-%03d:" % i + b"x" * 90) for i in range(39)]
            print("inserted", len(ids), "records; free bytes:", page.free_space())
            for s in ids[::2]:
                page.delete(s)                                      # punch 20 holes
            print("after deleting 20, free bytes before compaction:", page.free_space())
            new = page.insert(b"a-larger-record:" + b"y" * 400)     # forces compaction
            print("new record got slot", new, "; slot 1 still reads", page.get(1)[:8])
            print("free bytes now:", page.free_space())
```

Output:

```
inserted 39 records; free bytes: 114
after deleting 20, free bytes before compaction: 114
new record got slot 39 ; slot 1 still reads b'row-001:'
free bytes now: 1654
```

What to notice:

- 39 records of 98 bytes plus 4-byte slots fill $4 + 39 \times 102 = 3982$ of 4096 bytes, leaving 114 — the formula with $H = 4$, $r = 98$, $s = 4$.
- Deleting does not raise `free_space()`: holes are unusable until `compact()` runs (PostgreSQL compacts during page pruning and `VACUUM`).
- After compaction slot 1 returns the same record from a different offset; that stability lets indexes store `(page, slot)`.

Alternatives and trade-offs

Fixed-length records need no slots (record i sits at offset $H + i \times r$). Clustered tables such as InnoDB's keep rows inside B-tree leaves, with slots in key order for binary search (§13.14.1). Values too large for a page move out of line (PostgreSQL TOAST above roughly 2 KB; InnoDB overflow pages).

COMMON MISTAKE

“A row is the unit of I/O.” It never is. Updating one 50-byte field dirties a whole 8 or 16 KiB page, which is eventually written in full and, after a checkpoint, may also be copied into the log (§13.12). Reasoning in rows is the commonest cause of under-estimated write volume.

13.4 Buffer pools

13.4.1 What a buffer pool does

DEFINITION

Definition — Buffer pool. A region of memory, managed by the database itself, divided into page-sized **frames** that cache disk pages. A **page table** maps page IDs to frames; each frame carries a **pin count** (how many threads are using it right now), a **dirty flag** (whether it differs from the disk copy) and bookkeeping for the **replacement policy** that chooses which page to evict.

First-principles explanation

Think of a workbench with 1,000 trays beside a warehouse of a billion boxes. You fetch a box into a free tray and note its number on a whiteboard (the page table). While using it you clip the tray (a **pin**) so nobody swaps it out; if you change its contents you add a red dot (**dirty**), meaning it must be shelved before the tray is reused. When the trays are full, you pick an unclipped tray not needed for a while (the **replacement policy**).

Why does this exist?

RAM is about 1,000 times faster than NVMe for a random access (~100 ns versus ~80 µs, §5.6.5), and most databases’ working sets are far smaller than the data. The database could leave caching to the kernel’s page cache (§6.6.6), but it knows what the kernel does not: which pages are index roots and which belong to a one-off scan, and which dirty pages must not be written before their log records (§13.5).

How it works internally

A thread that needs page 42 calls `fetch(42)`:

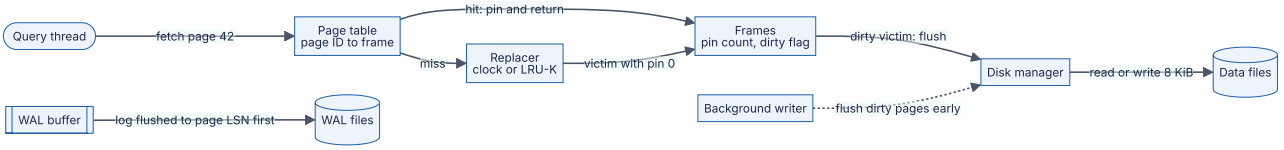
1. **Look up** 42 in the page table (a latched hash table). On a **hit**, increment the pin count, note the access for the replacer, return the frame.
2. On a **miss**, take a frame from the free list or ask the **replacer** for a victim with pin count 0.

3. If the victim is dirty, ensure the log is durable up to its page LSN (the **WAL rule**, §13.5.1), then write it.
4. Read page 42 into the frame, verify its checksum (§13.12), update the page table, pin and return it.
5. The caller later calls `unpin(42, dirty)`. Pinned pages are never evicted, so a leaked pin slowly shrinks the pool.

Background writers (PostgreSQL's bgwriter and checkpointer, InnoDB's page cleaners) flush dirty pages ahead of demand so step 3 rarely waits.

Architecture diagram

Figure 13.2 — Buffer pool components and the page request path.



Reading the diagram. Solid arrows are the synchronous request path: page table, then frame on a hit, or replacer and disk manager on a miss. A dirty victim is written only after the WAL is flushed past its page LSN, an ordering constraint between two storage boundaries (log files and data files). The background writer cleans frames asynchronously (dotted). Failure points: a crash loses dirty frames (recovered from the WAL); leaked pins exhaust evictable frames; a hot page-table latch limits concurrency, so InnoDB partitions large pools into instances. The pool scales vertically with RAM.

13.4.2 Eviction policies: LRU, clock and LRU-K

LRU evicts the page whose last access is oldest. Inside a database it has two problems: moving a list node on every hit makes the shared list a contention point, and **sequential flooding** — one scan of a 100 GB table pushes every hot index page out of a 32 GB pool although no scanned page is read again.

Clock approximates LRU cheaply: frames sit in a circle with a reference bit set on access; a sweeping hand clears set bits and evicts the first unpinned frame whose bit is already clear. PostgreSQL's **clock sweep** uses a usage count (capped at 5) instead of a bit, so frequently used pages survive several sweeps [PostgreSQL source: `src/backend/storage/buffer/README`].

LRU-K evicts the page whose K-th most recent access is oldest [O'Neil, O'Neil and Weikum 1993]. With $K = 2$, a page touched once by a scan is evicted first, so LRU-K resists scans and captures frequency, at the cost of access history. Related designs are **2Q** [Johnson and Shasha 1994] and ARC (§20.9).

InnoDB uses **midpoint insertion**: new pages enter the head of an "old" sublist (37% of the pool) and are promoted only if accessed again after `innodb_old_blocks_time` (1,000 ms by default), so a one-pass scan never reaches the "new" sublist [MySQL Docs: Making the Buffer Pool Scan Resistant]. PostgreSQL gives large scans and `VACUUM` a small private **ring buffer** of frames.

Table 13.2 — Buffer replacement policies.

Policy	How it works	Advantages	Disadvantages	Used in
LRU	Evict oldest last access; list move on every hit	Simple, good for recency	Hit path contention; scan flooding	Textbook baseline; with sublists in InnoDB
Clock / clock sweep	Reference bit or usage count; sweeping hand	Cheap hits; approximates LRU or LFU	Sweep cost when many pages are hot	PostgreSQL; OS page replacement
LRU-K (K = 2)	Evict oldest K-th last access	Scan resistant; frequency aware	History memory; tuning of correlation period	Research, commercial engines
Midpoint / 2Q	Probation queue before main queue	Scan resistant; cheap	Two parameters to tune	InnoDB (midpoint); 2Q variants
Ring buffer for scans	Bulk operations reuse a few private frames	Protects the main pool completely	Applies only to recognized bulk operations	PostgreSQL

13.4.3 Double buffering, `O_DIRECT` and the page cache

Reading files with ordinary `read()` caches every page twice, in the kernel page cache and in the buffer pool. This **double buffering** wastes RAM, and kernel read-ahead and write-back compete with the database's own. `O_DIRECT` transfers data directly between the device and aligned database buffers; the engine must then do its own read-ahead and asynchronous I/O, and still needs `fsync`, because `O_DIRECT` does not flush the drive cache (§6.14.3).

InnoDB owns memory: a buffer pool of most of RAM with `O_DIRECT`, the Linux default from MySQL 8.4 [MySQL Docs: InnoDB Startup Options]. **PostgreSQL** uses buffered I/O, with `shared_buffers` starting around 25% of RAM and the page cache as a second-level cache [PostgreSQL Docs: Resource Consumption]; direct I/O is only a developer option (`debug_io_direct`, version 16), and version 18 added asynchronous I/O (as of 2025). **RocksDB** defaults to buffered I/O plus its own block cache, with optional direct I/O [RocksDB Wiki: Direct IO]. **LMDB** maps the file with `mmap` and lets the kernel be its buffer pool, trading control for simplicity (§6.6.6, [Crotty et al. 2022]).

Mathematical foundations

How much RAM should the buffer pool have? Gray and Putzolu's **five-minute rule** answers by comparing the cost of keeping a page in memory with the cost of the device capacity needed to re-read it [Gray and Putzolu 1987; Gray and Graefe 1997].

FORMULA

Break-even reference interval (the "five-minute rule")

$$T_{BE} = \frac{C_{dev} / IOPS_{dev}}{C_{RAM} \times P}$$

Symbol	Meaning	Units
T_{BE}	Break-even interval: cache a page if it is re-referenced more often than once per T_{BE}	seconds
C_{dev}	Price of one storage device	dollars
$IOPS_{dev}$	Random reads per second the device sustains	reads/s
C_{RAM}	Price of memory	dollars/byte
P	Page size	bytes

WORKED EXAMPLE

Worked example — Break-even for HDD and NVMe (illustrative prices, as of 2025).

Assume DRAM at \$4/GB, so $C_{RAM} = 4 \times 10^{-9}$ \$/byte, and $P = 8192$ bytes. RAM cost of one cached page: $4 \times 10^{-9} \times 8192 = \$3.28 \times 10^{-5}$.

1. **HDD:** $C_{dev} = \$300$, $IOPS_{dev} = 200$. Cost per unit of read capacity: $300/200 = \$1.50$ per read/s. $T_{BE} = 1.50/(3.28 \times 10^{-5}) \approx 45\,700$ s ≈ 12.7 hours.
2. **NVMe SSD:** $C_{dev} = \$400$, $IOPS_{dev} = 400\,000$. Cost: $400/400\,000 = \$0.001$ per read/s. $T_{BE} = 0.001/(3.28 \times 10^{-5}) \approx 30.5$ s.

Interpretation. On HDDs, a page touched about twice a day earns its RAM; on NVMe, only pages re-referenced every half minute or so do. **Engineering implication.** With flash, size the pool for the hot working set and the latency target, not the whole database. CPU cost per I/O and latency SLOs, ignored here, push toward a somewhat larger pool.

Failure scenarios

Table 13.3 — Buffer pool failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Throughput collapses after a report	Scan floods the pool	Hit ratio drops	Re-warm	Scan-resistant policy; reports on a replica
Stalls waiting for free frames	All frames dirty	Wait events; dirty % near 100	Flush faster	Tune page cleaners or bgwriter
High p99 after restart	Cold pool	Low hit ratio	Warm before taking traffic	Dump/reload page list; `pg_prewarm`

COMMON MISTAKE

"A 99% buffer hit ratio means the disk does not matter." At 50,000 page requests per second, 1% misses is 500 random reads per second, each 80 µs on NVMe or 10 ms on an HDD — enough to saturate a hard disk (§5.6.1). Hit ratios must be read together with the absolute miss rate and the device's IOPS.

13.5 Write-ahead logging and recovery

13.5.1 The problem: fast commits without losing data

DEFINITION

Definition — Write-ahead log (WAL). An append-only file of **log records** describing every change to the database, written *before* the changed pages themselves reach disk. Each record is identified by a monotonically increasing **log sequence number (LSN)**, usually its byte offset in the log. Also called the redo log (InnoDB) or journal.

First-principles explanation

A shopkeeper writes every sale as one line at the bottom of a cash book the moment it happens, and tidies the stock ledger later. If a fire alarm interrupts the tidying, nothing is lost: she replays the cash book from the last time the ledger was known to be correct. Appending a line is fast; reorganizing shelves is slow. The cash book is the WAL, the ledger is the data pages, and "last known correct" is a **checkpoint**.

Why does this exist?

Writing a transaction's ten random pages at commit would cost ten random writes, and a crash midway would leave some pages new and some old. The WAL replaces them with one small *sequential* append and one `fsync` per commit group (§6.14.3). Two freedoms follow [Härder and Reuter 1983]: **no-force** — committed changes need not reach their pages at commit, because the log can **redo** them; and **steal** — pages with *uncommitted* changes may be written (say, on eviction), because the log can **undo** them. Steal/no-force gives the fastest commits and freest buffer pool, at the price of needing both redo and undo — the design point of ARIES.

How it works internally

An update record holds its LSN, transaction ID, `prevLSN` (the transaction's previous record, chaining backward), page ID, and **redo** and **undo** information. Records are usually **physiological**: physical to a page, logical within it ("insert at slot 3 of page 7"). Two rules make the WAL correct:

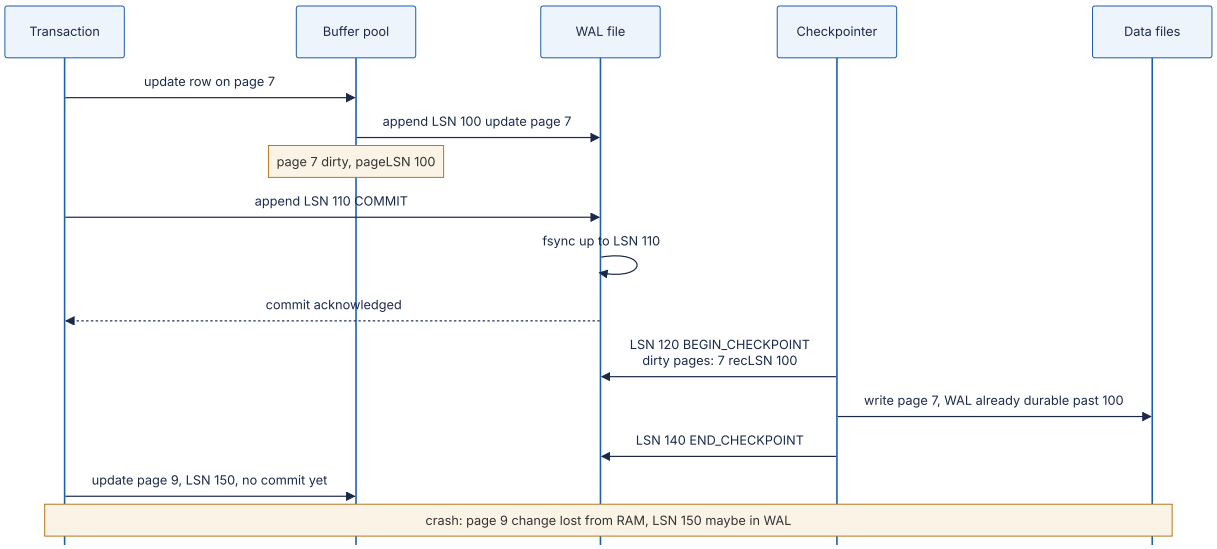
1. **WAL rule:** before a dirty page is written, the log must be durable up to that page's `pageLSN`: $\text{pageLSN} \le \text{flushedLSN}$. Otherwise a change could reach disk with no record able to undo it.
2. **Commit rule:** a transaction is acknowledged only after its commit record is durable.

Because each page stores the LSN of the last change applied to it, recovery can tell whether a record is already on the page, which makes redo **idempotent**.

13.5.2 Checkpoints

A **checkpoint** bounds how much log recovery must read and how much log must be kept. Rather than freezing the database to flush everything, engines take **fuzzy checkpoints**: log the **dirty page table** (each dirty page with its `recLSN`, the LSN that first dirtied it) and the **transaction table**, then flush pages gradually in the background. Redo starts from the smallest `recLSN`; older log can be recycled.

Figure 13.3 — WAL, commit and checkpoint timeline.



Reading the diagram. The WAL file and data files are separate durable storage boundaries; the buffer pool is RAM. Solid arrows are synchronous; the dashed arrow is the acknowledgment. The commit waits only for a sequential log `fsync` up to LSN 110, never for page 7, which the checkpointer writes later, respecting the WAL rule. At the crash, page 9's uncommitted change exists only in RAM and perhaps the log tail; recovery must ensure it does not appear. Scaling: the log is one sequential stream, so its device's flush latency bounds commit throughput; put it on fast, power-loss-protected storage.

13.5.3 ARIES recovery: analysis, redo, undo

ARIES, developed at IBM, is the reference design for steal/no-force recovery and underlies DB2 and SQL Server among others [Mohan et al. 1992]:

1. **Analysis** (forward from the last checkpoint): rebuild the dirty page and transaction tables; committed transactions are **winners**, active ones **losers**.
2. **Redo** (forward from the smallest `recLSN`): **repeat history** — reapply every change, losers' included, whose LSN exceeds the page's `pageLSN`, restoring the exact pre-crash state.
3. **Undo** (backward): roll back losers along their `prevLSN` chains, logging each step as a **compensation log record (CLR)** whose `undoNextLSN` says what to undo next. A crash during recovery redoes the CLRs and resumes, never undoing twice.

Repeating history first is what allows row-level locking and logical undo: after redo, pages are exactly as the undo records expect, even if committed transactions changed them since.

Figure 13.4 — The three ARIES passes over the log.



Reading the diagram. The boxes are positions in one WAL file, ordered by LSN. Analysis scans forward from the checkpoint; redo scans forward from the smallest `recLSN`, which precedes the checkpoint because fuzzy checkpoints do not flush pages; undo (dotted: it follows `prevLSN` chains, not a sequential scan) walks back to the oldest loser record, writing CLRs. The log is read; data pages are read and written through the buffer pool. Failure point: a long-running transaction pushes point A far back, lengthening undo and log retention. Scaling: redo parallelizes by page.

Mathematical foundations

FORMULA

Redo work after a crash

$$W_{redo} \approx r_{WAL} \times \Delta t_{ckpt}, \qquad T_{redo} \approx \frac{W_{redo}}{b_{replay}}$$

Symbol	Meaning	Units
W_{redo}	Log volume between the redo start point and the crash	bytes
r_{WAL}	Rate at which the workload generates log	bytes/s
Δt_{ckpt}	Time between checkpoints (worst case, crash just before the next one)	s
T_{redo}	Duration of the redo pass	s
b_{replay}	Replay throughput, usually limited by random page reads, not log reads	bytes/s

WORKED EXAMPLE

Worked example — Choosing a checkpoint interval.

A PostgreSQL server generates r_{WAL} = 30 MB/s of WAL at peak. Replay runs at b_{replay} = 150 MB/s (single-threaded redo, mostly cached pages).

1. Checkpoint every 5 minutes: $W_{redo} = 30 \times 300 = 9000$ MB $= 9$ GB. $T_{redo} = 9000/150 = 60$ s.
2. Checkpoint every 1 minute: $W_{redo} = 30 \times 60 = 1.8$ GB; $T_{redo} = 12$ s.
3. The cost: with full-page writes (§13.12.3), the first change to each page after a checkpoint logs the whole 8 KiB page. If 200,000 distinct pages are dirtied per minute, 1-minute checkpoints add up to $200\,000 \times 8192 \approx 1.64$ GB per minute, ≈ 27 MB/s — nearly doubling the WAL rate.

Interpretation. Checkpoint frequency trades recovery time against steady-state write volume. **Engineering implication.** Derive the interval from the recovery-time objective (RTO, §39.8) and the measured WAL rate; write-heavy PostgreSQL systems often raise `checkpoint_timeout` (default 5 min) and `max_wal_size` (default 1 GB), letting a standby failover (§27.11) hide the longer recovery.

13.5.4 Group commit and relaxed durability

Group commit lets one log `fsync` cover every commit that arrived during the previous flush (formula in §6.14.3). Engines also trade durability for speed explicitly: InnoDB's `innodb_flush_log_at_trx_commit = 1` (default) flushes at every commit, while `2` flushes about once per second, so an OS crash can lose about a second of commits. PostgreSQL's `synchronous_commit = off` risks the last fraction of a second of commits but never corruption, because the WAL rule still holds.

Real-world industry applications

INDUSTRY EXAMPLE

PostgreSQL versus InnoDB recovery. PostgreSQL recovers by redo only: MVCC never overwrites rows in place, so an aborted transaction's versions stay in the heap, invisible because `pg_xact` marks it aborted, until `VACUUM` removes them. The same WAL feeds replication and point-in-time recovery. InnoDB is ARIES-like: redo log plus separate **undo logs** that serve rollback and consistent reads; recovery applies redo, then rolls back incomplete transactions. *Evidence: Primary* — [PostgreSQL Docs: Reliability and the Write-Ahead Log; MySQL Docs: InnoDB Redo Log; InnoDB Undo Logs]

Failure scenarios

Table 13.4 — WAL and recovery failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Recovery takes 20 minutes	Long checkpoint interval, high WAL rate	Redo progress in logs	Fail over to a replica	Size interval from the RTO
Disk fills with WAL	Stalled checkpoints or unconsumed replication slot	WAL size alert	Fix consumer; drop slot	Monitor retention and slot lag
Acknowledged commits lost	Drive cache not flushed; relaxed settings	Gaps versus clients	Restore from replica	Power-loss protection; audit `fsync` settings
Undo grows without bound	Long-running transaction	Oldest transaction age	Kill or commit it	Transaction timeouts

COMMON MISTAKE

"The checkpoint is what makes a commit durable." Commits are durable when their log records are flushed. A checkpoint only bounds recovery work and lets old log be recycled.

13.6 B-tree storage engines

13.6.1 Storing a B+ tree in pages

DEFINITION

Definition — B-tree storage engine. A storage engine that keeps each table or index as a B+ tree of fixed-size pages and **updates pages in place**: a change to a key rewrites the page that holds it. Internal pages hold separator keys and child page numbers; leaf pages hold the keys with either the rows themselves (a clustered index) or pointers to them.

The logical structure of B+ trees is covered in §11.8. Here the questions are physical: how the tree grows in pages, what a lookup costs in I/O, and how threads share it.

First-principles explanation

Picture a filing cabinet with guide cards: one card per drawer, one per folder. To find a document you read three cards, not a million documents. When a folder overflows you split it and add a guide card above; rarely, the drawer overflows too. The cabinet stays balanced because it grows only by splitting, bottom up.

How it works internally

Lookup. From the root, binary-search separator keys and fetch the child through the buffer pool until reaching a leaf.

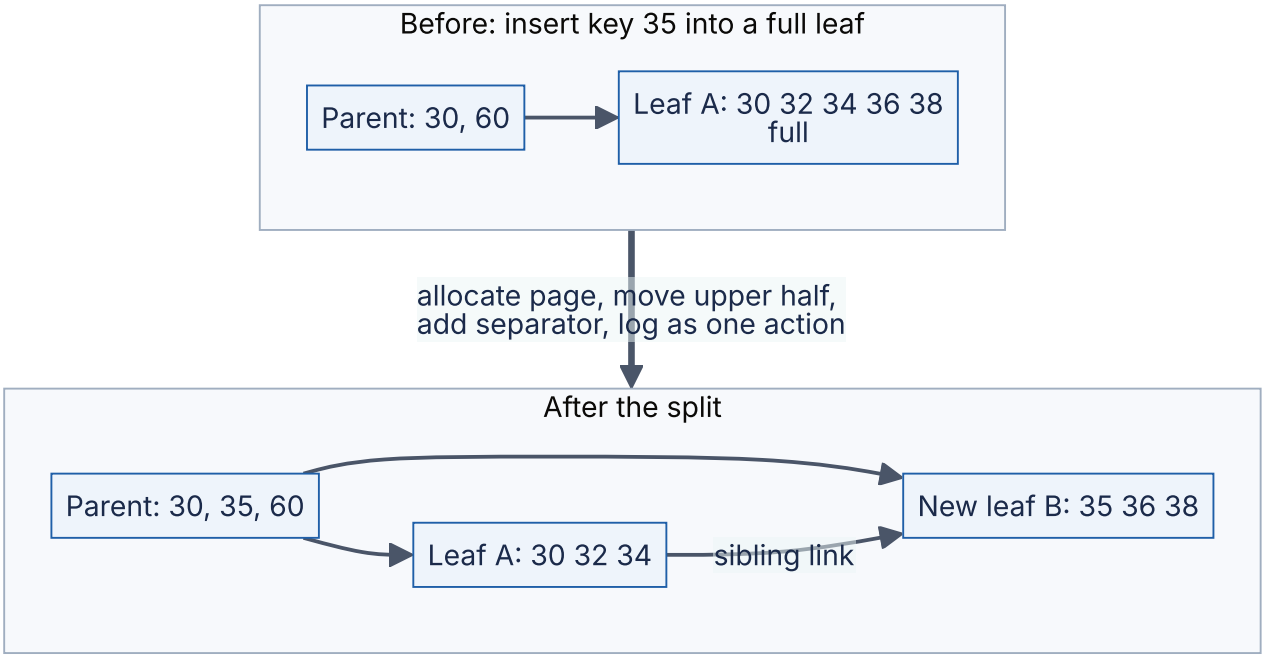
Insert and split. If the leaf has room, insert into its slotted page (§13.3) and log it. Otherwise:

1. Allocate a new page and move the upper half of the entries to it.
2. Link it into the leaf sibling chain.
3. Insert a separator into the parent, splitting the parent the same way if it is full; a root split adds a level.
4. Log the split as one unit. ARIES treats it as a **nested top action**: it is not undone if the triggering transaction aborts, because others may already use the new page [Mohan et al. 1992].

For rightmost inserts (auto-increment keys, timestamps), PostgreSQL and InnoDB split unevenly, leaving the old page nearly full instead of a trail of half-empty pages.

Delete and merge. Merging under-full pages reverses a split, but eager merging thrashes. InnoDB attempts a merge below `MERGE_THRESHOLD` (50% by default); PostgreSQL reclaims only pages `VACUUM` finds completely empty [MySQL Docs: Configuring the Merge Threshold for Index Pages; PostgreSQL source: `src/backend/access/nbtree/README`].

Figure 13.5 — A leaf page split propagating a separator to the parent.



Reading the diagram. Each box is a page in one data file; arrows are synchronous pointers. Inserting 35 into full leaf A allocates leaf B, moves the upper entries, links A to

B for range scans, and adds a separator to the parent. One logical insert dirties three pages — write amplification. Failure point: a crash between writing B and updating the parent would orphan B, hence logging the split atomically. Scaling: with 57 rows per leaf and fan-out 716 (below), a leaf splits about every 28 inserts into it and a parent about every 358 leaf splits.

Mathematical foundations

The height formula $h = 1 + \lceil \log_f \lceil n/L \rceil \rceil$ (fan-out f, L rows per leaf) is derived in §11.8.2, which works a billion-row InnoDB table at 80% usable space: fan-out 819, 65 rows per leaf, four levels, 294 MiB of internal pages. The physical question here is how sensitive those numbers are to page fill.

WORKED EXAMPLE

Worked example — A 1-billion-row InnoDB table.

Repeat the §11.8.2 example with 16 KiB pages filled to 70% (a typical steady state after random inserts; random B-tree fill averages about $\ln 2 \approx 69\%$ [Yao 1978]), rows of 200 bytes, and internal entries of 16 bytes (an 8-byte key, a page number and record overhead).

1. Usable bytes per page: $16\,384 \times 0.7 = 11\,469$.
2. Rows per leaf: $\lfloor 11\,469/200 \rfloor = 57$. Leaves: $10^9/57 = 17\,543\,860$.
3. Fan-out: $\lfloor 11\,469/16 \rfloor = 716$.
4. Height: $\log_{716} 17\,543\,860 = \ln(1.754 \times 10^7)/\ln 716 = 16.68/6.57 = 2.54$, so $\lceil 2.54 \rceil = 3$ internal levels and $h = 4$.
5. Internal pages: $17\,543\,860/716 \approx 24\,503$, then $\lceil 24\,503/716 \rceil = 35$, then 1 root: $24\,539$ pages $\times$ 16 KiB ≈ 402 MB.

Interpretation. Four levels index a billion rows, as at 80% fill, and the internal levels fit in about 400 MB (about 30% more than the 308 MB, or 294 MiB, of §11.8.2), so a point lookup costs at most **one** random read. **Engineering implication.** Read cost is nearly flat in table size while internal pages stay cached; the real danger is a working set of *leaves* larger than RAM. Wide keys cut fan-out and can add a level, one reason to prefer compact primary keys.

13.6.2 Concurrency: latches and latch crabbing

A **latch** is a short-term mutex on an in-memory structure such as a page frame, held for microseconds; a **lock** protects logical data for a transaction's lifetime and takes part in deadlock detection (§11.15). **Latch crabbing** descends like a crab: hold the parent's latch, acquire the child's, and release the parent when safe [Bayer and Schkolnick 1977].

1. **Readers** latch the child shared, then release the parent: at most two latches at a time.
2. **Writers** latch exclusively and release all ancestors on reaching a **safe** node, one that cannot split (has free space) or merge on this operation.
3. **Optimistically**, since splits are rare, writers descend with shared latches and latch only the leaf exclusively, restarting pessimistically if a split is needed. The root is then almost never exclusively latched.

B-link trees [Lehman and Yao 1981] add a high key and right-sibling pointer to every page, so a reader arriving just after a concurrent split follows the right-link instead of latching the parent. PostgreSQL's B-tree is based on this design.

13.6.3 Copy-on-write B-trees

A **copy-on-write (COW)** or shadowing B-tree never modifies a page reachable from the current root. Changing a leaf writes a new leaf, a new parent pointing to it, and so on to a new root; a commit atomically switches the root pointer. Readers that began earlier keep the old root — a consistent snapshot without locks — and no WAL is needed [Rodeh 2008].

> INDUSTRY EXAMPLE
>
> **LMDB.** Written for the read-dominated OpenLDAP directory server, LMDB memory-maps its file, allows one writer and any number of lock-free readers, and alternates between two **meta pages** recording the current root; after a crash the meta page with the higher valid transaction ID *is* the database, so there is no recovery procedure. Freed pages are reused once no reader needs them. Trade-offs: each commit copies the root-to-leaf path — with height 4 and 4 KiB pages, 16 KiB for a 100-byte update, about 160× write amplification unless a transaction batches many keys — long readers bloat the file, and one writer bounds write concurrency. Btrfs and ZFS use the same idea (§6.14.2). *Evidence: Primary* — [LMDB Docs]

Advantages and disadvantages

B-trees give predictable one-I/O point reads, efficient range scans along sibling links, and mature row-level locking. Their weaknesses come from in-place updates: random writes, page-sized write amplification (§13.11), partly empty pages ($SA \approx 1/0.69 \approx 1.45$ from fill factor alone) and torn-page risk (§13.12).

COMMON MISTAKE

“B-trees need one disk write per insert.” At commit only a small sequential log record is written; the page is written later, by which time many inserts may share it. Insert cost therefore depends on key locality (§13.11.2).

13.7 Log-structured storage

13.7.1 The idea: never overwrite, only append

DEFINITION

Definition — Log-structured storage. A storage design in which every write — insert, update or delete — is appended to the end of a file, and existing data is never modified in place. Old versions become garbage that a background **compaction** (or merge) process removes by copying live data into new files.

First-principles explanation

Suppose the shopkeeper’s cash book of §13.5 becomes *the* record: a balance is the most recent line about a customer. Writing is one append and recovery is trivial, but finding the latest line needs an index, and obsolete lines must be tidied away. Every log-structured engine answers those two problems; the idea goes back to the log-structured file system [Rosenblum and Ousterhout 1992].

Why does this exist?

Sequential writes are 100–600× faster than random ones on HDDs (§5.6.1) and still better on SSDs, where they also cut the drive’s internal write amplification (§5.6.3). Log structuring makes *all* writes sequential and moves the cost to reads and background compaction.

13.7.2 Bitcask: a hash index over a log

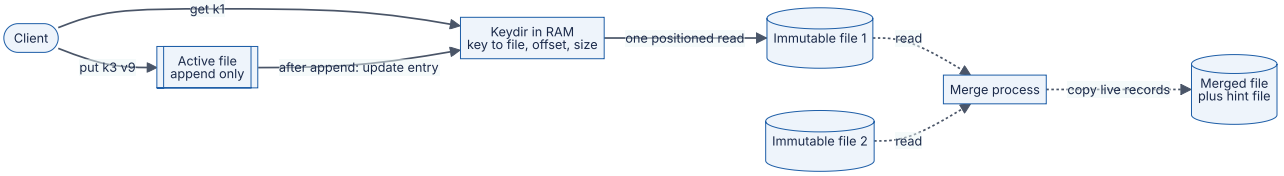
Bitcask, the default storage backend of the Riak key-value store, is the simplest complete log-structured engine and a good mental model [Sheehy and Smith 2010].

How it works internally

1. **Data files.** One **active** file takes appends of `CRC | timestamp | key size | value size | key | value`; full files become immutable. A delete appends a **tombstone**.
2. **Keydir.** An in-memory hash table maps every key to the file, position and size of its latest record.

3. **Read.** One keydir lookup and **one** positioned read.
4. **Merge.** In the background, copy records the keydir still references into new files, delete the old ones, and write **hint files** (keys and positions) for fast restarts.
5. **Recovery.** Rebuild the keydir from hint files and the active file's tail; a CRC mismatch marks a torn final record.

Figure 13.6 — Bitcask: an in-memory hash index over append-only files.



Reading the diagram. Writes append synchronously to the active file and then update the keydir; reads take one keydir lookup and one positioned read. Merging (dotted) is asynchronous. The storage boundary is stark: the whole index in RAM, all data on disk. Failure points: a crash loses the keydir, rebuilt from hint files; a torn final record fails its CRC. Scaling limit: every key must fit in RAM.

Mathematical foundations

With 200 million 32-byte keys and an assumed ~50 bytes of per-entry overhead (illustrative), the keydir needs $2 \times 10^8 \times 82 = 16.4$ GB of RAM whatever the value sizes: fine for 1 KB values (200 GB of data), absurd for 10-byte values.

Advantages and disadvantages

Bitcask gives one append per write, one seek per read and simple recovery, but no **range queries** (a hash has no order) and a RAM-bound key count. Sorting the files by key and allowing a sparse index removes both limits — which is the LSM tree.

13.8 LSM trees: memtables and SSTables

13.8.1 From a log to a log-structured merge tree

DEFINITION

Definition — Log-structured merge tree (LSM tree). A storage structure that buffers writes in a sorted in-memory table (the **memtable**), periodically writes it out as an immutable sorted file (a **sorted string table, SSTable**), and merges SSTables in the background (**compaction**) into progressively larger sorted runs organized in **levels**. Introduced by O'Neil, Cheng, Gawlick and O'Neil [O'Neil et al. 1996]; the memtable/SSTable design popularized by Google's Bigtable [Chang et al. 2006] and implemented in LevelDB, RocksDB, Cassandra, HBase, ScyllaDB and many others.

First-principles explanation

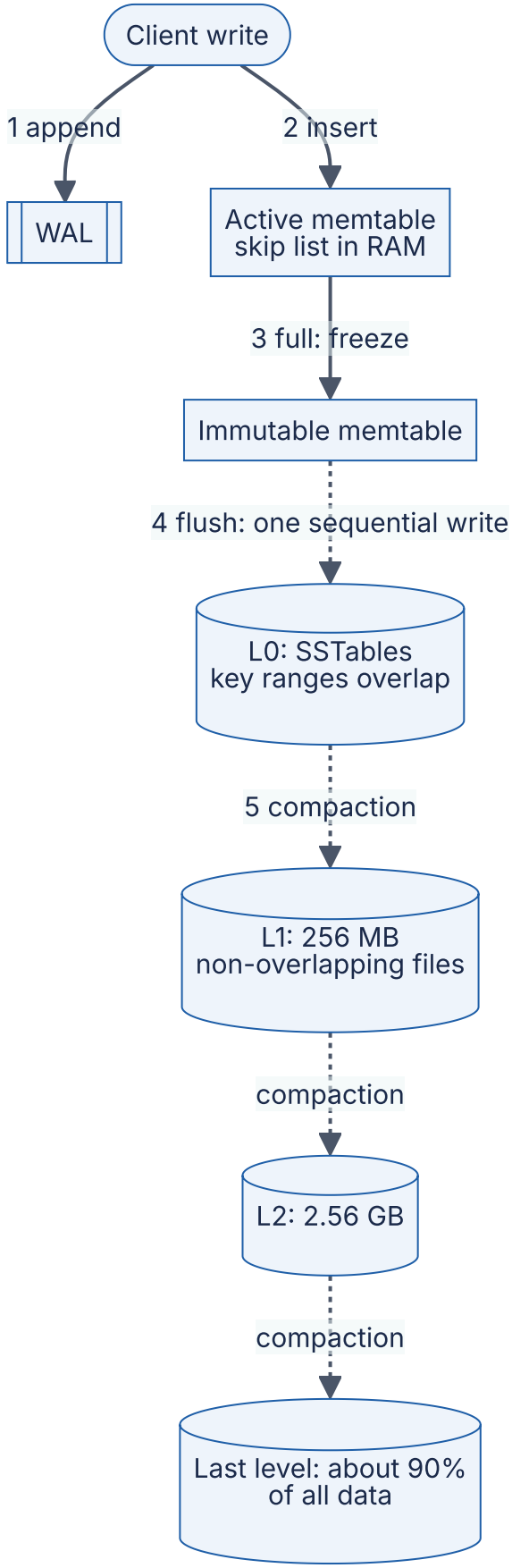
A librarian sorts returned books on a trolley instead of walking to a random shelf for each; when it is full she shelves the sorted batch on a side shelf in one pass. To find a book you check the trolley, then the newest batch, then older ones; nightly she merges batches into the main shelves. Trolley = memtable, batch = SSTable, nightly merge = compaction.

How it works internally: the write path

1. **Log:** append to the WAL (§13.5.4), which exists only to rebuild the memtable after a crash.
2. **Memtable:** insert into a sorted in-memory structure — a skip list in LevelDB and RocksDB (§17.7). A delete inserts a **tombstone**.
3. **Freeze:** at the size limit (RocksDB `write_buffer_size`, 64 MB by default) the memtable becomes immutable and a new one starts, so writes never wait.
4. **Flush:** write it as a new **level 0** SSTable in one sequential pass; drop the WAL segment it covered.
5. **Compact:** merge SSTables into larger non-overlapping runs in deeper levels (§13.9), discarding overwritten values and, at the bottom, tombstones.

An **SSTable** is immutable and sorted: **data blocks** (RocksDB default 4 KiB before compression), an **index block** with one entry per block, a **filter block** with a Bloom filter (§13.10), and a **footer** locating them. Sorting allows a sparse index and lets several SSTables be merged sequentially into a new one.

Figure 13.7 — LSM tree write path and levels.

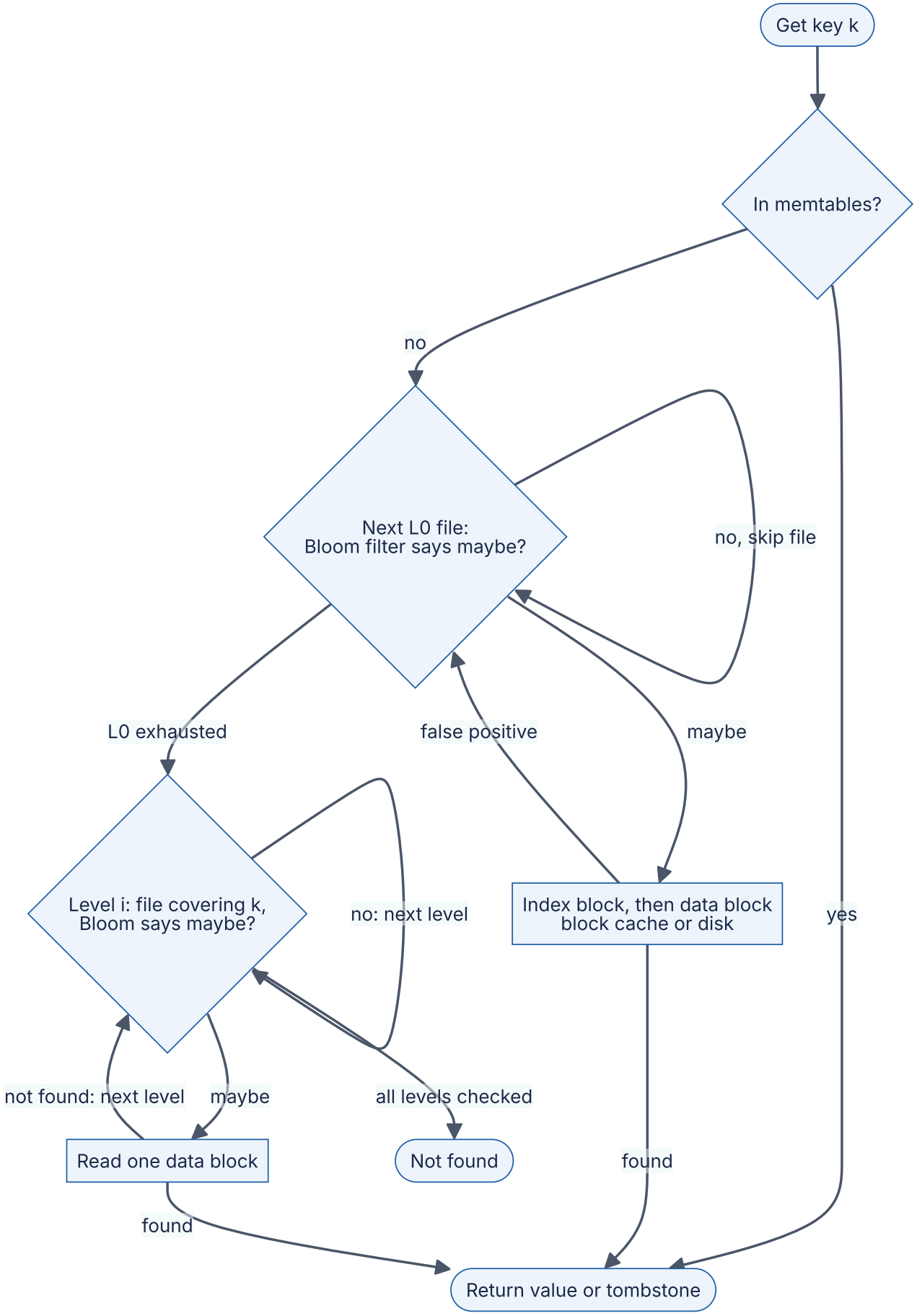

Client write
1 append
2 insert
WAL
Active memtable
skip list in RAM
3 full: freeze
Immutable memtable
4 flush: one sequential write
L0: SSTables
key ranges overlap
5 compaction
L1: 256 MB
non-overlapping files
compaction
L2: 2.56 GB
compaction
Last level: about 90%
of all data

Reading the diagram. Only steps 1 and 2 are synchronous: a sequential WAL append and an in-memory insert. Flushing and compaction are asynchronous (dotted). L0 holds whole flushes with overlapping key ranges; from L1 down, each level is one sorted run in non-overlapping files, about ten times larger than the level above (RocksDB defaults: `max_bytes_for_level_base` 256 MB, multiplier 10). Memtables live in RAM; WAL and SSTables on disk. Failure points: if compaction lags, L0 files pile up and the engine slows or stops writes (a **write stall**); a crash loses memtables, rebuilt from the WAL. Scaling is bounded by compaction bandwidth.

How it works internally: the read path

A point lookup searches newest to oldest and stops at the first version found (a tombstone means deleted): the memtables; every L0 file, newest first, since their ranges overlap; then the one file per deeper level whose range covers the key. In each file it checks the Bloom filter, then the index block, then reads one data block, from the **block cache** if present. A **range scan** cannot stop early: it merges iterators over every memtable and overlapping SSTable, and Bloom filters do not help it.

Figure 13.8 — LSM point-lookup read path.


Get key k
In memtables?
yes
no
Next L0 file:
Bloom filter says maybe?
no, skip file
maybe
false positive
L0 exhausted
Level i: file covering k,
Bloom says maybe?
no: next level
Index block, then data block
block cache or disk
found
all levels checked
not found: next level
maybe
Read one data block
Not found
found
Return value or tombstone

Reading the diagram. Every step is synchronous. Memtables are checked in RAM; each L0 file is tried in turn (the loop), and its in-memory Bloom filter usually rules it out without I/O; deeper levels need one file each. Only a “maybe” reaches the block cache or disk — the storage boundary where latency jumps from microseconds to tens of microseconds. A false positive wastes one block read. Failure points: L0 backlog lengthens every lookup; an undersized block cache turns index and data reads into I/O. Cost grows with the number of sorted runs, which compaction keeps small.

Mathematical foundations

FORMULA

Number of levels in a leveled LSM tree

$$L = \lceil \log_T \frac{N}{S_1} \rceil + 1 \quad (\text{levels } L_1 \ldots L_L, \text{ for } N > S_1)$$

Symbol	Meaning	Units
L	Number of levels below L0	levels
N	Total data size	bytes
S_1	Target size of level 1	bytes
T	Size ratio (fan-out) between adjacent levels	dimensionless

WORKED EXAMPLE

Worked example — Levels for 1 TB in RocksDB with default sizing.

$S_1 = 256$ MB, $T = 10$, $N = 1$ TB $= 10^{12}$ bytes.

1. $N/S_1 = 10^{12}/(2.56 \times 10^8) = 3906$.
2. $\log_{10} 3906 = 3.59$; $\lceil 3.59 \rceil = 4$; $L = 4 + 1 = 5$.
3. Check: capacities are L1 256 MB, L2 2.56 GB, L3 25.6 GB, L4 256 GB, L5 2.56 TB. Four levels hold at most 284 GB, so 1 TB needs L5.

Interpretation. Five levels plus L0 cover a terabyte; with upper levels sized backward from the last (`level_compaction_dynamic_level_bytes` , default since RocksDB 8.4), about 90% of data sits in the last level [RocksDB Wiki: Leveled Compaction]. **Engineering implication.** A lookup probes at most ~5 levels plus a few L0 files, and every level is another rewrite of each byte (§13.9); the size ratio T trades one against the other.

Code example

The program below is a complete, runnable toy LSM tree in about 250 lines of Python: WAL with replay, memtable, SSTables with a sparse index and Bloom filter, a manifest that records which SSTables are live, and full merge compaction. It counts the bytes written to measure write amplification. It is saved as `code/ch13/toy_lsm.py`.

```
        """A toy LSM tree: write-ahead log, memtable, SSTable flush, merge compaction,
        manifest and WAL replay. Teaching code: single-threaded, JSON-lines files."""
        import bisect
        import hashlib
        import heapq
        import json
        import os

        TOMBSTONE = None          # a deleted key is stored as key -> None


        def _fsync_dir(path):
            fd = os.open(path, os.O_RDONLY)
            try:
                os.fsync(fd)
            finally:
                os.close(fd)


        class BloomFilter:
            """m bits and k hash positions per key, derived from one SHA-256 digest."""

            def __init__(self, n_keys, bits_per_key=10, k=7, bits=0):
                self.m = max(64, n_keys * bits_per_key)
                self.k = k
                self.bits = bits

            def _positions(self, key):
                d = hashlib.sha256(key.encode()).digest()
                h1, h2 = int.from_bytes(d[:8], "big"), int.from_bytes(d[8:16], "big")
                return [(h1 + i * h2) % self.m for i in range(self.k)]   # double hash-
ing

            def add(self, key):
                for p in self._positions(key):
                    self.bits |= 1 << p

            def may_contain(self, key):
                return all(self.bits >> p & 1 for p in self._positions(key))


        class SSTable:
            """Immutable sorted file: one JSON record per line, then a footer line hold-
ing
            a sparse index (every INDEX_EVERY-th key -> byte offset) and a Bloom fil-
ter."""
            INDEX_EVERY = 16

            def __init__(self, path):
                self.path = path
                with open(path, "rb") as f:
                    f.seek(0, os.SEEK_END)
                    end = f.tell()
                    f.seek(max(0, end - 8))
                    footer_len = int(f.read(8))           # last 8 bytes: footer length
                    f.seek(end - 8 - footer_len)
```

```
                    footer = json.loads(f.read(footer_len))
                self.index_keys = [k for k, _ in footer["index"]]
                self.index_offsets = [o for _, o in footer["index"]]
                self.data_end = footer["data_end"]
                self.bloom = BloomFilter(0, bits=int(footer["bloom"], 16))
                self.bloom.m, self.bloom.k = footer["m"], footer["k"]
                self.size = end

            @staticmethod
            def write(path, items, sync=True):
                """items: iterable of (key, value) in ascending key order. Returns bytes
written."""
                items = list(items)
                bloom = BloomFilter(len(items))
                index, tmp = [], path + ".tmp"
                with open(tmp, "wb") as f:
                    for i, (k, v) in enumerate(items):
                        if i % SSTable.INDEX_EVERY == 0:
                            index.append((k, f.tell()))
                        bloom.add(k)
                        f.write((json.dumps([k, v]) + "\n").encode())
                    footer = json.dumps({"index": index, "data_end": f.tell(), "m":
bloom.m,
                                         "k": bloom.k, "bloom": format(bloom.bits,
"x")}).encode()
                    f.write(footer + b"%08d" % len(footer))
                    if sync:
                        f.flush()
                        os.fsync(f.fileno())
                    size = f.tell()
                os.replace(tmp, path)                    # appears atomically, complete
or not at all
                return size

            def get(self, key):
                """Returns (found, value). Reads at most INDEX_EVERY records from
disk."""
                if not self.bloom.may_contain(key):
                    return False, None                   # definitely absent: no disk
read
                i = bisect.bisect_right(self.index_keys, key) - 1
                if i < 0:
                    return False, None
                with open(self.path, "rb") as f:
                    f.seek(self.index_offsets[i])
                    for _ in range(self.INDEX_EVERY):
                        if f.tell() >= self.data_end:
                            break
                        k, v = json.loads(f.readline())
                        if k == key:
                            return True, v
                        if k > key:
                            break
                return False, None

            def scan(self):
                with open(self.path, "rb") as f:
```

```
                    while f.tell() < self.data_end:
                        k, v = json.loads(f.readline())
                        yield k, v


        class LSMTree:
            def __init__(self, directory, memtable_limit=256, compact_at=4, sync=True):
                self.dir, self.limit, self.compact_at, self.sync = directory,
memtable_limit, compact_at, sync
                os.makedirs(directory, exist_ok=True)
                self.stats = {"user": 0, "wal": 0, "flush": 0, "compaction": 0}
                self.memtable = {}
                manifest = os.path.join(directory, "MANIFEST")
                names = json.load(open(manifest)) if os.path.exists(manifest) else []
                self.tables = [SSTable(os.path.join(directory, n)) for n in names]  #
oldest first
                self.next_id = 1 + max([int(n[4:10]) for n in names], default=0)
                for n in os.listdir(directory):           # delete orphans left by a
crash
                    if n.startswith("sst-") and n not in names:
                        os.remove(os.path.join(directory, n))
                self.wal_path = os.path.join(directory, "wal.log")
                self._replay_wal()
                self.wal = open(self.wal_path, "ab")

            # ---------- durability ----------
            def _replay_wal(self):
                if not os.path.exists(self.wal_path):
                    return
                with open(self.wal_path, "rb") as f:
                    for line in f:
                        try:
                            k, v = json.loads(line)
                        except ValueError:
                            break                         # torn final record from a
crash: stop
                        self.memtable[k] = v

            def _save_manifest(self):
                tmp = os.path.join(self.dir, "MANIFEST.tmp")
                with open(tmp, "w") as f:
                    json.dump([os.path.basename(t.path) for t in self.tables], f)
                    if self.sync:
                        f.flush()
                        os.fsync(f.fileno())
                os.replace(tmp, os.path.join(self.dir, "MANIFEST"))
                if self.sync:
                    _fsync_dir(self.dir)

            # ---------- write path ----------
            def put(self, key, value):
                record = (json.dumps([key, value]) + "\n").encode()
                self.wal.write(record)                    # 1. log first (write-ahead)
                self.wal.flush()
                if self.sync:
                    os.fsync(self.wal.fileno())           #    durable before we ac-
knowledge
```

```
                self.stats["user"] += len(key) + len(value or "")
                self.stats["wal"] += len(record)
                self.memtable[key] = value                # 2. then apply to the
memtable
                if len(self.memtable) >= self.limit:
                    self.flush()

            def delete(self, key):
                self.put(key, TOMBSTONE)

            def _new_path(self):
                path = os.path.join(self.dir, "sst-%06d.jsonl" % self.next_id)
                self.next_id += 1
                return path

            def flush(self):
                if not self.memtable:
                    return
                path = self._new_path()
                self.stats["flush"] += SSTable.write(path,
sorted(self.memtable.items()), self.sync)
                self.tables.append(SSTable(path))
                self._save_manifest()                     # the new table is now part of
the tree
                self.memtable = {}
                self.wal.close()
                self.wal = open(self.wal_path, "wb")      # its records are in the
SSTable: truncate
                if len(self.tables) >= self.compact_at:
                    self.compact()

            def compact(self):
                """Full merge of every SSTable into one. Newest version of each key
wins;
                tombstones are dropped because no older table remains below the out-
put."""
                def tagged(age, table):                   # age 0 = newest
                    for k, v in table.scan():
                        yield k, age, v
                streams = [tagged(age, t) for age, t in
enumerate(reversed(self.tables))]
                merged, last = [], object()
                for k, _, v in heapq.merge(*streams):     # sorted by key, then newest
first
                    if k != last:
                        last = k
                        if v is not TOMBSTONE:
                            merged.append((k, v))
                path = self._new_path()
                self.stats["compaction"] += SSTable.write(path, merged, self.sync)
                old, self.tables = self.tables, [SSTable(path)]
                self._save_manifest()                     # switch atomically, then
delete inputs
                for t in old:
                    os.remove(t.path)

            # ---------- read path ----------
```

```
            def get(self, key):
                if key in self.memtable:
                    return self.memtable[key]             # may be TOMBSTONE -> None
                for table in reversed(self.tables):       # newest to oldest
                    found, value = table.get(key)
                    if found:
                        return value
                return None

            def write_amplification(self):
                disk = self.stats["wal"] + self.stats["flush"] + self.stats["com-
paction"]
                return disk / max(1, self.stats["user"])


        if __name__ == "__main__":
            import random
            import tempfile

            rng = random.Random(13)
            with tempfile.TemporaryDirectory() as d:
                db, model = LSMTree(d, sync=False), {}
                for i in range(20_000):
                    key = "user:%04d" % rng.randrange(2_000)
                    if rng.random() < 0.1:
                        db.delete(key)
                        model.pop(key, None)
                    else:
                        value = "v%d-%s" % (i, "x" * rng.randrange(20, 60))
                        db.put(key, value)
                        model[key] = value
                stats, wa = dict(db.stats), db.write_amplification()
                unflushed = len(db.memtable)

                db.wal.close()                            # simulate a crash: memtable
is lost
                db = LSMTree(d, sync=False)               # reopen: manifest + WAL re-
play
                assert len(db.memtable) == unflushed, "WAL replay lost records"
                for k in ("user:%04d" % n for n in range(2_000)):
                    assert db.get(k) == model.get(k), k
                print("all 2000 keys match after crash and WAL replay")
                print("live keys:", len(model), " SSTables:", len(db.tables),
                      " memtable entries replayed:", unflushed)
                print("bytes by user=%(user)d wal=%(wal)d flush=%(flush)d compaction=%
(compaction)d" % stats)
                print("write amplification = %.1f" % wa)
```

Output (Python 3.13):

```
all 2000 keys match after crash and WAL replay
live keys: 1799  SSTables: 1  memtable entries replayed: 12
bytes by user=1007515 wal=1191501 flush=1189724 compaction=2721866
write amplification = 5.1
```

What to notice:

- **Write-ahead order:** `put` logs before touching the memtable. After the simulated crash, 12 unflushed entries return from the WAL and all 2,000 keys (deleted ones included) match the reference dictionary.
- **Torn tail:** replay stops at the first unparsable record, as real engines stop at the first failed checksum (§13.12).
- **Manifest:** an SSTable joins the tree only when the atomically replaced `MANIFEST` names it (§6.14.3); orphans are deleted at startup, like RocksDB's MANIFEST. Switching the manifest before deleting compaction inputs stops an old input resurrecting a key whose tombstone was dropped.
- **Write amplification 5.1** = WAL 1.18 + flush 1.18 + compaction 2.70 (the 18% is JSON framing). Full merges rewrite everything, so real engines compact incrementally (§13.9).
- **Bloom filter:** 10 bits per key, 7 hashes; about 1% "maybe" on 10,000 absent keys, as §13.10 predicts.

13.9 Compaction strategies

13.9.1 Why compaction strategy is the central LSM decision

DEFINITION

Definition — Compaction. The background merge of sorted runs in an LSM tree that removes overwritten values and tombstones and bounds the number of runs a read must consult. A **compaction strategy** decides which runs to merge and when; it sets the engine's read, write and space amplification.

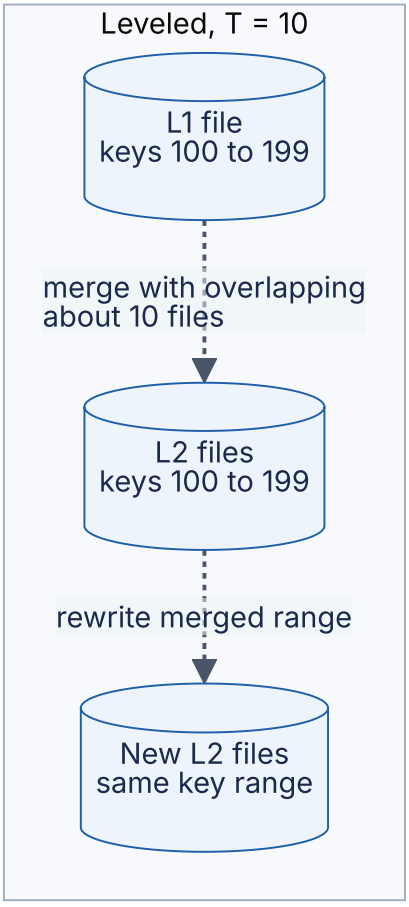
Every byte is rewritten by compaction several times on its way down. Merging eagerly makes reads cheap and writes expensive; merging lazily does the reverse.

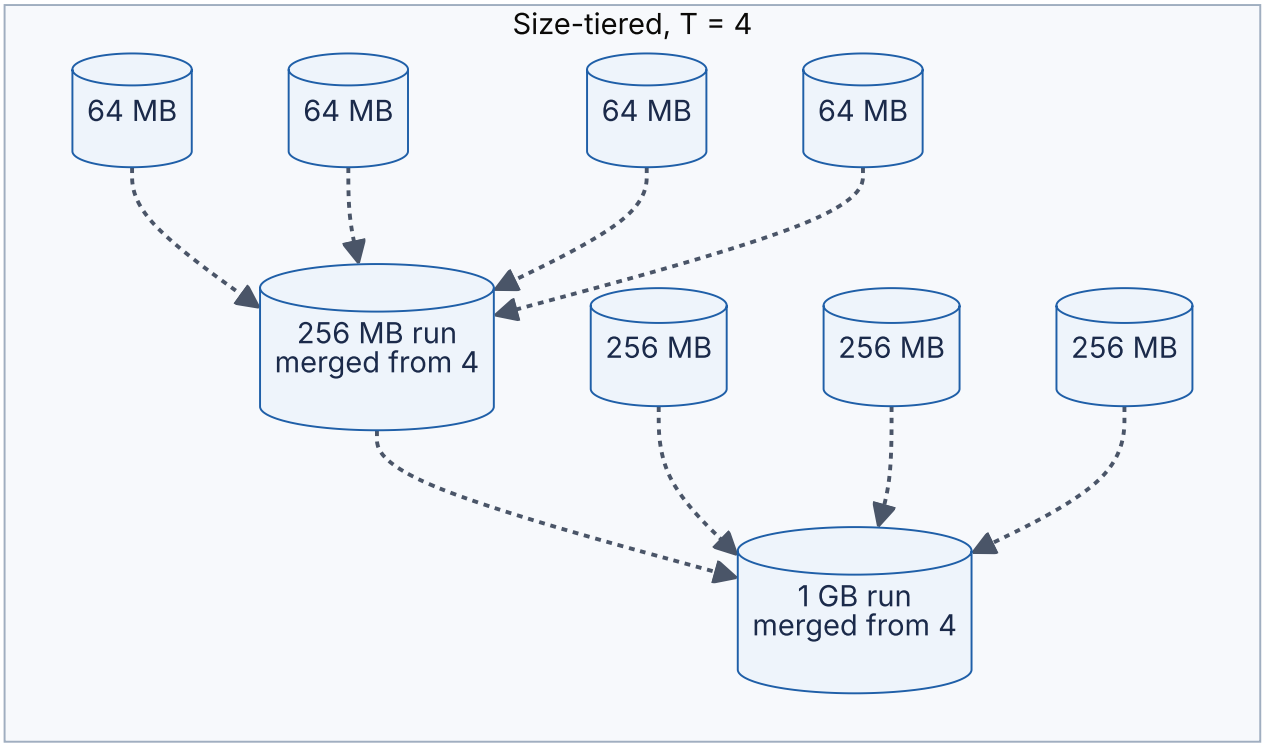
Size-tiered (tiering). When about T runs of similar size exist (Cassandra's default `min_threshold` is 4), merge them into one run of the next tier. Each byte is rewritten about once per tier, but up to $T - 1$ runs per tier may hold versions of a key, and merging the largest runs needs equal free space. Used by Cassandra STCS and RocksDB **universal** compaction.

Leveled (leveling). Each level is one sorted run of non-overlapping files, T times larger than the level above; an overflowing level pushes one file into the *overlapping* files of the next. Reads probe one file per level and obsolete data is bounded, but each byte entering a level is merged with up to T times its size. Used by LevelDB, RocksDB (default) and Cassandra LCS.

Hybrids. RocksDB’s default is already one: L0 tiered, deeper levels leveled. **Lazy leveling** tiers all levels but the last, which holds most data [Dayan and Idreos 2018]; Cassandra 5.0’s Unified Compaction Strategy slides between the two per level (as of 2025). **Time-window** compaction never merges across time windows, so expired windows are dropped whole.

Figure 13.9 — Size-tiered versus leveled compaction.





Reading the diagram. All arrows are asynchronous background compactions; clients wait only if a backlog triggers a write stall. Tiering copies each byte once per tier but

leaves up to three runs per tier. Leveling merges one L1 file (keys 100–199) with the ~10 overlapping L2 files and rewrites them: about ten bytes rewritten per byte promoted. All runs are SSTables on disk, read and written sequentially. Failure points: tiering's largest merge briefly needs double space; leveling can fall behind under heavy writes. Scaling: compaction parallelizes across key ranges and is rate-limited to protect foreground latency.

Mathematical foundations

FORMULA

Approximate amplification of compaction strategies

$$WA_{level} \approx 2 + L\,\frac{T+1}{2}, \qquad WA_{tier} \approx 2 + L, \qquad WA_{lazy} \approx 2 + (L-1) + \frac{T+1}{2}$$

$$R_{level} \approx L, \qquad R_{tier} \approx (T-1)L, \qquad R_{lazy} \approx (T-1)(L-1) + 1$$

Symbol	Meaning	Units
WA	Write amplification: bytes written to storage per byte written by the application (the 2 counts the WAL and the memtable flush)	dimensionless
R	Sorted runs a point lookup may probe in the worst case (excluding memtables)	runs
L	Number of levels (tiers), $\lceil \log_T (N/M) \rceil$	levels
T	Size ratio between levels; runs merged at once in tiering	dimensionless
N	Data size	bytes
M	Memtable flush size	bytes

The leveled term uses a simple average-case model: a byte entering a level is rewritten between about once (level nearly empty) and T times (level nearly full), so $(T+1)/2$ per level on average. The asymptotic analyses give leveling $O(T \cdot L)$ and tiering $O(L)$ write cost [Dayan et al. 2017]; real engines differ in constants (the RocksDB wiki notes leveled write amplification "often larger than 10").

WORKED EXAMPLE

Worked example — 1 TB of data, 100 MB flushes.

$N/M = 10^{12}/10^{8} = 10^{4}$.

1. **Leveled,** $T = 10$**:** $L = \log_{10} 10^{4} = 4$. $WA = 2 + 4 \times 5.5 = 24$. $R = 4$.
2. **Size-tiered,** $T = 4$**:** $L = \lceil \log_{4} 10^{4} \rceil = \lceil 6.64 \rceil = 7$. $WA = 2 + 7 = 9$. $R = 3 \times 7 = 21$.
3. **Lazy leveling,** $T = 10$**:** $L = 4$. $WA = 2 + 3 + 5.5 = 10.5$. $R = 9 \times 3 + 1 = 28$.
4. **Daily device writes** if the application writes 50 GB/day: leveled $24 \times 50 = 1200$ GB; tiered $9 \times 50 = 450$ GB; lazy $10.5 \times 50 = 525$ GB.
5. **Space amplification:** leveled $\approx 1 + 1/(T - 1) = 1.11$ (upper levels together are at most a ninth of the last); tiered, in the worst case of the whole dataset being overwritten, up to $T - 1 = 3$ copies at the largest tier plus temporary space for the merge in progress; lazy leveling ≈ 1.11, because its last level is leveled.

Interpretation. Tiering writes 2.7× less than leveling but may probe 5× more runs. Bloom filters (§13.10) make that cheap for point lookups — 21 runs at 1% false positives cost 0.21 wasted reads — but not for range scans. **Engineering implication.** Level for read-heavy, space-sensitive flash workloads; tier for write-heavy ingest. The SSD's own WAF multiplies on top: 1.2 TB/day at WAF 2 is 2.4 TB/day of flash wear (§5.6.3).

Table 13.5 — Compaction strategies compared.

	Size-tiered	Leveled	Tiered + leveled (lazy)
What it is	Merge runs of similar size	One sorted run per level	Tiers above, one run at the bottom
Write amplification	Low ($\approx L$)	High ($\approx T \cdot L/2$)	Low-medium
Point read (with Bloom filters)	Good	Best	Good
Range scan	Poor: many runs	Best	Medium
Space amplification	High; merge needs free space	~1.1	~1.1
Examples	Cassandra STCS, RocksDB universal	LevelDB, RocksDB default, Cassandra LCS	Dostoevsky; Cassandra UCS settings
Choose when	Ingest-heavy, few reads, ample disk	Read-heavy, updates, SSD cost matters	Mixed, write-heavy with point reads

Failure scenarios

Table 13.6 — LSM failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Writes stall or stop	Compaction cannot keep up; L0 file or pending-bytes limits hit	Stall counters; L0 file count	Add compaction threads; reduce ingest	Provision compaction I/O; tiered strategy for ingest
Disk full during compaction	Size-tiered merge needs free space for output	Free-space alerts during large merges	Add disk; switch strategy	Keep ~50% free with STCS; leveled compaction
Reads slow after mass delete	Tombstones not yet compacted away; scans skip millions	Tombstones scanned per query	Force compaction of the range	Range deletes; TTL-aware compaction
Deleted data reappears	Replica missed a tombstone that was purged before repair (Cassandra)	Repair or audit finds old values	Re-delete; repair	Repair within `gc_grace_seconds`

COMMON MISTAKE

"Compaction is housekeeping you can ignore." Compaction *is* the cost of an LSM tree: often 10–30 times the application's write volume, competing with reads for I/O, and its backlog decides whether writes stall. Budget compaction bandwidth, not just ingest.

13.10 Bloom filters in storage

A **Bloom filter** is a compact bit array answering "definitely not" or "probably yes" to set membership (full treatment in §17.4). LSM engines keep one per SSTable in memory, so most files lacking a key are skipped without I/O.

FORMULA

Bloom filter false-positive rate (summary of §17.4)

$$p \approx \left(1 - e^{-kn/m}\right)^{k}, \qquad k_{opt} = \frac{m}{n} \ln 2, \qquad p_{opt} \approx 0.6185^{m/n}$$

Symbol	Meaning	Units
p	Probability that an absent key gets "probably yes"	dimensionless
m	Bits in the filter	bits
n	Keys inserted	keys
k	Hash functions	count

WORKED EXAMPLE

Worked example — What filters save on a 1-billion-key LSM tree.

1. At $m/n = 10$ bits per key (the common RocksDB setting): $p = 0.6185^{10} = e^{10 \ln 0.6185} = e^{-4.805} \approx 0.0082$, with $k = 10 \ln 2 \approx 7$.
2. Memory: $10^9 \times 10/8 = 1.25$ GB.
3. A lookup for an absent key in a tree with 4 L0 files and 5 levels would otherwise read one data block per run: 9 reads. With filters: $9 \times 0.0082 \approx 0.07$ reads.
4. For a key that exists in the last level: 1 necessary read plus $8 \times 0.0082 \approx 0.07$ wasted reads.

Interpretation. Filters turn 9 I/Os into about one. **Engineering implication.** Keep filter memory (1.25 GB here) resident; evicted filter blocks make lookups regress sharply. Because ~90% of keys are in the last level, skipping its filters (RocksDB `optimize_filters_for_hits`) saves most of the memory when lookups usually succeed, and giving smaller levels more bits per key minimizes lookup cost for a fixed budget [Dayan et al. 2017].

13.11 Read, write and space amplification

13.11.1 The RUM conjecture

DEFINITION

Definition — RUM conjecture. For any access method, bounding two of **R**ead overhead, **U**pdate overhead and **M**emory (space) overhead forces the third to be worse: you can optimize for at most two [Athanassoulis et al. 2016].

A sorted array minimizes read and space overhead but makes inserts expensive; an append-only log minimizes update cost but reads slowly; a sparse hash table is fast both ways but wastes space. B-trees and LSM trees are two points in this triangle, and knobs such as fill factor, T and Bloom bits move an engine within it.

FORMULA

The three amplifications

$$RA = \frac{\text{bytes read from storage}}{\text{bytes returned}}, \qquad WA = \frac{\text{bytes written to storage}}{\text{bytes written by the application}}, \qquad SA = \frac{\text{bytes on storage}}{\text{bytes of live data}}$$

Symbol	Meaning	Units
RA	Read amplification (often counted instead as I/Os per lookup)	dimensionless
WA	Write amplification at the engine level, before the SSD's own WAF	dimensionless
SA	Space amplification	dimensionless

13.11.2 B-tree versus LSM, with numbers

WORKED EXAMPLE

Worked example — 100-byte random updates on a 2 TB table.

B-tree (InnoDB-style). Page $P = 16\,384$ bytes, doublewrite doubles each page write (§13.12), redo record about 150 bytes per update. Write amplification $WA \approx \frac{2P}{u \cdot r} + \frac{w}{r}$, where u is the number of updates a page absorbs before it is flushed, $r = 100$ and $w = 150$.

1. Uniformly random keys, data far larger than the buffer pool, so $u \approx 1$: $WA \approx 32\,768/100 + 1.5 = 329$.
2. Skewed keys so each flushed page carries $u = 20$ updates: $WA \approx 32\,768/2000 + 1.5 = 17.9$.

LSM (leveled, $T = 10$, from §13.9): $WA \approx 24$ regardless of key distribution, because compaction rewrites data in large sorted batches whatever the access pattern.

Reads. B-tree point lookup: 1 leaf read (internal pages cached). LSM: $1 + \approx 0.07$ block reads with Bloom filters; a range scan must merge every level, costing more CPU and up to one extra block per level.

Space. B-tree at 69% fill: $SA \approx 1.45$, and page compression is limited by fixed page boundaries. Leveled LSM: $SA \approx 1.11$, and SSTables compress well because blocks are packed densely.

Interpretation. For random small updates on large data the LSM tree writes about 14× less (329/24) and uses about 25% less space; with skewed updates the B-tree can write *less*. **Engineering implication.** Key locality, update size and read mix decide, not brand: measure bytes written to disk per byte written by the application on your workload.

INDUSTRY EXAMPLE

Uber's move from PostgreSQL to MySQL (2016). Uber engineers reported that a PostgreSQL row update writes a new tuple version at a new physical location, so every index must be updated, multiplying writes and the WAL shipped to replicas; InnoDB's secondary indexes point to the primary key, so updates to unindexed columns touch only the clustered index. Write amplification and replication bandwidth were among their reasons to migrate. Trade-off: InnoDB secondary lookups traverse two trees, and PostgreSQL HOT updates (§13.14.3) avoid index updates when no indexed column changes and the page has room. *Evidence: Primary* — [Klitzke 2016]

13.12 Durability: fsync, checksums and torn writes

`fsync`, its pitfalls and group commit are covered in §6.14.3. This section adds the database's defenses against storage that returns the wrong bytes.

13.12.1 Checksums

Storage can return the wrong bytes: **bit rot**, firmware bugs, **misdirected writes** and **lost writes** (acknowledged but never made). Engines checksum every layer: page headers, verified on each read (InnoDB CRC-32C by default; PostgreSQL's 16-bit page checksum, enabled by default by `initdb` from version 18); every WAL record, so recovery stops at the first torn record (as the toy LSM does); and every SSTable block. Checksums detect; repair comes from a replica, a backup or, for torn writes, the mechanisms below.

13.12.2 Torn writes

> DEFINITION
>
> **Definition — Torn write (torn page).** A page write that is interrupted by a crash so that part of the page on disk is new and part is old. It arises because the database page (8 or 16 KiB) is larger than the unit the device or file system writes atomically (commonly 4 KiB or 512 bytes).

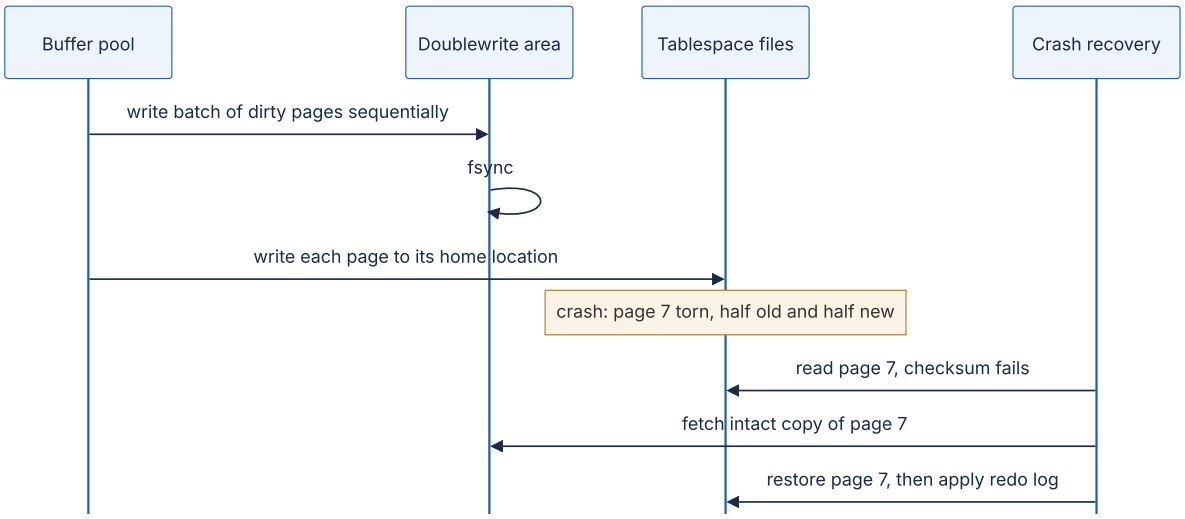
Physiological redo records assume an internally consistent page; applied to a half-old, half-new page they produce garbage, so the engine needs an intact copy from somewhere.

13.12.3 Full-page writes and the doublewrite buffer

PostgreSQL full-page writes. With `full_page_writes = on` (default), the first change to each page after a checkpoint logs the *whole* page; recovery restores the image and replays later records onto it [PostgreSQL Docs: WAL Configuration]. The cost is extra WAL after each checkpoint (§13.5.3).

InnoDB doublewrite buffer. Dirty pages are first written sequentially to a **doublewrite** area and flushed, then written in place; a page torn in place is restored from its intact copy. Because the extra writes are sequential batches with one flush, the cost is well under 2× [MySQL Docs: Doublewrite Buffer].

Figure 13.10 — InnoDB doublewrite protecting against a torn page.



Reading the diagram. Every arrow is synchronous. An intact copy of each page is durable in the doublewrite area (separate files since MySQL 8.0.20) before any home location is touched. At recovery, a checksum failure flags the torn page, its copy replaces it, and redo proceeds. A crash before the doublewrite flush leaves the home page intact, so redo alone suffices. The doublewrite area costs bandwidth, not IOPS. Failure point: disabling it on storage without atomic page writes silently reopens the torn-page window.

Both can be disabled when the layer below writes pages atomically (ZFS, some SSDs and cloud volumes). LSM trees and Bitcask never overwrite: a torn SSTable never entered the manifest, and a torn log tail fails its CRC.

Table 13.7 — Durability mechanisms in storage engines.

Threat	Mechanism	Cost	Examples
Lost committed transaction	WAL + `fsync` at commit (§6.14.3)	One flush per commit group	All engines
Torn data page	Full-page images in WAL	Extra WAL after checkpoints	PostgreSQL
Torn data page	Doublewrite buffer	Second sequential page write	InnoDB
Torn data page	Never overwrite in place	Path copying or compaction	LMDB, ZFS, LSM trees
Silent corruption	Page, record and block checksums	CPU per read; detection only	InnoDB, PostgreSQL, RocksDB
Corruption detected	Re-copy from replica or backup	Operational	All

13.13 Columnar storage

13.13.1 Rows versus columns

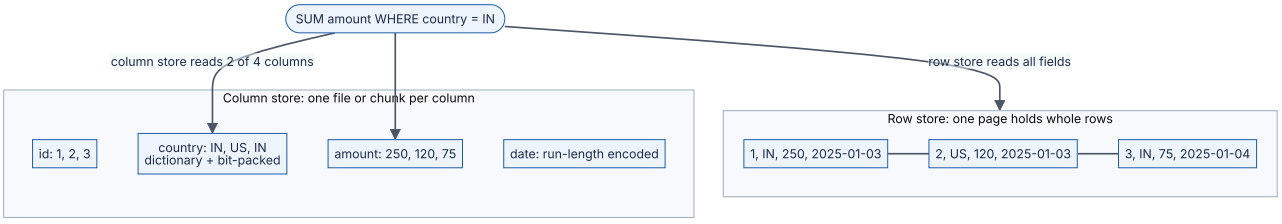
DEFINITION

Definition — Columnar storage (column store). A layout that stores each column of a table contiguously — all values of `price`, then all values of `country` — instead of storing each row's fields together. Column stores are optimized for analytical queries that read a few columns of many rows [Abadi et al. 2013].

First-principles explanation

A school keeps one card per student. To find one address, cards are perfect; to average 10,000 ages you handle 10,000 cards and ignore most of each, whereas one sheet of ages would do. Transaction processing (OLTP) touches whole rows by key: rows win. Analytics (OLAP) aggregates a few columns over millions of rows: columns win.

Figure 13.11 — Row-oriented versus column-oriented layout of the same table.



Reading the diagram. The same rows stored two ways; arrows are synchronous reads. The row store must read every field of every scanned row; the column store reads only `country` and `amount`, each encoded to suit its values. Failure point: a single-row insert or update touches every column file, so column stores buffer writes in a small row-oriented store and convert in batches. Scaling: columns are scanned in parallel or skipped entirely.

13.13.2 Encoding, vectorized execution and Parquet

Values in one column share a type and often repeat, so they compress far better than rows. Three encodings do most of the work:

- **Dictionary encoding** replaces each value with a small integer code into a dictionary of distinct values.
- **Bit-packing** stores codes in exactly $\lceil \log_2 d \rceil$ bits for d distinct values, with no byte padding.
- **Run-length encoding (RLE)** stores `(value, count)` pairs, ideal for sorted or clustered columns.

Queries can often run on encoded data directly: a filter `country = 'IN'` becomes “code = 3”, and on RLE data a count is a sum of run lengths.

```
        """Dictionary encoding, run-length encoding and bit-packing for one column."""
        import math
        import random


        def dictionary_encode(values):
            dictionary = sorted(set(values))
            code = {v: i for i, v in enumerate(dictionary)}
            return dictionary, [code[v] for v in values]


        def run_length_encode(codes):
            runs = []                                   # list of [value, run_length]
            for c in codes:
                if runs and runs[-1][0] == c:
                    runs[-1][1] += 1
                else:
                    runs.append([c, 1])
            return runs


        def bit_pack(codes, width):
            packed = 0
            for i, c in enumerate(codes):
                packed |= c << (i * width)              # width bits per value, no byte
padding
            return packed.to_bytes(math.ceil(len(codes) * width / 8), "little")


        def bit_unpack(data, width, n):
            packed, mask = int.from_bytes(data, "little"), (1 << width) - 1
            return [(packed >> (i * width)) & mask for i in range(n)]


        if __name__ == "__main__":
            rng = random.Random(7)
            countries = ["BR", "DE", "FR", "IN", "JP", "US"]
            # 100,000 rows sorted by country, as a column store would often keep them
            column = sorted(rng.choice(countries) for _ in range(100_000))
            raw = sum(len(v.encode()) for v in column)

            dictionary, codes = dictionary_encode(column)
            width = max(1, math.ceil(math.log2(len(dictionary))))
            packed = bit_pack(codes, width)
            runs = run_length_encode(codes)
            assert bit_unpack(packed, width, len(codes)) == codes

            print("raw bytes:", raw)
            print("dictionary:", dictionary, "-> width", width, "bits per value")
            print("bit-packed bytes:", len(packed), " ratio %.1fx" % (raw /
len(packed)))
            print("RLE runs:", len(runs), "-> about", len(runs) * 5, "bytes (1-byte code
+ 4-byte length)")
            # a filter can run on codes: count rows WHERE country = 'IN' without decod-
ing strings
```

```
target = dictionary.index("IN")
print("rows with IN:", sum(length for c, length in runs if c == target))
```

Output:

```
raw bytes: 200000
dictionary: ['BR', 'DE', 'FR', 'IN', 'JP', 'US'] -> width 3 bits per value
bit-packed bytes: 37500  ratio 5.3x
RLE runs: 6 -> about 30 bytes (1-byte code + 4-byte length)
rows with IN: 16536
```

What to notice:

- Six distinct values need $\lceil \log_2 6 \rceil = 3$ bits, so 100,000 values pack into $100\,000 \times 3/8 = 37\,500$ bytes.
- Sorted, the column collapses to six runs; the `WHERE country = 'IN'` count is computed from runs without decoding any string. Sort order is therefore a physical design decision in column stores.

FORMULA

Size of a dictionary-encoded, bit-packed column

$$S \approx \frac{N \lceil \log_2 d \rceil}{8} + d \cdot \bar{v}$$

Symbol	Meaning	Units
S	Encoded column size	bytes
N	Number of rows	rows
d	Number of distinct values	values
$\bar{v}$	Average size of one dictionary entry	bytes

WORKED EXAMPLE

Worked example — Scanning one column of a billion-row fact table.

A table has $N = 10^9$ rows and 20 columns averaging 8 bytes: 1.6×10^{11} bytes $= 160$ GB as rows. A query sums `amount` (8 bytes, uncompressed here) where `country` has $d = 200$ values of about 10 bytes.

1. Row store: reads all 160 GB.
2. Column store, `country`: $\lceil \log_2 200 \rceil = 8$ bits; $S = 10^9 \times 8/8 + 200 \times 10 = 1.0$ GB (the 2 KB dictionary is negligible).
3. Column store, `amount`: $10^9 \times 8 = 8$ GB. Total: 9 GB.
4. At 2 GB/s of scan bandwidth: row store $160/2 = 80$ s; column store $9/2 = 4.5$ s, about 18× faster before any further compression of `amount` or skipping of blocks.

Interpretation. Reading only needed columns, tightly encoded, cuts bytes scanned by an order of magnitude. **Engineering implication.** Use column formats for analytics, with per-block min/max statistics (**zone maps**) so filters skip blocks; the bottleneck then moves to CPU.

That CPU bottleneck is attacked with **vectorized execution**: operators process batches of about a thousand values from one column in tight loops, instead of one row at a time through a chain of virtual function calls. The loops stay in the CPU cache, avoid branch mispredictions and use SIMD instructions (§5.11) [Boncz et al. 2005]. **Late materialization** keeps data as column positions as long as possible and assembles full rows only for the final result.

Apache Parquet, the most widely used open columnar file format, applies all of this to files: a file is divided into **row groups**, each row group stores one **column chunk** per column, chunks are divided into pages with dictionary, RLE and bit-packed encodings, and a footer holds the schema and per-chunk statistics used to skip data [Apache Parquet Docs]. Parquet, ORC, and lakehouse table formats built on them are treated in Chapter 61; distributed scans over them in Chapter 29.

COMMON MISTAKE

"A column store is just a row store with column indexes." A column store changes what is read, how it is compressed and how operators execute. Single-row writes and lookups are expensive in it, so OLTP stays in row stores, with changes streamed to a column store for analytics (§32.6).

13.14 Case studies: InnoDB, RocksDB and the PostgreSQL heap

13.14.1 InnoDB: a clustered B+ tree with redo, undo and doublewrite

InnoDB, MySQL's default engine, stores each table as a **clustered index**: a B+ tree keyed by the primary key whose leaf pages (16 KiB) hold the full rows. Secondary indexes store the primary-key value, not a physical address, so a secondary lookup traverses two trees, but rows can move between pages during splits without touching secondary indexes. The buffer pool uses midpoint-insertion LRU (§13.4.2) and, on Linux from MySQL 8.4, `O_DIRECT`. Durability combines the redo log, undo logs for rollback and MVCC reads (§13.5.4), and the doublewrite buffer (§13.12.3). Choose it for OLTP workloads accessed mostly by primary key, with compact, roughly increasing primary keys: random keys such as UUIDv4 cause splits all over the tree and poor page fill. *Evidence: Primary* — [MySQL Docs: InnoDB Architecture].

13.14.2 RocksDB: an embeddable LSM engine

RocksDB began at Facebook in 2012 as a fork of Google's LevelDB, optimized for flash and many cores. It is a library, not a server: applications embed it and get the full machinery of §13.8–§13.10 — WAL, skip-list memtables, block-based SSTables with Bloom (or the more compact Ribbon) filters, a block cache, leveled compaction by default with universal (tiered) and FIFO alternatives, and **column families** that share a WAL but have separate memtables and SSTables. It is the storage layer of many distributed databases and stream processors, including TiKV and Kafka Streams state stores.

> INDUSTRY EXAMPLE
>
> **MyRocks at Facebook.** Facebook built MyRocks, a MySQL storage engine on RocksDB, and migrated its main user database from compressed InnoDB to it. The motivation was space and write amplification on flash: B-tree pages were partly empty and compressed poorly at page granularity, and random page writes wore SSDs, whereas leveled LSM compaction stores data densely and writes sequentially. The authors report substantial reductions in storage and write volume, and describe the engineering needed to make LSM reads, range scans and tombstone handling competitive. *Evidence: Primary* — [Matsunobu et al. 2020; Dong et al. 2017]

13.14.3 PostgreSQL heap: append-friendly pages with MVCC versions

PostgreSQL stores a table as an unordered **heap** of 8 KiB slotted pages (§13.3). A row version (tuple) carries `xmin` / `xmax`, the IDs of the transactions that created and deleted it (visibility rules: §11.14.2). An `UPDATE` never overwrites: it writes a new tuple version — on the same page if there is room, else elsewhere — and marks the old one deleted. All in-

dexes are secondary and point to a tuple's physical location `(block, slot)`, the `ctid`. Two optimizations limit the cost. A **heap-only tuple (HOT)** update, possible when no indexed column changes and the new version fits on the same page, links the versions within the page and adds no index entries; a lower `fillfactor` leaves room for it. **`VACUUM`** later removes dead versions, records reusable space in the **free space map**, and marks all-visible pages in the **visibility map**, which enables index-only scans. Large values are moved out of line by TOAST. Recovery is redo-only from the WAL with full-page writes (§13.5, §13.12.3). *Evidence: Primary* — [PostgreSQL Docs: Database Physical Storage; Routine Vacuuming].

Table 13.8 — B-tree versus LSM storage engines.

	B-tree (in place)	LSM tree
What it is	Pages updated in place in a balanced tree	Memtable flushed to immutable sorted runs, merged by compaction
How it works	WAL + buffer pool; splits on overflow	WAL + memtable; flush; tiered or leveled compaction
Advantages	One I/O point reads; fast range scans; mature locking	Sequential writes; low WA for random updates; dense, compressible storage
Disadvantages	Random writes; page-sized WA; fragmentation; torn-page protection	Many runs per read; compaction debt and write stalls; tombstones
Performance	Read-optimized; writes depend on key locality	Write-optimized; point reads near B-tree with Bloom filters; scans slower
Operational complexity	Fill factor, checkpoints, `VACUUM` (PostgreSQL)	Compaction strategy, L0 limits, filters, rate limits: many knobs
Cost	More space (SA ≈ 1.45); more flash wear for random updates	Less space (SA ≈ 1.1); compaction CPU and I/O
Use cases	OLTP, relational databases, secondary-index-heavy workloads	Ingest-heavy, key-value, time series, wide-column stores
Choose when	Read-heavy or locality-rich updates; latency predictability	Write-heavy or random updates on large data; space-sensitive SSD fleets

13.15 Summary and revision notes

SUMMARY

- Engines read, write, cache and log in **pages** (4–16 KiB). **Slotted pages** give records stable `(page, slot)` IDs while allowing variable lengths and in-page compaction; per-row and per-page overhead alone can make a table ~1.4× its raw data.
- A **buffer pool** maps page IDs to frames with pin counts and dirty flags. Clock and LRU-K approximate LRU cheaply and resist scans; `O_DIRECT` avoids double buffering. The five-minute rule says that on NVMe only pages re-referenced every ~30 s are worth caching on cost.
- **WAL** turns random page writes into sequential log appends. The WAL rule ($\text{pageLSN} \le \text{flushedLSN}$) and commit rule enable steal/no-force. **ARIES** recovers with analysis, redo (repeat history) and undo (with CLRs). Checkpoint interval trades recovery time (W_{redo}/b_{replay}) against full-page-image volume.
- **B-trees** of height 3–4 index billions of rows; lookups cost about one I/O when internal pages are cached. Splits are logged atomically; latch crabbing and B-link trees provide concurrency; **COW B-trees** (LMDB) need no WAL but copy the root-to-leaf path on each commit.
- **Log-structured** storage appends everything. **Bitcask** keeps every key in RAM for one-seek reads; **LSM trees** add sorted memtables, SSTables with sparse indexes and Bloom filters, and compaction.
- **Compaction**: leveled $WA \approx 2 + L(T+1)/2$, few runs, $SA \approx 1.1$; tiered $WA \approx 2 + L$, many runs, more space; hybrids sit between. Compaction bandwidth must be capacity-planned.
- **Bloom filters** at 10 bits/key give ~0.8% false positives and reduce absent-key lookups from one read per run to a few hundredths of a read.
- **RUM**: you can bound only two of read, update and space overhead. For random small updates, LSM WA (~24) can be >10× lower than a B-tree's (~329); for skewed updates the B-tree can win.
- **Durability** needs `fsync` plus checksums (detection) plus torn-page protection (full-page writes, doublewrite, or never overwriting).
- **Column stores** read only needed columns, encode them (dictionary, bit-packing, RLE) and execute vectorized; Parquet brings this to files.

13.16 Knowledge checkpoint

1. Why does a slotted page store slot numbers in index entries rather than byte offsets?

2. Compute the rows per 8 KiB page for 50-byte rows in PostgreSQL (24-byte tuple header, 8-byte alignment, 4-byte slots, 24-byte page header).
3. What are a pin count and a dirty flag for, and what happens if a thread forgets to unpin?
4. Explain sequential flooding and two mechanisms that prevent it.
5. State the WAL rule and the commit rule.
6. What do steal and no-force mean, and which one requires undo?
7. Why does ARIES redo the changes of loser transactions before undoing them?
8. What is a CLR and why does it make recovery safe against crashes during recovery?
9. Calculate the height of a B+ tree with 500 million rows, 100 rows per leaf and fan-out 500.
10. What limits Bitcask's scalability?
11. List the steps of an LSM point lookup.
12. For $N/M = 10^4$, compute the write amplification of leveled ($T = 10$) and tiered ($T = 4$) compaction with the chapter's model.
13. What is a torn write and how do PostgreSQL and InnoDB each defend against it?
14. How many bits per value does bit-packing need for a column with 1,000 distinct values?

Answers

1. So records can move within the page (updates, compaction) without changing their identifier or any index entry.
2. Tuple $24 + 50 = 74$, aligned to 80; with slot 84 bytes. $\lfloor 8168/84 \rfloor = 97$ rows.
3. Pin count prevents eviction of a page in use; dirty flag means it must be written back before reuse. A leaked pin keeps the frame unevictable forever, shrinking the pool.
4. A large scan touches each page once and evicts the hot set under LRU. Prevent with LRU-K/2Q/midpoint insertion (one-touch pages never reach the protected list) or ring buffers for bulk scans.
5. WAL rule: log records up to a page's LSN must be durable before the page is written. Commit rule: a commit is acknowledged only after its commit record is durable.
6. Steal: uncommitted changes may be written to disk (requires undo). No-force: committed changes need not be written at commit (requires redo).
7. Repeating history restores the exact pre-crash page state, so logical undo records apply correctly and fine-grained locking is supported.
8. A compensation log record logs each undo action with `undoNextLSN`; after a crash during recovery, CLRs are redone and undo resumes where it stopped, never undoing twice.
9. Leaves $= 5 \times 10^6$; $\log_{500} 5 \times 10^6 = 15.42/6.21 = 2.48 \rightarrow 3$ internal levels; $h = 4$.
10. Every key must fit in the in-memory keydir; no range queries.
11. Active and immutable memtables; each L0 file newest first; one file per deeper level; within each, Bloom filter, index block, one data block; first version found wins;

tombstone means deleted.

12. Leveled: $L = 4$, $WA = 2 + 4 \times 5.5 = 24$. Tiered: $L = 7$, $WA = 9$.
13. A crash leaves a page part old, part new. PostgreSQL logs a full page image on first modification after a checkpoint; InnoDB writes pages to a doublewrite area and flushes before writing in place.
14. $\lceil \log_2 1000 \rceil = 10$ bits.

13.17 Interview questions

Basic

Q1. What is a write-ahead log and why does it make databases fast as well as safe? Answer. An append-only log of changes, made durable before the changed pages. Commits need one sequential append and `fsync` (shared across concurrent commits by group commit) instead of random page writes; pages are flushed later in batches. After a crash, redo restores committed changes and undo removes uncommitted ones.

Q2. When would you choose an LSM-based store over a B-tree-based one? Answer. For write-heavy workloads with random keys on large data — event ingestion, time series, key-value at scale — and when space on SSD matters: LSM writes sequentially with lower write and space amplification. Prefer a B-tree for read-heavy OLTP, range scans and predictable latency. Then check compaction bandwidth and read amplification.

Intermediate

Q3. Your RocksDB-backed service shows periodic write latency spikes of several seconds. What is happening and what do you do? Answer. Most likely write stalls: compaction falls behind, L0 file count or pending compaction bytes exceed thresholds, and RocksDB slows or stops writes. Confirm with stall statistics and L0 counts. Fixes: more compaction threads and I/O budget, larger memtables, a rate limiter to smooth compaction, faster disks, or a tiered strategy if writes dominate; reduce ingest bursts with backpressure (§22.10).

Q4. Why do many databases bypass the operating system page cache, and why does PostgreSQL traditionally not? Answer. Bypassing with `O_DIRECT` avoids double caching, gives the engine control over eviction, read-ahead and write ordering, and makes memory use predictable — InnoDB's design. PostgreSQL uses buffered I/O with a moderate `shared_buffers`, letting the page cache act as a second-level cache; this is simpler and portable but double-caches some pages and cedes writeback control to the kernel. Either way durability still requires `fsync`.

Advanced

Q5. Estimate and compare write amplification for a B-tree and a leveled LSM tree on 100-byte random updates, and say when the answer flips. Answer. B-tree: each flushed 16 KiB page, doubled by doublewrite, carries about one update when keys are random and data exceeds RAM: $32\,768/100 + 1.5 \approx 329$. Leveled LSM with $T = 10$, $L = 4$: ≈ 24. The answer flips with locality: if each page absorbs 20+ updates before flushing, the B-tree drops below 20. Add the SSD's own WAF on top and compare `bytes written / bytes ingested` in production.

Q6. Walk through crash recovery in an ARIES-style engine after a crash during a long transaction. Answer. Analysis from the last checkpoint rebuilds the dirty page table and finds the long transaction as a loser. Redo from the minimum `recLSN` reapplies every change whose LSN exceeds the page's `pageLSN`, including the loser's. Undo follows the loser's `prevLSN` chain backward, writing CLRs; a crash during undo is handled by redoing CLRs and continuing from `undoNextLSN`. Undo time grows with the transaction's size, which is why long transactions lengthen recovery and should be bounded.

13.18 Practical exercises

1. **Extend the toy LSM.** Replace full-merge compaction in `toy_lsm.py` with leveled compaction (L0 plus levels of non-overlapping files with $T = 4$). Measure write amplification as data grows and compare with $2 + L(T + 1)/2$.
2. **Measure Bloom filters.** Count SSTable reads per `get` with and without Bloom filters for absent keys in the toy LSM; vary bits per key from 2 to 16 and compare with $0.6185^{m/n}$.
3. **Observe PostgreSQL pages.** With the `pageinspect` extension, insert rows into a table, run `UPDATE`s, and inspect `page_header` and `heap_page_items` before and after `VACUUM`; identify HOT chains and dead line pointers.
4. **Checkpoint trade-off.** On a test PostgreSQL instance under `pgbench`, measure WAL volume per minute with `checkpoint_timeout` at 1 and 15 minutes, then crash it (`kill -9`) and time recovery. Compare with §13.5.3.

13.19 Design challenges

1. **Metrics ingestion store.** Design the storage engine configuration for 2 million data points per second (16 bytes each), 30-day retention and queries over the last hour. Choose B-tree, leveled, tiered or time-window compaction; estimate write amplification, daily device writes, disk size and SSD lifetime (§5.6.3); explain how expired data is deleted cheaply.

2. **User-profile database.** 500 million profiles of 2 KB, 50,000 reads/s by user ID, 5,000 updates/s of single fields, p99 read latency under 5 ms. Compare InnoDB, PostgreSQL and RocksDB; size the buffer pool or block cache with the five-minute rule; estimate write and space amplification; describe torn-page and checksum protection.

13.20 References

1. Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H., and Schwarz, P. “ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging.” *ACM Transactions on Database Systems* 17(1), 1992.
2. O’Neil, P., Cheng, E., Gawlick, D., and O’Neil, E. “The Log-Structured Merge-Tree (LSM-Tree).” *Acta Informatica* 33(4), 1996.
3. Athanassoulis, M., Kester, M. S., Maas, L. M., Stoica, R., Idreos, S., Ailamaki, A., and Callaghan, M. “Designing Access Methods: The RUM Conjecture.” *EDBT*, 2016.
4. RocksDB Wiki. *Leveled Compaction*; *Universal Compaction*; *Direct IO*; *RocksDB Tuning Guide*. https://github.com/facebook/rocksdb/wiki
5. Sheehy, J., and Smith, D. *Bitcask: A Log-Structured Hash Table for Fast Key/Value Data*. Basho Technologies, 2010.
6. Abadi, D., Boncz, P., Harizopoulos, S., Idreos, S., and Madden, S. “The Design and Implementation of Modern Column-Oriented Database Systems.” *Foundations and Trends in Databases* 5(3), 2013.
7. Petrov, A. *Database Internals*. O’Reilly, 2019.
8. Kleppmann, M. *Designing Data-Intensive Applications*, chapter 3 “Storage and Retrieval”. O’Reilly, 2017.
9. PostgreSQL Global Development Group. *PostgreSQL Documentation*: Database Page Layout; Database Physical Storage; Reliability and the Write-Ahead Log; WAL Configuration; Resource Consumption; Routine Vacuuming. https://www.postgresql.org/docs/current/
10. PostgreSQL source tree. `src/backend/storage/buffer/README`; `src/backend/access/nbtree/README`. https://git.postgresql.org/
11. Oracle. *MySQL 8.4 Reference Manual*: InnoDB Architecture; Making the Buffer Pool Scan Resistant; InnoDB Startup Options; Redo Log; Undo Logs; Doublewrite Buffer; Configuring the Merge Threshold for Index Pages. https://dev.mysql.com/doc/refman/8.4/en/
12. O’Neil, E. J., O’Neil, P. E., and Weikum, G. “The LRU-K Page Replacement Algorithm for Database Disk Buffering.” *SIGMOD*, 1993.
13. Johnson, T., and Shasha, D. “2Q: A Low Overhead High Performance Buffer Management Replacement Algorithm.” *VLDB*, 1994.

14. Gray, J., and Putzolu, F. “The 5 Minute Rule for Trading Memory for Disc Accesses and the 10 Byte Rule for Trading Memory for CPU Time.” *SIGMOD*, 1987.
15. Gray, J., and Graefe, G. “The Five-Minute Rule Ten Years Later, and Other Computer Storage Rules of Thumb.” *SIGMOD Record* 26(4), 1997.
16. Crotty, A., Leis, V., and Pavlo, A. “Are You Sure You Want to Use MMAP in Your Database Management System?” *CIDR*, 2022.
17. Härder, T., and Reuter, A. “Principles of Transaction-Oriented Database Recovery.” *ACM Computing Surveys* 15(4), 1983.
18. Bayer, R., and Schkolnick, M. “Concurrency of Operations on B-Trees.” *Acta Informatica* 9(1), 1977.
19. Lehman, P. L., and Yao, S. B. “Efficient Locking for Concurrent Operations on B-Trees.” *ACM Transactions on Database Systems* 6(4), 1981.
20. Yao, A. C. “On Random 2–3 Trees.” *Acta Informatica* 9(2), 1978.
21. Rodeh, O. “B-trees, Shadowing, and Clones.” *ACM Transactions on Storage* 3(4), 2008.
22. Symas Corporation. *LMDB: Lightning Memory-Mapped Database Manager documentation*. http://www.lmdb.tech/doc/
23. Rosenblum, M., and Ousterhout, J. K. “The Design and Implementation of a Log-Structured File System.” *ACM Transactions on Computer Systems* 10(1), 1992.
24. Chang, F., et al. “Bigtable: A Distributed Storage System for Structured Data.” *OSDI*, 2006.
25. Dayan, N., Athanassoulis, M., and Idreos, S. “Monkey: Optimal Navigable Key-Value Store.” *SIGMOD*, 2017.
26. Dayan, N., and Idreos, S. “Dostoevsky: Better Space-Time Trade-Offs for LSM-Tree Based Key-Value Stores via Adaptive Removal of Superfluous Merging.” *SIGMOD*, 2018.
27. Dong, S., Callaghan, M., Galanis, L., Borthakur, D., Savor, T., and Strum, M. “Optimizing Space Amplification in RocksDB.” *CIDR*, 2017.
28. Matsunobu, Y., Dong, S., and Lee, H. “MyRocks: LSM-Tree Database Storage Engine Serving Facebook’s Social Graph.” *PVLDB* 13(12), 2020.
29. Klitzke, E. “Why Uber Engineering Switched from Postgres to MySQL.” Uber Engineering Blog, 2016. https://www.uber.com/blog/postgres-to-mysql-migration/
30. Boncz, P., Zukowski, M., and Nes, N. “MonetDB/X100: Hyper-Pipelining Query Execution.” *CIDR*, 2005.
31. Apache Software Foundation. *Apache Parquet Documentation: File Format; Encodings*. https://parquet.apache.org/docs/

13.21 Open verification items

- MySQL 8.4 Linux default `innodb_flush_method = O_DIRECT` (confirmed via secondary sources and the 8.4 manual summary; primary page not fetched).

- PostgreSQL `debug_io_direct` introduced in version 16 and the version-18 asynchronous I/O subsystem: wording and defaults.
- PostgreSQL clock-sweep usage-count cap of 5 and ring-buffer behavior (from the buffer README; not re-read for this edition).
- Exact storage and write reductions reported in [Matsunobu et al. 2020]; deliberately not quoted.
- Cassandra guidance to keep ~50% free disk for STCS; Cassandra 5.0 UCS description.
- Uber blog URL (original path may have changed).
- The compaction formulas are a simplified average-case model, not taken verbatim from one source.

Chapter 14 — Database Scaling: Replication, Partitioning and Sharding

> **Level:** 2–3 · **Track:** A (Traditional) · **Estimated reading time:** 6 h

A single database server is the most comfortable place your data will ever live: every row in one place, every transaction consistent, every join just a join. Three limits end that comfort. Reads outgrow one machine, writes outgrow one machine, or the data outgrows its disks. Chapter 2 sketched this for QuickBite: one PostgreSQL primary handles 60 orders per second easily, 600 needs read replicas, and 6,000 probably needs partitioning (§2.10).

Replication keeps copies of the same data on several nodes. It adds read capacity and survives failures but adds no write capacity or storage. **Partitioning** (also called **sharding**) splits the data into subsets on different nodes. It adds write capacity and storage but makes queries that span subsets harder. Real systems do both: every partition is replicated. Consistency guarantees are treated in depth in Chapter 25, leaderless replication and conflict resolution in Chapter 27, and distributed transactions in Chapter 28.

14.1 Learning objectives

After this chapter you should be able to:

- Draw a primary–replica architecture and compare synchronous, asynchronous and semi-synchronous replication by latency, durability and availability.
- Explain how replication lag causes read-your-writes and monotonic-read anomalies, and design routing that prevents them.
- Distinguish statement-based, row-based, physical and logical replication, and say when multi-primary replication is worth its conflicts.
- Compare range, hash, list and directory partitioning, choose a shard key, and compute the number of shards from data size, write rate and per-node limits.
- Build a consistent-hash ring with virtual nodes, derive how load imbalance shrinks with the number of virtual nodes V, and compare it with jump and rendezvous hashing.
- Plan an online resharding, mitigate hot partitions and choose local or global secondary indexes.
- Explain the cost of cross-shard queries and transactions, and place PgBouncer, ProxySQL and Vitess in an architecture.

14.2 Prerequisites

- Transactions, ACID and isolation levels — §11.11–§11.13 (replication and sharding change what a transaction can guarantee).
- Indexes and B+ trees — §11.7–§11.8 (secondary indexes become a distributed problem).
- Write-ahead logging — §13.5 (physical replication ships the WAL).
- Little's Law and connection pools — §4.11 and the pool-sizing examples in Chapter 4 (used again in §14.13).
- Hash functions — §17.3 at an intuitive level: a hash maps any key to a well-mixed number.

14.3 Read replicas and primary–replica architecture

14.3.1 The primary–replica model

> DEFINITION
>
> **Definition — Replication.** Keeping a copy of the same data on more than one node. Each copy is a **replica**. In **primary-replica replication**, exactly one node, the **primary** (called the "master" in older documentation), accepts writes and streams its changes to the **replicas**, which apply them in the same order and serve reads.

First-principles explanation

Think of a newspaper. One newsroom writes the stories, and printing plants in many cities print identical copies. Readers get the paper quickly from a local plant, but every correction must go through the newsroom, every plant must print every story, and a distant reader may hold yesterday's edition for a while. The newsroom is the primary, the plants are replicas and the delay is **replication lag**. Reads scale with the number of plants. Writes do not scale at all.

Why does this exist?

Most applications read far more than they write, often 10 to 100 reads per write. Copying the data to more nodes and spreading the reads is the cheapest way to multiply read capacity, with no schema change. Replicas are also a **hot standby**: a replica that already holds nearly all the data can be promoted in seconds if the primary dies.

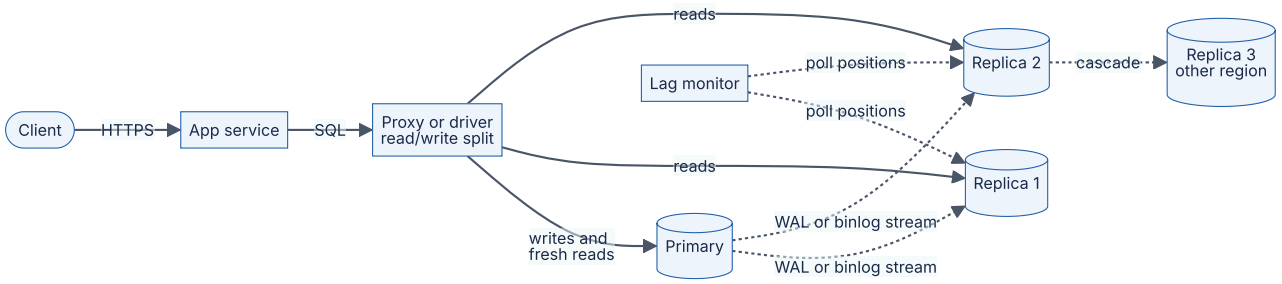
How it works internally

1. A client sends a write (`INSERT` , `UPDATE` , `DELETE`) to the primary.

2. The primary records the change in its **replication log**: the write-ahead log (WAL, §13.5) in PostgreSQL, the **binary log** (binlog) in MySQL.
3. It commits, waiting first for replica acknowledgments if the replication mode requires it (§14.4).
4. A sender process streams new log records to each replica over a long-lived TCP connection. Each replica logs them and **applies** them in order.
5. Read-only queries go to replicas, routed by the application, a driver or a proxy (§14.13).
6. Each replica tracks its applied position: a log sequence number (LSN) in PostgreSQL or a global transaction identifier (GTID) set in MySQL. The gap to the primary's position is the replication lag.

Architecture diagram

Figure 14.1 — Primary–replica architecture with read/write splitting.



Reading the diagram. A proxy or driver sends writes and must-be-fresh reads to the primary and other reads to replicas, synchronously. Replication (dotted) is asynchronous by default, so replicas lag. Replica 3 is fed by Replica 2 (**cascading replication**), sparing the primary's fan-out to distant regions. Each node stores a full copy. The lag monitor removes lagging replicas from rotation. Failure points: the primary, until a replica is promoted (§27.11), and the intermediate replica for everything below it.

Mathematical foundations

Every write is applied on every node, so the write stream consumes part of each replica's capacity.

FORMULA

Read capacity of a replicated group

$$R_{\max} = n \cdot (u \cdot c - w) \qquad \text{valid only while } w < u \cdot c$$

Symbol	Meaning	Units
$R_{\max}$	Maximum sustainable read rate across the group	reads/s
n	Number of nodes serving reads (replicas, plus the primary if it serves reads)	nodes
c	Capacity of one node, expressed in read-equivalent operations	ops/s
u	Target utilization, leaving headroom for spikes and failover	dimensionless (0–1)
w	Cost of applying the full write stream on one node, in read-equivalent operations	ops/s

WORKED EXAMPLE

Worked example — When do more replicas stop helping? Assume one node handles $c = 20,000$ read-equivalent operations per second, and you run it at $u = 0.6$, so 12,000 ops/s of budget. Applying one write costs about as much as 3 reads.

1. At 1,000 writes/s, $w = 3,000$. Each replica has $12,000 - 3,000 = 9,000$ ops/s for reads. Four replicas give $R_{\max} = 4 \times 9,000 = 36,000$ reads/s.
2. At 3,000 writes/s, $w = 9,000$. Each replica has 3,000 ops/s left. Four replicas give only 12,000 reads/s. Even 12 replicas give 36,000.
3. At 4,000 writes/s, $w = 12,000 = u \cdot c$. Replicas have no read budget left at all, and they can barely keep up with the write stream.

Interpretation. Replicas multiply *spare* capacity, and the write stream eats into every node's spare capacity equally. **Engineering implication.** Read replicas are the right first move when writes are a small fraction of load. Once applying writes uses about half a node's budget, partition (§14.6).

Real-world industry applications

INDUSTRY EXAMPLE

PostgreSQL and MySQL. PostgreSQL streams WAL to **hot standby** servers, which serve read-only queries while applying changes. MySQL replicas pull binlog events and apply them with one or more parallel applier threads. Managed services such as Amazon RDS expose the same mechanism as "read replicas". *Evidence: Primary* — [PostgreSQL Docs: High Availability], [MySQL Docs: Replication]

Advantages and disadvantages

Replicas are simple to add, give a warm standby and can isolate heavy analytical queries. But they add no write or storage capacity, each one is a full copy, reads may be stale, and asynchronous failover can lose the last few transactions (§14.4).

Failure scenarios

Table 14.1 — Failure scenarios for read replicas.

What goes wrong	Why	Detection	Recovery	Prevention
Users see stale data	Replica applies changes seconds behind	Lag metric; user reports "my edit vanished"	Route that user's reads to primary	Read-your-writes routing (§14.4.4)
Replica falls ever further behind	Single-threaded apply, or a long query blocks apply	Lag grows steadily	Kill blocking queries; enable parallel apply	Parallel apply; cap query time on replicas
Data lost on failover	Async replica promoted before receiving last commits	Gap in GTIDs or LSNs after promotion	Reconcile from old primary if it is reachable	Semi-sync or sync replica as failover target

Common misconceptions

COMMON MISTAKE

Mistake — "Add replicas until the database is fast enough." Replicas help only reads that tolerate staleness. If the primary is busy with writes or must-be-fresh reads, more replicas change nothing.

14.4 Synchronous, asynchronous and semi-synchronous replication

14.4.1 Three modes of acknowledgment

DEFINITION

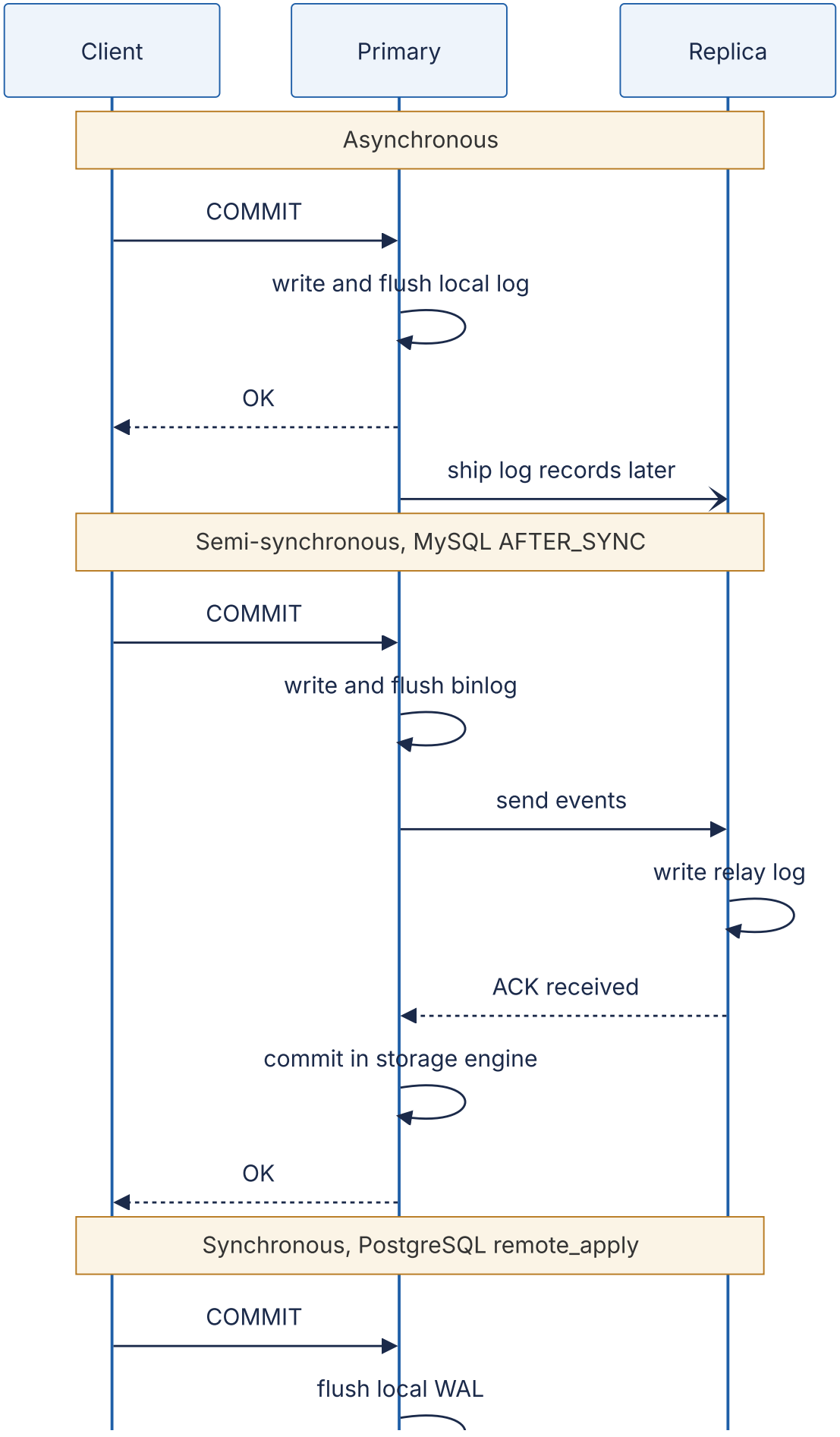
Definition — Replication modes. In **synchronous replication**, the primary reports a commit only after at least one designated replica has durably received (or applied) it. In **asynchronous replication**, it reports success after its local commit and ships the change afterwards. **Semi-synchronous replication**, MySQL's middle ground, waits until a replica has *received and logged* the change and falls back to asynchronous mode after a timeout.

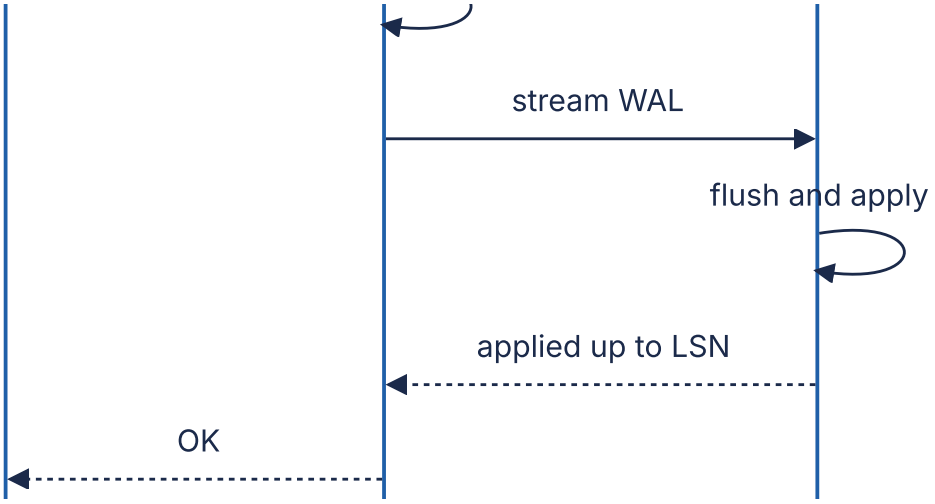
First-principles explanation

Imagine posting an important letter. Asynchronous is dropping it in a mailbox and walking away: fast, but if the post office burns down tonight the letter is lost. Synchronous is waiting for a signed receipt from the recipient: slow, and you are stuck if the courier is. Semi-synchronous is waiting only until the courier confirms the letter is in the van, and giving up after ten seconds. The trade-off is **durability on failover versus commit latency and availability**.

How it works internally

Figure 14.2 — Commit paths under asynchronous, semi-synchronous and synchronous replication.


Client
Primary
Replica
Asynchronous
COMMIT
write and flush local log
OK
ship log records later
Semi-synchronous, MySQL AFTER_SYNC
COMMIT
write and flush binlog
send events
write relay log
ACK received
commit in storage engine
OK
Synchronous, PostgreSQL remote_apply
COMMIT
flush local WAL



Reading the diagram. Solid arrows are steps the client waits for. In the asynchronous path, shipping (open arrowhead) happens after the reply, so a primary crash then loses the transaction. MySQL's lossless semi-sync (`AFTER_SYNC`) waits for a replica to write its **relay log** before the engine commit, so no client sees a transaction no replica has. PostgreSQL's `remote_apply` waits until the standby has *applied* the change. Storage boundaries are the two logs. The failure point is the replica: a slow synchronous replica stalls every commit, which quorum commit over several replicas mitigates.

The two products expose different knobs:

- **PostgreSQL.** `synchronous_standby_names` chooses which standbys count: `FIRST 1 (a, b)` by priority or `ANY 2 (a, b, c)` as a quorum. `synchronous_commit` sets how far to wait: `off`, `local`, `remote_write`, `on` (standby flushed) or `remote_apply` (standby applied) [PostgreSQL Docs: synchronous_commit].
- **MySQL.** Semi-synchronous replication has a wait point (`AFTER_SYNC`, lossless and the default since 5.7, or `AFTER_COMMIT`), a number of replicas that must acknowledge, and a timeout after which the primary silently reverts to asynchronous mode [MySQL Docs: Semisynchronous Replication]. Semi-sync protects you only while replicas are healthy.

Mathematical foundations

FORMULA

Commit latency with k acknowledgments

$$T_{\text{commit}} \approx T_{\text{local}} + \text{RTT}_{(k)} + T_{\text{remote}}$$

Symbol	Meaning	Units
T_{commit}	Time from the client's `COMMIT` to the primary's reply	ms
T_{local}	Time to flush the log on the primary	ms
$\text{RTT}_{(k)}$	The k-th smallest round-trip time among the eligible replicas	ms
T_{remote}	Work the replica does before acknowledging (write, flush or apply)	ms
k	Number of replica acknowledgments required ($k = 0$ for asynchronous)	count

WORKED EXAMPLE

Worked example — Where to put the synchronous replica. A primary flushes its WAL in 0.2 ms on NVMe with power-loss protection (a drive without it can take about 2 ms per `fsync`, §6.14.3). Replica A is in the same availability zone (RTT 0.3 ms), replica B is in another zone of the same region (RTT 1.2 ms), and replica C is in another region (RTT 70 ms). Each replica flushes in 0.2 ms.

1. Asynchronous ($k = 0$): $T \approx 0.2$ ms.
2. Synchronous, `ANY 1 (A, B)`: the fastest acknowledgment is A's. $T \approx 0.2 + 0.3 + 0.2 = 0.7$ ms.
3. Synchronous, `ANY 2 (A, B, C)`: the second-fastest is B. $T \approx 0.2 + 1.2 + 0.2 = 1.6$ ms.
4. Synchronous to C only: $T \approx 0.2 + 70 + 0.2 \approx 70.4$ ms.
5. A single session committing sequentially manages $1/0.7\,\text{ms} \approx 1,430$ commits/s in case 2 but only $1/70.4\,\text{ms} \approx 14$ in case 4. Group commit helps concurrent sessions, but each transaction still pays the latency.

Interpretation. A cross-zone acknowledgment costs about a millisecond. A cross-region one costs the inter-region RTT on every commit. **Engineering implication.** A common pattern is synchronous replication to a replica in another zone of the same region plus asynchronous replicas in remote regions. A zone loss then loses no data, and a region loss risks a few seconds of writes (a nonzero recovery point objective, §39.8).

Table 14.2 — Replication modes compared.

	Asynchronous	Semi-synchronous (MySQL)	Synchronous / quorum (PostgreSQL)
Client waits for	Local flush only	≥ 1 replica has *received*	*k* replicas have flushed or applied
Commit latency	Lowest	+ 1 RTT	+ RTT of *k*-th fastest replica
Data loss on primary crash	Possible (seconds of writes)	None while healthy; possible after timeout fallback	None, if failover picks an acknowledged replica
Availability if replicas fail	Unaffected	Unaffected after timeout	Commits block until *k* replicas return
Stale reads on replicas	Yes	Yes (received, not yet applied)	Only `remote_apply` avoids them on the waited-for replicas
Choose when	Throughput first; remote regions	MySQL fleets wanting near-zero loss	Financial data; zone-level failure tolerance

Common misconceptions

COMMON MISTAKE

Mistake — "Synchronous replication means replicas never serve stale data." PostgreSQL's default synchronous setting (`on`) waits for the standby to *flush*, not *apply*, so a read there a millisecond later can miss the change. Only `remote_apply` closes the gap, and only on the standbys waited for.

14.4.2 Replication lag

DEFINITION

Definition — Replication lag. The delay between a change committing on the primary and becoming visible on a replica, measured in seconds, bytes of log or transactions.

Within one region, asynchronous lag is typically well under a second, but it is unbounded. It grows whenever the replica applies changes more slowly than the primary produces them: a single-threaded applier, a bulk `UPDATE` of millions of rows, a replica returning from maintenance, a network partition, or replica queries that block replay. Catching up is draining a queue.

FORMULA

Replica catch-up time

$$T_{\text{catch-up}} = \frac{B}{r_{\text{apply}} - r_{\text{in}}} \qquad (r_{\text{apply}} > r_{\text{in}})$$

Symbol	Meaning	Units
$T_{\text{catch-up}}$	Time until the replica's lag returns to near zero	s
B	Backlog of unapplied changes when catch-up starts	transactions
r_{apply}	Rate at which the replica can apply changes	transactions/s
r_{in}	Rate at which the primary generates new changes during catch-up	transactions/s

WORKED EXAMPLE

Worked example — A replica returns from 10 minutes of maintenance. The primary commits $r_{\text{in}} = 5,000$ transactions/s. The replica can apply $r_{\text{apply}} = 8,000$ transactions/s with parallel apply.

1. Backlog: $B = 5,000 \times 600\,\text{s} = 3,000,000$ transactions.
2. Net drain rate: $8,000 - 5,000 = 3,000$ transactions/s.
3. Catch-up time: $3,000,000/3,000 = 1,000$ s, about 17 minutes.
4. At 5,500 transactions/s the net drain is 500/s and catch-up takes 6,000 s (100 minutes); at 5,000/s it never ends.

Interpretation. Catch-up time depends on the *margin* between apply and arrival rates. A replica with 10 % spare apply capacity takes ten times as long to recover as the outage lasted. **Engineering implication.** Keep replicas out of the read pool until lag is within your staleness budget, and track apply headroom $r_{\text{apply}}/r_{\text{in}}$ as a capacity metric.

14.4.3 Anomalies caused by lag

Lag is invisible until it violates a user's expectations. Two violations are common. Chapter 25 (§25.6) treats these **session guarantees** formally.

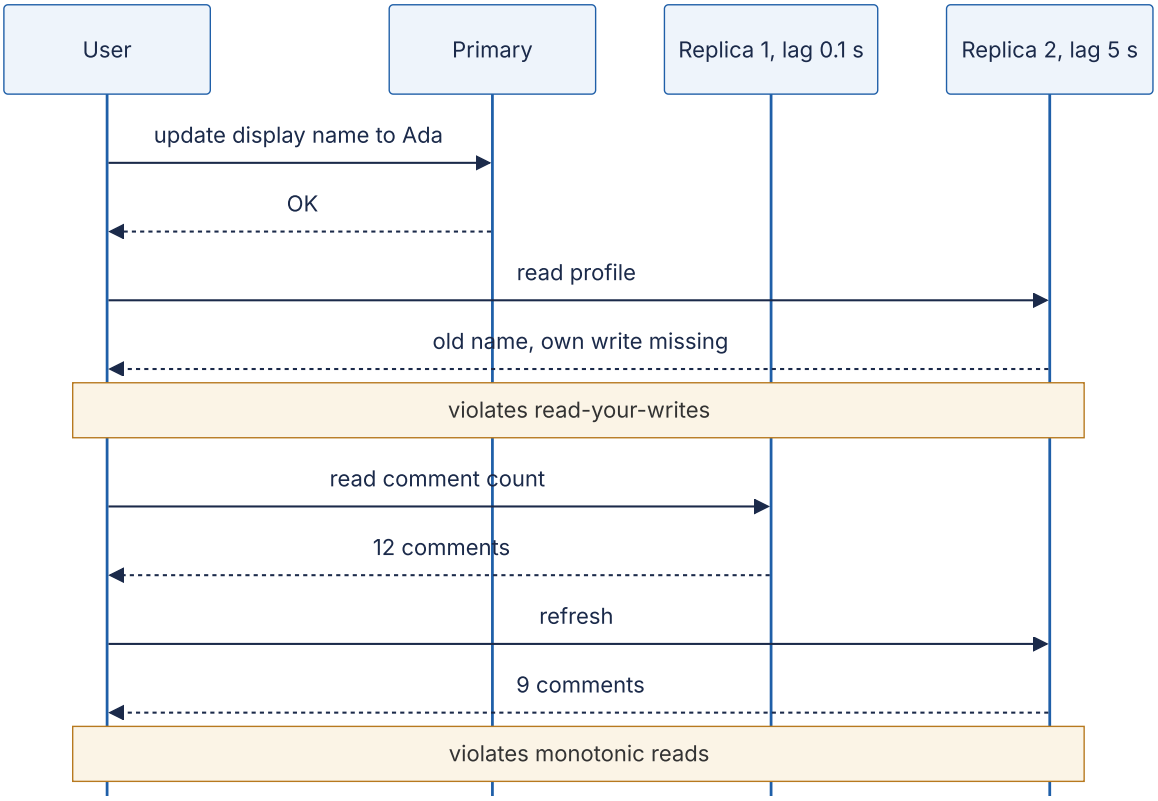
DEFINITION

Definition — Read-your-writes consistency. After a client writes a value, its own later reads return that value or a newer one. Other clients get no such promise.

DEFINITION

Definition — Monotonic reads. A client never sees data move backwards: after reading some version, its later reads return that version or newer.

Figure 14.3 — Two lag anomalies: lost own write and time going backwards.



Reading the diagram. Calls are synchronous; replication (not drawn) is asynchronous. The read after the write lands on Replica 2, five seconds behind, so the edit seems lost. Then successive reads hit different replicas, and the comment count falls from 12 to 9. The failure point is the router's freedom to pick any replica, and it bites more often as replicas are added.

14.4.4 Designing around lag

Each fix narrows the router's freedom and so costs some read scalability.

1. **Read recent writers from the primary.** For a window after a user writes, such as 10 seconds, route that user's reads to the primary. PhotoShare used this rule (Figure 1.6, §1.9). It is simple but fails silently if lag exceeds the window.
2. **Track the write's log position.** Return the primary's LSN or GTID after a write and read only from replicas that have applied it, or wait briefly until one has. MySQL offers `WAIT_FOR_EXECUTED_GTID_SET()`, and PostgreSQL exposes `pg_last_wal_replay_lsn()`. This is precise but needs a token plumbed through the application.
3. **Sticky replica per session.** Always read a session from the same replica. This gives monotonic reads but not read-your-writes.
4. **Synchronous apply where it matters.** Use `remote_apply` for the replica that serves a specific critical read path.

The program below implements option 2 with a per-session watermark.

```
        """Read-your-writes and monotonic reads with log-position tokens (simula-
tion)."""
        import random

        class Replica:
            def __init__(self, name):
                self.name, self.applied = name, 0       # applied log position (LSN)

        class Router:
            """Routes reads to any replica that has applied the session's watermark."""

            def __init__(self, replicas):
                self.primary_lsn = 0
                self.replicas = replicas
                self.watermark = {}                     # session -> minimum LSN to
read

            def write(self, session):
                self.primary_lsn += 1                   # commit on the primary
                self.watermark[session] = self.primary_lsn
                return self.primary_lsn

            def read(self, session):
                need = self.watermark.get(session, 0)
                fresh = [r for r in self.replicas if r.applied >= need]
                if not fresh:
                    return "primary"                    # nobody is fresh enough yet
                chosen = random.choice(fresh)
                # monotonic reads: later reads must be at least as fresh as this one
                self.watermark[session] = max(need, chosen.applied)
                return chosen.name

        if __name__ == "__main__":
            random.seed(7)
            r1, r2 = Replica("replica-1"), Replica("replica-2")
            router = Router([r1, r2])
            lsn = router.write("alice")                 # alice writes; LSN 1
            print("right after write  ->", router.read("alice"))   # nobody has LSN 1
            r1.applied = lsn                            # replica-1 catches up
            print("replica-1 caught up ->", router.read("alice"))
            print("bob (no writes)     ->", router.read("bob"))    # any replica is fine
```

What to notice:

- Only the user who wrote pays for freshness. Other sessions read from any replica.
- With no fresh replica, the router falls back to the primary. A production router might wait a few milliseconds first to avoid a stampede.
- Raising the watermark to the replica just read also gives monotonic reads.

14.4.5 What travels on the wire: statement, row, physical and logical replication

DEFINITION

Definition — Replication log formats. What the log describes: the SQL statements executed (**statement-based**), the changed rows (**row-based**), the storage engine's page-level byte changes (**physical**, or WAL shipping), or decoded, engine-independent row changes per table (**logical**).

How each works

- **Statement-based.** Replicas re-execute the primary's SQL text. The log is compact, but non-determinism (`NOW()`, `RAND()`, `UPDATE ... LIMIT` without `ORDER BY`) makes copies diverge silently. MySQL's `MIXED` mode falls back to rows for statements it knows are unsafe.
- **Row-based.** The log carries each changed row's key and new values, so replay is deterministic, at the cost of volume: a million-row `UPDATE` yields a million events. `ROW` has been MySQL's default since 5.7.7 [MySQL Docs: Replication Formats].
- **Physical (WAL shipping).** PostgreSQL streams the WAL itself, page-level byte changes, so the replica is a byte-for-byte copy including indexes and replay is cheap. The coupling is tight: same major version, the whole cluster, no extra tables or indexes.
- **Logical.** The primary decodes its WAL into per-table row changes and publishes them; in PostgreSQL (since version 10) a **publication** on the source feeds a **subscription** on the target [PostgreSQL Docs: Logical Replication]. This enables cross-version upgrades, partial copies and **change data capture (CDC)** into brokers and search indexes (§32.6). Built-in logical replication does not carry DDL, and a writable subscriber can conflict.

Table 14.3 — Replication log formats compared.

	Statement-based	Row-based	Physical (WAL)	Logical
What is shipped	SQL text	Changed rows	Page-level byte changes	Decoded row changes per table
Log volume for bulk update	Tiny	Large	Large (includes indexes)	Large
Determinism	Fragile	Exact	Exact	Exact
Version coupling	Loose	Loose	Same major version	Cross-version
Replica can differ (extra indexes, subset)	Partly	Partly	No	Yes
Feeds CDC and other systems	No	Yes (binlog readers)	No	Yes

RECOMMENDATION

Recommendation — Default choices. Use row-based binlogs in MySQL and physical streaming for PostgreSQL high-availability replicas. Add logical replication for upgrades, partial copies or CDC.

Failure scenarios

Table 14.4 — Failure scenarios in replication modes and formats.

What goes wrong	Why	Detection	Recovery	Prevention
All commits hang	Sole synchronous standby is down	Commit latency spikes to timeout	Remove it from `synchronous_standby_names`	Quorum `ANY k` over ≥ $k+1$ standbys
Silent loss of the durability guarantee	Semi-sync timed out and fell back to async	Semi-sync status counters	Restore replica health	Alert on fallback, not just on lag
Split brain after failover	Old primary still accepts writes	Two nodes report primary role	Fence old primary; reconcile	Fencing tokens and leases (§26.11–§26.13)

14.5 Multi-primary replication

14.5.1 Writes in more than one place

DEFINITION

Definition — Multi-primary replication. More than one node accepts writes for the same data and replicates to the others (also **multi-leader** or **active-active**). Concurrent uncoordinated changes to the same row create **write conflicts** that must be detected and resolved.

First-principles explanation

Two offices each keep a copy of the customer address book and exchange changes every evening. Each can edit at full speed even when the phone lines are down, but if both change Mrs. Rivera's address on the same day, somebody needs a rule for which one wins.

Why does this exist?

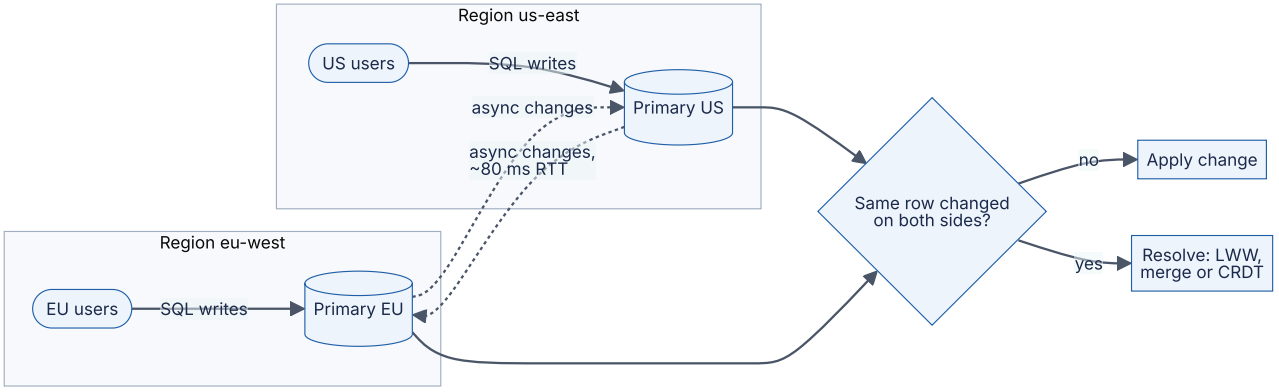
Single-primary replication makes distant users pay 70–150 ms per write and stops all writes if the primary's region is cut off. Multi-primary puts a writable primary in each re-

gion. Offline-capable clients, such as a calendar app, are the same pattern with each device as a primary.

How it works internally

1. Each region's primary accepts local writes and commits them locally.
2. Changes are replicated asynchronously to the other primaries, tagged with their origin so they are not echoed back.
3. When a change arrives that touches a row also changed locally since the last sync, the receiving primary detects a conflict.
4. A **conflict-resolution** policy picks the outcome. Common policies are **last-writer-wins** (LWW, the change with the later timestamp wins, which silently discards the other), a custom merge function, keeping both versions for the application to resolve, or **conflict-free replicated data types** (CRDTs) whose merges are mathematically guaranteed to converge. Chapter 27 (§27.14) covers these in depth.

Figure 14.4 — Two-region multi-primary replication and a write conflict.



Reading the diagram. Each region writes synchronously to its local primary (about 1 ms), and the primaries exchange changes asynchronously, so the two storage boundaries differ until replication catches up. Every incoming change passes the conflict detector. The resolver is the failure point: LWW depends on clocks (§24.9) and silently drops a write, and uniqueness cannot be enforced across regions. Each added region adds pairs of primaries that can conflict.

Advantages, disadvantages and when to choose it

Multi-primary buys local write latency and partition tolerance at the cost of global constraints, linearizable reads, colliding auto-increment keys and much harder debugging. A common compromise is **partitioned ownership**: each row has one home region that is its only primary, which gives most of the latency benefit with no conflicts. Choose true multi-primary only when the same data must be written concurrently in several regions *and* you can define a merge.

INDUSTRY EXAMPLE

MySQL Group Replication. In multi-primary mode, all members accept writes. Transactions are certified through a group-communication protocol, and one that conflicts with an earlier-certified transaction is rolled back rather than merged. It needs a low-latency network, so it is typically used within one region. *Evidence: Primary* — [MySQL Docs: Group Replication]

COMMON MISTAKE

Mistake — "Active-active doubles write capacity." Every primary still applies every other primary's writes. Multi-primary improves write *latency* and *availability*, not total write throughput.

14.6 Partitioning: range, hash, list and directory-based

14.6.1 What partitioning is

DEFINITION

Definition — Partition (shard). A subset of a dataset's rows, stored and served together, such that every row belongs to exactly one partition. **Partitioning** spreads partitions over nodes; **sharding** is a synonym used mostly for separate database servers. This book uses *partition* and *shard* interchangeably and reserves **network partition** for network splits (§24.7). The columns that choose a row's partition form the **partition key** (or **shard key**).

First-principles explanation

A library that outgrows one room does not photocopy every book into a second room; that would be replication. It moves A–M to one room and N–Z to another. Each room holds and serves half the books, and a visitor seeking one title goes straight to the right room. A visitor asking "which books were published in 1987?" must search both. Capacity scales with rooms, lookups by the partitioning attribute stay cheap, and everything else gets more expensive.

Why does this exist?

Replication cannot fix three limits: one primary's write rate, one node's storage and one node's memory for the working set. Partitioning divides all three by the number of partitions. It is the only general way to scale writes horizontally. (PostgreSQL's declarative ta-

ble partitioning splits a table on the *same* node, which helps maintenance but adds no machines.)

How it works internally

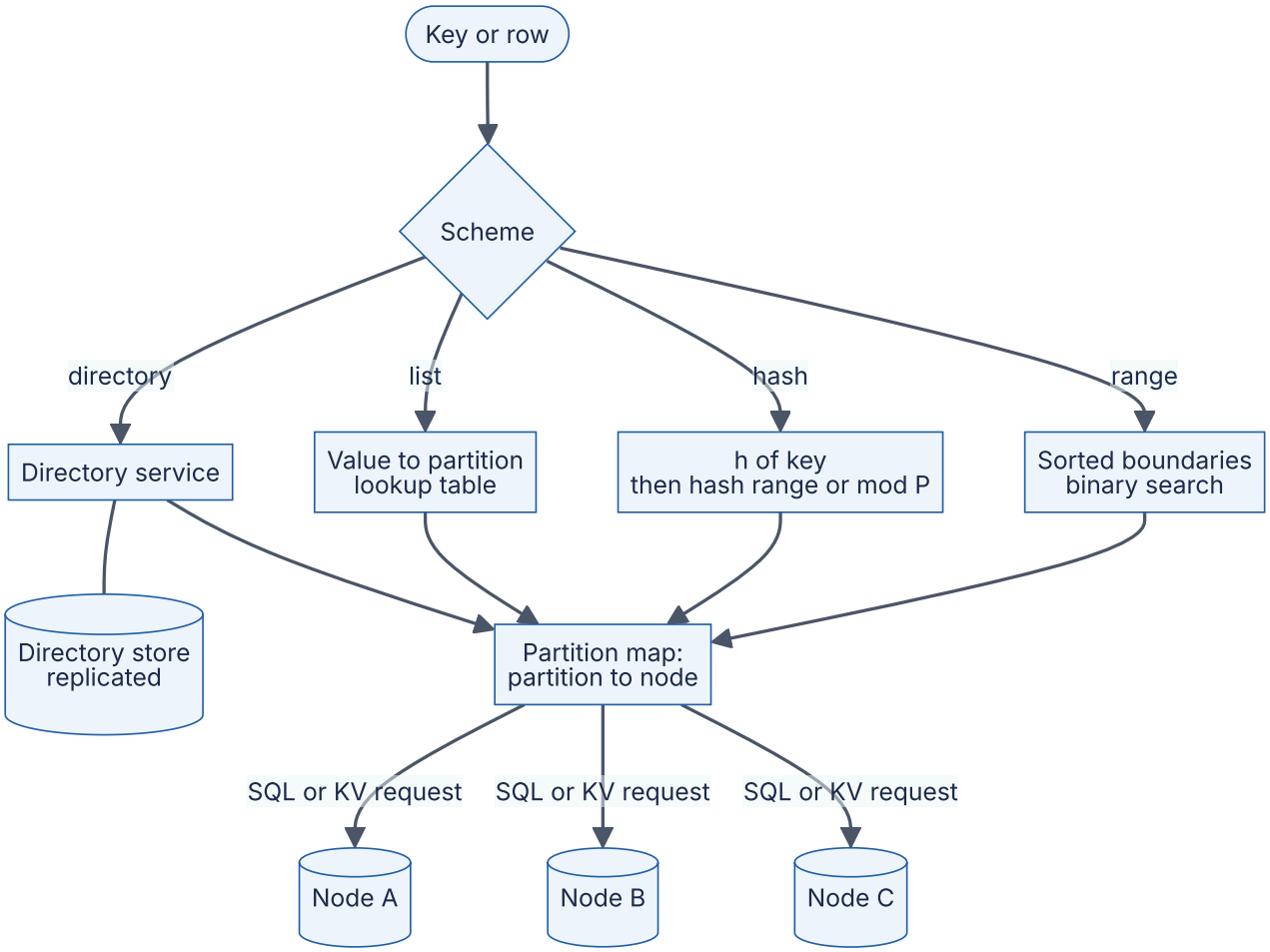
Every partitioned system answers two questions per request: **which partition holds this key?** and **which node holds that partition?** The schemes differ in the first answer.

1. **Range partitioning.** Each partition owns a contiguous key range, found by binary search over a small boundary table. Range scans touch only overlapping partitions. But sequential keys, such as timestamps or auto-increment IDs, send *all* new writes to the last partition. Bigtable, HBase and the distributed SQL systems of Chapter 28 range-partition and split ranges as they grow [Chang et al. 2006].
2. **Hash partitioning.** Partitions own ranges of $h(\text{key})$, or $h(\text{key}) \bmod P$ for a fixed P. Even sequential keys spread evenly, but range queries on the key must ask every partition. Mapping hashes to a changing set of nodes is §14.8.
3. **List partitioning.** Each partition owns an explicit list of values, such as `country IN ('DE', 'FR', 'IT')`. It suits low-cardinality attributes with business meaning, especially **data residency** (Chapter 44).
4. **Directory-based partitioning.** A lookup service maps each key, or group of keys such as a tenant, to its partition. Any placement is possible, and moving a tenant means copying its data and editing one entry. The directory is on every request's path, so it is cached and stored in a strongly consistent replicated store.

Compound keys combine schemes: Cassandra's primary key `((user_id), created_at)` hashes `user_id` to place the partition and keeps each user's rows sorted by time inside it, so "user 42's latest 20 posts" is one sequential read [Lakshman and Malik 2010].

Architecture diagram

Figure 14.5 — Four ways to answer "which partition holds this key?"



Reading the diagram. The scheme is a design-time choice, not a runtime branch. Range, hash and list are pure functions plus a small cached table; a directory adds a synchronous lookup unless cached. All yield a partition ID that a separate map turns into a node, so rebalancing edits only that map (§14.9). Failure points: a stale map (nodes must reject misrouted requests) and a directory outage. Scaling: add nodes and move partitions; the key-to-partition function never changes.

Table 14.5 — Partitioning schemes compared.

	Range	Hash	List	Directory
What it is	Contiguous key ranges	Ranges of hashed key	Explicit value lists	Explicit key-to-partition map
Load balance	Poor for sequential keys	Good	Depends on value sizes	Whatever you place
Range scans on key	Efficient	Scatter to all partitions	Efficient within a value	Depends
Routing cost	Binary search, cacheable	Arithmetic	Table lookup	Remote lookup, cached
Rebalancing	Split or merge ranges	Move hash ranges (§14.8–§14.9)	Edit lists	Move one entry at a time
Operational complexity	Medium	Low	Low	High (directory is critical)
Choose when	Time series, ordered scans	Point lookups, uniform load	Residency, regions	Tenants of very different sizes

Common misconceptions

COMMON MISTAKE

Mistake — "Hashing removes hot spots." Hashing spreads *distinct* keys. It cannot split one key's traffic. A celebrity key drawing 30 % of reads puts at least 30 % on its partition under any hash (§14.10).

14.7 Sharding strategies and choosing shard keys

14.7.1 Properties of a good shard key

The shard key is the least reversible decision in a partitioned database, because changing it means moving every row (§14.9). Evaluate candidates against five properties:

1. **High cardinality.** Many more distinct values than shards: `country` has about 200, `user_id` billions.
2. **Even load.** Data and request rate per value roughly uniform, or dominated by many small values.
3. **Query locality.** Frequent queries include the key and touch one shard. Others **scatter** to every shard and **gather** results.
4. **Transaction locality.** Rows changed atomically together share a key value, avoiding distributed commit (§14.12).
5. **Stability.** The key never changes. Otherwise a row must move between shards.

No key is perfect. List the top five queries by frequency and pick the candidate that keeps the most traffic on one shard.

14.7.2 Worked example: a multi-tenant SaaS product

Consider **TaskForge**, a hypothetical project-management SaaS with 40,000 customer organizations (tenants). Nearly every query is tenant-scoped: “this project’s open tasks”, “search this workspace”. Tenant sizes are heavy-tailed: the median has 15 users and the largest has 60,000 and about 2 % of all data.

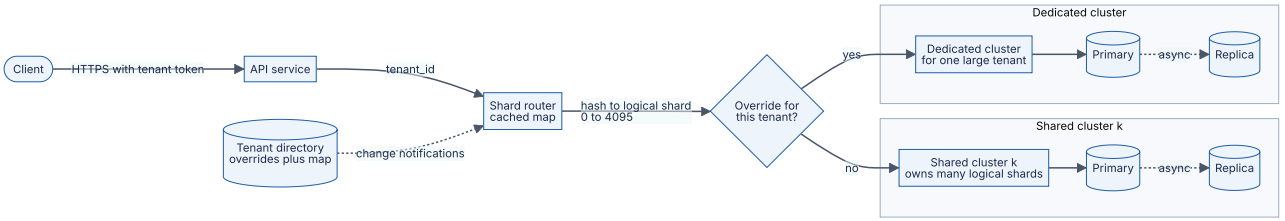
Table 14.6 — Shard-key candidates for TaskForge.

Candidate	Cardinality	Even load	Query locality	Transaction locality	Verdict
`task_id`	Very high	Excellent	Poor: “open tasks in project” scatters	Poor	Reject
`user_id`	High	Good	Poor: shared projects span users	Poor	Reject
`tenant_id`	40,000	Skewed by large tenants	Excellent: almost all queries	Excellent	**Choose**, with mitigation
`project_id`	High	Good	Good within a project; cross-project reports scatter	Good	Second choice

`tenant_id` keeps almost every query and transaction on one shard and simplifies tenant isolation, backup, residency and deletion. Its weakness is skew from giant tenants, fixed with directory placement on top of hashing:

1. Hash `tenant_id` into one of $L = 4,096$ **logical shards** (fixed partitions, §14.9).
2. Map logical shards to physical database clusters through a small directory.
3. Give the few giant tenants a **dedicated** cluster through a directory override. If one outgrows even that, partition it by `(tenant_id, project_id)`.

Figure 14.6 — Multi-tenant sharding with logical shards and a tenant directory.



Reading the diagram. The API takes the tenant ID from the authenticated token, never from user input. The router caches the shard-to-cluster map and override list, updated asynchronously from the directory, so databases must reject writes for logical shards they do not own. SQL to the chosen primary is synchronous, and each cluster is a replicated storage boundary (§14.3–§14.4). A cluster failure affects only its tenants, a smaller **blast radius** than one big database. Scaling: add clusters and move logical shards, or give a growing tenant its own cluster.

> INDUSTRY EXAMPLE
>
> **Notion.** Notion sharded its PostgreSQL block data by workspace ID, because each block belongs to one workspace. It created 480 logical shards (PostgreSQL schemas) spread over 32 physical databases, 15 each, with routing in application code. It chose 480 because its many divisors let the fleet grow to 40 or 48 hosts with equal shard counts, whereas 512 would force doubling. *Evidence: Primary* — [Fidalgo 2021]

14.7.3 Worked example: a social application

Now take **PhotoShare**, the hypothetical photo-sharing app of Chapter 1. Its top queries are “show user X’s profile and photos”, “post a photo”, “like a photo” and “build my home feed”.

- `photos` **by** `owner_user_id`**.** Profiles and posting become single-shard, and a compound key `((owner_user_id), created_at DESC)` keeps photos time-ordered. Celebrity profiles concentrate reads on one shard (§14.10).
- `likes` **by** `photo_id`**.** Like counts and “did I like it?” are single-shard. “Photos I liked” becomes a scatter query, served by an index or denormalized copy (§14.11).
- `follows`**, stored twice.** “Who does X follow?” wants partitioning by follower and “who follows X?” by followee, so write both copies, usually asynchronously (§32.6), and accept brief inconsistency.
- **Home feed.** Querying every followed user’s shard per page view is a large fan-out. Large social systems typically precompute feeds on write, except for celebrities whose posts are merged at read time [Kleppmann 2017].

> KEY CONCEPT
>
> **Key insight — Partition by the entity your hottest queries name, and denormalize for the rest.** No single key localizes every social-graph query. Pick the key for the dominant path and keep an eventually consistent, differently partitioned copy for the next.

INDUSTRY EXAMPLE

Instagram (2012). Instagram sharded PostgreSQL into several thousand **logical shards** (schemas) mapped onto far fewer servers. Each 64-bit ID embeds 41 bits of milliseconds since a custom epoch, 13 bits of logical shard ID and 10 bits of per-shard sequence, so any service can route from an ID alone. *Evidence: Primary* — [Instagram Engineering 2012]

14.7.4 How many shards? A capacity calculation

Shard count is set by whichever resource runs out first, computed at the end of the planning horizon because resharding is expensive.

FORMULA

Number of shards required

$$S = \left\lceil \max\left(\frac{D}{C_{\text{disk}} \cdot f},\ \frac{W}{w_{\text{node}} \cdot u},\ \frac{R}{n_{\text{read}} \cdot r_{\text{node}} \cdot u} \right) \right\rceil$$

Symbol	Meaning	Units
S	Number of shards (each one a replicated group)	shards
D	Data size at the planning horizon, one copy, including indexes	TB
C_{disk}	Usable storage per node	TB
f	Target fill fraction, leaving room for compaction, backups and growth between reshards	dimensionless (0–1)
W	Peak write rate at the horizon	writes/s
w_{node}	Sustainable write rate of one primary (from benchmarks)	writes/s
R	Peak read rate that reaches the database, after caches	reads/s
n_{read}	Nodes per shard that serve reads	nodes
r_{node}	Sustainable read rate of one node	reads/s
u	Target utilization at peak	dimensionless (0–1)

WORKED EXAMPLE

Worked example — Sharding PhotoShare's metadata store. SI units (1 TB = 10^{12} bytes). Assumptions are stated, not measured.

1. **Storage.** Metadata is 30 TB today, including indexes, and grows by 1.5 TB per month. The planning horizon is 24 months, so $D = 30 + 24 \times 1.5 = 66$ TB. Each node has 8 TB of usable NVMe, and you fill it to at most $f = 0.6$. Storage term: $66/(8 \times 0.6) = 66/4.8 = 13.75$.
2. **Writes.** Peak writes are 90,000/s today and expected to grow 1.5×, so $W = 135,000$/s. A benchmarked primary sustains $w_{\text{node}} = 10,000$ writes/s, and you run at $u = 0.6$. Write term: $135,000/(10,000 \times 0.6) = 135,000/6,000 = 22.5$.
3. **Reads.** Peak reads are 1,200,000/s, and the cache hit ratio is 90 %, so $R = 120,000$/s reach the database. Each shard has one primary and two replicas, and only the replicas serve reads, so $n_{\text{read}} = 2$. A replica sustains $r_{\text{node}} = 20,000$ reads/s, and $u = 0.6$. Read term: $120,000/(2 \times 20,000 \times 0.6) = 120,000/24,000 = 5$.
4. **Take the maximum.** $S = \lceil \max(13.75, 22.5, 5) \rceil = 23$. The store is **write-bound**.
5. **Round for operations.** Choose 24 shards so the fleet splits evenly over 3 availability zones (8 primaries per zone). Total nodes: $24 \times 3 = 72$.
6. **Logical shards.** Create $L = 960$ logical shards, 40 per physical shard. 960 is divisible by 24, 30, 32, 40, 48, 60, 64, 80 and 96, so the fleet can grow in modest steps and stay perfectly even.
7. **Sanity check after sharding.** Each shard stores $66/24 = 2.75$ TB (34 % of a node's disk) and takes $135,000/24 \approx 5,600$ writes/s (56 % of w_{node}) at the horizon.

Interpretation. Storage alone suggests 14 shards, which would under-provision by 40 %. **Engineering implication.** Compute all three terms and benchmark the binding one carefully: a 20 % error in w_{node} moves the answer by about five shards. Choose the logical count once, for years, and the physical count for the next 18–24 months.

The program below implements the formula and reproduces the example.

```
        """Shard-count calculator: the binding constraint among storage, writes and
reads."""
        import math

        def shards_needed(data_tb, disk_tb, fill, writes, write_cap, reads,
                          read_nodes, read_cap, util):
            terms = {
                "storage": data_tb / (disk_tb * fill),
                "writes": writes / (write_cap * util),
                "reads": reads / (read_nodes * read_cap * util),
            }
            binding = max(terms, key=terms.get)
            return math.ceil(terms[binding]), binding, terms

        def even_logical_counts(physical, candidates=(512, 720, 960, 1024, 4096)):
            """Logical-shard counts that divide evenly over `physical` shards."""
            return [c for c in candidates if c % physical == 0]

        if __name__ == "__main__":
            s, binding, terms = shards_needed(
                data_tb=30 + 24 * 1.5, disk_tb=8, fill=0.6,
                writes=90_000 * 1.5, write_cap=10_000,
                reads=1_200_000 * (1 - 0.90), read_nodes=2, read_cap=20_000, util=0.6)
            print({k: round(v, 2) for k, v in terms.items()})
            print(f"minimum shards = {s} ({binding}-bound)")
            physical = 3 * math.ceil(s / 3)          # round up to a multiple of 3 zones
            print(f"physical shards = {physical}, nodes = {physical * 3}, "
                  f"even logical counts = {even_logical_counts(physical)}")
```

Running it prints:

```
{'storage': 13.75, 'writes': 22.5, 'reads': 5.0}
minimum shards = 23 (writes-bound)
physical shards = 24, nodes = 72, even logical counts = [720, 960]
```

What to notice:

- It reports *which* term binds, which is where a real benchmark is needed.
- 1,024 and 4,096 do not divide evenly over 24 shards. Power-of-two counts suit fleets that double; highly composite counts suit fleets that grow in small steps.
- A cache outage multiplies R by 10, turning the read term from 5 into 50 (§20.13).

Failure scenarios

Table 14.7 — Failure scenarios from shard-key choice.

What goes wrong	Why	Detection	Recovery	Prevention
One shard runs out of disk	Key with large, uneven entities (a giant tenant)	Per-shard disk and growth dashboards	Move logical shards; split the tenant	Directory overrides; size by largest entity
All new writes hit one shard	Monotonic key under range partitioning	Per-shard write rate, one hot	Switch to hash or prefix the key	Avoid timestamps or sequences as the leading key
Cross-shard transactions everywhere	Related rows have different keys	2PC or saga volume	Re-key so related rows co-locate	Transaction locality as a key criterion

14.8 Consistent hashing and virtual nodes

This is the book's full treatment of consistent hashing. Section 3.3.2 used it as a study example, and Chapters 17, 19, 20 and 27 point back here.

14.8.1 The problem: `hash mod N` moves almost everything

The obvious rule, `node = h(key) mod N`, balances perfectly and needs no state, but a key keeps its node only if $h \bmod N = h \bmod (N+1)$, which holds for about $1/(N+1)$ of keys. Growing from 4 to 5 nodes moves about 80 % of keys, and from 100 to 101 about 99 %: a miss storm for a cache (§3.3.2) and a near-total copy for a database. We want a rule under which adding a node moves only the keys it should own.

DEFINITION

Definition — Consistent hashing. Keys and nodes are hashed onto the same circular space (the **ring**), and each key belongs to the first node position clockwise from its hash. Adding or removing a node changes the owner of an expected K/N of K keys for N nodes [Karger et al. 1997].

DEFINITION

Definition — Virtual node (vnode). One of V ring positions belonging to the same physical node, typically obtained by hashing labels such as `db-3#vn17`; also called a **token**.

14.8.2 Ring construction and lookup

First-principles explanation

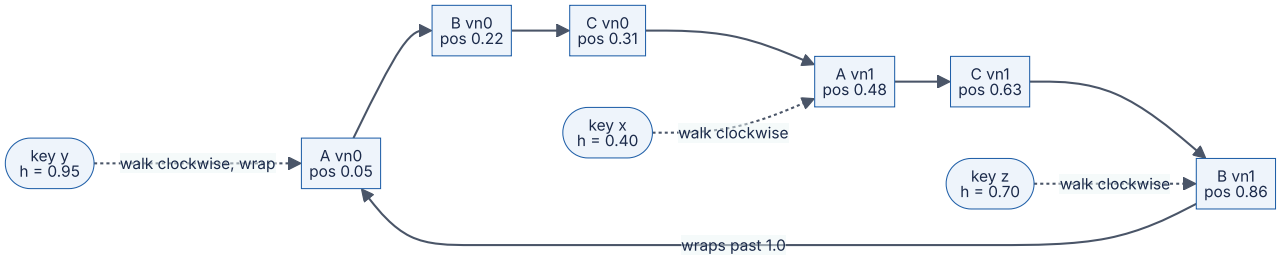
Picture a circular running track marked from 0 to just under 2^{64}. Each server stands at a few spots. Each key is dropped at the spot given by its hash and walks clockwise until it

meets a server, which owns it. A new server intercepts only the keys walking toward its spots; every other walk ends where it did before.

How it works internally

1. **Choose a hash** giving well-mixed 64-bit integers, identical in every router process (MD5 truncated to 64 bits, MurmurHash3, xxHash). Python's per-process randomized `hash()` is unsuitable.
2. **Place vnodes.** For each physical node and each i from 0 to $V - 1$, hash a label such as `db-3#vn17` to get a position. Store all positions in a sorted array, with a parallel array of owners.
3. **Look up a key.** Binary-search for the first position strictly greater than $h(\text{key})$, wrapping to index 0 past the end. Cost: $O(\log M)$ for M positions, about 15 comparisons for 100 nodes × 256 vnodes.
4. **Add a node.** Insert its positions. Each takes over the arc from its predecessor, so only keys in those arcs move, all *to* the new node.
5. **Remove a node.** Delete its positions. Its arcs merge into their clockwise successors, which, with scattered vnodes, belong to many different survivors.
6. **Weight nodes** by giving a bigger node proportionally more vnodes.
7. **Distribute membership** through a coordination service (§26.8–§26.9) or gossip (§27.10), versioned so a router can detect a stale copy.

Figure 14.7 — A ring with three nodes and two vnodes each.



Reading the diagram. Rectangles are vnode positions normalized to $[0, 1)$, linked clockwise and wrapping from 0.86 to 0.05. Each vnode owns the arc *ending* at it, for example A vn1 owns $(0.31, 0.48]$. Dotted lookups: x at 0.40 goes to A vn1, y at 0.95 wraps to A vn0, z at 0.70 goes to B vn1. Lookup happens in router memory; data lives on A, B and C, whose shares (0.36, 0.40, 0.24) are uneven with two vnodes each. Failure point: routers with different membership disagree about owners.

14.8.3 How many keys move?

FORMULA

Expected keys moved on a membership change

$$E[\text{moved on add}] = \frac{K}{N+1} \qquad E[\text{moved on remove}] = \frac{K}{N}$$

Symbol	Meaning	Units
K	Total number of keys	keys
N	Number of nodes before the change	nodes

By symmetry, a new node's expected share is $1/(N+1)$, and only keys in its new arcs move. A removed node's keys are its share, $1/N$ in expectation. These are the minimum any balanced scheme can achieve. Karger et al. call the property **monotonicity**: keys move only to the new node, never between old nodes [Karger et al. 1997].

WORKED EXAMPLE

Worked example — Adding a ninth database node. A store holds $K = 2 \times 10^9$ rows of 1 KB each (2 TB, SI units) on $N = 8$ nodes, ring-partitioned with 128 vnodes per node.

1. Rows moved: $2 \times 10^9 / 9 \approx 2.22 \times 10^8$ rows, about 222 GB.
2. Under `mod N`: about $8/9$ of rows, 1.78×10^9 rows or 1.78 TB, would move.
3. Time to move 222 GB if rebalancing is throttled to 100 MB/s to protect foreground traffic: $222 \times 10^9 / 10^8 = 2,220$ s, about 37 minutes. Moving 1.78 TB at the same rate takes 17,800 s, about 5 hours.
4. Sources: with 128 vnodes, the new node's 128 arcs are cut from all 8 existing nodes, each contributing roughly $222/8 \approx 28$ GB, so all 8 can stream in parallel.

Interpretation. The ring cuts data movement by a factor of N and spreads it over every node. **Engineering implication.** Adding capacity becomes a routine sub-hour operation, and the vnode count decides how many nodes share the copying.

The runnable ring below checks these claims. It supports weights and returns a replica **preference list**.

```
        """Consistent-hash ring with virtual nodes, weights and replica placement."""
        import bisect
        import hashlib
        from collections import Counter

        def h(key: str) -> int:
            """Map a string to a 64-bit ring position (MD5 used only for mixing)."""
            return int.from_bytes(hashlib.md5(key.encode("utf-8")).digest()[:8], "big")

        class HashRing:
            def __init__(self, vnodes_per_weight: int = 100):
                self.vpw = vnodes_per_weight
                self.weights = {}          # physical node -> weight
                self._positions = []       # sorted vnode positions
                self._owners = []          # owner of _positions[i]

            def _rebuild(self) -> None:
                points = sorted(
                    (h(f"{node}#vn{i}"), node)
                    for node, w in self.weights.items()
                    for i in range(self.vpw * w)
                )
                self._positions = [p for p, _ in points]
                self._owners = [n for _, n in points]

            def add_node(self, node: str, weight: int = 1) -> None:
                self.weights[node] = weight
                self._rebuild()            # one sort per membership change

            def remove_node(self, node: str) -> None:
                del self.weights[node]
                self._rebuild()

            def _index(self, key: str) -> int:
                i = bisect.bisect_right(self._positions, h(key))
                return 0 if i == len(self._positions) else i   # wrap around

            def owner(self, key: str) -> str:
                return self._owners[self._index(key)]

            def preference_list(self, key: str, r: int) -> list:
                """First r *distinct physical* nodes clockwise from the key."""
                r = min(r, len(self.weights))
                result, i = [], self._index(key)
                while len(result) < r:
                    node = self._owners[i]
                    if node not in result:
                        result.append(node)
                    i = (i + 1) % len(self._owners)
                return result

        if __name__ == "__main__":
            keys = [f"user:{i}" for i in range(200_000)]
            ring = HashRing(vnodes_per_weight=128)
            for n in range(8):
                ring.add_node(f"db-{n}")
```

```
before = {k: ring.owner(k) for k in keys}
ring.add_node("db-8")
after = {k: ring.owner(k) for k in keys}
moved = [k for k in keys if before[k] != after[k]]
print(f"add 9th node : {len(moved) / len(keys):.2%} of keys moved "
      f"(theory 1/9 = {1 / 9:.2%}); all moved keys went to "
      f"{set(after[k] for k in moved)}")

share = sum(1 for k in keys if after[k] == "db-3") / len(keys)
ring.remove_node("db-3")
gone = {k: ring.owner(k) for k in keys}
moved = [k for k in keys if after[k] != gone[k]]
receivers = Counter(gone[k] for k in moved)
print(f"remove db-3  : {len(moved) / len(keys):.2%} moved, exactly db-3's "
      f"share ({share:.2%}); spread over {len(receivers)} survivors, "
      f"largest receiver took {max(receivers.values()) / len(moved):.0%}")

ring.add_node("db-big", weight=2)        # twice the vnodes
load = Counter(ring.owner(k) for k in keys)
mean = len(keys) / (len(ring.weights) + 1)   # db-big counts as two
print(f"weighted node: db-big holds {load['db-big'] / mean:.2f}x a "
      f"weight-1 share")
print("replicas of user:42 ->", ring.preference_list("user:42", 3))
```

Running it with Python 3.8 or later prints:

```
add 9th node : 10.85% of keys moved (theory 1/9 = 11.11%); all moved keys went to {'db-
8'}
remove db-3  : 9.84% moved, exactly db-3's share (9.84%); spread over 8 survivors,
largest receiver took 20%
weighted node: db-big holds 1.88x a weight-1 share
replicas of user:42 -> ['db-0', 'db-5', 'db-big']
```

What to notice:

- Every moved key went to `db-8`: monotonicity, observed.
- Removal moved exactly the removed node's *actual* share, 9.84 % rather than the ideal 11.1 %; §14.8.4 quantifies that randomness.
- Those keys landed on all 8 survivors, the largest taking 20 %. With $V = 1$, one neighbor would take 100 %.
- `preference_list` skips nodes already chosen, so two replicas never share a machine.
- The weight-2 node got $1.88\times$, not $2\times$: weights set the *expected* share.

14.8.4 Virtual nodes and load balance: a derivation

With one position per node, §3.3.2 measured the busiest node at $2.6\times$ its fair share. Section 3.3.2 also claimed that the relative spread shrinks "roughly in proportion to $1/\sqrt{V}$". Here is the exact result for the standard model, checked by simulation.

The model. Assume an ideal hash, so each of the $M = NV$ vnode positions is independent and uniform on a ring of circumference 1. Each physical node owns V of them. A node's **load** L is the total length of the arcs ending at its vnodes, which is the fraction of a uniformly hashed key space it receives.

Step 1 — The arcs are Dirichlet-distributed. Cutting a circle at M independent uniform points gives M arc lengths that sum to 1 and are jointly $\text{Dirichlet}(1, \ldots, 1)$: fix one point as the origin by rotational symmetry, and the other $M - 1$ points cut the unit interval into pieces uniformly distributed on the simplex. This is the classical **uniform spacings** result [David and Nagaraja 2003]. The arcs are *exchangeable*, each with mean $1/M$.

Step 2 — A node's load is a Beta random variable. Which positions belong to which node depends only on labels, independent of where positions fall, so a node's arcs are in distribution any fixed V of the M exchangeable arcs. By the **aggregation property** of the Dirichlet distribution, their sum is $\text{Beta}(V, M - V)$:

$$L \sim \text{Beta}(V,\ (N-1)V).$$

Step 3 — Mean and variance. For $\text{Beta}(a, b)$, the mean is $a/(a+b)$ and the variance is $ab/((a+b)^2\,(a+b+1))$. With $a = V$ and $b = (N-1)V$:

$$E[L] = \frac{1}{N}, \qquad \text{Var}[L] = \frac{N-1}{N^2\,(NV+1)}.$$

Step 4 — Relative spread. Divide the standard deviation by the mean to get the coefficient of variation.

FORMULA

Load imbalance on a ring with virtual nodes

$$\mathrm{CV}(N,V) = \frac{\sqrt{\mathrm{Var}[L]}}{E[L]} = \sqrt{\frac{N-1}{NV+1}} \approx \frac{1}{\sqrt{V}} \quad (N \gg 1)$$

$$\frac{E[\max_i L_i]}{1/N} \approx 1 + z_N \cdot \mathrm{CV}(N,V) \quad (V \gtrsim 16), \qquad \frac{E[\max_i L_i]}{1/N} = H_N \quad (V = 1)$$

Symbol	Meaning	Units
L, L_i	Fraction of the key space owned by a node (by node i)	dimensionless (0–1)
N	Number of physical nodes	nodes
V	Virtual nodes per physical node	vnodes/node
CV	Coefficient of variation: standard deviation of a node's load divided by its mean	dimensionless
z_N	Expected maximum of N independent standard normal variables: 1.16 for $N = 5$, 1.54 for $N = 10$, 2.51 for $N = 100$, 3.24 for $N = 1,000$	dimensionless
H_N	N-th harmonic number $1 + \frac{1}{2} + \ldots + \frac{1}{N} \approx \ln N + 0.577$	dimensionless

Three remarks:

- **Why $1/\sqrt{V}$.** Each arc behaves almost like an independent exponential variable, whose standard deviation equals its mean. A sum of V of them has mean $\propto V$ and standard deviation $\propto \sqrt{V}$. The factor $(N-1)/N$ appears because the loads sum to 1.
- **The busiest node.** For large V each load is nearly normal, so the busiest of N sits about z_N standard deviations above the mean; z_N grows slowly, roughly as $\sqrt{2 \ln N}$. For $V = 1$, the longest of N uniform spacings has expected length H_N/N. The normal approximation slightly underestimates the maximum because the Beta distribution is right-skewed.
- **Agreement with published results.** Lamping and Veach report a 3.2 % standard error for a ring with 1,000 points per bucket, with about 1 % of buckets still at least 8 % off average [Lamping and Veach 2014]. The formula gives $\sqrt{1/1000} \approx 3.16$ %, and a two-sided 1 % normal tail sits at $2.58 \times 3.16\% \approx 8.1$ %. Karger et al. proved the asymptotic version: a logarithmic number of points per cache keeps every share within a constant factor of fair with high probability [Karger et al. 1997].

WORKED EXAMPLE

Worked example — Choosing V for a 100-node cluster. You want the busiest of $N = 100$ nodes to carry at most 10 % more than the mean, so $E[\max]/\text{mean} \le 1.10$.

1. Requirement: $z_{100} \cdot \text{CV} \le 0.10$, so $\text{CV} \le 0.10/2.51 = 0.0398$.
2. Solve $\sqrt{(N-1)/(NV+1)} \le 0.0398$: $(NV+1) \ge 99/0.0398^2 = 99/0.001584 \approx 62,500$, so $V \ge 625$.
3. Check a common default, $V = 256$: $\text{CV} = \sqrt{99/25,601} = 0.0622$, and $E[\max]/\text{mean} \approx 1 + 2.51 \times 0.0622 = 1.156$. A 30-ring simulation gives 1.17.
4. Check $V = 1$: $E[\max]/\text{mean} = H_{100} \approx 5.19$. The busiest node would carry over five times its fair share.
5. Memory for $V = 625$: 62,500 positions × about 16 bytes ≈ 1 MB per router, which is negligible.

Interpretation. Halving the imbalance costs four times the vnodes. **Engineering implication.** Either provision 15–20 % headroom per node with a few hundred vnodes, or place partitions deliberately: fixed equal-sized partitions (§14.9), token allocation, or bounded-load hashing (§14.8.6). More vnodes also mean more ranges to stream, repair and gossip.

The experiment below builds 200 random rings of 10 nodes for each V, computes each node’s exact share from the arc lengths and compares with the formula.

```
        """How load imbalance on a hash ring shrinks as virtual nodes V grow.

        For each V, build many independent rings of N nodes, compute every node's
        exact share of the key space (the total length of the arcs it owns) and
        compare the spread with the closed form sqrt((N-1)/(N*V+1)).
        """
        import hashlib
        import math
        import statistics

        RING = 2 ** 64
        N = 10            # physical nodes
        TRIALS = 200      # independent rings per value of V

        def h(key: str) -> int:
            return int.from_bytes(hashlib.md5(key.encode("utf-8")).digest()[:8], "big")

        def node_shares(n: int, v: int, trial: int) -> list:
            """Fraction of the ring owned by each node in one random ring."""
            points = sorted((h(f"t{trial}/node-{i}#vn{j}"), i)
                            for i in range(n) for j in range(v))
            share = [0] * n
            prev = points[-1][0] - RING          # wrap: arc ending at the first point
            for pos, owner in points:
                share[owner] += pos - prev       # a vnode owns the arc that ends at it
                prev = pos
            return [s / RING for s in share]

        def expected_max_std_normal(n: int, samples: int = 4000) -> float:
            """E[max of n standard normals], by a quick deterministic quadrature."""
            nd = statistics.NormalDist()
            total = 0.0
            for k in range(samples):                  # E[max] = integral of x d(F^n)
                u = (k + 0.5) / samples               # u = F(x)^n  =>  x =
F^-1(u^(1/n))
                total += nd.inv_cdf(u ** (1.0 / n))
            return total / samples

        if __name__ == "__main__":
            z = expected_max_std_normal(N)
            print(f"N = {N} nodes, {TRIALS} rings per row, E[max of {N} normals] =
{z:.3f}")
            print(f"{'V':>5} {'CV sim':>8} {'CV theory':>10} {'max/mean sim':>13} "
                  f"{'max/mean approx':>16}")
            rows = []
            for v in (1, 4, 16, 64, 256, 1024):
                trials = TRIALS if v <= 256 else TRIALS // 4
                cvs, maxes = [], []
                for t in range(trials):
                    s = node_shares(N, v, t)
                    cvs.append(statistics.pstdev(s) * N)      # std / mean, mean = 1/N
                    maxes.append(max(s) * N)
                # shares sum to exactly 1, so pstdev about the mean 1/N is unbiased
                cv_sim = math.sqrt(statistics.fmean(c * c for c in cvs))
                cv_th = math.sqrt((N - 1) / (N * v + 1))
                rows.append((v, cv_sim))
```

```
                print(f"{v:>5} {cv_sim:>8.4f} {cv_th:>10.4f} "
                      f"{statistics.fmean(maxes):>13.3f} {1 + z * cv_th:>16.3f}")
            harmonic = sum(1 / k for k in range(1, N + 1))
            print(f"V=1 exact: E[max/mean] = H_N = {harmonic:.3f}")
            (v1, c1), (v2, c2) = rows[1], rows[-1]
            slope = math.log(c2 / c1) / math.log(v2 / v1)
            print(f"log-log slope of CV vs V between V={v1} and V={v2}: {slope:.3f}")
```

Running it takes about two seconds and prints:

```
N = 10 nodes, 200 rings per row, E[max of 10 normals] = 1.539
    V   CV sim  CV theory  max/mean sim  max/mean approx
    1   0.8977     0.9045         2.912           2.392
    4   0.4655     0.4685         1.885           1.721
   16   0.2358     0.2364         1.416           1.364
   64   0.1187     0.1185         1.199           1.182
  256   0.0585     0.0593         1.095           1.091
 1024   0.0302     0.0296         1.050           1.046
V=1 exact: E[max/mean] = H_N = 2.929
log-log slope of CV vs V between V=4 and V=1024: -0.493
```

What to notice:

- The simulated CV matches $\sqrt{(N-1)/(NV+1)}$ within sampling error, and the log–log slope of -0.49 is the signature of $1/\sqrt{V}$ (slope -0.5). Each $4\times$ increase in V halves the spread.
- For $V = 1$, the simulated maximum (2.91) matches $H_{10} = 2.93$ and the normal approximation (2.39) is poor; use it only from about $V = 16$.
- With K actual keys, counts add sampling noise of CV about $\sqrt{N/K}$ (1 % for $K = 10^5$, $N = 10$), which is why the weight-2 node above showed $1.88\times$.
- The model assumes uniform key popularity. Hot keys are a separate problem (§14.10).

> NOTE
>
> **Note — Resolution of the $1/\sqrt{V}$ question.** Under the ideal-hash model a node's share is exactly $\mathrm{Beta}(V, (N-1)V)$, with coefficient of variation $\sqrt{(N-1)/(NV+1)} \to 1/\sqrt{V}$. The simulation and [Lamping and Veach 2014] confirm it, so §3.3.2's statement holds for the relative standard deviation. The busiest node's excess also shrinks as $1/\sqrt{V}$, multiplied by a slowly growing z_N.

14.8.5 Replication on the ring

On a ring, each key is stored on the first R **distinct physical** nodes clockwise, its preference list (R here is the replication factor, which §27.8 calls N because there R is the read quorum). Dynamo does this, typically with three replicas, and also skips nodes in the

same failure domain [DeCandia et al. 2007]. With vnodes, a failed node's replicas are spread over many nodes, so re-replication copies from all of them in parallel. Quorum reads and writes over preference lists are in §27.7–§27.9.

INDUSTRY EXAMPLE

Amazon Dynamo. Dynamo began with T random tokens per node. The paper reports that bootstrapping a node required scanning others' data and that shifting partition boundaries complicated archiving and Merkle-tree maintenance. Dynamo moved to Q equal-sized partitions with Q/S assigned to each of S nodes, which the paper reports gave the best load-balancing efficiency and the smallest membership metadata of the strategies compared: the fixed-partitions approach of §14.9. *Evidence: Primary* — [DeCandia et al. 2007]

INDUSTRY EXAMPLE

Apache Cassandra. Early Cassandra used one token per node and moved tokens to rebalance [Lakshman and Malik 2010]. Later releases defaulted to 256 random vnodes per node. Cassandra 4.0 lowered the default `num_tokens` to 16 with an allocation algorithm that places tokens to balance load for a replication factor, cutting repair and streaming cost (as of Cassandra 4.0). *Evidence: Primary* — [Apache Cassandra Docs]

14.8.6 Alternatives: jump consistent hash and rendezvous hashing

Two other algorithms give the same minimal movement with different trade-offs.

Jump consistent hash

Jump consistent hash maps a 64-bit key and a bucket count n to a bucket in $[0, n)$ using a few registers and no table [Lamping and Veach 2014]. Growing from b to $b+1$ buckets should move a key to the new bucket with probability exactly $1/(b+1)$. Seeding a pseudo-random generator with the key makes those jump decisions deterministic, and the algorithm draws the position of the key's *next* jump directly, so the loop runs only $O(\ln n)$ times on average. The Python port below follows the paper's C++ code and compares it with rendezvous hashing.

```
        """Jump consistent hash and rendezvous (HRW) hashing, compared on movement and
balance."""
        import hashlib
        import statistics

        MASK64 = (1 << 64) - 1

        def h64(s: str) -> int:
            return int.from_bytes(hashlib.md5(s.encode("utf-8")).digest()[:8], "big")

        def jump_hash(key: int, num_buckets: int) -> int:
            """Lamping & Veach (2014): 64-bit key -> bucket in [0, num_buckets)."""
            b, j = -1, 0
            while j < num_buckets:
                b = j
                key = (key * 2862933555777941757 + 1) & MASK64   # 64-bit LCG step
                j = int((b + 1) * (float(1 << 31) / float((key >> 33) + 1)))
            return b

        def rendezvous(key: str, nodes: list) -> str:
            """Highest random weight: every node scores the key; the top score wins."""
            return max(nodes, key=lambda node: h64(f"{node}|{key}"))

        def report(name, assign_before, assign_after, keys, n_after):
            moved = sum(assign_before(k) != assign_after(k) for k in keys) / len(keys)
            counts = {}
            for k in keys:
                b = assign_after(k)
                counts[b] = counts.get(b, 0) + 1
            mean = len(keys) / n_after
            cv = statistics.pstdev(counts.values()) / mean
            print(f"{name:<11} moved {moved:6.2%} (ideal {1 / n_after:.2%})  "
                  f"key-count CV {cv:.4f}  max/mean {max(counts.values()) / mean:.3f}")

        if __name__ == "__main__":
            keys = [f"order:{i}" for i in range(100_000)]
            hashed = {k: h64(k) for k in keys}
            n = 10
            names = [f"db-{i}" for i in range(n + 1)]
            report("jump", lambda k: jump_hash(hashed[k], n),
                   lambda k: jump_hash(hashed[k], n + 1), keys, n + 1)
            report("rendezvous", lambda k: rendezvous(k, names[:n]),
                   lambda k: rendezvous(k, names), keys, n + 1)
            # Binomial noise alone (perfectly fair assignment of 100,000 random keys)
            p = 1 / (n + 1)
            print(f"{'noise floor':<11} key-count CV of a perfect hash = "
                  f"{((1 - p) / (len(keys) * p)) ** 0.5:.4f}")
```

Running it prints:

```
jump        moved  9.10% (ideal 9.09%)  key-count CV 0.0081  max/mean 1.019
rendezvous  moved  9.14% (ideal 9.09%)  key-count CV 0.0133  max/mean 1.017
noise floor key-count CV of a perfect hash = 0.0100
```

What to notice:

- Both move almost exactly the ideal $1/(N+1)$ of keys.
- Both balance at the **noise floor**, the variation of assigning keys by fair coin; differences from 0.010 are sampling noise over just 11 buckets. A ring with $V = 100$ has a structural CV near 0.10 for $N = 10$.
- `& MASK64` reproduces C's unsigned 64-bit overflow, which Python integers lack.
- `rendezvous` computes N hashes per lookup, trivial for 10 nodes, significant for thousands.

Jump hash's limitation is structural: buckets are numbered 0 to $n-1$ and only the *last* can be added or removed. It suits logical shard numbers that never disappear, because each shard is a replicated group whose machines are replaced underneath it, not cache tiers where arbitrary nodes come and go.

Rendezvous (highest random weight) hashing

Rendezvous hashing, or **highest random weight (HRW) hashing**, scores every node with $h(\text{node} \parallel \text{key})$ and picks the highest [Thaler and Ravishankar 1998]. Adding a node moves an expected $1/(N+1)$ of keys. Removing one sends each of its keys to that key's second-highest scorer, spreading load evenly with no vnodes, and the top R scores form a replica list. Lookups cost $O(N)$; weighted and hierarchical variants exist.

Table 14.8 — Key-to-node assignment schemes compared.

	`hash mod N`	Ring + vnodes	Jump hash	Rendezvous (HRW)
Keys moved on add	$\sim N/(N+1)$	$\sim 1/(N+1)$	$\sim 1/(N+1)$	$\sim 1/(N+1)$
Load balance (CV)	Noise floor	$\sqrt{(N-1)/(NV+1)}$	Noise floor	Noise floor
Lookup cost	$O(1)$	$O(\log NV)$	$O(\log N)$ expected	$O(N)$
Memory	None	$O(NV)$ positions	None	$O(N)$ names
Remove arbitrary node	Remaps nearly all	Yes	No, only the last bucket	Yes
Weights	Awkward	More vnodes	No	Weighted scores
Replica list	Ad hoc	Walk to R distinct nodes	Ad hoc	Top-R scores
Choose when	N fixed forever	Caches, Dynamo-style stores	Numbered shards that only grow	Small N; simple, even spread

Table 3.2 gives a brief version. **Maglev hashing** fills a fixed lookup table for $O(1)$ lookups in load balancers (§19.4) at the cost of slightly more disruption [Eisenbud et al. 2016]. **Consistent hashing with bounded loads** caps each node at $(1+\varepsilon)$ times the average and forwards overflow clockwise, a hard bound random vnodes cannot give [Mirrokni et al. 2018].

Failure scenarios

Table 3.3 (§3.3.2) lists generic ring failures. Three more matter for databases.

Table 14.9 — Additional ring failure scenarios in partitioned databases.

What goes wrong	Why	Detection	Recovery	Prevention
Two replicas on one machine or rack	Preference list counted vnodes, not physical nodes or zones	Placement audit	Re-replicate	Skip duplicate nodes and zones when walking
Data unreachable after hash change	Library upgrade changed the hash function or seed	Sudden 100 % miss or not-found rate	Roll back the library	Pin hash algorithm and seed in the ring's metadata
Rebalance overloads cluster	Many vnodes stream at once after a join	Foreground p99 latency rises during join	Throttle streaming	Rate-limit streaming; join one node at a time

COMMON MISTAKE

Mistake — "More vnodes are always better." Balance improves only as $1/\sqrt{V}$, while ranges to repair and stream grow linearly with V. With $V = 256$ and three replicas, a node's ranges overlap most other nodes, so losing almost any two nodes leaves some range without a quorum.

14.9 Rebalancing

14.9.1 What rebalancing must achieve

DEFINITION

Definition — Rebalancing. Moving partitions, or parts of them, between nodes so that data and load stay even as nodes join, leave or fail and as data grows, while staying available and moving no more data than necessary.

First-principles explanation

When a third library room opens, you could renumber every shelf and carry two-thirds of the books around. Or, if the books were already packed in 300 numbered crates, you could carry 100 crates to the new room and update the crate list at the front desk. Rebalancing is cheap when the unit you move is decided *before* you need to move it. It must also be *online*: a store serving 100,000 requests per second cannot pause for a five-hour copy. Kleppmann describes three strategies [Kleppmann 2017].

Why does this exist?

14.9.2 Fixed number of partitions

Create many more partitions than nodes up front, say 1,024 for 16 nodes, and never change the number. A key's partition is fixed forever; only the **partition-to-node map** changes, and a joining node takes whole partitions from the others. Elasticsearch fixes primary shards at index creation, Riak and Couchbase (vBuckets) use fixed partition sets, and the logical shards of Notion and Instagram (§14.7) and Dynamo's final strategy are the same idea.

The difficulty is choosing P. Each partition has fixed overhead, so P cannot be huge, yet P caps the node count and sets the granularity of balance. Choose P at least 10× the largest plausible node count and divisible by likely fleet sizes (§14.7.4). If data grows 100×, each partition does too and may become slow to move.

14.9.3 Dynamic splitting

Start with one partition, or a few, and **split** any partition that passes a size threshold, merging ones that shrink. This adapts to data volume and suits range partitioning, where the key distribution is unknown. Bigtable splits tablets at roughly 100–200 MB [Chang et al. 2006]; HBase regions, MongoDB chunks and distributed SQL ranges split the same way. An empty database starts on one node, so bulk loads should **pre-split** on known boundaries.

14.9.4 Partitions proportional to nodes

Give each node a fixed number of partitions, such as Cassandra's `num_tokens`. A joining node picks that many split points and takes over half of each range it cuts, so the partition count grows with the cluster. This is consistent hashing with vnodes viewed as a rebalancing strategy.

Table 14.10 — Rebalancing strategies compared.

	Fixed partitions	Dynamic splitting	Proportional to nodes
What moves on node join	Whole partitions, map edit	Split halves	Arcs cut by new tokens
Partition count	Constant *P*	Grows with data	Grows with nodes
Partition size	Grows with data	Bounded by threshold	Grows with data per node
Range scans	With hash, no	Yes (range keys)	With hash, no
Main risk	*P* chosen too small or too large	Cold start; split storms	Random imbalance (§14.8.4)
Examples	Elasticsearch, Riak, Couchbase, logical shards, Dynamo	Bigtable, HBase, MongoDB, distributed SQL	Cassandra
Choose when	Predictable growth; simple ops	Range access; unknown key distribution	Peer-to-peer ring stores

WARNING / TRADE-OFF

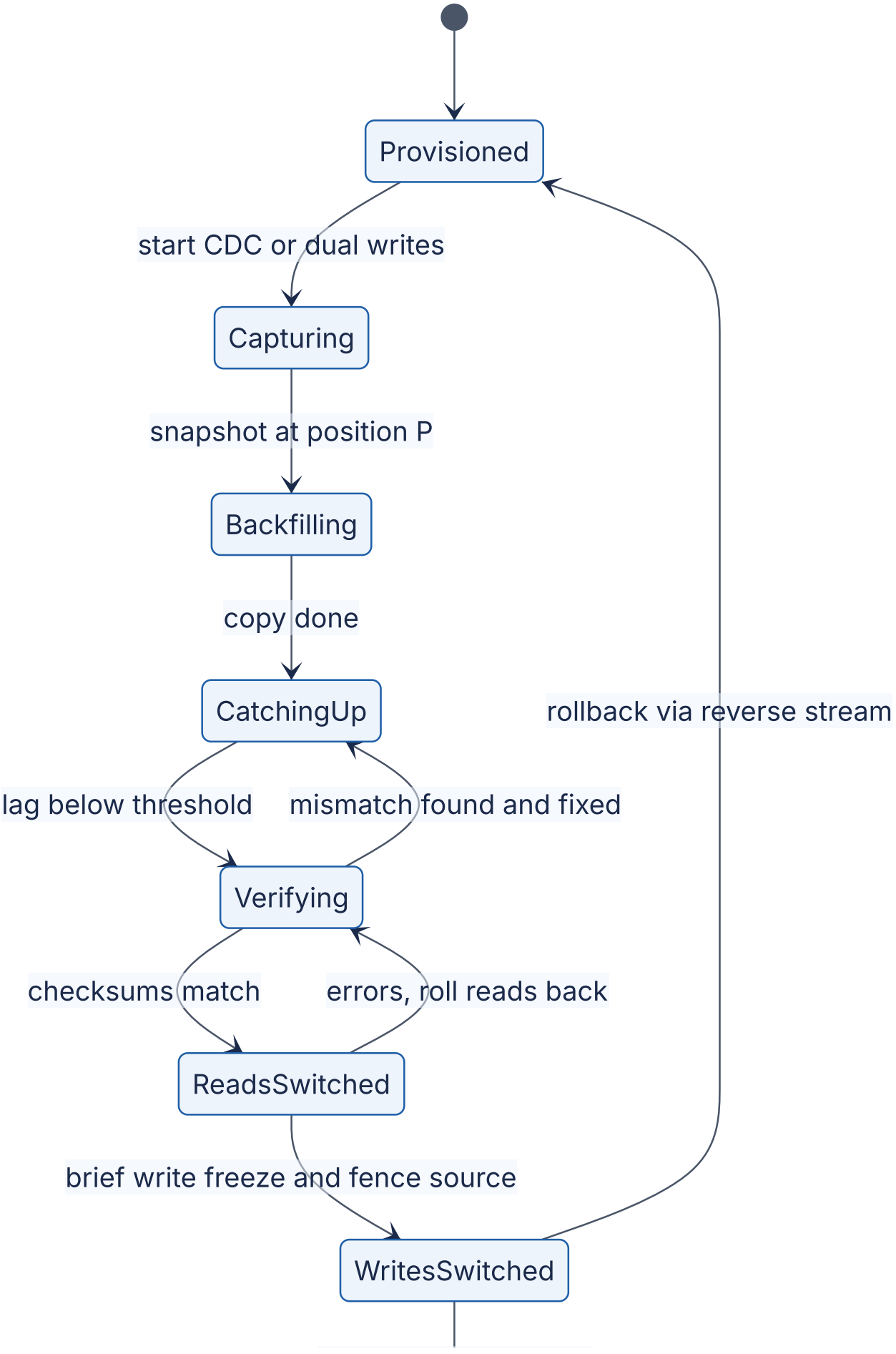
Warning — Fully automatic rebalancing can turn a slow node into an outage. If the failure detector mistakes an overloaded node for a dead one, streaming its partitions away adds load to it and its peers, more nodes look dead, and the cascade grows. Many operators let the system *propose* plans but require a human or rate-limited controller to execute them [Kleppmann 2017].

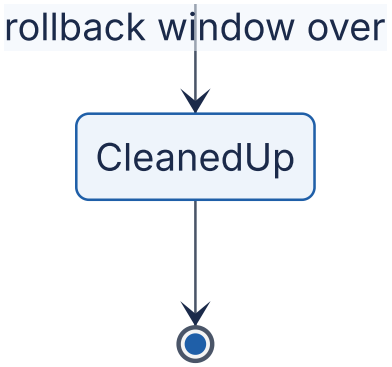
14.9.5 Online migration: dual writes, backfill and cutover

Changing the shard key, splitting a shard or moving a tenant means copying data while the application keeps writing. The safe procedure is the same whether done by Vitess VReplication, a CDC pipeline or by hand:

1. **Provision** the target shards with the new schema and partitioning.
2. **Start capturing changes** at a known source position *before* copying, from the change stream (CDC, §14.4.5) or application **dual writes**.
3. **Backfill** existing rows from a snapshot at that position, throttled.
4. **Catch up** by applying captured changes until the target is seconds behind.
5. **Verify** with per-range checksums and **shadow reads** that query both sides and log differences.
6. **Switch reads**, gradually if possible; the source still takes writes.
7. **Switch writes**: freeze writes for seconds, wait for the target to apply the last position, redirect writes and fence the source.
8. **Keep a reverse stream** to the source for a rollback window.
9. **Clean up** after the window closes.

Figure 14.8 — Online resharding as a state machine.


Provisioned
start CDC or dual writes
Capturing
snapshot at position P
Backfilling
copy done
CatchingUp
rollback via reverse stream
lag below threshold
mismatch found and fixed
Verifying
checksums match
errors, roll reads back
ReadsSwitched
brief write freeze and fence source
WritesSwitched



Reading the diagram. Capture starts *before* the snapshot so no write is missed. Until reads switch, the source alone serves users while the change stream updates the target asynchronously. Backward arrows are the safety net, including the reverse stream after the write switch. The write switch is the riskiest step, since two writable copies would be split brain, hence fencing. Shards migrate in parallel, each throttled.

WORKED EXAMPLE

Worked example — How long does a shard split take? A 3 TB shard is split in two, so 1.5 TB moves to a new shard (SI units). The backfill is throttled to 150 MB/s. During the copy, the moving half of the shard receives 2,000 writes/s, and the target can apply 6,000 writes/s.

1. Backfill: $1.5 \times 10^{12} / 1.5 \times 10^{8} = 10,000$ s, about 2.8 hours.
2. Backlog when the backfill ends: $B = 2,000 \times 10,000 = 2 \times 10^{7}$ writes.
3. Catch-up (§14.4.2): $T = 2 \times 10^{7} / (6,000 - 2,000) = 5,000$ s, about 1.4 hours. Production tools apply changes *during* the backfill, which shortens this.
4. Verification by range checksums: about 1 hour. The cutover freeze lasts a few seconds.

Interpretation. About 5 hours, with seconds of write unavailability. **Engineering implication.** Resharding takes hours per shard and competes for I/O, so start at 60–70 % of a shard's limit, not 95 %, and rehearse the runbook.

INDUSTRY EXAMPLE

Vitess resharding. Vitess's `Reshard` and `MoveTables` VReplication workflows copy rows while streaming binlog changes, compare source and target with `VDiff`, switch reads and writes as separate steps (`SwitchTraffic`) and keep reverse replication so the switch can be undone (`ReverseTraffic`) until the workflow completes. *Evidence: Primary* — [Vitess Docs]

Failure scenarios

Table 14.11 — Failure scenarios during rebalancing and migration.

What goes wrong	Why	Detection	Recovery	Prevention
Rows missing on target	Capture started after the snapshot	Count or checksum mismatch	Re-copy affected ranges	Start capture first; snapshot at a known position
Lost or duplicated writes on cutover	Writes accepted on both sides	Divergent checksums after switch	Reconcile from logs	Freeze and fence the source; idempotent apply
Production latency rises	Backfill saturates disk or network	p99 latency during copy	Lower the throttle	Throttle by measured headroom

COMMON MISTAKE

Mistake — "Dual writes are simple: just write to both." Two independent writes are not atomic. A crash between them, or concurrent writers applying in different orders, makes the stores diverge silently. Prefer deriving the copy from the source's ordered change log (CDC), or reconcile continuously.

14.10 Hot partitions and skew

14.10.1 Causes, detection and mitigation

DEFINITION

Definition — Hot partition and hot key. A **hot partition** receives enough excess requests or data to saturate its node while peers idle. A **hot key** is a single key causing that excess. **Skew** is uneven distribution of data or load in general.

First-principles explanation

A supermarket assigns customers to ten lanes by surname initial. Then a celebrity named K signs autographs at lane 6, and its queue reaches the car park while nine cashiers wait. An eleventh lane does not help; the celebrity's crowd still follows the letter K. The fix is a *different rule for the celebrity*.

Why does this exist?

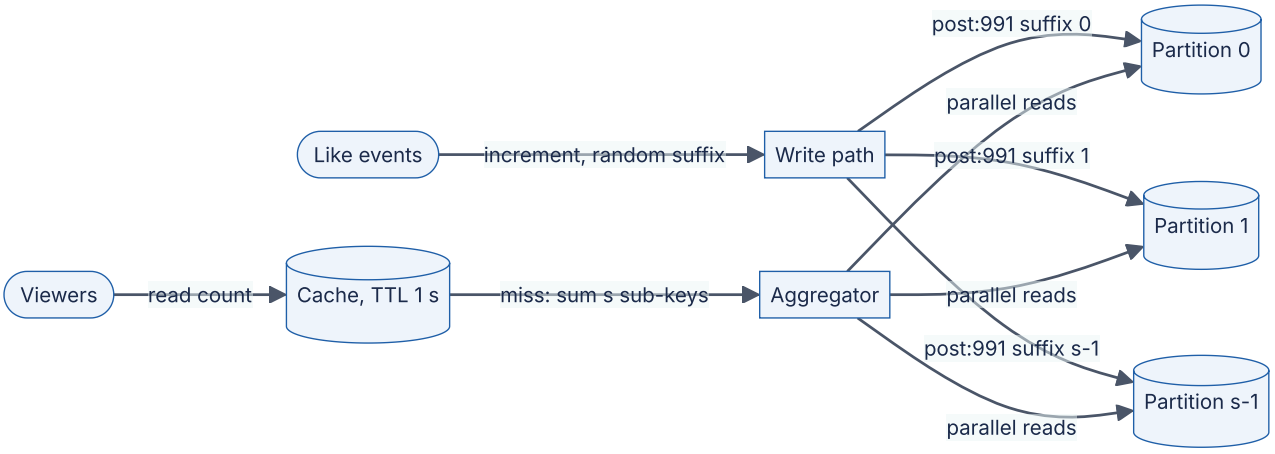
Popularity is typically heavy-tailed (§4.7.2): celebrity accounts, flash sales, the newest partition of a timestamp-ranged table, a giant tenant. Partitioning divides capacity evenly, so any concentration of load becomes a bottleneck. Detect it with per-partition request,

CPU, size and throttling metrics, alerting on the busiest-to-median ratio, and find hot keys with sampling or a heavy-hitters sketch (§17.6). Distinguish **data skew** from **load skew**; they need different fixes.

Mitigation techniques

1. **Key salting.** Append a suffix $0..s-1$ to a hot key, such as `post:991#7`, spreading its writes over s partitions; readers query and combine all s sub-keys. Good for counters and append-only data, bad for objects read as one consistent unit.
2. **Caching and replicas** for hot reads (Chapter 20), with request coalescing so one miss per key reaches the database (§20.11).
3. **Special-case the giants**: dedicated shards (§14.7.2) or fan-out on read for celebrities (§14.7.3).
4. **Avoid monotonic leading keys**: prefix timestamps with a hashed or bucketed component, such as `(sensor_id, day)`.
5. **Bound partition size** with a time bucket, such as `(channel_id, bucket)`.
6. **Adaptive capacity**: the store lends hot partitions unused capacity or splits them by traffic, not just size.
7. **Rate-limit** keys heated by buggy or malicious clients (Chapter 22).

Figure 14.9 — Salting a hot counter across partitions.



Reading the diagram. Each increment gets a random suffix, so each of the s partitions receives $1/s$ of the hot key's writes. Viewers read a cached total (TTL 1 s); only a miss makes the aggregator read all s sub-keys in parallel. Failure points: one slow sub-key partition delays the aggregate, and a cache failure turns every view into s reads. Raise s only for keys detected as hot.

FORMULA

Salt factor for a hot key

$$s = \left\lceil \frac{\lambda_{\text{key}}}{u \cdot c_{\text{part}}} \right\rceil \qquad \text{read amplification} = s$$

Symbol	Meaning	Units
s	Number of sub-keys (salt factor)	sub-keys
λ_{key}	Write rate to the hot key	writes/s
c_{part}	Write capacity of one partition	writes/s
u	Target utilization of a partition	dimensionless (0–1)

WORKED EXAMPLE

Worked example — A viral post's like counter. A post receives $\lambda_{\text{key}} = 50,000$ likes per second at its peak. The store's partitions each sustain $c_{\text{part}} = 1,000$ writes/s, a figure of the same order as DynamoDB's documented per-partition limit of 1,000 write capacity units per second (as of 2026; §12.4.5), and you target $u = 0.7$.

1. Salt factor: $s = \lceil 50,000/(0.7 \times 1,000) \rceil = \lceil 71.4 \rceil = 72$ sub-keys.
2. Writes per sub-key: $50,000/72 \approx 694$ writes/s, which is 69 % of a partition's capacity.
3. Reads without a cache: 20,000 viewers per second each read the count, so $20,000 \times 72 = 1,440,000$ partition reads/s. That is far worse than the problem you solved.
4. Reads with a 1-second cached total: at most 1 aggregation per second per cache node, so with 10 cache nodes, about $10 \times 72 = 720$ partition reads/s.

Interpretation. Salting turns one impossible hot spot into 72 manageable ones but multiplies read cost by s. **Engineering implication.** Pair it with cached or pre-aggregated reads, apply it only to detected hot keys, and accept a count that lags by the TTL.

INDUSTRY EXAMPLE

Amazon DynamoDB. DynamoDB documents per-partition throughput limits plus **burst capacity** (briefly using recently unused capacity) and **adaptive capacity** (letting a hot partition use more than an even share of the table's throughput and isolating frequently accessed items). The 2022 paper describes the evolution from static per-partition allocation, which throttled unevenly accessed tables, to these mechanisms and to splitting partitions by observed traffic. *Evidence: Primary* — [AWS Docs: DynamoDB], [Elhemali et al. 2022]

INDUSTRY EXAMPLE

Discord. Discord stored messages in Cassandra keyed by channel ID plus a time bucket, so a busy channel's history spans many bounded partitions rather than one ever-growing partition, which the post identifies as a performance problem. *Evidence: Primary* — [Discord Engineering 2017]

Failure scenarios

Table 14.12 — Failure scenarios for hot partitions.

What goes wrong	Why	Detection	Recovery	Prevention
One shard throttles while others idle	Celebrity or viral key	Busiest-to-median partition ratio	Cache or salt the key	Hot-key detection; per-key special handling
Newest partition saturated	Timestamp as leading range key	All writes on last range	Re-key with hash prefix	Hash or bucket the leading key
Salted reads become slow	Read fan-out over s partitions	Read p99 grows with s	Cache aggregate	Pre-aggregate; salt only hot keys

14.11 Secondary indexes in partitioned databases

14.11.1 Local and global indexes

Queries by anything other than the partition key, such as orders by `status` in a table partitioned by `customer_id`, need a **secondary index**, and its key differs from the data's key. There are two designs.

DEFINITION

Definition — Local and global secondary indexes. A **local** (document-partitioned) index lives in each partition and covers only its rows. A **global** (term-partitioned) index covers all rows and is partitioned by the *indexed value*, so all entries for one value live together [Kleppmann 2017].

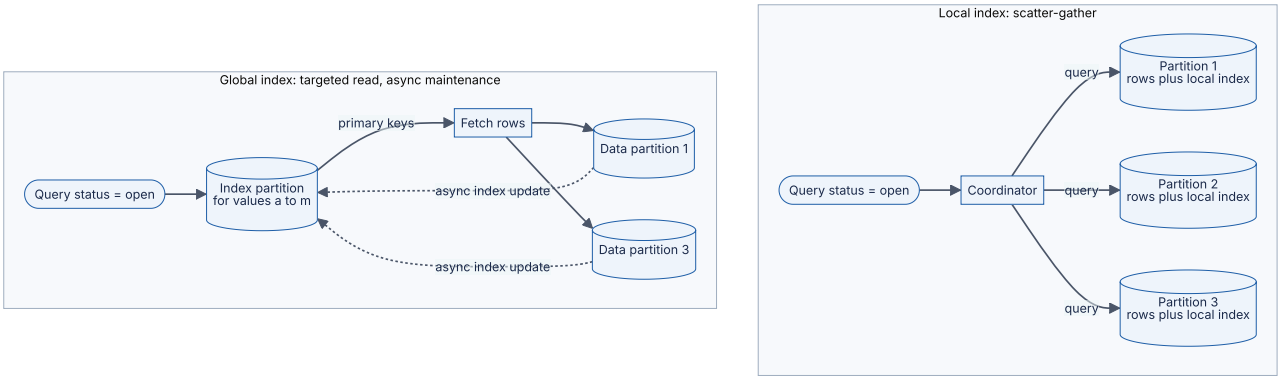
First-principles explanation

Each library room can keep a card catalog of its own books, so finding every Le Guin means checking every room: a local index. Or one central author catalog, itself split by author name across rooms, says where each book is: one visit finds Le Guin, but every new book updates its own room *and* perhaps a catalog section elsewhere: a global index.

How it works internally

- **Local index.** Queries scatter to every partition and a coordinator gathers results. Writes stay in one partition and one local transaction.
- **Global index.** A query visits the one index partition owning the value, gets primary keys, then fetches rows. A write must also update index partitions on other nodes; doing so synchronously needs a distributed transaction (§14.12), so most systems update global indexes **asynchronously** and they briefly lag.

Figure 14.10 — Local versus global secondary index for a query on `status`.



Reading the diagram. Top: the coordinator queries every partition synchronously and waits for the slowest. Bottom: the query reads the one index partition owning `open`, then fetches only the needed rows; dotted arrows show asynchronous index maintenance, so new rows may briefly be missing. Local indexes suffer from slow partitions; global ones from lag and from hot values concentrating on one index partition.

Mathematical foundations

Scatter-gather pays the fan-out tail cost of §4.8.4: if each partition is slow with probability q, a query over n partitions waits for at least one slow response with probability $P_{\text{slow}} = 1 - (1 - q)^n$.

WORKED EXAMPLE

Worked example — A local-index query across 32 shards. Each shard answers within 10 ms at p99, so $q = 0.01$.

1. With $n = 1$, $P_{\text{slow}} = 0.01$.
2. With $n = 32$, $P_{\text{slow}} = 1 - 0.99^{32} = 1 - 0.725 = 0.275$.
3. With $n = 128$, $P_{\text{slow}} = 1 - 0.99^{128} = 1 - 0.276 = 0.724$.

Interpretation. At 128 shards the *median* scatter-gather query is as slow as one shard's p99. **Engineering implication.** Keep scatter-gather off hot paths: use a global index or denormalized copy for frequent non-key lookups, and hedged requests (§23.7) or partial results where fan-out is unavoidable.

Table 14.13 — Local and global secondary indexes compared.

	Local (document-partitioned)	Global (term-partitioned)
Write cost	One partition, local transaction	Row partition + index partition(s) on other nodes
Read by indexed value	All partitions (scatter-gather)	One index partition, then targeted fetches
Index freshness	Synchronous with the row	Usually asynchronous: briefly stale
Tail latency	Grows with partition count	Independent of partition count
Hot indexed values	Spread over partitions	Concentrated on one index partition
Examples	Elasticsearch shards, MongoDB, Cassandra secondary indexes, DynamoDB LSIs	DynamoDB GSIs, distributed SQL global indexes, Vitess lookup vindexes
Choose when	Writes dominate; queries usually include the partition key	Frequent lookups by non-key attribute

INDUSTRY EXAMPLE

Amazon DynamoDB indexes. A **local secondary index** shares the table's partition key, lives with the base data and supports strongly consistent reads. A **global secondary index** has its own partition key, is maintained asynchronously with eventually consistent reads, and has its own throughput, which can throttle base-table writes if it falls behind. *Evidence: Primary* — [AWS Docs: DynamoDB]

COMMON MISTAKE

Mistake — "A global index is as consistent as the table." Usually it is an asynchronously updated copy, so querying it right after a write can miss the row: read-your-writes again (§14.4.3).

14.12 Cross-shard queries and distributed transactions

Partitioning makes single-key operations cheap and everything else harder. This is a preview; Chapter 28 covers distributed transactions and Chapter 32 sagas.

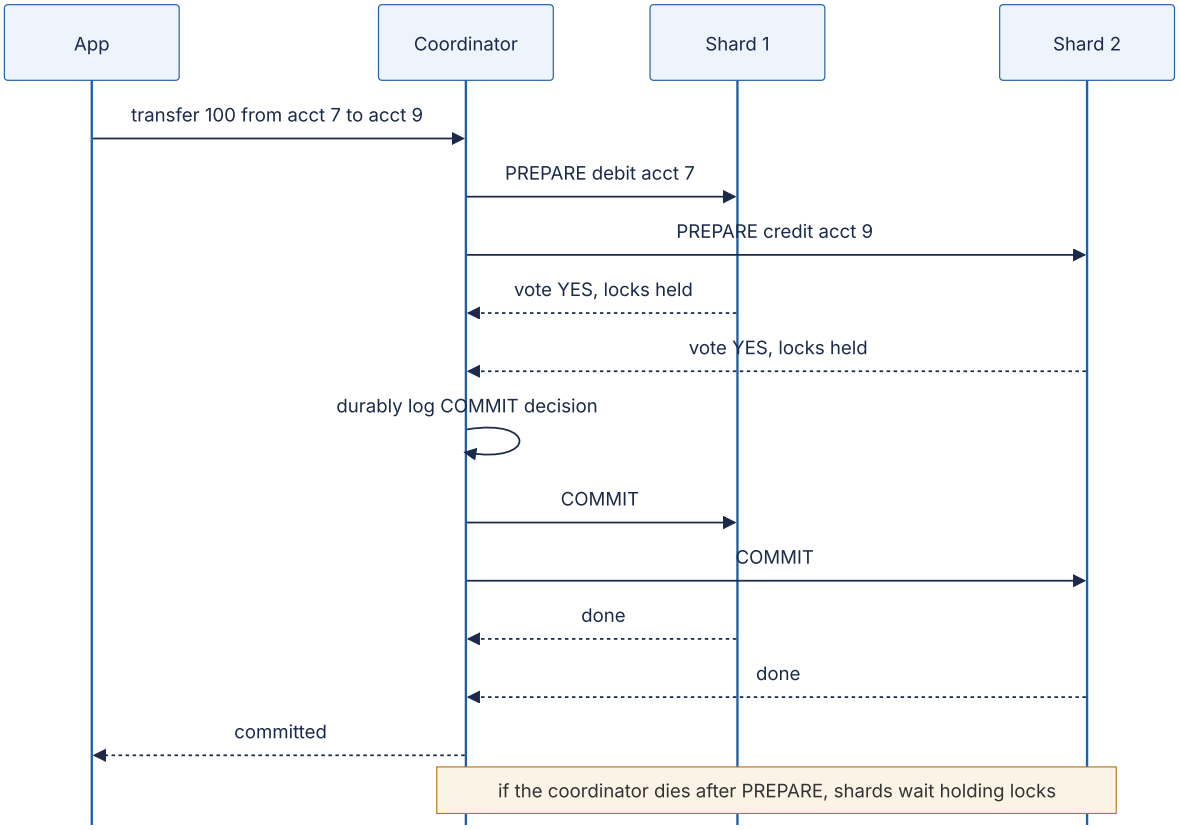
14.12.1 Cross-shard queries

1. **Scatter-gather aggregates** must be decomposed: sum per-shard `COUNT`s, compute `AVG` from per-shard sums and counts (never average averages), and merge per-shard top 10s for `ORDER BY ... LIMIT 10`. Routers such as VTGate do this automatically for supported queries.
2. **Joins** between tables sharded on the same key, such as `orders` and `order_items` by `customer_id`, run locally (**co-location**). Small reference tables can be copied to every shard. Other joins belong in analytical systems (Chapter 29).
3. **Global uniqueness** on a non-key column such as `email` needs a lookup table partitioned by that column.
4. **Global IDs** cannot come from per-shard auto-increment; embed the shard (§14.7.3) or use §17.10's schemes.

14.12.2 Cross-shard transactions: a preview

A transaction writing two shards must commit on both or neither. **Two-phase commit (2PC)** has a coordinator ask every participant to *prepare* (make the change durable but invisible and promise to commit); only if all vote yes does it log the decision and tell them to *commit*.

Figure 14.11 — Two-phase commit across two shards (preview of Chapter 28).



Reading the diagram. Every call is synchronous: two round trips per participant plus a durable decision log at the coordinator. After voting YES, a shard holds locks and *cannot* decide alone, so a coordinator crash blocks those rows (the note). Latency is at least two RTTs plus several fsyncs, and throughput falls as participants increase.

Teams avoid distributed transactions in roughly this order: (1) choose the shard key so transactions stay on one shard (§14.7.2); (2) use a saga of local transactions with compensations (Chapter 32); (3) use a database that implements distributed transactions internally (Chapter 28); (4) use explicit 2PC with careful recovery of in-doubt transactions.

WARNING / TRADE-OFF

Warning — Multi-shard writes through a proxy may not be atomic. A proxy may commit shards one after another, so a failed second commit leaves the first committed. Vitess documents this best-effort default and an optional two-phase commit mode with its own constraints.

14.13 Connection pooling and proxies

14.13.1 Why connections become a scaling problem

DEFINITION

Definition — Connection pooler. A component that keeps a few open database connections and lends them to many clients per session, transaction or statement, either inside the application or as a shared proxy.

First-principles explanation

A restaurant employs waiters for the tables being served, not one per customer who might walk in. Database connections are waiters: each costs server memory even when idle, and most application connections are idle most of the time.

Why does this exist?

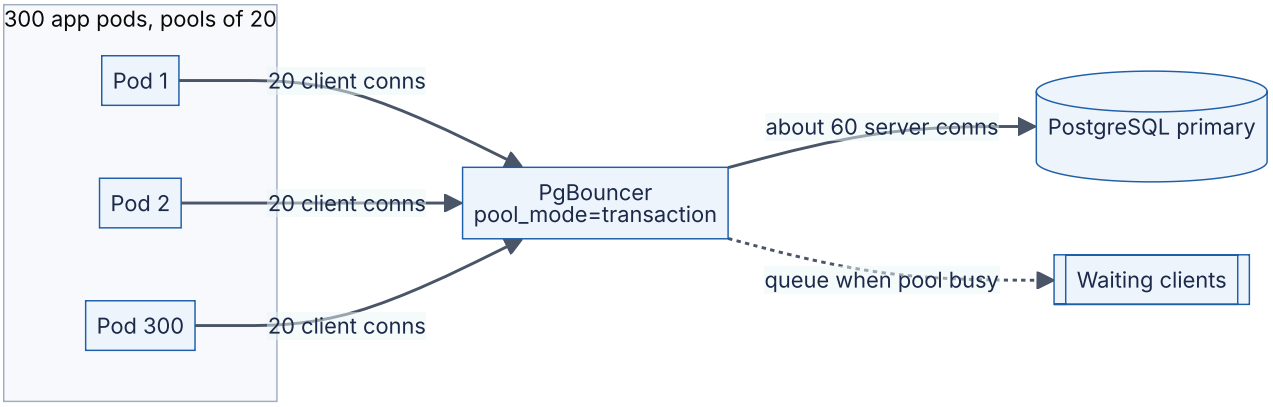
PostgreSQL runs a process per connection and MySQL a thread, so practical limits are hundreds to low thousands per server. Yet 300 application pods with pools of 20 open 6,000 connections while Little's Law says far fewer are busy (§4.11). A pooler accepts thousands of cheap client connections and multiplexes them onto tens of server connections.

How it works internally: PgBouncer

PgBouncer is a lightweight, single-process, event-driven pooler that speaks the PostgreSQL protocol, keeps a pool per database and user pair, and offers three **pool modes** [PgBouncer Docs]:

- **Session pooling** lends a server connection for the client's whole session: fully compatible, small savings.
- **Transaction pooling** lends it for one transaction, the source of the big savings. Session state breaks: session `SET`, `LISTEN`, session advisory locks, temporary tables and `WITH HOLD` cursors. Protocol-level prepared statements were supported in this mode from PgBouncer 1.21 (as of 2026).
- **Statement pooling** returns it after each statement, forbidding multi-statement transactions.

Figure 14.12 — PgBouncer in transaction-pooling mode.



Reading the diagram. PgBouncer holds 6,000 cheap client connections in one event loop and forwards each transaction synchronously over one of about 60 server connections, queuing the rest (dotted). It stores no data. Failure points: PgBouncer itself (run several), long transactions starving the pool, and session features breaking silently. It is single-threaded, so very high connection counts need several instances.

FORMULA

Server connections needed behind a transaction pooler (Little’s Law)

$$C_{\text{server}} = \lambda_{\text{tx}} \cdot W_{\text{tx}} \cdot (1 + h)$$

Symbol	Meaning	Units
C_{server}	Server connections the pool needs	connections
λ_{tx}	Transaction rate through the pooler	transactions/s
W_{tx}	Time a transaction holds a server connection, from `BEGIN` to `COMMIT`	s
h	Headroom fraction for bursts	dimensionless

WORKED EXAMPLE

Worked example — Sizing PgBouncer for 300 pods. The application runs 15,000 transactions/s at peak. Each holds a server connection for 3 ms on average. You add $h = 0.3$ headroom.

1. Average busy connections: $L = 15,000 \times 0.003 = 45$.
2. With headroom: $C_{\text{server}} = 45 \times 1.3 = 58.5$, so configure a pool of about 60.
3. Direct connections would be $300 \times 20 = 6,000$ server processes, 100× more.
4. If a slow query raises W_{tx} to 30 ms, demand becomes 450 connections; clients queue in PgBouncer, *but* the database still sees only 60 concurrent transactions.

Interpretation. The pooler turns unbounded concurrency at the database into a bounded queue in front of it. **Engineering implication.** Size the server pool by Little's Law, `max_client_conn` by client count, and add transaction timeouts so incidents fail fast.

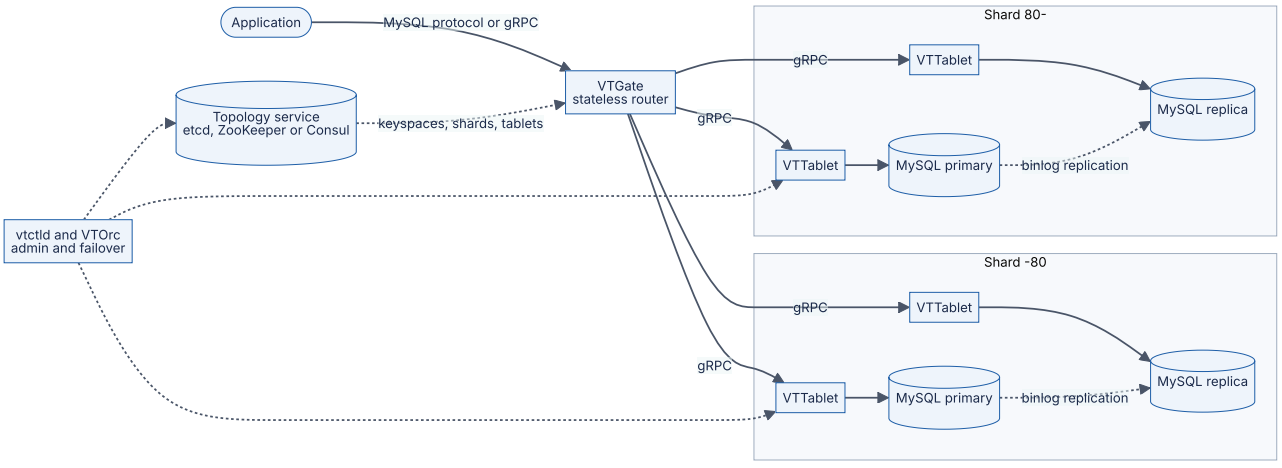
14.13.2 ProxySQL: protocol-aware routing for MySQL

ProxySQL is a MySQL-protocol proxy that groups backends into **hostgroups** (writer, readers) and uses **query rules** to route statements, giving read/write splitting without code changes. It multiplexes connections when safe and can stop sending reads to replicas whose lag exceeds a configured maximum, lag-aware routing at the proxy layer [ProxySQL Docs].

14.13.3 Vitess: a sharding layer for MySQL

Vitess makes a fleet of sharded MySQL databases look like one database. It began at YouTube and is a Cloud Native Computing Foundation project [Vitess Docs].

Figure 14.13 — Vitess architecture.



Reading the diagram. Applications speak MySQL protocol to **VTGate**, a stateless router that uses the **VSchema** and its **vindexes** (sharding functions) to compute each row's **keyspace ID** and route the query, scattering queries that lack the key. Shard `-80` holds keyspace IDs whose first byte is below hex 80 and `80-` the rest (§14.6). A **VTTablet** beside each MySQL instance pools connections and reports health. The strongly consistent topology service holds the shard map, which VTGate watches asynchronously; tablets reject queries for shards they do not serve, catching stale views. Inside a shard, binlog replication is asynchronous or semi-synchronous, and **VTOrc** promotes a replica when a primary fails. Scaling: more VTGates, and `Reshard` to split shards (§14.9).

INDUSTRY EXAMPLE

Slack. Slack described migrating its MySQL datastores to Vitess to shard by keys other than workspace, including channel for its largest customers, while keeping MySQL tooling; it moved table by table, comparing results with shadow traffic first. *Evidence: Primary* — [Slack Engineering 2020]

Table 14.14 — Connection poolers and sharding proxies compared.

	PgBouncer	ProxySQL	Vitess
What it is	PostgreSQL connection pooler	MySQL-protocol routing proxy	Sharding and management layer for MySQL
Pools connections	Yes	Yes	Yes (in VTTablet)
Read/write split	No (separate pools by name)	Yes (query rules)	Yes (tablet types)
Understands shards	No	Only via manual rules	Yes (VSchema, vindexes)
Resharding	No	No	Yes (VReplication workflows)
Failover	No	Detects and reroutes	Yes (VTOrc)
Operational complexity	Low	Medium	High
Choose when	Any PostgreSQL with many clients	MySQL replica fleets	MySQL beyond one primary's write capacity

RECOMMENDATION

Recommendation — Layer the tools in order of need. Start with a client-side pool sized by Little's Law, add PgBouncer or ProxySQL when instance counts make direct connections expensive, and adopt Vitess or distributed SQL (Chapter 28) only when write capacity or data size forces partitioning. Each layer adds a fraction of a millisecond per hop and real operational work.

Failure scenarios

Table 14.15 — Failure scenarios for poolers and proxies.

What goes wrong	Why	Detection	Recovery	Prevention
Wrong data from session settings	`SET search_path` leaks between clients in transaction mode	Intermittent wrong-schema errors	Move setting into each transaction	Use `SET LOCAL`; ban session state
All requests stall	Long transactions hold every server connection	Pool wait time; `cl_waiting` grows	Kill long transactions	Transaction time-outs; separate pools
Proxy becomes a single point of failure	One pooler instance	Health checks fail; total outage	Restart or fail over	Multiple instances; one per host

14.14 Summary and revision notes

SUMMARY

- **Replication** copies data for read capacity and availability; **partitioning** splits data for write capacity and storage. Production systems replicate every partition.
- Replicas cannot scale writes: each applies the whole write stream, so read capacity is $n(uc - w)$.
- Asynchronous replication is fast but can lose recent commits on failover; synchronous or quorum commit adds the k-th fastest replica's RTT; MySQL semi-sync silently degrades to async after a timeout.
- Lag is unbounded and drains at $r_{\text{apply}} - r_{\text{in}}$. It breaks read-your-writes and monotonic reads; fix with primary reads after writes, LSN/GTID watermarks or sticky replicas (§25.6 for theory).
- Prefer row-based or physical replication; use logical replication for upgrades, subsets and CDC. Multi-primary buys local writes at the cost of conflicts; partitioned ownership avoids most of them.
- Range partitioning serves scans but suffers monotonic keys; hash balances but scatters scans; list serves residency; directories place giant tenants. Separate key→partition from partition→node.
- Good shard keys have high cardinality, even load, query and transaction locality, and stability. Shard count is the maximum of storage, write and read terms at the planning horizon.
- Consistent hashing moves an expected $K/(N+1)$ keys on add. With V random vnodes a node's share is $\text{Beta}(V, (N-1)V)$, so its CV is $\sqrt{(N-1)/(NV+1)} \approx 1/\sqrt{V}$ and the busiest node sits near $1 + z_N/\sqrt{V}$.
- Jump hash balances perfectly with no memory but only for numbered buckets; rendezvous hashing balances perfectly at $O(N)$ per lookup.
- Rebalance with fixed partitions, dynamic splitting or per-node tokens; migrate online by capture → backfill → catch-up → verify → switch reads → switch writes → reverse stream.
- Hot keys defeat hashing: salt writes (read cost $\times s$), cache reads, special-case giants, bucket unbounded partitions.
- Local indexes make reads scatter; global indexes make writes cross partitions and lag. Avoid cross-shard transactions by co-location; 2PC blocks on coordinator failure (Chapter 28).
- Poolers turn unbounded database concurrency into a bounded queue sized by Little's Law; Vitess adds routing, resharding and failover for MySQL.

14.15 Knowledge checkpoint

1. Why does adding read replicas eventually stop increasing read capacity?
2. What exactly does MySQL's `AFTER_SYNC` semi-synchronous mode wait for, and when does it stop protecting you?
3. A replica was offline for 20 minutes with $r_{\text{in}} = 2,000$/s and $r_{\text{apply}} = 2,500$/s. How long does catch-up take?
4. Give a sequence of events that violates monotonic reads.
5. Why is statement-based replication unsafe for `UPDATE ... LIMIT 10` without `ORDER BY`?
6. Why does range partitioning on an auto-increment ID create a hot partition?
7. Name the five properties of a good shard key.
8. Under `hash mod N`, what fraction of keys moves going from 9 to 10 nodes? Under consistent hashing?
9. Compute the load CV for $N = 20$ nodes and $V = 50$ vnodes.
10. Why can jump consistent hash not remove an arbitrary failed bucket?
11. Why must change capture start before the backfill snapshot?
12. A counter key receives 12,000 writes/s and partitions sustain 1,000 writes/s at $u = 0.75$. What salt factor do you need?
13. Why are global secondary indexes usually eventually consistent?
14. Which PostgreSQL features break under PgBouncer transaction pooling?

Answers

1. Each replica applies every write, so per-node read budget is $uc - w$; as writes grow it approaches zero no matter how many replicas exist.
2. A replica writing the events to its relay log before the primary commits in the storage engine. After the timeout with no acknowledgment, the primary falls back to asynchronous replication.
3. $B = 2,000 \times 1,200 = 2.4 \times 10^6$; $T = 2.4 \times 10^6 / 500 = 4,800$ s, 80 minutes.
4. A user reads a fresh replica (12 comments), then a more lagged replica (9 comments).
5. Which 10 rows are updated depends on scan order, which can differ on the replica, so copies diverge silently.
6. Every new row has the largest key, so all inserts land in the last range.
7. High cardinality, even load, query locality, transaction locality, stability.
8. $9/10 = 90$ %; about $1/10 = 10$ %.
9. $\sqrt{19/1001} \approx 0.138$, about 14 %.
10. Buckets must be numbered $0..n-1$; only the last can be removed without remapping, so failures are handled inside a replicated bucket.
11. Writes between snapshot and capture start would be in neither copy.
12. $\lceil 12,000/750 \rceil = 16$.

13. Updating index partitions on other nodes synchronously needs a distributed transaction, so most systems update them asynchronously.
14. Session-level `SET`, `LISTEN`, session advisory locks, temporary tables and `WITH HOLD` cursors (and protocol prepared statements before PgBouncer 1.21).

14.16 Interview questions

Basic

Q1. When would you add read replicas, and when would you shard? Answer. Add replicas when reads dominate, can tolerate some staleness and the primary has write headroom; they also provide failover. Shard when writes, data size or working set exceed one primary, or when applying writes consumes most of each replica's budget. State the read/write ratio, freshness needs and growth, then show which limit binds using the shard-count formula.

Q2. Explain consistent hashing and why virtual nodes are used. Answer. Keys and nodes hash onto a ring; each key belongs to the next node clockwise, so adding a node moves only about $K/(N+1)$ keys, all to the new node, versus nearly all under mod-N. With one position per node, shares vary widely (busiest about H_N times fair). With V random vnodes each share is a sum of V arcs, CV $\approx 1/\sqrt{V}$, and a failed node's load spreads over many survivors. Mention weights, preference lists and the alternatives (jump, rendezvous, bounded loads).

Intermediate

Q3. Choose a shard key for a B2B SaaS product where one customer is 500× the median. Answer. `tenant_id`, because nearly all queries and transactions are tenant-scoped, giving locality, isolation and per-tenant deletion. Hash tenants into many logical shards mapped to physical clusters by a directory, put the giant on a dedicated cluster via an override, and if it outgrows that, sub-partition it by a second key such as project. Discuss cross-tenant analytics (move to a warehouse), routing cache staleness and fencing, and the resharding procedure.

Q4. Users report that a post disappears right after they publish it, then reappears. Diagnose and fix. Answer. Reads after the write are served by lagging asynchronous replicas: a read-your-writes violation. Confirm with lag metrics and request traces. Fixes: return the commit LSN/GTID and read only from replicas that have applied it (or wait briefly), or route the author's reads to the primary for a window; add sticky sessions for monotonic reads. Keep lagging replicas out of rotation and investigate why lag spikes (single-threaded apply, bulk jobs).

Advanced

Q5. Reshard a live 40 TB MySQL fleet from 8 to 32 shards with no more than seconds of write downtime. Answer. Introduce a routing layer (Vitess, or a proxy plus directory) if one does not exist. Choose logical shards so future growth needs only moves. For each source shard: start binlog-based capture, snapshot and backfill the target ranges with throttling, catch up, verify with checksums and shadow reads, switch reads, then freeze writes for seconds, wait for the final position, switch writes and fence the source, keeping reverse replication for rollback. Estimate time: splitting each shard into four moves three quarters of the data, 30 TB; at 150 MB/s per stream with 8 streams in parallel that is about 7 hours of copy plus catch-up. Address hot shards, cross-shard queries during transition, and monitoring of lag and p99.

Q6. Your ring-partitioned store has 200 nodes with 32 random vnodes each, and the busiest node runs at 1.6× the mean. Is that expected, and what would you do? Answer. CV $= \sqrt{199/6401} \approx 0.176$, and $z_{200} \approx 2.7$, so the expected busiest node is about $1 + 2.7 \times 0.176 \approx 1.48\times$; $1.6\times$ is plausible from randomness alone, though request skew may contribute, so check per-key heat first. To reduce imbalance: raise V (to reach $1.1\times$ needs about $(2.7/0.1)^2 \approx 730$ vnodes, with repair and streaming costs), allocate tokens deliberately, move to fixed equal-sized partitions, or use bounded-load assignment. For hot keys, salt or cache instead.

14.17 Practical exercises

1. Extend the ring in §14.8.3 with zone awareness so that a preference list never places two replicas in one zone. Measure the balance cost.
2. Modify the experiment in §14.8.4 to measure the busiest node for $N = 1,000$ and compare with $1 + z_N \cdot \mathrm{CV}$.
3. Implement weighted rendezvous hashing and verify that a weight-2 node receives twice the keys of a weight-1 node.
4. Set up PostgreSQL with one streaming standby, run `pgbench`, and measure commit latency under `synchronous_commit` set to `local`, `on` and `remote_apply`.
5. Put PgBouncer in front of PostgreSQL in transaction mode and demonstrate one session-state feature that breaks; then fix the application code.

14.18 Design challenges

1. **Global chat history.** 500 million messages per day, channels from two members to 2 million. Choose partition keys, bounded partitions and indexes for “latest 50 messages in channel” and “search my messages”.

2. **Multi-region SaaS with residency.** EU tenants' data must stay in the EU, writes must be local, and tenants occasionally move region. Combine list partitioning, directories and the migration procedure of §14.9.5.
3. **Flash-sale inventory.** One product receives 200,000 purchase attempts per second for 5 minutes. Design inventory decrements that neither oversell nor saturate one partition.

14.19 References

1. Karger, D., Lehman, E., Leighton, T., Panigrahy, R., Levine, M., Lewin, D. *Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web*. ACM STOC, 1997.
2. Lamping, J., Veach, E. *A Fast, Minimal Memory, Consistent Hash Algorithm*. arXiv:1406.2294, 2014. https://arxiv.org/abs/1406.2294
3. Thaler, D. G., Ravishankar, C. V. *Using Name-Based Mappings to Increase Hit Rates*. IEEE/ACM Transactions on Networking 6(1), 1998.
4. DeCandia, G., et al. *Dynamo: Amazon's Highly Available Key-value Store*. SOSP, 2007.
5. Kleppmann, M. *Designing Data-Intensive Applications* (Ch. 5 "Replication", Ch. 6 "Partitioning"). O'Reilly, 2017.
6. Lakshman, A., Malik, P. *Cassandra: A Decentralized Structured Storage System*. ACM SIGOPS Operating Systems Review 44(2), 2010.
7. Chang, F., et al. *Bigtable: A Distributed Storage System for Structured Data*. OSDI, 2006.
8. David, H. A., Nagaraja, H. N. *Order Statistics* (3rd ed.). Wiley, 2003.
9. Eisenbud, D. E., et al. *Maglev: A Fast and Reliable Software Network Load Balancer*. NSDI, 2016.
10. Mirrokni, V., Thorup, M., Zadimoghaddam, M. *Consistent Hashing with Bounded Loads*. ACM-SIAM SODA, 2018. https://arxiv.org/abs/1608.01350
11. Elhemali, M., et al. *Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service*. USENIX ATC, 2022.
12. Fidalgo, G. *Herding Elephants: Lessons Learned from Sharding Postgres at Notion*. Notion Blog, 2021. https://www.notion.com/blog/sharding-postgres-at-notion
13. Instagram Engineering. *Sharding & IDs at Instagram*. Instagram Engineering Blog, 2012.
14. Discord Engineering. *How Discord Stores Billions of Messages*. Discord Blog, 2017.
15. Slack Engineering. *Scaling Datastores at Slack with Vitess*. Slack Engineering Blog, 2020.
16. PostgreSQL Global Development Group. *PostgreSQL Documentation: High Availability, Load Balancing, and Replication; Logical Replication; synchronous_commit*. https://www.postgresql.org/docs/current/

17. Oracle. *MySQL Reference Manual: Replication (formats, semisynchronous replication) and Group Replication*. https://dev.mysql.com/doc/refman/8.4/en/replication.html
18. PgBouncer project. *PgBouncer Documentation: Configuration and Features*. https://www.pgbouncer.org/
19. ProxySQL project. *ProxySQL Documentation*. https://proxysql.com/documentation/
20. The Vitess Authors. *Vitess Documentation: Architecture, VSchema, VReplication workflows*. https://vitess.io/docs/
21. Amazon Web Services. *Amazon DynamoDB Developer Guide: partition key design, burst and adaptive capacity, secondary indexes*. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/
22. Apache Software Foundation. *Apache Cassandra Documentation: data distribution and virtual nodes*. https://cassandra.apache.org/doc/latest/

14.20 Open verification items

- Cassandra 4.0 default `num_tokens = 16` with the token-allocation algorithm, and the availability rationale for lowering it.
- PgBouncer version (1.21) that added protocol-level prepared statements in transaction mode.
- DynamoDB per-partition limit (1,000 WCU/s) and burst/adaptive capacity wording, as of 2026.
- Vitess two-phase-commit mode status and default multi-shard transaction semantics.
- URLs and authors for Instagram (2012), Discord (2017) and Slack (2020) engineering posts; Slack migration details.
- Lamping and Veach 3.2 % / 8 % figures confirmed via search snippets only (arXiv not reachable).
- Bigtable tablet split size (~100–200 MB).

Chapter 15 — Object, Block and File Storage

> **Level:** 2–3 · **Track:** A (Traditional) · **Estimated reading time:** 2.5 h

Chapters 11–14 were about databases: systems that store small, structured records and answer questions about them. Most of the bytes a company stores are not records at all. They are photos, videos, log files, backups, machine-learning datasets and database snapshots: large, mostly immutable blobs that are written once and read many times or never. In Chapter 1, PhotoShare's ADR-0003 moved every photo into "managed object storage", and at 10 million daily users that meant about 5 TB of new data per day (§1.9). This chapter opens that box. You will learn the three storage interfaces (blocks, files, objects), how S3-style object stores and GFS-style file systems are built, how erasure coding buys high durability for 1.4× raw capacity instead of 3×, why metadata is the real scaling limit, and how chunking, multipart upload and lifecycle tiers keep petabytes affordable.

15.1 Learning objectives

After this chapter you should be able to:

- Choose between block, file and object storage by interface, semantics, latency and throughput.
- Draw an S3-style object store (front end, metadata/index, placement, storage nodes) and trace a PUT and a GET through it.
- Explain GFS (single master, 64 MB chunks, leases, record append) and HDFS NameNode high availability.
- Calculate the storage overhead, fault tolerance, approximate durability and repair traffic of 3× replication, Reed–Solomon RS(10,4) and an Azure-style local reconstruction code.
- Estimate metadata memory and explain how namespaces are partitioned.
- Size a multipart upload, use presigned URLs safely, and explain why content-defined chunking survives insertions.
- Design and cost a lifecycle policy across hot, warm, cold and archive tiers.

15.2 Prerequisites

- Disks, SSDs, IOPS and throughput — §5.6 (block devices are the bottom layer of every design here).

- File systems, `fsync` and the page cache — §6.6 and §6.14 (file semantics and durability).
- MTBF, AFR and durability as a race between failure and repair — §4.15–§4.16 (the loss formula is generalized in §15.6).
- Hash functions and HMAC — §10.4.1 and §10.4.6 (content addressing and pre-signed URLs).
- Partitioning and consistent hashing — §14.6 and §14.8 (metadata and data placement).
- Units: in this chapter **MB/GB/TB/PB** are powers of ten for capacity and cost; **KiB/MiB/GiB** appear where a system defines a limit in powers of two (for example S3's 5 MiB minimum part size).

15.3 Block, file and object storage compared

15.3.1 Three interfaces to the same disks

> DEFINITION
>
> **Definition — Block, file and object storage. Block storage** exposes a numbered array of fixed-size blocks (typically 512 B or 4 KiB) that a client reads and writes by offset; the client's own file system or database gives the blocks meaning. **File storage** exposes a hierarchical namespace of directories and byte-addressable files with POSIX-like operations (open, read at offset, write at offset, rename, lock), shared by many clients over a protocol such as NFS or SMB. **Object storage** exposes a flat namespace of immutable **objects**, each a byte string plus metadata addressed by a key within a **bucket**, accessed over HTTP with whole-object PUT, GET (optionally a byte range), DELETE and LIST.

First-principles explanation

Picture a warehouse. **Block storage** rents you a bare floor marked in numbered squares: you decide what goes where and keep your own map; the warehouse knows nothing about your boxes. **File storage** is a filing cabinet shared by an office: folders inside folders, anyone can open a document, change page 7 and put it back, and the cabinet makes sure two people do not scribble on the same page at once. **Object storage** is a coat check: you hand over a sealed parcel, get a ticket (the key), and later hand the ticket back to get the identical parcel. You cannot change one page of a parcel; you hand in a new parcel under the same ticket.

The more an interface promises (in-place updates, shared locks, atomic rename of a directory tree), the more coordination every operation needs and the harder it is to scale

out. Object storage promises the least, which is why it routinely scales to trillions of items.

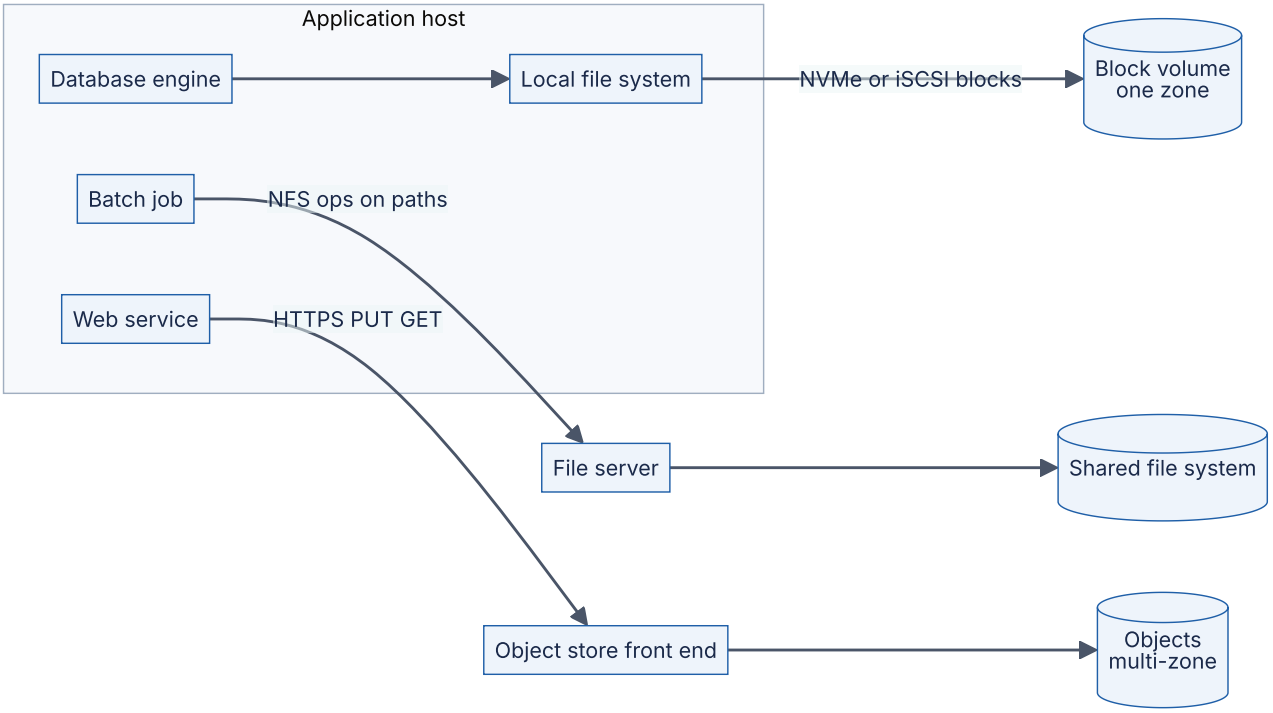
Why does this exist?

All three exist because applications need different contracts. A database engine (Chapter 13) wants raw blocks so that it controls page layout, write-ahead logging and `fsync` ordering itself. Many machines sharing a directory of model checkpoints want a POSIX file system. A web application storing uploads wants unlimited capacity, HTTP access and someone else to worry about durability; it never rewrites byte 4,000 of a photo.

How it works internally

1. **Block.** A volume is a virtual disk. Cloud services such as Amazon EBS and Google Persistent Disk attach a network volume to one VM over NVMe or iSCSI-style protocols, replicate its blocks within one zone, and let the guest run ext4, XFS or a database on it.
2. **File.** An NFS or SMB server (or a distributed service such as Amazon EFS, which speaks NFSv4.1) owns the namespace. Clients send `LOOKUP`, `READ(offset, count)`, `WRITE`, `RENAME`; the server keeps inodes, directories, permissions and locks and serializes conflicting updates.
3. **Object.** A client sends `PUT /bucket/key` with the full body; the service stores the bytes redundantly, records key → location in an index, and acknowledges. GET looks up the key and streams the bytes. There is no rename, append or partial overwrite: a “change” is a new PUT of the whole object.

Figure 15.1 — Three storage interfaces over shared disks.



Reading the diagram. Three clients on one host use three interfaces. The *database engine* runs its own file system on a *block volume*: the protocol carries block reads and writes by offset, and all meaning (files, pages) lives on the host. The *batch job* sends path-based NFS operations to a *file server*, which owns the namespace and locking for every client that mounts it. The *web service* sends whole-object HTTPS requests to an *object store front end*. All arrows are synchronous request/response. Storage boundaries: the block volume lives in one zone and attaches to one VM; the file system is shared; objects are spread across zones. Failure points: a zone outage takes the block volume with it (restore from snapshots); the file server's metadata path is a shared bottleneck. Scaling: block volumes scale up (bigger, faster volume); file systems scale up or partition the namespace (§15.7); object stores scale out horizontally.

Mathematical foundations

The single most useful model for comparing the three is the time to fetch S bytes: a fixed first-byte latency plus a transfer term.

FORMULA

Access time for one read

$$t \approx t_{\text{first}} + \frac{S}{B} \qquad t_{\text{parallel}} \approx t_{\text{first}} + \frac{S}{\min(c \cdot B,\ B_{\text{NIC}})}$$

Symbol	Meaning	Units
t	Time to read the whole request	s
t_{first}	Time to first byte (network round trip, lookup, seek or flash read)	s
S	Bytes requested	bytes
B	Throughput of one stream (connection or volume)	bytes/s
c	Number of parallel streams (for example byte-range GETs)	count
B_{NIC}	Client network interface limit	bytes/s

WORKED EXAMPLE

Worked example — Small random reads versus bulk reads. Use illustrative values: a network SSD block volume with $t_{\text{first}} = 0.5$ ms and $B = 125$ MB/s; an object store with $t_{\text{first}} = 30$ ms and $B = 80$ MB/s per connection; a 10 Gb/s client NIC ($B_{\text{NIC}} = 1.25$ GB/s).

1. 4 KiB (4,096 B) read, block: $0.5 + 4096/125,000,000 \times 1000 = 0.5 + 0.033 = 0.53$ ms.
2. 4 KiB read, object: $30 + 4096/80,000,000 \times 1000 = 30 + 0.05 = 30.05$ ms, about 57× slower.
3. 1 GB read, block (one volume): $0.0005 + 10^9/1.25 \times 10^8 = 8.0$ s.
4. 1 GB read, object, one connection: $0.03 + 10^9/8 \times 10^7 = 12.5$ s.
5. 1 GB read, object, 16 parallel range GETs: $\min(16 \times 80, 1250) = 1250$ MB/s, so $0.03 + 0.8 = 0.83$ s.

Interpretation. For tiny random reads the fixed latency dominates and the block volume wins by more than an order of magnitude. For bulk reads the transfer term dominates, and the object store wins because it lets you open many independent streams to many storage nodes, while one volume has a fixed throughput cap.
Engineering implication. Put latency-sensitive random I/O (database pages) on block storage; put large, sequentially read data (media, logs, training data, backups) on object storage and read it with parallel range requests. Never use an object store as if it were a disk for a database.

Table 15.1 — Block, file and object storage compared. Latency figures are typical orders of magnitude (as of 2026), not guarantees.

Aspect	Block	File	Object
What it is	Virtual disk of numbered blocks	Shared directory tree of files	Flat bucket of immutable key → blob
Interface	NVMe/iSCSI/SCSI commands	NFS, SMB, POSIX	HTTP REST: PUT, GET, DELETE, LIST
Update model	Overwrite any block in place	Overwrite/append at any offset; rename; locks	Replace whole object (new version)
Sharing	One host (multi-attach rare)	Many hosts concurrently	Unlimited clients over the internet
Typical first-byte latency	~0.1–2 ms (network SSD)	~1–10 ms	~10–100+ ms (single-digit ms for premium one-zone classes)
Throughput scaling	Per-volume cap	Per file system; often bursts	Near-linear with parallel requests
Consistency	Strong (single writer)	Close-to-open (NFS) to strong	Strong read-after-write (S3 since Dec 2020; GCS; Azure Blob)
Cost (relative)	Highest per GB, pay for provisioned size	High	Lowest, pay for bytes stored and requests
Examples	Amazon EBS, Google Persistent Disk, Azure Managed Disks	NFS, Amazon EFS, Google Filestore, Azure Files, Lustre	Amazon S3, Google Cloud Storage, Azure Blob Storage, Ceph RGW, MinIO
Choose when	Databases, boot disks, low-latency random I/O	Shared POSIX access, legacy apps, HPC scratch	Media, backups, logs, data lakes, ML datasets

Real-world industry applications

INDUSTRY EXAMPLE

Cloud block, file and object services. AWS documents EBS gp3 volumes with a baseline of 3,000 IOPS and 125 MiB/s that can be provisioned higher, and EBS volumes are created in a single Availability Zone; Amazon EFS is accessed with NFSv4.1 and stores data across multiple zones in its regional classes; Amazon S3 is designed for 99.999999999% durability and, since December 2020, provides strong read-after-write consistency for PUT and DELETE of objects and for LIST, automatically and at no extra cost [AWS S3 consistency docs; AWS EBS docs]. Google Cloud Storage documents strong global consistency for read-after-write, read-after-delete and object listing [Google Cloud Storage consistency docs]. *Evidence: Primary* — [AWS S3 consistency docs; AWS EBS docs; Google Cloud Storage consistency docs]

Common misconceptions

COMMON MISTAKE

“S3 is a file system.” Tools that mount a bucket as a directory make it look like one, but `rename` becomes copy-plus-delete of every object, appending means rewriting the whole object, and there are no locks. Programs that assume POSIX semantics (for example, writing a file in place and renaming it atomically) can be slow or incorrect on such mounts.

15.4 Object storage architecture (the S3 model)

15.4.1 Buckets, keys and a flat namespace

DEFINITION

Definition — Bucket, key, object and flat namespace. A **bucket** is a named container with its own access policy, region and configuration. An **object** is an immutable byte string (from zero bytes to tens of terabytes) plus system metadata (size, content type, checksum, version ID) and user metadata. A **key** is the object’s full name within the bucket, for example `photos/2026/10/u42/abc.jpg`. The namespace is **flat**: the slashes are ordinary characters, and “folders” are an illusion produced by listing keys that share a **prefix** and grouping them at a **delimiter** (usually `/`).

First-principles explanation

A hierarchical file system is a tree, and trees need coordination: renaming a directory atomically moves everything beneath it, and creating a file must check and update its parent directory. If millions of clients create files in the same directory, that one directory becomes a lock everyone fights over. A flat namespace removes the tree. Each key is independent, so the system can sort all keys and split the sorted list into ranges, giving each range to a different metadata server, exactly like range partitioning a database table (§14.6). Listing “the folder” `photos/2026/10/` becomes a range scan from that prefix to the next one.

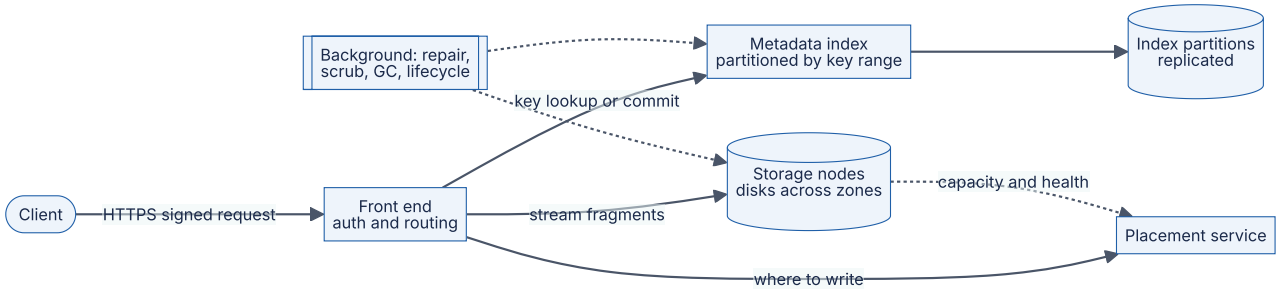
Why does this exist?

The web needed storage reachable over HTTP that never filled up. Immutable whole objects make that tractable: no partial overwrites to replicate, no locks to hold across a network partition, and bytes that never change, so caches and CDNs (Chapter 21) can store them safely.

15.4.2 Components of an S3-style object store

AWS has published only fragments of S3's internals (cited below). The design that follows is a generic, teaching architecture that matches the public behavior of S3, Google Cloud Storage, Azure Blob Storage and open-source systems such as Ceph's RADOS Gateway; it is not a description of any one product.

Figure 15.2 — Generic S3-style object store (illustrative).



Reading the diagram. The *client* sends an HTTPS request signed with its credentials. The stateless *front end* authenticates, authorizes and routes it. The *metadata index* maps (bucket, key, version) to the locations of the object's data; it is partitioned by key range so that LIST by prefix is a range scan, and each partition is replicated (a consensus-replicated store is the usual choice, Chapter 26). The *placement service* chooses which storage nodes, in which failure domains, receive a new object's fragments, using asynchronous health and capacity reports (dotted). *Storage nodes* own disks and store fragments with checksums. All client-facing arrows are synchronous. *Background workers* (dotted, asynchronous) re-create lost fragments, scrub checksums, garbage-collect unreferenced data and apply lifecycle rules (§15.9). Storage boundaries: small, hot, strongly consistent metadata is separated from huge, append-only data. Failure points: an unavailable index partition makes intact data unreachable; storage-node failures are routine and repaired in the background. Scaling: add front ends for request rate, split index partitions for key-space hot spots, add storage nodes for capacity.

How it works internally

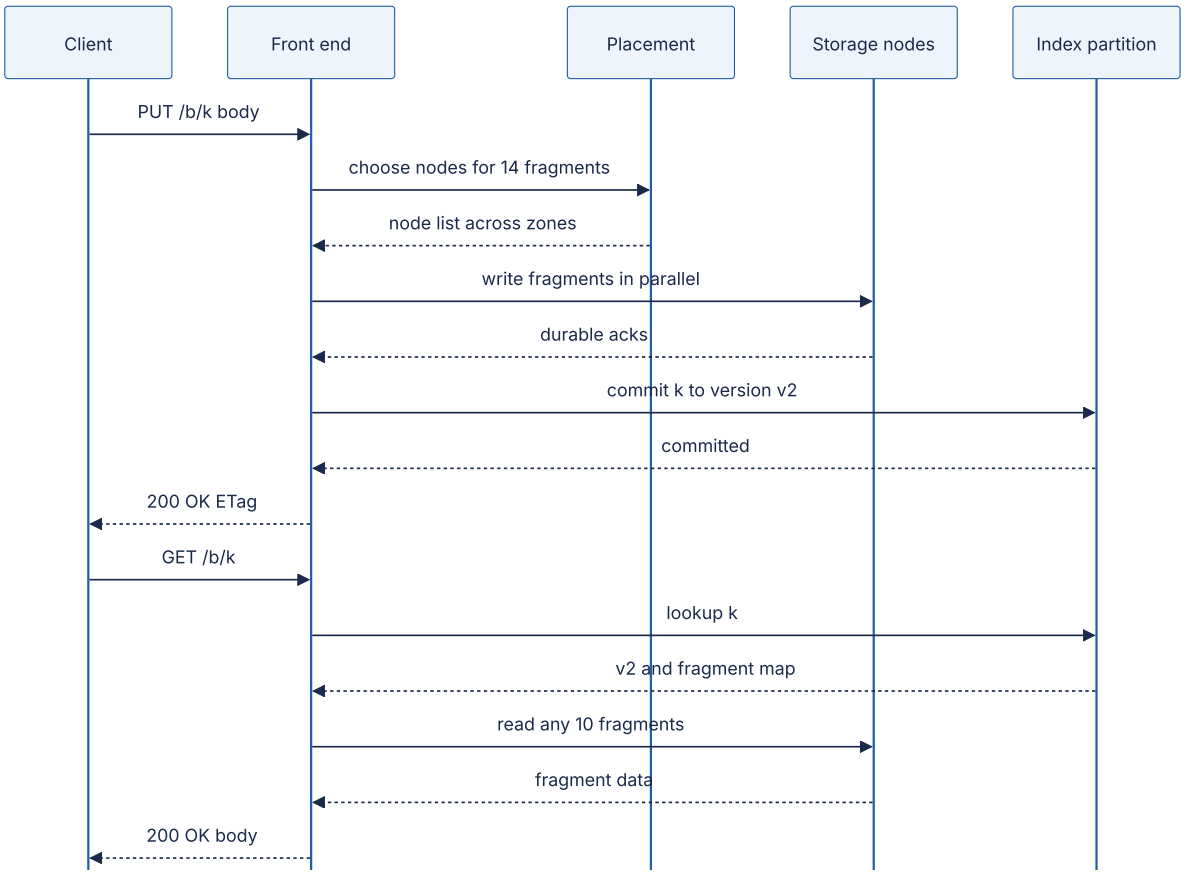
PUT (write path):

1. The front end authenticates the request (for example AWS Signature Version 4, an HMAC over the request, §10.4.6) and checks the bucket policy.
2. It asks the placement service for a set of storage nodes spread across failure domains (disks, racks, zones).
3. It streams the body, splits it into chunks if it is large, and writes each chunk redundantly: replicas or erasure-coded fragments (§15.6). It computes a checksum and verifies any checksum the client sent.

4. Only after enough fragments are durable does it **commit** a new metadata record (key, version ID, size, checksum, fragment locations) to the index partition that owns the key. This commit is the linearization point: before it, readers see the old version (or nothing); after it, every reader sees the new one.
5. It returns `200 OK` with an ETag. Data written by a PUT that failed before step 4 is unreferenced and later removed by garbage collection.

GET (read path): authenticate → look up the key's latest committed version in the index → read fragments from storage nodes (only k of $k+m$ are needed for an erasure-coded object, so the front end can ignore the slowest) → verify checksums → stream to the client.

Figure 15.3 — PUT then GET in an S3-style store (illustrative).



Reading the diagram. Five participants exchange synchronous HTTP or RPC messages, top to bottom in time. On the write, data flows to the *storage nodes* before any metadata changes; the *index partition* commit is the single step that makes version v2 visible. On the read, the *front end* first resolves the key to a version and a fragment map, then fetches fragments; in this RS(10,4) illustration any 10 of 14 suffice. Failure points: if the front end crashes after writing fragments but before the commit, the client sees an error and retries, and the orphaned fragments are garbage; if it crashes after the commit but before replying, the object exists and a retry simply writes it again (PUT of the same bytes is idempotent). Scaling: the fragment writes and reads fan out in parallel, so large objects are limited by aggregate node bandwidth, not by one disk.

Consistency: what “strong read-after-write” means

Because the index commit is a single atomic step per key, a store that keeps each index partition linearizable (§25.3) can promise that once a PUT returns, any subsequent GET or LIST anywhere sees it. Historically S3 cached metadata in a way that let reads miss recent writes; AWS described adding a cache-coherence protocol in which a “witness” component is notified of every object change, so that front ends can tell when their cached metadata is stale [Vogels 2021]. Two caveats remain. Concurrent PUTs to one key are not merged: the last commit wins, so read-modify-write needs conditional writes (`If-None-Match`, `If-Match`, supported by S3 since 2024). Bucket configuration and cross-region replication remain asynchronous.

Mathematical foundations

Index partitions have finite throughput, so request rate per key range is a design input. AWS documents that an application can achieve at least 3,500 PUT/COPY/POST/DELETE or 5,500 GET/HEAD requests per second per partitioned prefix, and that S3 scales by partitioning further as load grows [AWS S3 performance docs].

FORMULA

Prefixes needed for a target request rate

$$N_{\text{prefix}} = \lceil \frac{R_{\text{peak}}}{r_{\text{prefix}}} \rceil$$

Symbol	Meaning	Units
N_{prefix}	Distinct key prefixes over which requests must be spread	count
R_{peak}	Peak request rate of one type (writes or reads)	requests/s
r_{prefix}	Sustained rate one partitioned prefix supports	requests/s

WORKED EXAMPLE

Worked example — Log ingestion into an object store. A logging pipeline writes 20,000 small objects per second at peak with keys like `logs/2026-10-30T12/<host>/<seq>`.

1. $N_{\text{prefix}} = \lceil 20,000/3,500 \rceil = \lceil 5.71 \rceil = 6$ prefixes at minimum.
2. Problem: every key in the current hour shares one leading prefix, so all writes land in one key range until the store splits it.
3. Fix: put a short hash of the host first, for example `logs/a7/2026-10-30T12/<host>/<seq>` with 16 hex buckets, giving at least 16 prefixes and headroom of $16 \times 3,500 = 56,000$ writes/s.

Interpretation. Time-ordered keys concentrate load on the newest range (the hot-partition problem of §14.10). **Engineering implication.** Design keys for the index, not for humans: spread writes with a hashed component and keep the time component later in the key. Better still, batch 20,000 tiny records per second into a few larger objects (§15.8), which also cuts request fees.

Real-world industry applications

INDUSTRY EXAMPLE

Amazon S3 — what AWS has published. AWS's report on the February 2017 US-EAST-1 disruption names two S3 subsystems: an *index subsystem* that manages the metadata and location information of all objects in the region, and a *placement subsystem* that allocates storage for new objects and depends on the index subsystem; both had to be fully restarted [AWS 2017]. AWS engineers have described ShardStore, the key-value storage engine on S3 storage nodes that holds shards of customer objects, and its validation with lightweight formal methods [Bornholt et al. 2021]. Vogels described the metadata cache-coherence protocol behind strong consistency [Vogels 2021]. Anything beyond these statements in Figure 15.2 is illustrative. *Evidence: Primary* — [AWS 2017; Bornholt et al. 2021; Vogels 2021]

INDUSTRY EXAMPLE

Facebook Haystack — packing small objects. Storing each photo as a file cost Facebook several disk reads per photo just for directory and inode metadata. Haystack appends many photos ("needles") into large volume files and keeps an in-memory index from photo ID to (volume, offset, size), so a read needs about one disk operation [Beaver et al. 2010]. The lesson generalizes: an object store's per-object metadata cost, not its byte capacity, is what small objects stress. *Evidence: Primary* — [Beaver et al. 2010]

Advantages and disadvantages

Object storage gives unlimited capacity, high durability, parallel throughput and the lowest price per GB, in exchange for tens of milliseconds of first-byte latency, per-request charges, no in-place updates or multi-key rename, and coordination left to the application.

Failure scenarios

What goes wrong	Why	Detection	Recovery	Prevention
Orphaned storage	PUTs that failed after writing data, abandoned multipart uploads	Storage used exceeds sum of indexed sizes	Garbage collection; abort-incomplete-upload lifecycle rule	Make GC and lifecycle rules part of the design
Lost update	Two clients read-modify-write the same key	Missing data reported by users	Restore from versioning	Conditional writes (`If-Match`) or a database as the source of truth

Common misconceptions

COMMON MISTAKE

“Renaming a folder is cheap.” There is no folder. Renaming `photos/2026/` means copying and deleting every object under that prefix, one request each. Design keys so that you never need to rename them, and keep human-facing names in a database.

15.5 Distributed file systems: GFS, HDFS, Colossus and Tectonic

15.5.1 The Google File System (GFS)

DEFINITION

Definition — Distributed file system. A file system whose namespace and data are spread across many machines, so that files can be larger than any one disk and throughput can grow with the number of machines, while clients still use file-like operations (create, open, read, write or append, delete).

First-principles explanation

Think of a library with one head librarian and a thousand shelves in separate rooms. The librarian keeps only the catalog: which book is split into which volumes and which rooms hold each volume. Readers ask the librarian once, then walk directly to the rooms. The li-

brarian never carries books, so one person can manage a huge library. That is the GFS design.

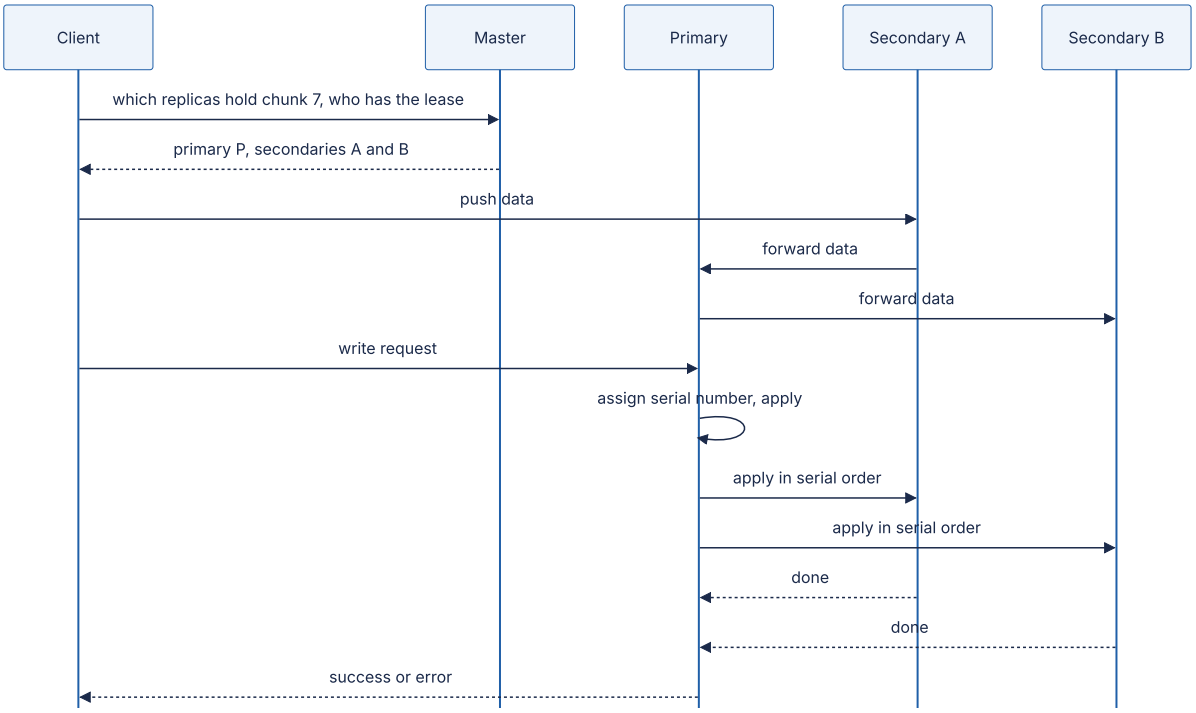
Why does this exist?

Google needed to store web crawls and indexes on thousands of commodity machines, some always failing, for workloads dominated by large sequential reads and concurrent *appends*. GFS was designed for exactly that: component failure as the norm, multi-gigabyte files, appends, and high sustained bandwidth over low latency [Ghemawat et al. 2003].

How it works internally

1. **One master, many chunkservers.** Files are split into fixed **64 MB chunks**, each identified by a globally unique 64-bit handle and stored as a Linux file on **chunkservers**, by default as three replicas. A single **master** holds all metadata in memory: the namespace, the file → chunk mapping, and chunk locations. The master persists namespace and mapping changes in an **operation log** replicated to other machines, with periodic checkpoints; it does *not* persist chunk locations but learns them from chunkservers at start-up and through heartbeats.
2. **Why 64 MB.** Large chunks mean few chunks, so the master needs less than 64 bytes of metadata per chunk and clients rarely need to ask it anything; a client caches chunk locations and talks to chunkservers directly. The cost: a popular small file is one chunk on three servers, a hot spot.
3. **Reads.** The client converts (file, byte offset) into a chunk index, asks the master for the handle and replica locations, caches them, and reads from the nearest replica.
4. **Leases and mutation order.** For each chunk being modified the master grants a **lease** (initially 60 seconds, renewable through heartbeats) to one replica, the **primary**. The primary picks a serial order for all mutations to that chunk, so all replicas apply them in the same order without the master on the data path. (Leases in general: §26.11.)
5. **Decoupled data and control flow.** The client pushes data to all replicas along a chain of chunkservers, each forwarding to the next nearest while still receiving (pipelining), so each machine's full outbound bandwidth is used. Only after all replicas hold the data does the client send the small write request to the primary.
6. **Record append.** The client supplies only data; the primary chooses the offset and appends the record atomically *at least once*. A failed append is retried, so replicas may contain duplicates or padding, which applications skip using checksums and unique record IDs.
7. **Housekeeping.** Each chunk has a version number to detect stale replicas that missed mutations while down; deleted files are renamed to hidden names and garbage-collected lazily; chunkservers checksum every 64 KB block on read.

Figure 15.4 — GFS write with a lease-holding primary.



Reading the diagram. The *client* asks the *master* (control flow) which replica holds the lease, and caches the answer. Data flows along a chain ordered by network topology, not roles, so the client uploads each byte once. The small write request goes to the *primary*, which assigns a serial number and tells both *secondaries* to apply mutations in that order. All messages are synchronous RPCs. Storage boundaries: metadata on the master, chunk bytes only on chunkservers. Failure points: a failed secondary makes the client retry (the region is inconsistent until it succeeds); a dead primary's lease expires and the master grants a new one. Scaling: the master handles only small metadata RPCs, so data bandwidth grows with the number of chunkservers.

Mathematical foundations

FORMULA

Metadata memory for an in-memory namespace

$$M = N_{\text{files}} \cdot b_{\text{file}} + N_{\text{chunks}} \cdot b_{\text{chunk}} \qquad N_{\text{chunks}} \approx \sum_{\text{files}} \lceil \frac{\text{file size}}{C} \rceil$$

Symbol	Meaning	Units
M	Memory needed by the metadata server	bytes
N_{files}	Number of files (and directories)	count
N_{chunks}	Number of chunks or blocks	count
$b_{\text{file}}, b_{\text{chunk}}$	Bytes of metadata per file and per chunk	bytes
C	Chunk (block) size	bytes

WORKED EXAMPLE

Worked example — Why chunk size and file count limit the master. Use GFS's figure of under 64 bytes per chunk and a commonly quoted HDFS rule of thumb of about 150 bytes per file or block object in the NameNode heap.

1. GFS, 1 PB of large files in 64 MB chunks: $10^{15} / 6.4 \times 10^{7} = 1.56 \times 10^{7}$ chunks; $\times$ 64 B $= 1.0 \times 10^{9}$ B ≈ 1 GB. One machine copes easily.
2. Same 1 PB in 4 MB chunks: 16× as many chunks, about 16 GB. Still feasible, but every client request now also hits the master 16× as often.
3. HDFS cluster with 300 million files averaging 1.5 blocks: $N = 3 \times 10^{8} + 4.5 \times 10^{8} = 7.5 \times 10^{8}$ objects; $\times$ 150 B $= 1.1 \times 10^{11}$ B ≈ 113 GB of heap.
4. The same bytes stored as 3 billion small files: at least 6×10^{9} objects, about 900 GB, beyond any practical single heap.

Interpretation. A single metadata server scales with the *number* of files and chunks, not with bytes. **Engineering implication.** Single-master designs work for large files; small-file workloads must either pack files together (Haystack, §15.4; HAR or sequence files in Hadoop) or move to a partitioned metadata service (§15.7).

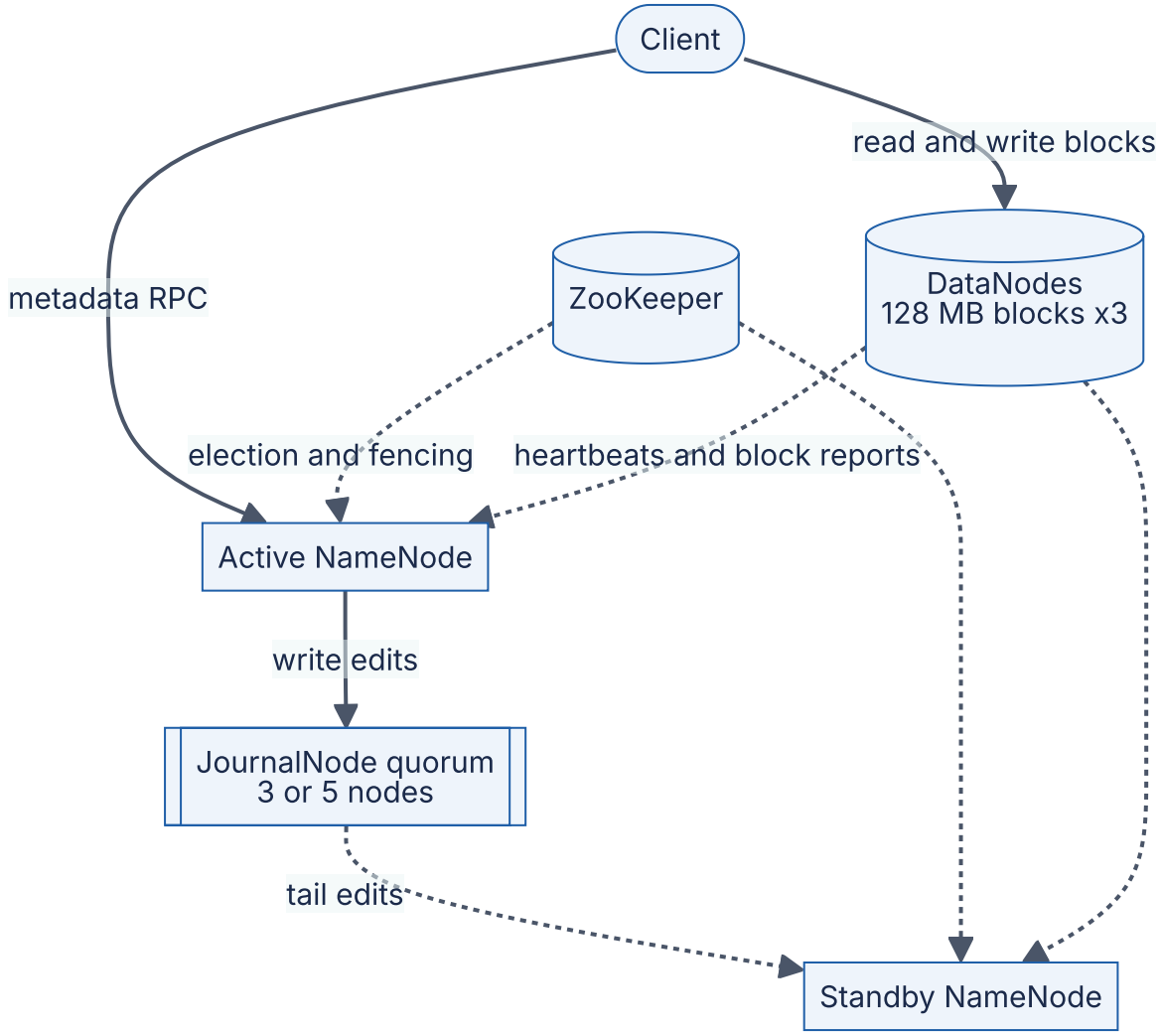
15.5.2 HDFS and NameNode high availability

HDFS, the Hadoop Distributed File System, follows the GFS design with different names: a **NameNode** (the master) keeps the namespace and block map in RAM, and **DataNodes** (chunkservers) store **blocks**, typically 128 MB, with a default replication factor of three

[Shvachko et al. 2010]. DataNodes send heartbeats every few seconds and periodic block reports; a DataNode silent for about ten minutes is considered dead and its blocks are re-replicated. Writes go through a pipeline of DataNodes, as in GFS. HDFS supports single-writer append rather than concurrent record append, and places replicas rack-aware.

The original NameNode was a single point of failure. Hadoop 2 added **high availability**: an *active* and a *standby* NameNode share the edit log through a quorum of **JournalNodes**, DataNodes report blocks to both, and a ZooKeeper-based failover controller elects the active one and fences the old one [Apache Hadoop HA docs]. **Federation** runs several independent NameNodes, each owning part of the namespace, to go past one heap's limit.

Figure 15.5 — HDFS with NameNode high availability.



Reading the diagram. The *client* sends metadata RPCs to the *active NameNode* and streams block data directly to and from *DataNodes*. The active NameNode writes every namespace change to a majority of *JournalNodes* (synchronously) before acknowledging; the *standby* tails those edits asynchronously and stays nearly current. *DataNodes* report

to both, so the standby knows block locations at failover. *ZooKeeper* (dotted) holds the election lock; failover controllers on each NameNode use it to promote the standby and fence the old active, preventing split brain (§26.12–§26.13). Storage boundaries: the namespace is in NameNode RAM backed by the journal; block bytes are on DataNodes. Failure points: active NameNode loss costs seconds of metadata unavailability; losing a JournalNode majority blocks writes. Scaling: data scales with DataNodes; metadata scales only by federation.

15.5.3 Colossus and Tectonic: distributing the master

Both GFS and HDFS hit the same wall: one metadata server's memory and request rate. Their successors keep the client library and chunk servers but move metadata into a scalable database.

INDUSTRY EXAMPLE

Google Colossus. Google has published only an overview. A 2021 Google Cloud blog describes GFS's successor as a client library, scalable metadata services ("curators") storing metadata in Bigtable, "D" file servers holding data, and background "custodians" for durability and rebalancing; data is replicated or Reed–Solomon encoded per workload [Hildebrand and Serenyi 2021]. *Evidence: Primary* — [Hildebrand and Serenyi 2021]

INDUSTRY EXAMPLE

Facebook Tectonic. Tectonic, an exabyte-scale multitenant file system, consolidated Facebook's blob storage (Haystack and f4) and its data-warehouse HDFS clusters. Its metadata is split into Name, File and Block layers (Figure 15.8) stored in ZippyDB, a sharded key-value store, behind stateless metadata services; data flows directly between client library and chunk-store nodes. ZippyDB lacks cross-shard transactions, so some recursive operations are restricted. Blocks are replicated or Reed–Solomon encoded; blob storage appends small writes to replicas and Reed–Solomon encodes them later [Pan et al. 2021]. *Evidence: Primary* — [Pan et al. 2021]

Alternatives and trade-offs

Table 15.2 — GFS, HDFS and Tectonic compared.

Aspect	GFS (2003)	HDFS (2010)	Tectonic (2021)
Metadata	Single in-memory master + replicated log	NameNode in RAM; HA standby; federation	Sharded key-value store, stateless services
Unit	64 MB chunk	128 MB block (configurable)	Block of chunks
Redundancy	3× replication	3× replication; erasure coding from Hadoop 3	Replication or Reed–Solomon per tenant
Write semantics	Concurrent record append, at least once	Single writer, append	Single writer per file; tenant-specific
Scaling limit	Master memory and RPC rate	NameNode heap	Key-value store shards (horizontally scalable)

Failure scenarios

What goes wrong	Why	Detection	Recovery	Prevention
Duplicate records after append	Client retried a partially failed record append	Duplicate record IDs downstream	Deduplicate on read	Unique record IDs and checksums in the payload
Re-replication storm	Rack failure loses thousands of blocks at once	Under-replicated block count spikes	Throttled, prioritized re-replication (blocks with one copy first)	Rack-aware placement; repair bandwidth budgets

15.6 Replication versus erasure coding

15.6.1 Reed–Solomon codes

DEFINITION

Definition — Erasure code and RS(k, m). An **erasure code** transforms k data fragments into $k + m$ fragments such that the original data can be rebuilt from some subset of them after the others are lost (“erased”). A **Reed-Solomon** code RS(k, m) is a **maximum distance separable (MDS)** code: *any* k of the $k + m$ fragments suffice, so it tolerates the loss of any m fragments. A code is **systematic** when the first k fragments are the unmodified data, so healthy reads need no decoding.

First-principles explanation

Store 5, 7 and 2 together with their sum, 14. Lose any one of the four and you can recompute it: $14 - 5 - 2 = 7$. RAID-5 does this with XOR: one parity block protects against any one loss. Reed–Solomon generalizes it. Treat the k data symbols as the values of a polynomial of degree less than k at k known points. Two points fix a line and three fix a parabola; in general k points fix a degree-$(k - 1)$ polynomial. Evaluate that polynomial at m extra points

and store those values as parity. Any k surviving values pin down the polynomial again, and therefore all the data. The arithmetic is done in a finite field (in practice GF(2^8), so each symbol is one byte) so that values never grow and division always works [Reed and Solomon 1960; Plank 1997].

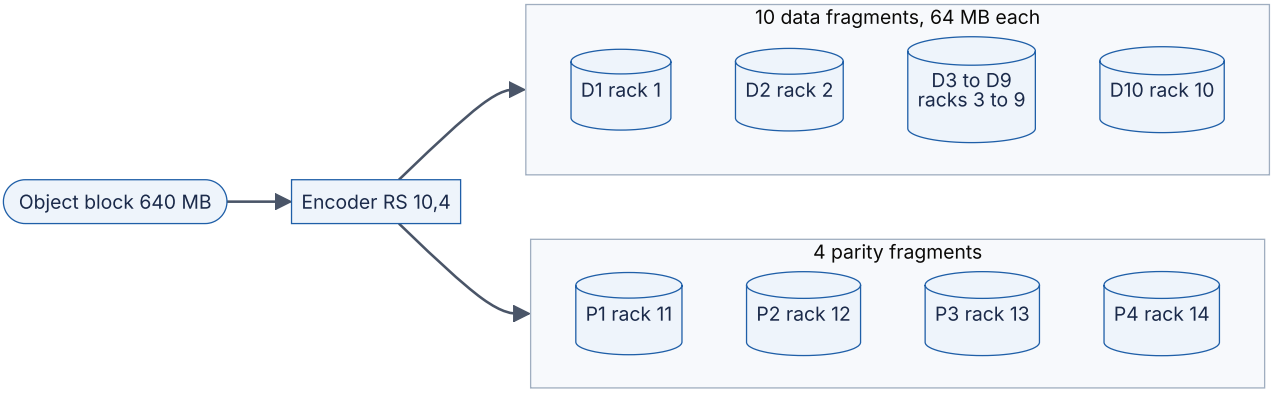
Why does this exist?

Three-way replication tolerates two failures but stores 3 bytes per byte. At exabyte scale the extra copies cost real money: hundreds of thousands of disks, the racks and the power for them. Erasure coding reaches the same or better durability for 1.2–1.5× overhead. Facebook's f4 warm-blob store reports moving from an effective replication factor of 3.6 in Haystack to 2.1 by using RS(10,4) within a datacenter and XOR across datacenters [Muralidhar et al. 2014].

How it works internally

1. Split an object (or a large block) into k equal data fragments, padding the last.
2. Compute m parity fragments, byte position by byte position, by multiplying the data symbols by a fixed encoding matrix (or, equivalently, evaluating the polynomial at the parity points).
3. Place all $k + m$ fragments of a **stripe** on different disks, ideally different racks or zones, so one failure domain holds at most a few fragments.
4. Reads normally fetch the k data fragments (systematic code). If one is slow or missing, fetch any parity fragment instead and decode: a degraded read.
5. Repair: when a disk dies, for each stripe that lost a fragment, read k surviving fragments, decode, and write the lost one to a new disk.

Figure 15.6 — An RS(10,4) stripe across 14 failure domains.



Reading the diagram. A 640 MB block enters the *encoder*, which writes ten 64 MB data fragments and four 64 MB parity fragments, each to a different rack. Arrows are synchronous writes. Failure points: any four of the fourteen racks can fail with no data loss; a fifth concurrent loss before repair loses the stripe. Raw capacity used is $14 \times 64 = 896$ MB for 640 MB of data (1.4×), versus 1,920 MB for three replicas. Scaling: stripes are inde-

pendent, so encoding and repair parallelize across the cluster, but rebuilding one fragment reads ten, from ten racks.

Mathematical foundations

FORMULA

Overhead, fault tolerance and repair reads of a (k, m) code

$$\text{overhead} = \frac{k+m}{k} \qquad \text{tolerated losses} = m \qquad \text{repair read per lost fragment} = r \times F$$

Symbol	Meaning	Units
k	Data fragments per stripe	count
m	Parity fragments per stripe	count
overhead	Raw bytes stored per logical byte	dimensionless (×)
r	Fragments read to rebuild one: 1 for replication, k for RS, local group size for LRC	count
F	Fragment size	bytes

Replication is the special case $k = 1$: three-way replication is a (1, 2) code with overhead 3× and $r = 1$.

FORMULA

Approximate annual probability of losing a stripe (generalizing §4.16)

$$P_{\text{loss/yr}} \approx \frac{n!}{(n-m-1)!}\,\lambda^{m+1}\,T^{m} \times H, \qquad n = k+m,\ \ \lambda T \ll 1$$

Symbol	Meaning	Units
n	Fragments per stripe (replicas for replication)	count
m	Losses tolerated	count
λ	Per-fragment (per-disk) failure rate, AFR/8,760	failures/hour
T	Mean time to detect and rebuild one lost fragment	hours
H	Hours per year (8,760)	hours/year

Derivation. Exactly as in §4.16: the first loss happens at rate $n\lambda$; the stripe is lost if m more of the survivors fail, one after another, each before the previous repair completes, with probabilities $(n-1)\lambda T, (n-2)\lambda T, \ldots, (n-m)\lambda T$. Multiplying gives $n(n-1)\cdots(n-m)\,\lambda^{m+1}T^m$ per hour. For three replicas ($n=3$, $m=2$) this is $6\lambda^3T^2$, the §4.16 result.

WORKED EXAMPLE

Worked example — 3× replication versus RS(10,4): durability and cost. Use the §4.16 assumptions: AFR 2%, so $\lambda = 2.28\times10^{-6}$ per hour; $T = 24$ h, so $\lambda T = 5.48\times10^{-5}$. Note that $\lambda^{m+1}T^mH = \mathrm{AFR}\times(\lambda T)^m$.

1. 3× replication: $\frac{3!}{0!} = 6$; $P = 6\times0.02\times(5.48\times10^{-5})^2 = 6\times0.02\times3.00\times10^{-9} = 3.6\times10^{-10}$ per group per year.
2. RS(10,4): $n=14$, $m=4$; $\frac{14!}{9!} = 14\times13\times12\times11\times10 = 240,240$; $(\lambda T)^4 = 9.01\times10^{-18}$; $P = 240,240\times0.02\times9.01\times10^{-18} = 4.3\times10^{-14}$ per stripe per year.
3. Per petabyte with 64 MB fragments: replication has $10^{15}/6.4\times10^7 = 1.56\times10^7$ groups, so 5.6×10^{-3} expected loss events per year; RS(10,4) stripes hold 640 MB of data, so 1.56×10^6 stripes and 6.8×10^{-8} loss events per year.
4. Cost for 10 PB of logical data at an illustrative $10 per raw TB-month: replication needs 30 PB raw, $300,000 per month; RS(10,4) needs 14 PB raw, $140,000 per month, saving $1.92 million per year.

Interpretation. Under independent failures RS(10,4) is both cheaper (1.4× against 3×) and far more durable, because it tolerates four losses instead of two: about 8,000 times less likely to lose a given stripe, and, since each stripe holds ten times as much data as a replica group, about 80,000 times fewer expected loss events per petabyte.
Engineering implication. Erasure-code large, cold or warm data. The independence assumption is optimistic for both schemes (§4.16): correlated failures dominate real loss [Ford et al. 2010], so spread a stripe's fragments across racks and zones, and remember the hidden costs below — CPU for encoding, latency for small objects, and repair traffic.

FORMULA

Daily repair read traffic in a fleet

$$R_{\text{day}} = \frac{N_{\text{disk}} \times \text{AFR}}{365} \times D_{\text{used}} \times r$$

Symbol	Meaning	Units
R_{day}	Bytes read per day to rebuild failed disks	bytes/day
N_{disk}	Disks in the fleet	count
AFR	Annualized failure rate	fraction/year
D_{used}	Data stored per disk	bytes
r	Fragments read per fragment rebuilt	count

WORKED EXAMPLE

Worked example — Repair bandwidth for a 10,000-disk cluster. Disks of 20 TB at 60% full ($D_{\text{used}} = 12$ TB), AFR 2%.

1. Failures per day: $10,000 \times 0.02/365 = 0.548$.
2. Data lost per day: $0.548 \times 12 = 6.58$ TB.
3. 3× replication ($r = 1$): 6.58 TB/day of reads.
4. RS(10,4) ($r = 10$): 65.8 TB/day, a sustained $65.8 \times 10^{12}/86,400 = 761$ MB/s, about 6.1 Gb/s, most of it crossing racks.
5. LRC(12,2,2) (below, $r = 6$ for one lost data fragment): 39.5 TB/day, about 457 MB/s.

Interpretation. Erasure coding saves capacity but multiplies repair traffic by k. Briefly unavailable machines (reboots, upgrades) add more reconstruction: a study of a Facebook warehouse cluster using RS(10,4) reported recovery traffic on the order of a hundred terabytes per day crossing racks [Rashmi et al. 2013]. **Engineering implication.** Budget network capacity for repair, delay repair of transient failures for a few minutes, and consider codes that reduce r.

Code example

The program below implements XOR parity and a toy systematic Reed–Solomon-style code. For readability it works over the prime field GF(257) with Lagrange interpolation, so a parity symbol can be 256; production libraries (for example Intel ISA-L or Jerasure) use GF(2^8), but the "any k of $k + m$" property is identical.

```
        # A toy erasure code: XOR parity (m = 1) and a Reed-Solomon-style (k, m) code.
        # The RS code works over the prime field GF(257) so the arithmetic is plain
        # modular integer maths; production systems use GF(2^8) so every symbol is a
byte.
        import itertools
        import random

        P = 257  # prime modulus; symbols are integers 0..256


        def xor_parity(blocks):
            """RAID-5 style parity: byte-wise XOR of all data blocks."""
            parity = bytearray(len(blocks[0]))
            for b in blocks:
                for i, byte in enumerate(b):
                    parity[i] ^= byte
            return bytes(parity)


        def lagrange_at(points, x):
            """Evaluate, at x, the unique polynomial of degree < len(points)
            passing through points [(xi, yi)], all arithmetic mod P."""
            total = 0
            for i, (xi, yi) in enumerate(points):
                num, den = 1, 1
                for j, (xj, _) in enumerate(points):
                    if j != i:
                        num = num * (x - xj) % P
                        den = den * (xi - xj) % P
                total = (total + yi * num * pow(den, P - 2, P)) % P  # pow(.., P-2) =
inverse
            return total


        def rs_encode(data, k, m):
            """Systematic encoding of one stripe column: fragment i (i = 1..k) holds
            data[i-1]; parity fragments k+1..k+m hold the polynomial's values there."""
            pts = list(zip(range(1, k + 1), data))
            return data + [lagrange_at(pts, x) for x in range(k + 1, k + m + 1)]


        def rs_decode(survivors, k):
            """Rebuild the k data symbols from ANY k surviving (index, value) pairs."""
            pts = survivors[:k]
            return [lagrange_at(pts, x) for x in range(1, k + 1)]


        if __name__ == "__main__":
            # 1. XOR parity: lose any one block, rebuild it from the others.
            d = [b"GFS!", b"HDFS", b"S3ok"]
            p = xor_parity(d)
            rebuilt = xor_parity([d[0], d[2], p])  # pretend d[1] was lost
            print("XOR rebuilt block 1:", rebuilt, rebuilt == d[1])

            # 2. RS(4, 2): 4 data symbols, 2 parity symbols, survives ANY 2 losses.
            k, m = 4, 2
```

```
            data = [ord(c) for c in "EC4u"]
            stripe = rs_encode(data, k, m)
            print("stripe (data + parity):", stripe)
            ok = 0
            for lost in itertools.combinations(range(k + m), m):
                survivors = [(i + 1, v) for i, v in enumerate(stripe) if i not in lost]
                ok += rs_decode(survivors, k) == data
            print(f"RS({k},{m}) recovered data in {ok} of "
                  f"{len(list(itertools.combinations(range(k + m), m)))} two-loss pat-
terns")

            # 3. RS(10, 4) on random bytes: lose 4 random fragments, decode, compare.
            rng = random.Random(7)
            k, m = 10, 4
            data = [rng.randrange(256) for _ in range(k)]
            stripe = rs_encode(data, k, m)
            lost = set(rng.sample(range(k + m), m))
            survivors = [(i + 1, v) for i, v in enumerate(stripe) if i not in lost]
            print("RS(10,4) lost fragments", sorted(lost), "-> recovered:",
                  rs_decode(survivors, k) == data, "| symbols read for repair:", k)
```

Output (Python 3):

```
XOR rebuilt block 1: b'HDFS' True
stripe (data + parity): [69, 67, 52, 117, 98, 88]
RS(4,2) recovered data in 15 of 15 two-loss patterns
RS(10,4) lost fragments [1, 6, 12, 13] -> recovered: True | symbols read for repair: 10
```

What to notice:

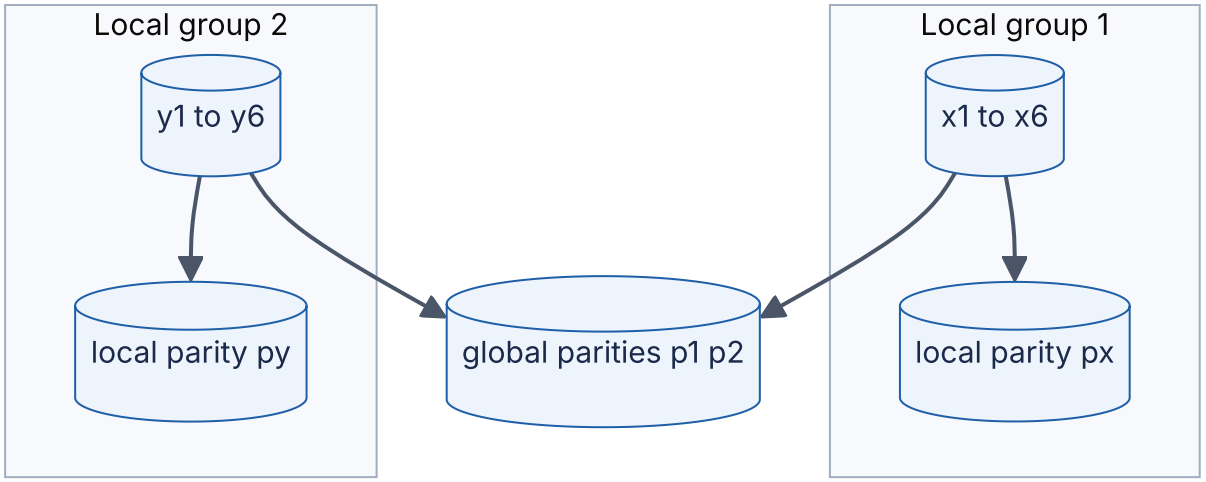
- XOR parity is its own inverse: XOR-ing the survivors with the parity yields the missing block. It tolerates exactly one loss.
- `rs_encode` keeps the data unchanged (systematic) and appends parity, so healthy reads never decode.
- The RS(4,2) loop tries all $\binom{6}{2} = 15$ ways to lose two fragments and recovers every time: the MDS property.
- Rebuilding any fragment reads k others; that is the repair-traffic multiplier of the formula above. Real codes apply this per byte position across fragments of many megabytes.

15.6.2 Local reconstruction codes

Repair cost led Microsoft to design **local reconstruction codes (LRC)** for Windows Azure Storage. LRC(k, l, g) splits the k data fragments into l local groups, adds one XOR-like **local parity** per group, and adds g **global parities** computed over all data. A single lost data fragment, by far the most common case, is rebuilt from its local group only. Azure's LRC(12,2,2) uses two groups of six: overhead $16/12 = 1.33\times$, a single-fragment repair reads 6 fragments instead of 12, it tolerates any three failures, and it also survives a

large fraction of four-failure patterns [Huang et al. 2012]. Compared with RS(6,3), which also reads 6 fragments per repair, LRC(12,2,2) stores less (1.33× against 1.5×) with similar durability.

Figure 15.7 — Azure-style LRC(12,2,2) stripe.



Reading the diagram. Twelve data fragments form two *local groups* of six. Each group has its own *local parity*, computed only from that group; the two *global parities* are computed from all twelve. Arrows are write-time dependencies; all sixteen fragments sit in different failure domains. Failure points and recovery: if `x3` fails, the system reads `x1`, `x2`, `x4`, `x5`, `x6` and `px` (six fragments) and never touches group 2; only rarer multi-failure patterns need the global parities and wider reads. Scaling: smaller repair reads mean less cross-rack traffic and faster repair, which also shrinks T in the durability formula.

Table 15.3 — Redundancy schemes compared (independent failures, AFR 2%, T = 24 h).

Scheme	Overhead	Losses tolerated	Reads per single repair	Approx. $P_{\text{loss/yr}}$ per group	Typical use
3× replication	3.0×	2	1	3.6×10^{-10}	Hot data, small objects, metadata
RS(6,3)	1.5×	3	6	1.0×10^{-11}	HDFS default EC policy (RS-6-3)
LRC(12,2,2)	1.33×	any 3 (some 4)	6	$\leq 1.4 \times 10^{-10}$ (counting only 3-loss tolerance)	Azure storage
RS(10,4)	1.4×	4	10	4.3×10^{-14}	f4; HDFS-RAID [Sathiamoorthy et al. 2013]

Alternatives and trade-offs

Replication wins for small objects (a 4 KB object should not become ten 400-byte fragments), hot data and low write latency. Erasure coding wins for large, warm and cold data. Many systems use both: write new data replicated (fast, simple), then **re-encode** it asynchronously once it is sealed or cold, as Azure does for sealed extents and as Tectonic does for blob storage [Huang et al. 2012; Pan et al. 2021].

Failure scenarios

What goes wrong	Why	Detection	Recovery	Prevention
Repair saturates the network	*k*-fold repair reads after a rack failure	Cross-rack link utilization, repair queue depth	Throttle and prioritize stripes with the fewest survivors	LRC or smaller *k*; repair bandwidth budget
Too many fragments in one domain	Placement ignores racks or zones	Placement audit	Migrate fragments	Placement constraint: at most *m* per failure domain, ideally fewer

COMMON MISTAKE

"RS(10,4) means 4 out of 10 can fail." It means 4 out of 14 fragments can fail, any 4, data or parity. And it does not mean "40% overhead buys 4 failures" for free: every single repair reads 10 fragments.

15.7 Metadata services

15.7.1 Scaling the namespace

DEFINITION

Definition — Metadata service. The component that maps names to data: directory entries, file or object attributes (size, owner, permissions, version, checksum) and the locations of each chunk or fragment. It is on the path of every open, create, list and delete, while the data path bypasses it.

First-principles explanation

Return to the library. Adding shelves is easy; the catalog is the hard part, because every reader consults it and every new book changes it. Data scales by adding disks; metadata scales only if the catalog itself can be split without breaking operations that span several entries, such as moving a directory.

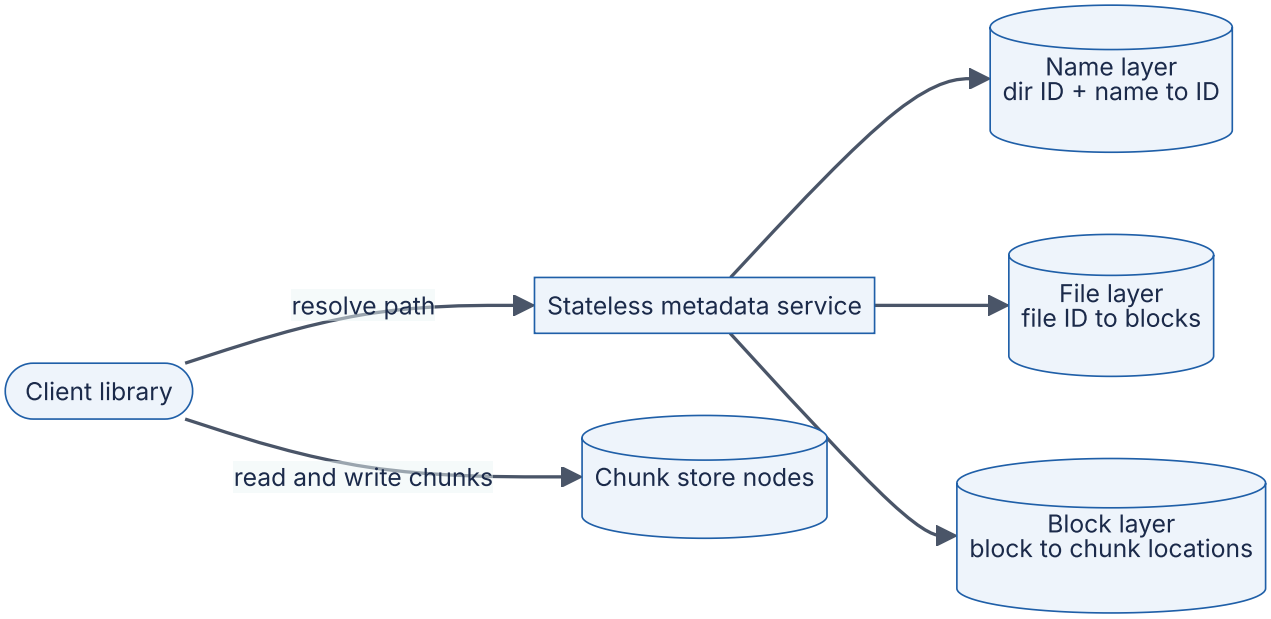
How it works internally

Five techniques recur, usually in combination:

1. **One in-memory server plus a replicated log** (GFS, HDFS): simple and fast, limited by RAM and request rate (§15.5 example).
2. **Static namespace partitioning**: several independent NameNodes or file systems under one tree (HDFS federation); cross-volume rename becomes a copy and load is balanced by hand.
3. **Dynamic subtree partitioning**: Ceph's metadata servers migrate hot directory subtrees between servers as load shifts [Weil et al. 2006].
4. **Hash partitioning in a scalable database**: entries keyed by (parent-directory ID, name) and (file ID, block index) in a sharded store, as in Tectonic (ZippyDB) and Colossus (Bigtable). Listing a directory stays on one shard; cross-shard operations such as directory rename need distributed transactions (Chapter 28) or are restricted.
5. **Compute placement instead of storing it**: Ceph's CRUSH function derives an object's storage nodes from its name and the cluster map, the same idea as consistent hashing (§14.8), so the metadata service need not store per-object locations at all [Weil et al. 2006].

Object stores avoid the hardest part, directory semantics, by having a flat namespace partitioned by key range (§15.4).

Figure 15.8 — Layered, partitioned file-system metadata (after Tectonic).



Reading the diagram. The *client library* resolves a path through a *stateless metadata service*, which reads three layers stored in a sharded key-value store: the *Name layer* (directory entries, sharded by directory ID), the *File layer* (file → block list, sharded by file

ID) and the *Block layer* (block → chunk locations, sharded by block ID). All metadata calls are synchronous. Data then flows directly between client and *chunk store nodes*. Storage boundary: metadata in the key-value store, bytes on chunk nodes. Failure points: a hot directory concentrates load on one Name-layer shard; a multi-shard operation can fail halfway without transactions. Scaling: each layer scales independently by adding shards, and the metadata service scales by adding stateless instances [Pan et al. 2021].

Advantages, disadvantages and failure scenarios

Partitioned metadata removes the single-server ceiling but brings back every problem of a partitioned database: hot partitions (one directory receiving millions of creates), cross-partition operations and consistency between layers.

RECOMMENDATION

Design for metadata first. Estimate the number of files or objects and the metadata operations per second before estimating bytes. Prefer fewer, larger objects; avoid huge single directories; and avoid workflows that rename or list large trees.

15.8 Multipart upload, chunking and deduplication

15.8.1 Multipart upload

DEFINITION

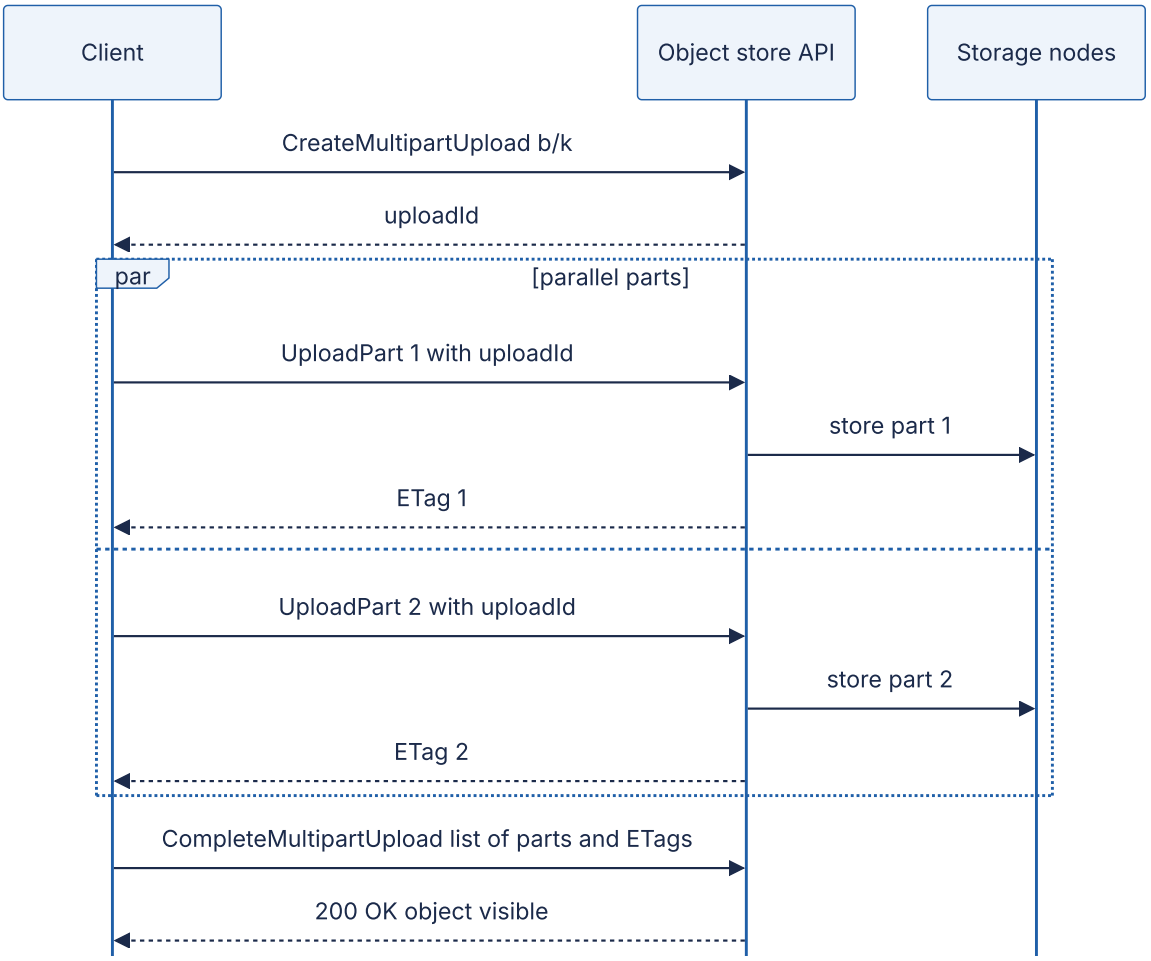
Definition — Multipart upload. A protocol for uploading one large object as independently uploaded **parts**: the client *initiates* an upload and receives an upload ID, uploads numbered parts (in any order, in parallel, retrying any part), then *completes* the upload with the list of part numbers and their ETags, at which point the store assembles the object atomically. Until completion, or after an *abort*, the object is not visible.

Why does this exist?

A single 200 GB PUT over one connection at 50 MB/s takes over an hour, and a network blip at 95% forces a restart from zero. Splitting the object makes the upload parallel (faster) and restartable (only the failed part is retried). As of 2026, Amazon S3 documents parts of 5 MiB to 5 GiB (the last part may be smaller), up to 10,000 parts per upload, a maximum single PUT of 5 GB, and a maximum object size of about 50 TB (48.8 TiB, raised from the long-standing 5 TB) [AWS S3 multipart docs].

How it works internally

Figure 15.9 — Multipart upload sequence.



Reading the diagram. The *client* initiates an upload and receives an `uploadId`. It then uploads parts concurrently (the `par` block), each a synchronous HTTP request stored durably on *storage nodes* and acknowledged with an ETag (a checksum the client keeps). The final *Complete* request is the commit point: the index records the object as the ordered concatenation of the listed parts, and only then does it become visible. Failure points: a failed part is retried; an upload never completed or aborted leaves billed, invisible parts for a lifecycle rule to delete (§15.9). Scaling: parts upload over many connections and land on many storage nodes, so throughput grows until the client's NIC saturates.

FORMULA

Part sizing and upload time

$$p_{\min} = \max\left(5\ \text{MiB}, \frac{S}{N_{\max}}\right) \qquad N = \left\lceil \frac{S}{p} \right\rceil \qquad t \approx \frac{S}{\min\left(c \cdot b,\ B_{\text{NIC}}\right)}$$

Symbol	Meaning	Units
S	Object size	bytes
p, $p_{\min}$	Chosen and minimum part size	bytes
$N_{\max}$	Maximum parts per upload (10,000 in S3)	count
N	Number of parts	count
c	Concurrent part uploads	count
b	Throughput per connection	bytes/s
B_{NIC}	Client network limit	bytes/s

WORKED EXAMPLE

Worked example — Uploading a 200 GB database backup. One connection achieves 50 MB/s; the NIC is 10 Gb/s (1.25 GB/s).

1. Minimum part size: $200 \times 10^9 / 10,000 = 20$ MB, which is above 5 MiB (5.24 MB), so $p_{\min} = 20$ MB.
2. Choose $p = 64$ MiB $= 67.1$ MB: $N = \lceil 2 \times 10^{11} / 6.71 \times 10^7 \rceil = 2,981$ parts.
3. Single PUT: not allowed (above 5 GB), and would take $2 \times 10^{11} / 5 \times 10^7 = 4,000$ s.
4. Sixteen concurrent parts: $\min(16 \times 50, 1250) = 800$ MB/s, so $t = 2 \times 10^{11} / 8 \times 10^8 = 250$ s.
5. Cost of a network error: retry one 67 MB part, about 1.3 s at 50 MB/s, instead of restarting.

Interpretation. Parallelism gives a 16× speed-up until the NIC is the limit (at 25 connections). **Engineering implication.** Choose a part size that keeps N well under 10,000 with headroom for growth, use 8–32 concurrent parts, verify each part's checksum, and always configure an abort rule for incomplete uploads.

15.8.2 Presigned URLs

A **presigned URL** is an object URL that carries the bucket, key, HTTP method, an expiry time and a signature computed with the issuer's credentials (an HMAC under AWS Signature Version 4, §10.4.6). Anyone holding it can perform exactly that operation until expiry (at most seven days with SigV4) [AWS S3 presigned URL docs]. PhotoShare uses them so that a phone uploads a photo *directly* to object storage: the API server authenti-

cates the user, chooses the key and returns a presigned PUT URL valid for a few minutes, so upload bandwidth never touches the application tier.

WARNING / TRADE-OFF

A presigned URL is a bearer token. Keep expiries short, scope each URL to one key and one method, enforce size and content type (for example with signed POST policies or post-upload validation), never log URLs in full, and note that the URL's power ends when the signing credential is revoked or expires.

15.8.3 Content-defined chunking and deduplication

DEFINITION

Definition — Content-defined chunking (CDC). Splitting a byte stream into variable-size chunks whose boundaries are chosen by the content itself: a rolling hash is computed over a small sliding window, and a boundary is declared wherever the hash matches a pattern (for example, its low 13 bits equal a fixed value). Each chunk is then identified by a cryptographic hash of its bytes, so identical chunks are stored once (**deduplication**).

First-principles explanation

Cut a book into pieces of exactly 1,000 characters, insert one word on page 1, and every later piece shifts: a backup system sees a new book. Cut instead after every sentence ending in a particular word, and the insertion changes only the first piece; later cuts stay put relative to the text. That is content-defined chunking. LBFS introduced it for network file systems, using Rabin fingerprints over a 48-byte window and declaring a boundary when the low 13 bits matched, for an expected 8 KB chunk, with minimum and maximum chunk sizes of 2 KB and 64 KB [Muthitacharoen et al. 2001; Rabin 1981].

Mathematical foundations

FORMULA

Expected chunk size and deduplication index size

$$E[\text{chunk}] \approx s_{\min} + 2^{b} \qquad M_{\text{index}} = \frac{U}{E[\text{chunk}]} \times e$$

Symbol	Meaning	Units
b	Number of hash bits that must match (boundary probability 2^{-b} per byte)	bits
$s_{\min}$	Minimum chunk size (no boundary is accepted earlier)	bytes
$E[\text{chunk}]$	Expected chunk size (ignoring the maximum cap)	bytes
U	Unique data stored	bytes
e	Index entry size (hash plus location)	bytes

WORKED EXAMPLE

Worked example — LBFS parameters and a backup index. With $b = 13$ and $s_{\min} = 2$ KiB: $E \approx 2,048 + 8,192 = 10,240$ B. A backup service with $U = 100$ TB of unique data and 40-byte entries (a 32-byte SHA-256 plus an 8-byte location):

1. Chunks: $10^{14}/10,240 = 9.77 \times 10^{9}$.
2. Index: $9.77 \times 10^{9} \times 40 = 3.9 \times 10^{11}$ B ≈ 391 GB.
3. With $b = 20$ (about 1 MiB chunks): $10^{14}/(2,048 + 1,048,576) \times 40 \approx 3.8$ GB.

Interpretation. Smaller chunks find more duplicates but the index grows inversely with chunk size and soon exceeds RAM. **Engineering implication.** Choose the chunk size from the index budget and the expected edit granularity; large-scale systems use larger chunks, sampled or tiered indexes, and keep only hot index entries in memory.

Code example

```
        # Content-defined chunking (CDC) versus fixed-size chunking, LBFS-style parame-
ters.
        # A Rabin-Karp polynomial rolling hash stands in for a true Rabin fingerprint
        # (which uses polynomial arithmetic over GF(2)); the boundary logic is the same.
        import hashlib
        import random

        WINDOW = 48                      # bytes in the rolling window (as in LBFS)
        MASK = (1 << 13) - 1             # 13 low bits -> expected chunk ~ 2^13 = 8 KiB
        MAGIC = 0x78                     # boundary when (hash & MASK) == MAGIC
        MIN_CHUNK, MAX_CHUNK = 2 * 1024, 64 * 1024
        BASE, MOD = 257, (1 << 61) - 1
        BASE_OUT = pow(BASE, WINDOW, MOD)  # weight of the byte leaving the window


        def cdc_chunks(data):
            chunks, start, h = [], 0, 0
            for i, byte in enumerate(data):
                h = (h * BASE + byte) % MOD
                if i >= WINDOW:
                    h = (h - data[i - WINDOW] * BASE_OUT) % MOD   # roll the oldest byte
out
                size = i - start + 1
                if (size >= MIN_CHUNK and (h & MASK) == MAGIC) or size >= MAX_CHUNK:
                    chunks.append(data[start:i + 1])
                    start = i + 1
            if start < len(data):
                chunks.append(data[start:])
            return chunks


        def fixed_chunks(data, size=8 * 1024):
            return [data[i:i + size] for i in range(0, len(data), size)]


        def shared_fraction(old, new):
            """Fraction of the new version's bytes found in chunks already stored."""
            stored = {hashlib.sha256(c).digest() for c in old}
            reused = sum(len(c) for c in new if hashlib.sha256(c).digest() in stored)
            return reused / sum(len(c) for c in new)


        if __name__ == "__main__":
            rng = random.Random(42)
            v1 = bytes(rng.randrange(256) for _ in range(2_000_000))   # 2 MB "file"
            v2 = b"inserted header!" + v1[:900_000] + b"edit" + v1[900_000:]

            for name, chunker in (("fixed 8 KiB", fixed_chunks), ("content-defined",
cdc_chunks)):
                a, b = chunker(v1), chunker(v2)
                avg = sum(map(len, a)) / len(a)
                print(f"{name:16s} chunks={len(a):4d} avg={avg:8.0f} B "
                      f"reused after edit={shared_fraction(a, b):6.1%}")
```

Output (Python 3, about 2 s):

```
fixed 8 KiB      chunks= 245 avg=    8163 B reused after edit=  0.0%
content-defined  chunks= 185 avg=   10811 B reused after edit= 98.5%
```

What to notice:

- The hash depends only on the last 48 bytes, so after an insertion the boundaries re-synchronize within one chunk, and every later chunk is byte-identical to before.
- Fixed-size chunking reuses nothing: a 16-byte insertion at the front shifts every block boundary.
- The measured average (about 10.8 KB) matches $s_{\min} + 2^{13} \approx 10.2$ KB within sampling noise.

COMMON MISTAKE

“Deduplicate everything, including encrypted data.” Encryption with random IVs makes identical plaintext produce different ciphertext, so deduplication must happen before encryption or use deliberate (and privacy-weakening) convergent encryption. Cross-user deduplication can also leak whether a file exists elsewhere; many services deduplicate only within one account.

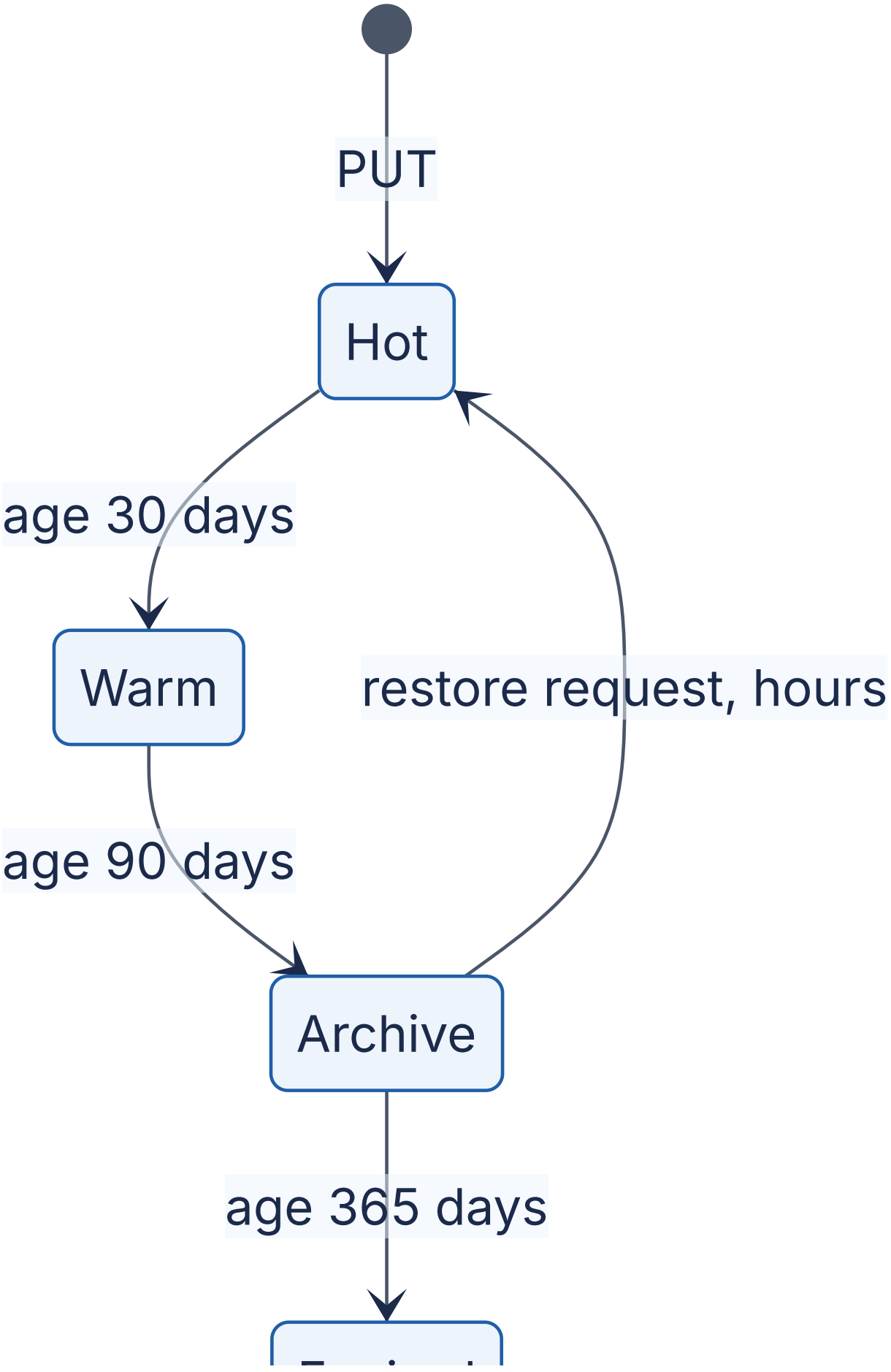
15.9 Storage tiers and lifecycle policies

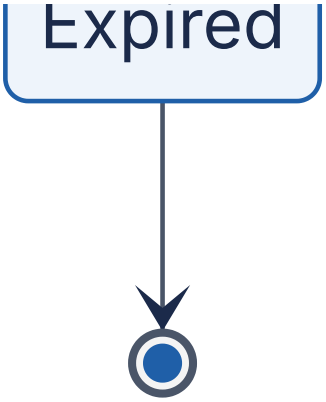
DEFINITION

Definition — Storage tier and lifecycle policy. A **storage tier** (storage class) is a price/performance point for the same data: **hot** (frequent access, highest storage price, no retrieval fee), **warm** or infrequent access (cheaper storage, per-GB retrieval fee, minimum storage duration), **cold** (cheaper again, slower or costlier retrieval) and **archive** (cheapest storage, retrieval in hours). A **lifecycle policy** is a set of rules, usually by prefix, tag or age, that automatically transitions objects between tiers and expires (deletes) them.

Access frequency falls with age: photos are viewed for days, logs queried for a week and kept for years. Facebook built f4 specifically for such “warm” blobs, moving them from triple-replicated Haystack to erasure-coded storage [Muralidhar et al. 2014].

Figure 15.10 — A lifecycle policy for log objects.


PUT
Hot
age 30 days
Warm
restore request, hours
age 90 days
Archive
age 365 days



Reading the diagram. Each state is a storage class; transitions are applied asynchronously by the store's lifecycle engine, typically once a day, so ages are approximate. *Hot* serves reads in milliseconds; *Warm* serves them in milliseconds but charges per GB retrieved; *Archive* requires a restore that takes hours before the object can be read and creates a temporary hot copy. Failure points: a rule on the wrong prefix makes data unreadable for hours or deletes it; test rules and enable versioning. Every transition is a billed request.

Table 15.4 — Illustrative object-storage classes (Amazon S3, us-east-1 list prices, as of 2026).

Tier	Example class	Storage $/GB-month	Retrieval	Minimum duration
Hot	S3 Standard	0.023	none	none
Warm	S3 Standard-IA	0.0125	$0.01/GB	30 days; 128 KB minimum billable size
Cold	S3 Glacier Instant Retrieval	0.004	$0.03/GB, milliseconds	90 days
Archive	S3 Glacier Deep Archive	0.00099	hours (about 12 h standard)	180 days

FORMULA

Cost of one cohort of data over its lifetime

$$C = \sum_{\text{tiers } i} V \cdot p_i \cdot t_i + \sum_{\text{transitions } j} N_{\text{obj}} \cdot q_j + V_{\text{read}} \cdot g$$

Symbol	Meaning	Units
V	Volume of the cohort	GB
p_i, t_i	Storage price and months spent in tier i	$/GB-month, months
N_{obj}	Number of objects	count
q_j	Price per lifecycle transition request into tier j	$/request
V_{read}, g	Data retrieved and its per-GB retrieval fee	GB, $/GB

WORKED EXAMPLE

Worked example — One month of logs kept for a year (as of 2026, illustrative list prices). A cohort of $V = 100$ TB $= 100,000$ GB of logs in 1 million objects of 100 MB. Assume transition requests cost $\$0.01$ per 1,000 into Standard-IA and $\$0.05$ per 1,000 into Deep Archive (verify current prices).

1. All Standard for 12 months: $100,000 \times 0.023 \times 12 = \$27,600$.
2. Lifecycle: 1 month Standard $= \$2,300$; 2 months Standard-IA $= 100,000 \times 0.0125 \times 2 = \$2,500$; 9 months Deep Archive $= 100,000 \times 0.00099 \times 9 = \$891$. Storage subtotal $\$5,691$.
3. Transitions: $10^6 \times 0.01/1000 + 10^6 \times 0.05/1000 = \$10 + \$50 = \$60$.
4. Total $\approx \$5,751$: a 79% saving.
5. Small-object trap: the same 100 TB as 1 billion 100 KB objects makes the Deep Archive transitions alone cost $10^9 \times 0.05/1000 = \$50,000$, more than the whole Standard-only bill, and every 100 KB object in Standard-IA is billed as 128 KB.

Interpretation. Tiering is worth roughly 4–5× for large objects that cool predictably. **Engineering implication.** Aggregate small objects before tiering them, respect minimum durations (deleting a Deep Archive object after 30 days still bills 180), and use automatic access-based tiering when access patterns are unknown.

RECOMMENDATION

Lifecycle hygiene. Always add rules to abort incomplete multipart uploads after a few days and to expire noncurrent versions in versioned buckets. Without them, invisible parts and old versions quietly become a large share of the bill.

15.10 Summary and revision notes

SUMMARY

- **Block** storage exposes numbered blocks to one host (databases, boot disks; sub-millisecond to millisecond latency). **File** storage exposes a shared POSIX tree (NFS/SMB; shared access, metadata-bound). **Object** storage exposes immutable key → blob over HTTP (unlimited scale, tens of ms first byte, parallel throughput, cheapest per GB).
- An S3-style store separates a stateless **front end**, a key-range-partitioned **metadata index**, a **placement** service and **storage nodes**. A PUT writes data first and commits metadata last; the commit is the linearization point that gives strong read-after-write consistency (S3 since December 2020).
- **GFS**: one in-memory master off the data path, 64 MB chunks, three replicas, leases that let a primary order mutations, pipelined data flow, at-least-once record append. **HDFS** copies the design; HA adds a standby NameNode, a JournalNode quorum and ZooKeeper fencing. **Colossus** and **Tectonic** move metadata into scalable databases (Bigtable, ZippyDB).
- **Erasure coding** RS(k,m): overhead $(k+m)/k$, survives any m losses, repair reads k fragments. RS(10,4) costs 1.4× versus 3× for replication and, under independence, is far more durable; loss $\approx \frac{n!}{(n-m-1)!}\lambda^{m+1}T^{m}H$. **LRC** cuts single-fragment repair reads (Azure LRC(12,2,2): 1.33×, 6 reads).
- Metadata, not bytes, limits scale: memory ≈ objects × bytes per entry. Partition it by subtree, by hash in a sharded store, or compute placement (CRUSH).
- **Multipart upload** makes large uploads parallel and restartable (S3: 5 MiB–5 GiB parts, ≤ 10,000 parts). **Presigned URLs** delegate one operation for a limited time. **Content-defined chunking** keeps chunk boundaries stable under insertion, enabling deduplication; index size ∝ 1/chunk size.
- **Lifecycle policies** move cooling data to cheaper tiers (4–5× savings for large objects); minimum durations, transition fees and minimum billable sizes punish small objects.

15.11 Knowledge checkpoint

1. Name one workload that fits each of block, file and object storage, and say why.
2. In an object-store PUT, why is data written before metadata is committed?
3. What did S3's December 2020 change guarantee, and what remains asynchronous?
4. Why did GFS choose 64 MB chunks? Give one benefit and one cost.
5. What is a lease in GFS and what problem does it solve?
6. Compute the overhead and tolerated losses of RS(6,3) and RS(10,4).
7. Why does RS(10,4) generate about ten times the repair traffic of replication?
8. How does LRC(12,2,2) repair a single lost data fragment?
9. Estimate NameNode heap for 200 million files of one block each at 150 bytes per object.
10. What is the minimum part size for a 300 GB object in S3, and why?
11. Why does inserting one byte at the start of a file defeat fixed-size deduplication but not CDC?

Answers

1. Block: a PostgreSQL data directory (random page I/O, one host). File: shared model checkpoints read by many training nodes (POSIX, shared). Object: user photos (immutable, large, HTTP, unlimited).
2. So that the key never points at incomplete data: the metadata commit is the atomic visibility point, and orphaned data from failed PUTs can be garbage-collected.
3. Strong read-after-write for PUT and DELETE of objects and for LIST. Bucket configuration changes and cross-region replication remain eventually consistent.
4. Benefit: few chunks, so small master memory and few master requests. Cost: small files occupy one chunk on few servers and can become hot spots.
5. A time-limited grant making one replica the primary for a chunk; it orders concurrent mutations without involving the master, and expires safely if the primary fails.
6. RS(6,3): 1.5×, any 3. RS(10,4): 1.4×, any 4.
7. Rebuilding one fragment requires reading $k = 10$ surviving fragments, versus one surviving replica.
8. It reads the other five data fragments of its local group plus that group's local parity, six fragments in all.
9. $(2 \times 10^8 + 2 \times 10^8) \times 150 = 6 \times 10^{10}$ B, about 60 GB.
10. $3 \times 10^{11} / 10,000 = 30$ MB, because at most 10,000 parts are allowed (and 30 MB exceeds the 5 MiB floor).
11. Fixed boundaries are at fixed offsets, so every block shifts; CDC boundaries depend on local content and re-synchronize after the insertion.

15.12 Interview questions

Basic

Q1. When would you use object storage rather than a block volume? Answer. For large, immutable, independently addressed data read sequentially or by many clients: media, backups, logs, datasets. It offers unlimited capacity, high durability and parallel throughput at the lowest price, but tens of milliseconds of first-byte latency and no in-place updates. Databases and anything needing low-latency random I/O stay on block storage.

Q2. How would PhotoShare let users upload photos without loading the API servers? Answer. The API authenticates the user, writes a pending photo row, and returns a short-lived presigned PUT URL for a server-chosen key. The phone uploads directly; an upload event triggers thumbnail jobs and activates the row. Videos use multipart upload. Keep expiries short; validate size and type.

Intermediate

Q3. Explain replication versus erasure coding and when you would choose each. Answer. Replication: simple, fast, cheap repair (read one copy), 3× overhead. RS(k,m): overhead $(k+m)/k$ (1.4× for RS(10,4)), tolerates any m losses, but repair reads k fragments and small objects encode poorly. Replicate hot data, small objects and metadata; erasure-code large warm and cold data, often by re-encoding after it is sealed.

Q4. Walk through a GFS write and explain the role of the lease. Answer. The client asks the master which replica holds the chunk's lease, pushes data along a chain of all replicas, then sends the write to the primary. The primary assigns a serial order, applies it, forwards it to the secondaries and replies when all apply. The lease (60 s, renewable) lets the primary order mutations without the master and expires safely if it dies.

Q5. A team writes a million files per hour into one HDFS directory and the cluster slows down. Why, and what do you do? Answer. Every create is a NameNode metadata operation and each small file costs heap. Pack records into larger files (Parquet, hourly compaction), spread directories, use federation, or move to an object store with hashed prefixes.

Advanced

Q6. Design the durability scheme for a 50 PB archive with eleven-nines durability and minimal cost. Answer. Use a wide erasure code such as RS(10,4) or an LRC, with each stripe's fragments spread across racks and, for zone failure, across three zones (or a geo-level XOR as in f4). Compute loss probability with the formula of §15.6 using a pes-

simistic AFR and realistic repair time, then add margin for correlated failures. Budget repair bandwidth, scrub checksums, keep versioning or an offline copy against bugs and deletion, and tier by age.

Q7. How does an object store provide strong read-after-write consistency at scale? Answer. Make each key's metadata update a single atomic commit in a linearizable, replicated index partition, after data is durable. Caches in front of the index must not serve stale entries: invalidate or validate them, for example against a component that records the latest change per key, as AWS describes with its witness. Concurrent writers still race; offer conditional writes for compare-and-swap.

15.13 Practical exercises

1. Extend the RS program to GF(2^8) using log and antilog tables and verify that RS(10,4) recovers all $\binom{14}{4} = 1,001$ four-loss patterns.
2. Using the §15.6 formula, tabulate $P_{\text{loss/yr}}$ for RS(6,3), RS(10,4) and 3× replication with $T = 6$, 24 and 168 hours.
3. Write a lifecycle policy for PhotoShare: originals to infrequent access after 60 days, thumbnails kept hot, abort incomplete uploads after 7 days. Estimate the first-year saving for 5 TB/day of new photos.

15.14 Design challenges

1. **Video archive.** Design storage for a service ingesting 2 PB of video per month, with hot playback of the newest 30 days and rare access afterwards. Choose redundancy per tier, a chunking and upload protocol, metadata design, and a lifecycle policy; estimate raw capacity, repair traffic and monthly cost.
2. **Training-data lake.** Thousands of GPU nodes read a 5 PB dataset of 50 billion small images each epoch. Choose a parallel file system or object storage, pack samples into shards, and keep metadata off the critical path (see Chapter 61).

15.15 References

1. Ghemawat, S., Gobioff, H., and Leung, S.-T. "The Google File System." *SOSP*, 2003.
2. Shvachko, K., Kuang, H., Radia, S., and Chansler, R. "The Hadoop Distributed File System." *IEEE MSST*, 2010.
3. Huang, C., Simitci, H., Xu, Y., Ogus, A., Calder, B., Gopalan, P., Li, J., and Yekhanin, S. "Erasure Coding in Windows Azure Storage." *USENIX ATC*, 2012.
4. Pan, S., et al. "Facebook's Tectonic Filesystem: Efficiency from Exascale." *USENIX FAST*, 2021. https://www.usenix.org/conference/fast21/presentation/pan
5. Muralidhar, S., et al. "f4: Facebook's Warm BLOB Storage System." *OSDI*, 2014.

6. Muthitacharoen, A., Chen, B., and Mazières, D. “A Low-bandwidth Network File System.” *SOSP*, 2001.
7. Rabin, M. O. *Fingerprinting by Random Polynomials*. Technical Report TR-15-81, Harvard University, 1981.
8. Reed, I. S., and Solomon, G. “Polynomial Codes over Certain Finite Fields.” *Journal of the Society for Industrial and Applied Mathematics* 8(2), 1960.
9. Plank, J. S. “A Tutorial on Reed-Solomon Coding for Fault-Tolerance in RAID-like Systems.” *Software: Practice and Experience* 27(9), 1997.
10. Beaver, D., Kumar, S., Li, H. C., Sobel, J., and Vajgel, P. “Finding a Needle in Haystack: Facebook’s Photo Storage.” *OSDI*, 2010.
11. Weil, S. A., Brandt, S. A., Miller, E. L., Long, D. D. E., and Maltzahn, C. “Ceph: A Scalable, High-Performance Distributed File System.” *OSDI*, 2006.
12. Rashmi, K. V., Shah, N. B., Gu, D., Kuang, H., Borthakur, D., and Ramchandran, K. “A Solution to the Network Challenges of Data Recovery in Erasure-coded Distributed Storage Systems: A Study on the Facebook Warehouse Cluster.” *USENIX HotStorage*, 2013.
13. Bornholt, J., et al. “Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3.” *SOSP*, 2021.
14. Vogels, W. “Diving Deep on S3 Consistency.” *All Things Distributed*, 2021. https://www.allthingsdistributed.com/2021/04/s3-strong-consistency.html
15. Amazon Web Services. *Summary of the Amazon S3 Service Disruption in the Northern Virginia (US-EAST-1) Region*. 2017. https://aws.amazon.com/message/41926/
16. Hildebrand, D., and Serenyi, D. “Colossus under the hood: a peek into Google’s scalable storage system.” *Google Cloud Blog*, 2021.
17. Amazon Web Services. *Amazon S3 User Guide*: consistency model; multipart upload limits; performance design patterns; presigned URLs; lifecycle management. https://docs.aws.amazon.com/AmazonS3/latest/userguide/ (accessed 2026).
18. Amazon Web Services. *Amazon EBS User Guide*: “Amazon EBS volume types”. Accessed 2026.
19. Amazon Web Services. *Amazon S3 pricing*. https://aws.amazon.com/s3/pricing/ (accessed 2026).
20. Google Cloud. *Cloud Storage documentation*: “Consistency”. Accessed 2026.
21. Apache Software Foundation. *Apache Hadoop documentation*: “HDFS High Availability Using the Quorum Journal Manager”; “HDFS Erasure Coding”. https://hadoop.apache.org/docs/
22. Sathiamoorthy, M., et al. “XORing Elephants: Novel Erasure Codes for Big Data.” *PVLDB* 6(5), 2013.
23. Ford, D., et al. “Availability in Globally Distributed Storage Systems.” *OSDI*, 2010.

15.16 Open verification items

- S3 maximum object size: AWS pages state both “50 TB” and 48.8 TiB (10,000 × 5 GiB); confirm the current figure and announcement date (docs.aws.amazon.com was unreachable from the authoring environment).
- S3 conditional writes: `If-None-Match` (2024) and `If-Match` (late 2024) availability dates and scope.
- Table 15.4 prices, transition-request prices ($0.01 and $0.05 per 1,000), Deep Archive retrieval times and minimum durations: verify against the AWS pricing page as of 2026.
- Exact title and URL of the Google Cloud Colossus blog post [Hildebrand and Serenyi 2021].
- HDFS “about 150 bytes per namespace object” rule of thumb: confirm a primary source.
- Recovery-traffic magnitude in [Rashmi et al. 2013] (“on the order of a hundred terabytes per day”).
- Typical object-store first-byte latency range in Table 15.1.

Chapter 16 — Search Systems

> **Level:** 3 · **Track:** A (Traditional) · **Estimated reading time:** 5 h

Type “wireless noise cancelling headphones” into an online store and, within about a tenth of a second, a ranked list comes back from tens of millions of products. A B-tree index (§11.8) cannot do this: it finds rows whose key *equals* or *starts with* a value, so `WHERE description LIKE '%noise%'` reads every row, and even then the database has no notion of which match is *best*.

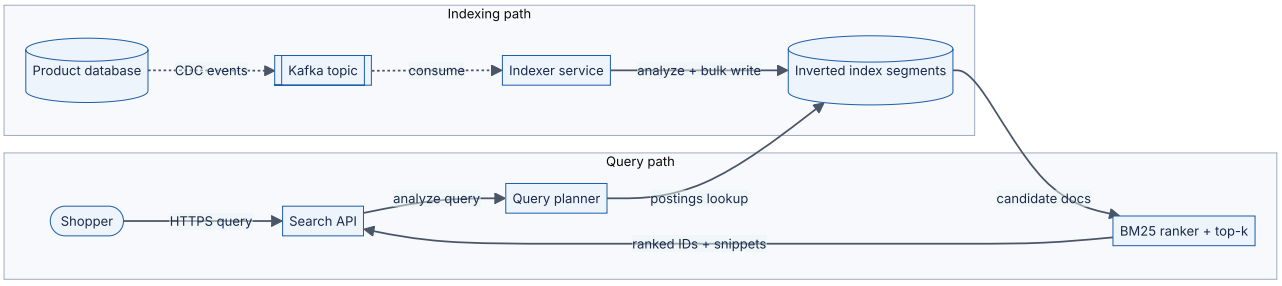
Search systems solve three problems that ordinary databases do not:

1. **Finding documents by the words inside them**, quickly, across billions of words. The answer is the **inverted index** (§16.3), together with the text analysis that decides what counts as a “word” (§16.4).
2. **Ordering matches by relevance**, so the most useful result comes first. The answer is a scoring function, today usually **BM25** (§16.5).
3. **Doing both at scale and in near real time**, while documents are being added, changed and deleted. The answer is immutable index segments that are merged in the background (§16.6), partitioned and replicated across nodes (§16.7). Elasticsearch and OpenSearch package all of this (§16.8), and an indexing pipeline keeps the index in step with the source of truth (§16.10).

Autocomplete (§16.9) has a latency budget so tight that it gets its own data structures, and §16.11 bridges to semantic and hybrid retrieval (Chapters 74–75). These ideas are stable: Brin and Page’s 1998 Google prototype already had an inverted index, position “hit lists” and a ranking function combining text scores with PageRank [Brin and Page 1998].

The running example is product search for an online electronics store. Figure 16.1 shows its two halves.

Figure 16.1 — The two paths of a search system.



Reading the diagram. The left subgraph is the **indexing path**. The product database is the source of truth; change data capture (CDC) publishes every insert, update and delete as an event to a Kafka topic. Those dotted arrows are asynchronous: the shopper never waits for them. The indexer consumes events, runs text analysis and writes to the index in bulk. The right subgraph is the **query path**, which is synchronous (solid arrows) from the shopper's HTTPS request to the ranked response. The query is analyzed with the *same* rules as the documents, the planner looks up postings lists, and the ranker scores candidates and keeps the top k. There are two storage boundaries, the product database and the index, and they are eventually consistent with each other; the lag between them is the **freshness** of search (§16.10). Failure points: if Kafka or the indexer stalls, search keeps serving but grows stale; if the index is unavailable, search fails even though the database is healthy. The two paths scale independently: indexing by adding partitions and indexer consumers, querying by adding replicas (§16.7).

16.1 Learning objectives

After this chapter you should be able to:

- Explain how an inverted index, postings lists, positions and skip pointers answer keyword, Boolean and phrase queries, and estimate an index's size with delta and variable-byte compression.
- Design an analysis chain (tokenizer, lowercasing, stemming or lemmatization, stop words, n-grams, synonyms) and predict how each choice changes recall, precision and index size.
- Compute TF-IDF and BM25 scores by hand, explain the roles of k_1 and b, and apply field boosting.
- Describe how Lucene builds indexes from immutable segments, handles deletes with tombstones, and merges segments, and estimate merge write amplification.
- Compare document and term partitioning, compute scatter–gather tail latency, and explain query-then-fetch, deep-pagination cost and the global-versus-local IDF problem.
- Describe the Elasticsearch/OpenSearch cluster model: node roles, shards, replicas, the translog, refresh, flush and near-real-time search.
- Design an autocomplete service with a trie or FST holding precomputed top-k completions and meet a 100 ms end-to-end budget.
- Build a near-real-time indexing pipeline with CDC and Kafka, and reason about its freshness latency and refresh-interval trade-offs.

16.2 Prerequisites

- B-trees and secondary indexes — §11.7–§11.8 (the inverted index is a different answer to the same "avoid a full scan" problem).
- Log-structured storage, LSM trees and compaction — §13.7–§13.9 (Lucene segments are an LSM-like design; this chapter cross-references rather than repeats).
- Partitioning and replication — §14.3 and §14.6–§14.7 (search clusters apply them to indexes).
- Percentiles and fan-out amplification — §4.8 (scatter–gather latency builds directly on p^N).
- Logarithms — §4.4 (IDF and merge-tier arithmetic use them).
- Tries — §17.11 gives the data structure; this chapter applies it to autocomplete.
- Python 3.10 or later to run the listings; all use only the standard library.

16.3 Inverted indexes

16.3.1 From the index at the back of a book to an inverted index

> DEFINITION
>
> **Definition — Inverted index.** A mapping from each distinct **term** (a normalized word or token) to a **postings list**: the sorted list of identifiers of the documents that contain the term, optionally with the term's frequency and positions in each document. The set of all terms is the **dictionary** (or **vocabulary**, or **lexicon**).

First-principles explanation

Open a textbook at the back. The index lists "Bloom filter … 211, 340, 512". Nobody reads the whole book to find Bloom filters; you look up the word and jump to the pages. The book's body maps *page → words*. The index at the back is that mapping turned inside out: *word → pages*. That is all an inverted index is. The "forward" direction (document → terms) is how documents are written; the "inverted" direction (term → documents) is how they are searched.

Three refinements make it a search index: page numbers become small-integer **document identifiers (docIDs)**; each list is kept **sorted** by docID so lists combine cheaply; and each entry can carry the **term frequency** (tf) and **positions** of the word, for ranking and phrase matching.

Why does this exist?

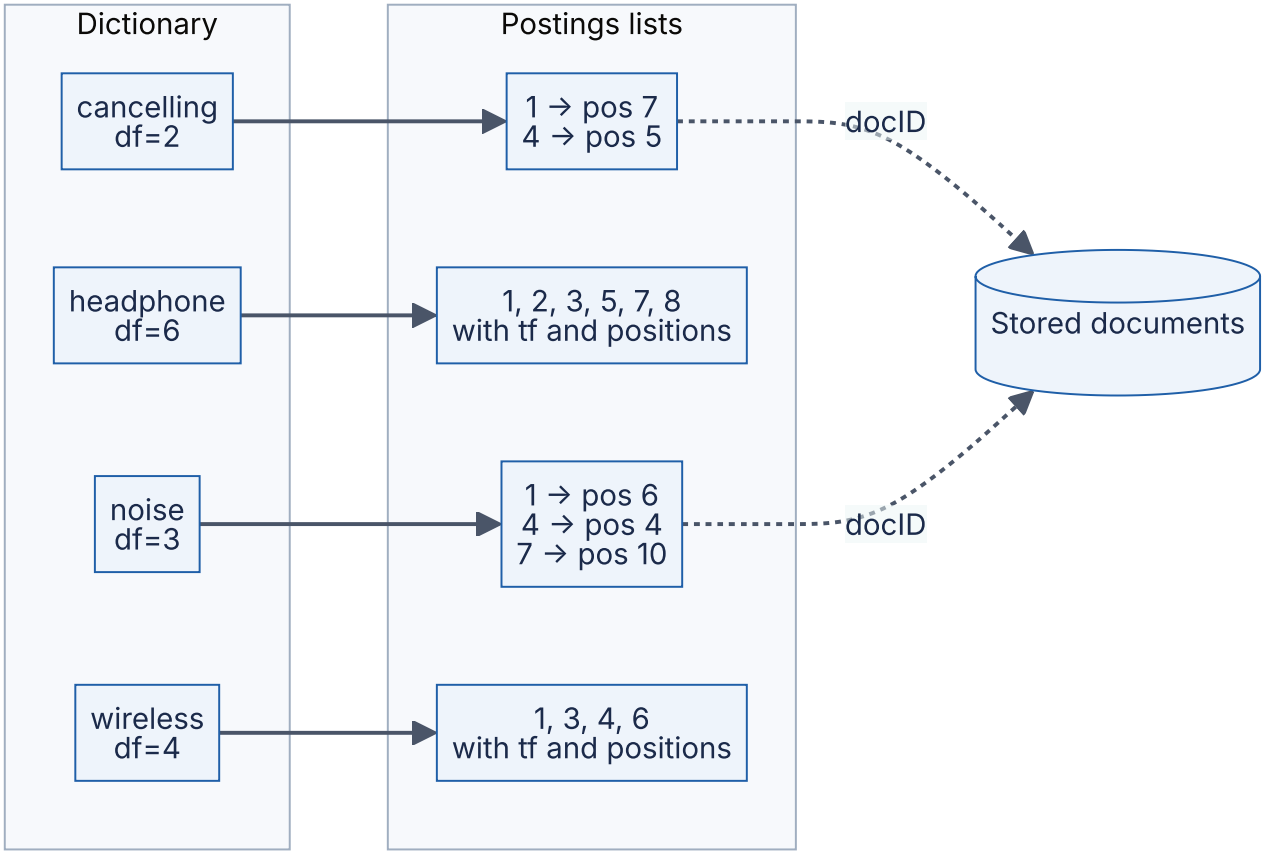
Consider answering “documents containing *noise* and *headphones*” by scanning. A catalog of 100 million documents averaging 2.4 KB of text is 240 GB. Reading 240 GB at a generous 10 GB/s of memory bandwidth takes 24 s per query on one core, before any matching work. An inverted index instead reads two postings lists. If *noise* appears in 1 million documents and *headphones* in 100,000, the engine touches about 1.1 million entries; at roughly 2.4 bytes per compressed entry (§16.3.5) that is about 2.6 MB, read in well under a millisecond. The index trades storage and indexing work for a 10,000-fold reduction in query work.

How it works internally

1. **Analyze** each document into terms with positions (§16.4).
2. **Assign** a docID; Lucene assigns them in insertion order within each segment, so postings are naturally sorted (§16.6).
3. **Append** the docID (with tf and positions) to the postings list of each distinct term.
4. **Store** the dictionary in a structure supporting exact and prefix lookup; Lucene uses a block tree indexed by a finite-state transducer (§16.9.3).
5. **Compress** postings (§16.3.4) and write them sequentially.

Architecture diagram

Figure 16.2 — Dictionary, postings lists and positions.



Reading the diagram. The left subgraph is the dictionary: one entry per distinct term with its **document frequency** (df, the number of documents containing it). Each solid arrow is a pointer, in practice a file offset, from a dictionary entry to its postings list. The right subgraph shows postings for the body field of this chapter's eight-product example (§16.5.4): *noise* occurs in documents 1, 4 and 7, at positions 6, 4 and 10. The dotted arrows show the last step of a query: docIDs are turned into displayable results by fetching stored fields, a separate storage structure. The dictionary is small and usually memory-resident; postings are large and read sequentially; stored documents are read only for the final top results. Failure points are corruption of any of the three structures, which is why Lucene checksums every file. Scaling is by partitioning documents across shards, each with its own complete dictionary and postings (§16.7).

16.3.2 Query evaluation with postings lists

Because postings are sorted, Boolean operators become list operations:

- **AND** (*noise* AND *wireless*) is an **intersection**: walk both lists in step, advancing whichever pointer holds the smaller docID, and output docIDs found in both. Cost is $O(L_1 + L_2)$ for lists of lengths L_1 and L_2.
- **OR** is a **union**, also a linear merge; ranked retrieval is usually OR-like, with scoring sorting candidates. **NOT** is a list difference.

- A **phrase query** (“noise cancelling”) first intersects the lists, then checks positions: a document matches only if *cancelling* occurs at position $p+1$ for some position p of *noise*. Positional data typically makes the index two to four times larger, which is why some fields are indexed without positions (in Elasticsearch, `index_options: docs` or `freqs`).

Process the **rarest term first**: an intersection is never larger than its shortest list.

16.3.3 Skip pointers

Intersecting a list of 200,000 docIDs with one of 1,000 by linear merge costs about 200,000 comparisons, almost all of them spent stepping through the long list past docIDs that cannot match. **Skip pointers** are shortcuts stored every s entries of a postings list: “the entry s positions ahead holds docID X”. When the target docID is beyond X, the iterator jumps s entries at once.

Lucene exposes this as an iterator operation, `advance(target)`: move to the first docID $\geq$ target. Its postings formats store multi-level skip data so `advance` can take long jumps, then shorter ones, then scan within one compressed block [Lucene Docs: postings format]. The skip interval trades jump distance against scan length. Manning et al. suggest $\sqrt{L}$ for a list of length L [Manning et al. 2008, §2.3], but that heuristic assumes lists of similar length. When a short list of m targets drives a long list of length L, the total cost is roughly L/s skip-follows plus $m \cdot s/2$ in-block steps, minimized at $s = \sqrt{2L/m}$.

```
        """Intersect two sorted postings lists (AND query), with and without skip point-
ers,
        counting comparisons. The skip version drives from the shorter list and calls
advance(target),
        the operation Lucene's postings iterators expose."""
        import math
        import random

        def intersect_linear(a, b):
            """Textbook merge: step whichever pointer holds the smaller docID."""
            i = j = comparisons = 0
            out = []
            while i < len(a) and j < len(b):
                comparisons += 1
                if a[i] == b[j]:
                    out.append(a[i]); i += 1; j += 1
                elif a[i] < b[j]:
                    i += 1
                else:
                    j += 1
            return out, comparisons

        class SkipPostings:
            """A postings list with a skip pointer every `step` entries."""
            def __init__(self, doc_ids, step=None):
                self.ids = doc_ids
                self.step = step or max(1, int(math.sqrt(len(doc_ids))))
                self.pos = 0
                self.comparisons = 0

            def advance(self, target):
                """Move to the first docID >= target; return it (or None when ex-
hausted)."""
                ids, s = self.ids, self.step
                nxt = (self.pos // s + 1) * s                    # next skip-pointer po-
sition
                while nxt < len(ids) and ids[nxt] <= target:     # follow skips while
they do not overshoot
                    self.comparisons += 1
                    self.pos, nxt = nxt, nxt + s
                while self.pos < len(ids) and ids[self.pos] < target:   # then scan in-
side one block
                    self.comparisons += 1
                    self.pos += 1
                return ids[self.pos] if self.pos < len(ids) else None

        def intersect_with_skips(short, long_list, step=None):
            it = SkipPostings(long_list, step)
            out = []
            for doc in short:                                    # drive from the rarer
term
                it.comparisons += 1
                found = it.advance(doc)
                if found is None:
                    break
                if found == doc:
```

```
                    out.append(doc)
            return out, it.comparisons

        if __name__ == "__main__":
            random.seed(7)
            N = 1_000_000
            common = sorted(random.sample(range(N), 200_000))   # e.g. "wireless": 20%
of docs
            rare = sorted(random.sample(range(N), 1_000))       # e.g. "anc": 0.1% of
docs
            r1, c1 = intersect_linear(common, rare)
            assert r1 == sorted(set(common) & set(rare))
            print(f"matches={len(r1)}  linear merge: {c1:,} comparisons")
            for step in (447, 128, 20):                         # sqrt(L), Lucene's block
size, sqrt(2L/m)
                r2, c2 = intersect_with_skips(rare, common, step)
                assert r2 == r1
                print(f"skip every {step:>3}: {c2:>7,} comparisons  ({c1 / c2:.0f}x
fewer)")
```

Output (Python 3.13):

```
matches=184  linear merge: 200,611 comparisons
skip every 447: 121,210 comparisons  (2x fewer)
skip every 128:  53,587 comparisons  (4x fewer)
skip every  20:  20,909 comparisons  (10x fewer)
```

What to notice:

- The intersection is **driven by the rare list**: 1,000 calls to `advance`, not 200,000 steps.
- The $\sqrt{L} = 447$ heuristic helps only 2× here because each `advance` then scans about 224 entries inside a block. Spacing tuned to the list-length ratio, $\sqrt{2 \times 200{,}000/1{,}000} = 20$, gives 10×. Multi-level skip lists, as in Lucene, get coarse and fine jumps without choosing one spacing.
- In a real engine the bigger win is **decoding**: a skipped compressed block is never decompressed.

16.3.4 Compression: delta encoding and variable-byte codes

A docID stored as a 32-bit integer costs 4 bytes. Postings lists hold one entry per (term, document) pair, so they dominate index size. Two observations make them highly compressible.

Delta (gap) encoding. A sorted list such as 824, 829, 215406 can be stored as its first value followed by differences: 824, 5, 214577. A frequent term has small gaps (a term in 20% of documents has an average gap of 5), and small numbers need few bits. Rare terms have large gaps but short lists, so they contribute little to total size.

Variable-byte (VByte) codes. Encode each gap in as many bytes as it needs, using 7 data bits per byte and the eighth bit as a flag. In Lucene's `VInt` convention the low-order 7 bits come first and a high bit of 1 means "more bytes follow" [Lucene Docs: DataOutput.writeVInt]. Manning et al. describe the same code with the flag set on the *last* byte instead [Manning et al. 2008, §5.3.1]; the two are equivalent in size.

FORMULA

Variable-byte length of an integer

$$\text{bytes}(g) = \max\left(1, \left\lceil \frac{\lfloor \log_2 g \rfloor + 1}{7} \right\rceil\right)$$

Symbol	Meaning	Units
g	The integer to encode, here a docID gap ($g \geq 1$)	count
$\lfloor \log_2 g \rfloor + 1$	Number of significant bits in g	bits
$\text{bytes}(g)$	Encoded length, 7 payload bits per byte	bytes

Table 16.1 — Variable-byte length by gap size.

Gap range	Significant bits	VByte bytes	Saving versus 4-byte integer
1–127	1–7	1	75%
128–16,383	8–14	2	50%
16,384–2,097,151	15–21	3	25%
2,097,152–268,435,455	22–28	4	0%

WORKED EXAMPLE

Worked example — encoding 824, 829, 215406.

1. Gaps: 824, 829 − 824 = 5, 215406 − 829 = 214577.
2. 824 has 10 significant bits (512 ≤ 824 < 1024), so $\lceil 10/7 \rceil = 2$ bytes. Split into 7-bit groups: $824 = 6 \times 128 + 56$. Low group first with the continuation bit set: $56 + 128 = 184$ = `10111000`; then `00000110` (6, no continuation).
3. 5 fits in one byte: `00000101`.
4. 214577 has 18 bits ($2^{17} = 131072 \le 214577 < 262144$), so 3 bytes. $214577 = 13 \times 16384 + 12 \times 128 + 49$, giving `10110001` (49 + 128), `10001100` (12 + 128), `00001101` (13).
5. Total: 6 bytes instead of 12.

Interpretation. Half the space is saved even here; frequent terms, whose gaps fit in one byte, save 75%. **Engineering implication.** The postings that make queries slow (common terms) compress best, so compression cuts I/O on the hot path, not just disk.

The listing below implements delta + VByte, checks the worked example, and measures the average bytes per posting on a synthetic collection of 200,000 documents whose term document frequencies follow Zipf’s law (a few very common terms, a long tail of rare ones).

```
        """Delta (gap) encoding + variable-byte (VByte) compression of a postings list,
        and a measurement of bytes per posting on a synthetic Zipf-distributed collec-
tion."""
        import random

        def vbyte_encode(n):
            """Lucene-style VInt: 7 data bits per byte, low-order group first,
            high bit = 1 means 'more bytes follow'."""
            out = bytearray()
            while n >= 0x80:
                out.append((n & 0x7F) | 0x80)
                n >>= 7
            out.append(n)
            return bytes(out)

        def vbyte_decode_all(data):
            nums, n, shift = [], 0, 0
            for byte in data:
                n |= (byte & 0x7F) << shift
                if byte & 0x80:
                    shift += 7
                else:
                    nums.append(n)
                    n, shift = 0, 0
            return nums

        def compress_postings(doc_ids):
            gaps = [doc_ids[0]] + [b - a for a, b in zip(doc_ids, doc_ids[1:])]
            return b"".join(vbyte_encode(g) for g in gaps)

        def decompress_postings(data):
            doc_ids, total = [], 0
            for gap in vbyte_decode_all(data):
                total += gap
                doc_ids.append(total)
            return doc_ids

        if __name__ == "__main__":
            ids = [824, 829, 215406]            # the example used in Manning et al.
(2008), ch. 5
            blob = compress_postings(ids)
            print("gaps:", [ids[0]] + [b - a for a, b in zip(ids, ids[1:])])
            print("bytes:", " ".join(f"{b:08b}" for b in blob), f"({len(blob)} bytes vs
{4*len(ids)} raw)")
            assert decompress_postings(blob) == ids

            # Synthetic collection: N docs, vocabulary with Zipfian document frequen-
cies.
            random.seed(16)
            N, M, total_postings_target = 200_000, 50_000, 10_000_000
            weights = [1 / r for r in range(1, M + 1)]                # Zipf, exponent
1
            scale = total_postings_target / sum(weights)
            raw_bytes = vb_bytes = postings = 0
            for w in weights:
                df = max(1, min(N, int(w * scale)))
```

```
                doc_ids = sorted(random.sample(range(N), df))
                vb_bytes += len(compress_postings(doc_ids))
                raw_bytes += 4 * df
                postings += df
            print(f"postings={postings:,}  raw={raw_bytes/1e6:.1f} MB  vbyte=
{vb_bytes/1e6:.1f} MB  "
                  f"bytes/posting={vb_bytes/postings:.2f}  ratio=
{vb_bytes/raw_bytes:.2f}")
```

Output (about 7 s):

```
gaps: [824, 5, 214577]
bytes: 10111000 00000110 00000101 10110001 10001100 00001101 (6 bytes vs 12 raw)
postings=8,946,910  raw=35.8 MB  vbyte=12.5 MB  bytes/posting=1.39  ratio=0.35
```

What to notice:

- Decoding has a data-dependent branch per byte, which is why production engines moved to **block** codes.
- On the Zipfian collection, docID postings shrink to **1.39 bytes each**, 35% of their raw size. Manning et al. report the same order for the Reuters RCV1 collection: about 100 million postings take 400 MB as 32-bit integers and 116 MB with variable-byte gaps [Manning et al. 2008, Table 5.6].

Lucene's postings formats (Lucene 5 through at least 9) group docID deltas and frequencies into blocks of 128 and store each block with **frame-of-reference bit packing**: every value uses the bit width of the block's largest value, decoded without branches; list tails use VInt [Lucene Docs: postings format]. Bit packing decodes several times faster than VByte, especially with SIMD [Lemire and Boytsov 2015]. Zobel and Moffat survey the wider family (Elias, Golomb, VByte) and its ratio-versus-speed trade-off [Zobel and Moffat 2006].

16.3.5 Estimating index size

FORMULA

Inverted index size (one copy)

$$S_{\text{index}} \approx \underbrace{N \cdot u \cdot (b_d + b_f)}_{\text{docIDs + frequencies}} + \underbrace{N \cdot L \cdot b_p}_{\text{positions}} + \underbrace{M \cdot b_t}_{\text{dictionary}}$$

Symbol	Meaning	Units
N	Number of documents	documents
u	Average number of *distinct* terms per document (postings per document)	postings/document
L	Average number of tokens per document (one position each)	tokens/document
b_d, b_f	Compressed bytes per docID gap and per term frequency	bytes
b_p	Compressed bytes per position gap	bytes
M	Number of distinct terms in the collection	terms
b_t	Dictionary bytes per term (term text + pointers + statistics)	bytes

WORKED EXAMPLE

Worked example — 100 million product and review documents. Assume $N = 10^8$, $L = 400$ tokens, $u = 200$ distinct terms per document, $M = 5 \times 10^7$ distinct terms (part numbers and misspellings make vocabularies large), $b_d = 1.4$ bytes (measured above), $b_f = 1$ byte, $b_p = 1.5$ bytes, $b_t = 20$ bytes. Sizes in SI units (1 GB = 10^9 bytes).

1. Postings: $N \cdot u = 10^8 \times 200 = 2 \times 10^{10}$.
2. DocIDs and frequencies: $2 \times 10^{10} \times (1.4 + 1) = 4.8 \times 10^{10}$ bytes = 48 GB.
3. Positions: $N \cdot L = 4 \times 10^{10}$ tokens × 1.5 bytes = 6×10^{10} bytes = 60 GB.
4. Dictionary: $5 \times 10^7 \times 20 = 10^9$ bytes = 1 GB.
5. Total inverted index ≈ 48 + 60 + 1 = **109 GB**.
6. Uncompressed for comparison (4-byte docID, tf and position): $2 \times 10^{10} \times 8 + 4 \times 10^{10} \times 4 = 160 + 160 = 320$ GB, about 3× larger.
7. The raw text is 4×10^{10} tokens × about 6 bytes = 240 GB, so the positional index is about 45% of the text and the non-positional part about 20%.

Interpretation. Positions cost more than docIDs and frequencies combined. **Engineering implication.** Elasticsearch also stores the original JSON (`_source`), **doc values** (a column store for sorting and aggregations) and **norms** (per-field length factors); budget roughly the index again for these, then multiply by $1 + r$ for r replicas: about $2 \times (109 + 110) \approx 440$ GB with one replica, before merge headroom (§16.6). Dropping positions from fields never used in phrases is the largest single saving.

Real-world industry applications

INDUSTRY EXAMPLE

Early Google: barrels and hit lists. Brin and Page's prototype partitioned its forward index into 64 "barrels" by word ID and sorted each barrel to produce the inverted index. Each posting carried a compact "hit list" (two bytes per hit, including capitalization and font size), and "fancy" hits from titles and anchor text were kept separately so queries could consult short, high-value lists first [Brin and Page 1998]. That idea survives as field-specific and tiered indexes. *Evidence: Primary* — [Brin and Page 1998]

Failure scenarios

Table 16.2 — Inverted index failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Query on a common term is slow	Postings list of tens of millions of entries is fully decoded and scored	Slow-query log; per-term latency	Add top-*k* pruning (§16.5.6); cache results	Block-max skipping; avoid stop-word-only queries on huge fields
Index far larger than estimated	Positions and n-gram fields indexed everywhere	Per-field disk usage API	Reindex with `index_options: freqs` where phrases are unused	Review mappings before launch
Corrupted segment after disk fault	Bit rot or torn write	Checksum mismatch on open	Recover shard from a replica or snapshot	Replicas on separate failure domains; checksums (Lucene has them)

Common misconceptions

COMMON MISTAKE

Mistake — "An inverted index is just a secondary index on a text column." A B-tree maps one value per row; an inverted index maps *many* terms per document, keeps term statistics for ranking and positions for phrases, and is built and compressed in bulk. That is why relational databases ship separate full-text index types (PostgreSQL's GIN, for example).

16.4 Tokenization and analysis

16.4.1 The analysis chain

DEFINITION

Definition — Analysis. The process that converts raw text into the terms stored in (or looked up from) an inverted index. An **analyzer** is a pipeline of zero or more **character filters** (rewrite the raw character stream), exactly one **tokenizer** (split the stream into tokens with positions and offsets) and zero or more **token filters** (modify, remove or add tokens).

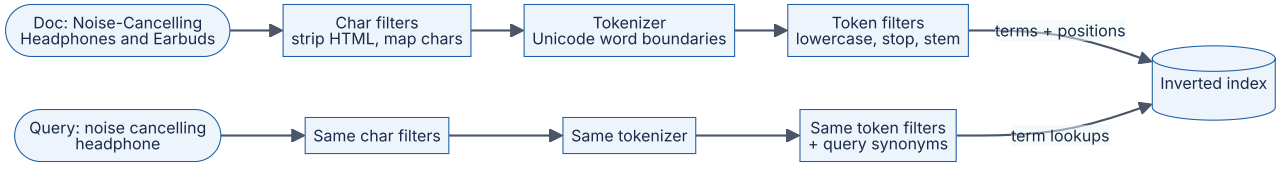
First-principles explanation

An inverted index matches terms *exactly*: `Headphones` and `headphone` are different keys, though a shopper does not care. Analysis decides which strings count as "the same word", as a librarian filing "Colour", "color" and "COLORS" under one card does by hand. Every decision moves the balance between two quality measures. **Recall** is the fraction of relevant documents that are returned; **precision** is the fraction of returned documents

that are relevant. Aggressive normalization (stemming, synonyms, folding accents) raises recall and risks precision: “university” and “universe” both stem to “univers” under the Porter stemmer. Conservative analysis does the opposite.

How it works internally

Figure 16.3 — An analysis chain applied at index time and at query time.



Reading the diagram. Two inputs enter the same pipeline: a document at index time (top) and a query at search time (bottom). Character filters run first on raw text (stripping HTML, mapping “&” to “and”); the tokenizer then splits text into tokens with positions; token filters normalize them. The index stores terms with positions; the query side only looks terms up. All steps are synchronous, in-process transformations; the only storage boundary is the index. The critical failure point is **asymmetry**: if the two pipelines diverge, for example because the index was built with stemming and the query analyzer was changed later, queries look up terms that were never stored and silently return nothing. Changing index-time analysis therefore requires a full reindex (§16.10.2); query-time-only filters such as synonyms can change without one. Analysis is CPU-bound and scales with indexer nodes.

Table 16.3 — Analysis chain components and their trade-offs.

Component	Example	Raises	Risks
Char filter	Strip HTML; map “½” to “1/2”	Clean tokens	Losing meaningful symbols
Standard tokenizer	Unicode word boundaries (UAX #29)	Language-neutral splitting	“C++”, “U.S.A.”, “e-mail” split oddly
Lowercase	“Wi-Fi” → “wi”, “fi”	Recall	“US” (country) vs “us” (pronoun)
ASCII folding	“café” → “cafe”	Recall for unaccented queries	Precision in languages where accents matter
Stop words	Drop “the”, “of”	Index size, speed	Queries like “The Who”, “to be or not to be”
Stemming	“running” → “run”	Recall	Over-stemming (“universe” ~ “university”)
Synonyms	“tv” → “television”	Recall	Drift (“apple” fruit vs company)
Edge n-grams	“head” → h, he, hea, head	Prefix matching	Index 3–10× larger

16.4.2 Tokenizers and normalization

A **whitespace tokenizer** splits on spaces and keeps punctuation attached ("head-phones,"). Lucene's **standard tokenizer** implements the Unicode word-break rules of UAX #29. Specialized tokenizers handle emails, URLs and paths, and `keyword` fields (SKUs, IDs, tags) keep the whole value as one token. **Unicode normalization** (NFC/NFKC) makes "é" as one code point equal to "e" plus a combining accent; normalization bugs are a classic cause of "the product is indexed but search cannot find it".

16.4.3 Stemming versus lemmatization

Stemming chops suffixes by rule. The Porter stemmer [Porter 1980] applies several dozen suffix-rewriting rules in five steps and maps "connection", "connected" and "connecting" to "connect". The "S" stemmer removes only plural endings [Harman 1991]; Lucene ships it as a minimal English stemmer, and this chapter's code uses it. **Lemmatization** maps a word to its dictionary form (lemma) using a vocabulary and part-of-speech information: "better" → "good", "mice" → "mouse", "was" → "be".

Table 16.4 — Stemming versus lemmatization.

	Stemming	Lemmatization
What it is	Rule-based suffix stripping	Dictionary + morphology lookup
Output	A stem, possibly not a word ("univers")	A real word (lemma)
Speed	Very fast, no dictionary	Slower; needs POS tagging for accuracy
Recall	High (aggressive)	High for irregular forms
Precision	Can conflate unrelated words	Better
Languages	Per-language rule sets (Snowball)	Needs a lexicon per language
Choose when	Large English corpora, speed matters	Morphologically rich languages, precision matters

Manning et al. observe that stemming helps recall but hurts precision, and its net effect on English retrieval quality is modest; for languages with richer morphology (Finnish, Hungarian, Arabic) normalization matters much more [Manning et al. 2008, §2.2.4].

16.4.4 Stop words

Stop words ("the", "a", "of") have the longest postings lists, so dropping them once saved much space. Modern engines usually *keep* them: their small gaps compress to a byte, BM25's IDF already gives them near-zero weight (§16.5.3), and dropping them breaks phrases and queries such as "The Who". Elasticsearch's standard analyzer keeps

them by default. If you remove them, keep the position gaps (as this chapter's analyzer does) so phrase matching stays correct.

16.4.5 n-grams and edge n-grams

A **character n-gram** is a substring of length n. Indexing all trigrams of "headphone" (hea, ead, adp, dph, …) lets the engine match substrings, misspellings and languages without spaces. An **edge n-gram** keeps only prefixes (h, he, hea, head, …), which supports search-as-you-type (§16.9.4). Both multiply the number of postings: a word of length ℓ produces $\ell - n + 1$ n-grams, so a trigram field is several times larger than a word field and its postings lists for common grams are very long.

16.4.6 Synonyms

Synonyms map equivalent terms ("tv" ⇔ "television"). At **index time** they enlarge the index and every change needs a reindex; at **query time** they cost slightly more per query but can be updated freely, so most teams apply them there. Multi-word synonyms ("ny" ⇔ "new york") produce expansions of different lengths, which Lucene represents as a token *graph*; its synonym-graph filter is meant for query time.

16.4.7 Language issues

- **No word separators.** Chinese, Japanese and Thai need dictionary-based segmenters (such as Lucene's Kuromoji for Japanese) or overlapping character bigrams, which need no dictionary but index more terms.
- **Compounds.** German "Kopfhörerverstärker" (headphone amplifier) needs decompounding so that "Kopfhörer" finds it.
- **Script rules.** Arabic needs letter-variant and diacritic normalization; Turkish has dotted and dotless "i", so naive lowercasing is wrong.
- **Mixed-language catalogs.** Give each language its own field and analyzer (`title.en`, `title.de`) rather than forcing one analyzer onto all text.

An Elasticsearch analyzer is declared in the index settings and attached to fields in the mapping:

```
        {
          "settings": {
            "analysis": {
              "filter": {
                "product_synonyms": { "type": "synonym_graph", "synonyms": ["tv, televi-
sion", "anc, noise cancelling"] }
              },
              "analyzer": {
                "product_index":  { "tokenizer": "standard", "filter": ["lowercase",
"asciifolding", "porter_stem"] },
                "product_search": { "tokenizer": "standard", "filter": ["lowercase",
"asciifolding", "product_synonyms", "porter_stem"] }
              }
            }
          },
          "mappings": {
            "properties": {
              "title": { "type": "text", "analyzer": "product_index",
"search_analyzer": "product_search" },
              "sku":   { "type": "keyword" }
            }
          }
        }
```

What to notice: the index and search analyzers share tokenizer and normalization but the synonym graph is applied only at search time; `sku` is a `keyword` field, indexed as a single unanalyzed token for exact matching.

Real-world industry applications

> INDUSTRY EXAMPLE
>
> **GitHub code search.** Source code defeats word analyzers: users search for punctuation (`foo(`), want no stemming or stop-word removal, and use regular expressions. GitHub's engine (Blackbird, written in Rust) therefore indexes character n-grams and answers a query by intersecting the postings lists of the query's n-grams, then verifying candidate documents; GitHub reported an index of about 25 TB built from 115 TB of source content [Clem 2023]. Trade-off: n-gram postings are numerous and long, which GitHub offset by indexing each unique file content only once and by sharding. *Evidence: Primary* — [Clem 2023]

Failure scenarios

Table 16.5 — Analysis failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Known document not found	Index and query analyzers differ	`_analyze` API on both strings	Align analyzers; reindex	One analyzer definition shared by both sides; tests
Irrelevant matches after a release	Aggressive stemmer or bad synonym	Relevance test set regression	Roll back query-time synonyms	Offline relevance evaluation gate
Index size jumps 5×	n-gram filter added to a large field	Disk usage alert	Move n-grams to a dedicated subfield	Size test on a sample before deploy

Common misconceptions

COMMON MISTAKE

Mistake — "Better analysis is always more normalization." Every normalization merges distinct strings. For SKUs, error codes, names and code, normalization destroys meaning. Analyze each field according to how people search it, and keep an unanalyzed `keyword` subfield for exact matching.

16.5 Ranking: TF-IDF and BM25

16.5.1 What a ranking function does

DEFINITION

Definition — Relevance score. A number computed for each (query, document) pair that estimates how well the document satisfies the query. A **ranking function** computes it; the engine returns the k highest-scoring documents.

First-principles explanation

A sensible librarian asked for "noise cancelling headphones" reasons three ways. A book mentioning *noise cancelling* ten times is more on topic than one mentioning it once, but not ten times more (**term frequency, with diminishing returns**). *Headphones* appears in half the catalog, so matching it says little, while rare *cancelling* is informative (**inverse document frequency**). And a leaflet mentioning *noise* once is more focused than an encyclopedia mentioning it once (**length normalization**). BM25 writes these three intuitions as one formula.

The theoretical justification is the **probability ranking principle**: if documents are ranked by their probability of relevance to the query, the ranking is optimal for any measure that rewards putting relevant documents first [Robertson 1977]. BM25 approximates that probability from term statistics alone [Robertson and Zaragoza 2009].

16.5.2 TF-IDF

Karen Spärck Jones observed in 1972 that a term's value for retrieval falls as the number of documents containing it rises [Spärck Jones 1972]. Combined with term frequency, this gives the classic **TF-IDF** weight, the basis of the vector space model in which documents and queries are vectors of term weights compared by cosine similarity (§4.9.2).

FORMULA

TF-IDF weight of term t in document d

$$w_{t,d} = \mathrm{tf}_{t,d} \times \log_{10} \frac{N}{\mathrm{df}_t} \qquad \mathrm{score}(q,d) = \sum_{t \in q} w_{t,d}$$

Symbol	Meaning	Units
$\mathrm{tf}_{t,d}$	Occurrences of term t in document d	count
N	Documents in the collection	documents
df_t	Documents containing t	documents
$w_{t,d}$	Weight of t in d	dimensionless

WORKED EXAMPLE

Worked example — TF-IDF on the product catalog. The body field of the eight-product catalog (§16.5.4) has $N = 8$. *noise* occurs in 3 documents and *headphone* in 6.

1. $\mathrm{idf}(\textit{noise}) = \log_{10}(8/3) = \log_{10} 2.667 = 0.426$.
2. $\mathrm{idf}(\textit{headphone}) = \log_{10}(8/6) = 0.125$.
3. A document containing *noise* once scores 0.426 for that term; one containing *noise* 10 times scores 4.26.

Interpretation. IDF makes *noise* 3.4× more valuable than *headphone*, as intended, but raw tf grows without limit (keyword stuffing wins) and nothing corrects for length. **Engineering implication.** Lucene's TF-IDF variant (with $\sqrt{\mathrm{tf}}$ and length factors) was the default until Lucene 6 and Elasticsearch 5.0 switched to BM25 [Elastic Docs: similarity].

16.5.3 BM25

BM25 ("Best Matching 25", the 25th weighting scheme in the Okapi experiments at City University London) comes from the probabilistic relevance framework [Robertson and Zaragoza 2009]. It keeps IDF, makes term frequency **saturate**, and normalizes by document length relative to the average.

FORMULA

BM25 inverse document frequency (Lucene form)

$$\mathrm{IDF}(t) = \ln\left(1 + \frac{N - n_t + 0.5}{n_t + 0.5}\right)$$

Symbol	Meaning	Units
N	Documents in the collection (or shard; see §16.7.5)	documents
n_t	Documents containing term t (document frequency)	documents
$\mathrm{IDF}(t)$	Weight of a match on t	dimensionless

The classic Robertson–Spärck Jones form, $\ln \frac{N - n_t + 0.5}{n_t + 0.5}$, becomes *negative* for terms in more than half the documents; Lucene adds 1 inside the logarithm so that every match contributes a small positive amount.

FORMULA

BM25 score

$$\mathrm{score}(q, d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{f_{t,d}\,(k_1 + 1)}{f_{t,d} + k_1\left(1 - b + b \cdot \frac{|d|}{\mathrm{avgdl}}\right)}$$

Symbol	Meaning	Units
$f_{t,d}$	Frequency of term t in document (field) d	count
$\lvert d\rvert$	Length of d in terms after analysis	terms
avgdl	Average length of that field across the collection	terms
k_1	Term-frequency saturation parameter; Lucene default 1.2 (typical 1.2–2.0)	dimensionless
b	Length-normalization strength, 0 (none) to 1 (full); default 0.75	dimensionless
$\mathrm{score}(q, d)$	Relevance score; only comparable within one query	dimensionless

Two properties follow from the formula. As $f_{t,d} \to \infty$ the fraction approaches $k_1 + 1$, so a term can never contribute more than $(k_1 + 1) \cdot \mathrm{IDF}$: repetition has diminishing returns. And

the denominator grows with $|d|/\text{avgdl}$, so the same tf in a longer document scores less; b controls how strongly.

Table 16.6 — BM25 term-frequency factor ($k_1 = 1.2$, $b = 0.75$) versus raw tf.

tf	Raw TF-IDF factor	BM25, $\|d\|$ = 0.5 avgdl	BM25, $\|d\|$ = avgdl	BM25, $\|d\|$ = 2 avgdl
1	1	1.257	1.000	0.710
2	2	1.600	1.375	1.073
5	5	1.913	1.774	1.549
10	10	2.047	1.964	1.818
∞	∞	2.200	2.200	2.200

Reading down a column shows saturation: 1 to 10 occurrences raises the BM25 factor by less than 2.6×, against 10× for raw tf. Reading across a row shows length normalization: the same tf is worth less in a longer document.

WORKED EXAMPLE

Worked example — BM25 for *noise* in two product descriptions. From the catalog in §16.5.4: $N = 8$ body fields, $n_{noise} = 3$, avgdl = 68/8 = 8.5 terms. Document 4 (earbuds) has $|d| = 7$; document 7 (kids' headphones) has $|d| = 12$. Each contains *noise* once. $k_1 = 1.2$, $b = 0.75$.

1. IDF: $\ln(1 + \frac{8 - 3 + 0.5}{3 + 0.5}) = \ln(1 + 1.5714) = \ln 2.5714 = 0.9445$.
2. Length factor, document 4:
   $k_1(1 - b + b \cdot 7/8.5) = 1.2 \times (0.25 + 0.6176) = 1.2 \times 0.8676 = 1.0412$.
3. TF part, document 4: $\frac{1 \times 2.2}{1 + 1.0412} = \frac{2.2}{2.0412} = 1.0778$.
4. Score, document 4: $0.9445 \times 1.0778 = 1.018$.
5. Length factor, document 7: $1.2 \times (0.25 + 0.75 \times 12/8.5) = 1.2 \times 1.3088 = 1.5706$; TF part $2.2/2.5706 = 0.8558$; score $0.9445 \times 0.8558 = 0.808$.
6. Saturation check for document 4: with tf = 2 the TF part would be $4.4/3.0412 = 1.447$, with tf = 5 it would be $11/6.0412 = 1.821$, never exceeding 2.2.

Interpretation. The same single mention of *noise* is worth 26% more in the short, focused earbuds description than in the longer kids' description, and repeating it five times would raise its weight by only 69%. The listing below reproduces 1.0179 and 0.8083. **Engineering implication.** BM25 resists keyword stuffing and long-document bias out of the box, which is why it is the default in Lucene, Elasticsearch, OpenSearch and Solr. Tune k_1 and b per field only against a relevance test set; for very short fields such as titles, a lower b is common because length differences carry little meaning.

> NOTE
>
> **Lucene's scores versus the textbook formula.** Lucene 8 removed the constant factor $(k_1 + 1)$ from the numerator because it does not change the ranking; Elasticsearch preserves the classic score values by applying the constant as a boost, which is why `explain` output in Elasticsearch shows a factor of 2.2 [Lucene MIGRATE notes for 8.0]. Lucene also stores document length in a lossy one-byte **norm**, so its scores differ slightly from exact arithmetic.

16.5.4 A complete engine in one listing

The listing below implements the analysis chain of §16.4 (tokenize, lowercase, stop words keeping position gaps, S-stemmer), a positional inverted index over two fields, BM25 with per-field boosts, and phrase matching.

```
        """A tiny search engine: analysis, a positional inverted index, BM25 with field
boosts.
        Standard library only. Run: python3 search_engine.py"""
        import math
        import re
        from collections import defaultdict

        STOP_WORDS = {"a", "an", "and", "the", "for", "with", "of", "in", "to", "is",
"are"}

        def s_stem(token):
            """Harman's 'S' stemmer: strips only plural endings (like Lucene's
EnglishMinimalStemFilter)."""
            if len(token) > 3 and token.endswith("ies") and not token.endswith(("eies",
"aies")):
                return token[:-3] + "y"
            if len(token) > 3 and token.endswith("es") and not token.endswith(("aes",
"ees", "oes")):
                return token[:-1]
            if len(token) > 3 and token.endswith("s") and not token.endswith(("us",
"ss")):
                return token[:-1]
            return token

        def analyze(text):
            """Tokenize on non-alphanumerics, lowercase, drop stop words, stem. Keeps
positions."""
            out = []
            for pos, raw in enumerate(re.findall(r"[a-z0-9]+", text.lower())):
                if raw not in STOP_WORDS:
                    out.append((s_stem(raw), pos))
            return out

        class InvertedIndex:
            def __init__(self, fields):
                self.fields = fields
                # postings[field][term] -> {doc_id: [positions]}  (doc_ids inserted in
increasing order)
                self.postings = {f: defaultdict(dict) for f in fields}
                self.doc_len = {f: {} for f in fields}
                self.n_docs = 0

            def add(self, doc_id, doc):
                self.n_docs += 1
                for f in self.fields:
                    tokens = analyze(doc[f])
                    self.doc_len[f][doc_id] = len(tokens)
                    for term, pos in tokens:
                        self.postings[f][term].setdefault(doc_id, []).append(pos)

            def avgdl(self, f):
                return sum(self.doc_len[f].values()) / self.n_docs

            def idf(self, f, term):
                n = len(self.postings[f].get(term, {}))
                return math.log(1 + (self.n_docs - n + 0.5) / (n + 0.5))   # Lucene's
```

```
BM25 IDF

            def bm25_term(self, f, term, doc_id, k1=1.2, b=0.75):
                tf = len(self.postings[f].get(term, {}).get(doc_id, []))
                if tf == 0:
                    return 0.0
                norm = k1 * (1 - b + b * self.doc_len[f][doc_id] / self.avgdl(f))
                return self.idf(f, term) * tf * (k1 + 1) / (tf + norm)

            def search(self, query, boosts, k=5):
                terms = [t for t, _ in analyze(query)]
                scores = defaultdict(float)
                for f, boost in boosts.items():
                    for term in terms:                          # OR semantics: any match-
ing term scores
                        for doc_id in self.postings[f].get(term, {}):
                            scores[doc_id] += boost * self.bm25_term(f, term, doc_id)
                return sorted(scores.items(), key=lambda kv: (-kv[1], kv[0]))[:k]

            def phrase(self, f, phrase):
                """Docs where the analyzed terms appear at consecutive positions (stop-
word gaps kept)."""
                toks = analyze(phrase)
                first, offsets = toks[0][0], [p - toks[0][1] for _, p in toks]
                hits = []
                for doc_id, positions in self.postings[f].get(first, {}).items():
                    for p in positions:
                        if all(p + off in self.postings[f].get(t, {}).get(doc_id, [])
                               for (t, _), off in zip(toks[1:], offsets[1:])):
                            hits.append(doc_id)
                            break
                return sorted(hits)

        DOCS = {
            1: {"title": "Wireless noise cancelling headphones",
                "body": "Over-ear wireless headphones with active noise cancelling and a
30 hour battery."},
            2: {"title": "Wired studio headphones",
                "body": "Closed-back wired headphones for mixing and mastering in the
studio."},
            3: {"title": "Portable Bluetooth speaker",
                "body": "Waterproof wireless speaker with deep bass. Pairs with phones
and headphones."},
            4: {"title": "Noise cancelling wireless earbuds",
                "body": "True wireless earbuds with noise cancelling and a charging
case."},
            5: {"title": "Headphone stand",
                "body": "Aluminum stand for headphones with a cable organizer."},
            6: {"title": "Wireless charging pad",
                "body": "Fast wireless charger for phones and earbuds. Charging cable
included."},
            7: {"title": "Kids headphones",
                "body": "Volume-limited wired headphones for kids. Foldable, light, and
less noise on long trips."},
            8: {"title": "Travel case for headphones and earbuds",
                "body": "Hard shell case that protects headphones, earbuds, cables and
chargers on trips."},
```

```
        }

        if __name__ == "__main__":
            idx = InvertedIndex(["title", "body"])
            for doc_id, doc in DOCS.items():
                idx.add(doc_id, doc)

            print("analyze('Noise-Cancelling Headphones'):", analyze("Noise-Cancelling
Headphones"))
            print("postings[body]['noise']:", dict(idx.postings["body"]["noise"]))
            print(f"N={idx.n_docs}  avgdl(title)={idx.avgdl('title'):.3f}  avgdl(body)=
{idx.avgdl('body'):.3f}")
            q = "wireless noise cancelling headphones"
            for f in ("title", "body"):
                row = {t: (len(idx.postings[f].get(t, {})), round(idx.idf(f, t), 4))
for t, _ in analyze(q)}
                print(f"df, idf in {f}:", row)
            print("\nBM25, body only:")
            for d, s in idx.search(q, {"body": 1.0}, k=8):
                print(f"  doc {d}: {s:.4f}  ({DOCS[d]['title']})")
            print("\nBM25, title^2 + body:")
            for d, s in idx.search(q, {"title": 2.0, "body": 1.0}, k=8):
                print(f"  doc {d}: {s:.4f}  ({DOCS[d]['title']})")
            print("\nterm 'noise' in body of doc 7:", round(idx.bm25_term("body",
"noise", 7), 4),
                  " len =", idx.doc_len["body"][7])
            print("term 'noise' in body of doc 4:", round(idx.bm25_term("body", "noise",
4), 4),
                  " len =", idx.doc_len["body"][4])
            print("phrase 'noise cancelling' in body:", idx.phrase("body", "noise can-
celling"))
            print("phrase 'cancelling noise' in body:", idx.phrase("body", "cancelling
noise"))
```

Output:

```
analyze('Noise-Cancelling Headphones'): [('noise', 0), ('cancelling', 1), ('headphone',
2)]
postings[body]['noise']: {1: [6], 4: [4], 7: [10]}
N=8  avgdl(title)=3.125  avgdl(body)=8.500
df, idf in title: {'wireless': (3, 0.9445), 'noise': (2, 1.2809), 'cancelling': (2,
1.2809), 'headphone': (5, 0.4925)}
df, idf in body: {'wireless': (4, 0.6931), 'noise': (3, 0.9445), 'cancelling': (2,
1.2809), 'headphone': (6, 0.3254)}

BM25, body only:
  doc 4: 3.1456  (Noise cancelling wireless earbuds)
  doc 1: 3.0255  (Wireless noise cancelling headphones)
  doc 7: 1.0868  (Kids headphones)
  doc 3: 1.0437  (Portable Bluetooth speaker)
  doc 6: 0.7102  (Wireless charging pad)
  doc 5: 0.3913  (Headphone stand)
  doc 2: 0.3507  (Wired studio headphones)
  doc 8: 0.2905  (Travel case for headphones and earbuds)

BM25, title^2 + body:
  doc 1: 10.2012  (Wireless noise cancelling headphones)
  doc 4: 9.4376  (Noise cancelling wireless earbuds)
  doc 6: 2.6306  (Wireless charging pad)
  doc 7: 2.2419  (Kids headphones)
  doc 5: 1.5464  (Headphone stand)
  doc 2: 1.3521  (Wired studio headphones)
  doc 8: 1.1742  (Travel case for headphones and earbuds)
  doc 3: 1.0437  (Portable Bluetooth speaker)

term 'noise' in body of doc 7: 0.8083  len = 12
term 'noise' in body of doc 4: 1.0179  len = 7
phrase 'noise cancelling' in body: [1, 4]
phrase 'cancelling noise' in body: []
```

What to notice:

- **Body-only BM25 ranks the earbuds above the headphones** that exactly match the query. Document 4's description is shorter (7 terms versus 10), and *headphone* is so common (in 6 of 8 bodies, IDF 0.33) that matching it barely helps. Lexical scoring has no idea that headphones are what the shopper wants.
- **Boosting the title field 2×** fixes the order: the title "Wireless noise cancelling headphones" matches all four terms, adding $2 \times 3.588 = 7.18$ to document 1. Field structure is one of the strongest relevance signals available.
- The stop word "with" was removed, but positions were kept, so *noise* in document 4 is at position 4, not 3; phrase queries stay correct, and word order matters ("cancelling noise" matches nothing).
- A real engine never scores every matching document this way; see §16.5.6.

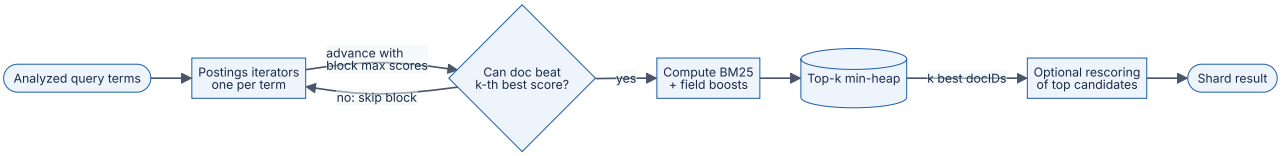
16.5.5 Field boosting and beyond

Field boosting weights each field's score, as in Elasticsearch's `multi_match` with `"fields": ["title^2", "body"]`. Summing per-field scores double-counts a term that appears in both fields, each saturating separately; **BM25F** combines weighted term frequencies across fields *before* saturation [Robertson et al. 2004], as Elasticsearch's `combined_fields` query does. Text relevance is rarely the whole ranking: **function scoring** adds signals such as $\log(1 + \text{sales})$, and **learning to rank** reorders a few hundred BM25 candidates with a trained model (Chapter 81).

16.5.6 Retrieving the top *k* efficiently

Scoring every document that matches any query term is wasteful: an OR query containing *headphone* may match millions of documents, but the user sees ten. Because each term's contribution is bounded by $(k_1 + 1) \cdot \text{IDF}(t)$, an engine can compute an upper bound on any document's score and skip documents that cannot beat the current k-th best score. **WAND** and its refinement **Block-Max WAND** store the maximum score per postings block and skip whole blocks [Ding and Suel 2011]. Lucene 8 added block-max WAND, and Elasticsearch 7 accordingly stopped counting total hits exactly by default (`track_total_hits` defaults to 10,000), since an exact count would force every match to be visited [Elastic Docs: track_total_hits].

Figure 16.4 — Ranked query pipeline inside one shard.



Reading the diagram. Each query term opens an iterator over its postings list. Before fully scoring a candidate, the WAND decision compares the sum of per-block maximum scores with the score of the current k-th best document held in a min-heap; if the bound is too low, whole blocks are skipped (the loop back to the iterators). Survivors are scored exactly and offered to the heap, an in-memory structure of size k. An optional rescoring stage applies more expensive signals (phrase proximity, a learned model) to only the top few hundred. All steps are synchronous and CPU-bound within one shard; the storage boundary is the postings files, usually served from the operating system page cache. Failure points are queries whose terms are all very common (little skipping possible) and very large k (deep pagination, §16.7.4). Scaling is by splitting the index into more shards, each running this pipeline in parallel.

Real-world industry applications

INDUSTRY EXAMPLE

Wikipedia's CirrusSearch. Wikimedia runs on-wiki search for Wikipedia and its sister projects with CirrusSearch, a MediaWiki extension backed by Elasticsearch. Pages are indexed with per-language analysis chains, and ranking combines text relevance with signals such as incoming links and page popularity, with rescoring profiles configurable per wiki [MediaWiki: CirrusSearch]. *Evidence: Primary* — [MediaWiki: CirrusSearch]

Failure scenarios

Table 16.7 — Ranking failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Accessories outrank the product searched for	Text match ignores intent; short bodies win	Click-through and relevance judgments	Boost title; add category signals	Offline evaluation set (NDCG)
Keyword stuffing by sellers	Unbounded tf in a custom scorer	Manual review, anomaly scores	Switch to BM25; cap field length	BM25 saturation; quality signals

Common misconceptions

COMMON MISTAKE

Mistake — "A BM25 score of 12 means a better match than a score of 6 on another query." BM25 scores depend on the query's terms, their IDF and the collection statistics. They rank documents *within* one query and have no absolute meaning; never threshold them globally or compare them across queries or indexes. Normalize, or use rank positions, when fusing result lists (§16.11).

16.6 Index construction and segment merging

16.6.1 Why indexes are built from immutable segments

DEFINITION

Definition — Segment. A self-contained, immutable mini-index (its own dictionary, postings, stored fields, norms and doc values) covering a subset of documents. A Lucene index is a set of segments plus a **commit point** file (`segments_N`) listing which segments are live; a search runs over every live segment and combines the results.

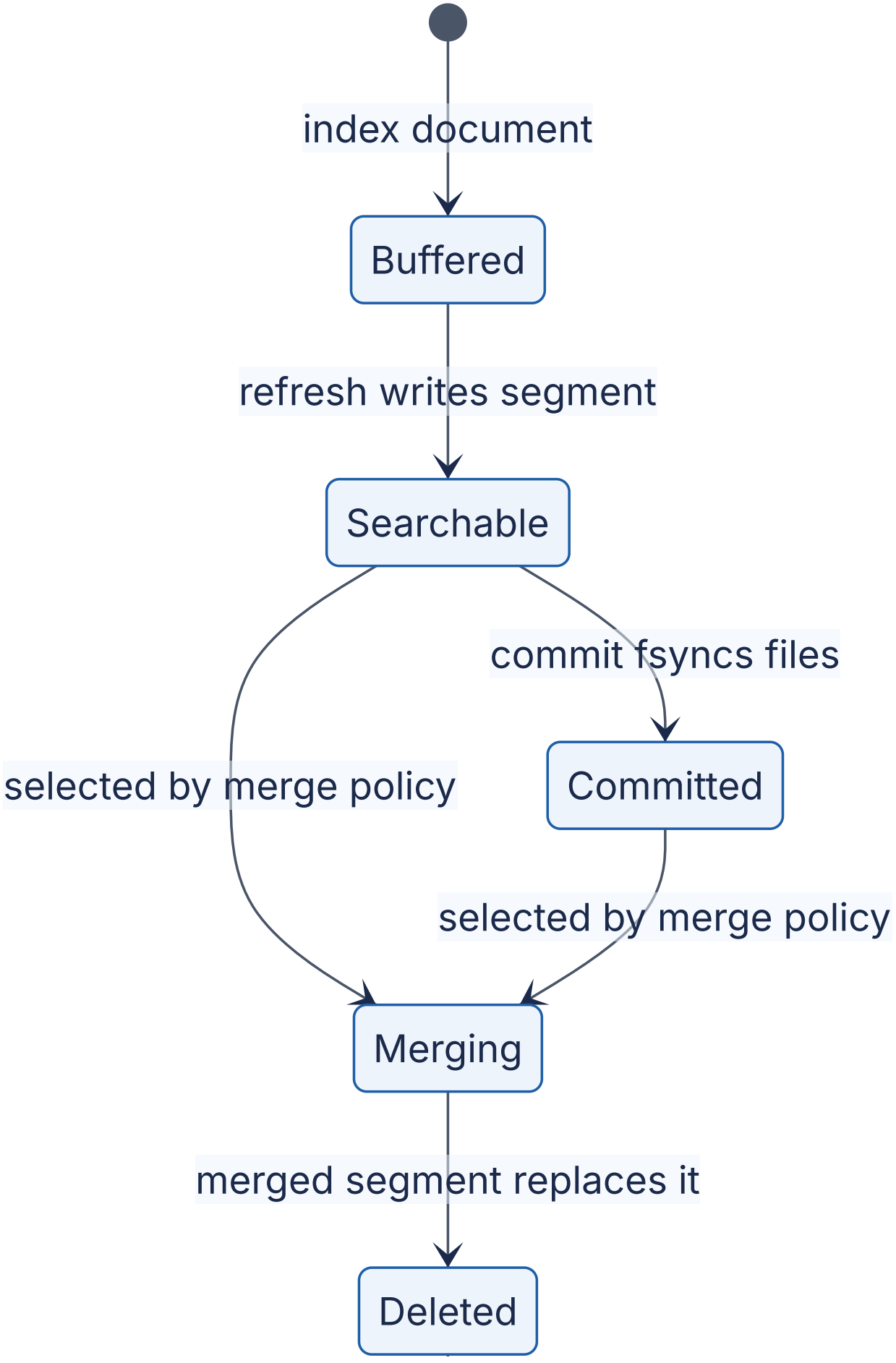
First-principles explanation

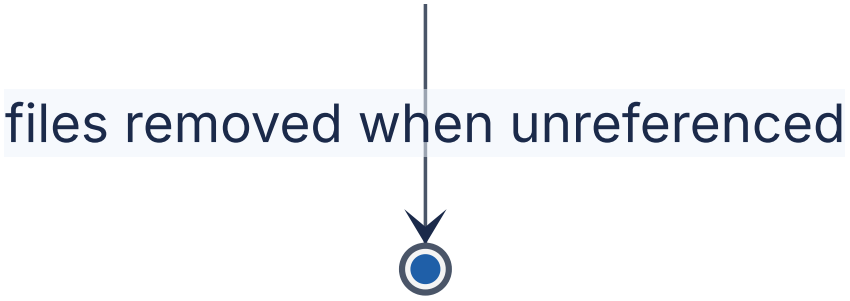
Compressed, sorted postings are excellent to read and terrible to update: inserting a docID mid-list means rewriting it. So search engines do what log-structured databases do (§13.7–§13.8): buffer new documents in memory, write the buffer out as a new small segment, and **merge** small segments into larger ones in the background, with segments playing the role of SSTables. Manning et al. describe the batch ancestors, blocked sort-based and single-pass in-memory indexing [Manning et al. 2008, ch. 4]. Immutable segments can be cached without invalidation, read without locks, copied as plain files for backup or replication, and compressed aggressively.

How it works internally

1. **Buffer.** The index writer analyzes incoming documents into an in-memory buffer. Documents in the buffer are *not* searchable.
2. **Refresh (flush to a segment).** The buffer is written as a new segment and a new searcher is opened that includes it. In Lucene terms this is opening a near-real-time reader; Elasticsearch calls it a **refresh** and by default does it every second (§16.8.4). The segment may sit only in the operating system's page cache: it is searchable but not yet durable.
3. **Commit.** Periodically, segment files are fsynced and a new `segments_N` file is written atomically. Only committed segments survive a crash without replay; Elasticsearch covers the gap with its translog (§16.8.3).
4. **Merge.** A background merge scheduler picks groups of segments, merges their postings into one new segment (dropping deleted documents), then swaps it in and deletes the old files once no reader uses them.

Figure 16.5 — Lifecycle of a Lucene segment.


index document
Buffered
refresh writes segment
Searchable
commit fsyncs files
selected by merge policy
Committed
selected by merge policy
Merging
merged segment replaces it
Deleted



Reading the diagram. A document enters the in-memory buffer (not yet visible). A refresh writes the buffer out as a segment that searches can see, a cheap operation because it need not wait for the disk. A commit fsyncs segment files and records them in the commit point, making them crash-durable; this is the expensive storage boundary. Either a searchable or a committed segment can be chosen for merging; once the merged segment is in place, the originals are marked deleted and their files removed when the last reader using them closes. All transitions are asynchronous background work relative to queries. Failure points: a crash loses searchable-but-uncommitted segments (recovered from the translog in Elasticsearch); a merge that runs out of disk fails and must be retried. Merging is throttled so it does not starve queries of I/O.

16.6.2 Deletes and updates as tombstones

Immutable segments cannot remove a document. Instead Lucene records deletions in a per-segment **live-docs bitset** (one bit per document) written as a separate small file: a **tombstone**. Searches skip documents whose bit is cleared. An **update** is a delete of the old version plus an add of the new one, which lands in a new segment. The space and the CPU spent iterating past deleted postings are reclaimed only when the segment is merged.

Consequences: deleted documents still count in term statistics until merged away, so replicas with different merge histories can score slightly differently; and update-heavy fields (prices, stock) generate many deletes, so keep volatile fields out of the text index where possible.

16.6.3 Merge policy

A **merge policy** decides which segments to merge. Lucene's default, `TieredMergePolicy`, groups segments into size tiers, allows about 10 segments per tier, merges up to 10 similar-sized segments at a time, favors merges that reclaim many deletes, and by default stops producing segments larger than about 5 GB [Lucene Docs: TieredMergePolicy]. The analysis is the same as size-tiered compaction in LSM stores (§13.9): each byte is rewritten once per tier it climbs.

FORMULA

Write amplification of tiered merging

$$W \approx 1 + \log_m \frac{S_{\max}}{S_{\text{flush}}}$$

Symbol	Meaning	Units
W	Bytes written to disk per byte of new index data	dimensionless
m	Merge factor: segments merged together per tier	count
$S_{\max}$	Maximum merged segment size	bytes
S_{flush}	Typical size of a newly flushed segment	bytes

WORKED EXAMPLE

Worked example — merge cost at a 1-second refresh. An index receives 2,000 documents/s; each adds about 1 KB to the index; refresh every 1 s gives $S_{\text{flush}} = 2$ MB. $S_{\max} = 5$ GB, $m = 10$.

1. $S_{\max}/S_{\text{flush}} = 5,000/2 = 2,500$.
2. $\log_{10} 2,500 = 3.40$, so $W \approx 4.4$.
3. Write bandwidth: $2,000 \times 1\ \text{KB} \times 4.4 = 8.8$ MB/s, plus the translog.
4. With a 30-second refresh, $S_{\text{flush}} = 60$ MB, ratio 83, $\log_{10} 83 = 1.92$, $W \approx 2.9$: one-third less merge I/O.

Interpretation. Frequent refreshes create many tiny segments that are rewritten several times. **Engineering implication.** For bulk loads and write-heavy indexes, lengthen or disable refresh (§16.10.1). For read-only indexes (yesterday's logs), **force-merge** to one segment once: queries then search one dictionary instead of dozens.

The following simulation of an idealized tiered policy confirms the formula: with three tiers above the flush size and $m = 10$, every unit of data is written exactly four times.

```
        """Simulate a simplified tiered merge policy and measure write amplification:
        every refresh flushes one small segment; whenever `merge_factor` segments of the
same tier
        exist they are merged into one segment of the next tier, until the size cap is
reached."""
        import math

        def simulate(flushes, merge_factor=10, max_tier=3):
            tiers = [0] * (max_tier + 1)      # number of live segments in each tier
            written = 0                       # units written: flushes + merge output
            for _ in range(flushes):
                tiers[0] += 1
                written += 1                  # the flush itself writes one unit
                t = 0
                while t < max_tier and tiers[t] == merge_factor:
                    tiers[t] = 0
                    tiers[t + 1] += 1
                    written += merge_factor ** (t + 1)    # merged segment size in units
                    t += 1
            return written / flushes, sum(tiers)

        if __name__ == "__main__":
            for flushes in (1_000, 100_000):
                wa, live = simulate(flushes)
                print(f"flushes={flushes:>7,}  write amplification={wa:.2f}  live
segments={live}")
            print("predicted 1 + log10(max/min) =", 1 + math.log10(10 ** 3))
```

Output:

```
flushes=  1,000  write amplification=4.00  live segments=1
flushes=100,000  write amplification=4.00  live segments=100
predicted 1 + log10(max/min) = 4.0
```

What to notice: once segments reach the cap they stop merging, so the number of live segments grows linearly with data (100 capped segments after 100,000 flushes); that is why large indexes are split into shards and time-based indexes rather than grown without bound.

COMMON MISTAKE

Mistake — “Deleting documents frees disk space immediately.” A delete sets a bit. Disk is reclaimed only when a merge rewrites the segment, and a merge temporarily needs space for both old and new segments. Clusters that are nearly full can fail to merge, accumulate deletes, and degrade; keep at least 15–20% free disk on data nodes and watch Elasticsearch’s disk watermarks.

16.7 Distributed search

16.7.1 Document partitioning versus term partitioning

When an index outgrows one node, or one node cannot serve the query rate, the index is partitioned. Search engines use the word **shard** for a partition (§14.6); this chapter uses the two interchangeably. There are two ways to cut an inverted index [Manning et al. 2008, §20.3].

- **Document partitioning** (local index): each shard holds *all* terms for a subset of documents, a complete small index. Every query goes to every shard, and a coordinator merges the per-shard top-k lists.
- **Term partitioning** (global index): each shard holds the *complete* postings lists for a subset of terms. A query visits only the shards owning its terms, but a multi-term query must ship long postings lists between nodes to intersect or score them.

Figure 16.6 — Document partitioning versus term partitioning.



Reading the diagram. In the upper subgraph the query fans out synchronously to every document shard; each shard runs the complete pipeline of Figure 16.4 on its own documents and returns only its top k (a few kilobytes), which a coordinator merges. In the lower subgraph the query goes only to the shards that own its terms, but each returns an entire postings list, potentially megabytes, which must be intersected and scored centrally. Each shard is its own storage boundary. Failure points: under document partitioning a slow or dead shard affects every query (mitigated by replicas); under term partitioning a hot term overloads one shard, and a dead shard makes every query containing its terms incomplete. Document partitioning scales by adding shards and replicas; term partitioning scales poorly for multi-term queries because network transfer grows with postings length.

Table 16.8 — Document versus term partitioning.

	Document partitioning	Term partitioning
What it is	Each shard indexes a subset of documents	Each shard owns all postings of a subset of terms
Query fan-out	All shards	Only shards owning query terms
Network per query	Small (top-k per shard)	Large (full postings lists)
Load balance	Even (random document assignment)	Skewed (Zipfian term popularity)
Indexing a document	Writes to one shard	Writes to many shards (one per term)
Tail latency	Max of S shard latencies	Fewer shards, but long transfers
Used by	Lucene-based engines, Google (as published), Earlybird	Research systems; rarely in production
Choose when	Almost always	Very high query rate of single-term lookups

Document partitioning wins in practice because it is simple, balanced and fault-tolerant; Dean's retrospective describes Google's serving system as replicated document-partitioned shards behind fan-out servers [Dean 2009].

16.7.2 Replicas

Each shard has r **replicas** on other nodes and zones, for **availability** and **query throughput** (any copy can answer). Replicas do not speed up indexing: every copy indexes every write. If each query costs c CPU-seconds on each of S shards at peak rate λ, the cluster needs about $\lambda S c / U$ cores at utilization U: 2,000 queries/s × 10 shards × 4 ms ÷ 0.5 = 160 cores, or 10 sixteen-core nodes holding, say, 20 shard copies.

16.7.3 Scatter–gather and tail latency

A document-partitioned query is a **scatter-gather**: a coordinator scatters the query to one copy of each of S shards and gathers their answers. The response time is the *maximum* of S shard latencies, so the fan-out amplification of §4.8.4 applies directly.

FORMULA

Scatter-gather tail latency

$$P(\text{query fast}) = p^S \qquad q_{\text{shard}} = q_{\text{query}}^{1/S}$$

Symbol	Meaning	Units
p	Probability one shard answers within threshold T	dimensionless
S	Number of shards the query waits for	count
$P(\text{query fast})$	Probability all S shards answer within T	dimensionless
q_{query}	Target percentile for the whole query (e.g. 0.99)	dimensionless
q_{shard}	Percentile each shard must meet at T	dimensionless

WORKED EXAMPLE

Worked example — 20 and 100 shards. Each shard's latency is log-normal with median 8 ms and p99 25.6 ms (simulated below). Assume independence.

1. $S = 20$: $P(\text{all} \le 25.6\text{ ms}) = 0.99^{20} = 0.818$; 18% of queries wait longer than the shard p99.
2. For the *query* p99 to be 25.6 ms, each shard needs $0.99^{1/20} = 0.99950$, its p99.95 at 25.6 ms.
3. $S = 100$: $0.99^{100} = 0.366$. The query median becomes the shard's $0.5^{1/100} =$ p99.3, which is 27.4 ms, more than three times the shard median, and the query p99 is about 51 ms.

Interpretation. Adding shards makes every shard's work smaller but makes the query wait for the unluckiest of more shards. **Engineering implication.** Use the fewest shards that meet size and throughput needs (§16.8.2), and use tail-tolerant techniques: **hedged requests** to a second replica after a delay [Dean and Barroso 2013], **adaptive replica selection** that routes to the replica with the best recent latency and queue length (on by default in Elasticsearch since 7.0, based on the C3 algorithm [Suresh et al. 2015]), and per-shard **timeouts with partial results**, accepting slightly incomplete answers rather than slow ones.

```
        """Scatter-gather tail latency: a query waits for the slowest of S shards.
        Monte Carlo check of P(all fast) = p**S, and the latency percentiles of max(S
shard latencies)."""
        import math
        import random

        def shard_latency_ms(rng):
            # Log-normal per-shard latency: median 8 ms, a heavy-ish right tail (p99 ~
25.6 ms).
            return 8.0 * math.exp(rng.gauss(0.0, 0.5))

        def percentile(sorted_vals, q):
            return sorted_vals[min(len(sorted_vals) - 1, int(q * len(sorted_vals)))]

        if __name__ == "__main__":
            rng = random.Random(42)
            trials = 20_000
            single = sorted(shard_latency_ms(rng) for _ in range(200_000))
            t99 = percentile(single, 0.99)
            print(f"one shard: p50={percentile(single, 0.50):.1f} ms  p99={t99:.1f} ms")
            print(" S   P(all<shard p99) sim  0.99**S   query p50   query p99")
            for S in (1, 10, 50, 100, 200):
                maxes = sorted(max(shard_latency_ms(rng) for _ in range(S)) for _ in
range(trials))
                fast = sum(m <= t99 for m in maxes) / trials
                print(f"{S:>3}   {fast:>18.3f}  {0.99 ** S:>7.3f}   {percentile(maxes,
0.5):>6.1f} ms"
                      f"   {percentile(maxes, 0.99):>6.1f} ms")
```

Output:

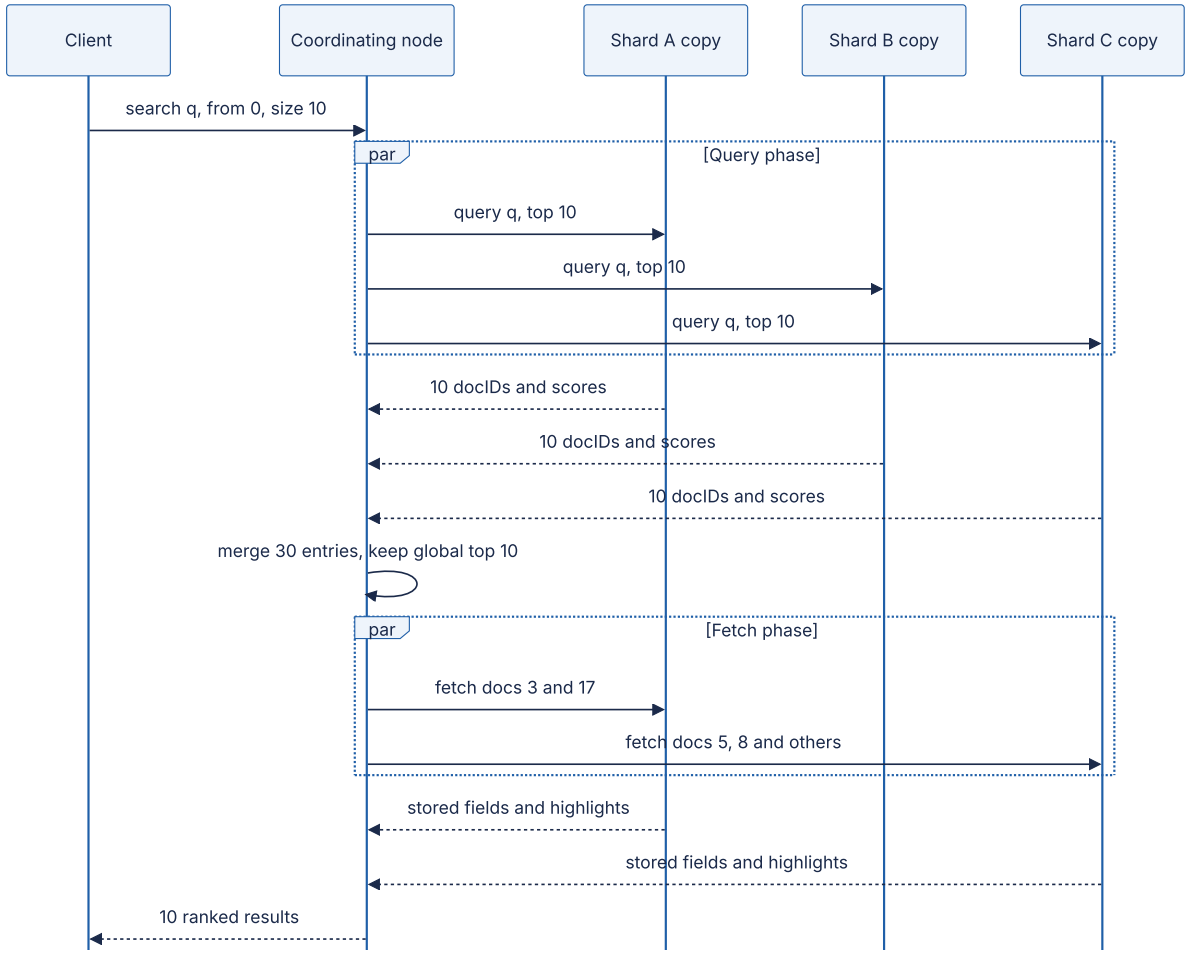
```
one shard: p50=8.0 ms  p99=25.6 ms
 S   P(all<shard p99) sim  0.99**S   query p50   query p99
  1               0.991    0.990      8.0 ms     25.2 ms
 10               0.904    0.904     16.9 ms     37.7 ms
 50               0.614    0.605     24.0 ms     46.9 ms
100               0.367    0.366     27.4 ms     51.1 ms
200               0.136    0.134     30.9 ms     56.3 ms
```

What to notice: the simulation matches 0.99^S; the query *median* at 100 shards (27.4 ms) is worse than a single shard's p99. Real shard latencies are also correlated (garbage-collection pauses, merges, a noisy neighbor), which makes the tail worse than the independent model.

16.7.4 Two-phase query-then-fetch

Shipping full documents from every shard would waste bandwidth, since most are discarded during the merge. Elasticsearch therefore runs a search in two phases [Elastic Docs: search_type].

Figure 16.7 — Query-then-fetch across three shards.



Reading the diagram. The client sends one request to any node, which acts as **coordinating node**. In the **query phase** it sends the query in parallel to one copy of every shard; each returns only docIDs and scores for its local top 10. The coordinator merges the 30 entries into a global top 10. In the **fetch phase** it asks only the shards that own those 10 documents for stored fields (`_source`) and highlighted snippets; shard B contributed nothing to the top 10, so it is not contacted. All calls are synchronous RPCs over the cluster transport protocol; each shard copy is a storage boundary. Failure points: a slow shard delays the query phase for everyone (§16.7.3); a shard copy that fails between phases forces a retry on another replica. The design scales because the expensive document payload is read for only k documents, not $S \times k$.

FORMULA

Deep pagination cost at the coordinator

$$E = S \cdot (from + size)$$

Symbol	Meaning	Units
S	Shards queried	count
from	Offset of the first requested result	results
size	Results per page	results
E	Entries every query phase must return and the coordinator must merge	entries

WORKED EXAMPLE

Worked example — page 500. $S = 20$ shards, 20 results per page.

1. Page 1: $E = 20 \times (0 + 20) = 400$ entries.
2. Page 500: $from = 499 \times 20 = 9,980$, so $E = 20 \times 10,000 = 200,000$ entries, 500× more, and each shard must also keep a heap of 10,000.

Interpretation. Deep pages cost in proportion to their depth. **Engineering implication.** Elasticsearch rejects requests with *from*+*size* above `index.max_result_window`, 10,000 by default. Use `search_after` with a sort tiebreaker (and a point-in-time snapshot) for deep scrolling, and the scroll or point-in-time APIs for exports.

16.7.5 Global versus local IDF

Each shard computes BM25 with its own N, n_t and avgdl. When documents are spread uniformly at random, local statistics are close to global ones. When they are not, scores from different shards are not comparable.

WORKED EXAMPLE

Worked example — skewed term distribution. Two shards hold 1,000,000 documents each. Custom routing by seller put most “anc” listings on shard B: shard A has 10 documents containing *anc*, shard B has 1,000.

1. Local IDF on A: $\ln(1 + 999,990.5/10.5) = 11.46$.
2. Local IDF on B: $\ln(1 + 999,000.5/1,000.5) = 6.91$.
3. Global IDF ($N = 2,000,000$, $n = 1,010$): $\ln(1 + 1,998,990.5/1,010.5) = 7.59$.

Interpretation. An *anc* match on shard A scores 66% higher than an identical match on shard B, purely because of where documents live, so shard A’s weak matches crowd out shard B’s strong ones. **Engineering implication.** Elasticsearch’s default `query_then_fetch` uses local statistics. `dfs_query_then_fetch` adds a preliminary round trip that collects term statistics from all shards and scores with global values, at the cost of extra latency [Elastic Docs: search_type]. Prefer random document routing so local statistics converge; use DFS for small or skewed indexes and in relevance tests.

Routing is the other side of the trade-off. By default Elasticsearch places a document on a shard by hashing its ID. A custom routing key (tenant ID, seller ID) places all of one tenant’s documents on one shard, so a tenant-scoped query hits one shard instead of S, removing fan-out at the cost of skew and the IDF effect above.

Real-world industry applications

INDUSTRY EXAMPLE

Twitter Earlybird. Twitter’s real-time search engine kept recent tweets in in-memory inverted indexes partitioned by document across many Earlybird servers, with a front-end (Blender) scattering each query and merging results. Indexing used a single writer and many concurrent readers per index, coordinated with memory barriers rather than locks, so that tweets became searchable within about 10 seconds of creation while queries averaged about 50 ms at billions of queries per day (2012 figures) [Busch et al. 2012]. *Evidence: Primary* — [Busch et al. 2012]

16.8 Elasticsearch and OpenSearch architecture

Elasticsearch wraps Lucene in a distributed system: a JSON-over-HTTP API, cluster membership, shard placement, replication and recovery. OpenSearch is a fork of Elasticsearch 7.10.2 created in 2021 after Elastic changed its license; the two share the architecture described here, though they have diverged in features since [OpenSearch Docs].

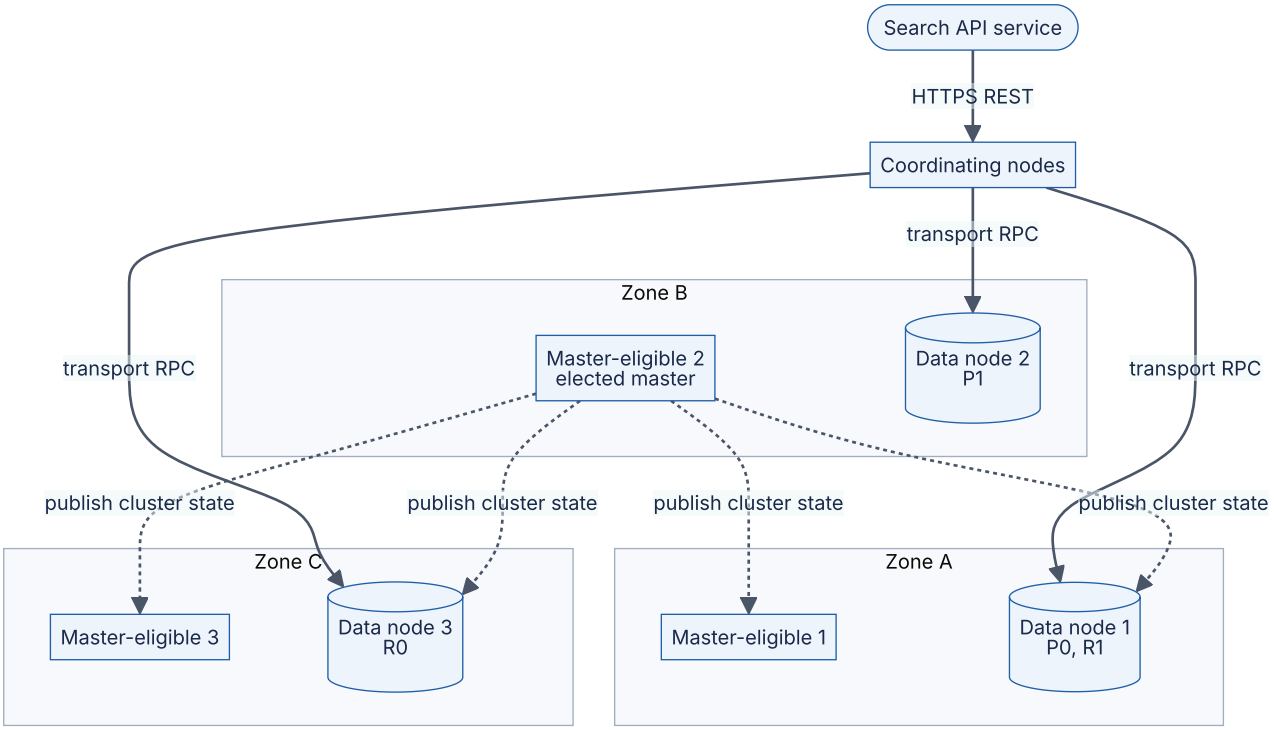
16.8.1 Cluster, nodes and roles

DEFINITION

Definition — Elasticsearch cluster. A set of nodes that share one cluster state (index definitions, mappings and the location of every shard copy). An **index** is a logical collection of documents divided into **primary shards**; each primary has zero or more **replica shards**; every shard is one Lucene index.

Nodes take one or more **roles** [Elastic Docs: node roles]: **master-eligible** nodes elect one elected master that owns and publishes the cluster state (a quorum-based protocol since 7.0, so three master-eligible nodes tolerate the loss of one); **data** nodes hold shards and execute queries and indexing (subdivided into hot, warm, cold and frozen tiers for time-based data); **ingest** nodes run preprocessing pipelines; and **coordinating-only** nodes route requests and merge results.

Figure 16.8 — A three-zone Elasticsearch cluster with two primaries and one replica each.



Reading the diagram. Clients call coordinating nodes over HTTPS; coordinating nodes talk to data nodes over Elasticsearch's binary transport protocol (synchronous RPCs, solid arrows). The index has two primary shards, P0 and P1, and one replica of each, R0 and R1; the allocator never places a primary and its replica on the same node and, with allocation awareness configured, not in the same zone. The elected master publishes cluster-state updates asynchronously (dotted arrows) to every node; it does not sit on the data path. Storage boundaries are the data nodes' local disks. Failure points: losing data

node 2 promotes R1 to primary and starts rebuilding a replica elsewhere; losing one master-eligible node leaves a quorum of two; losing a whole zone still leaves one copy of each shard. Scaling: add data nodes (shards rebalance automatically), add replicas for query throughput, and add coordinating nodes for heavy aggregations.

16.8.2 Shards, routing and sizing

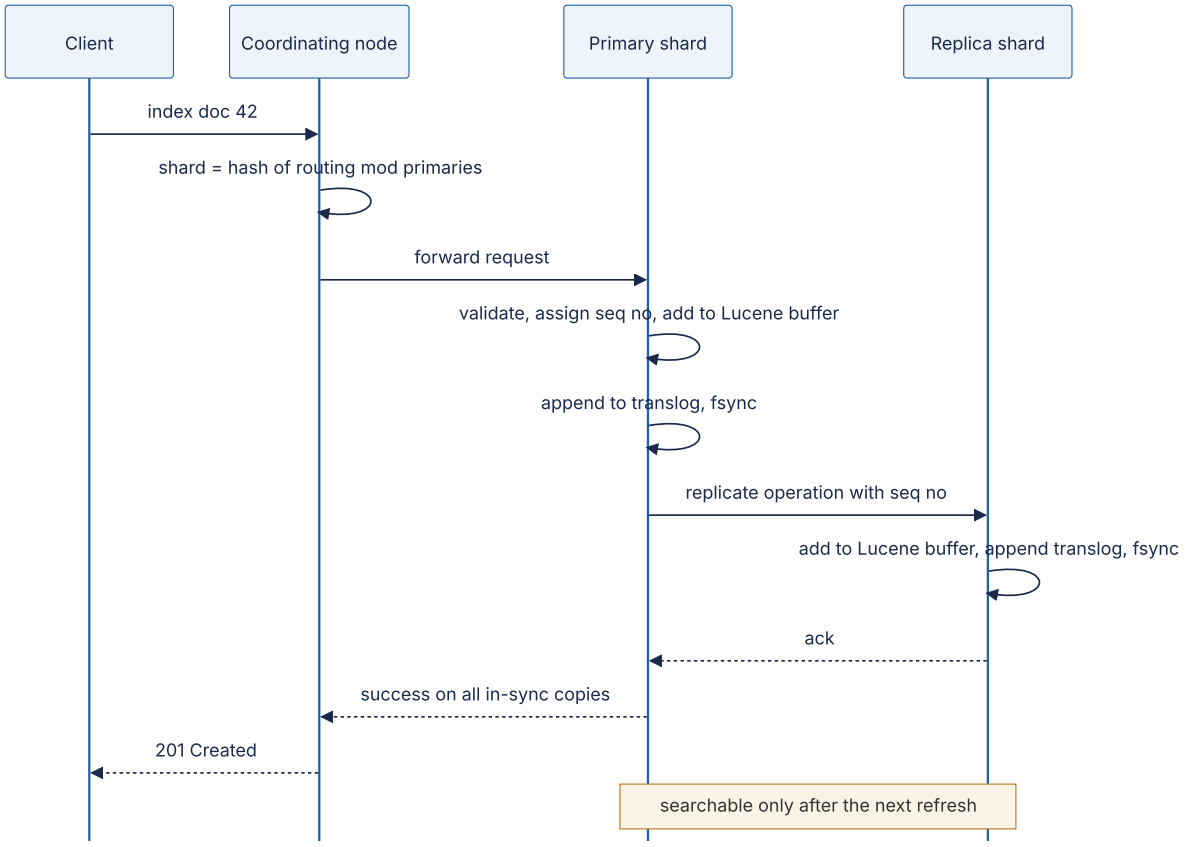
A document goes to shard $\text{hash}(\textit{routing}) \bmod N_{\text{primaries}}$, where routing defaults to the document ID and the hash is Murmur3 [Elastic Docs: _routing]. Because the formula depends on the number of primaries, that number is fixed when the index is created; changing it requires the split or shrink APIs (by integer factors) or a reindex. Replica count can be changed at any time. Since Elasticsearch 7.0 a new index has one primary and one replica by default.

Elastic's sizing guidance (as of 2025) is to aim for shards of roughly 10–50 GB and fewer than about 200 million documents each [Elastic Docs: size your shards]. Too-large shards recover slowly after node failure; too many small shards waste per-shard memory and inflate fan-out (§16.7.3). For the 440 GB catalog of §16.3.5 (about 220 GB of primary data), five primaries of roughly 44 GB fit the guidance. For logs and events, use **time-based indexes** with rollover and lifecycle policies, so old indexes can be force-merged, moved to cheaper tiers and deleted whole.

16.8.3 The write path: primary–backup replication and the translog

Elasticsearch uses primary–backup replication, modeled on Microsoft's PacificA protocol [Elastic Docs: data replication model].

Figure 16.9 — Indexing one document with one replica.



Reading the diagram. The coordinating node computes the shard and forwards the request to the node holding that shard's primary. The primary validates the document, assigns a sequence number (used to bring replicas back in sync after failures), indexes it into Lucene's in-memory buffer and appends it to the **translog**, a per-shard write-ahead log (§13.5) that by default is fsynced before the request is acknowledged (`index.translog.durability: request`). The primary then forwards the operation to all **in-sync** replicas in parallel and acknowledges only when they have all responded. Every arrow is synchronous; the disk boundary is the translog fsync on each copy. Failure points: a replica that fails to respond is reported to the master and removed from the in-sync set rather than blocking writes; a primary failure promotes an in-sync replica. The acknowledged document is durable but not yet visible to search; visibility comes at the next refresh. Indexing throughput scales with primaries, not replicas.

Table 16.9 — Refresh, flush and merge in Elasticsearch.

Operation	What it does	Default trigger	Makes data	Cost
Index (ack)	Buffer + translog fsync	Each request	Durable, not searchable	One fsync per request or bulk
Refresh	Write buffer as segment, reopen searcher	Every 1 s if the index was searched in the last 30 s	Searchable	New small segment; merge load
Flush	Lucene commit (fsync segments), trim translog	Translog size threshold	Durable without replay	Heavy fsync
Merge	Combine segments, drop deletes	Merge policy	Smaller, faster index	Background I/O and CPU

This split is what **near-real-time** means: a write is durable at acknowledgement (thanks to the translog) and searchable about one refresh interval later, without paying for a Lucene commit on every write [Elastic Docs: near real-time search]. A client that must read its own write can pass `refresh=wait_for`, which blocks the response until the next scheduled refresh.

16.8.4 Read path and memory

Queries rely on the operating system **page cache** for segment files (leave about half of RAM outside the JVM heap for it), the **shard request cache** for aggregations and the **node query cache** for filter bitsets. Elastic recommends a heap of at most half of RAM and below the roughly 31 GB compressed-pointer limit [Elastic Docs: heap size]; **doc values** serve sorting and aggregations from disk-based columns rather than heap.

Table 16.10 — Elasticsearch, OpenSearch and Solr compared (as of 2025).

	Elasticsearch	OpenSearch	Apache Solr
What it is	Lucene-based distributed engine by Elastic	Community fork of Elasticsearch 7.10.2 (Linux Foundation)	Apache Lucene project's search server
Coordination	Built-in quorum-based cluster coordination	Inherited from Elasticsearch	ZooKeeper (SolrCloud)
Replication	Document replication; stateless serverless offering	Document or segment replication	Replica types NRT, TLOG, PULL
Advantages	Feature velocity, ecosystem	Open-source license, AWS managed service	Mature, flexible configuration
Disadvantages	Licensing changes over time	Features lag or differ from Elasticsearch	Smaller operational ecosystem today
Choose when	You want the reference product and its tooling	You want Apache-licensed software or AWS OpenSearch Service	Existing Solr expertise or ZooKeeper-based operations

OpenSearch's **segment replication** copies segment files from primary to replicas instead of re-indexing each document on every replica, trading replica freshness for lower replica CPU [OpenSearch Docs: segment replication].

Failure scenarios

Table 16.11 — Distributed search and cluster failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Cluster status red	All copies of a primary lost	Cluster health API	Restore shard from snapshot; reindex from source	Replicas across zones; snapshots
Mapping explosion	Dynamic mapping of arbitrary JSON keys creates thousands of fields	Field count; cluster-state size	Reindex with strict mapping	`dynamic: strict`; field limits
Hot shard	Custom routing on a skewed key	Per-shard latency and CPU	Reroute; split shard	Random routing; routing partitions
Long GC pauses, node drops out	Heap too large or full	GC logs; node-left events	Reduce heap pressure; restart	Heap ≤ 50% RAM; circuit breakers
Stale results	Indexing pipeline lag	Freshness metric (§16.10.3)	Scale consumers	Alert on consumer lag

COMMON MISTAKE

Mistake — "Elasticsearch can be our primary database." It is durable per shard and replicated, but it has no multi-document transactions, near-real-time rather than immediate visibility, and mapping changes often require reindexing. Treat it as a derived, rebuildable view of a system of record (§16.10.2).

16.9 Typeahead and autocomplete

16.9.1 Requirements and the latency budget

DEFINITION

Definition — Autocomplete (typeahead). A service that, given the prefix a user has typed so far, returns a short ranked list (typically 5–10) of likely completions, usually popular past queries or entity names, on every keystroke or after a short pause.

First-principles explanation

Autocomplete is a friend finishing your sentences: useful only if the suggestion arrives before your next keystroke, 150–300 ms later. Nielsen's guideline is that about 0.1 s feels instantaneous [Nielsen 1993], so the whole round trip must fit in roughly 100 ms. That pushes design toward precomputation: answers for popular prefixes are computed offline and looked up, not searched.

Table 16.12 — End-to-end latency budget for one suggestion request.

Stage	Budget	Notes
Client event handling	≤ 5 ms	Plus an optional 30–50 ms debounce, outside the budget
Network round trip	30–50 ms	Reused HTTP/2 or HTTP/3 connection; nearby edge
Load balancer and gateway	2–5 ms	No per-request authentication call
Suggestion lookup	≤ 5 ms at p99	In-memory trie or FST; no disk
Serialization and render	~16 ms	One frame at 60 Hz
Total	**≈ 55–80 ms**	Leaves headroom under 100 ms

FORMULA

Autocomplete load and precomputed-index memory

$$\lambda_{ac} = \lambda_{search} \cdot c \cdot f \qquad M_{top\text{-}k} \approx P \cdot k \cdot b$$

Symbol	Meaning	Units
λ_{search}	Searches started per second	searches/s
c	Characters typed per search before selecting or submitting	characters
f	Fraction of keystrokes that send a request (after debounce and client caching)	dimensionless
λ_{ac}	Suggestion request rate	requests/s
P	Number of trie nodes (distinct prefixes) with stored suggestions	nodes
k	Suggestions stored per prefix	count
b	Bytes per stored suggestion reference	bytes
$M_{top\text{-}k}$	Memory for the precomputed lists	bytes

WORKED EXAMPLE

Worked example — sizing typeahead for the store. 50 million searches/day, peak-to-average ratio 2, $c = 8$, $f = 0.5$. The suggestion corpus is 10 million distinct past queries, giving an assumed $P = 6 \times 10^7$ trie nodes after prefix sharing; $k = 10$, $b = 4$ bytes (a query ID).

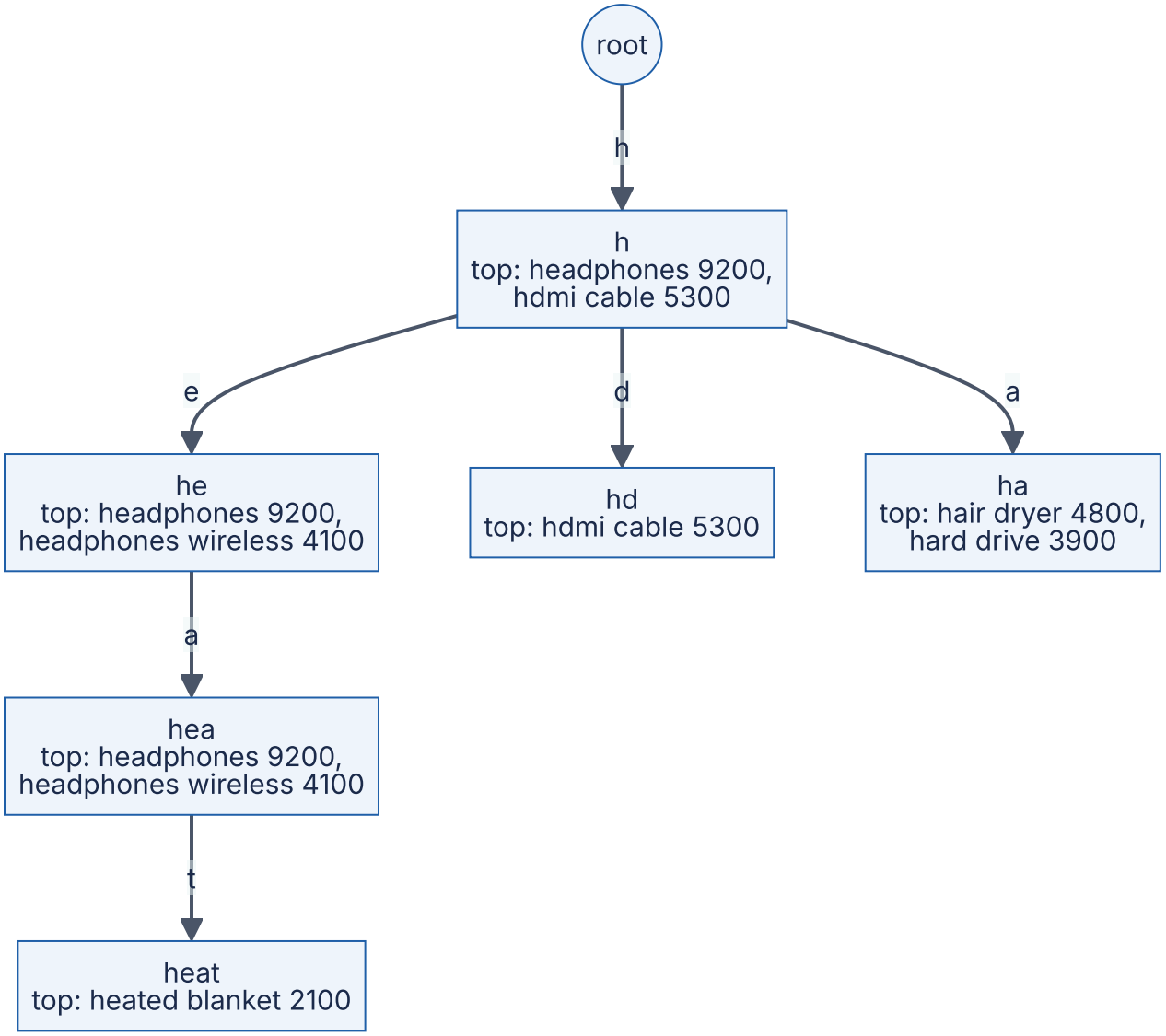
1. Average search rate: $5 \times 10^7 / 86,400 = 579$ searches/s; peak $\approx 1,157$/s.
2. Peak suggestion rate: $1,157 \times 8 \times 0.5 \approx 4,630$ requests/s, four times the search rate.
3. Memory for top-10 lists: $6 \times 10^7 \times 10 \times 4 = 2.4 \times 10^9$ bytes = 2.4 GB, plus the trie nodes themselves and the query strings (10 M × ~20 bytes = 200 MB).

Interpretation. Autocomplete usually generates more requests than search itself, but each is a memory lookup. **Engineering implication.** A few replicated in-memory nodes handle this load. Memory, not CPU, is the constraint, so cap precomputation at short prefixes (for example up to 10 characters), drop rare queries, and cache the responses for the most popular one- and two-character prefixes at the CDN or in the client.

16.9.2 Tries with precomputed top-*k*

A **trie** (§17.11) stores strings as paths from the root, one character per edge; all strings with a given prefix live under one node. Finding that node takes $O(\ell)$ steps for a prefix of length ℓ. The naive next step, walking the whole subtree to find the most popular completions, can touch millions of nodes for a one-letter prefix. The fix is to **store the top-*k* completions at every node**, computed bottom-up offline: a node's list is the best k among its own query (if one ends there) and its children's lists.

Figure 16.10 — A trie with top-2 completions stored at each node.



Reading the diagram. Each node is a prefix; edges are labeled with the next character. Each node stores its precomputed top-2 list with popularity counts from the query log. A lookup for “hea” follows three edges from the root and returns the list stored at that node with no further traversal. “heated blanket” lives under “heat” and appears in the lists of “he” and “hea” only when $k \geq 3$, because two headphone queries outrank it. The structure is read-only in memory; it is rebuilt offline and swapped in atomically. Failure points: an outdated build (stale trends) and memory exhaustion as the corpus grows. It scales by replication, and by partitioning on the first character or two of the prefix if one node cannot hold it.

```
        """Trie-based autocomplete with the top-k completions precomputed at every node.
        Lookup cost is O(len(prefix)) and independent of how many queries share the pre-
fix."""
        import heapq

        class Node:
            __slots__ = ("children", "count", "top")
            def __init__(self):
                self.children = {}      # char -> Node
                self.count = 0          # popularity if a query ends here, else 0
                self.top = []           # precomputed [(count, query)] best-first,
length <= k

        class Autocomplete:
            def __init__(self, k=3):
                self.root, self.k, self.nodes = Node(), k, 1

            def insert(self, query, count):
                node = self.root
                for ch in query:
                    if ch not in node.children:
                        node.children[ch] = Node()
                        self.nodes += 1
                    node = node.children[ch]
                node.count += count

            def build_top_k(self, node=None, prefix=""):
                """Post-order pass: a node's top-k = best k of (itself, children's top-k
lists)."""
                node = node or self.root
                candidates = [(node.count, prefix)] if node.count else []
                for ch, child in node.children.items():
                    self.build_top_k(child, prefix + ch)
                    candidates.extend(child.top)
                node.top = heapq.nlargest(self.k, candidates, key=lambda c: c[0])

            def suggest(self, prefix):
                node = self.root
                for ch in prefix.lower():
                    node = node.children.get(ch)
                    if node is None:
                        return []
                return [q for _, q in node.top]

            def suggest_by_dfs(self, prefix):
                """Without precomputation: walk the whole subtree. Returns (suggestions,
nodes visited)."""
                node = self.root
                for ch in prefix:
                    node = node.children.get(ch)
                    if node is None:
                        return [], 0
                found, stack, visited = [], [(node, prefix)], 0
                while stack:
                    n, p = stack.pop()
                    visited += 1
```

```
                    if n.count:
                        found.append((n.count, p))
                    stack.extend((c, p + ch) for ch, c in n.children.items())
                best = heapq.nlargest(self.k, found, key=lambda c: c[0])
                return [q for _, q in best], visited

        QUERY_LOG = {   # query -> times searched in the last 7 days (illustrative)
            "headphones": 9200, "headphones wireless": 4100, "headphone stand": 800,
            "headphones for kids": 1500, "head torch": 650, "heated blanket": 2100,
            "hdmi cable": 5300, "hard drive": 3900, "hair dryer": 4800, "hammock": 300,
        }

        if __name__ == "__main__":
            ac = Autocomplete(k=3)
            for q, c in QUERY_LOG.items():
                ac.insert(q, c)
            ac.build_top_k()
            for prefix in ("h", "he", "hea", "head", "headphones ", "x"):
                print(f"{prefix!r:>15} -> {ac.suggest(prefix)}")
            dfs, visited = ac.suggest_by_dfs("h")
            assert dfs == ac.suggest("h")
            print(f"trie nodes={ac.nodes}; DFS for 'h' visits {visited} nodes, precom-
puted lookup reads 1 node")
```

Output:

```
            'h' -> ['headphones', 'hdmi cable', 'hair dryer']
           'he' -> ['headphones', 'headphones wireless', 'heated blanket']
          'hea' -> ['headphones', 'headphones wireless', 'heated blanket']
         'head' -> ['headphones', 'headphones wireless', 'headphones for kids']
  'headphones ' -> ['headphones wireless', 'headphones for kids']
            'x' -> []
trie nodes=82; DFS for 'h' visits 81 nodes, precomputed lookup reads 1 node
```

What to notice: the build pass is one post-order traversal costing $O(\text{nodes} \cdot k \log k)$, done offline; the online lookup reads ℓ nodes regardless of corpus size; and a ten-query log already needs 82 nodes, which is why production systems compress the trie.

16.9.3 Finite-state transducers in Lucene

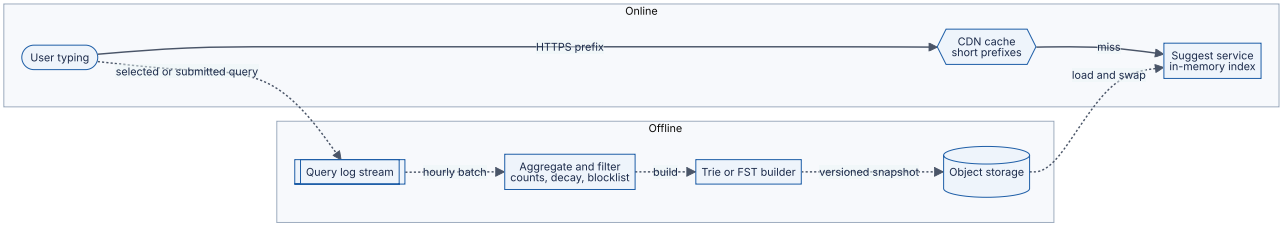
A trie shares common *prefixes* only. A **finite-state transducer (FST)** is a minimal automaton that shares both prefixes and suffixes (“-phones”, “-ing”) and attaches an output, such as a weight or an ordinal, to each accepted string. Minimal acyclic automata can be built incrementally from sorted input in linear time [Daciuk et al. 2000]. Lucene uses FSTs for its term-dictionary index, and Elasticsearch’s **completion suggester** (the `completion` field type) stores weighted suggestions in a per-segment FST held in memory, giving top-k lookups in microseconds with optional fuzzy matching and context filters [Elastic Docs: completion suggester]. Because the FST lives inside segments, suggestions update through normal indexing and refresh.

Table 16.13 — Autocomplete approaches.

Approach	How it works	Advantages	Disadvantages	Choose when
Trie with top-*k* per node	Offline build from query logs	Fastest; full control of ranking	Custom service; rebuilds	Very high traffic; query-log suggestions
FST completion suggester	Weighted FST per segment	Built in; compact; fuzzy	Prefix-only; weights static per document	Entity names in Elasticsearch
Edge n-gram field	Index every prefix as a term	Matches inner words, full query features	Index 3–10× larger; slower	Suggestions need filters and scoring
`search_as_you_type` field	Managed shingles and edge n-grams	Easy, infix-friendly	Same costs as edge n-grams	Small to medium catalogs

16.9.4 Serving architecture

Figure 16.11 — Autocomplete: offline build and online serving.



Reading the diagram. The offline subgraph runs asynchronously (dotted arrows): query logs are aggregated with time decay so trends surface, filtered against a blocklist of offensive or unsafe completions, built into a trie or FST, and published as a versioned snapshot to object storage. Online, the user's prefix goes over HTTPS to a CDN that caches responses for popular short prefixes; misses reach stateless suggest-service replicas that hold the whole structure in memory and swap in new snapshots atomically. The user's eventual query flows back into the log, closing the loop. Storage boundaries are the log, object storage and each replica's memory. Failure points: a bad snapshot (roll back to the previous version), a poisoned log (rate-limit and filter per user), and stale caches during breaking news (short TTLs). Scaling: add replicas; partition by prefix if memory runs out; personalize by blending global suggestions with the user's own recent queries at request time.

16.10 Near-real-time indexing

16.10.1 The refresh interval trade-off

DEFINITION

Definition — Near-real-time (NRT) search. A search system in which newly written documents become searchable after a short, bounded delay (typically one second to a minute) rather than immediately, because making them visible requires creating a new segment and reopening the searcher.

Each refresh creates a segment and a new searcher; refreshing more often gives fresher results but more tiny segments, more merging (§16.6.3) and lower indexing throughput.

Table 16.14 — Choosing a refresh interval.

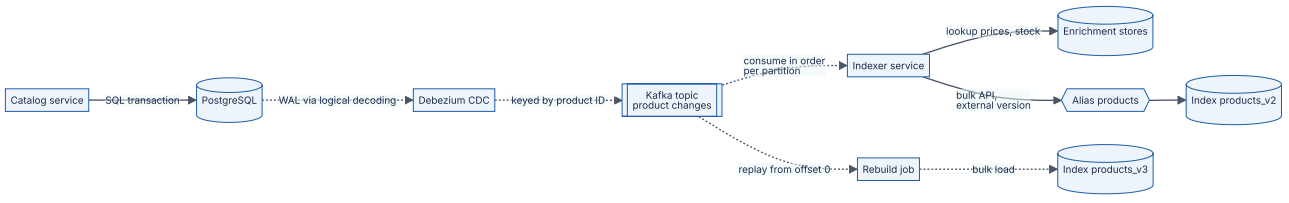
Setting	Freshness	Indexing throughput	Merge load	Typical use
1 s (default)	~1 s	Lowest	Highest ($W \approx 4.4$ in §16.6.3)	Product, user-facing content
30 s	~30 s	Higher	Lower ($W \approx 2.9$)	Logs, metrics, analytics
`-1` (disabled) during bulk load, replicas 0	None until re-enabled	Highest	Lowest	Initial load or full reindex
`refresh=wait_for` on a request	Caller waits for next refresh	Unchanged	Unchanged	Read-your-writes for one client

Never call an explicit refresh after every write: it turns each document into its own segment and can collapse indexing throughput.

16.10.2 The indexing pipeline: CDC and Kafka

The source of truth is usually a relational database. Writing to the database *and* to the search index from application code (**dual writes**) loses updates whenever one write succeeds and the other fails, and can apply concurrent updates in different orders to the two stores [Kleppmann 2017, ch. 11]. The robust pattern is to derive the index from the database's own change log.

Figure 16.12 — Near-real-time indexing pipeline with CDC and Kafka.



Reading the diagram. The catalog service writes only to PostgreSQL, synchronously. Debezium reads the write-ahead log through logical decoding and publishes each committed change to Kafka asynchronously, keyed by product ID so that all changes to one product land in one partition, in order. The indexer consumes partitions, enriches each event into a complete search document (prices, stock, review counts from other stores), and writes in bulk through an **alias**, a stable name pointing at the current physical index. Each write carries an external version (for example the source log sequence number), so a replayed or reordered older event is rejected and the pipeline stays idempotent under at-least-once delivery (§22.15). Deletes arrive as delete events and become delete requests. Storage boundaries: the database, the Kafka log, the enrichment stores and the index. Failure points: indexer lag (search becomes stale, never wrong), poison events (send to a dead-letter topic), and Kafka retention shorter than a rebuild. Scaling: more Kafka partitions and indexer consumers. The lower path shows a full **rebuild**: a job replays the topic (or a snapshot plus the topic) into `products_v3` with refresh disabled; when it catches up, the alias is switched atomically from v2 to v3, a zero-downtime reindex used for analyzer or mapping changes.

16.10.3 Freshness budget

FORMULA

End-to-end freshness latency

$$T_{\text{fresh}} \approx T_{\text{cdc}} + T_{\text{queue}} + T_{\text{batch}} + T_{\text{index}} + T_{\text{refresh}}$$

Symbol	Meaning	Units
T_{cdc}	Commit to event published in Kafka	s
T_{queue}	Time the event waits for the indexer (consumer lag)	s
T_{batch}	Wait for a bulk request to fill or its timer to fire	s
T_{index}	Bulk request execution on primary and replicas	s
T_{refresh}	Wait for the next refresh: uniform in $[0, I]$, mean $I/2$	s
T_{fresh}	Commit in source database to searchable	s

WORKED EXAMPLE

Worked example — how fresh is a price change? T_{cdc} = 0.2 s, T_{queue} = 0.3 s at normal load, bulk requests flush every 1 s (mean wait 0.5 s, worst 1 s), T_{index} = 0.1 s, refresh interval I = 1 s (mean 0.5 s, worst 1 s).

1. Mean: 0.2 + 0.3 + 0.5 + 0.1 + 0.5 = 1.6 s.
2. Worst case without backlog: 0.2 + 0.3 + 1 + 0.1 + 1 = 2.6 s.
3. With I = 30 s: mean 1.1 + 15 = 16.1 s, worst 31.6 s.
4. During a backlog of 600,000 events drained at 5,000 events/s: T_{queue} grows by 120 s.

Interpretation. At default settings the pipeline, not Lucene, dominates freshness, and backlog dominates everything. **Engineering implication.** Publish freshness as a service-level objective (for example 99% of changes searchable within 5 s), measure it end to end by writing canary documents and timing when they become searchable, and alert on consumer lag.

COMMON MISTAKE

Mistake — "Kafka gives us exactly-once, so the index is always consistent." Consumers see events at least once and may restart from older offsets; enrichment lookups may read newer data than the event. Make every index write idempotent with external versions, design documents to be fully rebuilt from current source state, and reconcile periodically by comparing counts or checksums between source and index.

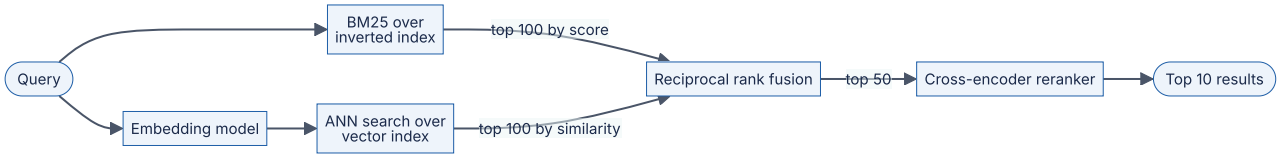
16.11 Beyond lexical matching: a bridge to semantic and hybrid search

BM25 matches *words*. It cannot know that "ANC earbuds" and "noise cancelling in-ear headphones" describe the same thing unless synonyms are configured by hand, and the listing in §16.5.4 ranked earbuds above headphones because it understands nothing. **Semantic (dense) retrieval** encodes queries and documents as embedding vectors with a neural model and retrieves nearest neighbors by cosine similarity (§4.9.2), using approximate nearest neighbor (ANN) indexes such as HNSW. It captures meaning but can miss exact identifiers (part numbers, error codes) that BM25 matches perfectly.

Hybrid retrieval runs both and fuses the lists. Because BM25 scores and cosine similarities live on different scales (§16.5 misconception), fusion usually uses ranks: **reciprocal rank fusion** adds $1/(60 + \text{rank})$ from each list [Cormack et al. 2009]. A cross-encoder reranker may then reorder the top few dozen. Lucene, Elasticsearch and OpenSearch

now include vector fields and ANN search alongside inverted indexes, so one cluster can serve both (as of 2025).

Figure 16.13 — Hybrid retrieval (preview of Chapters 74–75).



Reading the diagram. The query takes two synchronous paths in parallel: lexical retrieval over the inverted index of this chapter, and an embedding model followed by ANN search over a vector index. Each returns its top 100 candidates; rank fusion merges them without comparing raw scores, and an optional reranker spends expensive model inference on only the top 50. The two indexes are separate storage boundaries that must be kept in step by the same indexing pipeline (§16.10.2), with embeddings computed at indexing time. Failure points: the embedding model adds latency and a GPU or CPU dependency on the query path; if it fails, falling back to BM25 alone keeps search available. Chapter 74 covers vector indexes and Chapter 75 covers hybrid retrieval, fusion and reranking in depth.

16.12 Summary and revision notes

SUMMARY

- An **inverted index** maps each term to a sorted postings list of docIDs (with tf and positions). AND is intersection, OR is union, phrases check positions; evaluate the rarest term first. **Skip pointers** (`advance(target)`) let a short list drive a long one; spacing near $\sqrt{2L/m}$ beats the $\sqrt{L}$ heuristic for unequal lists.
- **Compression:** delta-encode docIDs, then VByte (7 bits per byte) or block bit-packing (Lucene: blocks of 128). Measured ≈ 1.4 bytes per docID posting. Index size $\approx Nu(b_d + b_f) + NLb_p + Mb_t$; positions often cost more than docIDs; budget again as much for `_source` , doc values and norms, then multiply by replicas.
- **Analysis** (char filters → tokenizer → token filters) must match at index and query time. Stemming raises recall and risks precision; lemmatization is more precise and costlier; keep stop words but preserve positions; n-grams enable substring and prefix search at large size cost; apply synonyms at query time; analyze each language in its own field.
- **BM25:** IDF = $\ln(1 + (N - n + 0.5)/(n + 0.5))$; tf saturates at $k_1 + 1$ ($k_1 = 1.2$); length normalization via $b = 0.75$. Scores are comparable only within one query. Field boosts or BM25F add structure; WAND and Block-Max WAND skip blocks that cannot reach the top k.
- **Segments** are immutable; refresh makes them searchable, commit makes them durable; deletes are live-docs tombstones; updates are delete + add; tiered merging rewrites each byte about $1 + \log_m(S_{\max}/S_{\text{flush}})$ times.
- **Distributed search** uses document partitioning, replicas for availability and throughput, scatter–gather ($P = p^S$, so use few shards and tail-tolerant techniques), query-then-fetch, bounded pagination ($S(from + size)$), and local IDF unless `dfs_query_then_fetch` is used.
- **Elasticsearch/OpenSearch:** master-eligible, data, ingest and coordinating roles; shard = hash(routing) mod primaries; shards of about 10–50 GB; primary–backup replication with a fsynced translog; refresh every 1 s gives near-real-time visibility.
- **Autocomplete** must fit ≈ 100 ms end to end: precompute top-k per prefix in a trie or FST, serve from memory, cache short prefixes, rebuild offline from query logs with decay and blocklists.
- **NRT pipeline:** CDC → Kafka (keyed by document ID) → indexer with external versions → alias; freshness ≈ CDC + lag + batch + index + refresh; reindex by replay into a new index and alias swap.

16.13 Knowledge checkpoint

1. Why can a B-tree index not answer `LIKE '%noise%'` efficiently, and what does an inverted index store instead?
2. Encode the postings list 3, 130, 131 with delta + VByte. How many bytes?
3. When does a skip pointer help an intersection, and what spacing suits a 1,000-entry list driving a 1,000,000-entry list?
4. Give one risk each of stemming, stop-word removal and index-time synonyms.
5. What happens if the query analyzer applies a stemmer that the index analyzer did not?
6. Compute the BM25 IDF of a term in 1 of 8 documents.
7. What value does the BM25 term-frequency factor approach as tf grows, and why does that matter?
8. Why is an update in Lucene more expensive than an insert?
9. A cluster refreshes every 1 s with 1 MB flushes and a 5 GB segment cap, merge factor 10. Estimate write amplification.
10. A query fans out to 50 shards whose p99 is 30 ms. What fraction of queries wait longer than 30 ms?
11. Why does Elasticsearch limit `from + size` to 10,000 by default?
12. When does local IDF distort ranking, and what is the fix?
13. Why is the number of primary shards fixed at index creation?
14. Why is autocomplete precomputed rather than searched per keystroke?
15. Name two ways the indexing pipeline stays correct under at-least-once delivery.

Answers

1. A B-tree orders whole keys, so it helps only with equality or prefix predicates; an infix pattern forces a full scan. An inverted index stores term → sorted docIDs, so each word is a direct lookup.
2. Gaps 3, 127, 1 → each fits in 7 bits → 3 bytes (`00000011 01111111 00000001`), versus 12 bytes raw.
3. When one list is much shorter, so many entries of the long list can be jumped. $\sqrt{2 \times 10^6 / 10^3} \approx 45$.
4. Stemming: conflates unrelated words (“universe”/“university”). Stop words: breaks queries like “The Who”. Index-time synonyms: changes need a full reindex and inflate the index.
5. Query terms are stems that were never indexed in that form, so matches silently disappear.
6. $\ln(1 + 7.5/1.5) = \ln 6 = 1.79$.
7. $k_1 + 1$ (2.2 at default). It caps the benefit of repetition, defeating keyword stuffing.
8. It is a delete (tombstone bit in the old segment) plus a full re-index of the new version into a new segment, and the dead copy occupies space until merged.

9. $1 + \log_{10}(5,000/1) = 1 + 3.7 = 4.7$.
10. $1 - 0.99^{50} = 1 - 0.605 = 39.5\%$.
11. Every shard must return and the coordinator must merge $S \times (from + size)$ entries; deep pages are expensive and can exhaust memory. Use `search_after` instead.
12. When a term's documents are unevenly spread across shards (custom routing, small indexes); use `dfs_query_then_fetch` or random routing.
13. Routing computes hash(routing) mod number of primaries; changing it would move most documents. Split/shrink by integer factors or reindex.
14. The budget is ≈ 100 ms end to end, request rate exceeds search rate, and answers for a prefix change slowly, so a memory lookup of a precomputed list is cheaper and faster.
15. Key events by document ID for per-document ordering, and write with external versions so stale or replayed events are rejected; additionally reconcile periodically.

16.14 Interview questions

Basic

Q1. Explain how a search engine finds documents containing two words.

Answer. Text is analyzed into terms, and an inverted index maps each term to a sorted postings list of docIDs. The query is analyzed the same way, both postings lists are fetched, and they are intersected by walking them in docID order, starting from the shorter list and using skip pointers to jump through the longer one. Matching docIDs are scored (BM25) and the top k are returned, with stored fields fetched only for those.

Q2. What is BM25 and how does it improve on TF-IDF?

Answer. BM25 sums, over query terms, IDF times a term-frequency factor $f(k_1 + 1)/(f + k_1(1 - b + b|d|/\text{avgdl}))$. Unlike raw TF-IDF, term frequency saturates (repetition has diminishing returns, bounded by $k_1 + 1$), and document length is normalized against the average with strength b. Defaults $k_1 = 1.2$, $b = 0.75$ work well and are tuned per field against a relevance set.

Intermediate

Q3. Search results take up to 30 seconds to reflect product edits. Where do you look?

Answer. Decompose freshness into CDC lag, Kafka consumer lag, bulk batching, indexing time and refresh interval. Check whether the index has `refresh_interval: 30s` (common copy-paste from logging setups) or is search-idle; check consumer lag and whether a hot partition or slow enrichment lookups throttle the indexer; check for bulk rejections and

retries. Fix the dominant term, add an end-to-end canary freshness metric and alert on lag.

Q4. How many shards would you give a 300 GB product index serving 3,000 queries per second?

Answer. Size shards by data first: with 10–50 GB guidance, 6–10 primaries (say 8 of about 37 GB). Then size copies for throughput: measure per-shard query cost, e.g. 5 ms CPU; $3,000 \times 8 \times 5\ \text{ms} = 120$ core-seconds/s, so 240 cores at 50% utilization, about 15 sixteen-core nodes, so 8 primaries with enough replicas (for example 2 replicas, 24 copies) spread across three zones. Avoid many small shards: fan-out raises tail latency as p^S. Validate with a load test.

Advanced

Q5. Design search for a multi-tenant SaaS with 50,000 tenants of very different sizes.

Answer. Shared indexes with tenant ID filters are efficient for small tenants; route by tenant ID so a tenant's query hits one shard, using routing partitions (`routing_partition_size`) so large tenants spread over several shards. Give the largest tenants dedicated indexes to isolate load and allow custom analyzers. Watch the local-IDF problem: per-tenant statistics differ, so test relevance per tenant or accept local statistics. Enforce tenant filters in the search API, never in the client; cap per-tenant request rates; and use aliases with filters per tenant for migrations between shared and dedicated indexes.

Q6. Your 200-shard cluster's p99 latency is 400 ms while median shard latency is 10 ms. Diagnose and fix.

Answer. Scatter–gather means each query waits for the slowest of 200 shards: $0.99^{200} = 13\%$ of queries are fully fast at the shard p99. Look for correlated stalls (GC pauses, merges, hot nodes, uneven shard placement) using per-node latency. Fixes in order: reduce shard count by merging small indexes (shrink API, larger shards); enable or tune adaptive replica selection and add replicas so slow copies are avoided; hedge requests after the shard p95; set per-shard timeouts with partial results; and move heavy aggregations to separate coordinating nodes.

16.15 Practical exercises

1. **Extend the engine.** Add Boolean AND with skip-pointer intersection to the §16.5.4 listing, then add BM25F-style scoring (combine weighted tf across fields before saturation) and compare the ranking for "wireless noise cancelling headphones".

2. **Measure compression.** Change the Zipf exponent in the §16.3.4 listing to 0.8 and 1.2 and report bytes per posting; then implement frame-of-reference bit packing for blocks of 128 gaps and compare size and decoding time with VByte.
3. **Analyzer laboratory.** Run Elasticsearch or OpenSearch locally and use the `_analyze` API to compare the standard, English and a custom analyzer on ten product titles containing hyphens, accents, model numbers and plurals.
4. **Refresh experiment.** Bulk-index one million synthetic documents with refresh intervals of 1 s, 30 s and disabled; record indexing throughput, segment counts over time and merge statistics, and compare with the write-amplification formula.
5. **Autocomplete from logs.** Feed a real or synthetic query log into the §16.9.2 trie with exponential time decay on counts and a blocklist; measure memory per million queries and lookup latency.

16.16 Design challenges

1. **Marketplace product search.** 200 million listings, 20,000 searches/s peak, prices and stock updated 5,000 times/s, freshness SLO of 5 s for stock. Decide what goes in the text index versus a fast-changing side store, shard and replica counts, the CDC pipeline and how stock filters are applied without reindexing documents for every change.
2. **Log search.** 2 TB/day of application logs, 30-day retention, most queries over the last hour. Design time-based indexes, rollover size, hot/warm tiers, refresh interval and force-merge policy, and compute the storage footprint with one replica.
3. **Global autocomplete.** 300 million users in 40 languages, suggestions within 100 ms p99 worldwide, trending queries reflected within 15 minutes, unsafe suggestions filtered. Design the build pipeline, per-language tries or FSTs, edge caching and the rollback mechanism.

16.17 References

1. Manning, C. D., Raghavan, P., Schütze, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008. https://nlp.stanford.edu/IR-book/
2. Robertson, S., Zaragoza, H. *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval 3(4), 2009.
3. Zobel, J., Moffat, A. *Inverted Files for Text Search Engines*. ACM Computing Surveys 38(2), 2006.
4. Brin, S., Page, L. *The Anatomy of a Large-Scale Hypertextual Web Search Engine*. Computer Networks and ISDN Systems 30(1–7), 1998.
5. Spärck Jones, K. *A Statistical Interpretation of Term Specificity and Its Application in Retrieval*. Journal of Documentation 28(1), 1972.

6. Robertson, S. E. *The Probability Ranking Principle in IR*. Journal of Documentation 33(4), 1977.
7. Robertson, S., Zaragoza, H., Taylor, M. *Simple BM25 Extension to Multiple Weighted Fields*. Proceedings of CIKM, 2004.
8. Porter, M. F. *An Algorithm for Suffix Stripping*. Program 14(3), 1980.
9. Harman, D. *How Effective Is Suffixing?* Journal of the American Society for Information Science 42(1), 1991.
10. Ding, S., Suel, T. *Faster Top-k Document Retrieval Using Block-Max Indexes*. Proceedings of SIGIR, 2011.
11. Lemire, D., Boytsov, L. *Decoding Billions of Integers per Second through Vectorization*. Software: Practice and Experience 45(1), 2015.
12. Daciuk, J., Mihov, S., Watson, B. W., Watson, R. E. *Incremental Construction of Minimal Acyclic Finite-State Automata*. Computational Linguistics 26(1), 2000.
13. Dean, J., Barroso, L. A. *The Tail at Scale*. Communications of the ACM 56(2), 2013.
14. Dean, J. *Challenges in Building Large-Scale Information Retrieval Systems*. Keynote, ACM WSDM, 2009.
15. Suresh, L., Canini, M., Schmid, S., Feldmann, A. *C3: Cutting Tail Latency in Cloud Data Stores via Adaptive Replica Selection*. USENIX NSDI, 2015.
16. Busch, M., Gade, K., Larson, B., Lok, P., Luckenbill, S., Lin, J. *Earlybird: Real-Time Search at Twitter*. IEEE ICDE, 2012.
17. Clem, T. *The Technology behind GitHub's New Code Search*. GitHub Blog, 2023. https://github.blog/engineering/architecture-optimization/the-technology-behind-githubs-new-code-search/
18. Cormack, G. V., Clarke, C. L. A., Büttcher, S. *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. Proceedings of SIGIR, 2009.
19. Kleppmann, M. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
20. Nielsen, J. *Usability Engineering*. Academic Press, 1993 (response-time limits; also "Response Times: The 3 Important Limits", Nielsen Norman Group).
21. Apache Lucene. *Lucene documentation: postings format, DataOutput.writeVInt, TieredMergePolicy, BM25Similarity, MIGRATE notes for 8.0*. https://lucene.apache.org/core/
22. Elastic. *Elasticsearch Reference: near real-time search; data replication model; search_type; track_total_hits; _routing; node roles; similarity; completion suggester; heap size*. https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html
23. Elastic. *Size your shards*. Elastic documentation. https://www.elastic.co/docs/deploy-manage/production-guidance/optimize-performance/size-shards
24. OpenSearch Project. *OpenSearch documentation, including segment replication*. https://opensearch.org/docs/latest/
25. Wikimedia. *Extension:CirrusSearch*. MediaWiki. https://www.mediawiki.org/wiki/Extension:CirrusSearch

16.18 Open verification items

- Lucene postings layout (blocks of 128 with frame-of-reference packing, multi-level skip data) is stated for Lucene 5–9; confirm against Lucene 10’s current postings format.
- `TieredMergePolicy` defaults (about 10 segments per tier, merge 10 at once, 5 GB maximum merged segment) — confirm current Lucene values.
- Lucene 8 removal of $(k_1 + 1)$ (LUCENE-8563) and Elasticsearch’s compensating 2.2 boost in `explain` — confirmed via issue tracker and forum summaries only.
- Manning et al. Table 5.6 figures (400 MB raw, 116 MB VByte for RCV1) quoted from memory.
- Earlybird “within 10 seconds” and “50 ms average” confirmed via search snippets, not full text.
- GitHub 25 TB / 115 TB figures from search summaries of the blog.
- Elastic Cloud Serverless “stateless” description and Elasticsearch BM25 switch version (5.0) — check docs.
- Workload numbers in worked examples (catalog size, query rates, $P = 6 \times 10^7$ trie nodes) are Illustrative.

Chapter 17 — Data Structures and Algorithms for Scale

> **Level:** 2–3 · **Track:** A (Traditional) · **Estimated reading time:** 6 h

A hash table, a sorted array and a B-tree will carry you a long way. Then the numbers grow. You need to know whether one of 3 billion usernames is taken without reading a disk. You need to count the distinct visitors to a site that gets 50 billion page views a day. You need the 100 most-shared links of the last hour in a stream of a million events per second. You need every driver within a kilometer of a rider, out of a million moving cars. And you need IDs for all of these things, generated on a thousand machines at once, without the machines talking to each other.

Exact answers cost memory, disk reads or coordination in proportion to the data. The structures in this chapter accept a small, *bounded* and *quantifiable* error, or a little randomness, in exchange for kilobytes of memory, constant-time operations and no coordination. A Bloom filter answers “have I seen this?” in about 10 bits per item at a 1% false-positive rate. A HyperLogLog counts a billion distinct items in 12 KB with 0.81% standard error. A Snowflake ID lets a thousand machines mint 4 billion IDs per second without talking to one another.

For each structure you get the idea, the mechanism, the sizing formula, a worked example and runnable Python that checks the formula against a measurement.

17.1 Learning objectives

After this chapter you should be able to:

- State the properties a non-cryptographic hash function must have, measure uniformity with a chi-square test, and explain how hash flooding works and how keyed hashes such as SipHash stop it.
- Size a Bloom filter for a target false-positive rate (bits per element and number of hash functions), predict its false-positive rate, and verify the prediction by measurement.
- Choose between Bloom, counting Bloom and cuckoo filters when deletions are needed.
- Explain how HyperLogLog estimates cardinality from leading zeros, compute its memory and standard error, and merge sketches across servers.

- Size a Count-Min sketch from an error target ε and confidence δ, and build a heavy-hitters (top-k) pipeline with a heap or the Space-Saving algorithm.
- Explain why skip lists give expected $O(\log n)$ search and why Redis uses them for sorted sets.
- Build a Merkle tree, produce and verify an inclusion proof, and explain how replicas use Merkle trees for anti-entropy.
- Encode a geohash, choose a precision for a search radius, and compare geohash, quadtrees, S2 and H3 for proximity search.
- Compare UUIDv4, UUIDv7, Snowflake, ULID and ticket servers, and compute the capacity and lifetime of a Snowflake bit layout.
- Use a trie for prefix lookup and reservoir sampling for uniform samples from a stream of unknown length.

17.2 Prerequisites

- Powers of two and logarithms — §4.3, §4.4 (every sizing formula here is a logarithm or an exponent).
- Probability essentials: complement, independence, expected value — §4.6 (false-positive rates and error bounds are probabilities).
- Cryptographic hash functions and the birthday bound — §10.4.1 (this chapter covers the fast, non-cryptographic kind and reuses the birthday bound for IDs).
- B-trees and indexes — §11.7, §11.8 (geospatial indexes reduce two dimensions to one so that a B-tree or sorted set can serve them).
- LSM trees, memtables and SSTables — §13.8 (the most common home of Bloom filters and skip lists).
- Partitioning by hash — §14.6 (hash quality decides partition balance).

17.3 Hash functions at scale

17.3.1 What a hash function must do

DEFINITION

Definition — Hash function. A deterministic function h that maps an input of any length to a fixed-size integer (for example 32, 64 or 128 bits), called the **hash** or **digest**. A **non-cryptographic hash function** is designed for speed and statistical uniformity on ordinary inputs. A **cryptographic hash function** additionally resists an adversary who tries to find collisions or preimages (§10.4.1).

First-principles explanation

Imagine a cloakroom with 1,000 numbered hooks. You want to hang each coat on a hook chosen by a rule, so that later you can go straight to the right hook instead of searching all of them. A good rule looks at the whole ticket and scatters tickets evenly: no hook gets 50 coats while others are empty. It must give the same hook for the same ticket every time. And it must be fast, because the queue is long. That rule is a hash function, and the hooks are **buckets**.

Every structure in this chapter starts with a hash: hash tables pick buckets, partitioned databases pick nodes (§14.6), Bloom filters pick bits, HyperLogLog reads output bits as coin flips. A poor hash passes its flaw to all of them.

Why does this exist?

At scale three demands sharpen. **Throughput**: a cache node may hash millions of keys per second. **Uniformity on structured input**: real keys look like `user:1`, `user:2`, and a weak hash maps near-identical keys to near-identical outputs. **Adversarial input**: an attacker who controls keys (HTTP parameters, JSON field names) can choose keys that all collide (§17.3.4).

How it works internally

MurmurHash3 and its relatives process the input in fixed-size blocks, repeatedly multiplying by large odd constants, rotating bits and XOR-ing into a running state; a **finalizer** then scrambles the state so that every input bit affects every output bit. Four properties matter:

1. **Determinism.** Same input and seed, same output, on every machine and run. A hash that changes between versions re-routes every key.
2. **Uniformity.** For realistic key sets, outputs are spread evenly over the output range, so `h(key) mod B` spreads evenly over B buckets.
3. **Avalanche.** Flipping one input bit flips each output bit with probability close to 1/2. This is what makes `user:1` and `user:2` land far apart.
4. **Speed.** A few CPU cycles per byte and low overhead for short keys.

A **seed** mixed into the initial state gives effectively independent hash functions from the same code; keyed hashes (§17.3.4) keep the seed secret.

17.3.2 MurmurHash3 and xxHash

MurmurHash3 (Austin Appleby, published with the SMHasher test suite) has 32- and 128-bit variants and is Apache Cassandra's default partitioner hash (`Murmur3Partitioner`) [Appleby: SMHasher; Apache Cassandra Docs]. **xxHash** (Yann Collet) offers XXH32, XXH64 and XXH3; its author reports higher throughput than MurmurHash3, especially on

long inputs [Collet: xxHash]. Both pass SMHasher's statistical tests, and neither is cryptographic.

Table 17.1 — Hash functions you will meet in system design.

Function	Output	Keyed / adversary-resistant?	Relative speed	Typical use
CRC32	32 bits	No; designed for error detection	Very fast (hardware instruction)	Checksums, some sharding (Redis Cluster uses CRC16)
FNV-1a	32/64 bits	No	Fast for short keys, weak mixing	Small in-process tables
MurmurHash3	32/128 bits	No (seeded, not secret)	Fast	Partitioners, Bloom filters, sketches
xxHash (XXH64, XXH3)	64/128 bits	No	Very fast	Checksums, sketches, deduplication
SipHash	64 bits	Yes, with a secret 128-bit key	Moderate	Hash tables exposed to untrusted keys
SHA-256, BLAKE2/BLAKE3	256 bits	Yes (cryptographic)	Slowest of these, still GB/s for BLAKE3	Content addressing, Merkle trees, integrity

Speeds are qualitative (as of 2025); benchmark on your hardware. This chapter's code uses BLAKE2b from Python's standard library for portability, not speed.

> RECOMMENDATION
>
> **Pick the hash by threat model, then by speed.** Untrusted keys in an in-memory table: SipHash or a bounded-worst-case structure. Keys from your own system (partitioning, sketches, filters): MurmurHash3 or xxHash. Integrity: a cryptographic hash. Never change a live system's partitioning hash without a migration plan.

17.3.3 Measuring uniformity

Mathematical foundations

Under a uniform hash, bucket counts are binomial, close to Poisson with mean n/B.

FORMULA

Expected bucket load and the chi-square uniformity test

$$\mu = \frac{n}{B}, \qquad \sigma \approx \sqrt{\frac{n}{B}}, \qquad \chi^2 = \sum_{i=1}^{B} \frac{(c_i - \mu)^2}{\mu}, \qquad \mathbb{E}[\chi^2] \approx B - 1, \quad \mathrm{sd}(\chi^2) \approx \sqrt{2(B-1)}$$

Symbol	Meaning	Units
n	Number of keys hashed	keys
B	Number of buckets	buckets
μ	Expected keys per bucket	keys/bucket
σ	Standard deviation of a bucket's count under a uniform hash	keys/bucket
c_i	Observed number of keys in bucket i	keys
χ^2	Chi-square statistic: total squared deviation from uniform	dimensionless

WORKED EXAMPLE

Worked example — what "uniform" looks like for 100,000 keys in 1,024 buckets.

1. Mean load: $\mu = 100,000/1,024 = 97.7$ keys per bucket.
2. Spread: $\sigma \approx \sqrt{97.7} = 9.9$. About 95% of buckets should hold between $97.7 - 2 \times 9.9 = 78$ and $97.7 + 2 \times 9.9 = 118$ keys.
3. The fullest of 1,024 buckets is typically about 3.2 standard deviations above the mean: $97.7 + 3.2 \times 9.9 \approx 129$ keys.
4. The chi-square statistic for a uniform hash should be near $B - 1 = 1,023$, with standard deviation $\sqrt{2 \times 1,023} = 45$. Values above about $1,023 + 3 \times 45 = 1,158$ suggest non-uniformity.

Interpretation. Even a perfect hash leaves a 30% spread between emptiest and fullest bucket. *Engineering implication.* Provision partitions for the fullest one, not the average; the relative spread $1/\sqrt{\mu}$ is one reason systems use many virtual nodes per server (§14.8).

Code example

The program implements MurmurHash3 (x86, 32-bit) and compares it with a deliberately bad hash on 100,000 structured keys.

```
        def murmur3_32(data: bytes, seed: int = 0) -> int:
            """MurmurHash3 x86_32 (Appleby), pure Python. Returns an unsigned 32-bit
int."""
            c1, c2, mask = 0xCC9E2D51, 0x1B873593, 0xFFFFFFFF
            h = seed & mask
            n_blocks = len(data) // 4
            for i in range(n_blocks):                       # body: 4-byte little-endian
blocks
                k = int.from_bytes(data[4 * i:4 * i + 4], "little")
                k = (k * c1) & mask
                k = ((k << 15) | (k >> 17)) & mask          # rotl32(k, 15)
                k = (k * c2) & mask
                h ^= k
                h = ((h << 13) | (h >> 19)) & mask          # rotl32(h, 13)
                h = (h * 5 + 0xE6546B64) & mask
            tail, k = data[4 * n_blocks:], 0                # tail: remaining 1-3 bytes
            for i, byte in enumerate(tail):
                k |= byte << (8 * i)
            if tail:
                k = (k * c1) & mask
                k = ((k << 15) | (k >> 17)) & mask
                k = (k * c2) & mask
                h ^= k
            h ^= len(data)                                  # finalization ("fmix32")
            h ^= h >> 16
            h = (h * 0x85EBCA6B) & mask
            h ^= h >> 13
            h = (h * 0xC2B2AE35) & mask
            h ^= h >> 16
            return h

        def additive_hash(data: bytes) -> int:
            """A deliberately bad hash: sum of byte values."""
            return sum(data)

        def bucket_stats(hash_fn, keys, buckets):
            counts = [0] * buckets
            for key in keys:
                counts[hash_fn(key) % buckets] += 1
            expected = len(keys) / buckets
            chi2 = sum((c - expected) ** 2 / expected for c in counts)
            return max(counts), sum(1 for c in counts if c == 0), chi2

        print(f"murmur3_32('hello') = {murmur3_32(b'hello'):#010x}")   # reference:
0x248bfa47

        # 100,000 realistic, highly structured keys: "user:0" ... "user:99999"
        keys = [f"user:{i}".encode() for i in range(100_000)]
        B = 1024                                            # expected ~97.7 keys per
bucket
        for name, fn in [("additive", additive_hash), ("murmur3", murmur3_32)]:
            mx, empty, chi2 = bucket_stats(fn, keys, B)
            print(f"{name:9s} max bucket = {mx:6d}  empty buckets = {empty:4d}  chi-
square = {chi2:12.0f}")
```

Output (Python 3.13):

```
murmur3_32('hello') = 0x248bfa47
additive  max bucket =   5520  empty buckets =  888  chi-square =      3598840
murmur3   max bucket =    132  empty buckets =    0  chi-square =         1105
```

What to notice:

- The implementation matches the reference value `0x248bfa47` for `"hello"` (and matched the C library on further test strings).
- MurmurHash3's fullest bucket (132) is close to the predicted ~129, and its chi-square (1,105) is within two standard deviations of 1,023: statistically indistinguishable from uniform.
- The additive hash leaves 888 buckets empty: the byte sums of `user:NNNNN` take fewer than 150 distinct values (553–790), so one bucket holds 5,520 keys.
- With a power-of-two B, `mod` keeps only low bits, which is safe only for hashes that mix well into them.

17.3.4 Hash flooding and keyed hashes

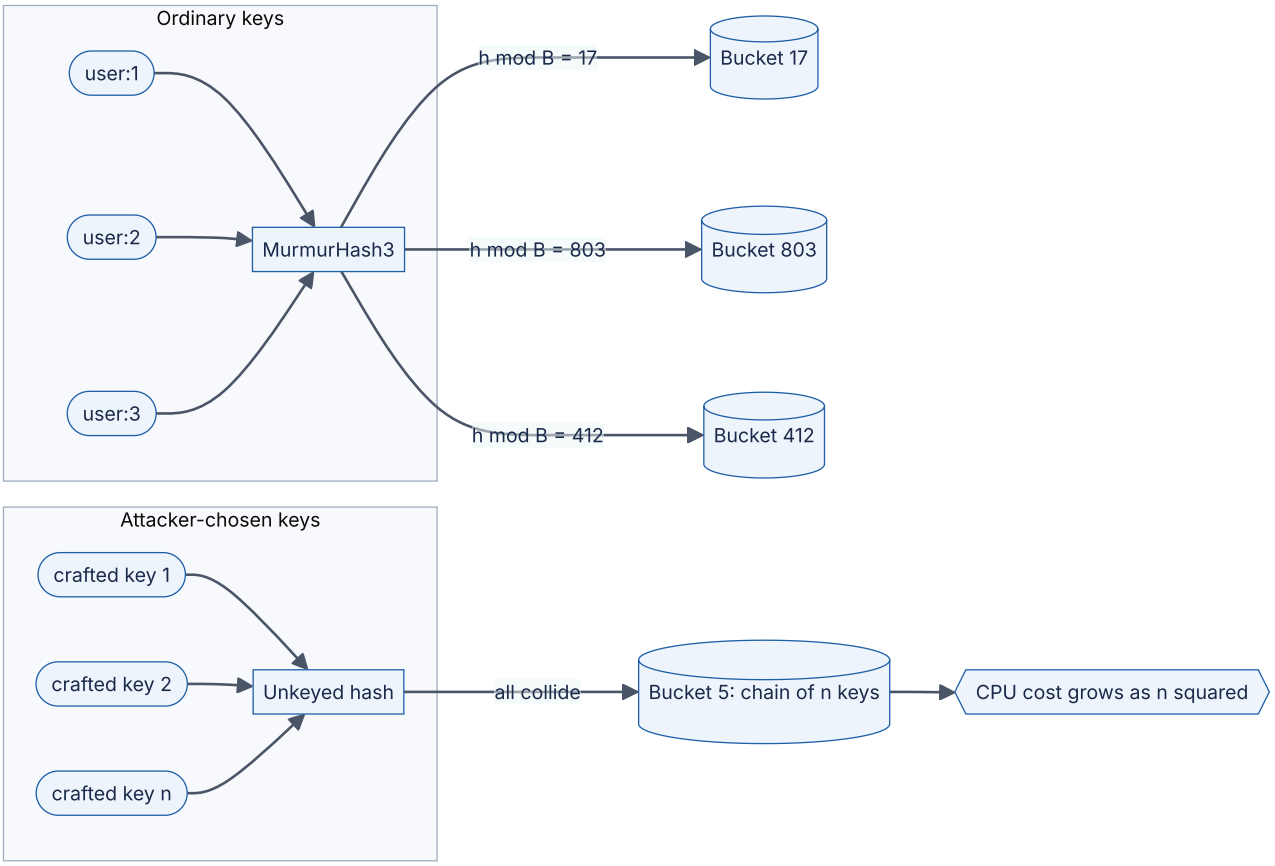
A chained hash table does $O(1)$ work per operation *on average*. If every key lands in one bucket, each insert walks a chain of all earlier keys, and inserting n keys costs $\sum_{i=1}^{n} i \approx n^2/2$ comparisons. Crosby and Wallach showed in 2003 that attackers can exploit this **algorithmic complexity attack** by sending inputs crafted to collide [Crosby & Wallach 2003]. In December 2011, Klink and Wälde demonstrated it against the hash tables that web frameworks in PHP, Java, Python, Ruby and ASP.NET used to store HTTP form parameters [Klink & Wälde 2011]. A single request with many colliding parameter names could occupy a CPU core for a long time. This is **hash flooding**.

> WORKED EXAMPLE
>
> **Worked example — the cost of 100,000 colliding keys.**
>
> 1. Uniform hash: about $100,000 \times 1 = 10^5$ bucket operations.
> 2. All keys colliding: about $n^2/2 = (10^5)^2/2 = 5 \times 10^9$ key comparisons.
> 3. At an optimistic 10^9 comparisons per second per core, that is 5 s of CPU for one request; at a more realistic 10^8 per second for string comparisons in an interpreted runtime, 50 s.
>
> *Interpretation.* One request of a few megabytes costs you seconds of CPU.
> *Engineering implication.* Any hash table filled from untrusted input needs a secret-keyed hash or a bounded worst case.

Three defenses are used in practice:

1. **Keyed hashes.** SipHash is a fast pseudorandom function keyed with a 128-bit secret chosen at process start, so attackers cannot predict collisions [Aumasson & Bernstein 2012]. Python uses it for `str` and `bytes` since 3.4 [PEP 456]; Rust's `HashMap` uses a SipHash variant. Consequence: Python's `hash()` differs between processes, so never use it for partitioning or persistence.
2. **Bounded worst case.** Since Java 8, `HashMap` turns long bucket chains into balanced trees, $O(\log n)$ per operation [JEP 180].
3. **Input limits.** Cap parameters, JSON keys and headers per request.

Figure 17.1 — Uniform hashing versus hash flooding.



Reading the diagram. Upper group: three structured keys pass through MurmurHash3 and, thanks to avalanche, land in unrelated buckets, so operations stay constant-time. Lower group: an attacker who knows an unkeyed hash precomputes keys that all map to bucket 5; every insert walks the whole chain and total work grows quadratically (the hexagon is the external effect, CPU exhaustion). Arrows are synchronous in-process calls; buckets are in-memory storage. The failure point is the hash function, not capacity, so adding servers does not help; the fixes are an unpredictable bucket choice (secret SipHash key) or bounded per-bucket cost (tree bins).

17.3.5 From hashes to partitions

With `hash mod N`, changing N moves almost every key (a key stays only if `h mod N` equals `h mod (N+1)`, about $1/(N+1)$ of keys). **Consistent hashing** places nodes and keys on a ring so a membership change moves only about $1/N$ of keys; jump and rendezvous hashing are alternatives. Chapter 14 owns this topic (§14.8; walk-through in §3.3.2). It assumes a uniform underlying hash.

Failure scenarios

Table 17.2 — Hash-function failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
One partition gets 5× the traffic of others	Weak hash on structured keys, or `mod` of a power of two keeping poorly mixed low bits	Per-partition request and size metrics; chi-square on a key sample	Re-hash with a better function (requires data migration)	Use MurmurHash3/xxHash; test with real key samples
CPU pinned by a few requests	Hash flooding of an unkeyed table	Profiling shows time inside hash-table insert; requests with thousands of parameters	Rate-limit offending clients; restart with patched runtime	Keyed hash (SipHash), tree bins, parameter-count limits
Keys "move" after a deploy	Language runtime's built-in `hash()` is randomized per process or changed between versions	Cache hit ratio collapses after a deploy; data written under one hash not found	Roll back; rebuild index	Use an explicitly specified, versioned hash for anything persisted or shared
Different services disagree on a key's partition	One service hashes UTF-8 bytes, another UTF-16, or seeds differ	Cross-service lookups miss	Fix encoding and seed; backfill	Specify encoding, algorithm, seed and byte order in the interface contract

Common misconceptions

> COMMON MISTAKE
>
> **"A 64-bit hash never collides, so I can use it as a unique ID."** By the birthday bound (§10.4.1), collisions among random 64-bit values become likely at about $2^{32} \approx 4.3$ billion items. A fingerprint is fine for sharding or deduplication hints, but anything that must be unique needs either a collision check or a much larger space (§17.10).

COMMON MISTAKE

"Cryptographic hashes are always better." They are slower and solve a different problem; for bucketing ordinary keys they are no more uniform than MurmurHash3. Use them where an adversary benefits from a collision.

17.4 Bloom filters and cuckoo filters

17.4.1 The membership problem

DEFINITION

Definition — Bloom filter. A probabilistic data structure for set membership, introduced by Burton Bloom in 1970, that stores a set of n items in an array of m bits using k hash functions [Bloom 1970]. A query answers either "definitely not in the set" or "probably in the set". It never gives a **false negative** (saying "no" for an item that was added), and gives a **false positive** (saying "yes" for an item that was not added) with a probability p that depends on m, n and k.

First-principles explanation

Picture a bouncer with a terrible memory but a stamp pad. Each guest who enters stamps three positions on a wall chart, chosen by the letters of their name. Asked "Was Alice here?", the bouncer checks Alice's three positions. If any is blank, she was certainly never here. If all are stamped, she was *probably* here, though other guests' stamps might happen to cover her positions. The chart is tiny, never forgets an entrant, and occasionally vouches for a stranger.

A Bloom filter is useful wherever "definitely not" lets you skip something expensive and an occasional "maybe" costs only a little wasted work.

Why does this exist?

Bloom's 1970 paper used a small in-memory filter so a hyphenation program could skip a disk-resident dictionary for most words [Bloom 1970]. The same shape recurs at scale:

- An LSM-tree database (§13.8) keeps many immutable files; a per-file filter in memory says "not in this file" for most of them.
- A crawler must not refetch any of billions of URLs; an exact URL set would take hundreds of gigabytes.
- A CDN should not write an object to disk on its first request, because most objects are never requested again.

In each case the filter turns most memory- or I/O-heavy exact lookups into a check of a few bits.

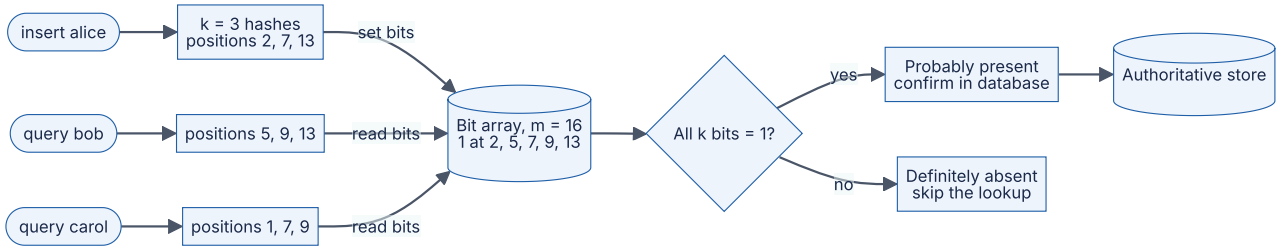
17.4.2 How it works internally

1. **Create** an array of m bits, all 0, and choose k independent hash functions $h_1, \ldots, h_k$, each mapping an item to a position in $[0, m)$.
2. **Insert** item x: compute $h_1(x), \ldots, h_k(x)$ and set those k bits to 1. Bits already set stay set.
3. **Query** item y: compute the same k positions. If any bit is 0, answer "definitely not present". If all are 1, answer "probably present".
4. **Never delete** by clearing bits: a bit may belong to several items, so clearing it creates false negatives (§17.4.6 covers deletable variants).

You do not need k unrelated hash functions: two hashes combined as $g_i(x) = h_a(x) + i \cdot h_b(x) \mod m$ give the same asymptotic false-positive rate [Kirsch & Mitzenmacher 2006], so one 128-bit hash split in two suffices. Filters with identical m, k and hashes merge by bitwise OR into a filter of the **union**.

Architecture diagram

Figure 17.2 — Inserting and querying a Bloom filter with k = 3.



Reading the diagram. Inserting `alice` sets bits 2, 7 and 13; an earlier insert set 5 and 9. A query for `bob` hits 5, 9 and 13, all set, so the filter says "probably present" although `bob` was never inserted: a false positive whose only cost is an unnecessary check against the authoritative store. A query for `carol` hits bit 1, which is 0, so the expensive lookup is skipped. All arrows are synchronous in-memory operations costing nanoseconds; the only I/O is the confirmation. Losing the filter never harms correctness (rebuild it from the data); undersizing it makes "maybe" frequent and erases the savings. Scale it by partitioning alongside the data it summarizes, one filter per file or shard.

17.4.3 Mathematical foundations

After inserting n items with k hashes into m bits, the probability that a particular bit is still 0 is $(1 - 1/m)^{kn} \approx e^{-kn/m}$, because each of the kn bit-settings misses that bit with probability $1 - 1/m$. A false positive needs all k positions of a new item to be 1. Treating the posi-

tions as independent gives the standard approximation [Bloom 1970; Broder & Mitzenmacher 2004].

FORMULA

Bloom filter false-positive rate

$$p \approx (1 - e^{-kn/m})^k$$

Symbol	Meaning	Units
p	Probability that a query for an item not in the set returns "probably present"	probability (0–1)
n	Number of items inserted	items
m	Size of the bit array	bits
k	Number of hash functions (bit positions per item)	count
$e^{-kn/m}$	Approximate fraction of bits still 0	fraction

WORKED EXAMPLE

Worked example — predicting the false-positive rate. You allocate $m = 10,000,000$ bits (1.25 MB) for $n = 1,000,000$ items with $k = 7$.

1. Exponent: $kn/m = 7 \times 10^6 / 10^7 = 0.7$.
2. Fraction of bits still 0: $e^{-0.7} = 0.4966$. So $1 - 0.4966 = 0.5034$ of the bits are 1, which is about half.
3. False-positive rate: 0.5034^7. Take logs: $7 \times \ln 0.5034 = 7 \times (-0.6864) = -4.805$; $e^{-4.805} = 0.0082$.

Interpretation. 82 in 10,000 absent-item queries return "maybe". *Engineering implication.* At 10,000 absent-key queries per second, each "maybe" costing a 10 ms disk read, the filter turns 10,000 disk reads per second into 82 for 1.25 MB of memory.

More hashes check more bits per query (lowering p) but set more bits per insert (raising it). Minimizing p over k gives:

FORMULA

Optimal number of hash functions

$$k^* = \frac{m}{n} \ln 2 \approx 0.693 \frac{m}{n}, \qquad p|_{k = k^*} = \left(\frac{1}{2}\right)^{k^*}$$

Symbol	Meaning	Units
k^*	Number of hash functions that minimizes p (round to an integer)	count
m/n	Bits per element	bits/item
$\ln 2$	Natural logarithm of 2, ≈ 0.693	dimensionless

WORKED EXAMPLE

Worked example — choosing k. With $m/n = 10$ bits per element: $k^* = 10 \times 0.693 = 6.93$, so use $k = 7$. At the optimum, exactly half of the bits are 1 ($e^{-k^* n/m} = e^{-\ln 2} = 1/2$) and $p = 0.5^{6.93} = 0.0082$, matching the previous example.

Interpretation. At the optimum the array is half full. *Engineering implication.* The fraction of 1 bits is a cheap health metric: alert when it passes about 0.5.

Usually you start from a tolerable p and solve for memory:

FORMULA

Bits per element for a target false-positive rate

$$\frac{m}{n} = \frac{-\ln p}{(\ln 2)^2} \approx 1.44 \log_2 \frac{1}{p}, \qquad k = -\log_2 p$$

Symbol	Meaning	Units
m/n	Bits of filter needed per inserted item	bits/item
p	Target false-positive rate	probability
$(\ln 2)^2$	≈ 0.4805	dimensionless
k	Matching number of hash functions (rounded)	count

WORKED EXAMPLE

Worked example — a username-availability filter for 1 billion accounts. You want “is this username taken?” answered in memory with a 1% false-positive rate.

1. Bits per element: $-\ln 0.01/0.4805 = 4.605/0.4805 = 9.585$ bits.
2. Total: 9.585×10^9 bits $= 1.198 \times 10^9$ bytes ≈ 1.2 GB (decimal).
3. Hash functions: $k = -\log_2 0.01 = 6.64$, so 7.
4. For 0.1%: $-\ln 0.001/0.4805 = 6.908/0.4805 = 14.38$ bits per element, 1.8 GB, $k = 10$.
5. Compare with an exact set: 1 billion usernames averaging 12 bytes, plus about 40 bytes of hash-table and object overhead each, is roughly 52×10^9 bytes = 52 GB.

Interpretation. Each 10× reduction in p costs a fixed $\ln 10/(\ln 2)^2 = 4.79$ extra bits per element, whatever the item size. *Engineering implication.* The filter fits in one server’s memory (or on every API node), where the exact set needs a cluster. “Taken” answers are confirmed against the database; “free” answers are always correct, and those are the ones that must be fast while the user types.

Table 17.3 — Bloom filter sizing at the optimal k.

Target p	Bits per element	Hash functions k	Memory for 10^9 items
10%	4.79	3	0.60 GB
1%	9.59	7	1.20 GB
0.1%	14.38	10	1.80 GB
0.01%	19.17	13	2.40 GB
0.001%	23.96	17	3.00 GB

The approximation assumes independent bit positions; it slightly underestimates the true rate for small filters [Bose et al. 2008], negligibly at millions of bits.

17.4.4 Implementation and measured validation

```
        import hashlib
        import math

        class BloomFilter:
            """Standard Bloom filter using Kirsch-Mitzenmacher double hashing."""

            def __init__(self, n_expected: int, fp_target: float):
                # m = -n ln p / (ln 2)^2 ; k = (m / n) ln 2
                self.m = math.ceil(-n_expected * math.log(fp_target) / math.log(2) ** 2)
                self.k = max(1, round(self.m / n_expected * math.log(2)))
                self.bits = bytearray((self.m + 7) // 8)

            def _positions(self, item: str):
                digest = hashlib.blake2b(item.encode(), digest_size=16).digest()
                h1 = int.from_bytes(digest[:8], "little")
                h2 = int.from_bytes(digest[8:], "little") | 1     # odd, so it never
sticks at h1
                return [(h1 + i * h2) % self.m for i in range(self.k)]

            def add(self, item: str) -> None:
                for pos in self._positions(item):
                    self.bits[pos >> 3] |= 1 << (pos & 7)

            def __contains__(self, item: str) -> bool:
                return all(self.bits[pos >> 3] & (1 << (pos & 7)) for pos in
self._positions(item))

        def predicted_fp(m: int, n: int, k: int) -> float:
            return (1 - math.exp(-k * n / m)) ** k

        n, probes = 100_000, 1_000_000
        print(f"{'target p':>9} {'m (bits)':>10} {'bits/item':>9} {'k':>3}
{'predicted':>10} {'measured':>10}")
        for target in (0.05, 0.01, 0.001):
            bf = BloomFilter(n, target)
            for i in range(n):
                bf.add(f"member-{i}")
            assert all(f"member-{i}" in bf for i in range(n))    # no false negatives,
ever
            fps = sum(f"stranger-{i}" in bf for i in range(probes))
            print(f"{target:9.3f} {bf.m:10d} {bf.m / n:9.2f} {bf.k:3d} "
                  f"{predicted_fp(bf.m, n, bf.k):10.5f} {fps / probes:10.5f}")

        # What happens if you insert 3x more items than the filter was sized for?
        bf = BloomFilter(n, 0.01)
        for i in range(3 * n):
            bf.add(f"member-{i}")
        fps = sum(f"stranger-{i}" in bf for i in range(probes))
        print(f"overfilled 3x: predicted {predicted_fp(bf.m, 3 * n, bf.k):.4f}, measured
{fps / probes:.4f}")
```

Output:

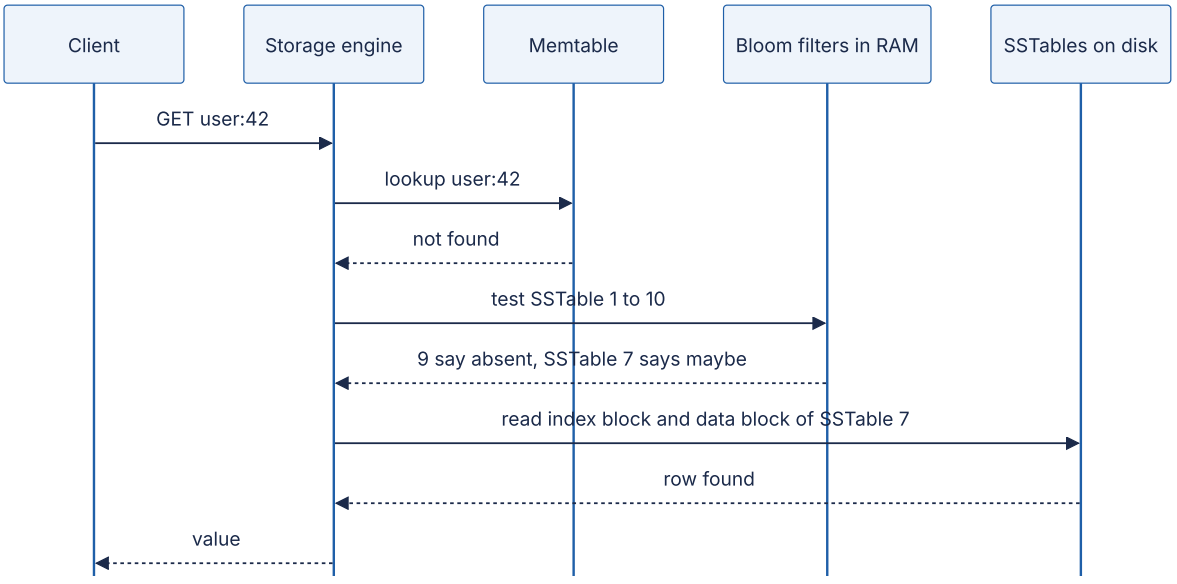
```
 target p   m (bits) bits/item   k  predicted   measured
    0.050     623523      6.24   4    0.05027    0.05044
    0.010     958506      9.59   7    0.01004    0.01018
    0.001    1437759     14.38  10    0.00100    0.00098
overfilled 3x: predicted 0.4360, measured 0.4376
```

What to notice:

- The sizing rule reproduces Table 17.3: 9.59 bits and $k = 7$ for 1%, 14.38 bits and $k = 10$ for 0.1%.
- Measured rates match predictions within sampling noise: with 10^6 probes at $p = 0.001$ you expect $1,000 \pm 32$ false positives, and 980 is inside that. (A 200,000-probe run measured 0.00117, 2.4 standard deviations high; use enough probes to resolve the rate.)
- The `assert` confirms the defining guarantee: every inserted item is found.
- Overfilling is the most common real failure: 3× the design load gives 44% false positives. Size for peak n or rebuild as n grows.

17.4.5 Bloom filters in system architecture

Figure 17.3 — Bloom filters on the read path of an LSM-tree storage engine.



Reading the diagram. After a memtable miss (§13.8), the engine would otherwise search every SSTable on disk. Instead it tests the key against each SSTable's RAM-resident filter (well under a microsecond each); nine say "absent" and are never touched, and only SSTable 7 costs disk reads. All calls are synchronous; the storage boundary separates RAM (memtable, filters) from disk (SSTables). For a missing key, ten files at $p = 1\%$ waste $10 \times 0.01 = 0.1$ reads instead of 10. Failure points: filters evicted from memory turn lookups back into disk reads, and range scans cannot use point filters (prefix Bloom filters

help within a known prefix). Scaling is automatic: each new SSTable carries its own filter. §13.10 covers the storage-engine side.

INDUSTRY EXAMPLE

Google Bigtable and Apache Cassandra. Bigtable lets clients enable Bloom filters per locality group so that most lookups for non-existent rows or columns never touch disk [Chang et al. 2006]. Cassandra keeps a Bloom filter per SSTable, tuned by the table option `bloom_filter_fp_chance` [Apache Cassandra Docs]. *Evidence: Primary* — [Chang et al. 2006; Apache Cassandra Docs]

INDUSTRY EXAMPLE

Akamai's "one-hit wonders". Akamai engineers reported that a large share of objects requested from their CDN servers are requested only once, so caching on first request wastes disk bandwidth and evicts useful content. Their servers record first requests in a Bloom filter and cache an object only on its second request [Maggs & Sitaraman 2015]. *Evidence: Primary* — [Maggs & Sitaraman 2015]

17.4.6 Counting Bloom filters

A **counting Bloom filter** replaces each bit with a small counter, typically 4 bits [Fan et al. 2000]. Insert increments the k counters, delete decrements them, and a query checks that all are non-zero, so the false-positive rate equals that of a bit filter with the same m and k. Costs: 4× the memory; counter overflow, which is astronomically unlikely with optimal k and 4-bit counters [Fan et al. 2000; Broder & Mitzenmacher 2004]; and false negatives if you delete an item that was never inserted. Fan and colleagues introduced it for the **Summary Cache** protocol, in which web proxies exchanged compact, constantly changing summaries of their cache contents.

17.4.7 Cuckoo filters

A **cuckoo filter** supports deletion with less memory than a counting Bloom filter and, at low false-positive rates, less than a plain Bloom filter [Fan et al. 2014].

DEFINITION

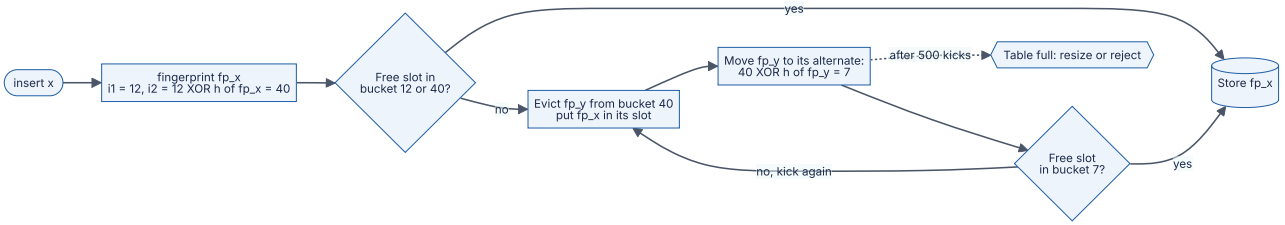
Definition — Cuckoo filter. A hash table of b-slot buckets that stores a short **fingerprint** of each item (an f-bit hash) in one of two candidate buckets. Membership checks two buckets for a matching fingerprint; deletion removes one copy of the fingerprint.

How it works internally

The name comes from the cuckoo bird, which lays its egg in another bird's nest and pushes an existing egg out.

1. For item x, compute a fingerprint $ = $ an f-bit hash of x, and the first bucket $i_1 = h(x) \mod B$.
2. Compute the alternate bucket $i_2 = i_1 \oplus h(fp)$. This is **partial-key cuckoo hashing**: because XOR is its own inverse, $i_1 = i_2 \oplus h(fp)$, so you can find an entry's other bucket from the fingerprint alone, without the original item.
3. **Insert**: if either bucket has a free slot, store fp there. Otherwise pick one of the two buckets, evict a random fingerprint from it, store fp in its place, and move the evicted fingerprint to *its* alternate bucket, repeating up to a limit (for example 500 kicks). If the limit is reached, the table is full.
4. **Lookup**: return true if either bucket contains fp.
5. **Delete**: remove one copy of fp from whichever bucket holds it.

Figure 17.4 — Inserting into a full cuckoo filter by relocation.



Reading the diagram. If either candidate bucket has room, the insert ends at once, the common case below about 90% load. Otherwise the filter displaces `fp_y` and moves it to its alternate bucket, computed from `fp_y` alone, repeating until a slot frees up. All steps are synchronous memory operations. Insert latency climbs as the table fills, and the dotted edge marks the failure point: after the kick limit the filter is full and must be rebuilt larger, since original items are not stored. Lookups and deletes always touch exactly two buckets.

Mathematical foundations

A lookup compares against up to $2b$ stored fingerprints, each matching by chance with probability 2^{-f}.

FORMULA

Cuckoo filter false-positive bound and space cost

$$\varepsilon \le \frac{2b}{2^f}, \qquad f \ge \log_2 \frac{2b}{\varepsilon}, \qquad \text{bits per item} = \frac{f}{\alpha} \approx \frac{\log_2(1/\varepsilon) + \log_2(2b)}{\alpha}$$

Symbol	Meaning	Units
ε	False-positive rate	probability
b	Slots per bucket (4 is the common choice)	slots
f	Fingerprint length	bits
α	Load factor reached before inserts fail (≈ 0.95 for $b = 4$)	fraction

WORKED EXAMPLE

Worked example — cuckoo versus Bloom at 1% and 0.1%. Take $b = 4$ and $\alpha = 0.95$, so $\log_2(2b) = 3$.

1. At $\varepsilon = 1\%$: $\log_2 100 = 6.64$; cuckoo needs $(6.64 + 3)/0.95 = 10.1$ bits per item. Bloom needs 9.59.
2. At $\varepsilon = 0.1\%$: $\log_2 1000 = 9.97$; cuckoo needs $(9.97 + 3)/0.95 = 13.7$ bits. Bloom needs 14.38.
3. Crossover: cuckoo wins when $1.44x > (x + 3)/0.95$, where $x = \log_2(1/\varepsilon)$, i.e. $1.37x > x + 3$, so $x > 8.1$ and $\varepsilon < 0.4\%$. Fan and colleagues' **semi-sorting** optimization saves one bit per item, which moves the crossover to about 3% [Fan et al. 2014].

Interpretation. Bloom filters pay 44% over the minimum $\log_2(1/\varepsilon)$; cuckoo filters pay 2–3 bits plus load-factor loss. *Engineering implication.* For low rates or deletion, prefer a cuckoo filter; for rates of 3% or more on append-only sets, a Bloom filter is smaller, simpler, and degrades gracefully when overfilled instead of refusing inserts.

Code example

```
        import hashlib
        import random

        class CuckooFilter:
            """Cuckoo filter (Fan et al. 2014): b slots per bucket, f-bit fingerprints,
            partial-key cuckoo hashing. Supports delete. num_buckets must be a power of
two."""

            def __init__(self, num_buckets: int, bucket_size: int = 4, fp_bits: int = 8,
max_kicks: int = 500):
                assert num_buckets & (num_buckets - 1) == 0
                self.nb, self.b, self.f, self.max_kicks = num_buckets, bucket_size,
fp_bits, max_kicks
                self.buckets = [[] for _ in range(num_buckets)]
                self.count = 0
                self.victim = None          # (bucket, fp) left homeless by a failed in-
sert
                self.rng = random.Random(7)

            def _hash(self, data: bytes) -> int:
                return int.from_bytes(hashlib.blake2b(data, digest_size=8).digest(),
"little")

            def _fingerprint_and_index(self, item: str):
                h = self._hash(item.encode())
                fp = (h >> 32) & ((1 << self.f) - 1) or 1       # fingerprint 0 is re-
served for "empty"
                return fp, (h & 0xFFFFFFFF) % self.nb

            def _alt_index(self, index: int, fp: int) -> int:
                # i2 = i1 XOR hash(fp): applying it twice returns the original bucket
                return (index ^ self._hash(fp.to_bytes(4, "little"))) % self.nb

            def add(self, item: str) -> bool:
                if self.victim is not None:
                    return False                                 # full: refuse further
inserts
                fp, i1 = self._fingerprint_and_index(item)
                i2 = self._alt_index(i1, fp)
                for i in (i1, i2):
                    if len(self.buckets[i]) < self.b:
                        self.buckets[i].append(fp)
                        self.count += 1
                        return True
                i = self.rng.choice((i1, i2))                    # both full: evict
("kick") a victim
                for _ in range(self.max_kicks):
                    j = self.rng.randrange(self.b)
                    fp, self.buckets[i][j] = self.buckets[i][j], fp
                    i = self._alt_index(i, fp)
                    if len(self.buckets[i]) < self.b:
                        self.buckets[i].append(fp)
                        self.count += 1
                        return True
```

```
                self.victim = (i, fp)                                   # keep the homeless
fingerprint
                self.count += 1                                         # so nothing is ever
lost
                return False

            def __contains__(self, item: str) -> bool:
                fp, i1 = self._fingerprint_and_index(item)
                i2 = self._alt_index(i1, fp)
                if self.victim is not None and self.victim[1] == fp and self.victim[0]
in (i1, i2):
                    return True
                return fp in self.buckets[i1] or fp in self.buckets[i2]

            def delete(self, item: str) -> bool:
                fp, i1 = self._fingerprint_and_index(item)
                i2 = self._alt_index(i1, fp)
                if self.victim is not None and self.victim[1] == fp and self.victim[0]
in (i1, i2):
                    self.victim = None
                    self.count -= 1
                    return True
                for i in (i1, i2):
                    if fp in self.buckets[i]:
                        self.buckets[i].remove(fp)
                        self.count -= 1
                        return True
                return False

        cf = CuckooFilter(num_buckets=2 ** 15, bucket_size=4, fp_bits=8)   # 131,072
slots
        capacity = cf.nb * cf.b
        inserted = 0
        while cf.add(f"member-{inserted}"):
            inserted += 1
        inserted += 1                                                   # the failed item is
held as the victim
        print(f"filter full at load factor {inserted / capacity:.3f}")
        assert all(f"member-{i}" in cf for i in range(inserted))  # no false negatives

        probes = 500_000
        fps = sum(f"stranger-{i}" in cf for i in range(probes))
        bound = 2 * cf.b / 2 ** cf.f
        print(f"false-positive rate: measured {fps / probes:.4f}, bound 2b/2^f =
{bound:.4f}")
        print(f"bits per stored item: {cf.f * capacity / inserted:.2f}")

        for i in range(0, inserted, 2):                                 # delete every second
member
            assert cf.delete(f"member-{i}")
        still_there = all(f"member-{i}" in cf for i in range(1, inserted, 2))
        print(f"after deleting half: count = {cf.count}, all survivors found =
{still_there}")
```

Output:

```
filter full at load factor 0.963
false-positive rate: measured 0.0298, bound 2b/2^f = 0.0312
bits per stored item: 8.30
after deleting half: count = 63135, all survivors found = True
```

What to notice:

- With $b = 4$ the table reached 96.3% load before the first failure, consistent with Fan and colleagues' ~95%.
- The measured rate (2.98%) sits under the bound $8/256 = 3.1\%$, which assumes every slot is full.
- Deletion works, but deleting a never-inserted item could remove another item's matching fingerprint; delete only known members.
- The `victim` slot, as in the reference implementation, keeps the last displaced fingerprint so a failed insert never causes a false negative.

Alternatives and trade-offs

Table 17.4 — Approximate membership filters compared.

	Bloom	Counting Bloom	Cuckoo	Static filters (XOR, ribbon)
What it is	Bit array, *k* hashes	Bloom with 4-bit counters	Fingerprints in 2 candidate buckets	Fingerprints placed by solving equations over a fixed set
Delete	No	Yes (only items present)	Yes (only items present)	No; rebuild
Bits per item at 1%	9.6	~38	~10 (~9 with semi-sorting)	~8–9.8
Lookup memory accesses	*k* (scattered)	*k*	2 buckets	~3
Behavior when overfilled	Degrades smoothly (*p* rises)	Degrades; counters may overflow	Inserts fail	Must know the set in advance
Union of two filters	Bitwise OR	Add counters	Not directly	Not directly
Choose when	Append-only sets, simplicity, mergeability	Deletion with modest memory pressure	Deletion; low *p*	Immutable sets, e.g. per-file filters

XOR filters [Graf & Lemire 2020] and RocksDB's ribbon filters are built once for a known set, which suits immutable per-file filters. Quotient filters are another fingerprint design supporting deletes and merges.

Failure scenarios

Table 17.5 — Bloom and cuckoo filter failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
False-positive rate far above target	More items inserted than the filter was sized for	Fraction of 1 bits > 0.5; measured "maybe then not found" ratio	Rebuild a larger filter from the source data	Size for peak n with headroom; scalable (layered) filters that add a new filter when one fills
False negatives appear	Bits cleared on delete, or counting-filter delete of a non-member	Items known to exist are reported absent	Rebuild from source	Never clear bits; delete only confirmed members
Disk reads spike after restart or memory pressure	Filters evicted from RAM or not yet loaded	Block-cache misses; read latency up	Pin filter blocks in memory	Budget filter memory explicitly (≈ 1.2 bytes/key at 1%)
Union gives garbage	Filters built with different m, k or hash seeds were OR-ed	Union reports nearly everything present	Rebuild with shared parameters	Version the parameters with the filter
Cuckoo insert fails in production	Table reached ~95% load	Insert error rate > 0	Allocate a larger filter and re-insert	Monitor load factor; resize at 80–90%

Common misconceptions

COMMON MISTAKE

"A Bloom filter tells me an item is in the set." Only *probably*; the useful answer is the negative one. Acting on a positive without confirmation means being wrong p of the time.

COMMON MISTAKE

"Bigger items need bigger filters." Filter size depends only on the number of items and the target false-positive rate, not on item size. A 2 KB URL and an 8-byte integer each cost 9.6 bits at 1%.

17.5 HyperLogLog

17.5.1 Counting distinct items from leading zeros

DEFINITION

Definition — HyperLogLog (HLL). A probabilistic algorithm that estimates the number of distinct items (the **cardinality**) in a multiset using m small registers, with relative standard error about $1.04/\sqrt{m}$, independent of the cardinality itself [Flajolet et al. 2007].

First-principles explanation

A friend flips coins in sessions and reports only the longest opening run of heads. Five heads in a row has probability $1/32$, so they probably ran about 32 sessions; twenty heads suggests about a million.

Read each item's hash as coin flips. The chance that its first 1-bit is at position r or later is $2^{-(r-1)}$, so if the largest such position over all items is R, there were about 2^R *distinct* items; duplicates hash identically and cannot raise the maximum. One maximum is very noisy, so HLL keeps many and averages them.

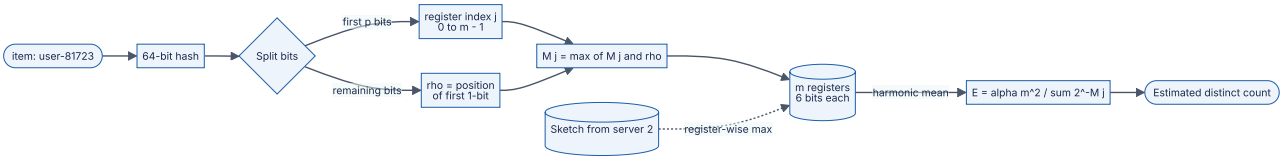
Why does this exist?

Exact distinct counting needs memory proportional to the distinct items: 1 billion 8-byte user IDs is at least 8 GB, per page, per day, per country. Summing per-server distinct counts double-counts users seen by two servers; HLL sketches merge without double counting.

How it works internally

1. Choose precision p; allocate $m = 2^p$ registers, all 0.
2. For each item, compute a 64-bit hash. The first p bits choose register j. In the remaining $64 - p$ bits, find ρ, the position of the leftmost 1-bit (1 if the first bit is 1).
3. Set $M[j] = \max(M[j], \rho)$.
4. Estimate with a bias-corrected **harmonic mean** of $2^{M[j]}$, which damps a few unusually large registers.
5. For small cardinalities, when many registers are still 0, switch to **linear counting**: $m \ln(m/V)$, where V is the number of zero registers.
6. **Merge** sketches by register-wise maximum, which equals the sketch of the combined stream.

Figure 17.5 — The HyperLogLog update and estimate path.



Reading the diagram. Each item is hashed once; the leading p bits pick a register and the first-1-bit position of the rest is the observation, kept only if it exceeds the register's maximum, so memory never grows. Updates are synchronous constant-time memory operations. The dotted edge is the asynchronous merge path: sketches from other servers or hours combine by register-wise max in a batch or query layer, with the same accuracy as one sketch. Failure points: a poor hash breaks the coin-flip assumption, and merging

sketches with different hashes or precisions gives nonsense. Scale by sharding the stream freely and merging later.

Mathematical foundations

FORMULA

HyperLogLog estimate and standard error

$$E = \alpha_m m^2 \left(\sum_{j=1}^{m} 2^{-M[j]} \right)^{-1}, \qquad \alpha_m \approx \frac{0.7213}{1 + 1.079/m}, \qquad \text{SE} \approx \frac{1.04}{\sqrt{m}}, \qquad \text{memory} = m \times 6 \text{ bits}$$

Symbol	Meaning	Units
E	Estimated number of distinct items	items
m	Number of registers, $m = 2^p$	registers
$M[j]$	Value of register j: the largest first-1-bit position seen	count
α_m	Bias-correction constant (valid for $m \geq 128$)	dimensionless
SE	Relative standard error of E	fraction
6 bits	Register width: enough to hold values up to 63, sufficient for a 64-bit hash	bits

WORKED EXAMPLE

Worked example — Redis's HyperLogLog. Redis uses $p = 14$.

1. Registers: $m = 2^{14} = 16,384$.
2. Standard error: $1.04/\sqrt{16,384} = 1.04/128 = 0.0081$, i.e. 0.81%.
3. Memory: $16,384 \times 6 = 98,304$ bits $= 12,288$ bytes $= 12$ KiB.
4. For 1 billion distinct users, one standard error is ±8.1 million, and about 95% of estimates fall within two standard errors, ±16.3 million.
5. An exact set of 1 billion 8-byte IDs needs at least 8 GB, about 650,000 times more memory.
6. Daily uniques for 1 million pages: about 12.3 GB dense, far less with the **sparse representation** most small sketches use.

Interpretation. Accuracy depends only on m; quadrupling memory halves the error ($p = 16$: 48 KiB, 0.41%). *Engineering implication.* 0.81% is invisible on dashboards; billing needs exact counts.

HyperLogLog++ [Heule et al. 2013], used by most production systems, adds a 64-bit hash (removing the large-range correction of the 32-bit original), empirical bias correction below about $5m$, and a sparse, higher-precision representation for small sets.

Code example

```
        import hashlib
        import math

        class HyperLogLog:
            """HyperLogLog (Flajolet et al. 2007) with 2^p registers and a 64-bit hash.
            Includes the linear-counting correction for small cardinalities."""

            def __init__(self, p: int = 14, salt: bytes = b""):
                self.p, self.m = p, 1 << p
                self.registers = bytearray(self.m)
                self.salt = salt
                self.alpha = 0.7213 / (1 + 1.079 / self.m)        # bias constant for m
>= 128

            def add(self, item: str) -> None:
                x = int.from_bytes(hashlib.blake2b(item.encode(), digest_size=8,
salt=self.salt).digest(), "big")
                j = x >> (64 - self.p)                            # first p bits choose
the register
                w = x & ((1 << (64 - self.p)) - 1)                # remaining 64 - p
bits
                rank = (64 - self.p) - w.bit_length() + 1         # position of the
leftmost 1-bit
                if rank > self.registers[j]:
                    self.registers[j] = rank

            def count(self) -> float:
                z = sum(2.0 ** -r for r in self.registers)
                estimate = self.alpha * self.m * self.m / z       # normalized harmonic
mean
                zeros = self.registers.count(0)
                if estimate <= 2.5 * self.m and zeros:            # small range: linear
counting
                    estimate = self.m * math.log(self.m / zeros)
                return estimate

            def merge(self, other: "HyperLogLog") -> None:
                self.registers = bytearray(max(a, b) for a, b in zip(self.registers,
other.registers))

        n, trials = 100_000, 50
        print(f"{'p':>3} {'registers':>9} {'memory':>9} {'predicted SE':>12} {'measured
RMS err':>16}")
        for p in (10, 12, 14):
            errors = []
            for t in range(trials):
                h = HyperLogLog(p, salt=t.to_bytes(4, "little"))
                for i in range(n):
                    h.add(f"user-{i}")
                errors.append((h.count() - n) / n)
            rms = math.sqrt(sum(e * e for e in errors) / trials)
            mem = h.m * 6 / 8                                     # 6-bit registers, as
in Redis
            print(f"{p:3d} {h.m:9d} {mem:7.0f} B {1.04 / math.sqrt(h.m):12.4f}
```

```
{rms:16.4f}")

        # Mergeability: two days of visitors with 50,000 in common
        day1, day2 = HyperLogLog(14), HyperLogLog(14)
        for i in range(0, 150_000):
            day1.add(f"user-{i}")
        for i in range(100_000, 250_000):
            day2.add(f"user-{i}")
        day1.merge(day2)
        print(f"union estimate {day1.count():,.0f} (true 250,000)")
```

Output:

```
  p registers    memory predicted SE measured RMS err
 10      1024     768 B       0.0325           0.0383
 12      4096    3072 B       0.0163           0.0162
 14     16384   12288 B       0.0081           0.0073
union estimate 248,111 (true 250,000)
```

What to notice:

- Each trial uses a different salt (an independent hash), so the RMS error over trials estimates the standard error, itself uncertain by about ±10% with 50 trials. Rows $p = 12$ and 14 match; $p = 10$ is 18% high, and a 200-trial run at $n = 20,000$ measured 0.0330 against 0.0325.
- The sketch for 100,000 distinct users is 12 KB whether the stream has 100,000 events or 100 billion.
- The merged sketch estimates the two-day union at 248,111, an error of −0.76%, without double-counting the 50,000 users present on both days.
- `salt` is a keyed BLAKE2b parameter. Every sketch that you intend to merge must use the same hash and salt.

INDUSTRY EXAMPLE

Redis **`PFADD`, `PFCOUNT`, `PFMERGE`.** Redis stores an HLL as a string of 16,384 6-bit registers (12 KB) with a sparse encoding for small sets, and documents a 0.81% standard error [Redis Docs: HyperLogLog; Sanfilippo 2014]. A typical pattern is one key per page per day, merged with `PFMERGE` into weekly keys. Google BigQuery exposes HyperLogLog++ through `HLL_COUNT` functions so partial sketches can be stored and merged later [Google Cloud Docs: BigQuery HLL functions]. *Evidence: Primary* — [Redis Docs: HyperLogLog; Google Cloud Docs]

Failure scenarios and misconceptions

What goes wrong	Why	Detection	Recovery	Prevention
Intersection estimate is meaningless	Computed as $\|A\| + \|B\| - \|A \cup B\|$; errors scale with the large sets	Negative or wildly varying estimates	Use exact counts or MinHash-based estimates for intersections	Only rely on HLL for unions and single-set counts
Merged count is far too high	Sketches built with different hashes, salts or precisions	Union exceeds sum of parts	Rebuild from raw data	Version the hash and p with the sketch
Small counts are biased	Raw estimator range below ~$5m$	Systematic error in tests with known small sets	Use HLL++ bias correction or linear counting	Use a library that implements HLL++

COMMON MISTAKE

"HLL counts events." It counts *distinct* items; adding a user a million times changes nothing. For per-item frequencies use Count-Min (§17.6).

COMMON MISTAKE

"Intersections work like unions." With two 10-million-user sets overlapping by 100,000, each count's standard error at $p = 14$ is about 81,000, so the inclusion–exclusion estimate's error exceeds the intersection itself.

17.6 Count-Min sketch and heavy hitters

17.6.1 The Count-Min sketch

DEFINITION

Definition — Count-Min sketch. A $d \times w$ array of counters with one hash function per row that estimates the frequency of any item in a stream. Each update increments one counter per row; each query returns the minimum of the item's d counters. The estimate never underestimates and, with probability at least $1 - \delta$, overestimates by at most εN, where N is the total of all updates [Cormode & Muthukrishnan 2005].

First-principles explanation

Imagine counting votes on a few small tally sheets, each assigning every candidate to one shared box by a different rule. Every box overcounts, but a candidate's box-mates differ between sheets, so the least-inflated box, the minimum, is close to the truth. More boxes

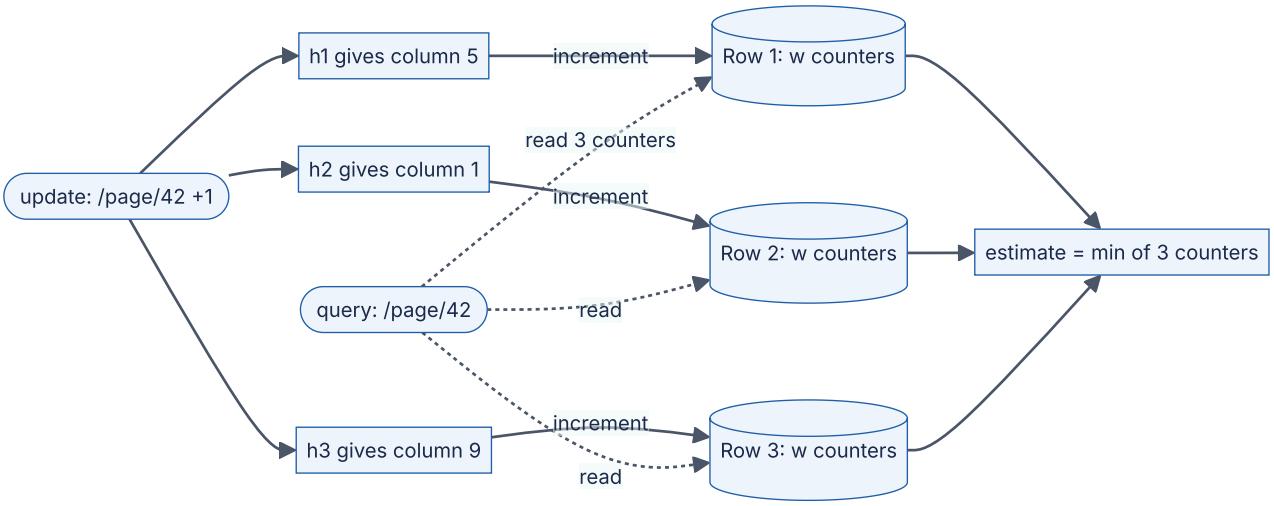
per sheet reduce inflation; more sheets reduce the chance that every box is badly inflated.

Why does this exist?

Per-item counts over millions of distinct keys (requests per IP, plays per song) need a hash map proportional to the key count, and attackers can create unbounded keys. The sketch has a fixed size, merges by adding arrays (same hashes) and carries a guaranteed bound.

Architecture diagram

Figure 17.6 — A Count-Min sketch with d = 3 rows.



Reading the diagram. An update increments one counter per row, *d* memory writes regardless of how many distinct items exist. A query (dotted, typically from a separate reader such as a rate limiter) reads the same *d* counters and returns the minimum; counters are shared storage cells, so estimates only err upward. Partitions keep their own sketches with identical hashes and a merger adds the arrays. Failure points: counter overflow (use 64-bit counters), merging sketches with different seeds, and error growing with *N* in a sketch that is never reset or decayed.

Mathematical foundations

FORMULA

Count-Min sizing and error bound

$$w = \lceil \frac{e}{\varepsilon} \rceil, \qquad d = \lceil \ln \frac{1}{\delta} \rceil, \qquad f_x \le \hat{f}_x \le f_x + \varepsilon N \text{ with probability } \ge 1 - \delta$$

Symbol	Meaning	Units
w	Counters per row (width)	counters
d	Rows, one hash function each (depth)	rows
ε	Error as a fraction of the stream total	fraction
δ	Probability that the error bound is exceeded for a given query	probability
N	Sum of all updates in the stream	events
f_x, $\hat{f}_x$	True and estimated count of item x	events
e	Euler's number, ≈ 2.718	dimensionless

WORKED EXAMPLE

Worked example — sizing a per-URL request counter. You want errors of at most 0.1% of traffic ($\varepsilon = 0.001$) with 99% confidence ($\delta = 0.01$).

1. Width: $w = \lceil 2.718/0.001 \rceil = 2,719$.
2. Depth: $d = \lceil \ln 100 \rceil = \lceil 4.605 \rceil = 5$.
3. Counters: $2,719 \times 5 = 13,595$; with 4-byte counters, 54.4 KB.
4. For a stream of $N = 10^6$ requests, each estimate is within $+1,000$ of the truth with probability 99%.
5. A day of $N = 10^{10}$ requests gives a bound of 10^7: a URL with 50,000 requests is lost in noise; one with 10^8 is within 10%.

Interpretation. The error is additive and proportional to the *whole stream*: accurate for heavy items, useless for rare ones. *Engineering implication.* Use it for heavy hitters, abusive-key rate limiting and frequency-based cache admission (TinyLFU, §20.9), and reset or decay it per window.

Conservative update increments only counters equal to the current minimum, reducing overestimation substantially while keeping the no-underestimate guarantee; such sketches no longer merge exactly by addition.

17.6.2 Heavy hitters and top-k

Heavy hitters exceed a frequency threshold (say 0.1% of traffic); **top-k** asks for the k most frequent items.

1. **Count-Min plus a min-heap.** After each update, if the item's estimate exceeds the heap minimum, replace that minimum. Memory: the sketch plus k entries.
2. **Space-Saving** [Metwally et al. 2005]. Keep k (item, count) counters. Increment a monitored item; otherwise take a free counter, or evict the smallest count $c_{\min}$ and give the newcomer $c_{\min} + 1$, an upper bound on its unseen occurrences.

FORMULA

Space-Saving guarantee

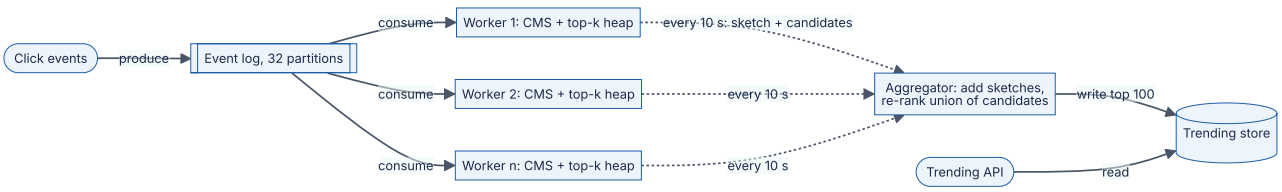
$$c_x - f_x \le c_{\min} \le \frac{N}{k}, \qquad f_x > \frac{N}{k} \Rightarrow x \text{ is monitored}$$

Symbol	Meaning	Units
k	Number of counters	counters
N	Stream length	events
c_x, f_x	Reported and true count of monitored item x	events
$c_{\min}$	Smallest counter value	events

WORKED EXAMPLE

Worked example — 100 counters over 1 million events. $N/k = 10^6/100 = 10,000$: every URL with over 10,000 requests (1% of traffic) is guaranteed monitored, and reported counts overestimate by at most 10,000. *Interpretation.* Error is relative to the stream; memory is k entries. *Engineering implication.* For a reliable top 10, monitor several times more than 10 counters so ranks 11–100 absorb churn.

Figure 17.7 — A distributed top-k pipeline.



Reading the diagram. Workers consume partitions of the event log (Chapter 30), each keeping a local Count-Min sketch and candidate heap. Every 10 seconds they ship sketches and candidates asynchronously (dotted) to an aggregator, which adds the

sketches, re-estimates candidates and writes the global top 100 to a store the API reads synchronously. An item moderately frequent everywhere can miss every local top-k; ship more candidates (top 1,000 for a global top 100) or partition the log by item key. Failure points: slow workers delay windows (event-time windows with lateness bounds, §29.8), and the aggregator must restart from the log. Scale by adding workers and partitions.

Code example

```
        import hashlib
        import heapq
        import math
        import random
        from collections import Counter

        class CountMinSketch:
            """Count-Min sketch (Cormode & Muthukrishnan 2005): d rows x w counters."""

            def __init__(self, epsilon: float, delta: float):
                self.w = math.ceil(math.e / epsilon)              # width from accuracy
                self.d = math.ceil(math.log(1 / delta))           # depth from confi-
dence
                self.table = [[0] * self.w for _ in range(self.d)]
                self.total = 0

            def _columns(self, item: str):
                digest = hashlib.blake2b(item.encode(), digest_size=16).digest()
                h1 = int.from_bytes(digest[:8], "little")
                h2 = int.from_bytes(digest[8:], "little")
                return [(h1 + r * h2) % self.w for r in range(self.d)]

            def add(self, item: str, count: int = 1) -> None:
                self.total += count
                for row, col in enumerate(self._columns(item)):
                    self.table[row][col] += count

            def estimate(self, item: str) -> int:
                return min(self.table[row][col] for row, col in
enumerate(self._columns(item)))

        class TopK:
            """Heavy hitters: a Count-Min sketch plus a min-heap of the k largest esti-
mates."""

            def __init__(self, k: int, epsilon: float, delta: float):
                self.k, self.cms = k, CountMinSketch(epsilon, delta)
                self.heap, self.members = [], {}                  # heap of (estimate,
item)

            def add(self, item: str) -> None:
                self.cms.add(item)
                est = self.cms.estimate(item)
                if item in self.members:
                    self.members[item] = est                      # lazily refreshed
below
                elif len(self.heap) < self.k:
                    self.members[item] = est
                    heapq.heappush(self.heap, (est, item))
                elif est > self.heap[0][0]:
                    # refresh stale heap entries before deciding whom to evict
                    self.heap = [(self.members[i], i) for _, i in self.heap]
                    heapq.heapify(self.heap)
                    if est > self.heap[0][0]:
```

```
                        _, evicted = heapq.heapreplace(self.heap, (est, item))
                        del self.members[evicted]
                        self.members[item] = est

            def top(self):
                return sorted(((c, i) for i, c in self.members.items()), reverse=True)

        def space_saving(stream, k):
            """Metwally et al. 2005: k counters; the newcomer replaces the minimum and
inherits its count."""
            counters = {}
            for item in stream:
                if item in counters:
                    counters[item] += 1
                elif len(counters) < k:
                    counters[item] = 1
                else:
                    victim = min(counters, key=counters.get)      # O(k); production
uses a stream-summary list
                    counters[item] = counters.pop(victim) + 1
            return sorted(((c, i) for i, c in counters.items()), reverse=True)

        # A Zipf-like stream: 1,000,000 events over 100,000 distinct URLs (exponent 1.1)
        rng = random.Random(1)
        universe = 100_000
        weights = [1 / (r ** 1.1) for r in range(1, universe + 1)]
        stream = [f"/page/{r}" for r in rng.choices(range(1, universe + 1),
weights=weights, k=1_000_000)]
        truth = Counter(stream)

        eps, delta = 0.001, 0.01
        cms = CountMinSketch(eps, delta)
        for item in stream:
            cms.add(item)
        N = cms.total
        errors = [cms.estimate(x) - c for x, c in truth.items()]
        assert min(errors) >= 0                                    # never underesti-
mates
        print(f"w = {cms.w}, d = {cms.d}, counters = {cms.w * cms.d:,} vs distinct items
= {len(truth):,}")
        print(f"error bound eps*N = {eps * N:,.0f}; max error = {max(errors)}; "
              f"mean error = {sum(errors) / len(errors):.1f}; "
              f"items over bound = {sum(e > eps * N for e in errors)} of {len(er-
rors):,} (delta allows {delta:.0%})")

        tk = TopK(10, eps, delta)
        for item in stream:
            tk.add(item)
        true_top = [i for i, _ in truth.most_common(10)]
        print("CMS+heap top-10 recall:", len(set(i for _, i in tk.top()) &
set(true_top)) / 10)
        ss = space_saving(stream, 100)
        print("Space-Saving (100 counters) top-10 recall:", len(set(i for _, i in
ss[:10]) & set(true_top)) / 10)
        print("true top-3:", truth.most_common(3))
        print("Space-Saving top-3:", [(i, c) for c, i in ss[:3]])
```

Output:

```
w = 2719, d = 5, counters = 13,595 vs distinct items = 64,589
error bound eps*N = 1,000; max error = 2474; mean error = 61.1; items over bound = 1 of
64,589 (delta allows 1%)
CMS+heap top-10 recall: 1.0
Space-Saving (100 counters) top-10 recall: 1.0
true top-3: [('/page/1', 134738), ('/page/2', 63178), ('/page/3', 40195)]
Space-Saving top-3: [('/page/1', 134738), ('/page/2', 63178), ('/page/3', 40195)]
```

What to notice:

- Width 2,719 and depth 5 reproduce the worked example. The `assert` confirms that no estimate is below the truth.
- Only 1 of 64,589 items exceeded the $\varepsilon N = 1,000$ bound, far below the 1% that δ allows; the mean overestimate is 61.
- Both methods recover the true top 10 on this skewed (Zipf) stream, typical of real traffic; on a near-uniform stream there are no heavy hitters to find.
- Space-Saving's top three counts are exact because those items were never evicted. Production code uses a "stream-summary" structure for $O(1)$ eviction.

INDUSTRY EXAMPLE

Twitter Algebird and RedisBloom. Twitter's open-source Algebird library provides mergeable Count-Min sketches, HyperLogLog and Bloom filters for Hadoop, Scalding and Storm jobs: each partition builds a sketch and the framework adds them [Twitter: Algebird]. The RedisBloom module (in Redis Stack, and in Redis Open Source from Redis 8, as of 2025) provides `CMS.INCRBY` / `CMS.QUERY` and `TOPK.ADD` / `TOPK.LIST` [Redis Docs: Count-min sketch; Top-K]. *Evidence: Primary* — [Twitter: Algebird; Redis Docs]

COMMON MISTAKE

"Count-Min gives each item's count within ε percent." The bound is εN, a fraction of the *entire stream*; a rare item can be overestimated 100-fold. Report estimates only for items well above εN.

17.7 Skip lists

17.7.1 Probabilistic levels instead of rebalancing

DEFINITION

Definition — Skip list. An ordered linked list augmented with extra levels of "express" pointers. Each node is promoted to the next level with probability p (typically 1/2 or 1/4), so level i contains about a fraction p^i of the nodes. Search, insert and delete take expected $O(\log n)$ time [Pugh 1990].

First-principles explanation

A subway has local trains stopping everywhere and express trains stopping at every fourth station: ride the express to the last stop before your destination, then switch to the local. A skip list stacks several express lines and picks each station's express status by coin flip, so nothing needs rebalancing as stations are added.

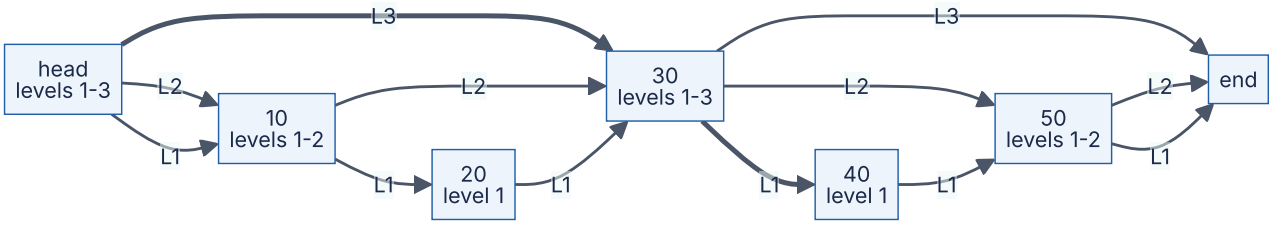
Why does this exist?

Balanced trees (red–black, AVL) guarantee $O(\log n)$, but their rotations are intricate and hard to make concurrent. Pugh showed that randomization gives the same *expected* cost with a far simpler algorithm: an insert changes only its predecessors' pointers [Pugh 1990].

How it works internally

1. **Search** for key x: start at the head's top level. Move right while the next key is less than x; when it is not, drop down a level. At level 0, the next node is x if x is present.
2. **Insert** x: search as above, remembering the last node visited on each level (the update path). Draw a random level ℓ: start at 1 and add a level while a coin with probability p comes up heads. Splice the new node in after the remembered node on each of levels $1..\ell$.
3. **Delete**: search, then unlink the node from each level it occupies.
4. **Range scan** $[a, b]$: search for a, then walk level 0 until passing b. This is why skip lists suit ordered data such as leaderboards and memtables.

Figure 17.8 — A skip list with three levels; the bold path finds 40.



Reading the diagram. Each box is one node, labeled with the levels it occupies; each arrow is a forward pointer labeled with its level (L1 links every node, L2 about a quarter to a half of them, L3 fewer still). Searching for 40 follows the bold path: at level 3 the head jumps straight to 30; the next L3 and L2 pointers from 30 overshoot (end, 50), so the search drops to level 1 and steps once to 40, two moves instead of four along the bottom list, a saving that grows with n. Everything is in memory and every step is a synchronous pointer read. Failure points: unlucky coin flips can slow one search (the guarantee is probabilistic), and pointer chasing causes CPU cache misses (§5.4), which is why on-disk indexes prefer B-trees. Concurrency scales well because an insert changes only a few predecessor pointers.

Mathematical foundations

FORMULA

Skip list levels, search cost and memory

$$L(n) = \log_{1/p} n, \qquad \mathbb{E}[\text{search path}] \le \frac{L(n)}{p} + \frac{1}{1-p}, \qquad \mathbb{E}[\text{pointers per node}] = \frac{1}{1-p}$$

Symbol	Meaning	Units
n	Number of elements	elements
p	Probability of promoting a node to the next level	probability
$L(n)$	Expected number of levels in use	levels
Search path	Forward moves plus level drops during one search	steps
Pointers per node	Average forward pointers stored per element	pointers

WORKED EXAMPLE

Worked example — choosing p for one million elements.

1. $p = 1/2$: $L = \log_2 10^6 = 19.9$ levels; search path $\leq 19.9/0.5 + 1/0.5 = 41.9$ steps; 2.00 pointers per node.
2. $p = 1/4$: $L = \log_4 10^6 = 6/\log_{10} 4 = 6/0.602 = 9.97$ levels; search path $\leq 9.97/0.25 + 1/0.75 = 41.2$ steps; 1.33 pointers per node.

Interpretation. Both give about 40 steps, but $p = 1/4$ stores a third fewer pointers.
Engineering implication. That is why Redis uses $p = 1/4$.

Code example and measured validation

```
        import math
        import random

        class Node:
            __slots__ = ("key", "forward")
            def __init__(self, key, level):
                self.key, self.forward = key, [None] * level

        class SkipList:
            """Pugh's skip list with promotion probability p (Redis uses p = 0.25, max
level 32)."""

            def __init__(self, p: float = 0.25, max_level: int = 32, seed: int = 0):
                self.p, self.max_level = p, max_level
                self.head = Node(None, max_level)
                self.level = 1
                self.pointers = 0                                # total forward point-
ers allocated
                self.rng = random.Random(seed)

            def _random_level(self) -> int:
                lvl = 1
                while self.rng.random() < self.p and lvl < self.max_level:
                    lvl += 1
                return lvl

            def insert(self, key) -> None:
                update, x = [self.head] * self.max_level, self.head
                for i in reversed(range(self.level)):            # descend, remembering
the last node per level
                    while x.forward[i] and x.forward[i].key < key:
                        x = x.forward[i]
                    update[i] = x
                lvl = self._random_level()
                self.level = max(self.level, lvl)
                node = Node(key, lvl)
                self.pointers += lvl
                for i in range(lvl):                             # splice in at each of
its levels
                    node.forward[i], update[i].forward[i] = update[i].forward[i], node

            def search(self, key):
                """Return (found, number of key comparisons)."""
                x, steps = self.head, 0
                for i in reversed(range(self.level)):
                    while x.forward[i] and x.forward[i].key < key:
                        x, steps = x.forward[i], steps + 1
                    steps += 1                                   # the comparison that
stopped us
                x = x.forward[0]
                return (x is not None and x.key == key), steps

            def range(self, lo, hi):
                """All keys in [lo, hi]: find lo in O(log n), then walk level 0."""
```

```
                x = self.head
                for i in reversed(range(self.level)):
                    while x.forward[i] and x.forward[i].key < lo:
                        x = x.forward[i]
                x, out = x.forward[0], []
                while x and x.key <= hi:
                    out.append(x.key)
                    x = x.forward[0]
                return out

        for p in (0.5, 0.25):
            for n in (10_000, 100_000, 1_000_000):
                sl = SkipList(p=p, seed=42)
                keys = list(range(n))
                random.Random(1).shuffle(keys)
                for k in keys:
                    sl.insert(k)
                probe = random.Random(2).sample(range(n), 2_000)
                avg = sum(sl.search(k)[1] for k in probe) / len(probe)
                L = math.log(n, 1 / p)
                bound = L / p + 1 / (1 - p)                    # Pugh's expected-cost
bound
                print(f"p={p:<4} n={n:>9,}  levels={sl.level:2d}  log_1/p(n)={L:5.1f}  "
                      f"avg comparisons={avg:5.1f}  Pugh bound={bound:5.1f}  "
                      f"pointers/node={sl.pointers / n:4.2f} (expect {1 / (1 -
p):4.2f})")
        print("range(500, 505):", sl.range(500, 505))
```

Output (about 40 s in CPython):

```
p=0.5  n=   10,000  levels=14  log_1/p(n)= 13.3  avg comparisons= 27.5  Pugh bound= 28.6
pointers/node=1.98 (expect 2.00)
p=0.5  n=  100,000  levels=17  log_1/p(n)= 16.6  avg comparisons= 32.7  Pugh bound= 35.2
pointers/node=2.00 (expect 2.00)
p=0.5  n=1,000,000  levels=20  log_1/p(n)= 19.9  avg comparisons= 38.4  Pugh bound= 41.9
pointers/node=2.00 (expect 2.00)
p=0.25 n=   10,000  levels= 7  log_1/p(n)=  6.6  avg comparisons= 23.8  Pugh bound= 27.9
pointers/node=1.33 (expect 1.33)
p=0.25 n=  100,000  levels= 9  log_1/p(n)=  8.3  avg comparisons= 31.9  Pugh bound= 34.6
pointers/node=1.34 (expect 1.33)
p=0.25 n=1,000,000  levels=11  log_1/p(n)= 10.0  avg comparisons= 37.4  Pugh bound= 41.2
pointers/node=1.33 (expect 1.33)
range(500, 505): [500, 501, 502, 503, 504, 505]
```

What to notice:

- The number of levels in use tracks $\log_{1/p} n$, and the measured search path stays below Pugh's bound at every size.
- Going from 10^4 to 10^6 elements (100×) adds only about 11–14 comparisons: logarithmic growth.
- Pointers per node match $1/(1-p)$ exactly, confirming the memory saving of $p = 1/4$.

INDUSTRY EXAMPLE

Redis sorted sets and LSM memtables. A large Redis sorted set is a hash table (member → score) plus a skip list ordered by (score, member) with $p = 1/4$ and up to 32 levels; each forward pointer stores a *span* so `ZRANK` sums spans along the search path in $O(\log n)$. Small sets use a compact listpack [Redis Docs: Sorted sets; Redis source `t_zset.c`]. LevelDB and RocksDB use a skip list as the default memtable, for concurrent inserts and ordered flushing to SSTables [RocksDB Wiki: MemTable]; Java's `ConcurrentSkipListMap` is a lock-free sorted map on the same idea. *Evidence: Primary* — [Redis Docs; RocksDB Wiki]

COMMON MISTAKE

"Skip lists guarantee O(log n)." Only in expectation over the coin flips; slow searches are exponentially unlikely, unless an adversary can predict your random generator.

17.8 Merkle trees

17.8.1 Hash trees for verification and anti-entropy

DEFINITION

Definition — Merkle tree (hash tree). A tree in which every leaf holds the hash of a data block and every interior node holds the hash of its children's hashes. The **root hash** summarizes the entire data set: changing any block changes the root [Merkle 1987].

First-principles explanation

Two libraries want to confirm their 1-million-book catalogs match without reading every entry over the phone. Each writes a summary per shelf, then per aisle from shelf summaries, then per floor, then for the building. They compare building summaries; on a mismatch they compare floors, then only aisles on mismatched floors, and so on. Work grows with the number of *differences* times the tree depth, not with catalog size.

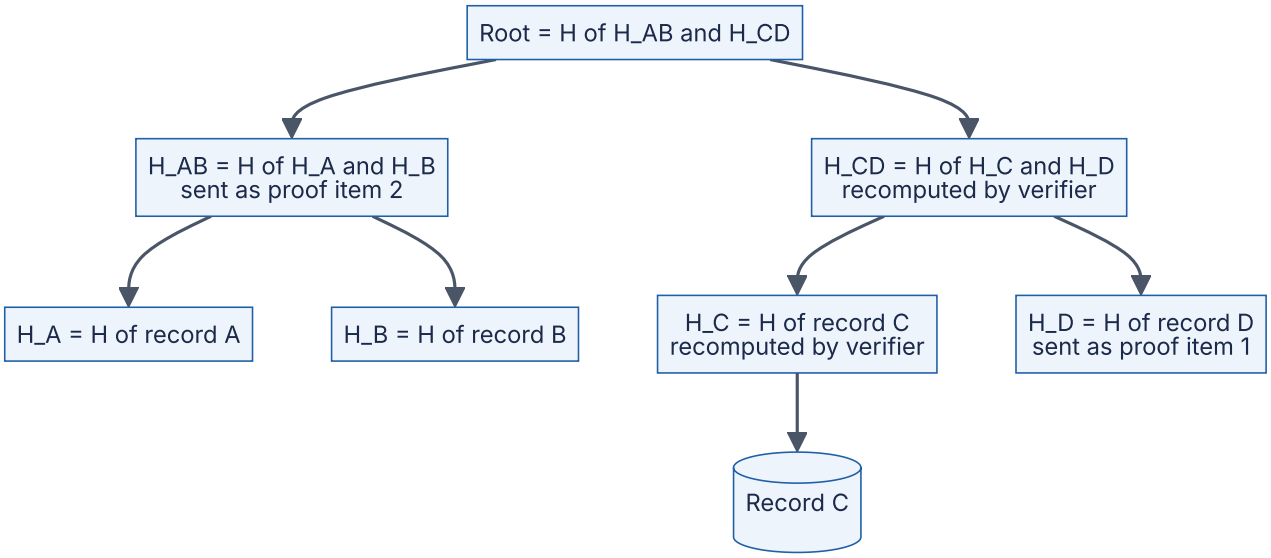
Why does this exist?

Merkle introduced hash trees so one signature over a root could authenticate many values [Merkle 1987]. Distributed systems use them for **verification** (prove a record belongs to a data set with a trusted root) and **anti-entropy** (find where replicas differ by exchanging a few hashes).

How it works internally

1. Hash each record into a leaf: a cryptographic hash such as SHA-256 if forgery matters, a cheaper one if only accidental divergence does.
2. Hash adjacent pairs level by level up to the root, prefixing leaf and interior inputs with different bytes (`0x00`, `0x01`, as in Certificate Transparency [RFC 6962]) so interior nodes cannot pose as leaves.
3. **Inclusion proof** for record *i*: the sibling hash at each level on the path from leaf *i* to the root. A verifier recomputes the root from the record and the siblings and compares it with the trusted root.
4. **Anti-entropy**: two replicas compare roots; if they differ, they compare children and descend only into subtrees whose hashes differ, until they reach the divergent leaves.

Figure 17.9 — A Merkle tree over four records and the inclusion proof for record C.



Reading the diagram. Leaves hash records, parents hash children, the root summarizes all. To prove C is present, the server sends C plus the two siblings marked as proof items, H_D and H_{AB}; the verifier recomputes H_C, H_{CD} and the root and compares it with a root obtained through a trusted channel (signed log head, block header), all synchronously and locally. Records live in storage; hashes can be cached or stored beside them. Failure points: a stale or untrusted root, and rebuild cost after many updates (incremental trees rehash only the changed leaf's path). Proofs grow logarithmically: 30 hashes for a billion records.

FORMULA

Merkle proof size and anti-entropy cost

$$h = \lceil \log_2 n \rceil, \qquad \text{proof size} = h \times H_{\text{bytes}}, \qquad \text{hashes compared} \le 1 + 2\,k\,h$$

Symbol	Meaning	Units
n	Number of leaves (records or key ranges)	leaves
h	Tree height	levels
H_{bytes}	Size of one hash (32 for SHA-256)	bytes
k	Number of divergent leaves between two replicas	leaves

WORKED EXAMPLE

Worked example — 1,048,576 records on two replicas.

1. Height: $h = \log_2 2^{20} = 20$.
2. Inclusion proof: $20 \times 32 = 640$ bytes, compared with sending all leaf hashes ($2^{20} \times 32$ B = 33.6 MB) or the data itself.
3. Anti-entropy with $k = 2$ divergent records: at most $1 + 2 \times 2 \times 20 = 81$ hash comparisons out of about 2.1 million tree nodes.

Interpretation. Both costs grow with $\log n$, and anti-entropy cost with the number of differences. *Engineering implication.* Replicas can run anti-entropy continuously at negligible network cost when mostly in sync.

```
        import hashlib

        def h(data: bytes) -> bytes:
            return hashlib.sha256(data).digest()

        def leaf_hash(record: bytes) -> bytes:
            return h(b"\x00" + record)          # domain separation: leaves and interior
nodes
        def node_hash(left: bytes, right: bytes) -> bytes:
            return h(b"\x01" + left + right)    # can never be confused (cf. RFC 6962)

        def build(records):
            """Return all levels, leaves first. An odd node is promoted unchanged."""
            level = [leaf_hash(r) for r in records]
            levels = [level]
            while len(level) > 1:
                nxt = [node_hash(level[i], level[i + 1]) for i in range(0, len(level) -
1, 2)]
                if len(level) % 2:
                    nxt.append(level[-1])
                levels.append(nxt)
                level = nxt
            return levels

        def proof(levels, index):
            """Sibling hashes from leaf to root: the audit path for one record."""
            path = []
            for level in levels[:-1]:
                sibling = index ^ 1
                if sibling < len(level):
                    path.append((sibling < index, level[sibling]))   # (sibling is on
the left?, hash)
                index //= 2
            return path

        def verify(record, path, root):
            acc = leaf_hash(record)
            for sibling_is_left, sib in path:
                acc = node_hash(sib, acc) if sibling_is_left else node_hash(acc, sib)
            return acc == root

        def diff(a, b, level=None, index=0, found=None):
            """Anti-entropy: descend only into subtrees whose hashes differ. Returns
(leaf indexes, hashes compared)."""
            if found is None:
                level, found = len(a) - 1, ([], [0])
            found[1][0] += 1
            if a[level][index] == b[level][index]:
                return found
            if level == 0:
                found[0].append(index)
                return found
            for child in (2 * index, 2 * index + 1):
                if child < len(a[level - 1]):
                    diff(a, b, level - 1, child, found)
            return found
```

```
        records = [f"key-{i}=value-{i}".encode() for i in range(1 << 20)]       #
1,048,576 records
        replica_a = build(records)
        records_b = list(records)
        for i in (12_345, 777_777):                                              # two
divergent records
            records_b[i] = b"stale"
        replica_b = build(records_b)

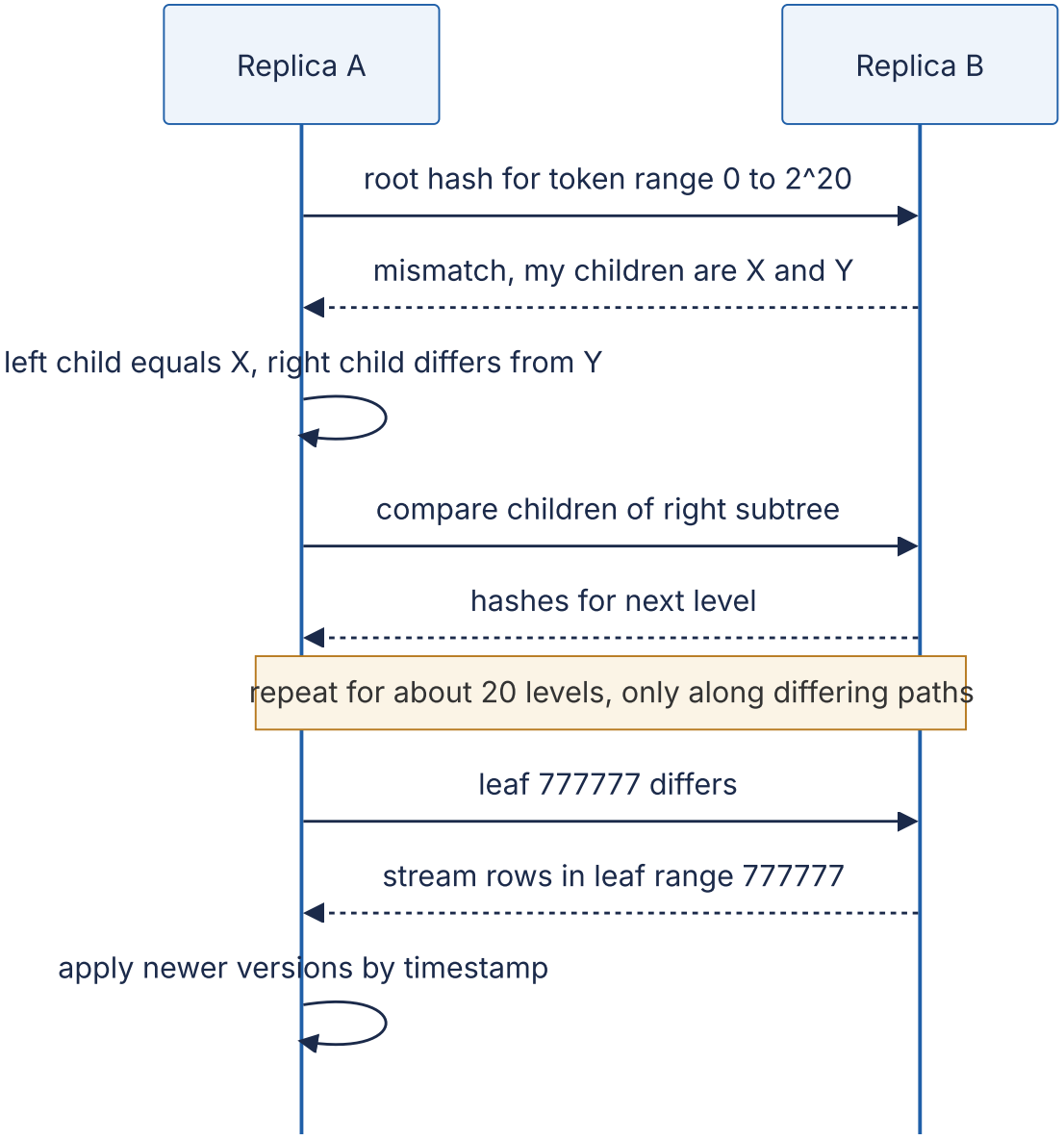
        root = replica_a[-1][0]
        p = proof(replica_a, 424_242)
        print(f"tree height = {len(replica_a) - 1}, proof length = {len(p)} hashes =
{32 * len(p)} bytes")
        print("valid record verifies:", verify(records[424_242], p, root))
        print("tampered record verifies:", verify(b"key-424242=forged", p, root))
        leaves, compared = diff(replica_a, replica_b)
        print(f"roots equal: {replica_a[-1][0] == replica_b[-1][0]}; divergent leaves:
{leaves}; "
              f"hashes compared: {compared[0]} of {sum(len(l) for l in replica_a):,}")
```

Output:

```
tree height = 20, proof length = 20 hashes = 640 bytes
valid record verifies: True
tampered record verifies: False
roots equal: False; divergent leaves: [12345, 777777]; hashes compared: 79 of 2,097,151
```

What to notice: the proof is exactly 20 hashes (640 bytes); a forged record fails verification against the same proof; and the anti-entropy walk finds both divergent records after 79 comparisons, just under the bound of 81.

Figure 17.10 — Merkle-tree anti-entropy between two replicas.



Reading the diagram. Replica A starts a background repair of one key range by sending its root; B returns its children; A finds only the right subtree differs and descends there, about 20 levels along differing paths only. At the leaves the replicas stream the divergent rows and keep the newer versions (§27.14). Each message is synchronous request/response, but repair is asynchronous to client traffic. Trees are built by scanning local data, an I/O-heavy step. Failure points: tree building competes with client reads; coarse leaves cause “overstreaming” of whole ranges for one changed row; and trees built at different moments show spurious differences for in-flight writes. Systems repair ranges incrementally on a schedule.

INDUSTRY EXAMPLE

Dynamo, Cassandra, Git and Certificate Transparency. Dynamo keeps a Merkle tree per key range and uses it to repair replicas after failures [DeCandia et al. 2007]. Cassandra's `nodetool repair` builds Merkle trees over token ranges, compares them and streams mismatched ranges [Apache Cassandra Docs: Repair]. Git names every file, directory and commit by the hash of its content, so equal commit hashes imply identical histories [Chacon & Straub 2014]. Certificate Transparency logs give logarithmic proofs of inclusion and append-only growth (§10.5) [RFC 6962]. *Evidence: Primary* — [DeCandia et al. 2007; Apache Cassandra Docs; Chacon & Straub 2014; RFC 6962]

Chapter 27 (§27.13) places Merkle-tree anti-entropy alongside read repair and hinted handoff in leaderless replication.

17.9 Geospatial indexing

17.9.1 Why location needs its own index

"Find all drivers within 1 km" looks like `WHERE lat BETWEEN ... AND lon BETWEEN ...`, but a B-tree orders keys along one dimension: an index on `lat` (or `(lat, lon)`) returns a strip around the planet, filtered afterwards by longitude. The fix is a **space-filling curve**: divide the map into **cells** whose IDs are close when the cells are close, index points by cell ID, look up a few cells per query and filter candidates by exact distance.

17.9.2 Geohash

DEFINITION

Definition — Geohash. A string encoding of a latitude/longitude cell, published by Gustavo Niemeyer in 2008, formed by alternately bisecting the longitude and latitude ranges, interleaving the resulting bits (longitude first) and writing every 5 bits as one base-32 character [Niemeyer 2008]. Each additional character subdivides the cell into 32 smaller cells, so a geohash that is a prefix of another names a cell containing it.

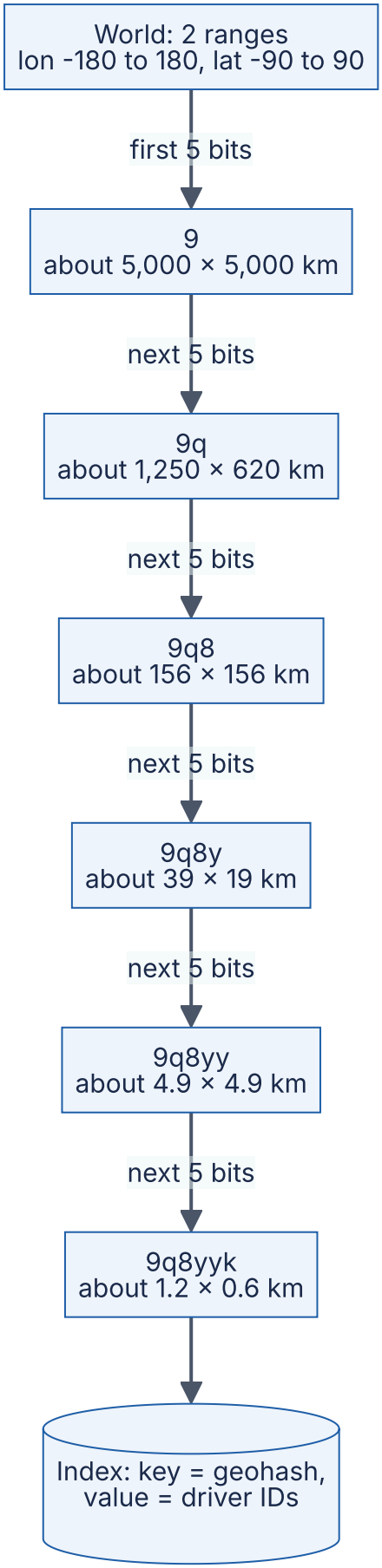
How it works internally

1. Start with longitude range $[-180, 180]$ and latitude range $[-90, 90]$.
2. Take the longitude midpoint. If the point is east of it, emit bit 1 and keep the upper half; otherwise emit 0 and keep the lower half.
3. Do the same with latitude, and keep alternating.

4. Group bits in fives and map each group to the alphabet `0123456789bcdefghjkmnpqrstuvwxyz` (the letters a, i, l and o are omitted to avoid confusion with digits).

The interleaving traces a **Z-order (Morton) curve**. A long shared prefix means nearby points, but not conversely: points a meter apart across a major boundary (the equator, the prime meridian) can share no prefix. Proximity search must always examine neighboring cells, never a prefix alone.

Figure 17.11 — Geohash refinement: each character narrows the cell.


World: 2 ranges
lon -180 to 180, lat -90 to 90
first 5 bits
9
about 5,000 × 5,000 km
next 5 bits
9q
about 1,250 × 620 km
next 5 bits
9q8
about 156 × 156 km
next 5 bits
9q8y
about 39 × 19 km
next 5 bits
9q8yy
about 4.9 × 4.9 km
next 5 bits
9q8yyk
about 1.2 × 0.6 km
Index: key = geohash,
value = driver IDs

Reading the diagram. Each arrow appends one character, shrinking the cell's area 32-fold (sizes at the equator; widths shrink by $\cos(\text{latitude})$). Because a longer geohash extends a shorter one, a range scan over keys starting `9q8yy` in any ordered store (B-tree, sorted set, LSM tree) returns every point in that 4.9 km cell: a 2-D query becomes 1-D range scans. Encoding is pure computation; the only storage is the index. The failure point is the boundary effect; the scaling decision is precision, trading cells scanned against false candidates per cell.

FORMULA

Geohash cell size

$$b_{\text{lon}} = \lceil \frac{5L}{2} \rceil, \quad b_{\text{lat}} = \lfloor \frac{5L}{2} \rfloor, \quad W = \frac{360^\circ}{2^{b_{\text{lon}}}} \times 111.32 \cos\varphi \text{ km}, \quad H = \frac{180^\circ}{2^{b_{\text{lat}}}} \times 110.57 \text{ km}$$

Symbol	Meaning	Units
L	Geohash length	characters
b_{lon}, b_{lat}	Bits spent on longitude and latitude	bits
W, H	Cell width (east–west) and height (north–south)	km
φ	Latitude of the cell	degrees
111.32, 110.57	Approximate km per degree of longitude at the equator and of latitude	km/degree

WORKED EXAMPLE

Worked example — a 6-character cell in San Francisco ($\varphi = 37.77^\circ$).

1. Bits: $5 \times 6 = 30$, so $b_{\text{lon}} = 15$, $b_{\text{lat}} = 15$.
2. Width: $360/32,768 = 0.010986^\circ$; $\times 111.32 = 1.223$ km at the equator; $\times \cos 37.77^\circ = 0.7905$ gives 0.967 km.
3. Height: $180/32,768 = 0.005493^\circ \times 110.57 = 0.607$ km.

Interpretation. The cell is about 970 m × 610 m, not square. *Engineering implication.* The 3 × 3 block around a point reaches at least 610 m in every direction: enough for a 500 m radius, **not** for 1 km. The program below measures the consequence.

Table 17.6 — Geohash precision (cell size at the equator, computed by the program below).

Length	Bits	Width	Height	Typical use
1	5	5,009 km	4,976 km	Continent
2	10	1,252 km	622 km	Large country region
3	15	157 km	156 km	Metropolitan region
4	20	39.1 km	19.4 km	City
5	25	4.9 km	4.9 km	District; 1–2 km searches
6	30	1.2 km	0.61 km	Neighborhood; ~500 m searches
7	35	153 m	152 m	Block
8	40	38 m	19 m	Building
9	45	4.8 m	4.8 m	Doorway (beyond typical GPS accuracy)
10	50	1.2 m	0.59 m	Below GPS accuracy

Code example: geohash encoding and a proximity search

```
        import math
        import random
        from collections import defaultdict

        BASE32 = "0123456789bcdefghjkmnpqrstuvwxyz"          # no a, i, l, o

        def encode(lat: float, lon: float, length: int) -> str:
            """Interleave longitude and latitude bisection bits (longitude first), 5
bits per character."""
            lat_rng, lon_rng = [-90.0, 90.0], [-180.0, 180.0]
            out, bits, bit_count, even = [], 0, 0, True
            while len(out) < length:
                rng, val = (lon_rng, lon) if even else (lat_rng, lat)
                mid = (rng[0] + rng[1]) / 2
                if val >= mid:
                    bits, rng[0] = (bits << 1) | 1, mid
                else:
                    bits, rng[1] = bits << 1, mid
                even, bit_count = not even, bit_count + 1
                if bit_count == 5:
                    out.append(BASE32[bits])
                    bits, bit_count = 0, 0
            return "".join(out)

        def decode_box(gh: str):
            """Return (lat_min, lat_max, lon_min, lon_max) of a geohash cell."""
            lat_rng, lon_rng, even = [-90.0, 90.0], [-180.0, 180.0], True
            for ch in gh:
                v = BASE32.index(ch)
                for shift in range(4, -1, -1):
                    rng = lon_rng if even else lat_rng
                    mid = (rng[0] + rng[1]) / 2
                    if (v >> shift) & 1:
                        rng[0] = mid
                    else:
                        rng[1] = mid
                    even = not even
            return lat_rng[0], lat_rng[1], lon_rng[0], lon_rng[1]

        def neighbors(gh: str):
            """The 8 surrounding cells (center-offset method; ignores the poles and the
antimeridian)."""
            la0, la1, lo0, lo1 = decode_box(gh)
            dlat, dlon = la1 - la0, lo1 - lo0
            clat, clon = (la0 + la1) / 2, (lo0 + lo1) / 2
            return [encode(clat + i * dlat, clon + j * dlon, len(gh))
                    for i in (-1, 0, 1) for j in (-1, 0, 1) if (i, j) != (0, 0)]

        def covering(lat: float, lon: float, radius_km: float, length: int):
            """All cells of this length that intersect the circle's bounding box (exact
recall)."""
            la0, la1, lo0, lo1 = decode_box(encode(lat, lon, length))
            dlat, dlon = la1 - la0, lo1 - lo0
            ri = math.ceil(radius_km / (dlat * 111.0))            # rows needed above
```

```
and below
            rj = math.ceil(radius_km / (dlon * 111.0 * math.cos(math.radians(lat))))
            clat, clon = (la0 + la1) / 2, (lo0 + lo1) / 2
            return [encode(clat + i * dlat, clon + j * dlon, length)
                    for i in range(-ri, ri + 1) for j in range(-rj, rj + 1)]

        def haversine_km(lat1, lon1, lat2, lon2):
            r = 6371.0088
            p1, p2 = math.radians(lat1), math.radians(lat2)
            dp, dl = p2 - p1, math.radians(lon2 - lon1)
            a = math.sin(dp / 2) ** 2 + math.cos(p1) * math.cos(p2) * math.sin(dl / 2)
** 2
            return 2 * r * math.asin(math.sqrt(a))

        print("encode(57.64911, 10.40744, 11) =", encode(57.64911, 10.40744, 11))   #
reference: u4pruydqqvj

        print(f"{'len':>3} {'bits':>4} {'cell width (equator)':>21} {'cell
height':>12}")
        for length in range(1, 11):
            lon_bits = math.ceil(5 * length / 2)
            lat_bits = 5 * length // 2
            w_km = 360 / 2 ** lon_bits * 111.32
            h_km = 180 / 2 ** lat_bits * 110.57
            fmt = lambda km: f"{km:,.1f} km" if km >= 1 else f"{km * 1000:,.2f} m"
            print(f"{length:3d} {5 * length:4d} {fmt(w_km):>21} {fmt(h_km):>12}")

        # Proximity search: 200,000 drivers around San Francisco; find those within 1 km
of a rider
        rng = random.Random(3)
        drivers = [(37.70 + rng.random() * 0.12, -122.52 + rng.random() * 0.14) for _
in range(200_000)]
        rider = (37.7749, -122.4194)
        truth = {i for i, d in enumerate(drivers) if haversine_km(*rider, *d) <= 1.0}
        for P, method in ((6, "3x3"), (5, "3x3"), (6, "cover")):
            index = defaultdict(list)                       # geohash -> driver ids (in
production: a sorted key range)
            for i, (lat, lon) in enumerate(drivers):
                index[encode(lat, lon, P)].append(i)
            cell = encode(*rider, P)
            cells = [cell] + neighbors(cell) if method == "3x3" else covering(*rider,
1.0, P)
            candidates = [i for c in cells for i in index[c]]
            hits = {i for i in candidates if haversine_km(*rider, *drivers[i]) <= 1.0}
            la0, la1, lo0, lo1 = decode_box(cell)
            print(f"P={P} {method:5s} cell {cell}: {haversine_km(la0, lo0, la0, lo1) *
1000:,.0f} m x "
                  f"{haversine_km(la0, lo0, la1, lo0) * 1000:,.0f} m; {len(cells)} cells
-> "
                  f"{len(candidates):,} candidates, {len(hits):,} within 1 km, "
                  f"recall {len(hits) / len(truth):.3f} (brute force finds
{len(truth):,})")
```

Output:

```
encode(57.64911, 10.40744, 11) = u4pruydqqvj
len bits  cell width (equator)  cell height
  1    5            5,009.4 km   4,975.6 km
  2   10            1,252.3 km     622.0 km
  3   15              156.5 km     155.5 km
  4   20               39.1 km      19.4 km
  5   25                4.9 km       4.9 km
  6   30                1.2 km     607.38 m
  7   35              152.87 m     151.84 m
  8   40               38.22 m      18.98 m
  9   45                4.78 m       4.75 m
 10   50                1.19 m       0.59 m
P=6 3x3   cell 9q8yyk: 966 m x 611 m; 9 cells -> 6,470 candidates, 3,601 within 1 km,
recall 0.942 (brute force finds 3,822)
P=5 3x3   cell 9q8yy: 3,864 m x 4,887 m; 9 cells -> 130,946 candidates, 3,822 within 1
km, recall 1.000 (brute force finds 3,822)
P=6 cover cell 9q8yyk: 966 m x 611 m; 25 cells -> 17,908 candidates, 3,822 within 1 km,
recall 1.000 (brute force finds 3,822)
```

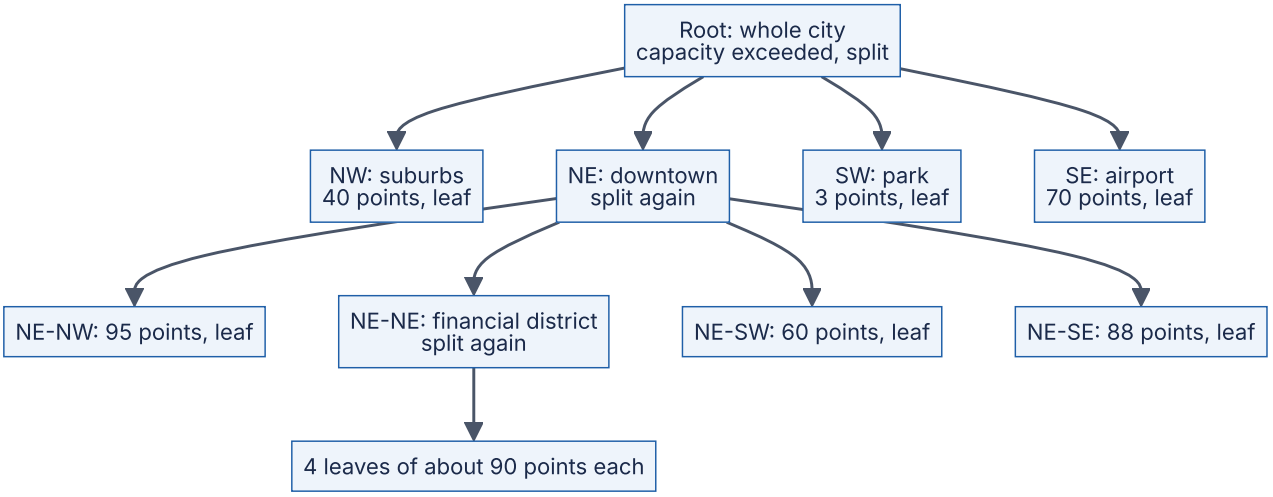
What to notice:

- The encoder reproduces the widely used reference geohash `u4pruydqqvj` for (57.64911, 10.40744), and the precision table reproduces Table 17.6.
- **The 3 × 3 trick at 6 characters silently misses 5.8% of drivers within 1 km**, exactly as the worked example predicted: the cells are only 611 m tall.
- Dropping to 5 characters restores full recall but scans 130,946 candidates, 65% of all drivers, because each cell is now about 4 km on a side.
- Covering the circle's bounding box with 25 six-character cells gives full recall with 17,908 candidates, 7× fewer. Rule: pick precision from the radius, then cover the area with as many cells as needed.
- Drivers are uniformly spread at about 1,220 per km² in this test, so the expected count within 1 km is $\pi \times 1^2 \times 1,220 \approx 3,830$; brute force found 3,822.

17.9.3 Quadtrees

A geohash grid is fixed, so a downtown cell may hold thousands of drivers and a lake cell none. A **quadtree** adapts [Finkel & Bentley 1974]: each node covers a rectangle holding up to a capacity (say 100 points) and splits into four children when it overflows. Dense areas grow deep, empty ones stay shallow, and queries descend only into nodes intersecting the search circle.

Figure 17.12 — An adaptive quadtree: dense areas split further.



Reading the diagram. The root split on overflow; three quadrants are leaves, downtown split again and its financial district a third time, so a query near the park reads one leaf while a downtown query descends three levels. The tree lives in memory and is walked synchronously by parent–child pointers. Failure points: moving points force constant reinsertion, splits and merges, complicating concurrency; a single in-memory tree is a single point of failure, so production systems shard by region and rebuild from the location stream after restart. For persistent indexes in ordinary databases, fixed cells (geohash, S2) fit B-trees better.

17.9.4 Google S2

S2 is Google's open-source spherical geometry library [S2 Geometry Docs]. It projects the sphere onto a cube's six faces, divides each face as a quadtree and orders cells along a **Hilbert curve**, whose consecutive cells always share an edge (the Z-curve jumps). A 64-bit **cell ID** encodes face, path and level (0–30); a cell's descendants form a contiguous ID range, so "points in this cell" is an integer range scan. A **region coverer** approximates any circle or polygon with a few cells at mixed levels.

FORMULA

Average S2 cell area at level L

$$\bar{A}_L \approx \frac{4\pi R^2}{6 \times 4^L} = \frac{510.1 \times 10^6 \text{ km}^2}{6 \times 4^L}$$

Symbol	Meaning	Units
$\bar{A}_L$	Average area of a level-L cell	km²
R	Earth's mean radius, ≈ 6,371 km	km
L	Cell level, 0 (a cube face) to 30	level

WORKED EXAMPLE

Worked example — which S2 level for a 1 km search? Level 12: $85.0 \times 10^6 / 4^{12} = 85.0 \times 10^6 / 16,777,216 = 5.07$ km², about 2.3 km across. Level 13: 1.27 km², about 1.1 km across. Level 30: 7.4×10^{-11} km² ≈ 0.74 cm². *Interpretation.* Each level quarters the area; 31 levels span a sixth of the planet to under a square centimeter. *Engineering implication.* Index points at a fine level and let the coverer choose coarse query cells, so one index serves every radius.

17.9.5 Uber H3

H3 is Uber's open-source hexagonal grid [H3 Docs; Brodsky 2018]. It projects the globe onto an icosahedron and tiles it with hexagons at 16 resolutions (0–15): 122 base cells, about 7× more cells per finer resolution, and 12 unavoidable pentagons per resolution. A hexagon's six neighbors all share an edge and are equidistant, whereas a square has edge and corner neighbors at different distances, so hexagons suit movement analysis, neighborhood smoothing and area pricing. The trade-off: seven children do not exactly tile their parent, so containment is approximate.

Table 17.7 — Selected H3 resolutions (average hexagon, from the H3 library version 4).

Resolution	Average area	Average edge length	Cells on Earth	Typical use
0	4,357,449 km²	1,281 km	122	Continental
5	252.9 km²	9.85 km	2,016,842	Regional demand
7	5.16 km²	1.41 km	98,825,162	City districts
8	0.737 km²	531 m	691,776,122	Neighborhood supply and demand
9	0.105 km²	201 m	4,842,432,842	Pickup zones, street blocks
10	0.0150 km²	75.9 m	33,897,029,882	Fine-grained locations
15	0.90 m²	0.58 m	about 5.7×10^{14}	Sub-meter (beyond GPS accuracy)

A k-ring query (`grid_disk` in H3 v4) returns a hexagon and all hexagons within k steps: 7 cells for $k = 1$ and 19 for $k = 2$ ($1 + 3k(k+1)$).

INDUSTRY EXAMPLE

Uber and Redis. Uber built H3 to analyze supply, demand and pricing by hexagon, using equidistant neighbors to smooth across adjacent areas, and open-sourced it in 2018 [Brodsky 2018; H3 Docs]. Redis's `GEOADD` / `GEOSEARCH` store each member's 52-bit interleaved geohash as its sorted-set score, so a radius search becomes range scans over covering cells plus a distance filter [Redis Docs: Geospatial]. *Evidence: Primary* — [Brodsky 2018; H3 Docs; Redis Docs]

17.9.6 Putting it together: a proximity service

Figure 17.13 — A nearby-drivers service built on a cell index.



Reading the diagram. Drivers report every 4 seconds; the location service computes each position's cell and moves the driver between cells in an in-memory geo index sharded by region. Updates are also appended asynchronously (dotted) to a location stream for analytics and for rebuilding a failed shard. A rider's request is synchronous: the query service covers the circle with cells, reads candidates, filters by exact distance and ranks by arrival time. Load: 1 million drivers every 4 s is 250,000 writes per second, hence memory and partitioning. Failure points: stale entries from dead apps (use a TTL), hot shards during events (finer cells or shard splits), and circles straddling shard boundaries (two-shard fan-out). Chapter 55 (§55.1, §55.2) develops the full design.

Table 17.8 — Geospatial indexes compared.

	Geohash	Quadtree	S2	H3	R-tree
Cell shape	Lat/lon rectangles	Adaptive rectangles	Spherical quadrilaterals	Hexagons (+12 pentagons)	Bounding boxes of data
Key	Base-32 string (prefix = parent)	Tree path	64-bit ID (range = parent)	64-bit ID	None (tree pages)
Adapts to density	No	Yes	No (coverer mixes levels)	No	Yes
Works in any ordered store	Yes	No (in memory)	Yes	Yes (exact match per cell)	No (specialized index)
Distortion	High near poles	Depends on projection	Low	Low	—
Neighbor queries	8 neighbors, boundary quirks	Tree walk	Edge neighbors	Uniform rings	Box overlap
Choose when	Simple prefix/range scans, Redis, Elasticsearch	In-memory dynamic points	Global coverage, polygons, ranges on integers	Movement analytics, aggregation, pricing	Polygons and shapes in PostGIS (GiST)

COMMON MISTAKE

"Searching the 3 × 3 block of geohash cells finds everything within the radius." Only if the cell's smaller side is at least the radius, and widths shrink with latitude; the measurement above lost 5.8% at 1 km. Derive precision from radius and latitude, or use a region coverer.

17.10 Unique ID generation

17.10.1 What an ID scheme must provide

Every row, message, order and event needs an identifier. On one database, an auto-increment column does the job. Across hundreds of services and partitioned databases, you need IDs that are:

1. **Unique** across all generators, forever.
2. **Generated without coordination** on the hot path, so that ID creation adds no network round trip and survives the loss of any central service.
3. **Roughly time-ordered** (**k-sortable**): later IDs sort later, keeping B-tree inserts at the index's right edge (§11.8, §13.6) and enabling pagination and recent-item range scans.
4. **Compact**: 64 bits fits a `BIGINT` and halves index size compared with 128 bits.
5. **Not leaking** information you would rather hide, such as exact creation time, order volume or which machine created the record.

No scheme maximizes all five. The rest of this section compares the main families.

17.10.2 UUIDv4 and UUIDv7

RFC 9562 (May 2024) defines the current 128-bit **UUID** versions and replaces RFC 4122 [RFC 9562]. **UUIDv4** is 122 random bits plus version and variant bits. **UUIDv7** starts with a 48-bit Unix millisecond timestamp, then the version, 12 bits of randomness or sub-millisecond counter, the variant and 62 random bits, so it sorts by creation time. Random UUIDv4 keys hit a random B-tree leaf on every insert; once the index exceeds memory, most inserts read a page and split pages all over the tree. UUIDv7 inserts land on the hot rightmost pages.

Collisions between random IDs are governed by the birthday bound (§10.4.1).

FORMULA

Birthday bound for random identifiers

$$P_{\text{collision}} \approx 1 - e^{-n^2/(2N)} \approx \frac{n^2}{2N} \text{ (when small)}, \qquad n_{50\%} \approx \sqrt{2N \ln 2} \approx 1.18\sqrt{N}$$

Symbol	Meaning	Units
n	Number of IDs generated within one collision domain	IDs
N	Number of possible values, $2^{\text{random bits}}$	values
$P_{\text{collision}}$	Probability that at least two IDs are equal	probability
$n_{50\%}$	Number of IDs at which a collision becomes a coin flip	IDs

WORKED EXAMPLE

Worked example — how safe are UUIDv4 and UUIDv7?

1. UUIDv4: $N = 2^{122} = 5.3 \times 10^{36}$. For one trillion IDs, $n^2/(2N) = 10^{24}/(1.06 \times 10^{37}) = 9.4 \times 10^{-14}$. A coin-flip collision needs $n_{50\%} = 1.18 \times \sqrt{5.3 \times 10^{36}} = 2.7 \times 10^{18}$ IDs.
2. UUIDv7 with all 74 non-timestamp bits random: only IDs created in the same millisecond can collide, so $N = 2^{74} = 1.9 \times 10^{22}$ per millisecond. At 1 million IDs per second (1,000 per ms): $10^6/(3.8 \times 10^{22}) = 2.6 \times 10^{-17}$ per millisecond; over a year of 3.16×10^{10} ms, 8.3×10^{-7}.

Interpretation. Both are safe for realistic systems. *Engineering implication.* The real risk is bad randomness (cloned VM random state, time-seeded generators); use the OS cryptographic source and keep a unique constraint.

17.10.3 Twitter Snowflake

In 2010, as Twitter moved storage from MySQL toward Cassandra, it needed roughly time-ordered 64-bit IDs generated at high rate without coordination. **Snowflake**, a small Thrift service, composed IDs from a timestamp, a machine number and a counter, with requirements of at least 10,000 IDs per second per process and 2 ms response time plus network latency [Twitter: Snowflake 2010].

Figure 17.14 — The Snowflake 64-bit ID layout.



Reading the diagram. Fields run from the most significant bit (left) to the least. The sign bit stays 0 so IDs are positive in signed 64-bit types. The 41-bit timestamp counts milliseconds since a custom epoch (Twitter's is 1288834974657, 4 November 2010 01:42:54.657 UTC [Twitter: Snowflake IdWorker]); in the high bits it makes IDs sort by time across machines, within clock skew. The 10-bit machine ID separates generators, so they never communicate; the only coordination is assigning machine IDs once, by configuration or a coordination service (§26.8). The 12-bit sequence counts IDs per millisecond per machine. Failure points: duplicated machine IDs, a clock moving backward, and exhausting 4,096 IDs in one millisecond (wait for the next).

FORMULA

Snowflake capacity and lifetime

$$\text{rate}_{\text{machine}} = 2^{S} \times 1000 \text{ IDs/s}, \qquad \text{rate}_{\text{cluster}} = 2^{M} \times \text{rate}_{\text{machine}}, \qquad \text{lifetime} = \frac{2^{T}}{1000 \times 3600 \times 24 \times 365.25} \text{ yea}$$

Symbol	Meaning	Units
T	Timestamp bits (millisecond resolution)	bits
M	Machine-ID bits	bits
S	Sequence bits	bits
rate	Maximum sustained ID generation rate	IDs/s
lifetime	Time from the epoch until the timestamp field overflows	years

WORKED EXAMPLE

Worked example — the 41/10/12 layout.

1. Per machine: $2^{12} = 4,096$ IDs per millisecond, or $4,096,000$ IDs/s.
2. Cluster: $2^{10} = 1,024$ machines × 4,096,000 = 4.19×10^{9} IDs/s.
3. Lifetime: $2^{41} = 2,199,023,255,552$ ms $= 2.199 \times 10^{12} / 3.156 \times 10^{10} = 69.7$ years. From Twitter's 2010 epoch, IDs run out in July 2080.
4. Instagram's variant uses 41 timestamp bits, 13 bits of logical shard ID and 10 sequence bits, generated inside PostgreSQL [Instagram Engineering 2012]: 8,192 shards × 1,024 IDs per millisecond each.

Interpretation. Moving a bit between fields doubles one limit and halves another. *Engineering implication.* Choose a late epoch, size machine bits for your largest fleet including short-lived containers, and document the layout: once clients store IDs it is a public contract.

Code example

```
        import datetime as dt
        import os
        import threading
        import time
        import uuid

        TWITTER_EPOCH_MS = 1288834974657           # 2010-11-04 01:42:54.657 UTC, from
Twitter's IdWorker
        TIMESTAMP_BITS, MACHINE_BITS, SEQUENCE_BITS = 41, 10, 12
        MAX_MACHINE = (1 << MACHINE_BITS) - 1      # 1023
        MAX_SEQUENCE = (1 << SEQUENCE_BITS) - 1    # 4095

        class ClockMovedBackwards(Exception):
            pass

        class Snowflake:
            """64-bit IDs: 1 unused sign bit | 41-bit ms timestamp | 10-bit machine id |
12-bit sequence."""

            def __init__(self, machine_id: int, epoch_ms: int = TWITTER_EPOCH_MS,
clock=lambda: time.time_ns() // 1_000_000):
                if not 0 <= machine_id <= MAX_MACHINE:
                    raise ValueError("machine id must fit in 10 bits")
                self.machine_id, self.epoch_ms, self.clock = machine_id, epoch_ms, clock
                self.last_ms, self.sequence = -1, 0
                self.lock = threading.Lock()

            def next_id(self) -> int:
                with self.lock:
                    now = self.clock()
                    if now < self.last_ms:                    # NTP stepped the clock
back
                        raise ClockMovedBackwards(f"refusing to issue IDs for
{self.last_ms - now} ms")
                    if now == self.last_ms:
                        self.sequence = (self.sequence + 1) & MAX_SEQUENCE
                        if self.sequence == 0:                # 4,096 IDs used this
ms: wait for the next
                            while now <= self.last_ms:
                                now = self.clock()
                    else:
                        self.sequence = 0
                    self.last_ms = now
                    return ((now - self.epoch_ms) << (MACHINE_BITS + SEQUENCE_BITS)) \
                        | (self.machine_id << SEQUENCE_BITS) | self.sequence

        def decode(snowflake_id: int, epoch_ms: int = TWITTER_EPOCH_MS):
            ms = (snowflake_id >> (MACHINE_BITS + SEQUENCE_BITS)) + epoch_ms
            machine = (snowflake_id >> SEQUENCE_BITS) & MAX_MACHINE
            seq = snowflake_id & MAX_SEQUENCE
            return dt.datetime.fromtimestamp(ms / 1000, dt.timezone.utc), machine, seq

        def uuid7() -> uuid.UUID:
            """RFC 9562 UUIDv7: 48-bit Unix ms | ver=7 | 12 random bits | var=10 | 62
```

```
random bits."""
            ms = time.time_ns() // 1_000_000
            rand = int.from_bytes(os.urandom(10), "big")      # 80 random bits, 74 of
them used
            value = (ms & ((1 << 48) - 1)) << 80
            value |= 0x7 << 76                                 # version
            value |= ((rand >> 62) & 0xFFF) << 64              # rand_a (12 bits)
            value |= 0b10 << 62                                # variant
            value |= rand & ((1 << 62) - 1)                    # rand_b (62 bits)
            return uuid.UUID(int=value)

        gen = Snowflake(machine_id=37)
        ids = [gen.next_id() for _ in range(200_000)]
        assert len(set(ids)) == len(ids) and ids == sorted(ids)   # unique and strictly
increasing
        print("sample id:", ids[-1], "->", decode(ids[-1]))
        start = time.perf_counter()
        for _ in range(200_000):
            gen.next_id()
        print(f"this Python generator: {200_000 / (time.perf_counter() - start):,.0f}
IDs/s")

        per_machine = (MAX_SEQUENCE + 1) * 1000
        print(f"capacity: {per_machine:,} IDs/s per machine, {per_machine * (MAX_MACHINE
+ 1):,} IDs/s cluster")
        lifetime_ms = 1 << TIMESTAMP_BITS
        years = lifetime_ms / (1000 * 3600 * 24 * 365.25)
        end = dt.datetime.fromtimestamp((TWITTER_EPOCH_MS + lifetime_ms) / 1000,
dt.timezone.utc)
        print(f"lifetime: 2^41 ms = {lifetime_ms:,} ms = {years:.1f} years; Twitter-
epoch IDs run out {end:%Y-%m-%d}")

        # A clock that jumps back 5 ms
        fake = iter([1000, 1001, 996, 1002])
        g = Snowflake(1, epoch_ms=0, clock=lambda: next(fake))
        g.next_id(); g.next_id()
        try:
            g.next_id()
        except ClockMovedBackwards as e:
            print("clock step:", e)

        u = [uuid7() for _ in range(3)]
        print("uuid7:", u[0], "version", u[0].version, "| uuid4:", uuid.uuid4())
```

Output (IDs, dates and the rate vary by run and machine):

```
sample id: 2116059594966061194 -> (datetime.datetime(2026, 10, 30, 6, 48, 16, 583000,
tzinfo=datetime.timezone.utc), 37, 138)
this Python generator: 1,671,382 IDs/s
capacity: 4,096,000 IDs/s per machine, 4,194,304,000 IDs/s cluster
lifetime: 2^41 ms = 2,199,023,255,552 ms = 69.7 years; Twitter-epoch IDs run out 2080-
07-10
clock step: refusing to issue IDs for 5 ms
uuid7: 01a18b90-fa5a-78bc-a410-f69281e34317 version 7 | uuid4: 07ac5358-bdb5-42c6-bed7-
46d2df7a34d6
```

What to notice:

- 200,000 IDs from one generator are unique and strictly increasing, and decoding recovers the creation time, machine 37 and the sequence number.
- Even this Python generator manages about 1.7 million IDs per second; the real ceiling is 4,096 per millisecond.
- On a 5 ms backward clock step the generator refuses to issue IDs, as Twitter's `IdWorker` does [Twitter: Snowflake IdWorker]. Alternatives: wait out small steps, or reuse the last timestamp and borrow sequence numbers. Configure NTP to slew rather than step (§24.9).
- This `uuid7()` does not guarantee ordering among IDs created in the same millisecond. RFC 9562 §6.2 describes counter-based methods that do.

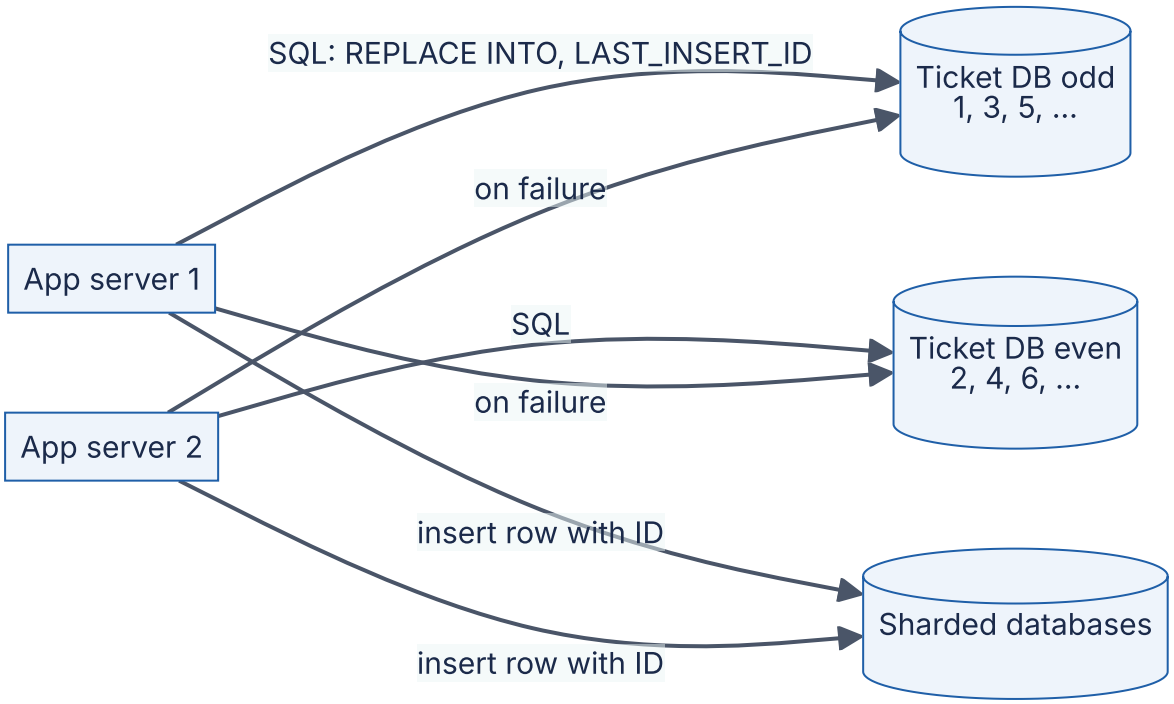
17.10.4 ULID

A **ULID** is 128 bits: a 48-bit millisecond timestamp and 80 random bits, written as 26 Crockford base-32 characters that sort in time order [ULID Spec], convenient for string keys (object paths, sort keys). Its timestamp lasts until the year 10889, and a monotonic mode increments the random part within a millisecond. UUIDv7 now offers the same properties in standard form.

17.10.5 Database sequences and ticket servers

Flickr scaled database sequences with **ticket servers**: dedicated MySQL instances that only hand out numbers via `REPLACE INTO Tickets64 (stub) VALUES ('a'); SELECT LAST_INSERT_ID();` on a one-row auto-increment table [Flickr: Ticket Servers 2010]. Two servers issue odd and even numbers (`auto_increment_increment = 2`, offsets 1 and 2), so either can fail without stopping ID generation.

Figure 17.15 — Flickr-style ticket servers.



Reading the diagram. App servers fetch an ID with one synchronous SQL round trip, then insert rows into the sharded databases. The ticket servers never communicate, since odd and even cannot collide; on failure clients switch servers (cross arrows), losing global ordering. Failure points: a round trip per ID and a write-path dependency whose capacity is one single-row MySQL update stream. Fetching **blocks** of, say, 1,000 IDs divides load by the block size at the cost of only rough ordering.

Table 17.9 — Unique ID schemes compared.

	UUIDv4	UUIDv7	Snowflake-style	ULID	Sequence / ticket server
Size	128 bits	128 bits	64 bits	128 bits (26 chars)	64 bits
Time-ordered	No	Yes (ms)	Yes (ms, within clock skew)	Yes (ms)	Yes per server; roughly with blocks
Coordination	None	None	Assign machine IDs once	None	Round trip per ID or block
Uniqueness	Probabilistic (122 bits)	Probabilistic per ms (74 bits)	Guaranteed if machine IDs unique and clock monotonic	Probabilistic per ms (80 bits)	Guaranteed
Leaks	Nothing	Creation time	Creation time, machine, volume	Creation time	Volume (sequential)
Index locality	Poor (random inserts)	Good	Good	Good	Best
Main failure mode	Bad randomness	Bad randomness	Duplicate machine ID; clock steps	Bad randomness	Ticket tier outage
Choose when	Unguessable public IDs; low insert rates	Default 128-bit key	Compact keys at very high rates	String keys that must sort	Small systems; strict ordering needed

COMMON MISTAKE

“Time-ordered IDs give a total order of events.” IDs from different machines are ordered only within clock skew (§24.9); for causal order use logical clocks or a single sequencer (§24.10).

COMMON MISTAKE

“Sequential IDs in URLs are harmless.” They reveal growth and invite enumeration. Authorization must stop access either way (§10.7.1), but an opaque public ID mapped to the internal key removes the leak.

17.11 Tries

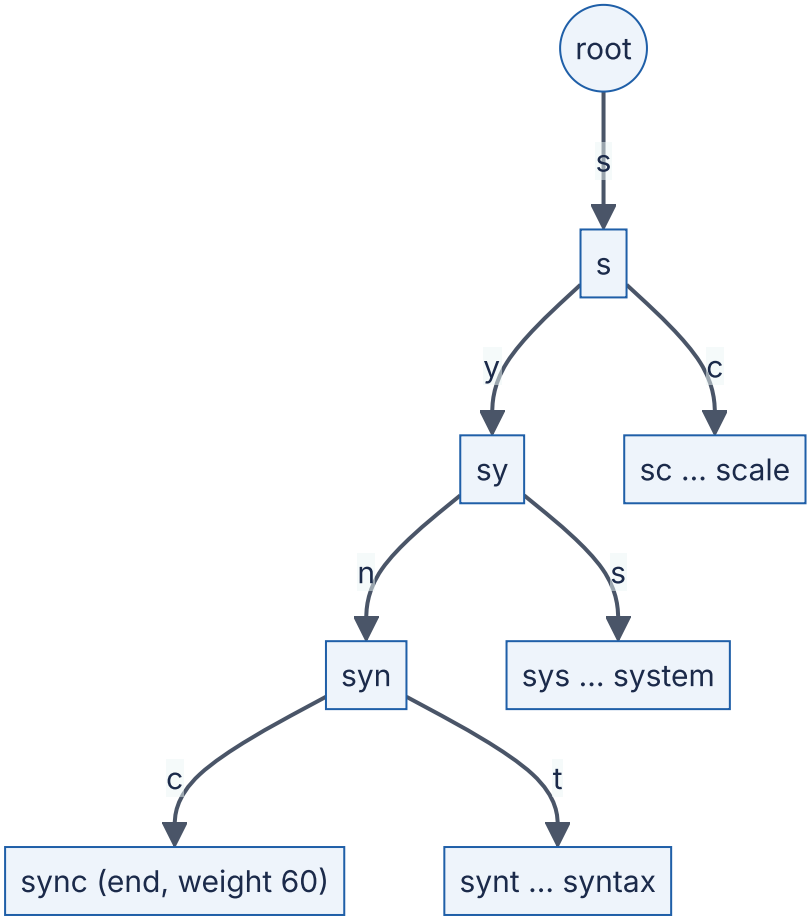
17.11.1 Prefix trees

DEFINITION

Definition — Trie (prefix tree). A tree in which each edge is labeled with a character and each node represents the string spelled by the path from the root; nodes that end a stored key are marked. Looking up a key or a prefix of length L takes $O(L)$ steps, independent of how many keys are stored [Fredkin 1960].

A trie answers the question a hash table cannot: “which stored keys *start with* this prefix?” Walk down L edges to the prefix node; its subtree holds every completion. **Radix trees** (Patricia tries) compress chains of single-child nodes into one edge labeled with a string, which cuts node count and pointer overhead dramatically. Tries are used for autocomplete, longest-prefix matching in IP routing tables, and as compact sorted dictionaries; Redis uses a radix tree internally for streams [Redis Docs: Streams]. Chapter 16 (§16.9) builds a production autocomplete on tries and finite-state transducers, including precomputed top-k completions at each node; this section gives only the structure.

Figure 17.16 — A trie holding sync, syntax, system and scale.



Reading the diagram. Each edge adds one character; the ellipses abbreviate chains of single-child nodes that a radix tree would store as one edge. A query for the prefix `syn` follows three edges and then enumerates the subtree: `sync` and `syntax`. All operations are in-memory pointer walks. The main failure point is memory: a naive trie with one node per character can use tens of bytes per character, so large dictionaries use radix trees, arrays of children, or succinct encodings. Tries scale by sharding on the first characters of the key, with care for skewed prefixes (many queries start with “s” and few with “x”).

```
        import heapq

        class TrieNode:
            __slots__ = ("children", "weight")
            def __init__(self):
                self.children, self.weight = {}, 0              # weight > 0 marks the
end of a stored word

        class Trie:
            def __init__(self):
                self.root, self.nodes = TrieNode(), 1

            def insert(self, word: str, weight: int) -> None:
                node = self.root
                for ch in word:
                    if ch not in node.children:
                        node.children[ch] = TrieNode()
                        self.nodes += 1
                    node = node.children[ch]
                node.weight += weight

            def top_k(self, prefix: str, k: int):
                """Walk to the prefix node (O(len(prefix))), then collect its subtree's
k heaviest words."""
                node = self.root
                for ch in prefix:
                    node = node.children.get(ch)
                    if node is None:
                        return []
                found, stack = [], [(node, prefix)]
                while stack:
                    cur, word = stack.pop()
                    if cur.weight:
                        found.append((cur.weight, word))
                    stack.extend((child, word + ch) for ch, child in
cur.children.items())
                return heapq.nlargest(k, found)

        t = Trie()
        for word, w in [("system", 90), ("systems", 40), ("syscall", 25), ("sync", 60),
                        ("synchronous", 30), ("syntax", 20), ("scale", 80), ("schema",
50)]:
            t.insert(word, w)
        print(t.top_k("sy", 3))
        print(t.top_k("syn", 5))
        print("nodes:", t.nodes, "for", sum(len(w) for w in ["system", "systems",
"syscall", "sync",
              "synchronous", "syntax", "scale", "schema"]), "characters")
```

Output:

```
[(90, 'system'), (60, 'sync'), (40, 'systems')]
[(60, 'sync'), (30, 'synchronous'), (20, 'syntax')]
nodes: 32 for 52 characters
```

What to notice: shared prefixes are stored once (32 nodes for 52 characters), and the cost of `top_k` is the prefix walk plus the size of the subtree. For a short prefix such as “s” in a large dictionary the subtree is huge, which is why production autocomplete stores pre-computed top-k lists at each node (§16.9).

17.12 Reservoir sampling and streaming algorithms

17.12.1 Reservoir sampling

DEFINITION

Definition — Reservoir sampling. An algorithm that maintains a uniform random sample of k items from a stream whose length n is not known in advance, using $O(k)$ memory and a single pass: after any number of items, every item seen so far is in the sample with probability k/n [Vitter 1985].

First-principles explanation and mechanism

You want 1,000 representative requests from a day of traffic for debugging, but you do not know how many requests the day will bring, and you cannot store them all. Sampling each request with a fixed probability gives the wrong sample size if traffic differs from your guess. Reservoir sampling (**Algorithm R**, which Knuth attributes to Alan Waterman) adjusts as it goes:

1. Put the first k items in the reservoir.
2. For the n-th item ($n > k$), draw a random integer j in $[0, n)$. If $j < k$, replace reservoir slot j with the new item; otherwise discard it.

FORMULA

Reservoir sampling inclusion probability

$$P(\text{item } i \text{ in sample after } n \text{ items}) = \frac{k}{i} \times \prod_{j=i+1}^{n} \left(1 - \frac{1}{j}\right) = \frac{k}{i} \times \frac{i}{n} = \frac{k}{n}, \qquad \mathbb{E}[\text{replacements}] \approx k \ln \frac{n}{k}$$

Symbol	Meaning	Units
k	Reservoir size	items
n	Items seen so far	items
i	Arrival position of a particular item ($i > k$)	position
$1 - 1/j$	Probability that item j does not evict this item: it is chosen with probability k/j and then evicts one of k slots	probability

WORKED EXAMPLE

Worked example — sampling traces. A service handles 600,000 requests per minute and keeps a reservoir of $k = 1,000$ per minute. Each request is kept with probability $1,000/600,000 = 1/600$. The expected number of reservoir writes per minute is about $1,000 + 1,000 \times \ln(600,000/1,000) = 1,000 + 1,000 \times 6.40 = 7,400$, not 600,000. *Interpretation.* Late items are rarely written, so the cost is dominated by drawing a random number per item. *Engineering implication.* Vitter's Algorithm Z [Vitter 1985] and Li's Algorithm L [Li 1994] compute how many items to *skip* before the next replacement, drawing about $O(k \log(n/k))$ random numbers instead of n; use them when random-number generation dominates.

Code example and measured validation

```
        import math
        import random
        from collections import Counter

        def reservoir_sample(stream, k, rng=random):
            """Algorithm R: a uniform sample of k items from a stream of unknown length,
in O(k) memory."""
            reservoir = []
            for n, item in enumerate(stream, start=1):
                if n <= k:
                    reservoir.append(item)                    # fill the reservoir
first
                else:
                    j = rng.randrange(n)                      # uniform in [0, n)
                    if j < k:                                 # happens with probabil-
ity k / n
                        reservoir[j] = item                   # evict a uniformly cho-
sen resident
            return reservoir

        def reservoir_sample_skip(stream, k, rng=random):
            """Algorithm L (Li 1994): same distribution, but jumps over items, so it
draws
            O(k log(n/k)) random numbers instead of n."""
            it = iter(stream)
            reservoir = [x for _, x in zip(range(k), it)]
            w = math.exp(math.log(rng.random()) / k)
            while True:
                skip = math.floor(math.log(rng.random()) / math.log(1 - w))
                for _ in range(skip):                         # discard without drawing
randomness
                    if next(it, None) is None:
                        return reservoir
                item = next(it, None)
                if item is None:
                    return reservoir
                reservoir[rng.randrange(k)] = item
                w *= math.exp(math.log(rng.random()) / k)

        # Validate uniformity: every one of n = 1,000 items should be chosen with proba-
bility k/n = 0.01
        n, k, trials = 1_000, 10, 50_000
        rng = random.Random(5)
        for name, fn in (("Algorithm R", reservoir_sample), ("Algorithm L",
reservoir_sample_skip)):
            counts = Counter()
            for _ in range(trials):
                counts.update(fn(range(n), k, rng))
            freqs = [counts[i] / trials for i in range(n)]
            expected = k * trials / n                         # 500 selections per item
            chi2 = sum((counts[i] - expected) ** 2 / expected for i in range(n))
            print(f"{name}: inclusion prob. min {min(freqs):.4f}  max {max(freqs):.4f}
"
                  f"(target {k / n:.4f}); first decile {sum(freqs[:100]) / 100:.4f}, "
```

```
                    f"last decile {sum(freqs[-100:]) / 100:.4f}; chi-square {chi2:.0f} on
{n - 1} dof")
```

Output:

```
Algorithm R: inclusion prob. min 0.0087  max 0.0115  (target 0.0100); first decile
0.0100, last decile 0.0100; chi-square 957 on 999 dof
Algorithm L: inclusion prob. min 0.0087  max 0.0113  (target 0.0100); first decile
0.0100, last decile 0.0100; chi-square 969 on 999 dof
```

What to notice: early and late items are equally likely to be chosen (first and last deciles both 0.0100), and the chi-square statistics (957 and 969 on 999 degrees of freedom) are what a uniform sample produces. The spread from 0.0087 to 0.0115 is ordinary sampling noise over 50,000 trials. The skip-based Algorithm L produces the same distribution with far fewer random draws. (The `None` sentinel in this sketch assumes the stream contains no `None` items.)

Two extensions are common. **Weighted reservoir sampling** keeps items with probability proportional to a weight, for example to oversample slow or failed requests [Efraimidis & Spirakis 2006]. **Distributed reservoir sampling** tags each item with a random key and keeps the k items with the smallest keys; because "smallest k keys" merges across partitions, each partition samples independently and a coordinator merges the results.

17.12.2 The streaming toolbox

Every structure in this chapter is a **streaming algorithm**: one pass, memory far smaller than the data, and an answer whose error you can state in advance. Most are also **mergeable**, which is what makes them fit distributed systems: compute per partition, combine centrally.

Table 17.10 — Streaming and probabilistic structures at a glance.

Question	Structure	Memory	Error	Mergeable	Section
Have I seen *x*?	Bloom / cuckoo filter	~10 bits per item at 1%	False positives only	Bloom: OR	§17.4
How many distinct items?	HyperLogLog	12 KB for 0.81%	$\pm 1.04/\sqrt{m}$ relative	Register max	§17.5
How often did *x* occur?	Count-Min sketch	$\lceil e/\varepsilon \rceil \times \lceil \ln(1/\delta) \rceil$ counters	$+\varepsilon N$, probability $1-\delta$	Add arrays	§17.6
Which items are most frequent?	Space-Saving; CMS + heap	k counters	$\leq N/k$	Approximately	§17.6.2
What is the p99?	HDR histogram, t-digest, KLL sketch	KBs	Bounded relative or rank error	Yes	§4.8, Chapter 40
Give me a fair sample	Reservoir sampling	k items	Exact distribution	Smallest-k random keys	§17.12.1
Do two replicas differ?	Merkle tree	Hash per range	Exact (with cryptographic hash)	—	§17.8

17.13 Summary and revision notes

SUMMARY

- **Hashing.** Good non-cryptographic hashes (MurmurHash3, xxHash) are deterministic, uniform and avalanche well. Test uniformity with chi-square ($\approx B - 1$ for a good hash). Use keyed SipHash for tables filled from untrusted input (hash flooding), cryptographic hashes for integrity. Consistent hashing is in §14.8.
- **Bloom filters.** $p \approx (1 - e^{-kn/m})^k$; optimal $k = (m/n) \ln 2$, where half the bits are set; $m/n = -\ln p/(\ln 2)^2$: 9.6 bits at 1%, +4.8 bits per 10× lower p. No false negatives; no deletes; OR merges. Overfilling 3× pushed a 1% filter to 44%.
- **Deletion.** Counting Bloom filters (4-bit counters, 4× memory) or cuckoo filters (fingerprints in two buckets, ~95% load, $\varepsilon \leq 2b/2^f$, smaller than Bloom below about 3% with semi-sorting).
- **HyperLogLog.** Leading zeros of hashes estimate cardinality; $m = 2^p$ registers, SE $= 1.04/\sqrt{m}$; Redis: 12 KB, 0.81%. Merge by register-wise max; unions are accurate, intersections are not. HLL++ adds 64-bit hashing, bias correction and sparse storage.
- **Count-Min.** $w = \lceil e/\varepsilon \rceil$, $d = \lceil \ln(1/\delta) \rceil$; never underestimates; error $\leq \varepsilon N$ with probability $1 - \delta$, relative to the whole stream. Heavy hitters: CMS + heap, or Space-Saving with error $\leq N/k$.
- **Skip lists.** Random levels with probability p; expected $O(\log n)$; $1/(1-p)$ pointers per node. Redis sorted sets ($p = 1/4$, spans for rank) and LSM memtables.
- **Merkle trees.** Root hash summarizes data; inclusion proofs of $\lceil \log_2 n \rceil$ hashes; anti-entropy cost grows with differences × depth (Dynamo, Cassandra, Git, Certificate Transparency).
- **Geospatial.** Map 2-D to 1-D cells. Geohash: interleaved bits, base-32, prefix = containment, boundary quirks; choose precision from radius and latitude or cover the area. Quadtrees adapt to density. S2: cube + Hilbert curve + 64-bit IDs, area /4 per level. H3: hexagons, aperture 7, uniform neighbors.
- **IDs.** UUIDv4 (random, poor index locality), UUIDv7 (time-ordered, RFC 9562), Snowflake (41/10/12 bits: 4.1 M IDs/s per machine, 69.7 years), ULID, ticket servers. Time-ordered IDs are ordered only within clock skew.
- **Streaming.** Tries answer prefix queries in $O(L)$. Reservoir sampling keeps a uniform k-sample with probability k/n in one pass. Prefer mergeable sketches in distributed systems: compute per partition, combine centrally.

17.14 Knowledge checkpoint

1. Why does the sum-of-bytes hash perform so badly on keys such as `user:12345`?

2. What is hash flooding, and name two defenses.
3. A Bloom filter has $m = 8 \times 10^6$ bits, $n = 10^6$ items and $k = 6$. What is its false-positive rate?
4. How many bits per element and how many hash functions do you need for $p = 0.1\%$?
5. Why can you not delete from a standard Bloom filter?
6. In a cuckoo filter, why is the alternate bucket computed as $i_1 \oplus h(fp)$ rather than from the original item?
7. What are the memory and standard error of a HyperLogLog with $p = 12$?
8. Why can HyperLogLog sketches be merged without double counting?
9. Size a Count-Min sketch for $\varepsilon = 0.0001$ and $\delta = 0.001$.
10. With Space-Saving and $k = 500$ counters over $N = 10^7$ events, what is the maximum overestimate?
11. What are the expected levels and pointers per node of a skip list with $p = 1/4$ and $n = 4^{10}$?
12. How many hashes are in a Merkle inclusion proof for 10^9 records?
13. How large is a 6-character geohash cell at latitude 60°?
14. Why do H3 hexagons suit neighborhood smoothing better than square cells?
15. How long do 41-bit millisecond timestamps last, and how many IDs per second can one Snowflake machine issue?
16. Why do random UUIDv4 primary keys slow down inserts into a large B-tree?
17. What is the probability that the 10th item of a stream is in a reservoir of size 5 after 1,000 items?

Answers

1. The sum takes fewer than 150 distinct values for these keys, so they crowd into a few buckets; there is no avalanche.
2. An attacker sends keys that all collide in an unkeyed hash table, making inserts $O(n)$ each and total work $O(n^2)$. Defenses: a keyed hash with a secret per-process key (SipHash), tree-structured buckets (Java 8), and limits on parameter counts.
3. $kn/m = 0.75$; $1 - e^{-0.75} = 0.5276$; $0.5276^6 = 0.0216$, about 2.2%.
4. $6.908/0.4805 = 14.4$ bits per element; $k = 10$.
5. A bit may have been set by several items; clearing it for one would cause false negatives for others.
6. Because only the fingerprint is stored. XOR with the fingerprint's hash is its own inverse, so either bucket can compute the other from the fingerprint alone during relocation.
7. $m = 4,096$ registers × 6 bits = 3,072 bytes; SE $= 1.04/64 = 1.63\%$.
8. Registers keep maxima, and the maximum is idempotent and order-independent: a register-wise max equals the sketch of the combined stream.
9. $w = \lceil 2.718/0.0001 \rceil = 27,183$; $d = \lceil \ln 1000 \rceil = 7$; 190,281 counters.
10. $N/k = 10^7/500 = 20,000$.
11. $\log_4 4^{10} = 10$ levels; $1/(1 - 0.25) = 1.33$ pointers per node.

12. $\lceil \log_2 10^9 \rceil = 30$ hashes (960 bytes with SHA-256).
13. Width $1.223 \times \cos 60^\circ = 0.61$ km; height 0.61 km, about 610 m × 610 m.
14. All six neighbors share an edge and are equidistant from the center, so a k-ring is a uniform neighborhood; squares have edge and corner neighbors at different distances.
15. 2^{41} ms = 69.7 years; $4,096 \times 1,000 = 4,096,000$ IDs per second.
16. Each insert targets a random leaf page; once the index exceeds memory, most inserts need a disk read and cause page splits across the tree. Time-ordered keys append near the rightmost pages, which stay cached.
17. $k/n = 5/1,000 = 0.5\%$, the same as every other item.

17.15 Interview questions

Basic

Q1. How would you check username availability for 1 billion users as the user types? Answer. Put a Bloom filter of all usernames in memory on each API node: at 1% false positives that is 9.6 bits per name, about 1.2 GB, with $k = 7$. A “definitely absent” answer is shown immediately; a “maybe taken” answer is confirmed against the database (a unique index on the username column), which also handles races at sign-up time. Rebuild or grow the filter as the user base grows, and add new names to it on sign-up.

Q2. How would you count daily unique visitors per page for a site with 10 million pages? Answer. One HyperLogLog per page per day (Redis `PFADD`), 12 KB dense at 0.81% error, much less for small pages thanks to the sparse encoding. Weekly and monthly uniques come from `PFMERGE`, not from summing daily counts. Exact counts are needed only for billing-grade metrics.

Q3. Why might you choose UUIDv7 over UUIDv4 for primary keys? Answer. UUIDv7 is time-ordered, so inserts go to the right edge of the B-tree index, keeping hot pages in memory and reducing page splits; it also allows rough time-range queries and pagination by ID. UUIDv4 is better when IDs must reveal nothing, including creation time.

Intermediate

Q4. Design a “trending hashtags in the last 10 minutes” feature for 500,000 tweets per second. Answer. Partition the stream (by hashtag, so each tag’s counts land on one worker). Each worker keeps per-minute Count-Min sketches and a Space-Saving or CMS-plus-heap candidate list. Every few seconds workers send their top candidates to an aggregator, which sums the last ten one-minute windows and ranks by a velocity score (current versus baseline rate) rather than raw counts, so permanently popular tags do not

dominate. With $\varepsilon = 10^{-5}$ the per-window error is $10^{-5} \times 3 \times 10^{7} = 300$ mentions, small next to trending tags with tens of thousands. Discuss spam filtering and late events.

Q5. Your geohash-based "nearby restaurants" search misses results near cell edges. Why, and how do you fix it? Answer. Points close to the query point but across a cell boundary have different prefixes, and a 3 × 3 block covers less than the radius when cells are smaller than the radius. Fix: choose precision from the radius and latitude, compute the full set of cells covering the circle's bounding box, scan all of them, then filter by exact distance. Or use S2's region coverer.

Q6. Two replicas of a 2 TB key-value store may have diverged after a network partition. How do you find and fix the differences efficiently? Answer. Build a Merkle tree per token range on each replica, compare roots, and descend into differing subtrees; at the leaves, stream only the differing ranges and resolve conflicts with the system's rule (timestamps, vector clocks, §27.14). Cost scales with the number of differences times tree depth. Leaf granularity trades tree size against overstreaming. Throttle tree building, which requires scanning data.

Advanced

Q7. An LSM-based store's read latency for missing keys doubled after a data-size increase. What could be wrong with the Bloom filters? Answer. Possibilities: filters sized by an outdated n or a configured bits-per-key too low for the new number of SSTables (expected wasted reads = files × p); filter blocks evicted from the block cache because total filter memory grew (≈ 1.2 bytes per key at 1%), so filters themselves now cost disk reads; or many more levels/files after compaction changes. Check filter-useful and filter-miss metrics and block-cache hit ratio, pin filter and index blocks, lower p for the deepest level, or use a partitioned filter.

Q8. Design ID generation for a multi-region payments platform producing 2 million IDs per second. Answer. A Snowflake-style 64-bit layout with an explicit region field: for example 41 bits of milliseconds from a 2026 epoch (69.7 years), 3 bits of region, 7 bits of worker within the region, 12 bits of sequence (2^{10} generators × 4.1 M/s gives ample headroom). Assign worker IDs through a lease in a coordination service so that two processes never share one, and refuse to issue IDs while a lease is lost. Handle clock steps by waiting out small ones and failing over large ones. Expose IDs externally only through opaque tokens if volume should stay private. Keep a unique constraint on storage as a final defense.

Q9. A product manager wants "users who visited both page A and page B this month" from your HyperLogLog sketches. What do you say? Answer. HLL estimates unions well, but an intersection via inclusion–exclusion has an absolute error on the order of the errors of the large sets: with 10 M users each and a 100,000-user overlap at 0.81%, the noise exceeds the answer. Options: MinHash-based sketches (estimate the

Jaccard similarity, then multiply by the union), exact computation in a batch job over the raw logs, or theta sketches that support set operations with stated error.

17.16 Practical exercises

1. **Hash uniformity.** Replace `murmur3_32` in §17.3.3 with Python's FNV-1a and with CRC32 (`zlib.crc32`). Compare chi-square values on `user:N` keys and on 100,000 random UUID strings.
2. **Bloom filter health.** Extend the §17.4.4 program to print the fraction of 1 bits as you insert from 0 to 3× the design capacity, and plot it against the measured false-positive rate.
3. **Scalable Bloom filter.** Implement a filter that adds a new, larger Bloom filter with a tighter p each time the current one reaches capacity, so the overall false-positive rate stays below a target.
4. **HyperLogLog bias.** Run the §17.5 program for n from 100 to 200,000 with $p = 12$ and plot the relative error. Find where the raw estimator is biased and where linear counting takes over.
5. **Conservative update.** Modify the Count-Min sketch to use conservative update and compare mean and maximum error on the Zipf stream.
6. **Geohash versus H3.** Repeat the proximity search with H3 (`pip install h3`) at resolution 8 using `grid_disk`, and compare candidate counts with the geohash covering approach.

17.17 Design challenges

1. **Web crawler frontier.** Design deduplication of URLs for a crawler that discovers 5 billion URLs per day and keeps 100 billion known URLs. Size the filter (or filters), decide how to handle false positives (missed pages) versus false negatives (impossible by design), and explain how the filter is partitioned across crawler nodes.
2. **Real-time analytics.** A product analytics service ingests 2 million events per second and must answer, within 5 seconds of an event: unique users per feature per hour, the top 50 features by usage, and p99 latency per endpoint. Choose a sketch for each, size it, show how per-partition sketches are merged, and state the error each answer carries.
3. **Nearby search at global scale.** Design the location index for a delivery app with 3 million active couriers in 800 cities, updates every 5 seconds, and queries for the 20 nearest couriers within 3 km. Compare geohash, S2 and H3 for this use, choose a cell size, estimate write load and memory per shard, and describe how the system behaves when a stadium event concentrates 20,000 couriers in a few cells.

17.18 References

1. Apache Cassandra Documentation. Bloom filters; Repair; Partitioners. https://cassandra.apache.org/doc/latest/
2. Appleby, A. *SMHasher* (MurmurHash3 reference implementation and test suite). https://github.com/aappleby/smhasher
3. Aumasson, J.-P., Bernstein, D. J. “SipHash: A Fast Short-Input PRF.” *INDOCRYPT 2012*, LNCS 7668. Springer, 2012.
4. Bloom, B. H. “Space/Time Trade-offs in Hash Coding with Allowable Errors.” *Communications of the ACM* 13(7):422–426, 1970.
5. Bose, P., Guo, H., Kranakis, E., Maheshwari, A., Morin, P., Morrison, J., Smid, M., Tang, Y. “On the False-Positive Rate of Bloom Filters.” *Information Processing Letters* 108(4):210–213, 2008.
6. Brodsky, I. “H3: Uber’s Hexagonal Hierarchical Spatial Index.” Uber Engineering Blog, 2018. https://www.uber.com/blog/h3/
7. Broder, A., Mitzenmacher, M. “Network Applications of Bloom Filters: A Survey.” *Internet Mathematics* 1(4):485–509, 2004.
8. Chacon, S., Straub, B. *Pro Git*, 2nd ed. Apress, 2014. https://git-scm.com/book
9. Chang, F., Dean, J., Ghemawat, S., Hsieh, W. C., Wallach, D. A., Burrows, M., Chandra, T., Fikes, A., Gruber, R. E. “Bigtable: A Distributed Storage System for Structured Data.” *OSDI*, 2006.
10. Collet, Y. *xxHash*. https://github.com/Cyan4973/xxHash
11. Cormode, G., Muthukrishnan, S. “An Improved Data Stream Summary: The Count-Min Sketch and Its Applications.” *Journal of Algorithms* 55(1):58–75, 2005.
12. Crosby, S. A., Wallach, D. S. “Denial of Service via Algorithmic Complexity Attacks.” *USENIX Security Symposium*, 2003.
13. DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., Vogels, W. “Dynamo: Amazon’s Highly Available Key-Value Store.” *SOSP*, 2007.
14. Efraimidis, P. S., Spirakis, P. G. “Weighted Random Sampling with a Reservoir.” *Information Processing Letters* 97(5):181–185, 2006.
15. Fan, B., Andersen, D. G., Kaminsky, M., Mitzenmacher, M. D. “Cuckoo Filter: Practically Better Than Bloom.” *ACM CoNEXT*, 2014.
16. Fan, L., Cao, P., Almeida, J., Broder, A. Z. “Summary Cache: A Scalable Wide-Area Web Cache Sharing Protocol.” *IEEE/ACM Transactions on Networking* 8(3):281–293, 2000.
17. Finkel, R. A., Bentley, J. L. “Quad Trees: A Data Structure for Retrieval on Composite Keys.” *Acta Informatica* 4(1):1–9, 1974.
18. Flajolet, P., Fusy, É., Gandouet, O., Meunier, F. “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm.” *Analysis of Algorithms (AofA)*, DMTCS Proceedings, 2007.

19. Flickr Engineering. “Ticket Servers: Distributed Unique Primary Keys on the Cheap.” code.flickr.net, 2010.
20. Fredkin, E. “Trie Memory.” *Communications of the ACM* 3(9):490–499, 1960.
21. Google Cloud. *BigQuery documentation: HyperLogLog++ functions*. https://cloud.google.com/bigquery/docs/reference/standard-sql/hll_functions
22. Graf, T. M., Lemire, D. “Xor Filters: Faster and Smaller Than Bloom and Cuckoo Filters.” *ACM Journal of Experimental Algorithmics* 25, 2020.
23. H3 Project. *H3 documentation*. https://h3geo.org/docs/
24. Heule, S., Nunkesser, M., Hall, A. “HyperLogLog in Practice: Algorithmic Engineering of a State of the Art Cardinality Estimation Algorithm.” *EDBT*, 2013.
25. Instagram Engineering. “Sharding & IDs at Instagram.” Instagram Engineering Blog, 2012.
26. Kirsch, A., Mitzenmacher, M. “Less Hashing, Same Performance: Building a Better Bloom Filter.” *European Symposium on Algorithms (ESA)*, 2006.
27. Klink, A., Wälde, J. “Efficient Denial of Service Attacks on Web Application Platforms.” *28th Chaos Communication Congress (28C3)*, 2011.
28. Li, K.-H. “Reservoir-Sampling Algorithms of Time Complexity O(n(1 + log(N/n))).” *ACM Transactions on Mathematical Software* 20(4):481–493, 1994.
29. Maggs, B. M., Sitaraman, R. K. “Algorithmic Nuggets in Content Delivery.” *ACM SIGCOMM Computer Communication Review* 45(3), 2015.
30. Merkle, R. C. “A Digital Signature Based on a Conventional Encryption Function.” *Advances in Cryptology — CRYPTO ’87*, LNCS 293. Springer, 1988.
31. Metwally, A., Agrawal, D., El Abbadi, A. “Efficient Computation of Frequent and Top-k Elements in Data Streams.” *International Conference on Database Theory (ICDT)*, 2005.
32. Niemeyer, G. Geohash (public-domain geocoding system, geohash.org), 2008.
33. OpenJDK. *JEP 180: Handle Frequent HashMap Collisions with Balanced Trees*. https://openjdk.org/jeps/180
34. Python Software Foundation. *PEP 456 — Secure and Interchangeable Hash Algorithm*. https://peps.python.org/pep-0456/
35. Pugh, W. “Skip Lists: A Probabilistic Alternative to Balanced Trees.” *Communications of the ACM* 33(6):668–676, 1990.
36. Redis Documentation. HyperLogLog; Sorted sets; Geospatial; Streams; Count-min sketch; Top-K. https://redis.io/docs/
37. Laurie, B., Langley, A., Kasper, E. *RFC 6962: Certificate Transparency*. IETF, 2013. https://www.rfc-editor.org/rfc/rfc6962
38. Davis, K., Peabody, B., Leach, P. *RFC 9562: Universally Unique IDentifiers (UUIDs)*. IETF, 2024. https://www.rfc-editor.org/rfc/rfc9562
39. RocksDB Wiki. *MemTable*. https://github.com/facebook/rocksdb/wiki/MemTable
40. S2 Geometry. *S2 Geometry Library documentation*. https://s2geometry.io/
41. Sanfilippo, S. “Redis New Data Structure: the HyperLogLog.” antirez.com, 2014.

42. Twitter. *snowflake* (archived repository, tag `snowflake-2010`: README and `IdWorker.scala`). https://github.com/twitter-archive/snowflake
43. Twitter. *Algebird: Abstract Algebra for Scala*. https://github.com/twitter/algebird
44. ULID. *Universally Unique Lexicographically Sortable Identifier specification*. https://github.com/ulid/spec
45. Vitter, J. S. “Random Sampling with a Reservoir.” *ACM Transactions on Mathematical Software* 11(1):37–57, 1985.

17.19 Open verification items

- Twitter’s 2010 “Announcing Snowflake” blog post (author, exact date, URL) could not be fetched; layout, epoch and requirements were verified against the archived GitHub repository (tag `snowflake-2010`) instead.
- Flickr ticket-servers post: confirm author (believed Kay Kremerskothen) and date (February 2010).
- Akamai one-hit-wonder share and the second-request caching rule: confirm wording in Maggs & Sitaraman (2015).
- Counting Bloom filter overflow bound (4-bit counters suffice): confirm statement in Fan et al. (2000).
- Redis sorted-set constants ($p = 1/4$, 32 levels) and listpack thresholds: confirm against current `t_zset.c` / `redis.conf` (as of 2025). RedisBloom bundling in Redis 8: confirm.
- RocksDB ribbon filters and default skip-list memtable: confirm current RocksDB wiki.
- Niemeyer (2008) geohash: no formal publication; confirm citation form.
- xxHash versus MurmurHash3 speed claims: author benchmarks only.

Part IV — Scalability and Performance

Chapter 18 — Scalability

> **Level:** 2 · **Track:** A (Traditional) · **Estimated reading time:** 3.5 h

A system that works for a thousand users does not automatically work for ten million. Somewhere between the two, a CPU saturates, a connection limit is reached, a lock becomes hot or a bill becomes absurd. **Scalability** is the discipline of growing capacity *predictably*: knowing which resource runs out next, what more of it costs, and how long it takes to add. Chapter 2 sketched QuickBite, our hypothetical food-delivery platform, and estimated its load at 2 million daily active users (§2.11). This chapter carries QuickBite from a 1,000-user pilot to 10 million users, one architecture per stage. Along the way you learn the two basic moves (bigger machines or more machines), why the second move requires stateless services, how to find what stops scaling, how autoscalers decide, and how to write a capacity plan with explicit headroom.

Several topics are only summarized here because other chapters own them: load-balancing algorithms (Chapter 19), caching (Chapter 20), replication and sharding (Chapter 14), queueing mathematics (§4.10–§4.13) and the estimation handbook (Chapter 89).

18.1 Learning objectives

After this chapter you should be able to:

- Explain the limits and cost curve of vertical scaling, and name the situations in which it is the right answer.
- State the preconditions for horizontal scaling and design a stateless service by externalizing session and other state.
- Use the resource model and the Universal Scalability Law to predict where adding nodes stops paying off.
- Configure target-tracking autoscaling, choose its signal and target, and size headroom for warm-up delay.
- Check a connection-pool configuration against a database’s connection limit before the autoscaler breaks it.
- Choose a database-scaling option from a decision table and place a design on the X, Y and Z axes of the scale cube.
- Produce a capacity plan with peak-to-average ratios, N+1/N+2 or zone-level redundancy, a growth forecast and lead times.

18.2 Prerequisites

- The resource model, service demand and the utilization law — §2.6 (used to find bottlenecks).
- Back-of-the-envelope estimation and the QuickBite assumptions — §2.7 and §2.11 (the running example).
- Queueing delay versus utilization — §4.10 (why target utilization is below 100%).
- Little's Law — §4.11 (connection and concurrency sizing).
- Amdahl's Law and the Universal Scalability Law — §4.12–§4.13 (applied here, not re-derived).
- Statelessness, cookies and sessions — §8.3.2 and §8.10 (externalized state).
- Connection pooling and PgBouncer — §14.13 (connection budgets).

Units: KB, MB, GB are powers of ten; RPS means requests per second; all QuickBite figures are teaching assumptions, not measurements of a real company.

18.3 Vertical scaling

18.3.1 Scaling up

DEFINITION

Definition — Scalability. The ability of a system to handle a growing workload by adding resources, with throughput rising roughly in proportion to the resources added and latency staying within its objective. A system is *not* scalable for a workload if adding resources yields little or no extra throughput, or if the cost per unit of work rises sharply.

DEFINITION

Definition — Vertical scaling (scaling up). Increasing the capacity of a single node by giving it more or faster CPU cores, memory, storage or network bandwidth, for example by moving a database from an 8-core to a 64-core machine.

First-principles explanation

A bakery with one oven is falling behind. The first fix is obvious: buy a bigger oven. Nothing else changes. The recipes, the staff and the shop layout all stay the same. That is vertical scaling. It works until no bigger oven exists, until the biggest ovens cost far more per loaf than two smaller ones, or until the baker, not the oven, is the limit.

Why does this exist?

Vertical scaling is the cheapest move in *engineering effort*. The software is unchanged, so there is no new failure mode, no data distribution and no coordination protocol to design. A single relational database primary (Chapter 11) holds all its data in one place and gives you transactions and joins for free. Keeping it on one node as long as possible is often the most valuable architectural decision a young product makes.

How it works internally

1. Measure which resource is saturated (§2.6). Scaling up the wrong resource buys nothing.
2. Move to a larger node. In the cloud this is usually a stop, resize and restart, or a failover to a pre-built larger replica. On owned hardware it means adding memory or replacing the server.
3. Re-tune software limits that were sized for the old machine: thread pools, buffer pool size (§13.4), `max_connections`, file-descriptor limits. A 64-core database with a 16-thread pool is still a 16-thread database.
4. Re-measure. The bottleneck may now be a lock, a single-threaded component or memory bandwidth (§5.9, §5.10).

The limits of vertical scaling

Table 18.1 — Where vertical scaling stops.

Limit	What happens	Example
Hardware ceiling	No larger node exists	As of 2026, general-purpose cloud instances top out at a few hundred vCPUs and a few TB of memory
Serial code	Extra cores sit idle	A single-threaded event loop or a global lock caps throughput (Amdahl, §4.12)
Coherency inside the node	Throughput flattens or falls	Cache-line contention and NUMA traffic between sockets (§5.10)
Availability	One node is one failure domain	Resizing needs a restart or failover; the node is a single point of failure
Cost	Price per unit of throughput rises	Large nodes deliver less throughput per dollar (below)
Blast radius	Every user shares the node	One bad query affects everyone

The cost curve

Two effects shape the cost of scaling up. First, **price per core**. Within one cloud instance family, price is roughly proportional to size as of 2026: twice the vCPUs costs about twice

as much. Beyond the largest standard size, specialized high-memory or multi-socket machines carry a premium, and on owned hardware high-end multi-socket servers have long cost more per unit of performance than commodity ones Barroso et al. 2018. Second, **throughput per core** falls as the node grows, because contention and coherency inside the node grow with core count. Combining a linear price with USL throughput (§4.13) gives a simple model of the cost curve.

FORMULA

Relative cost per unit of throughput on one node

$$k(N) = \frac{N}{C(N)} = 1 + \alpha(N - 1) + \beta N(N - 1)$$

Symbol	Meaning	Units
N	Cores (or vCPUs) in the node; price assumed proportional to N	count
$C(N)$	USL relative capacity: throughput at N cores divided by throughput at 1 core	dimensionless
α	Contention coefficient inside the node	dimensionless
β	Coherency coefficient inside the node	dimensionless
$k(N)$	Cost per unit of throughput relative to a 1-core node	dimensionless (×)

The formula is just the USL denominator: if price grows with N and throughput with $C(N)$, the ratio is what is left. It assumes linear pricing; on owned hardware or specialized instances, multiply by the price premium as well.

WORKED EXAMPLE

Worked example — Is the 192-core database node worth it? A load test of QuickBite's order database at several core counts fits $\alpha = 0.02$ and $\beta = 0.00005$ (assumed values for the exercise).

1. $N = 8$: $C = 8/(1 + 0.14 + 0.0028) = 8/1.1428 = 7.00$; $k = 1.14$.
2. $N = 32$: $C = 32/(1 + 0.62 + 0.0496) = 32/1.6696 = 19.17$; $k = 1.67$.
3. $N = 96$: $C = 96/(1 + 1.90 + 0.456) = 96/3.356 = 28.61$; $k = 3.36$.
4. $N = 192$: $C = 192/(1 + 3.82 + 1.8336) = 192/6.6536 = 28.86$; $k = 6.65$.

Interpretation. Doubling from 96 to 192 cores doubles the price and adds less than 1% throughput. Each unit of throughput on the 192-core node costs 6.65 times as much as on a single core, and twice as much as on the 96-core node. **Engineering implication.** Scale up while $k(N)$ is still close to linear (here, up to roughly 32 cores). Past the knee, spend effort on removing contention (shorter transactions, fewer hot rows) or on scaling out. Fit α and β from measurements at several sizes (§23.10); a two-point test cannot reveal the knee (§4.13).

When vertical scaling is the right answer

- **The workload needs a single coherent copy of state**, such as a transactional database primary, a lock manager or an in-memory matching engine. Splitting it would add coordination that costs more than the bigger node.
- **You are below the knee of the cost curve.** Going from 4 to 16 cores typically costs little in efficiency.
- **Time matters more than elegance.** Resizing takes minutes to hours; sharding takes months. Scaling up buys time to build the horizontal design properly.
- **The team is small.** One large node has far fewer moving parts to operate than ten small ones.
- **Per-node software licences** are not a factor, or favor fewer, larger nodes.

INDUSTRY EXAMPLE

Stack Overflow. Stack Overflow has publicly described serving its very large Q&A traffic with a small number of servers, including a handful of SQL Server machines with very large memory and on the order of ten web servers, preferring to scale up and to optimize code rather than to distribute the database. *Evidence: Primary* — [Craver 2016]

COMMON MISTAKE

"Vertical scaling is old-fashioned; real systems scale out." Most successful systems scaled up first and still run their most coherent component (often the primary database) on one large node. The mistake is not scaling up; it is scaling up *past the knee* without a plan for what comes next.

18.4 Horizontal scaling

18.4.1 Scaling out

DEFINITION

Definition — Horizontal scaling (scaling out). Increasing capacity by adding more nodes that share the workload, for example by running 20 identical application servers behind a load balancer instead of one. Removing nodes when load falls is **scaling in**.

First-principles explanation

The bakery opens a second shop with its own oven and staff, then a third. Each shop is ordinary and cheap, and the chain can keep opening shops. But now customers need to be sent to a shop with spare capacity, every shop needs the same recipes, and anything that must be shared (the one supplier of flour, the single account book) becomes the new limit.

Why does this exist?

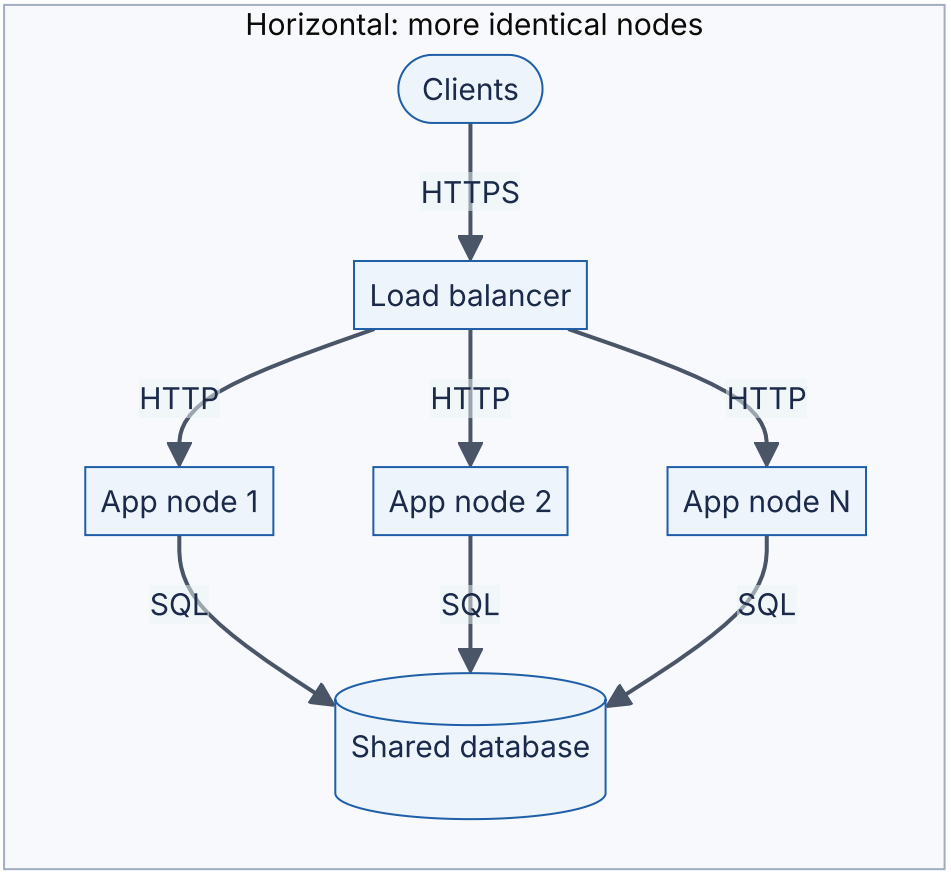
Horizontal scaling removes the hardware ceiling, keeps cost per unit of throughput nearly constant while the work is independent, and turns failures from outages into reduced capacity: losing one of 20 nodes removes 5% of capacity. Barroso, Hölzle and Ranganathan argue that warehouse-scale computers are built from large numbers of commodity servers for exactly these reasons: cost efficiency and tolerance of individual failures through software Barroso et al. 2018.

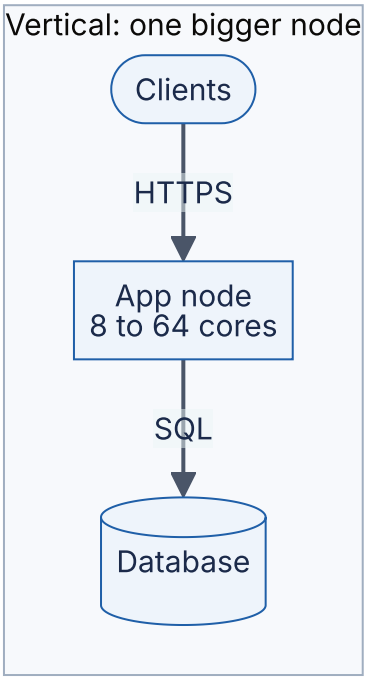
How it works internally

1. **Make nodes interchangeable.** Any node can serve any request, which requires that no node holds state that others need (§18.5).
2. **Distribute the load.** A load balancer, DNS or the client spreads requests across nodes (§18.6, Chapter 19).
3. **Detect and remove failed nodes.** Health checks stop traffic to a node that fails (§19.5).

4. **Add and remove nodes automatically** from a template (a machine image or container image) as load changes (§18.8).
5. **Push shared state down to a tier that scales differently**: a database, cache or partitioned store (§18.9).

Figure 18.1 — Vertical versus horizontal scaling.





Reading the diagram. On the left, clients reach one large node over HTTPS; capacity grows only by enlarging it, and it is a single failure point. On the right, a load balancer spreads HTTP requests over *N* identical nodes, and the others absorb a failed node's share. All arrows are synchronous. The storage boundary in both is the database, and on

the right all *N* nodes converge on it: scaling out the application tier *moves* the bottleneck to the shared tier (§18.7), which scales separately (§18.9).

Advantages, disadvantages and alternatives

Table 18.2 — Vertical and horizontal scaling compared.

	Vertical scaling	Horizontal scaling
What it is	Bigger node	More nodes
How it works	Resize, restart or fail over	Load balancer plus identical, replaceable nodes
Advantages	No code change; transactions and joins stay local	No hardware ceiling; failures reduce capacity instead of causing outages
Disadvantages	Ceiling; single failure domain; cost knee	Needs stateless design, distribution, coordination for shared state
Performance	Lowest latency (no network hops between parts)	Near-linear throughput while work is independent
Operational complexity	Low	Medium (fleet, deploys, autoscaling) to high (distributed data)
Cost	Linear, then superlinear past the knee	Roughly linear, plus coordination overhead
Choose when	Coherent state; below the knee; urgent relief	Stateless tiers; growth beyond one node; high availability

In practice you use both: scale out the stateless tiers, scale up the stateful tier until it reaches its knee, then partition it (Chapter 14).

Failure scenarios

Table 18.3 — Horizontal scaling failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Throughput flat after adding nodes	Bottleneck is a shared tier	Per-tier utilization; USL fit	Scale or relieve the shared tier	Find the bottleneck before scaling (§18.7)
Users lose sessions on scale-in	State held in node memory	Logouts after deploys or scale-in	Move state out	Stateless design (§18.5)
New nodes overloaded on arrival	Cold caches, JIT warm-up	High p99 on young nodes	Slow-start traffic ramp	Warm-up and readiness probes (§18.8)
Database connection limit hit	Every node opens its own pool	`too many clients` errors	Pooler; smaller pools	Connection budget (§18.8.5)

COMMON MISTAKE

"Adding nodes always adds capacity." Only while the work is independent. With contention or coherency, throughput flattens and then falls (§4.13). Measure the gain from each added node (§18.7).

18.5 Stateless vs stateful services

18.5.1 What "stateless" really means

DEFINITION

Definition — Stateless service. A service whose instances keep no client-specific or request-spanning data in local memory or disk that another instance would need. Any instance can serve any request, and an instance can be killed at any moment without losing anything except the requests it is processing (§8.3.2).

DEFINITION

Definition — Stateful service. A service whose instances hold data that must survive between requests and that clients or other instances depend on: a database, a cache node, a WebSocket server holding live connections, or an application server keeping sessions in memory.

First-principles explanation

Chapter 8 used two waiters: one memorizes your order, the other pins it to a shared board. The second waiter is stateless *because the board exists*. State did not disappear; it moved to a place built to hold it. "Stateless" is therefore a property of one tier, bought by making another tier stateful on purpose.

Why does this exist?

Every scaling move in this chapter (adding nodes, removing nodes, replacing a failed node, rolling out a new version, shifting traffic between zones) is trivial for a stateless tier and hard for a stateful one. Concentrating state in a few tiers that are designed for it (databases, caches, session stores) leaves the large, expensive compute tier free to scale elastically.

How it works internally: where state goes

Table 18.4 — Externalizing state from an application server.

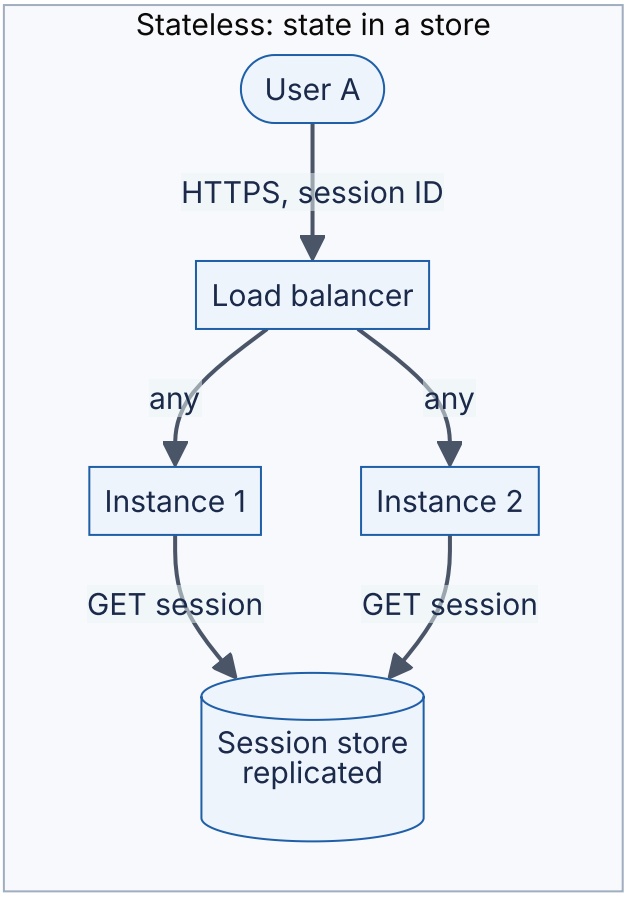
State in the server	Where it moves	Cost per request	See
Login session	Shared session store keyed by session ID, or a signed token in the client	~0.5–1 ms store lookup, or signature check	§8.10, §8.13
Shopping cart, draft order	Database or session store	One read or write	Chapter 11
Cached query results	Shared cache tier	~0.5 ms network round trip	Chapter 20
Uploaded files in progress	Object storage, multipart or presigned upload	Direct client upload	§15.8
Scheduled jobs, timers	Queue or job scheduler	Asynchronous	Chapter 30
Rate-limit counters	Shared counter store	One atomic increment	§22.8
Local disk logs	Log shipping to a central store	Asynchronous	Chapter 40

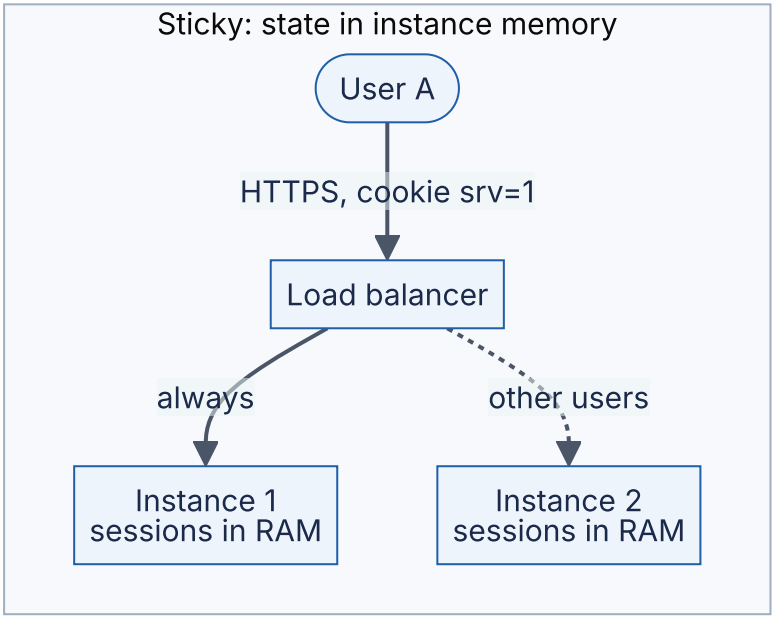
Some local state is harmless: a **read-only cache** that can be rebuilt (a menu cache with a TTL), compiled code and configuration loaded at start. The test is simple: *if this instance vanished now, would any user or other instance notice anything beyond a retry?* If not, the instance is stateless for scaling purposes.

Sticky sessions

Session affinity, also called **sticky sessions**, makes the load balancer send all requests from one client to the same instance, usually with a cookie or a hash of the client address (§19.6). It lets a stateful server *look* stateless to the load balancer.

Figure 18.2 — Sticky sessions versus an externalized session store.





Reading the diagram. On the left, the load balancer pins User A to instance 1 by a cookie, and A's session lives only in instance 1's memory. If scale-in, a deploy or a crash removes instance 1, A is logged out; heavy users stay stuck on an overloaded instance. On the right, any instance can serve any request and fetches the session synchronously from a replicated session store (storage boundary) in about a millisecond. The store becomes a

critical dependency that must be replicated (Chapter 20), but instances can be added, drained or killed freely, and scale independently of session capacity.

Table 18.5 — Ways to handle session state.

	Sticky sessions	Shared session store	Client-side token (signed cookie, JWT)
How it works	LB pins client to instance	Instances look up session by ID	Client carries signed state
Advantages	No code change; fastest local access	Instances disposable; revocation is one delete	No server lookup; no store to run
Disadvantages	Session loss on instance loss; uneven load; blocks scale-in	Extra hop; store must be highly available	Size limits; revocation is hard before expiry
Choose when	Short-term migration of a legacy app	Default for server-rendered and API sessions	Short-lived access tokens (§8.13)

Affinity is not always a smell. It is a legitimate **performance** optimization when the state is a rebuildable cache, for example routing a user to the instance that already holds their data warm, provided correctness never depends on it. Long-lived connections such as WebSockets are inherently stateful at the connection level; they are scaled with connection-aware techniques (§9.9).

Failure scenarios

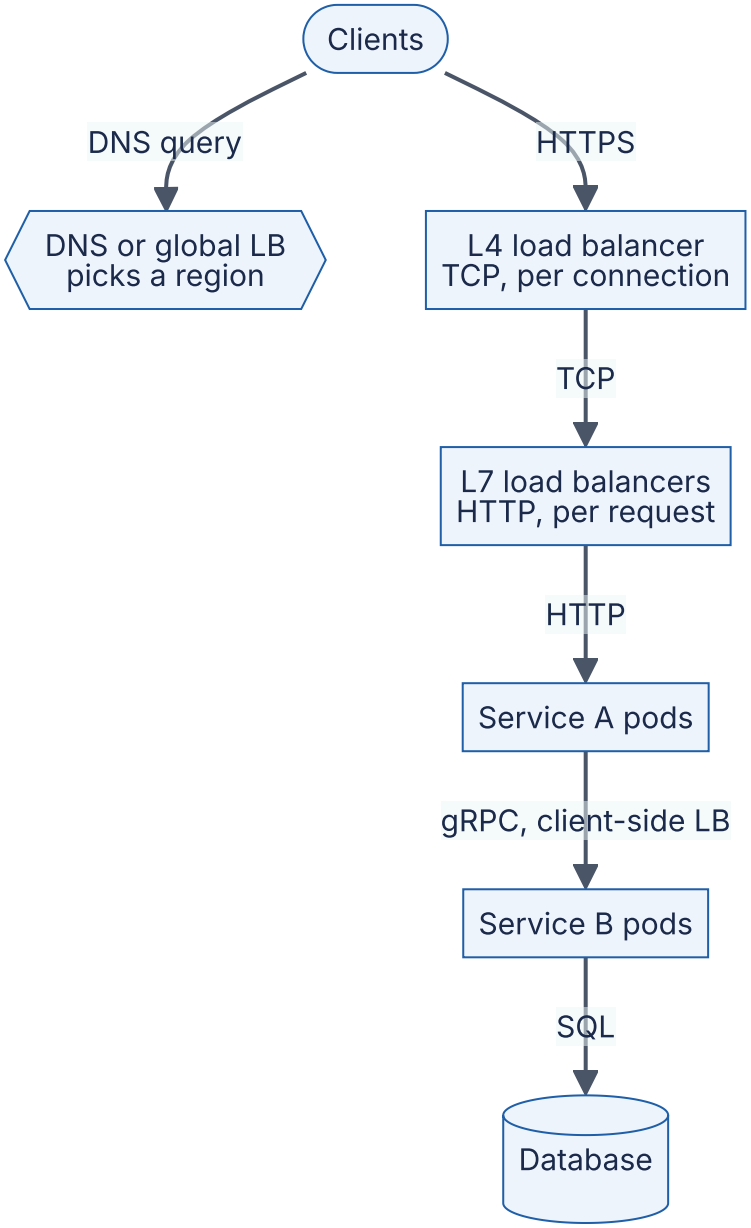
The classic state-related failures (mass logout on deploy, sessions lost at scale-in, a session-store outage that logs everyone out, a hot instance under affinity) appear in Table 8.1 and Table 18.3. All share one prevention: decide where each piece of state lives *before* the tier needs to scale, and replicate that place.

> COMMON MISTAKE
>
> **"Stateless means the system has no state."** It means *this tier* keeps none between requests. The engineering work is choosing where state lives and making that place durable, available and fast enough.

18.6 Load distribution

Horizontal scaling needs something to spread requests across nodes. This section is a map; Chapter 19 is the territory.

Figure 18.3 — Layers of load distribution.



Reading the diagram. DNS or global server load balancing (hexagon) steers each client to a region, at the granularity of DNS answers and their TTLs (§19.9). A **layer-4 load balancer** spreads TCP connections without reading requests; a **layer-7 load balancer** routes each HTTP request by path, header or cookie (§19.3). Inside, **client-side load balancing** lets service A pick a pod of service B from a registry (§19.10). All arrows are synchronous. Failure points: the load balancers, made redundant with pairs, anycast or ECMP (§19.8), and stale DNS answers. Each layer scales horizontally; the database at the bottom does not, by this mechanism.

The algorithm matters most when requests differ in cost. Round robin suits uniform requests; least-outstanding-requests and **power of two choices** avoid piling work on a slow node; consistent hashing keeps related requests together for cache locality (§14.8, §19.4).

Good algorithms approximate one shared queue in front of all servers, which beats separate queues (Figure 4.4).

18.7 Bottleneck identification

18.7.1 Where scaling stops

DEFINITION

Definition — Scaling bottleneck. The resource that limits throughput as load or node count grows: the resource with the largest service demand per unit of capacity (§2.6). For a horizontally scaled system, it is usually a resource that *all* nodes share and that cannot itself be scaled out by adding application nodes.

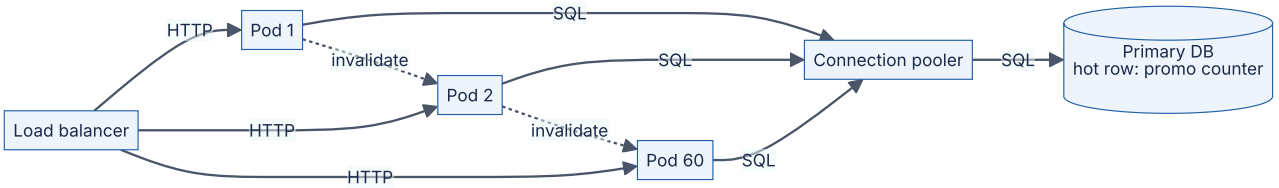
First-principles explanation

Adding checkout lanes to a supermarket helps until every lane waits for the single price-check clerk. The clerk was always there; with two lanes she was idle most of the time, and with twenty she is the whole story. Horizontal scaling multiplies the parts you replicate and leaves untouched the parts you share, so the shared parts eventually dominate.

How it works internally

1. **List the resources on the request path** using the resource model of Table 2.7: CPU, memory, disk, network and software resources (locks, pools, single-threaded components).
2. **Classify each as per-node or shared.** Per-node resources (application CPU) grow with N; shared ones (the database primary, a global counter, a leader, a license server, a third-party rate limit) do not.
3. **Compute utilization at the target load** with the utilization law $U = XD/c$ (§2.6). The largest is the bottleneck now.
4. **Ask what grows with N.** Connection counts, health-check traffic, cache-invalidation fan-out and membership gossip all grow with node count, sometimes as N^2. These are the coherency terms of the USL.
5. **Fit the USL** to load-test throughput at several node counts (§4.13, §23.10) [Gunther 2007]. Its coefficients tell you whether you face contention (α), coherency (β) or neither.
6. **Relieve, then repeat.** The bottleneck moves.

Figure 18.4 — The shared-resource funnel.



Reading the diagram. Sixty pods scale out freely behind the load balancer (synchronous HTTP) but converge, through a pooler, on one primary database and one hot row: a promotion counter that every order updates under a lock, a *contention* cost (USL α). The dotted arrows are asynchronous cache invalidations between pods; their number grows with pod pairs, a *coherency* cost (USL β). Storage boundary: the database. Failure point: the hot row saturates while pod CPUs look idle. Adding pods worsens both costs.

Mathematical foundations

The USL itself is in §4.13. For a capacity decision you need one derived quantity: what the *next* node buys.

FORMULA

Marginal throughput of the N-th node

$$\Delta X(N) = X_1\,[C(N) - C(N-1)] \qquad E(N) = \frac{C(N)}{N}$$

Symbol	Meaning	Units
X_1	Throughput of one node (measured)	requests/s
$C(N)$	USL relative capacity at N nodes (§4.13)	dimensionless
$\Delta X(N)$	Extra throughput gained by adding node N	requests/s
$E(N)$	Scaling efficiency: fraction of linear scaling achieved	dimensionless (0–1)

WORKED EXAMPLE

Worked example — How many pods can QuickBite usefully run? A load test of the order API, where every order updates a promotion counter row and pods broadcast invalidations, fits $\alpha = 0.03$ and $\beta = 0.0002$. One pod sustains $X_1 = 800$ RPS. The product team wants 15,000 RPS for a national promotion.

1. Peak concurrency: $N^* = \sqrt{(1 - 0.03)/0.0002} = \sqrt{4,850} = 69.6$ pods.
2. Maximum throughput:
   $C(70) = 70/(1 + 0.03 \times 69 + 0.0002 \times 70 \times 69) = 70/(1 + 2.07 + 0.966) = 70/4.036 = 17.34$, so
   $X_{max} \approx 17.34 \times 800 = 13,875$ RPS.
3. Marginal gain at 40 pods: $C(40) = 16.116$ and $C(39) = 16.008$, so
   $\Delta X(40) = 800 \times 0.108 \approx 87$ RPS, about one-ninth of what the first pod delivered.
4. Efficiency at 40 pods: $E(40) = 16.116/40 = 0.40$. You pay for 40 pods and get the work of 16.

Interpretation. No number of pods reaches 15,000 RPS; the system peaks at about 13,900 RPS near 70 pods and degrades beyond. **Engineering implication.** Stop scaling out and remove the shared costs: split the counter into per-shard sub-counters that are summed periodically (§14.10), and replace pod-to-pod invalidation with a TTL or a pub/sub channel (§20.10). Re-test and re-fit; if α falls to 0.005 and β to 0.00001, N^* rises to about 315 pods.

WARNING / TRADE-OFF

The autoscaler cannot see the knee. A CPU-based autoscaler adds pods whenever pods are busy. Past N^*, extra pods add contention, latency rises, pods stay busy, and the autoscaler adds still more. Cap `maxReplicas` below the measured knee and alert when the cap is reached.

COMMON MISTAKE

"Low utilization everywhere means we can scale." If latency rises with load while every hardware dashboard is calm, the bottleneck is a software resource: a lock, a pool or a single-threaded component (§2.6).

18.8 Scaling application servers

18.8.1 Preconditions

A stateless application tier scales out only if four things are true:

1. **Instances are disposable**: built from an immutable image, configured from the environment, holding no state (§18.5).
2. **Readiness is explicit.** A **readiness probe** tells the load balancer when a new instance can take traffic, separately from a **liveness probe** that says whether it should be restarted.
3. **Shutdown is graceful.** On a termination signal the instance stops accepting new requests, finishes in-flight ones within a deadline (**connection draining**), closes pools and exits.
4. **Dependencies can take the extra load.** Each new instance opens connections and sends queries to shared tiers (§18.8.5).

18.8.2 Autoscaling and its signals

DEFINITION

Definition — Autoscaling. Automatically changing the number of instances of a service in response to a measured or forecast signal, within configured minimum and maximum bounds. **Reactive** autoscaling responds to current metrics; **scheduled** autoscaling follows a calendar; **predictive** autoscaling acts on a forecast.

A thermostat is the right analogy: it measures the room, compares it with a set point and switches the heating on or off. Like a thermostat, an autoscaler reacts *after* the temperature changes and the heater takes time to warm up, so the set point must leave room for that delay.

Table 18.6 — Autoscaling signals.

Signal	Good for	Weakness
CPU utilization	CPU-bound stateless services	Misses I/O-bound services whose threads wait idle; noisy
Requests per second per instance	Uniform request cost; needs a measured per-instance capacity	Breaks when the request mix changes
In-flight requests (concurrency)	I/O-bound services; follows Little's Law ($L = \lambda W$)	Rises when a dependency slows, which more instances may not fix
Queue depth or consumer lag per worker	Asynchronous workers (Chapter 30)	Needs a target derived from the latency objective
Latency (p95, p99)	Guard rail and alerting	Lagging, nonlinear, and caused by dependencies as often as by load
Schedule	Known daily or weekly peaks	Misses surprises
Forecast	Ramps faster than reaction time	Wrong when the forecast is wrong

The best primary signal is one that is **proportional to load and inversely proportional to instance count**: doubling the instances should halve it. CPU and per-instance request rate usually qualify; latency usually does not.

18.8.3 Target tracking

Target tracking keeps a per-instance metric near a set point by resizing the fleet in proportion to the error. It is the algorithm of the Kubernetes Horizontal Pod Autoscaler (HPA) and of AWS target-tracking policies [Kubernetes Docs: HPA], [AWS Docs: EC2 Auto Scaling].

FORMULA

Target-tracking replica count (Kubernetes HPA)

$$n_{desired} = \lceil n_{current} \times \frac{m_{current}}{m_{target}} \rceil \quad \text{if } |\frac{m_{current}}{m_{target}} - 1| > \tau$$

Symbol	Meaning	Units
$n_{current}$	Current number of replicas	pods
$n_{desired}$	Replica count the autoscaler requests, clamped to [min, max]	pods
$m_{current}$	Current per-replica metric, averaged over replicas	metric units (e.g. % CPU, RPS)
m_{target}	Target per-replica metric	same units
τ	Tolerance band within which no action is taken (default 0.1 as of 2026)	dimensionless

WORKED EXAMPLE

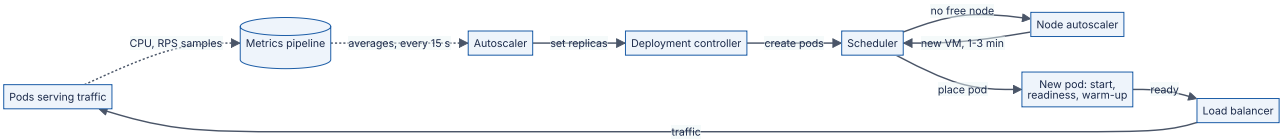
Worked example — Two HPA decisions for QuickBite.

CPU target. 10 replicas average 90% CPU against a 60% target. Ratio $= 90/60 = 1.5$, outside the ±0.1 band. $n_{desired} = \lceil 10 \times 1.5 \rceil = 15$. After scaling, the same load spread over 15 pods averages $90 \times 10/15 = 60\%$, exactly the target.

Request-rate target. 14 pods average 650 RPS against a target of 500 RPS per pod (assuming a measured 830 RPS per pod at the p99 objective, so 500 is about 60%). Ratio $= 1.3$; $n_{desired} = \lceil 14 \times 1.3 \rceil = \lceil 18.2 \rceil = 19$. If instead they averaged 530 RPS, the ratio 1.06 is inside the band and nothing happens.

Interpretation. The rule is proportional: double the per-pod metric, double the pods. The tolerance stops it reacting to noise. **Engineering implication.** Choose m_{target} from a load test, not from intuition: it is the per-instance throughput at which latency still meets the objective, times a safety factor for reaction time (next section).

Figure 18.5 — The autoscaling control loop.



Reading the diagram. Pods report metrics asynchronously (dotted) to a metrics pipeline, the storage boundary for recent samples. The autoscaler reads averages periodically (every 15 s by default in Kubernetes) and sets a new replica count; the deployment controller creates pods and the scheduler places them. If no node has room, a node autoscaler must first create a virtual machine, typically adding one to several minutes. The new pod pulls its image, starts and passes readiness before receiving traffic, often with cold caches. Failure points: stale metrics, image-registry outages, exhausted quotas. The sum of these delays is the **reaction time** that headroom must cover.

Flapping and scale-in

Scaling in too eagerly causes **flapping**: remove pods, utilization rises, add pods, repeat. Autoscalers therefore scale in slowly. The Kubernetes HPA, for example, uses by default the highest recommendation of the last five minutes when scaling down (a **stabilization window**) [Kubernetes Docs: HPA]. Scale up fast, scale down slowly: the cost of a few spare pods for five minutes is small; the cost of being under capacity during a rebound is an incident.

18.8.4 Warm-up and reaction time

A new instance is not useful the moment it is requested. Its delay is the sum of metric collection, the evaluation period, possibly provisioning a machine, pulling the image, starting the process, passing readiness, and **warm-up**: the period in which caches are empty, connection pools are filling and just-in-time compilers have not yet optimized hot code. During all of this the *existing* instances absorb the growing load.

FORMULA

Target utilization that survives reaction time

$$u_{target} \le \frac{u_{safe}}{1 + r \times T_{react}}$$

Symbol	Meaning	Units
u_{target}	Utilization set point for the autoscaler	fraction
u_{safe}	Highest utilization at which latency still meets the objective (from load tests and §4.10)	fraction
r	Fastest expected load growth, relative to current load	fraction per minute
T_{react}	Reaction time: metric delay + evaluation + provisioning + start + warm-up	minutes

The derivation is one line: during T_{react} load grows by a factor of about $1 + rT_{react}$ while capacity stays fixed, so utilization must start that factor below u_{safe}. It is a linear approximation, good when rT_{react} is below about 0.5.

WORKED EXAMPLE

Worked example — Choosing QuickBite's CPU target. Load tests show p99 stays within objective up to $u_{safe} = 0.85$. Dinner ramps grow load by up to $r = 15\%$ per minute. Reaction time: metric delay 0.5 min, evaluation 0.25 min, pod start and readiness 1 min, warm-up equivalent to 0.5 min of lost capacity, so $T_{react} = 2.25$ min.

1. $1 + rT_{react} = 1 + 0.15 \times 2.25 = 1.3375$.
2. $u_{target} \le 0.85/1.3375 = 0.636$. Choose **60%**.
3. If the cluster has no spare nodes and a node autoscaler adds 2 min: $T_{react} = 4.25$, $1 + 0.6375 = 1.6375$, $u_{target} \le 0.52$. Choose **50%**, which means about 20% more pods at all times.

Interpretation. Slow provisioning is paid for with permanently idle capacity. **Engineering implication.** Shorten T_{react} instead of lowering the target: keep a small buffer of spare nodes (low-priority placeholder pods that real pods can evict), use pre-initialized warm pools, slim images, and send a new instance a slowly increasing share of traffic (**slow start**) while it warms.

The simulation below checks the formula against a model of the HPA with startup and warm-up delays. Load doubles from 6,000 to 12,000 RPS over five minutes, a growth of 10–20% per minute.

```
        """Target-tracking autoscaler with startup delay and warm-up (Section 18.8).

        A simplified model of the Kubernetes HPA rule
            desired = ceil(current * current_metric / target_metric)
        applied every 15 s to CPU utilization. New pods serve nothing for STARTUP_S sec-
onds,
        then serve at half capacity for WARMUP_S seconds (cold caches, JIT compilation).
        Load ramps linearly from 6,000 to 12,000 RPS over five minutes.
        """
        import math

        POD_RPS = 1_000        # requests/s one warm pod handles at 100% CPU
        STARTUP_S = 60         # scheduling + image pull + process start + readiness
probe
        WARMUP_S = 60          # serving at half capacity while caches and JIT warm up
        SYNC_S = 15            # autoscaler evaluation period
        METRIC_LAG_S = 30      # age of the utilization sample the autoscaler sees
        TOLERANCE = 0.10       # no action while |ratio - 1| <= 10%
        RAMP_S = 300           # load doubles over five minutes, starting at t = 60 s


        def load(t):
            return 6_000 + 6_000 * min(max(t - 60, 0), RAMP_S) / RAMP_S


        def run(target):
            pods = [-10_000] * math.ceil(6_000 / (POD_RPS * target))   # start times;
all warm
            worst, overloaded = 0.0, 0
            for t in range(0, 1_200):
                cap = 0.0
                for start in pods:
                    age = t - start
                    if age >= STARTUP_S + WARMUP_S:
                        cap += POD_RPS
                    elif age >= STARTUP_S:
                        cap += POD_RPS / 2
                util = load(t) / cap
                worst = max(worst, util)
                overloaded += util > 1.0
                if t % SYNC_S == 0 and t >= METRIC_LAG_S:
                    # Average CPU over all pods as the autoscaler sees it; pods that are
                    # not ready count as idle, as the HPA assumes when scaling up.
                    seen = load(t - METRIC_LAG_S) / (len(pods) * POD_RPS)
                    ratio = seen / target
                    if abs(ratio - 1) > TOLERANCE:
                        desired = math.ceil(len(pods) * ratio)
                        if desired > len(pods):
                            pods += [t] * (desired - len(pods))
            return worst, overloaded, len(pods)


        print("target  peak util  seconds >100%  final pods")
        for target in (0.5, 0.6, 0.7, 0.8):
```

```
        worst, over, final = run(target)
        print(f"{target:5.0%}   {worst:8.0%}   {over:12d}   {final:9d}")
```

Output:

```
target  peak util  seconds >100%  final pods
  50%        74%              0          23
  60%        88%              0          19
  70%       102%              9          17
  80%       116%            184          15
```

What to notice:

- At an 80% target the fleet spends about three minutes above 100% utilization, during which queues grow without bound (§4.10). At 60% the worst moment is 88%, close to $u_{safe} = 0.85$. That is consistent with the formula, because this ramp starts at 20% per minute, a little faster than the 15% assumed in the worked example.
- The final pod counts differ by only 8 pods (23 versus 15), but the overload differs by minutes. Headroom is cheap insurance against reaction time.
- The model ignores backlog carried over from overloaded seconds, so real overload lasts longer than shown.

> INDUSTRY EXAMPLE
>
> **Predictive autoscaling.** Netflix described Scryer, a system that forecasts each service's traffic from historical daily and weekly patterns and scales capacity *ahead* of the predicted demand, with reactive autoscaling kept as a safety net for surprises. Cloud providers offer similar predictive scaling for virtual-machine groups. *Evidence: Primary* — [Netflix 2013], [AWS Docs: EC2 Auto Scaling]

18.8.5 Connection pools versus database limits

Every application instance keeps a **connection pool** to the database (§7.16, §14.13). Pool size is set per instance, but the database's connection limit is set per database. Horizontal scaling multiplies the first and leaves the second fixed. PostgreSQL runs one backend process per connection, reserves a few slots for superusers, and refuses the rest with `FATAL: sorry, too many clients already` [PostgreSQL Docs: Connections].

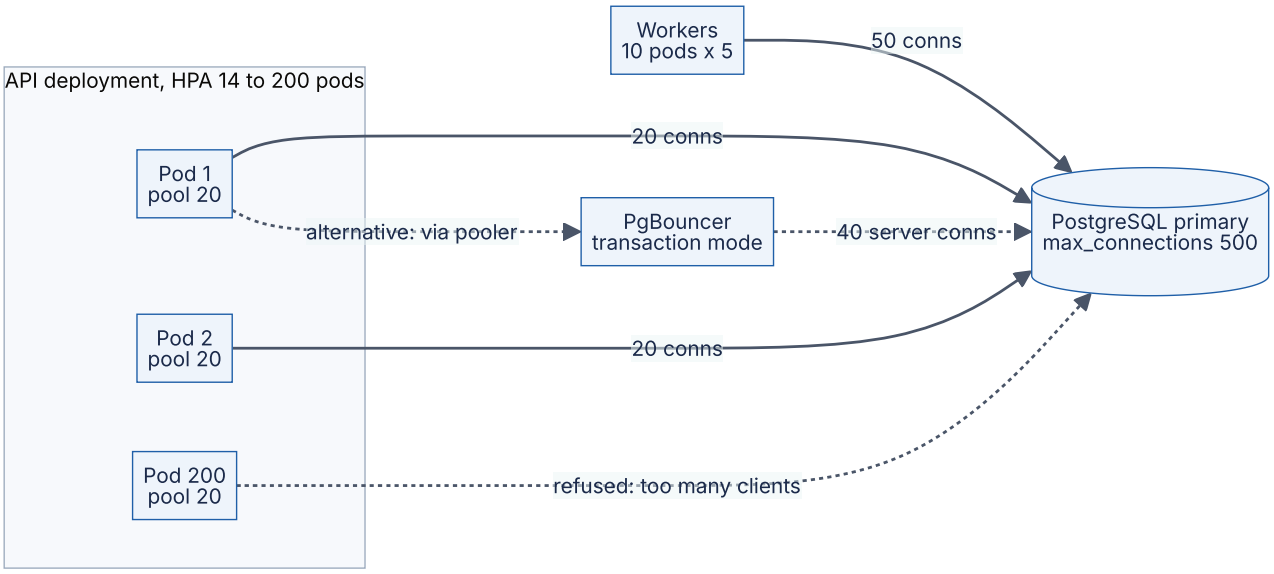
FORMULA

Connection budget

$$n_{max} \times p + \sum_{j} n_j p_j \leq M - R$$

Symbol	Meaning	Units
n_{max}	Maximum replicas of the scaled service (autoscaler `maxReplicas`)	pods
p	Maximum pool size per replica	connections
n_j, p_j	Replicas and pool size of every other client service j	pods, connections
M	Database `max_connections`	connections
R	Connections reserved for superusers, replication, monitoring and migrations	connections

Figure 18.6 — Two hundred pods against one connection limit.



Reading the diagram. The API deployment grows from 14 to 200 pods with pools of up to 20, and workers hold another 50 connections. All speak the PostgreSQL protocol synchronously to one primary whose `max_connections` of 500 is the hard limit (storage boundary). The dotted edge from pod 200 shows connections refused once the budget is spent. The dotted path through PgBouncer is the alternative: the pooler multiplexes client connections onto about 40 server connections (§14.13), decoupling pod count from database connections. Failure point: the limit is reached exactly when traffic peaks and the autoscaler is adding pods.

WORKED EXAMPLE

Worked example — QuickBite's promotion-night outage, on paper. PostgreSQL primary: $M = 500$. Reserved: 3 superuser slots plus 27 for replication, monitoring and migrations, so $R = 30$. Workers: $10 \times 5 = 50$. API pods: pool 20, HPA from 14 to 200 replicas.

1. Budget for the API: $500 - 30 - 50 = 420$ connections.
2. At the minimum of 14 pods: $14 \times 20 = 280$. Fits.
3. The autoscaler crosses $420/20 = 21$ pods during a normal dinner peak. Pod 22 cannot fill its pool; its readiness check fails or its requests error. CPU stays high, so the autoscaler adds more pods, which also fail.
4. At the maximum of 200 pods: $200 \times 20 = 4,000$ connections, 9.5 times the budget.
5. What the database actually needs, by Little's Law (§4.11): at peak about 4,000 queries/s that reach the database (most browse reads are served by the cache) at about 2 ms each, plus 60 order transactions/s at about 15 ms, gives $4,000 \times 0.002 + 60 \times 0.015 = 8 + 0.9 \approx 9$ busy connections on average. With 3× burst headroom, about 30.

Three fixes.

- **Shrink the pool:** $p = \lfloor 420/200 \rfloor = 2$. At 14 pods each pod then has 2 connections for about $9/14 = 0.64$ busy on average: workable, but a single slow query blocks half a pod's pool.
- **Cap the autoscaler:** `maxReplicas` = 21. Safe for the database, but caps the API tier at 21 pods even when the bottleneck is CPU.
- **Add a transaction-mode pooler** (§14.13): pods keep pools of 10 to PgBouncer (2,000 cheap client connections), and PgBouncer holds 40 server connections, comfortably above the 30 needed. Pod count and database connections are now independent.

Interpretation. The configuration that worked at 14 pods was a latent outage at 22. **Engineering implication.** Check the connection budget whenever you change `maxReplicas`, pool size or the number of client services, and put a pooler in front of PostgreSQL before the API tier autoscales. Size the *server* side of the pool from Little's Law, not from pod count.

Failure scenarios

Table 18.7 — Application-tier scaling failures.

What goes wrong	Why	Detection	Recovery	Prevention
Overload during ramps	Reaction time longer than the headroom covers	Utilization > u_{safe} while scaling	Shed load (§22.9)	Lower target; shorten T_{react}; scheduled scaling
Runaway scale-out	Bottleneck downstream keeps pods busy	Replicas at max; DB saturated	Cap replicas; fix bottleneck	`maxReplicas` below the knee
Connection exhaustion	Pods × pool > `max_connections`	`too many clients` errors	Lower pool; restart pooler	Connection budget; transaction pooler
Flapping	Scale-in too fast	Oscillating replica count	Lengthen stabilization	Scale down slowly
Cold-start storms	Many new pods with empty caches hit the DB at once	DB load spike after scale-out	Slow start	Warm caches; shared cache tier
Quota or capacity exhaustion	Cloud account or zone out of instances	Pending pods; API errors	Other instance types or zones	Quota reviews in the capacity plan

COMMON MISTAKE

"Autoscaling replaces capacity planning." An autoscaler only moves between `minReplicas` and `maxReplicas`, using nodes, quotas and database connections that someone had to plan. It handles hourly variation, not next year's growth.

18.9 Scaling databases (overview)

Once the application tier is stateless, the database is usually the bottleneck, because it is the tier that holds the shared, coherent state. Chapters 11–14 treat its scaling options in depth. This section is a decision aid: start from the symptom, try the cheapest option that addresses it, and escalate only when measurements say so.

Table 18.8 — Database scaling options: a decision table.

Option	Fixes this symptom	Adds read capacity	Adds write capacity	Adds storage	Cost and complexity	See
Query and index tuning	Slow queries, full scans	Yes	Yes	No	Low	§11.7–§11.10
Connection pooler	Too many connections; per-connection memory	Indirectly	Indirectly	No	Low	§14.13
Vertical scaling	Any resource saturated, below the cost knee	Yes	Yes	Yes	Low, until the knee	§18.3
Cache in front of the DB	Read-heavy, repetitive reads	Yes (large)	No	No	Medium: invalidation	Chapter 20
Read replicas	Read-heavy; analytics on the primary	Yes	No	No	Medium: replication lag	§14.3–§14.4
Functional split (Y axis)	Unrelated tables compete for one server	Yes	Yes	Yes	Medium: no cross-DB joins	§18.10
Storage engine choice	Write-heavy ingest; space amplification	Depends	Yes (LSM)	Depends	Medium	§13.7–§13.11
Partitioning or sharding (Z axis)	Writes or data beyond one primary	Yes	Yes	Yes	High: shard key, resharding, cross-shard queries	§14.6–§14.12
Distributed SQL (NewSQL)	Need SQL and transactions beyond one node	Yes	Yes	Yes	High: latency, cost	§12.11
NoSQL key-value or wide-column store	Simple access patterns at very high scale	Yes	Yes	Yes	Medium to high: data-model redesign	Chapter 12
Separate read model (CQRS, search index)	Queries the primary schema serves badly	Yes	No	Yes	Medium to high: asynchronous updates	§32.4, Chapter 16
Archiving and tiering	Table and index growth from old data	Indirectly	Indirectly	Frees space	Low to medium	§15.9

Three rules summarize the table. **Reads and writes scale differently:** caches and replicas multiply reads; only partitioning or a bigger primary multiplies writes, so measure the read-to-write ratio first (117:1 for QuickBite browsing, §2.11). **Cheap fixes first:** index-

ing, pooling, caching and a bigger node often buy an order of magnitude together, while sharding is a one-way door (§2.8) that costs months. **Plan the hard move early:** §18.11.4 turns this into a lead-time calculation.

18.10 The scale cube

18.10.1 Three axes of scaling

DEFINITION

Definition — Scale cube. A model by Martin Abbott and Michael Fisher that classifies every scaling technique along three independent axes [Abbott and Fisher 2015]: the **X axis**, cloning identical copies of a service or its data behind a load balancer; the **Y axis**, splitting by function, service or resource so that each part handles different work; and the **Z axis**, splitting by customer, requester or other key so that each part handles the same work for a subset of the data or users.

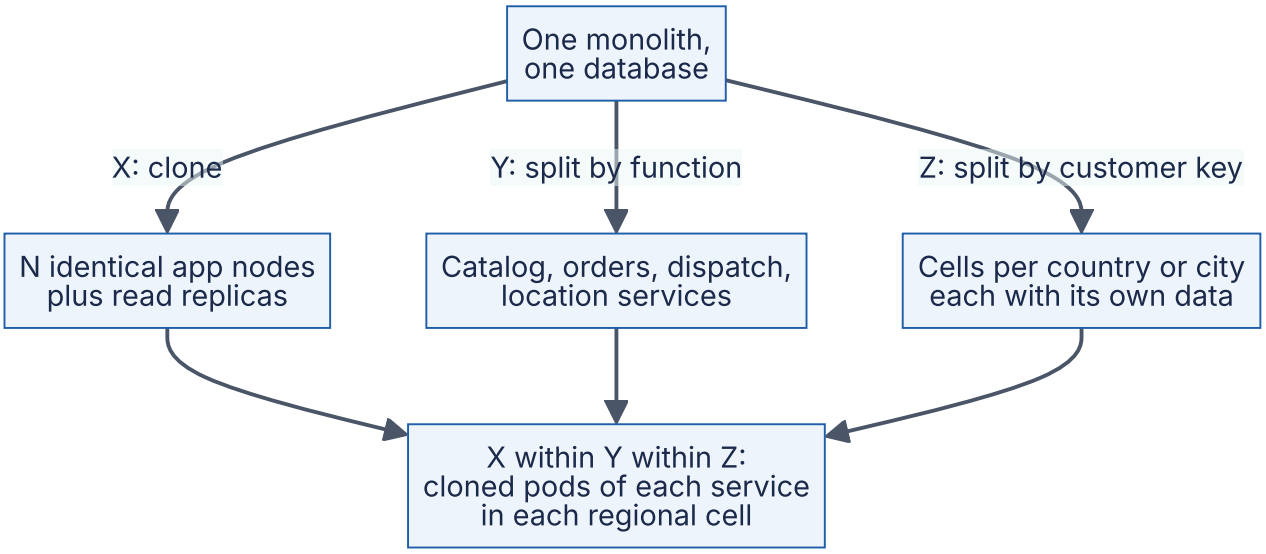
First-principles explanation

The bakery chain again. **X axis**: open more identical shops, each making everything. **Y axis**: split the work so one kitchen makes bread, another cakes, another sandwiches, each sized for its own demand. **Z axis**: give each city its own complete operation that serves only that city's customers. The axes combine: each city (Z) can have specialized kitchens (Y), each with several identical ovens (X).

Why does this exist?

Engineers tend to reach for one familiar technique. The cube forces the question "which kind of split solves *this* bottleneck?" Each axis relieves a different limit and costs something different.

Figure 18.7 — The scale cube applied to QuickBite.



Reading the diagram. The top node is the starting point: one deployable unit and one database. Arrows are design moves, not request flows, so no protocols apply. X clones the application and read-only data; the shared primary remains. Y splits by function into services, each with its own data store (a storage boundary per service). Z splits by a customer key such as country, so each cell holds only its customers' data and contains its own failures. Large systems end up at the bottom node, combining all three. Failure points differ by axis: X still shares the database, Y adds network calls, Z adds a routing layer and cross-cell operations.

Table 18.9 — The three axes compared.

	X axis: clone	Y axis: split by function	Z axis: split by key
What it is	Identical copies behind a load balancer	Different services for different work	Same service, different subsets of customers or data
Relieves	Compute and read capacity	Codebase size, team coupling, mixed workloads	Write capacity, data size, blast radius, data residency
Advantages	Simplest; no code change if stateless	Independent scaling and deployment	Near-linear scaling; failure isolation
Disadvantages	Shared data tier remains; caches duplicated	Network calls, distributed transactions	Routing layer; cross-cell queries and rebalancing
Operational complexity	Low	Medium to high	High
Examples	Stateless pods, read replicas	Microservices (Chapter 34), separate location service	Sharding (Chapter 14), cells (§33.12), regional stacks (Chapter 44)
Choose when	First, for stateless tiers	Parts of the system have very different load or owners	One primary or one failure domain is too big

WORKED EXAMPLE

Worked example — Which axis for which QuickBite bottleneck? At 2 million daily users (§2.11):

1. *Browse reads, 7,000 RPS.* Independent requests, read-only, cacheable. **X axis**: about 18 cloned API pods plus a menu cache (§18.11).
2. *Courier location writes, 5,000/s, about 83 times the order write rate.* A different workload from orders in rate, data shape and consistency. **Y axis**: a separate location service with its own in-memory store, as §2.8 decided.
3. *Order writes heading for the WAL limit of about 500/s* (§2.10) as QuickBite expands into several countries. **Z axis**: partition orders by country or city, which also satisfies data-residency rules and contains failures.

Interpretation. Each bottleneck has a natural axis, determined by whether the work is identical (X), different (Y) or identical on separable data (Z). **Engineering implication.** Name the axis explicitly in design reviews; “we will shard” and “we will add microservices” are answers to different problems.

INDUSTRY EXAMPLE

Cells and shuffle sharding at AWS. AWS publishes guidance on **cell-based architecture**: dividing a service into independent cells, each a complete copy of the stack serving a subset of customers, behind a thin routing layer [AWS 2023]. Its Builders’ Library also describes **shuffle sharding**, which assigns each customer to a random combination of nodes so that one customer’s harmful traffic affects few others [MacCárthaigh 2019]. Both are Z-axis splits, chosen as much to limit the blast radius of failures as to add capacity. *Evidence: Primary* — [AWS 2023], [MacCárthaigh 2019]

COMMON MISTAKE

“Z axis means database sharding.” Sharding a database is one Z-axis split. Abbott and Fisher’s Z axis can split the *whole stack* (application pods, caches and data) by customer, which also isolates failures. Conversely, splitting into microservices is a Y-axis move and adds no write capacity to a shared database.

18.11 Capacity planning

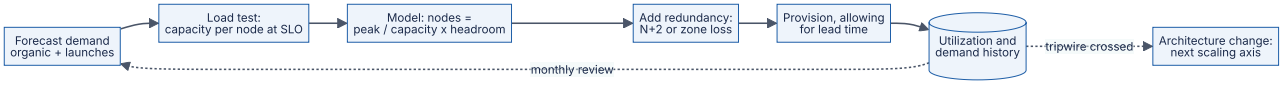
18.11.1 The planning loop

DEFINITION

Definition — Capacity planning. The recurring process of forecasting demand, translating it into the resources needed to serve it within the service-level objective with stated headroom and redundancy, and acquiring those resources before they are needed, allowing for the lead time to obtain them.

Capacity planning is the scalability counterpart of a household budget: forecast income and expenses, keep a reserve for emergencies, and order large purchases early enough to arrive on time. Google's SRE book lists its mandatory steps as an accurate forecast of organic demand that extends beyond the lead time for acquiring capacity, the inclusion of inorganic demand such as launches and marketing campaigns, and regular load testing to relate raw resources to service capacity [Beyer et al. 2016].

Figure 18.8 — The capacity planning loop.



Reading the diagram. This is a process, not a running system. Load tests convert requests per second into nodes; the per-node capacity c must be measured at the latency objective, not at saturation. The model adds queueing headroom, the redundancy step adds failure headroom, and provisioning starts early enough to cover lead time: minutes for cloud instances, weeks for quotas or reservations, months for hardware or a re-architecture. Utilization history (a storage boundary) feeds the next forecast (dotted). The second dotted exit matters most: when a tripwire (§2.7) fires, the answer is a design change, not more nodes.

18.11.2 Headroom and redundancy

FORMULA

Provisioned capacity with headroom

$$N = \lceil \frac{R_{peak}\,(1+g)^{L}}{c \times u_{target}} \rceil \qquad N_{total} = N + k \quad \text{or} \quad N_{total} = \lceil N \times \frac{Z}{Z-f} \rceil$$

Symbol	Meaning	Units
R_{peak}	Current design peak load	requests/s
g	Demand growth rate per period	fraction per month
L	Lead time to add capacity	months
c	Per-node throughput at which latency meets the objective (measured)	requests/s per node
u_{target}	Target utilization (queueing and reaction-time headroom, §18.8.4)	fraction
N	Nodes needed to serve the load with headroom	nodes
k	Spare nodes for failures and deploys (N+1: $k = 1$; N+2: $k = 2$)	nodes
Z, f	Failure domains (zones) used, and how many may fail at once	count
N_{total}	Nodes to provision	nodes

Three kinds of headroom hide in this formula: **queueing headroom** ($u_{target} < 1$, because latency grows as $1/(1-\rho)$, §4.10), **growth headroom** ($(1+g)^{L}$, because capacity takes time to arrive), and **failure headroom** (k or $Z/(Z-f)$). N+2 is the common rule for services: one spare for an unplanned failure, a second for the node that is down for maintenance or a deploy at the same moment [Beyer et al. 2016]. The zone form provisions so that losing f of Z zones leaves enough capacity *without any scaling action*, a property AWS calls **static stability** [Weiss and Furr 2020].

WORKED EXAMPLE

Worked example — QuickBite's API fleet at 2 million users. From §2.11: $R_{peak} = 6,667$ RPS (rounded to 7,000 in Chapter 2), $c = 1,000$ RPS per node, $u_{target} = 0.6$.

1. *Servers needed now:* $6,667/(1,000 \times 0.6) = 11.1$, so $N = 12$.
2. *N+2:* $N_{total} = 14$, the figure Chapter 2 used. Spread over 3 zones as 5, 5 and 4, losing a 5-node zone leaves 9 nodes: $6,667/9,000 = 74\%$ utilization. The service survives, but above its 60% target and with no room for a burst.
3. *Zone-safe,* $Z = 3$*,* $f = 1$*:* $N_{total} = \lceil 12 \times 3/2 \rceil = 18$, six per zone. Losing a zone leaves 12 nodes at $6,667/12,000 = 56\%$.
4. *With growth headroom:* demand grows $g = 8\%$ per month and new reserved capacity takes $L = 3$ months to arrange. $(1.08)^3 = 1.26$, so plan for $6,667 \times 1.26 = 8,398$ RPS: $N = \lceil 8,398/600 \rceil = 14$, and zone-safe $N_{total} = 21$.

Interpretation. Moving from N+2 to zone-level static stability costs 4 more nodes (29% more) and buys survival of a whole-zone outage within target. **Engineering implication.** Decide which failure the fleet must absorb without scaling, write it down in the capacity plan, and size for it; autoscalers may not be able to add capacity during a zone outage, when every other customer is trying to do the same.

18.11.3 Peak-to-average ratio and the cost of peaks

FORMULA

Peak-to-average ratio and average fleet utilization

$$PAR = \frac{R_{peak}}{R_{avg}} \qquad \bar{u} = \frac{R_{avg}}{N_{total} \times c}$$

Symbol	Meaning	Units
R_{avg}	Average load over a day (or week)	requests/s
PAR	Peak-to-average ratio	dimensionless
$\bar{u}$	Average utilization of a statically provisioned fleet	fraction

WORKED EXAMPLE

Worked example — How idle is a fleet sized for dinner? QuickBite's browse load averages $2,000,000 \times 30/86,400 = 694$ RPS and peaks at 6,667 RPS.

1. $PAR = 6,667/694 = 9.6$. This follows from the assumptions: 20% of the day's traffic in one hour is $0.2 \times 24 = 4.8$ times the daily average, and the burst factor of 2 doubles it.
2. Static, zone-safe fleet of 18 nodes: $\bar{u} = 694/(18 \times 1,000) = 3.9\%$.
3. With autoscaling between a floor of 6 nodes (two per zone) and 18, the fleet follows demand; if it averages 8 nodes, $\bar{u} = 694/8,000 = 8.7\%$, and the fleet costs less than half as much.

Interpretation. A fleet sized for a sharp peak, with queueing and failure headroom, is idle most of the day. Barroso and Hölzle reported that servers in large datacenters typically operate at 10–50% of their maximum utilization [Barroso and Hölzle 2007].
Engineering implication. The higher the PAR, the more autoscaling, scheduled scaling or sharing capacity with batch work pays off. A low PAR (a steady workload) favors reserved, statically sized capacity.

18.11.4 Forecasting and lead time

Forecasts combine **organic growth**, extrapolated from history (often as a compound monthly rate with daily and weekly seasonality), with **inorganic** steps: launches, marketing campaigns, new countries, and events such as holidays [Beyer et al. 2016]. The crucial output is not a number but a *date*: when will each component cross its usable capacity, and is that further away than the time needed to fix it?

FORMULA

Time until a component's capacity is exhausted

$$D(t) = D_0(1+g)^t \qquad t_{exhaust} = \frac{\ln(C_{usable}/D_0)}{\ln(1+g)} \qquad t_{start} = t_{exhaust} - L_{fix}$$

Symbol	Meaning	Units
D_0	Current peak demand on the component	operations/s
g	Compound growth rate per period	fraction per month
C_{usable}	Usable capacity: limit × target utilization	operations/s
$t_{exhaust}$	Periods until demand reaches usable capacity	months
L_{fix}	Lead time of the fix, including a safety margin	months
t_{start}	Latest month to start the fix	months

WORKED EXAMPLE

Worked example — When must QuickBite start partitioning orders? The order database's WAL disk caps it at about 500 orders/s (§2.6); at $u_{target} = 0.6$, $C_{usable} = 300$ orders/s. Today $D_0 = 56$ orders/s at 2 million users (the unrounded 55.6 of §2.11, which Chapter 2 rounds up to 60), growing 8% per month.

1. $C_{usable}/D_0 = 300/56 = 5.36$; $\ln 5.36 = 1.678$; $\ln 1.08 = 0.0770$.
2. $t_{exhaust} = 1.678/0.0770 = 21.8$ months.
3. Partitioning by region (§14.7) is estimated at 6 months of work plus a 3-month margin: $L_{fix} = 9$. So $t_{start} = 21.8 - 9 \approx 13$ months from now.
4. Sensitivity: at 12% monthly growth, $t_{exhaust} = 1.678/\ln 1.12 = 1.678/0.1133 = 14.8$ months, and the project must start in about 6 months.

Interpretation. The same headroom means two years at one growth rate and barely more than one at another. **Engineering implication.** Put t_{start} for every architectural limit into the capacity plan, recompute it monthly from measured growth, and treat crossing it as a tripwire (§2.7). Group commit (§13.5) could raise the limit about 4× (an assumed factor, §2.10) and push $t_{exhaust}$ out by $\ln 4/\ln 1.08 = 18$ months, a cheaper first move worth evaluating before partitioning.

INDUSTRY EXAMPLE

Intent-based capacity planning at Google. The SRE book describes Auxon, a tool in which teams state capacity *intent* (dependencies, service-level requirements, N+2 redundancy, budget) and a solver produces the resource allocation, replacing hand-maintained spreadsheets of machine counts. *Evidence: Primary* — [Beyer et al. 2016]

18.11.5 Worked capacity plan: QuickBite from 1,000 to 10 million users

The plan reuses Chapter 2's per-user assumptions: 30 browse requests and 0.25 orders per daily active user (DAU), 20% of daily traffic in the busiest hour, a burst factor of 2, one active courier per 100 DAU at peak sending a location every 4 s, and $c = 1,000$ RPS per API node at $u_{target} = 0.6$. Two zones are used for the first two stages and three thereafter, and no stage runs fewer than two API nodes. The program below computes every row of Table 18.10.

```
        """QuickBite capacity plan from 1k to 10M daily active users (Section 18.11).

        All inputs are teaching assumptions carried over from Chapter 2 (Table 2.10,
Section 2.11).
        """
        import math

        BROWSE_PER_DAU = 30      # browse requests per DAU per day
        ORDERS_PER_DAU = 0.25    # orders per DAU per day
        BUSY_HOUR_SHARE = 0.20   # share of daily traffic in the busiest hour
        BURST = 2.0              # burst factor above the busiest-hour average
        COURIERS_PER_DAU = 0.01  # couriers active at peak per DAU (20,000 at 2M DAU)
        UPDATE_INTERVAL_S = 4    # courier location update interval
        NODE_RPS = 1_000         # sustained RPS per API node (measured, Section 2.11)
        U_TARGET = 0.6           # target utilization per node


        def peak(daily):
            return daily * BUSY_HOUR_SHARE / 3_600 * BURST


        def api_nodes(rps, zones):
            work = max(1, math.ceil(rps / (NODE_RPS * U_TARGET)))
            if zones == 1:
                return work, work + 1                    # N+1 on one zone
            return work, math.ceil(work * zones / (zones - 1))  # survive one zone loss


        stages = [  # (name, DAU, zones)
            ("S1 pilot", 1_000, 2),
            ("S2 city", 20_000, 2),
            ("S3 region", 200_000, 3),
            ("S4 country", 2_000_000, 3),
            ("S5 multi-country", 10_000_000, 3),
        ]

        print(f"{'stage':17} {'DAU':>10} {'browse/s':>9} {'avg/s':>7} {'PAR':>4} "
              f"{'orders/s':>8} {'loc w/s':>8} {'work':>5} {'total':>5} {'avg
util':>8}")
        for name, dau, zones in stages:
            browse_peak = peak(dau * BROWSE_PER_DAU)
            browse_avg = dau * BROWSE_PER_DAU / 86_400
            orders_peak = peak(dau * ORDERS_PER_DAU)
            loc_writes = dau * COURIERS_PER_DAU / UPDATE_INTERVAL_S
            work, total = api_nodes(browse_peak, zones)
            total = max(total, 2)                        # never fewer than two nodes
            avg_util = browse_avg / (total * NODE_RPS)
            print(f"{name:17} {dau:>10,} {browse_peak:>9,.0f} {browse_avg:>7,.0f} "
                  f"{browse_peak / browse_avg:>4.1f} {orders_peak:>8.2f}
{loc_writes:>8,.0f} "
                  f"{work:>5} {total:>5} {avg_util:>8.1%}")
```

Output:

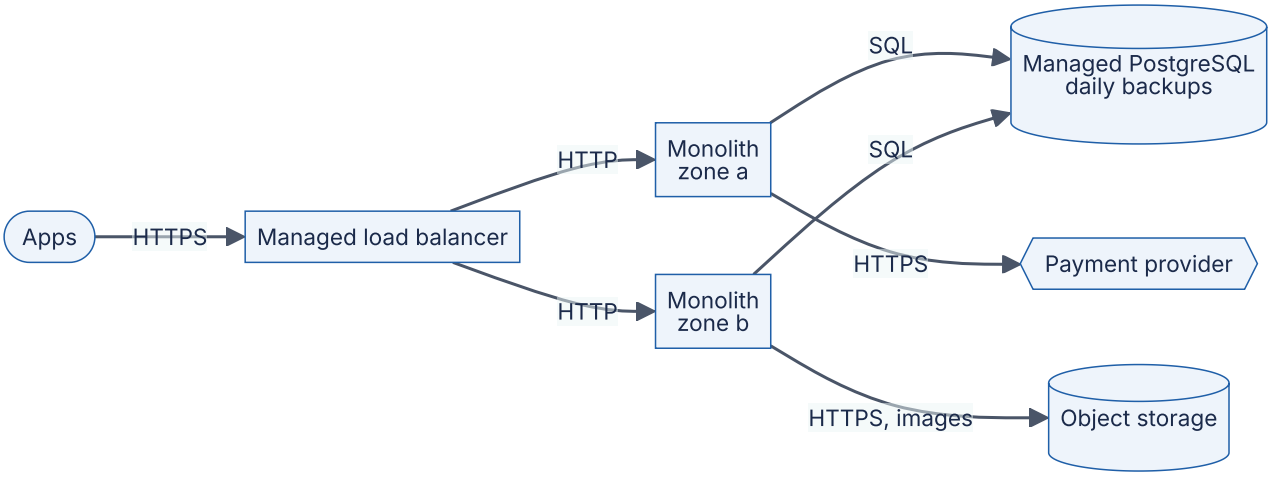
```
stage                    DAU  browse/s   avg/s  PAR orders/s  loc w/s  work total avg
util
S1 pilot               1,000         3       0  9.6     0.03        2     1     2
0.0%
S2 city               20,000        67       7  9.6     0.56       50     1     2
0.3%
S3 region            200,000       667      69  9.6     5.56      500     2     3
2.3%
S4 country         2,000,000     6,667     694  9.6    55.56    5,000    12    18
3.9%
S5 multi-country  10,000,000    33,333   3,472  9.6   277.78   25,000    56    84
4.1%
```

What to notice: the API fleet is never the hard part; at every stage the interesting limits are elsewhere (the order database, the location write path, availability). The minimum of two nodes dominates the first three stages, which is why their average utilization rounds to zero.

Table 18.10 — QuickBite capacity plan by stage.

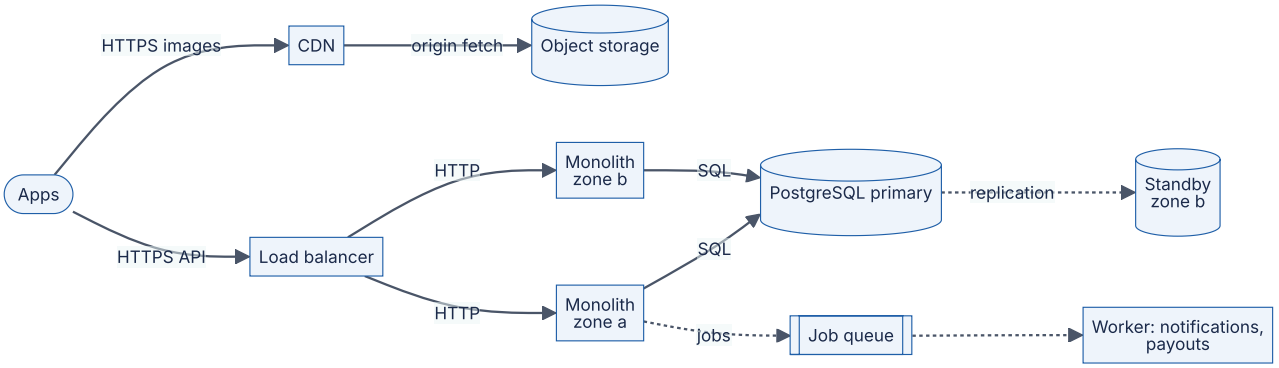
Stage	DAU	Peak browse RPS	Peak orders/s	Location writes/s	API nodes	What changes, and why
S1 pilot	1,000	3	0.03	2.5	2	Monolith; managed DB; second node only for availability
S2 city	20,000	67	0.6	50	2	Standby DB, CDN for images, job queue
S3 region	200,000	667	5.6	500	3 (autoscaled)	Cache, pooler, location moves to an in-memory store
S4 country	2,000,000	6,667	56	5,000	18 (autoscaled)	Y-axis location service, event log, read replicas
S5 multi-country	10,000,000	33,333	278	25,000	84 in 4 cells	Z-axis regional cells; orders partitioned by region

Figure 18.9 — Stage S1: 1,000 DAU.



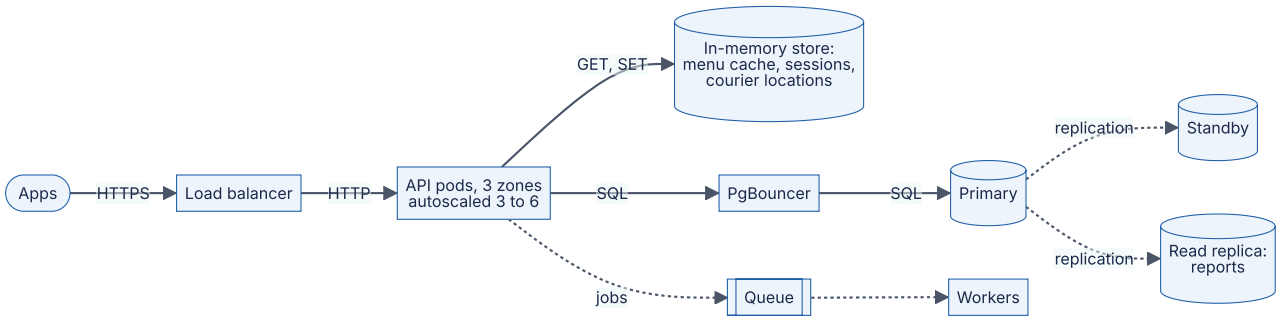
Reading the diagram. The three apps call a managed load balancer over HTTPS, which forwards synchronous HTTP to two copies of the modular monolith (§2.8) in different zones. Sessions live in the database from day one, so the nodes are stateless. One managed PostgreSQL instance with backups is the only database; images go to object storage; the payment provider is external. Failure point: the single database instance, acceptable for a pilot. Peak load (3 RPS) would fit on a laptop; the second node exists for deploys and failures, not capacity.

Figure 18.10 — Stage S2: 20,000 DAU (one city).



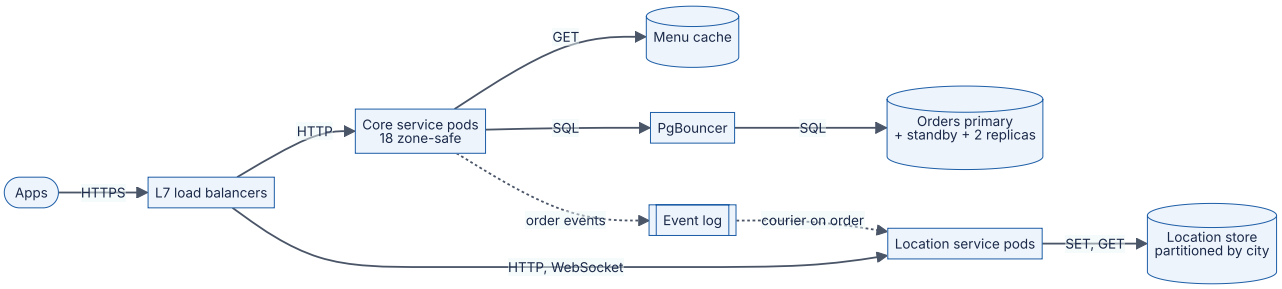
Reading the diagram. Paying customers arrive, so availability matters more than capacity. A CDN serves menu images. The database gains a standby in a second zone, fed by replication (dotted), with automatic failover (§14.4). Notifications and payouts move asynchronously to a job queue and worker, so a slow third party cannot hold an API request. Storage boundaries: object storage, primary and standby, the queue. The API tier is still two nodes at 67 RPS.

Figure 18.11 — Stage S3: 200,000 DAU (one region).



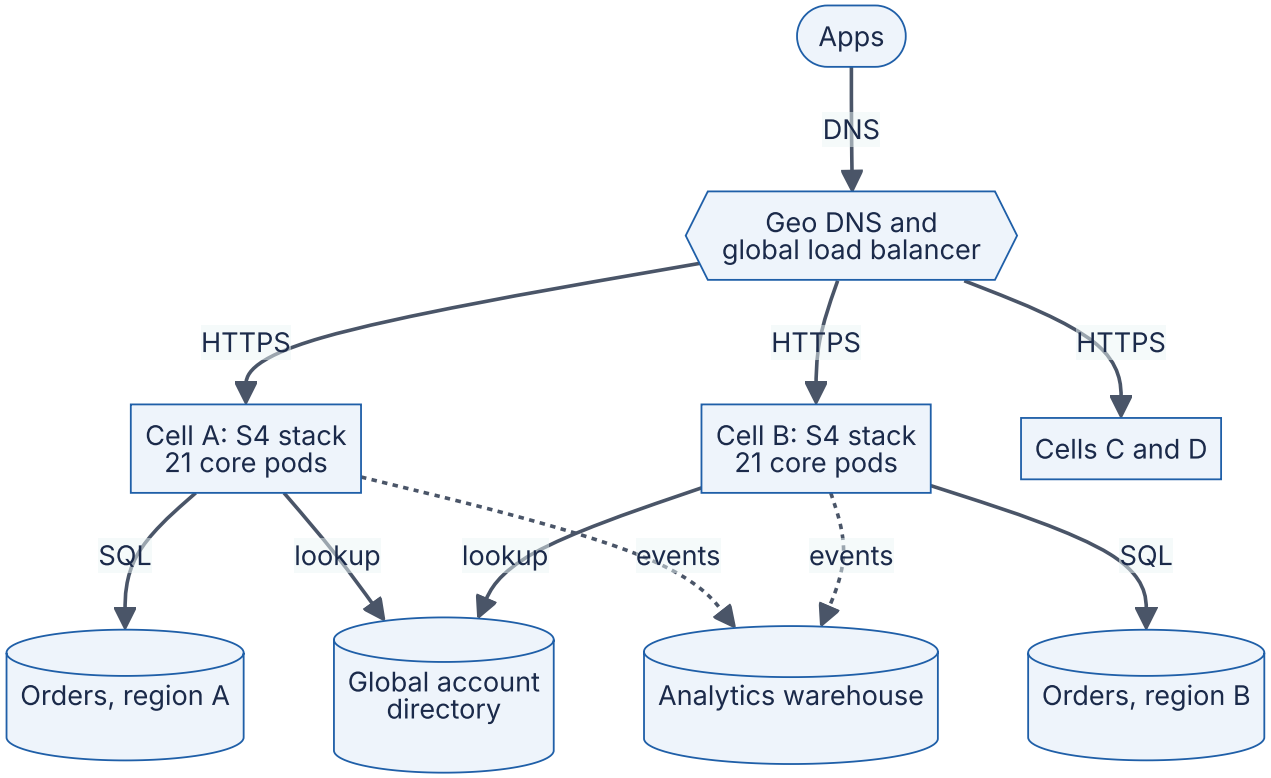
Reading the diagram. Menu queries are now the database's largest cost, so a replicated in-memory store caches menus with a TTL and holds sessions (Chapter 20). Courier locations, 500 writes/s of short-lived data, move there too. The API runs as autoscaled pods across three zones behind a transaction-mode pooler, which fixes the connection budget before autoscaling can break it (§18.8.5). A read replica serves reports. Failure point: the in-memory store is on the critical path for sessions, so it is replicated across zones.

Figure 18.12 — Stage S4: 2,000,000 DAU (one country).



Reading the diagram. This is Chapter 2's container view (Figure 2.7) sized by this chapter's arithmetic. The location path (5,000 writes/s) is a separate service with a store partitioned by city: a Y-axis split that also serves the 2,500 tracking reads/s. The core service runs 18 zone-safe pods (§18.11.2) and publishes order events asynchronously to a durable log that the location service consumes. The orders primary runs at about 11% of its WAL limit (56 of 500 orders/s), so it is not partitioned yet, but §18.11.4 has started the clock.

Figure 18.13 — Stage S5: 10,000,000 DAU (several countries).



Reading the diagram. Order writes would reach 278/s, 93% of one primary's usable 300/s, and expansion brings data-residency rules, so the system splits on the Z axis into four regional **cells**, each a complete S4 stack with its own orders database serving one region. Assuming four equal cells, each peaks at 8,333 browse RPS: $\lceil 8,333/600 \rceil = 14$ working pods, 21 zone-safe, 84 in total. Global DNS routes users to their cell; a small global account directory maps accounts to cells; events flow asynchronously to a global analytics warehouse. Failure is contained to one cell. Cross-cell operations, such as a traveler ordering abroad, become explicit exceptions (Chapter 44, §33.12).

Failure scenarios and misconceptions

Table 18.11 — Capacity planning failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Outage at a known peak	Planned for average load	Saturation at the same hour daily	Emergency scale-up; shed load	Plan with PAR and burst factors
Zone loss overloads survivors	Sized N+k, not zone-safe	Utilization jump on zone failure	Shed non-critical traffic	Static stability, $Z/(Z-f)$
Capacity arrives too late	Lead time ignored	Forecast crosses usable capacity before the fix is ready	Vertical stopgap	t_{start} tripwires
Wrong per-node capacity	c measured at saturation, or with an unrealistic request mix	p99 breach below planned load	Re-measure	Load test at the latency objective
Launch traffic unplanned	Inorganic demand missed	Marketing calendar surprises	Pre-scale; queue sign-ups	Include launches in forecasts

COMMON MISTAKE

“Plan for average load and let the autoscaler handle peaks.” QuickBite’s PAR is 9.6. An autoscaler with a two-minute reaction time cannot add 9× capacity in time for a dinner rush that ramps in minutes, and it is bounded by `maxReplicas`, quotas and database connections that only a plan provides. Use scheduled scaling for known peaks and the autoscaler for the surprises.

COMMON MISTAKE

“Headroom is waste.” Queueing headroom keeps latency within objective (§4.10), failure headroom keeps the service up when a zone fails, and growth headroom covers lead time. Waste is headroom nobody can justify; write the reason for each factor into the plan.

18.12 Summary and revision notes

SUMMARY

- **Vertical scaling** is the cheapest move in effort and keeps state coherent, but it hits a hardware ceiling, keeps one failure domain, and gets expensive past the knee: with linear pricing, cost per unit of throughput is $k(N) = N/C(N) = 1 + \alpha(N-1) + \beta N(N-1)$.
- **Horizontal scaling** removes the ceiling and turns failures into lost capacity, but needs interchangeable nodes, load distribution, and a shared tier that can take the combined load.
- **Stateless** means a tier keeps no state another instance needs; state moves to session stores, databases, caches and clients. Sticky sessions hide state rather than removing it.
- **Bottlenecks** in scaled-out systems are shared resources. Use the resource model (§2.6) and fit the USL; the marginal gain $\Delta X(N) = X_1[C(N) - C(N-1)]$ shows when another node stops paying. Cap autoscalers below the knee.
- **Target tracking:** $n_{desired} = \lceil n \cdot m_{current}/m_{target} \rceil$ with a tolerance band; scale up fast, down slowly. Choose $u_{target} \le u_{safe}/(1 + rT_{react})$ and shorten reaction time with warm pools and spare nodes.
- **Connection budget:** $n_{max} p + \sum n_j p_j \le M - R$. Autoscaling multiplies pools; put a transaction pooler in front of PostgreSQL and size server connections by Little's Law.
- **Databases:** caches and replicas scale reads; only partitioning (or a bigger primary) scales writes. Try cheap fixes first (Table 18.8).
- **Scale cube:** X clones, Y splits by function, Z splits by customer or key. Each relieves a different bottleneck.
- **Capacity planning:** $N = \lceil R_{peak}(1+g)^L/(c\,u_{target}) \rceil$, then N+2 or zone-safe $\lceil NZ/(Z-f) \rceil$. PAR reveals idle cost. Turn forecasts into dates: $t_{exhaust} = \ln(C_{usable}/D_0)/\ln(1+g)$, and start fixes L_{fix} earlier.

18.13 Knowledge checkpoint

1. Give two reasons vertical scaling stops being the right answer.
2. With linear pricing and USL coefficients $\alpha = 0.02$, $\beta = 0.00005$, what is the relative cost per unit of throughput at 32 cores?
3. What test tells you whether an application instance is stateless?
4. Why do sticky sessions make scale-in harder?
5. 12 pods average 75% CPU with a 50% target. How many pods does the HPA request?

6. Why do autoscalers scale down more slowly than they scale up?
7. u_{safe} = 0.8, load grows at 10% per minute, reaction time is 3 minutes. What is the highest safe target?
8. A database has `max_connections` = 300 and 20 reserved. Pods have pools of 10. What is the largest safe `maxReplicas` with no other clients?
9. Which database-scaling options add write capacity?
10. Classify: read replicas; a separate payments service; one database per country.
11. Why is N+2 not the same as zone-safe for 14 nodes in 3 zones?
12. Peak 9,000 RPS, average 1,500 RPS. What is the PAR, and what does it suggest?
13. Usable capacity is 4 times current demand and demand grows 5% per month. When is it exhausted?

Answers

1. Any two of: no larger node exists; cost per unit of throughput rises past the knee; a single node is one failure domain; serial code or in-node coherency leaves added cores idle.
2. $k(32) = 1 + 0.02 \times 31 + 0.00005 \times 32 \times 31 = 1 + 0.62 + 0.0496 = 1.67$.
3. If it vanished now, would any user or other instance notice anything beyond a retry? If not, it is stateless.
4. Removing an instance destroys the sessions pinned to it, so scale-in either logs users out or must wait for sessions to expire.
5. $\lceil 12 \times 75/50 \rceil = 18$.
6. To avoid flapping: removing capacity too early causes a rebound that needs it back, and under-capacity is costlier than a few minutes of spare pods.
7. $0.8/(1 + 0.1 \times 3) = 0.8/1.3 = 0.615$, so about 60%.
8. $\lfloor (300 - 20)/10 \rfloor = 28$ pods.
9. A bigger primary, a functional split onto separate databases, partitioning or sharding, distributed SQL, and write-optimized or partitioned NoSQL stores. Caches and read replicas do not.
10. X axis; Y axis; Z axis.
11. N+2 tolerates any two node failures, but 14 nodes in 3 zones put 5 in one zone; losing that zone removes 5. Zone-safe sizing provisions $Z/(Z - f)$ times the needed nodes.
12. PAR = 6. A statically sized fleet would be idle most of the day; autoscaling or scheduled scaling pays off.
13. $\ln 4 / \ln 1.05 = 1.386/0.0488 = 28.4$ months.

18.14 Interview questions

Basic

Q1. What is the difference between vertical and horizontal scaling, and when would you choose each? Answer. Vertical adds resources to one node: no code change, transactions stay local, but there is a ceiling, one failure domain and a rising cost per unit of throughput past the knee. Horizontal adds nodes: no ceiling and graceful degradation, but it needs stateless nodes, load distribution and a shared tier that can keep up. Scale the stateless tiers out from the start; scale the database up until its knee, then partition.

Q2. How do you make a service stateless? Answer. Move every piece of request-spanning state to a tier built for it: sessions to a replicated session store or signed tokens, carts and drafts to the database, cached data to a shared cache, uploads to object storage, timers to a queue. Keep only rebuildable local caches. Add readiness probes and graceful shutdown so instances can be added and removed safely.

Intermediate

Q3. Your API autoscales from 10 to 100 pods during a sale, and the database starts refusing connections. What happened and how do you fix it? Answer. Pods times pool size exceeded `max_connections` minus reserved slots. Short term: lower pool sizes or cap replicas. Proper fix: a transaction-mode pooler such as PgBouncer, with server connections sized by Little's Law (transaction rate × hold time × headroom), and a connection-budget check whenever replicas or pools change. Also verify that the database itself is not the real bottleneck.

Q4. How do you choose the autoscaling metric and target? Answer. Pick a metric proportional to load and inversely proportional to replicas: CPU for CPU-bound services, in-flight requests for I/O-bound ones, backlog per worker for queues. Load-test to find the per-pod throughput at which p99 meets the objective (u_{safe}), then lower the target by the load growth expected during the reaction time: $u_{safe}/(1 + rT_{react})$. Add scheduled scaling for known peaks.

Advanced

Q5. Adding servers no longer increases throughput. How do you investigate? Answer. Measure throughput at several node counts and fit the USL. A large α points to a shared, serialized resource (a hot row, a global lock, a single primary); a significant β points to coordination that grows with node pairs (invalidation broadcasts, gossip, distributed locks). Confirm with per-resource utilization (USE, §2.6) and profiling. Fix the shared cost by partitioning or removing it; cap the autoscaler below the knee meanwhile.

Q6. Walk through a capacity plan for a service growing from one city to several countries. Answer. Convert users to peak rates (busy-hour share, burst factor); measure per-node capacity at the latency objective; compute nodes at a target utilization; add N+2 or zone-safe redundancy and growth over lead time. For each shared component, compute when demand crosses usable capacity and subtract the fix's lead time. Map each limit to an axis: X for stateless tiers, Y for very different workloads such as location updates, Z (cells, partitions) for write limits and data residency.

18.15 Practical exercises

1. Run the autoscaler simulation with a 2-minute load ramp and with `STARTUP_S = 180`. Find the highest target that avoids overload in each case and compare it with the formula in §18.8.4.
2. Extend the capacity-plan program to compute PgBouncer server connections per stage, assuming 2 ms per query and that 40% of browse requests miss the cache.
3. Load-test any small web service at 1, 2, 4, 8 and 16 workers, fit the USL coefficients and report N^*.

18.16 Design challenges

1. **Ticket on-sale.** A concert-ticket service sees 100 times its normal load for 10 minutes when sales open. Design the scaling strategy: what is pre-scaled, what autoscales, what queues users, and how the inventory database avoids a hot row.
2. **QuickBite in a new continent.** Extend the S5 design to a continent with strict data-residency laws and a different payment provider. Decide what is global, what is per cell, and how a traveler's account works across cells.

18.17 References

1. Abbott, M. L., and Fisher, M. T. *The Art of Scalability: Scalable Web Architecture, Processes, and Organizations for the Modern Enterprise*, 2nd ed. Addison-Wesley, 2015.
2. Amazon Web Services. *Amazon EC2 Auto Scaling User Guide*: target tracking scaling policies, instance warm-up, warm pools and predictive scaling. Accessed 2026.
3. Amazon Web Services. *Reducing the Scope of Impact with Cell-Based Architecture*. AWS Well-Architected whitepaper, 2023.
4. Barroso, L. A., and Hölzle, U. "The Case for Energy-Proportional Computing." *IEEE Computer* 40(12), 2007.
5. Barroso, L. A., Hölzle, U., and Ranganathan, P. *The Datacenter as a Computer: Designing Warehouse-Scale Machines*, 3rd ed. Morgan & Claypool, 2018.

6. Beyer, B., Jones, C., Petoff, J., and Murphy, N. R. (eds.). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016. Chapters 1, 2 and 18. https://sre.google/sre-book/table-of-contents/
7. Craver, N. "Stack Overflow: The Architecture – 2016 Edition." nickcraver.com, 2016.
8. Gunther, N. J. *Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable Applications and Services*. Springer, 2007.
9. Kubernetes Documentation. "Horizontal Pod Autoscaling." kubernetes.io, accessed 2026.
10. MacCárthaigh, C. "Workload isolation using shuffle-sharding." *Amazon Builders' Library*, 2019.
11. Netflix Technology Blog. "Scryer: Netflix's Predictive Auto Scaling Engine." 2013.
12. PostgreSQL Global Development Group. *PostgreSQL Documentation*: "Connections and Authentication" (`max_connections`, `superuser_reserved_connections`). Accessed 2026.
13. Weiss, B., and Furr, M. "Static stability using Availability Zones." *Amazon Builders' Library*, 2020.

18.18 Open verification items

- Kubernetes HPA defaults (tolerance 0.1, 15 s sync period, 300 s scale-down stabilization) and the documentation URL, as of 2026.
- Largest general-purpose cloud instance sizes in Table 18.1, as of 2026.
- Stack Overflow server counts in [Craver 2016] and the post's URL.
- Publication years of [Weiss and Furr 2020] and [Netflix 2013]; exact title of [AWS 2023].
- That the SRE book's N+2 guidance and Auxon description are in the chapters cited.

Chapter 19 — Load Balancing

> **Level:** 2 · **Track:** A (Traditional) · **Estimated reading time:** 5 h

Chapter 18 made the case for horizontal scaling: run many identical, stateless copies of a service instead of one big machine. That decision creates a new problem the moment you make it. Clients know one name, `api.example.com`, but there are now forty servers behind it. Something must pick a server for every connection or request, notice when a server dies, and keep doing so while servers come and go. That something is a **load balancer**, and it sits on the critical path of almost every request in a modern system.

Load balancing looks simple ("send each request to the next server") and hides some of the most instructive ideas in system design. A balancer that ignores load can push slow servers into collapse while fast ones idle. A balancer that chases the least-loaded server can stampede it. A balancer that remembers which server holds a user's session cannot also spread load perfectly. And a balancer is itself a server that can fail, so you must balance the balancers. This chapter works through each of these problems with the mechanism, the arithmetic and a simulation that you can run.

19.1 Learning objectives

After this chapter you should be able to:

- Distinguish layer 4 from layer 7 load balancing, and choose between TLS termination, re-encryption and passthrough, and between NAT, full-proxy and direct server return forwarding.
- Implement round robin, smooth weighted round robin, least connections, EWMA-based least response time and power of two choices, and predict how each behaves on heterogeneous servers.
- State the maximum-load results for one and two random choices, and use a simulation to compare p99 queue lengths across algorithms, including with stale load information.
- Explain consistent hashing with bounded loads and Maglev hashing, build a Maglev lookup table and measure its disruption when a backend fails.
- Configure active and passive health checks, compute detection time and the requests lost during it, and avoid flapping and ejection cascades.
- Weigh session affinity methods (cookies, source-IP hashing, consistent hashing) against their costs.

- Design redundant load balancer tiers with VRRP, anycast and ECMP, compute failover times, and drain connections safely.
- Explain where DNS-based global server load balancing fits and where it falls short, and compare proxy, client-side, sidecar and lookaside load balancing.
- Compare NGINX, HAProxy, Envoy, AWS ALB/NLB and Google Maglev for a given workload.

19.2 Prerequisites

- IP routing, BGP and anycast — §7.5 (anycast and ECMP are how balancers themselves are made redundant).
- DNS resolution, TTLs and DNS-based load balancing — §7.6.4, §7.6.5 (global load balancing starts in DNS).
- TCP handshakes, TLS 1.3 and HTTP/2 multiplexing — §7.7, §7.9, §7.10 (they decide what a balancer can see and what a connection costs).
- Reverse proxies and NAT — §7.12, §7.13 (a load balancer is a reverse proxy or a NAT device with a selection policy).
- Queueing basics: utilization, M/M/1, shared versus separate queues, Little's Law — §4.10, §4.11 (every algorithm here is an attempt to approximate one shared queue).
- Percentiles and tail latency — §4.8.
- Consistent hashing and virtual nodes — §14.8 (this chapter builds on it and does not repeat the derivations).
- Stateless versus stateful services — §18.5.

19.3 Layer 4 versus layer 7 load balancing

19.3.1 What a load balancer is

DEFINITION

Definition — Load balancer. A component that receives traffic addressed to a single service endpoint, the **virtual IP address (VIP)** or service name, and distributes it across a **pool** of **backends** (also called *upstreams*, *targets* or *real servers*) according to a **balancing algorithm**, while excluding backends that are unhealthy.

DEFINITION

Definition — Layer 4 (L4) and layer 7 (L7) load balancing. An **L4 load balancer** makes its decision from transport-layer information (IP addresses, ports, protocol) and balances **connections or flows**: every packet of one TCP connection goes to the same backend. An **L7 load balancer** terminates the client connection, parses the application protocol (usually HTTP), and balances **requests**: two requests on one connection may go to different backends.

First-principles explanation

Picture a post office with twenty counters. An L4 balancer is a doorman who looks only at the envelope (sender, entrance) and sends each customer to a counter for the whole visit, however many errands they have. An L7 balancer is a receptionist who opens each letter, reads the errand, and sends each errand to the right counter. She is far more flexible, but opening letters takes time and trust: she must hold the keys to the envelopes (TLS keys).

Why does this exist?

Both exist because of a trade-off between **cost per byte** and **intelligence per request**. Network-level balancers carry terabits per second by touching only packet headers on commodity servers [Eisenbud et al. 2016]. Routing by path, retrying an idempotent `GET` elsewhere, or spreading gRPC calls that share one HTTP/2 connection needs the request itself. Large sites stack both: L4 in front for scale, L7 behind for routing (Figure 19.9).

How it works internally

An **L4 balancer** comes in two forms:

1. **Packet-forwarding L4** (Linux IPVS, Maglev, Meta's Katran, AWS Network Load Balancer). It does not terminate TCP. For the first packet of a flow it picks a backend by hashing the **5-tuple** (source IP, source port, destination IP, destination port, protocol) or by algorithm, records the choice in a **connection-tracking table**, and rewrites or encapsulates each packet toward that backend. The backend completes the TCP handshake with the client itself.
2. **Connection-terminating L4 proxy** (HAProxy `mode tcp`, NGINX `stream`). It accepts the client's TCP connection, opens a second TCP connection to a chosen backend, and copies bytes in both directions without parsing them.

An **L7 balancer** (NGINX `http`, HAProxy `mode http`, Envoy, AWS Application Load Balancer) always terminates:

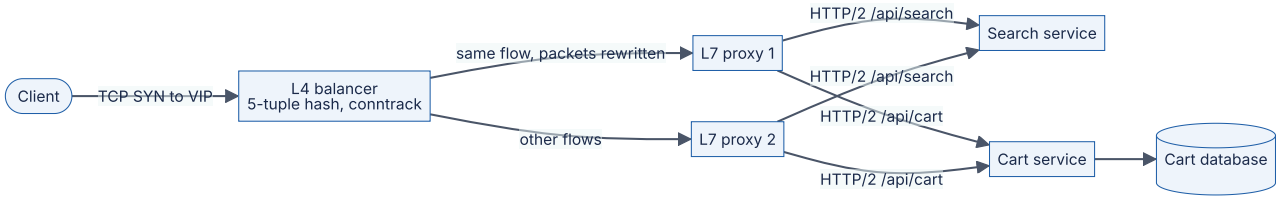
1. Accept the client TCP connection and, for HTTPS, complete the TLS handshake with the site's certificate (**TLS termination**).

2. Parse each HTTP request: method, path, `Host`, headers.
3. Apply **routing rules** (host- and path-based) to choose a pool, then the balancing algorithm to choose a backend in it.
4. Forward the request over a pooled, often long-lived connection to the backend, adding headers such as `X-Forwarded-For` (§7.12).
5. Stream the response back, optionally compressing, caching or retrying on failure.

TLS choices. With **termination**, the balancer decrypts and forwards plaintext inside the private network. With **re-encryption** (TLS bridging), it decrypts, inspects, and opens a new TLS connection to the backend, which zero-trust networks (§10.10) often require. With **passthrough**, an L4 balancer forwards encrypted bytes untouched and may route on the **Server Name Indication (SNI)** field, which travels unencrypted in the TLS ClientHello; it cannot see paths or headers, but certificates and keys stay on the backends.

Architecture diagram

Figure 19.1 — An L4 balancer forwards flows; an L7 balancer terminates connections and routes requests.



Reading the diagram. The client opens one TCP connection to the VIP. The L4 balancer hashes the 5-tuple, records the choice in its connection table, and sends every packet of the flow to the same L7 proxy. The proxy terminates TCP and TLS, reads each HTTP request, and routes `/api/search` and `/api/cart` to different services over pooled HTTP/2 connections. All arrows are synchronous. The cart database is the only storage boundary. Failure points: a lost L4 node breaks its flows unless peers can rebuild the mapping (§19.8); a lost L7 proxy breaks the connections it terminated. Scaling: add L4 nodes behind ECMP and L7 proxies behind the L4 tier.

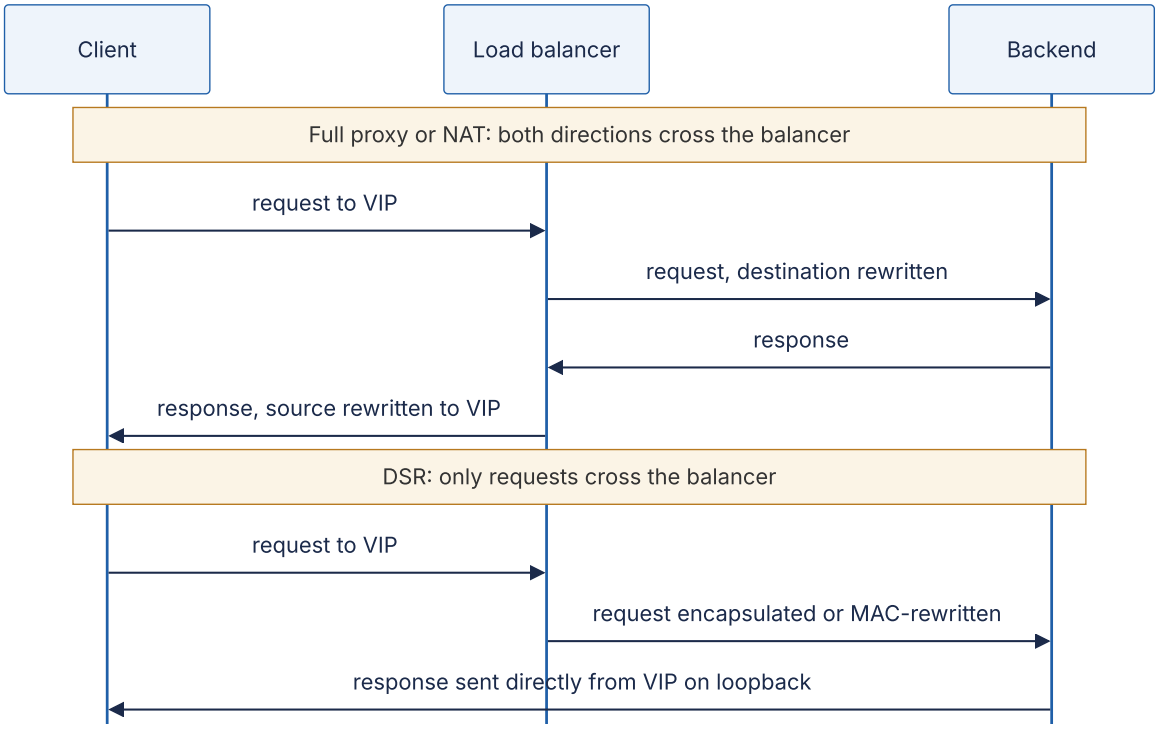
Forwarding modes and direct server return

A packet-forwarding balancer must decide how replies get back to the client.

DEFINITION

Definition — Direct server return (DSR). A forwarding mode in which the balancer delivers request packets to the backend but the backend sends responses **directly to the client**, using the VIP as its source address, bypassing the balancer on the return path.

Figure 19.2 — Full-proxy (NAT) forwarding versus direct server return.



Reading the diagram. In the upper half every byte, in both directions, passes through the balancer, which rewrites addresses each way; the backend learns the client's address only if the balancer passes it (PROXY protocol or `X-Forwarded-For`). In the lower half the balancer forwards only requests, by rewriting the destination MAC address (same layer-2 segment) or by wrapping the packet in an outer IP header (IP-in-IP, GRE, GUE). The backend holds the VIP on a loopback interface and replies from it straight to the client. All flows are synchronous. Failure points: with DSR the balancer never sees responses, so it cannot notice a backend that never answers, and tunnel headers shrink the usable MTU. Scaling: response bytes never load the balancer.

FORMULA

Balancer bandwidth by forwarding mode

$$B_{\text{proxy}} = R\,(s_{\text{req}} + s_{\text{resp}}), \qquad B_{\text{DSR}} = R\,s_{\text{req}}$$

Symbol	Meaning	Units
B_{proxy}	Bandwidth through the balancer when both directions cross it	bytes/second
B_{DSR}	Bandwidth through the balancer with direct server return	bytes/second
R	Request rate	requests/second
s_{req}, s_{resp}	Average request and response size on the wire	bytes

WORKED EXAMPLE

Worked example — a video-thumbnail service. $R = 100,000$ requests/s, $s_{\text{req}} = 2$ KB, $s_{\text{resp}} = 50$ KB (SI units).

1. Full proxy: $100,000 \times (2,000 + 50,000) = 5.2 \times 10^{9}$ bytes/s $= 41.6$ Gbit/s.
2. DSR: $100,000 \times 2,000 = 2 \times 10^{8}$ bytes/s $= 1.6$ Gbit/s.
3. Ratio: $41.6/1.6 = 26$.

Interpretation. With DSR, one balancer with a 10 Gbit/s NIC has room to spare; a full proxy needs at least five such NICs before any headroom. *Engineering implication.* For download-heavy L4 traffic, DSR is the cheapest scaling lever; it is unavailable at L7, because an L7 proxy must see the response to do its job.

Table 19.1 — Forwarding modes of an L4 balancer.

Mode	How it works	Client IP at backend	Return path	Constraints
Destination NAT	Rewrite destination IP; backend's gateway is the balancer	Preserved	Through balancer	Backends must route replies via the balancer
Full NAT / L4 proxy	Rewrite source and destination, or terminate TCP	Lost (use PROXY protocol)	Through balancer	Port exhaustion on the balancer's source IPs
DSR, L2 (MAC rewrite)	Rewrite destination MAC only	Preserved	Direct	Same layer-2 segment; VIP on loopback; ARP suppressed
DSR, tunnel	Encapsulate in IP-in-IP, GRE or GUE	Preserved	Direct	MTU and MSS clamping; backend decapsulates

These are the modes of Linux Virtual Server (LVS-NAT, LVS-DR and LVS-TUN), which IPVS implements in the kernel [Zhang 2000]. Maglev encapsulates packets to backends in GRE and relies on DSR [Eisenbud et al. 2016].

Advantages and disadvantages

Table 19.2 — Layer 4 versus layer 7 load balancing.

Aspect	Layer 4	Layer 7
What it is	Flow or connection balancer	Request balancer and reverse proxy
Unit of balancing	TCP connection or UDP flow	HTTP request or gRPC call
Sees	IPs, ports, protocol (and SNI if it peeks)	Method, path, headers, cookies, body
TLS	Passthrough (or termination in an L4 proxy)	Termination or re-encryption
Advantages	Very high throughput, protocol-agnostic, DSR possible, low added latency	Content routing, retries, per-request balancing, header rewriting, observability per request
Disadvantages	Long-lived connections pin load; no content routing	CPU for TLS and parsing; holds keys; more complex failure modes
Performance	Line rate on commodity servers (10 Gbit/s per Maglev machine)	Bounded by TLS handshakes and request parsing per core
Operational complexity	Routing integration (BGP, ECMP), tunnels	Configuration surface: routes, timeouts, retries
Choose when	Non-HTTP protocols, extreme volume, first tier of a stack	HTTP/gRPC services, microservices, canary routing

COMMON MISTAKE

"An L4 balancer cannot handle HTTPS." It can: with passthrough it forwards encrypted bytes and never needs keys, and with SNI it can even pick a pool per hostname. What it cannot do is route on anything *inside* the encrypted stream, such as the URL path.

COMMON MISTAKE

"Putting HTTP/2 behind an L4 balancer spreads load." It spreads *connections*. A client that multiplexes all its calls over one HTTP/2 connection sends all of them to one backend. This is the classic surprise when teams adopt gRPC behind a cloud L4 balancer (§8.8).

19.4 Load-balancing algorithms

A balancing algorithm answers one question per connection or request: *which backend?* Section 4.10.4 showed why the answer matters: one shared queue feeding all servers beats separate queues by a wide margin, because no server idles while work waits elsewhere. A load balancer cannot build one shared queue across machines; it must commit each request to one server's queue at arrival. Every algorithm below is a different approximation of the shared queue, trading information for cost.

19.4.1 Round robin and weighted round robin

DEFINITION

Definition — Round robin (RR). Send the k-th request to backend $k \bmod N$. **Weighted round robin (WRR)** gives backend i an integer weight w_i and sends it a share $w_i / \sum_j w_j$ of requests in each cycle.

First-principles explanation

A teacher handing out exam papers walks along the row giving one to each desk in turn. Nobody gets two before everyone has one, and the teacher needs no information about the students. That is round robin: perfectly fair in *count*, blind to *effort*. If some desks hold students who work twice as fast, you give them two papers per round: weighted round robin.

Why does this exist?

Round robin needs only a counter and no feedback from backends, so it is cheap and predictable; it is the default in NGINX, HAProxy, Envoy and AWS ALB. Weights handle the commonest exception: a mixed fleet.

How it works internally

A naive WRR repeats each backend w_i times in a row: with weights $a{:}\,5, b{:}\,1, c{:}\,1$ it sends `a a a a a b c`, five requests in a burst to `a`. NGINX uses **smooth weighted round robin**, which interleaves [NGINX source: ngx_http_upstream_round_robin.c]:

1. Each backend keeps a *current weight*, initially 0.
2. On each pick, add every backend's configured weight to its current weight.
3. Choose the backend with the largest current weight.
4. Subtract the total of all weights from the chosen backend's current weight.

Over one cycle of $\sum w_i$ picks each backend is chosen exactly w_i times, and the chosen backend is pushed to the back of the line each time, so picks are spread out.

Mathematical foundations

FORMULA

Weighted round-robin share

$$\lambda_i = \lambda \cdot \frac{w_i}{\sum_{j=1}^{N} w_j}, \qquad \rho_i = \frac{\lambda_i}{\mu_i}$$

Symbol	Meaning	Units
λ	Total request rate into the pool	requests/second
λ_i	Request rate sent to backend i	requests/second
w_i	Configured weight of backend i	dimensionless
N	Number of backends	count
μ_i	Capacity (service rate) of backend i	requests/second
ρ_i	Utilization of backend i	dimensionless

WORKED EXAMPLE

Worked example — a mixed fleet. Three backends with 8, 16 and 32 cores; each core serves 125 requests/s, so $\mu = 1,000$, $2,000$ and $4,000$ requests/s. Total load $\lambda = 5,600$ requests/s.

1. Unweighted RR: each gets $5,600/3 = 1,867$ requests/s. Utilizations: $1,867/1,000 = 1.87$ (overloaded, queue grows without bound), 0.93 and 0.47.
2. Weights $1 : 2 : 4$ (proportional to cores), $\sum w = 7$: $\lambda_i = 800$, $1,600$, $3,200$ requests/s.
3. Utilizations: $800/1,000 = 0.8$, $1,600/2,000 = 0.8$, $3,200/4,000 = 0.8$.

Interpretation. Total capacity is 7,000 requests/s, so the pool is at 80%, yet unweighted RR drives the smallest machine to 187% while the largest idles at 47%. *Engineering implication.* RR is only safe on homogeneous pools; with mixed instance types, set weights proportional to measured capacity, or use a load-aware algorithm.

Code example

```
        """Smooth weighted round robin, the algorithm NGINX uses for weighted up-
streams."""
        from collections import Counter


        class SmoothWRR:
            def __init__(self, weights: dict):
                self.weights = dict(weights)              # server -> configured weight
                self.current = {s: 0 for s in weights}    # server -> current weight
                self.total = sum(weights.values())

            def pick(self) -> str:
                # 1. Every server earns its weight.
                for s, w in self.weights.items():
                    self.current[s] += w
                # 2. The richest server wins (ties: first in insertion order).
                best = max(self.current, key=self.current.get)
                # 3. The winner pays the total, so it falls behind the others.
                self.current[best] -= self.total
                return best


        def naive_wrr(weights: dict) -> list:
            """Repeat each server w times in a row: correct shares, bursty order."""
            return [s for s, w in weights.items() for _ in range(w)]


        if __name__ == "__main__":
            w = {"a": 5, "b": 1, "c": 1}
            lb = SmoothWRR(w)
            seq = [lb.pick() for _ in range(7)]
            print("naive :", " ".join(naive_wrr(w)))
            print("smooth:", " ".join(seq))
            assert seq == ["a", "a", "b", "a", "c", "a", "a"]

            w2 = {"s1": 1, "s2": 2, "s3": 4}               # 8, 16 and 32 cores
            lb2 = SmoothWRR(w2)
            counts = Counter(lb2.pick() for _ in range(7_000))
            print("shares over 7,000 picks:", dict(counts))
            assert counts == {"s1": 1_000, "s2": 2_000, "s3": 4_000}
```

What to notice:

- The smooth sequence for weights 5, 1, 1 is `a a b a c a a`, the sequence documented in the NGINX source, instead of five `a`s in a row.
- Shares are exact over each cycle: 1,000, 2,000 and 4,000 picks out of 7,000.
- The algorithm is $O(N)$ per pick; NGINX accepts that because pools are small. Large pools use precomputed schedules or randomized weighted choice.

INDUSTRY EXAMPLE

Google: weighted round robin driven by backend reports. Google's SRE book describes datacenter load balancing in which clients weight backends using utilization and error rates that the backends report in their responses, after finding that plain round robin and "least-loaded" round robin left large utilization differences between the most and least loaded tasks. gRPC's `weighted_round_robin` policy, which takes weights from backend load reports, follows the same idea. *Evidence: Primary* — [Beyer et al. 2016, ch. 20; gRPC Docs: Load balancing]

19.4.2 Least connections

DEFINITION

Definition — Least connections. Send each new connection or request to the backend with the fewest **active** (in-flight) connections or requests, as counted by the balancer. *Weighted least connections* divides each count by the backend's weight. Envoy calls the request-counting variant **least request**; AWS ALB calls it **least outstanding requests**.

First-principles explanation

At a supermarket you do not join checkouts in turn; you join the shortest line. A fast cashier's line empties quickly, so she gets more customers without anyone measuring her speed. Least connections works the same way: the count of in-flight requests is a free, real-time signal that combines how much work a backend has and how fast it clears it.

How it works internally

1. The balancer increments a counter for a backend when it forwards a request (or opens a connection) and decrements it when the response completes (or the connection closes).
2. For each new request it scans for the minimum count, breaking ties randomly (ties in a fixed order send every tie to backend 0).
3. Weighted versions compare active_i / w_i.

The count is **local to one balancer**. With ten balancer instances, each sees only the connections it opened itself.

Mathematical foundations

FORMULA

Little's Law per backend (§4.11)

$$L_i = \lambda_i W_i \quad \Rightarrow \quad \lambda_i = \frac{L_i}{W_i}$$

Symbol	Meaning	Units
L_i	Average in-flight requests at backend i (its "connections")	requests
λ_i	Request rate sent to backend i	requests/second
W_i	Average time a request spends at backend i	seconds

WORKED EXAMPLE

Worked example — least connections finds the fast server. A pool has a fast backend (W = 50 ms) and a slow one (W = 200 ms, perhaps on a noisy host). Least connections keeps both at about the same count, say L = 20.

1. Fast: λ = 20/0.050 = 400 requests/s.
2. Slow: λ = 20/0.200 = 100 requests/s.
3. Split: 400/(400 + 100) = 80% to the fast backend, with no configuration.
4. Round robin at the same 500 requests/s sends 250 to each; the slow one then holds $250 \times 0.2 = 50$ in-flight requests (more, as its queue grows), $2.5\times$ the fast one.

Interpretation. Equalizing in-flight counts makes the rate to each backend inversely proportional to its latency. *Engineering implication.* Least connections is the robust default when request costs vary widely (search queries, report generation) or backends differ, and essential for long-lived connections (WebSockets, §9.6), where round robin balances connection *arrivals* but not connection *lifetimes*.

WARNING / TRADE-OFF

The fast-failure sinkhole. A backend that fails instantly (returns `503` in 1 ms because its database is down) has the fewest in-flight requests, so least connections sends it *more* traffic. The Google SRE book describes the same trap for "least-loaded" balancing [Beyer et al. 2016, ch. 20]. Pair load-aware algorithms with passive health checks that count errors (§19.5.1).

19.4.3 Least response time and EWMA

DEFINITION

Definition — Least response time. Choose the backend with the lowest recent latency, usually an **exponentially weighted moving average (EWMA)** of observed response times, often multiplied by the in-flight count so that both queue length and speed count.

An EWMA is a running average that forgets: every new sample pulls the estimate a fraction α of the way toward itself. It needs one number per backend, reacts to changes, and ignores old history.

FORMULA

Exponentially weighted moving average

$$S_k = \alpha x_k + (1-\alpha) S_{k-1}, \qquad S_k - x^* = (1-\alpha)^k (S_0 - x^*) \text{ after a step to } x^*, \qquad \alpha = 1 - e^{-\Delta t/\tau}$$

Symbol	Meaning	Units
S_k	Latency estimate after k samples	ms
x_k	The k-th observed latency	ms
α	Smoothing factor, $0 < \alpha \le 1$	dimensionless
x^*	New steady latency after a sudden change	ms
Δt, τ	Time since previous sample; decay time constant (time-based variant)	seconds

::: example **Worked example — how fast does the estimate notice a slowdown?** A backend's latency jumps from 20 ms to 200 ms (garbage-collection storm). $\alpha = 0.1$.

1. After k samples: $S_k = 200 - 180 \times 0.9^k$.
2. $k = 10$: $0.9^{10} = 0.349$, $S = 200 - 62.8 = 137$ ms.
3. $k = 22$: $0.9^{22} = 0.098$, $S = 200 - 17.7 = 182$ ms, within 10% of the truth.
4. At 100 requests/s to this backend, 22 samples arrive in 0.22 s; at 1 request/s, it takes 22 s.

Interpretation. Per-sample smoothing reacts at the speed of traffic, so a backend that gets little traffic (because it looked slow) updates slowly; the time-based form fixes the memory at τ. *Engineering implication.* Choose τ near the timescale of the problems to dodge (seconds for GC pauses), and decay idle estimates toward the pool average so a once-slow backend is retried.

INDUSTRY EXAMPLE

Twitter Finagle and Linkerd: power of two choices with peak EWMA. Finagle, Twitter's RPC library, offers a "peak EWMA" balancer: it tracks an EWMA of round-trip time that jumps *up* immediately on a slow response and decays slowly, multiplies it by the outstanding request count, and compares two randomly chosen backends. Linkerd, which grew out of Finagle, uses the same latency-aware power-of-two-choices approach. *Evidence: Primary* — [Twitter: Finagle documentation, Load balancers]

19.4.4 Power of two choices

DEFINITION

Definition — Power of two choices (P2C). For each request, pick **two** backends uniformly at random and send the request to the less loaded of the two (by in-flight count, EWMA latency or another load signal).

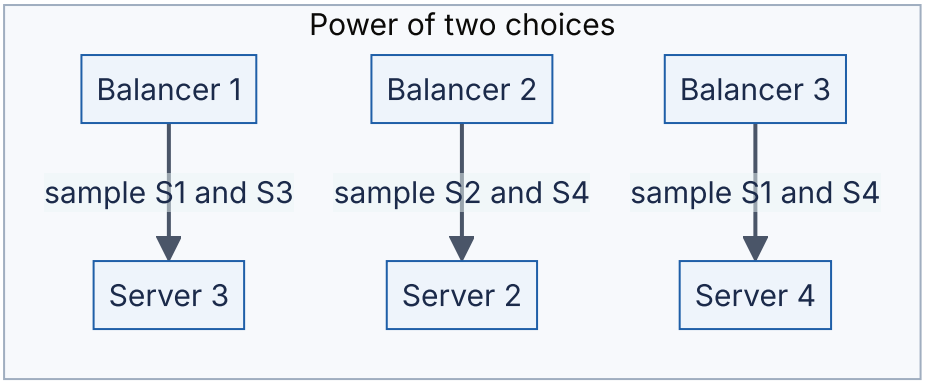
First-principles explanation

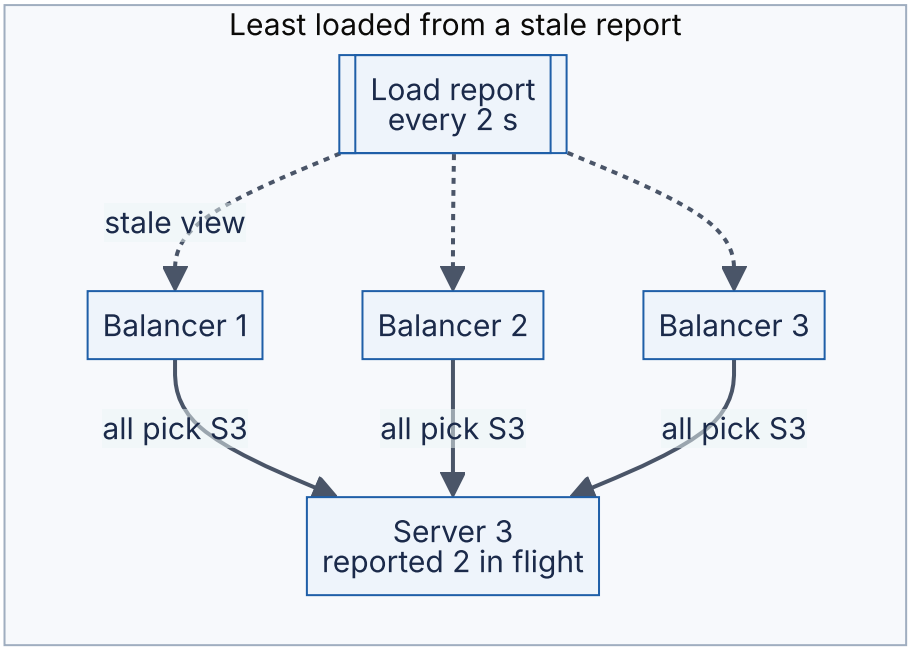
Return to the supermarket, but now the store is enormous, with 500 checkouts, and you cannot see all the lines. If you join a random line, sooner or later you join a terrible one. If you glance at just *two* random lines and join the shorter, you almost never join a terrible line, because both would have to be terrible. The surprise is how much that one extra glance buys: the worst line in the store shrinks *exponentially*.

Why does this exist?

Least connections needs a global, current view of every backend's load. That is cheap for one balancer and 10 backends, but not for 50 balancers and 5,000 backends: each balancer sees only its own traffic, and load reports arrive late. When many balancers all pick "the least loaded backend" from the same stale information, they all pick the *same* backend and swamp it, a **herd effect** (Figure 19.3). Random choice avoids herds but tolerates bad luck. Two random choices keep almost all the benefit of full information, avoid the herd, and cost two lookups [Mitzenmacher 2001].

Figure 19.3 — Herd behavior with stale load information, and how two random choices avoid it.





Reading the diagram. On the left, three balancers receive the same load report asynchronously (dotted) every 2 seconds. Server 3 looked idlest, so all three forward every new request to it (solid, synchronous) until the next report, by which time it is the busiest and the herd moves on. On the right, each balancer samples two random servers per request and picks the lighter, so choices scatter even with stale information. No storage is involved; the failure point is the shared stale view. P2C needs no coordination, so it scales to any number of balancers.

Mathematical foundations

The classic model is **balls into bins**: throw n balls into n bins, each ball choosing its bin(s) at random.

FORMULA

Maximum load with one and with d random choices [Azar et al. 1999]

one choice: max load $= \frac{\ln n}{\ln \ln n}(1 + o(1))$, $d \geq 2$ choices: max load $= \frac{\ln \ln n}{\ln d} + \Theta(1)$ (with high pr

Symbol	Meaning	Units
n	Number of balls (requests) and of bins (servers)	count
d	Number of random bins examined per ball	count
maxload	Largest number of balls in any bin	balls
$o(1)$, $\Theta(1)$	A term that vanishes as n grows; a bounded constant	—

WORKED EXAMPLE

Worked example — one million balls into one million bins.

1. $\ln n = \ln 10^6 = 13.82$; $\ln \ln n = \ln 13.82 = 2.63$.
2. One choice, leading term: $13.82/2.63 = 5.26$.
3. Two choices, leading term: $2.63/\ln 2 = 2.63/0.693 = 3.79$.
4. Measured (listing below): 9.0 with one choice, 4.0 with two, 3.0 with three.

Interpretation. The leading term underestimates the one-choice case at practical sizes (the $o(1)$ factor is large), but the shape is right: the worst bin with one choice grows almost like $\log n$, while with two choices it grows like $\log \log n$, which is below 5 for any n you will meet. Going from two choices to three helps far less than going from one to two. *Engineering implication.* Two samples are the sweet spot: almost all the benefit, no coordination, and no herd.

Mitzenmacher analyzed the dynamic version, the **supermarket model**: requests arrive as a Poisson stream at rate λn to n servers of rate 1, each joining the shorter of d random queues [Mitzenmacher 2001].

FORMULA

Queue-length tail in the supermarket model (large n)

$$s_i = \lambda^i \quad (d = 1), \qquad s_i = \lambda^{\frac{d^i - 1}{d - 1}} \quad (d \geq 2), \qquad \text{so for } d = 2\text{: } s_i = \lambda^{2^i - 1}$$

Symbol	Meaning	Units
s_i	Fraction of servers with at least i requests (queued or in service)	dimensionless
λ	Utilization: arrival rate per server divided by service rate	dimensionless
d	Number of queues sampled per request	count
i	Queue-length threshold	requests

WORKED EXAMPLE

Worked example — at 90% utilization. $\lambda = 0.9$.

1. Random ($d = 1$): $s_5 = 0.9^5 = 0.59$, so 59% of servers have 5 or more requests. $s_{10} = 0.9^{10} = 0.35$.
2. Two choices: $s_5 = 0.9^{2^5 - 1} = 0.9^{31} = 0.038$, so 3.8%. $s_{10} = 0.9^{1023} \approx 10^{-47}$: effectively never.

Interpretation. With one choice the tail decays geometrically; with two it decays **doubly exponentially**. *Engineering implication.* At high utilization the difference is between a long tail of overloaded servers and none, which is exactly what p99 latency measures.

Code example

```
        """Balls into bins: maximum load with d = 1, 2, 3 random choices."""
        import math
        import random


        def max_load(n: int, d: int, rng: random.Random) -> int:
            """Throw n balls into n bins; each ball goes to the least loaded of d random
bins."""
            bins = [0] * n
            for _ in range(n):
                if d == 1:
                    i = rng.randrange(n)
                else:
                    i = min((rng.randrange(n) for _ in range(d)), key=bins.__getitem__)
                bins[i] += 1
            return max(bins)


        if __name__ == "__main__":
            rng = random.Random(42)
            trials = 5
            print(f"{'n':>9} | {'d=1':>5} {'d=2':>5} {'d=3':>5} | "
                  f"{'ln n/ln ln n':>12} {'ln ln n/ln 2':>12} {'ln ln n/ln 3':>12}")
            for n in (1_000, 10_000, 100_000, 1_000_000):
                t = trials if n < 1_000_000 else 2
                row = [sum(max_load(n, d, rng) for _ in range(t)) / t for d in (1, 2,
3)]
                lnln = math.log(math.log(n))
                print(f"{n:>9,} | {row[0]:5.1f} {row[1]:5.1f} {row[2]:5.1f} | "
                      f"{math.log(n) / lnln:12.2f} {lnln / math.log(2):12.2f} {lnln /
math.log(3):12.2f}")
```

Output (Python 3.13):

```
        n |   d=1   d=2   d=3 | ln n/ln ln n ln ln n/ln 2 ln ln n/ln 3
    1,000 |   5.4   3.0   2.2 |         3.57         2.79         1.76
   10,000 |   6.6   3.0   3.0 |         4.15         3.20         2.02
  100,000 |   7.4   3.2   3.0 |         4.71         3.53         2.22
1,000,000 |   9.0   4.0   3.0 |         5.26         3.79         2.39
```

What to notice:

- Going from 10^3 to 10^6 bins, the one-choice maximum grows from 5.4 to 9.0; the two-choice maximum barely moves (3.0 to 4.0).
- The measured values sit above the leading terms by a roughly constant amount, the $\Theta(1)$ and $o(1)$ terms.

INDUSTRY EXAMPLE

Envoy and NGINX ship P2C. Envoy's default least-request balancer, when all hosts have equal weight, picks two random hosts and chooses the one with fewer active requests; the number of choices is configurable. NGINX supports `random two least_conn`, which its documentation recommends for setups in which several balancers serve the same backends (as of 2026). *Evidence: Primary* — [Envoy Docs: Load balancers; NGINX Docs: ngx_http_upstream_module]

19.4.5 Simulation: algorithms on heterogeneous servers

Real fleets are mixed. The simulation below has ten FIFO servers, five of rate 1 and five of rate 2 (capacity 15 requests per time unit), Poisson arrivals and exponential service. It records the **queue length each request finds** at its chosen server and its latency, and reports mean, p99 and maximum after a warm-up. The third experiment refreshes the load view only every 2 time units, modeling many balancers that share periodic load reports.

```
        """Compare load-balancing policies on heterogeneous servers.

        Ten FIFO servers: five 'slow' (1 request/s each) and five 'fast' (2 requests/s
each).
        Poisson arrivals, exponential service times. For each policy we record, for ev-
ery
        request, the queue length it finds at its chosen server (requests already there)
        and its latency (waiting + service). Times are in units of the slow server's
mean
        service time.
        """
        import random
        from collections import deque

        SPEEDS = [1.0] * 5 + [2.0] * 5          # service rate of each server,
requests/s


        def percentile(sorted_xs, p):
            """Nearest-rank percentile of an already sorted list."""
            k = max(0, min(len(sorted_xs) - 1, int(round(p / 100 * len(sorted_xs) +
0.5)) - 1))
            return sorted_xs[k]


        class Policy:
            def __init__(self, n, rng):
                self.n, self.rng, self.i = n, rng, -1

            def pick(self, qlen):               # qlen[i] = requests at server i now
                raise NotImplementedError


        class Random(Policy):
            def pick(self, qlen):
                return self.rng.randrange(self.n)


        class RoundRobin(Policy):
            def pick(self, qlen):
                self.i = (self.i + 1) % self.n
                return self.i


        class WeightedRR(Policy):               # smooth WRR, weights = server speeds
            def __init__(self, n, rng):
                super().__init__(n, rng)
                self.cur = [0.0] * n

            def pick(self, qlen):
                for s in range(self.n):
                    self.cur[s] += SPEEDS[s]
                best = max(range(self.n), key=self.cur.__getitem__)
                self.cur[best] -= sum(SPEEDS)
                return best
```

```
        class LeastConnections(Policy):        # global view, random tie-break
            def pick(self, qlen):
                m = min(qlen)
                return self.rng.choice([s for s in range(self.n) if qlen[s] == m])


        class PowerOfTwo(Policy):              # sample two distinct servers, keep the
shorter queue
            def pick(self, qlen):
                a, b = self.rng.sample(range(self.n), 2)
                if qlen[a] == qlen[b]:
                    return self.rng.choice((a, b))
                return a if qlen[a] < qlen[b] else b


        def simulate(policy_cls, load, refresh=0.0, requests=200_000, seed=1):
            """refresh > 0: the policy sees queue lengths that are refreshed only every
            `refresh` time units, as when many balancers share load reports."""
            rng = random.Random(seed)
            n = len(SPEEDS)
            lam = load * sum(SPEEDS)                       # arrival rate for target
utilization
            policy = policy_cls(n, random.Random(seed + 1))
            departures = [deque() for _ in range(n)]       # FIFO departure times per
server
            t, found, latency = 0.0, [], []
            view, next_refresh = [0] * n, 0.0
            for _ in range(requests):
                t += rng.expovariate(lam)
                for dq in departures:                      # drop requests that finished
                    while dq and dq[0] <= t:
                        dq.popleft()
                qlen = [len(dq) for dq in departures]
                if t >= next_refresh:                      # refresh = 0 means a live
view
                    view, next_refresh = qlen, t + refresh
                s = policy.pick(view)
                start = departures[s][-1] if departures[s] else t
                done = start + rng.expovariate(SPEEDS[s])
                departures[s].append(done)
                found.append(qlen[s])
                latency.append(done - t)
            warm = requests // 10                          # discard warm-up
            found, latency = sorted(found[warm:]), sorted(latency[warm:])
            return {
                "q_mean": sum(found) / len(found), "q_p99": percentile(found, 99),
                "q_max": found[-1], "lat_mean": sum(latency) / len(latency),
                "lat_p99": percentile(latency, 99),
            }


        POLICIES = [Random, RoundRobin, WeightedRR, LeastConnections, PowerOfTwo]

        if __name__ == "__main__":
            for load, refresh in ((0.6, 0.0), (0.85, 0.0), (0.85, 2.0)):
                print(f"\nLoad {load:.0%} of capacity, arrival rate {load *
```

```
sum(SPEEDS):.2f}/s, "
                      + ("live load view" if refresh == 0 else f"load view refreshed ev-
ery {refresh:g} s"))
                print(f"{'policy':<17}{'queue mean':>11}{'queue p99':>10}{'queue
max':>10}"
                      f"{'lat mean':>10}{'lat p99':>9}")
                for cls in (POLICIES if refresh == 0 else POLICIES[2:]):   # load-aware
only
                    r = simulate(cls, load, refresh)
                    print(f"{cls.__name__:<17}{r['q_mean']:>11.2f}{r['q_p99']:>10d}
{r['q_max']:>10d}"
                          f"{r['lat_mean']:>10.2f}{r['lat_p99']:>9.2f}")
```

Output (Python 3.13; times in units of the slow server's mean service time):

```
Load 60% of capacity, arrival rate 9.00/s, live load view
policy            queue mean queue p99 queue max  lat mean  lat p99
Random                  4.74        35        85      5.28    37.99
RoundRobin              2.37        19        33      3.05    21.39
WeightedRR              0.61         4        11      1.06     5.54
LeastConnections        0.08         1         2      0.75     4.01
PowerOfTwo              0.52         3         5      1.11     5.58

Load 85% of capacity, arrival rate 12.75/s, live load view
policy            queue mean queue p99 queue max  lat mean  lat p99
Random               1154.35      4168      4415   1151.50  4157.78
RoundRobin           1200.44      4280      4465   1199.30  4277.65
WeightedRR              2.92        15        26      2.59    13.79
LeastConnections        0.58         3         7      1.09     5.52
PowerOfTwo              1.75         6        10      1.98     8.93

Load 85% of capacity, arrival rate 12.75/s, load view refreshed every 2 s
policy            queue mean queue p99 queue max  lat mean  lat p99
WeightedRR              2.92        15        26      2.59    13.79
LeastConnections        6.14        27        42      4.91    25.42
PowerOfTwo              2.53         8        15      2.58    11.26
```

What to notice:

- **Load-oblivious policies fail on mixed fleets.** At 60% load, random and round robin send 0.9 requests per time unit to each slow server (90% busy) while fast ones idle at 45%: p99 queue 19 for RR and 35 for random, against 1 to 4 for the others. At 85% they send 1.275 to servers that can do 1.0; those queues grow without bound, so the numbers only reflect the run length.

19.4.6 Consistent hashing for request routing

Hash-based balancing sends all requests with the same **key** (user ID, cache key, video ID) to the same backend, so its cache or local state is reused. **Consistent hashing** places backends and keys on a ring so that adding or removing one of N backends moves only

about $1/N$ of keys, where `hash mod N` would move nearly all; the derivations, code and alternatives (jump, rendezvous hashing) are in §14.8.

Two limits of the plain ring matter here. **Balance depends on keys, not arcs**: an even split of the key space still sends a hot key's traffic to one server. And **lookups cost** $O(\log M)$ over M ring points, while per-packet L4 balancers want $O(1)$. The next two techniques address these limits. Ring hashing is available in HAProxy (`hash-type consistent`), NGINX (`hash ... consistent`, Ketama) and Envoy (`RING_HASH`).

19.4.7 Consistent hashing with bounded loads

DEFINITION

Definition — Consistent hashing with bounded loads. A variant of consistent hashing in which every backend has a **capacity** of $\lceil (1+\varepsilon)\,m/n \rceil$ for m current items (requests, sessions) and n backends. An item goes to the first backend clockwise from its hash that is below capacity, so overflow from a full backend walks on to the next [Mirrokni, Thorup & Zadimoghaddam 2018].

First-principles explanation

Each customer has a favorite checkout (her hash). The store rules that no line may exceed the average by more than 25%; if her favorite is at the limit, she walks on to the next checkout under it. Most customers still get their favorite, and no line passes the cap even when one cashier is everyone's favorite.

Why does this exist?

Hash routing exists for locality, but popularity is skewed (§14.10): one viral video can draw a large share of requests and melt its server. Bounded loads keep most of the locality while making overload impossible by construction. When an item or backend is added or removed, the expected number of moved items exceeds the minimum by only a factor $O(1/\varepsilon^2)$ for $\varepsilon \le 1$, and $1 + O(\log c/c)$ for $c = 1+\varepsilon \ge 2$ [Mirrokni, Thorup & Zadimoghaddam 2018].

FORMULA

Bounded-load capacity

$$\text{cap} = \lceil (1 + \varepsilon) \frac{m}{n} \rceil$$

Symbol	Meaning	Units
cap	Maximum items any backend may hold	items
ε	Allowed overload above the average, $\varepsilon > 0$	dimensionless
m	Items currently assigned (for request routing: in-flight requests)	items
n	Number of backends	count

WORKED EXAMPLE

Worked example — 10,000 in-flight requests on 40 cache servers.

1. Average: $m/n = 10,000/40 = 250$ requests per server.
2. $\varepsilon = 0.25$: cap $= \lceil 1.25 \times 250 \rceil = \lceil 312.5 \rceil = 313$.
3. $\varepsilon = 1$: cap $= \lceil 2 \times 250 \rceil = 500$.
4. With Zipf-distributed popularity over 2,000 videos, the most popular video alone draws about $1/H_{2000} \approx 1/8.2 \approx 12\%$ of requests, roughly 1,200, nearly five servers' worth, where H_{2000} is the 2,000th harmonic number.

Interpretation. A plain ring must put all ~1,200 requests for the top video on one server ($\approx 5\times$ average); the bounded ring spills them across about four consecutive servers on the ring. *Engineering implication.* ε is a dial between cache locality (large ε) and balance (small ε); values between 0.25 and 1 are common starting points.

```
        """Consistent hashing with bounded loads (Mirrokni, Thorup & Zadimoghaddam)."""
        import bisect
        import hashlib
        import math
        import random
        from collections import Counter


        def h(s: str) -> int:
            return int.from_bytes(hashlib.md5(s.encode()).digest()[:8], "big")


        class Ring:
            def __init__(self, servers, vnodes=100):
                pts = sorted((h(f"{s}#{v}"), s) for s in servers for v in range(vnodes))
                self.pos = [p for p, _ in pts]
                self.owner = [s for _, s in pts]
                self.servers = list(servers)

            def assign(self, keys, eps=None):
                """Place each key (a request or session) in order. eps=None: plain
                consistent hashing. Otherwise no server may exceed ceil((1+eps)*m/n)."""
                cap = math.inf if eps is None else math.ceil((1 + eps) * len(keys) /
len(self.servers))
                load, where = Counter(), []
                for k in keys:
                    i = bisect.bisect_right(self.pos, h(k)) % len(self.pos)
                    while load[self.owner[i]] >= cap:         # full: keep walking
clockwise
                        i = (i + 1) % len(self.pos)
                    load[self.owner[i]] += 1
                    where.append(self.owner[i])
                return load, where


        if __name__ == "__main__":
            rng = random.Random(7)
            servers = [f"cache-{i}" for i in range(40)]
            # 10,000 in-flight requests for 2,000 objects with Zipf(1.0) popularity:
            # every request for the same object hashes to the same point.
            objects = [f"video:{i}" for i in range(2_000)]
            zipf_w = [1 / (r + 1) for r in range(len(objects))]
            requests = rng.choices(objects, weights=zipf_w, k=10_000)
            avg = len(requests) / len(servers)
            ring = Ring(servers)
            for eps in (None, 1.0, 0.25):
                load, where = ring.assign(requests, eps)
                first = sum(1 for k, s in zip(requests, where)
                            if s == ring.owner[bisect.bisect_right(ring.pos, h(k)) %
len(ring.pos)])
                label = "plain ring" if eps is None else f"bounded eps={eps}"
                print(f"{label:<18} max load {max(load.values()):5d}
({max(load.values()) / avg:4.2f}x avg)"
                      f"  on first-choice server {first / len(requests):6.1%}")

            # Membership change: add a 41st server, count requests that change server.
```

```
            bigger = Ring(servers + ["cache-40"])
            for eps in (None, 1.0, 0.25):
                _, before = ring.assign(requests, eps)
                _, after = bigger.assign(requests, eps)
                moved = sum(1 for a, b in zip(before, after) if a != b) / len(requests)
                print(f"add 41st server, eps={eps}: {moved:5.1%} of requests change
server")
```

Output (Python 3.13):

```
plain ring         max load  1534 (6.14x avg)  on first-choice server 100.0%
bounded eps=1.0    max load   500 (2.00x avg)  on first-choice server  83.2%
bounded eps=0.25   max load   313 (1.25x avg)  on first-choice server  72.5%
add 41st server, eps=None:  1.0% of requests change server
add 41st server, eps=1.0:  1.9% of requests change server
add 41st server, eps=0.25:  3.3% of requests change server
```

What to notice:

- The plain ring's busiest server holds 6.1 times the average because every request for a hot video lands on the same point. The cap is met exactly: 500 and 313.
- With $\varepsilon = 0.25$, 72.5% of requests still reach their first-choice server; the rest are hot-object overflow.
- Bounded loads cost some stability: adding a server moves 3.3% of requests instead of 1.0%, because changing n changes the cap and shifts overflow chains.
- For request routing, m is the number of in-flight requests, so the cap moves continuously with traffic; the code takes a static snapshot for clarity.

INDUSTRY EXAMPLE

Vimeo and HAProxy. Vimeo's video-delivery team found that plain consistent hashing across its cache servers overloaded the servers holding popular videos, implemented the bounded-loads algorithm in HAProxy, and reported a large drop in cache-tier bandwidth imbalance. HAProxy exposes it as `hash-balance-factor` (a value of 125 corresponds to $\varepsilon = 0.25$). Google has reported using the algorithm in Google Cloud Pub/Sub. *Evidence: Primary* — [Rodland 2016; HAProxy Docs: hash-balance-factor; Mirrokni & Zadimoghaddam 2017]

19.4.8 Maglev hashing

DEFINITION

Definition — Maglev hashing. A consistent-hashing scheme that builds a **lookup table** of prime size M in which every entry names a backend. Each backend has its own pseudo-random **preference permutation** of table positions; backends take turns claiming their next preferred empty position until the table is full. A packet's backend is `table[hash(5-tuple) mod M]` : one array read [Eisenbud et al. 2016].

First-principles explanation

Picture 3 students sharing 7 lockers. Each has a private, shuffled preference list; in rounds, each claims the first free locker on their list. Everyone gets almost the same number, and if one student leaves and the rounds are rerun, most lockers keep their owners, because the preference lists have not changed.

Why does this exist?

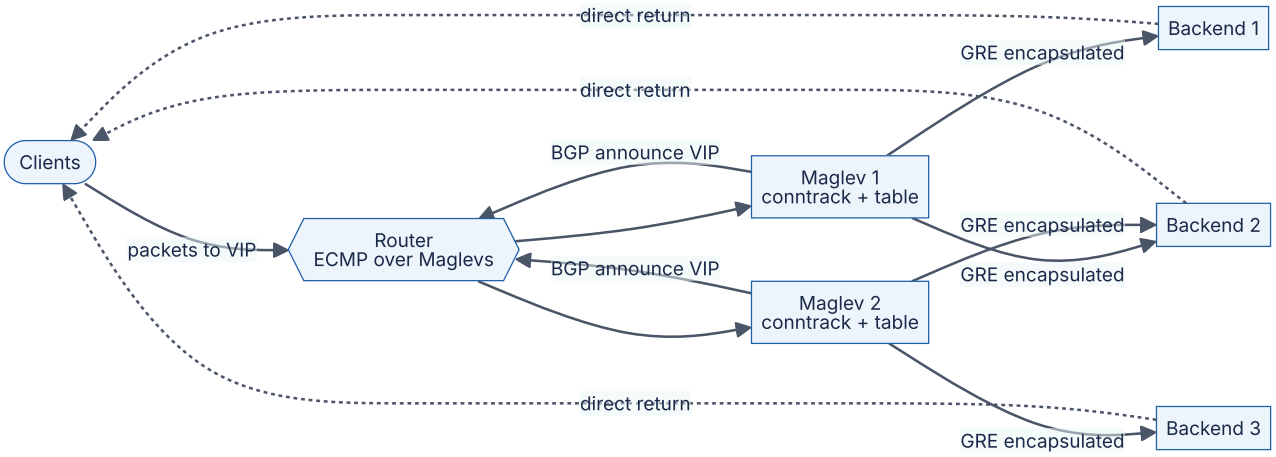
Google's Maglev is a software L4 balancer that has served Google's traffic since 2008, running on commodity Linux servers behind routers that spread packets across Maglev machines with ECMP [Eisenbud et al. 2016]. Two requirements shaped its hashing:

1. **Any Maglev machine must route any packet of a flow to the same backend**, even if the packet reaches a different Maglev machine after a router change, so the choice must be a deterministic function of the 5-tuple and the backend set.
2. **Per-packet cost must be tiny**: the paper reports a single machine saturating a 10 Gbit/s link with small packets, which leaves no room for a binary search per packet.

A ring gives (1) but $O(\log M)$ lookups and imperfect balance; Maglev hashing gives near-perfect balance and $O(1)$ lookup, at the price of slightly more than minimal disruption.

How it works internally

Figure 19.4 — Maglev: ECMP spreads packets over balancer machines; each machine maps flows to backends with the same lookup table.



Reading the diagram. Each Maglev machine announces the VIPs to the router by BGP, and the router spreads packets across them with ECMP (§19.8.3). A Maglev machine looks the flow up in its connection-tracking table; on a miss it reads `table[hash(5-tuple) mod M]`, records the result and forwards the packet in a GRE tunnel. Backends reply directly to clients (dotted: the asymmetric return path, still synchronous TCP). Tables are computed from configuration and connection state is soft, so there is no storage boundary. Failure point: if a Maglev machine dies, its flows rehash to machines with no connection entries, but those compute the same table, so most flows keep their backend. Scaling: add Maglev machines or backends.

The table is filled as follows:

1. For each backend i, compute offset_i and skip_i from two hashes of its name.
2. Its preference list is $\text{perm}_i[j] = (\text{offset}_i + j \cdot \text{skip}_i) \mod M$ for $j = 0, 1, \ldots, M-1$. Because M is prime and $1 \le \text{skip}_i < M$, this visits every position exactly once.
3. Set every entry empty. Loop over backends in turn: backend i walks its list from where it stopped to the first empty position, claims it, and the turn passes. Stop when all M entries are full.
4. To look up a packet, compute `table[hash(5-tuple) mod M]`.

Because each round gives every backend one entry, entry counts differ by at most one. Weights can be added by letting heavier backends take more turns. The paper chooses M much larger than the number of backends (it suggests $M > 100N$) so that a one-entry difference is a small fraction of each backend's share.

FORMULA

Maglev preference permutation

$$\text{offset}_i = h_1(\text{name}_i) \bmod M, \qquad \text{skip}_i = h_2(\text{name}_i) \bmod (M-1) + 1, \qquad \text{perm}_i[j] = (\text{offset}_i + j \cdot \text{skip}_i) \text{ ı}$$

Symbol	Meaning	Units
M	Lookup-table size, a prime	entries
h_1, h_2	Two independent hash functions	—
offset_i	Backend i's first preferred position	entry index
skip_i	Step between backend i's successive preferences, $1 \le \text{skip}_i \le M-1$	entries
$\text{perm}_i[j]$	Backend i's j-th preferred position	entry index

WORKED EXAMPLE

Worked example — filling a table of size 7. Backends B0 (offset 3, skip 4), B1 (offset 0, skip 2), B2 (offset 3, skip 1).

1. Preference lists: B0: $3, 0, 4, 1, 5, 2, 6$ (each step adds 4 mod 7). B1: $0, 2, 4, 6, 1, 3, 5$. B2: $3, 4, 5, 6, 0, 1, 2$.
2. Round 1: B0 claims 3. B1 claims 0. B2 wants 3 (taken), claims 4.
3. Round 2: B0's next is 0 (taken), then 4 (taken), claims 1. B1 claims 2. B2 claims 5.
4. Round 3: B0 tries 5, 2 (taken), claims 6. Table full: `[B1, B0, B1, B0, B2, B2, B0]` , counts 3, 2, 2.
5. B1 fails; rebuild with B0 and B2: B0 claims 3, B2 claims 4; B0 claims 0, B2 claims 5; B0 claims 1, B2 claims 6; B0 claims 2. Table: `[B0, B0, B0, B0, B2, B2, B2]` .
6. Compare: entries 0 and 2 (B1's) had to move. Entry 6 moved from B0 to B2 although B0 is still alive.

Interpretation. Balance is exact up to one entry, but disruption is not minimal: one of seven entries (14%) moved between survivors. *Engineering implication.* With realistic sizes the extra disruption is small (listing below), and Maglev's connection tracking hides it for established flows; only new flows, or flows that move to another Maglev machine, follow the new table.

```
        """Maglev lookup-table population (Eisenbud et al., NSDI 2016) and its disrup-
tion."""
        import hashlib
        from collections import Counter


        def _h(s: str, salt: str) -> int:
            return int.from_bytes(hashlib.md5((salt + s).encode()).digest()[:8], "big")


        def offset_skip(name: str, m: int):
            return _h(name, "offset") % m, _h(name, "skip") % (m - 1) + 1


        def populate(backends, m, params=None):
            """Return a lookup table of size m (m prime). params optionally maps a
            backend name to a fixed (offset, skip) pair, for hand-checkable examples."""
            perms = []
            for b in backends:
                off, skip = params[b] if params else offset_skip(b, m)
                perms.append((off, skip))
            nxt = [0] * len(backends)        # next preference index for each backend
            table = [None] * m
            filled = 0
            while True:
                for i, (off, skip) in enumerate(perms):     # one turn per backend
                    c = (off + nxt[i] * skip) % m
                    while table[c] is not None:              # preferred slot taken: try
next
                        nxt[i] += 1
                        c = (off + nxt[i] * skip) % m
                    table[c] = backends[i]
                    nxt[i] += 1
                    filled += 1
                    if filled == m:
                        return table


        if __name__ == "__main__":
            # Small example: M = 7, three backends with hand-picked offset and skip.
            p = {"B0": (3, 4), "B1": (0, 2), "B2": (3, 1)}
            print("B0..B2 table:", populate(["B0", "B1", "B2"], 7, p))
            print("B1 removed  :", populate(["B0", "B2"], 7, p))

            # Realistic size: M = 65,537 (prime), 100 backends; then backend-42 fails.
            M = 65_537
            names = [f"backend-{i}" for i in range(100)]
            before = populate(names, M)
            counts = Counter(before)
            print(f"entries per backend: min {min(counts.values())}, max
{max(counts.values())}, "
                  f"ideal {M / len(names):.1f}")
            after = populate([n for n in names if n != "backend-42"], M)
            changed = sum(1 for a, b in zip(before, after) if a != b)
            collateral = sum(1 for a, b in zip(before, after) if a != b and a != "back-
end-42")
```

```
        print(f"backend-42 removed: {changed / M:.2%} of entries changed "
              f"(minimum {counts['backend-42'] / M:.2%}); "
              f"{collateral / M:.2%} moved between surviving backends")
```

Output (Python 3.13):

```
B0..B2 table: ['B1', 'B0', 'B1', 'B0', 'B2', 'B2', 'B0']
B1 removed  : ['B0', 'B0', 'B0', 'B0', 'B2', 'B2', 'B2']
entries per backend: min 655, max 656, ideal 655.4
backend-42 removed: 1.66% of entries changed (minimum 1.00%); 0.66% moved between sur-
viving backends
```

What to notice:

- With 100 backends and $M = 65,537$, every backend owns 655 or 656 entries: imbalance 0.15%, far better than a ring with a few hundred vnodes per node (§14.8.4).
- Removing one backend changes 1.66% of entries: the unavoidable 1.00% plus 0.66% collateral movement between survivors, a trade Maglev accepts for $O(1)$ lookups and balance.
- Building the table is $O(M \log M)$ expected work and takes well under a second here; the paper's implementation rebuilds when backend health changes.

> INDUSTRY EXAMPLE
>
> **Open-source Maglev-style L4 balancers.** Meta's **Katran** is an L4 balancer built on the Linux kernel's XDP and eBPF facilities that uses a Maglev-style lookup table and IP-in-IP encapsulation toward backends. GitHub's **GLB Director** takes a different route: it uses rendezvous hashing to build a forwarding table in which every row lists a primary and a secondary backend, and a backend that receives a packet for a flow it does not know forwards it to the secondary ("second chance"), so connections survive backend set changes without shared connection state. *Evidence: Primary* — [Meta Engineering 2018 (Katran); Julienne 2018 (GLB)]

19.4.9 Choosing an algorithm

Table 19.3 — Load-balancing algorithms compared.

Algorithm	Load signal	State	Strength	Weakness	Choose when
Round robin	None	Counter	Simple, predictable	Overloads slow backends	Identical backends, uniform requests
Smooth weighted RR	Static weights	Weights	Handles known capacity differences	Weights go stale	Mixed instance types
Least connections	Local in-flight count	Count per backend	Adapts to speed and request cost	Herds with many balancers; fast-failure sinkhole	Single balancer tier, variable request cost, long-lived connections
Least response time (EWMA)	Observed latency	EWMA per backend	Avoids slow backends	Feedback oscillation; slow to retry	Latency-sensitive RPC
Power of two choices	In-flight or EWMA, two samples	Same as above	Near-optimal, no herding, $O(1)$	Slightly worse than perfect information	Many balancers or client-side balancing
Ring consistent hash	Key	Ring, $O(\log M)$	Locality, minimal movement	Hot keys overload one backend	Caches, sticky routing
Bounded-load hash	Key + in-flight count	Ring + counts	Locality with a load cap	More movement; needs load counts	Caches with skewed popularity
Maglev hash	5-tuple or key	Table of M entries	$O(1)$ lookup, even balance	Small extra disruption	L4 packet balancers

RECOMMENDATION

A default ladder. Start with round robin for identical stateless backends. Move to least request with P2C (Envoy's default, NGINX `random two least_conn`, HAProxy `leastconn` or `random(2)`) as soon as requests vary in cost or several balancers share backends. Use a hash, bounded if keys are skewed, only when locality pays for itself, and measure the cache hit ratio to prove it.

19.5 Health checks

19.5.1 Active and passive checks

DEFINITION

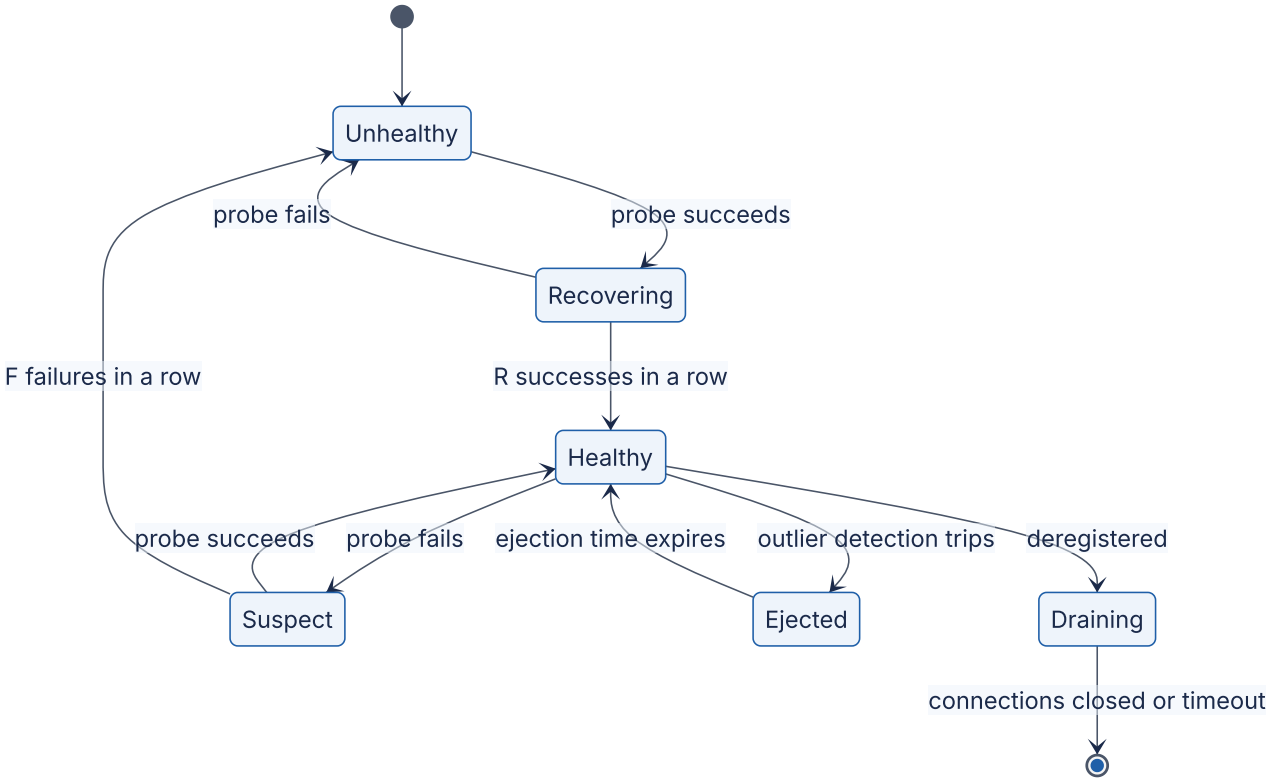
Definition — Health check. A mechanism by which a balancer decides whether a backend should receive traffic. An **active** check sends synthetic probes (a TCP connect, an HTTP `GET /healthz`, a gRPC health RPC) on a timer. A **passive** check, also called **outlier detection**, watches real traffic for errors, timeouts or latency and **ejects** misbehaving backends for a period.

A lifeguard does two things: every few minutes she scans the whole pool (active), and she reacts at once when she hears a shout (passive). Neither alone is enough. Active checks find dead backends even when no traffic flows, but are slow and test only what the probe touches. Passive checks react within milliseconds on busy services, but need traffic and cannot tell a broken backend from a bad request.

How it works internally

1. **Probe.** Every interval *I* (HAProxy default 2 s), send a probe with a timeout.
2. **Thresholds.** Mark a backend down after *F* consecutive failures (*fall*, HAProxy default 3) and up after *R* consecutive successes (*rise*, default 2) [HAProxy Docs]. AWS ALB target groups default to a 30 s interval, 2 failures and 5 successes (as of 2026).
3. **Outlier detection.** Envoy, by default, ejects a host after 5 consecutive `5xx` responses, for a base time of 30 s multiplied by the number of times it has been ejected, and never ejects more than 10% of a cluster at once (as of 2026) [Envoy Docs: Outlier detection]. NGINX open source marks a server unavailable for `fail_timeout` (10 s) after `max_fails` (1) failures.
4. **Fail open.** If too many backends look unhealthy, the checks are probably wrong. Envoy's **panic threshold** (default 50%) makes it ignore health and balance across all hosts; AWS ALB routes to all targets when all fail their checks.

Figure 19.5 — Health state of one backend as seen by the balancer.



Reading the diagram. A new backend starts unhealthy and must pass *R* probes before receiving traffic. One failed probe makes it *suspect* but keeps it in rotation; only *F* consecutive failures remove it. This **hysteresis** prevents flapping on one lost packet. Passive outlier detection can eject a backend immediately, for a time that grows with repeat offenses. *Draining* is the orderly exit (§19.8.4). Transitions are asynchronous (probe timers, response observations), and state is per balancer, so balancers may briefly disagree. Failure point: a probe that skips the real dependency path reports healthy while users fail. Scaling: probe traffic grows with balancers × backends.

FORMULA

Detection time and requests lost to a dead backend

$$T_{\text{detect}} \approx F \cdot I + T_o, \qquad N_{\text{lost}} \approx \frac{\lambda}{n} T_{\text{detect}}$$

Symbol	Meaning	Units
T_{detect}	Worst-case time from failure to removal	seconds
F	Consecutive failures required (fall threshold)	count
I	Probe interval	seconds
T_o	Probe timeout	seconds
N_{lost}	Requests sent to the dead backend meanwhile (without retries)	requests
λ, n	Total request rate; number of backends	requests/second; count

WORKED EXAMPLE

Worked example — active versus passive detection. 10 backends, $\lambda = 5,000$ requests/s, so 500 requests/s each. A backend's process hangs.

1. HAProxy defaults: $F = 3$, $I = 2$ s, timeout 2 s: $T_{\text{detect}} \approx 3 \times 2 + 2 = 8$ s; $N_{\text{lost}} \approx 500 \times 8 = 4,000$ requests.
2. ALB defaults: $F = 2$, $I = 30$ s, timeout 5 s: $T_{\text{detect}} \approx 65$ s; $N_{\text{lost}} \approx 32,500$ requests.
3. Passive, 5 consecutive errors (if the hang yields fast errors): ejection after 5 requests, about $5/500 = 10$ ms.

Interpretation. Active checks alone cost thousands of failed requests per incident; passive checks cut that by orders of magnitude on busy pools. *Engineering implication.* Use both, retry idempotent requests on another backend (§22.12), and tune active intervals for low-traffic pools where passive detection has nothing to observe.

19.5.2 What a health check should test

A **shallow** check (process up, port open) misses a backend whose database pool is exhausted. A **deep** check that calls every dependency fails the other way: when a *shared* dependency slows, every backend fails at once and the balancer ejects the whole pool, turning degradation into outage. The usual compromise: check the backend's own ability to serve (threads free, caches loaded, not shutting down), report dependency trouble as

metrics, and keep the panic threshold as a backstop. Kubernetes separates **liveness** (restart me) from **readiness** (send me traffic) probes (§42.7).

Probe load is real at scale: 1,000 sidecar proxies each checking 100 backends every 5 s generate $1,000 \times 100/5 = 20,000$ probes per second. Meshes therefore lean on passive detection and on health status pushed by the control plane.

Failure scenarios

Table 19.4 — Health-check failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Whole pool ejected	Deep checks fail on a shared dependency	Healthy-host count drops to 0	Panic threshold / fail open	Shallow readiness checks; dependency signals as metrics
Flapping	Thresholds of 1; probe timeout near p99 latency	Backend toggles every few seconds	Raise rise/fall; longer timeout	Hysteresis; ejection time growing with repeats
Healthy but broken	Probe path skips the failing code path	Errors rise while health is green	Passive ejection	Probe a representative path; outlier detection
Ejection cascade	Overload makes backends fail; ejecting them overloads the rest	Ejections rise with load	Cap ejected fraction (Envoy 10%)	Load shedding (§22.9) instead of ejection
Thundering return	Recovered backend receives a full share while caches are cold	Latency spike after recovery	Slow start	Ramp weights over 30–60 s (HAProxy `slowstart`, Envoy slow start)

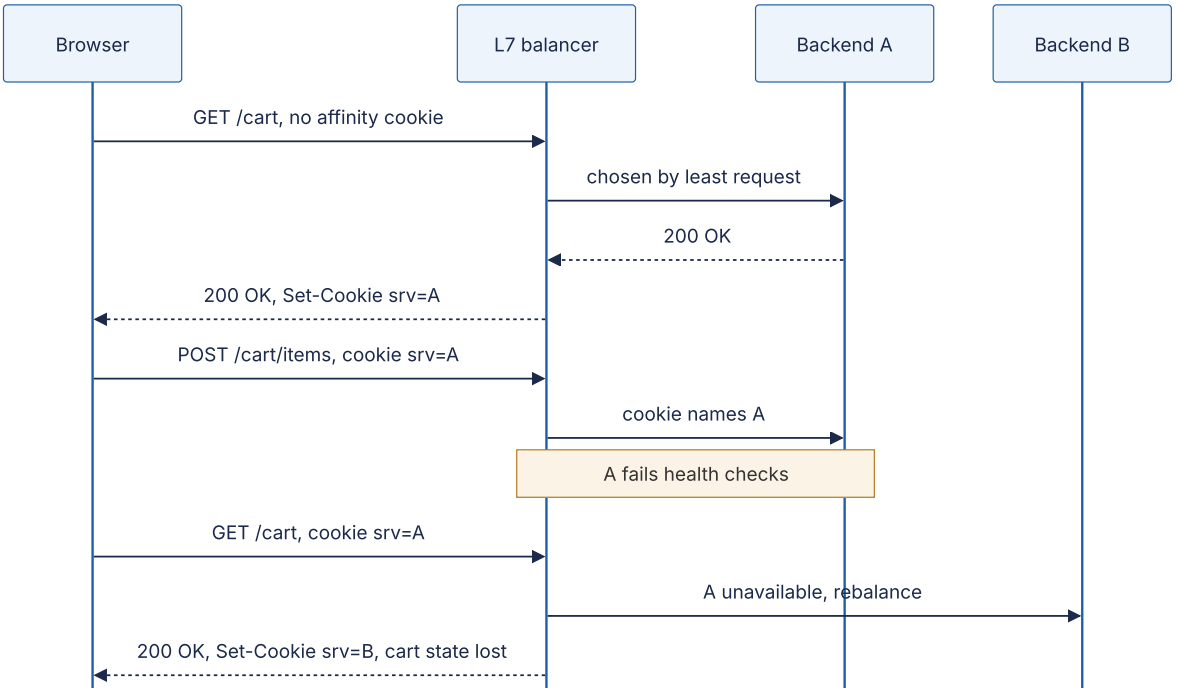
19.6 Session affinity

DEFINITION

Definition — Session affinity (sticky sessions). A balancer policy that sends all requests of one client session to the same backend, identified by a cookie, the client's IP address or a request key.

Affinity exists because some backends keep per-user state in memory: a shopping cart, a WebSocket subscription buffer (§9.6), a local cache of the user's data. It is the balancing equivalent of always seeing the same doctor: convenient, until she is on holiday.

Figure 19.6 — Balancer-inserted affinity cookie.



Reading the diagram. The first request has no cookie, so the balancer chooses normally and sets a cookie naming the backend (in practice an opaque or encrypted value). Later requests carry it and go to A. When A fails, the balancer must break affinity and pick B, and state held only in A's memory, the storage boundary here, is lost. All calls are synchronous. Scaling cost: heavy sessions stay pinned, so load skews.

Table 19.5 — Session affinity methods.

Method	How it works	Advantages	Disadvantages	Choose when
Balancer cookie	Balancer sets a cookie naming the backend	Precise, survives client IP changes	HTTP only; L7 needed	Web apps with in-memory sessions
Application cookie	Balancer hashes an app's session cookie	Uses existing session ID	Same as above	Frameworks with session IDs
Source-IP hash	Hash client IP (L4 works)	No cookies, any protocol	Carrier-grade NAT puts many users on one IP; mobile clients change IP	Non-HTTP protocols, coarse affinity
Consistent hash on key	Hash user ID or header (§19.4.6–19.4.8)	Minimal remapping on resize; works without the client's help	Hot keys; needs a key	Caches, sharded in-memory state

RECOMMENDATION

Prefer statelessness to stickiness. Keep session state in a shared store (Redis, a database) or in a signed token (§8.13), and make every backend able to serve every request (§18.5). Then affinity becomes an *optimization* for cache locality, not a *correctness* requirement, and deploys, autoscaling and failures stop logging users out.

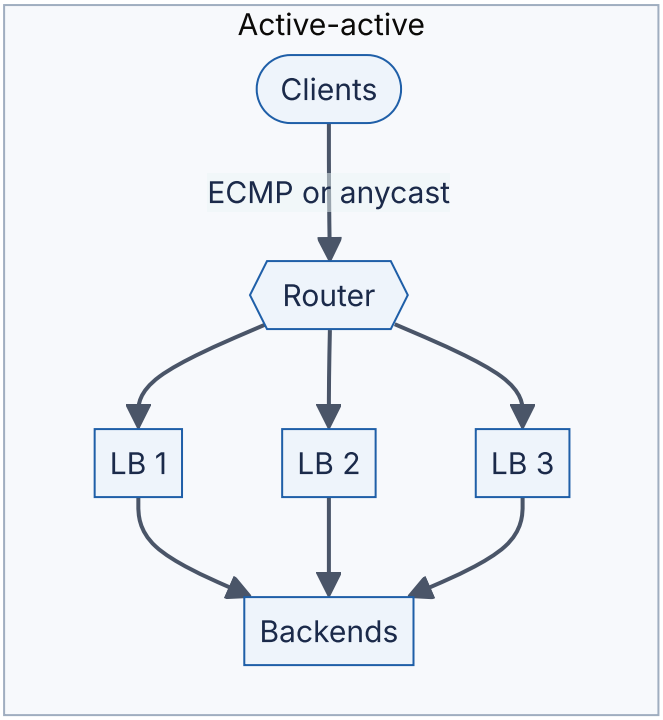
19.7 Active–passive and active–active balancer pairs

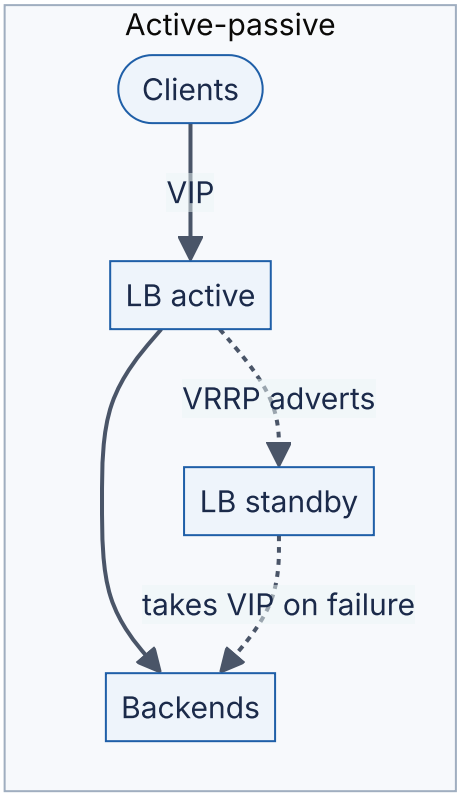
A balancer is a single point of failure until it has a partner.

DEFINITION

Definition — Active-passive and active-active. In an **active-passive** pair, one balancer carries all traffic while a standby monitors it and takes over the VIP on failure. In **active-active**, all balancers carry traffic at once (spread by DNS, ECMP or anycast), and survivors absorb a failed member’s share.

Figure 19.7 — Active-passive versus active-active balancer tiers.





Reading the diagram. Left: the active balancer carries all VIP traffic; the standby receives only heartbeat advertisements (dotted, asynchronous) and claims the VIP when they stop. Right: a router spreads flows across three active balancers and rehashes a failed member’s flows to the others. Neither tier stores data beyond connection tables. Failure points: failover time and split brain on the left; flows moving between balancers

on the right (§19.8.3). Scaling: active–passive grows only vertically; active–active by adding members.

FORMULA

Safe utilization for active-active with k tolerated failures

$$U_{\max} = \frac{N - k}{N}$$

Symbol	Meaning	Units
$U_{\max}$	Highest normal utilization per balancer that still fits all traffic after k failures	dimensionless
N	Balancers in the active–active group	count
k	Simultaneous failures to tolerate	count

WORKED EXAMPLE

Worked example — how many balancers? Peak traffic needs 30 Gbit/s; each balancer handles 10 Gbit/s; tolerate one failure ($k = 1$).

1. $N = 4$: $U_{\max} = 3/4 = 75\%$; normal load per balancer $30/4 = 7.5$ Gbit/s $= 75\%$. Exactly at the limit.
2. $N = 5$: $U_{\max} = 80\%$; normal load 6 Gbit/s $= 60\%$, leaving margin.
3. An active–passive pair would need two 30 Gbit/s balancers, with $U_{\max} = 50\%$ of the installed hardware.

Interpretation. Active–passive wastes half of the hardware but is simple; active–active wastes k/N, which shrinks as N grows. *Engineering implication.* Use active–passive for small sites and appliances, active–active behind ECMP or anycast once traffic exceeds one balancer. Either way, **test** failover: a standby that has never taken traffic is a hope, not a plan.

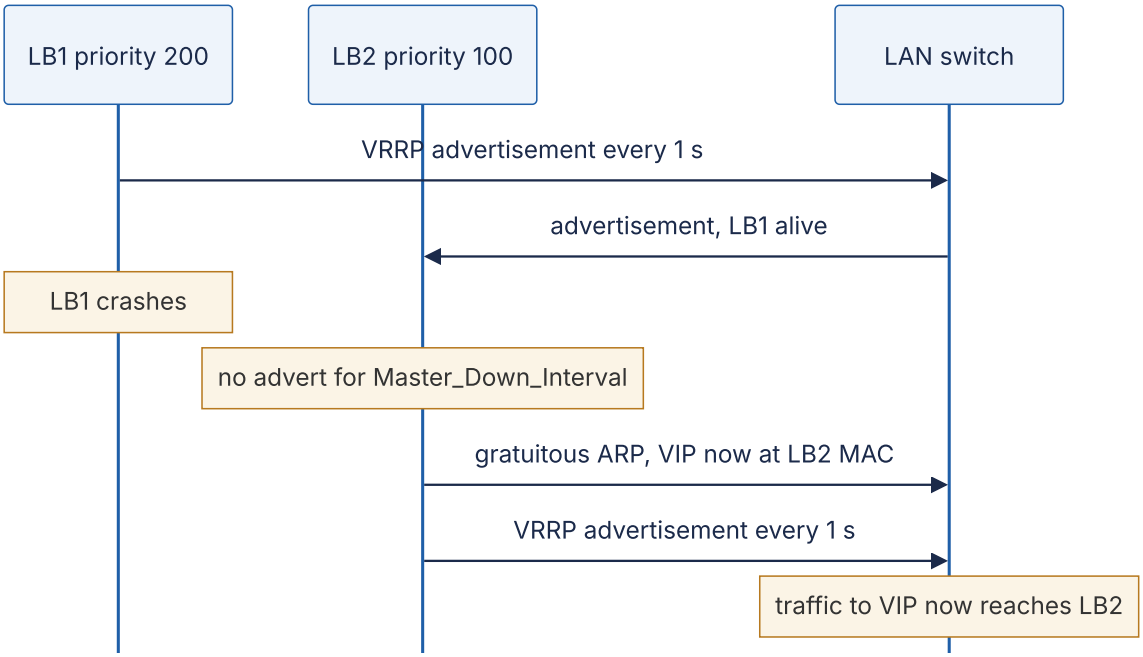
19.8 Balancer failure and redundancy

19.8.1 VRRP and keepalived

The **Virtual Router Redundancy Protocol (VRRP)** lets several machines on one LAN share a virtual IP: the highest-priority machine is the active owner (the RFC's "Master" state) and multicasts advertisements every `Advertisement_Interval`; backups take over when advertisements stop and announce the VIP's new location with gratuitous ARP [RFC

5798]. **keepalived** implements VRRP on Linux and is the usual way to make an HAProxy or NGINX pair highly available.

Figure 19.8 — VRRP failover between two balancers.



Reading the diagram. LB1 owns the VIP and advertises periodically (asynchronous multicast). When LB1 dies, LB2 waits out the master-down interval, then sends a gratuitous ARP so the switch and neighbors map the VIP to its MAC address, and begins advertising itself. Connections in flight through LB1 are lost unless connection state was synchronized (keepalived can pair with `conntrackd`). Failure points: if advertisements are blocked while both machines are alive (a firewall rule, a partitioned link), both claim the VIP, which is **split brain** (§26.12). Many clouds do not support multicast or gratuitous ARP; there, use the provider's managed balancer or a floating-IP API.

FORMULA

VRRP failover time [RFC 5798, §6.1]

$$T_{\text{skew}} = \frac{(256 - P)\,A}{256}, \qquad T_{\text{down}} = 3A + T_{\text{skew}}$$

Symbol	Meaning	Units
A	Advertisement interval (VRRPv3 allows centisecond values)	seconds
P	Backup router's priority, 1–254	dimensionless
T_{skew}	Extra wait so higher-priority backups take over first	seconds
T_{down}	Master-down interval: silence before a backup takes over	seconds

WORKED EXAMPLE

Worked example — default versus tuned timers.

1. $A = 1$ s, $P = 100$: $T_{\text{skew}} = 156/256 = 0.61$ s; $T_{\text{down}} = 3 + 0.61 = 3.61$ s.
2. $A = 0.1$ s, $P = 100$: $T_{\text{skew}} = 0.061$ s; $T_{\text{down}} = 0.361$ s.
3. At 20,000 new connections/s, a 3.61 s gap affects about 72,000 connection attempts, most of which clients retry.

Interpretation. Failover takes seconds with defaults and a few hundred milliseconds when tuned. *Engineering implication.* Shorter intervals detect faster but raise the risk of false failovers under CPU or network jitter; 100 ms is common on dedicated links, 1 s across busier ones.

19.8.2 Anycast

At Internet scale, the same VIP is announced by BGP from many sites (§7.5.3). Routers send each client to a nearby site, and a site that fails withdraws its route. Anycast makes the *first* hop redundant across sites and absorbs DDoS traffic, but it relies on BGP convergence (seconds to minutes) and can move long-lived TCP flows to a site without their state. Section 7.5.3 describes the 2021 Facebook outage in which every site withdrew at once.

19.8.3 ECMP and consistent hashing

Inside a site, the router in front of the balancers uses **equal-cost multipath (ECMP)**: it has several next hops for the VIP (one per balancer, each announcing the VIP by BGP)

and picks one per packet by hashing header fields, so all packets of a flow take the same path [RFC 2992]. When a balancer fails, the router removes one next hop and **rehashes**.

FORMULA

Flows remapped when one of N next hops is removed

$$f_{\text{mod}} \approx 1 - \frac{1}{N}, \qquad \tfrac{1}{4} \lesssim f_{\text{threshold}} \leq \tfrac{1}{2}, \qquad f_{\text{min}} = \frac{1}{N}$$

Symbol	Meaning	Units
f_{mod}	Fraction of flows that change next hop with `hash mod N`	dimensionless
$f_{\text{threshold}}$	Same, for the hash-threshold method analyzed in RFC 2992 (depends on which region is removed)	dimensionless
f_{min}	Minimum: only the failed hop's flows move (resilient or consistent hashing)	dimensionless
N	Next hops (balancers) before the failure	count

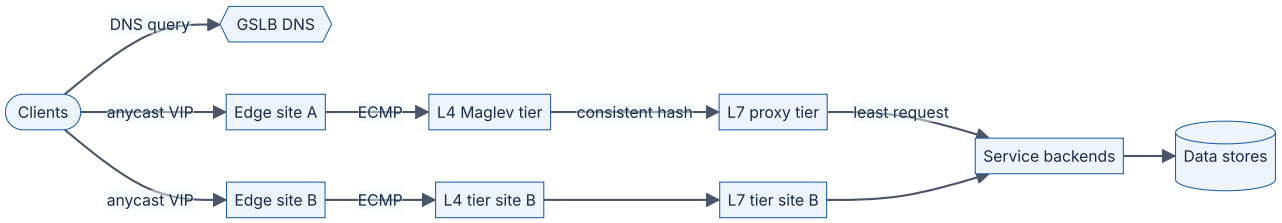
WORKED EXAMPLE

Worked example — eight balancers, one fails.

1. Modulo hashing: $1 - 1/8 = 87.5\%$ of flows land on a different balancer.
2. Hash-threshold: between 28.6% (a middle region removed) and 50% (an end region removed), computed for $N = 8$.
3. Resilient hashing: only the failed balancer's 12.5%.

Interpretation. With 1 million flows, 875,000 land on a balancer with no connection entry for them. *Engineering implication.* Balancers behind ECMP must choose backends with a **consistent** function of the 5-tuple (Maglev hashing, §19.4.8) so that a flow that changes balancer still reaches the same backend, and switches should use resilient ECMP hashing where available.

Figure 19.9 — A layered, redundant front end.



Reading the diagram. DNS (§19.9) picks a region or returns an anycast address; anycast picks the nearest edge site; ECMP spreads flows across the L4 tier, which consistently hashes them to L7 proxies; the proxies balance requests across backends with a load-aware algorithm. All arrows are synchronous request paths, and the data stores are the storage boundary. Each layer covers a failure the next cannot: anycast covers site loss, ECMP balancer loss, consistent hashing rehashing, health checks backend loss. Each tier scales horizontally. Google describes this shape for its frontend: DNS, Maglev, then Google Front End proxies [Beyer et al. 2016, ch. 19].

19.8.4 Connection draining

Removing a backend or balancer on purpose (deploy, scale-in) should break nothing. **Connection draining** (AWS: *deregistration delay*, default 300 s as of 2026; Google SRE: **lame duck** state) works in four steps:

1. Mark the backend *draining*: no new connections or requests, but in-flight ones continue.
2. For HTTP/1.1, send `Connection: close` on responses; for HTTP/2 and gRPC, send `GOAWAY` so clients open new connections elsewhere.
3. Wait until in-flight requests reach zero or a drain timeout expires (set it above the p99.9 request duration).
4. Close remaining long-lived connections (WebSockets, streams), preferably with a reconnect hint and jitter, so clients do not reconnect in one wave (§9.6).

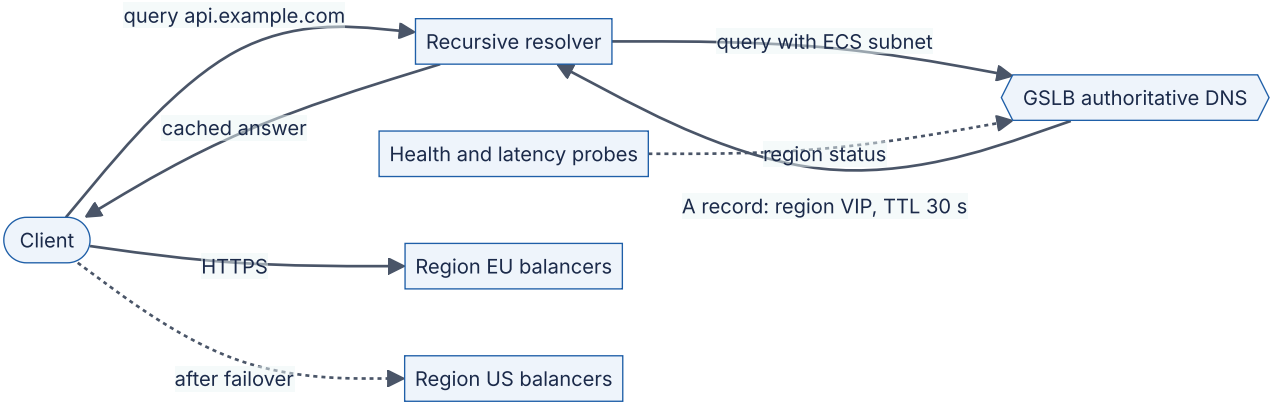
Table 19.6 — Balancer-tier failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Bad configuration everywhere	Global push of a broken rule	Error spike across all sites at once	Roll back	Staged rollout; config validation; canary balancers
Split brain	VRRP adverts lost while both nodes live	Duplicate-IP alerts; ARP flapping	Fence one node	Dedicated heartbeat links; unicast VRRP; managed VIPs
Mass connection resets	ECMP rehash after a balancer change	Reset spike on scale events	Clients reconnect	Consistent hashing; resilient ECMP; graceful BGP withdraw

19.9 DNS and global server load balancing

Global server load balancing (GSLB) chooses a *region or site* before any connection exists, usually by answering DNS queries differently per client. Section 7.6.5 covered DNS load balancing and its limits; Chapter 44 covers global traffic management in depth. Here is the essential shape.

Figure 19.10 — DNS-based global server load balancing.



Reading the diagram. The client's resolver asks the GSLB's authoritative server, which chooses a region from the resolver's location (or the client subnet, if the resolver sends EDNS Client Subnet), from latency measurements, configured weights, and region health fed asynchronously by probes. The answer is cached for its TTL by the resolver, then used by the client to connect synchronously to that region's balancers. Failure point: when a region fails, clients keep using cached answers until the TTL expires, and some clients and resolvers keep them longer. Scaling: DNS answers are cheap; the regional balancer stack (Figure 19.9) does the heavy lifting.

Policies include **geo-DNS** (nearest region), **latency-based** routing (lowest measured latency from the resolver's network, as in AWS Route 53), **weighted** answers and **failover**. The limits are those of §7.6.5: caching makes control coarse and failover TTL-bound, the resolver is not the client, and DNS sees no server load. Anycast (§19.8.2) is the main alternative and fails over at BGP speed; many providers combine both.

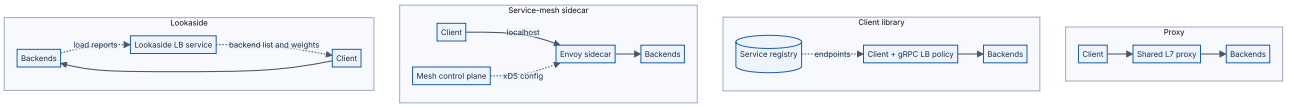
19.10 Client-side load balancing

Every proxy is an extra network hop, an extra machine to run, and an extra failure point. Inside a datacenter, the client can balance for itself.

DEFINITION

Definition — Client-side load balancing. The calling process obtains the list of backend endpoints from service discovery (§34.7) and chooses a backend for each request itself, with no proxy in the path. A **lookaside** balancer is an external service that tells clients *which* backends to use (and with what weights) while the traffic still flows directly from client to backend.

Figure 19.11 — Four places to put the balancing decision.



Reading the diagram. Solid arrows are synchronous requests; dotted arrows are asynchronous control traffic. The **proxy** adds a shared hop. The **library** (gRPC's `round_robin`, `least_request`, `weighted_round_robin`, `ring_hash` policies) balances per call inside the client, using endpoints from a registry, the storage boundary. The **sidecar** moves that logic into a local proxy usable from any language, configured by a control plane over Envoy's xDS APIs (§34.10). **Lookaside** centralizes the decision but keeps traffic direct: backends report load to a service that hands clients weighted backend lists [gRPC Docs: Load balancing]. Failure points: a stale registry, and a control-plane outage, during which sidecars must keep their last configuration. Scaling: no proxy tier; the cost moves into every client.

Client-side balancing gives per-call balancing for gRPC, saves a hop (often 0.1–1 ms in a datacenter) and removes a tier. The costs: every client needs the logic (or a sidecar), and connections multiply. With 1,000 clients and 1,000 backends, a full mesh is 10^6 connections. Google's answer is **subsetting**: each client connects to a deterministic subset of backends, chosen so every backend gets about the same number of clients; the SRE book discusses subsets of 20 to 100 [Beyer et al. 2016, ch. 20]. With subsets of 20, connections fall to 20,000, and P2C works well inside a subset because it needs no global view.

INDUSTRY EXAMPLE

Kubernetes Services. A Kubernetes `ClusterIP` Service is a form of client-side L4 balancing: `kube-proxy` on every node programs iptables, IPVS or nftables rules so that connections to the Service IP are spread over pod endpoints on the *client's* node, with no central balancer. Because it balances connections, long-lived gRPC connections still need L7 or library balancing (§42.9). *Evidence: Primary* — [Kubernetes Docs: Virtual IPs and Service Proxies]

19.11 Choosing a load balancer: products compared

Table 19.7 — Load balancer products (as of 2026; check current documentation).

Product	Layer	Algorithms (selection)	Health checking	Notable traits	Choose when
NGINX (open source)	L7 HTTP; L4 `stream`	Smooth WRR, `least_conn`, `ip_hash`, `hash ... consistent`, `random two least_conn`	Passive (`max_fails`); active in NGINX Plus	Also a web server and cache; huge install base	Web front ends, simple reverse proxying
HAProxy	L4 and L7	`roundrobin`, `leastconn`, `source`, `uri`, `hdr`, `random(<draws>)`, consistent hash with `hash-balance-factor`	Active (rise/fall, agent checks) and passive (`observe`)	Very detailed statistics, stick tables, mature	High-performance self-managed balancing
Envoy	L4 and L7 (HTTP/1, HTTP/2, HTTP/3, gRPC)	Round robin, least request (P2C), ring hash, Maglev, random	Active, outlier detection, panic threshold	Dynamic xDS configuration; service-mesh data plane	Microservices, gRPC, meshes
AWS Application Load Balancer	L7 managed	Round robin, least outstanding requests, weighted random	Active per target group	Managed TLS, path/host routing, WAF integration	HTTP services on AWS
AWS Network Load Balancer	L4 managed (TCP, UDP, TLS)	Flow hash	Active per target group	Static IP per zone, client IP preserved, very high scale	Non-HTTP, extreme volume, static IPs
Google Maglev	L4 software, in-house	Maglev hashing + connection tracking	Backend health drives table	ECMP + BGP, GRE to backends, DSR; underlies Google Cloud's network load balancing	Not a product you run; the reference design for software L4

Also worth knowing: IPVS (in the Linux kernel, used by keepalived and `kube-proxy`) and eBPF/XDP balancers such as Katran.

19.12 Summary and revision notes

SUMMARY

- **L4 versus L7.** L4 balances flows on headers (line rate, any protocol, DSR: $B = Rs_{\text{req}}$); L7 terminates TLS and balances requests (routing, retries, per-call gRPC). Large sites stack ECMP → L4 → L7.
- **Algorithms approximate one shared queue.** RR and smooth WRR ignore load; unweighted RR drove a slow server to 187% at 80% pool load. Least connections equalizes $L_i = \lambda_i W_i$, so rate $\propto 1/W_i$, but herds on stale data and rewards fast failures. EWMA latency adapts at the speed of traffic.
- **Power of two choices.** Max load falls from $\approx \ln n / \ln \ln n$ to $\ln \ln n / \ln 2 + O(1)$; queue tails from λ^i to $\lambda^{2^i - 1}$. Simulated p99 queue: 6 live, 8 stale, versus 27 for least connections on a stale view.
- **Hashing.** Consistent hashing (§14.8) gives locality; bounded loads cap backends at $\lceil (1+\varepsilon)m/n \rceil$; Maglev gives $O(1)$ lookup and balance within one entry, with ~0.66% collateral disruption at 100 backends.
- **Health.** Active detection $\approx F \cdot I + T_o$ (8 s with HAProxy defaults); passive in milliseconds. Use hysteresis, ejection caps, panic thresholds and shallow readiness checks.
- **Affinity** trades balance and resilience for locality; externalize state.
- **Redundancy.** Active–passive wastes 50%, active–active k/N. VRRP fails over in $3A + (256 - P)A/256$ (3.61 s by default). ECMP rehash can move $1 - 1/N$ of flows, so L4 tiers hash consistently. Drain before removal.
- **Global and client-side.** DNS GSLB picks regions within TTL limits; anycast fails over at BGP speed. Library, sidecar and lookaside balancing remove a hop; subsetting bounds connections.

19.13 Knowledge checkpoint

1. Why can an L4 balancer not route `/api/search` and `/api/cart` to different pools when TLS is passed through?
2. A service returns 40 KB responses to 1 KB requests at 50,000 requests/s. What bandwidth crosses the balancer with a full proxy and with DSR?
3. List the smooth WRR sequence for weights $a{:}\,3, b{:}\,1$.
4. Two backends have mean latencies of 20 ms and 80 ms. Under least connections, what fraction of traffic goes to the faster one?
5. Why does least connections herd when many balancers share periodic load reports?
6. For $\lambda = 0.8$, what fraction of servers have at least 4 requests with one and with two random choices?
7. With 9,000 sessions, 30 servers and $\varepsilon = 0.5$, what is the bounded-load capacity?

8. Why must the Maglev table size be prime?
9. A pool uses $I = 5$ s, $F = 3$, timeout 1 s, and receives 200 requests/s per backend. How many requests reach a dead backend before removal?
10. Why can deep health checks cause a full outage?
11. Six active–active balancers must survive two failures. What is the highest safe utilization?
12. Why do balancers behind ECMP need consistent hashing to backends?

Answers

1. The path is inside the encrypted stream; without terminating TLS the balancer sees only IPs, ports and the SNI hostname.
2. Full proxy: $50,000 \times 41,000 = 2.05$ GB/s $= 16.4$ Gbit/s. DSR: $50,000 \times 1,000 = 50$ MB/s $= 0.4$ Gbit/s.
3. Current weights start at 0. Picks: *a* (3,1 → −1,1), *a* (2,2 → tie, *a* first → −2,2), *b* (1,3 → 1,−1), *a* (4,0 → 0,0). Sequence `a a b a`.
4. $\lambda \propto 1/W$: $(1/20)/(1/20 + 1/80) = 0.05/0.0625 = 80\%$.
5. All balancers see the same “least loaded” backend until the next report and all send it traffic, overloading it; the next report then moves the herd elsewhere.
6. One choice: $0.8^4 = 0.41$. Two choices: $0.8^{15} = 0.035$.
7. $\lceil 1.5 \times 9,000/30 \rceil = \lceil 450 \rceil = 450$.
8. So that every skip value in $1..M-1$ is coprime with M, making each preference list a full permutation of positions.
9. $T_{\text{detect}} \approx 3 \times 5 + 1 = 16$ s; $200 \times 16 = 3,200$ requests.
10. When a shared dependency degrades, every backend fails its check together and the balancer ejects all of them.
11. $(6 - 2)/6 = 66.7\%$.
12. A router rehash moves flows to other balancers that have no connection state; only a deterministic, consistent mapping sends those flows to the same backend.

19.14 Interview questions

Basic

Q1. What is the difference between an L4 and an L7 load balancer, and when would you use each? Answer. L4 balances TCP/UDP flows on headers, so it is fast, protocol-agnostic and can use DSR, but every request on a connection goes to one backend. L7 terminates TLS, parses HTTP, and balances and routes each request, at higher CPU cost. Use L4 for non-HTTP traffic and as the high-volume first tier; L7 for HTTP and gRPC services, path routing and retries. Large systems use both.

Q2. Your backends are a mix of 8-core and 32-core machines. Which algorithm do you use? Answer. Not plain round robin, which gives equal shares and overloads small machines. Weighted round robin with weights 1:4 works if capacity is stable; least request with power of two choices adapts automatically to actual speed and is safer when performance drifts.

Intermediate

Q3. Explain power of two choices and why it beats "always pick the least loaded server" in large systems. Answer. Sample two random backends and pick the less loaded. The maximum load drops from about $\ln n / \ln \ln n$ to $\ln \ln n / \ln 2$, and queue tails fall doubly exponentially. "Least loaded" needs a global, fresh view; with many balancers and stale reports they all pick the same backend (herding). P2C's randomness scatters choices, needs no coordination and tolerates staleness: in our simulation its p99 queue was 8 versus 27 for least connections with 2-second-old data.

Q4. Design health checking for 300 backends behind a fleet of balancers. Answer. Active readiness checks (2–5 s interval, fall 3, rise 2) that test the backend's own ability to serve; passive outlier detection (eject after 5 consecutive 5xx, ejection time growing with repeats, at most 10% ejected); a panic threshold that fails open; slow start on recovery; retries of idempotent requests elsewhere. At mesh scale, prefer passive signals to limit probe volume.

Advanced

Q5. Design the front-end load-balancing stack for a global service at 5 Tbit/s. Answer. GSLB DNS or anycast to pick a site; within the site, ECMP across a software L4 tier (Maglev-style hashing plus connection tracking, DSR or encapsulation) sized for $N - k$ survivors; an L7 proxy tier that terminates TLS and routes by path with least request/P2C; client-side or mesh balancing inside. Discuss consistent hashing across ECMP changes, connection draining, staged configuration rollouts, DDoS absorption, and failure domains per site.

Q6. A cache tier uses consistent hashing on object ID and one server is at 100% CPU during a viral event. What do you do? Answer. The hot key is pinned to one point on the ring. Switch to consistent hashing with bounded loads ($\varepsilon \approx 0.25$) so overflow spills to the next servers, or replicate hot keys to several servers and choose among them with P2C; add a small in-process cache in front for the hottest objects. Accept slightly lower hit ratio for a capped load.

Q7. Your gRPC service sits behind an AWS NLB and one pod receives most of the traffic. Why, and how do you fix it? Answer. NLB balances TCP connections, and gRPC multiplexes all calls from a client over one long-lived HTTP/2 connection, so a few clients

pin to a few pods. Fixes: an L7 balancer that balances per call (ALB with gRPC, Envoy), client-side balancing with a gRPC policy over headless-service endpoints, or a mesh sidecar; as a stopgap, a maximum connection age forcing periodic reconnects.

19.15 Practical exercises

1. Add a `WeightedLeastConnections` policy to the §19.4.5 simulation and compare it with P2C at 85% load, with live and stale views.
2. Extend the simulation to model 5 independent balancers, each counting only its own in-flight requests. Which policy wins now?
3. Change the §19.4.8 program to measure disruption when a backend is *added*, and plot collateral movement against M/N.
4. Configure HAProxy with three backends and `hash-balance-factor 125`; drive skewed traffic with a load generator and record per-server request counts.
5. Set up two VMs with keepalived and measure failover time for advertisement intervals of 1 s and 0.1 s.

19.16 Design challenges

1. **Video edge cache.** 200 cache servers per site serve 2 million objects with Zipf popularity. Choose the routing scheme from the L7 tier to the caches, set ε, estimate hit ratio loss versus plain consistent hashing, and describe behavior during a cache server failure.
2. **Internal RPC platform.** 3,000 services, 40,000 pods, mostly gRPC. Choose between proxy, sidecar, library and lookaside balancing; size subsets; define health checking and outlier detection; explain what happens when the control plane is unavailable for 10 minutes.

19.17 References

1. Amazon Web Services. *Elastic Load Balancing User Guides: Application Load Balancers; Network Load Balancers* (target groups, health checks, routing algorithms, deregistration delay). https://docs.aws.amazon.com/elasticloadbalancing/
2. Azar, Y., Broder, A. Z., Karlin, A. R., Upfal, E. “Balanced Allocations.” *SIAM Journal on Computing* 29(1):180–200, 1999.
3. Beyer, B., Jones, C., Petoff, J., Murphy, N. R. (eds.). *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly, 2016. Chapter 19, “Load Balancing at the Frontend”; Chapter 20, “Load Balancing in the Datacenter.” https://sre.google/sre-book/table-of-contents/

4. Eisenbud, D. E., Yi, C., Contavalli, C., Smith, C., Kononov, R., Mann-Hielscher, E., Cilingiroglu, A., Cheyney, B., Shang, W., Hosein, J. D. “Maglev: A Fast and Reliable Software Network Load Balancer.” *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2016.
5. Envoy Project. *Envoy documentation: Load balancing; Health checking; Outlier detection; Panic threshold*. https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/upstream
6. gRPC Authors. *Load Balancing in gRPC*. https://github.com/grpc/grpc/blob/master/doc/load-balancing.md
7. HAProxy Technologies. *HAProxy Configuration Manual* (`balance`, `hash-type`, `hash-balance-factor`, health-check options). https://docs.haproxy.org/
8. Hopps, C. *RFC 2992: Analysis of an Equal-Cost Multi-Path Algorithm*. IETF, 2000. https://www.rfc-editor.org/rfc/rfc2992
9. Julienne, T. “GLB: GitHub’s open source load balancer.” GitHub Blog, 2018. https://github.blog/
10. Kubernetes Authors. *Virtual IPs and Service Proxies*. https://kubernetes.io/docs/reference/networking/virtual-ips/
11. Meta Engineering. “Open-sourcing Katran, a scalable network load balancer.” engineering.fb.com, 2018. https://github.com/facebookincubator/katran
12. Mirrokni, V., Thorup, M., Zadimoghaddam, M. “Consistent Hashing with Bounded Loads.” *Proceedings of the 29th ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 587–604, 2018. arXiv:1608.01350.
13. Mirrokni, V., Zadimoghaddam, M. “Consistent Hashing with Bounded Loads.” Google Research Blog, 2017. https://research.google/blog/consistent-hashing-with-bounded-loads/
14. Mitzenmacher, M. “How Useful Is Old Information?” *IEEE Transactions on Parallel and Distributed Systems* 11(1):6–20, 2000.
15. Mitzenmacher, M. “The Power of Two Choices in Randomized Load Balancing.” *IEEE Transactions on Parallel and Distributed Systems* 12(10):1094–1104, 2001.
16. Nadas, S. (ed.). *RFC 5798: Virtual Router Redundancy Protocol (VRRP) Version 3 for IPv4 and IPv6*. IETF, 2010. https://www.rfc-editor.org/rfc/rfc5798
17. NGINX. *Module ngx_http_upstream_module* and source file `src/http/ngx_http_upstream_round_robin.c`. https://nginx.org/en/docs/http/ngx_http_upstream_module.html
18. Rodland, A. “Improving load balancing with a new consistent-hashing algorithm.” Vimeo Engineering Blog, 2016.
19. Twitter. *Finagle documentation: Clients — Load balancing*. https://twitter.github.io/finagle/guide/Clients.html
20. Zhang, W. “Linux Virtual Server for Scalable Network Services.” *Ottawa Linux Symposium*, 2000.

19.18 Open verification items

- Maglev paper: confirm GRE encapsulation wording, the $M > 100N$ guidance and the table sizes used in its evaluation (65,537 is used here as a typical prime); full author list.
- AWS defaults (ALB health-check interval 30 s, thresholds 5/2, deregistration delay 300 s) and Envoy outlier-detection defaults (5 consecutive 5xx, 30 s base, 10% max): confirm against current documentation (as of 2026).
- HAProxy: default `<draws>` for `balance random`; `hash-balance-factor` semantics.
- Vimeo post (Rodland 2016): exact title, date and reported results; Google Cloud Pub/Sub use per Google Research blog (2017).
- Katran blog post: authors and date.
- Google SRE book subset sizes (20–100): confirm wording in ch. 20.

Chapter 20 — Caching

> **Level:** 2 · **Track:** A (Traditional) · **Estimated reading time:** 5 h

Almost every large system you will design reads far more than it writes, and reads the same few things again and again. A product page, a user's profile, a feed's first screen: each is computed from durable storage that takes milliseconds to answer and can serve only so many queries per second. A **cache** keeps a copy of recent answers in faster memory, closer to whoever asks, so that most requests never reach the slow store. Done well, a cache cuts average latency by an order of magnitude and database load by two. Done badly, it serves stale data, melts the database when it empties, and hides bugs that appear only under load.

You already met caches inside a CPU (§5.4) and inside a database (§13.4). This chapter is about the caches you add yourself: in the browser, at the edge, in your application and in dedicated cache servers such as Redis and Memcached. It explains why a cache holding 1% of your data can absorb more than half the traffic, how to read and write through a cache without corrupting it, how to choose an eviction policy, and how to survive the three classic failure modes of production caches: stampedes, penetration and avalanches.

20.1 Learning objectives

After this chapter you should be able to:

- Compute effective latency and backend load from a hit ratio, and explain why the miss ratio, not the hit ratio, is the number to watch.
- Explain with the Zipf distribution why small caches capture most requests, and estimate the hit ratio of a cache of a given size.
- Place caches in the right layers (client, CDN, gateway, in-process, remote, database) and choose between in-process and remote caches.
- Describe how Redis and Memcached work internally: event loop, data structures, persistence, replication, Redis Cluster hash slots, the slab allocator and segmented LRU.
- Draw the sequence of cache-aside, read-through, write-through, write-back and write-around, and identify the consistency races in each.
- Choose TTLs, add jitter, and choose among LRU, LFU, ARC and W-TinyLFU using measured hit ratios.

- Design invalidation with TTLs, explicit deletes, change data capture and versioned keys.
- Prevent cache stampedes with locks, leases, probabilistic early expiration (XFetch) and stale-while-revalidate, and size XFetch's parameter.
- Defend against cache penetration, avalanches and hot keys.

20.2 Prerequisites

- Latency numbers and percentiles — §5.9.4, §4.8 (every argument here compares memory, network and disk latencies, and tails matter).
- CPU caches and average memory access time — §5.4.5 (the same formula governs application caches).
- Buffer pools and eviction (LRU, clock, LRU-K) — §13.4 (database-side caching and scan resistance).
- Read replicas and replication lag — §14.3–§14.4 (caches and replicas are both stale copies).
- Consistent hashing — §14.8 (how keys are spread over cache nodes).
- Bloom filters and Count-Min sketches — §17.4, §17.6 (used against penetration, for hot-key detection and in TinyLFU).

20.3 Cache fundamentals and hit ratio

20.3.1 What a cache is

> DEFINITION
>
> **Definition — Cache.** A store that holds copies of data whose authoritative version lives elsewhere (the **backing store** or **source of truth**), placed so that reading the copy is faster or cheaper than reading the original. A lookup that finds the data is a **hit**; one that does not is a **miss**. The **cache hit ratio** h is hits divided by lookups; the **miss ratio** is $1 - h$.

First-principles explanation

A reference librarian keeps a shelf of the 200 books people ask for most behind the desk. Most questions are answered from that shelf in seconds; the rare request for an obscure title means a trip to the stacks, after which the book may replace something on the shelf that nobody has asked for lately. Three questions decide how well this works: how often the shelf has the book (hit ratio), how much faster the shelf is than the stacks (the latency gap), and what happens when a book on the shelf is out of date because a new edition came out (staleness).

Why does this exist?

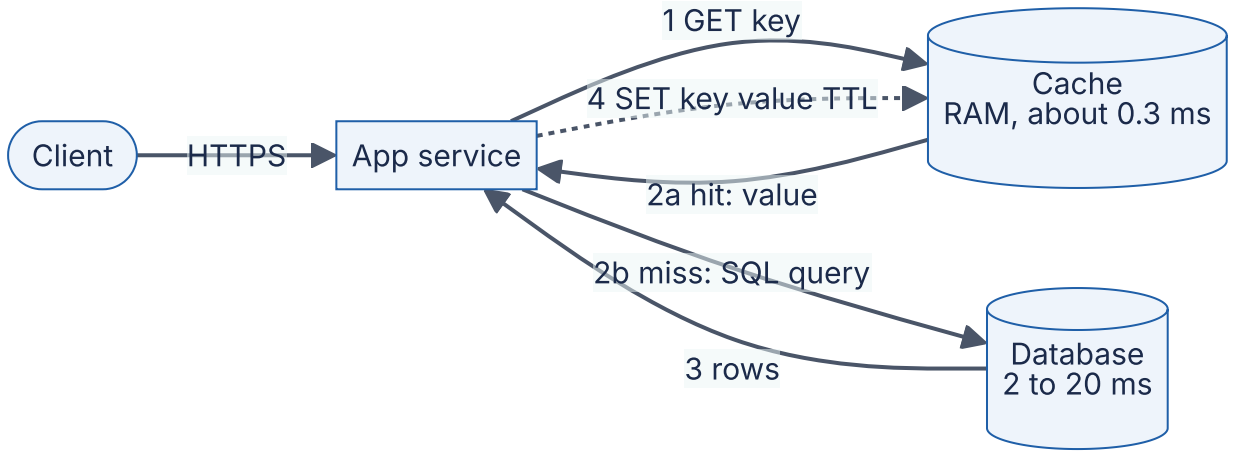
Two engineering problems motivate every cache. **Latency**: a key–value read from memory on another machine in the same datacenter costs about 0.2–0.5 ms, while a database query that touches disk or does real work costs 2–50 ms; a page that makes 20 such reads cannot meet a 200 ms budget without caching. **Capacity**: a database primary may sustain 10,000–50,000 simple queries per second, and every extra read replica adds cost and replication lag (§14.3). A cache node serving 100,000+ reads per second from RAM is cheaper per read by one to two orders of magnitude. Caches buy both, at the price of **staleness**, **memory cost** and **another component that can fail**.

How it works internally

Every cache, from an L1 CPU cache to a CDN, implements the same five decisions:

1. **Lookup**: map a **cache key** (for example `user:42:profile:v3`) to a stored entry, usually through a hash table.
2. **Fill**: on a miss, fetch from the backing store and decide whether to store the result (**admission**).
3. **Expiry**: decide how long an entry is trusted (**time to live**, TTL, §20.8).
4. **Eviction**: when memory is full, decide which entry to discard (§20.9).
5. **Invalidation**: when the source changes, remove or update the copy (§20.10).

Figure 20.1 — The hit and miss paths of a cache in front of a database.



Reading the diagram. The service first asks the cache over TCP; on a hit (2a) it answers immediately, on a miss (2b) it queries the database and fills the cache (4) so the next reader hits. Steps 1–3 are synchronous; the fill (dotted) need not delay the response. Storage boundaries: the disposable cache in RAM and the durable database. Failure points: a cache outage sends every request down path 2b (§20.13); a fill racing a write can store stale data (§20.7.2). Scaling: partition keys over cache nodes (§20.14); the database sees only misses.

20.3.2 Hit ratio, effective latency and backend load

Mathematical foundations

You met this formula for CPU caches as average memory access time (§5.4.5). For an application cache, every request pays for the lookup, and a miss adds the backend read.

FORMULA

Effective latency and backend load of a look-aside cache

$$T_{\text{eff}} = t_c + (1 - h)\, t_b \qquad R_{\text{db}} = (1 - h)\, R_{\text{read}} + R_{\text{write}}$$

Symbol	Meaning	Units
T_{eff}	Mean latency of a read through the cache	ms
t_c	Cache lookup latency (paid on hits and misses)	ms
t_b	Backend read latency, including the fill	ms
h	Cache hit ratio	dimensionless (0–1)
R_{read}	Read requests reaching the cache	requests/s
R_{write}	Writes, which go to the database regardless	requests/s
R_{db}	Requests reaching the database	requests/s

WORKED EXAMPLE

Worked example — What one percentage point of hit ratio is worth. A profile service receives $R_{\text{read}} = 50,000$ reads/s and $R_{\text{write}} = 500$ writes/s. A Redis lookup costs $t_c = 0.3$ ms; a database read costs $t_b = 8$ ms.

1. At $h = 0.90$: $T_{\text{eff}} = 0.3 + 0.10 \times 8 = 1.1$ ms; $R_{\text{db}} = 0.10 \times 50,000 + 500 = 5,500$ queries/s.
2. At $h = 0.98$: $T_{\text{eff}} = 0.3 + 0.02 \times 8 = 0.46$ ms; $R_{\text{db}} = 1,000 + 500 = 1,500$ queries/s.
3. At $h = 0.99$: $T_{\text{eff}} = 0.3 + 0.01 \times 8 = 0.38$ ms; $R_{\text{db}} = 500 + 500 = 1,000$ queries/s.
4. Going from 99% to 98% raises *read* traffic at the database from 500 to 1,000 queries/s: a one-point drop in hit ratio **doubles** the database's read load.
5. Tail: with a 2% miss ratio, more than 1% of requests are misses, so the p99 latency is a miss (about 8.3 ms) even though the mean is 0.46 ms.

Interpretation. Gains at high hit ratios look small as percentages but are large as ratios of miss ratios. **Engineering implication.** Monitor and alert on the miss ratio and on absolute database queries per second, size the database for the miss traffic you would see after a cache restart (§20.13), and remember that the mean hides the tail: p99 latency is set by misses whenever the miss ratio exceeds 1%.

COMMON MISTAKE

"Our cache hit ratio is 95%, so the cache is working." 95% of 1,000 RPS leaves 50 misses/s; 95% of 1,000,000 RPS leaves 50,000, enough to overwhelm most databases. And a page with 20 lookups and one miss is a "95% hit ratio" but a 100% slow page.

20.3.3 Popularity skew: why small caches work

First-principles explanation

If every key were equally popular, a cache holding 1% of the keys would serve 1% of requests, and caching would rarely be worth it. Real access patterns are highly skewed: a few celebrities, products and videos receive most of the traffic, and a long tail of items is requested rarely. Skew is what makes caching pay.

DEFINITION

Definition — Zipf (Zipf-like) distribution. A popularity law in which the item of popularity rank i is requested with probability proportional to $1/i^s$, where $s > 0$ is the **skew exponent**. Breslau et al. found that requests to web proxies follow a Zipf-like law with an exponent below 1 and that, as a consequence, hit ratio grows roughly logarithmically with cache size [Breslau et al. 1999].

Mathematical foundations

FORMULA

Zipf request probability and the hit ratio of an ideal top-C cache

$$p_i = \frac{1/i^s}{H_{N,s}}, \qquad H_{n,s} = \sum_{j=1}^{n} \frac{1}{j^s}, \qquad h_{\text{ideal}}(C) = \frac{H_{C,s}}{H_{N,s}} \overset{s=1}{\approx} \frac{\ln C + \gamma}{\ln N + \gamma}$$

Symbol	Meaning	Units
N	Number of distinct keys	keys
C	Cache capacity	entries
i	Popularity rank (1 = most popular)	dimensionless
s	Zipf skew exponent	dimensionless
p_i	Probability that a request is for rank i	dimensionless
$H_{n,s}$	Generalized harmonic number	dimensionless
$h_{\text{ideal}}(C)$	Hit ratio of a cache that always holds the top C keys	dimensionless
γ	Euler–Mascheroni constant, ≈ 0.5772	dimensionless

WORKED EXAMPLE

Worked example — One percent of the catalog, two thirds of the traffic. $N = 10^6$ keys with $s = 1$; the cache holds $C = 10^4$ entries (1%).

1. $\ln C + \gamma = \ln 10^4 + 0.5772 = 9.210 + 0.577 = 9.787$.
2. $\ln N + \gamma = \ln 10^6 + 0.5772 = 13.816 + 0.577 = 14.393$.
3. $h_{\text{ideal}} = 9.787/14.393 = 0.680$: 1% of the keys can serve 68% of requests (the exact sum, computed by the program below, is also 68.0%).
4. Doubling the cache to 2×10^4: $(\ln(2 \times 10^4) + 0.577)/14.393 = (9.903 + 0.577)/14.393 = 0.728$. Twice the memory buys only 4.8 more points.

Interpretation. Under Zipf, each *doubling* of cache size adds a roughly constant number of points ($\ln 2/14.393 \approx 4.8$ here), so the first gigabytes of cache are worth far more than the last. **Engineering implication.** Size a cache by plotting hit ratio against size from a real trace and stop where the next doubling no longer pays for itself in database capacity; for lower skew ($s < 1$) the curve is flatter and you need a larger fraction of the data.

Code example

The program computes the ideal hit ratio and then simulates a real LRU cache on a sampled Zipf trace, for three skews and three cache sizes.

```
        """Why small caches work: the ideal hit ratio of a cache that holds the C most
        popular of N keys under Zipf popularity, versus a simulated LRU cache."""
        import itertools
        import random
        from collections import OrderedDict

        def zipf_weights(n, s):
            """Unnormalized Zipf weights 1/i^s for ranks 1..n."""
            return [1.0 / i ** s for i in range(1, n + 1)]

        def ideal_hit_ratio(weights, c):
            """Fraction of requests for the top-c keys: H(c, s) / H(n, s)."""
            return sum(weights[:c]) / sum(weights)

        def zipf_trace(weights, length, seed=1):
            """Sample `length` requests; key i has probability weights[i] / sum."""
            cum = list(itertools.accumulate(weights))
            return random.Random(seed).choices(range(len(weights)), cum_weights=cum,
k=length)

        def lru_hit_ratio(trace, capacity, warmup):
            cache, hits = OrderedDict(), 0
            for t, key in enumerate(trace):
                if key in cache:
                    cache.move_to_end(key)
                    hits += t >= warmup
                else:
                    cache[key] = None
                    if len(cache) > capacity:
                        cache.popitem(last=False)          # evict least recently used
            return hits / (len(trace) - warmup)

        if __name__ == "__main__":
            N, LENGTH, WARMUP = 100_000, 1_000_000, 200_000
            sizes = [100, 1_000, 10_000]                    # 0.1%, 1%, 10% of the keys
            print(f"N = {N:,} keys, {LENGTH:,} requests; hit ratio by cache size")
            print("   s   policy        " + "".join(f"{c / N:>9.1%}" for c in sizes))
            for s in (0.7, 0.9, 1.0):
                w = zipf_weights(N, s)
                trace = zipf_trace(w, LENGTH)
                ideal = [ideal_hit_ratio(w, c) for c in sizes]
                lru = [lru_hit_ratio(trace, c, WARMUP) for c in sizes]
                print(f"{s:>4}   ideal top-C    " + "".join(f"{h:>9.1%}" for h in ideal))
                print(f"{s:>4}   LRU (sim)      " + "".join(f"{h:>9.1%}" for h in lru))
            w = zipf_weights(1_000_000, 1.0)
            print(f"worked example: N = 1e6, s = 1, C = 1e4 -> ideal "
                  f"{ideal_hit_ratio(w, 10_000):.1%}")
```

Output (Python 3.13, about 4 s):

```
N = 100,000 keys, 1,000,000 requests; hit ratio by cache size
   s   policy             0.1%     1.0%    10.0%
 0.7   ideal top-C       10.2%    23.1%    48.8%
 0.7   LRU (sim)          2.4%    10.9%    34.7%
 0.9   ideal top-C       29.0%    47.4%    70.7%
 0.9   LRU (sim)         15.6%    34.2%    60.5%
 1.0   ideal top-C       42.9%    61.9%    81.0%
 1.0   LRU (sim)         29.0%    50.6%    73.7%
worked example: N = 1e6, s = 1, C = 1e4 -> ideal 68.0%
```

What to notice:

- Skew dominates: with 1% of keys cached, the ideal hit ratio is 23% at $s = 0.7$ but 62% at $s = 1.0$. Measure your skew before promising a hit ratio.
- Each tenfold increase in size adds a roughly constant number of points, the logarithmic growth Breslau et al. observed.
- LRU falls 8–13 points short of the ideal because it keeps whatever was used last, including one-off keys from the tail. §20.9 measures policies that close most of that gap.
- `warmup` excludes the first 200,000 requests, so the numbers describe a warm cache; a cold cache after a restart does much worse (§20.13).

20.4 CPU cache versus application cache

The principles are the same at every level of the memory hierarchy: locality, a hit/miss formula, eviction and coherence. The engineering is very different, as Table 20.1 shows.

Table 20.1 — CPU caches (Chapter 5) versus application caches (this chapter).

Aspect	CPU cache (L1–L3)	Application cache (Redis, Memcached, in-process)
Unit	64-byte cache line	Whole object or query result, bytes to megabytes
Hit latency	~1–15 ns	~50 ns (in-process) to ~0.3 ms (remote)
Managed by	Hardware, transparently	Your code: keys, TTLs, fills and invalidation are explicit
Placement and eviction	Set-associative, pseudo-LRU (§5.4.4)	Hash table plus LRU, LFU, W-TinyLFU or approximations
Coherence	Hardware protocol (MESI, §5.4.6) keeps copies consistent	None by default: you invalidate, expire or accept staleness
Failure	Never “down”	Can fail, restart empty or be partitioned from clients
What a miss costs	~100 ns DRAM access	2–50 ms database query, and load on a shared database

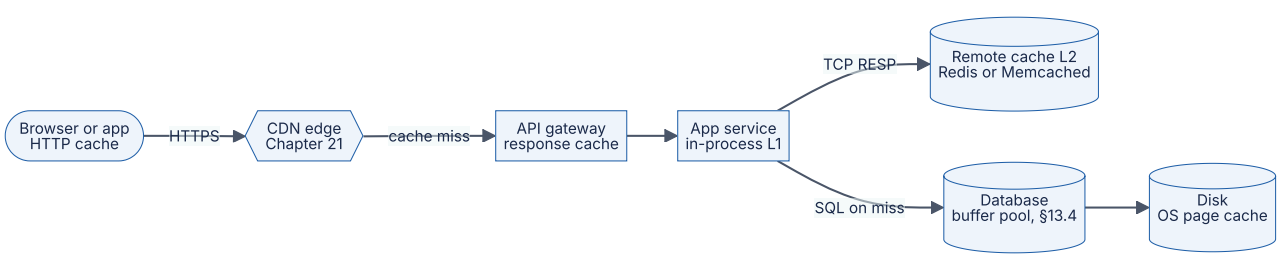
The most important row is coherence. A CPU guarantees that no core reads a stale line after another core’s write commits. An application cache gives you no such guarantee: you must design how copies are invalidated (§20.10), and you must decide how stale is acceptable for each piece of data. The second most important row is failure: a CPU cache never disappears, but a cache cluster can restart empty and expose the database to the full read load.

20.5 Where to cache

20.5.1 The layers

A request passes through several places where a copy can be kept. The closer to the client, the more latency and backend work a hit saves, but the less control you have over invalidation and the more copies there are to keep fresh.

Figure 20.2 — Cache layers from the client to the database.



Reading the diagram. From left to right the copies get more authoritative and more expensive to reach. The browser’s HTTP cache can answer with no network at all; the CDN

edge (an external system, Chapter 21) answers from a nearby point of presence; the gateway caches whole responses; the service's in-process L1 cache answers in microseconds; the remote L2 cache answers in a fraction of a millisecond over the Redis or Memcached protocol; the database's buffer pool and the operating system's page cache avoid disk reads. All arrows are synchronous request paths; each layer forwards only its misses. Storage boundaries: client device, CDN, application memory, cache cluster, database. Failure points: a copy in a layer you do not control (browser, CDN) cannot be deleted instantly; each extra layer adds a place where data can be stale. Scaling: each layer absorbs a share of traffic so the next sees only misses.

Table 20.2 — What each cache layer is good for.

Layer	Typical hit latency	Stores	Invalidation	Best for
Client (browser, mobile)	0 (no network)	HTTP responses, app state	Only by TTL or validation (`ETag`)	Static assets, per-user data that may be slightly old
CDN edge	5–30 ms from the user	Public HTTP responses	TTL, purge API (seconds, §21.6)	Images, video, scripts, public pages
API gateway	Sub-ms after reaching it	Whole API responses	TTL, explicit purge	Expensive public `GET`s
In-process (L1)	50 ns–1 µs	Objects in the heap	TTL; broadcast or none	Very hot, small, read-mostly data
Remote (L2)	0.2–0.5 ms in-datacenter	Serialized objects, counters, sessions	Delete on write, TTL, CDC	Shared hot data across many service instances
Database	Inside the query	Pages, query plans	Automatic, coherent	Everything; you tune its size (§13.4.3)

20.5.2 Client and HTTP caching

HTTP caching is standardized in RFC 9111 [RFC 9111]; §21.6.1 treats it in depth for CDNs. The origin controls client and intermediary caches with headers:

- `Cache-Control: max-age=600` permits reuse for 600 s; `s-maxage` sets the lifetime for shared caches; `private` keeps per-user responses out of shared caches; `no-store` forbids storage.
- **Validators** (`ETag`, `Last-Modified`): a stale copy is revalidated with `If-None-Match`, and `304 Not Modified` saves the body but not the round trip.
- `stale-while-revalidate` serves a stale copy while refreshing in the background; `stale-if-error` serves it when the origin fails [RFC 5861]. §20.11.3 applies the idea to server-side caches.

For static assets, the standard trick is **cache busting**: put a content hash in the file name (`app.3f9a2c.js`) and set a TTL of a year. The URL changes whenever the content changes,

so no invalidation is ever needed.

20.5.3 CDN and gateway

A **CDN** is a geographically distributed set of HTTP caches (Chapter 21). An **API gateway** or reverse proxy (NGINX, Envoy, Varnish) caches whole responses keyed by method, URL and selected headers. At both layers the risk is the **cache key**: if a response varies by a header (language, authorization) missing from the key, one user's response is served to another.

20.5.4 Application caches: in-process versus remote

An **in-process cache** (Caffeine in Java, an LRU dictionary in Python, `ristretto` in Go) lives in the service's own heap. A **remote cache** (Redis, Memcached) is a separate cluster shared by all instances.

Table 20.3 — In-process versus remote application caches.

Aspect	In-process (L1)	Remote (L2)
What it is	A map in the service's memory	A separate in-memory server cluster
How it works	Direct object reference; no serialization	Network round trip; serialize and deserialize
Advantages	~100 ns hits; no network; survives cache-cluster outage	Shared by all instances; one copy to invalidate; large capacity
Disadvantages	One copy per instance (N× memory, N× misses); hard to invalidate; lost on deploy	~0.3 ms per hit; network and serialization CPU; another system to operate
Performance	Millions of lookups/s per core	100,000+ simple ops/s per Redis node, more with I/O threads or Memcached threads
Operational complexity	Low, but GC pressure in managed runtimes	Medium: clustering, failover, memory tuning
Cost	Uses service RAM (multiplied by instance count)	Dedicated nodes
Use cases	Configuration, feature flags, hot keys (§20.15)	Sessions, profiles, rendered fragments, counters
Choose when	Data is small, very hot and tolerates seconds of staleness	Data is shared, larger, or must be invalidated reliably

Production systems often use both: an L1 with a TTL of seconds in front of an L2 with a TTL of minutes. The L1 absorbs hot keys; the L2 provides a single place to invalidate.

20.5.5 Database-side caching

The database already caches pages in its buffer pool (§13.4) and the operating system caches file blocks (§6.6.6). These caches are **coherent**: they never return stale data, because the database owns both the copy and the original. They cannot, however, avoid the cost of parsing, planning and executing a query, or of a network hop to the database. MySQL removed its query cache in version 8.0 because invalidating every cached result on every write to a table scaled poorly under concurrency [MySQL Server Team 2017]. Materialized views (§11.5.5) and read replicas (§14.3) are the database's own answers to repeated expensive reads.

RECOMMENDATION

Cache as close to the client as freshness allows. Static, versioned assets: browser and CDN, one-year TTL. Public pages: CDN with short TTL plus purge. Per-user API data: remote cache with delete-on-write. Very hot small objects: add an in-process L1 with a TTL of a few seconds. Never add a layer without stating how it is invalidated and what happens when it is empty.

20.6 Redis and Memcached internals

Redis and Memcached are the two dominant remote caches. Memcached is a deliberately minimal, multithreaded cache of opaque byte strings; Redis is a single-threaded data-structure server that also persists and replicates. §20.16 compares them.

20.6.1 Redis: a single-threaded event loop

DEFINITION

Definition — Event loop. A single thread that waits on many sockets at once with an operating-system readiness API (`epoll` on Linux, `kqueue` on BSD and macOS, §6.13.2–§6.13.4) and handles each ready socket in turn, without blocking on any one of them.

First-principles explanation

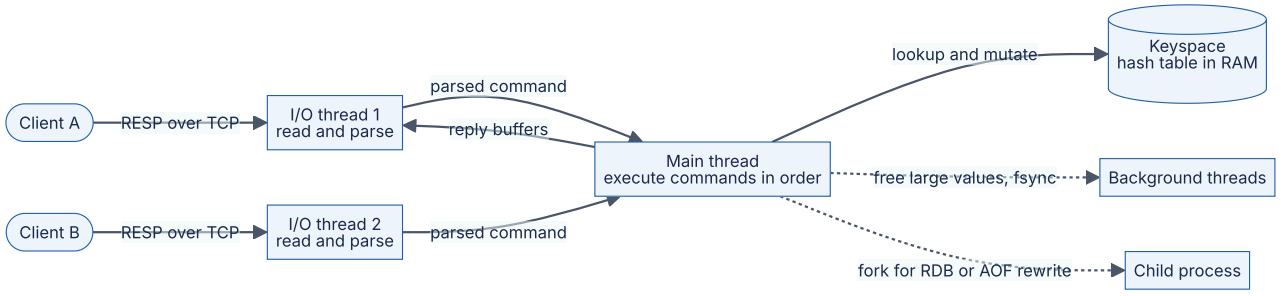
Picture one very fast chef who never waits: while one pot heats, he chops for the next order, and because only he touches the kitchen, nobody locks the knife drawer. Redis executes every command on one main thread, to completion, so every command (including a Lua script) is **atomic** without locks.

How it works internally

1. The main thread calls `epoll_wait` to learn which client sockets are readable.
2. For each, it reads bytes, parses commands in the **RESP** protocol and looks up the key in the **keyspace**, a hash table that grows by **incremental rehashing** (two tables; each operation moves a few buckets, so no single command pays for a full resize).
3. It executes the command against the in-memory data structure and appends the reply to the client's output buffer.
4. Before sleeping again it writes pending replies and runs timers: **active expiry**, which samples keys with TTLs and deletes expired ones, background rehashing and replication bookkeeping.

Background threads free large values (`UNLINK`) and `fsync` the append-only file. **I/O threads** (Redis 6.0 onward, `io-threads`) parse requests and write replies in parallel while execution stays single-threaded [Redis Docs: Redis configuration]. Valkey 8.0 reworked them to run concurrently with the main thread and reports about 1.2 million requests/s on one AWS r7g instance (as of 2024) [Valkey Blog 2024].

Figure 20.3 — Redis's event loop with I/O threads.



Reading the diagram. Clients send RESP commands over TCP. Optional I/O threads parse and write in parallel; the single main thread executes every command against the in-RAM keyspace. Side work runs asynchronously (dotted): background threads free memory and `fsync`, a forked child writes snapshots. Storage boundary: RAM for data, disk only for persistence files. Failure points: one slow command (`KEYS *`, a huge `DEL`, a long script) blocks *all* clients for tens of milliseconds, so use `SCAN`, `UNLINK` and `SLOWLOG`; forking a large process pauses the main thread. Scaling: execution uses one core, so scale out with Redis Cluster (§20.6.4).

20.6.2 Data structures and encodings

Redis values are typed. **Strings** (binary-safe, up to 512 MB) hold serialized objects and counters (`INCR`); **hashes** hold objects field by field; **lists** are queues; **sets** and **sorted sets** support membership and ranking (`ZADD`, `ZRANGE`), the latter as a skip list plus hash table (§17.7); **streams** are append-only logs; and HyperLogLog, bitmaps and geospatial

indexes are built on strings and sorted sets (§17.5, §17.9) [Redis Docs: Data types]. Small collections use a compact contiguous **listpack** encoding and are converted to hash tables or skip lists when they exceed thresholds such as `hash-max-listpack-entries` (128 by default), trading CPU for memory.

20.6.3 Persistence: RDB and AOF

A cold restart of a large cache is expensive (§20.13), and Redis also holds data that must survive restarts.

Table 20.4 — Redis persistence options.

Option	How it works	Data lost on crash	Cost
None	Pure in-memory	Everything	None
RDB snapshot	`fork()`; the child writes a compact binary snapshot while copy-on-write (§6.6.5) preserves the parent's view	Writes since the last snapshot (minutes)	Fork pause; up to 2× memory if many pages change during the dump
AOF, `appendfsync everysec`	Every write command appended to a log; `fsync` once per second; the log is periodically rewritten	About 1 s of writes	Disk bandwidth; rewrite cost
AOF, `appendfsync always`	`fsync` before acknowledging	Nothing acknowledged	Throughput limited by `fsync` latency
RDB + AOF (RDB preamble)	Rewritten AOF starts with an RDB image, then appends	About 1 s	Fast restart, small log

[Redis Docs: Persistence]. For a pure cache, many teams disable AOF and keep RDB only for faster warm restarts.

20.6.4 Replication and Redis Cluster

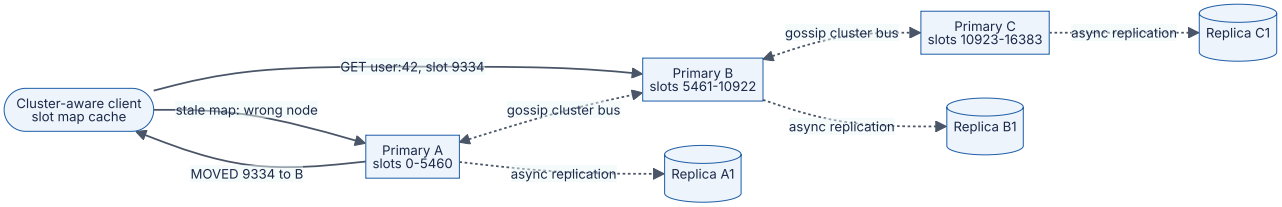
Replication is asynchronous: a primary streams its writes to replicas; a briefly disconnected replica resumes from a **replication backlog** (`PSYNC` with replication ID and offset), otherwise it receives a full snapshot. A failover can lose the last acknowledged writes; `WAIT` reduces but does not eliminate this [Redis Docs: Replication]. **Redis Sentinel** handles failover for unpartitioned deployments.

DEFINITION

Definition — Hash slot (Redis Cluster). One of 16,384 buckets into which Redis Cluster divides the keyspace: `slot = CRC16(key) mod 16384`. Each primary owns a set of slots. If a key contains `{...}`, only the substring inside the braces (a **hash tag**) is hashed, so related keys can be forced into one slot [Redis Docs: Cluster specification].

A node asked for a slot it does not own replies `MOVED <slot> <host:port>`; a slot under migration returns `ASK` for keys already moved. Nodes gossip over a **cluster bus** (client port + 10,000), and a majority of primaries authorizes a replica to take over a failed primary's slots. Multi-key commands work only within one slot.

Figure 20.4 — Redis Cluster: hash slots, redirects and replicas.



Reading the diagram. The client computes the slot and sends the command directly to the owning primary (slot 9334 on B). With a stale map it reaches the wrong node, which answers `MOVED`; the client updates its map and retries. Primaries replicate asynchronously (dotted) and gossip on the cluster bus. Storage boundaries: each primary holds only its slots. Failure points: failover loses unreplicated writes; the minority side of a network partition stops accepting writes. Scaling: add primaries and move slots, a fixed-partition scheme (§14.9.2) rather than a hash ring (§20.14).

20.6.5 Memcached: threads, slabs and segmented LRU

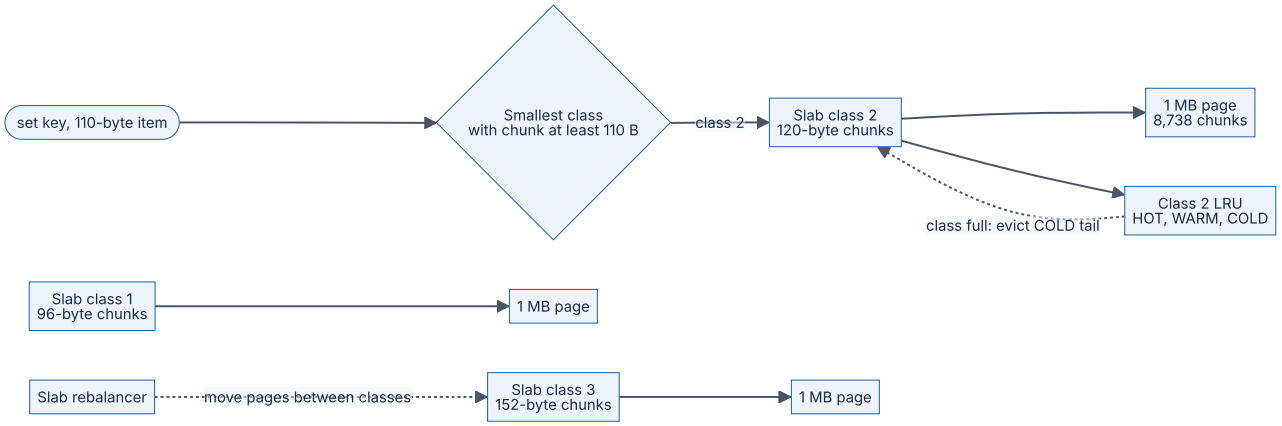
Memcached stores opaque values (1 MB maximum by default) with `get`, `set`, `delete`, `incr`, compare-and-set (`cas`) and a **meta protocol** whose flags can serve stale items while electing one client to refresh them [Memcached Docs: Meta commands]. It has no built-in persistence or replication; clients spread keys by consistent hashing (§20.14).

Threads. A listener thread hands connections to `-t` worker threads (4 by default), each with its own `libevent` loop, sharing one hash table under striped locks, so one process uses many cores [Memcached Docs: Overview].

The slab allocator. `malloc` and `free` on millions of variable-size objects fragment the heap. Memcached instead carves memory into 1 MB **pages**, assigns each to a **slab class** and cuts it into equal **chunks** whose sizes grow by the factor `-f` (1.25 by default): 96,

120, 152, 192, 240, 304 bytes and so on. An item (key, value and a header of about 50 bytes) takes the smallest chunk that fits.

Figure 20.5 — Memcached's slab allocator and per-class LRUs.



Reading the diagram. A 110-byte item goes to class 2; a 1 MB page holds $\lfloor 1,048,576/120 \rfloor = 8,738$ such chunks. Each class has its own LRU, so a full class evicts its coldest item even if other classes have free chunks. Background maintenance is dotted. Storage boundary: RAM (optional flash extension, extstore). Failure point: **slab calcification**, pages stuck in classes the current size mix no longer needs, which the slab rebalancer fixes by moving pages. Scaling: add memory or servers.

FORMULA

Slab internal fragmentation

$$c_{k+1} \approx f \cdot c_k, \qquad \text{waste}(x) = c_k - x \text{ for } c_{k-1} < x \le c_k, \qquad \frac{\text{waste}}{c_k} < 1 - \frac{1}{f}, \qquad \mathbb{E}[\frac{\text{waste}}{c_k}] \approx \frac{1}{2}(1 - \frac{1}{f})$$

Symbol	Meaning	Units
c_k	Chunk size of slab class k (rounded to 8 bytes)	bytes
f	Growth factor between classes (`-f`)	dimensionless
x	Item size, including header	bytes
waste	Unused bytes in the chunk	bytes

WORKED EXAMPLE

Worked example — Memory for 20 million session objects. Each item is a 40-byte key, a 300-byte value and a 50-byte header: $x = 390$ bytes.

1. Classes near 390 bytes: 304, then $304 \times 1.25 = 380$ (chunk 384), then $384 \times 1.25 = 480$. The item exceeds 384, so it uses a 480-byte chunk.
2. Waste per item: $480 - 390 = 90$ bytes, 18.8% of the chunk, close to the worst case $1 - 1/1.25 = 20\%$.
3. Memory: $20 \times 10^6 \times 480 = 9.6 \times 10^9$ bytes = 9.6 GB, versus 7.8 GB of item bytes. With mixed sizes, expected waste is about $\frac{1}{2}(1 - 0.8) = 10\%$.

Interpretation. Slab allocation trades 10–20% of memory for no external fragmentation and constant-time allocation. **Engineering implication.** If one item size dominates, check `stats slabs` and consider a smaller growth factor (for example `-f 1.08`) so a class lands just above your common size; budget memory by chunk size, not value size.

Segmented LRU. Since version 1.5.0, each class's LRU is split into **HOT**, **WARM** and **COLD** segments (plus **TEMP** for very short TTLs). New items enter HOT; items reaching a segment's tail move to COLD unless accessed, in which case they go to WARM; evictions take COLD's tail. A background **LRU maintainer** moves items, so a hit only sets a flag instead of relinking a locked list [Memcached Blog: Modern LRU]: InnoDB's scan-resistant midpoint idea (§13.4.2) applied to a cache.

INDUSTRY EXAMPLE

Netflix EVCache. Netflix built EVCache, a client library and tooling around Memcached, to cache data for its streaming services; writes are replicated to copies in several availability zones and regions by the client so a zone failure does not empty the cache, and it was open-sourced [Netflix: EVCache]. The design shows the Memcached philosophy: keep the server simple and put replication and routing in the client or a proxy. *Evidence: Primary* — [Netflix: EVCache]

20.7 Caching patterns: cache-aside, read-through, write-through, write-back, write-around

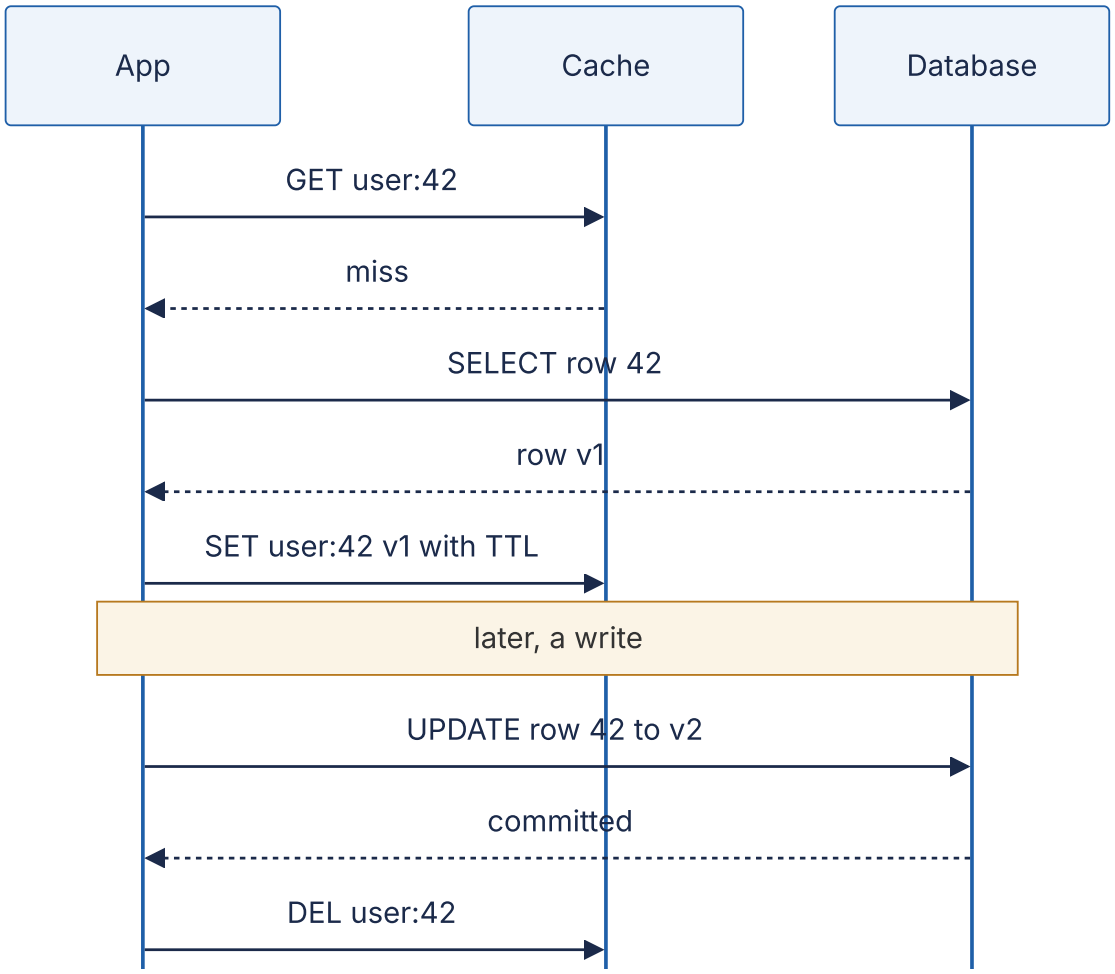
A caching pattern answers two questions: *who* fills the cache on a read miss (your code or the cache), and *what happens to the cache* on a write. Each answer moves the consistency risk somewhere else.

20.7.1 Cache-aside (look-aside)

DEFINITION

Definition — Cache-aside. The application reads the cache first; on a miss it reads the database and populates the cache itself. On a write it updates the database and then **deletes** (invalidates) the cache entry. The cache never talks to the database. Also called **look-aside** or **demand-filled** caching.

Figure 20.6 — Cache-aside read and write paths.



Reading the diagram. All calls are synchronous and made by the application. On a miss it reads the database and sets the value with a TTL; on a write it commits first, then deletes the key. Storage boundaries: cache and database, with no transaction spanning both. Failure points: a failed delete (the TTL bounds staleness) and a concurrent read re-inserting old data (§20.7.2). Scaling: memory follows the read working set.

Cache-aside is the default for remote caches because it is simple, keeps the cache optional (if the cache is down, reads go to the database) and caches only data that is actu-

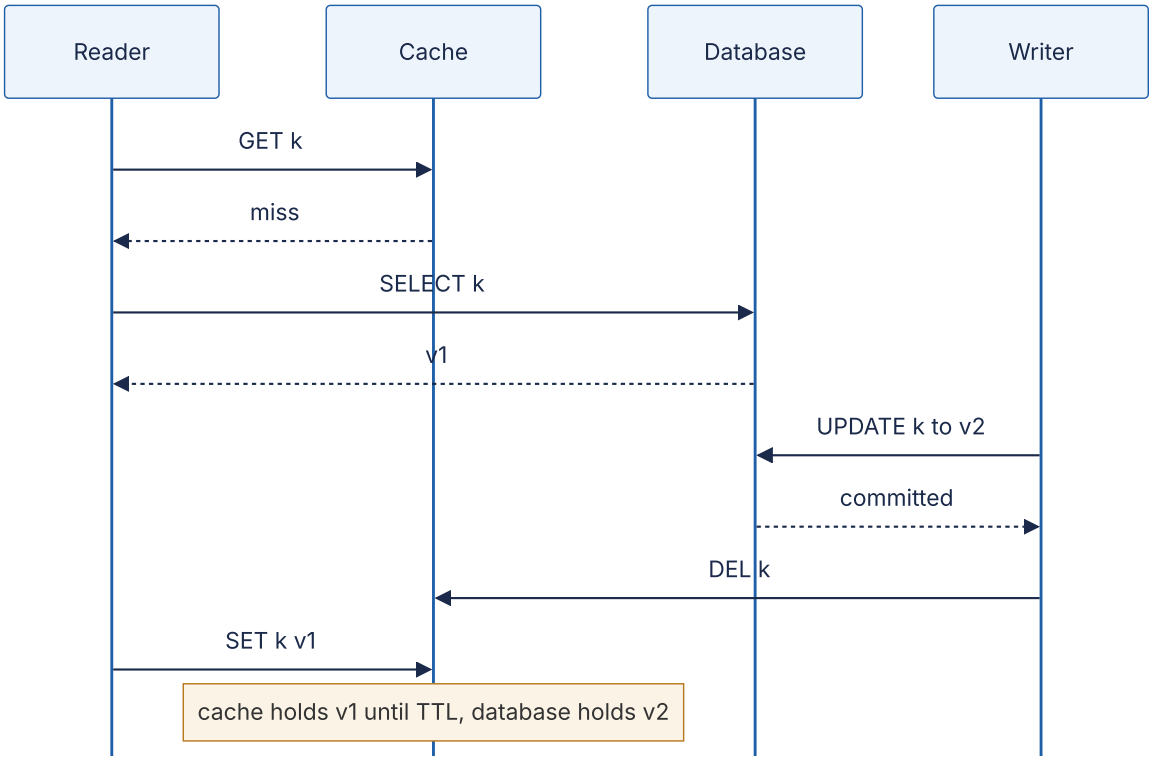
ally read. Facebook's Memcache deployment is the canonical large-scale example [Nishtala et al. 2013].

Why delete instead of update? If writers *set* the new value, two concurrent writers can commit to the database in one order and set the cache in the other, leaving the older value cached indefinitely. A delete is idempotent: whichever order deletes arrive in, the result is "absent", and the next read loads the committed value. Nishtala et al. chose deletes for exactly this reason [Nishtala et al. 2013].

20.7.2 Consistency races in cache-aside

Deleting after the write narrows the window for stale data but does not close it.

Figure 20.7 — The stale-set race in cache-aside.



Reading the diagram. Time flows downward. The reader misses and reads v_1 but is delayed (a GC pause, a slow network) before setting it. Meanwhile the writer commits v_2 and deletes the key, which is already absent. The reader's late `SET` then installs v_1, and every subsequent reader sees stale data until the TTL expires. All steps are synchronous; the hazard is the gap between the reader's database read and its cache write. The race is rare, but at millions of reads per second rare events happen every day.

The opposite ordering is worse. If the writer deletes the cache *before* updating the database, any reader arriving between the delete and the commit refills the cache with the old value, a window as long as the write transaction.

Mitigations, from simplest to strongest:

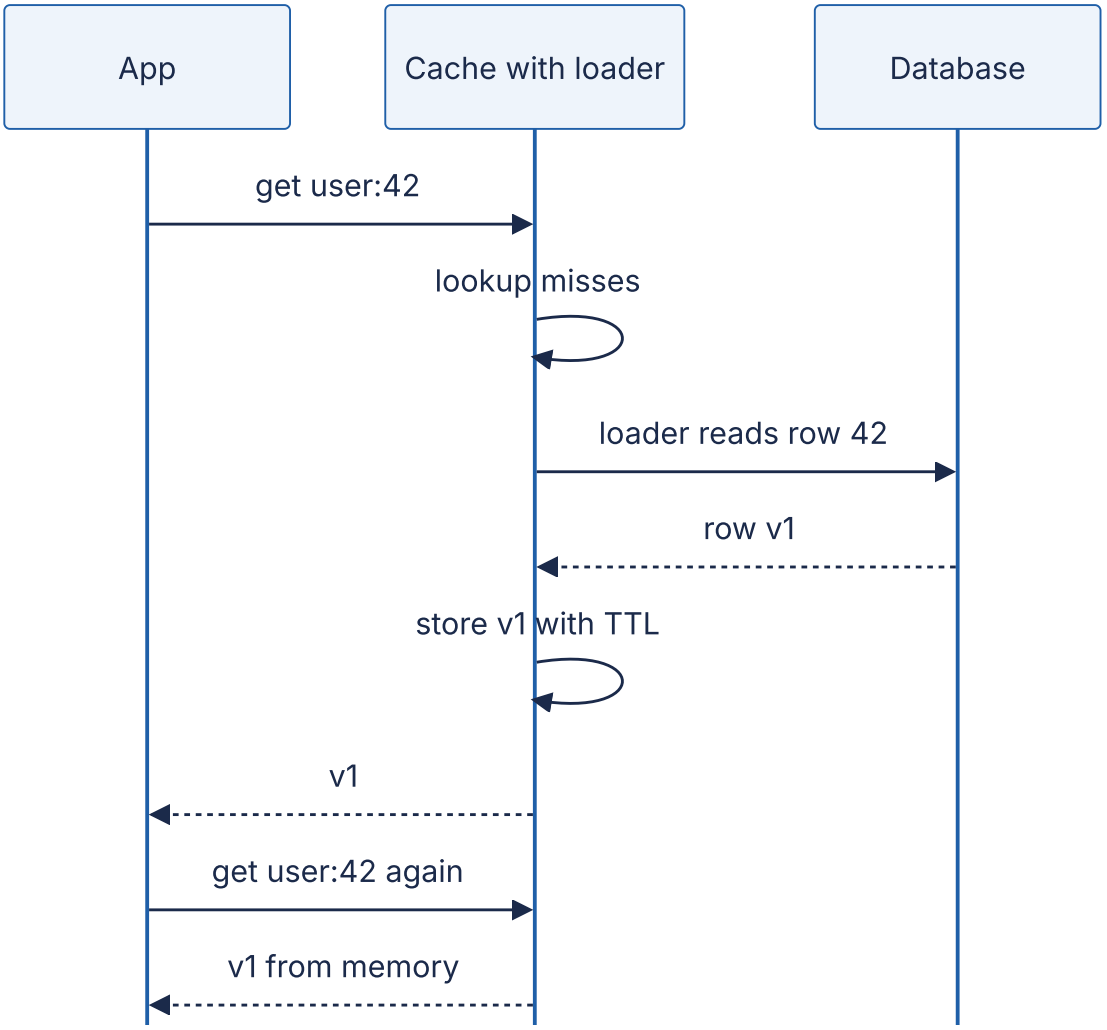
1. **Short TTLs** bound how long a stale entry can survive. Always set one.
2. **Delayed double delete**: delete, commit, then delete again after a delay longer than a typical read–fill (for example 500 ms). It shrinks the window but is a heuristic.
3. **Leases**: on a miss the cache hands the reader a token; a delete invalidates outstanding tokens, and a `SET` with an invalid token is rejected. In Figure 20.7 the writer's `DEL` would void the reader's lease, so the stale `SET` fails [Nishtala et al. 2013]. Memcached's `cas` and meta-protocol tokens provide similar building blocks.
4. **Versioned sets**: store a version (a database row version or commit LSN) with the value and set only if the cached version is older, using compare-and-set or a server-side script.
5. **Invalidation from the database's change stream** (§20.10.2): deletes are issued after commit by a process that tails the log, so no application crash can skip them.

20.7.3 Read-through

DEFINITION

Definition — Read-through. The application asks only the cache; on a miss the *cache* loads the value from the database through a configured loader, stores it and returns it.

Figure 20.8 — Read-through.



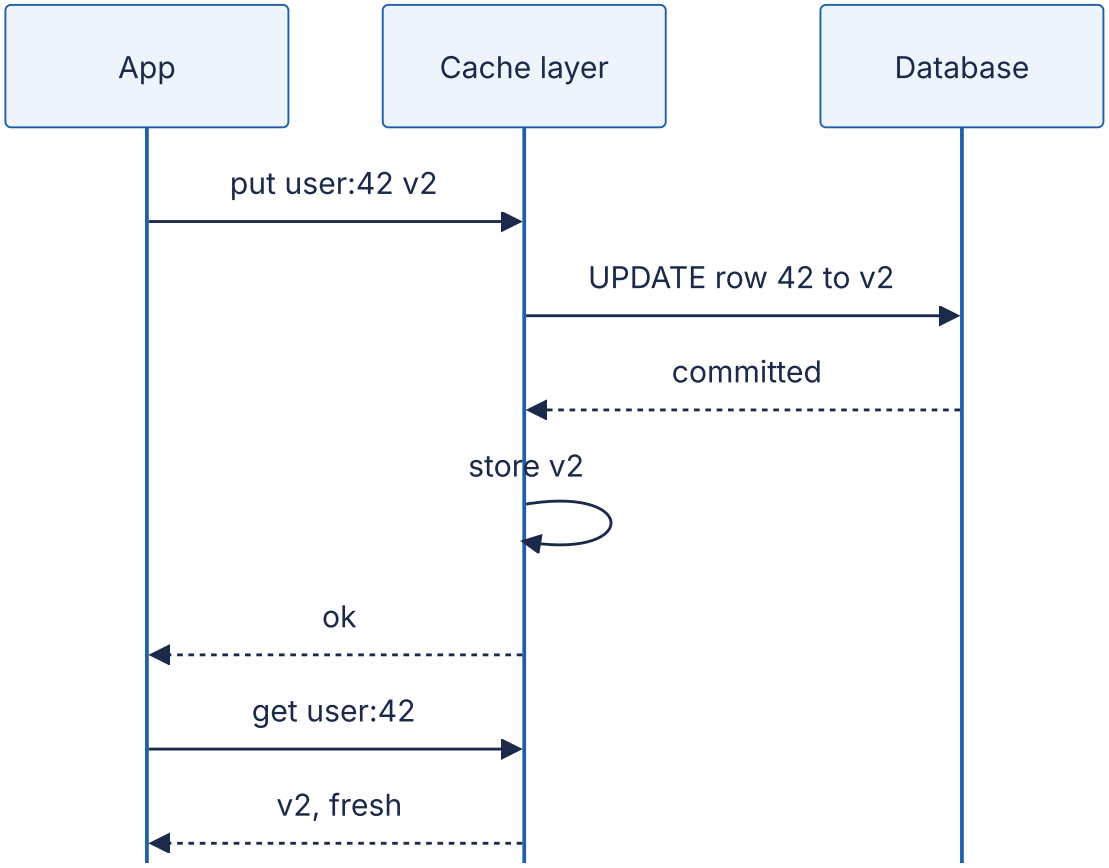
Reading the diagram. The application sees one synchronous interface; the cache loads. Because the loader sits inside the cache, concurrent misses for a key can be **coalesced** into one database read (§20.11). Storage boundaries are as in cache-aside, but a cache outage also blocks reads unless the client falls back. Caffeine's `LoadingCache` is an in-process example; remote read-through usually comes from a proxy or loader plug-in.

20.7.4 Write-through

DEFINITION

Definition — Write-through. Every write goes to the cache, which synchronously writes it to the database before acknowledging; the cache is updated and the database is updated as one client-visible operation.

Figure 20.9 — Write-through.

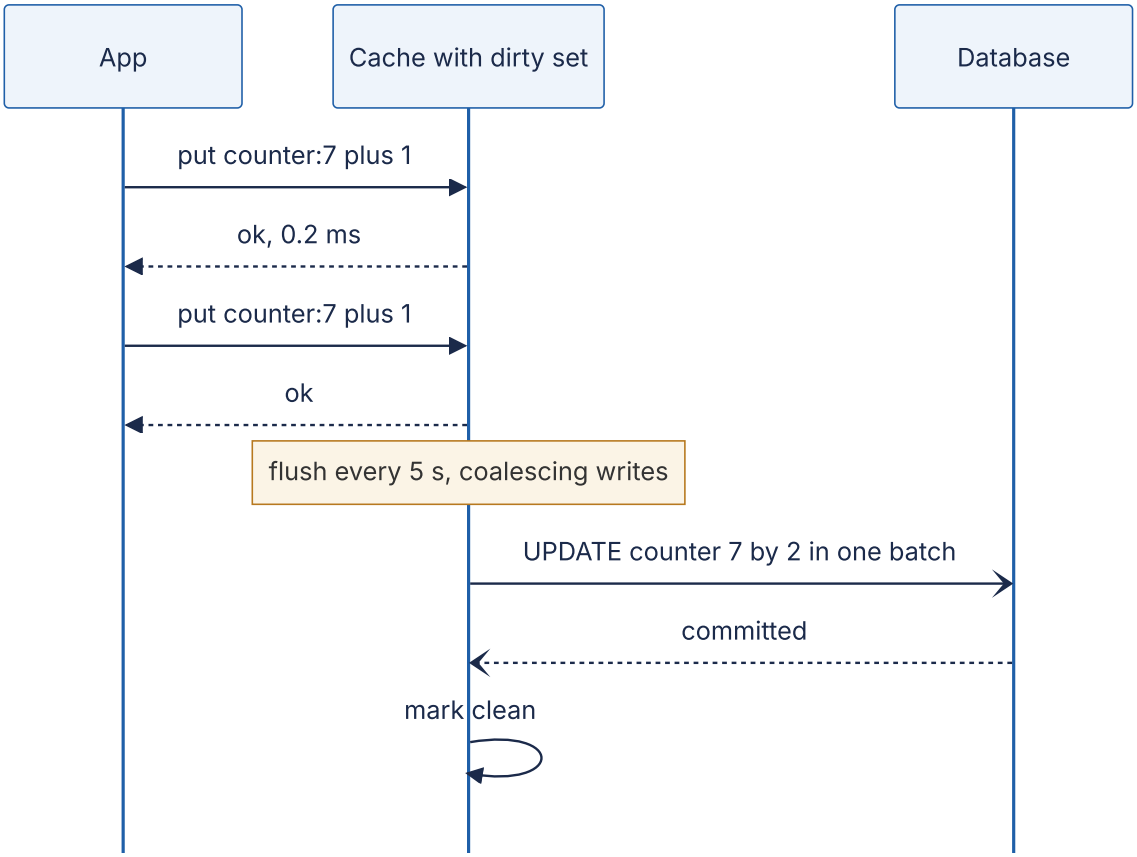


Reading the diagram. The write is acknowledged only after both the database commit and the cache update, so a subsequent read through the same cache sees the new value. Write latency is the sum of both, and every written key is cached, even if never read. Failure points: a crash between the database commit and the cache store leaves the cache without the value (harmless) or, if ordered the other way, with a value the database never committed (harmful), so the database must be written first. Writes that bypass the cache layer (an admin script, another service) still make it stale.

20.7.5 Write-back (write-behind)

> DEFINITION
>
> **Definition — Write-back.** Writes update only the cache and are acknowledged immediately; the cache marks entries **dirty** and flushes them to the database asynchronously, in batches or after a delay.

Figure 20.10 — Write-back.



Reading the diagram. Writes are acknowledged from memory; the flush to the database is asynchronous (open arrowheads) and coalesces many writes to the same key into one. This is how CPU caches (§5.4) and database buffer pools (§13.4) handle writes. The storage boundary that matters is the dirty set: until flushed, acknowledged data exists only in cache memory. Failure point: a cache node crash loses every unflushed write unless the cache itself is replicated or logged. Scaling: write-back absorbs write bursts that the database could not take directly.

FORMULA

Data at risk under write-back

$$W_{\text{risk}} \approx r_w \times T_{\text{flush}}, \qquad \text{write reduction} = \frac{r_w}{r_{\text{flush}}}$$

Symbol	Meaning	Units
W_{risk}	Acknowledged writes that a crash can lose	writes
r_w	Write rate into the cache	writes/s
T_{flush}	Maximum time a dirty entry waits before flushing	s
r_{flush}	Database writes per second after coalescing	writes/s

WORKED EXAMPLE

Worked example — View counters. A video service counts 200,000 views/s spread over 50,000 videos active in any 5-second window, flushing every $T_{\text{flush}} = 5$ s.

1. Writes at risk: $200,000 \times 5 = 10^6$ view increments lost if a node crashes just before a flush.
2. Database load after coalescing: at most 50,000 rows updated per 5 s, so $r_{\text{flush}} = 10,000$ writes/s.
3. Write reduction: $200,000/10,000 = 20\times$.

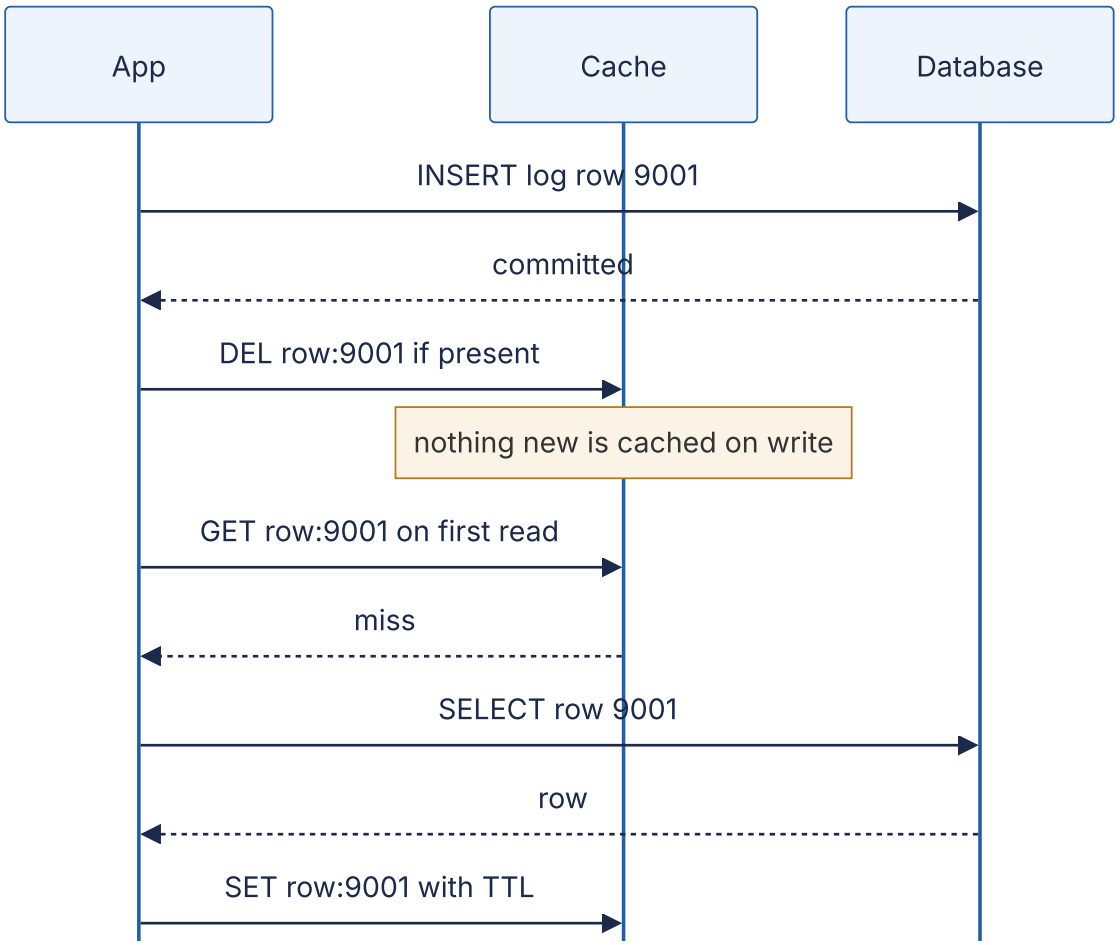
Interpretation. For view counts, losing a few seconds of increments is acceptable, and a 20× write reduction is decisive. **Engineering implication.** Use write-back only for data whose loss you can tolerate or reconstruct (counters, metrics, presence); for orders and payments, the database must commit first.

20.7.6 Write-around

DEFINITION

Definition — Write-around. Writes go directly to the database and bypass the cache; the cache is populated only by reads (usually cache-aside), and the written key is invalidated or left to expire.

Figure 20.11 — Write-around.



Reading the diagram. Writes touch only the database (and delete a stale key); only a later read caches the row, so write-once data such as logs and audit rows never evicts hot entries. The cost: the first read after a write misses. Cache-aside with delete-on-write *is* write-around; the name matters in contrast with write-through.

20.7.7 Choosing a pattern

Table 20.5 — Caching patterns compared.

Pattern	Read miss filled by	Write path	Consistency risk	Write latency	Choose when
Cache-aside	Application	DB, then delete key	Stale-set race; missed deletes	DB only	Default for remote caches
Read-through	Cache loader	(combine with a write pattern)	As cache-aside	—	In-process caches; coalescing misses
Write-through	Cache loader	Cache writes DB synchronously	Writes bypassing the layer	DB + cache	Read-after-write freshness via one layer
Write-back	Cache loader	Cache only; async flush	Loss of unflushed writes	Cache only	Counters, bursts, tolerable loss
Write-around	Application	DB only	First read misses	DB only	Write-heavy, rarely re-read data

COMMON MISTAKE

"Write-through means the cache and database are always consistent." Only for writers that go through that cache layer, and only if the database is written first. Concurrent writers, a second cache tier, or a batch job writing the database directly all reintroduce staleness. No pattern gives you linearizability across two stores without a protocol (Chapter 25); pick the pattern whose failure mode you can tolerate.

20.8 Time to live (TTL)

DEFINITION

Definition — Time to live (TTL). The maximum time an entry may be served after it was stored. After the TTL the entry is treated as absent (or, with stale-while-revalidate, as stale). Redis removes expired keys lazily when they are accessed and actively by periodically sampling keys that have a TTL [Redis Docs: EXPIRE].

A TTL is your **staleness budget** and your safety net: even if every invalidation mechanism fails, no entry is wrong for longer than its TTL. Choose it per data type from the question "how wrong may a user see this, and for how long?": seconds for prices and inventory, minutes for profiles, hours for rendered public pages, a year for content-addressed assets. Shorter TTLs cost more misses: an entry read λ times per second with TTL T misses about once per T seconds, a miss ratio near $1/(\lambda T)$ for hot keys.

The danger is synchronized expiry. If a deploy, a batch job or a cache restart writes a million keys in one minute with the same TTL, they all expire in the same minute an hour

later, and the database receives a million refills at once. **Jitter** spreads them out.

FORMULA

TTL with uniform jitter

$$T_i = T_{\text{base}} + U_i, \quad U_i \sim \text{Uniform}(0, J), \qquad \text{peak refills/s} \approx \frac{K}{\max(J, t_{\text{write}})}$$

Symbol	Meaning	Units
T_i	TTL given to key i	s
T_{base}	Base TTL	s
J	Jitter range	s
K	Keys written together	keys
t_{write}	Time span over which the keys were originally written	s

WORKED EXAMPLE

Worked example — Choosing a jitter range. A nightly job writes $K = 1,200,000$ product keys in 60 s with $T_{\text{base}} = 3,600$ s. The database can absorb 2,000 extra refill queries/s.

1. Without jitter: $1,200,000/60 = 20,000$ refills/s for a minute, an hour later. Ten times the safe level.
2. Required spread: $J \geq 1,200,000/2,000 = 600$ s.
3. Use $T_i = 3,600 + \text{Uniform}(0, 600)$ s. Peak refill rate: $1,200,000/600 = 2,000$ queries/s.
4. The cost: the longest-lived entries are now 4,200 s old, so the staleness bound grows from 60 to 70 minutes.

Interpretation. Jitter converts a spike into a plateau whose height you choose. **Engineering implication.** Add jitter of 10–20% to every TTL by default; for the very hottest keys combine it with the stampede defenses of §20.11, because jitter spreads *different* keys but cannot help one key that everyone reads.

20.9 Eviction: LRU, LFU, ARC and TinyLFU

When memory is full, the cache must choose a **victim**. The ideal is Belady's MIN: evict the entry whose next use is furthest in the future [Belady 1966]. It needs knowledge of the future, so real policies predict the future from the past using **recency** (used lately?),

frequency (used often?) or both. §13.4.2 covered LRU, clock and LRU-K inside databases; here we focus on policies used by application caches.

20.9.1 LRU and LFU

LRU (least recently used) keeps entries in a list ordered by last access, moving an entry to the head on each hit and evicting from the tail; a hash map plus a doubly linked list gives $O(1)$ operations. It adapts instantly to change but is polluted by **scans** and one-hit wonders: a burst of keys read once pushes out keys that are read every second.

LFU (least frequently used) evicts the entry with the smallest access count. It protects consistently popular keys from scans but has two flaws: it must store counts, and old counts never fade, so yesterday's hits keep today's useless entries in the cache (**cache pollution by history**). Practical LFUs age counts by halving or decay.

20.9.2 ARC

ARC (Adaptive Replacement Cache) keeps two LRU lists of cached entries, T_1 for keys seen once recently and T_2 for keys seen at least twice, plus two **ghost lists** B_1 and B_2 that remember only the *keys* recently evicted from each [Megiddo & Modha 2003]. A hit in ghost list B_1 means "T_1 was too small" and shifts the target size p toward recency; a hit in B_2 shifts it toward frequency. ARC therefore tunes itself between LRU and LFU with no parameters, resists scans, and costs $O(1)$ per access with metadata for $2C$ keys. OpenZFS's ARC is a modified version of it [OpenZFS Docs]. IBM held a patent on ARC, which kept it out of some open-source projects.

20.9.3 TinyLFU and W-TinyLFU

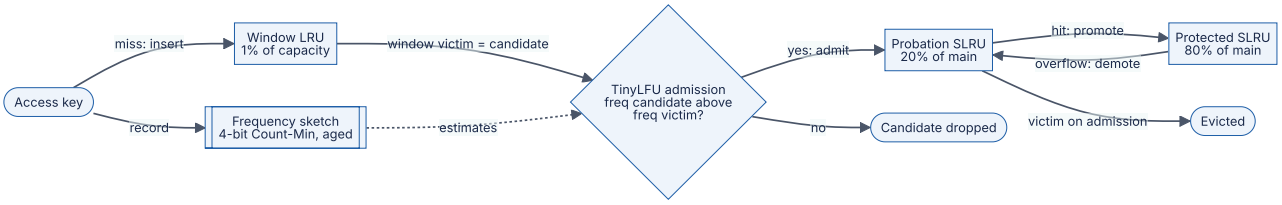
DEFINITION

Definition — TinyLFU. An **admission policy**: when the cache is full and a new candidate arrives, admit it only if its estimated access frequency exceeds that of the victim the eviction policy would remove. Frequencies are estimated by a small approximate sketch (a Count-Min sketch with 4-bit counters, §17.6) over a sample of recent requests, and **aged** by halving every counter after a fixed number of requests [Einziger, Friedman & Manes 2017].

Admission solves LRU's problem from the other side: instead of choosing a better victim, it refuses to let a one-hit wonder in at all. Pure TinyLFU, however, rejects new keys that are about to become popular, because they have no history yet. **W-TinyLFU** adds a small **window** LRU (1% of capacity by default) that admits everything; entries leaving the window compete through TinyLFU for a place in the **main** area, a segmented LRU with a probation and a protected segment. A **doorkeeper** Bloom filter can absorb first occur-

rences so they do not take sketch counters. Caffeine, the standard Java cache library, uses W-TinyLFU and adapts the window size with a hill-climbing algorithm [Caffeine Wiki: Efficiency].

Figure 20.12 — W-TinyLFU: window, admission filter and segmented main cache.



Reading the diagram. Every access updates the frequency sketch. A missed key first enters the small window LRU, which gives new keys a chance to prove themselves. When the window overflows, its LRU entry becomes a candidate and is compared with the probation segment's LRU victim; the one with the higher estimated frequency stays. Hits in probation promote entries to the protected segment, whose overflow is demoted back to probation. All of this is in-process memory and synchronous; the sketch is shared state read by the admission decision (dotted). Failure points: a stale sketch (fixed by periodic halving) and hash-flooding attacks on admission (Caffeine adds a little randomness). Scaling: sketch memory is about 8 bytes per cache entry.

20.9.4 Redis's approximated LRU and LFU

Redis does not keep a global LRU list, which would cost two pointers per key and a list update on every read. Each object header instead stores a 24-bit clock of its last access. When memory exceeds `maxmemory`, Redis **samples** `maxmemory-samples` keys (5 by default), adds the best candidates to a small pool of 16, and evicts the one idle longest; 10 samples come very close to true LRU [Redis Docs: Key eviction]. In LFU mode the same 24 bits hold a 16-bit decay timestamp and an 8-bit **logarithmic counter** (a Morris counter), incremented with probability that falls as the counter grows (`lfu-log-factor`, default 10) and decremented as time passes (`lfu-decay-time`, default 1 minute); new keys start at 5 so they are not evicted immediately. The policy is chosen with `maxmemory-policy`: `noeviction` (the default, which rejects writes when full), `allkeys-lru`, `allkeys-lfu`, `allkeys-random`, and `volatile-*` variants that consider only keys with a TTL, plus `volatile-ttl`.

> WARNING / TRADE-OFF
>
> **A Redis used as a cache must not run with `noeviction`.** With the default policy a full Redis rejects writes with an out-of-memory error, so cache fills fail and every request becomes a miss. Set `maxmemory` below physical RAM (leave room for fork copy-on-write and fragmentation) and choose `allkeys-lru` or `allkeys-lfu`.

20.9.5 Measuring policies on a Zipf trace

The program implements LRU, LFU (with least-recently-used tie-breaking), ARC from the paper’s pseudocode, and a simplified W-TinyLFU (no doorkeeper or hill climbing), and runs them on three traces of 400,000 requests over 50,000 keys: stationary Zipf ($s = 0.9$); the same Zipf interrupted every 8,000 requests by a scan of 2,000 never-repeated keys; and Zipf whose popularity ranks are reshuffled four times.

```
        """Hit ratios of LRU, LFU, ARC and a simplified W-TinyLFU on three synthetic
traces."""
        import itertools
        import random
        from collections import OrderedDict, defaultdict

        class LRU:
            def __init__(self, capacity):
                self.c, self.d = capacity, OrderedDict()
            def access(self, key):                      # returns True on a hit
                if key in self.d:
                    self.d.move_to_end(key)
                    return True
                self.d[key] = None
                if len(self.d) > self.c:
                    self.d.popitem(last=False)
                return False

        class LFU:
            """O(1) in-cache LFU: evict the lowest count, least recent among ties."""
            def __init__(self, capacity):
                self.c, self.freq = capacity, {}
                self.buckets, self.min_f = defaultdict(OrderedDict), 0
            def access(self, key):
                if key in self.freq:
                    f = self.freq[key]
                    del self.buckets[f][key]
                    if not self.buckets[f] and self.min_f == f:
                        self.min_f = f + 1
                    self.freq[key] = f + 1
                    self.buckets[f + 1][key] = None
                    return True
                if len(self.freq) >= self.c:
                    victim, _ = self.buckets[self.min_f].popitem(last=False)
                    del self.freq[victim]
                self.freq[key], self.min_f = 1, 1
                self.buckets[1][key] = None
                return False

        class ARC:
            """Adaptive Replacement Cache [Megiddo & Modha 2003]: T1/T2 hold data,
            B1/B2 are ghost lists of recently evicted keys, p is T1's target size."""
            def __init__(self, capacity):
                self.c, self.p = capacity, 0
                self.t1, self.t2, self.b1, self.b2 = (OrderedDict() for _ in range(4))
            def _replace(self, in_b2):
                if self.t1 and (not self.t2 or len(self.t1) > self.p
                                or (in_b2 and len(self.t1) == self.p)):
                    k, _ = self.t1.popitem(last=False)
                    self.b1[k] = None
                else:
                    k, _ = self.t2.popitem(last=False)
                    self.b2[k] = None
            def access(self, x):
                if x in self.t1:                        # second hit: recency -> fre-
quency
```

```
                    del self.t1[x]
                    self.t2[x] = None
                    return True
                if x in self.t2:
                    self.t2.move_to_end(x)
                    return True
                if x in self.b1:                          # T1 was too small: grow p
                    self.p = min(self.c, self.p + max(len(self.b2) // len(self.b1), 1))
                    self._replace(False)
                    del self.b1[x]
                    self.t2[x] = None
                    return False
                if x in self.b2:                          # T2 was too small: shrink p
                    self.p = max(0, self.p - max(len(self.b1) // len(self.b2), 1))
                    self._replace(True)
                    del self.b2[x]
                    self.t2[x] = None
                    return False
                l1 = len(self.t1) + len(self.b1)
                if l1 == self.c:
                    if len(self.t1) < self.c:
                        self.b1.popitem(last=False)
                        self._replace(False)
                    else:
                        self.t1.popitem(last=False)
                else:
                    total = l1 + len(self.t2) + len(self.b2)
                    if total >= self.c:
                        if total == 2 * self.c:
                            self.b2.popitem(last=False)
                        self._replace(False)
                self.t1[x] = None
                return False

        SEEDS = (0x9E3779B97F4A7C15, 0xC2B2AE3D27D4EB4F, 0x165667B19E3779F9,
0xD6E8FEB86659FD93)

        class CountMin4:
            """Count-Min sketch (4 rows) with 4-bit counters (cap 15) and periodic halv-
ing."""
            def __init__(self, capacity):
                self.bits = max(4, (capacity * 4 - 1).bit_length())   # width >= 4 x ca-
pacity
                self.rows = [[0] * (1 << self.bits) for _ in SEEDS]
                self.samples, self.reset_at = 0, 10 * capacity
            def _cols(self, key):                         # multiplicative hashing of int
keys
                return ((((key + 1) * s) & 0xFFFFFFFFFFFFFFFF) >> (64 - self.bits) for s
in SEEDS]
            def add(self, key):
                for row, col in zip(self.rows, self._cols(key)):
                    if row[col] < 15:
                        row[col] += 1
                self.samples += 1
                if self.samples >= self.reset_at:         # aging: halve every counter
                    self.rows = [[v >> 1 for v in row] for row in self.rows]
                    self.samples //= 2
```

```
            def estimate(self, key):
                return min(row[col] for row, col in zip(self.rows, self._cols(key)))

        class WTinyLFU:
            """1% window LRU, then a TinyLFU admission filter guarding a segmented LRU
            main area (20% probation, 80% protected), as in Caffeine (simplified)."""
            def __init__(self, capacity):
                self.wcap = max(1, capacity // 100)
                main = capacity - self.wcap
                self.prot_cap = int(main * 0.8)
                self.main_cap = main
                self.window, self.prob, self.prot = OrderedDict(), OrderedDict(),
OrderedDict()
                self.sketch = CountMin4(capacity)
            def access(self, key):
                self.sketch.add(key)
                if key in self.window:
                    self.window.move_to_end(key)
                    return True
                if key in self.prot:
                    self.prot.move_to_end(key)
                    return True
                if key in self.prob:                        # promote on second hit
                    del self.prob[key]
                    self.prot[key] = None
                    if len(self.prot) > self.prot_cap:
                        k, _ = self.prot.popitem(last=False)
                        self.prob[k] = None                 # demote to probation MRU
                    return True
                self.window[key] = None
                if len(self.window) > self.wcap:
                    cand, _ = self.window.popitem(last=False)
                    if len(self.prob) + len(self.prot) < self.main_cap:
                        self.prob[cand] = None
                    elif self.prob or self.prot:
                        seg = self.prob if self.prob else self.prot
                        victim = next(iter(seg))
                        if self.sketch.estimate(cand) > self.sketch.estimate(victim):
                            del seg[victim]                 # admit candidate, evict victim
                            self.prob[cand] = None          # else: candidate is dropped
                return False

        def zipf_cum(n, s):
            return list(itertools.accumulate(1.0 / i ** s for i in range(1, n + 1)))

        def stationary(n, length, rng, s=0.9):
            return rng.choices(range(n), cum_weights=zipf_cum(n, s), k=length)

        def with_scans(n, length, rng, s=0.9, burst=8_000, scan=2_000):
            """Zipf bursts interleaved with scans of keys that are never seen again."""
            cum, out, fresh = zipf_cum(n, s), [], n
            while len(out) < length:
                out += rng.choices(range(n), cum_weights=cum, k=burst)
                out += range(fresh, fresh + scan)
                fresh += scan
            return out[:length]
```

```
        def shifting(n, length, rng, s=0.9, phases=4):
            """Popularity ranks are reshuffled at the start of every phase."""
            cum, out = zipf_cum(n, s), []
            for _ in range(phases):
                perm = list(range(n))
                rng.shuffle(perm)
                out += [perm[r] for r in rng.choices(range(n), cum_weights=cum,
                                                     k=length // phases)]
            return out

        def hit_ratio(policy, trace, warmup):
            hits = 0
            for t, key in enumerate(trace):
                if policy.access(key) and t >= warmup:
                    hits += 1
            return hits / (len(trace) - warmup)

        if __name__ == "__main__":
            N, LENGTH = 50_000, 400_000
            rng = random.Random(7)
            traces = {"stationary Zipf": stationary(N, LENGTH, rng),
                      "Zipf + scans": with_scans(N, LENGTH, rng),
                      "shifting Zipf": shifting(N, LENGTH, rng)}
            policies = [("LRU", LRU), ("LFU", LFU), ("ARC", ARC), ("W-TinyLFU",
WTinyLFU)]
            for cap in (500, 2_500):
                print(f"cache = {cap:,} entries ({cap / N:.0%} of {N:,} keys)")
                print(f"  {'trace':<17}" + "".join(f"{name:>11}" for name, _ in poli-
cies))
                for tname, trace in traces.items():
                    row = [hit_ratio(cls(cap), trace, LENGTH // 10) for _, cls in poli-
cies]
                    print(f"  {tname:<17}" + "".join(f"{h:>11.1%}" for h in row))
```

Output (about 13 s):

```
cache = 500 entries (1% of 50,000 keys)
  trace                    LRU        LFU        ARC  W-TinyLFU
  stationary Zipf        32.3%      42.5%      43.0%      43.5%
  Zipf + scans           25.2%      33.8%      34.3%      34.6%
  shifting Zipf          32.1%      14.8%      42.1%      41.9%
cache = 2,500 entries (5% of 50,000 keys)
  trace                    LRU        LFU        ARC  W-TinyLFU
  stationary Zipf        50.1%      58.3%      58.3%      59.4%
  Zipf + scans           36.4%      46.2%      46.9%      47.1%
  shifting Zipf          49.7%      33.3%      55.7%      54.2%
```

What to notice:

- On the stationary trace the ideal top-C hit ratios are 45.8% (1%) and 62.0% (5%). W-TinyLFU gets within 2.5 points of ideal; LRU is 12–13 points behind.
- Scans hurt LRU most (−7 to −14 points); in this trace 20% of requests are unrepeatable scan keys, so 80% of the ideal, about 36.6% at 1%, is the ceiling, and ARC

and W-TinyLFU reach about 34%.

- In-cache LFU collapses on the shifting trace (14.8%) because entries that were hot in an earlier phase keep their high counts. Aging in W-TinyLFU and adaptation in ARC avoid this.
- The sketch's hash matters. An earlier version of this program that indexed rows with Python's tuple `hash((row, key))` produced correlated rows and lost 2–7 points; a multiplicative hash per row fixed it (§17.3).
- These are synthetic traces; Caffeine's comparisons on real traces mostly show W-TinyLFU at or near the top [Caffeine Wiki: Efficiency], but replay your own trace before switching.

Table 20.6 — Eviction and admission policies.

Policy	Signal	Scan resistant	Adapts to change	Metadata	Used in
LRU	Recency	No	Instantly	List pointers per entry	Defaults in many libraries
Sampled LRU	Approximate recency	No	Instantly	24-bit clock per key	Redis `allkeys-lru`
LFU with decay	Frequency	Yes	Slowly	Counter per entry	Redis `allkeys-lfu`
Segmented LRU	Recency with probation	Partly	Quickly	Segment flag	Memcached (HOT/WARM/COLD), InnoDB midpoint
ARC	Recency and frequency, self-tuning	Yes	Yes	Ghost keys for $2C$ entries	OpenZFS (modified)
W-TinyLFU	Frequency-based admission plus recency	Yes	Yes (aging, adaptive window)	~8 bytes per entry sketch	Caffeine

20.10 Cache invalidation

Invalidation keeps copies from outliving the truth. There are four mechanisms, and serious systems combine them.

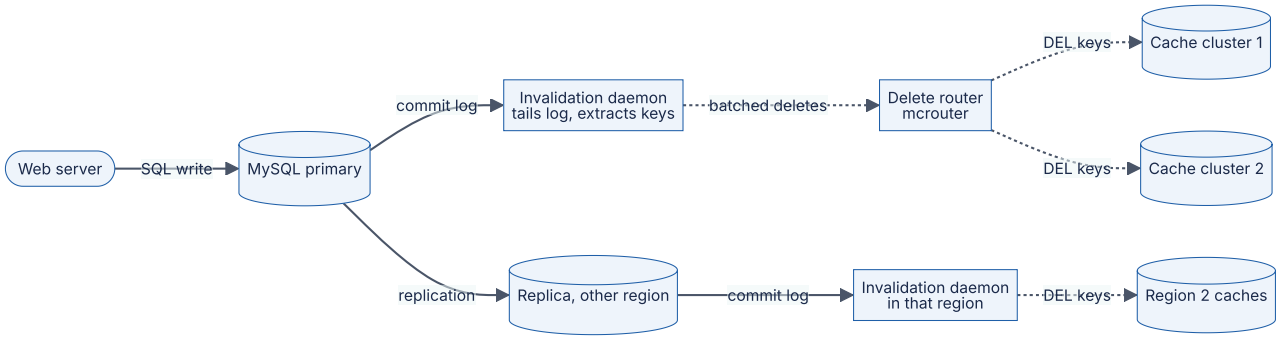
20.10.1 TTL and explicit deletes

TTL-only invalidation is simple and self-healing but guarantees staleness up to the TTL. **Explicit deletes** from the code path that writes (cache-aside, §20.7.1) make data fresh within milliseconds, but every write path must remember to delete every affected key: the profile, the list it appears in, the rendered page fragment. A missed path is a staleness bug that the TTL eventually hides, which is why you always keep a TTL as a backstop.

20.10.2 Event-driven invalidation from the change stream

Instead of trusting every writer, derive deletes from the database's own log. A **change data capture** (CDC) process tails the commit log (MySQL binlog, PostgreSQL logical decoding, §32.6), maps each changed row to cache keys and issues deletes. Because the log contains only committed changes, in commit order, nothing is invalidated too early and no application crash can skip an invalidation.

Figure 20.13 — CDC-driven invalidation across regions (after Facebook's mcsqueal).



Reading the diagram. The web server writes only to the database (synchronous). A daemon on the database host reads the commit log, extracts keys, batches them and sends deletes asynchronously (dotted) through a router to every cache cluster. In another region the replica's own log drives deletes, so local caches are invalidated only after the replica has the new data. Storage boundaries: database, log, cache clusters. Failure points: daemon lag, lost batches (replay from the log position) and deletes overtaking replication (hence per-region daemons). Scaling: batching reduces packet rates.

INDUSTRY EXAMPLE

Facebook: mcsqueal and leases. Facebook's Memcache deployment served billions of requests per second as a demand-filled look-aside cache in front of MySQL. Invalidation daemons called **mcsqueal** run on each database, extract deletes from committed transactions and broadcast them, batched, through **mcrouter** to all front-end clusters; in remote regions the replica's daemons do the same. **Leases** (64-bit tokens issued on a miss and voided by a delete) prevent stale sets, and, because a server returns a token only once every 10 seconds per key, they also throttle thundering herds; the paper reports a peak database query rate falling from 17K/s to 1.3K/s in one instance. A small **gutter** pool (about 1% of servers) takes over for failed cache servers with short TTLs, so a failure does not dump its load onto the databases [Nishtala et al. 2013]. *Evidence: Primary* — [Nishtala et al. 2013]

20.10.3 Versioned keys

Instead of deleting, change the key. Store a version number per entity (or per namespace) and include it in every cache key: `product:42:v17`. A write increments the version in the database (or in a small, strongly consistent version store); readers fetch the current version, then the versioned key. Old entries become unreachable and age out by TTL or eviction. Versioned keys make invalidation of whole groups cheap (bump `catalog_version` to invalidate every catalog page) and never suffer the stale-set race, because a stale value is written under an old key nobody reads. The cost is an extra lookup for the version and wasted memory until old versions are evicted. Content-hashed asset URLs (§20.5.2) are the same idea.

COMMON MISTAKE

"There are only two hard things in computer science: cache invalidation and naming things." The quip, usually attributed to Phil Karlton, is often used to excuse not designing invalidation. The hard part is enumerating which keys depend on which rows. Write the dependency down: for each cached object, list the tables and columns it is derived from and the mechanism (TTL, delete, CDC, version) that invalidates it.

20.11 Cache stampede

DEFINITION

Definition — Cache stampede (thundering herd, dog-piling). When a popular entry expires or is evicted, many concurrent requests miss at once and all recompute or reload it, multiplying load on the backend exactly when it is busiest.

If a key is read λ times per second and recomputing it takes δ seconds, about $\lambda\delta$ requests miss before the first refill lands. For $\lambda = 2,000$ requests/s and a 500 ms query, that is 1,000 identical queries.

20.11.1 Locks, leases and single-flight

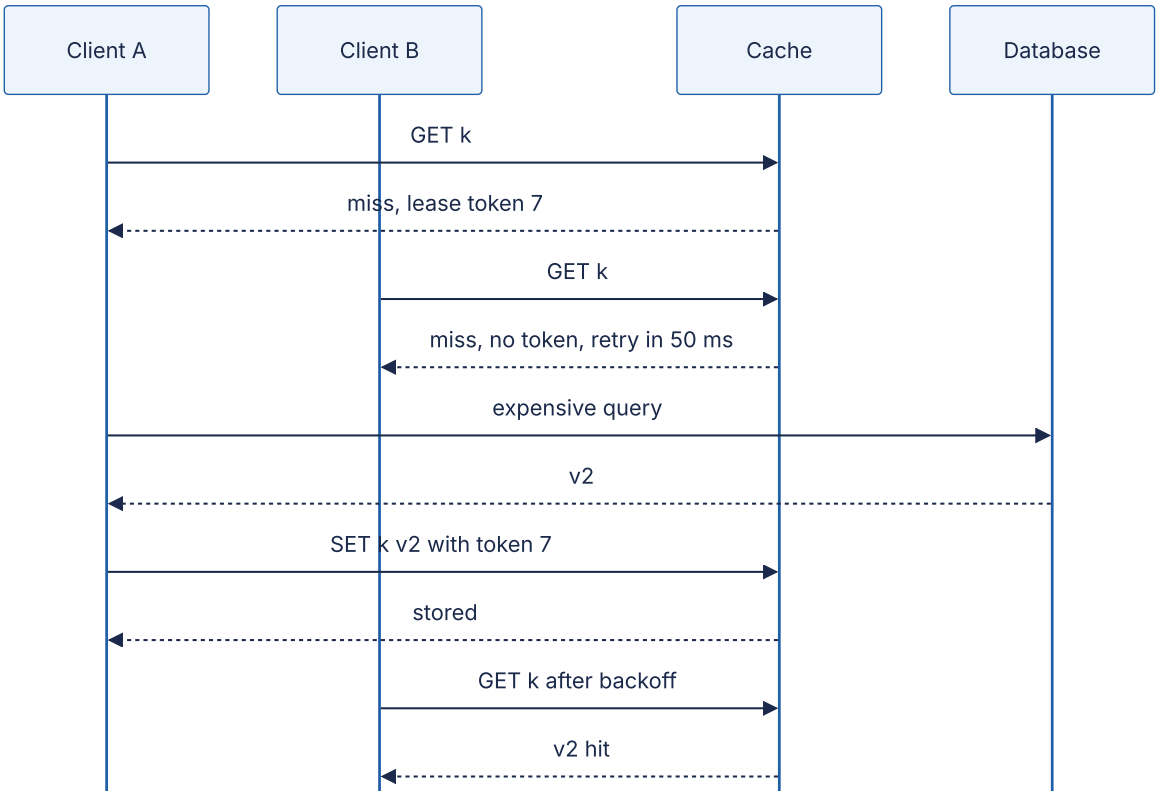
Let one request recompute and make the others wait or use a stale copy.

- **Single-flight (request coalescing)** inside a process: concurrent misses for a key share one in-flight load (Go's `singleflight`, Caffeine's `LoadingCache`). Across A app instances this still allows A loads.
- **Distributed lock**: the first misser sets `SET lock:k token NX PX 3000`; others wait briefly and retry the cache, or serve stale data. The lock needs a timeout in case the

holder dies (§26.10).

- **Leases** in Memcache (§20.10.2): the cache itself hands out one token per key per interval and tells other clients to retry shortly.

Figure 20.14 — Stampede protection with a lease: one recompute, others wait or serve stale.



Reading the diagram. Only the first misser gets a lease token; later missers are told to retry (or get a stale value), so only A queries the database. The `SET` needs a valid token, which also blocks the stale-set race of Figure 20.7. All calls are synchronous. Failure points: if A dies, its token expires and another client gets one; waiting adds latency. Scaling: one database query per refresh regardless of λ.

20.11.2 Probabilistic early expiration (XFetch)

Locks add coordination. **XFetch** avoids it: each reader independently decides, with a probability that rises as expiry approaches, to recompute *before* the entry expires, while everyone else keeps getting hits [Vattani, Chierichetti & Lowenstein 2015].

FORMULA

XFetch early-recomputation rule and its consequences (Poisson arrivals)

$$\text{recompute if } \; t - \delta\beta \, \ln(u) \geq t_{\text{exp}}, \quad u \sim \text{Uniform}(0, 1] \qquad \Rightarrow \qquad P(\text{early} \mid r) = e^{-r/(\delta\beta)}$$

$$\mathbb{E}[\text{lead}] \approx \delta\beta(\ln(\lambda\delta\beta) + \gamma), \qquad \mathbb{E}[\text{recomputes per refresh}] \approx e^{1/\beta}$$

Symbol	Meaning	Units
t	Current time	s
t_{exp}	Entry's expiry time	s
$r = t_{\text{exp}} - t$	Time remaining before expiry	s
δ	Measured recompute time, stored with the value	s
β	Aggressiveness; 1 is the recommended default	dimensionless
u	Uniform random number per request	dimensionless
λ	Request rate for the key	requests/s
γ	Euler–Mascheroni constant, ≈ 0.5772	dimensionless

Since $-\ln u$ is exponentially distributed, triggers form a Poisson process of intensity $\lambda e^{-r/(\delta\beta)}$; the remaining time at the first trigger is Gumbel-distributed, which gives the expected lead, and requests during the δ-second recompute add $e^{1/\beta} - 1$ extra triggers on average, independent of λ (§20.19, Q7).

WORKED EXAMPLE

Worked example — A hot key with a 2-second recompute. $\lambda = 100$ requests/s, $\delta = 2$ s, $\beta = 1$, TTL 60 s.

1. Per-request trigger probability with 10 s left: $e^{-10/2} = e^{-5} = 0.0067$; with 2 s left: $e^{-1} = 0.37$.
2. Expected lead: $\lambda\delta\beta = 200$; $\ln 200 = 5.298$; $2 \times (5.298 + 0.577) = 11.75$ s before expiry.
3. Recomputes per refresh: $e^{1} \approx 2.72$, against $\lambda\delta = 200$ with no protection.
4. Extra cost: the entry is refreshed every $60 - 11.75 + 2 \approx 50$ s instead of 62 s, about 23% more refreshes.
5. With $\beta = 2$: lead $4 \times (\ln 400 + 0.577) = 4 \times 6.569 = 26.3$ s and $e^{0.5} = 1.65$ recomputes; with $\beta = 0.5$: lead 5.2 s and $e^{2} = 7.4$ recomputes.

Interpretation. β trades earlier refreshes for fewer duplicate recomputations; the lead grows only logarithmically with traffic, so XFetch scales to any λ. **Engineering implication.** Store δ with each value, start with $\beta = 1$, and if duplicate recomputes are expensive combine XFetch with a non-blocking lock so only the first trigger recomputes; no request ever waits.

The simulation below checks these numbers.

```
        """One hot key under Poisson traffic: recomputations and misses per refresh for
        no protection, a recompute lock, and XFetch [Vattani et al. 2015]."""
        import math
        import random

        def simulate(policy, rate=100.0, ttl=60.0, delta=2.0, beta=1.0,
                     hours=3.0, seed=42):
            rng = random.Random(seed)
            expiry = ttl                          # value computed at t = 0
            inflight = []                         # finish times of running recomputes
            t, end = 0.0, hours * 3600
            recomputes = misses = requests = episodes = 0
            last_episode, leads = -1e9, []

            def start(now):
                nonlocal recomputes, episodes, last_episode
                recomputes += 1
                inflight.append(now + delta)
                if now > last_episode + delta:    # first recompute of a new refresh
                    episodes += 1
                    last_episode = now
                    leads.append(expiry - now)    # > 0 means refreshed before expiry

            while True:
                t += rng.expovariate(rate)        # next request (Poisson arrivals)
                if t >= end:
                    break
                requests += 1
                for f in [f for f in inflight if f <= t]:
                    inflight.remove(f)            # a recompute finished: store value
                    expiry = f + ttl
                if t >= expiry:                   # value missing: the request misses
                    misses += 1
                    if policy == "none" or not inflight:
                        start(t)                  # "lock" lets only one recompute run
                elif policy.startswith("xfetch"):
                    early = t - delta * beta * math.log(rng.random()) >= expiry
                    if early and (policy == "xfetch" or not inflight):
                        start(t)                  # "xfetch+lock" also skips duplicates
            return dict(requests=requests, episodes=episodes,
                        per_refresh=recomputes / episodes, miss_ratio=misses / requests,
                        mean_lead=sum(leads) / len(leads))

        if __name__ == "__main__":
            rate, delta = 100.0, 2.0
            print(f"rate {rate:.0f} req/s, TTL 60 s, recompute time {delta:.0f} s, 3
hours")
            print(f"{'policy':<14}{'beta':>5}{'recomputes/refresh':>20}{'miss
ratio':>12}"
                  f"{'mean lead (s)':>15}{'theory lead':>13}")
            runs = [("none", 1.0), ("lock", 1.0), ("xfetch", 0.5), ("xfetch", 1.0),
                    ("xfetch", 2.0), ("xfetch+lock", 1.0)]
            for policy, beta in runs:
                r = simulate(policy, rate=rate, delta=delta, beta=beta)
                theory = (f"{delta * beta * (math.log(rate * delta * beta) +
0.5772):.2f}"
```

```
                            if policy.startswith("xfetch") else "-")
                print(f"{policy:<14}{beta:>5}{r['per_refresh']:>20.2f}
{r['miss_ratio']:>12.4%}"
                      f"{r['mean_lead']:>15.2f}{theory:>13}")
```

Output:

```
rate 100 req/s, TTL 60 s, recompute time 2 s, 3 hours
policy         beta  recomputes/refresh  miss ratio  mean lead (s)  theory lead
none            1.0              202.77     3.1517%          -0.01            -
lock            1.0                1.00     3.2451%          -0.01            -
xfetch          0.5                6.71     0.0000%           5.30         5.18
xfetch          1.0                2.50     0.0000%          12.05        11.75
xfetch          2.0                1.66     0.0000%          26.58        26.27
xfetch+lock     1.0                1.00     0.0000%          11.83        11.75
```

What to notice:

- Without protection each refresh costs about 200 recomputations ($\lambda\delta$) and 3.2% of requests miss ($\delta/(\text{TTL}+\delta) = 2/62$).
- A lock fixes the duplicate work but not the misses: requests during the recompute still wait.
- XFetch removes misses entirely, and its leads match the formula within half a second. Duplicate recomputes (6.71, 2.50, 1.66) sit within sampling error of $e^{1/\beta}$ (7.39, 2.72, 1.65); averaged over 40 simulated hours they are 7.50, 2.71 and 1.63.
- XFetch plus a non-blocking lock gives exactly one recompute and no misses.

20.11.3 Stale-while-revalidate

Keep entries past their **soft TTL**: after it, serve the stale value immediately and trigger one background refresh; delete only at a later **hard TTL**. This is HTTP's `stale-while-revalidate` (§20.5.2) applied to an application cache, and it gives the same result as XFetch plus a lock when traffic is steady. It also enables **stale-if-error**: if the database is down, keep serving the last good value. The price is explicit staleness, so use it only where showing a value a few seconds old is acceptable.

20.12 Cache penetration

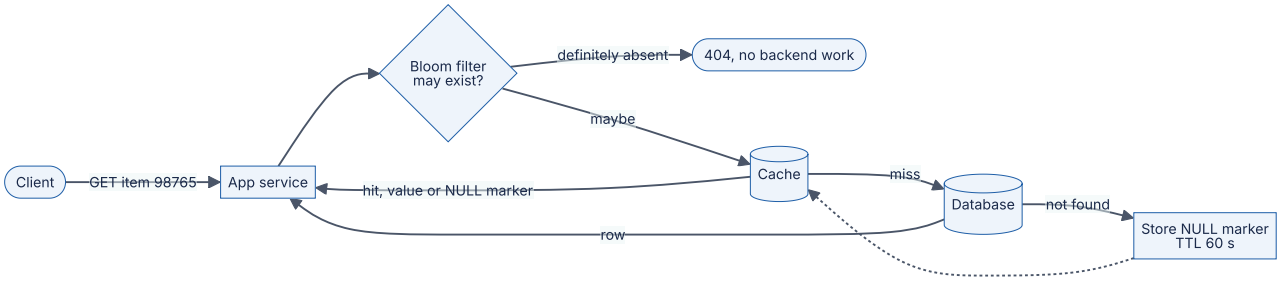
DEFINITION

Definition — Cache penetration. Requests for keys that do not exist in the database. Because there is nothing to cache, every such request misses and reaches the database. A bug or an attacker iterating random IDs can push the database's full read load through the cache.

Two defenses work together:

1. **Negative caching**: cache the “not found” result under the key with a short TTL (for example 30–60 s), so repeated lookups of the same missing key hit. It fails against random keys, each requested once, and wastes memory on them.
2. **A Bloom filter of existing keys** in front of the cache: if the filter says “definitely absent”, return 404 without touching cache or database. False positives (1% at about 9.6 bits per key, §17.4.3) only let a small fraction through. The filter must be updated on insert and rebuilt periodically to remove deleted keys, or use a cuckoo filter that supports deletion (§17.4.7).

Figure 20.15 — Defenses against cache penetration.



Reading the diagram. The in-memory Bloom filter rejects most nonexistent IDs before any network call. Survivors go to the cache, which returns a value or a cached “not found” marker; only true misses reach the database, and a not-found result is stored with a short TTL (dotted). All checks are synchronous. Storage boundaries: filter, cache, database. Failure points: a filter that lags behind inserts returns false 404s, so add keys before they become visible, and delete negative markers on creation. Scaling: at 1% false positives, 100,000 random IDs/s let about 1,000 through.

Rate limiting per client (§22.3) and validating ID formats before lookup complete the defense.

20.13 Cache avalanche

DEFINITION

Definition — Cache avalanche. A large fraction of cache entries becomes unavailable at once, because many keys expire together or because cache nodes fail or restart, so the database suddenly receives the full read load.

At $h = 0.99$ the database sees 1% of reads; with the cache gone it sees 100%, a 100× jump, and the resulting timeouts and retries add more load (§22.12).

Defenses:

- **TTL jitter** (§20.8) so mass-written keys do not expire together.
- **Replicated, highly available cache** (Redis replicas with automatic failover; a gutter pool, §20.10.2), so one node failure does not empty its share of keys.
- **Warming**: before a new or restarted cache takes traffic, load the hottest keys from a key list, a snapshot (RDB) or a warm peer; Facebook warms a new cluster by letting it read from a warm cluster's caches instead of the database [Nishtala et al. 2013].
- **Tiered fallback**: an in-process L1 (§20.5.4) and stale-if-error serving keep answering hot keys during an L2 outage.
- **Protect the database**: admission control, load shedding and circuit breakers (§22.9, §22.13) so it degrades by rejecting a fraction of requests rather than collapsing.

Table 20.7 — The three classic cache failure modes.

What goes wrong	Why	Detection	Recovery	Prevention
Stampede	One hot key expires; $\lambda\delta$ concurrent recomputes	Spikes of identical queries; per-key miss bursts	Shed load; serve stale	Leases or single-flight, XFetch, stale-while-revalidate
Penetration	Requests for nonexistent keys bypass the cache	High miss ratio with many "not found" results	Block abusive clients	Negative caching, Bloom filter, rate limits, ID validation
Avalanche	Many keys expire together, or cache nodes fail	Hit ratio collapses; database queries/s jump	Throttle, warm caches, fail over	Jitter, replication, warming, L1 and stale-if-error, load shedding

20.14 Distributed caching and consistent hashing

A single cache node is limited by its RAM (tens to hundreds of GB) and, for Redis, by one execution core. A distributed cache partitions keys over N nodes. With `hash(key) mod N`, adding a node remaps about $N/(N+1)$ of keys, which for a cache means that fraction of entries effectively vanishes: a self-inflicted avalanche. **Consistent hashing** with virtual nodes moves only about $1/(N+1)$ of keys and spreads the moved load evenly; §14.8 derives it, and §3.3.2 works through a cache example. Three routing designs exist:

- **Client-side routing**: the client library holds the ring (most Memcached clients, Ketama hashing).
- **Proxy routing**: a proxy such as mcrouter or Twemproxy routes, replicates and fails over on behalf of thin clients.
- **Server-assisted routing**: Redis Cluster's fixed 16,384 slots with `MOVED` redirects (§20.6.4), which moves whole slots on resharding rather than ring arcs.

When a node fails, its keys move to their successors cold. Replication (Redis replicas, EVCache copies) or a gutter pool avoid that miss spike.

20.15 Hot keys

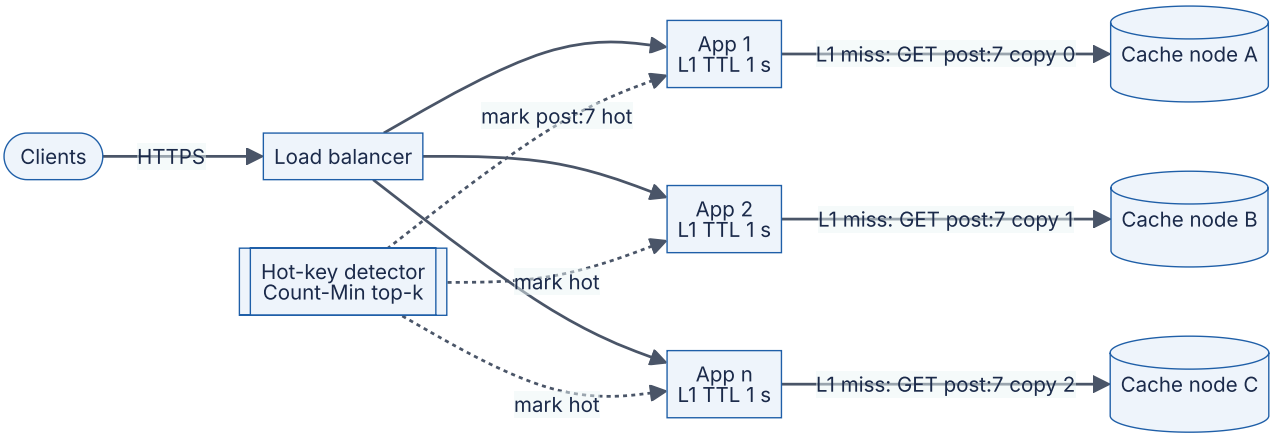
DEFINITION

Definition — Hot key. A single key whose request rate exceeds what the node holding it can serve, so partitioning cannot help: all requests for one key go to one node.

A celebrity's post, a flash-sale product or a global configuration key can draw hundreds of thousands of requests per second. **Detect** hot keys by sampling requests into a Count-Min sketch or top-*k* structure per cache client or proxy (§17.6), or with server tools such as `redis-cli --hotkeys` (which needs an LFU eviction policy). Then **mitigate**:

1. **Local L1 caches** with a short TTL in each app instance, so each instance asks the remote cache at most once per TTL. Redis 6's **client-side caching** sends invalidation messages to clients that track the keys they cached [Redis Docs: Client-side caching].
2. **Key replication**: store copies under `k#0` … `k#r-1` on different nodes, read a random copy and write or delete all copies.
3. **Request coalescing** at the proxy so concurrent identical gets become one.

Figure 20.16 — Hot-key mitigation with L1 caches and replicated keys.



Reading the diagram. Each of *n* app instances keeps hot keys in its L1 for one second, so it sends at most one request per second per hot key to the remote tier, choosing one of three replicated copies at random. A detector fed by sampled requests marks keys hot asynchronously (dotted). Storage boundaries: process memory and three cache nodes. Failure points: L1 staleness up to its TTL; writes and deletes must reach every copy. Scaling: remote load grows with instance count, so lengthen the L1 TTL or add copies.

FORMULA

Remote load from a hot key with L1 caches and replication

$$R_{\text{remote}} \le \min\left(\lambda_k, \frac{A}{T_{L1}}\right), \qquad R_{\text{per node}} = \frac{R_{\text{remote}}}{r}$$

Symbol	Meaning	Units
λ_k	Request rate for the hot key	requests/s
A	Number of app instances with an L1 cache	instances
T_{L1}	L1 TTL	s
r	Number of replicated copies of the key	copies
R_{remote}, $R_{\text{per node}}$	Requests to the remote tier, and to each node holding a copy	requests/s

WORKED EXAMPLE

Worked example — A viral post. $\lambda_k = 600,000$ requests/s; one cache node can serve about 100,000 requests/s (illustrative).

1. Without mitigation: 600,000 requests/s to one node, six times its capacity.
2. Replication alone with $r = 8$: $600,000/8 = 75,000$ per node, under capacity but with little headroom.
3. L1 caches on $A = 300$ instances with $T_{L1} = 1$ s: $R_{\text{remote}} \le 300/1 = 300$ requests/s.

Interpretation. A one-second L1 cuts remote load by a factor of 2,000 at the cost of up to one second of staleness. **Engineering implication.** Use L1 caching for read-hot keys whenever a second of staleness is acceptable; reserve key replication for keys that must stay fresher or for write-hot counters (which need sharded counters instead, §14.10).

20.16 Redis versus Memcached

Table 20.8 — Redis (and Valkey) versus Memcached.

Aspect	Redis / Valkey	Memcached
What it is	In-memory data-structure server	In-memory cache of opaque blobs
How it works	Single-threaded command execution; optional I/O threads	Multithreaded workers sharing one hash table; slab allocator
Advantages	Rich types (hashes, sorted sets, streams), atomic scripts, persistence, replication, Cluster, pub/sub	Simple; scales vertically across cores; predictable memory; very mature
Disadvantages	One core executes commands; fork for persistence; more to tune	Strings only; no persistence or replication (client or proxy must provide them)
Performance	~100,000+ ops/s per core; more with I/O threads or pipelining	Millions of ops/s per server with many threads
Operational complexity	Medium–high (Sentinel or Cluster, persistence, memory policy)	Low (stateless nodes, client-side hashing)
Cost	Higher memory overhead per small key; replicas double RAM	Low overhead; slab waste 10–20%
Use cases	Sessions, leaderboards, rate limiters, queues, caches needing structure	Large look-aside caches of serialized objects
Choose when	You need data structures, atomic operations or durability	You need a fast, simple, horizontally scaled cache

Licensing (as of 2026). Redis was BSD-licensed until March 2024, when Redis 7.4 moved to a dual RSALv2/SSPLv1 license, which is not an open-source license under the OSI definition. The Linux Foundation then launched **Valkey**, a BSD-licensed fork of Redis 7.2.4 supported by several cloud providers. Redis 8.0 (2025) added the OSI-approved AGPLv3 as a third licensing option [Redis Blog 2024; Redis Blog 2025; Linux Foundation 2024]. Managed services such as AWS ElastiCache offer Valkey, Redis OSS and Memcached engines. For system design the engines are protocol-compatible for the features in this chapter; check the license and feature set of the specific version you deploy.

20.17 Summary and revision notes

SUMMARY

- A cache trades staleness, memory and a new failure mode for latency and backend capacity. $T_{\text{eff}} = t_c + (1-h)t_b$; database reads $= (1-h)R$. Watch the miss rate: 99% → 98% doubles database reads, and p99 is a miss once the miss ratio exceeds 1%.
- Zipf skew makes small caches effective: the ideal top-C hit ratio is $H_{C,s}/H_{N,s}$, about 68% for 1% of a million keys at $s = 1$, and grows by a roughly constant amount per doubling.
- Cache at every layer that freshness allows: browser and CDN for versioned and public content, in-process L1 for tiny hot data, remote L2 as the shared, invalidatable tier; the database's buffer pool is the only coherent one.
- Redis: one thread executes commands (atomic, but one slow command blocks all), optional I/O threads, RDB/AOF persistence, asynchronous replication, 16,384 hash slots with `MOVED` / `ASK`. Memcached: multithreaded, slab allocator (10–20% waste), segmented HOT/WARM/COLD LRU, no persistence or replication.
- Cache-aside with delete-after-commit is the default; it still has a stale-set race, closed by leases, versioned sets or CDC deletes, and bounded by TTLs. Write-through adds write latency; write-back risks $r_w T_{\text{flush}}$ writes; write-around avoids polluting the cache.
- Every entry needs a TTL; add jitter so peak refills are K/J.
- LRU is scan-sensitive, in-cache LFU cannot forget; ARC and W-TinyLFU combine recency and frequency and come within a few points of ideal. Redis samples keys to approximate LRU and LFU; never run a cache with `noeviction`.
- Invalidate with TTL + explicit deletes, CDC from the commit log (mcsqueal), or versioned keys.
- Stampede: leases/single-flight, XFetch ($t - \delta\beta \ln u \geq t_{\text{exp}}$; lead $\approx \delta\beta(\ln \lambda\delta\beta + \gamma)$; $e^{1/\beta}$ recomputes), stale-while-revalidate. Penetration: negative caching and Bloom filters. Avalanche: jitter, replication, warming, L1 and stale-if-error, load shedding.
- Distribute keys by consistent hashing or slots; hot keys need L1 caches ($\leq A/T_{L1}$ remote requests/s) or replicated copies.

20.18 Knowledge checkpoint

1. A cache lookup costs 0.4 ms and a database read 12 ms. What is the mean read latency at $h = 0.95$?
2. Why can a 1-point drop in hit ratio double database load?

3. Using the $s = 1$ approximation, what is the ideal hit ratio for $C = 10^5$ of $N = 10^7$ keys?
4. Why are Redis commands atomic without locks, and what is the operational cost of that design?
5. How does Redis Cluster map a key to a node, and how do you make two keys land in the same slot?
6. An item with header is 200 bytes. Which Memcached chunk (growth factor 1.25) holds it, and what fraction is wasted?
7. Describe the stale-set race in cache-aside and one mechanism that prevents it.
8. Why delete the cache entry after a write instead of setting the new value?
9. A write-back cache receives 50,000 writes/s and flushes every 2 s. How many acknowledged writes can a crash lose?
10. 600,000 keys were written in 30 s with the same TTL. The database tolerates 1,500 refills/s. What jitter range do you need?
11. Why does in-cache LFU do badly when popularity shifts, and how does W-TinyLFU avoid it?
12. What do ARC's ghost lists record, and what does a ghost hit change?
13. For XFetch with $\lambda = 400$ requests/s, $\delta = 0.5$ s, $\beta = 1$, what is the expected lead before expiry?
14. 200 app instances use a 2 s L1 TTL for a hot key. What is the maximum remote request rate for it?

Answers

1. $0.4 + 0.05 \times 12 = 1.0$ ms.
2. Database reads are proportional to the miss ratio; going from 99% to 98% doubles the miss ratio from 1% to 2%.
3. $(\ln 10^5 + 0.577)/(\ln 10^7 + 0.577) = (11.513 + 0.577)/(16.118 + 0.577) = 12.090/16.695 \approx 0.724$.
4. One thread executes all commands sequentially. Any slow command (`KEYS` , large `DEL` , long scripts) blocks every client, and a process uses only one core for execution.
5. `CRC16(key) mod 16384` gives the slot; each primary owns a set of slots. Put the same hash tag in both keys, for example `{user:42}:profile` and `{user:42}:cart` .
6. Classes: 152, 192, 240. 200 > 192, so the 240-byte chunk; waste $40/240 = 16.7\%$.
7. A reader reads v_1 from the database, a writer commits v_2 and deletes the key, and the reader then sets v_1, which stays until TTL. Leases (the delete voids the reader's token), versioned compare-and-set, or CDC-issued deletes after commit prevent or repair it.
8. Concurrent writers can set values in a different order from their database commits, leaving an old value cached; deletes are idempotent and order-insensitive.
9. $50,000 \times 2 = 100,000$ writes.
10. $J \geq 600,000/1,500 = 400$ s.
11. Counts of formerly hot entries stay high, so they are never evicted. W-TinyLFU halves sketch counters periodically (aging) and admits new keys through a recency

window.
12. The keys (not values) recently evicted from T_1 and T_2. A hit in B_1 increases the target size of T_1 (more recency); a hit in B_2 decreases it (more frequency).
13. $\lambda\delta\beta = 200$; $0.5 \times (\ln 200 + 0.577) = 0.5 \times 5.875 = 2.94$ s.
14. $200/2 = 100$ requests/s.

20.19 Interview questions

Basic

Q1. When would you add a cache, and what do you need to state when you do? Answer. When reads repeat, the store is too slow or near capacity, and the data tolerates bounded staleness. State the hit ratio and resulting database load, the pattern, TTL and invalidation, the outage behavior and the memory size.

Q2. Explain cache-aside versus write-through. Answer. Cache-aside: the application reads the cache, fills it from the database on a miss and deletes the key after writing the database; the cache is optional and holds only read data. Write-through: writes go through the cache layer, which writes the database synchronously and updates itself, so reads through that layer are fresh, at the cost of higher write latency and caching data that may never be read.

Q3. Redis or Memcached? Answer. Memcached for a simple, multithreaded, large look-aside cache of serialized objects. Redis (or Valkey) when you need data structures, atomic operations, persistence or replication: sessions, leaderboards, rate limiters. Mention the Redis licensing change and Valkey as of 2026.

Intermediate

Q4. Users sometimes see their old profile picture for an hour after changing it. Diagnose. Answer. The one-hour duration points to a TTL acting as backstop for a stale-set race, a missed delete on some write path, delete-before-write ordering, or a CDN or browser layer. Fix: delete after commit, leases or versioned keys, CDC-driven invalidation, and a versioned image URL.

Q5. Your database melts down every day at 09:00. Why might the cache be involved? Answer. A cache avalanche: a job warms or rewrites many keys at the same time each day with identical TTLs, or a daily cache restart empties it. Add TTL jitter, warm caches gradually, keep replicas, add stale-if-error serving, and protect the database with load shedding.

Q6. How do you protect a hot key that expires? Answer. Make sure only one request recomputes it: single-flight in process plus a lease or lock across processes, or XFetch to

refresh before expiry with $\beta \approx 1$, or stale-while-revalidate. For read volume beyond one node, add L1 caches and key replication.

Advanced

Q7. Derive why XFetch's duplicate recomputations do not grow with traffic. Answer. Each request triggers with probability $e^{-r/(\delta\beta)}$; Poisson arrivals give trigger intensity $\lambda e^{-r/(\delta\beta)}$. Let $G = \lambda\delta\beta e^{-R/(\delta\beta)}$ at the first trigger time R; since $P(\text{no trigger before } r) = \exp(-\lambda\delta\beta e^{-r/(\delta\beta)})$, G is exponential with mean 1. Triggers in the following δ seconds have expected count $G(e^{1/\beta} - 1)$, so the mean total is $e^{1/\beta}$ for any λ, while the lead $\delta\beta(\ln \lambda\delta\beta + \gamma)$ grows only logarithmically.

Q8. Design invalidation for a multi-region cache in front of a MySQL primary with cross-region replicas. Answer. Writers delete local keys after commit; a CDC daemon per database (primary and each replica) tails the commit log and issues batched deletes to that region's caches, so a region's caches are invalidated only after its replica has applied the change, preventing refills from lagging replicas. Keep TTLs as a backstop, use leases against stale sets, and for read-your-writes after a user's own update, route that user's reads to the primary or mark the key so misses read the primary (Facebook's remote markers) [Nishtala et al. 2013].

Q9. A team proposes replacing LRU with LFU in a 200 GB cache. How do you decide? Answer. Replay a production trace through LRU, LFU with aging, ARC and W-TinyLFU at several sizes and compare hit ratios and database queries per second (§20.9.5); scans hurt LRU, popularity shifts hurt pure LFU. In Redis this is `allkeys-lfu` versus `allkeys-lru`.

20.20 Practical exercises

1. **Measure skew.** Extend the §20.3.3 program to read a log of keys (one per line), fit the Zipf exponent by linear regression of log frequency on log rank, and plot measured LRU hit ratio against cache size.
2. **Add a doorkeeper.** Add a Bloom-filter doorkeeper (§17.4) to the W-TinyLFU class in §20.9.5 and report the hit-ratio change on the three traces.
3. **Reproduce the race.** With Redis and PostgreSQL in Docker, write a reader that sleeps 200 ms between its database read and its cache `SET`, and a writer that updates and deletes. Count stale entries over 10,000 iterations; then add a version check with a Lua script and count again.
4. **Stampede test.** Put a 1 s sleep behind a cached endpoint, expire the key under 500 concurrent clients, and measure database calls with no protection, with a `SET NX PX` lock, and with XFetch.

5. **Redis eviction.** Fill a Redis instance with `maxmemory 100mb` under `allkeys-lru` with `maxmemory-samples` 1, 5 and 10, replay a Zipf trace and compare hit ratios with exact LRU from §20.9.5.

20.21 Design challenges

1. **Product catalog for a flash sale.** 5 million products, 400,000 reads/s at peak with one product drawing 30% of traffic during the sale, prices that must be correct within 2 s. Design the cache layers, TTLs, invalidation and hot-key handling, and compute database load at the expected hit ratio.
2. **Session store.** 20 million active sessions of 1 KB, 100,000 reads/s, sessions must survive a single node failure. Compare Redis with replicas against Memcached with client-side replication, including memory with slab overhead.
3. **Social feed cache.** Feeds are lists of post IDs per user, rebuilt from a fan-out service. Design the keys, eviction policy, invalidation on post deletion, and the warm-up plan for a new region.

20.22 References

1. Belady, L. A. *A Study of Replacement Algorithms for a Virtual-Storage Computer*. IBM Systems Journal 5(2), 1966.
2. Breslau, L., Cao, P., Fan, L., Phillips, G., Shenker, S. *Web Caching and Zipf-like Distributions: Evidence and Implications*. IEEE INFOCOM, 1999.
3. Caffeine project. *Caffeine Wiki: Efficiency* and *Design*. GitHub, accessed 2026. https://github.com/ben-manes/caffeine/wiki
4. Einziger, G., Friedman, R., Manes, B. *TinyLFU: A Highly Efficient Cache Admission Policy*. ACM Transactions on Storage 13(4), 2017.
5. Linux Foundation. *Linux Foundation Launches Open Source Valkey Community* (press release). 2024.
6. Megiddo, N., Modha, D. S. *ARC: A Self-Tuning, Low Overhead Replacement Cache*. USENIX Conference on File and Storage Technologies (FAST), 2003.
7. Memcached project. *Replacing the cache replacement algorithm in memcached* (Modern LRU). memcached.org blog, 2018. https://memcached.org/blog/modern-lru/
8. Memcached project. *protocol.txt* (text and meta protocols) and wiki (overview, slabs, threads). Accessed 2026. https://github.com/memcached/memcached
9. MySQL Server Team. *MySQL 8.0: Retiring Support for the Query Cache*. MySQL Server Blog, 2017.
10. Netflix. *EVCache*. GitHub repository, accessed 2026. https://github.com/Netflix/EVCache
11. Nishtala, R., Fugal, H., Grimm, S., Kwiatkowski, M., Lee, H., Li, H. C., McElroy, R., Paleczny, M., Peek, D., Saab, P., Stafford, D., Tung, T., Venkataramani, V. *Scaling*

Memcache at Facebook. USENIX NSDI, 2013. https://www.usenix.org/conference/nsdi13/technical-sessions/presentation/nishtala
12. OpenZFS project. *OpenZFS documentation* (Adaptive Replacement Cache). Accessed 2026.
13. Redis. *Redis documentation*: Data types; Persistence; Replication; Cluster specification; Key eviction; EXPIRE; Client-side caching; Redis configuration (`io-threads`). Accessed 2026. https://redis.io/docs/
14. Redis. *Redis adopts dual source-available licensing* (2024) and *Redis 8 general availability, AGPLv3* (2025). Redis Blog.
15. Fielding, R., Nottingham, M., Reschke, J. *HTTP Caching*. RFC 9111, IETF, 2022. https://www.rfc-editor.org/rfc/rfc9111
16. Nottingham, M. *HTTP Cache-Control Extensions for Stale Content*. RFC 5861, IETF, 2010. https://www.rfc-editor.org/rfc/rfc5861
17. Valkey project. *Valkey 8.0: Delivering Enhanced Performance and Reliability*. Valkey Blog, 2024. https://valkey.io/blog/valkey-8-0-0-rc1/
18. Vattani, A., Chierichetti, F., Lowenstein, K. *Optimal Probabilistic Cache Stampede Prevention*. Proceedings of the VLDB Endowment 8(8), 2015.

20.23 Open verification items

- Nishtala et al. 2013: lease rate of one token per 10 s per key, the 17K/s → 1.3K/s peak database query figure and the ~1% gutter pool size were confirmed only via secondary summaries (the USENIX site was unreachable); check against the paper.
- Breslau et al. 1999: exponent range of the traces (stated here only as “below 1”).
- Valkey 8.0’s 1.2M requests/s figure is a vendor benchmark on AWS r7g; Redis license history and versions (7.4 dual license; 8.0 AGPLv3 option) as of 2026.
- Caffeine’s admission randomization against hash flooding, and default window/probation proportions.
- Exact URLs for the MySQL query-cache blog post, the Linux Foundation press release and Redis blog posts.

Chapter 21 — Content Delivery Networks

> **Level:** 2 · **Track:** A (Traditional) · **Estimated reading time:** 3 h

At 10 million daily users, PhotoShare serves about 28 Gbit/s of images at peak (§1.9). Chapter 1 drew the box that makes this affordable, a **CDN**, as a hexagon labeled "external". This chapter opens the hexagon. A CDN is a geographically distributed cache built from the ideas of Chapters 7 and 20: anycast and DNS steer each user to a nearby site, TLS terminates close to the user, and a hierarchy of caches keeps most requests away from your servers. You will learn how shielding turns hundreds of misses into one, how cache keys and headers decide what is shared, how to purge in a fraction of a second, how video adapts to the network, and how Netflix built a CDN inside other people's networks.

21.1 Learning objectives

After this chapter you should be able to:

- Draw a CDN (points of presence, mid-tier, origin) and trace a cache hit and a cache miss end to end.
- Compare DNS-based request routing with anycast and choose between them.
- Compute request and byte offload ratios, compose hit ratios across cache tiers, and estimate origin bandwidth and monthly egress cost.
- Explain why origin shielding reduces origin load and quantify the reduction with a Poisson model and a simulation.
- Design cache keys and `Cache-Control` policies (`max-age`, `s-maxage`, `stale-while-revalidate`, `stale-if-error`, `ETag`) under RFC 9111, and choose between URL purge, tag-based purge and versioned URLs.
- Explain dynamic site acceleration and edge-side includes, and estimate when they help.
- Size video delivery for an adaptive bitrate ladder and compare throughput-based and buffer-based bitrate adaptation.
- Describe edge compute runtimes, Netflix Open Connect and the CDN's role in security.

21.2 Prerequisites

- Anycast, BGP, DNS resolution and TTLs — §7.5–§7.6 (both request-routing methods build on them).
- TCP handshake, slow start, TLS 1.3, HTTP/2 and HTTP/3 — §7.7, §7.9, §7.10 (the latency a CDN removes is round trips).
- HTTP methods, status codes, `ETag` and conditional requests — §8.6.3–§8.6.4; cookies — §8.10.1.
- Cache hit ratio, TTL, eviction and stampedes — §20.3, §20.8, §20.9, §20.11 (a CDN is a cache; this chapter does not repeat the general theory).
- Consistent hashing — §14.8 (used inside each CDN site).
- Units: **MB/GB/TB/PB** are powers of ten; bit rates use **Mb/s** and **Gb/s** (10^6 and 10^9 bits per second). One byte is eight bits.

21.3 CDN architecture

21.3.1 What a CDN is and why it exists

> DEFINITION
>
> **Definition — Content delivery network (CDN).** A distributed system of caching reverse proxies (§7.12) deployed in many network locations, which serves content on behalf of an **origin** (the authoritative server for the content) from the location nearest each user, fetching from the origin only on a cache miss.

First-principles explanation

A bestselling author lives in one city. If every reader had to travel there to buy a book, the queue would stretch for kilometers and most readers would give up. Instead, the publisher prints copies and ships them to bookshops in every town. Readers walk to a local shop; only when a shop runs out does it order from the warehouse. A CDN is that network of bookshops: the **origin** is the author's house, the **edge** servers are the bookshops, and a **cache miss** is a shop reordering from the warehouse.

Why does this exist?

A CDN solves four problems at once:

1. **Latency.** Light in fiber covers about 200 km per millisecond, so a Sydney user talking to a server in Virginia pays roughly 200 ms per round trip, and a fresh HTTPS fetch needs several round trips (§7.15; [Grigorik 2013]). An edge 10 ms away cuts each round trip twentyfold.

2. **Origin load.** A hit is served without touching the origin, so the origin is sized for misses, not for total traffic.
3. **Egress cost and backbone capacity.** Bytes served from inside or near the user's network do not cross expensive long-haul links or the origin's metered egress.
4. **Surges and attacks.** Hundreds of sites with terabits of aggregate capacity absorb flash crowds and DDoS floods that would saturate any single data center (§10.11).

Mathematical foundations

FORMULA

Time to fetch an object over a new connection

$$T_{\text{fetch}} \approx (n_{\text{hs}} + \lceil \log_2 (\frac{S}{W_0} + 1) \rceil) \times \text{RTT} + t_{\text{server}}$$

Symbol	Meaning	Units
n_{hs}	Handshake round trips before the request (TCP + TLS 1.3 = 2; QUIC = 1; reused connection = 0)	round trips
S	Response size	bytes
W_0	Initial congestion window (typically 10 segments × 1,460 B ≈ 14.6 KB)	bytes
$\lceil \log_2 (S/W_0 + 1) \rceil$	Slow-start rounds to deliver S, the first of which carries the request (§7.7.7)	round trips
RTT	Round-trip time between client and server	s
t_{server}	Server processing time	s

WORKED EXAMPLE

Worked example — A 100 KB photo from Virginia versus a Sydney edge.

1. Slow-start rounds: $S/W_0 = 100,000/14,600 = 6.85$; $\log_2(7.85) = 2.97$, so 3 rounds (14.6 + 29.2 + 58.4 = 102.2 KB ≥ 100 KB).
2. Round trips: $n_{hs} = 2$ (TCP + TLS 1.3), so $2 + 3 = 5$.
3. From the origin (RTT 200 ms): $5 \times 200 = 1,000$ ms, plus server time.
4. From a Sydney edge (RTT 10 ms), on a cache hit: $5 \times 10 = 50$ ms.

Interpretation. The same bytes arrive 20 times sooner, and no amount of origin bandwidth could close that gap: the cost is round trips, not throughput. **Engineering implication.** For small and medium objects, moving the server closer beats every other optimization; on a cache miss the edge still saves the handshakes, as §21.7.2 shows.

21.3.2 Points of presence

DEFINITION

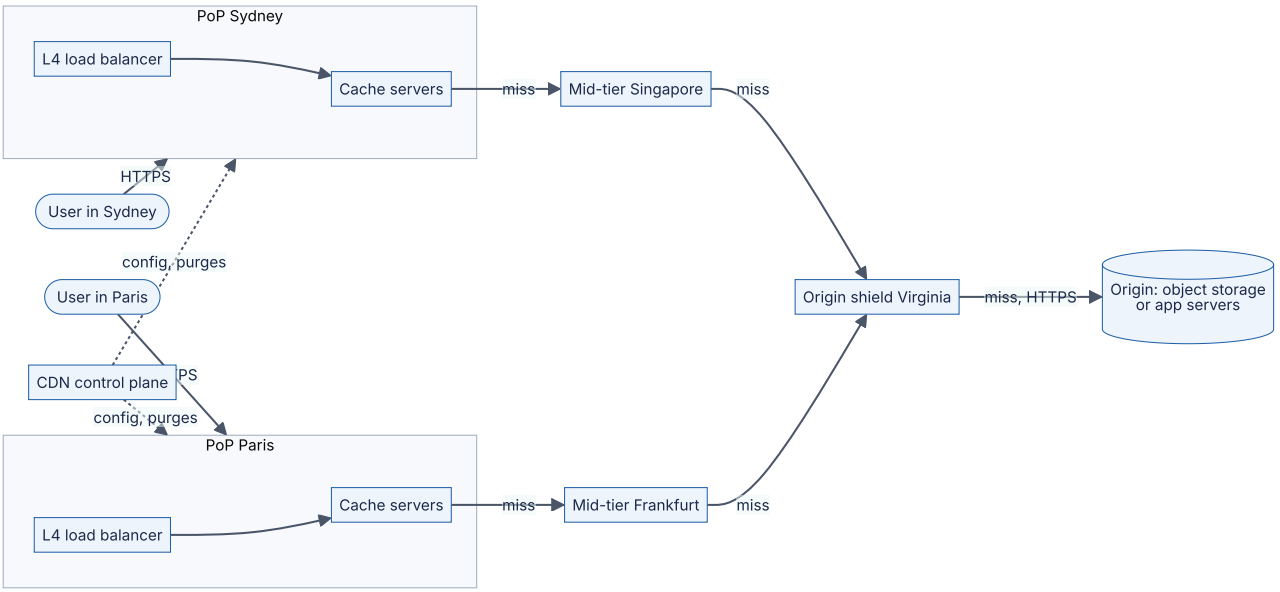
Definition — Point of presence (PoP). A CDN site in one metropolitan area or interconnection facility, containing load balancers, caching proxy servers with RAM and SSD storage, and connections to local networks. Large CDNs operate hundreds of PoPs; some also place servers inside Internet service provider (ISP) networks.

Inside a PoP, a request passes through three layers:

1. **Layer 4 load balancing.** Routers spread packets across servers with equal-cost multipath (ECMP) and a stateless L4 load balancer (§19.3, §19.8).
2. **TLS termination and HTTP processing.** A proxy (for example Varnish, NGINX, Apache Traffic Server, or a CDN's own software) decrypts the request, applies rules, and computes the cache key (§21.6.2).
3. **Cache lookup across the PoP.** Rather than store each popular object on all 100 servers, the PoP hashes the cache key to an "owner" server with consistent hashing (§14.8), so each object is stored roughly once per PoP. Consistent hashing was invented for exactly this problem [Karger et al. 1997].

The cache itself is a two-level store: hot objects in RAM and the long tail on SSD, with eviction policies of the kind described in §20.9.

Figure 21.1 — A CDN: users, PoPs, mid-tier and origin.



Reading the diagram. Users (stadiums) connect over HTTPS to the nearest PoP, where an L4 load balancer spreads connections across cache servers. A miss moves one hop inward at a time: to a regional mid-tier cache, then to an origin shield near the origin, and finally to the origin (object storage or application servers). Solid arrows are synchronous request/response paths; the dotted arrows from the control plane are asynchronous pushes of configuration and purge commands. Storage boundaries: every PoP, mid-tier and shield holds a cache (RAM and SSD); only the origin holds the authoritative copy. Failure points: a PoP outage (users are rerouted, §21.3.3), a shield outage (misses must bypass it), and the origin itself. Scaling: add servers inside a PoP, add PoPs for coverage, and add tiers to protect the origin (§21.5).

21.3.3 Request routing: DNS versus anycast

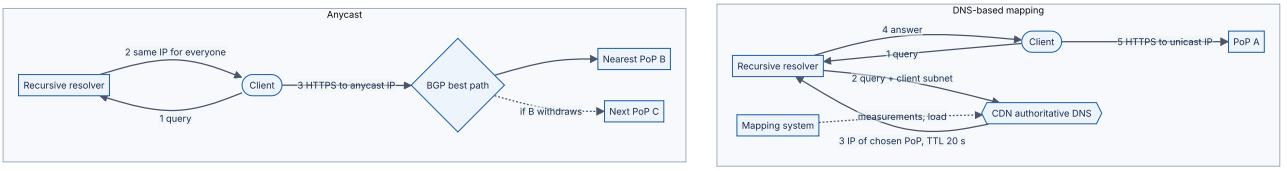
Before a user can hit a cache, something must decide *which* PoP serves them. Two mechanisms dominate, both introduced in Chapter 7.

DNS-based routing. The site's hostname is a CNAME to a CDN-controlled name (`img.photoshare.example` → `photoshare.cdn-provider.example`). The CDN's authoritative DNS answers each query with a PoP chosen by a **mapping system** from network measurements, PoP health and load, and the resolver's location. Akamai describes this design, with short TTLs so users can be re-steered quickly [Nygren et al. 2010]. The weakness from §7.6.5 applies: the DNS server sees the *resolver's* address, not the user's. The **EDNS Client Subnet** extension lets a resolver pass a truncated client prefix, such as a /24, to the authoritative server [RFC 7871]; Akamai reported substantial latency improvements for users of public resolvers after adopting it ("end-user mapping") [Chen et al. 2015].

Anycast routing. Every PoP announces the same IP prefix over BGP (§7.5.3), and routing delivers each packet to the "closest" site by BGP policy; DNS returns the same address

everywhere. Cloudflare describes serving its services from every data center over anycast [Cloudflare Docs: Network]. A PoP that withdraws its announcement disappears from routing within seconds to minutes. The weaknesses: BGP's "closest" is not always lowest latency, and load is hard to steer without changing announcements that affect many networks. A measurement study of Microsoft's anycast CDN found that anycast picked a good PoP for most clients but a noticeably worse one for a meaningful minority [Calder et al. 2015].

Figure 21.2 — Request routing with DNS mapping versus anycast.



Reading the diagram. In the DNS mode (top), steps 1–4 are synchronous DNS exchanges over UDP; the mapping system feeds the authoritative servers asynchronously (dotted). The choice is cached for the answer's TTL, so a PoP failure is corrected only as TTLs expire. In the anycast mode (bottom), BGP chooses per packet; when PoP B withdraws its route (dotted), packets flow to PoP C and in-flight TCP connections reset. No storage is involved; failure points are the resolver, the mapping system and routing instability. Scaling: add PoPs (and, for DNS, measurements).

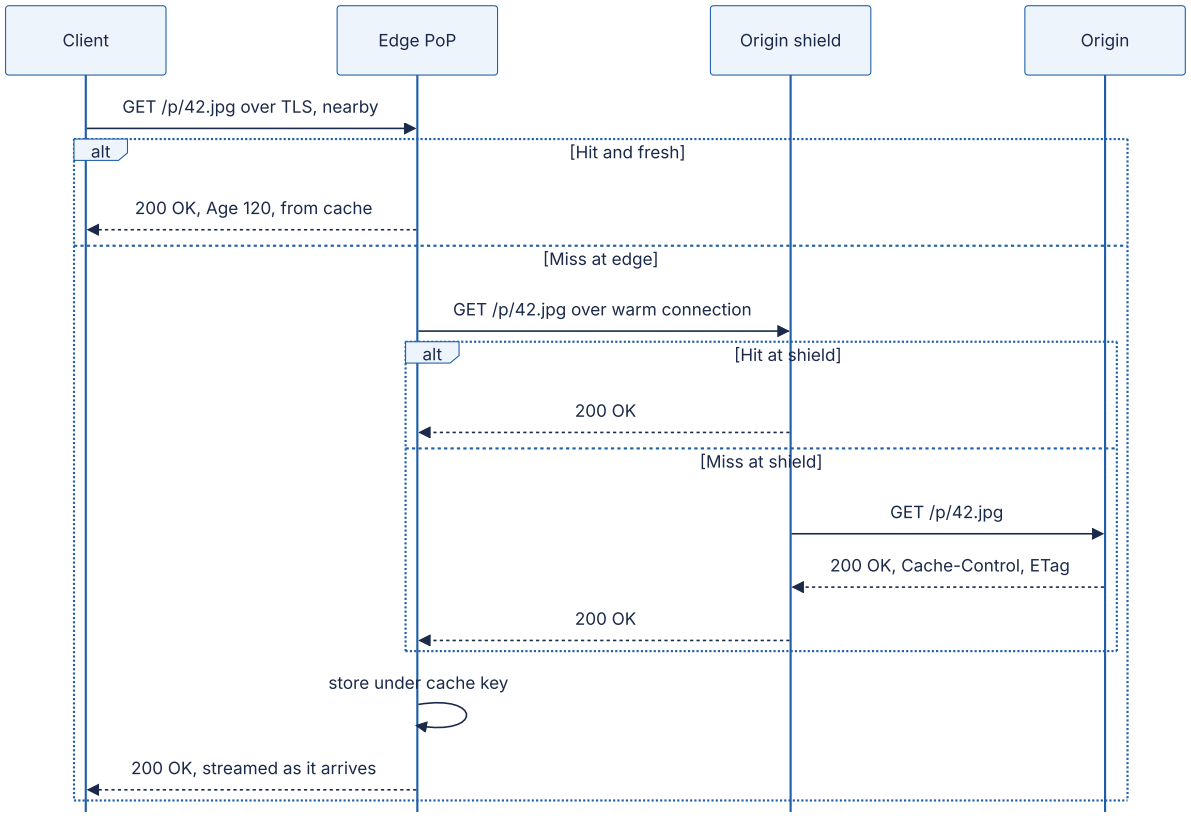
Table 21.1 — DNS-based routing versus anycast.

	DNS-based mapping	Anycast
Who chooses the PoP	CDN's mapping system, per DNS answer	BGP, per packet
Granularity	Per resolver (per client subnet with ECS)	Per network path
Load control	Fine: change answers per network	Coarse: change announcements
Failover speed	Bounded by DNS TTL and client caching	Seconds to minutes (BGP convergence)
DDoS absorption	Attack aimed at one PoP's IPs	Attack spread across all PoPs automatically
Risks	Resolver far from user; stale answers	Suboptimal paths; connection resets on route change
Choose when	You need precise steering, many small or ISP-embedded sites	You want simplicity, resilience and DDoS dispersion

Real CDNs combine the two, for example anycast for DNS servers and DNS steering for some customers. Global traffic management across your own regions uses the same tools (§19.9, Chapter 44).

21.3.4 Anatomy of a cache hit and a cache miss

Figure 21.3 — A cache hit and a cache miss at an edge PoP.



Reading the diagram. Time runs downward. TLS ends at the nearby edge. On a fresh hit the edge answers at once, adding an `Age` header (seconds spent in cache). On a miss it forwards over a long-lived connection to the shield; only a shield miss reaches the origin. All exchanges are synchronous, and responses are **streamed**: the edge forwards bytes while still receiving them. Storage boundaries: edge and shield caches, and the origin. Failure points: a slow or down shield or origin (the edge may serve stale, §21.6.1). Scaling: more tiers or bigger caches raise the hit ratio.

Three details matter in production:

- **Request collapsing.** If 500 users request the same uncached object within 50 ms, a naive edge sends 500 origin requests. CDNs **collapse** concurrent misses for one cache key into one upstream fetch, the stampede defense of §20.11 applied at the edge.
- **Negative caching.** Error responses such as `404` can be cached briefly, so a burst of requests for a missing object does not become a burst at the origin.
- **Range requests.** For large files the edge caches fixed-size chunks (say 1–2 MB), so a seek into a video need not fetch the first gigabyte.

Table 21.2 — CDN architecture failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
PoP outage	Power, fiber or software failure	Synthetic probes per PoP; traffic drop	Withdraw anycast route or remove from DNS answers	Capacity headroom in neighboring PoPs
Users mapped far away	Resolver location, BGP policy	Real-user latency by network	Adjust mapping or announcements	ECS; per-network latency monitoring
Origin overwhelmed after a purge	Every PoP misses at once	Origin RPS spike; edge 5xx	Serve stale; rate-limit fetches	Shielding, request collapsing, soft purge
Bad configuration pushed globally	One change reaches all PoPs	Error-rate jump everywhere	Roll back config	Staged rollout to canary PoPs

21.4 Edge, mid-tier and origin

21.4.1 Three tiers

DEFINITION

Definition — Cache tiers. The **edge** tier is the set of PoPs that accept user connections. A **mid-tier** (also *parent, regional* or *upper-tier* cache) is a smaller set of larger caches that edges consult on a miss. An **origin shield** is a mid-tier designated as the single path to a given origin. The **origin** is the customer's own infrastructure that holds the authoritative content.

First-principles explanation

Return to the bookshops. A town shop stocks the 2,000 titles its customers buy most; for an obscure title it phones the regional warehouse, which stocks 200,000, and only the warehouse contacts the publisher. Each tier holds more items, serves larger areas, and sees only what the tier below could not answer.

Why does this exist?

Edges must be numerous and close to users, so each one is small relative to the catalog and sees only a slice of the audience. For long-tail content (most images, most video titles, most software versions) each edge misses often. A mid-tier aggregates the misses of many edges, so an object requested once in each of 50 cities becomes popular *at the mid-tier* and is fetched from the origin once.

Table 21.3 — Roles of the cache tiers.

	Edge PoP	Mid-tier / shield	Origin
Location	Near users, often in ISPs or exchange points	Few, well-connected regions; shield near the origin	Your region(s) or cloud
Count	Tens to thousands	A handful to tens	One to a few
Cache size per site	Small relative to catalog	Large	Holds everything (authoritative)
Sees	All user requests	Edge misses	Shield misses
Main job	Latency, TLS termination, absorbing traffic	Aggregating misses, protecting origin	Correctness and freshness

The origin can be **object storage** (static assets, images, video files; §15.4), an **application tier** behind a load balancer (HTML, APIs), or a **media packager** (live video). Whatever it is, the origin should accept traffic only from the CDN: allow-list the CDN's address ranges or require a shared secret header or mutual TLS on origin pulls (§21.11), otherwise attackers bypass the cache and hit the origin directly.

INDUSTRY EXAMPLE

Akamai, Amazon CloudFront and Cloudflare tiering. Akamai's 2010 description of its platform includes **tiered distribution**: edge clusters fetch from a smaller set of well-provisioned "parent" clusters rather than from the origin, which Akamai reports substantially reduces origin load, especially for less popular content [Nygren et al. 2010]. CloudFront documents **regional edge caches** between its edge locations and the origin, and an optional **Origin Shield** layer through which all requests for an origin pass [AWS Docs: CloudFront Origin Shield]. Cloudflare documents **Tiered Cache**, in which lower-tier data centers fetch from upper-tier data centers instead of the origin [Cloudflare Docs: Tiered Cache]. *Evidence: Primary* — [Nygren et al. 2010]; [AWS Docs: CloudFront Origin Shield]; [Cloudflare Docs: Tiered Cache]

21.5 Cache hierarchy and origin shielding

21.5.1 Offload ratio

DEFINITION

Definition — Offload ratio. The fraction of traffic a CDN serves without contacting the origin. **Request offload** counts requests; **byte offload** counts bytes. Both are a cache hit ratio (§20.3) measured at the boundary between the CDN and the origin.

FORMULA

Request and byte offload

$$O_R = 1 - \frac{N_{\text{origin}}}{N_{\text{total}}} \qquad O_B = 1 - \frac{V_{\text{origin}}}{V_{\text{total}}}$$

Symbol	Meaning	Units
O_R, O_B	Request offload, byte offload	dimensionless (0–1)
N_{origin}	Requests the CDN sent to the origin	requests
N_{total}	Requests users sent to the CDN	requests
V_{origin}	Bytes the origin sent to the CDN	bytes
V_{total}	Bytes the CDN sent to users	bytes

WORKED EXAMPLE

Worked example — Two offloads for the same day. A CDN receives $N_{\text{total}} = 10^9$ requests carrying $V_{\text{total}} = 500$ TB, and sends $N_{\text{origin}} = 4 \times 10^7$ requests to the origin, which return $V_{\text{origin}} = 40$ TB.

1. $O_R = 1 - 4 \times 10^7 / 10^9 = 1 - 0.04 = 0.96$.
2. $O_B = 1 - 40/500 = 1 - 0.08 = 0.92$.

Interpretation. Misses are disproportionately large objects (long-tail videos, rarely downloaded installers), so byte offload is lower than request offload here; with other mixes it can be higher. **Engineering implication.** Origin CPU and request rate follow O_R; origin egress cost and bandwidth follow O_B. Track both, and do not quote one when the decision depends on the other.

21.5.2 Composing tiers

A request reaches the origin only if it misses at *every* tier, so the miss ratios multiply.

FORMULA

Origin fraction and origin bandwidth across tiers

$$f_{\text{origin}} = \prod_{i=1}^{k} (1 - h_i) \qquad B_{\text{origin}} = B_{\text{users}} \times (1 - O_B)$$

Symbol	Meaning	Units
h_i	Hit ratio of tier i, measured on the requests that *reach* that tier	dimensionless
k	Number of cache tiers (browser, edge, mid-tier, ...)	count
f_{origin}	Fraction of all requests that reach the origin, $1 - O_R$	dimensionless
B_{users}	Bandwidth delivered to users	Gb/s
B_{origin}	Bandwidth the origin must serve	Gb/s

WORKED EXAMPLE

Worked example — Facebook's photo-serving stack. A study of Facebook photo traffic found that browser caches served 65.5% of requests, edge caches 20.0%, the origin cache (a shield-like tier in Facebook's data centers) 4.6%, and the Haystack storage backend the remaining 9.9% [Huang et al. 2013].

1. Browser: $h_1 = 0.655$; 34.5% of requests continue.
2. Edge: $h_2 = 20.0/34.5 = 0.580$; 14.5% continue.
3. Origin cache: $h_3 = 4.6/14.5 = 0.317$.
4. Check: $(1 - 0.655)(1 - 0.580)(1 - 0.317) = 0.345 \times 0.420 \times 0.683 = 0.099$, the 9.9% that reached storage.

Now apply the bandwidth form to PhotoShare's 28 Gb/s peak: at $O_B = 0.90$, $B_{\text{origin}} = 28 \times 0.10 = 2.8$ Gb/s; at $O_B = 0.98$, only 0.56 Gb/s.

Interpretation. No single tier is spectacular (the edge hit ratio is 58%, the origin cache's 32%), yet together they keep 90% of requests away from storage. A tier with a modest hit ratio still pays off when the tier behind it is expensive. **Engineering implication.** Measure each tier's hit ratio on *its own* input, and remember that raising offload from 90% to 98% cuts origin load fivefold.

21.5.3 Origin shielding

DEFINITION

Definition — Origin shielding. Routing every cache miss for an origin through one designated cache tier (the **shield**), so that the origin sees at most one request per object per freshness period, regardless of how many edge PoPs missed.

Why does this exist?

Without a shield, each PoP is an independent client of your origin. An object that is cold everywhere is fetched once *per PoP*, and when a TTL expires, every PoP that holds it revalidates separately. With 300 PoPs, a purge of a popular object turns into up to 300 near-simultaneous origin requests, the CDN-scale version of a cache stampede (§20.11).

FORMULA

Origin fetches per object per TTL window, without and with a shield

$$F_{\text{no shield}} \approx N(1 - e^{-\lambda T/N}) \qquad F_{\text{shield}} \approx 1$$

Symbol	Meaning	Units
N	Number of edge PoPs, with requests spread evenly among them	count
λ	Global request rate for the object	requests/s
T	Freshness lifetime (TTL) of the object	s
$1 - e^{-\lambda T/N}$	Probability that a given PoP sees at least one request in T (Poisson arrivals)	dimensionless
F	Expected origin fetches for the object per window T	fetches

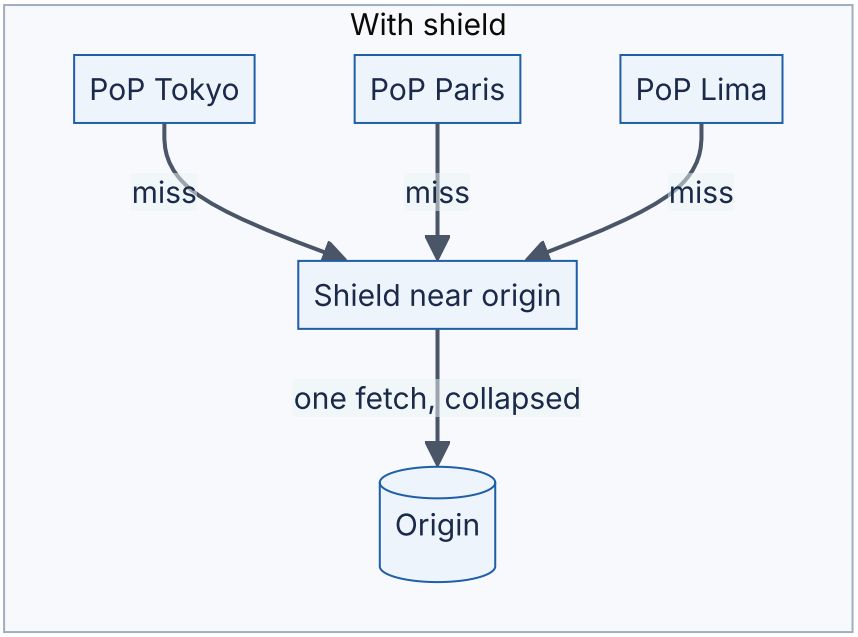
WORKED EXAMPLE

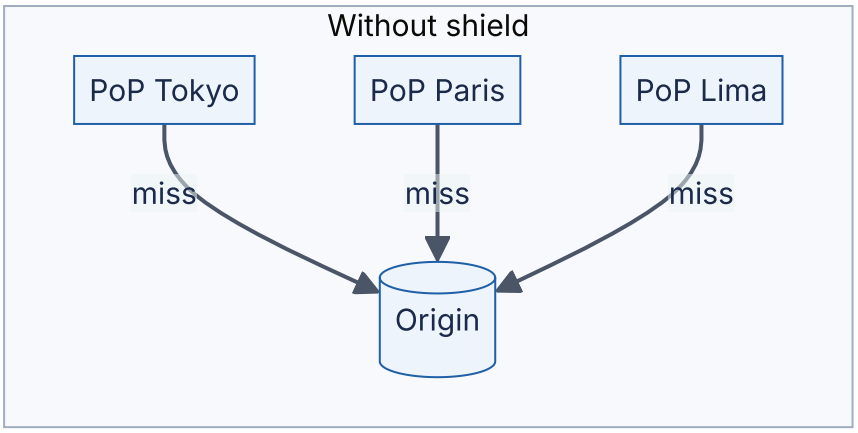
Worked example — A long-tail photo and a popular photo across 100 PoPs, TTL 1 hour.

1. Long-tail photo, $\lambda = 0.01$ req/s: $\lambda T = 36$ requests per hour; $\lambda T/N = 0.36$; $e^{-0.36} = 0.698$.
2. $F_{\text{no shield}} = 100 \times (1 - 0.698) = 30.2$ origin fetches per hour, out of 36 requests: the edges' hit ratio for this object is $1 - 30.2/36 = 16\%$.
3. Popular photo, $\lambda = 1$ req/s: $\lambda T/N = 36$, so $e^{-36} \approx 0$ and $F_{\text{no shield}} \approx 100$ fetches per hour out of 3,600 requests (97.2% edge hit ratio).
4. With a shield that retains both objects, $F_{\text{shield}} = 1$ per hour in each case.

Interpretation. The shield cuts origin fetches 30-fold for the long-tail photo and 100-fold for the popular one. **Engineering implication.** Shielding matters most when there are many PoPs, a long tail of content, short TTLs, or frequent purges; it matters little for a tiny catalog of very hot objects.

Figure 21.4 — Origin traffic without and with a shield.





Reading the diagram. On the left, three PoPs (rectangles) each send their miss for the same object straight to the origin (cylinder), so the origin serves it three times, or 300 times with 300 PoPs. On the right, every miss goes to a shield located close to the origin; the shield collapses concurrent requests and fetches once. All arrows are synchronous HTTPS. Storage boundaries: every PoP and the shield are caches; the origin is authoritative. Failure points: the shield is a concentration point, so it is itself a cluster of servers, and edges must be able to bypass it (or use a backup shield) if it fails. Scaling: shield capacity scales by consistent hashing objects across shield servers; a second shield region adds redundancy.

21.5.4 Origin egress and cost

FORMULA

Monthly delivery cost

$$C = V \times p_{\text{cdn}} + V \times (1 - O_B) \times p_{\text{origin}}$$

Symbol	Meaning	Units
V	Bytes delivered to users per month	GB
p_{cdn}	CDN delivery price	$/GB
p_{origin}	Origin egress price (cloud Internet egress)	$/GB
O_B	Byte offload	dimensionless
C	Monthly delivery cost (excluding request fees and commitments)	$

WORKED EXAMPLE

Worked example — PhotoShare's image bill, with and without a shield. Prices are **illustrative, as of 2026**: cloud Internet egress list prices at petabyte-scale tiers are on the order of $0.05/GB, and large customers negotiate CDN delivery far below list prices; assume $p_{\text{cdn}} = \$0.01$/GB and $p_{\text{origin}} = \$0.05$/GB.

1. Volume: 9.3 Gb/s average (§1.9) $= 1.16$ GB/s; $\times\ 2,592,000$ s per 30-day month $= 3.01 \times 10^6$ GB, about 3 PB.
2. CDN delivery: $3.0 \times 10^6 \times 0.01 = \$30,000$.
3. Origin egress without shield ($O_B = 0.90$): 3.0×10^5 GB $\times\ 0.05 = \$15,000$.
4. Origin egress with shield ($O_B = 0.98$): 6.0×10^4 GB $\times\ 0.05 = \$3,000$.
5. Without any CDN, the origin would serve everything: $3.0 \times 10^6 \times 0.05 = \$150,000$, and would need about 28 Gb/s of peak serving capacity.

Interpretation. The shield saves $12,000 per month ($144,000 per year) on egress and cuts peak origin bandwidth from 2.8 to 0.56 Gb/s. **Engineering implication.** Check whether your cloud waives origin-to-CDN egress for its own CDN (several do), include shield request fees, and remember that offload also shrinks the origin fleet, which is often the larger saving.

21.5.5 Simulating shielding

The model above assumes caches never evict. The program below drops that assumption: 20 edge PoPs, each a byte-capacity LRU cache holding 3% of a 50,000-object catalog, receive one million Zipf-distributed requests (exponent 1.0, the skew typical of web content), with and without a shield.

```
        # Two-tier CDN simulation: how much traffic reaches the origin with and
        # without an origin shield. Edge PoPs and the shield are byte-capacity LRU
caches;
        # object popularity follows a Zipf distribution.
        import random
        from collections import OrderedDict


        class LRUCache:
            """Least-recently-used cache whose capacity is measured in bytes."""

            def __init__(self, capacity_bytes):
                self.capacity = capacity_bytes
                self.used = 0
                self.items = OrderedDict()          # key -> size, oldest first

            def get(self, key):
                if key in self.items:
                    self.items.move_to_end(key)     # mark as most recently used
                    return True
                return False

            def put(self, key, size):
                if size > self.capacity:
                    return                          # too big to cache at all
                if key in self.items:
                    self.used -= self.items.pop(key)
                while self.used + size > self.capacity:
                    _, old_size = self.items.popitem(last=False)   # evict LRU
                    self.used -= old_size
                self.items[key] = size
                self.used += size


        def make_workload(n_objects, n_requests, n_pops, alpha, seed):
            """Return object sizes and a trace of (pop, object) requests."""
            rng = random.Random(seed)
            sizes = [rng.choice([20_000, 50_000, 200_000, 1_000_000]) for _ in
range(n_objects)]
            cum, total = [], 0.0
            for rank in range(1, n_objects + 1):    # Zipf: weight of rank k is
1/k^alpha
                total += 1.0 / rank ** alpha
                cum.append(total)
            objs = rng.choices(range(n_objects), cum_weights=cum, k=n_requests)
            pops = [rng.randrange(n_pops) for _ in range(n_requests)]
            return sizes, list(zip(pops, objs))


        def simulate(sizes, trace, n_pops, edge_bytes, shield_bytes=None):
            """Replay the trace. shield_bytes=None means edges go straight to origin."""
            edges = [LRUCache(edge_bytes) for _ in range(n_pops)]
            shield = LRUCache(shield_bytes) if shield_bytes else None
            s = dict(requests=0, edge_hits=0, shield_hits=0, origin_fetches=0,
                     bytes_served=0, origin_bytes=0)
            for pop, obj in trace:
```

```
                size = sizes[obj]
                s["requests"] += 1
                s["bytes_served"] += size
                if edges[pop].get(obj):
                    s["edge_hits"] += 1
                    continue
                if shield is not None and shield.get(obj):
                    s["shield_hits"] += 1
                else:                                  # fetch from origin
                    s["origin_fetches"] += 1
                    s["origin_bytes"] += size
                    if shield is not None:
                        shield.put(obj, size)
                edges[pop].put(obj, size)
            s["request_offload"] = 1 - s["origin_fetches"] / s["requests"]
            s["byte_offload"] = 1 - s["origin_bytes"] / s["bytes_served"]
            return s


        if __name__ == "__main__":
            N_OBJ, N_REQ, N_POPS = 50_000, 1_000_000, 20
            sizes, trace = make_workload(N_OBJ, N_REQ, N_POPS, alpha=1.0, seed=21)
            catalog = sum(sizes)
            edge = int(0.03 * catalog)                 # each PoP holds 3% of the catalog
            print(f"catalog {catalog / 1e9:.1f} GB, {N_POPS} PoPs x {edge / 1e9:.2f} GB
each")
            for label, shield in [("no shield", None),
                                  ("shield 10% of catalog", int(0.10 * catalog)),
                                  ("shield 30% of catalog", int(0.30 * catalog))]:
                r = simulate(sizes, trace, N_POPS, edge, shield)
                print(f"{label:<22} edge hit {r['edge_hits'] / r['requests']:6.1%}  "
                      f"origin fetches {r['origin_fetches']:>7,}  "
                      f"request offload {r['request_offload']:6.1%}  "
                      f"byte offload {r['byte_offload']:6.1%}  "
                      f"origin egress {r['origin_bytes'] / 1e9:5.1f} GB")
```

Output (Python 3.13, about 3 s):

```
catalog 15.8 GB, 20 PoPs x 0.47 GB each
no shield              edge hit  58.3%  origin fetches 417,280  request offload  58.3%
byte offload  62.4%  origin egress 132.4 GB
shield 10% of catalog  edge hit  58.3%  origin fetches 278,623  request offload  72.1%
byte offload  74.9%  origin egress  88.2 GB
shield 30% of catalog  edge hit  58.3%  origin fetches 156,952  request offload  84.3%
byte offload  85.9%  origin egress  49.5 GB
```

What to notice:

- The edge hit ratio is identical in all three runs: the shield does not change what edges see, only where their misses go.
- A shield holding 30% of the catalog absorbs $1 - 156,952/417,280 = 62\%$ of edge misses and cuts origin egress from 132.4 GB to 49.5 GB. It aggregates 20 PoPs' long tails, so objects that are rare at each PoP are popular at the shield.

- `put` is byte-based, so one 1 MB object evicts fifty 20 KB ones; size-aware policies (§20.9) can do better.
- Tests in `code/ch21/test_listing_01.py` check LRU eviction, that shield hits plus origin fetches equal edge misses, that a shield never increases origin fetches, and that with unlimited caches the origin sees one fetch per object with a shield but one per (PoP, object) pair without.

Advantages, disadvantages and failure scenarios

A shield adds a hop (cheap if it sits near the origin) and concentrates traffic, so it must be redundant. Place it closest to the origin, not to users, with one shield per origin region.

COMMON MISTAKE

"Our CDN hit ratio is 95%, so the origin is safe." A dashboard hit ratio usually counts edge hits only, often by request. The origin cares about requests and bytes that cross the CDN–origin boundary, including revalidations and purge-driven refetches. Measure offload at the origin's own access logs.

21.6 Cache keys, HTTP caching headers and invalidation

21.6.1 Freshness and validation under RFC 9111

A CDN is a **shared cache** in the vocabulary of RFC 9111, which defines HTTP caching: it stores responses and reuses them for many users. The origin controls that reuse with response headers.

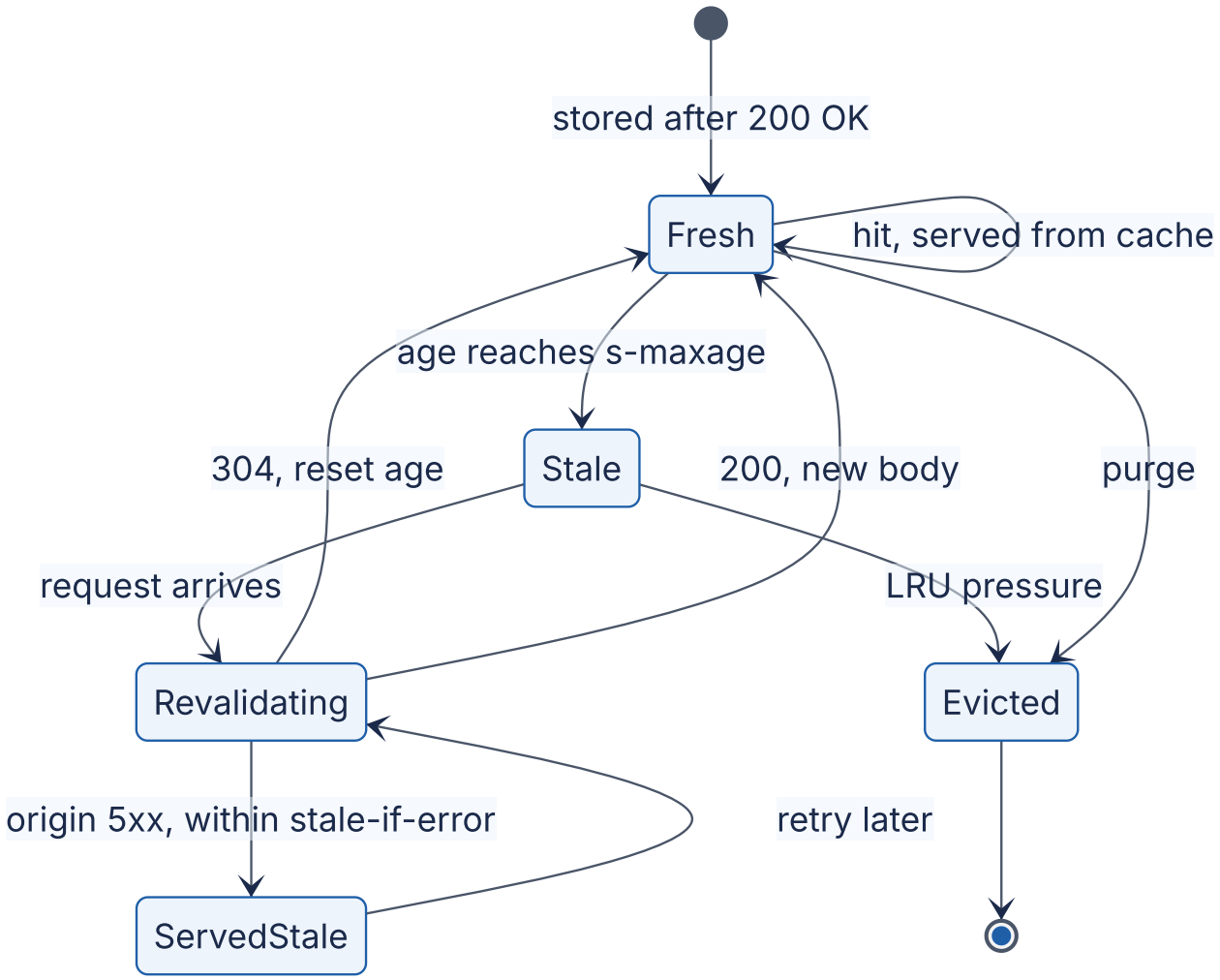
DEFINITION

Definition — Freshness and validation. A stored response is **fresh** while its age is below its **freshness lifetime** and may then be served without contacting the origin; once **stale**, it must normally be **revalidated**, by sending a conditional request (`If-None-Match` with the stored `ETag` , or `If-Modified-Since`), to which the origin answers `304 Not Modified` (keep using the copy) or `200 OK` with a new body [RFC 9111; RFC 9110].

Table 21.4 — Caching directives a CDN designer uses.

Header or directive	Meaning	Typical use
`max-age=N`	Fresh for N seconds in any cache	Browser and CDN lifetime
`s-maxage=N`	Overrides `max-age` in shared caches only	CDN longer than browsers
`no-cache`	May store, but must revalidate before every use	HTML that must be current
`no-store`	Must not store at all	Sensitive responses
`private`	Only the user's own browser may store it	Personalized pages
`immutable`	Will never change during freshness; skip reload revalidation [RFC 8246]	Versioned assets
`stale-while-revalidate=N`	Serve stale for up to N s while refreshing in background [RFC 5861]	Hide origin latency
`stale-if-error=N`	Serve stale for up to N s if the origin errors [RFC 5861]	Survive origin outages
`ETag` / `If-None-Match`	Validator and conditional request; `304` if unchanged	Cheap revalidation
`Age`	Seconds the response has spent in caches	Lets browsers subtract CDN time
`CDN-Cache-Control`	Same directives, aimed only at CDNs [RFC 9213]	Separate CDN and browser policy

Figure 21.5 — Life of a cached response at the edge.



Reading the diagram. States describe one stored response at one edge. A **Fresh** response is served directly until its age reaches the shared-cache lifetime. A request for a **Stale** response triggers revalidation, synchronously by default or in the background within the `stale-while-revalidate` window (the user gets the stale copy at once). A `304` or new `200` makes it fresh; an origin error within the `stale-if-error` window means stale content is served instead of an error. Purges and eviction remove it. The storage boundary is the edge cache; the failure point is the origin during revalidation; every PoP runs this machine independently, which is why shields matter (§21.5.3).

FORMULA

Freshness test in a shared cache

$$\text{fresh} \iff a < L, \qquad L = \begin{cases} \texttt{s-maxage} & \text{if present} \\ \texttt{max-age} & \text{otherwise} \end{cases}$$

Symbol	Meaning	Units
a	Current age of the stored response, including time spent in upstream caches (`Age`)	s
L	Freshness lifetime for a shared cache (RFC 9111 §4.2.1; `Expires` or heuristics apply if neither directive is present)	s

WORKED EXAMPLE

Worked example — One header, three behaviors. The origin sends `Cache-Control: max-age=60, s-maxage=600, stale-while-revalidate=30, stale-if-error=86400` with `ETag: "v7"` .

1. **Edge:** fresh for 600 s. Between ages 600 and 630 s it serves the stale copy and revalidates in the background. A `304` costs a few hundred bytes instead of resending, say, a 2 MB body.
2. **Browser:** uses `max-age=60` , but the edge forwards `Age` . A copy fetched from the edge at age 50 s is fresh in the browser for only $60 - 50 = 10$ s.
3. **Origin outage:** for up to 86,400 s after a copy goes stale, the edge may serve it instead of a `5xx` .

Interpretation. The CDN holds content ten times longer than browsers, because the CDN can be purged and browsers cannot. **Engineering implication.** Give browsers short lifetimes for anything that might need recall, give the CDN long lifetimes plus a purge path, and always add `stale-if-error` : it turns many origin outages into non-events.

21.6.2 Cache keys

DEFINITION

Definition — Cache key. The value a cache uses to decide whether two requests may receive the same stored response. By default it is the method plus the full URL (scheme, host, path and query string), extended by any request headers listed in the response's `Vary` header.

Two requests that should get the same bytes but differ in their key produce two cache entries and two origin fetches: the hit ratio **fragments**. Two requests that should get *different* bytes but share a key leak one user's response to another. Cache-key design steers between these.

- **Normalize.** Lowercase the host, sort query parameters, drop parameters that do not change the response (tracking tags such as `utm_source`, session IDs). An allow-list of meaningful parameters is safer than a deny-list.
- **Tame `Vary`.** `Vary: Accept-Encoding` is necessary for compressed text, but browsers send dozens of distinct `Accept-Encoding` strings; normalize them to a few values (`br`, `gzip`, `identity`). `Vary: User-Agent` creates thousands of variants and effectively disables caching; derive a small device class at the edge instead.
- **Strip cookies.** Cookies are usually ignored or excluded from the key for static assets; a response that depends on a cookie must be `private` or keyed on a small derived value (for example a language), never on the raw cookie.

```
        # Cache-key normalization: map many equivalent requests to one cache key.
        from urllib.parse import parse_qsl, urlencode, urlsplit

        KEEP_PARAMS = {"v", "lang"}   # allow-list: only params that change the bytes


        def cache_key(url, accept_encoding=""):
            parts = urlsplit(url)
            host = parts.hostname.lower()             # hostnames are case-insensitive
            path = parts.path or "/"                  # paths are case-sensitive: keep
            params = sorted((k.lower(), v) for k, v in parse_qsl(parts.query)
                            if k.lower() in KEEP_PARAMS)
            # Collapse Accept-Encoding to the few variants the edge actually stores.
            enc = accept_encoding.lower()
            enc = "br" if "br" in enc else "gzip" if "gzip" in enc else "identity"
            query = urlencode(params)
            return f"{host}{path}" + (f"?{query}" if query else "") + f"|{enc}"


        if __name__ == "__main__":
            requests = [
                ("https://CDN.example.com/js/app.js?v=7&lang=en", "gzip, deflate, br"),
                ("https://cdn.example.com/js/app.js?lang=en&v=7&utm_source=mail",
"br;q=1.0, gzip;q=0.8"),
                ("https://cdn.example.com/js/app.js?LANG=en&v=7&sessionid=abc123",
"br"),
                ("https://cdn.example.com/js/app.js?v=7&lang=en", "gzip, deflate"),
                ("https://cdn.example.com/js/app.js?v=7&lang=en", ""),
            ]
            keys = [cache_key(u, ae) for u, ae in requests]
            for k in keys:
                print(k)
            print("distinct keys:", len(set(keys)), "of", len(keys), "requests")
```

Output:

```
cdn.example.com/js/app.js?lang=en&v=7|br
cdn.example.com/js/app.js?lang=en&v=7|br
cdn.example.com/js/app.js?lang=en&v=7|br
cdn.example.com/js/app.js?lang=en&v=7|gzip
cdn.example.com/js/app.js?lang=en&v=7|identity
distinct keys: 3 of 5 requests
```

What to notice: five raw requests collapse to three keys, one per encoding the edge must store; parameter order, host case, tracking tags and session IDs no longer fragment the cache; the path keeps its case because URL paths are case-sensitive.

WARNING / TRADE-OFF

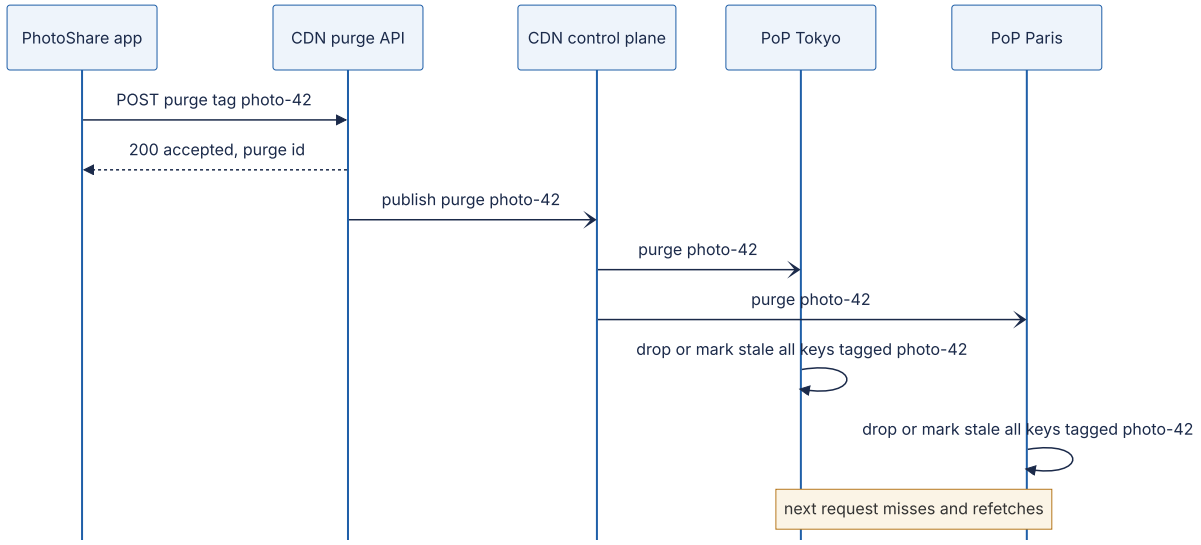
Unkeyed inputs are a security boundary. In **web cache poisoning**, an attacker sends a request whose *unkeyed* header (for example `X-Forwarded-Host`) changes the response, so the poisoned response is stored under a normal key and served to everyone [Kettle 2018]. In **web cache deception**, an attacker lures a victim to `/account/profile.css`; a lenient origin returns the private profile page, and a CDN that caches by extension stores it publicly [Gil 2017]. Defenses: key on every input that affects the response, cache by explicit `Cache-Control` rather than by file extension, and mark personalized responses `private`.

21.6.3 Invalidation and purge

Expiry is passive. When content must change *now* (a corrected price, a deleted photo, a legal takedown), you need active invalidation (§20.10). CDNs offer three tools.

1. **Purge by URL.** An API call removes one cache key from every PoP. Simple, but you must know every URL and variant affected.
2. **Purge by surrogate key (tag).** The origin labels responses with tags, for example `Surrogate-Key: user-881 photo-42` on Fastly or `Cache-Tag` on Cloudflare; one API call then invalidates every response carrying a tag, across all variants and pages [Fastly Docs: Purging; Cloudflare Docs: Purge by tag]. A **soft purge** marks matching objects stale instead of deleting them, so `stale-while-revalidate` and `stale-if-error` still protect the origin.
3. **Versioned URLs.** Put a content hash or version in the URL (`app.3f9a1c.js`), cache it for a year with `immutable`, and "invalidate" by publishing new HTML that references a new URL. Nothing is purged; the old object simply stops being requested.

Figure 21.6 — Tag-based purge propagation.



Reading the diagram. The application calls the purge API synchronously; propagation to PoPs is asynchronous (open arrowheads), fanned out by the control plane. Each PoP keeps an index from tag to cache keys and applies the purge locally. Storage boundaries are the per-PoP caches. Failure points: a PoP partitioned from the control plane keeps serving old content until it reconnects or the TTL expires, so purge is fast in practice but not linearizable; mass refetching can stampede the origin unless a shield or soft purge is used. Scaling: purge is a broadcast whose cost grows with PoP count, not traffic.

INDUSTRY EXAMPLE

Fastly Instant Purge. Fastly built its CDN around fast invalidation of surrogate keys, broadcast to all PoPs. A Fastly developer advocate has written that a surrogate-key purge starts taking effect within a few milliseconds and completes at every PoP worldwide in around 150 ms, while "purge all" uses a different, slower mechanism [Betts, Fastly Blog]. Fast purge lets customers cache content once considered dynamic, such as news articles and API responses, with long TTLs and purge on change. *Evidence: Primary* — [Betts, Fastly Blog]; [Fastly Docs: Purging]

Table 21.5 — Invalidation methods compared.

	URL purge	Tag purge	Versioned URL
How it works	Delete one key everywhere	Delete all keys with a tag	New URL; old one ages out
Advantages	Simple, universal	One call covers many pages	No purge needed; perfect browser caching
Disadvantages	Must enumerate variants	Tagging discipline; vendor-specific	Needs a reference that changes (HTML, manifest)
Origin impact	Refetch of that key	Refetch of every tagged key	None until new URL is requested
Choose when	Rare, specific fixes	Content shown on many pages (a photo in feeds)	Static assets (JS, CSS, images by hash)

RECOMMENDATION

A default policy. Version every static asset and cache it for a year with `immutable`. Serve HTML and API responses with a short browser lifetime, a longer `s-maxage`, `stale-while-revalidate` and `stale-if-error`, and tag them by the entities they display so a write can purge exactly what changed. For PhotoShare, a deleted photo is purged by tag `photo-42`, removing it from every feed page that embedded it.

21.7 Static versus dynamic content

21.7.1 What can be cached

Table 21.6 — Content classes and CDN treatment.

Class	Examples	Cacheable?	CDN treatment
Static, versioned	JS, CSS, fonts, images by hash	Yes, for a year	`immutable`, versioned URLs
Static, mutable	Photos, product images, video segments	Yes, hours to days	Long `s-maxage` plus tag purge
Shared dynamic	Home page, article, public API list	Yes, seconds to minutes	**Micro-caching** (1–10 s), `stale-while-revalidate`, request collapsing
Personalized	Feed, cart, account page	Mostly no	Cache fragments (ESI), or accelerate only
Writes	`POST` upload, checkout	No	Dynamic acceleration, WAF, rate limits

Micro-caching is underrated: a shared page requested 2,000 times per second and cached for just 1 s sends at most one request per second per cache to the origin, a 99.95% offload at a cost of one second of staleness.

21.7.2 Dynamic site acceleration

DEFINITION

Definition — Dynamic site acceleration (DSA). Routing uncacheable requests through the CDN anyway, so that TCP and TLS terminate near the user and the edge forwards each request to the origin over pre-established, tuned, long-lived connections, sometimes along an overlay path chosen from measurements.

The insight is that most of a dynamic request's latency is handshakes and slow start on a long path, not origin processing. Akamai's platform paper describes this suite: persistent connections between edge and origin, transport tuning, and overlay routing that picks a faster path through Akamai's own servers than BGP's default [Nygren et al. 2010].

FORMULA

Time to first byte, direct versus through an edge

$$T_{\text{direct}} = (n_{\text{hs}} + 1)\,\text{RTT}_{uo} + t_{\text{app}} \qquad T_{\text{edge}} = (n_{\text{hs}} + 1)\,\text{RTT}_{ue} + \text{RTT}_{eo} + t_{\text{app}}$$

Symbol	Meaning	Units
n_{hs}	Client handshake round trips (2 for TCP + TLS 1.3, 0 if reused)	round trips
RTT_{uo}	Round trip, user to origin	ms
RTT_{ue}	Round trip, user to edge	ms
RTT_{eo}	Round trip, edge to origin over a warm, already-open connection	ms
t_{app}	Origin processing time	ms

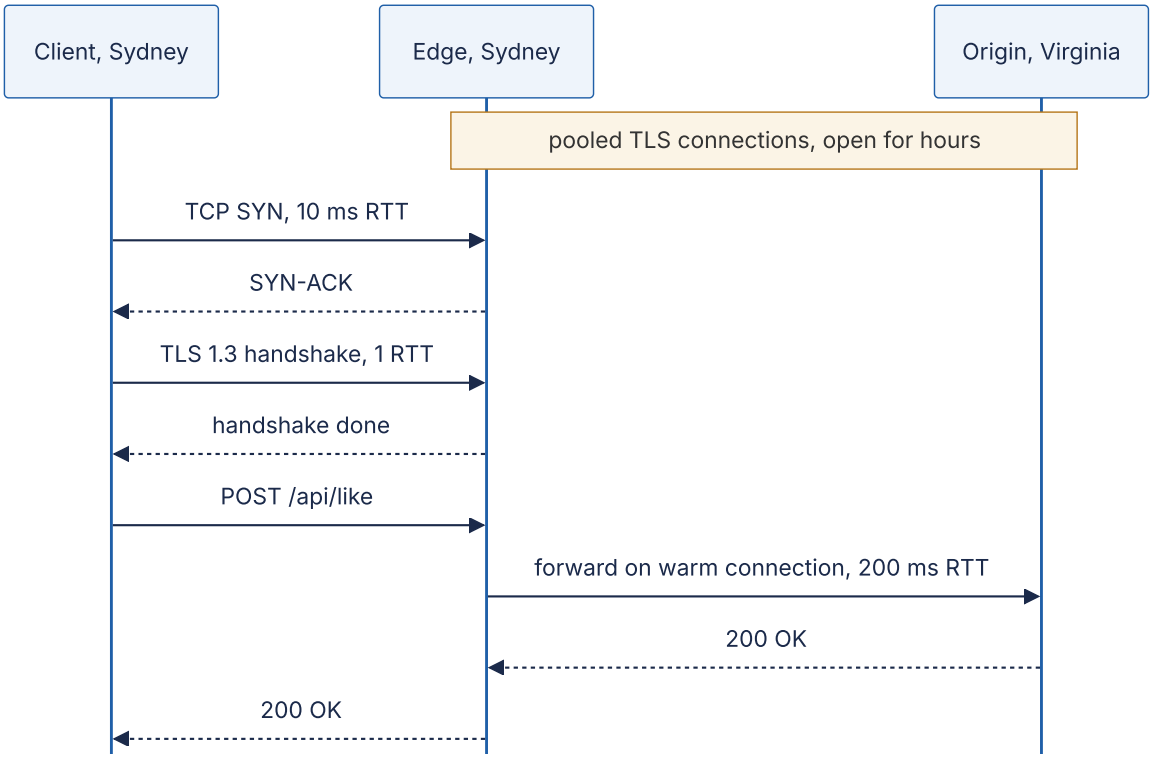
WORKED EXAMPLE

Worked example — A Sydney user calling a Virginia API. $\text{RTT}_{uo} = \text{RTT}_{eo} = 200$ ms, $\text{RTT}_{ue} = 10$ ms, $t_{\text{app}} = 50$ ms.

1. New connection, direct: $(2 + 1) \times 200 + 50 = 650$ ms.
2. New connection, via edge: $(2 + 1) \times 10 + 200 + 50 = 280$ ms, saving 370 ms (57%).
3. Reused connection ($n_{\text{hs}} = 0$), direct: $200 + 50 = 250$ ms; via edge: $10 + 200 + 50 = 260$ ms.

Interpretation. The edge wins on new connections and loses about one user-edge RTT when the client already holds a warm connection. **Engineering implication.** DSA helps most for users far from the origin making few requests per connection (mobile apps, first page loads). Large responses also gain because the edge–origin connection's congestion window is already open, so slow start is not repeated.

Figure 21.7 — Dynamic site acceleration.



Reading the diagram. Time runs downward. The client's handshakes are short because the edge is 10 ms away; the edge already holds encrypted, warmed-up connections to the origin (the note), so the request crosses the ocean exactly once each way. Every exchange is synchronous; nothing is cached, so the storage boundary is the origin alone. Failure points: the edge–origin pool (stale connections must be health-checked) and the origin itself. Scaling: the pool multiplexes many users' requests over few connections (HTTP/2), reducing the origin's connection count, a benefit of its own (§7.16).

21.7.3 Edge-side includes

A personalized page is often 95% shared content plus a small personal fragment. **Edge-side includes (ESI)**, a markup language specified in 2001 by Akamai and others, lets the origin return a cacheable page template containing include tags; the edge fetches each fragment separately, with its own cache policy, and assembles the page [Tsimelzon et al. 2001]:

```
<html><body>
  <esi:include src="/fragments/header?user=$(HTTP_COOKIE{uid})"/>  <!-- private, un-
cached -->
  <esi:include src="/fragments/trending" />                         <!-- cached 60 s -->
</body></html>
```

ESI is supported by several CDNs and by Varnish in subsets; today the same composition is often written as edge compute code (§21.9) or done in the browser.

21.8 Video delivery

Video dominates Internet traffic by volume, and it is a CDN's ideal workload: large, immutable, highly shared files. Two ideas make it work: cut the video into small HTTP objects so ordinary caches can serve it, and let the player choose quality continuously.

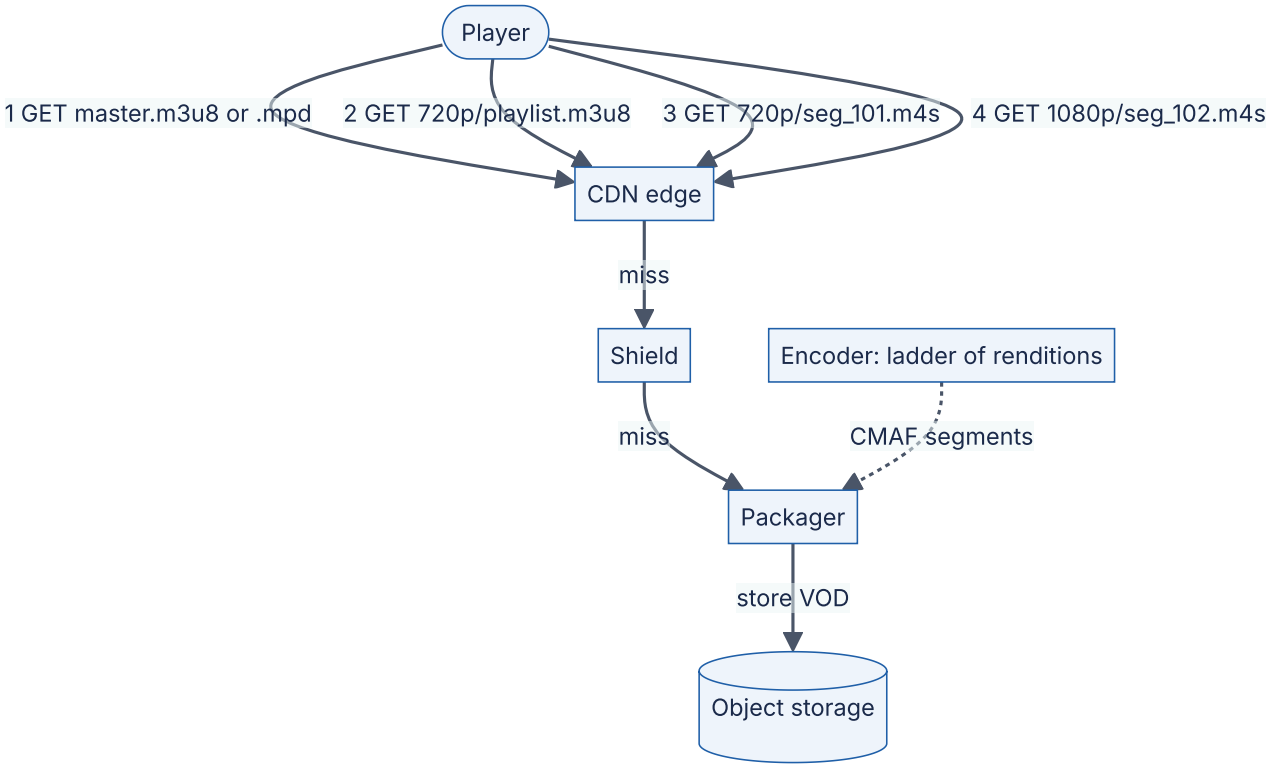
21.8.1 Segments, manifests and the bitrate ladder

> DEFINITION
>
> **Definition — HTTP adaptive streaming.** Video is encoded at several quality levels (a **bitrate ladder**), each cut into **segments** of a few seconds stored as ordinary files. A **manifest** lists the levels and segment URLs. The player downloads the manifest, then fetches segments one by one over HTTP, choosing a level for each segment.

Two formats dominate. **HLS** (HTTP Live Streaming), created by Apple and published as RFC 8216, uses text playlists (`.m3u8`): a multivariant (master) playlist listing renditions and a media playlist per rendition listing segments [RFC 8216]. **MPEG-DASH**, the international standard ISO/IEC 23009-1, uses an XML **Media Presentation Description** (MPD) [ISO/IEC 23009-1]. Historically HLS used MPEG-2 transport-stream segments and DASH used fragmented MP4, so a service supporting both stored everything twice. **CMAF** (Common Media Application Format, ISO/IEC 23000-19) defines one fragmented-MP4 segment format that both manifests can reference, halving storage and doubling cache efficiency.

```
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=3300000,RESOLUTION=1280x720,CODECS="avc1.64001f,mp4a.40.2"
720p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=6600000,RESOLUTION=1920x1080,CODECS="avc1.640028,mp4a.40.2"
1080p/playlist.m3u8
```

Figure 21.8 — HLS and DASH structure on a CDN.



Reading the diagram. The player first fetches the master manifest, then a media playlist, then segments; step 4 switches to 1080p for the next segment, which is just a different URL. All player requests are synchronous HTTP GETs to the edge; misses go through the shield to the packager or to object storage. The encoder produces renditions asynchronously (dotted) as CMAF segments. Storage boundaries: edge and shield caches, and object storage for video on demand (VOD). Failure points: for live video, the packager and encoder (a missing segment stalls every viewer); for VOD, cold long-tail titles. Scaling: segments are immutable and cacheable for a long time; live media playlists change every segment and get TTLs of about half a segment duration.

21.8.2 Bandwidth arithmetic

FORMULA

Edge egress and storage for a bitrate ladder

$$B_{\text{edge}} = V \times \sum_r s_r b_r \qquad B_{\text{origin,live}} \approx M \times \sum_r b_r \qquad S_{\text{title}} = \frac{D}{8} \sum_r b_r$$

Symbol	Meaning	Units
V	Concurrent viewers	viewers
b_r	Bitrate of rendition r	Mb/s
s_r	Share of viewers on rendition r ($\sum s_r = 1$)	dimensionless
M	Number of shields (or PoPs, if un-shielded) fetching each live rendition	count
D	Title duration	s
S_{title}	Stored size of one encoding of the full ladder	Mb ÷ 8 = MB

WORKED EXAMPLE

Worked example — A live match for two million viewers. Ladder: 0.4, 0.8, 1.5, 3.0, 4.5, 6.0 Mb/s (sum 16.2 Mb/s). Viewer shares: 5% at 0.8, 15% at 1.5, 30% at 3.0, 30% at 4.5, 20% at 6.0.

1. Mean bitrate: $0.04 + 0.225 + 0.9 + 1.35 + 1.2 = 3.715$ Mb/s.
2. Edge egress: $2 \times 10^6 \times 3.715 = 7.43 \times 10^6$ Mb/s $= 7.43$ Tb/s.
3. Origin egress with 10 shields: $10 \times 16.2 = 162$ Mb/s. Without shields, 200 PoPs each pull every rendition: $200 \times 16.2 = 3,240$ Mb/s.
4. Byte offload with shields: $1 - 162/(7.43 \times 10^6) = 99.998\%$.
5. Storage for a 2-hour VOD title: $7,200 \times 16.2/8 = 14,580$ MB ≈ 14.6 GB per codec.

Interpretation. The origin's job is trivial; the hard part is 7.4 Tb/s of edge capacity in the right places, which is why large live events are often spread across several CDNs. **Engineering implication.** Plan live events from the edge inward (capacity per metro and per ISP), and shield so the packager is never exposed to PoP count.

21.8.3 Adaptive bitrate algorithms

The player's **adaptive bitrate (ABR)** algorithm chooses the rendition for each segment. Too high and the buffer drains and video **rebuffers** (freezes); too low and quality suffers

needlessly; too jumpy and viewers notice switches.

- **Throughput-based.** Estimate bandwidth from recent segment downloads, typically a harmonic mean, which discounts outliers that would inflate an arithmetic mean, and choose the highest rendition below a safety fraction of it [Jiang et al. 2012].
- **Buffer-based (BBA).** Ignore the estimate in steady state and choose the rate from the buffer level: the minimum rate below a **reservoir**, the maximum above reservoir plus **cushion**, and linearly in between [Huang et al. 2014].
- **Hybrids and learning.** Model-predictive control combines both signals [Yin et al. 2015]; BOLA derives a buffer-based rule from utility maximization and is used in the open-source dash.js player [Spiteri et al. 2016]; Pensieve learns a policy with reinforcement learning [Mao et al. 2017].

FORMULA

Throughput rule and buffer-based rate map

$$\hat{C} = \frac{n}{\sum_{i=1}^{n} 1/c_i}, \quad R_{\text{tput}} = \max\{b_r \le \beta \hat{C}\} \qquad R_{\text{BBA}}(x) = \begin{cases} b_{\min} & x \le x_r \\ b_{\min} + \frac{x - x_r}{x_c}(b_{\max} - b_{\min}) & x_r < x < x_r + x_c \\ b_{\max} & x \ge x_r + x_c \end{cases}$$

Symbol	Meaning	Units
c_i	Measured throughput of the i-th most recent segment download	Mb/s
n	Number of recent segments averaged	count
$\hat{C}$	Harmonic-mean throughput estimate	Mb/s
β	Safety factor (e.g. 0.8)	dimensionless
x	Current buffer level	s
x_r, x_c	Reservoir and cushion	s
$b_{\min}$, $b_{\max}$	Lowest and highest ladder rates; the BBA result is rounded down to a ladder rung	Mb/s

WORKED EXAMPLE

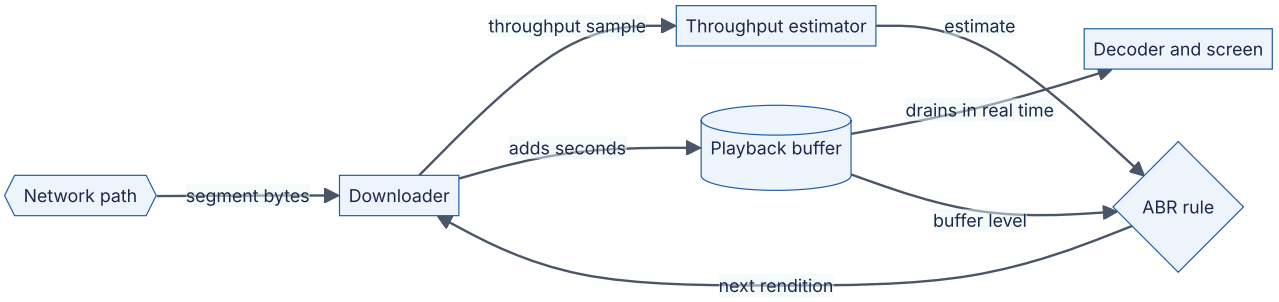
Worked example — Both rules on the ladder above. Throughput rule: the last five segments downloaded at 8, 7, 6, 2 and 8 Mb/s.

1. Harmonic mean: $5/(0.125 + 0.143 + 0.167 + 0.5 + 0.125) = 5/1.060 = 4.72$ Mb/s; $\times 0.8 = 3.78$ → choose 3.0 Mb/s.
2. The arithmetic mean would be 6.2 Mb/s; $\times 0.8 = 4.96$ → 4.5 Mb/s, betting on a link that just delivered 2 Mb/s.

BBA with $x_r = 10$ s, $x_c = 30$ s: at a 25 s buffer, $0.4 + (15/30)(5.6) = 3.2$ → 3.0 Mb/s; at 8 s → 0.4 Mb/s; at 45 s → 6.0 Mb/s.

Interpretation. The throughput rule reacts to the network, the buffer rule to the consequence that matters, a draining buffer. **Engineering implication.** Throughput estimates are essential at startup, when the buffer is empty and says nothing; in steady state the buffer is the more reliable signal.

Figure 21.9 — The ABR control loop.



Reading the diagram. The downloader fetches one segment at a time across the network (hexagon, outside the player's control), adding its duration to the buffer (cylinder), which the decoder drains at one second per second. The ABR rule reads the throughput estimate, the buffer level or both, and picks the next rendition, closing a feedback loop. Fetches are synchronous; the buffer decouples download from playback. The failure point is an empty buffer (a rebuffer). Each client runs its own loop; CDN load is the sum of their choices.

```
        # Throughput-based versus buffer-based (BBA-style) bitrate selection on the
        # same synthetic bandwidth trace. Rates in Mb/s, times in seconds.
        LADDER = [0.4, 0.8, 1.5, 3.0, 4.5, 6.0]   # bitrate ladder, Mb/s
        SEG = 4.0                                  # segment duration, s
        B_MAX = 60.0                               # player buffer limit, s
        DT = 0.05                                  # simulation step, s


        def bandwidth(t):
            """6 Mb/s, except a 60 s collapse to 0.6 Mb/s starting at t = 30 s."""
            return 0.6 if 30 <= t < 90 else 6.0


        def throughput_rule(history, buffer):
            if not history:
                return LADDER[0]
            recent = history[-5:]
            estimate = len(recent) / sum(1 / x for x in recent)   # harmonic mean
            fits = [r for r in LADDER if r <= 0.8 * estimate]     # 20% safety margin
            return fits[-1] if fits else LADDER[0]


        def buffer_rule(history, buffer, reservoir=10.0, cushion=30.0):
            if buffer <= reservoir:
                return LADDER[0]
            if buffer >= reservoir + cushion:
                return LADDER[-1]
            target = LADDER[0] + (buffer - reservoir) / cushion * (LADDER[-1] -
LADDER[0])
            return max(r for r in LADDER if r <= target)


        def play(rule, duration=200.0):
            t = buffer = stall = 0.0
            history, chosen, started = [], [], False
            while t < duration:
                rate = rule(history, buffer)
                bits, start = rate * SEG, t
                while bits > 0:                    # download one segment
                    bits -= bandwidth(t) * DT
                    t += DT
                    if started:
                        if buffer > 0:
                            buffer = max(0.0, buffer - DT)
                        else:
                            stall += DT            # empty buffer: the video freezes
                history.append(rate * SEG / (t - start))
                buffer += SEG
                started = True
                chosen.append(rate)
                if buffer > B_MAX - SEG:           # buffer full: idle until room
                    t += buffer - (B_MAX - SEG)
                    buffer = B_MAX - SEG
            switches = sum(a != b for a, b in zip(chosen, chosen[1:]))
            return sum(chosen) / len(chosen), stall, switches
```

```
        if __name__ == "__main__":
            for name, rule in [("throughput-based", throughput_rule), ("buffer-based",
buffer_rule)]:
                avg, stall, switches = play(rule)
                print(f"{name:<17} mean bitrate {avg:4.2f} Mb/s  stalled {stall:4.1f} s
"
                      f"switches {switches}")
```

Output:

```
throughput-based  mean bitrate 3.48 Mb/s  stalled 17.3 s  switches 9
buffer-based      mean bitrate 3.64 Mb/s  stalled  0.0 s  switches 15
```

What to notice:

- Just before the collapse at $t = 30$ s, both rules request a 4.5 Mb/s segment (18 Mb), which takes about 29 s at 0.6 Mb/s. The throughput rule ramped up at once and holds only 15 s of buffer, so the video freezes; the buffer rule started conservatively, built 38.5 s of buffer, and rides it out.
- Afterwards the throughput rule's history still contains fast samples, so it overestimates and stalls again. The buffer rule oscillates between 0.4 and 0.8 Mb/s near its reservoir, hence more switches; real players add hysteresis.
- This is one synthetic trace, an illustration rather than a benchmark.

INDUSTRY EXAMPLE

Netflix and buffer-based adaptation. Netflix and Stanford researchers ran buffer-based algorithms in a randomized experiment on Netflix's production service. They reported that BBA reduced the rebuffer rate by roughly 10–20% compared with the then-default ABR algorithm while delivering a similar average video rate, and that throughput estimates remained useful during startup, when the buffer is still filling [Huang et al. 2014]. *Evidence: Primary* — [Huang et al. 2014]

21.8.4 Live latency

For live video, the player starts several segments behind the newest one so that it has a buffer; RFC 8216 says a client should not start playback less than three target durations from the end of the playlist [RFC 8216].

FORMULA

Glass-to-glass latency of segmented live video

$$L \approx t_{enc} + k \times D_{seg} + t_{poll}$$

Symbol	Meaning	Units
t_{enc}	Capture, encode and package delay	s
k	Segments (or parts) held back by the player	count
D_{seg}	Segment (or part) duration	s
t_{poll}	Delay before the player learns a new segment exists, plus download	s
L	Delay from camera to screen	s

WORKED EXAMPLE

Worked example — Classic HLS versus low-latency parts. Classic: $t_{enc} = 2$ s, $k = 3$, $D_{seg} = 6$ s, $t_{poll} \approx 3$ s, so $L \approx 2 + 18 + 3 = 23$ s. With 1 s parts and blocking playlist reloads: $L \approx 1 + 3 \times 1 + 0.5 = 4.5$ s.

Interpretation. Segment duration dominates. **Engineering implication.** Cutting latency means smaller units and push-like signaling, which multiplies requests per viewer and makes request collapsing at the CDN essential.

Low-Latency HLS (introduced by Apple in 2019) adds **partial segments** published while the full segment is still being encoded, **blocking playlist reloads** (the edge holds a playlist request until the next part exists, removing polling delay) and **preload hints** [Apple Docs: Low-Latency HLS]. **Low-latency DASH** achieves similar results by delivering CMAF chunks of a segment with HTTP chunked transfer as they are encoded. Both demand CDNs that can hold requests open and stream objects that are still being written.

21.9 Edge computing on CDNs

Since every request already passes through a programmable proxy 10 ms from the user, you can run your own code there: validating tokens, A/B-test assignment, redirects, image-format choice, custom cache keys, page assembly. The problem is isolation: thousands of customers' code must share each server safely and start far faster than a container or virtual machine can.

Table 21.7 — Edge compute runtimes (brief; as of 2026).

Platform	Isolation model	Where it runs	Notable limits and traits
Cloudflare Workers	V8 **isolates**: many customers' scripts in one process, each in its own JavaScript heap	Every Cloudflare data center	JavaScript and WebAssembly; isolates start in milliseconds, avoiding container cold starts [Bloom 2018]
CloudFront Functions	Lightweight JavaScript runtime	CloudFront edge locations	Viewer request and response only; sub-millisecond execution budget; no network calls [AWS 2021]
Lambda@Edge	AWS Lambda functions	Regional edge caches	Node.js and Python; network access; up to 5 s on viewer triggers and 30 s on origin triggers [AWS 2021]
Fastly Compute	WebAssembly module instantiated per request	Fastly PoPs	Any language compiling to Wasm (Rust, Go, JavaScript); sandbox per request [Fastly Docs: Compute]

The design rule is to keep edge code stateless and cheap, and to treat edge storage (key-value stores, durable objects) as a separate consistency question. Runtimes, cold starts, Firecracker microVMs and edge data patterns are covered in depth in Chapter 48.

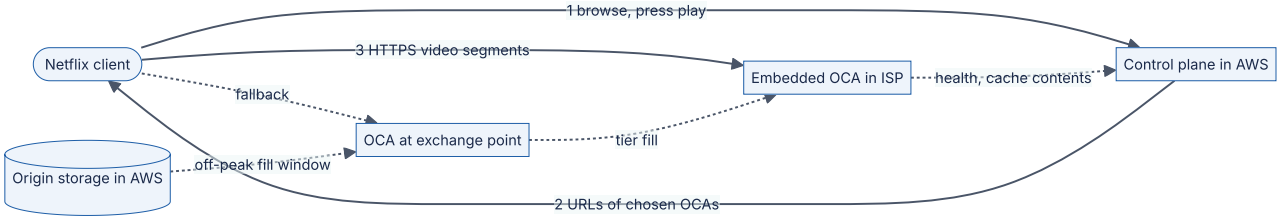
21.10 Netflix Open Connect

Netflix is the extreme case of video delivery: at peak it is one of the largest sources of Internet traffic, and its content is long-form, licensed, and therefore known in advance. Since 2012 it has delivered video through its own CDN, **Open Connect**.

INDUSTRY EXAMPLE

Netflix Open Connect. Netflix designs its own cache servers, **Open Connect Appliances (OCAs)**, and deploys them in two ways: **embedded** inside ISP networks, where Netflix provides the hardware to qualifying ISPs at no charge and the ISP provides space, power and connectivity, and at **Internet exchange points**, where Netflix peers with ISPs directly. The split of responsibilities is clean: services running in AWS handle everything before playback (browsing, authorization, choosing which OCAs a client should use) and return URLs pointing at OCAs; the OCAs then serve the video bytes. Rather than fill caches on demand, Netflix predicts what each OCA's population will watch and **proactively fills** OCAs during an off-peak **fill window** agreed with each ISP, so that peak-hour traffic is served almost entirely from caches that are already full. Fill is tiered: an OCA copies from a peer in the same location first, then from an upstream OCA, and only then from origin storage in AWS [Netflix TechBlog 2016]. *Evidence: Primary* — [Netflix Open Connect Overview]; [Netflix TechBlog 2016]

Figure 21.10 — Open Connect: control plane in the cloud, bytes from the ISP.



Reading the diagram. Steps 1–2 are synchronous API calls to Netflix's control plane in AWS, which knows which OCAs hold each file and how healthy and loaded they are. Step 3 streams segments from an embedded OCA inside the user's ISP; an OCA at an exchange point is the fallback. Dotted edges are asynchronous: content is pushed from origin storage to exchange-point OCAs and then to embedded OCAs during off-peak fill windows, and OCAs report health and contents to the control plane. Storage boundaries: origin storage in AWS and each OCA's disks and flash. Failure points: an OCA failure (the client is directed to another), a missed fill (the title is served from a farther OCA), and the control plane (playback cannot start, but running streams continue). Scaling: add OCAs in ISPs, which also removes traffic from the ISP's transit links.

21.10.1 Why it works, and when it would not

Proactive fill turns caching into **replica placement**: with a known catalog and good popularity predictions, peak-hour hit ratios approach those of a perfect cache, and fill uses idle off-peak capacity. Embedding removes transit costs for both parties, which is why ISPs accept the boxes. It depends on a curated catalog, predictable demand and enor-

mous volume; a site with viral user-generated content or modest volume is better served by a commercial CDN with tiered caching.

> COMMON MISTAKE
>
> **"Netflix streams from AWS."** Netflix runs its control plane in AWS, but the video bytes come from Open Connect appliances, mostly inside or next to the viewer's ISP. Conflating the two leads to wildly wrong egress estimates in interviews (§54.2 develops the full design).

21.11 CDN security

A CDN sits in front of everything, so it becomes the natural place for several security controls (Chapter 10 holds the foundations).

- **DDoS absorption.** Anycast spreads volumetric floods across every PoP, and the aggregate capacity of a large CDN exceeds any single attack's bandwidth; stateless edge proxies absorb SYN floods (Figure 10.9, §10.11).
- **WAF, bot management and rate limiting.** Application-layer rules, challenges and per-client limits (§22.3) run at the edge, before traffic reaches the origin.
- **TLS certificate management.** To terminate TLS, the CDN needs a certificate and private key for your domain. CDNs automate issuance and renewal with ACME [RFC 8555] and select certificates per hostname with SNI. If policy forbids handing over the private key, designs such as Cloudflare's **Keyless SSL** keep the key on the customer's server and perform only the private-key operation there [Sullivan 2014]. Certificate expiry at the edge is a classic outage (§10.5).
- **Origin protection.** Accept origin traffic only from the CDN, using address allowlists plus authenticated origin pulls (mutual TLS) or a secret header; otherwise attackers bypass every control above.
- **Private content.** For PhotoShare's private photos, the API issues a short-lived **signed URL** or token (an HMAC over path and expiry, §10.4.6), and the edge verifies it before serving from cache. The signature is excluded from the cache key, so one cached copy serves all authorized viewers while unauthorized requests are rejected at the edge.

21.12 Summary and revision notes

SUMMARY

- A **CDN** is a hierarchy of caching reverse proxies: **edge PoPs** near users, **mid-tier/shield** caches, and the authoritative **origin**. It cuts latency (round trips, not bandwidth), origin load, egress cost and DDoS exposure.
- **Request routing**: DNS mapping steers precisely but per resolver and bounded by TTLs (ECS helps); **anycast** steers by BGP, fails over automatically and disperses attacks, but with coarse load control. Real CDNs combine them.
- **Offload**: O_R (requests) and O_B (bytes) differ; miss ratios multiply across tiers, $f_{\text{origin}} = \prod (1 - h_i)$; $B_{\text{origin}} = B_{\text{users}}(1 - O_B)$. Going from 90% to 98% offload cuts origin load fivefold.
- **Shielding** funnels all misses through one tier: per object and TTL the origin sees about 1 fetch instead of $N(1 - e^{-\lambda T/N})$. It matters most with many PoPs, long tails, short TTLs and frequent purges.
- **RFC 9111 headers**: `s-maxage` for the CDN, short `max-age` for browsers, `stale-while-revalidate` and `stale-if-error` for resilience, `ETag` with `304` for cheap revalidation, `private` / `no-store` for personal data.
- **Cache keys**: normalize (host case, sorted allow-listed parameters, collapsed `Accept-Encoding`), avoid `Vary: User-Agent` and raw cookies; unkeyed inputs enable cache poisoning.
- **Invalidation**: URL purge, **tag (surrogate-key) purge** in about 150 ms on fast CDNs, soft purge, and **versioned URLs** with `immutable` for static assets.
- **Dynamic content**: micro-caching, ESI fragments, and **dynamic site acceleration** (TLS near the user, warm connections to origin).
- **Video**: HLS (RFC 8216) and DASH (ISO/IEC 23009-1) cut renditions into segments listed in manifests; CMAF unifies segments. ABR is throughput-based (harmonic mean × safety factor), buffer-based (reservoir and cushion) or hybrid; live latency ≈ encode + held-back segments × duration, reduced by LL-HLS parts.
- **Netflix Open Connect** embeds appliances in ISPs and fills them proactively off-peak; the control plane lives in AWS. Edge compute (isolates, Wasm, Lambda@Edge) runs small stateless logic at PoPs (Chapter 48).

21.13 Knowledge checkpoint

1. Why does a CDN reduce the time to fetch a 100 KB object even when the origin has plenty of bandwidth?
2. Give one advantage and one disadvantage of anycast relative to DNS-based routing.
3. A CDN’s request offload is 97% but its byte offload is 85%. What does that suggest?

4. Edge hit ratio is 80% and shield hit ratio (on edge misses) is 60%. What fraction of requests reaches the origin?
5. Why does a shield help a long-tail object more than its edge hit ratio suggests?
6. With `Cache-Control: max-age=300` and `Age: 280` from the CDN, how long is the browser's copy fresh?
7. What does `stale-if-error=3600` allow?
8. Why is `Vary: User-Agent` harmful at a CDN?
9. Compare tag purge with versioned URLs.
10. When does dynamic site acceleration *not* reduce latency?
11. Why did HLS and DASH originally require storing video twice, and what fixed it?
12. Compute the harmonic mean of 4 and 1 Mb/s and explain why ABR uses it.
13. Why can Netflix fill caches before demand arrives, and why can most sites not?
14. How should a CDN serve private photos without caching one copy per user?

Answers

1. Fetch time is dominated by handshake and slow-start round trips; the edge shortens each round trip (for example from 200 ms to 10 ms).
2. Advantage: automatic failover and DDoS dispersion with one IP. Disadvantage: BGP may choose a non-lowest-latency PoP, and load is hard to steer.
3. Misses are disproportionately large objects (long-tail video or downloads); origin egress is driven by those.
4. $0.2 \times 0.4 = 0.08$, i.e. 8%.
5. The object is rare at each PoP but common across all of them; the shield aggregates those misses into one fetch per TTL.
6. $300 - 280 = 20$ s.
7. Serving the stale response for up to an hour after it becomes stale if the origin returns errors or is unreachable.
8. It creates a separate cache entry for every distinct User-Agent string (thousands), destroying the hit ratio.
9. Tag purge actively removes everything labeled with an entity, across pages, but needs tagging and a purge API; versioned URLs need no purge and cache forever, but need a referring document that changes.
10. When the client already has a warm connection to the origin, or the user is near the origin: the edge then adds about one user–edge RTT.
11. HLS used MPEG-2 transport stream and DASH fragmented MP4; CMAF defines one fragmented-MP4 format both manifests can reference.
12. $2/(1/4 + 1/1) = 1.6$ Mb/s. It is dominated by slow samples, so a single fast outlier does not lead the player to choose a rate the link cannot sustain.
13. Its catalog is curated and demand is predictable from viewing history; user-generated, viral content is not predictable.
14. Verify a short-lived signed URL or token at the edge and exclude the signature from the cache key, so one cached copy serves all authorized viewers.

21.14 Interview questions

Basic

Q1. What is a CDN and when would you add one? Answer. A distributed set of caching reverse proxies near users that serves content for an origin. Add one when users are far from your servers, when static or shareable content dominates bandwidth, when origin egress or load is expensive, or when you need DDoS protection. Quantify the gain: round trips saved per request and origin bandwidth $B_{\text{users}}(1 - O_B)$.

Q2. How does a user get routed to a nearby PoP? Answer. Either the CDN's DNS answers with a PoP chosen by a mapping system (using resolver location, ECS, load and health), or all PoPs announce the same anycast prefix and BGP delivers packets to the nearest. DNS gives finer control but is bounded by TTLs; anycast fails over automatically and spreads attacks.

Intermediate

Q3. Your origin falls over every time marketing purges the home page. Why, and what do you change? Answer. Every PoP misses simultaneously and fetches independently, a CDN-wide stampede. Enable origin shielding and request collapsing, use soft purge so stale content is served while revalidating, add `stale-while-revalidate`, purge by tag rather than everything, and cache the page briefly (micro-caching) instead of marking it uncacheable.

Q4. Design the caching policy for PhotoShare. Answer. Images: URL carries photo ID and rendition, `s-maxage` of days, tagged by photo ID for deletion, private photos via edge-verified signed URLs. JS and CSS: hashed filenames, `max-age=31536000, immutable`. Feed API: `private`, or micro-cache public parts. Normalize keys, add `stale-if-error`, shield near the origin, and measure offload at the origin.

Q5. How does a video player decide which quality to download? Answer. Throughput-based rules pick the highest ladder rung below a safety fraction of a harmonic-mean bandwidth estimate; buffer-based rules map buffer level to rate (minimum below a reservoir, maximum above reservoir plus cushion). Production players use hybrids: throughput at startup, buffer in steady state, with hysteresis to limit switching.

Advanced

Q6. Estimate CDN and origin capacity for a 5-million-viewer live event. Answer. With a mean bitrate of about 3.7 Mb/s, edge egress is $5 \times 10^6 \times 3.7 \approx 18.5$ Tb/s, so plan capacity per metro and per ISP and probably use several CDNs. Origin egress is only shields × the ladder sum (for example $20 \times 16.2 = 324$ Mb/s). Use 2–4 s segments or LL-HLS parts,

short TTLs on live playlists, long TTLs on segments, request collapsing, redundant encoders and packagers, and a lower maximum rung as a load-shedding lever.

Q7. Would you build your own CDN? Answer. Only at very large, predictable volume where transit costs dominate and you can run hardware and ISP partnerships, as Netflix does. Otherwise commercial CDNs (possibly several, steered by DNS) give reach, DDoS capacity and features at lower total cost; a middle path is a commercial CDN plus your own shield.

21.15 Practical exercises

1. Run the shielding simulation with Zipf exponents 0.7, 1.0 and 1.2 and with 5, 20 and 100 PoPs. Plot origin fetches with and without a 30% shield, and explain which parameter makes shielding most valuable.
2. Configure NGINX or Varnish as a caching reverse proxy in front of a small origin. Verify `s-maxage`, `stale-while-revalidate` and `stale-if-error` with `curl -I` while stopping the origin, and confirm request collapsing with 100 concurrent requests.
3. Extend the ABR program with a hybrid rule (throughput during the first 20 s, buffer afterwards, switch only if the new rate differs for two consecutive decisions). Compare stalls and switches on three bandwidth traces.

21.16 Design challenges

1. **Global news site.** 50 million page views per day, breaking-news articles that must update within 5 seconds of an edit, personalized headers, and an origin in one region. Choose cache keys, TTLs, tagging and purge strategy, ESI or edge compute for personalization, and shielding; estimate offload and origin request rate.
2. **Live sports streaming.** 3 million concurrent viewers across three continents with a 5-second latency target. Choose segment and part durations, ladder, CDN strategy (single or multi-CDN), shield placement and failure handling for an encoder outage; estimate edge and origin bandwidth.

21.17 References

1. Nygren, E., Sitaraman, R. K., and Sun, J. “The Akamai Network: A Platform for High-Performance Internet Applications.” *ACM SIGOPS Operating Systems Review* 44(3), 2010.
2. Fielding, R., Nottingham, M., and Reschke, J. *RFC 9111: HTTP Caching*. IETF, 2022. https://www.rfc-editor.org/rfc/rfc9111
3. Fielding, R., Nottingham, M., and Reschke, J. *RFC 9110: HTTP Semantics*. IETF, 2022. https://www.rfc-editor.org/rfc/rfc9110

4. Nottingham, M. *RFC 5861: HTTP Cache-Control Extensions for Stale Content*. IETF, 2010.
5. McManus, P. *RFC 8246: HTTP Immutable Responses*. IETF, 2017.
6. Ludin, S., Nottingham, M., and Wu, Y. *RFC 9213: Targeted HTTP Cache Control*. IETF, 2022.
7. Pantos, R., and May, W. *RFC 8216: HTTP Live Streaming*. IETF, 2017. https://www.rfc-editor.org/rfc/rfc8216
8. ISO/IEC 23009-1. *Information technology — Dynamic adaptive streaming over HTTP (DASH) — Part 1: Media presentation description and segment formats*.
9. ISO/IEC 23000-19. *Information technology — Multimedia application format (MPEG-A) — Part 19: Common media application format (CMAF) for segmented media*.
10. Huang, T.-Y., Johari, R., McKeown, N., Trunnell, M., and Watson, M. “A Buffer-Based Approach to Rate Adaptation: Evidence from a Large Video Streaming Service.” *ACM SIGCOMM*, 2014.
11. Jiang, J., Sekar, V., and Zhang, H. “Improving Fairness, Efficiency, and Stability in HTTP-based Adaptive Video Streaming with FESTIVE.” *ACM CoNEXT*, 2012.
12. Yin, X., Jindal, A., Sekar, V., and Sinopoli, B. “A Control-Theoretic Approach for Dynamic Adaptive Video Streaming over HTTP.” *ACM SIGCOMM*, 2015.
13. Spiteri, K., Urgaonkar, R., and Sitaraman, R. K. “BOLA: Near-Optimal Bitrate Adaptation for Online Videos.” *IEEE INFOCOM*, 2016.
14. Mao, H., Netravali, R., and Alizadeh, M. “Neural Adaptive Video Streaming with Pensieve.” *ACM SIGCOMM*, 2017.
15. Huang, Q., Birman, K., van Renesse, R., Lloyd, W., Kumar, S., and Li, H. C. “An Analysis of Facebook Photo Caching.” *ACM SOSP*, 2013.
16. Grigorik, I. *High Performance Browser Networking*. O’Reilly, 2013. https://hpbn.co
17. Karger, D., Lehman, E., Leighton, T., Panigrahy, R., Levine, M., and Lewin, D. “Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web.” *ACM STOC*, 1997.
18. Contavalli, C., van der Gaast, W., Lawrence, D., and Kumari, W. *RFC 7871: Client Subnet in DNS Queries*. IETF, 2016.
19. Chen, F., Sitaraman, R. K., and Torres, M. “End-User Mapping: Next Generation Request Routing for Content Delivery.” *ACM SIGCOMM*, 2015.
20. Calder, M., Flavel, A., Katz-Bassett, E., Mahajan, R., and Padhye, J. “Analyzing the Performance of an Anycast CDN.” *ACM IMC*, 2015.
21. Tsimelzon, M., et al. *ESI Language Specification 1.0*. W3C Note, 2001.
22. Kettle, J. “Practical Web Cache Poisoning.” PortSwigger Research, 2018.
23. Gil, O. “Web Cache Deception Attack.” Black Hat USA, 2017.
24. Netflix. *Open Connect Overview*. https://openconnect.netflix.com/Open-Connect-Overview.pdf (accessed 2026).
25. Netflix Technology Blog. “Netflix and Fill.” 2016.
26. Bloom, Z. “Cloud Computing without Containers.” *Cloudflare Blog*, 2018.

27. Amazon Web Services. “Introducing CloudFront Functions – Run Your Code at the Edge with Low Latency at Any Scale.” *AWS News Blog*, 2021.
28. Amazon Web Services. *Amazon CloudFront Developer Guide*: “Using Amazon CloudFront Origin Shield”; regional edge caches. Accessed 2026.
29. Cloudflare. *Cloudflare documentation*: Network; Tiered Cache; Cache keys; Purge cache by tag. https://developers.cloudflare.com/ (accessed 2026).
30. Fastly. *Fastly documentation*: Purging (surrogate keys); Compute. https://www.fastly.com/documentation/ (accessed 2026).
31. Betts, A. “Is purging still the hardest problem in computer science?” *Fastly Blog*. https://www.fastly.com/blog/is-purging-still-the-hardest-problem-in-computer-science
32. Apple. *Enabling Low-Latency HTTP Live Streaming (HLS)*; *HLS Authoring Specification for Apple Devices*. Apple Developer documentation. Accessed 2026.
33. Sullivan, N. “Keyless SSL: The Nitty Gritty Technical Details.” *Cloudflare Blog*, 2014.
34. Barnes, R., Hoffman-Andrews, J., McCarney, D., and Kasten, J. *RFC 8555: Automatic Certificate Management Environment (ACME)*. IETF, 2019.

21.18 Open verification items

- Fastly purge timing (~150 ms) and the publication date of [Betts, Fastly Blog]; fastly.com was unreachable from the authoring environment.
- Netflix Open Connect claims (no-charge embedded OCAs, AWS control plane, fill-window and tiered-fill order, 2012 launch): openconnect.netflix.com was unreachable; confirm against the Overview PDF and the 2016 “Netflix and Fill” post (exact date and authors).
- Huang et al. 2014: confirm the “10–20% rebuffer reduction” wording.
- Illustrative 2026 prices in §21.5.4 ($0.05/GB origin egress tier, $0.01/GB negotiated CDN rate).
- CloudFront Functions and Lambda@Edge limits (sub-millisecond; 5 s/30 s) as of 2026.
- LL-HLS details (part hold-back of about three parts; 2019 introduction).
- Exact title and authors of the ESI 1.0 W3C Note.

Chapter 22 — API and Traffic Management

> **Level:** 2–3 · **Track:** A (Traditional) · **Estimated reading time:** 3 h

Every service eventually receives more requests than it can handle: from success (a launch), abuse (a scraper) or, very often, itself (a slow database makes callers time out, they retry, and the retries triple the load). As utilization approaches 100%, queueing delay grows without bound (§4.10.3); this chapter is the toolbox for staying on the safe side of that curve. It covers **rate limiting** (deciding how much each client may send), **load shedding** and **backpressure** (what a server does when total demand exceeds capacity), the client-side disciplines of **timeouts, deadlines, retries, circuit breakers and bulkheads**, and finally **idempotency keys**, which make retries safe for operations that must never happen twice, such as charging a card.

The running example is PhotoShare (Chapter 1), which now offers a public API and sells a "PhotoShare Pro" subscription through an external payment provider: the API needs quotas, the upload service needs overload protection, and the purchase must never charge twice. The traffic figures in this chapter describe PhotoShare several years after the v3 stage of §1.9, with rates 15–30× higher than Chapter 1's; each worked example states its own capacity and load, so treat them as independent scenarios rather than one consistent fleet.

22.1 Learning objectives

After this chapter you should be able to:

- Choose where to enforce a rate limit (client, edge, gateway, service) and what to key it on, and signal limits correctly with HTTP 429, `Retry-After` and the draft `RateLimit` header fields.
- Implement and parameterize the token bucket, leaky bucket (including GCRA), fixed window, sliding log and sliding window counter algorithms, and compare their accuracy, memory and burst behavior with numbers.
- Design a distributed rate limiter (centralized Redis with Lua, local counting with synchronization, quota distribution) and quantify its latency/accuracy trade-off.
- Explain load shedding, priority-based shedding, CoDel-style queue control and adaptive LIFO, and compute when an overloaded server's goodput collapses.
- Size bounded queues and connection pools with Little's Law, and explain backpressure from TCP to reactive streams.

- Propagate deadlines across a call chain and allocate a timeout budget.
- Configure retries with exponential backoff, full/equal/decorrelated jitter and retry budgets, and calculate retry amplification in a deep call graph.
- Implement a circuit breaker and bulkheads, and choose their thresholds.
- Design an idempotency-key layer: key scope, request fingerprinting, storage schema, concurrency control, leases and retention.

22.2 Prerequisites

- Queueing, utilization and the M/M/1 latency curve — §4.10 (why overload makes latency explode).
- Little's Law, $L = \lambda W$ — §4.11 (sizing queues, pools and in-flight limits).
- TCP flow control and the receive window — §7.7.6 (the original backpressure mechanism).
- HTTP methods, safety and idempotency; status codes 429 and 503 — §8.6.3–§8.6.4 (this chapter builds the full idempotency design on them).
- gRPC status codes and deadlines — §8.8 (deadline propagation in RPC frameworks).
- Redis basics (in-memory key-value store) — §12.4 (the usual home for shared rate-limit counters; internals in §20.6).
- Units: time in milliseconds (ms) and seconds (s); request rates in requests per second (RPS); storage in decimal units (1 KB = 10^3 bytes, 1 GB = 10^9 bytes).

22.3 Rate limiting

22.3.1 What rate limiting is and why it exists

> DEFINITION
>
> **Definition — Rate limiting.** Enforcing an upper bound on how many requests (or how much work) a given **client identity** may submit to a resource over time. Requests beyond the bound are rejected (or, less often, delayed). The bound is usually stated as a **rate** (for example 100 requests per minute) plus a **burst** allowance.

First-principles explanation

A popular museum caps entries per half hour and per booking, so that one tour bus of 200 people cannot crowd out everyone else. A rate limiter does the same for an API: it caps how many requests each client may make per unit of time, so no single client can consume the shared capacity.

Why does this exist?

Rate limits solve five distinct problems: **abuse** (credential stuffing, scraping, application-layer DoS, §10.11; 5 login attempts per minute turns a million-password attack into a multi-year project), **fairness** between tenants, **protecting downstream capacity**, **cost control** for expensive calls (paid APIs, GPU inference), and **product tiers** ("Free: 60 requests/minute; Pro: 5,000").

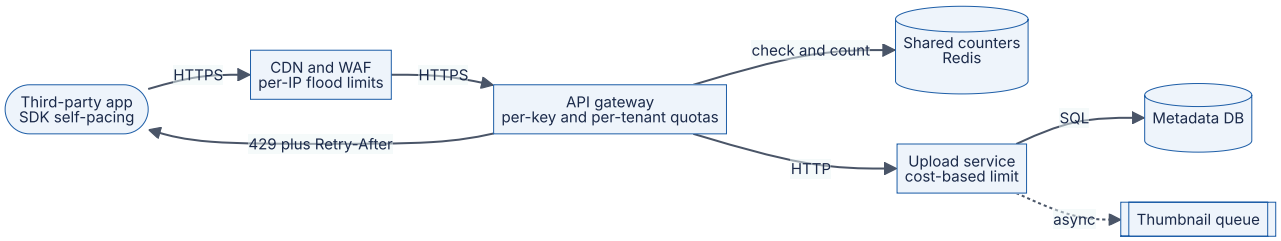
Rate limits answer "is this client within its contract?"; load shedding (§22.9) answers "can I afford this request right now?". Keep the two separate.

22.3.2 Where to enforce limits

Mature systems enforce limits in layers:

1. **Client side.** SDKs pace themselves and honor `Retry-After`; useful, but never trusted for protection.
2. **Edge (CDN or WAF).** Coarse per-IP limits stop floods before they reach your network (§10.11).
3. **API gateway.** After authentication it can limit per API key, user, tenant and route; the usual home of contractual quotas.
4. **Service.** Only the service knows a request's true cost (rows scanned, bytes uploaded, tokens generated), so it can limit on cost rather than count.

Figure 22.1 — Layered rate limiting for the PhotoShare API.



Reading the diagram. A third-party app sends HTTPS requests through three enforcement layers: the *CDN/WAF* (cheap per-IP limits), the *API gateway* (per-key and per-tenant quotas checked against *shared counters* in Redis, a storage boundary shared by all gateway nodes, §22.8) and the *upload service* (a cost-based limit in bytes per minute). Solid arrows are synchronous; the thumbnail queue is asynchronous. Whichever layer rejects returns 429 with `Retry-After`. Failure point: the counter store is on every request's hot path, so its outage policy (§22.8.5) must be decided in advance. Scaling: edge and gateway scale horizontally; counters are partitioned by key.

22.3.3 Choosing the key

The **rate-limit key** is the identity whose usage is counted.

Table 22.1 — Rate-limit keys.

Key	Strengths	Weaknesses	Typical use
Client IP address	Available before authentication	Many users share one IP behind carrier-grade NAT (§7.13); attackers rotate IPs	Edge flood control, login, sign-up
User or account ID	Fair per person	Needs authentication; attackers create accounts	Per-user actions
API key or OAuth client ID	Maps to a contract and a bill	A leaked key consumes the owner's quota	Public developer APIs
Tenant	Isolates customers	One heavy user exhausts the tenant	Noisy-neighbor control
Route plus one of the above	Tighter limits on expensive endpoints	More counters	`POST /uploads`
Cost units	Reflects true resource use	Cost must be estimated up front	GraphQL cost (§8.7), LLM tokens

Production systems apply **hierarchical limits**: a request must pass every applicable bucket (per-IP *and* per-key *and* per-tenant *and* a route ceiling).

22.3.4 Telling the client: 429, Retry-After and RateLimit headers

- **429 Too Many Requests** [Nottingham and Fielding 2012] blames the client; **503 Service Unavailable** blames the server and is right for load shedding (§22.9).
- `Retry-After` (RFC 9110 §10.2.3) gives seconds or a date after which to retry; compute it from the limiter's state.
- **RateLimit header fields** are being standardized in the IETF Internet-Draft *RateLimit header fields for HTTP* [Polli et al., draft], as of 2026 still a **draft, not an RFC**. Early drafts used `RateLimit-Limit`, `-Remaining` and `-Reset`; recent drafts use `RateLimit-Policy` (quota `q`, window `w`) and `RateLimit` (remaining `r`, seconds to reset `t`). Many APIs still use their own `X-RateLimit-*` headers.

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/problem+json
Retry-After: 18
RateLimit-Policy: "default";q=100;w=60
RateLimit: "default";r=0;t=18

{"type": "https://api.photoshare.example/errors/rate-limited",
 "title": "Rate limit exceeded", "status": 429,
 "detail": "API key pk_live_42 is limited to 100 requests per 60 s."}
```

What to notice:

- The body uses RFC 9457 problem details; `Retry-After: 18` and `t=18` agree, so older clients still behave.
- Send the `RateLimit` fields on *successful* responses too, so clients slow down before being rejected.

INDUSTRY EXAMPLE

GitHub REST API. GitHub returns `x-ratelimit-limit`, `x-ratelimit-remaining`, `x-ratelimit-used`, `x-ratelimit-reset` (a Unix timestamp) and `x-ratelimit-resource`, answers excess requests with 403 or 429, and may send `retry-after` for secondary limits [GitHub Docs: REST rate limits]. De-facto conventions predate the IETF draft, so client libraries must handle several dialects. *Evidence: Primary* — [GitHub Docs: REST rate limits]

Table 22.2 — Rate-limiting failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Thousands of mobile users blocked at once	Per-IP limit applied to a carrier NAT address	429 spike concentrated on few IPs with many user agents	Raise or bypass the IP limit for authenticated traffic	Key on user or API key after authentication; IP limits only pre-auth
Clients hammer after 429	No `Retry-After`; SDK retries immediately	429 rate rises in lockstep with request rate	Return `Retry-After`; block abusive keys	SDK backoff with jitter (§22.12)
Legitimate burst rejected	Burst allowance smaller than the client's natural burst (app start-up sync)	429s at app launch time, low average usage	Increase bucket capacity	Size burst from observed per-session peaks (§22.4)

COMMON MISTAKE

Mistake — "Rate limiting protects my service from overload." 10,000 customers each within 100 RPS can still send 1,000,000 RPS in total. Capacity protection needs load shedding and backpressure (§22.9–§22.10).

22.4 Token bucket

22.4.1 The algorithm

DEFINITION

Definition — Token bucket. A rate limiter that keeps a counter of **tokens**, refilled continuously at **rate** *r* up to a maximum **capacity** (burst size) *b*. Each request consumes one token (or a cost *c*); a request that finds too few tokens is rejected. The long-run admitted rate is at most *r*, and up to *b* requests may be admitted at once.

First-principles explanation

A jar is topped up with one sweet every six seconds but holds only 20. After an hour away you can eat 20 in a row (a **burst**), then only as fast as sweets arrive. The jar size limits the burst; the top-up speed limits the average.

Why does this exist?

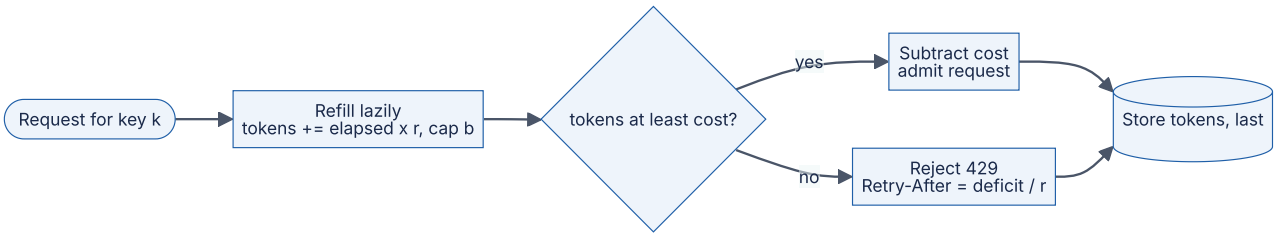
Real clients are bursty: an app opening fetches a profile, a feed and ten thumbnails in one second, then goes quiet. The token bucket separates average rate from burst size, exactly how capacity is planned; it came from network traffic policing and underlies most gateways' "rate and burst" settings.

How it works internally

No timer is needed. Store `tokens` and `last` (time of last update) per key and refill lazily:

1. `elapsed = now − last`; `tokens = min(b, tokens + elapsed × r)`; `last = now`.
2. If `tokens ≥ cost`: subtract the cost and admit.
3. Otherwise reject, and report `Retry-After = (cost − tokens) / r`.

Figure 22.2 — Token bucket admission decision.



Reading the diagram. A request triggers a lazy refill capped at *b*; the decision compares tokens with the request's cost; admitted requests subtract it, rejected ones compute when the deficit refills. Both paths write the two-number state back to storage (local memory, or a shared store in §22.8). Everything is synchronous and constant-time. Failure

point: the read-modify-write must be atomic when threads or nodes share a key (§22.8.2). Scaling: two numbers per key, so millions of keys are cheap.

Mathematical foundations

FORMULA

Token bucket state and admission bound

$$\text{tokens}(t) = \min\left(b,\ \text{tokens}(t_0) + r\,(t - t_0)\right) \qquad A(T) \le b + rT$$

Symbol	Meaning	Units
r	Refill rate (sustained allowed rate)	tokens/s (= requests/s at unit cost)
b	Bucket capacity (maximum burst)	tokens
t_0, t	Time of last update, current time	s
$A(T)$	Maximum requests admitted in any interval of length T	requests

WORKED EXAMPLE

Worked example — PhotoShare free tier: $r = 10$ RPS, $b = 20$.

1. A client idle for a while has a full bucket of 20. It fires 50 requests at $t = 0$: the first 20 are admitted, 30 are rejected. Deficit for the next request is 1 token, so `Retry-After` $= 1/10 = 0.1$ s.
2. The client waits 1 s: the bucket refills $1 \times 10 = 10$ tokens, so 10 more requests succeed.
3. Maximum admitted in any 60 s interval: $A(60) = 20 + 10 \times 60 = 620$ requests, an average of 10.3 RPS. The burst adds only 3% over a minute.
4. Time to refill from empty to full: $b/r = 20/10 = 2$ s.
5. Sustained 15 RPS for 10 s against a full bucket (150 requests over a 9.93 s span): the bound is $20 + 10 \times 9.93 = 119.3$, so at most 119 are admitted; the code below admits exactly 119.

Interpretation. r governs the minute-scale average; b only the first seconds.
Engineering implication. Set r from capacity planning and b from the largest legitimate client burst; if the downstream cannot absorb b requests at once from many clients, the burst, not the rate, will hurt you.

Code example

```
        import random
        import time


        class FakeClock:
            """A controllable clock so tests are deterministic (time in seconds)."""
            def __init__(self, t=0.0):
                self.t = t
            def __call__(self):
                return self.t
            def advance(self, dt):
                self.t += dt


        class TokenBucket:
            """Refill at `rate` tokens/s up to `capacity`; each request spends `cost`
tokens."""
            def __init__(self, rate, capacity, clock=time.monotonic):
                self.rate, self.capacity, self.clock = float(rate), float(capacity),
clock
                self.tokens = float(capacity)          # start full: a new client may
burst at once
                self.last = clock()

            def _refill(self):
                now = self.clock()
                elapsed = max(0.0, now - self.last)    # ignore a clock that steps back-
wards
                self.tokens = min(self.capacity, self.tokens + elapsed * self.rate)
                self.last = now

            def try_acquire(self, cost=1.0):
                self._refill()
                if self.tokens >= cost:
                    self.tokens -= cost
                    return True
                return False

            def retry_after(self, cost=1.0):
                """Seconds until `cost` tokens are available: the value for a Retry-
After header."""
                self._refill()
                return max(0.0, (cost - self.tokens) / self.rate)


        if __name__ == "__main__":
            clock = FakeClock()
            tb = TokenBucket(rate=10, capacity=20, clock=clock)

            # 1. A burst of 50 requests at t = 0
            admitted = sum(tb.try_acquire() for _ in range(50))
            print(f"t=0.0 s burst of 50: admitted {admitted}, retry after
{tb.retry_after():.2f} s")
            assert admitted == 20
```

```
            # 2. One second later the bucket has refilled 10 tokens
            clock.advance(1.0)
            admitted = sum(tb.try_acquire() for _ in range(50))
            print(f"t=1.0 s burst of 50: admitted {admitted}")
            assert admitted == 10

            # 3. Sustained 15 RPS for 10 s against a fresh (full) bucket
            clock, n_ok = FakeClock(), 0
            tb = TokenBucket(rate=10, capacity=20, clock=clock)
            for k in range(150):
                clock.t = k / 15
                n_ok += tb.try_acquire()
            bound = 20 + 10 * (149 / 15)               # b + r*T over the 9.93 s span
            print(f"15 RPS for 10 s: admitted {n_ok} of 150 (bound b + rT =
{bound:.1f})")
            assert n_ok <= bound
```

Output:

```
t=0.0 s burst of 50: admitted 20, retry after 0.10 s
t=1.0 s burst of 50: admitted 10
15 RPS for 10 s: admitted 119 of 150 (bound b + rT = 119.3)
```

What to notice:

- The injected clock makes tests exact. In production use a **monotonic** clock, never wall-clock time, which can jump backwards (§24.9).
- `retry_after` falls out of the state, so the 429 can say exactly when to return; a non-unit `cost` makes it a cost-based limiter.

Real-world industry applications

> INDUSTRY EXAMPLE
>
> **Stripe's API rate limiters.** Stripe described four limiters: a per-user **request rate limiter** (a token bucket in Redis, shared by all API servers), a **concurrent requests limiter**, a **fleet usage load shedder** reserving capacity for critical requests, and a **worker utilization load shedder** that sheds lower-priority traffic first. It recommends dark-launching new limiters, logging what would be rejected before enforcing [Tarjan 2017]. *Evidence: Primary* — [Tarjan 2017]

INDUSTRY EXAMPLE

Amazon API Gateway. AWS documents token-bucket throttling with a configurable *rate* and *burst* per account, stage, method or usage-plan client; excess requests receive 429 [AWS Docs: API Gateway throttling]. *Evidence: Primary* — [AWS Docs: API Gateway throttling]

Common misconceptions

COMMON MISTAKE

Mistake — "r = 10 RPS means no client ever exceeds 10 RPS." At b = 20, 20 requests can arrive in one millisecond. By Little's Law, in-flight requests $= \lambda W$ (§4.11), so a rate limit bounds concurrency only while latency stays bounded; add a **concurrency limit** too.

22.5 Leaky bucket

22.5.1 Two different algorithms with one name

DEFINITION

Definition — Leaky bucket. (a) **As a queue:** requests enter a FIFO buffer of capacity Q that is drained at a constant rate r; arrivals that find the buffer full are dropped. Output is perfectly smooth. (b) **As a meter:** a counter (the "water level") increases by one per admitted request and leaks at rate r; a request that would overflow capacity is rejected immediately. The meter form makes the same admit/reject decisions as a token bucket with the same rate and capacity.

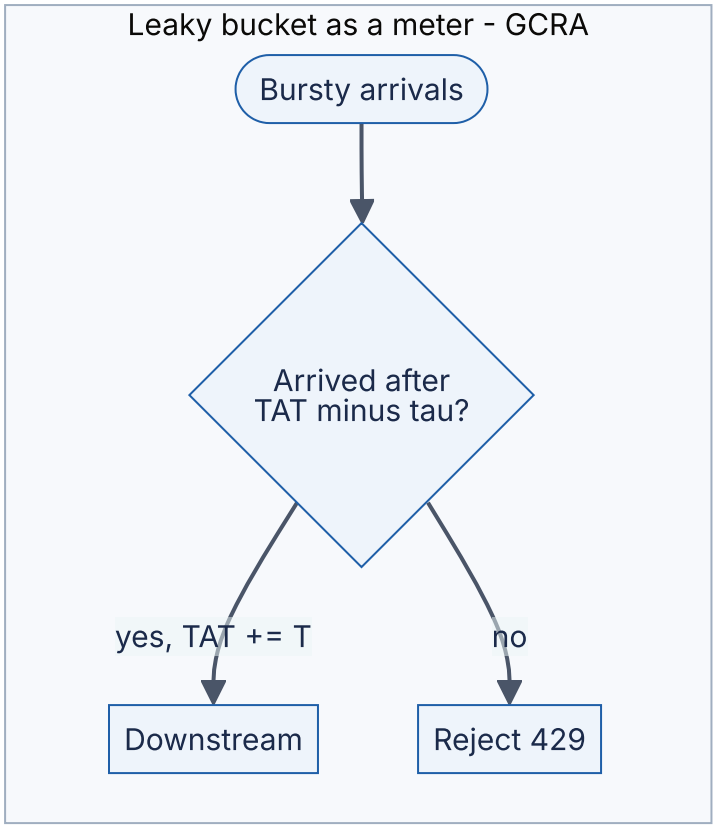
First-principles explanation

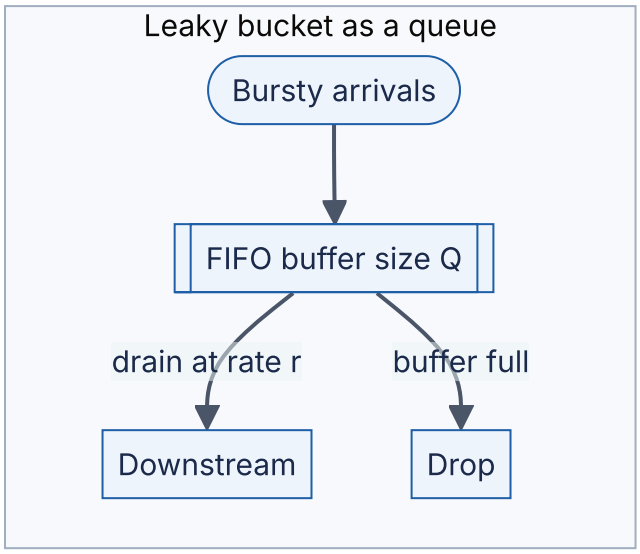
Water poured unevenly into a bucket with a hole leaks out at a steady drip; pour too fast and it overflows. In the **queue** form the water is the requests themselves: they wait and leave smoothly, but *delayed*. In the **meter** form the water is only a measurement: each request is admitted or rejected at once.

Why does this exist?

It comes from telecommunications, where switches policed and shaped fixed-size cells. For APIs, the queue form suits a downstream that needs smooth arrivals (a partner API allowing exactly 5 calls/s); the meter form needs only *one* number per key.

Figure 22.3 — Leaky bucket as a queue versus as a meter.





Reading the diagram. Left: arrivals are buffered in a FIFO (an in-memory storage boundary of size Q) and released asynchronously at rate r, adding up to Q/r latency; a full buffer drops arrivals. Right: nothing is buffered; each arrival is checked synchronously against one stored timestamp (TAT, below) and admitted or rejected. The queue *shapes*, the meter *polices*. Failure points: queued requests are lost on a crash and may time out while waiting. Scaling: the meter's single number per key shards trivially.

22.5.2 GCRA: the leaky bucket as a single timestamp

The **Generic Cell Rate Algorithm (GCRA)** is the meter form defined for ATM (Asynchronous Transfer Mode) networks in ITU-T Recommendation I.371 and the ATM Forum Traffic Management specification [ITU-T I.371; ATM Forum TM 4.0]. Of its two equivalent formulations, the "virtual scheduling" form is the one adapted for APIs: instead of counting tokens, it remembers when the next request is *due* at exactly the allowed rate.

FORMULA

GCRA (virtual scheduling form)

$$T = \frac{1}{r}, \qquad \tau = (b-1)\,T, \qquad \text{admit at time } t \;\Leftrightarrow\; t \geq \mathrm{TAT} - \tau, \qquad \text{then } \mathrm{TAT} \leftarrow \max(\mathrm{TAT}, t) + T$$

Symbol	Meaning	Units
r	Sustained allowed rate	requests/s
T	Emission interval: ideal spacing between requests	s
b	Burst size (requests that may arrive back to back)	requests
τ	Tolerance: how early a request may be relative to its ideal schedule	s
TAT	Theoretical arrival time of the next request (the only stored state)	s (timestamp)
t	Arrival time of the current request	s (timestamp)

WORKED EXAMPLE

Worked example — GCRA with $r = 10$ RPS, $b = 20$.

1. $T = 1/10 = 100$ ms; $\tau = 19 \times 100 = 1,900$ ms.
2. An idle client sends 50 requests at $t = 0$. TAT starts at 0 (or is reset to t because $\max(\mathrm{TAT}, t)$ ignores a stale past TAT).
3. Request 1: $0 \geq 0 - 1.9$ ✓; TAT $= 0.1$ s. Request 20: TAT before it is 1.9; $0 \geq 1.9 - 1.9 = 0$ ✓; TAT becomes 2.0.
4. Request 21: $0 \geq 2.0 - 1.9 = 0.1$? No → reject. `Retry-After` $= \mathrm{TAT} - \tau - t = 0.1$ s.

Interpretation. Identical to the token bucket of §22.4. **Engineering implication.** One timestamp per key and no refill arithmetic make GCRA attractive in shared stores such as Redis.

```
        import random


        class GCRA:
            """Generic Cell Rate Algorithm, virtual-scheduling form. State per key: one
integer (TAT).
            Times are integer nanoseconds so that repeated additions do not drift."""
            def __init__(self, rate, burst):
                self.T = round(1e9 / rate)          # emission interval (ns): ideal gap
between requests
                self.tau = (burst - 1) * self.T     # tolerance (ns): how far ahead of
schedule we accept
                self.tat = None                     # theoretical arrival time of the
next request

            def try_acquire(self, now_s):
                now = round(now_s * 1e9)
                tat = now if self.tat is None else max(self.tat, now)
                if tat - now > self.tau:            # arrived earlier than TAT - tau:
non-conforming
                    return False
                self.tat = tat + self.T             # conforming: push the schedule one
interval out
                return True

            def retry_after(self, now_s):
                """Seconds until the next request would conform."""
                return max(0.0, (self.tat - self.tau) / 1e9 - now_s)


        def token_bucket_decisions(rate, capacity, times):
            """Reference token bucket (starts full) for comparison."""
            tokens, last, out = float(capacity), times[0], []
            for t in times:
                tokens = min(capacity, tokens + (t - last) * rate)
                last = t
                ok = tokens >= 1.0
                tokens -= 1.0 if ok else 0.0
                out.append(ok)
            return out


        if __name__ == "__main__":
            g = GCRA(rate=10, burst=20)
            burst = [g.try_acquire(0.0) for _ in range(50)]
            print(f"burst of 50 at t=0: admitted {sum(burst)}, retry after
{g.retry_after(0.0):.2f} s")

            # Equivalence check on 100,000 random arrivals (Poisson, average 14 RPS)
            rng = random.Random(7)
            times, t = [], 0.0
            for _ in range(100_000):
                t += rng.expovariate(14.0)
                times.append(t)
            g = GCRA(rate=10, burst=20)
            gcra = [g.try_acquire(x) for x in times]
```

```
    tb = token_bucket_decisions(10, 20, times)
    mismatches = sum(a != b for a, b in zip(gcra, tb))
    print(f"admitted: GCRA {sum(gcra)}, token bucket {sum(tb)}; "
          f"decisions that differ: {mismatches}")
    assert sum(burst) == 20 and mismatches == 0
```

Output:

```
burst of 50 at t=0: admitted 20, retry after 0.10 s
admitted: GCRA 71137, token bucket 71137; decisions that differ: 0
```

What to notice:

- Identical decisions on 100,000 arrivals: GCRA with $\tau = (b-1)/r$ *is* a token bucket of capacity b.
- Integer nanoseconds matter: a floating-point version admitted only 19, because repeated addition of 0.1 slightly exceeded 1.9.
- `max(self.tat, now)` stops an idle client banking more than the burst.

Alternatives and trade-offs

Use the **queue form** only when the downstream needs smooth arrivals and callers can wait, keeping Q/r below their timeout (§22.10); use the **meter form** for API admission.

> COMMON MISTAKE
>
> **Mistake — Treating "leaky bucket" and "token bucket" as rivals.** The meter form and the token bucket are the same policy. The real distinction is *shaping* (queue and delay) versus *policing* (admit or reject); ask which one is meant.

22.6 Fixed window

22.6.1 Counting per calendar window

> DEFINITION
>
> **Definition — Fixed window counter.** A limiter that divides time into aligned windows of length W (for example each clock minute), keeps one counter per key per window, and admits a request if the current window's counter is below the limit L. The counter resets at each window boundary.

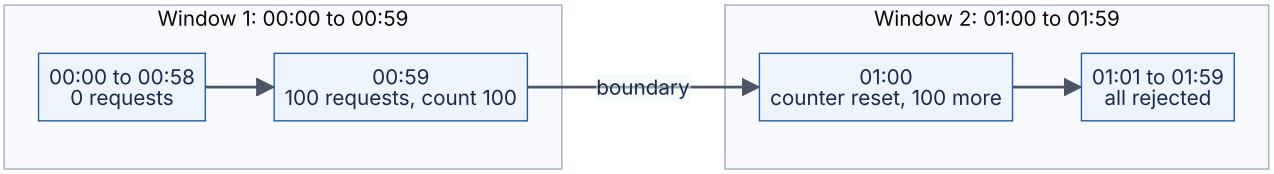
How it works internally

1. Window index $= \lfloor t/W \rfloor$; counter key = `key:window_index`.

2. Increment the counter atomically (in Redis: `INCR` plus `EXPIRE` set to W).
3. Admit if the new value is at most L.

It is simple (one integer, one atomic increment, automatic cleanup), but suffers the **boundary burst**: a full quota at the end of one window and another at the start of the next.

Figure 22.4 — The fixed-window boundary burst.



Reading the diagram. Two one-minute windows, limit 100 each. The client sends 100 requests in the last second of window 1 (counter reaches 100); the counter, a per-window storage record, is discarded at the boundary, so 100 more in the first second of window 2 are also admitted. The failure point is the downstream service, which receives 200 requests in two seconds; clients whose cron jobs fire on the minute produce such synchronized bursts.

FORMULA

Worst-case burst of a fixed window

$$A_{\max}(W) = 2L \qquad \text{peak short-term rate} \approx \frac{2L}{\delta}$$

Symbol	Meaning	Units
L	Limit per window	requests
W	Window length	s
$A_{\max}(W)$	Most requests admitted in some interval of length W that straddles a boundary	requests
δ	Duration of the straddling burst (can be arbitrarily small)	s

WORKED EXAMPLE

Worked example — 100 requests per minute.

1. Intended average rate: $100/60 = 1.67$ RPS.
2. Client sends 100 requests during 00:59.00–00:59.99 and 100 during 01:00.00–01:00.99. All 200 are admitted (the code in §22.7 confirms it).
3. In the 2 s interval the client achieved $200/2 = 100$ RPS, which is $100/1.67 = 60$ times the intended average, and any 60 s interval containing both bursts held $2L = 200$ requests.

Interpretation. The limit holds per calendar window, not per sliding minute.
Engineering implication. Use fixed windows only for coarse daily or monthly quotas; protect backends with a token bucket or sliding window.

22.7 Sliding window log and sliding window counter

22.7.1 Sliding window log

DEFINITION

Definition — Sliding window log. A limiter that stores the timestamp of every admitted request per key and admits a new request only if fewer than L timestamps fall in the interval $(t - W, t]$.

The log is exact (in Redis, a sorted set per key) but costs up to L timestamps per key: 1 million keys at 1,000 requests per hour need $10^6 \times 1,000 \times 8$ bytes $= 8$ GB raw, more with sorted-set overhead. Use it when L is small (5 login attempts per 15 minutes) or exactness is contractual.

22.7.2 Sliding window counter

DEFINITION

Definition — Sliding window counter. A limiter that keeps fixed-window counters for the current and previous windows and estimates the count over the last W seconds by weighting the previous window's count by the fraction of it that still overlaps the sliding interval.

First-principles explanation

You made 84 calls last hour and 36 so far this hour; it is quarter past. If last hour's calls were spread evenly, three-quarters (63) fall inside the sliding hour, so the estimate is $63 + 36 = 99$. The counter assumes exactly this uniformity.

FORMULA

Sliding window counter estimate

$$\hat{N}(t) = C_{\text{prev}} \cdot \frac{W - e}{W} + C_{\text{curr}}, \qquad \text{admit if } \hat{N}(t) + 1 \le L$$

Symbol	Meaning	Units
$\hat{N}(t)$	Estimated requests in the sliding interval $(t - W, t]$	requests
C_{prev}	Count in the previous fixed window	requests
C_{curr}	Count so far in the current fixed window	requests
e	Time elapsed since the current window started	s
W	Window length	s
L	Limit per sliding window	requests

WORKED EXAMPLE

Worked example — $L = 100$ per minute, 15 s into the current minute.

1. $C_{\text{prev}} = 84$, $C_{\text{curr}} = 36$, $e = 15$ s, $W = 60$ s.
2. Overlap weight: $(60 - 15)/60 = 0.75$; weighted previous count: $84 \times 0.75 = 63$.
3. $\hat{N} = 63 + 36 = 99$. Admit test: $99 + 1 = 100 \le 100 \rightarrow$ admit; C_{curr} becomes 37.
4. Next request at the same instant: $\hat{N} = 63 + 37 = 100$; $100 + 1 > 100 \rightarrow$ reject.
5. When does the next request fit? The previous window's contribution falls by $84/60 = 1.4$ requests per second, so after $1/1.4 \approx 0.71$ s the estimate drops to 99. `Retry-After` rounds up to 1 s.
6. Memory: two counters plus a window start, about 16 bytes per key, versus up to 8,000 bytes for a log at $L = 1,000$; for 1 million keys, 16 MB versus 8 GB.

Interpretation. Two integers per key give a smooth sliding limit. **Engineering implication.** It is the usual choice for large-scale per-minute or per-hour limits; its error depends on how uneven traffic is within a window, measured below.

Code example

```
        import math
        import random
        from collections import deque


        class FixedWindow:
            def __init__(self, limit, window):
                self.limit, self.window = limit, window
                self.start, self.count = None, 0

            def try_acquire(self, now):
                start = math.floor(now / self.window) * self.window
                if start != self.start:                      # a new window: reset the
counter
                    self.start, self.count = start, 0
                if self.count < self.limit:
                    self.count += 1
                    return True
                return False


        class SlidingLog:
            def __init__(self, limit, window):
                self.limit, self.window, self.log = limit, window, deque()

            def try_acquire(self, now):
                while self.log and self.log[0] <= now - self.window:
                    self.log.popleft()                       # forget requests older
than one window
                if len(self.log) < self.limit:
                    self.log.append(now)
                    return True
                return False


        class SlidingWindowCounter:
            """Two counters per key; estimate = prev * (overlap fraction) + curr."""
            def __init__(self, limit, window):
                self.limit, self.window = limit, window
                self.start, self.curr, self.prev = 0.0, 0, 0

            def estimate(self, now):
                start = math.floor(now / self.window) * self.window
                if start != self.start:                      # roll forward
                    gap = round((start - self.start) / self.window)
                    self.prev = self.curr if gap == 1 else 0  # an idle gap > 1 window
empties both
                    self.curr, self.start = 0, start
                overlap = 1.0 - (now - start) / self.window  # share of previous window
still inside
                return self.prev * overlap + self.curr

            def try_acquire(self, now):
                if self.estimate(now) + 1 <= self.limit:     # never let the estimate
```

```
exceed the limit
                    self.curr += 1
                    return True
                return False


        def boundary_burst(limiter):
            """100 requests in the last second of a minute, 100 in the first second of
the next."""
            times = [59 + i / 100 for i in range(100)] + [60 + i / 100 for i in
range(100)]
            return sum(limiter.try_acquire(t) for t in times)


        if __name__ == "__main__":
            L, W = 100, 60.0
            print("Boundary burst (limit 100 per 60 s, 200 requests within 2 s):")
            for name, cls in [("fixed window", FixedWindow), ("sliding log",
SlidingLog),
                              ("sliding counter", SlidingWindowCounter)]:
                print(f"  {name:16s} admitted {boundary_burst(cls(L, W))}")

            # Worked example of section 22.7: prev = 84, curr = 36, 15 s into the window
            swc = SlidingWindowCounter(L, W)
            swc.start, swc.prev, swc.curr = 60.0, 84, 36
            print(f"\nestimate at t=75 s: {swc.estimate(75.0):.1f}; "
                  f"next two requests: {swc.try_acquire(75.0)},
{swc.try_acquire(75.0)}")

            # Accuracy against the exact sliding log: 4 hours of random traffic
            def worst_window(ts):                           # most admitted requests in
any 60 s span
                best, j = 0, 0
                for i, x in enumerate(ts):
                    while ts[j] <= x - W:
                        j += 1
                    best = max(best, i - j + 1)
                return best

            print("\npattern  RPS  offered  | log: admitted worst60 | counter: admitted
worst60 "
                  "| fixed: admitted worst60")
            for pattern in ("steady", "on/off"):            # on/off: 40 s on, 20 s
silent
                for rate in (1.5, 3.0):
                    rng, times, t = random.Random(42), [], 0.0
                    while t < 4 * 3600:
                        t += rng.expovariate(rate)
                        if pattern == "steady" or int(t // 20) % 3 != 2:
                            times.append(t)
                    row = []
                    for cls in (SlidingLog, SlidingWindowCounter, FixedWindow):
                        lim = cls(L, W)
                        ok = [x for x in times if lim.try_acquire(x)]
                        row.append(f"{len(ok):>6} {worst_window(ok):>4}")
                    print(f"{pattern:7s} {rate:4.1f} {len(times):>8}  |  " + "      |
".join(row))
```

```
            assert boundary_burst(FixedWindow(L, W)) == 200
            assert boundary_burst(SlidingLog(L, W)) == 100
```

Output:

```
Boundary burst (limit 100 per 60 s, 200 requests within 2 s):
  fixed window     admitted 200
  sliding log      admitted 100
  sliding counter  admitted 101

estimate at t=75 s: 99.0; next two requests: True, False

pattern  RPS  offered  | log: admitted worst60 | counter: admitted worst60 | fixed: ad-
mitted worst60
steady   1.5    21574  |   21034  100      |   21166  113      |   21383  115
steady   3.0    43246  |   23726  100      |   23683  105      |   24001  137
on/off   1.5    14342  |   14342   86      |   14339   85      |   14342   86
on/off   3.0    28717  |   23110  100      |   17845  100      |   23929  137
```

What to notice:

- **Boundary burst:** fixed admits 200, log 100, counter 101 (one slips in 0.6 s after the boundary).
- **Steady traffic:** the counter admits within 1% of the log; its worst real 60 s window is 105–113 (5–13% over) versus 137 for the fixed window.
- **Periodic traffic** (40 s on, 20 s off, in step with the window) breaks the uniformity assumption: at 3 RPS the counter admits 23% fewer than the log (17,845 versus 23,110).

INDUSTRY EXAMPLE

Cloudflare's rate limiting. Cloudflare described rate limiting for millions of domains with this two-counter approximation, chosen for its tiny per-key memory; its analysis of 400 million requests from 270,000 sources found 0.003% wrongly allowed or limited [Desgats 2017]. *Evidence: Primary* — [Desgats 2017]

22.7.3 Comparing the algorithms

Table 22.3 — Rate-limiting algorithms compared.

	Token bucket	Leaky bucket (queue)	GCRA (meter)	Fixed window	Sliding log	Sliding counter
What it is	Tokens refill at r up to b	FIFO drained at r	One time-stamp (TAT)	Counter per aligned window	Timestamp per request	Two window counters, weighted
State per key	2 numbers	Queue of up to Q requests	1 number	1 counter	Up to L timestamps	2 counters
Burst behavior	Up to b at once	Absorbs bursts by delaying	Up to b at once	Up to $2L$ across a boundary	Exactly L per window	About L; small overshoot
Accuracy	Exact for its definition	Exact; adds latency	Exact (= to-ken bucket)	Poor at boundaries	Exact	Approximate (uniformity assumption)
Rejects or delays?	Rejects	Delays, drops when full	Rejects	Rejects	Rejects	Rejects
Operational complexity	Low	Medium (queue, timeouts)	Low	Lowest	Medium (memory)	Low
Choose when	API rate plus burst	Downstream needs smooth flow	Minimal shared state	Coarse daily or monthly quotas	Small exact limits	Large-scale per-minute limits

22.8 Distributed rate limiting

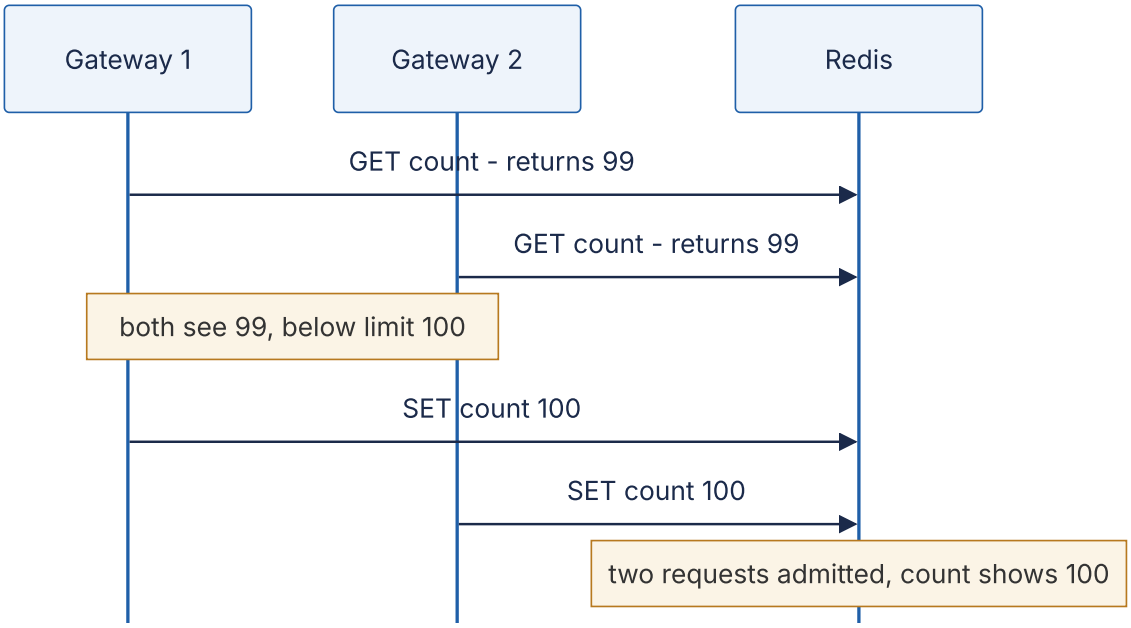
22.8.1 The problem

N gateway nodes must enforce *one* limit per key, but each sees only part of the traffic, rarely split evenly (long-lived connections concentrate a client on few nodes, §19.6), and every coordination step either adds latency or lets counts go stale.

22.8.2 Centralized counters: Redis with Lua

The simplest correct design keeps state in a shared store, usually Redis, partitioned by key. The naive implementation races.

Figure 22.5 — The read-modify-write race in a shared counter.



Reading the diagram. Two gateways issue synchronous `GET`s for the same key, both see 99, both admit and both write 100: the limit is exceeded and the stored count is wrong. The failure point is the gap between read and write, which more nodes and hotter keys widen. The fix moves the decision into the store so read, decide and write are one atomic step.

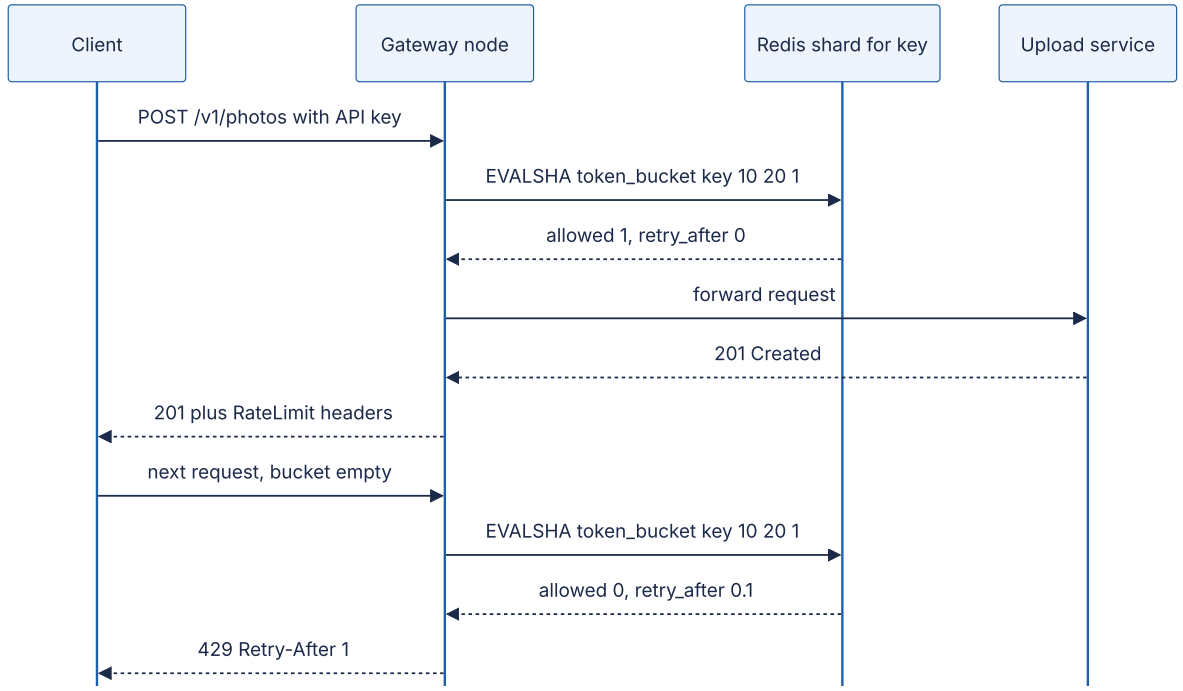
Redis executes a Lua script atomically: no other command runs on that shard while the script executes [Redis Docs: Lua scripting]. The token bucket of §22.4 becomes:

```
        -- Token bucket in Redis. KEYS[1] = bucket key; ARGV = rate (tokens/s), capac-
ity, cost
        local rate     = tonumber(ARGV[1])
        local capacity = tonumber(ARGV[2])
        local cost     = tonumber(ARGV[3])
        local t        = redis.call('TIME')                -- Redis server clock, not
the caller's
        local now      = tonumber(t[1]) + tonumber(t[2]) / 1000000
        local state    = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
        local tokens   = tonumber(state[1]) or capacity     -- unknown key: start with a
full bucket
        local ts       = tonumber(state[2]) or now
        tokens = math.min(capacity, tokens + math.max(0, now - ts) * rate)
        local allowed, retry_after = 0, 0
        if tokens >= cost then
          tokens, allowed = tokens - cost, 1
        else
          retry_after = (cost - tokens) / rate
        end
        redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
        redis.call('PEXPIRE', KEYS[1], math.ceil(capacity / rate * 1000) + 1000)  --
idle keys vanish
        return {allowed, tostring(retry_after)}            -- Lua numbers would be trun-
cated to integers
```

What to notice:

- Tested on Redis 7.0: 50 pipelined `EVALSHA` calls on a fresh key (`10 20 1`) returned 20 admits and 30 rejections.
- `TIME` uses the server's clock, so skewed gateway clocks (§24.9) cannot corrupt buckets; Redis 5+ replicates script *effects*, which makes this safe.
- Idle keys expire. In Redis Cluster all keys a script touches must share a shard (use a hash tag such as `{tenant42}`).

Figure 22.6 — Centralized rate limiting with an atomic script.



Reading the diagram. The gateway makes one synchronous round trip to the Redis shard owning the key (§14.8); the script runs atomically and returns a decision and retry hint. Admitted requests go to the upload service; rejected ones get 429 without touching it. The Redis shard is the storage boundary; failover may reset some buckets (usually acceptable). Failure points: Redis latency on every request, a failed shard blocking its keys, and hot keys. Scaling: add shards; pre-aggregate hot keys locally (§22.8.3).

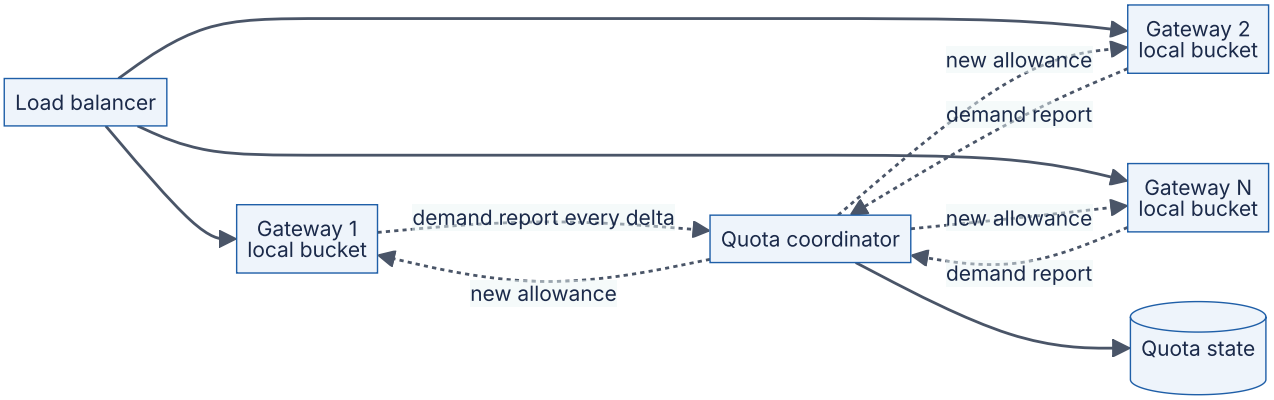
22.8.3 Local counting with synchronization, and quota distribution

Two families avoid a remote call on every request.

1. **Local counting with periodic synchronization.** Nodes count locally and exchange counts every Δ seconds, deciding in between on a stale global view.
2. **Quota distribution (leases).** A coordinator divides the limit into per-node allowances in proportion to recent demand, re-divided periodically; nodes enforce them with local buckets. Gateways offer related building blocks: Envoy, for example,

supports an external global rate-limit service and a local token-bucket filter [Envoy Docs: rate limiting].

Figure 22.7 — Local enforcement with synchronization or quota leases.



Reading the diagram. Each gateway decides synchronously against a local bucket, so the request path has no remote call. Asynchronously (dotted), nodes report demand to a coordinator, which persists state and pushes back allowances summing to the global limit (in the sync variant, nodes exchange counts instead). Failure points: an unreachable coordinator leaves nodes on their last allowance; allowances lag demand by one period. Scaling: coordination grows with N per period, not with request rate.

Mathematical foundations

FORMULA

Overshoot of local counting with synchronization interval Δ

$$E_{\max} \approx \lambda \cdot \Delta \qquad M_{\text{sync}} = \frac{N(N-1)}{\Delta} \text{ (all-to-all)} \quad \text{or} \quad \frac{2N}{\Delta} \text{ (via a store or coordinator)}$$

Symbol	Meaning	Units
$E_{\max}$	Worst-case requests admitted beyond the limit per window	requests
λ	Offered rate for the key across all nodes	requests/s
Δ	Synchronization interval	s
N	Number of gateway nodes	nodes
M_{sync}	Coordination messages per second (each message can carry all keys)	messages/s

WORKED EXAMPLE

Worked example — 10 gateways, limit 1,000 requests/s for one tenant, offered 3,000 RPS.

1. $\Delta = 100$ ms: $E_{\max} \approx 3,000 \times 0.1 = 300$ excess requests per second, up to 30% over. Messages: $10 \times 9/0.1 = 900$/s all-to-all.
2. $\Delta = 10$ ms: $E_{\max} \approx 30$ (3%); messages $= 9,000$/s.
3. Centralized Redis: zero overshoot, but one round trip per request: 3,000 Redis calls/s for this tenant alone, and roughly 0.2–1 ms added to each request inside a datacenter (§7.15).
4. Simulation (below): with $\Delta = 100$ ms the measured overshoot averages 147/s (bound 300); with $\Delta = 10$ ms, 9.5/s (bound 30).

Interpretation. Accuracy, latency and coordination traffic form a triangle; pick two.
Engineering implication. Centralize contractual limits on costly operations; use local-plus-sync or quotas on hot paths where a few percent of overshoot is harmless.

Code example: a Redis-free simulation

```
        """Simulate one tenant's global limit of 1,000 requests per 1-second window, en-
forced by
        10 gateway nodes that receive skewed traffic. No Redis needed: the 'central
store' is a dict."""
        import random

        LIMIT, NODES, SECONDS = 1000, 10, 20
        WEIGHTS = [0.5] + [0.5 / 9] * 9                 # node 0 gets half the tenant's
traffic


        def arrivals(offered_rps, seed=1):
            rng, events = random.Random(seed), []
            for node, w in enumerate(WEIGHTS):
                t = 0.0
                while True:
                    t += rng.expovariate(offered_rps * w)
                    if t >= SECONDS:
                        break
                    events.append((t, node))
            return sorted(events)


        def static_split(events):
            """Each node enforces LIMIT/N locally. Zero coordination."""
            counts = {}
            def allow(t, node):
                k = (int(t), node)
                if counts.get(k, 0) < LIMIT / NODES:
                    counts[k] = counts.get(k, 0) + 1
                    return True
                return False
            return allow, lambda: 0


        def centralized(events):
            """Every decision is an atomic increment on one shared counter (Redis + Lua
in production)."""
            counts, calls = {}, [0]
            def allow(t, node):
                calls[0] += 1                           # one network round trip per
request
                k = int(t)
                if counts.get(k, 0) < LIMIT:
                    counts[k] = counts.get(k, 0) + 1
                    return True
                return False
            return allow, lambda: calls[0]


        def local_with_sync(events, sync_interval):
            """Nodes count locally and exchange counts every sync_interval seconds."""
            local = [0] * NODES                         # this window's admits per node
(true values)
```

```
            seen = [0] * NODES                          # each node's last snapshot of
the others' total
            state = {"window": 0, "next_sync": sync_interval, "msgs": 0}
            def allow(t, node):
                while t >= state["next_sync"]:          # periodic all-to-all exchange
of counters
                    total = sum(local)
                    for i in range(NODES):
                        seen[i] = total - local[i]
                    state["msgs"] += NODES * (NODES - 1)
                    state["next_sync"] += sync_interval
                if int(t) != state["window"]:           # new window: everyone resets
                    state["window"] = int(t)
                    local[:] = [0] * NODES
                    seen[:] = [0] * NODES
                if seen[node] + local[node] < LIMIT:    # decide on a possibly stale
global view
                    local[node] += 1
                    return True
                return False
            return allow, lambda: state["msgs"]


        def quota_leases(events):
            """A coordinator re-divides the limit once per window in proportion to each
node's
            demand in the previous window; nodes then enforce their quota locally."""
            quota = [LIMIT / NODES] * NODES
            demand, used = [0] * NODES, [0] * NODES
            state = {"window": 0, "msgs": 0}
            def allow(t, node):
                if int(t) != state["window"]:           # window boundary: report de-
mand, get quotas
                    total = sum(demand) or 1
                    for i in range(NODES):
                        quota[i] = LIMIT * demand[i] / total
                    state["msgs"] += 2 * NODES          # N reports in, N quota grants
out
                    state["window"] = int(t)
                    demand[:], used[:] = [0] * NODES, [0] * NODES
                demand[node] += 1
                if used[node] + 1 <= quota[node]:
                    used[node] += 1
                    return True
                return False
            return allow, lambda: state["msgs"]


        def run(name, factory, events):
            allow, cost = factory(events)
            per_window = [0] * SECONDS
            for t, node in events:
                if allow(t, node):
                    per_window[int(t)] += 1
            steady = per_window[1:]                     # skip the first window (warm-
up)
            print(f"{name:26s} mean {sum(steady)/len(steady):7.1f}/s  max
```

```
{max(steady):5d}  "
                  f"coordination msgs/s {cost()/SECONDS:9.0f}")
            return per_window


        if __name__ == "__main__":
            for offered in (1500, 3000):
                events = arrivals(offered)
                print(f"Offered {offered} RPS, global limit {LIMIT}/s, {NODES} nodes,
node 0 gets 50%")
                s = run("static split (L/N each)", static_split, events)
                c = run("centralized counter", centralized, events)
                y100 = run("local + sync every 100 ms", lambda e: local_with_sync(e,
0.1), events)
                y10 = run("local + sync every 10 ms", lambda e: local_with_sync(e,
0.01), events)
                q = run("demand-weighted quotas", quota_leases, events)
                print()
                assert max(c) <= LIMIT and max(s) <= LIMIT and max(q) <= LIMIT
                assert max(y100) > LIMIT >= max(c)       # stale views overshoot; exact
counting does not
```

Output:

```
Offered 1500 RPS, global limit 1000/s, 10 nodes, node 0 gets 50%
static split (L/N each)    mean   840.1/s  max   895  coordination msgs/s         0
centralized counter        mean  1000.0/s  max  1000  coordination msgs/s      1495
local + sync every 100 ms  mean  1041.2/s  max  1079  coordination msgs/s       896
local + sync every 10 ms   mean  1005.5/s  max  1010  coordination msgs/s      8996
demand-weighted quotas     mean   995.4/s  max   997  coordination msgs/s        19

Offered 3000 RPS, global limit 1000/s, 10 nodes, node 0 gets 50%
static split (L/N each)    mean  1000.0/s  max  1000  coordination msgs/s         0
centralized counter        mean  1000.0/s  max  1000  coordination msgs/s      2979
local + sync every 100 ms  mean  1147.4/s  max  1170  coordination msgs/s       896
local + sync every 10 ms   mean  1009.5/s  max  1020  coordination msgs/s      8996
demand-weighted quotas     mean   994.5/s  max   996  coordination msgs/s        19
```

What to notice:

- **Static split under-admits under skew:** at 1,500 RPS the tenant gets only 840/s because node 0 sees 750 RPS but may admit 100.
- **Synchronization overshoots** in proportion to Δ, within $\lambda\Delta$.
- **Quotas** are accurate with 19 messages/s only because demand is stationary.

22.8.4 Comparison

Table 22.4 — Distributed rate-limiting strategies.

Strategy	Added latency	Accuracy	Coordination cost	Failure behavior	Choose when
Static split L/N	None	Under-admits with skewed traffic	None	Independent	Even traffic, rough limits
Centralized (Redis + Lua)	One round trip per request	Exact	One call per request	Store outage blocks the decision	Contractual or costly limits
Local + periodic sync	None	Overshoot up to $\lambda\Delta$	$O(N^2/\Delta)$ or $O(N/\Delta)$	Stale but working	Very high rates, tolerant limits
Quota distribution (leases)	None	Exact sum; lags demand shifts	$O(N/\Delta)$	Keeps last allowance	Large fleets, stable demand
Hybrid: local bucket batches reservations from a central store	Rare round trips	Bounded by batch size	One call per batch	Degrades to local	Hot keys at very high rates

22.8.5 Failure scenarios

Table 22.5 — Distributed limiter failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
All requests fail when Redis is down	Limiter fails *closed*	Limiter error spike	Fail open with local limits	Per limit: open for quotas, closed for login limits
One shard at 100% CPU	Hot key (one huge tenant)	Per-shard skew	Split key or pre-aggregate locally	Hot-key detection (§17.6, §20.15)
Buckets drift	Gateway clocks skewed	Inconsistent `Retry-After`	Server-side time	Redis `TIME`
Limiter adds p99 latency	Remote call per request	Gateway p99 tracks Redis p99	Short limiter timeout, then fail open	Cache “deny” until `Retry-After`

22.9 Throttling and load shedding

22.9.1 Goodput and overload

DEFINITION

Definition — Load shedding and throttling. Load shedding is a server rejecting some requests early and cheaply (usually with 503) because accepting them would push it past capacity, regardless of whether each client is within its quota. **Throttling** is slowing a client down, by delaying or rejecting its requests; rate limiting is the quota-based form. **Goodput** is the rate of requests completed *usefully*: correctly and before the client gave up.

A kitchen that accepts every order on a busy night serves every dish late; diners leave before food arrives, so it works flat out producing nothing useful. Turning new arrivals away keeps the seated diners served. The AWS Builders' Library describes this failure: once latency exceeds client timeouts, a server can run at maximum throughput while goodput falls toward zero [Yanacek: load shedding].

FORMULA

Time until goodput collapses without shedding

$$q(t) = (\lambda - \mu)\,t, \qquad w(t) = \frac{q(t)}{\mu}, \qquad t_{\text{collapse}} = \frac{\mu\, T_{\text{out}}}{\lambda - \mu} \quad (\lambda > \mu)$$

Symbol	Meaning	Units
λ	Offered request rate	requests/s
μ	Server capacity (maximum completion rate)	requests/s
$q(t)$	Queue length after t seconds of overload (unbounded FIFO)	requests
$w(t)$	Queueing delay seen by a request arriving at t	s
T_{out}	Client timeout	s
t_{collapse}	Time after which every completed request is already too late	s

WORKED EXAMPLE

Worked example — PhotoShare upload service, $\mu = 1,000$ RPS, clients time out after 1 s, a spike brings $\lambda = 1,500$ RPS.

1. The queue grows at $1,500 - 1,000 = 500$ requests per second.
2. $t_{\text{collapse}} = 1,000 \times 1/500 = 2$ s. After two seconds the queue holds 1,000 requests and every new request waits 1 s, the full timeout.
3. From then on the server still completes 1,000 requests/s, but goodput is roughly zero. Worse, clients retry, raising λ further (§22.12).
4. With shedding (a queue bounded at 200 requests, i.e. at most $200/1,000 = 200$ ms of waiting), the server rejects the excess 500 RPS in microseconds and goodput stays at 1,000 RPS.

Interpretation. Unbounded queueing turns a 50% overload into a total outage in two seconds. **Engineering implication.** Bound queues by *time*, and make rejection far cheaper than service.

22.9.2 Priority-based shedding

An upload or checkout matters more than a recommendations prefetch. **Priority-based shedding** tags each request with a **criticality** class at the edge, propagates it, and sheds the lowest class first. Google's SRE book describes four classes: `CRITICAL_PLUS`, `CRITICAL`, `SHEDDABLE_PLUS` and `SHEDDABLE`, propagated automatically with RPCs [Beyer et al. 2016, ch. 21]. Stripe's worker-utilization load shedder similarly sheds test-mode and other lower-priority traffic before critical API methods [Tarjan 2017].

Clients can shed too. In the SRE book's **client-side adaptive throttling**, each client tracks requests sent and accepted over the last two minutes and drops requests locally as rejections grow.

FORMULA

Client-side adaptive throttling (Google SRE)

$$P_{\text{reject}} = \max\left(0, \frac{\text{requests} - K \cdot \text{accepts}}{\text{requests} + 1}\right)$$

Symbol	Meaning	Units
requests	Requests the client attempted in the recent window	count
accepts	Requests the backend accepted (did not reject for overload)	count
K	Multiplier; how many requests per accept the client may still send (typically 2)	dimensionless
P_{reject}	Probability the client rejects a new request locally, without sending it	dimensionless

WORKED EXAMPLE

Worked example — A client in a two-minute window sent 10,000 requests and 4,000 were accepted, $K = 2$.

1. $K \times \text{accepts} = 8,000$.
2. $P_{\text{reject}} = (10,000 - 8,000)/10,001 = 0.20$.
3. The client now drops 20% of new requests locally at zero network cost. If the backend recovers and accepts everything, $\text{requests} < 2 \times \text{accepts}$ and P_{reject} returns to 0.

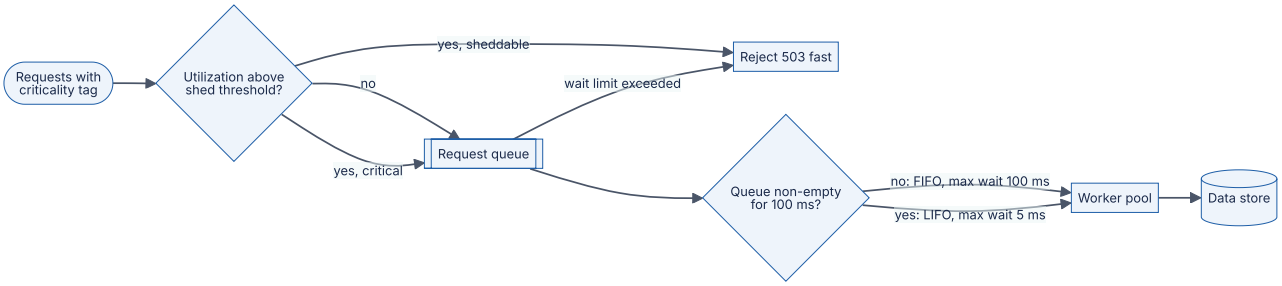
Interpretation. The client sends about K times what the backend accepts, enough to detect recovery. **Engineering implication.** Lower K sheds harder but recovers slower; $K = 2$ is a sensible default.

22.9.3 Controlling queue delay: CoDel and adaptive LIFO

Queue *length* is a poor overload signal; queue *delay* is better. **CoDel** (Controlled Delay), designed by Nichols and Jacobson for network routers, watches the time packets spend in the queue (their *sojourn time*). If the *minimum* sojourn time over an interval (100 ms by default) stays above a target (5 ms), the queue is a standing queue rather than a passing burst, and CoDel starts dropping, with the gap between drops shrinking as $\text{interval}/\sqrt{n}$ for the n-th drop: 100 ms, 70.7 ms, 57.7 ms, 50 ms, until the delay falls below target [Nichols and Jacobson 2012].

Facebook adapted this to RPC queues with **adaptive LIFO** [Maurer 2015]: when the queue has not been empty for *N* ms (100 ms), requests may wait at most *M* ms (5 ms), and the queue switches to **LIFO**, because the newest request's client is the one most likely still waiting.

Figure 22.8 — Overload control at a service: priority shedding and an adaptive queue.



Reading the diagram. Requests carry a criticality tag. Above a utilization threshold, sheddable requests get a fast 503 while critical ones are queued. The queue is FIFO while it drains regularly and LIFO with a 5 ms wait limit once it has stayed non-empty for 100 ms; requests over their limit are rejected. Workers call the data store synchronously. Failure points: tagging everything critical, or a threshold above where latency explodes. Scaling: all controls are per instance.

COMMON MISTAKE

Mistake — "Autoscaling makes load shedding unnecessary." Instances take minutes to start; a queue passes client timeouts in seconds (§22.9.1). You need both.

22.10 Backpressure

DEFINITION

Definition — Backpressure. A flow-control mechanism in which a slower downstream component signals upstream components to reduce their sending rate, by blocking, by advertising available capacity, or by rejecting, so that work in progress stays bounded instead of piling up in unbounded buffers.

When a packing station falls behind, the assembly line stops feeding it rather than heaping goods on the floor (an unbounded queue). Backpressure is that stop signal traveling upstream.

TCP is the canonical example (§7.7.6): when the receiver stops reading, its advertised window drops to zero and the sender's `write()` blocks. Systems break the chain by in-

serting **unbounded** buffers (a list a handler appends to, a task per message), turning backpressure into memory growth and latency.

Three rules give you backpressure at the application level:

1. **Bound every queue,** and size the bound from the latency you can tolerate, not from available memory (formula below).
2. **Decide what happens when it is full:** block the producer (pipelines inside one process), reject (request/response APIs: 503 or 429), or drop the oldest or least important (telemetry, video frames).
3. **Signal demand explicitly across asynchronous boundaries.** The **Reactive Streams** specification standardizes this for asynchronous stream libraries on the JVM: a subscriber calls `request(n)` to say it can accept *n* more items, and a publisher may never send more than has been requested [Reactive Streams 2015]. Java 9's `java.util.concurrent.Flow` adopted the same interfaces, and gRPC and HTTP/2 use per-stream flow-control windows for the same purpose.

Figure 22.9 — Backpressure through a processing pipeline.



Reading the diagram. Items flow left to right through bounded in-memory buffers (100 items) to a sink that writes to a database. Demand flows right to left as asynchronous `request(n)` signals (dotted). If the database slows, the sink requests less, buffers fill, and the producer stops being asked, so at most about 200 items are in flight. Failure point: any component accepting data without demand. Scaling: add parallel sinks; demand rises automatically.

FORMULA

Bounding a queue by time (Little's Law)

$$Q_{\max} = \mu \times D_{\text{queue}}$$

Symbol	Meaning	Units
$Q_{\max}$	Maximum queue length	items
μ	Rate at which the consumer drains the queue	items/s
D_{queue}	Maximum acceptable time an item may wait in the queue	s

WORKED EXAMPLE

Worked example — Sizing the upload service's request queue. Consider a single upload cell (half the fleet of §22.9's example) whose workers complete $\mu = 500$ requests/s; the gateway's timeout leaves at most 200 ms for queueing.

1. $Q_{max} = 500 \times 0.2 = 100$ requests.
2. A request arriving behind 100 others waits $100/500 = 0.2$ s, the full budget; request 101 should be rejected immediately rather than queued to time out.
3. If workers slow to $\mu = 250$/s (a slow database), the same 100-item queue now means 400 ms of waiting. Bound queues by *measured wait time* (as CoDel does) rather than by item count when service time varies.

Interpretation. A queue longer than $\mu\times$ timeout holds only doomed requests. **Engineering implication.** Framework default queue sizes are usually far too large for interactive services.

22.11 Timeouts and deadlines

DEFINITION

Definition — Timeout and deadline. A **timeout** is a relative duration after which a caller stops waiting for one operation (for example 300 ms). A **deadline** is an absolute point in time by which the *whole* request must complete; it is passed along the call chain so that every downstream call knows how much time remains.

Without a timeout, a caller waits forever on a hung dependency, holding a thread and a connection; by Little's Law (§4.11) held resources grow until the caller itself stops. Every network call needs connect and request timeouts.

Choosing a timeout. Set it from the dependency's measured latency distribution, not from intuition. The AWS Builders' Library recommends choosing an acceptable rate of false timeouts (for example 0.1%) and using the corresponding latency percentile (p99.9) of the downstream service, plus padding for network variance [Brooker: timeouts and retries].

Why deadlines must propagate. Suppose each hop independently uses a 1 s timeout. The user's app gives up after 1 s, but a service three hops down started 600 ms late and keeps working for its full second, then the database query beneath it keeps running too. All of that work is wasted. With **deadline propagation**, the edge sets an absolute deadline and each hop passes the *remaining* time downstream; gRPC does this automatically, sending the remaining time in the `grpc-timeout` header and failing calls with `DEADLINE_EXCEEDED` once it passes [gRPC Docs: deadlines]. Over HTTP you carry it in a

header of your own. Each service should check the remaining budget before starting expensive work and refuse work it cannot finish in time [Beyer et al. 2016, ch. 22].

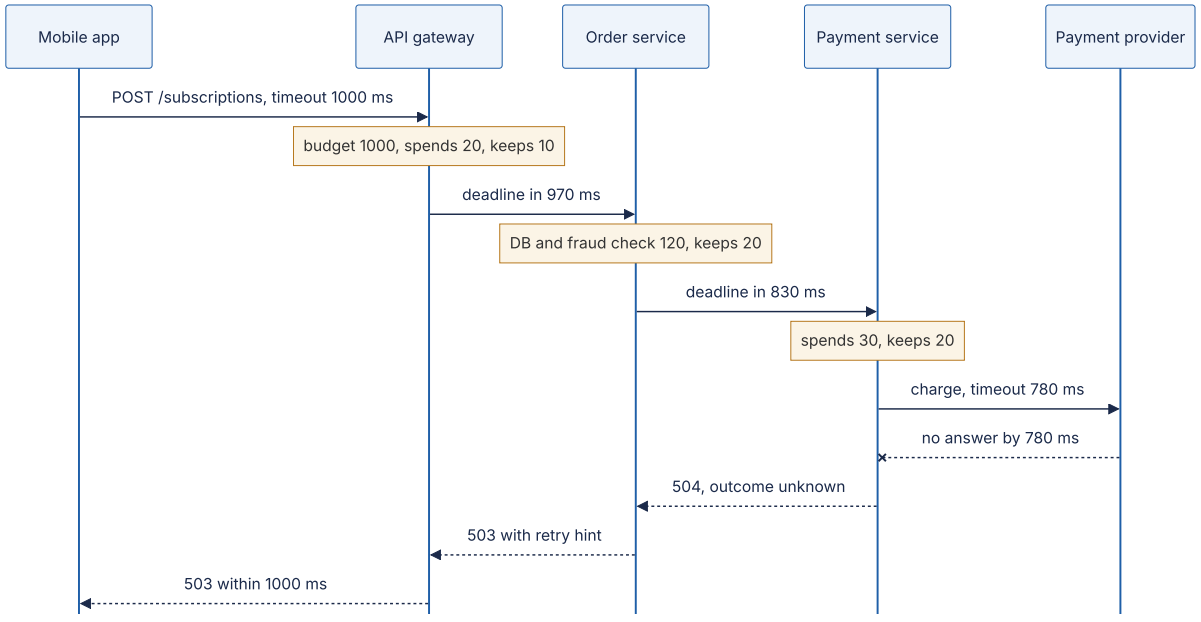
FORMULA

Timeout budget along a call chain

$$B_{i+1} = B_i - e_i - m_i, \qquad \text{start the call only if } B_{i+1} > \hat{s}_{i+1}$$

Symbol	Meaning	Units
B_i	Remaining budget when hop i receives the request	ms
e_i	Time hop i spends before calling hop $i+1$ (local work, earlier calls)	ms
m_i	Margin hop i reserves to build and return its own response	ms
B_{i+1}	Budget passed to hop $i+1$	ms
$\hat{s}_{i+1}$	Typical latency of hop $i+1$ (for example its p50)	ms

Figure 22.10 — Deadline propagation through the PhotoShare Pro purchase path.



Reading the diagram. The app waits at most 1,000 ms. Each hop subtracts elapsed time and a response margin before passing the budget on: 970, 830, then 780 ms for the external provider. All calls are synchronous. When the provider misses its budget, the payment service reports an *unknown outcome*, and every hop still has its margin to answer before the app gives up. The provider's state is the critical storage boundary, which is why re-

tries need an idempotency key (§22.15). Budgets are per request and need no shared state.

WORKED EXAMPLE

Worked example — Budget for the purchase path, client deadline 1,000 ms.

1. Gateway: $B_1 = 1,000$; $e_1 = 20$ (authentication, rate-limit check), $m_1 = 10 \rightarrow B_2 = 970$ ms.
2. Order service: $e_2 = 120$ (database read, fraud check), $m_2 = 20 \rightarrow B_3 = 830$ ms.
3. Payment service: $e_3 = 30$, $m_3 = 20 \rightarrow B_4 = 780$ ms for the provider call. The provider's p50 is 300 ms and p99.9 is 900 ms, so $B_4 > \hat{s}_4$ and the call starts.
4. Contrast with fixed per-hop timeouts of 1 s, 1 s and 2 s: when the provider hangs, the payment service, which started the call at 170 ms, waits until 2,170 ms: 1,170 ms after the app has given up; every thread on the path is held for work nobody receives.

Interpretation. The provider gets 78% of the user's budget, the most that is consistent with the earlier hops. **Engineering implication.** Propagate absolute deadlines (or remaining durations), subtract as you go, and make inner timeouts shorter than outer ones. Note the provider's p99.9 (900 ms) exceeds its budget, so roughly 0.1–1% of purchases will hit the deadline; either raise the client budget or make the purchase asynchronous (accept with 202 and confirm by push).

COMMON MISTAKE

Mistake — "A timeout means the operation failed." It means *you do not know*. The request may never have arrived, may have failed, or may have succeeded with the response lost. For reads this ambiguity is harmless; for writes it is the central problem that idempotency keys solve (§22.15).

22.12 Retries with exponential backoff and jitter

22.12.1 When to retry

Retries turn transient failures (lost packets, restarts, elections) into successes, but each is *extra load on a system that just failed*. Retry only when all hold:

1. The error is **retryable**: timeouts, connection resets, 429, 502, 503, 504, gRPC `UNAVAILABLE`. Never retry 400, 401, 403, 404 or 422; they will fail again.
2. The operation is **idempotent** or carries an idempotency key (§22.15).
3. **Time remains** in the deadline (§22.11).

4. The **retry budget** is not exhausted (§22.12.4).

22.12.2 Exponential backoff and jitter

Exponential backoff doubles the wait after each failure, up to a cap. Clients that failed together still retry in synchronized waves; **jitter** randomizes the wait to spread them. Brooker's analysis for the AWS Architecture Blog compared three variants [Brooker 2015]:

FORMULA

Backoff with jitter (attempt $k = 1, 2, \ldots$)

$$s_k = \min(\text{cap}, \text{base} \cdot 2^k)$$

full: $U(0, s_k)$ equal: $\frac{s_k}{2} + U(0, \frac{s_k}{2})$ decorrelated: $\min(\text{cap}, U(\text{base}, 3 w_{k-1}))$

Symbol	Meaning	Units
k	Retry number	count
base	Initial backoff unit	ms
cap	Maximum wait	ms
s_k	Exponential backoff ceiling for retry k	ms
$U(a, b)$	A uniformly random value between a and b	ms
w_{k-1}	The previous wait for decorrelated jitter ($w_0 = \text{base}$)	ms

WORKED EXAMPLE

Worked example — base 100 ms, cap 20 s.

1. Ceilings s_k for $k = 1…8$: 200, 400, 800, 1,600, 3,200, 6,400, 12,800, 20,000 (capped) ms.
2. Full jitter at $k = 3$: a wait anywhere in 0–800 ms, mean 400 ms. Equal jitter: 400–800 ms, mean 600 ms.
3. Cumulative worst-case wait before the 5th retry with full jitter: $200 + 400 + 800 + 1,600 + 3,200 = 6,200$ ms, so five retries do not fit a 1 s user deadline; they fit a background job.

Interpretation. Exponential growth reaches the cap quickly; jitter decides how evenly clients are spread within each ceiling. **Engineering implication.** For interactive requests, one or two retries with small bases are all the deadline allows; long backoff sequences belong to asynchronous work.

Code example: measuring contention

The simulation has N clients each needing one successful write to a contended resource with optimistic concurrency: a write conflicts and fails if another started less than 1 ms earlier. All clients begin within the same millisecond, as after a brief outage.

```
        """N clients each need one successful write to a contended resource (optimistic
        concurrency: if another write started less than WORK ms earlier, this one con-
flicts).
        Compare backoff strategies by total calls (work) and time until every client
succeeds."""
        import heapq
        import random

        BASE, CAP, WORK = 5.0, 2000.0, 1.0              # milliseconds


        def no_jitter(attempt, prev, rng):
            return min(CAP, BASE * 2 ** attempt)

        def full_jitter(attempt, prev, rng):
            return rng.uniform(0, min(CAP, BASE * 2 ** attempt))

        def equal_jitter(attempt, prev, rng):
            v = min(CAP, BASE * 2 ** attempt)
            return v / 2 + rng.uniform(0, v / 2)

        def decorrelated_jitter(attempt, prev, rng):
            return min(CAP, rng.uniform(BASE, prev * 3))


        def simulate(strategy, clients, seed, max_attempts=200):
            rng = random.Random(seed)
            # every client makes its first call within the same 1 ms (a synchronized
wave)
            queue = [(rng.uniform(0, 1.0), c, 0, BASE) for c in range(clients)]
            heapq.heapify(queue)
            busy_until, calls, finish = -1.0, 0, 0.0
            while queue:
                t, c, attempt, prev = heapq.heappop(queue)
                calls += 1
                if t >= busy_until:                     # no conflicting write in
flight: success
                    busy_until, finish = t + WORK, t
                elif attempt + 1 < max_attempts:        # conflict: back off and try
again
                    sleep = strategy(attempt + 1, prev, rng)
                    heapq.heappush(queue, (t + WORK + sleep, c, attempt + 1, sleep))
            return calls, finish


        if __name__ == "__main__":
            strategies = [("exponential, no jitter", no_jitter), ("full jitter",
full_jitter),
                          ("equal jitter", equal_jitter), ("decorrelated jitter",
decorrelated_jitter)]
            seeds = range(20)
            for n in (25, 100):
                print(f"{n} clients (mean of {len(seeds)} runs)     calls   calls/client
time to finish")
                results = {}
                for name, fn in strategies:
```

```
                    runs = [simulate(fn, n, s) for s in seeds]
                    calls = sum(r[0] for r in runs) / len(runs)
                    finish = sum(r[1] for r in runs) / len(runs)
                    results[name] = calls
                    print(f"  {name:24s} {calls:8.0f}   {calls / n:10.1f}
{finish:10.0f} ms")
                print()
                assert results["full jitter"] < results["exponential, no jitter"] / 3
```

Output:

```
25 clients (mean of 20 runs)     calls   calls/client   time to finish
  exponential, no jitter        325         13.0        34575 ms
  full jitter                    76          3.1           93 ms
  equal jitter                   83          3.3          139 ms
  decorrelated jitter            73          2.9          108 ms

100 clients (mean of 20 runs)     calls   calls/client   time to finish
  exponential, no jitter       5050         50.5       184650 ms
  full jitter                   459          4.6          498 ms
  equal jitter                  475          4.8          636 ms
  decorrelated jitter           450          4.5          406 ms
```

What to notice:

- **Without jitter, contention is quadratic.** The clients stay synchronized forever, so each wave lets exactly one through: $100 + 99 + \ldots + 1 = 5,050$ calls, and the last client finishes after three minutes because waits hit the 2 s cap.
- **Any jitter cuts work by 11×** (5,050 → about 460 calls) and completion time by over 300×. Full and decorrelated jitter are slightly better than equal jitter, matching Brooker's qualitative result; the exact numbers depend on the model.

22.12.3 Retry storms and amplification

Figure 22.11 — Retry amplification in a four-layer call graph.



Reading the diagram. Each layer, reasonably in isolation, makes up to three attempts. When the database fails every call, attempts multiply synchronously down the chain: 3, 9, 27, then 81 calls at the database, aimed precisely at the failing component: a **retry storm**. The fix is structural: retry at one layer, cap retries with budgets, and stop when a circuit breaker opens (§22.13).

FORMULA

Retry amplification

$$A = \prod_{i=1}^{d} a_i = a^d \text{ (identical layers)}, \qquad \mathbb{E}[\text{attempts per layer}] = \sum_{j=0}^{a-1} p^j = \frac{1-p^a}{1-p}$$

Symbol	Meaning	Units
A	Worst-case calls reaching the bottom layer per user request	calls
a_i	Maximum attempts (1 + retries) at layer i	attempts
d	Number of layers that retry	layers
p	Probability that one attempt fails ($p < 1$)	dimensionless

WORKED EXAMPLE

Worked example — 10,000 user RPS through four retrying layers.

1. Database hard down ($p = 1$): $A = 3^4 = 81$; the database receives $10,000 \times 81 = 810,000$ RPS of doomed calls.
2. Database partly overloaded, $p = 0.5$: expected attempts per layer $= (1 - 0.5^3)/(1 - 0.5) = 1.75$; if failures compound through layers, the multiplier approaches $1.75^4 \approx 9.4$, a 9× load increase on a system already failing half its calls.
3. Retry only at the SDK: $A = 3$, so 30,000 RPS at worst.
4. Retry budget of 10% at every layer: each layer adds at most 10%, so $A \le 1.1^4 = 1.46$, about 14,600 RPS.

Interpretation. Per-layer retries look harmless locally but compound exponentially with depth. **Engineering implication.** Retry at one layer, or bound retries by a budget that is a fraction of normal traffic; never let every layer retry independently.

22.12.4 Retry budgets

A **retry budget** caps retries as a fraction of successful or total requests, rather than per request. Google's SRE book describes a per-request limit (at most three attempts) combined with a per-client budget (retries no more than 10% of requests) [Beyer et al. 2016, ch. 21]. A simple implementation is a token bucket for retries: each success deposits 0.1 token, each retry spends 1, and when the bucket is empty the error is returned to the caller without a retry. When the dependency is healthy, occasional retries always find tokens; when it fails broadly, retries are limited to 10% extra load. The AWS SDKs implement a comparable client-side retry quota, and Envoy offers a retry budget expressed as

a percentage of active requests [AWS SDK Docs: retry behavior; Envoy Docs: retry budgets].

INDUSTRY EXAMPLE

AWS guidance on retries. The Builders' Library article on timeouts and retries recommends capped exponential backoff with jitter, retrying at a single point in the stack, local retry limits implemented with a token bucket so that retries stop when most calls fail, and treating timeouts as signals to be measured; it also warns that retries are "selfish": they spend server resources to raise one client's chance of success [Brooker: timeouts and retries]. *Evidence: Primary* — [Brooker: timeouts and retries]

Table 22.6 — Retry failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Outage worsens as clients retry	Every layer retries; no budget	Request rate rises while success rate falls	Disable retries via configuration; open breakers; shed	One retrying layer, budgets, jitter
Synchronized retry waves every few seconds	Backoff without jitter; reconnect after a shared failure	Periodic spikes at backoff intervals	Add jitter, server-side shedding	Full or decorrelated jitter (§9.9 reconnect storms)
Duplicate side effects	Retried non-idempotent `POST` after a timeout	Duplicate rows seconds apart	Reconcile and refund	Idempotency keys (§22.15)
Retries outlive the user	Retry delays ignore the deadline	Work completing after client timeout	Check remaining budget before retrying	Deadline propagation (§22.11)

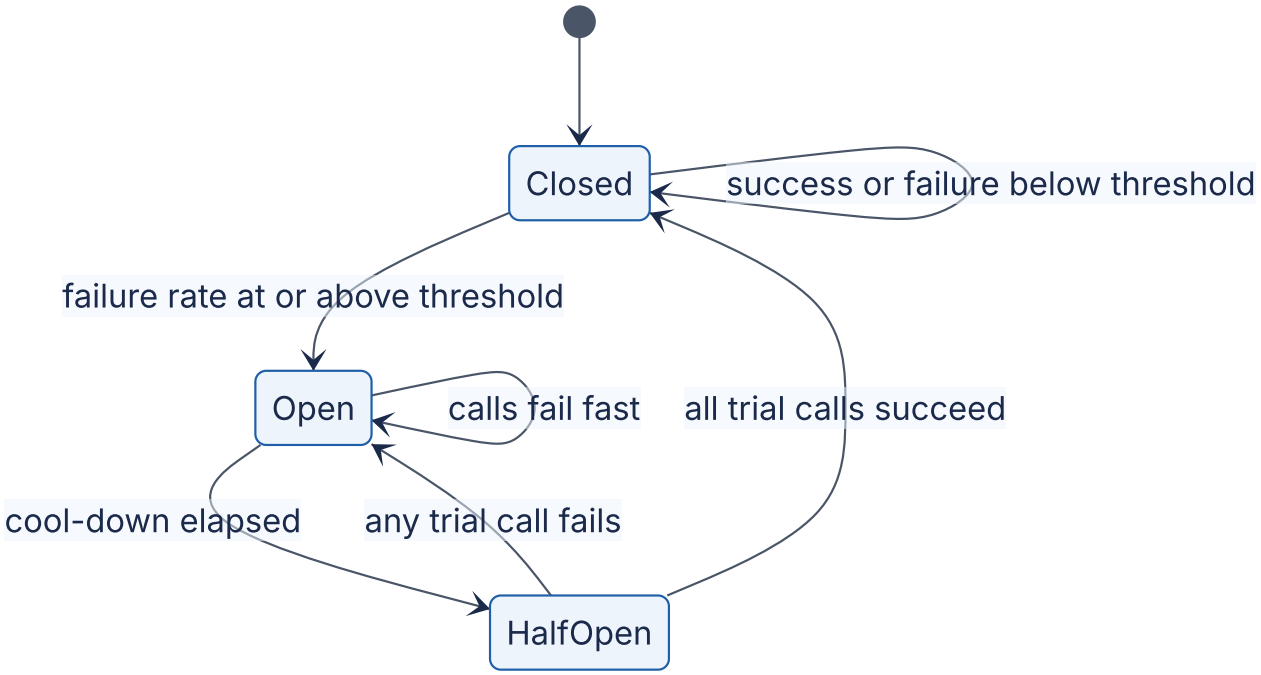
22.13 Circuit breakers

DEFINITION

Definition — Circuit breaker. A client-side proxy for calls to a dependency that tracks recent failures and, when they cross a threshold, **opens**: it fails calls immediately without contacting the dependency for a cool-down period, then lets a few trial calls through (**half-open**) and **closes** again only if they succeed.

An electrical circuit breaker cuts power when current is dangerous, so a fault in one appliance does not burn down the house; you reset it once the fault is fixed. The software version, popularized by Nygard's *Release It!* [Nygard 2018] and Fowler's description [Fowler 2014], protects two parties: the *caller*, which stops tying up threads waiting on a dead dependency, and the *dependency*, which gets breathing room to recover instead of a retry storm.

Figure 22.12 — Circuit breaker state machine.



Reading the diagram. In **Closed**, calls pass through and outcomes are recorded in a sliding window (the last *n* calls, or the last *t* seconds). When the window's failure rate reaches the threshold (and a minimum number of calls has been seen), the breaker moves to **Open**, where every call fails immediately with a local error, costing microseconds and no network traffic. After the cool-down, the next caller moves it to **HalfOpen**, where a small fixed number of trial calls reach the dependency; one failure reopens it, success of all trials closes it. Transitions are local to each client instance (no shared storage), so different instances may briefly disagree. Failure points: thresholds set too low trip on normal noise; too high trip only after the damage is done. Fallbacks (cached data, a default, a degraded page) decide what the user sees while open.

Choosing thresholds.

- **Window and minimum volume.** Count-based (last 20–100 calls) or time-based (last 10 s). Require a minimum number of calls so that 2 failures out of 3 at 3 a.m. do not trip it. Netflix's Hystrix library, for example, documented defaults of a 10 s rolling window, a minimum of 20 requests and a 50% error threshold, with a 5 s sleep window [Netflix Hystrix wiki]; Hystrix has been in maintenance mode since 2018, and Resilience4j is a common successor on the JVM.
- **What counts as failure.** Timeouts, connection errors and 5xx, but not 4xx: a client's bad request says nothing about the dependency's health. Slow calls above a latency threshold can also count.
- **Cool-down.** Long enough for typical recovery (seconds to a minute), ideally with jitter so that many instances do not probe in unison.

22.13.1 Implementation and simulation

```
        import time
        from collections import deque


        class CircuitOpenError(Exception):
            """Raised instead of calling the dependency while the breaker is open."""


        class CircuitBreaker:
            def __init__(self, failure_rate=0.5, window=20, open_for=5.0, probes=3,
                         clock=time.monotonic):
                self.failure_rate, self.window = failure_rate, window
                self.open_for, self.probes, self.clock = open_for, probes, clock
                self.state, self.opened_at = "closed", 0.0
                self.results = deque(maxlen=window)          # outcomes of the last
`window` calls
                self.probes_sent = self.probe_successes = 0
                self.transitions = []

            def _move(self, state):
                self.transitions.append((round(self.clock(), 1), state))
                self.state = state

            def _allow(self):
                if self.state == "open":
                    if self.clock() - self.opened_at < self.open_for:
                        return False                         # fail fast: do not touch
the dependency
                    self._move("half_open")
                    self.probes_sent = self.probe_successes = 0
                if self.state == "half_open":
                    if self.probes_sent >= self.probes:      # only a few trial calls at
a time
                        return False
                    self.probes_sent += 1
                return True

            def _record(self, ok):
                now = self.clock()
                if self.state == "half_open":
                    if not ok:                               # dependency still sick:
reopen
                        self.opened_at = now
                        self._move("open")
                    else:
                        self.probe_successes += 1
                        if self.probe_successes >= self.probes:
                            self.results.clear()
                            self._move("closed")
                    return
                self.results.append(ok)
                failures = self.results.count(False)
                if len(self.results) == self.window and failures / self.window >=
self.failure_rate:
```

```
                    self.opened_at = now
                    self._move("open")

            def call(self, fn, *args, **kwargs):
                if not self._allow():
                    raise CircuitOpenError("circuit open")
                try:
                    result = fn(*args, **kwargs)
                except Exception:
                    self._record(False)
                    raise
                self._record(True)
                return result


        if __name__ == "__main__":
            from itertools import count

            class Clock:
                t = 0.0
                def __call__(self):
                    return self.t

            clock = Clock()
            breaker = CircuitBreaker(clock=clock)
            reached = short_circuited = failed = 0

            def dependency():
                global reached
                reached += 1
                if 10.0 <= clock.t < 28.0:                    # outage from t = 10 s to t
= 28 s
                    raise ConnectionError("upstream down")
                return "ok"

            for i in count():                                 # 20 requests per second
for 60 s
                clock.t = i * 0.05
                if clock.t >= 60.0:
                    break
                try:
                    breaker.call(dependency)
                except CircuitOpenError:
                    short_circuited += 1
                except ConnectionError:
                    failed += 1

            print(f"requests 1200, reached dependency {reached}, failed {failed}, "
                  f"short-circuited {short_circuited}")
            print("transitions:", breaker.transitions)
            assert breaker.state == "closed" and short_circuited > 350 and failed < 20
```

Output:

```
requests 1200, reached dependency 804, failed 13, short-circuited 396
transitions: [(10.5, 'open'), (15.5, 'half_open'), (15.5, 'open'), (20.5, 'half_open'),
(20.5, 'open'), (25.5, 'half_open'), (25.5, 'open'), (30.5, 'half_open'), (30.6,
'closed')]
```

What to notice:

- The breaker trips 0.5 s into the 18 s outage (10 failures out of the last 20 calls) and probes every 5 s; only 13 calls fail against the dead dependency instead of 360.
- It closes at 30.6 s, 2.6 s after recovery: the cool-down trades recovery speed against probing load.
- Little's Law shows the saving: if each failed call had hung for a 2 s timeout, 360 calls over 18 s would have held $(360/18) \times 2 = 40$ threads continuously; with the breaker, almost none.

COMMON MISTAKE

Mistake — Using a breaker instead of a timeout. A breaker learns only from completed calls. If calls hang with no timeout, nothing is ever recorded as failed and the breaker never opens. Timeouts come first; breakers build on them.

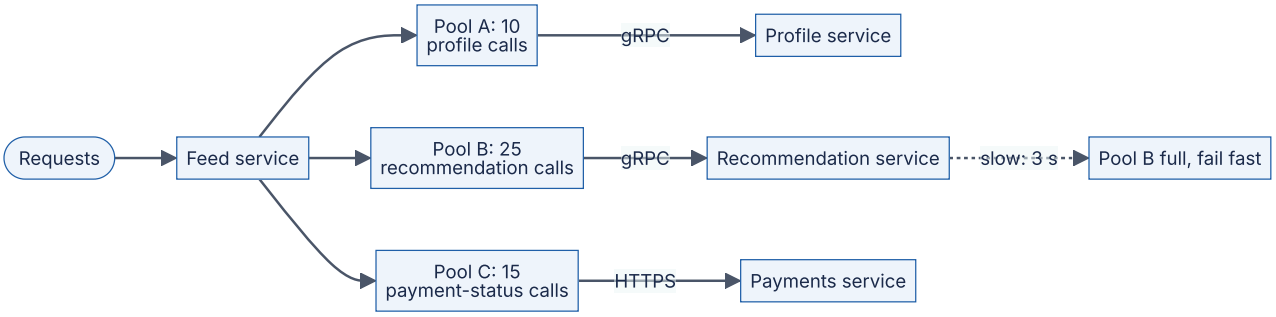
22.14 Bulkheads

DEFINITION

Definition — Bulkhead. Partitioning a service's limited resources (threads, connections, memory, instances) into isolated pools, one per dependency, tenant or workload class, so that exhaustion of one pool cannot starve the others.

A ship's hull is divided into watertight compartments (bulkheads): a breach floods one compartment, not the whole ship [Nygard 2018]. In a service, the classic failure is a shared thread pool: one slow dependency holds every thread, and requests that never touch that dependency also stall. Bulkheads give each dependency its own pool (or a semaphore limiting concurrent calls to it) sized from Little's Law, so that when a pool is full, calls to that dependency are rejected immediately while the rest of the service carries on. The same idea applies at larger scale: separate connection pools per database, separate instance groups for internal and public traffic, and cell-based architectures that confine a failure to one cell of customers (§33.12).

Figure 22.13 — Bulkheads isolating dependencies in the feed service.



Reading the diagram. The feed service makes synchronous calls to three dependencies, each through its own bounded pool of concurrent slots. When the recommendation service slows to 3 s, its 25 slots fill and further calls fail fast (the feed is rendered without recommendations, a graceful degradation), while profile and payment-status calls keep their own slots and continue normally. There is no shared storage between pools. Failure points: pools sized too small reject healthy traffic; too large a sum exhausts the host. Scaling: pools are per instance, so fleet capacity for a dependency is pool size × instances.

WORKED EXAMPLE

Worked example — Sizing the pools with Little's Law ($L = \lambda W$) and 1.5–2× headroom.

1. Profile: 300 RPS × 0.020 s = 6 in flight → pool 10.
2. Recommendations: 100 RPS × 0.150 s = 15 → pool 25.
3. Payment status: 20 RPS × 0.400 s = 8 → pool 15.
4. Incident: recommendations slow to 3 s. Demand becomes $100 \times 3 = 300$ concurrent calls. In a single shared 200-thread pool, all 200 threads would block on recommendations within about 2 s and the whole feed would stop. With bulkheads, the 25 slots stay busy and complete only $25/3 \approx 8$ recommendation calls per second, so about 92 of the 100 per second fail fast (the 275 calls of excess concurrency are never admitted), and profile and payment calls use their own 10 and 15 slots normally.

Interpretation. Bulkheads convert a total outage into a partial degradation.
Engineering implication. Give every remote dependency its own concurrency limit, combine it with timeouts and a breaker, and design the fallback the user sees when the bulkhead rejects.

22.15 Idempotency and idempotency keys

22.15.1 Why retries need idempotency

Section 8.6.3 defined an **idempotent** operation: performing it $N > 0$ times has the same effect as performing it once. `GET`, `PUT` and `DELETE` are idempotent by definition; `POST` and most `PATCH` operations are not. Every technique in §22.11–§22.13 ends in the same place: a client that timed out does not know whether its request took effect, and the only safe way to find out is to send it again. For “create a charge”, “send a message” or “place an order”, a naive retry can do the thing twice. This chapter owns the deep treatment of how to make such operations safe to retry; Chapters 30 and 32 apply the same idea to message consumers (§32.10).

DEFINITION

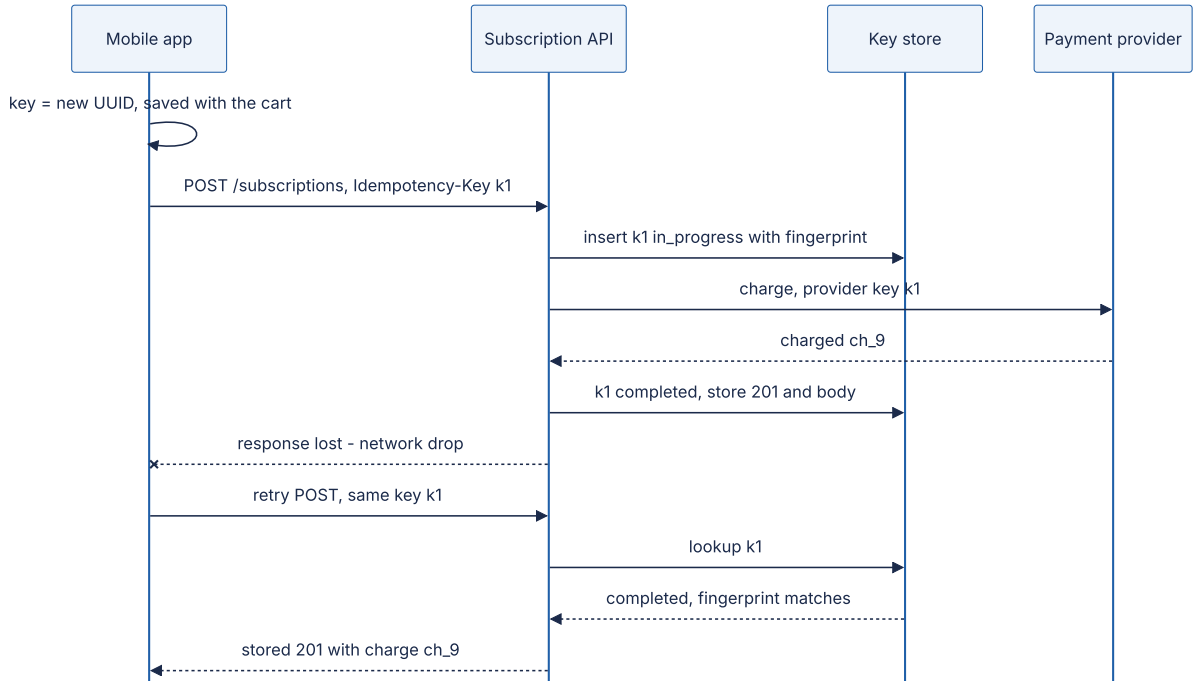
Definition — Idempotency key. A unique value, generated by the client once per *logical* operation and sent with every attempt of that operation (for example in an `Idempotency-Key` header), that the server uses to recognize retries: the first attempt executes and its result is stored under the key; later attempts with the same key return the stored result instead of executing again.

Think of a cloakroom ticket you write *before* handing over your coat: if unsure the attendant took it, you show the same number again, and the attendant finds ticket 417 already on the rack. The client must choose the key before the first attempt, because it may never see the server’s reply.

22.15.2 The protocol

Stripe’s API popularized the pattern: clients send an `Idempotency-Key` header (a random V4 UUID is recommended) on `POST` requests, and the server guarantees at-most-once execution per key within a retention window [Leach 2017; Stripe Docs: idempotent requests]. The IETF HTTPAPI working group has an Internet-Draft standardizing the header, *The Idempotency-Key HTTP Header Field* [Jena and Dalal, draft], which, as of its 2025 revision, suggests 409 Conflict when a retry arrives while the original is still being processed and 422 when a key is reused with a different payload; it is a draft, not an RFC.

Figure 22.14 — An idempotent purchase retried after a lost response.



Reading the diagram. The app generates `k1` before the first attempt and persists it with the cart, so even an app restart reuses it. The API records `k1` as in progress in the key store (a table in its own database, the storage boundary), calls the provider with a key derived from `k1`, records the response and replies; the reply is lost. The retry finds a completed record with a matching fingerprint and gets the stored response without a second charge. All calls are synchronous. Failure points: a crash between charge and "completed" write (§22.15.4) and a concurrent retry (the in-progress state). Scaling: partition the key store by account.

22.15.3 Storage, fingerprinting and concurrency

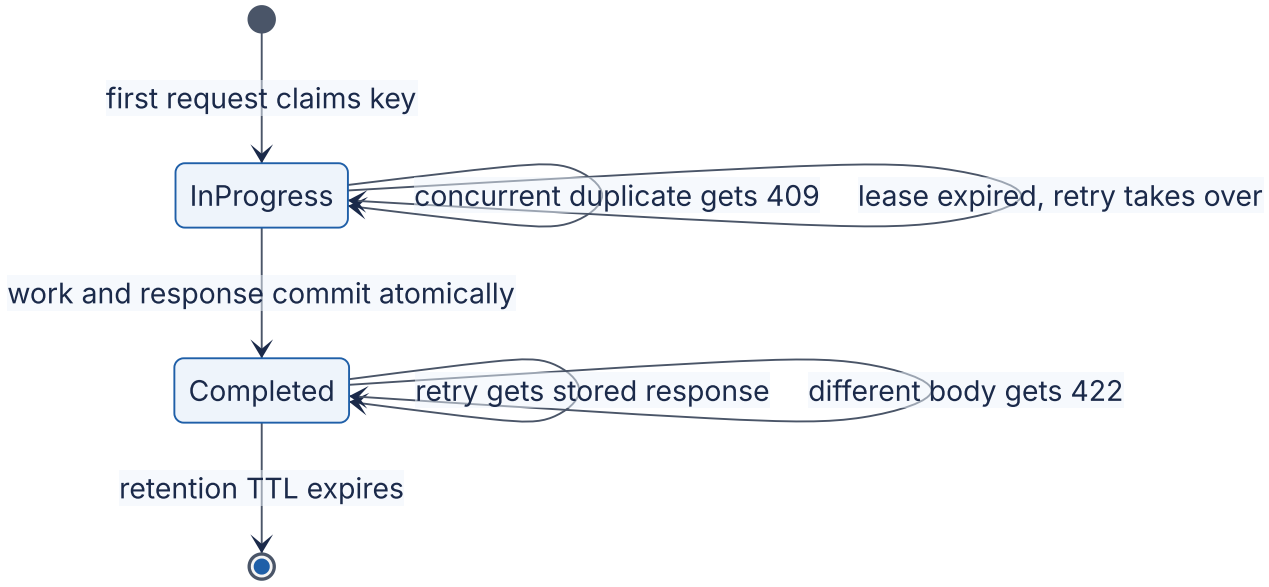
A production idempotency layer must answer five questions.

1. **Scope.** Keys are unique *per client account*, not globally: the primary key is `(account_id, idem_key)`, so one customer cannot collide with, or probe, another's keys.
2. **Fingerprint.** Store a hash of the method, path and canonicalized body. A retry with the same key but a different body is a client bug (or a reused key) and must be rejected (422), not answered with the first request's result.
3. **Concurrency.** Two attempts with the same key can arrive at once (a client that retries before the first attempt times out at the server). Exactly one may execute: insert the key with a uniqueness constraint (`INSERT ... ON CONFLICT DO NOTHING`); the loser receives 409 and retries later.
4. **Crash recovery.** A worker can die after claiming a key. Give the claim a **lease** (`locked_until`); once it expires, a retry may take over. A worker that was merely

slow must not then complete as well, so the final write checks that it still holds its lease, a **fencing token** (§26.13).

5. **Atomicity.** The business write (the charge row) and the "completed" record must commit in the *same* database transaction. Then a crash leaves either both or neither, and a takeover after lease expiry can safely re-execute.

Figure 22.15 — Lifecycle of an idempotency-key record.



Reading the diagram. The first request's insert creates the record **InProgress**, atomically thanks to the uniqueness constraint. Duplicates get 409 while the lease is live; after it expires a retry takes over and the original worker's late completion is fenced off. Work and response commit together into **Completed**; then retries get the stored response and mismatched bodies 422. At TTL expiry the record is deleted and the key would execute again. Storage boundary: one row per key in the business database, which enables the atomic commit.

Code example

```
        import hashlib
        import json
        import sqlite3
        import uuid

        SCHEMA = """
        CREATE TABLE idempotency_keys (
            account_id    TEXT    NOT NULL,
            idem_key      TEXT    NOT NULL,
            fingerprint   TEXT    NOT NULL,          -- hash of method + path + canoni-
cal body
            status        TEXT    NOT NULL,          -- 'in_progress' or 'completed'
            locked_until  REAL,                      -- lease for the worker executing
the request
            response_code INTEGER,
            response_body TEXT,
            created_at    REAL    NOT NULL,
            PRIMARY KEY (account_id, idem_key)       -- keys are scoped to the caller
        );
        CREATE TABLE charges (id TEXT PRIMARY KEY, account_id TEXT, amount INTEGER, cur-
rency TEXT);
        """


        def fingerprint(method, path, body):
            canonical = json.dumps(body, sort_keys=True, separators=(",", ":"))
            return hashlib.sha256(f"{method} {path}\n{canonical}".encode()).hexdigest()


        class LeaseLost(Exception):
            """Another worker took over this key; our business writes must roll back."""


        class IdempotencyLayer:
            def __init__(self, db, clock, lease=30.0, ttl=24 * 3600.0):
                self.db, self.clock, self.lease, self.ttl = db, clock, lease, ttl

            def claim(self, account, key, method, path, body):
                """Phase 1. Returns ('proceed', lease) or ('respond', (status,
body))."""
                fp, now = fingerprint(method, path, body), self.clock()
                with self.db:                                    # one transaction
                    self.db.execute("DELETE FROM idempotency_keys WHERE account_id=? AND
idem_key=? "
                                    "AND created_at < ?", (account, key, now -
self.ttl))
                    inserted = self.db.execute(
                        "INSERT INTO idempotency_keys VALUES
(?,?,?,'in_progress',?,NULL,NULL,?) "
                        "ON CONFLICT DO NOTHING", (account, key, fp, now + self.lease,
now)).rowcount
                    if inserted:
                        return "proceed", now + self.lease       # first time we see
this key
```

```
                    fp0, status, locked_until, code, resp = self.db.execute(
                        "SELECT fingerprint, status, locked_until, response_code,
response_body "
                        "FROM idempotency_keys WHERE account_id=? AND idem_key=?", (ac-
count, key)).fetchone()
                    if fp0 != fp:
                        return "respond", (422, {"error": "idempotency key reused with
different request"})
                    if status == "completed":
                        return "respond", (code, json.loads(resp))   # replay the stored
response
                    if locked_until > now:
                        return "respond", (409, {"error": "request with this key is in
progress"})
                    self.db.execute("UPDATE idempotency_keys SET locked_until=? WHERE
account_id=? "
                                    "AND idem_key=?", (now + self.lease, account, key))
                    return "proceed", now + self.lease           # previous worker died:
take over

            def execute(self, account, key, lease, handler, body):
                """Phase 2. Business writes and the stored response commit atomically,
and only
                if we still hold the lease we were given (the lease value acts as a
fencing token)."""
                with self.db:
                    code, resp = handler(self.db, account, body)
                    done = self.db.execute(
                        "UPDATE idempotency_keys SET status='completed',
locked_until=NULL, "
                        "response_code=?, response_body=? WHERE account_id=? AND
idem_key=? "
                        "AND status='in_progress' AND locked_until=?",
                        (code, json.dumps(resp), account, key, lease)).rowcount
                    if not done:
                        raise LeaseLost(key)                     # exception => transac-
tion rolls back
                return code, resp

            def handle(self, account, key, method, path, body, handler):
                action, value = self.claim(account, key, method, path, body)
                return value if action == "respond" else self.execute(account, key,
value, handler, body)


        def create_charge(db, account, body):
            charge_id = "ch_" + uuid.uuid4().hex[:12]
            db.execute("INSERT INTO charges VALUES (?,?,?,?)",
                       (charge_id, account, body["amount"], body["currency"]))
            return 201, {"id": charge_id, "amount": body["amount"], "status": "suc-
ceeded"}


        if __name__ == "__main__":
            class Clock:
                t = 1_000.0
                def __call__(self):
```

```
                    return self.t

            clock, db = Clock(), sqlite3.connect(":memory:")
            db.executescript(SCHEMA)
            api = IdempotencyLayer(db, clock)
            charges = lambda: db.execute("SELECT COUNT(*) FROM charges").fetchone()[0]
            body, key = {"amount": 2000, "currency": "usd"}, str(uuid.uuid4())

            first = api.handle("acct_1", key, "POST", "/v1/charges", body,
create_charge)
            retry = api.handle("acct_1", key, "POST", "/v1/charges", dict(body),
create_charge)
            print("1. first call:", first)
            print("2. retry     :", retry, "| charges in DB:", charges())
            assert first == retry and charges() == 1

            other = api.handle("acct_1", key, "POST", "/v1/charges", {"amount": 9999,
"currency": "usd"},
                               create_charge)
            print("3. same key, different body:", other[0])
            assert other[0] == 422

            k2 = str(uuid.uuid4())                                # two concurrent dupli-
cates
            a = api.claim("acct_1", k2, "POST", "/v1/charges", body)
            b = api.claim("acct_1", k2, "POST", "/v1/charges", body)
            print("4. concurrent duplicate while first in progress:", b[1][0])
            assert a[0] == "proceed" and b[1][0] == 409

            clock.t += 31                                         # first worker stalls
past its lease
            c = api.claim("acct_1", k2, "POST", "/v1/charges", body)
            api.execute("acct_1", k2, c[1], create_charge, body)  # the retry takes over
and finishes
            try:
                api.execute("acct_1", k2, a[1], create_charge, body)  # the stalled
worker wakes up
            except LeaseLost:
                print("5. stalled worker's late write rejected and rolled back")
            print("   retry took over:", c[0], "| charges in DB:", charges())
            assert c[0] == "proceed" and charges() == 2           # one charge per logi-
cal operation

            clock.t += 25 * 3600                                  # beyond the 24 h re-
tention window
            late = api.handle("acct_1", key, "POST", "/v1/charges", body, create_charge)
            print("6. same key after TTL expired:", late[0], "| charges in DB:",
charges())
            assert late[0] == 201 and charges() == 3              # the duplicate is no
longer caught
```

Output (charge IDs are random):

```
1. first call: (201, {'id': 'ch_56b3f43ff904', 'amount': 2000, 'status': 'succeeded'})
2. retry     : (201, {'id': 'ch_56b3f43ff904', 'amount': 2000, 'status': 'succeeded'}) |
charges in DB: 1
3. same key, different body: 422
4. concurrent duplicate while first in progress: 409
5. stalled worker's late write rejected and rolled back
   retry took over: proceed | charges in DB: 2
6. same key after TTL expired: 201 | charges in DB: 3
```

What to notice:

- The uniqueness constraint, not application logic, decides who executes: `INSERT ... ON CONFLICT DO NOTHING` (SQLite 3.24+, PostgreSQL 9.5+) returns a row count of 1 for exactly one claimant.
- Canonical JSON (`sort_keys=True`) makes `{"a":1,"b":2}` and `{"b":2,"a":1}` the same fingerprint; whitespace and key order must not turn a retry into a "different request".
- Case 5 is the subtle one: without the `locked_until=?` fencing condition, the stalled worker's late commit would have created a second charge.
- Case 6 shows the retention trade-off: after the TTL the key is forgotten and a very late retry executes again.

22.15.4 External side effects, retention and alternatives

Side effects outside your database. The atomic commit in case 5 works only when the effect lives in the same database as the key. A call to a payment provider, an email gateway or another service cannot join your transaction. The remedies, in order of preference: pass an idempotency key *downstream* (derived from yours, for example `k1:charge`), so the provider deduplicates; split the request into **atomic phases** with recorded **recovery points**, so a retry resumes after the last completed phase instead of starting over, as described in Leach's PostgreSQL implementation [Leach 2017b]; or, for effects that cannot be made idempotent, record intent first and reconcile later (the transactional outbox, §32.5).

FORMULA

Idempotency-key storage

$$S = \lambda_w \times T_{\text{ttl}} \times s_r$$

Symbol	Meaning	Units
S	Steady-state storage for key records	bytes
λ_w	Rate of keyed (non-idempotent) requests	requests/s
T_{ttl}	Retention window	s
s_r	Average record size (key, fingerprint, stored response)	bytes

WORKED EXAMPLE

Worked example — PhotoShare's write APIs. 2,000 keyed `POST` requests per second at peak (use peak as a conservative average), 24 h retention, records of about 1 KB (a 36-byte key, a 32-byte fingerprint, a stored JSON response of most of the rest).

1. Records retained: $2,000 \times 86,400 = 172.8$ million.
2. Storage: $172.8 \times 10^6 \times 1,000$ bytes $= 172.8$ GB.
3. Storing only the response's status and resource ID (about 200 bytes per record) and re-reading the resource on replay cuts this to 34.6 GB.

Interpretation. Retention is a capacity decision. **Engineering implication.** Make T_{ttl} exceed the longest retry horizon (including offline mobile queues), partition by time so expiry is a cheap partition drop, and document the window.

INDUSTRY EXAMPLE

Stripe idempotency keys. Stripe's documentation (as of 2026) describes keys of up to 255 characters, recommends V4 UUIDs, stores the status code and body of the first request for a key *whether it succeeded or failed* (including 500 errors) and returns them for retries, compares parameters and returns an error if a retry's parameters differ, and allows keys to be removed after they are at least 24 hours old. Leach's 2017 engineering post explains the motivation: networks fail in ambiguous ways, so clients must be able to retry any mutating request safely, with backoff and jitter [Stripe Docs: idempotent requests; Leach 2017]. *Evidence: Primary* — [Stripe Docs: idempotent requests; Leach 2017]

INDUSTRY EXAMPLE

Amazon EC2 client tokens. EC2's `RunInstances` accepts a `ClientToken`; retrying with the same token returns the original result instead of launching more instances, and reusing a token with different parameters fails with an `IdempotentParameterMismatch` error. The AWS Builders' Library article on idempotent APIs explains why the service, not the client, must detect parameter mismatch and why the response to a retry should be semantically equivalent to the original [Yanacek: idempotent APIs]. *Evidence: Primary* — [Yanacek: idempotent APIs]

Alternatives. Not every operation needs a key store. A **client-chosen resource ID** with `PUT /subscriptions/{uuid}` is naturally idempotent. **Conditional requests** (`If-Match` with an ETag, §8.6) make updates safe to retry. A **natural unique constraint** (one active subscription per user) rejects duplicates in the database. Idempotency keys are the general solution when the operation is a `POST` with server-assigned identity and real-world effects.

Table 22.7 — Idempotency failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Customer charged twice	Key generated per *attempt*, not per operation	Two charges, two different keys, same cart	Refund and reconcile	Generate the key once and persist it with the cart
Retry gets the wrong result	No fingerprint; key reused for a new purchase	Responses that do not match request bodies	Return 422 on mismatch	Fingerprint method, path and canonical body
Double execution under concurrency	Check-then-insert race on the key	Duplicate effects milliseconds apart	Reconcile	Uniqueness constraint plus lease and fencing
Charge made, record says in progress	Crash between external call and commit	Stuck in-progress keys past lease	Retry passes the same downstream key; provider deduplicates	Downstream keys, recovery points
Late retry executes again	Retry after the retention TTL	Duplicates spaced by more than the TTL	Reconcile	TTL longer than client retry horizon

COMMON MISTAKE

Mistake — “We return the stored response, so the operation is exactly-once.” Idempotency keys give *at-most-once execution per key within the retention window* on top of *at-least-once delivery* by the client: effectively-once processing. They say nothing about a client that generates a new key per attempt, a key that has expired, or a side effect that escaped your transaction. Exactly-once delivery is not achievable over an unreliable network; effectively-once processing is (§32.9).

22.16 Summary and revision notes

SUMMARY

- **Rate limits are per-client contracts; load shedding protects total capacity.** You need both. Enforce limits in layers (edge per IP, gateway per key/tenant, service per cost) and signal them with 429, `Retry-After`, and the draft `RateLimit-Policy` / `RateLimit` fields.
- **Token bucket:** rate r, burst b, admitted $A(T) \le b + rT$; two numbers per key; `Retry-After` = deficit/r. **GCRA** is the same policy stored as one timestamp. The leaky bucket *as a queue* shapes traffic by delaying it.
- **Fixed window** admits up to $2L$ across a boundary. The **sliding log** is exact but stores L timestamps per key. The **sliding window counter** $\hat{N} = C_{\text{prev}}(W - e)/W + C_{\text{curr}}$ is cheap and close to exact for random traffic, but biased for traffic phase-locked to the window.
- **Distributed limiting:** centralized Redis with an atomic Lua script is exact but costs a round trip; local counting with sync overshoots by up to $\lambda\Delta$; quota distribution is cheap but lags demand shifts. Decide fail-open versus fail-closed per limit.
- **Overload:** without shedding, goodput collapses after $t = \mu T_{\text{out}}/(\lambda - \mu)$. Shed by priority, bound queues by time ($Q_{\max} = \mu D$), and use CoDel-style delay targets and adaptive LIFO.
- **Backpressure** propagates "slow down" upstream: bounded buffers, blocking or rejection, and explicit demand (`request(n)`, TCP windows).
- **Deadlines** propagate absolute time budgets: $B_{i+1} = B_i - e_i - m_i$. A timeout means "unknown outcome", not "failed".
- **Retries:** only retryable errors, idempotent operations, remaining budget. Use capped exponential backoff with full or decorrelated jitter. Amplification is a^d across d retrying layers; retry at one layer and cap with a budget (about 10%).
- **Circuit breakers** fail fast when a dependency's failure rate crosses a threshold, probe in half-open, and need timeouts underneath. **Bulkheads** give each dependency its own Little's-Law-sized pool.
- **Idempotency keys:** client-generated once per logical operation; scoped per account; fingerprinted; claimed with a uniqueness constraint; leased and fenced; committed atomically with the business write; retained longer than the retry horizon ($S = \lambda_w T_{\text{ttl}} s_r$).

22.17 Knowledge checkpoint

1. Why is a rate limit not sufficient protection against overload?

2. A token bucket has $r = 50$ RPS and $b = 200$. What is the most it admits in any 10 s interval, and how long does it take to refill from empty?
3. What is the difference between the leaky bucket as a queue and as a meter?
4. With a fixed window of 1,000 requests per hour, what is the most a client can send in any one-hour interval?
5. Sliding window counter, $L = 60$/min: previous window 40, current 25, 30 s elapsed. Is the next request admitted?
6. Why must a Redis-based limiter use a Lua script (or equivalent atomic operation) rather than `GET` then `SET`?
7. With 20 gateways synchronizing every 50 ms and 4,000 RPS offered for one key, what is the worst-case overshoot per second?
8. A server handles 400 RPS; clients time out after 2 s; 600 RPS arrive. When does goodput collapse without shedding?
9. Why does adaptive LIFO help an overloaded server?
10. What does a timeout tell the caller about a write operation?
11. Five layers each make up to 2 attempts. What is the worst-case amplification at the bottom?
12. Why does a circuit breaker need a minimum call volume?
13. What does the fencing check in an idempotency layer prevent?
14. Name three request properties that should go into an idempotency fingerprint.

Answers

1. It bounds each client individually; the sum of all clients' allowed traffic can still exceed capacity. Global protection needs load shedding and backpressure.
2. $A(10) = 200 + 50 \times 10 = 700$ requests; refill time $b/r = 200/50 = 4$ s.
3. The queue form buffers requests and releases them at a constant rate (delaying them, smoothing output); the meter form admits or rejects immediately and is equivalent to a token bucket.
4. $2L = 2,000$, by spending a full quota at the end of one window and another at the start of the next.
5. $\hat{N} = 40 \times 0.5 + 25 = 45$; $45 + 1 \le 60$, so yes.
6. Between a read and a write, other nodes can read the same value and also admit, exceeding the limit and losing increments. A script runs atomically on the shard.
7. $E_{\max} \approx \lambda\Delta = 4,000 \times 0.05 = 200$ requests.
8. $t = \mu T/(\lambda - \mu) = 400 \times 2/200 = 4$ s.
9. Under a standing queue the oldest requests have probably timed out at the client; serving the newest first spends capacity on requests someone is still waiting for.
10. Nothing definite: the write may not have arrived, may have failed, or may have succeeded with the response lost.
11. $2^5 = 32$.
12. Without it, a handful of failures at low traffic (2 of 3 calls) would exceed a percentage threshold and trip the breaker on noise.

13. A slow worker whose lease expired committing its result after another worker took over, which would execute the operation twice.
14. HTTP method, path (resource), and canonicalized body; optionally the API version and relevant headers.

22.18 Interview questions

Basic

Q1. Design a rate limiter for a public API. Answer. Clarify keys (API key; per-IP before authentication), tiers (rate and burst) and accuracy. Token bucket per key in Redis, updated by an atomic Lua script from the gateway; 429 with `Retry-After` and `RateLimit` headers. Discuss the added round trip, hot keys, fail-open versus fail-closed, and hierarchical tenant and route limits.

Q2. Why add jitter to exponential backoff? Answer. Clients that failed together back off by identical amounts and retry together, so each wave collides again. Randomizing the wait spreads retries over time; in this chapter's simulation, jitter reduced calls from 5,050 to about 460 for 100 contending clients.

Intermediate

Q3. Compare token bucket, fixed window and sliding window counter for a 100-requests-per-minute limit. Answer. Token bucket (r = 1.67/s, b chosen for bursts) is exact, two numbers per key, controls bursts explicitly. Fixed window is one counter but admits up to 200 across a boundary. Sliding counter is two counters and approximates a true sliding minute well for random traffic (5–13% worst-window overshoot in simulation) but is biased for periodic traffic. Choose token bucket for API admission, fixed windows for coarse daily quotas.

Q4. A dependency gets slow and your service's p99 explodes even for endpoints that don't call it. Why and what do you change? Answer. The slow dependency holds threads from a shared pool (Little's Law: in-flight $= \lambda W$ grows with W), starving other endpoints. Add timeouts set from the dependency's p99.9, a bulkhead (dedicated pool or semaphore) sized for normal concurrency, a circuit breaker to fail fast, and a fallback.

Advanced

Q5. Design idempotent payment creation across your API and an external payment provider. Answer. Client generates a UUID key per checkout and persists it; server scopes keys per account, stores fingerprint and status with a uniqueness constraint; concurrent duplicates get 409; mismatched bodies 422. The server passes a de-

rived key to the provider so that a crash between charge and commit is safe to retry; it splits work into atomic phases with recovery points; stores the final response atomically with local state; leases claims and fences completions; retains keys longer than the client's retry horizon; and reconciles stuck in-progress keys with the provider.

Q6. A five-service call chain melts down during a database brownout. Explain the dynamics and a fix. Answer. Each layer retries three times, so load at the database multiplies up to $3^5 = 243\times$; queues grow beyond client timeouts so goodput collapses; fixed per-hop timeouts keep inner calls running after the user has given up. Fix: retry at one layer with jittered backoff and a 10% budget, propagate deadlines, shed sheddable traffic by priority with bounded queues, add circuit breakers at each dependency, and use adaptive client throttling.

22.19 Practical exercises

1. Extend the `TokenBucket` class with per-request cost and a `RateLimit` header generator; test with a fake clock that a 5-token upload request is rejected when 3 tokens remain and that `Retry-After` is correct.
2. Run the Lua script from §22.8.2 against a local Redis, then write a version *without* atomicity (separate `HMGET` and `HSET` from 20 concurrent threads) and measure how many requests exceed the limit.
3. Modify the distributed limiter simulation so that node 0's share of traffic jumps from 10% to 60% at $t = 10$ s. Measure how long each strategy under- or over-admits.
4. Add a retry budget (10% token bucket) to the backoff simulation and measure total calls when the resource fails every call for 5 s.
5. Port the idempotency layer to PostgreSQL and test concurrent claims from two processes with `INSERT ... ON CONFLICT DO NOTHING`.

22.20 Design challenges

1. **Multi-tier quota service.** A SaaS platform with 50,000 tenants, 200 gateway nodes in three regions and 1 million RPS must enforce per-tenant per-minute quotas and per-user burst limits with less than 1 ms added p99 latency. Choose algorithms, storage and synchronization; quantify overshoot and coordination traffic; define behavior during a region partition.
2. **Overload-proof checkout.** Design PhotoShare Pro's purchase path to survive a 5× traffic spike and a 10-minute payment-provider brownout without double charges: deadlines, retries, breakers, bulkheads, shedding priorities and the idempotency layer, with a failure-mode table.

22.21 References

1. Amazon Web Services. *Throttle requests to your REST APIs for better throughput in API Gateway*. AWS Documentation, accessed 2026. https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-request-throttling.html
2. Amazon Web Services. *Retry behavior*. AWS SDKs and Tools Reference Guide, accessed 2026. https://docs.aws.amazon.com/sdkref/latest/guide/feature-retry-behavior.html
3. ATM Forum. *Traffic Management Specification Version 4.0* (af-tm-0056.000). ATM Forum, 1996.
4. Beyer, B., Jones, C., Petoff, J., Murphy, N. R. (eds.). *Site Reliability Engineering: How Google Runs Production Systems*, ch. 21 “Handling Overload” and ch. 22 “Addressing Cascading Failures”. O’Reilly, 2016. https://sre.google/sre-book/table-of-contents/
5. Brooker, M. *Exponential Backoff and Jitter*. AWS Architecture Blog, 2015. https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/
6. Brooker, M. *Timeouts, retries, and backoff with jitter*. Amazon Builders’ Library, 2019. https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/
7. Desgats, J. *How we built rate limiting capable of scaling to millions of domains*. Cloudflare Blog, 2017.
8. Envoy Project. *Global rate limiting*, *Local rate limit filter* and *Circuit breaking* (retry budgets). Envoy Proxy documentation, accessed 2026. https://www.envoyproxy.io/docs
9. Fowler, M. *CircuitBreaker*. martinfowler.com, 2014. https://martinfowler.com/bliki/CircuitBreaker.html
10. GitHub. *Rate limits for the REST API*. GitHub Docs, accessed 2026. https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api
11. gRPC Authors. *Deadlines*. gRPC documentation, accessed 2026. https://grpc.io/docs/guides/deadlines/
12. ITU-T. *Recommendation I.371: Traffic control and congestion control in B-ISDN*. International Telecommunication Union (several editions since 1993).
13. Jena, J., Dalal, S. *The Idempotency-Key HTTP Header Field*. IETF Internet-Draft draft-ietf-httpapi-idempotency-key-header, work in progress (revision -07, October 2025).
14. Leach, B. *Designing robust and predictable APIs with idempotency*. Stripe Blog, 2017. https://stripe.com/blog/idempotency
15. Leach, B. *Implementing Stripe-like Idempotency Keys in Postgres*. brandur.org, 2017. https://brandur.org/idempotency-keys (cited as Leach 2017b)
16. Maurer, B. *Fail at Scale: Reliability in the face of rapid change*. ACM Queue 13(8), 2015.

17. Netflix. *Hystrix Configuration*. Hystrix wiki on GitHub, accessed 2026. https://github.com/Netflix/Hystrix/wiki/Configuration
18. Nichols, K., Jacobson, V. *Controlling Queue Delay*. ACM Queue 10(5), 2012.
19. Nottingham, M., Fielding, R. *Additional HTTP Status Codes*. RFC 6585, IETF, 2012.
20. Nottingham, M., Wilde, E., Dalal, S. *Problem Details for HTTP APIs*. RFC 9457, IETF, 2023.
21. Fielding, R., Nottingham, M., Reschke, J. (eds.). *HTTP Semantics*. RFC 9110, IETF, 2022.
22. Nygard, M. T. *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Pragmatic Bookshelf, 2018.
23. Polli, R., Martínez Ruiz, A., Miller, D. *RateLimit header fields for HTTP*. IETF Internet-Draft draft-ietf-httpapi-ratelimit-headers, work in progress (revisions through 2025–2026).
24. Reactive Streams. *Reactive Streams Specification for the JVM*, version 1.0.0, 2015. https://www.reactive-streams.org/
25. Redis. *Scripting with Lua*. Redis documentation, accessed 2026. https://redis.io/docs/latest/develop/interact/programmability/eval-intro/
26. Stripe. *Idempotent requests*. Stripe API reference, accessed 2026. https://docs.stripe.com/api/idempotent_requests
27. Tarjan, P. *Scaling your API with rate limiters*. Stripe Blog, 2017. https://stripe.com/blog/rate-limiters
28. Yanacek, D. *Using load shedding to avoid overload*. Amazon Builders' Library. https://aws.amazon.com/builders-library/using-load-shedding-to-avoid-overload/
29. Yanacek, D. *Making retries safe with idempotent APIs*. Amazon Builders' Library. https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-apis/

22.22 Open verification items

- ITU-T I.371 edition year and the exact ATM Forum TM 4.0 document number for the GCRA definition (cited without a specific edition).
- Current revision and field syntax of the IETF RateLimit header draft (search results indicated draft-10, September 2025, and draft-11, May 2026; ietf.org was unreachable from the build environment).
- Idempotency-Key draft: status after revision -07 (October 2025) and whether its 409/422 recommendations changed.
- Cloudflare post (Desgats 2017): exact URL and the "0.003% of 400 million requests" figure.
- Stripe documentation details (255-character keys, 24-hour pruning, stored 500 responses) as of 2026.
- Hystrix default values and AWS SDK retry-quota behavior; ACM Queue URLs for Nichols and Jacobson 2012 and Maurer 2015.

Chapter 23 — Performance Engineering

> **Level:** 3 · **Track:** A (Traditional) · **Estimated reading time:** 6 h

A checkout page that was fast last month now takes two seconds for one user in fifty. The dashboards show 40% CPU. Someone suggests doubling the instance count; someone else blames the database; a third engineer wants to rewrite the JSON serializer. All three are guessing. Performance engineering replaces the guessing with method: a checklist that finds the saturated resource, a profiler that finds the hot code, a load test that measures what users actually experience, and a model that predicts how many machines next year's traffic will need.

Chapter 4 gave you the mathematics (percentiles, queueing, Little's Law, the Universal Scalability Law) and Chapter 2 the bottleneck and the utilization law. This chapter turns them into a practice: where to look first, how to measure without fooling yourself, why the slowest 1% of requests dominates large systems, and how to cut that tail. Four runnable Python programs reproduce the key effects.

23.1 Learning objectives

After this chapter you should be able to:

- Apply USE to resources and RED or the four golden signals to services, following a workflow from symptom to cause.
- Choose sampling or instrumenting profilers, read flame graphs, use off-CPU analysis, and profile Python with `cProfile` and `tracemalloc`.
- Distinguish open and closed workload models, demonstrate coordinated omission and correct for it, and record latency with HDR histograms.
- Quantify fan-out tail amplification and list the main sources of latency variability (garbage collection, queueing, background work, noisy neighbors, CPU throttling).
- Design hedged and tied requests, compute their latency benefit and extra load, and explain when hedging becomes dangerous.
- Size batches and pipelines with a latency–throughput model and decide when compression pays for its CPU cost.
- Explain generational GC, stop-the-world pauses, collector trade-offs (G1, ZGC, Shenandoah, Go) and JIT warm-up.
- Fit the Universal Scalability Law to benchmark data, predict the throughput peak and convert the model into a fleet size.

23.2 Prerequisites

- Percentiles, histograms and fan-out amplification — §4.8 (tail latency is the central quantity of this chapter).
- Queueing theory, Little's Law and the M/M/1 latency curve — §4.10, §4.11 (load testing and hedging both change utilization).
- Amdahl's Law and the Universal Scalability Law — §4.12, §4.13 (§23.10 fits the USL to data).
- The resource model, utilization law and USE method overview — §2.6 (§23.3 extends them into a full workflow).
- CPU caches, CPU-bound versus I/O-bound workloads — §5.4, §5.8 (what a CPU profile can and cannot show).
- Threads, scheduling and context switches — §6.3–§6.5 (on-CPU versus off-CPU time).
- TCP behavior, including Nagle's algorithm — §7.7.8 (the original batching trade-off).

23.3 Performance methodology

23.3.1 What performance engineering is

> DEFINITION
>
> **Definition — Performance engineering.** The practice of measuring, explaining and improving the **latency**, **throughput** and **resource efficiency** of a system against explicit targets, using repeatable methods rather than intuition. Its outputs are diagnoses (why is it slow?), changes (make it faster or cheaper) and predictions (how much capacity will we need?).

First-principles explanation

A doctor facing a fever does not start with antibiotics. She takes vital signs in a fixed order, forms a hypothesis, orders a targeted test, treats, and follows up. In performance work the vital signs are a few metrics per resource and per service, the tests are profilers and tracers, and the follow-up is a measurement confirming the fix.

Brendan Gregg names the **anti-methods** that replace this discipline [Gregg 2020]: the *streetlight* anti-method (look only where your favorite tool shines), *random change* (tweak settings until the problem goes away) and *blame-someone-else* ("it's the network"). The methods below are evidence-driven and take minutes.

23.3.2 The USE method

DEFINITION

Definition — USE method. For every **resource**, check **U**tilization (the fraction of time the resource was busy, or the fraction of its capacity in use), **S**aturation (the amount of extra work queued because the resource could not service it) and **E**rrors (the count of error events) [Gregg USE]. A resource is any physical or software component with finite capacity: CPUs, memory, disks, network interfaces, controllers and interconnects, but also locks, thread pools, connection pools and file descriptors.

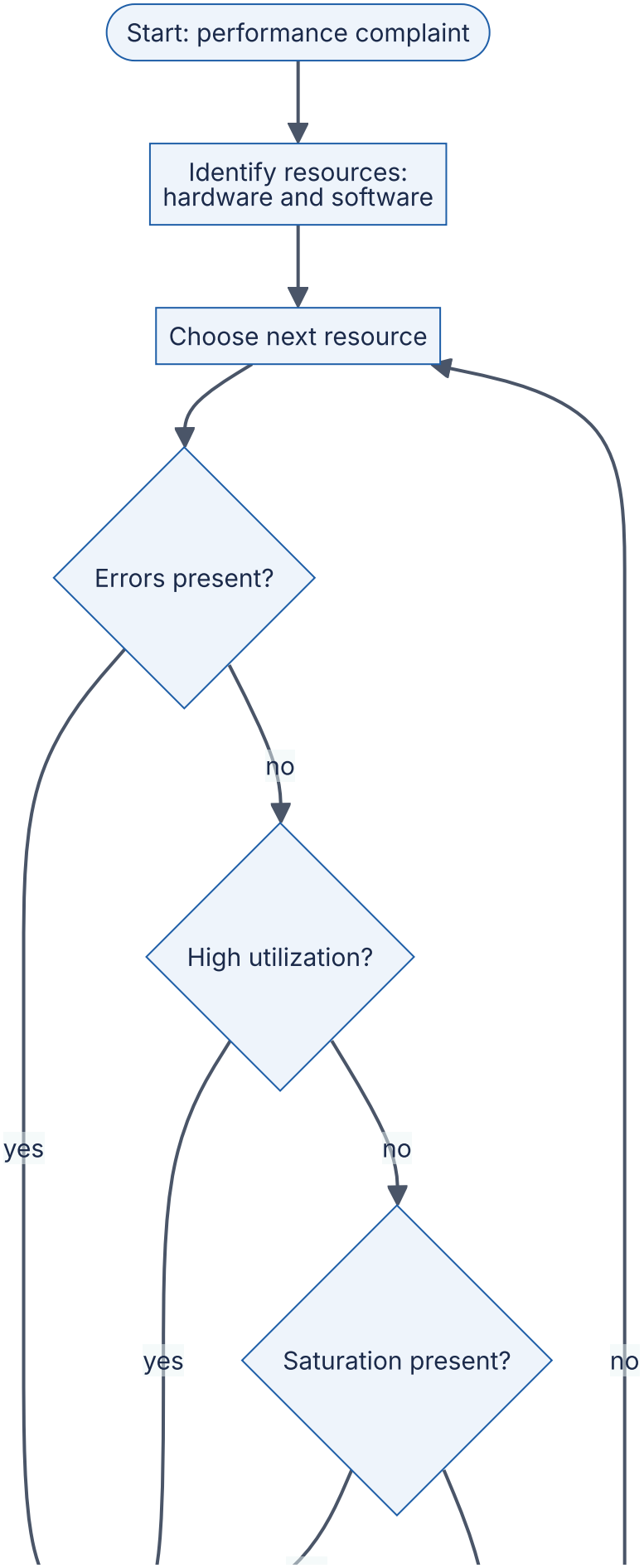
Why does this exist?

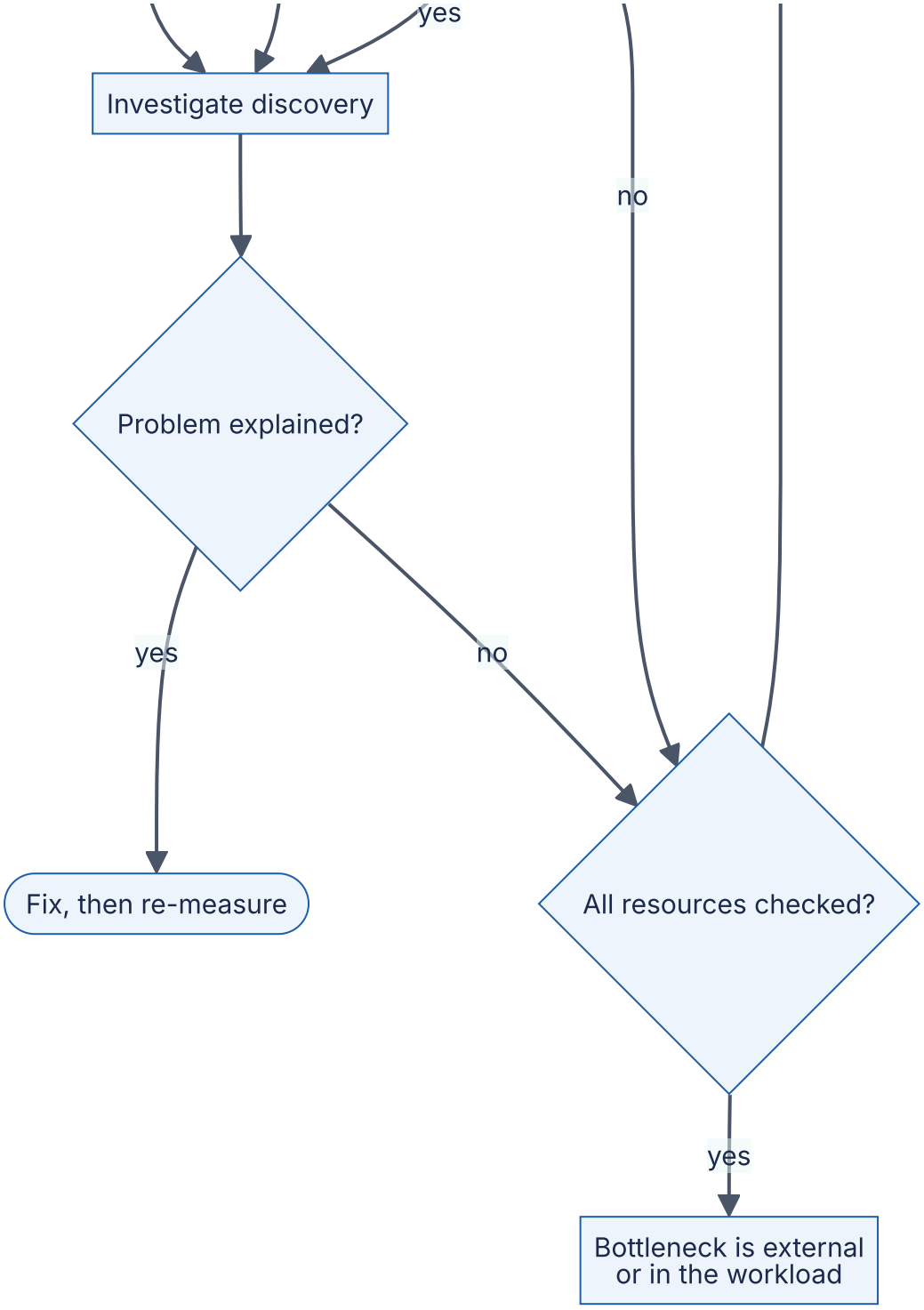
Dashboards built from whatever metrics were easy to collect are both overwhelming and incomplete. USE inverts the process: start from the finite list of resources and ask three questions of each, so no saturated resource is missed because nobody graphed it. Chapter 2 introduced USE as a bottleneck finder (§2.6); here it becomes a complete procedure.

How it works internally

1. **Enumerate resources** from a block diagram of the server: CPUs, memory, disks and controllers, network interfaces, plus software resources.
2. **Check errors first.** They are quick to interpret (disk retries, NIC drops) and can explain everything else.
3. **Check utilization** per device and per CPU: an 8-core average of 25% can hide one core at 100% running a single-threaded event loop.
4. **Check saturation**: run-queue length, swapping, disk queue size, drops and retransmits, waiters on locks and pools. Sustained saturation is always a problem, because queued work is latency (§4.10).
5. **Iterate** until the problem is found or every resource is cleared.

Figure 23.1 — The USE method procedure.


Start: performance complaint
Identify resources: hardware and software
Choose next resource
Errors present?
yes
no
High utilization?
yes
no
Saturation present?
no



Reading the diagram. This is a reasoning loop, not a running system: there are no protocols or storage boundaries, and every arrow is a step the engineer takes. Iterating over a finite resource list is what makes the method complete, and each check is one counter, so a pass takes minutes. A finding must *explain* the complaint (a busy disk may be a symptom of swapping), otherwise the loop continues. The method's failure point is the resource list: a forgotten resource (a connection pool, a file-descriptor limit, a cloud API quota) can never be found. If every resource is clear, look outside the host (§23.3.3). The same checks scale to fleets per instance and resource class.

Table 23.1 — USE metrics for common resources (Linux tools in parentheses).

Resource	Utilization	Saturation	Errors
CPU	Per-CPU busy % (`mpstat -P ALL`)	Run-queue length; scheduler latency (`vmstat` r column, `runqlat`)	Machine-check events (rare)
Memory	Used vs. available (`free`)	Swapping, page scanning, OOM kills (`vmstat` si/so, `dmesg`)	Allocation failures
Disk	Busy % per device (`iostat -x`)	Average queue size; await time (`iostat -x`)	Device errors (`smartctl`, kernel log)
Network interface	Throughput vs. link speed (`sar -n DEV`)	Drops, overruns, TCP retransmits (`nstat`, `ss -ti`)	Interface errors (`ip -s link`)
Container CPU limit	CPU time vs. quota	Throttled periods (`cpu.stat` in cgroup)	—
Thread / connection pool	Busy workers / pool size	Requests waiting for a worker	Rejected tasks, acquisition timeouts
Lock	Fraction of time held	Waiting threads; contention time	Deadlock detections, timeouts

The container row deserves emphasis: a container that exhausts its CPU **quota** is **throttled** for the rest of the period even if the host is idle (§23.6.3), so host CPU looks low while latency spikes.

23.3.3 The RED method and the four golden signals

USE looks at a system from the inside; users experience it request by request. Two request-centric checklists cover that view.

DEFINITION

Definition — RED method. For every **service**, monitor **R**ate (requests per second), **E**rrors (failed requests per second) and **D**uration (the distribution of request latency, as a histogram from which percentiles are read). Tom Wilkie proposed it while at Weaveworks as a service-level complement to USE [Wilkie 2018].

DEFINITION

Definition — Four golden signals. Google's site reliability engineering guidance says that if you can measure only four metrics of a user-facing system, measure **latency** (time to serve a request, tracking successful and failed requests separately), **traffic** (demand, such as requests per second), **errors** (rate of failed requests, explicit or implicit) and **saturation** (how "full" the service is, emphasizing its most constrained resource) [Beyer et al. 2016, ch. 6].

The golden signals are RED plus saturation. Separating success and failure latency matters: a backend failing fast in 2 ms makes the latency histogram look *better* while users suffer. Saturation is the leading signal: a pool at 90% occupancy says users are *about* to hurt.

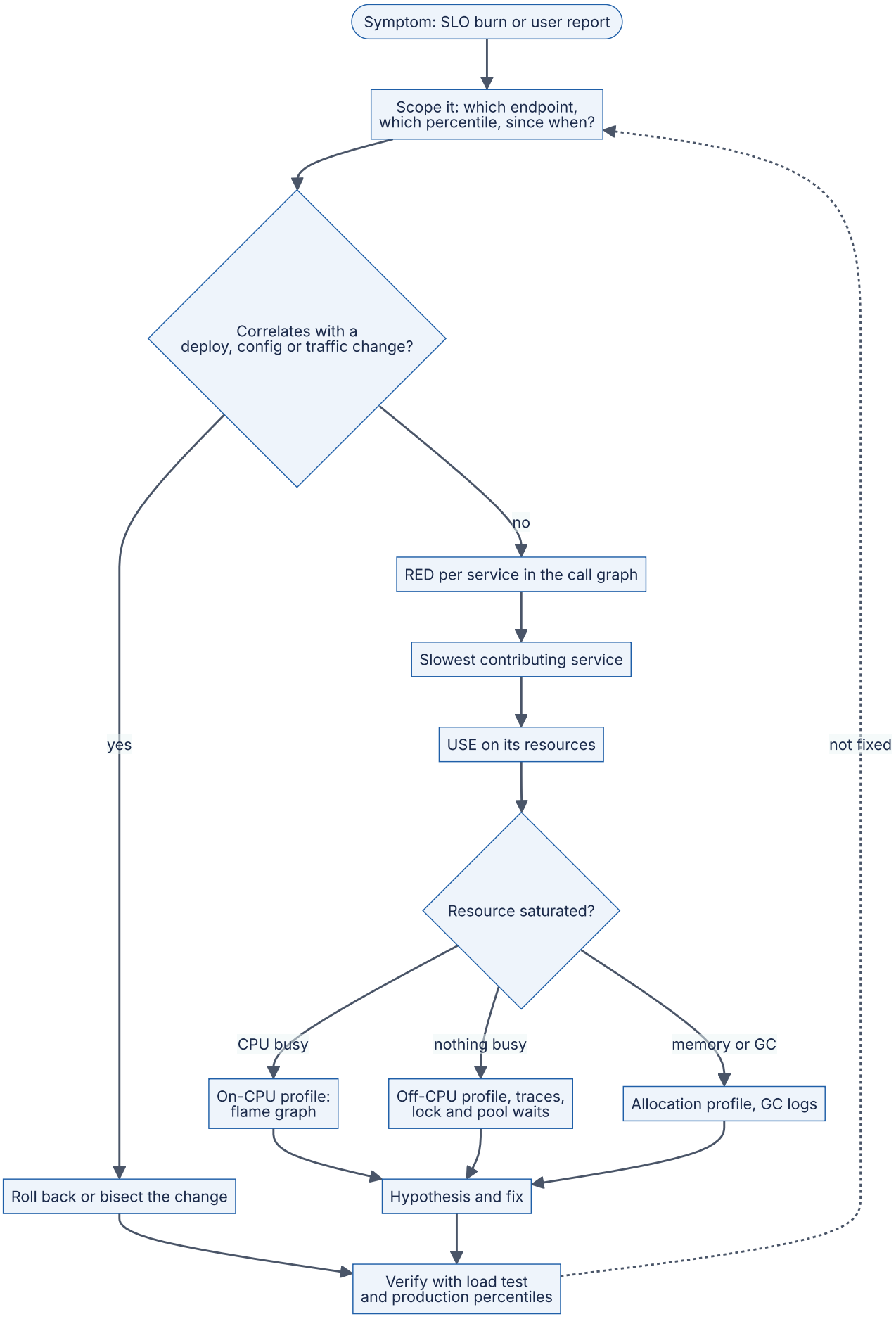
Table 23.2 — USE, RED and the four golden signals compared.

	USE	RED	Four golden signals
Unit of analysis	Resource	Service (endpoint)	User-facing service
Questions	Busy? Queued? Failing?	How much? How many fail? How long?	RED + how full?
Best for	Root-causing on one host or component	Finding *which* service in a call graph is slow	Alerting and SLOs
Typical source	OS counters, cgroup stats, pool gauges	Request middleware, service mesh, load balancer logs	Same as RED plus capacity gauges
Blind spot	Code-level causes; external dependencies	Why a service is slow	Why a service is slow
Origin	Gregg (2012 onward)	Wilkie (Weaveworks)	Google SRE book (2016)

23.3.4 From symptom to cause

A real investigation chains the checklists: RED narrows the problem to a service and endpoint, USE finds the saturated resource, a profiler (§23.4) or trace (§40.6) finds the responsible code or call, and a load test (§23.5) confirms the fix.

Figure 23.2 — Workflow from symptom to cause.


Symptom: SLO burn or user report
Scope it: which endpoint, which percentile, since when?
Correlates with a deploy, config or traffic change?
yes
no
RED per service in the call graph
Slowest contributing service
USE on its resources
Resource saturated?
CPU busy
nothing busy
memory or GC
On-CPU profile: flame graph
Off-CPU profile, traces, lock and pool waits
Allocation profile, GC logs
Roll back or bisect the change
Hypothesis and fix
Verify with load test and production percentiles
not fixed

Reading the diagram. Start from a user-visible symptom and first ask *what changed*: most regressions follow a deployment, configuration change or traffic shift, and rollback is the fastest fix. Otherwise RED metrics across the call graph (from middleware or a service mesh, over HTTP or gRPC) locate the slowest service, and USE on its resources selects the instrument: a CPU flame graph if CPUs are busy; off-CPU analysis and traces if nothing is busy but requests wait (locks, pools, dependencies); allocation profiles and GC logs if memory dominates. Solid steps are synchronous actions; the dotted edge reopens the investigation if verification fails. The main failure point is skipping scoping: “the site is slow” without an endpoint, percentile and start time cannot converge.

WORKED EXAMPLE

Worked example — Locating the bottleneck with the utilization law. RED shows the `/search` endpoint’s p99 rose from 120 ms to 900 ms at 2,400 RPS across 12 instances. USE on one instance (200 RPS) shows 8 CPUs at 35%, and a database connection pool of 16 connections reporting 14.8 busy on average with 40 waiters. Each request holds a connection for 74 ms.

1. Connection pool utilization by Little’s Law (§4.11): $L = \lambda W = 200 \times 0.074 = 14.8$ connections, so $U = 14.8/16 = 92.5\%$.
2. CPU utilization is 35% per CPU, far from saturation, and the run queue is short.
3. Saturation evidence: 40 waiters on the pool. At 92.5% utilization a queue is expected (§4.10).
4. Hypothesis: connection hold time rose. Tracing shows a new query taking 50 ms inside each request; before the change hold time was 24 ms, giving $200 \times 0.024 = 4.8$ busy connections (30%).

Interpretation. The CPU dashboard (35%) said “plenty of capacity”; the pool said “saturated”. **Engineering implication.** Software resources must be in your USE list and on your dashboards. The fix is to make the query faster or move it off the request path, not to add instances: twelve more instances would double the connection count the database must serve.

COMMON MISTAKE

Mistake — Treating utilization as the only signal. A disk at 60% utilization can still be the bottleneck if its queue is long and its requests are bursty, and a CPU at 100% may be harmless for a batch job. Utilization tells you how busy a resource is; saturation tells you whether work is *waiting* for it. Latency comes from waiting.

23.4 Profiling CPU and memory

23.4.1 Sampling versus instrumentation

DEFINITION

Definition — Profiler. A tool that attributes a resource (CPU time, wall-clock time, allocated bytes, lock wait time) to the code locations that consumed it. A **sampling profiler** interrupts the program at a fixed frequency and records the current call stack; a function's share of samples estimates its share of time. An **instrumenting** (deterministic, tracing) profiler inserts hooks at every function entry and exit and records exact call counts and durations.

First-principles explanation

To learn how a colleague spends the day you could follow them with a stopwatch (instrumentation) or glance at their desk a few hundred times at random and tally what you see (sampling). The stopwatch is exact but intrusive; the glances are cheap and give percentages accurate to a point or two. Production profiling almost always samples, because overhead is fixed by the sampling rate, not by how many functions the program calls.

How it works internally

1. A timer or hardware counter fires an interrupt or signal.
2. The handler walks the running thread's stack (via frame pointers, DWARF unwind tables or a runtime API), including kernel frames.
3. Addresses are later **symbolized** into function names, and identical stacks are counted; each count divided by the total estimates that stack's share of CPU time.

Linux `perf` typically samples at 99 Hz rather than 100 Hz to avoid lockstep with periodic activity [Gregg 2020].

FORMULA

Sampling error of a profile

$$n = f \times T \times c \qquad \hat{p} = \frac{k}{n} \qquad \mathrm{SE}(\hat{p}) = \sqrt{\frac{\hat{p}\,(1-\hat{p})}{n}}$$

Symbol	Meaning	Units
f	Sampling frequency per CPU	samples/second
T	Profiling duration	seconds
c	Number of busy CPUs sampled	count
n	Total samples collected	samples
k	Samples whose stack contains the function	samples
$\hat{p}$	Estimated fraction of CPU time in the function	dimensionless
SE	Standard error of the estimate (binomial approximation)	dimensionless

WORKED EXAMPLE

Worked example — How long to profile. A service runs on 8 busy CPUs. You sample at 99 Hz for 30 s.

1. $n = 99 \times 30 \times 8 = 23,760$ samples.
2. A function at $\hat{p} = 2\%$: $\mathrm{SE} = \sqrt{0.02 \times 0.98/23,760} = \sqrt{8.25 \times 10^{-7}} = 0.00091$, i.e. ± 0.09 percentage points; a 95% interval is about 2.0% ± 0.18%.
3. A function at $\hat{p} = 0.1\%$ (about 24 samples): $\mathrm{SE} = \sqrt{0.001 \times 0.999/23,760} = 0.000205$, a *relative* error of about 20%.

Interpretation. Thirty seconds measures anything that matters (above 1%) precisely; tiny functions are noisy, but they are not where the time is. **Engineering implication.** Profile for tens of seconds under representative load, and do not chase differences smaller than two standard errors between two profiles.

Instrumentation has exact counts but overhead proportional to call frequency: 10 million calls per second at 50 ns of hook cost adds 0.5 CPU-seconds per second, distorting exactly the small hot functions you want to measure. It suits call counts, coarse spans (distributed tracing, §40.6) and offline analysis.

23.4.2 Flame graphs

A profile of a real service contains tens of thousands of distinct stacks, far too many to read as text. Gregg's **flame graph** visualizes them [Gregg 2016]: each box is a function in a stack; the y-axis is stack depth (callers below, callees above); the **width** of a box is proportional to the number of samples containing it; boxes are sorted alphabetically along the x-axis so identical stacks merge. The x-axis is *not* time.

Figure 23.3 — From samples to a flame graph.

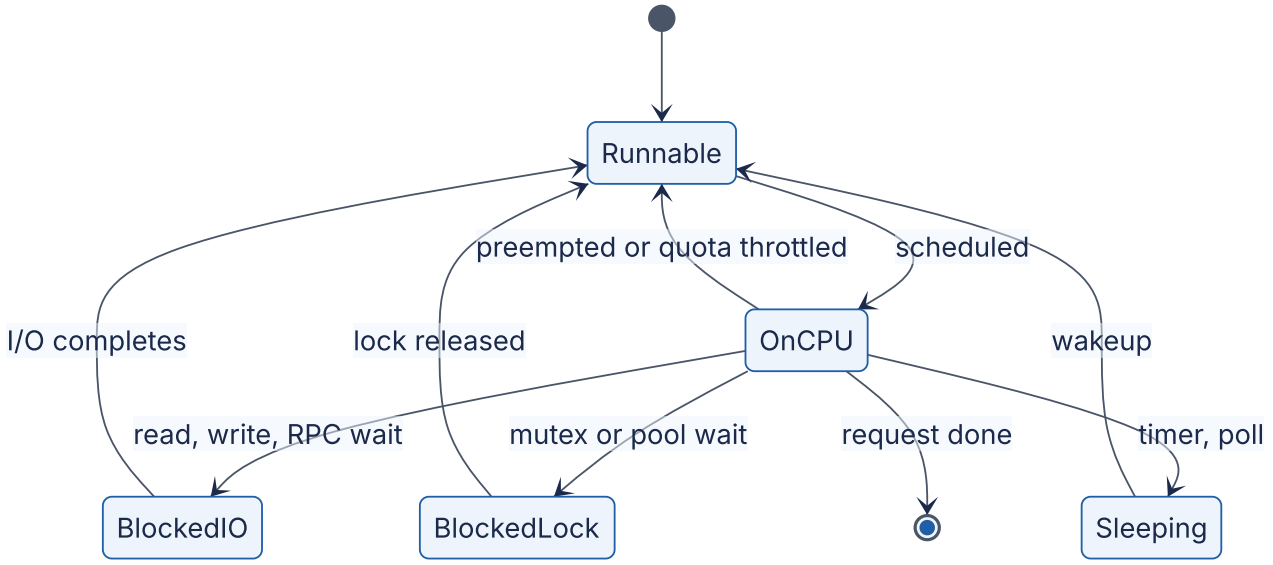


Reading the diagram. The sampler observes the service (perf and eBPF from the kernel, py-spy by reading process memory, async-profiler inside the JVM) and writes raw stacks to a profile file, the storage boundary; everything after it runs offline. Arrows are batch hand-offs, not request paths. Symbolization fails when symbols are stripped or JIT code lacks a symbol map, leaving hexadecimal frames. Collapsing merges identical stacks into counts that the flame graph draws as widths. Look first for wide **plateaus**, boxes with nothing on top, which burn CPU themselves. Continuous profiling on every host scales this to a fleet.

23.4.3 Off-CPU analysis

A CPU profile shows only time spent *running*. A request that takes 300 ms of wall-clock time but uses 5 ms of CPU is invisible to it. The missing 295 ms is **off-CPU time**: the thread was blocked on disk or network I/O, waiting for a lock, sleeping, or runnable but waiting for a CPU.

Figure 23.4 — On-CPU and off-CPU thread states.



Reading the diagram. Only `OnCPU` appears in a CPU profile. `Runnable` time is scheduler latency (CPU saturation or throttling, measured by `runqlat`); `BlockedIO` is waiting on storage or network calls; `BlockedLock` is mutex or pool contention; `Sleeping` is voluntary. Transitions out of `OnCPU` are context switches (§6.4). **Off-CPU profiling** records the stack and blocked duration at each switch-out, typically with eBPF (`offcputime`), and draws a flame graph whose width is blocked *time*. Wall-clock profilers (async-profiler's `wall` mode) sample all threads in any state. Pitfall: idle pool workers dominate off-CPU profiles, so filter to threads serving requests.

Request latency equals on-CPU plus off-CPU time. Latency problems in I/O-heavy services are usually off-CPU; throughput and cost problems usually on-CPU.

23.4.4 Tools by runtime

Table 23.3 — Common profilers (as of 2026).

Tool	Runtime	Method	Notable features
`perf`	Any (Linux)	Sampling via kernel perf events	CPU, hardware counters, kernel stacks; needs frame pointers or DWARF for user stacks
eBPF tools (BCC, bpftrace)	Any (Linux)	In-kernel programs on events	Low-overhead off-CPU, run-queue, block I/O latency histograms
py-spy	CPython	Sampling by reading process memory	No code change; attaches to a running PID; flame graph output
async-profiler	JVM	Sampling with perf events and JVM APIs	Avoids safepoint bias; CPU, allocation, lock and wall-clock modes
Java Flight Recorder	JVM	Built-in event recording	Low overhead, always-on in production; GC and lock events
pprof	Go	Built-in sampling (CPU 100 Hz), heap sampling	`net/http/pprof` endpoint; CPU, heap, goroutine, block, mutex profiles

Safepoint bias: older JVM profilers sampled threads only at **safepoints**, so time in loops without safepoints was misattributed, and a study of four Java profilers found they often disagreed about the hottest method [Mytkowicz et al. 2010]. Signal-based asynchronous profilers such as async-profiler avoid this.

23.4.5 Allocation profiling

In garbage-collected runtimes the **allocation rate** sets how often the collector runs (§23.9). Allocation profilers answer two questions: *who allocates most* (drives GC frequency) and *who holds memory now* (drives heap size); diffing heap snapshots taken minutes apart finds leaks. The program below profiles CPU with `cProfile` and memory with `tracemalloc`, counting distinct users in a request log with a list (O(n) membership test) and with a set.

```
        import cProfile
        import json
        import linecache
        import pstats
        import random
        import tracemalloc

        def make_events(n, seed=7):
            """Synthetic request log: JSON lines with a user id and a latency."""
            rng = random.Random(seed)
            return [json.dumps({"user": f"u{rng.randrange(5_000)}",
                                "ms": round(rng.lognormvariate(3, 1), 2)})
                    for _ in range(n)]

        def distinct_users_slow(events):
            seen = []                           # list: membership test is O(n)
            for line in events:
                user = json.loads(line)["user"]
                if user not in seen:
                    seen.append(user)
            return len(seen)

        def distinct_users_fast(events):
            return len({json.loads(line)["user"] for line in events})  # set: O(1)

        def report(events):
            return distinct_users_slow(events), distinct_users_fast(events)

        events = make_events(50_000)

        # 1. CPU: cProfile is a deterministic (instrumenting) profiler
        profiler = cProfile.Profile()
        profiler.enable()
        print("distinct users (slow, fast):", report(events))
        profiler.disable()
        stats = pstats.Stats(profiler)
        print(f"{'function':<22}{'calls':>8}{'self s':>9}{'total s':>9}")
        rows = sorted(((k, v) for k, v in stats.stats.items() if not k[2].startswith("
<")),
                      key=lambda kv: kv[1][3], reverse=True)[:6]       # by total time
        for (filename, line, name), (cc, nc, tottime, cumtime, _) in rows:
            print(f"{name[:21]:<22}{nc:>8}{tottime:>9.3f}{cumtime:>9.3f}")

        # 2. Memory: tracemalloc attributes live allocations to source lines
        tracemalloc.start()
        more_events = make_events(50_000)
        snapshot = tracemalloc.take_snapshot()
        tracemalloc.stop()
        for stat in snapshot.statistics("lineno")[:2]:
            frame = stat.traceback[0]
            code = linecache.getline(frame.filename, frame.lineno).strip()
            print(f"{stat.size / 1024:6.0f} KiB in {stat.count:6d} blocks: {code[:38]}")
```

Output (Python 3.13; timings vary by machine):

```
distinct users (slow, fast): (5000, 5000)
function                 calls   self s  total s
report                       1    0.000    1.486
distinct_users_slow          1    1.028    1.266
loads                   100000    0.092    0.436
decode                  100000    0.142    0.314
distinct_users_fast          1    0.021    0.220
raw_decode              100000    0.072    0.072
  3442 KiB in  50000 blocks: return _iterencode(o, 0)
   434 KiB in      3 blocks: return [json.dumps({"user": f"u{rng.ra
```

What to notice:

- **Self time versus total time.** `distinct_users_slow` has 1.03 s of *self* time: the `user not in seen` scan is an operator, not a function call, so cProfile charges it to the enclosing function. Its JSON parsing (`loads` , about 0.22 s per 50,000 calls) is the same in both versions. The fix (a set) makes the function about 6 times faster overall.
- **Call counts are exact**: 100,000 calls to `loads` (50,000 per version). A sampling profiler would give percentages but not counts.
- **Instrumentation overhead.** Each of the 100,000 `loads` calls passes through cProfile's hooks, so the absolute times are inflated; compare ratios, not absolutes.
- **tracemalloc** attributes 3.4 MiB of live memory to a line inside the JSON encoder: 50,000 strings of about 70 bytes each. The list comprehension holding them costs 434 KiB of pointers. In a production service the same view, diffed over time, finds leaks.

For a running production Python process you would use a sampling profiler instead: `py-spy record -o profile.svg --pid <PID> --duration 30` produces a flame graph without restarting or modifying the program.

Real-world industry applications

> INDUSTRY EXAMPLE
>
> **Google-Wide Profiling.** Google runs continuous, low-overhead sampling profiling across its fleet, collecting CPU and heap profiles from a small random subset of machines at a time and aggregating them in a central repository. The fleet-wide view lets engineers rank the most expensive functions across all services and attribute cost to specific libraries, which guides optimization work with the largest fleet-wide payoff. Trade-off: individual profiles are sparse, so the value comes from aggregation over many machines and long periods. *Evidence: Primary* — [Ren et al. 2010]

INDUSTRY EXAMPLE

Netflix — mixed-mode flame graphs for Java. Netflix engineers produced CPU flame graphs showing Java, JIT-compiled, kernel and native frames together by running the JVM with frame pointers preserved and exporting a symbol map for JIT code so `perf` can name it. Trade-off: frame pointers cost a little CPU, accepted for the visibility. *Evidence: Primary* — [Gregg and Spier 2015]

COMMON MISTAKE

Mistake — Optimizing the function with the largest *total* time. `main` has 100% total time and nothing to optimize. Look for large *self* time (the plateaus of a flame graph). Conversely, a function with small self time but huge total time is a good target only if you can avoid *calling* the expensive callees.

23.5 Load testing and benchmarking pitfalls

23.5.1 Kinds of performance test

Table 23.4 — Performance test types.

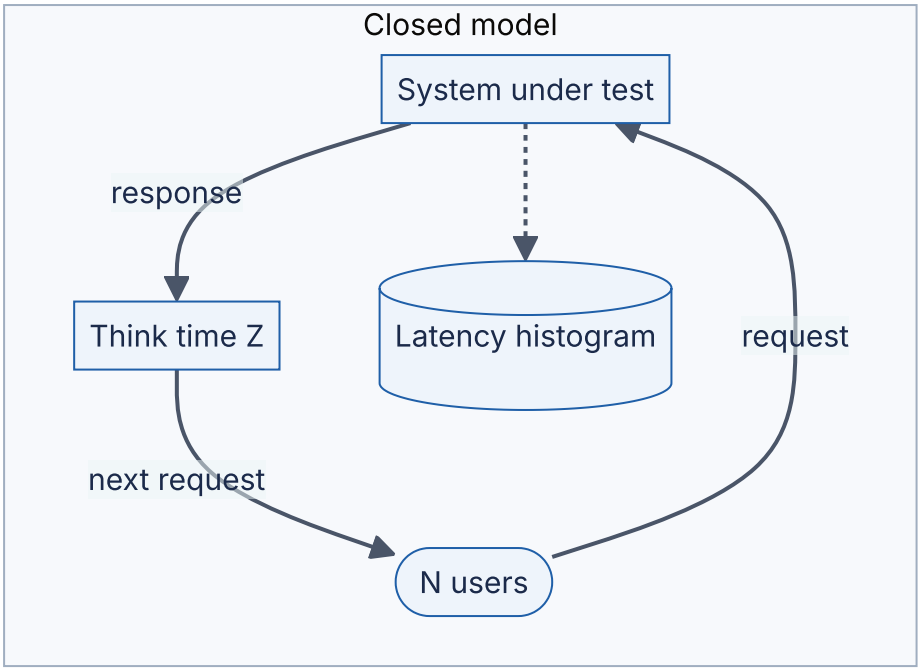
Test	Question it answers	Typical shape
Microbenchmark	How fast is this function or data structure?	Tight loop in a harness (JMH, `pytest-benchmark`, Go `testing.B`)
Load test	Does the system meet its SLO at expected peak?	Steady target rate at forecast peak for 30–60 min
Stress test	Where and how does it break?	Rate ramps until errors or latency explode
Soak (endurance) test	Does it degrade over hours?	Moderate load for 8–72 h; watch memory, handles, GC
Spike test	Does it survive a sudden surge, and recover?	Step from normal to 3–10× for minutes
Capacity benchmark	Throughput as a function of concurrency or nodes	Series of steady runs at increasing *N* (§23.10)

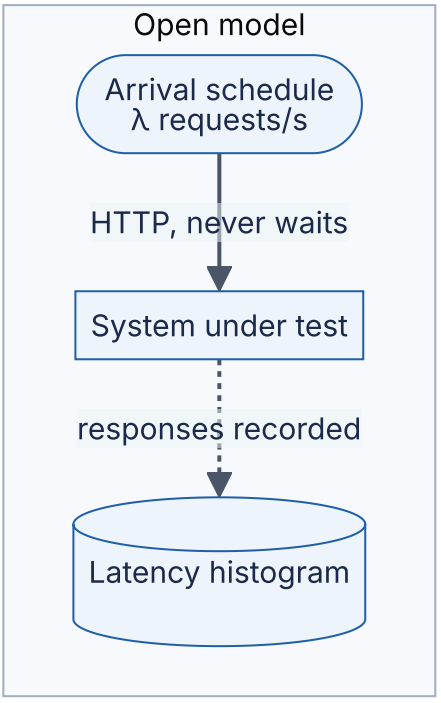
23.5.2 Open versus closed workload models

DEFINITION

Definition — Open and closed workload models. In an **open** model, new requests arrive at a rate that is independent of how fast the system answers: users on the internet do not wait for each other. In a **closed** model, a fixed population of N users (or load-generator threads) each sends a request, waits for the response, optionally pauses for a **think time** Z, and only then sends the next. A **partly open** model has sessions that arrive openly but issue several requests each, closed-loop, within the session [Schroeder et al. 2006].

Figure 23.5 — Open and closed load generators.





Reading the diagram. In the open model a scheduler issues requests (for example, HTTP) at rate λ and never waits, so when the system slows down requests pile up inside it, as real internet traffic does; responses are recorded asynchronously (dotted) into a histogram, the test's storage boundary. In the closed model each of N users loops synchronously: send, wait, think, repeat. When the system slows, users send *less*, so the generator reduces load exactly when the system struggles. That is the closed model's failure point: it cannot reach the queueing regime real users would cause. Open tests scale by

adding generator machines; closed tests by adding users, which raises load only while the system keeps up.

FORMULA

Interactive response time law (closed systems)

$$X = \frac{N}{R + Z} \qquad \Leftrightarrow \qquad R = \frac{N}{X} - Z$$

Symbol	Meaning	Units
N	Number of users (or load-generator threads) in the closed loop	count
X	Throughput	requests/second
R	Mean response time	seconds
Z	Mean think time between a response and the next request	seconds

This is Little's Law (§4.11) applied to the whole loop of users plus system [Lazowska et al. 1984].

WORKED EXAMPLE

Worked example — A closed test that cannot find the problem. A load test uses $N = 50$ threads with no think time ($Z = 0$). With $R = 10$ ms the generator produces $X = 50/0.010 = 5,000$ RPS. Production peak is 5,000 RPS, open.

1. Suppose a regression doubles service time so $R = 20$ ms at the same concurrency. The closed test now produces $X = 50/0.020 = 2,500$ RPS: it halves the offered load and reports latency of 20 ms, "a bit slower".
2. In production the 5,000 RPS keeps arriving. If capacity has fallen to about 2,500 RPS, utilization exceeds 100%, the queue grows without bound (§4.10.3), and latency rises until requests time out.

Interpretation. The closed test turns an outage into a mild regression, because it measures the system at whatever rate the system can sustain. **Engineering implication.** To check an SLO at a target rate, use an open (constant-arrival-rate) generator. Use closed tests only for systems that really are closed (a fixed pool of batch workers, a connection-limited client) or to measure capacity curves as a function of concurrency (§23.10).

Schroeder, Wierman and Harchol-Balter showed that at the same load the two models give very different response times and respond differently to scheduling policies, and that

many standard benchmarks used closed generators although web traffic is better described as open or partly open [Schroeder et al. 2006].

23.5.3 Coordinated omission

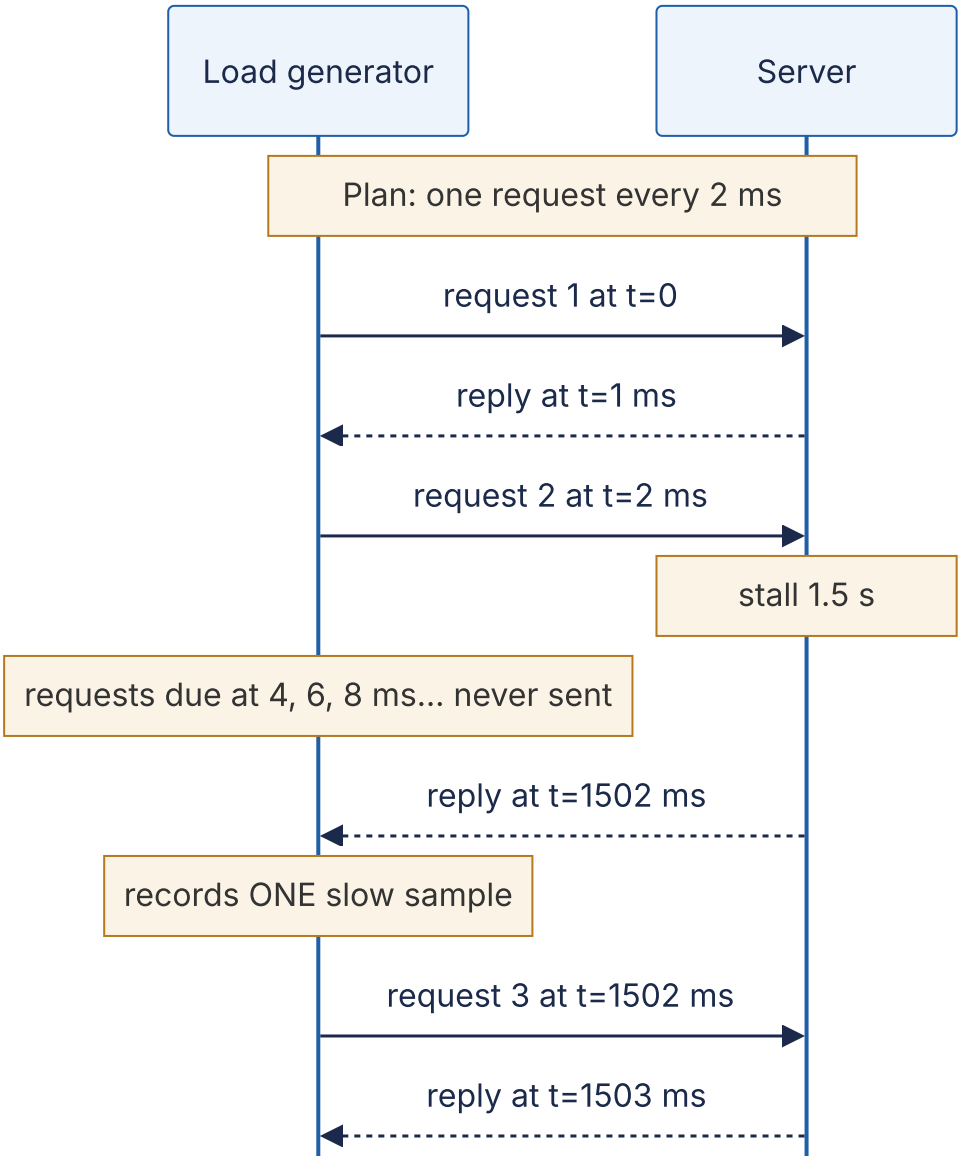
DEFINITION

Definition — Coordinated omission. A measurement error in which the load generator, by waiting for slow responses before sending more requests, fails to send (and therefore fails to measure) the requests that would have experienced the slowness. The tester and the system "coordinate" to omit the worst samples. The term and diagnosis are due to Gil Tene [Tene, How NOT to Measure Latency].

First-principles explanation

A manager measures checkout waits by sending one shopper through every two minutes. At noon the till jams for 30 minutes; the shopper reports one bad wait, and the next goes at 12:32. Fifteen real customers who would have waited 2–30 minutes were never sampled, so the report shows one bad wait in hundreds: a perfect-looking p99.

Figure 23.6 — How a closed-loop tester omits the stall.



Reading the diagram. The generator plans one request every 2 ms over a single connection and waits synchronously for each reply. When the server stalls for 1.5 s (a GC pause, a failover), request 2 waits throughout, the ~750 requests scheduled during the stall are never sent, and one 1.5 s sample is recorded. In an open model those requests would have queued at the server, and the backlog would have delayed requests after the stall too. The failure point is coupling sending to receiving; the fix is to decouple them and measure each request from its *intended* send time.

FORMULA

Damage done by a stall in an open system

$$\text{missed by closed tester} \approx \frac{T_s}{\Delta} \qquad t_{\text{drain}} = \frac{\lambda T_s}{\mu - \lambda} \qquad \text{affected requests} = \lambda (T_s + t_{\text{drain}}) = \frac{\lambda \mu T_s}{\mu - \lambda}$$

Symbol	Meaning	Units
T_s	Duration of the stall	seconds
$\Delta = 1/\lambda$	Intended interval between requests	seconds
λ	Intended (open) arrival rate	requests/second
μ	Service rate once the server resumes ($> \lambda$)	requests/second
t_{drain}	Time to drain the backlog after the stall	seconds

WORKED EXAMPLE

Worked example — A 1.5 s stall in a 60 s test. $\lambda = 500$ RPS, $\Delta = 2$ ms, $\mu = 1,000$ RPS (1 ms per request), $T_s = 1.5$ s, test length 60 s (30,000 intended requests).

1. Missed by the closed tester: $1.5/0.002 = 750$ requests (about 2.5% of the test).
2. Backlog at the end of the stall: $500 \times 1.5 = 750$ requests; it drains at the net rate $\mu - \lambda = 500$/s, so $t_{\text{drain}} = 1.5$ s.
3. Affected requests: $500 \times (1.5 + 1.5) = 1,500$, which is 5% of the test. Their latencies fall linearly from 1.5 s to about 0.
4. True p99 is the 300th worst of these: $1.5 - 300 \times (1.5/1,500) = 1.2$ s.

Interpretation. The real p99 is 1.2 s; a closed tester reports the ordinary 1–2 ms. **Engineering implication.** Generators that wait for responses under-report every percentile a stall touches, often by orders of magnitude.

The following program simulates this exact scenario with three measurement strategies: the naive closed loop, the closed loop with HdrHistogram's after-the-fact correction (which adds the "missing" samples $v - \Delta, v - 2\Delta, \ldots$ for each long sample v), and an open-loop generator that measures from the intended send time.

```
        import math
        import random

        RATE = 500                    # intended request rate, requests/second
        INTERVAL = 1 / RATE           # 2 ms between scheduled sends
        DURATION = 60.0               # seconds of test
        STALL = (30.0, 31.5)          # server freezes for 1.5 s (GC pause, failover...)

        class Server:
            """Single-threaded FIFO server; service time ~1 ms; one 1.5 s stall."""
            def __init__(self, seed=1):
                self.rng = random.Random(seed)
                self.free_at = 0.0
            def submit(self, t):
                start = max(t, self.free_at)
                if STALL[0] <= start < STALL[1]:          # frozen: wait for the stall
to end
                    start = STALL[1]
                service = 0.0008 + self.rng.expovariate(1 / 0.0002)
                if start < STALL[0] < start + service:    # stall hits mid-request
                    service += STALL[1] - STALL[0]
                self.free_at = start + service
                return self.free_at                       # completion time

        def closed_loop():
            """Naive tester: one connection; waits for each reply before sending the
            next; measures from the actual send time (like wrk or a simple script)."""
            server, t, lat = Server(), 0.0, []
            while t < DURATION:
                done = server.submit(t)
                lat.append(done - t)
                t = max(t + INTERVAL, done)               # late? just carry on from now
            return lat

        def corrected(lat):
            """HdrHistogram-style correction: a reply that took v > INTERVAL implies
            missed sends that would have seen v - INTERVAL, v - 2*INTERVAL, ..."""
            out = []
            for v in lat:
                out.append(v)
                missing = v - INTERVAL
                while missing > INTERVAL:
                    out.append(missing)
                    missing -= INTERVAL
            return out

        def open_loop():
            """Correct tester: sends on a fixed schedule whatever happens and measures
            from the *intended* send time (like wrk2, or k6 constant-arrival-rate)."""
            server, lat = Server(), []
            for i in range(int(DURATION * RATE)):
                t = i * INTERVAL
                lat.append(server.submit(t) - t)
            return lat

        def pct(sorted_ms, q):
```

```
            return sorted_ms[max(math.ceil(q / 100 * len(sorted_ms)), 1) - 1]

        print(f"{'tester':<22}{'count':>7}{'p50':>8}{'p90':>8}{'p99':>8}"
              f"{'p99.9':>8}{'max':>8}   (ms)")
        for name, lat in (("closed loop (naive)", closed_loop()),
                          ("closed + correction", corrected(closed_loop())),
                          ("open loop (truth)", open_loop())):
            ms = sorted(v * 1000 for v in lat)
            print(f"{name:<22}{len(ms):>7}" +
                  "".join(f"{pct(ms, q):>8.1f}" for q in (50, 90, 99, 99.9)) +
                  f"{ms[-1]:>8.0f}")
```

Output:

```
tester                 count     p50     p90     p99   p99.9     max   (ms)
closed loop (naive)    29245     0.9     1.3     1.7     2.1    1500
closed + correction    29994     0.9     1.3   902.2  1442.2    1500
open loop (truth)      30000     1.0     1.4  1199.8  1471.4    1501
```

What to notice:

- The naive tester sent 755 fewer requests and reports p99 = 1.7 ms. The truth is 1,200 ms, as the worked example predicted: a 700-fold understatement. Only the max (1.5 s) hints at the stall, and averages and p99 dashboards hide it.
- The correction recovers most of the tail (p99 902 ms) but not all: it reconstructs the requests missed *during* the stall but not the backlog that would have delayed requests *after* it, because the naive tester never built a queue.
- The open-loop tester's numbers are the only ones that describe what users would see. Its generator must itself never stall: real generators use many connections or asynchronous I/O so that a slow reply does not delay the next scheduled send.

23.5.4 Warm-up, think time and other pitfalls

- **Warm-up.** Early minutes run with cold caches, empty pools and interpreted code (§23.9.3). Report warm-up separately: it is what users see after every deployment.
- **Think time.** Closed tests with $Z = 0$ model nobody. Use measured think times, or an open model if you need a target rate.
- **Generator saturation.** A generator at 100% CPU delays its own sends, which is coordinated omission from the client side. Apply USE to generators too.
- **Unrealistic data.** Repeated keys hit caches; replay production traffic or sample keys from the measured popularity distribution.
- **Too-short runs.** Near saturation averages converge slowly (§4.10.3), and leaks and GC behavior appear only in soak tests.
- **Averaging percentiles.** Merge histograms, then read the percentile (§4.8.3).

23.5.5 Recording latency with HDR histograms

Fixed-width buckets lose resolution at one end of the range. **HdrHistogram** preserves a chosen number of **significant decimal digits** over a wide range by splitting power-of-two buckets into linear sub-buckets, so *relative* error stays below 10^{-d} [Tene, HdrHistogram].

FORMULA

HdrHistogram footprint (simplified from the reference implementation)

$$s = 2^{\lceil \log_2 (2 \times 10^d) \rceil} \qquad b = 1 + \lceil \log_2 \frac{H}{s \cdot L} \rceil \qquad \text{counters} = (b+1)\frac{s}{2} \qquad \text{memory} \approx 8 \times \text{counters}$$

Symbol	Meaning	Units
d	Significant decimal digits preserved	digits
s	Sub-buckets per bucket	count
L, H	Lowest discernible and highest trackable values	e.g. µs
b	Number of power-of-two buckets	count
memory	Size with 64-bit counters	bytes

WORKED EXAMPLE

Worked example — 1 µs to 1 hour at 3 significant digits. $d = 3$, $L = 1$ µs, $H = 3.6 \times 10^9$ µs.

1. $s = 2^{\lceil \log_2 2{,}000 \rceil} = 2^{11} = 2{,}048$.
2. $H/(sL) = 3.6 \times 10^9 / 2{,}048 = 1.76 \times 10^6$; $\log_2$ of that is 20.7, so $b = 1 + 21 = 22$.
3. Counters $= 23 \times 1{,}024 = 23{,}552$; memory $= 23{,}552 \times 8 = 188{,}416$ bytes ≈ 184 KiB.

Interpretation. Under 200 KB records any latency from 1 µs to an hour with 0.1% relative error, and histograms merge by adding counters. **Engineering implication.** Record latency in HDR or similar log-linear histograms in generators and services, and ship histograms, not precomputed percentiles.

23.5.6 Load-testing tools

Table 23.5 — Load generators and their workload models (as of 2026).

Tool	Scripting	Open model	Closed model	Notes
wrk2	Lua	Yes: constant throughput, `-R` rate	—	Measures from intended send time; HdrHistogram output
k6	JavaScript	`constant-arrival-rate`, `ramping-arrival-rate` executors	`constant-vus`, `ramping-vus` executors	Documents the open/closed distinction explicitly
Gatling	Scala, Java, Kotlin	`constantUsersPerSec`, `rampUsersPerSec` injection	`constantConcurrentUsers` injection	Detailed HTML reports
Locust	Python	Approximate via `constant_throughput` per user	Default: users with `wait_time`	Easy to write; one user per greenlet
wrk (original)	Lua	—	Yes	Fast, but subject to coordinated omission

INDUSTRY EXAMPLE

Facebook — Kraken load tests with live traffic. Instead of synthetic load, Kraken shifts a controlled share of *real* user traffic onto a target cluster or region by adjusting load-balancer weights, monitors health metrics, and stops automatically if latency or errors degrade. Regular tests exposed bottlenecks synthetic tests missed. Trade-off: the "test load" is real users, so safety stops must be fast and automatic. *Evidence: Primary* — [Veeraraghavan et al. 2016]

COMMON MISTAKE

Mistake — "The load test passed at 10,000 RPS" when the generator achieved 6,000. Closed-loop generators report whatever rate the system allowed. Print the *achieved* rate next to latency results, and fail the test if it falls short.

23.6 Tail latency at scale

23.6.1 Fan-out amplification, quantified

Section 4.8.4 showed that a root waiting for N independent leaves is slow whenever *any* leaf is slow: with a leaf p99 threshold, $1 - 0.99^N$ of root requests exceed it. The same idea gives the root's whole latency distribution.

FORMULA

Percentiles of a fan-out request

$$P(T_{\text{root}} \le t) = F(t)^N \qquad t_q^{\text{root}} = F^{-1}(q^{1/N}) \qquad \text{exponential leaves: } t_q^{\text{root}} = m \ln \frac{1}{1 - q^{1/N}}$$

Symbol	Meaning	Units
$F(t)$	CDF of one leaf's latency	dimensionless
N	Number of leaves the root waits for	count
q	Desired root percentile (as a fraction)	dimensionless
t_q^{root}	Root latency at percentile q	ms
m	Mean leaf latency for exponentially distributed leaves	ms

WORKED EXAMPLE

Worked example — 100 leaves with exponential latency. Leaf latency is exponential with mean $m = 10$ ms.

1. One leaf's p99: $10 \ln 100 = 46.1$ ms.
2. Root p50 with $N = 100$: $0.5^{1/100} = 0.99309$, so $t = 10 \ln (1/0.00691) = 10 \times 4.975 = 49.8$ ms.
3. Root p99: $0.99^{1/100} = 0.9998995$, so $t = 10 \ln (1/0.0001005) = 10 \times 9.205 = 92.1$ ms.

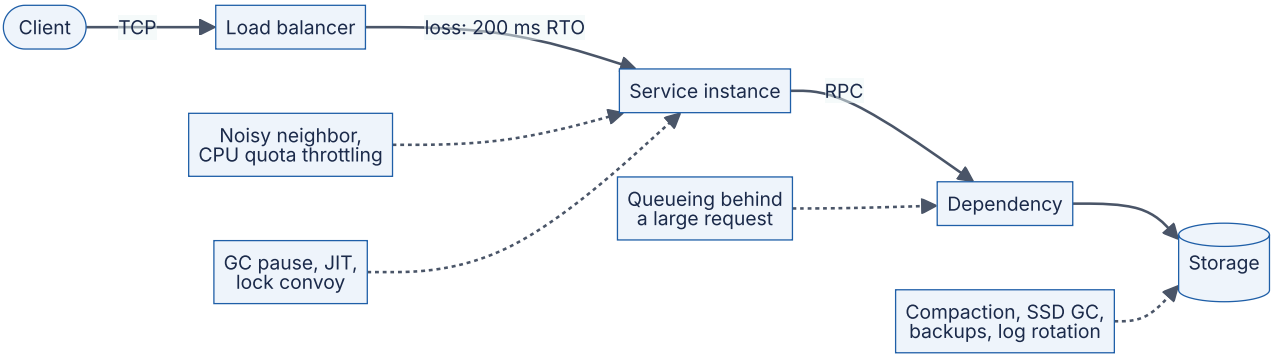
Interpretation. The root's *median* (49.8 ms) is worse than one leaf's p99 (46.1 ms). **Engineering implication.** In fan-out services the leaf tail *is* the user-facing median: engineer leaves to extreme percentiles or use tail-tolerant techniques (§23.7).

Real leaves are not fully independent: a shared cause (a switch hiccup) slows many at once. Dean and Barroso measured a large fan-out service at Google and reported that waiting for the slowest 5% of leaf requests accounted for roughly half of the root's p99 latency [Dean and Barroso 2013].

23.6.2 Sources of variability

Dean and Barroso list the usual suspects: contention with co-located work, daemons and background activities, shared global resources (switches, file systems), maintenance, queueing at many layers, power and thermal throttling, SSD garbage collection and runtime garbage collection [Dean and Barroso 2013].

Figure 23.7 — Where variability enters a request.



Reading the diagram. Solid arrows are the synchronous request path (TCP, then HTTP or RPC, then a storage read at the storage boundary). Each dotted arrow is an asynchronous disturbance that delays the request without belonging to it: a dropped packet costs a ≥ 200 ms retransmission timeout (§7.7); co-located tenants, CPU quotas (§23.6.3) and runtime pauses (§23.9) stall the instance; a request queued behind an expensive one inherits its delay; compaction (§13.9), SSD garbage collection (§5.6) and backups steal storage I/O. Every box is a tail-latency failure point, and scaling out multiplies their combined effect.

Table 23.6 — Common sources of tail latency.

Source	Typical magnitude	Detection	Mitigation
GC stop-the-world pause	1 ms – seconds	GC logs; pause histograms	Lower allocation; concurrent collector (§23.9)
Queueing behind large requests	Size of the large request	Per-request size vs. latency	Split large requests; priority classes
Background tasks (compaction, backups)	10s–100s of ms	Latency correlates with job schedule	Throttle; schedule off-peak; stagger across replicas
Noisy neighbor	10s of % slowdown	Cache misses, CPU steal time	Isolation, dedicated hosts
CPU quota throttling	Rest of a 100 ms period	`nr_throttled` in cgroup `cpu.stat`	Raise or remove limits; fewer threads
Packet loss	≥ 200 ms per retransmission	TCP retransmit counters	Avoid drops; hedging (§23.7)

23.6.3 A worked trap: CPU quota throttling

Linux enforces container CPU limits (including Kubernetes limits) with the CFS bandwidth controller: in each **period** P (100 ms by default) a cgroup may use at most its **quota** Q of CPU time across all its threads.

FORMULA

Throttling under a CPU quota

$$t_{\text{run}} = \frac{Q}{k} \qquad t_{\text{throttled}} = P - \frac{Q}{k} \qquad (\text{when } k > Q/P)$$

Symbol	Meaning	Units
P	Enforcement period	ms
Q	CPU-time quota per period (2 CPUs = $2P$)	CPU·ms
k	Threads runnable at the same time	count
t_{run}	Wall time until the quota is exhausted	ms
$t_{\text{throttled}}$	Wall time the whole cgroup is frozen	ms

WORKED EXAMPLE

Worked example — Two CPUs of quota, eight busy threads. $P = 100$ ms, $Q = 200$ ms (a limit of 2 CPUs), and a burst makes $k = 8$ threads runnable on an 8-core host.

1. $t_{\text{run}} = 200/8 = 25$ ms; $t_{\text{throttled}} = 100 - 25 = 75$ ms.
2. A request needing 40 ms of CPU on one of those threads gets 25 ms, waits 75 ms, then finishes after 15 ms more: latency 115 ms instead of 40 ms.
3. Average CPU use over the period is still only 2 cores out of 8, so host dashboards show 25%.

Interpretation. A limit expressed as an *average* (2 CPUs) is enforced over a short window, so bursty multi-threaded work is frozen for most of each period.
Engineering implication. Watch throttled-period counters, size thread pools to the quota rather than the host's core count (runtimes such as the JVM and Go read the limit for this reason in recent versions), and consider CPU requests without hard limits for latency-sensitive services.

Within a server, mitigations are **service classes** (interactive before batch), **splitting large requests**, and **managing background activity** (throttle it, or synchronize it across machines so it disturbs one short window) [Dean and Barroso 2013]. Across servers, it is often better to *tolerate* variability, as the next section shows.

23.7 Hedged and tied requests

23.7.1 Hedged requests

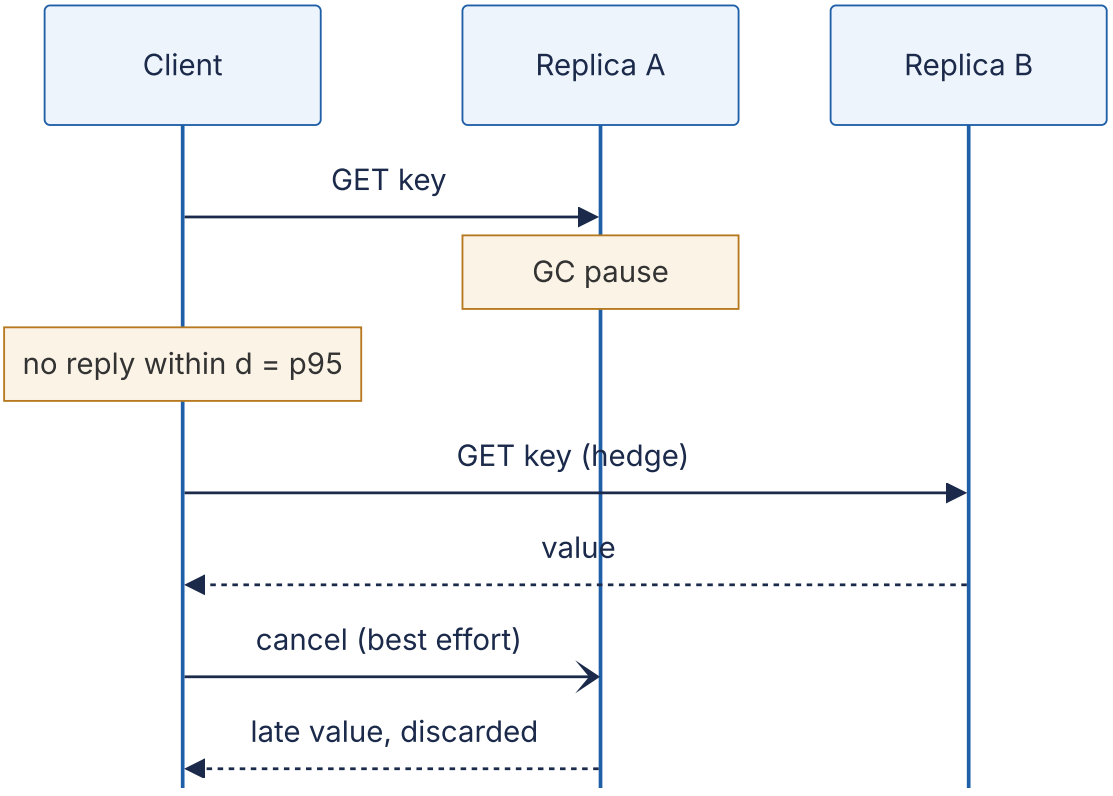
DEFINITION

Definition — Hedged request. A request sent to one replica and, if no response arrives within a **hedge delay** d, sent again to a second replica; the client uses whichever response arrives first and cancels or ignores the other [Dean and Barroso 2013].

First-principles explanation

You order a taxi through one app; if no driver accepts within five minutes, you also order through a second and take whichever car comes first. An occasional canceled booking buys protection from the rare 40-minute wait. The key choice is the delay: order from both at once and you double the load on drivers; wait too long and the hedge is useless.

Figure 23.8 — A hedged request.



Reading the diagram. The client sends a synchronous RPC to replica A, which is paused. When the hedge timer d expires, the client sends the same idempotent read to replica B and uses B's earlier answer. The cancel to A is asynchronous and best effort, so A's work may be wasted. Both replicas must hold the data (a replication boundary), and the operation must be safe to run twice (reads, or writes with an idempotency key,

§22.15). The failure point is load: if the whole system slows, many requests exceed d and hedging adds load when there is none to spare.

FORMULA

Hedged latency and extra load (independent replicas)

$$T = \min(X_1,\ d + X_2) \qquad P(T > t) = \begin{cases} P(X > t) & t < d \\ P(X > t)\,P(X > t - d) & t \geq d \end{cases} \qquad \text{extra load} = P(X > d)$$

Symbol	Meaning	Units
X_1, X_2	Latencies of the primary and the hedge, same distribution as X	ms
d	Hedge delay	ms
T	Latency seen by the client	ms
extra load	Fraction of requests that send a second copy	dimensionless

WORKED EXAMPLE

Worked example — Hedging a bimodal leaf. A replica answers in 8 ms with probability 0.98 and in 400 ms with probability 0.02 (a pause). Without hedging, p95 = 8 ms and p99 = 400 ms. Hedge after $d = 10$ ms.

1. With probability 0.98 the first reply arrives at 8 ms; no hedge is sent.
2. With probability 0.02 the hedge goes at 10 ms. It returns at $10 + 8 = 18$ ms with probability 0.98; otherwise both are slow and the reply arrives at 400 ms.
3. $P(T > 18\text{ ms}) = 0.02 \times 0.02 = 0.0004$. So p99 and p99.9 fall from 400 ms to 18 ms; only p99.96 and above remain at 400 ms.
4. Extra load $= P(X > 10\text{ ms}) = 2\%$.
5. With a 100-leaf fan-out: $P(\text{all leaves} \leq 18\text{ ms}) = (1 - 0.0004)^{100} = 0.961$, versus $0.98^{100} = 0.133$ unhedged.

Interpretation. Two percent more requests turn 87% slow fan-out queries into 4%. **Engineering implication.** Hedge idempotent operations after about the p95, bounding extra load near 5%.

23.7.2 Simulating hedging, including its cost

The worked example ignores a feedback: hedges add load, load adds queueing, and higher latency triggers more hedges. The simulation models 20 FIFO replicas with Poisson arrivals, about 1 ms of work per request, and a 50 ms hiccup on 0.1% of requests

that also delays everything queued behind it. It compares no hedging, hedging after the unhedged p95, hedging with cancellation (a copy not yet started when the original finishes is dropped), and hedging after the p50.

```
        import heapq
        import math
        import random

        SERVERS = 20
        N_REQ = 400_000

        def service_time(rng):
            """~1 ms of work; 0.1% of requests hit a 50 ms hiccup (GC, com-
paction...)."""
            t = 0.0005 + rng.expovariate(1 / 0.0005)
            if rng.random() < 0.001:
                t += 0.050
            return t

        def simulate(rho, hedge_after=None, cancel=False, seed=3):
            """FIFO replicas; Poisson arrivals; optional hedge to a second replica.
            Returns (latencies in ms, extra work as a fraction of original work)."""
            rng = random.Random(seed)
            lam = rho * SERVERS / 0.00105           # mean service time is 1.05 ms
            free_at = [0.0] * SERVERS
            events, t = [], 0.0                     # (submit_time, seq, req_id,
is_hedge, avoid)
            for i in range(N_REQ):
                t += rng.expovariate(lam)
                heapq.heappush(events, (t, i, i, False, -1))
            done = [0.0] * N_REQ
            sent = [0.0] * N_REQ
            seq, base_work, extra_work = N_REQ, 0.0, 0.0
            while events:
                now, _, rid, is_hedge, avoid = heapq.heappop(events)
                server = rng.randrange(SERVERS)
                while server == avoid:
                    server = rng.randrange(SERVERS)
                start = max(now, free_at[server])
                if is_hedge:
                    if cancel and done[rid] <= start:  # original already answered: drop
copy
                        continue
                    work = service_time(rng)
                    free_at[server] = start + work
                    extra_work += work
                    done[rid] = min(done[rid], free_at[server])
                else:
                    work = service_time(rng)
                    free_at[server] = start + work
                    base_work += work
                    sent[rid], done[rid] = now, free_at[server]
                    if hedge_after is not None and done[rid] > now + hedge_after:
                        seq += 1
                        heapq.heappush(events, (now + hedge_after, seq, rid, True,
server))
            lat = sorted((d - s) * 1000 for d, s in zip(done, sent))
            return lat, extra_work / base_work

        def pct(xs, q):
```

```
            return xs[max(math.ceil(q / 100 * len(xs)), 1) - 1]

        for rho in (0.5, 0.8):
            base, _ = simulate(rho)
            d = pct(base, 95) / 1000                # hedge after the unhedged p95
            print(f"utilization {rho:.0%}: hedge delay = p95 = {d*1000:.2f} ms")
            print(f"  {'policy':<16}{'p50':>8}{'p95':>8}{'p99':>8}{'p99.9':>8}{'extra
work':>12}")
            for name, kw in (("no hedging", {}),
                             ("hedge at p95", {"hedge_after": d}),
                             ("hedge + cancel", {"hedge_after": d, "cancel": True}),
                             ("hedge at p50", {"hedge_after": pct(base, 50) / 1000})):
                lat, extra = simulate(rho, **kw)
                print(f"  {name:<16}" + "".join(f"{pct(lat, q):>8.1f}" for q in (50, 95,
99, 99.9))
                      + f"{extra:>12.1%}")
```

Output (latencies in ms):

```
utilization 50%: hedge delay = p95 = 5.89 ms
  policy               p50     p95     p99   p99.9  extra work
  no hedging           1.3     5.9    44.6    61.0        0.0%
  hedge at p95         1.3     6.5     8.6    28.3        5.8%
  hedge + cancel       1.3     6.4     8.5    26.3        5.0%
  hedge at p50        48.6   244.1   401.8   579.2       98.7%
utilization 80%: hedge delay = p95 = 45.05 ms
  policy               p50     p95     p99   p99.9  extra work
  no hedging           2.7    45.0    68.2   127.0        0.0%
  hedge at p95      3456.8 10502.8 11149.5 11550.1       75.5%
  hedge + cancel       3.2    45.7    51.7    74.6        3.9%
  hedge at p50      7696.8 14771.3 15513.6 15978.4      100.2%
```

What to notice:

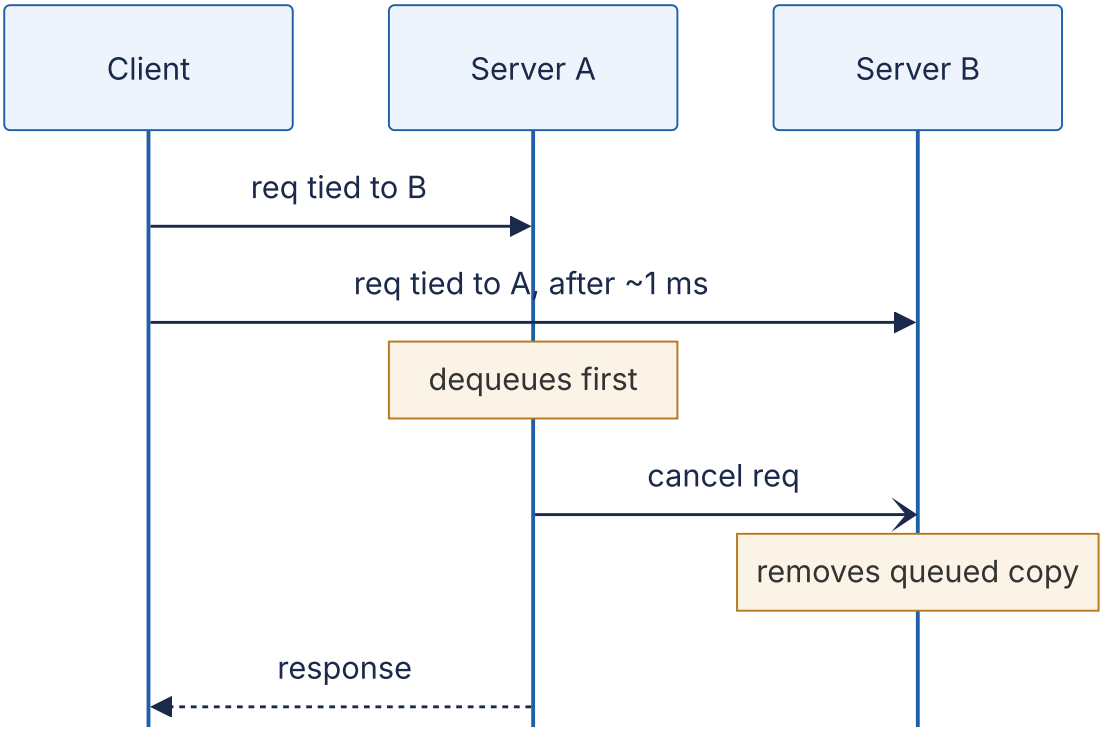
- **At 50% utilization, hedging at p95 cuts p99 from 44.6 ms to 8.6 ms** (5×) and p99.9 by more than half, for about 6% extra work. The median is untouched.
- **At 80% utilization the same policy without cancellation collapses** in a positive feedback loop: hedges raise utilization, queues grow, more requests exceed *d*. Extra work reaches 75%, demand exceeds capacity, and latency grows for as long as the run lasts. *With* cancellation the policy stays stable and still cuts the tail.
- **Hedging at p50 is a self-inflicted denial of service**, nearly doubling the work.
- The model cannot cancel a queued original; real tied requests (below) can.

23.7.3 Tied requests

Hedging waits before sending the second copy, so it helps only the slowest few percent. **Tied requests** send both copies (almost) at once, but each copy carries the identity of the other server, and whichever server *starts* executing the request first sends a cancellation to the other [Dean and Barroso 2013]. The request thus waits in the *shorter* of two

queues, which approximates a shared queue across servers (pooling, §4.10.4), and is executed only once in the common case.

Figure 23.9 — Tied requests with cross-server cancellation.



Reading the diagram. The client enqueues the request on two servers, delaying the second send by about one network message delay, and each copy names its twin. When A starts the request, it sends an asynchronous cancel to B, which drops its queued copy, so one server does the work and the client gets one response. If both dequeue before the cancel arrives, both execute (one duplicate). Unlike hedging, this needs server-side support for queue manipulation and peer messages.

Dean and Barroso report that in a BigTable benchmark, tied requests made the latency of a cluster running a concurrent large sort job nearly as good as that of an idle cluster without tied requests, at a disk-utilization cost below 1% [Dean and Barroso 2013]. They also report the hedged-request benchmark quoted in §4.8.4: hedging after 10 ms reduced the p99.9 of a 1,000-key read across 100 servers from 1,800 ms to 74 ms with 2% more requests.

Real-world industry applications

INDUSTRY EXAMPLE

Speculative execution in batch and storage systems. MapReduce schedules **backup tasks** for the last in-progress tasks of a job, taking whichever copy finishes first; the original paper reports a sort taking 44% longer with backup tasks disabled [Dean and Ghemawat 2004]. Apache Cassandra offers a per-table `speculative_retry` setting that sends a read to an extra replica when the first has not answered by a chosen percentile or fixed delay [Apache Cassandra Docs]. gRPC's retry design defines a `hedgingPolicy` with a maximum number of attempts and a hedging delay, and the server can throttle hedges with pushback [gRPC A6]. *Evidence: Primary* — [Dean and Ghemawat 2004], [Apache Cassandra Docs], [gRPC A6]

Failure scenarios

What goes wrong	Why	Detection	Recovery	Prevention
Overload spiral	Hedges add load as latency rises	Hedge rate climbs with latency	Disable hedging; shed load	Hedge budget (e.g., ≤ 5% of requests); adaptive delay
Duplicate side effects	Non-idempotent operation hedged	Duplicate records, double charges	Compensating actions	Hedge reads only, or use idempotency keys (§22.15)
No improvement	Replicas share the slow cause (same disk, same switch)	Hedge and primary equally slow	Choose replicas in other failure domains	Spread replicas across racks and zones
Hedges amplify retries	Hedging and retries both enabled at every layer	Request count ≫ user requests	Remove one layer's hedging	Hedge at one layer only, with a shared budget (§22.12)

COMMON MISTAKE

Mistake — Hedging with a fixed delay forever. A delay tuned to last quarter's p95 becomes the p60 after a regression, and the hedge rate jumps from 5% to 40%. Derive the delay from a recent latency histogram, cap the fraction of requests that may hedge, and turn hedging off when the service is overloaded.

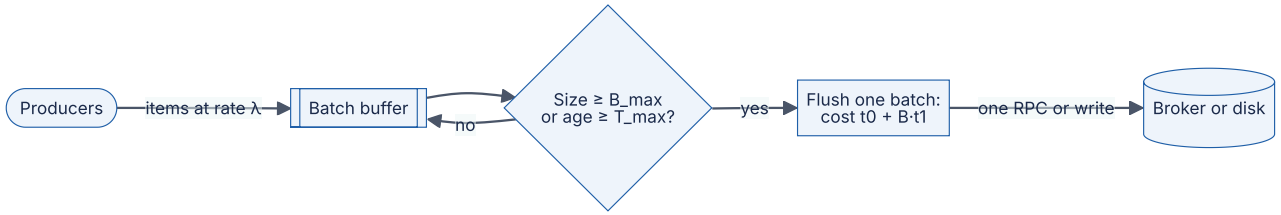
23.8 Batching, pipelining and compression

23.8.1 Batching with a size and a time limit

Section 4.18.6 modeled a batch of B items as costing $t_0 + Bt_1$: the fixed cost t_0 (a system call, a network round trip, a disk flush, a GPU kernel launch) is shared by all items. Real

batchers close a batch on whichever comes first, a **size limit** or a **time limit**, so that light traffic is not stuck waiting for a full batch. Kafka's producer `batch.size` and `linger.ms` settings are exactly this pair (§31.6).

Figure 23.10 — A batcher with size and time triggers.



Reading the diagram. Producers append asynchronously to an in-memory buffer, a failure point: its contents are lost on a crash unless callers wait for acknowledgment. Two triggers are checked on every append and by a timer; when either fires, the batch goes as one synchronous RPC or write to the sink (the storage boundary). Heavy load fills batches (maximum throughput); light load fires the timer, bounding added latency at $T_{\max}$. Scale with parallel batchers (per partition or core).

FORMULA

Batch size and latency with a time limit

$$B_{\text{eff}} \approx \min\left(B_{\max},\ \max\left(1, \lambda T_{\max}\right)\right) \qquad X_{\max} = \frac{B_{\text{eff}}}{t_0 + B_{\text{eff}}\, t_1} \qquad W_{\text{worst}} \approx T_{\max} + t_0 + B_{\text{eff}}\, t_1$$

Symbol	Meaning	Units
λ	Item arrival rate	items/second
$B_{\max}$, $T_{\max}$	Size and time limits of the batcher	items, seconds
B_{eff}	Typical batch size actually sent	items
t_0, t_1	Fixed cost per batch, marginal cost per item	seconds, seconds/item
$X_{\max}$	Throughput one sender can sustain	items/second
W_{worst}	Worst-case latency added by batching and sending	seconds

WORKED EXAMPLE

Worked example — A log shipper. Each send costs $t_0 = 0.5$ ms (round trip and system call) plus $t_1 = 10$ µs per record. Records arrive at $\lambda = 20,000$/s; one sender has one batch in flight at a time.

1. Unbatched ($B = 1$): $X_{\max} = 1/0.00051 = 1,961$/s. The sender falls 18,000 records/s behind.
2. With $T_{\max} = 5$ ms and $B_{\max} = 500$: $B_{\text{eff}} = \min(500, 20,000 \times 0.005) = 100$.
3. Batch time $= 0.5 + 100 \times 0.01 = 1.5$ ms, so $X_{\max} = 100/0.0015 = 66,667$/s: 3.3× headroom.
4. Worst-case added latency $\approx 5 + 1.5 = 6.5$ ms; the average record waits about half of $T_{\max}$, 2.5 ms.

Interpretation. A 5 ms latency budget buys a 34× throughput increase.
Engineering implication. Choose $T_{\max}$ from the latency budget, not from throughput; at high load the batch fills before the timer anyway.

The Nagle analogy. Nagle's algorithm (§7.7.8) is a batcher whose "timer" is the arrival of an acknowledgment: small writes are held while data is unacknowledged. It improves throughput for chatty senders, but combined with the receiver's delayed-ACK timer it adds tens of milliseconds to request/response traffic. The general lesson is that *two batchers in series can deadlock each other until a timer fires*. When you add batching to a client, check what the server does with small, incomplete messages.

23.8.2 Pipelining

Pipelining sends several requests on one connection without waiting for each response, overlapping network round trips with server work.

FORMULA

Sequential versus pipelined requests

$$T_{\text{seq}} = n(r + s) \qquad T_{\text{pipe}} \approx r + ns \quad (\text{window} \geq n)$$

Symbol	Meaning	Units
n	Number of requests	count
r	Network round-trip time	seconds
s	Server processing time per request	seconds

WORKED EXAMPLE

Worked example — 1,000 cache reads. $r = 0.5$ ms within a datacenter, $s = 5$ µs per GET in an in-memory store.

1. Sequential: $1,000 \times (0.5 + 0.005) = 505$ ms.
2. Pipelined: $0.5 + 1,000 \times 0.005 = 5.5$ ms, 92× faster.

Interpretation. For small operations the round trip, not the server, is the cost. **Engineering implication.** Use pipelining or multi-key commands (Redis pipelines, `MGET`) and multiplexed protocols such as HTTP/2 for many small requests. Strict in-order pipelining suffers head-of-line blocking: one slow response delays all later ones, which is why HTTP/1.1 pipelining failed and HTTP/2 multiplexes independent streams (§7.9).

23.8.3 Compression: ratio versus CPU

Compression trades CPU for bytes: it pays when bytes are the bottleneck (slow links, egress charges, storage, cache capacity).

Table 23.7 — Fast compressors on one CPU core (as of 2026; zstd project benchmark, lzbench on the Silesia corpus, Core i7-9700K).

Compressor and level	Ratio	Compression (MB/s)	Decompression (MB/s)
zstd 1.5.7, level 1	2.896	510	1,550
brotli 1.1.0, level 1	2.883	290	425
zlib 1.3.1, level 1 (gzip's DEFLATE)	2.743	105	390
lz4 1.10.0	2.101	675	3,850
snappy 1.2.1	2.089	520	1,500

Source: [Zstandard README]. Higher levels compress better and far more slowly; benchmark your own payloads.

FORMULA

When compression reduces transfer time (store-and-forward)

$$t_{\text{plain}} = \frac{S}{W} \qquad t_{\text{comp}} = \frac{S}{C_c} + \frac{S}{\rho W} + \frac{S}{C_d} \qquad \text{compress if } \frac{1}{C_c} + \frac{1}{\rho W} + \frac{1}{C_d} < \frac{1}{W}$$

Symbol	Meaning	Units
S	Uncompressed size	MB
W	Network (or disk) bandwidth	MB/s
ρ	Compression ratio (original / compressed)	dimensionless
C_c, C_d	Compression and decompression speed	MB/s

With streaming, stages overlap and time approaches the *slowest* stage rather than the sum.

WORKED EXAMPLE

Worked example — Shipping 100 MB. Use Table 23.7.

1. 1 Gb/s link ($W = 125$ MB/s): plain $= 0.80$ s. lz4: $0.148 + 100/262.6 + 0.026 = 0.148 + 0.381 + 0.026 = 0.555$ s. zstd-1: $0.196 + 100/362 + 0.065 = 0.196 + 0.276 + 0.065 = 0.537$ s. zlib-1: $0.952 + 0.292 + 0.256 = 1.50$ s.
2. 10 Gb/s link ($W = 1,250$ MB/s): plain $= 0.08$ s; lz4 $= 0.148 + 0.038 + 0.026 = 0.212$ s; zstd-1 $= 0.196 + 0.028 + 0.065 = 0.288$ s.

Interpretation. On a 1 Gb/s link, fast compressors cut transfer time by about a third and zlib nearly doubles it; on a 10 Gb/s datacenter link, compression *slows* a single-stream transfer by 2.6–3.6× (parallel compression across cores narrows the gap).
Engineering implication. Compress for slow or metered links, storage and cache capacity, and cross-region traffic (where egress is billed per byte); prefer zstd or lz4 over gzip for machine-to-machine traffic; compress batches rather than single small messages, since ratio improves with more context.

23.9 Garbage collection and runtime effects

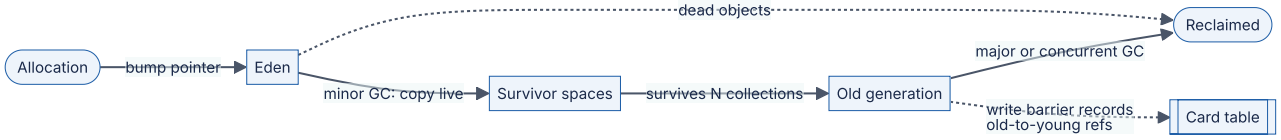
23.9.1 Generational garbage collection

DEFINITION

Definition — Garbage collection (GC). Automatic reclamation of heap memory that the program can no longer reach. A **tracing** collector finds live objects by following references from **roots** (stacks, globals, registers) and reclaims everything else. A **stop-the-world (STW) pause** is an interval in which all application threads (the **mutator**) are halted so the collector can work on a consistent heap [Jones et al. 2023].

A restaurant can clear tables while diners eat (concurrent collection) or close the room for ten minutes every hour (stop-the-world): simpler and cheaper overall, but everyone present waits. Most objects die young (request buffers, temporary strings), the **weak generational hypothesis**, which motivates **generational** collection [Ungar 1984]: allocate in a small **young generation** collected often by copying out the few survivors, and promote long-lived objects to an **old generation** collected rarely.

Figure 23.11 — Generational heap.



Reading the diagram. Allocation in Eden is a nearly free pointer bump. A minor GC copies live objects to a survivor space and reclaims the rest without touching it, so its cost scales with *survivors*, not garbage. Long-lived objects are promoted to the old generation. A **write barrier** on reference stores records old-to-young pointers in a card table (dotted bookkeeping) so minor GCs need not scan the old generation, which is collected by a major collection, stop-the-world or largely concurrent. Failure points: promotion faster than old-generation collection forces a long full STW collection; a heap too small for the live set causes back-to-back collections.

FORMULA

GC frequency and pause exposure

$$I_{\text{minor}} = \frac{E}{a} \qquad f_{\text{hit}} \approx \frac{p + w}{I_{\text{minor}}} \qquad \text{Go heap target} = H_{\text{live}} \times \left(1 + \frac{\text{GOGC}}{100}\right)$$

Symbol	Meaning	Units
E	Young-generation (Eden) size	MB
a	Allocation rate	MB/s
I_{minor}	Interval between minor collections	seconds
p	STW pause per minor collection	seconds
w	Request duration	seconds
f_{hit}	Approximate fraction of requests that overlap a pause	dimensionless
H_{live}	Live heap after the previous Go collection	MB
GOGC	Go's GC percentage setting (default 100)	percent

WORKED EXAMPLE

Worked example — GC in the tail. A JVM service allocates $a = 500$ MB/s with a 2 GB Eden; minor pauses are $p = 30$ ms. Its requests take 5 ms.

1. $I_{\text{minor}} = 2,000/500 = 4$ s between pauses.
2. Requests overlapping a pause: $f_{\text{hit}} \approx (30 + 5)/4,000 \approx 0.9\%$ (in flight when it starts, or arriving during it), each delayed by up to 30 ms.
3. So the service's p99 sits at the edge of the GC effect and its p99.9 includes most of a full pause; at 100-way fan-out, most root requests ($1 - 0.99125^{100} \approx 58\%$) see one.
4. Halving the allocation rate doubles the interval and halves the exposure. For a Go service with 1 GB live heap and GOGC = 100, the heap grows to $1 \times 2 = 2$ GB before the next cycle; GOGC = 50 caps it at 1.5 GB but runs GC twice as often.

Interpretation. GC pauses are a *rate* problem (how often) times a *size* problem (how long). **Engineering implication.** Reduce allocation first (it lowers both CPU and pause frequency), then choose a collector whose pause behavior fits the SLO.

23.9.2 Collectors in practice

Table 23.8 — Production collectors (as of 2026).

Collector	Runtime	Approach	Pause behavior	Choose when
Parallel	JVM	Generational, STW, multi-threaded	Long pauses, best throughput	Batch jobs where latency is irrelevant
G1	JVM (default since JDK 9)	Region-based, generational, mostly concurrent marking, STW evacuation	Pause-time *goal* (default 200 ms), typically tens of ms	General-purpose services
ZGC	JVM (production since JDK 15; generational since JDK 21)	Concurrent marking and relocation with load barriers	Sub-millisecond pauses independent of heap size	Large heaps with strict tail SLOs
Shenandoah	JVM (production since JDK 15, in some builds)	Concurrent compaction with barriers	Short pauses independent of heap size	Similar to ZGC
Go GC	Go	Concurrent, non-generational, non-moving mark-sweep	Short STW phases; *GC assist* slows allocating goroutines	Default; tune GOGC and GOMEMLIMIT
CPython	Python	Reference counting plus a cycle collector	Immediate frees; occasional cycle-collection pauses	Default

Concurrent collectors trade throughput and memory for short pauses: they add barriers to reference loads or stores, consume CPU for collector threads (Go's targets about 25% of available CPU during marking) and need heap headroom. Go 1.19 added `GOMEMLIMIT`, a soft limit that makes the collector run more often near it, which suits containers [Go GC Guide].

23.9.3 JIT warm-up and other runtime effects

A **just-in-time (JIT)** compiler (JVM, .NET, JavaScript engines) interprets code first, profiles it and compiles hot methods with increasing optimization, so a fresh instance runs several times slower for seconds to minutes. Every deployment and autoscaling event therefore causes a p99 spike, and tests without warm-up misstate steady-state performance. Mitigations: replay warm-up traffic before marking an instance ready, ramp traffic with load-balancer **slow start** (Chapter 19), or use ahead-of-time compilation and class-data sharing.

RECOMMENDATION

GC and runtime tuning heuristics.

- Measure before tuning: enable GC logging (`-Xlog:gc*` on the JVM, `GODEBUG=gctrace=1` in Go) and plot pause durations and frequency against latency percentiles.
- Reduce allocation rate with an allocation profiler (§23.4.5): reuse buffers, avoid boxing, stream instead of materializing.
- Give the collector headroom: a heap of a few times the live set is a common starting point; a nearly full heap means frequent collections.
- Pick the collector by goal: throughput (Parallel), balanced (G1), tail latency on large heaps (ZGC or Shenandoah).
- In containers, set the heap from the container limit (for example `-XX:MaxRAMPercentage`, or `GOMEMLIMIT` below the cgroup limit) so the runtime is not killed before it collects.

INDUSTRY EXAMPLE

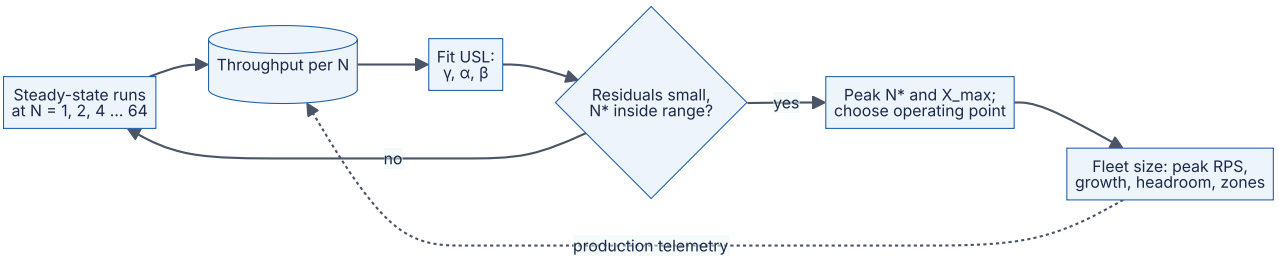
Discord — GC spikes in a Go service. Discord described a Go service holding a large in-memory cache that showed latency spikes about every two minutes. The cause was Go's forced garbage collection, which runs at least every two minutes even when little garbage exists; each cycle had to scan the large cache. Tuning reduced but did not remove the effect, and the team rewrote the service in Rust, which has no garbage collector, eliminating the spikes. Trade-off: a rewrite removed the runtime cause at significant engineering cost; reducing the number of heap pointers to scan is the usual cheaper alternative. *Evidence: Primary* — [Howarth 2020]

23.10 Capacity models from measurements

23.10.1 From benchmark to fleet

Section 4.13 introduced the Universal Scalability Law (USL) with known coefficients. In practice you *measure* throughput at several concurrency levels, *fit* the coefficients, and use the fitted model to predict the peak, choose an operating point and size the fleet [Gunther 2007].

Figure 23.12 — Capacity modeling workflow.



Reading the diagram. Each run is a warm, steady-state closed-loop test at fixed concurrency N (closed is right here, because concurrency is the independent variable); the throughputs are the stored data. If residuals are large or the fitted peak N^* lies outside the measured range, measure more points, because extrapolated peaks are unreliable. Operating point and fleet size follow by arithmetic, and production telemetry (dotted, asynchronous) refines the model.

FORMULA

USL for measured throughput, and its linearized form

$$X(N) = \frac{\gamma N}{1 + \alpha(N-1) + \beta N(N-1)} \qquad \frac{\gamma N}{X(N)} - 1 = \alpha(N-1) + \beta N(N-1)$$

Symbol	Meaning	Units
N	Concurrency (client threads, cores or nodes)	count
$X(N)$	Measured throughput at concurrency N	requests/second
γ	Throughput at $N = 1$ (the slope of ideal linear scaling)	requests/second
α, β	Contention and coherency coefficients (§4.13)	dimensionless

The right-hand form is linear in α and β, so with γ fixed at the measured $X(1)$ an ordinary least-squares fit gives a starting estimate; a nonlinear fit of all three parameters to the raw throughputs then refines it.

23.10.2 Fitting the USL in Python

The program fits benchmark data in two steps using only the standard library. The data are synthetic, generated from known coefficients ($\gamma = 950$, $\alpha = 0.03$, $\beta = 0.0004$) with 3% measurement noise, so you can see how well the fit recovers them.

```
        import math
        import random

        def usl(n, gamma, alpha, beta):
            """USL throughput: gamma = throughput at N = 1."""
            return gamma * n / (1 + alpha * (n - 1) + beta * n * (n - 1))

        def solve(a, b):
            """Solve the small linear system a x = b by Gaussian elimination."""
            n = len(b)
            m = [row[:] + [b[i]] for i, row in enumerate(a)]
            for c in range(n):
                p = max(range(c, n), key=lambda r: abs(m[r][c]))
                m[c], m[p] = m[p], m[c]
                for r in range(c + 1, n):
                    f = m[r][c] / m[c][c]
                    for k in range(c, n + 1):
                        m[r][k] -= f * m[c][k]
            x = [0.0] * n
            for r in range(n - 1, -1, -1):
                x[r] = (m[r][n] - sum(m[r][k] * x[k] for k in range(r + 1, n))) / m[r]
[r]
            return x

        def fit_linearized(ns, xs):
            """Step 1: N*gamma/X - 1 = alpha (N-1) + beta N (N-1), with gamma = X(1)."""
            gamma = xs[ns.index(1)]
            rows = [(n - 1, n * (n - 1)) for n in ns]
            ys = [n * gamma / x - 1 for n, x in zip(ns, xs)]
            ata = [[sum(r[i] * r[j] for r in rows) for j in range(2)] for i in range(2)]
            aty = [sum(r[i] * y for r, y in zip(rows, ys)) for i in range(2)]
            alpha, beta = solve(ata, aty)
            return gamma, alpha, beta

        def fit_nonlinear(ns, xs, p, iters=50):
            """Step 2: Gauss-Newton refinement of (gamma, alpha, beta) on throughput."""
            p = list(p)
            for _ in range(iters):
                r = [x - usl(n, *p) for n, x in zip(ns, xs)]
                jac = []
                for n in ns:
                    row = []
                    for k in range(3):
                        h = 1e-6 * max(abs(p[k]), 1e-6)
                        q = p[:]; q[k] += h
                        row.append((usl(n, *q) - usl(n, *p)) / h)
                    jac.append(row)
                jtj = [[sum(j[a] * j[b] for j in jac) for b in range(3)] for a in
range(3)]
                jtr = [sum(j[a] * ri for j, ri in zip(jac, r)) for a in range(3)]
                step = solve(jtj, jtr)
                p = [pi + si for pi, si in zip(p, step)]
                if max(abs(s / pi) for s, pi in zip(step, p)) < 1e-10:
                    break
            return p
```

```
        # "Measured" throughput (req/s) at each client concurrency N. Synthetic:
        # generated from gamma=950, alpha=0.03, beta=0.0004 with 3% noise.
        rng = random.Random(11)
        ns = [1, 2, 4, 8, 12, 16, 24, 32, 48, 64]
        xs = [round(usl(n, 950, 0.03, 0.0004) * rng.gauss(1, 0.03)) for n in ns]

        g, a, b = fit_linearized(ns, xs)
        print(f"linearized fit : gamma={g:7.1f}  alpha={a:.4f}  beta={b:.6f}")
        g, a, b = fit_nonlinear(ns, xs, (g, a, b))
        print(f"nonlinear fit  : gamma={g:7.1f}  alpha={a:.4f}  beta={b:.6f}")

        mean = sum(xs) / len(xs)
        ss_res = sum((x - usl(n, g, a, b)) ** 2 for n, x in zip(ns, xs))
        ss_tot = sum((x - mean) ** 2 for x in xs)
        n_star = math.sqrt((1 - a) / b)
        x_max = usl(n_star, g, a, b)
        print(f"R^2 = {1 - ss_res / ss_tot:.4f}   N* = {n_star:.1f}   X(N*) =
{x_max:,.0f} req/s")

        print(f"{'N':>4}{'measured':>10}{'model':>9}{'R=N/X ms':>10}{'efficiency':>12}")
        for n in ns + [96, 128]:
            model = usl(n, g, a, b)
            meas = f"{xs[ns.index(n)]:>10,}" if n in ns else f"{'-':>10}"
            print(f"{n:>4}{meas}{model:>9,.0f}{1000 * n / model:>10.2f}{model / (n *
g):>12.0%}")
```

Output:

```
linearized fit : gamma=  915.0  alpha=0.0238  beta=0.000502
nonlinear fit  : gamma=  885.4  alpha=0.0205  beta=0.000522
R^2 = 0.9983   N* = 43.3   X(N*) = 13,588 req/s
   N  measured    model  R=N/X ms  efficiency
   1       915      885      1.13        100%
   2     1,864    1,734      1.15         98%
   4     3,575    3,317      1.21         94%
   8     6,072    6,041      1.32         85%
  12     7,915    8,211      1.46         77%
  16     9,812    9,891      1.62         70%
  24    12,103   12,082      1.99         57%
  32    13,496   13,165      2.43         46%
  48    13,264   13,539      3.55         32%
  64    12,979   12,899      4.96         23%
  96         -   11,035      8.70         13%
 128         -    9,381     13.64          8%
```

What to notice:

- **The fit is excellent ($R^2 = 0.998$) yet the coefficients are off.** The true peak is at $N^* = \sqrt{0.97/0.0004} = 49$ with 13,770 req/s; the fit says 43 with 13,588 req/s. Throughput near the peak is flat, so the *location* of the peak is poorly determined while its *height* is well determined. Plan on throughput, not on the exact N^*.

- **The response-time column is Little's Law** for a closed test with no think time, $R = N/X$ (§23.5.2): latency rises more than fourfold from $N = 24$ to $N = 96$ while throughput falls.
- **Efficiency** $X(N)/(N\gamma)$ reaches 70% at $N = 16$ and 57% at 24: the knee, beyond which concurrency buys little throughput for much latency.
- Rows 96 and 128 are extrapolations; measure before relying on them.

23.10.3 Converting a model into a fleet size

FORMULA

Fleet size from per-instance capacity

$$n = \left\lceil \frac{\lambda_{\text{peak}}\,(1+g)}{u \cdot X_{\max}} \right\rceil \qquad n_{\text{zones}} = Z \cdot \left\lceil \frac{n}{Z-1} \right\rceil$$

Symbol	Meaning	Units
λ_{peak}	Forecast peak request rate	requests/second
g	Growth allowance over the planning horizon	dimensionless
$X_{\max}$	Fitted peak throughput of one instance	requests/second
u	Target fraction of $X_{\max}$ at peak (headroom for queueing and variance)	dimensionless
Z	Number of availability zones, sized to survive the loss of one	count

This is the provisioning formula of §18.11.2 with $c\,u_{target}$ replaced by the fitted $u \cdot X_{\max}$, the growth term written as one allowance g for the horizon, and the zone term rounded per zone so that every zone runs the same count (it can therefore exceed §18.11.2's $\lceil N \cdot Z/(Z-1) \rceil$ by up to $Z-2$ instances: 33 rather than 32 in the example below).

WORKED EXAMPLE

Worked example — Sizing from the fitted model. Forecast peak 150,000 RPS, growth allowance 30%, $X_{\max} = 13,588$ req/s per instance from the fit, target $u = 0.7$, three zones.

1. Per-instance planning capacity: $0.7 \times 13,588 = 9,512$ req/s. The fitted curve reaches this at $N \approx 16$, where $R \approx 1.6$ ms and efficiency is still 70%.
2. Demand with growth: $150,000 \times 1.3 = 195,000$ RPS.
3. $n = \lceil 195,000/9,512 \rceil = \lceil 20.5 \rceil = 21$ instances.
4. Survive a zone loss: each of the two remaining zones must carry the load, so $\lceil 21/2 \rceil = 11$ per zone, $3 \times 11 = 33$ instances.

Interpretation. Zone redundancy (33 vs 21) costs more than growth and headroom combined. **Engineering implication.** Re-run the benchmark and refit whenever code or instance type changes; the capacity estimate from Chapter 2 and Chapter 89 becomes a *measured* number only through this loop (§18.11 covers the planning process).

COMMON MISTAKE

Mistake — Planning to the peak of the curve. $X_{\max}$ is reached in a closed test at concurrency N^*, where latency is already several times its unloaded value; in production, arrivals are open and bursty, so running an instance near $X_{\max}$ puts it on the steep part of the queueing curve (§4.10.3). Plan to a fraction of the peak and verify the latency at that point with an open-model load test.

23.11 Summary and revision notes

SUMMARY

- **Method beats intuition.** USE (utilization, saturation, errors per resource) finds saturated resources, including locks and pools; RED and the four golden signals (rate, errors, duration, plus saturation) find the slow service. Scope the symptom, check what changed, then profile.
- **Profiling.** Sampling profilers have overhead fixed by frequency and error $\sqrt{p(1-p)/n}$; instrumenting profilers give exact counts but distort hot paths. Flame-graph width is sample share; look for wide plateaus. CPU profiles miss off-CPU time (I/O, locks, run-queue waits), which usually dominates latency problems.
- **Load testing.** Closed generators reduce load when the system slows and hide outages; use open (constant-arrival-rate) generators for SLO tests. Coordinated omission can understate p99 by orders of magnitude: measure from the intended send time. Record latency in HDR histograms and merge, never average, percentiles.
- **Tail at scale.** For N-way fan-out, root percentiles are leaf percentiles at $q^{1/N}$; the leaf tail becomes the user median. Variability comes from GC, queueing, background work, noisy neighbors, CPU throttling and packet loss.
- **Hedging.** Hedge idempotent requests after about the p95 for about 5% extra load; tied requests with cancellation do better. Without budgets and cancellation, hedging can collapse a loaded system.
- **Batching, pipelining, compression.** Bound batch latency with a timer; pipeline small requests; compress when bytes, not CPU, are the bottleneck.
- **Runtimes.** GC exposure is pause frequency (allocation rate / young size) times pause length; reduce allocation first, then choose a collector for the goal. Warm up JIT runtimes before they take traffic.
- **Capacity.** Fit the USL to measured throughput, plan to a fraction of the fitted peak, then add growth and zone redundancy.

23.12 Knowledge checkpoint

1. What are the three USE questions, and why are software resources part of the resource list?
2. How do the four golden signals differ from RED, and why should successful and failed request latencies be tracked separately?
3. A 20 s profile at 49 Hz on 4 busy CPUs shows a function in 5% of samples. What is the standard error?
4. Why can a CPU flame graph show nothing useful for a service whose p99 is 2 s?

5. State the interactive response time law and explain why a closed load test understates an overload.
6. Define coordinated omission and give two ways to avoid or correct it.
7. Why must percentiles from several load generators be computed from merged histograms?
8. Leaves have exponential latency with a 5 ms mean. What is the root's median with 50-way fan-out?
9. A container has a 1-CPU quota per 100 ms period and 4 runnable threads. How long is it throttled each period?
10. A replica answers in 5 ms with probability 0.97 and 300 ms otherwise. Hedging after 6 ms, what fraction of requests exceeds 11 ms, and what is the extra load?
11. Why did hedging without cancellation collapse at 80% utilization in the simulation of §23.7.2?
12. When does compression increase transfer time?
13. Why is the location of the USL peak N^* less reliable than its height?

Answers

1. Utilization, saturation, errors. Locks, pools and descriptor limits are finite and can saturate while hardware is idle, so omitting them hides real bottlenecks.
2. The golden signals add saturation, a leading indicator. Fast failures pull the latency distribution down, hiding user pain.
3. $n = 49 \times 20 \times 4 = 3,920$; $\mathrm{SE} = \sqrt{0.05 \times 0.95 / 3,920} = 0.0035$, i.e. ± 0.35 percentage points.
4. The time is off-CPU: waiting on I/O, locks, pools or the run queue. Use off-CPU or wall-clock profiling and traces.
5. $X = N/(R + Z)$. When R rises, a fixed population sends less, so offered load falls instead of queueing.
6. The generator waits for slow replies and so skips sends during stalls, omitting the worst samples. Use an open-model generator that measures from the intended send time (wrk2, k6 arrival-rate executors), or apply expected-interval correction.
7. Percentiles are not additive; averaging p99s from generators with different sample counts or distributions gives a number that is not the p99 of anything.
8. $0.5^{1/50} = 0.98623$; $t = 5 \ln(1/0.01377) = 5 \times 4.285 = 21.4$ ms.
9. $t_{\mathrm{run}} = 100/4 = 25$ ms, so it is throttled for 75 ms per period.
10. $P(T > 11) = 0.03 \times 0.03 = 0.0009$ (0.09%); extra load is $P(X > 6) = 3\%$.
11. Hedges raised utilization, which raised latency, which pushed more requests past the fixed hedge delay, a positive feedback loop that drove demand above capacity.
12. When CPU time to compress and decompress exceeds the bandwidth time saved: $1/C_c + 1/(\rho W) + 1/C_d > 1/W$, typically on fast links with a single stream.
13. Throughput is flat near the peak, so many values of N give nearly the same X; small noise moves N^* a lot but $X_{\max}$ little.

23.13 Interview questions

Basic

Q1. A service's average latency is fine but users complain. Where do you start? Answer. Check per-endpoint percentiles (p99, p99.9) from histograms, split by success and error, and what changed when complaints began; then RED across the call graph and USE on the slow service. Averages hide tails, and with fan-out the unlucky fraction is often most users.

Q2. What is coordinated omission? Answer. A load generator that waits for each response before sending the next stops sending during a stall, so the requests that would have suffered are never measured. Reported percentiles can be orders of magnitude too low. Use a constant-rate open generator that timestamps from the intended send time, and record into HDR histograms.

Intermediate

Q3. Design hedging for a key-value read path with a p99 problem. Answer. Hedge only idempotent reads. Set the delay from a rolling histogram near the p95, cap hedges at about 5% of requests with a token-bucket budget, choose the second replica in a different failure domain, and cancel the loser. Disable hedging when the service is overloaded (rising queue depth or error rate) to avoid feedback. Measure p99, p99.9 and hedge rate before and after; consider tied requests if the storage servers can cancel queued work.

Q4. CPU is at 30% but p99 is ten times the SLO. What could it be? Answer. Off-CPU causes: saturated pools, lock contention, slow dependencies, GC pauses, CPU quota throttling or retransmissions. Check pool waiters, cgroup `nr_throttled`, GC logs and retransmit counters, then take off-CPU profiles and traces of slow requests.

Advanced

Q5. How would you produce a capacity plan for a new service? Answer. Build a representative workload from production traces; run warm, steady-state closed tests at increasing concurrency on the target instance type; fit the USL; check residuals and that the peak lies within the measured range. Choose an operating point at a fraction of the fitted peak, verified by an open-model test at that rate against the latency SLO. Size $n = \lceil \lambda_{\text{peak}}(1+g)/(uX_{\max}) \rceil$, add zone redundancy, and refit after every major change, comparing with production telemetry.

Q6. A Java service's p99.9 spikes every few seconds. Walk through diagnosis and fixes. Answer. Correlate spikes with GC logs. If they match minor pauses, measure the allocation rate and young-generation size: frequency is E/a, and pause length scales with

survivors. Reduce allocation using an allocation profiler, size the heap with headroom, or move to a concurrent collector (ZGC or Shenandoah) for sub-millisecond pauses at some throughput and memory cost. Also rule out JIT warm-up after deployments, container throttling and background jobs.

23.14 Practical exercises

1. Run the coordinated-omission program of §23.5.3 with the stall moved to $t = 5$ s and lengthened to 5 s. Predict the true p99 with the formula of §23.5.3 before running.
2. Modify the hedging simulation to use an adaptive delay (the running p95 of the last 10,000 requests) and a budget capping hedges at 5% of requests. Does it remain stable at 80% and 90% utilization?
3. Profile a Python program of your own with `py-spy record` and with `cProfile`. Compare the hottest functions and explain any differences.
4. Use k6 with a `constant-vus` executor and then a `constant-arrival-rate` executor against a test server that sleeps 1 s every 10 s. Compare reported p99 and achieved rates.
5. Collect throughput at eight concurrency levels from any local service (for example, a database with a benchmark tool) and fit the USL with the program of §23.10.2.

23.15 Design challenges

1. **Search fan-out.** A query fans out to 200 index partitions with leaf p99 = 20 ms and p99.9 = 120 ms. The product requires root p99 ≤ 80 ms. Choose a combination of replication, hedging, tied requests and partial results, and quantify the extra load.
2. **Load-test platform.** Design an internal load-testing service for 100 teams: open-model generators, HDR histogram aggregation, safety limits for tests against shared dependencies, and automatic comparison with the previous release.

23.16 References

1. Apache Cassandra. *Apache Cassandra Documentation — CREATE TABLE: `speculative_retry` option*. https://cassandra.apache.org/doc/latest/
2. Beyer, B., Jones, C., Petoff, J., Murphy, N. R. (eds.). *Site Reliability Engineering: How Google Runs Production Systems*, ch. 6 “Monitoring Distributed Systems.” O’Reilly Media, 2016. https://sre.google/sre-book/monitoring-distributed-systems/
3. Dean, J., Barroso, L. A. “The Tail at Scale.” *Communications of the ACM* 56(2):74–80, 2013.
4. Dean, J., Ghemawat, S. “MapReduce: Simplified Data Processing on Large Clusters.” *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*,

2004.
5. The Go Authors. *A Guide to the Go Garbage Collector*. https://go.dev/doc/gc-guide
6. Gregg, B. “The Flame Graph.” *Communications of the ACM* 59(6):48–57, 2016.
7. Gregg, B. *Systems Performance: Enterprise and the Cloud*, 2nd ed. Addison-Wesley, 2020.
8. Gregg, B. *The USE Method*. brendangregg.com. https://www.brendangregg.com/usemethod.html
9. Gregg, B., Spier, M. “Java in Flames.” Netflix Technology Blog, 2015.
10. gRPC Authors. *gRFC A6: gRPC Retry Design* (includes hedging policy). https://github.com/grpc/proposal/blob/master/A6-client-retries.md
11. Gunther, N. J. *Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable Applications and Services*. Springer, 2007.
12. Howarth, J. “Why Discord Is Switching from Go to Rust.” Discord Blog, 2020. https://discord.com/blog/why-discord-is-switching-from-go-to-rust
13. Jones, R., Hosking, A., Moss, E. *The Garbage Collection Handbook: The Art of Automatic Memory Management*, 2nd ed. CRC Press, 2023.
14. Lazowska, E. D., Zahorjan, J., Graham, G. S., Sevcik, K. C. *Quantitative System Performance: Computer System Analysis Using Queueing Network Models*. Prentice Hall, 1984.
15. Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P. F. “Evaluating the Accuracy of Java Profilers.” *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2010.
16. Ren, G., Tune, E., Moseley, T., Shi, Y., Rus, S., Hundt, R. “Google-Wide Profiling: A Continuous Profiling Infrastructure for Data Centers.” *IEEE Micro* 30(4):65–79, 2010.
17. Schroeder, B., Wierman, A., Harchol-Balter, M. “Open Versus Closed: A Cautionary Tale.” *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2006.
18. Tene, G. “How NOT to Measure Latency.” Conference talk, presented at several venues (e.g., Strange Loop 2015).
19. Tene, G. *HdrHistogram: A High Dynamic Range Histogram*. http://hdrhistogram.org/
20. Ungar, D. “Generation Scavenging: A Non-disruptive High Performance Storage Reclamation Algorithm.” *ACM SIGSOFT/SIGPLAN Software Engineering Symposium on Practical Software Development Environments*, 1984.
21. Veeraraghavan, K., et al. “Kraken: Leveraging Live Traffic Tests to Identify and Resolve Resource Utilization Bottlenecks in Large Scale Web Services.” *USENIX OSDI*, 2016.
22. Wilkie, T. “The RED Method: How to Instrument Your Services.” Talk and Grafana Labs blog post, 2018.
23. Meta Platforms. *Zstandard (zstd) README — benchmark table*. https://github.com/facebook/zstd (accessed 2026).

23.17 Open verification items

- RED method: confirm Wilkie’s original venue and date (Weaveworks-era talk, reportedly 2015; Grafana Labs blog 2018) and the exact title.
- Dean and Barroso (2013): confirm that the slowest 5% of leaves account for about half of root p99; the tied-request result (sort-job cluster near idle-cluster latency, disk overhead below 1%); and the hedging delay wording.
- Gregg and Spier “Java in Flames”: confirm authorship, date and URL.
- Kraken (OSDI 2016): confirm the description of traffic shifting and safety stops; no numeric claims are made.
- Discord (2020): confirm the two-minute forced-GC explanation.
- Table 23.7: values copied from the zstd README as read in 2026; re-check before publication.
- JVM collector release versions (G1 default JDK 9; ZGC and Shenandoah production JDK 15; generational ZGC JDK 21) and Go’s 25% marking CPU target.
- Tene talk: confirm a canonical venue and year.

Part V — Distributed Systems

Chapter 24 — Distributed Systems Fundamentals

> **Level:** 2–3 · **Track:** A (Traditional) · **Estimated reading time:** 3.5 h

Replication (Chapter 14), caching (Chapter 20) and load balancing (Chapter 19) already showed that work spread across machines behaves differently from work on one computer: replicas lag, caches go stale, failovers can leave two primaries. This chapter names the reasons. Once machines talk only by messages over a network, you lose **a consistent view of what has failed**, **a shared clock** and **guaranteed delivery**. Consistency models (Chapter 25), consensus (Chapter 26), replication and failure detection (Chapter 27) and distributed transactions (Chapter 28) all respond to those losses. This chapter supplies the vocabulary: system models, partitions and partial failures, physical and logical clocks, the FLP result and the end-to-end argument.

The running example is PhotoShare (Chapter 1), now deployed in two regions, `us-east` and `eu-west`. Its examples are *illustrative*, not a description of any real company's system.

24.1 Learning objectives

After this chapter you should be able to:

- Define a distributed system by its three essential properties and explain the two generals problem.
- Justify distribution by scale, fault tolerance, latency or data residency, and quantify replication's availability gain.
- Give a concrete failure caused by each of the eight fallacies of distributed computing.
- State a protocol's timing and failure model and compute the replicas each fault model needs.
- Distinguish complete, partial and asymmetric partitions, citing measured failure evidence.
- Explain partial and gray failures, and set a timeout from latency data.
- Calculate clock drift and NTP offset, delay and error bound; explain PTP and leap smearing; show why wall-clock last-writer-wins loses data.
- Implement and choose among Lamport clocks, vector clocks, version vectors and hybrid logical clocks.

- Explain what FLP forbids and how systems escape it, and apply the end-to-end argument.

24.2 Prerequisites

- Network latency components and the speed-of-light floor — §7.15 (message delay is never zero and is variable).
- TCP acknowledgments and retransmission — §7.7.5 (what a transport-level ACK does and does not promise).
- Primary–replica, synchronous and asynchronous replication, and multi-primary conflicts — §14.3–§14.5 (the replication settings in which clocks and partitions matter).
- Timeouts, retries and idempotency keys — §22.11–§22.15 (the request-level tools this chapter justifies).
- Probability of independent events and availability arithmetic — §4.6 and §4.14 (replication and fleet-failure calculations).
- Units: time in seconds (s), milliseconds (ms, 10^{-3} s), microseconds (µs, 10^{-6} s) and nanoseconds (ns, 10^{-9} s); drift in parts per million (ppm, 10^{-6}); storage in decimal units (1 KB = 10^{3} bytes, 1 GB = 10^{9} bytes).

24.3 What is a distributed system?

24.3.1 Definition and the three defining properties

DEFINITION

Definition — Distributed system. A set of independent computers (**nodes**) that have no shared memory and no shared clock, communicate only by exchanging messages over a network, and cooperate to provide a service. Van Steen and Tanenbaum describe it from the user's side as "a collection of autonomous computing elements that appears to its users as a single coherent system" [van Steen and Tanenbaum 2017].

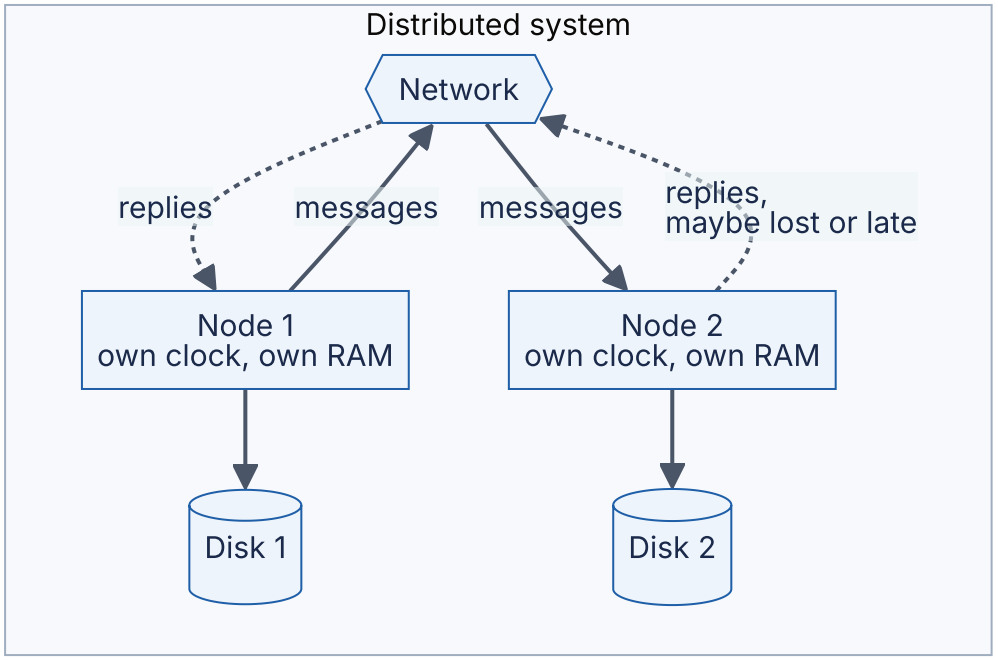
Leslie Lamport's definition is the one engineers remember. In an email of 28 May 1987 he wrote: "A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable" [Lamport 1987]. The joke is a precise observation: in a distributed system, your work depends on machines you cannot see, and their failures reach you in forms you did not expect.

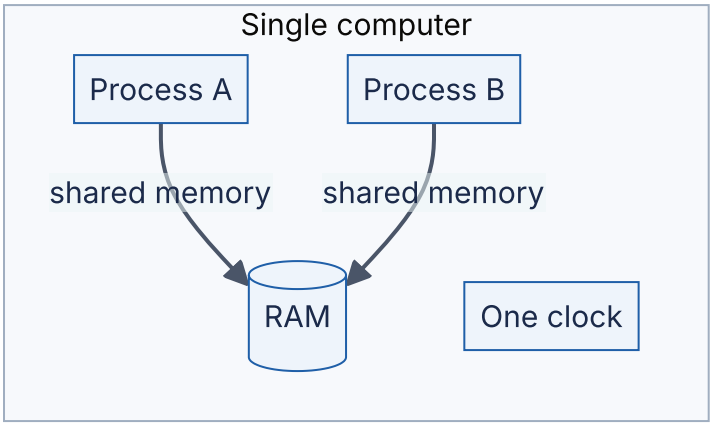
First-principles explanation

Imagine a restaurant chain whose managers communicate only by post: letters arrive late or never, no two branch clocks agree, and a manager who quits is noticed only when letters stop, which looks exactly like slow post. That chain has the three defining properties:

1. **Independent (partial) failure.** Some nodes fail while others run, and the survivors do not know exactly which failed.
2. **No global clock.** Clocks disagree, so the order of events on different nodes is not directly observable.
3. **Unreliable, asynchronous communication.** Messages may be delayed without bound, lost, duplicated or reordered, and are out of date when they arrive.

Figure 24.1 — One computer versus a distributed system.





Reading the diagram. On the left, two processes share one memory and one clock: a write is visible to the other process within nanoseconds, and if the machine fails both processes stop together. On the right, each node has its own memory, disk (a separate storage boundary) and clock; the only connection is the network. Requests travel synchronously in the sense that a caller waits, but the dotted reply paths show what the caller cannot control: a reply may arrive late or never. Failure points are each node, each disk and the network itself, and they fail independently. Scaling comes from adding nodes, at the price of more independent failure points.

24.3.2 Why this is hard: uncertainty about other nodes

Together the properties mean **a node can never be certain of another node's current state.** Silence from node 2 could mean a lost request, a slow node, a crash before or after processing, a lost reply or a reply still in flight; §24.8.1 draws all six. Every distributed algorithm is a strategy for acting correctly despite this ambiguity.

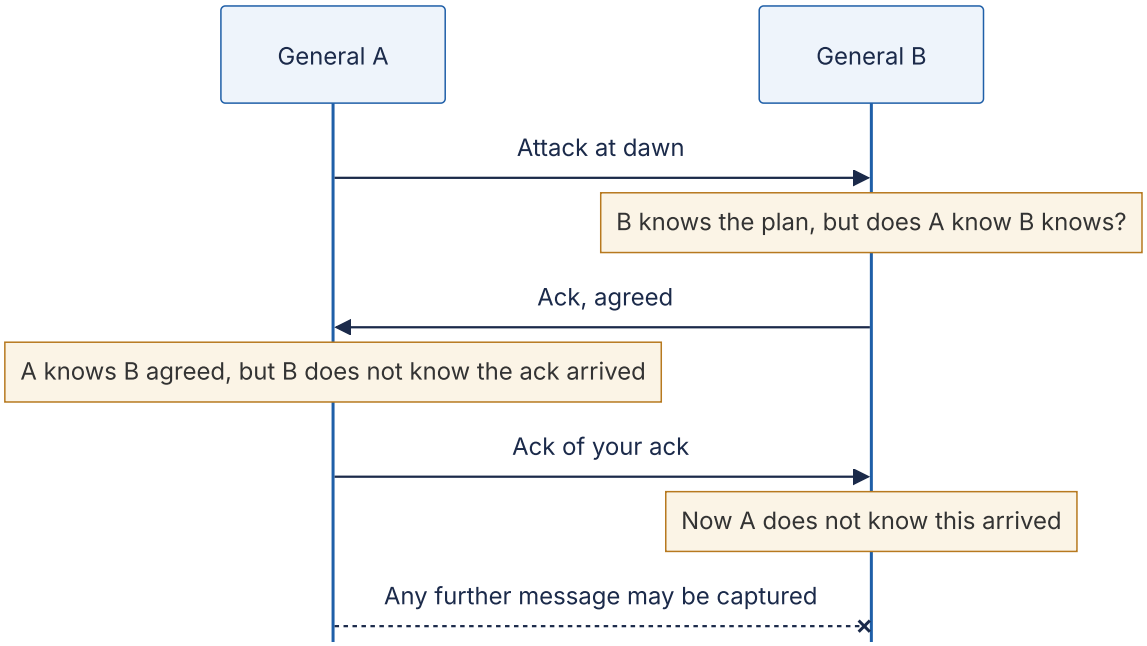
24.3.3 The two generals problem

The oldest illustration of this ambiguity is the **two generals problem**, first described by Akkoyunlu, Ekanadham and Huber [Akkoyunlu et al. 1975] and named by Jim Gray [Gray 1978].

DEFINITION

Definition — Two generals problem. Two armies camped on opposite hills must attack a city in the valley at the same time; either attacking alone is defeated. Their generals can communicate only by messengers who cross the valley and may be captured. The problem is to design a messaging protocol after which both generals are *certain* that they will attack together. No such protocol with a finite number of messages exists.

Figure 24.2 — The two generals problem: every acknowledgment needs another acknowledgment.



Reading the diagram. Solid arrows are messengers that arrived; the final arrow with a cross is a messenger captured in the valley. After each delivered message, the *sender* of the latest message is left unsure whether it arrived, so the other general's knowledge is always one level short of certainty. The failure point is the unreliable channel itself; adding more messengers raises the probability of agreement but never makes it certain, so this "protocol" does not scale toward certainty at any message count.

Why no finite protocol works

The proof is a short argument by contradiction. Suppose some protocol guarantees coordinated attack, and take the shortest such protocol, with k messages. Consider the last message. Its sender must decide to attack whether or not it arrives, because the sender cannot observe its loss. So the receiver must also decide to attack whether or not it arrives, since the protocol guarantees agreement. Then the last message affects no decision and can be removed, giving a working protocol with $k - 1$ messages. That contradicts "shortest", so no finite protocol exists.

What engineers do instead

Practical systems settle for less than certainty: **retry idempotent actions** until acknowledged (TCP, idempotency keys in §22.15); **accept high probability** (five messengers each captured with probability 0.1 all fail with probability 10^{-5}); or **persist state and allow blocking**, as two-phase commit does (§28.4).

COMMON MISTAKE

Mistake — "TCP solves the two generals problem." TCP makes delivery *reliable while the connection works,* by retransmitting. It cannot tell the sender whether its final bytes were processed by the application before the peer crashed or the connection broke (§7.7). The connection-close handshake itself has the two generals shape, which is one reason for the `TIME_WAIT` state.

24.4 Why distributed systems exist

24.4.1 Four reasons to pay the price

If distribution is this hard, why do it? Because a single computer cannot meet some requirements at all. Table 24.1 lists the four main reasons.

Table 24.1 — Why systems are distributed.

Reason	The limit of one machine	Distributed answer	What it costs
Scale	Even the largest single servers top out at hundreds of cores and a few tens of terabytes of memory (as of 2026); one disk array holds finite data	Partition data and work across nodes (§14.6, Ch 18)	Cross-node queries, rebalancing, coordination
Fault tolerance	One machine, power supply or datacenter is a single point of failure	Replicate across nodes, racks, zones and regions (Ch 14, Ch 27)	Keeping replicas consistent; failover
Latency and geography	The speed of light: New York–London RTT is at least about 56 ms over fiber (§7.15.1)	Place replicas and computation near users (Ch 21, Ch 44)	Cross-region consistency and conflicts
Data residency	Law may require that personal data stay within a jurisdiction	Region-pinned partitions and processing	Fragmented operations; per-region capacity

Some problems are also **inherently distributed** (banks in a payment network, offline edits on phones), and organizations split systems so teams deploy independently (Chapter 34). On residency, be precise: the EU GDPR does not generally require storage inside the EU, but its Chapter V restricts *transfers* of personal data outside it without safeguards [EU 2016]; some other jurisdictions mandate localization. Either way, partitions get pinned to regions.

Mathematical foundations: why replicate, and why failure is normal

FORMULA

Availability of k independent replicas, and the chance that something in a fleet fails

$$A_k = 1 - (1 - a)^k \qquad P_{\text{any}} = 1 - (1 - p)^n$$

Symbol	Meaning	Units
a	Availability of one replica (fraction of time it can serve)	dimensionless, 0–1
k	Number of replicas, any one of which can serve a request	count
A_k	Availability of the replicated service, assuming independent failures and instant failover	dimensionless
p	Probability that one node fails during a given period	dimensionless
n	Number of nodes in the fleet	count
P_{any}	Probability that at least one node fails in that period	dimensionless

WORKED EXAMPLE

Worked example — PhotoShare's metadata service.

1. One server has availability $a = 0.999$ (about 8.8 hours down per year).
2. With $k = 3$ independent replicas: $A_3 = 1 - (0.001)^3 = 1 - 10^{-9}$, nine nines on paper.
3. Now the fleet view. PhotoShare runs $n = 1,000$ nodes, each with probability $p = 0.001$ of failing on a given day. $P_{\text{any}} = 1 - 0.999^{1000}$. Since $\ln 0.999 \approx -0.0010005$, $0.999^{1000} \approx e^{-1.0005} \approx 0.368$, so $P_{\text{any}} \approx 0.632$.

Interpretation: replication makes the *service* far more available than any node, but the same scale guarantees that on about 63% of days *some* node fails. Failure is a daily routine, not an exception. Engineering implication: the nine nines are fiction, because replica failures are not independent (shared power, shared software bugs, shared network) and failover is not instant. The real gains come from placing replicas in separate failure domains and from making failover automatic and well tested, which is the subject of Chapter 27.

RECOMMENDATION

Recommendation — Distribute only for a stated reason. A single well-provisioned database with a hot standby serves many products and avoids most of this chapter's problems. When you do distribute, the reason tells you what to optimize: partitioning for scale, replication for fault tolerance, locality for latency, region pinning for residency.

24.5 The eight fallacies of distributed computing

24.5.1 Origin and the list

Around 1994, L. Peter Deutsch at Sun Microsystems listed seven false assumptions that newcomers to distributed applications make; James Gosling later added an eighth (Bill Joy and Tom Lyon are credited with the first four). The list circulated informally and was explained in detail by Rotem-Gal-Oz [Rotem-Gal-Oz 2006]. Each fallacy is the hidden assumption behind a class of outages; code that treats a remote call as a local one (§8.8) assumes several at once.

Table 24.2 — The eight fallacies and what each one breaks.

#	Fallacy	Reality	A concrete consequence	Mitigation
1	The network is reliable	Packets and connections are lost; switches and links fail daily	A payment request times out; the client retries; the customer is charged twice	Timeouts, retries with backoff, idempotency keys (§22.12, §22.15)
2	Latency is zero	Within a datacenter about 0.5 ms per round trip; across oceans 60–150 ms	A page making 100 sequential calls takes 50 ms in one region and about 7 s across regions	Batch calls, parallelize, colocate chatty services
3	Bandwidth is infinite	Links are finite and shared	Re-replicating 10 TB over a 1 Gbps link takes about 22 hours	Compress, ship deltas, throttle background traffic
4	The network is secure	Internal networks are routinely breached	Plaintext service-to-service traffic leaks credentials after one compromised host	Mutual TLS, zero trust (Ch 10)
5	Topology does not change	Nodes, routes and addresses change continuously	A client caches a server IP; the node is replaced; requests fail until restart	Service discovery, short DNS TTLs, re-resolution (§7.6)
6	There is one administrator	Many teams and providers control parts of the path	Another team changes a firewall rule or lets a certificate expire, and your service breaks	Explicit dependency contracts, certificate automation, change visibility
7	Transport cost is zero	Serialization costs CPU; providers charge for data transfer	Cross-zone chatter appears as a large line item on the cloud bill	Measure bytes per request, keep traffic zone-local where safe
8	The network is homogeneous	Mixed hardware, OS, protocol and software versions	During a rolling upgrade, v1 nodes cannot parse v2 messages and requests fail	Versioned, backward- and forward-compatible encodings

The arithmetic behind three rows: 100 calls × 0.5 ms = 50 ms, but × 70 ms = 7 s; $10 \times 10^{12} \times 8 / 10^{9} = 80{,}000$ s ≈ 22 h to copy 10 TB at 1 Gbps (which is why recovery spreads a node's data over many peers, §27.12); and at an *illustrative* $0.01/GB, 100 MB/s of cross-zone traffic is 8,640 GB/day, about $86 per day.

INDUSTRY EXAMPLE

AWS US-East, April 2011 — a topology change becomes an outage. During a network upgrade in one Availability Zone, traffic was shifted onto a lower-capacity network. Elastic Block Store (EBS) nodes lost contact with their replicas and all began re-mirroring at once; the storm exhausted capacity and degraded EBS and dependent services for days. Fallacies 1, 3 and 5 combined: a topology change made the network unreliable, and recovery traffic assumed plentiful bandwidth. *Evidence: Primary* — [AWS 2011]

COMMON MISTAKE

Mistake — "A remote procedure call is just a function call." A remote call can time out with an *unknown* outcome, take a million times longer, and run twice. Good RPC frameworks expose deadlines and status codes (§8.8, §22.11).

24.6 System models: timing and failures

24.6.1 Why a protocol needs a system model

A protocol is correct only *relative to its assumptions*: Raft (§26.7) tolerates nodes that crash but not nodes that lie, and a clock-based lease (§26.11) is safe only if drift is bounded.

DEFINITION

Definition — System model. A precise statement of what a distributed algorithm may assume about (1) **timing**: bounds on message delay, processing speed and clock drift, and (2) **faults**: how nodes and links may misbehave. Proofs of correctness hold only within the stated model.

Like a chess strategy proven under standard rules, a protocol proven for crash-stop nodes says nothing about a node that returns garbage from a corrupted disk.

24.6.2 Timing models

Table 24.3 — Timing models.

Model	What is assumed	Strength of the assumption	What it allows	Realistic?
Synchronous	Known upper bounds on message delay, processing time and clock drift	Strongest	Perfect failure detection by timeout; consensus despite failures, even Byzantine with enough nodes	Only in special hardware (avionics buses); not in commodity networks
Partially synchronous	Bounds exist but are unknown, or known bounds hold only after an unknown **global stabilization time** (GST)	Middle	Consensus that is always safe and terminates once the system behaves	Yes: the model behind Paxos, Raft and most production systems
Asynchronous	No timing bounds at all; messages are eventually delivered	Weakest	Safety-only reasoning; deterministic consensus impossible with one crash (§24.13)	Safe to assume, but too weak to build consensus on alone

The **partially synchronous** model of Dwork, Lynch and Stockmeyer [Dwork et al. 1988] fits real datacenters: usually timely, occasionally not (pauses, congestion, partitions) for an unknown time. A protocol for it must never violate **safety** (nothing bad happens) in bad periods and must make progress (**liveness**) once a good period lasts long enough.

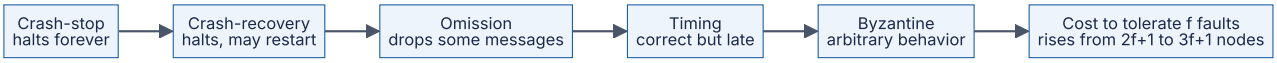
24.6.3 Failure models

Table 24.4 — Failure models, from most benign to most severe.

Model	What a faulty node may do	Real-world example	Detectable by others?	Nodes needed to tolerate f faults in consensus	Typical systems
Crash-stop	Stops forever; until then behaves correctly	Hardware dies; VM terminated	Only by timeout (no certainty in async)	$2f + 1$	Textbook protocols; Paxos variants
Crash-recovery	Crashes, later restarts; keeps only what it wrote to stable storage	Process restart after an OOM kill; power cycle	Only by timeout	$2f + 1$, with durable logs	Raft, ZooKeeper, most databases
Omission	Fails to send or receive some messages, otherwise correct	Full NIC buffer; overloaded switch dropping packets	Only by timeout	$2f + 1$	Covered by retransmission and quorums
Timing (performance)	Responds correctly but outside the assumed time bound	30 s garbage-collection pause; fail-slow disk	Only relative to a bound	Depends on the timing model	Lease-based systems are vulnerable
Byzantine (arbitrary)	Anything: wrong values, contradictory messages to different peers, forged messages	Memory corruption, software bug, malicious operator	Not in general	$3f + 1$	Blockchains, some aerospace and financial systems

Clarifications: a **fail-stop** processor, as defined by Schlichting and Schneider, halts *detectably* [Schlichting and Schneider 1983]; plain crash-stop does not, which is the realistic case. Under **crash-recovery**, anything a node promised (a vote, an acknowledgment) must reach stable storage before the promise is sent, which is why Raft persists its term and vote (§26.7). The classic **timing** fault is a lease holder that pauses for 30 s and then writes as if it still held the lease; fencing tokens (§26.13) exist for it. **Byzantine** faults, named in [Lamport et al. 1982], include bit flips and bugs, not just malice. Links have models too: **fair-loss**, **stubborn** (retransmit forever) and **perfect** (exactly-once delivery, built with acknowledgments and duplicate suppression) [Cachin et al. 2011]; TCP approximates a perfect link only while a connection lasts.

Figure 24.3 — Failure models nest: each one includes the behaviors of those inside it.



Reading the diagram. Each arrow points to a more general model: a crash is a special omission, an omission is an infinitely late message, and everything is a special case of

Byzantine. Tolerating a model on the right tolerates all to its left, at higher cost (final box). The hierarchy is conceptual, with no protocols, storage boundaries or sync/async paths; the practical failure point is assuming a model weaker than reality.

Mathematical foundations: replicas needed per fault model

FORMULA

Minimum replicas for consensus in the partially synchronous model

$$n_{\text{crash}} \geq 2f + 1 \qquad n_{\text{Byz}} \geq 3f + 1$$

Symbol	Meaning	Units
f	Maximum number of nodes that may be faulty at once	count
n_{crash}	Nodes needed when faults are crashes or omissions	count
n_{Byz}	Nodes needed when faults may be arbitrary	count

The intuition for $2f + 1$: any decision must be backed by a **majority quorum** of $f + 1$ nodes, and any two majorities overlap in at least one node, so two conflicting decisions cannot both gather a quorum. For Byzantine faults, quorums of $2f + 1$ out of $3f + 1$ overlap in at least $f + 1$ nodes, so at least one honest node is in every overlap. Quorums in depth are in §26.4.

WORKED EXAMPLE

Worked example — How many replicas for PhotoShare's coordination service?

1. Requirement: survive one zone outage, where each zone holds one replica, so $f = 1$.
2. Crash faults: $n \geq 2(1) + 1 = 3$ replicas in 3 zones. A majority is 2, so with one zone down the other two still decide.
3. Survive two simultaneous zone losses ($f = 2$): $n \geq 5$, so 5 replicas, which needs 5 failure domains.
4. If the same service had to tolerate one *Byzantine* replica: $n \geq 3(1) + 1 = 4$, and every decision needs $2f + 1 = 3$ matching votes.

Interpretation: Byzantine tolerance costs a third more replicas at $f = 1$ and many more messages per decision. Engineering implication: inside one organization, use crash-recovery protocols plus checksums (§24.14); reserve Byzantine tolerance for mutually distrustful parties.

INDUSTRY EXAMPLE

Raft clusters such as etcd. Raft clusters typically run 3 or 5 members: 3 tolerate one failure, 5 tolerate two, and 4 still tolerate only one (a majority is 3), so even sizes add cost without tolerance. Raft assumes crash-recovery faults and partial synchrony. *Evidence: Primary* — [Ongaro and Ousterhout 2014]

24.7 Network partitions

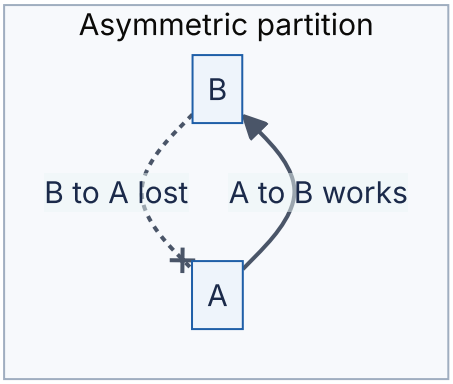
24.7.1 Definition and types

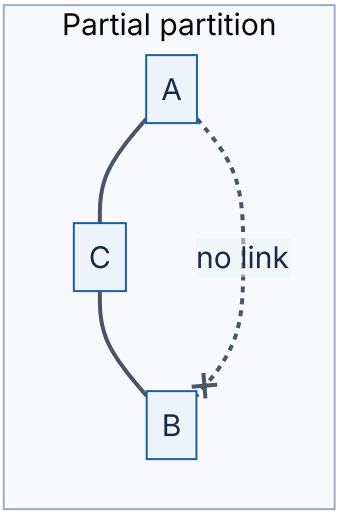
DEFINITION

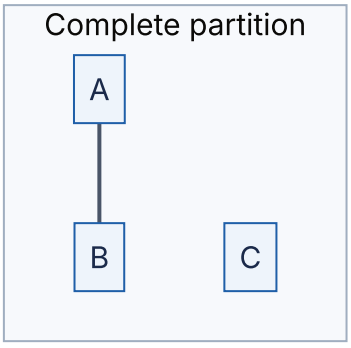
Definition — Network partition. A network fault after which some nodes that are running correctly cannot exchange messages with some other running nodes for a period of time. (This book reserves "partition" alone for a subset of data, §14.6; a network fault is always a **network partition**.)

From inside a node, a partition looks exactly like remote crashes, except that the "crashed" nodes keep running, may serve clients, and may believe *you* crashed. The less obvious shapes cause the worst bugs.

Figure 24.4 — Three kinds of network partition.







Reading the diagram. In the *complete* partition, the cluster splits cleanly: A and B talk to each other, C is cut off from both. In the *partial* partition, A and B cannot reach each other, yet both reach C, so C sees a healthy cluster while A and B each see a missing peer. In the *asymmetric* (one-way) partition, messages from A to B arrive but B's replies to A are lost. Solid lines are working links; dotted lines with crosses are failed directions. All traffic is the cluster's own heartbeat and data traffic; the failure points are the links, not the nodes, and adding nodes multiplies the number of links that can fail independently.

Table 24.5 — Partition types and the trouble each one causes.

Type	What happens	Typical cause	Why it is dangerous
Complete	The network splits into groups that cannot talk to each other	Lost uplink, failed inter-region link	Each side may elect a leader or accept writes: **split brain** (§26.12)
Partial	Some pairs cannot communicate while others can	Misconfigured firewall rule, failed link in a mesh, one bad switch port	Nodes disagree about who is alive; leadership can flap as nodes reach different peers
Asymmetric (one-way)	Traffic flows in one direction only	Saturated queue in one direction, ACL or routing asymmetry	A leader receives requests and heartbeats but its replies are lost, so it believes it is healthy while followers elect another leader
Intermittent (flapping)	Connectivity repeatedly comes and goes	Faulty optics, overloaded link	Constant failovers; each one costs work and risks inconsistency

24.7.2 How often do networks fail?

Bailis and Kingsbury's ironically titled survey "The Network is Reliable" collects the evidence [Bailis and Kingsbury 2014]:

- **Microsoft datacenters.** A measurement study of several Microsoft datacenters over a year [Gill et al. 2011] reported on average 5.2 device failures and 40.8 link failures per day, with a median time to repair of about five minutes and some failures lasting much longer. It also found that network redundancy was far less effective than hoped, improving the median traffic carried during failures by only about 43%.
- **Google clusters.** Jeff Dean's description of a typical first year of a new cluster included, among many other events, around 20 rack failures, about 5 racks that "go wonky" with half their packets lost, around 8 network maintenances of which several cause about 30 minutes of random connectivity loss, and about 3 router failures [Dean 2009].
- **Failures in deployed software.** Alquraan et al. studied 136 failures caused by network partitions in 25 widely used systems, including databases, coordination services and message brokers [Alquraan et al. 2018]. Most had catastrophic effects such as data loss, reappearance of deleted data and broken locks; many could be triggered by isolating a single node; and *partial* partitions were a significant and previously under-studied cause.

A large fleet therefore sees many small network faults every day: assume *some* pair of nodes cannot talk *right now*.

INDUSTRY EXAMPLE

Cloudflare, November 2020 — a partial partition and a Raft cluster. A partially failed switch left some members of an etcd (Raft) cluster able to reach each other and some not. The cluster could not keep a stable leader, so it could not accept writes; dependent automation that manages database primaries triggered failovers that overloaded other databases, and Cloudflare's API and dashboard were degraded for several hours. The post-mortem's title reflects that partial connectivity gave nodes contradictory views of the cluster. *Evidence: Primary* — [Lianza and Snook 2020]

24.7.3 What a partition forces you to choose

A node cut off from its peers can **refuse** (keeping consistency, losing availability on the minority side) or **answer** from local data and reconcile later (keeping availability, allowing divergence to be resolved with version vectors, §24.11, and conflict resolution, §27.14). CAP and PACELC (§25.8–§25.9) make this precise; you choose not *whether* partitions happen, only how to behave during one.

Table 24.6 — Partition failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Two primaries accept conflicting writes	Each side of a complete partition elects a leader	Two nodes report the primary role; divergent replication positions	Pick one history; reconcile or discard the other's writes	Majority quorums; fencing tokens (§26.13)
Leader flapping during a partial partition	Different nodes see different sets of peers	Election count and term number climbing quickly	Isolate the bad link or node	Pre-vote and leader-stickiness checks in Raft (§26.7); per-link health monitoring
"Healthy" leader that nobody hears	Asymmetric partition drops the leader's outbound traffic	Followers time out while the leader's own metrics look normal	Step the leader down on loss of quorum acknowledgments	Leaders check that a quorum acknowledges them within a lease period
Recovery storm after healing	All nodes resynchronize at once	Bandwidth and CPU spikes right after connectivity returns	Throttle resynchronization	Rate-limited, prioritized anti-entropy (§27.13)

COMMON MISTAKE

Mistake — "Partitions mean the cable was cut." Most partitions in practice are partial, short, or caused by software and configuration: a firewall change, a full queue, a long garbage-collection pause that makes a node unreachable for 20 s. A node that is alive but paused is, to its peers, partitioned.

24.8 Partial failures, gray failures and timeouts

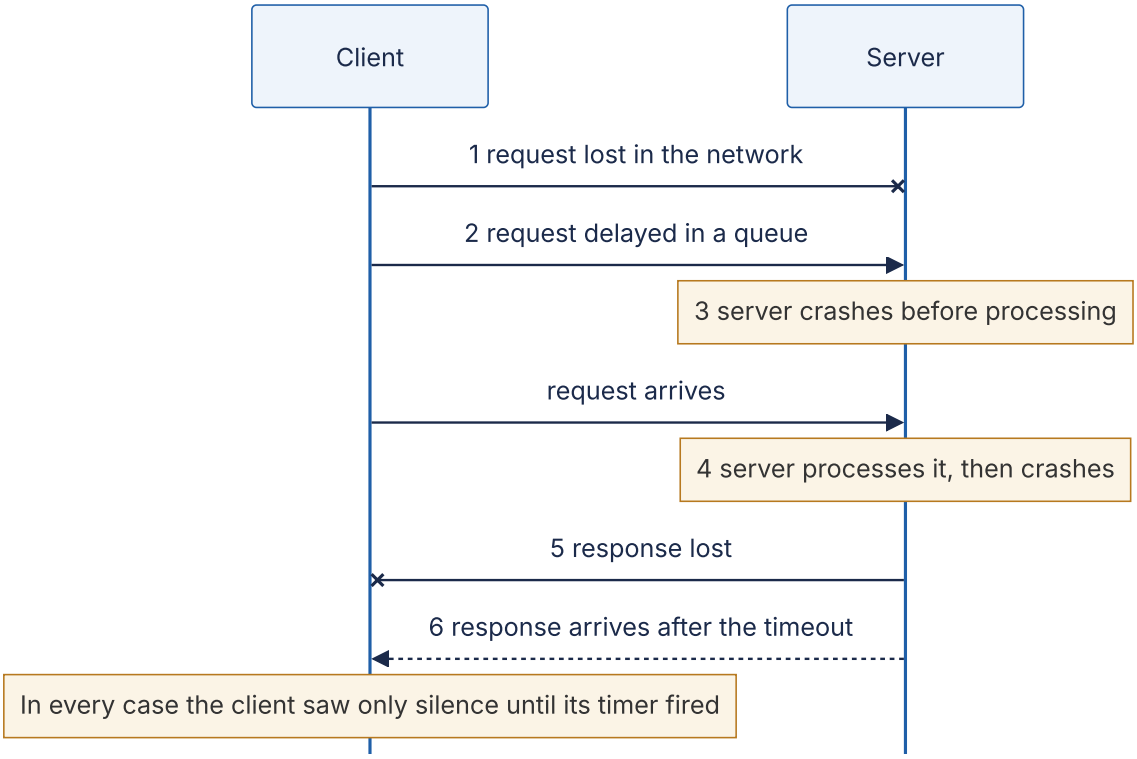
24.8.1 Partial failure

DEFINITION

Definition — Partial failure. A state in which some components of a system have failed while others continue to work, so that an operation may be applied in some places and not others, and the working components cannot reliably tell which components failed.

On one computer, partial failure is rare enough to ignore; in a fleet it is normal (§24.4.1). The hard part is the *ambiguity*.

Figure 24.5 — Six things that can happen to one request, all of which look the same to the client.



Reading the diagram. The client sends a synchronous request (solid arrows) and waits for a response (dashed arrow). Outcome 1 loses the request; 2 delays it; 3 and 4 are server crashes before and after the side effect; 5 loses the response; 6 delivers it too late to matter. Outcomes 4, 5 and 6 mean the operation *happened*; 1 and 3 mean it did not; 2 means it may still happen. The storage boundary is the server's database, which may or may not contain the effect. The failure points are the network in both directions and the server. Retrying is safe only if the operation is idempotent (§22.15), because in half of these cases the retry is a duplicate.

24.8.2 Gray failure

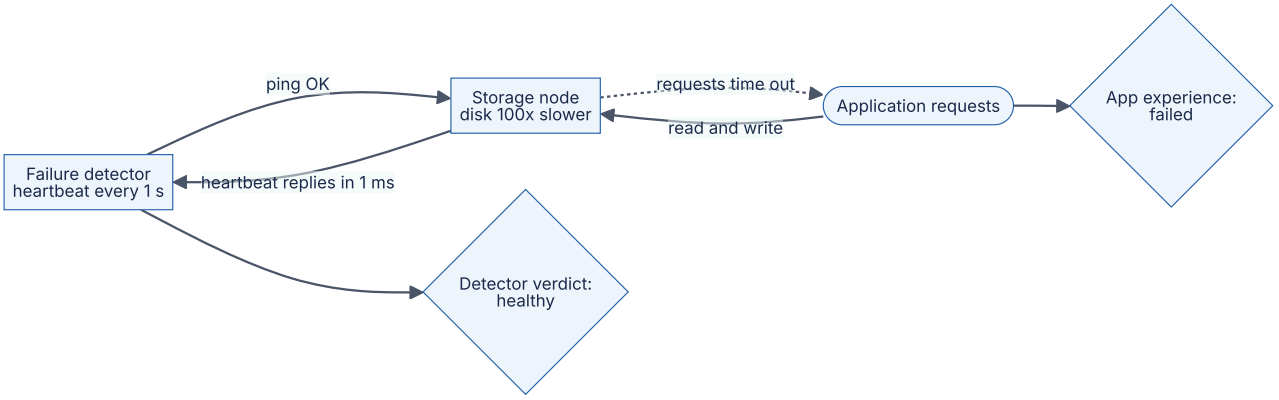
Crashes are the easy case. Huang et al. found that many serious incidents in Microsoft's cloud came from something subtler, **gray failure** [Huang et al. 2017].

DEFINITION

Definition — Gray failure. A component failure that is *differentially observable*: some observers (often the system's own failure detector) consider the component healthy, while others (often the applications using it) experience it as failed or badly degraded.

Examples of gray failure include a network interface that drops 1% of packets (TCP throughput collapses, §7.15.3, but pings succeed); a disk that completes every I/O but takes 500 ms instead of 5 ms (a **fail-slow** fault; Gunawi et al. collected over a hundred such hardware cases from large production systems [Gunawi et al. 2018]); and a process whose heartbeat thread runs happily while its request-handling threads are deadlocked.

Figure 24.6 — Gray failure is a disagreement between observers.



Reading the diagram. The failure detector probes the storage node with a cheap heartbeat that never touches the disk, so it gets fast replies and concludes "healthy". The application's synchronous read and write requests go through the slow disk and time out (dotted arrow: the response effectively never arrives in time). The storage boundary, the node's disk, is the failure point, and it is invisible to the probe. Because the detector says healthy, the load balancer keeps sending traffic and no failover happens; adding more nodes does not help, since the sick node still receives its share. The fix is to make the detector observe what the application observes.

Huang et al. recommend closing the observation gap: detect with what applications experience. In practice: **health checks that exercise the real dependency path**, **outlier ejection** of nodes whose client-observed errors or latency are far worse than their peers' (passive health checks, §19.5), and treating "too slow" as down.

24.8.3 Timeouts: the only failure detector you have

Without timing bounds you cannot *know* that a node crashed; you only see silence. Every failure detector, from a timeout to phi-accrual or SWIM (§27.10), turns silence into a *suspicion*.

Mathematical foundations: choosing a timeout

FORMULA

Timeout from bounded delays, and its practical percentile form

$$T = 2d + r \qquad T_{\text{practical}} = \text{RTT}_{p99.9} + r_{p99.9} + m$$

Symbol	Meaning	Units
d	Upper bound on one-way network delay	s
r	Upper bound on the time the remote node takes to process the request	s
T	Timeout after which the absence of a reply proves a fault (only if the bounds are real)	s
$\text{RTT}_{p99.9}$	Measured 99.9th-percentile round-trip time	s
$r_{p99.9}$	Measured 99.9th-percentile processing time	s
m	Safety margin for pauses (garbage collection, scheduling)	s

With real bounds, silence beyond $2d + r$ would prove a fault. Real networks have none, so the practical form accepts a small, chosen probability of mistaking slow for dead.

WORKED EXAMPLE

Worked example — Timeout for PhotoShare's metadata replicas.

1. Measured inside one region: RTT p50 = 0.4 ms, p99.9 = 4 ms; processing p99.9 = 20 ms.
2. Garbage-collection pauses on these JVM nodes reach 150 ms a few times a day; choose $m = 200$ ms.
3. $T_{\text{practical}} = 4 + 20 + 200 = 224$ ms; round to 250 ms for a request timeout.
4. For declaring a *node* dead (which triggers failover), require several consecutive misses: with heartbeats every 250 ms and 4 misses, detection takes about 1 s.

Interpretation: the pause margin, not the network, dominates. Too low a T causes false failovers that can cascade; too high leaves users waiting. Engineering implication: measure pauses and tail latency first, use a short timeout for retrying a request (§22.11) and a longer, multi-miss rule for failing over a node, and prefer adaptive detectors (§27.10).

Table 24.7 — Partial-failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Duplicate side effect after timeout	Request succeeded, response lost; client retried	Duplicate rows, double charges	Compensate or deduplicate	Idempotency keys (§22.15)
Healthy node declared dead	Timeout shorter than a garbage-collection pause	Failover events correlated with pause logs	Let the node rejoin as follower	Timeouts from measured p99.9 plus pause margin; several misses
Sick node kept in rotation	Gray failure: health check misses the slow path	Client-side p99 and error rate high for one node	Eject the node manually	Deep health checks; outlier ejection
Cascading slowness	Callers wait the full timeout on a slow dependency	Thread pools full; latency rises everywhere	Shed load; open circuit breakers	Deadlines, bulkheads, breakers (§22.11–§22.14)

COMMON MISTAKE

Mistake — "If the node answered the health check, it is up." A health check answers one narrow question. Health is what clients experience.

24.9 Physical clocks: drift, NTP, PTP and skew

24.9.1 Two clocks in every computer

DEFINITION

Definition — Time-of-day clock and monotonic clock. A **time-of-day** (wall-clock) clock returns the current date and time, for example as seconds since the Unix epoch (Linux `CLOCK_REALTIME`, Python `time.time()`). It is synchronized with other machines and *can jump* forward or backward when corrected. A **monotonic** clock (Linux `CLOCK_MONOTONIC`, Python `time.monotonic()`) only moves forward at a steady rate from an arbitrary starting point. It is meaningless across machines but correct for measuring durations on one machine.

Use the monotonic clock for timeouts, rate-limiter refill (§22.4) and latency; never assume two machines' wall clocks agree.

24.9.2 Quartz drift

A computer counts the vibrations of a quartz crystal whose frequency varies with manufacturing tolerance, temperature and age. The error is expressed in **parts per million (ppm)**: 1 ppm is 1 µs per second, or 86.4 ms per day. Commodity server oscillators typically drift by tens of ppm, and Google's Spanner design conservatively assumes a worst case of 200 ppm [Corbett et al. 2012].

FORMULA

Clock drift and the resynchronization interval

$$\Delta(t) = \rho\, t \qquad \Delta_{AB}(t) \le 2\rho\, t \qquad T_{\text{sync}} \le \frac{\varepsilon}{2\rho}$$

Symbol	Meaning	Units
ρ	Maximum drift rate of one clock	ppm (1 ppm = 10^{-6} s/s)
t	Time since the clock was last synchronized	s
$\Delta(t)$	Worst-case error of one clock relative to true time	s
$\Delta_{AB}(t)$	Worst-case skew between two clocks drifting in opposite directions	s
ε	Largest skew you can tolerate between any two clocks	s
T_{sync}	Longest interval between synchronizations that keeps skew within ε	s

WORKED EXAMPLE

Worked example — Drift on PhotoShare's servers, and Spanner's sawtooth.

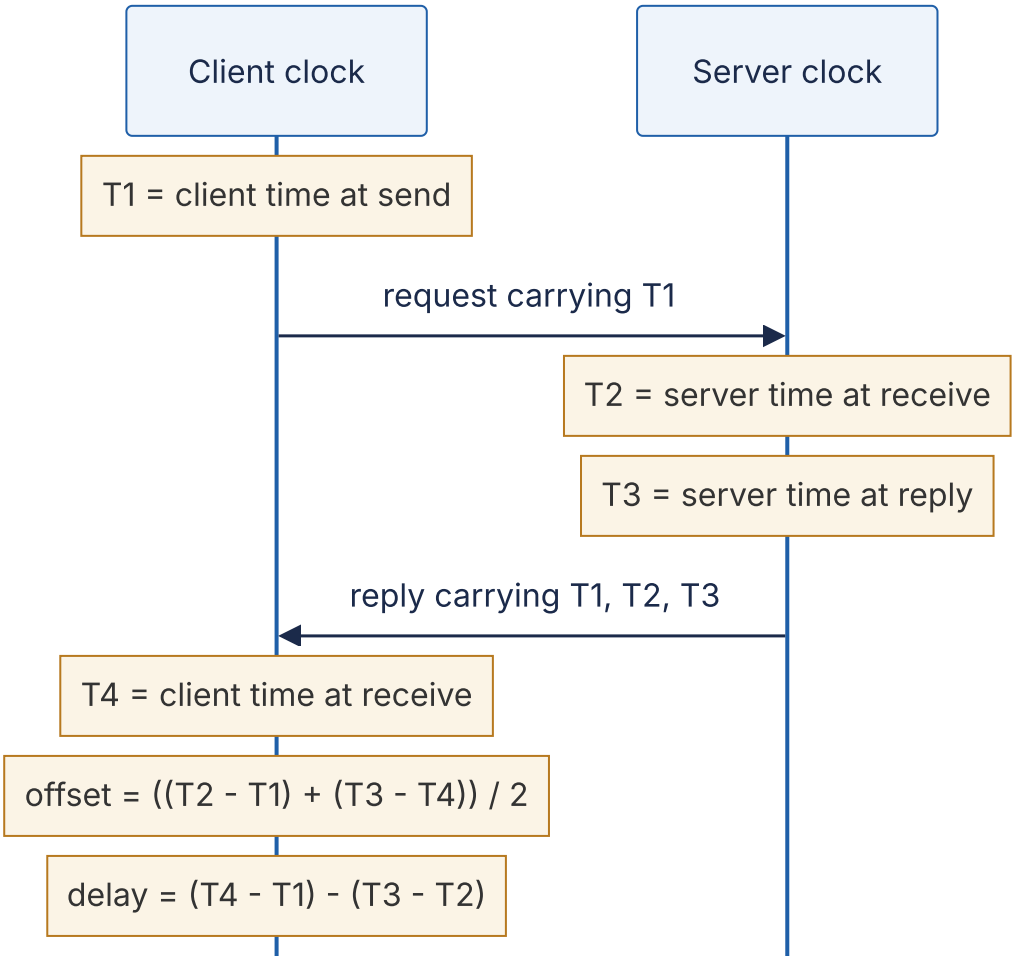
1. A server drifts at $\rho = 50$ ppm $= 50 \times 10^{-6}$ s/s. Unsynchronized for a day: $\Delta = 50 \times 10^{-6} \times 86,400 = 4.32$ s.
2. Two servers, one fast and one slow at 50 ppm, diverge at $2 \times 50 = 100$ µs per second.
3. To keep them within $\varepsilon = 1$ ms: $T_{\text{sync}} \le 0.001/(2 \times 50 \times 10^{-6}) = 10$ s.
4. Spanner: each server polls its time masters every 30 s and assumes 200 ppm, so uncertainty grows by up to $200 \times 10^{-6} \times 30 = 6$ ms between polls. Corbett et al. describe exactly this sawtooth of 0 to 6 ms on top of about 1 ms of communication delay, giving a total uncertainty ε of about 1 to 7 ms [Corbett et al. 2012].

Interpretation: drift is tiny per second but unbounded over time, so clocks must be resynchronized continually. Engineering implication: alert on *synchronization loss*, not just offset; a host that silently stopped syncing drifts further every hour.

24.9.3 NTP: synchronizing over a network

The **Network Time Protocol** (NTP), designed by David Mills and specified as NTPv4 in RFC 5905 [Mills et al. 2010], measures a round trip, assumes equal delay in both directions, and computes the offset from four timestamps.

Figure 24.7 — The NTP on-wire exchange.



Reading the diagram. One synchronous UDP request/response (port 123): T_1 and T_4 are read from the client's clock, T_2 and T_3 from the server's. No storage is involved. Failure points: asymmetric delay (biases the offset) and a wrong server clock (handled by querying several servers). NTP scales through a server hierarchy (Figure 24.8).

FORMULA

NTP offset, round-trip delay and error bound (RFC 5905)

$$\theta = \frac{(T_2 - T_1) + (T_3 - T_4)}{2} \qquad \delta = (T_4 - T_1) - (T_3 - T_2) \qquad |\theta - \theta_{\text{true}}| \le \frac{\delta}{2}$$

Symbol	Meaning	Units
T_1	Client clock when the request is sent	s
T_2	Server clock when the request arrives	s
T_3	Server clock when the reply is sent	s
T_4	Client clock when the reply arrives	s
θ	Estimated offset: server clock minus client clock; the client adds θ to correct itself	s
δ	Round-trip network delay, excluding the server's processing time	s
θ_{true}	Actual offset	s

If the request takes d_1 and the reply d_2, the error is $(d_1 - d_2)/2$, at most $\delta/2$ because $d_1 + d_2 = \delta$; hence NTP prefers low-delay samples.

WORKED EXAMPLE

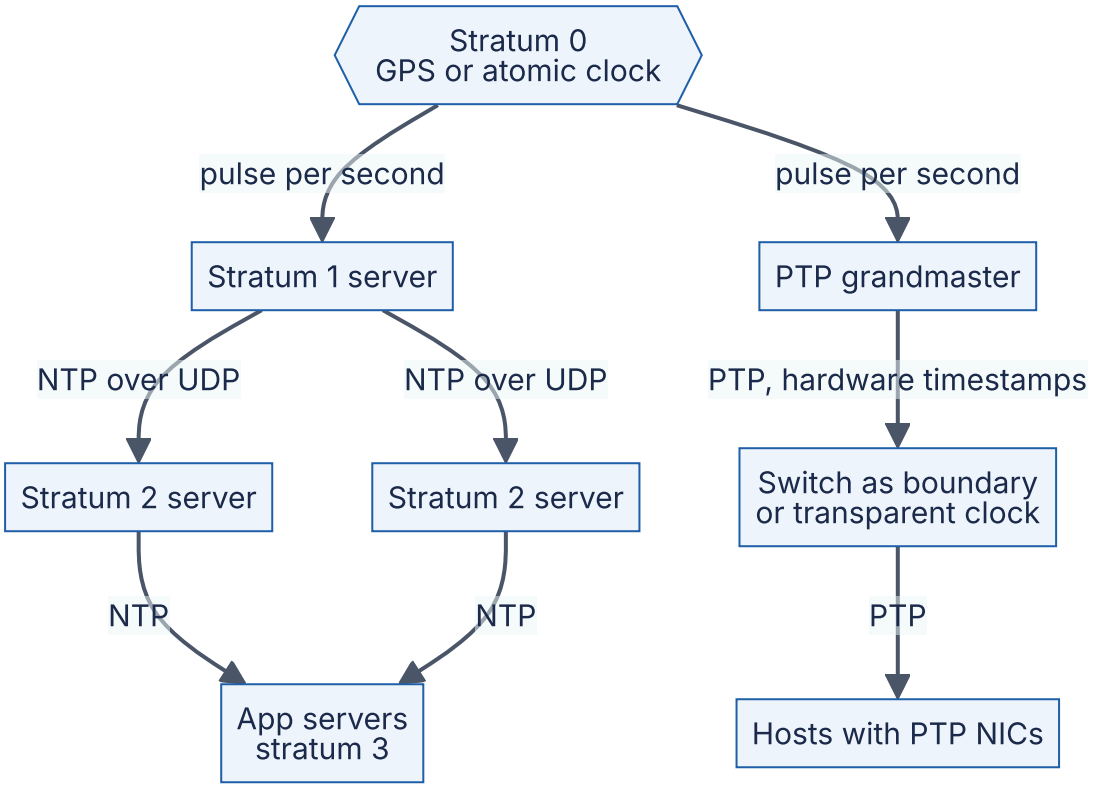
Worked example — One NTP exchange, symmetric and asymmetric.

Timestamps in seconds after 10:00:00: $T_1 = 0.000$ (client), $T_2 = 0.120$ (server), $T_3 = 0.121$ (server), $T_4 = 0.061$ (client).

1. Delay: $\delta = (0.061 - 0.000) - (0.121 - 0.120) = 0.061 - 0.001 = 0.060$ s $= 60$ ms.
2. Offset: $\theta = [(0.120 - 0.000) + (0.121 - 0.061)]/2 = (0.120 + 0.060)/2 = 0.090$ s $= 90$ ms. The client's clock is 90 ms behind the server's.
3. Check: if each direction took 30 ms and the client was 90 ms behind, the request sent at client time 0.000 (server time 0.090) arrives at server time 0.120, and the reply sent at 0.121 arrives at server time 0.151, which is client time 0.061. Both match.
4. Asymmetric case: suppose the request actually took 50 ms and the reply 10 ms (same $\delta = 60$ ms). NTP still computes 90 ms, but the true offset is $0.120 - 0.050 = 0.070$ s, an error of $(50 - 10)/2 = 20$ ms, within the bound $\delta/2 = 30$ ms.

Interpretation: NTP cannot see path asymmetry, so one sample over a 60 ms path is good only to ±30 ms. Engineering implication: run stratum-1 or stratum-2 servers inside the datacenter, where sub-millisecond round trips shrink the bound.

Figure 24.8 — The time-distribution hierarchy: NTP strata and a PTP domain.



Reading the diagram. A **stratum** is the distance from a reference clock: stratum-0 devices (GPS, atomic clocks) feed stratum-1 servers directly, and each NTP hop adds one (16 means unsynchronized). Clients query several servers so a bad one (a **falseticker**) is outvoted. The right-hand PTP domain distributes time from a grandmaster through PTP-aware switches to hosts with hardware-timestamping NICs. Arrows are periodic request/response exchanges; failure points are reference loss, falsetickers and asymmetry; the tree scales because each server serves many clients.

How an NTP client turns samples into a clock

A daemon such as `ntpd` or `chrony` (1) polls several servers repeatedly (64–1,024 s by default in `ntpd`), (2) keeps the lowest-delay samples, (3) **selects** the largest group of servers whose error intervals overlap, discarding falsetickers, (4) combines the survivors, and (5) disciplines the clock: small offsets are **slewed** (`ntpd` runs the clock at most 500 ppm fast or slow), large ones (over 128 ms by default) are **stepped**, possibly *backwards*. Typical results are milliseconds over a good wide-area path and well under a millisecond on a quiet LAN, with no hard guarantee: congestion, asymmetry or a paused virtual machine can produce offsets of seconds.

24.9.4 PTP: sub-microsecond synchronization

The **Precision Time Protocol** (PTP, IEEE 1588-2008 and its 2019 revision) [IEEE 2008; IEEE 2019] uses the same four-timestamp idea with three improvements: **hardware timestamping** in the NIC removes operating-system stack jitter, the main error in software NTP; switches act as **transparent clocks** (adding their queuing time to a correction field) or **boundary clocks** (re-serving time downstream), so asymmetry is measured rather than guessed; and **frequent exchanges** (`Sync`, `Delay_Req`, `Delay_Resp`) track drift closely. A well-built network achieves sub-microsecond synchronization, at the cost of PTP-capable NICs, switches and GPS-referenced grandmasters.

> INDUSTRY EXAMPLE
>
> **Precision time in public clouds and at Meta.** Meta has described deploying PTP across its datacenters for sub-microsecond precision [Obleukhov and Byagowi 2022]. AWS announced microsecond-accurate time from the Amazon Time Sync Service on supported instances in 2023, and publishes ClockBound, a daemon that returns the time together with an *error bound* [AWS 2023]. Applications increasingly ask “what is the time, *and how wrong could it be*?” *Evidence: Primary* — [Obleukhov and Byagowi 2022; AWS 2023]

24.9.5 Leap seconds and smearing

UTC is kept within 0.9 s of the Earth's irregular rotation by **leap seconds**: a minute ending `23:59:60` on 30 June or 31 December. Since 1972, 27 have been added, the latest at the end of 2016 (TAI − UTC = 37 s). Unix time cannot represent `23:59:60`, so systems typically repeat a second and time appears to go backwards. The 2012 leap second triggered a Linux kernel timer bug that left many Java and MySQL processes spinning at full CPU; at the end of 2016, Cloudflare's DNS software computed a negative duration and some resolutions failed [Graham-Cumming 2017]. The mitigation is **leap smearing**: servers run slightly slow over a long window, absorbing the second gradually. Google smears linearly over 24 hours, noon to noon UTC [Google: Leap Smear].

FORMULA

Leap-smear rate

$$r_{\text{smear}} = \frac{1\ \text{s}}{W} \qquad e_{\max} = 0.5\ \text{s (midpoint offset from UTC)}$$

Symbol	Meaning	Units
W	Smear window	s
r_{smear}	Rate by which smeared clocks run slow during the window	ppm
$e_{\max}$	Offset between a smeared clock and true UTC at the leap second instant	s

WORKED EXAMPLE

Worked example — Google-style 24-hour smear.

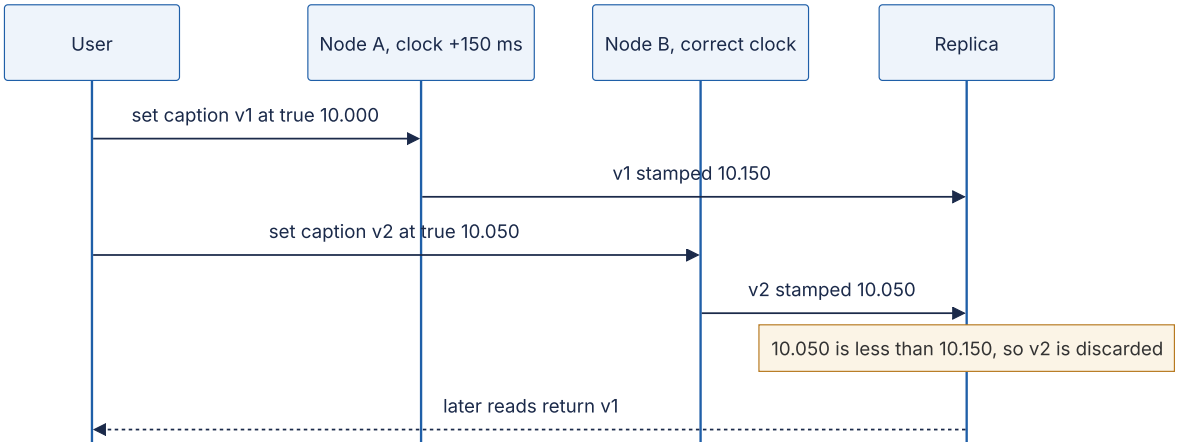
1. $W = 24 \times 3,600 = 86,400$ s.
2. $r_{\text{smear}} = 1/86,400 = 1.157 \times 10^{-5}$ s/s ≈ 11.6 ppm.
3. At the leap second, a smeared clock is half-way through: it differs from UTC by 0.5 s.

Interpretation: 11.6 ppm is an ordinary slew, so no client sees a step. Engineering implication: never mix smeared and unsmeared sources, which would disagree by up to 0.5 s. In 2022 the CGPM resolved to let UT1 − UTC grow larger by 2035, in effect ending leap seconds [BIPM 2022]; until then, systems must handle them.

24.9.6 Why wall-clock last-writer-wins loses data

Under **last-writer-wins (LWW)**, the write with the higher timestamp survives (Cassandra per cell, §12.6; many multi-primary systems, §14.5). With wall-clock timestamps, skew rather than event order decides the winner.

Figure 24.9 — Last-writer-wins discards the later write when one clock runs fast.



Reading the diagram. A user edits a PhotoShare caption through node A, then 50 ms later through node B. Each write is synchronous to its node and replicated asynchronously to the replica (the storage boundary). A's clock is 150 ms fast, so the earlier write carries the larger stamp and v2 is silently discarded. The failure point is A's clock; more replicas do not help, since all apply the same rule.

WORKED EXAMPLE

Worked example — How large is the danger window?

1. Skew between A and B: $\varepsilon = 150$ ms.
2. A write through B at true time t loses to an earlier write through A at true time $t - x$ whenever A's stamp is larger: $(t - x) + 0.150 > t$, i.e., $x < 150$ ms.
3. So any two writes to the same key through these two nodes that are less than 150 ms apart may be reordered. PhotoShare's caption edits are rare, but an inventory counter updated 50 times per second from several nodes would lose updates constantly.

Interpretation: the skew is a window within which LWW silently discards *causally later* writes. Engineering implication: use wall-clock LWW only where losing one of two near-simultaneous writes is acceptable; otherwise use logical clocks (§24.10–§24.12), version vectors (§24.11), conditional writes, or CRDTs (§27.14).

Code example

```
        """Clock arithmetic: NTP offset/delay, drift-based resync interval,
        and how last-writer-wins (LWW) loses a write when clocks are skewed."""


        def ntp_offset_delay(t1, t2, t3, t4):
            """RFC 5905 on-wire calculation (all values in seconds).
            t1, t4 are read from the client clock; t2, t3 from the server clock."""
            offset = ((t2 - t1) + (t3 - t4)) / 2   # theta: server clock minus client
clock
            delay = (t4 - t1) - (t3 - t2)          # delta: round trip minus server hold
time
            return offset, delay


        def resync_interval(max_skew_s, drift_ppm):
            """Longest interval between syncs that keeps two clocks within max_skew_s
            when each may drift up to drift_ppm in opposite directions."""
            return max_skew_s / (2 * drift_ppm * 1e-6)


        class LWWRegister:
            """A replica that keeps the value with the highest wall-clock timestamp."""

            def __init__(self):
                self.value, self.ts = None, float("-inf")

            def apply(self, value, ts):
                if ts > self.ts:
                    self.value, self.ts = value, ts
                    return "applied"
                return "discarded"


        if __name__ == "__main__":
            theta, delta = ntp_offset_delay(0.000, 0.120, 0.121, 0.061)
            print(f"NTP: offset {theta * 1000:.1f} ms, delay {delta * 1000:.1f} ms, "
                  f"error bound +/-{delta / 2 * 1000:.1f} ms")

            print(f"Resync every {resync_interval(0.001, 50):.0f} s to stay within 1 ms
at 50 ppm")

            # Node A's clock runs 150 ms fast; node B's clock is correct.
            skew = {"A": 0.150, "B": 0.000}
            writes = [  # (true time in s, node, value): the second write happens 50 ms
later
                (10.000, "A", "caption v1"),
                (10.050, "B", "caption v2"),
            ]
            replica = LWWRegister()
            for true_t, node, value in writes:
                stamp = true_t + skew[node]                    # what the node's clock
says
                print(f"t={true_t:.3f} node {node} writes {value!r:13} stamp=
{stamp:.3f}"
```

```
                    f" -> {replica.apply(value, stamp)}")
    print(f"Final value: {replica.value!r} (the later write was lost)")
```

Output:

```
NTP: offset 90.0 ms, delay 60.0 ms, error bound +/-30.0 ms
Resync every 10 s to stay within 1 ms at 50 ppm
t=10.000 node A writes 'caption v1'  stamp=10.150 -> applied
t=10.050 node B writes 'caption v2'  stamp=10.050 -> discarded
Final value: 'caption v1' (the later write was lost)
```

What to notice:

- `ntp_offset_delay` is a direct transcription of the RFC 5905 formulas; the sign convention is "server minus client".
- `LWWRegister.apply` returns `"discarded"` for the user's *latest* edit, and nothing in the system raises an error: LWW data loss is silent.
- Even if the replica received v2 first, the outcome is the same, because LWW orders by timestamp, not by arrival.

24.9.7 Clock sources and bounded uncertainty

Table 24.8 — Clock synchronization options.

Option	How it works	Typical accuracy (as of 2026)	Cost	Use when
Unsynchronized quartz	Free-running oscillator	Drifts seconds per day	None	Never for timestamps shared across machines
NTP over the Internet	Software timestamps, public servers	Milliseconds to tens of milliseconds	Free	Laptops, small fleets, logging
NTP inside a datacenter	Local stratum-1/2 servers, short paths	Sub-millisecond to a few ms	Low	Default for server fleets
PTP with hardware timestamps	NIC and switch timestamping	Sub-microsecond in a well-built LAN	PTP-capable NICs, switches, grandmasters	Finance, telecom, databases that exploit tight bounds
Bounded-uncertainty APIs	Return an interval [earliest, latest] instead of one value	Interval width of microseconds to milliseconds	Infrastructure plus discipline in using the bound	Correctness that depends on time (leases, commit wait)

The last row matters most for Part V. Google's **TrueTime** returns an interval guaranteed to contain true time, and Spanner waits out its width before making a commit visible (**commit wait**), turning bounded uncertainty into externally consistent transactions (§28.8).

Table 24.9 — Clock failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Negative durations; timeouts fire early or never	Wall clock used for durations; NTP stepped it	Negative latency metrics; stuck timers	Restart affected processes	Monotonic clocks for all durations
Lost writes under LWW	Skew larger than gap between conflicting writes	Users report edits "reverting"; audit logs disagree	Restore from log if kept	Logical or hybrid clocks, version vectors, CRDTs
Two lease holders at once	Lease holder's clock slow or process paused	Two nodes acting as owner	Fencing rejects stale owner	Fencing tokens (§26.13); bounded-uncertainty clocks
One node far off after hypervisor pause or failed sync	Synchronization stopped silently	Offset and "last sync" metrics	Step clock; quarantine node	Alert on sync loss; nodes refuse to serve when offset exceeds a bound
Leap second crash or 0.5 s disagreement	Repeated second; mixed smeared and unsmeared sources	Errors clustered at 23:59:60 UTC	Restart; fix time sources	Smear consistently across the fleet

COMMON MISTAKE

Mistake — "NTP keeps all our clocks in sync, so timestamps give a total order." NTP gives an *estimate*, usually off by milliseconds and occasionally by seconds; events closer than the skew cannot be ordered by timestamp (§17.10 makes the same point for IDs). Use logical clocks for ordering and explicit uncertainty bounds for real-time guarantees.

24.10 Logical clocks: Lamport clocks

24.10.1 Happened-before

Lamport's 1978 insight was that most systems need to know not *when* events happened but which could have *influenced* which [Lamport 1978].

DEFINITION

Definition — Happened-before (→). The smallest relation on events such that: (1) if a and b occur in the same process and a comes first, $a \to b$; (2) if a is the sending of a message and b its receipt, $a \to b$; (3) if $a \to b$ and $b \to c$, then $a \to c$. Two distinct events are **concurrent** ($a \parallel b$) if neither $a \to b$ nor $b \to a$.

Happened-before is **potential causality**: information from a *could* have reached b.

24.10.2 The algorithm

Think of a diary page counter: turn a page per event, write the page number on each letter sent, and on receiving a letter skip past its page number if higher.

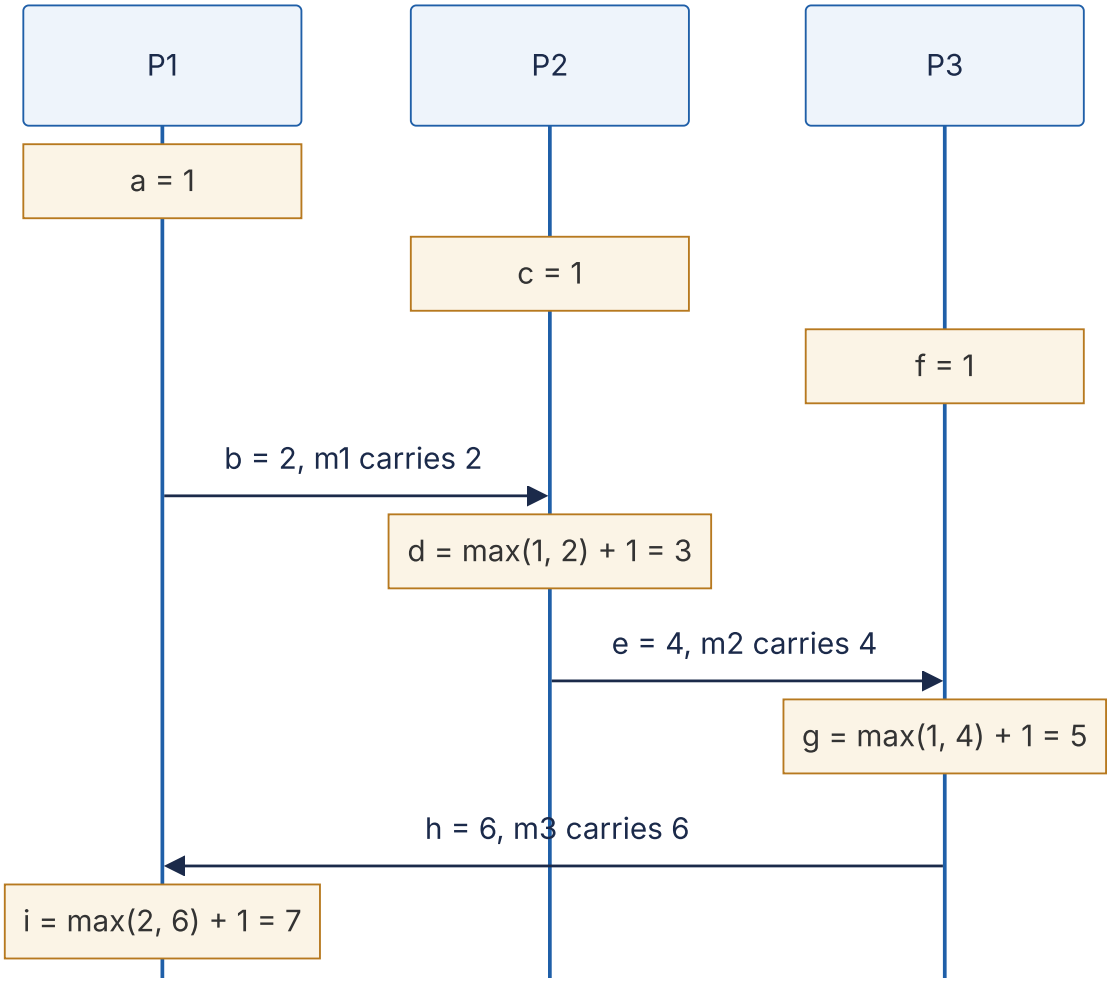
DEFINITION

Definition — Lamport clock. Each process P_i keeps an integer counter C_i, initially 0. **Rule 1:** before each event (local, send or receive), $C_i \leftarrow C_i + 1$; a send attaches the new C_i to the message. **Rule 2:** on receiving a message with timestamp t_m, set $C_i \leftarrow \max(C_i, t_m)$, then apply rule 1. The event's timestamp is the resulting C_i.

The rules guarantee the **clock condition**: if $a \to b$ then $C(a) < C(b)$. The converse does *not* hold: $C(a) < C(b)$ says only that b did not happen before a.

24.10.3 A worked trace

Figure 24.10 — Lamport timestamps on three processes.



Reading the diagram. Lifelines are processes, notes are events with timestamps, and arrows are asynchronous messages carrying the sender's timestamp. P2's receipt `d` jumps to $\max(1, 2) + 1 = 3$, and the chain continues through P3 back to P1. There is no storage or failure point; only causality is shown. Note the limitation: `c` (1) < `b` (2), yet they are concurrent. The cost is one integer per message, at any scale.

24.10.4 Code example

```
        """Lamport logical clocks: replay the trace of Figure 24.10 and derive a total
order."""


        class LamportProcess:
            def __init__(self, pid):
                self.pid, self.clock, self.log = pid, 0, []

            def _tick(self, name, kind):
                self.clock += 1                          # rule 1: increment before every
event
                self.log.append((self.clock, self.pid, name, kind))
                return self.clock

            def local(self, name):
                return self._tick(name, "local")

            def send(self, name):
                return self._tick(name, "send")          # the message carries this time-
stamp

            def receive(self, name, msg_ts):
                self.clock = max(self.clock, msg_ts)     # rule 2: jump past the sender's
time
                return self._tick(name, "receive")


        if __name__ == "__main__":
            p1, p2, p3 = LamportProcess("P1"), LamportProcess("P2"),
LamportProcess("P3")
            p1.local("a")
            m1 = p1.send("b")            # m1: P1 -> P2
            p2.local("c")
            p2.receive("d", m1)
            m2 = p2.send("e")            # m2: P2 -> P3
            p3.local("f")
            p3.receive("g", m2)
            m3 = p3.send("h")            # m3: P3 -> P1
            p1.receive("i", m3)

            events = p1.log + p2.log + p3.log
            for ts, pid, name, kind in sorted(events):  # total order: (timestamp,
process id)
                print(f"{ts:>2} {pid} {name} ({kind})")
```

Output:

```
1 P1 a (local)
1 P2 c (local)
1 P3 f (local)
2 P1 b (send)
3 P2 d (receive)
4 P2 e (send)
5 P3 g (receive)
6 P3 h (send)
7 P1 i (receive)
```

What to notice:

- Sorting by the pair (timestamp, process ID) gives a **total order** that every process can compute identically and that is consistent with causality; ties such as `a`, `c`, `f` at 1 are broken by process ID.
- The total order is *arbitrary* among concurrent events: it places `c` before `b`, though neither influenced the other.

24.10.5 Uses and limits

Lamport used this total order for distributed mutual exclusion [Lamport 1978], the seed of **state machine replication** (§26.3). Raft's **terms** behave like a Lamport clock: a node adopts any higher term it sees and rejects lower ones [Ongaro and Ousterhout 2014]; fencing tokens (§26.13) are similar. What Lamport clocks cannot do is detect concurrency.

24.11 Vector clocks and version vectors

24.11.1 The algorithm

Vector clocks were introduced independently by Fidge and Mattern [Fidge 1988; Mattern 1989]. Instead of one counter, each process keeps a counter *for every process*: its knowledge of how many events each process has performed.

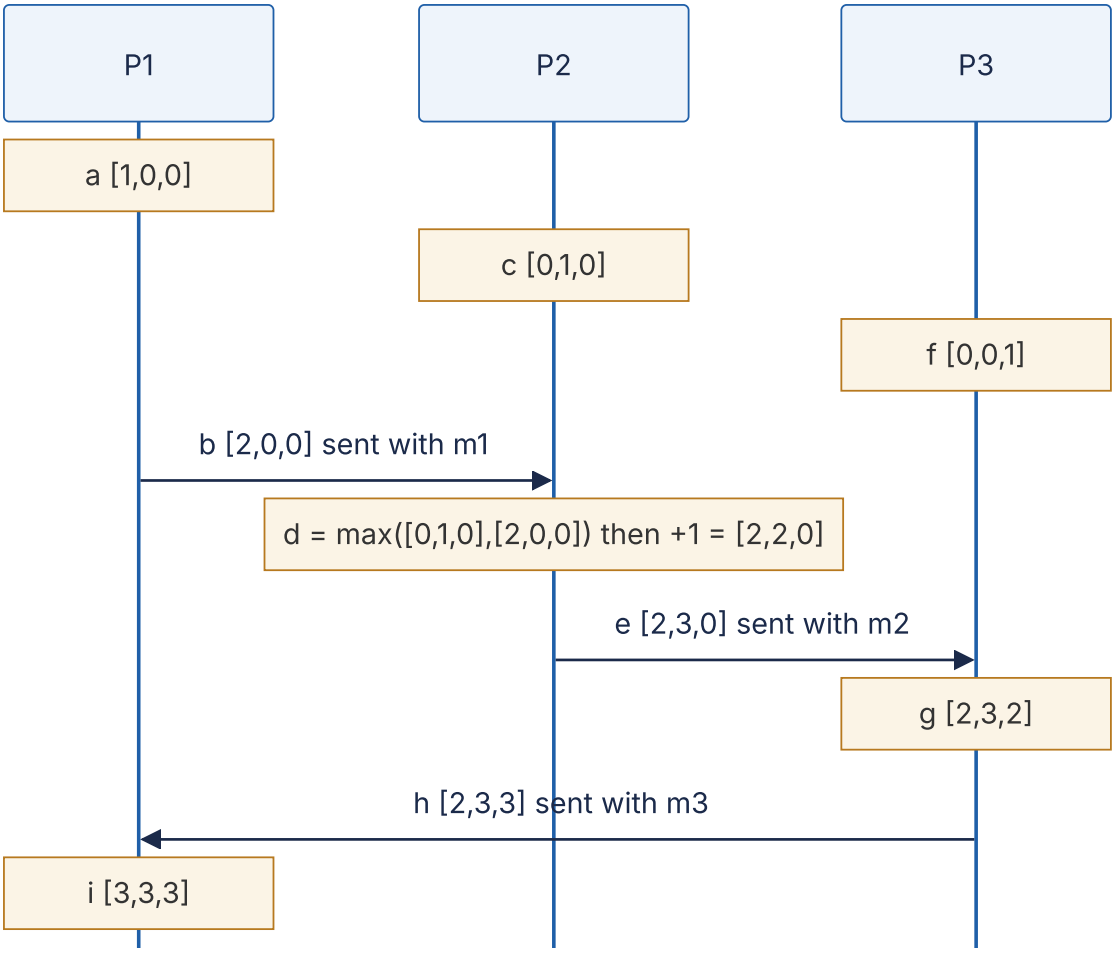
DEFINITION

Definition — Vector clock. With n processes, each process P_i keeps a vector $V_i[1..n]$, initially all zeros. Before each event, P_i increments $V_i[i]$; a send attaches a copy of V_i. On receiving a message carrying V_m, P_i sets $V_i[k] \leftarrow \max(V_i[k], V_m[k])$ for every k, then increments $V_i[i]$.

Comparison rules: $V \le W$ if $V[k] \le W[k]$ for all k; $V < W$ if $V \le W$ and $V \ne W$; $V \parallel W$ (concurrent) if neither $V \le W$ nor $W \le V$. Unlike Lamport clocks, vector clocks **characterize** causality

exactly: $a \rightarrow b$ if and only if $V(a) < V(b)$.

Figure 24.11 — Vector clocks on the same trace.



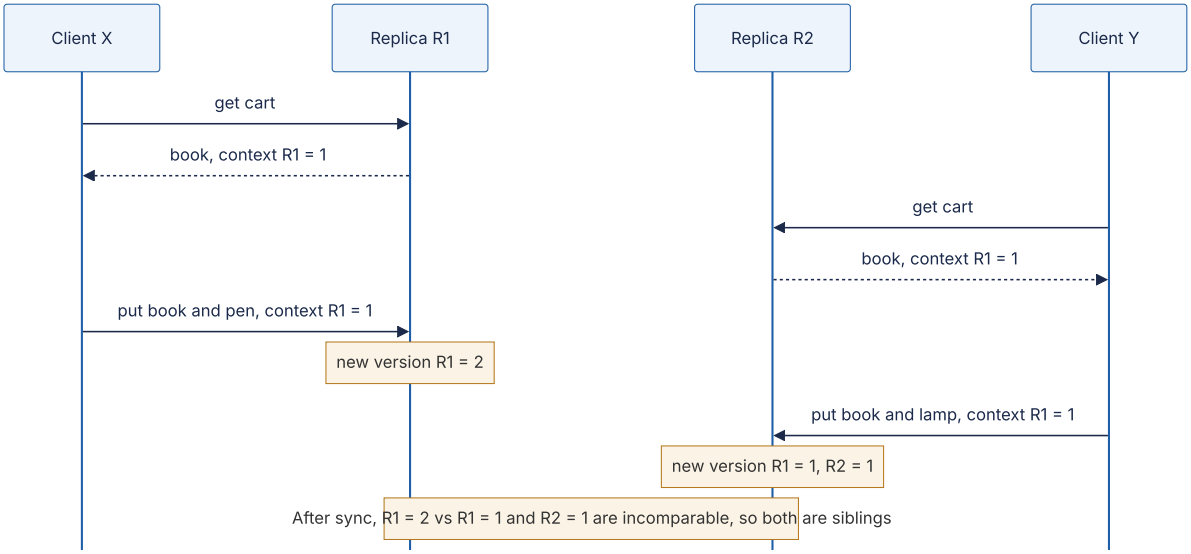
Reading the diagram. Entries are [P1, P2, P3]; each event increments its own entry, and each receipt first takes the element-wise maximum. Asynchronous messages now carry n integers. Concurrency is visible: `c` = [0,1,0] and `b` = [2,0,0] are incomparable, while `b` ≤ `g` = [2,3,2] entry by entry, so `b` → `g`. The scaling limit is vector size, which grows with the number of processes.

24.11.2 Version vectors

A **version vector** applies the same idea to *replicas of a data item* rather than to events in processes [Parker et al. 1983]. Each replica that accepts writes for a key has an entry counting the updates it has issued to that key. When two versions of a key have incomparable version vectors, they were written concurrently and are kept as **siblings** for the application (or a merge function) to reconcile; when one dominates, it replaces the other. The client passes the version vector it read (its **context**) with each write, so the store knows which versions the client had seen. This section introduces the mechanism only;

how replicated stores use siblings in practice (the Dynamo cart walk-through, Riak's causal contexts, merge functions and their pitfalls) is in §27.14.2.

Figure 24.12 — Version vectors detect a concurrent shopping-cart update.



Reading the diagram. Both clients read context {R1: 1} and write synchronously through different replicas, which later synchronize asynchronously. Neither {R1: 2} nor {R1: 1, R2: 1} dominates, so both values are kept as siblings; the next reader gets both and writes back a merge. The storage boundary is each replica's copy. Unlike LWW (Figure 24.9), nothing is silently lost, at the cost of application merging.

24.11.3 Code example

```
        """Vector clocks: comparison, the Figure 24.11 trace, and sibling detection with
version vectors."""


        def compare(v, w):
            """Return 'equal', 'before' (v happened before w), 'after' or 'concur-
rent'."""
            keys = set(v) | set(w)
            le = all(v.get(k, 0) <= w.get(k, 0) for k in keys)
            ge = all(v.get(k, 0) >= w.get(k, 0) for k in keys)
            if le and ge:
                return "equal"
            if le:
                return "before"
            if ge:
                return "after"
            return "concurrent"


        def merge(v, w):
            return {k: max(v.get(k, 0), w.get(k, 0)) for k in sorted(set(v) | set(w))}


        class VCProcess:
            def __init__(self, pid):
                self.pid, self.vc, self.events = pid, {}, {}

            def _tick(self, name):
                self.vc = dict(self.vc)
                self.vc[self.pid] = self.vc.get(self.pid, 0) + 1   # increment own entry
                self.events[name] = self.vc
                return self.vc

            def local(self, name):
                return self._tick(name)

            def send(self, name):
                return self._tick(name)                # the message carries a copy of
vc

            def receive(self, name, msg_vc):
                self.vc = merge(self.vc, msg_vc)       # element-wise maximum, then in-
crement
                return self._tick(name)


        class VersionedStore:
            """One key replicated with version vectors keyed by replica id. Concurrent
            writes are kept as siblings instead of being silently overwritten."""

            def __init__(self):
                self.siblings = []                     # list of (version_vector,
value)
```

```
            def get(self):
                context = {}
                for vv, _ in self.siblings:
                    context = merge(context, vv)
                return [val for _, val in self.siblings], context

            def put(self, replica, value, context):
                new_vv = dict(context)
                counter = max((vv.get(replica, 0) for vv, _ in self.siblings),
default=0)
                new_vv[replica] = counter + 1           # replica issues its next
counter value
                # Keep only siblings that the new write did not causally overwrite.
                self.siblings = [(vv, val) for vv, val in self.siblings
                                 if compare(vv, new_vv) == "concurrent"]
                self.siblings.append((new_vv, value))


        def fmt(v):
            return "[" + ",".join(str(v.get(p, 0)) for p in ("P1", "P2", "P3")) + "]"


        if __name__ == "__main__":
            p1, p2, p3 = VCProcess("P1"), VCProcess("P2"), VCProcess("P3")
            p1.local("a"); m1 = p1.send("b")
            p2.local("c"); p2.receive("d", m1); m2 = p2.send("e")
            p3.local("f"); p3.receive("g", m2); m3 = p3.send("h")
            p1.receive("i", m3)
            ev = {**p1.events, **p2.events, **p3.events}
            for name in "abcdefghi":
                print(name, fmt(ev[name]))
            for x, y in [("b", "g"), ("c", "b"), ("f", "e"), ("a", "i")]:
                print(f"{x} vs {y}: {compare(ev[x], ev[y])}")

            store = VersionedStore()
            store.put("R1", "cart=[book]", {})
            _, ctx = store.get()                            # both clients read the same
version
            store.put("R1", "cart=[book, pen]", ctx)        # client 1 writes via replica
R1
            store.put("R2", "cart=[book, lamp]", ctx)       # client 2 writes via R2 con-
currently
            values, ctx = store.get()
            print("siblings:", values, "context:", ctx)
            store.put("R1", "cart=[book, pen, lamp]", ctx) # client merges and writes
back
            print("after merge:", store.get())

            # Pitfall: two clients write concurrently through the SAME replica.
            store = VersionedStore()
            store.put("R1", "v0", {})
            _, ctx = store.get()
            store.put("R1", "from client X", ctx)           # gets {R1: 2}
            store.put("R1", "from client Y", ctx)           # gets {R1: 3}, which domi-
nates {R1: 2}
            print("same-replica concurrent writes:", store.get()[0])
```

Output:

```
a [1,0,0]
b [2,0,0]
c [0,1,0]
d [2,2,0]
e [2,3,0]
f [0,0,1]
g [2,3,2]
h [2,3,3]
i [3,3,3]
b vs g: before
c vs b: concurrent
f vs e: concurrent
a vs i: before
siblings: ['cart=[book, pen]', 'cart=[book, lamp]'] context: {'R1': 2, 'R2': 1}
after merge: (['cart=[book, pen, lamp]'], {'R1': 3, 'R2': 1})
same-replica concurrent writes: ['from client Y']
```

What to notice:

- `compare` treats missing entries as 0, so vectors can be sparse maps rather than fixed arrays.
- The merged write carries the merged context {R1: 2, R2: 1}, so its new vector {R1: 3, R2: 1} dominates both siblings and replaces them.
- The last case is a real flaw of per-replica version vectors: two *concurrent* writes through the same replica get {R1: 2} and {R1: 3}, the second appears to dominate the first, and client X's write is lost. Dotted version vectors (§24.11.5) fix this.

24.11.4 Size growth

FORMULA

Vector clock metadata size

$$S = n(b_{\text{id}} + b_{\text{ctr}}) \qquad M = K \cdot S$$

Symbol	Meaning	Units
n	Number of entries (processes, replicas or clients that have written)	count
b_{id}	Bytes to identify an entry's owner	bytes
b_{ctr}	Bytes per counter	bytes
S	Metadata per version	bytes
K	Number of keys stored	count
M	Total version metadata	bytes

WORKED EXAMPLE

Worked example — Server IDs versus client IDs.

1. Entries keyed by replica: $n = 3$, 8-byte IDs and 8-byte counters. $S = 3 \times 16 = 48$ bytes. For $K = 10^9$ keys, $M = 48 \times 10^9$ bytes = 48 GB, acceptable next to the data.
2. Entries keyed by *client*, for a popular key written by 10,000 distinct users: $S = 10,000 \times 16 = 160,000$ bytes = 160 KB of metadata on one key, transferred on every read and write.
3. Over a fleet of 1,000 processes tracking event causality: $S = 1,000 \times 16 = 16$ KB on *every message*.

Interpretation: size grows with the number of writers, and Charron-Bost proved no smaller timestamp can characterize causality in general [Charron-Bost 1991]. Engineering implication: key vectors by a small, stable set of replicas; if you must prune entries, accept occasional false conflicts.

INDUSTRY EXAMPLE

Amazon Dynamo. Dynamo tracked versions of shopping-cart objects with vector clocks of (node, counter) pairs, returned divergent versions to the client for semantic reconciliation, and truncated the vector by removing the oldest pair when it exceeded a threshold (about 10 pairs), accepting the small risk of losing causality information [DeCandia et al. 2007]. *Evidence: Primary* — [DeCandia et al. 2007]

24.11.5 Dotted version vectors

Dotted version vectors (DVVs) fix the same-replica flaw [Preguiça et al. 2010]. A DVV pairs the writer's causal context with a **dot**, the (replica, counter) pair naming *this* write. Writes through R1 with dots (R1, 2) and (R1, 3) are recognized as concurrent because neither context contains the other's dot. DVVs stay bounded by the replica count while avoiding lost updates and client-keyed "sibling explosion"; Riak adopted them in version 2.0 [Riak Docs: Dotted Version Vectors].

Table 24.10 — Logical clocks compared.

Clock	Size	Detects concurrency?	Close to real time?	Typical use
Lamport	1 integer	No	No	Total order, terms, fencing tokens
Vector clock	*n* entries (processes)	Yes	No	Causality tracking, debugging, causal broadcast
Version vector	*n* entries (replicas) per key	Yes	No	Conflict detection in replicated stores
Dotted version vector	Replica entries plus one dot	Yes, including same-replica writes	No	Riak-style key-value stores
Hybrid logical clock	64 bits	No	Yes, within skew	Timestamps for MVCC and snapshot reads

24.12 Hybrid logical clocks

24.12.1 Why combine physical and logical time

Multi-version databases need timestamps that (1) respect causality, (2) stay close to real time so "read as of 10:00" means something, and (3) fit in 64 bits. Lamport clocks miss (2); physical clocks miss (1). Kulkarni et al.'s **hybrid logical clock** (HLC) provides all three [Kulkarni et al. 2014].

DEFINITION

Definition — Hybrid logical clock. A timestamp (l, c) where l is the largest physical clock value the node has seen (from its own clock or from any message) and c is a logical counter that orders events sharing the same l. Timestamps compare lexicographically: first by l, then by c.

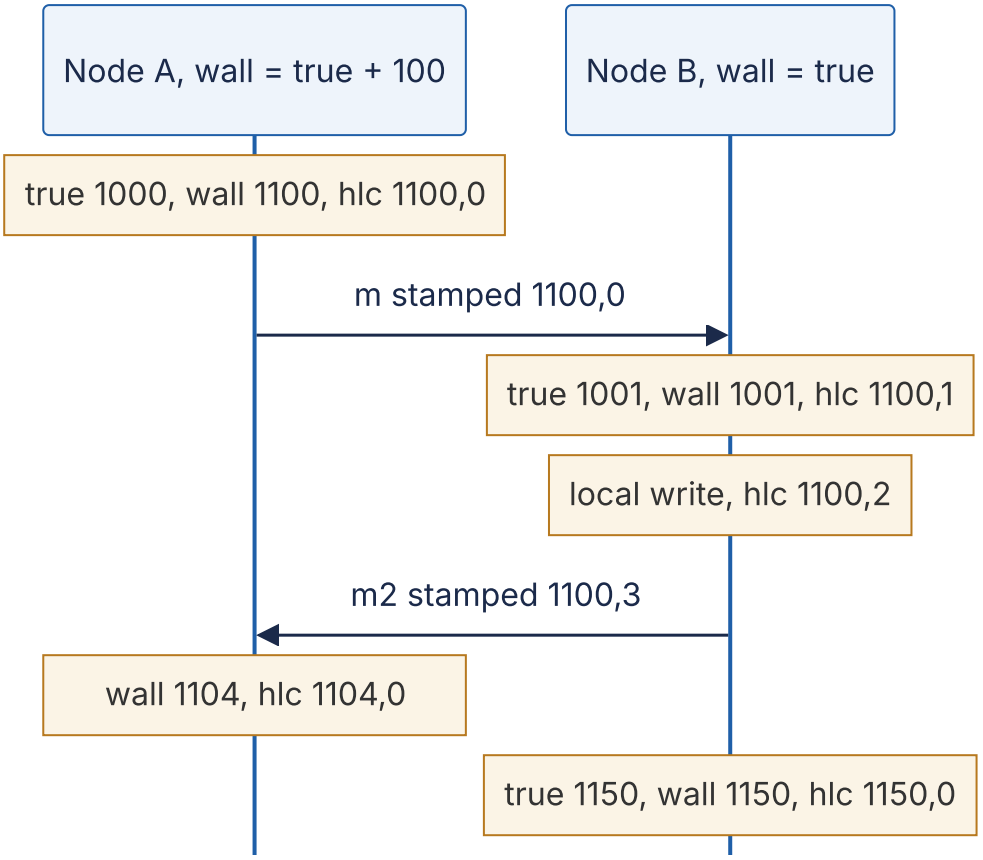
24.12.2 The algorithm

For a local or send event at a node with physical clock pt: set $l' \leftarrow l$, $l \leftarrow \max(l', pt)$; if $l = l'$ then $c \leftarrow c + 1$, else $c \leftarrow 0$.

For a receive of a message stamped (l_m, c_m): set $l \leftarrow \max(l', l_m, pt)$; then $c \leftarrow \max(c, c_m) + 1$ if l equals both l' and l_m; $c \leftarrow c + 1$ if $l = l'$ only; $c \leftarrow c_m + 1$ if $l = l_m$ only; otherwise $c \leftarrow 0$.

The paper proves that $e \rightarrow f$ implies $\text{hlc}(e) < \text{hlc}(f)$, that $l - pt \le \varepsilon$ (the clock skew) because l is always some node's clock reading, and that c stays small, so 48 bits of l plus 16 bits of c fit a 64-bit time field.

Figure 24.13 — HLC keeps causal order when node A's clock runs 100 ms fast.



Reading the diagram. Times are in milliseconds. A's message reaches B when B's wall clock reads 1001, *before* the send stamp 1100; physical timestamps would invert the order. B's HLC adopts $l = 1100$ and counts with c, ordering the receipt and B's next write after the send. Messages are asynchronous; the clock has no storage boundary. At 1150, l returns to physical time. The failure point is unbounded skew: one wildly fast clock drags every HLC forward, so production systems bound and monitor offsets.

24.12.3 Code example

```
        """Hybrid logical clock (Kulkarni et al. 2014) on two nodes whose clocks dis-
agree by 100 ms."""


        class HLC:
            def __init__(self, physical_clock):
                self.pt = physical_clock      # function returning this node's wall
clock, in ms
                self.l, self.c = 0, 0         # l: largest physical time seen; c: logi-
cal counter

            def now(self):
                """Timestamp a local or send event."""
                old_l = self.l
                self.l = max(old_l, self.pt())
                self.c = self.c + 1 if self.l == old_l else 0
                return (self.l, self.c)

            def update(self, msg):
                """Timestamp a receive event for a message stamped msg = (l_m, c_m)."""
                l_m, c_m = msg
                old_l = self.l
                self.l = max(old_l, l_m, self.pt())
                if self.l == old_l == l_m:
                    self.c = max(self.c, c_m) + 1
                elif self.l == old_l:
                    self.c += 1
                elif self.l == l_m:
                    self.c = c_m + 1
                else:
                    self.c = 0
                return (self.l, self.c)


        def pack(ts):
            """48 bits of milliseconds plus a 16-bit counter fit in one 64-bit inte-
ger."""
            l, c = ts
            assert c < 2**16
            return (l << 16) | c


        if __name__ == "__main__":
            true_ms = 1000
            a = HLC(lambda: true_ms + 100)    # node A's clock runs 100 ms fast
            b = HLC(lambda: true_ms)          # node B's clock is correct

            def show(event, node, ts, wall):
                print(f"true={true_ms} {event:<15} {node}: wall={wall}  hlc={ts}")

            ts = a.now()
            show("A sends m", "A", ts, true_ms + 100)
            true_ms += 1
            ts = b.update(ts)
```

```
            show("B receives m", "B", ts, true_ms)
            true_ms += 1
            ts = b.now()
            show("B local write", "B", ts, true_ms)
            true_ms += 1
            ts = b.now()
            show("B sends m2", "B", ts, true_ms)
            true_ms += 1
            ts = a.update(ts)
            show("A receives m2", "A", ts, true_ms + 100)
            true_ms = 1150
            ts = b.now()
            show("B local", "B", ts, true_ms)
            print("packed:", hex(pack(ts)), "->", ts == (pack(ts) >> 16, pack(ts) &
0xFFFF))
```

Output:

```
true=1000 A sends m       A: wall=1100  hlc=(1100, 0)
true=1001 B receives m    B: wall=1001  hlc=(1100, 1)
true=1002 B local write   B: wall=1002  hlc=(1100, 2)
true=1003 B sends m2      B: wall=1003  hlc=(1100, 3)
true=1004 A receives m2   A: wall=1104  hlc=(1104, 0)
true=1150 B local         B: wall=1150  hlc=(1150, 0)
packed: 0x47e0000 -> True
```

What to notice:

- `update` is the only place causality enters: taking the maximum with l_m guarantees the receipt is stamped after the send.
- l never runs ahead of the fastest physical clock (1100 here), so HLC timestamps stay within the skew of real time.
- Like a Lamport clock, an HLC is a scalar and cannot detect concurrency.

24.12.4 HLC in production: uncertainty intervals

HLC orders causally related events, but two *unrelated* transactions on different nodes are ordered only up to the clock skew. CockroachDB, which uses HLC timestamps, handles this with a configured **maximum clock offset** (500 ms by default, as of 2026) and an **uncertainty interval** [Cockroach Labs Docs; Kimball 2016].

FORMULA

Read uncertainty interval

$$t_r < t_w \le t_r + \varepsilon_{\max} \;\Rightarrow\; \text{restart the read at } t_w$$

Symbol	Meaning	Units
t_r	The reading transaction's HLC timestamp	s
t_w	Commit timestamp of a value found by the read	s
$\varepsilon_{\max}$	Configured maximum clock offset between any two nodes	s

Such a value *might* have committed before the read began in real time (the writer's clock may have been ahead), so the read moves up to t_w and retries.

WORKED EXAMPLE

Worked example — A read meets uncertain writes.

1. $\varepsilon_{\max} = 500$ ms; a read starts at $t_r = $ 10:00:00.000.
2. It finds a value at 10:00:00.300: inside $(t_r, t_r + 0.5]$, so it restarts at $t_r' = $ 10:00:00.300 and returns that value.
3. A value at 10:00:00.700 lies beyond the window: it definitely happened after the read began, and is ignored.

Interpretation: correctness depends on the configured bound holding, so CockroachDB nodes monitor their offset against peers and shut down rather than serve near the maximum. Engineering implication: better synchronization buys performance; halving $\varepsilon_{\max}$ halves the restart window. Spanner goes further with TrueTime (§28.8).

INDUSTRY EXAMPLE

MongoDB and CockroachDB. MongoDB's causal consistency sessions use a cluster-wide logical clock of the hybrid type, a (seconds, counter) pair gossiped on every message and signed to prevent malicious advancement [Tyulenev et al. 2019]. CockroachDB stamps every transaction with an HLC and uses the uncertainty interval above. *Evidence: Primary* — [Tyulenev et al. 2019; Cockroach Labs Docs]

24.13 The FLP impossibility result

24.13.1 The statement

DEFINITION

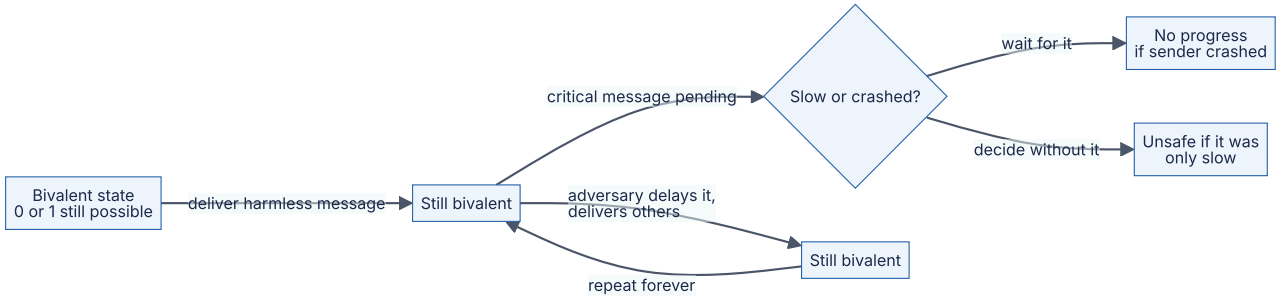
Definition — FLP impossibility. In a fully asynchronous message-passing system with reliable links, no *deterministic* protocol can solve **consensus** (all correct processes decide the same value, the value was proposed by some process, and every correct process eventually decides) if even one process may crash-stop [Fischer et al. 1985].

The assumptions are generous (no lost messages, one clean crash); only a delay bound is missing.

24.13.2 Intuition

Call a system state **bivalent** if both decisions (0 and 1) are still reachable, and **univalent** once the outcome is fixed. FLP shows that (1) some initial state is bivalent, and (2) from any bivalent state, an adversarial scheduler can always deliver messages in an order that keeps the state bivalent. The key step is that there is always some pending message whose delivery would decide the outcome; the scheduler delays it. Because the system is asynchronous, a delayed message is indistinguishable from a crashed sender, so the protocol cannot safely decide without it, and the scheduler can repeat the trick forever.

Figure 24.14 — The adversary keeps the protocol undecided.



Reading the diagram. Nodes are global states of the protocol; edges are message deliveries chosen by the adversarial scheduler, which is how the asynchronous network is modeled. When a critical message is pending, the protocol faces the decision box: waiting risks blocking forever if the sender crashed, deciding risks disagreement if the sender was merely slow. The adversary always picks the delivery that keeps the system bivalent. There are no storage boundaries; the failure point is the inability to tell slow from crashed, and adding processes does not help.

24.13.3 What FLP does and does not forbid

Table 24.11 — Reading FLP correctly.

FLP forbids	FLP does not forbid	How real systems escape
A deterministic protocol that *guarantees* termination in every asynchronous execution with one crash	Protocols that are always *safe* and terminate when the network behaves	**Partial synchrony** [Dwork et al. 1988]: Paxos and Raft never decide inconsistently and make progress after GST
	Protocols that terminate with probability 1	**Randomization** [Ben-Or 1983]: coin flips defeat the adversarial schedule
	Protocols that use an oracle to suspect failures	**Failure detectors** [Chandra and Toueg 1996]: an eventually accurate detector suffices
	Solving weaker problems	Eventual consistency and CRDTs avoid consensus altogether (§27.14)

In practice, a Raft cluster may fail to elect a leader during a storm of slow messages, stays *safe* throughout, and recovers once messages flow; its randomized election timeouts break symmetry, though they are not Ben-Or's algorithm [Ongaro and Ousterhout 2014].

COMMON MISTAKE

Mistake — "FLP proves consensus is impossible, so systems like etcd cannot work." FLP rules out a *guarantee of termination* in a purely asynchronous model. Real systems keep safety unconditionally and obtain liveness from timeouts, i.e., from the assumption that the network is eventually timely. FLP is also distinct from CAP (§25.8), which concerns availability of reads and writes during a partition.

24.14 The end-to-end argument

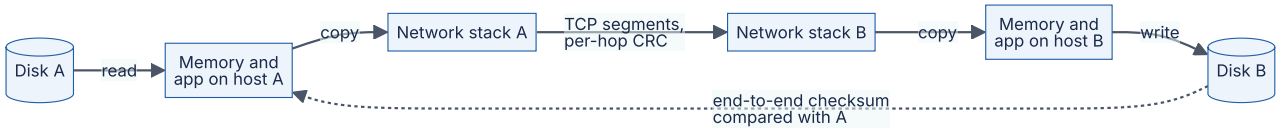
24.14.1 The principle

Saltzer, Reed and Clark's 1984 paper asks where in a layered system a function such as reliable delivery should be implemented [Saltzer et al. 1984]. Their answer, paraphrased: a function that can be implemented completely and correctly only with knowledge held by the application at the endpoints cannot be provided as a feature of the communication system alone. A lower layer may still provide a partial version, but only as a *performance optimization*.

Their example is **careful file transfer** from A's disk to B's disk. A reliable network still cannot prevent corruption in A's memory, a buffer copy or B's disk; only an application

check at both ends (compare checksums of the stored and original file, retry on mismatch) covers every step.

Figure 24.15 — Where errors can enter a file transfer, and which check covers them.



Reading the diagram. Data flows left to right between two storage boundaries (the disks) through several copies; every arrow is a failure point. TCP and link-layer checks protect only the middle arrow. The dotted arrow is the asynchronous end-to-end verification, which covers every step and scales at one checksum per file regardless of path length.

FORMULA

Undetected-corruption probability over many transfers

$$P_{\text{bad}} = 1 - (1 - q)^N$$

Symbol	Meaning	Units
q	Probability that one transfer suffers corruption no lower layer catches	dimensionless
N	Number of transfers	count
P_{bad}	Probability that at least one transfer is silently corrupted	dimensionless

WORKED EXAMPLE

Worked example — Rare errors at fleet scale.

1. Assume, for illustration, that one transfer in 10^{10} suffers corruption missed by TCP and link checks: $q = 10^{-10}$.
2. PhotoShare's replication moves $N = 10^9$ blocks per day: $P_{\text{bad}} = 1 - (1 - 10^{-10})^{10^9} \approx 1 - e^{-0.1} \approx 0.095$.
3. Over a year: $N = 3.65 \times 10^{11}$, $P_{\text{bad}} \approx 1 - e^{-36.5} \approx 1$.

Interpretation: a probability that looks negligible per transfer becomes a near-certainty at scale. Engineering implication: verify integrity end to end (checksums stored with the data and checked on read). Stone and Partridge measured that real Internet traffic contains errors the TCP checksum fails to catch at rates far above what checksum theory predicts [Stone and Partridge 2000].

INDUSTRY EXAMPLE

Amazon S3, July 2008. A single corrupted bit in messages of S3's internal gossip protocol, which carried no checksum of its own, spread incorrect server-state information through the system and caused a multi-hour outage. AWS's remedy included adding checksums to that internal state and its messages [AWS 2008]. *Evidence: Primary* — [AWS 2008]

24.14.2 Applying the argument

- **Delivery versus processing.** A TCP acknowledgment means the peer's kernel buffered the bytes (§7.7), not that the application processed them. A message broker should be acknowledged *after* processing (Chapters 30–31).
- **Exactly-once effects.** No network can deliver exactly once; the endpoint achieves *effectively-once* processing with at-least-once delivery plus deduplication by idempotency key (§22.15).
- **Security.** Encryption hop by hop leaves data exposed at every intermediary; end-to-end encryption does not (Chapter 10).
- **Where lower layers still help.** Wi-Fi retransmits frames locally because an end-to-end retransmission would be far slower: the permitted "performance optimization".

COMMON MISTAKE

Mistake — "The network (or storage system) is reliable, so the application needn't check." Each layer's guarantee covers only that layer. If the requirement is "the user's photo is stored intact and saved once", only the application can verify it.

24.15 Summary and revision notes

SUMMARY

- Distributed systems have **independent failures, no global clock and unreliable messages**; the **two generals problem** shows certainty of agreement over a lossy channel is impossible.
- Distribute for **scale, fault tolerance, latency or data residency**. Replication gives $A_k = 1 - (1 - a)^k$ under independence, but fleets fail daily: $P_{\text{any}} = 1 - (1 - p)^n$.
- Each of the **eight fallacies** maps to a class of outages.
- State a **system model**: timing (synchronous, partially synchronous, asynchronous) and faults (crash-stop, crash-recovery, omission, timing, Byzantine). Consensus needs $2f + 1$ nodes for crashes, $3f + 1$ for Byzantine faults.
- **Network partitions** are complete, partial or asymmetric, and occur daily in large fleets; partial ones cause many catastrophic bugs.
- A timeout means **unknown outcome**. **Gray failures** are differentially observable; detect them with what clients experience. Set timeouts from measured p99.9 plus pause margin.
- Quartz drifts (1 ppm = 86.4 ms/day); NTP estimates $\theta = [(T_2 - T_1) + (T_3 - T_4)]/2$ with error up to $\delta/2$; PTP with hardware timestamps reaches sub-microsecond. Use monotonic clocks for durations; smear leap seconds consistently.
- **Wall-clock LWW** silently discards causally later writes that fall within the skew window.
- **Lamport clocks**: $a \to b \Rightarrow C(a) < C(b)$, total order with tie-breaking, no concurrency detection. **Vector clocks** characterize causality exactly at $O(n)$ size; **version vectors** detect concurrent writes as siblings; **dotted version vectors** fix same-replica lost updates.
- **HLC** (l, c) is causal, within skew of real time, and 64 bits; real-time guarantees still need bounded skew (uncertainty intervals, TrueTime in §28.8).
- **FLP**: no deterministic asynchronous consensus guarantees termination with one crash; systems stay safe always and live under partial synchrony, randomization or failure detectors.
- **End-to-end argument**: integrity, deduplication and processing acknowledgments belong at the endpoints; lower layers only optimize.

24.16 Knowledge checkpoint

1. Name the three properties that make a system distributed.
2. Why can no finite protocol solve the two generals problem?
3. One server is 99.9% available. What is the availability of 3 independent replicas, and why is that number optimistic?

4. Give a concrete consequence of the fallacy “latency is zero”.
5. How many nodes does consensus need to tolerate 2 crash faults? 2 Byzantine faults?
6. What is the difference between a partial and an asymmetric partition?
7. Define gray failure and give one example.
8. A clock drifts at 20 ppm. How far off can it be after one hour?
9. NTP timestamps: $T_1 = 0$, $T_2 = 50$, $T_3 = 52$, $T_4 = 22$ ms. Compute offset, delay and error bound.
10. Under LWW, node A’s clock is 80 ms fast. Which writes through node B can A’s earlier writes override?
11. Lamport timestamps $C(a) = 3$, $C(b) = 5$. What can you conclude?
12. Compare [2,1,0] and [1,2,0]; then [1,1,0] and [2,1,3].
13. What problem do dotted version vectors solve?
14. Does FLP mean Raft can lose data? Explain.
15. State the end-to-end argument and give one application.

Answers

1. Independent (partial) failure, no shared clock, communication only by unreliable messages with unbounded delay.
2. The sender of the last message cannot know whether it arrived, so both must decide the same way without it; the message is then unnecessary, contradicting the assumption of a shortest protocol.
3. $1 - 0.001^3 = 1 - 10^{-9}$. Failures are correlated (shared power, network, software) and failover is not instant.
4. A chatty page with 100 sequential calls takes 50 ms at 0.5 ms per call in one region but about 7 s at 70 ms per call across regions.
5. $2f + 1 = 5$; $3f + 1 = 7$.
6. Partial: some pairs cannot communicate while others can (A and B both reach C but not each other). Asymmetric: one direction works and the other does not.
7. A failure seen by some observers but not others, e.g., a disk 100× slower that passes the heartbeat health check while application requests time out.
8. $20 \times 10^{-6} \times 3,600 = 0.072$ s $= 72$ ms.
9. $\delta = (22 - 0) - (52 - 50) = 20$ ms; $\theta = [(50 - 0) + (52 - 22)]/2 = 40$ ms (client behind by 40 ms); bound ± 10 ms.
10. Any B write within 80 ms after an A write to the same key.
11. Only that $b \not\rightarrow a$; a may have happened before b or they may be concurrent.
12. [2,1,0] and [1,2,0] are concurrent; [1,1,0] < [2,1,3], so the first happened before the second.
13. Concurrent writes through the same replica, which per-replica version vectors misorder as one dominating the other, losing a write.
14. No. Raft is always safe; FLP only means that in bad asynchronous periods it may fail to make progress (elect a leader or commit).

15. A function that needs endpoint knowledge must be implemented at the endpoints; lower layers can only optimize. Example: checksum the stored file end to end, or deduplicate by idempotency key.

24.17 Interview questions

Basic

Q1. Why is building a distributed system harder than a single-machine one? Answer. Three losses: partial failure (some parts fail, others continue, and nobody knows exactly which), no global clock (events on different machines cannot be ordered by time) and unreliable messaging (loss, delay, duplication). A timeout cannot distinguish a slow node from a dead one or tell whether a write happened, so designs need idempotency, quorums and logical ordering.

Q2. Can you use server timestamps to order events across machines? Answer. Only coarsely. NTP skew is milliseconds and occasionally seconds; clocks can step backwards. Events closer together than the skew may be misordered, which makes LWW lose writes. Use Lamport or hybrid logical clocks for causal order, vector clocks to detect concurrency, or a single sequencer; use bounded-uncertainty clocks when real-time order matters.

Intermediate

Q3. Design conflict handling for a shopping cart replicated across two regions. Answer. Avoid LWW, which drops items. Track versions with version vectors keyed by replica (or dotted version vectors), keep concurrent versions as siblings, and merge on read (union of adds); represent removals with tombstones or use an observed-remove set CRDT (§27.14) so deletions are not resurrected. Discuss metadata size, sibling limits and the user experience of merges.

Q4. A node passes health checks, yet its clients see 30% errors. What is happening and what would you change? Answer. A gray failure: the probe does not exercise the failing path (disk, dependency, thread pool). Make health checks exercise real dependencies, add client-side outlier detection that ejects nodes with abnormal error rates or latency, and alert on client-observed rather than self-reported health.

Advanced

Q5. Explain FLP and how Raft copes with it. Answer. In an asynchronous system no deterministic consensus protocol can guarantee termination if one process may crash, because a slow process is indistinguishable from a crashed one and an adversarial schedule

can keep the system undecided. Raft keeps safety unconditionally (majority quorums, term checks, persisted votes) and gets liveness from partial synchrony: timeouts detect leader failure, and randomized election timeouts break split votes. During a bad network period it may stall, but never diverges.

Q6. Your database uses HLC timestamps and a 500 ms maximum offset. Why do some reads restart, and how would you reduce restarts? Answer. A read at t_r that finds a value committed in $(t_r, t_r + 500\ \text{ms}]$ cannot tell whether it committed before the read in real time, so it moves its timestamp forward and retries. Reduce restarts by improving clock synchronization (PTP or a precise cloud time service) and lowering the configured maximum offset, by monitoring offsets so nodes with bad clocks are removed, and by reducing contention on hot keys.

24.18 Practical exercises

1. Extend the Lamport listing with Lamport's mutual-exclusion algorithm (request, reply, release) for three processes and check that the lock is granted in timestamp order.
2. Modify the version-vector store to use dotted version vectors and show that the "same-replica" case keeps both writes as siblings.
3. Simulate 10 nodes with random drift between −50 and +50 ppm and NTP corrections every 64 s with ±2 ms error. Plot the maximum pairwise skew and count LWW inversions for writes 10 ms apart.
4. Run `chronyc tracking` (or `ntpq -p`) on a Linux machine; identify the stratum, offset, root delay and root dispersion, and explain each in terms of §24.9.3.
5. Using `tc netem` on two containers, create a one-way partition and observe how an etcd or Redis Sentinel cluster behaves.

24.19 Design challenges

1. **Global event log.** PhotoShare wants a per-user activity feed assembled from events produced in three regions, ordered so that a reply never appears before the comment it answers. Choose clocks (Lamport, HLC, vector), state the system model, quantify metadata overhead per event, and describe behavior during an inter-region partition.
2. **Clock-safe leases.** Design a leader lease for a job scheduler whose nodes use NTP with up to 50 ms skew and may pause for 2 s during garbage collection. Size lease duration and renewal interval, and explain why fencing tokens (§26.13) are still needed.

24.20 References

1. Akkoyunlu, E. A., Ekanadham, K., Huber, R. V. *Some Constraints and Tradeoffs in the Design of Network Communications*. Proc. 5th ACM Symposium on Operating Systems Principles (SOSP), 1975.
2. Alquraan, A., Takruri, H., Alfatafta, M., Al-Kiswany, S. *An Analysis of Network-Partitioning Failures in Cloud Systems*. 13th USENIX OSDI, 2018. https://www.usenix.org/conference/osdi18/presentation/alquraan
3. Amazon Web Services. *Amazon S3 Availability Event: July 20, 2008*. AWS Service Health Dashboard post-mortem, 2008. (Cited as AWS 2008.)
4. Amazon Web Services. *Summary of the Amazon EC2 and Amazon RDS Service Disruption in the US East Region*. AWS, April 2011. (Cited as AWS 2011.)
5. Amazon Web Services. *Amazon Time Sync Service now supports microsecond-accurate time*. AWS What's New, November 2023; and *ClockBound*. https://github.com/aws/clock-bound (Cited as AWS 2023.)
6. Bailis, P., Kingsbury, K. *The Network is Reliable: An informal survey of real-world communications failures*. ACM Queue 12(7), 2014; also Communications of the ACM 57(9), 2014.
7. Ben-Or, M. *Another Advantage of Free Choice: Completely Asynchronous Agreement Protocols*. Proc. ACM PODC, 1983.
8. BIPM. *Resolution 4 of the 27th CGPM: On the use and future development of UTC*. 2022.
9. Cachin, C., Guerraoui, R., Rodrigues, L. *Introduction to Reliable and Secure Distributed Programming*, 2nd ed. Springer, 2011.
10. Chandra, T. D., Toueg, S. *Unreliable Failure Detectors for Reliable Distributed Systems*. Journal of the ACM 43(2), 1996.
11. Charron-Bost, B. *Concerning the Size of Logical Clocks in Distributed Systems*. Information Processing Letters 39(1), 1991.
12. Cockroach Labs. *Operational FAQs* and *cockroach start* (`--max-offset`). CockroachDB documentation, accessed 2026. https://www.cockroachlabs.com/docs/
13. Corbett, J. C., et al. *Spanner: Google's Globally-Distributed Database*. 10th USENIX OSDI, 2012.
14. Dean, J. *Designs, Lessons and Advice from Building Large Distributed Systems*. Keynote, LADIS workshop, 2009.
15. DeCandia, G., et al. *Dynamo: Amazon's Highly Available Key-value Store*. Proc. ACM SOSP, 2007.
16. Dwork, C., Lynch, N., Stockmeyer, L. *Consensus in the Presence of Partial Synchrony*. Journal of the ACM 35(2):288–323, 1988.
17. European Union. *Regulation (EU) 2016/679 (General Data Protection Regulation)*, Chapter V. Official Journal of the EU, 2016. (Cited as EU 2016.)
18. Fidge, C. J. *Timestamps in Message-Passing Systems That Preserve the Partial Ordering*. Proc. 11th Australian Computer Science Conference, 1988, pp. 56–66.

19. Fischer, M. J., Lynch, N. A., Paterson, M. S. *Impossibility of Distributed Consensus with One Faulty Process*. Journal of the ACM 32(2):374–382, 1985.
20. Gill, P., Jain, N., Nagappan, N. *Understanding Network Failures in Data Centers: Measurement, Analysis, and Implications*. Proc. ACM SIGCOMM, 2011.
21. Google. *Leap Smear*. Google Public NTP documentation, accessed 2026. https://developers.google.com/time/smear
22. Graham-Cumming, J. *How and why the leap second affected Cloudflare DNS*. Cloudflare Blog, January 2017.
23. Gray, J. *Notes on Data Base Operating Systems*. In *Operating Systems: An Advanced Course*, LNCS 60, Springer, 1978.
24. Gunawi, H. S., et al. *Fail-Slow at Scale: Evidence of Hardware Performance Faults in Large Production Systems*. 16th USENIX FAST, 2018.
25. Huang, P., Guo, C., Zhou, L., Lorch, J. R., Dang, Y., Chintalapati, M., Yao, R. *Gray Failure: The Achilles' Heel of Cloud-Scale Systems*. Proc. HotOS XVI, ACM, 2017.
26. IEEE. *IEEE Std 1588-2008: Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems*. 2008; revised as IEEE Std 1588-2019. (Cited as IEEE 2008; IEEE 2019.)
27. Kimball, S. *Living Without Atomic Clocks*. Cockroach Labs Blog, 2016.
28. Kleppmann, M. *Designing Data-Intensive Applications*, ch. 8 "The Trouble with Distributed Systems". O'Reilly, 2017.
29. Kulkarni, S. S., Demirbas, M., Madappa, D., Avva, B., Leone, M. *Logical Physical Clocks*. Proc. OPODIS 2014, LNCS 8878, Springer, 2014.
30. Lamport, L. *Time, Clocks, and the Ordering of Events in a Distributed System*. Communications of the ACM 21(7):558–565, 1978.
31. Lamport, L. Email message of 28 May 1987 defining a distributed system. Reproduced on Lamport's writings pages. https://lamport.azurewebsites.net/pubs/distributed-system.txt (Cited as Lamport 1987.)
32. Lamport, L., Shostak, R., Pease, M. *The Byzantine Generals Problem*. ACM TOPLAS 4(3):382–401, 1982.
33. Lianza, T., Snook, C. *A Byzantine failure in the real world*. Cloudflare Blog, 27 November 2020. https://blog.cloudflare.com/a-byzantine-failure-in-the-real-world/
34. Mattern, F. *Virtual Time and Global States of Distributed Systems*. In Cosnard, M., et al. (eds.), *Parallel and Distributed Algorithms*, North-Holland, 1989, pp. 215–226.
35. Mills, D., Martin, J., Burbank, J., Kasch, W. *Network Time Protocol Version 4: Protocol and Algorithms Specification*. RFC 5905, IETF, 2010.
36. Obleukhov, O., Byagowi, A. *How Precision Time Protocol is being deployed at Meta*. Engineering at Meta blog, 2022.
37. Ongaro, D., Ousterhout, J. *In Search of an Understandable Consensus Algorithm*. USENIX ATC, 2014.
38. Parker, D. S., et al. *Detection of Mutual Inconsistency in Distributed Systems*. IEEE Transactions on Software Engineering SE-9(3), 1983.

39. Preguiça, N., Baquero, C., Almeida, P. S., Fonte, V., Gonçalves, R. *Dotted Version Vectors: Logical Clocks for Optimistic Replication*. arXiv preprint, 2010.
40. Riak. *Dotted Version Vectors*. Riak KV documentation, accessed 2026. (Cited as Riak Docs: Dotted Version Vectors.)
41. Rotem-Gal-Oz, A. *Fallacies of Distributed Computing Explained*. White paper, 2006.
42. Saltzer, J. H., Reed, D. P., Clark, D. D. *End-to-End Arguments in System Design*. ACM Transactions on Computer Systems 2(4):277–288, 1984.
43. Schlichting, R. D., Schneider, F. B. *Fail-Stop Processors: An Approach to Designing Fault-Tolerant Computing Systems*. ACM Transactions on Computer Systems 1(3), 1983.
44. Stone, J., Partridge, C. *When the CRC and TCP Checksum Disagree*. Proc. ACM SIGCOMM, 2000.
45. Tyulenev, M., et al. *Implementation of Cluster-wide Logical Clock and Causal Consistency in MongoDB*. Proc. ACM SIGMOD, 2019.
46. van Steen, M., Tanenbaum, A. S. *Distributed Systems*, 3rd ed. distributed-systems.net, 2017.

24.21 Open verification items

- Lamport’s 1987 email: exact URL on Lamport’s site (the `distributed-system.txt` path was not confirmed).
- Gill et al. 2011 figures (5.2 device and 40.8 link failures per day; ~5-minute median repair; ~43% redundancy effectiveness) and Dean 2009 “first year of a cluster” counts, quoted from secondary summaries.
- Alquraan et al. 2018: exact share of catastrophic failures and of partial-partition failures.
- Attribution of the first four fallacies to Joy and Lyon, and the year Gosling added the eighth.
- CockroachDB default maximum offset (500 ms) and self-termination rule as of the current release.
- Meta PTP blog authors/date; AWS microsecond time announcement date; Riak 2.0 DVV adoption; Gunawi et al. case count.
- URLs for the AWS 2008 and 2011 post-mortems.

Chapter 25 — Consistency and Availability

> **Level:** 3 · **Track:** A (Traditional) · **Estimated reading time:** 6 h

A customer changes her shipping address and presses “Place order”. The order service reads the address from a replica in another zone that has not yet received the change, and the parcel goes to her old flat. Nothing crashed, no disk failed and every component did exactly what its documentation promised. The bug is a mismatch between the guarantee the code assumed (“I will read what was just written”) and the guarantee the database gave (“replicas converge eventually”).

This chapter gives you a precise vocabulary for those guarantees. A **consistency model** is a contract between a data store and its clients: it says which values a read is allowed to return, given the writes that happened before or alongside it. Strong models make replicated data behave like a single copy and cost latency and availability. Weak models are fast and always answer, but let clients observe anomalies. Between the two lie sequential consistency, causal consistency and the session guarantees, each of which forbids a specific set of anomalies at a specific price.

Transaction isolation (§11.13) is a different axis; §25.3.1 and §25.7.5 keep the two apart. Three runnable Python programs accompany the chapter: a brute-force linearizability checker, a version-vector implementation of session guarantees, and a Monte Carlo simulation of probabilistically bounded staleness.

25.1 Learning objectives

After this chapter you should be able to:

- Define linearizability through real-time order and a legal sequential history, locate linearization points, and decide by hand or with a checker whether a register history is linearizable.
- Explain why a linearizable read or write costs at least one round trip to a quorum, and compute that latency for a given replica placement.
- Distinguish sequential consistency from linearizability with a concrete history, and explain why sequential consistency is not compositional.
- Define causal consistency, give an anomaly it prevents, and describe how COPS tracks and checks dependencies.
- State the four session guarantees of Terry et al., draw the anomaly each prevents and implement each with sticky routing, version tokens or log positions.

- Quantify eventual consistency with probabilistically bounded staleness (PBS) using a closed form and a simulation.
- State the CAP theorem as Gilbert and Lynch proved it, define its terms formally, and refute “pick two” and “CA system” claims.
- Classify real systems with PACELC, including configuration caveats, and choose consistency settings in Cassandra, DynamoDB, Cosmos DB, MongoDB and Spanner.
- Describe how Jepsen records histories, injects faults and checks them with Knossos-style and Elle-style checkers, and summarize published findings accurately.

25.2 Prerequisites

- Replication, replication lag and the two lag anomalies — §14.3, §14.4 (this chapter formalizes them).
- Transactions, isolation levels and anomalies — §11.12, §11.13 (to keep isolation and consistency separate).
- System models, network partitions and partial failures — §24.6–§24.8 (the CAP proof is a statement about an asynchronous network that can lose messages).
- Lamport clocks, the happens-before relation and vector clocks — §24.10, §24.11 (causal consistency is defined by happens-before; session guarantees are implemented with version vectors).
- Clock skew and NTP — §24.9 (why “latest” cannot simply be decided by wall-clock timestamps).
- Percentiles and latency distributions — §4.8 (PBS is a statement about latency distributions).

25.3 Linearizability

25.3.1 Histories, objects and two kinds of guarantee

Before defining any model you need the material every model is stated over: a **history**.

DEFINITION

Definition — Operation, history. An **operation** is one call by a client on a shared object, such as `write(x, 1)` or `read(x)`. It has an **invocation** event (the client sends the request, at real time t_{inv}) and, if the client hears back, a **response** event (at t_{resp}, carrying the return value). A **history** is the sequence of invocation and response events observed in one execution. An operation without a response is **pending**: it may or may not have taken effect.

Two operations are **concurrent** if their intervals $[t_{\text{inv}}, t_{\text{resp}}]$ overlap. If operation *a* responds before operation *b* is invoked ($a.t_{\text{resp}} < b.t_{\text{inv}}$), then *a* **precedes** *b* in **real-time order**. Notice that real time is the time of an imaginary global observer; no node needs to know it, but the definitions use it.

A consistency model is a set of allowed histories; a store "provides" it if every history it can produce, under every allowed fault, is in the set [Viotti & Vukolić 2016]. This turns vague adjectives ("strongly consistent") into a test you can run.

Models come in two families that are often confused.

Table 25.1 — Single-object consistency models versus transaction isolation models.

	Single-object (non-transactional) consistency	Transaction isolation
Unit of reasoning	One operation on one object (register, key, queue)	A transaction: a group of operations on many objects
Question answered	Which value may a read return, given writes by others and real time?	Which interleavings of concurrent transactions are allowed?
Classic strongest model	Linearizability [Herlihy & Wing 1990]	Serializability [Papadimitriou 1979]
Weaker models	Sequential, causal, session guarantees, eventual	Snapshot isolation, repeatable read, read committed
Mentions real time?	Linearizability does; sequential and causal do not	Plain serializability does not; strict serializability does
Typical anomalies	Stale read, reading back in time, missing own write	Dirty read, lost update, write skew (§11.13)
Where in this book	This chapter	§11.13–§11.15; distributed transactions Ch 28

The two meet at **strict serializability**: transactions are serializable *and* the serial order respects real time. Strict serializability is to transactions what linearizability is to single operations; Spanner calls it **external consistency** [Corbett et al. 2012]. A database can be serializable without being linearizable (a serializable replica that serves a consistent but stale snapshot), and linearizable without being serializable (a key-value store whose single-key operations are linearizable but which has no multi-key transactions). Keep the two axes apart in interviews and designs: "Which isolation level?" and "Which consistency model?" are separate questions.

25.3.2 The definition

DEFINITION

Definition — Linearizability. A history is **linearizable** if each operation can be assigned a single instant, its **linearization point**, somewhere between its invocation and its response, such that (1) the operations, ordered by these points, form a **legal sequential history** of the object (every read returns the value of the most recent preceding write), and (2) therefore whenever *a* precedes *b* in real time, *a* comes before *b* in that order. Pending operations may be either included (with a linearization point after their invocation) or dropped. An object is linearizable if all its histories are [Herlihy & Wing 1990].

First-principles explanation

Imagine a single notebook on a table in a room. People walk in, write a value or read the latest value, and walk out. Each visit takes a moment, and two visitors can be in the room at once, but the notebook itself is changed at one instant. Anyone who arrives after you left must see your entry or a later one. Linearizability is the promise that a replicated system behaves *exactly* like that single notebook, even though behind the scenes there are five copies on three continents. Kleppmann calls this the **single-copy illusion** [Kleppmann 2017].

Consequently, once any client has read a new value, no client can afterwards read an older one, and "afterwards" is real time as observed by anybody, including a colleague you phone to say "I've just paid; can you see it?"

Why does this exist?

Herlihy and Wing introduced linearizability in 1990 as a correctness condition for concurrent objects (queues, counters, registers) in shared-memory multiprocessors [Herlihy & Wing 1990]. Distributed systems adopted it because many problems require a single, agreed, up-to-date value: a lock or lease must have one holder; a username must be unique; a bank balance must not be spent twice; a leader must be the only leader. In each case a stale read leads to two parties acting on contradictory information. Linearizability is also what makes higher-level protocols easy to reason about: a linearizable compare-and-set register is enough to build leader election and fencing tokens (Chapter 26).

Herlihy and Wing also proved that linearizability is **local** (compositional): a history over many objects is linearizable if and only if each object's sub-history is. Independently linearizable partitions therefore compose into a linearizable store. Section 25.4 shows that sequential consistency lacks this property.

25.3.3 Linearization points and checking a history

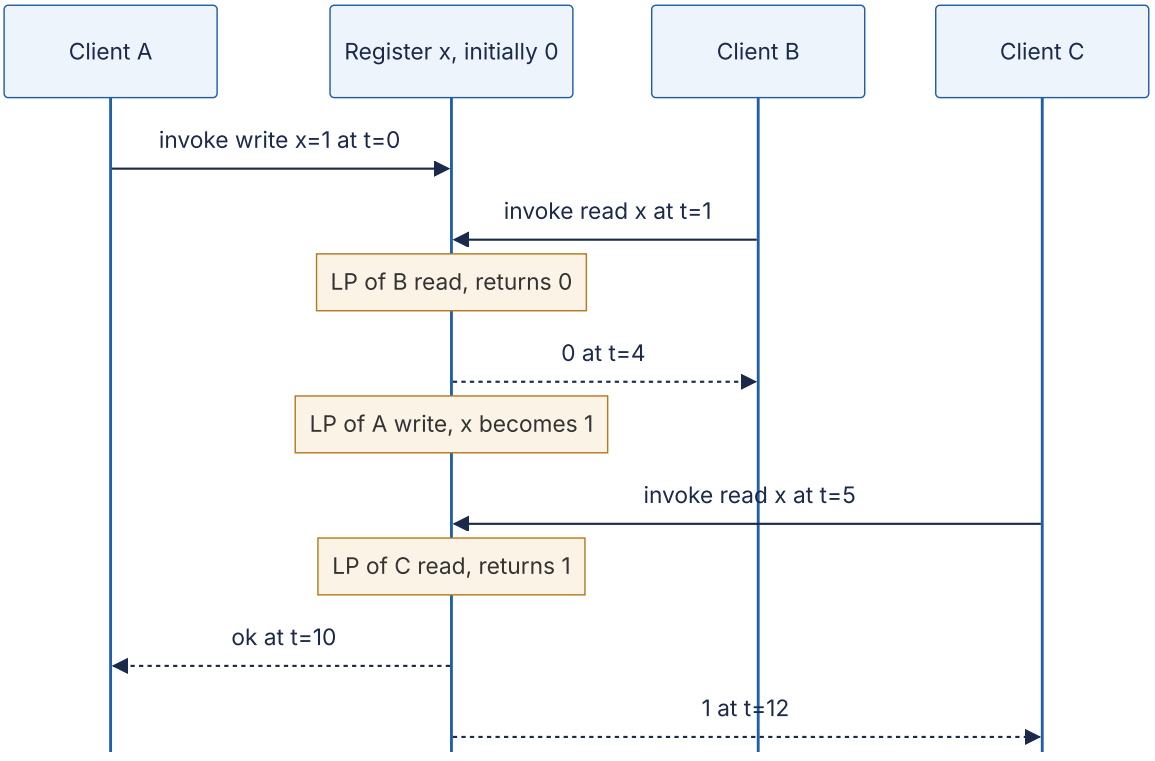
How it works internally

To check a history by hand:

1. Draw each operation as an interval on a timeline, one row per client.
2. For every operation, you may place its linearization point anywhere inside its interval.
3. Look for a placement in which reads return the latest preceding write. If one exists, the history is linearizable; the placement is a **witness**.
4. If every placement fails, the history is not linearizable.

The search is exponential in the worst case: Gibbons and Korach showed that checking whether a history of a read/write register is linearizable is NP-complete in general [Gibbons & Korach 1997]. Practical checkers prune aggressively and handle histories of thousands of operations when concurrency is modest (§25.11).

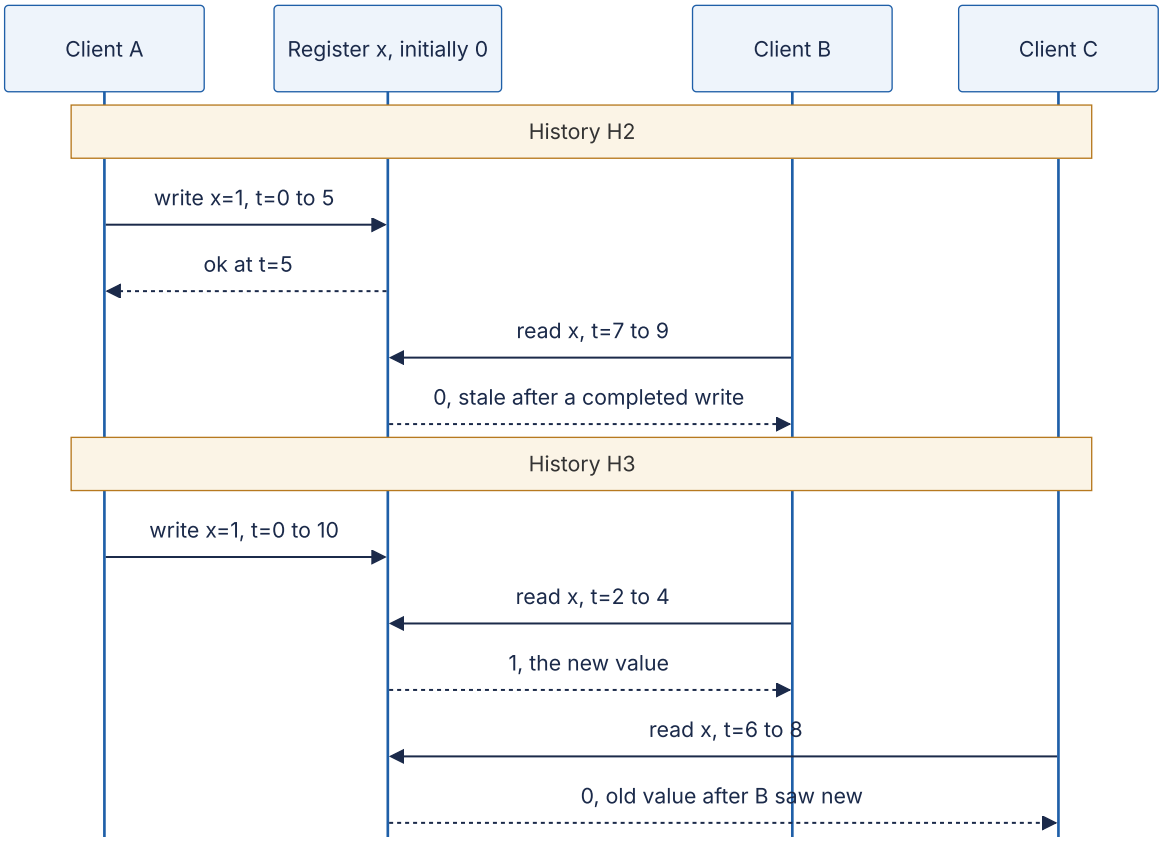
Figure 25.1 — A linearizable history and its linearization points.



Reading the diagram. Time runs downward; solid arrows are invocations and dashed arrows responses, all synchronous request/response calls on one logical register. Client A's write lasts from t = 0 to t = 10, so it is concurrent with both reads. "LP" marks a chosen linearization point. B's read (t = 1 to 4) can be linearized before the write takes effect, so returning the old value 0 is legal; C's read (t = 5 to 12) is linearized after the write, so returning 1 is legal. The register behind the arrows may be five replicas, but the

history is indistinguishable from one copy. The failure point to watch is a read that starts *after* a completed write or *after* another read that saw the new value; Figure 25.2 shows both.

Figure 25.2 — Two non-linearizable histories: a stale read, and a new-then-old inversion.



Reading the diagram. In H2 the write has *completed* (acknowledged at t = 5) before B's read begins at t = 7, so real-time order forces the write before the read; a read returning 0 has no legal placement. This is the classic asynchronous-replica bug: the write was acknowledged by the primary and the read went to a lagging replica. H3 is subtler: the write is still in progress, so either value is individually allowed, but once B has returned 1 (its linearization point is after the write's), C's read, which begins after B finished, must also see 1. Returning 0 means the system "un-wrote" a value someone already observed. H3 happens in leaderless quorum stores when a write has reached one replica but not a quorum, and two readers contact different replica sets (§25.3.4).

Code example

The program below is a brute-force checker for histories of read/write registers. It tries to build a legal sequential order one operation at a time, allowing an operation to go next only if no unplaced operation must precede it. With `real_time=True` the precedence rule is real-time order (linearizability); with `real_time=False` it is only each client's program or-

der (sequential consistency, §25.4). Memoizing on the set of placed operations and the register state is the core idea of the Wing–Gong–Lowe family of checkers used by Knossos and Porcupine [Wing & Gong 1993; Lowe 2017].

```
        """Brute-force checker: is a history of register operations linearizable,
        or at least sequentially consistent? Each operation is
            (process, kind, key, value, invoke_time, response_time)
        kind is "w" (write value) or "r" (read that returned value).
        response_time None means the client never heard back (the op may or may not
        have taken effect). All registers start at 0."""
        from functools import lru_cache

        INF = float("inf")

        def check(history, real_time=True):
            ops = [(p, k, key, v, inv, INF if resp is None else resp)
                   for p, k, key, v, inv, resp in history]
            n = len(ops)
            done = frozenset(i for i in range(n) if ops[i][5] != INF)

            def must_precede(a, b):
                """True if op a has to be ordered before op b."""
                if real_time:                       # linearizability: real-time order
                    return ops[a][5] < ops[b][4]
                # sequential consistency: only each process's own program order
                return ops[a][0] == ops[b][0] and ops[a][4] < ops[b][4]

            @lru_cache(maxsize=None)
            def search(placed, state):              # placed: bitmask; state: sorted
(key, value) pairs
                if all(placed >> i & 1 for i in done):
                    return ()                       # every completed op is placed
                regs = dict(state)
                for i in range(n):
                    if placed >> i & 1:
                        continue
                    # i can go next only if no unplaced op must precede it
                    if any(not placed >> j & 1 and j != i and must_precede(j, i)
                           for j in range(n)):
                        continue
                    p, kind, key, v, _, _ = ops[i]
                    if kind == "r" and regs.get(key, 0) != v:
                        continue                    # read would return the wrong value
                    new = dict(regs)
                    if kind == "w":
                        new[key] = v
                    rest = search(placed | 1 << i, tuple(sorted(new.items())))
                    if rest is not None:
                        return (i,) + rest
                return None

            order = search(0, ())
            return None if order is None else [ops[i][:4] for i in order]

        def describe(name, h):
            lin, sc = check(h, real_time=True), check(h, real_time=False)
            print(f"{name}: linearizable={lin is not None}, sequential={sc is not
None}")
            if lin:
                print("   linearization:", " -> ".join(f"{p}:{k}({key}={v})" for p, k,
```

```
key, v in lin))
            elif sc:
                print("   sequential order:", " -> ".join(f"{p}:{k}({key}={v})" for p,
k, key, v in sc))

        H1 = [("A", "w", "x", 1, 0, 10),   # write overlaps both reads
              ("B", "r", "x", 0, 1, 4),
              ("C", "r", "x", 1, 5, 12)]
        H2 = [("A", "w", "x", 1, 0, 5),    # write finished at t=5 ...
              ("B", "r", "x", 0, 7, 9)]    # ... yet a later read sees the old value
        H3 = [("A", "w", "x", 1, 0, 10),   # new-then-old inversion during a write
              ("B", "r", "x", 1, 2, 4),
              ("C", "r", "x", 0, 6, 8)]
        H4 = [("A", "w", "x", 1, 0, 2),    # a process misses its own write
              ("A", "r", "x", 0, 3, 4)]
        H5 = [("A", "w", "x", 1, 0, 2), ("A", "r", "y", 0, 3, 4),   # store buffering
              ("B", "w", "y", 1, 0, 2), ("B", "r", "x", 0, 3, 4)]
        H6 = [("A", "w", "x", 1, 0, None), # write timed out: outcome unknown
              ("B", "r", "x", 1, 5, 6),
              ("C", "r", "x", 1, 7, 8)]

        for name, h in [("H1", H1), ("H2", H2), ("H3", H3), ("H4", H4), ("H5", H5),
("H6", H6)]:
            describe(name, h)
        x_only = [op for op in H5 if op[2] == "x"]
        y_only = [op for op in H5 if op[2] == "y"]
        print("H5 per key sequential:", check(x_only, False) is not None, check(y_only,
False) is not None)
```

Running it prints:

```
H1: linearizable=True, sequential=True
   linearization: B:r(x=0) -> A:w(x=1) -> C:r(x=1)
H2: linearizable=False, sequential=True
   sequential order: B:r(x=0) -> A:w(x=1)
H3: linearizable=False, sequential=True
   sequential order: C:r(x=0) -> A:w(x=1) -> B:r(x=1)
H4: linearizable=False, sequential=False
H5: linearizable=False, sequential=False
H6: linearizable=True, sequential=True
   linearization: A:w(x=1) -> B:r(x=1) -> C:r(x=1)
H5 per key sequential: True True
```

What to notice:

- `must_precede` is the only difference between the two models. Linearizability constrains order by real time across all clients; sequential consistency only by each client's own program order.
- H2 and H3 (Figure 25.2) fail linearizability but pass sequential consistency: the checker reorders B's or C's stale read before the write, which is legal when real time does not count.

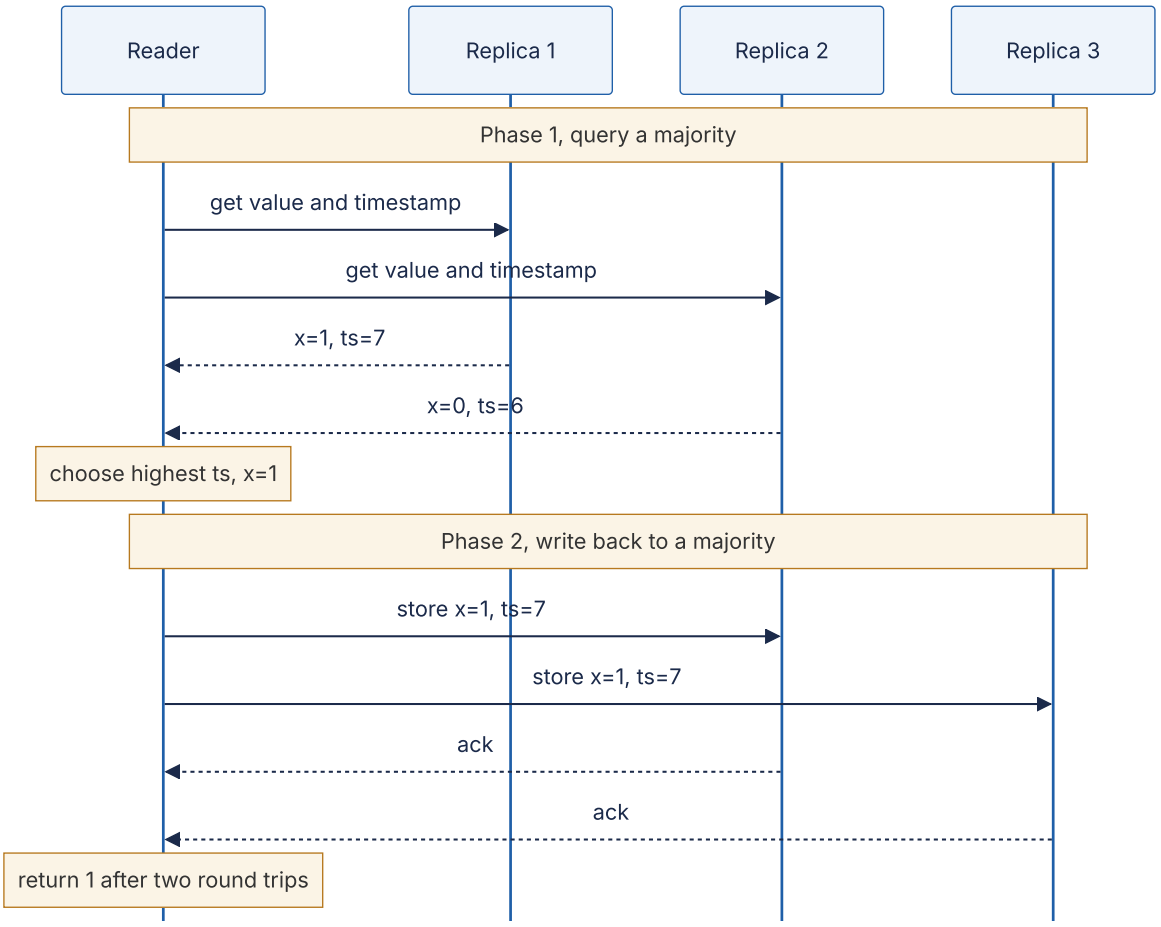
- H6 contains a write whose client timed out. The checker may include it (it evidently took effect, since reads return 1). Treating timeouts as failures would wrongly flag this history; real checkers model them as **indeterminate**, exactly as here.
- H5 is discussed in §25.4.

25.3.4 Implementing linearizability

You can obtain linearizability in three ways, all of which route every operation through some point of agreement:

1. **Single copy or single leader with synchronous reads.** All reads and writes go to one node (a single PostgreSQL server, or the leader of a partition). It is linearizable as long as there is exactly one such node. After failover, the old leader must be prevented from serving, which needs leases or a consensus round (Chapter 26). A leader that serves reads from local state without confirming it is still leader can return stale data after a new leader has been elected elsewhere.
2. **Consensus-replicated log.** Raft, Multi-Paxos and ZAB (Chapter 26) order writes through a majority. Linearizable reads either go through the log too, or the leader confirms its leadership with a quorum heartbeat (Raft's ReadIndex) or relies on a time-based lease bounded by clock-drift assumptions [Ongaro 2014].
3. **Quorum registers without a leader.** The **ABD** algorithm of Attiya, Bar-Noy and Dolev [Attiya et al. 1995] implements a linearizable read/write register over N replicas tolerating $f < N/2$ crashes, with no leader and no consensus. Each value carries a timestamp. A write sends (value, timestamp) to all replicas and waits for a majority to acknowledge. A read asks a majority for their (value, timestamp), picks the highest, and **writes it back** to a majority before returning.

Figure 25.3 — ABD quorum read with write-back on three replicas.



Reading the diagram. All calls are synchronous RPCs; the reader waits for two of three replies in each phase. Replica 1 holds a write that has reached only one node, the partial-write situation of history H3. Without phase 2, this reader returns 1 but a later reader querying Replicas 2 and 3 would return 0, an inversion. Phase 2 makes sure that any value a reader returns is stored on a majority, so every later majority overlaps it. The failure points are replica crashes (tolerated while a majority answers) and a network partition that leaves the reader with only a minority (the read blocks: this is the C-over-A choice of §25.8). Scaling is per object: different registers can live on different replica groups because linearizability is compositional.

Mathematical foundations

Every one of the three approaches needs a message exchange with a majority (or with a leader that has itself confirmed a majority) on the critical path. That is the price of linearizability.

FORMULA

Latency of a quorum operation

$$T_{\text{op}} \approx k \times \text{RTT}_{(q)} + T_{\text{proc}}, \qquad q = \lfloor N/2 \rfloor + 1$$

Symbol	Meaning	Units
T_{op}	Latency of one linearizable read or write as seen by the client	ms
k	Sequential round trips the protocol needs (ABD read: 2; ABD single-writer write: 1; Raft write via leader: 1 to the followers plus the client-to-leader trip)	count
$\text{RTT}_{(q)}$	Round-trip time to the q-th closest replica, counting the coordinator itself if it holds a replica	ms
q	Majority quorum size for N replicas	replicas
N	Number of replicas	replicas
T_{proc}	Processing and disk flush time	ms

WORKED EXAMPLE

Worked example — Five replicas across regions. A register is replicated in five regions. From the coordinator in Virginia, the RTTs are: Virginia 0 ms (local replica), Ohio 12 ms, Oregon 65 ms, Ireland 75 ms, Frankfurt 90 ms. Take $T_{\text{proc}} = 1$ ms.

1. $q = \lfloor 5/2 \rfloor + 1 = 3$. Sorted RTTs: 0, 12, 65, 75, 90. The third smallest is $\text{RTT}_{(3)} = 65$ ms (Oregon).
2. One-round-trip write: $T \approx 1 \times 65 + 1 = 66$ ms.
3. ABD read with write-back: $T \approx 2 \times 65 + 1 = 131$ ms.
4. Now move the Oregon replica to a second Virginia zone (RTT 1 ms). Sorted: 0, 1, 12, 75, 90; $\text{RTT}_{(3)} = 12$ ms. Write ≈ 13 ms; read ≈ 25 ms.

Interpretation. Quorum latency is set by the *median* replica's distance, not the nearest one (the same order statistic appears as $\text{RTT}_{(q-1)}$ over followers for Raft commits in §26.4.2 and for *R/W* quorums in §27.8.3). Clustering a majority close together makes linearizable operations cheap for nearby clients, at the cost of a region failure taking down the majority. **Engineering implication.** Linearizable global data pays a cross-region round trip somewhere; you choose where. Spanner and CockroachDB place leaders and voting replicas near the dominant writers for that reason (§25.10, Chapter 28).

Is this cost an artifact of particular algorithms, or fundamental? Lipton and Sandberg, and later Attiya and Welch, proved lower bounds in a model where message delays are between $d - u$ and d [Lipton & Sandberg 1988; Attiya & Welch 1994].

FORMULA

Lower bound for sequentially consistent and linearizable registers

$$t_{\text{read}} + t_{\text{write}} \geq d$$

Symbol	Meaning	Units
t_{read}	Worst-case time for a read to return	ms
t_{write}	Worst-case time for a write to return	ms
d	Maximum one-way message delay between replicas	ms

The bound holds already for sequential consistency, and linearizability, being stronger, inherits it. Attiya and Welch also showed that sequential consistency can make *either* reads *or* writes fast (local), while linearizability requires both reads and writes to wait for time proportional to the delay uncertainty. Causal consistency, by contrast, can be implemented with purely local reads and writes (§25.5). Kleppmann uses exactly this distinction to argue that latency, not partitions, is the more useful lens on consistency [Kleppmann 2015].

WORKED EXAMPLE

Worked example — What the bound forbids. Two data centers are 35 ms apart one way ($d = 35$ ms). A team wants a sequentially consistent register with 5 ms reads and 5 ms writes everywhere.

1. $t_{\text{read}} + t_{\text{write}} = 5 + 5 = 10$ ms.
2. $10 < 35$, so the bound is violated: no algorithm can achieve this, however clever.
3. Options: make writes slow ($t_{\text{write}} \geq 30$ ms, keeping 5 ms reads), make reads slow, or weaken the model to causal consistency, which allows both to be local.

Interpretation. This is an impossibility, not an engineering gap. **Engineering implication.** Push back early on “strongly consistent and single-digit-millisecond everywhere” and ask which operations truly need it.

Real-world industry applications

INDUSTRY EXAMPLE

Amazon S3 strong read-after-write consistency (2020). S3 historically offered eventual consistency for overwrites and listings. In December 2020 it moved to strong read-after-write consistency for all PUTs and LISTs at no extra cost, using a cache-coherence protocol in which a witness component tracks object changes so front ends can tell when cached metadata is stale (§15.4). The change removed a whole class of data-lake bugs in which a job listed a prefix and missed files just written. *Evidence: Primary* — [Vogels 2021; AWS Docs: S3 Consistency]

INDUSTRY EXAMPLE

etcd and Kubernetes. Kubernetes stores cluster state in etcd, a Raft-replicated key-value store. Leader election among controllers relies on linearizable compare-and-set on a lease object. Jepsen's 2020 analysis found etcd's key-value operations, including multi-key transactions, to be strictly serializable under partitions, pauses, crashes and clock skew, while its lock API could not guarantee mutual exclusion (§25.11). *Evidence: Primary* — [Kingsbury 2020a]

Failure scenarios

Table 25.2 — Failure scenarios that break linearizability.

What goes wrong	Why	Detection	Recovery	Prevention
Stale read from a deposed leader	Old leader is partitioned but still believes it leads and serves reads locally	Jepsen-style test with partitions; two nodes claim leadership	Step down on lease expiry	ReadIndex or quorum reads; leases with bounded clock drift
Read from asynchronous replica	Write acknowledged before replica applied it	Read-after-write probes; replica lag metrics	Route to primary	Synchronous apply, LSN-tracked reads (§25.6)
New-old inversion in quorum store	Partial write visible to one reader's quorum, not the next	Linearizability checker on recorded histories	Read repair	Write-back on read (ABD), or a consensus-replicated log
Clock-based "latest" wins	Last-writer-wins by wall clock picks a write that happened earlier	Lost-update tests with clock skew	Restore from log	Logical or hybrid timestamps (§24.12); consensus ordering

Common misconceptions

COMMON MISTAKE

Mistake — "$R + W > N$ makes a Dynamo-style store linearizable." Overlapping quorums guarantee that a read sees the latest *completed* write in the absence of failures, but they do not prevent the new-then-old inversion of H3 when a write is still in flight or partially failed, and sloppy quorums (§27.9) and last-writer-wins conflict resolution break the overlap entirely. Linearizability needs write-back on read or a consensus protocol on top of the quorum [Kleppmann 2017].

COMMON MISTAKE

Mistake — "Linearizability means operations take effect when they are sent." The linearization point may be anywhere between invocation and response. A client that sent a write and timed out does not know whether it happened; a correct client treats the outcome as unknown and retries idempotently (§22.15).

25.4 Sequential consistency

25.4.1 Definition and contrast with linearizability

DEFINITION

Definition — Sequential consistency. A history is **sequentially consistent** if there is a single total order of all operations that is a legal sequential history of the objects and that respects each client's **program order** (the order in which that client issued its own operations). In Lamport's words, "the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program" [Lamport 1979].

First-principles explanation

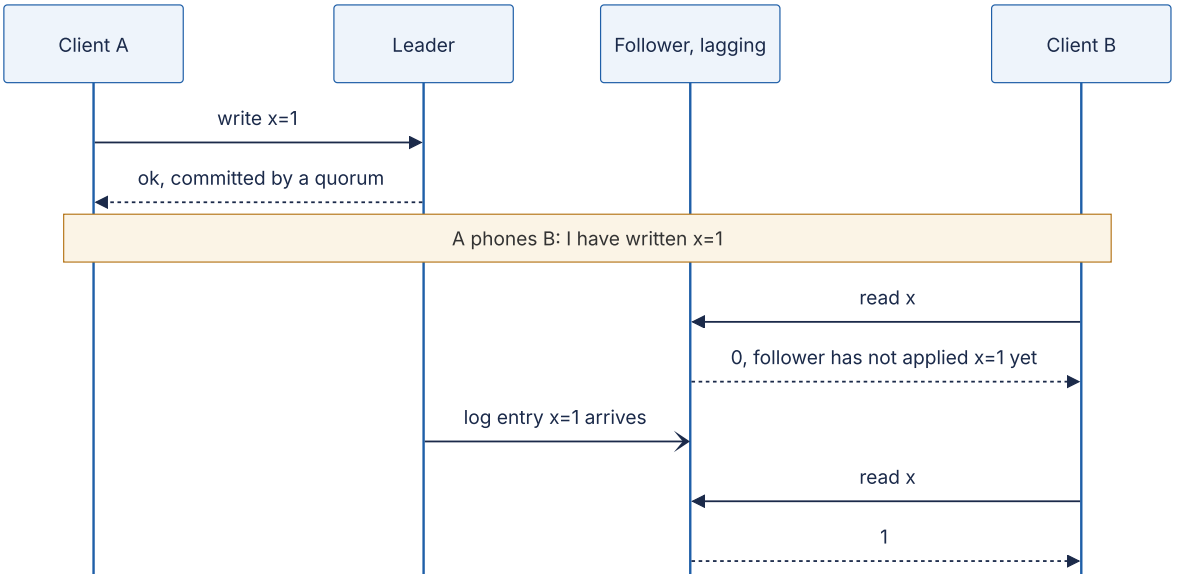
Go back to the notebook. Sequential consistency still promises that everyone sees the *same story*, entries in one order, and that each person's own entries appear in the order that person wrote them. What it drops is the clock on the wall. The story may be told late: you may read the notebook as it was ten minutes ago, as long as everyone agrees on the sequence and you never see your own entries out of order. Linearizability is sequential consistency plus the rule that the agreed order must also match real time.

Lamport defined it in 1979 for multiprocessor memory. Most modern processors provide something weaker (x86-TSO, Arm's relaxed model) and offer fence instructions to recover it where needed (cache coherence, the layer underneath, is §5.4.6).

How it works internally

A common implementation in replicated stores is: **writes go through a single ordered log; reads are served by any replica that has applied a prefix of the log, and a client never reads from a replica behind the point it has already seen.** Every replica applies the same writes in the same order, so all clients see one sequence, but a replica may lag, so a read can return a value that is already overwritten elsewhere. That is the gap between Figures 25.1 and 25.4.

Figure 25.4 — A sequentially consistent but not linearizable history (follower read).



Reading the diagram. Writes are synchronous to a quorum through the leader; log shipping to this follower (open arrowhead) is asynchronous. B's first read starts after A's write completed in real time, so returning 0 violates linearizability. But the history is sequentially consistent: the total order "B reads 0, A writes 1, B reads 1" respects each client's program order. The out-of-band phone call is the failure point: it carries causality the storage system cannot see. Followers scale reads horizontally; the cost is exactly this staleness.

ZooKeeper behaves like this by design. Writes are linearizable through the ZAB atomic broadcast, but reads are served locally by whichever server the client is connected to, so reads may be stale. ZooKeeper guarantees that each client's operations are applied in FIFO order and that a client never sees an older state after a newer one, even after reconnecting to a different server. A client that needs the latest value calls `sync` before

reading, which makes the server catch up with the leader at that moment [Hunt et al. 2010].

25.4.2 Sequential consistency is not compositional

Unlike linearizability, sequential consistency is not local: two objects can each be sequentially consistent while the system as a whole is not. History H5 from the checker of §25.3.3 is the textbook case, called **store buffering**:

- Client A writes `x = 1`, then reads `y` and gets 0.
- Client B writes `y = 1`, then reads `x` and gets 0.

Restricted to `x` alone (A's write, B's read of 0), the history is sequentially consistent: order B's read first. The same holds for `y` alone. The checker confirms both (`H5 per key sequential: True True`). But no single order works for both: whichever write comes first in the total order precedes, in program order, the other client's read, which must then return 1. The checker reports `sequential=False` for the combined history. Real multiprocessors produce exactly this outcome because each write sits in a store buffer while the following read goes to memory.

The consequence: sequentially consistent shards do not make a sequentially consistent store without cross-shard coordination, whereas linearizable shards do make a linearizable one.

INDUSTRY EXAMPLE

ZooKeeper reads and `sync`. ZooKeeper's paper describes reads served from a server's local replica as the source of its read throughput, which scales with the number of servers, while writes go through the leader. Applications that need a fresh read after an external signal (Figure 25.4's phone call) issue `sync` first, paying one leader round trip only on the paths that need it. *Evidence: Primary* — [Hunt et al. 2010]

COMMON MISTAKE

Mistake — "Sequential consistency is the same as serializability." Sequential consistency orders individual operations on objects and says nothing about grouping operations into atomic transactions. Serializability orders transactions and, in its plain form, says nothing about real time or about each client's sessions. Neither implies the other.

25.5 Causal consistency

25.5.1 Definition

Recall from §24.10 Lamport's **happens-before** relation: $a \to b$ if a and b are on the same process and a comes first, or a is the sending of a message and b its receipt, or by transitivity. For a data store, the messages are writes and reads: if a client reads a value written by w, every later operation of that client is causally after w.

DEFINITION

Definition — Causal consistency. Writes that are **causally related** must be seen by every client in their causal order; writes that are **concurrent** (neither happens before the other) may be seen in different orders by different clients. Formally, each client's view must be a legal sequential history that respects the causal order induced by program order and reads-from [Ahamad et al. 1995].

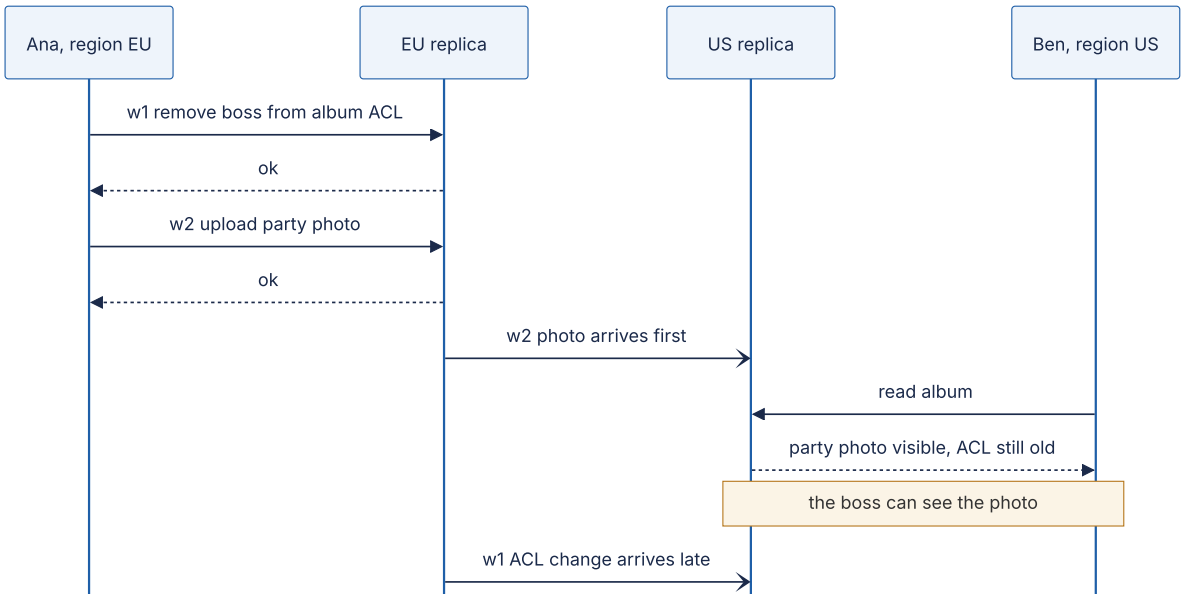
First-principles explanation

Picture a group chat. Ana asks "Who wants pizza?" and Ben replies "Me!". Causality says nobody may ever see Ben's reply without Ana's question above it, because Ben wrote it *after reading* the question. But if Ana and Chen post unrelated messages at the same moment, some phones may show Ana's first and others Chen's first; nothing is wrong with that. Causal consistency enforces the first rule and allows the second. It forbids exactly the anomalies that confuse humans (answers before questions, a comment on a photo that does not exist yet) while letting unrelated updates flow independently.

Why does this exist?

Causal consistency is a sweet spot for geo-replicated systems. It can be implemented so that every read and write completes at the local data center, with no cross-region wait, and it remains available during network partitions as long as a client stays connected to the same replica (it is **sticky available**, §25.7.5). Mahajan, Alvisi and Dahlin argued that no consistency model stronger than a variant they call real-time causal consistency can be both available under partitions and convergent [Mahajan et al. 2011]. Causal consistency is, in that sense, about the strongest model you can have while still choosing A in CAP.

Figure 25.5 — A causality violation across regions.



Reading the diagram. Ana's two writes go synchronously to her local EU replica; replication to the US (open arrowheads) is asynchronous and, here, reorders them because they travel on different partitions or connections. Ana issued w1 before w2, so w1 happens before w2. A replica that applies w2 without w1 lets the photo leak to the boss. The storage boundary is between the regions; the failure point is applying a write before its causal dependencies. A causally consistent store would hold w2 at the US replica until w1 is applied, and the scaling challenge is doing that check without serializing all writes.

25.5.2 How COPS tracks dependencies

COPS (Clusters of Order-Preserving Servers) is a geo-replicated key-value store from Lloyd, Freedman, Kaminsky and Andersen that provides **causal+ consistency**: causal consistency plus **convergent conflict handling**, so that replicas that receive concurrent writes to the same key resolve them identically and converge [Lloyd et al. 2011]. Its design shows the general recipe.

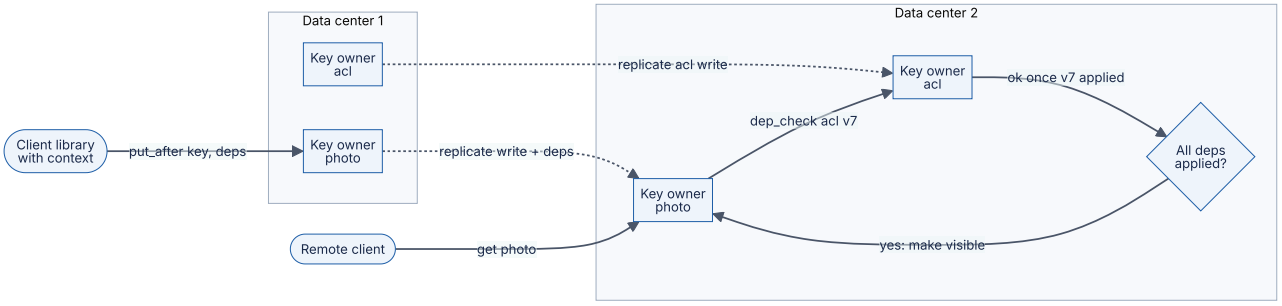
How it works internally

1. **The client library keeps a context.** Every value the client reads or writes adds an entry (key, version) to the client's context. These are the operations its next write may depend on.
2. **Writes carry dependencies.** A `put` is sent to the local cluster as `put_after(key, value, deps)`, where `deps` is the context. The local cluster applies it immediately, assigns it a version (a Lamport timestamp, so concurrent writes have a deterministic winner) and returns. The write is local: no cross-region wait.
3. **Nearest dependencies only.** Tracking every ancestor would grow without bound. Because dependencies are transitive, a write needs to list only its **nearest** depen-

dencies; COPS clears the client's context after each put and replaces it with the new write, since that write already depends on everything before it.

4. **Remote clusters check before applying.** When a replicated write arrives at a remote cluster, the node owning that key issues `dep_check` calls to the nodes owning each dependency. Each call returns once that dependency (or a later version) is applied locally. Only then is the write made visible.
5. **Reads are local.** `get` returns whatever the local cluster has; every visible value already has its dependencies visible.
6. **COPS-GT** adds read-only **get transactions** that return a causally consistent snapshot of several keys.

Figure 25.6 — COPS: local writes, asynchronous replication, dependency checks.



Reading the diagram. The client's write is acknowledged by the local key owner synchronously; replication between data centers (dotted) is asynchronous. In data center 2 the owner of `photo` asks the owner of `acl` whether version 7, the nearest dependency, is applied; the `dep_check` call blocks until it is. Only then does the photo become visible to remote clients, so no reader sees the photo without the ACL change. Each data center is a storage boundary holding a full replica, partitioned by consistent hashing within it. Failure points: a partition between data centers delays visibility of new writes there but never blocks local reads or writes. Scaling is horizontal because dependency checks touch only the partitions named in the dependency list, not a global sequencer.

Advantages and disadvantages

Causal consistency gives local-latency reads and writes in every region, survives partitions between regions, and forbids the anomalies users notice most. Its costs are metadata (dependency lists or vector clocks on every write), extra checks on the remote apply path, and **visibility delay**: a write whose dependency is stuck in a slow partition waits, so a hot dependency can hold up many writes. It also cannot enforce invariants such as uniqueness: two regions can concurrently register the same username, because concurrent writes are, by definition, allowed. Such operations still need coordination, so systems typically combine causal consistency for most data with linearizable operations for a few keys.

INDUSTRY EXAMPLE

MongoDB causally consistent sessions. Since version 3.6, MongoDB clients can open a causally consistent session. The driver carries an `operationTime` and `clusterTime` from each response into the next request, and a secondary that has not yet reached that time waits before answering. With read concern `majority` and write concern `majority`, the session provides read-your-writes, monotonic reads, monotonic writes and writes-follow-reads even when reads go to secondaries. *Evidence: Primary* — [MongoDB Docs: Causal Consistency and Read and Write Concerns]

COMMON MISTAKE

Mistake — "Causal consistency needs synchronized clocks." It needs *logical* ordering: Lamport timestamps, vector clocks or explicit dependency lists (§24.10–§24.11). Physical clocks, or hybrid logical clocks (§24.12), are sometimes added to make versions human-meaningful or to support snapshot reads, but correctness does not depend on clock accuracy.

25.6 Session guarantees

25.6.1 Four promises to one client

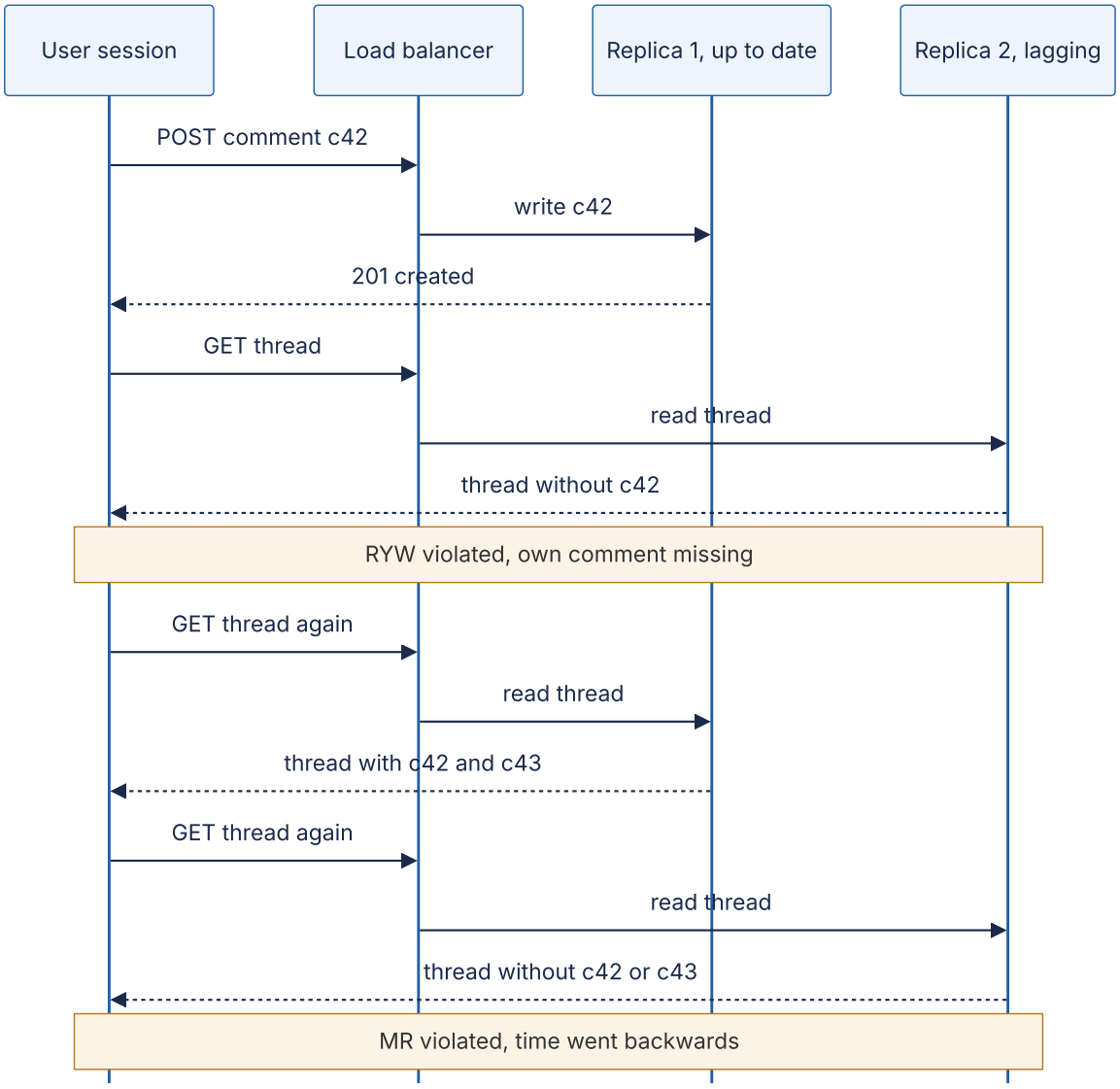
Global models constrain what *all* clients see. Often a weaker, cheaper promise suffices: that the system looks sensible *to each client individually*. Terry and colleagues, building the Bayou weakly connected replicated storage system at Xerox PARC, defined four such **session guarantees** [Terry et al. 1994]. A **session** is the sequence of operations a single client (an application instance, a browser tab, a user) performs; the guarantees hold within it.

DEFINITION

Definition — Session guarantees. Read-your-writes (RYW): if a session writes w and later reads, the read reflects w (or a later write). **Monotonic reads (MR):** if a session reads a value reflecting a set of writes, later reads reflect at least that set; the session never sees time go backwards. **Monotonic writes (MW):** a session's writes are applied at every replica in the order the session issued them. **Writes-follow-reads (WFR):** if a session reads a value produced by write w_1 and then writes w_2, every replica applies w_1 before w_2 (the new write is ordered after what the session saw).

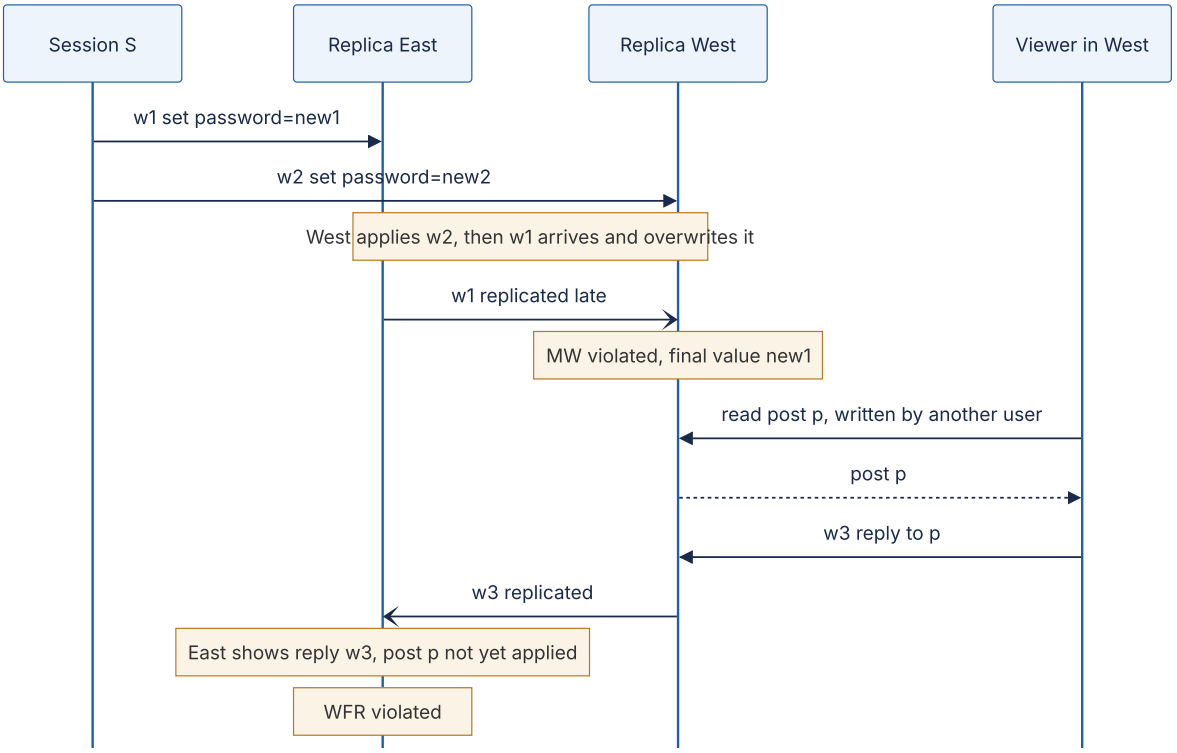
Together the four guarantees give each session a causally consistent view of its own interactions; causal consistency (§25.5) extends the same ordering across sessions that communicate through the data. Chapter 14 showed the first two failing with lagging read replicas (Figure 14.3). Figure 25.7 shows them in a multi-region setting, and Figure 25.8 shows the two write-side guarantees.

Figure 25.7 — Read-your-writes and monotonic-reads violations behind a load balancer.



Reading the diagram. All calls are synchronous HTTP requests; replication between the replicas (not drawn) is asynchronous. The load balancer is the failure point: by choosing a replica per request with no memory of the session, it can send any read to the lagging replica. Adding replicas makes the anomaly more likely, not less. The fixes below constrain routing per session, not globally, so they keep most of the read scaling.

Figure 25.8 — Monotonic-writes and writes-follow-reads violations.



Reading the diagram. Writes go synchronously to whichever replica the session reaches; replication between regions (open arrowheads) is asynchronous and unordered. In the first half, the session's two writes arrive at West in the wrong order, so West ends with the older password: the session's own write order was not preserved. In the second half, the viewer's reply depends on a post it read, but East receives the reply before the post, so readers in East see an answer to nothing. Both failures sit in the replication path; the fix is to carry ordering metadata with writes so replicas delay out-of-order ones.

25.6.2 Implementation techniques

Terry et al. implemented all four guarantees with two sets per session, stored at the client: the **write-set** (identifiers of writes the session issued) and the **read-set** (identifiers of writes that produced values the session read). Compressed as **version vectors** (one counter per origin replica, §24.11), each set is small. A replica may serve a request only if its own vector *dominates* the required one:

- RYW: before a read, the replica must cover the write-set.
- MR: before a read, the replica must cover the read-set.
- MW: before a write, the replica must cover the write-set (so earlier writes are applied first).
- WFR: before a write, the replica must cover the read-set.

If no reachable replica qualifies, the client waits or tries another replica. Production systems use the same idea in simpler forms.

Table 25.3 — Session guarantees and practical implementation techniques.

Guarantee	Anomaly prevented	Sticky sessions	Version token / LSN tracking	Other technique
Read-your-writes	Own write missing	Works only if writes and reads hit the same replica	Client keeps the commit LSN or GTID of its last write; read from a replica with applied LSN ≥ token, else wait or go to primary	Read from primary for a fixed window after a write (fails if lag exceeds window)
Monotonic reads	Data moving backwards	Pin a session to one replica: sufficient while it is up	Keep the highest LSN or vector observed; never read from a replica behind it	—
Monotonic writes	Own writes reordered	Single primary for the key already orders them	Sequence number per session; replica buffers out-of-order writes	Route all of a session's writes to one leader
Writes-follow-reads	Reply before the post it answers	—	Attach read-set vector to the write; remote replica delays apply until covered	Single-leader log orders all writes anyway

Two practical points matter. First, **single-leader systems give MW and WFR for free** within one partition, because one log orders all writes; the session problems there are RYW and MR on followers (Chapter 14). Multi-leader and leaderless systems need all four. Second, **tokens must travel with the session**, not the connection: a user who writes on a phone and reads on a laptop gets RYW only if the token is stored server-side with the user or carried in a cookie.

Code example

The program below implements the Bayou approach with version vectors. A `Session` records the vectors of what it wrote and read; a replica may serve it only if its vector dominates what the enabled guarantees require.

```
        """Bayou-style session guarantees with version vectors (after Terry et al.
1994).
        A replica's vector counts how many writes from each origin it has applied.
        The session remembers the vectors of what it wrote and what it read, and a
        replica may serve the session only if its vector dominates the required one."""

        def dominates(a, b):
            return all(a.get(k, 0) >= v for k, v in b.items())

        def merge(a, b):
            return {k: max(a.get(k, 0), b.get(k, 0)) for k in a.keys() | b.keys()}

        class Replica:
            def __init__(self, name):
                self.name, self.vv, self.log = name, {}, []
            def apply(self, origin, seq, item):        # anti-entropy delivers writes in
order
                if self.vv.get(origin, 0) == seq - 1:
                    self.vv[origin] = seq
                    self.log.append(item)
            def local_write(self, item):
                seq = self.vv.get(self.name, 0) + 1
                self.apply(self.name, seq, item)
                return self.name, seq

        class Session:
            def __init__(self, ryw=True, mr=True, mw=True, wfr=True):
                self.ryw, self.mr, self.mw, self.wfr = ryw, mr, mw, wfr
                self.read_vv, self.write_vv = {}, {}

            def can_read(self, rep):
                need = {}
                if self.ryw: need = merge(need, self.write_vv)   # see my own writes
                if self.mr:  need = merge(need, self.read_vv)    # never go backwards
                return dominates(rep.vv, need)

            def can_write(self, rep):
                need = {}
                if self.mw:  need = merge(need, self.write_vv)   # my writes stay in or-
der
                if self.wfr: need = merge(need, self.read_vv)    # writes after what I
read
                return dominates(rep.vv, need)

            def read(self, replicas):
                for rep in replicas:                             # try nearest first
                    if self.can_read(rep):
                        self.read_vv = merge(self.read_vv, rep.vv)
                        return rep.name, list(rep.log)
                return None, "no replica is fresh enough: wait or go to another site"

            def write(self, replicas, item):
                for rep in replicas:
                    if self.can_write(rep):
                        origin, seq = rep.local_write(item)
                        self.write_vv = merge(self.write_vv, {origin: seq})
```

```
                        return rep.name
                return None

        def own_write_then_read(**g):
            east, west = Replica("east"), Replica("west")
            s = Session(**g)
            s.write([east], "name=Ada")            # write lands in east
            return s.read([west, east])            # next request is routed to west
first

        def read_twice(**g):
            east, west = Replica("east"), Replica("west")
            east.local_write("comment 1")          # another user's write, not yet in
west
            s = Session(**g)
            first = s.read([east])
            second = s.read([west, east])          # load balancer prefers west now
            return first, second

        off = dict(ryw=False, mr=False, mw=False, wfr=False)
        print("RYW off:", own_write_then_read(**off))
        print("RYW on :", own_write_then_read())
        print("MR off :", read_twice(**off))
        print("MR on  :", read_twice())
```

Output:

```
RYW off: ('west', [])
RYW on : ('east', ['name=Ada'])
MR off : (('east', ['comment 1']), ('west', []))
MR on  : (('east', ['comment 1']), ('east', ['comment 1']))
```

What to notice:

- `read_vv` absorbs the *whole* vector of the replica read, a conservative over-approximation of what the session saw. Bayou did the same; precise tracking would need per-write dependencies, as in COPS.
- Replacing the vector with a single log sequence number (LSN) gives the single-leader version used with PostgreSQL and MySQL replicas (§14.4.4): one counter suffices because there is one origin.

INDUSTRY EXAMPLE

Azure Cosmos DB session consistency. Cosmos DB's default consistency level, *session*, gives read-your-writes, monotonic reads, monotonic writes and writes-follow-reads within a session by returning a **session token** with every write; the SDK sends it with later reads, and a replica that has not caught up to the token either waits or the request is retried elsewhere. Clients that share a token (for example, by passing it between web servers) share the session. *Evidence: Primary* — [Microsoft Docs: Cosmos DB Consistency Levels]

COMMON MISTAKE

Mistake — "Sticky sessions give read-your-writes." They give monotonic reads while the pinned replica is healthy. If writes go to a primary and reads to the pinned replica, the replica may not yet have the session's own write. And when the pinned replica fails, the session moves to another replica that may be further behind, breaking monotonic reads too, unless a token travels with it.

25.7 Eventual consistency

25.7.1 Definition: a liveness promise, not a safety promise

DEFINITION

Definition — Eventual consistency. If no new updates are made to an object, eventually all reads return the last updated value [Vogels 2009]. Equivalently, replicas that have received the same set of updates **converge** to the same state. **Strong eventual consistency** strengthens this: replicas that have received the same updates are in the same state immediately, with no further coordination, which is what conflict-free replicated data types (CRDTs) guarantee (§27.14).

Eventual consistency is a **liveness** property ("something good eventually happens") with no **safety** content ("nothing bad ever happens"). It says nothing about what you read *while* updates are flowing, and no bound on how long "eventually" is. A store that returned arbitrary values for an hour and then converged would technically comply [Bailis & Ghodsi 2013]. The engineering question is therefore quantitative: **how stale, how often?**

Eventual consistency also needs a **conflict resolution** rule for concurrent writes to the same key: last-writer-wins by timestamp (which silently discards one write), application merge (the Dynamo shopping cart unions concurrent versions [DeCandia et al. 2007]), or

CRDTs. Chapter 27 (§27.13–§27.14) covers anti-entropy, read repair and conflict resolution.

25.7.2 Probabilistically bounded staleness

Bailis, Venkataraman, Franklin, Hellerstein and Stoica introduced **probabilistically bounded staleness (PBS)** to answer "how eventual?" for Dynamo-style stores with N replicas, write quorum W and read quorum R [Bailis et al. 2012]. When $R + W \le N$ (a **partial quorum**), a read may miss the latest write. PBS predicts two quantities:

- **t-visibility:** the probability that a read issued t ms after a write commits returns that write (or a newer one).
- **k-staleness:** the probability that a read returns a value no more than k versions old.

Mathematical foundations

The simplest model assumes a write has reached exactly W of N replicas when it commits and has not spread further, and that a read contacts R replicas chosen uniformly at random.

FORMULA

Probability that a partial-quorum read misses the latest write

$$p_s = \frac{\binom{N-W}{R}}{\binom{N}{R}}, \qquad P(\text{older than all } k \text{ most recent versions}) = p_s^k$$

Symbol	Meaning	Units
p_s	Probability that all R replicas read hold none of the latest write's W copies	probability
N	Replication factor	replicas
W	Replicas that acknowledged the write when it committed	replicas
R	Replicas whose responses the read waits for	replicas
k	Number of most recent versions, any one of which counts as fresh enough (the read may miss $k-1$ of them)	versions

WORKED EXAMPLE

Worked example — Closed-form staleness for common settings.

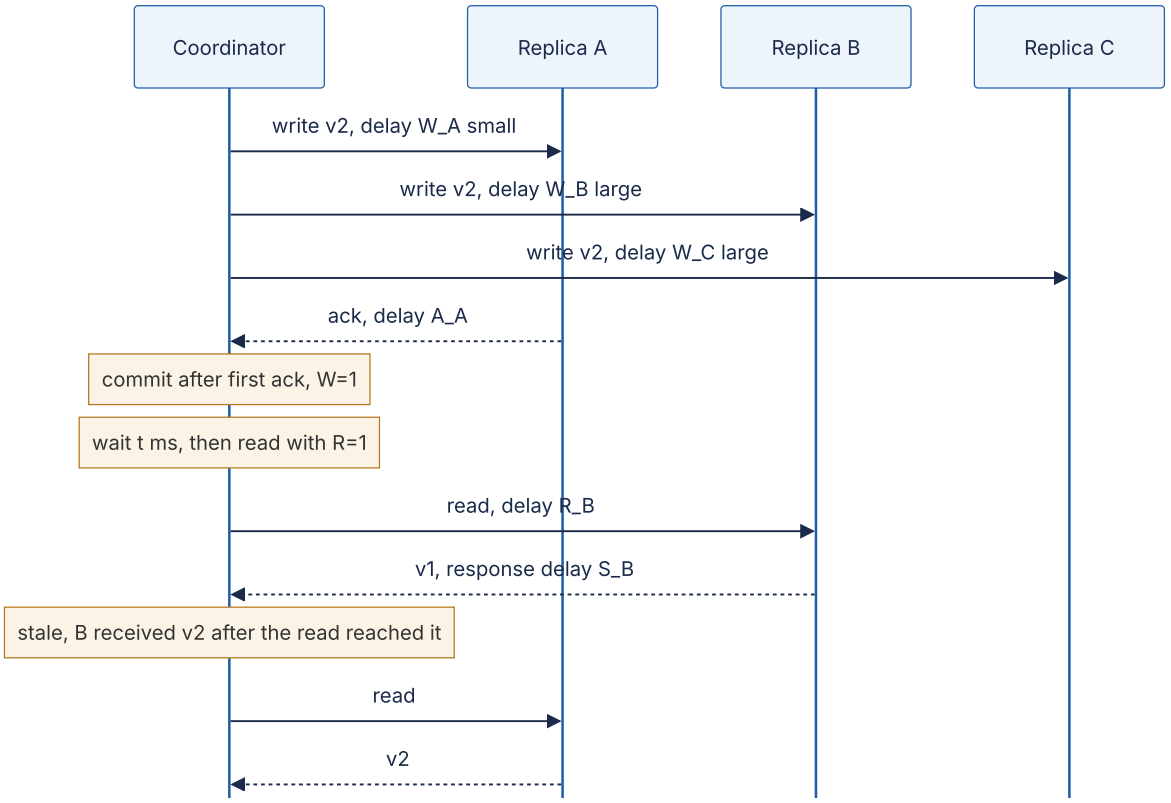
1. $N = 3$, $R = W = 1$: $p_s = \binom{2}{1}/\binom{3}{1} = 2/3 \approx 0.667$. Right after commit, two reads in three could miss the write if nothing propagates further.
2. $N = 3$, $R = 1$, $W = 2$: $p_s = \binom{1}{1}/\binom{3}{1} = 1/3$.
3. $N = 3$, $R = W = 2$: $\binom{1}{2} = 0$, so $p_s = 0$: the quorums overlap ($R + W > N$).
4. $N = 5$, $R = W = 2$: $p_s = \binom{3}{2}/\binom{5}{2} = 3/10 = 0.3$.
5. k-staleness for case 1 with $k = 3$ (the read may miss the last two versions but not three in a row): $p_s^3 = (2/3)^3 = 8/27 \approx 0.30$. With a newer write every few milliseconds, a $R = W = 1$ read lands on a replica missing three consecutive writes about 30% of the time in this pessimistic model.

Interpretation. The closed form is a worst case: it ignores the fact that the write keeps propagating to the other $N - W$ replicas after commit. **Engineering implication.** To get realistic numbers you must model propagation latency, which is what the Monte Carlo below does. The closed form is still useful for one thing: it shows that partial quorums are stale immediately after commit far more often than intuition suggests.

How it works internally: the WARS model

PBS models four latency distributions per replica, which give it its name, **WARS**: **W** (write request reaches the replica and is applied), **A** (the acknowledgment returns to the coordinator), **R** (a later read request reaches the replica), **S** (the replica's response returns). The write commits when the W-th acknowledgment arrives. A read starting t ms later returns the first R responses; it is consistent if at least one of those replicas had applied the write before the read request reached it.

Figure 25.9 — The WARS model of a partial-quorum write and read (N = 3, W = 1, R = 1).



Reading the diagram. All messages are asynchronous one-way deliveries with random delays; the coordinator only waits for the first W acknowledgments and the first R read responses. The write is applied at A quickly and at B and C later. The read happens to be answered first by B, which had not yet applied v2, so it returns v1. Had the coordinator waited for two responses ($R = 2$), A's v2 would have been included and the higher version returned. Staleness depends on the race between write propagation (W) and the read's arrival (t + R), which is why t-visibility improves with t and with slower reads. Failure points: a replica with a slow disk widens W; scaling the number of replicas without raising R or W makes misses more likely.

Code example

```
        """PBS-style Monte Carlo (WARS model, after Bailis et al. 2012): probability
        that a read issued t ms after a write commits returns that write (or newer)."""
        import random
        from math import comb

        def one_trial(n, r, w, t, lat):
            W = [lat("W") for _ in range(n)]           # write reaches replica i
            A = [lat("A") for _ in range(n)]           # replica i's ack returns
            commit = sorted(W[i] + A[i] for i in range(n))[w - 1]   # w-th ack
            R = [lat("R") for _ in range(n)]           # read request reaches replica i
            S = [lat("S") for _ in range(n)]           # replica i's response returns
            first_r = sorted(range(n), key=lambda i: R[i] + S[i])[:r]
            # consistent if some responder already had the write when the read arrived
            return any(W[i] <= commit + t + R[i] for i in first_r)

        def lat(kind, rng=random.Random(7)):
            if kind == "W":                            # propagate + apply: slow tail
                return 1.0 + rng.expovariate(1 / 4.0)  # mean 5 ms
            return 0.25 + rng.expovariate(1 / 0.5)     # A, R, S: mean 0.75 ms

        def p_consistent(n, r, w, t, trials=100_000):
            return sum(one_trial(n, r, w, t, lat) for _ in range(trials)) / trials

        if __name__ == "__main__":
            n = 3
            print("closed form, no propagation after commit: P(stale) = C(N-
W,R)/C(N,R)")
            for r, w in [(1, 1), (1, 2), (2, 1), (2, 2)]:
                print(f"  N={n} R={r} W={w}: {comb(n - w, r) / comb(n, r):.3f}")
            ts = [0, 1, 2, 5, 10, 20]
            print("\nMonte Carlo P(consistent read) by delay t after commit (ms)")
            print("R W  " + "".join(f"{t:>8}" for t in ts))
            for r, w in [(1, 1), (1, 2), (2, 1), (2, 2)]:
                row = [p_consistent(n, r, w, t) for t in ts]
                print(f"{r} {w}  " + "".join(f"{p:8.4f}" for p in row))
```

Output (about 10 seconds; the seed makes it reproducible):

```
closed form, no propagation after commit: P(stale) = C(N-W,R)/C(N,R)
  N=3 R=1 W=1: 0.667
  N=3 R=1 W=2: 0.333
  N=3 R=2 W=1: 0.333
  N=3 R=2 W=2: 0.000

Monte Carlo P(consistent read) by delay t after commit (ms)
R W         0       1       2       5      10      20
1 1    0.5000  0.6100  0.6975  0.8579  0.9600  0.9964
1 2    0.7568  0.8089  0.8514  0.9299  0.9792  0.9985
2 1    0.8253  0.8928  0.9354  0.9853  0.9989  1.0000
2 2    1.0000  1.0000  1.0000  1.0000  1.0000  1.0000
```

What to notice:

- The latency distributions are **illustrative**, not measurements: write application has a 1 ms floor plus an exponential tail with a 4 ms mean, network legs 0.25 ms plus a 0.5 ms exponential. Bailis et al. fitted distributions from production traces; with your own system, replace `lat` with samples from your latency histograms.
- At $t = 0$, $R = W = 1$ is consistent 50% of the time, better than the closed form's $1 - 2/3 = 33\%$, because propagation continues while the read is in flight.
- $R = 2, W = 1$ beats $R = 1, W = 2$ at every t here even though both have $R + W = 3$: waiting for two read responses samples more replicas *after* propagation has progressed. Which is better depends on whether reads or writes dominate your latency budget.

WORKED EXAMPLE

Worked example — Reading the simulation as an SLO. A product team accepts that a profile edit may be invisible to *other* users for a short while, but wants at least 99% of reads issued 10 ms or more after a write to be fresh. With $N = 3$:

1. From the table, at $t = 10$ ms: $(R, W) = (1, 1)$ gives 0.9600, $(1, 2)$ gives 0.9792, $(2, 1)$ gives 0.9989.
2. Only $(2, 1)$ meets 99%. It also keeps writes at one acknowledgment.
3. Cost: every read waits for the second-fastest of three responses, which raises read p99. The author's own writes still need read-your-writes (§25.6).

Interpretation. PBS turns "eventually" into a curve you can put in a service level objective. **Engineering implication.** Measure the four WARS distributions in your deployment, rerun the simulation and choose (R, W) per table; revisit when disks, load or topology change.

25.7.3 Where eventual consistency fits

Eventual consistency is the right default for data where a short window of staleness is harmless and availability or latency dominates: counters of likes and views, recommendation features, product catalogs, DNS, caches, shopping carts with merge semantics. It is the wrong choice for anything whose correctness depends on a single current value: balances, inventory reservations, uniqueness, locks. Many systems combine the two per operation, which is the theme of §25.10.

COMMON MISTAKE

Mistake — "Eventual consistency means data is usually stale." Most reads are usually fresh. The danger is *unbounded* staleness during incidents (a partitioned replica, a stalled replication stream) and the anomalies the bare model permits. Add session guarantees and monitor replication lag.

25.7.4 Models compared

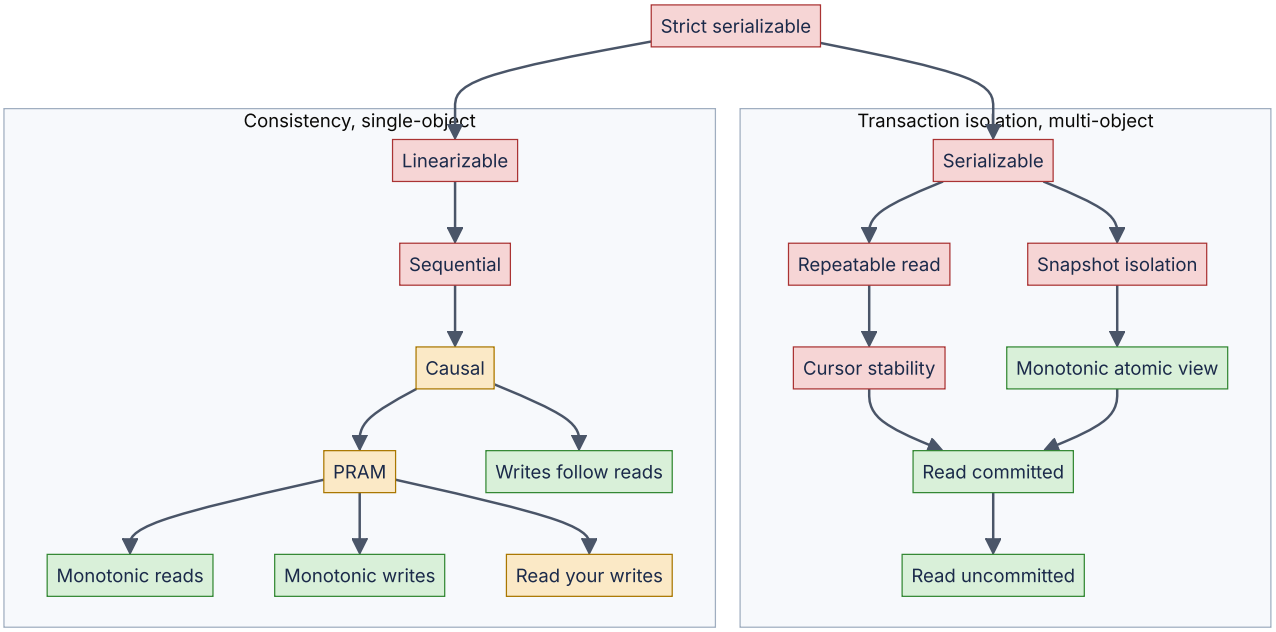
Table 25.4 — Single-object consistency models compared.

Model	What it guarantees	Allows	Latency of reads and writes	Available under partition?
Linearizable	One copy, real-time order	Nothing beyond concurrency	Quorum or leader round trip for both	No
Sequential	One agreed order, program order	Stale reads (order in the past)	One of reads or writes can be local	No
Causal	Causal order seen by all	Concurrent writes seen in different orders	Both local	Sticky available
Session guarantees (PRAM and friends)	Sensible view per session	Cross-session staleness and reordering	Local unless the replica is behind the token	Sticky or totally available
Eventual	Convergence when writes stop	Anything in between	Both local	Totally available

25.7.5 The map of consistency models

The models of §25.3–§25.7 and the isolation levels of §11.13 form a partial order: model *A* is **stronger** than *B* if every history allowed by *A* is allowed by *B*. Figure 25.10 draws that order after Jepsen's map of consistency models, which is itself based on Viotti and Vukolić and on Bailis et al.'s work on highly available transactions [Kingsbury: Consistency Models; Viotti & Vukolić 2016; Bailis et al. 2014].

Figure 25.10 — The consistency hierarchy: transactional isolation (left) and single-object consistency (right).



Reading the diagram. An arrow from *A* to *B* means *A* implies *B* (every *A* history is a *B* history); strength decreases downward. The left subgraph holds transaction isolation models, which constrain groups of operations over many objects; the right holds single-object consistency models; strict serializable at the top is the meeting point (serializable plus real time). Colors give availability under network partition as classified by Bailis et al. and Jepsen: red models cannot be totally available (some requests must block or fail in a partition); amber models are **sticky available**, meaning available as long as each client keeps talking to the same replicas; green models are **totally available**, so any non-failed replica can answer. Eventual consistency, a liveness property, is not drawn; it can be combined with any green or amber model. PRAM (pipelined RAM, [Lipton & Sandberg 1988]) is the combination of read-your-writes, monotonic reads and monotonic writes.

Two lessons from the map are worth memorizing. First, **the red-amber boundary is exactly the CAP boundary**: everything at or above sequential consistency cannot stay available in a partition, while causal consistency and below can (§25.8). Second, **isolation and consistency are orthogonal** until the very top: a serializable system can serve stale reads, and a linearizable key-value store can have no transactions at all.

25.8 The CAP theorem

25.8.1 The precise statement

Brewer conjectured in a 2000 keynote that a web service cannot be simultaneously consistent, available and partition-tolerant [Brewer 2000]. Gilbert and Lynch turned the conjecture into a theorem by defining each word [Gilbert & Lynch 2002].

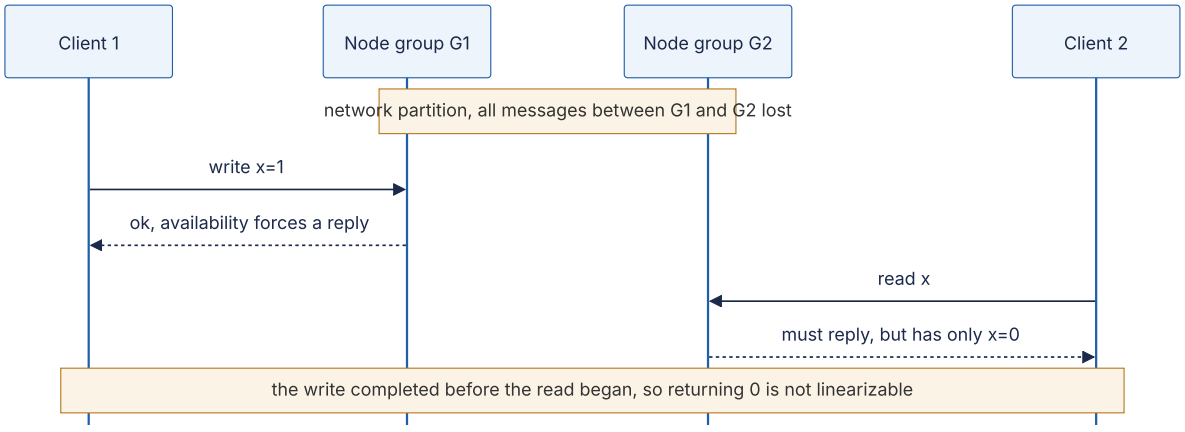
DEFINITION

Definition — CAP theorem (Gilbert and Lynch). In an asynchronous network model, where messages may be delayed arbitrarily and there are no clocks, it is impossible to implement a read/write data object that guarantees both of the following in all fair executions, including those in which messages are lost: **Consistency (C):** the object is **atomic**, that is, linearizable (§25.3). **Availability (A):** every request received by a non-failing node must eventually result in a response. **Partition tolerance (P)** is not a property the system chooses; it is the assumption that the network may lose arbitrarily many messages between nodes.

Each term is narrower than its everyday meaning. “C” is linearizability of a single register, not ACID consistency. “A” requires *every* non-failing node to answer, with no time bound: a system that answers 99.99% of requests in 100 ms but refuses the minority side is not CAP-available, and one that answers after an hour is. “P” is a network fault, not a design option.

How the proof works

Figure 25.11 — The CAP proof scenario.



Reading the diagram. The nodes are split into two groups whose messages to each other are lost. Requests are synchronous from each client’s point of view. Availability forces G1 to acknowledge the write without contacting G2, and forces G2 to answer the read without contacting G1. G2 cannot know about the write, so it returns the old value after the write completed: a violation of linearizability. The only escapes are for G1 to withhold its acknowledgment or G2 to withhold its answer until the partition heals, which gives up availability. Gilbert and Lynch also show the result holds in a partially synchronous model with local clocks.

25.8.2 Common misreadings

COMMON MISTAKE

Mistake — "Pick two of three." You cannot "pick" P: partitions happen whether you choose them or not, so the real choice is what each operation does *during* a partition, C or A. When there is no partition, CAP says nothing, and a system can be both linearizable and available [Brewer 2012]. Drawing CAP as a triangle with three equal corners is the source of most confusion.

COMMON MISTAKE

Mistake — "Our database is CA." A single-node database is not partition-tolerant in any useful sense; it is simply not distributed. A distributed system that claims CA means "we assume partitions don't happen," and when one does, it will in fact sacrifice C or A, usually without having designed which [Kleppmann 2015].

COMMON MISTAKE

Mistake — "System X is CP" or "AP" as a whole. Most systems are neither in the strict sense. A Raft-based store is not CAP-available (the minority side refuses requests), but its follower reads are not CAP-consistent either. A Dynamo-style store with quorum reads and writes is not linearizable even without partitions (§25.3.4). Kleppmann argues that the labels should be dropped in favor of naming the actual consistency model and failure behavior of each operation [Kleppmann 2015].

25.8.3 Brewer's "CAP twelve years later"

In 2012 Brewer revisited the theorem [Brewer 2012]. His main points:

1. **Partitions are rare**, so systems should not give up C or A when there is no partition.
2. **The choice is fine-grained**: per operation, per data item, per user, and it can change over time. A system can refuse to sell the last seat during a partition while still letting users browse.
3. **Partitions should be managed explicitly in three steps**: *detect* the partition (typically by timeout), enter a **partition mode** that limits some operations, then *recover* by merging state and **compensating** for mistakes, for example by refunding an oversold item. CRDTs and commutative operations make the merge mechanical.
4. **Latency and partitions are the same thing in practice**: a timeout is the only way to detect a partition, so every timeout decision is a small CAP decision.

Earlier, Fox and Brewer had framed the trade-off as **harvest** (the fraction of the data reflected in an answer) versus **yield** (the fraction of requests answered), pointing out that degrading harvest, such as a search result missing one shard, is often better than failing the request [Fox & Brewer 1999].

25.8.4 What CAP leaves out

Kleppmann's critique identifies the gaps [Kleppmann 2015]. CAP covers one consistency model (linearizability), one fault (a partition), and one notion of availability that no real system measures. It says nothing about latency, about crashed nodes, or about any model weaker than linearizability. His proposed alternative is to state, for each operation, how its latency depends on network delay d: linearizable operations take time proportional to d, causal operations can complete in constant time. This is the same insight as the lower bound of §25.3.4 and the motivation for PACELC.

INTERVIEW TIP

Interview tip — Answer CAP questions in three sentences. (1) CAP says that during a network partition a replicated register cannot be both linearizable and able to answer on every node. (2) So the real decision is per operation: refuse or delay (consistency) versus answer possibly stale or conflicting (availability), plus a plan to reconcile. (3) Outside partitions the trade-off is latency versus consistency, which is PACELC.

25.9 PACELC

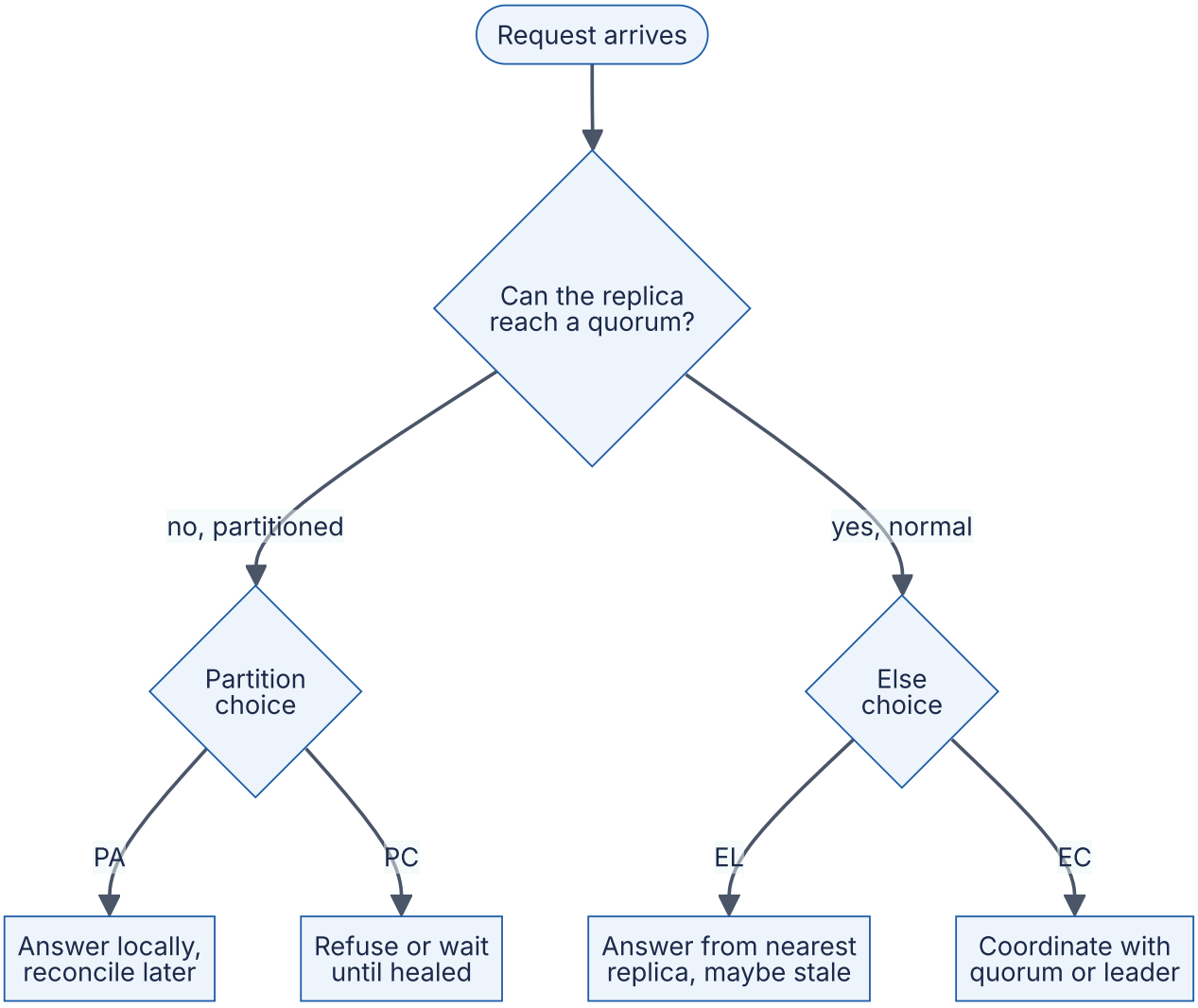
25.9.1 The formulation

Abadi observed that the consistency trade-off most systems actually make has nothing to do with partitions [Abadi 2012]. Keeping replicas synchronized costs round trips on every request, partition or not.

DEFINITION

Definition — PACELC. If there is a **P**artition, how does the system trade off **A**vailability and **C**onsistency; **E**lse, when the system is running normally, how does it trade off **L**atency and **C**onsistency? A system is labeled by its two choices, for example PA/EL or PC/EC.

Figure 25.12 — The PACELC decision for one operation.



Reading the diagram. A request takes the left branch only while the replica cannot reach the nodes it needs; that is the CAP decision. The right branch is the everyday path, where the choice is between answering from the closest replica (low latency, possibly stale) and paying a synchronous round trip for consistency. Both decisions are made per operation and are often configurable. The failure point is the “can it reach a quorum?” test itself, which in practice is a timeout.

25.9.2 Classifying real systems

Abadi’s 2012 paper labeled Dynamo, Cassandra and Riak as PA/EL; VoltDB/H-Store and Megastore as PC/EC; MongoDB as PA/EC; and Yahoo’s PNUTS as PC/EL [Abadi 2012]. Configurations have changed since; Table 25.5 gives labels for typical settings as of 2026. Treat every label as a statement about a configuration, not about a product.

Table 25.5 — PACELC classification of real systems, by configuration (as of 2026).

System	Typical configuration	P choice	E choice	Caveat
Cassandra	`ONE` or `LOCAL_ONE` reads and writes	PA	EL	`QUORUM` both ways gives PC/EC per request; lightweight transactions use Paxos (PC/EC)
DynamoDB (single region)	Default eventually consistent reads	PA	EL	`ConsistentRead=true` reads the leader replica: EC, and fails rather than serve stale data
DynamoDB global tables	Multi-Region eventual consistency (default)	PA	EL	Multi-Region strong consistency mode (GA 2025) is PC/EC, chosen at table creation
Cosmos DB	Session (default)	PA	EL within session rules	Strong is PC/EC; bounded staleness is PC within its bound
MongoDB	`w: majority`, reads from primary	PC	EC	Reads from secondaries are EL; `w: 1` (pre-5.0 default) risks rollback
Spanner	Read-write transactions, strong reads	PC	EC	Stale reads at a past timestamp are EL
CockroachDB	Default	PC	EC	Follower reads with `AS OF SYSTEM TIME` are EL
etcd, ZooKeeper	Default	PC	EC for writes	ZooKeeper reads are local (EL) unless preceded by `sync`; etcd serializable reads are EL
PostgreSQL with async replicas	Reads routed to replicas	PC for writes (one primary)	EL	Failover to an async replica can lose acknowledged writes

Mathematical foundations

The cost of “EC” can be priced directly. DynamoDB bills reads in **read capacity units (RCUs)**: one RCU is one strongly consistent read per second of an item up to 4 KB, or two eventually consistent reads [AWS Docs: DynamoDB Read Consistency].

FORMULA

DynamoDB read capacity for a read rate

$$\text{RCU} = r \times \left\lceil \frac{s}{4\ \text{KB}} \right\rceil \times f, \qquad f = \begin{cases} 1 & \text{strongly consistent} \\ 0.5 & \text{eventually consistent} \end{cases}$$

Symbol	Meaning	Units
RCU	Provisioned read capacity needed	RCU
r	Read rate	reads/s
s	Item size	KB
f	Consistency factor	RCU per 4 KB read

WORKED EXAMPLE

Worked example — Paying for EC. A profile service reads 20,000 items per second, each 6 KB.

1. Size factor: $\lceil 6/4 \rceil = 2$.
2. Eventually consistent: $20,000 \times 2 \times 0.5 = 20,000$ RCU.
3. Strongly consistent: $20,000 \times 2 \times 1 = 40,000$ RCU.
4. If only the 5% of reads that follow the user's own edit need RYW and use strong reads: $20,000 \times 2 \times (0.95 \times 0.5 + 0.05 \times 1) = 40,000 \times 0.525 = 21,000$ RCU.

Interpretation. Strong reads double read cost; applying them only where a session guarantee needs them costs 5% more than all-eventual. **Engineering implication.** Choose consistency per call site, not per table; the latency and cost difference is a property of each operation.

25.10 Consistency trade-offs in real databases

25.10.1 Tunable consistency

Modern databases rarely offer one model; they offer knobs, usually per request.

- **Cassandra** sets a consistency level per statement: `ONE`, `QUORUM`, `LOCAL_QUORUM` (a majority in the local data center), `EACH_QUORUM`, `ALL`, and `SERIAL` / `LOCAL_SERIAL` for Paxos-based lightweight transactions such as `INSERT ... IF NOT EXISTS` [Apache Cassandra Docs]. `LOCAL_QUORUM` reads and writes are the common production choice: overlapping quorums within a region, asynchronous replication to other regions (EL across regions).

- **DynamoDB** makes reads eventually consistent by default; `ConsistentRead=true` makes a `GetItem`, `Query` or `Scan` strongly consistent within a region, at twice the read cost (§25.9.2). Global secondary indexes support only eventually consistent reads [AWS Docs: DynamoDB Read Consistency].
- **MongoDB** combines write concern (`w: 1`, `w: majority`), read concern (`local`, `majority`, `linearizable`, `snapshot`) and read preference (primary or secondaries). `linearizable` read concern on the primary confirms with a majority before returning [MongoDB Docs: Read Concern].
- **Follower reads.** CockroachDB and Spanner let a query read a slightly old, consistent snapshot from the nearest replica: CockroachDB with `AS OF SYSTEM TIME follower_read_timestamp()`, Spanner with **bounded-staleness** or **exact-staleness** reads [CockroachDB Docs: Follower Reads; Google Cloud Docs: Spanner Reads]. The read is serializable but not linearizable: it is a correct view of the past.

25.10.2 Cosmos DB's five levels

Azure Cosmos DB exposes five named levels, which map neatly onto this chapter's models [Microsoft Docs: Cosmos DB Consistency Levels].

Table 25.6 — Azure Cosmos DB consistency levels (as of 2026).

Level	Guarantee	Closest model in this chapter	Notes
Strong	Reads see the latest committed write	Linearizable	Writes wait for a majority in every region; not offered with multiple write regions
Bounded staleness	Reads lag writes by at most K versions or T seconds	Linearizable outside the window; consistent prefix inside it	Minimum bounds depend on single- or multi-region writes
Session (default)	RYW, MR, MW, WFR within a session token	Session guarantees (§25.6)	Other sessions see consistent prefix
Consistent prefix	Never see writes out of order or with gaps	Ordered prefix of a single log, no recency bound	Reads may be arbitrarily old
Eventual	Convergence only	Eventual	Lowest latency and cost

25.10.3 External consistency in Spanner

Spanner provides **external consistency**, which is strict serializability for transactions: if transaction T_1 commits before T_2 starts, T_1's commit timestamp is smaller [Corbett et al. 2012]. It achieves this with TrueTime, a clock API that returns an interval guaranteed to contain true time, and a **commit wait**: a transaction does not report success until its timestamp is definitely in the past everywhere. The cost is a commit wait of about twice the clock uncertainty (about 8 ms at the paper's typical uncertainty of 4 ms), which over-

laps the Paxos round trip that logs the commit, so a commit takes roughly the larger of the two. Chapter 28 (§28.8.2) treats the mechanism.

INDUSTRY EXAMPLE

Amazon DynamoDB global tables with multi-Region strong consistency. Global tables historically replicated asynchronously between regions with last-writer-wins resolution. In June 2025 AWS made a multi-Region strong consistency (MRSC) mode generally available, so that strongly consistent reads in any participating region return the latest write, with a recovery point objective of zero. The mode is selected when the table is created; writes pay cross-region replication latency. *Evidence: Primary* — [AWS 2025: DynamoDB MRSC]

RECOMMENDATION

Recommendation — Choose consistency per operation. List each operation and ask: does a stale or reordered answer cause a wrong decision (money, inventory, uniqueness, authorization) or only a cosmetic glitch? Give the first group linearizable operations (leader reads, quorum reads, conditional writes, strong reads); give user-facing reads of a user's own data session guarantees; serve everything else eventually consistent from the nearest replica. Write the choice into the API documentation.

25.11 Jepsen-style testing

25.11.1 Why test consistency?

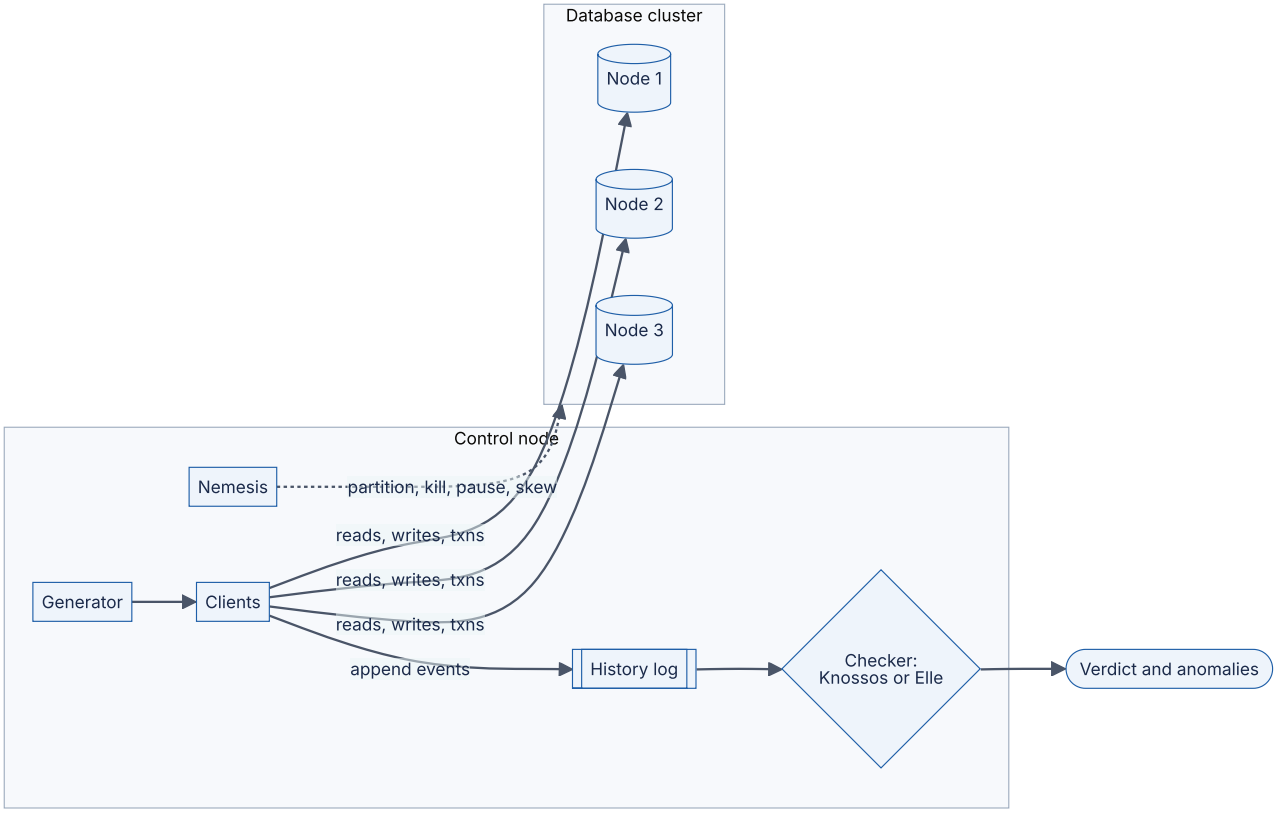
A consistency model is a claim about *all* histories under *all* allowed faults. Unit tests exercise a handful of interleavings without faults, where most systems behave linearizably. Bugs live in rare windows: a leader paused by garbage collection while a new one is elected, a partition during a membership change, a clock jump during lease renewal. Kyle Kingsbury's **Jepsen** project, started in 2013, tests real databases by generating concurrent operations, injecting faults, recording the history, and checking it against a model [Kingsbury: Jepsen].

How it works internally

1. **Set up** a cluster (typically five nodes) and a control node that runs the test.
2. **Generate** a workload: many concurrent clients issue randomized operations, such as reads, writes and compare-and-set on registers, or transactions appending to lists.
3. **Inject faults** with a **nemesis**: network partitions (via firewall rules), process crashes and pauses (`SIGSTOP`), clock skew, disk faults, membership changes.

4. **Record a history**: every invocation and completion with its result, marking timed-out operations as **indeterminate** (`:info`), exactly like H6 in §25.3.3.
5. **Check** the history: a linearizability checker for single-object workloads, or a transactional checker for isolation levels.

Figure 25.13 — Architecture of a Jepsen test.



Reading the diagram. The generator schedules operations for many client threads, which call the database over its native protocol synchronously. The nemesis acts asynchronously on the nodes over SSH. Every event goes to the history log (the storage boundary for test evidence), and the checker analyzes it offline after the run. Failure points are deliberate: the nemesis creates them. The checker is the scaling bottleneck: linearizability checking is NP-complete, so tests keep concurrency and history length modest or partition the history by key.

25.11.2 Checkers

Table 25.7 — History checkers.

Checker	What it checks	Method	Notes
Knossos	Linearizability of single objects	Search for a linearization, Wing–Gong–Lowe style	Jepsen's original checker (Clojure); cost grows quickly with concurrency
Porcupine	Linearizability	Same family, optimized, with per-key partitioning	Go; used in MIT 6.824 labs
Elle	Transaction isolation, from read uncommitted to strict serializable	Infers write-read, write-write and read-write dependencies, then finds cycles (Adya's phenomena)	Linear-time on suitable workloads such as list-append [Kingsbury & Alvaro 2020]

Elle's key idea is to design the workload so that the history *reveals* the version order: if each transaction appends unique values to lists, a read of `[1, 2, 5]` proves that the append of 2 came before the append of 5. Dependency graphs can then be built directly, and any cycle is a named anomaly (G0, G1c, G-single, G2-item) [Adya 1999; Kingsbury & Alvaro 2020].

25.11.3 Published findings

Jepsen reports are public and detailed. Three examples show the range:

INDUSTRY EXAMPLE

etcd 3.4.3 (2020). Key-value operations, including multi-key transactions, appeared strictly serializable under partitions, pauses, crashes, clock skew and membership changes, and watches delivered changes in order apart from an undocumented edge case. etcd's locks, however, could not guarantee mutual exclusion in an asynchronous network, a fundamental limitation of lease-based locks that needs fencing tokens (§26.13). *Evidence: Primary* — [Kingsbury 2020a]

INDUSTRY EXAMPLE

MongoDB 4.2.6 (2020). Even with the strongest read and write concerns, transactions exhibited read skew, cyclic information flow, duplicate writes and internal consistency violations, so they did not provide snapshot isolation; under default settings transactions could lose writes. MongoDB changed the default write concern to `majority` in version 5.0. *Evidence: Primary* — [Kingsbury 2020b; MongoDB Docs: Jepsen]

INDUSTRY EXAMPLE

PostgreSQL 12.3 (2020). Using Elle, Jepsen found that the SERIALIZABLE level allowed G2-item anomalies (anti-dependency cycles) in healthy clusters without any faults, a bug in serializable snapshot isolation that the PostgreSQL developers then fixed. The same report noted that PostgreSQL's REPEATABLE READ is snapshot isolation. *Evidence: Primary* — [Kingsbury 2020c]

RECOMMENDATION

Recommendation — Test your own system's claims. Record histories in integration tests (client, operation, invocation and completion time, result), run them through Porcupine, Knossos or Elle, and add fault injection: partition the leader, pause it beyond its lease, skew clocks. Treat every timeout as indeterminate. A single non-linearizable history with a short witness is worth more than months of green dashboards.

25.12 Summary and revision notes

SUMMARY

- **Two axes.** Single-object consistency models (linearizable, sequential, causal, session, eventual) say which values reads may return; isolation levels (§11.13) say which transaction interleavings are allowed. Strict serializability is where they meet.
- **Linearizability** = a linearization point inside each operation's interval, giving a legal sequential history that respects real time; the single-copy illusion. It is compositional. Checking is NP-complete in general; brute force with memoization works for small histories. It costs a round trip to a quorum or a confirmed leader ($T \approx k \cdot \mathrm{RTT}_{(q)}$), and $t_{\text{read}} + t_{\text{write}} \geq d$ already for sequential consistency.
- **Sequential consistency** drops real time but keeps one agreed order and program order; follower reads in ZooKeeper behave like this. It is not compositional (store buffering).
- **Causal consistency** orders only causally related writes, allows local reads and writes and is sticky available; COPS carries nearest dependencies and checks them before applying remote writes.
- **Session guarantees** (RYW, MR, MW, WFR) are implemented with read-sets and write-sets as version vectors, LSN tokens or sticky routing; tokens must travel with the session.
- **Eventual consistency** is liveness only. PBS quantifies it: $p_s = \binom{N-W}{R} / \binom{N}{R}$ immediately after commit, and a WARS simulation gives t-visibility curves you can put in an SLO.
- **CAP** (Gilbert and Lynch): in an asynchronous network that can lose messages, no register is both linearizable and available on every non-failed node. Not "pick two"; no meaningful "CA"; choose per operation what happens during a partition.
- **PACELC** adds the everyday trade-off: else, latency versus consistency. Labels describe configurations, not products.
- **Jepsen** records histories under injected faults and checks them with Knossos/Porcupine (linearizability) or Elle (isolation, via dependency cycles). Treat timeouts as indeterminate.

25.13 Knowledge checkpoint

1. What makes two operations concurrent, and when does a precede b in real time?
2. Why is history H3 (a read returns 1 during a write, a later read returns 0) not linearizable, even though each read alone could be?

3. State the locality property of linearizability and why it matters for partitioned stores.
4. Give a history that is sequentially consistent but not linearizable.
5. Why does $R + W > N$ not by itself give linearizability?
6. Five replicas have RTTs 0, 2, 30, 80 and 85 ms from the coordinator. What is the latency of an ABD read?
7. Which session guarantee does a sticky replica provide, and which does it fail to provide?
8. In COPS, why does a client need to track only nearest dependencies?
9. Compute p_s for $N = 5$, $R = 1$, $W = 2$.
10. What exactly do C, A and P mean in Gilbert and Lynch's proof?
11. Why is "CA system" a misleading label?
12. Classify Cassandra at `LOCAL_ONE` and Spanner strong reads in PACELC.
13. What does Elle need from a workload to infer the version order?
14. Why must a Jepsen history record timed-out operations as indeterminate?

Answers

1. Their invocation–response intervals overlap. *a* precedes *b* if *a*'s response happens before *b*'s invocation.
2. Once B returned 1, B's linearization point is after the write's. C's read starts after B finished, so its point is later still and must see 1.
3. A history is linearizable if and only if each object's sub-history is. Independently linearizable partitions compose into a linearizable store without cross-partition coordination for single-object operations.
4. H2: A writes $x = 1$ and completes; afterwards B reads 0. The order "B reads, then A writes" respects program order.
5. Partial writes, in-flight writes, sloppy quorums and last-writer-wins can let one reader see a new value and a later reader an old one; write-back or consensus is needed.
6. $q = 3$, $\mathrm{RTT}_{(3)} = 30$ ms; two round trips, about 60 ms plus processing.
7. Monotonic reads while the replica is healthy; not read-your-writes if writes go elsewhere, and not monotonic reads across failover.
8. Dependencies are transitive: the nearest dependency already depends on everything before it, and the remote cluster applied it only after its own dependencies.
9. $\binom{3}{1}/\binom{5}{1} = 3/5 = 0.6$.
10. C: linearizable read/write register. A: every request to a non-failing node eventually gets a response. P: the network may lose arbitrarily many messages.
11. Partitions are not optional in a distributed system; a "CA" system just has not decided whether it will lose C or A when one happens.
12. Cassandra `LOCAL_ONE`: PA/EL. Spanner strong reads: PC/EC.
13. A way to recover each object's version order from reads, for example appends of unique values to lists, so dependencies can be inferred directly.

14. The operation may have taken effect; treating it as failed produces false alarms, and treating it as succeeded can hide real anomalies.

25.14 Interview questions

Basic

Q1. What is the difference between linearizability and serializability? Answer. Linearizability is a recency guarantee for single operations on single objects: each operation appears to take effect at one instant between its call and return, respecting real time. Serializability is an isolation guarantee for transactions over many objects: the outcome equals some serial order, which need not respect real time. Strict serializability combines both.

Q2. A user updates her profile and the page still shows the old one. What happened and how do you fix it? Answer. The read went to a lagging replica, violating read-your-writes. Return the commit LSN or a session token with the write and read only from replicas that have applied it, or read from the primary for that user's own profile after a write.

Intermediate

Q3. Explain CAP precisely and say what your design does during a partition. Answer. In an asynchronous network that can lose messages, a register cannot be both linearizable and answer on every non-failing node. Per operation, I decide: payments and inventory decrements refuse on the minority side (PC); browsing, carts and likes keep serving locally with merge or compensation afterwards (PA). Outside partitions, PACELC's latency–consistency choice applies: strong reads where a stale value causes a wrong decision, nearest-replica reads elsewhere.

Q4. How would you implement monotonic reads across a fleet of read replicas? Answer. Track, per session, the highest LSN (or version vector) it has observed; route reads only to replicas at or beyond it, waiting briefly or falling back to the primary otherwise. Sticky routing alone fails on replica failure.

Advanced

Q5. Your team wants a globally replicated, linearizable key-value store with 5 ms reads and writes on three continents. Respond. Answer. Impossible: for sequential consistency already $t_r + t_w \geq d$, and cross-continent d is tens of milliseconds. Offer alternatives: place each key's quorum near its main writer (fast for local users, slow for others);

linearizable writes with follower reads that are bounded-stale; or causal consistency with local operations plus linearizable operations only for uniqueness and money.

Q6. How would you test that a new replicated store is linearizable? Answer. Jepsen-style: concurrent clients on a small key set doing reads, writes and compare-and-set; a nemesis partitioning, pausing and killing leaders and skewing clocks; record every invocation and completion, timeouts as indeterminate; check with Porcupine or Knossos per key. For transactions, use list-append workloads and Elle. Minimize failing histories and turn them into regression tests.

25.15 Practical exercises

1. Extend the checker of §25.3.3 with a compare-and-set operation and construct a history in which two CAS operations both succeed but the history is not linearizable.
2. Add a third replica and a write-side scenario to the session-guarantee program that demonstrates a monotonic-writes violation when `mw=False`.
3. Modify the PBS simulation so one of the three replicas has a 10× slower write path. How do the curves for $(R,W) = (1,1)$ and $(2,1)$ change?
4. Run Porcupine on histories produced by a toy key-value store with an asynchronous replica, and find the shortest non-linearizable history.

25.16 Design challenges

1. **Global collaborative documents.** Users in three regions edit shared documents. Choose consistency models for document contents, access control lists and document names (which must be unique per folder), and justify the latency each user sees.
2. **Ticketing.** A concert ticketing service must never oversell seats but must keep browsing available during a regional network partition. Describe partition detection, partition mode and recovery in Brewer's terms.

25.17 References

1. Abadi, D. "Consistency Tradeoffs in Modern Distributed Database System Design: CAP Is Only Part of the Story." *IEEE Computer* 45(2):37–42, 2012.
2. Adya, A. *Weak Consistency: A Generalized Theory and Optimistic Implementations for Distributed Transactions*. PhD thesis, MIT, 1999.
3. Ahamad, M., Neiger, G., Burns, J. E., Kohli, P., Hutto, P. W. "Causal Memory: Definitions, Implementation, and Programming." *Distributed Computing* 9(1):37–49, 1995.

4. Apache Cassandra. *Documentation — Consistency levels and lightweight transactions*. https://cassandra.apache.org/doc/latest/
5. Attiya, H., Bar-Noy, A., Dolev, D. “Sharing Memory Robustly in Message-Passing Systems.” *Journal of the ACM* 42(1):124–142, 1995.
6. Attiya, H., Welch, J. L. “Sequential Consistency versus Linearizability.” *ACM Transactions on Computer Systems* 12(2):91–122, 1994.
7. AWS. *Amazon DynamoDB Developer Guide — Read consistency*. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html
8. AWS. “Amazon DynamoDB global tables with multi-Region strong consistency is now generally available.” AWS What’s New, June 2025. https://aws.amazon.com/about-aws/whats-new/2025/06/amazon-dynamo-db-global-tables-multi-region-strong-consistency-generally-available/
9. AWS. *Amazon S3 User Guide — Amazon S3 data consistency model*. https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html
10. Bailis, P., Venkataraman, S., Franklin, M. J., Hellerstein, J. M., Stoica, I. “Probabilistically Bounded Staleness for Practical Partial Quorums.” *PVLDB* 5(8):776–787, 2012.
11. Bailis, P., Davidson, A., Fekete, A., Ghodsi, A., Hellerstein, J. M., Stoica, I. “Highly Available Transactions: Virtues and Limitations.” *PVLDB* 7(3):181–192, 2014.
12. Bailis, P., Ghodsi, A. “Eventual Consistency Today: Limitations, Extensions, and Beyond.” *ACM Queue* 11(3), 2013.
13. Brewer, E. “Towards Robust Distributed Systems.” Keynote, *ACM Symposium on Principles of Distributed Computing (PODC)*, 2000.
14. Brewer, E. “CAP Twelve Years Later: How the ‘Rules’ Have Changed.” *IEEE Computer* 45(2):23–29, 2012.
15. Cockroach Labs. *CockroachDB Docs — Follower Reads*. https://www.cockroachlabs.com/docs/stable/follower-reads
16. Corbett, J. C., et al. “Spanner: Google’s Globally-Distributed Database.” *USENIX OSDI*, 2012.
17. DeCandia, G., et al. “Dynamo: Amazon’s Highly Available Key-value Store.” *ACM SOSP*, 2007.
18. Fox, A., Brewer, E. A. “Harvest, Yield, and Scalable Tolerant Systems.” *Workshop on Hot Topics in Operating Systems (HotOS-VII)*, 1999.
19. Gibbons, P. B., Korach, E. “Testing Shared Memories.” *SIAM Journal on Computing* 26(4):1208–1244, 1997.
20. Gilbert, S., Lynch, N. “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services.” *ACM SIGACT News* 33(2):51–59, 2002.
21. Google Cloud. *Spanner documentation — Reads (strong, bounded-staleness and exact-staleness)*. https://cloud.google.com/spanner/docs/reads
22. Herlihy, M. P., Wing, J. M. “Linearizability: A Correctness Condition for Concurrent Objects.” *ACM Transactions on Programming Languages and Systems* 12(3):463–

492, 1990.
23. Hunt, P., Konar, M., Junqueira, F. P., Reed, B. “ZooKeeper: Wait-free Coordination for Internet-scale Systems.” *USENIX ATC*, 2010.
24. Kingsbury, K. *Consistency Models* (map and model pages). jepsen.io. https://jepsen.io/consistency
25. Kingsbury, K. *Jepsen analyses and tools*. jepsen.io. https://jepsen.io/analyses
26. Kingsbury, K. “Jepsen: etcd 3.4.3.” jepsen.io, 2020 (cited as 2020a). https://jepsen.io/analyses/etcd-3.4.3
27. Kingsbury, K. “Jepsen: MongoDB 4.2.6.” jepsen.io, 2020 (2020b). https://jepsen.io/analyses/mongodb-4.2.6
28. Kingsbury, K. “Jepsen: PostgreSQL 12.3.” jepsen.io, 2020 (2020c). https://jepsen.io/analyses/postgresql-12.3
29. Kingsbury, K., Alvaro, P. “Elle: Inferring Isolation Anomalies from Experimental Observations.” *PVLDB* 14(3):268–280, 2020.
30. Kleppmann, M. “A Critique of the CAP Theorem.” arXiv:1509.05393, 2015.
31. Kleppmann, M. *Designing Data-Intensive Applications*, ch. 9 “Consistency and Consensus.” O’Reilly Media, 2017.
32. Lamport, L. “How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs.” *IEEE Transactions on Computers* C-28(9):690–691, 1979.
33. Lipton, R. J., Sandberg, J. S. “PRAM: A Scalable Shared Memory.” Technical Report CS-TR-180-88, Princeton University, 1988.
34. Lloyd, W., Freedman, M. J., Kaminsky, M., Andersen, D. G. “Don’t Settle for Eventual: Scalable Causal Consistency for Wide-Area Storage with COPS.” *ACM SOSP*, 2011.
35. Lowe, G. “Testing for Linearizability.” *Concurrency and Computation: Practice and Experience* 29(4), 2017.
36. Mahajan, P., Alvisi, L., Dahlin, M. “Consistency, Availability, and Convergence.” Technical Report TR-11-22, University of Texas at Austin, 2011.
37. Microsoft. *Azure Cosmos DB documentation — Consistency levels*. https://learn.microsoft.com/azure/cosmos-db/consistency-levels
38. MongoDB. *MongoDB Manual — Causal Consistency and Read and Write Concerns; Read Concern*. https://www.mongodb.com/docs/manual/
39. MongoDB. *MongoDB and Jepsen*. https://www.mongodb.com/jepsen
40. Ongaro, D. *Consensus: Bridging Theory and Practice*. PhD thesis, Stanford University, 2014.
41. Papadimitriou, C. H. “The Serializability of Concurrent Database Updates.” *Journal of the ACM* 26(4):631–653, 1979.
42. Terry, D. B., Demers, A. J., Petersen, K., Spreitzer, M. J., Theimer, M. M., Welch, B. B. “Session Guarantees for Weakly Consistent Replicated Data.” *International Conference on Parallel and Distributed Information Systems (PDIS)*, 1994.
43. Viotti, P., Vukolić, M. “Consistency in Non-Transactional Distributed Storage Systems.” *ACM Computing Surveys* 49(1), Article 19, 2016.

44. Vogels, W. “Eventually Consistent.” *Communications of the ACM* 52(1):40–44, 2009.
45. Vogels, W. “Diving Deep on S3 Consistency.” *All Things Distributed*, 2021. https://www.allthingsdistributed.com/2021/04/s3-strong-consistency.html
46. Wing, J. M., Gong, C. “Testing and Verifying Concurrent Objects.” *Journal of Parallel and Distributed Computing* 17(1–2):164–182, 1993.
47. Athalye, A. *Porcupine: A fast linearizability checker in Go*. https://github.com/anishathalye/porcupine

25.18 Open verification items

- Attiya and Welch (1994): confirm the exact bounds attributed in §25.3.4 ($t_r + t_w \geq d$ for sequential consistency, credited also to Lipton and Sandberg; linearizability needing both operations to wait in proportion to delay uncertainty).
- Mahajan et al. (2011): confirm the precise statement of the real-time causal result.
- Jepsen map (Figure 25.10): re-check edges and availability colors against the current jepsen.io page (jepsen.io was unreachable from the build environment).
- PostgreSQL 12.3: confirm the fix release (believed 12.4).
- Cosmos DB: confirm minimum bounded-staleness K/T values and that strong consistency is unavailable with multiple write regions.
- Spanner commit-wait magnitude (about $2\bar{\varepsilon} \approx 8$ ms, as derived in §28.8.2 from the paper’s average ε of about 4 ms); Brewer 2012 page numbers.

Chapter 26 — Consensus and Coordination

> **Level:** 3–4 · **Track:** A (Traditional) · **Estimated reading time:** 7 h

At 02:14 a database primary stops answering heartbeats. The failover controller promotes a replica. Ninety seconds later the old primary finishes a long garbage-collection pause, finds its network link healthy again, and keeps accepting writes, because nothing told it that it had been replaced. For a few minutes two nodes both believe they are the primary, and customers who wrote to the "wrong" one lose their orders when the clusters are reconciled. Nothing here was a bug in the usual sense: every node acted correctly on the information it had. The failure was a missing *agreement* about who was in charge.

This chapter is about that agreement. **Consensus** gets unreliable machines to decide one thing and never change their minds; **coordination** builds on it: leader election, locks, leases, configuration and membership. You will learn why majority quorums work, how Paxos and Raft agree despite crashes and lost messages, how ZooKeeper, etcd and Chubby package consensus as a service, and why even a correct lock service cannot stop a paused client unless storage checks a **fencing token**.

26.1 Learning objectives

After this chapter you should be able to:

- State the consensus problem (agreement, validity, termination) and explain how a replicated state machine turns single-value consensus into a fault-tolerant service.
- Compute quorum sizes, fault tolerance, availability and commit latency for 3-, 5- and 7-node clusters, and explain flexible quorums.
- Compare bully, ring and lease-based leader election and explain why electing a leader does not, by itself, give safety.
- Trace single-decree Paxos through prepare/promise and accept/accepted, including competing proposers, and explain how Multi-Paxos removes phase 1 from the common path.
- Explain Raft's terms, elections, log replication, log matching, commit rule and the "commit only current-term entries" subtlety; size election timeouts; describe snapshots, membership changes, ReadIndex and lease reads; and run a Raft simulator.
- Use ZooKeeper, etcd, Consul and Chubby primitives (ephemeral nodes, watches, revisions, leases, sequencers) to build election, locks and barriers.

- Evaluate distributed-lock designs, including the Redlock debate, and design leases with an explicit clock-safety margin.
- Diagnose split brain and prevent it with quorums, witnesses, STONITH and fencing tokens.

26.2 Prerequisites

- System models, partial failure and network partitions — §24.6–§24.8 (consensus algorithms are defined relative to a failure model).
- Physical clocks and clock skew — §24.9 (leases and lease reads depend on bounded clock drift).
- FLP impossibility — §24.13 (explains why every practical algorithm needs timeouts).
- Linearizability — §25.3 (the consistency level consensus-backed services provide).
- Primary–replica replication and failover — §14.3–§14.4 (consensus fixes the failover problems introduced there).
- Write-ahead logs and `fsync` — §13.5, §13.12 (every vote and log entry must be durable before it is acknowledged).

26.3 The consensus problem

26.3.1 Agreement, validity, termination

> DEFINITION
>
> **Definition — Consensus.** A set of processes, each of which may **propose** a value, must **decide** on a single value such that: **agreement** — no two correct processes decide differently; **validity** (integrity) — the decided value was proposed by some process; **termination** — every correct process eventually decides. Once decided, a value never changes.

First-principles explanation

Picture five friends choosing a restaurant by text message. Messages arrive late or not at all, and any friend's phone may die mid-conversation. They want three things. Everyone who chooses must choose the *same* restaurant (agreement). The chosen restaurant must be one somebody actually suggested, not a default made up by the protocol (validity). And they must eventually stop arguing and eat (termination). The first two are **safety** properties: "nothing bad ever happens." The third is a **liveness** property: "something good eventually happens."

The difficulty is that a silent phone is ambiguous: is Dana's phone dead, or her reply just slow? Wait forever and the group may never decide; decide without her and Dana may later act on an older, different decision. Every consensus algorithm is a careful answer to this ambiguity.

Why does this exist?

Many problems reduce to consensus: choosing one leader, committing or aborting a distributed transaction, granting a lock to one client, agreeing on membership, and ordering writes to a replicated database. Solve consensus once and you can build all of these, which is why it sits underneath ZooKeeper, etcd, Consul, Chubby, Spanner, CockroachDB, TiKV and Kafka's KRaft controller (Chapter 31).

The impossibility result and how practice escapes it

Section 24.13 introduced the **FLP result** [Fischer, Lynch and Paterson 1985]: in a fully *asynchronous* system, where message delays have no upper bound, no deterministic algorithm can guarantee consensus if even one process may crash. The reason is exactly Dana's silent phone: the algorithm can never distinguish "crashed" from "slow," and an adversarial schedule can keep it undecided forever.

Practical algorithms **never compromise safety** under any timing, and guarantee **termination only when the network behaves** for long enough, a model called **partial synchrony** [Dwork, Lynch and Stockmeyer 1988]. Paxos and Raft use timeouts to suspect failures; a wrong suspicion costs time, never correctness.

> KEY CONCEPT
>
> **Key insight — Safety from quorums, liveness from timers.** In Paxos, Raft and ZAB, the rules preventing two different decisions depend only on counting votes from overlapping majorities; timeouts only decide *when to try again*. A badly tuned timeout makes these systems slow, never inconsistent. Designs that use clocks for safety (leases, §26.11) need separate scrutiny.

The failure model matters. Paxos and Raft assume **crash-recovery** faults (§24.6): nodes stop, may restart with their durable state intact, and never lie. Tolerating **Byzantine** nodes that send arbitrary or malicious messages needs different algorithms and $3f + 1$ nodes; that is the subject of Chapter 45.

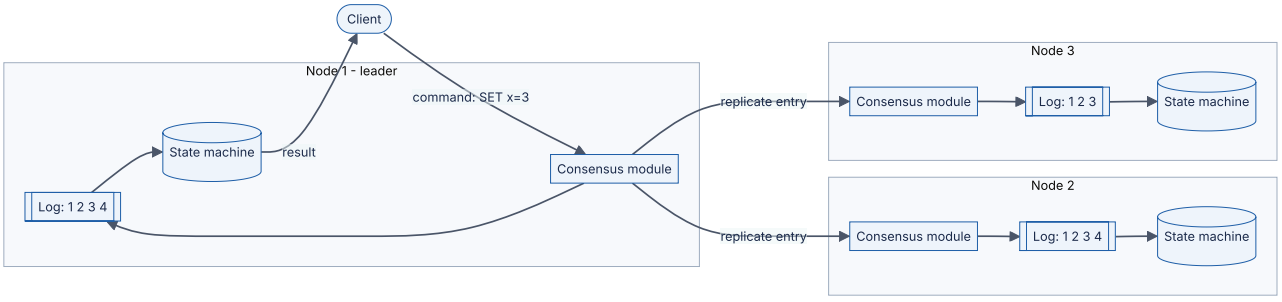
26.3.2 Replicated state machines

DEFINITION

Definition — Replicated state machine (RSM). A service implemented as a deterministic state machine that is copied onto several nodes. Each node applies the same sequence of commands, taken from a replicated **log**, in the same order, so every node passes through the same sequence of states and produces the same outputs [Schneider 1990].

Single-value consensus decides one thing. A database needs to decide millions of things in order. The **replicated state machine** approach bridges the gap: run one instance of consensus per log position (slot 1, slot 2, …). Slot *i* decides "the *i*-th command." Because each replica is deterministic and applies the same commands in the same order, the replicas stay identical without ever comparing their states.

Figure 26.1 — A replicated state machine built on a consensus log.



Reading the diagram. The client sends a command to the consensus module on the leader (Node 1). The module appends the command to its durable log (subroutine shape) and replicates it, over RPC, to the modules on the other nodes, which append it to their own logs. When a majority of logs contain the entry, it is **committed**; each node then feeds committed entries, in order, to its state machine (cylinder), and the leader returns the result. Write-path arrows are synchronous: the client waits until a majority has stored the entry. Node 3 is one entry behind, which is allowed, because a majority (Nodes 1 and 2) holds entry 4. Storage: each node owns its log and state; nothing is shared. Failure points: if Node 1 crashes, Nodes 2 and 3 elect a new leader whose log contains every committed entry. Scaling: adding nodes improves fault tolerance but not write throughput, because every node stores every entry.

Two consequences follow. The state machine must be **deterministic** (no local clock, random numbers or unordered iteration inside `apply`; the leader writes such values into the command). And the log is the source of truth; state is a cache of "the log applied so far," which is why snapshots (§26.7.8) can replace the log prefix.

COMMON MISTAKE

Misconception — "Consensus makes the system faster or more scalable." Consensus makes a service *fault-tolerant*, not fast. Every write waits for a majority round trip and a disk flush, and every node does every write. A 5-node etcd cluster has lower write throughput than a single node. Systems that need scale partition the data into many independent consensus groups (CockroachDB ranges, TiKV regions, Spanner splits), each small (§14.6, §28.10).

26.4 Quorums

26.4.1 Majority quorums and the intersection property

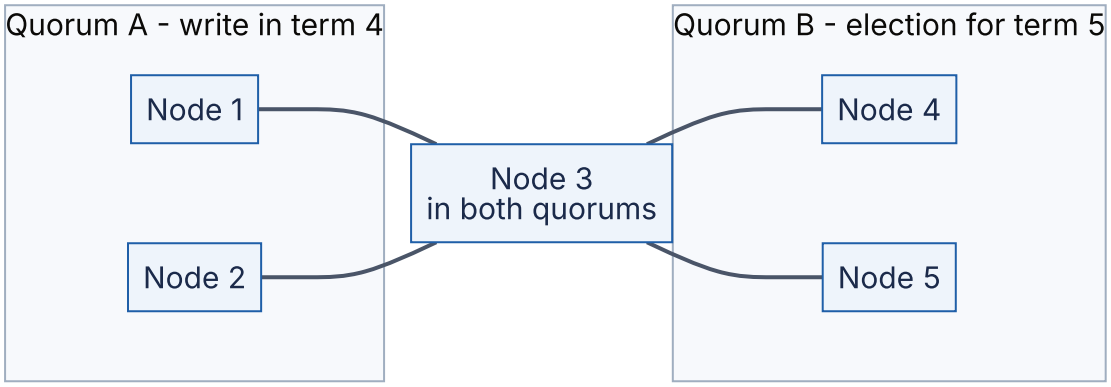
DEFINITION

Definition — Quorum. A set of nodes whose agreement is sufficient for an operation to take effect. A **majority quorum** in a cluster of N voting nodes is any set of more than $N/2$ nodes. The essential **intersection property**: any two majority quorums share at least one node.

First-principles explanation

Think of a committee of five where a decision needs three signatures. Two rival decisions would need six signatures from five members, so someone signed both. If no member signs two conflicting decisions, conflicts are impossible. The overlapping member is the system's memory: whoever comes later meets at least one node that saw what came before.

Figure 26.2 — Two majority quorums in a five-node cluster always intersect.



Reading the diagram. Quorum A is {1, 2, 3}, the nodes that stored a write in term 4. Quorum B is {3, 4, 5}, the nodes that voted in a later election. Links show membership

only; no data flows. Both quorums have three of five members, so they share a node, here Node 3. Node 3 carries the term-4 write into the term-5 election: Raft's voting rule lets it refuse any candidate whose log lacks that write (§26.7.6). The failure point the figure guards against is the **minority partition**: {4, 5} alone cannot form a quorum, so it can neither elect a leader nor commit writes. Scaling the cluster changes the arithmetic but not the principle.

Mathematical foundations

FORMULA

Majority quorum size and fault tolerance

$$q = \lfloor \frac{N}{2} \rfloor + 1 \qquad f = N - q = \lfloor \frac{N-1}{2} \rfloor \qquad \Leftrightarrow \qquad N = 2f + 1$$

Symbol	Meaning	Units
N	Number of voting nodes	nodes
q	Majority quorum size	nodes
f	Crash failures tolerated while still able to form a quorum	nodes

WORKED EXAMPLE

Worked example — 3, 4, 5 and 7 nodes.

1. $N = 3$: $q = \lfloor 1.5 \rfloor + 1 = 2$; $f = 3 - 2 = 1$.
2. $N = 4$: $q = 2 + 1 = 3$; $f = 4 - 3 = 1$.
3. $N = 5$: $q = 2 + 1 = 3$; $f = 5 - 3 = 2$.
4. $N = 7$: $q = 3 + 1 = 4$; $f = 7 - 4 = 3$.

Interpretation. The fourth node buys nothing: a 4-node cluster tolerates one failure, like a 3-node cluster, but needs three acknowledgments instead of two and has one more machine that can fail. **Engineering implication.** Run an odd number of voters: 3 for most services, 5 when you want to survive one failure *during* planned maintenance of another, 7 rarely. Add read capacity with non-voting members (ZooKeeper observers, etcd learners) rather than voters.

Fault tolerance is not the same as availability. If each node is independently available with probability a, the cluster is available when at least q nodes are up.

FORMULA

Availability of a majority-quorum cluster (independent failures)

$$A_{\text{cluster}} = \sum_{k=q}^{N} \binom{N}{k} a^{k} (1-a)^{N-k}$$

Symbol	Meaning	Units
A_{cluster}	Probability that a quorum of nodes is up	dimensionless
a	Availability of one node	dimensionless
N, q	Cluster size and quorum size	nodes
$\binom{N}{k}$	Number of ways to choose which k nodes are up	count

WORKED EXAMPLE

Worked example — nodes that are each 99% available.

1. $N = 3$, $q = 2$:
   $A = a^3 + 3a^2(1-a) = 0.970299 + 3 \times 0.9801 \times 0.01 = 0.970299 + 0.029403 = 0.999702$.
   Unavailability 2.98×10^{-4}, about 157 minutes per year.
2. $N = 5$, $q = 3$:
   $A = 0.99^5 + 5 \times 0.99^4 \times 0.01 + 10 \times 0.99^3 \times 0.01^2 = 0.950990 + 0.048030 + 0.000970 = 0.999990$.
   About 5.2 minutes per year.
3. $N = 7$, $q = 4$: $A \approx 0.99999966$, about 0.18 minutes (11 seconds) per year.
4. $N = 4$, $q = 3$: $A = 0.99^4 + 4 \times 0.99^3 \times 0.01 = 0.960596 + 0.038812 = 0.999408$, worse than $N = 3$.

Interpretation. Each step from 3 to 5 to 7 multiplies unavailability by roughly 0.03. **Engineering implication.** These numbers assume independent failures; a shared switch, zone outage or bad configuration push breaks that. Place voters in distinct failure domains, or the arithmetic is fiction.

Table 26.1 — Majority quorums at a glance.

Voters N	Quorum q	Failures tolerated f	Unavailability at $a = 0.99$	Typical use
1	1	0	1.0×10^{-2}	Development only
3	2	1	3.0×10^{-4}	Default for etcd, ZooKeeper, Consul
4	3	1	5.9×10^{-4}	Avoid: worse than 3
5	3	2	9.9×10^{-6}	Critical control planes, Chubby cells
7	4	3	3.4×10^{-7}	Rare; write latency and load grow

26.4.2 The latency trade-off

Larger clusters are more available but slower to commit, because the leader must wait for more acknowledgments. The leader waits only for the fastest $q - 1$ followers (it counts itself), so commit latency is an **order statistic** of follower round-trip times.

FORMULA

Quorum commit latency

$$L_{\text{commit}} \approx t_{\text{fsync}} + \text{RTT}_{(q-1)}$$

Symbol	Meaning	Units
L_{commit}	Time from the leader receiving a write to the entry being committed	ms
t_{fsync}	Time to flush the entry to durable storage (leader and followers flush in parallel)	ms
$\text{RTT}_{(k)}$	The k-th smallest round-trip time from the leader to its followers, including the followers' flush	ms
q	Quorum size	nodes

WORKED EXAMPLE

Worked example — a cluster spread across regions. The leader is in Virginia. Candidate replica sites have round-trip times of 12 ms (Ohio), 25 ms (Montreal), 70 ms (Oregon), 140 ms (Frankfurt), 150 ms (London) and 210 ms (Tokyo). Assume $t_{\text{fsync}} = 1$ ms.

1. $N = 3$ (Virginia, Ohio, Montreal): $q - 1 = 1$ follower needed; fastest RTT is 12 ms; $L \approx 1 + 12 = 13$ ms.
2. $N = 5$ (add Oregon, Frankfurt): $q - 1 = 2$; second-fastest is 25 ms; $L \approx 26$ ms.
3. $N = 7$ (add London, Tokyo): $q - 1 = 3$; third-fastest is 70 ms; $L \approx 71$ ms.

Interpretation. Going from 3 to 7 replicas multiplies commit latency by about 5.5, because the quorum must now reach across the continent. **Engineering implication.** Choose voter placement as carefully as voter count, for example 5 voters in three nearby regions (2 + 2 + 1), and place the leader where $\text{RTT}_{(q-1)}$ is smallest.

26.4.3 Flexible quorums

A majority is sufficient for intersection, but it is not *necessary*. Howard, Malkhi and Spiegelman observed that Paxos uses quorums in two different phases: **phase 1** (leader election, “who is in charge and what might already be decided?”) and **phase 2** (replication, “store this value”). Safety only requires that *every phase-1 quorum intersects every phase-2 quorum*; two phase-2 quorums need not intersect, because only one leader issues phase-2 requests for a given proposal number [Howard, Malkhi and Spiegelman 2016].

FORMULA

Flexible Paxos quorum condition

$$|Q_1| + |Q_2| > N$$

Symbol	Meaning	Units
$\lvert Q_1 \rvert$	Size of a leader-election (phase-1) quorum	nodes
$\lvert Q_2 \rvert$	Size of a replication (phase-2) quorum	nodes
N	Number of acceptors	nodes

WORKED EXAMPLE

Worked example — 7 acceptors.

1. Majority quorums: $|Q_1| = |Q_2| = 4$; $4 + 4 = 8 > 7$. Each write waits for 3 followers; elections and replication each tolerate 3 failures.
2. Flexible choice: $|Q_2| = 3$, so $|Q_1| \geq 7 - 3 + 1 = 5$. Check: $3 + 5 = 8 > 7$.
3. With a stable leader, each write now waits for only 2 followers (the second-fastest RTT instead of the third), and replication continues while the leader and any 2 other nodes are up, tolerating 4 failures among the followers.
4. A leader change needs 5 of 7 nodes, so it tolerates only 2 failures.

Interpretation. Flexible quorums move fault tolerance from the rare path (elections) to the common path (writes). **Engineering implication.** Useful when leader changes are rare and write latency dominates; risky when failures are correlated (a zone loss kills the leader *and* several nodes), so most production systems keep majorities.

26.5 Leader election

26.5.1 Election algorithms

DEFINITION

Definition — Leader election. A protocol by which the nodes of a group choose one node, the **leader**, to coordinate some activity (ordering writes, running a scheduler, owning a partition), and choose a replacement when it fails. The goal is *at most one* leader at a time (safety) and *eventually some* leader (liveness).

Why does this exist?

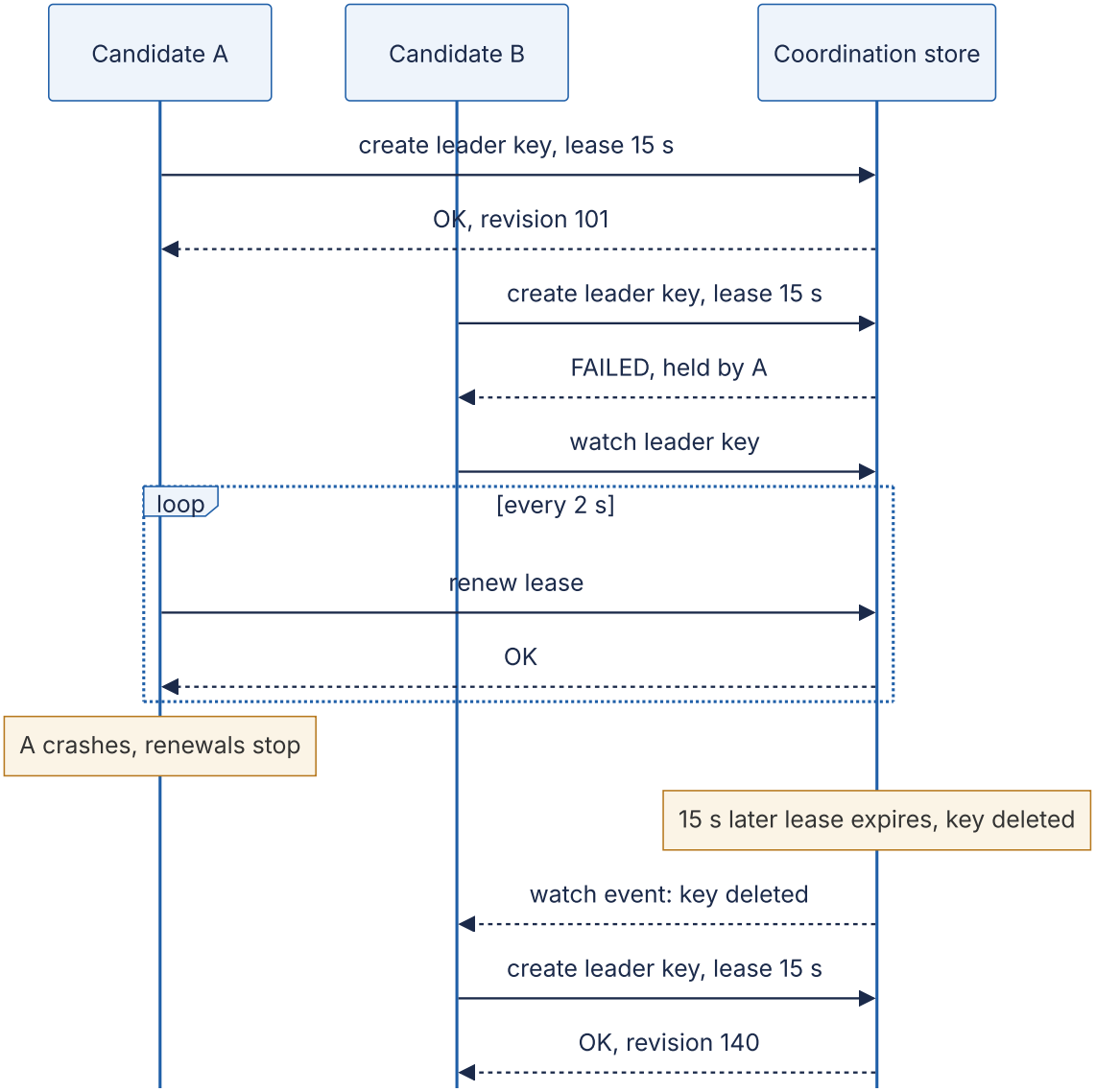
Most replicated systems are far simpler with one coordinator. A single leader can order writes without conflicts, assign work without double assignment, and answer "what is the latest value?" from its own memory. The cost is that the leader is a single point of failure, so the group needs a way to replace it quickly and without ever having two.

How it works internally: three classic approaches

1. **Bully algorithm** [Garcia-Molina 1982]. A node that suspects the leader has failed sends `ELECTION` to all higher-ID nodes. If none answers within a timeout, it announces itself with `COORDINATOR`; otherwise a higher node takes over. The highest-ID live node wins. Worst case $O(n^2)$ messages; it assumes reliable failure detection, which real networks do not provide.

2. **Ring algorithm** [Chang and Roberts 1979]. IDs travel clockwise around a logical ring; each node forwards only IDs larger than its own. A node that receives its own ID back is the maximum and announces itself. Average $O(n \log n)$ messages, worst case $O(n^2)$; a single broken link stalls it.
3. **Lease-based election on a consensus store.** Candidates race to create or acquire a key (say `/services/scheduler/leader`) in ZooKeeper, etcd or Consul. The store, itself replicated by consensus, lets exactly one candidate succeed and attaches a **lease** (a time-limited grant, §26.11). The winner renews the lease periodically; if it stops, the lease expires and another candidate wins. This is how almost every production system elects leaders today, including Kubernetes controllers (§26.9).

Figure 26.3 — Lease-based leader election through a coordination service.



Reading the diagram. Both candidates call the coordination store with synchronous RPCs (solid: request; dashed: response). The store is a consensus-backed cluster whose

atomic “create if absent” guarantees a single winner. A’s lease is renewed every 2 s; when A crashes, nothing deletes the key until the 15 s lease expires. B learns of the expiry through a **watch** (an asynchronous notification it registered earlier) rather than by polling. The revision numbers (101, 140) increase with every write and can serve as fencing tokens (§26.13). Failure points: if A is merely *paused* rather than crashed, it may still believe it is leader after its lease expired; the store cannot reach into A to stop it. Scaling: one store cluster can host thousands of election keys for different services.

26.5.2 Why election alone does not give safety

Every algorithm above produces a node that *believes* it leads; none can make a deposed leader *stop believing it*. A partitioned, paused or slow old leader overlaps with its successor, and if both act on shared state you have split brain (§26.12).

Safe systems therefore add one of two mechanisms on top of election:

- **Every action is itself checked by a quorum.** In Raft and Paxos, a leader cannot commit anything without a majority acknowledging it *in its current term*. A deposed leader’s requests carry an old term, and any majority contains at least one node that has seen the newer term and rejects them. The old leader may *believe* it leads, but it cannot *do* anything.
- **Every action carries an epoch the resource can check.** When the protected resource is outside the consensus group (a database, a file in object storage), the leader must present a monotonically increasing token, and the resource rejects stale ones. This is fencing (§26.13).

> COMMON MISTAKE
>
> **Misconception — “Once we have leader election, only one node acts as leader.”** Election guarantees at most one leader *per term or lease epoch* as seen by the election service. It says nothing about what an old leader does with its stale belief. Without quorum-checked actions or fencing, two nodes will eventually act as leader at the same time.

26.6 Paxos

Leslie Lamport described Paxos as the legislature of a fictional Greek island whose part-time legislators wander in and out of the chamber [Lamport 1998]. The paper was written around 1990 and published in 1998; its allegory made it famously hard to read, and Lamport later restated the algorithm, beginning: “The Paxos algorithm, when presented in plain English, is very simple” [Lamport 2001]. This section follows that plain-English version.

26.6.1 Single-decree Paxos

DEFINITION

Definition — Single-decree (basic) Paxos. A consensus algorithm in which **proposers** suggest values, **acceptors** vote on them and **learners** find out the decided value. A value is **chosen** when a majority of acceptors has accepted it with the same proposal number. Paxos guarantees safety under any message delays, loss or reordering, and makes progress whenever one proposer can run both phases without interference from another.

First-principles explanation

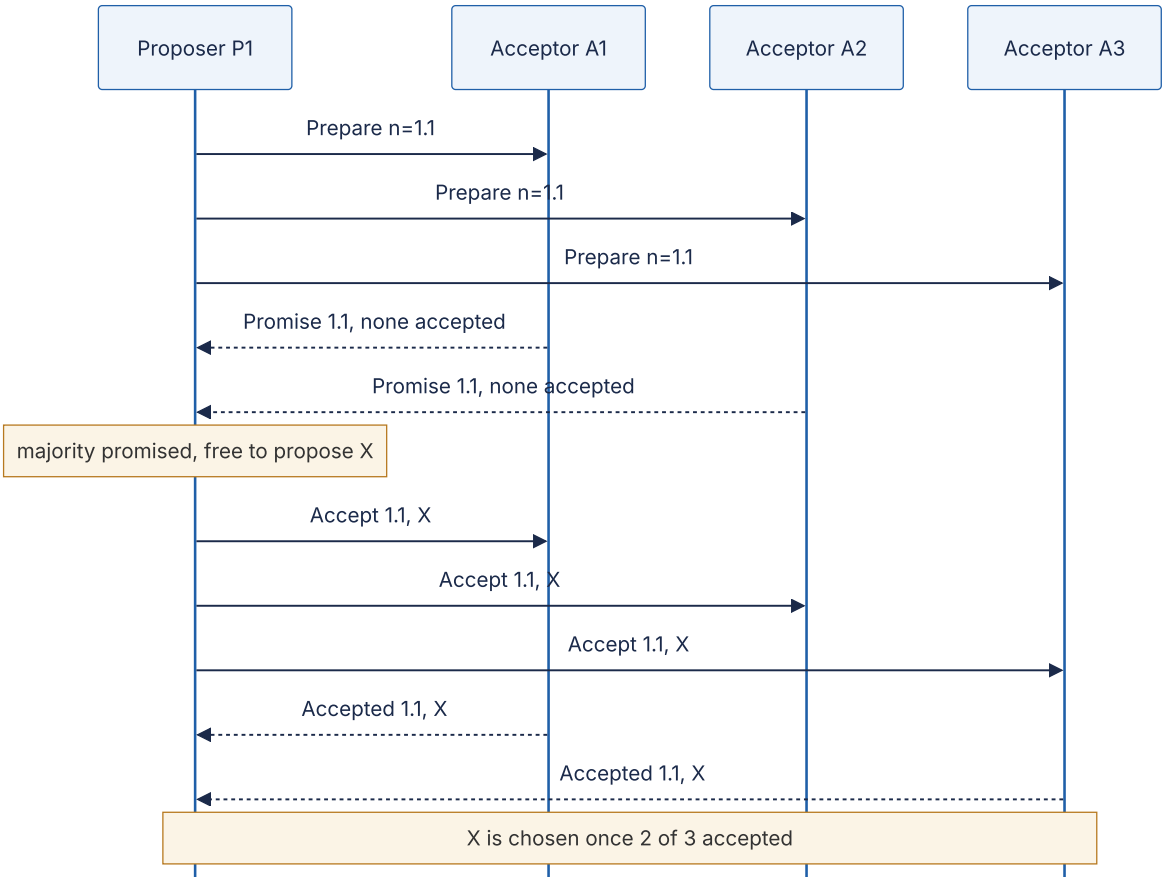
Imagine booking a room through five receptionists, each with a paper diary. A rival may be asking a different three at the same moment, so you first present a *ticket number* higher than any they have seen ("promise to ignore lower tickets") and ask whether they have already written a booking. If any has, you must *finish that booking* instead of yours. Only after a majority promises do you ask them to write the booking under your ticket. The first step discovers and protects the past; the second decides.

How it works internally

Each acceptor stores, durably: `promised_n` (highest proposal number it promised) and `(accepted_n, accepted_v)` (the last proposal it accepted, if any). Proposal numbers must be unique and totally ordered; the usual construction is a pair `(round, proposer_id)` compared lexicographically, so proposers never collide.

1. **Phase 1a — Prepare.** A proposer picks a proposal number n higher than any it has used and sends `Prepare(n)` to the acceptors.
2. **Phase 1b — Promise.** An acceptor that receives `Prepare(n)` with $n >$ `promised_n` sets `promised_n = n`, writes it to disk, and replies `Promise(n, accepted_n, accepted_v)`. Otherwise it ignores the request or replies with a rejection containing `promised_n` (a useful hint).
3. **Value selection.** If the proposer receives promises from a majority, it looks at the accepted proposals they reported. If any acceptor reported one, the proposer **must** propose the value v of the highest-numbered accepted proposal. Only if none reported anything may it propose its own value.
4. **Phase 2a — Accept.** The proposer sends `Accept(n, v)` to the acceptors.
5. **Phase 2b — Accepted.** An acceptor that receives `Accept(n, v)` with $n \geq$ `promised_n` records `(accepted_n, accepted_v) = (n, v)` durably and replies `Accepted(n, v)` to the proposer and the learners. Otherwise it rejects.
6. **Learning.** When a learner sees `Accepted(n, v)` from a majority for the same n, the value v is chosen.

Figure 26.4 — Single-decree Paxos without contention.



Reading the diagram. P1 uses proposal number 1.1 (round 1, proposer 1). All arrows are point-to-point RPCs; each acceptor must persist its promise or acceptance before replying, so every reply implies a disk flush. Two promises out of three form a majority, so P1 does not wait for A3's promise. Because no promise reported an accepted value, P1 may propose its own value X. Two `Accepted` replies (from A1 and A3) form a majority, and X is chosen at that instant, even before anyone knows it. Failure points: one acceptor can fail without blocking progress; if the proposer fails, a later proposer's phase 1 discovers X. Cost: two round trips and two durable writes per value.

Why the value-adoption rule makes Paxos safe

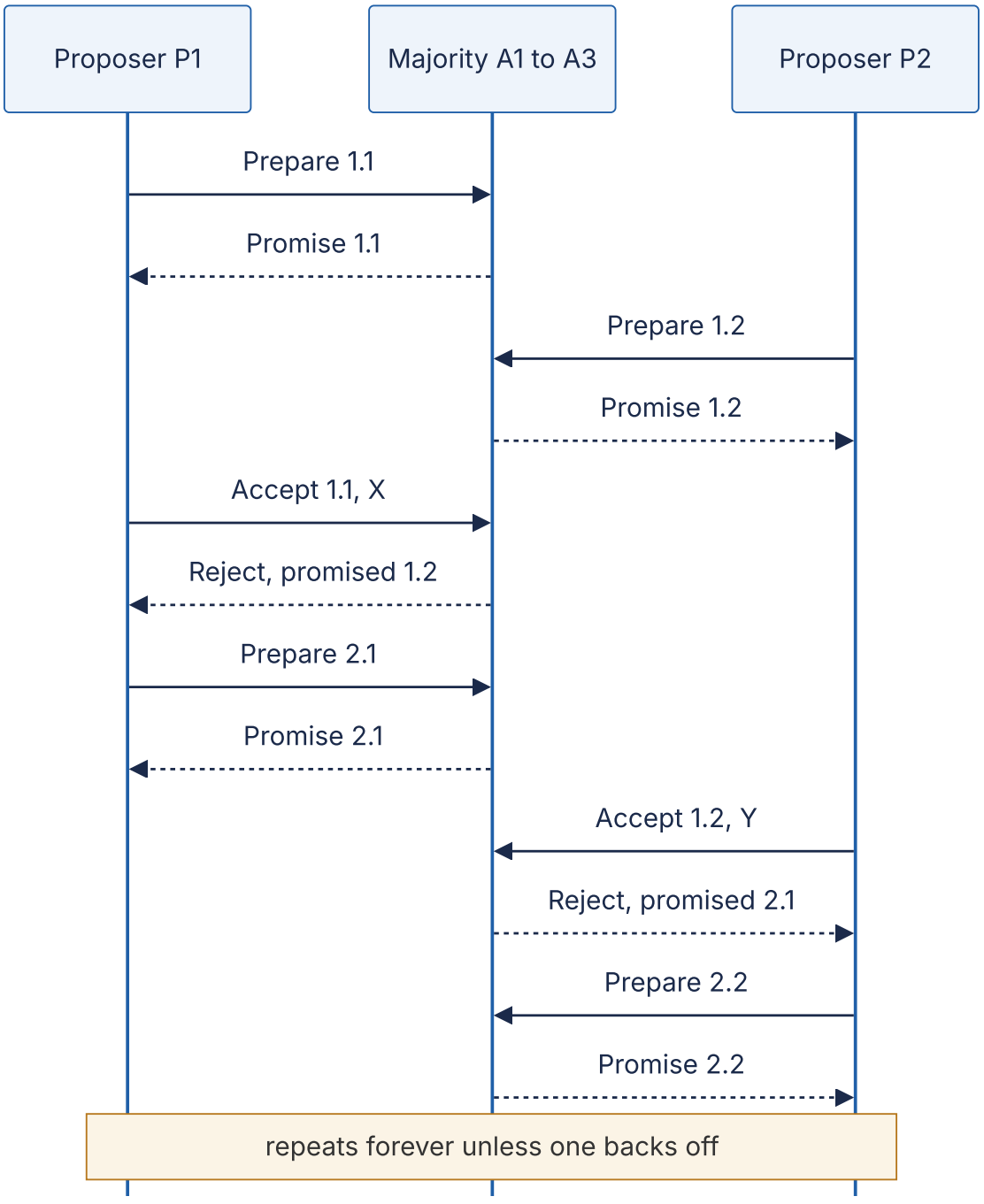
Suppose X was chosen with proposal 1.1, meaning a majority accepted it. Any later proposer with number $n > 1.1$ must collect promises from a majority, and that majority intersects the accepting majority (§26.4.1). So at least one promise reports "I accepted (1.1, X)." The adoption rule then forces the new proposer to propose X again. By induction on proposal numbers, every proposal after X was chosen carries X. Lamport's invariant: *if a value v is chosen with number n, every higher-numbered proposal issued has value v* [Lamport 2001].

Note that a value can be chosen without its proposer knowing (replies lost), and an acceptor may hold an accepted value that is never chosen; neither breaks safety.

Dueling proposers

Paxos is safe but not guaranteed to terminate, and the classic way it stalls is two proposers repeatedly pre-empting each other.

Figure 26.5 — Dueling proposers: safe but not live.



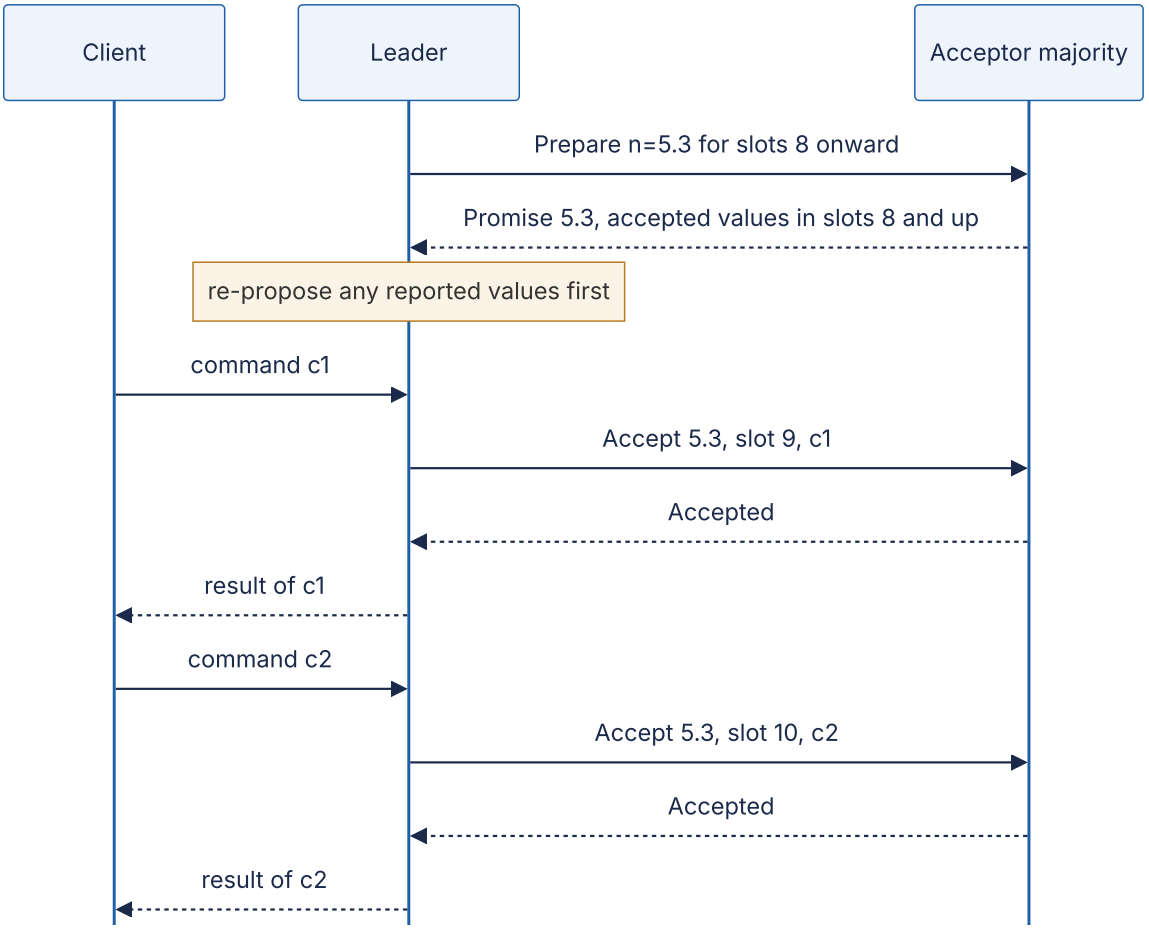
Reading the diagram. The acceptors are drawn as one participant; each arrow stands for messages to and replies from a majority. Before P1's accept arrives, P2's higher prepare (1.2) raises every promise, so P1's accept is rejected; P1's retry (2.1) then invalidates P2's accept, and so on. Nothing wrong is decided, but nothing is decided at all: the FLP adversarial schedule (§24.13) made concrete. The fixes are liveness mechanisms:

randomized back-off and, above all, a single **distinguished proposer** (leader), which Multi-Paxos and Raft provide.

26.6.2 Multi-Paxos

Running two round trips per log slot is expensive. **Multi-Paxos** observes that phase 1 does not depend on the value, so a stable leader can run phase 1 *once* for all future slots: `Prepare(n)` for "slot *i* and every slot after it." Each acceptor's promise covers all those slots and reports any values it accepted in them. From then on, each new command needs only phase 2: one round trip and one durable write, the same cost as Raft's AppendEntries.

Figure 26.6 — Multi-Paxos: phase 1 once, then one round trip per command.



Reading the diagram. The new leader (5.3) prepares every slot from 8 onward in one synchronous exchange with a majority. Promises report values already accepted there; slot 8 had one, so the leader re-proposes it first. Each client command then costs one `Accept` / `Accepted` round trip plus a durable write per acceptor, and the leader applies commands in slot order. Failure point: a successor leader must fill **gaps** (slots never chosen) with no-ops. Scaling: batch commands per slot and pipeline slots.

Paxos in production

Google's **Paxos Made Live** is the best antidote to the idea that knowing the algorithm is enough [Chandra, Griesemer and Redstone 2007]. Building Chubby's replicated log needed additions the papers did not specify: master leases for local reads, epoch numbers to detect leader changes mid-operation, disk-corruption handling (a replica that lost its state must not vote as if it remembered its promises), snapshots, membership changes and extensive randomized testing. The authors conclude that there are significant gaps between the algorithm's description and a real system's needs.

> INDUSTRY EXAMPLE
>
> **Google Chubby's replicated log.** Problem: a lock service for thousands of machines that must survive replica failures. Architecture: five replicas per cell running Multi-Paxos with a leader ("master") holding a lease; the replicated log stores the lock-service database. Why chosen: safety without timing assumptions. Trade-off: most engineering effort went outside the core algorithm. *Evidence: Primary* — [Chandra, Griesemer and Redstone 2007]; [Burrows 2006]

Table 26.2 — Paxos family vocabulary mapped to Raft.

Concept	Paxos / Multi-Paxos	Raft	ZAB
Leader identity epoch	Proposal number (round, id)	Term	Epoch (high 32 bits of zxid)
Phase 1	Prepare / Promise	RequestVote (plus log check)	Discovery and synchronization
Phase 2	Accept / Accepted	AppendEntries	Proposal / Ack / Commit
Log position	Slot (may have gaps)	Index (no gaps)	Counter (low 32 bits of zxid)
Who can become leader	Anyone; must learn and adopt prior values	Only a node with an up-to-date log	Node with the highest zxid, then synchronizes followers

26.7 Raft

Raft was designed by Diego Ongaro and John Ousterhout with **understandability** as its primary goal [Ongaro and Ousterhout 2014]. It matches Multi-Paxos in fault tolerance and performance, but splits consensus into leader election, log replication and safety, and removes freedoms Paxos allows, such as log gaps or a leader with a stale log. etcd, Consul, CockroachDB, TiKV, Vault's integrated storage and Kafka's KRaft (Chapter 31) all use Raft or close variants. Ongaro's thesis adds membership changes, compaction, client interaction and read optimizations [Ongaro 2014].

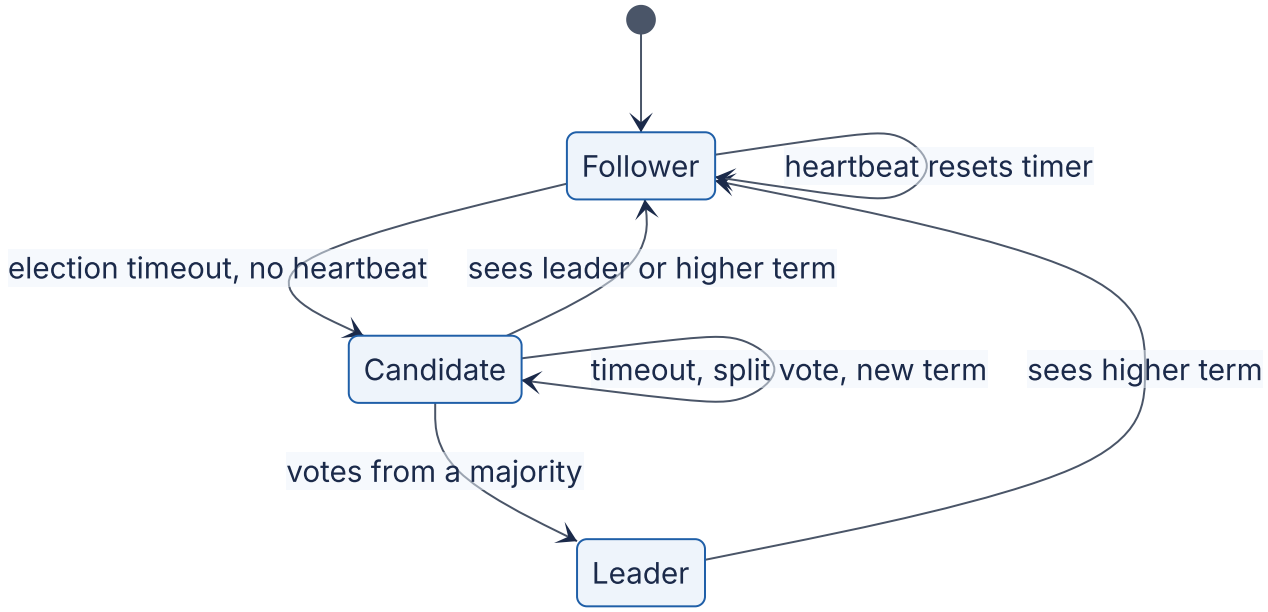
26.7.1 Terms, roles and state

DEFINITION

Definition — Raft term. A logical clock that divides time into consecutive, numbered periods. Each term begins with an election; if a candidate wins, it is the **leader** for the rest of the term, and there is at most one leader per term. Every RPC carries the sender's term; a node that sees a higher term immediately adopts it and becomes a follower; a request with a lower term is rejected.

Every node is in one of three roles: **follower** (passive; answers RPCs), **candidate** (seeking votes) or **leader** (handles all client writes and replicates the log). Each node persists `currentTerm`, `votedFor` (the candidate it voted for in this term, if any) and its log, and must flush them to disk *before* replying to any RPC that changed them. Volatile state includes `commitIndex` (highest entry known committed) and `lastApplied`; leaders also keep `nextIndex[]` and `matchIndex[]` per follower.

Figure 26.7 — Raft role transitions.



Reading the diagram. Every node starts as a follower. A follower that hears no leader for its randomized election timeout increments its term and becomes a candidate. A candidate becomes leader with a majority of votes, reverts to follower on hearing a legitimate leader or a higher term, and starts a new term if its timeout expires first (typically after a split vote). A leader steps down only on seeing a higher term. All transitions are local decisions from messages and timers. Failure point: a leader that is partitioned away stays leader in its own view, but cannot commit anything, and steps down as soon as it hears the higher term.

26.7.2 Leader election

1. A follower's election timer expires. It increments `currentTerm`, votes for itself, persists both, and sends `RequestVote(term, candidateId, lastLogIndex, lastLogTerm)` to all peers.
2. A node grants its vote if the candidate's term is at least its own, it has not voted for someone else in this term, and the **candidate's log is at least as up-to-date** as its own: the candidate's last entry has a higher term, or the same term and an index at least as large. Each node votes for at most one candidate per term, first come, first served.
3. A candidate with votes from a majority becomes leader and immediately sends empty `AppendEntries` heartbeats to assert authority.
4. If two candidates split the vote, both time out and retry in a higher term. Raft makes this rare by drawing each timeout at random, for example uniformly from 150–300 ms, the range the paper recommends [Ongaro and Ousterhout 2014].

Raft's timing requirement is broadcastTime ≪ electionTimeout ≪ MTBF: heartbeats must arrive well within the timeout, and the timeout must be far shorter than the mean time between failures.

FORMULA

Raft election timing and expected failover time

$$h \approx \mathrm{RTT}_{\mathrm{typ}}, \qquad T_{\min} \geq 10 \times \mathrm{RTT}_{\mathrm{typ}}, \qquad T_{\mathrm{failover}} \approx T_{\min} + \frac{W}{n+1} - \frac{h}{2} + \mathrm{RTT} + t_{\mathrm{fsync}}$$

Symbol	Meaning	Units
h	Heartbeat interval	ms
$\mathrm{RTT}_{\mathrm{typ}}$	Typical round-trip time between members	ms
$T_{\min}, W$	Election timeout is drawn uniformly from $[T_{\min}, T_{\min} + W]$	ms
n	Followers still alive after the leader fails	nodes
T_{failover}	Expected time from leader crash to a new leader	ms

The heartbeat and 10× rules are the etcd tuning guidance [etcd Docs: Tuning]. The failover estimate adds three pieces: the earliest of n uniform timers fires on average $W/(n+1)$ after $T_{\min}$; timers were last reset by a heartbeat on average $h/2$ before the crash; and the winner then needs one vote round trip with a disk flush.

WORKED EXAMPLE

Worked example — two deployments.

1. *Single data center*, RTT about 0.5 ms, Raft-paper settings: $h = 50$ ms, $T_{\min} = 150$ ms, $W = 150$ ms, 5 nodes so $n = 4$ survivors.
   $T_{\text{failover}} \approx 150 + 150/5 - 50/2 + 0.5 + 1 = 150 + 30 - 25 + 1.5 = 156.5$ ms.
2. *Cross-region*, RTT 70 ms: $h = 100$ ms (etcd's default), and $T_{\min} \geq 10 \times 70 = 700$ ms, so keep etcd's default of 1,000 ms; etcd's Raft library randomizes over $[T, 2T)$, so $W = 1,000$ ms. $T_{\text{failover}} \approx 1,000 + 1,000/5 - 50 + 70 + 1 = 1,221$ ms.

Interpretation. The simulator in §26.7.7, which uses the first deployment's settings, measures a median failover of 155 ms over 200 seeded runs, matching the estimate. **Engineering implication.** Failover time is dominated by the election timeout, which must grow with network RTT. Lowering it below about 10 RTTs trades faster failover for spurious elections whenever a p99.9 network hiccup or GC pause delays heartbeats.

Randomization does not eliminate competing candidates; it makes them unlikely.

FORMULA

Probability that a second candidate starts before the first's votes arrive

$$P_{\text{contend}} = 1 - \left(1 - \frac{\delta}{W}\right)^{n}$$

Symbol	Meaning	Units
δ	Time for the first candidate's RequestVote to reach the others (one-way delay plus flush)	ms
W	Width of the randomized timeout range	ms
n	Number of followers whose timers are running	nodes
P_{contend}	Probability that the gap between the first two timeouts is below δ	dimensionless

WORKED EXAMPLE

Worked example — choosing the randomization width. With $n = 4$ and $\delta = 10$ ms:

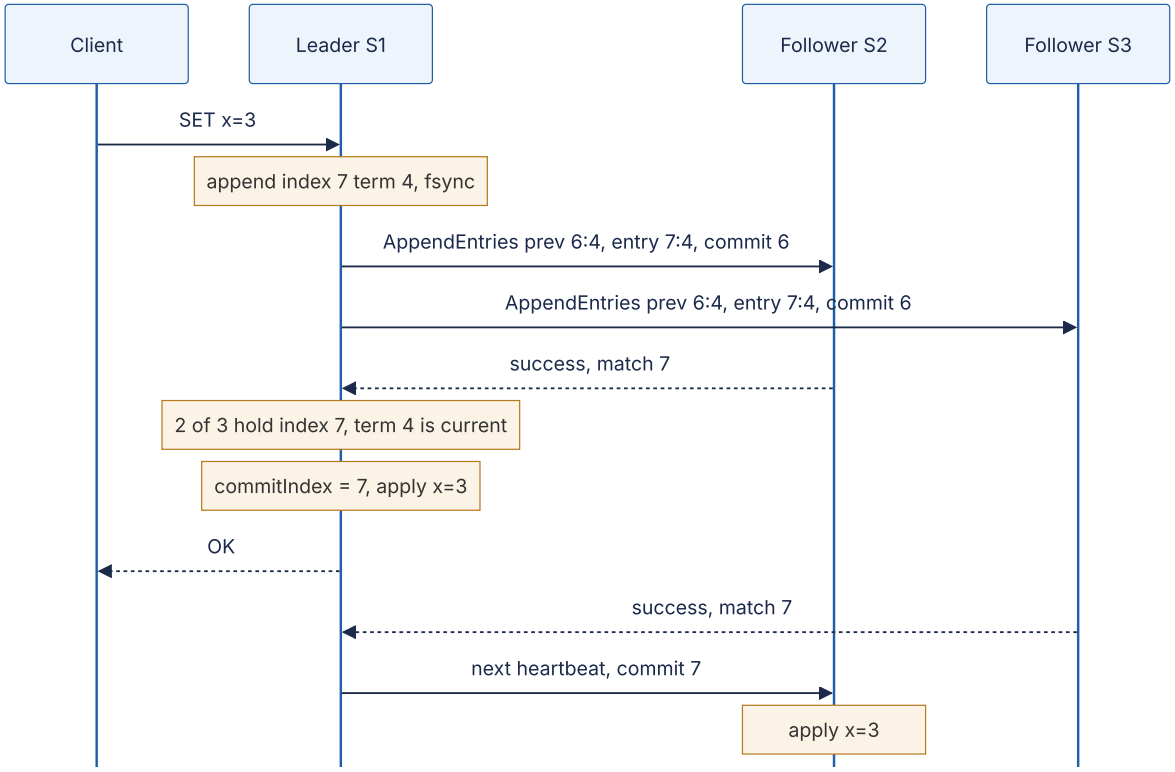
1. $W = 150$ ms: $1 - (1 - 10/150)^4 = 1 - 0.9333^4 = 1 - 0.7588 = 0.241$.
2. $W = 1,000$ ms: $1 - 0.99^4 = 1 - 0.9606 = 0.039$.
3. $W = 150$ ms but $\delta = 2$ ms (fast LAN, NVMe flush): $1 - 0.98667^4 = 0.052$.

Interpretation. This bounds split votes from above; contention does not always split the vote. **Engineering implication.** Keep W at least an order of magnitude above δ. A 200,000-trial Monte Carlo check of the first case gives 0.241, matching the formula.

26.7.3 Log replication

Once elected, the leader accepts client commands, appends each as a new log entry `(term, index, command)`, and replicates it with `AppendEntries(term, leaderId, prevLogIndex, prevLogTerm, entries[], leaderCommit)`. Heartbeats are simply AppendEntries with no entries.

Figure 26.8 — Replicating and committing one entry.



Reading the diagram. The client's write goes synchronously to the leader, which appends entry 7 (term 4) and flushes it. AppendEntries carries `prev 6:4`: "my entry before this one is index 6 with term 4"; a follower accepts only if its own log has that entry. As

soon as one follower plus the leader (a majority of three) hold the entry, the leader advances `commitIndex`, applies the command and answers the client; S3's slower reply is not needed. Followers learn the new commit index on the next AppendEntries (asynchronous to the client) and apply then. Each log is on its node's own disk. Failure points: if the leader crashes after committing but before replying, the client must retry, so commands should be idempotent or deduplicated by client ID and sequence number (§22.15). Scaling: leaders batch many commands per AppendEntries and pipeline requests without waiting for replies.

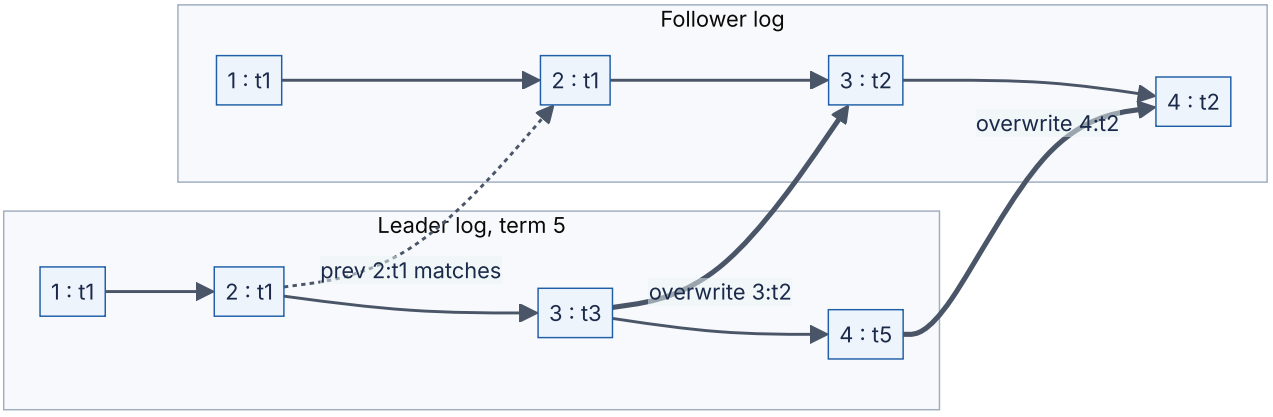
26.7.4 The log matching property

DEFINITION

Definition — Log matching property. If two logs contain an entry with the same index and term, then (1) the entries store the same command, and (2) the logs are identical in all preceding entries.

Part (1) holds because a leader creates at most one entry per index in its term and never changes it. Part (2) is enforced inductively by the **consistency check**: a follower accepts AppendEntries only if it has an entry matching `(prevLogIndex, prevLogTerm)`. When the check fails, the leader decrements `nextIndex` for that follower and retries until they find the last point of agreement; the follower then deletes everything after that point and appends the leader's entries. The leader never deletes or overwrites its own entries (**leader append-only**).

Figure 26.9 — Repairing a divergent follower log.



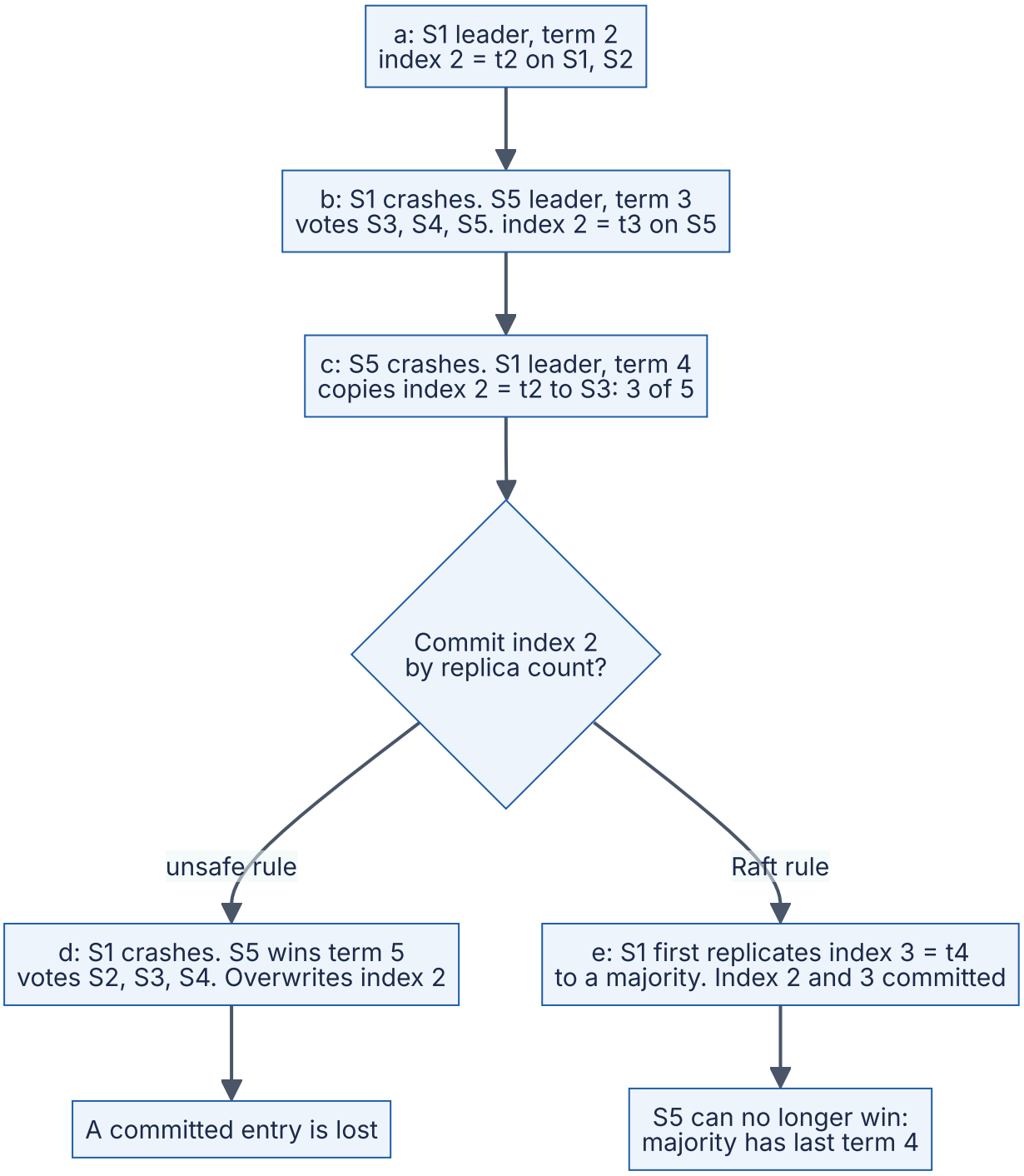
Reading the diagram. Each box is a log entry labeled `index : term`. The follower holds uncommitted entries 3 and 4 from a term-2 leader that crashed. The leader's AppendEntries with `prev 4:t5` fails, then `prev 3:t3` fails, then `prev 2:t1` succeeds (dotted arrow). The follower truncates after index 2 and takes the leader's entries 3 and 4 (thick arrows). Data flows only leader to follower. Failure point: backing off one entry per

round trip is slow for long-lagging followers, so implementations skip a whole conflicting term per round trip or send a snapshot (§26.7.8).

26.7.5 The commit rule and the “current term only” subtlety

A leader marks an entry committed when it is stored on a majority, **but only if the entry is from the leader’s current term**. Entries from earlier terms are never committed by counting replicas; they become committed *indirectly* when a later current-term entry after them commits, by the log matching property. This is the scenario in Figure 8 of the Raft paper.

Figure 26.10 — Why a leader must not commit old-term entries by counting replicas.



Reading the diagram. Read top to bottom. In (c), the term-2 entry at index 2 sits on a majority (S1, S2, S3), so a naive leader would call it committed. But S5's last entry has term 3, "more up-to-date" than term 2, so in (d) S5 wins votes from S2, S3 and S4 and overwrites index 2: the "committed" entry vanishes. Under Raft's rule (e), S1 first puts a term-4 entry on a majority; those nodes refuse to vote for S5, and index 2 commits safely as a prefix of index 3. Hence a new leader immediately appends a **no-op entry** in its term, as the simulator in §26.7.7 does.

26.7.6 Why Raft is safe

The core guarantee is **state machine safety**: if any node has applied a log entry at index i, no node ever applies a different entry at i. It follows from a chain of properties [Ongaro and Ousterhout 2014]:

1. **Election safety.** At most one leader per term, because each node votes once per term and two majorities intersect.
2. **Leader append-only** and **log matching** (§26.7.4).
3. **Leader completeness.** If an entry is committed in term T, it is present in the logs of all leaders of terms greater than T. Sketch: the entry is on a majority M_1; a later winner received votes from a majority M_2; some voter is in both; that voter only votes for a candidate whose log is at least as up-to-date as its own; with the current-term commit rule, "at least as up-to-date" implies the candidate's log contains the entry.
4. Since every future leader contains every committed entry, and followers only take entries from leaders, nodes apply the same entry at each index.

The thesis includes a formal proof of the single-node algorithm, and the authors model-checked a TLA+ specification [Ongaro 2014].

26.7.7 A runnable Raft simulator

The program below implements Raft's election and replication rules in a deterministic, tick-based simulation of five nodes. Messages take 2–8 ms, timeouts are drawn from 150–300 ms, heartbeats are sent every 50 ms (new entries ride on the next heartbeat), and a seeded random generator makes every run reproducible. It omits snapshots, membership changes and client sessions.

```
        """A simplified, deterministic Raft simulator: leader election + log replica-
tion.

        Time advances in 1 ms ticks. Messages take 2-8 ms. Nodes can crash and restart
        (persistent state survives a crash; volatile state does not). Not production
code:
        no snapshots, no membership changes, no client sessions.
        """
        import random

        HEARTBEAT_MS = 50
        ELECTION_MIN_MS, ELECTION_MAX_MS = 150, 300


        class Node:
            def __init__(self, nid, peers, rng):
                self.id, self.peers, self.rng = nid, peers, rng
                # Persistent state (survives crashes)
                self.term, self.voted_for, self.log = 0, None, []   # log entries:
(term, command)
                self.up = True
                self.reset_volatile(now=0)

            def reset_volatile(self, now):
                self.role, self.commit, self.votes = "follower", 0, set()
                self.next_idx, self.match_idx = {}, {}
                self.reset_election_timer(now)

            def reset_election_timer(self, now):
                self.deadline = now + self.rng.randint(ELECTION_MIN_MS, ELECTION_MAX_MS)

            def last_index(self):
                return len(self.log)

            def term_at(self, index):          # log indices are 1-based; index 0 has
term 0
                return self.log[index - 1][0] if index > 0 else 0


        class Cluster:
            def __init__(self, n=5, seed=7):
                self.rng = random.Random(seed)
                ids = list(range(1, n + 1))
                self.nodes = {i: Node(i, [p for p in ids if p != i], self.rng) for i in
ids}
                self.inbox, self.now = [], 0                       # (deliver_at, dst,
msg)

            def log_event(self, text):
                print(f"t={self.now:5d} ms  {text}")

            def send(self, src, dst, msg):
                msg["src"] = src
                self.inbox.append((self.now + self.rng.randint(2, 8), dst, msg))

            # ---------------- message handlers ----------------
```

```
            def step_down_if_stale(self, n, term):
                if term > n.term:
                    n.term, n.voted_for, n.role = term, None, "follower"

            def handle(self, n, m):
                self.step_down_if_stale(n, m["term"])
                kind = m["type"]
                if kind == "RequestVote":
                    up_to_date = (m["last_term"], m["last_index"]) >=
(n.term_at(n.last_index()), n.last_index())
                    grant = (m["term"] == n.term and n.voted_for in (None, m["src"])
and up_to_date)
                    if grant:
                        n.voted_for = m["src"]
                        n.reset_election_timer(self.now)
                    self.send(n.id, m["src"], {"type": "Vote", "term": n.term,
"granted": grant})
                elif kind == "Vote" and n.role == "candidate" and m["term"] == n.term
and m["granted"]:
                    n.votes.add(m["src"])
                    if len(n.votes) > (len(n.peers) + 1) // 2:
                        self.become_leader(n)
                elif kind == "AppendEntries":
                    ok = False
                    if m["term"] == n.term:
                        n.role = "follower"
                        n.reset_election_timer(self.now)
                        prev = m["prev_index"]
                        if prev <= n.last_index() and n.term_at(prev) == m["prev_term"]:
                            ok = True
                            for k, entry in enumerate(m["entries"]):       # log match-
ing: fix conflicts
                                idx = prev + 1 + k
                                if idx <= n.last_index() and n.term_at(idx) != entry[0]:
                                    del n.log[idx - 1:]
                                if idx > n.last_index():
                                    n.log.append(entry)
                            n.commit = max(n.commit, min(m["commit"], prev + len(m["en-
tries"])))
                    self.send(n.id, m["src"], {"type": "AppendReply", "term": n.term,
"ok": ok,
                                               "match": m["prev_index"] + len(m["en-
tries"])})
                elif kind == "AppendReply" and n.role == "leader" and m["term"] ==
n.term:
                    p = m["src"]
                    if m["ok"]:
                        n.match_idx[p] = max(n.match_idx[p], m["match"])
                        n.next_idx[p] = n.match_idx[p] + 1
                        self.advance_commit(n)
                    else:
                        n.next_idx[p] = max(1, n.next_idx[p] - 1)          # back up and
retry

            # ---------------- leader logic ----------------
            def become_leader(self, n):
                n.role = "leader"
```

```
                n.next_idx = {p: n.last_index() + 1 for p in n.peers}
                n.match_idx = {p: 0 for p in n.peers}
                self.log_event(f"n{n.id} becomes LEADER for term {n.term} (votes from
{sorted(n.votes)})")
                n.log.append((n.term, "no-op"))                          # commit
something in our term
                n.hb_due = self.now

            def advance_commit(self, n):
                for idx in range(n.last_index(), n.commit, -1):
                    replicas = 1 + sum(1 for p in n.peers if n.match_idx[p] >= idx)
                    # Commit rule: majority AND entry from the current term (Raft paper,
Figure 8)
                    if replicas > (len(n.peers) + 1) // 2 and n.term_at(idx) == n.term:
                        self.log_event(f"n{n.id} commits up to index {idx} (on {repli-
cas} nodes)")
                        n.commit = idx
                        break

            def replicate(self, n):
                for p in n.peers:
                    prev = n.next_idx[p] - 1
                    self.send(n.id, p, {"type": "AppendEntries", "term": n.term,
"prev_index": prev,
                                        "prev_term": n.term_at(prev), "entries":
n.log[prev:],
                                        "commit": n.commit})

            def client_write(self, command):
                leader = next((n for n in self.nodes.values() if n.up and n.role ==
"leader"), None)
                if leader is None:
                    self.log_event(f"client write {command!r} rejected: no leader")
                    return
                leader.log.append((leader.term, command))
                self.log_event(f"client write {command!r} -> leader n{leader.id} at in-
dex {leader.last_index()}")

            # ---------------- main loop ----------------
            def tick(self):
                self.now += 1
                due = [x for x in self.inbox if x[0] <= self.now]
                self.inbox = [x for x in self.inbox if x[0] > self.now]
                for _, dst, msg in sorted(due, key=lambda x: (x[0], x[1])):
                    if self.nodes[dst].up and self.nodes[msg["src"]].up:   # crashed
nodes drop traffic
                        self.handle(self.nodes[dst], msg)
                for n in self.nodes.values():
                    if not n.up:
                        continue
                    if n.role == "leader" and self.now >= n.hb_due:
                        self.replicate(n)
                        n.hb_due = self.now + HEARTBEAT_MS
                    elif n.role != "leader" and self.now >= n.deadline:
                        n.role, n.term, n.voted_for, n.votes = "candidate", n.term + 1,
n.id, {n.id}
                        n.reset_election_timer(self.now)
```

```
                        self.log_event(f"n{n.id} times out, starts election for term
{n.term}")
                        for p in n.peers:
                            self.send(n.id, p, {"type": "RequestVote", "term": n.term,
                                                "last_index": n.last_index(),
                                                "last_term": n.term_at(n.last_index())})

            def run_until(self, t):
                while self.now < t:
                    self.tick()

            def crash(self, nid):
                self.nodes[nid].up = False
                self.log_event(f"n{nid} CRASHES")

            def restart(self, nid):
                n = self.nodes[nid]
                n.up = True
                n.reset_volatile(self.now)
                self.log_event(f"n{nid} restarts as follower (term {n.term}, log length
{n.last_index()})")


        def check_safety(cluster):
            """State machine safety: committed prefixes on all nodes must agree."""
            committed = [n.log[:n.commit] for n in cluster.nodes.values()]
            longest = max(committed, key=len)
            return all(c == longest[:len(c)] for c in committed)


        def failover_experiment(runs=200):
            """Crash the leader in many seeded runs; measure failover time and check
safety."""
            import contextlib, io
            times, violations = [], 0
            for seed in range(1, runs + 1):
                with contextlib.redirect_stdout(io.StringIO()):          # silence per-
run events
                    c = Cluster(n=5, seed=seed)
                    c.run_until(400)
                    for k in range(6):
                        c.client_write(f"op{k}")
                        c.run_until(c.now + 60)
                    old = next(n for n in c.nodes.values() if n.role == "leader")
                    crashed_at = c.now
                    c.crash(old.id)
                    while not any(n.up and n.role == "leader" for n in
c.nodes.values()):
                        c.tick()
                    times.append(c.now - crashed_at)
                    c.run_until(c.now + 300)
                    c.restart(old.id)
                    c.run_until(c.now + 400)
                violations += not check_safety(c)
            times.sort()
            print(f"{runs} seeded runs: safety violations={violations}, failover ms: "
                  f"median={times[runs // 2]}, p95={times[int(runs * 0.95)]}, max=
```

```
{times[-1]}")


        if __name__ == "__main__":
            c = Cluster(n=5, seed=7)
            c.run_until(400)
            for cmd in ["x=1", "y=2"]:
                c.client_write(cmd)
            c.run_until(500)
            old = next(n.id for n in c.nodes.values() if n.role == "leader")
            c.crash(old)
            c.run_until(900)
            c.client_write("x=3")
            c.run_until(1000)
            c.restart(old)
            c.run_until(1200)
            print()
            for n in c.nodes.values():
                print(f"n{n.id} {n.role:9s} term={n.term} commit={n.commit} "
                      f"log={[f'{t}:{cmd}' for t, cmd in n.log]}")
            print("state machine safety holds:", check_safety(c))
            failover_experiment()
```

Running it prints the following (deterministic for seed 7):

```
t=  162 ms  n4 times out, starts election for term 1
t=  167 ms  n4 becomes LEADER for term 1 (votes from [1, 4, 5])
t=  172 ms  n4 commits up to index 1 (on 3 nodes)
t=  400 ms  client write 'x=1' -> leader n4 at index 2
t=  400 ms  client write 'y=2' -> leader n4 at index 3
t=  426 ms  n4 commits up to index 3 (on 3 nodes)
t=  500 ms  n4 CRASHES
t=  638 ms  n3 times out, starts election for term 2
t=  648 ms  n3 becomes LEADER for term 2 (votes from [2, 3, 5])
t=  662 ms  n3 commits up to index 4 (on 3 nodes)
t=  900 ms  client write 'x=3' -> leader n3 at index 5
t=  956 ms  n3 commits up to index 5 (on 3 nodes)
t= 1000 ms  n4 restarts as follower (term 1, log length 3)

n1 follower  term=2 commit=5 log=['1:no-op', '1:x=1', '1:y=2', '2:no-op', '2:x=3']
n2 follower  term=2 commit=5 log=['1:no-op', '1:x=1', '1:y=2', '2:no-op', '2:x=3']
n3 leader    term=2 commit=5 log=['1:no-op', '1:x=1', '1:y=2', '2:no-op', '2:x=3']
n4 follower  term=2 commit=5 log=['1:no-op', '1:x=1', '1:y=2', '2:no-op', '2:x=3']
n5 follower  term=2 commit=5 log=['1:no-op', '1:x=1', '1:y=2', '2:no-op', '2:x=3']
state machine safety holds: True
200 seeded runs: safety violations=0, failover ms: median=155, p95=276, max=360
```

What to notice:

- **Persistent versus volatile state.** `term`, `voted_for` and `log` survive `crash()`; `reset_volatile` wipes role, commit index and leader bookkeeping, as a real restart would. n4 rejoins with term 1, adopts term 2 from the first message it sees, and is brought up to date by the consistency check.

- **The up-to-date check** is one tuple comparison, `(last_term, last_index) >= (...)`, which compares terms first and indices second.
- **The commit rule** in `advance_commit` requires both a majority and `term_at(idx) == n.term`; each new leader appends a `no-op` so earlier entries commit through it (index 4 commits indices 1–4 for n3).
- **Writes wait for the next heartbeat** (400 ms write, 426 ms commit), a simplification; real leaders send AppendEntries immediately and batch.
- **Failover statistics** (median 155 ms) match the formula in §26.7.2. Change `ELECTION_MIN_MS` or the message delays and rerun to see the trade-offs.

26.7.8 Snapshots and log compaction

A log that grows forever exhausts disk and makes restarts replay hours of history. Each node independently takes a **snapshot** of its state machine at some applied index, records the `lastIncludedIndex` and `lastIncludedTerm` (needed for the consistency check of the next entry), and discards the log prefix. When a follower is so far behind that the leader has already discarded the entries it needs, the leader sends **InstallSnapshot** chunks instead of AppendEntries. Snapshotting must not block the state machine for long: implementations use copy-on-write structures or the storage engine's own consistent snapshot (etcd snapshots its bbolt database; CockroachDB and TiKV ship RocksDB-family SST files). Snapshot frequency is a trade-off between disk space and the cost of each snapshot; etcd, for example, triggers one after a configurable number of applied entries.

26.7.9 Membership changes

Changing the voter set changes the quorum definition. Switching all nodes at once is unsafe: during the transition some nodes count old-configuration majorities and others new ones, and two disjoint majorities could elect two leaders in one term.

- **Joint consensus** (Raft paper). The leader first commits a configuration entry $C_{\text{old,new}}$; while it is in effect, every election and every commit needs a majority of C_{old} *and* a majority of C_{new}. Once $C_{\text{old,new}}$ is committed, the leader commits C_{new}. Any change, including replacing several nodes at once, is safe.
- **Single-server changes** (thesis). If you add or remove only *one* voter at a time, every majority of the old configuration intersects every majority of the new one, so no joint phase is needed. Each configuration takes effect as soon as it is appended (not committed), and only one change may be in progress. In 2015 Ongaro reported a bug in the original single-server description that could lose a committed entry when changes overlapped with leader changes; the fix is that a new leader must commit an entry in its own term (the same no-op as in §26.7.5) before starting a membership change.

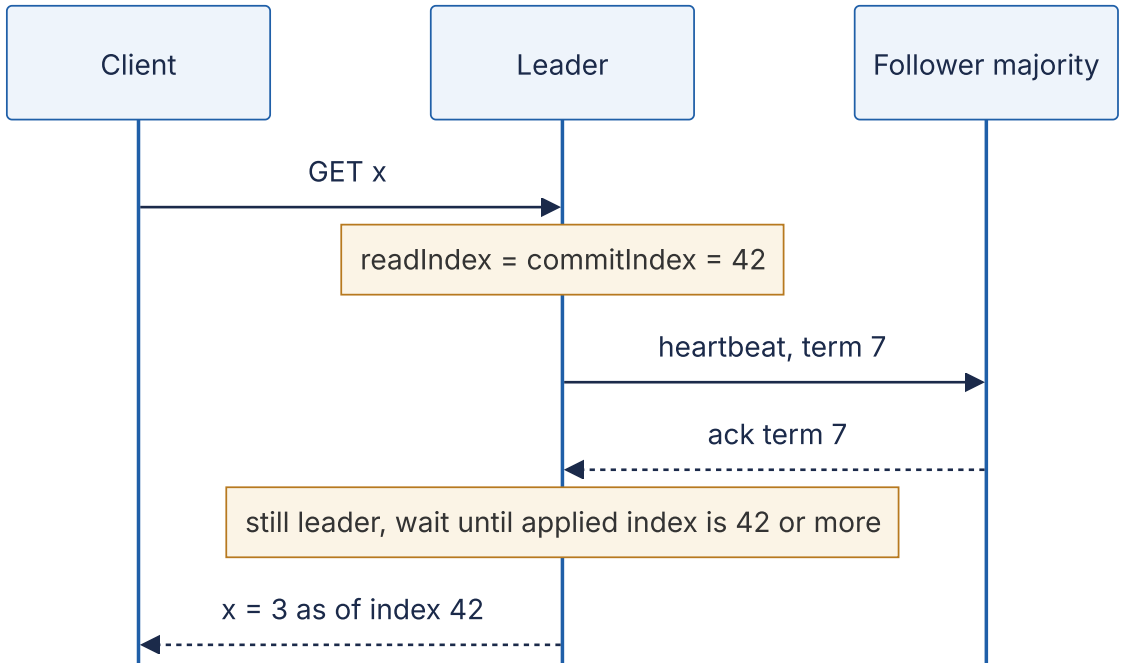
Production systems add **learners** (etcd) or observers (ZooKeeper): a new node catches up without affecting the quorum and is promoted when current. Otherwise adding an empty fourth node (quorum 3) plus one unrelated failure blocks all writes.

26.7.10 Linearizable reads: ReadIndex and lease reads

A read from the leader's local state is *not* automatically linearizable (§25.3): a partitioned leader may not know it was replaced and return a value already overwritten. Three remedies exist [Ongaro 2014]:

1. **Log reads.** Append the read to the log like a write. Correct, but costs a disk write per read.
2. **ReadIndex.** The leader records its current `commitIndex` as the read index, confirms it is still leader by exchanging a heartbeat round with a majority, waits until its state machine has applied up to the read index, then answers locally. Cost: one round trip, no disk write. Followers can serve reads the same way by asking the leader for a read index. A new leader must first commit its no-op entry, or its `commitIndex` may be stale.
3. **Lease reads.** After a successful heartbeat round started at time t, the leader assumes no other leader can exist before $t + T_{\text{min}}$ (followers will not start an election sooner), shortened by a clock-drift margin. Within that window it answers reads with no network round trip. This trades a timing assumption for latency: a long pause on the leader, or clock rates that differ more than the margin, can return stale data.

Figure 26.11 — ReadIndex: linearizable reads without writing to the log.



Reading the diagram. The leader notes the commit index (42) when the read arrives. The synchronous heartbeat round to a majority proves no newer leader existed then: a majority would have a higher term and reject it. The leader waits to apply index 42, so the read reflects every earlier committed write, and replies from memory. Nothing is written to disk. Failure point: if the heartbeat round fails, the leader aborts the read rather than risk staleness. Scaling: one heartbeat round can confirm leadership for a whole batch of pending reads.

etcd uses ReadIndex for its default linearizable reads and also offers **serializable** reads, served from any member's local state without a round trip, which may be stale [etcd Docs: API guarantees].

Failure scenarios

Table 26.3 — Raft failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Repeated elections	Timeout below network, GC or `fsync` jitter	Term climbing	Raise timeout; faster dedicated disks	Size timeouts from p99.9 RTT and flush latency
Disruptive rejoining node	A partitioned node's inflated term forces the leader to step down	Term jumps after healing	Automatic, after an extra election	**Pre-vote** [Ongaro 2014]: campaign only if a majority would vote
Writes blocked	Quorum lost	Commit index stalls	Restore nodes; else rebuild from backup, accepting loss	5 voters across 3 zones; learners
Stale reads	Local reads on a deposed leader	Jepsen-style tests (§25.11)	Switch to ReadIndex	ReadIndex, or lease reads with margin
Data loss after crash	Acknowledged before `fsync`	Divergent committed logs	Rebuild replica from peers	Never disable `fsync`; checksums

26.8 ZooKeeper and ZAB

26.8.1 The coordination-kernel idea

DEFINITION

Definition — ZooKeeper. A replicated coordination service that exposes a small, file-system-like tree of data nodes (**znodes**) with linearizable writes, FIFO per-client ordering, ephemeral and sequential nodes, and change notifications (**watches**). Applications build locks, leader election, barriers and configuration management on top of these primitives [Hunt et al. 2010].

Before ZooKeeper (built at Yahoo!, now an Apache project), each distributed system wrote its own, usually buggy, election and membership code. ZooKeeper deliberately offers no locks; it offers a **wait-free** API (no operation blocks on another client) from which clients compose **recipes**.

How it works internally

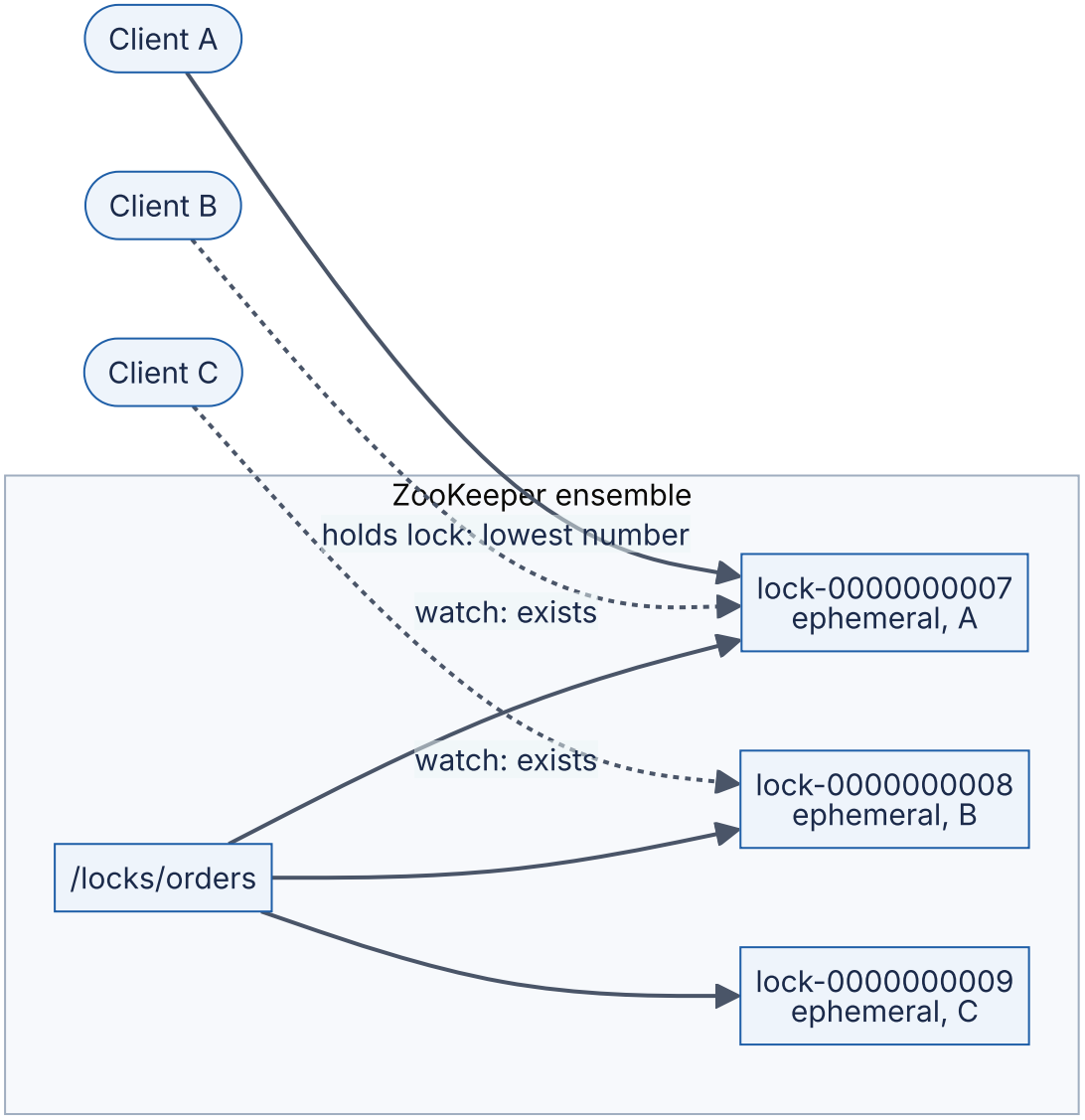
- **Znodes.** Each holds a small blob of data (the default limit is about 1 MB; coordination data should be bytes to kilobytes) plus a version number used for conditional updates (`setData(path, data, expectedVersion)`).
- **Ephemeral znodes** exist only while the client **session** that created them is alive. A session is kept alive by heartbeats; if the server ensemble hears nothing for the session timeout, it expires the session and deletes its ephemeral nodes. This is ZooKeeper's failure detector.
- **Sequential znodes** get a monotonically increasing 10-digit suffix from their parent, for example `lock-0000000007`, giving a total order among competing clients.
- **Watches** are one-time triggers set by a read (`getData`, `exists`, `getChildren`); the server sends one notification when the node changes, and the client must re-read and re-arm. Version 3.6 added persistent and recursive watches.
- **Ordering guarantees.** Writes are linearizable and go through the leader. Reads come from the connected server's local replica, fast but possibly stale; call `sync()` first to read the latest. Each client's operations execute in issue order, and a client sees a watch notification before the data that triggered it.
- **Observers** are non-voting servers that receive committed updates and serve reads, adding read capacity without slowing writes.

26.8.2 ZAB: ZooKeeper Atomic Broadcast

ZooKeeper replicates its tree with **ZAB**, an atomic broadcast protocol designed for primary–backup systems [Junqueira, Reed and Serafini 2011]. Each update is a state change produced by the leader (not the client's original request), labeled with a 64-bit **zxid**: the high 32 bits are the leader's **epoch**, the low 32 bits a counter. In the broadcast phase the leader sends a proposal, followers write it to disk and acknowledge, and the leader commits once a quorum has acknowledged, much like Raft's AppendEntries. After a leader failure, ZAB runs **discovery** (the new leader learns the latest epoch and history from a quorum) and **synchronization** (it brings followers to its history) before broadcasting again. ZAB's distinguishing guarantee is **primary order**: all of a leader's changes are delivered in order, and a new leader's changes come after every change from previous leaders that was delivered. ZooKeeper's paper reports that read-dominated workloads reach tens to hundreds of thousands of operations per second [Hunt et al. 2010].

26.8.3 Recipes: leader election, locks and barriers

Figure 26.12 — The ZooKeeper lock recipe with sequential ephemeral znodes.



Reading the diagram. Each client creates an ephemeral sequential child of `/locks/orders` (synchronous write through the ZooKeeper leader) and lists the children. A's node has the lowest number, so A holds the lock. B and C do not poll: each sets a watch (dotted, asynchronous notification) on the node *immediately below its own*. When A releases the lock by deleting its node, or A's session expires and ZooKeeper deletes it, only B is notified; B re-lists the children, sees it is lowest, and proceeds. Watching only the predecessor avoids the **herd effect** (every waiter waking on every release). The waiter queue lives inside the replicated tree. Failure point: A can be paused past its session timeout and still believe it holds the lock; the node's creation zxid should be passed to the protected resource as a fencing token (§26.13).

- **Leader election** is the same recipe: the candidate with the lowest sequential node under `/election` is leader, and each other candidate watches its predecessor.
- **Barrier.** Workers wait until a node `/barrier/ready` exists (watch on `exists`); the coordinator creates it to release everyone. A **double barrier** has workers register ephemeral children on entry and waits until the count reaches *k*, then waits for all to leave.
- **Configuration and group membership.** Store configuration in a znode and watch it; each live member registers an ephemeral child under `/members`.

> INDUSTRY EXAMPLE
>
> **Apache HBase, Hadoop HDFS and Kafka.** HBase uses ZooKeeper to elect its active master and track live region servers; HDFS high availability uses it to elect and fence the active NameNode (§15.5.2). Kafka historically stored cluster metadata and its controller election in ZooKeeper, then replaced it with KRaft, a Raft-based metadata quorum; Kafka 4.0 (2025) removed ZooKeeper mode entirely (Chapter 31). The trend reflects operational cost: running a separate ZooKeeper ensemble alongside every cluster was a frequent source of outages and configuration mistakes. *Evidence: Primary* — [Apache HBase Reference Guide]; [Apache Kafka KIP-500]

26.9 etcd

etcd is a distributed key–value store replicated with Raft, created at CoreOS and now a Cloud Native Computing Foundation project. It is best known as the database behind Kubernetes [etcd Docs].

26.9.1 Data model: revisions, watches, leases and transactions

- **Flat, ordered key space.** Keys are byte strings; range queries over prefixes replace ZooKeeper's tree.
- **MVCC and revisions.** The store keeps a single cluster-wide **revision** counter that increases with every modification. Each key records `create_revision`, `mod_revision` and `version`. You can read any key "as of" an older revision until **compaction** removes history older than a chosen revision.
- **Watches** start from a revision and stream every subsequent change, so a client that reconnects at revision *r* misses nothing (unless *r* was compacted). This fixes ZooKeeper's classic one-shot watch gap.
- **Leases.** A lease has a TTL; keys can be attached to it; the client renews with `LeaseKeepAlive`. When the lease expires, all its keys are deleted, the equivalent of ephemeral znodes.
- **Transactions.** `Txn(If compare..., Then ops..., Else ops...)` executes atomically, enabling compare-and-swap on a key's value, version or revision.

- **Consistency.** Writes are linearizable through Raft; reads are linearizable by default (ReadIndex, §26.7.10) or serializable on request.

The `concurrency` package of the Go client implements election and mutexes from these primitives: each contender writes a key under a common prefix attached to its lease, and the contender whose key has the lowest `create_revision` holds the lock; the others watch the key just before theirs, mirroring Figure 26.12. That revision is a natural fencing token.

26.9.2 etcd in Kubernetes

The Kubernetes API server is etcd's only client. Objects live under keys such as `/registry/pods/<namespace>/<name>`, `resourceVersion` derives from etcd's revision, and every controller's **watch** is an etcd watch multiplexed by the API server. etcd's health defines the control plane's; Kubernetes documentation recommends running an odd number of members (typically 3 or 5) on dedicated, low-latency disks [Kubernetes Docs: etcd]. The default storage quota is 2 GB; keeping the dataset small (compaction, defragmentation, no large blobs) is a standing task.

> INDUSTRY EXAMPLE
>
> **Kubernetes controller leader election.** Only one replica of `kube-controller-manager` or `kube-scheduler` acts at a time. Rather than etcd leases, they compete to update a `coordination.k8s.io Lease` object through the API server, with optimistic concurrency on `resourceVersion`. The defaults are a 15 s lease duration, a 10 s renew deadline and a 2 s retry period, so a crashed leader is replaced in roughly 15–17 s. Trade-off: this is coarse failover, chosen to avoid churn, and it relies on clocks running at similar rates. *Evidence: Primary* — [Kubernetes Docs: Leases]; [Kubernetes Docs: kube-controller-manager]

26.10 Distributed locks

26.10.1 Why distributed locks are hard

A single-process lock is simple because the kernel sees every thread. A distributed lock has no such comfort:

1. **The holder can fail silently.** So locks need an expiry (a lease), or a crashed client holds the lock forever.
2. **Expiry means the holder can lose the lock without knowing.** A process paused for 20 s (garbage collection, VM migration, page faults on a swapped heap, a `SIGSTOP`) resumes believing it still holds a lock that expired 10 s ago.

3. **Messages are delayed.** A write sent while holding the lock can arrive at the storage system after the lock has passed to someone else.

Kleppmann distinguishes two purposes [Kleppmann 2016]. An **efficiency lock** prevents duplicate work (two workers computing the same report); an occasional double holder costs some wasted CPU. A **correctness lock** protects an invariant (two writers corrupting one file); a double holder causes data loss. The right design depends on which you have.

26.10.2 The Redlock debate

The Redis documentation describes **Redlock**, an algorithm for a lock across *N* independent Redis primaries (typically 5) with no replication between them [Redis Docs: Distributed Locks]:

1. Record the current time. Try to acquire the lock on each instance in turn with `SET resource random_value NX PX ttl` and a short per-instance timeout.
2. The lock is acquired if a majority (3 of 5) succeeded and the total elapsed time is less than the TTL. The remaining **validity time** is the TTL minus the elapsed time minus an allowance for clock drift.
3. Otherwise, release on all instances (a script deletes the key only if it still holds `random_value`) and retry after a random delay.

In 2016 Martin Kleppmann published “How to do distributed locking,” arguing that Redlock is unsuitable for correctness locks [Kleppmann 2016]. Salvatore Sanfilippo (antirez), Redis’s creator, replied in “Is Redlock safe?” [Sanfilippo 2016]. The debate is a good model of how to reason about timing assumptions, so both sides are summarized here.

Table 26.4 — The Redlock debate in brief.

Issue	Kleppmann's argument	antirez's reply
Paused or delayed clients	Any lease-based lock can expire while the holder is paused; Redlock offers no **fencing token** (the random value is not monotonic), so storage cannot reject the stale holder	This affects every lock with auto-release; Redlock checks elapsed time after acquisition, and the unique random value can be used for check-and-set at the resource
Timing assumptions	Safety depends on bounded network delay, bounded pauses and bounded clock drift; a clock jump on one instance (for example, an NTP step) can let two clients hold a majority	Redlock needs bounded clock *rate* error, not synchronized clocks; operators should configure time sync to slew, not step
What to use instead	For efficiency, a single Redis node is enough; for correctness, use a consensus system (ZooKeeper, etcd) **with fencing tokens**	If the resource can check tokens, it is already a linearizable store; Redlock with its assumptions is adequate for many uses

A fair reading: both agree an auto-expiring lock cannot alone protect a resource from a paused holder; they disagree about how realistic the timing failures are. Redlock is reasonable as an efficiency lock, though one Redis instance is simpler. For correctness, use a consensus-backed lock and pass a monotonic token to the resource, or make the resource itself the arbiter (a conditional write such as compare-and-set on a version).

RECOMMENDATION

Recommendation — Choosing a lock. (1) Can you avoid the lock? An idempotent operation, a conditional write or a single-writer partition (§14.6) is usually better. (2) If you need a lock for efficiency, a single Redis `SET NX PX` with a sensible TTL is fine. (3) If you need it for correctness, use etcd, ZooKeeper or Consul, and make the protected resource check a fencing token (§26.13). (4) Never hold a lock across an unbounded operation without renewing it and checking validity before each side effect.

26.11 Leases

26.11.1 Leases and clocks

DEFINITION

Definition — Lease. A time-bounded grant of a right (to cache data, to act as leader, to hold a lock) from a grantor to a holder. Within the term the holder may act without consulting the grantor; after the term, the right lapses automatically, even if the holder has crashed or is unreachable [Gray and Cheriton 1989].

First-principles explanation

A lease is a library loan: you keep the book until the due date without phoning in, and if you vanish the loan simply ends. Gray and Cheriton introduced leases for cache consistency in the V file system: a server could update a file once every client lease on it expired, even if a client was unreachable. Leases turn “wait forever” into “wait at most T,” at the price of depending on clocks: holder and grantor each measure T with their own oscillator.

Mathematical foundations

The holder should start its timer *before* sending the request (so network delay counts against it), and the grantor starts its timer when it grants. Both clocks may run fast or slow relative to real time by at most a rate ρ.

FORMULA

Lease safety margin

$$T_{\text{holder}} = T(1-\rho) - d_{\max} \qquad T_{\text{grantor}} = T(1+\rho)$$

Symbol	Meaning	Units
T	Nominal lease duration	s
ρ	Maximum clock rate error relative to real time (drift bound)	dimensionless (e.g. 10^{-4} = 100 ppm)
$d_{\max}$	Maximum time between the holder checking validity and its action taking effect (processing, network, budgeted pause)	s
T_{holder}	Local elapsed time after which the holder must stop acting	s
T_{grantor}	Local elapsed time after grant before the grantor may grant the right to someone else	s

WORKED EXAMPLE

Worked example — a 10 s leader lease. $T = 10$ s, $\rho = 10^{-4}$ (100 ppm, a generous bound for server oscillators), $d_{\max} = 0.5$ s.

1. Holder: $T(1-\rho) = 10 \times 0.9999 = 9.999$ s; minus $d_{\max}$: $9.999 - 0.5 = 9.499$ s.
2. Grantor: $T(1+\rho) = 10 \times 1.0001 = 10.001$ s.
3. Real-time gap: the holder stops at most $9.499/(1-\rho) + 0.5 \approx 10.0$ s after its request was sent; the grantor reassigns no earlier than $10.001/(1+\rho) = 10.0$ s after granting, which is later than the request was sent. The intervals do not overlap.
4. Renewal: renew about every $T/3$ (3.3 s), so two consecutive renewals can fail before the lease lapses at 9.5 s; renewing every $T/2$ (5 s) tolerates only one.

Interpretation. Clock drift contributes only about 1 ms of margin over 10 s; the 500 ms action budget dominates. **Engineering implication.** The real enemy is not drift but **unbounded pauses**: a 15 s GC pause between the validity check and the write blows through any margin. That is why leases need fencing tokens (§26.13) whenever the action has effects outside the holder's process, and why clock *steps* (an NTP correction that jumps time) must be prevented on lease holders.

Leases appear throughout this book: GFS chunk leases (§15.5.1), Chubby sessions (§26.14), etcd and Consul session TTLs, Raft lease reads (§26.7.10), Kubernetes `Lease` objects (§26.9.2) and the idempotency-key claims of §22.15.

26.12 Split brain

26.12.1 Causes and prevention

DEFINITION

Definition — Split brain. A failure in which two or more parts of a cluster each believe they are the authoritative primary (or leader) and accept conflicting writes, so the system's state diverges.

Causes

- **Failover without a quorum.** A two-node pair loses its heartbeat link and each side promotes itself; neither can tell a partition from a crash.
- **A paused primary** resumes after being replaced and keeps serving (the chapter opening).
- **Asymmetric partitions.** A reaches B but not vice versa, so they disagree about who is alive.

- **Operator error.** Promoting a replica while the old primary is still reachable.

Prevention

1. **Quorum.** Only a side holding a majority may act; the minority stops.
2. **Witness (tie-breaker).** Two-site deployments add a lightweight third vote in a third location (a file-share or cloud witness in Windows Server Failover Clustering, an arbiter in MongoDB, a third etcd member), so one site can form a majority.
3. **STONITH ("shoot the other node in the head").** Before taking over, the survivor *forcibly* powers off or isolates the old primary through an out-of-band channel: a power distribution unit, a server management controller (IPMI), or a cloud API that stops the instance. Brutal, but it turns "maybe still running" into "definitely off."
4. **Resource-level fencing.** Revoke the old primary's access to shared storage (SCSI persistent reservations, revoking a volume attachment) or have the resource reject stale epochs (§26.13).

INDUSTRY EXAMPLE

GitHub, October 2018. A 43-second loss of connectivity between GitHub's East Coast network hub and its primary East Coast data center led the cluster-management tool Orchestrator, which uses Raft among its own nodes, to promote MySQL primaries in the West Coast data center. Writes accepted on the East Coast during the brief window had not replicated West, so both sites now held writes the other lacked; GitHub chose data integrity over availability and spent about 24 hours reconciling and restoring service. The lesson: consensus among the *failover controllers* does not prevent divergence in the *data* they manage, and topology-aware failover policies matter. *Evidence: Primary* — [GitHub 2018]

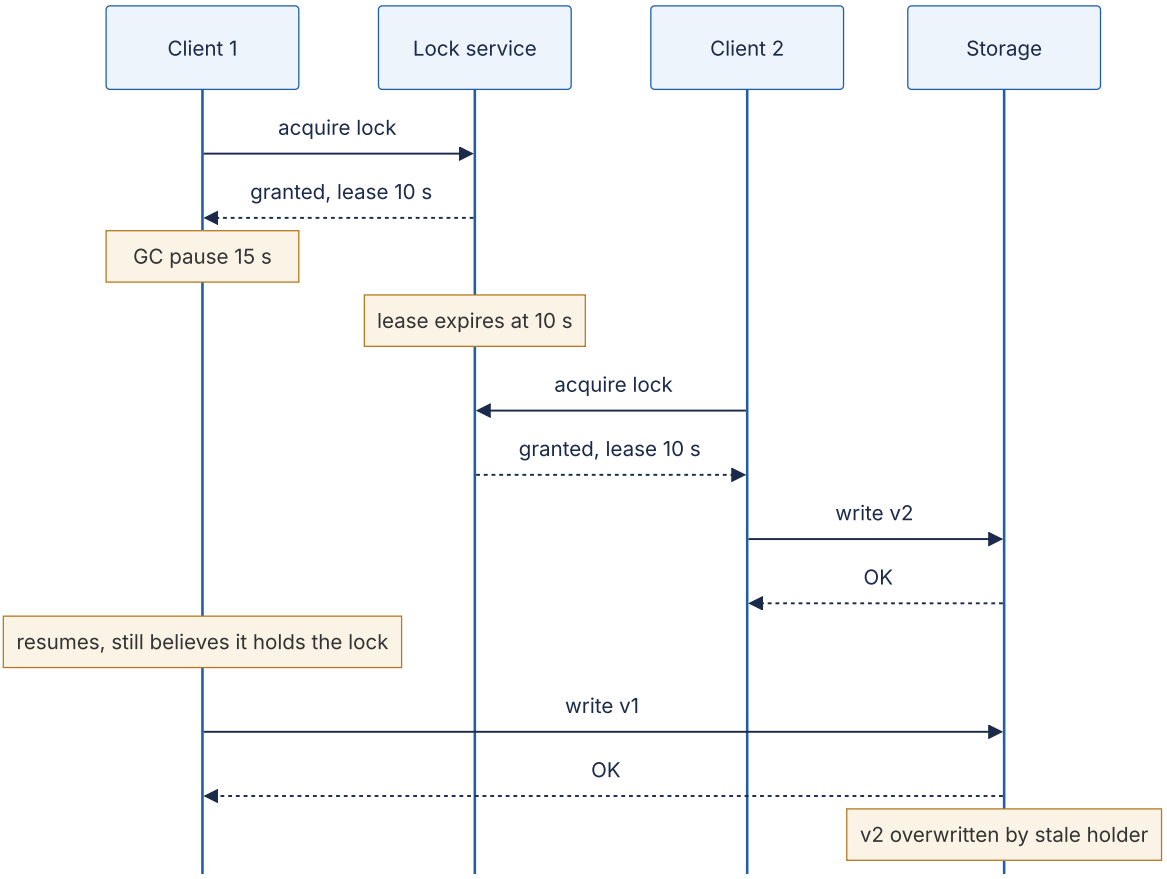
26.13 Fencing tokens

DEFINITION

Definition — Fencing token. A number, increasing with every grant of a lock or lease, that the holder attaches to every request to the protected resource. The resource remembers the highest token it has accepted and rejects any request carrying a lower one.

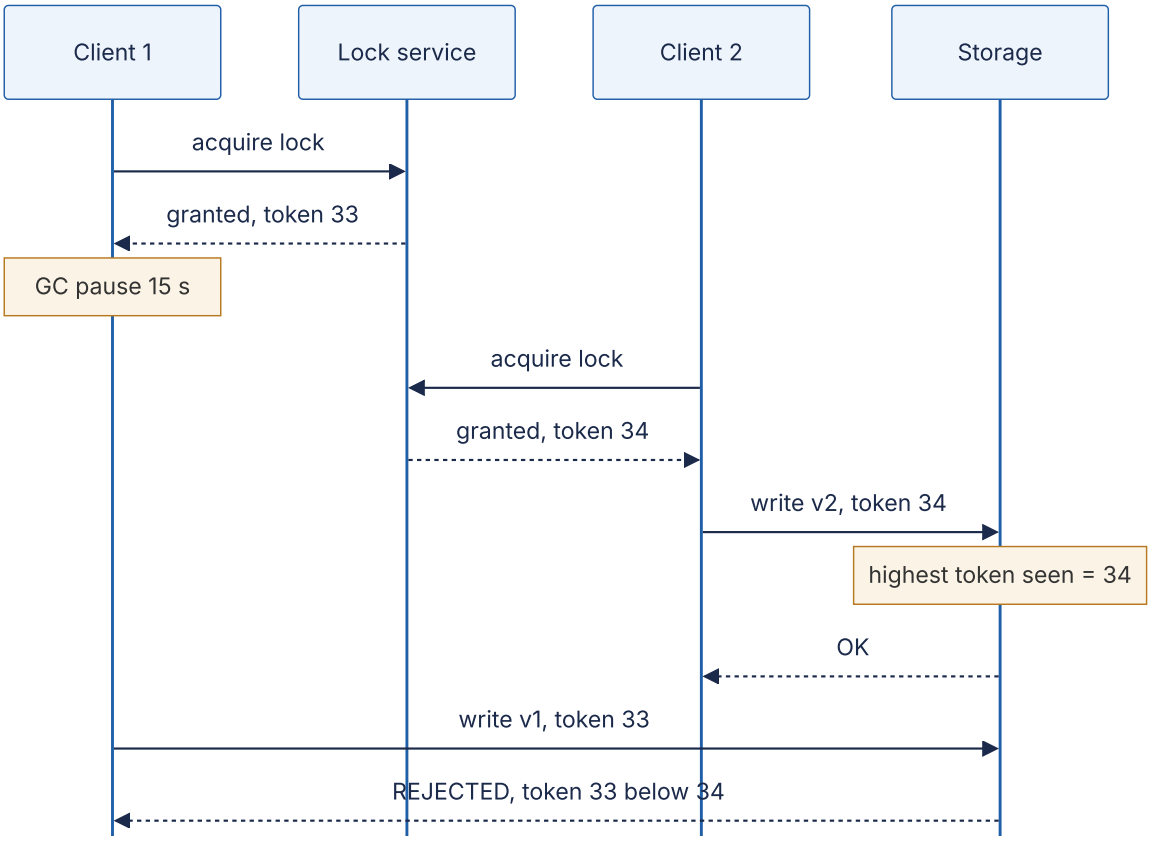
Fencing moves the final safety check from the client, which cannot know it has been paused, to the resource, which sees requests in the order they actually arrive. The two diagrams below follow Kleppmann's example [Kleppmann 2016].

Figure 26.13 — Without fencing: a paused client corrupts storage.



Reading the diagram. All arrows are synchronous RPCs. Client 1 holds a valid lease, then stops for 15 s without being able to observe its pause. The lock service correctly grants the lock to Client 2, which writes. Client 1 resumes and writes; storage cannot distinguish this from a legitimate write. Every component behaved correctly, yet Client 2's update is lost. The failure point is the gap between "checked the lock" and "write took effect," which no lock service can close from outside the client.

Figure 26.14 — With fencing: the storage service rejects the stale holder.



Reading the diagram. The lock service now hands out increasing tokens (33, then 34). Storage records the highest token it has accepted, durably and atomically with the write. Client 1's late write carries 33, which is lower than 34, so it is rejected and Client 1 learns it lost the lock. Ordering is decided at the storage boundary, where requests serialize. Failure points: storage must check and record tokens atomically, and every path to the resource must carry the token. Had Client 1's write arrived *before* Client 2's, accepting it would be correct.

Good token sources are values the consensus system already increments: a Raft or ZAB index, a ZooKeeper zxid or znode version, an etcd revision, a Chubby **sequencer**, or a Kubernetes-style generation number. If the resource cannot check tokens (a legacy file server, an external payment API), fencing is impossible and you must fall back to idempotency keys (§22.15), STONITH or long lock-delays, accepting the residual risk.

COMMON MISTAKE

Misconception — "A random lock value works as a fencing token." A random value lets the holder release only its own lock, but storage cannot tell whether value `a9f3` is newer or older than `71c2`. Fencing needs a *monotonic* order. The alternative is for the resource to support compare-and-set against the exact current owner value, which makes the resource itself the lock.

26.14 Coordination services in practice

26.14.1 Chubby

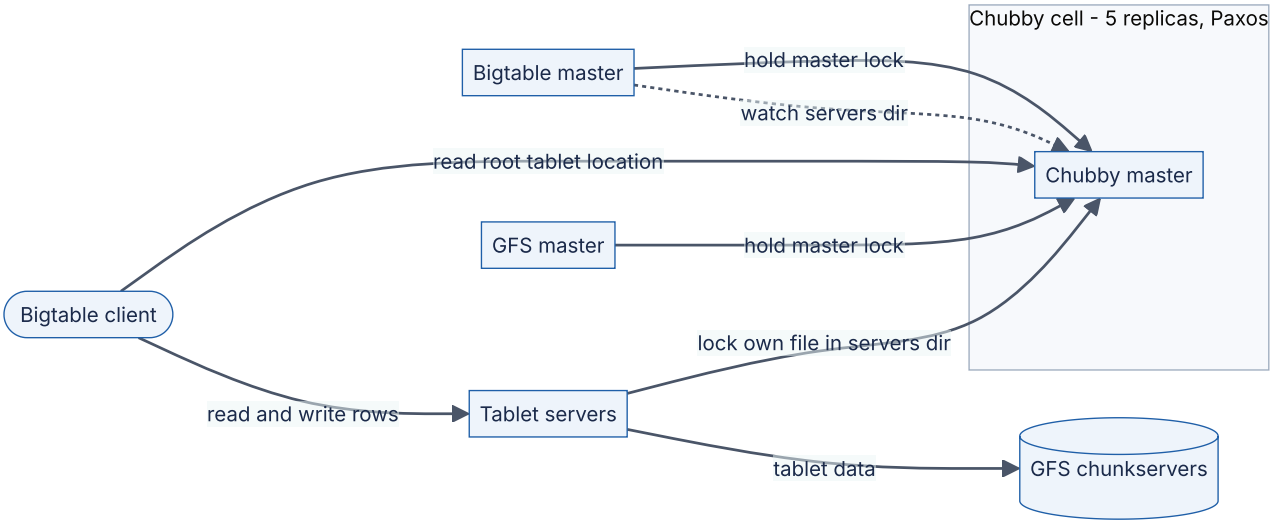
DEFINITION

Definition — Chubby. Google's lock service, providing coarse-grained advisory locks and small files in a file-system-like namespace, replicated across a five-node **cell** with Paxos, and used chiefly for leader election, naming and storing small amounts of configuration [Burrows 2006].

Burrows chose a *lock service* over a consensus library deliberately: taking a lock and writing a file was far easier to add to existing programs than embedding Paxos. Key design points:

- **Coarse-grained locks**, held for hours or days, not milliseconds, so load is low and brief outages do not disturb current holders.
- **Sessions and KeepAlives.** Each client session has a lease; the master holds each KeepAlive RPC until the lease nears expiry, then replies with a new lease (12 s by default) plus cache invalidations and events. If the lease lapses, the client enters **jeopardy**, disables its cache, and waits a **grace period** (45 s by default) for a new master before declaring the session lost.
- **Master election and master leases.** Replicas elect a master through Paxos; the master holds a lease of a few seconds during which no other master can be elected, letting it serve reads locally.
- **Consistent client caching** with invalidation: the master blocks a write until every caching client has invalidated, so reads are almost always local.
- **Sequencers and lock-delay.** A lock holder can obtain a **sequencer** (lock name, mode and generation number) and pass it to servers, which check it with Chubby: fencing tokens under another name. For servers that cannot check sequencers, Chubby imposes a **lock-delay** (up to one minute) before a lock released by a failed holder can be re-acquired.

Figure 26.15 — How Bigtable and GFS use Chubby.



Reading the diagram. Solid arrows are synchronous RPCs; the dotted arrow is an asynchronous event subscription. Bigtable's and GFS's masters each hold an exclusive lock so at most one master acts. Each tablet server locks a uniquely named file in a servers directory; the Bigtable master watches it to discover servers and, when one loses its lock, deletes its file and reassigns its tablets. Clients read the root tablet's location from Chubby, then talk to tablet servers, which store data in GFS; Chubby is off the data path. Failure point: if Chubby is unavailable for long, Bigtable becomes unavailable; the Bigtable paper measured this effect as small but nonzero [Chang et al. 2006]. Scaling: clients cache Chubby data, and Chubby proxies can aggregate KeepAlives.

INDUSTRY EXAMPLE

Chubby as Google's name service. Burrows reports that Chubby's most popular use was not locking but naming: storing small files mapping service names to addresses, where Chubby's consistent caching outperformed time-based DNS caching. Trade-off: this concentrated a great deal of read load on one service, handled by caching and proxies. *Evidence: Primary* — [Burrows 2006]

26.14.2 Choosing a coordination service

Table 26.5 — ZooKeeper vs etcd vs Consul vs Chubby (as of 2026).

	ZooKeeper	etcd	Consul	Chubby
What it is	Apache coordination service; hierarchical znodes	CNCF key–value store; Kubernetes' database	HashiCorp service-discovery and KV platform	Google-internal lock service
How it works	ZAB; leader orders writes; any server serves (possibly stale) reads	Raft; MVCC revisions; gRPC API	Raft among servers; gossip (Serf) among agents	Multi-Paxos in a 5-replica cell; master lease
Advantages	Mature recipes (Apache Curator); ephemeral and sequential nodes; observers	Revision-based watches; leases; transactions; linearizable reads by default	Health checks, DNS interface, multi-datacenter federation, sessions with lock-delay	Sequencers; lock-delay; consistent client caching
Disadvantages	JVM tuning; one-shot watches (before 3.6); stale reads unless `sync`	Dataset must stay small (2 GB default quota); sensitive to disk latency	KV is secondary to discovery; more moving parts	Not available outside Google
Performance	Reads scale with servers; writes bounded by leader and quorum	Writes bounded by leader, quorum and `fsync`	Similar to etcd for KV; gossip scales membership	Designed for low load: coarse locks, cached reads
Operational complexity	Medium–high (separate JVM ensemble)	Medium (defrag, compaction, backups)	Medium–high (servers plus agent on every node)	Run by a central team
Cost	Open source; 3–5 dedicated nodes	Open source; 3–5 nodes with fast SSDs	Open source plus commercial edition	Internal
Use cases	Hadoop, HBase, older Kafka, Solr	Kubernetes, CoreDNS, Vitess topology, many cloud-native systems	Service mesh, discovery, config, locks	GFS, Bigtable master election; naming
Choose when	You run the Hadoop ecosystem or need its recipes	You build on Kubernetes or need watches with history	You need discovery and health checking with locks	(Model for your designs)

Many systems now **embed** a Raft library (etcd's `raft`, HashiCorp's `raft`) instead of running an external service, as Kafka did with KRaft; and conditional writes in managed stores can serve as leases or fencing points without running a quorum yourself.

26.15 Summary and revision notes

SUMMARY

- **Consensus**: agreement, validity, termination. FLP forbids guaranteed termination under asynchrony, so algorithms keep safety always and liveness under partial synchrony: *safety from quorums, liveness from timeouts*.
- A **replicated state machine** runs one consensus decision per log slot; deterministic replicas applying the same log stay identical.
- **Majority quorums** intersect. $N = 2f + 1$ tolerates f crashes; use 3 or 5 voters, never an even number. Commit latency is the $(q - 1)$-th fastest follower RTT, so placement matters as much as count. Flexible Paxos needs only $|Q_1| + |Q_2| > N$.
- **Leader election** yields a node that *believes* it leads; safety needs quorum-checked actions or fencing.
- **Paxos**: prepare/promise discovers and locks out the past; accept/accepted decides; a proposer must adopt the highest-numbered accepted value it hears. Dueling proposers can livelock; Multi-Paxos uses a stable leader and skips phase 1.
- **Raft**: terms, randomized election timeouts, RequestVote with an up-to-date check, AppendEntries with a consistency check (log matching), commit only current-term entries (Figure 8), no-op on election, snapshots, joint or single-server membership changes, ReadIndex or lease reads.
- **ZooKeeper** (ZAB) offers znodes, ephemeral and sequential nodes and watches for recipes; **etcd** offers revisions, history-preserving watches, leases and transactions and backs Kubernetes; **Chubby** pioneered coarse-grained locks, sessions, KeepAlives, sequencers and lock-delay.
- **Expiring locks** cannot stop a paused holder; the Redlock debate centers on timing assumptions and missing fencing. **Leases** need margins for drift and, above all, pauses.
- **Split brain** is prevented by quorums, witnesses, STONITH and **fencing tokens**, which let the resource reject stale holders.

26.16 Knowledge checkpoint

1. Which consensus properties are safety properties and which is a liveness property?
2. How many failures does a 6-voter cluster tolerate, and why is it a poor choice?
3. Five voters have follower RTTs of 2, 3, 40 and 45 ms. What is the commit latency with 1 ms flushes?
4. With $N = 9$ and $|Q_2| = 3$, what is the minimum phase-1 quorum in Flexible Paxos?
5. In Paxos, a proposer receives promises reporting accepted proposals (2.1, A) and (3.2, B). Which value must it propose?

6. Why can dueling proposers prevent Paxos from deciding, and what fixes it?
7. A Raft node's last entry is (index 9, term 3). A candidate's last entry is (index 12, term 2). Does the node grant its vote?
8. Why does a new Raft leader append a no-op entry?
9. What does the log matching property guarantee, and which check enforces it?
10. Why are single-server membership changes safe without joint consensus?
11. How does ReadIndex make a leader read linearizable without writing to the log?
12. Why do ZooKeeper lock waiters watch their predecessor rather than the lock holder?
13. A lease is 6 s, $\rho = 2 \times 10^{-4}$, $d_{\max} = 1$ s. When must the holder stop?
14. Why is a random lock value not a fencing token?

Answers

1. Agreement and validity are safety; termination is liveness.
2. $q = 4$, $f = 2$, the same as 5 voters, with one more machine to fail and a larger quorum to wait for.
3. $q - 1 = 2$ followers; the second-fastest RTT is 3 ms; $L \approx 1 + 3 = 4$ ms.
4. $|Q_1| \geq 9 - 3 + 1 = 7$.
5. B, the value of the highest-numbered accepted proposal (3.2).
6. Each proposer's higher prepare invalidates the other's pending accept, forever. A single distinguished leader plus randomized back-off.
7. No. The node's last term (3) is higher than the candidate's (2), so its log is more up-to-date regardless of length.
8. Entries from earlier terms can only be committed indirectly; committing a current-term entry commits them and enables ReadIndex.
9. Equal (index, term) implies identical prefixes; the AppendEntries `(prevLogIndex, prevLogTerm)` consistency check.
10. Any majority of a configuration with N voters intersects any majority of one with $N \pm 1$ voters.
11. It fixes the commit index at the read's start, confirms leadership with a heartbeat round to a majority, and waits to apply up to that index.
12. To avoid the herd effect: each release wakes exactly one waiter.
13. $6 \times (1 - 0.0002) - 1 = 5.9988 - 1 = 4.9988$ s after sending the request.
14. Storage cannot order random values, so it cannot tell a newer holder from a stale one.

26.17 Interview questions

Basic

Q1. Why do consensus clusters usually have 3 or 5 nodes? Answer. A majority quorum lets $2f + 1$ nodes tolerate f crashes. Three tolerates one, five tolerates two (one failure during maintenance of another). Even sizes add cost and latency without extra tolerance, and more than five adds write latency and leader load for little availability gain under correlated failures.

Q2. What is a fencing token and when do you need one? Answer. A monotonically increasing number issued with each lock or lease grant; the protected resource rejects requests with a token lower than the highest it has seen. You need it whenever a lock protects correctness and the resource is outside the lock service, because a paused or delayed holder can act after its lock expired.

Intermediate

Q3. Walk through a Raft leader failure and recovery. Answer. Followers stop receiving heartbeats; the first whose randomized timeout fires increments its term and requests votes. Nodes vote once per term, only for candidates whose log is at least as up-to-date. With a majority, it becomes leader, appends a no-op, and repairs followers' logs through AppendEntries consistency checks. Uncommitted entries from the old leader may be overwritten; committed ones survive by leader completeness. Failover takes roughly one election timeout plus a round trip.

Q4. Design leader election for a fleet of cron schedulers. Answer. Use a lease-based election in etcd or ZooKeeper: create a key with a lease (for example, 15 s, renewed every 5 s), watch it if you lose. The leader checks lease validity before each job, carries the key's revision as a fencing token into the job store, and the store rejects stale tokens. Make jobs idempotent to tolerate the rare double run. State the failover time (about one lease duration).

Advanced

Q5. A team proposes Redlock to guard writes to a shared S3 object. Critique it. Answer. Redlock's safety depends on bounded pauses, delays and clock drift, and its random values cannot fence. A GC pause can let two clients write. Instead use conditional writes on the object (compare an ETag or version, where supported) so the store itself arbitrates, or a consensus-backed lock whose revision the writer stores and checks atomically. If a lock is only for efficiency, a single Redis node suffices.

Q6. Explain the Raft "Figure 8" problem and its fix. Answer. A leader of term 4 replicates an entry from term 2 to a majority, then crashes. A node holding a conflicting term-3 entry can still win the next election, because term 3 is more up-to-date than term 2, and overwrite the "majority-replicated" entry. Raft therefore commits only entries from the leader's current term by counting replicas; earlier entries commit indirectly, which is why new leaders write a no-op immediately.

26.18 Practical exercises

1. Modify the simulator of §26.7.7 to partition the leader with one follower (a minority) for 500 ms. Verify that the minority leader cannot commit and that a new leader is elected on the majority side.
2. Remove the `n.term_at(idx) == n.term` check from `advance_commit` and construct a schedule reproducing Figure 26.10. Does `check_safety` detect the violation?
3. Set the election timeout range to 150–155 ms and measure split votes and failover times over 200 seeds; compare with the formula in §26.7.2.
4. Start a three-member etcd cluster locally, take a lock with `etcdctl lock`, pause the holder with `kill -STOP`, and observe the lease expiry and the next holder's revision.
5. Implement the ZooKeeper lock recipe with Apache Curator or `kazoo`, and verify that a release wakes only one waiter.

26.19 Design challenges

1. **Global configuration service.** Store feature flags read by 50,000 servers in three regions, updated a few times per hour. Choose voter count and placement, decide which reads must be linearizable, design watch fan-out (proxies or caches), and estimate commit latency and availability.
2. **Fenced job runner.** Design a system in which exactly one worker processes each partition of a queue, workers can pause for up to 30 s, and the output database supports conditional writes. Specify lease duration, renewal, fencing and recovery after a pause.

26.20 References

1. Burrows, M. "The Chubby Lock Service for Loosely-Coupled Distributed Systems." *USENIX OSDI*, 2006.
2. Chandra, T. D., Griesemer, R., Redstone, J. "Paxos Made Live: An Engineering Perspective." *ACM PODC*, 2007.
3. Chang, E., Roberts, R. "An Improved Algorithm for Decentralized Extrema-Finding in Circular Configurations of Processes." *Communications of the ACM* 22(5), 1979.

4. Chang, F., Dean, J., Ghemawat, S., et al. “Bigtable: A Distributed Storage System for Structured Data.” *USENIX OSDI*, 2006.
5. Dwork, C., Lynch, N., Stockmeyer, L. “Consensus in the Presence of Partial Synchrony.” *Journal of the ACM* 35(2), 1988.
6. etcd Authors. *etcd Documentation* (API guarantees; Tuning; Learner design). https://etcd.io/docs/
7. Fischer, M. J., Lynch, N. A., Paterson, M. S. “Impossibility of Distributed Consensus with One Faulty Process.” *Journal of the ACM* 32(2), 1985.
8. Garcia-Molina, H. “Elections in a Distributed Computing System.” *IEEE Transactions on Computers* C-31(1), 1982.
9. GitHub. “October 21 post-incident analysis.” GitHub Blog, 2018. https://github.blog/news-insights/company-news/oct21-post-incident-analysis/
10. Gray, C. G., Cheriton, D. R. “Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency.” *ACM SOSP*, 1989.
11. Howard, H., Malkhi, D., Spiegelman, A. “Flexible Paxos: Quorum Intersection Revisited.” arXiv:1608.06696, 2016 (also OPODIS 2016).
12. Hunt, P., Konar, M., Junqueira, F. P., Reed, B. “ZooKeeper: Wait-free Coordination for Internet-scale Systems.” *USENIX ATC*, 2010.
13. Junqueira, F. P., Reed, B. C., Serafini, M. “Zab: High-performance Broadcast for Primary-Backup Systems.” *IEEE/IFIP DSN*, 2011.
14. Kleppmann, M. “How to do distributed locking.” Blog post, 8 February 2016. https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html
15. Kleppmann, M. *Designing Data-Intensive Applications*, chapters 8–9. O’Reilly Media, 2017.
16. Kubernetes Authors. *Kubernetes Documentation*: “Leases”; “Operating etcd clusters for Kubernetes”; `kube-controller-manager` reference. https://kubernetes.io/docs/
17. Lamport, L. “The Part-Time Parliament.” *ACM Transactions on Computer Systems* 16(2):133–169, 1998.
18. Lamport, L. “Paxos Made Simple.” *ACM SIGACT News* 32(4):51–58, 2001.
19. Ongaro, D., Ousterhout, J. “In Search of an Understandable Consensus Algorithm.” *USENIX ATC*, 2014. https://raft.github.io/
20. Ongaro, D. *Consensus: Bridging Theory and Practice*. PhD thesis, Stanford University, 2014.
21. Redis. *Distributed Locks with Redis* (Redlock). Redis documentation.
22. Sanfilippo, S. (antirez). “Is Redlock safe?” Blog post, February 2016. http://antirez.com/news/101
23. Schneider, F. B. “Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial.” *ACM Computing Surveys* 22(4), 1990.
24. Apache Software Foundation. *Apache HBase Reference Guide*; *KIP-500: Replace ZooKeeper with a Self-Managed Metadata Quorum*. https://hbase.apache.org/ ; https://cwiki.apache.org/confluence/display/KAFKA/

26.21 Open verification items

- Ongaro’s 2015 raft-dev report of the single-server membership-change bug and its no-op fix (date, wording).
- GitHub October 2018 incident figures (43 s partition, about 24 h degradation) and the current blog URL.
- antirez reply URL (`antirez.com/news/101`) and the Redis documentation URL for Redlock.
- ZooKeeper throughput phrasing in Hunt et al. 2010; persistent recursive watches introduced in 3.6.0.
- etcd defaults as of 2026: 100 ms heartbeat, 1,000 ms election timeout, randomization over $[T, 2T)$, 2 GB quota; tuning guidance (heartbeat ≈ RTT, election ≥ 10 × RTT).
- Kubernetes controller defaults (15 s / 10 s / 2 s) as of 2026.
- Chubby defaults (12 s lease extension, 45 s grace, lock-delay up to 1 min) and Bigtable’s measured Chubby-caused unavailability.
- Kafka 4.0 (2025) removing ZooKeeper mode.

Chapter 27 — Replication and Fault Tolerance

> **Level:** 3 · **Track:** A (Traditional) · **Estimated reading time:** 6 h

Chapter 14 introduced replication as a database scaling tool: one primary, a few read replicas, and a choice between synchronous and asynchronous commit (§14.3–§14.5). That picture stops being enough when you ask harder questions. What happens when the primary dies at the wrong moment? How does anyone *know* it died? Can a store keep accepting writes while half its nodes are unreachable, and what does it give up to do so? When two copies disagree, which one is right?

This chapter surveys the five families of replication strategy, works through primary–backup, chain, multi-leader and leaderless replication, and then treats the machinery that keeps replicas alive and consistent: quorums, hinted handoff, failure detection, failover, recovery, anti-entropy and conflict-free replicated data types (CRDTs). Consensus belongs to Chapter 26, consistency definitions to Chapter 25 and distributed transactions to Chapter 28.

27.1 Learning objectives

After this chapter you should be able to:

- Choose among primary–backup, chain, multi-leader, leaderless and consensus-based replication for a workload, and justify the choice.
- Compute asynchronous failover loss, chain write latency and CRAQ read capacity, and estimate multi-leader conflict rates.
- State what $R + W > N$ does and does not guarantee, and compute quorum latency from the k-th fastest replica.
- Size hinted handoff, compute phi-accrual suspicion, SWIM detection time and gossip rounds, and build failover and recovery time budgets.
- Use read repair and Merkle anti-entropy, choose among last-writer-wins, version vectors and CRDTs, and implement and test four CRDTs.

27.2 Prerequisites

- Primary–replica replication, replication modes and lag — §14.3–§14.5 (this chapter builds on them rather than repeating them).

- Consistent hashing, virtual nodes and preference lists — §14.8 (leaderless stores place replicas on a ring).
- Merkle trees — §17.8 (the data structure behind anti-entropy).
- Availability, MTBF/MTTR and durability arithmetic — §4.14–§4.16 (every redundancy argument here reduces to them).
- Percentiles and tail latency — §4.8 (quorum latency is an order statistic).
- Failure models, partial failures, clocks and version vectors — §24.6–§24.11 (what can fail, and how causality is tracked).

27.3 Replication strategies overview

27.3.1 Five ways to keep copies

DEFINITION

Definition — Replication strategy. The rule that decides *which* replicas may accept a write, *in what order* the replicas apply writes, and *when* a write counts as done. The five families are **primary-backup**, **chain replication**, **multi-leader**, **leaderless** and **consensus-based** replication.

First-principles explanation

A family shares a shopping list on several phones. One person can forward every change (primary–backup); changes can pass along a line of phones, with the last one answering questions (chain); everyone can edit locally and sync later (multi-leader); anyone can write to any two of three phones and read any two (leaderless quorums); or the family can vote on each change (consensus). Each rule trades latency, availability during failures and how surprising the list looks to a reader.

Why does this exist?

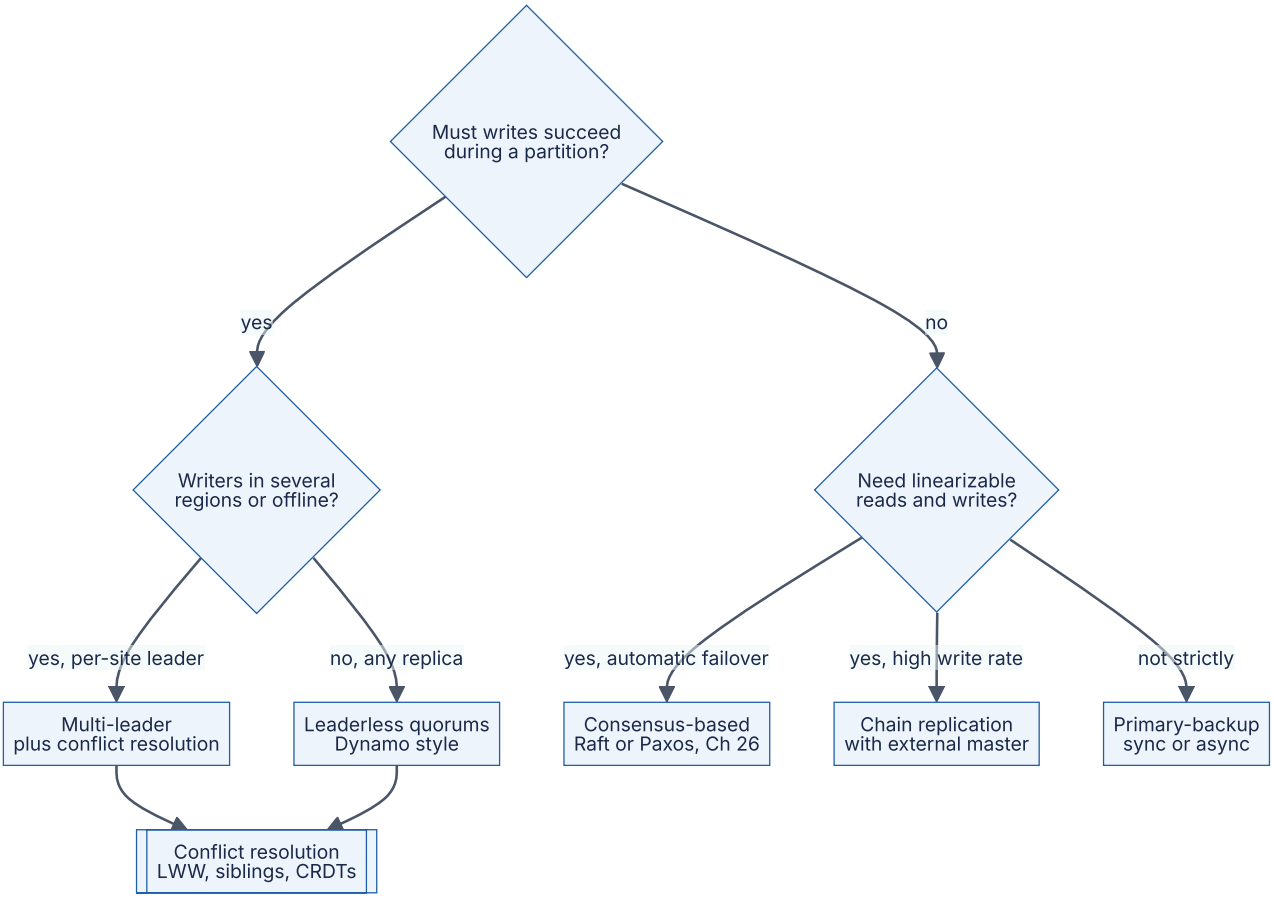
Replication exists for three reasons: **fault tolerance** (survive node, disk and zone failures), **latency** (serve users from a nearby copy) and **throughput** (spread reads, and sometimes writes, over more machines). No single strategy optimizes all three, because a write that must reach distant copies before it is acknowledged is slow, and a write that is acknowledged before reaching them can be lost or can conflict.

Decision table

Table 27.1 — Replication strategies compared.

	Primary–backup	Chain	Multi-leader	Leaderless	Consensus-based
Who accepts writes	One primary	Head of chain	Every leader	Any replica, via a coordinator	Elected leader
Ordering	Primary's order	Chain order	None globally; conflicts	None; version vectors or timestamps	Agreed log
Write done when	Backups ack (sync) or at once (async)	Tail has it	Local leader commits	W of N ack	Majority ack
Strong reads	From primary	From tail	No	No (not even with $R + W > N$)	From leader with lease, or via log
Failover	External detector promotes a backup	Master service splices chain	Not needed for writes	Not needed	Built into protocol
Tolerates	$n - 1$ crashes, if detector is correct	$n - 1$ crashes	Region loss, partitions	Partitions, slow nodes	f of $2f + 1$ crashes
Typical systems	PostgreSQL, MySQL, Kafka ISR	FAWN-KV, CRAQ	Multi-region databases, offline apps	Dynamo, Cassandra, Riak	etcd, ZooKeeper, Spanner (Ch 26)
Choose when	One region, simple ops	High write throughput with strong reads	Multi-region writes or offline clients	Always-writable, latency-sensitive keys	Metadata, locks, strong consistency

Figure 27.1 — Choosing a replication strategy.



Reading the diagram. The diagram is a decision path, not a data flow. The first question splits designs that remain writable on both sides of a network partition (right branch, which must accept conflicts and resolve them, §27.14) from those that keep one ordering of writes (left branch). On the ordered side, consensus-based replication gives automatic, safe failover; chain replication gives high throughput with strong reads but relies on a separately replicated master service; primary–backup is simplest but its safety depends on the failure detector and fencing. Both availability-first branches feed into conflict resolution, the shared failure point: whichever resolver you choose decides which concurrent writes survive.

KEY CONCEPT

Concept — Replication is ordering plus acknowledgment. Every strategy answers two questions: who decides the order of writes, and how many copies must hold a write before the client hears “done”. Primary–backup, chain and consensus fix the order at one place; multi-leader and leaderless let order emerge and repair disagreements afterwards.

27.4 Primary–backup replication

27.4.1 State transfer and operation shipping

DEFINITION

Definition — Primary-backup replication. One replica, the **primary**, receives all requests, executes them and propagates the result to one or more **backups**. If the primary fails, a backup is **promoted** to primary. It is also called **passive replication** [Alsberg and Day 1976; Budhiraja et al. 1993].

First-principles explanation

A head chef cooks while an apprentice keeps an identical copy of each dish. The chef can photograph each finished plate (**state transfer**: send the resulting state) or read out each instruction, “add 5 g salt”, for the apprentice to perform (**operation shipping**, also called log shipping, or **active** replication when every replica executes it). Photographs are bigger but exact; instructions are small but work only if every step gives the same result on both stoves.

Why does this exist?

Primary–backup is the oldest and simplest fault-tolerance pattern. One node decides the order of operations, so there are no write conflicts and reads from the primary are linearizable while the primary is genuinely the only primary. Most relational databases (§14.3), Kafka partitions and many file systems use it.

How it works internally

1. The client sends a request to the primary.
2. The primary executes it and produces either a state delta (changed pages, rows) or an operation record (a log entry).
3. The primary sends the delta or record to the backups, in the order it executed them.
4. In **synchronous** mode the primary waits for the backups’ acknowledgments before replying; in asynchronous mode it replies at once (§14.4 compares the modes).
5. A **failure detector** (§27.10) watches the primary. When it suspects a crash, a backup is chosen, made consistent with the most up-to-date surviving copy, **fenced** off from the old primary (§26.13) and promoted.

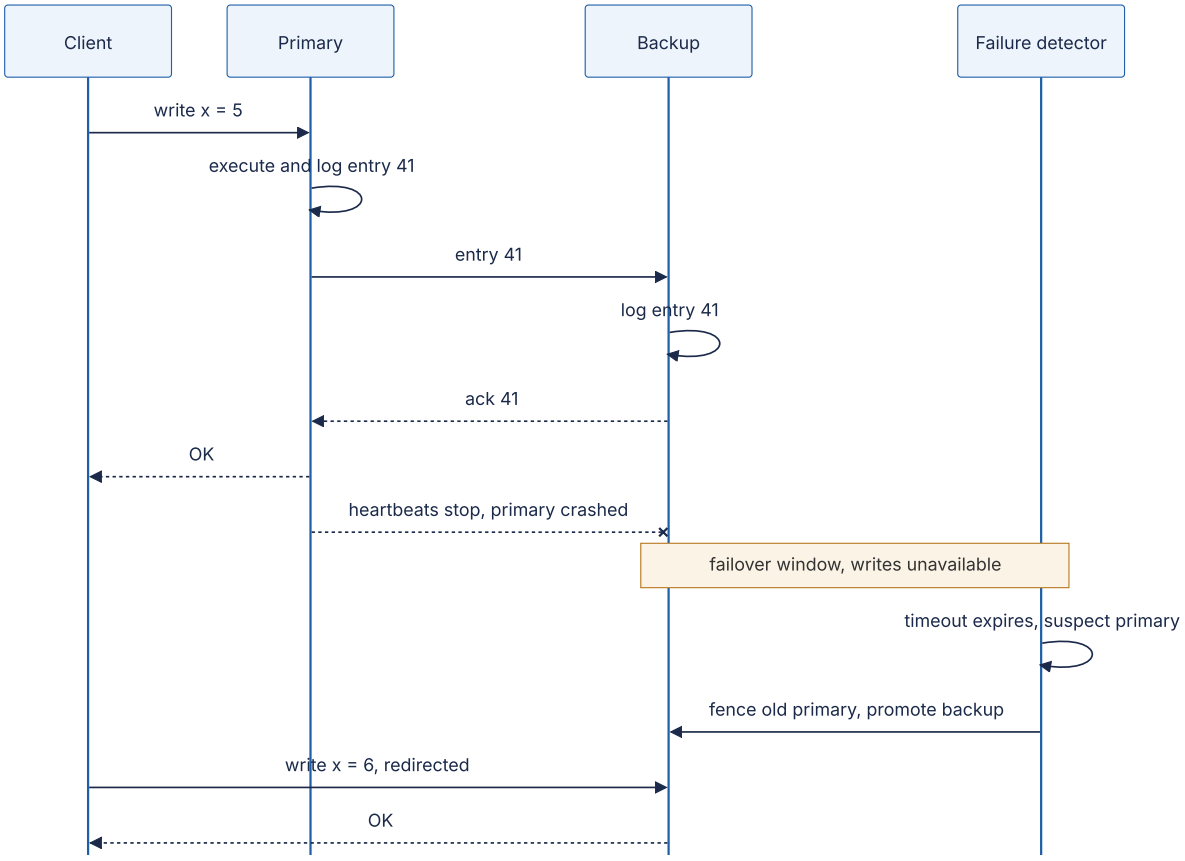
The trade-off between the two forms is concrete:

- **State transfer** is deterministic by construction, works for any computation and is the basis of physical (WAL) replication (§14.4.5). It costs bandwidth proportional to

the amount of state changed, which can be large: one `UPDATE` that touches a million rows ships a million row images.

- **Operation shipping** sends the request, so it is compact, but every replica must execute it **deterministically**. Clocks, random numbers, thread scheduling and unordered iteration break determinism. Systems either forbid non-deterministic operations, log their results (for example, the value of `NOW()`), or replay at a very low level, as VMware Fault Tolerance does.

Figure 27.2 — Primary-backup with a synchronous backup and the failover window.



Reading the diagram. Solid arrows are synchronous messages; the client waits until the backup has logged entry 41, so a primary crash after the reply loses nothing. The crossed arrow marks the crash. From then until the detector's timeout expires and the backup is promoted, no write can succeed: that interval is the **failover window**, and it is dominated by detection time (§27.11). The storage boundaries are the two logs. The failure points are the detector (too slow lengthens the window; too eager causes needless failovers and possible split brain) and the backup itself (if it is down, synchronous commits stall). Scaling adds backups, which adds acknowledgments to wait for and copies for the primary to send.

Mathematical foundations

With asynchronous shipping, everything the primary acknowledged but had not yet shipped is lost when a backup is promoted.

FORMULA

Writes lost in an asynchronous failover

$$L = r_w \times \ell$$

Symbol	Meaning	Units
L	Acknowledged writes lost when the most up-to-date backup is promoted	writes
r_w	Write rate on the primary just before the crash	writes/s
ℓ	Replication lag of the promoted backup at the moment of the crash	s

WORKED EXAMPLE

Worked example — Sizing the loss. An order service commits $r_w = 2,000$ writes/s. Its asynchronous backup normally lags by 50 ms, but under a burst the lag reaches 500 ms.

1. Normal lag: $L = 2,000 \times 0.05 = 100$ acknowledged orders lost.
2. During the burst: $L = 2,000 \times 0.5 = 1,000$ orders lost.
3. With one synchronous backup in the same region: $\ell = 0$ for acknowledged writes, so $L = 0$, at a cost of one intra-region round trip per commit (about 0.5–1 ms, §14.4.1).

Interpretation. Asynchronous loss scales with load *and* lag, and lag grows exactly when the system is busiest. **Engineering implication.** If any loss is unacceptable, make at least one backup synchronous, or make failover choose only a backup known to hold every acknowledged write, and alert on lag in seconds as well as bytes.

There is also a bandwidth cost. With n replicas the primary sends every update $n-1$ times, so its outbound network link limits write throughput: $X_{\max} = B_{\text{out}}/((n-1)\cdot s)$ for updates of size s. That bottleneck is what chain replication removes (§27.5).

Real-world industry applications

INDUSTRY EXAMPLE

VMware vSphere Fault Tolerance. VMware FT replicates an entire virtual machine by operation shipping at the instruction level. The primary's hypervisor logs every non-deterministic event (interrupts, network packets, timer reads) and sends it over a logging channel; the backup replays the same execution. The **output rule** makes the primary delay any externally visible output until the backup has acknowledged the log entry that produced it, so a failover never contradicts what the outside world has already seen. Failure detection uses heartbeats plus an atomic test-and-set on shared storage to prevent both copies from going live. *Evidence: Primary* — [Scales et al. 2010]

INDUSTRY EXAMPLE

Apache Kafka in-sync replicas. Each Kafka partition has a leader and followers that fetch from it. Followers that keep up form the **in-sync replica set (ISR)**. A producer with `acks=all` is acknowledged once every current ISR member has the record, and `min.insync.replicas` sets how small the ISR may shrink before such writes are refused. A replica outside the ISR may become leader only if unclean leader election is enabled, which trades data loss for availability. *Evidence: Primary* — [Apache Kafka Docs: Replication]

Failure scenarios

Table 27.2 — Failure scenarios in primary-backup replication.

What goes wrong	Why	Detection	Recovery	Prevention
Acknowledged writes vanish after failover	Async backup promoted while lagging	Gap in log positions; user reports	Replay old primary's unshipped log if disk survives	Sync backup or lossless semi-sync; promote most-advanced replica
Two primaries accept writes	Old primary was slow, not dead; detector misfired	Two nodes report primary role	Fence old primary; reconcile divergent writes	Leases and fencing tokens (§26.11–§26.13)
Replicas diverge silently	Non-deterministic operation shipped	Checksums differ between replicas	Rebuild backup by state transfer (§27.12)	Row-based or physical logs; log non-deterministic values
All commits stall	Sole synchronous backup is down	Commit latency spikes	Drop to async or add a backup	Quorum acks over ≥ 2 backups

COMMON MISTAKE

Mistake — "A backup is a backup, so failover is safe." A backup is only as current as its last acknowledged entry, and failover is only safe if the old primary is stopped. Without fencing, a primary paused for 20 s by garbage collection can wake up and accept writes alongside the new one.

27.5 Chain replication

27.5.1 Writes at the head, reads at the tail

DEFINITION

Definition — Chain replication. Replicas are arranged in a linear **chain**. Every update enters at the **head** and is forwarded node by node to the **tail**; the tail replies to the client. Every query is answered by the tail. A separate, fault-tolerant **master** service detects failures and reconfigures the chain [van Renesse and Schneider 2004].

First-principles explanation

Think of a bucket brigade. Buckets pass along the line, and only when the last person pours one on the fire does it count, so anyone asking "how much water has reached the fire?" asks the last person. Each person passes each bucket once, so no one is overloaded; the price is that every bucket travels the whole line.

Why does this exist?

In primary–backup the primary sends every update to every backup. Van Renesse and Schneider designed chain replication for storage needing both high throughput and strong consistency: each node forwards each update once, and because the tail holds only updates every other node already has, tail reads are linearizable without extra coordination.

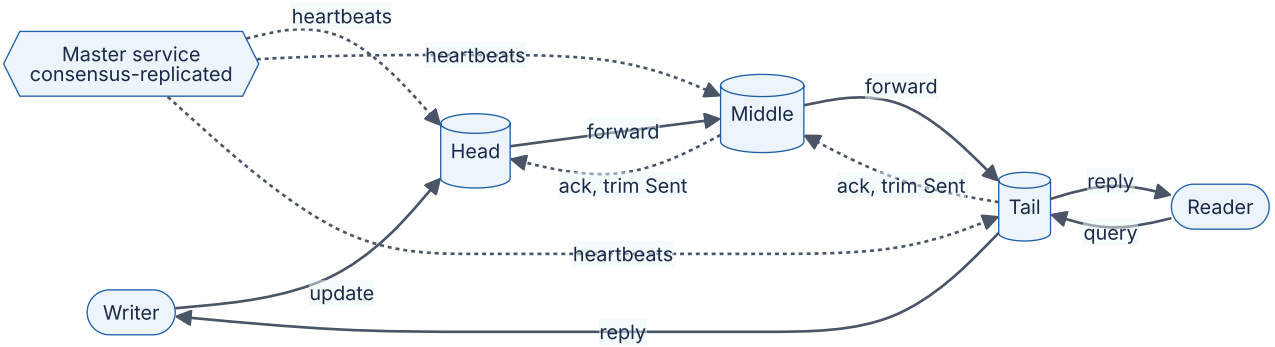
How it works internally

1. The client sends an update to the head. The head applies it and forwards it to its successor.
2. Each middle node applies the update, records it in a **Sent** list of updates forwarded but not yet confirmed, and forwards it.
3. The tail applies the update, replies to the client and sends an acknowledgment back up the chain, so predecessors can trim their Sent lists.
4. Queries go only to the tail, which reflects exactly the set of fully replicated updates.

5. The master (itself replicated with consensus, Chapter 26) detects crashes and repairs the chain:
    - **Head fails:** its successor becomes the head. Updates the old head received but had not forwarded are lost, but none had been acknowledged.
    - **Tail fails:** its predecessor becomes the tail. That node already holds everything the old tail held, and possibly more.
    - **Middle node fails:** the master links its predecessor to its successor, and the predecessor resends everything in its Sent list that the successor may have missed.
6. A new node is added at the tail end: it copies the current tail's state, then takes over as tail.

A chain of n nodes tolerates $n - 1$ crashes, because any surviving node holds every acknowledged update.

Figure 27.3 — Chain replication with an external master.



Reading the diagram. Updates flow synchronously left to right over TCP; the writer's reply comes from the tail, so the write's latency spans every hop. Readers talk only to the tail. Dotted arrows are asynchronous: acknowledgments flowing back up the chain to trim each node's Sent list, and the master's failure detection. Each node stores a full copy. Failure points are any chain node (repaired by the master as listed above) and the master itself, which is why the master must be consensus-replicated. Scaling: a longer chain adds fault tolerance and latency but no read capacity, since only the tail serves reads; systems scale out by running many chains, each owning a range of keys.

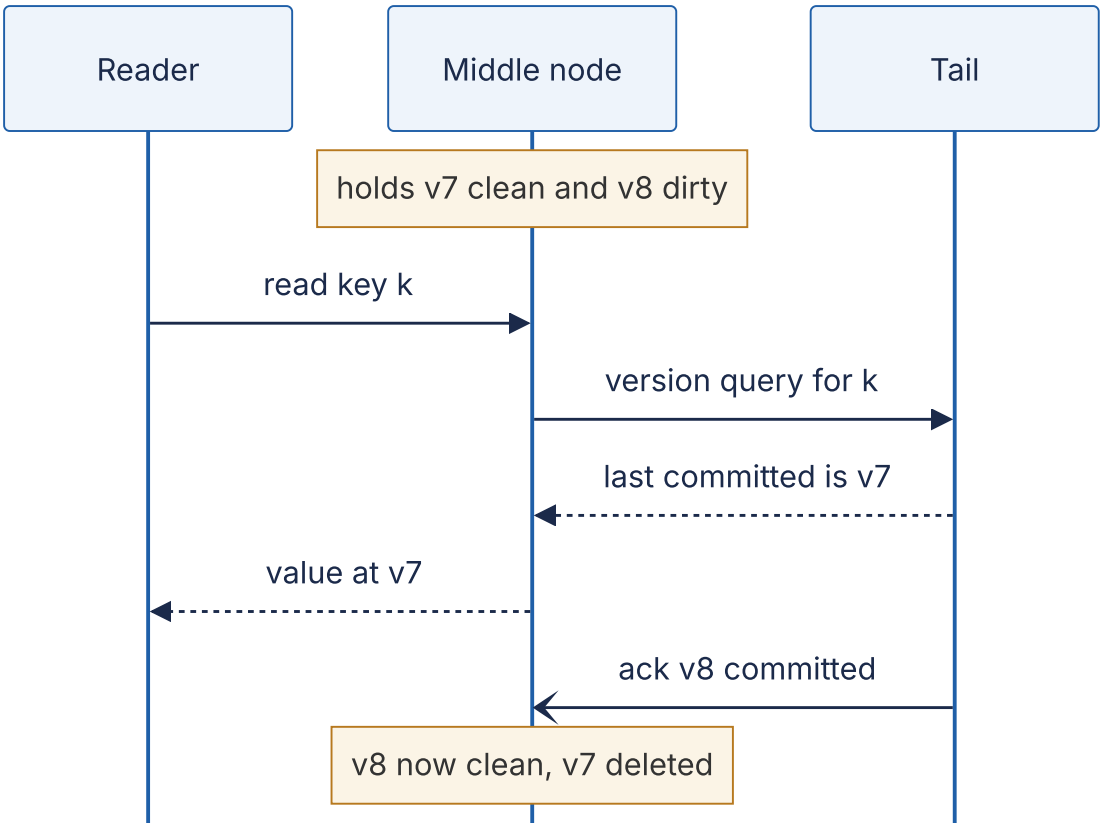
27.5.2 CRAQ: reading from every node

Chain replication has an obvious waste: $n - 1$ nodes hold the data but only the tail can answer queries. **CRAQ** (Chain Replication with Apportioned Queries) lets any node answer reads while keeping strong consistency [Terrace and Freedman 2009].

1. Each node may store several versions of an object. A version is **clean** once the node knows the tail has it, and **dirty** before that.

2. On an update, each node appends a new dirty version and forwards it. When the tail applies it, the version is committed; acknowledgments flow back, each node marks that version clean and deletes older ones.
3. On a read at any node: if the latest local version is clean, return it. If it is dirty, send a small **version query** to the tail, which returns its latest committed version number, and return that version (which the node has, because the chain delivers versions in order).

Figure 27.4 — A CRAQ read that hits a dirty object.



Reading the diagram. The reader contacts a middle node synchronously. Because the newest local version (v8) is dirty, that node asks the tail which version is committed, a message carrying only a version number, and returns v7, the value every other reader would also see. The later asynchronous acknowledgment marks v8 clean, after which the node answers locally. Storage holds several versions per key during an update. The failure point is the tail: dirty reads depend on it, so a write-heavy hot key concentrates version queries there. Read capacity scales with chain length when most objects are clean.

Mathematical foundations

FORMULA

Chain write latency and read capacity

$$T_{\text{write}} \approx (n+1)\,d + n\,s \qquad X_{\text{read}}^{\text{CR}} = c \qquad X_{\text{read}}^{\text{CRAQ}} \approx n\,c \text{ (clean reads)}$$

Symbol	Meaning	Units
n	Number of nodes in the chain	nodes
d	One-way network delay per hop	ms
s	Processing time per node for an update	ms
c	Read capacity of one node	reads/s
T_{write}	Client-observed write latency (client → head → … → tail → client)	ms
X_{read}	Read capacity of one chain	reads/s

WORKED EXAMPLE

Worked example — Chain versus primary-backup in one datacenter. Take $d = 0.25$ ms, $s = 0.05$ ms, $c = 50,000$ reads/s, 4 KiB updates and 10 Gb/s network interfaces (1.25 GB/s each way).

1. Chain of 3: $T_{\text{write}} = 4 \times 0.25 + 3 \times 0.05 = 1.15$ ms. Chain of 5: $6 \times 0.25 + 5 \times 0.05 = 1.75$ ms.
2. Primary–backup with parallel fan-out: client → primary → backups → primary → client is 4 one-way hops regardless of n, so $4 \times 0.25 + 2 \times 0.05 = 1.1$ ms.
3. Write throughput limit from the busiest outbound link, $n = 5$: the primary sends 4×4 KiB $= 16$ KiB per write, so $1.25 \times 10^9 / 16,384 \approx 76,000$ writes/s. In a chain each node sends each update once: $1.25 \times 10^9 / 4,096 \approx 305,000$ writes/s.
4. Reads with $n = 3$: chain replication serves 50,000 reads/s at the tail; CRAQ serves up to about $3 \times 50,000 = 150,000$ reads/s if most reads find clean objects.

Interpretation. Chains trade latency that grows linearly with length for throughput that does not depend on length. **Engineering implication.** Keep chains short (3 is typical) and inside one datacenter; use CRAQ or many independent chains when reads dominate.

Real-world industry applications

INDUSTRY EXAMPLE

FAWN-KV. FAWN-KV, a key-value store built on clusters of low-power nodes, places nodes on a consistent-hash ring and replicates each key range with chain replication along the ring, with a front-end that routes puts to the head and gets to the tail. *Evidence: Primary* — [Andersen et al. 2009]

INDUSTRY EXAMPLE

CRAQ across datacenters. The CRAQ paper describes chains that span datacenters, with chain membership kept in ZooKeeper, and offers eventual-consistency and bounded-staleness read modes in addition to strong reads, so applications can trade freshness for local reads. *Evidence: Primary* — [Terrace and Freedman 2009]

Advantages and disadvantages

Chain replication gives linearizable reads and writes, balanced network load and simple recovery. Its costs are write latency proportional to chain length, every write waiting for the slowest node, and dependence on an external master for reconfiguration.

27.6 Multi-leader replication

27.6.1 Use cases and topologies

Chapter 14 introduced multi-primary replication for databases (§14.5). Here we generalize: **multi-leader replication** lets each of several leaders accept writes and asynchronously exchange them, and it reappears wherever writers cannot wait for a single authority.

DEFINITION

Definition — Replication topology. The graph along which leaders forward writes to each other: a **circular** ring, a **star** with a hub, or **all-to-all**.

Why does this exist?

Three use cases recur:

1. **Multi-datacenter operation.** Each region has a leader, so writes take a local round trip (about 1 ms) instead of a cross-region one (70–150 ms), and a region keeps writing when cut off.

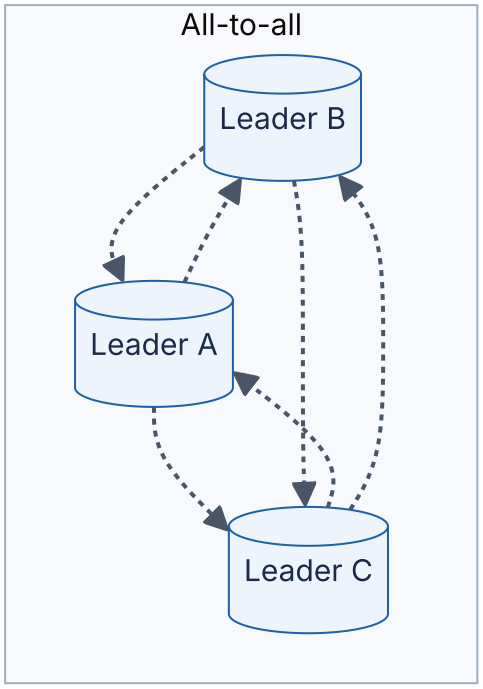
2. **Offline clients.** A calendar or notes app on a phone is a leader with a local database; it syncs when the network returns, possibly days later.
3. **Collaborative editing.** Every participant's editor applies its own keystrokes immediately and exchanges changes, so typing never waits for a server round trip.

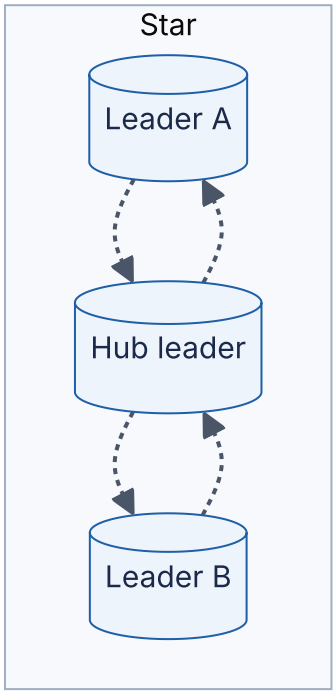
All three accept the same consequence: two leaders can change the same item concurrently, and nobody finds out until the changes meet.

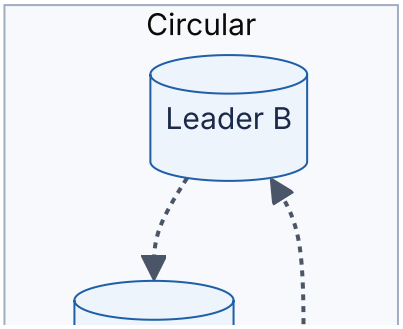
How it works internally

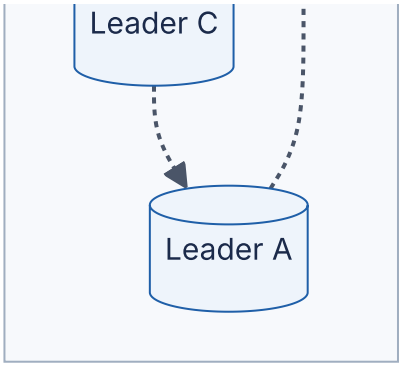
1. Each leader commits a write locally and tags it with its origin (a leader ID) and a version (a log position, timestamp or version vector).
2. The write is forwarded along the topology. A node that receives a write it has already seen, recognized by origin and version, drops it, which prevents infinite loops in rings.
3. A receiving leader **detects a conflict** if the incoming write and a local write touched the same item and neither causally precedes the other. Version vectors detect this exactly (§27.14); timestamps cannot.
4. The conflict is resolved by a policy (§27.14) or recorded for the application.

Figure 27.5 — Multi-leader topologies.


All-to-all
Leader B
Leader A
Leader C


Star
Leader A
Hub leader
Leader B


Circular
Leader B



Reading the diagram. Every arrow is asynchronous replication of committed writes, typically a log stream over TCP. In the circular topology each write travels through every other leader in turn, so one failed or slow leader stops propagation for everyone downstream until the ring is reconfigured. In the star, the hub is a single point of failure for replication (though not for local writes). All-to-all has no single point of failure but sends every write along $m(m-1)$ links for m leaders and allows writes to overtake each other on different paths. Each leader stores a full copy. Scaling beyond a handful of leaders multiplies both links and potential conflicts.

Mathematical foundations

How often do conflicts happen? Two writes to the same item conflict if they are made at different leaders within one replication delay of each other.

FORMULA

Conflict rate for one item written at two leaders

$$\kappa \approx 2\,\lambda_A\,\lambda_B\,\Delta \qquad (\lambda_A\Delta \ll 1,\ \lambda_B\Delta \ll 1)$$

Symbol	Meaning	Units
κ	Expected conflicts per second for the item	conflicts/s
λ_A, λ_B	Write rate to the item at leaders A and B	writes/s
Δ	Replication delay between the leaders	s

Derivation. Each write at A conflicts if B wrote the same item within Δ before or after it, a window of length 2Δ, which happens with probability about $2\Delta\lambda_B$ when that is small. Multiply by A's write rate.

WORKED EXAMPLE

Worked example — Rare per key, common in aggregate. Two regions replicate with $\Delta = 0.2$ s.

1. A busy shared document written 0.1 times per second in each region: $\kappa = 2 \times 0.1 \times 0.1 \times 0.2 = 0.004$ conflicts/s, about 14 per hour.
2. One million ordinary keys, each written 0.001 times per second in each region: per key $2 \times 10^{-6} \times 0.2 = 4 \times 10^{-7}$ conflicts/s; over all keys 0.4 conflicts/s, about 34,600 per day.
3. Partitioned ownership, where each key is written in only one region: $\lambda_B = 0$, so $\kappa = 0$.

Interpretation. Conflicts grow with the product of the two write rates and with the delay; a partition that makes Δ last for minutes multiplies them. **Engineering implication.** Conflict resolution cannot be an afterthought you hope never runs. Either give each item a home leader, or make the resolution rule correct and tested (§27.14).

Real-world industry applications

INDUSTRY EXAMPLE

Amazon DynamoDB global tables. Global tables replicate a table across Regions with every replica writable. For the multi-Region eventually consistent mode, AWS documents that concurrent updates to the same item are reconciled by last-writer-wins, so applications needing different semantics must avoid concurrent cross-Region writes to an item (as of 2026; AWS has since added a multi-Region strongly consistent option). *Evidence: Primary* — [AWS Docs: DynamoDB Global Tables]

INDUSTRY EXAMPLE

CouchDB. CouchDB replicates between any pair of databases, including offline devices. Each document has a revision tree; when replicas diverge, every node deterministically picks the same winning revision so readers agree, while the losing revisions are kept as conflicts that the application can read and merge. *Evidence: Primary* — [Apache CouchDB Docs: Replication and Conflict Model]

Failure scenarios

Table 27.3 — Failure scenarios in multi-leader replication.

What goes wrong	Why	Detection	Recovery	Prevention
Replication stops for a subset of leaders	Ring or hub node failed	Replication lag grows on downstream leaders	Reconfigure ring or hub	All-to-all, or redundant hubs
Update applied before its insert	Messages overtook on different links	Apply errors, missing parents	Buffer and retry	Version vectors; causal delivery
Uniqueness violated	Two leaders accept same username	Duplicate detected after sync	Rename or merge manually	Route uniqueness checks to one home leader

27.7 Leaderless replication

27.7.1 Dynamo-style coordination

DEFINITION

Definition — Leaderless replication. Any replica of a key may accept reads and writes; the client, or a **coordinator** node acting for it, sends each request to several replicas in parallel and treats it as successful once enough of them respond. There is no failover because there is no leader [DeCandia et al. 2007].

First-principles explanation

A club keeps its membership list on three noticeboards. The secretary pins each notice on at least two boards; a reader checks any two and believes the newer notice. Any two boards share one with the secretary's two, so the reader always sees the latest notice, and losing one board stops nothing.

Why does this exist?

Amazon built Dynamo for the shopping cart, where refusing an "add to cart" during a failure costs money. A leader is a single point that must be detected dead and replaced; during that window writes stop. Leaderless replication keeps writing as long as *enough* replicas answer, and it masks slow replicas by not waiting for the slowest ones [DeCandia et al. 2007].

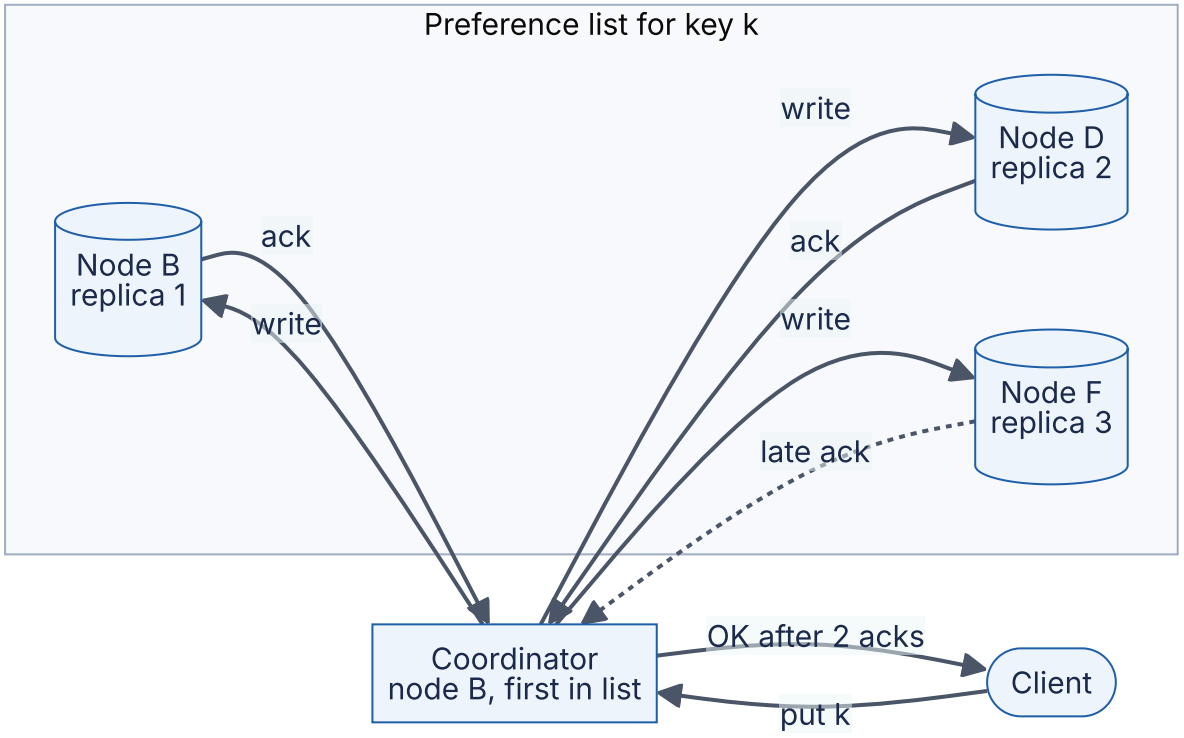
How it works internally

1. **Placement.** Keys are hashed onto a consistent-hash ring (§14.8). The **preference list** for a key is the first N distinct physical nodes clockwise from its position; the list kept is longer than N so that substitutes are known when nodes fail.
2. **Coordinator.** A request arrives at a partition-aware client library or at any node, which forwards it to a coordinator, typically the first healthy node in the key's pref-

erence list.

3. **Write.** The coordinator attaches a new version (a vector clock in Dynamo, a timestamp in Cassandra), writes locally and sends the write to the other replicas in parallel. It reports success once *W* replicas, counting itself, acknowledge.
4. **Read.** The coordinator asks the replicas and waits for *R* responses. If the versions differ, it returns the newest, or all causally concurrent versions for the client to reconcile (§27.14), and it may repair stale replicas (§27.13).
5. **Background repair.** Hinted handoff (§27.9) and Merkle-tree anti-entropy (§27.13) bring replicas that missed writes back into agreement.

Figure 27.6 — A Dynamo-style write with N = 3 and W = 2.



Reading the diagram. The client sends the write to a coordinator, here node B, which owns the first position in the key's preference list. The coordinator sends the write to all three replicas in parallel over the internal RPC protocol and replies once two acknowledgments arrive; the third (dotted) arrives later and is not waited for. Each replica stores the key in its local storage engine. Failure points: a replica that never receives the write (repaired later), and the coordinator crashing after some but not all replicas applied the write, leaving partial state (§27.8). Scaling adds nodes to the ring; each key still has exactly *N* replicas.

INDUSTRY EXAMPLE

Apache Cassandra. Cassandra adopted Dynamo's ring, replication and repair mechanisms with a Bigtable-style data model [Lakshman and Malik 2010]. Any node can coordinate; a replication strategy chooses replicas per datacenter; each request carries a **consistency level** such as `ONE`, `QUORUM`, `LOCAL_QUORUM` or `ALL` that sets how many replica responses the coordinator waits for. Concurrent writes to a cell are resolved by timestamp (§27.14). *Evidence: Primary* — [Lakshman and Malik 2010; Apache Cassandra Docs]

27.8 Read and write quorums: R + W > N

27.8.1 The overlap argument

DEFINITION

Definition — Quorum. With N replicas per key, a write is acknowledged after W replicas confirm and a read returns after R replicas respond. The configuration is a **strict quorum** when $R + W > N$, so every read set intersects every write set.

FORMULA

Quorum overlap and fault tolerance

$$|S_R \cap S_W| \geq R + W - N \qquad f_{\text{read}} = N - R \qquad f_{\text{write}} = N - W$$

Symbol	Meaning	Units
N	Replicas per key (replication factor)	replicas
R	Responses a read waits for	replicas
W	Acknowledgments a write waits for	replicas
S_R, S_W	The sets of replicas that answered a read and acknowledged a write	sets
$f_{\text{read}}, f_{\text{write}}$	Unavailable replicas a read or write can tolerate	replicas

Derivation. Both sets are drawn from N replicas, so $|S_R \cup S_W| \leq N$. Since $|S_R \cap S_W| = |S_R| + |S_W| - |S_R \cup S_W|$, the overlap is at least $R + W - N$, which is at least 1 exactly when $R + W > N$.

WORKED EXAMPLE

Worked example — The N = 3 configurations.

1. $R = 2, W = 2$: overlap ≥ 1. Reads and writes each survive one unavailable replica. This is Dynamo's common configuration and Cassandra's `QUORUM` with RF = 3.
2. $R = 1, W = 3$: overlap ≥ 1. Reads are fastest and survive two failures, but a single unavailable replica blocks every write.
3. $R = 3, W = 1$: overlap ≥ 1. Writes survive two failures; a single unavailable replica blocks strong reads.
4. $R = 1, W = 1$: overlap ≥ -1, meaning none is guaranteed. A read can miss the latest write entirely. This is `ONE` / `ONE`: fast and available, eventually consistent.
5. $R = 2, W = 1$ or $R = 1, W = 2$: $R + W = 3 = N$, no guaranteed overlap.

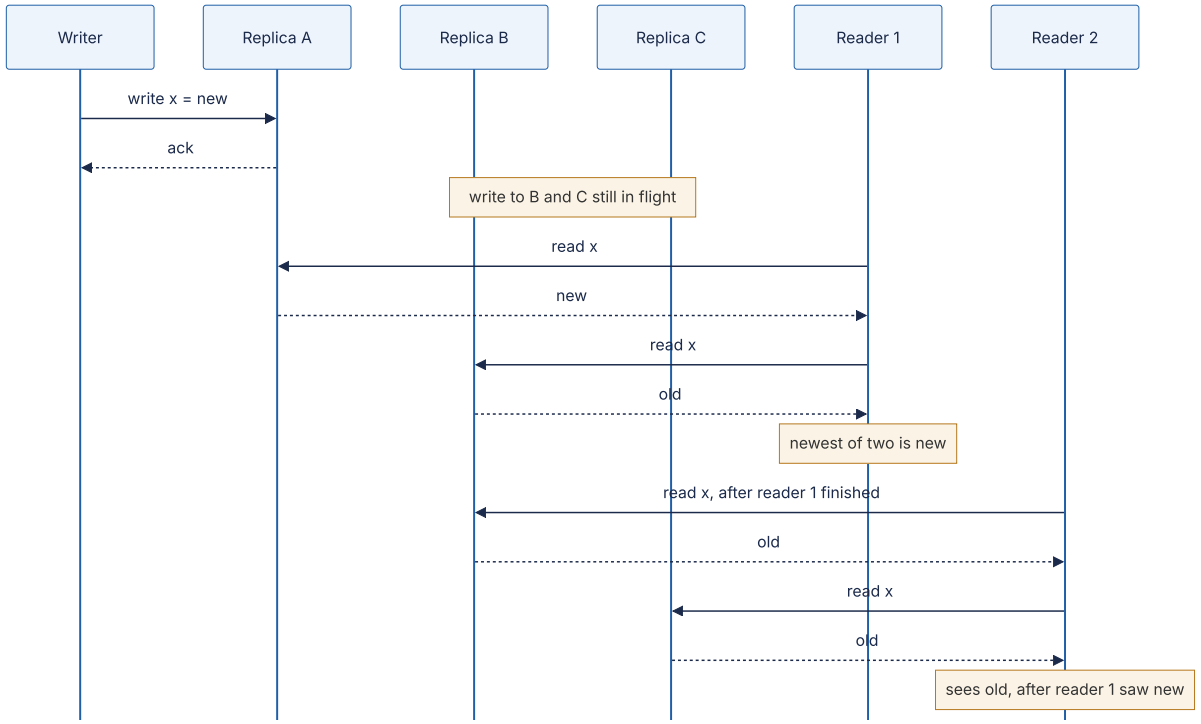
Interpretation. $R + W > N$ trades read cost against write cost along a line; it does not change total fault tolerance much. **Engineering implication.** Choose W by how many failures writes must survive and how slow a write may be, then set $R = N - W + 1$, the cheapest read that still overlaps.

27.8.2 What R + W > N does not guarantee

The overlap argument says that *some* replica in every read set holds the latest *completed* write. It is tempting to conclude that quorum reads are linearizable (§25.3). They are not, for five reasons [Kleppmann 2017, ch. 5]:

1. **Concurrent writes.** Two clients write the same key at the same time and their writes reach replicas in different orders. Overlap guarantees you see both, not that there is an agreed winner. With timestamps (last-writer-wins), one write is silently dropped (§27.14).
2. **A write concurrent with a read.** If a write has reached only some replicas when a read arrives, the read may or may not see it.
3. **Failed writes leave partial state.** If a write reaches one replica but fewer than W, the client is told it failed, but the one replica is not rolled back. Later reads may return the "failed" value, and may later stop returning it.
4. **Sloppy quorums.** If writes may go to substitute nodes outside the key's N home replicas (§27.9), the read and write sets need not intersect at all.
5. **Lost replicas.** If a replica holding the new value fails and is rebuilt from a replica with the old value, the number of copies of the new value can fall below W.

Figure 27.7 — R = W = 2, N = 3, and a read that goes back in time.



Reading the diagram. All calls are synchronous RPCs. The writer's update has reached only replica A. Reader 1 happens to include A in its quorum and returns `new`. Reader 2 starts strictly after Reader 1 finished but picks B and C, both still `old`. Each read overlapped every *completed* write (none has completed yet), so $R + W > N$ is not violated, yet the history is not linearizable: a later read observed an earlier state. The fix is for Reader 1 to write `new` back to a quorum before returning (**blocking read repair**, §27.13), which ABD-style register protocols do and which Cassandra performs for `QUORUM` reads that find a mismatch. Even then, operations that need compare-and-set require consensus (Chapter 26).

27.8.3 Latency: waiting for the k-th fastest replica

A quorum operation sends to all N replicas and waits for the first k replies, where $k = W$ for writes and $k = R$ for reads. Its latency is the **k-th order statistic** of the replica latencies: waiting for fewer replies hides the slow ones.

FORMULA

Latency of the k-th fastest of N replicas (independent, identically distributed)

$$P(T_{(k)} > t) = \sum_{j=0}^{k-1} \binom{N}{j} F(t)^j \, (1 - F(t))^{N-j} \qquad \mathbb{E}[T_{(k)}] = \mu \sum_{i=N-k+1}^{N} \frac{1}{i} \ \text{(exponential)}$$

Symbol	Meaning	Units
$T_{(k)}$	Time until the k-th reply arrives	ms
$F(t)$	Probability that one replica replies within t	dimensionless
N, k	Replicas contacted; replies required	count
j	Number of replicas that replied within t (summation index)	count
μ	Mean latency of one replica, if exponentially distributed	ms

Derivation. The k-th reply is later than t exactly when fewer than k replicas have replied by t; the number that have is binomial with success probability $F(t)$. For exponential latencies, the gap before the first of N replies has mean μ/N, the next gap $\mu/(N-1)$, and so on, by memorylessness.

WORKED EXAMPLE

Worked example — How much does W cost? $N = 3$.

1. *Tail model.* Each replica is independently "slow" (a garbage-collection pause, say 50 ms) with probability $p = 0.01$, so $F(t) = 0.99$ for t below 50 ms. The operation is slow when fewer than k replicas are fast:
   - $k = 1$: $p^3 = 10^{-6}$.
   - $k = 2$: $p^3 + 3p^2(1-p) = 10^{-6} + 2.97 \times 10^{-4} = 2.98 \times 10^{-4}$.
   - $k = 3$: $1 - 0.99^3 = 0.0297$.
2. *Mean model.* Exponential replica latency with $\mu = 2$ ms: $k = 1$: $2 \times \frac{1}{3} = 0.67$ ms; $k = 2$: $2 \times (\frac{1}{3} + \frac{1}{2}) = 1.67$ ms; $k = 3$: $2 \times (\frac{1}{3} + \frac{1}{2} + 1) = 3.67$ ms.

Interpretation. Waiting for all three replicas exposes about 3% of operations, so the p99, to the 50 ms pause; waiting for two exposes 0.03%, beyond even the p99.9. **Engineering implication.** Quorums of two out of three keep the slowest replica off the tail. This is why Dynamo-style systems contact all N replicas even when they need only R or W answers, and why "read from one replica, retry on timeout" is slower in the tail than a parallel quorum.

The program below verifies the overlap table and both latency models.

```
        """Quorum overlap and the latency of waiting for the k-th fastest of N repli-
cas."""
        from itertools import combinations
        from math import comb
        import random

        def min_overlap(n, r, w):
            """Smallest possible intersection of any read set and any write set."""
            replicas = range(n)
            return min(len(set(rs) & set(ws))
                       for rs in combinations(replicas, r)
                       for ws in combinations(replicas, w))

        def p_kth_slow(n, k, p_slow):
            """P(the k-th fastest reply is slow) = P(fewer than k replicas are fast)."""
            q = 1 - p_slow
            return sum(comb(n, j) * q**j * p_slow**(n - j) for j in range(k))

        def mean_kth_exponential(n, k, mean):
            """E[k-th smallest of n i.i.d. exponential latencies with the given
mean]."""
            return mean * sum(1 / i for i in range(n - k + 1, n + 1))

        if __name__ == "__main__":
            n = 3
            print("R W  R+W>N  min overlap  read faults ok  write faults ok")
            for r in range(1, n + 1):
                for w in range(1, n + 1):
                    print(f"{r} {w}  {str(r + w > n):5}  {min_overlap(n, r, w):^11}  "
                          f"{n - r:^14}  {n - w:^15}")

            print("\nP(k-th reply is slow), each replica slow with p = 0.01")
            for k in (1, 2, 3):
                print(f"  k = {k}: {p_kth_slow(n, k, 0.01):.3e}")

            random.seed(1)
            trials = 200_000
            print("\nk-th fastest of 3 exponential replies, mean 2 ms")
            for k in (1, 2, 3):
                sim = sum(sorted(random.expovariate(1 / 2.0) for _ in range(n))[k - 1]
                          for _ in range(trials)) / trials
                print(f"  k = {k}: formula {mean_kth_exponential(n, k, 2.0):.3f} ms, "
                      f"simulated {sim:.3f} ms")
```

Output (abridged to the $R = W$ rows and the latency results):

```
R W  R+W>N  min overlap  read faults ok  write faults ok
1 1  False       0             2               2
2 2  True        1             1               1
3 3  True        3             0               0

P(k-th reply is slow), each replica slow with p = 0.01
  k = 1: 1.000e-06
  k = 2: 2.980e-04
  k = 3: 2.970e-02

k-th fastest of 3 exponential replies, mean 2 ms
  k = 1: formula 0.667 ms, simulated 0.667 ms
  k = 2: formula 1.667 ms, simulated 1.666 ms
  k = 3: formula 3.667 ms, simulated 3.663 ms
```

What to notice:

- `min_overlap` checks every pair of read and write sets by brute force, confirming that the overlap is positive exactly when $R + W > N$.
- The simulated means match the harmonic-sum formula to three digits.

COMMON MISTAKE

Mistake — "QUORUM reads and writes make Cassandra linearizable." They give overlap, not linearizability: concurrent writes are resolved by timestamp, failed writes are not rolled back, and only blocking read repair prevents the read anomaly of Figure 27.7. Compare-and-set needs Cassandra's Paxos-based lightweight transactions.

27.9 Sloppy quorums and hinted handoff

27.9.1 Writing to a stand-in

DEFINITION

Definition — Sloppy quorum and hinted handoff. In a **sloppy quorum**, a write that cannot reach W of a key's N home replicas is accepted by the next healthy nodes on the ring instead, and still counts toward W. Each stand-in stores the write with a **hint** naming the intended home replica and **hands it off** when that replica recovers [DeCandia et al. 2007].

First-principles explanation

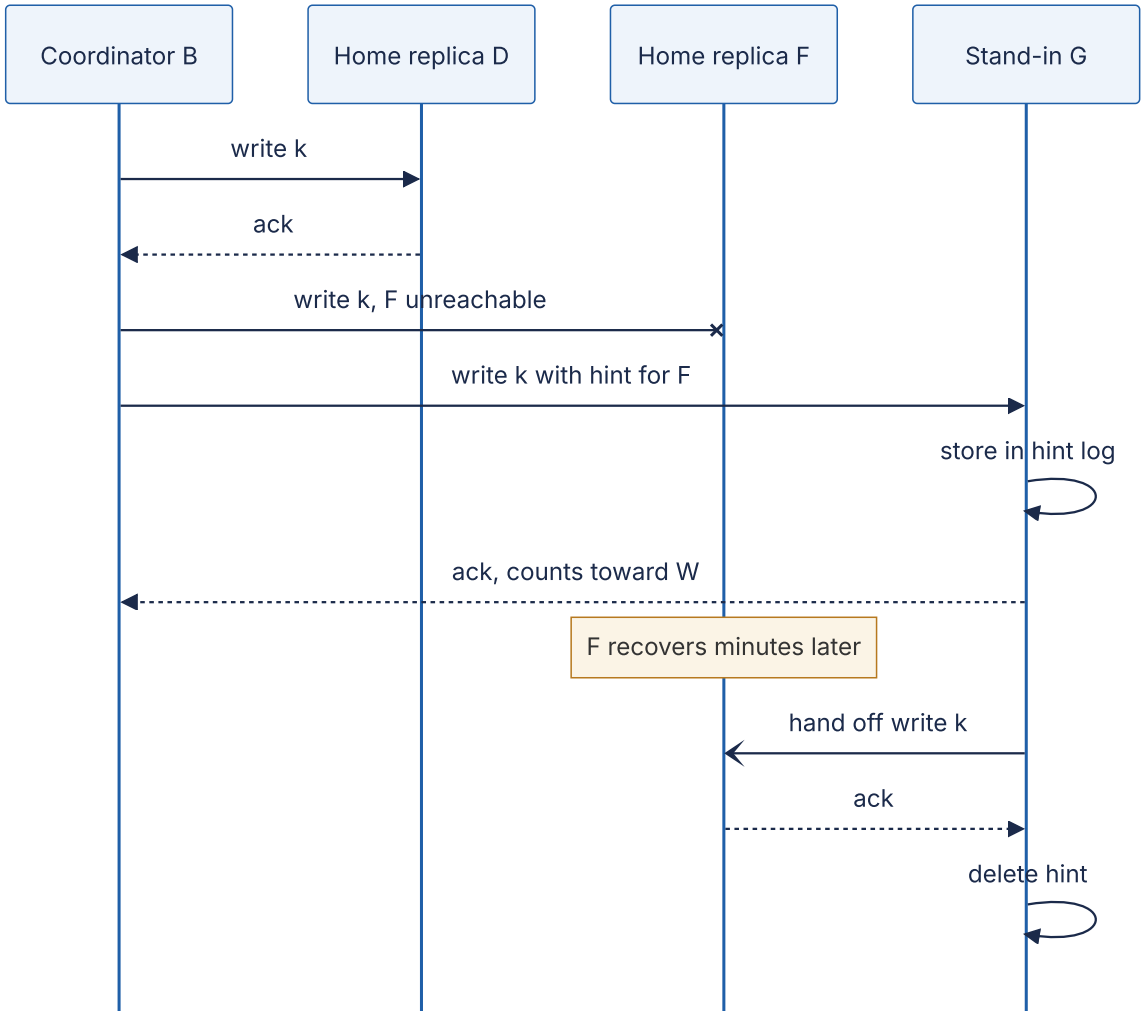
A courier leaves a parcel next door with a note, "for number 12", and the neighbor hands it over when number 12 returns. Nothing is lost, but anyone asking at number 12 in the

meantime will not find it.

How it works internally

1. The coordinator walks the preference list beyond the first *N* nodes when home replicas are unreachable, choosing the first *N healthy* nodes.
2. A stand-in stores the write in a separate local hint store, tagged with the intended owner.
3. The stand-in periodically checks whether the owner is reachable again and, when it is, replays the hint and deletes it.
4. If the owner stays down longer than a configured **hint window**, the stand-in stops collecting hints, and Merkle-tree repair (§27.13) must restore the replica.

Figure 27.8 — A sloppy quorum write and later handoff.



Reading the diagram. The coordinator's writes are synchronous RPCs. Home replica F is unreachable (crossed arrow), so stand-in G, the next node on the ring, accepts the write into a separate hint log, and its acknowledgment counts toward *W*. When F returns, G replays the write asynchronously (open arrowhead) and deletes the hint. Storage

boundaries are the hint log on G and the main store on F. Failure points: G failing before handoff (the hint is lost, so anti-entropy must repair F), and reads in the meantime that go to the home replicas and miss the write, because G is not in their read set.

Mathematical foundations

FORMULA

Hint volume during an outage

$$V_h = \frac{r_w \cdot N}{n} \cdot T_{\text{down}} \cdot s$$

Symbol	Meaning	Units
V_h	Total hints accumulated for the failed node across the cluster	bytes
r_w	Cluster-wide client write rate	writes/s
N	Replication factor	replicas
n	Nodes in the cluster	nodes
T_{down}	Time the node is down (capped by the hint window)	s
s	Size of one write including overhead	bytes

WORKED EXAMPLE

Worked example — A one-hour outage. A 12-node cluster takes $r_w = 60,000$ writes/s with $N = 3$ and 1 KB writes.

1. Replica writes per node: $60,000 \times 3/12 = 15,000$ writes/s.
2. Hints for a one-hour outage: $15,000 \times 3,600 = 5.4 \times 10^7$ writes, $5.4 \times 10^7 \times 1$ KB $= 54$ GB.
3. Spread over the 11 survivors: about 4.9 GB each.
4. Replay into the recovered node at 100 MB/s total: $54 \times 10^9 / 10^8 = 540$ s, about 9 minutes, during which that node still serves stale reads.

Interpretation. Hints are cheap for short outages and grow linearly with downtime. **Engineering implication.** Cap the hint window (Cassandra's default is 3 hours, as of Cassandra 4) so stand-ins do not fill their disks, alert when a node exceeds it, and run a full repair before the node rejoins the read path.

Real-world industry applications

INDUSTRY EXAMPLE

Dynamo versus Cassandra. Dynamo used sloppy quorums: hinted writes count toward W, maximizing write availability at the cost of quorum overlap [DeCandia et al. 2007]. Cassandra stores hints too, but by default they do not count toward the consistency level; only `ANY` accepts a write stored solely as a hint. Cassandra's quorums are therefore strict, and hints are a best-effort repair mechanism [Apache Cassandra Docs: Hints]. *Evidence: Primary* — [DeCandia et al. 2007; Apache Cassandra Docs: Hints]

WARNING / TRADE-OFF

Warning — Sloppy quorums break the overlap guarantee. A write acknowledged by stand-ins G and H and a later read from home replicas D and F can share no node, even though $R + W > N$. Treat a sloppy quorum as a durability promise ("W nodes hold this write"), not a consistency promise.

27.10 Failure detection

27.10.1 Heartbeats and timeouts

DEFINITION

Definition — Failure detector. A component that outputs, for each monitored process, whether it currently **suspects** that process has crashed. It is **complete** if every crashed process is eventually suspected and **accurate** if live processes are not suspected. In an asynchronous network no detector can be both perfectly complete and perfectly accurate, because a slow process is indistinguishable from a dead one [Chandra and Toueg 1996].

First-principles explanation

You phone a friend every evening. One evening there is no call. Did the friend's phone die, are they busy, or is the network down? If you raise the alarm after five minutes, you will often worry about nothing; if you wait a week, you will be late when something really happened. Every failure detector picks a point on this curve between **detection time** and **false-suspicion rate**.

How it works internally

1. Each node sends a **heartbeat** every h seconds (push), or a monitor sends **pings** and expects replies (pull).
2. The monitor records the arrival time of the last heartbeat.
3. If nothing arrives within a **timeout** T_{out}, the node is suspected.

A crash happens at a random point in the heartbeat cycle, so the crash-to-suspicion time is between $T_{\text{out}} - h$ and T_{out}, about $T_{\text{out}} - h/2$ on average. A fixed timeout must exceed the worst normal gap from jitter, garbage-collection pauses and CPU starvation; three to five heartbeat intervals is a common rule. But one number cannot suit both a quiet LAN and a congested cross-region link.

27.10.2 The phi-accrual detector

Hayashibara et al. replaced the binary verdict with a continuous **suspicion level** φ that adapts to the observed heartbeat history; each application then picks its own threshold [Hayashibara et al. 2004].

FORMULA

Phi-accrual suspicion level (normal model)

$$\varphi(t_{\text{now}}) = -\log_{10}\left(1 - F(t_{\text{now}} - t_{\text{last}})\right), \qquad F(\delta) = \Phi\left(\frac{\delta - \mu}{\sigma}\right)$$

Symbol	Meaning	Units
φ	Suspicion level; the detector's false-suspicion probability is about $10^{-\varphi}$	dimensionless
$t_{\text{now}} - t_{\text{last}}$	Time since the last heartbeat arrived, δ	ms
μ, σ	Mean and standard deviation of recent inter-arrival times (sliding window)	ms
F	Estimated cumulative distribution of inter-arrival times	dimensionless
Φ	Standard normal cumulative distribution function	dimensionless

$1 - F(\delta)$ is the probability that a live node's next heartbeat would arrive even later than δ. If you suspect the node now, that is roughly the probability you are wrong. So $\varphi = 1$ means a 10% chance of a mistake, $\varphi = 2$ means 1%, and $\varphi = 8$ means 10^{-8}.

WORKED EXAMPLE

Worked example — When does phi reach 8? Heartbeats arrive every $\mu = 1,000$ ms with $\sigma = 100$ ms.

1. After 1,200 ms of silence: $z = (1,200 - 1,000)/100 = 2$; $1 - \Phi(2) = 0.0228$; $\varphi = -\log_{10} 0.0228 = 1.64$.
2. After 1,400 ms: $z = 4$; $1 - \Phi(4) = 3.17 \times 10^{-5}$; $\varphi = 4.50$.
3. After 1,500 ms: $z = 5$; 2.87×10^{-7}; $\varphi = 6.54$.
4. After 1,600 ms: $z = 6$; 9.87×10^{-10}; $\varphi = 9.01$.
5. Threshold 8 means $1 - \Phi(z) = 10^{-8}$, so $z = 5.61$: suspicion after $1,000 + 5.61 \times 100 = 1,561$ ms.
6. On a jittery link with $\sigma = 400$ ms, the same threshold waits $1,000 + 5.61 \times 400 = 3,245$ ms.

Interpretation. The same threshold produces a short timeout on a steady link and a long one on a noisy link, because it is a statement about error probability, not about time. **Engineering implication.** Set a floor on σ (otherwise a very regular sender makes the detector hair-trigger) and an allowance for expected pauses, then tune one threshold for the whole fleet.

```
        """A phi-accrual failure detector (Hayashibara et al. 2004) with a normal
model."""
        from collections import deque
        from math import erfc, log10, sqrt
        import random
        import statistics

        class PhiAccrualDetector:
            def __init__(self, window=1000, min_std_ms=50.0):
                self.intervals = deque(maxlen=window)   # recent heartbeat inter-arrival
times
                self.last = None
                self.min_std = min_std_ms              # floor so a very regular sender
is not hair-trigger

            def heartbeat(self, now_ms):
                if self.last is not None:
                    self.intervals.append(now_ms - self.last)
                self.last = now_ms

            def phi(self, now_ms):
                if len(self.intervals) < 2:
                    return 0.0
                mu = statistics.fmean(self.intervals)
                sigma = max(statistics.stdev(self.intervals), self.min_std)
                z = (now_ms - self.last - mu) / sigma
                p_later = 0.5 * erfc(z / sqrt(2))       # P(next heartbeat arrives even
later)
                return -log10(max(p_later, 1e-300))

        if __name__ == "__main__":
            random.seed(42)
            d = PhiAccrualDetector()
            t = 0.0
            for _ in range(1000):                       # heartbeats every ~1,000 ms, sd
~100 ms
                t += random.gauss(1000, 100)
                d.heartbeat(t)
            mu = statistics.fmean(d.intervals)
            sd = statistics.stdev(d.intervals)
            print(f"learned mean {mu:.0f} ms, std {sd:.0f} ms")
            for silence in (1000, 1200, 1400, 1500, 1600, 2000):
                print(f"silent for {silence:>4} ms -> phi = {d.phi(t + silence):6.2f}")
            s = 1000
            while d.phi(t + s) < 8:                     # when does phi cross the
threshold of 8?
                s += 1
            print(f"phi reaches 8 after {s} ms of silence")
```

Output:

```
learned mean 998 ms, std 102 ms
silent for 1000 ms -> phi =   0.31
silent for 1200 ms -> phi =   1.62
silent for 1400 ms -> phi =   4.39
silent for 1500 ms -> phi =   6.36
silent for 1600 ms -> phi =   8.74
silent for 2000 ms -> phi =  22.34
phi reaches 8 after 1571 ms of silence
```

What to notice: the detector learns μ and σ from a sliding window of 1,000 intervals; `erfc` computes the normal tail without losing precision; and the learned values (998 ms, 102 ms) give a crossing at 1,571 ms, close to the 1,561 ms of the worked example.

INDUSTRY EXAMPLE

Cassandra and Akka. Cassandra marks a node down when its phi value exceeds `phi_convict_threshold`, default 8. Its implementation models gossip inter-arrival times with an exponential rather than a normal distribution, so φ grows linearly with silence [Apache Cassandra Docs; Cassandra source, `FailureDetector`]. Akka Cluster uses the normal model with a configurable threshold, a minimum standard deviation and an "acceptable heartbeat pause" [Akka Docs: Phi Accrual Failure Detector]. *Evidence: Primary* — [Apache Cassandra Docs; Akka Docs]

27.10.3 Gossip and SWIM

All-to-all heartbeating costs $n(n-1)/h$ messages per second: 999,000 per second for 1,000 nodes at one per second. Large clusters instead spread information **epidemically**: each node periodically contacts a few random peers and exchanges what it knows, as an infection spreads [Demers et al. 1987].

DEFINITION

Definition — Gossip protocol. A protocol in which, every round, each node exchanges state with f randomly chosen peers (the **fanout**). In **push** gossip informed nodes send updates; in **pull** gossip nodes ask peers for news; **push–pull** does both. **Anti-entropy** gossip exchanges full state; **rumor mongering** spreads only recent updates and stops after a while [Demers et al. 1987].

FORMULA

Rounds for push gossip to reach every node

$$T_{\text{rounds}} \approx \log_{1+f} n + \frac{\ln n}{f} \qquad (\text{for } f = 1:\ \log_2 n + \ln n + O(1))$$

Symbol	Meaning	Units
T_{rounds}	Rounds until every node is informed	rounds
n	Number of nodes	nodes
f	Peers contacted per node per round	peers/round

Derivation. Early on almost every target is uninformed, so the informed set grows by a factor of about $1 + f$ per round: $\log_{1+f} n$ rounds to reach most nodes. At the end, each uninformed node escapes all f pushes from nearly n informed nodes with probability about e^{-f} per round, so the uninformed count shrinks by e^{-f} per round and the last one is reached after about $\ln n/f$ more rounds. Pittel proved the $f = 1$ case [Pittel 1987]. Pull shrinks the uninformed fraction from p to about p^2 per round at the end, so push–pull finishes in $\log_3 n + O(\log \log n)$ rounds [Karp et al. 2000].

WORKED EXAMPLE

Worked example — Spreading a membership change to 1,000 nodes.

1. Push, $f = 1$: $\log_2 1,000 + \ln 1,000 = 9.97 + 6.91 = 16.9$ rounds.
2. Push, $f = 3$: $\log_4 1,000 + 6.91/3 = 4.98 + 2.30 = 7.3$ rounds.
3. Growth with size, $f = 1$: going to 10,000 nodes adds $\log_2 10 + \ln 10 = 3.32 + 2.30 = 5.6$ rounds.

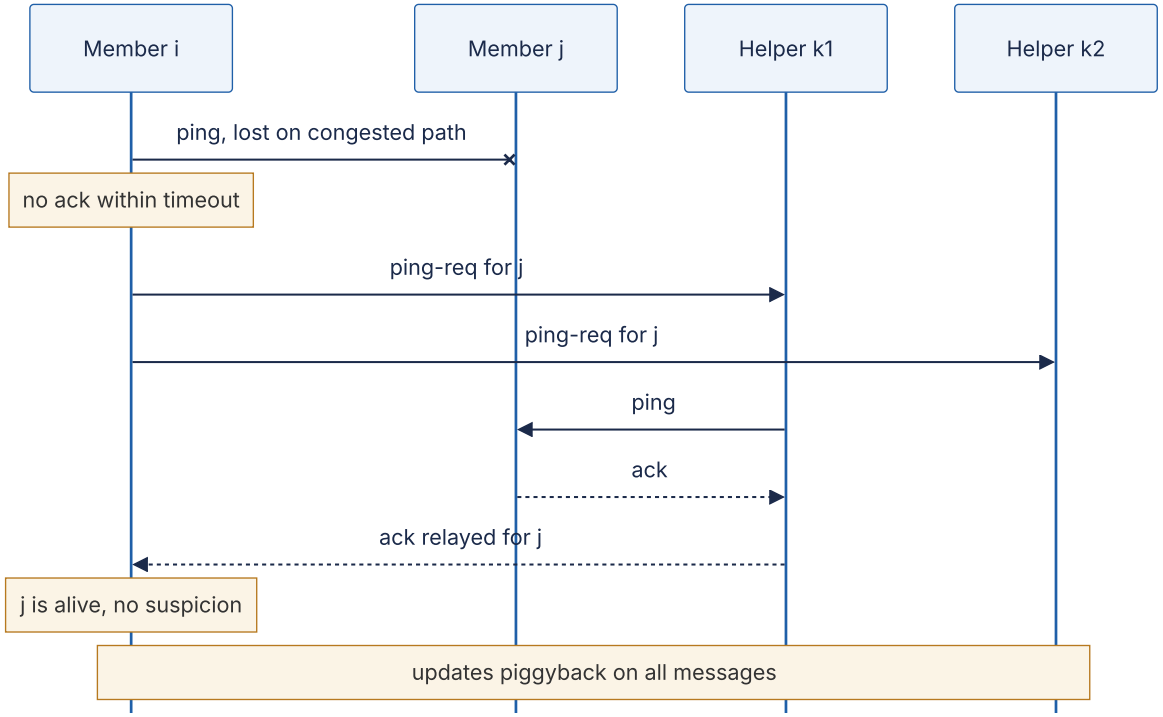
Interpretation. Dissemination time grows with $\log n$, and each node's message load is constant (f per round) regardless of cluster size. **Engineering implication.** Gossip scales membership and failure news to thousands of nodes, but a full spread takes tens of rounds; anything that must take effect everywhere *at once* belongs in a consensus-backed store (Chapter 26).

SWIM combines a randomized failure detector with gossip dissemination [Das et al. 2002]. In each **protocol period** T', every member:

1. Picks a random member M_j and sends it a `ping`.
2. If no `ack` arrives within a short timeout, sends `ping-req` to k other random members, asking them to ping M_j and relay any ack. This indirect probe avoids blaming M_j for a congested link between two particular nodes.

3. If still no ack, marks M_j **suspect**, rather than dead, and spreads the suspicion. M_j can refute it by gossiping an “alive” message with a higher **incarnation number**; if nobody refutes it within a suspicion timeout, M_j is declared failed.
4. Membership updates (joins, suspicions, failures) are **piggybacked** on ping and ack messages, so dissemination costs no extra messages.

Figure 27.9 — A SWIM protocol period with an indirect probe.



Reading the diagram. All messages are small UDP datagrams in typical implementations. Member *i*’s direct ping is lost, so it asks two helpers to probe *j* for it; one helper’s ping succeeds and its ack is relayed back, so *j* is not suspected. Had both indirect probes failed, *i* would gossip “*j* suspect”, and *j* would have a suspicion timeout to refute it. No member stores more than the membership list. Failure points: a member that is itself overloaded may miss acks and wrongly suspect others, which the Lifeguard extensions address by lengthening its own timeouts [Dadgar et al. 2017]. Message load per member is constant as the group grows.

FORMULA

SWIM expected time to first detection

$$\mathbb{E}[T_{\text{detect}}] = \frac{T'}{1 - (1 - \frac{1}{n-1})^{n-1}} \quad \overset{n \to \infty}{\to} \quad T' \cdot \frac{e}{e-1} \approx 1.58\,T'$$

Symbol	Meaning	Units
T'	Protocol period	s
n	Group size	members
$\mathbb{E}[T_{\text{detect}}]$	Expected time until some member first probes the failed member	s

Derivation. Each of the $n-1$ live members picks the failed member with probability $1/(n-1)$, so the chance that none does in a period is $(1 - \frac{1}{n-1})^{n-1} \approx e^{-1}$. The number of periods until someone does is geometric with success probability $1 - e^{-1}$, giving a mean of $1/(1 - e^{-1}) = e/(e-1)$.

WORKED EXAMPLE

Worked example — Detection budget for 1,000 members. $T' = 1$ s, $n = 1,000$.

1. $(1 - 1/999)^{999} = 0.3677$, so $\mathbb{E}[T_{\text{detect}}] = 1/(1 - 0.3677) = 1.58$ s.
2. Add a suspicion timeout of, say, 5 s for refutation: about 6.6 s from crash to declaration.
3. Spreading the verdict by piggybacking takes $O(\log n)$ further periods, about 10–17 at this size.
4. Message load: about 2 messages (ping and ack) per member per period, about 2,000 per second cluster-wide, versus 999,000 for all-to-all heartbeats.

Interpretation. Detection time is independent of group size; only dissemination grows, logarithmically. **Engineering implication.** SWIM-style membership is the standard choice for clusters of hundreds to thousands of nodes; tune the suspicion timeout, not the probe rate, to trade false positives for speed.

The simulation below measures push, pull and push–pull dissemination and SWIM’s first-detection time.

```
        """Rounds for a rumor to reach every node: push, pull and push-pull gossip,
        plus SWIM's expected time to first detection of a crashed member."""
        import math
        import random
        import statistics

        def spread(n, mode, fanout=1, rng=random):
            """Synchronous rounds: every node contacts `fanout` random peers per
round."""
            informed = [False] * n
            informed[0] = True
            count, rounds = 1, 0
            while count < n:
                rounds += 1
                new = informed[:]                       # updates take effect next
round
                for node in range(n):
                    for _ in range(fanout):
                        peer = rng.randrange(n - 1)
                        peer += peer >= node            # a random peer other than it-
self
                        if mode in ("push", "push-pull") and informed[node]:
                            new[peer] = True            # push: informed node tells
peer
                        if mode in ("pull", "push-pull") and informed[peer]:
                            new[node] = True            # pull: node asks peer for news
                informed = new
                count = sum(informed)
            return rounds

        def swim_first_detection(n, rng=random):
            """Protocol periods until some live member pings the one crashed member."""
            periods = 0
            while True:
                periods += 1
                # each of the n-1 live members pings one random other member
                if any(rng.randrange(n - 1) == 0 for _ in range(n - 1)):
                    return periods

        if __name__ == "__main__":
            rng = random.Random(2026)
            n, trials = 1000, 100
            print(f"n = {n}, fanout 1, {trials} trials each")
            for mode in ("push", "pull", "push-pull"):
                r = [spread(n, mode, 1, rng) for _ in range(trials)]
                print(f"  {mode:9s}: mean {statistics.fmean(r):5.2f} rounds, max
{max(r)}")
            print(f"  model log2(n) + ln(n) = {math.log2(n) + math.log(n):.2f} rounds
(push)")
            for f in (2, 3):
                r = [spread(n, "push", f, rng) for _ in range(trials)]
                model = math.log(n, 1 + f) + math.log(n) / f
                print(f"  push, fanout {f}: mean {statistics.fmean(r):5.2f}, model
{model:5.2f}")
            for size in (100, 1000, 10000):
                print(f"  push, n = {size:>5}: mean "
```

```
                      f"{statistics.fmean(spread(size, 'push', 1, rng) for _ in
range(30)):5.2f} rounds")

            d = [swim_first_detection(n, rng) for _ in range(5000)]
            print(f"SWIM first detection: mean {statistics.fmean(d):.3f} periods, "
                  f"e/(e-1) = {math.e / (math.e - 1):.3f}")
```

Output:

```
n = 1000, fanout 1, 100 trials each
  push     : mean 17.99 rounds, max 22
  pull     : mean 13.66 rounds, max 19
  push-pull: mean  9.13 rounds, max 10
  model log2(n) + ln(n) = 16.87 rounds (push)
  push, fanout 2: mean 10.62, model  9.74
  push, fanout 3: mean  8.28, model  7.29
  push, n =   100: mean 12.23 rounds
  push, n =  1000: mean 18.33 rounds
  push, n = 10000: mean 23.70 rounds
SWIM first detection: mean 1.597 periods, e/(e-1) = 1.582
```

What to notice:

- Push gossip takes about one round more than the model, the $O(1)$ term; each tenfold increase in n adds about 5.6 rounds, as predicted.
- Push–pull halves the rounds because pull mops up the last uninformed nodes quadratically fast.
- SWIM's simulated detection time matches $e/(e-1)$ periods.

COMMON MISTAKE

Mistake — "A failure detector tells you a node is dead." It tells you a node is *suspected*. A suspected primary may still be running and writing. Every action taken on suspicion, such as promoting a replica, must fence the suspect (§26.13).

27.11 Failover

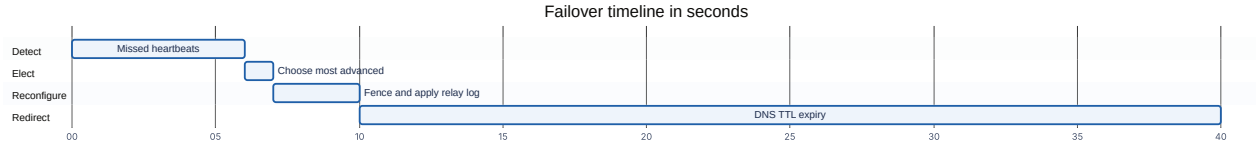
27.11.1 From detection to redirection

DEFINITION

Definition — Failover. The sequence that replaces a failed node in a role: **detection**, **election** of a replacement, **reconfiguration** (making the replacement consistent and fencing the old node) and **client redirection**.

Leaderless systems need no failover for writes; primary–backup, chain and consensus systems do. Each step consumes part of the availability budget.

Figure 27.10 — A failover time budget with DNS redirection.



Reading the diagram. Time runs left to right from the crash; each bar must finish before the next starts, so the steps are sequential. Detection waits for the timeout; election picks the replica holding the most acknowledged log (or runs a consensus election, §26.5); reconfiguration fences the old primary, applies any unapplied log and switches the replica to read–write mode; redirection waits for clients to learn the new address. The DNS bar dominates. The failure points are a premature detection (split brain if fencing fails) and redirection that never completes for clients that cache addresses beyond the TTL.

FORMULA

Failover time budget

$$T_{\text{fo}} = T_{\text{detect}} + T_{\text{elect}} + T_{\text{reconfig}} + T_{\text{redirect}} \qquad D_{\text{yr}} = v \cdot T_{\text{fo}}$$

Symbol	Meaning	Units
T_{fo}	Time from crash until clients write successfully again	s
T_{detect}	Time to suspect the failure	s
T_{elect}	Time to choose a replacement	s
T_{reconfig}	Time to fence, catch up and promote	s
T_{redirect}	Time until clients reach the replacement	s
v	Failovers per year	1/year
D_{yr}	Write downtime per year from failovers	s/year

WORKED EXAMPLE

Worked example — DNS versus a proxy. Heartbeats every 2 s with a 6 s timeout; election 1 s; reconfiguration 3 s.

1. DNS redirection with a 30 s TTL: $T_{\text{fo}} = 6 + 1 + 3 + 30 = 40$ s.
2. Proxy or virtual IP that switches in 1 s: $T_{\text{fo}} = 6 + 1 + 3 + 1 = 11$ s.
3. With $v = 6$ failovers a year (crashes, kernel patches, zone events): $6 \times 40 = 240$ s versus $6 \times 11 = 66$ s.
4. Against a 99.99% target (52.6 min = 3,156 s per year, §4.14): DNS failovers use 7.6% of the budget, the proxy 2.1%. Against a monthly budget of 4.32 min, one DNS failover uses 15%.

Interpretation. Redirection, not detection, is often the largest term. Consensus-replicated systems shrink detection and election together to about 0.16–1.2 s (§26.7.2), which leaves redirection even more dominant. **Engineering implication.** Put a proxy, a virtual IP or a topology-aware client in front of a replicated primary, and measure each term in game days rather than assuming it.

Dangers

- **Split brain.** If the old primary is merely slow or partitioned, it may still accept writes. Both "primaries" then diverge (§26.12). Only fencing, by lease expiry, a fencing token checked by storage, or cutting power, makes promotion safe.
- **Data loss with asynchronous replication.** The promoted replica lacks the last $r_w \ell$ writes (§27.4). Worse, when the old primary returns, its unreplicated writes conflict with new writes that reused the same keys or IDs.

INDUSTRY EXAMPLE

GitHub, October 2018. A 43-second loss of connectivity between GitHub's US East Coast network hub and its primary East Coast datacenter led its MySQL failover tooling (Orchestrator) to promote replicas in the US West Coast. When connectivity returned, the East Coast databases held writes not present in the West and vice versa, so GitHub could not simply fail back. It chose data integrity over speed and ran degraded for 24 hours and 11 minutes while reconciling; its follow-ups included preventing automatic cross-region promotion. *Evidence: Primary* — [GitHub 2018]

27.12 Recovery

27.12.1 Catching up, snapshotting and rebuilding

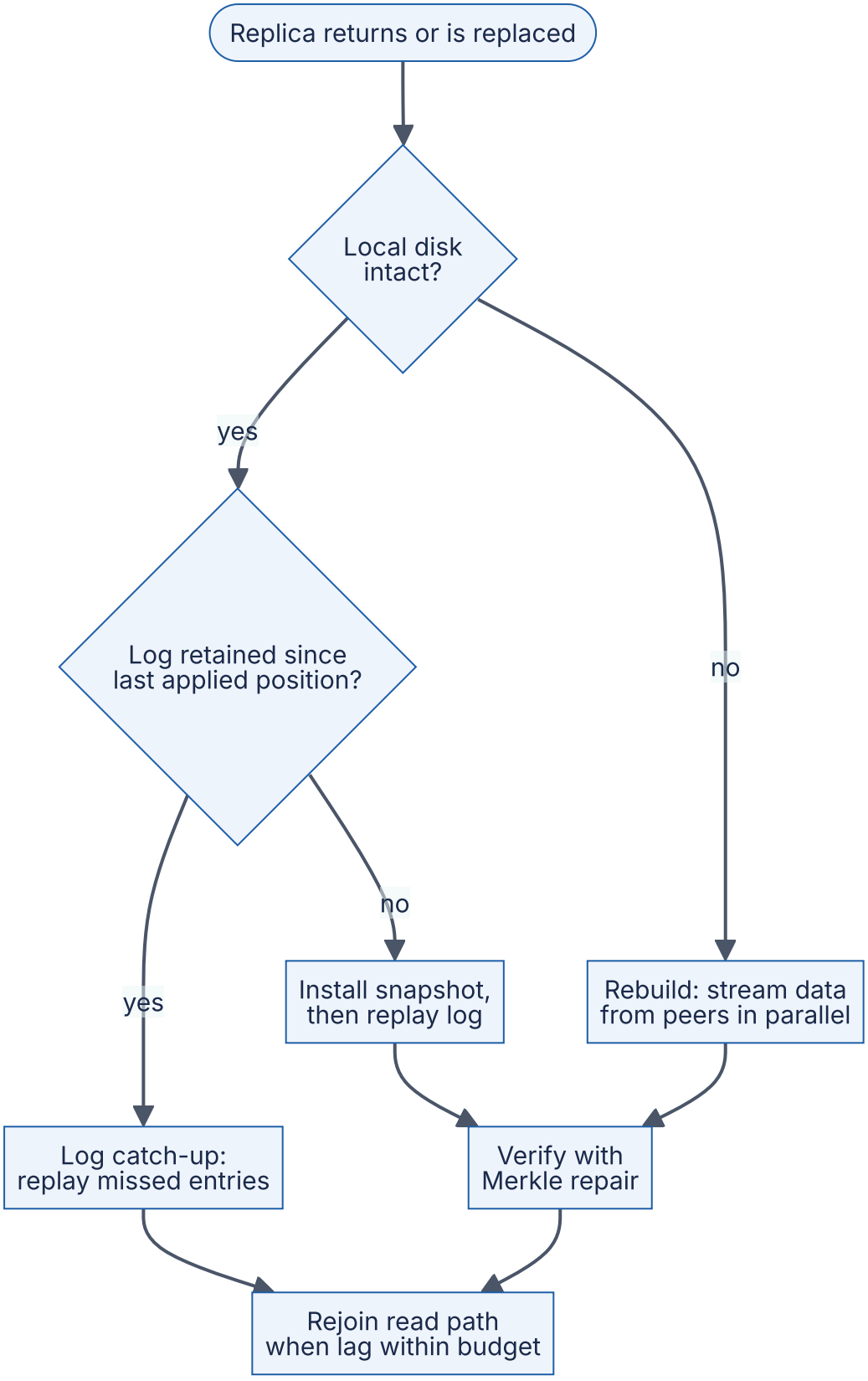
DEFINITION

Definition — Replica recovery. Returning a replica to full membership after a crash or replacement, by **log catch-up** (replaying missed log entries), **snapshot plus log** (installing a copy of the state, then replaying from the snapshot's position) or **rebuild** (copying all data from peers).

How it works internally

1. **Log catch-up.** If the node's disk survived and the primary still retains every log entry since the node's last applied position, the node replays them (§14.4.2). PostgreSQL retains WAL for a replica through `wal_keep_size` or a **replication slot**; MySQL keeps binlogs for `binlog_expire_logs_seconds`.
2. **Snapshot plus log.** If the log has been truncated past the node's position, it installs a snapshot taken at a known log position and replays from there. PostgreSQL's `pg_basebackup` and Raft's `InstallSnapshot` (§26.7) work this way.
3. **Rebuild.** A replacement node in a leaderless store streams its token ranges from the surviving replicas; with virtual nodes (§14.8), many peers send in parallel. New writes for those ranges go to the joining node while it streams.

Figure 27.11 — Choosing a recovery path.



Reading the diagram. Decisions run top to bottom. Every path ends at the same gate: the replica rejoins the read path only once its lag is within the staleness budget. Snapshot and rebuild paths transfer bulk data over the network and are throttled to protect the sources; log catch-up streams only missed entries. Storage boundaries are the source’s

log retention (which decides the middle branch) and the new node's disk. Failure points: a second failure during a long rebuild, which leaves the data with fewer copies (§4.16), and a source overwhelmed by streaming.

FORMULA

Recovery time for snapshot plus log

$$T_{\text{rec}} = \frac{D}{B_{\text{eff}}} + \frac{r_{\text{in}} \cdot D/B_{\text{eff}}}{r_{\text{apply}} - r_{\text{in}}}, \qquad B_{\text{eff}} = \min\left(\sum_i B_{\text{src},i},\ B_{\text{dst}}\right)$$

Symbol	Meaning	Units
T_{rec}	Time until the replica is current	s
D	Size of the snapshot or data to copy	bytes
B_{eff}	Effective copy bandwidth	bytes/s
$B_{\text{src},i}$	Throttled send rate of source i	bytes/s
B_{dst}	Receiver limit: NIC or disk write rate	bytes/s
r_{in}	Rate of new log entries during the copy	entries/s
r_{apply}	Rate at which the replica replays entries	entries/s

The second term is the catch-up time of §14.4.2 applied to the backlog accumulated during the copy.

WORKED EXAMPLE

Worked example — Rebuilding a 2 TB replica. New writes arrive at $r_{\text{in}} = 5,000$ entries/s and the replica applies $r_{\text{apply}} = 8,000$ entries/s.

1. One source throttled to 250 MB/s: copy $= 2 \times 10^{12} / 2.5 \times 10^{8} = 8,000$ s (2.2 h). Backlog $= 5,000 \times 8,000 = 4 \times 10^{7}$ entries; catch-up $= 4 \times 10^{7} / 3,000 = 13,333$ s (3.7 h). Total about 5.9 h.
2. Twenty sources at 100 MB/s each (2 GB/s) into a 10 Gb/s NIC (1.25 GB/s): $B_{\text{eff}} = 1.25$ GB/s; copy $= 1,600$ s (27 min). Backlog 8×10^{6}; catch-up 2,667 s (44 min). Total about 71 min.
3. Durability during recovery: from §4.16, the chance of losing a three-way replicated item grows with the square of repair time, so cutting 21,333 s to 4,267 s improves it by about $(21,333/4,267)^2 = 5^2 = 25$ times.

Interpretation. Recovery time is dominated by copy bandwidth and by apply headroom, and parallel sources help only until the receiver saturates. **Engineering implication.** Spread replicas with virtual nodes, keep apply headroom, and retain enough log that short outages never need a full copy.

WARNING / TRADE-OFF

Warning — A rejoining replica can resurrect deleted data. Leaderless stores record deletes as **tombstones** and purge them after a grace period (Cassandra's `gc_grace_seconds`, 10 days by default). A replica that was down longer than that, or never repaired within it, still holds the deleted rows and will spread them back. Repair every replica within the grace period, and never return a long-dead node without a full repair.

27.13 Anti-entropy and read repair

27.13.1 Repairing on the read path and in the background

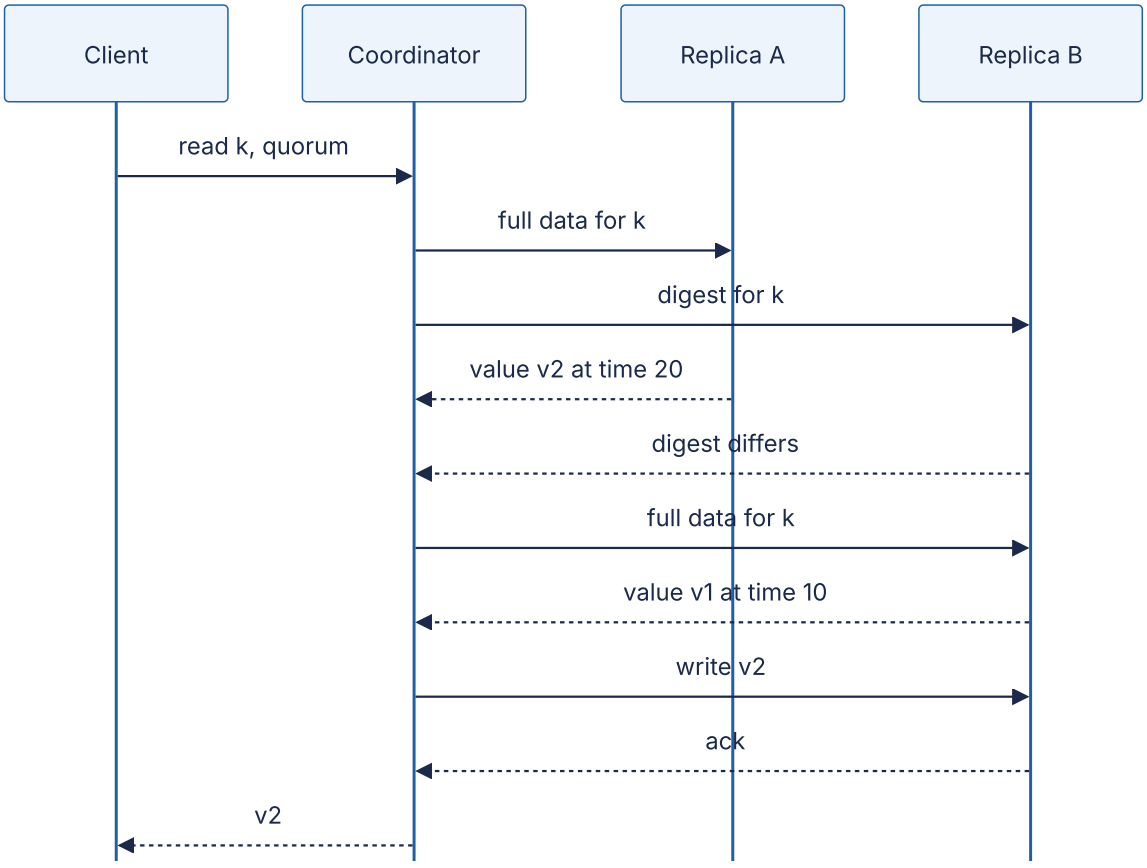
DEFINITION

Definition — Read repair and anti-entropy. Read repair fixes stale replicas discovered while serving a read: the coordinator writes the newest version back to replicas that returned older ones. **Anti-entropy** is a background process that compares whole replicas, typically with Merkle trees, and copies the differences [DeCandia et al. 2007; Demers et al. 1987].

Hints can be lost and cover only short outages. Two further mechanisms close the gaps:

1. **Read repair.** On a read the coordinator compares versions, usually by requesting full data from one replica and digests (hashes) from the others. On a mismatch it fetches full data, computes the newest version, returns it and writes it back to the stale replicas. **Blocking** read repair finishes the write-back before replying, which is what prevents the anomaly of Figure 27.7; **asynchronous** read repair replies first. Read repair only fixes keys that are read, so rarely read keys can stay divergent indefinitely.
2. **Merkle-tree anti-entropy.** Each replica builds a hash tree over each key range (§17.8). Two replicas compare roots, descend only into differing subtrees and stream only the divergent leaf ranges. For two replicas of a million-leaf tree that differ in two leaves, this compares about 80 hashes instead of the whole data set (§17.8).

Figure 27.12 — Blocking read repair with R = 2.



Reading the diagram. All calls are synchronous. Asking one replica for data and others for digests saves bandwidth when replicas agree, the common case. On a mismatch the coordinator fetches B's value, picks the newer one by timestamp (or by version vector), writes it to B and only then replies, so any later quorum read must meet a replica holding v2. The storage boundary is each replica's local store. Failure points: the extra round trips on mismatches inflate read tail latency, and a write-back racing a newer concurrent write must not overwrite it, which version comparison prevents.

INDUSTRY EXAMPLE

Cassandra repair. Cassandra's `nodetool repair` builds Merkle trees per token range on each replica, compares them and streams mismatched ranges; incremental repair limits the work to data written since the last repair. Operators are told to repair every node within `gc_grace_seconds` so that tombstones reach every replica before they are purged. *Evidence: Primary* — [Apache Cassandra Docs: Repair]

27.14 Conflict resolution: LWW, version vectors and CRDTs

27.14.1 Last-writer-wins

DEFINITION

Definition — Last-writer-wins (LWW). Each write carries a timestamp, and when replicas hold different values for an item, the one with the greatest timestamp is kept and the others are discarded.

LWW is simple, needs no application code and always converges, which is why Cassandra and DynamoDB global tables use it. Its cost is **silent data loss**: of two concurrent writes, one is discarded even though both were acknowledged. Worse, "last" is decided by clocks that can disagree by milliseconds or more (§24.9; §24.9.6 derives the danger window), so a write that happened *later* in real time can lose to one whose node's clock runs fast.

WORKED EXAMPLE

Worked example — Losing a later write to clock skew. Node A's clock runs 5 ms fast. At real time 98 ms a client removes "milk" from a list through A, stamped 103. At real time 100 ms another client re-adds "milk" through B, stamped 100.

1. Real-time order: remove (98), then add (100). The user expects milk on the list.
2. LWW order: add (100) < remove (103), so the remove wins on every replica.
3. Any two writes closer together than the skew bound ε can be misordered; with $\varepsilon = 5$ ms and a key written 10 times per second, about $10 \times 0.005 = 5\%$ of consecutive write pairs fall inside the window.

Interpretation. LWW is correct only when concurrent writes to the same item are rare or interchangeable. **Engineering implication.** Use LWW for immutable values or keys written by one client (insert-only events, cache entries), and something else for shared mutable state.

27.14.2 Version vectors and siblings

Instead of picking a winner, a store can **detect** concurrency and keep every concurrent value as a **sibling**, letting the application merge. Detection uses **version vectors** (§24.11.2; the runnable example of §24.11.3 also shows the same-replica flaw that dotted version vectors, §24.11.5, fix): a map from replica ID to a counter of writes coordinated by that replica.

FORMULA

Comparing version vectors

$$V \le V' \iff \forall i:\ V[i] \le V'[i] \qquad V \parallel V' \iff V \not\le V' \ \wedge\ V' \not\le V \qquad \text{merge: } (V \sqcup V')[i] = \max(V[i], V'[i])$$

Symbol	Meaning	Units
V, V'	Version vectors of two versions of the same item	counters per replica
$V[i]$	Number of writes to the item coordinated by replica i that this version has seen	count
$V \le V'$	V' descends from V, so V can be discarded	—
$V \parallel V'$	The versions are concurrent: keep both as siblings	—
$\sqcup$	Element-wise maximum, the vector of a merged version	—

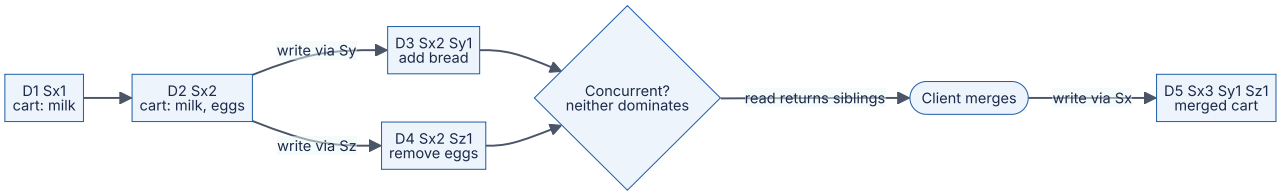
WORKED EXAMPLE

Worked example — The Dynamo shopping cart. Following the Dynamo paper's example, with servers S_x, S_y, S_z:

1. A write coordinated by S_x creates D1 with $\{S_x:1\}$; an update through S_x creates D2, $\{S_x:2\}$. Since $\{S_x:1\} \le \{S_x:2\}$, D1 is discarded.
2. Two clients read D2 and write concurrently: through S_y, D3 $= \{S_x:2, S_y:1\}$; through S_z, D4 $= \{S_x:2, S_z:1\}$.
3. Compare D3 and D4: S_y is 1 vs 0 and S_z is 0 vs 1, so neither dominates. A read returns both siblings.
4. The client merges them (for a cart, the union of items) and writes back through S_x with the merged context: D5 $= \{S_x:3, S_y:1, S_z:1\}$, which dominates both [DeCandia et al. 2007].

Interpretation. No write was lost, but the application had to merge, and a union merge re-adds an item that one sibling had deleted. **Engineering implication.** Siblings move conflict resolution into application code; give that code a deterministic, tested merge, or use a CRDT.

Figure 27.13 — Version vectors creating and resolving siblings.



Reading the diagram. Boxes are versions of one key, labeled with their version vectors; arrows mean "descends from". D3 and D4 are written through different coordinators from the same parent, so their vectors are incomparable and the store keeps both. A read returns both siblings with their combined context; the client's merged write dominates both and replaces them. Storage holds every sibling until a merge, so a key with many unmerged concurrent writers grows. The failure point is the merge function: a union resurrects "eggs". Dynamo truncated vectors beyond a threshold of about 10 entries, which can cause false conflicts.

INDUSTRY EXAMPLE

Riak siblings. With `allow_mult` enabled, Riak keeps concurrent writes as siblings and returns them with a causal context that the client sends back with its resolved write; Riak 2.0 added dotted version vectors (§24.11.5) to avoid sibling explosion and lost updates when many clients write through the same node, and built-in CRDT data types (counters, sets, maps, registers, flags) that merge automatically. *Evidence: Primary* — [Riak KV Docs: Conflict Resolution; Riak KV Docs: Data Types; Preguiça et al. 2010]

In production Dynamo, siblings were rare: the paper reports that 99.94% of requests saw exactly one version over a 24-hour period of the shopping cart service [DeCandia et al. 2007].

27.14.3 CRDTs

DEFINITION

Definition — Conflict-free replicated data type (CRDT). A data type whose replicas can be updated independently and concurrently without coordination and are guaranteed to converge to the same state once they have received the same updates, a property called **strong eventual consistency** [Shapiro et al. 2011a].

First-principles explanation

Two scorekeepers each keep a sheet with one column per scorekeeper, and each only ever increases its own column. Whenever they compare sheets, they take the larger number in each column and agree, however often and in whatever order they compare. A CRDT is a data structure designed so that "compare and combine" cannot go wrong.

How it works internally

There are two styles, and each can emulate the other [Shapiro et al. 2011b]:

- **State-based CRDTs (CvRDTs)** ship whole states and combine them with a **merge** function. Convergence requires the states to form a **join semilattice**: merge must be **commutative** ($a \sqcup b = b \sqcup a$), **associative** ($(a \sqcup b) \sqcup c = a \sqcup (b \sqcup c)$) and **idempotent** ($a \sqcup a = a$), and every update must move the state upward. Then gossip may deliver states in any order, any number of times. **Delta-state** CRDTs ship only recent changes to cut bandwidth [Almeida et al. 2018].
- **Operation-based CRDTs (CmRDTs)** ship operations. Concurrent operations must commute, and the delivery layer must deliver every operation exactly once and in causal order.

The four classic designs are:

1. **G-Counter** (grow-only counter): one slot per replica; a replica increments only its own slot; value is the sum; merge takes the per-slot maximum.
2. **PN-Counter:** two G-Counters, one for increments and one for decrements; value is the difference.
3. **LWW-Register:** a value with a (timestamp, replica ID) stamp; merge keeps the larger stamp. It is LWW, packaged so that merge is a lattice join.
4. **OR-Set** (observed-remove set): each add creates a unique tag; a remove deletes only the tags it has observed. A concurrent add's new tag survives, so **add wins**. Removed tags must be remembered as tombstones, unless an optimized design summarizes them with version vectors [Bieniusa et al. 2012].

The program implements all four and checks the three merge laws, plus convergence under every merge order, on random histories.

```
        """State-based CRDTs: G-Counter, PN-Counter, LWW-Register and OR-Set,
        with randomized tests that merge is commutative, associative and idempotent."""
        import copy
        import itertools
        import random

        class GCounter:
            """Grow-only counter: one slot per replica; merge takes the per-slot max."""
            def __init__(self, replica):
                self.replica, self.slots = replica, {}
            def increment(self, n=1):
                assert n >= 0, "a G-Counter only grows"
                self.slots[self.replica] = self.slots.get(self.replica, 0) + n
            def value(self):
                return sum(self.slots.values())
            def merge(self, other):
                for r, v in other.slots.items():
                    self.slots[r] = max(self.slots.get(r, 0), v)
            def state(self):
                return dict(sorted(self.slots.items()))

        class PNCounter:
            """Counter with decrements: one G-Counter for increments, one for decre-
ments."""
            def __init__(self, replica):
                self.p, self.n = GCounter(replica), GCounter(replica)
            def increment(self, k=1):
                self.p.increment(k)
            def decrement(self, k=1):
                self.n.increment(k)
            def value(self):
                return self.p.value() - self.n.value()
            def merge(self, other):
                self.p.merge(other.p)
                self.n.merge(other.n)
            def state(self):
                return (self.p.state(), self.n.state())

        class LWWRegister:
            """Last-writer-wins register; ties on timestamp broken by replica ID."""
            def __init__(self, replica):
                self.replica, self.value_, self.stamp = replica, None, (0, "")
            def assign(self, value, timestamp):
                stamp = (timestamp, self.replica)
                if stamp > self.stamp:
                    self.value_, self.stamp = value, stamp
            def value(self):
                return self.value_
            def merge(self, other):
                if other.stamp > self.stamp:
                    self.value_, self.stamp = other.value_, other.stamp
            def state(self):
                return (self.value_, self.stamp)

        class ORSet:
            """Observed-remove set: each add creates a unique tag; remove deletes only
```

```
            the tags it has observed, so a concurrent add survives (add wins)."""
            def __init__(self, replica):
                self.replica, self.adds, self.removes, self.seq = replica, set(), set(),
0
            def add(self, element):
                self.seq += 1                           # tag = (replica, sequence):
globally unique
                self.adds.add((element, (self.replica, self.seq)))
            def remove(self, element):
                self.removes |= {t for t in self.adds if t[0] == element}
            def value(self):
                return {e for (e, tag) in self.adds - self.removes}
            def merge(self, other):
                self.adds |= other.adds
                self.removes |= other.removes
            def state(self):
                return (sorted(self.adds), sorted(self.removes))

        def merged(a, b):
            """Return a copy of a merged with b, leaving both inputs unchanged."""
            out = copy.deepcopy(a)
            out.merge(b)
            return out

        def random_replicas(cls, rng, ops=30):
            replicas = [cls(name) for name in "ABC"]
            for step in range(ops):
                r = rng.choice(replicas)
                if cls is GCounter:
                    r.increment(rng.randint(1, 5))
                elif cls is PNCounter:
                    (r.increment if rng.random() < 0.6 else r.decrement)(rng.randint(1,
5))
                elif cls is LWWRegister:
                    r.assign(rng.choice("xyz"), timestamp=rng.randint(1, 20))
                else:
                    (r.add if rng.random() < 0.6 else r.remove)(rng.choice("abcd"))
                if rng.random() < 0.3:                  # occasional partial gossip
                    r.merge(rng.choice(replicas))
            return replicas

        def check_laws(cls, trials=200, seed=7):
            rng = random.Random(seed)
            for _ in range(trials):
                a, b, c = random_replicas(cls, rng)
                assert merged(a, b).state() == merged(b, a).state(), "commutativity"
                assert merged(merged(a, b), c).state() == merged(a, merged(b,
c)).state(), "associativity"
                assert merged(a, a).state() == a.state(), "idempotence"
                finals = [merged(merged(x, y), z).state() for x, y, z in
itertools.permutations((a, b, c))]
                assert all(s == finals[0] for s in finals), "convergence in every merge
order"
            return f"{cls.__name__:11s}: {trials} random histories, all laws hold"

        class SumCounter(GCounter):
            """A broken design: merge adds slots instead of taking the maximum."""
```

```
            def merge(self, other):
                for r, v in other.slots.items():
                    self.slots[r] = self.slots.get(r, 0) + v

        if __name__ == "__main__":
            for cls in (GCounter, PNCounter, LWWRegister, ORSet):
                print(check_laws(cls))

            bad = SumCounter("A"); bad.increment(5)
            print("SumCounter: value 5, after merging a duplicate delivery:",
merged(bad, bad).value())

            # Add-wins: A removes "milk" while B concurrently re-adds it.
            a, b = ORSet("A"), ORSet("B")
            a.add("milk"); b.merge(a)                    # both replicas see milk
            a.remove("milk")                             # A removes the tag it observed
            b.add("milk")                                # B adds again with a fresh tag
            a.merge(b); b.merge(a)
            print("OR-Set after concurrent remove and add:", a.value(), b.value())

            # The same story with LWW on timestamps loses B's add if A's clock is ahead.
            ra, rb = LWWRegister("A"), LWWRegister("B")
            ra.assign("<removed>", timestamp=103)        # A removes at real time 98;
its clock is 5 ms fast
            rb.assign("milk", timestamp=100)             # B re-adds later, at real time
100
            ra.merge(rb); rb.merge(ra)
            print("LWW register after the same history:", ra.value(), rb.value())
```

Output:

```
GCounter   : 200 random histories, all laws hold
PNCounter  : 200 random histories, all laws hold
LWWRegister: 200 random histories, all laws hold
ORSet      : 200 random histories, all laws hold
SumCounter: value 5, after merging a duplicate delivery: 10
OR-Set after concurrent remove and add: {'milk'} {'milk'}
LWW register after the same history: <removed> <removed>
```

What to notice:

- Each `merge` is a lattice join: per-slot `max`, set union, or the larger stamp. That is why all three laws hold on every random history, including histories with partial gossip between replicas.
- `SumCounter` looks plausible but is not idempotent: a duplicated gossip message doubles the count. Idempotence is what lets state-based CRDTs ride on unreliable, repeating gossip.
- The OR-Set keeps “milk” after a concurrent remove and re-add, matching user intent; the LWW register replays the clock-skew example of §27.14.1 and drops the later add.

Real-world industry applications

INDUSTRY EXAMPLE

Redis active-active databases and collaborative editors. Redis Enterprise's Active-Active databases replicate across regions with every replica writable and resolve concurrent updates with CRDT-based data types, such as counters that sum concurrent increments and sets with add-wins semantics [Redis Docs: Active-Active]. Libraries such as Automerge and Yjs apply sequence CRDTs to collaborative text and JSON documents. *Evidence: Primary* — [Redis Docs: Active-Active; Automerge and Yjs project documentation]

Alternatives and trade-offs

Table 27.4 — Conflict-resolution strategies.

	Last-writer-wins	Version vectors + siblings	CRDTs
What it is	Keep greatest timestamp	Keep all concurrent values	Types whose merge always converges
Data loss	Silent, on concurrent writes	None	None, within the type's semantics
Application work	None	Write a merge for every type	Choose a type that fits
Metadata	One timestamp	One counter per coordinating replica	Per-replica slots or tags; tombstones
Clock dependence	Yes	No	Only for LWW-based types
Choose when	Immutable or single-writer data	Domain-specific merges	Counters, sets, maps, collaborative documents

Failure scenarios

Table 27.5 — Failure scenarios in conflict resolution.

What goes wrong	Why	Detection	Recovery	Prevention
Acknowledged update disappears	LWW discarded a concurrent write	Audit log shows the write; final value lacks it	Replay from audit log	Siblings or CRDTs for shared mutable data
Deleted item reappears	Union merge of siblings, or tombstone purged before repair	User reports	Delete again	OR-Set semantics; repair within grace period
Sibling explosion	Many clients write without causal context	Value size and sibling count metrics	Merge and write back	Always read before write; dotted version vectors
CRDT state grows without bound	Tombstones and per-replica entries never collected	State size metrics	Compaction at causal stability	Optimized OR-Set; retire replica IDs

COMMON MISTAKE

Mistake — "CRDTs make every invariant safe." CRDTs guarantee convergence, not application invariants. A PN-Counter for stock can converge to −3 because two regions each sold the last two items. Invariants such as "never below zero" or "unique username" still need coordination (Chapter 26) or escrow-style partitioning of the allowance.

27.15 Summary and revision notes

SUMMARY

- Replication decides who orders writes and how many copies hold a write before acknowledgment; multi-leader and leaderless designs diverge and repair afterwards.
- Primary–backup ships state or operations; asynchronous failover loses $r_w \ell$ writes, and safe promotion needs fencing.
- Chain replication: writes at the head, strong reads at the tail, $T_{\text{write}} \approx (n+1)d + ns$; CRAQ serves clean reads from any node.
- Multi-leader serves multi-region, offline and collaborative writers; ring and star have single points of failure, all-to-all reorders messages; conflicts per item $\approx 2\lambda_A \lambda_B \Delta$.
- $R + W > N$ guarantees overlap with completed writes, not linearizability. Quorum latency is the k-th order statistic: 2-of-3 hides the slowest replica.
- Sloppy quorums and hinted handoff trade overlap for write availability; hints grow linearly with downtime.
- Phi-accrual turns silence into an error probability; SWIM detects in about $1.58\,T'$ with constant load per member; gossip spreads in $\approx \log_{1+f} n + \ln n/f$ rounds.
- Failover time is detection + election + reconfiguration + redirection; redirection often dominates. Recovery time is copy time plus catch-up of the backlog.
- Read repair fixes keys that are read; Merkle anti-entropy fixes everything, and must run within the tombstone grace period.
- LWW loses concurrent writes silently; version vectors keep siblings; CRDTs converge because merge is commutative, associative and idempotent, but do not protect invariants.

27.16 Knowledge checkpoint

1. What is the difference between state transfer and operation shipping, and which needs determinism?
2. An asynchronous backup lags by 300 ms when the primary, taking 4,000 writes/s, crashes. How many acknowledged writes are lost?
3. Why are reads at the tail of a chain linearizable?
4. For $N = 5$, what is the smallest R that overlaps $W = 3$, and how many failures can each tolerate?
5. Give two reasons why $R + W > N$ does not give linearizability.
6. With $N = 3$ and each replica independently slow with probability 0.02, what fraction of $W = 2$ writes are slow?
7. Heartbeats have $\mu = 500$ ms and $\sigma = 50$ ms. What is φ after 600 ms of silence?

8. Estimate push-gossip rounds with fanout 1 for 4,096 nodes.
9. Detection 4 s, election 2 s, reconfiguration 2 s, DNS TTL 60 s: what is the failover time, and what is the biggest lever?
10. Why must a state-based CRDT merge be idempotent?

Answers

1. State transfer sends resulting state; operation shipping sends the operation, which every replica must execute deterministically.
2. $4,000 \times 0.3 = 1,200$ writes.
3. The tail holds only updates every other chain node already has, and it is the only node that replies to writes and reads.
4. $R = 3$ ($3 + 3 > 5$); reads and writes each tolerate 2 unavailable replicas.
5. Any two of: concurrent writes resolved by timestamp; writes concurrent with reads; failed writes not rolled back; sloppy quorums; replicas restored from stale copies.
6. $p^3 + 3p^2(1-p) = 8 \times 10^{-6} + 3 \times 0.0004 \times 0.98 = 0.001184$, about 0.12%.
7. $z = 2$, $1 - \Phi(2) = 0.0228$, $\varphi = 1.64$.
8. $\log_2 4,096 + \ln 4,096 = 12 + 8.3 \approx 20$ rounds, plus about one.
9. $4 + 2 + 2 + 60 = 68$ s; replace DNS redirection with a proxy, virtual IP or topology-aware client.
10. Gossip may deliver the same state many times; a non-idempotent merge (such as summing) would count it repeatedly.

27.17 Interview questions

Basic

Q1. What does R + W > N give you? Answer. Every read set intersects every completed write's set, so a read sees at least one replica with that write. It does not order concurrent writes, roll back failed ones or prevent a later read seeing an older value without blocking read repair.

Q2. How do you detect a failed node? Answer. Heartbeats or pings with a timeout, ideally adaptive (phi-accrual) and, in large clusters, SWIM-style probing with indirect pings, suspicion and gossip. A detector only suspects; actions taken on suspicion must fence.

Intermediate

Q3. Walk through a database failover and its time budget. Answer. Detection (timeout), election (most advanced replica or consensus), reconfiguration (fence, apply remaining log, promote), redirection (proxy, VIP or DNS). Quantify each term; fence against split brain; expect $r_w \ell$ loss if replication is asynchronous.

Q4. Chain replication or primary-backup? Answer. Chain spreads network load and gives strong tail reads, but write latency grows with length and one slow node slows every write. Primary–backup has constant hops but the primary sends $n-1$ copies. Choose chain or CRAQ for high-throughput storage with a reliable master.

Advanced

Q5. Design an inventory counter replicated in three regions with local writes. Answer. A PN-Counter converges but cannot enforce "stock ≥ 0". Partition the stock into per-region allowances (escrow), sell locally within the allowance, and rebalance allowances with a coordinated transfer when one runs low; strong checks only at the boundary.

Q6. A Cassandra cluster shows deleted rows reappearing after a node replacement. Explain and fix. Answer. A replica missed deletes and was not repaired within `gc_grace_seconds`; others purged their tombstones, so repair copied the old rows back. Repair every node within the grace period and alert on repair age.

Q7. Compare gossip dissemination and a consensus-backed store for cluster membership. Answer. Gossip spreads changes in $O(\log n)$ rounds with constant per-node load, but views differ briefly. Consensus gives one agreed view and fencing at the cost of quorum round trips. Many systems use gossip for liveness and consensus for authoritative ownership.

27.18 Practical exercises

1. Extend the quorum program to estimate p99 read and write latency for $N = 5$ from a measured latency sample, for every (R, W) with $R + W > N$.
2. Add a suspicion timeout and incarnation numbers to the SWIM simulation and measure false positives with 5% packet loss.
3. Implement a delta-state G-Counter that ships only changed slots and measure bytes sent against the full-state version.

27.19 Design challenges

1. **Global shopping cart.** Three regions, writes always accepted, no lost additions. Choose replication, conflict resolution and repair, and explain deleted-item behavior.
2. **Metadata store failover.** A primary–backup metadata service must fail over in under 10 s with zero acknowledged-write loss. Produce the time budget and fencing design.
3. **Membership for 5,000 nodes.** Choose protocol parameters so a crash is declared within 10 s with a false-positive rate below one per day.

27.20 References

1. Almeida, P. S., Shoker, A., Baquero, C. *Delta State Replicated Data Types*. Journal of Parallel and Distributed Computing 111, 2018.
2. Alsberg, P. A., Day, J. D. *A Principle for Resilient Sharing of Distributed Resources*. Proc. 2nd International Conference on Software Engineering (ICSE), 1976.
3. Andersen, D. G., Franklin, J., Kaminsky, M., Phanishayee, A., Tan, L., Vasudevan, V. *FAWN: A Fast Array of Wimpy Nodes*. ACM SOSP, 2009.
4. Akka. *Phi Accrual Failure Detector*. Akka documentation, accessed 2026.
5. Apache Cassandra. *Documentation*: Dynamo architecture; Hints; Read repair; Repair; `cassandra.yaml`. Accessed 2026. https://cassandra.apache.org/doc/latest/
6. Apache CouchDB. *Replication and Conflict Model*. CouchDB documentation, accessed 2026.
7. Apache Kafka. *Documentation: Replication*. Accessed 2026. https://kafka.apache.org/documentation/
8. Attiya, H., Bar-Noy, A., Dolev, D. *Sharing Memory Robustly in Message-Passing Systems*. Journal of the ACM 42(1), 1995.
9. AWS. *Amazon DynamoDB Developer Guide: Global Tables*. Accessed 2026.
10. Bieniusa, A., Zawirski, M., Preguiça, N., Shapiro, M., Baquero, C., Balegas, V., Duarte, S. *An Optimized Conflict-free Replicated Set*. arXiv, 2012.
11. Budhiraja, N., Marzullo, K., Schneider, F. B., Toueg, S. *The Primary-Backup Approach*. In Mullender, S. (ed.), *Distributed Systems*, 2nd ed., ACM Press/Addison-Wesley, 1993.
12. Chandra, T. D., Toueg, S. *Unreliable Failure Detectors for Reliable Distributed Systems*. Journal of the ACM 43(2), 1996.
13. Dadgar, A., Phillips, J., Currey, J. *Lifeguard: SWIM-ing with Situational Awareness*. arXiv, 2017.
14. Das, A., Gupta, I., Motivala, A. *SWIM: Scalable Weakly-consistent Infection-style Process Group Membership Protocol*. IEEE/IFIP DSN, 2002.
15. DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., Vogels, W. *Dynamo: Amazon's Highly Available Key-value Store*. ACM SOSP, 2007.
16. Demers, A., Greene, D., Hauser, C., Irish, W., Larson, J., Shenker, S., Sturgis, H., Swinehart, D., Terry, D. *Epidemic Algorithms for Replicated Database Maintenance*. ACM PODC, 1987.
17. GitHub. *October 21 post-incident analysis*. GitHub Blog, 2018. https://github.blog/news-insights/company-news/oct21-post-incident-analysis/
18. Hayashibara, N., Défago, X., Yared, R., Katayama, T. *The φ Accrual Failure Detector*. IEEE SRDS, 2004.
19. Karp, R., Schindelhauer, C., Shenker, S., Vöcking, B. *Randomized Rumor Spreading*. IEEE FOCS, 2000.

20. Kleppmann, M. *Designing Data-Intensive Applications*, ch. 5 “Replication”. O’Reilly, 2017.
21. Lakshman, A., Malik, P. *Cassandra: A Decentralized Structured Storage System*. ACM SIGOPS Operating Systems Review 44(2), 2010.
22. Pittel, B. *On Spreading a Rumor*. SIAM Journal on Applied Mathematics 47(1), 1987.
23. Preguiça, N., Baquero, C., Almeida, P. S., Fonte, V., Gonçalves, R. *Dotted Version Vectors: Logical Clocks for Optimistic Replication*. arXiv, 2010.
24. Redis. *Active-Active geo-distributed Redis*. Redis Enterprise documentation, accessed 2026.
25. Riak. *Riak KV documentation*: Conflict Resolution; Causal Context; Data Types. Accessed 2026.
26. Scales, D. J., Nelson, M., Venkitachalam, G. *The Design of a Practical System for Fault-Tolerant Virtual Machines*. ACM SIGOPS Operating Systems Review 44(4), 2010.
27. Shapiro, M., Preguiça, N., Baquero, C., Zawirski, M. *Conflict-free Replicated Data Types*. SSS 2011, LNCS 6976, 2011 (cited as 2011a).
28. Shapiro, M., Preguiça, N., Baquero, C., Zawirski, M. *A Comprehensive Study of Convergent and Commutative Replicated Data Types*. INRIA Research Report RR-7506, 2011 (cited as 2011b).
29. Terrace, J., Freedman, M. J. *Object Storage on CRAQ: High-throughput Chain Replication for Read-mostly Workloads*. USENIX ATC, 2009.
30. van Renesse, R., Schneider, F. B. *Chain Replication for Supporting High Throughput and Availability*. USENIX OSDI, 2004.

27.21 Open verification items

- GitHub 2018 incident figures (43 s partition, 24 h 11 min degradation) confirmed only via search summaries; github.blog was unreachable from this session.
- Cassandra’s exponential-distribution phi model and the 3-hour default hint window should be checked against current (5.x) source and docs.
- Dynamo’s “99.94% of requests saw one version” figure and the ~10-entry vector-clock truncation threshold: check against the paper text.
- DynamoDB global tables’ LWW and strong-consistency mode wording (as of 2026).
- Exact titles/URLs for Akka, CouchDB, Redis and Riak documentation pages.

Chapter 28 — Distributed Transactions

> **Level:** 4 · **Track:** A (Traditional) · **Estimated reading time:** 6 h

On one database server a transaction is a solved problem: the write-ahead log, locks and MVCC of Chapter 11 make a transfer between two accounts all-or-nothing, isolated and durable. Partition the accounts across two servers (Chapter 14) and the guarantee disappears. Each server can still commit or abort *its own* half, but nothing forces the two halves to agree, and either server, or the network between them, can fail at any moment.

This chapter is about getting the guarantee back. It starts with the **atomic commitment problem** and the classic answer, two-phase commit, including exactly what must be logged and why it can block. It then follows the ideas that made distributed transactions practical at scale: running 2PC over replicated consensus groups, Percolator's timestamp oracle and primary lock, Spanner's TrueTime and commit wait, Calvin's deterministic ordering, and the commit protocols inside CockroachDB, YugabyteDB and TiDB. It ends with a decision procedure, because the cheapest distributed transaction is the one your data model avoids. Consensus itself is Chapter 26's subject, hybrid logical clocks are introduced in §24.12, and sagas and the transactional outbox are treated in depth in Chapter 32.

28.1 Learning objectives

After this chapter you should be able to:

- State the atomic commitment problem and explain why partial failure makes it harder than local commit.
- Walk through two-phase commit, naming every forced log write, and show where and why it blocks when the coordinator fails.
- Compute 2PC latency, the availability of a transaction spanning k partitions (A^k) and the expected number of in-doubt transactions per coordinator crash.
- Explain presumed abort, XA and PostgreSQL prepared transactions, and recover an in-doubt transaction by hand.
- Explain why three-phase commit is non-blocking only under synchrony assumptions and why it is rarely deployed.
- Describe Percolator's write and read paths, Spanner's commit wait and external-consistency argument, and Calvin's deterministic execution, and compute their latencies.

- Compare CockroachDB, YugabyteDB, TiDB and Spanner, and choose a transaction strategy for a given workload.

28.2 Prerequisites

- Transactions, ACID, isolation anomalies (especially write skew), MVCC and two-phase locking — §11.11–§11.15 (this chapter distributes those mechanisms).
- Write-ahead logging and `fsync` — §13.5 and §6.14 (every protocol here is defined by what it forces to the log).
- Partitioning and the cross-shard transaction preview — §14.6–§14.7 and §14.12.
- Partial failure, clock skew and hybrid logical clocks — §24.8, §24.9 and §24.12.
- Consensus and leases — §26.3, §26.7 and §26.11 (2PC is made highly available by running it over consensus groups).
- Idempotency keys — §22.15 (commit and abort messages are retried and must be harmless when duplicated).

28.3 Why distributed transactions are hard

28.3.1 The atomic commitment problem

DEFINITION

Definition — Atomic commitment. A protocol by which a set of **participants** (nodes that each executed part of a transaction) reach a single outcome, commit or abort, such that: (1) all participants that decide reach the same decision; (2) a participant cannot reverse a decision once made; (3) commit is decided only if every participant voted yes; (4) if there are no failures and every participant voted yes, the decision is commit; and (5) once failures are repaired and no new ones occur for long enough, every participant eventually decides [Bernstein et al. 1987].

First-principles explanation

Picture a wedding ceremony. The officiant asks each partner “Do you?” Each partner, before answering, must be *certain* they can follow through, because after saying “I do” they may not back out on their own. Only when both have said yes does the officiant pronounce the marriage, and from that moment it is binding on both. If the officiant faints after hearing two yeses but before pronouncing, the couple stands at the altar unable to leave and unable to celebrate. That picture, used by Gray, is two-phase commit, weakness included [Gray 1978].

The single-server version is easy because there is one log and one decision. One `fsync` of a commit record is the instant at which the whole transaction becomes durable. Across servers there is no shared instant. Each server can promise only things about its own log, and it learns about the others solely through messages that may be delayed, lost or never sent because the sender died.

Why does this exist?

Partitioning scales writes (Chapter 14), but business operations ignore partition boundaries: a transfer touches two accounts, an order touches stock and orders, a global index entry lives apart from its row (§14.11). Each needs all-or-nothing behavior across nodes.

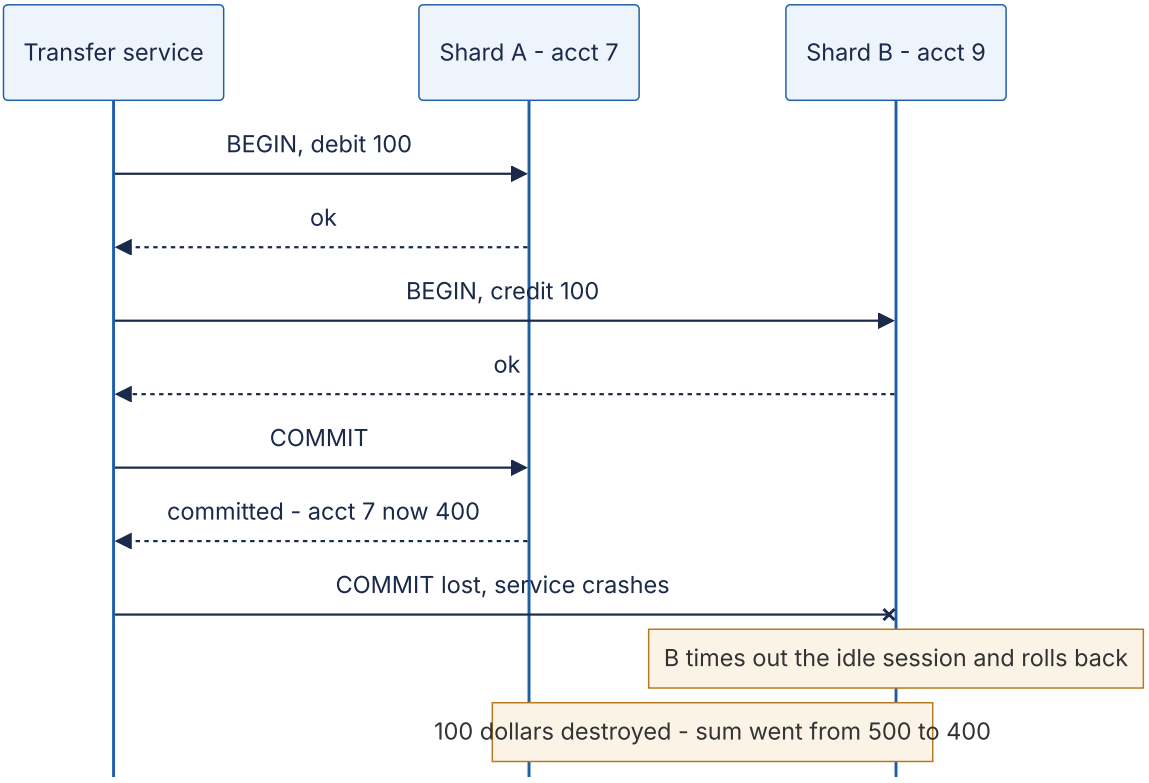
28.3.2 Partial failure: a money transfer across two shards

DEFINITION

Definition — Partial failure. A failure in which some components of a distributed system fail, or become unreachable, while others keep running, and the running components cannot reliably tell which (§24.8).

Take two PostgreSQL shards. Account 7 (balance $500) lives on shard A and account 9 (balance $0) on shard B. A service transfers $100 by running an ordinary local transaction on each shard, one after the other.

Figure 28.1 — A naive cross-shard transfer that loses money.



Reading the diagram. The transfer service talks to each shard over its own synchronous SQL connection; each shard has its own write-ahead log, so there are two independent storage boundaries. Shard A's commit is durable before the service even sends B's commit. The service crashes (the crossed arrow), B's session times out and B rolls back its half. Committing B first fails symmetrically, creating money. No ordering of two independent commits is safe, and scaling changes nothing: the flaw is in the protocol.

Three facts make the problem hard:

1. **No participant may commit until it knows every other participant can.** Shard B might fail a `CHECK` constraint, run out of disk or deadlock. A must not make its half permanent until that is ruled out.
2. **A participant that has promised cannot change its mind.** Once B has said "I can commit", it must keep that promise through crashes, so the promise itself must be durable.
3. **Silence is ambiguous.** A coordinator that hears nothing from B cannot distinguish "B crashed", "B is slow" and "the network dropped B's reply" (§24.7). Any protocol must be safe in all three cases.

KEY CONCEPT

Concept — Atomic commitment is a consensus problem with veto. Every participant must agree on one value (commit or abort), and any single participant can force abort. The FLP result (§24.13) says no deterministic protocol can guarantee that agreement terminates in an asynchronous system with even one crash. Every protocol in this chapter therefore gives up something: two-phase commit gives up termination while the coordinator is down; three-phase commit assumes bounded delays; consensus-based commit needs a majority of replicas to be up.

28.3.3 Isolation across partitions: snapshot isolation versus serializability

Atomicity is half the job. A distributed transaction must also be isolated (§11.13), which needs two things a single server gets for free:

1. **A consistent snapshot across partitions.** A reader must see either both halves of the transfer or neither. On one server a snapshot is a list of committed transaction IDs (§11.14). Across servers you need a common notion of "before" and "after": a central **timestamp oracle** (Percolator, TiDB), tightly bounded physical clocks (Spanner) or hybrid logical clocks with uncertainty handling (CockroachDB, YugabyteDB).
2. **Conflict detection across partitions.** Write-write conflicts are easy, because both writers touch the same key on the same partition. Read-write conflicts, the ones that cause **write skew**, are not: transaction T1 reads key *x* on partition 1 and writes

y on partition 2, while T2 reads *y* and writes *x*. Neither partition sees both edges of the dependency cycle.

That second point explains a pattern you will see throughout this chapter. **Snapshot isolation** (SI) needs only per-key write-write checks, so it distributes cheaply, and Percolator and TiDB offer it [Peng and Dabek 2010; Huang et al. 2020]. SI permits write skew: two transfers that each check "sum of a customer's two accounts ≥ 0" and each debit a *different* account can both commit and overdraw the customer, exactly the doctors-on-call anomaly of Figure 11.8 with the rows on different machines (§28.7.4 reproduces it). **Serializable** isolation needs either read locks held to commit (Spanner's two-phase locking for read-write transactions [Corbett et al. 2012]) or validation of reads at commit time (CockroachDB's timestamp ordering with read refresh [Taft et al. 2020]). Serializable snapshot isolation (SSI), which PostgreSQL implements on one node by tracking read-write dependencies [Cahill et al. 2008], is hard to distribute because the dependency graph spans partitions.

RECOMMENDATION

Recommendation — Know which isolation your distributed database gives by default. Spanner and CockroachDB default to serializable; TiDB defaults to snapshot isolation (exposed as `REPEATABLE READ`); YugabyteDB's YSQL defaults to snapshot isolation in recent versions (as of 2026). Under SI, protect cross-row invariants with `SELECT ... FOR UPDATE` or a materialized conflict row (§11.13).

COMMON MISTAKE

Common misconception — "Distributed transactions are just slow local transactions." The extra latency is the visible cost. The deeper costs are new failure modes (a transaction stuck between commit and abort), new availability arithmetic (a transaction needs *every* participant), and anomalies that local snapshot isolation never had to handle across machines.

28.4 Two-phase commit

28.4.1 The protocol

DEFINITION

Definition — Two-phase commit (2PC). An atomic commitment protocol in which one node, the **coordinator** (also called the transaction manager), first asks every participant (resource manager) to **prepare**, that is, to make its part of the transaction durable while keeping it invisible and to promise to commit on request; and then, only if every participant voted yes, durably records a **commit** decision and tells all participants to commit. Any no vote or timeout leads to abort [Gray 1978; Bernstein et al. 1987].

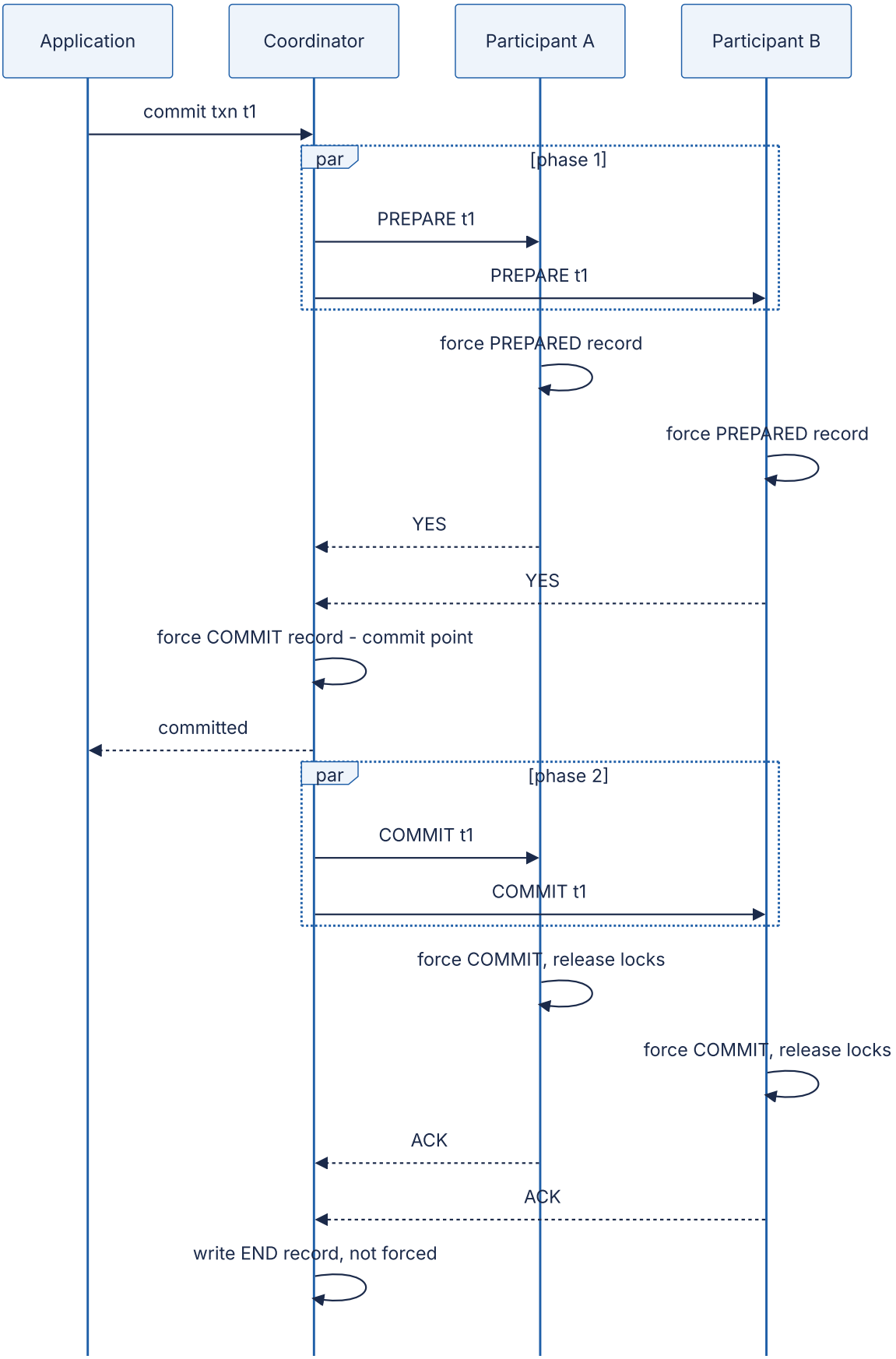
Why does this exist?

Figure 28.1 failed because each shard decided alone. 2PC separates the *ability* to commit from the *decision* to commit. Phase 1 collects durable promises; phase 2 distributes one decision, made in one place and recorded in one log. The coordinator's log record is the distributed equivalent of the single-server commit record.

How it works internally

1. **Execution.** The application runs the transaction's reads and writes on each participant as usual; participants take locks (or write MVCC intents) and keep undo and redo information.
2. **Prepare.** When the application says commit, the coordinator sends `PREPARE` to every participant.
3. **Vote.** Each participant checks it can commit (constraints satisfied, no deadlock victim, resources available). If yes, it **forces a prepared record** to its log, containing enough to redo or undo its writes and to re-acquire its locks after a crash, then replies `YES`. From this moment it is **in doubt**: it may not commit or abort on its own. If it cannot commit, it aborts locally and replies `NO`.
4. **Decision.** If every vote is `YES`, the coordinator **forces a commit record** to its log. This write is the **commit point** of the whole distributed transaction. Any `NO` or a timeout leads to abort.
5. **Completion.** The coordinator sends `COMMIT` (or `ABORT`) to each participant, retrying until it is acknowledged. Each participant logs the outcome, releases its locks and sends `ACK`.
6. **Forget.** When all acknowledgments are in, the coordinator writes a non-forced end record and may discard its memory of the transaction.

Figure 28.2 — Two-phase commit, happy path.



Reading the diagram. Three logs are involved: the coordinator's and each participant's, and every self-arrow is a write to one of them. Prepare messages go out in parallel, so

phase 1 costs one round trip plus the slowest participant's forced write, not the sum. The coordinator may answer the application as soon as its commit record is durable: the outcome is now decided, and phase 2 only propagates it. Phase 2 messages are retried until acknowledged, so participants must treat duplicate `COMMIT`s as no-ops. The failure point is the interval between a participant's `YES` and its receipt of the decision (Figure 28.4). Messages grow as $4k$ for k participants; latency stays two round trips.

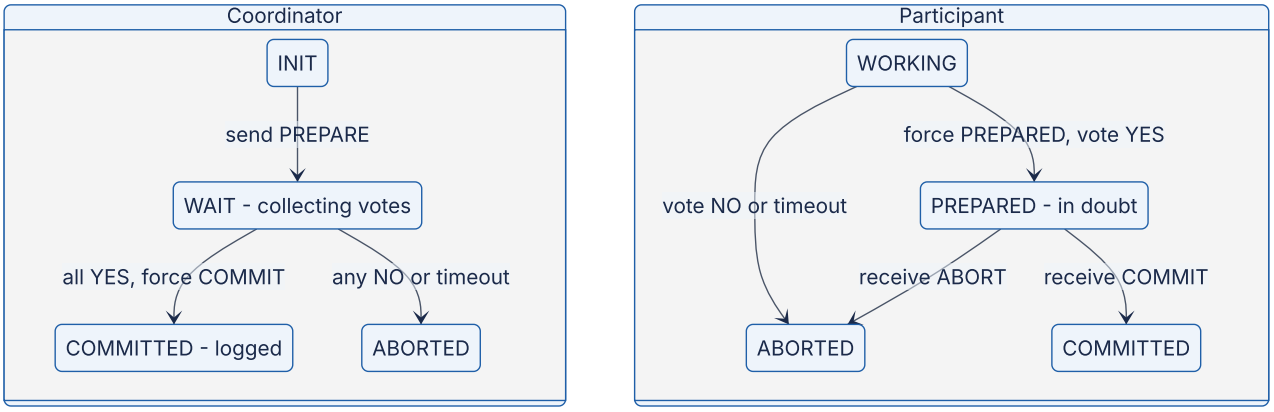
28.4.2 What must be logged, and when

2PC is defined as much by its log writes as by its messages. A **forced** write is one the node `fsync`s (§6.14) before sending the next message, because the message makes a promise the node must keep after a crash.

Table 28.1 — Log records in two-phase commit (basic protocol).

Who	Record	Forced?	When	Why it must be durable
Participant	PREPARED (redo/undo, lock list, coordinator ID)	Yes	Before sending `YES`	After a crash it must still be able to commit, and must keep others from seeing or changing the rows
Coordinator	COMMIT (transaction ID, participant list)	Yes	After all `YES`, before sending any `COMMIT`	The decision must survive a crash, or a recovered coordinator could abort a transaction some participant already committed
Participant	COMMIT	Yes	Before `ACK`	Once it acknowledges, the coordinator may forget, so the participant must not need to ask again
Coordinator	ABORT	Yes in basic 2PC; no under presumed abort	On any `NO` or timeout	See §28.4.5
Participant	ABORT	Yes in basic 2PC; no under presumed abort	On receiving `ABORT`	See §28.4.5
Coordinator	END	No	After all `ACK`s	Lets the coordinator garbage-collect; losing it causes only a harmless resend

Figure 28.3 — Coordinator and participant state machines.



Reading the diagram. Each box is one node's local state, recovered from its log after a crash. The participant has a free choice only in WORKING: it may abort for any reason, including the coordinator taking too long to send `PREPARE`. PREPARED is the dangerous state, because it has no outgoing transition the participant can take alone; both exits are triggered by a message from the coordinator. The coordinator, by contrast, never blocks: in WAIT it may time out and abort. This asymmetry is the root of 2PC's blocking behavior.

Recovery rules. After a restart, each node reads its log:

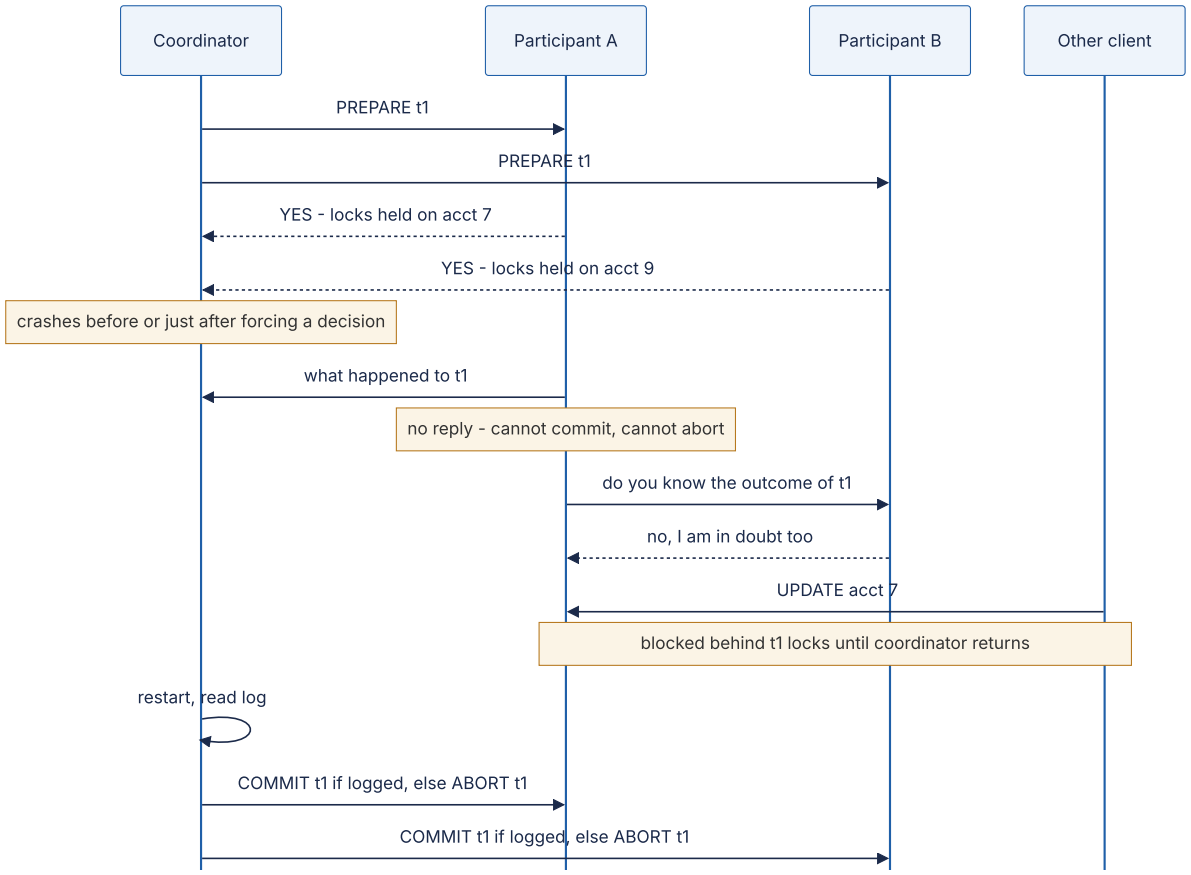
- **Participant, no PREPARED record:** abort locally; the coordinator cannot have decided commit without its vote.
- **Participant, PREPARED plus COMMIT or ABORT:** redo or undo accordingly.
- **Participant, PREPARED only:** in doubt. Re-acquire the locks listed in the record, then ask the coordinator for the outcome, repeatedly, until it answers.
- **Coordinator, COMMIT without END:** resend `COMMIT` to every listed participant until all acknowledge.
- **Coordinator, no COMMIT record:** the transaction aborted (it may have been mid-vote at the crash); answer `ABORT` to any inquiry.

28.4.3 Blocking and in-doubt transactions

DEFINITION

Definition — In-doubt (blocked) transaction. A transaction at a participant that has voted yes but has not learned the outcome. It must hold its locks and cannot decide until the coordinator, or another participant that knows the outcome, answers.

Figure 28.4 — Coordinator failure after the votes: participants block.



Reading the diagram. Both participants are in PREPARED and their locks are durable promises. A asks the coordinator (no answer), then tries the **cooperative termination protocol**: asking other participants. If any participant had not yet voted, or had already received a decision, A could decide. Here B is equally in doubt, and neither can tell whether the coordinator logged commit before dying; deciding either way could contradict a decision the coordinator already made. Unrelated client X now waits on account 7 for as long as the coordinator is down. The storage boundary that matters is the coordinator's log: only it holds the answer. Replicating that log (§28.4.8) is the fix.

How likely is this? A transaction is exposed while any participant is in doubt: from the first `YES` until every participant has the decision.

FORMULA

Exposure to in-doubt blocking

$$w \approx \mathrm{RTT} + t_{f,c} \qquad P_{\mathrm{doubt}} = 1 - e^{-\lambda w} \approx \lambda w \qquad N_{\mathrm{doubt}} = X w$$

Symbol	Meaning	Units
w	Window from the first YES vote reaching the coordinator's path until all participants hold the decision: half an RTT for votes, the coordinator's forced write, half an RTT for decisions	seconds
RTT	Network round-trip time between coordinator and participants	seconds
$t_{f,c}$	Time of the coordinator's forced log write	seconds
λ	Coordinator crash rate	crashes/second
P_{doubt}	Probability a given transaction is caught in doubt by a coordinator crash	dimensionless
X	Rate of distributed transactions through this coordinator	transactions/second
N_{doubt}	Expected number of transactions left in doubt by one crash (Little's Law, §4.11)	transactions

WORKED EXAMPLE

Worked example — How many transactions does a coordinator crash strand? A payments service coordinates $X = 2,000$ cross-shard transfers per second. RTT $= 0.5$ ms, coordinator forced write $t_{f,c} = 1$ ms, and the coordinator process crashes about once a month ($\lambda = 1/2,592,000$ s $= 3.86 \times 10^{-7}$ /s). Restarting it with its log takes 10 minutes when an operator must intervene.

1. Window: $w = 0.5 + 1 = 1.5$ ms $= 0.0015$ s.
2. Per-transaction probability: $P_{\text{doubt}} \approx 3.86 \times 10^{-7} \times 0.0015 = 5.8 \times 10^{-10}$, about one in 1.7 billion.
3. Stranded per crash: $N_{\text{doubt}} = 2,000 \times 0.0015 = 3$ transactions.
4. Each holds its row locks for the 10-minute outage. If one of those rows is a merchant's settlement account receiving 50 transfers/s, $50 \times 600 = 30,000$ transfers queue behind the lock or time out.

Interpretation. The probability per transaction is negligible; the *consequence* is not, because a few in-doubt transactions can freeze hot rows for the whole recovery time. **Engineering implication.** Minimize coordinator recovery time (automatic restart, a replicated coordinator log), keep hot rows out of distributed transactions, and alert on prepared transactions older than a few seconds.

28.4.4 Latency and availability

FORMULA

Two-phase commit latency and participant lock hold time

$$T_{2PC} \approx 2\,\mathrm{RTT} + t_{f,p} + t_{f,c} \qquad T_{\mathrm{hold}} \approx t_{f,p} + \mathrm{RTT} + t_{f,c}$$

Symbol	Meaning	Units
T_{2PC}	Commit latency until every participant has received the decision	seconds
RTT	Round trip between coordinator and the slowest participant	seconds
$t_{f,p}$	Slowest participant's forced write of its PREPARED record (writes happen in parallel)	seconds
$t_{f,c}$	Coordinator's forced write of the COMMIT record	seconds
T_{hold}	Extra time a participant holds locks during commit, from forcing PREPARED until the decision arrives	seconds

The commit phase adds two sequential round trips and two sequential forced writes. Replying right after the coordinator's commit record (Figure 28.2) hides the second round trip from the client, but not from participants' lock hold time.

WORKED EXAMPLE

Worked example — 2PC inside a datacenter and across regions. Forced writes take $t_f = 1$ ms on cloud block storage; a local commit therefore costs about 1 ms.

1. Same datacenter, RTT $= 0.5$ ms: $T_{2PC} = 2(0.5) + 1 + 1 = 3$ ms, three times a local commit. Lock hold: $1 + 0.5 + 1 = 2.5$ ms.
2. Participants in two regions, RTT $= 70$ ms: $T_{2PC} = 140 + 2 = 142$ ms; lock hold $= 72$ ms.
3. A hot row protected by row locks can commit at most $1/T_{\mathrm{hold}}$ transactions per second (§11.15.3): about 400/s in one datacenter, about 14/s across regions.

Interpretation. Inside a datacenter, 2PC's latency is tolerable; its lock hold time, not its latency, is what limits contended rows. Across regions both explode.
Engineering implication. Keep coordinators near participants, avoid multi-region 2PC on hot rows, and use group commit so that many transactions share each forced write.

The availability arithmetic is harsher still. A distributed transaction commits only if the coordinator and *every* participant are reachable for its duration.

FORMULA

Availability of a transaction spanning k partitions

$$A_{\text{txn}} = A_c \prod_{i=1}^{k} A_i \approx A^k \quad \text{(equal partitions, coordinator co-located with one of them)}$$

Symbol	Meaning	Units
A_{txn}	Probability that the transaction can commit at a random moment	dimensionless
A_c	Availability of the coordinator and its log	dimensionless
A_i	Availability of partition i	dimensionless
k	Number of partitions the transaction touches	count

WORKED EXAMPLE

Worked example — Ten shards at three nines versus ten consensus groups at five nines. Assume independent failures and a 30-day month (43,200 minutes).

1. One unreplicated shard at $A = 0.999$: unavailable $0.001 \times 43,200 = 43.2$ min/month.
2. A transaction over $k = 10$ such shards: $0.999^{10} = 0.990045$, unavailable $0.009955 \times 43,200 \approx 430$ min, more than 7 hours a month.
3. If each partition is a consensus group (§26.7) with $A = 0.99999$: $0.99999^{10} = 0.99990$, unavailable about 4.3 min/month.

Interpretation. Fan-out multiplies unavailability almost linearly ($1 - A^k \approx k(1 - A)$ for small $1 - A$). **Engineering implication.** Never run 2PC over unreplicated nodes for anything that must stay up; make each participant a replicated group, and keep k small by co-locating data that is written together (§14.7).

28.4.5 Presumed abort and other optimizations

Basic 2PC forces records and collects acknowledgments for aborts as well as commits. Mohan, Lindsay and Obermarck, building IBM's R* distributed database, observed that the coordinator can make one outcome the *default* for transactions it has no record of, and then skip logging for that outcome [Mohan et al. 1986].

DEFINITION

Definition — Presumed abort (PA). A 2PC variant in which a coordinator that has no information about a transaction answers inquiries with “abort”. Aborts therefore need no forced coordinator record, no forced participant record and no acknowledgments.

The argument is short. Under PA the coordinator forgets an aborted transaction immediately. If a participant later asks, the coordinator finds nothing and says “abort”, which is correct. Commits are unchanged: the coordinator must force its commit record, and keep it until every participant has acknowledged, because forgetting a committed transaction would turn it into a presumed abort.

Table 28.2 — Cost of commit protocol variants for one coordinator and n participants.

Variant	Commit: forced writes	Commit: messages	Abort: forced writes	Abort: messages	Notes
Basic 2PC	$1 + 2n$	$4n$	$1 + 2n$	$4n$	Every outcome logged and acknowledged
Presumed abort	$1 + 2n$	$4n$	n prepares only, nothing for the outcome	$3n$ (no ACK)	No information means abort; the usual choice in commercial systems
Presumed commit	$2 + n$	$3n$ (no ACK)	$2 + 2n$	$4n$	Coordinator forces a “collecting” record listing participants before prepare
Read-only participant	0 for that participant	2 for that participant	—	—	Votes READ in phase 1 and drops out of phase 2
One participant	1 local commit	2	—	—	Skip 2PC entirely (“one-phase commit”)

Three other optimizations matter in practice:

1. **Read-only participants.** A participant that only read data has nothing to commit; it releases its read locks and votes READ, and the coordinator omits it from phase 2. If every participant is read-only, nothing is logged at all.
2. **One-phase commit.** A transaction that touched one participant needs no vote: the coordinator just commits it locally. Transaction managers do this automatically,

which is why designing for single-partition transactions (§28.11) eliminates 2PC rather than merely shortening it.

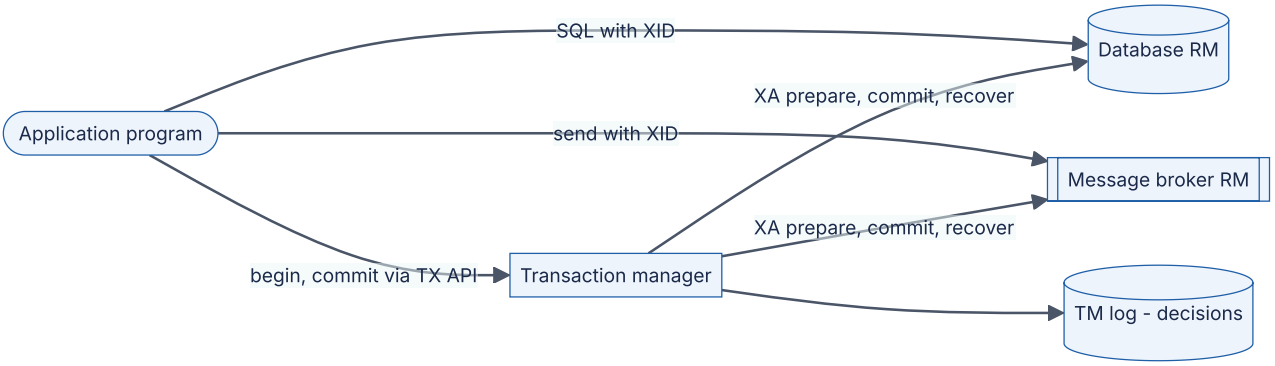
3. **Coordinator co-located with a participant** (sometimes called the last-agent optimization). The coordinator's log is the same as one participant's, saving one round trip and one forced write; Spanner, CockroachDB and TiDB all place the coordinator's state in one of the participating partitions.

28.4.6 XA and prepared transactions in practice

DEFINITION

Definition — XA. The X/Open Distributed Transaction Processing standard interface between a **transaction manager** (TM, the coordinator) and **resource managers** (RMs: databases, message brokers), with an **application program** (AP) demarcating transactions. Its main calls are `xa_start` and `xa_end` (associate work with a global transaction ID, the XID), `xa_prepare`, `xa_commit`, `xa_rollback` and `xa_recover` (list in-doubt XIDs after a crash) [X/Open 1991].

Figure 28.5 — The X/Open DTP model.



Reading the diagram. The application sends ordinary work directly to each resource manager, tagged with a global transaction ID; the transaction manager never sees the data. At commit, the TM runs 2PC over the XA interface and keeps its decisions in its own durable log, a separate storage boundary from every RM. All arrows are synchronous calls. Failure points: the TM log (lose it and in-doubt XIDs must be resolved by hand) and any RM that cannot answer `xa_recover`. Java's JTA is the best-known binding of this model.

PostgreSQL exposes the participant side directly. `PREPARE TRANSACTION 'gid'` performs phase 1, writing the transaction's state durably and dissociating it from the session; `COMMIT PREPARED` or `ROLLBACK PREPARED` performs phase 2, from any session; the view `pg_prepared_xacts` lists in-doubt transactions. The feature is disabled by default (`max_prepared_transactions = 0`) because orphaned prepared transactions are dangerous

[PostgreSQL Docs: PREPARE TRANSACTION]. MySQL offers `XA START`, `XA END`, `XA PREPARE`, `XA COMMIT` and `XA RECOVER` [MySQL Docs: XA Transactions].

The following session ran against two PostgreSQL 16 servers (shard A on port 5433, shard B on 5434, both with `max_prepared_transactions = 10`), with the human operator acting as coordinator:

```
        -- On shard A (acct 7 = 50000 cents) and shard B (acct 9 = 0): phase 1
        BEGIN;                                              -- shard A
        UPDATE accounts SET balance_cents = balance_cents - 10000 WHERE account_id = 7;
        PREPARE TRANSACTION 'xfer-42';
        BEGIN;                                              -- shard B
        UPDATE accounts SET balance_cents = balance_cents + 10000 WHERE account_id = 9;
        PREPARE TRANSACTION 'xfer-42';

        -- The "coordinator" disappears. On shard A:
        SELECT gid, prepared FROM pg_prepared_xacts;
        SET lock_timeout = '1s';
        UPDATE accounts SET balance_cents = balance_cents - 1 WHERE account_id = 7;

        -- Restart shard A (pg_ctl restart), then:
        SELECT gid FROM pg_prepared_xacts;
        SELECT * FROM accounts;

        -- The coordinator's log says COMMIT: phase 2 on both shards
        COMMIT PREPARED 'xfer-42';                          -- shard A, then shard B
```

Output (shard A unless noted):

```
   gid   |          prepared
---------+---------------------------
 xfer-42 | 2026-10-30 11:08:33.859+00

ERROR:  canceling statement due to lock timeout
CONTEXT:  while updating tuple (0,1) in relation "accounts"

   gid          <- after restart: the prepared transaction survived
---------
 xfer-42

 account_id | balance_cents      <- still invisible before COMMIT PREPARED
------------+---------------
          7 |         50000

 account_id | balance_cents      <- after COMMIT PREPARED on both shards
------------+---------------
          7 |         40000      (shard B: account 9 = 10000)
```

What to notice:

- The prepared transaction outlives its session *and a server restart*: that is the durable promise of phase 1, taken from the PREPARED record in the WAL.
- An unrelated update to account 7 blocks and times out: in-doubt transactions hold row locks indefinitely. PostgreSQL will never resolve them on its own, and they also hold back VACUUM (§11.14.2).

WARNING / TRADE-OFF

Warning — Heuristic decisions break atomicity. When an in-doubt transaction blocks production for too long, operators (or XA resource managers configured to do so) may force a **heuristic commit** or **heuristic abort** without the coordinator. If the guess contradicts the coordinator's logged decision, the transaction is no longer atomic and the damage must be repaired by hand. XA reports this as a *heuristic hazard* or *heuristic mixed* outcome. Treat a heuristic decision as an incident, and record which XIDs were forced.

28.4.7 A 2PC simulation with coordinator crash and recovery

The program below implements presumed-abort 2PC with durable logs that survive crashes and volatile lock tables that do not. Crash points can be injected into the coordinator, and the tests check that recovery from the logs alone always restores atomicity.

```
        """Two-phase commit with presumed abort: coordinator crash and recovery from its
log.

        Each node has a durable log (a list that survives crash()) and volatile state
        (locks, pending writes) that is lost on a crash and rebuilt from the log.
        Run it directly with python3 to execute the scenario tests.
        """
        import unittest


        class Crash(Exception):
            """Raised when a node is killed at an injected crash point."""


        class Participant:
            def __init__(self, name, balances):
                self.name, self.up = name, True
                self.data = dict(balances)        # committed state (durable)
                self.log = []                     # durable log: (record, txid, payload)
                self.locks, self.pending = {}, {}  # volatile: key -> txid, txid -> new
values

            def prepare(self, txid, deltas):
                if not self.up:
                    raise Crash(self.name)        # the coordinator sees a timeout
                new = {}
                for key, delta in deltas.items():
                    if self.locks.get(key, txid) != txid:
                        return "NO"               # row locked by an in-doubt or active
txn
                    new[key] = self.data[key] + delta
                    if new[key] < 0:
                        return "NO"               # invariant: no overdraft
                for key in new:
                    self.locks[key] = txid
                self.pending[txid] = new
                self.log.append(("PREPARED", txid, new))   # forced before voting YES
                return "YES"

            def finish(self, txid, decision):
                """Apply COMMIT or ABORT. Idempotent: duplicates are harmless."""
                if not self.up:
                    raise Crash(self.name)
                new = self.pending.pop(txid, None)
                if new is None:
                    return "ACK"                  # already finished, or never prepared
                if decision == "COMMIT":
                    self.data.update(new)
                    self.log.append(("COMMIT", txid, None))   # forced, then ACK
                else:
                    self.log.append(("ABORT", txid, None))    # not forced under pre-
sumed abort
                for key in new:
                    del self.locks[key]
                return "ACK"
```

```
            def crash(self):
                self.up, self.locks, self.pending = False, {}, {}

            def recover(self):
                """Redo from the log: a PREPARED txn with no outcome is in doubt and re-
locked."""
                self.up = True
                done = {txid for rec, txid, _ in self.log if rec in ("COMMIT", "ABORT")}
                for rec, txid, new in self.log:
                    if rec == "PREPARED" and txid not in done:
                        self.pending[txid] = new
                        for key in new:
                            self.locks[key] = txid

            def in_doubt(self):
                return sorted(self.pending)

            def resolve(self, coordinator):
                """Termination protocol: ask the coordinator about each in-doubt txn."""
                for txid in self.in_doubt():
                    if not coordinator.up:
                        return False              # blocked: cannot decide alone
                    self.finish(txid, coordinator.outcome(txid))
                return True


        class Coordinator:
            def __init__(self, participants):
                self.parts = {p.name: p for p in participants}
                self.log = []                     # durable
                self.up, self.crash_at = True, None

            def _point(self, where):
                if self.crash_at == where:
                    self.up = False
                    raise Crash("coordinator at " + where)

            def run(self, txid, plan):
                """plan: {participant name: {key: delta}}. Returns COMMIT or ABORT."""
                votes = {}
                for name, deltas in plan.items():          # phase 1: prepare
                    try:
                        votes[name] = self.parts[name].prepare(txid, deltas)
                    except Crash:
                        votes[name] = "NO"                 # timeout counts as NO
                self._point("after_votes")
                if all(v == "YES" for v in votes.values()):
                    self.log.append(("COMMIT", txid, sorted(plan)))   # forced: commit
point
                    self._point("after_decision")
                    self._send(txid, "COMMIT", sorted(plan))
                    return "COMMIT"
                # presumed abort: no forced log record, no ACKs needed
                yes = [n for n, v in votes.items() if v == "YES"]
                for name in yes:
                    try:
                        self.parts[name].finish(txid, "ABORT")
```

```
                    except Crash:
                        pass                                # it will ask later; answer
is ABORT
                return "ABORT"

            def _send(self, txid, decision, names):
                acked = 0
                for i, name in enumerate(names):
                    try:
                        self.parts[name].finish(txid, decision)
                        acked += 1
                    except Crash:
                        pass
                    if i == 0:
                        self._point("mid_phase2")
                if acked == len(names):
                    self.log.append(("END", txid, None))   # not forced; allows log
truncation

            def outcome(self, txid):
                """Answer an inquiry. No COMMIT record means ABORT (presumed abort)."""
                return "COMMIT" if any(r == "COMMIT" and t == txid for r, t, _ in
self.log) else "ABORT"

            def recover(self):
                """Re-drive phase 2 for every COMMIT without END; everything else is
aborted."""
                self.up, self.crash_at = True, None
                ended = {t for r, t, _ in self.log if r == "END"}
                for rec, txid, names in list(self.log):
                    if rec == "COMMIT" and txid not in ended:
                        self._send(txid, "COMMIT", names)


        def bank():
            a, b = Participant("A", {"acct7": 50_000}), Participant("B", {"acct9": 0})
            return a, b, Coordinator([a, b])


        TRANSFER = {"A": {"acct7": -10_000}, "B": {"acct9": +10_000}}


        class TwoPhaseCommitTests(unittest.TestCase):
            def total(self, a, b):
                return a.data["acct7"] + b.data["acct9"]

            def test_happy_path(self):
                a, b, c = bank()
                self.assertEqual(c.run("t1", TRANSFER), "COMMIT")
                self.assertEqual((a.data["acct7"], b.data["acct9"]), (40_000, 10_000))
                self.assertEqual([r for r, _, _ in c.log], ["COMMIT", "END"])
                self.assertEqual(a.locks, {})

            def test_no_vote_aborts_without_coordinator_log(self):
                a, b, c = bank()
                plan = {"A": {"acct7": -60_000}, "B": {"acct9": +60_000}}   # overdraft
                self.assertEqual(c.run("t1", plan), "ABORT")
```

```
                self.assertEqual(c.log, [])                # presumed abort: nothing
logged
                self.assertEqual((a.data["acct7"], b.data["acct9"]), (50_000, 0))
                self.assertEqual((a.locks, b.locks), ({}, {}))

            def test_crash_before_decision_blocks_then_aborts(self):
                a, b, c = bank()
                c.crash_at = "after_votes"
                with self.assertRaises(Crash):
                    c.run("t1", TRANSFER)
                self.assertEqual((a.in_doubt(), b.in_doubt()), (["t1"], ["t1"]))
                self.assertFalse(a.resolve(c))             # blocked while coordinator
is down
                self.assertEqual(a.prepare("t2", {"acct7": -1}), "NO")   # row is locked
                c.recover()
                self.assertTrue(a.resolve(c) and b.resolve(c))
                self.assertEqual((a.data["acct7"], b.data["acct9"]), (50_000, 0))
                self.assertEqual((a.locks, b.locks), ({}, {}))

            def test_crash_after_decision_commits_on_recovery(self):
                a, b, c = bank()
                c.crash_at = "after_decision"
                with self.assertRaises(Crash):
                    c.run("t1", TRANSFER)
                self.assertEqual(self.total(a, b), 50_000)
                self.assertEqual(a.data["acct7"], 50_000)  # nothing applied yet
                c.recover()                                # finds COMMIT without END
                self.assertEqual((a.data["acct7"], b.data["acct9"]), (40_000, 10_000))
                self.assertEqual(c.log[-1][0], "END")

            def test_crash_mid_phase2_is_completed(self):
                a, b, c = bank()
                c.crash_at = "mid_phase2"
                with self.assertRaises(Crash):
                    c.run("t1", TRANSFER)
                self.assertEqual((a.data["acct7"], b.in_doubt()), (40_000, ["t1"]))
                self.assertFalse(b.resolve(c))             # still down
                c.recover()
                self.assertEqual(b.data["acct9"], 10_000)
                self.assertEqual(self.total(a, b), 50_000)  # money conserved

            def test_participant_crash_after_prepare_recovers_from_its_log(self):
                a, b, c = bank()
                c.crash_at = "after_decision"
                with self.assertRaises(Crash):
                    c.run("t1", TRANSFER)
                b.crash()                                  # loses locks and pending
writes
                b.recover()                                # rebuilds them from PREPARED
                self.assertEqual((b.in_doubt(), b.locks), (["t1"], {"acct9": "t1"}))
                c.recover()
                self.assertEqual(b.data["acct9"], 10_000)

            def test_unreachable_participant_means_abort(self):
                a, b, c = bank()
                b.crash()
                self.assertEqual(c.run("t1", TRANSFER), "ABORT")
```

```
                self.assertEqual((a.data["acct7"], a.locks), (50_000, {}))

            def test_duplicate_commit_is_idempotent(self):
                a, b, c = bank()
                c.run("t1", TRANSFER)
                a.finish("t1", "COMMIT")
                c.recover()
                self.assertEqual(a.data["acct7"], 40_000)


        if __name__ == "__main__":
            unittest.main(verbosity=1)
```

Output (Python 3.13; the timing varies):

```
........
----------------------------------------------------------------------
Ran 8 tests in 0.000s

OK
```

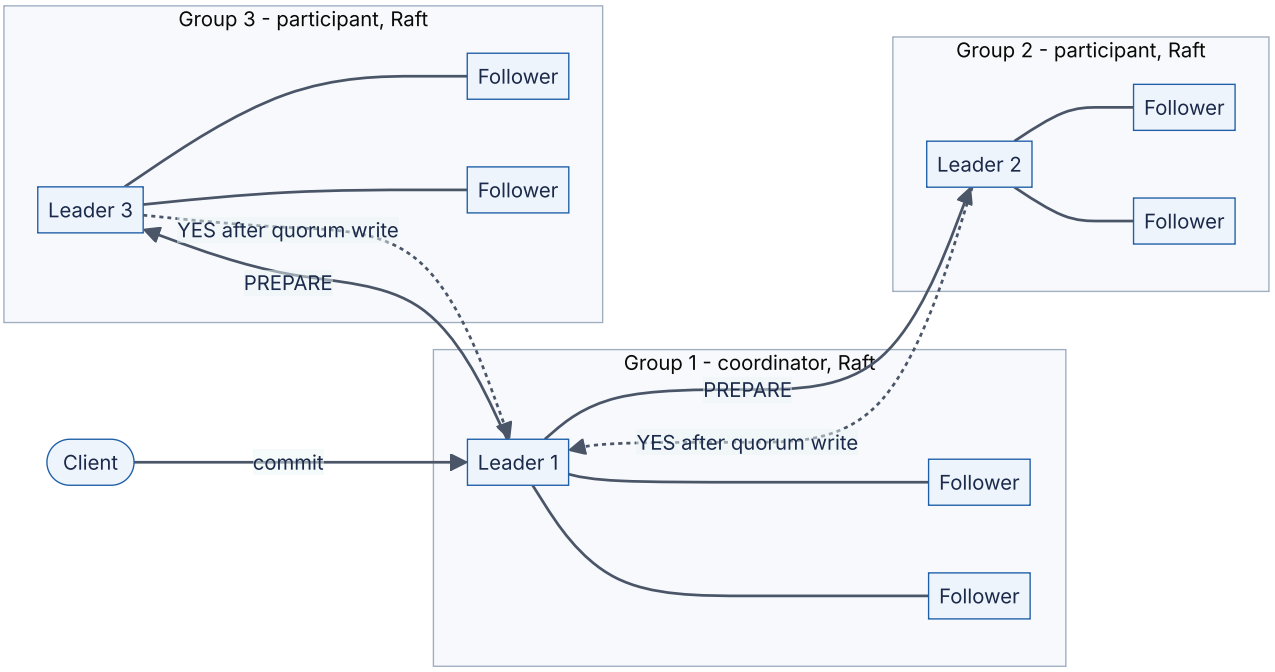
What to notice:

- The only thing that survives a coordinator crash is `self.log`, and recovery reads nothing else. A `COMMIT` without `END` is re-driven; any other transaction is aborted by presumption, which is why `test_no_vote_aborts_without_coordinator_log` can assert an empty log.
- `test_crash_before_decision_blocks_then_aborts` reproduces Figure 28.4: while the coordinator is down, `resolve` returns `False` and a competing transaction on account 7 is refused because the in-doubt transaction still holds the lock.
- `test_crash_mid_phase2_is_completed` shows a moment where A has committed and B has not. This is not an atomicity violation: the outcome was fixed at the commit record and recovery converges B to it.
- `outcome` is valid only for transactions the coordinator is no longer running; a real coordinator answers “ask later” for active ones.
- `finish` is idempotent because phase 2 messages are retried (§22.15).

28.4.8 Two-phase commit over consensus groups

Every blocking scenario above has the same cause: the decision lives in one node’s log. Replace that log with a **replicated log** (Raft or Multi-Paxos, §26.6–§26.7) and the coordinator’s crash is no longer a long outage, because a new leader of the coordinator’s group takes over within an election timeout, reads the decision from the replicated log and finishes phase 2. Do the same for each participant, and a participant’s prepared record survives the loss of any minority of its replicas.

Figure 28.6 — Two-phase commit across consensus groups.



Reading the diagram. Each subgraph is a consensus group of three replicas, typically in three availability zones; “forcing a log record” now means replicating it to a quorum (two of three) of the group. The coordinator is the leader of one group, often a group that is also a participant. Solid arrows are synchronous requests; the dotted votes are replies that arrive only after the participant’s prepared record is replicated. Failure points move from nodes to groups: losing one replica of any group is invisible to the transaction; losing a majority of one group blocks the transactions that touch it, as for any consensus system. Each transaction pays its slowest group’s quorum write, twice.

Gray and Lamport generalized the idea as **Paxos Commit**, which runs one Paxos instance per participant’s vote and tolerates F failures of the acceptors with $2F + 1$ acceptors; with $F = 0$ it reduces to 2PC [Gray and Lamport 2006]. Most production systems use the simpler structure of Figure 28.6, and so does nearly every system in the rest of this chapter.

INDUSTRY EXAMPLE

Apache Kafka — 2PC with a replicated coordinator log. Kafka transactions let a producer write atomically to several partitions and commit consumer offsets. A *transaction coordinator* records each transaction’s state in the internal, replicated `__transaction_state` topic: it writes a prepare-commit entry (the commit point), then writes commit markers into every partition involved, then a completion entry. If the coordinator broker fails, a new leader for that `__transaction_state` partition reads the log and finishes the markers. Consumers in `read_committed` mode skip data whose markers are not yet written. See §31.11. *Evidence: Primary* — [Kafka KIP-98 2017]

28.4.9 Advantages, disadvantages and failure scenarios

Advantages. 2PC gives true atomicity and, combined with locking, serializability across independent resource managers. It is standardized (XA), supported by most relational databases and several brokers, and its logic is simple enough to verify.

Disadvantages. It blocks when the coordinator fails after votes; it holds locks across at least one round trip plus two forced writes; its availability is the product of its participants'; and it couples otherwise independent services in time, which is why microservice architectures avoid it (§28.6, Chapter 34).

Table 28.3 — Two-phase commit failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Rows locked for minutes	Coordinator down while participants are in doubt	Age of `pg_prepared_xacts` / `XA RECOVER` entries	Restart coordinator from its log; resolve by GID	Replicated coordinator log; fast automatic restart
Orphaned prepared transaction	Coordinator's log lost or GID never recorded	Prepared transactions with unknown GID	Manual heuristic decision, then audit	Log GID and participant list before phase 2; never lose the TM log
Inconsistent outcome	Heuristic commit/abort contradicting the coordinator	Heuristic-mixed report; reconciliation	Manual compensation	Forbid heuristics without the coordinator's log
Transaction aborts under load	Participant times out waiting for `PREPARE` and aborts unilaterally	Abort rate and reason codes	Retry the transaction	Tune timeouts; avoid slow work between execution and commit
Duplicate commit applied twice	Retried phase 2 message not deduplicated	Balance reconciliation	Repair data	Idempotent `finish` keyed by transaction ID
VACUUM or purge stalls	Long-lived prepared transaction pins old versions	Table bloat, `age(datfrozenxid)`	Resolve the prepared transaction	Alert on prepared transactions older than seconds

COMMON MISTAKE

Common misconception — "If the coordinator dies, participants can just time out and abort." Before voting yes, they can. After voting yes, they cannot: the coordinator may have logged commit and told another participant, which has already made its half visible. A timeout-driven abort at that point violates atomicity.

COMMON MISTAKE

Common misconception — "2PC guarantees all participants commit at the same instant." It guarantees they reach the same *decision*. Participants apply it at different times, so a reader can see A's half before B's unless the system also provides consistent snapshots, which is what Percolator's timestamps and Spanner's commit wait add.

28.5 Three-phase commit

28.5.1 How 3PC removes blocking

DEFINITION

Definition — Three-phase commit (3PC). A commit protocol that inserts a **pre-commit** phase between voting and committing, so that no participant can commit until every operational participant knows that all voted yes. Under a synchronous system model with reliable failure detection and no network partitions, it lets the surviving participants always reach a decision without the failed coordinator [Skeen 1981].

Why does this exist?

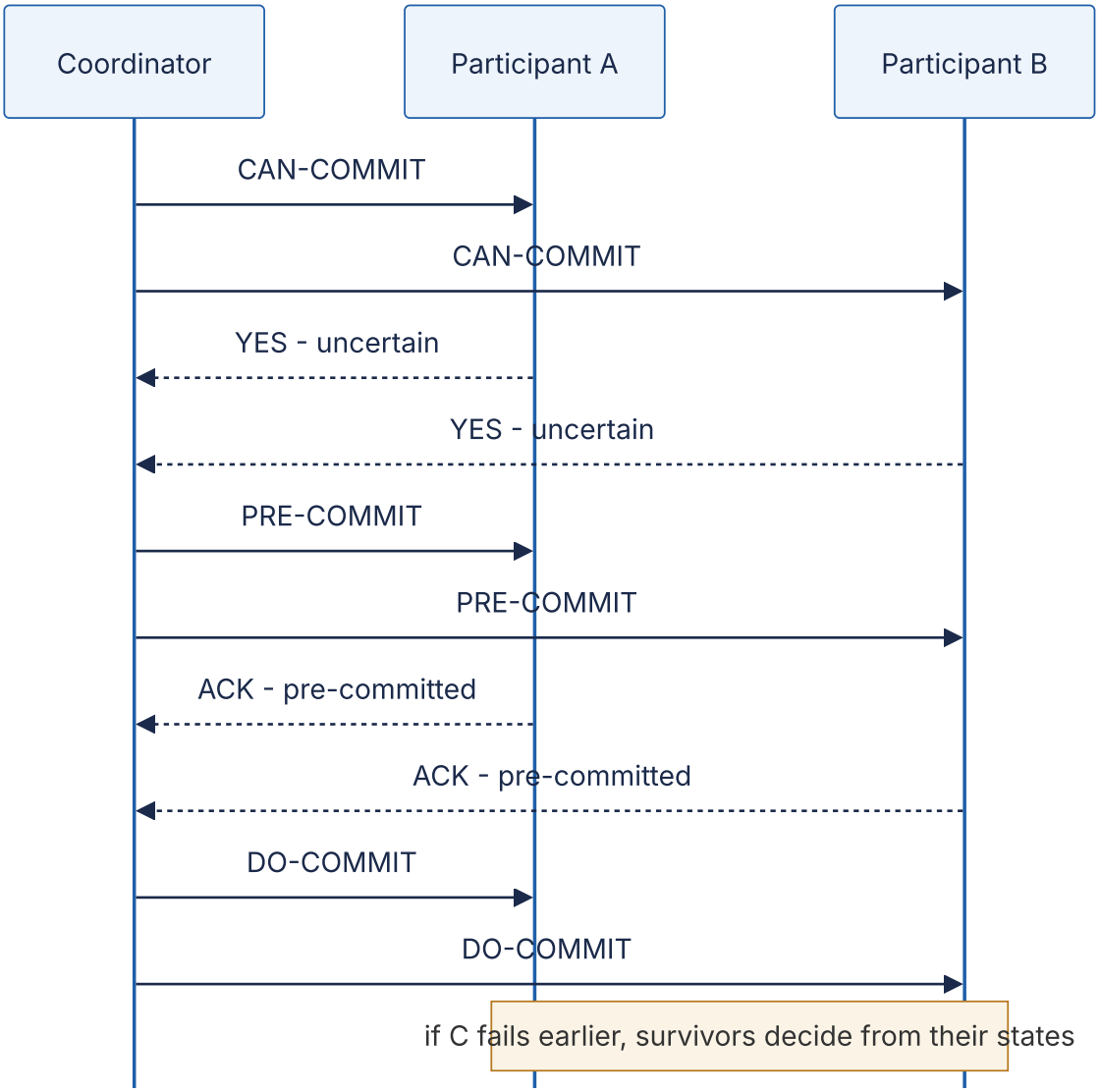
2PC blocks because a prepared participant cannot tell whether some other node has already committed. Skeen proved that a commit protocol is non-blocking (for crash failures, in a synchronous system) only if no participant can be in a state from which *both* commit and abort are reachable in one step, and no non-committable state is adjacent to commit [Skeen 1981]. 2PC's PREPARED state violates the first rule. 3PC adds a buffer state so that "someone may have committed" implies "everyone already knows the vote was unanimous".

How it works internally

1. **CanCommit.** The coordinator asks each participant whether it can commit; each replies yes (and becomes *uncertain*, as in 2PC's PREPARED) or no.
2. **PreCommit.** If all said yes, the coordinator sends `PRE-COMMIT`; each participant logs it, enters the *pre-committed* state and acknowledges. Pre-committed means "everyone voted yes; I will commit unless I fail".
3. **DoCommit.** After all acknowledgments, the coordinator sends `COMMIT`.
4. **Termination when the coordinator fails.** The survivors elect a new coordinator, which collects their states. If any has committed, or any is pre-committed, it drives everyone to commit (sending `PRE-COMMIT` first to those still uncertain). If any has

aborted, or *all* are merely uncertain, it aborts. Aborting is safe in the last case because nobody can have committed without first being pre-committed.

Figure 28.7 — Three-phase commit.



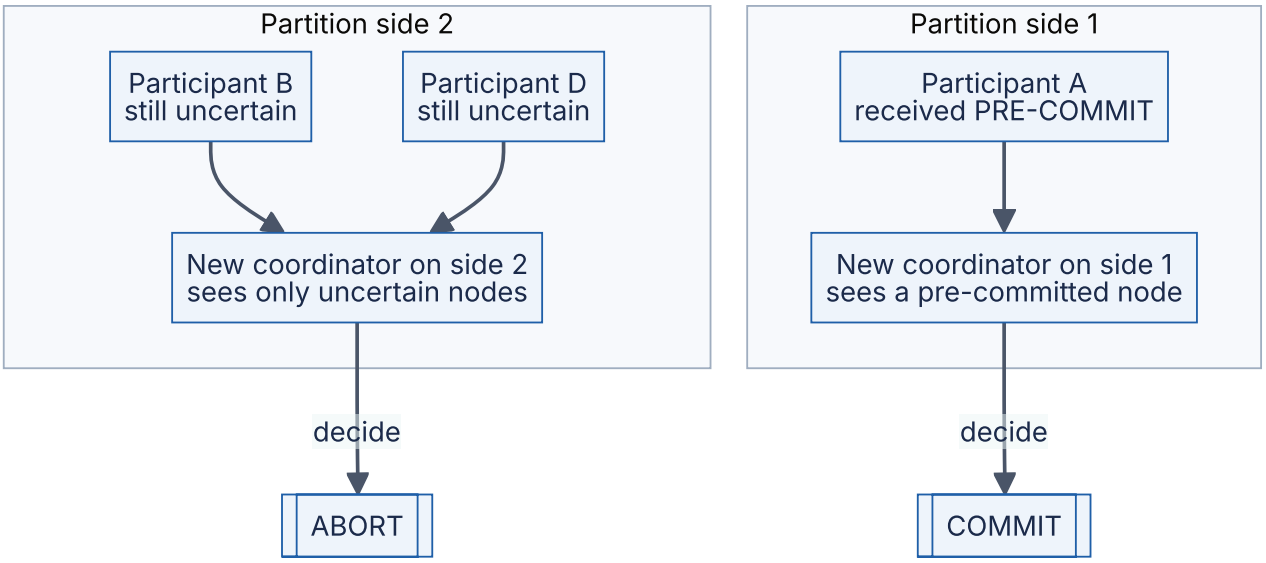
Reading the diagram. Each phase is a synchronous parallel fan-out, so 3PC costs three round trips instead of two; in a datacenter with RTT $= 0.5$ ms and 1 ms forced writes, about $3(0.5) + 2(1) = 3.5$ ms before the decision is known everywhere, versus 3 ms for 2PC. The extra round trip buys the invariant shown in the note: whenever any node may have committed, every live node is at least pre-committed, so the survivors can always decide among themselves. Failure points are the timeouts used to declare the coordinator dead, which must be accurate for the protocol to be safe.

28.5.2 Why 3PC fails under network partitions

The termination rule “all uncertain means abort” assumes that a node which cannot be reached has *crashed*. In an asynchronous network it may simply be on the other side of a

partition (§24.7).

Figure 28.8 — A partition makes 3PC decide both ways.



Reading the diagram. The original coordinator sent `PRE-COMMIT` to A and then lost connectivity; the network splits A from B and D. Each side runs the termination protocol with the nodes it can see. Side 1 contains a pre-committed node, so it commits; side 2 sees only uncertain nodes, so it aborts. Both decisions are durable, and the transaction is no longer atomic. The failure point is the timeout-based failure detector, which cannot distinguish crashed from partitioned. No amount of scaling fixes it; only a majority-based decision does.

3PC is rarely deployed for three reasons: real networks partition and timeouts misfire under GC pauses and overload (§23.9); it adds a round trip and a forced write to every commit; and running 2PC over consensus groups (§28.4.8) tolerates crashes *and* partitions, staying safe even when timeouts are wrong.

Table 28.4 — Commit protocols compared.

Property	2PC	3PC	2PC over consensus groups
Message rounds to decide	2	3	2 (each a quorum write)
Blocks if coordinator crashes	Yes, until it recovers	No (synchronous model)	Only until a new leader is elected
Safe under network partition	Yes (blocks, never inconsistent)	No	Yes (minority side blocks)
Failure assumption	Crash-recovery	Crash-stop, bounded delays	Crash-recovery, majority up
Used in practice	XA, databases, brokers	Rarely	Spanner, CockroachDB, TiDB, YugabyteDB, Kafka

COMMON MISTAKE

Common misconception — "3PC is 2PC without blocking." It removes blocking only by assuming perfect failure detection. In an asynchronous network it trades 2PC's *blocking* for the far worse possibility of *inconsistency*.

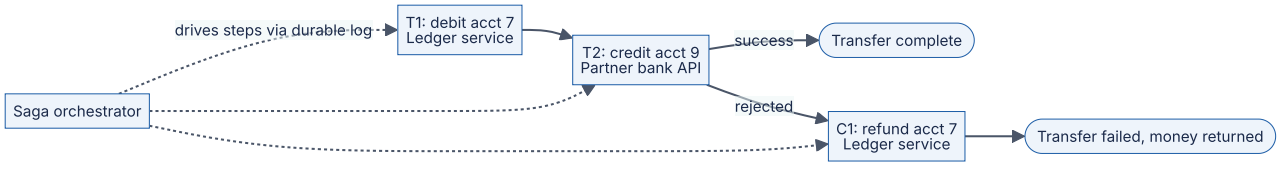
28.6 Sagas: a preview

DEFINITION

Definition — Saga. A long-lived transaction written as a sequence of local transactions $T_1, T_2, \ldots, T_n$, each committed independently, where each T_i has a **compensating transaction** C_i that semantically undoes it. Either $T_1 \ldots T_n$ all complete, or $T_1 \ldots T_j, C_j \ldots C_1$ run [Garcia-Molina and Salem 1987].

A saga replaces atomicity with **semantic atomicity**: the system ends in a state equivalent to "all happened" or "none happened" as the *business* defines it, not byte for byte. Refunding a charge does not erase the charge; it adds a refund. Because each step commits locally, a saga holds no locks across services and never blocks on a coordinator, which is why it suits long-running business processes and microservices.

Figure 28.9 — A transfer saga with compensation.



Reading the diagram. Each box is a local transaction in a different service with its own database; between steps, other transactions can see the intermediate state (account 7 debited, account 9 not yet credited). The orchestrator persists its progress and drives steps asynchronously (dotted arrows), retrying each step until it succeeds, so steps and compensations must be idempotent (§22.15). The failure point is the gap between steps: a saga provides no isolation, so another transaction might read the debited balance and refuse a payment it should have allowed.

The price is isolation, plus a compensation for every step, even steps that cannot truly be undone (an email sent). Chapter 32 covers sagas in depth (§32.7–§32.8) and the **transactional outbox** (§32.5), which makes "commit locally *and* publish a message" atomic without 2PC.

28.7 Percolator-style transactions

28.7.1 The problem Percolator solved

DEFINITION

Definition — Percolator transaction. A snapshot-isolated, multi-row, multi-table transaction built entirely by a client library on top of a store that offers only single-row atomic updates and multiple timestamped versions per cell. Ordering comes from a central **timestamp oracle**; atomicity comes from making one row's lock, the **primary lock**, the commit point for the whole transaction [Peng and Dabek 2010].

First-principles explanation

Think of a team editing many pages of a shared binder. Before editing, each writer clips a sticky note to every page they will change, and every note says "see page 1". Page 1's note is special: whoever removes it and stamps page 1 "done at 14:02" has committed the whole edit, even if they collapse before unclipping the other notes. A reader who finds a stale note on page 37 just turns to page 1 to learn whether that edit happened.

Why does this exist?

Google's web index was rebuilt in large MapReduce batches, so a newly crawled page could take days to appear in search. Updating the index *incrementally* meant many concurrent workers changing billions of Bigtable rows (§14.6), each document touching many rows, with no stale or half-applied results. Bigtable offered only single-row transactions. Percolator added cross-row snapshot-isolated transactions and notifications on top, and Google reported that moving indexing to it cut the average age of documents in search results by 50% [Peng and Dabek 2010].

28.7.2 How it works internally

Every logical column is stored as several Bigtable columns, each holding timestamped versions.

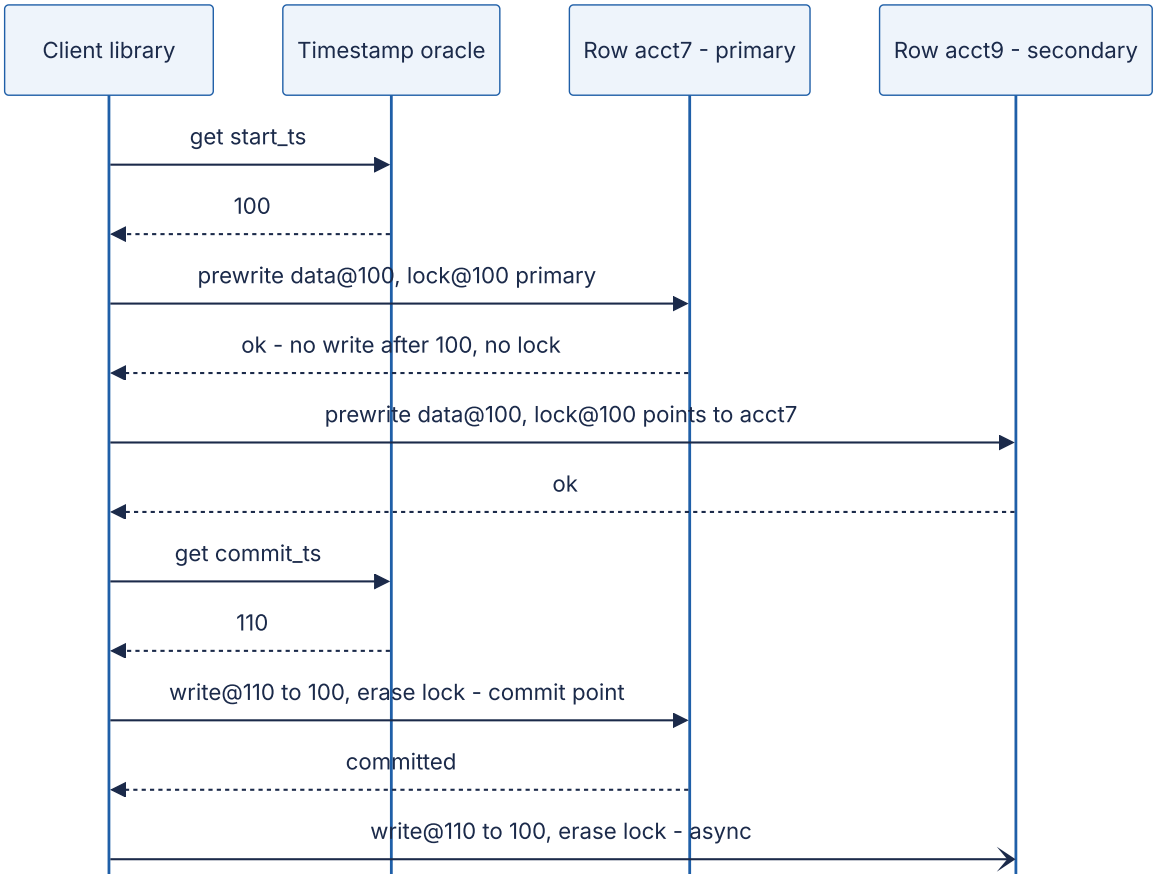
Table 28.5 — Percolator's columns for one logical cell.

Column	Version timestamp	Contents	Meaning
`data`	transaction start timestamp	The value	Written during prewrite, invisible until a `write` record points to it
`lock`	transaction start timestamp	Location of the primary lock (or "primary")	An uncommitted transaction holds this cell
`write`	transaction commit timestamp	Start timestamp of the committed `data` version	Makes `data@start_ts` visible to snapshots at or after `commit_ts`

Write path.

1. **Start.** Get `start_ts` from the timestamp oracle. Buffer writes at the client.
2. **Prewrite (phase 1).** Pick one written cell as the **primary**; the rest are secondaries. For each cell, primary first, run a Bigtable single-row transaction that aborts if there is a `write` record with timestamp $\geq$ `start_ts` (a write-write conflict with a transaction that committed after our snapshot) or *any* `lock` (a concurrent writer); otherwise it writes `data@start_ts` and `lock@start_ts` pointing at the primary.
3. **Commit (phase 2).** Get `commit_ts` from the oracle. In one single-row transaction on the primary, check that our lock is still there, write `write@commit_ts` $\rightarrow$ `start_ts` and erase the lock. **This is the commit point.**
4. **Secondaries.** Write `write@commit_ts` and erase the lock on each secondary. This step can happen lazily, after replying to the client, because the outcome is already fixed.

Figure 28.10 — A Percolator write of two cells.



Reading the diagram. All coordination lives in the client library; there is no coordinator process and no coordinator log, because the primary row's lock *is* the coordinator's state, stored durably in replicated Bigtable. The oracle is a single logical service on every transaction's path, twice. Phase 1 messages to different rows can run in parallel after the primary's. The final secondary update is asynchronous (open arrowhead). Failure points: the client can die anywhere; after the primary's `write` record exists the transaction is committed and readers will roll the secondaries forward, before it they will roll everything back. Scaling: the oracle batches requests, and Peng and Dabek report a single oracle machine serving about two million timestamps per second.

Read path and lazy cleanup. A reader with snapshot `start_ts` checks for a `lock` with timestamp $\leq$ `start_ts` on the cell. If one exists, a writer that may commit *before* our snapshot is in flight, so the reader waits. If the lock looks abandoned (Percolator checks writer liveness through Chubby, §26.14, and lock age), the reader cleans up by looking at the primary: if the primary's lock is still there, it erases it, rolling the transaction back; if the primary has a `write` record for that `start_ts`, it rolls this secondary forward by writing the same `write` record. Otherwise the reader returns the newest `data` version referenced by a `write` record with timestamp $\leq$ `start_ts`.

KEY CONCEPT

Concept — Why a single-row atomic operation is enough. 2PC needs a coordinator log so that one durable record decides the outcome. Percolator makes that record the primary cell's `write` entry, written by a single-row transaction in a replicated store. Every participant (every secondary lock) names the primary, so anyone can find the decision. There is no blocking on a coordinator: the decision is always readable, and an abandoned transaction can be resolved by whichever reader trips over it.

28.7.3 TiKV and TiDB

TiDB is a MySQL-compatible distributed SQL database. Its storage layer, TiKV, implements Percolator's protocol over **Regions**, contiguous key ranges each replicated by Raft, and its **Placement Driver** (PD) acts as the timestamp oracle, handing out hybrid timestamps (physical milliseconds plus a logical counter) in batches [Huang et al. 2020]. Since v3.0.8, new clusters default to **pessimistic transactions**, which acquire Percolator-style locks during statement execution, as MySQL users expect, rather than discovering conflicts only at commit [TiDB Docs: Transactions]. Two later optimizations shorten the protocol: **async commit** returns to the client once all prewrites succeed, computing the commit timestamp from the participants' prewrite responses instead of a second oracle call; and **one-phase commit** writes a transaction confined to one Region in a single step.

FORMULA

Percolator commit latency

$$T_{\text{perc}} \approx 2\,\text{RTT}_{\text{TSO}} + 2\,(\text{RTT} + t_w) \qquad T_{\text{async}} \approx \text{RTT}_{\text{TSO}} + \text{RTT} + t_w$$

Symbol	Meaning	Units
T_{perc}	Timestamp and commit overhead of a classic Percolator transaction (start and commit timestamps, prewrite, primary commit)	ms
T_{async}	The same with async commit (no commit timestamp fetch, no wait for the primary commit)	ms
RTT_{TSO}	Round trip to the timestamp oracle	ms
RTT	Round trip from the client to the storage node	ms
t_w	Durable, replicated write on the storage node (Raft quorum plus `fsync`)	ms

WORKED EXAMPLE

Worked example — TiDB in one region. Take $\text{RTT}_{\text{TSO}} = 0.5$ ms, RTT = 0.5 ms and $t_w = 2$ ms for a Raft write across three availability zones.

1. Classic: $2(0.5) + 2(0.5 + 2) = 1 + 5 = 6$ ms.
2. Async commit: $0.5 + 0.5 + 2 = 3$ ms, half the overhead.
3. If the PD leader is in another region with $\text{RTT}_{\text{TSO}} = 60$ ms, classic costs $120 + 5 = 125$ ms.

Interpretation. Percolator's latency is dominated by replicated writes inside a region, and by the oracle as soon as it is far away. **Engineering implication.** Keep the timestamp oracle in the region where transactions run, and treat a single global oracle as the scaling and geo-distribution limit of this design; Spanner (§28.8) removes it with clocks.

28.7.4 A Percolator model you can run

```
        """A Percolator-style transaction on a multi-version key-value store.

        Each row has three columns, each a map timestamp -> value (as in Bigtable):
          data[ts]  = value written by the txn that started at ts
          lock[ts]  = name of the primary row, while the txn that started at ts is in
flight
          write[cts] = start ts of the txn whose data became visible at commit ts cts
        Assumes the holder of any lock a reader finds is dead (Percolator checks live-
ness first).
        """
        import itertools


        class Store:
            def __init__(self):
                self.rows = {}

            def row(self, key):
                return self.rows.setdefault(key, {"data": {}, "lock": {}, "write": {}})


        class Oracle:                                   # the timestamp oracle
            def __init__(self):
                self._next = itertools.count(1)

            def ts(self):
                return next(self._next)


        class Txn:
            def __init__(self, store, oracle):
                self.s, self.o = store, oracle
                self.start = oracle.ts()                # snapshot timestamp
                self.writes = {}

            def get(self, key):
                r = self.s.row(key)
                for lts in [t for t in r["lock"] if t <= self.start]:
                    self._cleanup(key, lts)             # stale lock below our snap-
shot
                visible = [c for c in r["write"] if c <= self.start]
                return r["data"][r["write"][max(visible)]] if visible else None

            def set(self, key, value):
                self.writes[key] = value

            def _prewrite(self, key, primary):
                r = self.s.row(key)
                if any(c >= self.start for c in r["write"]) or r["lock"]:
                    return False                        # write-write conflict or
locked
                r["data"][self.start] = self.writes[key]
                r["lock"][self.start] = primary
                return True
```

```
            def commit(self, crash=None):
                keys = sorted(self.writes)
                primary, secondaries = keys[0], keys[1:]
                for k in keys:                              # phase 1: prewrite, primary
first
                    if not self._prewrite(k, primary):
                        self._rollback(keys)
                        return False
                if crash == "before_primary":
                    return None                             # client dies holding all
locks
                commit_ts = self.o.ts()
                p = self.s.row(primary)                     # phase 2: single-row txn on
primary
                if self.start not in p["lock"]:
                    return False                            # someone rolled us back
                p["write"][commit_ts] = self.start
                del p["lock"][self.start]                   # <- the commit point
                if crash == "after_primary":
                    return True                             # committed; secondaries still
locked
                for k in secondaries:
                    r = self.s.row(k)
                    r["write"][commit_ts] = self.start
                    r["lock"].pop(self.start, None)
                return True

            def _rollback(self, keys):
                for k in keys:
                    r = self.s.row(k)
                    if r["lock"].get(self.start) is not None:
                        del r["lock"][self.start]
                        del r["data"][self.start]

            def _cleanup(self, key, lts):
                """Roll a dead transaction forward or back, decided by its primary
row."""
                r, primary = self.s.row(key), self.s.row(key)["lock"][lts]
                p = self.s.row(primary)
                if lts in p["lock"]:                        # primary never committed:
roll back
                    del p["lock"][lts]
                    p["data"].pop(lts, None)
                committed = [c for c, st in p["write"].items() if st == lts]
                if committed and key != primary:            # roll forward this secondary
                    r["write"][committed[0]] = lts
                elif not committed:
                    r["data"].pop(lts, None)
                r["lock"].pop(lts, None)


        def setup():
            s, o = Store(), Oracle()
            t = Txn(s, o)
            t.set("acct7", 500)
            t.set("acct9", 0)
```

```
            assert t.commit()
            return s, o


        def transfer(s, o, amount, crash=None):
            t = Txn(s, o)
            t.set("acct7", t.get("acct7") - amount)
            t.set("acct9", t.get("acct9") + amount)
            return t.commit(crash)


        def balances(s, o):
            t = Txn(s, o)
            return t.get("acct7"), t.get("acct9")


        if __name__ == "__main__":
            # 1. Snapshot isolation: a reader that started earlier keeps its snapshot.
            s, o = setup()
            old_reader = Txn(s, o)
            assert transfer(s, o, 100)
            assert (old_reader.get("acct7"), old_reader.get("acct9")) == (500, 0)
            assert balances(s, o) == (400, 100)

            # 2. Write-write conflict: the second concurrent writer aborts.
            s, o = setup()
            t1, t2 = Txn(s, o), Txn(s, o)
            t1.set("acct7", 1)
            t2.set("acct7", 2)
            assert t1.commit() and not t2.commit()

            # 3. Client dies after committing the primary: a reader rolls the secondary
forward.
            s, o = setup()
            assert transfer(s, o, 100, crash="after_primary")
            assert balances(s, o) == (400, 100)

            # 4. Client dies before the commit point: a reader rolls everything back.
            s, o = setup()
            transfer(s, o, 100, crash="before_primary")
            assert balances(s, o) == (500, 0)

            # 5. Write skew is allowed: invariant acct7 + acct9 >= 0 is violated.
            s, o = setup()
            a, b = Txn(s, o), Txn(s, o)
            if a.get("acct7") + a.get("acct9") >= 400:
                a.set("acct7", a.get("acct7") - 400)      # writes acct7 only
            if b.get("acct7") + b.get("acct9") >= 400:
                b.set("acct9", b.get("acct9") - 400)      # writes acct9 only
            assert a.commit() and b.commit()
            print("after write skew:", balances(s, o), "sum =", sum(balances(s, o)))
            print("all Percolator scenarios passed")
```

Output:

```
after write skew: (100, -400) sum = -300
all Percolator scenarios passed
```

What to notice:

- Scenario 3 is the heart of the protocol: the client "dies" right after deleting the primary lock, yet the next reader returns (400, 100) because `_cleanup` finds the primary's `write` record and rolls `acct9` forward.
- Scenario 4 is the mirror image: with the primary lock still present, the reader rolls the whole transaction back. No coordinator was needed in either case.
- Scenario 5 is write skew (§28.3.3). The two transactions write disjoint rows, so no write-write check fires, and the customer ends $300 overdrawn. TiDB users avoid this with `SELECT ... FOR UPDATE` on both rows.

INDUSTRY EXAMPLE

Google Percolator — incremental web indexing. Google replaced batch MapReduce index builds with Percolator-based incremental processing for its Caffeine indexing system. Transactions spanning many Bigtable rows and tables update the index as each document is crawled, with observers triggered by notifications. The trade-off was explicit: per-document processing used substantially more resources than batch processing, in exchange for much fresher results. *Evidence: Primary* — [Peng and Dabek 2010]

COMMON MISTAKE

Common misconception — "A leftover lock means data loss." A Percolator lock left by a dead client is an unresolved, not a lost, transaction. Its fate is already fixed by the primary row; the first reader that needs the cell resolves it. What it costs is latency: readers wait, then pay for the cleanup.

28.8 Spanner and TrueTime

28.8.1 The TrueTime API

DEFINITION

Definition — TrueTime. Google's clock API that returns time as an interval with a guaranteed error bound instead of a single value. `TT.now()` returns $[earliest, latest]$, guaranteed to contain the absolute time at which it was called; `TT.after(t)` is true if t has definitely passed, and `TT.before(t)` is true if t has definitely not arrived. The half-width of the interval is the **uncertainty** ε [Corbett et al. 2012].

First-principles explanation

Ordinary clocks lie with confidence: a server says 12:00:00.000 when the true time might be 11:59:59.950 (§24.9). TrueTime is a clock that admits its error: "it is between 11:59:59.996 and 12:00:00.004". To be sure an event happens after time s everywhere, wait until your clock's *earliest* possible reading has passed s.

How it works internally

Each datacenter has **time masters** with GPS receivers and some with atomic clocks, which fail in different ways. A daemon on every machine polls several masters, rejects outliers and derives ε from the masters' uncertainty, communication delay and a conservatively assumed local drift between polls (the paper uses 200 µs/s). In the 2012 paper, ε followed a sawtooth from about 1 to 7 ms over each 30-second polling interval, averaging about 4 ms [Corbett et al. 2012]. If a machine's clock drifts or loses contact with masters, ε grows; it never silently becomes wrong.

28.8.2 Read-write transactions and commit wait

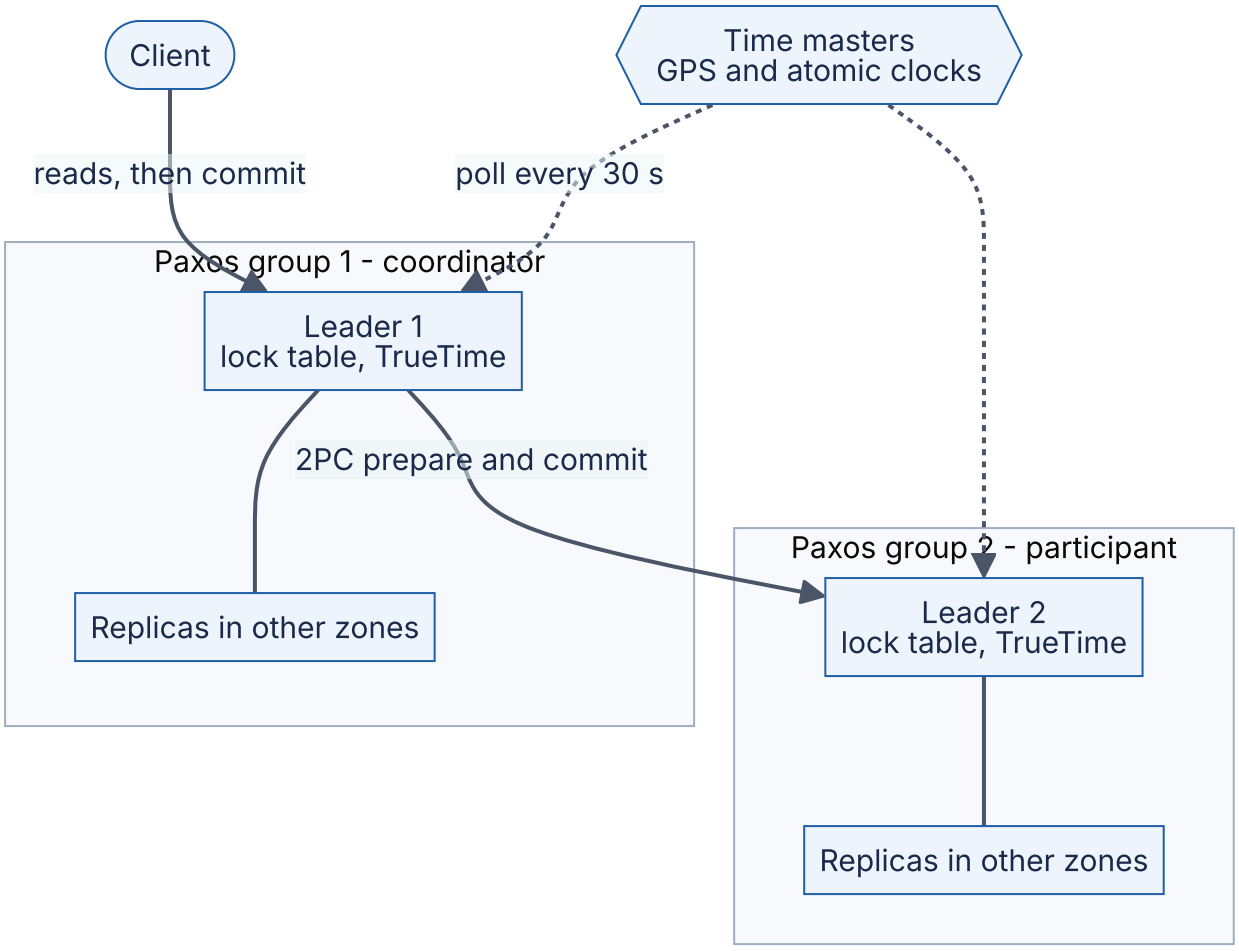
Spanner shards data into **splits** replicated by Multi-Paxos groups spread over zones; each group's leader holds a time-based leader lease (§26.11) and a lock table, and runs two-phase locking for read-write transactions. A transaction that touches several groups runs 2PC across them (§28.4.8), with one group's leader as coordinator.

1. The client buffers writes and acquires read locks at participant leaders as it reads.
2. At commit, each participant leader acquires write locks, chooses a **prepare timestamp** larger than any timestamp it has assigned, and logs a prepare record through Paxos.
3. The coordinator leader, after receiving all prepare timestamps, chooses the **commit timestamp** s: at least every prepare timestamp, greater than `TT.now().latest` at

the moment it received the commit request, and greater than any timestamp it has assigned.

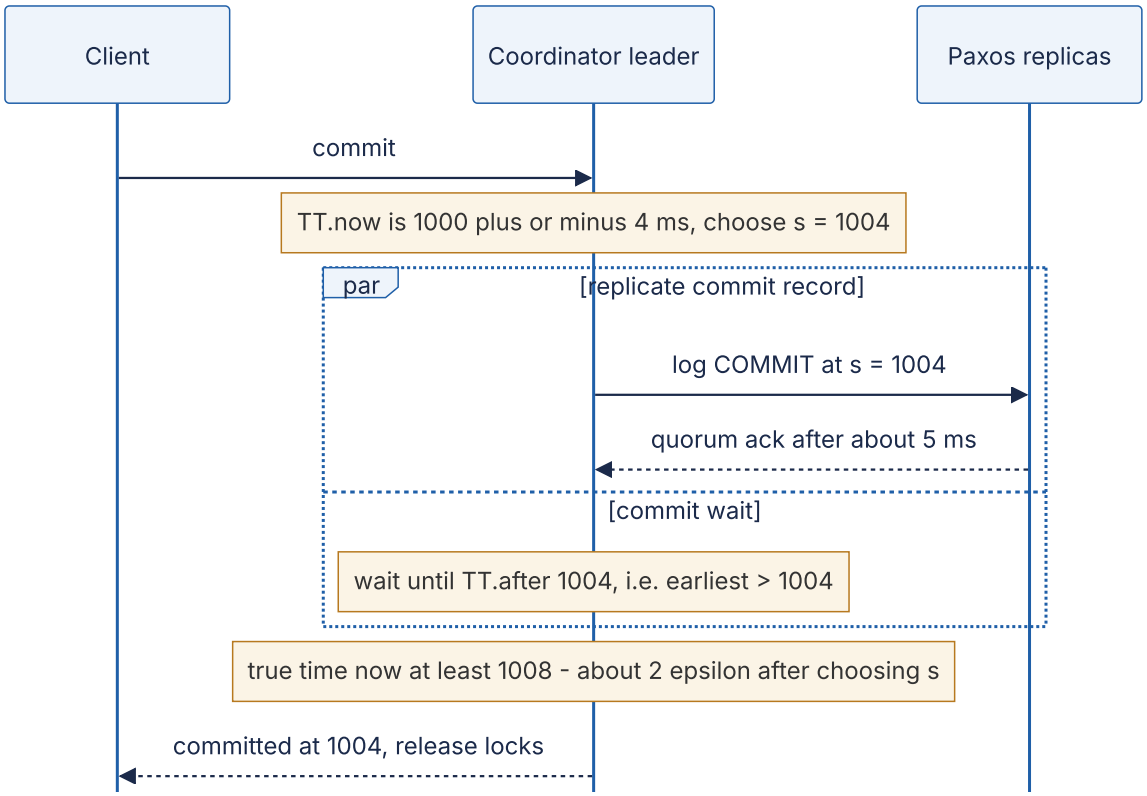
4. It logs the commit record through Paxos and, *in parallel*, performs **commit wait**: it does not let anyone see the commit until `TT.after(s)` is true.
5. It then tells the client and the participants, which apply the writes at timestamp s and release locks.

Figure 28.11 — Spanner: transactions across Paxos groups.



Reading the diagram. Each subgraph is a Paxos group whose leader serves locks and timestamps; replicas live in other zones or regions, so every “log” step is a Paxos round. 2PC runs only between leaders, synchronously. The time masters are an external dependency polled asynchronously (dotted); they bound ε but sit on no transaction’s critical path. Failure points: a leader failure costs a lease expiry and election; time master failures or network trouble widen ε, which slows commits but never breaks correctness. Scaling comes from adding splits and groups.

Figure 28.12 — Commit wait timeline at the coordinator leader.



Reading the diagram. All times are milliseconds on an illustrative scale. When the request arrives, true time is somewhere in [996, 1004], so the leader picks $s = 1004$, the latest possible “now”. It then waits until even the earliest possible reading exceeds 1004, which takes about $2\varepsilon = 8$ ms if ε stays at 4 ms. Paxos replication runs in parallel, so the client sees roughly the larger of the two delays. The storage boundary is the Paxos log; locks are held until the end of commit wait, so wait time directly adds to contention on hot rows.

FORMULA

Commit wait

$$T_{\text{wait}} \approx 2\bar{\varepsilon} \qquad T_{\text{commit}} \approx \max(T_{\text{Paxos}}, 2\bar{\varepsilon})$$

Symbol	Meaning	Units
T_{wait}	Time from choosing s = `TT.now().latest` until `TT.after(s)` holds	ms
$\bar{\varepsilon}$	Average TrueTime uncertainty (half-width) during the wait	ms
T_{Paxos}	Time to replicate the commit record to a quorum	ms
T_{commit}	Commit latency at the coordinator leader, ignoring 2PC phase 1	ms

WORKED EXAMPLE

Worked example — When does commit wait hurt?

1. Regional configuration, replicas 1 ms apart, $T_{\text{Paxos}} \approx 2$ ms; with $\bar{\varepsilon} = 4$ ms, $T_{\text{wait}} = 8$ ms. $T_{\text{commit}} = \max(2, 8) = 8$ ms: commit wait dominates.
2. Multi-region configuration, quorum RTT 50 ms: $T_{\text{commit}} = \max(50, 8) = 50$ ms: commit wait is hidden entirely.
3. Clock trouble widens $\bar{\varepsilon}$ to 100 ms: $T_{\text{wait}} = 200$ ms in both configurations, and hot-row throughput under locking falls to at most $1/0.2 = 5$ commits/s.

Interpretation. Spanner converts clock uncertainty directly into commit latency, so the quality of the time infrastructure is a performance parameter. **Engineering implication.** Commodity NTP is usually accurate to milliseconds but gives no guaranteed bound (§24.9), so a safe configured bound must be hundreds of milliseconds (CockroachDB's default maximum offset is 500 ms, §28.10.1), which would make commit wait unacceptable; this is why systems without TrueTime-class clocks use HLCs and uncertainty intervals instead (§28.10).

28.8.3 External consistency: a proof sketch

DEFINITION

Definition — External consistency. If transaction T_1 commits before T_2 starts, in absolute (wall-clock) time, then T_1's commit timestamp is smaller than T_2's. Combined with serializability, this is **strict serializability**: the serial order respects real-time order, as linearizability does for single objects (§25.3).

Let e_i^{start}, e_i^{server} and e_i^{commit} be the events where T_i's commit request reaches the coordinator, where it is assigned timestamp s_i, and where it becomes visible; let $t_{\text{abs}}(e)$ be absolute time. Suppose $t_{\text{abs}}(e_1^{\text{commit}}) < t_{\text{abs}}(e_2^{\text{start}})$. Then [Corbett et al. 2012]:

1. $s_1 < t_{\text{abs}}(e_1^{\text{commit}})$, by **commit wait**: T_1 is not visible until `TT.after(s_1)`.
2. $t_{\text{abs}}(e_1^{\text{commit}}) < t_{\text{abs}}(e_2^{\text{start}})$, the assumption.
3. $t_{\text{abs}}(e_2^{\text{start}}) \le t_{\text{abs}}(e_2^{\text{server}})$, by causality.
4. $t_{\text{abs}}(e_2^{\text{server}}) \le s_2$, by the **start rule**: $s_2 \ge$ `TT.now().latest` at that moment, which is at least the absolute time.

Chaining the inequalities gives $s_1 < s_2$. Both rules are needed: without commit wait, s_1 could be in the future of T_2's start; without the start rule, s_2 could lag real time.

28.8.4 Read-only transactions and snapshot reads

A read-only transaction takes $s_{\text{read}} =$ `TT.now().latest` and reads at that timestamp, without locks, from *any* replica that is sufficiently up to date. A replica can serve a read at t once $t \le t_{\text{safe}} = \min(t_{\text{safe}}^{\text{Paxos}}, t_{\text{safe}}^{\text{TM}})$: it has applied every Paxos write up to t, and no prepared-but-undecided transaction could still commit at or below t. Because commit timestamps are externally consistent, such a read sees every transaction that committed before it started. Reads at a timestamp in the past (**stale reads**, for example 10 s old) are cheaper still: any replica can serve them immediately. Read-only transactions are therefore never blocked by, and never block, read-write transactions.

28.8.5 Spanner and the CAP theorem

CAP (§25.8) says that during a network partition a system must give up either consistency or availability. Spanner chooses consistency: a minority side of a partition cannot commit. Brewer argues that Spanner is nevertheless "effectively CA": partitions are rare on Google's private network, and Spanner's measured availability exceeds five nines, so users experience a system that is both consistent and available. In the incident data he reports, network problems were a small minority of Spanner's outages, below user error, overload and bugs [Brewer 2017]. The lesson: availability is an engineering number, and

a consistent system on a well-run network can beat a weakly consistent one on a poor network.

INDUSTRY EXAMPLE

Google F1 on Spanner — the AdWords backend. Google migrated its advertising backend from a sharded MySQL deployment to F1, a distributed SQL system built on Spanner, to get cross-shard transactions, consistent secondary indexes and resharding without application changes. The authors describe higher commit latency than MySQL, offset by hierarchical schemas that cluster related rows and by batching and parallelism. *Evidence: Primary* — [Shute et al. 2013]

COMMON MISTAKE

Common misconception — "Spanner works because Google's clocks are perfectly synchronized." Its clocks are not perfect; they are *honest*. Correctness requires only that the true time lie inside TrueTime's interval. Tight intervals determine performance, not correctness.

28.9 Calvin and deterministic databases

28.9.1 Agree on the order first, then execute

DEFINITION

Definition — Deterministic database. A database in which replicas and partitions first agree on a global order of transaction *inputs*, then execute transactions so that the outcome depends only on that order and the initial state. Every node that executes the same log reaches the same state, so no node needs to ask another whether to commit [Thomson et al. 2012].

First-principles explanation

2PC is a meeting held *after* the work: "did everyone manage to do their part?" Calvin holds the meeting *before*: everyone agrees on the list of jobs and their order, then works through the list independently. If every worker follows the same list with the same rules, they cannot disagree about the outcome, so there is nothing to vote on.

Why does this exist?

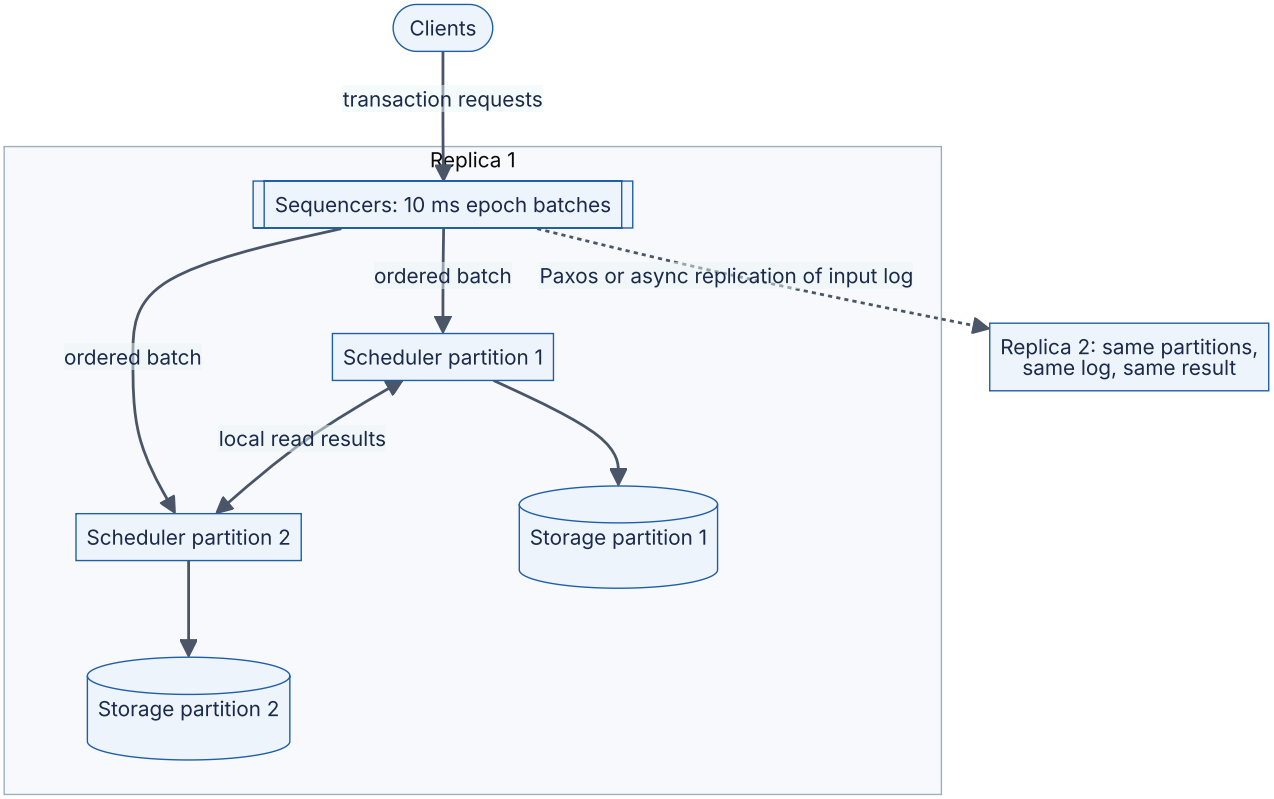
In 2PC a participant can abort for *nondeterministic* reasons: a deadlock victim, a node crash, a timeout. The vote exists to catch those. Thomson and colleagues observed that if

the system removes nondeterministic aborts, the vote becomes unnecessary, and locks no longer need to be held across a commit protocol. Contended transactions then hold locks only for their execution time, which is where Calvin's throughput advantage under contention comes from.

How it works internally

1. **Sequencing.** Clients send transaction requests to a sequencer on any node. Sequencers collect requests into **epochs** of 10 ms, replicate each batch (with Paxos, or asynchronously for lower latency and weaker durability) and send it to the schedulers of every partition the transactions touch. The merged batches form the global input log.
2. **Scheduling.** Each partition's scheduler acquires locks in global log order, deterministically: if T5 precedes T9 in the log and both want key *x*, T5 always gets it first. This rules out deadlocks.
3. **Execution.** Each participating partition performs its local reads and sends the results to the partitions that have writes; each of those runs the transaction logic and applies its own writes. A transaction that fails a constraint aborts on every partition alike, because every partition computes the same result.
4. **Failures.** A crashed node does not cause aborts; a replica, or the node after restart, replays the input log from a checkpoint and reaches the same state.

Figure 28.13 — Calvin's architecture.



Reading the diagram. The sequencing layer is the only place where nodes coordinate synchronously, and what it replicates is the *input* (subroutine shape: a log), not the resulting data. Schedulers within a replica exchange read results directly (two-way arrow) and never exchange votes. Replica 2 receives the same log asynchronously or through Paxos and computes the same state independently, so storage on each replica is a pure function of the log. Failure points: an unreplicated batch lost with its sequencer, and a slow transaction stalling later conflicting ones. Scaling: add partitions; throughput grew nearly linearly in the paper's experiments on up to 100 machines.

FORMULA

Calvin transaction latency

$$T_{\text{Calvin}} \approx \frac{e}{2} + T_{\text{rep}} + T_{\text{exec}}$$

Symbol	Meaning	Units
e	Epoch length; a request waits on average half an epoch to be batched	ms
T_{rep}	Time to replicate the batch (Paxos round, or 0 for asynchronous)	ms
T_{exec}	Lock wait, read forwarding and execution	ms
T_{Calvin}	Expected time from request to result	ms

WORKED EXAMPLE

Worked example — Calvin versus 2PC across regions. Epoch $e = 10$ ms; three replicas in three regions with quorum RTT 70 ms; $T_{\text{exec}} = 2$ ms.

1. Calvin: $5 + 70 + 2 = 77$ ms, and locks are held for only about 2 ms.
2. 2PC over geo-replicated consensus groups: two quorum rounds, ≈ 140 ms, with locks held across the second round of 70 ms or more.
3. In one region with $T_{\text{rep}} = 2$ ms: Calvin needs $5 + 2 + 2 = 9$ ms, slower than about 5 ms for 2PC over consensus groups.

Interpretation. Calvin wins across regions and under contention, because one agreement round replaces two and locks are not held during it; in a single region, batching delay can make it slower. **Engineering implication.** Calvin's lock hold time, not its latency, is what makes it attractive for high-contention workloads such as inventory or ticketing.

28.9.2 Trade-offs and lineage

Advantages. No 2PC and no distributed deadlocks; cheap active replication of inputs instead of data; high throughput on contended keys; easy recovery by replay.

Disadvantages. Transactions must declare their read and write sets before execution. A transaction whose keys depend on data it reads (look up an index, then update the row it points to) needs a **reconnaissance** read first and must restart if the prediction proves wrong; the paper calls this optimistic lock location prediction. Interactive SQL sessions (`BEGIN`, think, `UPDATE`, think, `COMMIT`) do not fit at all. Every request pays half an epoch of batching. One slow or stalled transaction delays every later transaction that conflicts with it, because the order is fixed.

Calvin influenced commercial systems most visibly through **FaunaDB**, whose transaction engine was publicly described as Calvin-inspired; Fauna's hosted service shut down on 30 May 2025 [Fauna Docs: End of Life FAQ]. Later research, such as SLOG, extends deterministic ordering to geo-distributed data with region-local "home" data, so that single-home transactions avoid cross-region ordering [Ren et al. 2019].

> COMMON MISTAKE
>
> **Common misconception — "Deterministic databases have no coordination."** They move all coordination to one place, before execution. The global sequencer is still a consensus problem; it is simply solved once per batch rather than once per transaction.

28.10 Distributed SQL: CockroachDB, YugabyteDB and TiDB

28.10.1 CockroachDB

CockroachDB is a PostgreSQL-wire-compatible distributed SQL database that runs serializable transactions without specialized clocks [Taft et al. 2020].

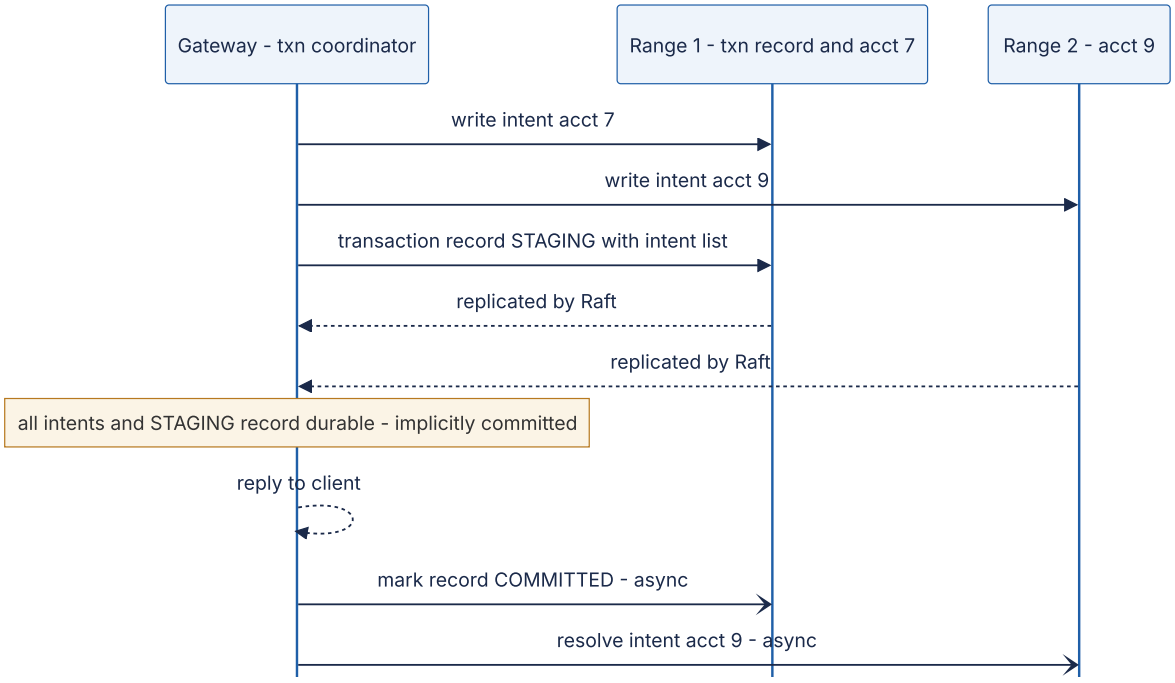
How it works internally

1. **Ranges and leaseholders.** Data is a sorted key space split into **ranges** (512 MiB maximum by default in recent versions) each replicated with Raft. One replica holds a lease and serves reads and coordinates writes for the range.
2. **HLC timestamps.** Every transaction gets a timestamp from the gateway node's **hybrid logical clock** (§24.12), and MVCC versions are tagged with HLC timestamps.
3. **Transaction record and write intents.** A transaction writes provisional values called **write intents**, each pointing to a **transaction record** stored in the range of the transaction's first key. The record's status (`PENDING`, `STAGING`, `COMMITTED`,

`ABORTED`) is the single source of truth, playing the role of Percolator's primary lock. Readers that meet an intent consult the record and may push or wait for the writer.

4. **Serializability by timestamp ordering.** Each leaseholder remembers the latest timestamp at which each key was read (the timestamp cache). A write below that timestamp is pushed to a higher one; before committing at the higher timestamp the transaction must **refresh** its reads, checking they are unchanged in between, or else retry.
5. **Uncertainty intervals.** Clocks may differ by up to a configured maximum offset (500 ms by default). A read at timestamp t that finds a value with timestamp in $(t, t + \text{max offset}]$ cannot know whether that write happened before it in real time, so it restarts at a higher timestamp. Nodes whose clocks drift too far from their peers shut themselves down rather than risk anomalies.
6. **Parallel commits.** Instead of writing intents and *then* a committed record (two sequential consensus rounds), the coordinator writes the intents and a `STAGING` record listing them in parallel. A transaction is **implicitly committed** once the staging record and all its intents are replicated; the client is answered then, and the record is marked `COMMITTED` asynchronously [Cockroach Labs 2019].

Figure 28.14 — CockroachDB parallel commits.



Reading the diagram. The three writes are independent Raft proposals in different ranges, issued concurrently, so the client waits for one consensus round, not two. "Implicitly committed" is a property any node can verify: a reader that finds a `STAGING` record checks whether every listed intent exists at the staged timestamp, and if so treats the transaction as committed. The asynchronous arrows clean up afterwards. If the gate-

way dies, a recovering transaction completes it if all intents exist, or prevents the missing intent from being written and aborts it.

FORMULA

Commit latency in consensus rounds

$$T_{\text{serial}} \approx 2\,T_{\text{c}} \qquad T_{\text{parallel}} \approx T_{\text{c}}$$

Symbol	Meaning	Units
T_{c}	One consensus (Raft) round for the slowest range	ms
T_{serial}	Commit with intents then a committed record	ms
T_{parallel}	Commit with parallel commits	ms

WORKED EXAMPLE

Worked example — A multi-region cluster. Ranges have voters in three regions and $T_{\text{c}} = 40$ ms. Serial commit: $2 \times 40 = 80$ ms. Parallel commit: 40 ms, halving commit latency. In one region with $T_{\text{c}} = 2$ ms the saving is 2 ms, worth having but not decisive. **Engineering implication.** Commit protocols matter most where consensus rounds are expensive, which is exactly where geo-distributed SQL is deployed.

28.10.2 YugabyteDB

YugabyteDB reuses PostgreSQL's query layer code above a distributed storage engine, **DocDB**, which splits tables into **tablets** replicated by Raft. Multi-tablet transactions write **provisional records** (intents) to each tablet and keep their state in a **transaction status tablet**, again a replicated record that decides the outcome. Timestamps come from **hybrid time**, an HLC, with a bounded maximum clock skew used for read uncertainty, as in CockroachDB. Single-row and single-tablet operations take a fast path without a status record. It supports snapshot, serializable and read committed isolation [YugabyteDB Docs: Distributed Transactions].

28.10.3 TiDB

TiDB, described in §28.7.3, runs Percolator over Raft Regions with a central timestamp oracle in PD, defaulting to snapshot isolation with pessimistic locking, with async commit and one-phase commit as optimizations [Huang et al. 2020].

Table 28.6 — Distributed SQL databases compared (as of 2026).

Row	Spanner	CockroachDB	YugabyteDB	TiDB
What it is	Google's globally distributed database, managed service	PostgreSQL-wire-compatible distributed SQL	PostgreSQL-compatible distributed SQL	MySQL-compatible distributed SQL with HTAP
How it works	Paxos groups, 2PL, 2PC across groups, TrueTime commit wait	Raft ranges, HLC, intents, transaction record, parallel commits	Raft tablets, hybrid time, intents, status tablet	Raft Regions, PD timestamp oracle, Percolator 2PC
Default isolation	Serializable, externally consistent	Serializable	Snapshot (YSQL)	Snapshot (`REPEATABLE READ`), pessimistic locking
Advantages	Strongest guarantee; lock-free snapshot reads anywhere	No special clocks; geo-partitioning; serializable default	PostgreSQL feature compatibility; flexible isolation	MySQL compatibility; columnar replicas for analytics
Disadvantages	Google Cloud only; commit wait	Retries under contention; uncertainty restarts	Clock-skew bound; younger ecosystem	Central oracle limits geo-distribution
Performance	Bounded by Paxos RTT and $2\bar{\varepsilon}$	One consensus round per commit with parallel commits	One consensus round for single-tablet writes	Oracle RTTs plus prewrite and commit; async commit saves one
Operational complexity	Low (managed)	Moderate; clock sync must be monitored	Moderate	Moderate to high: TiDB, TiKV, PD, TiFlash
Cost	Managed pricing per node or processing unit	Open-source core and commercial licenses; managed cloud	Open source; managed cloud	Open source; managed cloud
Use cases	Global financial, advertising, inventory systems	Multi-region OLTP with data residency	Migrating PostgreSQL apps that outgrew one node	Migrating sharded MySQL; real-time analytics on OLTP data
Choose when	You are on Google Cloud and need strict serializability globally	You need serializable SQL across regions on any cloud	You need PostgreSQL semantics at scale	You run MySQL and want to drop manual sharding

INDUSTRY EXAMPLE

CockroachDB — geo-distributed OLTP. Taft and colleagues give the goals as surviving zone and region failures automatically while keeping serializable SQL, with per-table or per-row placement for latency and residency; parallel commits and pipelined writes help most in multi-region deployments. *Evidence: Primary* — [Taft et al. 2020]

COMMON MISTAKE

Common misconception — "A distributed SQL database makes cross-region transactions fast." It makes them *correct*. Physics still sets the cost: any transaction whose data has voters in several regions pays at least one cross-region consensus round. Fast systems place data so that most transactions are regional.

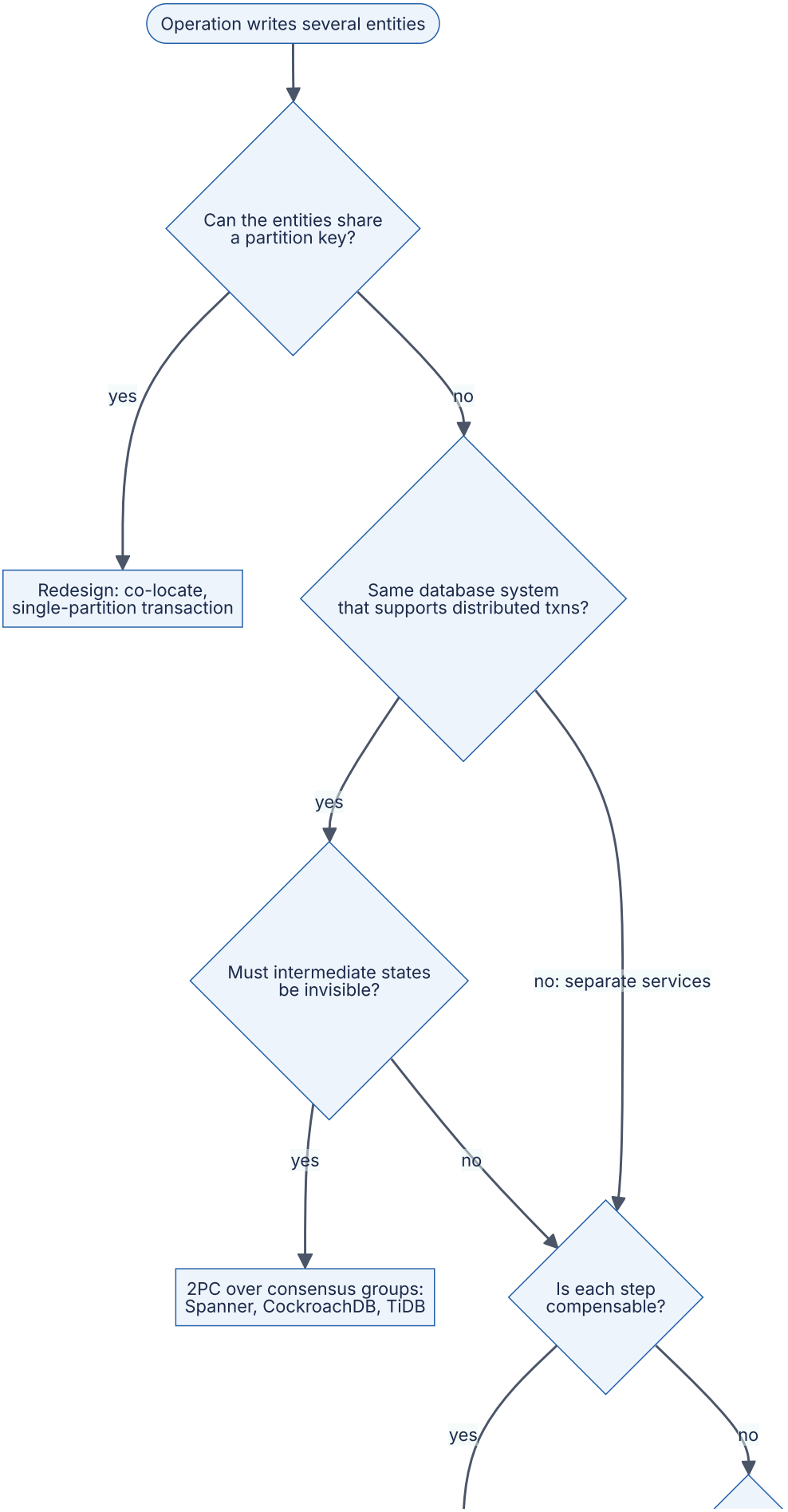
Failure scenarios in distributed SQL

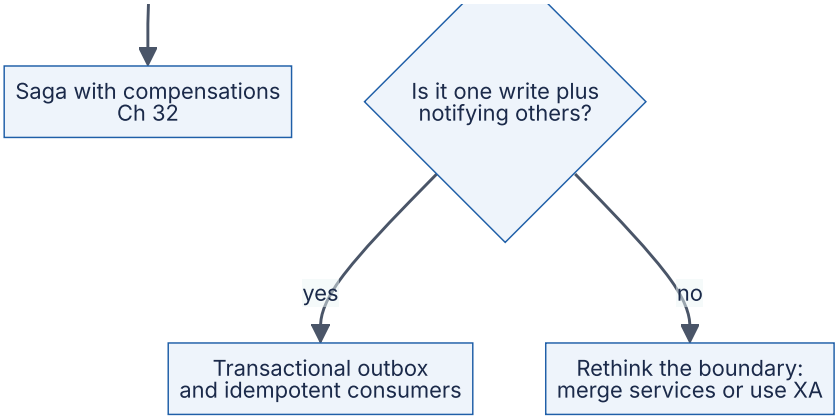
What goes wrong	Why	Detection	Recovery	Prevention
Spike in transaction retry errors	Contention, timestamp pushes failing read refresh	Retry and restart metrics by statement	Client retry loop with backoff	Shorter transactions; avoid hot keys; `SELECT ... FOR UPDATE`
Clock offset alarms, nodes exiting	NTP failure or VM pause beyond max offset	Clock-offset metrics	Fix time sync; restart nodes	Reliable time sync (PTP or cloud time service); alert early
Long-held locks after client crash	Abandoned intents or Percolator locks	Lock wait time	Lock resolution by readers, TTL expiry	Heartbeating transaction records; short TTLs

28.11 Choosing a transaction strategy

Every mechanism in this chapter costs latency, availability or isolation. The best strategy therefore starts by *not needing* a distributed transaction, and climbs the ladder only as far as the requirements force it.

Figure 28.15 — Choosing a transaction strategy.


Operation writes several entities
Can the entities share a partition key?
yes
no
Redesign: co-locate, single-partition transaction
Same database system that supports distributed txns?
yes
no: separate services
Must intermediate states be invisible?
yes
no
2PC over consensus groups: Spanner, CockroachDB, TiDB
Is each step compensable?
yes
no



Reading the diagram. Decision nodes are questions about the data and the business, not the technology. The first question removes most distributed transactions: if a customer's orders and order lines share `customer_id` as partition key (§14.7), the transaction is local. Inside one distributed database, its own 2PC over consensus groups is the right tool when intermediate states must stay invisible; if they may be visible, the cheaper cross-service patterns also apply. Across services with separate databases, sagas (asynchronous, compensating) and the outbox (atomic local write plus at-least-once publication) replace atomic commitment with eventual, semantic consistency. Explicit XA across heterogeneous systems is the last resort.

Table 28.7 — Transaction strategies compared.

Strategy	Atomicity	Isolation	Latency	Availability	Choose when
Single-partition transaction	Full	Full (local)	One local or quorum write	One partition's	You can choose the partition key
2PC over consensus groups	Full	Serializable or SI	2 quorum rounds (1 with parallel commits)	Product over groups, each highly available	Data is in one distributed database
Explicit 2PC / XA	Full	Depends on RMs	2 RTT + 2 forced writes	Product over unreplicated RMs; blocks	Heterogeneous resources, short transactions, no alternative
Saga	Semantic	None between steps	Sum of local steps, asynchronous	High; steps retry independently	Long-running cross-service processes
Transactional outbox	Local write plus guaranteed publish	Local only	One local commit	One service's	One owner of the write, others react

RECOMMENDATION

Recommendation — A default order of preference. (1) Model data so that transactions are single-partition. (2) If not possible and the data lives in one system, use a distributed SQL database's native transactions and keep them short and regional. (3) Across service boundaries, use sagas and the outbox (Chapter 32), with idempotent consumers. (4) Use XA only for short transactions between a few resources you control, and operate its recovery deliberately: monitor in-doubt transactions, protect the transaction manager's log, forbid heuristic decisions without its records.

WORKED EXAMPLE

Worked example — A wallet service. Each user's wallet and transaction history share partition key `user_id`, so a purchase is single-partition (step 1). Peer-to-peer transfers touch two users. At 500 transfers/s in one region on CockroachDB, a parallel commit adds one consensus round, about 2 ms, which is acceptable, so native transactions are chosen (step 2). Paying out to an external bank cannot join any transaction, so the payout is a saga: debit the wallet, call the bank, and refund on rejection; the debit and the "payout requested" event are written together with an outbox (step 3).

28.12 Summary and revision notes

SUMMARY

- **Atomic commitment** needs every participant to reach the same irreversible decision, with any one able to veto. Partial failure and ambiguous silence make independent local commits unsafe (Figure 28.1).
- **2PC**: participants force a PREPARED record and vote; the coordinator forces the COMMIT record (the commit point) and propagates it. A prepared participant cannot decide alone, so a coordinator crash **blocks** in-doubt transactions and their locks.
- Exposure per crash is $N_{\text{doubt}} = Xw$ with $w \approx \text{RTT} + t_{f,c}$; latency is about $2\,\text{RTT} + 2t_f$; availability is A^k.
- **Presumed abort** removes forced writes and acknowledgments for aborts; read-only and one-phase optimizations remove 2PC for many transactions. XA and PostgreSQL `PREPARE TRANSACTION` expose 2PC; heuristic decisions break atomicity.
- **3PC** avoids blocking only with perfect failure detection; partitions make it inconsistent. Production systems run **2PC over consensus groups** instead.
- **Sagas** trade isolation for availability with compensations (Chapter 32).
- **Percolator** builds SI transactions from single-row atomicity: a timestamp oracle, prewrite locks pointing to a primary, and the primary's `write` record as commit point; readers roll abandoned transactions forward or back. TiDB uses it over Raft.
- **Spanner** assigns commit timestamps from TrueTime and waits about 2ε before exposing a commit, yielding external consistency and lock-free snapshot reads.
- **Calvin** agrees on input order first and executes deterministically, eliminating 2PC at the cost of declared read/write sets and batching delay.
- **CockroachDB** uses HLCs, uncertainty intervals, transaction records and parallel commits (one consensus round); YugabyteDB uses hybrid time and a status tablet.
- Snapshot isolation distributes cheaply but permits **write skew**; serializability needs read locks or read validation.
- Prefer single-partition design, then native distributed transactions, then sagas and the outbox; use XA last.

28.13 Knowledge checkpoint

1. Why can a participant abort unilaterally before voting yes but not after?
2. Which two log writes must be forced in 2PC's commit path, and what goes wrong if each is not?

3. What does a recovering coordinator do with a COMMIT record that has no END record?
4. Under presumed abort, why may the coordinator forget an aborted transaction immediately?
5. A coordinator handles 5,000 distributed transactions/s with RTT 1 ms and a 2 ms forced write. How many are left in doubt by a crash?
6. What is the availability of a transaction spanning 4 partitions at 99.9 % each?
7. Why does 3PC become unsafe under a network partition?
8. What is Percolator's commit point, and how does a reader resolve a lock left by a dead client?
9. Why must Spanner's coordinator perform commit wait?
10. With $\bar{\varepsilon} = 3$ ms and regional Paxos latency 4 ms, what is Spanner's approximate commit latency?
11. How does Calvin avoid 2PC, and what does it require from transactions?
12. What makes a CockroachDB transaction "implicitly committed"?
13. Why does snapshot isolation allow write skew across partitions?
14. Give the first strategy to try before any distributed transaction.

Answers

1. Before voting, no decision can include its yes vote, so abort is safe. After voting yes, the coordinator may already have logged commit.
2. The participant's PREPARED record (else a crash loses the promise and the participant might later be unable to commit) and the coordinator's COMMIT record (else a recovered coordinator could presume abort while some participant has committed).
3. Resends COMMIT to every listed participant until all acknowledge, then writes END.
4. Inquiries about a transaction it has no record of are answered "abort", which is correct for aborted transactions.
5. $w = 1 + 2 = 3$ ms; $N = 5,000 \times 0.003 = 15$.
6. $0.999^4 \approx 0.996$, about 173 minutes of unavailability per 30 days.
7. Each side runs the termination protocol; one side may see a pre-committed node and commit while the other sees only uncertain nodes and aborts.
8. Writing the primary cell's `write` record and erasing its lock. A reader checks the primary: lock still present means roll back; a `write` record for that start timestamp means roll the secondary forward.
9. To guarantee that its timestamp s is in the past everywhere before anyone sees the commit, so any later transaction gets a larger timestamp (external consistency).
10. $\max(4, 2 \times 3) = 6$ ms.
11. It fixes a global input order before execution and executes deterministically, so there are no nondeterministic aborts to vote on. Transactions must declare read and write sets in advance.

12. A STAGING transaction record and all intents it lists are durably written at the staged timestamp.
13. SI checks only write-write conflicts; the two transactions write different keys, and the read-write dependencies span partitions that never compare notes.
14. Choose partition keys so the entities written together share a partition, making the transaction local.

28.14 Interview questions

Basic

Q1. Explain two-phase commit and its main weakness. Answer. A coordinator sends prepare; each participant forces a prepared record and votes; if all vote yes, the coordinator forces a commit record and tells everyone to commit, otherwise abort. The weakness is blocking: participants that voted yes cannot decide if the coordinator fails, and hold locks until it recovers. Also mention two round trips, two forced writes and A^k availability.

Q2. When would you use a saga instead of 2PC? Answer. When the steps cross service or organizational boundaries, run for seconds to days, involve external APIs that cannot participate in 2PC, or must stay available when one service is down. Each step must have a compensation and be idempotent, and the business must tolerate visible intermediate states.

Intermediate

Q3. How does Percolator achieve atomicity without a coordinator process? Answer. The primary lock stores the coordinator's state. Prewrite locks every cell, each secondary lock pointing to the primary. Committing writes the primary's `write` record at the commit timestamp in one single-row transaction; that is the commit point. Any reader finding a stale secondary lock looks at the primary to roll forward or back. Timestamps from the oracle give snapshot isolation; costs are two oracle calls and SI's write skew.

Q4. Your XA-based order system has rows locked for 20 minutes after a deploy. Diagnose. Answer. Likely in-doubt prepared transactions: the deploy killed the transaction manager after participants voted yes. Check `pg_prepared_xacts` or `XA RECOVER` for old XIDs; bring the transaction manager back with its log so it re-drives outcomes; if the log is lost, resolve manually by auditing each XID. Prevent with graceful shutdown that drains transactions, a durable replicated TM log, and alerts on prepared-transaction age.

Advanced

Q5. Explain why Spanner's commit wait gives external consistency, and its cost. Answer. The coordinator picks $s \geq$ `TT.now().latest` and does not expose the commit until `TT.after(s)`, so s is in the past when anyone can observe it. A transaction starting later gets a timestamp at least its own `TT.now().latest`, which exceeds true time and therefore s. Cost: about $2\bar{\varepsilon}$ per commit with locks held, overlapped with Paxos replication; it grows if clock uncertainty grows.

Q6. Design transfers between accounts in a multi-region bank where most accounts are regional. Answer. Partition by home region and customer so most transfers are regional single- or multi-range transactions in a distributed SQL database with serializable isolation and parallel commits. Cross-region transfers either pay one cross-region consensus round, or become a saga through a regional clearing account with idempotent steps and an outbox. Quantify: regional commit about 2–5 ms, cross-region 70+ ms. Discuss write skew on overdraft checks and hot merchant accounts.

28.15 Practical exercises

1. Extend the simulation in §28.4.7 with a cooperative termination protocol: an in-doubt participant asks the others, and decides if any knows the outcome or has not voted. Add a test where it succeeds.
2. Convert the simulation to presumed commit and count forced writes and messages in both outcomes; compare with Table 28.2.
3. Repeat the PostgreSQL session of §28.4.6 and measure how long an unrelated `UPDATE` waits; then watch `age(datfrozenxid)` while a prepared transaction stays open.
4. Modify the Percolator model to prevent write skew in scenario 5 by having each transaction also lock the row it only read.
5. Compute Spanner commit latency for $\bar{\varepsilon} \in \{1, 4, 10\}$ ms against Paxos latencies of 2, 20 and 80 ms, and identify where commit wait matters.

28.16 Design challenges

1. **Ticketing flash sale.** 100,000 buyers compete for 5,000 seats in one stadium. Compare Calvin-style deterministic ordering, a single-partition design and 2PC across seat partitions on contention and latency.
2. **Cross-service checkout.** Order, inventory, payment and shipping services each own a database. Design checkout using sagas and an outbox; list each compensation and the isolation anomalies you accept.

3. **Global ledger.** A ledger must be strictly serializable across three continents with p99 commit below 200 ms. Choose a database, data placement and transaction pattern, and estimate latency.

28.17 References

1. Gray, J. *Notes on Data Base Operating Systems*. In *Operating Systems: An Advanced Course*, LNCS 60, Springer, 1978.
2. Bernstein, P. A., Hadzilacos, V., Goodman, N. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987. Free online from Microsoft Research.
3. Skeen, D. *Nonblocking Commit Protocols*. ACM SIGMOD, 1981.
4. Mohan, C., Lindsay, B., Obermarck, R. *Transaction Management in the R* Distributed Database Management System*. ACM Transactions on Database Systems 11(4), 1986.
5. Gray, J., Lamport, L. *Consensus on Transaction Commit*. ACM Transactions on Database Systems 31(1), 2006.
6. The Open Group (X/Open). *Distributed Transaction Processing: The XA Specification*. 1991.
7. Garcia-Molina, H., Salem, K. *Sagas*. ACM SIGMOD, 1987.
8. Peng, D., Dabek, F. *Large-scale Incremental Processing Using Distributed Transactions and Notifications*. USENIX OSDI, 2010.
9. Corbett, J. C., et al. *Spanner: Google's Globally-Distributed Database*. USENIX OSDI, 2012.
10. Brewer, E. *Spanner, TrueTime and the CAP Theorem*. Google, 2017.
11. Shute, J., et al. *F1: A Distributed SQL Database That Scales*. PVLDB 6(11), 2013.
12. Thomson, A., Diamond, T., Weng, S.-C., Ren, K., Shao, P., Abadi, D. J. *Calvin: Fast Distributed Transactions for Partitioned Database Systems*. ACM SIGMOD, 2012.
13. Ren, K., Li, D., Abadi, D. J. *SLOG: Serializable, Low-latency, Geo-replicated Transactions*. PVLDB 12(11), 2019.
14. Taft, R., et al. *CockroachDB: The Resilient Geo-Distributed SQL Database*. ACM SIGMOD, 2020.
15. Cockroach Labs. *Parallel Commits: An Atomic Commit Protocol for Globally Distributed Transactions*. Cockroach Labs Blog, 2019. https://www.cockroachlabs.com/blog/parallel-commits/
16. Huang, D., et al. *TiDB: A Raft-based HTAP Database*. PVLDB 13(12), 2020.
17. PingCAP. *TiDB Documentation: Transactions Overview; Pessimistic Transaction Mode*. https://docs.pingcap.com/tidb/stable/transaction-overview
18. Yugabyte. *YugabyteDB Documentation: Distributed Transactions*. https://docs.yugabyte.com/
19. Apache Kafka. *KIP-98: Exactly Once Delivery and Transactional Messaging*. 2017. https://cwiki.apache.org/confluence/display/KAFKA/KIP-98+-

+Exactly+Once+Delivery+and+Transactional+Messaging
20. Cahill, M. J., Röhm, U., Fekete, A. D. *Serializable Isolation for Snapshot Databases*. ACM SIGMOD, 2008.
21. PostgreSQL Global Development Group. *PostgreSQL 16 Documentation: PREPARE TRANSACTION; pg_prepared_xacts*. https://www.postgresql.org/docs/16/sql-prepare-transaction.html
22. Oracle. *MySQL 8.4 Reference Manual: XA Transactions*. https://dev.mysql.com/doc/refman/8.4/en/xa.html
23. Fauna. *Fauna Service End of Life FAQ*. https://docs.fauna.com/fauna/current/eol-faq
24. Kleppmann, M. *Designing Data-Intensive Applications* (Ch. 7 “Transactions”, Ch. 9 “Consistency and Consensus”). O’Reilly, 2017.

28.18 Open verification items

- Table 28.2 forced-write and message counts for presumed abort and presumed commit, against Mohan et al. 1986.
- Percolator oracle throughput (~2 million timestamps/s) and the 50 % document-age reduction.
- Brewer 2017: exact share of Spanner incidents attributed to the network (a secondary source says under 8 %).
- Calvin’s near-linear scaling to 100 machines; FaunaDB’s description as Calvin-inspired.
- Defaults as of 2026: CockroachDB range size (512 MiB), max offset (500 ms); YugabyteDB YSQL default isolation; TiDB async commit and 1PC status.

Chapter 29 — Distributed Data Processing

> **Level:** 3 · **Track:** Shared (A Traditional and B ML/AI) · **Estimated reading time:** 3 h

Chapters 11–15 stored data; Chapters 24–28 kept copies of it consistent. This chapter is about *computing over* data too large for one machine: counting a month of click logs, joining two billion orders with their customers, or keeping a live per-minute fraud score for every card. The same few ideas recur in every engine: split the input, run the same function on each piece, move records with the same key to the same place (the **shuffle**), and recover from failures by recomputing rather than replicating. You will meet them in MapReduce, where every step is visible; in Apache Spark, which keeps data in memory and plans jobs as graphs; and in stream processing and Apache Flink, where the input never ends. Chapter 61 (ML data pipelines) builds directly on this chapter.

29.1 Learning objectives

After this chapter you should be able to:

- Explain immutable inputs and throughput-first design, and estimate a job's scan time.
- Trace a MapReduce job from splits to reduce, estimate its run time, and quantify what a combiner saves.
- Explain recovery by re-execution and lineage, and how backup tasks cut straggler delay.
- Choose between a sort-merge join and a broadcast join by cost.
- Describe RDDs, narrow and wide dependencies, stages, caching, DataFrames, Catalyst and Adaptive Query Execution, and write a correct PySpark job.
- Calculate shuffle time on a rack-structured cluster, the slowdown from key skew, and the effect of salting.
- Explain data locality and why disaggregated storage changed its importance.
- Define event time, windows, watermarks, triggers and allowed lateness, and compute the late fraction for a watermark delay.
- Explain Flink's state, barrier snapshots and savepoints, and size a checkpoint interval.
- Compare Lambda and Kappa, batch and stream, and Spark and Flink, and place Apache Beam.

29.2 Prerequisites

- Object storage and GFS/HDFS — §15.4–§15.5 (where batch inputs live).
- Columnar storage and Parquet — §13.13.
- Partitioning, hash and range — §14.6 and hot partitions §14.10 (the shuffle is partitioning on the fly).
- Datacenter networking and oversubscription — §7.17 (shuffle time is a network problem).
- Probability and Amdahl's Law — §4.6 and §4.12 (stragglers and the serial parts of a job).
- Clocks and logical time — §24.9–§24.10 (event time; the happens-before reasoning behind the Chandy–Lamport snapshots of §29.9.2).
- Backpressure and idempotency — §22.10 and §22.15 (streaming operators slow their sources; sinks must tolerate replays).
- Units: **MB/GB/TB** are powers of ten; network speeds are in bits per second (Gb/s), so 25 Gb/s = 3.125 GB/s.

29.3 Batch processing concepts

29.3.1 Bounded, immutable inputs and throughput-first design

> DEFINITION
>
> **Definition — Batch processing.** Running a computation over a **bounded** dataset (one whose size is known when the job starts, such as "yesterday's logs") that reads immutable input files, writes new output files, and is judged mainly by **throughput** (bytes or records processed per second) and total completion time rather than by the latency of individual records.

First-principles explanation

Imagine counting every word in a library. You could ask one person to read every book: correct, but it takes years. Or you could give each of 1,000 volunteers a shelf, have each write a tally sheet, then have a second group add the tally sheets together letter by letter: one person totals words beginning with "a", another "b", and so on. Nobody edits a book; the books are only read. If a volunteer goes home sick, someone else redoes the same shelf. That is batch processing: **split, process in parallel, regroup by key, combine**, over inputs that do not change.

The Unix shell already works this way on one machine. To find the five most common URLs in a web server log you can write:

```
cat access.log | awk '{print $7}' | sort | uniq -c | sort -rn | head -5
```

Each program reads a stream, does one job, and writes a stream; the input file is never modified; and `sort` is the step that brings equal keys together so `uniq -c` can count them. MapReduce and its successors are this pipeline spread over thousands of machines, with the distributed file system playing the role of the pipe [Kleppmann 2017, ch. 10].

Why does this exist?

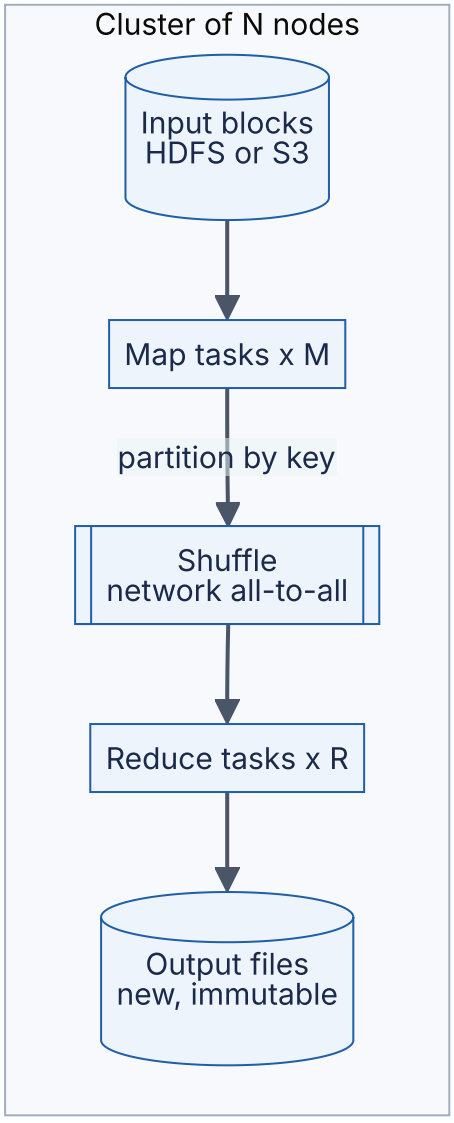
Three properties make large computations tractable:

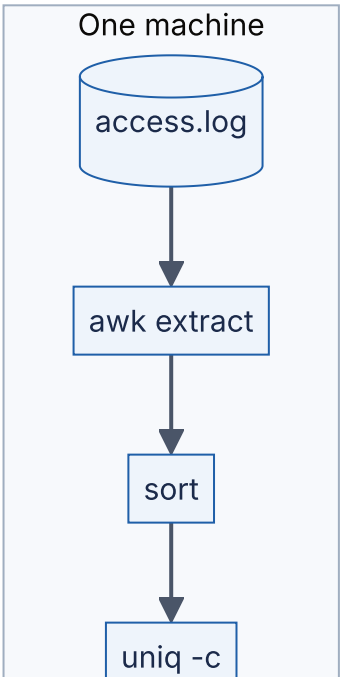
1. **Immutable inputs.** If inputs are never modified, any piece of work can be rerun and will produce the same output. Fault tolerance becomes “do it again” rather than distributed transactions (Chapter 28). A buggy job can simply be fixed and rerun.
2. **Bounded datasets.** Knowing the whole input exists lets the system sort it, count it, plan the work in waves, and decide when the job is finished.
3. **Throughput over latency.** Nobody waits on one record, so the system can use large sequential I/O, keep every disk and core busy, and accept tens of seconds of scheduling overhead.

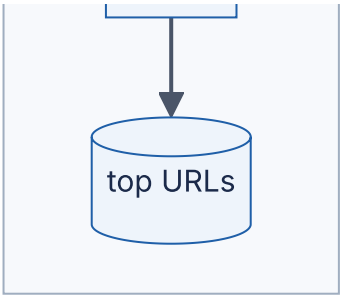
How it works internally

1. Store the input as large files split into blocks (128 MB in HDFS, §15.5.2) or objects.
2. Divide the input into **splits**, usually one per block, and run one **task** per split.
3. Run tasks in parallel on many nodes, each writing intermediate output locally.
4. Regroup intermediate records by key across the network.
5. Write the final output as new immutable files; a job “commits” by atomically renaming or registering its output directory.

Figure 29.1 — A Unix pipeline and the distributed batch job that generalizes it.


Cluster of N nodes
Input blocks
HDFS or S3
Map tasks x M
partition by key
Shuffle
network all-to-all
Reduce tasks x R
Output files
new, immutable


One machine
access.log
awk extract
sort
uniq -c



Reading the diagram. The top row is a single-machine pipeline: `awk` extracts the URL, `sort` brings equal URLs together, `uniq -c` counts them, and the input file is only read. The bottom row is the cluster equivalent: *input blocks* feed *M map tasks* in parallel; the *shuffle* (asynchronous, all-to-all) plays the role of `sort`, delivering every record with one key to one of *R reduce tasks*, which write *new output files*. Storage: inputs and outputs are durable and replicated; intermediate output sits on local disks. Failure points: a dead node loses running tasks and local files, recovered by rerunning tasks. Scaling: add nodes to raise M and R.

Mathematical foundations

The first estimate for any batch job is how long it takes just to read the input.

FORMULA

Scan time of a bounded dataset

$$T_{\text{scan}} = \frac{D}{N \cdot b}$$

Symbol	Meaning	Units
T_{scan}	Time to read the whole input once, ignoring stragglers	s
D	Input size	bytes
N	Number of nodes reading in parallel	count
b	Sustained read-and-process rate per node	bytes/s

WORKED EXAMPLE

Worked example — Why a 10 TB log needs a cluster. A day of clickstream logs is $D = 10$ TB $= 10^{13}$ B. One node can read and parse at $b = 500$ MB/s.

1. One node: $10^{13}/(5 \times 10^{8}) = 20,000$ s ≈ 5.6 hours.
2. 100 nodes: $10^{13}/(100 \times 5 \times 10^{8}) = 200$ s ≈ 3.3 minutes.
3. Reading only 3 of 30 columns from Parquet (§13.13) cuts effective D to about 1 TB: 20 s on 100 nodes.

Interpretation. Parallelism turns hours into minutes, but start-up, the slowest task and serial steps do not shrink (Amdahl's Law, §4.12). **Engineering implication.** Reduce D first (columnar formats, date pruning), then raise N; later sections add shuffle and straggler terms.

COMMON MISTAKE

"Batch means old-fashioned or slow." Batch is a contract about bounded inputs and throughput, not about hardware. A modern Spark job over Parquet can finish in seconds; a streaming job can lag hours behind. Choose by whether the input is bounded and how fresh the output must be (§29.11).

29.4 MapReduce

29.4.1 The programming model and word count

DEFINITION

Definition — MapReduce. A programming model and execution system in which the user writes two functions: **map**, which turns one input record into zero or more intermediate key–value pairs, and **reduce**, which receives one intermediate key with *all* values emitted for it and produces output. The system handles splitting, scheduling, regrouping by key, and failures [Dean and Ghemawat 2004].

First-principles explanation

The model works because many computations have the same shape: *look at each record independently* (map), then *combine everything that shares a key* (reduce). Counting words, building an inverted index (word → documents), computing per-user totals and joining two tables by ID all fit. The user writes only the two functions; the framework supplies the parallelism. The signatures are:

```
map:    (k1, v1)        -> list of (k2, v2)
reduce: (k2, list of v2) -> list of v3
```

For word count, map receives one line of text and emits `(word, 1)` for each word; reduce receives `("the", [1, 1, 1, …])` and emits `("the", count)`.

Why does this exist?

Google in the early 2000s ran hundreds of special-purpose programs over crawled documents and request logs. Each reimplemented parallelization, data distribution and failure handling. MapReduce moved that plumbing into a library so engineers without distributed-systems experience could use thousands of machines, and Google rewrote its production indexing system on it [Dean and Ghemawat 2004]. Hadoop MapReduce, the open-source reimplementation on HDFS, brought the same model to the rest of the industry.

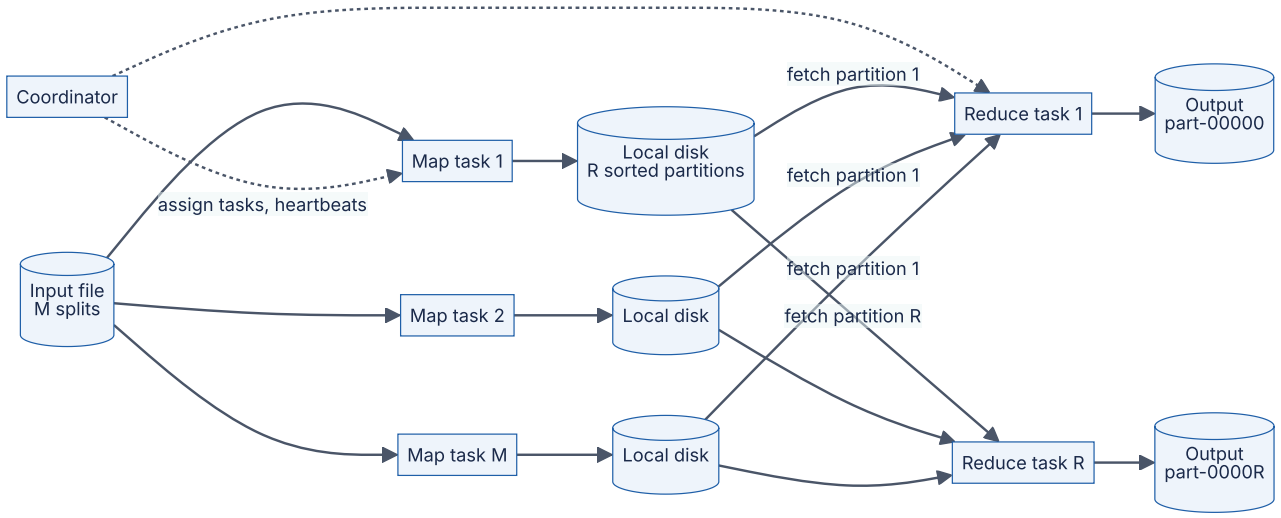
29.4.2 Execution: splits, map, partition, sort and spill, shuffle, reduce

How it works internally

1. **Split.** The input is divided into M splits, typically one per 64–128 MB block. A single **coordinator** (Google's "master"; on YARN an ApplicationMaster) assigns map tasks to idle workers, preferring a worker that stores the split locally (§29.7).
2. **Map.** The worker parses records from its split and calls `map` on each one.
3. **Partition.** Each intermediate pair is assigned to one of R reduce partitions, by default $\text{hash}(k) \bmod R$, so every pair with the same key goes to the same reducer no matter which mapper emitted it.
4. **Buffer, sort and spill.** Pairs accumulate in a memory buffer (100 MB by default in Hadoop). When it fills, the worker sorts the buffer by (partition, key), optionally runs the **combiner**, and **spills** a sorted run to local disk. At task end, spills are merged into one partitioned, indexed file.
5. **Shuffle.** Each reduce task fetches its partition from *every* map task's output over the network, usually starting while maps are still finishing. This all-to-all copy is the shuffle.
6. **Merge.** The reducer merge-sorts the M sorted runs it fetched, spilling to disk if they do not fit in memory, so that all values for a key are adjacent.
7. **Reduce.** The reducer calls `reduce` once per key, streaming the sorted values, and writes one output file per reduce task (`part-00000`, …), made visible by an atomic rename on commit.

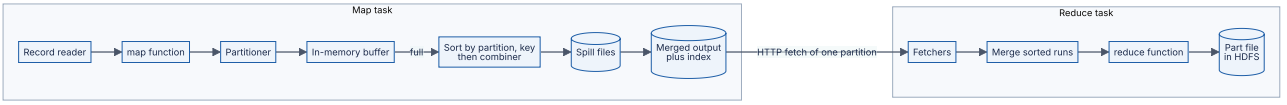
A **combiner** is a reduce function run on the map side, on one mapper's output, before the shuffle. It is legal only for associative and commutative operations (sum, count, max).

Figure 29.2 — MapReduce job execution.



Reading the diagram. The *coordinator* (dotted arrows) assigns tasks and receives heartbeats; it never touches data. Each *map task* reads one split of the replicated *input* and writes R sorted partitions to *local disk*. Each *reduce task* fetches its partition from every map output (synchronous HTTP fetches), merges, reduces and writes one *output* file to the distributed file system. Storage: input and output are replicated; intermediate data is not, so a lost node forces its map tasks to rerun. Scaling: M follows input size, R is chosen by the user, and the shuffle is $M \times R$ transfers.

Figure 29.3 — Inside a map task and a reduce task.



Reading the diagram. Within a map task, records flow from the *record reader* through the user's *map function* and the *partitioner* into a memory *buffer*. Each time the buffer fills, it is sorted by (partition, key), the *combiner* runs, and a *spill file* is written; at the end all spills are merged into one partitioned file with an index giving each partition's byte range. The reduce task's *fetchers* pull its partition from every map output, the *merge* step produces one sorted stream, and the *reduce function* writes a part file to HDFS. Failure points: a full disk fails the task; a fetch failure reruns the map. Scaling: larger buffers mean fewer spills.

Code example

The program below is a complete MapReduce engine in about 70 lines: map tasks with partitioning, per-partition sorting and an optional combiner; a shuffle that gives reducer *r* run *r* from every mapper; a k-way merge; and reduce. Map tasks that "crash" are re-executed. It runs word count with and without a combiner and a reduce-side join, and checks every answer against plain Python.

```
        """A mini MapReduce engine: splits -> map -> (combine) -> partition -> sort ->
shuffle -> reduce.
        Map tasks are pure functions of their split, so a failed task is simply re-exe-
cuted."""
        import heapq
        import itertools
        import zlib
        from collections import Counter, defaultdict


        def hash_partition(key, num_reducers):
            # Deterministic across processes (Python's built-in hash() is salted per
process).
            return zlib.crc32(repr(key).encode()) % num_reducers


        class TaskFailure(Exception):
            pass


        class MiniMapReduce:
            def __init__(self, mapper, reducer, num_reducers=4, combiner=None,
                         partitioner=hash_partition, max_attempts=4):
                self.mapper, self.reducer, self.combiner = mapper, reducer, combiner
                self.R, self.partitioner, self.max_attempts = num_reducers, partitioner,
max_attempts
                self.stats = Counter()

            def _map_task(self, split, crash):
                if crash:                               # simulate a worker dying mid-
task
                    raise TaskFailure("worker lost")
                buckets = [defaultdict(list) for _ in range(self.R)]
                for record in split:
                    for key, value in self.mapper(record):
                        self.stats["map_output_records"] += 1
                        buckets[self.partitioner(key, self.R)][key].append(value)
                runs = []                               # one sorted run per reducer
partition
                for bucket in buckets:
                    run = []
                    for key in sorted(bucket):
                        values = bucket[key]
                        if self.combiner:               # local pre-aggregation before
the shuffle
                            values = list(self.combiner(key, values))
                        run.extend((key, v) for v in values)
                    runs.append(run)
                return runs

            def run(self, splits, crash_first_attempt=()):
                map_outputs = []                        # map_outputs[m][r] = sorted run
                for m, split in enumerate(splits):
                    for attempt in range(1, self.max_attempts + 1):
                        try:
                            crash = m in crash_first_attempt and attempt == 1
```

```
                            map_outputs.append(self._map_task(split, crash))
                            break
                        except TaskFailure:
                            self.stats["map_reexecutions"] += 1
                    else:
                        raise RuntimeError(f"map task {m} failed {self.max_attempts}
times")
                result, reducer_input = {}, []
                for r in range(self.R):                 # shuffle: reducer r fetches run
r from every mapper
                    runs = [map_outputs[m][r] for m in range(len(map_outputs))]
                    self.stats["shuffled_records"] += sum(len(run) for run in runs)
                    reducer_input.append(sum(len(run) for run in runs))
                    merged = heapq.merge(*runs, key=lambda kv: kv[0])   # k-way merge of
sorted runs
                    for key, group in itertools.groupby(merged, key=lambda kv: kv[0]):
                        for out in self.reducer(key, (v for _, v in group)):
                            result[key] = out
                self.stats["max_reducer_input"] = max(reducer_input)
                return result


        # ---- Job 1: word count ---------------------------------------------------
        def wc_map(line):
            for word in line.lower().split():
                yield word.strip(".,;:!?\"'"), 1


        def wc_sum(key, values):
            yield sum(values)


        # ---- Job 2: reduce-side (sort-merge) join of users and orders ---------------
        def join_map(record):
            table, row = record
            if table == "users":                            # ("users", (user_id, country))
                yield row[0], ("U", row[1])
            else:                                           # ("orders", (order_id, user_id,
amount))
                yield row[1], ("O", (row[0], row[2]))


        def join_reduce(user_id, tagged):
            country, orders = None, []
            for tag, payload in tagged:
                if tag == "U":
                    country = payload
                else:
                    orders.append(payload)
            yield sorted((oid, amt, country) for oid, amt in orders)   # orders en-
riched; None = no profile


        if __name__ == "__main__":
            text = ["the quick brown fox", "jumps over the lazy dog", "the dog barks",
                    "The fox runs. The dog sleeps!", "a quick test of the engine"] * 200
            splits = [text[i:i + 100] for i in range(0, len(text), 100)]        # 10 in-
```

```
put splits
            expected = Counter(w.strip(".,;:!?\"'") for line in text for w in
line.lower().split())

            plain = MiniMapReduce(wc_map, wc_sum, num_reducers=4)
            out_plain = plain.run(splits)
            combined = MiniMapReduce(wc_map, wc_sum, num_reducers=4, combiner=wc_sum)
            out_comb = combined.run(splits, crash_first_attempt={2, 7})
            assert out_plain == dict(expected) == out_comb
            print("distinct words:", len(out_comb), " 'the' =", out_comb["the"])
            print("map output records:", plain.stats["map_output_records"])
            print("shuffled without combiner:", plain.stats["shuffled_records"])
            print("shuffled with combiner:   ", combined.stats["shuffled_records"])
            print("re-executed map tasks:", combined.stats["map_reexecutions"])

            users = [("users", (u, c)) for u, c in [(1, "IN"), (2, "US"), (3, "DE")]]
            orders = [("orders", (100 + i, 1 + i % 4, 10 * i)) for i in range(8)]   #
user 4 has no profile
            joined = MiniMapReduce(join_map, join_reduce, num_reducers=2).run([users,
orders[:4], orders[4:]])
            assert joined[1] == [(100, 0, "IN"), (104, 40, "IN")]
            assert joined[4] == [(103, 30, None), (107, 70, None)]   # orphan orders
keep country None
            print("join, user 2:", joined[2])
            print("all tests passed")
```

Output:

```
distinct words: 15  'the' = 1200
map output records: 4800
shuffled without combiner: 4800
shuffled with combiner:    150
re-executed map tasks: 2
join, user 2: [(101, 10, 'US'), (105, 50, 'US')]
all tests passed
```

What to notice:

- `hash_partition` uses CRC32, not Python's `hash()`, because every mapper must send a key to the *same* reducer; a per-process salted hash would scatter a key across reducers and give wrong counts.
- The combiner cut shuffled records from 4,800 to 150 (10 splits × 15 distinct words) without changing the answer, because addition is associative and commutative.
- Two map tasks crashed and were rerun; the output is identical, because a map task depends only on its immutable split.
- `heapq.merge` is the reduce-side merge of sorted runs: it never loads all values for all keys at once, which is how real reducers handle partitions larger than memory.

Mathematical foundations

FORMULA

MapReduce job time (first-order estimate)

$$T_{\text{job}} \approx T_0 + \left\lceil \frac{M}{Ns_m} \right\rceil \frac{C}{r_m} + \frac{\sigma D}{NB} + \left\lceil \frac{R}{Ns_r} \right\rceil \frac{\sigma D/R}{r_r}, \qquad M = \left\lceil \frac{D}{C} \right\rceil$$

Symbol	Meaning	Units
T_0	Job start-up and commit overhead	s
D	Input size	bytes
C	Split size	bytes
M, R	Number of map and reduce tasks	count
N	Worker nodes	count
s_m, s_r	Map and reduce slots (concurrent tasks) per node	count
r_m, r_r	Per-task processing rate of map and reduce	bytes/s
σ	Shuffle ratio: map output bytes ÷ input bytes, after any combiner	dimensionless
B	Effective per-node shuffle network bandwidth	bytes/s

WORKED EXAMPLE

Worked example — A 1 TB log aggregation on 50 nodes. $D = 10^{12}$ B, $C = 128$ MB, $N = 50$ nodes with $s_m = 16$ map slots and $s_r = 8$ reduce slots, $r_m = 10$ MB/s (parsing-heavy), $\sigma = 0.3$, $B = 1$ GB/s, $R = 400$, $r_r = 20$ MB/s, $T_0 = 20$ s.

1. Map tasks: $M = \lceil 10^{12} / 1.28 \times 10^{8} \rceil = 7,813$. Slots: $50 \times 16 = 800$. Waves: $\lceil 7,813/800 \rceil = 10$.
2. One map task: $128/10 = 12.8$ s, so the map phase takes $10 \times 12.8 = 128$ s.
3. Shuffle: $\sigma D = 300$ GB; per node $300/50 = 6$ GB at 1 GB/s $= 6$ s (much of it overlaps the map phase).
4. Reduce: $50 \times 8 = 400$ slots, so one wave. Each reducer gets 300 GB$/400 = 750$ MB, taking $750/20 = 37.5$ s.
5. Total: $20 + 128 + 6 + 37.5 \approx 192$ s, about 3.2 minutes.

Interpretation. The map phase dominates, and since $7,813/800 = 9.77$, the last wave runs 77% full. **Engineering implication.** Keep tasks to tens of seconds so the last wave is short, and treat the estimate as a *lower bound*: one slow last-wave task adds its whole delay (§29.4.3).

FORMULA

Shuffle volume with and without a combiner

$$V_{\text{no comb}} = n_{\text{pairs}} \cdot b_{\text{pair}}, \qquad V_{\text{comb}} \approx M \cdot u \cdot b_{\text{pair}}$$

Symbol	Meaning	Units
V	Bytes moved across the network in the shuffle	bytes
n_{pairs}	Total intermediate pairs emitted by all mappers	count
b_{pair}	Average serialized size of one key–value pair	bytes
M	Map tasks	count
u	Distinct keys per map task after combining ($u \leq$ pairs per task)	count

WORKED EXAMPLE

Worked example — Word count over 1 TB of text. Assume 6 bytes per word on average (including the space), so a 128 MB split holds about $1.28 \times 10^8 / 6 \approx 2.1 \times 10^7$ words; $M = 7,813$; $b_{\text{pair}} = 12$ B; about $u = 200,000$ distinct words per split.

1. Without a combiner: $n_{\text{pairs}} = 10^{12}/6 = 1.67 \times 10^{11}$ pairs; $V = 1.67 \times 10^{11} \times 12 = 2.0 \times 10^{12}$ B $= 2$ TB, twice the input.
2. With a combiner: $V = 7,813 \times 2 \times 10^5 \times 12 = 1.9 \times 10^{10}$ B ≈ 19 GB.
3. Reduction: $2,000/19 \approx 105\times$.

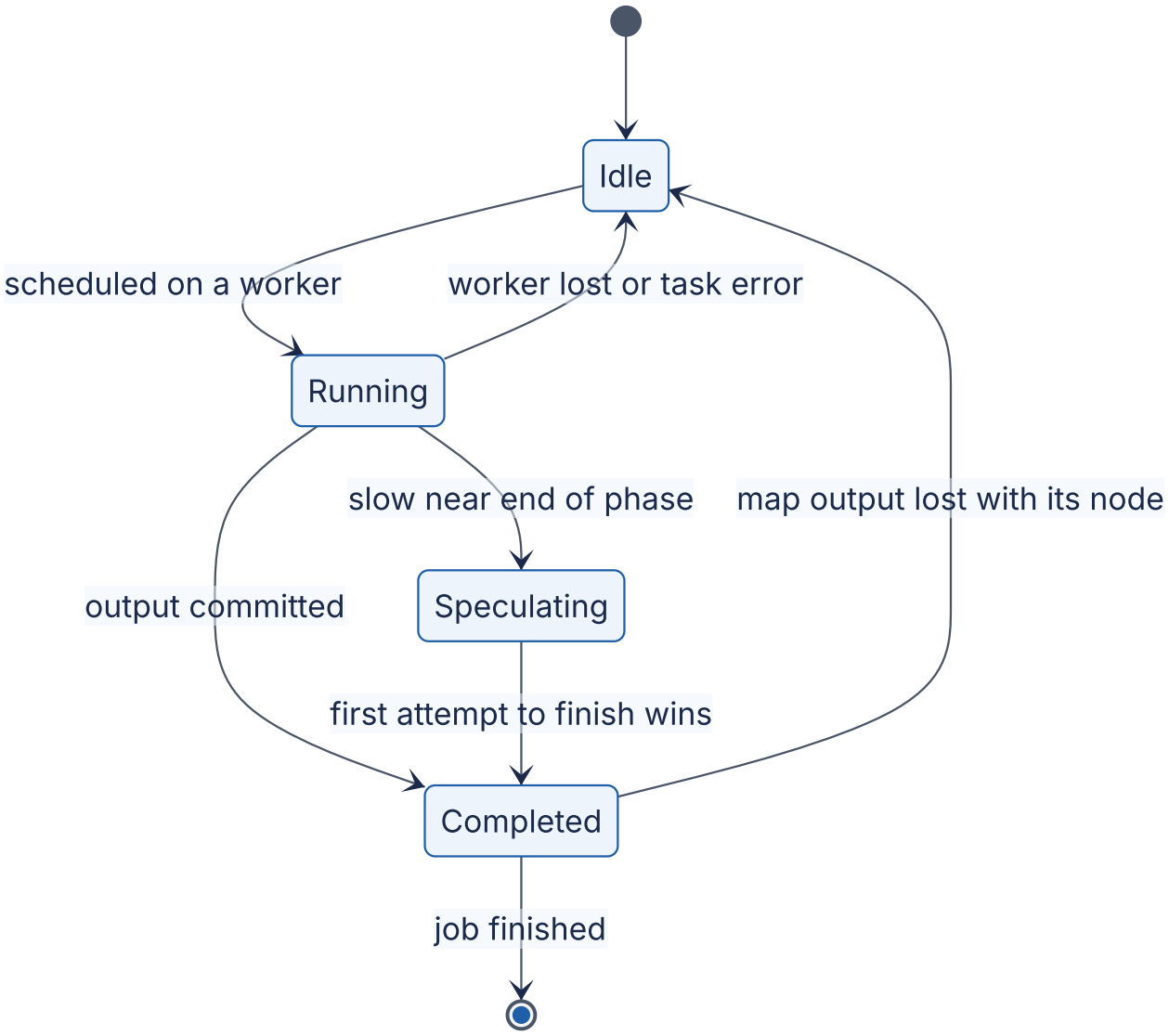
Interpretation. A 12-byte pair replaces a 6-byte word, so naive map output doubles the data; words repeat heavily within a split, so local aggregation removes almost all of it. **Engineering implication.** Always pre-aggregate before a shuffle when the operation allows it; Spark's `reduceByKey` and DataFrame `groupBy().agg()` do this automatically (the `partial_count` step in the Spark plan of §29.5.4), while `groupByKey` does not.

29.4.3 Fault tolerance: re-execution, stragglers and backup tasks

How it works internally

1. **Worker failure.** The coordinator pings workers. If one stops responding, every map task it *completed* is reset to idle and rescheduled, because its output lived on that worker's local disk and is now unreachable; its in-progress map and reduce tasks are rescheduled too. Completed reduce tasks are not rerun, because their output is already in the replicated file system. Reducers that had not yet fetched from the dead worker are told to fetch from the new attempt.
2. **Deterministic re-execution.** Because tasks are functions of immutable inputs, a rerun produces the same output; atomic commit keeps only one attempt's output.
3. **Stragglers and backup tasks.** A **straggler** is a task that runs far slower than its peers because of a bad disk, a noisy neighbor, or a misconfigured machine. Near the end of a phase the coordinator launches **backup** (speculative) copies of the remaining in-progress tasks on other workers and takes whichever finishes first. Google reported that a sort of 1 TB took 44% longer with backup tasks disabled [Dean and Ghemawat 2004].
4. **Bad records** that crash user code deterministically can be skipped after repeated failures.

Figure 29.4 — Lifecycle of a task under re-execution and speculation.



Reading the diagram. Each *task* starts *Idle*, is *Running* once a worker accepts it, and becomes *Completed* when its output is committed. A lost worker or task error sends it back to *Idle* for another attempt (up to a retry limit, four by default in Hadoop). A slow task near the end of a phase enters *Speculating*: a second attempt runs elsewhere, the first to commit wins and the other is killed. The unusual edge is *Completed* → *Idle*: a completed map task's output sits on unreplicated local disk, so losing the node invalidates it. Storage boundary: only committed reduce output is durable. Scaling: speculation costs a few percent extra work and removes most of the tail.

Mathematical foundations

FORMULA

Probability that a phase contains at least one straggler

$$P(\text{at least one straggler}) = 1 - (1-p)^n \approx 1 - e^{-np}$$

Symbol	Meaning	Units
p	Probability that any one task is a straggler	probability
n	Number of tasks in the phase (or in its last wave)	count

WORKED EXAMPLE

Worked example — Why big jobs always have a straggler. Suppose $p = 0.001$: one task in a thousand lands on a machine that runs it 5× slower.

1. Whole map phase of the previous example: $n = 7,813$; $np = 7.8$; $P = 1 - e^{-7.8} = 0.9996$.
2. Only the last wave matters for completion time, because earlier slow tasks are hidden by later waves: $n = 613$ (the tasks in wave 10); $np = 0.613$; $P = 1 - e^{-0.613} = 0.46$.
3. Cost of that straggler: $5 \times 12.8 = 64$ s instead of 12.8 s, adding about 51 s to a 192 s job (+27%).
4. With a backup task launched when the phase is about 95% done, the copy finishes about 13 s after launch, so the delay drops to roughly one task duration.

Interpretation. At scale, “rare” slow machines are present in almost every job, and the slowest task, not the average, sets the finish time (the same tail effect as in §23.6). **Engineering implication.** Enable speculative execution for idempotent tasks, keep tasks short, and do not speculate tasks with side effects (writes to external databases) unless the writes are idempotent (§22.15).

INDUSTRY EXAMPLE

Google MapReduce (2004). The paper describes commodity PCs with 2–4 GB of RAM, 100 Mb/s or 1 Gb/s networking and local disks, with GFS storing input and output. In its sort benchmark (10^{10} 100-byte records, about 1 TB, on roughly 1,800 machines), normal execution took 891 s; with backup tasks disabled it took 1,283 s, 44% longer; and deliberately killing 200 worker processes mid-job increased the time by only about 5%, because the lost tasks were simply re-executed. *Evidence: Primary* — [Dean and Ghemawat 2004]

29.4.4 Joins in MapReduce: sort-merge and broadcast

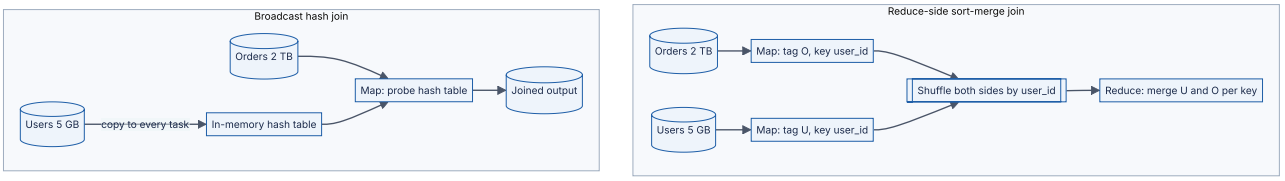
DEFINITION

Definition — Reduce-side (sort-merge) join and broadcast (map-side) join. In a **reduce-side join**, both inputs are mapped to `(join key, tagged record)`, shuffled so that all records with the same key meet at one reducer, sorted, and combined there. In a **broadcast join**, the smaller input is copied in full to every map task, loaded into an in-memory hash table, and each record of the larger input is joined locally by lookup, with no shuffle of the larger input.

How it works internally

1. **Reduce-side join.** Mappers tag each record with its source (`U` or `O` in the code above) and both tables are partitioned by key. A **secondary sort** can put the user record before its orders, so the reducer streams orders past one buffered user. It works for any sizes but shuffles *both* tables.
2. **Broadcast hash join.** The small table (say 5 GB of user profiles) is shipped to every task. Each mapper streams the large table and probes the hash table. Nothing large crosses the network, and no sort is needed. It fails if the small side does not fit in a task's memory.
3. **Bucketed map-side join.** If both inputs were *already* written sorted into the same number of buckets by the join key, task *i* merges bucket *i* of each without a shuffle.

Figure 29.5 — Reduce-side join versus broadcast join.



Reading the diagram. In the *reduce-side join*, mappers key both tables by `user_id` and the *shuffle* (asynchronous, all-to-all) brings matching records to one *reducer*. In the *broadcast join*, *users* are copied to every task as a *hash table* and *orders* are streamed and joined in place. Storage: inputs and outputs are durable; the hash table lives in task memory. Failure points: a broadcast side larger than memory causes out-of-memory failures; a skewed key overloads one reducer (§29.6.3). Scaling: broadcast cost grows with the number of tasks, shuffle cost with both inputs.

Mathematical foundations

FORMULA

Network cost of a shuffle join versus a broadcast join

$$V_{\text{shuffle}} \approx (|L| + |S|) \cdot \frac{N-1}{N}, \qquad V_{\text{broadcast}} \approx |S| \cdot (N-1)$$

Broadcast is cheaper when $|S| \cdot N < |L| + |S|$, roughly $|S| < |L|/(N-1)$, and $|S|$ fits in each executor's memory.

Symbol	Meaning	Units
$\|L\|, \|S\|$	Size of the large and small input	bytes
N	Number of executors or nodes receiving data	count
V	Bytes sent over the network	bytes

WORKED EXAMPLE

Worked example — Orders and users on 100 executors. $|L| = 2$ TB of orders, $N = 100$.

1. $|S| = 5$ GB. Shuffle: $2.005\ \text{TB} \times 0.99 = 1.985$ TB. Broadcast: $5\ \text{GB} \times 99 = 495$ GB. Broadcast moves 4× less and skips sorting 2 TB.
2. $|S| = 50$ GB. Broadcast: $50 \times 99 = 4.95$ TB versus a shuffle of $2.05 \times 0.99 = 2.03$ TB. Shuffle wins, and 50 GB would not fit in most executors anyway.
3. Break-even: $|S| \approx 2\ \text{TB}/99 \approx 20$ GB.

Interpretation. Broadcasting trades a copy per executor for avoiding a shuffle of the big side, which pays only while the small side is really small. **Engineering implication.** Spark broadcasts automatically below `spark.sql.autoBroadcastJoinThreshold` (10 MB by default, deliberately conservative), and Adaptive Query Execution can switch to a broadcast at run time when the actual size after filtering turns out small (§29.5.4). Raise the threshold cautiously and only with memory headroom.

Failure scenarios

What goes wrong	Why	Detection	Recovery	Prevention
Job never finishes; one reducer at 99%	Hot key or straggler	Task duration histogram; one task 10–100× median	Kill and resubmit with salting or speculation	Sample key frequencies; salting (§29.6.3); speculation
Map phase restarts late in the job	Node holding completed map output died	Fetch-failure counters in reducers	Coordinator reruns those maps	External shuffle service or replicated shuffle storage
Executors die with out-of-memory errors in a join	Broadcast side bigger than estimated	OOM logs; broadcast size in plan	Fall back to sort-merge join	Fresh table statistics; AQE
Duplicate rows in output	Two attempts both wrote output without atomic commit	Row counts exceed input-derived bound	Rerun with commit protocol	Commit by rename or table-format transaction

COMMON MISTAKE

"A combiner is just the reducer run early, so any reducer can be a combiner." Only associative and commutative operations qualify, and the combiner may run zero, one or many times. An average computed in the combiner and then averaged again in the reducer is wrong; emit `(sum, count)` pairs and divide only in the reducer.

29.5 Apache Spark

29.5.1 Resilient Distributed Datasets and lineage

DEFINITION

Definition — Resilient Distributed Dataset (RDD). A read-only, partitioned collection of records that can be created only by deterministic operations (**transformations** such as `map`, `filter`, `join`) on stable storage or on other RDDs. Each RDD records its **lineage**, the graph of transformations that produced it, so a lost partition can be recomputed from its parents instead of being restored from a replica [Zaharia et al. 2012].

First-principles explanation

A recipe card is cheaper to keep than a fridge full of prepared dishes. If someone drops the sauce, you do not need a backup sauce; you need the recipe and the raw ingredients, and you redo only the sauce. An RDD is a recipe plus a pointer to ingredients in durable

storage. Spark keeps the dishes (partitions) in memory when it can, and if a node dies it reruns the recipe only for the partitions that node held.

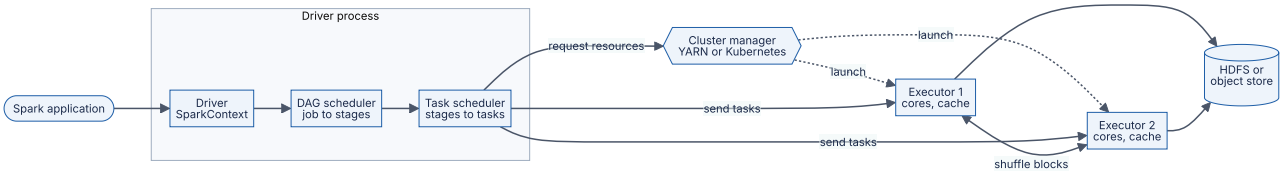
Why does this exist?

MapReduce writes every job's output to the replicated distributed file system, and multi-step pipelines are chains of jobs. That is wasteful for **iterative** algorithms (logistic regression, k-means, PageRank) that reread the same data many times, and for **interactive** analysis. Replicating in-memory data instead would cost network bandwidth comparable to the writes it saved. Lineage gives fault tolerance by logging the *coarse-grained* transformation once per dataset rather than every record [Zaharia et al. 2012].

How it works internally

1. **Transformations are lazy.** `rdd.map(f)` returns a new RDD object describing the computation; nothing runs.
2. **Actions trigger jobs.** `count()`, `collect()` or `saveAsTextFile()` submit a **job** for the final RDD's lineage graph.
3. **The driver plans, executors run.** The **driver** program holds the `SparkContext`, the **DAG scheduler** and the **task scheduler**. A **cluster manager** (YARN, Kubernetes or Spark's standalone manager) grants **executors**, long-lived JVM processes on worker nodes, each with a number of cores and a memory region for cached partitions and shuffle buffers.
4. **Recovery.** When an executor dies, the driver identifies which partitions it held, walks the lineage back to the nearest available ancestors (cached partitions, surviving shuffle files, or the input), and reruns only the needed tasks.
5. **Checkpointing** (`rdd.checkpoint()`) writes an RDD to reliable storage and truncates very long lineages.

Figure 29.6 — Spark runtime: driver, cluster manager and executors.



Reading the diagram. The *driver* turns each action into a job; the *DAG scheduler* splits it into stages and the *task scheduler* places tasks, preferring executors that hold the data. The *cluster manager* starts *executors* asynchronously (dotted). Tasks are sent synchronously; executors read and write *durable storage* and exchange *shuffle blocks*. Storage: cached partitions and shuffle files are unreplicated, on executors. Failure points: a dead executor's partitions are recomputed from lineage; a dead driver loses the application unless restarted in cluster mode. Scaling: add executors, statically or by dynamic allocation.

29.5.2 Narrow and wide dependencies, stages and tasks

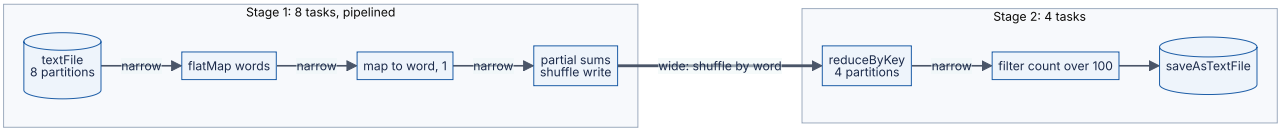
DEFINITION

Definition — Narrow and wide dependencies. A **narrow dependency** exists when each partition of the parent RDD is used by at most one partition of the child (`map` , `filter` , `union` , a join of co-partitioned inputs). A **wide (shuffle) dependency** exists when a child partition may depend on many or all parent partitions (`groupByKey` , `reduceByKey` , a join of unpartitioned inputs), which requires a shuffle.

The distinction drives both performance and recovery. Narrow transformations are **pipelined**: one task applies `map` , `filter` and `map` to each record in a single pass, and recovering a child partition needs one parent partition. A wide dependency is a barrier: all parent partitions must write shuffle files before any child starts, and losing a child may require recomputing many parents.

The DAG scheduler therefore cuts the lineage graph at every wide dependency. Each piece is a **stage**: a chain of narrow transformations ending in a shuffle write (a *shuffle map stage*) or in the action's result (a *result stage*). Each stage runs as one **task** per partition. Stages whose shuffle output already exists are skipped.

Figure 29.7 — Lineage of a word-count job cut into stages at the wide dependency.



Reading the diagram. Each of Stage 1's 8 tasks pipelines `flatMap` , `map` and the partial sum (all narrow) record by record and ends by writing shuffle files partitioned by word. The thick arrow is the *wide* dependency: each of Stage 2's 4 tasks fetches its partition from all 8 outputs (asynchronous fetch), then reduces, filters and writes to durable storage. Shuffle files sit on executor disks between stages. Failure points: a lost Stage 2 executor reruns only its tasks if the shuffle files survive, otherwise also the Stage 1 tasks that wrote them. Scaling: Stage 2 parallelism is the shuffle partition count.

29.5.3 Caching and persistence

`rdd.persist(level)` or `df.cache()` asks Spark to keep the computed partitions after the first action so later actions reuse them. **Storage levels** choose the trade-off: `MEMORY_ONLY` recomputes partitions that do not fit; `MEMORY_AND_DISK` (the DataFrame `cache()` default) spills them to local disk; serialized and replicated (`_2`) variants trade CPU for memory or faster recovery. `cache()` is lazy (the first action fills it), and it is a hint for reuse, not durability: lost cached partitions are recomputed from lineage.

FORMULA

Speedup from caching an iterative job

$$S = \frac{k\,(T_{\text{load}} + T_{\text{iter}})}{T_{\text{load}} + k\,T_{\text{iter}}} \;\xrightarrow{k \to \infty}\; 1 + \frac{T_{\text{load}}}{T_{\text{iter}}}$$

Symbol	Meaning	Units
S	Speedup of the cached version over reloading every iteration	dimensionless
k	Number of iterations (or repeated queries)	count
T_{load}	Time to read, parse and build the working dataset from storage	s
T_{iter}	Compute time of one iteration over the in-memory data	s

WORKED EXAMPLE

Worked example — Ten iterations of a training-feature job. A logistic-regression job reads 1 TB of Parquet from object storage and parses it: $T_{\text{load}} = 60$ s on the cluster. Each gradient pass over the in-memory data takes $T_{\text{iter}} = 6$ s. $k = 10$.

1. Without caching: $10 \times (60 + 6) = 660$ s.
2. With caching: $60 + 10 \times 6 = 120$ s.
3. Speedup: $660/120 = 5.5\times$; the asymptotic limit is $1 + 60/6 = 11\times$.

Interpretation. Caching helps when loading dominates and data is reused; for one pass it only costs memory. **Engineering implication.** Cache the reused dataset after expensive parsing and filtering, check in the Spark UI that it fits (deserialized objects are often several times larger than Parquet), and `unpersist()` it when done.

INDUSTRY EXAMPLE

RDD paper and the 100 TB sort. The RDD paper reported Spark up to 20× faster than Hadoop for iterative applications and interactive queries over a 1 TB dataset in 5–7 s [Zaharia et al. 2012]. In 2014 Databricks reported sorting 100 TB with Spark in 23 minutes on 206 cloud machines, against a previous Hadoop MapReduce record of 72 minutes on 2,100 machines; the run did not rely on in-memory caching but on a new sort-based shuffle and network layer [Xin 2014]. *Evidence: Primary* — [Zaharia et al. 2012; Xin 2014]

29.5.4 DataFrames, Catalyst, Tungsten and Adaptive Query Execution

DEFINITION

Definition — DataFrame, Dataset and Catalyst. A **DataFrame** is a distributed table with a known schema (named, typed columns), manipulated with relational operators or SQL. A **Dataset** (Scala and Java) adds compile-time types to the same abstraction; in Python and R, every DataFrame is untyped at compile time. **Catalyst** is Spark SQL's extensible query optimizer, which turns DataFrame or SQL queries into an optimized physical plan using rule-based and cost-based transformations of trees [Armbrust et al. 2015].

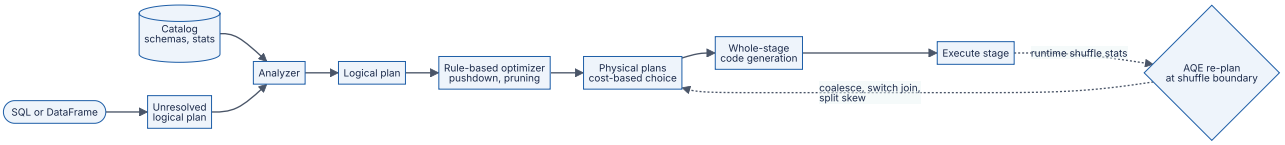
Why does this exist?

An RDD of Python or Java objects is opaque: Spark cannot see that a `lambda` only reads two fields, or that a filter could run before a join. DataFrames make the computation **declarative**: you say *what* (group by country, count), and the engine decides *how*. With a schema, Spark can read only needed columns from Parquet, push filters into the scan, choose join strategies from table sizes, and store rows in compact binary form instead of objects.

How it works internally

1. **Analysis.** The query is parsed into an unresolved logical plan; column and table names are resolved against the catalog.
2. **Logical optimization.** Rules rewrite the tree: predicate pushdown, column pruning, constant folding, combining filters and projections.
3. **Physical planning.** Candidate physical plans are generated (for example broadcast hash join versus sort-merge join) and one is chosen, using size statistics.
4. **Code generation. Tungsten**, Spark's execution layer, stores rows in a compact binary format and uses **whole-stage code generation** to fuse a chain of operators into one generated function (the `*(1)` markers in the plan below).
5. **Adaptive Query Execution (AQE).** Statistics guessed before running are often wrong. With AQE (on by default since Spark 3.2), Spark re-optimizes the remaining plan at each shuffle boundary using the *actual* sizes of completed shuffle outputs. It can (a) coalesce many small shuffle partitions into fewer, right-sized ones (target 64 MB by default); (b) switch a sort-merge join to a broadcast join when one side turned out small; and (c) split skewed partitions, those over 5× the median and over 256 MB by default, into smaller tasks [Apache Spark Docs: Performance Tuning].

Figure 29.8 — Catalyst query planning with Adaptive Query Execution.



Reading the diagram. The *analyzer* resolves names against the *catalog*; the *optimizer* applies rewrite rules; *physical planning* picks operators using statistics; *code generation* compiles each stage; stages *execute* on executors. The dotted loop is AQE: when a shuffle-producing stage finishes, real partition sizes flow back asynchronously and the remaining plan is re-optimized. Planning happens in the driver. Failure points: stale statistics cause bad initial choices, which AQE corrects only at shuffle boundaries. Scaling: planning cost is independent of data size.

Code example

The PySpark program computes page views with the RDD API and the DataFrame API, joins events with users using a broadcast hint, prints the physical plan, and checks every result against plain Python. It was executed with PySpark 4.0.1 in local mode on Java 21.

```
        """PySpark: the same aggregation with the RDD API and the DataFrame API, plus a
broadcast join.
        Tested with PySpark 4.0.1 in local mode (Java 21); results are checked against
plain Python."""
        import random
        from collections import Counter

        from pyspark.sql import SparkSession, functions as F

        spark = (SparkSession.builder.master("local[4]").appName("ch29-demo")
                 .config("spark.sql.shuffle.partitions", "8")          # default is 200
                 .config("spark.sql.adaptive.enabled", "true")          # AQE (default
on since 3.2)
                 .getOrCreate())
        spark.sparkContext.setLogLevel("ERROR")

        rng = random.Random(29)
        pages = ["/home", "/search", "/item", "/cart", "/checkout"]
        events = [(rng.randrange(1_000), rng.choice(pages)) for _ in range(50_000)]
        users = [(u, rng.choice(["IN", "US", "DE", "BR"])) for u in range(1_000)]

        # 1. RDD API: explicit functions, lineage parallelize -> map -> reduceByKey
        rdd = spark.sparkContext.parallelize(events, numSlices=8)
        views_rdd = dict(rdd.map(lambda e: (e[1], 1)).reduceByKey(lambda a, b: a +
b).collect())

        # 2. DataFrame API: declarative, optimized by Catalyst
        ev = spark.createDataFrame(events, ["user_id", "page"]).cache()      # reused
twice below
        us = spark.createDataFrame(users, ["user_id", "country"])
        views_df = {r["page"]: r["n"] for r in
ev.groupBy("page").agg(F.count("*").alias("n")).collect()}

        # 3. Join + aggregate; the small side is broadcast, so the large side is never
shuffled for the join
        by_country = (ev.join(F.broadcast(us), "user_id")
                        .groupBy("country", "page").agg(F.count("*").alias("views")))
        by_country.explain()                        # physical plan: BroadcastHashJoin,
then one Exchange
        result = {(r["country"], r["page"]): r["views"] for r in by_country.collect()}

        # Check against plain Python
        truth_pages = Counter(p for _, p in events)
        country = dict(users)
        truth_join = Counter((country[u], p) for u, p in events)
        assert views_rdd == views_df == dict(truth_pages)
        assert result == dict(truth_join)
        print("views per page:", dict(sorted(views_df.items())))
        print("country/page groups:", len(result), "- all checks passed")
        spark.stop()
```

Output (log lines removed; plan identifiers such as `#13` vary between runs):

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[country#13, page#1], functions=[count(1)])
   +- Exchange hashpartitioning(country#13, page#1, 8), ENSURE_REQUIREMENTS,
[plan_id=90]
      +- HashAggregate(keys=[country#13, page#1], functions=[partial_count(1)])
         +- Project [page#1, country#13]
            +- BroadcastHashJoin [user_id#0L], [user_id#12L], Inner, BuildRight, false
               :- Filter isnotnull(user_id#0L)
               :  +- InMemoryTableScan [user_id#0L, page#1], [isnotnull(user_id#0L)]
               :        +- InMemoryRelation [user_id#0L, page#1], StorageLevel(disk,
memory, deserialized, 1 replicas)
               :              +- *(1) Scan ExistingRDD[user_id#0L,page#1]
               +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, bigint,
false]),false), [plan_id=85]
                  +- Filter isnotnull(user_id#12L)
                     +- Scan ExistingRDD[user_id#12L,country#13]

views per page: {'/cart': 9945, '/checkout': 10053, '/home': 10073, '/item': 9954,
'/search': 9975}
country/page groups: 20 - all checks passed
```

What to notice:

- Read the plan bottom-up. The users side goes through a `BroadcastExchange`; the events side is joined in place by `BroadcastHashJoin`, so the large input is never shuffled for the join.
- There is exactly one `Exchange` (shuffle), for the `groupBy`. Before it, `partial_count` is the map-side combiner of §29.4.2; after it, the final `count` merges partial counts.
- Catalyst added `Filter isnotnull(user_id)` itself: an inner join never matches a null key.
- `AdaptiveSparkPlan isFinalPlan=false`: AQE may still rewrite the plan after the shuffle.
- The RDD version gives the same answer, but Catalyst cannot see inside its lambdas and every record crosses the JVM–Python boundary. Prefer DataFrames and built-in functions.

29.6 Partitioning and shuffle

29.6.1 Hash and range partitioning

Every shuffle must decide which partition each key goes to. The two schemes are the same ones databases use for storage (§14.6), applied on the fly:

- **Hash partitioning**, $p = \text{hash}(k) \mod P$, spreads keys evenly regardless of their values and is the default for aggregations and joins. Keys in one partition are in no useful order.

- **Range partitioning** assigns contiguous key ranges to partitions, so partition i holds keys smaller than those in partition $i+1$. Concatenating sorted partitions gives a globally sorted output (`ORDER BY`, sort benchmarks); the engine first **samples** the input to choose balanced boundaries.

Both schemes balance *keys*, not *records*: a key with 20% of the records lands entirely in one partition under either. That is the skew problem of §29.6.3.

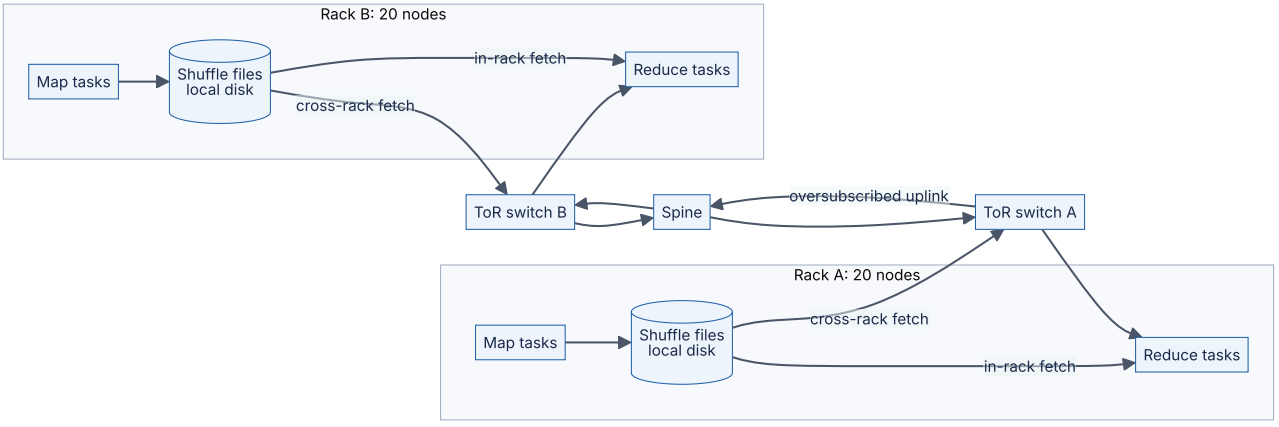
29.6.2 The cost of a shuffle

DEFINITION

Definition — Shuffle. The all-to-all redistribution of intermediate data between two stages, in which every task of the producing stage writes its output partitioned by key and every task of the consuming stage reads its partition from every producer. It costs serialization, local disk writes and reads, and network transfer, and it is a barrier between stages.

On a cluster of N nodes with keys spread evenly, a fraction $(N-1)/N$ of the shuffled data leaves its node. In a datacenter built from racks behind oversubscribed uplinks (§7.17), a fraction $(K-1)/K$ of it also leaves its rack when there are K racks, and the rack uplinks are usually the tightest pipe.

Figure 29.9 — A shuffle crossing rack uplinks.



Reading the diagram. Map tasks in each rack write partitioned *shuffle files* to local disk. Reduce tasks fetch their partition from every file: fetches within a rack go through the top-of-rack switch only, while cross-rack fetches climb through the *ToR switch*, an *oversubscribed uplink* and the *spine*. All fetches are pull-based TCP transfers that start as soon as producers finish. Shuffle files are on unreplicated local disks. Failure points: a congested uplink slows every reducer; a lost node invalidates its files. Scaling: more partitions create $M \times R$ ever-smaller blocks and random I/O.

FORMULA

Shuffle time lower bound on a rack-structured cluster

$$T_{\text{shuffle}} \approx \max\left(\frac{V(N-1)/N}{N\eta B_{\text{NIC}}},\ \frac{V(K-1)/K}{KU},\ \frac{2V}{NB_{\text{disk}}}\right)$$

Symbol	Meaning	Units
V	Shuffled data volume	bytes
N, K	Nodes and racks	count
B_{NIC}	Network interface speed per node	bytes/s
η	Achievable fraction of line rate	dimensionless
U	Uplink capacity per rack	bytes/s
B_{disk}	Local disk throughput per node (each byte written once, read once)	bytes/s

WORKED EXAMPLE

Worked example — Shuffling 10 TB on a 100-node cluster. $V = 10$ TB; $N = 100$ in $K = 5$ racks of 20; 25 Gb/s NICs ($B_{\text{NIC}} = 3.125$ GB/s) at $\eta = 0.7$; rack uplinks oversubscribed 4:1, so $U = 20 \times 3.125/4 = 15.6$ GB/s; NVMe $B_{\text{disk}} = 2$ GB/s.

1. NIC: each node sends $10\ \text{TB}/100 \times 0.99 = 99$ GB at $0.7 \times 3.125 = 2.19$ GB/s: 45 s.
2. Uplinks: cross-rack data $= 10 \times 0.8 = 8$ TB, so each rack sends 1.6 TB through 15.6 GB/s: 102 s.
3. Disk: each node writes 100 GB and reads 100 GB at 2 GB/s: 100 s.
4. Lower bound: $\max(45, 102, 100) \approx 102$ s, about 1.7 minutes; serialization, compression and stragglers typically add 2–3× in practice.
5. Block count: with 128 MB input splits from 10 TB of input, $M \approx 78{,}000$ map tasks; with $R = 2{,}000$ reducers there are 1.56×10^8 shuffle blocks averaging $10^{13}/1.56 \times 10^8 = 64$ KB each.

Interpretation. Moving 10 TB is dominated by oversubscribed uplinks and local disks, not NICs; and over a hundred million 64 KB random reads would cripple hard disks. **Engineering implication.** Shrink V first (filter and project before the shuffle, pre-aggregate, compress), then avoid it (broadcast joins, co-partitioned or bucketed inputs), and keep partitions around 100–200 MB so blocks stay large.

INDUSTRY EXAMPLE

LinkedIn Magnet: push-based shuffle. LinkedIn found that pull-based shuffle on hard disks produced huge numbers of small random reads and frequent fetch failures. In Magnet, map tasks *push* blocks to remote shuffle services that merge each reduce partition into larger files, turning random reads into sequential ones and keeping a second copy. It became Spark's push-based shuffle for YARN in Spark 3.2. *Evidence: Primary* — [Shen et al. 2020; Apache Spark Docs: Configuration]

29.6.3 Skew and salting

DEFINITION

Definition — Partition skew and salting. Skew is an uneven distribution of records across partitions, usually caused by a few very frequent keys (a celebrity user, a null or default value, a bot). **Salting** appends a random suffix from a small range to hot keys so their records spread over several partitions, then removes the suffix in a second, much smaller aggregation (or, for joins, replicates the matching rows of the other side once per salt value).

A stage finishes only when its slowest task finishes, so the largest partition, not the average, sets the stage time.

FORMULA

Slowdown from a single hot key

$$\text{skew} = \frac{\max_i L_i}{\bar{L}} \approx fP + (1 - f), \qquad \text{with salt } s\text{: skew} \approx \frac{fP}{s} + (1 - f)$$

Symbol	Meaning	Units
L_i, $\bar{L}$	Records in partition i; mean records per partition	records
f	Fraction of all records carrying the hot key	dimensionless
P	Number of shuffle partitions	count
s	Salt range (hot key spread over s partitions), $s \le P$	count

WORKED EXAMPLE

Worked example — A bot account with 20% of the events. $P = 200$, $f = 0.2$, and an average task takes 30 s.

1. Unsalted: $\text{skew} = 0.2 \times 200 + 0.8 = 40.8$. The hot partition takes $40.8 \times 30 = 1,224$ s (20 minutes) while 199 tasks finish in 30 s.
2. Salt $s = 8$: $40/8 + 0.8 = 5.8$, so 174 s.
3. Salt $s = 64$: $40/64 + 0.8 = 1.4$ in theory; the simulation below measures 2.1 because 64 salted keys hashed into 200 partitions collide.

Interpretation. Adding partitions does not help: one key goes to one partition however many there are, and fP even grows with P. **Engineering implication.** Find hot keys by sampling, salt only those, and let AQE's skew-join handling split oversized join partitions; filter out junk keys (nulls, test accounts) before the shuffle.

```
        """Partition skew and key salting: per-partition load before and after a two-
stage aggregation."""
        import random
        import zlib
        from collections import Counter


        def partition(key, n):
            return zlib.crc32(repr(key).encode()) % n


        def stage_loads(keys, n):
            loads = Counter(partition(k, n) for k in keys)
            return [loads.get(p, 0) for p in range(n)]


        def skew_ratio(loads):
            return max(loads) / (sum(loads) / len(loads))     # slowest task / average
task


        rng = random.Random(1)
        P = 200                                               # shuffle partitions
        records = [("hot" if rng.random() < 0.20 else f"user{rng.randrange(100_000)}",
1)
                   for _ in range(1_000_000)]                 # one key carries 20% of
all rows
        truth = Counter()
        for k, v in records:
            truth[k] += v

        # Plain aggregation: every row of a key goes to one partition.
        plain = stage_loads((k for k, _ in records), P)
        print(f"plain  : max={max(plain):>7,}  mean={sum(plain) / P:>7,.0f}  skew=
{skew_ratio(plain):5.1f}x")

        sample = Counter(k for k, _ in rng.sample(records, 10_000))   # find hot keys
from a 1% sample
        hot = {k for k, c in sample.items() if c > 0.01 * 10_000}
        print("hot keys found by sampling:", hot)

        for salt in (8, 64):
            # Stage 1: append a random salt to hot keys only, spreading each over `salt`
partitions.
            salted = [((k, rng.randrange(salt) if k in hot else 0), v) for k, v in
records]
            s1 = stage_loads((sk for sk, _ in salted), P)
            partial = Counter()
            for (k, s), v in salted:
                partial[(k, s)] += v                          # partial sums per (key,
salt)
            # Stage 2: strip the salt and combine at most `salt` partial sums per key.
            s2 = stage_loads((k for k, _ in partial), P)
            final = Counter()
            for (k, _), v in partial.items():
                final[k] += v
```

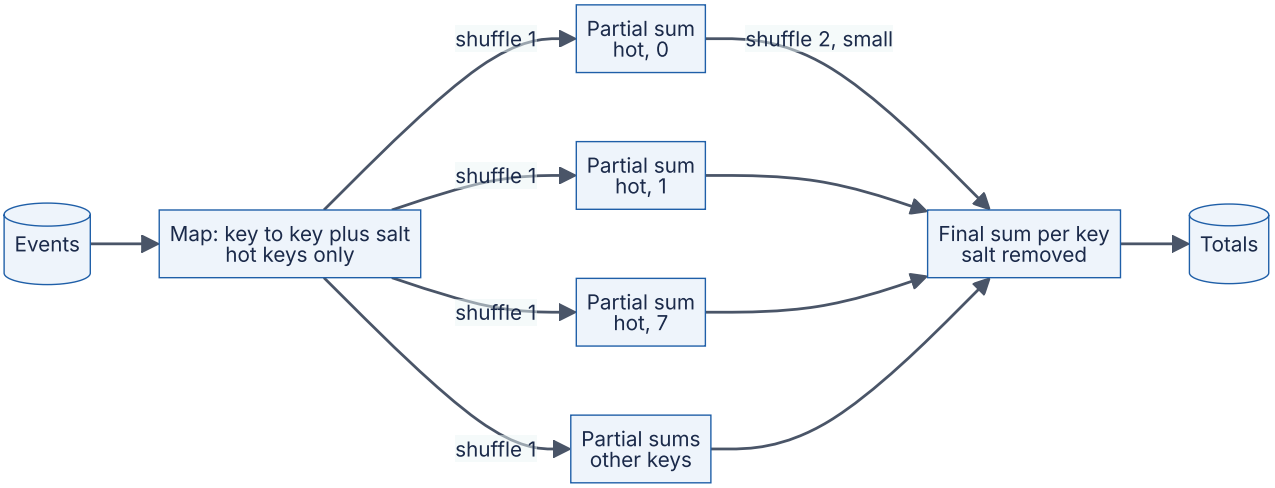
```
            assert final == truth                                  # salting never changes the
answer
            print(f"salt={salt:<3}: stage1 max={max(s1):>7,} skew={skew_ratio(s1):5.1f}x
| "
                  f"stage2 rows={len(partial):,} skew={skew_ratio(s2):4.1f}x")
```

Output:

```
plain  : max=204,210  mean=  5,000  skew= 40.8x
hot keys found by sampling: {'hot'}
salt=8  : stage1 max= 29,459 skew=  5.9x | stage2 rows=99,975 skew= 1.1x
salt=64 : stage1 max= 10,306 skew=  2.1x | stage2 rows=100,031 skew= 1.1x
```

What to notice: the measured unsalted skew (40.8×) matches the formula exactly; stage 2 handles only about 100,000 partial sums, so its cost is negligible; and the assertion proves the two-stage aggregation gives the same totals.

Figure 29.10 — Two-stage aggregation with a salted hot key.



Reading the diagram. The first map step appends a salt (0–7) to the hot key only; the first shuffle spreads `hot` over eight partitions, each computing a *partial sum*, while other keys aggregate normally. The second shuffle carries at most eight rows per hot key, so it is tiny; the final stage removes the salt and adds the partial sums. Both shuffles are asynchronous all-to-all transfers. Failure point: salting a non-associative operation (a median) gives wrong answers. Scaling: choose s so each salted piece is near the average partition size.

29.7 Data locality

DEFINITION

Definition — Data locality. Scheduling a task on or near the node that stores its input, so that the input is read from local disk or memory instead of over the network. Schedulers rank placements as **process-local** (data cached in the same executor), **node-local**, **rack-local** and **off-rack** (any).

In 2004 the network was the scarcest resource: 1 Gb/s NICs behind oversubscribed rack uplinks, while local disks delivered a few hundred MB/s per node. Moving computation to data was essential: GFS and HDFS expose block locations (§15.5), the MapReduce coordinator tried node-local then rack-local placement, and most input in large jobs was read locally [Dean and Ghemawat 2004]. Spark waits briefly for a better locality level before accepting a worse one (`spark.locality.wait`, 3 s by default), a technique called **delay scheduling** [Zaharia et al. 2010].

Two changes reduced locality's importance. Networks moved to 25–100 Gb/s NICs and fabrics with low oversubscription (§7.17), so remote reads approach local NVMe speed. And cloud deployments **disaggregated storage from compute**: data lives in object stores (§15.4) or shared file systems such as Colossus or Tectonic (§15.5.3), while stateless compute clusters scale independently, even to zero. Parquet decoding often becomes the bottleneck before the network does.

WORKED EXAMPLE

Worked example — How much locality is worth, then and now. Each node must read 100 GB of input.

1. 2004-style node: local disks 200 MB/s, effective remote rate about 25 MB/s (125 MB/s NIC behind 5:1 rack oversubscription). Fully local: $100,000/200 = 500$ s. Fully remote: $100,000/25 = 4,000$ s, 8× slower.
2. 2026-style node: local NVMe 6 GB/s; 100 Gb/s NIC delivering about 3 GB/s from an object store with parallel range reads. Local: $100/6 = 17$ s; remote: $100/3 = 33$ s, 2× slower, before columnar pruning, which applies equally to both.

Interpretation. Locality shrank from an 8× factor to 2×, which elasticity usually outweighs. **Engineering implication.** Keep columnar tables in object storage, cache hot data on local SSDs, and tune shuffle and skew instead; the shuffle remains node-to-node.

Object stores bring their own limits: per-prefix request rates (Amazon S3 documents at least 3,500 writes and 5,500 reads per second per prefix, as of 2026), slow listing, and re-

name-as-copy. Table formats such as Apache Iceberg, Delta Lake and Apache Hudi therefore track files in metadata and commit by swapping a metadata pointer (Chapter 61).

29.8 Stream processing concepts

29.8.1 Unbounded data, event time and processing time

DEFINITION

Definition — Stream processing, event time and processing time. Stream processing runs a long-lived computation over an **unbounded** dataset, one that grows forever, producing results incrementally. Each record has an **event time**, when the event actually happened (stamped by the device or service that produced it), and a **processing time**, when the system observes it. The difference is the record's **event-time skew**, which is variable and unbounded in the worst case.

First-principles explanation

Picture postcards from a group holiday. Each card shows the date it was written (event time), but they arrive in your mailbox days apart and out of order (processing time); one is stuck in a mailbag for a month. If you want "how many cards were written on Tuesday", you cannot answer when the first Wednesday card arrives; you have to decide how long to wait for stragglers, and what to do if a Tuesday card turns up after you announced the answer. Every streaming system makes exactly those two decisions.

Why does this exist?

Fraud scores, live dashboards and ML features over the last five minutes (Chapter 62) need answers continuously, and rerunning batch jobs every few minutes is wasteful. Grouping by processing time is easy but wrong: a phone that reconnects after a flight delivers yesterday's events today, and a consumer that restarts reprocesses old events in a burst, so processing-time counts change with operational accidents. Correct results require grouping by **event time** while data arrives out of order.

29.8.2 Windows

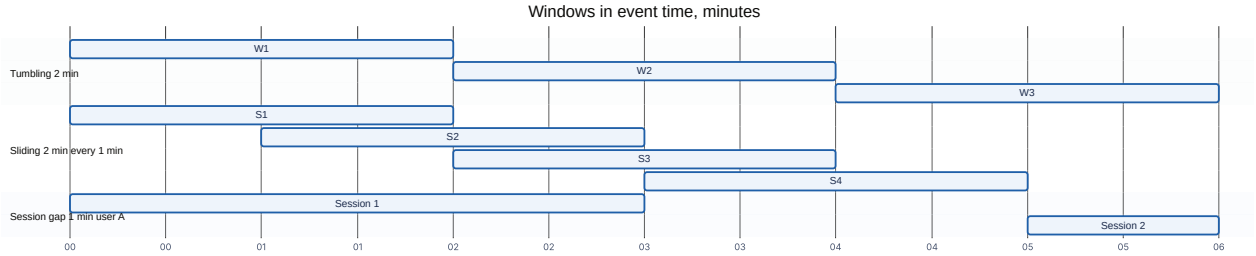
A **window** cuts an unbounded stream into finite chunks of event time so that aggregates can be computed and emitted.

- **Tumbling (fixed) windows** have size W and do not overlap: [12:00, 12:01), [12:01, 12:02). Each event belongs to exactly one.
- **Sliding (hopping) windows** have size W and slide $S < W$, so they overlap and each event belongs to W/S windows ("last 10 minutes, updated every minute" puts each

event in 10 windows, multiplying state and output).

- **Session windows** are per key and data-driven: a session continues while events arrive within a **gap** g of each other and closes after g of inactivity. Windows are merged when a new event bridges two of them.

Figure 29.11 — Tumbling, sliding and session windows over the same events.



Reading the diagram. The horizontal axis is event time in minutes. *Tumbling* windows tile the axis without overlap. *Sliding* windows of two minutes start every minute, so any instant is covered by two windows. *Session* windows belong to one key (user A): events from minute 0 to minute 3 arrived less than a minute apart and form one session, then a gap of over a minute closes it, and a new session starts at minute 5. The engine keeps state per key per open window, so sliding windows multiply state by W/S. Failure point: a bot sending every 30 s keeps a session open forever unless capped.

29.8.3 Watermarks, triggers, late data and accumulation

DEFINITION

Definition — Watermark. A watermark with value t is a statement flowing through the stream: "no more events with event time $\leq t$ are expected." It lets event-time operators decide that a window is complete. A **perfect** watermark is possible only with full knowledge of the sources; real systems use **heuristic** watermarks, such as the maximum event time seen minus a bound δ on out-of-orderness, which can be wrong. Events that arrive with event time behind the watermark are **late data**.

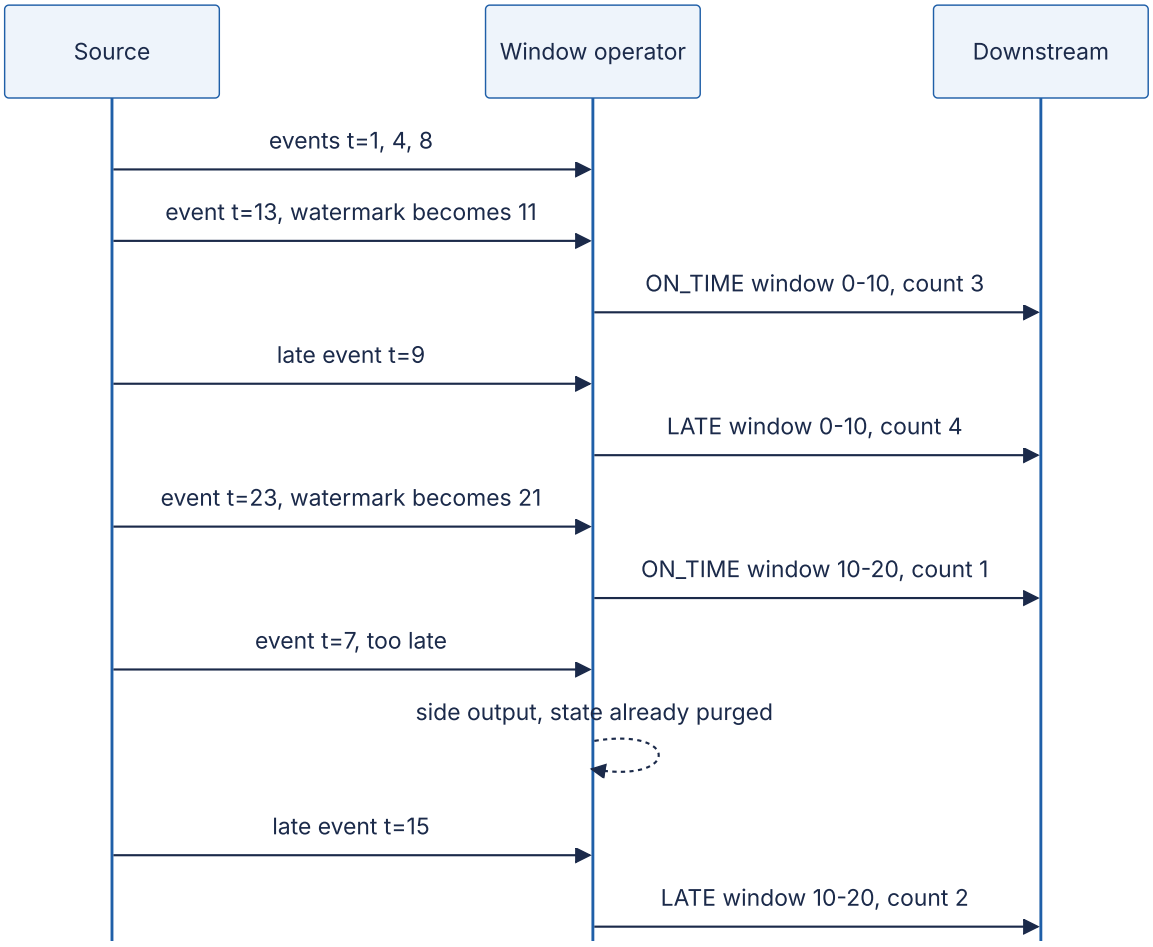
The Dataflow Model, published by Google and the basis of Google Cloud Dataflow and Apache Beam, separates four questions [Akidau et al. 2015]:

1. **What** results are computed? The transformation: sums, joins, model features.
2. **Where** in event time are they grouped? The windowing.
3. **When** in processing time are results emitted? **Triggers**: at the watermark (the on-time result), optionally early (speculative results every minute before the watermark) and late (an update for each late event).
4. **How** do successive results for one window relate? **Accumulation mode**: *discarding* (only new data), *accumulating* (the full total so far) or *accumulating and retracting*

(also withdrawing the previous value).

Allowed lateness bounds how long a window's state is kept after the watermark passes its end. Late events within it update the result; events beyond it are dropped or sent to a **side output** for separate handling. Without such a bound, state for every window ever opened would be kept forever.

Figure 29.12 — A 10 s window with a watermark, a late update and a dropped event.



Reading the diagram. This is the scenario tested by the simulator below: 10 s tumbling windows, watermark = maximum event time − 2 s, allowed lateness 5 s, accumulating mode. Event 13 moves the watermark to 11, past the end of [0, 10), so the *operator* emits its on-time count. Event 9 is late but within allowed lateness, so it triggers an update. At watermark 21 the state for [0, 10) is purged ($21 \geq 10 + 5$), so event 7 goes to the *side output*. Emissions are asynchronous. Failure point: a consumer that adds every emission would double-count; with accumulating mode it must upsert per window.

Mathematical foundations

FORMULA

Fraction of events that miss their tumbling window

$$f_{\text{late}} \approx \frac{\mu}{W} e^{-\delta/\mu}\left(1 - e^{-W/\mu}\right) \qquad \text{added output latency} \approx \delta$$

Derivation: with exponentially distributed delay d (mean μ) and the watermark close to arrival time minus δ, an event at offset u before its window's end misses the window if $d > \delta + u$; averaging $e^{-(\delta+u)/\mu}$ over u uniform on $[0, W)$ gives the formula.

Symbol	Meaning	Units
f_{late}	Fraction of events arriving after their window fired	dimensionless
μ	Mean delay between event time and arrival	s
δ	Watermark delay (bounded out-of-orderness)	s
W	Window size	s

WORKED EXAMPLE

Worked example — Choosing a watermark delay. Mobile click events have mean delay $\mu = 2$ s; windows are $W = 60$ s.

1. $\delta = 0$: $f = (2/60)(1)(1 - e^{-30}) = 0.0333$, so 3.3% of events miss their window.
2. $\delta = 5$ s: $e^{-2.5} = 0.0821$; $f = 0.0333 \times 0.0821 = 0.0027$ (0.27%), and every result is emitted 5 s later.
3. $\delta = 10$ s: $e^{-5} = 0.0067$; $f = 0.022\%$, for 10 s of extra latency.

Interpretation. Each extra μ of waiting cuts late data by $e \approx 2.7\times$, and costs that much latency on *every* result. Real delay distributions have heavy tails (phones offline for hours), so no practical δ makes f zero. **Engineering implication.** Pick δ for the latency the consumer can accept, set allowed lateness to absorb the tail (with late firings as updates), route anything later to a side output, and monitor the late fraction as a data-quality metric.

Code example

The simulator implements tumbling and sliding event-time windows with a bounded-out-of-orderness watermark, allowed lateness, on-time and late firings, state garbage collection and a side output. It checks the scenario of Figure 29.12, sliding-window member-

ship, and that no event is ever lost silently, then measures the late fraction for several watermark delays against the formula.

```
        """Event-time windowing with a bounded-out-of-orderness watermark, allowed late-
ness,
        on-time and late firings, and a side output for events that arrive too late."""
        import math
        import random


        class EventTimeWindower:
            def __init__(self, size, slide=None, max_out_of_orderness=0.0,
allowed_lateness=0.0):
                self.size, self.slide = size, slide or size        # slide == size ->
tumbling windows
                self.delta, self.lateness = max_out_of_orderness, allowed_lateness
                self.max_ts = -math.inf
                self.state = {}             # keyed window state: (key, window_start) ->
count
                self.fired = set()          # windows that already emitted their on-time
result
                self.output = []            # (kind, key, start, end, count, emitted_at)
                self.dropped = []           # side output: (event_time, key,
window_start)

            @property
            def watermark(self):            # "no more events older than this are ex-
pected"
                return self.max_ts - self.delta

            def window_starts(self, ts):
                start = math.floor(ts / self.slide) * self.slide
                starts = []
                while start > ts - self.size:                       # every window [s,
s+size) holding ts
                    starts.append(start)
                    start -= self.slide
                return starts

            def on_event(self, ts, key, now):
                for start in self.window_starts(ts):
                    w = (key, start)
                    if self.watermark >= start + self.size + self.lateness:
                        self.dropped.append((ts, key, start))       # state already
purged: too late
                        continue
                    self.state[w] = self.state.get(w, 0) + 1
                    if w in self.fired:                             # late, but within
allowed lateness
                        self.output.append(("LATE", key, start, start + self.size,
self.state[w], now))
                self.max_ts = max(self.max_ts, ts)
                self._on_watermark(now)

            def _on_watermark(self, now):
                wm = self.watermark
                for w in sorted(self.state):
                    key, start = w
                    end = start + self.size
```

```
                    if wm >= end and w not in self.fired:           # event-time trigger
                        self.fired.add(w)
                        self.output.append(("ON_TIME", key, start, end, self.state[w],
now))
                    if wm >= end + self.lateness:                   # garbage-collect
window state
                        del self.state[w]
                        self.fired.discard(w)

            def end_of_input(self, now):
                self.max_ts = math.inf                              # final watermark
flushes everything
                self._on_watermark(now)


        def run(events, **cfg):
            """events: (event_time, key, arrival_time); processed in arrival order."""
            op = EventTimeWindower(**cfg)
            for ts, key, arrival in sorted(events, key=lambda e: e[2]):
                op.on_event(ts, key, arrival)
            op.end_of_input(max(e[2] for e in events))
            return op


        def late_fraction_model(mean_delay, delta, size):
            """P(event misses its tumbling window) for exponential delays, uniform posi-
tion in window."""
            return (mean_delay / size) * math.exp(-delta / mean_delay) * (1 - math.exp(-
size / mean_delay))


        if __name__ == "__main__":
            # Scenario: 10 s tumbling windows, watermark 2 s behind max event time, 5 s
allowed lateness.
            ev = [(1, "a", 1.2), (4, "a", 4.1), (8, "a", 8.3), (13, "a", 13.0),   # wm
reaches 11 -> fire [0,10)
                  (9, "a", 13.5),                                                 #
late, within lateness
                  (23, "a", 23.0),                                                # wm
21 -> fire [10,20)
                  (7, "a", 23.4),                                                 # wm
21 >= 10+5 -> dropped
                  (15, "a", 23.6)]                                                # late
update for [10,20)
            op = run(ev, size=10, max_out_of_orderness=2, allowed_lateness=5)
            for row in op.output:
                print(row)
            print("dropped:", op.dropped)
            assert op.output[0] == ("ON_TIME", "a", 0, 10, 3, 13.0)
            assert op.output[1] == ("LATE", "a", 0, 10, 4, 13.5)
            assert ("ON_TIME", "a", 10, 20, 1, 23.0) in op.output
            assert ("LATE", "a", 10, 20, 2, 23.6) in op.output
            assert op.dropped == [(7, "a", 0)]

            # Sliding windows: size 10, slide 5 -> each event lands in two windows.
            sl = run([(7, "k", 7), (12, "k", 12), (40, "k", 40)], size=10, slide=5)
            assert [(r[2], r[4]) for r in sl.output if r[0] == "ON_TIME"][:3] == [(0,
```

```
1), (5, 2), (10, 1)]

            # Simulation: 50 events/s for 2 h, exponential network delay (mean 2 s), 60
s windows.
            rng = random.Random(42)
            events, t = [], 0.0
            while t < 7200:
                t += rng.expovariate(50)
                events.append((t, "clicks", t + rng.expovariate(1 / 2.0)))
            print(f"{'delta s':>7} {'late model':>10} {'late sim':>9} {'emit delay
s':>12}")
            for delta in (0, 2, 5, 10):
                op = run(events, size=60, max_out_of_orderness=delta,
allowed_lateness=0)
                sim = len(op.dropped) / len(events)
                counted = sum(r[4] for r in op.output if r[0] == "ON_TIME")
                assert counted + len(op.dropped) == len(events)            # nothing
lost silently
                on_time = [r for r in op.output if r[0] == "ON_TIME" and r[3] < 7200]
                lag = sum(r[5] - r[3] for r in on_time) / len(on_time)      # emission
time - window end
                print(f"{delta:>7} {late_fraction_model(2.0, delta, 60):>10.4%}
{sim:>9.4%} {lag:>12.2f}")
            print("all tests passed")
```

Output:

```
('ON_TIME', 'a', 0, 10, 3, 13.0)
('LATE', 'a', 0, 10, 4, 13.5)
('ON_TIME', 'a', 10, 20, 1, 23.0)
('LATE', 'a', 10, 20, 2, 23.6)
('ON_TIME', 'a', 20, 30, 1, 23.6)
dropped: [(7, 'a', 0)]
delta s late model  late sim emit delay s
      0    3.3333%   2.9084%         0.25
      2    1.2263%   1.0811%         2.27
      5    0.2736%   0.2373%         5.25
     10    0.0225%   0.0172%        10.24
all tests passed
```

What to notice:

- The late check uses the watermark *before* the event advances it, so an event cannot rescue its own window.
- Window [20, 30) fires only at `end_of_input`, which sends a final infinite watermark; Flink does the same when a bounded source finishes.
- The simulation tracks the model closely but slightly below it: the maximum event time trails the arrival clock by about 0.25 s, adding that much extra slack (the “emit delay” column), and $e^{-0.25/2} = 0.88$ explains the gap.
- With $\delta = 0$ the operator is fast but loses about 3% of events; each extra 2 s of delay divides losses by about e.

29.8.4 State and exactly-once processing

Windows, joins, deduplication and per-user features all need **state** that outlives individual events: counts per open window, the last event per user, a model's running statistics. State is partitioned by key like a shuffle. After a failure it must be recovered *consistently with the input position*: restoring counts from 12:00 but resuming input from 12:05 loses five minutes; resuming from 11:55 double-counts five.

Engines achieve **exactly-once state semantics** (effectively-once) by periodically taking a **consistent distributed snapshot** of all operator state plus source offsets and, on failure, rolling back to it and replaying input from those offsets. Events may be *processed* twice, but each affects state once. The classic algorithm is Chandy and Lamport's: a **marker** travels along every channel; a process records its state at the first marker and records in-flight messages on channels whose markers have not arrived, yielding a consistent global state without stopping the system [Chandy and Lamport 1985]. Flink's adaptation is in §29.9.2. External results are exactly-once only with idempotent or transactional sinks (§32.9; Kafka transactions in Chapter 31).

INDUSTRY EXAMPLE

Google MillWheel and Cloud Dataflow. MillWheel, Google's earlier stream processor, introduced low watermarks computed from the oldest pending work, per-key persistent state, and exactly-once delivery by deduplicating record IDs [Akidau et al. 2013]. The Dataflow Model generalized these ideas and underlies Google Cloud Dataflow; Apache Beam is its open-source SDK [Akidau et al. 2015]. *Evidence: Primary* — [Akidau et al. 2013; Akidau et al. 2015]

COMMON MISTAKE

"Exactly-once means every event is processed exactly once." It means the *effect on managed state* is as if each event were processed once. After a failure, events since the last checkpoint are reprocessed, and any non-transactional side effect (an email, an HTTP call, a write to a database without an idempotency key) happens again.

29.9 Apache Flink

29.9.1 Architecture and state

DEFINITION

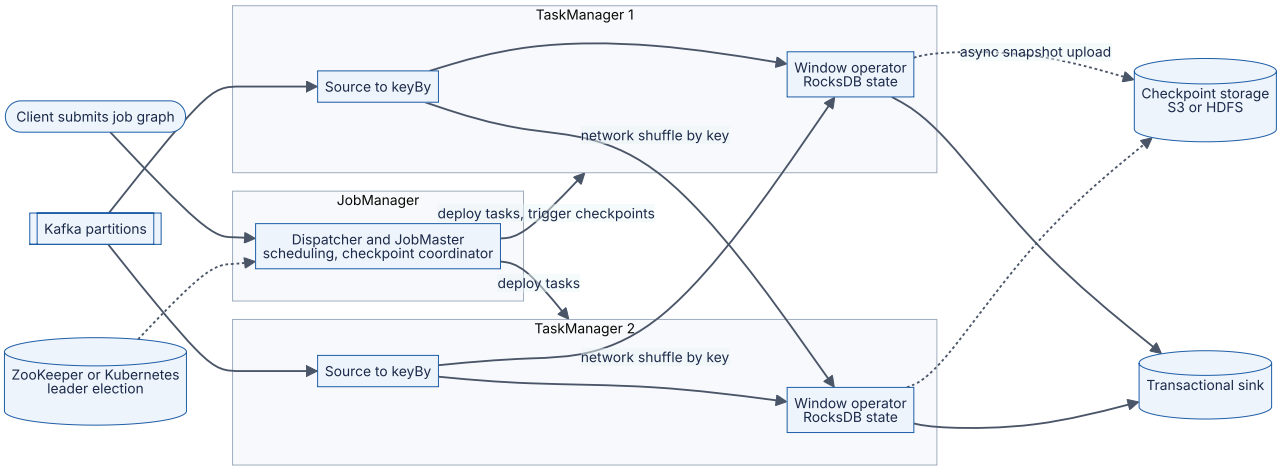
Definition — Apache Flink. An open-source distributed engine that executes dataflow programs as a graph of long-running, pipelined operators, treating batch as a bounded special case of streaming. It provides event-time processing with watermarks, large keyed state, and exactly-once state consistency through asynchronous distributed snapshots [Carbone et al. 2015a].

A Flink cluster has a **JobManager** (a resource manager, a dispatcher, and one **JobMaster** per job that schedules tasks and coordinates checkpoints) and several **TaskManagers**, worker processes that each offer a fixed number of **task slots**. A job graph is turned into parallel subtasks; consecutive operators with the same parallelism and a one-to-one connection are **chained** into one task, so records pass between them as method calls. Unlike Spark's stage-at-a-time batch model, all stages of a streaming job run at once, and records flow continuously through network buffers between them, with **credit-based flow control** providing backpressure when a downstream task is slow (§22.10).

State comes in two kinds. **Keyed state** (value, list, map state) is scoped to the current key of a `keyBy` stream and partitioned with the stream; it is organized into **key groups**, whose number equals the job's **maximum parallelism**, so rescaling moves whole key groups between subtasks without rehashing keys. **Operator state** belongs to a parallel operator instance regardless of key; the classic example is a source's partition offsets.

State backends decide where working state lives. The **HashMapStateBackend** keeps state as heap objects: fastest, but bounded by memory and GC pressure. The **EmbeddedRocksDBStateBackend** keeps state in an embedded RocksDB LSM tree (§13.8) on local disk, serialized, so state can exceed memory by orders of magnitude at the cost of serialization on every access; it supports **incremental checkpoints** that upload only new SST files. Flink 2.0 (2025) added a disaggregated state backend, ForSt, that keeps the primary copy of state in remote storage (as of 2026). Checkpoints go to durable storage such as S3 or HDFS [Apache Flink Docs: State Backends; Carbone et al. 2017].

Figure 29.13 — Flink cluster architecture.



Reading the diagram. The *client* submits a job graph to the *JobManager*, whose JobMaster deploys tasks into *TaskManager* slots and coordinates checkpoints. Sources read *Kafka partitions* and re-partition records by key to the *window operators*, which keep state in RocksDB on local disk. Solid arrows are pipelined record streams with back-pressure; dotted arrows are asynchronous snapshot uploads and leader election for high availability. Failure points: a lost TaskManager restarts tasks from the last checkpoint; a lost JobManager fails over to a standby. Scaling: restart from a savepoint with higher parallelism.

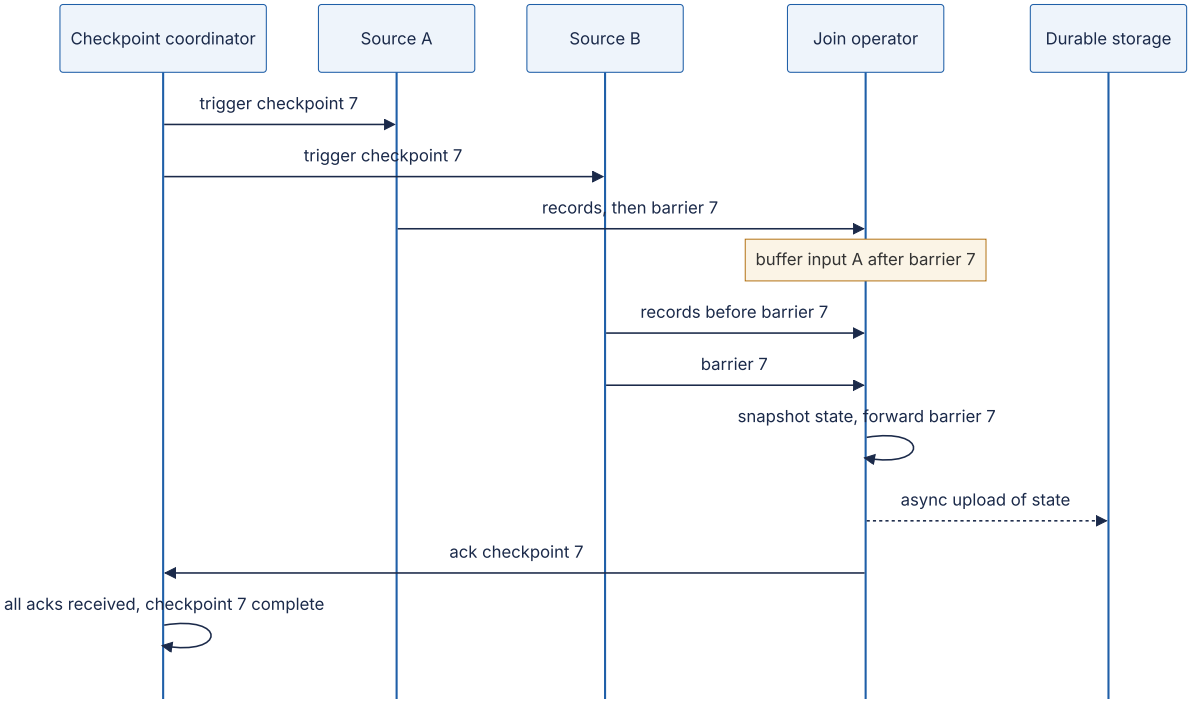
29.9.2 Asynchronous barrier snapshotting and savepoints

Flink's checkpointing is a variant of Chandy–Lamport for directed dataflow graphs, called **asynchronous barrier snapshotting** [Carbone et al. 2015b]:

1. The checkpoint coordinator tells every source to start checkpoint n. Each source records its input offset and emits a **barrier** n into its output streams, between records.
2. An operator with several inputs **aligns**: once barrier n arrives on one input, it buffers further records from that input until barrier n has arrived on all inputs. Records before the barrier belong to checkpoint n; records after it to $n+1$.
3. When aligned, the operator takes a snapshot of its state and forwards barrier n downstream. The snapshot is **asynchronous**: a quick copy-on-write or RocksDB snapshot is taken synchronously, and the upload to durable storage happens in the background while processing continues.
4. Sinks acknowledge; when every task has acknowledged, the coordinator marks checkpoint n complete. Transactional sinks then commit the output written during that checkpoint (two-phase commit), which is how Flink achieves end-to-end exactly-once with Kafka.
5. On failure, all tasks restart, load their state from checkpoint n, and sources rewind to the recorded offsets.

For acyclic graphs, alignment means in-flight channel contents need not be saved, unlike in Chandy–Lamport. Under heavy backpressure, barriers queue behind buffered data; **unaligned checkpoints** (Flink 1.11+) let barriers overtake buffered records and save those buffers in the snapshot.

Figure 29.14 — Barrier alignment in a two-input operator.



Reading the diagram. The *coordinator* triggers checkpoint 7 at both *sources*, which record their offsets and inject barriers. Barrier 7 reaches the *join operator* first from source A, so the operator pauses input A (buffering) while it keeps consuming B until B's barrier arrives. Then it snapshots state, forwards the barrier, uploads state asynchronously (dotted) to *durable storage* and acknowledges. Failure points: if any task fails before acknowledging, checkpoint 7 is abandoned and recovery uses checkpoint 6. Scaling: alignment time grows with backpressure and skew between inputs, which is what unaligned checkpoints address.

A **savepoint** is a user-triggered checkpoint in a canonical format that is never deleted automatically. To upgrade code, change parallelism or migrate clusters, stop with a savepoint, deploy the new version and resume from it; operators need stable IDs (`uid`) so state maps to the new graph.

FORMULA

Checkpoint interval: overhead versus recovery

$$T_{\text{opt}} \approx \sqrt{2CM_f}, \qquad T_{\text{catch-up}} \approx \frac{\lambda(T/2 + T_{\text{restart}} + T_{\text{restore}})}{\mu - \lambda}$$

Symbol	Meaning	Units
T, T_{opt}	Checkpoint interval; interval minimizing total lost time (Young's approximation)	s
C	Throughput-equivalent cost of one checkpoint	s
M_f	Mean time between failures of the job	s
λ	Input rate	events/s
μ	Maximum processing rate of the job	events/s
T_{restart}, T_{restore}	Time to reschedule tasks; time to download state	s
$T_{\text{catch-up}}$	Time to work off the backlog after restoring	s

WORKED EXAMPLE

Worked example — A fraud-feature job with 2 TB of state. $\lambda = 200,000$ events/s, $\mu = 500,000$ events/s; state is 2 TB over 50 TaskManagers (40 GB each), restored at 1 GB/s per node, so $T_{\text{restore}} = 40$ s; $T_{\text{restart}} = 20$ s; the job fails about once a day ($M_f = 86,400$ s); each asynchronous checkpoint costs about $C = 2$ s of throughput.

1. Young's interval: $\sqrt{2 \times 2 \times 86,400} = \sqrt{345,600} = 588$ s, about 10 minutes.
2. Catch-up at $T = 600$ s: backlog $= 200,000 \times (300 + 20 + 40) = 7.2 \times 10^7$ events; $7.2 \times 10^7 / 300,000 = 240$ s. Results are stale for $20 + 40 + 240 = 300$ s.
3. At $T = 60$ s: backlog $= 200,000 \times 90 = 1.8 \times 10^7$; catch-up 60 s; stale for 120 s. Overhead rises to $2/60 = 3.3\%$ of throughput, from 0.3%.
4. With a transactional sink, output is visible only at checkpoint completion, so $T = 600$ s adds up to 10 minutes of latency.

Interpretation. Young's formula ignores two streaming costs: catch-up, which grows with T and explodes as $\lambda \to \mu$, and sink visibility latency, which equals T.
Engineering implication. Checkpoint every 10 s to a few minutes, keep $\mu \geq 2\lambda$, use incremental checkpoints and local recovery to cut T_{restore}, and alert when checkpoint duration nears the interval.

Failure scenarios

What goes wrong	Why	Detection	Recovery	Prevention
Checkpoints time out repeatedly	Backpressure delays barriers; slow uploads	Checkpoint duration and alignment metrics	Unaligned checkpoints; scale out the slow operator	Capacity headroom; incremental checkpoints
State grows without bound	Session windows or keys never expire	State size per checkpoint rising steadily	Restart from savepoint with state TTL	State TTL, allowed-lateness bounds
Watermark stops advancing	An idle source partition emits no events	Watermark lag metric flat	Mark idle sources (`withIdleness`)	Idleness timeouts on every source
Duplicates downstream after restart	Non-transactional sink replayed	Duplicate keys in sink	Deduplicate by event ID	Transactional or idempotent sink

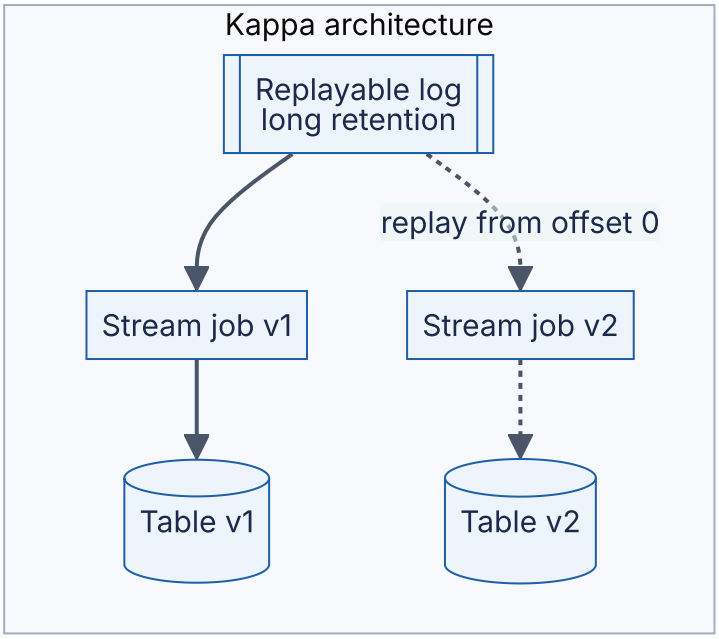
29.10 Lambda and Kappa architectures

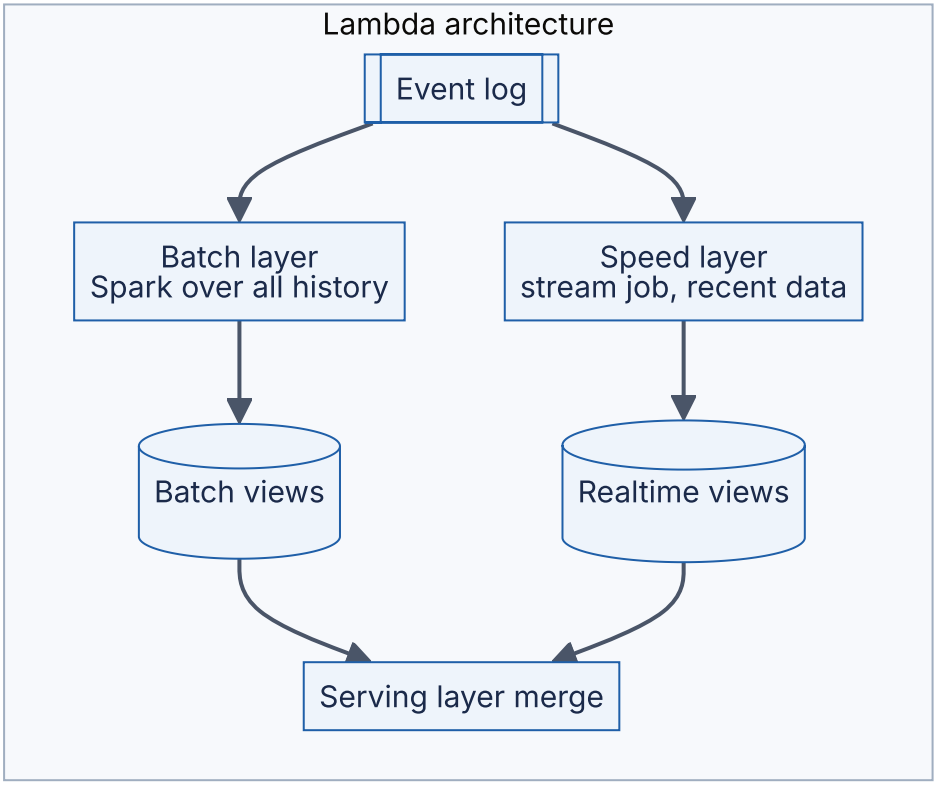
DEFINITION

Definition — Lambda and Kappa architectures. The **Lambda architecture** runs two parallel paths over the same immutable master dataset: a **batch layer** that periodically recomputes complete, accurate views from all history, and a **speed layer** that processes recent data incrementally with low latency; a **serving layer** merges the two at query time [Marz and Warren 2015]. The **Kappa architecture** uses a single stream-processing path over a replayable log; to change logic or fix a bug, it reprocesses history by replaying the log through a new version of the job [Kreps 2014].

Lambda arose when stream processors were approximate (early Storm offered at-least-once processing and no consistent state) while batch jobs were trustworthy but hours behind; batch recomputation healed the speed layer's errors. The cost, as Kreps argued, is maintaining **the same logic twice** in two frameworks and keeping them identical. Kappa removes the duplication once the stream engine is correct (exactly-once state, event time) and the log retains enough history: to deploy version 2, run a second job from the start of the log into a new table, let it catch up, switch readers, and delete version 1's output.

Figure 29.15 — Lambda versus Kappa.





Reading the diagram. In *Lambda*, one *event log* feeds a *batch layer* that recomputes *batch views* from all history every few hours and a *speed layer* that covers data since the last batch run; the *serving layer* merges both. In *Kappa*, one *stream job* consumes a *replayable log*; to change logic, *job v2* replays the log (dotted, asynchronous) into a new table and readers switch once it catches up. Storage: Lambda keeps history in files or objects; Kappa needs the log (Chapter 31) or tiered storage. Failure points: divergent logic

(Lambda); slow replay (Kappa). Scaling: replay parallelism is bounded by the log's partition count.

Replay speed decides whether Kappa is practical. Reprocessing 30 days of events at an average of 100,000 events/s means 2.6×10^{11} events; with 200 parallel tasks at 50,000 events/s each (10 million events/s), replay takes $2.6 \times 10^{11} / 10^{7} = 26,000$ s, about 7.2 hours, which is acceptable for a logic change but not for an urgent fix.

INDUSTRY EXAMPLE

Summingbird at Twitter and Kappa at LinkedIn. Twitter built Summingbird to reduce Lambda's double implementation: one Scala program is compiled to both a Hadoop batch job and a Storm streaming job, with results merged at query time [Boykin et al. 2014]. Kreps, then at LinkedIn, proposed instead reprocessing by replaying Kafka into a new job instance, which became known as the Kappa architecture [Kreps 2014]. *Evidence: Primary* — [Boykin et al. 2014; Kreps 2014]

Table 29.1 — Lambda and Kappa compared.

Aspect	Lambda	Kappa
What it is	Batch layer plus speed layer, merged at serving	One stream path; reprocess by replay
Code paths	Two, kept semantically equal	One
Correctness source	Batch recomputation heals stream errors	Exactly-once stream engine with event time
History store	Files or objects (cheap, columnar)	Log with long retention or tiered storage
Reprocessing	Rerun batch job	Replay log into a new job version
Operational complexity	High: two systems, merge logic	Medium: one system, replay capacity
Choose when	Heavy historical recomputation; stream engine cannot be trusted	Logic fits streaming; log retains enough history

In practice many teams now run a hybrid: one engine (Spark or Flink, or Beam over either) with shared code, writing streaming results into lakehouse tables that batch jobs also read and backfill (Chapters 61–62).

29.11 Batch versus stream; Spark versus Flink

Batch and streaming are two points on one spectrum: a batch job is a stream job over a bounded input whose watermark jumps to infinity at the end. The Dataflow Model and

Flink both treat them this way; Spark Structured Streaming does the reverse, running a stream as a series of small incremental batch queries [Armbrust et al. 2018].

Table 29.2 — Batch versus stream processing.

Aspect	Batch	Stream
What it is	Computation over a bounded, complete dataset	Continuous computation over an unbounded dataset
How it works	Read all input, stage by stage, write output	Long-running operators, state, watermarks, checkpoints
Advantages	Simple reasoning, easy reruns, cheap per byte	Seconds of freshness, incremental cost
Disadvantages	Hours of latency; recomputes unchanged data	Completeness is a heuristic; state and late data to manage
Performance	Throughput-optimized, minutes to hours	Latency of milliseconds to seconds
Operational complexity	Scheduler, retries, backfills	24×7 on-call, state migrations, lag monitoring
Cost	Pay per run; can use spot capacity	Always-on resources
Use cases	Reports, model training sets, backfills, index builds	Fraud, alerting, live features, dashboards
Choose when	Freshness of an hour or more is acceptable	Value decays within minutes

Table 29.3 — Apache Spark versus Apache Flink (as of 2026).

Aspect	Spark (incl. Structured Streaming)	Flink
What it is	Batch-first engine with SQL, ML and streaming	Stream-first engine; batch as bounded streams
How it works	Stages of tasks; streaming as micro-batches, plus experimental continuous mode	Pipelined long-running operators; continuous record flow
Advantages	Best-in-class batch SQL (Catalyst, AQE), huge ecosystem, Python support	Low latency, rich event-time and state APIs, large state on RocksDB
Disadvantages	Micro-batch latency floor; less flexible custom state and timers	Smaller batch and Python ecosystem; more operational tuning
Performance	Micro-batch latency of about 100 ms and up, exactly-once; continuous mode about 1 ms, at-least-once	Milliseconds, exactly-once state
Operational complexity	Lower for batch; streaming state stores simpler	Higher: checkpoints, savepoints, state backends
Cost	Efficient for large scans; can stop between runs	Always-on clusters sized for peak plus recovery headroom
Use cases	ETL, analytics, feature backfills, ML preprocessing	Fraud, CEP, real-time features, streaming ETL
Choose when	Batch dominates, latency of seconds is fine	Sub-second latency or complex event-time state

The micro-batch and continuous-mode figures are from the Spark documentation [Apache Spark Docs: Structured Streaming]. **Apache Beam** sits above both: a pipeline written once with the Dataflow Model's windowing, triggers and accumulation runs on a chosen **runner** (Google Cloud Dataflow, Flink, Spark), trading some engine-specific features for portability. Chapter 61 uses Beam and Spark for ML data pipelines.

29.12 Summary and revision notes

SUMMARY

- **Batch** processes bounded, immutable inputs for throughput; rerunning a task is safe. Scan time is $D/(Nb)$: shrink D before adding nodes.
- **MapReduce**: map → partition → sort/spill → shuffle → merge → reduce. A combiner pre-aggregates associative, commutative operations and can cut shuffle volume by 100×. Failures are handled by re-executing tasks; backup tasks tame stragglers, which appear in almost every large job ($1-(1-p)^n$).
- **Joins**: sort-merge shuffles both sides; broadcast copies the small side to every task and wins while $|S| < |L|/(N-1)$ and $|S|$ fits in memory.
- **Spark**: RDD lineage replaces replication; narrow dependencies pipeline within a stage, wide dependencies end it with a shuffle. Cache reused data. DataFrames let Catalyst prune, push down and choose joins; Tungsten generates code; AQE re-plans at shuffle boundaries (coalesce, switch to broadcast, split skew).
- **Shuffle** cost is set by the tightest of NIC, rack uplink and local disk; a 10 TB shuffle on 100 nodes needs about 100 s at best. A hot key with fraction f slows a stage by fP; salt hot keys and aggregate in two stages.
- **Locality** mattered when networks were slow; today compute and storage are disaggregated.
- **Streams**: group by event time; windows are tumbling, sliding or session; watermarks trade latency (δ) against late data ($\propto e^{-\delta/\mu}$); triggers and accumulation modes say when and how results are emitted; allowed lateness bounds state.
- **Flink**: JobManager and TaskManagers; keyed state in key groups; RocksDB for large state; asynchronous barrier snapshots give exactly-once state; savepoints enable upgrades and rescaling. Checkpoint interval trades overhead against catch-up time and sink latency.
- **Lambda** duplicates logic for freshness plus correctness; **Kappa** reprocesses by replay. Spark leads for batch, Flink for low-latency stateful streaming; Beam provides portability.

29.13 Knowledge checkpoint

1. Why do immutable inputs make batch fault tolerance simple?
2. List the steps between a map function’s output and a reduce function’s input.
3. When may the reduce function be used as a combiner?
4. A phase has 2,000 tasks and each is a straggler with probability 0.0005. What is the chance of at least one straggler?

5. Using the join-cost formula, should you broadcast a 1 GB table to 200 executors when the other side is 500 GB?
6. What is the difference between a narrow and a wide dependency, and why does it define stage boundaries?
7. What three things can Adaptive Query Execution change at run time?
8. A hot key carries 10% of rows over 400 partitions. What is the skew ratio, and what is it with a salt of 20?
9. Define event time, processing time and watermark.
10. For $\mu = 3$ s, $W = 60$ s and $\delta = 6$ s, what fraction of events misses its window?
11. What do allowed lateness and side outputs each protect against?
12. Why does a two-input Flink operator align barriers, and what do unaligned checkpoints change?
13. What is the main cost of Lambda that Kappa removes, and what does Kappa require instead?

Answers

1. A task's output depends only on its input, so rerunning it gives the same result; no undo or coordination is needed.
2. Partitioning by key, buffering, sorting by (partition, key), optional combining, spilling and merging on the map side; fetching (shuffle) and merge-sorting on the reduce side.
3. When the operation is associative and commutative and its input and output types match, so applying it zero or more times to partial groups does not change the result.
4. $1 - e^{-2,000 \times 0.0005} = 1 - e^{-1} = 0.63$.
5. Broadcast moves $1 \times 199 = 199$ GB; shuffle moves about $501 \times 0.995 = 499$ GB. Broadcast is cheaper if 1 GB fits comfortably in each executor's memory.
6. Narrow: each parent partition feeds at most one child partition, so operators pipeline in one task. Wide: a child needs many parents, requiring a shuffle and a barrier, so the scheduler starts a new stage.
7. It coalesces small shuffle partitions, converts a sort-merge join to a broadcast join, and splits skewed join partitions.
8. $0.1 \times 400 + 0.9 = 40.9$; with salt 20: $40/20 + 0.9 = 2.9$.
9. Event time: when the event happened. Processing time: when the system observes it. Watermark: an assertion that no events with earlier event time are expected.
10. $(3/60)\,e^{-2}(1 - e^{-20}) = 0.05 \times 0.135 = 0.68\%$.
11. Allowed lateness bounds state retention while still updating results for moderately late data; side outputs keep events that arrive later still from being silently lost.
12. So that the snapshot reflects exactly the records before the barrier on every input. Unaligned checkpoints let barriers overtake buffered records and save in-flight data, avoiding long alignment under backpressure.

13. Lambda maintains the same logic in two systems; Kappa needs a correct stream engine and a log retaining enough history to replay fast enough.

29.14 Interview questions

Basic

Q1. Explain MapReduce with word count. Answer. Map emits `(word, 1)` per word; a combiner sums locally; pairs are hash-partitioned by word, sorted and shuffled so every count for a word reaches one reducer, which sums them and writes the result. Failed tasks rerun because inputs are immutable.

Q2. Why is Spark faster than MapReduce for iterative jobs? Answer. MapReduce writes each job's output to replicated storage and rereads it; Spark keeps reused datasets in memory, pipelines narrow transformations within a stage, and recovers lost partitions from lineage rather than replicas. Catalyst and code generation add CPU efficiency for DataFrames.

Intermediate

Q3. A Spark job's last task runs for an hour while others take a minute. Diagnose and fix. Answer. Check the stage's task-duration and shuffle-read-size distributions. If one task reads far more data, it is key skew: find hot keys by sampling, filter junk keys, enable AQE skew-join handling, or salt and aggregate in two stages; broadcast the small side if one exists. If sizes are equal, it is a straggling node: enable speculation and check that machine.

Q4. Design per-minute click counts per page from mobile clients with delayed uploads. Answer. Key by page, one-minute tumbling event-time windows, a bounded-out-of-orderness watermark (for example 10 s from measured delay), allowed lateness of hours with accumulating late firings upserted by (page, window), a side output for later events, a nightly batch reconciliation if needed, Kafka source with checkpointed offsets, and an idempotent or transactional sink.

Advanced

Q5. How does Flink provide exactly-once results end to end with Kafka? Answer. Sources store Kafka offsets in checkpoints; barriers create consistent snapshots of all operator state; on failure, state and offsets roll back together. The Kafka sink writes inside a transaction per checkpoint, pre-commits on the barrier, and commits when the checkpoint completes; consumers read with `read_committed`. External side effects outside this protocol still need idempotency keys.

Q6. Estimate the shuffle time of a 20 TB join on 200 nodes with 25 Gb/s NICs, 10 racks and 4:1 oversubscription. Answer. NIC: about 100 GB per node at 2.19 GB/s ≈ 46 s. Uplinks (10 racks of 20): cross-rack $20 \times 0.9 = 18$ TB, 1.8 TB per rack through $20 \times 3.125/4 = 15.6$ GB/s ≈ 115 s. Disk (2 GB/s): 200 GB per node ≈ 100 s. Bound ≈ 2 minutes; expect 4–6 minutes. Then question the shuffle: can one side be broadcast or pre-bucketed?

29.15 Practical exercises

1. Extend the mini MapReduce engine with a range partitioner built from a sample, and verify that concatenating reducer outputs gives globally sorted keys.
2. Add speculative execution to the engine: give each map task a random duration with 1% stragglers at 5×, and measure job time with and without backup tasks.
3. Run the PySpark program with `spark.sql.autoBroadcastJoinThreshold=-1` and without the broadcast hint; compare the physical plans and count the `Exchange` operators.
4. Add session windows (gap g) to the windowing simulator, including merging two sessions when a late event bridges them.
5. Rerun the simulator with lognormal delays of the same mean and compare the late fraction with the exponential model.

29.16 Design challenges

1. **Daily training-set build.** 50 TB of events must be joined with a 200 GB user table and a 2 GB product table and aggregated into per-user features by 06:00 each day. Choose join strategies, partition counts and cluster size, and estimate run time and shuffle volume.
2. **Real-time fraud features.** Compute per-card counts and amounts over 1-minute, 1-hour and 24-hour sliding windows for 300,000 events/s with p99 freshness under 5 s. Choose the engine, state backend, watermark, checkpoint interval and sink, and size the state.

29.17 References

1. Akidau, T., et al. “MillWheel: Fault-Tolerant Stream Processing at Internet Scale.” *Proceedings of the VLDB Endowment* 6(11), 2013.
2. Akidau, T., et al. “The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing.” *Proceedings of the VLDB Endowment* 8(12), 2015.
3. Akidau, T., Chernyak, S., Lax, R. *Streaming Systems*. O’Reilly Media, 2018.

4. Apache Flink. *Documentation: Flink Architecture; State Backends; Checkpointing; Savepoints*. https://nightlies.apache.org/flink/
5. Apache Spark. *Documentation: Performance Tuning (Adaptive Query Execution); Configuration; RDD Programming Guide; Structured Streaming Programming Guide*. https://spark.apache.org/docs/latest/
6. Armbrust, M., et al. “Spark SQL: Relational Data Processing in Spark.” *ACM SIGMOD*, 2015.
7. Armbrust, M., et al. “Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark.” *ACM SIGMOD*, 2018.
8. Boykin, O., et al. “Summingbird: A Framework for Integrating Batch and Online MapReduce Computations.” *Proceedings of the VLDB Endowment* 7(13), 2014.
9. Carbone, P., et al. “Apache Flink: Stream and Batch Processing in a Single Engine.” *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering* 38(4), 2015. (Carbone et al. 2015a)
10. Carbone, P., et al. “Lightweight Asynchronous Snapshots for Distributed Dataflows.” arXiv:1506.08603, 2015. (Carbone et al. 2015b)
11. Carbone, P., et al. “State Management in Apache Flink: Consistent Stateful Distributed Stream Processing.” *Proceedings of the VLDB Endowment* 10(12), 2017.
12. Chandy, K. M., Lamport, L. “Distributed Snapshots: Determining Global States of Distributed Systems.” *ACM Transactions on Computer Systems* 3(1):63–75, 1985.
13. Dean, J., Ghemawat, S. “MapReduce: Simplified Data Processing on Large Clusters.” *USENIX OSDI*, 2004.
14. Kleppmann, M. *Designing Data-Intensive Applications*, ch. 10–11. O’Reilly Media, 2017.
15. Kreps, J. “Questioning the Lambda Architecture.” O’Reilly Radar, 2014. https://www.oreilly.com/radar/questioning-the-lambda-architecture/
16. Marz, N., Warren, J. *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning, 2015.
17. Shen, M., Zhou, Y., Singh, C. “Magnet: Push-based Shuffle Service for Large-scale Data Processing.” *Proceedings of the VLDB Endowment* 13(12), 2020.
18. Xin, R. “Apache Spark Officially Sets a New Record in Large-Scale Sorting.” Databricks Blog, 2014.
19. Young, J. W. “A First Order Approximation to the Optimum Checkpoint Interval.” *Communications of the ACM* 17(9):530–531, 1974.
20. Zaharia, M., et al. “Improving MapReduce Performance in Heterogeneous Environments.” *USENIX OSDI*, 2008.
21. Zaharia, M., et al. “Delay Scheduling: A Simple Technique for Achieving Locality and Fairness in Cluster Scheduling.” *EuroSys*, 2010.
22. Zaharia, M., et al. “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing.” *USENIX NSDI*, 2012.
23. Amazon Web Services. *Amazon S3 User Guide: Best practices design patterns — optimizing Amazon S3 performance*.

https://docs.aws.amazon.com/AmazonS3/latest/userguide/optimizing-performance.html

29.18 Open verification items

- Dean and Ghemawat (2004) sort benchmark figures (891 s; 1,283 s without backup tasks; about 5% slower with 200 workers killed; ~1,800 machines).
- Xin (2014): 100 TB in 23 minutes on 206 machines versus 72 minutes on 2,100 machines; exact blog title and URL.
- Zaharia et al. (2012): “up to 20×” and “5–7 s” interactive figures.
- Magnet: Spark 3.2 as the release adding push-based shuffle; author list.
- Flink 2.0 ForSt disaggregated state backend status and naming (as of 2026); unaligned checkpoints introduced in 1.11.
- Spark Structured Streaming latency figures (~100 ms micro-batch, ~1 ms continuous) as stated in current docs.
- S3 per-prefix request-rate figures (as of 2026).

Part VI — Messaging and Event-Driven Systems

Chapter 30 — Messaging Fundamentals

> **Level:** 2 · **Track:** A (Traditional) · **Estimated reading time:** 2.5 h

Until now most of the services in this book have talked to each other the way people talk on the phone: one calls, the other answers, and both must be present at the same moment. This chapter introduces the alternative, the way people use a postbox. A **message broker** sits between services and holds messages until the receiver is ready, so the sender can carry on without waiting and without even knowing who will read what it sent. That single change — putting durable storage between two services — buys availability, absorbs load spikes and lets one event feed many consumers. It also brings new problems that you must design for explicitly: duplicates, reordering, messages that can never be processed, and backlogs that take hours to drain.

The running example is PhotoShare (Chapter 1). When a user uploads a photo, PhotoShare must store it, generate thumbnails, run content moderation, update the search index, notify followers and record analytics. You will see what happens when those steps are chained synchronously, then rebuild the pipeline with queues and topics, size its consumers, choose a broker, and handle the failures that messaging introduces. Kafka is introduced only briefly here; its internals are the subject of Chapter 31, and the patterns built on top of messaging (transactional outbox, sagas, CQRS, event sourcing) are the subject of Chapter 32.

30.1 Learning objectives

After this chapter you should be able to:

- Explain temporal coupling and compute the availability of a synchronous call chain versus a queue-decoupled design.
- Describe point-to-point queues, competing consumers, acknowledgments and visibility timeouts, and size a consumer fleet with Little’s Law.
- Calculate backlog drain time after an outage and choose a visibility timeout from processing-time percentiles.
- Distinguish publish/subscribe from queues, durable from ephemeral subscriptions, and estimate fan-out cost.
- Distinguish events, commands and messages, and event notification from event-carried state transfer.

- Explain the models of RabbitMQ (exchanges, bindings, quorum queues, streams), Amazon SQS and SNS (standard and FIFO, message groups), Google Cloud Pub/Sub and Apache Pulsar (brokers over BookKeeper).
- Define at-most-once, at-least-once and effectively-once delivery, and trace where duplicates come from.
- Explain per-key ordering, what breaks it, and the throughput limit it imposes.
- Design retries, poison-message handling and dead-letter queues with redrive and alerting, and implement them in a small tested queue.
- Choose between Kafka, RabbitMQ, SQS and Pulsar for a given workload.

30.2 Prerequisites

- Availability arithmetic for series and parallel components — §4.14 (the sync-versus-async comparison is a series calculation).
- Queueing and Little's Law — §4.10–§4.11 (consumer sizing and backlog latency).
- HTTP and RPC request/response — §8.6 and §8.8 (the synchronous baseline that messaging replaces).
- Timeouts, retries with backoff and jitter — §22.11–§22.12 (consumers retry too).
- Backpressure — §22.10 (what a bounded system does when consumers fall behind; owned there).
- Idempotency keys — §22.15 (the consumer-side defense against duplicates; owned there).
- Raft consensus — §26.7 (how replicated queues such as RabbitMQ quorum queues stay durable and available).
- Units: **KB/MB/GB** are powers of ten; a "30-day month" is 43,200 minutes.

30.3 Synchronous vs asynchronous communication

30.3.1 Temporal coupling

DEFINITION

Definition — Synchronous communication. An interaction in which the sender waits for the receiver to process a request and return a response before it continues; both parties must be running and reachable at the same time.

DEFINITION

Definition — Asynchronous (message-based) communication. An interaction in which the sender hands a **message** to an intermediary — a **message broker** — and continues without waiting for the receiver to process it. The broker stores the message until a receiver takes it, so sender and receiver need not be available at the same time.

First-principles explanation

A phone call needs both people on the line. If your friend is in a meeting, the call fails and you must try again later. A letter does not: you drop it in a postbox, the postal service keeps it, and your friend reads it when they get home. The postal service has removed **temporal coupling** — the requirement that both parties be available at the same moment.

Eugster et al. identify three kinds of decoupling that a messaging intermediary can provide [Eugster et al. 2003]:

1. **Time decoupling.** Sender and receiver need not be running at the same time; the broker stores messages in between.
2. **Space decoupling.** The sender need not know who, or how many, the receivers are; it addresses a queue or topic, not a host.
3. **Synchronization decoupling.** The sender is not blocked while the message is processed, and the receiver is notified (or polls) in its own thread of control.

An HTTP call provides none of these. A queue provides all three.

Why does this exist?

Three engineering problems drive services toward messaging:

1. **Availability multiplication.** A chain of synchronous calls is up only if *every* link is up, so availability falls with every hop (formula below).
2. **Load mismatch.** Producers are bursty (a celebrity posts and a million followers react); consumers have a fixed capacity. A queue is a buffer that lets consumers work at their own steady rate while the backlog absorbs the burst.
3. **Latency of slow, non-essential work.** A user uploading a photo needs to know it was stored, not that thumbnails exist in five sizes and every follower has been notified. Moving the slow work out of the request path cuts user-visible latency.

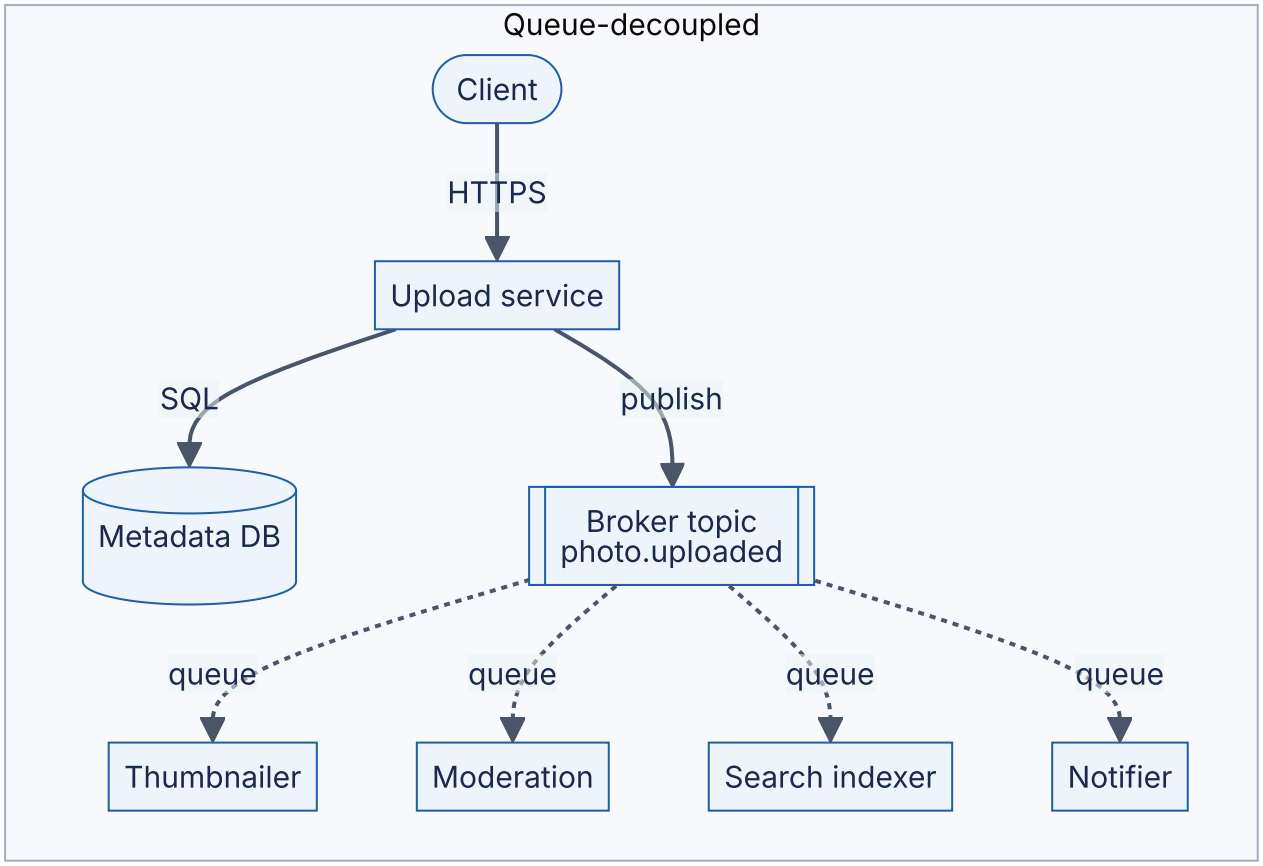
How it works internally

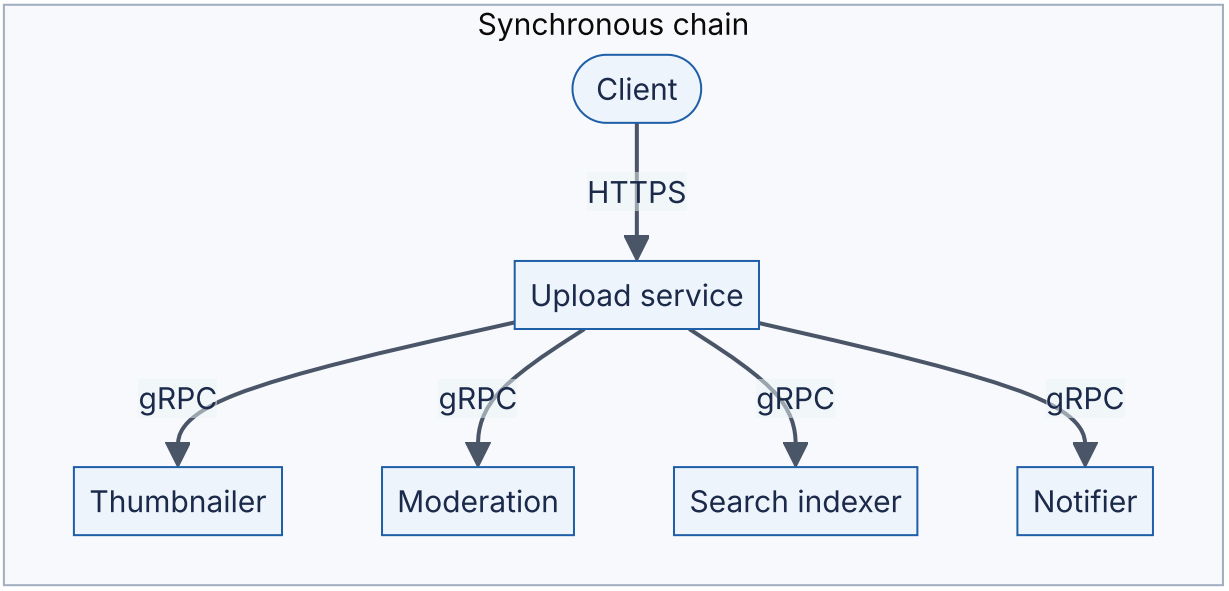
1. The producer serializes the work as a message (a payload plus headers such as an ID, type and timestamp).

2. It sends the message to the broker and waits only for the broker's acknowledgment that the message is durably stored — typically a few milliseconds.
3. The producer returns to its own client ("upload accepted").
4. Later — milliseconds or hours — a consumer retrieves the message, processes it and **acknowledges** it, after which the broker deletes it or marks it consumed.

Architecture diagram

Figure 30.1 — PhotoShare upload: synchronous chain versus queue-decoupled.





Reading the diagram. In the top half, the client's HTTPS request reaches the upload service, which makes four synchronous gRPC calls (solid arrows) before it can answer; the client waits for the slowest call, and a failure of any of the four services fails the upload. In the bottom half, the upload service does two synchronous things — writes the photo's metadata to its database (the storage boundary) and publishes one `photo.uploaded` message to the broker — then answers the client. Each downstream ser-

vice consumes from its own queue asynchronously (dotted arrows) at its own pace. Failure points move: a downstream outage now delays its work instead of failing uploads, while the broker becomes a new critical component that must itself be replicated. Scaling is independent: the thumbnailer can run 50 consumers while the notifier runs 5.

Mathematical foundations

FORMULA

Availability of synchronous and queue-decoupled paths (independent failures)

$$A_{\text{sync}} = \prod_{i=1}^{n} A_i \qquad A_{\text{async}} = A_{\text{front}} \times A_{\text{broker}} \times \prod_{j \in \text{sync deps}} A_j$$

Symbol	Meaning	Units
A_{sync}	Availability of a request that synchronously requires all n services	dimensionless
A_i	Availability of service i on the synchronous path	dimensionless
n	Number of services on the synchronous path, including the front service	count
A_{async}	Availability of *accepting* the request when the other work is queued	dimensionless
A_{front}	Availability of the service that receives the client request	dimensionless
A_{broker}	Availability of the broker for publishing	dimensionless
A_j	Availability of each dependency that must still be called synchronously	dimensionless

With n identical services this is simply A^n: availability decays geometrically with chain length. The asynchronous formula measures something different and weaker — that the request was *accepted* durably. The queued work completes later provided each consumer recovers before the broker's retention period expires.

WORKED EXAMPLE

Worked example — How much downtime does decoupling remove? Every PhotoShare service has $A = 0.999$; the broker has $A_{\text{broker}} = 0.9999$. Use a 30-day month of 43,200 minutes.

1. Fully synchronous (upload service plus four downstream calls, $n = 5$): $A_{\text{sync}} = 0.999^5 = 0.99501$. Unavailable fraction 0.00499, so $0.00499 \times 43,200 \approx 216$ minutes (3.6 hours) per month of failed uploads.
2. Ten services in the chain: $0.999^{10} = 0.99004$, about 430 minutes (7.2 hours) per month.
3. Hybrid: the upload service still writes synchronously to its metadata database ($A = 0.999$) and publishes to the broker: $0.999 \times 0.999 \times 0.9999 = 0.99790$, about 91 minutes per month.
4. Front service plus broker only: $0.999 \times 0.9999 = 0.99890$, about 48 minutes per month.

Interpretation. Removing four synchronous dependencies cuts failed-upload time from about 216 to about 91 minutes per month, and adding services downstream no longer costs any user-facing availability. **Engineering implication.** Keep on the synchronous path only what the client must know before it continues (here: “your photo is stored”), and make the broker more available than the services it decouples, or it becomes the new weakest link.

Advantages and disadvantages

Table 30.1 — Synchronous versus asynchronous communication.

Aspect	Synchronous (RPC, HTTP)	Asynchronous (broker)
Coupling	Time, space and synchronization	None of the three
Availability	Product of the chain	Front service × broker
User latency	Sum or max of calls	Publish time (a few ms)
Result to caller	Immediate, including errors	Later, via another message or polling
Load spikes	Must be served or rejected now	Buffered as backlog
Debugging	One trace, one stack	Distributed, needs correlation IDs
Consistency	Read-your-writes is easy	Downstream state lags (eventual consistency)
New failure modes	Timeouts, cascades	Duplicates, reordering, poison messages, backlog

Asynchrony is not free. The caller cannot get an answer, so anything the user must see immediately — a payment authorization, a login, a search result — stays synchronous. Downstream data is stale for as long as the backlog takes to drain, and the system's behavior is spread across many processes connected only by messages, which makes it harder to follow (tracing across brokers is covered in §40.6).

Failure scenarios

Table 30.2 — Failure scenarios: synchronous versus asynchronous.

What goes wrong	Why	Detection	Recovery	Prevention
Cascading timeouts in a call chain	One slow service holds every caller's threads	Rising p99, thread-pool saturation	Shed load, open circuit breakers (§22.13)	Shorter chains, deadlines, queue non-essential work
Broker outage	The new single dependency fails	Publish errors and latency	Fail over to replicas; buffer locally briefly	Replicated broker, multi-zone deployment
Silent backlog growth	Consumers slower than producers	Queue depth and age of oldest message	Scale consumers; drain	Alert on message age, autoscale on lag
User sees stale data	Work still queued	Complaints, read-after-write checks	Show "processing" state	Design the UI for eventual consistency

COMMON MISTAKE

"Asynchronous means faster." Publishing returns sooner, but the *work* takes at least as long — plus queueing time. If the user needs the result to continue, making the call asynchronous only moves the wait into polling. Use messaging for work whose result the caller does not need before it continues.

COMMON MISTAKE

"Async calls with callbacks are messaging." A non-blocking HTTP client (`async` / `await`) removes synchronization coupling inside one process, but the receiver must still be up at that moment. Only a durable intermediary removes time coupling.

30.4 Message queues

30.4.1 Point-to-point channels and competing consumers

DEFINITION

Definition — Message queue (point-to-point channel). A broker-managed, durable buffer of messages in which each message is delivered to, and processed by, exactly one of the queue's consumers; once a consumer acknowledges a message, the broker removes it [Hohpe and Woolf 2003].

DEFINITION

Definition — Competing consumers. A pattern in which several consumer processes read from the same queue concurrently, each message going to whichever consumer takes it first, so that adding consumers raises throughput [Hohpe and Woolf 2003].

First-principles explanation

Think of the ticket rail in a restaurant kitchen. Waiters clip orders to the rail; any free cook takes the next ticket; when the dish leaves the pass, the ticket is spiked and discarded. No order is cooked twice, and when the dinner rush arrives the manager adds cooks, not waiters. The rail is the queue, the cooks are competing consumers, and spiking the ticket is the acknowledgment.

The essential property is **destructive, shared consumption**: a message is work to be done once, by someone. That makes queues the right tool for *tasks* — resize this image, send this email, charge this order — and the wrong tool when several independent parties each need their own copy (§30.5).

How it works internally

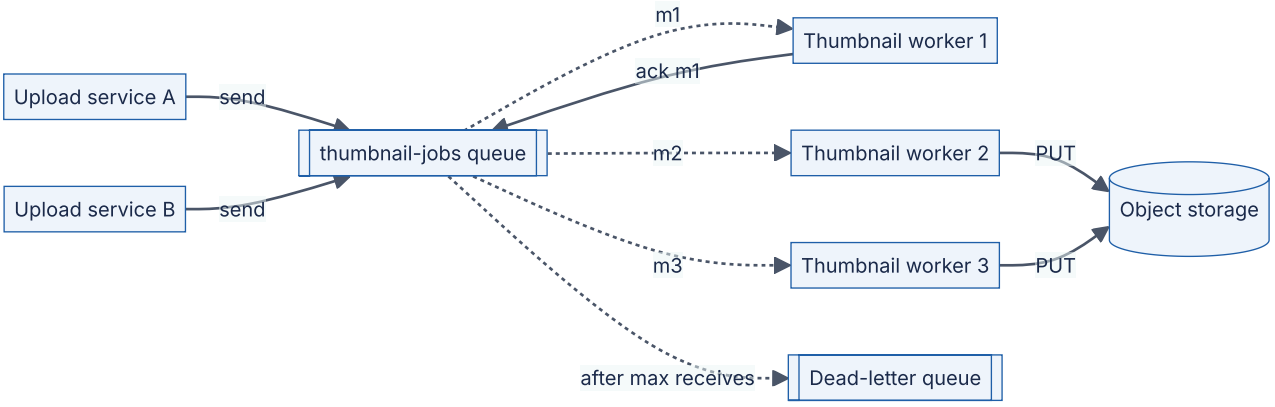
1. **Send.** The producer sends a message; the broker persists it (usually replicated across nodes or availability zones) and confirms.
2. **Receive.** A consumer asks for up to *b* messages. Brokers either **push** messages to subscribed consumers up to a **prefetch** limit (RabbitMQ) or let consumers **pull** with long polling (SQS waits up to 20 seconds for a message to arrive rather than returning empty).
3. **Lease.** The broker does not delete a delivered message. It marks it **in flight** — invisible to other consumers — for a lease period: the **visibility timeout** in SQS, the **ack deadline** in Google Cloud Pub/Sub. RabbitMQ instead keeps a delivered mes-

sage unacknowledged for as long as the consumer's channel stays open, and re-queues it if the channel or connection closes.
4. **Process.** The consumer does the work.
5. **Acknowledge.** The consumer tells the broker it is done (SQS `DeleteMessage` with the delivery's **receipt handle**; AMQP `basic.ack`; Pub/Sub `acknowledge`). Only now is the message removed.
6. **Redeliver on failure.** If the consumer crashes, or the lease expires before the acknowledgment, the message becomes visible again and another consumer receives it. A consumer can also **negatively acknowledge** (`basic.nack`, or setting the SQS visibility timeout to zero) to release it immediately.

Acknowledging *after* processing is what makes a queue reliable: a crash at any point before step 5 leads to redelivery rather than loss. It is also why every queue that never loses messages can deliver one twice (§30.8).

Architecture diagram

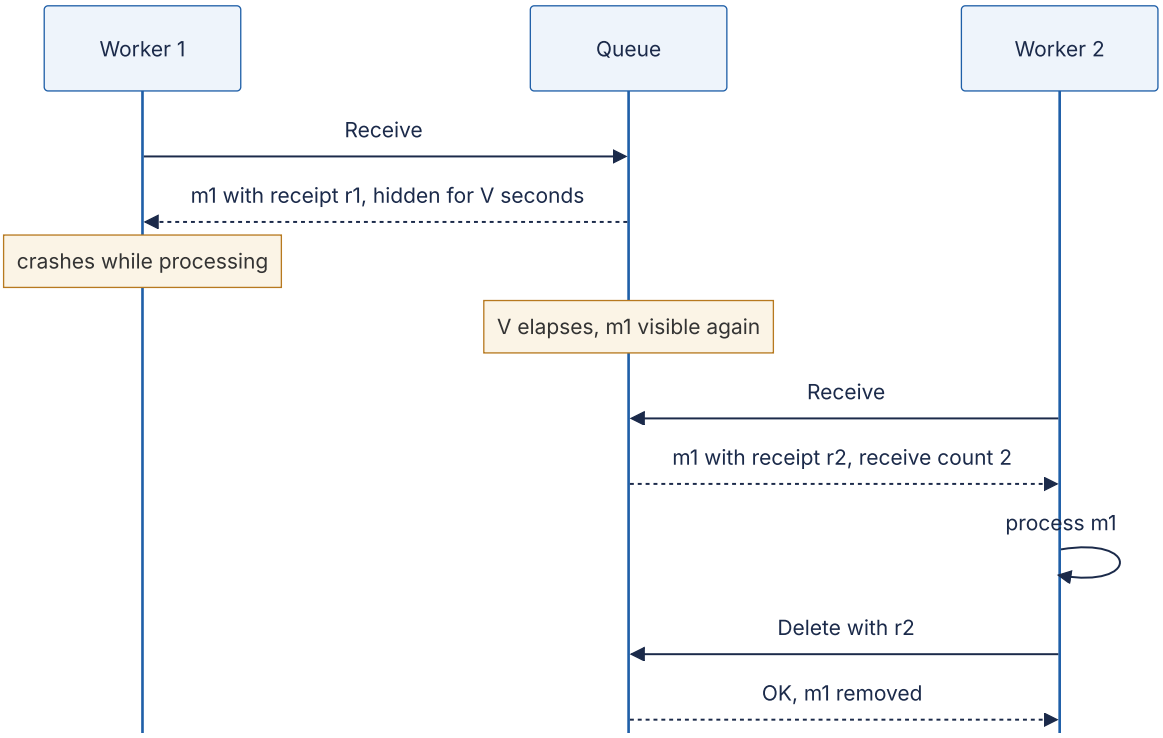
Figure 30.2 — A point-to-point queue with competing consumers.



Reading the diagram. Two upload-service instances send jobs to one queue (solid arrows: the send is a synchronous call to the broker). The queue delivers each message to exactly one of three workers (dotted: delivery is asynchronous and decoupled in time). Each worker writes thumbnails to object storage (the storage boundary outside the broker) and then acknowledges, which deletes the message; worker 1's acknowledgment is drawn as an example. A message whose processing keeps failing is moved to a dead-letter queue after a maximum number of deliveries (§30.11). Failure points: a worker crash leaves its message in flight until the lease expires; a broker-node failure is covered by replication inside the broker. Scaling: add workers until the queue's own throughput or the downstream storage becomes the limit.

30.4.2 Acknowledgments and the visibility timeout

Figure 30.3 — The visibility timeout: success, crash and redelivery.



Reading the diagram. Worker 1 receives message m1 with receipt r1, and the queue hides m1 for *V* seconds. Worker 1 crashes before acknowledging, so nothing happens until the timeout elapses; m1 then becomes visible and worker 2 receives it with a *new* receipt r2 and a receive count of 2. Worker 2 processes it and deletes it using r2. All calls are synchronous request/response between worker and queue; the redelivery itself is time-driven. The failure point to notice is the gap: m1 waited a full *V* before anyone retried it. Had worker 1 merely been slow rather than dead, both workers would have processed m1 — a duplicate.

The visibility timeout is therefore a trade-off with no free setting. Too short, and slow-but-healthy consumers lose their lease and the message is processed twice. Too long, and a crash delays the retry by the full timeout. Long-running consumers escape the dilemma with a **heartbeat**: start with a modest timeout and periodically extend it while work progresses (SQS `ChangeMessageVisibility`; Pub/Sub `modifyAckDeadline`, which its client libraries do automatically). As of 2026, SQS defaults to 30 seconds and allows 0 seconds to 12 hours [AWS Docs: SQS]; Pub/Sub defaults to 10 seconds and allows up to 600 seconds per extension [Google Cloud Docs: Pub/Sub]; RabbitMQ closes a channel whose delivery stays unacknowledged longer than its consumer timeout (30 minutes by default) and requeues its messages [RabbitMQ Docs: Confirms].

FORMULA

Visibility-timeout sizing

$$V \geq b \cdot S_{\text{tail}} + \delta$$

Symbol	Meaning	Units
V	Visibility timeout (lease length)	s
b	Messages received per batch and processed sequentially under one lease	messages
S_{tail}	High-percentile (for example p99.9) processing time per message	s
δ	Margin for acknowledgment latency, pauses and retries inside processing	s

WORKED EXAMPLE

Worked example — Sizing the thumbnail queue's timeout. Workers receive batches of $b = 10$ messages and process them one after another. Measured processing time per message: p50 0.4 s, p99.9 2.5 s. Allow $\delta = 10$ s.

1. Worst plausible batch: $10 \times 2.5 = 25$ s.
2. $V \geq 25 + 10 = 35$ s; round to 40 s.
3. The default of 30 s would expire during an unlucky batch and cause the last few messages to be processed twice.
4. Crash cost: a dead worker's 10 messages reappear only after up to 40 s.
5. Alternative: receive one message at a time ($b = 1$) with $V = 15$ s, or keep $b = 10$ with $V = 30$ s and extend the lease after each message.

Interpretation. The timeout must cover the *batch*, not the average message. **Engineering implication.** Size V from tail latency and batch size; when processing time is unbounded (video transcoding), use heartbeats rather than an hours-long timeout that would make crash recovery equally slow.

30.4.3 Sizing consumers and draining backlogs

How many consumers does a queue need? Little's Law (§4.11) answers directly: the number of messages being processed at once equals the arrival rate multiplied by the processing time.

FORMULA

Consumer count from throughput (Little's Law)

$$N = \lceil \frac{\lambda \cdot S}{k \cdot \rho} \rceil$$

Symbol	Meaning	Units
N	Number of consumer instances	instances
λ	Average arrival rate of messages	messages/s
S	Mean processing time per message, including acknowledgment	s
λS	Average messages in processing (Little's L)	messages
k	Concurrent messages per instance (threads, coroutines or prefetch)	messages
ρ	Target utilization of each processing slot	dimensionless (0–1)

WORKED EXAMPLE

Worked example — Thumbnail workers. PhotoShare receives $\lambda = 2,000$ uploads per second at peak; generating thumbnails takes $S = 150$ ms; each worker instance runs $k = 10$ concurrent jobs; target utilization $\rho = 0.7$.

1. Messages in processing: $L = \lambda S = 2,000 \times 0.15 = 300$.
2. Slots needed at 70% utilization: $300/0.7 = 428.6$.
3. Instances: $428.6/10 = 42.9$, so $N = 43$.
4. Capacity of 43 instances: $\mu = Nk/S = 43 \times 10/0.15 = 2,867$ messages/s.

Interpretation. About 43 instances keep pace with peak with 30% headroom. **Engineering implication.** Headroom is not only for spikes; it sets how quickly you recover from a backlog, as the next formula shows. Re-derive N whenever S changes — a slower downstream store raises S and silently raises N.

When consumers stop — a bad deploy, a downstream outage — producers keep sending and the backlog grows at rate λ. When consumers return, they drain it only with their *surplus* capacity.

FORMULA

Backlog drain time

$$T_{\text{drain}} = \frac{B}{\mu - \lambda}, \qquad \mu = \frac{Nk}{S}, \qquad B = \lambda \cdot T_{\text{out}}$$

Symbol	Meaning	Units
T_{drain}	Time to return to a near-empty queue	s
B	Backlog when consumers resume	messages
T_{out}	Duration of the consumer outage	s
μ	Maximum consumer throughput	messages/s
λ	Arrival rate during the drain	messages/s

WORKED EXAMPLE

Worked example — Recovering from a 30-minute outage. Thumbnail workers are down for $T_{\text{out}} = 1,800$ s while uploads continue at $\lambda = 2,000$/s.

1. Backlog: $B = 2,000 \times 1,800 = 3.6$ million messages.
2. With 43 instances, $\mu = 2,867$/s; surplus $= 867$/s; $T_{\text{drain}} = 3.6 \times 10^6 / 867 \approx 4,150$ s ≈ 69 minutes.
3. Autoscaling to 100 instances: $\mu = 100 \times 10/0.15 = 6,667$/s; surplus $4,667$/s; $T_{\text{drain}} \approx 771$ s ≈ 13 minutes — if object storage can absorb 6,667 writes per second.
4. Age of the oldest message when workers resume: 30 minutes. With Little's Law (§4.11.2), a queue depth of 3.6 million at 2,867/s means a new message waits about $3.6 \times 10^6 / 2,867 \approx 21$ minutes.

Interpretation. A 30-minute outage costs more than an hour of stale thumbnails at normal headroom. **Engineering implication.** Alert on the **age of the oldest message** (SQS exposes `ApproximateAgeOfOldestMessage`), not just depth; autoscale consumers on backlog per instance; and check that retention (4 days by default in SQS, up to 14 [AWS Docs: SQS]) comfortably exceeds the longest outage you can imagine.

INDUSTRY EXAMPLE

Slack's job queue. Slack runs much of its asynchronous work (message fan-out, link unfurling, notifications) through a job queue originally built on Redis. Slack's engineers describe an incident in which a database slowdown made jobs run slower, the backlog grew until Redis ran out of memory, and the queue could no longer accept or hand out jobs. They responded by putting Kafka in front of Redis as a durable, disk-backed buffer, so that a backlog grows on disk rather than in a memory-bound store [Slack Engineering 2020]. *Evidence: Primary* — [Slack Engineering 2020]

Failure scenarios

Table 30.3 — Failure scenarios: queues and consumers.

What goes wrong	Why	Detection	Recovery	Prevention
Duplicate processing	Lease expired while a slow consumer still worked	Same message ID processed twice	Idempotent consumer absorbs it	Size V from tail latency; heartbeat
Messages stuck for minutes	Consumer crashed holding a large prefetch or batch	Age of oldest message jumps by V	Wait for lease expiry	Small batches; health-checked consumers
Backlog never drains	$\mu \leq \lambda$ after a slowdown	Depth rising steadily	Scale out; shed optional work	Headroom; autoscaling on age
Downstream overload during drain	Many consumers hammer the database	DB latency and errors	Cap consumer concurrency	Rate-limit consumers (§22.10)
Broker memory exhaustion	Unbounded in-memory backlog	Broker memory alarms	Block publishers, add storage	Disk-backed queues; length limits

COMMON MISTAKE

"Acknowledge as soon as you receive." Acknowledging before processing turns a reliable queue into an at-most-once one: if the consumer crashes after the acknowledgment, the work is lost. Acknowledge after the side effects are durable.

30.5 Publish/subscribe

30.5.1 Topics, subscriptions and fan-out

DEFINITION

Definition — Publish/subscribe (pub/sub). A messaging style in which a publisher sends a message to a named **topic** without knowing its receivers, and the broker delivers a copy to every **subscription** registered on that topic [Hohpe and Woolf 2003; Eugster et al. 2003].

First-principles explanation

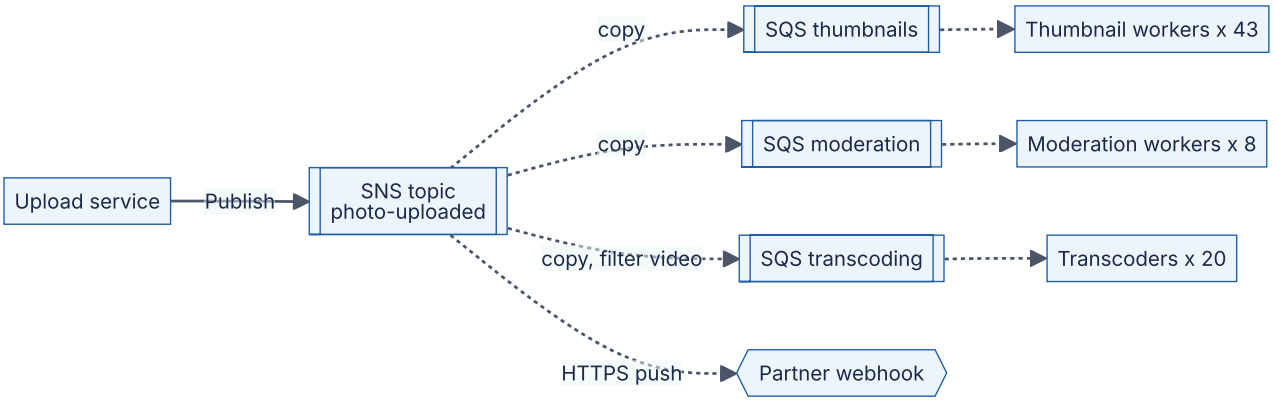
A queue is a ticket rail: each ticket is cooked once. A topic is a newspaper: every subscriber gets their own copy, and the newsroom neither knows nor cares how many there are. Adding a subscriber requires no change to the publisher — which is the point. When PhotoShare later adds a “memories” feature that needs every upload, the team creates a new subscription; the upload service is untouched. Inside a single process the same idea is the Observer pattern (§37.8), which is synchronous and keeps no messages; Table 37.1 compares the two.

The two styles compose. Each *subscription* usually behaves like a queue, with its own competing consumers, acknowledgments and backlog. Fan-out happens between subscriptions; load balancing happens within one.

How it works internally

1. Subscribers register subscriptions on the topic, optionally with a **filter** (for example, only messages whose attribute `type = "video"`).
2. The publisher sends a message to the topic; the broker stores it once or per subscription, depending on the design.
3. The broker delivers a copy to every matching subscription. Each subscription tracks its own acknowledgments, so a slow subscriber’s backlog never delays a fast one.

Figure 30.4 — Fan-out from one topic to independent subscriptions (SNS to SQS).



Reading the diagram. The upload service publishes once to an SNS topic (a synchronous API call). SNS copies the message asynchronously to three SQS queues and one external HTTPS endpoint; the transcoding subscription carries a filter, so it receives only videos. Each SQS queue is a durable storage boundary with its own pool of competing consumers. Failure points: a dead moderation fleet grows only the moderation queue; an unreachable webhook is retried by SNS under its delivery policy and can be given its own dead-letter queue. Scaling: subscriptions and their consumer pools grow independently, which is why this **topic-to-queue fan-out** is the standard AWS pattern [AWS Docs: SNS].

30.5.2 Durable and ephemeral subscriptions

DEFINITION

Definition — Durable subscription. A subscription whose messages the broker retains while the subscriber is disconnected, so the subscriber receives everything published since it subscribed, within the retention period. An **ephemeral** (non-durable) subscription exists only while the subscriber is connected; messages published while it is away are never delivered to it.

Hohpe and Woolf call the first a **Durable Subscriber**. Use durable subscriptions for business work that must not be skipped (search indexing, billing) and ephemeral ones where missed messages are worthless after a few seconds (live cursor positions, cache-invalidation hints that are also covered by TTLs, dashboards). An exclusive, auto-delete RabbitMQ queue bound to a fanout exchange, a Redis Pub/Sub channel and an SNS-to-HTTP push to a server that is down are all ephemeral in effect; an SQS queue subscribed to SNS, a Pub/Sub subscription and a Pulsar subscription cursor are durable.

Durability has a cost: a subscriber that disappears keeps accumulating messages until retention expires. Abandoned durable subscriptions are a classic source of storage alarms.

FORMULA

Fan-out delivery rate, egress and orphaned backlog

$$D = \lambda \cdot N_s, \qquad E = \lambda \cdot s \cdot N_s, \qquad B_{\text{orphan}} = \lambda \cdot s \cdot T_{\text{away}}$$

Symbol	Meaning	Units
D	Deliveries the broker performs	deliveries/s
λ	Publish rate	messages/s
N_s	Number of subscriptions (after filtering)	count
E	Broker egress	bytes/s
s	Mean message size	bytes
B_{orphan}	Storage held for one disconnected durable subscriber	bytes
T_{away}	Time the subscriber has been away	s

WORKED EXAMPLE

Worked example — PhotoShare event fan-out. The platform publishes $\lambda = 5,000$ events/s of $s = 2$ KB each to a topic with $N_s = 12$ subscriptions.

1. Deliveries: $5,000 \times 12 = 60,000$ per second, about 5.2 billion per day — and per-request pricing applies to each.
2. Egress: $5,000 \times 2,000 \times 12 = 120$ MB/s, close to 1 Gb/s.
3. One subscriber's service is turned off for 6 hours: $5,000 \times 21,600 = 108$ million messages, 216 GB retained for it alone.

Interpretation. Each subscription multiplies delivery and egress cost, and an unattended durable subscription grows by hundreds of gigabytes per day. **Engineering implication.** Filter at the broker so subscribers receive only what they need, treat subscriptions as owned resources with alarms on backlog age, and delete subscriptions whose consumers are retired.

COMMON MISTAKE

"Pub/sub replaces queues." Pub/sub decides *who* gets a copy; queues decide *how* one subscriber shares work among its instances. Nearly every production pub/sub system has a queue (or queue-like subscription) at each leaf.

30.6 Event-driven architecture

30.6.1 Events, commands and messages

DEFINITION

Definition — Message, command, event. A **message** is the transport unit a broker carries: headers plus a payload. A **command** is a message that asks one specific receiver to do something (“ResizePhoto”); it may be refused. An **event** is a message stating a fact that has already happened (“PhotoUploaded”); it cannot be refused, only reacted to, and its publisher does not know who reacts [Hohpe and Woolf 2003].

The distinction is about **direction of knowledge**. A command couples the sender to the receiver’s capability: the upload service must know that a thumbnailer exists and what it needs. An event inverts that: the upload service announces what happened, and the thumbnailer decides to care. **Event-driven architecture (EDA)** is a style in which services communicate mainly by publishing and reacting to events, so that new behavior is added by adding subscribers rather than changing publishers. Its architectural treatment is in §33.8; this section covers the messaging mechanics.

Table 30.4 — Commands versus events.

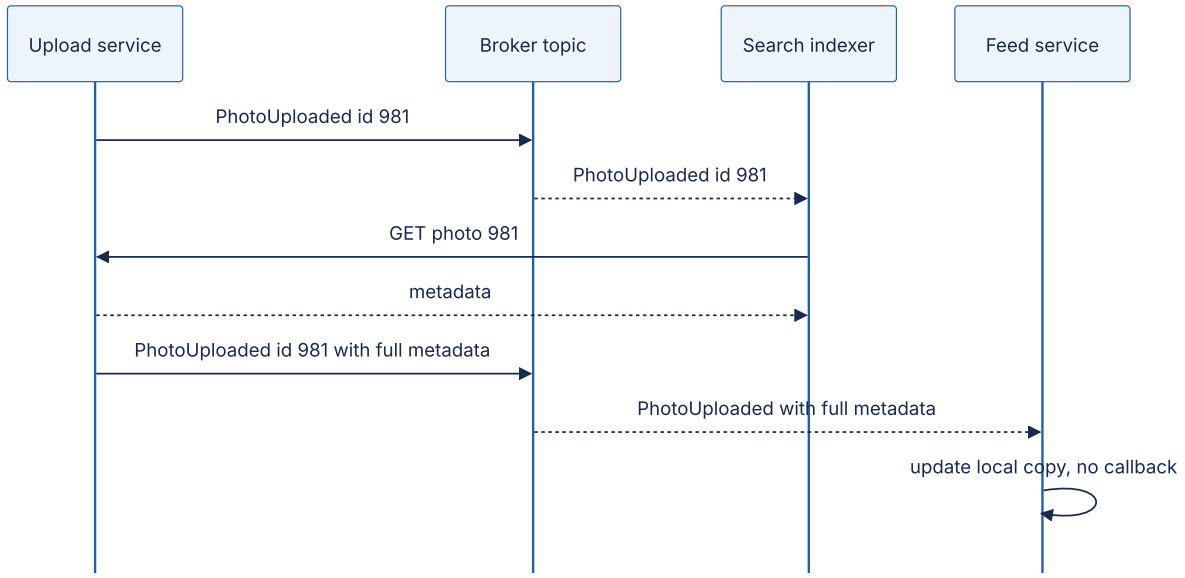
Property	Command	Event
Grammar	Imperative: `ChargeCard`	Past tense: `CardCharged`
Receivers	Exactly one logical receiver	Zero or more unknown subscribers
Channel	Queue (point-to-point)	Topic (pub/sub)
Can be rejected?	Yes, with a reply or failure	No; it already happened
Coupling	Sender knows receiver’s capability	Publisher knows only its own facts
Typical use	Delegating a task	Announcing a state change

30.6.2 Event notification versus event-carried state transfer

Fowler distinguishes patterns that all get called “event-driven” [Fowler 2017]. Two concern us here (event sourcing and CQRS are in §32.3–§32.4):

- **Event notification.** The event says *that* something changed and carries little more than an identifier: `{type: "PhotoUploaded", photoId: 981}`. Subscribers that need details call back to the publisher’s API.
- **Event-carried state transfer.** The event carries the state subscribers need: `{type: "PhotoUploaded", photoId: 981, ownerId: 7, url: "...", width: 4032, tags: [...]}`. Subscribers keep their own local copy and never call back.

Figure 30.5 — Event notification (with callback) versus event-carried state transfer.



Reading the diagram. In the first exchange (event notification), the upload service publishes a thin event; the broker delivers it asynchronously to the search indexer, which makes a synchronous HTTP call back to the upload service for details. The upload service is therefore still on the indexer's critical path, and a burst of events becomes a burst of callbacks. In the second exchange (event-carried state transfer), the event carries the metadata; the feed service updates its own storage without calling back. Failure points: the callback path fails when the upload service is down; the fat-event path fails when subscribers depend on fields the publisher later changes. Scaling: fat events remove callback load but enlarge every message and every subscriber's storage.

Choose event notification when payloads are large or sensitive, when subscribers need the *current* state rather than the state at event time, or when few subscribers need details. **Choose** event-carried state transfer when many subscribers need the same data, when you want subscribers to keep working while the publisher is down, and when the data is small. In both cases the event schema is a public API: version it, add fields compatibly, and validate it with a schema registry (§8.9.3 covers schema evolution and compatibility). The CNCF **CloudEvents** specification standardizes common envelope attributes (`id`, `source`, `type`, `time`, `subject`) so events can cross brokers and organizations [CNCF CloudEvents 2019].

> COMMON MISTAKE
>
> **"Events should be named after what the receiver must do."** `SendWelcomeEmail` published to a topic is a command in disguise; the publisher still knows its receiver. Name events after facts in the publisher's own domain (`AccountCreated`) and let the email service decide.

30.7 Event brokers

The ideas so far are broker-neutral. Real brokers package them differently, and the packaging decides what is easy. This section describes four widely used brokers as of 2026; Kafka, the fifth, is introduced in §30.12 and covered fully in Chapter 31. Product limits change often, so treat every number here as "as of 2026" and confirm it in current documentation before relying on it.

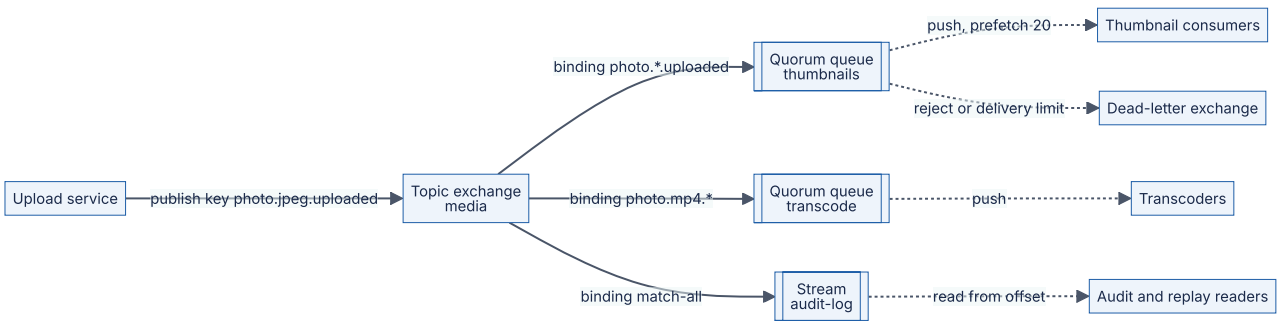
30.7.1 RabbitMQ and the AMQP 0-9-1 model

RabbitMQ is an open-source broker, written in Erlang, that implements the **AMQP 0-9-1** protocol (and, since RabbitMQ 4.0, AMQP 1.0 natively, plus MQTT and STOMP). Its central idea is that producers never write to queues directly. They publish to an **exchange** with a **routing key**; the exchange routes copies to every queue whose **binding** matches; consumers read from queues [RabbitMQ Docs: AMQP Concepts]. Routing is therefore configuration, not code.

Table 30.5 — RabbitMQ exchange types.

Exchange type	Routing rule	Typical use
Direct	Binding key equals routing key exactly	Task queues by job type; the default exchange routes to the queue named by the key
Topic	Pattern over dot-separated words: `*` matches one word, `#` matches zero or more	`photo.*.uploaded`, `audit.#`
Fanout	Ignores the key; copies to every bound queue	Broadcast to all subscribers
Headers	Matches message header values (`x-match` all or any)	Routing on several attributes

Figure 30.6 — The AMQP 0-9-1 model: exchanges, bindings and queues.



Reading the diagram. The upload service publishes over AMQP to the `media` topic exchange with routing key `photo.jpeg.uploaded`. The exchange evaluates each binding: the

key matches `photo.*.uploaded` and the match-all binding `#`, so copies go to the `thumbnails` quorum queue and the `audit-log` stream but not to `transcode`. Consumers receive from queues asynchronously; RabbitMQ pushes up to the **prefetch** count of unacknowledged messages per consumer (set with `basic.qos`). Storage lives in queues and streams, not exchanges. Messages rejected without requeue, expired, or over a delivery limit are republished to a **dead-letter exchange** (§30.11). Failure points: publishing to an exchange with no matching binding silently drops the message unless the publisher sets the `mandatory` flag or the exchange has an alternate exchange. Scaling: one queue's throughput is bounded by the node hosting its leader, so high throughput comes from many queues spread across nodes.

Reliability on both sides. Publisher confirms make the broker acknowledge each published message once it is safely stored (for quorum queues, once a majority of replicas has it); without them a publisher cannot know whether a message survived a broker crash. **Consumer acknowledgments** (`basic.ack`, `basic.nack`, `basic.reject`) work as in §30.4 [RabbitMQ Docs: Confirms].

Queue types. RabbitMQ offers three, as of 2026:

1. **Classic queues** — a single node's queue, now unreplicated: classic queue mirroring was deprecated and then removed in RabbitMQ 4.0.
2. **Quorum queues** — replicated with Raft (§26.7) across, typically, three or five nodes. A publish is confirmed when a majority stores it, and a leader failure elects a new leader without losing confirmed messages. Quorum queues track a per-message delivery count and support a **delivery limit**, after which a message is dead-lettered or dropped [RabbitMQ Docs: Quorum Queues]. They are the recommended type for data that must survive node failures.
3. **Streams** — introduced in RabbitMQ 3.9, an append-only, replicated log in which consuming does *not* remove messages. Consumers attach at an offset (first, last, next, or a timestamp), several consumers can read the same data independently, and retention is by size or age. **Super streams** partition one logical stream across several streams [RabbitMQ Docs: Streams]. Streams bring Kafka-like replay to RabbitMQ.

30.7.2 Amazon SQS and SNS

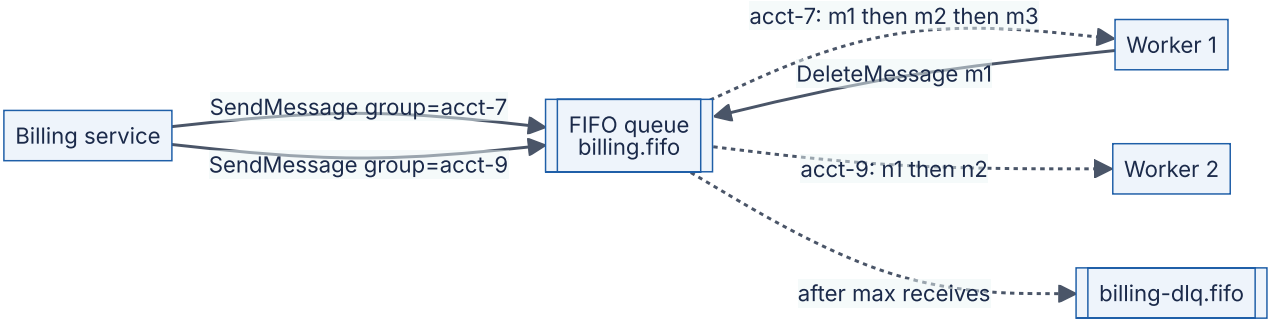
Amazon SQS is a fully managed queue service: there are no brokers to run, and you pay per request. It offers two queue types [AWS Docs: SQS]:

- **Standard queues** — at-least-once delivery, *best-effort* ordering, and a nearly unlimited number of API calls per second. Occasionally a message is delivered twice or out of order.
- **FIFO queues** — strict ordering *within a message group* and deduplication of sends: a message sent again with the same **deduplication ID** (or the same body, with con-

tent-based deduplication) within a 5-minute interval is accepted but not enqueued twice. Throughput is quota-limited (300 API calls per second per action without batching in the basic mode; much higher in high-throughput mode, varying by region).

The key FIFO concept is the **message group ID**. Messages with the same group ID are delivered in order and *one batch at a time*: while any message of a group is in flight, SQS returns no later message from that group. Different groups proceed in parallel, so the group ID is both the unit of ordering and the unit of parallelism.

Figure 30.7 — SQS FIFO message groups: ordered within a group, parallel across groups.



Reading the diagram. The billing service sends messages with a group ID per account. SQS hands account 7's messages to worker 1 strictly in order, releasing m2 only after m1 is deleted or its visibility timeout expires; account 9's messages flow to worker 2 in parallel. All calls are synchronous HTTPS API calls; delivery is pull-based. Storage is inside SQS, replicated across availability zones. Failure points: a message that keeps failing blocks *its group* (head-of-line blocking) until it moves to the FIFO dead-letter queue; a hot group is limited to one consumer's speed (§30.9). Scaling: more groups, not more workers, is what raises a FIFO queue's parallelism.

Other SQS behaviors worth knowing: long polling up to 20 s per `ReceiveMessage`; batches of up to 10 messages per call; per-message delays up to 15 minutes; retention from 60 s to 14 days; a **redrive policy** with `maxReceiveCount` that moves a message to a dead-letter queue (§30.11); and, since 2025, *fair queues*, in which standard queues use the message group ID to stop one noisy tenant's backlog from delaying the others [AWS Docs: SQS].

Amazon SNS is the managed pub/sub companion: a topic delivers to SQS queues, AWS Lambda functions, HTTP/S endpoints, email, SMS and mobile push. Subscriptions can carry **filter policies** on message attributes or payload fields, and each subscription can have its own dead-letter queue for deliveries that exhaust their retries. **SNS FIFO topics** preserve per-group order and deduplication when fanning out to SQS queues [AWS Docs: SNS]. SNS-to-SQS fan-out (Figure 30.4) is the AWS idiom for "publish once, many durable consumers".

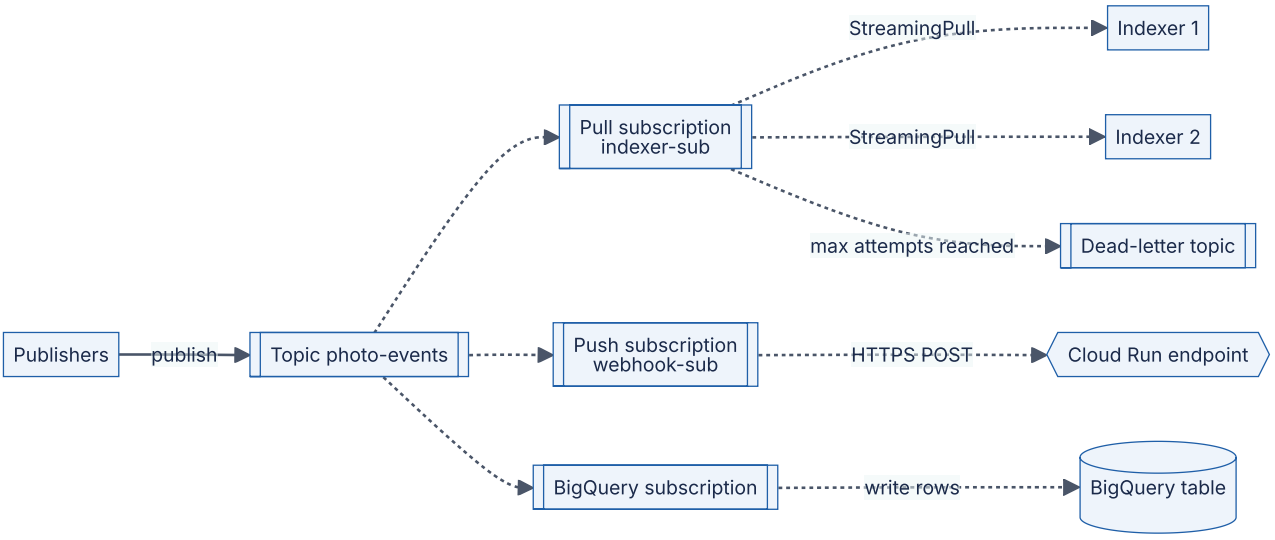
INDUSTRY EXAMPLE

Amazon SQS. SQS became generally available in 2006 and is one of the oldest AWS services. Its design choice — a standard queue that gives up strict ordering and exactly-once delivery in exchange for essentially unlimited scale — pushed a generation of engineers to write idempotent consumers, and the later FIFO type made the ordering-versus-throughput trade-off an explicit per-queue choice [AWS Docs: SQS]. *Evidence: Primary* — [AWS Docs: SQS]

30.7.3 Google Cloud Pub/Sub

Google Cloud Pub/Sub is a global, fully managed service that unifies topics and queues in one model: a **topic** fans out to any number of **subscriptions**, and each subscription is a durable queue shared by its subscribers [Google Cloud Docs: Pub/Sub].

Figure 30.8 — Google Cloud Pub/Sub: topics, subscriptions and delivery modes.



Reading the diagram. Publishers send to a topic; every message is copied to all three subscriptions (fan-out). The indexer subscription is **pull**: two indexer instances hold StreamingPull connections and share its messages (competing consumers). The webhook subscription is **push**: Pub/Sub POSTs each message over HTTPS and treats a success status as the acknowledgment. The **BigQuery subscription** writes messages directly into a table, a storage boundary with no consumer code. All delivery is asynchronous; each subscription keeps its own unacknowledged backlog. Failure points: a message that is not acknowledged within the ack deadline is redelivered; after a configured number of attempts it goes to a dead-letter topic. Scaling is automatic; the main levers are the number of subscriber instances and flow-control settings.

Features as of 2026: an ack deadline from 10 s (default) to 600 s, extended automatically by client libraries; retention of unacknowledged messages up to 7 days per subscription

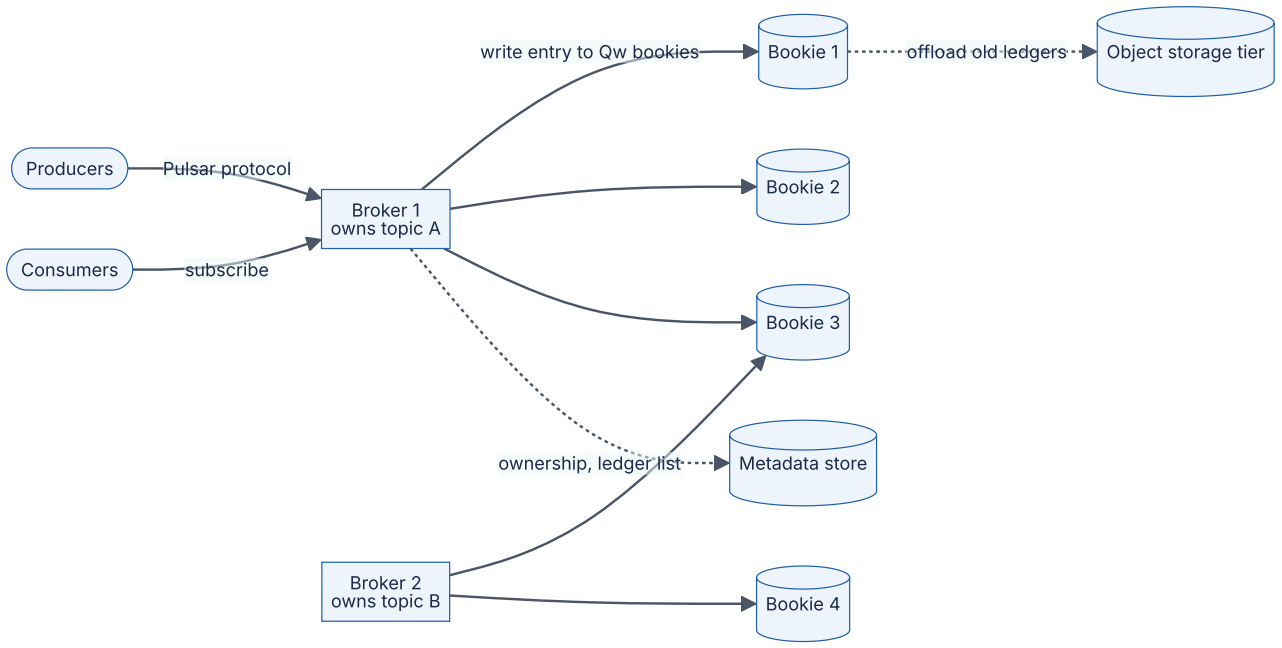
and, optionally, up to 31 days on the topic; **seek** to a timestamp or snapshot for replay; **ordering keys**, which deliver messages with the same key in publish order when the subscription enables ordering; **exactly-once delivery** for pull subscriptions within a region, which guarantees that a successfully acknowledged message is not redelivered; **dead-letter topics** with 5–100 maximum delivery attempts; and retry policies with exponential backoff [Google Cloud Docs: Pub/Sub].

30.7.4 Apache Pulsar

Apache Pulsar, created at Yahoo and an Apache top-level project since 2018, separates **serving** from **storage** [Apache Pulsar Docs]:

- **Brokers** are stateless. Each topic is owned by one broker at a time, which accepts publishes, dispatches to consumers and caches recent entries.
- **Apache BookKeeper** stores the data. A topic is a sequence of **ledgers** (segments); each ledger is written to an **ensemble** of E storage nodes called **bookies**, with each entry written to Q_w of them and acknowledged after Q_a confirm (for example $E = 3$, $Q_w = 3$, $Q_a = 2$). When a ledger closes, a new one may be placed on different bookies [Junqueira et al. 2013].
- A **metadata store** (ZooKeeper, or alternatives in newer releases) holds topic ownership and ledger lists.

Figure 30.9 — Pulsar: stateless brokers over BookKeeper segment storage.



Reading the diagram. Producers and consumers connect to the broker that owns their topic. Broker 1 writes each entry synchronously to Q_w bookies and confirms to the producer after Q_a acknowledgments; topic B's ledgers sit on a different set of bookies. Ownership and ledger metadata live in the metadata store (dotted: read and updated

asynchronously relative to the data path). Sealed ledgers can be offloaded to object storage (**tiered storage**). Failure points: if broker 1 dies, another broker takes ownership of topic A in seconds without copying data, because the data is in BookKeeper; if a bookie dies, its ledgers are re-replicated from the surviving copies. Scaling: add brokers for serving capacity and bookies for storage independently.

Pulsar supports both queueing and streaming through **subscription types**: *exclusive* and *failover* (one active consumer, ordered, stream-like), *shared* (competing consumers, queue-like) and *key_shared* (competing consumers with per-key ordering). Each subscription is a durable **cursor** over the topic, with individual acknowledgments, negative acknowledgments, a redelivery count, retry-letter and dead-letter topics in the clients, and built-in geo-replication between clusters [Apache Pulsar Docs].

INDUSTRY EXAMPLE

DistributedLog at Twitter. Twitter built DistributedLog, a replicated log service on BookKeeper, to serve as durable storage for its messaging and database replication workloads; the design segments each log into BookKeeper ledgers so that serving and storage scale separately — the same architecture Pulsar uses [Guo et al. 2017]. *Evidence: Primary* — [Guo et al. 2017]

30.8 Delivery guarantees

DEFINITION

Definition — Delivery guarantees. At-most-once: each message is processed zero or one times; failures lose messages but never duplicate them. **At-least-once**: each message is processed one or more times; failures cause duplicates but never loss. **Exactly-once** (more precisely **effectively-once**): the *observable effect* of processing is as if each message were processed exactly once, even though delivery may happen more than once.

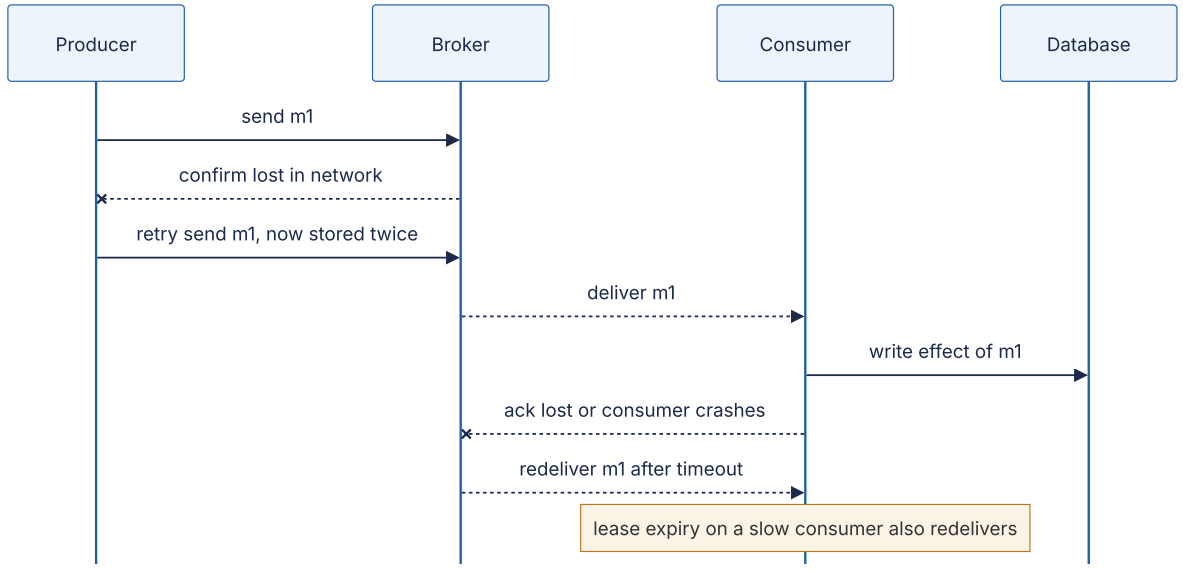
The guarantee is decided by **when the consumer acknowledges relative to its side effect**, and by whether the producer retries sends:

- Acknowledge, then process → at-most-once. A crash between the two loses the message.
- Process, then acknowledge → at-least-once. A crash between the two causes redelivery of a message whose effect already happened.
- Process idempotently, then acknowledge → effectively-once. Redeliveries happen, but the second processing changes nothing.

There is no fourth option in which the broker alone delivers each message exactly once to an arbitrary side effect: the consumer's write to its database and its acknowledgment to the broker are two different systems, and one can succeed while the other fails (the two generals problem, §24.3.3). "Exactly-once" features narrow the window rather than close it.

30.8.1 Where duplicates come from

Figure 30.10 — Three sources of duplicates.



Reading the diagram. First, the *producer* sends m1, the broker stores it, but the confirmation is lost; the producer cannot distinguish this from a lost send, retries, and the broker stores a second copy. Second, the *consumer* writes m1's effect to its database, then crashes or loses the acknowledgment; the broker redelivers m1 after the visibility timeout. Third (the note), a lease that expires while a slow consumer is still working hands the message to a second consumer (Figure 30.3). Solid arrows are synchronous calls; crossed arrows are failed messages. The storage boundary that matters is the consumer's database: it already holds m1's effect when the duplicate arrives. Rebalancing in partitioned logs is a fourth source (Chapter 31).

FORMULA

Expected duplicates under at-least-once delivery

$$E[\text{deliveries}] = \frac{1}{1-p}, \qquad r_{\text{dup}} = \lambda \cdot \frac{p}{1-p}$$

Symbol	Meaning	Units
p	Probability that a processed delivery is not successfully acknowledged (crash, lost ack, lease expiry), independent per attempt	dimensionless
$E[\text{deliveries}]$	Mean deliveries per message until one is acknowledged	deliveries
λ	Message rate	messages/s
r_{dup}	Rate of duplicate processings	duplicates/s

WORKED EXAMPLE

Worked example — How rare is rare? A notification consumer handles $\lambda = 10,000$ messages/s; one delivery in a thousand ends without an effective acknowledgment ($p = 0.001$).

1. $E[\text{deliveries}] = 1/0.999 = 1.001$.
2. $r_{\text{dup}} = 10,000 \times 0.001/0.999 \approx 10$ duplicates per second.
3. Per day: $10 \times 86,400 \approx 864,000$ duplicate notifications.

Interpretation. A 0.1% failure rate sounds negligible but produces nearly a million duplicates a day at this scale. **Engineering implication.** Every at-least-once consumer must be idempotent: record processed message IDs (or business keys) in the same transaction as the effect, or make the effect naturally idempotent (an upsert, a “set status to SHIPPED”). The full design — key scope, storage, retention and concurrency — is in §22.15; consumer patterns are in §32.10.

Table 30.6 — Delivery guarantees compared.

Guarantee	Mechanism	Loses messages?	Duplicates?	Use for
At-most-once	Ack before processing; no producer retries	Yes, on failure	No	Metrics samples, live presence
At-least-once	Ack after processing; producer retries	No	Yes, on failure	The default for business work
Effectively-once	At-least-once plus idempotent or transactional processing	No	Delivered, but without repeated effect	Payments, inventory, counters

Broker features help at specific boundaries: SQS FIFO deduplicates *sends* within 5 minutes; Pub/Sub exactly-once delivery prevents redelivery after a successful acknowledgment; Kafka's idempotent producer and transactions make read-process-write *within Kafka* atomic (§31.11). None of them can make your consumer's call to a payment provider or an email service happen once; only idempotency at that boundary can, and §32.9 discusses the limits [Kleppmann 2017, ch. 11; Helland 2007].

COMMON MISTAKE

"We use a broker with exactly-once delivery, so our consumer need not be idempotent." Broker-level guarantees stop at the broker. A consumer that sends an email and then crashes before acknowledging will send it again on redelivery, whatever the broker promised.

30.9 Ordering

DEFINITION

Definition — Message ordering. Global (total) ordering: all consumers observe all messages in one sequence. **Per-key (partial) ordering**: messages that share a key — an account, an order, a device — are processed in send order, while messages with different keys may be processed in any relative order.

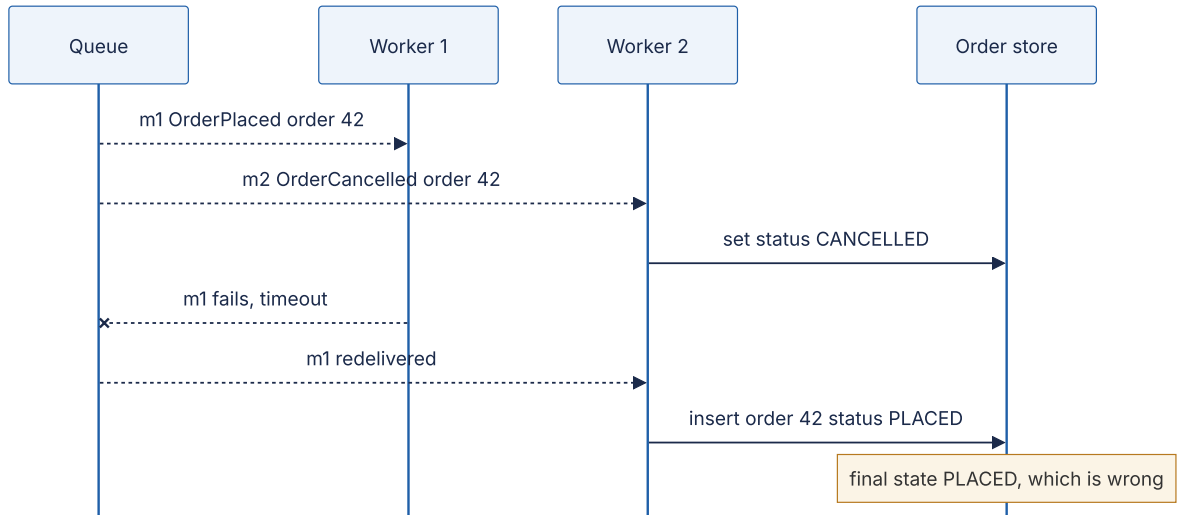
Global ordering requires a single sequence and, in effect, a single consumer at a time: its throughput is that of one consumer. Almost no business process needs it. What it needs is per-key ordering: `OrderPlaced` before `OrderCancelled` *for the same order*. Brokers express the key differently — the SQS FIFO message group ID, the Pub/Sub ordering key, the Pulsar key in a key_shared subscription, the Kafka partition key (§31.5) — but the guarantee is the same: same key, same lane, one at a time.

30.9.1 What breaks ordering

Even a broker that stores messages in order can deliver them out of order to your code:

1. **Parallel consumers.** With competing consumers, m1 goes to a slow worker and m2 to a fast one; m2's effect lands first.
2. **Retries and redelivery.** m1 fails and is redelivered after its visibility timeout, by which time m2 has been processed. Kleppmann notes that combining load balancing with redelivery inevitably reorders messages [Kleppmann 2017, ch. 11].
3. **Producer retries.** The producer sends m1, the confirmation times out, it sends m2, then retries m1: the broker now holds m2 before m1. (Kafka's idempotent producer prevents this within a partition; §31.6.)
4. **Multiple producers.** Two instances publishing events for the same key have no common clock; "send order" is undefined (§24.9).
5. **Dead-lettering.** A message moved to the DLQ and redriven later lands behind newer messages for its key.

Figure 30.11 — How parallel consumers and redelivery reorder messages for one key.



Reading the diagram. Both messages concern order 42 but go to different workers. Worker 2 applies the cancellation first; worker 1's attempt at m1 fails, and m1 is redelivered after the cancellation, so the order ends up "PLACED". Deliveries are asynchronous; writes to the order store (the storage boundary) are synchronous. Fixes: route both messages through one ordered lane (same group or partition key), or make the consumer **order-tolerant** — store a per-order version or sequence number and reject updates older than the stored one, which also defends against the other causes above.

FORMULA

Throughput limit of per-key ordering

$$X_{\text{key}} \le \frac{1}{S}, \qquad X_{\text{total}} \le \frac{\min(G, c)}{S}$$

Symbol	Meaning	Units
X_{key}	Maximum processing rate for one key (group)	messages/s
S	Mean processing time per message	s
X_{total}	Maximum processing rate for the queue	messages/s
G	Number of keys with messages waiting	count
c	Total consumer slots (instances × concurrency)	count

WORKED EXAMPLE

Worked example — A hot account in a FIFO queue. Billing events are grouped by account; $S = 50$ ms; there are $c = 200$ consumer slots and thousands of active accounts.

1. Per account: $X_{\text{key}} \le 1/0.05 = 20$ messages/s.
2. Queue: $X_{\text{total}} \le 200/0.05 = 4,000$ messages/s.
3. One large customer sends 50 messages/s. Its backlog grows by $50 - 20 = 30$ per second: after 10 minutes, 18,000 messages, which take $18,000/20 = 900$ s to process even after the burst stops.

Interpretation. Ordering turns a hot key into a single-threaded bottleneck that no amount of consumer scaling fixes. **Engineering implication.** Choose the narrowest key that preserves the invariant you need (order ID rather than customer ID), reduce S for hot keys (batch updates per key), and monitor backlog per key or group, not only per queue.

COMMON MISTAKE

"A FIFO queue guarantees the database sees updates in order." It guarantees *delivery* order per group. If your consumer hands messages to a thread pool, or performs writes asynchronously, it reintroduces the reordering the broker removed.

30.10 Retries and poison messages

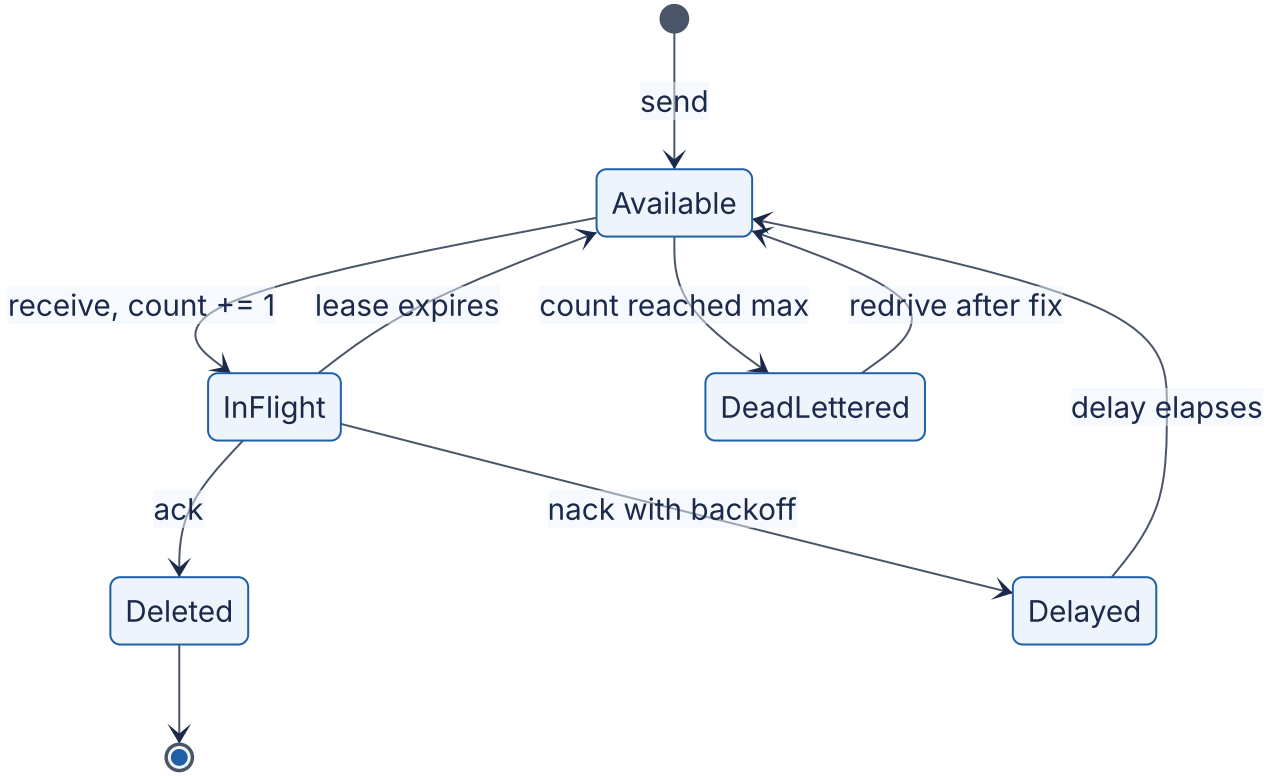
DEFINITION

Definition — Poison message. A message that fails processing every time it is delivered — because it is malformed, refers to data that does not exist, or triggers a bug — and that, if redelivered indefinitely, wastes consumer capacity and (in ordered lanes) blocks every message behind it.

Consumer failures come in two kinds, and they need opposite treatment. **Transient** failures — a downstream timeout, a throttled API, a database failover — succeed if you wait and retry. **Permanent** failures — a corrupt JPEG, a schema violation, a reference to a deleted user — fail identically on every attempt. A good consumer retries the first kind with backoff and stops retrying the second kind quickly.

1. **Classify the error.** Treat validation and parsing errors as permanent: send the message to the dead-letter queue (or an "invalid message channel" [Hohpe and Woolf 2003]) on the first attempt. Treat timeouts, 429s and 5xx responses as transient.
2. **Back off.** Retrying a transient failure immediately just repeats it, and a fleet of consumers retrying together hammers the recovering dependency. Use exponential backoff with jitter (§22.12). Brokers provide the delay in different ways: change the message's visibility timeout to the backoff delay (SQS), negatively acknowledge with a retry policy (Pub/Sub, Pulsar), or republish to delay or retry topics (the common Kafka pattern, below).
3. **Cap attempts.** A **maximum receive count** (SQS `maxReceiveCount`, the quorum-queue delivery limit, Pub/Sub maximum delivery attempts, Pulsar `maxRedeliverCount`) bounds the damage. After it, the broker moves the message to a dead-letter queue (§30.11).

Figure 30.12 — Lifecycle of a message with retries and a delivery limit.



Reading the diagram. A sent message is *Available*. Each receive makes it *InFlight* and increments its receive count. An acknowledgment deletes it. A negative acknowledgment with a backoff, or a visibility change, makes it *Delayed*; a lease that expires without an ack makes it *Available* again immediately. When an available message has already been received the maximum number of times, the broker moves it to the dead-letter queue instead of delivering it. A redrive returns it to *Available*, usually with a reset count. All transitions occur inside the broker's storage; the failure points are the two paths back to *Available*, which are where duplicates and reordering arise.

FORMULA

Time and work before a poison message is dead-lettered

$$T_{\text{DLQ}} \approx \sum_{k=1}^{R} (S_k + d_k), \qquad W_{\text{wasted}} = R \cdot \bar{S}$$

Symbol	Meaning	Units
T_{DLQ}	Time from first delivery until the message reaches the DLQ	s
R	Maximum receive count (delivery limit)	deliveries
S_k	Time attempt k runs before failing (up to the visibility timeout if the consumer hangs or crashes)	s
d_k	Delay before the next delivery (backoff; 0 if the lease simply expires)	s
W_{wasted}	Consumer time spent on the message	slot-seconds
$\bar{S}$	Mean time per failed attempt	s

WORKED EXAMPLE

Worked example — A corrupt video in a FIFO lane. The transcoding queue has visibility timeout $V = 60$ s and $R = 5$. A corrupt video makes the transcoder hang until its lease expires.

1. Each attempt costs the full lease: $S_k = 60$ s, $d_k = 0$. $T_{\text{DLQ}} \approx 5 \times 60 = 300$ s.
2. If the consumer instead detects the failure in 2 s and nacks with exponential backoff of 10, 20, 40, 80 and 160 s: $T_{\text{DLQ}} \approx 5 \times 2 + 310 = 320$ s — similar time, but only 10 s of consumer work instead of 300 s.
3. In a FIFO queue grouped by user, that user's other uploads wait behind the poison message for the whole 5 minutes.

Interpretation. Retries cost time and capacity, and in ordered lanes they cost *other messages'* latency. **Engineering implication.** Fail fast on permanent errors; use backoff for transient ones; keep R small (3–5) for ordered queues and larger (10–20) for unordered queues whose dependencies have longer outages. RabbitMQ 4.0 made a delivery limit of 20 the default for quorum queues for exactly this reason [RabbitMQ Docs: Quorum Queues].

INDUSTRY EXAMPLE

Retry topics at Uber. Uber's engineers describe a Kafka-based design in which a consumer that fails a message publishes it to a sequence of **retry topics** with increasing delays, each consumed by a retry consumer that waits until the message's delay has elapsed, and finally to a **dead-letter topic** for manual inspection. Failed messages therefore do not block the main topic's partitions, and the main consumers keep processing new messages [Xia 2018]. *Evidence: Primary* — [Xia 2018]

COMMON MISTAKE

"Infinite retries guarantee eventual success." They guarantee eventual success only for transient errors. For poison messages they guarantee wasted capacity forever and, in ordered lanes, a permanent stall.

30.11 Dead-letter queues

30.11.1 Redrive, alerting and replay

DEFINITION

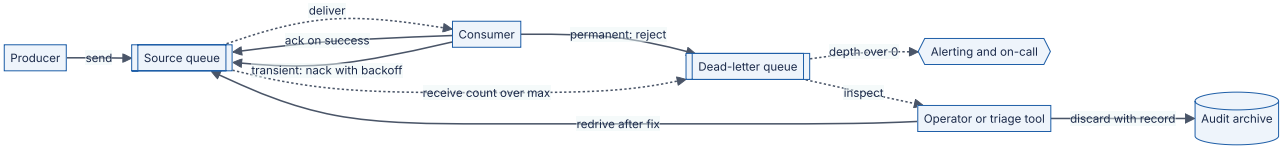
Definition — Dead-letter queue (DLQ). A separate queue or topic to which a broker or consumer moves messages that cannot be processed (delivery limit exceeded, rejected as invalid, expired), preserving them for inspection and later reprocessing instead of losing or endlessly retrying them [Hohpe and Woolf 2003, "Dead Letter Channel"].

A DLQ is a hospital ward, not a graveyard. Its purpose is to get sick messages out of the way of healthy ones while keeping them safe until someone diagnoses and fixes the cause. That gives it three operational duties:

1. **Alerting.** Any message in a DLQ means something is broken. Alert on DLQ depth greater than zero (or on its rate of arrival for noisy domains) and on the age of its oldest message. A DLQ nobody watches converts errors into silent data loss, delayed by the retention period.
2. **Diagnosis.** Keep enough context to understand the failure: the original queue, the receive count, the timestamps and, ideally, the last error. RabbitMQ adds an `x-death` header recording the reason and source queue; Pulsar and Kafka retry patterns add headers; with SQS, consumers should log the error with the message ID before giving up.

3. **Redrive and replay.** After the bug is fixed or the missing data exists, move messages back to the source queue (SQS offers a managed DLQ redrive that moves messages back with a configurable rate; Pub/Sub messages can be republished from the dead-letter subscription; RabbitMQ messages can be shovelled back). Redriven messages arrive *behind* newer messages for the same key, so the consumer must tolerate reordering (§30.9). Retention matters: a message moved from a standard SQS queue keeps its original enqueue timestamp in the DLQ, so set the DLQ's retention longer than the source queue's [AWS Docs: SQS].

Figure 30.13 — Retry, dead-letter and redrive flow.



Reading the diagram. Messages flow from producer to source queue to consumer. The consumer acknowledges successes, negatively acknowledges transient failures with a backoff delay, and rejects permanent failures straight to the DLQ; the broker itself moves messages that exceed the receive count. The DLQ triggers an alert asynchronously and is inspected by an operator or triage tool, which either redrives messages to the source queue after a fix or discards them with a record in an audit archive (the storage boundary for deliberate drops). Failure points: an unmonitored DLQ, a DLQ with shorter retention than the source, and a redrive that re-poisons the consumer because the fix was not deployed. Scaling: redrive at a controlled rate so the backlog does not overload downstream services.

30.11.2 A runnable in-memory queue with visibility timeout and DLQ

The following program implements the mechanics of this chapter in about 100 lines plus tests: competing consumers, leases, receipts, acknowledgments, heartbeats, backoff via visibility changes, a maximum receive count, a dead-letter queue and redrive. It uses a fake clock so the tests can "wait" instantly. Run it with `python3`; it prints `all queue tests passed`.

```
        """In-memory queue with visibility timeout, acks, redelivery and a dead-letter
queue."""
        import heapq
        import itertools
        from dataclasses import dataclass


        class FakeClock:
            """A controllable clock so tests can 'wait' without sleeping."""
            def __init__(self):
                self.now = 0.0

            def __call__(self):
                return self.now

            def advance(self, seconds):
                self.now += seconds


        @dataclass
        class _Message:
            id: int
            body: str
            receive_count: int = 0
            receipt: str = ""      # receipt of the latest delivery; "" = never deliv-
ered
            version: int = 0       # bumps on every reschedule; stale heap entries are
skipped


        @dataclass(frozen=True)
        class Delivery:
            receipt: str
            body: str
            receive_count: int


        class StaleReceipt(Exception):
            """The message was acked already or redelivered under a newer receipt."""


        class InMemoryQueue:
            def __init__(self, clock, visibility_timeout=30.0, max_receive_count=None,
dlq=None):
                if max_receive_count is not None and dlq is None:
                    raise ValueError("max_receive_count needs a dead-letter queue")
                self.clock, self.vt = clock, visibility_timeout
                self.max_receive_count, self.dlq = max_receive_count, dlq
                self._msgs = {}        # id -> _Message, for every message not yet acked
                self._heap = []        # (visible_at, seq, id, version): who becomes
visible when
                self._seq = itertools.count()
                self._ids = itertools.count(1)

            def _make_visible_at(self, msg, t):
                msg.version += 1
```

```
                heapq.heappush(self._heap, (t, next(self._seq), msg.id, msg.version))

            def send(self, body, delay=0.0):
                msg = _Message(next(self._ids), body)
                self._msgs[msg.id] = msg
                self._make_visible_at(msg, self.clock() + delay)
                return msg.id

            def receive(self, max_messages=1):
                now, out = self.clock(), []
                while self._heap and self._heap[0][0] <= now and len(out) <
max_messages:
                    _, _, mid, version = heapq.heappop(self._heap)
                    msg = self._msgs.get(mid)
                    if msg is None or msg.version != version:
                        continue                                    # acked or rescheduled
                    if self.max_receive_count and msg.receive_count >=
self.max_receive_count:
                        del self._msgs[mid]                         # poison: move, don't
deliver
                        self.dlq.send(msg.body)
                        continue
                    msg.receive_count += 1
                    msg.receipt = f"{mid}:{msg.receive_count}"      # new receipt per de-
livery
                    self._make_visible_at(msg, now + self.vt)       # in flight until then
                    out.append(Delivery(msg.receipt, msg.body, msg.receive_count))
                return out

            def _current(self, receipt):
                msg = self._msgs.get(int(receipt.split(":")[0]))
                if msg is None or msg.receipt != receipt:
                    raise StaleReceipt(receipt)
                return msg

            def ack(self, receipt):
                del self._msgs[self._current(receipt).id]

            def change_visibility(self, receipt, timeout):
                """Extend a lease (heartbeat) or, with a delay, schedule a retry
(nack)."""
                self._make_visible_at(self._current(receipt), self.clock() + timeout)

            def depth(self):
                return len(self._msgs)


        def redrive(dlq, target, limit=None):
            """Move messages from a dead-letter queue back to a source queue."""
            moved = 0
            while limit is None or moved < limit:
                batch = dlq.receive(1)
                if not batch:
                    return moved
                target.send(batch[0].body)
                dlq.ack(batch[0].receipt)
                moved += 1
```

```
            return moved


        def test_queue():
            clock = FakeClock()
            dlq = InMemoryQueue(clock, visibility_timeout=60)
            q = InMemoryQueue(clock, visibility_timeout=30, max_receive_count=3,
dlq=dlq)

            # 1. Competing consumers get different messages; ack deletes.
            q.send("resize 1"); q.send("resize 2")
            a, b = q.receive(1)[0], q.receive(1)[0]
            assert (a.body, b.body) == ("resize 1", "resize 2") and q.receive(1) == []
            q.ack(a.receipt); q.ack(b.receipt)
            assert q.depth() == 0

            # 2. A crash (no ack) hides the message until the timeout, then redelivers
it.
            q.send("thumb 9")
            first = q.receive()[0]
            clock.advance(29); assert q.receive() == []
            clock.advance(1); second = q.receive()[0]
            assert second.body == "thumb 9" and second.receive_count == 2
            try:
                q.ack(first.receipt); raise AssertionError("stale receipt accepted")
            except StaleReceipt:
                pass                                    # the slow worker's ack is re-
fused
            q.ack(second.receipt)

            # 3. A heartbeat keeps a long job invisible; a nack with delay is a backoff
retry.
            q.send("transcode")
            d = q.receive()[0]
            for _ in range(4):
                clock.advance(20); q.change_visibility(d.receipt, 30)
                assert q.receive() == []
            q.change_visibility(d.receipt, 5)           # failed: retry in 5 s
            clock.advance(4); assert q.receive() == []
            clock.advance(1); d = q.receive()[0]; q.ack(d.receipt)

            # 4. A poison message moves to the DLQ on the receive after the 3rd deliv-
ery.
            q.send("corrupt.jpg")
            for attempt in (1, 2, 3):
                d = q.receive()[0]
                assert d.receive_count == attempt
                clock.advance(30)                       # consumer crashes each time
            assert q.receive() == [] and q.depth() == 0 and dlq.depth() == 1

            # 5. After a fix, redrive puts it back on the source queue.
            assert redrive(dlq, q) == 1 and dlq.depth() == 0
            d = q.receive()[0]
            assert d.body == "corrupt.jpg" and d.receive_count == 1
            q.ack(d.receipt)
            print("all queue tests passed")
```

```
if __name__ == "__main__":
    test_queue()
```

What to notice:

- **One heap for all timing.** Newly sent, delayed, and in-flight messages all sit in the heap keyed by the time they become visible. A redelivery is not a special code path: an in-flight message whose lease expires simply reaches the top of the heap again.
- **Lazy invalidation.** Acknowledging or rescheduling does not search the heap; it bumps the message's `version` (or deletes it), and `receive` skips heap entries whose version no longer matches.
- **A new receipt per delivery.** `ack` and `change_visibility` require the *latest* receipt, so a slow worker whose lease expired cannot delete a message that another worker now holds. (SQS has a weaker rule — a delete with an old receipt handle may succeed without deleting the message — so do not rely on stale handles there either.)
- **Dead-lettering happens on receive.** As in SQS, a message is moved when it becomes available after its last permitted delivery, not at the moment of failure.
- **What it leaves out:** durability, replication, per-group ordering and concurrency control. Adding a `group_id` that blocks later messages of a group while one is in flight is practical exercise 1.

> RECOMMENDATION
>
> **DLQ checklist.** One DLQ per source queue (or per subscription), of the same type (FIFO for FIFO). Retention longer than the source's. An alert on depth greater than zero routed to the owning team. A runbook for triage, redrive and discard. A rate-limited redrive tool. A dashboard of DLQ arrivals by error class, because a sudden spike usually means a bad deploy rather than bad messages.

30.12 Kafka vs RabbitMQ (and SQS, Pulsar)

30.12.1 Queues versus logs

Brokers fall into two families [Kleppmann 2017, ch. 11]. **Queue-style brokers** (RabbitMQ classic and quorum queues, SQS) track each message individually and delete it once acknowledged; consumers share work message by message, and replay is impossible once a message is gone. **Log-based brokers** (Kafka, Pulsar, RabbitMQ streams) append messages to a partitioned, retained log; a consumer is just a position (an **offset**) in each partition, so many consumer groups can read the same data independently and any of them can rewind and replay. Parallelism in a log is per partition: within a consumer

group each partition is read by one consumer, which gives per-partition ordering for free but means a slow message holds up its whole partition.

Apache Kafka, created at LinkedIn [Kreps et al. 2011], is the dominant log-based broker. As of 2026 Kafka 4.x runs without ZooKeeper (metadata is managed by its own Raft-based KRaft quorum), and **share groups** (KIP-932, “Queues for Kafka”), introduced in the 4.x line, add queue-style consumption with per-message acknowledgment and delivery counts on top of the log. Chapter 31 covers partitions, replication, offsets, transactions and performance.

Table 30.7 — Kafka, RabbitMQ, SQS and Pulsar compared (as of 2026).

	Kafka	RabbitMQ	Amazon SQS (+ SNS)	Apache Pulsar
What it is	Distributed, partitioned, replicated log	General-purpose broker with flexible routing	Fully managed queue (and topic) service	Log-based broker with separate storage layer
How it works	Append to partitions; consumers track offsets	Exchanges route to queues; per-message acks; quorum queues use Raft	HTTPS API; leases via visibility timeout	Stateless brokers over BookKeeper ledgers; durable cursors
Advantages	Very high throughput, replay, many independent readers, stream processing ecosystem	Rich routing, per-message acks, priorities, low latency, many protocols	No operations, elastic scale, pay per use, DLQ and redrive built in	Queue and stream semantics, independent scaling of serving and storage, geo-replication, tiered storage
Disadvantages	Per-partition head-of-line blocking; partition count limits parallelism; no per-message retry (until share groups)	Per-queue throughput bound by one node; large backlogs are costly; replay only with streams	AWS only; standard queues unordered and duplicate-prone; FIFO throughput quotas	Most moving parts (brokers, bookies, metadata store); smaller ecosystem
Performance	Hundreds of MB/s per broker with batching; latency from milliseconds	Low-millisecond latency; tens of thousands of msg/s per queue (order of magnitude)	Effectively unlimited for standard; per-group serial for FIFO; tens of ms per call	Comparable to Kafka; low publish latency
Operational complexity	Medium–high (fewer parts since KRaft)	Medium	Very low	High
Cost	Infrastructure, or managed services	Infrastructure, or managed services	Per request and per GB; cheap at low volume, can be costly at sustained high volume	Infrastructure, or managed services
Use cases	Event streaming, CDC, logs and metrics pipelines, event sourcing	Task queues, RPC over messaging, complex routing	Decoupling AWS services, job queues, fan-out	Multi-tenant platforms, mixed queue and stream workloads, geo-distributed messaging
Choose when	You need retention, replay and many consumers of the same high-volume stream	You need per-message routing, acks and priorities on moderate volume	You are on AWS and want zero broker operations	You need both semantics, multi-tenancy or geo-replication and can operate it

Performance figures are orders of magnitude, not benchmarks: real numbers depend on message size, batching, replication, acknowledgment settings and hardware, so measure your own workload. Google Cloud Pub/Sub sits in the same niche on Google Cloud as SQS plus SNS on AWS, with fan-out and per-subscription queues in one service.

RECOMMENDATION

A default decision path. Start with the question “does more than one consumer need the *same* data, now or later?” If yes, or if you need replay, use a log (Kafka, Pulsar, RabbitMQ streams, or Pub/Sub with retention). If no, and the work is a task to be done once, use a queue (SQS, RabbitMQ quorum queues). Then let operations decide: on a cloud provider with no platform team, prefer the managed service; at sustained high volume with a platform team, self-managed or managed Kafka is usually cheaper per message.

COMMON MISTAKE

“Kafka is a faster RabbitMQ.” They answer different questions. Kafka retains an ordered log that many readers replay; RabbitMQ routes and tracks individual messages until each is acknowledged. Using Kafka as a task queue with slow, variable jobs suffers head-of-line blocking per partition; using RabbitMQ classic queues as a long-term event archive suffers memory and disk pressure.

30.13 Summary and revision notes

SUMMARY

- A **broker** removes time, space and synchronization coupling. A synchronous chain's availability is $\prod A_i$ ($0.999^5 \approx 99.5\%$); a queued design's is front service × broker. Keep synchronous only what the caller needs now.
- **Queues** deliver each message to one of many **competing consumers**. Messages are leased (visibility timeout, ack deadline) and deleted on **acknowledgment**; size the lease as $b \cdot S_{\text{tail}} + \delta$ or use heartbeats.
- Size consumers with Little's Law, $N = \lceil \lambda S/(k\rho) \rceil$; backlogs drain at the surplus, $B/(\mu - \lambda)$. Alert on the age of the oldest message.
- **Pub/sub** copies each message to every subscription; each subscription is usually a queue. Durable subscriptions retain messages while consumers are away and cost storage when abandoned.
- **Commands** ask one receiver to act; **events** state facts to unknown subscribers. Event notification is thin and invites callbacks; event-carried state transfer is fat and self-sufficient.
- **RabbitMQ**: exchanges (direct, topic, fanout, headers), bindings, quorum queues (Raft), streams. **SQS**: standard (at-least-once, best-effort order) or FIFO (message groups, 5-minute deduplication). **Pub/Sub**: topics, pull/push/BigQuery subscriptions, ordering keys, exactly-once delivery. **Pulsar**: stateless brokers over BookKeeper ledgers, four subscription types.
- Delivery: at-most-once (ack first), at-least-once (ack after), effectively-once (at-least-once + idempotent processing, §22.15). Duplicates come from producer retries, lost acks and expired leases; at 0.1% they are $\lambda p/(1-p)$ per second.
- Ordering is per key; parallel consumers, redelivery, producer retries and redrive break it. One key processes at most $1/S$ messages per second.
- Classify failures; back off transient ones; cap deliveries; dead-letter poison messages; alert on any DLQ depth; redrive after the fix.
- Queues (RabbitMQ, SQS) track messages; logs (Kafka, Pulsar, RabbitMQ streams) retain and replay them.

30.14 Knowledge checkpoint

1. Name the three kinds of decoupling a broker provides.
2. Six services each with availability 0.998 are called in series. What is the chain's availability?
3. What happens to an SQS message whose consumer crashes before deleting it?
4. A queue receives 1,200 messages/s, processing takes 250 ms, each consumer runs 8 concurrent jobs at 75% target utilization. How many consumers?

5. Those consumers stop for 20 minutes, and capacity after restart is 1,800 messages/s. How long does draining take?
6. What is the difference between a durable and an ephemeral subscription?
7. Is `ResizeThumbnail` an event or a command? Why?
8. In a RabbitMQ topic exchange, does the routing key `photo.png.uploaded` match `photo.#` and `*.uploaded`?
9. Which three mechanisms produce duplicates in an at-least-once system?
10. Why does an SQS FIFO queue's parallelism depend on the number of message groups?
11. A poison message hangs its consumer until a 45 s lease expires, with a maximum receive count of 4. Roughly when does it reach the DLQ?
12. Why should a DLQ's retention exceed its source queue's?

Answers

1. Time, space and synchronization decoupling [Eugster et al. 2003].
2. $0.998^6 \approx 0.988$, about 8.6 hours of unavailability per 30-day month.
3. It stays invisible until the visibility timeout expires, then becomes visible and is redelivered with an incremented receive count.
4. $L = 1,200 \times 0.25 = 300$; $300/(8 \times 0.75) = 50$ consumers.
5. $B = 1,200 \times 1,200 = 1.44$ million; surplus 600/s; $T = 2,400$ s $= 40$ minutes.
6. A durable subscription retains messages while the subscriber is disconnected; an ephemeral one exists only while it is connected, so messages sent in between are lost to it.
7. A command: it is imperative and addressed to a receiver with a known capability. The event would be `PhotoUploaded`.
8. Yes to `photo.#` (`#` matches zero or more words). No to `*.uploaded`: `*` matches exactly one word, but the key has three.
9. Producer retries after a lost confirm; a consumer crash or lost acknowledgment after the side effect; a lease expiring while a slow consumer still works.
10. Messages of one group are delivered one batch at a time, in order; only different groups can be processed concurrently.
11. About $4 \times 45 = 180$ s after the first delivery, when it next becomes available.
12. Because the message's age may count from its original enqueue time; a shorter DLQ retention can delete it before anyone inspects it.

30.15 Interview questions

Basic

Q1. When would you use a message queue instead of a synchronous API call?
Answer. When the caller does not need the result to continue (sending email, generating

thumbnails), when the receiver may be down or slower than the producer, or when bursts must be smoothed. Keep it synchronous when the user needs the answer now (authorization, search). Mention the availability product, backlog monitoring and idempotent consumers.

Q2. Explain the visibility timeout. Answer. A received message is hidden for a lease period rather than deleted. If the consumer deletes it in time, it is gone; if not, it reappears for another consumer. Too short causes duplicates for slow consumers; too long delays recovery after crashes; heartbeats extend the lease for long jobs.

Intermediate

Q3. Your queue's depth has grown for an hour. Walk through the diagnosis. Answer. Compare arrival and completion rates; check consumer count, errors and processing time per message (a slow dependency raises S and so the required N); look for poison messages consuming retries and for a hot key in ordered lanes; check DLQ arrivals. Mitigate by scaling consumers within downstream limits, shedding optional work, and fixing the cause; estimate drain time with $B/(\mu - \lambda)$.

Q4. How do you guarantee an order is never charged twice when payment requests arrive by queue? Answer. Accept that delivery is at-least-once. Give each request an idempotency key (order ID plus attempt), record it in the same transaction as the charge record, and pass it to the payment provider's idempotency API; on redelivery, look up the stored outcome and acknowledge. Order-sensitive steps go through a per-order lane or check a version number.

Advanced

Q5. Design the messaging for PhotoShare's upload pipeline at 2,000 uploads/s. Answer. The upload service writes metadata and publishes `PhotoUploaded` (event-carried state: IDs, URLs, type) to a topic, ideally via an outbox (§32.5). Subscriptions per consumer: thumbnails (about 43 workers, $V = 40$ s), moderation, search, notifications (video-only filter for transcoding). At-least-once with idempotent writes keyed by photo ID. Per-photo ordering is unnecessary except for edits and deletes, which carry versions. Retries with backoff, $R = 5$, a DLQ per subscription with alerts, autoscaling on age of oldest message, retention of days.

Q6. Kafka or RabbitMQ for a job system with jobs lasting 100 ms to 30 minutes? Answer. Highly variable job lengths suit per-message leasing: RabbitMQ quorum queues (low prefetch, per-message acks, delivery limit, DLX) or SQS with heartbeats. In classic Kafka consumer groups a 30-minute job blocks its partition, and parallelism is capped by partitions; share groups or a separate job queue fed from Kafka avoid that. Choose Kafka if the same events also feed analytics and need replay.

30.16 Practical exercises

1. Add FIFO message groups to the in-memory queue: while any message of a group is in flight, `receive` must skip later messages of that group. Test that two groups proceed in parallel and that a poison message blocks only its group until dead-lettered.
2. Write a consumer loop around the queue that classifies errors as transient or permanent, retries transient ones with full-jitter backoff via `change_visibility`, and rejects permanent ones to the DLQ.
3. Simulate 2,000 messages/s with exponentially distributed processing times and measure duplicate rates for visibility timeouts of 1, 2 and 5 times the mean batch time.
4. Set up RabbitMQ locally with a topic exchange, a quorum queue with a delivery limit of 3 and a dead-letter exchange; send a message the consumer always rejects and inspect its `x-death` header.

30.17 Design challenges

1. **Order notifications for a marketplace.** 20,000 order events per second at peak feed email, SMS, push, analytics and a partner webhook. Choose broker, topology, delivery guarantee, ordering key, retry and DLQ policy, and size each consumer pool.
2. **Migrating from synchronous to asynchronous.** A checkout makes seven synchronous calls with 99.9% availability each. Decide which calls stay synchronous, compute the new availability, and design how the user learns of failures that now happen after checkout returns.

30.18 References

1. Amazon Web Services. *Amazon Simple Queue Service Developer Guide* (visibility timeout, FIFO queues, dead-letter queues and redrive, quotas). https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/ [AWS Docs: SQS]
2. Amazon Web Services. *Amazon Simple Notification Service Developer Guide* (fan-out, filter policies, FIFO topics). https://docs.aws.amazon.com/sns/latest/dg/ [AWS Docs: SNS]
3. Apache Software Foundation. *Apache Pulsar Documentation: Architecture Overview; Messaging Concepts*. https://pulsar.apache.org/docs/ [Apache Pulsar Docs]
4. CNCF. *CloudEvents Specification v1.0*. 2019. https://cloudevents.io/ [CNCF CloudEvents 2019]

5. Eugster, P. T., Felber, P. A., Guerraoui, R., Kermarrec, A.-M. “The Many Faces of Publish/Subscribe.” *ACM Computing Surveys* 35(2):114–131, 2003.
6. Fowler, M. “What do you mean by ‘Event-Driven’?” martinfowler.com, 2017. https://martinfowler.com/articles/201701-event-driven.html
7. Google Cloud. *Pub/Sub Documentation* (subscriptions, ack deadlines, ordering, exactly-once delivery, dead-letter topics). https://cloud.google.com/pubsub/docs [Google Cloud Docs: Pub/Sub]
8. Guo, S., Dhamankar, R., Stewart, L. “DistributedLog: A High Performance Replicated Log Service.” *IEEE ICDE*, 2017.
9. Helland, P. “Life beyond Distributed Transactions: an Apostate’s Opinion.” *CIDR*, 2007.
10. Hohpe, G., Woolf, B. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
11. Junqueira, F. P., Kelly, I., Reed, B. “Durability with BookKeeper.” *ACM SIGOPS Operating Systems Review* 47(1), 2013.
12. Kleppmann, M. *Designing Data-Intensive Applications*, ch. 11. O’Reilly Media, 2017.
13. Kreps, J., Narkhede, N., Rao, J. “Kafka: A Distributed Messaging System for Log Processing.” *NetDB Workshop*, 2011.
14. RabbitMQ. *AMQP 0-9-1 Model Explained*. https://www.rabbitmq.com/tutorials/amqp-concepts [RabbitMQ Docs: AMQP Concepts]
15. RabbitMQ. *Consumer Acknowledgements and Publisher Confirms*. https://www.rabbitmq.com/docs/confirms [RabbitMQ Docs: Confirms]
16. RabbitMQ. *Quorum Queues*. https://www.rabbitmq.com/docs/quorum-queues [RabbitMQ Docs: Quorum Queues]
17. RabbitMQ. *Streams and Super Streams*. https://www.rabbitmq.com/docs/streams [RabbitMQ Docs: Streams]
18. Slack Engineering. “Scaling Slack’s Job Queue.” Slack Engineering Blog, 2020. https://slack.engineering/scaling-slacks-job-queue/ [Slack Engineering 2020]
19. Xia, N. “Building Reliable Reprocessing and Dead Letter Queues with Apache Kafka.” Uber Engineering Blog, 2018. [Xia 2018]
20. Apache Kafka. “KIP-932: Queues for Kafka.” Kafka Improvement Proposals. https://cwiki.apache.org/confluence/display/KAFKA/KIP-932%3A+Queues+for+Kafka

30.19 Open verification items

- SQS figures as of 2026: FIFO quotas (300 calls/s basic; high-throughput mode by region); “fair queues” for standard queues (2025); DLQ enqueue-timestamp behavior for standard versus FIFO queues.
- RabbitMQ 4.0: removal of classic queue mirroring; default quorum-queue delivery limit of 20; consumer timeout default of 30 minutes.

- Pub/Sub limits: ack deadline 10–600 s; 7-day subscription and 31-day topic retention; 5–100 delivery attempts.
- Slack (2020): incident details (Redis memory exhaustion) and URL; Uber (Xia 2018) URL.
- KIP-932 share-group status in Kafka 4.x releases.
- Pulsar metadata-store alternatives (etcd, Oxia) by release.

Chapter 31 — Kafka Deep Dive

> **Level:** 3 · **Track:** A (Traditional) · **Estimated reading time:** 3 h

Chapter 30 introduced queues, publish/subscribe and delivery guarantees in general terms. This chapter takes one system apart: **Apache Kafka**, the distributed log built at LinkedIn in 2010–2011 that now sits at the center of most large event-driven architectures. Kafka is not "a queue with extra features" but a replicated, partitioned, append-only log, and nearly every behavior you will meet (per-partition ordering, consumers that rewind, durability that depends on three settings, exactly-once that stops at Kafka's edge) follows from that decision. You will follow a record from the producer's batch buffer to a consumer's committed offset and size partitions, disks and brokers with real arithmetic. Version-specific facts are stated **as of 2026** (Kafka 4.x).

31.1 Learning objectives

After this chapter you should be able to:

- Explain the log abstraction and why a shared, ordered, replayable log simplifies data integration.
- Describe a Kafka cluster in KRaft mode, the role of the controller quorum, and why ZooKeeper mode was removed.
- Size a topic's partition count from target throughput, and calculate how many keys move when the partition count changes.
- Configure a producer for throughput (batching, compression) and safety (`acks`, idempotence, in-flight limits), and explain each trade-off.
- Explain consumer groups, partition assignment, offsets and commit strategies, and derive at-least-once or at-most-once behavior from commit timing.
- Trace replication with ISR, log end offset, high watermark and leader epochs; choose `min.insync.replicas`; and explain what unclean leader election costs.
- Choose between time/size retention and log compaction.
- Explain Kafka transactions and `read_committed`, and state precisely where exactly-once ends.
- Compare eager, cooperative and KIP-848 rebalancing.
- Explain Kafka's throughput (sequential I/O, page cache, zero-copy, end-to-end batching) and estimate broker count, disk capacity, lag drain time and end-to-end latency.
- Place Kafka Streams, Kafka Connect and tiered storage in an architecture, and monitor a cluster.

31.2 Prerequisites

- Log-structured storage and compaction — §13.7 and §13.9 (a Kafka partition is a log-structured file set).
- Page cache, `mmap` and `fsync` — §6.6.6 and §6.14.3 (Kafka delegates caching and flushing to the OS).
- Sequential versus random I/O — §5.6 (the root of Kafka's speed).
- Hash partitioning and hot partitions — §14.6 and §14.10.
- Raft, ZooKeeper and fencing tokens — §26.7, §26.8 and §26.13 (KRaft, leader epochs and producer epochs are all fencing schemes).
- Primary–backup replication and failover — §27.4 and §27.11 (Kafka's ISR replication is a variant).
- Two-phase commit — §28.4 (Kafka transactions use it, §28.4.8).
- Idempotency keys and backpressure — §22.15 and §22.10.
- Messaging vocabulary — Chapter 30 (queues, pub/sub, delivery guarantees, ordering, dead-letter queues).
- Units: **KB/MB/GB/TB** are powers of ten in all estimates; **KiB/MiB/GiB** appear only where Kafka's defaults are powers of two (for example, 1 GiB segments). Network speeds are in bits per second: 25 Gb/s = 3.125 GB/s.

31.3 The log abstraction

31.3.1 What a log is

> DEFINITION
>
> **Definition — Log.** An append-only, totally ordered sequence of records. Each record is assigned a monotonically increasing integer position called its **offset** when it is appended. Records are never modified in place; readers read sequentially from any offset they choose.

First-principles explanation

Think of a ship's logbook. Entries are written one after another, each line gets the next number, nobody erases a line, and anyone can read from line 1 or from line 9,000. Two readers can be at different places in the book at the same time without disturbing each other or the writer. If a reader loses its place, it only needs to remember one number.

A database's write-ahead log (§13.5) is a log hidden inside one machine; Raft's replicated log (§26.7) is a log copied across machines to agree on an order. Kafka exposes the same structure to applications, partitions it for scale and replicates it for durability.

Why does this exist?

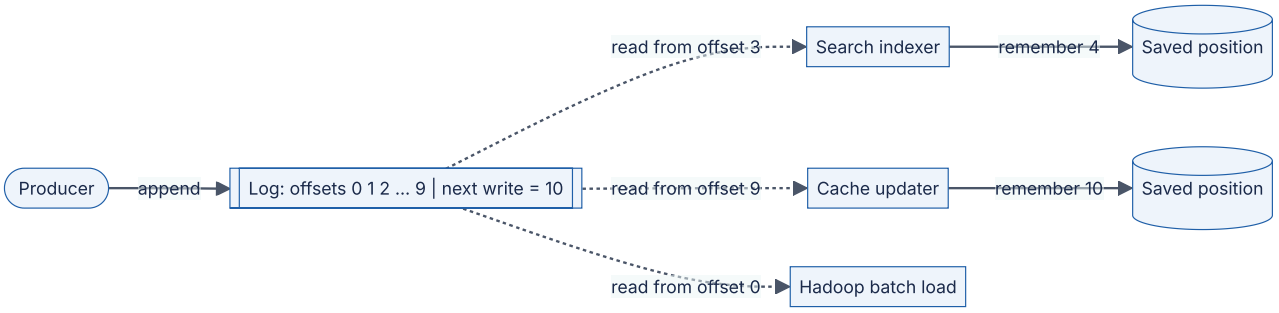
At LinkedIn around 2010, dozens of systems needed the same data: page views, profile updates and metrics flowed into Hadoop, search indexes, the social graph, monitoring and recommendation services. Each pair of systems had its own custom pipeline, so N sources and M destinations needed up to $N \times M$ integrations, each with its own failure modes. Existing message brokers kept messages in memory, deleted them once consumed, and handled neither the volume (billions of events per day) nor slow batch consumers such as Hadoop that read once an hour [Kreps et al. 2011].

Jay Kreps's 2013 essay "The Log" argued for a single abstraction: put every change into an ordered log and let every system **subscribe** at its own pace [Kreps 2013]. Integrations fall from $N \times M$ to $N + M$; a new consumer can **replay** history to build its own copy; a slow consumer does not slow the producer, because the log holds data on disk. And because the order is fixed, two consumers that apply the same log reach the same state: **state machine replication** (§26.3.2) applied to whole data systems.

How it works internally

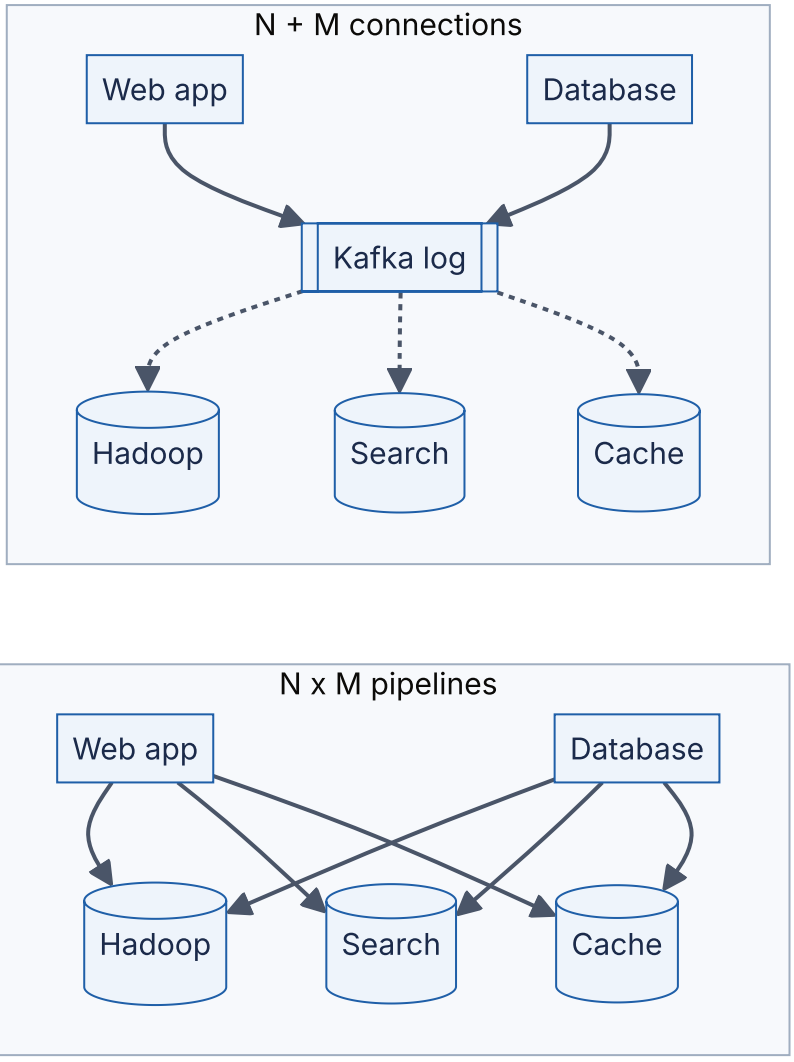
1. A producer appends a record; the log assigns the next offset (0, 1, 2, …).
2. The record is stored durably and becomes visible to readers.
3. Each consumer keeps its own **position**: the offset of the next record it will read.
4. Records are deleted only by a **retention policy** (age, size or compaction, §31.10), never because they were read.

Figure 31.1 — One log, one writer position, many independent readers.



Reading the diagram. The *producer* appends synchronously to the end of the *log*; the next write will get offset 10. Three consumers read asynchronously (dotted arrows) at different positions: the *search indexer* is at offset 3, the *cache updater* is fully caught up, and the *Hadoop batch load* is replaying from the beginning. Each consumer stores only one number, its position, in its own storage boundary. Failure points: a crashed consumer restarts from its saved position and loses nothing; if retention deletes offsets a consumer has not yet read, that consumer has fallen too far behind. Scaling: readers are independent, so adding a reader costs read bandwidth but no coordination with the writer.

Figure 31.2 — Point-to-point integration versus a central log.



Reading the diagram. On the left, every source has a custom pipeline to every destination: six links, each a separate failure point and format. On the right, sources write once to the *Kafka log* and destinations subscribe asynchronously, so a new destination adds one link and can backfill by replaying. The log is a storage boundary decoupling producers and consumers in time and speed; it is also the new single failure point, which is why the rest of the chapter replicates and partitions it.

INDUSTRY EXAMPLE

LinkedIn's activity pipeline. Kafka was built at LinkedIn to collect user activity events and operational metrics for both online consumers and offline Hadoop loads, replacing per-system pipelines. The original design deliberately used a pull model, stored messages in segment files on disk, kept no per-message acknowledgment state on the broker (consumers track offsets), and relied on the OS page cache [Kreps et al. 2011]. Later work added intra-cluster replication and described its design and LinkedIn's production use [Wang et al. 2015]. *Evidence: Primary* — [Kreps et al. 2011]; [Wang et al. 2015]

Common misconceptions

COMMON MISTAKE

"Kafka is a queue, so a message disappears once read." In a classic queue (§30.4) consumption deletes the message. In Kafka, reading only advances that consumer's offset; the record stays until retention removes it, and other consumer groups read it independently. This is why Kafka can replay, and also why "who has processed what" is the consumer's responsibility (§31.8). Kafka 4.x adds share groups for queue-like consumption (§31.7.4), but the log underneath is unchanged.

31.4 Brokers and clusters: ZooKeeper mode versus KRaft

31.4.1 Brokers, controllers and metadata

DEFINITION

Definition — Broker and controller. A **broker** is a Kafka server process that stores partition replicas and serves produce and fetch requests. The **controller** is the component that owns cluster **metadata** (which brokers are alive, which topics and partitions exist, which replica leads each partition, and the in-sync replica sets) and makes decisions such as leader election when a broker fails.

First-principles explanation

A cluster is like a library with many branches, each storing some books (partitions). Someone must keep the catalog of which branch holds the authoritative copy of each book and who takes over when a branch closes. That is the controller, and because the catalog must never have two conflicting versions, the controller itself is a consensus problem (Chapter 26).

Why does this exist? From ZooKeeper to KRaft

From 2011 until KRaft became production-ready in 2022 (and in many clusters until 4.0 removed the option), Kafka stored its metadata in **Apache ZooKeeper** (§26.8). One broker won an election in ZooKeeper to become the controller; it read all metadata from ZooKeeper, made decisions, wrote them back, and pushed them to other brokers with RPCs. This had three costs that grew with scale [KIP-500]:

1. **Two systems to operate**, each with its own configuration, security, monitoring and upgrades.
2. **Slow controller failover.** A new controller had to load every partition's state from ZooKeeper before acting, so recovery time grew linearly with partition count, capping cluster size.
3. **Divergence.** Brokers held cached metadata updated by RPC, which could disagree with ZooKeeper.

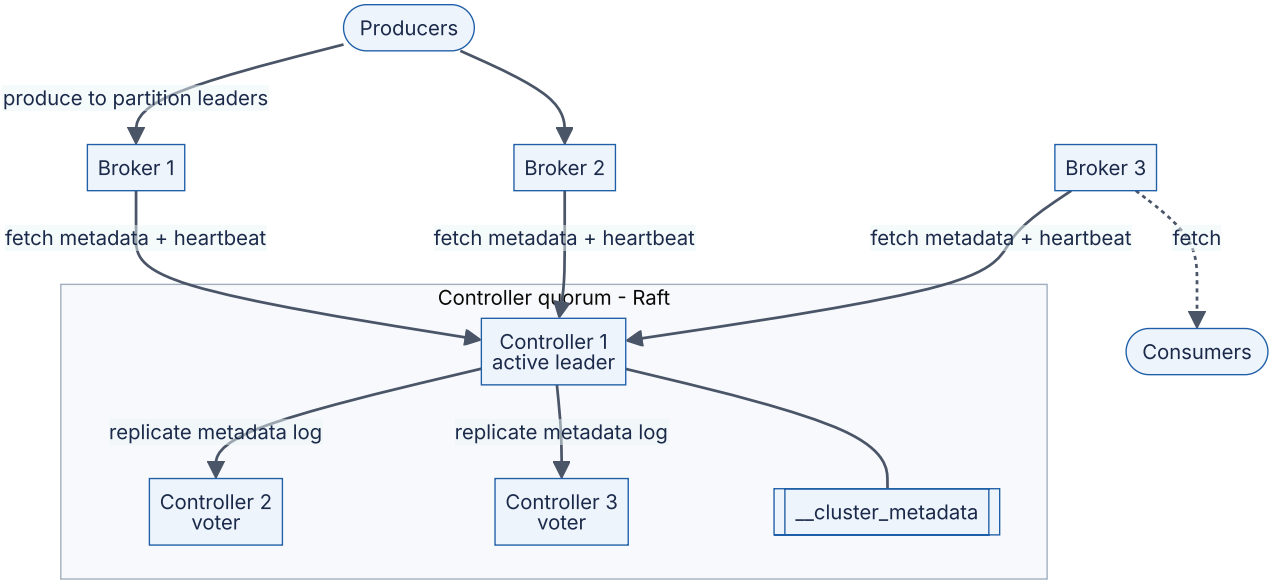
KIP-500 proposed replacing ZooKeeper with a **self-managed metadata quorum** that stores metadata as a Kafka log replicated with a Raft variant. The result, **KRaft** (Kafka Raft), was marked production-ready in Kafka 3.3 (2022); ZooKeeper mode was deprecated in 3.5, and **Kafka 4.0 (March 2025) removed ZooKeeper mode entirely**. As of 2026, every supported Kafka release runs only in KRaft mode, and a ZooKeeper-based cluster must first migrate to KRaft on a 3.x bridge release before upgrading to 4.x [Apache Kafka 4.0 Release Notes].

How it works internally

1. A small set of **controller nodes** (typically 3 or 5) forms a Raft quorum (§26.7). Their elected Raft leader is the **active controller**; the others are hot standbys.
2. All metadata changes (topic creation, partition reassignment, leader changes, ISR changes, broker registrations, configuration) are appended as records to a single-partition internal log, `__cluster_metadata`. A change is committed when a majority of controllers has it.
3. **Brokers** fetch the metadata log from the active controller exactly as followers fetch a partition, and apply records to an in-memory metadata image. Every broker therefore sees the same ordered history; a lagging broker knows exactly how far behind it is (its metadata offset).
4. Brokers **register** with the controller and send **heartbeats**. A broker that stops heartbeating is **fenced**: the controller moves leadership of its partitions to other in-sync replicas by writing new leader records.
5. The quorum takes periodic **snapshots** of the metadata image so the log can be truncated, like Raft snapshotting (§26.7.8).
6. If the active controller fails, a standby that *already has the metadata in memory* becomes Raft leader, so failover no longer reloads state and clusters can hold far more partitions.

Development setups may run broker and controller in one process (**combined mode**); production usually runs dedicated controllers.

Figure 31.3 — A KRaft cluster: controller quorum and brokers.



Reading the diagram. The *controller quorum* replicates `__cluster_metadata` from the active controller to two voters; a record commits when two of three have it, so one controller may fail. *Brokers* pull metadata and send heartbeats over TCP; each request is synchronous, but propagation is eventually consistent, measured by each broker's metadata offset. *Producers* and *consumers* talk only to brokers. Failure points: losing a controller majority stops metadata changes (no elections, no new topics) while existing leaders keep serving; losing a broker triggers leader moves. Scaling: add brokers for capacity; controllers stay at 3 or 5.

Alternatives and trade-offs

Table 31.1 — ZooKeeper mode versus KRaft (as of 2026).

Aspect	ZooKeeper mode (removed in 4.0)	KRaft mode
Metadata store	External ZooKeeper ensemble	Internal `__cluster_metadata` log, Raft quorum
Systems to operate	Two	One
Controller failover	Reload all metadata from ZooKeeper; grows with partitions	Standby already has metadata in memory
Metadata propagation	Controller RPCs to brokers	Brokers fetch an ordered log
Partition scale	Limited by controller reload time	Much higher (KIP-500 targeted millions)
Status	Deprecated 3.5; removed 4.0 (2025)	Only mode in 4.x

Failure scenarios

What goes wrong	Why	Detection	Recovery	Prevention
Two of three controllers down	Quorum lost	`ActiveControllerCount` = 0 cluster-wide	Restore a majority	Spread controllers across zones; use 5 for two-failure tolerance
Broker isolated from controller but serving clients	Network partition	Broker fenced; clients get `NOT_LEADER`	Clients refresh metadata and move to new leaders	Leader epochs (§31.9.4) reject the stale leader's replication
Controllers co-located with overloaded brokers	Combined mode in production	Metadata commit latency rises	Separate roles	Dedicated controller nodes

COMMON MISTAKE

"KRaft means Kafka partitions are replicated with Raft." Only the *metadata* log uses Raft. Ordinary partitions still use the leader–follower ISR protocol of §31.9, which commits when all in-sync replicas have a record rather than when a majority does. The two protocols make different trade-offs, compared in Table 31.4.

31.5 Topics and partitions

31.5.1 Topics, partitions and records

DEFINITION

Definition — Topic, partition and record. A **topic** is a named category of records (for example `orders`). A topic is split into **partitions**, each an independent, ordered, replicated log with its own offsets starting at 0. A **record** carries an optional **key**, a **value** (opaque bytes), a **timestamp** and optional **headers**. Kafka guarantees order *within* a partition, not across partitions of a topic.

First-principles explanation

One logbook can be served only as fast as one machine allows, so Kafka gives a topic many logbooks (partitions) on different brokers, written and read in parallel. The price is that there is no order across them: record 5 in partition 0 and record 5 in partition 1 have no "before" relationship. Two events that must stay ordered (an order's "created" before its "paid") must land in the same partition, which is what the record key is for.

How it works internally: segments and indexes

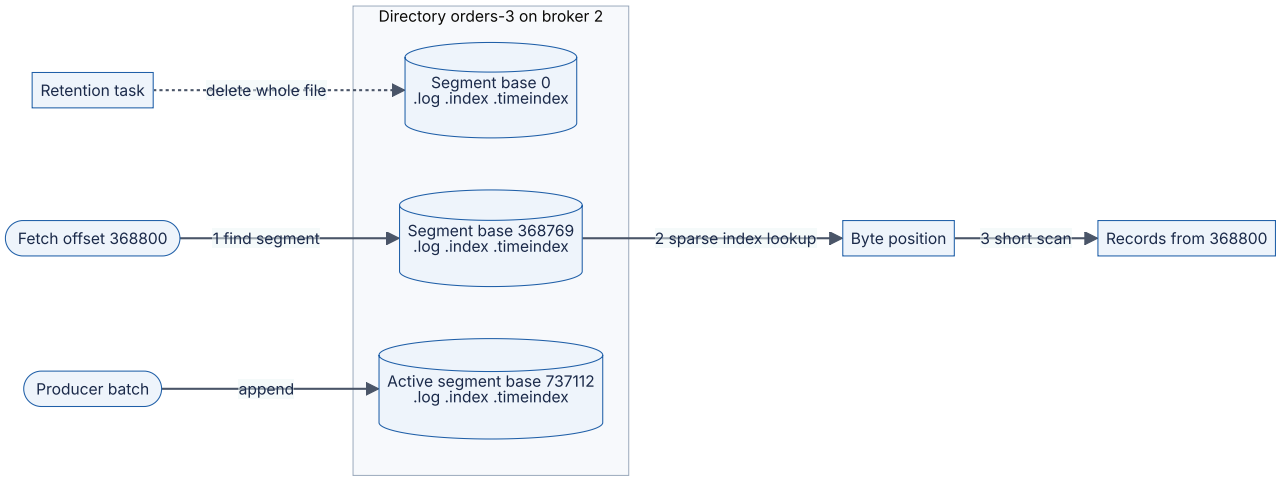
A partition is a directory on the broker's disk, named `<topic>-<partition>` (for example `orders-3/`). Inside, the log is split into **segments**, so old data can be deleted a file at a time:

1. `.log` **files** hold record batches in append order. Each file is named after its **base offset**, the first offset it contains, padded to 20 digits: `00000000000000000000.log`, `00000000000000368769.log`.
2. Only the newest segment, the **active segment**, receives appends. It is **rolled** (closed and a new one opened) when it reaches `segment.bytes` (default 1 GiB) or `segment.ms` (default 7 days).
3. `.index` **files** are **sparse** maps from offset to byte position in the `.log` file. Kafka adds an entry roughly every `index.interval.bytes` (default 4,096 bytes) of appended data, so the index is small enough to memory-map.
4. `.timeindex` **files** map timestamps to offsets, which lets a consumer start "from 09:00 yesterday".
5. `.txnindex` **files** record aborted transactions (§31.11), and a `leader-epoch-checkpoint` file records where each leader epoch began (§31.9.4).

To read offset 368,800, the broker (a) binary-searches the segment list by base offset and picks `00000000000000368769.log`, (b) binary-searches that segment's `.index` for the largest indexed offset ≤ 368,800, and (c) scans forward from that byte position, at most about 4 KB. A lookup is therefore two in-memory binary searches and one short sequential read, regardless of how large the partition is.

Records are written in **record batches**: a header (base offset, count, producer ID and sequence, CRC, compression codec, timestamps) followed by records whose offsets and timestamps are stored as small deltas. The broker validates the batch and assigns offsets but stores the producer's (possibly compressed) bytes as they arrived. Batches are the unit of compression, replication and transfer, which matters for performance (§31.13).

Figure 31.4 — A partition on disk: segments and sparse indexes.



Reading the diagram. Only the *active segment* receives appends. A *fetch* for offset 368,800 picks the segment by base offset, uses the sparse *index* to find a nearby byte position, then scans a few kilobytes. The *retention task* runs asynchronously and deletes whole closed segments. The storage boundary is the local file system and page cache. Failure points: a torn final batch after a crash is detected by CRC and truncated; indexes can be rebuilt from the log. Scaling: smaller segments allow finer deletion but more open files.

31.5.2 Choosing the number of partitions

Why does this exist?

Partitions are Kafka's unit of parallelism. A partition is written by one leader and, within a consumer group, read by at most one consumer (§31.7). So the partition count sets the ceiling on both write and read parallelism for a topic. Too few partitions and you cannot scale consumers; too many and you pay in open files, memory for batches, replication traffic, longer leader elections after a broker failure, and more work for the controller.

Mathematical foundations

FORMULA

Partition count from target throughput

$$P = \lceil h \cdot \max\left(\frac{T}{t_p}, \frac{T}{t_c}\right) \rceil$$

Symbol	Meaning	Units
P	Number of partitions for the topic	count
T	Target peak throughput of the topic	MB/s
t_p	Measured producer throughput into one partition	MB/s per partition
t_c	Measured throughput of one consumer instance on one partition, including its processing	MB/s per partition
h	Headroom factor for growth and uneven keys (≥ 1)	dimensionless

This rule of thumb comes from Kafka's committers [Rao 2015]: measure what one partition can do on the produce side and on the consume side, and provision for the slower of the two. The consumer side is usually the binding constraint because consumers do real work (database writes, enrichment calls), while a broker can append to one partition at tens of MB/s.

WORKED EXAMPLE

Worked example — Partitions for an order-events topic. Peak traffic is 50,000 events/s at 2 KB each, so $T = 100$ MB/s. A load test shows a producer can push $t_p = 20$ MB/s into one partition, and one consumer instance, which writes each event to a database, sustains $t_c = 4$ MB/s. You want headroom $h = 1.5$ for two years of growth.

1. Producer bound: $T/t_p = 100/20 = 5$ partitions.
2. Consumer bound: $T/t_c = 100/4 = 25$ partitions.
3. Take the maximum, 25, and apply headroom: $1.5 \times 25 = 37.5$, so $P = 38$.
4. Round to a number with many divisors so consumer counts divide evenly: 40 or 48. Choose 48 (divisible by 2, 3, 4, 6, 8, 12, 16, 24).

Interpretation. The consumer side needs five times as many partitions as the producer side. With 48 partitions you can run up to 48 consumer instances; a 49th would sit idle. **Engineering implication.** Set the partition count for keyed topics at creation with headroom, because increasing it later breaks key-to-partition mapping (§31.5.3). For unkeyed topics you can add partitions later with little harm.

WARNING / TRADE-OFF

Do not over-partition. Each partition replica is a set of open files, an entry in every broker's metadata image, a slot in leader elections and a separate producer batch. Thousands of partitions per broker are routine, but a topic with 10,000 partitions at 1 MB/s total produces tiny batches, poor compression and higher end-to-end latency, because each partition gets one record every few seconds [Rao 2015].

31.5.3 Keys, hashing and ordering

How it works internally

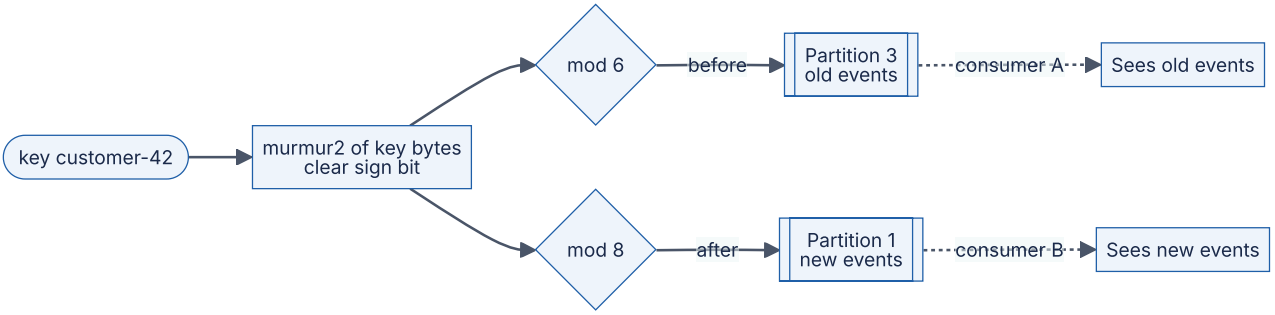
The producer chooses the partition for each record:

1. If the application names a partition explicitly, use it.
2. If the record has a **key**, the default partitioner computes `toPositive(murmur2(keyBytes)) % numPartitions`: a 32-bit MurmurHash2 of the serialized key bytes, with the sign bit cleared, modulo the partition count.
3. If the key is null, the **sticky partitioner** fills a batch for one partition and then switches to another, rather than spraying single records across all partitions (KIP-480, refined in KIP-794 to favor partitions on less loaded brokers). This produces fuller batches and lower latency than round-robin.

Because the same key always hashes to the same partition (while the count is fixed), all events for `customer-42` are totally ordered relative to each other. This is the basis of per-

entity ordering in Kafka.

Figure 31.5 — Key-to-partition hashing, and what changes when partitions are added.



Reading the diagram. The producer hashes the key `customer-42` and takes the result modulo the partition count. With 6 partitions the key maps to partition 3; after the topic grows to 8 partitions the same key maps to partition 1 (the code below computes both). Old events stay in partition 3 and new ones go to partition 1, read asynchronously by possibly different consumers, so a consumer may process a new “paid” event before an old “created” event still waiting in partition 3. The failure point is the moment of the partition change; the storage boundary is unaffected because Kafka never moves existing records between partitions.

Mathematical foundations

FORMULA

Fraction of keys that move when the partition count changes (hash mod P)

$$F_{\text{move}} = 1 - \frac{\min(P, P')}{\operatorname{lcm}(P, P')}$$

Symbol	Meaning	Units
F_{move}	Fraction of keys whose partition changes	fraction
P	Old partition count	count
P'	New partition count	count
lcm	Least common multiple	count

Why: a key stays put when $h \bmod P = h \bmod P' = a$. Then $h - a$ is divisible by both counts, so $h \equiv a \pmod{\operatorname{lcm}(P, P')}$ with $a < \min(P, P')$. Exactly $\min(P, P')$ of every $\operatorname{lcm}(P, P')$ hash values stay, assuming a uniform hash.

WORKED EXAMPLE

Worked example — Growing from 6 to 8, 12 and 13 partitions.

1. $6 \to 8$: $\text{lcm} = 24$, $\min = 6$, so $F = 1 - 6/24 = 0.75$. Three-quarters of keys move.
2. $6 \to 12$: $\text{lcm} = 12$, $F = 1 - 6/12 = 0.50$. Doubling is the friendliest change: each old partition splits into two.
3. $12 \to 13$: $\text{lcm} = 156$, $F = 1 - 12/156 = 0.923$. Adding “just one” partition moves 92% of keys.

Interpretation. Unlike consistent hashing (§14.8), `hash mod P` remaps most keys on almost any change. **Engineering implication.** For keyed topics, either over-provision at creation, or migrate by creating a new topic with the new count, dual-writing or replaying into it, and cutting consumers over once the old topic has drained.

Code example

The following runnable port of Kafka’s default partitioner is checked against test vectors from Kafka’s own unit tests and measures the moved fractions above on 100,000 keys.

```
        """Kafka's default key partitioner: murmur2(key) & 0x7fffffff, mod parti-
tions."""

        M32 = 0xFFFFFFFF


        def murmur2(data: bytes) -> int:
            """Port of org.apache.kafka.common.utils.Utils.murmur2; returns a signed
int32."""
            m, r = 0x5BD1E995, 24
            h = (0x9747B28C ^ len(data)) & M32
            n4 = len(data) // 4
            for i in range(n4):
                k = int.from_bytes(data[4 * i:4 * i + 4], "little")
                k = (k * m) & M32
                k ^= k >> r
                k = (k * m) & M32
                h = (h * m) & M32
                h ^= k
            tail = data[4 * n4:]
            if len(tail) == 3:
                h ^= tail[2] << 16
            if len(tail) >= 2:
                h ^= tail[1] << 8
            if len(tail) >= 1:
                h ^= tail[0]
                h = (h * m) & M32
            h ^= h >> 13
            h = (h * m) & M32
            h ^= h >> 15
            return h - (1 << 32) if h & 0x80000000 else h


        def partition_for(key: str, num_partitions: int) -> int:
            return (murmur2(key.encode()) & 0x7FFFFFFF) % num_partitions


        # Test vectors from Kafka's own unit tests.
        assert murmur2(b"21") == -973932308
        assert murmur2(b"foobar") == -790332482
        assert murmur2(b"a-little-bit-long-string") == -985981536
        assert murmur2(b"a-little-bit-longer-string") == -1486304829
        assert murmur2(b"lkjh234lh9fiuh90y23oiuhsafujhadof229phr9h19h89h8") == -58897971
        assert murmur2(b"abc") == 479470107

        keys = [f"customer-{i}" for i in range(100_000)]
        for old, new in [(6, 8), (6, 12), (12, 13)]:
            moved = sum(partition_for(k, old) != partition_for(k, new) for k in keys)
            print(f"{old:>2} -> {new:>2} partitions: {moved / len(keys):.1%} of keys
change partition")
        print("customer-42 ->", partition_for("customer-42", 6), "with 6,",
              partition_for("customer-42", 8), "with 8")
```

Output:

```
 6 ->  8 partitions: 75.1% of keys change partition
 6 -> 12 partitions: 49.9% of keys change partition
12 -> 13 partitions: 92.3% of keys change partition
customer-42 -> 3 with 6, 1 with 8
```

What to notice:

- Java's `int` arithmetic wraps at 32 bits; masking with `M32` after each multiplication reproduces it, and the asserts prove the port matches Kafka.
- Kafka clears the sign bit rather than taking `abs()`, because `abs(Integer.MIN_VALUE)` is still negative in Java.
- The measured fractions match the formula: 75%, 50% and 92%.
- Every client must use the same hash. Some non-Java clients (for example, librd-kafka-based ones) default to a different hash unless configured for murmur2.

Failure scenarios

What goes wrong	Why	Detection	Recovery	Prevention
Hot partition	One key (a celebrity, a big tenant) dominates	Per-partition bytes-in skew; one consumer lags	Split the key (salting, §14.10) if order allows	Choose keys with many distinct, balanced values
Out-of-order events after scaling	Partition count changed on a keyed topic	Consumers see "paid" before "created"	Reprocess from a new topic	Fix the count at creation; migrate via a new topic
Mixed-language producers disagree	Different default hash functions	Same key in two partitions	Re-key	Configure all clients for murmur2

31.6 Producers

31.6.1 Batching and compression

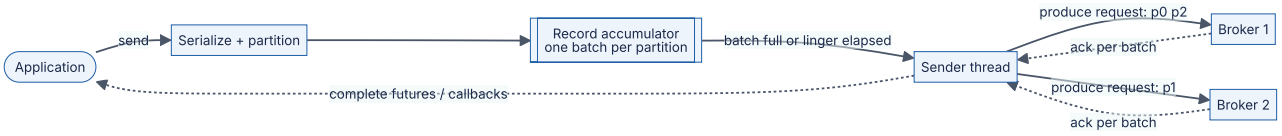
How it works internally

A producer is not a thin RPC client. Calling `send()` only serializes the record, picks a partition and appends it to an in-memory **record accumulator**; a background **sender thread** later ships whole batches.

1. `send()` serializes key and value, runs the partitioner (§31.5.3) and appends the record to the open batch for that partition. It returns a future immediately.
2. A batch becomes sendable when it reaches `batch.size` bytes (default 16,384) or when it has waited `linger.ms` (default 5 ms as of Kafka 4.0; 0 before 4.0, KIP-1030).
3. The sender groups ready batches by destination broker into one produce request per broker, so one network round trip carries batches for many partitions.

4. Each batch is compressed as a unit with `compression.type` (`none`, `gzip`, `snappy`, `lz4` or `zstd`). Compressing 16 records of similar JSON together gives far better ratios than compressing each alone.
5. If `buffer.memory` (default 32 MiB) fills because brokers are slow, `send()` blocks for up to `max.block.ms`: built-in backpressure (§22.10).

Figure 31.6 — Inside the producer: accumulator, sender and per-broker requests.



Reading the diagram. The application thread only serializes and appends to the *accumulator* (memory, not disk). The *sender thread* drains full or expired batches and sends one produce request per broker over TCP, each carrying batches for the partitions that broker leads. Acknowledgments return asynchronously and complete the futures. Failure points: a full accumulator blocks `send()`; a failed request is retried by the sender (§31.6.3). Scaling: larger batches and compression raise throughput per request at the cost of up to `linger.ms` of extra latency.

Mathematical foundations

FORMULA

Records per batch and time to fill a batch

$$n_{\text{batch}} = \min\left(\lambda_p \cdot \ell,\ \frac{B}{s}\right), \qquad t_{\text{fill}} = \min\left(\ell,\ \frac{B}{s \cdot \lambda_p}\right)$$

Symbol	Meaning	Units
n_{batch}	Records per batch sent	records
λ_p	Record rate into one partition from one producer	records/s
ℓ	`linger.ms`	s
B	`batch.size`	bytes
s	Average serialized record size	bytes
t_{fill}	Time a batch waits before it is sendable	s

WORKED EXAMPLE

Worked example — Is `linger.ms` or `batch.size` the limit? A producer writes 48,000 records/s of $s = 500$ B spread over 48 partitions, so $\lambda_p = 1,000$ records/s per partition. Defaults: $\ell = 5$ ms, $B = 16,384$ B.

1. Linger bound: $1,000 \times 0.005 = 5$ records per batch.
2. Size bound: $16,384/500 = 32$ records.
3. $n_{\text{batch}} = \min(5, 32) = 5$ records (2.5 KB); $t_{\text{fill}} = 5$ ms. Linger is the limit.
4. Batches per second: $48,000/5 = 9,600$ batches/s.
5. Raise ℓ to 20 ms: $n = \min(20, 32) = 20$ records; batches fall to 2,400/s, a 4× cut in per-batch overhead, at the cost of up to 15 ms more latency.

Interpretation. At moderate rates per partition, `linger.ms` decides batch size; `batch.size` matters only for hot partitions. **Engineering implication.** Fewer partitions per producer, or a larger linger, gives bigger batches, better compression and fewer requests. Latency-critical paths keep linger small and accept the overhead.

31.6.2 Acknowledgments: `acks=0`, `1` and `all`

The `acks` setting decides when the leader answers the producer:

Table 31.2 — Producer acknowledgment modes.

Setting	Leader replies when	Latency	Loss window
`acks=0`	Never; producer does not wait	Lowest	Any network or broker failure; no retries possible
`acks=1`	Leader has appended to its log (page cache)	Low	Leader dies before followers copy the record
`acks=all` (`-1`)	Every replica currently in the ISR has the record	Higher (one follower fetch round trip)	None for committed records while one ISR replica survives; needs `min.insync.replicas` (§31.9.3)

Since Kafka 3.0 the producer defaults are `acks=all` and `enable.idempotence=true` (KIP-679). Note that “has the record” means *in memory or the page cache* of each replica, not fsynced to disk; Kafka’s durability comes from copies on several machines, not from flushing one disk (§31.13).

31.6.3 Retries, idempotence and ordering

Why does this exist?

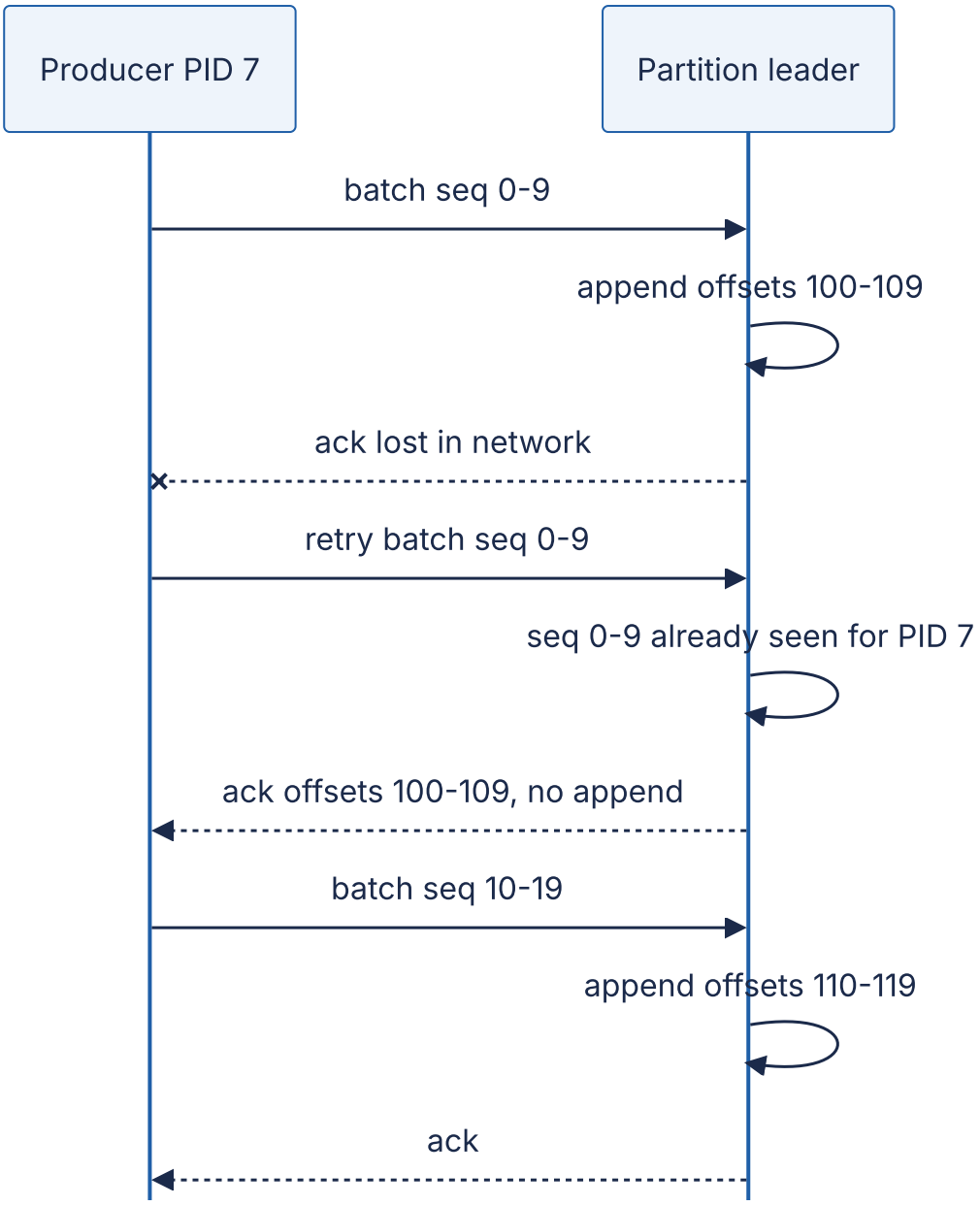
Networks lose acknowledgments. If the broker appended a batch but the reply was lost, the producer retries and the batch is written twice. Retries also threaten ordering: with several requests in flight, batch 1 can fail and be retried after batch 2 succeeds, reversing them.

How it works internally

The **idempotent producer** (KIP-98) applies the idempotency-key idea of §22.15 inside Kafka:

1. On start-up the producer obtains a **producer ID (PID)** and an **epoch** from a broker.
2. Every batch carries the PID, epoch and a **sequence number** per partition, starting at 0 and increasing by the record count.
3. The partition leader remembers, for each PID, the sequence numbers of the last five batches it appended. A batch whose sequence it has already seen is acknowledged again but **not** appended (duplicate). A batch with a gap (sequence too high) is rejected with `OutOfOrderSequenceException` so the producer can resend in order.
4. With idempotence on, up to `max.in.flight.requests.per.connection` = 5 requests may be outstanding and ordering is still preserved, because the leader accepts batches only in sequence order. Without idempotence, preserving order under retries requires `max.in.flight.requests.per.connection=1`, which caps throughput at one batch per round trip.
5. Retries continue until `delivery.timeout.ms` (default 120 s) expires.

Figure 31.7 — Idempotent producer: a lost acknowledgment does not create a duplicate.



Reading the diagram. The producer sends a batch tagged with PID 7 and sequences 0–9; the leader appends it, but the acknowledgment is lost. The retry carries the *same* PID and sequences, so the leader recognizes the duplicate and returns the original offsets without appending. The next batch continues at sequence 10. All calls are synchronous request/response over TCP. Limits: deduplication is per PID and per partition, and a PID is new each time the producer restarts unless it uses a `transactional.id` (§31.11), so idempotence protects against retries inside one producer session, not against an application that crashes and re-sends.

COMMON MISTAKE

"Idempotent producer means exactly-once." It removes duplicates caused by the producer's own retries to one partition during one session. It does not deduplicate an application that restarts and re-sends, does not make a consumer's processing idempotent, and does not make writes to several partitions atomic. Those need transactions (§31.11) and idempotent consumers (§32.10).

31.7 Consumers and consumer groups

31.7.1 The pull model and the poll loop

Kafka consumers **pull** ("partition 3 from offset 5,120"), so each goes at its own speed, batch jobs read in large chunks, and backpressure is automatic [Kreps et al. 2011]. On an idle topic a fetch waits at the broker until `fetch.min.bytes` (default 1 byte) arrive or `fetch.max.wait.ms` (default 500 ms) passes. The application loops: `poll()` returns up to `max.poll.records` (default 500), it processes them, and polls again [Shapira et al. 2021].

31.7.2 Consumer groups and partition assignment

DEFINITION

Definition — Consumer group. A set of consumer instances sharing a `group.id` that divide a topic's partitions among themselves so that **each partition is consumed by exactly one member** at a time. Different groups consume the same topic independently, each with its own offsets.

First-principles explanation

A consumer group is a team sharing a pile of logbooks: each logbook is given to one team member, so no entry is read twice within the team and entries in one logbook stay in order. A second team (another group) gets its own copies of the bookmarks and reads the same logbooks independently. Within a group Kafka behaves like a queue (work is divided); across groups it behaves like publish/subscribe (everyone gets everything), which is how one system provides both models of §30.4–§30.5.

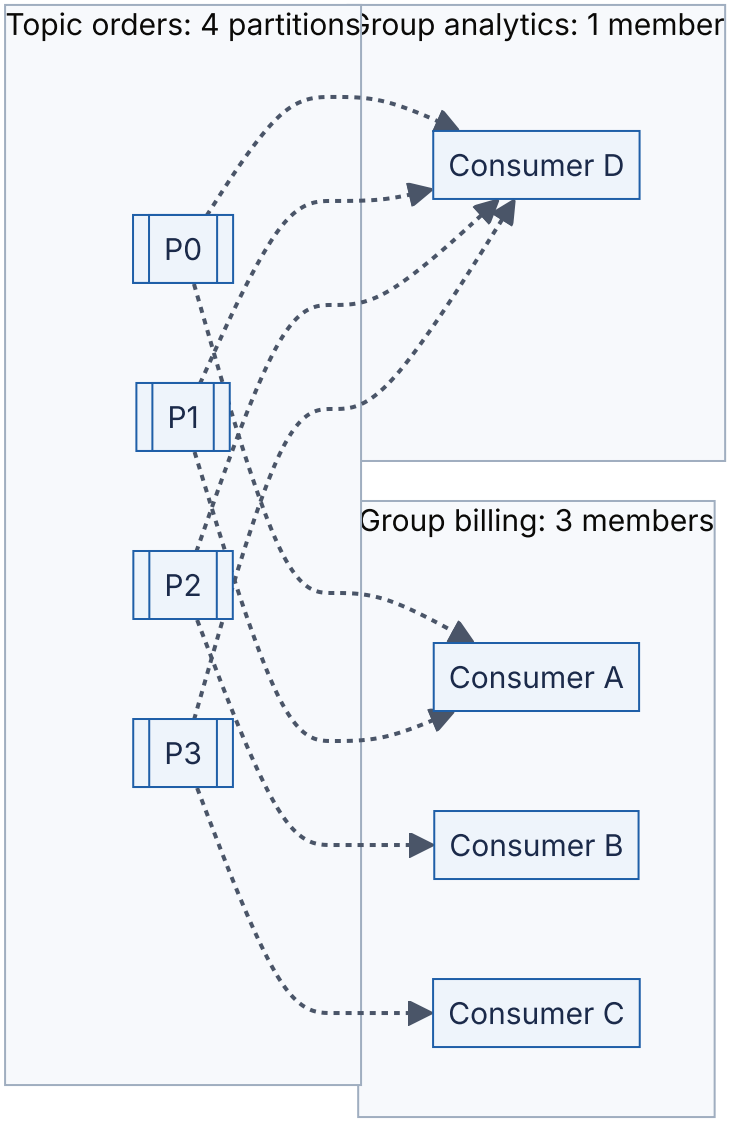
How it works internally

1. Each group has a **group coordinator**: the broker leading the partition of the internal `__consumer_offsets` topic that `hash(group.id)` maps to.
2. Members join the group and heartbeat to the coordinator. A member is considered dead if no heartbeat arrives within `session.timeout.ms` (default 45 s), or if it fails to

call `poll()` within `max.poll.interval.ms` (default 5 minutes), which catches a consumer stuck in processing.

3. When membership or the topic's partitions change, the group **rebalances**: partitions are reassigned by an **assignor** (range, round-robin, sticky or cooperative-sticky). §31.12 covers the protocols.
4. Each member fetches its assigned partitions from their leaders (or, with rack-aware **follower fetching**, KIP-392, from a replica in its own availability zone to avoid cross-zone transfer charges).

Figure 31.8 — Two consumer groups reading one four-partition topic.



Reading the diagram. The *billing* group's three members share four partitions: A owns two, B and C one each. The *analytics* group's single member reads all four independently, with its own offsets. Arrows are asynchronous pull fetches. Failure point: if B crashes, P2 is unconsumed by billing until a rebalance gives it to A or C. Scaling: billing can use at most four members.

31.7.3 Parallelism ceiling and consumer lag

The **parallelism ceiling** of a group is the partition count: with P partitions, at most P members do work. If processing is the bottleneck and members are already at P, the options are more partitions (with the key-remapping cost of §31.5.3), faster processing per record, or concurrency *inside* a member (for example a thread pool keyed by record key to keep per-key order).

DEFINITION

Definition — Consumer lag. For one partition, lag = **log end offset** (or high watermark) minus the group's **committed offset**: the number of records written but not yet processed by the group. Group lag is the sum over partitions.

FORMULA

Lag drain time

$$t_{\text{drain}} = \frac{L}{\mu - \lambda} \quad (\mu > \lambda)$$

Symbol	Meaning	Units
t_{drain}	Time for lag to return to about zero	s
L	Current lag	records
μ	Aggregate consumption rate of the group	records/s
λ	Current production rate	records/s

WORKED EXAMPLE

Worked example — Recovering from a 20-minute consumer outage. Producers write $\lambda = 40,000$ records/s. The consumer group was down for 20 minutes, so $L = 40,000 \times 1,200 = 48,000,000$ records. Twelve members each process 5,000 records/s, so $\mu = 60,000$ records/s.

1. Net drain rate: $\mu - \lambda = 20,000$ records/s.
2. $t_{\text{drain}} = 48,000,000/20,000 = 2,400$ s $= 40$ minutes.
3. If the topic has 24 partitions, scaling to 24 members gives $\mu = 120,000$: $48 \times 10^6/80,000 = 600$ s $= 10$ minutes.
4. With 12 partitions, extra members are idle and recovery stays at 40 minutes.

Interpretation. A 20-minute outage costs 40 minutes of staleness after recovery unless spare capacity exists, and spare consumers help only up to the partition count. **Engineering implication.** Provision consumers for roughly 1.5–2× peak and partitions for the scaled-up count; alert on lag *in seconds* (lag ÷ production rate), which is what users feel, rather than in records. Also check retention: if lag ever exceeds retention, records are deleted before they are read.

31.7.4 Share groups (queues for Kafka)

Consumer groups tie parallelism to partitions and order per partition. Some workloads, such as a pool of workers processing independent jobs, want queue semantics instead: many workers on one partition, per-record acknowledgment and redelivery. **KIP-932** adds **share groups**, in which records of a partition are handed to several consumers with per-record acquisition locks and delivery counts. Share groups arrived as early access in Kafka 4.0 and were still maturing through the 4.x line as of 2026; check your version's documentation before relying on them.

31.8 Offsets

31.8.1 Where offsets live

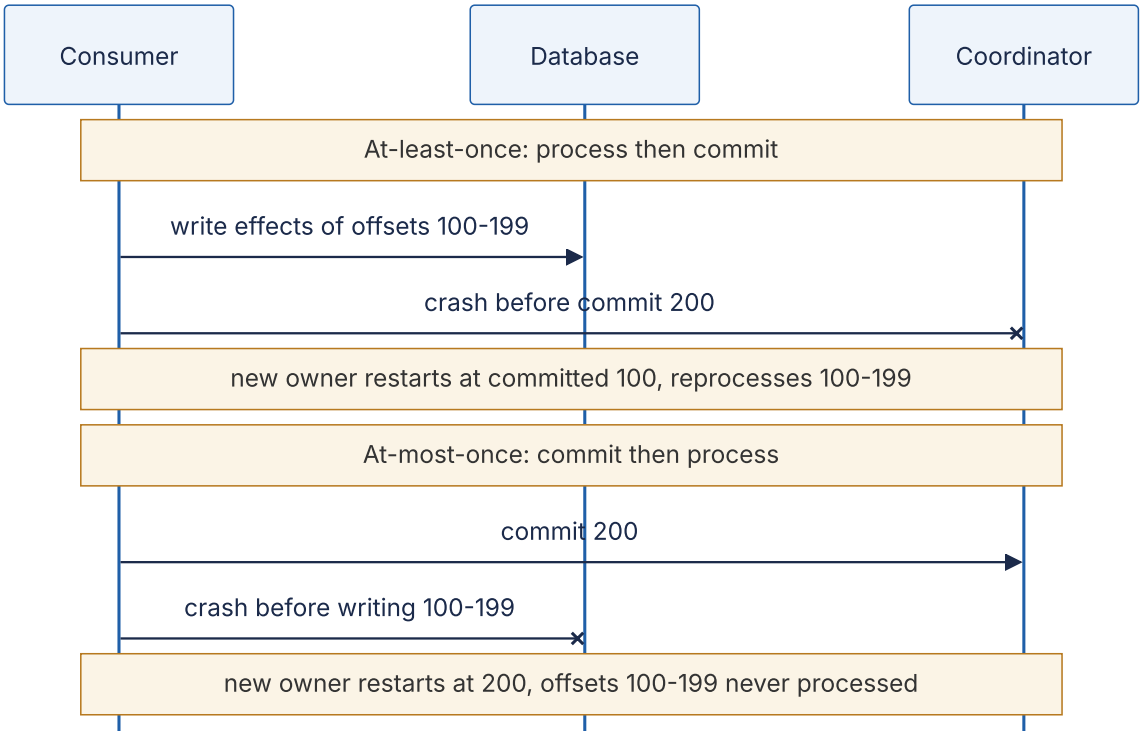
A consumer group's progress is a map from partition to **committed offset**, by convention the offset of the *next* record to read. Since Kafka 0.9 these commits are records in the internal topic `__consumer_offsets` (50 partitions by default, replicated, **compacted** so only the latest commit per group and partition survives, §31.10). Storing offsets in a Kafka log makes commits a cheap append and lets the coordinator rebuild its cache by reading the partition after failover. When a group has no committed offset (or the committed offset has been deleted by retention), `auto.offset.reset` decides where to start: `latest` (the default), `earliest` or `none` (fail).

31.8.2 Commit strategies and delivery semantics

The moment of the commit relative to processing decides the delivery guarantee (§30.8):

- **Commit after processing → at-least-once.** If the consumer crashes after processing but before committing, the next owner of the partition reprocesses those records. Duplicates are possible; loss is not.
- **Commit before processing → at-most-once.** If the consumer crashes after committing but before processing, those records are skipped. Loss is possible; duplicates are not.

Figure 31.9 — Commit timing decides duplicates or loss.



Reading the diagram. Upper half: effects reach the *database* first, the crash prevents the commit, and the new owner reprocesses 100–199, duplicating effects unless writes are idempotent. Lower half: the commit reaches the *coordinator* first, the crash prevents processing, and the new owner starts at 200, silently losing 100–199. Both calls are synchronous; the failure point is the gap between two writes to different storage systems, which cannot be atomic without transactions or an idempotent sink.

Table 31.3 — Offset commit strategies.

Strategy	How	Guarantee	Cost	Use when
Auto commit	Default; commits polled position every 5 s inside `poll()`	At-least-once if processing finishes before the next `poll()`; at-most-once if handed off asynchronously	Up to 5 s reprocessed after a crash	Simple, idempotent processing
Synchronous commit	`commitSync()` after processing	At-least-once; blocks until acknowledged	One round trip per commit; lower throughput	Correctness first, moderate rate
Asynchronous commit	`commitAsync()`	At-least-once; failures not retried (could overwrite a newer offset)	Non-blocking	High throughput; `commitSync()` on shutdown and revocation
Offsets stored with results	Write offset in the same database transaction as the output; seek on start-up	Effectively-once for that sink	Requires transactional sink	Database sinks needing no duplicates
Kafka transactions	`sendOffsetsToTransaction()` (§31.11)	Exactly-once for Kafka-to-Kafka	Transaction overhead	Read-process-write pipelines

RECOMMENDATION

Default to at-least-once with idempotent processing. Disable auto commit for anything non-trivial, commit after processing (synchronously every few hundred records, or asynchronously with a final synchronous commit), and make the processing idempotent with an idempotency key or upsert (§22.15, §32.10). At-most-once is acceptable only for data where loss is cheaper than duplicates, such as some metrics.

31.9 Replication, ISR and leader election

31.9.1 Leaders, followers and the in-sync replica set

DEFINITION

Definition — ISR, LEO and high watermark. Each partition has a **replication factor** (RF) of replicas on different brokers; one is the **leader**, which handles all writes, and the others are **followers**, which fetch from the leader. The **in-sync replica set (ISR)** is the leader plus the followers that have fully caught up within `replica.lag.time.max.ms` (default 30 s). Each replica's **log end offset (LEO)** is the offset its next appended record will get. The **high watermark (HW)** is the offset below which every ISR member has the record; records below the HW are **committed** and are the only ones consumers can read.

How it works internally

1. The leader appends a producer batch, advancing its LEO.
2. Each follower sends fetch requests (the same API consumers use) with its own LEO. The leader returns new records and learns from the fetch offset how far each follower has got.
3. The leader advances the HW to the minimum LEO across the ISR, and tells followers the new HW in the next fetch response; followers' HWs therefore lag the leader's by one fetch round.
4. An `acks=all` produce request waits in a holding area ("purgatory") until the HW passes its last offset, then the leader acknowledges.
5. A follower that has not caught up for `replica.lag.time.max.ms` is removed from the ISR (ISR **shrink**); it rejoins (ISR **expand**) when it catches up again. The leader records ISR changes through the controller.

FORMULA

High watermark

$$\mathrm{HW} = \min_{r \in \mathrm{ISR}} \mathrm{LEO}_r$$

Symbol	Meaning	Units
HW	High watermark: first offset not yet committed	offset
LEO_r	Log end offset of replica r	offset
ISR	Current in-sync replica set, including the leader	set of replicas

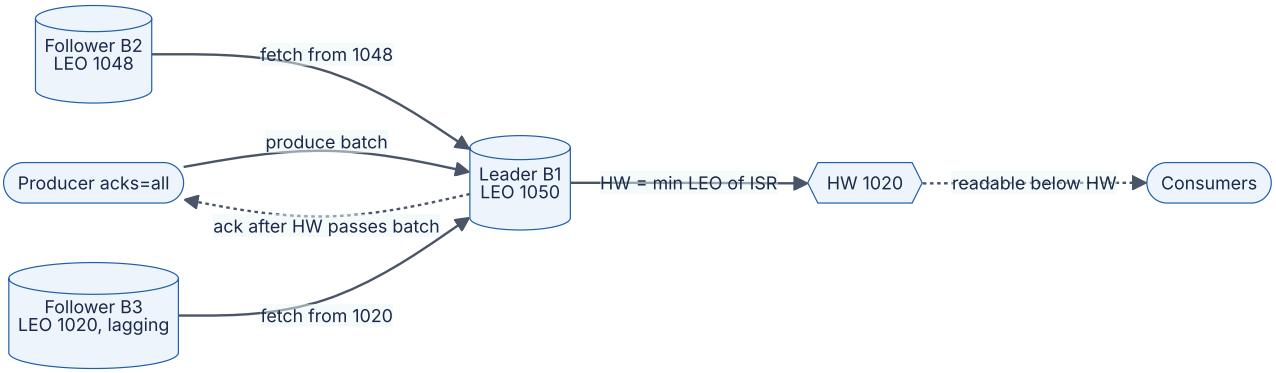
WORKED EXAMPLE

Worked example — What a consumer can see. RF = 3. The leader's LEO is 1,050, follower F1's is 1,048, follower F2's is 1,020, and all three are in the ISR.

1. HW = min(1,050, 1,048, 1,020) = 1,020. Consumers can read offsets up to 1,019.
2. An `acks=all` producer whose batch ended at offset 1,049 is still waiting.
3. F2 has been behind for 35 s, exceeding the 30 s limit, so the leader shrinks the ISR to {leader, F1}. HW jumps to min(1,050, 1,048) = 1,048, and requests up to 1,047 are acknowledged.

Interpretation. A slow follower holds back both visibility and `acks=all` latency until it is dropped; dropping it trades redundancy for progress. **Engineering implication.** Alert on ISR shrink rate and under-replicated partitions (§31.15), because a partition running with ISR = {leader} has no redundancy even though writes still succeed (unless `min.insync.replicas` stops them).

Figure 31.10 — Replica state on one partition with RF = 3.



Reading the diagram. The *producer* writes to the *leader* on broker B1. *Followers* pull with fetches carrying their LEOs, so replication is asynchronous to the write, but the `acks=all` acknowledgment waits until the *HW* passes the batch. *Consumers* read only below the HW, never seeing records that could vanish in a failover. Each replica is a separate broker disk. Failure points: lagging B3 holds back the HW until removed from the ISR; leader failure triggers election (§31.9.3). Scaling: replication traffic is (RF − 1) times produce traffic.

31.9.2 Kafka's ISR versus majority quorums

Table 31.4 — ISR replication versus a Raft-style majority quorum.

Aspect	Kafka ISR (data partitions)	Majority quorum (Raft, KRaft metadata)
Commit rule	All current ISR members have the record	A majority has the record
Replicas to tolerate f failures without losing committed data	$f + 1$	$2f + 1$
Effect of one slow replica	Holds commits until dropped from ISR (up to 30 s)	None; the majority proceeds
Who decides membership	External controller (itself a Raft quorum)	The protocol itself

The ISR design needs fewer copies for the same data safety, because it waits for every in-sync replica, and it relies on the controller to make ISR changes and leader choices consistent [Wang et al. 2015]. The price is sensitivity to slow followers, bounded by the lag timeout.

31.9.3 Leader election, `min.insync.replicas` and unclean election

When a leader's broker is fenced, the controller picks a new leader **from the ISR**, which by definition holds every committed record, increments the partition's **leader epoch**, and records the change in the metadata log. Brokers and clients learn the new leader from metadata. Two settings decide the durability/availability trade-off:

- **`min.insync.replicas`** (topic or broker setting, default 1): an `acks=all` write is rejected with `NOT_ENOUGH_REPLICAS` if the ISR is smaller than this. With RF = 3 and `min.insync.replicas=2`, every acknowledged record is on at least two brokers.
- **`unclean.leader.election.enable`** (default false): if no ISR member is alive, an out-of-sync replica may become leader. The partition comes back online, but every record it lacks, including acknowledged ones, is lost and later truncated from returning replicas.

Mathematical foundations

FORMULA

Write unavailability of one partition (independent broker failures)

$$U_w = \sum_{k=R-m+1}^{R} \binom{R}{k} q^k (1-q)^{R-k}$$

Symbol	Meaning	Units
U_w	Probability that fewer than m replicas are up, so `acks=all` writes are refused	probability
R	Replication factor	count
m	`min.insync.replicas`	count
q	Probability that a given broker is unavailable at a random moment	probability
k	Number of unavailable replicas	count

The formula optimistically treats “up” as “in sync”. Committed data survives as long as at least one ISR replica survives, so data loss on clean election requires all R replicas of the ISR to fail.

WORKED EXAMPLE

Worked example — RF = 3 with `min.insync.replicas` of 1, 2 and 3. Assume each broker is unavailable with $q = 0.01$ (about 7 hours a month, including rolling restarts).

1. $m = 2$: writes stop when 2 or 3 replicas are down. $U_w = 3q^2(1-q) + q^3 = 3(0.0001)(0.99) + 0.000001 = 0.000298$, about 0.03% of the time.
2. $m = 3$: writes stop when any replica is down. $U_w = 1 - 0.99^3 = 0.0297$, about 3%, which means every rolling restart blocks writes.
3. $m = 1$: writes stop only when all are down, $q^3 = 10^{-6}$, but an acknowledged write may exist on only one broker; if that broker's disk dies before followers catch up, the record is gone.
4. With $m = 2$ the cluster tolerates one broker failure with no write impact and no committed-data loss; with two failures writes stop but committed data is intact on the survivor.

Interpretation. RF = 3, `min.insync.replicas=2`, `acks=all` is the standard durability configuration because it survives a single failure (including planned restarts) with neither loss nor downtime. **Engineering implication.** RF = 3 with $m = 3$ is a common mistake; it converts every restart into an outage. Spread replicas across racks or availability zones with `broker.rack` so one zone failure removes only one replica.

Table 31.5 — Durability configurations (RF = 3).

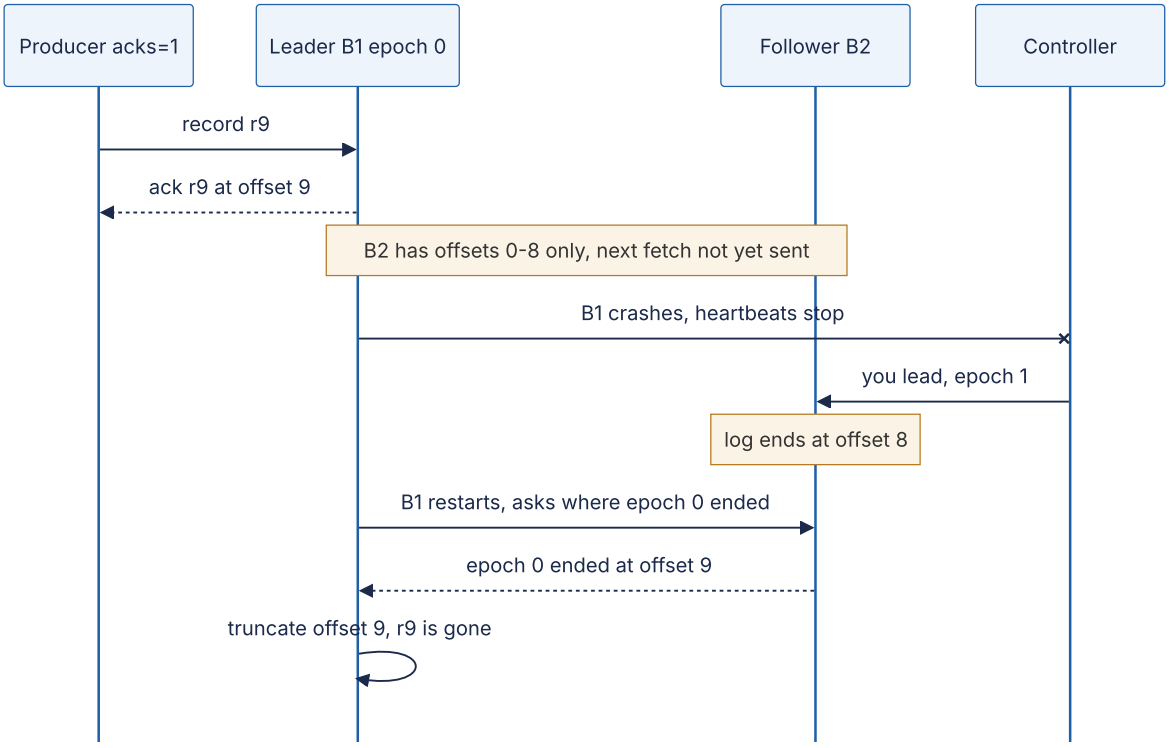
`acks`	`min.insync.replicas`	Unclean election	Acknowledged data lost when	Writes stop when
1	any	false	Leader dies before followers fetch	Leader unavailable (until re-election)
all	1	false	ISR shrank to leader alone, then leader's disk lost	All replicas down
all	2	false	Two brokers holding the record both lost	Two replicas down
all	2	true	Out-of-sync replica elected after ISR loss	Rarely; availability favored

31.9.4 Leader epochs and log truncation

After a leader change, a follower may hold records the new leader never received; those records were never committed and must be removed. Early Kafka truncated each follower to its own high watermark, but because followers learn the HW one fetch late, two failures in quick succession could truncate committed records or leave replicas with divergent logs [KIP-101]. Since Kafka 0.11 each record batch carries its **leader epoch**, every

replica records the start offset of each epoch, and a follower asks the new leader "where did epoch *e* end?" (`OffsetsForLeaderEpoch`). It truncates exactly to that point and then fetches. Leader epochs are fencing tokens (§26.13): a deposed leader with an old epoch cannot get its stale records accepted, and clients include the epoch in fetch requests to detect stale leaders.

Figure 31.11 — Leader failure with `acks=1`: an acknowledged record disappears.



Reading the diagram. With `acks=1` the *leader* acknowledges r9 as soon as it is in its own log. Before the *follower* fetches it, B1 crashes; the *controller* detects missing heartbeats and makes B2 leader in epoch 1. When B1 returns, the leader-epoch exchange tells it that epoch 0 ended at offset 9 on the new leader, so it truncates its own r9. The producer was told r9 succeeded, yet no replica has it. With `acks=all`, the leader would not have acknowledged r9 until B2 had it, so the producer would have retried instead.

31.9.5 A runnable replication simulation

The program below models one partition with RF = 3. One record is produced per tick; followers fetch every few ticks; the leader shrinks the ISR when a follower lags too long, computes the HW, and fails at the end. It reports acknowledged records that the new leader does not have.

```
        """Toy Kafka partition: leader, followers, ISR, high watermark, leader epochs.

        Compares acks=1 and acks=all under leader failure. One record is produced per
        tick; followers copy from the leader every `fetch_every` ticks.
        """


        class NotEnoughReplicas(Exception):
            pass


        class Replica:
            def __init__(self, rid, fetch_every):
                self.rid, self.fetch_every = rid, fetch_every   # fetch_every=0 means
down
                self.log = []               # list of (leader_epoch, value); index =
offset
                self.caught_up_at = 0       # last tick this follower matched the leader

            @property
            def leo(self):                  # log end offset: next offset to be written
                return len(self.log)


        class Partition:
            def __init__(self, fetch_every=(1, 3, 3), min_insync=1, lag_max=4,
unclean=False):
                self.r = {i: Replica(i, f) for i, f in enumerate(fetch_every)}
                self.leader, self.epoch, self.tick = 0, 0, 0
                self.isr = set(self.r)
                self.hw = 0                 # offsets below hw are committed and read-
able
                self.min_insync, self.lag_max, self.unclean = min_insync, lag_max, un-
clean

            def produce(self, value, acks):
                if acks == "all" and len(self.isr) < self.min_insync:
                    raise NotEnoughReplicas(f"ISR={sorted(self.isr)}")
                self.r[self.leader].log.append((self.epoch, value))
                return self.r[self.leader].leo - 1

            def step(self):
                """Advance one tick: followers fetch, ISR shrinks or grows, HW moves."""
                self.tick += 1
                lead = self.r[self.leader]
                for f in self.r.values():
                    if f.rid == self.leader or f.fetch_every == 0:
                        if f.fetch_every == 0:
                            self.isr.discard(f.rid)          # a dead broker leaves the
ISR
                        continue
                    if self.tick % f.fetch_every == 0:
                        f.log = f.log + lead.log[f.leo:]     # fetch everything new
                    if f.leo == lead.leo:
                        f.caught_up_at = self.tick
                        self.isr.add(f.rid)
```

```
                elif self.tick - f.caught_up_at > self.lag_max:
                    self.isr.discard(f.rid)              # like
replica.lag.time.max.ms
            self.hw = min(self.r[i].leo for i in self.isr)

        def is_acked(self, offset, acks):
            if acks == "1":
                return True                              # leader appended it: ack
sent
            # every ISR member has it, and the ISR is still large enough
            return offset < self.hw and len(self.isr) >= self.min_insync

        def fail_leader(self):
            old = self.leader
            self.r[old].fetch_every = 0
            self.isr.discard(old)
            live = [i for i in self.r if self.r[i].fetch_every > 0]
            candidates = [i for i in self.isr if i in live]
            if not candidates:
                if not self.unclean:
                    raise RuntimeError("no live ISR member: partition OFFLINE")
                candidates = live                        # unclean leader election
            self.leader = max(candidates, key=lambda i: self.r[i].leo)
            self.epoch += 1
            for i in live:                               # followers truncate to the
new
                self.r[i].log = list(self.r[self.leader].log)   # leader's log
            self.isr = set(candidates)
            return old


    def run(title, acks, n=10, **cfg):
        print(title)
        p = Partition(**cfg)
        acked, pending = [], []
        for t in range(n):
            try:
                pending.append((p.produce(f"r{t}", acks), f"r{t}"))
            except NotEnoughReplicas as e:
                print(f"  t={t}: write r{t} REJECTED, {e}")
            p.step()
            acked += [v for off, v in pending if p.is_acked(off, acks)]
            pending = [(off, v) for off, v in pending if not p.is_acked(off, acks)]
        print(f"  before crash: leader LEO={p.r[p.leader].leo} HW={p.hw} ISR=
{sorted(p.isr)}")
        try:
            old = p.fail_leader()
        except RuntimeError as e:
            print(f"  acked={len(acked)} unacked(retry)={len(pending)}; leader
crashed: {e}")
            return
        kept = {v for _, v in p.r[p.leader].log}
        lost = [v for v in acked if v not in kept]
        print(f"  leader {old} crashed -> new leader {p.leader}, epoch {p.epoch}, "
              f"log has {len(kept)} records")
        print(f"  acked={len(acked)} unacked(retry)={len(pending)} "
              f"ACKED BUT LOST={len(lost)} {lost}")
```

```
        if __name__ == "__main__":
            run("A: acks=1", "1", fetch_every=(1, 3, 3))
            run("B: acks=all, min.insync.replicas=2", "all", fetch_every=(1, 3, 3),
min_insync=2)
            run("C: acks=all, min.insync=1, followers stalled, unclean election on",
"all",
                fetch_every=(1, 50, 0), min_insync=1, unclean=True)
            run("D: same stall, min.insync.replicas=2", "all",
                fetch_every=(1, 50, 0), min_insync=2)
```

Output:

```
A: acks=1
  before crash: leader LEO=10 HW=9 ISR=[0, 1, 2]
  leader 0 crashed -> new leader 1, epoch 1, log has 9 records
  acked=10 unacked(retry)=0 ACKED BUT LOST=1 ['r9']
B: acks=all, min.insync.replicas=2
  before crash: leader LEO=10 HW=9 ISR=[0, 1, 2]
  leader 0 crashed -> new leader 1, epoch 1, log has 9 records
  acked=9 unacked(retry)=1 ACKED BUT LOST=0 []
C: acks=all, min.insync=1, followers stalled, unclean election on
  before crash: leader LEO=10 HW=10 ISR=[0]
  leader 0 crashed -> new leader 1, epoch 1, log has 0 records
  acked=10 unacked(retry)=0 ACKED BUT LOST=10 ['r0', 'r1', 'r2', 'r3', 'r4', 'r5', 'r6',
'r7', 'r8', 'r9']
D: same stall, min.insync.replicas=2
  t=5: write r5 REJECTED, ISR=[0]
  t=6: write r6 REJECTED, ISR=[0]
  t=7: write r7 REJECTED, ISR=[0]
  t=8: write r8 REJECTED, ISR=[0]
  t=9: write r9 REJECTED, ISR=[0]
  before crash: leader LEO=5 HW=5 ISR=[0]
  acked=0 unacked(retry)=5; leader crashed: no live ISR member: partition OFFLINE
```

What to notice:

- **A versus B** share replication timing. With `acks=1`, r9 was acknowledged from the leader's log alone and vanished in the failover (Figure 31.11). With `acks=all`, r9 was never acknowledged, so the producer retries it against the new leader; nothing acknowledged is lost.
- **C** shows `acks=all` alone is not enough: the ISR shrank to the leader, so "all ISR members" meant one broker, and unclean election then promoted an empty replica, losing ten acknowledged records.
- **D** adds `min.insync.replicas=2`: writes are refused once the ISR shrinks, and the partition goes offline rather than losing data. Availability is traded for durability.
- The model omits HW propagation delay, retries and timeouts, but its safety argument is the one Kafka makes.

31.10 Retention and log compaction

31.10.1 Time and size retention

With `cleanup.policy=delete` (the default), a partition keeps data for `retention.ms` (default 7 days) and, optionally, up to `retention.bytes` per partition (default unlimited). A background task deletes **whole closed segments** whose newest record is older than the limit, or the oldest segments when the partition exceeds the size limit. Because the active segment is never deleted, a low-traffic partition can hold data longer than `retention.ms` until its segment rolls. Retention is independent of consumption: Kafka deletes unread data on schedule and keeps read data until then.

31.10.2 Log compaction

> DEFINITION
>
> **Definition — Log compaction.** A retention policy (`cleanup.policy=compact`) that keeps **at least the latest record for each key** and discards older records with the same key, so the partition converges to a snapshot of the latest value per key while preserving the offsets and order of the records that remain. A record with a null value, a **tombstone**, marks a key as deleted and is itself removed after `delete.retention.ms` (default 24 hours).

First-principles explanation

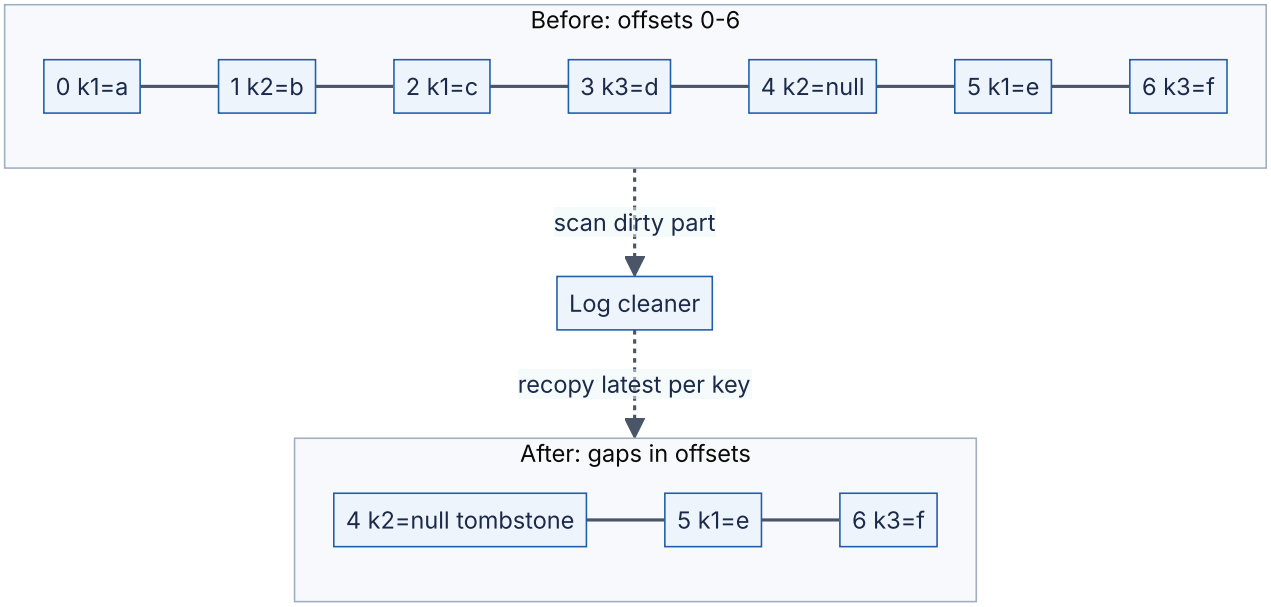
A phone book printed yearly does not list every address a person has ever had, only the current one; but if you photocopy it each day you can still rebuild today's town. A compacted topic is that phone book: it is the **changelog** of a table, and replaying it from offset 0 rebuilds the table's latest state, however long the topic has existed. This is the same idea as log-structured merge compaction (§13.9), applied to a replicated, readable log.

How it works internally

1. The log has a **clean** part (already compacted) and a **dirty** part (written since the last pass). The cleaner runs on a partition when the dirty fraction exceeds `min.cleanable.dirty.ratio` (default 0.5).
2. A cleaner thread scans the dirty part and builds an in-memory map from key to its latest offset.
3. It recopies the clean and dirty segments, keeping a record only if its offset is the latest for its key, and swaps in the new segments. Offsets are not renumbered, so the compacted log has gaps.
4. The active segment is never compacted, so recent history is always complete.

5. Tombstones survive long enough (`delete.retention.ms`) for consumers that are replaying to see the deletion, then disappear.

Figure 31.12 — Compaction keeps the latest value per key.



Reading the diagram. Before compaction, keys k1, k2 and k3 have several versions. The *log cleaner* runs asynchronously and keeps only the latest record per key: k1 = e at offset 5, k3 = f at offset 6, and the tombstone for k2 at offset 4, which will itself be removed after `delete.retention.ms` . Offsets 0–3 are gone but the survivors keep their original offsets and order. Failure points: a consumer that replays slower than `delete.retention.ms` can miss a tombstone and keep a deleted key; compaction needs disk headroom for recopying. Scaling: cleaner threads and their memory bound how fast large compacted topics converge.

Table 31.6 — Delete versus compact retention.

	`delete`	`compact`	`compact,delete`
Keeps	Everything for a time or size window	Latest record per key, forever	Latest per key, within a time window
Typical use	Event streams, logs, clickstream	Changelogs, `__consumer_offsets` , Kafka Streams state, CDC snapshots	Latest-state caches that may expire
Rebuild state by replay	Only within the window	Always	Within the window

INDUSTRY EXAMPLE

The New York Times publishing pipeline. The Times described storing every published asset since 1851 in a single-partition, log-compacted Kafka topic used as the source of truth; services such as search and personalization build their own views by replaying it from the beginning, and new derived systems are created by replay rather than by bulk export. The single partition preserves a total order of edits across assets. *Evidence: Primary* — [Svingen 2017]

COMMON MISTAKE

"Compaction makes Kafka a database." A compacted topic has no secondary indexes, no point reads by key (only sequential scans), and keeps several versions until the cleaner runs. It is a durable changelog from which a database, cache or Kafka Streams state store is built.

31.11 Delivery semantics and transactions

31.11.1 From at-least-once to exactly-once

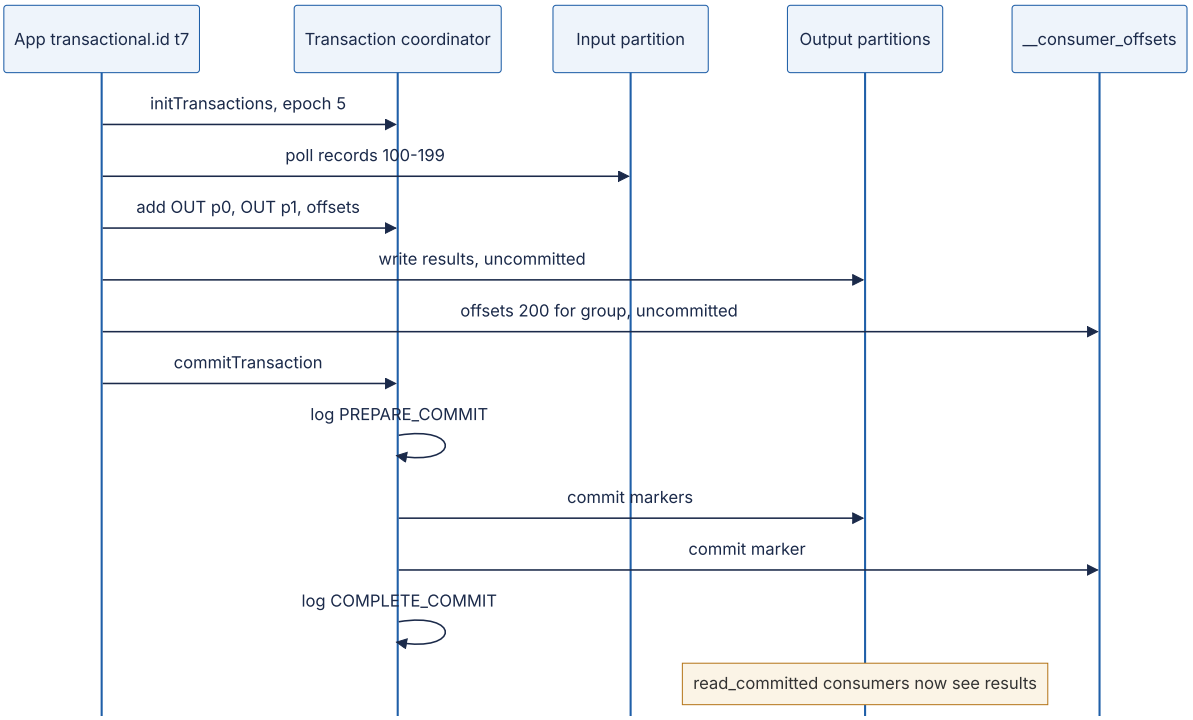
Kafka offers all three guarantees of §30.8: at-most-once (commit before processing, or `acks=0`), at-least-once (the default: retries plus commit after processing), and **exactly-once semantics (EOS)**, which Kafka defines narrowly: in a **read-process-write** pipeline from Kafka topics to Kafka topics, each input record's effects (output records *and* the input offset commit) appear exactly once, even across producer retries and application crashes [KIP-98]. Exactly-once delivery over a network is impossible in general; what Kafka provides is effectively-once *processing* by making the output and the offset commit atomic.

31.11.2 How transactions work

1. The application sets a stable **`transactional.id`** (for example, `billing-processor-7`). On `initTransactions()`, the **transaction coordinator** (the broker leading the partition of `__transaction_state` chosen by hashing the id) returns the producer ID and bumps its **epoch**. Any older instance with the same id now has a stale epoch and is **fenced**: its writes and commits are rejected. This is how a zombie, a paused instance whose replacement has already started, is stopped.
2. `beginTransaction()` starts a transaction locally. Before the first write to each partition, the producer registers that partition with the coordinator.
3. The producer writes output records normally (idempotently). They are in the logs but marked as belonging to an open transaction.

4. `sendOffsetsToTransaction()` adds the consumed input offsets to the same transaction, as writes to `__consumer_offsets`.
5. `commitTransaction()` runs two-phase commit (§28.4.8): the coordinator writes `PREPARE_COMMIT` to its log (the commit point), writes a **commit marker** (a control record) into every involved partition, then writes `COMPLETE_COMMIT`. Aborts write abort markers instead.
6. Consumers with `isolation.level=read_committed` read only up to the **last stable offset (LSO)**, the first offset of any still-open transaction, and skip records of aborted transactions using the `.txnindex`. The default, `read_uncommitted`, returns everything.

Figure 31.13 — A read-process-write transaction.



Reading the diagram. Outputs and input offsets are written inside one transaction with synchronous calls; the coordinator's replicated `__transaction_state` log records the decision. Once `PREPARE_COMMIT` is durable the transaction commits even if the coordinator crashes, because its successor replays the log and finishes the markers. If the application crashes first, the transaction is aborted (on timeout, `transaction.timeout.ms`, default 60 s, or when the restarted instance calls `initTransactions()`); `read_committed` consumers never see the partial output, and the restart resumes from the last committed offsets.

31.11.3 Limits of exactly-once

> WARNING / TRADE-OFF
>
> **Exactly-once stops at Kafka's boundary.** Transactions cover records written to Kafka and offsets committed to Kafka. They do not cover a database write, an email, a payment API call or any other side effect inside the processing step; if the application crashes after calling the payment API but before committing, the retry calls it again. For external sinks you need idempotent writes, offsets stored atomically with the output, or a transactional outbox and CDC; §32.9 examines these limits and §32.10 the idempotent-consumer pattern. Transactions also add latency (a commit every interval) and `read_committed` consumers wait for the slowest open transaction, which a hung producer can hold for up to `transaction.timeout.ms`.

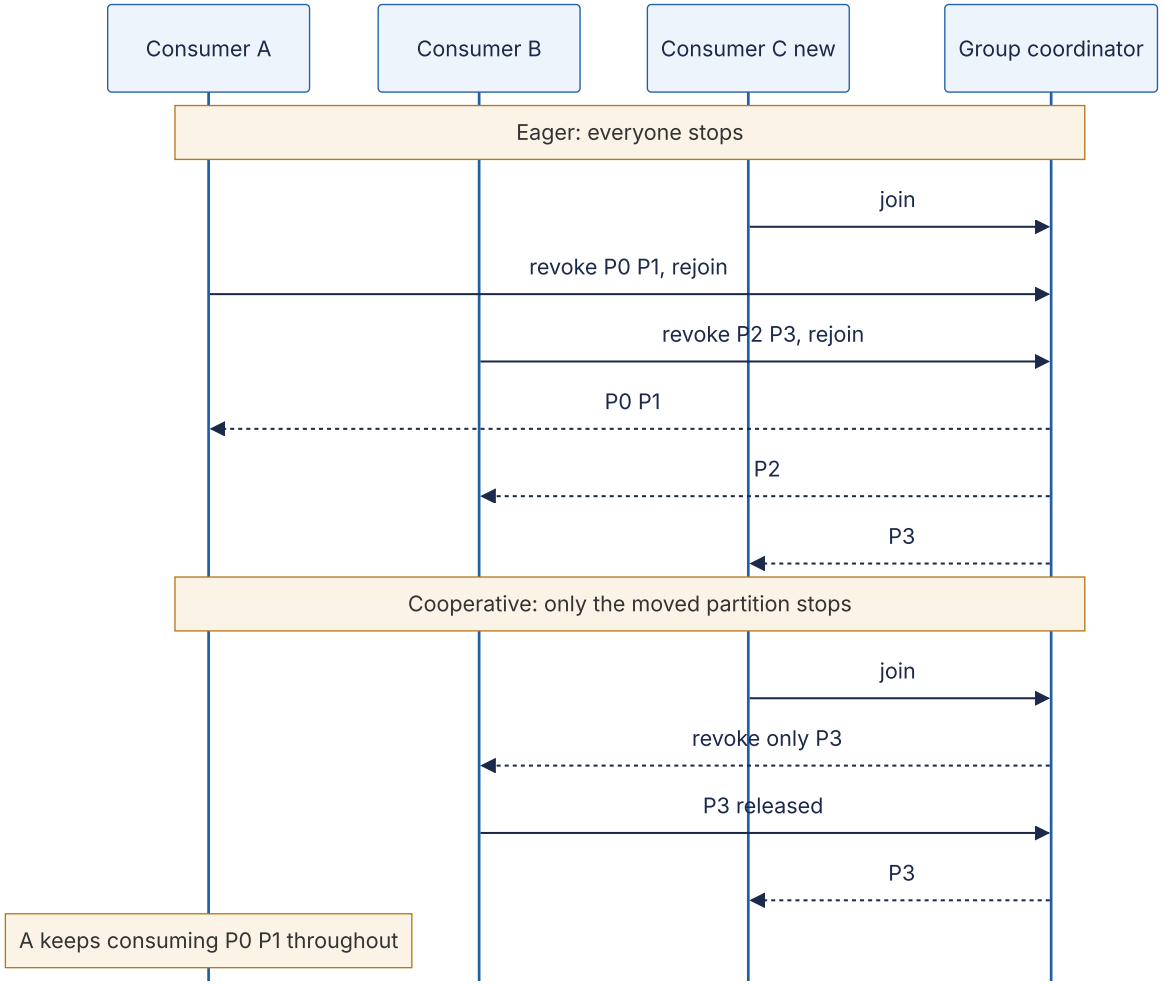
Kafka Streams enables this machinery with one setting, `processing.guarantee=exactly_once_v2`, committing by default every 100 ms under EOS. Stream processors such as Flink achieve the same end-to-end effect by tying Kafka transactions to their checkpoints (§29.9.2).

31.12 Rebalancing protocols

A **rebalance** reassigns partitions when a member joins, leaves, crashes or misses `max.poll.interval.ms`, or when partitions are added. Its cost is a gap in consumption and, if offsets were not committed, reprocessing.

1. **Eager rebalancing (classic protocol).** Every member revokes *all* its partitions, rejoins, the group leader (a client) computes a new assignment, and members resume. The whole group stops even if only one partition moves: "stop the world".
2. **Cooperative incremental rebalancing (KIP-429, Kafka 2.4).** With the `CooperativeStickyAssignor`, members keep partitions that are not moving; only partitions that change owner are revoked, over two shorter rounds. Most members never stop.
3. **Static membership (KIP-345, Kafka 2.3).** A member with a fixed `group.instance.id` that restarts within `session.timeout.ms` gets its old partitions back without any rebalance, which makes rolling deployments cheap.
4. **Next-generation consumer rebalance protocol (KIP-848).** The group coordinator on the broker, not a client leader, computes assignments and drives them incrementally through ordinary heartbeats; there is no group-wide synchronization barrier, so one member's slow join cannot stall the others. KIP-848 reached general availability in Kafka 4.0 and is enabled on brokers by default; as of 2026 Java consumers opt in with `group.protocol=consumer`.

Figure 31.14 — Eager versus cooperative rebalancing when consumer C joins.



Reading the diagram. In the eager half, C's arrival forces A and B to give up all four partitions, so nothing is consumed until the new assignment arrives. In the cooperative half, the coordinator asks only B to release P3, then assigns it to C; A never pauses and B pauses only P3. All messages are synchronous group-protocol calls. Failure point: one member slow to rejoin stretches an eager rebalance for the whole group, a cost that grows with group size.

Table 31.7 — Rebalance protocols (as of 2026).

Protocol	Who assigns	Pause during rebalance	Since
Eager (range, round-robin, sticky)	Client group leader	Whole group	Early versions
Cooperative sticky	Client group leader	Only moved partitions	2.4
Static membership (add-on)	Either	None on quick restart	2.3
KIP-848 consumer protocol	Broker coordinator	Only moved partitions, no global barrier	GA in 4.0

RECOMMENDATION

Reduce rebalances before speeding them up. Keep processing per `poll()` well under `max.poll.interval.ms` (lower `max.poll.records` if needed), use static membership for stateful or slow-starting consumers, commit offsets in the partition-revoked callback, and use a cooperative or KIP-848 protocol.

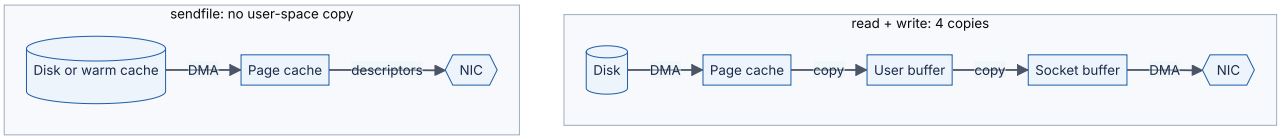
31.13 Performance

31.13.1 Why Kafka is fast

Kafka's throughput comes from a few hardware facts (Chapter 5) applied consistently [Apache Kafka Docs: Design]:

1. **Sequential I/O.** Appends and most reads are sequential, which disks and SSDs serve far faster than random access (§5.6).
2. **The page cache.** Brokers write to the OS page cache rather than a JVM-heap cache and let the kernel flush (§6.6.6); consumers near the head read from memory. This avoids double caching and garbage-collection pressure and survives broker restarts. Kafka does not `fsync` each write by default; durability comes from replication.
3. **Zero-copy transfer.** For plaintext connections the broker uses `sendfile` to move bytes from the page cache to the socket without user-space copies. TLS requires encryption in user space, which loses this (kernel TLS recovers part of it).
4. **End-to-end batching.** The producer's compressed batch is stored, replicated and sent to consumers as-is, so per-record costs are paid once per batch.

Figure 31.15 — Read path with and without zero-copy.



Reading the diagram. In the classic path, data crosses the kernel–user boundary twice, costing CPU copies and system calls. With `sendfile`, the kernel hands page-cache pages straight to the network card, so the broker's CPU barely touches the bytes. Both serve the same synchronous fetch. Failure point: a consumer reading old data causes disk reads that evict hot pages. Scaling: consumers of recent data are served from memory at network speed.

31.13.2 Sizing brokers for throughput

FORMULA

Brokers needed for network and disk throughput

$$N = \lceil \max\left(\frac{WR}{uD_b}, \frac{W(R-1+F)}{uE_b}\right) \rceil$$

Symbol	Meaning	Units
N	Brokers needed	count
W	Total produce throughput into the cluster (after compression)	MB/s
R	Replication factor	count
F	Number of consumer groups reading each record from brokers (fan-out)	count
D_b	Sustained disk write throughput per broker	MB/s
E_b	Network egress capacity per broker	MB/s
u	Target utilization (headroom for failures and catch-up)	fraction

Each byte is written to disk R times and leaves brokers $R-1$ times for replication plus F times for consumers. Network ingress (WR) is usually smaller than egress.

WORKED EXAMPLE

Worked example — A 400 MB/s cluster. $W = 400$ MB/s, $R = 3$, $F = 4$ consumer groups, each broker has $D_b = 1,000$ MB/s of disk write and a 25 Gb/s NIC ($E_b = 3,125$ MB/s), and you target $u = 0.5$ so a failed broker's load fits on the rest.

1. Disk: $400 \times 3 = 1,200$ MB/s cluster-wide; $1,200/(0.5 \times 1,000) = 2.4$.
2. Egress: $400 \times (3 - 1 + 4) = 2,400$ MB/s; $2,400/(0.5 \times 3,125) = 1.54$.
3. $N = \lceil 2.4 \rceil = 3$ brokers for throughput.
4. Storage (§31.15) and partition counts often require more; take the largest requirement.

Interpretation. At this scale disk writes, not the network, set the minimum, and consumers that read from the page cache add egress but almost no disk reads. **Engineering implication.** A lagging consumer that reads from disk adds disk *reads* not in this formula; keep headroom, or use tiered storage so historical reads come from object storage.

31.13.3 End-to-end latency budget

FORMULA

Produce-to-consume latency

$$T_{e2e} = t_{\text{linger}} + t_{\text{send}} + t_{\text{append}} + t_{\text{repl}} + t_{\text{fetch}} + t_{\text{proc}}$$

Symbol	Meaning	Units
t_{linger}	Time a record waits in the producer batch (≤ `linger.ms`)	ms
t_{send}	Network transfer to the leader	ms
t_{append}	Leader validation and append to page cache	ms
t_{repl}	Time until followers fetch it and the HW advances	ms
t_{fetch}	Consumer fetch wait and transfer after the record is committed	ms
t_{proc}	Consumer processing before the effect is visible	ms

WORKED EXAMPLE

Worked example — Budgeting a 50 ms p99 target. Within one region (round trips about 1 ms; cross-zone about 1–2 ms):

1. $t_{\text{linger}} = 5$ ms (default linger at low load).
2. $t_{\text{send}} = 1$ ms; $t_{\text{append}} = 1$ ms.
3. t_{repl}: followers fetch continuously across zones, typically 2–5 ms; at p99 under load, 10 ms.
4. t_{fetch}: a waiting consumer fetch returns as soon as data is committed, about 2 ms.
5. $t_{\text{proc}} = 10$ ms (one database write).
6. Total p99 estimate: $5 + 1 + 1 + 10 + 2 + 10 = 29$ ms, inside 50 ms with 21 ms to spare.

Interpretation. Replication and processing dominate, not disk. Consumers see records only after the HW advances, so with `acks=1` the *producer* is acknowledged sooner but the *consumer* still waits for replication. **Engineering implication.** If p99 is too high, check for slow followers holding back the HW, garbage-collection pauses, `read_committed` waiting on open transactions, and consumer lag, which adds queueing delay far larger than any term above.

INDUSTRY EXAMPLE

“2 million writes per second on three cheap machines.” A 2014 LinkedIn benchmark of Kafka on three commodity brokers reported about two million small-record writes per second with replication, and showed throughput staying flat as stored data grew, because reads and writes remain sequential and unaffected by log size [Kreps 2014]. The exact numbers depend on 2014 hardware and versions; the scaling argument is what transfers. *Evidence: Primary* — [Kreps 2014]

31.14 Kafka Streams and Kafka Connect

31.14.1 Kafka Streams

Kafka Streams is a Java library (not a cluster) for stream processing on Kafka. Its two core abstractions express the **stream–table duality**:

- A **KStream** is an unbounded sequence of independent events (“order 17 was paid”).
- A **KTable** is a changelog interpreted as a table: each record is an upsert for its key, and a tombstone deletes it. Aggregating a KStream produces a KTable; reading a KTable’s changes produces a KStream. A **GlobalKTable** is replicated in full to every instance for joins without repartitioning.

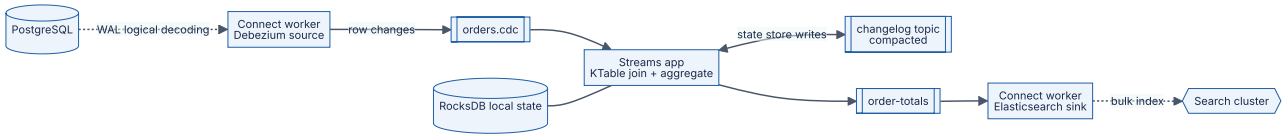
Stateful operations (aggregations, joins, windows, §29.8) keep **state stores**, by default embedded RocksDB instances (§13.14.2) on the instance's local disk. Every store write is also sent to a compacted **changelog topic**, so when an instance dies, another rebuilds the store by replaying the changelog; **standby replicas** (`num.standby.replicas`) keep warm copies to shorten that recovery. Re-keying a stream writes it through an internal **repartition topic** so records with the new key meet in one partition. Scaling is by partitions: each input partition becomes a **task**, and tasks are spread over instances using the consumer-group protocol.

31.14.2 Kafka Connect and Debezium

Kafka Connect is a framework for moving data between Kafka and other systems without custom code. A cluster of **workers** runs **connectors**, each split into **tasks**. **Source connectors** read from an external system into topics; **sink connectors** write topics to an external system. **Converters** serialize data (JSON, Avro or Protobuf, often with a schema registry), **single message transforms** make light per-record changes, and sink errors can be routed to a dead-letter topic (§30.11). In distributed mode, connector configurations, source offsets and status live in three internal compacted topics, so workers are stateless and restartable.

Debezium is a family of source connectors for **change data capture (CDC)**: it reads a database's own replication log (PostgreSQL logical decoding of the WAL, the MySQL binlog), takes an initial snapshot, then emits one event per committed row change, typically to a topic per table keyed by primary key [Debezium Docs]. Chapter 32 builds the transactional outbox and CDC patterns on it (§32.5–§32.6).

Figure 31.16 — Connect, Streams and CDC around one cluster.



Reading the diagram. A *Debezium* source connector tails PostgreSQL's WAL asynchronously and publishes row changes to `orders.cdc`. A *Kafka Streams* application joins and aggregates them, keeping state in local *RocksDB* backed by a compacted *changelog topic*, and writes results to `order-totals`. A *sink connector* bulk-indexes those into a search cluster. Failure points: a dead Streams instance's tasks move elsewhere and restore state from the changelog; a sink outage builds lag but loses nothing within retention. Scaling: add workers or instances up to the partition counts.

31.15 Operating Kafka

31.15.1 What to monitor

Table 31.8 — Core health signals.

Signal	Healthy value	What it means when wrong
`UnderReplicatedPartitions`	0	Followers lagging or a broker down; redundancy reduced
`UnderMinIsrPartitionCount`	0	`acks=all` writes are being refused for those partitions
`OfflinePartitionsCount`	0	No leader: partitions unavailable
`ActiveControllerCount`	1 across the cluster	0 means the KRaft quorum has no leader
ISR shrink/expand rate	Near 0	Flapping followers: network, disk or GC problems
Consumer lag (records and seconds)	Bounded, not growing	Consumers too slow or stuck (§31.7.3)
Produce/fetch request latency p99	Stable	Disk saturation, page-cache misses, overloaded broker

Routine operations include **partition reassignment** (moving replicas to new brokers, throttled so replication does not starve clients), **rolling restarts** with controlled leadership transfer, **quotas** per client to stop one tenant saturating a broker, and automated balancing tools such as LinkedIn's Cruise Control.

31.15.2 Capacity planning for disk

FORMULA

Local disk required for retention

$$S = \frac{W \cdot t_{\text{ret}} \cdot R}{u}$$

Symbol	Meaning	Units
S	Total provisioned broker disk	bytes
W	Average produce throughput after compression	bytes/s
t_{ret}	Retention period	s
R	Replication factor	count
u	Maximum safe disk utilization	fraction

WORKED EXAMPLE

Worked example — Seven days at 200 MB/s. W = 200 MB/s average, t_{ret} = 7 days = $604,800$ s, R = 3, u = 0.7.

1. One copy: $200 \times 10^6 \times 604,800 = 1.2096 \times 10^{14}$ B = 121 TB.
2. Three replicas: 363 TB.
3. Divide by 0.7: $S \approx 518$ TB.
4. With 12 TB usable per broker: 518/12 = 43.2, so 44 brokers, far more than the 2 that throughput needs at 200 MB/s by the formula of §31.13.2 (the 400 MB/s example there needed 3).
5. With **tiered storage** keeping 1 day locally: local $= 200 \times 10^6 \times 86,400 \times 3/0.7 = 74$ TB (7 brokers), plus 121 TB of one logical copy in object storage, whose durability is the object store's.

Interpretation. Retention, not throughput, sized this cluster. **Engineering implication.** Long retention on broker disks couples storage to compute; tiered storage decouples them and makes broker replacement and rebalancing faster because there is less local data to copy.

31.15.3 Tiered storage (KIP-405)

With **tiered storage**, closed segments are copied to a remote store (typically object storage) through a pluggable `RemoteStorageManager`, and deleted locally after `local.retention.ms`; the topic's `retention.ms` still governs total retention. Consumers reading old offsets are served transparently from the remote tier, at higher latency. Tiered storage was early access in Kafka 3.6 and declared production-ready in 3.9 (2024) [KIP-405]; as of 2026 it requires a third-party or vendor plugin for the specific object store.

Failure scenarios

What goes wrong	Why	Detection	Recovery	Prevention
Disk fills on a broker	Retention too long, skewed partitions	Disk usage alerts	Shorten retention, reassign partitions	Formula above with $u \le 0.7$; tiered storage
Lag grows without bound	Consumers slower than producers, or a poison record	Lag in seconds rising	Scale consumers, skip or dead-letter the record	Capacity headroom; DLQ handling (§30.11)
Under-min-ISR during maintenance	Two brokers restarted at once	`UnderMinIsrPartitionCount` > 0	Bring brokers back	Restart one broker at a time; wait for ISR to recover
Cross-zone transfer cost spike	Consumers fetch from leaders in other zones	Cloud bill, network metrics	Enable follower fetching (KIP-392)	Rack-aware placement and clients

31.16 Summary and revision notes

SUMMARY

- Kafka is a **partitioned, replicated, append-only log**. Consumers track their own offsets, so reading does not delete, many groups read independently, and history can be replayed (§31.3).
- **KRaft** stores metadata in a Raft-replicated `__cluster_metadata` log; ZooKeeper mode was removed in Kafka 4.0 (2025). Data partitions still use ISR replication (§31.4).
- Order holds **within a partition only**. Keys pick partitions by `murmur2 mod P`; changing P moves $1 - \min(P, P')/\operatorname{lcm}(P, P')$ of keys, so size keyed topics up front: $P = \lceil h \max(T/t_p, T/t_c) \rceil$ (§31.5).
- Producers batch by `linger.ms` and `batch.size`, compress per batch, and default to `acks=all` with idempotence (PID + sequence numbers, ordering preserved with up to 5 in-flight requests) (§31.6).
- A consumer group gives each partition to one member; parallelism ≤ partitions; lag drains in $L/(\mu - \lambda)$ (§31.7).
- Commit after processing = at-least-once; commit before = at-most-once; offsets live in compacted `__consumer_offsets` (§31.8).
- HW = min LEO over the ISR; consumers read below it. RF = 3, `min.insync.replicas=2`, `acks=all`, unclean election off survives one failure with no loss and no write outage. Leader epochs fence stale leaders and make truncation safe (§31.9).
- Retention deletes whole segments by time or size; compaction keeps the latest value per key and turns a topic into a changelog (§31.10).
- Transactions (2PC through a coordinator, fenced by `transactional.id` epochs, read via `read_committed`) give exactly-once for Kafka-to-Kafka read-process-write only (§31.11).
- Cooperative rebalancing, static membership and KIP-848 (GA in 4.0) cut rebalance pauses (§31.12).
- Speed comes from sequential I/O, the page cache, `sendfile` zero-copy and end-to-end batching; size brokers by disk, egress and especially retention: $S = W t_{\text{ret}} R/u$; tiered storage decouples retention from broker disks (§31.13–§31.15).

31.17 Knowledge checkpoint

1. Why can a Kafka consumer replay data that a classic queue consumer cannot?
2. What problem with ZooKeeper-mode controller failover did KRaft solve?
3. How does a broker find offset 368,800 in a partition with many segments?

4. A keyed topic grows from 10 to 15 partitions. What fraction of keys move?
5. With `linger.ms=5` and 400 records/s into a partition, how many records are in each batch?
6. Why does an idempotent producer preserve order with five requests in flight?
7. A group has 8 members and the topic has 6 partitions. What happens?
8. Which commit timing gives at-most-once delivery, and why?
9. Leader LEO 500, follower LEOs 480 and 470, all in the ISR. What is the HW, and what can consumers read?
10. Why is RF = 3 with `min.insync.replicas=3` a poor choice?
11. What does unclean leader election trade?
12. What does a tombstone do in a compacted topic, and why is it kept for a while?
13. What exactly does Kafka's exactly-once guarantee cover?
14. Why does TLS reduce Kafka's zero-copy benefit?
15. Retention 3 days, 50 MB/s, RF = 3, $u = 0.75$: how much disk?

Answers

1. Reading only advances the consumer's own offset; records remain until retention removes them, so any consumer can seek back.
2. A new controller had to reload all metadata from ZooKeeper, taking time proportional to partition count; KRaft standbys already hold the metadata log in memory.
3. Binary-search segments by base offset, binary-search that segment's sparse `.index`, then scan forward a few kilobytes.
4. $\text{lcm}(10, 15) = 30$, $\min = 10$: $1 - 10/30 = 2/3 \approx 67\%$.
5. $400 \times 0.005 = 2$ records (assuming they fit in `batch.size`).
6. The leader accepts batches only in sequence order per producer and partition; an out-of-order batch is rejected and resent.
7. Six members get one partition each; two are idle standbys.
8. Commit before processing: a crash after the commit skips the uncommitted work, losing it but never duplicating it.
9. HW = 470; consumers read offsets 0–469.
10. Any single broker outage, including a rolling restart, shrinks the ISR below 3 and blocks `acks=all` writes.
11. It restores availability by electing an out-of-sync replica, losing acknowledged records that replica lacked.
12. It marks the key deleted; it is retained for `delete.retention.ms` so consumers replaying the log see the deletion before it disappears.
13. Atomic output to Kafka topics plus input offset commits in a read-process-write pipeline, seen by `read_committed` consumers; not external side effects.
14. Data must be encrypted in user space, so the kernel cannot send page-cache pages directly to the socket.
15. $50 \times 10^6 \times 259{,}200 \times 3/0.75 = 5.18 \times 10^{13}$ B $\approx$ 52 TB.

31.18 Interview questions

Basic

Q1. How does Kafka guarantee ordering? Answer. Only within a partition: the leader assigns increasing offsets and consumers read in offset order. Records that must stay ordered share a key so they hash to one partition. There is no order across partitions, and changing the partition count breaks per-key order for in-flight data.

Q2. What is a consumer group? Answer. Consumers sharing a `group.id` that divide partitions so each partition has one owner; different groups read independently. Parallelism is capped by partition count.

Intermediate

Q3. Configure a topic so an acknowledged write survives any single broker failure without making writes unavailable. Answer. RF = 3 across three racks or zones, `min.insync.replicas=2`, producer `acks=all` with idempotence, unclean leader election disabled. One failure leaves two in-sync replicas, so writes continue and every acknowledged record is on at least two brokers.

Q4. Consumer lag is growing after a deploy. How do you diagnose it? Answer. Check whether lag grows on all partitions (throughput: processing slowed, too few consumers, downstream database slow) or a few (hot key, poison record, stuck consumer). Look for frequent rebalances (processing exceeding `max.poll.interval.ms`), compare production rate with consumption rate, compute drain time, and scale consumers only up to the partition count.

Advanced

Q5. Design exactly-once processing from a Kafka topic into PostgreSQL. Answer. Kafka transactions do not cover PostgreSQL. Either write results and the consumed offset in one PostgreSQL transaction and seek to the stored offset on start-up, or make writes idempotent (upsert keyed by event ID or a processed-events table) with at-least-once commits. Mention zombie consumers after rebalance: the database transaction or a fencing check on partition ownership must reject stale writers.

Q6. Why does Kafka use ISR replication for data but Raft for metadata? Answer. ISR commits when all in-sync replicas have a record, tolerating f failures with $f + 1$ copies, which halves storage relative to majority quorums for large data volumes; it relies on an external authority to manage ISR membership and elections. That authority must itself be consistent without external help, so the metadata quorum uses Raft, where majority commit needs no outside arbiter and slow members do not block progress.

31.19 Practical exercises

1. Run the simulation of §31.9.5 with `fetch_every=(1, 1, 1)` and `acks=1`. Is anything lost? Explain why the loss window depends on replication lag.
2. Extend the simulation with HW propagation delay: followers learn the HW one fetch later. Construct a double failure that loses committed data if followers truncate to their own HW, then fix it with leader-epoch truncation.
3. Using the partitioner code, measure per-partition counts for 1,000,000 keys of the form `tenant-<n>` with 48 partitions, then for keys where 5% of traffic is one tenant. Compute the hottest partition's share.
4. Write a consumer loop (pseudo-code) that commits asynchronously during normal operation and synchronously on revocation and shutdown.
5. Size a cluster for 1 GB/s, 3-day retention, RF = 3, 5 consumer groups, 20 TB disks and 25 Gb/s NICs, using the formulas of §31.13.2 and §31.15.2.

31.20 Design challenges

1. **Order event backbone.** Design topics, keys, partition counts and durability settings for an e-commerce platform with 30,000 orders/s at peak, per-order ordering, a 30-day replay requirement and consumers for billing, search and analytics. Justify each number and plan how to grow partitions in two years.
2. **Multi-region log.** Two regions must both consume all orders, and the platform must survive a region loss with at most 5 seconds of data loss. Compare stretched clusters with asynchronous cross-cluster replication, and state the recovery point and recovery time of each.
3. **Changelog-backed cache.** Replace nightly cache rebuilds with a compacted topic fed by CDC. Specify tombstone handling, bootstrap time for a new cache node, and how consumers detect that they have caught up.

31.21 References

1. Kreps, J., Narkhede, N., Rao, J. *Kafka: a Distributed Messaging System for Log Processing*. NetDB Workshop, 2011.
2. Kreps, J. *The Log: What every software engineer should know about real-time data's unifying abstraction*. LinkedIn Engineering blog, 2013. https://engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying
3. Wang, G., Koshy, J., Subramanian, S., Paramasivam, K., Zadeh, M., Narkhede, N., Rao, J., Kreps, J., Stein, J. *Building a Replicated Logging System with Apache Kafka*. Proceedings of the VLDB Endowment 8(12), 2015.

4. Shapira, G., Palino, T., Sivaram, R., Petty, K. *Kafka: The Definitive Guide*, 2nd ed. O'Reilly Media, 2021.
5. Apache Software Foundation. *Apache Kafka Documentation* (Design, Replication, Configuration, Operations). Accessed 2026. https://kafka.apache.org/documentation/
6. Apache Software Foundation. *Apache Kafka 4.0.0 Release Announcement and Upgrade Notes*. March 2025. https://kafka.apache.org/blog
7. Apache Kafka. *KIP-500: Replace ZooKeeper with a Self-Managed Metadata Quorum*. https://cwiki.apache.org/confluence/display/KAFKA/KIP-500%3A+Replace+ZooKeeper+with+a+Self-Managed+Metadata+Quorum
8. Apache Kafka. *KIP-98: Exactly Once Delivery and Transactional Messaging*. 2017. https://cwiki.apache.org/confluence/display/KAFKA/KIP-98+-+Exactly+Once+Delivery+and+Transactional+Messaging
9. Apache Kafka. *KIP-101: Alter Replication Protocol to use Leader Epoch rather than High Watermark for Truncation*. 2017. https://cwiki.apache.org/confluence/display/KAFKA/
10. Apache Kafka. *KIP-405: Kafka Tiered Storage*. https://cwiki.apache.org/confluence/display/KAFKA/KIP-405%3A+Kafka+Tiered+Storage
11. Apache Kafka. *KIP-848: The Next Generation of the Consumer Rebalance Protocol*. https://cwiki.apache.org/confluence/display/KAFKA/
12. Apache Kafka. *KIP-345 (static membership), KIP-392 (follower fetching), KIP-429 (incremental cooperative rebalancing), KIP-480 and KIP-794 (sticky partitioner), KIP-679 (producer safety defaults), KIP-932 (queues for Kafka), KIP-1030 (default value changes in 4.0)*. https://cwiki.apache.org/confluence/display/KAFKA/Kafka+Improvement+Proposals
13. Rao, J. *How to choose the number of topics/partitions in a Kafka cluster?* Confluent blog, 2015.
14. Kreps, J. *Benchmarking Apache Kafka: 2 Million Writes Per Second (On Three Cheap Machines)*. LinkedIn Engineering blog, 2014.
15. Svingen, B. *Publishing with Apache Kafka at The New York Times*. Confluent blog, 2017.
16. Debezium Project. *Debezium Documentation*. Accessed 2026. https://debezium.io/documentation/

31.22 Open verification items

- Exact Kafka release for each status as of 2026: KRaft production-ready 3.3, ZooKeeper deprecated 3.5, removed 4.0 (March 2025); KIP-848 GA in 4.0 with client opt-in via `group.protocol=consumer`; tiered storage production-ready in 3.9.
- KIP-932 share groups: maturity level in the current 4.x release.

- Defaults quoted: `linger.ms` 5 ms (4.0, KIP-1030), `session.timeout.ms` 45 s, `replica.lag.time.max.ms` 30 s, `__consumer_offsets` 50 partitions, Streams EOS commit interval 100 ms.
- Kreps 2014 benchmark figures and Svingen 2017 details (single-partition compacted topic) against the original posts.
- Exact URLs for KIP-101, KIP-848 and the 4.0 release announcement.
- The brief listed *Kafka: The Definitive Guide* (2nd ed.) as Narkhede, Shapira and Palino; the 2021 edition's authors are Shapira, Palino, Sivaram and Petty (Narkhede co-authored the 2017 first edition).

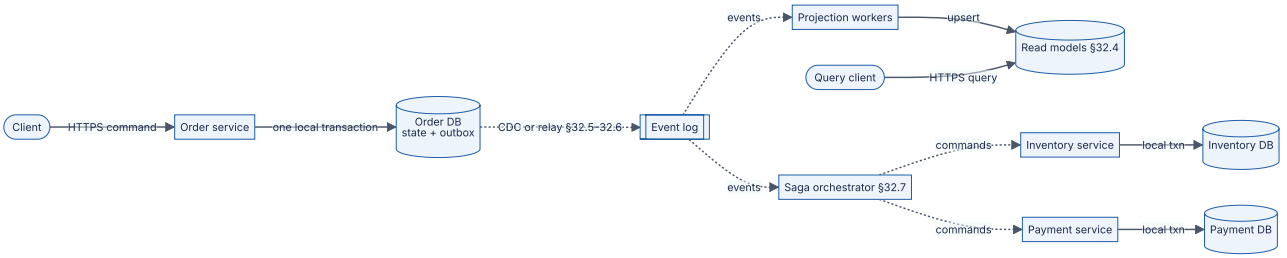
Chapter 32 — Event-Driven Architecture Patterns

> **Level:** 3–4 · **Track:** A (Traditional) · **Estimated reading time:** 5 h

Chapter 30 introduced messages, brokers and delivery guarantees, and Chapter 31 showed how Kafka stores events in a replicated, partitioned log. This chapter is about what you *build* with those tools when a business process spans several services, each owning its own database. Four problems recur. How do you record what happened so that you can rebuild, audit and re-interpret it later (event sourcing)? How do you serve queries that the write model answers badly (CQRS)? How do you change a database *and* tell the world about it without losing or inventing events (the transactional outbox and change data capture)? And how do you keep a multi-step business transaction correct when 2PC is not an option (sagas, choreography, orchestration and workflow engines)?

Underneath all four sits one uncomfortable fact: messages are delivered at least once, so every consumer must be safe to run twice. Sections 32.9 and 32.10 make that precise. The running example is the online shop of Chapter 11, now split into order, inventory, payment and shipping services; it is *illustrative*, not a description of any real company. Two-phase commit is Chapter 28's subject; API idempotency keys are owned by §22.15 and only applied here.

Figure 32.1 — How the patterns in this chapter fit together.



Reading the diagram. The only synchronous write a user waits for is the order service's local transaction, which stores the new state and, in the same transaction, an outbox row (or, with event sourcing, the events themselves). Everything after that is asynchronous (dotted arrows): a relay or change data capture (CDC) process copies committed events to the event log; projection workers build read models; a saga orchestrator sends commands to the inventory and payment services, each of which commits locally. Each database is a separate storage boundary, and no transaction spans two of them. Failure points: the relay, the log, each consumer and each service can crash between any two

steps, so every arrow is retried and every receiver deduplicates. Scaling is per component: partition the log by order ID, run more projection workers per partition, and shard the orchestrator by saga ID.

32.1 Learning objectives

After this chapter you should be able to:

- Design an event-sourced aggregate with optimistic concurrency, snapshots and schema upcasting, and estimate its replay time.
- Explain how event sourcing interacts with the GDPR right to erasure and apply crypto-shredding.
- Build CQRS read models from an event stream, estimate projection lag, and say when CQRS is not worth it.
- Show with a failure sequence why dual writes lose or invent events, and implement a transactional outbox with a polling or log-tailing relay.
- Describe how log-based change data capture (Debezium) snapshots and streams a database, and the role of a schema registry.
- Design a saga with compensating, pivot and retriable steps, analyze a failure at each step, and choose countermeasures for its missing isolation.
- Compare choreography and orchestration, and decide when a workflow engine (Temporal, Step Functions) beats a hand-rolled saga.
- State precisely what “exactly-once” means in Kafka and why end-to-end effectively-once needs idempotent consumers.

32.2 Prerequisites

- Messages, events versus commands, delivery guarantees and dead-letter queues — Chapter 30 (this chapter assumes at-least-once delivery).
- Kafka partitions, offsets, idempotent producers and transactions — §31.5–§31.11 (the event log underneath most examples).
- Local transactions, isolation anomalies and the write-ahead log — §11.11–§11.15 and §13.5 (the outbox and CDC rely on them).
- Logical replication — §14.4 (CDC is a consumer of it).
- Two-phase commit and the saga preview — §28.4 and §28.6 (sagas are the alternative to 2PC).
- Idempotency keys — §22.15 (owned there; applied to consumers here).
- The end-to-end argument — §24.14 (why exactly-once must be finished by the application).

32.3 Event sourcing

32.3.1 Storing facts instead of current state

> DEFINITION
>
> **Definition — Event sourcing.** A persistence pattern in which the system of record for an entity is the append-only, ordered sequence of **domain events** that changed it (`AccountOpened`, `MoneyDeposited`, …). Current state is not stored as truth; it is derived by replaying the events, a **left fold** of a pure function `apply(state, event)` over the sequence [Fowler 2005].

First-principles explanation

An accountant never erases a ledger line. If a payment was booked wrongly, a correcting entry is added, and the balance is the sum of all lines. A bank statement is a *view* computed from the ledger. Event sourcing applies the same discipline to software: instead of `UPDATE accounts SET balance = 4000`, you append `MoneyWithdrawn{amount: 1000}`, and the balance is whatever the events add up to.

Why does this exist?

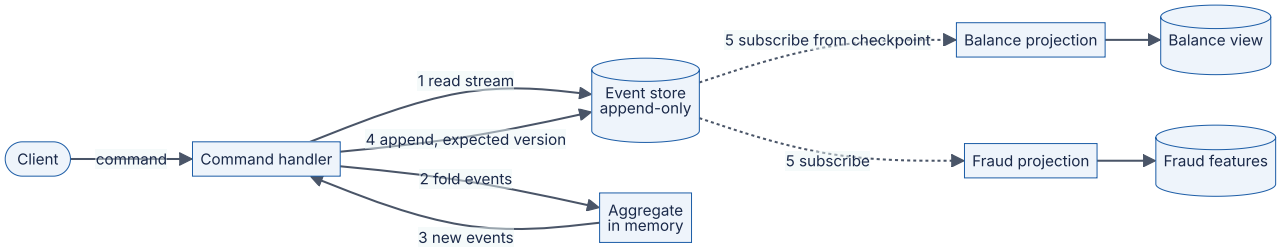
A conventional table that is updated in place answers "what is true now?" and forgets "how did we get here?". Many domains need the second question: auditors, fraud analysts, customer-support agents ("why was I charged twice?"), and engineers who want to build a *new* view of old data, such as "average time from cart to checkout last year", without having logged it in advance. With state-oriented storage, history survives only if someone remembered to write an audit table, and audit tables drift from the truth because they are a second write. With event sourcing the history *is* the truth, so it cannot drift. Other benefits follow: temporal queries ("the balance on 31 March"), debugging by replaying production events, and natural integration, because the events can be published to other services.

How it works internally

1. **Streams and aggregates.** Events are grouped into **streams**, one per **aggregate**: a cluster of objects that is a consistency boundary, such as one account or one order (a term from domain-driven design). An invariant ("balance never negative") is enforced inside one aggregate; nothing is atomic across aggregates.
2. **Command handling.** A command (`Withdraw 4,000`) loads the aggregate by replaying its stream, validates the command against current state, and, if valid, produces new events. The command can be rejected; an event cannot, because it describes something that already happened.

3. **Optimistic concurrency.** The append carries the **expected version**, the stream length the handler saw. If another writer appended in between, the store refuses the append and the handler reloads and retries. This is the event-store equivalent of a compare-and-set, and it serializes writes per aggregate without locks.
4. **The event store.** Any durable, ordered, append-only storage works: a relational table with a unique key on `(stream_id, version)`, a purpose-built store such as EventStoreDB, or a log. Kafka is a poor fit as the *only* store, because it has no per-key expected-version append and loading one aggregate means scanning a partition; it is an excellent transport for the events afterwards [Kleppmann 2017, ch. 11].
5. **Projections.** Subscribers fold events into read-optimized views (§32.4), each tracking a **checkpoint**, the global position of the last event it applied.

Figure 32.2 — The write path of an event-sourced service.

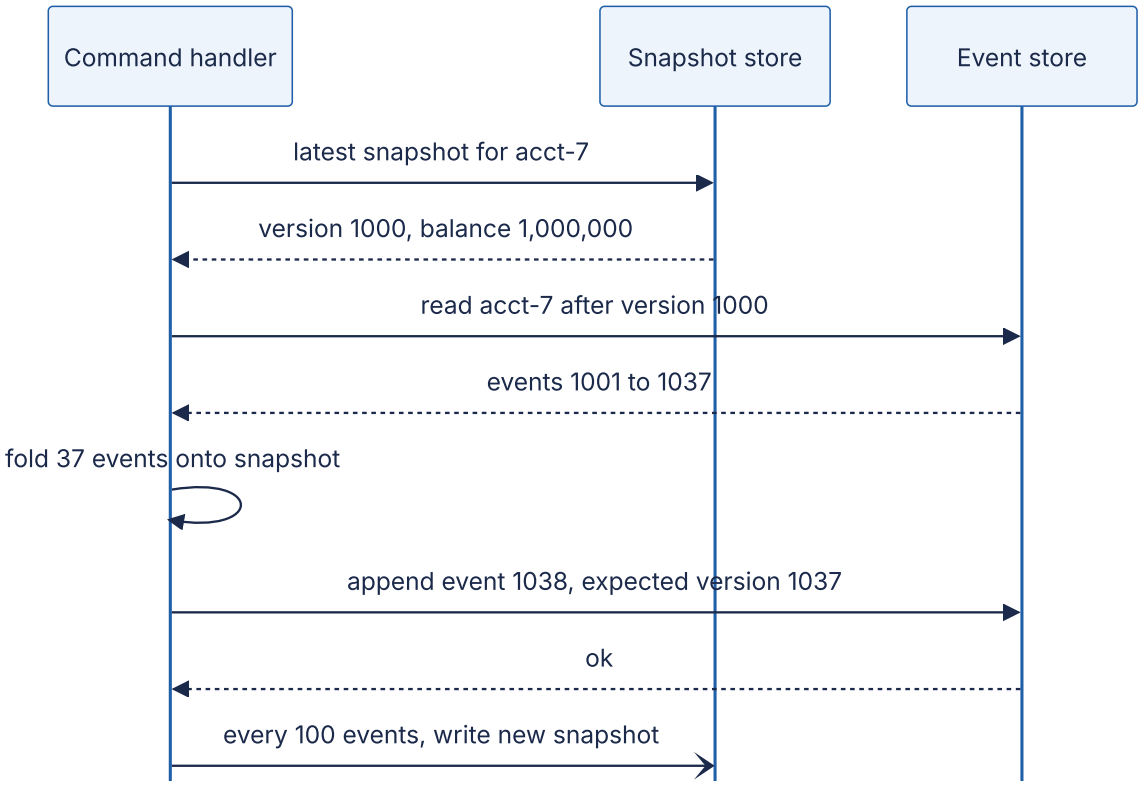


Reading the diagram. Steps 1–4 are synchronous and run inside the command handler; the aggregate exists only in memory, rebuilt from the event store for each command (or cached). The append in step 4 is the single atomic write and the commit point; an expected-version mismatch fails it and the handler retries from step 1. Step 5 is asynchronous: each projection pulls events after its own checkpoint and updates its own store, so views lag the event store and can be deleted and rebuilt. Failure points: a crash before step 4 loses nothing (no event was recorded, the client retries with an idempotency key, §22.15); a projection crash leaves its view stale until it resumes from its checkpoint. Scaling: streams partition naturally by aggregate ID; projections scale by partitioning the event feed.

32.3.2 Rebuilding state and snapshots

Replaying a stream of a few dozen events is cheap. Long-lived aggregates (a merchant account with millions of transactions) and full projection rebuilds are not. A **snapshot** stores the aggregate's state at version v; loading reads the latest snapshot and replays only events after v. A snapshot is a cache: it can be deleted and recomputed, and it must record the schema version of the state it holds, because changing the `apply` logic invalidates old snapshots.

Figure 32.3 — Loading an aggregate from a snapshot plus its tail.



Reading the diagram. Both stores are durable storage boundaries; they may be two tables in one database. The handler reads synchronously from both, folds at most one snapshot interval of events, and appends with the expected version. Writing the snapshot is asynchronous and optional (the half-arrow): if it fails or the process crashes, the next load simply replays more events. The failure to watch is a stale snapshot built with an older `apply` function; version the snapshot schema and discard mismatches. Snapshot frequency trades storage and write work against load latency.

Mathematical foundations

FORMULA

Replay time with and without snapshots

$$T_{\text{full}} = \frac{N}{r} \qquad T_{\text{snap}} \approx t_s + \frac{S}{2r}$$

Symbol	Meaning	Units
N	Events in the stream (or in the whole store, for a projection rebuild)	events
r	Replay throughput: events read and applied per second	events/s
S	Snapshot interval; on average $S/2$ events follow the latest snapshot	events
t_s	Time to read and deserialize one snapshot	s
$T_{\text{full}}, T_{\text{snap}}$	Time to rebuild state from scratch or from a snapshot	s

WORKED EXAMPLE

Worked example — A busy merchant account and a full rebuild.

1. A merchant account has $N = 50,000$ events; the handler replays $r = 20,000$ events/s. $T_{\text{full}} = 50,000/20,000 = 2.5$ s per command, unacceptable for a payment API.
2. Snapshot every $S = 500$ events with $t_s = 2$ ms: $T_{\text{snap}} = 0.002 + 250/20,000 = 0.002 + 0.0125 = 14.5$ ms.
3. A new fraud projection must process the whole store: $N = 2 \times 10^9$ events at $r = 100,000$ events/s gives $T_{\text{full}} = 20,000$ s ≈ 5.6 h on one worker. With 16 partitions processed in parallel at the same per-worker rate, about 21 minutes.

Interpretation: snapshots turn per-command cost from $O(N)$ into $O(S)$; they do nothing for projection rebuilds, which need parallelism. Engineering implication: snapshot long-lived aggregates (or, better, model them so streams stay short, for example one stream per merchant per day), and design projections to be partitionable from the start.

32.3.3 Code example: an event-sourced account with snapshots

The program below implements an event store with expected-version appends, an account aggregate, a repository that snapshots every S events, and an upcaster for an old event schema (§32.3.4).

```
        """Event-sourced bank account with optimistic concurrency, snapshots and upcast-
ing.

        The event store is append-only. Account state is never stored directly: it is
        rebuilt by replaying events, optionally starting from a snapshot.
        Run it directly with python3 to execute the tests.
        """
        import unittest


        class ConcurrencyError(Exception):
            """Another writer appended to the stream since we loaded it."""


        class EventStore:
            def __init__(self):
                self.streams = {}    # stream_id -> list of event dicts (version = index
+ 1)
                self.snapshots = {}  # stream_id -> (version, state dict)
                self.reads = 0       # events read from storage, to measure replay cost

            def append(self, stream_id, events, expected_version):
                stream = self.streams.setdefault(stream_id, [])
                if len(stream) != expected_version:          # optimistic concurrency
check
                    raise ConcurrencyError(f"{stream_id}: expected {expected_version},
at {len(stream)}")
                stream.extend(events)
                return len(stream)

            def read(self, stream_id, after_version=0):
                events = self.streams.get(stream_id, [])[after_version:]
                self.reads += len(events)
                return [upcast(e) for e in events]           # old shapes are upgraded
on read


        def upcast(event):
            """Schema v1 (no "schema" field) stored float dollars; v2 stores integer
cents."""
            if "schema" not in event and "amount" in event:
                event = {"type": event["type"], "schema": 2,
                         "amount_cents": round(event["amount"] * 100)}
            return event                                     # a new dict: storage is
untouched


        class InsufficientFunds(Exception):
            pass


        class Account:
            """Aggregate: decides with current state, records decisions as events."""

            def __init__(self, account_id):
                self.id, self.version = account_id, 0
```

```
                self.balance_cents, self.opened = 0, False
                self.pending = []                           # new events not yet saved

            # --- state transitions: pure functions of (state, event) ---
            def apply(self, event):
                kind = event["type"]
                if kind == "AccountOpened":
                    self.opened = True
                elif kind == "MoneyDeposited":
                    self.balance_cents += event["amount_cents"]
                elif kind == "MoneyWithdrawn":
                    self.balance_cents -= event["amount_cents"]
                self.version += 1

            # --- commands: validate, then emit events ---
            def _emit(self, event):
                event["schema"] = 2
                self.apply(event)
                self.pending.append(event)

            def open(self):
                if self.opened:
                    raise ValueError("already open")
                self._emit({"type": "AccountOpened"})

            def deposit(self, cents):
                self._emit({"type": "MoneyDeposited", "amount_cents": cents})

            def withdraw(self, cents):
                if cents > self.balance_cents:
                    raise InsufficientFunds(f"balance {self.balance_cents}, asked
{cents}")
                self._emit({"type": "MoneyWithdrawn", "amount_cents": cents})

            def snapshot(self):
                return {"balance_cents": self.balance_cents, "opened": self.opened}


        class Repository:
            def __init__(self, store, snapshot_every=None):
                self.store, self.every = store, snapshot_every

            def load(self, account_id):
                acct, start = Account(account_id), 0
                if self.every and account_id in self.store.snapshots:
                    start, state = self.store.snapshots[account_id]
                    acct.balance_cents, acct.opened = state["balance_cents"],
state["opened"]
                    acct.version = start
                for event in self.store.read(account_id, after_version=start):
                    acct.apply(event)
                return acct

            def save(self, acct):
                expected = acct.version - len(acct.pending)
                new_version = self.store.append(acct.id, acct.pending, expected)
                acct.pending = []
```

```
                if self.every and new_version // self.every > expected // self.every:
                    self.store.snapshots[acct.id] = (new_version, acct.snapshot())


        class Tests(unittest.TestCase):
            def history(self, repo, n):
                acct = repo.load("acct-7")
                acct.open()
                repo.save(acct)
                for _ in range(n):
                    acct = repo.load("acct-7")
                    acct.deposit(1_000)
                    repo.save(acct)

            def test_snapshot_and_full_replay_agree(self):
                store = EventStore()
                self.history(Repository(store, snapshot_every=100), 1_000)
                full = Repository(store).load("acct-7")              # no snapshots
                fast = Repository(store, snapshot_every=100).load("acct-7")
                self.assertEqual((full.balance_cents, full.version),
(fast.balance_cents, fast.version))
                self.assertEqual(full.balance_cents, 1_000_000)

            def test_snapshot_bounds_replay_cost(self):
                store = EventStore()
                repo = Repository(store, snapshot_every=100)
                self.history(repo, 1_000)
                store.reads = 0
                repo.load("acct-7")
                self.assertLess(store.reads, 100)                    # only the tail af-
ter the snapshot
                store.reads = 0
                Repository(store).load("acct-7")
                self.assertEqual(store.reads, 1_001)

            def test_concurrent_writers_conflict(self):
                repo = Repository(EventStore())
                a = repo.load("acct-7"); a.open(); a.deposit(5_000); repo.save(a)
                teller, atm = repo.load("acct-7"), repo.load("acct-7")
                teller.withdraw(4_000); atm.withdraw(4_000)          # each sees 5,000
                repo.save(teller)
                with self.assertRaises(ConcurrencyError):            # the ATM must
reload and retry
                    repo.save(atm)
                retry = repo.load("acct-7")
                with self.assertRaises(InsufficientFunds):           # invariant holds
after reload
                    retry.withdraw(4_000)

            def test_v1_events_are_upcast_on_read(self):
                store = EventStore()
                store.streams["acct-9"] = [{"type": "AccountOpened"},
                                           {"type": "MoneyDeposited", "amount": 12.34}]
# v1, 2019
                acct = Repository(store).load("acct-9")
                self.assertEqual(acct.balance_cents, 1_234)
                self.assertNotIn("schema", store.streams["acct-9"][1])  # history is im-
```

```
mutable


        if __name__ == "__main__":
            unittest.main(verbosity=1)
```

Output (Python 3.13; the timing varies):

```
....
----------------------------------------------------------------------
Ran 4 tests in 0.041s

OK
```

What to notice:

- `apply` never validates and never fails; validation lives in the command methods. Replaying history must always succeed, even for events that today's rules would reject.
- `test_concurrent_writers_conflict` is the lost-update anomaly of §11.13 prevented by the expected version: both writers saw 5,000, only the first append wins, and the reloaded second writer is refused by the invariant.
- With a snapshot every 100 events, loading reads 1 event instead of 1,001. The snapshot is written *after* the append succeeds, so it never contains uncommitted state.
- The upcaster converts the stored v1 event into a new dictionary on every read; the stored history is never rewritten.

32.3.4 Schema evolution and upcasting

Events are kept for years, but the code that reads them changes weekly. A message schema in Chapter 30 must survive only until its consumers drain the queue; an event schema must survive forever. Every change must therefore be **backward compatible for readers**: the current code must understand every version ever written.

Table 32.1 — Strategies for evolving event schemas.

Strategy	How it works	Use when	Cost
Additive change	Add optional fields with defaults; never rename or retype	Most changes	Lowest; enforce with a schema registry (§32.6.3)
Weak schema	Readers map JSON by name and ignore unknown fields	Polyglot consumers	Silent misreads if meaning changes
Upcasting	A function converts version k to $k+1$ at read time; chain them	Renames, unit changes, splits	Every read pays the conversion; chains grow
New event type	`OrderShippedV2` alongside the old type	Meaning changed	Every consumer handles both forever
Copy-and-transform migration	Write a new store with converted events, switch over	Accumulated upcasters slow reads	A large batch job; history now differs from what was originally written

Young's guidance is to prefer additive changes and upcasting, and to treat rewriting history as a last resort, because the event log is also an audit record [Young 2017].

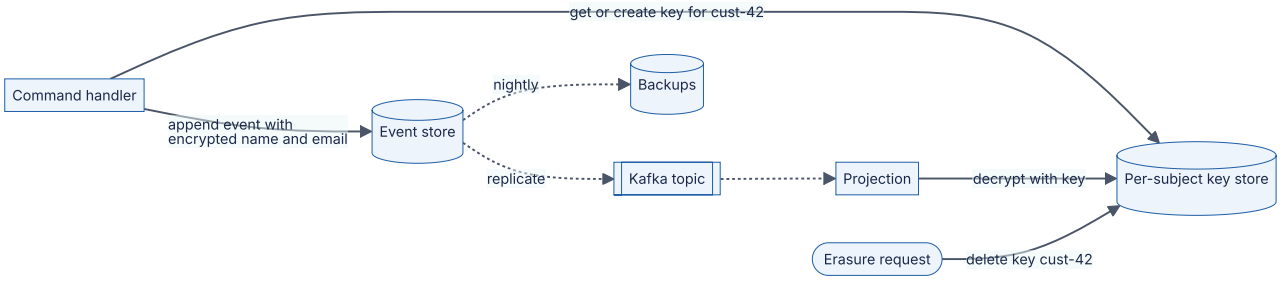
32.3.5 Deletion, the GDPR and crypto-shredding

Immutability collides with law. The EU General Data Protection Regulation gives people a **right to erasure** (Article 17) of their personal data in many circumstances [EU 2016/679]. An append-only store that holds `CustomerRegistered{name, email}` cannot simply delete it, and neither can the Kafka topics, backups and projections that copied it.

Three techniques, usually combined:

1. **Keep personal data out of events.** Events carry an opaque `customer_id`; the name and address live in a mutable store that can delete rows. This is the cleanest answer but not always possible (a shipping event needs an address).
2. **Crypto-shredding.** Encrypt each data subject's personal fields with a per-subject key held in a key store. To erase, delete the key: every copy of the ciphertext, in every log, backup and replica, becomes unreadable at once.
3. **Compaction or rewriting** for systems where events can be replaced, such as a Kafka compacted topic with a tombstone (§31.10). This removes data only where compaction reaches; backups still need handling.

Figure 32.4 — Crypto-shredding personal fields in an event store.



Reading the diagram. Only the key store is mutable and deletable; the event store, the Kafka topic and the backups hold ciphertext. Writes and decrypting reads call the key store synchronously; replication and backup are asynchronous copies. After the erasure request deletes the key, projections replaying old events receive a “key not found” error and must render the fields as erased rather than fail. Failure points: the key store becomes a critical dependency on every read and write (cache keys with a short TTL, and replicate the key store), and a key leaked before deletion defeats the scheme. Scaling: one key per subject means millions of keys, so use envelope encryption with a key-management service.

WARNING / TRADE-OFF

Warning — Crypto-shredding is an argument, not a guarantee. Whether deleting a key satisfies erasure under a particular regulator’s reading is a legal question; non-personal fields (amounts, timestamps) remain and may still identify someone in combination. Involve your privacy counsel, and keep personal data out of events where you can.

32.3.6 Trade-offs, industry use and failure scenarios

INDUSTRY EXAMPLE

LMAX exchange — event sourcing in memory. LMAX’s retail trading platform runs its business logic on a single thread over in-memory state. Every input event is journaled and replicated before processing, and state is recovered after a restart by loading a nightly snapshot and replaying the journal. Fowler reports the business logic processor handling about six million orders per second, possible because no business step waits on a database. *Evidence: Primary* — [Fowler 2011a]

Advantages. A complete, trustworthy audit trail; the ability to build new read models from old history; temporal queries; replay for debugging; natural event publication; append-only writes that are cheap and contention-free across aggregates.

Disadvantages. A steep learning curve; eventual consistency for every read model; schema evolution forever; hard deletion; no ad hoc queries against the event store; and

aggregate boundaries that are expensive to change once events exist.

Table 32.2 — Event sourcing failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Lost update	Append without expected version	Invariant violations in reconciliation	Compensating events	Always append with expected version
Replay crashes on old event	`apply` validates, or code dropped an old type	Rebuild jobs fail	Hotfix the upcaster, replay	Never validate in `apply`; replay tests over production samples
Stale or corrupt snapshot	Snapshot written by older logic	Snapshot-vs-replay comparison	Delete snapshots, rebuild	Version snapshots; sample-verify against full replay
Commands slow down over time	Streams grow without bound	Load latency vs stream length	Add snapshots	Short-lived streams; "closing the books" events
Projection serves wrong data	Bug in projection code	Diffs against a rebuilt view	Fix and rebuild from position 0	Blue-green projections

COMMON MISTAKE

Common misconception — "Event sourcing means using Kafka as the database." Event sourcing needs per-aggregate reads and expected-version appends; a Kafka topic offers neither directly. Store events in a database or event store and *publish* them to Kafka (§32.5, §32.6).

COMMON MISTAKE

Common misconception — "Every service should be event sourced." Most CRUD data (user preferences, product descriptions) gains nothing from history and pays every cost above. Use event sourcing for the few aggregates where history *is* the business: ledgers, orders, inventory movements.

32.4 CQRS

32.4.1 Separating the write model from read models

DEFINITION

Definition — Command Query Responsibility Segregation (CQRS). An architecture that uses one model to process commands (writes that enforce invariants) and one or more separate models to answer queries, each read model shaped for the queries it serves and typically updated asynchronously from the write side's events [Young 2010; Fowler 2011b].

First-principles explanation

A restaurant kitchen works from order tickets organized for cooking; the front-of-house display shows "table 12: mains in 5 minutes", organized for waiters. Nobody makes the chefs rearrange tickets to answer waiters' questions; a runner copies the relevant facts to the display. CQRS gives the "runner" a name: a **projection** that copies facts from the write model into views built for readers.

Why does this exist?

One schema rarely serves both sides well. The order write model is normalized around invariants (an order, its lines, a version column). The queries are varied: "a customer's last 20 orders with product images" (a document per customer), "full-text search over orders" (Chapter 16), "revenue per region per hour" (a column store). Forcing all of them through one schema means heavy joins on the write database and indexes that slow every write. Read and write load also scale differently: shops commonly serve tens to hundreds of reads per write. Bertrand Meyer's **command-query separation** applied the idea to methods; Young applied it to whole models [Young 2010].

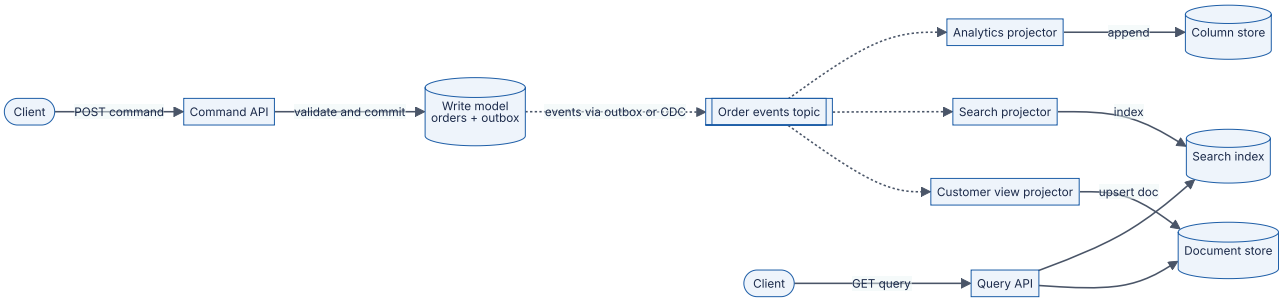
How it works internally

1. Commands go to the write model, which validates them and commits state changes (and, in an event-sourced system, events).
2. Each change is published as an event, through the event store's subscription, an outbox (§32.5) or CDC (§32.6).
3. A **projection** consumes events in order per aggregate and updates its read store with idempotent upserts, recording its checkpoint in the *same* transaction as the update, so a restart neither skips nor double-applies an event.
4. Queries read only from read stores. Each store can be a different technology.
5. To add or fix a view, deploy a new projection that starts from position 0 (or from a snapshot of the source data), let it catch up, then switch readers to it (**blue-green**

projection).

CQRS does not require event sourcing, and event sourcing does not strictly require CQRS, but they are usually paired: an event store cannot answer “orders over $100 last week”, so event-sourced systems almost always need projections.

Figure 32.5 — CQRS with three read models.



Reading the diagram. The command side (top left) is synchronous and strongly consistent within one database. The event topic decouples it from three projectors, each consuming at its own pace (dotted arrows) and writing to its own storage boundary. The query API reads only read models, so a query never touches the write database. Failure points: a projector that falls behind serves stale data; a projector bug corrupts only its own view, which can be rebuilt from the topic. Scaling: read models are replicated and scaled independently, and projectors scale by topic partition (§31.5).

32.4.2 Eventual consistency of reads

Every read model lags the write model by the **projection lag**: the time from commit to visibility in the view. In steady state it is the sum of relay or CDC delay, broker latency and projector processing, typically tens to hundreds of milliseconds. After an outage or a deploy pause it can be minutes.

The classic symptom is **read-your-writes** failure: a customer places an order, is redirected to “My orders”, and the list does not show it. Remedies, cheapest first:

- Return the new state in the command response and render it on the client.
- Return the write’s **position** (log offset or version) and have the query API wait until the projection’s checkpoint passes it, with a timeout (a *consistency token*).
- Route that user’s reads to the write model for a few seconds after a write.
- Show the item as “processing” in the UI, which is honest and often sufficient.

FORMULA

Catch-up time of a lagging projection

$$T_{\text{catch}} = \frac{\lambda D}{\mu - \lambda} \qquad (\mu > \lambda)$$

Symbol	Meaning	Units
λ	Event arrival rate on the source topic	events/s
μ	Maximum processing rate of the projection	events/s
D	Duration of the pause or outage	s
λD	Backlog accumulated during the pause	events
T_{catch}	Time after resuming until lag returns to steady state	s

WORKED EXAMPLE

Worked example — A projector paused by a bad deploy.

1. The order topic carries $\lambda = 2,000$ events/s. A projector with capacity $\mu = 5,000$ events/s is stopped for $D = 10$ min $= 600$ s.
2. Backlog: $2,000 \times 600 = 1,200,000$ events.
3. It drains at the spare rate $\mu - \lambda = 3,000$ events/s: $T_{\text{catch}} = 1,200,000/3,000 = 400$ s, almost 7 minutes, so total staleness peaks at about 17 minutes.
4. If peak traffic reaches $\lambda = 4,500$ events/s, the spare rate is 500 events/s and the same backlog takes 40 minutes.

Interpretation: catch-up time explodes as λ approaches μ, the same $1/(1-\rho)$ shape as queueing delay (§4.10). Engineering implication: provision projectors at two to three times peak rate, alert on lag measured in *time* (age of the oldest unapplied event), not only in events, and check that topic retention far exceeds the longest outage you plan to survive.

32.4.3 When not to use CQRS

Fowler's own warning is that CQRS adds risky complexity and suits only specific parts of a system [Fowler 2011b].

Table 32.3 — When CQRS pays off.

Use CQRS when	Avoid it when
Read shapes differ sharply from the write model (search, feeds, analytics)	Reads are CRUD views of the same rows
Read and write loads differ by orders of magnitude	One database with a few indexes and replicas meets the load (§14.4)
You already publish events (event sourcing, outbox)	Users must always see their writes immediately and you cannot add consistency tokens
Several teams need different views of the same facts	The team has no experience operating asynchronous pipelines

COMMON MISTAKE

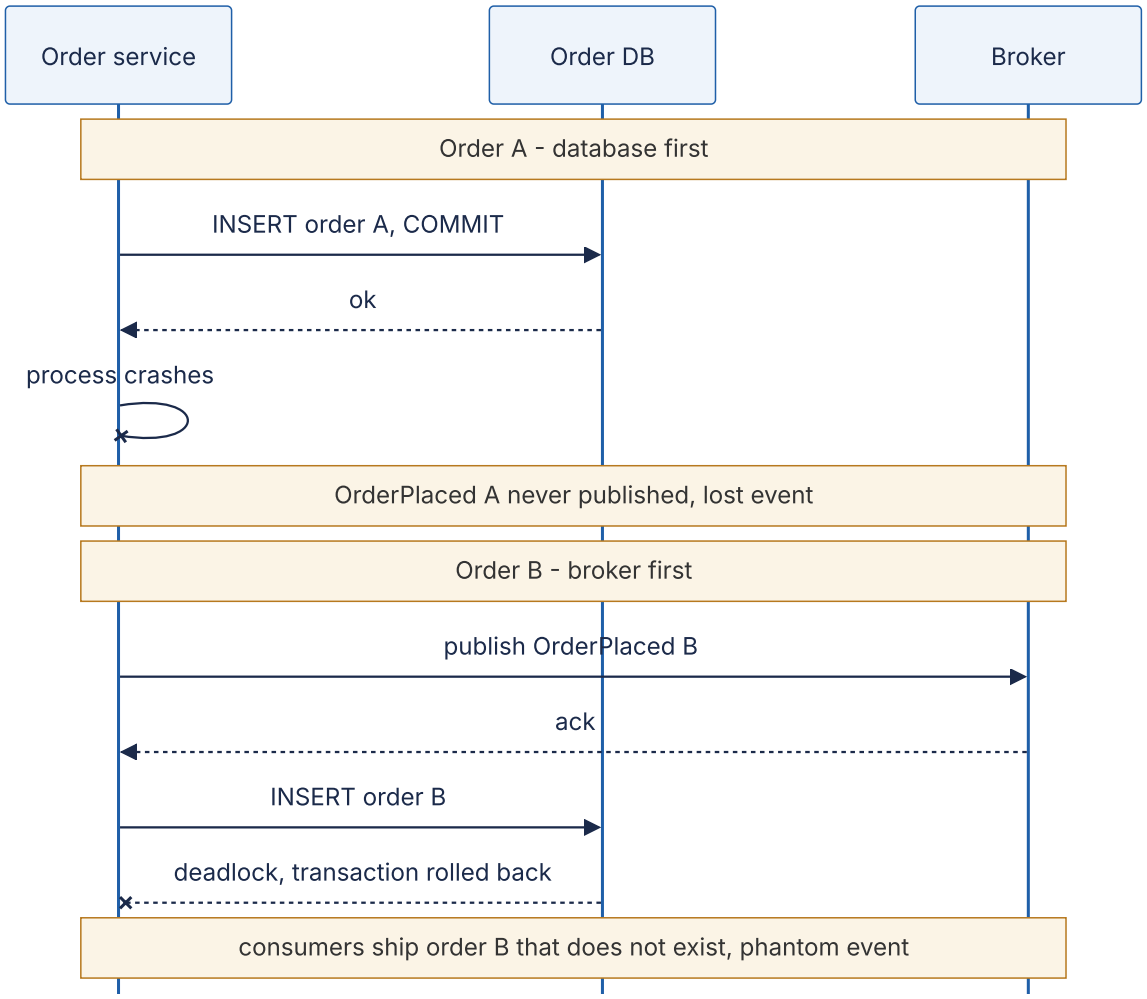
Common misconception — "CQRS means two databases." The essence is two *models*. A materialized view or a read replica with denormalized tables inside the same database is a legitimate, much simpler CQRS. Separate stores are a later step, taken when the view needs a different technology.

32.5 Transactional outbox

32.5.1 The dual-write problem

A service that changes its database and must tell other services faces two independent systems, the database and the broker, with no shared transaction. Writing to both from application code is a **dual write**, and every ordering of the two writes has a failure window.

Figure 32.6 — Two ways a dual write goes wrong.



Reading the diagram. Both scenarios use synchronous calls to two storage boundaries. In the first, the database commits and the process dies before publishing: the order exists but no consumer ever hears of it, so it is never shipped. In the second, the broker accepts the event and the database then rolls back: consumers act on a **phantom event**. Retrying does not close either window, because the crash can happen during the retry too; even a successful publish can time out and be retried, producing a duplicate. Scaling makes it worse: at 1,000 orders/s, a failure window hit once per million writes occurs about 86 times a day.

Distributed transactions (XA, §28.4.6) could commit both, but most brokers do not support XA, and 2PC couples the service's availability to the broker's. The outbox removes the second write instead.

32.5.2 The outbox table and the relay

DEFINITION

Definition — Transactional outbox. A pattern in which a service writes the events it wants to publish into an **outbox table** in its own database, in the same local transaction as the state change, and a separate **message relay** later reads committed outbox rows and publishes them to the broker, retrying until the broker acknowledges [Richardson 2018].

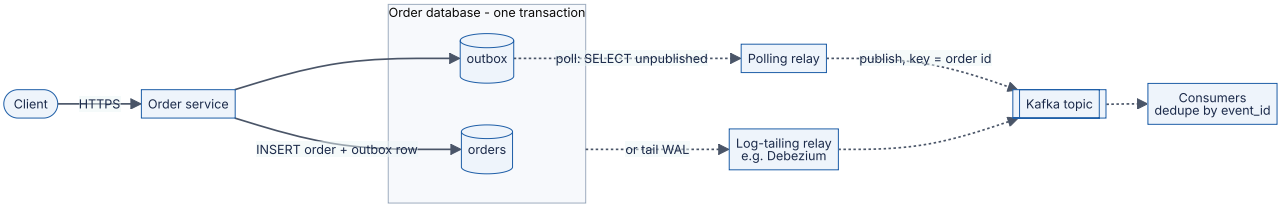
The local transaction makes “state changed” and “event recorded” atomic: both commit or neither does. The relay then turns “recorded” into “published” with at-least-once delivery. A crash anywhere loses nothing (the row is durable) and invents nothing (rolled-back transactions leave no row). The remaining imperfection is duplicates, when the relay publishes and crashes before marking the row, which idempotent consumers absorb (§32.10).

```
        CREATE TABLE outbox (
          id            BIGSERIAL PRIMARY KEY,         -- relay order
          event_id      UUID NOT NULL UNIQUE,          -- consumers deduplicate on this
          aggregate_id  TEXT NOT NULL,                 -- becomes the Kafka message key
          event_type    TEXT NOT NULL,
          payload       JSONB NOT NULL,
          created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
          published_at  TIMESTAMPTZ                    -- NULL until the relay publishes
        );
        CREATE INDEX outbox_unpublished ON outbox (id) WHERE published_at IS NULL;

        BEGIN;
        INSERT INTO orders (id, customer_id, status, total_cents) VALUES ('o-981', 'c-
42', 'PLACED', 4599);
        INSERT INTO outbox (event_id, aggregate_id, event_type, payload)
          VALUES (gen_random_uuid(), 'o-981', 'OrderPlaced', '{"order_id": "o-981",
"total_cents": 4599}');
        COMMIT;
```

What to notice: the partial index keeps the relay’s scan cheap even when the table holds millions of published rows; `aggregate_id` becomes the message key so all events of one order land in one partition (§31.5); `gen_random_uuid()` is built into PostgreSQL 13 and later. Delete published rows in batches, or drop time-based partitions, to keep the table small.

Figure 32.7 — Outbox with a relay, polling or log-tailing.



Reading the diagram. The client waits only for the single local transaction inside the database boundary. A relay, either a polling process or a log-tailing CDC connector (choose one), copies committed rows to Kafka asynchronously, keyed by order ID. Failure points: the relay can crash after publishing and before recording progress, so the broker can receive duplicates; consumers therefore deduplicate by `event_id`. If the relay is down, events wait safely in the table and their latency grows; nothing is lost. Scaling: one relay per database (or per outbox partition) with leader election so that two relays do not race (§26.5, §26.11).

32.5.3 Polling versus log-tailing relays

Table 32.4 — Polling versus log-tailing relays.

	Polling publisher	Log-tailing (CDC) relay
How it works	`SELECT ... WHERE published_at IS NULL ORDER BY id LIMIT n`, publish, then `UPDATE`	Reads committed outbox inserts from the WAL or binlog (§32.6)
Latency	Poll interval / 2 plus cycle time when idle	Tens of milliseconds
Database load	A query every interval plus an update per batch	Reads the log; no queries on the table
Ordering	By `id`; but sequence values are assigned at insert, not commit, so a concurrent slower transaction can commit a lower `id` after the relay has passed it	Commit order, exactly as the database applied it
Operational complexity	Low: a small process	Higher: replication slots, connectors, CDC monitoring
Choose when	Moderate volume, simplicity first	High volume, strict ordering, many tables

The polling ordering trap deserves emphasis. If transaction T1 takes `id = 100` and T2 takes `id = 101`, but T2 commits first, a relay that remembers “last published id = 101” will never see 100. The schema above avoids that by selecting on `published_at IS NULL` instead of a high-water mark; per-aggregate order still holds when each aggregate’s writes are serialized (row lock or version check), because its later event cannot commit before its earlier one. Log tailing avoids the issue entirely.

FORMULA

Polling relay throughput and added latency

$$X = \frac{B}{t_q + B\,t_p + t_m} \qquad L_{\text{relay}} \approx \frac{I}{2} + t_q + B\,t_p + t_m$$

Symbol	Meaning	Units
B	Batch size: rows fetched per cycle	events
t_q	Time to query the batch	s
t_p	Amortized publish time per event (pipelined, batched sends)	s/event
t_m	Time to mark the batch published	s
I	Polling interval when the outbox is empty	s
X	Maximum relay throughput	events/s
L_{relay}	Typical extra delivery latency at low load	s

WORKED EXAMPLE

Worked example — Sizing one relay for 5,000 events/s.

1. Batched, pipelined sends: $B = 500$, $t_q = 4$ ms, $t_p = 0.04$ ms ($500 \times 0.04 = 20$ ms per batch), $t_m = 6$ ms. Cycle $= 4 + 20 + 6 = 30$ ms; $X = 500/0.030 \approx 16,700$ events/s, over three times the required rate.
2. Naive relay that sends each event synchronously and waits for `acks=all` at $t_p = 2$ ms: cycle $= 4 + 1,000 + 6 = 1,010$ ms; $X \approx 495$ events/s, ten times too slow.
3. Latency at low load with $I = 100$ ms: $L \approx 50 + 30 = 80$ ms.

Interpretation: batching dominates relay throughput, and the poll interval dominates its latency. Engineering implication: send asynchronously with the producer's batching and wait for all acknowledgments before marking the batch; if 80 ms of extra latency is too much, poll more often (more load) or switch to log tailing.

32.5.4 Code example: dual writes versus the outbox under crashes

The simulation injects crashes, with probability 5 % at each marked point, into the order service, the broker, the relay and the consumer. Durable objects (database, broker log, the consumer's dedup table and offset) survive; a crash only aborts the current call.

```
        """Dual writes versus the transactional outbox, under randomly injected crashes.

        The database and the broker log are durable; process memory is not. A Crash
        can strike the order service, the relay or the consumer at any marked point.
        Run it directly with python3 to execute the tests.
        """
        import random
        import unittest


        class Crash(Exception):
            """The current process died at an injected crash point."""


        class Chaos:
            def __init__(self, seed, p):
                self.rng, self.p = random.Random(seed), p

            def point(self, name):
                if self.rng.random() < self.p:
                    raise Crash(name)


        class Database:
            """Durable tables. transaction() applies all of its writes or none."""

            def __init__(self):
                self.orders, self.outbox, self.next_id = {}, [], 1

            def transaction(self, order_id, status, event=None):
                if self.orders.get(order_id) == status:
                    return                                  # natural idempotency: al-
ready done
                self.orders[order_id] = status
                if event is not None:                       # same transaction as the
state change
                    self.outbox.append(dict(event, id=self.next_id, published=False))
                    self.next_id += 1


        class Broker:
            """An append-only log. A crash may strike after the append but before the
ack."""

            def __init__(self, chaos):
                self.log, self.chaos = [], chaos

            def publish(self, event):
                self.chaos.point("before append")
                self.log.append({k: event[k] for k in ("event_id", "order_id", "type")})
                self.chaos.point("after append, before ack")


        def event_for(order_id, status):
            kind = {"PLACED": "OrderPlaced", "PAID": "OrderPaid"}[status]
            return {"event_id": f"{order_id}:{kind}", "order_id": order_id, "type":
```

```
kind}


        # ---------- Version 1: dual write (two systems, no atomicity between them) ----
------
        def dual_write(db, broker, chaos, order_id, status, db_first=True):
            event = event_for(order_id, status)
            if db_first:
                db.transaction(order_id, status)
                chaos.point("between the two writes")       # crash here: lost event
                broker.publish(event)
            else:
                broker.publish(event)
                chaos.point("between the two writes")       # crash here: phantom event
                db.transaction(order_id, status)


        # ---------- Version 2: transactional outbox plus a relay ----------
        def write_with_outbox(db, chaos, order_id, status):
            chaos.point("before commit")
            db.transaction(order_id, status, event_for(order_id, status))


        def relay_once(db, broker, chaos, batch=5):
            """Polling relay: publish unpublished rows in id order, then mark them."""
            rows = [r for r in db.outbox if not r["published"]][:batch]
            for row in rows:
                broker.publish(row)
                chaos.point("after publish, before mark")   # crash here: duplicate
later
                row["published"] = True


        class Consumer:
            """Idempotent consumer: the dedup record and the effect commit together."""

            def __init__(self, chaos):
                self.offset, self.seen, self.applied, self.chaos = 0, set(), [], chaos

            def poll(self, broker):
                while self.offset < len(broker.log):
                    msg = broker.log[self.offset]
                    if msg["event_id"] not in self.seen:    # one local transaction:
                        self.seen.add(msg["event_id"])      #   dedup row + business ef-
fect
                        self.applied.append((msg["order_id"], msg["type"]))
                    self.chaos.point("after effect, before offset commit")
                    self.offset += 1


        def survive(fn, *args):
            try:
                fn(*args)
            except Crash:
                pass                                        # the process restarts; mem-
ory is gone
```

```
        def run_outbox(seed, orders=200, p=0.05):
            chaos = Chaos(seed, p)
            db, broker, consumer = Database(), Broker(chaos), Consumer(chaos)
            for oid in range(orders):
                survive(write_with_outbox, db, chaos, oid, "PLACED")
                if oid in db.orders:
                    survive(write_with_outbox, db, chaos, oid, "PAID")
                if chaos.rng.random() < 0.3:
                    survive(relay_once, db, broker, chaos)
                    survive(consumer.poll, broker)
            while any(not r["published"] for r in db.outbox) or consumer.offset <
len(broker.log):
                survive(relay_once, db, broker, chaos)      # eventually every row is
published
                survive(consumer.poll, broker)
            return db, broker, consumer


        def run_dual_write(seed, db_first, orders=200, p=0.05):
            chaos = Chaos(seed, p)
            db, broker = Database(), Broker(chaos)
            for oid in range(orders):
                survive(dual_write, db, broker, chaos, oid, "PLACED", db_first)
            published = {m["order_id"] for m in broker.log}
            lost = set(db.orders) - published
            phantom = published - set(db.orders)
            return lost, phantom


        class Tests(unittest.TestCase):
            def test_dual_write_db_first_loses_events(self):
                lost, _ = run_dual_write(seed=1, db_first=True)
                self.assertGreater(len(lost), 0)

            def test_dual_write_broker_first_creates_phantoms(self):
                _, phantom = run_dual_write(seed=1, db_first=False)
                self.assertGreater(len(phantom), 0)

            def test_outbox_no_lost_or_phantom_events(self):
                for seed in range(50):
                    db, broker, consumer = run_outbox(seed)
                    expected = {(oid, event_for(oid, s)["type"])
                                for oid, final in db.orders.items()
                                for s in (("PLACED", "PAID") if final == "PAID" else
("PLACED",))}
                    self.assertEqual(set(consumer.applied), expected)          # nothing
lost or invented
                    self.assertEqual(len(consumer.applied), len(expected))     # each
effect exactly once

            def test_outbox_duplicates_reach_broker_but_not_effects(self):
                db, broker, consumer = run_outbox(seed=3)
                self.assertGreater(len(broker.log), len(consumer.applied))     # at-
least-once delivery

            def test_per_order_event_order_is_preserved(self):
```

```
                for seed in range(50):
                    _, _, consumer = run_outbox(seed)
                    first = {}
                    for i, (oid, kind) in enumerate(consumer.applied):
                        first.setdefault((oid, kind), i)
                    for (oid, kind), i in first.items():
                        if kind == "OrderPaid":
                            self.assertLess(first[(oid, "OrderPlaced")], i)


        if __name__ == "__main__":
            unittest.main(verbosity=1)
```

Output (Python 3.13; the timing varies):

```
.....
----------------------------------------------------------------------
Ran 5 tests in 0.332s

OK
```

What to notice:

- With seed 1, each dual-write variant fails about 22 times in 200 orders: database-first loses events, broker-first invents them. No retry inside `dual_write` could fix this, because the crash ends the process that would retry.
- In `run_outbox`, across 50 seeds and about 2,000 crash opportunities each, the set of effects equals the set of committed state changes. Seed 3 puts 428 messages on the broker for 385 effects: duplicates are real and harmless.
- `Database.transaction` is naturally idempotent (setting a status that is already set does nothing), so a client that retries after an ambiguous failure does not create a second order or a second outbox row (§22.15).
- The consumer's `seen` set, `applied` list and `offset` model durable state in the consumer's own database. Because the dedup record commits with the effect, the crash between effect and offset commit only causes a redelivery that is skipped.
- Per-order ordering holds because the relay always republishes from the oldest unpublished row; a relay that skipped a failed row and continued would break it.

Table 32.5 — Outbox failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Duplicate events	Relay crashed between publish and mark	Consumer dedup hit rate	None needed if consumers are idempotent	Unique `event_id`; idempotent consumers (§32.10)
Growing delivery latency	Relay down or slower than the write rate	Age of oldest unpublished row	Restart or scale the relay	Alert on outbox age; *X* sized above peak
Skipped events	High-water-mark polling past an uncommitted lower `id`	Gaps in per-aggregate sequence numbers	Republish from the gap	Select on `published_at IS NULL`, or tail the log
Two relays publish out of order	No single-writer guarantee	Out-of-order sequence numbers at consumers	Consumers reorder or reject by version	Leader election with fencing
Outbox table bloat	Published rows never deleted	Table size, slow relay queries	Batch delete, vacuum	Time-partitioned outbox with partition drops

32.6 Change data capture

32.6.1 Reading the database's own log

DEFINITION

Definition — Change data capture (CDC). Extracting every committed insert, update and delete from a database, in commit order, as a stream of change events that other systems consume. **Log-based CDC** reads the database's replication log (PostgreSQL's write-ahead log via logical decoding, MySQL's binary log) rather than querying tables.

First-principles explanation

A database already keeps a diary of every change, because it needs one for crash recovery and replication (§13.5, §14.4). CDC asks the database for a copy of that diary, decoded into rows. Since the diary contains only committed changes in the order they were applied, anything built from it agrees with the database by construction.

Why does this exist?

Search indexes, caches, data warehouses and other services all need to know when data changes. The alternatives are worse: dual writes from application code (§32.5.1); periodic queries on an `updated_at` column, which miss hard deletes, miss intermediate states and load the database; or database triggers, which add latency to every transaction. Log-based CDC has none of these problems, and it captures changes made by *any* writer, in-

cluding batch jobs and manual fixes that bypass the application [Kleppmann 2017, ch. 11]. LinkedIn's Databus, an early large-scale CDC system, was built for exactly this purpose: keeping search indexes, caches and replicas consistent with the primary databases [Das et al. 2012].

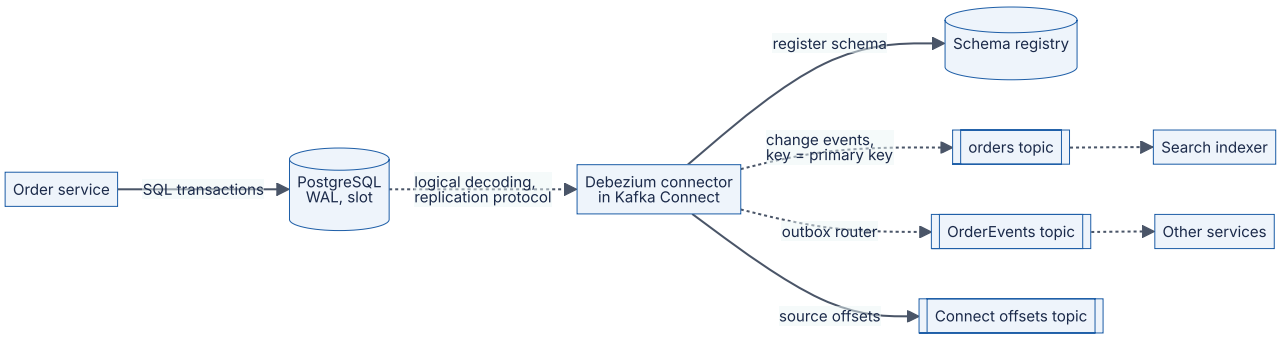
How it works internally

1. **Enable a logical log.** PostgreSQL needs `wal_level = logical` and a **replication slot**, which records how far the consumer has read and makes the server retain WAL until then. MySQL needs the binary log in `ROW` format, so each event carries row images rather than SQL statements.
2. **Initial snapshot.** The connector reads existing table contents in a consistent snapshot and notes the log position at which the snapshot was taken.
3. **Streaming.** It then reads the log from that position onward, decoding each change into an event with the row *before* and *after* the change, the operation (`c`reate, `u`pdate, `d`elete, or `r`ead for snapshot rows) and source metadata such as the log position and commit timestamp.
4. **Publishing.** Events go to a topic per table, keyed by primary key, so all changes to one row stay ordered in one partition. A delete is followed by a **tombstone** (null value) so that log compaction can eventually remove the key (§31.10).
5. **Progress.** The connector periodically records the last log position it published. After a crash it resumes from there, so events after the last recorded position are published again: delivery is at-least-once.

32.6.2 Debezium architecture

Debezium is an open-source CDC platform built as a set of Kafka Connect source connectors (§31.14), one per database type; it can also run standalone as Debezium Server for non-Kafka destinations [Debezium Docs].

Figure 32.8 — Debezium capturing PostgreSQL changes into Kafka.



Reading the diagram. The application writes only to PostgreSQL. The connector streams decoded WAL records over PostgreSQL's replication protocol (asynchronous), registers each record schema with a schema registry, and publishes change events keyed

by primary key. Rows of the outbox table can be routed by Debezium's outbox event router to per-aggregate topics, which is how §32.5's log-tailing relay is usually built. Progress (the last WAL position) is stored in Kafka Connect's offsets topic. Failure points: if the connector is down, the replication slot forces PostgreSQL to keep WAL, which can fill the disk; after a restart, events since the last stored offset are re-sent. Scaling: one connector task per database (the log is a single ordered stream), with throughput limited by WAL decoding, and many independent consumers downstream.

Snapshot plus stream, without stopping the world. A naive initial snapshot of a large table takes hours and holds a long transaction open. Netflix's DBLog showed how to interleave chunked table reads with log streaming using low and high **watermarks** written to the log, so that a chunk's rows are reconciled with concurrent changes and snapshots can be paused, resumed or re-run for one table [Andreakis and Papapanagiotou 2020]. Debezium's **incremental snapshots**, triggered through a signaling table, use the same idea [Debezium Docs].

> WARNING / TRADE-OFF
>
> **Warning — An abandoned replication slot fills the disk.** A PostgreSQL slot whose consumer has stopped retains WAL indefinitely. Monitor slot lag in bytes (`pg_replication_slots`), alert well before disk exhaustion, and on PostgreSQL 13 and later set `max_slot_wal_keep_size` to cap retention, accepting that the connector then needs a fresh snapshot.

32.6.3 Schema registry and evolution

A CDC topic exposes your table schema to every consumer, which turns a column rename into a cross-team breaking change. A **schema registry** stores each version of a topic's schema (Avro, Protobuf or JSON Schema); producers register a schema and embed its ID in each message, and consumers fetch the schema by ID to decode. The registry enforces a **compatibility mode** at registration: *backward* (new readers can read old data; Confluent Schema Registry's default), *forward* (old readers can read new data), *full* (both), with *transitive* variants that check against all earlier versions rather than only the previous one [Confluent Docs: Schema Registry]. For event streams that are replayed from the beginning, as in §32.3.4, use a transitive mode.

RECOMMENDATION

Recommendation — Do not publish your tables as your API. Raw CDC events couple consumers to internal column names and normalization. For integration between teams, publish deliberate domain events through an outbox (routed by CDC if you like) and keep raw table CDC for derived data you own: caches, search indexes, the warehouse.

INDUSTRY EXAMPLE

Netflix DBLog. Netflix needed to keep many derived stores, such as search indexes, in sync with MySQL and PostgreSQL sources, including full re-syncs without pausing writes. DBLog streams the transaction log and interleaves chunked selects bounded by watermarks, so dumps can be triggered at any time per table. Trade-off: the connector must write watermark rows into the source database. *Evidence: Primary* — [Andreakis and Papapanagiotou 2020]

Table 32.6 — Change data capture failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Source disk fills	Stalled slot retains WAL	Slot lag bytes	Restart connector or drop slot and re-snapshot	`max_slot_wal_keep_size`, alerts
Consumers break on deploy	Column renamed or retyped	Deserialization errors	Roll back schema; upcast	Registry compatibility checks in CI
Missing intermediate states	Query-based polling instead of log	Audits	Switch to log-based CDC	Log-based CDC
Duplicates after restart	Offsets stored periodically	Dedup counters	None if idempotent	Idempotent sinks keyed by primary key and log position

32.7 The saga pattern

32.7.1 Long-lived transactions without distributed locks

Section 28.6 previewed the saga. This section owns the deep treatment.

DEFINITION

Definition — Saga. A long-lived transaction (LLT) written as a sequence of local transactions $T_1, \ldots, T_n$, each of which commits independently and releases its resources, where each T_i (except possibly the last) has a **compensating transaction** C_i. The system guarantees that either $T_1, T_2, \ldots, T_n$ executes, or $T_1, \ldots, T_j, C_j, \ldots, C_1$ executes for some $j < n$ [Garcia-Molina and Salem 1987].

First-principles explanation

Booking a trip with a hotel, a flight and a car is a saga. You book the hotel, then the flight; if no car is available, you cancel the flight and the hotel. Nobody locks the hotel's whole reservation system while you shop for cars, and nobody expects the cancellation to erase the booking from history: you get a cancellation record and perhaps a fee. That is **semantic atomicity**: the end state is "all done" or "equivalently undone" by the business's definition.

Why does this exist?

Garcia-Molina and Salem observed in 1987 that long-lived transactions, which hold locks for minutes or hours, delay every other transaction that touches the same data and are more likely to abort. Their proposal was to break an LLT into ordinary transactions that may interleave with others, and to undo committed work with application-defined compensations rather than rollbacks [Garcia-Molina and Salem 1987]. Microservices recreate the same problem at a larger scale: each service owns its database, external APIs (payment providers) cannot join a 2PC, and a coordinator that blocks while holding locks across services (§28.4.3) couples their availability. Helland argued that at scale, applications must live without distributed transactions and coordinate entities through messages, tentative operations and confirmations or cancellations [Helland 2007]; the saga is the standard shape for that.

How it works internally

Richardson classifies saga steps into three kinds, which determine how failures are handled [Richardson 2018]:

1. **Compensatable transactions** come before the point of no return and have a compensation (create order → reject order; reserve stock → release stock).
2. The **pivot transaction** is the go/no-go step. If it commits, the saga will complete; if it fails, the saga compensates. It is often the step that cannot be undone or is the most likely to fail, such as charging a card.
3. **Retriable transactions** follow the pivot and are guaranteed to succeed eventually if retried (approve order, notify shipping). They never compensate; they retry forward.

Compensations must themselves be idempotent and retriable until they succeed, because the saga has no other way out. Garcia-Molina and Salem call undoing **backward recovery**; with save points, a saga can also use **forward recovery**, retrying from the last save point instead of compensating.

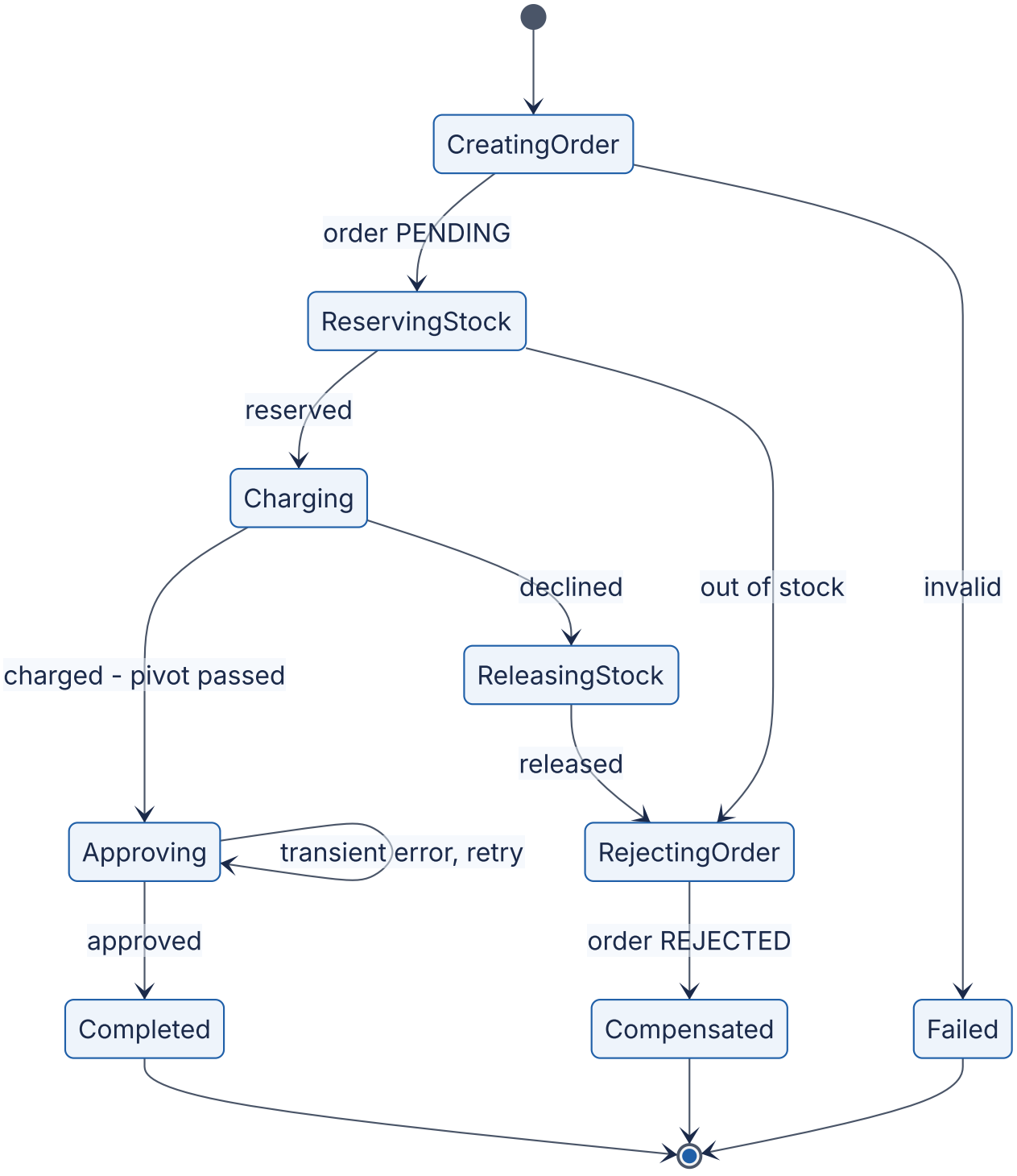
32.7.2 Worked example: order, inventory and payment

The shop's checkout saga has four steps. Table 32.7 lists each step and what happens if it fails.

Table 32.7 — The checkout saga and a failure at each step.

Step	Service	Action	Kind	Compensation	If this step fails
T_1	Order	Create order in `PENDING`	Compensatable	C_1: set `REJECTED`	Validation error to client; nothing to undo
T_2	Inventory	Reserve 2 units	Compensatable	C_2: release reservation	Out of stock: run C_1
T_3	Payment	Charge card $300	Pivot	(refund only by a new business process)	Declined: run C_2, then C_1
T_4	Order	Set `APPROVED`, emit `OrderApproved`	Retriable	—	Retry until it succeeds; never compensate
—	Orchestrator	Crash between any two steps	—	—	Resume from the saga log; steps are idempotent
—	Any	Compensation C_i fails transiently	—	—	Retry C_i; after a limit, alert a human

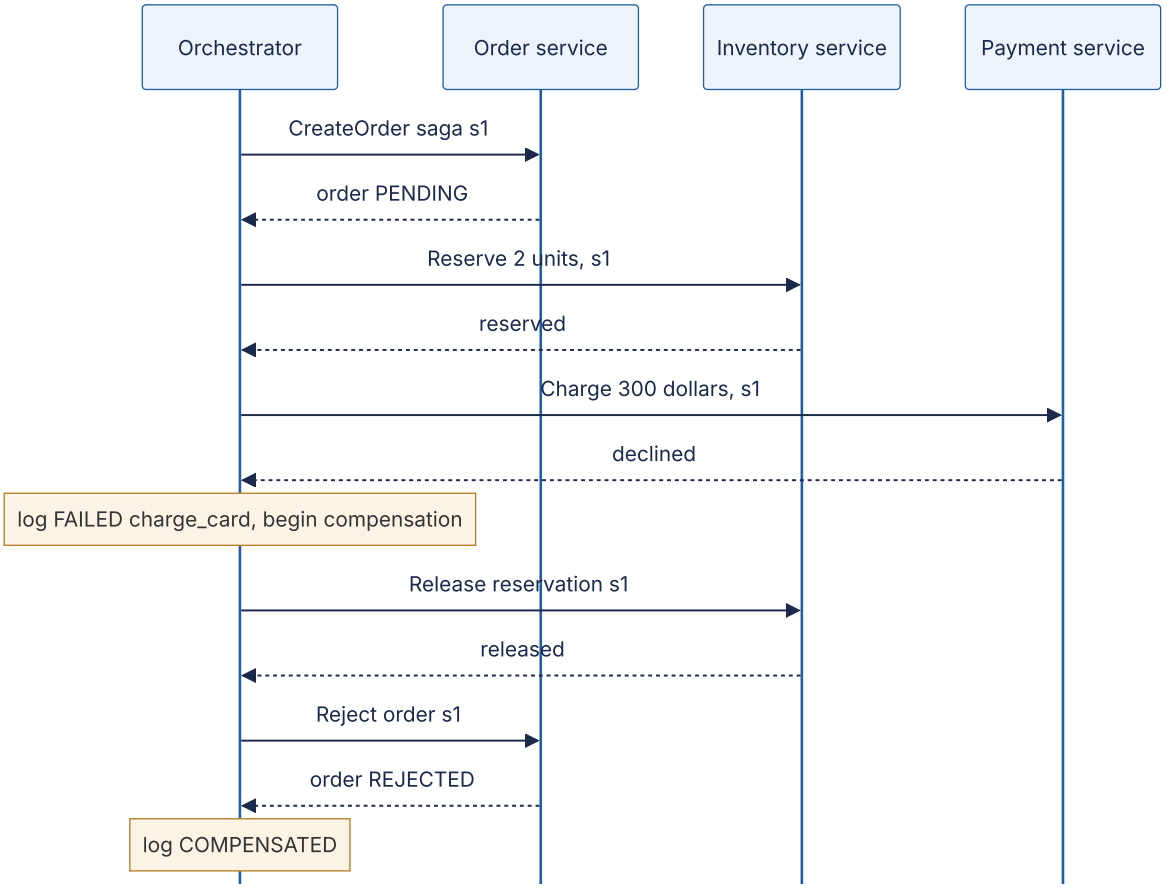
Figure 32.9 — State machine of the checkout saga orchestrator.



Reading the diagram. Each state is persisted in the orchestrator's saga log (a table in its own database) before the next command is sent, so a restarted orchestrator re-enters the same state and re-sends the same command; the receiving services deduplicate by saga ID. Transitions are triggered by asynchronous replies from the order, inventory and payment services. The left branch (before `Charging` succeeds) is backward recovery through compensations in reverse order; after the pivot the only exit is forward, with retries. Failure points: a compensation that keeps failing leaves the saga stuck in

`ReleasingStock` or `RejectingOrder`, which must raise an alert. Scaling: sagas are independent, so partition the saga log by saga ID across orchestrator instances.

Figure 32.10 — The card is declined: compensation in reverse order.



Reading the diagram. Each arrow is a command or a reply, usually sent as messages through a broker with the request–reply pattern, so the whole exchange is asynchronous even though it reads like calls. Each service runs one local transaction per command and stores the saga ID to make retries idempotent. The orchestrator writes a log record after every reply. Between “reserved” and “released”, two units are invisible to other shoppers, an intermediate state that a saga cannot hide (§32.7.3). If the orchestrator crashes after logging the failure, it restarts compensation from the log; if the inventory service is down, the release is retried until it succeeds.

32.7.3 Living without isolation

A saga is ACD without I: atomic in the semantic sense, consistent within each service, durable, but not isolated. Other transactions see its intermediate states, which causes three anomalies: **lost updates** (another saga overwrites a change, and a compensation later overwrites theirs), **dirty reads** (a reader sees stock reserved by a saga that will be compensated) and **fuzzy reads** (two steps of one saga read different versions of the same

data). Richardson collects countermeasures, drawing on Frank and Zahle's work on semantic ACID properties in multidatabases [Richardson 2018; Frank and Zahle 1998].

Table 32.8 — Countermeasures for the lack of isolation.

Countermeasure	How it works	Checkout example
Semantic lock	A compensatable step sets an application-level flag (`*_PENDING`) that other requests check	Canceling an order in `PENDING` is refused or queued until the saga ends
Commutative updates	Design operations so their order does not matter	`reserve(+2)` and `release(-2)` as deltas, not "set stock = 8"
Pessimistic view	Reorder steps so that risky reads happen in retriable steps	Increase a loyalty balance only after the pivot
Reread value	Reread data before overwriting and abort if it changed (optimistic locking)	Approve only if order version is unchanged
Version file	Record operations so out-of-order arrivals can be reordered	Payment service tolerates `Cancel` arriving before `Authorize`
By value	Choose the mechanism by business risk	High-value orders use a 2PC-backed path or manual review

KEY CONCEPT

Key insight — Reservations are the saga-friendly form of locking. "Reserve 2 units for 15 minutes" is a semantic lock with a timeout, held in business data rather than in a database lock manager. It survives crashes, is visible to other users ("2 left, 1 in someone's cart"), and expires on its own if a saga dies. Helland's tentative operations followed by confirmation or cancellation are the same idea [Helland 2007].

32.7.4 Code example: an orchestrated saga with compensation

```
        """An orchestrated order saga: order -> inventory -> payment, with compensation.

        Each service owns its data and exposes idempotent operations keyed by saga ID.
        The orchestrator persists its progress in a durable saga log, so a crashed
        orchestrator resumes where it stopped. Run with python3 to execute the tests.
        """
        import unittest


        class Crash(Exception):
            """The orchestrator process died."""


        class Rejected(Exception):
            """A business failure: the step cannot succeed, so the saga must compen-
sate."""


        class Unavailable(Exception):
            """A transient failure: retry the same step later."""


        class Orders:
            def __init__(self):
                self.status = {}

            def create(self, saga):                          # T1, compensatable
                self.status.setdefault(saga, "PENDING")      # PENDING is the semantic
lock

            def reject(self, saga):                          # C1
                self.status[saga] = "REJECTED"

            def approve(self, saga):                         # T4, retriable
                self.status[saga] = "APPROVED"


        class Inventory:
            def __init__(self, stock):
                self.stock, self.reserved = stock, {}

            def reserve(self, saga, qty):                    # T2, compensatable
                if saga in self.reserved:
                    return                                   # duplicate request: already
done
                if qty > self.stock:
                    raise Rejected("out of stock")
                self.stock -= qty
                self.reserved[saga] = qty

            def release(self, saga, qty):                    # C2
                self.stock += self.reserved.pop(saga, 0)     # no-op if never reserved or
released
```

```
        class Payments:
            def __init__(self, limit):
                self.limit, self.charges = limit, {}

            def charge(self, saga, cents):                  # T3, the pivot transaction
                if saga in self.charges:
                    return
                if cents > self.limit:
                    raise Rejected("card declined")
                self.charges[saga] = cents


        class Orchestrator:
            def __init__(self, orders, inventory, payments, log, crash_after=None,
flaky=0):
                self.o, self.i, self.p, self.log = orders, inventory, payments, log
                self.crash_after, self.flaky = crash_after, flaky   # fault injection
                self.steps = [   # (name, action, compensation); None = nothing to undo
                    ("create_order", lambda s, c: self.o.create(s), lambda s, c:
self.o.reject(s)),
                    ("reserve_stock", lambda s, c: self.i.reserve(s, c["qty"]),
                     lambda s, c: self.i.release(s, c["qty"])),
                    ("charge_card", lambda s, c: self.p.charge(s, c["cents"]), None),
# pivot
                    ("approve_order", self.approve, None),                          #
retriable
                ]
                self.pivot = 2

            def approve(self, saga, cmd):
                if self.flaky:
                    self.flaky -= 1
                    raise Unavailable("order service down")
                self.o.approve(saga)

            def record(self, saga, entry):
                self.log.setdefault(saga, []).append(entry)  # durable, written before
moving on
                if entry == self.crash_after:
                    raise Crash(entry)

            def run(self, saga, cmd):
                """Start or resume a saga. Safe to call again after a crash."""
                entries = self.log.get(saga, [])
                if "COMPLETED" in entries or "COMPENSATED" in entries:
                    return entries[-1]
                if any(e.startswith("FAILED") for e in entries):
                    return self.compensate(saga, cmd)
                done = sum(1 for e in entries if e.startswith("DONE"))
                for k in range(done, len(self.steps)):
                    name, action, _ = self.steps[k]
                    while True:
                        try:
                            action(saga, cmd)                # idempotent, so re-run-
ning is safe
                            break
```

```
                        except Unavailable:
                            if k < self.pivot:                # before the pivot: give
up and undo
                                self.record(saga, f"FAILED {name}")
                                return self.compensate(saga, cmd)
                            continue                          # after the pivot: retry
until it works
                        except Rejected:
                            self.record(saga, f"FAILED {name}")
                            return self.compensate(saga, cmd)
                    self.record(saga, f"DONE {name}")
                self.record(saga, "COMPLETED")
                return "COMPLETED"

            def compensate(self, saga, cmd):
                done = [e[5:] for e in self.log[saga] if e.startswith("DONE")]
                undone = {e[5:] for e in self.log[saga] if e.startswith("UNDO")}
                for name, _, comp in reversed(self.steps):
                    if name in done and name not in undone and comp:
                        comp(saga, cmd)                       # idempotent compensation
                        self.record(saga, f"UNDO {name}")
                self.record(saga, "COMPENSATED")
                return "COMPENSATED"


        def world(stock=10, limit=50_000):
            return Orders(), Inventory(stock), Payments(limit), {}


        class Tests(unittest.TestCase):
            CMD = {"qty": 2, "cents": 30_000}

            def test_happy_path(self):
                o, i, p, log = world()
                self.assertEqual(Orchestrator(o, i, p, log).run("s1", self.CMD),
"COMPLETED")
                self.assertEqual((o.status["s1"], i.stock, p.charges["s1"]),
("APPROVED", 8, 30_000))

            def test_out_of_stock_rejects_order(self):
                o, i, p, log = world(stock=1)
                self.assertEqual(Orchestrator(o, i, p, log).run("s1", self.CMD),
"COMPENSATED")
                self.assertEqual((o.status["s1"], i.stock, p.charges), ("REJECTED", 1,
{}))

            def test_card_declined_releases_stock_then_rejects(self):
                o, i, p, log = world(limit=10_000)
                self.assertEqual(Orchestrator(o, i, p, log).run("s1", self.CMD),
"COMPENSATED")
                self.assertEqual((o.status["s1"], i.stock, p.charges), ("REJECTED", 10,
{}))
                self.assertEqual(log["s1"][-3:], ["UNDO reserve_stock", "UNDO
create_order", "COMPENSATED"])

            def test_failure_after_pivot_is_retried_forward(self):
                o, i, p, log = world()
```

```
                self.assertEqual(Orchestrator(o, i, p, log, flaky=3).run("s1",
self.CMD), "COMPLETED")
                self.assertEqual((o.status["s1"], i.stock), ("APPROVED", 8))

            def test_crash_at_every_point_then_resume(self):
                points = ["DONE create_order", "DONE reserve_stock", "DONE charge_card",
                          "DONE approve_order", "COMPLETED"]
                for point in points:
                    o, i, p, log = world()
                    with self.assertRaises(Crash):
                        Orchestrator(o, i, p, log, crash_after=point).run("s1",
self.CMD)
                    self.assertEqual(Orchestrator(o, i, p, log).run("s1", self.CMD),
"COMPLETED")
                    self.assertEqual((i.stock, len(p.charges)), (8, 1), point)   # noth-
ing doubled

            def test_step_executed_but_not_logged_is_not_doubled(self):
                o, i, p, log = world()
                with self.assertRaises(Crash):
                    Orchestrator(o, i, p, log, crash_after="DONE
create_order").run("s1", self.CMD)
                i.reserve("s1", 2)        # the next step ran, but the crash hit before
its DONE record
                self.assertEqual(Orchestrator(o, i, p, log).run("s1", self.CMD),
"COMPLETED")
                self.assertEqual(i.stock, 8)

            def test_crash_during_compensation_then_resume(self):
                o, i, p, log = world(limit=10_000)
                with self.assertRaises(Crash):
                    Orchestrator(o, i, p, log, crash_after="UNDO
reserve_stock").run("s1", self.CMD)
                self.assertEqual(Orchestrator(o, i, p, log).run("s1", self.CMD),
"COMPENSATED")
                self.assertEqual((o.status["s1"], i.stock), ("REJECTED", 10))


        if __name__ == "__main__":
            unittest.main(verbosity=1)
```

Output (Python 3.13; the timing varies):

```
.......
----------------------------------------------------------------------
Ran 7 tests in 0.001s

OK
```

What to notice:

- `record` writes to the saga log *before* the orchestrator moves on, and `run` reconstructs its position from the log alone. That is what lets

`test_crash_at_every_point_then_resume` kill the orchestrator after every step and still finish with exactly one reservation and one charge.

- `test_step_executed_but_not_logged_is_not_doubled` covers the other crash window: the action ran but its `DONE` record did not. The re-sent `reserve` is a no-op because the inventory service deduplicates by saga ID.
- `Unavailable` after the pivot is retried forever; a `Rejected` before the pivot triggers compensation in reverse order. A real orchestrator would back off between retries (§22.12) and alert after a limit.
- `release` and `reject` tolerate being called twice or for work that never happened, so a crash during compensation is also recoverable (`test_crash_during_compensation_then_resume`).
- The order's `PENDING` status is the semantic lock of Table 32.8.

32.7.5 Failure scenarios and misconceptions

Table 32.9 — Saga failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Double charge or double reservation	Retried step not idempotent	Reconciliation between services	Refund, release	Idempotency by saga ID in every participant
Saga stuck in compensation	Compensation fails permanently (e.g. refund rejected)	Saga age by state	Human workflow	Design compensations that cannot fail for business reasons
Compensation runs before the action	Messages reordered; action arrives after cancel	Orphaned reservations	Timeout-based cleanup	Participants record "cancelled" tombstones for the saga ID
Oversold stock	Dirty read of uncommitted reservations	Negative stock alarms	Back-order process	Semantic locks, commutative updates
Lost saga	Orchestrator state only in memory	Orders stuck in `PENDING`	Sweep and resume	Persist the saga log before each command

COMMON MISTAKE

Common misconception — "Compensation is rollback." A rollback makes it as if nothing happened; a compensation is a new transaction that others can observe, which may itself have effects (a refund email, a fee). Some actions have no compensation at all (an email sent, a parcel dispatched); place them after the pivot.

COMMON MISTAKE

Common misconception — "A saga gives the same guarantees as a distributed transaction, just faster." It gives semantic atomicity and eventual consistency without isolation. If an invariant must never be visibly violated, even briefly, keep the data in one database (§28.11).

32.8 Choreography versus orchestration

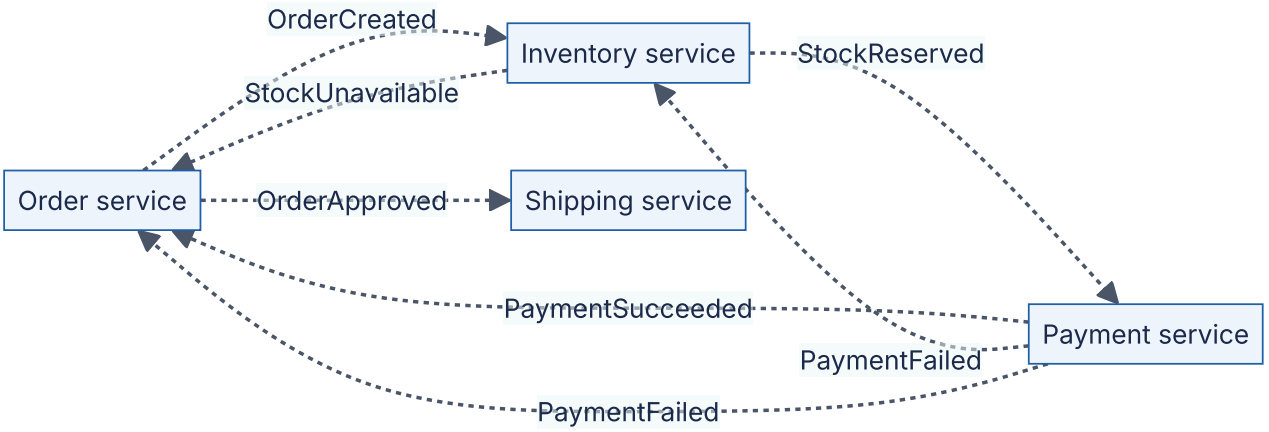
A saga needs something to decide which step runs next. There are two styles.

DEFINITION

Definition — Choreography and orchestration. In **choreography**, each service reacts to other services' events and publishes its own; the saga's control flow is distributed across the participants and exists nowhere as one piece of code. In **orchestration**, a dedicated **orchestrator** sends commands to participants, receives their replies and decides the next step; the control flow lives in one place [Richardson 2018].

The analogy is a dance troupe versus an orchestra. Dancers who know their cues need no conductor, which works beautifully for four dancers and becomes chaos for forty. An orchestra has a conductor who is a single point of control, and of failure, but anyone can read the score.

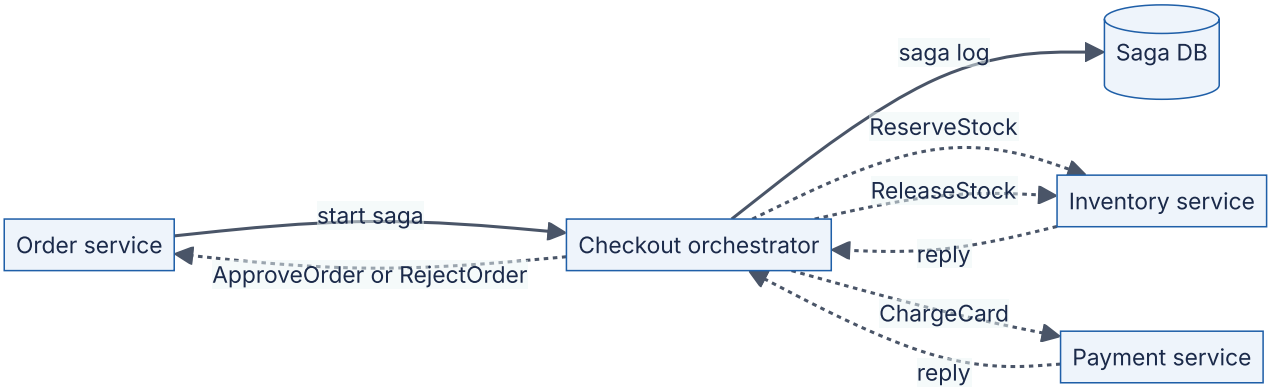
Figure 32.11 — The checkout saga by choreography.



Reading the diagram. Every arrow is an asynchronous event published to the broker (omitted for clarity) through each service's outbox; no service calls another. Each service subscribes to the events it cares about: inventory reacts to `OrderCreated` and to `PaymentFailed` (its compensation trigger), and the order service reacts to the final out-

come. Each service keeps its own saga state in its own database. Failure points: a lost subscription or a consumer bug silently stalls a saga, and nobody owns the end-to-end view; cyclic dependencies (order ↔ payment) make deploys and schema changes harder. Scaling is excellent, since there is no central component.

Figure 32.12 — The same saga by orchestration.



Reading the diagram. The orchestrator persists its state in its own database (synchronous, solid arrow) and exchanges asynchronous commands and replies with participants over request and reply channels. Participants know nothing about the saga; they expose commands such as `ReserveStock` and `ReleaseStock`. Failure points: the orchestrator is a central dependency, so it must be stateless apart from the saga log and run as several instances; a slow participant delays only the sagas waiting on it. Scaling: partition sagas by ID across orchestrator instances.

Table 32.10 — Choreography versus orchestration.

	Choreography	Orchestration
What it is	Services react to each other's events	A coordinator issues commands
Coupling	Each service knows which events to react to	Participants know nothing of the flow; orchestrator knows all
Visibility	Flow is implicit; needs tracing (§40.6) to reconstruct	Flow is explicit in one place; saga state is queryable
Adding a step	Change several services	Change the orchestrator
Failure handling	Compensation logic scattered across services	Centralized, testable
Single point of failure	None	The orchestrator, mitigated by replication
Risk	Cyclic dependencies, "event spaghetti"	Orchestrator accumulating business logic
Choose when	2–4 steps, stable flow, autonomous teams	Many steps, branching, timeouts, compliance audits

RECOMMENDATION

Recommendation — Choreograph between domains, orchestrate within a process. Broadcast facts (`OrderPlaced`) by choreography so other domains (analytics, loyalty, email) can subscribe without coordination. Model a business process with compensations and deadlines, such as checkout or loan approval, as an orchestrated saga, preferably on a workflow engine (§32.11). Keep business rules in participants; the orchestrator should sequence, not compute.

INDUSTRY EXAMPLE

Netflix Conductor. Netflix's content and media-processing pipelines grew into flows that were hard to follow when implemented as peer-to-peer events. Netflix built Conductor, a JSON-defined workflow orchestrator with a central server that schedules tasks onto worker queues and tracks every execution's state, and open-sourced it in 2016. The stated motivations were visibility into flows, the ability to pause, resume and restart them, and reuse of workers across flows. *Evidence: Primary* — [Netflix Technology Blog 2016]

32.9 Exactly-once processing and its limits

The phrase "exactly-once" causes more confusion than any other in messaging. Be precise about *what* happens exactly once, and *where*.

- **Exactly-once delivery** over an unreliable network is impossible. A sender that gets no acknowledgment cannot know whether the message arrived (the two generals problem, §24.3.3), so it must either resend (risking duplicates) or not (risking loss).
- **Exactly-once processing**, better called **effectively-once**, is achievable: a message may be delivered and processed several times, but its *effect* on state is applied once. By the end-to-end argument (§24.14), only the endpoint that owns the state can guarantee this, with at-least-once delivery plus deduplication or idempotence.

effectively-once = at-least-once delivery + idempotent effect

32.9.1 What Kafka's exactly-once semantics cover

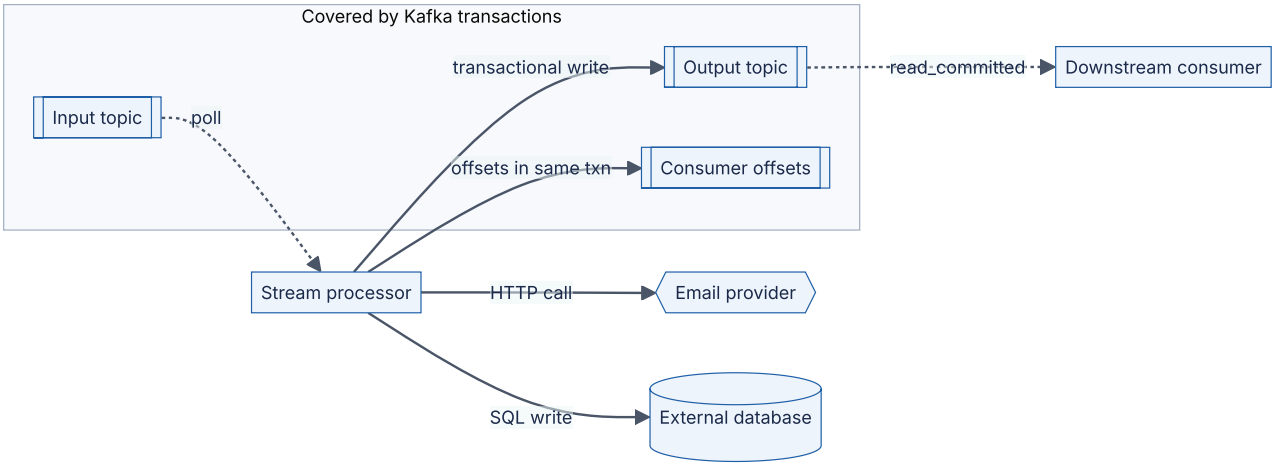
Kafka's exactly-once semantics (EOS) combine three mechanisms [Kafka KIP-98 2017; §31.11]:

1. **Idempotent producer.** The broker assigns a producer ID, and the producer numbers each batch per partition; the broker discards a retried batch whose sequence number it has already written. This removes duplicates caused by producer retries, within one producer session.

2. **Transactions.** A transactional producer writes to several partitions *and* commits the consumer's input offsets in one atomic transaction (`sendOffsetsToTransaction`), using the replicated transaction coordinator described in §28.4.8.
3. **`read_committed` consumers** see only committed transactional data.

Together these make a **consume-transform-produce** loop inside Kafka effectively-once: input offsets and outputs commit together, so a crash replays input but aborted outputs are never seen. Kafka Streams enables this with `processing.guarantee=exactly_once_v2`.

Figure 32.13 — The boundary of Kafka's exactly-once guarantee.



Reading the diagram. Everything inside the subgraph lives in Kafka and commits atomically: the output records and the input offsets. A crash before commit aborts both, and the replayed input produces the same outputs again, which `read_committed` consumers see once. The two arrows leaving the boundary are outside the guarantee: if the processor sends an email or writes a database row and then crashes before committing, the replay sends the email again. Failure points therefore cluster at external side effects. Scaling: transactions add a few milliseconds per commit, amortized by committing every 100 ms or so rather than per record.

Table 32.11 — What Kafka exactly-once does and does not cover.

Covered	Not covered
Duplicates from producer retries (idempotent producer)	A producer application that restarts and resends with a new producer ID, unless it uses a stable `transactional.id`
Atomic writes to several partitions	Writes to databases, caches, search indexes, HTTP APIs
Atomic commit of outputs with input offsets	Consumers using `read_uncommitted`
Kafka Streams state stores (changelog topics)	Side effects in user code: emails, payments, logs

COMMON MISTAKE

Common misconception — "We enabled Kafka EOS, so our payment consumer is exactly-once." EOS ends where Kafka ends. A consumer that charges a card through an API and commits its offset afterwards will charge twice after a crash between the two. Pass an idempotency key derived from the event ID to the provider (§22.15), or record the charge and the offset in one database transaction (§32.10).

32.10 Idempotent consumers

DEFINITION

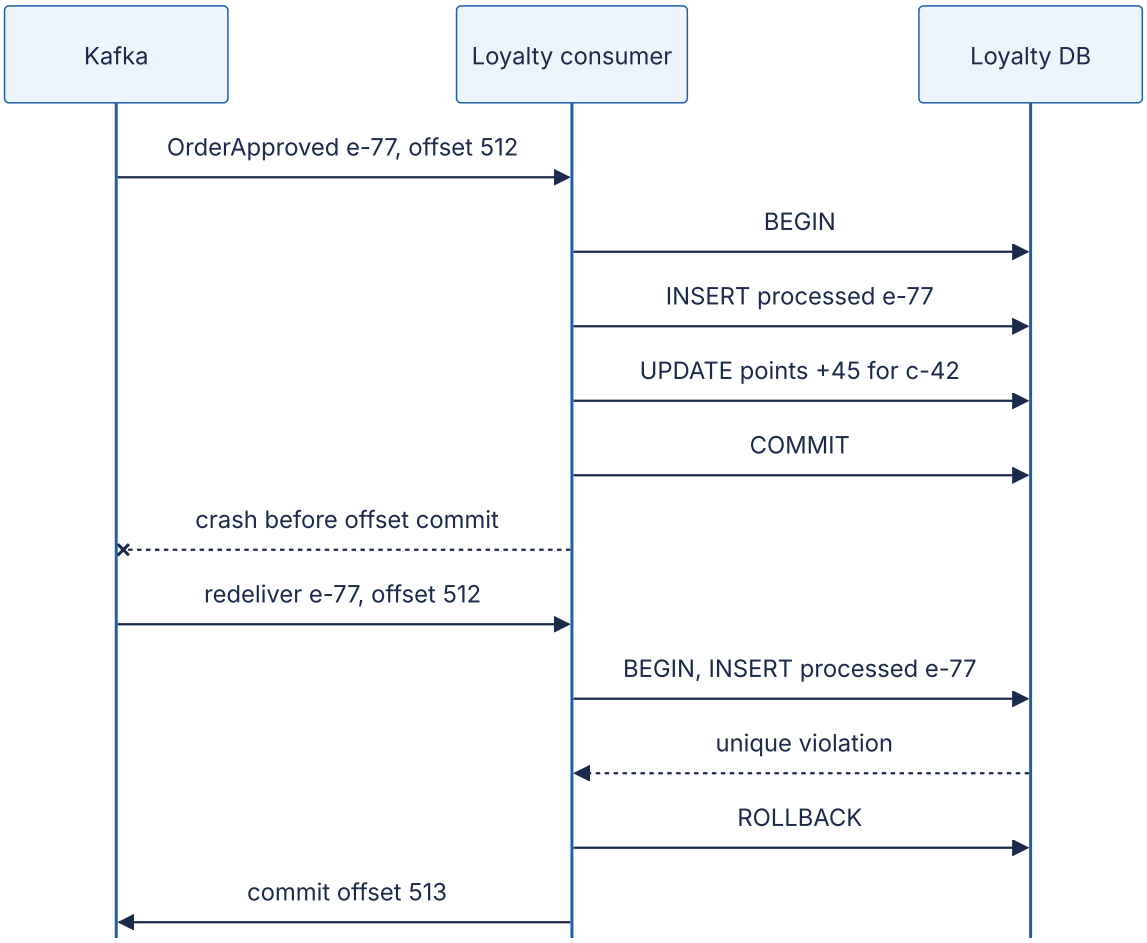
Definition — Idempotent consumer. A message consumer whose effect on state is the same whether a given message is processed once or many times, achieved either because the operation is naturally idempotent or because the consumer detects and skips messages it has already processed.

Section 22.15 owns idempotency keys for APIs; consumers use the same idea with the **event ID** as the key. Three techniques cover nearly every case.

Table 32.12 — Idempotent consumer techniques.

Technique	How it works	Strength	Limitation
Natural idempotency	Operation is idempotent by nature: `SET status = 'SHIPPED'`, upsert by key, add to a set	No extra state	Not available for increments, sends, charges
Dedup table	`INSERT INTO processed(event_id)` in the same transaction as the effect; a unique-key conflict means "already done"	Works for any effect in the same database	Storage grows with rate × window; useless if the effect is outside that database
Versioned (conditional) writes	Store the last applied version per entity; apply only if `event.version > stored.version`	Constant storage per entity; also rejects reordered stale events	Needs a per-entity monotonic version from the producer

Figure 32.14 — A consumer that deduplicates in the same transaction as its effect.



Reading the diagram. The broker delivers at least once; the consumer's database is the storage boundary where deduplication happens. The dedup row and the points increment commit in one local transaction, so they cannot diverge. The crash after commit but before the offset commit causes redelivery, and the unique key on `event_id` turns the duplicate into a no-op. Failure points: an effect outside the database (an email) cannot join this transaction and needs a downstream idempotency key or an outbox. Scaling: partition the dedup table with the consumer's data and expire old rows.

An alternative is to store the consumer's **offset** in the same database as the effect (and seek to it on startup), which deduplicates without per-message rows, provided one consumer owns each partition.

FORMULA

Storage needed by a dedup table

$$S = \lambda \times W \times b$$

Symbol	Meaning	Units
λ	Messages consumed per second	messages/s
W	Dedup window: how long an ID must be remembered	s
b	Bytes per dedup row including index overhead	bytes
S	Storage required	bytes

WORKED EXAMPLE

Worked example — How long to remember event IDs.

1. The loyalty consumer handles $\lambda = 5,000$ messages/s. Redeliveries normally arrive within minutes, but a replay from a backup may resend a week of events, so $W = 7$ days $= 604,800$ s.
2. Rows: $5,000 \times 604,800 = 3.024 \times 10^{9}$. At $b = 60$ bytes (16-byte UUID, timestamp, B-tree overhead): $S \approx 1.8 \times 10^{11}$ bytes ≈ 181 GB (decimal).
3. With $W = 24$ h: $5,000 \times 86,400 \times 60 \approx 26$ GB.

Interpretation: a long window is expensive. Engineering implication: partition the dedup table by day and drop old partitions; or switch to versioned writes, which need one row per *entity* (a few million customers, a few hundred megabytes) and remember forever.

COMMON MISTAKE

Common misconception — "Checking whether we processed the message, then processing it, is idempotent." A check followed by a separate write is a race: two redeliveries processed concurrently both pass the check. The dedup insert must be atomic with the effect, enforced by a unique constraint.

32.11 Workflow engines

32.11.1 Durable execution

The orchestrator in §32.7.4 is about 60 lines of saga logic, and a production version needs much more: timers ("cancel the reservation if unpaid in 15 minutes"), backoff, versioning

while thousands of sagas are in flight, visibility ("where is order 981?"), and a scalable, replicated saga log. **Workflow engines** package all of it.

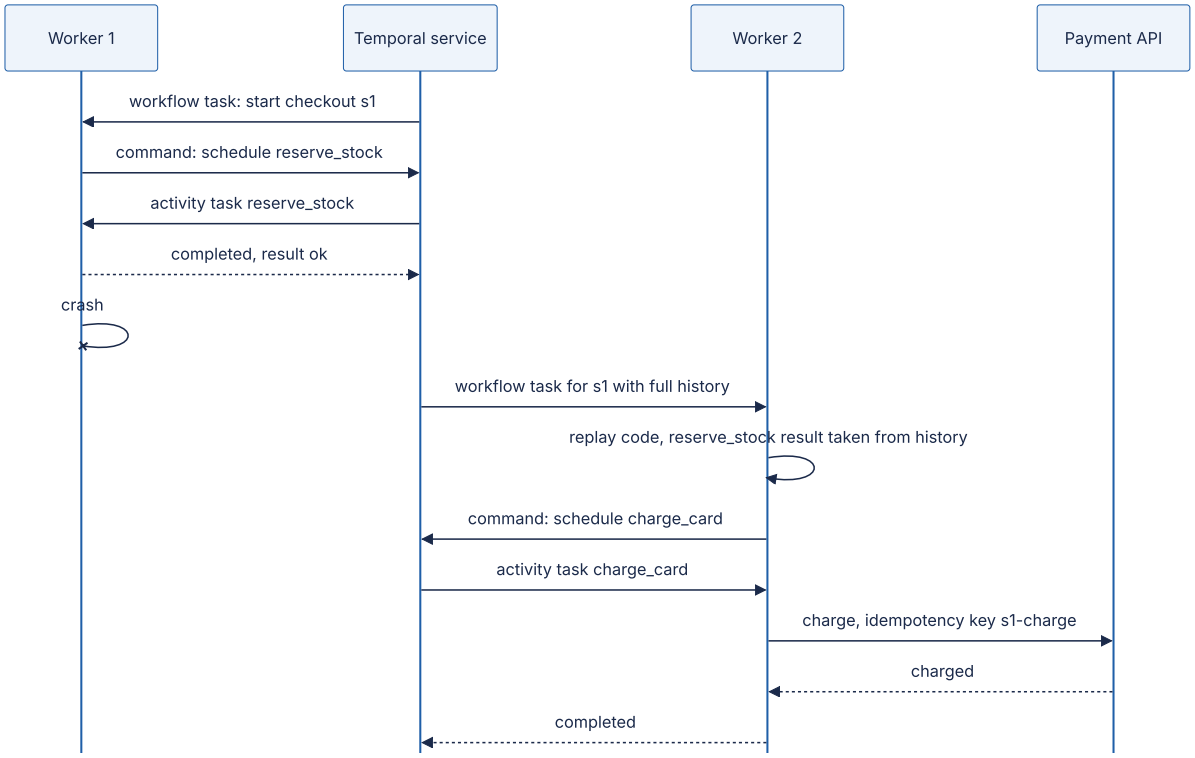
> DEFINITION
>
> **Definition — Durable execution.** A programming model in which a function's progress is persisted by an engine as it runs, so that the function survives process crashes, deploys and arbitrarily long waits and continues exactly where it left off, as if the process had never died [Temporal Docs].

How Temporal works internally

Temporal, a fork of Uber's Cadence by Cadence's original authors, implements durable execution with **event-history replay** [Temporal Docs].

1. **Workflow code** is ordinary code (Go, Java, Python, TypeScript, .NET and others) that orchestrates; **activities** are functions that do the side-effecting work (call the payment API, write a database).
2. The Temporal service stores, for each workflow execution, an append-only **event history**: `WorkflowExecutionStarted`, `ActivityTaskScheduled`, `ActivityTaskCompleted` with its result, `TimerFired`, and so on.
3. **Workers** in your own deployment poll **task queues**. When a workflow needs to make progress, a worker runs the workflow code until it blocks on an activity or timer, and returns the resulting **commands** ("schedule activity `charge_card`"), which the service records as events.
4. If the worker crashes or evicts the workflow from its cache, any other worker **replays** the history: it re-runs the workflow code from the start, and whenever the code asks for an activity that the history says already completed, the SDK returns the recorded result instead of executing it. The code reaches the same point with the same local variables, then continues.
5. Activities run at least once and are retried per a **retry policy** (initial interval, back-off coefficient, maximum interval, maximum attempts, non-retryable error types) bounded by timeouts: *Start-To-Close* (one attempt), *Schedule-To-Close* (all attempts) and *Heartbeat* (liveness of long activities). So activities must be idempotent (§32.10).

Figure 32.15 — A Temporal worker crashes and another worker replays the history.



Reading the diagram. The Temporal service owns durable state (the history, in its own replicated database); workers are stateless and in the application's network, connected over gRPC long polls. Workflow tasks run deterministic orchestration code; activity tasks perform side effects. When worker 1 crashes, worker 2 receives the history and replays: `reserve_stock` is *not* executed again because its completion is recorded. The failure point that remains is an activity that crashes after calling the payment API but before reporting completion; it will be retried, which is why the call carries an idempotency key. Scaling: add workers per task queue; the service shards histories by workflow ID.

The determinism constraint

Replay works only if the workflow code, given the same history, issues the same commands in the same order. Workflow code must therefore not read the wall clock, generate random numbers, call the network or the file system, use threads, or iterate over unordered collections in ways that change between runs; SDKs provide deterministic substitutes (`workflow.now()`, deterministic random, durable timers). Changing workflow code while executions are in flight needs explicit **versioning** (a patch marker that lets old executions follow the old branch), or old histories will fail to replay with a *non-determinism error*. Histories also have size limits, so a workflow that loops indefinitely periodically starts a fresh execution with **Continue-As-New**.

The checkout saga as a Temporal workflow in Python:

```
        # illustrative: needs the temporalio package, a Temporal service and a worker
        from datetime import timedelta

        from temporalio import workflow
        from temporalio.common import RetryPolicy
        from temporalio.exceptions import ActivityError

        with workflow.unsafe.imports_passed_through():
            from shop.activities import (approve_order, charge_card, create_order,
                                         reject_order, release_stock, reserve_stock)

        STEP = dict(start_to_close_timeout=timedelta(seconds=30),
                    retry_policy=RetryPolicy(maximum_attempts=3))       # then give up
and compensate
        FOREVER = dict(start_to_close_timeout=timedelta(seconds=30),
                       retry_policy=RetryPolicy(maximum_interval=timedelta(minutes=5)))
# no attempt cap


        @workflow.defn
        class CheckoutWorkflow:
            @workflow.run
            async def run(self, order: dict) -> str:
                await workflow.execute_activity(create_order, order, **STEP)
                undo = []
                try:
                    await workflow.execute_activity(reserve_stock, order, **STEP)
                    undo.append(release_stock)
                    await workflow.execute_activity(charge_card, order, **STEP)      #
pivot
                except ActivityError:
                    for compensation in reversed(undo):
                        await workflow.execute_activity(compensation, order, **FOREVER)
                    await workflow.execute_activity(reject_order, order, **FOREVER)
                    return "COMPENSATED"
                await workflow.execute_activity(approve_order, order, **FOREVER)    #
retriable
                return "COMPLETED"
```

What to notice: compensation is a `try`/`except` with a list of undo steps, readable top to bottom; the pivot and retriable distinctions of §32.7 become retry policies (`STEP` gives up after three attempts, `FOREVER` never does); there is no saga log in user code, because the engine's event history is the saga log.

32.11.2 AWS Step Functions

AWS Step Functions defines workflows as state machines in the JSON-based **Amazon States Language** (ASL): `Task`, `Choice`, `Parallel`, `Map`, `Wait`, `Pass`, `Succeed` and `Fail` states, with `Retry` and `Catch` fields on tasks [AWS Docs: Step Functions]. **Standard workflows** run for up to one year with exactly-once execution of each state transition and are priced per transition; **Express workflows** run for up to five minutes with at-least-

once (asynchronous) or at-most-once (synchronous) semantics and are priced per request and duration, suiting high-volume event processing (as of 2026).

```
        {
          "Comment": "Checkout saga: compensate before the pivot, retry after it",
          "StartAt": "CreateOrder",
          "States": {
            "CreateOrder": {
              "Type": "Task",
              "Resource": "arn:aws:lambda:us-east-1:123456789012:function:create-order",
              "Next": "ReserveStock"
            },
            "ReserveStock": {
              "Type": "Task",
              "Resource": "arn:aws:lambda:us-east-1:123456789012:function:reserve-
stock",
              "Retry": [{"ErrorEquals": ["States.Timeout"], "IntervalSeconds": 1,
"MaxAttempts": 3, "BackoffRate": 2.0}],
              "Catch": [{"ErrorEquals": ["States.ALL"], "ResultPath": "$.error",
"Next": "RejectOrder"}],
              "Next": "ChargeCard"
            },
            "ChargeCard": {
              "Type": "Task",
              "Resource": "arn:aws:lambda:us-east-1:123456789012:function:charge-card",
              "Catch": [{"ErrorEquals": ["States.ALL"], "ResultPath": "$.error",
"Next": "ReleaseStock"}],
              "Next": "ApproveOrder"
            },
            "ApproveOrder": {
              "Type": "Task",
              "Resource": "arn:aws:lambda:us-east-1:123456789012:function:approve-or-
der",
              "Retry": [{"ErrorEquals": ["States.ALL"], "IntervalSeconds": 2,
"MaxAttempts": 20, "BackoffRate": 2.0}],
              "End": true
            },
            "ReleaseStock": {
              "Type": "Task",
              "Resource": "arn:aws:lambda:us-east-1:123456789012:function:release-
stock",
              "Retry": [{"ErrorEquals": ["States.ALL"], "IntervalSeconds": 2,
"MaxAttempts": 20, "BackoffRate": 2.0}],
              "Next": "RejectOrder"
            },
            "RejectOrder": {
              "Type": "Task",
              "Resource": "arn:aws:lambda:us-east-1:123456789012:function:reject-order",
              "Retry": [{"ErrorEquals": ["States.ALL"], "IntervalSeconds": 2,
"MaxAttempts": 20, "BackoffRate": 2.0}],
              "Next": "OrderRejected"
            },
            "OrderRejected": {"Type": "Fail", "Error": "OrderRejected", "Cause": "Saga
compensated"}
          }
        }
```

What to notice: `Catch` routes a failure before the pivot into the compensation chain (`ReleaseStock` → `RejectOrder`); states after the pivot and all compensations use generous `Retry` blocks instead of `Catch`; `ResultPath` keeps the original input so compensations still know the order. Each Task invocation is at-least-once from the function's point of view, so the functions must be idempotent.

32.11.3 When to use an engine

Table 32.13 — Hand-rolled saga versus workflow engine.

	Hand-rolled (saga table + messages)	Temporal or Cadence	AWS Step Functions
How it works	Your orchestrator, your saga log, your timers	Code-first durable execution with history replay	JSON state machine, managed service
Advantages	No new infrastructure; full control	Arbitrary logic in a real language; timers, signals, versioning, visibility	Fully managed; deep AWS integrations; visual console
Disadvantages	You rebuild retries, timers, visibility, versioning	Operate a cluster (or buy the cloud service); determinism discipline	AWS lock-in; complex logic awkward in JSON; per-transition cost
Performance	As fast as your broker	Tens of milliseconds per step	Tens of milliseconds per transition; Express faster
Choose when	2–3 steps, no timers, team already runs a broker	Many long-running processes, complex branching, multi-cloud	Serverless AWS workloads, simple to moderate flows

RECOMMENDATION

Recommendation — Count your sagas' features, not their steps. Once a process needs two of {timers longer than a request, human approval, more than four steps, versioned rollouts while instances are in flight, an operator UI to find and repair stuck instances}, a workflow engine is cheaper than building those features yourself.

INDUSTRY EXAMPLE

Uber Cadence and Temporal. Cadence was built at Uber to run long-lived, fault-tolerant business workflows as code and was open-sourced; its original creators later forked it as Temporal and founded a company around it. The core design, event-history replay with deterministic workflow code and retried idempotent activities, is shared by both. *Evidence: Primary* — [Temporal Docs]

COMMON MISTAKE

Common misconception — "A workflow engine gives exactly-once side effects." It gives exactly-once *workflow progress*: each step's completion is recorded once and never replayed. The activity itself is at-least-once, so a payment activity still needs an idempotency key.

32.12 Summary and revision notes

SUMMARY

- **Event sourcing** stores an aggregate's append-only events as the source of truth; state is a fold of `apply` over them. Appends carry an **expected version** (optimistic concurrency). Replay costs N/r; **snapshots** cut per-command cost to about $t_s + S/(2r)$ but do not speed up projection rebuilds.
- Events live forever: evolve schemas additively, **upcast** old versions on read, and treat rewriting history as a last resort. Keep personal data out of events or **crypto-shred** it with per-subject keys.
- **CQRS** separates the write model from read models built by projections; reads are eventually consistent. Catch-up after a pause takes $\lambda D/(\mu - \lambda)$. Use consistency tokens for read-your-writes. Do not use CQRS for plain CRUD.
- **Dual writes** to a database and a broker lose events (database first) or invent them (broker first). The **transactional outbox** writes the event in the same local transaction; a relay publishes it at least once. Polling relays are simple; log-tailing relays give commit order and lower latency. Relay throughput is $B/(t_q + Bt_p + t_m)$.
- **Log-based CDC** (Debezium) snapshots tables, then streams the WAL or binlog in commit order with before and after images; watch replication-slot retention and enforce schema compatibility with a registry.
- A **saga** is $T_1 \dots T_n$ or $T_1 \dots T_j, C_j \dots C_1$. Steps are compensatable, the **pivot**, or retriable. Sagas lack isolation; use semantic locks, commutative updates, reread values and reservations.
- **Choreography** suits short, stable flows and broadcasting facts; **orchestration** suits complex processes with compensations and deadlines.
- Exactly-once *delivery* is impossible; **effectively-once = at-least-once + idempotent effect**. Kafka EOS covers Kafka-to-Kafka processing only.
- **Idempotent consumers** use natural idempotency, a dedup table committed with the effect ($S = \lambda W b$), or versioned writes.
- **Workflow engines** (Temporal, Step Functions) provide durable execution: the event history is the saga log, workflow code must be deterministic, and activities are at-least-once.

32.13 Knowledge checkpoint

1. Why must an event-sourced `apply` function never validate?
2. What does the expected version prevent, and what does a writer do when it fails?
3. A stream has 80,000 events, replay runs at 40,000 events/s, snapshots every 1,000 events with $t_s = 1$ ms. Compute load time with and without snapshots.

4. Name two ways to evolve an event schema without rewriting history.
5. How does crypto-shredding erase data from backups you cannot modify?
6. A projection at $\mu = 3,000$ events/s, with $\lambda = 1,000$ events/s, was down for 30 minutes. How long does it take to catch up?
7. Give the two failure sequences of a dual write and the anomaly each causes.
8. Why can a polling relay that remembers the highest published `id` skip events?
9. What does a Debezium change event contain, and why is delivery at-least-once?
10. What is a pivot transaction, and how are failures before and after it handled?
11. Name three countermeasures for the missing isolation of sagas.
12. When would you choose choreography over orchestration?
13. What exactly does a Kafka transaction make atomic?
14. A consumer handles 2,000 messages/s and must remember IDs for 3 days at 50 bytes each. How much storage?
15. Why must Temporal workflow code be deterministic, and what happens if it is not?

Answers

1. Replay must always reproduce history. An old event that today's rules reject still happened; if `apply` throws, the aggregate can never be loaded.
2. Lost updates from concurrent writers on the same aggregate. The writer reloads the stream, re-validates the command against the new state, and retries or rejects.
3. Without: $80,000/40,000 = 2$ s. With: $0.001 + 500/40,000 = 0.001 + 0.0125 = 13.5$ ms.
4. Additive optional fields with defaults; upcasters that convert old versions on read (or a new event type).
5. Backups contain only ciphertext; deleting the per-subject key from the key store makes every copy unreadable.
6. Backlog $1,000 \times 1,800 = 1.8 \times 10^6$ events; $T = 1.8 \times 10^6 / 2,000 = 900$ s, 15 minutes.
7. Database commit then crash before publish: lost event. Publish then database rollback or crash: phantom event.
8. IDs are assigned at insert but become visible at commit; a transaction with a lower `id` can commit after the relay has passed it.
9. Before and after row images, operation type, and source metadata (log position, commit time, table). Offsets are stored periodically, so after a restart events since the last stored offset are re-sent.
10. The go/no-go step after which the saga must complete. Failures before or at the pivot trigger compensations in reverse order; failures after it are retried forward.
11. Any three of: semantic lock, commutative updates, pessimistic view, reread value, version file, by value.
12. For short, stable flows of two to four steps across autonomous teams, and for broadcasting facts that many domains consume.
13. Writes to several partitions plus the consumer's input offsets, made visible together to `read_committed` consumers.
14. $2,000 \times 259,200 \times 50 \approx 2.6 \times 10^{10}$ bytes ≈ 26 GB.

15. A replaying worker must regenerate exactly the commands recorded in the history to rebuild state. Nondeterminism produces different commands and a non-determinism error that stalls the execution.

32.14 Interview questions

Basic

Q1. What is the transactional outbox and why is it needed? Answer. A service must update its database and publish an event, but there is no transaction across the two systems, so a crash between writes loses an event or publishes a phantom. The outbox inserts the event into a table in the same local transaction as the state change; a relay (polling or CDC) publishes committed rows and marks them, at least once. Consumers deduplicate by event ID. Mention ordering by aggregate key and monitoring outbox age.

Q2. What is the difference between event sourcing and CQRS? Answer. Event sourcing is a storage choice: events are the source of truth and state is derived. CQRS is a model split: commands and queries use different models. They pair well because an event store cannot answer ad hoc queries, but either can be used alone. Both introduce eventual consistency in read models.

Intermediate

Q3. Design checkout across order, inventory and payment services without 2PC. Answer. An orchestrated saga: create order `PENDING` (compensate: reject), reserve stock with a 15-minute expiry (compensate: release), charge card (pivot), approve order (retriable). Persist saga state before each command; every participant deduplicates by saga ID; commands and events flow through outboxes. Walk through a failure at each step, the semantic lock on `PENDING`, and what users see during compensation. Consider a workflow engine if timers and visibility matter.

Q4. Your team says Kafka exactly-once makes the email consumer safe. Respond. Answer. EOS makes Kafka-to-Kafka read-process-write atomic. Sending an email is an external side effect: a crash after sending and before committing offsets re-sends it. Make it effectively-once by recording the send in a database with the event ID in the same transaction as the decision, using an outbox for the email request, and passing an idempotency key to the provider if it supports one. Some duplicates may remain possible; quantify and accept or reconcile.

Advanced

Q5. An event-sourced ledger has 3 billion events and a new regulatory report needs a projection over all of them by next week. Plan it. Answer. Estimate: at 100,000 events/s per worker, a single pass takes about 8.3 hours; partition by account into 32 workers to bring it under 20 minutes, subject to event-store read bandwidth. Build the projection blue-green beside existing ones, checkpoint per partition, make upserts idempotent, verify against sampled full replays, then switch readers. Watch upcaster cost on old events, read load on the production store (replay from a replica or an archive copy), and snapshot schema versions.

Q6. When would you choose Temporal over a hand-rolled saga, and what discipline does it impose? Answer. Choose it when processes are long-running, have timers, human steps, many branches, or need operator visibility and safe versioning. It imposes determinism in workflow code (no clocks, randomness or I/O except through SDK APIs), explicit versioning for code changes while executions are in flight, idempotent activities with sensible retry policies and timeouts, history-size management with Continue-As-New, and operating the service or paying for the cloud service.

32.15 Practical exercises

1. Extend the event-sourced account of §32.3.3 with a `snapshot_schema` field and make `load` ignore snapshots whose schema differs; add a test that changes `apply` and still loads correctly.
2. Modify the outbox simulation to use a high-water-mark relay with two concurrent writers whose commits can reorder, and show a test that loses events; then fix it.
3. Add a 15-minute reservation timeout to the saga in §32.7.4 so that a crashed orchestrator’s reservations expire, and test that a late `charge_card` after expiry is refused.
4. Run Debezium against a local PostgreSQL, stop the connector for ten minutes under write load, and graph replication-slot lag in bytes.
5. Implement the loyalty consumer of Figure 32.14 with SQLite: a dedup table, then versioned writes; compare storage after one million messages.

32.16 Design challenges

1. **Ride-hailing trip lifecycle.** Model a trip (request, match, pickup, drop-off, payment, rating) as either an event-sourced aggregate or a workflow. Identify the pivot, the compensations, and the read models drivers and riders need, with their acceptable lag.

2. **Search index for 500 million products.** Keep it consistent with a sharded MySQL catalog using CDC. Plan the initial load without downtime, schema changes, and the response to a connector outage of six hours.
3. **Cross-border payment.** A transfer debits a local account, converts currency through a partner, and credits a foreign bank via an API that may take two days to confirm. Choose choreography, orchestration or a workflow engine, and specify every idempotency key.

32.17 References

1. Garcia-Molina, H., Salem, K. *Sagas*. Proceedings of ACM SIGMOD, 1987.
2. Fowler, M. *Event Sourcing*. martinfowler.com, 2005. https://martinfowler.com/eaaDev/EventSourcing.html
3. Fowler, M. *The LMAX Architecture*. martinfowler.com, 2011. https://martinfowler.com/articles/lmax.html
4. Fowler, M. *CQRS*. martinfowler.com bliki, 2011. https://martinfowler.com/bliki/CQRS.html
5. Young, G. *CQRS Documents*. 2010.
6. Young, G. *Versioning in an Event Sourced System*. Leanpub, 2017.
7. Richardson, C. *Microservices Patterns: With Examples in Java*. Manning, 2018. Pattern pages at https://microservices.io/patterns/
8. Helland, P. *Life beyond Distributed Transactions: an Apostate's Opinion*. CIDR, 2007.
9. Kleppmann, M. *Designing Data-Intensive Applications* (Ch. 11 "Stream Processing", Ch. 12 "The Future of Data Systems"). O'Reilly, 2017.
10. Frank, L., Zahle, T. U. *Semantic ACID Properties in Multidatabases Using Remote Procedure Calls and Update Propagations*. Software: Practice and Experience 28(1), 1998.
11. Debezium Project. *Debezium Documentation: PostgreSQL connector, Outbox Event Router, incremental snapshots*. https://debezium.io/documentation/
12. Andreakis, A., Papapanagiotou, I. *DBLog: A Watermark Based Change-Data-Capture Framework*. arXiv, 2020.
13. Das, S., et al. *All Aboard the Databus! LinkedIn's Scalable Consistent Change Data Capture Platform*. ACM Symposium on Cloud Computing (SoCC), 2012.
14. Confluent. *Schema Registry Documentation: Schema Evolution and Compatibility*. https://docs.confluent.io/
15. Apache Kafka. *KIP-98: Exactly Once Delivery and Transactional Messaging*. 2017. https://cwiki.apache.org/confluence/display/KAFKA/KIP-98+-+Exactly+Once+Delivery+and+Transactional+Messaging
16. Temporal Technologies. *Temporal Documentation: Workflows, Activities, Event History, Retry Policies*. https://docs.temporal.io/

17. Amazon Web Services. *AWS Step Functions Developer Guide*. https://docs.aws.amazon.com/step-functions/
18. Netflix Technology Blog. *Netflix Conductor: A Microservices Orchestrator*. 2016.
19. European Parliament and Council. *Regulation (EU) 2016/679 (General Data Protection Regulation)*, Article 17. Official Journal of the European Union L 119, 2016.
20. PostgreSQL Global Development Group. *PostgreSQL 16 Documentation: Logical Decoding; Replication Slots*. https://www.postgresql.org/docs/16/logicaldecoding.html

32.18 Open verification items

- LMAX throughput (about six million orders per second) and nightly-snapshot recovery, against Fowler 2011.
- Young 2017 title and publisher; exact URL of Young's 2010 *CQRS Documents*.
- Frank and Zahle 1998 as the origin of Richardson's countermeasure list.
- DBLog arXiv identifier and Netflix Conductor blog URL.
- Debezium's default delivery guarantee as of 2026 (exactly-once support via Kafka Connect may exist for some connectors).
- Step Functions Standard/Express limits and semantics, and Temporal history limits, as of 2026; per-step latency figures in Table 32.13.
- Whether crypto-shredding satisfies Article 17 is a legal question; no source asserted.

Part VII — Software Architecture and Design

Chapter 33 — Architecture Patterns

> **Level:** 2 · **Track:** A (Traditional) · **Estimated reading time:** 2.5 h

An **architecture pattern** (or **architectural style**) is a reusable answer to the question "how should the parts of this system be arranged, and how may they depend on each other?" Each pattern trades some properties for others: a monolith buys simplicity with coupling, microservices buy team independence with network calls, cells buy a small blast radius with duplicated infrastructure. None is best in general. This chapter shows eleven patterns on one running example, **Folio**, a fictional online bookstore that sells books from a catalog, takes payments through an external payment provider, reserves inventory, ships parcels and emails customers. You will see Folio drawn in each style, so the differences are about structure rather than about the problem. Chapter 34 then goes deep on microservices and domain-driven design.

33.1 Learning objectives

After this chapter you should be able to:

- Distinguish logical layers from physical tiers, strict from relaxed layering, and recognize the architecture sinkhole anti-pattern.
- State the real benefits of a monolith and quantify how deployment coupling grows with the number of teams.
- Design a modular monolith with explicit module interfaces and mechanically enforced dependency rules.
- Contrast service-oriented architecture (SOA) with microservices, and place event-driven architecture among the styles.
- Explain serverless execution, cold starts and concurrency, and compute the break-even request rate against always-on servers.
- Implement a port and two adapters in Python and test the domain core with a fake.
- Explain the dependency rule of clean architecture and relate hexagonal, onion and clean architecture.
- Size cells, compute blast radius, and calculate shuffle-sharding overlap probabilities.
- Choose between monolith, modular monolith and microservices using team size, deployment frequency, domain maturity and Conway's law.

33.2 Prerequisites

- Decomposition, components and weighted decision matrices — §2.3 and §2.8 (patterns are pre-packaged decompositions).
- Availability in series and parallel — §4.14.2 (why network hops reduce availability).
- Little's Law — §4.11 (serverless concurrency).
- Combinations and probability — §4.6 (shuffle-sharding arithmetic).
- Stateless services and the scale cube — §18.5 and §18.10 (cells are Z-axis splits).
- Bulkheads — §22.14 (cells are bulkheads for whole stacks).
- Messaging and events — §30.3–§30.6 (event-driven architecture in brief).

33.3 Layered architecture

33.3.1 Layers, tiers and the flow of a request

> DEFINITION
>
> **Definition — Layered architecture.** A style that organizes code into horizontal **layers**, each with one kind of responsibility (presentation, business logic, persistence), where each layer may depend only on layers below it. A **tier** is a *physical* deployment boundary (browser, application server, database server); an **n-tier architecture** maps layers onto separately deployed tiers.

First-principles explanation

Think of a restaurant. The waiter takes orders and presents dishes but never cooks. The kitchen applies the recipes but never talks to diners. The storeroom keeps ingredients but has no idea what a recipe is. Each group can be trained, replaced or reorganized without retraining the others, because each talks only to its neighbor through a narrow interface: an order ticket, a requisition slip. Software layers apply the same idea. The **presentation layer** renders pages and parses requests; the **business layer** (also called the service or domain layer) enforces rules such as "you cannot order more copies than are in stock"; the **persistence layer** turns objects into SQL; the **database** stores rows.

Why does this exist?

Layering was the default enterprise style from the 1990s because it matched how organizations were staffed: UI developers, back-end developers and database administrators. It gives **separation of concerns** (a page designer does not need SQL), **layers of isolation** (replacing the persistence technology touches one layer), and a structure every new hire already recognizes [Richards and Ford 2020]. It is also the natural starting point of most

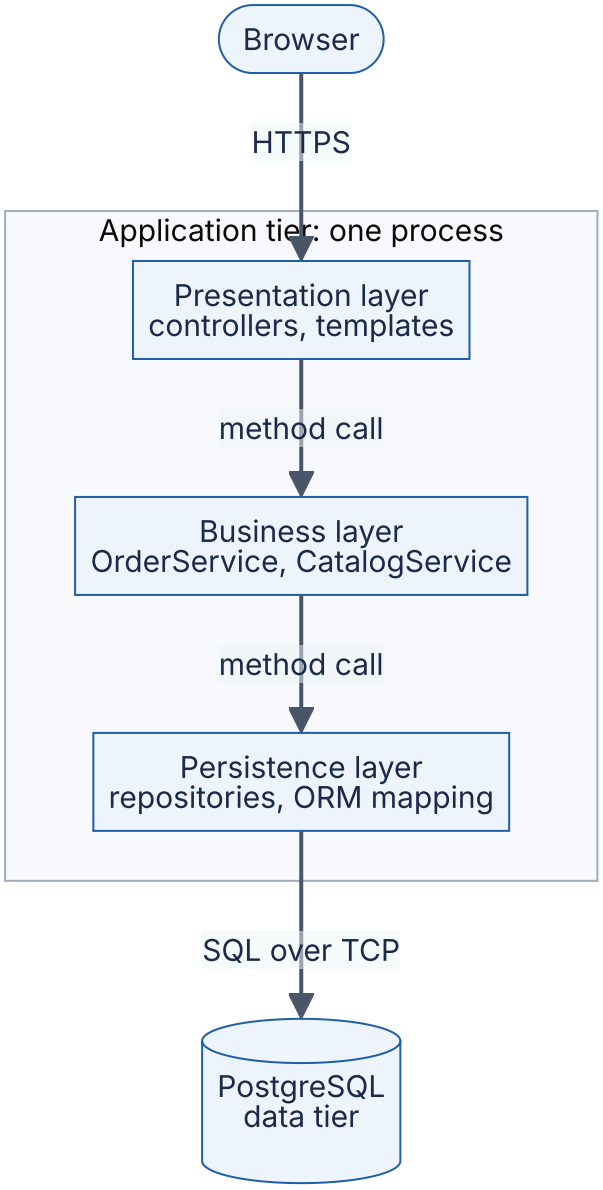
web frameworks, whose project templates already contain `controllers`, `services` and `repositories` folders.

How it works internally

1. A browser sends `POST /orders` to the presentation layer, which validates the input format and converts it into a call such as `order_service.place(isbn, qty)`.
2. The business layer checks stock, computes the price, applies discounts and decides whether the order is valid.
3. The persistence layer converts the decision into SQL statements, inside a transaction.
4. The database executes the statements; the result returns upward through the same layers.

Layers may be **closed** or **open**. A closed layer must be passed through: presentation may call business but not persistence. An open layer may be skipped. **Strict layering** makes every layer closed; **relaxed layering** opens some, typically a shared “utilities” or persistence layer, so upper layers can reach it directly [Richards and Ford 2020; Bass et al. 2021].

Figure 33.1 — Folio as a strict layered application deployed on three tiers.



Reading the diagram. The browser is the client tier; it sends HTTPS requests to the application tier. Inside that one process, three logical layers call each other synchronously with ordinary method calls (solid arrows), always downward: no layer knows about the one above it. Only the persistence layer crosses the storage boundary, speaking SQL over TCP to PostgreSQL in the separate data tier. Failure points are the application process (a crash takes all three layers with it) and the database (every request depends on it). The application tier scales by running identical copies behind a load balancer (§18.4); the data tier scales with replicas and partitioning (Chapter 14). Note that the three layers are not three deployments: layers are logical, tiers are physical.

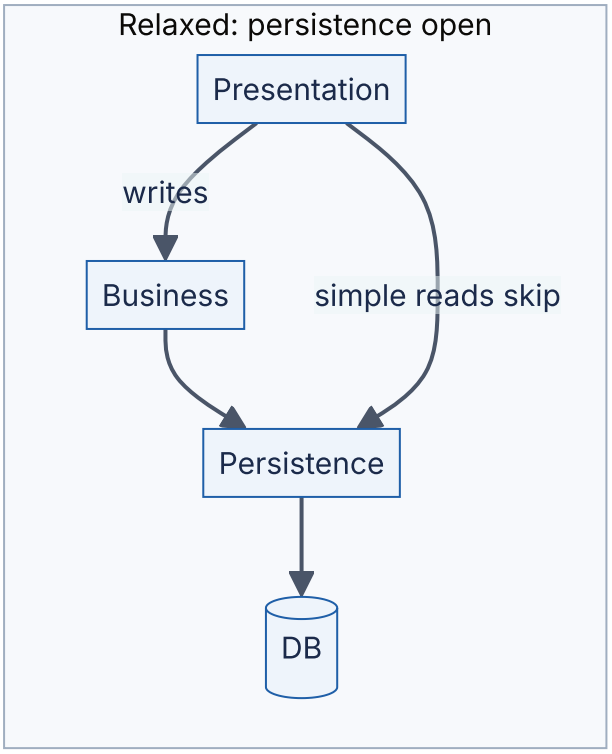
33.3.2 Strict versus relaxed layering and the sinkhole anti-pattern

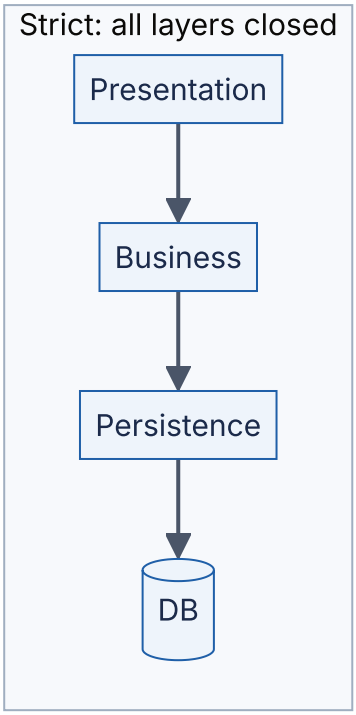
Strict layering maximizes isolation: if every query must go through the business layer, then replacing the ORM touches only persistence, and every rule is enforced in one place.

Its cost is boilerplate. Consider Folio's "show book details" page. The controller calls `CatalogService.get_book(isbn)`, which does nothing but call `BookRepository.find(isbn)`, which runs one `SELECT`. The business layer adds no logic; it only forwards. This is the **architecture sinkhole anti-pattern**: requests that fall through layers without any processing [Richards and Ford 2020]. Each pass-through costs code, a mapping between nearly identical objects, and a test, and multiplies the files you touch for a small change.

Every layered system has *some* sinkholes; simple reads are pass-throughs by nature. Richards suggests an 80–20 heuristic: if roughly 20% of requests are sinkholes that is normal, but if most are, the layering is not paying for itself, and you should either open the layer for those paths or choose a different style [Richards 2015]. Opening a layer is a deliberate trade: the presentation layer now depends on persistence directly, so a persistence change can break the UI.

Figure 33.2 — Strict layering, relaxed layering and a sinkhole path.





Reading the diagram. On the left, every request traverses each layer in order; for a book-detail read the business layer only forwards the call, which is the sinkhole. On the right, the persistence layer is marked open: writes still go through the business layer, where rules live, while simple reads skip it. All calls are synchronous in-process calls; the only storage boundary is the database. The relaxed variant has one more dependency edge (presentation to persistence), which is the failure point to watch: a schema change

can now break pages directly. Neither variant changes how the system scales; layering is a code-organization decision.

Advantages and disadvantages

Layering is easy to understand, matches framework conventions, isolates technology choices, and lets specialists work in parallel. Its central weakness is that it partitions code by *technical* role rather than by *business domain*. Adding "audiobook" as a new product format at Folio touches a template, a controller, a service, a repository and a migration: one feature, five layers, often several teams. Richards and Ford call this **technical partitioning**, as opposed to **domain partitioning**, where code is grouped by business capability (catalog, ordering, shipping) [Richards and Ford 2020]. Most of the remaining patterns in this chapter are, in one way or another, domain-partitioned. Layering also says nothing about deployment, fault isolation or scaling: a layered application is usually a monolith (§33.4).

Failure scenarios

Table 33.1 — Layered architecture failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Business rules leak into controllers or SQL	Deadline pressure; open layers	Same rule implemented twice; inconsistent behavior	Move logic into the business layer	Code review rule; architecture tests
Sinkhole bloat	Strict layering applied to CRUD paths	Most service methods are one-line forwards	Open the layer for reads	Apply the 80–20 check during design
Lower layer depends upward	Callback or import "just this once"	Import cycle between packages	Invert with an interface (§36.7)	Automated dependency checks (§33.5)
Schema change breaks every layer	Database types used as domain objects	Large diffs for small schema changes	Introduce mapping at the persistence boundary	Keep persistence types inside persistence

COMMON MISTAKE

"Layered means n-tier." Three layers can run in one process on one machine, and a three-tier deployment can contain a single tangled layer. Layers describe code dependencies; tiers describe where processes run. Say which one you mean in a design review.

33.4 Monolithic architecture

33.4.1 What a monolith is, and what it is good at

DEFINITION

Definition — Monolith. A system whose functionality is built, tested and deployed as **one deployable unit**, typically one process image running as identical replicas and backed by one shared database. “Monolith” describes the *deployment* unit; it says nothing about whether the code inside is well structured.

First-principles explanation

A monolith is a single building that houses every department of a company. Walking from sales to accounting takes seconds, everyone shares one mailroom, and renovating means closing the whole building for a weekend. For a company of twenty people this is the best possible arrangement; for twenty thousand it becomes a problem. The same is true of code. Folio’s first version is one Python or Java application containing catalog, cart, ordering, payments and shipping code, deployed as one artifact.

Why does this exist?

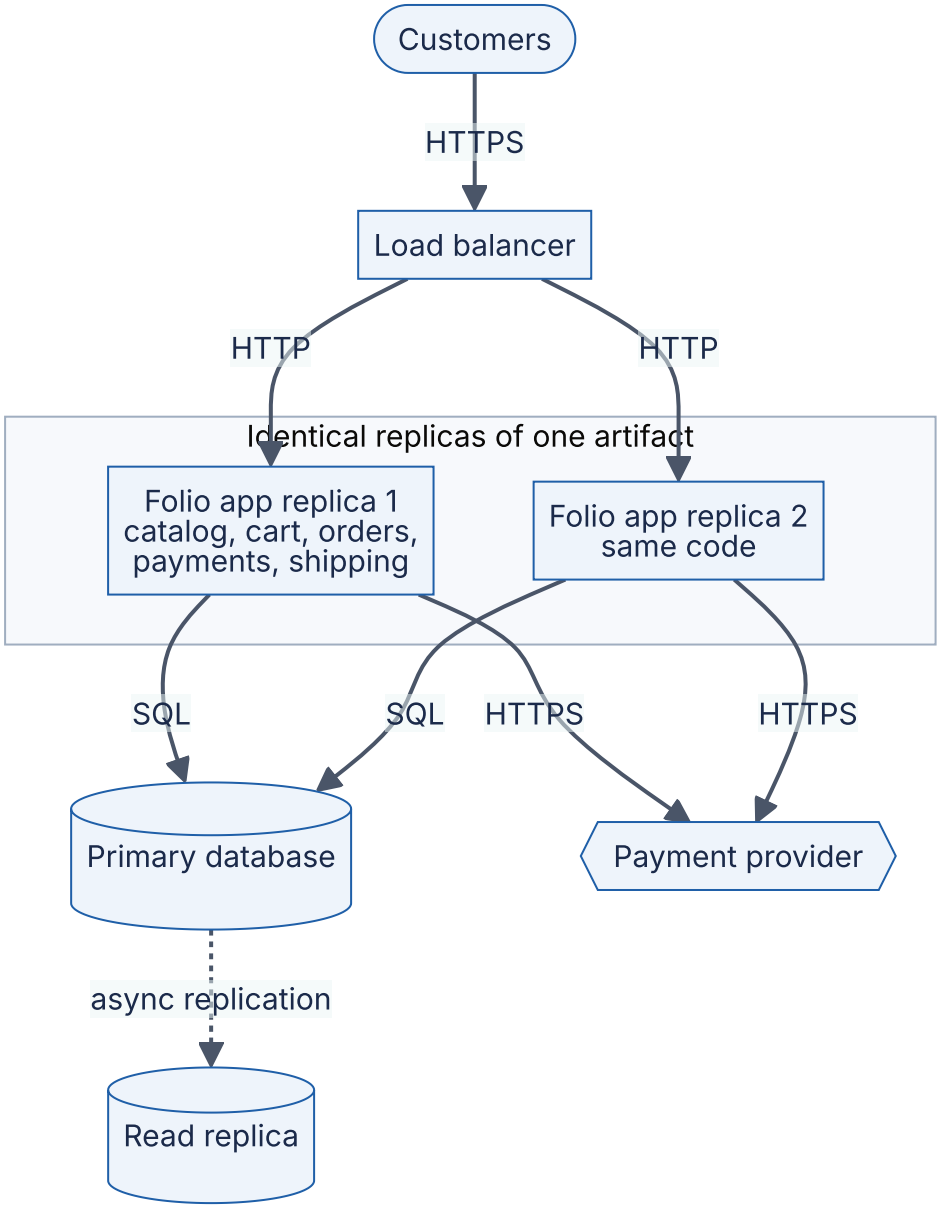
The monolith is not a mistake that later styles corrected; it is the default because its benefits are large and immediate. Stated honestly:

- **Calls are cheap and reliable.** An in-process function call costs nanoseconds and cannot time out. A call over the network within a datacenter costs roughly 0.5 ms round trip and can fail in ways an in-process call cannot (§24.5).
- **Transactions are easy.** Placing an order, decrementing stock and recording a payment intent can share one ACID transaction (§11.12). Across services, the same operation needs a saga (§32.7).
- **Refactoring is cheap.** Renaming a function or moving a field is an IDE operation checked by the compiler or test suite, not a versioned API migration across teams.
- **Operations are simple.** One build pipeline, one deployment, one log stream, one stack trace that shows the whole request.
- **It scales further than people expect.** A stateless monolith scales horizontally behind a load balancer exactly as a stateless microservice does (§18.4–§18.5).

Fowler’s “MonolithFirst” argues that almost all successful microservice systems he had seen started as monoliths that were split later, and that projects started as microservices often ran into trouble, because the right service boundaries are only visible once the domain is understood [Fowler 2015].

How it works internally

Figure 33.3 — Folio deployed as a monolith.



Reading the diagram. Customers reach a load balancer over HTTPS, which spreads synchronous HTTP requests across identical replicas of one artifact; every replica contains every feature. All replicas share one primary database (the storage boundary), which streams changes asynchronously (dotted arrow) to a read replica for reporting and failover. The payment provider is an external system called synchronously. Failure points: a bug that crashes the process crashes every feature on that replica at once, and the primary database is shared by all features. Scaling is by adding replicas (all features scale together) and by database replicas; you cannot scale only the catalog.

33.4.2 The cost: deployment coupling

The monolith's costs grow with the number of people working in it rather than with traffic. Every change, from every team, ships in the same artifact. If any one change is bad, the release is held or rolled back for everyone. This is **deployment coupling**.

Mathematical foundations

FORMULA

Probability that a coupled release is clean

$$P_{\text{clean}} = (1 - p)^{m}, \qquad m = T \times c$$

Symbol	Meaning	Units
P_{clean}	Probability a release contains no release-blocking defect	probability (0–1)
p	Probability that one change contains a release-blocking defect (assumed independent)	probability
m	Number of changes bundled in one release	changes
T	Number of teams contributing to the release	teams
c	Changes per team per release	changes/team

WORKED EXAMPLE

Worked example — Folio's release train at 12 teams. Folio has grown to $T = 12$ teams, each landing $c = 4$ changes into a weekly release; historically $p = 1\%$ of changes contain a defect serious enough to stop a release.

1. Changes per release: $m = 12 \times 4 = 48$.
2. $P_{\text{clean}} = 0.99^{48} = e^{48 \ln 0.99} = e^{48 \times (-0.01005)} = e^{-0.4824} \approx 0.617$.
3. So about 38% of weekly releases are held or rolled back, and when that happens all 12 teams wait.
4. If each team deployed its own unit with its own 4 changes: $0.99^4 \approx 0.961$. Only 4% of releases fail, and a failure delays one team.

Interpretation. Coupling turns one team's defect into everyone's delay; the clean-release probability decays exponentially in the number of teams. **Engineering implication.** Before splitting the deployment, attack p and c: better tests lower p, and deploying smaller batches more often lowers c. With $c = 1$ (continuous deployment of single changes) the 12-team monolith ships clean $0.99^{12} \approx 0.886$ of the time and a failure blocks only the next change. When neither lever is enough, independent deployment units (§33.7) remove the coupling.

Real-world industry applications

INDUSTRY EXAMPLE

Stack Overflow. Stack Overflow has publicly described serving its question-and-answer sites from a monolithic .NET application on a small fleet of web servers, with heavy caching and a large SQL Server installation, rather than from many services. The design works because the team is small relative to traffic, the domain is stable, and the database is tuned and scaled vertically [Craver 2016]. *Evidence: Primary* — [Craver 2016]

Advantages and disadvantages

Beyond deployment coupling, a monolith has three structural costs. **Scaling granularity**: a memory-hungry recommendation feature forces every replica to be sized for it. **Fault isolation**: a memory leak in shipping-label generation exhausts the heap of the process that also serves checkout. **Technology lock-in**: every module uses the same language, runtime and library versions; upgrading the framework is an all-or-nothing project. Build and test times also grow with the code base, which slows every team's feedback loop.

Table 33.2 — Monolith failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
One feature's leak crashes all features	Shared process and heap	Memory alarms; correlated errors on unrelated pages	Roll back; restart replicas	Resource limits per worker pool; bulkheads (§22.14)
Release train blocked	Deployment coupling	Release failure rate rises with team count	Revert offending change	Feature flags (§43.7), smaller batches
"Big ball of mud"	No enforced internal boundaries	Every change touches many packages	Modularize incrementally	Modular monolith (§33.5)
Shared database hot spot	All features share one primary	Lock waits, CPU saturation on primary	Read replicas, caching	Per-module schemas; later partitioning

COMMON MISTAKE

"A monolith is a big ball of mud." Foote and Yoder's **big ball of mud** is code without structure [Foote and Yoder 1997]. A monolith is a deployment choice. A well-modularized monolith can be cleaner than a set of microservices whose boundaries cut through the domain.

When the monolith is right

Choose a monolith when the team fits in one or two groups that can coordinate in a single stand-up (very roughly, up to 10–20 engineers), when the domain is new and its boundaries will move, when strong consistency across features matters, and when there is no platform team to run service infrastructure. That describes most new products.

33.5 Modular monolith

33.5.1 Domain modules inside one deployable

DEFINITION

Definition — Modular monolith. A monolith whose code is divided into **modules** aligned with business domains, where each module exposes a small **public interface**, hides its internals and data, and may depend only on an explicitly declared, acyclic set of other modules. Boundaries are enforced by tooling, not just convention.

First-principles explanation

Return to the single building. A modular monolith keeps one building but gives each department a locked suite with a reception desk. Other departments may leave requests at

the desk (the public interface); they may not walk into the filing room (internal classes and tables). You keep the short walking distance, and you gain the ability to reorganize a suite without the rest of the building noticing. If a department later moves to its own building (a separate service), its reception desk becomes its API.

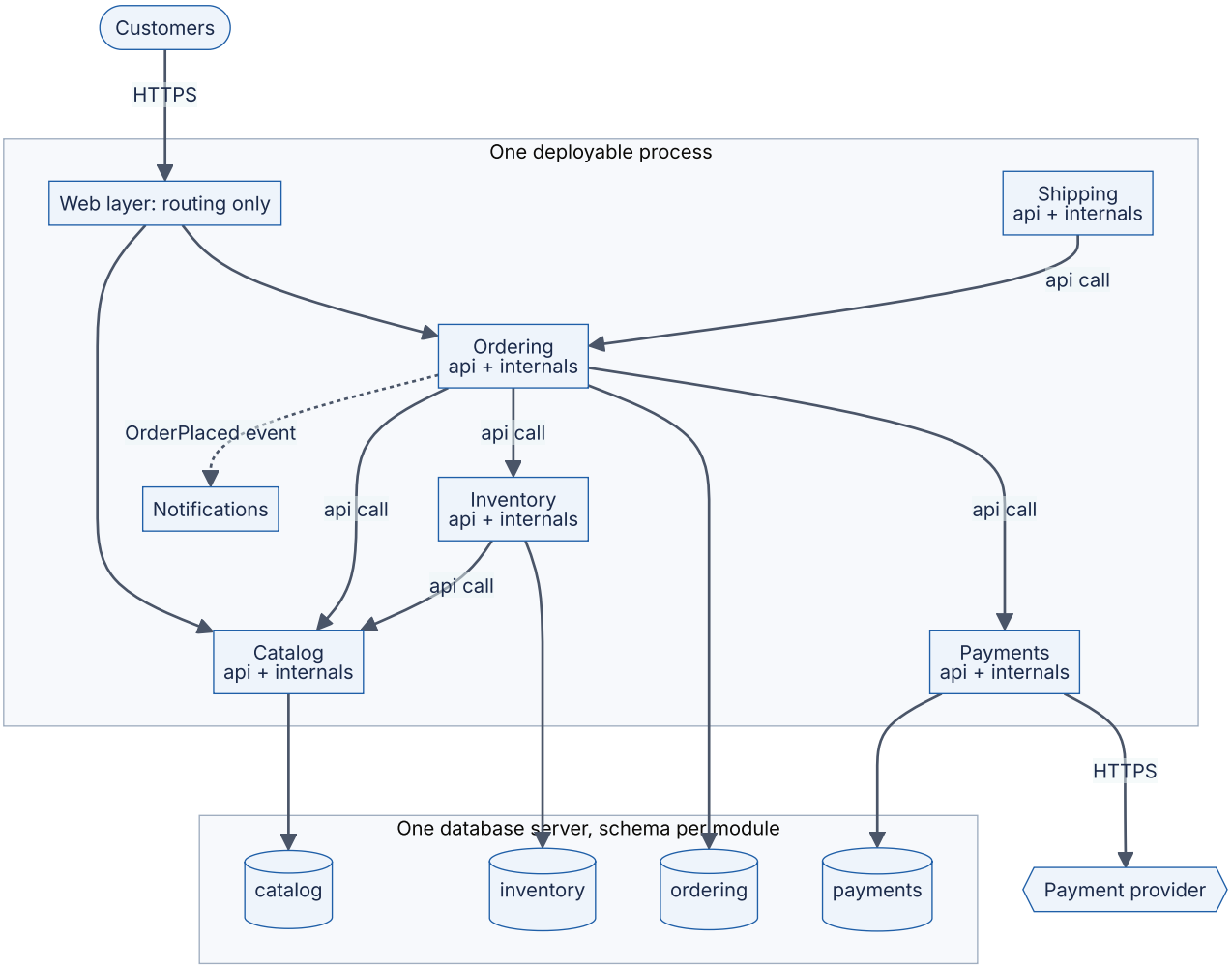
Why does this exist?

It captures most of the monolith's operational simplicity and most of the microservices' maintainability. The idea is old: Parnas argued in 1972 that modules should be chosen to hide design decisions likely to change ("information hiding"), not to mirror steps of processing [Parnas 1972]. The modular monolith applies that rule at the level of business domains, which is why Chapter 2 chose it for QuickBite v1 (§2.8.1).

How it works internally

1. **Partition by domain.** Folio's modules are Catalog, Inventory, Ordering, Payments, Shipping and Notifications. Each owns a package or directory.
2. **Publish a narrow interface.** Each module exposes an `api` package (functions, data-transfer objects, events). Everything else is internal.
3. **Own your data.** Each module owns its tables, ideally in its own database schema. No module joins another module's tables; it calls the other module's interface instead. This is the rule that makes later extraction possible.
4. **Declare dependencies.** A configuration lists which modules each module may use; the graph must be acyclic.
5. **Enforce mechanically.** A build step fails on any import that crosses a boundary illegally. Tools include ArchUnit (Java), Spring Modulith, the Java Platform Module System, `internal` packages in Go and .NET, import-linter (Python) and Packwerk (Ruby).
6. **Decouple with in-process events where order does not matter.** Ordering publishes `OrderPlaced`; Notifications subscribes. Same process, no broker, but the dependency points the right way.

Figure 33.4 — Folio as a modular monolith with declared, acyclic dependencies.



Reading the diagram. All modules run in one process; solid arrows are synchronous in-process calls through each target's public `api`, and every arrow appears in the declared dependency list. The dotted arrow is an in-process event: Ordering does not depend on Notifications, so adding a new subscriber needs no change to Ordering. The storage boundary is one database server divided into schemas, each touched by exactly one module; there are no cross-schema joins. The graph has no cycles: Shipping depends on Ordering, never the reverse. Failure points remain those of a monolith (shared process, shared database server). Scaling is still whole-process, but any module whose interface is clean, Payments for example, can be extracted to its own service by replacing its in-process `api` with a network client (§34.13).

Code example

The enforcement step is simple enough to write yourself. This sketch parses Python imports and checks them against declared dependencies; production teams use import-linter or an equivalent, but the logic is the same.

```
        """Fail the build when a module imports another module's internals or an unde-
clared module."""
        import ast

        # Declared dependencies of Folio's modules (must form a DAG).
        ALLOWED = {
            "catalog": set(),
            "inventory": {"catalog"},
            "ordering": {"catalog", "inventory", "payments"},
            "payments": set(),
            "shipping": {"ordering"},
        }

        # A tiny in-memory "source tree": path -> code.
        SOURCES = {
            "ordering/service.py": "from catalog.api import get_price\n"
                                   "from inventory.api import reserve\n",
            "shipping/service.py": "from ordering.api import order_details\n"
                                   "from ordering.models import OrderRow\n",      #
reaches into internals
            "inventory/service.py": "import shipping.api\n",                     # unde-
clared dependency
        }


        def imported_modules(code: str):
            for node in ast.walk(ast.parse(code)):
                if isinstance(node, ast.Import):
                    yield from (alias.name for alias in node.names)
                elif isinstance(node, ast.ImportFrom) and node.module:
                    yield node.module


        def violations(sources, allowed):
            for path, code in sources.items():
                owner = path.split("/")[0]
                for name in imported_modules(code):
                    target, _, sub = name.partition(".")
                    if target == owner or target not in allowed:
                        continue                                  # own code or third-
party library
                    if target not in allowed[owner]:
                        yield f"{path}: '{owner}' may not depend on '{target}'"
                    elif sub != "api":
                        yield f"{path}: '{name}' is internal to '{target}'; use
'{target}.api'"


        for v in violations(SOURCES, ALLOWED):
            print("VIOLATION", v)
```

Output:

```
VIOLATION shipping/service.py: 'ordering.models' is internal to 'ordering'; use
'ordering.api'
VIOLATION inventory/service.py: 'inventory' may not depend on 'shipping'
```

What to notice:

- Two rules cover most erosion: *only declared dependencies* and *only through* `api`. Ordering's imports pass both.
- The check reads source, not running code, so it runs in seconds in continuous integration and fails the pull request before the coupling exists.
- Imports of names outside `ALLOWED` (standard library, third-party packages) are ignored; a fuller tool would also check that `ALLOWED` itself is acyclic.

Real-world industry applications

> INDUSTRY EXAMPLE
>
> **Shopify's modular monolith.** Shopify has written publicly about its large Ruby on Rails monolith. Rather than splitting it into microservices, Shopify reorganized the code into components aligned with business domains, each with a public interface, and built tooling to detect references that cross component boundaries; it later open-sourced Packwerk, a static-analysis tool for enforcing such boundaries in Rails applications [Westeinde 2019; Shopify Packwerk]. The stated motivation was developer productivity: keeping the benefits of one code base and one deployment while making ownership and boundaries explicit. *Evidence: Primary* — [Westeinde 2019], [Shopify Packwerk]

Advantages and disadvantages

A modular monolith keeps in-process calls, single deployment and the option of ACID transactions (used sparingly, since cross-module transactions are coupling). It localizes change, makes ownership visible, and makes later extraction a mechanical step. It does *not* fix deployment coupling, scaling granularity or fault isolation; those still require separate processes. Its main risk is erosion: without enforcement, boundaries decay one "temporary" import at a time.

Table 33.3 — Modular monolith failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Boundary erosion	Unchecked imports of internals	Dependency-graph diff in CI	Refactor to use `api`	Fail the build on violations
Cross-module SQL joins	Shared database makes it easy	Query log shows tables of two modules	Replace with interface call or read model	Schema-per-module with restricted grants
Dependency cycle	Two modules call each other	Cycle detection in CI	Extract shared concept or invert via event	Acyclic dependency rule
Wrong boundaries	Modules chosen by technical layer	Features always touch many modules	Re-partition by domain (§34.4)	Domain analysis before coding

33.6 Service-oriented architecture

33.6.1 Services, the enterprise service bus and the reuse goal

DEFINITION

Definition — Service-oriented architecture (SOA). An enterprise integration style, prominent from roughly 2000 to 2010, in which business capabilities are exposed as coarse-grained, reusable **services** with formal contracts, typically SOAP web services described in WSDL, connected through an **enterprise service bus (ESB)** that routes, transforms and orchestrates messages between them.

First-principles explanation

Picture a large company whose departments each bought their own software: an ERP for inventory, a CRM for customers, a mainframe for billing. SOA is the attempt to make these systems talk through a common switchboard. Each system wraps its capability in a service (“get customer”, “reserve stock”), and the switchboard (the ESB) knows who to call, how to translate one system’s message format into another’s, and in what sequence to call several services to complete a business process.

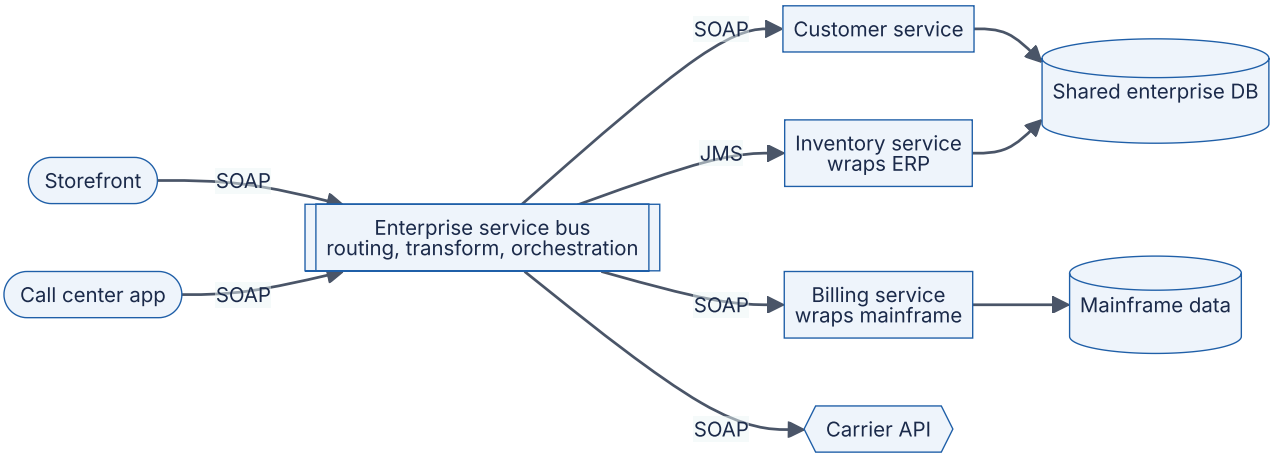
Why does this exist?

SOA answered an integration problem, not a scaling problem. Enterprises had dozens of incompatible applications and point-to-point integrations growing as $n(n-1)/2$; with 30 systems that is up to 435 links. A bus reduces each system to one connection. The second goal was **reuse**: build “customer service” once and have every application call it.

How it works internally

1. Each provider publishes a contract (WSDL) and, in the original vision, registers it in a directory (UDDI).
2. Consumers send SOAP messages to the ESB, not to providers.
3. The ESB performs **protocol mediation** (HTTP to message queue to mainframe), **transformation** (into a **canonical data model** shared by the enterprise), **routing** and often **orchestration** of multistep processes, sometimes expressed in BPEL.
4. Cross-cutting policies (security, reliable messaging, transactions) are standardized by the WS-* specifications.

Figure 33.5 — Folio integrated in the SOA style through an ESB.



Reading the diagram. Two consumer applications send SOAP messages to the ESB, drawn as a message channel because it is middleware. The bus translates each request into the canonical model and calls providers with whatever protocol they speak: SOAP for the customer service, a JMS queue for the ERP wrapper, SOAP for billing and the carrier. Calls are mostly synchronous request–response, though JMS legs may be asynchronous. Two services share an enterprise database, a common SOA pattern that couples them at the storage boundary. The ESB is the obvious failure point and bottleneck: every interaction passes through it, and every change to routing or transformation is made by the team that owns it. Scaling means scaling the bus product, usually vertically.

33.6.2 SOA versus microservices

Microservices share SOA's core idea of independently useful services with contracts, and Lewis and Fowler describe them partly by contrast: **"smart endpoints and dumb pipes"** instead of a smart bus, decentralized data instead of shared databases, and decentralized governance instead of an enterprise architecture board [Lewis and Fowler 2014]. Table 33.4 summarizes the contrast.

Table 33.4 — SOA (ESB era) compared with microservices.

Dimension	SOA (ESB era)	Microservices
Primary goal	Integrate existing enterprise applications; reuse	Independent change and deployment of one product's parts
Scope	Enterprise-wide	One application or product line
Communication	Smart pipe (ESB) with routing and transformation	Dumb pipes (HTTP, gRPC, a log or broker); logic in endpoints
Data	Often shared enterprise databases; canonical model	Database per service; each service owns its model
Contracts	SOAP, WSDL, WS-* standards	REST/JSON, gRPC/protobuf, event schemas
Governance	Central architecture board and ESB team	Team-owned services within platform guardrails
Granularity	Coarse (a whole ERP capability)	Smaller (one bounded context, Ch 34)
Typical failure	ESB becomes bottleneck and change queue	Distributed monolith; operational sprawl (§34.14)

WARNING / TRADE-OFF

The ESB recreates Conway's problem. When all routing and transformation logic lives in the bus, every cross-team change queues behind the bus team. The architecture's coupling point becomes an organizational bottleneck. Microservices moved that logic back into the services, at the price of each team handling more operations itself.

33.7 Microservices (overview)

33.7.1 Independently deployable services

DEFINITION

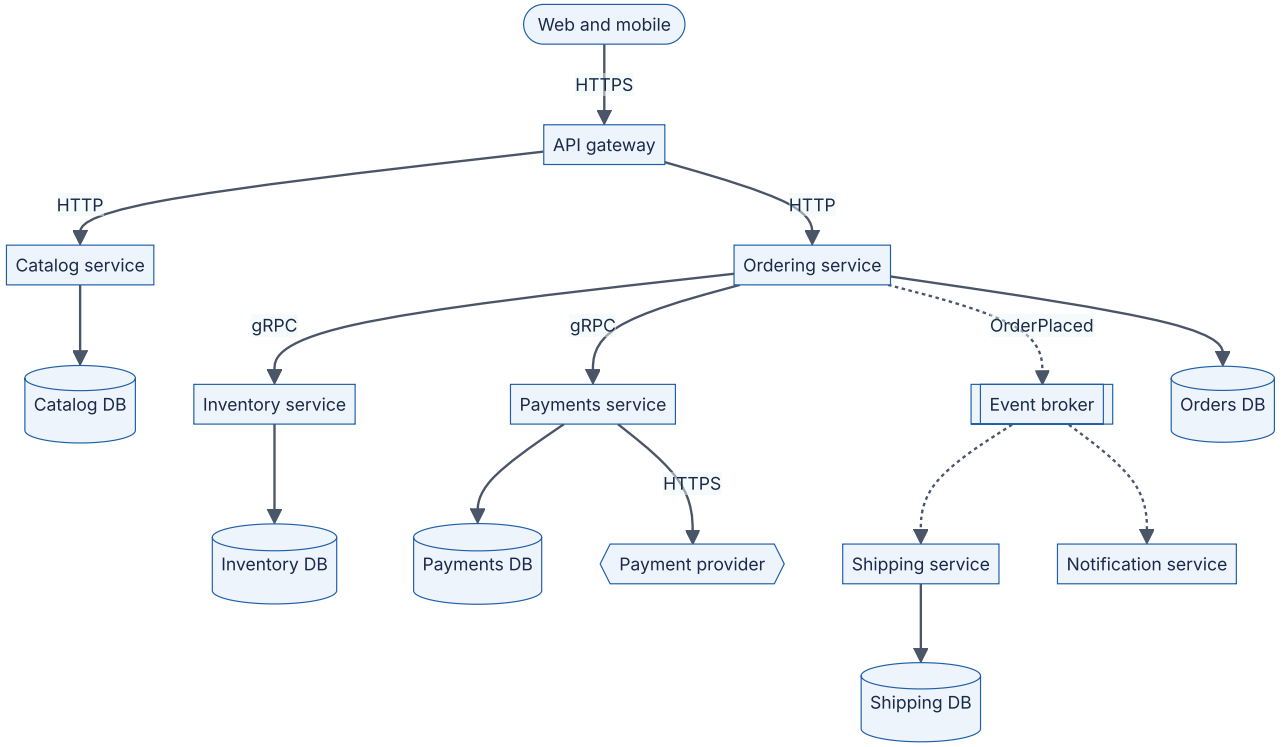
Definition — Microservices. An architectural style that builds one application as a suite of small services, each running in its own process, owning its own data, communicating over lightweight network mechanisms, built around a business capability, and **independently deployable** by the team that owns it [Lewis and Fowler 2014].

First-principles explanation and why it exists

Return to the building analogy: microservices give each department its own building with its own mailroom and its own front door. Departments renovate whenever they like, but every interaction becomes a letter rather than a walk down the corridor. The style exists

because, beyond a certain number of teams, the coordination cost of one deployable (§33.4.2) exceeds the cost of distribution. Removing deployment coupling, scaling hot services independently, isolating failures to one process and allowing different technologies per service are the benefits.

Figure 33.6 — Folio as microservices.



Reading the diagram. Clients call an API gateway over HTTPS (§34.6), which routes to catalog and ordering. Ordering calls inventory and payments synchronously over gRPC, so a checkout depends on all three being up; it also publishes an `OrderPlaced` event asynchronously to a broker, from which shipping and notifications consume at their own pace. Each service has its own database, so there are five storage boundaries and no shared tables; consistency between them is eventual or coordinated by a saga (§32.7). Failure points multiply: any network hop can time out, and the broker is shared infrastructure. Each service scales independently, so the catalog can run 40 replicas while payments runs 3.

The price in numbers

Distribution has a measurable cost. If checkout makes three sequential synchronous calls and each callee is available 99.9% of the time, the chain's availability is at most $0.999^3 \approx 0.997$ (§4.14.2), and each hop adds roughly 0.5–1 ms of network latency plus serialization, against nanoseconds for an in-process call. Data that was joined in SQL must now be fetched from two services or replicated by events. Each service needs a pipeline, dash-

boards, alerts and on-call ownership. Chapter 34 covers how to choose boundaries, communicate, own data and migrate; the comparison and decision framework are in §33.13.

COMMON MISTAKE

“Microservices are about small code size.” The defining property is independent deployability by an owning team, not lines of code. A tiny service that must be deployed in lockstep with three others is part of a **distributed monolith** (§34.14): all the costs of distribution and none of the benefits.

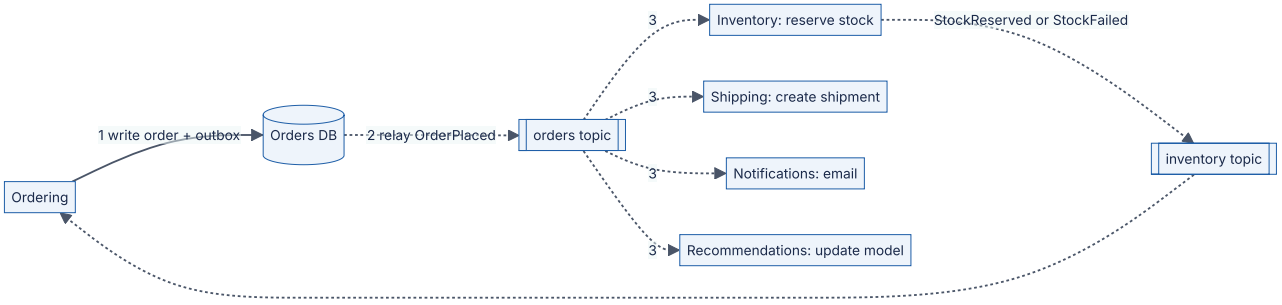
33.8 Event-driven architecture (overview)

DEFINITION

Definition — Event-driven architecture (EDA). A style in which components communicate mainly by producing and consuming **events**, immutable records of facts that happened (“OrderPlaced”), through a broker or log, so that producers do not know which consumers exist or when they will run.

Think of a newspaper: the publisher prints the news once and has no idea who reads it or when. Contrast a phone call, where the caller waits for the callee to answer. EDA exists to decouple components in **time** (the consumer may be down when the event is published) and in **knowledge** (adding a consumer does not change the producer). Richards and Ford distinguish two topologies [Richards and Ford 2020]: in the **broker topology**, components react to each other’s events with no central coordinator (choreography); in the **mediator topology**, an orchestrator receives an initiating event and directs the steps (orchestration). Chapter 32 compares the two in depth (§32.8).

Figure 33.7 — Folio’s order flow, event-driven with a broker topology.



Reading the diagram. Ordering writes the order and an outbox row in one local transaction (§32.5), then a relay publishes `OrderPlaced` to the orders topic; everything after step 1 is asynchronous (dotted arrows). Four consumers each react independently; recommendations was added last without changing ordering. Inventory reports success or

failure on its own topic, which ordering consumes to confirm or cancel the order, a choreographed saga (§32.7). Storage boundaries are each component's own store plus the broker's durable log (Chapter 31). Failure points: a slow consumer builds lag but does not slow ordering; a poison message needs a dead-letter queue (§30.11); duplicates require idempotent consumers (§32.10). The topic scales by partitions, consumers by consumer-group size.

EDA's advantages are loose coupling, natural buffering of load spikes, easy extension and an audit trail of facts. Its costs are eventual consistency (the customer may see "order received" before stock is confirmed), harder debugging because no single stack trace shows the whole flow (distributed tracing, §40.6, becomes necessary), and the need to reason about ordering, duplicates and schema evolution. EDA combines with any deployment style: a modular monolith can use in-process events (§33.5), and microservices commonly use a broker. Chapters 30–32 own the details.

33.9 Serverless architecture

33.9.1 Functions, managed services and scale to zero

DEFINITION

Definition — Serverless. An architecture built from **functions as a service (FaaS)**, short-lived functions that the platform runs in response to events, combined with **managed services** (databases, queues, object storage, authentication) that the provider operates, such that the team provisions no servers, capacity scales automatically including **to zero**, and billing is per request and per unit of execution time rather than per running machine [Jonas et al. 2019].

First-principles explanation

Owning servers is like owning a car: you pay for it whether it is parked or driving. Serverless is a taxi: you pay per trip, someone else maintains the vehicle, and at 2 a.m. when nobody travels you pay nothing. Taxis are cheaper for occasional trips and far more expensive for a daily 100 km commute, and you wait for one to arrive. The same three facts, per-use pricing, no maintenance and a wait, define serverless.

Why does this exist?

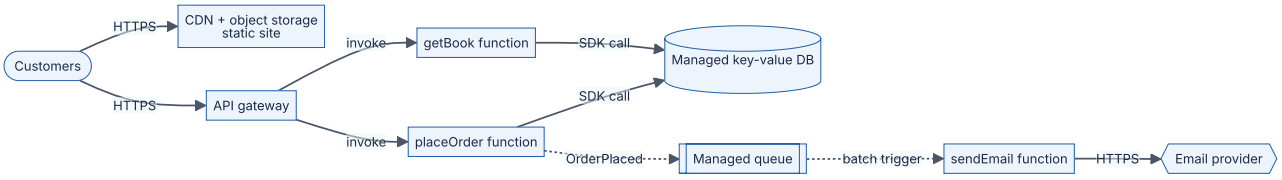
Most services are idle most of the time, yet a server must be sized for peak and kept running. FaaS removes capacity planning, patching and idle cost, and makes event wiring trivial: "run this function when a file lands in this bucket". The Berkeley view of server-

less argued that it does for cloud programming what high-level languages did for assembly, by hiding resource management [Jonas et al. 2019].

How it works internally

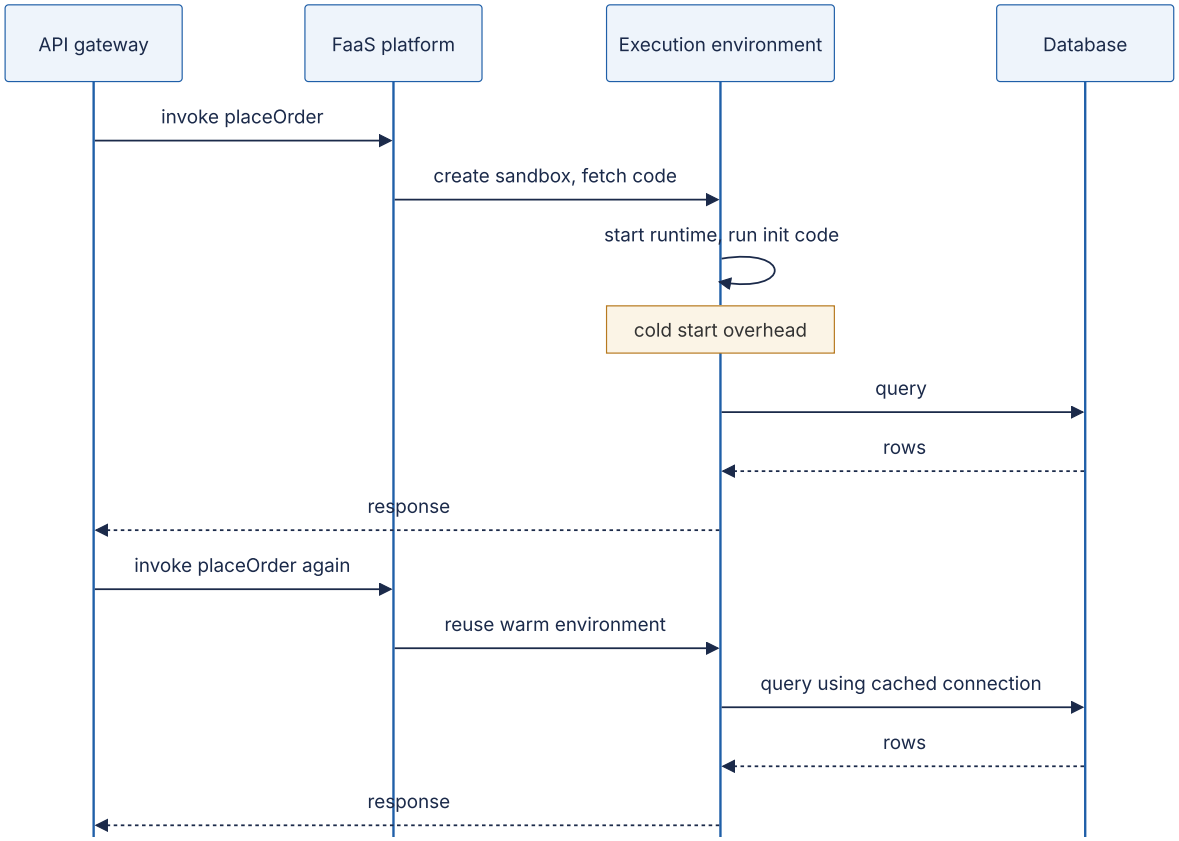
1. An event arrives: an HTTP request through an API gateway, a message on a queue, an object upload.
2. The platform looks for an idle, already initialized **execution environment** (a lightweight sandbox; AWS Lambda uses Firecracker microVMs, Chapter 48) for that function version.
3. If one exists, it runs the handler immediately: a **warm start**.
4. If none exists, the platform performs a **cold start**: allocate a sandbox, download the code package or image, start the language runtime, and run the function's initialization code (imports, SDK clients, database connections). Only then does the handler run.
5. Each environment processes one request at a time on AWS Lambda; concurrent requests need concurrent environments. By Little's Law (§4.11), concurrency = arrival rate × duration, so 200 RPS at 100 ms needs about $200 \times 0.1 = 20$ environments.
6. Environments idle for a while are reclaimed, so traffic that pauses produces cold starts again.

Figure 33.8 — Folio's order path built serverless.



Reading the diagram. Static pages come from a CDN backed by object storage. API calls go through a managed API gateway, which invokes functions synchronously; `placeOrder` and `getBook` call a managed key-value database through the provider's SDK. `placeOrder` also enqueues a message (dotted, asynchronous), and the queue triggers `sendEmail` in batches. Every box except the functions is a managed service; the storage boundaries are the database and the queue. Failure points: cold starts add latency on the synchronous path; account-level concurrency limits can throttle all functions at once; downstream systems that are not elastic (a relational database with a connection limit) can be overwhelmed when functions scale out. Scaling is automatic per request up to those limits.

Figure 33.9 — A cold start compared with a warm start.



Reading the diagram. The first invocation finds no idle environment, so the platform creates a sandbox, fetches code and starts the runtime; the function's initialization code then runs before the handler. All calls shown are synchronous. The second invocation, arriving while the environment is still alive, skips straight to the handler and reuses objects created during initialization, such as database connections. The failure point is the cold path: if initialization is slow or a dependency is unreachable, the first request in each new environment pays for it. Scaling out creates more environments, so a burst produces a burst of cold starts.

33.9.2 Cold starts

Cold-start time depends mostly on package size, runtime and initialization work. As of 2026, small functions in interpreted runtimes commonly cold-start in a few hundred milliseconds, while large JVM functions with heavy frameworks can take seconds; warm invocations add only milliseconds of platform overhead. Mitigations, in order of preference: keep packages small and initialization lazy; choose a fast-starting runtime for latency-sensitive paths; use **provisioned concurrency** (pre-initialized environments, billed while idle); use snapshot-based start (AWS Lambda SnapStart restores a snapshot of an initialized environment); and keep cold starts off the critical path by putting asynchronous work behind queues. "Keep-warm" pings are a weak workaround: they keep one environment warm, not the twenty a burst needs. Chapter 48 covers the internals.

33.9.3 The cost model and the break-even point

Mathematical foundations

FORMULA

Serverless versus always-on monthly cost

$$C_{\text{FaaS}} = R\,(c_{\text{req}} + M\,D\,c_{\text{GBs}}), \qquad C_{\text{srv}} = n\,c_{\text{inst}}, \qquad R^{*} = \frac{C_{\text{srv}}}{c_{\text{req}} + M\,D\,c_{\text{GBs}}}$$

Symbol	Meaning	Units
C_{FaaS}	Monthly cost of the functions	$/month
R	Invocations per month	requests/month
c_{req}	Per-request charges (function, plus gateway if used)	$/request
M	Memory allocated to the function	GB
D	Billed duration per invocation	s
c_{GBs}	Price per GB-second of execution	$/GB-s
C_{srv}	Monthly cost of always-on servers	$/month
n	Number of always-on instances (including redundancy)	instances
c_{inst}	Monthly price of one instance	$/instance-month
R^{*}	Break-even invocations per month	requests/month

WORKED EXAMPLE

Worked example — Folio's `getBook` API. Prices (AWS Lambda x86 list prices in us-east-1, as of 2026 [AWS Lambda Pricing]; verify for your region): $c_{\text{GBs}} = \$0.0000166667$, function requests \$0.20 per million so $c_{\text{req}} = \$2 \times 10^{-7}$. The function uses $M = 0.5$ GB and $D = 0.1$ s. The always-on alternative is $n = 2$ small instances (two for availability) at an illustrative \$60/month each, so $C_{\text{srv}} = \$120$. Assume two instances can carry the load in question. Free tiers are ignored.

1. GB-seconds per invocation: $0.5 \times 0.1 = 0.05$ GB-s.
2. Compute cost per invocation: $0.05 \times 0.0000166667 = \$8.33 \times 10^{-7}$.
3. Total per invocation: $8.33 \times 10^{-7} + 2.0 \times 10^{-7} = \$1.033 \times 10^{-6}$.
4. Break-even: $R^* = 120/(1.033 \times 10^{-6}) \approx 116$ million invocations per month.
5. As an average rate over a 30-day month (2.592×10^6 s): $116 \times 10^6 / 2.592 \times 10^6 \approx 45$ RPS.
6. Add an API gateway at an illustrative \$1.00 per million requests: per invocation $\$2.033 \times 10^{-6}$, so $R^* \approx 59$ million per month, about 23 RPS.
7. Check two points. At 10 RPS average (25.9 million/month) with the gateway: $25.9 \times 10^6 \times 2.033 \times 10^{-6} \approx \$53$, cheaper than \$120. At 200 RPS (518 million/month): about \$1,054, nearly nine times the servers' cost.

Interpretation. Below roughly 20–45 RPS of *average* load, serverless is cheaper; above it, always-on capacity wins, and the gap widens linearly. Spiky traffic favors serverless because servers must be sized for the peak while functions bill only the average. **Engineering implication.** Compute R^* with your real duration and memory, and remember the omitted terms on both sides: engineer time to operate servers (favoring serverless) and per-request charges of other managed services, data transfer and logging (often favoring servers at high volume). Re-run the numbers as traffic grows.

Real-world industry applications

INDUSTRY EXAMPLE

Prime Video's audio/video monitoring service. Amazon's Prime Video team described a stream-quality monitoring tool first built as distributed components orchestrated by AWS Step Functions with Lambda functions passing frames through object storage. At high volume, per-transition orchestration charges and storage round trips dominated cost and limited scale. The team moved the components into a single process running on containers, passing data in memory, and reported infrastructure cost falling by about 90% [Kolny 2023]. The lesson is the cost model above: a fine-grained serverless design suited a low-volume start, and a consolidated always-on design suited sustained high volume. *Evidence: Primary* — [Kolny 2023]

Advantages, disadvantages and failure scenarios

Serverless removes server operations, scales automatically, costs nothing when idle and suits event glue, spiky APIs, scheduled jobs and prototypes. Its disadvantages are cold-start latency, execution limits (for example, a 15-minute maximum duration on AWS Lambda, as of 2026), no local state between invocations, provider lock-in through event formats and managed services, harder local testing, and costs that exceed servers at sustained load.

Table 33.5 — Serverless failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
p99 latency spikes after quiet periods	Cold starts	Init-duration metric; p99 vs p50 gap	Provisioned concurrency on hot paths	Small packages; lazy init; async paths
Database connection exhaustion	Hundreds of environments each open connections	DB connection count at limit	Throttle function concurrency	Connection proxy; reserved concurrency
Throttling of unrelated functions	Shared account concurrency limit	Throttle metrics across functions	Raise or partition limits	Reserved concurrency per critical function
Runaway bill	Recursive trigger or retry loop	Cost and invocation anomaly alerts	Disable trigger	Guard against self-triggering; budgets
Duplicate side effects	At-least-once event delivery and retries	Duplicate emails or charges	Compensate	Idempotency keys (§22.15)

33.10 Hexagonal architecture (ports and adapters)

33.10.1 Ports, adapters and a technology-free core

DEFINITION

Definition — Hexagonal architecture (ports and adapters). A style, described by Alistair Cockburn, in which the application's core logic interacts with the outside world only through **ports**, interfaces defined by and in the language of the core, and each external technology (web framework, database, message broker, test harness) connects through an **adapter** that implements or calls a port [Cockburn 2005]. **Driving (primary) ports** are how the outside asks the application to do something; **driven (secondary) ports** are how the application asks the outside for something.

First-principles explanation

A laptop has a USB-C port. The laptop's designers defined what the port expects; they did not design your monitor, charger or keyboard. Each device connects through a cable or dongle that adapts it to the port. You can test the laptop with a cheap test fixture plugged into the same port, and you can switch monitor brands without opening the laptop. In software, the core is the laptop, interfaces such as `OrderRepository` are the ports, and `SqliteOrderRepository` or `InMemoryOrderRepository` are the dongles.

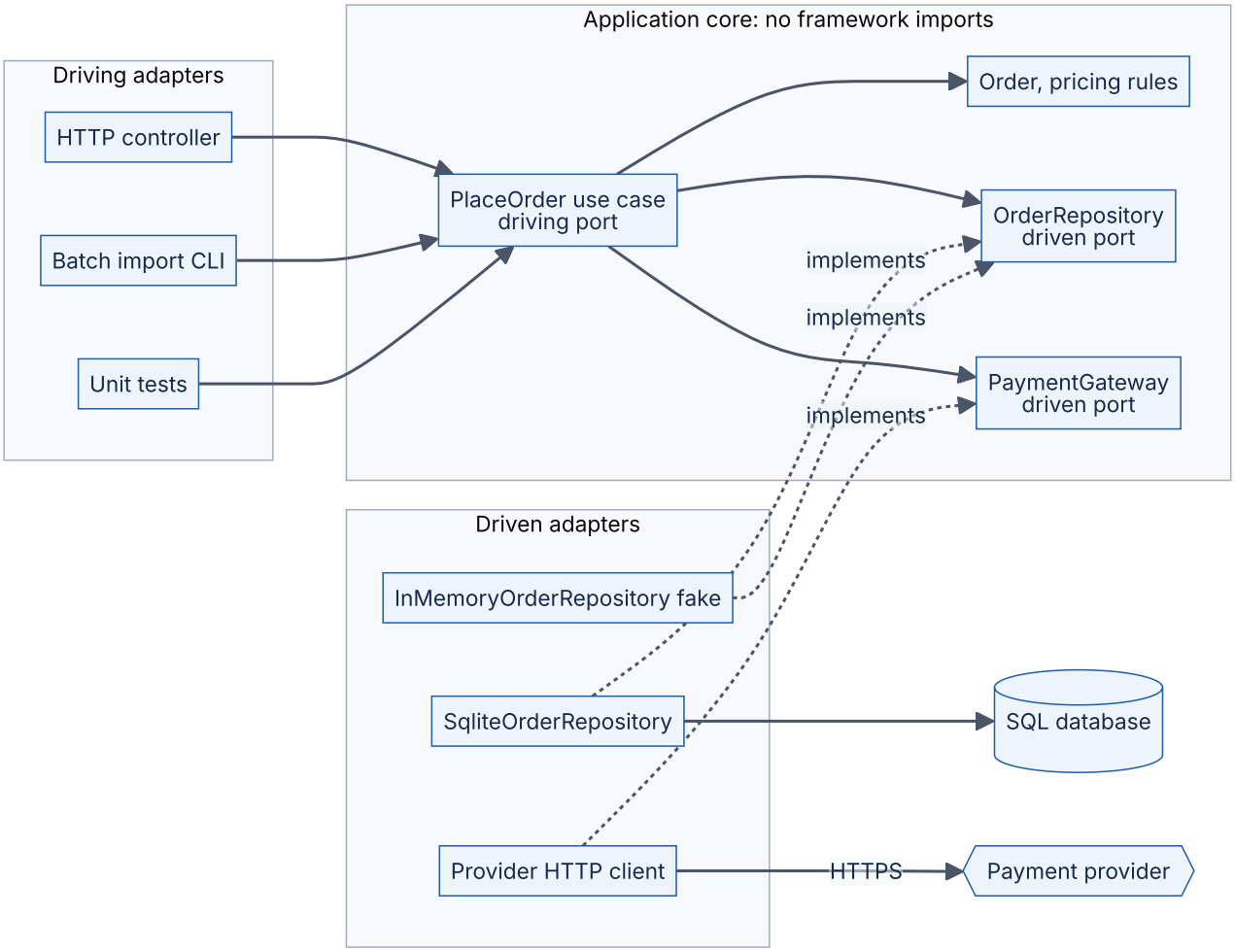
Why does this exist?

Cockburn's motivation was that business logic kept leaking into user-interface code and depending directly on databases, so it could not be tested automatically or driven by anything except the UI. His goal was an application that can be driven equally by users, programs, automated tests or batch scripts, and developed and tested in isolation from its eventual devices and databases [Cockburn 2005]. The hexagon shape has no meaning beyond leaving room to draw several ports; the point is that there is an inside and an outside instead of a top and a bottom. Unlike layering, where business logic depends downward on persistence, here persistence depends inward on an interface the core owns: this is the dependency inversion principle (§36.7) applied to a whole application.

How it works internally

1. The core defines use cases (driving ports) such as `PlaceOrder`, and the interfaces it needs (driven ports) such as `OrderRepository` and `PaymentGateway`. The core imports no framework or database library.
2. Driving adapters (an HTTP controller, a CLI command, a message consumer, a test) translate an external request into a call on a driving port.
3. Driven adapters (a SQL repository, a payment-provider client, a fake) implement driven ports.
4. A **composition root**, usually `main`, constructs the adapters and injects them into the core (dependency injection, §37.14).

Figure 33.10 — Folio's ordering core with ports and adapters.



Reading the diagram. On the left, three driving adapters call the same use case synchronously; the core does not know which one is calling. Inside the core, the use case applies domain rules and calls two driven ports, which are interfaces. On the right, adapters implement those ports; the dotted “implements” edges are source-code dependencies pointing *into* the core, not runtime calls. Only the SQL adapter crosses the storage boundary and only the provider client crosses the network to the payment provider, so those are the failure points; the core itself has none beyond its own logic. Scaling is unaffected: hexagonal architecture is a code structure that can live inside a monolith, a microservice or a function.

Code example

The program below shows the core with one driven port, two adapters (SQLite and an in-memory fake) and one contract test suite that runs against both. It uses only the standard library; run it with `python3`.

```
        """Folio checkout in ports-and-adapters style (single file for teaching)."""
        import sqlite3
        import unittest
        from dataclasses import dataclass
        from typing import Optional, Protocol


        # ---------- Domain core: no imports from frameworks or databases ----------
        @dataclass(frozen=True)
        class Order:
            order_id: str
            isbn: str
            quantity: int
            total_cents: int


        class OutOfStock(Exception):
            pass


        class OrderRepository(Protocol):          # a driven (secondary) port
            def stock_of(self, isbn: str) -> int: ...
            def price_of(self, isbn: str) -> int: ...
            def save(self, order: Order) -> None: ...   # must also decrement stock
            def find(self, order_id: str) -> Optional[Order]: ...


        class PlaceOrder:                          # a use case; its method is the driv-
ing port
            def __init__(self, repo: OrderRepository) -> None:
                self.repo = repo

            def __call__(self, order_id: str, isbn: str, quantity: int) -> Order:
                if quantity <= 0:
                    raise ValueError("quantity must be positive")
                if self.repo.stock_of(isbn) < quantity:
                    raise OutOfStock(isbn)
                order = Order(order_id, isbn, quantity, self.repo.price_of(isbn) * quan-
tity)
                self.repo.save(order)
                return order


        # ---------- Adapter 1: SQLite (production-like) ----------
        class SqliteOrderRepository:
            def __init__(self, conn: sqlite3.Connection) -> None:
                self.conn = conn
                conn.execute("CREATE TABLE IF NOT EXISTS books"
                             "(isbn TEXT PRIMARY KEY, price_cents INT, stock INT)")
                conn.execute("CREATE TABLE IF NOT EXISTS orders"
                             "(id TEXT PRIMARY KEY, isbn TEXT, qty INT, total INT)")

            def add_book(self, isbn: str, price_cents: int, stock: int) -> None:
                self.conn.execute("INSERT INTO books VALUES (?, ?, ?)", (isbn,
price_cents, stock))
```

```
            def stock_of(self, isbn: str) -> int:
                row = self.conn.execute("SELECT stock FROM books WHERE isbn=?",
(isbn,)).fetchone()
                return row[0] if row else 0

            def price_of(self, isbn: str) -> int:
                return self.conn.execute("SELECT price_cents FROM books WHERE isbn=?",
                                         (isbn,)).fetchone()[0]

            def save(self, order: Order) -> None:
                with self.conn:                     # one ACID transaction
                    cur = self.conn.execute(
                        "UPDATE books SET stock = stock - ? WHERE isbn=? AND stock >=
?",
                        (order.quantity, order.isbn, order.quantity))
                    if cur.rowcount != 1:
                        raise OutOfStock(order.isbn)
                    self.conn.execute("INSERT INTO orders VALUES (?, ?, ?, ?)",
                                      (order.order_id, order.isbn, order.quantity,
order.total_cents))

            def find(self, order_id: str) -> Optional[Order]:
                row = self.conn.execute("SELECT id, isbn, qty, total FROM orders WHERE
id=?",
                                        (order_id,)).fetchone()
                return Order(*row) if row else None


        # ---------- Adapter 2: in-memory fake (for tests and local runs) ----------
        class InMemoryOrderRepository:
            def __init__(self) -> None:
                self.books: dict[str, list[int]] = {}   # isbn -> [price_cents, stock]
                self.orders: dict[str, Order] = {}

            def add_book(self, isbn: str, price_cents: int, stock: int) -> None:
                self.books[isbn] = [price_cents, stock]

            def stock_of(self, isbn: str) -> int:
                return self.books.get(isbn, [0, 0])[1]

            def price_of(self, isbn: str) -> int:
                return self.books[isbn][0]

            def save(self, order: Order) -> None:
                if self.books[order.isbn][1] < order.quantity:
                    raise OutOfStock(order.isbn)
                self.books[order.isbn][1] -= order.quantity
                self.orders[order.order_id] = order

            def find(self, order_id: str) -> Optional[Order]:
                return self.orders.get(order_id)


        # ---------- Tests: the same contract runs against both adapters ----------
        class PlaceOrderContract:
            def make_repo(self):
                raise NotImplementedError
```

```
        def setUp(self):
            self.repo = self.make_repo()
            self.repo.add_book("978-0", price_cents=2500, stock=3)
            self.place = PlaceOrder(self.repo)

        def test_places_order_and_decrements_stock(self):
            order = self.place("o-1", "978-0", 2)
            self.assertEqual(order.total_cents, 5000)
            self.assertEqual(self.repo.stock_of("978-0"), 1)
            self.assertEqual(self.repo.find("o-1"), order)

        def test_rejects_when_out_of_stock(self):
            with self.assertRaises(OutOfStock):
                self.place("o-2", "978-0", 4)
            self.assertIsNone(self.repo.find("o-2"))


    class TestWithFake(PlaceOrderContract, unittest.TestCase):
        def make_repo(self):
            return InMemoryOrderRepository()


    class TestWithSqlite(PlaceOrderContract, unittest.TestCase):
        def make_repo(self):
            return SqliteOrderRepository(sqlite3.connect(":memory:"))


    if __name__ == "__main__":
        unittest.main(verbosity=2)
```

Running it prints four passing tests: two against the fake, two against SQLite.

What to notice:

- `PlaceOrder` imports nothing from `sqlite3`; it depends only on the `OrderRepository` protocol. The port is defined by what the core needs, in the core's vocabulary (`stock_of`, `save`), not by what SQL offers.
- The two adapters differ completely inside, yet both satisfy the same port. The SQLite adapter uses a conditional `UPDATE ... WHERE stock >= ?` inside a transaction, so a concurrent order cannot oversell; the fake checks in memory.
- The **contract test** (`PlaceOrderContract`) is written once and run against each adapter. This is what makes a fake trustworthy: if the fake ever behaves differently from the real adapter, the SQLite run of the same test catches it.
- Tests of the use case through the fake need no database and run in milliseconds, so business rules can be tested exhaustively; the slower SQLite run is reserved for the adapter contract.

Advantages, disadvantages and misconceptions

Hexagonal architecture makes the core testable, keeps technology decisions reversible, and lets new delivery mechanisms (a queue consumer next to an HTTP API) reuse the same use cases. Its costs are indirection and mapping code; for a thin CRUD service whose "logic" is the SQL itself, ports add ceremony without benefit, the sinkhole of §33.3.2 in another form.

COMMON MISTAKE

"Every class needs an interface." Ports are needed at the boundary to things that are slow, nondeterministic, external or likely to change: databases, networks, clocks, randomness. Domain classes inside the core call each other directly. Wrapping everything in interfaces adds cost and hides the real boundaries.

33.11 Clean architecture

33.11.1 The dependency rule

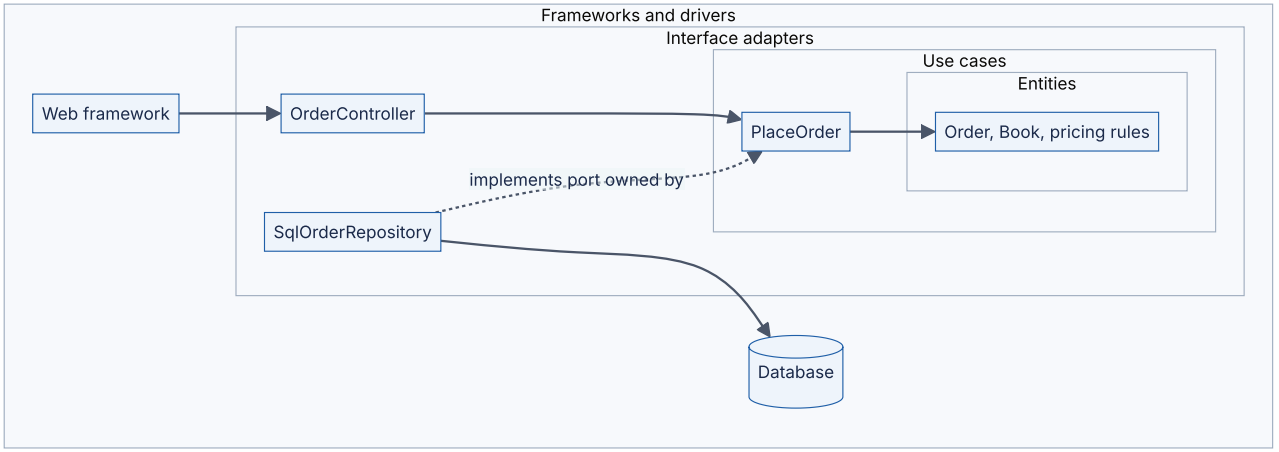
DEFINITION

Definition — Clean architecture. Robert C. Martin's synthesis of hexagonal, onion and similar architectures, drawn as concentric circles: **entities** (enterprise business rules) at the center, then **use cases** (application business rules), then **interface adapters** (controllers, presenters, gateways), then **frameworks and drivers** (web, database, devices) at the edge, governed by one **dependency rule**: source-code dependencies point only inward [Martin 2017].

First-principles explanation and why it exists

Imagine the policies of a bank written down in a rulebook. The rulebook does not mention which brand of teller software the branches use; the teller software is written to follow the rulebook. If the software changes, the rulebook does not. Martin's argument is that the most stable, most valuable code (business rules) should not depend on the most volatile code (frameworks, UI, database drivers), so the details plug into the policy, never the reverse. When data must flow outward, for example a use case presenting results, the inner circle defines an interface (an output port) that the outer circle implements: control flows outward while the source-code dependency still points inward.

Figure 33.11 — Clean architecture's circles for Folio, with the dependency rule.



Reading the diagram. Nested boxes are the circles: entities innermost, then use cases, interface adapters and frameworks. Every solid arrow is a synchronous call that also points inward as a source dependency: the web framework calls the controller, the controller calls the use case, and the use case uses entities. The repository is the interesting edge: at run time the use case calls it, but in source code the repository implements a port owned by the use-case circle (dotted arrow), so the dependency still points inward. The database is the only storage boundary and sits at the outermost circle with other details. Failure points and scaling are those of whatever deployment hosts the code.

33.11.2 Onion, hexagonal and clean compared

Jeffrey Palermo's **onion architecture** (2008) makes the same move with different labels: the domain model at the center, then domain services, then application services, with UI, infrastructure and tests in the outer ring, and all coupling toward the center [Palermo 2008]. Table 33.6 relates the three.

Table 33.6 — Hexagonal, onion and clean architecture.

Aspect	Hexagonal (2005)	Onion (2008)	Clean (2012 blog, 2017 book)
Central metaphor	Inside versus outside, ports	Concentric rings	Concentric circles
Prescribes inner structure	No; "the application"	Domain model, domain services, application services	Entities, use cases
Key rule	Core talks to outside only via ports	Coupling toward the center	Dependencies point inward only
Emphasis	Symmetry of driving and driven sides; testability	Domain model at center; infrastructure outermost	Policy versus detail; use cases as first-class

All three are applications of dependency inversion around a technology-free core; they differ in vocabulary and in how much inner layering they prescribe. In practice, the deci-

sions that matter are the same: which interfaces the core owns, and where the composition root lives.

WARNING / TRADE-OFF

Ceremony is a real cost. Clean architecture applied literally to a small CRUD service produces request models, response models, presenters and mappers for data that passes through unchanged. Use the full structure where business rules are rich and long-lived; use a thin hexagonal boundary, or plain layering, where they are not.

33.12 Cell-based architecture

33.12.1 Cells, the cell router and blast radius

DEFINITION

Definition — Cell-based architecture. A style in which a workload is deployed as multiple independent, complete copies called **cells**, each with its own compute and data and each serving a fixed subset of customers (or other partition key), placed behind a thin **cell router** that sends each request to its customer's cell. Cells share nothing at run time, so a failure, bad deployment or poison request affects only one cell. The fraction of customers affected by a failure is its **blast radius** [AWS 2023].

First-principles explanation

A ship is divided into watertight compartments so that a hole floods one compartment rather than sinking the vessel. Bulkheads (§22.14) apply that idea to thread pools inside one service. Cells apply it to the entire stack: load balancer, services, caches, queues and databases are all duplicated per compartment. Every architecture so far, even microservices, has shared components whose failure affects every customer: the one orders database, the one broker. Cells remove that sharing.

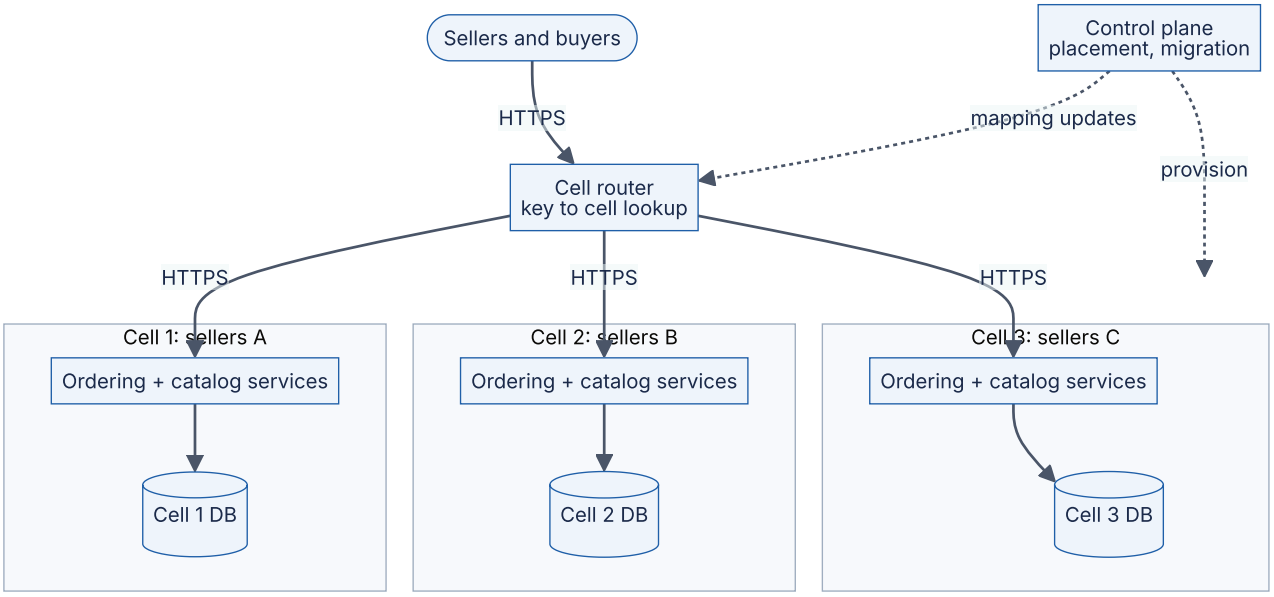
Why does this exist?

At large scale the dominant causes of outages are not hardware failures, which redundancy handles, but correlated failures: a bad configuration push, a software bug triggered by one customer's unusual request, a capacity overload. Redundant replicas of the same code share the same bug, and a load balancer will happily send a crashing "poison pill" request to every replica in turn until all are down. Cells limit such events to one partition of customers, and they bound the size of each copy, so a cell's maximum load can be tested in advance instead of being discovered in production [AWS 2023].

How it works internally

1. **Choose a partition key**: usually the customer or tenant ID; for Folio's marketplace, the seller account.
2. **Size cells** to a fixed maximum (customers, requests per second, data volume) that you have load-tested. Grow by adding cells, not by growing cells.
3. **Route**: the cell router looks up the key in a mapping table (cached aggressively) and forwards the request. The router must be the simplest, most reliable component in the system; it contains no business logic.
4. **Control plane**: a separate service places new customers into cells, provisions new cells and migrates customers between cells. Its failure must not stop the data plane from serving.
5. **Deploy in waves**: release to one cell, watch, then progressively more. The first wave's exposure equals one cell's blast radius.
6. **Handle what cannot be partitioned** (global search, cross-customer analytics) with separate, asynchronously fed systems.

Figure 33.12 — Folio's seller platform as cells behind a cell router.



Reading the diagram. The router receives every request over HTTPS, maps the seller ID to a cell and forwards the request synchronously; it holds no other logic. Each cell is a complete stack with its own services and its own database, so there are as many storage boundaries as cells and no shared data path between them. The control plane updates the router's mapping and provisions cells asynchronously (dotted); if it fails, routing continues from the cached mapping, a property called **static stability** [Weiss and Furr 2020]. Failure points: a fault inside a cell stays inside it; the router is the one shared component and must be kept minimal and redundant. Scaling is by adding cells; a single customer larger than a cell is the main limit.

Mathematical foundations

FORMULA

Blast radius of a cell failure

$$B = \frac{1}{N_c}, \qquad N_c = \lceil \frac{U}{u_{\max}} \rceil$$

Symbol	Meaning	Units
B	Fraction of customers affected when one cell fails (customers evenly spread)	fraction
N_c	Number of cells	cells
U	Total customers	customers
$u_{\max}$	Tested maximum customers per cell	customers/cell

WORKED EXAMPLE

Worked example — Sizing Folio's seller cells. Folio has $U = 1,000,000$ sellers and has load-tested one cell up to $u_{\max} = 50,000$ sellers.

1. $N_c = \lceil 1,000,000/50,000 \rceil = 20$ cells.
2. $B = 1/20 = 5\%$: a poison-pill request or bad deploy that crashes one cell affects 50,000 sellers instead of 1,000,000.
3. Deployment waves of 1, 2, 4, 8 and 5 cells expose 5%, then 10%, 20%, 40% and 25% of sellers at each stage; a defect caught in wave 1 touches 5%.
4. Cost check: if each cell carries a fixed overhead of, say, 3 instances for redundancy and control functions, 20 cells carry 60 instances of overhead against perhaps 2 or 3 for an unpartitioned design.

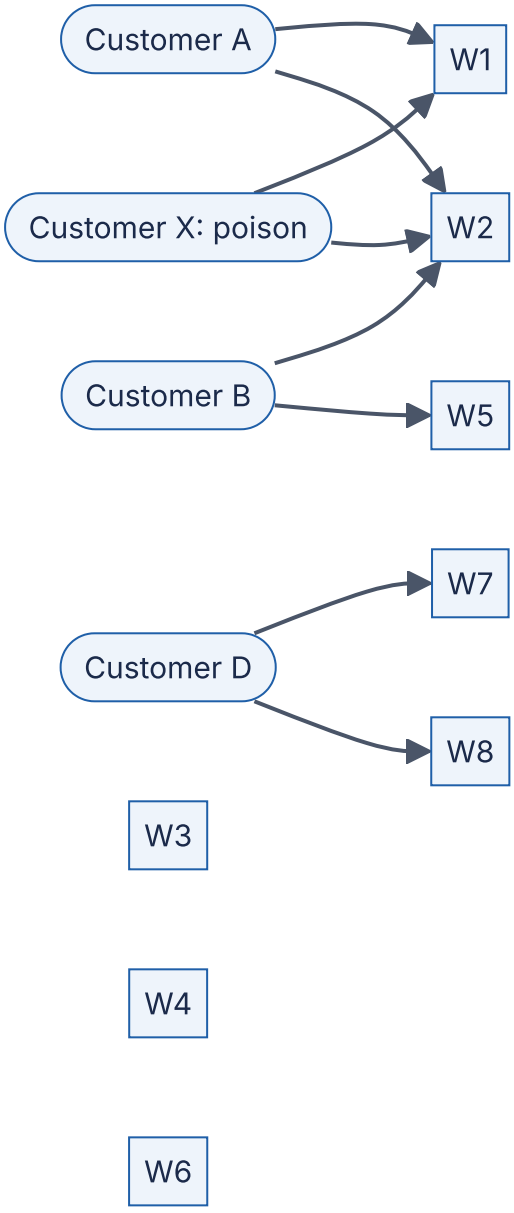
Interpretation. Blast radius falls as $1/N_c$, but per-cell fixed cost rises linearly in N_c. **Engineering implication.** Choose $u_{\max}$ from load tests and business tolerance for a partial outage, not as small as possible; make each cell large enough that its fixed overhead is a small fraction of its useful capacity.

33.12.2 Shuffle sharding

Cells give each customer exactly one compartment. **Shuffle sharding** does better for stateless fleets: assign each customer a random *combination* of k workers out of N, so that two customers rarely share the same full set [MacCárthaigh 2019]. If one customer's requests crash workers, only that customer's k workers are hit, and another customer is

fully cut off only if its entire combination is contained in the failed set; any customer with at least one healthy worker continues, provided clients retry on another worker in their shard.

Figure 33.13 — Shuffle sharding: eight workers, two per customer.



Reading the diagram. Eight stateless workers serve all customers; each customer's requests may go only to its two assigned workers, chosen at random when the customer is onboarded and resolved by the router. Calls are synchronous. Customer X sends requests that crash workers, taking out W1 and W2. Customer A, whose shard is exactly {W1, W2}, loses service. Customer B shares only W2, so its retries succeed on W5. Customer D shares nothing and is unaffected. There is no storage boundary here (workers are stateless); the failure point is the overlap between shards, and capacity scales by adding workers, which also increases the number of possible combinations.

FORMULA

Shuffle-sharding overlap

$$S = \binom{N}{k}, \qquad P(\text{overlap} = j) = \frac{\binom{k}{j}\binom{N-k}{k-j}}{\binom{N}{k}}, \qquad P(\text{full overlap}) = \frac{1}{\binom{N}{k}}$$

Symbol	Meaning	Units
N	Workers in the fleet	workers
k	Shard size: workers assigned to each customer	workers
S	Number of distinct shards (combinations)	count
j	Workers that another customer shares with the failed shard	workers
$P(\text{overlap} = j)$	Probability a random other customer shares exactly j workers	probability

WORKED EXAMPLE

Worked example — Isolation from a poison customer.

Small fleet, $N = 8$, $k = 2$.

1. $S = \binom{8}{2} = 28$ combinations.
2. Full overlap (all of another customer's workers fail): $1/28 \approx 3.6\%$ of customers. Compare a design without sharding, where all workers serve everyone and the poison request eventually crashes all eight: 100%.
3. No overlap: $\binom{6}{2}/28 = 15/28 \approx 53.6\%$ unaffected; the remaining $12/28 \approx 42.9\%$ share one worker and survive by retrying on their other worker.

Larger fleet, $N = 100$, $k = 5$, *10,000 customers.*

1. $S = \binom{100}{5} = 75,287,520$.
2. Full overlap: $1/S \approx 1.3 \times 10^{-8}$; expected fully affected customers among the other 9,999: $9,999 \times 1.3 \times 10^{-8} \approx 0.0001$.
3. Overlap of 3 or more workers (at most 2 of 5 left): $P(3) + P(4) + P(5) = 5.93 \times 10^{-4} + 6.3 \times 10^{-6} + 1.3 \times 10^{-8} \approx 6.0 \times 10^{-4}$, about 6 customers, who keep service at reduced capacity.
4. $P(0) = \binom{95}{5}/\binom{100}{5} \approx 77.0\%$ share no worker at all.

Interpretation. With the same 100 workers, 20 cells of 5 would put each customer in one of 20 compartments, so a poison customer fully affects 5% of the others; shuffle sharding makes full overlap a one-in-75-million event. **Engineering implication.** Shuffle sharding needs clients or routers that retry across the shard and workers that are interchangeable; for stateful partitions holding a customer's data, use cells, and combine the two by shuffle sharding within or across cells.

Real-world industry applications

INDUSTRY EXAMPLE

AWS cells and shuffle sharding. AWS's Well-Architected guidance describes cell-based architecture, including cell routers, cell sizing, placement and migration, as a way to reduce the scope of impact of failures [AWS 2023], and its Builders' Library describes shuffle sharding with the eight-worker, two-per-customer illustration used above [MacCárthaigh 2019]. Route 53 is described as using shuffle sharding to assign name servers to hosted zones. *Evidence: Primary* — [AWS 2023], [MacCárthaigh 2019]

Table 33.7 — Cell-based architecture failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Router outage takes down all cells	Router is the one shared component	Global error rate, all cells healthy	Fail over router; serve from cached map	Minimal router logic; static stability
Hot customer outgrows its cell	Uneven customer sizes	Cell nearing tested limits	Migrate customer to a larger or dedicated cell	Placement by size; per-customer quotas
Same bug in every cell	Simultaneous global deploy	Errors rise in all cells together	Roll back	Wave deployments, one cell first
Cross-cell dependency appears	Shared service added “temporarily”	Dependency map shows shared component	Duplicate it per cell	Review rule: cells share nothing in the data plane

Chapter 44 extends cells across regions, and Chapter 47 treats tenant placement and isolation levels.

33.13 Monolith versus microservices

33.13.1 Side-by-side comparison

Table 33.8 compares the three deployment styles that most teams actually choose between. Serverless and cells are orthogonal: a modular monolith can run in cells, and microservices can be built from functions.

Table 33.8 — Monolith, modular monolith and microservices.

Row	Monolith	Modular monolith	Microservices
What it is	One deployable, unstructured or layered inside	One deployable, domain modules with enforced boundaries	Many independently deployable services
How it works	In-process calls, one shared database	In-process calls through module APIs; schema per module	Network calls and events; database per service
Advantages	Simplest to build, debug and deploy; ACID everywhere	Monolith's simplicity plus clear ownership and cheap extraction	Independent deploys, scaling, failure isolation, technology choice
Disadvantages	Coupling grows with team count; erosion	Deployment and scaling still coupled	Network failures, distributed data, sagas, operational load
Performance	Nanosecond calls; joins in SQL	Same as monolith	~0.5–1 ms per hop; fan-out tail latency (§4.8.4)
Operational complexity	Low: one pipeline, one service	Low, plus boundary tooling	High: per-service pipelines, tracing, discovery, on-call
Cost	Lowest at small scale	Low	Higher baseline (platform, per-service overhead)
Use cases	Early products; small teams; stable small domains	Most products up to several teams	Many teams; heterogeneous scaling or isolation needs
Choose when	One team; domain still moving	2–8 teams sharing one release; boundaries known	Release coordination is the bottleneck and a platform exists

33.13.2 Conway's law and the cost of coordination

Conway observed that organizations which design systems are constrained to produce designs that copy their communication structures [Conway 1968]. The practical reading is that architecture and team structure must match: if Ordering and Inventory are separate services owned by one team, they will be changed together and drift into a distributed monolith; if one module is owned by three teams, it will grow three internal sub-architectures. The **inverse Conway maneuver** designs team boundaries first, so that the desired architecture is the one the organization naturally produces [Skelton and Pais 2019].

Mathematical foundations

FORMULA

Communication paths among teams

$$L_{\text{all}} = \frac{T(T-1)}{2}, \qquad L_{\text{declared}} = \frac{T\,\bar{d}}{2}$$

Symbol	Meaning	Units
L_{all}	Pairwise coordination paths when every team may need every other	paths
T	Number of teams	teams
L_{declared}	Paths when teams coordinate only along declared module or service dependencies	paths
$\bar{d}$	Average number of neighbor teams per team in the dependency graph	teams

WORKED EXAMPLE

Worked example — Folio at 3 and 12 teams.

1. At $T = 3$: $L_{\text{all}} = 3 \times 2/2 = 3$ paths. Everyone can attend one planning meeting; a monolith is fine.
2. At $T = 12$ with one shared code base and release: $L_{\text{all}} = 12 \times 11/2 = 66$ paths.
3. At $T = 12$ with enforced boundaries where each team depends on about $\bar{d} = 2$ others: $L_{\text{declared}} = 12 \times 2/2 = 12$ paths, an 82% reduction.

Interpretation. Coordination grows quadratically with team count unless the architecture limits who must talk to whom. **Engineering implication.** Module boundaries (§33.5) already cut L from 66 to 12 for code changes; independent deployment (§33.7) additionally removes the release-train coupling of §33.4.2. Split deployments when that second effect is what is hurting.

33.13.3 A decision framework

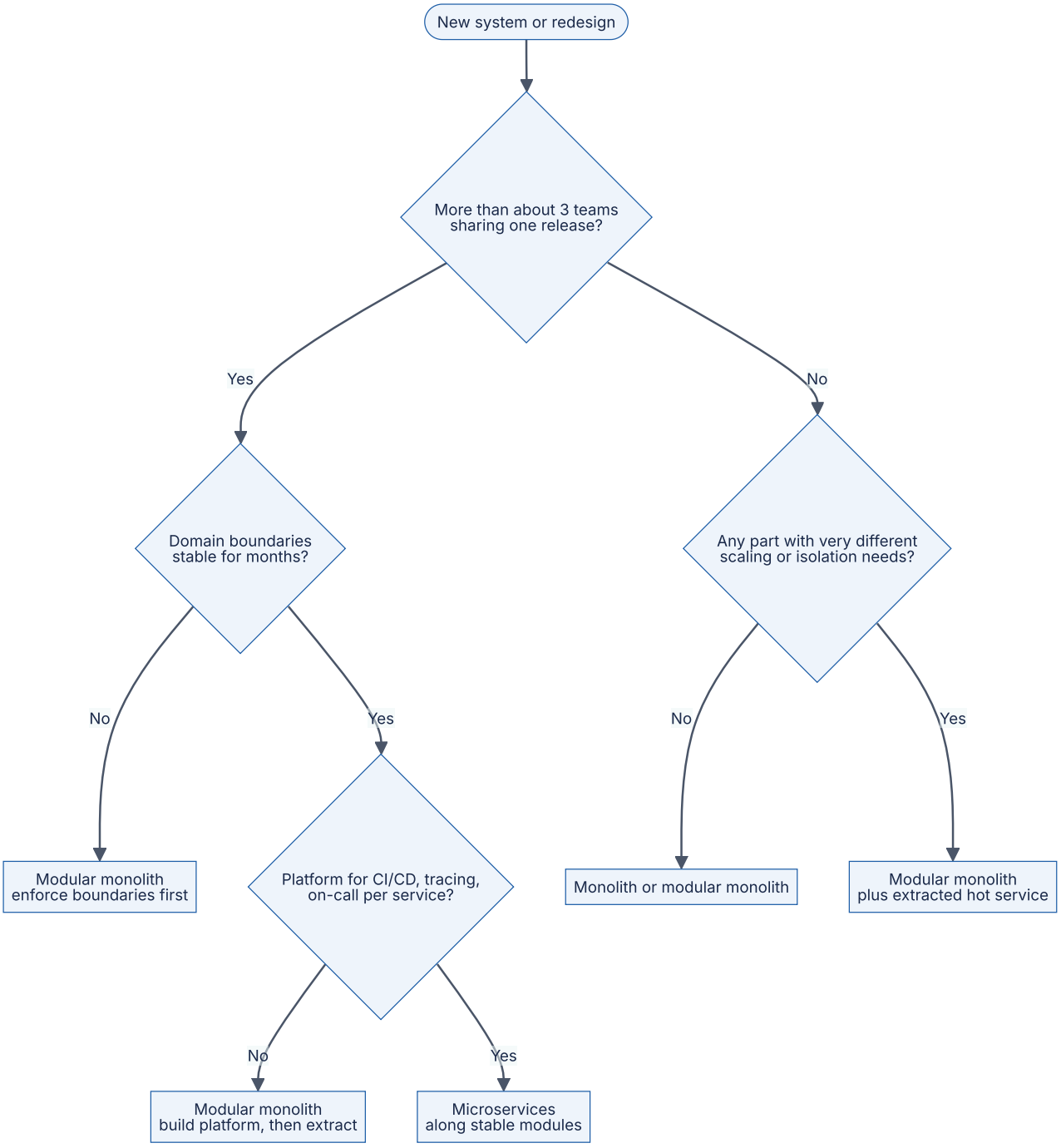
Use four questions, in order. Each has a measurable signal.

1. **Team size and structure.** One or two teams: monolith or modular monolith. More teams than can share one release without regular blocking (from §33.4.2, release failure rates above roughly 20–30%, or queues to merge): consider independent deployment for the teams most blocked.

2. **Deployment frequency.** If teams want to deploy several times a day and the shared pipeline or test suite takes hours, independent deployables help. If the business releases monthly, they do not.
3. **Domain maturity.** If boundaries are still moving (a new product, frequent changes that touch many modules), keep them in-process where moving a boundary is a refactor. Extract only modules whose interfaces have been stable for months [Fowler 2015; Newman 2019].
4. **Heterogeneous runtime needs.** If one part needs very different scaling, technology, security isolation or availability (Folio's search indexing, payment-card handling), extract *that part* first.

Fowler's "MonolithFirst" and Tilkov's counterargument "Don't start with a monolith" (both 2015) disagree mainly on question 3: Tilkov argues that when the domain is well understood, for example a rewrite of an existing system, starting with separate services avoids a costly later split [Fowler 2015; Tilkov 2015].

Figure 33.14 — Choosing a deployment style.



Reading the diagram. Decision diamonds are evaluated top to bottom; each answer leads to the next question or to a recommendation. The first split is organizational: few teams lead to a monolith or modular monolith, with a hybrid when one component has distinct runtime needs (as QuickBite's location service did in §2.8.1). Many teams lead to microservices only when boundaries are stable and a platform exists; otherwise the answer is still a modular monolith, with extraction deferred. There are no runtime components, protocols or storage boundaries here; the failure point the chart guards against is the premature split, and "scaling" means revisiting the chart as teams and traffic grow.

Real-world industry applications

INDUSTRY EXAMPLE

Segment's return to a monolith. Segment described splitting its event-delivery destinations into well over a hundred separate services, one per destination, each with its own queue. Shared-library versions drifted across services, operational load grew with each new destination, and a small team could not keep them all healthy. Segment consolidated the destinations into a single service with a shared test suite and reported large gains in developer productivity [Noonan 2018]. The case illustrates the decision framework: service count far exceeded team count, so the coordination microservices remove was not the problem they had. *Evidence: Primary* — [Noonan 2018]

RECOMMENDATION

Default path for a new product. Start with a modular monolith with enforced boundaries and per-module schemas. Instrument release failure rate, lead time and cross-module change frequency. Extract a module into a service when a measured signal (blocked releases, a scaling or isolation need) justifies the operational cost, and use the strangler fig pattern for the extraction (§34.13).

COMMON MISTAKE

"Microservices are what large companies use, so they are more scalable." Request throughput scales by replication in every style. Microservices scale *organizations*, by letting many teams deploy independently, and permit per-component scaling. A 10-engineer team adopting them pays the costs without the organizational benefit.

33.14 Summary and revision notes

SUMMARY

- **Layered**: horizontal technical layers; layers are logical, tiers physical. Strict layering isolates, relaxed layering opens chosen layers; most requests passing through without logic is the **sinkhole** anti-pattern. Layering is technical partitioning, so one feature touches every layer.
- **Monolith**: one deployable. Real benefits: in-process calls, ACID transactions, easy refactoring, simple operations. Main cost is **deployment coupling**: $P_{\text{clean}} = (1-p)^{Tc}$ falls exponentially with team count.
- **Modular monolith**: domain modules, public `api`, schema per module, acyclic declared dependencies, enforced in CI. Keeps the monolith's simplicity; makes extraction cheap.
- **SOA**: enterprise integration through a smart ESB and canonical model; microservices invert it to smart endpoints, dumb pipes and database per service.
- **Microservices**: independently deployable, team-owned services; pay in network hops, distributed data and operations (Ch 34).
- **Event-driven**: producers emit facts; consumers react asynchronously; broker versus mediator topology; eventual consistency (Ch 30–32).
- **Serverless**: FaaS plus managed services; scale to zero; cold starts; concurrency = rate × duration. Break-even $R^{*} = C_{\text{srv}}/(c_{\text{req}} + MDc_{\text{GBs}})$, about 23–45 RPS in the worked example.
- **Hexagonal**: the core owns ports; adapters implement them; test the core with fakes validated by contract tests. **Clean**: dependencies point inward only; onion, hexagonal and clean are variants of dependency inversion.
- **Cells**: complete independent stacks per customer partition behind a thin router; blast radius $1/N_c$; deploy in waves. **Shuffle sharding**: full-overlap probability $1/\binom{N}{k}$.
- **Choosing**: Conway's law; coordination paths $T(T-1)/2$; default to a modular monolith and extract on measured signals.

33.15 Knowledge checkpoint

1. What is the difference between a layer and a tier?
2. Define the architecture sinkhole anti-pattern and give one remedy.
3. Why does layered architecture make adding a new product type expensive?
4. A release bundles 30 changes, each with a 2% chance of a blocking defect. What is the probability the release is clean?
5. Name the two rules a modular-monolith boundary checker enforces.
6. What does "smart endpoints and dumb pipes" contrast with?

7. A function receives 300 RPS and runs for 200 ms. How many concurrent execution environments does it need?
8. Using the worked-example prices, what is the per-invocation cost of a 1 GB, 250 ms function without a gateway?
9. In hexagonal architecture, what is the difference between a driving and a driven port?
10. Why should a fake adapter be run through the same contract tests as the real adapter?
11. State the dependency rule. How does a use case send data outward without violating it?
12. A service has 2,000,000 customers and a tested cell size of 80,000. How many cells, and what is the blast radius?
13. With $N = 10$ workers and $k = 3$, what fraction of customers share all three workers with a poison customer?
14. How many coordination paths do 8 teams have if every team may need every other?

Answers

1. A layer is a logical grouping of code by responsibility; a tier is a physical deployment boundary where a process runs. Several layers can share one tier.
2. Requests that pass through layers without any processing. Open the layer for those paths, or adopt a style with less ceremony where most operations are simple.
3. Code is partitioned by technical role, so a new domain concept needs changes in the presentation, business and persistence layers and the schema, often owned by different people.
4. $0.98^{30} = e^{30 \ln 0.98} = e^{-0.606} \approx 0.545$.
5. A module may depend only on its declared modules, and only through their public `api`.
6. SOA's enterprise service bus, which contains routing, transformation and orchestration logic in the middleware.
7. $300 \times 0.2 = 60$ environments (Little's Law).
8. $1 \times 0.25 = 0.25$ GB-s; $0.25 \times 0.0000166667 = \$4.17 \times 10^{-6}$; plus $\$2 \times 10^{-7}$ gives about $\$4.37 \times 10^{-6}$.
9. A driving port is how external actors invoke the application (its use cases); a driven port is how the application calls external resources (repositories, gateways). Driving adapters call the core; driven adapters are called by it.
10. To prove the fake behaves like the real adapter; otherwise fast tests may pass against behavior production never exhibits.
11. Source-code dependencies point only inward. The inner circle defines an output interface that an outer-circle presenter or gateway implements, so control flows outward while the dependency points inward.
12. $\lceil 2,000,000/80,000 \rceil = 25$ cells; blast radius $1/25 = 4\%$.
13. $1/\binom{10}{3} = 1/120 \approx 0.83\%$.
14. $8 \times 7/2 = 28$.

33.16 Interview questions

Basic

Q1. When is a monolith the right choice? Answer. When one or two teams own the product, the domain is still being discovered, strong consistency across features matters and there is no platform for running many services. It offers in-process calls, ACID transactions, easy refactoring and simple operations. Structure it as a modular monolith so that extraction stays possible.

Q2. What is hexagonal architecture? Answer. The core defines ports, interfaces in its own vocabulary, for use cases it offers and resources it needs. Adapters connect technologies: HTTP controllers drive the core, SQL repositories and API clients implement driven ports. The core imports no framework, so it can be tested with fakes and its technologies replaced.

Intermediate

Q3. Your 15-team organization ships one monolith weekly and half the releases are delayed. What do you do? Answer. Measure first: change failure rate, batch size, pipeline duration and which modules cause holds. Reduce p with better tests and c with smaller, more frequent deploys and feature flags. Enforce module boundaries and per-module schemas. Then extract the modules whose teams are most blocked and whose interfaces are stable, one at a time with the strangler fig pattern, on a platform providing pipelines, tracing and on-call tooling.

Q4. Should an API with 5 RPS average and 500 RPS spikes be serverless? Answer. Likely yes. At 5 RPS average, serverless costs a fraction of the break-even (about 23–45 RPS in the chapter's example), while servers would be sized for 500 RPS. Check cold-start impact on the latency SLO during spikes, concurrency limits ($500\times$ duration environments), and whether downstream databases can absorb connection bursts; add provisioned concurrency or a connection proxy if not.

Advanced

Q5. Design fault isolation for a multi-tenant API where one tenant's malformed request has crashed the whole fleet. Answer. Explain that redundancy fails because every replica runs the same code and retries spread the poison request. Partition into cells with a thin, statically stable router and tested cell size; blast radius is $1/N_c$. Within cells, shuffle-shard stateless workers so full overlap is $1/\binom{N}{k}$. Add per-tenant quotas, wave deployments, and quarantine of request signatures that crash workers.

Q6. Contrast SOA, microservices and cells. Answer. SOA integrates enterprise applications through a smart ESB and canonical model, centralizing logic and governance. Microservices split one product by business capability into independently deployable services with their own data and dumb pipes, optimizing team autonomy. Cells duplicate an entire stack, monolith or microservices, per customer partition to limit blast radius. They answer integration, organizational and fault-isolation problems respectively and can be combined.

33.17 Practical exercises

1. Extend the boundary checker to detect cycles in `ALLOWED` and to read real files from a directory tree; run it on a project you know.
2. Add a `PaymentGateway` driven port to the hexagonal example with a fake that can be told to decline, and test that a declined payment leaves stock unchanged in both repository adapters.
3. Write a script that computes R^* for memory sizes from 128 MB to 2 GB and durations from 20 ms to 1 s, and plot the break-even RPS.
4. Simulate shuffle sharding: assign 10,000 customers random 5-of-100 shards, fail one shard, and compare the measured overlap distribution with the hypergeometric formula.

33.18 Design challenges

1. **Folio's growth plan.** Folio has 6 engineers today and expects 60 in three years, in eight countries with data-residency rules. Propose an architecture for year one and the measured triggers for each later step (module extraction, cells per country), with an ADR (§2.12.4) for each.
2. **Serverless or containers for image processing.** Folio generates cover thumbnails: 2 million uploads a month, bursty around publisher catalog drops, 1.5 GB memory and 3 s per image. Compare serverless and always-on costs, latency and failure handling, and recommend a design.

33.19 References

1. Amazon Web Services. *AWS Lambda Pricing* and *AWS Lambda Developer Guide* (execution environment lifecycle, provisioned concurrency, SnapStart, quotas). Accessed 2026. https://aws.amazon.com/lambda/pricing/ (AWS Lambda Pricing)
2. Amazon Web Services. *Reducing the Scope of Impact with Cell-Based Architecture*. AWS Well-Architected whitepaper, 2023. (AWS 2023)

3. Bass, L., Clements, P., Kazman, R. *Software Architecture in Practice*, 4th ed. Addison-Wesley, 2021.
4. Cockburn, A. “Hexagonal Architecture.” alistair.cockburn.us, 2005. https://alistair.cockburn.us/hexagonal-architecture/
5. Conway, M. E. “How Do Committees Invent?” *Datamation* 14(4), 1968.
6. Craver, N. “Stack Overflow: The Architecture – 2016 Edition.” nickcraver.com, 2016.
7. Foote, B., Yoder, J. “Big Ball of Mud.” *Pattern Languages of Programs (PLoP)*, 1997.
8. Fowler, M. “MonolithFirst.” martinfowler.com, 2015. https://martinfowler.com/bliki/MonolithFirst.html
9. Jonas, E., et al. “Cloud Programming Simplified: A Berkeley View on Serverless Computing.” Technical Report UCB/EECS-2019-3, University of California, Berkeley, 2019. arXiv:1902.03383.
10. Kolny, M. “Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%.” Prime Video Tech blog, 2023.
11. Lewis, J., Fowler, M. “Microservices: a definition of this new architectural term.” martinfowler.com, 2014. https://martinfowler.com/articles/microservices.html
12. MacCárthaigh, C. “Workload isolation using shuffle-sharding.” *Amazon Builders’ Library*, 2019. https://aws.amazon.com/builders-library/workload-isolation-using-shuffle-sharding/
13. Martin, R. C. “The Clean Architecture.” The Clean Code Blog, 2012. https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html
14. Martin, R. C. *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017.
15. Newman, S. *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O’Reilly Media, 2019.
16. Noonan, A. “Goodbye Microservices: From 100s of problem children to 1 superstar.” Segment Engineering Blog, 2018.
17. Palermo, J. “The Onion Architecture: part 1.” jeffreypalermo.com, 2008.
18. Parnas, D. L. “On the Criteria To Be Used in Decomposing Systems into Modules.” *Communications of the ACM* 15(12):1053–1058, 1972.
19. Richards, M. *Software Architecture Patterns*. O’Reilly Media, 2015.
20. Richards, M., Ford, N. *Fundamentals of Software Architecture: An Engineering Approach*. O’Reilly Media, 2020.
21. Shopify. *Packwerk*: a Ruby gem for enforcing boundaries and modularization in Rails applications. GitHub repository. https://github.com/Shopify/packwerk (Shopify Packwerk)
22. Skelton, M., Pais, M. *Team Topologies: Organizing Business and Technology Teams for Fast Flow*. IT Revolution, 2019.
23. Tilkov, S. “Don’t start with a monolith.” martinfowler.com, 2015. https://martinfowler.com/articles/dont-start-monolith.html
24. Weiss, B., Furr, M. “Static stability using Availability Zones.” *Amazon Builders’ Library*, 2020.

25. Westeinde, K. “Deconstructing the Monolith: Designing Software that Maximizes Developer Productivity.” Shopify Engineering blog, 2019.

33.20 Open verification items

- AWS Lambda prices ($0.0000166667 per GB-s, $0.20 per million requests, x86, us-east-1), 15-minute maximum duration, one-request-per-environment concurrency model, and whether initialization (INIT) time is billed for all functions, as of 2026.
- API gateway price of about $1.00 per million requests (HTTP APIs) used as illustrative, as of 2026.
- Richards’s 80–20 sinkhole heuristic: confirm it appears in Richards 2015 (and/or Richards and Ford 2020).
- Kolny 2023: exact title, “over 90%” figure and current URL (the post may have moved).
- Noonan 2018: exact title and the number of destination services (“well over a hundred”).
- Westeinde 2019 exact title; Packwerk’s description and current status of its privacy checks.
- Route 53 use of shuffle sharding as described in MacCárthaigh 2019; the brief attributed the shuffle-sharding article to Brooker, but the Builders’ Library byline is believed to be Colm MacCárthaigh (consistent with Ch 18).
- Exact title and year of the AWS cell-based architecture whitepaper.
- Cockburn and Palermo URLs.

Chapter 34 — Microservices

> **Level:** 3 · **Track:** A (Traditional) · **Estimated reading time:** 6 h

Chapter 33 placed microservices among the architecture styles. This chapter is about doing microservices well, and about recognizing when not to. Most microservice failures come from where the lines are drawn, not from the technology. Split a system along the wrong seams and every feature needs five teams, every release needs a coordinated deploy, and every request crosses a dozen network hops. Split it along the right seams and teams ship independently, failures stay contained, and each part scales on its own.

The chapter has three parts. **Boundaries**: cohesion, change coupling, team ownership and domain-driven design. **Plumbing**: gateways, backends for frontends, discovery, communication, contract testing, tracing and service meshes, each with its latency and cost arithmetic. **Data and change**: database-per-service, independent deployment, monolith migration and anti-patterns. QuickBite, the food-delivery platform of Chapter 2, is the running example.

34.1 Learning objectives

After this chapter you should be able to:

- Choose service boundaries using cohesion, change coupling, team ownership and sizing heuristics.
- Apply entities, value objects, aggregates and domain events, and size an aggregate with a contention estimate.
- Draw a context map and implement an anticorruption layer.
- Place an API gateway and backends for frontends, and compute the latency saved by aggregating calls.
- Compare client-side and server-side discovery and compute a registry’s stale-endpoint window.
- Choose synchronous or asynchronous communication, version APIs and write a consumer-driven contract test.
- Explain trace context propagation and service-mesh data and control planes, including ambient mode, and estimate mesh overhead.
- Apply database-per-service, and query across services with API composition or CQRS views.

- Plan a strangler-fig migration and diagnose anti-patterns, including the A^n availability of call chains.

34.2 Prerequisites

- Monolith, modular monolith and microservices overview — §33.4, §33.5, §33.7.
- Conway's law and the QuickBite decomposition — §1.4, §2.3.
- Fan-out tail latency and series availability — §4.8.4, §4.14.2.
- REST, gRPC and API versioning — §8.6, §8.8, §8.15.
- Client-side load balancing — §19.10.
- Timeouts, retries, circuit breakers and idempotency — §22.11–§22.15.
- Messaging, outbox and sagas — Chapters 30 and 32 (summarized here, not repeated).

34.3 Service boundaries

34.3.1 What a microservice is

> DEFINITION
>
> **Definition — Microservice.** A service built around one business capability, owned by one team, deployable independently, and the sole owner of its data, which others reach only through its API or events. Lewis and Fowler describe the style as "an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms," built around business capabilities and independently deployable [Lewis and Fowler 2014].

First-principles explanation

A large restaurant kitchen is divided into **stations**: grill, sauce, pastry. Each has its own staff, tools and ingredients, and stations communicate through one fixed protocol, the ticket. The pastry chef can change suppliers without asking the grill. The division is by *outcome* (desserts), not by *activity* (one person chops for everyone), because outcomes change together: a new dessert touches only pastry. A microservice is a station: it owns an outcome, its data and its team, and others reach it only through its API.

Why does this exist?

A monolith shares one codebase, one deployable and one database. Beyond a few dozen engineers this causes **release coupling** (every change waits for the slowest change in the deploy), **scaling coupling** (the hottest component forces the whole application to scale) and **failure coupling** (a reporting job's memory leak takes down checkout).

Microservices trade these for distributed-systems pains. Newman's summary: microservices buy options, chiefly independent deployability, at a substantial cost in complexity [Newman 2021].

34.3.2 Cohesion and coupling at service scale

DEFINITION

Definition — Cohesion and coupling (service level). Cohesion is the degree to which code inside one service changes together for the same reasons. **Coupling** is the degree to which a change in one service forces a change, redeploy or coordinated release in another. Newman's shorthand: "the code that changes together, stays together" [Newman 2021].

Newman orders coupling from mild to damaging: **domain** coupling (A uses B's capability; unavoidable), **temporal** coupling (B must be up now; removed by events, §34.8), **pass-through** coupling (A carries data only because C needs it), **common** coupling (shared data) and **content** coupling (A changes B's database directly, the worst) [Newman 2021].

How it works internally: measuring change coupling

Version-control history records which code changes together: every commit touching two modules is evidence that they change for the same reason. Tornhill popularized mining such change-coupling metrics from Git [Tornhill 2015].

FORMULA

Change coupling between two modules (Jaccard index of co-changes)

$$C(A,B) = \frac{n_{AB}}{n_A + n_B - n_{AB}}$$

Symbol	Meaning	Units
n_A	Number of commits (or pull requests) that touch module A in the window	commits
n_B	Number of commits that touch module B in the window	commits
n_{AB}	Number of commits that touch both A and B	commits
$C(A,B)$	Fraction of all changes to either module that touched both; 0 = independent, 1 = always together	dimensionless

WORKED EXAMPLE

Worked example — Should QuickBite's payment code be its own service? In six months of monolith history, 120 pull requests touched `ordering`, 80 touched `payments` and 60 touched both; 90 touched `catalog`, of which 6 also touched `ordering`.

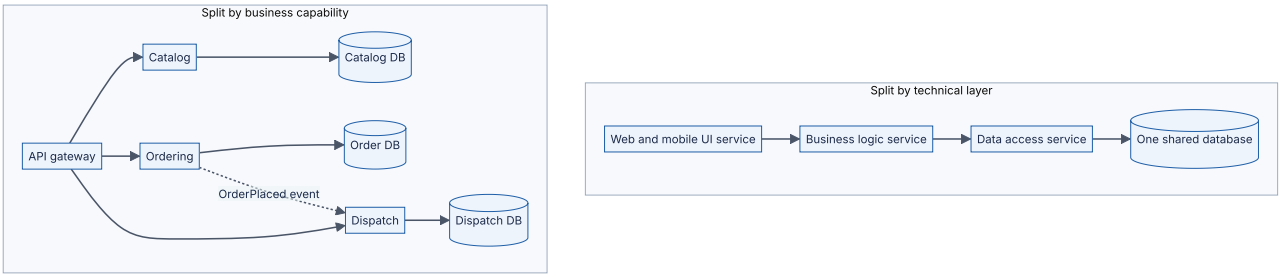
1. Ordering–payments: $C = 60/(120 + 80 - 60) = 60/140 = 0.43$.
2. Ordering–catalog: $C = 6/(120 + 90 - 6) = 6/204 = 0.029$.

Interpretation. As separate services, ordering and payments would need cross-team coordination on 43% of their changes; ordering and catalog on 3%. **Engineering implication.** Extract catalog first. Before splitting payments, inspect the 60 shared pull requests and redesign the interface (for example a `PaymentAuthorized` event) until the co-change disappears; otherwise you build a distributed monolith (§34.14). Treat $C > 0.2$ as a prompt to investigate, not a rule.

34.3.3 Team ownership and Conway's law

Conway observed that systems copy the communication structure of the organizations that build them [Conway 1968] (§1.4; §33.13.2 counts the coordination paths). The **inverse Conway maneuver** shapes teams first so the desired architecture is the natural outcome. Skelton and Pais add that a service should fit within one **stream-aligned team's** cognitive load [Skelton and Pais 2019]. Practical rules: one owning team per service (a team may own several); a change in one business capability should land in one team; and teams with high communication cost (time zones, companies) need stable, documented interfaces between them.

Figure 34.1 — Two ways to split QuickBite: by technical layer and by business capability.



Reading the diagram. The top split follows technical layers: UI, logic and data-access services call each other synchronously over one shared database, so a feature such as "scheduled orders" changes all three services and the schema, every request crosses two extra hops, and each service is a single failure point for every capability. The bottom split follows capabilities: the gateway routes each request to one service (synchronous HTTPS or gRPC), each service owns its database (separate storage boundaries), and ordering in-

forms dispatch asynchronously with an `OrderPlaced` event (dotted), so a dispatch outage does not block ordering. Each capability scales independently: read-heavy catalog adds replicas and caches; dispatch scales with courier activity.

34.3.4 Sizing heuristics

Size follows from boundaries, not lines of code. Each heuristic below is a test you can apply to a proposed boundary.

Table 34.1 — Service sizing heuristics.

Heuristic	The test	Too big when	Too small when
Business capability	Nameable in two or three business words?	Name needs “and”	Name is a verb on another’s noun (“order-validator”)
Team ownership	One stream-aligned team can own it?	Several teams commit weekly	One team owns twenty services
Change coupling	Changes usually stay inside?	Many unrelated reasons to change	Most changes also change a neighbor
Data ownership	Owns its data outright?	Unrelated tables with separate invariants	Needs another service’s data on every request
Transaction boundary	Invariants enforced inside?	—	An invariant spans it and a neighbor, forcing sagas
Independent deployability	Ships on its own schedule?	Blocked by unrelated work inside	Always deploys with a neighbor

The most important row is the **transaction boundary**: if two candidates must change state atomically for a business rule to hold, they belong together, because splitting them turns one local transaction into a saga with compensations (§32.7).

RECOMMENDATION

Recommendation — Start coarse, split on evidence. Begin with a modular monolith (§33.5) or a few coarse services. Split when there is a concrete reason (team boundary, scaling profile, failure isolation, release cadence) and history shows low change coupling. Merging services later costs far more than splitting a well-modularized module.

COMMON MISTAKE

Common misconception — "Micro means small, so smaller is better." *Micro* refers to scope of responsibility. A service that is too small moves its complexity into the network: more calls, failure modes, versioned interfaces and pipelines. Fowler's "Microservice Premium" argues that the style carries a fixed productivity cost that pays off only above a certain system complexity [Fowler 2015].

34.4 Domain-driven design

Domain-driven design (DDD) is the most widely used discipline for finding and modeling business capabilities. Evans wrote it for complicated domains such as shipping and banking, a decade before microservices [Evans 2003]. **Strategic design** divides a domain into models and relates them (§34.5); **tactical design** builds one model well (this section).

DEFINITION

Definition — Domain-driven design (DDD). An approach that places a model of the business domain, built jointly by developers and domain experts, at the center of the design, expresses it directly in code, and divides large domains into explicitly bounded models [Evans 2003; Vernon 2013].

34.4.1 Ubiquitous language

Ask three people at QuickBite what an "order" is. Support means the thing a customer complains about, refund included; restaurant operations means a ticket the kitchen accepts; finance means a settlement line that is final 14 days after delivery. Code serving all three with one `Order` class becomes a 4,000-line class with 60 nullable fields.

DEFINITION

Definition — Ubiquitous language. A rigorous vocabulary built by developers and domain experts for one model and used consistently in conversation, tests and code, so that each term means exactly one thing within that model [Evans 2003].

The language is ubiquitous *within a boundary*. In ordering, `Order.accept()` means the restaurant confirmed it can cook; finance has no `accept`, only `Settlement.finalize()`. When code says `status = 3` and the business says "accepted", translation happens in people's heads and fails silently.

34.4.2 Entities and value objects

DEFINITION

Definition — Entity and value object. An **entity** has a continuous identity that persists as its attributes change: two orders for the same pizza are two orders. A **value object** is defined entirely by its attributes, has no identity and is usually immutable: `Money(24.50, "USD")`, `Address` [Evans 2003].

Prefer value objects. A bare `float` loses its currency and precision; a `Money` value object that refuses negative amounts makes whole classes of bugs impossible to write.

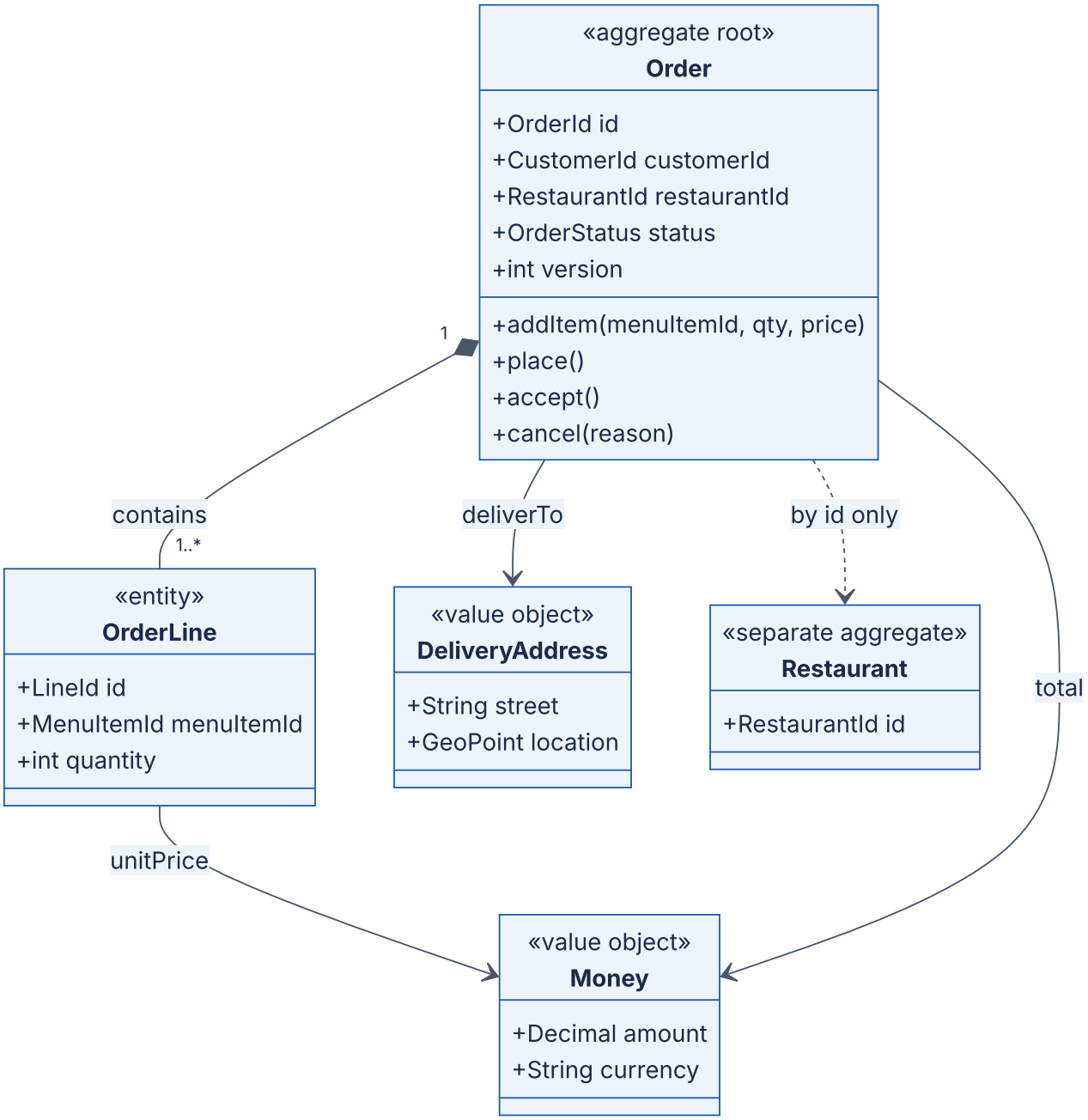
34.4.3 Aggregates and consistency boundaries

DEFINITION

Definition — Aggregate. A cluster of entities and value objects treated as one unit for data changes, with one **root** entity. Outside objects reference only the root, every change goes through it, and it enforces the aggregate's **invariants**. One transaction modifies one aggregate [Evans 2003; Vernon 2013].

In a large object graph any code can change anything, so nobody can say which rules hold, and locking the whole graph does not scale. An aggregate draws a small boundary around exactly the data one invariant needs: consistency is immediate inside (one ACID transaction) and eventual across boundaries (via domain events). Vernon's rules: model true invariants in consistency boundaries; design small aggregates; reference other aggregates by identity; use eventual consistency outside the boundary [Vernon 2013].

Figure 34.2 — The QuickBite `Order` aggregate.



Reading the diagram. The `Order` root is the only entry point; `OrderLine` entities live and die with it (composition). `Money` and `DeliveryAddress` are immutable value objects. `Restaurant` is a separate aggregate referenced only by identifier (dotted), so loading an order never loads a menu, and the two are stored and locked separately. The root enforces invariants such as “total equals the sum of lines”. The `version` field supports optimistic concurrency: a write commits only if the version is unchanged since the read. The aggregate is the unit of storage, locking and partitioning (all its data shares the `OrderId` key).

Mathematical foundations: how big may an aggregate be?

FORMULA

Optimistic-concurrency conflict probability for one aggregate

$$P_{\text{conflict}} = 1 - e^{-\lambda w}$$

Symbol	Meaning	Units
λ	Rate of *other* writes to the same aggregate (Poisson assumption)	writes/s
w	Read-modify-write window of one transaction (read version to commit)	s
P_{conflict}	Probability that at least one other write commits inside the window, so this write must retry	dimensionless

WORKED EXAMPLE

Worked example — A restaurant aggregate versus an order aggregate. A first design made `Restaurant` one aggregate containing all active orders, to enforce "at most 30 open orders" in one transaction. At the Friday peak a busy restaurant receives $\lambda = 2$ writes/s; each command takes $w = 100$ ms from read to commit.

1. Big aggregate: $\lambda w = 0.2$; $P = 1 - e^{-0.2} = 0.181$. About 18% of writes retry, and retries raise λ further.
2. Order as its own aggregate: about 4 updates over a 40-minute life, $\lambda \approx 4/2,400 = 0.00167$ writes/s; $P \approx 0.00017$, one write in 6,000.

Interpretation. Splitting cut conflicts about 1,000-fold. **Engineering implication.** The capacity rule becomes an eventually consistent counter fed by `OrderPlaced` events, or a tiny `KitchenCapacity` aggregate. Ask the business whether 31 open orders for a few seconds is truly unacceptable; the answer decides the boundary.

34.4.4 Domain events

DEFINITION

Definition — Domain event. An immutable record of something domain experts care about that happened, named in the past tense (`OrderPlaced` , `CourierAssigned`), carrying the identifiers and facts its consumers need [Evans 2003; Vernon 2013].

Inside a service, aggregates record events that handlers apply to other aggregates after commit. Across services, the same events go to a broker (Chapter 30), published reliably with the **transactional outbox**: the event row is written in the same local transaction as the aggregate change and relayed afterwards (§32.5). An *event* states a fact; a *command* asks for something and may be refused (§30.6).

COMMON MISTAKE

Common misconception — "DDD means repositories and factories." Most of DDD's value is strategic: finding boundaries and naming things in the business's language. A codebase full of `OrderRepository` classes that still has one "order" model for support, kitchen and finance has adopted the vocabulary without the design (the Repository and Factory patterns themselves are in §37.13 and §37.3). Evans reserves the full investment for the **core domain**, the part that differentiates the business [Evans 2003].

34.5 Bounded contexts and context maps

34.5.1 Bounded contexts

DEFINITION

Definition — Bounded context. An explicit boundary (in code, data and team ownership) within which one domain model and its ubiquitous language apply consistently. The same word may mean different things in different contexts [Evans 2003].

A "ticket" means different things at a railway station, in a police car and in a sprint backlog; nobody is confused because each word lives in a context. QuickBite's order therefore becomes several models keyed by a shared `order_id`: in **Ordering**, lines, a status machine and a total; in **Kitchen**, a `Ticket` with a promised-ready time; in **Delivery**, pickup, drop-off and courier; in **Billing**, a `Charge` and later a `Settlement`. Single enterprise-wide models, the goal of 1990s data modeling, became so general that they served nobody; bounded contexts make the translation between legitimate models explicit.

KEY CONCEPT

Concept — A bounded context is the best first candidate for a service boundary. It already has high cohesion and a deliberately narrow interface. Newman recommends contexts as the starting point: one context may later become several services, but a service should rarely span two contexts [Newman 2021].

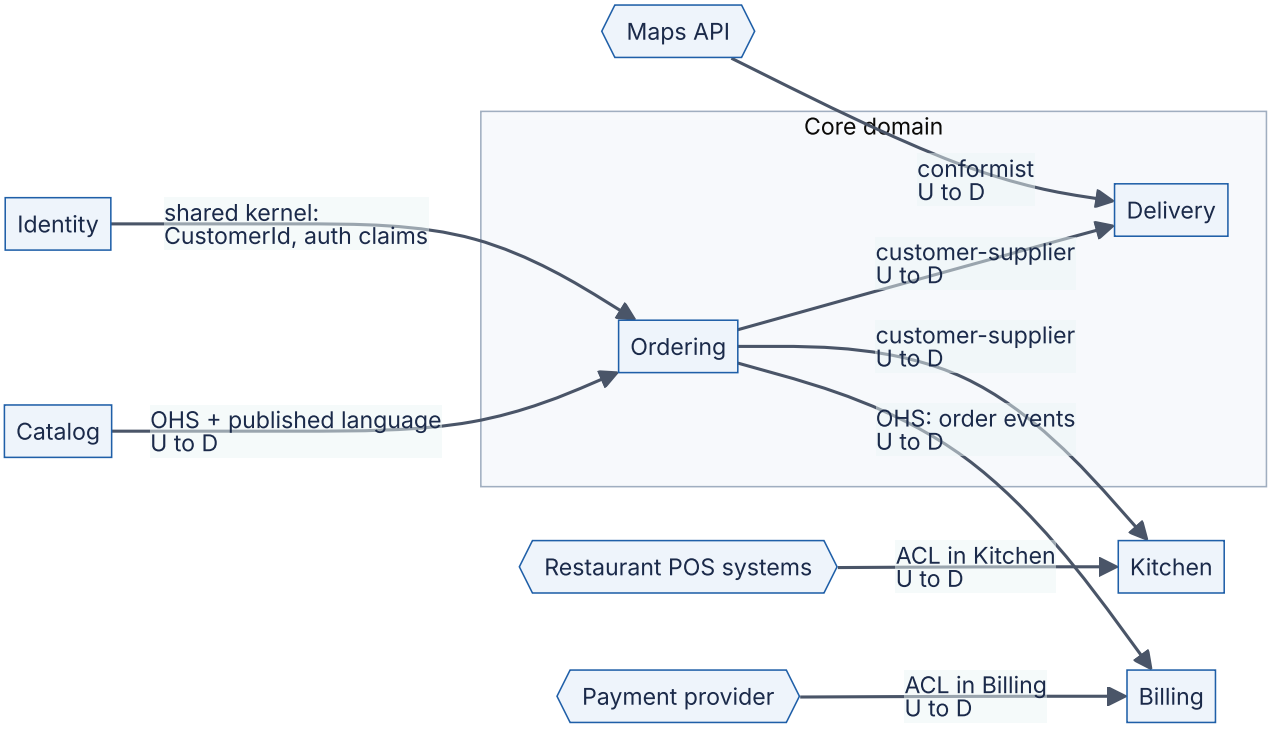
34.5.2 Context maps

A **context map** shows the bounded contexts and their relationships. Each relationship pattern describes both an integration style and a team relationship: **upstream** contexts influence **downstream** ones [Evans 2003; Vernon 2013].

Table 34.2 — Context-map relationship patterns.

Pattern	What it is	Choose when	Risk
Shared kernel	Small, jointly owned subset of the model	Two closely cooperating teams	Kernel grows into shared-library coupling
Customer–supplier	Upstream plans for downstream's needs	Teams with aligned goals	Supplier overwhelmed by customers
Conformist	Downstream adopts upstream's model	Upstream is large or external and acceptable	Upstream model distorts downstream
Anticorruption layer	Downstream translates upstream's model into its own	Upstream is legacy, poor or external	Translation code to maintain
Open host service	Upstream offers one stable protocol for all	Many consumers	Protocol must evolve compatibly
Published language	Documented shared exchange format	Many parties exchange data	Format governance
Separate ways	No integration	Integration costs more than it returns	Duplicated effort
Partnership	Two teams coordinate releases	Jointly delivered capability	Coupled release cadence

Figure 34.3 — QuickBite context map.



Reading the diagram. Arrows run from upstream to downstream and are labeled with the pattern; this models influence and translation, not request direction. Catalog's **open host service** publishes a menu schema that Ordering consumes. Ordering is a **supplier** to Delivery and Kitchen, which consume its events asynchronously. Billing receives order events through Ordering's open host service and shields itself from the payment provider with an **anticorruption layer**; Kitchen does the same for many restaurant point-of-sale systems. Delivery is **conformist** toward the maps provider, whose geometry types are acceptable. Identity and Ordering share a tiny **kernel**. Each box is a storage and ownership boundary. Failure points: Kitchen's ACL must stop malformed POS data; a maps model change forces Delivery to change. Each context scales independently.

34.5.3 The anticorruption layer in code

> DEFINITION
>
> **Definition — Anticorruption layer (ACL).** A layer owned by a downstream context that translates between an upstream model and its own, so the upstream's concepts, codes and quirks never appear in the downstream model [Evans 2003].

The payment provider speaks in statuses such as `AUTHORISED`, `CAPTURED`, `REFUSED` and `PENDING_3DS`, numeric refusal codes and string amounts in minor units. Billing knows three outcomes: authorized, declined (with a business reason) and "customer must act". The ACL below is the only code that knows the provider's vocabulary.

```
from dataclasses import dataclass
from decimal import Decimal
from enum import Enum


# ---- Billing context: the model we want to protect ------------------------
@dataclass(frozen=True)
class Money:                        # value object: equality by value, immutable
    amount: Decimal
    currency: str

    def __post_init__(self):
        if self.amount < 0 or len(self.currency) != 3:
            raise ValueError(f"invalid money {self.amount} {self.currency}")


class DeclineReason(Enum):
    INSUFFICIENT_FUNDS = "insufficient_funds"
    CARD_EXPIRED = "card_expired"
    SUSPECTED_FRAUD = "suspected_fraud"
    OTHER = "other"


@dataclass(frozen=True)
class PaymentAuthorized:
    order_id: str
    amount: Money
    provider_ref: str


@dataclass(frozen=True)
class PaymentDeclined:
    order_id: str
    reason: DeclineReason


@dataclass(frozen=True)
class PaymentNeedsCustomerAction:  # e.g. a 3-D Secure challenge
    order_id: str
    redirect_url: str


# ---- Anticorruption layer: the only code that knows the provider's model --
MINOR_UNITS = {"USD": 2, "EUR": 2, "JPY": 0}
DECLINE_CODES = {"51": DeclineReason.INSUFFICIENT_FUNDS,
                 "54": DeclineReason.CARD_EXPIRED,
                 "59": DeclineReason.SUSPECTED_FRAUD}


class ProviderTranslationError(Exception):
    pass


def translate(raw: dict):
    """Map one provider response (their language) to one domain outcome."""
    order_id = raw.get("merchant_reference")
```

```
            if not order_id:
                raise ProviderTranslationError("missing merchant_reference")
            status = raw.get("status", "").upper()
            if status in ("AUTHORISED", "CAPTURED"):          # two words, one meaning
                cur = raw["currency"].upper()
                if cur not in MINOR_UNITS:
                    raise ProviderTranslationError(f"unsupported currency {cur}")
                amount = Decimal(int(raw["amount_minor"])).scaleb(-MINOR_UNITS[cur])
                return PaymentAuthorized(order_id, Money(amount, cur), raw["psp_ref"])
            if status == "REFUSED":
                reason = DECLINE_CODES.get(str(raw.get("refusal_code")),
DeclineReason.OTHER)
                return PaymentDeclined(order_id, reason)
            if status == "PENDING_3DS":
                return PaymentNeedsCustomerAction(order_id, raw["action"]["url"])
            raise ProviderTranslationError(f"unknown provider status {status!r}")


        # ---- Tests --------------------------------------------------------------
        if __name__ == "__main__":
            ok = translate({"merchant_reference": "ord-17", "status": "Authorised",
                            "amount_minor": "2450", "currency": "usd", "psp_ref":
"PSP9"})
            assert ok == PaymentAuthorized("ord-17", Money(Decimal("24.50"), "USD"),
"PSP9")

            yen = translate({"merchant_reference": "ord-18", "status": "CAPTURED",
                             "amount_minor": "1500", "currency": "JPY", "psp_ref":
"PSP10"})
            assert yen.amount == Money(Decimal("1500"), "JPY")

            assert translate({"merchant_reference": "ord-19", "status": "REFUSED",
                              "refusal_code": 51}).reason is
DeclineReason.INSUFFICIENT_FUNDS
            assert translate({"merchant_reference": "ord-20", "status": "REFUSED",
                              "refusal_code": "99"}).reason is DeclineReason.OTHER

            action = translate({"merchant_reference": "ord-21", "status": "PENDING_3DS",
                                "action": {"url": "https://psp.example/3ds/abc"}})
            assert isinstance(action, PaymentNeedsCustomerAction)

            for bad in ({"status": "AUTHORISED"},
                        {"merchant_reference": "ord-22", "status":
"PARTIALLY_AUTHORISED"}):
                try:
                    translate(bad)
                    raise AssertionError("expected ProviderTranslationError")
                except ProviderTranslationError as e:
                    print("rejected:", e)
            print("all ACL tests passed")
```

Running it with `python3` prints:

```
rejected: missing merchant_reference
rejected: unknown provider status 'PARTIALLY_AUTHORISED'
all ACL tests passed
```

What to notice:

- **Two upstream words, one meaning.** `AUTHORISED` and `CAPTURED` both become `PaymentAuthorized`.
- **Units are translated at the edge.** The currency's exponent is applied, so a yen amount is not divided by 100.
- **Unknown codes are mapped deliberately.** An unknown refusal code safely becomes `OTHER`, but an unknown *status* raises, because guessing whether money moved is never safe; route it to a dead-letter queue and alert (§30.11).
- **Isolation.** When the provider changes, one file and its tests change, not Billing's model.

34.5.4 Event storming

DEFINITION

Definition — Event storming. A collaborative workshop technique created by Alberto Brandolini in which domain experts and developers place domain events (past tense, on sticky notes) along a timeline, then add commands, actors, policies ("whenever X, then Y"), external systems and hotspots (open questions), and group the result into aggregates and candidate bounded contexts [Brandolini: Introducing EventStorming].

How it works internally

1. **Chaotic exploration.** Everyone writes the events they know and sticks them roughly in time order.
2. **Enforce the timeline.** Sort, remove duplicates and mark disagreements as **hotspots**; disagreements reveal where one word has two meanings.
3. **Add causes.** For each event, add the command and actor, policy or external system that caused it.
4. **Find aggregates and boundaries.** Group commands around what decides whether they are allowed (aggregates); draw lines where the language or the owning department changes (candidate contexts).

Figure 34.4 — Event-storming output for QuickBite's order flow (simplified).



Reading the diagram. This is a modeling artifact: arrows show causality, not network calls. Stadiums are actors, parallelograms commands, rectangles domain events and hexagons policies. A customer's *Place order* produces `OrderPlaced`; a policy reacts with *Authorize payment*, and so on. The dotted arrow marks a human waiting step, where latency and timeouts will appear in the system. The vocabulary shifts around `PaymentAuthorized` (Billing), at "ticket" (Kitchen) and at "courier" (Delivery): these are the candidate boundaries of Figure 34.3. The final hotspot (support says delivery, finance says 14 days later) becomes a design decision instead of a production bug.

Failure scenarios

What goes wrong	Why	Detection	Recovery	Prevention
Upstream model leaks downstream	Accidental conformist	Upstream codes in downstream domain classes	Introduce an ACL	Decide each relationship explicitly
Shared kernel grows	Convenient home for common code	Kernel changes in most releases	Move code to owners	Kernel limited to identifiers and stable values
ACL silently mistranslates	Unknown codes mapped to defaults	"Declined: other" spikes	Fix mapping; replay dead letters	Fail loudly on unknown statuses

34.6 API gateways and backends for frontends

34.6.1 The API gateway

DEFINITION

Definition — API gateway. A reverse proxy (§7.12) that is the single entry point for external clients into a set of services. It routes requests and applies **edge concerns** once: TLS termination, authentication, rate limiting, request validation, caching, protocol translation and sometimes aggregation [Richardson 2018].

A hotel guest asks the front desk rather than searching the corridors for housekeeping; the desk checks who they are, knows where everything is and enforces house rules in one place. Without a gateway, a mobile app must know every service's address, every service must validate tokens and enforce rate limits in every language, and every internal refactoring breaks published clients. The gateway decouples the **external API**, which must stay stable for years because old app versions persist (§8.4.3), from the **internal topology**, which should be free to change weekly.

For each request it terminates TLS; authenticates (validates a JWT, §8.13, and attaches a verified identity header); applies rate limits (Chapter 22) and validation; matches the route and resolves the target through discovery (§34.7); optionally aggregates; and starts

or continues a trace (§34.9). Common choices include Kong, Envoy-based gateways, NGINX, cloud API gateways and Kubernetes Gateway API implementations (§42.10). The gateway holds no business state and scales horizontally.

WARNING / TRADE-OFF

Warning — Keep business logic out of the gateway. Every team needs route changes, so a gateway accumulating business rules becomes a shared monolith owned by one overloaded team. Lewis and Fowler's principle is "smart endpoints and dumb pipes" [Lewis and Fowler 2014].

34.6.2 Backends for frontends (BFF)

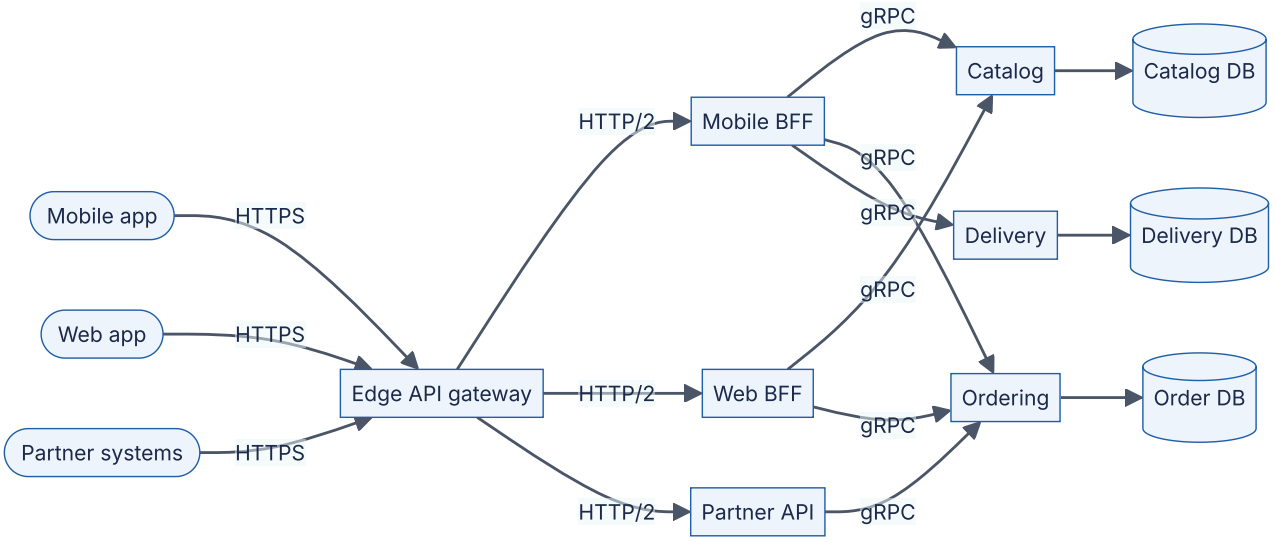
DEFINITION

Definition — Backend for frontend (BFF). A server-side component dedicated to one kind of user interface (iOS, web, partner API), owned by or aligned with that interface's team, which aggregates and reshapes downstream calls into exactly the API that interface needs [Newman 2015].

One general-purpose API serves no client well. The mobile home screen needs 6 fields from 5 services in one round trip over a cellular network; the web dashboard needs 40 fields; a partner needs a stable, versioned contract. A single gateway serving all three accumulates per-client conditionals and makes every UI change wait for the gateway team.

Origins, cited carefully. At **Netflix**, the API was redesigned around 2012 so that device-specific adapter code, written by each device's UI team, ran on the server and called general-purpose backend services, letting each device fetch what its screens needed in one request [Jacobson 2012]. At **SoundCloud**, per-client backends emerged from about 2013 during the move from a monolith to services; Lukasz Plotnicki described them, and Sam Newman, who interviewed SoundCloud engineers, named and popularized the pattern in 2015 [Plotnicki 2015; Newman 2015].

Figure 34.5 — Edge gateway with one BFF per client type.



Reading the diagram. All clients reach one edge gateway over HTTPS; it terminates TLS, authenticates, rate-limits and routes to a BFF over HTTP/2. Each BFF calls domain services synchronously over gRPC and merges the responses into a client-shaped payload. Domain services own their databases; BFFs own no business data, so they scale horizontally with their client's traffic. Failure points: a BFF's availability and tail latency are bounded by its dependencies (§34.8.3), so it needs deadlines, partial responses and fallbacks (Chapter 22).

Mathematical foundations: why aggregation near the services pays off

FORMULA

Latency of sequential and parallel call sequences

$$L_{\text{seq}} = \sum_{i=1}^{n} (R_i + s_i) \qquad L_{\text{par}} = \max_i (R_i + s_i)$$

Symbol	Meaning	Units
n	Number of calls needed to build one response	calls
R_i	Network round-trip time of call i (including connection reuse)	ms
s_i	Server processing time of call i	ms
L_{seq}	Total latency when each call waits for the previous one	ms
L_{par}	Total latency when independent calls run concurrently	ms

WORKED EXAMPLE

Worked example — The QuickBite home screen over a mobile network. The screen needs 8 calls, each taking $s = 20$ ms in its service. The phone's round trip is $R_{\text{mobile}} = 100$ ms; inside the datacenter $R_{\text{dc}} = 1$ ms.

1. Phone calls sequentially: $L = 8 \times (100 + 20) = 960$ ms.
2. Phone calls in parallel: $100 + 20 = 120$ ms in theory, but dependent calls (recommendations need the address) typically force about three waves: $3 \times 120 = 360$ ms.
3. BFF, one phone round trip, sequential inside: $L = 100 + 8 \times (1 + 20) = 268$ ms.
4. BFF with three parallel waves inside: $L = 100 + 3 \times 21 = 163$ ms.

Interpretation. Paying the expensive round trip once and the cheap ones many times saves 0.8 s against the naive design and 0.2 s against the best realistic client design. **Engineering implication.** Aggregate where round trips are cheapest, parallelize independent calls, and give each a deadline so late optional data (promotions) is dropped rather than awaited.

Table 34.3 — Gateway, BFF and GraphQL compared.

	Single API gateway	Gateway + BFF per client	GraphQL federation (§8.7)
Advantages	Simple; one place for policy	UI teams shape their APIs; fewer round trips	Clients choose fields; one endpoint
Disadvantages	Accumulates per-client logic	Duplication across BFFs; more deployables	Query cost control; harder caching
Choose when	Few, similar client types	Several client types and teams	Many clients with varied data needs

The main BFF risk is **duplication**: "is this restaurant open?" implemented three times and diverging. Push domain logic into the owning service; keep BFFs to aggregation and presentation.

34.7 Service discovery

DEFINITION

Definition — Service discovery. The mechanism by which a client finds the current network locations of healthy instances of a service. A **service registry** stores instances, locations and health; a **lookup** path lets a client (client-side discovery) or a router (server-side discovery) query it [Richardson 2018].

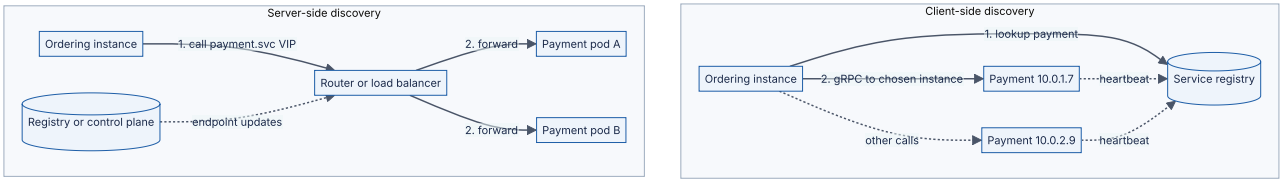
In containers, instances appear and disappear with autoscaling, deploys and failures, and each gets a new IP address; hard-coded addresses cannot keep up. Think of a hospital switchboard: doctors check in when they reach a ward and out when they leave, and the operator connects callers to wherever the doctor is. A doctor who forgets to check out gets calls put through to an empty room until someone notices. Registration is checking in; health checks are how the operator notices.

34.7.1 Registration and health

With **self-registration**, an instance registers on start-up, sends heartbeats and deregisters on shutdown; missing heartbeats expire its lease (Netflix Eureka) [Netflix Eureka Docs]. With **third-party registration**, the platform registers instances: Kubernetes maintains **EndpointSlices** for each Service, including only pods whose **readiness probe** passes [Kubernetes Docs: Services; Kubernetes Docs: Probes], and Consul agents register local services and run their checks [HashiCorp Consul Docs]. A registry listing dead instances is worse than none, so distinguish liveness (restart?), readiness (send traffic?) and **passive** health, where callers eject an instance after consecutive errors (Envoy's outlier detection) (§19.5).

34.7.2 Client-side versus server-side discovery

Figure 34.6 — Client-side and server-side service discovery.



Reading the diagram. Client-side (top): Ordering queries the registry (a replicated store, the storage boundary), caches the list, picks an instance with a client-side policy (§19.10) and calls it directly over gRPC; instances heartbeat asynchronously (dotted). Server-side (bottom): Ordering calls one stable name or virtual IP; a router (load balancer, `kube-proxy` rules or a sidecar) receives endpoint updates asynchronously and forwards the request. Failure points: stale entries send traffic to dead instances in both designs; the server-side router is an extra hop that must be redundant. Scaling: registries are read-heavy and serve cached lookups; clients must keep working on their last-known list if the registry is unreachable.

Table 34.4 — Client-side versus server-side discovery.

	Client-side	Server-side
Advantages	No extra hop; per-request balancing	Simple, language-agnostic clients
Disadvantages	Discovery logic in every language's library	Extra hop; router must be highly available
Examples	Eureka client; gRPC name resolvers; Consul API	Kubernetes Services and DNS; cloud load balancers; mesh sidecars
Choose when	Few languages, latency-sensitive RPC	Polyglot fleet, platform-managed infrastructure

A service mesh (§34.10) blurs the line: the sidecar balances client-side on the application's behalf.

34.7.3 Registries: Consul, Eureka and Kubernetes DNS

- **Consul** replicates its catalog across server nodes with Raft (§26.7), so catalog writes are strongly consistent; per-node agents run health checks and gossip membership. Lookups use an HTTP API or DNS (`payment.service.consul`) and return only passing instances [HashiCorp Consul Docs].
- **Eureka** favors availability: servers replicate asynchronously, clients cache the registry and keep using it if Eureka is unreachable, and in **self-preservation mode** Eureka stops expiring instances when heartbeats drop suddenly, assuming a network problem rather than mass death. It is an AP design (§25.8) [Netflix Eureka Docs].
- **Kubernetes DNS** gives each Service the name `<service>.<namespace>.svc.cluster.local` . A normal Service resolves to a virtual IP spread by `kube-proxy` ; a **headless** Service resolves to pod IPs, suiting client-side balancing such as gRPC's [Kubernetes Docs: DNS for Services and Pods].

WARNING / TRADE-OFF

Warning — DNS caching and long-lived connections defeat fast discovery. DNS is only as fresh as its TTL, and some clients cache longer. A gRPC client that resolved a headless Service once keeps its connections to old pods until they close. Use short TTLs, connection maximum ages or push-based discovery.

Mathematical foundations: how long does a dead instance keep receiving traffic?

FORMULA

Stale-endpoint window and failed requests

$$T_{\text{stale}} \le k \cdot h + t_{\text{prop}} + t_{\text{cache}} \qquad F \approx \frac{\lambda}{m} \cdot T_{\text{stale}} \cdot (1 - q)$$

Symbol	Meaning	Units
h	Heartbeat or health-check interval	s
k	Consecutive missed checks before the instance is removed	count
t_{prop}	Time for the registry change to reach clients or routers	s
t_{cache}	Maximum additional client cache age	s
T_{stale}	Worst-case time a dead instance stays in some client's list	s
λ	Total request rate to the service	requests/s
m	Number of instances (traffic share of the dead one is $1/m$)	count
q	Fraction of failed calls rescued by a retry on another instance or by passive ejection	dimensionless
F	Requests that fail during the window	requests

WORKED EXAMPLE

Worked example — A Payment instance dies abruptly. Payment receives $\lambda = 2,000$ requests/s over $m = 20$ instances. Option A: heartbeats with $h = 10$ s, $k = 3$, propagation 2 s, client caches refreshed every 30 s. Option B: Kubernetes readiness defaults (`periodSeconds: 10` , `failureThreshold: 3`), about 2 s propagation, no client cache.

1. Option A: $T = 3 \times 10 + 2 + 30 = 62$ s; the dead instance's share is $2,000/20 = 100$ requests/s, so without retries $F = 100 \times 62 = 6,200$ failed requests.
2. Option B: $T = 30 + 2 = 32$ s; $F = 3,200$. (Kubernetes often notices a crashed pod sooner through its container or node status.)
3. Add passive ejection after 5 consecutive errors per caller and one retry elsewhere for idempotent calls ($q \approx 0.99$): option A falls to about $6,200 \times 0.01 \approx 62$ failures.

Interpretation. Periodic detection plus caching leaves tens of seconds of failures. **Engineering implication.** Do not rely on the registry for fast failure handling: combine it with passive ejection and bounded retries of idempotent calls (§22.12, §22.15). Shortening checks to 1 s with $k = 1$ would eject healthy instances during every garbage-collection pause.

INDUSTRY EXAMPLE

Eureka at Netflix. Netflix built Eureka for AWS, where instance addresses changed constantly and the load balancers of the time did not suit mid-tier service-to-service traffic. Its availability-first design (client caching, self-preservation) reflects the judgment that during a network problem a possibly stale list beats no list. *Evidence: Primary* — [Netflix Eureka Docs]

Failure scenarios

What goes wrong	Why	Detection	Recovery	Prevention
Traffic to dead instances	Stale registry or caches	Errors to specific IPs	Passive ejection; retry elsewhere	Short leases; push updates
Registry outage	Lost quorum or overload	Lookup and registration errors	Clients use cached lists	Replicated registry; tolerate its absence
Mass deregistration	Health check tests a shared dependency	Zero healthy endpoints	Fail open: keep last endpoints	Readiness tests the instance only (§19.5.2)

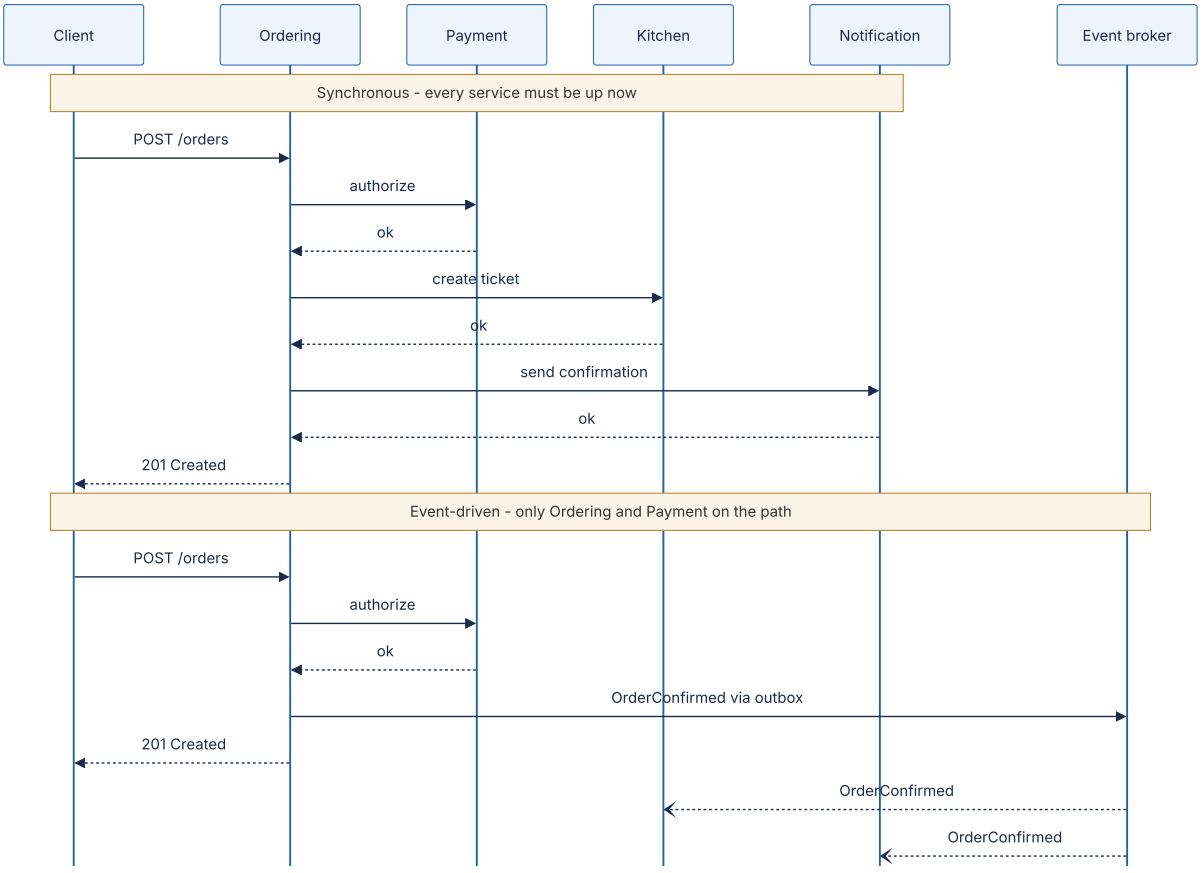
34.8 Inter-service communication

34.8.1 Synchronous versus asynchronous

DEFINITION

Definition — Synchronous and asynchronous inter-service communication. In **synchronous** communication the caller sends a request (REST, gRPC) and cannot complete its work until the response arrives, so caller and callee must be available at the same time. In **asynchronous** communication the sender publishes a message (event or command) to a broker or log and continues; the receiver processes it later (§30.3).

Figure 34.7 — Placing an order: synchronous chain versus event-driven flow.



Reading the diagram. In the first half, Ordering calls Payment, Kitchen and Notification synchronously in sequence; latencies add, and the request fails if any of four services is down. In the second half only payment authorization stays synchronous (the customer must know whether the card worked); Ordering commits the order and an outbox row in one local transaction (§32.5) and responds. A relay publishes `OrderConfirmed` to the broker, the durable boundary between services, and Kitchen and Notification consume it asynchronously (open arrowheads); if Notification is down for ten minutes, confirmations

are late but orders still succeed. The cost is eventual consistency, and failures after the response need retries, dead-letter queues and, where steps can fail, sagas (§32.7). Consumers scale on their backlog.

Table 34.5 — Choosing a communication style.

	Synchronous (REST, gRPC)	Asynchronous (events via broker)
Use for	Queries; commands whose result is needed now	Propagating facts; work nobody waits for
Advantages	Simple model; immediate errors	Temporal decoupling; load leveling; easy new consumers
Disadvantages	Availability and latency compound along chains	Eventual consistency; duplicates; harder debugging
Failure handling	Timeouts, retries, circuit breakers (Chapter 22)	Idempotent consumers, dead-letter queues (§30.11, §32.10)

REST over HTTP/JSON dominates at the edge; **gRPC** with Protocol Buffers (§8.8) is common internally for its compact encoding, generated stubs, streaming and built-in deadlines. For asynchronous paths, choose a log such as Kafka when you need replay and per-key ordering, and a broker such as RabbitMQ for routed work queues (§30.12).

RECOMMENDATION

Recommendation — Keep one synchronous call on the critical path where you can. For each synchronous call, ask whether the caller needs the answer before it can respond. If not, publish an event. If it needs data rather than an action (a restaurant's name), keep a local event-fed copy (§34.11.2).

34.8.2 Chatty calls

A function call in a monolith can become an expensive **N+1 call** pattern across the network. QuickBite's order-history page shows 50 orders, and a naive Ordering service calls Catalog once per order line for item names.

WORKED EXAMPLE

Worked example — Chatty versus batched calls. A Catalog call costs $R = 0.5$ ms plus $s = 2$ ms; a batched call costs $0.5 + 5$ ms. The page has 150 order lines.

1. One call per line, sequential: $L = 150 \times 2.5 = 375$ ms (formula of §34.6.2).
2. Ten concurrent: $15 \times 2.5 = 37.5$ ms, but at 200 page views/s Catalog receives $150 \times 200 = 30,000$ RPS for this one page.
3. One batched `GetItems(ids)` call: $L = 5.5$ ms and 200 RPS.
4. A local copy of item names fed by `MenuItemChanged` events: no network call at read time.

Interpretation. Batching is 68 times faster and puts 150 times less load on Catalog. **Engineering implication.** Design inter-service APIs for the caller's access pattern (batch lookups, coarse operations, pagination). Dozens of calls to one service per page signal an API that is too fine-grained or a misplaced boundary.

34.8.3 Fan-out and tail latency

A composer that calls N services in parallel waits for the slowest. By §4.8.4, if each is fast with probability p, all are fast with probability p^N.

WORKED EXAMPLE

Worked example — The home-screen fan-out. The Mobile BFF calls 6 services in parallel, each with p99 = 50 ms.

1. $P(\text{all six} \le 50 \text{ ms}) = 0.99^6 = 0.9415$.
2. So 5.9% of requests wait longer than 50 ms for some dependency: the BFF's p94 is roughly the dependencies' p99.
3. At 100-way fan-out, $0.99^{100} = 0.366$, the *Tail at Scale* example of §4.8.4.

Interpretation. Each parallel dependency pulls the overall tail toward the dependencies' tails. **Engineering implication.** Give each call a deadline, render without optional data when it is late, hedge idempotent reads (§23.6), and above all reduce the number of dependencies per request.

34.8.4 API versioning across services

The rules of §8.15 apply, with one advantage: you can know every internal consumer and verify compatibility before deploying.

1. **Additive changes by default.** Add optional fields; never remove, rename, retype or tighten in place. Consumers are **tolerant readers** that ignore unknown fields

[Fowler 2011: TolerantReader]. In Protobuf, removed tags are marked `reserved`.
2. **Expand and contract** for breaking changes: add the new form, migrate consumers, confirm the old form has no traffic, then remove it, each step a separate release.
3. **Explicit versions only when needed** (`/v2/payments`, `payments.v2`), keeping at most two alive.
4. **Version events too.** Consumers may replay old events, so use a schema registry with compatibility checks (§8.9, §30.6).

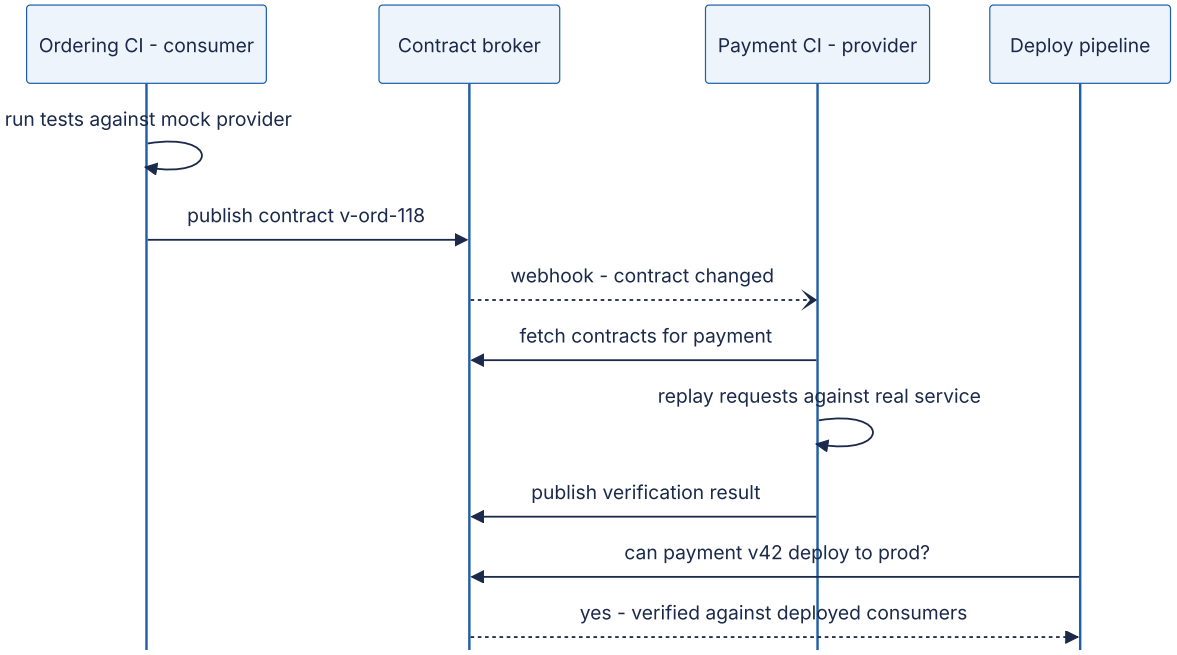
34.8.5 Contract testing with consumer-driven contracts

End-to-end tests that deploy every service together are slow, flaky and reintroduce the coordination microservices were meant to remove. Hand-written mocks test the consumer only against its own assumptions, which drift.

> DEFINITION
>
> **Definition — Consumer-driven contract (CDC).** A specification, generated by the consumer's tests, of the requests a consumer sends to a provider and the parts of the responses it relies on. The provider verifies every consumer's contract against its real implementation in its own build, so a breaking change fails before deployment [Robinson 2006; Pact Docs].

1. **Consumer test.** Ordering's tests run its real client code against a mock provider generated from declared interactions ("given payment p-1 is authorized, `GET /payments/p-1` returns 200 with a string `status`"), producing a contract (a *pact*).
2. **Publish** the contract to a **broker**, tagged with the consumer's version.
3. **Provider verification.** Payment's CI sets up each **provider state**, replays each request against the real service and checks the response, matching by type where the consumer asked.
4. **Deployment gate.** Before deploying, ask the broker whether this version is verified against the versions deployed in the target environment (Pact's `can-i-deploy`) [Pact Docs].

Figure 34.8 — Consumer-driven contract testing workflow.



Reading the diagram. The consumer's CI publishes the contract synchronously; the broker notifies the provider's CI asynchronously by webhook; the provider verifies against its real service and publishes the result. The broker is the storage boundary recording which versions are verified and deployed where, and the deploy pipeline queries it. Failure points: over-specified contracts (literal values) fail on harmless changes, under-specified ones miss breakage, and the broker becomes a release dependency. Cost grows with real consumer–provider pairs, not with the square of the number of services.

Code example

The program below implements the mechanism in about 60 lines (Pact adds many matchers, HTTP mock servers, message contracts and the broker). Provider version 1 passes despite returning extra fields; version 2 renames `status` to `state` and fails its own build.

```
        import json

        # ---- A tiny consumer-driven contract library (the idea behind Pact) --------
        def like(example):
            """Matcher: any value of the same type as the example is acceptable."""
            return {"$like": example}


        def matches(expected, actual, path="body"):
            """Return a list of mismatches; extra fields in `actual` are allowed."""
            if isinstance(expected, dict) and "$like" in expected:
                ex = expected["$like"]
                return [] if type(actual) is type(ex) else [f"{path}: expected
{type(ex).__name__}, got {actual!r}"]
            if isinstance(expected, dict):
                if not isinstance(actual, dict):
                    return [f"{path}: expected object"]
                errs = []
                for key, sub in expected.items():
                    if key not in actual:
                        errs.append(f"{path}.{key}: missing")
                    else:
                        errs += matches(sub, actual[key], f"{path}.{key}")
                return errs
            return [] if expected == actual else [f"{path}: expected {expected!r}, got
{actual!r}"]


        def example_of(expected):
            """Turn matchers back into a concrete example response for the mock."""
            if isinstance(expected, dict) and "$like" in expected:
                return expected["$like"]
            if isinstance(expected, dict):
                return {k: example_of(v) for k, v in expected.items()}
            return expected


        class Contract:
            def __init__(self, consumer, provider):
                self.doc = {"consumer": consumer, "provider": provider, "interactions":
[]}

            def given(self, state, request, response):
                self.doc["interactions"].append(
                    {"state": state, "request": request, "response": response})

            def mock_provider(self, request):
                for i in self.doc["interactions"]:
                    if i["request"] == request:
                        r = i["response"]
                        return r["status"], example_of(r["body"])
                raise AssertionError(f"unexpected request {request}")


        def verify(contract_json, provider_handler, set_state):
            """Provider side: replay every interaction against the real handler."""
```

```
            failures = []
            for i in json.loads(contract_json)["interactions"]:
                set_state(i["state"])
                status, body = provider_handler(i["request"])
                if status != i["response"]["status"]:
                    failures.append(f"[{i['state']}] status {status} != {i['response']
['status']}")
                failures += [f"[{i['state']}] {m}" for m in matches(i["response"]
["body"], body)]
            return failures


        # ---- Consumer side (Ordering service) ---------------------------------------
        def fetch_payment_status(http_get, payment_id):     # the consumer's real code
            status, body = http_get({"method": "GET", "path":
f"/payments/{payment_id}"})
            if status == 404:
                return None
            return body["status"], body["amount"]["minor"]


        contract = Contract("order-service", "payment-service")
        contract.given("payment p-1 is authorized",
                       {"method": "GET", "path": "/payments/p-1"},
                       {"status": 200, "body": {"id": "p-1", "status":
like("AUTHORIZED"),
                                                "amount": {"minor": like(2450),
                                                           "currency": like("USD")}}})
        contract.given("payment p-404 does not exist",
                       {"method": "GET", "path": "/payments/p-404"},
                       {"status": 404, "body": {}})

        # Consumer test runs against the mock built from the contract, then publishes
it.
        assert fetch_payment_status(contract.mock_provider, "p-1") == ("AUTHORIZED",
2450)
        assert fetch_payment_status(contract.mock_provider, "p-404") is None
        published = json.dumps(contract.doc)               # would go to a contract bro-
ker


        # ---- Provider side (Payment service), two versions --------------------------
        DB = {}

        def set_state(state):
            DB.clear()
            if state == "payment p-1 is authorized":
                DB["p-1"] = {"status": "AUTHORIZED", "minor": 9900, "currency": "EUR"}

        def handler_v1(req):
            pid = req["path"].rsplit("/", 1)[1]
            if pid not in DB:
                return 404, {}
            p = DB[pid]
            return 200, {"id": pid, "status": p["status"], "created_at": "2026-10-
30T12:00:00Z",
                         "amount": {"minor": p["minor"], "currency": p["currency"]}}
```

```
        def handler_v2(req):                    # a refactor renames status -> state
            status, body = handler_v1(req)
            if status == 200:
                body["state"] = body.pop("status")
            return status, body

        print("v1 failures:", verify(published, handler_v1, set_state))
        print("v2 failures:", verify(published, handler_v2, set_state))
        assert verify(published, handler_v1, set_state) == []
        assert verify(published, handler_v2, set_state) == [
            "[payment p-1 is authorized] body.status: missing"]
        print("contract tests behave as expected")
```

Running it with `python3` prints:

```
v1 failures: []
v2 failures: ['[payment p-1 is authorized] body.status: missing']
contract tests behave as expected
```

What to notice:

- **The consumer's real client code runs** against the mock, so the contract cannot claim what the client does not use.
- **Matching by type.** The provider returns 9900 EUR against an example of 2450 USD and still passes, because the consumer declared `like(...)`. Literal values are the commonest cause of brittle contracts.
- **Extra fields are allowed** (`created_at`): only what the consumer relies on is checked.
- **Provider states** seed data per interaction, making verification repeatable.
- **The breaking rename fails in Payment's build**, naming the consumer's expectation, before deployment.

> COMMON MISTAKE
>
> **Common misconception — "Contract tests replace provider functional tests."** A contract checks shape, not business correctness: it passes if Payment returns `AUTHORIZED` for a card that should be declined. Contracts replace most cross-service end-to-end tests, not unit tests [Pact Docs].

34.9 Distributed tracing (introduction)

A slow "place order" request passes through the gateway, BFF, Ordering, Payment and an external provider, each logging on a different machine. Which made it slow? This section

introduces the concepts the chapter needs; Chapter 40 (§40.6–§40.7) covers sampling, storage, OpenTelemetry and analysis.

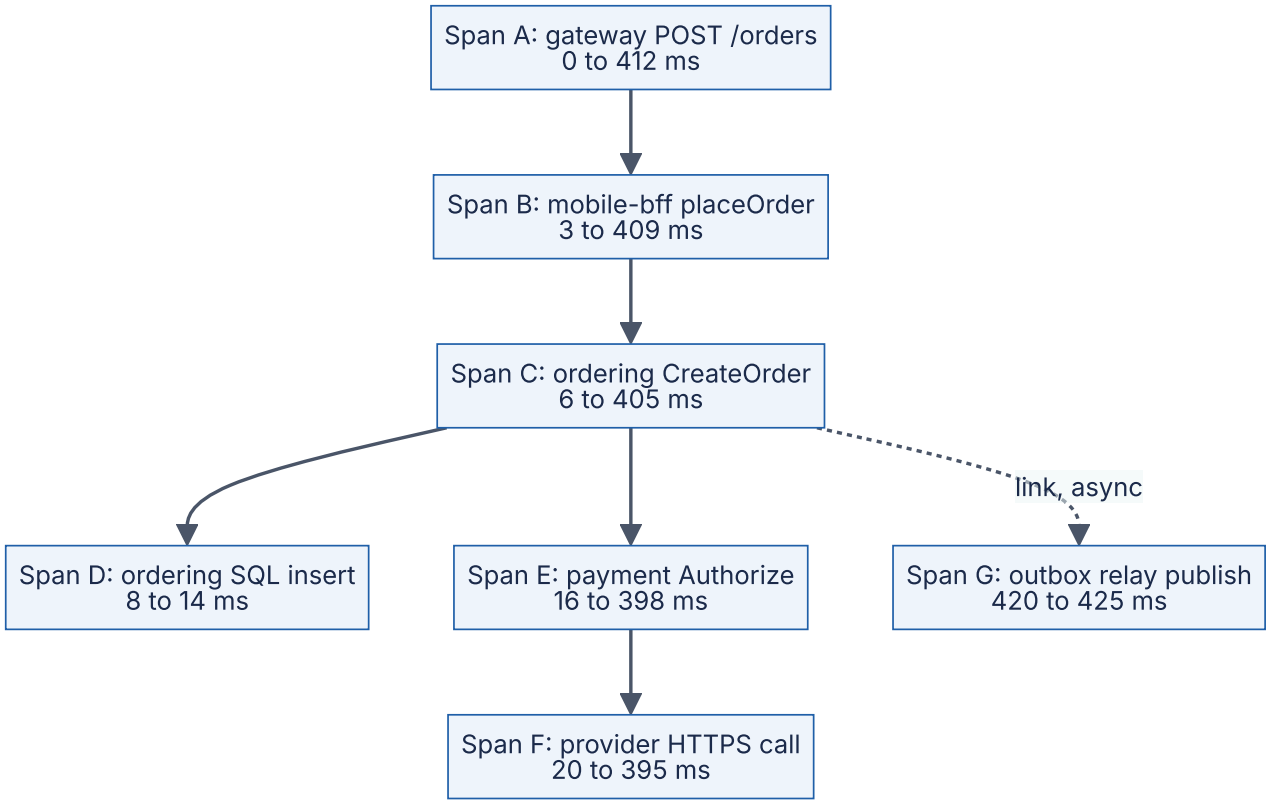
34.9.1 Traces, spans and context propagation

DEFINITION

Definition — Trace, span and context propagation. A **trace** records one request's path through a distributed system under a unique **trace ID**. It is a tree of **spans**, each recording one unit of work (an inbound request, an outbound call, a query) with service, operation, start time, duration, status, its span ID and its **parent span ID**. **Context propagation** passes the trace ID and current span ID from caller to callee (in HTTP headers or message metadata) so the callee's spans join the trace [Sigelman et al. 2010; W3C Trace Context].

Google's Dapper paper describes the motivating problem: a web search touched many services built by different teams, and engineers could not tell which caused a slow query. Dapper aimed for low overhead and application transparency by instrumenting shared RPC and threading libraries, and sampled requests (its first production version used a uniform probability of 1/1024) [Sigelman et al. 2010].

Figure 34.9 — A trace of "place order" as a span tree.



Reading the diagram. Each box is a span; solid arrows point from parent to child, all under one trace ID, with times as offsets from the root. Span F, the external provider's HTTPS call, takes 375 ms of 412 ms, so tuning Ordering's 6 ms SQL insert would not help. Span G starts after the response (420 ms) because the outbox relay publishes asynchronously; it is attached by a **span link** (dotted) rather than as a child. Services export spans asynchronously to a collector and trace store, never on the request path. Failure points: any hop that drops the context splits the trace. Scaling: traces are sampled (Chapter 40).

The first instrumented component creates the trace ID, root span and sampling decision. Each outbound call carries the W3C `traceparent` header with a version, 16-byte trace ID, 8-byte parent span ID and flags, for example `traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01` [W3C Trace Context]. Callees create child spans; message consumers usually start linked spans.

WARNING / TRADE-OFF

Warning — Propagation is only as good as the weakest hop. Thread pools, queues and proxies that drop headers break traces; use shared instrumentation (OpenTelemetry, §40.7). A service mesh emits spans per hop but cannot connect an inbound request to the application's outbound calls: the application must still forward the context headers [Istio Docs: Distributed Tracing].

34.10 Service mesh

34.10.1 What a mesh is and why it exists

Every inter-service call needs discovery and balancing (§34.7), timeouts, retries and circuit breaking (Chapter 22), mutual TLS (§10.10.2), telemetry (§34.9) and traffic control for canaries (§34.12). Libraries can provide all of this, as Netflix's Java libraries did, but that ties features to one language and makes every fix a fleet-wide library upgrade. A **service mesh** moves them into infrastructure.

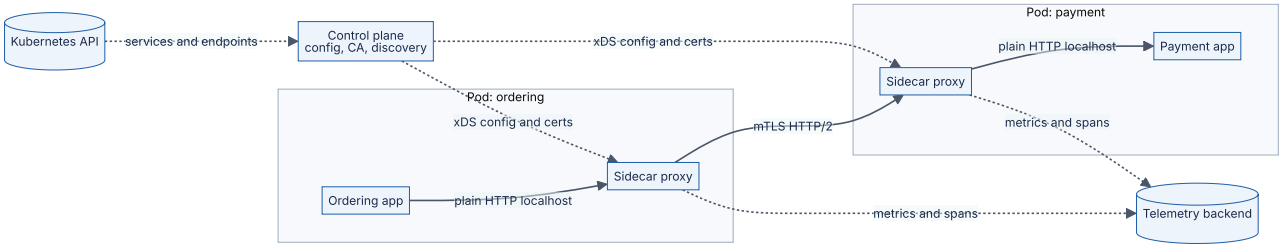
DEFINITION

Definition — Service mesh. An infrastructure layer for service-to-service communication. Its **data plane** is a set of proxies through which all service traffic passes (a **sidecar** per workload instance, or shared per-node proxies); its **control plane** configures the proxies, issues certificates and collects state. It provides traffic management, security (mutual TLS, authorization) and telemetry without application changes [Istio Docs: Architecture; Linkerd Docs: Architecture].

How it works internally

1. **Interception.** The mesh injects a proxy into each pod and redirects the pod's inbound and outbound TCP traffic through it (iptables or a CNI plugin). The application still connects in plain text to `payment:8080`.
2. **Outbound.** The caller's sidecar picks an instance from control-plane endpoints, opens or reuses an mTLS connection to the destination's sidecar, applies timeouts and retries, and records metrics and a span.
3. **Inbound.** The callee's sidecar terminates mTLS, verifies the caller's workload identity (a SPIFFE ID, §10.10.2), enforces policy ("only `ordering` may call `POST /charge`") and forwards on localhost.
4. **Control plane.** It watches services, endpoints and policy resources, pushes proxy configuration (Istio over Envoy's xDS APIs), and acts as a certificate authority issuing short-lived, automatically rotated certificates [Istio Docs: Architecture].

Figure 34.10 — Sidecar service mesh: data plane and control plane.



Reading the diagram. The Ordering application sends a plain request to localhost; its sidecar carries it over mutual TLS (HTTP/2) to the Payment sidecar, which checks identity and policy and forwards to the application. Solid arrows are the synchronous path: one logical call crosses two proxies. Dotted arrows are asynchronous: the control plane reads services and endpoints from the Kubernetes API (a storage boundary) and pushes configuration and certificates; proxies export telemetry. Failure points: if the control plane is down, proxies keep their last configuration, so traffic flows but deploys and certificate rotation stall; a bad pushed configuration can affect every service at once. Scaling: data-plane cost grows with pods; control-plane push cost grows with proxies times configuration size.

34.10.2 Features

- **Mutual TLS everywhere** with automatic certificates and identity-based authorization: often the main reason to adopt a mesh, since it delivers zero-trust service security without code changes.
- **Traffic management.** Per-route retries, timeouts, circuit breaking and outlier detection; **traffic shifting** by weight (5% to `payment` v2, §34.12); header-based routing; mirroring live traffic to a new version.

- **Telemetry.** Uniform request rate, error and latency metrics for every service pair, in any language.

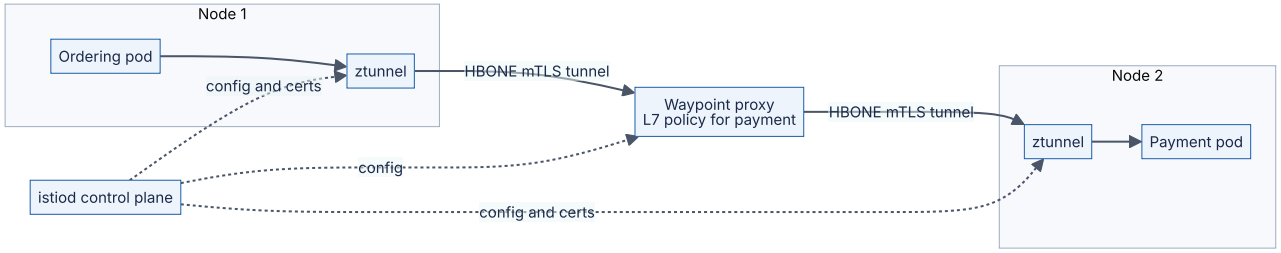
> WARNING / TRADE-OFF
>
> **Warning — Mesh retries multiply with application retries.** Three application attempts times three sidecar attempts is nine at the destination, more down a chain (§22.12.3). Choose one retrying layer.

34.10.3 Istio, Linkerd and sidecarless (ambient) mode

Istio uses Envoy sidecars and the `istiod` control plane. **Linkerd** uses its own lightweight Rust proxy, `linkerd2-proxy`, built specifically as a sidecar, emphasizing simplicity and low resource use [Linkerd Docs: Architecture]. Both offer automatic mTLS, retries, timeouts, traffic splitting and golden metrics; Istio has the larger feature and configuration surface.

Sidecars cost one proxy per pod, two proxy traversals per call, pod restarts for proxy upgrades and start-up ordering problems. As of 2026, Istio's **ambient mode**, generally available since Istio 1.24 (November 2024), removes the sidecar and splits the data plane in two [Istio Blog 2024]: **ztunnel**, a per-node Rust proxy providing L4 features (mTLS, identity, L4 authorization, TCP telemetry), and **waypoint proxies**, Envoy-based and deployed per namespace or service only where L7 features (HTTP routing, retries, L7 policy) are needed. Linkerd, as of 2026, remains sidecar-based; other sidecarless designs, such as Cilium's, move parts of the data plane into the kernel with eBPF. This area changes quickly; check current documentation.

Figure 34.11 — Istio ambient mode: per-node ztunnel with an optional waypoint.



Reading the diagram. Pods contain no proxy. Ordering's traffic is redirected to its node's ztunnel, which wraps it in an mTLS tunnel (HBONE) and, because Payment has L7 policy, sends it to Payment's waypoint, which applies HTTP routing, retries and authorization before forwarding through Node 2's ztunnel. Without L7 policy, traffic goes ztunnel to ztunnel. The control plane configures all three asynchronously. Failure points: a ztunnel failure affects every pod on its node, a larger blast radius than one sidecar; the waypoint is a shared hop to scale and replicate. Scaling: proxy count grows with nodes, not pods, and only services needing L7 pay for it.

Mathematical foundations: what does the mesh cost?

FORMULA

Mesh latency and resource overhead

$$\Delta L_{\text{chain}} \approx h \cdot \delta_{\text{hop}} \qquad C_{\text{CPU}} = \frac{\Lambda \cdot x}{1,000} \cdot c \qquad M = P \cdot m$$

Symbol	Meaning	Units
h	Number of sequential service-to-service hops on the request path	hops
δ_{hop}	Latency added per hop by the proxies on that hop (two sidecars, or two ztunnels)	ms
ΔL_{chain}	Added latency along the chain (an approximation: percentiles do not add exactly)	ms
Λ	Total internal service-to-service request rate	requests/s
x	Proxies traversed per call (2 for sidecar-to-sidecar)	count
c	Proxy CPU per 1,000 requests/s	vCPU
C_{CPU}	Fleet-wide CPU spent in proxies	vCPU
P	Number of proxies (pods for sidecars, nodes for ztunnels)	count
m	Memory per proxy	MB
M	Fleet-wide proxy memory	MB

WORKED EXAMPLE

Worked example — Meshing QuickBite. QuickBite runs 400 pods on 40 nodes, with $\Lambda = 20,000$ internal requests/s and a longest synchronous path of $h = 4$ hops. Istio's published measurements (1 KB HTTP requests, mutual TLS, specific hardware; as of 2026) report: two sidecars per hop add about 0.18 ms at p90 and 0.25 ms at p99; two ztunnels add about 0.17 ms and 0.20 ms; a two-worker sidecar uses about 0.20 vCPU and 60 MB per 1,000 requests/s; a ztunnel about 0.06 vCPU and 12 MB [Istio Docs: Performance and Scalability].

1. Sidecar latency on the longest path: $\Delta L \approx 4 \times 0.25 = 1.0$ ms at p99.
2. Sidecar CPU: $C = (20,000 \times 2/1,000) \times 0.20 = 8$ vCPU.
3. Sidecar memory: $M = 400 \times 60 = 24,000$ MB $= 24$ GB (an under-load figure; it grows with configuration size).
4. Ambient, L4 only: CPU $= (20,000 \times 2/1,000) \times 0.06 = 2.4$ vCPU (two ztunnels per call); memory $= 40 \times 12 = 480$ MB; latency $\approx 4 \times 0.20 = 0.8$ ms at p99, plus waypoints where L7 is needed.

Interpretation. About 1 ms of p99 is negligible in a 300 ms budget but half of a 2 ms cache-path budget; 8 vCPU and 24 GB is two or three extra nodes. **Engineering implication.** The mesh suits typical request/response services; for very high-rate, low-latency paths, measure first. Ambient mode cuts the per-pod memory cost most. Vendor numbers are best cases; benchmark on your hardware and payloads.

Advantages and disadvantages. A mesh gives uniform mTLS and policy without code changes, consistent retries and telemetry across languages, and traffic shifting. It costs a complex system to operate and upgrade, latency and resources, fleet-wide blast radius for configuration errors, and overlap with library features that must be reconciled.

RECOMMENDATION

Recommendation — Adopt a mesh for a specific problem: mandatory service-to-service encryption and authentication, a polyglot fleet needing uniform telemetry and traffic policy, or fine-grained traffic shifting. With ten services in one language, a well-configured RPC library is simpler.

34.11 Data ownership and database-per-service

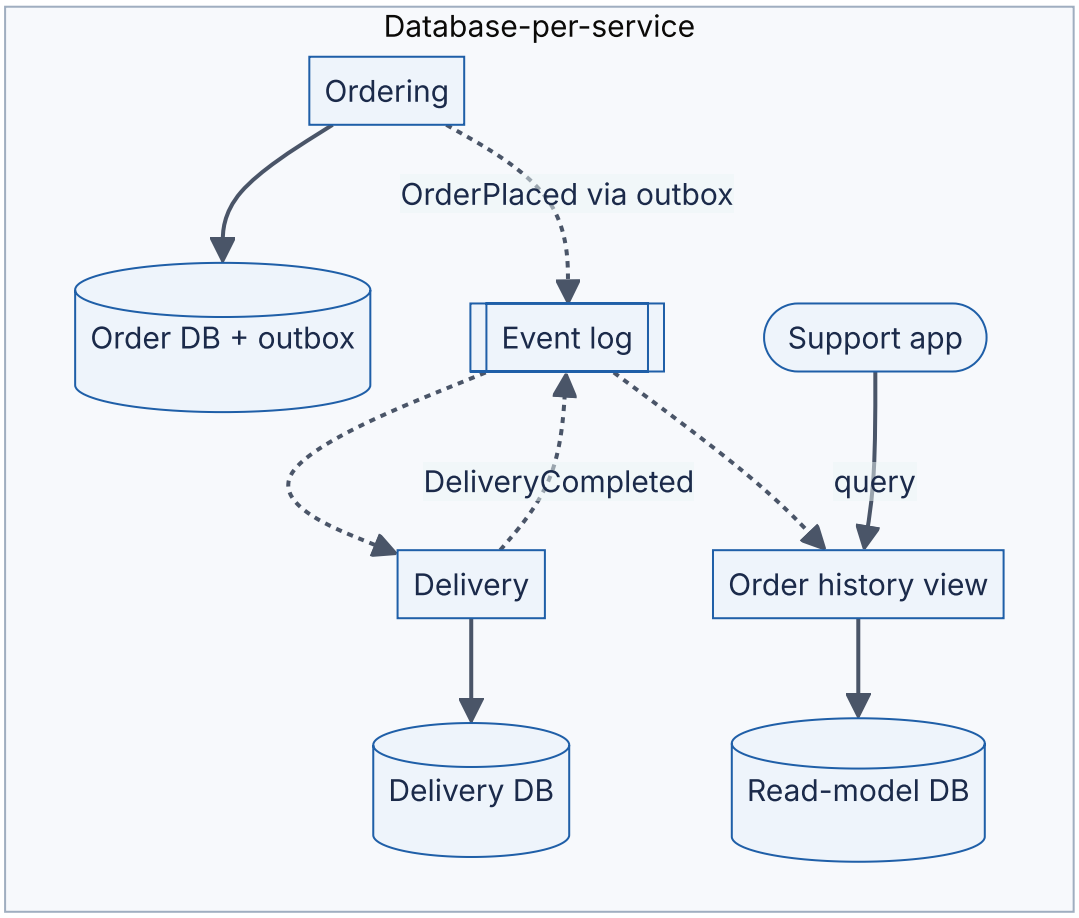
34.11.1 Database-per-service and the shared-database anti-pattern

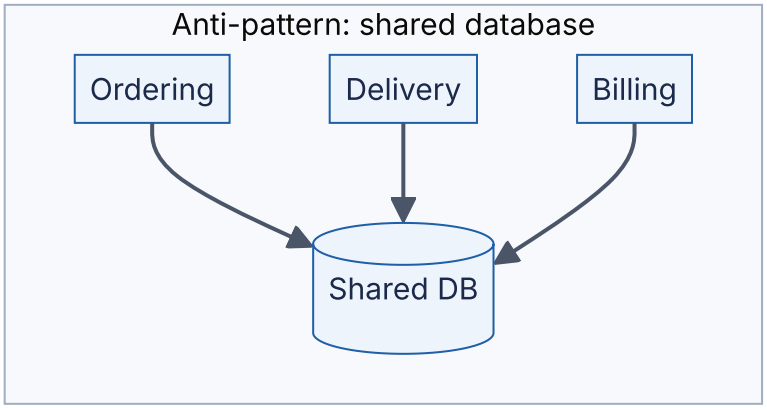
DEFINITION

Definition — Database-per-service. Each service's persistent data is private to that service and reachable by others only through its API or published events. The service may use its own database server, its own schema on a shared server, or private tables, provided no other service reads or writes them [Richardson 2018].

The opposite is the **shared database**: several services read and write the same tables. It looks efficient, since joins are free and there is one source of truth, but it is content coupling (§34.3.2) at scale. Nobody can change a column without finding every service that uses it; one service's heavy query slows the others; and invariants are enforced by whichever service wrote last. The schema becomes the real, unversioned API of the system.

Figure 34.12 — Shared database versus database-per-service with an event-fed read model.





Reading the diagram. On the left, three services share one database: there is one storage boundary for three owners, every table is a shared interface, and a lock or slow query from one service hurts all three. On the right, each service writes only its own database. Ordering commits an order and an outbox row in one local transaction, and a relay publishes `OrderPlaced` to the event log asynchronously (dotted). Delivery keeps the order fields it needs in its own database. An **order history view** service consumes events from both Ordering and Delivery into a denormalized read model that the support app queries synchronously. Failure points: if the event log or relay stops, copies go stale but nothing fails outright; consumers must be idempotent because delivery is at least once (§32.10).

Scaling: each database is sized for its own load, and the read model can be rebuilt by replaying the log.

34.11.2 Replicating data with events

When Delivery needs the customer's address and the restaurant's location, it does not call Ordering for every courier assignment. It keeps a local copy, updated from events. Rules for doing so safely:

1. **One owner per fact.** Ordering owns the order address; Delivery's copy is a cache that it never edits.
2. **Publish reliably.** Use the transactional outbox or change data capture (§32.5–§32.6) so the database and the event log cannot disagree.
3. **Consume idempotently and in order per key.** Partition events by aggregate ID so all events for one order arrive in order (§31.5), and ignore events whose version is not newer than the copy's.
4. **Carry enough state.** "Event-carried state transfer" puts the changed fields in the event so consumers need not call back.

Cross-service invariants then become **sagas**: a sequence of local transactions with compensations (refund the payment if no courier can be found), coordinated by choreography or an orchestrator. Chapter 32 owns that topic (§32.7–§32.8).

34.11.3 Querying across services

A support agent wants "all orders for customer X in the last month, with delivery times and refund status": data owned by three services. Two patterns exist [Richardson 2018]:

- **API composition.** A composer (a BFF or query service) calls each owner and joins in memory. Simple and always fresh, but its latency is that of the slowest owner, its availability is the product of the owners' (§34.14.3), and large joins (all orders with any refund) need inefficient fan-out.
- **CQRS view.** A dedicated read model, fed by events from the owners, stores the pre-joined answer (Figure 34.12). Queries are fast and independent of the owners' availability, at the cost of replication lag (typically under a second to seconds) and another datastore to run (§32.4).

Table 34.6 — Querying across services.

	API composition	CQRS view
Freshness	Current	Lags by the replication delay
Latency	Slowest owner plus composition	One local query
Availability	Product of owners' availability	Independent of owners at read time
Complexity	Low	Event handling, idempotency, rebuilds
Choose when	Few owners, small result sets	Frequent queries, joins across many owners, search

COMMON MISTAKE

Common misconception — "Database-per-service means one database server per service." It means private *ownership*. Several services may share a managed database cluster with separate schemas and credentials, which is often the cost-effective start, as long as no service can read another's schema. Enforce the boundary with database permissions, not with good intentions.

34.12 Deployment strategies

The defining property of a microservice is that it can be deployed alone. Chapter 43 covers rolling, blue–green and canary deployments and feature flags in depth (§43.4–§43.7); this section lists what microservices add.

1. **One pipeline per service.** Each service builds, tests (including contract verification, §34.8.5) and deploys from its own pipeline, without waiting for other services.
2. **Backward and forward compatibility at every step.** During a rolling deploy, old and new versions of a service run side by side, and consumers on either side may be old or new. Every API, event and schema change therefore uses expand and contract (§34.8.4); database changes too (§43.13).
3. **Progressive delivery.** Shift a small share of traffic to the new version (with a mesh, by weight, §34.10.2), compare its error rate and latency with the old version, and roll forward or back automatically.
4. **Decouple deploy from release.** Ship code dark behind a feature flag, then enable it gradually.

WARNING / TRADE-OFF

Warning — The lockstep release is a symptom. If a release plan says "deploy Payment v42, then Ordering v118, then Billing v77 within the same hour", the services are not independently deployable. Find the incompatible change and split it into compatible steps.

34.13 Migration from a monolith

34.13.1 The strangler fig pattern

> DEFINITION
>
> **Definition — Strangler fig application.** A migration strategy in which a new system is grown around the edges of an old one, taking over one capability at a time while the old system keeps running, until the old system can be retired. Fowler named it after strangler fig vines, which grow around a host tree until they stand on their own [Fowler 2004].

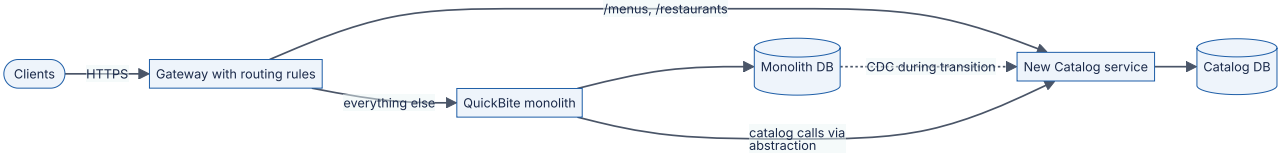
Why does this exist?

The alternative, a “big bang” rewrite, stops feature delivery for months or years, must reproduce every undocumented behavior of the old system, and delivers no value until the end. Incremental migration delivers value early, can be paused at any point, and keeps the old system as a fallback.

How it works internally

1. **Put a routing layer in front** of the monolith (a reverse proxy or the API gateway); initially it sends everything to the monolith.
2. **Pick a capability** with low change coupling to the rest (§34.3.2), clear data ownership and real value from extraction.
3. **Build it as a service**, with its own data. Synchronize data during the transition, for example with change data capture from the monolith’s tables.
4. **Redirect traffic** for that capability’s routes to the new service, gradually (by percentage or by customer), with the ability to switch back.
5. **Remove the dead code** from the monolith, then repeat.

Figure 34.13 — Strangler fig: extracting Catalog from the QuickBite monolith.



Reading the diagram. All clients reach the gateway over HTTPS; routing rules send catalog paths to the new Catalog service and everything else to the monolith (synchronous). During the transition, change data capture streams menu changes still written by old monolith screens into Catalog’s database (dotted, asynchronous), and monolith code that needs menu data calls Catalog through an internal abstraction instead of its own tables. Two storage boundaries now exist, and for a period the monolith database remains the

source of truth for some fields, so ownership must be explicit per field. Failure points: the routing layer (must be redundant), the CDC pipeline (lag shows stale menus) and divergent behavior between old and new code. Scaling: Catalog now scales independently with read traffic.

34.13.2 Branch by abstraction and parallel run

Routing works at the edge. Functionality deep inside the monolith, called by other monolith code, needs **branch by abstraction**: introduce an interface in front of the existing implementation, move all callers to the interface, build the new implementation (a client to the new service) behind it, switch with a flag, then delete the old implementation [Fowler 2014: BranchByAbstraction; Newman 2019]. All of this happens on the main branch with no long-lived feature branch.

For risky logic (pricing, payments), use a **parallel run**: call both old and new implementations, return the old result to the user, and compare the results, logging any differences. GitHub's Scientist library automates this for code paths inside an application [GitHub Scientist]. Only when differences are rare and understood do you switch to the new implementation as the source of truth. Make sure only one side has side effects (only one sends the email or charges the card).

34.13.3 A worked migration plan

Table 34.7 — Migrating QuickBite's monolith over about four quarters (illustrative).

Phase	Step	Exit criterion
0	Gateway in front of monolith; tracing and per-route metrics; modularize code by context	All traffic via gateway; module dependencies mapped
1	Extract Notification (async, low coupling) via `OrderPlaced` events from an outbox	Monolith no longer sends messages
2	Extract Catalog with CDC sync and route-by-route redirection	Catalog DB is source of truth; monolith reads via API
3	Extract Delivery; parallel-run courier assignment for two weeks	Assignment differences below 0.1%, all explained
4	Extract Payment behind branch by abstraction; ACL for provider	Payment tables removed from monolith DB
5	Remaining Ordering core stays a well-modularized service	Decide on evidence whether to split further

The order follows the change-coupling data: the least coupled, lowest-risk capabilities first, so the team learns the platform (pipelines, discovery, tracing) on cheap mistakes.

Payment comes late because it has high coupling with Ordering ($C = 0.43$ in §34.3.2). Phase 5 is a legitimate end state: nothing requires the core to be split.

INDUSTRY EXAMPLE

Shopify's modular monolith. Shopify chose not to split its large Ruby on Rails monolith into microservices, and instead enforced component boundaries inside it, with tooling that detects violations of the declared dependencies between components [Westeinde 2019]. This captures much of the boundary benefit without the network. *Evidence: Primary* — [Westeinde 2019]

34.14 Anti-patterns

34.14.1 The distributed monolith

DEFINITION

Definition — Distributed monolith. A system deployed as multiple services that cannot be changed, deployed or operated independently: services share databases, must be released together, or call each other synchronously so tightly that one failure stops all of them. It has the costs of distribution and the coupling of a monolith [Newman 2021].

Symptoms: coordinated releases (§34.12), a shared database (§34.11), long synchronous call chains, and a single change touching many repositories. The cause is almost always boundaries drawn without regard to cohesion, often by technical layer (Figure 34.1) or by noun ("Customer service", "Product service") instead of by capability.

34.14.2 Nanoservices and shared-library coupling

Nanoservices are services so fine-grained that the network and operational overhead outweighs their function: a service per database table, or per function. Each adds a pipeline, an on-call surface, a versioned API and network hops.

Shared-library coupling occurs when services share a library containing domain model classes or business logic. Updating the library forces every service to rebuild and redeploy, often in lockstep, and the library's model becomes a hidden shared kernel. Sharing infrastructure libraries (logging, tracing, an RPC client) is fine if versions can differ between services; sharing domain models is not.

34.14.3 Synchronous call chains

FORMULA

Availability and latency of a synchronous call chain

$$A_{\text{chain}} = \prod_{i=1}^{n} A_i = A^n \ \text{(equal services)} \qquad L_{\text{chain}} = \sum_{i=1}^{n} (R_i + s_i)$$

Symbol	Meaning	Units
n	Number of services that must all succeed for the request	count
A_i	Availability of service i (independent failures assumed)	dimensionless
A_{chain}	Availability of the request path	dimensionless
R_i, s_i	Round-trip and processing time of hop i	ms
L_{chain}	End-to-end latency of the chain	ms

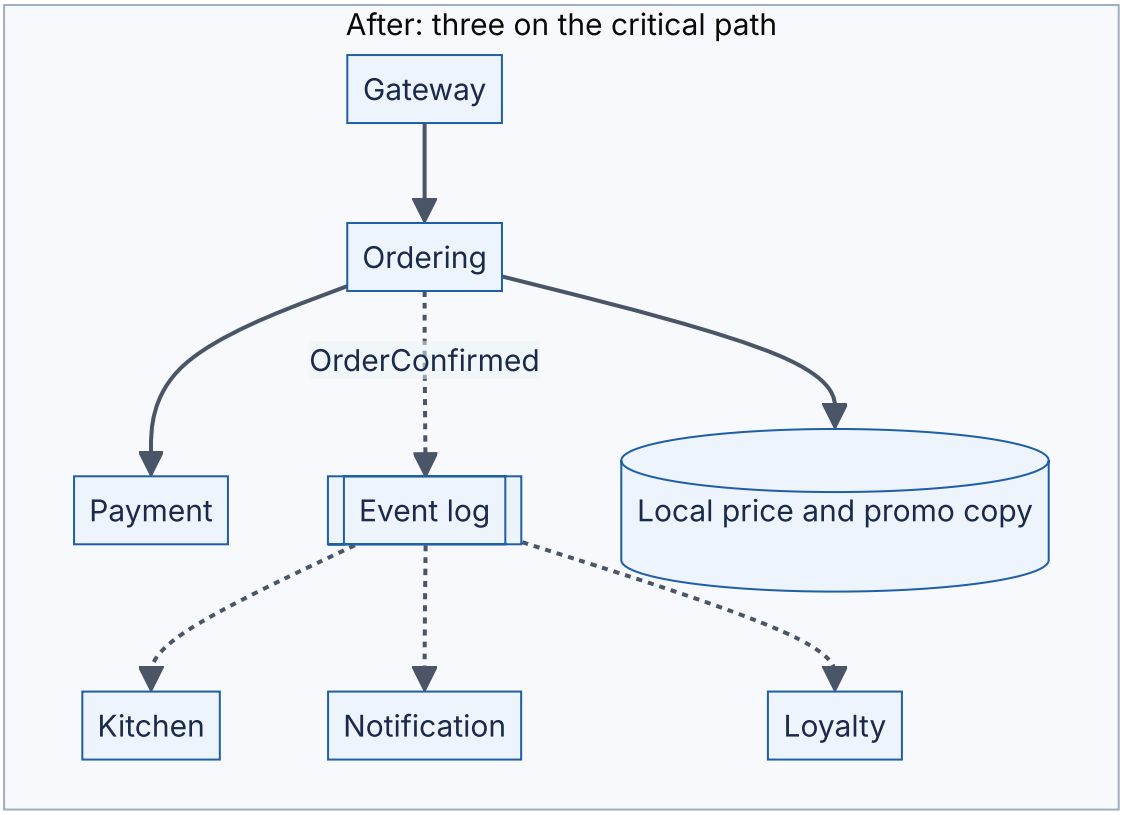
WORKED EXAMPLE

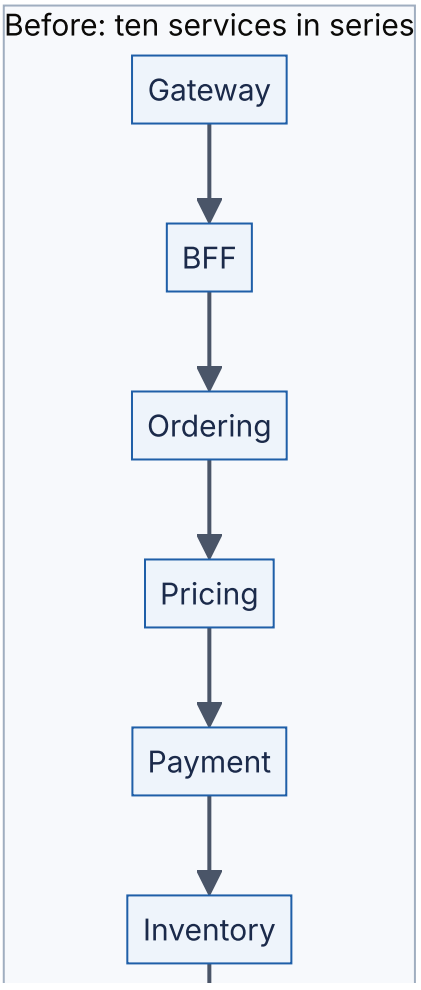
Worked example — QuickBite checkout as a chain. Each service is 99.9% available. A distributed-monolith checkout calls gateway → BFF → Ordering → Pricing → Payment, five in sequence. Over a 30-day month (720 hours):

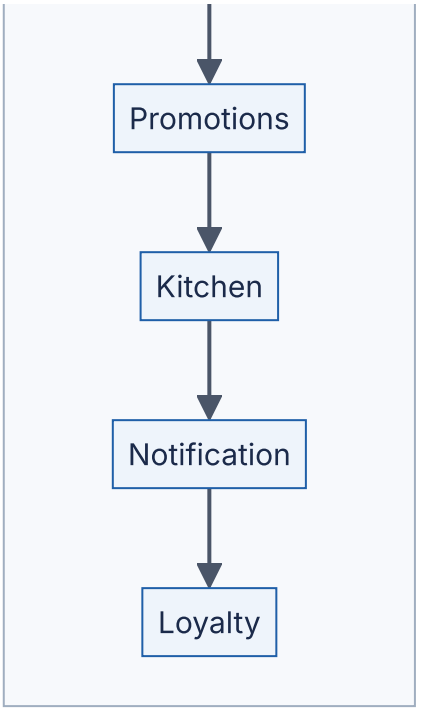
1. Five in series: $A = 0.999^5 = 0.99501$; unavailability $0.00499 \times 720 = 3.59$ hours/month.
2. Ten in series (add Inventory, Promotions, Kitchen, Notification, Loyalty synchronously): $A = 0.999^{10} = 0.99004$; $0.00996 \times 720 = 7.17$ hours/month.
3. Redesign with three services on the critical path (gateway, Ordering, Payment) and the rest asynchronous: $A = 0.999^3 = 0.99700$; 2.16 hours/month.
4. Latency at 0.5 ms network, 0.25 ms mesh and 10 ms processing per hop: ten hops cost $10 \times 10.75 = 107.5$ ms; three cost 32.25 ms.

Interpretation. Every synchronous dependency multiplies in its unavailability: ten 99.9% services give barely two nines. Retries help only against transient faults, and correlated failures (a shared database or cluster) make reality worse than the independence assumption. **Engineering implication.** Count the services on each critical path and keep it short; move non-essential work to events; and for remaining synchronous calls use timeouts, fallbacks and circuit breakers (Chapter 22) so a non-essential dependency degrades the response instead of failing it.

Figure 34.14 — Shortening the critical path.


After: three on the critical path
Gateway
Ordering
OrderConfirmed
Payment
Event log
Local price and promo copy
Kitchen
Notification
Loyalty


Before: ten services in series
Gateway
BFF
Ordering
Pricing
Payment
Inventory



Reading the diagram. Before, ten synchronous calls must all succeed in order; any failure fails checkout, and latency is the sum of all hops. After, only the gateway, Ordering and Payment are on the synchronous path. Pricing and promotion rules are replicated into a local copy that Ordering reads (a storage boundary inside Ordering, fed by events). Kitchen, Notification and Loyalty consume `OrderConfirmed` asynchronously from the event log. Inventory is checked from a local, event-fed availability copy (not shown). Failure points shrink to three services plus the outbox relay, whose failure delays but does not lose events. Scaling: asynchronous consumers scale on backlog.

INDUSTRY EXAMPLE

Two public reversals. Segment described consolidating more than a hundred destination microservices back into a single service after the operational cost of many tiny services, shared-library versions and per-service queues outgrew the benefit [Noonan 2018]. Amazon Prime Video's monitoring team reported cutting infrastructure cost by about 90% by moving an audio/video quality-inspection workload from distributed serverless components to a single process, because passing frames between components dominated the cost [Kolny 2023]. Neither is an argument against services in general; both show granularity chosen without regard to the data flow. *Evidence: Primary* — [Noonan 2018; Kolny 2023]

Table 34.8 — Anti-patterns at a glance.

Anti-pattern	Symptom	Fix
Distributed monolith	Lockstep releases	Re-cut boundaries by capability and change coupling; expand and contract
Shared database	Schema changes need many teams	Database-per-service; events; CQRS views
Nanoservices	More services than engineers	Merge into capability-sized services
Shared domain library	Library bump forces fleet redeploy	Share only infrastructure; each context owns its model
Long synchronous chains	A^n availability; summed latency	Events, local copies, fewer hops
Gateway with business logic	Every change goes through the platform team	Move logic into services or BFFs

34.15 Summary and revision notes

SUMMARY

- A microservice is built around one business capability, owned by one team, independently deployable and the sole owner of its data. Its main benefit is independent change; its main cost is distribution.
- Choose boundaries by cohesion and change coupling ($C = n_{AB}/(n_A + n_B - n_{AB})$ from version history), team ownership and transaction boundaries, not by technical layer or size.
- DDD supplies the model: ubiquitous language, entities, value objects, small aggregates (one transaction, one aggregate; conflict probability $1 - e^{-\lambda w}$) and domain events.
- Bounded contexts are the best first service boundaries. Context maps make relationships explicit; an anticorruption layer keeps foreign models out. Event storming finds the boundaries.
- An API gateway handles edge concerns; BFFs shape APIs per client and save expensive round trips. Keep business logic out of both.
- Service discovery combines a registry, health checks and client-side or server-side lookup. Stale entries persist for about $k \cdot h + t_{\text{prop}} + t_{\text{cache}}$, so add passive ejection and retries.
- Prefer one synchronous call on the critical path and events elsewhere; batch chatty calls; fan-out exposes you to p^N tails; version with expand and contract; verify compatibility with consumer-driven contracts.
- Tracing links spans across services by propagating trace context; Chapter 40 goes deeper.
- A service mesh moves mTLS, retries and telemetry into proxies; sidecars add roughly 0.2–0.25 ms per hop and per-pod memory; Istio’s ambient mode (GA in 1.24) uses per-node ztunnels and optional waypoints.
- Database-per-service is mandatory; share data by events, query with API composition or CQRS views, and keep invariants inside one service or use sagas.
- Migrate with the strangler fig, branch by abstraction and parallel run, least-coupled capabilities first.
- The distributed monolith is the main failure mode; synchronous chains have availability A^n.

34.16 Knowledge checkpoint

1. State four properties of a microservice.
2. Modules A and B were touched by 50 and 40 commits, 30 of them shared. Compute $C(A, B)$ and interpret it.

3. Why is content coupling worse than domain coupling?
4. Distinguish an entity from a value object, with one QuickBite example of each.
5. An aggregate receives 5 writes/s from others and each write takes 40 ms. What is the conflict probability?
6. What is the difference between a conformist relationship and an anticorruption layer?
7. Why should an unknown payment status raise an error in an ACL while an unknown decline code may map to a default?
8. A mobile screen needs 5 sequential calls, mobile RTT 80 ms, service time 10 ms, datacenter RTT 1 ms. Compare direct calls with a BFF calling sequentially.
9. Why does Eureka keep possibly stale instances during a heartbeat drop?
10. Health checks every 5 s, 3 misses, 1 s propagation, 10 s client cache: what is the stale window?
11. What does a consumer-driven contract check, and what does it not check?
12. What does the `traceparent` header carry?
13. Name the two layers of Istio ambient mode and what each handles.
14. With 300 pods, 60 MB per sidecar and 15,000 internal requests/s at 0.2 vCPU per 1,000 requests/s per proxy, estimate sidecar memory and CPU.
15. When would you choose a CQRS view over API composition?
16. What is the availability of eight services in series at 99.95% each, and the monthly downtime?

Answers

1. Built around one business capability; owned by one team; independently deployable; sole owner of its data.
2. $30/(50 + 40 - 30) = 0.5$: half of all changes to either touch both; they probably belong together or need a better interface.
3. Content coupling lets another service change your internal state directly, so you can no longer enforce invariants or change your schema; domain coupling only uses your published API.
4. An entity has identity over time (an `Order`); a value object is defined by its attributes and immutable (`Money`).
5. $1 - e^{-5 \times 0.04} = 1 - e^{-0.2} = 0.181$.
6. A conformist adopts the upstream model unchanged; an ACL translates it into the downstream's own model.
7. Mapping an unknown decline code to `OTHER` still declines the order safely; guessing whether an unknown status means money moved could ship unpaid orders or double-charge.
8. Direct: $5 \times 90 = 450$ ms. BFF: $80 + 5 \times 11 = 135$ ms.
9. A sudden drop in heartbeats is more likely a network problem than mass failure; a stale list is better than none (availability over consistency).
10. $3 \times 5 + 1 + 10 = 26$ s.

11. It checks that the provider still satisfies the request and response shapes the consumer relies on; it does not check business correctness.
12. A version, the trace ID, the parent span ID and trace flags such as sampled.
13. ztunnel, a per-node L4 proxy for mTLS, identity and L4 policy; waypoint proxies for L7 routing, retries and policy where needed.
14. Memory $300 \times 60 = 18,000$ MB $= 18$ GB; CPU $(15,000 \times 2/1,000) \times 0.2 = 6$ vCPU.
15. When the query is frequent, joins many owners or needs search or aggregation, and seconds of staleness are acceptable.
16. $0.9995^8 = 0.99601$; about $0.004 \times 720 = 2.87$ hours per month.

34.17 Interview questions

Basic

Q1. How do you decide service boundaries? Answer. Start from business capabilities and bounded contexts, found with event storming. Check that each candidate has one owning team, keeps its invariants inside one transaction and has low change coupling with its neighbors (measured from history). Prefer coarse services first and split on evidence such as different scaling or release needs.

Q2. What does an API gateway do, and what should it not do? Answer. It is the single entry point: TLS termination, authentication, rate limiting, routing, observability and sometimes aggregation. It should not contain business logic, which would make it a shared monolith that every team must change.

Intermediate

Q3. Orders, payments and inventory each have their own database. How do you place an order consistently? Answer. Keep payment authorization synchronous if the customer needs the result, and make the overall flow a saga: each service commits locally and publishes events through an outbox; failures trigger compensations (release stock, refund). Consumers are idempotent. Explain the accepted intermediate states and how the UI shows them.

Q4. How do you prevent a provider change from breaking consumers without end-to-end test environments? Answer. Consumer-driven contracts: consumers generate contracts from their tests and publish them to a broker; the provider verifies all of them in CI using provider states; deployment is gated on verification against the versions in the target environment. Combine with additive changes, tolerant readers and expand and contract.

Advanced

Q5. Should we adopt a service mesh? Answer. Identify the problem: uniform mTLS and identity-based policy, polyglot telemetry and traffic shifting are good reasons. Quantify cost: about 0.2–0.25 ms per hop at p99 for two proxies, CPU proportional to internal traffic, memory per pod for sidecars. Consider ambient mode to cut per-pod cost, and plan for control-plane operations, upgrades and retry duplication with libraries.

Q6. A company has 120 services, releases need coordination and incidents cascade. Diagnose and plan. Answer. Likely a distributed monolith. Measure change coupling between services, map synchronous critical paths and compute their A^n, and find shared databases and shared domain libraries. Merge strongly coupled services, move non-essential calls to events with local copies, introduce contract testing and expand-and-contract, and set ownership so each service has one team. Track deploy independence and critical-path length as metrics.

34.18 Practical exercises

1. Compute change coupling for every pair of top-level directories in a repository you know, using `git log --name-only`; list the five most coupled pairs.
2. Extend the ACL of §34.5.3 with a refund response and outbound translation of a `ChargeRequest`; add tests.
3. Extend the contract library of §34.8.5 with an array matcher (`each_like`) and a message (event) contract.
4. Simulate the stale-endpoint window of §34.7.3 with 20 instances, one crash and passive ejection after 5 errors; compare failures with and without retries.
5. Draw QuickBite's context map for a new "group ordering" feature and state each relationship pattern.

34.19 Design challenges

1. **Ride-hailing decomposition.** Propose bounded contexts, a context map and the synchronous critical path for "request a ride"; compute its availability at 99.9% per service.
2. **Strangling a billing monolith.** Plan a four-phase migration of invoicing out of a 15-year-old ERP, with ACLs, CDC and a parallel run; define exit criteria per phase.
3. **Mesh or library.** A trading platform has a 2 ms internal latency budget across 3 hops and 40,000 requests/s. Decide between sidecars, ambient mode and an RPC library, with numbers.

34.20 References

1. Evans, E. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.
2. Vernon, V. *Implementing Domain-Driven Design*. Addison-Wesley, 2013.
3. Newman, S. *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O'Reilly, 2021.
4. Newman, S. *Monolith to Microservices*. O'Reilly, 2019.
5. Richardson, C. *Microservices Patterns*. Manning, 2018.
6. Lewis, J., Fowler, M. *Microservices: a definition of this new architectural term*. martinfowler.com, 2014. https://martinfowler.com/articles/microservices.html
7. Fowler, M. *StranglerFigApplication*. martinfowler.com, 2004. https://martinfowler.com/bliki/StranglerFigApplication.html
8. Fowler, M. *BranchByAbstraction*. martinfowler.com, 2014. https://martinfowler.com/bliki/BranchByAbstraction.html
9. Fowler, M. *TolerantReader*. martinfowler.com, 2011. https://martinfowler.com/bliki/TolerantReader.html
10. Fowler, M. *MicroservicePremium*. martinfowler.com, 2015. https://martinfowler.com/bliki/MicroservicePremium.html
11. Conway, M. E. *How Do Committees Invent?* Datamation 14(4), 1968.
12. Skelton, M., Pais, M. *Team Topologies*. IT Revolution, 2019.
13. Tornhill, A. *Your Code as a Crime Scene*. Pragmatic Bookshelf, 2015.
14. Brandolini, A. *Introducing EventStorming*. Leanpub. https://leanpub.com/introducing_eventstorming
15. Newman, S. *Pattern: Backends For Frontends*. samnewman.io, 2015. https://samnewman.io/patterns/architectural/bff/
16. Plotnicki, L. *BFF @ SoundCloud*. ThoughtWorks Insights, 2015. https://www.thoughtworks.com/insights/blog/bff-soundcloud
17. Jacobson, D. *Embracing the Differences: Inside the Netflix API Redesign*. Netflix Technology Blog, 2012.
18. Netflix. *Eureka* (wiki and documentation). https://github.com/Netflix/eureka/wiki
19. HashiCorp. *Consul Documentation: Service discovery; health checks*. https://developer.hashicorp.com/consul/docs
20. Kubernetes. *Documentation: Services; DNS for Services and Pods; Liveness, Readiness and Startup Probes*. https://kubernetes.io/docs/
21. Robinson, I. *Consumer-Driven Contracts: A Service Evolution Pattern*. martinfowler.com, 2006. https://martinfowler.com/articles/consumerDrivenContracts.html
22. Pact Foundation. *Pact Documentation* (consumer tests, provider verification, provider states, can-i-deploy). https://docs.pact.io/
23. Sigelman, B. H., et al. *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Google Technical Report dapper-2010-1, 2010.

24. W3C. *Trace Context* (Recommendation). https://www.w3.org/TR/trace-context/
25. Istio. *Documentation: Architecture; Distributed Tracing; Performance and Scalability*. https://istio.io/latest/docs/
26. Istio. *Fast, Secure, and Simple: Istio's Ambient Mode Reaches General Availability in v1.24*. Istio Blog, 2024. https://istio.io/blog/2024/ambient-reaches-ga/
27. Linkerd. *Documentation: Architecture*. https://linkerd.io/2/reference/architecture/
28. Westeinde, K. *Deconstructing the Monolith: Designing Software that Maximizes Developer Productivity*. Shopify Engineering, 2019.
29. GitHub. *Scientist: a Ruby library for carefully refactoring critical paths*. https://github.com/github/scientist
30. Noonan, A. *Goodbye Microservices: From 100s of problem children to 1 superstar*. Segment Engineering Blog, 2018.
31. Kolny, M. *Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%*. Prime Video Tech Blog, 2023.

34.21 Open verification items

- Origin of the "BFF" name (Phil Calçado at SoundCloud) and exact dates of Plotnicki 2015 and Newman 2015.
- Netflix API redesign post (Jacobson 2012): exact title and date.
- Dapper's first production sampling rate of 1/1024.
- Istio performance figures quoted in §34.10.3 (1.21, 1.22 and 1.24 measurements) for the current release.
- Linkerd remaining sidecar-only as of 2026.
- Eureka default lease intervals and self-preservation behavior in current releases.
- Author attribution of the Prime Video 2023 post.

Chapter 35 — Object-Oriented Design

> **Level:** 1–2 · **Track:** A (Traditional) · **Estimated reading time:** 5 h

The high-level design decides which services exist and how they talk (§1.5); the low-level design decides what happens *inside* one service. Object-oriented design is the most common vocabulary for that inside view. It answers a practical question: how do you arrange thousands of lines of code so that a rule lives in one place, a change touches few files, and a mistake is caught where it happens?

This chapter builds that vocabulary from variables, functions and loops alone, using one running example: the checkout of a small online shop. You start with a product and a cart, protect the cart's rules, add kinds of products, discount rules and payment methods, untangle a hierarchy that grows out of control, write down each class's promises, and measure how well the pieces fit. Every idea is shown in Python and Java side by side; all programs were run (Python 3.13, Java 21).

35.1 Learning objectives

After this chapter you should be able to:

- Write classes whose constructors reject invalid state, and distinguish identity from equality.
- Protect invariants with encapsulation, avoid setter-driven designs, and apply "tell, don't ask".
- Use inheritance and overriding, and demonstrate the fragile base class problem.
- Explain dynamic dispatch, duck typing and simple generics.
- Choose between an abstract class and an interface.
- Refactor an inheritance hierarchy into composition and quantify the saving.
- Write preconditions, postconditions and invariants, and explain why a subclass may not strengthen a precondition.
- Classify coupling and cohesion, compute LCOM1, and apply the Law of Demeter.
- Recognize the anemic domain model, the god class and deep hierarchies.

35.2 Prerequisites

- Basic programming: variables, functions, `if`, loops, lists and dictionaries.

- High-level vs low-level design — §1.5 (this chapter is the toolbox for low-level design).

SOLID is Ch 36, design patterns Ch 37, and UML notation and full low-level design exercises Ch 38. The class diagrams here use a few Mermaid symbols, explained as they appear.

35.3 Classes and objects

35.3.1 State plus behavior

DEFINITION

Definition — Object. A value in a running program that bundles **state** (data, stored in **attributes** or **fields**) with **behavior** (functions that operate on that data, called **methods**). Each object has its own copy of the state.

DEFINITION

Definition — Class. A template that describes the attributes and methods every object of a kind will have. Creating an object from a class is called **instantiation**; the object is an **instance** of the class.

First-principles explanation

A blank paper order form is printed once; every filled-in copy is separate. A class is the blank form; an object is a filled-in copy that also carries the instructions for working with its contents: a cart object knows how to add an item and total itself.

Without objects, you keep data in a dictionary and write free-standing functions such as `cart_add(cart_dict, product, 2)`. That works until another team writes `cart_dict["items"].append(...)` directly, skipping the rules. Objects put the data and the sanctioned ways of changing it in one place.

Why does this exist?

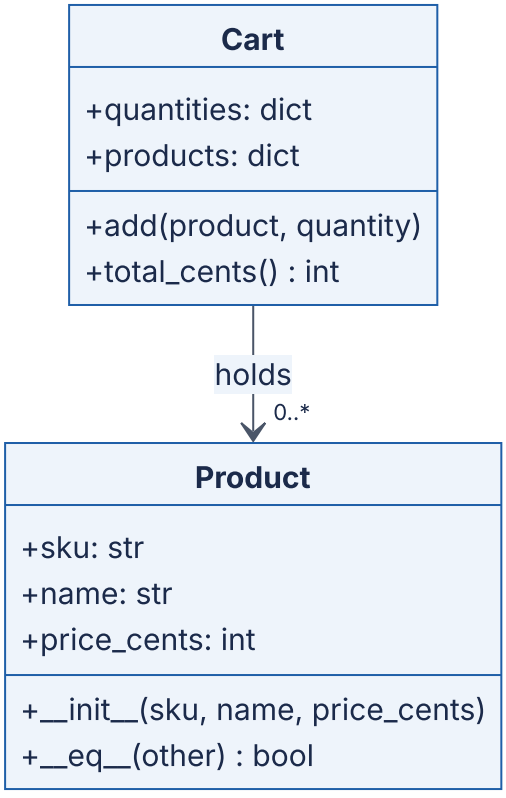
Programs model things that have data and rules: a cart has lines, and quantities must be positive. When data and rules live in different files, someone eventually changes the data without the rules. Classes (Simula 67, then Smalltalk, C++, Java, Python) keep them together.

How it works internally

1. The class definition is loaded once and holds the method code shared by all instances.
2. Calling the class (`Product(...)`, Java `new Product(...)`) allocates memory for a new object.
3. The **constructor** (`__init__`, or Java's method named after the class) puts the object into a valid starting state, or refuses by raising an exception.
4. `obj.method(args)` runs the method with `obj` passed implicitly (`self` in Python, `this` in Java), so it works on *that* object's state.
5. When nothing refers to the object any more, the garbage collector reclaims it.

Architecture diagram

Figure 35.1 — The first shop model: products in a cart.



Reading the diagram.

- **Components.** Each box is a class: name, then attributes (state), then methods (behavior). `+` means public.
- **Direction.** `Cart` refers to zero or more (`0..*`) products; a product knows nothing about carts.
- **Sync/async and storage.** Synchronous in-process calls; objects live in memory only.

- **Failure points.** Every attribute is public, so any code can set `price_cents` to −5 after construction. The constructor protects only the moment of creation; §35.4 fixes this.
- **Scaling.** A new field is added in one class and every instance gets it.

Code example

Prices are stored as integer cents, never floating point, because `0.1 + 0.2` is not exactly `0.3` in binary floating point and money must add up exactly.

```
        class Product:
            """Something the shop sells. Two products are equal if their SKUs match."""

            def __init__(self, sku, name, price_cents):
                if not sku:
                    raise ValueError("sku must be non-empty")
                if price_cents < 0:
                    raise ValueError("price_cents must be >= 0")
                self.sku = sku                  # state: three attributes
                self.name = name
                self.price_cents = price_cents

            def __eq__(self, other):            # equality: "same SKU"
                return isinstance(other, Product) and self.sku == other.sku

            def __hash__(self):                 # equal objects must hash equally
                return hash(self.sku)


        class Cart:
            """A shopping cart. Each cart is a distinct thing, even if two hold the same
items."""

            def __init__(self):
                self.quantities = {}            # sku -> quantity
                self.products = {}              # sku -> Product

            def add(self, product, quantity=1): # behavior
                self.products[product.sku] = product
                self.quantities[product.sku] = self.quantities.get(product.sku, 0) +
quantity

            def total_cents(self):
                return sum(self.products[s].price_cents * q for s, q in
self.quantities.items())


        mug_a = Product("SKU-MUG", "Mug", 1299)
        mug_b = Product("SKU-MUG", "Mug", 1299)
        print(mug_a == mug_b, mug_a is mug_b)        # True False

        cart1, cart2 = Cart(), Cart()
        cart1.add(mug_a, 2)
        cart2.add(mug_b, 2)
        print(cart1.total_cents())                   # 2598
        print(cart1 == cart2, cart1 is cart2)        # False False

        try:
            Product("SKU-BAD", "Broken", -5)
        except ValueError as e:
            print("rejected:", e)                    # rejected: price_cents must be >=
0
```

```
        import java.util.LinkedHashMap;
        import java.util.Map;

        final class Product {
            final String sku;                          // state: three fields
            final String name;
            final long priceCents;

            Product(String sku, String name, long priceCents) {   // constructor
                if (sku == null || sku.isEmpty())
                    throw new IllegalArgumentException("sku must be non-empty");
                if (priceCents < 0)
                    throw new IllegalArgumentException("priceCents must be >= 0");
                this.sku = sku;
                this.name = name;
                this.priceCents = priceCents;
            }

            @Override public boolean equals(Object o) {          // equality: "same SKU"
                return o instanceof Product other && sku.equals(other.sku);
            }

            @Override public int hashCode() {                    // equal objects, equal
hashes
                return sku.hashCode();
            }
        }

        class Cart {
            final Map<String, Integer> quantities = new LinkedHashMap<>();
            final Map<String, Product> products = new LinkedHashMap<>();

            void add(Product product, int quantity) {            // behavior
                products.put(product.sku, product);
                quantities.merge(product.sku, quantity, Integer::sum);
            }

            long totalCents() {
                long total = 0;
                for (var e : quantities.entrySet())
                    total += products.get(e.getKey()).priceCents * e.getValue();
                return total;
            }
        }

        class Main {
            public static void main(String[] args) {
                Product mugA = new Product("SKU-MUG", "Mug", 1299);
                Product mugB = new Product("SKU-MUG", "Mug", 1299);
                System.out.println(mugA.equals(mugB) + " " + (mugA == mugB));     //
true false

                Cart cart1 = new Cart(), cart2 = new Cart();
                cart1.add(mugA, 2);
                cart2.add(mugB, 2);
                System.out.println(cart1.totalCents());                           //
```

```
2598
                System.out.println(cart1.equals(cart2) + " " + (cart1 == cart2)); //
false false

                try {
                    new Product("SKU-BAD", "Broken", -5);
                } catch (IllegalArgumentException e) {
                    System.out.println("rejected: " + e.getMessage());
                }
            }
        }
```

What to notice:

- The constructor is the gatekeeper: an invalid `Product` is never created.
- Python creates attributes on first assignment; Java declares typed fields up front, and `final` means "assigned once".
- `mug_a == mug_b` is `True` because `Product` defines equality by SKU. `cart1 == cart2` is `False` even though both carts hold the same items, because `Cart` does not define equality and falls back to identity.

35.3.2 Identity versus equality

DEFINITION

Definition — Identity and equality. Identity asks "are these the same object in memory?" (Python `is`, Java `==` on references). **Equality** asks "do these objects represent the same value by a rule the class defines?" (Python `==` via `__eq__`, Java `equals`). Unless a class overrides equality, both languages default to identity.

Two copies of the same banknote are interchangeable; two customers named "Ada Lopez" are not. A **value object** (a price, an address) is defined by its attributes, so equality compares attributes and the object is usually immutable. An **entity** (a customer, an order, a cart) has an identity that persists while its attributes change; two carts with the same contents are still two carts.

Table 35.1 — Identity and equality in Python and Java.

Question	Python	Java	Default if not overridden
Same object?	`a is b`	`a == b` (references)	—
Same value?	`a == b` → `__eq__`	`a.equals(b)`	Identity
Hash consistent with equality	`__hash__`	`hashCode()`	Identity-based
Typical value object	`@dataclass(frozen=True)`	`record`	—

The rule that trips people up: **equal objects must have equal hashes**. Hash tables find a bucket by hash, then compare with equality. Override `equals` without `hashCode` in Java and a `HashSet` can hold "duplicates" [Bloch 2018, Items 10–11]. Python defining `__eq__` without `__hash__` makes instances unhashable, a loud `TypeError` instead of a silent duplicate.

Failure scenarios

Table 35.2 — Failure scenarios for classes, constructors and equality.

What goes wrong	Why	Detection	Recovery	Prevention
Duplicate products in a set	`equals` overridden, `hashCode` not	Set size larger than expected	Add `hashCode` from the same fields	Records/dataclasses generate both
Object found then "lost" in a map	Mutable field used in the hash changed after insertion	Lookups return `None` for present keys	Remove and reinsert	Hash only immutable fields
Invalid object deep in the system	Constructor accepted bad input	Errors far from the cause	Fix the creating code	Validate in the constructor

COMMON MISTAKE

Misconception — "Objects are just dictionaries with functions attached." Mechanically, a Python object nearly is one. The design point is different: a class decides *which* changes to its state are allowed. A dictionary allows every change, which is why the rules end up scattered across the code that uses it.

35.4 Encapsulation

35.4.1 Invariants and information hiding

DEFINITION

Definition — Encapsulation. Keeping an object's state private and allowing it to change only through the object's own methods, so the object can guarantee its own rules. **Information hiding** [Parnas 1972] is the broader design principle behind it: each module hides a design decision that is likely to change (a data layout, an algorithm) behind an interface that is not.

DEFINITION

Definition — Class invariant. A condition on an object's state that is true whenever no method of the object is running: after the constructor finishes and after every public method returns.

First-principles explanation

You cannot walk into a bank vault; you ask the teller, who checks your balance first. The counter is encapsulation, and "no account goes below zero" holds precisely because every withdrawal goes through the teller. One side door into the vault, and the rule becomes something every visitor must remember.

In Figure 35.1, any code can write `cart.quantities["SKU-MUG"] = -3`. The cart can promise nothing about its contents, so every reader must defend itself, and the one that forgets causes the bug.

Why does this exist?

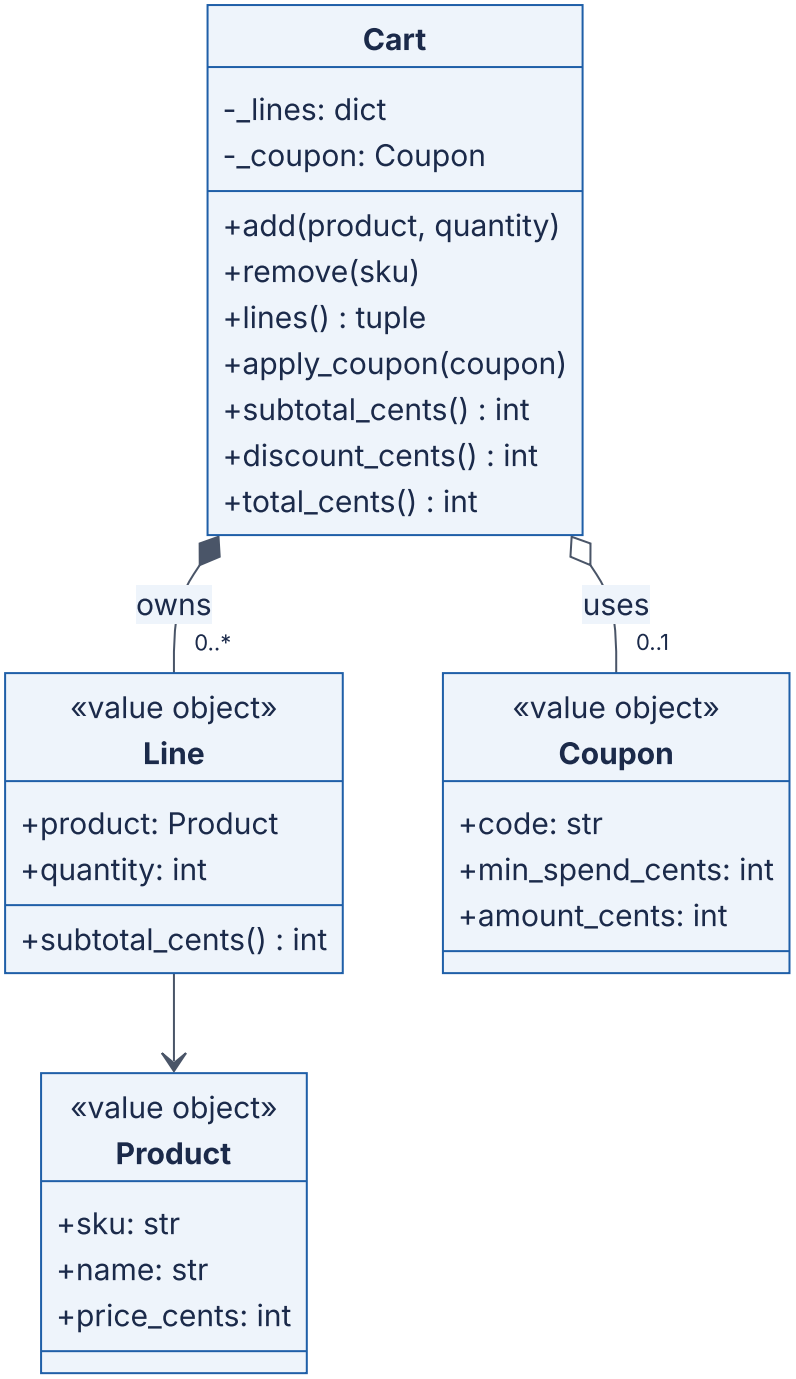
1. **Correctness.** If the only way to change quantities is `add` and `remove`, checking the rule in those two methods proves it for the whole program.
2. **Change.** If callers read `cart.quantities` directly, you cannot change how quantities are stored without editing every caller. Parnas's 1972 argument was exactly this: decompose by the decisions likely to change, and hide each one.

How it works internally

- **Java** enforces access in the compiler: `private` (class only), package-private (no modifier), `protected` (subclasses and package), `public`.
- **Python** has no enforced privacy. A leading underscore (`_lines`) is a convention meaning "internal"; a double underscore triggers **name mangling** (`_Cart__lines`), which avoids clashes in subclasses but stops nobody determined.
- The stronger tool in both is **immutability**: Python's `@dataclass(frozen=True)` and Java's `record` produce objects whose state cannot change after construction.
- Return a **copy** or **read-only view** of internal data, never the internal collection.

Architecture diagram

Figure 35.2 — The encapsulated cart.



Reading the diagram.

- **Components.** `Cart` is the entity that enforces the rules; `Line`, `Coupon` and `Product` are immutable value objects (`<<value object>>` is a free-text label). `-` marks private members: the only doors into `Cart` are its public methods.
- **Relationships.** The filled diamond (`*--`) means the cart *owns* its lines; the hollow diamond (`o--`) means it *uses* a coupon that exists independently (Ch 38 covers the notation).
- **Sync/async and storage.** Synchronous, in memory.

- **Failure points.** Only a bug inside `Cart` can break the invariants, and `lines()` returns an immutable snapshot, not a side door.
- **Scaling.** The dictionary could become a database-backed store without changing any caller.

Code example

```
        from dataclasses import dataclass

        MAX_QTY_PER_LINE = 10


        @dataclass(frozen=True)
        class Product:
            sku: str
            name: str
            price_cents: int


        @dataclass(frozen=True)
        class Line:                                 # immutable: safe to hand out
            product: Product
            quantity: int

            def subtotal_cents(self):
                return self.product.price_cents * self.quantity


        @dataclass(frozen=True)
        class Coupon:
            code: str
            min_spend_cents: int
            amount_cents: int


        class Cart:
            """Invariants:
            1. every line has 1 <= quantity <= MAX_QTY_PER_LINE
            2. 0 <= total_cents() <= subtotal_cents()
            """

            def __init__(self):
                self._lines = {}                    # sku -> Line; underscore = "private,
keep out"
                self._coupon = None

            def add(self, product, quantity=1):
                if quantity < 1:
                    raise ValueError("quantity must be >= 1")
                current = self._lines.get(product.sku)
                new_qty = quantity + (current.quantity if current else 0)
                if new_qty > MAX_QTY_PER_LINE:
                    raise ValueError(f"at most {MAX_QTY_PER_LINE} of {product.sku} per
order")
                self._lines[product.sku] = Line(product, new_qty)

            def remove(self, sku):
                self._lines.pop(sku, None)

            def lines(self):
                return tuple(self._lines.values())    # a read-only snapshot, not the
```

```
dict itself

            def apply_coupon(self, coupon):           # "tell": the cart decides
                self._coupon = coupon

            def subtotal_cents(self):
                return sum(line.subtotal_cents() for line in self._lines.values())

            def discount_cents(self):
                subtotal = self.subtotal_cents()
                if self._coupon is None or subtotal < self._coupon.min_spend_cents:
                    return 0
                return min(self._coupon.amount_cents, subtotal)

            def total_cents(self):
                return self.subtotal_cents() - self.discount_cents()


        mug = Product("SKU-MUG", "Mug", 1299)
        tea = Product("SKU-TEA", "Green tea", 2450)
        cart = Cart()
        cart.add(mug, 2)
        cart.add(tea, 1)
        cart.apply_coupon(Coupon("SAVE5", min_spend_cents=5000, amount_cents=500))
        print(cart.subtotal_cents(), cart.discount_cents(), cart.total_cents())  # 5048
500 4548

        cart.remove("SKU-TEA")                    # subtotal drops below the minimum
spend...
        print(cart.subtotal_cents(), cart.discount_cents(), cart.total_cents())  # 2598
0 2598

        for bad in (lambda: cart.add(mug, 0), lambda: cart.add(mug, 9)):
            try:
                bad()
            except ValueError as e:
                print("rejected:", e)
        try:
            cart.lines()[0].quantity = 1000       # the snapshot cannot be used to break
the cart
        except Exception as e:
            print("rejected:", type(e).__name__)  # rejected: FrozenInstanceError
```

```
        import java.util.LinkedHashMap;
        import java.util.List;
        import java.util.Map;

        record Product(String sku, String name, long priceCents) {}

        record Line(Product product, int quantity) {          // records are immutable
            long subtotalCents() { return product.priceCents() * quantity; }
        }

        record Coupon(String code, long minSpendCents, long amountCents) {}

        /** Invariants: 1 <= quantity <= MAX_QTY_PER_LINE for every line;
         *  0 <= totalCents() <= subtotalCents(). */
        class Cart {
            static final int MAX_QTY_PER_LINE = 10;
            private final Map<String, Line> lines = new LinkedHashMap<>();
            private Coupon coupon;                            // null when none applied

            void add(Product product, int quantity) {
                if (quantity < 1)
                    throw new IllegalArgumentException("quantity must be >= 1");
                Line current = lines.get(product.sku());
                int newQty = quantity + (current == null ? 0 : current.quantity());
                if (newQty > MAX_QTY_PER_LINE)
                    throw new IllegalArgumentException(
                        "at most " + MAX_QTY_PER_LINE + " of " + product.sku() + " per
order");
                lines.put(product.sku(), new Line(product, newQty));
            }

            void remove(String sku) { lines.remove(sku); }

            List<Line> lines() { return List.copyOf(lines.values()); }   // unmodifiable
copy

            void applyCoupon(Coupon coupon) { this.coupon = coupon; }    // "tell"

            long subtotalCents() {
                long sum = 0;
                for (Line l : lines.values()) sum += l.subtotalCents();
                return sum;
            }

            long discountCents() {
                long subtotal = subtotalCents();
                if (coupon == null || subtotal < coupon.minSpendCents()) return 0;
                return Math.min(coupon.amountCents(), subtotal);
            }

            long totalCents() { return subtotalCents() - discountCents(); }
        }

        class Main {
            public static void main(String[] args) {
                Product mug = new Product("SKU-MUG", "Mug", 1299);
```

```
                Product tea = new Product("SKU-TEA", "Green tea", 2450);
                Cart cart = new Cart();
                cart.add(mug, 2);
                cart.add(tea, 1);
                cart.applyCoupon(new Coupon("SAVE5", 5000, 500));
                System.out.println(cart.subtotalCents() + " " + cart.discountCents() + "
" + cart.totalCents());
                cart.remove("SKU-TEA");
                System.out.println(cart.subtotalCents() + " " + cart.discountCents() + "
" + cart.totalCents());
                try {
                    cart.add(mug, 9);
                } catch (IllegalArgumentException e) {
                    System.out.println("rejected: " + e.getMessage());
                }
                try {
                    cart.lines().clear();
                } catch (UnsupportedOperationException e) {
                    System.out.println("rejected: " + e.getClass().getSimpleName());
                }
            }
        }
```

What to notice:

- The cart stores no `total` field. The total is *derived* from the lines and the coupon each time it is asked for, so it can never disagree with them. When the tea is removed, the subtotal falls to 2,598 cents, below the 5,000-cent minimum, and the discount disappears automatically. A stored total updated through a setter would have kept the stale 500-cent discount.
- Both invariants are enforced in one place: adding 9 mugs to a line of 2 is refused because 11 > 10.
- `lines()` returns frozen dataclasses or records in an immutable collection: callers can read everything and change nothing.

35.4.2 Getters and setters: the pitfalls

A **getter** returns a field (`get_price()`), a **setter** assigns one (`set_price(x)`). Generating both for every private field satisfies the letter of encapsulation and defeats its purpose: a setter for every field is a public field with extra typing.

1. **Setters bypass invariants.** `line.set_quantity(-3)` creates a state no business rule permits. Expose operations named after the business action (`apply_coupon`, `remove`), which can check the rule.
2. **Getters leak representation.** `get_lines()` returning the internal mutable list is a side door: `cart.get_lines().clear()` empties the cart without going through any method. Bloch's example is a `Period` class whose start and end are mutable `java.util.Date` objects: a caller who keeps a reference can move the end before the

start. His remedy is **defensive copying** of mutable inputs and outputs [Bloch 2018, Item 50].

3. **Getters invite logic to move outside.** Callers that can read `quantity`, `price` and `coupon` start computing totals themselves, and the discount rule is copied into the checkout page, the email and the invoice.

Read-only access to immutable values (`product.sku`) is harmless and needed for display. The test is whether the caller then *makes a decision* the object should have made.

35.4.3 Tell, don't ask

> KEY CONCEPT
>
> **Tell, don't ask.** Rather than asking an object for its data and deciding what to do with it, tell the object what you want and let it decide [Fowler 2013, "TellDontAsk"]. The rule that uses the data then lives with the data.

```
        # Asking: the caller pulls data out and makes the decision.   # illustrative
        if cart.subtotal_cents() >= 5000:
            cart.discount_cents = 500          # requires a writable field; the rule is
now out here

        # Telling: the cart owns the rule.
        cart.apply_coupon(Coupon("SAVE5", min_spend_cents=5000, amount_cents=500))
```

```
        // Asking (illustrative): the rule lives in the caller and needs a setter.
        if (cart.subtotalCents() >= 5000) cart.setDiscountCents(500);

        // Telling: the cart owns the rule.
        cart.applyCoupon(new Coupon("SAVE5", 5000, 500));
```

The asking version needs a writable discount field, and when the tea is removed nobody re-checks the minimum spend. The telling version re-evaluates the rule every time because it lives inside the cart.

Real-world industry applications

INDUSTRY EXAMPLE

`java.util.Date` versus `java.time`. `java.util.Date` is mutable: a class that stores a `Date` passed by a caller, or returns its own `Date` from a getter, can have its state changed from outside, which is why *Effective Java* uses it as the example for defensive copying. The `java.time` API introduced in Java 8 (JSR 310) made its date and time classes immutable and documents them as thread-safe, which removes the need for defensive copies entirely. *Evidence: Primary* — [Bloch 2018, Items 17 and 50]; [Oracle: java.time package documentation]

Advantages and disadvantages

- **Advantages.** Invariants checked in one place; representation can change freely; immutable values can be shared, even across threads.
- **Disadvantages.** More methods to design; immutability means a new object per change (costly for large, often-updated collections); Python's privacy depends on discipline.

Failure scenarios

Table 35.3 — Failure scenarios for encapsulation.

What goes wrong	Why	Detection	Recovery	Prevention
Negative quantity reaches the invoice	Public field or setter	Data-quality checks, customer complaint	Correct data, add the check	Business-named methods with validation
Cart emptied by a report job	Getter returned the internal list	Hard to trace "spooky" changes	Return a copy	Copies or read-only views
Stale discount after item removal	Derived value stored and set from outside	Totals disagree with lines	Recompute	Derive, don't store

COMMON MISTAKE

Misconception — "Private fields plus getters and setters equals encapsulation." Encapsulation is about who may make which state changes. Ask which *operations* the business allows on the object, and expose those.

35.5 Inheritance

35.5.1 Is-a relationships and method overriding

> DEFINITION
>
> **Definition — Inheritance.** A mechanism by which a class (the **subclass**, **child** or **derived class**) is defined as an extension of another (the **superclass**, **parent** or **base class**). The subclass automatically has the parent's attributes and methods, may add new ones, and may **override** inherited methods by providing its own version with the same name and parameters.

First-principles explanation

The shop now sells a mug (physical: it has a weight and costs money to ship) and an e-book (digital: a download link, free shipping). Both have a SKU, name and price. Copying `Product` twice means every fix is made twice. Inheritance says "a physical product *is a* product, plus a weight and its own shipping rule": shared code is written once in the parent, and each child states only what differs.

The **is-a** test applies to *behavior*, not everyday language: everything the code does with a product must make sense for a physical product. §35.9 makes this precise.

Why does this exist?

It provides **reuse** (shared code in the parent) and, more importantly, **substitutability**: code written for `Product` works unchanged for every subclass, including ones written years later. That makes polymorphism (§35.6) possible. The GoF call inheritance-for-reuse **white-box reuse**, because the subclass sees the parent's internals, and warn that "inheritance breaks encapsulation" [Gamma et al. 1994, ch. 1]. §35.5.2 shows how.

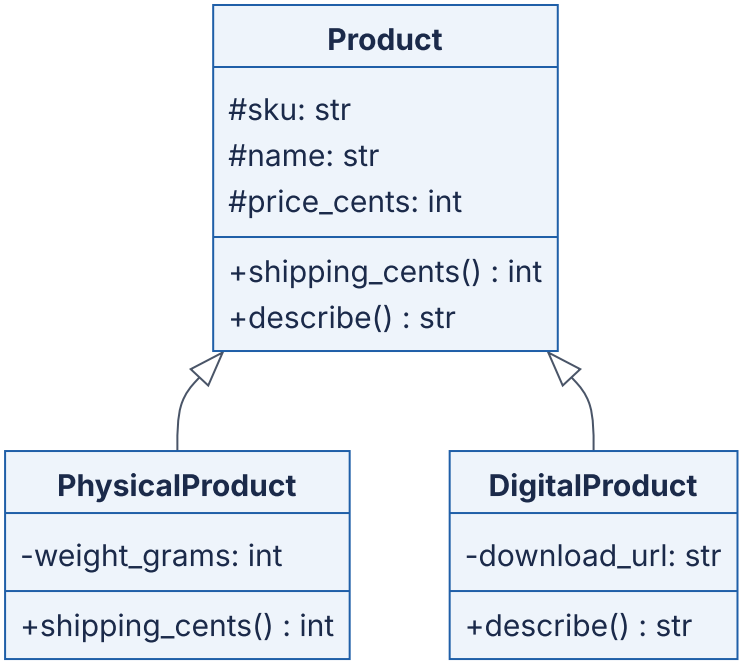
How it works internally

1. The subclass names its parent: `class PhysicalProduct(Product)` or `extends Product`. Java allows one parent class; Python allows several, resolved by the method resolution order.
2. The subclass constructor initializes the parent's part first with `super().__init__(...)` or `super(...)`.
3. A call `p.shipping_cents()` looks for the method in the object's actual class, then its parent, up the chain; the first match runs. That is how an override replaces inherited behavior.
4. An override can call the parent's version with `super` to extend rather than replace it.

5. Java's `@Override` makes the compiler confirm a parent method exists, catching typos that would silently create a new method.

Architecture diagram

Figure 35.3 — Products by inheritance.



Reading the diagram.

- **Components.** `PhysicalProduct` adds a weight and overrides `shipping_cents`; `DigitalProduct` adds a URL and overrides `describe`. `#` means protected (visible to subclasses): the white-box coupling.
- **Direction.** The hollow triangle (`<|--`) points from child to parent and reads "is a"; the parent does not know its children.
- **Sync/async and storage.** Synchronous in-process calls; no storage.
- **Failure points.** Every change to `Product` reaches both children; a parent method assuming every product has a weight breaks `DigitalProduct`.
- **Scaling.** A third kind of product is one subclass; a *second* axis of variation is a different story (§35.8).

Code example

Shipping is 399 cents plus 50 cents per started 100 g. The mug weighs 350 g, which starts 4 blocks of 100 g, so shipping is $399 + 4 \times 50 = 599$ cents.

```
        class Product:
            def __init__(self, sku, name, price_cents):
                self.sku = sku
                self.name = name
                self.price_cents = price_cents

            def shipping_cents(self):
                return 0                                  # default: nothing to ship

            def describe(self):
                return f"{self.name} ({self.price_cents / 100:.2f})"


        class PhysicalProduct(Product):                   # PhysicalProduct IS-A Product
            BASE_CENTS = 399
            CENTS_PER_100G = 50

            def __init__(self, sku, name, price_cents, weight_grams):
                super().__init__(sku, name, price_cents)  # let the parent set up its
part
                self.weight_grams = weight_grams

            def shipping_cents(self):                     # override
                hundreds = -(-self.weight_grams // 100)   # ceiling division
                return self.BASE_CENTS + self.CENTS_PER_100G * hundreds


        class DigitalProduct(Product):
            def __init__(self, sku, name, price_cents, download_url):
                super().__init__(sku, name, price_cents)
                self.download_url = download_url

            def describe(self):                           # override and extend
                return super().describe() + " [download]"


        items = [PhysicalProduct("SKU-MUG", "Mug", 1299, weight_grams=350),
                 DigitalProduct("SKU-EBOOK", "Tea guide", 899,
"https://example.com/dl/42")]
        for p in items:
            print(p.describe(), p.shipping_cents())
        # Mug (12.99) 599
        # Tea guide (8.99) [download] 0
```

```
        import java.util.List;

        class Product {
            protected final String sku;
            protected final String name;
            protected final long priceCents;

            Product(String sku, String name, long priceCents) {
                this.sku = sku;
                this.name = name;
                this.priceCents = priceCents;
            }

            long shippingCents() { return 0; }                      // default

            String describe() {
                return String.format("%s (%.2f)", name, priceCents / 100.0);
            }
        }

        class PhysicalProduct extends Product {                      // IS-A Product
            static final long BASE_CENTS = 399, CENTS_PER_100G = 50;
            private final int weightGrams;

            PhysicalProduct(String sku, String name, long priceCents, int weightGrams) {
                super(sku, name, priceCents);                        // parent's con-
structor first
                this.weightGrams = weightGrams;
            }

            @Override long shippingCents() {                         // override
                long hundreds = (weightGrams + 99) / 100;            // ceiling divi-
sion
                return BASE_CENTS + CENTS_PER_100G * hundreds;
            }
        }

        class DigitalProduct extends Product {
            private final String downloadUrl;

            DigitalProduct(String sku, String name, long priceCents, String download-
Url) {
                super(sku, name, priceCents);
                this.downloadUrl = downloadUrl;
            }

            @Override String describe() { return super.describe() + " [download]"; }
        }

        class Main {
            public static void main(String[] args) {
                List<Product> items = List.of(
                    new PhysicalProduct("SKU-MUG", "Mug", 1299, 350),
                    new DigitalProduct("SKU-EBOOK", "Tea guide", 899,
"https://example.com/dl/42"));
                for (Product p : items)
```

```
                System.out.println(p.describe() + " " + p.shippingCents());
            }
        }
```

What to notice:

- The loop never checks which kind of product it has; each object answers `shipping_cents()` its own way.
- The parent's default `shipping_cents` returns 0, so a future physical subclass that forgets to override ships free. §35.7 turns that silent bug into an error with an abstract method.

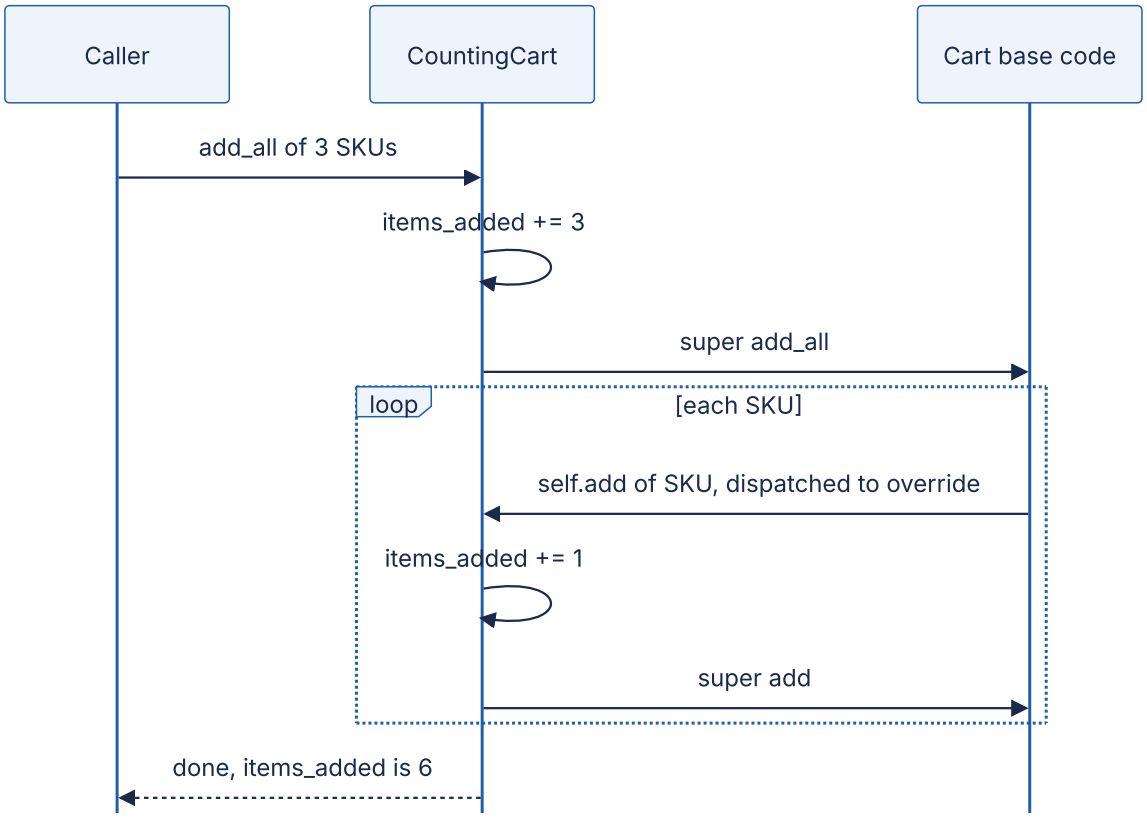
35.5.2 The fragile base class problem

> DEFINITION
>
> **Definition — Fragile base class problem.** A change to a base class that is correct on its own breaks subclasses, because the subclasses depend on implementation details of the base (which methods call which) that were never part of its documented interface.

First-principles explanation

The analytics team subclasses `Cart` to count every item ever added, overriding both `add` and `add_all`. Nobody told them that `add_all` calls `add` once per item, so the count doubles. This is the shop version of Bloch's `InstrumentedHashSet`, which reports 6 additions after adding 3 elements [Bloch 2018, Item 18].

Figure 35.4 — How self-use double-counts in the subclass.



Reading the diagram.

- **Components.** One object, two layers of code: the `CountingCart` overrides and the inherited `Cart` methods.
- **Direction and sync.** Every arrow is a synchronous call on the *same* object. In `Base->>CC`, base code calls `self.add` and dynamic dispatch (§35.6) sends it to the override.
- **Storage.** None; the counter is an in-memory field.
- **Failure point.** The undocumented internal call from `add_all` to `add` is the hidden dependency.
- **Scaling.** Every subclass of `Cart` shares this exposure; the more there are, the harder `Cart` is to change.

```
        class Cart:
            def __init__(self):
                self._quantities = {}

            def add(self, sku, quantity=1):
                self._quantities[sku] = self._quantities.get(sku, 0) + quantity

            def add_all(self, skus):
                for sku in skus:
                    self.add(sku)            # self-use: add_all is built on add (undoc-
umented)


        class CountingCart(Cart):
            """Counts every item a shopper ever puts in the cart, for analytics."""

            def __init__(self):
                super().__init__()
                self.items_added = 0

            def add(self, sku, quantity=1):
                self.items_added += quantity
                super().add(sku, quantity)

            def add_all(self, skus):
                self.items_added += len(skus)
                super().add_all(skus)        # ...which calls self.add, which counts
again


        cart = CountingCart()
        cart.add_all(["SKU-MUG", "SKU-TEA", "SKU-PEN"])
        print(cart.items_added)              # 6, not 3
```

```java
import java.util.HashMap;
import java.util.List;
import java.util.Map;

class Cart {
    private final Map<String, Integer> quantities = new HashMap<>();

    void add(String sku, int quantity) {
        quantities.merge(sku, quantity, Integer::sum);
    }

    void addAll(List<String> skus) {
        for (String sku : skus) add(sku, 1);       // self-use (undocumented)
    }
}

class CountingCart extends Cart {
    int itemsAdded = 0;

    @Override void add(String sku, int quantity) {
        itemsAdded += quantity;
        super.add(sku, quantity);
    }

    @Override void addAll(List<String> skus) {
        itemsAdded += skus.size();
        super.addAll(skus);                        // calls this.add(...) again
    }
}

class Main {
    public static void main(String[] args) {
        CountingCart cart = new CountingCart();
        cart.addAll(List.of("SKU-MUG", "SKU-TEA", "SKU-PEN"));
        System.out.println(cart.itemsAdded);       // 6, not 3
    }
}
```

Removing the `add_all` override fixes the count *today*. If the cart team later rewrites `add_all` to update the dictionary directly, a change that passes all of `Cart`'s tests, `CountingCart` silently *under*counts. Either way the subclass depends on a private choice in the parent. The fix, composition with forwarding, is in §35.8. Bloch's rule: "design and document for inheritance or else prohibit it" [Bloch 2018, Item 19], by documenting self-use or declaring the class `final` (Python's `typing.final` lets type checkers flag subclassing).

Real-world industry applications

INDUSTRY EXAMPLE

`java.util.Stack` and `java.util.Properties`. `Stack` extends `Vector`, so every stack exposes methods for inserting at arbitrary positions, breaking last-in, first-out. `Properties` extends `Hashtable`, whose inherited `put` accepts non-string objects. Bloch cites both as is-a violations that cannot be fixed without breaking compatibility; the `Stack` documentation now recommends `Deque` instead. *Evidence: Primary* — [Bloch 2018, Item 18]; [Oracle: java.util.Stack API documentation]

Advantages and disadvantages

- **Advantages.** Little code for genuine is-a variations; substitutability enables polymorphism.
- **Disadvantages.** White-box coupling to the parent; fixed at compile time; one parent in Java; deep hierarchies hide which method runs.

Failure scenarios

Table 35.4 — Failure scenarios for inheritance.

What goes wrong	Why	Detection	Recovery	Prevention
Counts doubled or missing	Fragile base class self-use	Metrics disagree with source data	Switch subclass to composition	Document self-use or make classes `final`
Subclass exposes forbidden operations	Inherited methods break child's rules (Stack/Vector)	Misuse in code review or production	Hide behind a wrapper	Apply the behavioral is-a test
New physical product ships free	Default implementation inherited silently	Revenue leak in reports	Add override	Abstract method instead of default

COMMON MISTAKE

Misconception — "Inheritance is mainly for code reuse." A helper object gives the same reuse without the coupling. Inherit when callers treat the child *as* the parent and the child keeps every promise the parent makes (§35.9).

35.6 Polymorphism

35.6.1 Subtype polymorphism and dynamic dispatch

DEFINITION

Definition — Polymorphism. The ability of one piece of code to work with values of different types. **Subtype polymorphism** means code written against a type also works with any of its subtypes, and each object responds to a method call with its own implementation. **Dynamic dispatch** is the run-time mechanism that picks which implementation runs, based on the object's actual class rather than the declared type of the variable.

First-principles explanation

A cashier says "Please pay" to everyone in the queue; one taps a card, one hands over cash, one scans a phone. The instruction never changes, even when a new payment method appears.

Marketing invents discount rules monthly: 10% off, 5 off orders over 50, buy two mugs get one free. Without polymorphism the checkout grows an `if rule.kind == "percent" ... elif ...` chain that must be edited and retested for every rule. With it, each rule is an object with a `discount_cents(lines)` method, and the checkout just calls it.

Why does this exist?

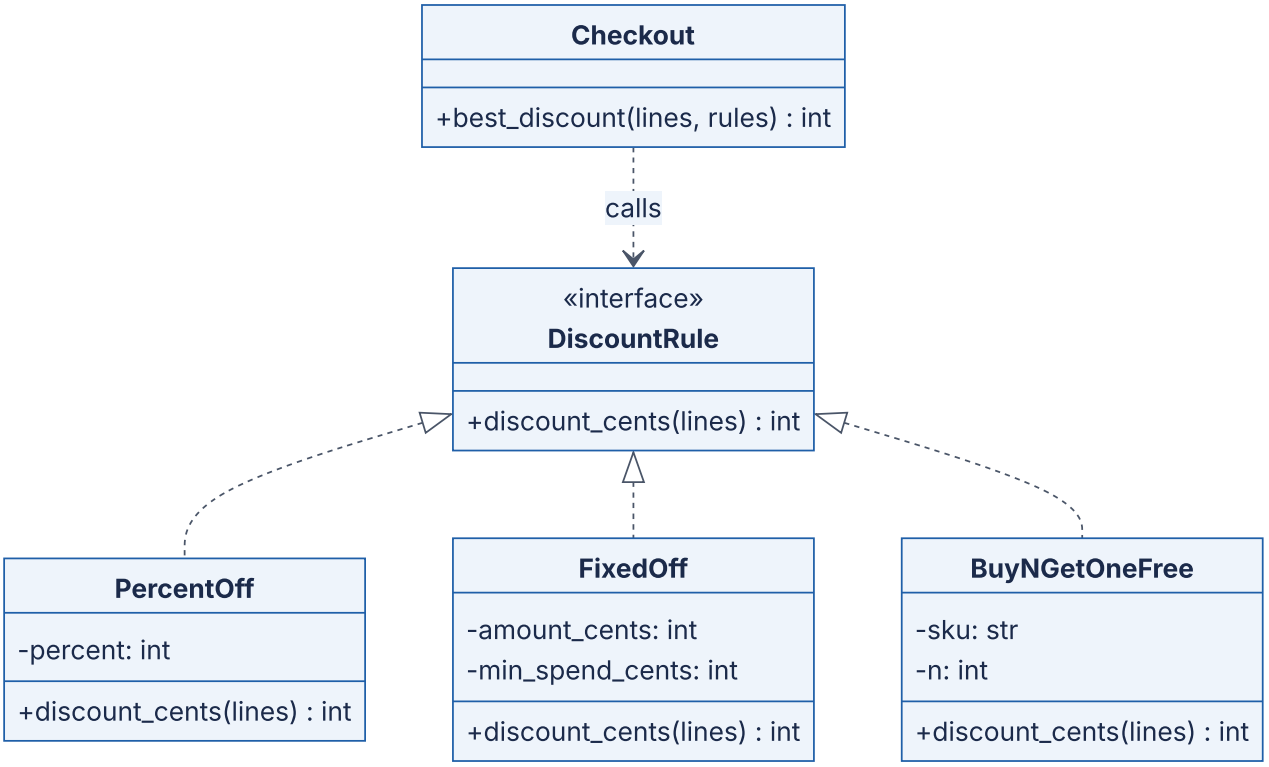
To let variations grow without editing the code that uses them: the Open/Closed Principle of §36.4, of which polymorphism is the mechanism.

How it works internally

1. **Java.** Each class has a **method table** (vtable) of pointers to its overridable methods, and every object points to its class. `rule.discountCents(lines)` becomes "load the class, load the table entry, jump": a couple of memory loads and an indirect jump. Just-in-time compilers inline the one or two classes a call site usually sees, so the overhead is usually negligible.
2. **Python.** `rule.discount_cents` is looked up by name at run time, in the object, then its class and parents. The classes need not share a parent.
3. **Static versus dynamic type.** In `DiscountRule rule = new PercentOff(10);` the compiler checks against `DiscountRule` (static type); the runtime runs `PercentOff`'s code (dynamic type).

Architecture diagram

Figure 35.5 — Discount rules behind one interface.



Reading the diagram.

- **Components.** `DiscountRule` is an interface (methods without implementation), implemented by three classes (`<|..`). `Checkout` depends (`..>`) only on the interface.
- **Direction and sync.** `Checkout` calls whichever rules it is handed, synchronously, via dynamic dispatch.
- **Storage.** In production the rule parameters come from a database or configuration, and a factory builds the objects (Ch 37).
- **Failure points.** A rule returning more than the subtotal would corrupt totals; the cart's invariant (§35.4) caps the discount.
- **Scaling.** A fourth rule is a new class; `Checkout` and its tests do not change.

Code example

The cart has 3 mugs at 1,299 and 1 tea at 2,450: subtotal $3 \times 1299 + 2450 = 6,347$ cents. Ten percent is 634 cents (integer division rounds down), the fixed rule gives 500, and "buy 2, get 1 free" on mugs gives one free mug, 1,299. The best is 1,299.

```
        from dataclasses import dataclass
        from typing import Protocol


        @dataclass(frozen=True)
        class Line:
            sku: str
            unit_cents: int
            quantity: int


        class DiscountRule(Protocol):                    # optional: documents the ex-
pected shape
            def discount_cents(self, lines: list[Line]) -> int: ...


        class PercentOff:
            def __init__(self, percent):
                self.percent = percent

            def discount_cents(self, lines):
                subtotal = sum(l.unit_cents * l.quantity for l in lines)
                return subtotal * self.percent // 100


        class FixedOff:
            def __init__(self, amount_cents, min_spend_cents):
                self.amount_cents = amount_cents
                self.min_spend_cents = min_spend_cents

            def discount_cents(self, lines):
                subtotal = sum(l.unit_cents * l.quantity for l in lines)
                return self.amount_cents if subtotal >= self.min_spend_cents else 0


        class BuyNGetOneFree:
            def __init__(self, sku, n):
                self.sku, self.n = sku, n

            def discount_cents(self, lines):
                return sum((l.quantity // (self.n + 1)) * l.unit_cents
                           for l in lines if l.sku == self.sku)


        def best_discount(lines, rules):
            # One call site, many behaviors: which discount_cents runs depends on the
object.
            return max((rule.discount_cents(lines) for rule in rules), default=0)


        lines = [Line("SKU-MUG", 1299, 3), Line("SKU-TEA", 2450, 1)]   # subtotal 6347
        rules = [PercentOff(10), FixedOff(500, 5000), BuyNGetOneFree("SKU-MUG", 2)]
        for r in rules:
            print(type(r).__name__, r.discount_cents(lines))
```

```
        print("best:", best_discount(lines, rules))
        # PercentOff 634 / FixedOff 500 / BuyNGetOneFree 1299 / best: 1299
```

```
        import java.util.List;

        record Line(String sku, long unitCents, int quantity) {}

        interface DiscountRule {
            long discountCents(List<Line> lines);
        }

        final class PercentOff implements DiscountRule {
            private final int percent;
            PercentOff(int percent) { this.percent = percent; }

            @Override public long discountCents(List<Line> lines) {
                long subtotal = 0;
                for (Line l : lines) subtotal += l.unitCents() * l.quantity();
                return subtotal * percent / 100;
            }
        }

        final class FixedOff implements DiscountRule {
            private final long amountCents, minSpendCents;
            FixedOff(long amountCents, long minSpendCents) {
                this.amountCents = amountCents;
                this.minSpendCents = minSpendCents;
            }

            @Override public long discountCents(List<Line> lines) {
                long subtotal = 0;
                for (Line l : lines) subtotal += l.unitCents() * l.quantity();
                return subtotal >= minSpendCents ? amountCents : 0;
            }
        }

        final class BuyNGetOneFree implements DiscountRule {
            private final String sku;
            private final int n;
            BuyNGetOneFree(String sku, int n) { this.sku = sku; this.n = n; }

            @Override public long discountCents(List<Line> lines) {
                long free = 0;
                for (Line l : lines)
                    if (l.sku().equals(sku)) free += (l.quantity() / (n + 1)) * l.unit-
Cents();
                return free;
            }
        }

        class Main {
            static long bestDiscount(List<Line> lines, List<DiscountRule> rules) {
                long best = 0;
                for (DiscountRule rule : rules)                    // dynamic dispatch
here
                    best = Math.max(best, rule.discountCents(lines));
                return best;
            }
```

```
            public static void main(String[] args) {
                List<Line> lines = List.of(new Line("SKU-MUG", 1299, 3), new Line("SKU-
TEA", 2450, 1));
                List<DiscountRule> rules = List.of(
                    new PercentOff(10), new FixedOff(500, 5000), new
BuyNGetOneFree("SKU-MUG", 2));
                for (DiscountRule r : rules)
                    System.out.println(r.getClass().getSimpleName() + " " +
r.discountCents(lines));
                System.out.println("best: " + bestDiscount(lines, rules));
            }
        }
```

What to notice:

- `best_discount` contains no `if` on the rule's kind.
- In Python the classes share no parent; each simply *has* `discount_cents` (duck typing, §35.6.2). In Java each must declare `implements DiscountRule`.

35.6.2 Duck typing

> DEFINITION
>
> **Definition — Duck typing.** A style of polymorphism in which an object is acceptable wherever it supports the operations actually used, regardless of its declared class ("if it walks like a duck and quacks like a duck"). Checking happens when the method is called. **Structural typing** is the static version: a type checker accepts an object if its shape matches a declared protocol.

Duck typing is short and flexible: any object with `discount_cents(lines)` is a discount rule, including a three-line test double. The price is late errors: a missing method raises `AttributeError` only when the line runs, perhaps in production. `typing.Protocol` [PEP 544] recovers much of the safety: a static checker such as mypy verifies the shape before the program runs, without inheritance. Java's default is **nominal typing**: a class is a `DiscountRule` only if it says so.

35.6.3 Parametric polymorphism (generics), briefly

"Return the cheapest item in a list" is the same code for products or shipping options. **Parametric polymorphism** (**generics**) writes it once with a type parameter `T`, and the type checker still knows a list of products gives back a product.

```
        from typing import Protocol, TypeVar


        class Priced(Protocol):
            @property
            def price_cents(self) -> int: ...


        T = TypeVar("T", bound=Priced)


        def cheapest(items: list[T]) -> T:
            """Works for a list of any priced type and returns that same type."""
            if not items:
                raise ValueError("items must be non-empty")
            return min(items, key=lambda item: item.price_cents)


        class Product:
            def __init__(self, sku, price_cents):
                self.sku, self.price_cents = sku, price_cents


        class ShippingOption:
            def __init__(self, carrier, price_cents):
                self.carrier, self.price_cents = carrier, price_cents


        print(cheapest([Product("SKU-MUG", 1299), Product("SKU-PEN", 250)]).sku)       #
SKU-PEN
        print(cheapest([ShippingOption("PostCo", 599), ShippingOption("FastCo",
999)]).carrier)  # PostCo
```

```java
        import java.util.List;

        interface Priced { long priceCents(); }

        record Product(String sku, long priceCents) implements Priced {}
        record ShippingOption(String carrier, long priceCents) implements Priced {}

        class Main {
            // <T extends Priced>: any type T, as long as it is Priced. Returns that
same T.
            static <T extends Priced> T cheapest(List<T> items) {
                if (items.isEmpty()) throw new IllegalArgumentException("items must be
non-empty");
                T best = items.get(0);
                for (T item : items)
                    if (item.priceCents() < best.priceCents()) best = item;
                return best;
            }

            public static void main(String[] args) {
                Product p = cheapest(List.of(new Product("SKU-MUG", 1299), new
Product("SKU-PEN", 250)));
                ShippingOption s = cheapest(List.of(new ShippingOption("PostCo", 599),
                                                    new ShippingOption("FastCo", 999)));
                System.out.println(p.sku() + " " + s.carrier());   // no casts needed
            }
        }
```

What to notice: the **bound** (`bound=Priced`, `T extends Priced`) restricts `T` to priced types. Java checks generics at compile time; Python's hints are checked only by external tools.

Table 35.5 — Kinds of polymorphism.

Kind	What varies	Mechanism	Checked	Example in this chapter
Subtype	Behavior per subclass	Overriding + dynamic dispatch	Compile time (Java)	`DiscountRule` implementations
Duck / structural	Any object with the right methods	Run-time lookup; `Protocol` for static check	Run time (Python), optional static	Python discount rules
Parametric (generics)	Element type	Type parameters	Compile time / type checker	`cheapest(List<T>)`
Ad hoc (overloading)	Argument types	Same name, different parameter lists	Compile time (Java)	Not used; mention only

The classification follows Cardelli and Wegner [Cardelli & Wegner 1985], who distinguish *universal* polymorphism (parametric and subtype) from *ad hoc* polymorphism (overloading and coercion).

Advantages and disadvantages

- **Advantages.** New variants without editing callers; easy fakes in tests; behavior chosen at run time.
- **Disadvantages.** A call site no longer shows which code runs; tiny classes for variations that never change are over-engineering; duck typing defers errors.

Alternatives and trade-offs

If new *kinds* arrive often, use polymorphism. If the kinds are fixed (say, order states) but new *operations* arrive often, a `switch` over a sealed set of types, which Java 21 checks for exhaustiveness, can be clearer.

COMMON MISTAKE

Misconception — "Polymorphism means using inheritance." It needs a shared *type*: a Java interface with no inherited code, or in Python nothing declared at all.

35.7 Abstraction

35.7.1 Abstract classes and interfaces

DEFINITION

Definition — Abstraction. Representing something by the operations that matter to its users while leaving out the details of how they are carried out. In code, an **abstract class** is a class that cannot be instantiated and may contain both implemented methods and **abstract methods** (declared, with no body) that subclasses must implement. An **interface** declares a set of methods a type promises to provide; in Java it has no instance state, and since Java 8 it may include `default` method bodies built only on its other methods.

First-principles explanation

You drive a petrol car and an electric car with the same pedals. Checkout code wants the same for payments: "charge 25.98 to this method" and get a receipt, without seeing card processors or gift-card balances.

Payment methods share two kinds of things. *Common work*: every payment checks the amount against a limit and records what was charged. *Capabilities* only some have: cards can be refunded, gift cards here cannot. Common work with one varying step fits an abstract class; a capability across unrelated classes fits an interface.

Why does this exist?

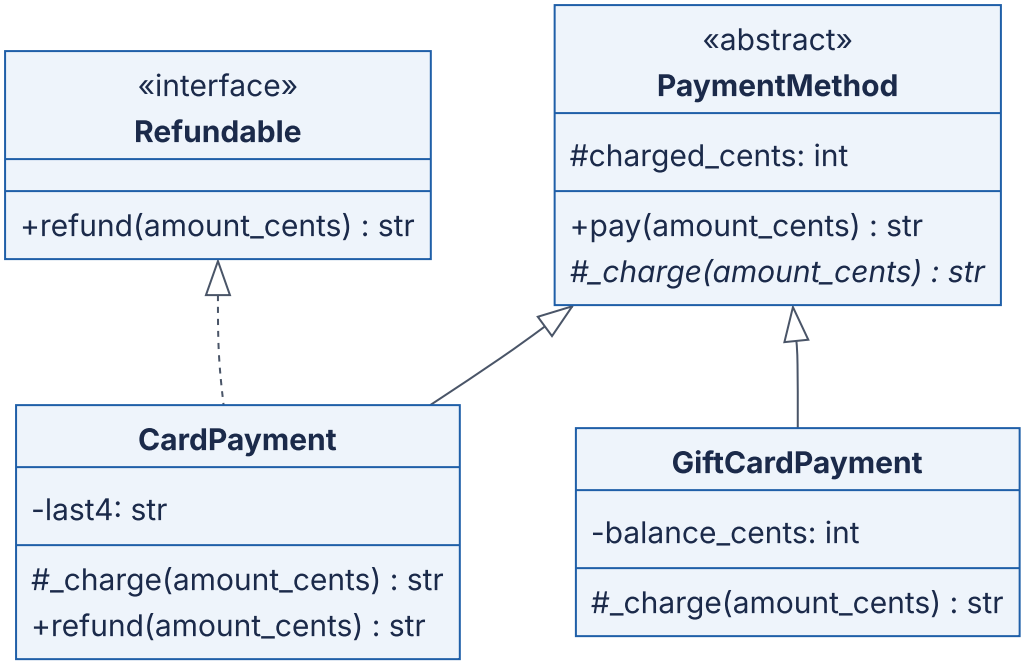
- To **force** subclasses to supply the varying part, turning the free-shipping bug of §35.5.1 into a compile error (Java) or instantiation error (Python).
- To let callers depend on a **small, stable surface**: "Program to an interface, not an implementation" [Gamma et al. 1994, ch. 1].
- To let one class play **several roles** (one parent class, many interfaces in Java).

How it works internally

- **Python.** Inherit from `abc.ABC` and mark methods `@abstractmethod`; instantiating raises `TypeError` while any remain unimplemented. A Python "interface" is an all-abstract ABC or a `typing.Protocol`.
- **Java.** An `abstract class` cannot be instantiated; `interface` methods are public and abstract unless `default` or `static`.
- A parent that fixes the steps in a `final` method and leaves one abstract is the Template Method pattern (§37.15).

Architecture diagram

Figure 35.6 — Payment methods: an abstract class plus a capability interface.



Reading the diagram.

- **Components.** `PaymentMethod` is abstract (`*` marks the abstract `_charge`); `Refundable` is an interface. `CardPayment` is both; `GiftCardPayment` only a payment method. Solid triangles are class inheritance, dotted ones interface implementation.

- **Direction and sync.** Checkout calls `pay`, which calls the subclass's `_charge`, synchronously; in production `_charge` is a slow, failure-prone network call to a provider.
- **Storage.** Balances are in memory here; really in a database or at the provider.
- **Failure points.** `charged_cents` changes only after `_charge` succeeds. "Charged but response lost" needs idempotency keys (§22.15).
- **Scaling.** A new method is one subclass; a new capability is one interface.

Code example

```
        from abc import ABC, abstractmethod


        class Refundable(ABC):
            """Interface: a pure promise, no state."""
            @abstractmethod
            def refund(self, amount_cents): ...


        class PaymentMethod(ABC):
            """Abstract class: shared state and shared steps, one step left open."""
            MAX_CHARGE_CENTS = 500_000

            def __init__(self):
                self.charged_cents = 0                      # shared state

            def pay(self, amount_cents):                    # shared, fixed algorithm
                if not 0 < amount_cents <= self.MAX_CHARGE_CENTS:
                    raise ValueError("amount out of range")
                receipt = self._charge(amount_cents)        # the varying step
                self.charged_cents += amount_cents
                return receipt

            @abstractmethod
            def _charge(self, amount_cents): ...            # subclasses must supply
this


        class CardPayment(PaymentMethod, Refundable):
            def __init__(self, last4):
                super().__init__()
                self.last4 = last4

            def _charge(self, amount_cents):
                return f"card ...{self.last4} charged {amount_cents}"

            def refund(self, amount_cents):
                if amount_cents > self.charged_cents:
                    raise ValueError("cannot refund more than was charged")
                self.charged_cents -= amount_cents
                return f"card ...{self.last4} refunded {amount_cents}"


        class GiftCardPayment(PaymentMethod):               # not Refundable in this shop
            def __init__(self, balance_cents):
                super().__init__()
                self.balance_cents = balance_cents

            def _charge(self, amount_cents):
                if amount_cents > self.balance_cents:
                    raise ValueError("insufficient gift-card balance")
                self.balance_cents -= amount_cents
                return f"gift card charged {amount_cents}, {self.balance_cents} left"
```

```
        for method in (CardPayment("4242"), GiftCardPayment(3000)):
            print(method.pay(2598))
            if isinstance(method, Refundable):
                print(method.refund(1299))

        try:
            PaymentMethod()
        except TypeError as e:
            print("TypeError:", e)
        # card ...4242 charged 2598
        # card ...4242 refunded 1299
        # gift card charged 2598, 402 left
        # TypeError: Can't instantiate abstract class PaymentMethod without an implemen-
tation
        #            for abstract method '_charge'
```

```
        interface Refundable {                                  // pure promise, no
state
            String refund(long amountCents);
        }

        abstract class PaymentMethod {                          // shared state + steps
            static final long MAX_CHARGE_CENTS = 500_000;
            protected long chargedCents = 0;

            final String pay(long amountCents) {                // final: subclasses
can't change it
                if (amountCents <= 0 || amountCents > MAX_CHARGE_CENTS)
                    throw new IllegalArgumentException("amount out of range");
                String receipt = charge(amountCents);           // the varying step
                chargedCents += amountCents;
                return receipt;
            }

            protected abstract String charge(long amountCents);  // subclasses must sup-
ply this
        }

        final class CardPayment extends PaymentMethod implements Refundable {
            private final String last4;
            CardPayment(String last4) { this.last4 = last4; }

            @Override protected String charge(long amountCents) {
                return "card ..." + last4 + " charged " + amountCents;
            }

            @Override public String refund(long amountCents) {
                if (amountCents > chargedCents)
                    throw new IllegalArgumentException("cannot refund more than was
charged");
                chargedCents -= amountCents;
                return "card ..." + last4 + " refunded " + amountCents;
            }
        }

        final class GiftCardPayment extends PaymentMethod {
            private long balanceCents;
            GiftCardPayment(long balanceCents) { this.balanceCents = balanceCents; }

            @Override protected String charge(long amountCents) {
                if (amountCents > balanceCents)
                    throw new IllegalArgumentException("insufficient gift-card bal-
ance");
                balanceCents -= amountCents;
                return "gift card charged " + amountCents + ", " + balanceCents + "
left";
            }
        }

        class Main {
            public static void main(String[] args) {
                PaymentMethod[] methods = { new CardPayment("4242"), new
```

```
        GiftCardPayment(3000) };
                for (PaymentMethod m : methods) {
                    System.out.println(m.pay(2598));
                    if (m instanceof Refundable r) System.out.println(r.refund(1299));
                }
                // new PaymentMethod();  // compile error: PaymentMethod is abstract
            }
        }
```

What to notice:

- `pay` is `final` in Java, so no subclass can skip the amount check; Python relies on convention.
- The `Refundable` check asks about a *capability*, not a concrete class, so new refundable methods need no edits.
- The abstract class carries shared *state*, which a Java interface cannot.

Table 35.6 — Abstract class versus interface.

Aspect	Abstract class	Interface
What it is	Partially implemented parent	A named set of operations
How it works	`extends` / inherit; abstract methods filled in by children	`implements` (Java); ABC or `Protocol` (Python)
Instance state	Yes	No (Java); by convention none (Python)
How many per class	One (Java); several (Python)	Many
Advantages	Shares code and state; can fix the algorithm with `final`	Loose coupling; unrelated classes can share a role; easy to fake in tests
Disadvantages	Couples children to parent internals; uses the single parent slot (Java)	Cannot hold state; adding a method breaks implementers unless it has a `default`
Choose when	Closely related variants share real work and state	You need a role or capability, or you are defining a boundary between modules

Bloch's advice combines the two: define the type as an interface, and optionally provide an abstract **skeletal implementation** class that implementers may extend for convenience [Bloch 2018, Item 20]. The Java collections follow this design.

Real-world industry applications

INDUSTRY EXAMPLE

Java collections and Python's `collections.abc`. Java defines collection types as interfaces (`List`, `Map`) with skeletal abstract classes (`AbstractList`, `AbstractMap`), so a new implementation writes only a few primitive methods. In Python, a subclass of `collections.abc.Sequence` that implements `__getitem__` and `__len__` receives `__contains__`, `__iter__`, `index` and `count` as mixin methods. *Evidence: Primary* — [Bloch 2018, Item 20]; [Python docs: collections.abc]

COMMON MISTAKE

Misconception — "Abstraction means adding interfaces everywhere." An interface with one implementation and no test fake is cost without benefit. Abstract where variation is real (payment providers, discount rules, storage).

35.8 Composition over inheritance

35.8.1 Has-a relationships and delegation

DEFINITION

Definition — Composition and delegation. Composition builds an object out of other objects that it holds in its fields (a **has-a** relationship). **Delegation** is the object forwarding part of its work to one of those parts by calling the part's methods. "Favor object composition over class inheritance" is the second principle stated in the GoF introduction [Gamma et al. 1994, ch. 1].

First-principles explanation

A computer *has* a keyboard plugged in through a standard socket; you can swap it without rebuilding the computer. Inheritance is like casting the keyboard into the case.

Orders now vary two independent ways: shipping (standard: free over 50, else 4.99; express: 12.99) and packaging (plain box, or gift wrap at 3.50). With inheritance every *combination* is a subclass (`StandardPlainOrder`, `ExpressGiftOrder`, ...), each new option multiplies the grid, and a customer who upgrades to express cannot change an existing object's class.

Why does this exist?

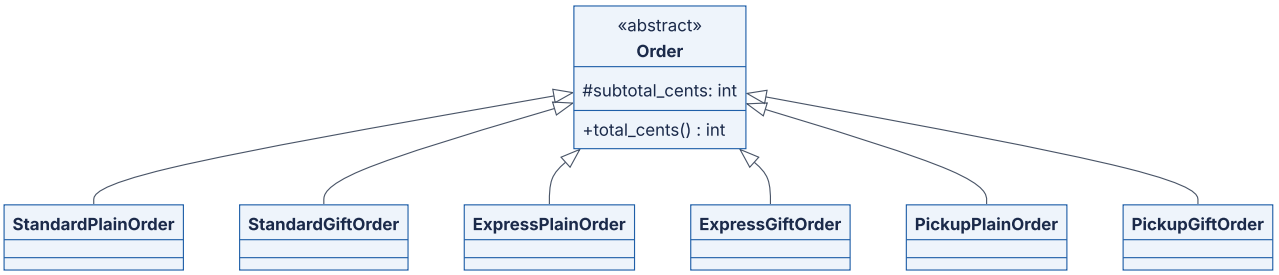
Inheritance is compile-time, single-axis and white-box. Requirements vary along several axes, change at run time, and come from teams who should not read each other's internals. Composition makes each axis a part behind a small interface, replaceable at run time and known only through its public methods (**black-box reuse**).

How it works internally

1. Identify each independent axis of variation (shipping, packaging).
2. For each axis, define a small interface (`ShippingPolicy.cost_cents(subtotal)` , `Packaging.cost_cents()`) and one class per variant.
3. Give the whole (`Order`) one field per axis, set in the constructor.
4. Implement the whole's methods by **delegating** to the parts.
5. To change behavior at run time, replace a part (`order.upgrade_shipping(ExpressShipping())`).

Architecture diagram

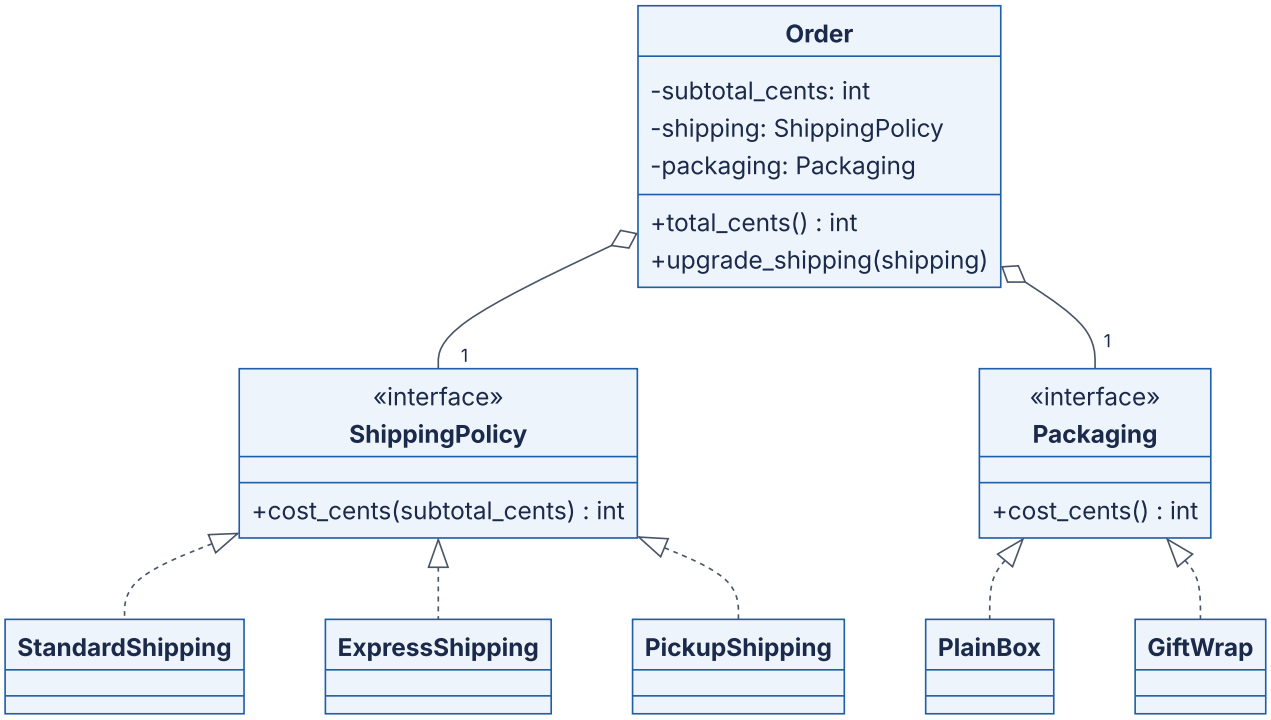
Figure 35.7 — Before: one subclass per combination.



Reading the diagram.

- **Components and direction.** One abstract `Order` and six subclasses, one per (shipping, packaging) pair for 3 × 2 options; all arrows are inheritance.
- **Sync/async and storage.** In-process only.
- **Failure points.** The gift-wrap price is duplicated in three subclasses; missing one in a price change gives inconsistent totals. Orders cannot switch shipping.
- **Scaling.** Classes grow as the *product* of options per axis: the "class explosion".

Figure 35.8 — After: an order composed of a shipping policy and a packaging.



Reading the diagram.

- **Components.** `Order` has (`o--`) one `ShippingPolicy` and one `Packaging`, each an interface with its own family of classes.
- **Direction and sync.** `total_cents()` synchronously calls both parts and adds the results: delegation.
- **Storage.** A database row would store the chosen *codes*; a factory rebuilds the objects (Ch 37).
- **Failure points.** Each price rule exists once; a missing part should be rejected by the constructor.
- **Scaling.** A new option is one class; classes grow as the *sum* of options.

Mathematical foundations

FORMULA

Class count: inheritance per combination versus composition per axis

$$N_{\text{inherit}} = \prod_{i=1}^{a} k_i \qquad N_{\text{compose}} = 1 + \sum_{i=1}^{a} k_i$$

Symbol	Meaning	Units
a	Number of independent axes of variation	count
k_i	Number of options on axis i	count
N_{inherit}	Concrete subclasses needed to cover every combination	classes
N_{compose}	Classes needed with composition: one whole plus one class per option	classes

WORKED EXAMPLE

Worked example — Adding payment timing as a third axis.

1. Current axes: shipping has $k_1 = 3$ options (standard, express, pickup), packaging $k_2 = 2$ (plain, gift).
2. Inheritance: $3 \times 2 = 6$ subclasses. Composition: $1 + 3 + 2 = 6$ classes. Equal counts, but composition already wins on duplication and run-time change.
3. Add payment timing, $k_3 = 3$ (now, on delivery, installments), and an eco box, so $k_2 = 3$.
4. Inheritance: $N = 3 \times 3 \times 3 = 27$ subclasses. Composition: $N = 1 + 3 + 3 + 3 = 10$ classes.
5. One more shipping option ($k_1 = 4$): inheritance needs $4 \times 3 \times 3 = 36$, that is 9 *new* classes; composition needs 11, that is 1 new class.

Interpretation. Each additional axis multiplies the inheritance count but only adds to the composition count.

Engineering implication. At the second independent axis, stop subclassing along it, while N_{inherit} is still small.

Code example

The code also fixes §35.5.2's `CountingCart`: instead of *being* a cart it *has* one and forwards to it, a **wrapper class** [Bloch 2018, Item 18]. The inner cart's `add_all` calls the inner `add`, never the wrapper's, so the count is right whatever the cart team changes.

```
        # --- Parts: each varies independently ------------------------------------
        class StandardShipping:
            def cost_cents(self, subtotal_cents):
                return 0 if subtotal_cents >= 5000 else 499

        class ExpressShipping:
            def cost_cents(self, subtotal_cents):
                return 1299

        class PlainBox:
            def cost_cents(self):
                return 0

        class GiftWrap:
            def __init__(self, message):
                self.message = message
            def cost_cents(self):
                return 350


        # --- The whole: an Order HAS-A shipping policy and HAS-A packaging --------
        class Order:
            def __init__(self, subtotal_cents, shipping, packaging):
                self.subtotal_cents = subtotal_cents
                self.shipping = shipping
                self.packaging = packaging

            def total_cents(self):                          # delegation: ask the parts
                return (self.subtotal_cents
                        + self.shipping.cost_cents(self.subtotal_cents)
                        + self.packaging.cost_cents())

            def upgrade_shipping(self, shipping):           # behavior can change at run
time
                self.shipping = shipping


        order = Order(4200, StandardShipping(), GiftWrap("Happy birthday!"))
        print(order.total_cents())                          # 4200 + 499 + 350 = 5049
        order.upgrade_shipping(ExpressShipping())
        print(order.total_cents())                          # 4200 + 1299 + 350 = 5849


        # --- Fixing the fragile CountingCart with a wrapper -----------------------
        class Cart:
            def __init__(self):
                self._quantities = {}
            def add(self, sku, quantity=1):
                self._quantities[sku] = self._quantities.get(sku, 0) + quantity
            def add_all(self, skus):
                for sku in skus:
                    self.add(sku)
            def quantity_of(self, sku):
                return self._quantities.get(sku, 0)

        class CountingCart:                                 # HAS-A Cart, not IS-A Cart
```

```
        def __init__(self, inner):
            self._inner = inner
            self.items_added = 0
        def add(self, sku, quantity=1):
            self.items_added += quantity
            self._inner.add(sku, quantity)           # forward
        def add_all(self, skus):
            self.items_added += len(skus)
            self._inner.add_all(skus)                # inner calls ITS add, not ours
        def quantity_of(self, sku):
            return self._inner.quantity_of(sku)

    cart = CountingCart(Cart())
    cart.add_all(["SKU-MUG", "SKU-TEA", "SKU-PEN"])
    print(cart.items_added)                          # 3
```

```
        import java.util.HashMap;
        import java.util.List;
        import java.util.Map;

        interface ShippingPolicy { long costCents(long subtotalCents); }
        interface Packaging { long costCents(); }

        final class StandardShipping implements ShippingPolicy {
            public long costCents(long subtotalCents) { return subtotalCents >= 5000 ?
0 : 499; }
        }
        final class ExpressShipping implements ShippingPolicy {
            public long costCents(long subtotalCents) { return 1299; }
        }
        final class PlainBox implements Packaging {
            public long costCents() { return 0; }
        }
        record GiftWrap(String message) implements Packaging {
            public long costCents() { return 350; }
        }

        final class Order {
            private final long subtotalCents;
            private ShippingPolicy shipping;                // HAS-A
            private final Packaging packaging;              // HAS-A

            Order(long subtotalCents, ShippingPolicy shipping, Packaging packaging) {
                this.subtotalCents = subtotalCents;
                this.shipping = shipping;
                this.packaging = packaging;
            }

            long totalCents() {                             // delegation
                return subtotalCents + shipping.costCents(subtotalCents) +
packaging.costCents();
            }

            void upgradeShipping(ShippingPolicy shipping) { this.shipping = shipping; }
        }

        // Wrapper (forwarding) version of CountingCart.
        interface CartOps {
            void add(String sku, int quantity);
            void addAll(List<String> skus);
            int quantityOf(String sku);
        }

        final class Cart implements CartOps {
            private final Map<String, Integer> quantities = new HashMap<>();
            public void add(String sku, int quantity) { quantities.merge(sku, quantity,
Integer::sum); }
            public void addAll(List<String> skus) { for (String s : skus) add(s, 1); }
            public int quantityOf(String sku) { return quantities.getOrDefault(sku, 0);
}
        }
```

```
        final class CountingCart implements CartOps {      // HAS-A CartOps
            private final CartOps inner;
            int itemsAdded = 0;
            CountingCart(CartOps inner) { this.inner = inner; }

            public void add(String sku, int quantity) { itemsAdded += quantity;
inner.add(sku, quantity); }
            public void addAll(List<String> skus) { itemsAdded += skus.size();
inner.addAll(skus); }
            public int quantityOf(String sku) { return inner.quantityOf(sku); }
        }

        class Main {
            public static void main(String[] args) {
                Order order = new Order(4200, new StandardShipping(), new
GiftWrap("Happy birthday!"));
                System.out.println(order.totalCents());          // 5049
                order.upgradeShipping(new ExpressShipping());
                System.out.println(order.totalCents());          // 5849

                CountingCart cart = new CountingCart(new Cart());
                cart.addAll(List.of("SKU-MUG", "SKU-TEA", "SKU-PEN"));
                System.out.println(cart.itemsAdded);             // 3
                System.out.println(new Order(6000, new StandardShipping(), new
PlainBox()).totalCents()); // 6000
            }
        }
```

What to notice:

- `upgrade_shipping` changes an existing order's behavior; inheritance would need a new object of another class.
- The wrapper implements `CartOps` like `Cart`, so it is substitutable without inheriting code, at the cost of one forwarding method per interface method.

Table 35.7 — Inheritance versus composition.

Aspect	Inheritance (is-a)	Composition (has-a)
What it is	Child class extends parent	Object holds parts and delegates
How it works	Method lookup up the class chain	Explicit calls to fields
Advantages	Little code for true subtypes; substitutability	Independent axes; run-time change; black-box reuse
Disadvantages	White-box coupling; fixed at compile time; class explosion	Forwarding boilerplate; more objects to wire together
Performance	One dispatch	One extra call per delegation (usually inlined by the JIT)
Operational complexity	Low at first, rises with depth	Moderate and stays flat
Choose when	Child keeps every promise of the parent and there is one axis	Several axes, run-time choice, or reuse across team boundaries

Real-world industry applications

INDUSTRY EXAMPLE

Java collection wrappers. `Collections.unmodifiableList`, `synchronizedList` and `checkedList` return wrappers that hold the original list and forward calls, adding read-only behavior, locking or type checks. Because they compose, each works with *any* `List` and they can be stacked, which subclassing `ArrayList` could not achieve. *Evidence: Primary* — [Oracle: java.util.Collections API documentation]; [Bloch 2018, Item 18]

INDUSTRY EXAMPLE

Shop order variants. The refactor from six combination subclasses to a composed order is a teaching example, not any company's system. *Evidence: Illustrative* — hypothetical design.

Failure scenarios

Table 35.8 — Failure scenarios when moving to composition.

What goes wrong	Why	Detection	Recovery	Prevention
`None` / `null` part causes crash at checkout	Part not supplied	Error on first total	Supply a default part	Validate parts in the constructor
Wrapper forgets a newly added method	Interface grew, wrapper not updated	Compile error (Java); `AttributeError` (Python)	Add the forwarder	Keep interfaces small; tests on wrappers
Too many tiny objects	Composition applied where nothing varies	Hard-to-follow wiring code	Inline unused parts	Compose along *real* axes only

COMMON MISTAKE

Misconception — "Composition over inheritance means never inherit." The GoF say *favor*. Implementing interfaces, extending classes designed for extension, and true single-axis subtypes are fine; composition is the default when unsure.

35.9 Interfaces and contracts

35.9.1 Design by contract

DEFINITION

Definition — Design by contract. A method of specifying software in which each routine states a **precondition** (what must be true when it is called: the caller's obligation), a **postcondition** (what it guarantees when it returns: the supplier's obligation), and each class states an **invariant** (what is true of every instance between calls). The approach, and the term, come from Bertrand Meyer and the Eiffel language [Meyer 1997].

First-principles explanation

A courier promises: "If the parcel weighs at most 30 kg and is properly addressed, we deliver within two days." The first half is your obligation (precondition), the second the courier's promise (postcondition). Send 40 kg and the courier owes you nothing.

`reserve(sku: str, quantity: int)` gives types but not that `quantity` must be positive and at most what is available, or that availability drops by exactly `quantity`. A contract makes those rules part of the interface.

Why does this exist?

Without contracts, either nobody checks (bad input corrupts state far from the cause) or every layer checks differently, bloating code and still leaving gaps [Meyer 1997]. A contract assigns each check to one side. It also defines correct inheritance: a subclass must honor its parent's contract, formalized as **behavioral subtyping** [Liskov & Wing 1994] and developed in §36.5.

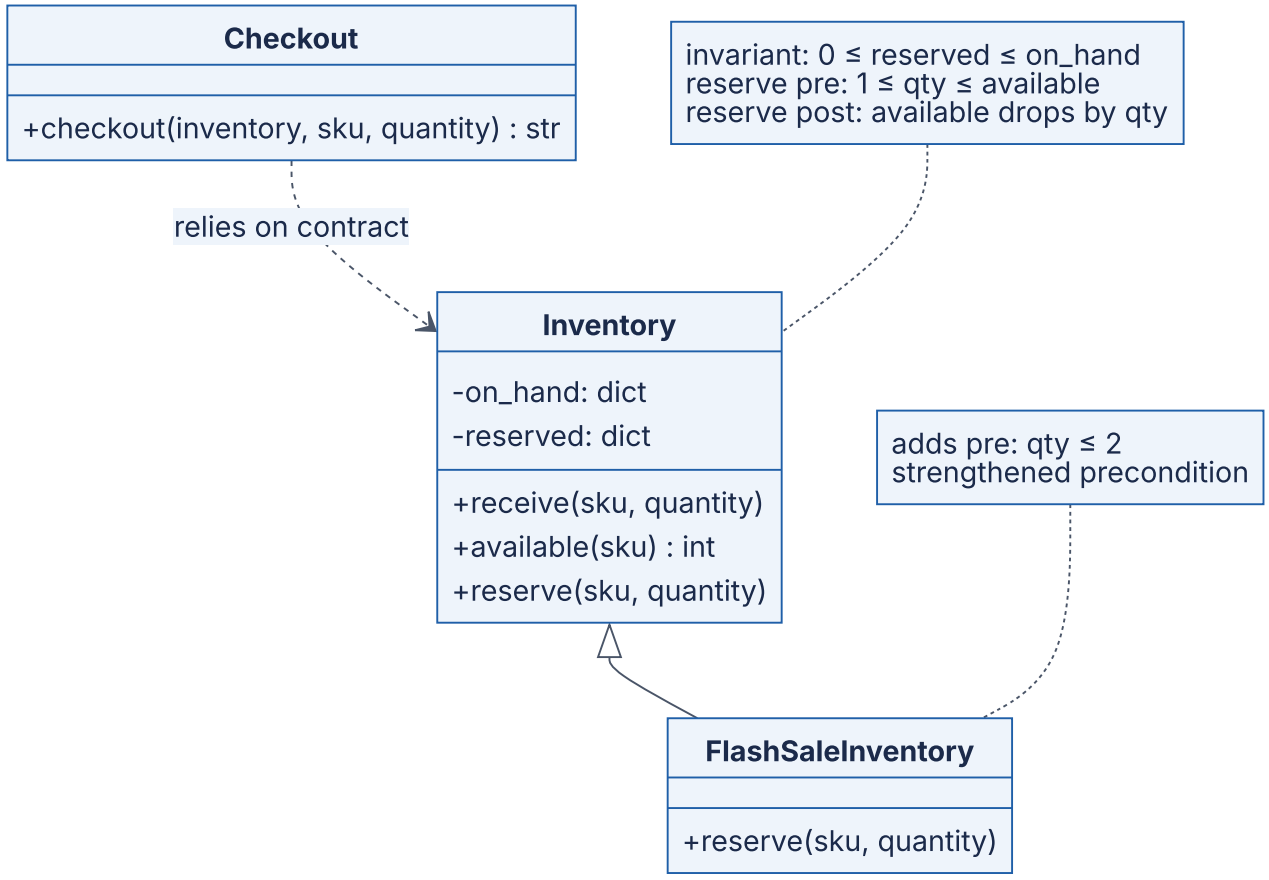
How it works internally

1. **Preconditions** are checked on entry; public APIs fail fast with `ValueError` or `IllegalArgumentException` [Bloch 2018, Item 49].
2. **Postconditions** relate the result to the inputs and the *old* state: save the old value, assert afterwards.
3. **Invariants** are checked at the end of the constructor and every state-changing public method.
4. **Assertions** can be switched off (`python -O`; Java without `-ea`), so never validate user input with `assert`.
5. **Meyer's rule.** A subclass may **weaken** the precondition and **strengthen** the postcondition, never the reverse, and must keep the invariant. Eiffel enforces this with

`require else` and `ensure then`.

Architecture diagram

Figure 35.9 — The inventory contract and a subclass that breaks it.



Reading the diagram.

- **Components.** `Inventory` tracks stock on hand and reserved; the notes record its contract and the subclass's extra condition. `Checkout` is client code.
- **Direction and sync.** `Checkout` calls `available` then `reserve` synchronously, written against `Inventory`'s contract but handed a `FlashSaleInventory`.
- **Storage.** In production "check then reserve" must be atomic in the database, for example one `UPDATE ... WHERE available >= qty` (Ch 11), or concurrent checkouts both pass.
- **Failure point.** The subclass rejects a call the parent allows; `Checkout` has no handling for an "impossible" failure.
- **Scaling.** Every new caller and subclass is checked against the same written rules.

Code example

```
        class Inventory:
            """Stock for each SKU.
            Class invariant: for every SKU, 0 <= reserved <= on_hand.
            """

            def __init__(self):
                self._on_hand = {}
                self._reserved = {}

            def receive(self, sku, quantity):
                # Precondition (caller's duty): quantity >= 1
                if quantity < 1:
                    raise ValueError("precondition: quantity >= 1")
                self._on_hand[sku] = self._on_hand.get(sku, 0) + quantity
                self._reserved.setdefault(sku, 0)
                self._check_invariant(sku)

            def available(self, sku):
                return self._on_hand.get(sku, 0) - self._reserved.get(sku, 0)

            def reserve(self, sku, quantity):
                # Preconditions (caller's duty)
                if quantity < 1:
                    raise ValueError("precondition: quantity >= 1")
                if quantity > self.available(sku):
                    raise ValueError("precondition: quantity <= available(sku)")
                old_available = self.available(sku)

                self._reserved[sku] += quantity

                # Postcondition (supplier's promise) and invariant
                assert self.available(sku) == old_available - quantity, "postcondition"
                self._check_invariant(sku)

            def _check_invariant(self, sku):
                assert 0 <= self._reserved.get(sku, 0) <= self._on_hand.get(sku, 0),
"invariant"


        inv = Inventory()
        inv.receive("SKU-MUG", 5)
        inv.reserve("SKU-MUG", 3)
        print(inv.available("SKU-MUG"))           # 2
        try:
            inv.reserve("SKU-MUG", 3)             # caller broke the precondition
        except ValueError as e:
            print("refused:", e)


        class FlashSaleInventory(Inventory):
            """BAD subclass: strengthens the precondition (max 2 per reservation)."""
            def reserve(self, sku, quantity):
                if quantity > 2:
                    raise ValueError("flash sale: at most 2 per reservation")
```

```
            super().reserve(sku, quantity)


    def checkout(inventory, sku, quantity):  # written against Inventory's contract
        if quantity <= inventory.available(sku):
            inventory.reserve(sku, quantity)  # contract says this cannot fail
            return "reserved"
        return "out of stock"


    for cls in (Inventory, FlashSaleInventory):
        store = cls()
        store.receive("SKU-TEA", 10)
        try:
            print(cls.__name__, checkout(store, "SKU-TEA", 4))
        except ValueError as e:
            print(cls.__name__, "surprise:", e)
    # 2
    # refused: precondition: quantity <= available(sku)
    # Inventory reserved
    # FlashSaleInventory surprise: flash sale: at most 2 per reservation
```

```
        import java.util.HashMap;
        import java.util.Map;

        /** Class invariant: for every SKU, 0 <= reserved <= onHand. */
        class Inventory {
            private final Map<String, Integer> onHand = new HashMap<>();
            private final Map<String, Integer> reserved = new HashMap<>();

            /** @pre quantity >= 1 */
            void receive(String sku, int quantity) {
                if (quantity < 1) throw new IllegalArgumentException("precondition:
quantity >= 1");
                onHand.merge(sku, quantity, Integer::sum);
                reserved.putIfAbsent(sku, 0);
                assert invariantHolds(sku) : "invariant";
            }

            int available(String sku) {
                return onHand.getOrDefault(sku, 0) - reserved.getOrDefault(sku, 0);
            }

            /** @pre 1 <= quantity <= available(sku)
             *  @post available(sku) == old available(sku) - quantity */
            void reserve(String sku, int quantity) {
                if (quantity < 1)
                    throw new IllegalArgumentException("precondition: quantity >= 1");
                if (quantity > available(sku))
                    throw new IllegalArgumentException("precondition: quantity <=
available(sku)");
                int oldAvailable = available(sku);

                reserved.merge(sku, quantity, Integer::sum);

                assert available(sku) == oldAvailable - quantity : "postcondition";
                assert invariantHolds(sku) : "invariant";
            }

            private boolean invariantHolds(String sku) {
                int r = reserved.getOrDefault(sku, 0);
                return 0 <= r && r <= onHand.getOrDefault(sku, 0);
            }
        }

        /** BAD subclass: strengthens the precondition. */
        class FlashSaleInventory extends Inventory {
            @Override void reserve(String sku, int quantity) {
                if (quantity > 2) throw new IllegalArgumentException("flash sale: at
most 2 per reservation");
                super.reserve(sku, quantity);
            }
        }

        class Main {
            static String checkout(Inventory inventory, String sku, int quantity) {
                if (quantity <= inventory.available(sku)) {
                    inventory.reserve(sku, quantity);     // the contract says this can-
```

```
not fail
                    return "reserved";
                }
                return "out of stock";
            }

            public static void main(String[] args) {
                Inventory inv = new Inventory();
                inv.receive("SKU-MUG", 5);
                inv.reserve("SKU-MUG", 3);
                System.out.println(inv.available("SKU-MUG"));                  // 2
                try {
                    inv.reserve("SKU-MUG", 3);
                } catch (IllegalArgumentException e) {
                    System.out.println("refused: " + e.getMessage());
                }
                for (Inventory store : new Inventory[] { new Inventory(), new
FlashSaleInventory() }) {
                    store.receive("SKU-TEA", 10);
                    String name = store.getClass().getSimpleName();
                    try {
                        System.out.println(name + " " + checkout(store, "SKU-TEA", 4));
                    } catch (IllegalArgumentException e) {
                        System.out.println(name + " surprise: " + e.getMessage());
                    }
                }
            }
        }
```

What to notice:

- Preconditions always run; postconditions and invariants use `assert` (Java was run with `-ea`). Eiffel's `old` keyword is a local variable here.
- `FlashSaleInventory` compiles and passes its own tests yet breaks `checkout`: types check *signatures*, only contracts catch *behavior*. Put the limit in the base contract instead (a queryable `max_per_reservation(sku)`).

Real-world industry applications

INDUSTRY EXAMPLE

Eiffel and mainstream languages. Eiffel has `require`, `ensure` (with `old`) and `invariant` clauses, monitored at run time per a compilation setting and inherited by subclasses. Java approximates this with Javadoc, exceptions, `Objects.requireNonNull`, and `assert` (off by default). *Evidence: Primary* — [Meyer 1997]; [Bloch 2018, Item 49]

Failure scenarios

Table 35.9 — Failure scenarios for contracts.

What goes wrong	Why	Detection	Recovery	Prevention
Subclass rejects valid calls	Strengthened precondition	Unhandled exception in client code	Move rule into the base contract	Meyer's rule; contract tests run against every subclass
Input validation silently off in production	Validation written with `assert`	Bad data in production	Replace with explicit checks	`assert` only for internal conditions
Overselling under load	Check-then-act across two calls, not atomic	Negative stock in reports	Compensate (cancel, refund)	Atomic conditional update in the store
Each layer re-validates differently	No written contract	Conflicting error messages	Document and centralize	One owner per check

COMMON MISTAKE

Misconception — "A subclass that adds a safety check is always safer." Extra demands on *inputs* break callers that obey the parent's contract; extra guarantees on *outputs* are safe.

35.10 Cohesion and coupling

35.10.1 Two measures of a good module

DEFINITION

Definition — Cohesion and coupling. Cohesion is how strongly the elements inside one module (here, one class) belong together: whether they serve a single purpose. **Coupling** is how strongly one module depends on others: how much it must know about them, and how likely a change in one is to force a change in the other. Good designs have **high cohesion and low coupling**. Both terms come from structured design [Stevens, Myers & Constantine 1974].

First-principles explanation

A well-run kitchen has stations: one cook does sauces, another grills. Everything at the sauce station is about sauces (high cohesion). The grill cook needs only "two portions of pepper sauce, please", not the recipe (low coupling). Now imagine one cook who does sauces, the accounts and the window cleaning, and a grill cook who walks over and adjusts the sauce burner directly. Any change to either person's routine breaks the other.

Every principle in this chapter so far is a way to get one or both properties. Encapsulation lowers coupling (callers know only methods). Tell, don't ask raises cohesion (rules sit with

their data). Composition lowers coupling (black-box parts). Contracts make the remaining coupling explicit.

Why does this exist?

Stevens, Myers and Constantine observed in 1974 that the cost of changing a program depends on how far a change spreads. If modules are cohesive, a change in requirements maps to a change in one module. If they are loosely coupled, the change stops there. Their paper and the later book by Yourdon and Constantine [Yourdon & Constantine 1979] ranked kinds of coupling and cohesion from worst to best, and these rankings are still the standard vocabulary.

How it works internally: kinds of coupling

Table 35.10 — Kinds of coupling, from tightest (worst) to loosest (best), with shop examples.

Kind	What it means	Shop example	Fix
Content	One module reads or changes another's internals	Report job edits `cart._lines` directly	Encapsulate (§35.4)
Common	Modules share global mutable data	Every module reads a global `CURRENT_DISCOUNT`	Pass values; one owner
External	Modules share an externally imposed format or device	Two classes parse the same payment-provider file layout	One adapter owns the format
Control	One module passes a flag that steers the other's logic	`total(cart, mode="gift")` with branches inside	Polymorphism or separate methods
Stamp	A whole structure is passed when only part is used	`shipping_cost(order)` reads only the weight	Pass what is needed
Data	Only needed elementary data passed as parameters	`shipping_cost(weight_grams)`	—
Message	Modules interact only by calling public methods	`order.settle()`	— (object-oriented ideal)

Message coupling is not in the 1974 paper; it is commonly added at the loose end for object-oriented systems. **Inheritance** is another strong form of coupling not in the original list: a subclass depends on its parent's implementation (§35.5.2). Coupling is not bad in itself: a system with no coupling does nothing. The goal is that each dependency is *necessary*, *narrow* and *visible*.

How it works internally: kinds of cohesion

Table 35.11 — Kinds of cohesion, from weakest to strongest.

Kind	Elements are grouped because…	Shop example
Coincidental	No real reason	`ShopUtils` with `format_price`, `send_sms`, `parse_csv`
Logical	They do the same *kind* of thing	`Validators` for carts, emails and addresses
Temporal	They run at the same time	`on_startup()` that loads config, warms cache, opens logs
Procedural	They run in a fixed sequence	`checkout_steps()` mixing pricing, stock and email
Communicational	They work on the same data	Methods that all read the order record
Sequential	The output of one is the input of the next	Price → apply discount → add tax
Functional	All contribute to one well-defined task	`Cart`: manage lines and compute totals

Mathematical foundations: the LCOM intuition

You can approximate cohesion mechanically. If the methods of a class use disjoint sets of fields, the class is probably several classes glued together. Chidamber and Kemerer turned this into a metric, **Lack of Cohesion in Methods** [Chidamber & Kemerer 1994].

FORMULA

LCOM1 (Chidamber–Kemerer lack of cohesion in methods)

$$\text{LCOM} = \begin{cases} |P| - |Q| & \text{if } |P| > |Q| \\ 0 & \text{otherwise} \end{cases}$$

Symbol	Meaning	Units
m	Number of methods in the class	count
I_j	Set of instance fields used by method j	set of fields
P	Pairs of methods (j, k) with $I_j \cap I_k = \varnothing$ (share no field)	pairs
Q	Pairs of methods with $I_j \cap I_k \neq \varnothing$ (share at least one field)	pairs
$\|P\| + \|Q\|$	Total method pairs, $m(m-1)/2$	pairs
LCOM	Lack of cohesion; 0 is best, larger is worse	pairs

A related variant, often called **LCOM4** (attributed to Hitz and Montazeri), counts the **connected components** of the graph in which methods are linked when they share a field. One component means one responsibility; three components suggest three classes.

WORKED EXAMPLE

Worked example — A `CheckoutManager` god class.

The class has six methods with these field uses: `add_item` {lines}; `subtotal` {lines}; `apply_coupon` {coupon, lines}; `send_receipt` {smtp, email_template}; `render_invoice` {pdf_engine, email_template}; `log_event` {logger}.

1. Total pairs: $m(m-1)/2 = 6 \times 5/2 = 15$.
2. Pairs that share a field (Q): add_item–subtotal, add_item–apply_coupon, subtotal–apply_coupon (all share `lines`), send_receipt–render_invoice (share `email_template`). $|Q| = 4$.
3. Pairs that share nothing: $|P| = 15 - 4 = 11$.
4. LCOM $= 11 - 4 = 7$.
5. Connected components: {add_item, subtotal, apply_coupon}, {send_receipt, render_invoice}, {log_event}: three.
6. After splitting out the cart methods into `Cart`: 3 methods, 3 pairs, all sharing `lines`, so $|P| = 0$, $|Q| = 3$ and LCOM $= 0$.

Interpretation. An LCOM of 7 on a six-method class means most pairs of methods have nothing to do with each other; the three components name the classes hiding inside: `Cart`, a receipt/invoice sender, and a logger that should be injected rather than owned.

Engineering implication. Use LCOM as a *pointer* to classes worth reading, not as a target to optimize. It ignores methods that call each other, treats getters as tiny methods that inflate P, and varies between tools that use different LCOM definitions. A high value plus a vague class name (`Manager`, `Helper`, `Utils`) is a strong signal.

The calculation is mechanical, so here it is as a short script (Python only, since it is a measuring tool rather than an object-oriented construct):

```
        from itertools import combinations

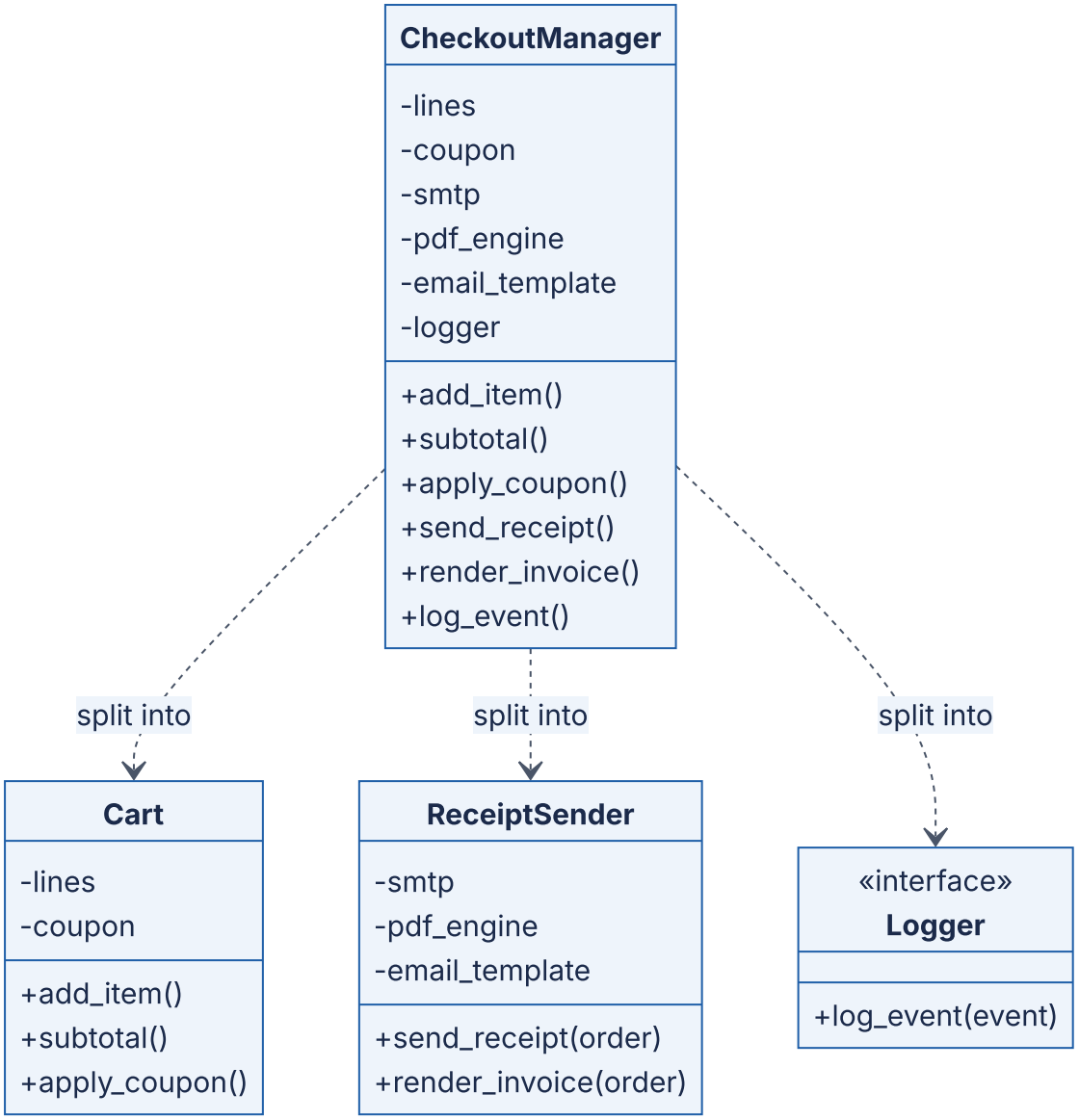

        def lcom1(uses):
            """Chidamber-Kemerer LCOM: uses maps method name -> set of attributes it
touches."""
            p = q = 0
            for a, b in combinations(uses, 2):
                if uses[a] & uses[b]:
                    q += 1                      # pair shares at least one attribute
                else:
                    p += 1                      # pair shares nothing
            return max(p - q, 0), p, q


        def components(uses):
            """Groups of methods linked by shared attributes (the LCOM4 intuition)."""
            groups = []
            for method, attrs in uses.items():
                merged = [g for g in groups if g[1] & attrs]
                names, fields = {method}, set(attrs)
                for g in merged:
                    names |= g[0]
                    fields |= g[1]
                    groups.remove(g)
                groups.append((names, fields))
            return [sorted(g[0]) for g in groups]


        checkout_manager = {
            "add_item":       {"lines"},
            "subtotal":       {"lines"},
            "apply_coupon":   {"coupon", "lines"},
            "send_receipt":   {"smtp", "email_template"},
            "render_invoice": {"pdf_engine", "email_template"},
            "log_event":      {"logger"},
        }
        print(lcom1(checkout_manager))          # (7, 11, 4)
        for group in components(checkout_manager):
            print(group)
        # ['add_item', 'apply_coupon', 'subtotal']
        # ['render_invoice', 'send_receipt']
        # ['log_event']

        cart = {k: v for k, v in checkout_manager.items() if k in ("add_item", "subto-
tal", "apply_coupon")}
        print(lcom1(cart))                      # (0, 0, 3)
```

Figure 35.10 — Splitting a low-cohesion class along its LCOM components.



Reading the diagram.

- **Components.** Left: the original god class with six fields and six unrelated methods. Right: three classes, one per connected component found by the script.
- **Relationships.** The dotted "split into" arrows show the refactoring, not run-time calls. After the split, an order-placement service would call `Cart` for totals and `ReceiptSender` after payment, and receive a `Logger` from outside.
- **Sync/async.** Sending a receipt is a natural candidate for asynchronous processing via a queue (Ch 30), which is only easy once it is its own class.
- **Storage.** `Cart` state goes to the cart store; `ReceiptSender` is stateless apart from its configuration.
- **Failure points.** In the god class, an SMTP outage can fail a method that also holds cart state; after the split, a receipt failure cannot corrupt the cart.
- **Scaling.** Each new class can change, be tested and, in a service architecture, be deployed independently.

35.10.2 The Law of Demeter

> DEFINITION
>
> **Definition — Law of Demeter (object form).** A method `M` of an object `O` should call methods only on: `O` itself; `M`'s parameters; objects `M` creates; and `O`'s own fields (its direct parts) [Lieberherr & Holland 1989]. Informally: "talk only to your immediate friends", or "use only one dot" (excluding fluent builders and value-object chains).

First-principles explanation

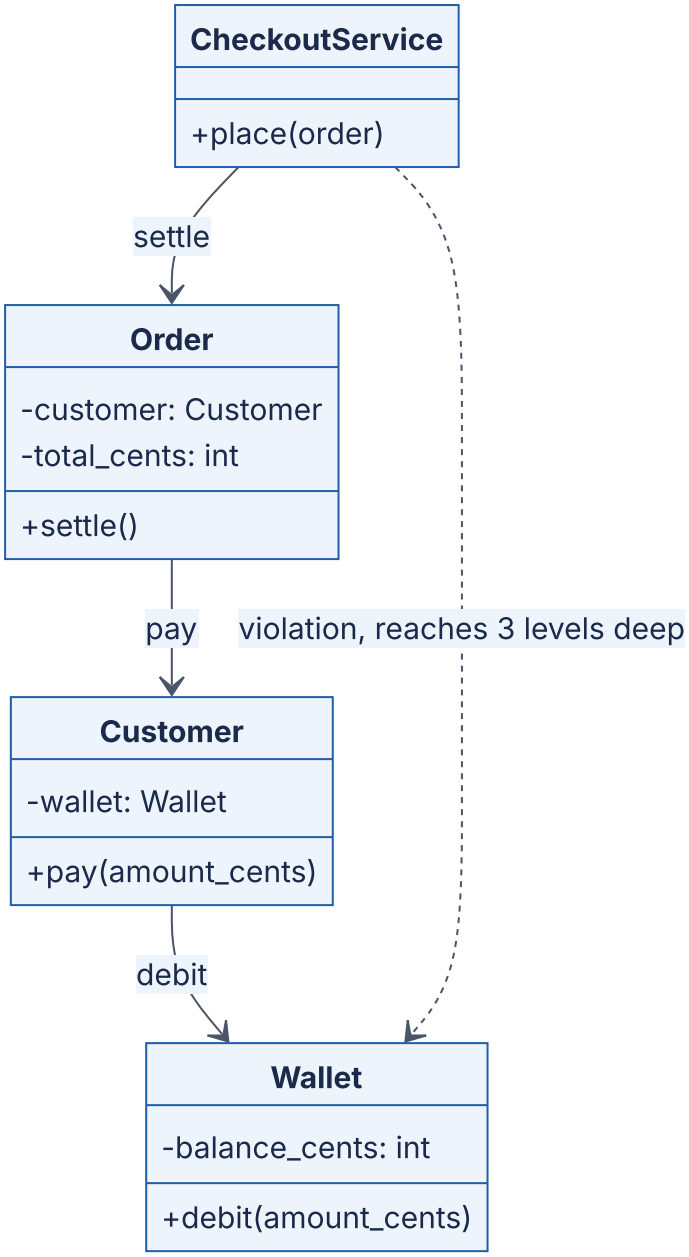
When you pay in a shop, the cashier asks you to pay. The cashier does not reach into your bag, open your wallet, take out a card and swipe it. If you change wallets, or pay by phone, the cashier's procedure does not change. Code like `order.customer.wallet.balance_cents -= total` is the cashier rummaging in your bag: the checkout now depends on `Order` having a customer, the customer having a wallet, and the wallet storing its balance in a field called `balance_cents`. Three classes' internals, one line. Martin calls such chains **train wrecks** [Martin 2008, ch. 6].

Why does this exist?

The Law of Demeter was formulated in the late 1980s by Karl Lieberherr's Demeter project at Northeastern University as a style rule that limits how much of the object structure any one method knows. Each method that obeys it depends only on its immediate neighbors, so a change to one class's structure affects only that class's neighbors. It is the object-level form of low coupling, and it pushes you towards tell, don't ask: instead of reaching through objects, you add a method to the nearest one.

Architecture diagram

Figure 35.11 — Reaching through objects versus talking to neighbors.



Reading the diagram.

- **Components.** A checkout service, an order, the customer who placed it and the customer's wallet.
- **Direction and sync.** Solid arrows are the compliant path: each object calls only its direct part, synchronously. The dotted arrow is the train-wreck dependency that compliant code removes.
- **Storage.** The wallet balance would be persisted by the payment component; only `Wallet` knows its representation.
- **Failure points.** With the violation, a change to `Wallet` (say, balances move to an external provider) breaks `CheckoutService`. With the compliant design, only `Customer` might change.

- **Scaling.** Adding a second way to pay (a company account) changes `Customer.pay`, not every caller.

Code example

```
        class Wallet:
            def __init__(self, balance_cents):
                self._balance_cents = balance_cents

            def debit(self, amount_cents):
                if amount_cents > self._balance_cents:
                    raise ValueError("insufficient funds")
                self._balance_cents -= amount_cents

            def balance_cents(self):
                return self._balance_cents


        class Customer:
            def __init__(self, name, wallet):
                self.name = name
                self._wallet = wallet

            def pay(self, amount_cents):                  # Customer talks to its own
part
                self._wallet.debit(amount_cents)


        class Order:
            def __init__(self, customer, total_cents):
                self._customer = customer
                self._total_cents = total_cents
                self.paid = False

            def settle(self):                             # Order talks to its own part
                self._customer.pay(self._total_cents)
                self.paid = True


        # Violates the Law of Demeter: the checkout reaches through three objects.
        #   order.customer.wallet.balance_cents -= order.total_cents
        # Follows it: the checkout talks only to the order.
        wallet = Wallet(10_000)
        order = Order(Customer("Ada", wallet), 4548)
        order.settle()
        print(order.paid, wallet.balance_cents())         # True 5452
```

```
        final class Wallet {
            private long balanceCents;
            Wallet(long balanceCents) { this.balanceCents = balanceCents; }

            void debit(long amountCents) {
                if (amountCents > balanceCents) throw new IllegalStateException("insuf-
ficient funds");
                balanceCents -= amountCents;
            }
            long balanceCents() { return balanceCents; }
        }

        final class Customer {
            private final String name;
            private final Wallet wallet;
            Customer(String name, Wallet wallet) { this.name = name; this.wallet = wal-
let; }

            String name() { return name; }
            void pay(long amountCents) { wallet.debit(amountCents); }   // talks to its
own field
        }

        final class Order {
            private final Customer customer;
            private final long totalCents;
            private boolean paid = false;
            Order(Customer customer, long totalCents) { this.customer = customer;
this.totalCents = totalCents; }

            void settle() { customer.pay(totalCents); paid = true; }    // talks to its
own field
            boolean isPaid() { return paid; }
        }

        class Main {
            public static void main(String[] args) {
                // Violation (would need getters all the way down):
                //   order.getCustomer().getWallet().debit(order.getTotalCents());
                Wallet wallet = new Wallet(10_000);
                Order order = new Order(new Customer("Ada", wallet), 4548);
                order.settle();
                System.out.println(order.isPaid() + " " + wallet.balanceCents());   //
true 5452
            }
        }
```

What to notice:

- Each method calls only its own fields: `Order.settle` → `customer.pay`; `Customer.pay` → `wallet.debit`. No class knows the structure two levels away.
- The cost is small **wrapper methods** (`Customer.pay`). If a class accumulates dozens of pass-through methods, that is a sign the caller needs a different object, not that

the law should be broken.
- The law is about *objects with behavior*. Chaining over data or fluent APIs (`text.strip().lower()`, a stream pipeline) returns new values rather than reaching into structure, and is not a violation in spirit.

Real-world industry applications

INDUSTRY EXAMPLE

Demeter project. The Law of Demeter was developed in the Demeter project on adaptive software at Northeastern University and published as a style rule with a formal statement and examples of mechanically transforming non-compliant programs into compliant ones. The authors present it as a heuristic that limits the spread of change, not an absolute law, and discuss the trade-off of extra wrapper methods. *Evidence: Primary* — [Lieberherr & Holland 1989]

Failure scenarios

Table 35.12 — Failure scenarios from poor cohesion and coupling.

What goes wrong	Why	Detection	Recovery	Prevention
Small change touches 15 files	High coupling (content, common, control)	Large diffs for small features; merge conflicts	Introduce interfaces and owners	Message/data coupling; Law of Demeter
SMTP outage blocks cart updates	Low cohesion: unrelated duties in one class	Incident shows unrelated failure	Split class	Review LCOM components and class names
Refactoring `Wallet` breaks checkout	Train-wreck chains	Compile errors far away	Add delegating methods	One-dot rule for behavior objects
Global discount flag changed by a test leaks into others	Common coupling	Flaky tests	Pass configuration explicitly	No global mutable state

COMMON MISTAKE

Misconception — "Low coupling means fewer classes." Merging classes reduces the number of dependencies *between classes* by hiding them inside one class, where they become low cohesion instead. The goal is the right boundaries: each class cohesive, each dependency narrow.

35.11 Common mistakes

The previous sections each ended with a misconception about one idea. Three mistakes combine several ideas and are the ones reviewers and interviewers look for first.

COMMON MISTAKE

Anemic domain model. Classes such as `Order`, `Cart` and `Customer` hold only fields with getters and setters, and all the rules live in "service" classes (`OrderService.calculateTotal(order)`, `CartService.applyCoupon(cart, coupon)`). It looks object-oriented but is procedural code with data structures: every invariant is enforced, or not, by whichever service touches the data, and the same rule is reimplemented in several services. Fowler named it an anti-pattern because it pays the cost of a domain model without the benefit [Fowler 2003]. **Fix:** move each rule next to the data it governs (tell, don't ask, §35.4.3); keep services for coordinating several objects and for infrastructure such as transactions. **Caveat:** for simple create-read-update-delete screens with almost no rules, plain data records and a thin service are a reasonable, honest choice.

COMMON MISTAKE

God class. One class, often named `Manager`, `Processor`, `Helper` or after the whole application, accumulates most of the system's behavior and data: thousands of lines, dozens of fields, and imports from every package. Riel's design heuristics warn against such classes and recommend distributing system intelligence evenly across top-level classes [Riel 1996]. Symptoms: high LCOM with several connected components (§35.10.1), most commits touch it, and its tests are slow and brittle. **Fix:** split along the components that LCOM reveals, starting with the least coupled group; move behavior to the classes whose data it uses.

COMMON MISTAKE

Deep inheritance hierarchies. `Product → ShippableProduct → FragileShippableProduct → FragileShippableImportedProduct → ...`. Each level adds coupling to every level above it (§35.5.2), a reader must open five files to know what one method does, and new requirements that cut across the hierarchy ("imported but not fragile") cause the class explosion of §35.8. Chidamber and Kemerer's depth-of-inheritance-tree metric was proposed partly because deeper classes are harder to predict [Chidamber & Kemerer 1994]. **Fix:** keep hierarchies shallow (one or two levels below an interface or abstract class is typical), turn independent axes into composed parts, and use interfaces for capabilities.

Table 35.13 — Common mistakes at a glance.

Mistake	Quick test	First fix
Anemic domain model	Do the entity classes have any methods besides getters and setters?	Move one rule into its entity
God class	Can you describe the class in one sentence without “and”?	Split along LCOM components
Deep hierarchy	More than two or three levels of concrete classes?	Extract an axis into a composed part
Setter for every field	Can a caller break an invariant with one call?	Replace setters with business operations
Inheritance for reuse only	Would callers ever use the child *as* the parent?	Delegate to a helper object

35.12 Summary and revision notes

SUMMARY

- **Objects** bundle state and behavior; a **class** is their template. Constructors must leave objects valid or refuse to create them. **Identity** (`is` , Java `==`) differs from **equality** (`==` / `__eq__` , `equals`); override equality and hashing together, value equality for value objects, identity for entities.
- **Encapsulation** protects **invariants** and hides decisions likely to change [Parnas 1972]. Expose business operations, not setters; return copies or immutable views; derive values instead of storing them; **tell, don't ask**.
- **Inheritance** models behavioral is-a and enables substitutability, but is white-box reuse: the **fragile base class problem** means subclasses depend on parent internals (`CountingCart` counted 6 instead of 3). Design and document for inheritance, or prohibit it.
- **Polymorphism**: one call, many behaviors. Subtype polymorphism uses **dynamic dispatch**; Python also allows **duck typing**, checkable with `Protocol` ; **generics** abstract over element types.
- **Abstraction**: abstract classes share state and steps and force subclasses to fill gaps; interfaces declare roles and allow many per class. Program to an interface.
- **Composition over inheritance**: one composed part per axis of variation turns $\prod k_i$ subclasses into $1 + \sum k_i$ classes (27 vs 10 for three axes of three) and allows run-time change. Wrappers with forwarding fix fragile subclasses.
- **Design by contract**: preconditions (caller's duty), postconditions (supplier's promise), invariants. Subclasses may weaken preconditions and strengthen postconditions, never the reverse (preview of §36.5).
- **High cohesion, low coupling**. Prefer data and message coupling; LCOM1 $= \max(|P| - |Q|, 0)$ and connected components point at classes to split. **Law of Demeter**: talk only to your immediate friends.
- Watch for the **anemic domain model**, the **god class** and **deep hierarchies**.

35.13 Knowledge checkpoint

1. What is the difference between a class and an object?
2. Two `Product` objects have the same SKU. Under the design of §35.3, what do `a == b` and `a is b` return, and why?
3. Why must `hashCode` change whenever you override `equals` in Java?
4. Name two ways a getter can break encapsulation.
5. In §35.4, why does the cart not store a `total` field?

6. What exactly goes wrong in the `CountingCart` subclass, and why is removing its `add_all` override not a real fix?
7. What is dynamic dispatch? What decides which method runs?
8. How does duck typing differ from Java's interfaces, and what does `typing.Protocol` add?
9. Give one thing an abstract class can do that a Java interface cannot, and one thing an interface can do that an abstract class cannot.
10. A design has 4 shipping options, 3 packagings and 2 payment timings. How many classes does inheritance-per-combination need, and how many does composition need?
11. State Meyer's rule for preconditions and postconditions in a subclass.
12. Why should user-input validation not be written with `assert`?
13. Rank data, control and content coupling from best to worst.
14. Compute LCOM1 for a class with 4 methods in which exactly one pair shares a field.
15. Rewrite `order.get_customer().get_address().get_city()` used to compute shipping so that it obeys the Law of Demeter.

Answers

1. A class is the template (fields and methods); an object is one instance with its own state, created from the class.
2. `a == b` is `True` because `Product.__eq__` compares SKUs; `a is b` is `False` because they are two distinct objects in memory.
3. Hash-based collections find a bucket by hash first; equal objects with different hashes land in different buckets, so lookups fail and sets hold duplicates.
4. Returning an internal mutable collection (callers can modify it), and exposing data that lets callers make decisions the object should make (logic leaks out).
5. The total is derived from lines and coupon; storing it would let it disagree with them, for example keeping a discount after an item removal drops the subtotal below the minimum.
6. `add_all` counts 3, then the base `add_all` calls the overridden `add` 3 times, counting 3 more: 6. Removing the override relies on the base still calling `add`; if the base changes, the count becomes wrong again.
7. Choosing the method implementation at run time from the object's actual class, not the variable's declared type.
8. Duck typing accepts any object that has the methods used, checked when called; Java requires an explicit `implements`. `Protocol` lets a static checker verify the shape before running, without inheritance.
9. Abstract class: hold instance state (or a `final` template method using it). Interface: be implemented by a class that already extends another class (many interfaces per class).
10. Inheritance: $4 \times 3 \times 2 = 24$. Composition: $1 + 4 + 3 + 2 = 10$.

11. A redefinition may only weaken the precondition and only strengthen the postcondition, and must keep the class invariant.
12. Assertions can be disabled (`python -O` , Java without `-ea`), so the check would silently disappear in production.
13. Data (best), control, content (worst).
14. 6 pairs; $|Q| = 1$, $|P| = 5$; LCOM $= 5 - 1 = 4$.
15. Add `order.shipping_city()` (which asks `customer.shipping_city()` , which asks its address), or better, tell: `order.shipping_cost(rate_table)` .

35.14 Interview questions

Basic

Q1. What are the four pillars of object-oriented programming, and how do they relate? Answer. Encapsulation (an object guards its state and invariants), abstraction (callers see what an object does, not how), inheritance (a class extends another, sharing code and type), and polymorphism (one call, behavior chosen by the object). Encapsulation and abstraction create boundaries; polymorphism lets you vary what is behind a boundary; inheritance is one way, and often not the best way, to get polymorphism. Mention that composition and interfaces deliver polymorphism with less coupling.

Q2. When would you use an abstract class rather than an interface? Answer. An abstract class when closely related variants share real state and an algorithm whose order must be fixed, such as `PaymentMethod.pay` validating, charging and recording. An interface for a role or capability that unrelated classes can play (`Refundable`), for module boundaries, and whenever a class may need several roles. A common combination is an interface for the type plus an optional skeletal abstract class.

Q3. Why are public setters for every field considered poor design? Answer. They let any caller put the object in a state the business rules forbid and spread the rules across callers. Replace them with operations named after business actions that check invariants (`apply_coupon` , `remove`), keep immutable values immutable, and derive computed values.

Intermediate

Q4. “Favor composition over inheritance.” When is inheritance still the right choice? Answer. When the child is a behavioral subtype (keeps every promise of the parent), there is a single axis of variation, the parent was designed and documented for extension, and both are maintained together. Implementing an interface is always fine. Otherwise, compose: separate axes into parts, delegate, and swap parts at run time. Quantify: $\prod k_i$ versus $1 + \sum k_i$ classes.

Q5. Explain the fragile base class problem with an example and two remedies. Answer. A subclass overrides `add` and `add_all` to count items; the base `add_all` internally calls `add`, so items are counted twice. Change the base and the subclass breaks the other way. Remedies: wrap instead of extend (forwarding wrapper implementing the same interface), or have the base class document its self-use or be declared `final`.

Q6. Implement `equals` for a `Money(amount_cents, currency)` value object. What must you remember? Answer. Compare both fields; handle `null` and other types; override `hashCode` with the same fields; make the class immutable so the hash cannot change while it is in a set; keep equality reflexive, symmetric and transitive (be careful with subclasses adding fields, which is why value classes are usually `final` or records).

Advanced

Q7. Design the classes for a shop checkout that supports multiple discount rules, payment methods and shipping options. Answer. `Cart` (entity) owns immutable `Line`s and enforces quantity invariants; `Product` and `Money` are value objects. `DiscountRule` interface with one class per rule; a pricing component picks or stacks rules with a cap at the subtotal. `ShippingPolicy` and `Packaging` as composed parts of `Order`. `PaymentMethod` abstract class with a fixed `pay` template and a `Refundable` capability interface. `Inventory` with a written contract for `reserve`, implemented atomically in storage. Walk through one checkout with a sequence of calls, then discuss extension (new rule = new class), testing (fakes for payment) and failure handling (payment timeouts need idempotency, §22.15).

Q8. Is a `Square` a `Rectangle`? Answer. Mathematically yes; behaviorally only if rectangles are immutable. If `Rectangle` has `set_width` with the postcondition "width changes, height unchanged", a `Square` must change both, which weakens the postcondition, so code written for rectangles breaks. Options: make shapes immutable, give them a common `Shape` interface without setters, or do not relate them. This is the behavioral subtyping idea of Liskov and Wing, developed in §36.5.

Q9. You inherit a 6,000-line `OrderManager`. How do you approach it? Answer. Add characterization tests around current behavior first. Map which methods use which fields (LCOM components) and which callers use which methods. Extract the most independent component first behind an interface, delegate from the old class so callers keep working, and repeat. Move rules into the entities they govern. Track size, coupling and change frequency to show progress, and avoid a big-bang rewrite.

35.15 Practical exercises

1. **Money value object.** Write `Money(amount_cents, currency)` in Python (frozen dataclass) and Java (record) with addition that refuses mixed currencies. Show that

equal values hash equally and work as set elements.

2. **Break and fix encapsulation.** Change §35.4's `lines()` to return the internal dictionary, write a caller that breaks invariant 1, then restore the snapshot and show the caller now fails.
3. **Fragile base, other direction.** Remove `add_all` from `CountingCart` (§35.5.2), confirm the count is 3, then rewrite the base `add_all` to update the dictionary directly and show the count becomes 0 for `add_all`. Then switch to the wrapper of §35.8 and repeat.
4. **A fourth discount.** Add `CategoryPercentOff` to §35.6 without editing `best_discount`. Then add a rule that stacks discounts but caps the total at the subtotal.
5. **Contract tests.** Write one test function that takes any `Inventory` and checks the `reserve` contract; run it against both classes of §35.9.
6. **Measure cohesion.** Run the LCOM script on a class from a codebase you know, by listing methods and the fields each uses. Do the components suggest a split?

35.16 Design challenges

1. **Library loans.** Design classes for a library that lends books, e-books and DVDs with different loan periods and late fees, and members with different borrowing limits. Identify axes of variation, decide inheritance versus composition, write the contract of `lend(item, member)`, and draw a class diagram.
2. **Notification system.** Orders must notify customers by email, SMS or push, in several languages, with immediate or daily-digest timing. Show the class explosion an inheritance design would cause, compute it, and give a composed design.
3. **Refactor a god class.** Given a `CheckoutManager` with pricing, stock reservation, payment, receipts and analytics, propose the target classes, the order of extraction, how callers keep working during the change, and how you would measure that cohesion improved.

35.17 References

1. Bloch, J. *Effective Java*, 3rd ed. Addison-Wesley, 2018.
2. Cardelli, L., Wegner, P. "On Understanding Types, Data Abstraction, and Polymorphism." *ACM Computing Surveys* 17(4):471–522, 1985.
3. Chidamber, S. R., Kemerer, C. F. "A Metrics Suite for Object Oriented Design." *IEEE Transactions on Software Engineering* 20(6):476–493, 1994.
4. Fowler, M. "AnemicDomainModel." martinfowler.com (bliki), 2003. https://martinfowler.com/bliki/AnemicDomainModel.html
5. Fowler, M. "TellDontAsk." martinfowler.com (bliki), 2013. https://martinfowler.com/bliki/TellDontAsk.html

6. Gamma, E., Helm, R., Johnson, R., Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*, ch. 1 “Introduction”. Addison-Wesley, 1994.
7. Hitz, M., Montazeri, B. “Measuring Coupling and Cohesion in Object-Oriented Systems.” *Proceedings of the International Symposium on Applied Corporate Computing*, 1995.
8. Levkivskyi, I., Lehtosalo, J., Langa, Ł. *PEP 544 — Protocols: Structural Subtyping (Static Duck Typing)*. Python Software Foundation, 2017. https://peps.python.org/pep-0544/
9. Lieberherr, K. J., Holland, I. M. “Assuring Good Style for Object-Oriented Programs.” *IEEE Software* 6(5):38–48, 1989.
10. Liskov, B. H., Wing, J. M. “A Behavioral Notion of Subtyping.” *ACM Transactions on Programming Languages and Systems* 16(6):1811–1841, 1994.
11. Martin, R. C. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
12. Meyer, B. *Object-Oriented Software Construction*, 2nd ed. Prentice Hall, 1997.
13. Oracle. *Java SE 21 API documentation*: `java.util.Stack`, `java.util.Collections`, `java.time` package. https://docs.oracle.com/en/java/javase/21/docs/api/
14. Parnas, D. L. “On the Criteria To Be Used in Decomposing Systems into Modules.” *Communications of the ACM* 15(12):1053–1058, 1972.
15. Python Software Foundation. *Python 3 documentation*: `abc` and `collections.abc` modules. https://docs.python.org/3/library/collections.abc.html
16. Riel, A. J. *Object-Oriented Design Heuristics*. Addison-Wesley, 1996.
17. Stevens, W. P., Myers, G. J., Constantine, L. L. “Structured Design.” *IBM Systems Journal* 13(2):115–139, 1974.
18. Yourdon, E., Constantine, L. L. *Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design*. Prentice Hall, 1979.

35.18 Open verification items

- Exact list and ordering of coupling kinds in Stevens, Myers & Constantine (1974) versus later refinements (Myers 1975; Yourdon & Constantine 1979); Table 35.10 uses the commonly taught seven-level ordering, with message coupling as a later addition.
- Publication venue of Hitz & Montazeri (1995) for LCOM4.
- Publication years of Fowler’s “TellDontAsk” (believed 2013) and “AnemicDomainModel” (believed 2003) bliki entries.
- Item numbers cited from *Effective Java*, 3rd ed. (10–11, 17–20, 49–50): confirm against the printed edition.

Chapter 36 — SOLID Principles

> **Level:** 2 · **Track:** A (Traditional) · **Estimated reading time:** 4 h

Chapter 35 gave you the raw materials of object-oriented design: classes, encapsulation, inheritance, polymorphism, interfaces, cohesion and coupling. Those tools say what you *can* build. They do not say which of the many possible arrangements will still be pleasant to change in two years, when the tax rules have changed three times, marketing has rewritten every email, and the payment provider has been replaced. **SOLID** is a set of five design principles that answer that question at the level of classes and modules. Robert C. Martin collected and named them in the late 1990s and early 2000s [Martin 2002]; the acronym itself came later.

Table 36.1 — The five SOLID principles at a glance.

Letter	Principle	One-line test
S	Single Responsibility	Does this module answer to exactly one group of people?
O	Open/Closed	Can I add the next expected variant without editing working code?
L	Liskov Substitution	Can every subtype be used wherever its base type is expected, without surprises?
I	Interface Segregation	Does each client depend only on the methods it calls?
D	Dependency Inversion	Does policy code depend on abstractions that it owns, not on concrete details?

All five serve one goal: to **limit how far a change spreads**. A design in which one business decision touches one module, which is recompiled, retested and redeployed alone, is cheap to change. A design in which every change ripples through ten files owned by four teams is expensive, slow and fragile. SOLID gives you five specific ways to stop the ripple.

The chapter uses one running example throughout: the **order billing and notification module** of an online shop. When a customer places an order, the module computes an invoice (merchandise, gift-wrap fees, sales tax), stores the order, charges the customer's card and tells the customer, by email, SMS, push notification or webhook. For each principle you will see a violating version, the pain it causes, and the refactor in Python and

Java. The final section lifts the principles to components and services, and shows when they do more harm than good.

36.1 Learning objectives

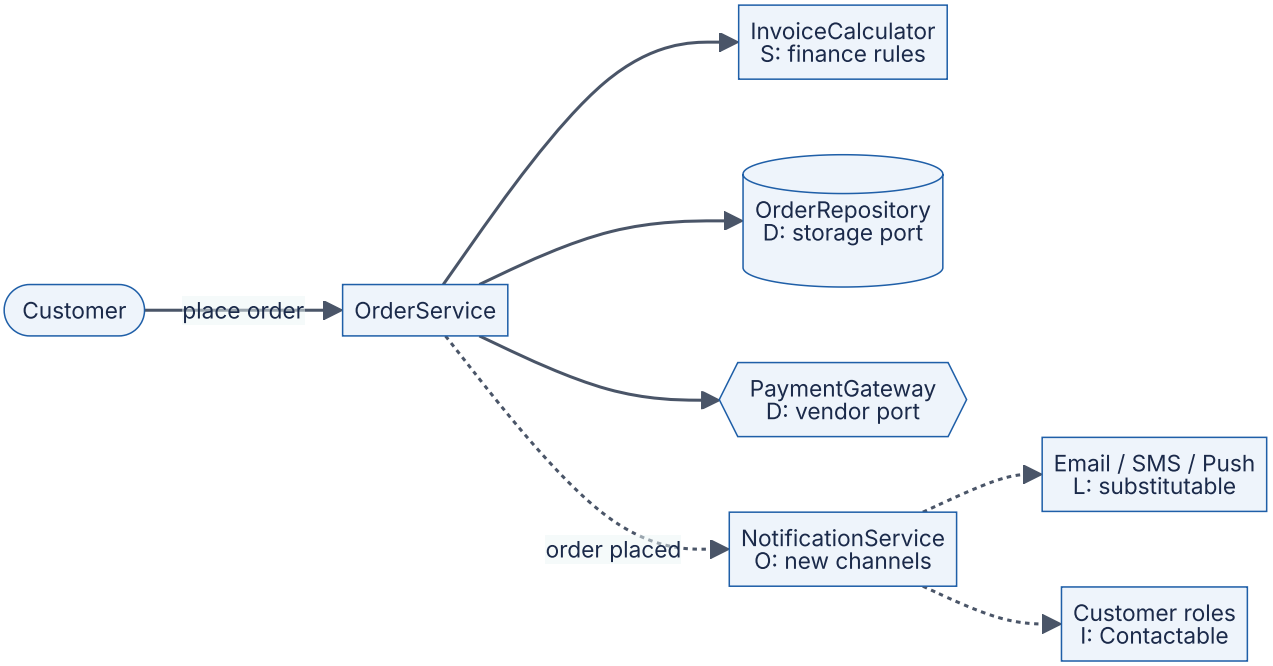
After this chapter you should be able to:

- State each SOLID principle precisely, including its original source, and explain the problem it solves in terms of change and dependency.
- Identify the **actors** behind a class and split it so that each module answers to one actor.
- Distinguish Meyer’s inheritance-based open–closed principle from Martin’s abstraction-based version, and use strategy-style polymorphism to add variants without editing existing code.
- Apply the Liskov–Wing rules (preconditions may be weakened, postconditions strengthened, invariants and history preserved) to decide whether a subtype is safe, and write a reusable contract test.
- Split a fat interface into role interfaces and explain the effect on coupling, rebuilds and stubs.
- Explain dependency inversion, dependency injection and inversion of control as three different ideas, and use fakes to test policy code without networks or databases.
- Map SOLID to component and service boundaries, compute Martin’s instability and abstractness metrics, and recognize when an abstraction is needless.
- Discuss SOLID trade-offs fluently in a low-level design interview.

36.2 Prerequisites

- Classes, encapsulation and interfaces — §35.3, §35.4, §35.9 (every principle is about how classes and interfaces are arranged).
- Inheritance and polymorphism — §35.5, §35.6 (open/closed and Liskov substitution are rules for using them safely).
- Composition over inheritance — §35.8 (most refactors in this chapter replace inheritance with composition).
- Design by contract — §35.9.1 (preconditions and postconditions are the language of Liskov substitution).
- Cohesion and coupling — §35.10 (single responsibility sharpens cohesion; dependency inversion manages coupling).
- Optional: layered and hexagonal architecture — §33.3, §33.10 (needed only for §36.8).

Figure 36.1 — The running example and where each principle acts.



Reading the diagram. The customer (stadium) places an order through `OrderService`, the coordinating class. Solid arrows are synchronous calls made while the customer waits: computing the invoice, storing the order (cylinder, the storage boundary) and charging the card through an external payment provider (hexagon, the main failure point outside your control). Dotted arrows are asynchronous: notification happens after the order is committed, so a slow SMS provider cannot block checkout. Each box is labeled with the principle that shapes it (S, O, L, I or D); the dependency-inverted ports are also what let storage and payments be scaled or replaced independently.

36.3 Single Responsibility Principle

36.3.1 Definition: one reason to change means one actor

> DEFINITION
>
> **Definition — Single Responsibility Principle (SRP).** A module should be responsible to one, and only one, **actor**, where an actor is a group of people (a role, a team, a stakeholder) who request changes to the module for the same reasons [Martin 2017]. The earlier formulation is “a class should have only one reason to change” [Martin 2002].

The name misleads almost everyone. “Single responsibility” sounds like “a class should do one thing”, which is a different and older rule about functions. A class can do many things (validate, compute, round, format a number) and still have one responsibility, because all of those things change for the same reason. Martin made this explicit in a 2014

essay: the principle is about *people*, and its practical form is “gather together the things that change for the same reasons; separate those things that change for different reasons” [Martin 2014].

First-principles explanation

Think of a shared kitchen notice board. If the cleaning rota, the menu and the fire-safety instructions are written on one sheet of paper, then every time the chef reprints the menu there is a chance the fire-safety section is lost or garbled, and the safety officer never asked for anything to change. Three people with unrelated concerns edit one artifact. Give each concern its own sheet and each person can change their sheet without touching anyone else’s.

Code behaves the same way. Each line in a source file was written to satisfy someone. When two groups of people with different priorities edit the same class, three things go wrong. Their changes **collide** (merge conflicts, contested reviews). One group’s change **breaks** behavior the other group relies on, often through shared helper code. And every change to the class forces **retesting and redeployment** of behavior that did not change.

Why does this exist?

The idea predates the name. Parnas argued in 1972 that a system should be split into modules so that each hides one design decision likely to change, rather than one step in a processing flowchart [Parnas 1972]. Cohesion (§35.10) captured “things in this module belong together”. SRP sharpens cohesion by giving a test you can actually apply: *list the people who would ask for this code to change*. If the list has more than one entry with different motives, the module has more than one responsibility.

How it works internally

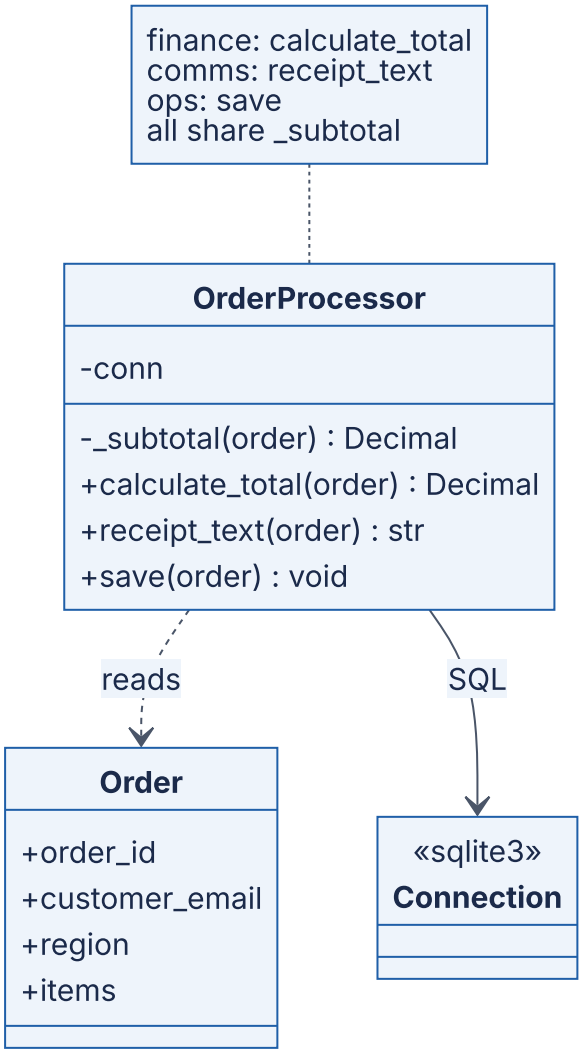
Applying SRP is a four-step procedure:

1. **List the operations** of the class.
2. **Name the actor** for each operation: who would file the ticket that changes it? Use roles (finance, customer communications, operations), not individuals.
3. **Look for shared code** used by operations with different actors. That is where accidental coupling hides.
4. **Split by actor**, give each part its own data or value objects, and, if callers want a single entry point, add a thin **facade** that sequences the parts but contains no business rule.

36.3.2 The violation: one class, three actors

Here is the first version of the billing module. One class computes the total, writes the receipt and saves the order.

Figure 36.2 — Before SRP: one class answers to three actors.



Reading the diagram. One class, `OrderProcessor`, holds a database connection and depends on the `Order` data class (dashed arrow, a usage dependency) and on a SQL connection (solid arrow, an association). Calls are synchronous, in-process method calls. The note lists which actor owns which method. The failure point is the private helper `_subtotal`, which all three public methods use: it is the one place where finance, communications and operations code meet. The storage boundary (the SQL connection) is inside the same class as the tax rules, so a schema change and a tax change produce edits to the same file. This class can only scale by deploying everything together.

```
        from dataclasses import dataclass, field
        from decimal import Decimal, ROUND_HALF_UP
        import sqlite3

        @dataclass
        class LineItem:
            sku: str
            unit_price: Decimal
            quantity: int
            gift_wrap: bool = False

        @dataclass
        class Order:
            order_id: str
            customer_email: str
            region: str
            items: list[LineItem] = field(default_factory=list)

        TAX_RATES = {"CA": Decimal("0.0725"), "NY": Decimal("0.04")}
        GIFT_WRAP_FEE = Decimal("4.00")

        class OrderProcessor:
            """Before SRP: finance, customer comms and operations all edit this
class."""

            def __init__(self, conn: sqlite3.Connection):
                self.conn = conn
                conn.execute("CREATE TABLE IF NOT EXISTS orders "
                             "(id TEXT PRIMARY KEY, email TEXT, total TEXT)")

            def _subtotal(self, order: Order) -> Decimal:      # shared helper
                total = Decimal("0")
                for item in order.items:
                    total += item.unit_price * item.quantity
                    if item.gift_wrap:
                        total += GIFT_WRAP_FEE
                return total

            # Actor 1 -- finance: tax and fee policy
            def calculate_total(self, order: Order) -> Decimal:
                sub = self._subtotal(order)
                tax = (sub * TAX_RATES[order.region]).quantize(Decimal("0.01"),
ROUND_HALF_UP)
                return sub + tax

            # Actor 2 -- customer communications: receipt wording and layout
            def receipt_text(self, order: Order) -> str:
                return (f"Order {order.order_id}: items {self._subtotal(order):.2f}, "
                        f"total due {self.calculate_total(order):.2f}")

            # Actor 3 -- operations: schema and storage
            def save(self, order: Order) -> None:
                self.conn.execute("INSERT INTO orders VALUES (?, ?, ?)",
                                  (order.order_id, order.customer_email,
                                   str(self.calculate_total(order))))
```

```
        if __name__ == "__main__":
            order = Order("A-1001", "ana@example.com", "CA",
                          [LineItem("MUG", Decimal("20.00"), 2, gift_wrap=True)])
            processor = OrderProcessor(sqlite3.connect(":memory:"))
            processor.save(order)
            print(processor.receipt_text(order))
            # Order A-1001: items 44.00, total due 47.19
```

What to notice:

- `_subtotal` mixes two concepts: the merchandise value and the gift-wrap fee. Both the tax computation and the receipt rely on it.
- The receipt calls `calculate_total`, so receipt wording now depends on tax policy.
- The class creates its own table. A database migration and a tax change land in the same file and the same deployment.

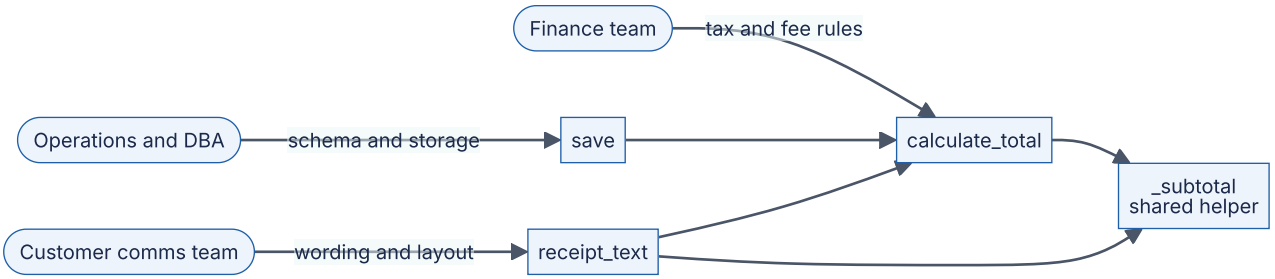
WORKED EXAMPLE

Worked example — How a finance change breaks the receipt. Finance learns that in its main state gift wrapping should not be taxed. A developer on the finance team edits `_subtotal` to return only merchandise, and adds the wrap fee after tax in `calculate_total`. For order A-1001 (two mugs at $20.00, one gift-wrapped line, California rate 7.25%):

1. Before the change: subtotal = 2 × 20.00 + 4.00 = 44.00; tax = 44.00 × 0.0725 = 3.19; total = 47.19. Receipt: “items 44.00, total due 47.19”. The $3.19 difference is tax, as the customer expects.
2. After the change: `_subtotal` = 40.00; tax = 40.00 × 0.0725 = 2.90; total = 40.00 + 2.90 + 4.00 = 46.90. Receipt: “items 40.00, total due 46.90”.
3. The customer now sees $6.90 between “items” and “total” with no explanation of the $4.00 wrap fee. Nobody on the communications team changed anything; their tests (if they exist) assert wording, not arithmetic.

Interpretation. The finance change was correct for finance, and the defect landed in another actor’s output through a shared helper. **Engineering implication.** Shared code between actors is not reuse; it is accidental coupling. Two pieces of code that look identical today but change for different reasons should be allowed to diverge.

Figure 36.3 — Actors map to methods; shared code couples them.

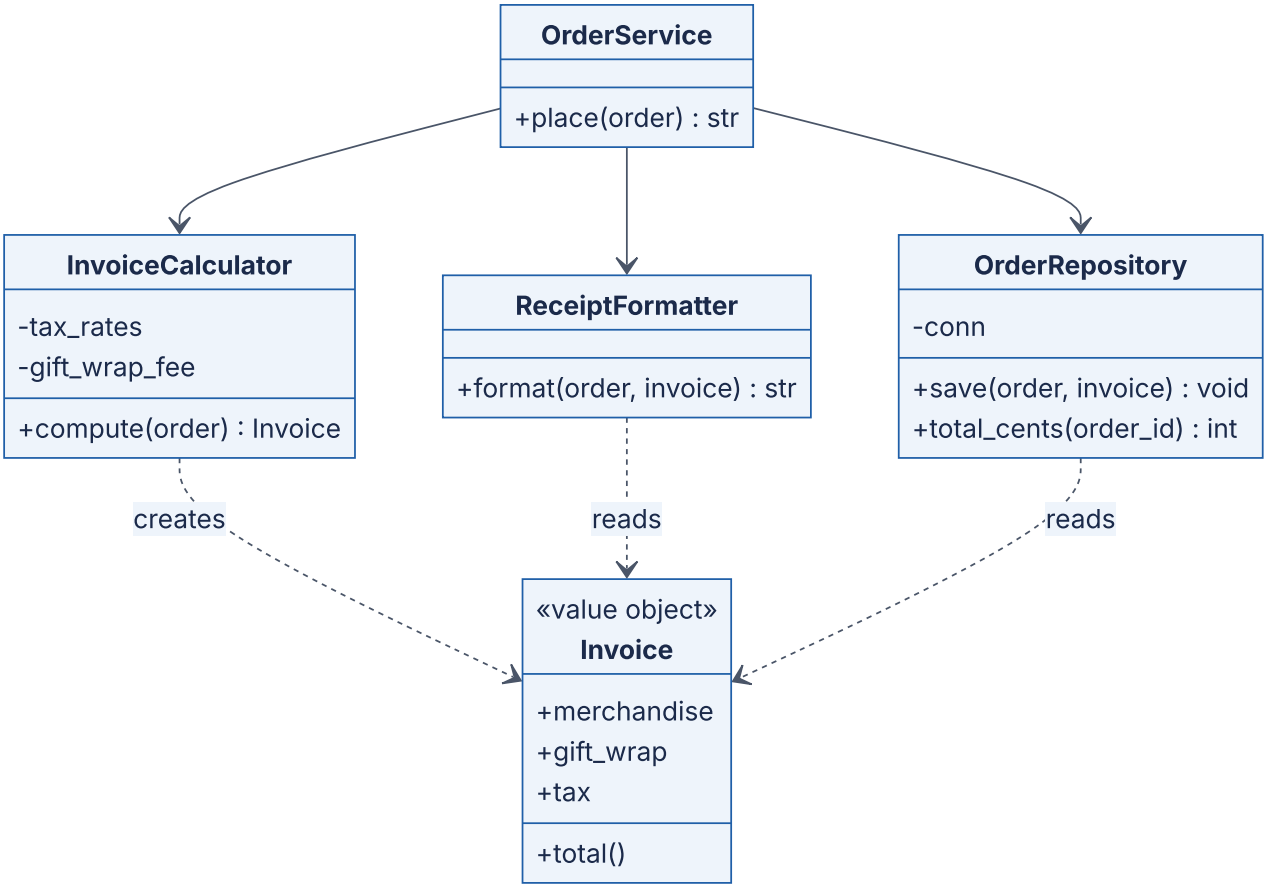


Reading the diagram. Stadium shapes are the three actors; rectangles are methods of the single class. Arrows from actors are change requests; arrows between methods are synchronous in-process calls. Follow them into `_subtotal`: a finance change reaches the receipt and the stored total. That convergence is the failure point SRP targets. Splitting into smaller methods does not help; the methods are already small, and what matters is that their *change paths* converge. No storage boundary is shown because the problem is code ownership.

36.3.3 The refactor: split by actor

The refactor creates one class per actor and a value object, `Invoice`, that carries finance's result to the others so nobody recomputes it.

Figure 36.4 — After SRP: one class per actor, coordinated by a facade.



Reading the diagram. `OrderService` is a facade: it holds references (solid arrows) to three collaborators and calls them synchronously in sequence. Each collaborator answers to one actor: `InvoiceCalculator` to finance, `ReceiptFormatter` to communications, `OrderRepository` to operations. The immutable `Invoice` value object (dashed arrows, usage) is created only by the calculator and read by the others, so the arithmetic exists in exactly one place. The storage boundary is now confined to `OrderRepository`; a schema migration touches only that class. Failure points are isolated too: a formatting bug cannot change a stored total.

Code example

```
        from dataclasses import dataclass
        from decimal import Decimal, ROUND_HALF_UP
        import sqlite3

        CENT = Decimal("0.01")

        @dataclass(frozen=True)
        class LineItem:
            sku: str
            unit_price: Decimal
            quantity: int
            gift_wrap: bool = False

        @dataclass(frozen=True)
        class Order:
            order_id: str
            customer_email: str
            region: str
            items: tuple[LineItem, ...]

        @dataclass(frozen=True)
        class Invoice:                        # value object passed between the parts
            merchandise: Decimal
            gift_wrap: Decimal
            tax: Decimal

            @property
            def total(self) -> Decimal:
                return self.merchandise + self.gift_wrap + self.tax

        class InvoiceCalculator:
            """Answers to finance. Changes only when tax or fee policy changes."""

            def __init__(self, tax_rates: dict[str, Decimal], gift_wrap_fee: Decimal,
                         tax_gift_wrap: bool):
                self.tax_rates = tax_rates
                self.gift_wrap_fee = gift_wrap_fee
                self.tax_gift_wrap = tax_gift_wrap

            def compute(self, order: Order) -> Invoice:
                merchandise = sum((i.unit_price * i.quantity for i in order.items),
Decimal("0"))
                wraps = self.gift_wrap_fee * sum(1 for i in order.items if i.gift_wrap)
                taxable = merchandise + (wraps if self.tax_gift_wrap else Decimal("0"))
                tax = (taxable * self.tax_rates[order.region]).quantize(CENT,
ROUND_HALF_UP)
                return Invoice(merchandise, wraps, tax)

        class ReceiptFormatter:
            """Answers to customer communications. Changes when wording or layout
changes."""

            def format(self, order: Order, invoice: Invoice) -> str:
                lines = [f"Order {order.order_id}",
```

```
                        f"  Merchandise {invoice.merchandise:>8.2f}"]
                if invoice.gift_wrap:
                    lines.append(f"  Gift wrap   {invoice.gift_wrap:>8.2f}")
                lines.append(f"  Tax         {invoice.tax:>8.2f}")
                lines.append(f"  Total       {invoice.total:>8.2f}")
                return "\n".join(lines)

        class OrderRepository:
            """Answers to operations. Changes when schema or storage changes."""

            def __init__(self, conn: sqlite3.Connection):
                self.conn = conn
                conn.execute("CREATE TABLE IF NOT EXISTS orders "
                             "(id TEXT PRIMARY KEY, email TEXT, total_cents INTEGER)")

            def save(self, order: Order, invoice: Invoice) -> None:
                self.conn.execute("INSERT INTO orders VALUES (?, ?, ?)",
                                  (order.order_id, order.customer_email,
                                   int(invoice.total * 100)))

            def total_cents(self, order_id: str) -> int:
                row = self.conn.execute("SELECT total_cents FROM orders WHERE id = ?",
                                        (order_id,)).fetchone()
                return row[0]

        class OrderService:
            """Facade: sequences the three parts; owns no business rule itself."""

            def __init__(self, calculator: InvoiceCalculator, formatter:
ReceiptFormatter,
                         repository: OrderRepository):
                self.calculator, self.formatter, self.repository = calculator, format-
ter, repository

            def place(self, order: Order) -> str:
                invoice = self.calculator.compute(order)
                self.repository.save(order, invoice)
                return self.formatter.format(order, invoice)

        if __name__ == "__main__":
            order = Order("A-1001", "ana@example.com", "CA",
                          (LineItem("MUG", Decimal("20.00"), 2, gift_wrap=True),))
            rates = {"CA": Decimal("0.0725"), "NY": Decimal("0.04")}
            for tax_wrap in (True, False):           # finance flips one policy flag
                repo = OrderRepository(sqlite3.connect(":memory:"))
                service = OrderService(InvoiceCalculator(rates, Decimal("4.00"),
tax_wrap),
                                       ReceiptFormatter(), repo)
                print(f"tax_gift_wrap={tax_wrap}")
                print(service.place(order))
                print("  stored cents:", repo.total_cents("A-1001"))
        # Output (abridged): Tax 3.19, Total 47.19, stored 4719; then Tax 2.90, Total
46.90, stored 4690
```

What to notice:

- Finance’s policy change is now one constructor argument in one class. The receipt shows a separate gift-wrap line, so it reconciles in both cases (40.00 + 4.00 + 2.90 = 46.90).
- `Invoice` is frozen. Communications and operations read finance’s numbers; they cannot alter them.
- The repository stores integer cents, an operations decision invisible to the other two classes.
- `OrderService` contains no `if`, no arithmetic and no SQL. A facade that starts accumulating rules is a sign that a fourth actor has appeared.

The same design in Java (compiled and run with `javac` / `java` 21):

```
        // File: SrpAfter.java  (Java 17+)
        import java.math.BigDecimal;
        import java.math.RoundingMode;
        import java.util.HashMap;
        import java.util.List;
        import java.util.Map;

        public class SrpAfter {
            record LineItem(String sku, BigDecimal unitPrice, int quantity, boolean
giftWrap) {}
            record Order(String orderId, String customerEmail, String region,
List<LineItem> items) {}
            record Invoice(BigDecimal merchandise, BigDecimal giftWrap, BigDecimal tax)
{
                BigDecimal total() { return merchandise.add(giftWrap).add(tax); }
            }

            /** Answers to finance. */
            static final class InvoiceCalculator {
                private final Map<String, BigDecimal> taxRates;
                private final BigDecimal giftWrapFee;
                private final boolean taxGiftWrap;
                InvoiceCalculator(Map<String, BigDecimal> taxRates, BigDecimal
giftWrapFee, boolean taxGiftWrap) {
                    this.taxRates = taxRates; this.giftWrapFee = giftWrapFee;
this.taxGiftWrap = taxGiftWrap;
                }
                Invoice compute(Order order) {
                    BigDecimal merchandise = BigDecimal.ZERO, wraps = BigDecimal.ZERO;
                    for (LineItem i : order.items()) {
                        merchandise =
merchandise.add(i.unitPrice().multiply(BigDecimal.valueOf(i.quantity())));
                        if (i.giftWrap()) wraps = wraps.add(giftWrapFee);
                    }
                    BigDecimal taxable = taxGiftWrap ? merchandise.add(wraps) : merchan-
dise;
                    BigDecimal tax = taxable.multiply(taxRates.get(order.region()))
                                            .setScale(2, RoundingMode.HALF_UP);
                    return new Invoice(merchandise, wraps, tax);
                }
            }

            /** Answers to customer communications. */
            static final class ReceiptFormatter {
                String format(Order order, Invoice inv) {
                    StringBuilder sb = new StringBuilder("Order " + order.orderId() +
"\n");
                    sb.append(String.format("  Merchandise %8.2f%n", inv.merchan-
dise()));
                    if (inv.giftWrap().signum() > 0) sb.append(String.format("  Gift
wrap   %8.2f%n", inv.giftWrap()));
                    sb.append(String.format("  Tax         %8.2f%n", inv.tax()));
                    sb.append(String.format("  Total       %8.2f", inv.total()));
                    return sb.toString();
                }
            }
```

```
            /** Answers to operations. An in-memory map stands in for the database. */
            static final class OrderRepository {
                private final Map<String, Long> totalCents = new HashMap<>();
                void save(Order order, Invoice inv) {
                    totalCents.put(order.orderId(), inv.total().movePointRight(2).long-
ValueExact());
                }
                long totalCents(String orderId) { return totalCents.get(orderId); }
            }

            /** Facade: sequences the parts, owns no rule. */
            static final class OrderService {
                private final InvoiceCalculator calculator;
                private final ReceiptFormatter formatter;
                private final OrderRepository repository;
                OrderService(InvoiceCalculator c, ReceiptFormatter f, OrderRepository
r) {
                    calculator = c; formatter = f; repository = r;
                }
                String place(Order order) {
                    Invoice invoice = calculator.compute(order);
                    repository.save(order, invoice);
                    return formatter.format(order, invoice);
                }
            }

            public static void main(String[] args) {
                Order order = new Order("A-1001", "ana@example.com", "CA",
                        List.of(new LineItem("MUG", new BigDecimal("20.00"), 2, true)));
                Map<String, BigDecimal> rates = Map.of("CA", new BigDecimal("0.0725"),
"NY", new BigDecimal("0.04"));
                for (boolean taxWrap : new boolean[] {true, false}) {
                    OrderRepository repo = new OrderRepository();
                    OrderService service = new OrderService(
                            new InvoiceCalculator(rates, new BigDecimal("4.00"),
taxWrap), new ReceiptFormatter(), repo);
                    System.out.println("taxGiftWrap=" + taxWrap + "\n" +
service.place(order));
                    System.out.println("  stored cents: " + repo.totalCents("A-1001"));
                }
            }
        }
```

What to notice:

- Records give immutable value objects for free; `BigDecimal` with an explicit `RoundingMode` avoids binary floating-point error in money (never use `double` for currency).
- `movePointRight(2).longValueExact()` throws if the total somehow had fractions of a cent, turning a silent rounding bug into a loud failure at the storage boundary.

Real-world industry applications

INDUSTRY EXAMPLE

`java.util.Date` and its successors. Java 1.0's `Date` combined three responsibilities: representing an instant, interpreting it as calendar fields (year, month, day) and parsing strings. JDK 1.1 deprecated the calendar-field and parsing methods in favor of `Calendar` and `DateFormat`, and Java 8's `java.time` package (JSR 310) separated the concepts further into `Instant`, `LocalDate`, `ZonedDateTime` and `DateTimeFormatter`. Each type now changes for one kind of reason: time-zone rules, calendar arithmetic or text formats. The cost was a long migration and two generations of deprecated API. *Evidence: Primary* — [Oracle Java SE API: java.util.Date; java.time]

Advantages and disadvantages

The benefits are fewer merge conflicts between teams, defects that stay inside one actor's code, smaller test surfaces and the freedom to deploy one part without the others. The costs are more classes, more names to learn and a coordination layer (the facade) that newcomers must find. Guessing the actors wrong creates seams in the wrong places.

Table 36.2 — SRP failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
One team's change breaks another's output	Shared helper across actors	Cross-team bug reports; tests in an unrelated area fail	Duplicate the helper per actor, then let copies diverge	Map actors before extracting "common" code
Constant merge conflicts on one file	Several teams edit one class	Version-control history: many authors, unrelated commits	Split by actor behind a facade	Review ownership in design reviews
Over-split code: one change touches six classes	Split by technical step, not by actor	Every feature ticket edits the same cluster of files	Merge classes that always change together	Split only along observed reasons to change

Common misconceptions

COMMON MISTAKE

"SRP means a class should do only one thing." That rule applies to functions. A class may do many related things. SRP asks *who* requests changes, and a 300-line class that only finance ever changes satisfies it better than five 60-line classes that every ticket touches together.

COMMON MISTAKE

“Duplicated code always violates DRY, so merge it.” Two blocks that look the same but answer to different actors are *accidental* duplicates. Merging them creates the coupling in the worked example above. Merge only when the code represents the same knowledge and will change for the same reason.

INTERVIEW TIP

SRP in LLD interviews. When you sketch classes for a parking lot or a billing system, name the actor behind each class out loud (“pricing changes when the business changes rates; the ticket printer changes when the hardware changes”). Interviewers listen for *reasons to change*, not class counts. If asked to “add a feature”, show that it lands in one class.

36.4 Open/Closed Principle

36.4.1 Definition: Meyer’s version and Martin’s version

DEFINITION

Definition — Open/Closed Principle (OCP). Software entities (classes, modules, functions) should be **open for extension** but **closed for modification**: you should be able to add new behavior by adding new code, without editing code that already works and is already depended upon [Martin 2002]. The principle was first stated by Bertrand Meyer [Meyer 1988].

The two halves sound contradictory until you see what each refers to. “Closed” refers to the existing source: its text, its tests, its users. “Open” refers to the set of behaviors the system can exhibit. The trick that reconciles them is an **abstraction** that stays fixed while the number of implementations behind it grows.

Meyer and Martin used different tricks, and interviewers sometimes ask about the difference.

Table 36.3 — Meyer’s and Martin’s open-closed principles.

Aspect	Meyer (1988, 1997)	Martin (1996, 2002)
Problem	A published library module must stay stable for its users, yet keep evolving	Client code must not be edited every time a new variant appears
“Closed” means	The module has a stable, well-defined interface and is available for use by other modules	The client’s source is not edited when variants are added
“Open” means	The module can still be extended, for example with new features or fields	New behavior can be added by writing new classes
Mechanism	Implementation inheritance: a subclass reuses and extends the frozen original	Abstract interfaces and polymorphism: clients depend on an interface; variants implement it
Typical risk	Deep, fragile inheritance trees	Speculative interfaces nobody extends

Meyer’s setting was library engineering: once a class is compiled, documented and used by others, you do not edit it; you inherit from it and add to the subclass [Meyer 1997]. Martin kept the goal but moved the mechanism to abstraction: the client depends on an interface, and new implementations plug in behind it. Martin’s version dominates, partly because implementation inheritance invites the substitution problems of §36.5. Larman later restated the same idea as **protected variations**: identify points of predicted variation and wrap a stable interface around them [Larman 2001].

First-principles explanation

A wall socket is open/closed. The socket (the interface) was fixed decades ago; lamps, chargers and kettles that did not exist when it was installed plug into it without anyone rewiring the house. The house is closed for modification and open for extension *along one axis*: things that use the standard plug. It is not open along other axes. A device needing three-phase power still requires an electrician. Every OCP design is like this: closed against the kinds of change you anticipated, and open only along those axes.

Why does this exist?

Editing working code is risky in proportion to how many things depend on it. Each edit can break existing behavior, forces re-review and retesting, and, in compiled languages, recompiles and redeploys every dependent module. When the same `if/elif` chain over a “type” field appears in five places, adding one new type means five coordinated edits; missing one is a production bug. OCP replaces “find every place that switches on type” with “add one class”.

How it works internally

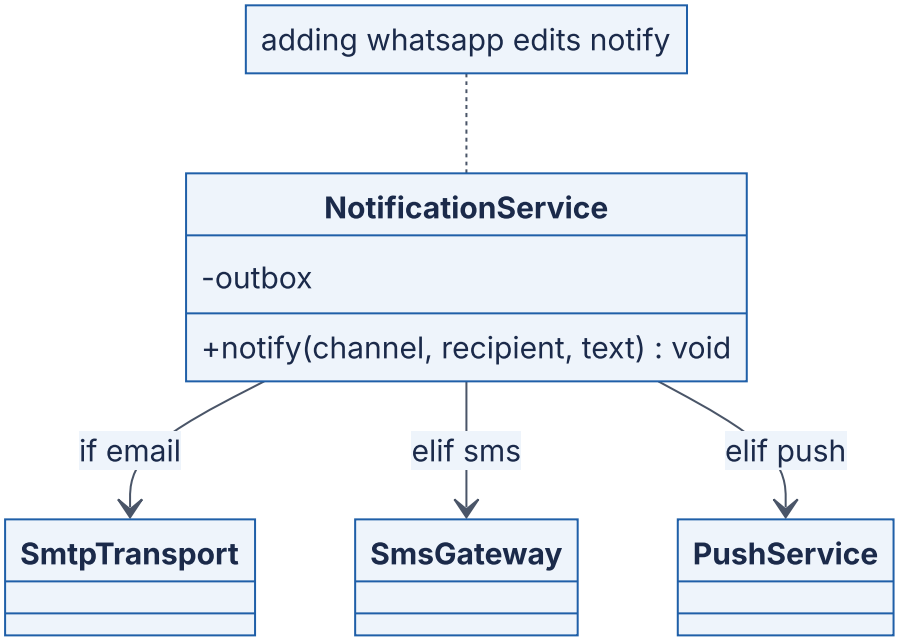
1. **Find the axis of variation**: the thing that keeps changing (channels, tax regimes, discount rules, file formats).
2. **Define an abstraction** that captures what clients need from every variant, in the client's vocabulary.
3. **Move each variant** into its own class implementing the abstraction (the Strategy pattern, §37.7).
4. **Make clients depend only on the abstraction**, and select variants in one place: a registry, a configuration file or a composition root.
5. **Add new variants as new classes**; existing code is untouched.

Martin calls the choice of axis **strategic closure**: no design is closed against every change, so you choose the changes you expect and close against those [Martin 2002].

36.4.2 The violation: a switch on channel

After the SRP refactor, the shop adds a notification step. The first version switches on a channel string.

Figure 36.5 — Before OCP: one method knows every channel.



Reading the diagram. A single class holds direct, synchronous references to three transports and chooses among them with a conditional; the edge labels show the branch that selects each one. Every transport is a failure point the service must know about, and every new transport adds an edge *and* an edit to `notify`. A change for WhatsApp re-runs and re-reviews email and SMS logic, and in a compiled language rebuilds every dependent.

```
        class NotificationService:
            """Before OCP: every new channel means editing this method."""

            def __init__(self, outbox: list):
                self.outbox = outbox                  # stands in for real transports

            def notify(self, channel: str, recipient: str, text: str) -> None:
                if channel == "email":
                    self.outbox.append(("smtp", recipient, f"Subject: Your
order\n\n{text}"))
                elif channel == "sms":
                    self.outbox.append(("sms-gateway", recipient, text[:160]))
                elif channel == "push":
                    self.outbox.append(("push", recipient, text[:100]))
                else:
                    raise ValueError(f"unknown channel: {channel}")

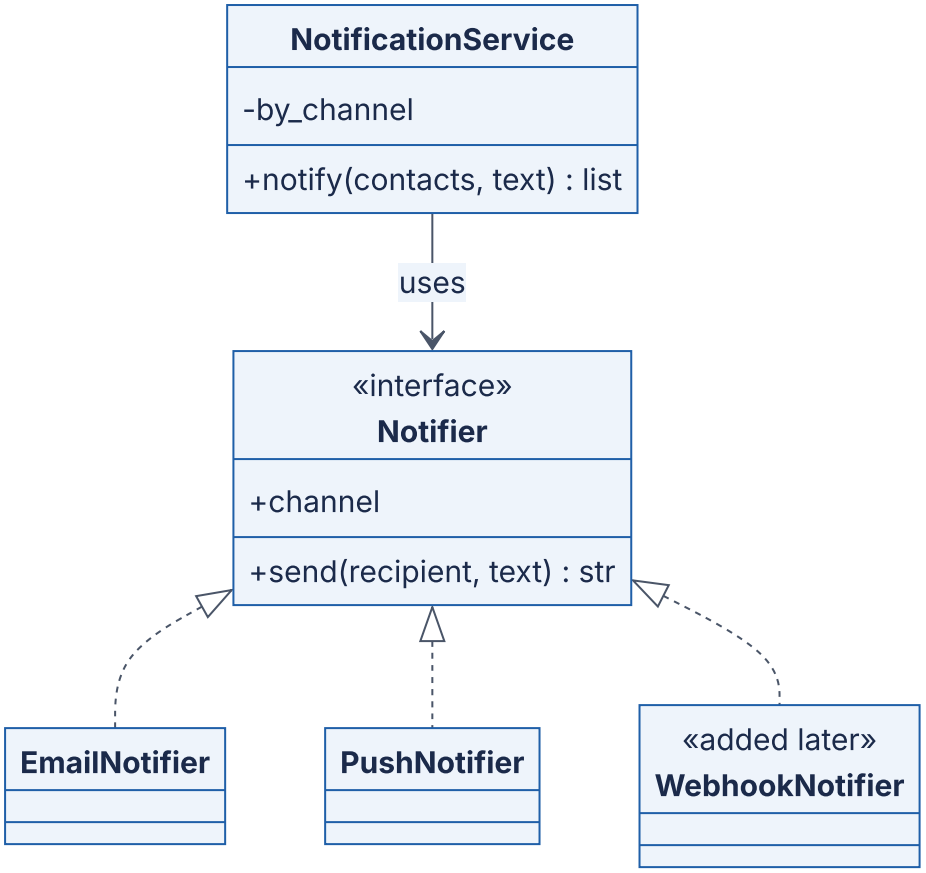
        if __name__ == "__main__":
            outbox = []
            service = NotificationService(outbox)
            service.notify("sms", "+15550100", "Order A-1001 shipped")
            try:
                service.notify("whatsapp", "+15550100", "Order A-1001 shipped")
            except ValueError as err:
                print("cannot extend without editing:", err)
```

What to notice:

- The only way to add a channel is to edit `notify`, a method every order flows through.
- `text[:160]` silently truncates SMS messages. Keep that line in mind; it returns as a Liskov violation in §36.5.

36.4.3 The refactor: depend on a Notifier abstraction

Figure 36.6 — After OCP: the service depends on an abstraction; channels plug in.



Reading the diagram. `NotificationService` holds a map from channel name to `Notifier` (solid arrow) and calls `send` synchronously through the interface. Three classes implement the interface (dashed arrows with hollow triangles, realization). `WebhookNotifier` was added months later in a new module; no arrow into or out of the service changed. Failure handling is now per implementation: a webhook timeout lives in `WebhookNotifier`. The service never imports a transport, so new channels do not rebuild or retest it. In production, `notify` would usually run asynchronously from a queue (Chapter 30), which Figure 36.1 shows as the dotted path.

Code example

```
        import itertools
        from typing import Protocol

        class Notifier(Protocol):
            """The stable abstraction. Contract: send() delivers the full text and
            returns a non-empty message id; it raises only for an invalid recipient."""
            channel: str
            def send(self, recipient: str, text: str) -> str: ...

        _ids = itertools.count(1)

        class EmailNotifier:
            channel = "email"
            def __init__(self, outbox: list):
                self.outbox = outbox
            def send(self, recipient: str, text: str) -> str:
                msg_id = f"em-{next(_ids)}"
                self.outbox.append((msg_id, recipient, f"Subject: Your
order\n\n{text}"))
                return msg_id

        class PushNotifier:
            channel = "push"
            def __init__(self, outbox: list):
                self.outbox = outbox
            def send(self, recipient: str, text: str) -> str:
                msg_id = f"pu-{next(_ids)}"
                self.outbox.append((msg_id, recipient, text))
                return msg_id

        class NotificationService:
            """Closed for modification: knows only the Notifier abstraction."""

            def __init__(self, notifiers: list[Notifier]):
                self._by_channel = {n.channel: n for n in notifiers}

            def notify(self, contacts: dict[str, str], text: str) -> list[str]:
                """contacts maps channel -> address, in the customer's preference or-
der."""
                sent = []
                for channel, address in contacts.items():
                    notifier = self._by_channel.get(channel)
                    if notifier is not None:          # unknown channels are skipped
                        sent.append(notifier.send(address, text))
                return sent

        # ---- Extension added months later, in a new module: no existing line edited --
--
        class WebhookNotifier:
            channel = "webhook"
            def __init__(self, outbox: list):
                self.outbox = outbox
            def send(self, recipient: str, text: str) -> str:
                msg_id = f"wh-{next(_ids)}"
```

```
                self.outbox.append((msg_id, recipient, {"event": "order.updated",
"text": text}))
                return msg_id

        if __name__ == "__main__":
            outbox: list = []
            service = NotificationService([EmailNotifier(outbox), PushNotifier(outbox),
                                           WebhookNotifier(outbox)])   # composition
root
            ids = service.notify({"email": "ana@example.com",
                                  "webhook": "https://erp.example.com/hooks/orders",
                                  "fax": "+15550199"}, "Order A-1001 shipped")
            print(ids)            # ['em-1', 'wh-2']  (no fax notifier registered)
```

What to notice:

- `typing.Protocol` gives **structural** typing: `WebhookNotifier` never names `Notifier`, yet a type checker accepts it because it has the right members [PEP 544].
- The docstring on `Notifier` states a **contract**, not just a signature. OCP only works if every implementation honors it, which is exactly the subject of §36.5.
- The constructor list is the single point where variants are chosen; moving it to configuration removes even that edit.

```
        // File: OcpAfter.java  (Java 17+)
        import java.util.ArrayList;
        import java.util.LinkedHashMap;
        import java.util.List;
        import java.util.Map;
        import java.util.function.Function;
        import java.util.stream.Collectors;

        public class OcpAfter {
            /** Stable abstraction: deliver the full text, return a non-empty message
id. */
            interface Notifier {
                String channel();
                String send(String recipient, String text);
            }

            static final List<String> OUTBOX = new ArrayList<>();   // stands in for
transports

            static final class EmailNotifier implements Notifier {
                public String channel() { return "email"; }
                public String send(String to, String text) {
                    OUTBOX.add("smtp to=" + to + " | Subject: Your order | " + text);
                    return "em-" + OUTBOX.size();
                }
            }

            static final class PushNotifier implements Notifier {
                public String channel() { return "push"; }
                public String send(String deviceToken, String text) {
                    OUTBOX.add("push to=" + deviceToken + " | " + text);
                    return "pu-" + OUTBOX.size();
                }
            }

            /** Closed for modification: depends only on Notifier. */
            static final class NotificationService {
                private final Map<String, Notifier> byChannel;
                NotificationService(List<Notifier> notifiers) {
                    byChannel =
notifiers.stream().collect(Collectors.toMap(Notifier::channel, Function.identity()));
                }
                List<String> notify(Map<String, String> contacts, String text) {
                    List<String> ids = new ArrayList<>();
                    contacts.forEach((channel, address) -> {
                        Notifier n = byChannel.get(channel);
                        if (n != null) ids.add(n.send(address, text));
                    });
                    return ids;
                }
            }

            // ---- Extension added later: a new class, no edits above this line ----
            static final class WebhookNotifier implements Notifier {
                public String channel() { return "webhook"; }
                public String send(String url, String text) {
```

```
                    OUTBOX.add("POST " + url + " |
{\"event\":\"order.updated\",\"text\":\"" + text + "\"}");
                    return "wh-" + OUTBOX.size();
                }
            }

            public static void main(String[] args) {
                NotificationService service = new NotificationService(
                        List.of(new EmailNotifier(), new PushNotifier(), new
WebhookNotifier()));
                Map<String, String> contacts = new LinkedHashMap<>();
                contacts.put("email", "ana@example.com");
                contacts.put("webhook", "https://erp.example.com/hooks/orders");
                contacts.put("fax", "+15550199");                    // no notifier:
skipped
                System.out.println(service.notify(contacts, "Order A-1001 shipped"));
// [em-1, wh-2]
            }
        }
```

What to notice: in Java the abstraction is nominal (`implements Notifier`), so the compiler rejects a class that forgets a method. A `LinkedHashMap` preserves the customer's preference order. In a real codebase, `java.util.ServiceLoader` can discover `Notifier` implementations on the class path, so even the composition root need not change.

Real-world industry applications

INDUSTRY EXAMPLE

Linux virtual file system. The Linux kernel's VFS layer calls file systems through tables of function pointers (`struct file_operations`, `struct inode_operations` and others). System-call code such as `read` and `write` dispatches through these tables and does not name ext4, XFS or NFS. A new file system is added by providing new operation tables and registering the file-system type, without editing the system-call layer. This is OCP in C, where an operations struct plays the role of an interface. *Evidence: Primary* — [Linux kernel docs: Overview of the Linux Virtual File System]

Advantages, disadvantages and alternatives

OCP cuts the cost of the *expected* change to "write one class and register it", keeps existing tests valid and allows separate teams to own variants. It costs indirection (a reader must find which implementation runs) and an up-front guess about the axis of change. A wrong guess is worse than no guess: the abstraction resists the change that actually arrives. Note too that polymorphism makes adding *types* cheap but adding *operations* expensive: a new method on `Notifier` must be implemented by every channel. If your system mostly gains operations over a fixed set of types, a `match` statement or the Visitor pattern can be the more open design. This tension is known as the expression problem.

Table 36.4 — OCP failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
New variant needs edits in five places	Type switches duplicated across clients	Feature diffs touch many files with the same `if` chain	Replace the switch with polymorphism or a registry	One place selects variants
Abstraction cannot express the new variant	Wrong axis of closure	The new class needs flags or downcasts	Redesign the interface with the new knowledge	Close only after two or three real variants exist
Indirection without benefit	Interface added for a variant that never came	Interfaces with exactly one implementation for years	Inline the interface	Apply YAGNI (§36.8.3)

Common misconceptions

COMMON MISTAKE

"Closed for modification means never edit the class." Bug fixes, refactors and changes along an axis you did not anticipate are edited normally. OCP is about the *anticipated* kind of change being additive.

INTERVIEW TIP

OCP in LLD interviews. When the interviewer says "now add a new vehicle type / payment method / pricing rule", the answer they hope for is "that is one new class implementing `X`; nothing else changes". Design your first class diagram so that the obvious extension follows that path, and say which axis you have chosen to keep open.

36.5 Liskov Substitution Principle

36.5.1 Definition: behavioral subtyping

DEFINITION

Definition — Liskov Substitution Principle (LSP). Subtypes must be substitutable for their base types [Martin 2002]. More precisely, S is a **behavioral subtype** of T if every property you can prove about objects of type T, using T's specification, also holds for objects of type S [Liskov and Wing 1994].

Barbara Liskov introduced the idea in her 1987 OOPSLA keynote, "Data Abstraction and Hierarchy", as a substitution property: if for each object o1 of type S there is an object o2 of type T such that, for all programs P defined in terms of T, the behavior of P is un-

changed when o1 is substituted for o2, then S is a subtype of T [Liskov 1987]. Liskov and Wing's 1994 paper turned this into checkable rules for preconditions, postconditions, invariants and the history of an object [Liskov and Wing 1994]. Martin's one-line version is the one engineers quote.

First-principles explanation

A car-rental company promises "a mid-size car with an automatic gearbox". You are handed a different model from the one in the brochure, and that is fine: it is a substitute. It would not be fine to be handed a car that only starts if you phone the depot first (a stronger *precondition*), or one whose fuel gauge reads full when the tank is half empty (a weaker *postcondition*). Substitution is about the *promise*, not about what the thing looks like.

The compiler checks only the shape: method names and parameter types. LSP is about the behavior those signatures promise. A subclass that compiles can still break every caller.

Why does this exist?

Polymorphism is the engine of OCP: clients call `notifier.send(...)` without knowing which class runs. That only works if every implementation keeps the promises the client relies on. If one subtype behaves differently, the client must find out which type it holds (`isinstance` checks, `instanceof` casts, special cases) and OCP collapses, because each new subtype now needs a client edit. LSP is the condition that keeps polymorphism safe.

How it works internally: the Liskov–Wing rules

FORMULA

Behavioral subtyping rules for a method m of base type T overridden in subtype S

$$\text{Pre}_T(m) \Rightarrow \text{Pre}_S(m) \qquad \text{Post}_S(m) \Rightarrow \text{Post}_T(m) \qquad \text{Inv}_S \Rightarrow \text{Inv}_T$$

Symbol	Meaning	Units
$\text{Pre}_T(m)$	Precondition: what the caller must guarantee before calling m on a T	predicate (true/false)
$\text{Pre}_S(m)$	Precondition of the overriding method in S	predicate
$\text{Post}_T(m)$	Postcondition: what m guarantees on return, per T's specification	predicate
$\text{Post}_S(m)$	Postcondition of the overriding method in S	predicate
Inv_T, Inv_S	Invariants: properties true of every T (or S) object between calls	predicate
$\Rightarrow$	Logical implication: whenever the left side is true, the right side must be true	—

Read the rules as: a subtype may **accept more** (weaker precondition) and **promise more** (stronger postcondition), never accept less or promise less; it must keep the base type's invariants; and, the **history constraint**, it must not allow state changes that the base type's methods could not produce. A further rule follows from the postcondition rule: a subtype must not throw exceptions that the base contract does not allow, because an unexpected exception is a broken promise.

WORKED EXAMPLE

Worked example — Checking two SMS notifiers against the Notifier contract. The `Notifier` contract says: precondition, the text has 1 to 1,600 characters; postcondition, the full text is delivered and a non-empty id is returned. A single SMS holds at most 160 GSM-7 characters; a message split into concatenated parts holds 153 characters per part, because each part carries a small header that reassembles the message on the handset.

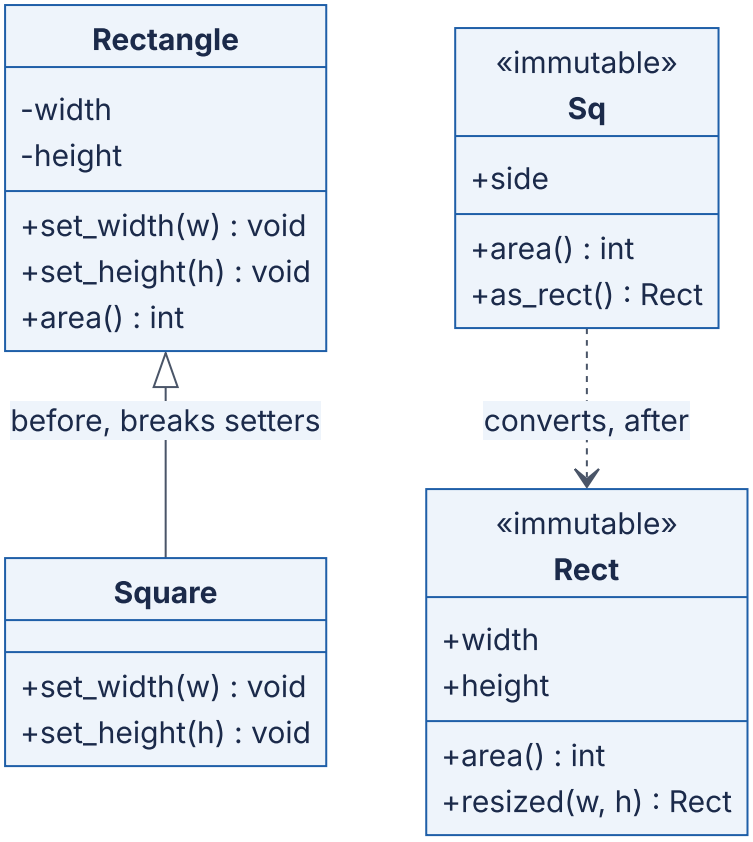
1. `StrictSmsNotifier` rejects text longer than 160. Take a 320-character receipt. Pre_T: $1 \le 320 \le 1{,}600$ is true. Pre_S: $320 \le 160$ is false. True ⇒ false is false, so the precondition rule fails: the subtype **strengthened** the precondition.
2. `TruncatingSmsNotifier` sends `text[:160]`. Preconditions match, but for 320 characters it delivers 160. Post_S ("first 160 characters delivered") does not imply Post_T ("all 320 delivered"), so the postcondition rule fails: the subtype **weakened** the postcondition.
3. `SegmentingSmsNotifier` splits long text into 153-character parts: $\lceil 320 / 153 \rceil = \lceil 2.09 \rceil = 3$ parts ($153 + 153 + 14 = 320$ characters). At the contract maximum, $\lceil 1{,}600 / 153 \rceil = \lceil 10.46 \rceil = 11$ parts. Both rules hold.

Interpretation. The strict notifier fails loudly at run time; the truncating one fails silently, which is worse, because customers receive half a receipt and no alert fires. **Engineering implication.** The base contract's limit of 1,600 characters is itself a design decision: a contract of 10,000 characters would force $\lceil 10{,}000 / 153 \rceil = 66$ SMS parts per message, which no business would pay for. Choose base-type preconditions that every reasonable implementation can honor.

36.5.2 The classic violation: Square extends Rectangle

Geometry says every square is a rectangle. Code says otherwise when `Rectangle` has independent setters.

Figure 36.7 — Rectangle and Square, before and after.



Reading the diagram. On the left, `Square` inherits from the mutable `Rectangle` (solid line, hollow triangle) and overrides both setters to keep its sides equal. That override breaks the base postcondition "`set_width` leaves the height unchanged", which is the failure point. On the right, the refactored types are immutable: no method mutates an object, so there is no setter whose postcondition a square could violate. `Sq` relates to `Rect` by conversion (dashed arrow), not inheritance; in Python an immutable `Square(Rectangle)` subclass is also safe, as the code shows. All calls are synchronous and in-memory; there is no storage boundary.

```
        class Rectangle:
            def __init__(self, width: int, height: int):
                self._w, self._h = width, height
            def set_width(self, w: int) -> None:
                self._w = w
            def set_height(self, h: int) -> None:
                self._h = h
            def area(self) -> int:
                return self._w * self._h

        class Square(Rectangle):
            """'A square is a rectangle' -- true in geometry, false for this con-
tract."""
            def __init__(self, side: int):
                super().__init__(side, side)
            def set_width(self, w: int) -> None:
                self._w = self._h = w             # keeps the square invariant ...
            def set_height(self, h: int) -> None:
                self._w = self._h = h             # ... by breaking Rectangle's postcon-
dition

        def resize_banner(r: Rectangle) -> int:
            """A client written against Rectangle's documented behavior."""
            r.set_width(5)
            r.set_height(4)
            assert r.area() == 20, f"expected area 20, got {r.area()}"
            return r.area()

        for shape in (Rectangle(1, 1), Square(1)):
            try:
                print(type(shape).__name__, "area", resize_banner(shape))
            except AssertionError as err:
                print(type(shape).__name__, "breaks the client:", err)
        # Rectangle area 20
        # Square breaks the client: expected area 20, got 16
```

The fix is to remove the operation that cannot be honored. With immutable values, a square really is a rectangle, because nothing can try to stretch it in place:

```
        from dataclasses import dataclass

        @dataclass(frozen=True)
        class Rectangle:
            width: int
            height: int
            def area(self) -> int:
                return self.width * self.height
            def resized(self, width: int, height: int) -> "Rectangle":
                """Postcondition: returns a Rectangle with exactly these sides."""
                return Rectangle(width, height)

        class Square(Rectangle):
            """Safe now: no operation lets a Square stop being square."""
            def __init__(self, side: int):
                super().__init__(side, side)
            @property
            def side(self) -> int:
                return self.width

        def resize_banner(r: Rectangle) -> int:
            resized = r.resized(5, 4)
            assert resized.area() == 20
            return resized.area()

        for shape in (Rectangle(1, 1), Square(1)):
            print(type(shape).__name__, "->", resize_banner(shape), "| original area",
shape.area())
        # Rectangle -> 20 | original area 1
        # Square -> 20 | original area 1
```

What to notice:

- The violation is not in `Square` alone or in `resize_banner` alone; it is in the *relationship*. LSP is a property of a subtype *with respect to a contract and its clients*.
- `resized` returns a plain `Rectangle`, so its postcondition ("a rectangle with these sides") holds for squares too.
- This is the history constraint at work: a mutable rectangle can reach states (width ≠ height) that a square must never reach, so a mutable square cannot be its subtype.

```
        // File: LspShapes.java  (Java 17+)
        public class LspShapes {
            // ----- Before: mutable Rectangle with a Square subclass -----
            static class Rectangle {
                protected int width, height;
                Rectangle(int w, int h) { width = w; height = h; }
                void setWidth(int w) { width = w; }     // postcondition: height un-
changed
                void setHeight(int h) { height = h; }   // postcondition: width un-
changed
                int area() { return width * height; }
            }
            static class Square extends Rectangle {
                Square(int side) { super(side, side); }
                @Override void setWidth(int w) { width = height = w; }
                @Override void setHeight(int h) { width = height = h; }
            }
            static int resizeBanner(Rectangle r) {
                r.setWidth(5);
                r.setHeight(4);
                return r.area();                        // the client expects 20
            }

            // ----- After: immutable value types; "resizing" returns a new Rectangle --
---
            record Rect(int width, int height) {
                int area() { return width * height; }
                Rect resized(int w, int h) { return new Rect(w, h); }
            }
            record Sq(int side) {
                int area() { return side * side; }
                Rect asRect() { return new Rect(side, side); }   // conversion, not sub-
typing
            }

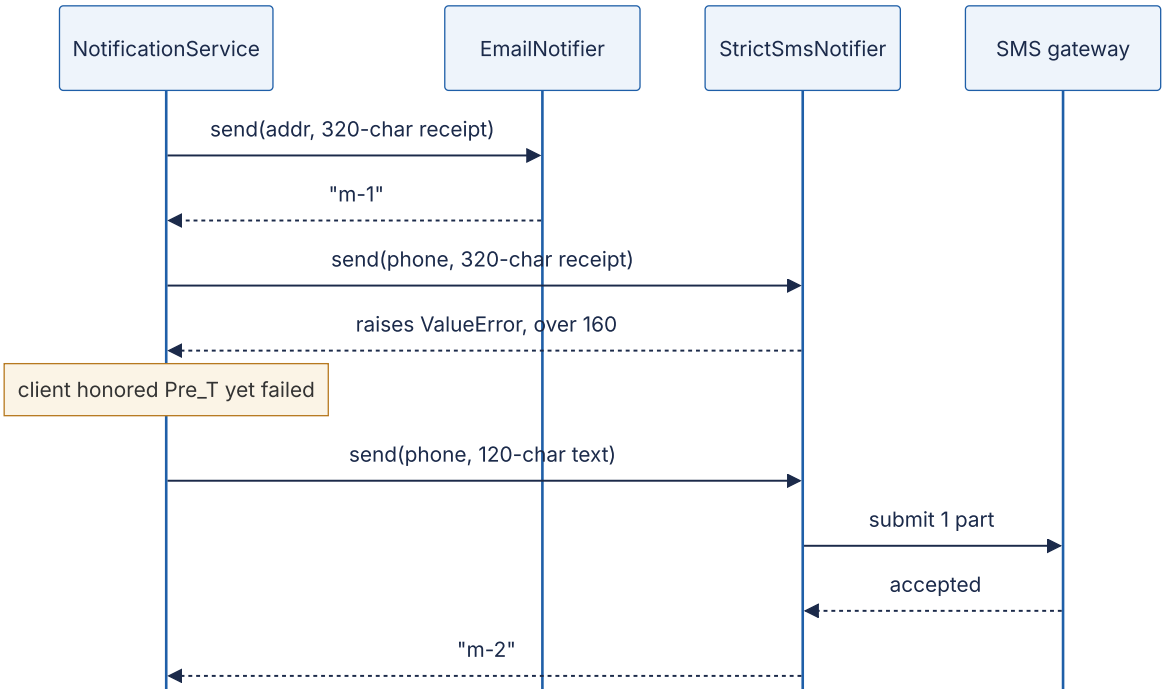
            public static void main(String[] args) {
                System.out.println("Rectangle: " + resizeBanner(new Rectangle(1, 1)));
// 20
                System.out.println("Square:    " + resizeBanner(new Square(1)) + "  <-
LSP violation");   // 16
                System.out.println("After, Rect: " + new Rect(1, 1).resized(5,
4).area());               // 20
                System.out.println("After, Sq:   " + new Sq(1).asRect().resized(5,
4).area());          // 20
            }
        }
```

What to notice: Java records are implicitly `final`, so the Java fix uses conversion (`asRect`) instead of inheritance. Both are valid; the point is that no type promises behavior it cannot keep.

36.5.3 The running example: substitutable notifiers and contract tests

The `text[:160]` line from §36.4.2 is a real LSP bug. Once channels become `Notifier` implementations, an SMS notifier that truncates or rejects long messages breaks the `NotificationService`, which was written against the contract "full text delivered".

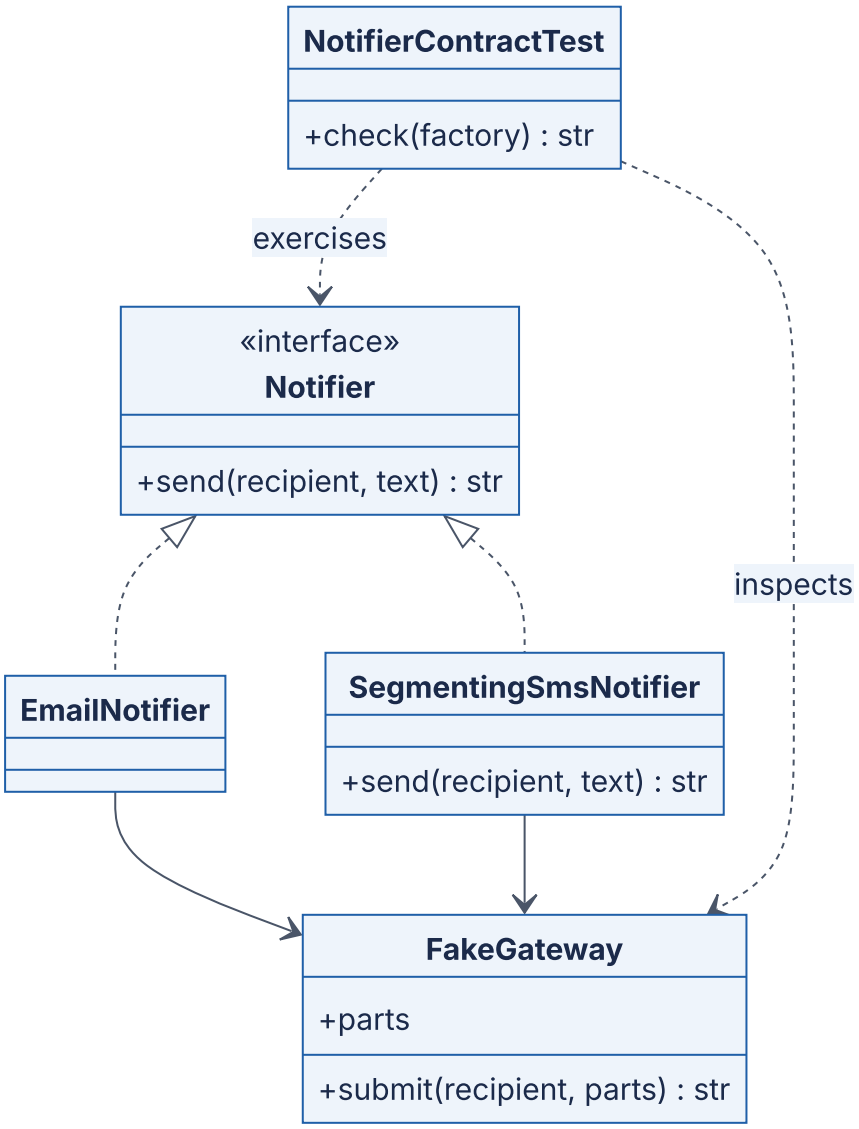
Figure 36.8 — A strengthened precondition fails at run time.



Reading the diagram. All calls are synchronous. The service sends the same valid 320-character receipt through two implementations of the same interface. The email notifier returns an id; the strict SMS notifier raises before contacting the external gateway, so the failure point is the subtype, not the network. The service did nothing wrong: it satisfied the base precondition. The only way the service could avoid this is to check the concrete type or the text length per channel, which reintroduces the switch that OCP removed. Short messages work, so such bugs pass casual testing and surface only with long receipts.

The defense is a **contract test**: one test function, written against the base type, run against every implementation. It turns LSP from a design-review opinion into a failing build.

Figure 36.9 — After LSP: every notifier passes one contract test.



Reading the diagram. Both notifiers realize `Notifier` and send through a gateway; in tests the gateway is a `FakeGateway` that records the parts submitted under each message id (the in-memory stand-in for the external SMS or SMTP provider). The contract test depends only on the interface and the fake (dashed arrows): it constructs each implementation through a factory, sends texts at the boundary lengths, and checks the postcondition by reassembling the parts. A new channel adds one line to the list under test, and the fake keeps the test fast and deterministic.

```
        import itertools
        from typing import Callable, Protocol

        MAX_TEXT = 1600          # Notifier contract: accept any text of 1..1,600 char-
acters
        SMS_SINGLE, SMS_SEGMENT = 160, 153   # GSM-7 limits: one SMS; one concatenated
part

        class FakeGateway:
            """Test double for a transport: records the parts sent under each message
id."""
            def __init__(self):
                self.parts: dict[str, list[str]] = {}
                self._ids = itertools.count(1)
            def submit(self, recipient: str, parts: list[str]) -> str:
                msg_id = f"m-{next(self._ids)}"
                self.parts[msg_id] = parts
                return msg_id

        class Notifier(Protocol):
            channel: str
            def send(self, recipient: str, text: str) -> str: ...

        class EmailNotifier:
            channel = "email"
            def __init__(self, gw: FakeGateway): self.gw = gw
            def send(self, recipient, text):
                return self.gw.submit(recipient, [text])

        class StrictSmsNotifier:                 # VIOLATION: strengthens the precondi-
tion
            channel = "sms"
            def __init__(self, gw: FakeGateway): self.gw = gw
            def send(self, recipient, text):
                if len(text) > SMS_SINGLE:
                    raise ValueError("SMS text limited to 160 characters")
                return self.gw.submit(recipient, [text])

        class TruncatingSmsNotifier:             # VIOLATION: weakens the postcondition
            channel = "sms"
            def __init__(self, gw: FakeGateway): self.gw = gw
            def send(self, recipient, text):
                return self.gw.submit(recipient, [text[:SMS_SINGLE]])

        class SegmentingSmsNotifier:             # honors the contract
            channel = "sms"
            def __init__(self, gw: FakeGateway): self.gw = gw
            def send(self, recipient, text):
                if len(text) <= SMS_SINGLE:
                    parts = [text]
                else:
                    parts = [text[i:i + SMS_SEGMENT] for i in range(0, len(text),
SMS_SEGMENT)]
                return self.gw.submit(recipient, parts)

        def check_notifier_contract(make: Callable[[FakeGateway], Notifier]) -> str:
```

```
            """One contract test, run against every implementation of Notifier."""
            gw = FakeGateway()
            notifier = make(gw)
            for length in (1, 160, 161, 320, MAX_TEXT):
                text = ("Order A-1001 update. " * 80)[:length]
                try:
                    msg_id = notifier.send("+15550100", text)
                except Exception as err:
                    return f"FAIL len={length}: raised {err!r}"
                if not msg_id:
                    return f"FAIL len={length}: empty message id"
                if "".join(gw.parts[msg_id]) != text:
                    return f"FAIL len={length}: text altered in transit"
            return "PASS"

        for cls in (EmailNotifier, StrictSmsNotifier, TruncatingSmsNotifier,
SegmentingSmsNotifier):
            print(f"{cls.__name__:<22} {check_notifier_contract(cls)}")

        gw = FakeGateway()
        mid = SegmentingSmsNotifier(gw).send("+15550100", "x" * 320)
        print("320 chars ->", len(gw.parts[mid]), "segments")
        # EmailNotifier          PASS
        # StrictSmsNotifier      FAIL len=161: raised ValueError('SMS text limited to
160 characters')
        # TruncatingSmsNotifier  FAIL len=161: text altered in transit
        # SegmentingSmsNotifier  PASS
        # 320 chars -> 3 segments
```

What to notice:

- The test probes the **boundaries** of the precondition (1, 160, 161 and 1,600 characters), where subtypes usually diverge.
- The postcondition is checked through the fake gateway, not by trusting the return value.
- In practice, write it as a parametrized pytest or an abstract JUnit class extended per implementation.

```
        // File: LspNotifiers.java  (Java 17+)
        import java.util.ArrayList;
        import java.util.HashMap;
        import java.util.List;
        import java.util.Map;
        import java.util.function.Function;

        public class LspNotifiers {
            static final int MAX_TEXT = 1600, SMS_SINGLE = 160, SMS_SEGMENT = 153;

            /** Test double for a transport: records the parts sent per message id. */
            static final class FakeGateway {
                final Map<String, List<String>> parts = new HashMap<>();
                String submit(String recipient, List<String> p) {
                    String id = "m-" + (parts.size() + 1);
                    parts.put(id, p);
                    return id;
                }
            }

            /** Contract: for 1 <= text.length() <= 1600, delivers the full text and
             *  returns a non-empty id. Throws only for an invalid recipient. */
            interface Notifier { String send(String recipient, String text); }

            record StrictSms(FakeGateway gw) implements Notifier {       // strengthens
precondition
                public String send(String to, String text) {
                    if (text.length() > SMS_SINGLE) throw new
IllegalArgumentException("SMS limited to 160 chars");
                    return gw.submit(to, List.of(text));
                }
            }
            record TruncatingSms(FakeGateway gw) implements Notifier {   // weakens
postcondition
                public String send(String to, String text) {
                    return gw.submit(to, List.of(text.substring(0,
Math.min(text.length(), SMS_SINGLE))));
                }
            }
            record SegmentingSms(FakeGateway gw) implements Notifier {   // honors the
contract
                public String send(String to, String text) {
                    List<String> p = new ArrayList<>();
                    if (text.length() <= SMS_SINGLE) p.add(text);
                    else for (int i = 0; i < text.length(); i += SMS_SEGMENT)
                        p.add(text.substring(i, Math.min(text.length(), i +
SMS_SEGMENT)));
                    return gw.submit(to, p);
                }
            }

            /** One contract test, reused for every implementation. */
            static String checkContract(Function<FakeGateway, Notifier> factory) {
                FakeGateway gw = new FakeGateway();
                Notifier n = factory.apply(gw);
                for (int len : new int[] {1, 160, 161, 320, MAX_TEXT}) {
```

```
                    String text = "x".repeat(len);
                    String id;
                    try { id = n.send("+15550100", text); }
                    catch (RuntimeException e) { return "FAIL len=" + len + ": threw "
+ e.getMessage(); }
                    if (id == null || id.isEmpty()) return "FAIL len=" + len + ": empty
id";
                    if (!String.join("", gw.parts.get(id)).equals(text)) return "FAIL
len=" + len + ": text altered";
                }
                return "PASS";
            }

            public static void main(String[] args) {
                System.out.println("StrictSms     " + checkContract(StrictSms::new));
// FAIL len=161: threw ...
                System.out.println("TruncatingSms " +
checkContract(TruncatingSms::new));  // FAIL len=161: text altered
                System.out.println("SegmentingSms " +
checkContract(SegmentingSms::new));  // PASS
            }
        }
```

Real-world industry applications

INDUSTRY EXAMPLE

`java.sql.Timestamp` extends `java.util.Date`. `Timestamp` adds nanosecond precision to `Date`. Its own documentation warns that `Timestamp.equals(Object)` is not symmetric with `Date.equals(Object)`, that `hashCode` uses only the underlying `Date` part, and that code should not treat `Timestamp` values generically as `Date` instances, because the inheritance "denotes implementation inheritance, and not type inheritance". A `HashSet<Date>` holding a mix of both can therefore contain values that compare unequal one way and equal the other. This is an LSP violation that the platform documents rather than fixes, for compatibility. *Evidence: Primary* — [Oracle Java SE API: java.sql.Timestamp]

INDUSTRY EXAMPLE

`Stack` extends `Vector`; `Properties` extends `Hashtable`. Bloch cites both as cases where inheritance exposed operations that break the subclass's intended abstraction: a `Stack` lets callers insert into the middle, and a `Properties` object, meant to hold string keys and values, accepts arbitrary objects through inherited `put`, after which some of its own methods no longer work as documented [Bloch 2018]. *Evidence: Primary* — [Bloch 2018], Item 18

Advantages and disadvantages

LSP lets clients be written once against an abstraction, keeps OCP's promise of "add a class, edit nothing", and makes contract tests reusable. Its cost is discipline: base types need written contracts (docstrings, Javadoc, types), and some intuitive "is-a" hierarchies must be abandoned for composition (§35.8). Over-strict contracts can also make legitimate implementations impossible; a contract must describe what clients need, not every detail of the first implementation.

Table 36.5 — LSP failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Client crashes with a new subtype	Strengthened precondition or new exception type	Exceptions only for certain implementations	Make the subtype accept the base input, or change the hierarchy	Contract tests at precondition boundaries
Silent data loss (truncation, rounding)	Weakened postcondition	Reconciliation or customer reports; no error logs	Restore the full postcondition	Contract tests that verify outputs, not just "no exception"
`isinstance` / `instanceof` checks spread in clients	Subtypes differ in behavior	Code search for type checks on an abstraction	Split the abstraction or fix the subtype	Treat type checks on an interface as a review flag
Mutable subtype of an immutable type	History constraint violated	Values change under a cache or a map key	Make the subtype immutable or stop subclassing	Do not subclass value types

Common misconceptions

COMMON MISTAKE

"If it type-checks, it is substitutable." Type checkers verify signatures. LSP is about behavior: preconditions, postconditions, invariants and exceptions. `Square` type-checks perfectly.

COMMON MISTAKE

"Throwing `NotImplementedError` or `UnsupportedOperationException` is a fine way to say a subtype cannot do something." It is fine only if the base contract allows it (Java's collections document some operations as optional, §36.6.4). Otherwise it is a strengthened precondition, and usually a sign that the interface is too fat, which is the next principle.

INTERVIEW TIP

LSP in LLD interviews. Expect "is a square a rectangle?" or "should `ElectricCar` extend `Car` if `refuel()` exists?". Answer with contracts: state the base method's pre-condition and postcondition, show the client that breaks, then offer the fix (immutability, a narrower base type, or composition). Mention contract tests to show you know how teams enforce it.

36.6 Interface Segregation Principle

36.6.1 Definition: clients own the shape of their interfaces

DEFINITION

Definition — Interface Segregation Principle (ISP). Clients should not be forced to depend on methods that they do not use [Martin 2002]. A **fat interface** serves many kinds of client; a **role interface** describes what one kind of client needs from its collaborator [Fowler 2006].

First-principles explanation

A restaurant's full menu has 120 dishes; the dessert menu has eight. A guest who only wants dessert can use either, but a dessert menu reprinted every time the kitchen changes a starter would be odd, and a printer who must typeset all 120 dishes to produce eight is wasting work. In code, depending on an interface means depending on *all* of its methods: their signatures, their parameter types, and every change to them. If a client uses one method of eight, seven-eighths of its dependency is pure risk.

Why does this exist?

Martin reports that ISP emerged from consulting work on a printer system at Xerox in which a single large `Job` class served printing, stapling and other tasks; every change to it forced long rebuilds of unrelated code [Martin 2002]. In C++ and Java, a change to an interface's signature recompiles every module that imports it. In dynamic languages the rebuild cost disappears, but two other costs remain. **Implementers** of a fat interface must provide every method, so classes that cannot honor some of them write stubs that throw, which is an LSP violation in waiting. **Readers** cannot tell from a parameter type which of the many methods a function will actually call.

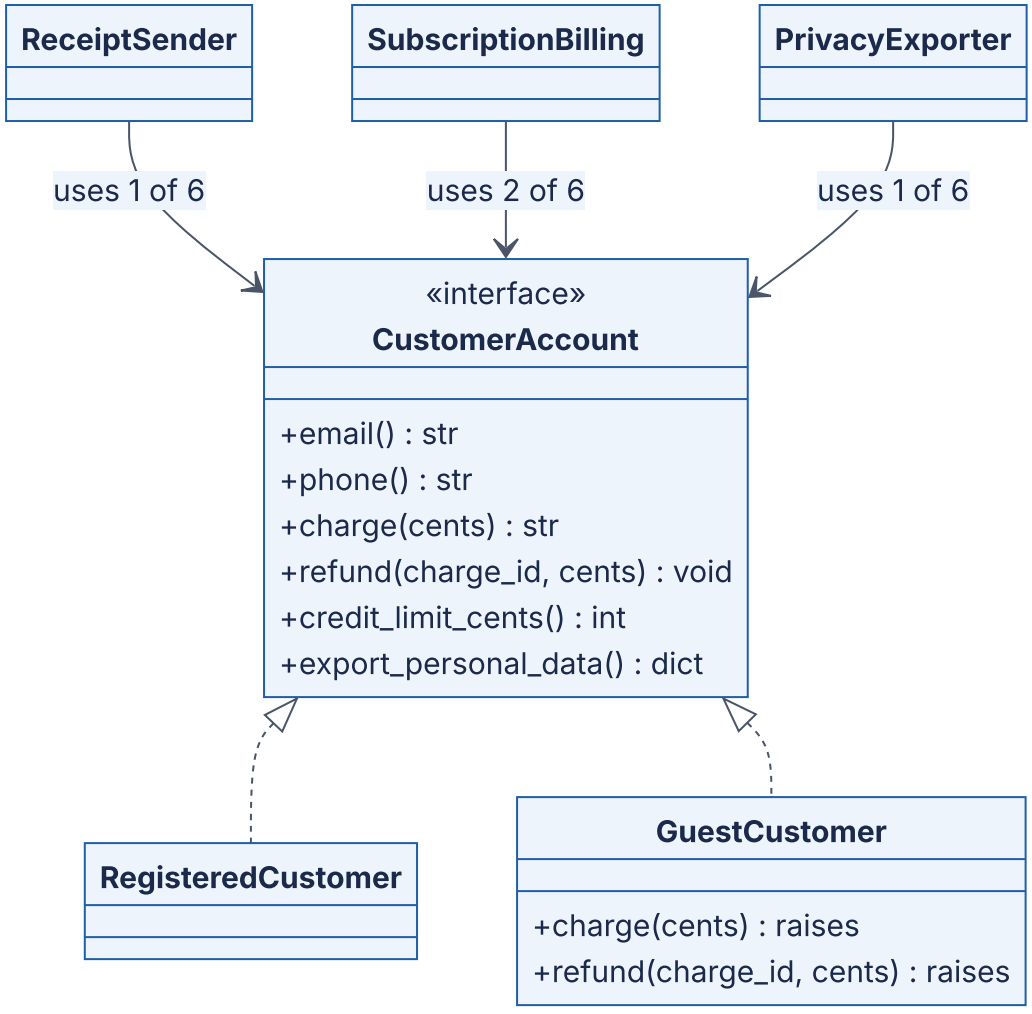
How it works internally

1. For each interface, list its **clients** and the methods each one calls.

2. Group methods by client role (notification needs contact details; billing needs charging).
3. Define one **role interface** per group, named after the role, not the implementer.
4. Let implementers implement several role interfaces; a class can still be “fat” internally.
5. Change each client’s parameter types to the narrowest role it needs.

36.6.2 The violation: a fat CustomerAccount

Figure 36.10 — Before ISP: three clients depend on one fat interface.



Reading the diagram. Three clients hold synchronous references to the same six-method interface, though each uses one or two methods (edge labels). Two classes implement it; `GuestCustomer` cannot charge (guests pay once at checkout and have no stored card), so it implements `charge` and `refund` by raising. That stub is the failure point: `SubscriptionBilling` accepts any `CustomerAccount`, so the type system lets a guest reach the code that charges. A billing signature change also recompiles `ReceiptSender` and `PrivacyExporter`, which never call those methods.

```
        from abc import ABC, abstractmethod

        class CustomerAccount(ABC):
            """Fat interface: every client depends on all six methods."""
            @abstractmethod
            def email(self) -> str: ...
            @abstractmethod
            def phone(self) -> str | None: ...
            @abstractmethod
            def charge(self, cents: int) -> str: ...
            @abstractmethod
            def refund(self, charge_id: str, cents: int) -> None: ...
            @abstractmethod
            def credit_limit_cents(self) -> int: ...
            @abstractmethod
            def export_personal_data(self) -> dict: ...

        class GuestCustomer(CustomerAccount):
            """A guest has no stored card, yet must implement billing methods."""
            def __init__(self, email: str):
                self._email = email
            def email(self): return self._email
            def phone(self): return None
            def charge(self, cents): raise NotImplementedError("guest has no stored
card")
            def refund(self, charge_id, cents): raise NotImplementedError("guest has no
stored card")
            def credit_limit_cents(self): return 0
            def export_personal_data(self): return {"email": self._email}

        def send_receipt(account: CustomerAccount, text: str) -> str:
            return f"to={account.email()} body={text}"     # uses 1 of 6 methods

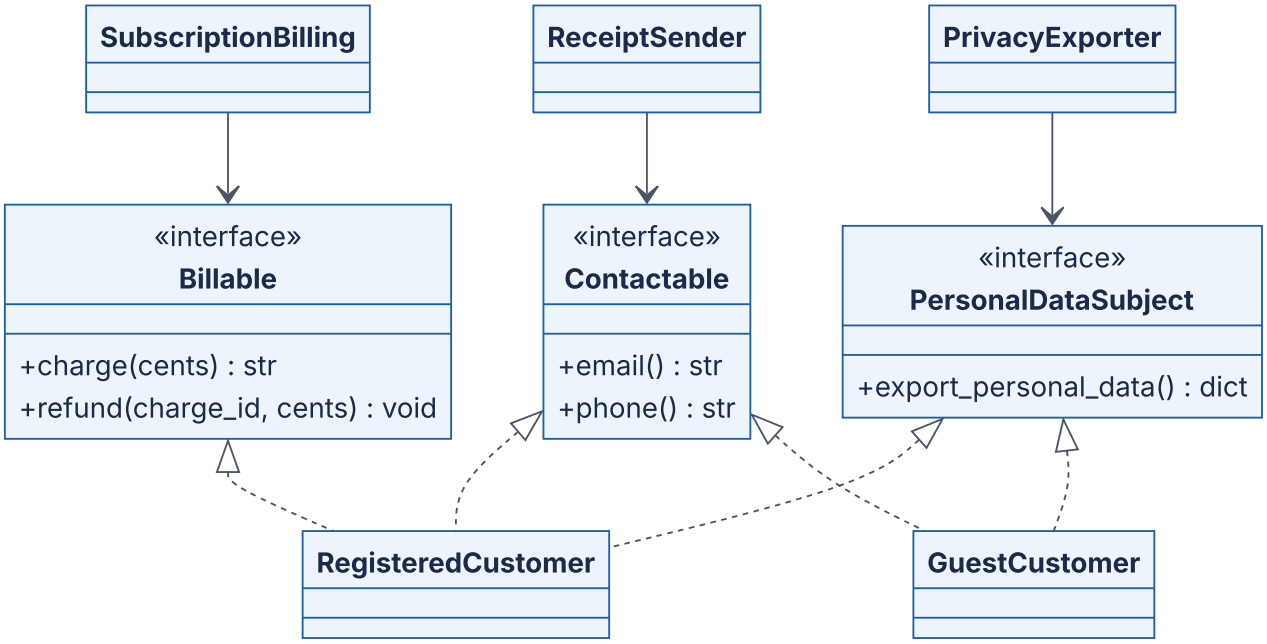
        def bill_monthly_subscriptions(accounts: list[CustomerAccount]) -> None:
            for acct in accounts:
                acct.charge(999)                          # the type checker says this
is fine

        guest = GuestCustomer("guest@example.com")
        print(send_receipt(guest, "Order A-1002 confirmed"))
        try:
            bill_monthly_subscriptions([guest])
        except NotImplementedError as err:
            print("runtime failure:", err)               # runtime failure: guest has no
stored card
```

What to notice: the error that a type checker could have caught is deferred to run time, inside a monthly batch job, because the interface claims every account can be charged.

36.6.3 The refactor: role interfaces

Figure 36.11 — After ISP: each client depends on one role.



Reading the diagram. The fat interface is replaced by three role interfaces, each named for what a client needs. `RegisteredCustomer` realizes all three; `GuestCustomer` realizes only the two it can honor, so it has no stubs. Each client's synchronous dependency now points at one small interface: a change to `Billable` cannot affect `ReceiptSender`. The failure point of Figure 36.10 disappears at compile or type-check time, because a guest is not a `Billable` and cannot be passed to `SubscriptionBilling`.

```
        from typing import Protocol, runtime_checkable

        @runtime_checkable
        class Contactable(Protocol):          # role used by notifications
            def email(self) -> str: ...
            def phone(self) -> str | None: ...

        @runtime_checkable
        class Billable(Protocol):             # role used by billing
            def charge(self, cents: int) -> str: ...
            def refund(self, charge_id: str, cents: int) -> None: ...

        @runtime_checkable
        class PersonalDataSubject(Protocol):  # role used by privacy tooling
            def export_personal_data(self) -> dict: ...

        class RegisteredCustomer:             # plays all three roles
            def __init__(self, email: str, phone: str, card_token: str):
                self._email, self._phone, self._card = email, phone, card_token
                self._charges: dict[str, int] = {}
            def email(self): return self._email
            def phone(self): return self._phone
            def charge(self, cents):
                charge_id = f"ch-{len(self._charges) + 1}"
                self._charges[charge_id] = cents
                return charge_id
            def refund(self, charge_id, cents):
                if cents > self._charges.get(charge_id, 0):
                    raise ValueError("refund exceeds captured amount")
                self._charges[charge_id] -= cents
            def export_personal_data(self):
                return {"email": self._email, "phone": self._phone, "charges":
dict(self._charges)}

        class GuestCustomer:                  # plays only the roles it can honor
            def __init__(self, email: str):
                self._email = email
            def email(self): return self._email
            def phone(self): return None
            def export_personal_data(self): return {"email": self._email}

        def send_receipt(who: Contactable, text: str) -> str:
            return f"to={who.email()} body={text}"

        def bill_monthly_subscriptions(accounts: list[Billable]) -> list[str]:
            return [acct.charge(999) for acct in accounts]

        def export_all(subjects: list[PersonalDataSubject]) -> list[dict]:
            return [s.export_personal_data() for s in subjects]

        ana = RegisteredCustomer("ana@example.com", "+15550100", "tok_visa")
        guest = GuestCustomer("guest@example.com")
        print(send_receipt(guest, "Order A-1002 confirmed"))
        print(bill_monthly_subscriptions([ana]))      # a guest here is a type-checker
error
```

```
        print(export_all([ana, guest]))
        print("guest is Billable?", isinstance(guest, Billable))   # False
```

What to notice:

- Neither customer class names a protocol; conformance is structural. A static checker such as mypy reports `bill_monthly_subscriptions([guest])` as an error.
- `@runtime_checkable` lets `isinstance` test conformance, but it checks only that the methods *exist*, not their signatures or behavior.

```java
        // File: IspAfter.java  (Java 17+)
        import java.util.ArrayList;
        import java.util.List;
        import java.util.Map;
        import java.util.Optional;

        public class IspAfter {
            interface Contactable { String email(); Optional<String> phone(); }
            interface Billable { String charge(long cents); void refund(String
chargeId, long cents); }
            interface PersonalDataSubject { Map<String, String> exportPersonalData(); }

            static final class RegisteredCustomer implements Contactable, Billable,
PersonalDataSubject {
                private final String email, phone;
                private final List<Long> charges = new ArrayList<>();
                RegisteredCustomer(String email, String phone) { this.email = email;
this.phone = phone; }
                public String email() { return email; }
                public Optional<String> phone() { return Optional.of(phone); }
                public String charge(long cents) { charges.add(cents); return "ch-" +
charges.size(); }
                public void refund(String chargeId, long cents) {
                    int idx = Integer.parseInt(chargeId.substring(3)) - 1;
                    if (cents > charges.get(idx)) throw new
IllegalArgumentException("refund exceeds capture");
                    charges.set(idx, charges.get(idx) - cents);
                }
                public Map<String, String> exportPersonalData() {
                    return Map.of("email", email, "phone", phone, "charges",
charges.toString());
                }
            }

            /** Implements only the roles it can honor: no stubs, no
UnsupportedOperationException. */
            static final class GuestCustomer implements Contactable, PersonalDataSubject
{
                private final String email;
                GuestCustomer(String email) { this.email = email; }
                public String email() { return email; }
                public Optional<String> phone() { return Optional.empty(); }
                public Map<String, String> exportPersonalData() { return
Map.of("email", email); }
            }

            // Each client names only the role it uses.
            static String sendReceipt(Contactable who, String text) { return "to=" +
who.email() + " body=" + text; }
            static List<String> billMonthly(List<? extends Billable> accounts) {
                List<String> ids = new ArrayList<>();
                for (Billable b : accounts) ids.add(b.charge(999));
                return ids;
            }

            public static void main(String[] args) {
```

```
                RegisteredCustomer ana = new RegisteredCustomer("ana@example.com",
"+15550100");
                GuestCustomer guest = new GuestCustomer("guest@example.com");
                System.out.println(sendReceipt(guest, "Order A-1002 confirmed"));
                System.out.println(billMonthly(List.of(ana)));                // [ch-1]
                // billMonthly(List.of(guest));   // does not compile: GuestCustomer is
not Billable
                List<PersonalDataSubject> subjects = List.of(ana, guest);
                subjects.forEach(s ->
System.out.println(s.exportPersonalData().get("email")));
            }
        }
```

What to notice: the commented-out line is the payoff. The defect from the “before” version is now a compile error, found in seconds rather than in a monthly batch run.

36.6.4 Industry practice, trade-offs and misconceptions

INDUSTRY EXAMPLE

Python’s `collections.abc` versus Java’s optional operations. Python defines many small abstract base classes (`Sized` , `Iterable` , `Container` , `Collection` , `Sequence` , `MutableSequence` , `Mapping`), and functions such as `len()` need only the one-method `Sized` role. Java’s collections framework went the other way for simplicity: `List` includes mutators such as `add` , and immutable or fixed-size implementations throw `UnsupportedOperationException` , which the Javadoc marks as “optional operations”. Oracle’s design FAQ explains the trade-off: separate interfaces for every combination of modifiable, fixed-size and append-only would have multiplied the number of interfaces. Java accepted a weaker static guarantee in exchange for a small API. *Evidence: Primary* — [Python docs: collections.abc]; [Oracle: Java Collections API Design FAQ]

ISP’s benefits are narrower blast radius for interface changes, no stub methods, clearer function signatures and errors moved from run time to type-check time. Its cost is more interface names, and the Java collections case shows the limit: if every client needs a slightly different subset, segregating perfectly produces a combinatorial explosion. Segregate along *roles that real clients play*, not along every possible subset.

Table 36.6 — ISP failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Implementers write throwing stubs	Interface bundles capabilities not all implementers have	Search for `NotImplementedError` / `UnsupportedOperationException` in implementations	Extract role interfaces	Design interfaces from client needs
Unrelated clients rebuild on every change	Fat interface in a compiled language	Build graph shows wide fan-out from one interface	Split by client role	Keep interfaces in the client's package (§36.7)
Dozens of one-method interfaces nobody can navigate	Segregation by every subset	Interfaces with one client and one implementer	Merge interfaces that always travel together	Segregate by real roles

COMMON MISTAKE

"ISP means every interface should have one method." It means each *client* should see only what it uses. A five-method interface used in full by every client is perfectly segregated.

INTERVIEW TIP

ISP in LLD interviews. In a vending machine or parking lot, a single `Machine` or `ParkingLot` interface with admin, payment and display methods is a red flag. Split it into roles such as `PaymentAcceptor`, `InventoryAdmin` and `Display`, and say which client uses which.

36.7 Dependency Inversion Principle

36.7.1 Definition: policy should not depend on detail

> DEFINITION
>
> **Definition — Dependency Inversion Principle (DIP).** (a) High-level modules should not depend on low-level modules; both should depend on abstractions. (b) Abstractions should not depend on details; details should depend on abstractions [Martin 2002]. **High-level** modules hold policy: the business rules that justify the system (checkout, invoicing). **Low-level** modules hold detail: mechanisms that could be swapped without changing the business (a payment vendor's SDK, SMTP, SQL).

First-principles explanation

A company writes a job description ("must hold a forklift license, work night shifts") and then hires someone who fits it. It does not rewrite its operations manual around whichever person it happened to hire. The job description is the abstraction, written in the company's terms and owned by the company; candidates conform to it. If the company instead wrote its procedures around one employee's personal habits, it could never replace that person.

In naive layered code the arrows run the other way: checkout imports the payment vendor's client, so checkout's source code names the vendor, its data shapes and its error codes. Every vendor change ripples upward into the business rules.

Why does this exist?

Source-code dependencies point the same way as control flow by default: if A calls B, A imports B. That makes the most important code (policy) depend on the most volatile code (vendors, frameworks, databases). DIP breaks that coupling by inserting an interface between them and, crucially, **placing the interface on the policy side**. Control still flows from checkout to the vendor at run time, but the source-code dependency now points from the vendor adapter *toward* checkout. That reversal is the "inversion".

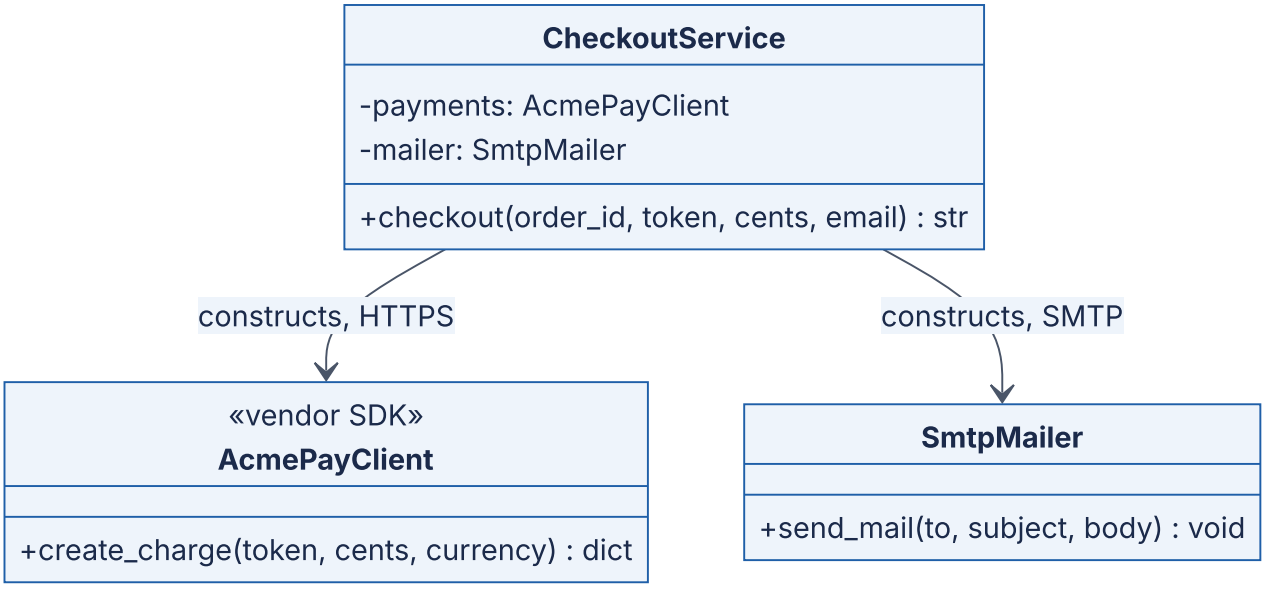
How it works internally

1. Identify policy code and the details it calls.
2. Define, **in the policy module**, an interface expressing what policy needs in its own vocabulary (`PaymentGateway.charge` returns a `ChargeResult`, not the vendor's JSON).
3. Write an **adapter** in a detail module that implements the interface by calling the vendor.

4. Give policy its collaborators from outside (dependency injection, §36.7.4), assembled in one **composition root**.
5. In tests, implement the same interfaces with **fakes**.

36.7.2 The violation: checkout constructs its details

Figure 36.12 — Before DIP: policy depends on concrete details.



Reading the diagram. The high-level `CheckoutService` creates and holds its two low-level collaborators (solid arrows): a vendor SDK that charges cards over HTTPS and an SMTP mailer. Both calls are synchronous. Both collaborators are external failure points, and the service cannot run, even in a unit test, without reaching them. The vendor's response shape leaks into the business rule, so a second provider means editing checkout. Every arrow points from policy to detail.

```
        class AcmePayClient:                        # low-level detail: a vendor SDK
            def create_charge(self, card_token: str, amount_cents: int,
                              currency: str = "usd") -> dict:
                print(f"[HTTPS api.acmepay.example] charge {amount_cents} {currency}")
                return {"id": "ch_1", "status": "succeeded"}

        class SmtpMailer:                           # low-level detail: a mail transport
            def __init__(self, host: str = "smtp.internal", port: int = 587):
                self.host, self.port = host, port
            def send_mail(self, to: str, subject: str, body: str) -> None:
                print(f"[SMTP {self.host}:{self.port}] to={to} subject={subject}")

        class CheckoutService:                      # high-level policy
            def __init__(self):
                self.payments = AcmePayClient()     # policy constructs and names details
                self.mailer = SmtpMailer()

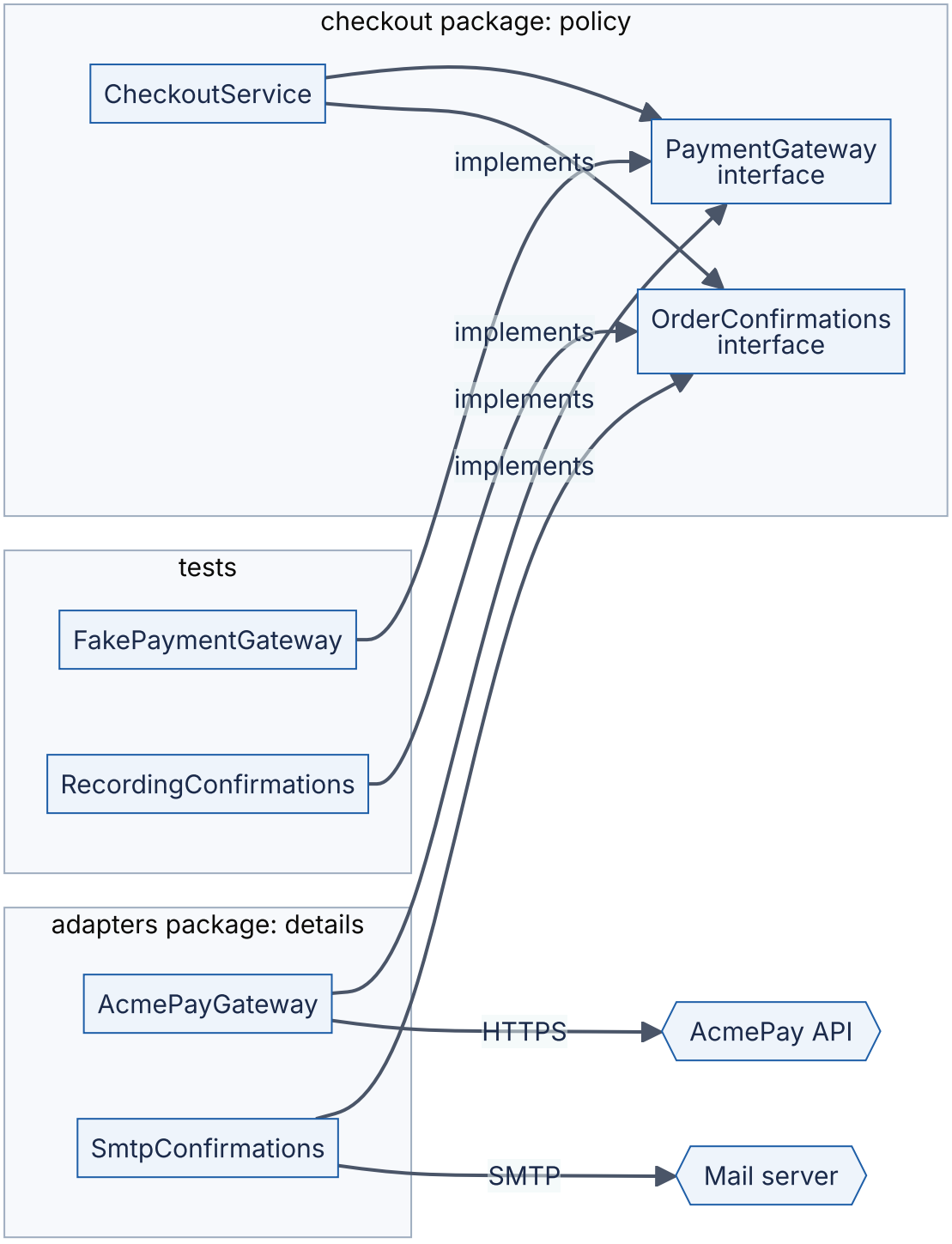
            def checkout(self, order_id: str, card_token: str, amount_cents: int,
                         email: str) -> str:
                result = self.payments.create_charge(card_token, amount_cents)
                if result["status"] != "succeeded":          # vendor vocabulary leaks
in
                    raise RuntimeError("payment declined")
                self.mailer.send_mail(email, "Order confirmed", f"Order {order_id} is
paid")
                return result["id"]

        print(CheckoutService().checkout("A-1001", "tok_visa", 4719, "ana@example.com"))
```

What to notice: to test the decline path you would need a real card that the vendor declines, or a patching library that rewrites `AcmePayClient` at run time. Both are signs that the dependency points the wrong way.

36.7.3 The refactor: ports owned by policy

Figure 36.13 — After DIP: interfaces live with policy; adapters point inward.



Reading the diagram. Arrows are source-code dependencies, which is what DIP is about. Inside the `checkout` package, the service depends only on two interfaces that the package itself defines. Both the production adapters and the test fakes depend *inward* on those interfaces; nothing in `checkout` names an adapter. Only adapters cross the system boundary to the external vendor and mail server (hexagons, the failure points), over HTTPS and SMTP. At run time, calls still flow synchronously from the service through an interface to an adapter; only the compile-time direction is inverted. A second payment

vendor is a new adapter; policy is untouched. Applied to a whole application, this arrangement is hexagonal architecture (ports and adapters, §33.10).

Code example

```
        from dataclasses import dataclass, field
        from typing import Protocol

        # ---------- checkout: high-level policy. It OWNS the ports it needs. ----------
        @dataclass(frozen=True)
        class ChargeResult:
            ok: bool
            charge_id: str = ""
            reason: str = ""

        class PaymentGateway(Protocol):            # port, named in checkout's language
            def charge(self, card_token: str, cents: int) -> ChargeResult: ...

        class OrderConfirmations(Protocol):        # port
            def confirm(self, email: str, order_id: str) -> None: ...

        class PaymentDeclined(Exception):
            pass

        class CheckoutService:
            def __init__(self, payments: PaymentGateway, confirmations:
OrderConfirmations):
                self.payments = payments           # injected through the constructor
                self.confirmations = confirmations

            def checkout(self, order_id: str, card_token: str, cents: int, email: str) -
> str:
                result = self.payments.charge(card_token, cents)
                if not result.ok:
                    raise PaymentDeclined(result.reason)
                self.confirmations.confirm(email, order_id)
                return result.charge_id

        # ---------- adapters: low-level details. They depend on checkout's ports. -----
-----
        class AcmePayClient:                       # vendor SDK (stand-in)
            def create_charge(self, card_token, amount_cents, currency="usd") -> dict:
                return {"id": "ch_1", "status": "succeeded"}

        class AcmePayGateway:                      # implements PaymentGateway
            def __init__(self, client: AcmePayClient):
                self.client = client
            def charge(self, card_token, cents):
                raw = self.client.create_charge(card_token, cents)
                if raw["status"] == "succeeded":
                    return ChargeResult(True, charge_id=raw["id"])
                return ChargeResult(False, reason=raw.get("failure_code", "unknown"))

        class SmtpConfirmations:                   # implements OrderConfirmations
            def __init__(self, host: str):
                self.host = host
            def confirm(self, email, order_id):
                print(f"[SMTP {self.host}] to={email} Order {order_id} is paid")
```

```
        def build_checkout() -> CheckoutService:   # composition root: the only place
            return CheckoutService(AcmePayGateway(AcmePayClient()),   # that names de-
tails
                                   SmtpConfirmations("smtp.internal"))

        # ---------- tests: fakes implement the same ports; no network, no mocks library
----------
        @dataclass
        class FakePaymentGateway:
            declined_tokens: set = field(default_factory=set)
            charges: list = field(default_factory=list)
            def charge(self, card_token, cents):
                if card_token in self.declined_tokens:
                    return ChargeResult(False, reason="card_declined")
                self.charges.append((card_token, cents))
                return ChargeResult(True, charge_id=f"fake-{len(self.charges)}")

        @dataclass
        class RecordingConfirmations:
            sent: list = field(default_factory=list)
            def confirm(self, email, order_id):
                self.sent.append((email, order_id))

        def test_successful_checkout_charges_once_and_confirms():
            pay, mail = FakePaymentGateway(), RecordingConfirmations()
            charge_id = CheckoutService(pay, mail).checkout("A-1", "tok_ok", 4719,
"a@x.com")
            assert charge_id == "fake-1"
            assert pay.charges == [("tok_ok", 4719)]
            assert mail.sent == [("a@x.com", "A-1")]

        def test_declined_card_sends_no_confirmation():
            pay, mail = FakePaymentGateway(declined_tokens={"tok_bad"}),
RecordingConfirmations()
            try:
                CheckoutService(pay, mail).checkout("A-2", "tok_bad", 1000, "b@x.com")
                raise AssertionError("expected PaymentDeclined")
            except PaymentDeclined as err:
                assert str(err) == "card_declined"
            assert mail.sent == []

        test_successful_checkout_charges_once_and_confirms()
        test_declined_card_sends_no_confirmation()
        print("tests passed")
        print("production wiring:", build_checkout().checkout("A-3", "tok_visa", 4719,
"ana@example.com"))
```

What to notice:

- `ChargeResult` and `PaymentDeclined` are checkout's vocabulary. The vendor's `"succeeded"` string appears only in the adapter.
- The decline test, which was hard to write before, is now four lines. A **fake** is a working, simplified implementation (here, an in-memory list); a **mock** is an object preprogrammed with expected calls [Meszaros 2007]. Fakes of owned interfaces tend

to produce tests that survive refactoring, because they check outcomes rather than call sequences.

- `build_checkout` is the only function that knows concrete classes. Everything else receives its collaborators.

```
        // File: DipAfter.java  (Java 17+)
        import java.util.ArrayList;
        import java.util.List;
        import java.util.Set;

        public class DipAfter {
            // ---------- checkout (policy): owns its ports ----------
            record ChargeResult(boolean ok, String chargeId, String reason) {}
            interface PaymentGateway { ChargeResult charge(String cardToken, long
cents); }
            interface OrderConfirmations { void confirm(String email, String orderId); }
            static final class PaymentDeclined extends RuntimeException {
                PaymentDeclined(String reason) { super(reason); }
            }

            static final class CheckoutService {
                private final PaymentGateway payments;
                private final OrderConfirmations confirmations;
                CheckoutService(PaymentGateway payments, OrderConfirmations confirma-
tions) {   // constructor injection
                    this.payments = payments; this.confirmations = confirmations;
                }
                String checkout(String orderId, String cardToken, long cents, String
email) {
                    ChargeResult r = payments.charge(cardToken, cents);
                    if (!r.ok()) throw new PaymentDeclined(r.reason());
                    confirmations.confirm(email, orderId);
                    return r.chargeId();
                }
            }

            // ---------- adapters (details): depend on checkout's ports ----------
            static final class AcmePayGateway implements PaymentGateway {
                public ChargeResult charge(String token, long cents) {
                    // A real adapter would call the vendor SDK here and translate its
response.
                    return new ChargeResult(true, "ch_1", "");
                }
            }
            static final class SmtpConfirmations implements OrderConfirmations {
                public void confirm(String email, String orderId) {
                    System.out.println("[SMTP] to=" + email + " Order " + orderId + " is
paid");
                }
            }

            // ---------- test fakes ----------
            static final class FakePaymentGateway implements PaymentGateway {
                final Set<String> declined; final List<Long> charges = new ArrayList<>
();
                FakePaymentGateway(Set<String> declined) { this.declined = declined; }
                public ChargeResult charge(String token, long cents) {
                    if (declined.contains(token)) return new ChargeResult(false, "",
"card_declined");
                    charges.add(cents);
                    return new ChargeResult(true, "fake-" + charges.size(), "");
```

```
                }
            }
            static final class RecordingConfirmations implements OrderConfirmations {
                final List<String> sent = new ArrayList<>();
                public void confirm(String email, String orderId) { sent.add(email +
":" + orderId); }
            }

            static void check(boolean cond, String msg) { if (!cond) throw new
AssertionError(msg); }

            public static void main(String[] args) {
                // Unit tests with fakes
                FakePaymentGateway pay = new FakePaymentGateway(Set.of("tok_bad"));
                RecordingConfirmations mail = new RecordingConfirmations();
                CheckoutService svc = new CheckoutService(pay, mail);
                check(svc.checkout("A-1", "tok_ok", 4719, "a@x.com").equals("fake-1"),
"charge id");
                check(mail.sent.equals(List.of("a@x.com:A-1")), "confirmation sent");
                try { svc.checkout("A-2", "tok_bad", 1000, "b@x.com"); check(false, "ex-
pected decline"); }
                catch (PaymentDeclined e) {
check(e.getMessage().equals("card_declined"), "reason"); }
                check(mail.sent.size() == 1, "no confirmation after decline");
                System.out.println("tests passed");

                // Composition root: the only place that names concrete classes
                CheckoutService prod = new CheckoutService(new AcmePayGateway(), new
SmtpConfirmations());
                System.out.println(prod.checkout("A-3", "tok_visa", 4719,
"ana@example.com"));
            }
        }
```

What to notice: in a real project the ports and `CheckoutService` sit in one package or module, adapters in another that depends on it, and fakes under `src/test`, with JUnit instead of `main`.

36.7.4 DI, DIP and IoC are three different things

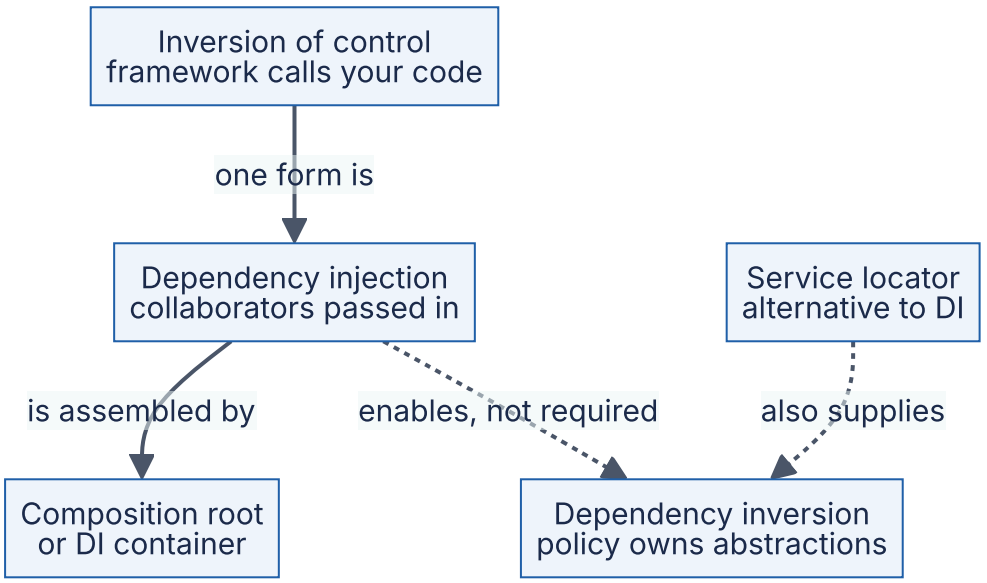
The three terms are often used interchangeably. They are related but distinct.

Table 36.7 — Dependency inversion, dependency injection and inversion of control.

Term	What it is	Question it answers	Example
Dependency Inversion Principle (DIP)	A design principle about the *direction of source-code dependencies*	Which module owns the interface, and which way do imports point?	`checkout` defines `PaymentGateway`; the adapter imports it
Dependency injection (DI)	A *technique*: an object receives its collaborators from outside instead of constructing them	Who creates the collaborators?	`CheckoutService(payments, confirmations)`
Inversion of control (IoC)	A broad *property of frameworks*: the framework calls your code, not the reverse	Who owns the main loop?	A web framework calls your handler; a test runner calls your tests

Fowler's 2004 article explains why he and others coined "dependency injection": "inversion of control" was too generic, since every framework inverts control, and the specific pattern being described was supplying dependencies to an object from outside [Fowler 2004]. He describes constructor injection, setter injection and interface injection, and compares injection with the **service locator** alternative, in which an object asks a registry for its dependencies. A service locator hides dependencies inside method bodies; constructor injection makes them visible in the signature, which is why it is the common default.

Figure 36.14 — How DIP, DI and IoC relate.



Reading the diagram. Read solid arrows as "is a kind of" or "is done by", and dotted arrows as "can support". Inversion of control is the broadest idea; dependency injection is one form of it, in which a composition root (hand-written, as in `build_checkout`, or a container such as Spring or Guice) creates objects and passes them in. DI makes DIP convenient, but you can inject a concrete class (DI without DIP), and you can satisfy DIP with a

service locator (DIP without DI). A common failure is a DI container used without DIP: everything is injected, but the injected types are concrete vendor classes.

Real-world industry applications

INDUSTRY EXAMPLE

JDBC and SLF4J. Java application code programs against `java.sql.Connection` and related interfaces defined by the platform; vendors ship drivers that implement them, and since JDBC 4.0 `DriverManager` discovers drivers through `ServiceLoader`, so the application's source never names the database vendor. SLF4J does the same for logging: libraries depend on the `org.slf4j.Logger` API, and the deployer chooses the logging backend at deployment time. In both cases the abstraction is owned by the client side (the platform or the facade), and the details depend on it. *Evidence: Primary* — [Oracle Java SE API: java.sql.DriverManager, java.util.ServiceLoader]; [SLF4J manual]

Advantages, disadvantages and failure scenarios

DIP makes policy testable in milliseconds, lets you replace vendors and databases by writing adapters, and keeps business rules readable in business terms. It costs an interface and an adapter per external dependency, plus a composition root to maintain. Interfaces that merely mirror one vendor's SDK method for method give the cost without the benefit, because a second vendor will not fit them.

Table 36.8 — DIP failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Swapping vendors requires editing business rules	Policy imports the SDK or its data types	Import graph: core package imports vendor package	Introduce a port in the core; move vendor code to an adapter	Architecture tests that forbid core-to-adapter imports
Interface changes whenever the vendor changes	Interface owned by, or copied from, the detail	Port signatures use vendor types or names	Rewrite the port in domain vocabulary	Define ports from the policy's needs
Tests pass but production fails	Fake diverges from the real adapter's behavior	Bugs found only in integration	Run the same contract tests (§36.5.3) on fake and real adapter	Contract-test every fake

COMMON MISTAKE

"Using a DI container means the code follows DIP." A container only wires objects. If `CheckoutService` is injected with an `AcmePayClient`, the dependency still points from policy to detail. DIP is about *which module owns the abstraction*.

INTERVIEW TIP

DIP in LLD interviews. When you introduce a repository or gateway interface, say who owns it ("the domain layer defines `OrderRepository`; the Postgres adapter implements it") and how you would test the domain with an in-memory fake. Distinguishing DI from DIP in one sentence is a common senior-level signal.

36.8 SOLID at the architecture level

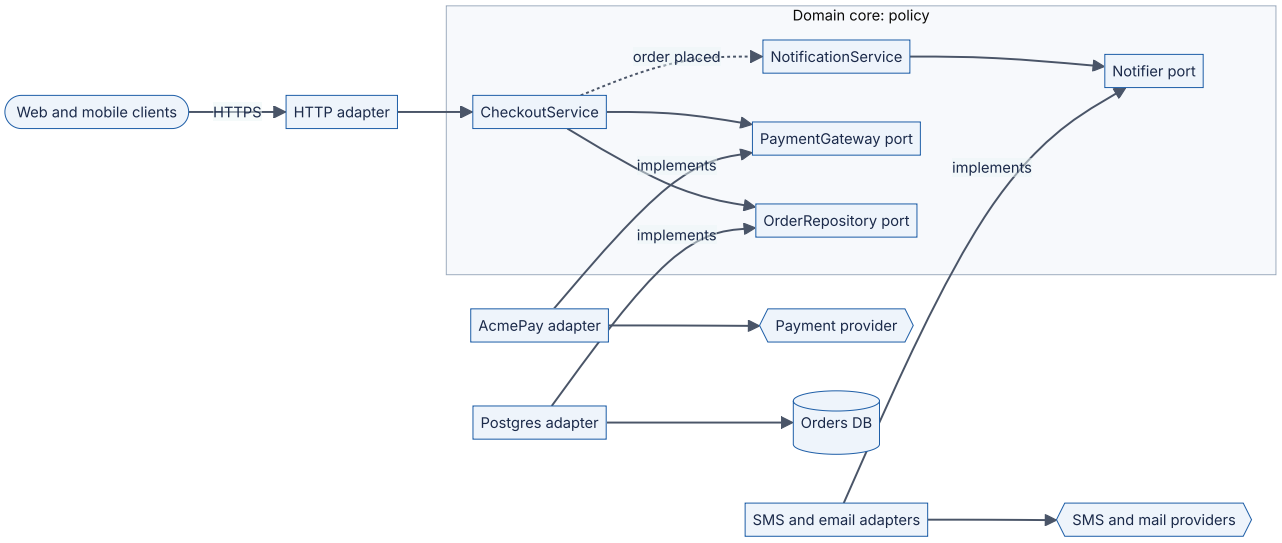
36.8.1 From classes to components and services

The same forces that act on classes act on packages, deployable components and services, only with higher costs: a change that crosses a service boundary needs coordinated releases, API versioning and sometimes a data migration. Martin restated the principles for components in his component cohesion and coupling principles [Martin 2002; Martin 2017].

Table 36.9 — Each SOLID principle at the component and service level.

Principle	Class level	Component and service level
SRP	One actor per class	**Common Closure Principle**: group into one component the classes that change for the same reasons; split services along business capabilities and teams (§34.3)
OCP	New variant = new class	**Plugin architecture**: the core exposes extension points; features ship as plugins or new services consuming events
LSP	Subtypes honor contracts	Every implementation of an API, and every new version of a service, honors the published contract; breaking changes need a new version
ISP	Role interfaces	**Common Reuse Principle**: do not force users of a component to depend on classes they do not use; per-client APIs such as the back-end-for-frontend (§34.6)
DIP	Policy owns interfaces	Hexagonal **ports and adapters** (§33.10) and the clean-architecture dependency rule (§33.11): source dependencies point toward the domain

Figure 36.15 — The order module as ports and adapters.



Reading the diagram. The domain core contains the policy classes and the ports they own; it imports nothing outside itself. Driving adapters (the HTTP adapter receiving HTTPS requests from clients) call into the core; driven adapters (payment, persistence, messaging) implement the core's ports and are the only components that talk to external systems (hexagons) and the database (cylinder, the storage boundary). Solid arrows are synchronous calls or source dependencies; the dotted arrow is the asynchronous order-placed event that triggers notification. Failure points are concentrated in adapters, where timeouts, retries and circuit breakers belong (§22.11–§22.13). Adapters scale and deploy independently.

36.8.2 Measuring dependency direction: stability and abstractness

DIP at the component level says that dependencies should point toward stable, abstract components. Martin made this measurable with three metrics [Martin 2002; Martin 2017].

FORMULA

Instability, abstractness and distance from the main sequence

$$I = \frac{C_e}{C_a + C_e} \qquad A = \frac{N_a}{N_c} \qquad D = |A + I - 1|$$

Symbol	Meaning	Units
C_a	Afferent couplings: number of other components that depend on this one (fan-in)	components
C_e	Efferent couplings: number of other components this one depends on (fan-out)	components
I	Instability: 0 = maximally stable (many dependents, no dependencies), 1 = maximally unstable	dimensionless, 0–1
N_a	Number of abstract classes and interfaces in the component	types
N_c	Total number of classes and interfaces in the component	types
A	Abstractness	dimensionless, 0–1
D	Distance from the “main sequence” line $A + I = 1$; 0 is ideal	dimensionless, 0–1

The **Stable Dependencies Principle** says to depend in the direction of stability (decreasing I). The **Stable Abstractions Principle** says a component should be as abstract as it is stable: stable components ($I \approx 0$) should be abstract ($A \approx 1$) so that they can be extended without modification, while unstable ones ($I \approx 1$) can be concrete. A component that is stable and concrete (D near 1 with low I, low A) sits in the **zone of pain**: many things depend on it, so it is hard to change, yet it is full of details that want to change.

WORKED EXAMPLE

Worked example — Metrics for the order module. The components of Figure 36.15 have these dependencies: `web-api` depends on `checkout-core` and `notification-core`; the payment and Postgres adapters depend on `checkout-core`; the SMS and SMTP adapters depend on `notification-core`; `checkout-core` depends on `notification-core`. `checkout-core` has 8 types, 3 of them interfaces; `notification-core` has 4 types, 2 of them interfaces.

1. `checkout-core`: $C_a = 3$ (web-api, payment adapter, Postgres adapter); $C_e = 1$ (notification-core). $I = 1/(3 + 1) = 0.25$. $A = 3/8 = 0.375$. $D = |0.375 + 0.25 − 1| = 0.375$.
2. `notification-core`: $C_a = 4$ (web-api, two adapters, checkout-core); $C_e = 0$. $I = 0/4 = 0$. $A = 2/4 = 0.5$. $D = |0.5 + 0 − 1| = 0.5$.
3. Each adapter: $C_a = 0$, $C_e = 1$, so $I = 1$; with no interfaces, $A = 0$ and $D = |0 + 1 − 1| = 0$.

Interpretation. All dependencies point from $I = 1$ (adapters, web-api) toward $I = 0.25$ and $I = 0$: the Stable Dependencies Principle holds. The adapters sit exactly on the main sequence. The two cores are more concrete than their stability suggests ($D =$ 0.375 and 0.5). **Engineering implication.** That is acceptable for concrete classes that rarely change, such as the `Invoice` value object; it becomes a problem if volatile logic lives in `notification-core`, because four components must be retested whenever it changes. Watch D over time rather than chasing 0.

The arithmetic is easy to automate from an import graph:

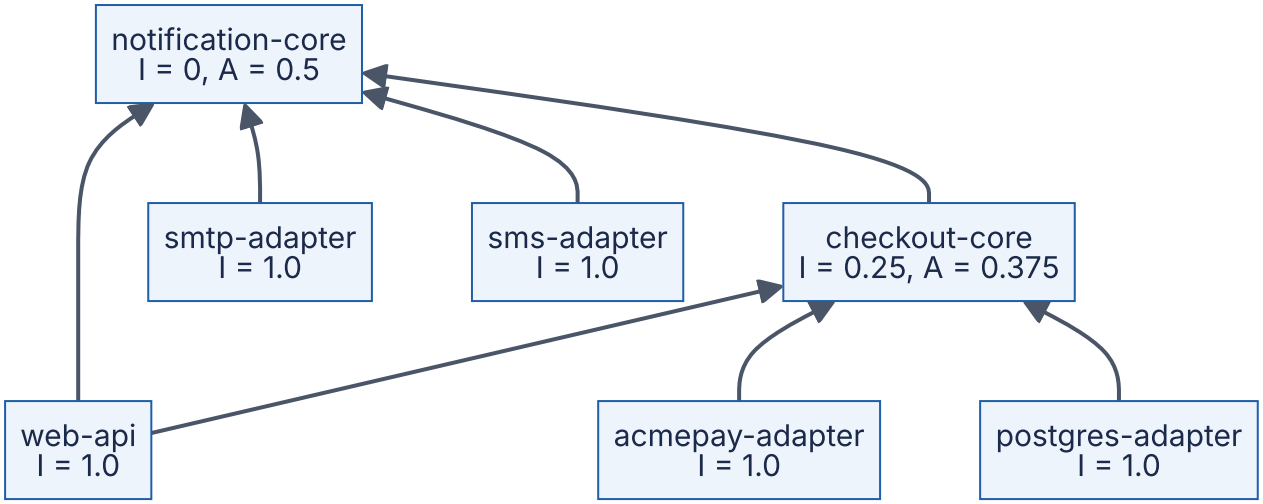
```
        # Martin's component metrics: instability I, abstractness A, distance D.
        DEPENDS_ON = {                      # component -> components it imports
            "web-api":           ["checkout-core", "notification-core"],
            "acmepay-adapter":   ["checkout-core"],
            "postgres-adapter":  ["checkout-core"],
            "smtp-adapter":      ["notification-core"],
            "sms-adapter":       ["notification-core"],
            "checkout-core":     ["notification-core"],
            "notification-core": [],
        }
        CLASSES = {                         # component -> (abstract types, all types)
            "web-api": (0, 6), "acmepay-adapter": (0, 2), "postgres-adapter": (0, 3),
            "smtp-adapter": (0, 1), "sms-adapter": (0, 2),
            "checkout-core": (3, 8), "notification-core": (2, 4),
        }

        def metrics(deps: dict, classes: dict) -> dict:
            fan_in = {c: 0 for c in deps}
            for targets in deps.values():
                for t in targets:
                    fan_in[t] += 1
            out = {}
            for c, targets in deps.items():
                ca, ce = fan_in[c], len(targets)
                i = ce / (ca + ce) if ca + ce else 0.0
                n_abstract, n_total = classes[c]
                a = n_abstract / n_total
                out[c] = (ca, ce, round(i, 3), round(a, 3), round(abs(a + i - 1), 3))
            return out

        print(f"{'component':<18} Ca Ce     I     A     D")
        for name, (ca, ce, i, a, d) in sorted(metrics(DEPENDS_ON, CLASSES).items(),
                                              key=lambda kv: kv[1][2]):
            print(f"{name:<18} {ca:>2} {ce:>2} {i:>5} {a:>5} {d:>5}")
        # notification-core   4  0   0.0   0.5   0.5
        # checkout-core       3  1  0.25 0.375 0.375
        # web-api             0  2   1.0   0.0   0.0   (adapters likewise: I=1, A=0,
D=0)
```

What to notice: an isolated component ($C_a = C_e = 0$) gets $I = 0$ here to avoid division by zero; real dependency maps come from import analysis.

Figure 36.16 — Component dependencies point toward stability.



Reading the diagram. Each node is a component (a package or deployable module) labeled with its computed metrics; arrows are compile-time dependencies, drawn upward toward stability. Every arrow goes from a higher *I* to a lower *I*, which is the Stable Dependencies Principle in picture form. A dependency arrow pointing downward, for example `notification-core` importing `sms-adapter`, would be a DIP violation and would raise the core's instability.

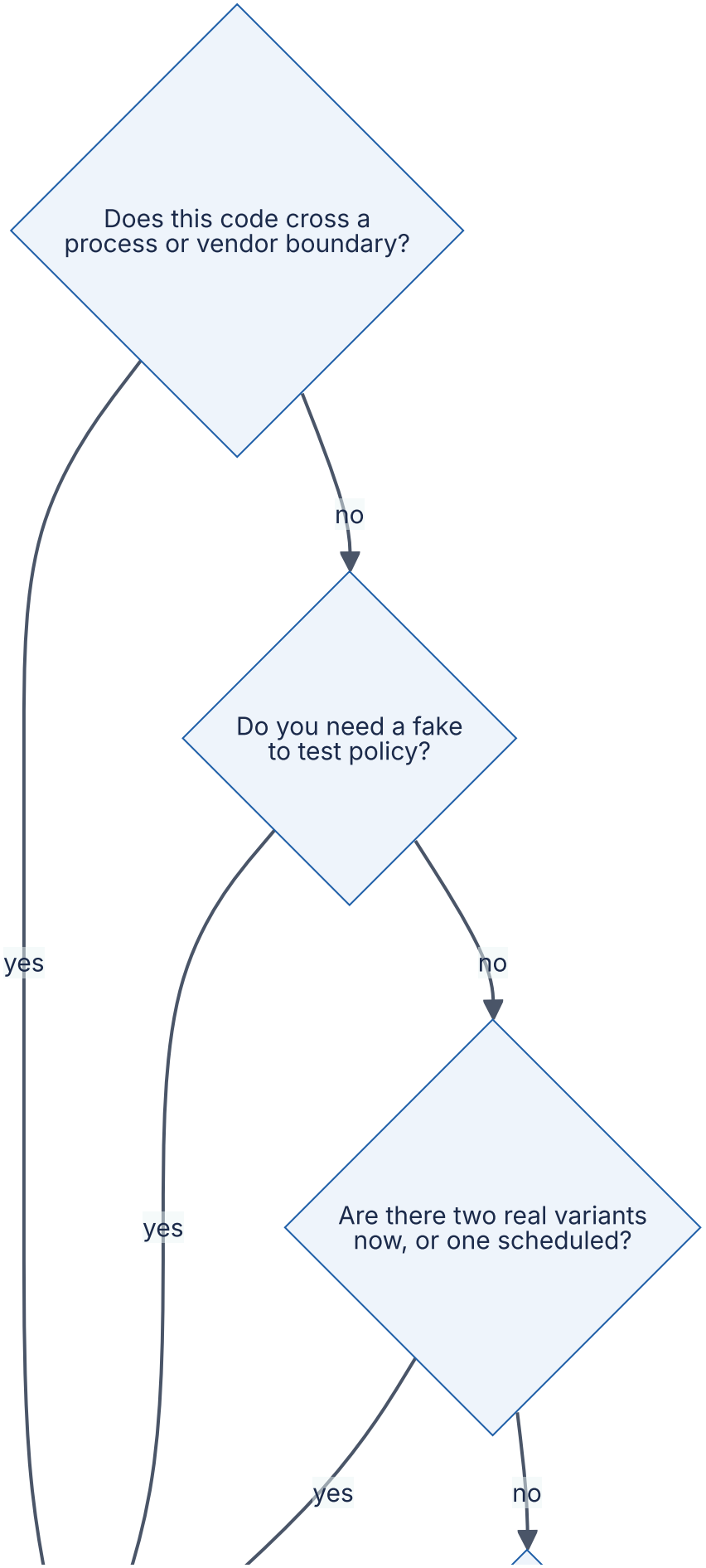
36.8.3 When SOLID is over-applied

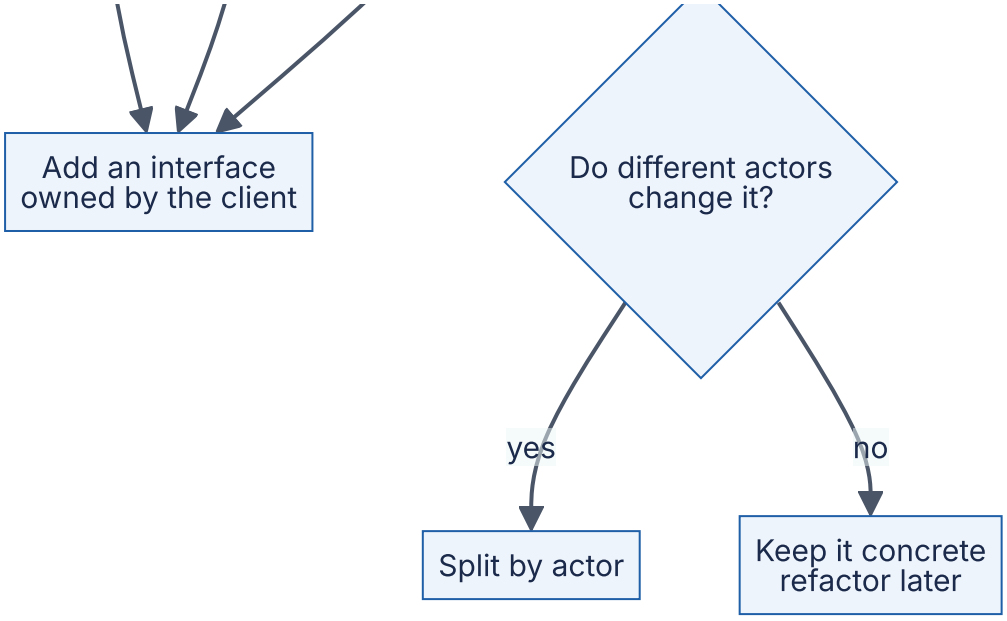
Every principle in this chapter adds structure: more classes, more interfaces, more indirection. Structure pays for itself only when the change it anticipates actually happens. Martin himself lists **needless complexity** among the symptoms of poor design, alongside rigidity and fragility [Martin 2002]. Extreme Programming's **YAGNI** ("you aren't gonna need it") rule says not to build capability for features you only presume you will need, because the presumed need often never arrives and the unused structure still costs effort to carry [Fowler 2015]. Fowler's catalog of code smells calls the result **speculative generality** [Fowler 2018].

Table 36.10 — Signs that SOLID has been over-applied.

Symptom	Example	Better default
Interface with exactly one implementation and no test fake	`IInvoiceCalculator` with `InvoiceCalculatorImpl`	Use the concrete class; extract an interface when a second implementation or a fake is needed
Abstraction for a variation that never came	`TaxStrategyFactoryProvider` for one country	Wait for the second real variant (“rule of three”)
Classes split by technical step, not by actor	`OrderValidator`, `OrderMapper`, `OrderHelper` all edited together	Merge classes that always change together
Every call goes through a DI container	Business logic unreadable without the container configuration	Hand-written composition root for small systems
Ports that mirror one vendor’s SDK	`PaymentGateway.createStripeCharge(...)`	Write ports in domain vocabulary, or skip the port

Figure 36.17 — Should you add an abstraction here?


Does this code cross a process or vendor boundary?
yes
no
Do you need a fake to test policy?
yes
no
Are there two real variants now, or one scheduled?
yes
no



Reading the diagram. Diamonds are decisions, evaluated top to bottom; rectangles are outcomes. The first two questions capture the cheap, almost always justified uses of DIP: external boundaries (databases, vendors, networks, clocks), which are failure points and slow in tests, and policy that needs a fake. The third applies OCP only when variation is real. The fourth applies SRP when more than one actor edits the code. The default is concrete code: extracting an interface later is a mechanical refactor, while removing a wrong abstraction is manual work.

RECOMMENDATION

A practical rule. Apply DIP at every boundary to I/O and to other teams' systems from day one; apply SRP when you can name two actors; apply OCP and ISP when you see the second variant or the second kind of client. Revisit when change history (which files change together, which teams edit them) tells you where the real axes are.

INTERVIEW TIP

SOLID in LLD interviews, overall. Interviewers rarely ask "recite SOLID". They give a design (parking lot, notification service, rate limiter) and ask for an extension. Show a small, concrete design first, name the axis of change you are protecting, and justify each interface by a client, a test fake or a real variant. Saying "I would not add an interface here yet, because…" is a strong signal, not a weak one.

36.9 Summary and revision notes

SUMMARY

- SOLID limits how far a change spreads. Each principle blocks one path along which changes ripple.
- **SRP**: a module answers to one actor. Shared helpers between actors are accidental coupling; split by actor behind a facade.
- **OCP**: add expected variants as new classes behind a stable abstraction. Meyer used implementation inheritance; Martin uses interfaces and polymorphism. Closure is strategic: choose the axis.
- **LSP**: subtypes may weaken preconditions and strengthen postconditions, never the reverse; they keep invariants and history. Enforce with contract tests at the boundaries.
- **ISP**: clients see role interfaces sized to their needs. Throwing stubs signal a fat interface.
- **DIP**: policy owns the interfaces; details implement them. DI is a technique, IoC a framework property, DIP a rule about dependency direction. Fakes of owned ports make policy testable without I/O.
- At architecture level: Common Closure (SRP), plugins (OCP), versioned contracts (LSP), Common Reuse and BFFs (ISP), ports and adapters (DIP). Instability $I = C_e/(C_a + C_e)$ should decrease along dependencies.
- Over-application costs real money: add abstractions for boundaries, fakes, real variants and real actors, not for imagined ones.

36.10 Knowledge checkpoint

1. Restate SRP using the word “actor”, and explain why “a class should do one thing” is a different rule.
2. In the worked example of §36.3.2, what was the receipt’s unexplained difference after finance’s change, and why?
3. How does Meyer’s mechanism for the open–closed principle differ from Martin’s?
4. What is “strategic closure”?
5. Write the Liskov–Wing precondition and postcondition rules as implications.
6. Why does a mutable `Square` subclass of a mutable `Rectangle` violate LSP, while an immutable one need not?
7. How many SMS parts does a 500-character message need under GSM-7 concatenation?
8. Which is worse, a subtype that strengthens a precondition or one that weakens a postcondition, and why?
9. Give two symptoms of a fat interface.

10. In DIP, which module should own the `PaymentGateway` interface, and why?
11. Distinguish DIP, DI and IoC in one sentence each.
12. Compute *I* for a component with 6 dependents and 2 dependencies.
13. Name two signs that SOLID has been over-applied.

Answers

1. A module should be responsible to one, and only one, actor: one group of people who request its changes for the same reasons. “Do one thing” is about function size; a class can do many things that all change for one actor’s reasons.
2. $6.90 between “items 40.00” and “total due 46.90”: the $4.00 wrap fee disappeared from the items line because the shared `_subtotal` changed for finance’s reasons.
3. Meyer closes a module by freezing it and extends it through implementation inheritance; Martin closes clients by having them depend on an abstraction, with new behavior added as new implementations.
4. Choosing which kinds of change a design is closed against, since no design is closed against all changes.
5. $\text{Pre}_T \Rightarrow \text{Pre}_S$ and $\text{Post}_S \Rightarrow \text{Post}_T$.
6. The mutable rectangle promises that setting the width leaves the height unchanged; a square cannot keep that promise. With no mutators there is no such promise to break, and a rectangle’s history cannot reach a non-square state through a square.
7. $\lceil 500 / 153 \rceil = \lceil 3.27 \rceil = 4$ parts.
8. Usually the weakened postcondition, because it fails silently (truncated text, lost data), while a strengthened precondition fails loudly.
9. Implementers write throwing stubs; unrelated clients rebuild or retest when methods they do not call change; signatures do not reveal what a function uses.
10. The high-level checkout module, so that the interface is written in policy terms and source dependencies point from the adapter toward policy.
11. DIP: policy and detail both depend on abstractions owned by policy. DI: collaborators are passed in rather than constructed. IoC: the framework calls your code.
12. $I = 2/(6 + 2) = 0.25$.
13. Interfaces with one implementation and no fake; abstractions for variants that never arrived; classes that always change together; ports that mirror one vendor’s SDK.

36.11 Interview questions

Basic

Q1. What is the Single Responsibility Principle? Give an example. Answer. A module should answer to one actor. An `Employee` or `Order` class that computes pay or tax (finance), formats reports (operations or communications) and saves itself (DBA) answers to

three actors; a change for one can break another through shared code. Split it into a calculator, a formatter and a repository, with a facade if callers want one entry point.

Q2. How would you add a new payment method to an existing checkout without modifying it? Answer. Checkout depends on a `PaymentMethod` (or `PaymentGateway`) interface; each method is a class implementing it, selected by a registry or configuration. The new method is a new class plus one registration line. That is OCP, and it holds only if the new class honors the interface's contract (LSP).

Intermediate

Q3. Is a square a rectangle? Answer as a software designer. Answer. It depends on the contract. If `Rectangle` has independent setters whose postconditions leave the other side unchanged, a mutable `Square` cannot honor them, and code like "set width 5, set height 4, expect area 20" breaks. If rectangles are immutable values, a square can be a rectangle, because no operation promises independent mutation.

Q4. What is the difference between dependency injection and dependency inversion? Answer. DI is a technique: pass collaborators in. DIP is a structural rule: high-level policy defines the interfaces it needs, and low-level details implement them, so source dependencies point toward policy. You can inject a concrete vendor class (DI without DIP) or follow DIP with a service locator.

Advanced

Q5. A team's codebase has an interface for every class, a DI container and 40% of its classes named `*Impl`. Changes are still slow. What is going on? Answer. Likely speculative generality: interfaces mirror classes one to one, so they add indirection without changing dependency direction or isolating actors. Check whether core packages still import infrastructure, whether interfaces have a second implementation or a fake, and which files change together. Remove single-implementation interfaces not at I/O boundaries, split by actor where change history shows conflicts, and move ports into the policy packages.

Q6. How do SOLID principles map to a microservice architecture, and where does the analogy break? Answer. SRP becomes services aligned to business capabilities and teams; OCP becomes event-driven extension (new consumers without changing producers); LSP becomes backward-compatible API versions; ISP becomes client-specific APIs (BFFs); DIP becomes ports and adapters inside each service. The analogy breaks on cost: crossing a service boundary adds network latency, partial failure and data consistency problems that do not exist between classes, so service boundaries should be fewer and coarser than class boundaries.

36.12 Practical exercises

1. **Actor audit.** Take a 300+ line class from a codebase you know. List each public method's actor, draw a Figure 36.3-style diagram, and propose a split.
2. **Contract test.** Write a pytest or JUnit contract test for a `Repository` interface (save, find by id, delete) and run it against an in-memory fake and a SQLite implementation.
3. **Remove a switch.** Find an `if/elif` or `switch` on a type code repeated in more than one place and replace it with polymorphism. Count files touched before and after when adding a new type.
4. **Metrics.** Run the §36.8.2 script on a real project's import graph (generate it with a tool such as `pydeps` or `jdeps`) and identify components in the zone of pain.

36.13 Design challenges

1. **Notification platform.** Extend the running example to support quiet hours, per-customer channel preferences and per-channel rate limits. Show which classes change, which are added, and which principle protects each existing class.
2. **Multi-provider payments.** Design ports and adapters for two payment providers with different refund semantics (one allows partial refunds, one does not). Make the base contract honest so that both adapters satisfy LSP.
3. **Plugin pricing engine.** Design a pricing core that lets regional teams add discount rules without redeploying the core. Decide what is closed, what is open, and how a faulty plugin is contained.

36.14 References

1. Bloch, J. *Effective Java*, 3rd ed. Addison-Wesley, 2018.
2. Cockburn, A. *Hexagonal Architecture*. alistair.cockburn.us, 2005.
3. Fowler, M. *Inversion of Control Containers and the Dependency Injection pattern*. martinfowler.com, 2004. https://martinfowler.com/articles/injection.html
4. Fowler, M. *RoleInterface*. martinfowler.com bliki, 2006. https://martinfowler.com/bliki/RoleInterface.html
5. Fowler, M. *Yagni*. martinfowler.com bliki, 2015. https://martinfowler.com/bliki/Yagni.html
6. Fowler, M. *Refactoring: Improving the Design of Existing Code*, 2nd ed. Addison-Wesley, 2018.
7. Larman, C. *Protected Variation: The Importance of Being Closed*. IEEE Software 18(3), 2001.
8. Levkivskyi, I., Lehtosalo, J., Langa, Ł. *PEP 544 — Protocols: Structural subtyping (static duck typing)*. Python Software Foundation, 2017.

https://peps.python.org/pep-0544/
9. Linux kernel documentation. *Overview of the Linux Virtual File System*. Accessed 2026. https://docs.kernel.org/filesystems/vfs.html
10. Liskov, B. *Data Abstraction and Hierarchy*. Keynote, OOPSLA ’87 Addendum to the Proceedings; reprinted in ACM SIGPLAN Notices 23(5), 1988.
11. Liskov, B. H., Wing, J. M. *A Behavioral Notion of Subtyping*. ACM Transactions on Programming Languages and Systems 16(6), 1994.
12. Martin, R. C. *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall, 2002.
13. Martin, R. C. *The Single Responsibility Principle*. The Clean Code Blog, 2014. https://blog.cleancoder.com/uncle-bob/2014/05/08/SingleReponsibilityPrinciple.html
14. Martin, R. C. *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017.
15. Meszaros, G. *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley, 2007.
16. Meyer, B. *Object-Oriented Software Construction*. Prentice Hall, 1988; 2nd ed. 1997.
17. Oracle. *Java SE API documentation*: `java.util.Date`, `java.time`, `java.sql.Timestamp`, `java.sql.DriverManager`, `java.util.ServiceLoader`; and *Java Collections API Design FAQ*. Accessed 2026.
18. Parnas, D. L. *On the Criteria To Be Used in Decomposing Systems into Modules*. Communications of the ACM 15(12), 1972.
19. Python Software Foundation. *collections.abc — Abstract Base Classes for Containers*. Python documentation, accessed 2026.
20. SLF4J project. *SLF4J user manual*. Accessed 2026. https://www.slf4j.org/manual.html

36.15 Open verification items

- The Xerox `Job`-class origin story of ISP: confirm it appears in Martin 2002 (it may come from Martin’s 1996 C++ Report column or a later talk).
- Exact URL of Martin’s 2014 SRP blog post (the path is spelled “SingleReponsibilityPrinciple” as recalled).
- Wording of Liskov’s 1987 substitution property quoted from the SIGPLAN Notices reprint; page numbers not checked.
- Attribution of the “SOLID” acronym (commonly credited to Michael Feathers, c. 2004) was not verified and is not asserted in the text.

Chapter 37 — Design Patterns

> **Level:** 2 · **Track:** A (Traditional) · **Estimated reading time:** 3.5 h

A carpenter who has built a hundred staircases does not reinvent the stringer, the tread and the riser each time. They recognize the problem ("connect two floors safely in this much space"), recall a solution shape that has worked before, and adapt it to the room. Software has the same kind of recurring problems: "create the right object without hard-coding its class", "add retries to a client without editing it", "let five subsystems react when an order is placed". A **design pattern** is a named, reusable solution shape for one of these problems. This chapter catalogs the patterns that keep appearing in the code behind large systems, shows each one in runnable Python, and connects each to the system-level idea it foreshadows: Observer to publish/subscribe, Proxy to API gateways and sidecars, Chain of Responsibility to middleware, producer–consumer to message queues.

DEFINITION

Definition — Design pattern. A named description of a recurring design problem, the core of a solution (a set of collaborating classes or functions and their responsibilities), and the trade-offs of applying it. A pattern is a template you adapt, not a library you import [Gamma et al. 1994].

The canonical catalog is the "Gang of Four" (GoF) book, which grouped 23 patterns into three families [Gamma et al. 1994]: **creational** patterns (how objects are made: Factory Method, Abstract Factory, Builder, Singleton, Prototype), **structural** patterns (how objects are composed: Adapter, Decorator, Proxy, Facade, Composite, Bridge, Flyweight) and **behavioral** patterns (how objects divide work and communicate: Strategy, Observer, Command, State, Template Method, Chain of Responsibility and others). Two further families matter for system design: **enterprise** patterns such as Repository [Fowler 2002] and Dependency Injection [Fowler 2004], and **concurrency** patterns such as producer–consumer and thread pools [Schmidt et al. 2000; Goetz et al. 2006].

KEY CONCEPT

Patterns depend on the language. The GoF book was written for C++ and Smalltalk. In a language with first-class functions, modules and duck typing, several patterns shrink: a Strategy becomes a function argument, a Singleton becomes a module, a Command becomes a closure. Norvig observed in 1996 that most GoF patterns are simpler or invisible in dynamic languages [Norvig 1996]. This chapter shows the classic structure first, because interviewers and Java codebases use it, and then the idiomatic Python form.

Every pattern below uses the same compact structure: intent, the problem it solves in a real system, the structure as a Mermaid class diagram, a Python implementation (all run under Python 3.13), a short Java sketch only where Java adds something language-specific, when to use and when not, related patterns, and one interview question. Most examples share one running system: the order service of an online store, which takes orders, charges payments, sends notifications and talks to cloud storage.

37.1 Learning objectives

After this chapter you should be able to:

- State the intent, structure and trade-offs of the Factory, Abstract Factory, Builder, Singleton, Strategy, Observer, Adapter, Decorator, Proxy, Command, Repository, State, Template Method and Chain of Responsibility patterns, and draw each as a class diagram.
- Implement each pattern in idiomatic Python and recognize where a function, module or closure replaces a class hierarchy.
- Explain why Singleton is often an anti-pattern, write a thread-safe variant, and explain why a “singleton” in a 50-pod deployment is really 50 objects.
- Distinguish the GoF Decorator from the Python `@decorator` syntax, and Adapter from Decorator from Proxy.
- Apply constructor injection and a composition root, and relate them to the Dependency Inversion Principle (§36.7).
- Build producer–consumer pipelines, thread pools and futures, size a thread pool with the utilization formula, and explain why double-checked locking is broken without a memory-model guarantee.
- Choose a pattern for a stated problem and name the trade-off you accept.

37.2 Prerequisites

- Classes, interfaces, polymorphism and composition over inheritance — §35.3–§35.9 (every pattern here is built from them).
- Cohesion and coupling — §35.10 (most patterns trade a little indirection for lower coupling).
- SOLID principles, especially Open/Closed (§36.4) and Dependency Inversion (§36.7) — patterns are the concrete mechanisms that realize them.
- Threads, locks, condition variables and read-write locks — §6.3, §6.9–§6.10 (needed for Singleton thread safety and §37.16).
- Little's Law — §4.11 (used to size thread pools).

37.3 Factory

Creating an object with `EmailNotifier()` is the simplest thing in programming, and it is also a hidden decision: the calling code now knows, and depends on, one concrete class. When the choice of class depends on configuration, input or environment, that decision should live in exactly one place. Factories put it there.

37.3.1 Simple factory

Intent

DEFINITION

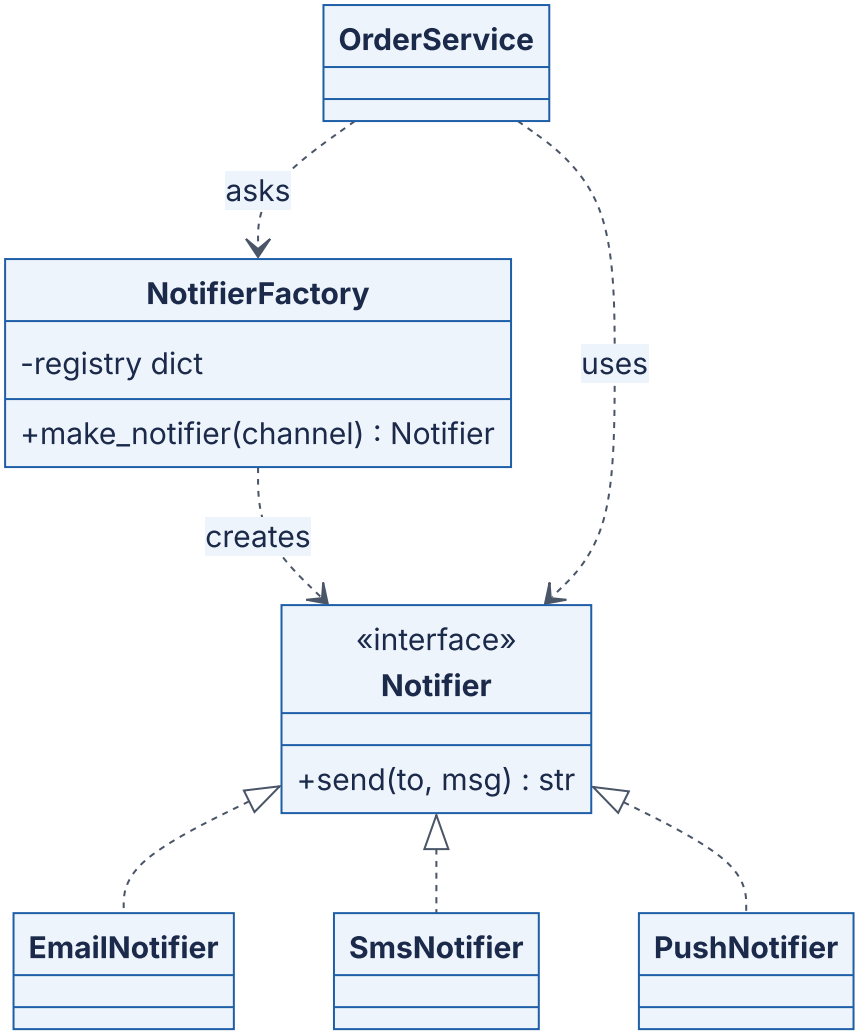
Definition — Simple factory. A function or static method that takes a description of what is wanted (a string, an enum, a config value) and returns an instance of the matching concrete class, behind a common interface. It is an idiom rather than a GoF pattern, but it is the most common factory in practice.

Problem it solves

The order service sends notifications by email, SMS or push, and the channel comes from each user's preferences. Without a factory, every call site contains the same `if channel == "email": ... elif ...` chain, and adding WhatsApp means editing all of them. With a factory, call sites ask for "a notifier for this channel" and only the factory knows the mapping.

Structure

Figure 37.1 — Simple factory for notifiers.



Reading the diagram. `Notifier` is the interface every channel implements; the three concrete classes realize it (dotted arrows with hollow heads). `NotifierFactory` owns a registry from channel name to class and is the only component that names concrete classes. `OrderService` depends on the factory and on the interface, never on a concrete notifier. All calls are synchronous, in-process method calls. The registry is the only state, held in memory. The failure point is an unknown channel name, which the factory should reject loudly rather than return `None`. Adding a channel extends the registry without touching callers, which is the Open/Closed Principle (§36.4) in action.

37.3.2 Factory method

Intent

DEFINITION

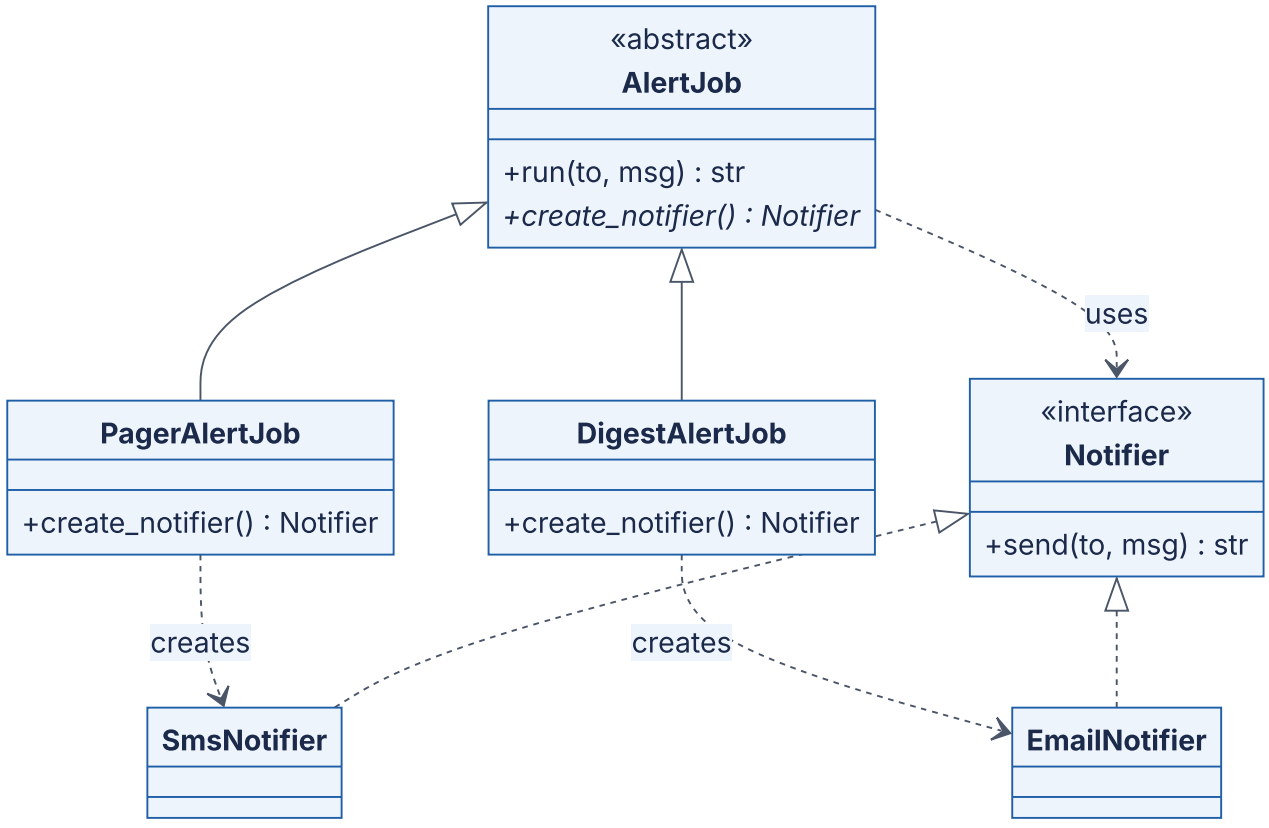
Definition — Factory method. A method declared in a base class that returns a product through an interface, which subclasses override to decide which concrete product to create. The base class's other methods use the product without knowing its class [Gamma et al. 1994].

Problem it solves

A framework defines the workflow and lets applications supply the parts. An alerting framework's `AlertJob.run()` always formats a message and sends it, but a paging job must use SMS and a daily digest job must use email. The base class cannot know which; the subclass can. Real frameworks are full of factory methods: Python's `socketserver` and `unittest` let subclasses decide the handler or test objects the framework creates, and Java's `Collection.iterator()` lets each collection return its own iterator class.

Structure

Figure 37.2 — Factory method.



Reading the diagram. The abstract creator `AlertJob` defines `run()`, which calls the abstract `create_notifier()` (the asterisk marks it abstract). Each concrete creator overrides that one method and names one concrete product. Control flows downward through inheritance (solid arrows with hollow heads) and the creation dependency points from each subclass to its product. Everything is synchronous and in-process; no state is stored beyond the objects themselves. The design scales by adding a creator subclass per product variant, which is also its weakness: two independent variation axes (channel and urgency, say) multiply subclasses, and composition (Strategy, §37.7, or injection, §37.14) is then the better tool.

Python implementation

```
from abc import ABC, abstractmethod


class Notifier(ABC):
    @abstractmethod
    def send(self, to: str, msg: str) -> str: ...


class EmailNotifier(Notifier):
    def send(self, to, msg):
        return f"email to {to}: {msg}"


class SmsNotifier(Notifier):
    def send(self, to, msg):
        return f"sms to {to}: {msg[:160]}"          # SMS segments are 160 chars


class PushNotifier(Notifier):
    def send(self, to, msg):
        return f"push to {to}: {msg}"


# --- Simple factory: one place owns the "which class?" decision ---
_REGISTRY: dict[str, type[Notifier]] = {
    "email": EmailNotifier, "sms": SmsNotifier, "push": PushNotifier,
}


def make_notifier(channel: str) -> Notifier:
    try:
        return _REGISTRY[channel]()
    except KeyError:
        raise ValueError(f"unknown channel {channel!r}") from None


# --- Factory method: subclasses decide what the base class creates ---
class AlertJob(ABC):
    @abstractmethod
    def create_notifier(self) -> Notifier: ...      # the factory method

    def run(self, to: str, msg: str) -> str:        # fixed workflow uses it
        return self.create_notifier().send(to, f"[ALERT] {msg}")


class PagerAlertJob(AlertJob):
    def create_notifier(self):
        return SmsNotifier()


class DigestAlertJob(AlertJob):
    def create_notifier(self):
        return EmailNotifier()
```

```
        if __name__ == "__main__":
            print(make_notifier("push").send("user-42", "Order shipped"))
            for job in (PagerAlertJob(), DigestAlertJob()):
                print(job.run("oncall", "p99 latency above SLO"))
            try:
                make_notifier("fax")
            except ValueError as e:
                print("rejected:", e)
```

Output:

```
push to user-42: Order shipped
sms to oncall: [ALERT] p99 latency above SLO
email to oncall: [ALERT] p99 latency above SLO
rejected: unknown channel 'fax'
```

What to notice:

- The registry is a dictionary of classes. In Python, classes are objects, so a simple factory needs no `if` chain and plugins can register themselves at import time.
- The factory fails loudly on unknown input. A factory that returns `None` moves the crash to a distant call site.
- `AlertJob.run()` never names a concrete class; only the one-line overrides do.

When to use and when not

Use a simple factory when the concrete class is chosen at run time from data (config, a request field, a feature flag). Use a factory method when a base class or framework owns a workflow and subclasses supply a part. Do not add a factory when there is only one implementation and no realistic second one: `Thing()` is clearer than `ThingFactory.create()`. In Python, passing a class or a callable (`notifier_cls=SmsNotifier`) often replaces a factory-method hierarchy.

Related patterns

Abstract Factory (§37.4) is a set of factory methods for a family of products. Template Method (§37.15.2) is the general form of “base class owns the workflow”. Dependency injection (§37.14) usually decides which factory is used.

Interview question

INTERVIEW TIP

Q. What is the difference between a simple factory and the factory method pattern?
A. A simple factory is one function that maps a parameter to a concrete class; callers pass data in. A factory method is an overridable method in a class hierarchy; the subclass, not a parameter, decides the product, and the base class's own logic uses it. The first centralizes a run-time choice; the second lets a framework defer a decision to its extensions.

37.4 Abstract factory

Intent

DEFINITION

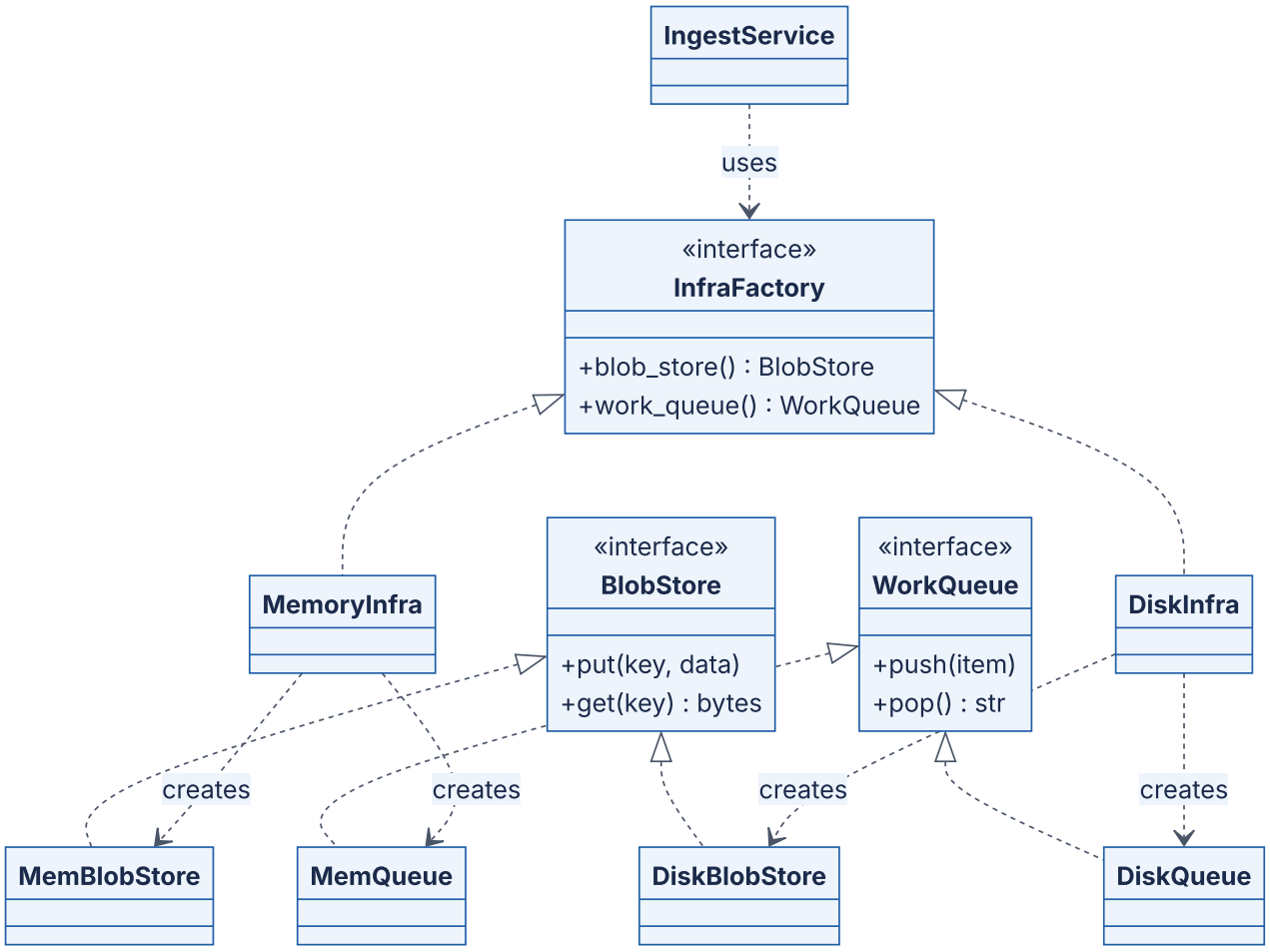
Definition — Abstract factory. An interface for creating a *family* of related objects (for example a blob store and a work queue) without naming their concrete classes, so that a client receives products that are guaranteed to belong together [Gamma et al. 1994].

Problem it solves

The order service stores invoices as blobs and enqueues work for a background worker. In production it uses a cloud object store and a cloud queue; in unit tests it uses in-memory fakes; on a laptop it uses the local disk. Mixing families (a disk blob store with a cloud queue whose workers cannot read the disk) is a bug that type checks cannot catch unless the families are created together. An abstract factory makes "which environment am I in?" one decision, made once, at startup. Portable libraries use the same idea: Java's `javax.xml.parsers.DocumentBuilderFactory` hides which XML parser implementation is installed.

Structure

Figure 37.3 — Abstract factory for infrastructure families.



Reading the diagram. There are two product interfaces (`BlobStore`, `WorkQueue`) and one factory interface with one creation method per product. Each concrete factory creates one consistent family: `MemoryInfra` creates the in-memory store and queue, `DiskInfra` the file-based ones. `IngestService` depends only on the three interfaces. Calls are synchronous and in-process here; in production the products would wrap network clients, and the storage boundary would sit behind `BlobStore.put` and `WorkQueue.push`, which are the failure points (timeouts, throttling). Adding a family (a cloud one) is easy; adding a product type (a cache) is hard, because every factory must change.

Python implementation

```
import os
import tempfile
from abc import ABC, abstractmethod
from collections import deque


class BlobStore(ABC):
    @abstractmethod
    def put(self, key: str, data: bytes) -> None: ...
    @abstractmethod
    def get(self, key: str) -> bytes: ...


class WorkQueue(ABC):
    @abstractmethod
    def push(self, item: str) -> None: ...
    @abstractmethod
    def pop(self) -> str | None: ...


class InfraFactory(ABC):
    @abstractmethod
    def blob_store(self) -> BlobStore: ...
    @abstractmethod
    def work_queue(self) -> WorkQueue: ...


# Family 1: in-memory, for unit tests
class MemBlobStore(BlobStore):
    def __init__(self): self._d: dict[str, bytes] = {}
    def put(self, key, data): self._d[key] = data
    def get(self, key): return self._d[key]


class MemQueue(WorkQueue):
    def __init__(self): self._q: deque[str] = deque()
    def push(self, item): self._q.append(item)
    def pop(self): return self._q.popleft() if self._q else None


class MemoryInfra(InfraFactory):
    def blob_store(self): return MemBlobStore()
    def work_queue(self): return MemQueue()


# Family 2: local disk, for a developer laptop
class DiskBlobStore(BlobStore):
    def __init__(self, root): self.root = root
    def put(self, key, data):
        with open(os.path.join(self.root, key), "wb") as f:
            f.write(data)
    def get(self, key):
        with open(os.path.join(self.root, key), "rb") as f:
            return f.read()
```

```
        class DiskQueue(WorkQueue):
            def __init__(self, root):
                self.path, self.read_pos = os.path.join(root, "queue.log"), 0
            def push(self, item):
                with open(self.path, "a") as f:
                    f.write(item + "\n")
            def pop(self):
                if not os.path.exists(self.path):
                    return None
                with open(self.path) as f:
                    lines = f.read().splitlines()
                if self.read_pos >= len(lines):
                    return None
                self.read_pos += 1
                return lines[self.read_pos - 1]


        class DiskInfra(InfraFactory):
            def __init__(self): self.root = tempfile.mkdtemp()
            def blob_store(self): return DiskBlobStore(self.root)
            def work_queue(self): return DiskQueue(self.root)


        def ingest(infra: InfraFactory, name: str, payload: bytes) -> str:
            store, queue = infra.blob_store(), infra.work_queue()   # one consistent
family
            store.put(name, payload)
            queue.push(name)
            key = queue.pop()
            return f"{type(infra).__name__}: {key} -> {store.get(key)!r}"


        if __name__ == "__main__":
            factories = {"test": MemoryInfra, "dev": DiskInfra}
            for env in ("test", "dev"):
                print(ingest(factories[env](), "invoice-17.txt", b"total=4999"))
```

Output:

```
MemoryInfra: invoice-17.txt -> b'total=4999'
DiskInfra: invoice-17.txt -> b'total=4999'
```

What to notice:

- `ingest` is written once and runs unchanged against both families; the environment is chosen in one dictionary lookup at the top.
- The factory guarantees compatibility: `DiskQueue` items name files that `DiskBlobStore` can read.
- The cost is many small classes. In Python you can often replace the factory class with a small dataclass of two callables, or a module per environment.

When to use and when not

Use an abstract factory when you have several product types that must vary together (per cloud, per database vendor, per UI theme, per test versus production). Do not use it when products vary independently, when there is only one family, or when you expect to add product types frequently, since each new product forces a change to every factory.

Related patterns

Each creation method is a factory method (§37.3.2). Concrete factories are often configured by dependency injection (§37.14) and are sometimes, unwisely, made singletons (§37.6). Hexagonal architecture (§33.10) generalizes the idea: ports are the product interfaces and each environment supplies adapters.

Interview question

> INTERVIEW TIP
>
> **Q.** Your service must run on two cloud providers. How would you keep provider-specific code out of business logic? **A.** Define ports for each capability (blob storage, queue, secrets) and one abstract factory per provider that builds a consistent set of adapters. Business logic receives the factory or the built adapters via constructor injection; provider choice is made once in the composition root from configuration. Contract tests run against every family. Accept that new capabilities require changes to every provider's factory.

37.5 Builder

Intent

> DEFINITION
>
> **Definition — Builder.** A separate object that assembles a complex product step by step, validates the combination of choices, and returns a finished (ideally immutable) product from a final `build()` call [Gamma et al. 1994; Bloch 2018].

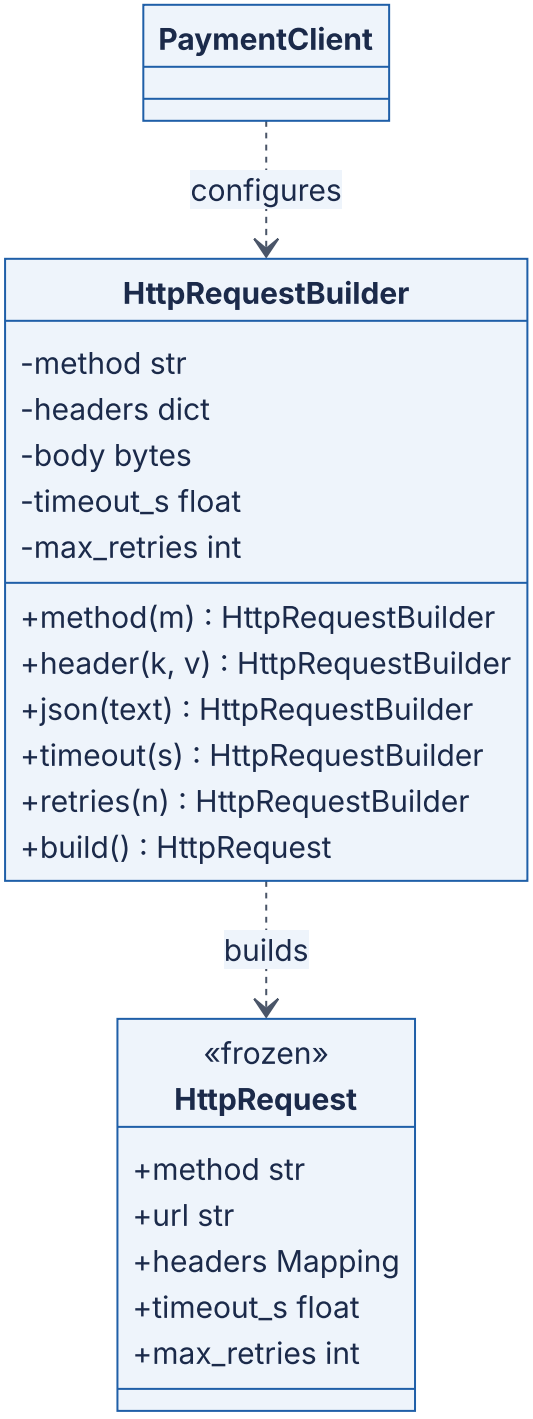
Problem it solves

An outbound HTTP request in the order service has a method, URL, headers, an optional body, a timeout and a retry count, and some combinations are invalid: a `GET` with a body, a timeout of 0, or a retried `POST` without an idempotency key (which risks charging a customer twice, §22.15). A constructor with eight positional arguments is unreadable, and

setters on a mutable object allow half-built requests to escape. A builder collects choices fluently, checks cross-field rules once, and emits a frozen object. Real APIs use exactly this shape: Java 11's `HttpRequest.newBuilder()`, and the `newBuilder()` methods that Protocol Buffers generates for every Java message class.

Structure

Figure 37.4 — Builder producing an immutable request.



Reading the diagram. The client (`PaymentClient`) talks only to the mutable builder, whose setters each return the builder so calls chain. `build()` is the single validation point and the only path to the immutable `HttpRequest`. Everything is synchronous and in memory; no I/O happens until a transport later sends the product. The failure point is `build()`, which raises on an invalid combination, so an invalid request can never exist. Immutable products are safe to share across threads and to cache.

Python implementation

```
        from dataclasses import dataclass
        from types import MappingProxyType


        @dataclass(frozen=True)
        class HttpRequest:
            method: str
            url: str
            headers: MappingProxyType
            body: bytes | None
            timeout_s: float
            max_retries: int


        class HttpRequestBuilder:
            def __init__(self, url: str):
                self._url, self._method = url, "GET"
                self._headers: dict[str, str] = {}
                self._body: bytes | None = None
                self._timeout_s, self._max_retries = 2.0, 0

            def method(self, m):   self._method = m.upper(); return self
            def header(self, k, v): self._headers[k.lower()] = v; return self
            def timeout(self, s):  self._timeout_s = s; return self
            def retries(self, n):  self._max_retries = n; return self

            def json(self, text: str):
                self._body = text.encode()
                return self.header("Content-Type", "application/json")

            def build(self) -> HttpRequest:
                if self._body is not None and self._method in ("GET", "HEAD"):
                    raise ValueError(f"{self._method} must not carry a body")
                if self._max_retries and self._method == "POST" \
                        and "idempotency-key" not in self._headers:
                    raise ValueError("retried POST needs an Idempotency-Key header")
                if not 0 < self._timeout_s <= 30:
                    raise ValueError("timeout must be in (0, 30] seconds")
                return HttpRequest(self._method, self._url,
                                   MappingProxyType(dict(self._headers)),  # read-only
copy
                                   self._body, self._timeout_s, self._max_retries)


        if __name__ == "__main__":
            req = (HttpRequestBuilder("https://payments.internal/charges")
                   .method("post").json('{"amount": 4999}')
                   .header("Idempotency-Key", "order-17-charge").timeout(0.5).retries(3)
                   .build())
            print(req.method, req.timeout_s, req.max_retries, dict(req.headers))
            try:

HttpRequestBuilder("https://payments.internal/charges").method("POST").retries(3).build(
)
```

```
        except ValueError as e:
            print("rejected:", e)
```

Output:

```
POST 0.5 3 {'content-type': 'application/json', 'idempotency-key': 'order-17-charge'}
rejected: retried POST needs an Idempotency-Key header
```

What to notice:

- Cross-field rules (retries require an idempotency key) live in `build()`, the one place that sees all fields together.
- The product is frozen and its headers are a read-only `MappingProxyType` copy, so later builder calls cannot mutate a request already in flight.

Java sketch

Python has keyword arguments with defaults, so `HttpRequest(url=..., timeout_s=0.5)` already reads well; a builder earns its keep mainly for cross-field validation. Java has no named or default parameters, so constructors with many parameters (“telescoping constructors”) are common, and the static nested Builder is the standard cure [Bloch 2018]:

```
        public final class RequestSpec {
            private final String url, method;
            private final int timeoutMs, maxRetries;

            private RequestSpec(Builder b) {                        // only the builder
can construct
                url = b.url; method = b.method; timeoutMs = b.timeoutMs; maxRetries =
b.maxRetries;
            }
            public static Builder newBuilder(String url) { return new Builder(url); }

            public static final class Builder {
                private final String url;
                private String method = "GET";
                private int timeoutMs = 2000, maxRetries = 0;
                private Builder(String url) { this.url = url; }
                public Builder method(String m)     { method = m; return this; }
                public Builder timeoutMs(int t)     { timeoutMs = t; return this; }
                public Builder maxRetries(int n)    { maxRetries = n; return this; }
                public RequestSpec build() {
                    if (timeoutMs <= 0) throw new IllegalStateException("timeout must be
positive");
                    return new RequestSpec(this);
                }
            }
        }
        // RequestSpec r = RequestSpec.newBuilder("https://api.example.com/orders")
        //
.method("POST").timeoutMs(500).maxRetries(3).build();
```

When to use and when not

Use a builder for objects with many optional parts, cross-field invariants, or a need for immutability after construction (requests, queries, configuration, test fixtures). Do not use one for objects with two or three fields; in Python, a frozen dataclass with keyword arguments and a `__post_init__` check is usually enough.

Related patterns

Abstract Factory (§37.4) returns a product in one call; Builder assembles one in steps. A builder is often created by a factory method (`newBuilder()`). Test-data builders are a common use in tests of Repository-based code (§37.13).

Interview question

> INTERVIEW TIP
>
> **Q.** Why make the built object immutable? **A.** Immutability means validation in `build()` holds forever: no later setter can break an invariant. Immutable objects can be shared between threads without locks, used as cache keys and dictionary keys, and retried safely because the request that is retried is exactly the one that was validated.

37.6 Singleton

Intent

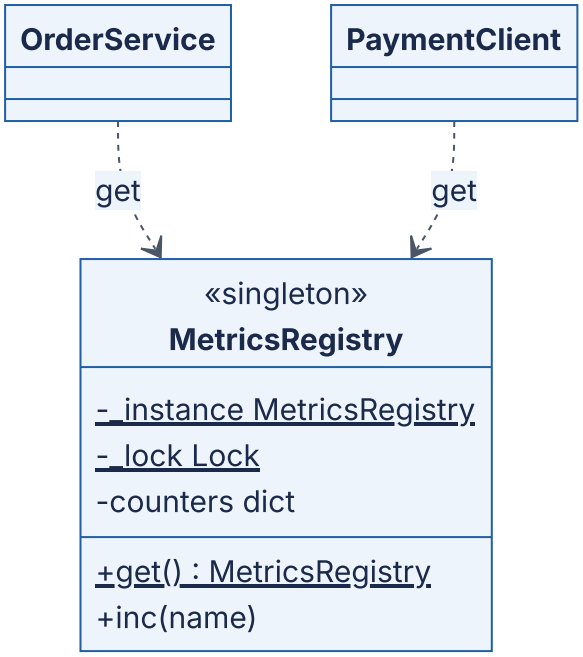
> DEFINITION
>
> **Definition — Singleton.** A class that guarantees it has exactly one instance within a scope and provides a global access point to it [Gamma et al. 1994]. The scope is almost always one process, a fact that matters greatly in distributed systems.

Problem it solves

Some resources should exist once per process: a metrics registry that aggregates counters before export, a connection pool, a configuration object loaded at startup, the logging hierarchy. Creating a second connection pool by accident doubles database connections; two metrics registries split counters so dashboards undercount. Python's `logging.getLogger(name)` returns the same logger object for the same name every time, which is a registry of singletons keyed by name.

Structure

Figure 37.5 — Singleton with lazy, thread-safe creation.



Reading the diagram. `MetricsRegistry` holds a class-level (static, underlined `$`) reference to its only instance and a lock that guards first creation. Any component calls the static `get()` and receives the same object, synchronously and in-process. The state (counters) lives in that process's memory only. The failure points are a race during first creation (two threads both see "no instance"), hidden coupling (callers do not declare the dependency, so tests cannot substitute it), and shared mutable state that every caller can corrupt. It does not scale across processes: each process has its own.

Python implementation

```
        import json
        import sys
        import threading
        import time


        class NaiveRegistry:
            _instance, created = None, 0

            @classmethod
            def get(cls):
                if cls._instance is None:           # check ...
                    time.sleep(0.05)                # slow initialization widens the
window
                    cls.created += 1
                    cls._instance = cls()           # ... then act: not atomic
                return cls._instance


        class SafeRegistry:
            _instance, created = None, 0
            _lock = threading.Lock()

            @classmethod
            def get(cls):
                if cls._instance is None:           # fast path: no lock after creation
                    with cls._lock:
                        if cls._instance is None:   # re-check while holding the lock
                            time.sleep(0.05)
                            cls.created += 1
                            cls._instance = cls()
                return cls._instance


        def hammer(getter, n_threads=8):
            barrier = threading.Barrier(n_threads)   # release all threads at once
            def worker():
                barrier.wait()
                getter()
            threads = [threading.Thread(target=worker) for _ in range(n_threads)]
            for t in threads: t.start()
            for t in threads: t.join()


        if __name__ == "__main__":
            hammer(NaiveRegistry.get)
            hammer(SafeRegistry.get)
            print("naive instances created:", NaiveRegistry.created)
            print("safe instances created: ", SafeRegistry.created)
            # The idiomatic Python singleton is a module: imports are cached.
            import json as again
            print("module is cached:", again is json is sys.modules["json"])
```

Output (the naive count can vary between runs; it was 8 in every run we tried):

```
naive instances created: 8
safe instances created:  1
module is cached: True
```

What to notice:

- Check-then-act without a lock is a race condition (§6.9): all eight threads saw `None` before any assigned.
- The safe version re-checks under the lock. In CPython this double-check is correct because an object is fully built before the assignment publishes it; the same code in Java needs `volatile` (§37.16.5).
- A **module-level singleton** is the Python idiom: put `registry = MetricsRegistry()` in `metrics.py`, and every `import metrics` returns the module object cached in `sys.modules`. Import itself is protected by the interpreter's import lock.

Java sketch

Java offers two idioms that use language guarantees instead of hand-written locking. An `enum` with one constant is serialization-safe and reflection-safe [Bloch 2018]; the **initialization-on-demand holder** relies on the JVM's guarantee that class initialization is lazy and thread-safe:

```
        public enum MetricsRegistry {
            INSTANCE;
            private final java.util.concurrent.ConcurrentHashMap<String, Long> counters
=
                new java.util.concurrent.ConcurrentHashMap<>();
            public void inc(String name) { counters.merge(name, 1L, Long::sum); }
        }

        final class AppConfig {
            private AppConfig() { /* read files, environment */ }
            private static final class Holder { static final AppConfig INSTANCE = new
AppConfig(); }
            static AppConfig get() { return Holder.INSTANCE; }   // Holder initialized
on first call
        }
```

When to use and when not

Singleton is often an **anti-pattern** because it is a global variable with a class around it. It hides dependencies (a function's signature does not reveal that it writes metrics), makes tests order-dependent (state leaks from one test to the next), prevents running two configurations side by side, and invites lock contention on a hot shared object. Prefer creat-

ing one instance in the composition root and passing it in (§37.14); you keep "exactly one" by construction and gain testability. Reserve true singletons for stateless or effectively immutable process-wide facilities (logging, a metrics registry), and give them a reset hook for tests.

COMMON MISTAKE

Mistake — "A singleton guarantees one instance in the system." It guarantees one per process (per class loader in Java). An order service running 50 pods with 4 worker processes each has 200 "singleton" connection pools; at 10 connections each, that is 2,000 database connections, which can exhaust a PostgreSQL server configured for a few hundred. A system-wide single instance (one scheduler, one leader) needs leader election and leases (§26.5, §26.11), not a class pattern.

Related patterns

Dependency injection (§37.14) is the usual replacement. Abstract factories and registries are sometimes singletons. Double-checked locking (§37.16.5) is the classic lazy-initialization idiom and its classic bug.

Interview question

INTERVIEW TIP

Q. How would you make a singleton testable? **A.** Stop calling `get()` from business code. Create the instance once at startup and inject it through constructors; tests then pass a fake. If legacy code must call a global accessor, provide a test-only `reset()` or an override hook, and keep the singleton's state immutable or behind a narrow interface.

37.7 Strategy

Intent

DEFINITION

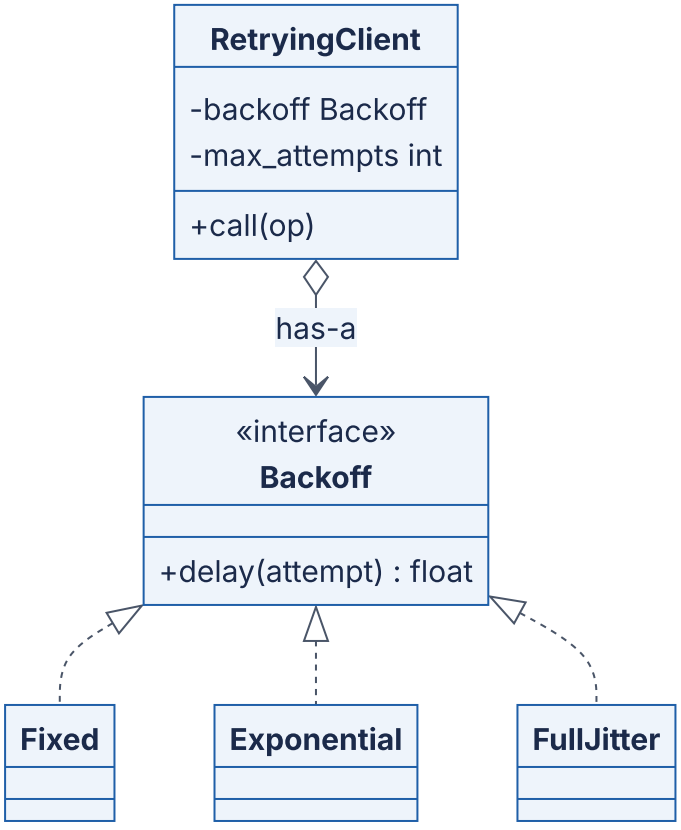
Definition — Strategy. A family of interchangeable algorithms, each behind a common interface, so that a client can be configured with one at run time without changing its code [Gamma et al. 1994].

Problem it solves

The order service retries calls to the payment provider. Which backoff policy is right depends on the dependency: a fixed delay for an internal cache, exponential backoff with **full jitter** for a shared external API, where synchronized retries from thousands of clients would otherwise arrive in waves (§22.12) [Brooker 2015]. Hard-coding the policy in the retry loop means editing and retesting the loop for every new policy. Strategy separates "when to retry" (the client's job) from "how long to wait" (the policy). The standard library is full of strategies: the `key=` argument of Python's `sorted`, Java's `Comparator`, and pluggable load-balancing policies (Chapter 19).

Structure

Figure 37.6 — Strategy for retry backoff.



Reading the diagram. `RetryingClient` (the context) holds a reference to one `Backoff` (aggregation, hollow diamond) and asks it for a delay after each failure. Three concrete strategies implement the same one-method interface. The call to the strategy is synchronous and in-process; the expensive, failure-prone call is the remote operation the client wraps. Strategies here are stateless except for the random generator, so one instance can be shared by many clients. Adding a policy adds a class (or function) and touches nothing else.

Python implementation

In Python a one-method interface is just a callable, so strategies are functions.

```
        import random
        from typing import Callable

        Backoff = Callable[[int], float]      # attempt number (0-based) -> seconds to
wait


        def fixed(delay: float) -> Backoff:
            return lambda attempt: delay


        def exponential(base: float, cap: float) -> Backoff:
            return lambda attempt: min(cap, base * 2 ** attempt)


        def full_jitter(base: float, cap: float, rng: random.Random) -> Backoff:
            return lambda attempt: rng.uniform(0, min(cap, base * 2 ** attempt))


        class RetryingClient:
            def __init__(self, backoff: Backoff, max_attempts: int = 5):
                self.backoff, self.max_attempts = backoff, max_attempts

            def call(self, op, sleep=lambda s: None):
                waits = []
                for attempt in range(self.max_attempts):
                    try:
                        return op(), waits
                    except ConnectionError:
                        wait = self.backoff(attempt)
                        waits.append(round(wait, 3))
                        sleep(wait)                    # injected so the demo runs in-
stantly
                raise TimeoutError(f"gave up after {self.max_attempts} attempts; waits=
{waits}")


        def flaky(fail_times: int):
            state = {"n": 0}
            def op():
                state["n"] += 1
                if state["n"] <= fail_times:
                    raise ConnectionError("503")
                return "charged"
            return op


        if __name__ == "__main__":
            rng = random.Random(7)
            for name, policy in [("fixed", fixed(0.2)),
                                 ("exponential", exponential(0.1, 2.0)),
                                 ("full jitter", full_jitter(0.1, 2.0, rng))]:
                result, waits = RetryingClient(policy).call(flaky(fail_times=4))
                print(f"{name:12} {result} after waits {waits}")
```

Output:

```
fixed        charged after waits [0.2, 0.2, 0.2, 0.2]
exponential  charged after waits [0.1, 0.2, 0.4, 0.8]
full jitter  charged after waits [0.032, 0.03, 0.26, 0.058]
```

What to notice:

- The retry loop never changes; swapping a closure changes the policy.
- Closures capture configuration (`base`, `cap`, `rng`), which is all a strategy object's fields would do.
- Injecting `sleep` and a seeded `rng` makes time-dependent code deterministic in tests, a small taste of §37.14.

When to use and when not

Use Strategy when several algorithms solve the same problem and the choice depends on configuration, data or tenant (pricing rules, compression codecs, eviction policies, load-balancing algorithms). Do not use it for a choice that never varies, and do not build a class hierarchy in Python when a function argument suffices.

Related patterns

State (§37.15.1) has the same structure, but the state object changes itself; a strategy is chosen from outside. Template Method (§37.15.2) varies steps by inheritance; Strategy varies them by composition. Decorator (§37.10) adds behavior around a call; Strategy replaces the algorithm inside.

Interview question

> INTERVIEW TIP
>
> **Q.** How does Strategy relate to the Open/Closed Principle? **A.** The context is closed for modification and open for extension: new behavior arrives as a new strategy, not as another `elif` in the context. The choice of strategy moves to configuration or the composition root, and each strategy can be unit-tested in isolation.

37.8 Observer

Intent

DEFINITION

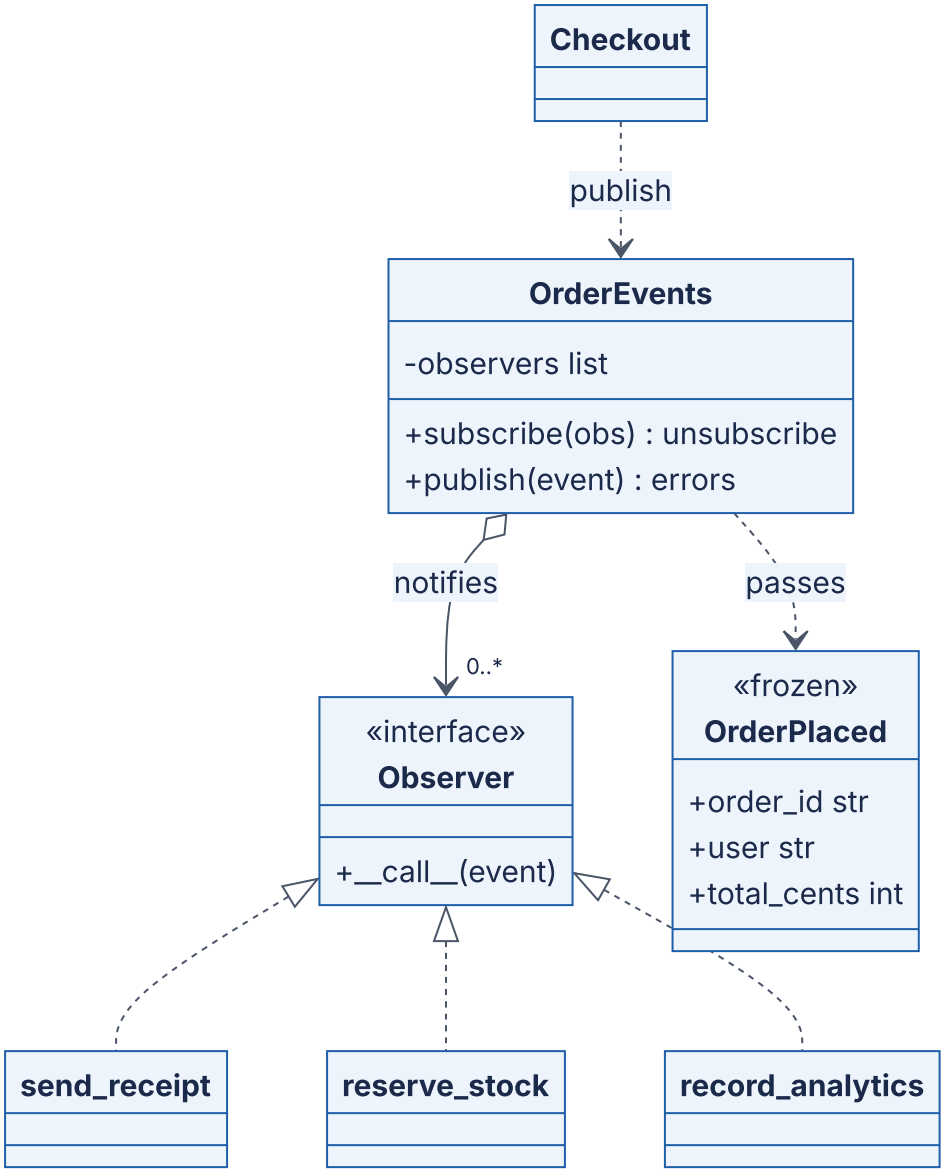
Definition — Observer. A one-to-many dependency in which a **subject** keeps a list of **observers** and notifies each of them when its state changes, so that the subject does not depend on what the observers do [Gamma et al. 1994].

Problem it solves

When an order is placed, the store must send a receipt, reserve stock and record analytics, and next quarter it will also award loyalty points. If `place_order()` calls each of these directly, the order module imports every other module, and adding loyalty means editing checkout. With Observer, checkout announces "an order was placed" and each interested module subscribes. GUI toolkits (button click listeners), Java's `PropertyChangeListener`, and Kubernetes client informers with registered event handlers all follow this shape.

Structure

Figure 37.7 — Observer for order events.



Reading the diagram. `Checkout` publishes an immutable `OrderPlaced` event to the subject, `OrderEvents`, which holds zero or more observers and calls each in turn. In the classic pattern every call is synchronous and in-process, so checkout's latency includes every observer's latency and an observer that blocks blocks checkout. The subscriber list is in-memory state; it is lost on restart, and so is any event being delivered when the process crashes. Failure points are a throwing observer (must be isolated) and a slow one (must be moved off the request path). Scaling beyond one process is where the pattern becomes publish/subscribe through a broker.

Python implementation

```
        from dataclasses import dataclass
        from typing import Callable


        @dataclass(frozen=True)
        class OrderPlaced:
            order_id: str
            user: str
            total_cents: int


        Observer = Callable[[OrderPlaced], None]


        class OrderEvents:                                  # the subject
            def __init__(self):
                self._observers: list[Observer] = []

            def subscribe(self, obs: Observer) -> Callable[[], None]:
                self._observers.append(obs)
                return lambda: self._observers.remove(obs)   # caller keeps the unsub-
scribe handle

            def publish(self, event: OrderPlaced) -> list[str]:
                errors = []
                for obs in list(self._observers):            # copy: observers may un-
subscribe
                    try:
                        obs(event)
                    except Exception as e:                   # isolate a failing ob-
server
                        errors.append(f"{obs.__name__}: {e}")
                return errors


        log = []
        def send_receipt(e):     log.append(f"receipt to {e.user}")
        def record_analytics(e): log.append(f"analytics +{e.total_cents}")
        def reserve_stock(e):
            if e.order_id == "o-2":
                raise RuntimeError("warehouse timeout")
            log.append(f"stock reserved for {e.order_id}")


        if __name__ == "__main__":
            events = OrderEvents()
            events.subscribe(send_receipt)
            events.subscribe(reserve_stock)
            stop_analytics = events.subscribe(record_analytics)
            print(events.publish(OrderPlaced("o-1", "ana", 4999)), log)
            log.clear(); stop_analytics()
            print(events.publish(OrderPlaced("o-2", "ben", 1999)), log)
```

Output:

```
[] ['receipt to ana', 'stock reserved for o-1', 'analytics +4999']
['reserve_stock: warehouse timeout'] ['receipt to ben']
```

What to notice:

- A failing observer is reported but does not stop the receipt from being sent; without the `try`, one bug in analytics would break checkout.
- `subscribe` returns an unsubscribe handle. Forgotten subscriptions are a classic memory leak (the "lapsed listener" problem): the subject keeps dead observers alive. `weakref.WeakMethod` is the Python remedy for bound methods.
- Observers run in subscription order and synchronously. Code must not depend on that order.

Observer and publish/subscribe

Observer is publish/subscribe inside one process. Chapter 30 (§30.5) takes the idea across the network.

Table 37.1 — Observer versus broker-based publish/subscribe.

Aspect	Observer (in-process)	Publish/subscribe (§30.5)
Coupling	Subject holds observer references	Publisher knows only a topic
Timing	Synchronous, same call stack	Asynchronous; subscribers may be offline
Durability	None; lost on crash	Broker persists messages
Failure isolation	Manual (`try` per observer)	Process boundary; retries, dead-letter queues (§30.11)
Latency	Microseconds	Milliseconds
Choose when	Same deployable, same transaction	Different services, independent scaling

A common production arrangement uses both: an in-process observer writes the event to an outbox table in the same database transaction, and a relay publishes it to the broker (the transactional outbox, §32.5).

When to use and when not

Use Observer when one change must trigger an open-ended set of reactions inside a process (UI updates, cache invalidation, domain events). Do not use it when the reactions must be atomic with the change and failure must roll it back (call them directly in the transaction), or when observers live in other services (use a broker).

Related patterns

Mediator centralizes many-to-many communication; Observer distributes one-to-many. Command (§37.12) objects are often what gets queued when observers become asynchronous. Publish/subscribe (§30.5) and event-driven architecture (§30.6) are the distributed forms.

Interview question

> INTERVIEW TIP
>
> **Q.** A slow analytics observer is adding 300 ms to checkout. What do you do? **A.** Take it off the request path: have the observer enqueue the event (an in-process queue drained by a worker, or a broker topic) and return immediately. If the event must not be lost, write it to an outbox in the order's transaction. Then decide the delivery guarantee (at-least-once plus an idempotent consumer is typical, §30.8).

37.9 Adapter

Intent

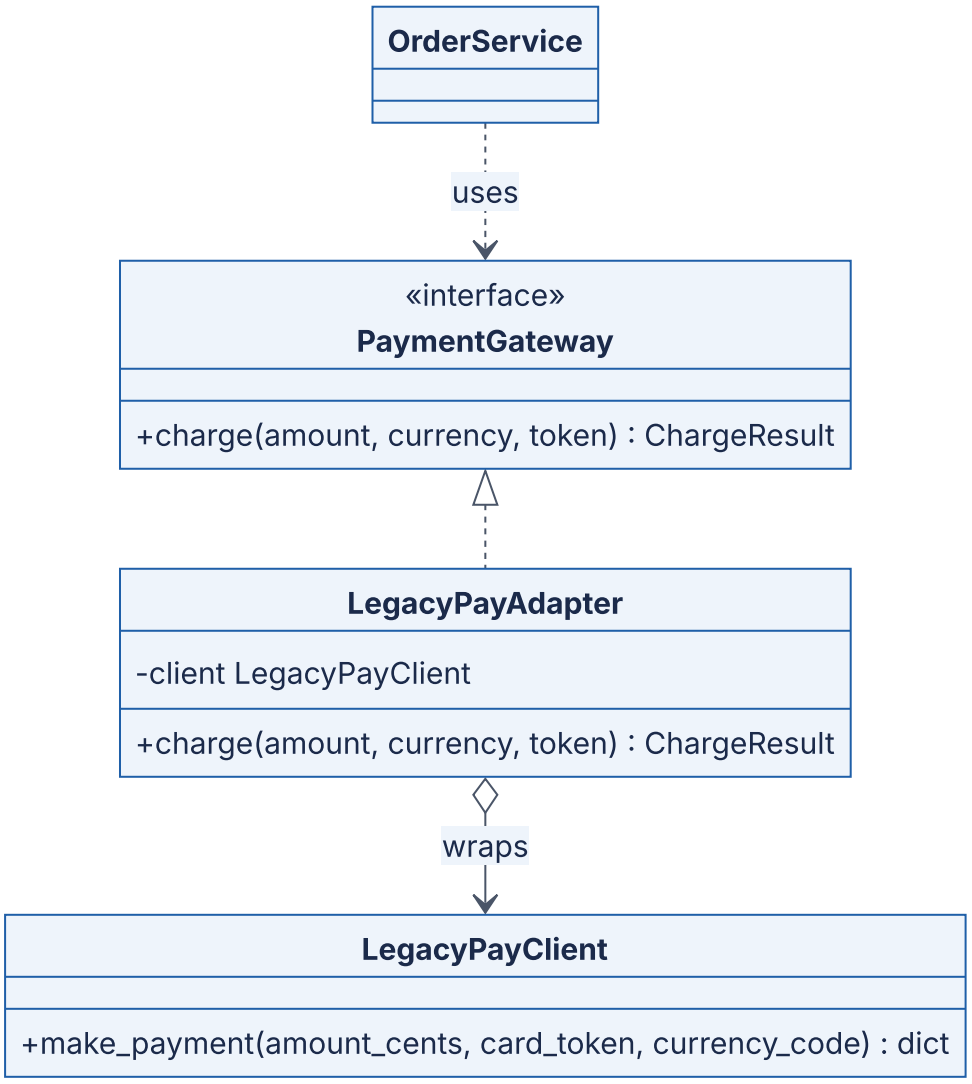
> DEFINITION
>
> **Definition — Adapter.** A wrapper that converts the interface of an existing class into the interface its clients expect, so that classes with incompatible interfaces can work together [Gamma et al. 1994].

Problem it solves

The order service defines its own `PaymentGateway.charge(amount, currency, token)` port, returning a `ChargeResult`. The acquiring bank's SDK, which you cannot modify, exposes `make_payment(amount_cents, card_token, currency_code)` and returns a dictionary of status codes. An adapter translates units (decimal to integer cents), argument order and error conventions in one class, so neither side changes. SLF4J's bindings, which route one logging API to Logback, Log4j or `java.util.logging`, are adapters, as are JDBC drivers that present each database's wire protocol through one Java interface.

Structure

Figure 37.8 — Object adapter for a legacy payment SDK.



Reading the diagram. `OrderService` depends on its own interface (the **target**). `LegacyPayAdapter` implements the target and holds the third-party **adaptee**, translating every call. Calls are synchronous; the adaptee call crosses the network to the payment processor, so it is the failure point (timeouts, declines, malformed responses), and the adapter is where vendor error codes become your domain's results. The adapter holds no state of its own. Swapping vendors means writing a new adapter, not editing the service.

Python implementation

```
        from abc import ABC, abstractmethod
        from dataclasses import dataclass
        from decimal import Decimal


        class LegacyPayClient:                        # third-party SDK: cannot be
changed
            def make_payment(self, amount_cents, card_token, currency_code):
                if card_token.startswith("tok_bad"):
                    return {"status_code": 402, "err": "card_declined"}
                return {"status_code": 200, "txn": f"T-{amount_cents}-{currency_code}"}


        @dataclass(frozen=True)
        class ChargeResult:
            ok: bool
            reference: str


        class PaymentGateway(ABC):                    # our port (the target interface)
            @abstractmethod
            def charge(self, amount: Decimal, currency: str, token: str) ->
ChargeResult: ...


        class LegacyPayAdapter(PaymentGateway):
            def __init__(self, client: LegacyPayClient):
                self._client = client

            def charge(self, amount, currency, token):
                cents = int((amount * 100).to_integral_value())     # Decimal -> integer
cents
                resp = self._client.make_payment(cents, token, currency.upper())
                if resp["status_code"] == 200:
                    return ChargeResult(True, resp["txn"])
                return ChargeResult(False, resp.get("err", f"http
{resp['status_code']}"))


        if __name__ == "__main__":
            gateway: PaymentGateway = LegacyPayAdapter(LegacyPayClient())
            print(gateway.charge(Decimal("49.99"), "usd", "tok_visa"))
            print(gateway.charge(Decimal("19.99"), "eur", "tok_bad_cvc"))
```

Output:

```
ChargeResult(ok=True, reference='T-4999-USD')
ChargeResult(ok=False, reference='card_declined')
```

What to notice:

- All translation lives in one place: unit conversion, argument order, casing, and error codes become domain values.
- Money uses `Decimal`; converting a binary float like `49.99` to cents can be off by one.
- Tests of `OrderService` use a fake `PaymentGateway`; only the adapter's own tests touch the SDK.

When to use and when not

Use an adapter at every boundary with code you do not own (vendor SDKs, legacy modules, another team's API) and when migrating between implementations. Do not use one to paper over your own inconsistent interfaces; fix those instead. An adapter that grows business logic has become a service and should be named and tested as one.

Related patterns

Decorator (§37.10) keeps the interface and adds behavior; Adapter changes the interface. Facade simplifies a whole subsystem behind a new, smaller interface. Hexagonal architecture (§33.10) is named "ports and adapters" after this pattern, and the anti-corruption layer of domain-driven design (§34.5) is an adapter at a bounded-context boundary.

Interview question

INTERVIEW TIP

Q. What is the difference between an object adapter and a class adapter? **A.** An object adapter holds an instance of the adaptee and delegates (composition); it works with any subclass of the adaptee and is the norm in Python and Java. A class adapter inherits from both the target and the adaptee (multiple inheritance in C++, or extending the adaptee class while implementing the target interface in Java); it can override adaptee behavior but couples you to one concrete class.

37.10 Decorator

Intent

DEFINITION

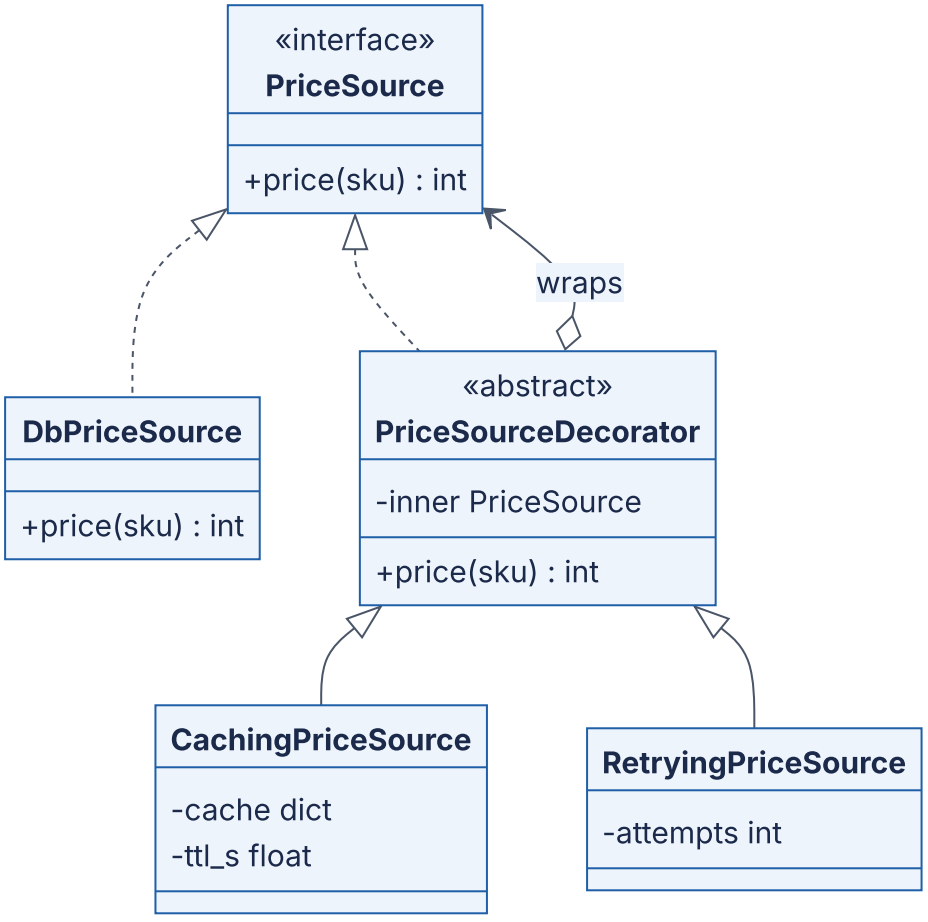
Definition — Decorator (GoF). An object that implements the same interface as the object it wraps, forwards calls to it, and adds behavior before or after them, so responsibilities can be added at run time and stacked in any order without subclassing [Gamma et al. 1994].

Problem it solves

The order service reads prices from a database-backed `PriceSource`. Some deployments need a cache, all need retries on transient errors, and some need metrics. Subclassing gives a class per combination (`CachingRetryingMetricsDbPriceSource`): with three optional behaviors that is $2^3 = 8$ classes. Decorators give three small wrappers that compose. Java's I/O library is the textbook case: `new BufferedInputStream(new GZIPInputStream(new FileInputStream(f)))` stacks buffering and decompression on a file stream, and Python's `io` module layers a `TextIOWrapper` over a `BufferedReader` over a raw `FileIO` in the same way.

Structure

Figure 37.9 — Decorators stacked around a price source.



Reading the diagram. Every decorator *is a* `PriceSource` and *has a* `PriceSource`; that double relationship is what lets wrappers nest to any depth. A call enters the outermost decorator and flows inward to `DbPriceSource`, the only component that crosses the storage boundary; results flow back out. Everything is synchronous. The cache holds in-process state with a TTL (stale reads are the trade-off, §20.8). Failure points: the database call, which the retry wrapper absorbs for transient errors, and the stacking order,

which changes behavior (a cache outside the retry hides retries on hits; inside, it does not).

Python implementation

```
        import functools
        import time
        from abc import ABC, abstractmethod


        class PriceSource(ABC):
            @abstractmethod
            def price(self, sku: str) -> int: ...


        class DbPriceSource(PriceSource):
            def __init__(self, failures_before_success=1):
                self.calls, self._fail = 0, failures_before_success
            def price(self, sku):
                self.calls += 1
                if self._fail:
                    self._fail -= 1
                    raise ConnectionError("db connection reset")
                return {"A1": 1999, "B2": 4999}[sku]


        class CachingPriceSource(PriceSource):            # GoF decorator 1
            def __init__(self, inner, ttl_s=60.0, clock=time.monotonic):
                self._inner, self._ttl, self._clock, self._cache = inner, ttl_s, clock,
{}
            def price(self, sku):
                hit = self._cache.get(sku)
                if hit and self._clock() - hit[1] < self._ttl:
                    return hit[0]
                value = self._inner.price(sku)
                self._cache[sku] = (value, self._clock())
                return value


        class RetryingPriceSource(PriceSource):           # GoF decorator 2
            def __init__(self, inner, attempts=3):
                self._inner, self._attempts = inner, attempts
            def price(self, sku):
                for i in range(self._attempts):
                    try:
                        return self._inner.price(sku)
                    except ConnectionError:
                        if i == self._attempts - 1:
                            raise


        # Python's @decorator syntax: wraps a *function* at definition time.
        def timed(fn):
            @functools.wraps(fn)                          # keep name, docstring
            def wrapper(*args, **kwargs):
                start = time.perf_counter()
                try:
                    return fn(*args, **kwargs)
                finally:
```

```
                    wrapper.last_ms = (time.perf_counter() - start) * 1000
            return wrapper


        @timed
        def quote(source: PriceSource, sku: str, qty: int) -> int:
            return source.price(sku) * qty


        if __name__ == "__main__":
            db = DbPriceSource()
            source = CachingPriceSource(RetryingPriceSource(db))     # stack at run time
            print(quote(source, "B2", 2), quote(source, "B2", 1), "db calls:", db.calls)
            print(quote.__name__, f"{quote.last_ms:.3f} ms" if quote.last_ms < 5 else
"slow")
```

Output (the time varies by machine):

```
9998 4999 db calls: 2
quote 0.002 ms
```

What to notice:

- Two database calls: the first failed and was retried; the second quote was a cache hit.
- The stack is built at run time from configuration; no class names the combination.
- `@timed` is Python syntax for `quote = timed(quote)`. `functools.wraps` copies `__name__` and `__doc__`, without which logs and tracebacks name every function `wrapper`.

> WARNING / TRADE-OFF
>
> **Python decorators are not the GoF Decorator.** The GoF pattern wraps an *object* at run time behind the *same interface*. Python's `@decorator` (PEP 318) is syntax that passes a function or class to another callable at *definition time* and rebinds the name to whatever it returns [PEP 318]. It can implement a GoF-style wrapper for a function (`@functools.lru_cache`, a retry decorator), but it is equally used for registration (`@app.route("/orders")` adds a URL mapping and returns the function unchanged), for `@property`, and for class transformations (`@dataclass`), none of which is the GoF pattern. Ask "does this wrap an object and keep its interface?"

When to use and when not

Use decorators for cross-cutting concerns that you want to combine freely: caching, retries, metrics, logging, authorization, compression, rate limiting. Do not use them when the wrapped interface is wide (every method must be forwarded; Python's `__getattr__`

helps but hides the contract), when order-dependent stacks become hard to reason about, or when identity matters (`wrapped is not inner`).

Related patterns

Proxy (§37.11) has the same structure but controls access rather than adding features. Adapter (§37.9) changes the interface. Chain of Responsibility (§37.15.3) is a decorator stack expressed as a pipeline; HTTP middleware is both.

Interview question

> INTERVIEW TIP
>
> **Q.** Design a client library that supports optional caching, retries and circuit breaking. **A.** Define one client interface; implement the network client; implement each concern as a decorator of that interface; build the stack in a factory from configuration. Document the order: retry inside the circuit breaker so an open circuit stops retries (§22.13), cache outermost so hits avoid both.

37.11 Proxy

Intent

> DEFINITION
>
> **Definition — Proxy.** A surrogate that implements the same interface as a real subject and controls access to it [Gamma et al. 1994]. A **virtual proxy** delays creating an expensive object until first use; a **protection proxy** checks permissions before forwarding; a **remote proxy** (a stub) makes an object in another address space look local.

Problem it solves

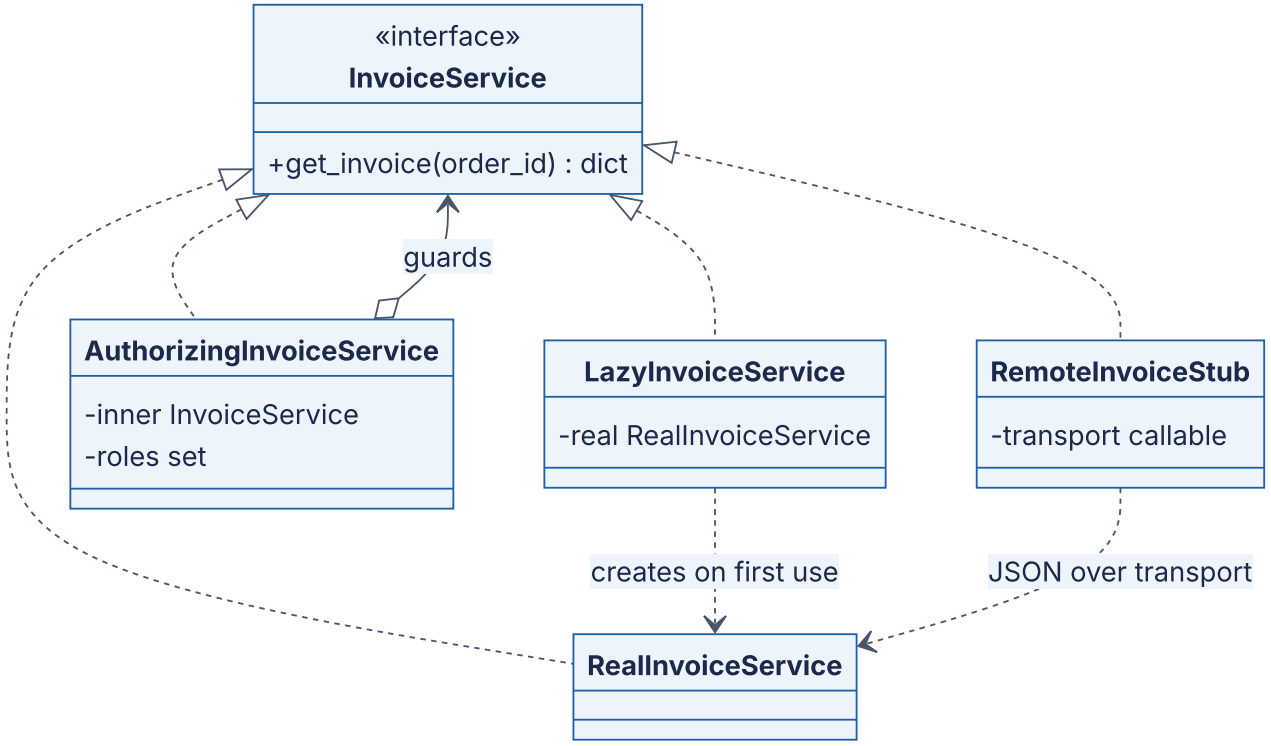
The invoice service is expensive to start (it loads templates and opens a connection pool) but most requests never need it; a virtual proxy defers that cost. Only users with the billing role may read invoices; a protection proxy enforces that at one choke point. In a microservice deployment the invoice service runs elsewhere; a remote proxy (a gRPC or REST client stub) lets callers write `invoices.get_invoice("o-17")`.

INDUSTRY EXAMPLE

Spring Framework transactions. Spring applies `@Transactional` and other aspects by wrapping a bean in a proxy, a JDK dynamic proxy when the bean implements an interface and a generated subclass otherwise. The proxy begins the transaction, calls the real method and commits or rolls back. The documented trade-off: a call from one method of the bean to another on `this` bypasses the proxy, so the annotation on the inner method has no effect. *Evidence: Primary* — [Spring Docs]

Structure

Figure 37.10 — Virtual, protection and remote proxies.



Reading the diagram. All four classes share one interface, so a client cannot tell which it holds. The virtual proxy creates the real subject on first call and caches it. The protection proxy wraps any `InvoiceService` and refuses calls that fail its check. The remote stub serializes the call, sends it over a transport (HTTP or gRPC in production; a function here), and deserializes the reply; that edge crosses a process and network boundary, so it adds latency (~0.5 ms in a datacenter at best) and new failure modes (timeouts, partial failure, retries). Scaling the remote side happens behind the stub (load balancing, §19).

Python implementation

```
        import json
        from abc import ABC, abstractmethod


        class InvoiceService(ABC):
            @abstractmethod
            def get_invoice(self, order_id: str) -> dict: ...


        class RealInvoiceService(InvoiceService):
            def __init__(self):
                print("  (expensive init: templates loaded, DB pool opened)")
                self._db = {"o-17": {"order_id": "o-17", "total_cents": 4999}}
            def get_invoice(self, order_id):
                return self._db[order_id]


        class LazyInvoiceService(InvoiceService):           # virtual proxy
            def __init__(self):
                self._real = None
            def get_invoice(self, order_id):
                if self._real is None:
                    self._real = RealInvoiceService()
                return self._real.get_invoice(order_id)


        class AuthorizingInvoiceService(InvoiceService):    # protection proxy
            def __init__(self, inner: InvoiceService, roles: set[str]):
                self._inner, self._roles = inner, roles
            def get_invoice(self, order_id):
                if "billing" not in self._roles:
                    raise PermissionError("billing role required")
                return self._inner.get_invoice(order_id)


        class RemoteInvoiceStub(InvoiceService):            # remote proxy (client
stub)
            def __init__(self, transport):
                self._send = transport
            def get_invoice(self, order_id):
                reply = json.loads(self._send(json.dumps(
                    {"method": "get_invoice", "args": [order_id]})))
                if "error" in reply:
                    raise RuntimeError(reply["error"])
                return reply["result"]


        def make_server(impl: InvoiceService):              # server skeleton
            allowed = {"get_invoice"}                       # never dispatch arbitrary
names
            def handle(wire: str) -> str:
                req = json.loads(wire)
                try:
                    if req["method"] not in allowed:
```

```
                        raise ValueError("no such method")
                    return json.dumps({"result": getattr(impl, req["method"])
(*req["args"])})
                except Exception as e:
                    return json.dumps({"error": repr(e)})
            return handle


        if __name__ == "__main__":
            lazy = LazyInvoiceService()
            print("proxy created; nothing loaded yet")
            print(lazy.get_invoice("o-17"))
            try:
                AuthorizingInvoiceService(lazy, {"support"}).get_invoice("o-17")
            except PermissionError as e:
                print("denied:", e)
            stub = RemoteInvoiceStub(make_server(lazy))
            print(stub.get_invoice("o-17"))
            try:
                stub.get_invoice("o-99")
            except RuntimeError as e:
                print("remote error:", e)
```

Output:

```
proxy created; nothing loaded yet
  (expensive init: templates loaded, DB pool opened)
{'order_id': 'o-17', 'total_cents': 4999}
denied: billing role required
{'order_id': 'o-17', 'total_cents': 4999}
remote error: KeyError('o-99')
```

What to notice:

- The expensive initialization ran once, on first use, not at construction.
- The remote stub changes failure semantics: a `KeyError` on the server becomes a `RuntimeError` carrying a string. Remote proxies cannot fully hide the network, which is the first fallacy of distributed computing (§24.5).
- The skeleton dispatches only allowed method names. Dispatching any name from the wire is a remote-code-execution hole.

Java sketch

Java can generate proxies at run time for any interface with `java.lang.reflect.Proxy`, which is how frameworks add transactions, security checks and tracing without hand-written wrapper classes:

```
        import java.lang.reflect.Proxy;

        interface Invoices { String get(String orderId); }

        class Proxies {
            @SuppressWarnings("unchecked")
            static <T> T timed(T target, Class<T> iface) {
                return (T) Proxy.newProxyInstance(iface.getClassLoader(), new Class<?>[]
{iface},
                    (proxy, method, args) -> {
                        long t0 = System.nanoTime();
                        try { return method.invoke(target, args); }
                        finally { System.out.printf("%s took %d us%n", method.getName(),
                                                    (System.nanoTime() - t0) / 1000); }
                    });
            }
        }
        // Invoices inv = Proxies.timed(orderId -> "invoice " + orderId,
Invoices.class);
```

Proxies at the system level

The same three roles reappear as infrastructure. A reverse proxy (§7.12) and an API gateway (§34.6) are protection proxies for whole services: they authenticate, authorize and rate-limit before forwarding. A service-mesh sidecar such as Envoy (§34.10) is a remote proxy deployed next to each service: the application talks to `localhost`, and the sidecar handles discovery, TLS, retries and telemetry. Content delivery networks (Chapter 21) act as caching proxies.

Table 37.2 — Four wrappers that look alike.

Pattern	Interface seen by client	Purpose	Typical example
Adapter	Different from the wrapped object	Make incompatible interfaces fit	Vendor SDK behind your port
Decorator	Same	Add behavior, stackable	Caching, retries, buffering
Proxy	Same	Control access: lazy, permission, remote	Lazy entity, auth check, RPC stub
Facade	New, simpler	Hide a subsystem	One `checkout()` over five services

When to use and when not

Use a proxy when access needs control: deferred creation of something expensive, a permission check at one point, a remote object, or a smart reference that counts or logs. Avoid remote proxies that make chatty remote calls look like cheap local ones; design coarse-grained remote interfaces instead.

Related patterns

Decorator (§37.10) is structurally identical; the difference is intent. Adapter changes the interface. API gateways and sidecars are proxies at deployment scale.

Interview question

> INTERVIEW TIP
>
> **Q.** Why are lazy-loading ORM proxies a common source of performance bugs? **A.** Each proxy triggers a query on first access, so iterating 100 orders and touching each order's lazily loaded customer issues 1 + 100 queries (the N+1 problem). The proxy hides the I/O. Fix it with eager fetching or a join for that query, and detect it by counting queries per request in tests.

37.12 Command

Intent

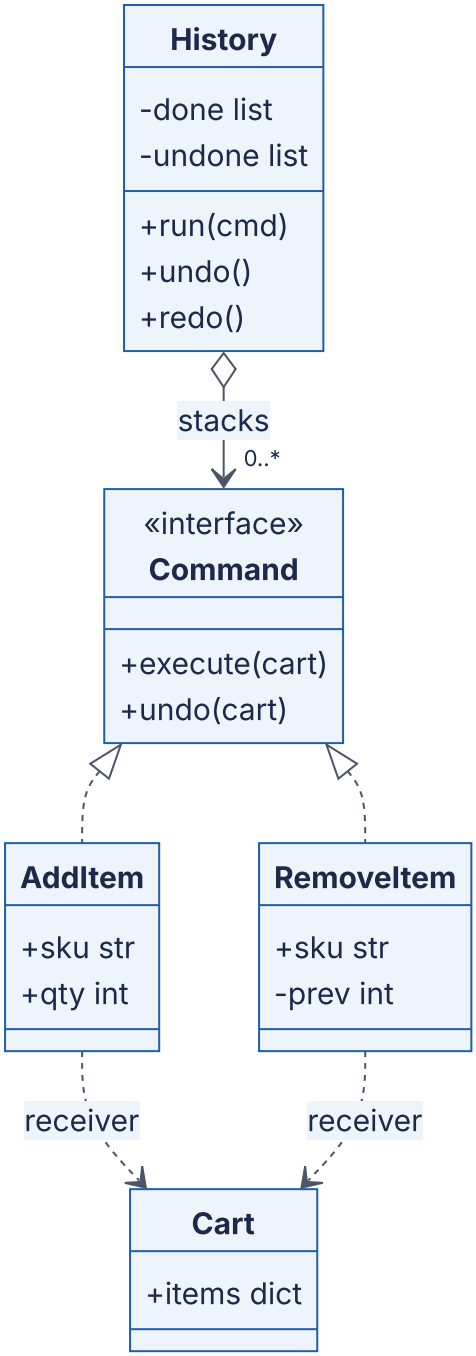
> DEFINITION
>
> **Definition — Command.** An object that encapsulates a request (the operation and its arguments) so that it can be stored, passed around, queued, logged, replayed and undone [Gamma et al. 1994]. In this book's terminology a command is a request to do something, as opposed to an event, a fact that has happened.

Problem it solves

A shopping cart editor needs undo and redo. A background job system needs to put "send receipt for order 17" on a queue and run it later on another machine. An audit log needs a durable record of every requested change. All three need the request itself to be a value, not just a method call that vanishes once made. Java's `Runnable` and `Callable` passed to an executor, Celery tasks, and the operations in a write-ahead log are commands.

Structure

Figure 37.11 — Command with undo/redo history.



Reading the diagram. The **invoker** (`History`) runs commands and keeps two stacks. Each concrete command knows its **receiver** (`Cart`) and how to reverse itself; `RemoveItem` stores the quantity it removed so that undo can restore it. Calls are synchronous here, but because a command is a value it can cross an asynchronous boundary: serialize it, put it on a queue, execute it on a worker. State lives in the receiver and, for undo, in the history stacks; those stacks are in memory unless you persist them. Failure points: a command whose undo is not the exact inverse, and a queued command executed twice (make handlers idempotent, §32.10).

Python implementation

```
        import json
        from abc import ABC, abstractmethod
        from dataclasses import asdict, dataclass, field


        class Cart:
            def __init__(self):
                self.items: dict[str, int] = {}


        class Command(ABC):
            @abstractmethod
            def execute(self, cart: Cart) -> None: ...
            @abstractmethod
            def undo(self, cart: Cart) -> None: ...


        @dataclass
        class AddItem(Command):
            sku: str
            qty: int
            def execute(self, cart):
                cart.items[self.sku] = cart.items.get(self.sku, 0) + self.qty
            def undo(self, cart):
                left = cart.items[self.sku] - self.qty
                if left:
                    cart.items[self.sku] = left
                else:
                    del cart.items[self.sku]


        @dataclass
        class RemoveItem(Command):
            sku: str
            _prev: int = field(default=0, repr=False)   # captured for undo
            def execute(self, cart):
                self._prev = cart.items.pop(self.sku, 0)
            def undo(self, cart):
                if self._prev:
                    cart.items[self.sku] = self._prev


        class History:                                  # the invoker
            def __init__(self, cart):
                self.cart, self._done, self._undone = cart, [], []
            def run(self, cmd):
                cmd.execute(self.cart); self._done.append(cmd); self._undone.clear()
            def undo(self):
                if self._done:
                    cmd = self._done.pop(); cmd.undo(self.cart);
self._undone.append(cmd)
            def redo(self):
                if self._undone:
                    cmd = self._undone.pop(); cmd.execute(self.cart);
```

```
self._done.append(cmd)


        TYPES = {"AddItem": AddItem, "RemoveItem": RemoveItem}

        def to_message(cmd: Command) -> str:            # commands are values: serial-
ize them
            data = {k: v for k, v in asdict(cmd).items() if not k.startswith("_")}
            return json.dumps({"type": type(cmd).__name__, **data})

        def from_message(text: str) -> Command:
            data = json.loads(text)
            return TYPES[data.pop("type")](**data)


        if __name__ == "__main__":
            h = History(Cart())
            for cmd in (AddItem("A1", 2), AddItem("B2", 1), RemoveItem("A1")):
                h.run(cmd)
            print("after 3 commands:", h.cart.items)
            h.undo(); h.undo()
            print("after 2 undos:   ", h.cart.items)
            h.redo()
            print("after redo:      ", h.cart.items)

            queue = [to_message(c) for c in (AddItem("A1", 2), RemoveItem("A1"),
AddItem("C3", 5))]
            print("queued:", queue[0])
            replica = Cart()
            for msg in queue:                           # a worker replays the queue
                from_message(msg).execute(replica)
            print("replayed cart:   ", replica.items)
```

Output:

```
after 3 commands: {'B2': 1}
after 2 undos:    {'A1': 2}
after redo:       {'A1': 2, 'B2': 1}
queued: {"type": "AddItem", "sku": "A1", "qty": 2}
replayed cart:    {'C3': 5}
```

What to notice:

- Running a new command clears the redo stack; otherwise redo would reapply a change on top of a different history.
- `RemoveItem` captures the state it destroys (a small memento) so undo is exact.
- Serialized commands are messages: a queue of them is a job queue, and a durable, ordered log of them is a replay log, the core idea behind write-ahead logs and, with events instead of commands, event sourcing (§32.3).

When to use and when not

Use Command for undo/redo, job queues, scheduled or retried work, macro recording, transactional scripts that must be logged before execution, and any API where the caller and executor are separated in time or space. In Python, a closure or `functools.partial` is enough when you only need deferred execution; reach for command classes when you need undo, serialization or introspection.

Related patterns

Memento stores the state that undo restores. Strategy (§37.7) varies *how* something is done; Command captures *what* to do. Producer–consumer (§37.16.1) and message queues (§30.4) transport commands. CQRS (§32.4) separates command handling from queries.

Interview question

> INTERVIEW TIP
>
> **Q.** How would you implement undo for operations that call external systems, such as "send email"? **A.** You cannot unsend an email, so undo must be a compensating action (send a correction) or the operation must be delayed until undo is no longer possible (an "undo send" window that queues the command and executes it after 10 s). The same reasoning drives compensating transactions in sagas (§32.7).

37.13 Repository

Intent

> DEFINITION
>
> **Definition — Repository.** An object that mediates between the domain model and data storage, presenting stored aggregates as an in-memory collection with methods such as `add`, `get` and domain-specific finders, so that domain code never sees SQL, ORM sessions or storage APIs [Fowler 2002; Evans 2003].

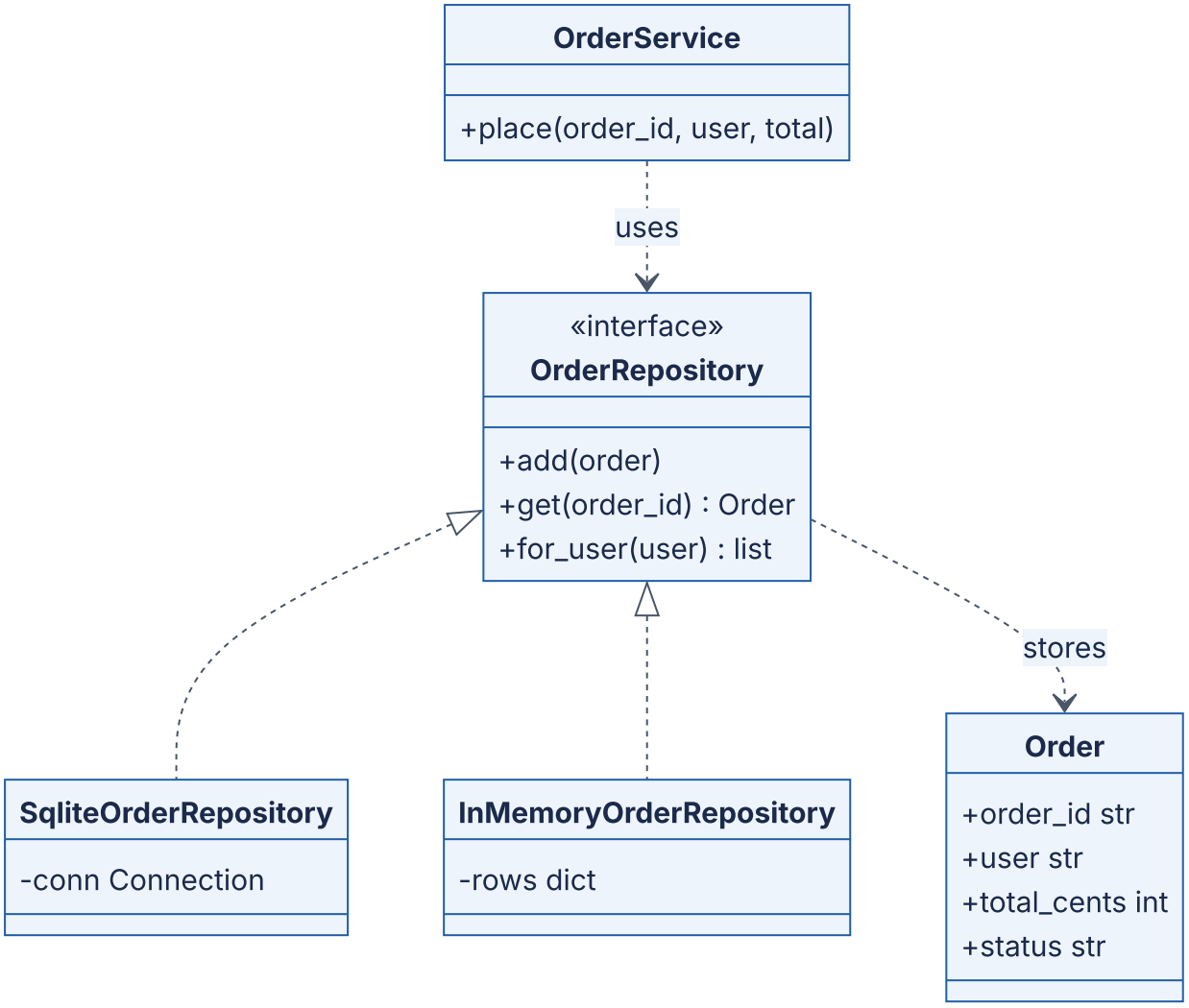
Problem it solves

If `OrderService.place()` builds SQL strings, then every business rule test needs a database, the service cannot move from PostgreSQL to DynamoDB without rewriting business logic, and query code is scattered. A repository gives the domain one interface (`OrderRepository`) expressed in domain terms ("orders for this user"), with a database implementation for production and an in-memory one for fast tests. Spring Data generates

repository implementations from interfaces such as `interface OrderRepository extends CrudRepository<Order, Long>` [Spring Data Docs].

Structure

Figure 37.12 — Repository with interchangeable storage.



Reading the diagram. `OrderService` depends on the `OrderRepository` interface and on the domain object `Order`, nothing else. Two implementations sit behind the interface: the SQLite one crosses the storage boundary (SQL over a database connection; in production a network hop to a database server, where timeouts and constraint violations occur) and the in-memory one keeps rows in a dictionary for tests. Calls are synchronous. Scaling concerns such as read replicas, partitioning (Chapter 14) and caching can be hidden inside an implementation without changing the service.

Python implementation

```
        import sqlite3
        from abc import ABC, abstractmethod
        from dataclasses import dataclass


        @dataclass(frozen=True)
        class Order:
            order_id: str
            user: str
            total_cents: int
            status: str = "PENDING"


        class OrderRepository(ABC):
            @abstractmethod
            def add(self, order: Order) -> None: ...
            @abstractmethod
            def get(self, order_id: str) -> Order | None: ...
            @abstractmethod
            def for_user(self, user: str) -> list[Order]: ...


        class InMemoryOrderRepository(OrderRepository):
            def __init__(self): self._rows: dict[str, Order] = {}
            def add(self, order): self._rows[order.order_id] = order
            def get(self, order_id): return self._rows.get(order_id)
            def for_user(self, user):
                return sorted((o for o in self._rows.values() if o.user == user),
                              key=lambda o: o.order_id)


        class SqliteOrderRepository(OrderRepository):
            def __init__(self, conn: sqlite3.Connection):
                self._conn = conn
                conn.execute("CREATE TABLE IF NOT EXISTS orders (order_id TEXT PRIMARY
KEY, "
                             "user TEXT NOT NULL, total_cents INTEGER NOT NULL, status
TEXT NOT NULL)")
                conn.execute("CREATE INDEX IF NOT EXISTS orders_user ON orders(user)")
            def add(self, order):
                with self._conn:                                    # one transaction
                    self._conn.execute("INSERT OR REPLACE INTO orders VALUES (?, ?, ?,
?)",
                                       (order.order_id, order.user, order.total_cents,
order.status))
            def get(self, order_id):
                row = self._conn.execute("SELECT * FROM orders WHERE order_id = ?",
                                         (order_id,)).fetchone()
                return Order(*row) if row else None
            def for_user(self, user):
                rows = self._conn.execute("SELECT * FROM orders WHERE user = ? ORDER BY
order_id",
                                          (user,)).fetchall()
                return [Order(*r) for r in rows]
```

```
        class OrderService:
            def __init__(self, repo: OrderRepository):
                self._repo = repo
            def place(self, order_id, user, total_cents):
                if total_cents <= 0:
                    raise ValueError("total must be positive")
                if self._repo.get(order_id):
                    return "duplicate"                                  # idempotent retry
                self._repo.add(Order(order_id, user, total_cents))
                return "placed"


        def contract_test(repo: OrderRepository) -> str:                # same test, every
implementation
            svc = OrderService(repo)
            assert svc.place("o-1", "ana", 4999) == "placed"
            assert svc.place("o-1", "ana", 4999) == "duplicate"
            svc.place("o-2", "ana", 1999); svc.place("o-3", "ben", 500)
            assert [o.order_id for o in repo.for_user("ana")] == ["o-1", "o-2"]
            assert repo.get("o-9") is None
            return f"{type(repo).__name__}: contract passed"


        if __name__ == "__main__":
            print(contract_test(InMemoryOrderRepository()))
            print(contract_test(SqliteOrderRepository(sqlite3.connect(":memory:"))))
```

Output:

```
InMemoryOrderRepository: contract passed
SqliteOrderRepository: contract passed
```

What to notice:

- `OrderService` contains the business rules and no SQL; it runs against either implementation.
- One **contract test** runs against every implementation. This is what keeps the in-memory fake honest; a fake that drifts from the real repository makes tests pass while production fails.
- Queries use placeholders (`?`), never string formatting, which prevents SQL injection (Chapter 10).

When to use and when not

Use a repository per **aggregate** (a cluster of objects changed together, such as an order and its lines, §34.4) when domain logic is rich enough to be worth testing without a database. Avoid a generic `Repository<T>` with `find_by_anything(**filters)`: it leaks query concerns back into the domain and becomes a weak ORM. For reporting and read-heavy

screens, query the database directly or use a separate read model (CQRS, §32.4); forcing analytics through aggregate repositories causes N+1 queries.

Related patterns

Unit of Work tracks changes across repositories and commits them in one transaction [Fowler 2002]. Data Mapper and Active Record are the alternatives Fowler catalogs for mapping objects to tables. Adapter (§37.9): a repository implementation is an adapter from a domain port to a storage technology. Dependency injection (§37.14) supplies the right implementation.

Interview question

INTERVIEW TIP

Q. Is a repository just a DAO? **A.** A data access object is organized around tables and storage operations; a repository is organized around domain aggregates and speaks the domain's language (`orders.for_user(ana)` rather than `select(table="orders", where=...)`). In small systems they converge. The distinction matters when one aggregate spans several tables or stores: the repository hides that, a DAO usually does not.

37.14 Dependency injection

Intent

DEFINITION

Definition — Dependency injection (DI). A technique in which an object receives the objects it depends on (its dependencies) from outside, typically through its constructor, instead of creating or looking them up itself. The code that creates and connects all objects is the **composition root** [Fowler 2004].

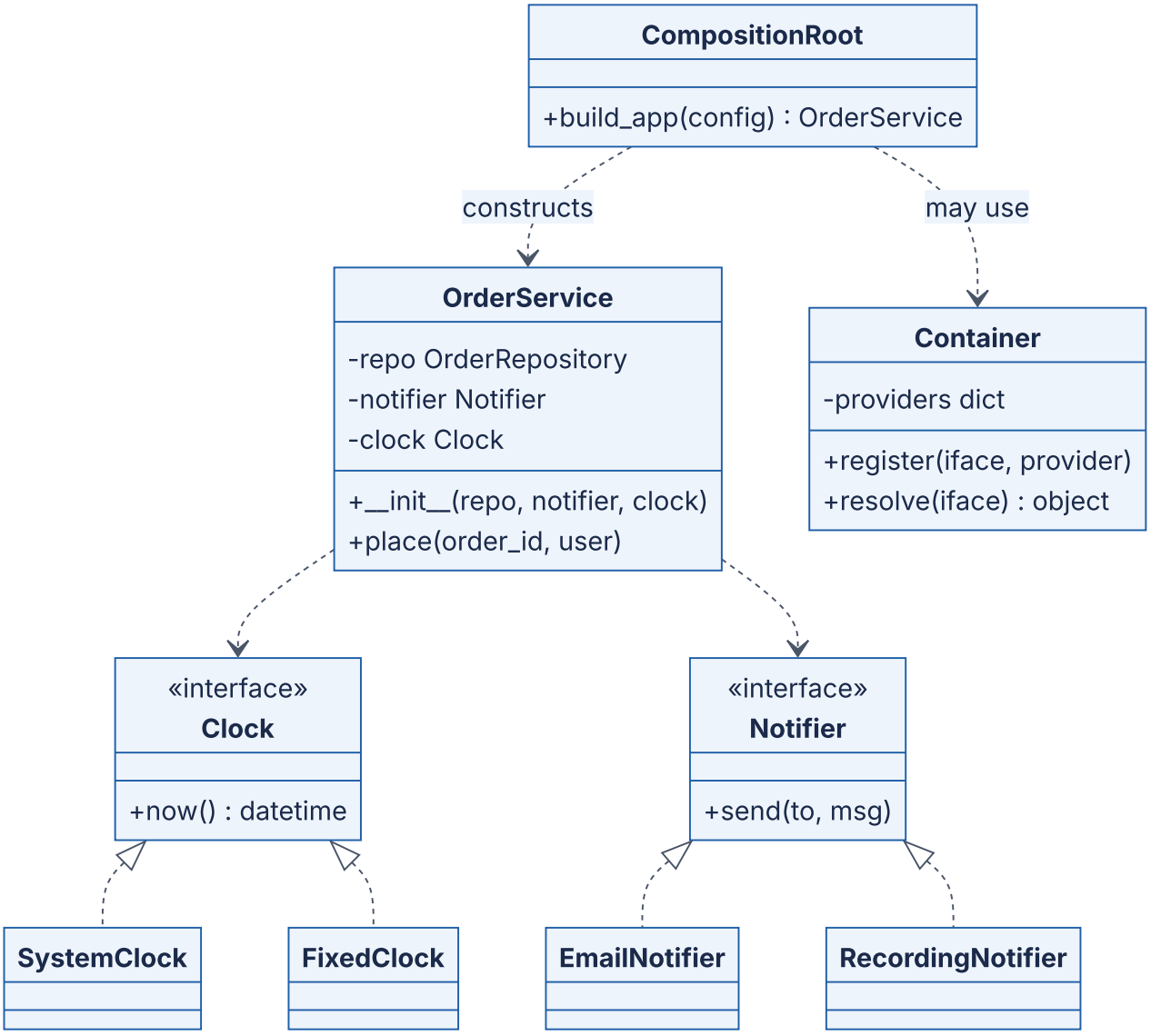
Problem it solves

An `OrderService` that calls `PostgresOrderRepository()` , `VendorPayClient()` and `datetime.now()` inside its methods cannot be tested without a database, a payment sandbox and a frozen clock, and it cannot be reconfigured without editing it. The Dependency Inversion Principle (§36.7) says high-level policy should depend on abstractions; DI is the mechanism that hands the concrete implementations of those abstractions in. Fowler named the pattern in 2004 to distinguish it from the broader term "inversion of control",

and contrasted it with the **Service Locator**, in which objects ask a global registry for dependencies [Fowler 2004]. Section 36.7.4 separates DI, dependency inversion and inversion of control in detail.

Structure

Figure 37.13 — Constructor injection with a composition root.



Reading the diagram. `OrderService` declares its dependencies as constructor parameters typed by interfaces and never constructs them. The composition root, at the program's entry point (`main`, the web app factory, the test fixture), chooses implementations: `SystemClock` and `EmailNotifier` in production, `FixedClock` and `RecordingNotifier` in tests. A container is optional automation of that wiring. Wiring happens once at startup, synchronously; the failure point moves there too, which is good: a missing dependency fails at boot rather than on the first request at 3 a.m.

Python implementation

```
        from abc import ABC, abstractmethod
        from datetime import datetime, timezone
        from typing import get_type_hints
        import inspect


        class Clock(ABC):
            @abstractmethod
            def now(self) -> datetime: ...

        class Notifier(ABC):
            @abstractmethod
            def send(self, to: str, msg: str) -> None: ...

        class SystemClock(Clock):
            def now(self): return datetime.now(timezone.utc)

        class FixedClock(Clock):
            def __init__(self): self._t = datetime(2026, 1, 1, 9, 0,
tzinfo=timezone.utc)
            def now(self): return self._t

        class RecordingNotifier(Notifier):
            def __init__(self): self.sent = []
            def send(self, to, msg): self.sent.append((to, msg))


        class OrderService:
            def __init__(self, notifier: Notifier, clock: Clock):    # constructor in-
jection
                self._notifier, self._clock = notifier, clock
            def place(self, order_id: str, user: str) -> str:
                stamp = self._clock.now().isoformat()
                self._notifier.send(user, f"order {order_id} placed at {stamp}")
                return stamp


        class Container:                                    # a minimal auto-wiring con-
tainer
            def __init__(self):
                self._providers, self._instances = {}, {}
            def register(self, iface, impl):
                self._providers[iface] = impl
            def resolve(self, iface):
                if iface not in self._instances:           # one instance per container
                    impl = self._providers.get(iface, iface)
                    hints = get_type_hints(impl.__init__) if inspect.isclass(impl) else
{}
                    deps = {n: self.resolve(t) for n, t in hints.items() if n != "re-
turn"}
                    self._instances[iface] = impl(**deps)
                return self._instances[iface]
```

```
        if __name__ == "__main__":
            # 1. Test: wire by hand with fakes -- no framework needed.
            notifier = RecordingNotifier()
            OrderService(notifier, FixedClock()).place("o-17", "ana")
            print(notifier.sent)
            # 2. Container: same graph, wired from registrations.
            c = Container()
            c.register(Notifier, RecordingNotifier)
            c.register(Clock, FixedClock)
            svc = c.resolve(OrderService)
            svc.place("o-18", "ben")
            print(c.resolve(Notifier).sent, c.resolve(Notifier) is c.resolve(Notifier))
```

Output:

```
[('ana', 'order o-17 placed at 2026-01-01T09:00:00+00:00')]
[('ben', 'order o-18 placed at 2026-01-01T09:00:00+00:00')] True
```

What to notice:

- The test needs no mocking library: fakes are ordinary classes passed to the constructor.
- The container reads constructor type hints, resolves each recursively, and caches instances; that cache gives you “one per application” without a Singleton class (§37.6).
- The container is the only code that knows concrete classes. Business code never calls `resolve`; if it did, the container would degrade into a service locator with hidden dependencies.

Java and frameworks

In Java, DI containers are the norm: Spring and Guice build the object graph from annotations or configuration, and Spring recommends constructor injection for required dependencies, which keeps fields `final` and objects complete after construction [Spring Docs]. Python services more often wire by hand in the composition root or use framework hooks such as FastAPI’s `Depends`. Either way the principle is the same; a container is a convenience, not the pattern.

When to use and when not

Use constructor injection by default for anything with I/O, time, randomness or configuration, which is what tests need to control. Prefer constructor injection over setter injection (objects are never half-built) and over field injection (dependencies stay visible in the signature). A constructor with eight parameters is a design smell revealing a class with too many responsibilities (§36.3), not a reason to switch to a locator. Do not inject value objects or pure utilities.

Related patterns

Factory (§37.3) and Abstract Factory (§37.4) are often what you inject when an object must create things later. Service Locator is the alternative Fowler contrasts it with. Strategy (§37.7) and Repository (§37.13) are the most commonly injected roles.

Interview question

INTERVIEW TIP

Q. What is the difference between dependency injection and the Dependency Inversion Principle? **A.** Dependency inversion is a design principle: high-level modules and low-level modules should both depend on abstractions, and the abstraction is owned by the high-level side (§36.7). Dependency injection is a technique for supplying concrete implementations from outside. You can inject concrete classes (DI without inversion) or depend on abstractions but look them up from a global (inversion without injection); good designs do both.

37.15 State, Template Method and Chain of Responsibility

These three behavioral patterns each organize *control flow*: State by the object's current mode, Template Method by a fixed skeleton with variable steps, Chain of Responsibility by a sequence of handlers.

37.15.1 State

Intent

DEFINITION

Definition — State. An object delegates its behavior to a separate state object and changes behavior by swapping that state object, so it appears to change its class; each state class handles the transitions that are legal from it [Gamma et al. 1994].

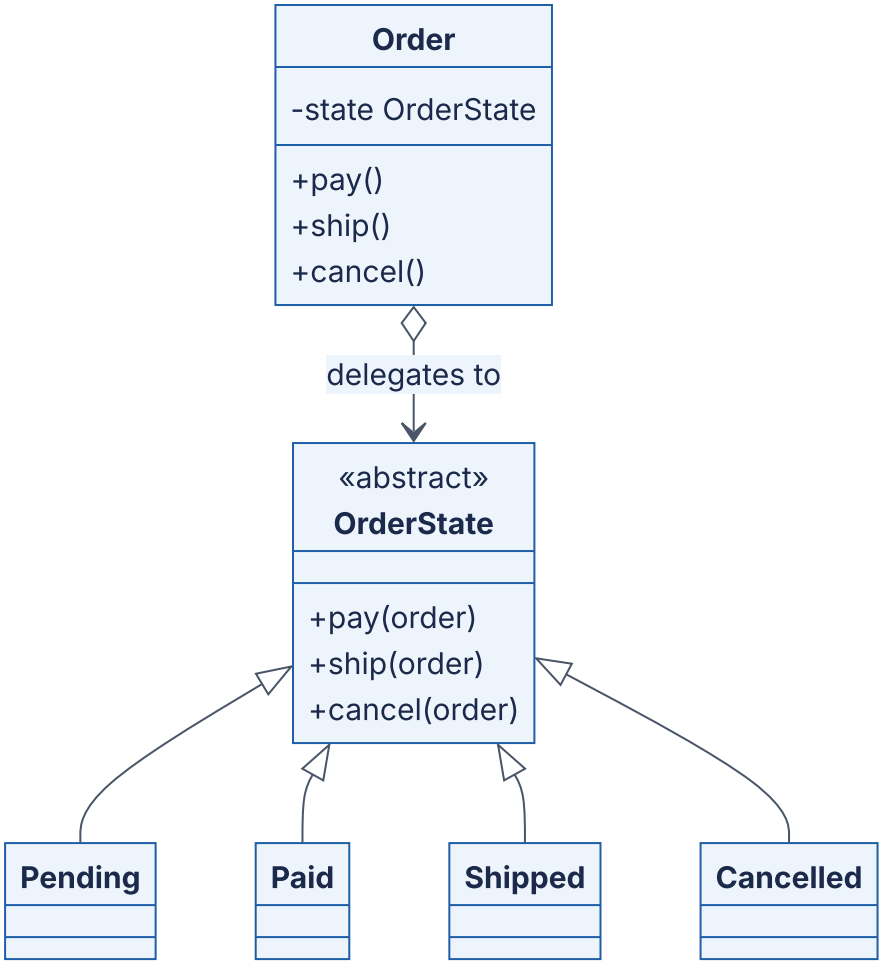
Problem it solves

An order moves from `PENDING` to `PAID` to `SHIPPED`, and can be canceled while pending or paid (with a refund if paid) but not after shipping. Implemented with `if self.status == ...` inside every method, the rules for one state are spread across all methods, and an illegal transition (shipping an unpaid order) slips through whichever method forgot a check. The GoF book's own motivating example was a `TCPConnection` whose `open()`,

`close()` and `acknowledge()` behave differently in the Established, Listen and Closed states.

Structure

Figure 37.14 — State pattern for the order lifecycle.



Reading the diagram. `Order` (the context) forwards every action to its current `OrderState`. The abstract state rejects every action by default; each concrete state overrides only the actions legal from it and performs the transition by replacing the context's state object. Calls are synchronous. The current state is the only data that must be persisted (a status column); the state objects are stateless and can be shared. The failure point is concurrent transitions on the same order from two requests; protect them with optimistic concurrency (a version column) in the repository.

Python implementation

```
        class IllegalTransition(Exception):
            pass


        class OrderState:
            name = "?"
            def _reject(self, action):
                raise IllegalTransition(f"cannot {action} an order that is {self.name}")
            def pay(self, order):    self._reject("pay")
            def ship(self, order):   self._reject("ship")
            def cancel(self, order): self._reject("cancel")


        class Pending(OrderState):
            name = "PENDING"
            def pay(self, order):    order.state = PAID
            def cancel(self, order): order.state = CANCELLED


        class Paid(OrderState):
            name = "PAID"
            def ship(self, order):   order.state = SHIPPED
            def cancel(self, order):
                order.refund_due = True
                order.state = CANCELLED


        class Shipped(OrderState):   name = "SHIPPED"
        class Cancelled(OrderState): name = "CANCELLED"

        PENDING, PAID, SHIPPED, CANCELLED = Pending(), Paid(), Shipped(), Cancelled()  # shared


        class Order:
            def __init__(self):
                self.state, self.refund_due = PENDING, False
            def pay(self):    self.state.pay(self)
            def ship(self):   self.state.ship(self)
            def cancel(self): self.state.cancel(self)


        if __name__ == "__main__":
            for actions in (["pay", "ship"], ["pay", "cancel"], ["ship"], ["pay", "ship", "cancel"]):
                o = Order()
                try:
                    for a in actions:
                        getattr(o, a)()
                    print(f"{'+'.join(actions):16} -> {o.state.name}, refund_due={o.refund_due}")
                except IllegalTransition as e:
                    print(f"{'+'.join(actions):16} -> rejected: {e}")
```

Output:

```
pay+ship         -> SHIPPED, refund_due=False
pay+cancel       -> CANCELLED, refund_due=True
ship             -> rejected: cannot ship an order that is PENDING
pay+ship+cancel  -> rejected: cannot cancel an order that is SHIPPED
```

What to notice: every rule about the `PAID` state is in one class; illegal transitions are rejected by default rather than by remembered checks; and the state objects hold no per-order data, so four shared instances serve every order. For small machines, a transition table (`{(PENDING, "pay"): PAID, ...}`) is a lighter alternative; Chapter 38 draws such machines as state diagrams.

When to use and when not

Use State when behavior differs substantially per mode and transitions have rules (orders, payments, connections, workflow steps, circuit breakers with closed, open and half-open states, §22.13). Use a table or an enum with a transition map when states differ only in which transitions are allowed. Related: Strategy (§37.7) has the same shape but is chosen externally. **Interview question:** *Where does the state live when the service has 20 replicas?* In the database row, not in memory; each request loads the order, applies the transition through the state object, and writes back with a version check so two replicas cannot both ship and cancel the same order.

37.15.2 Template Method

Intent

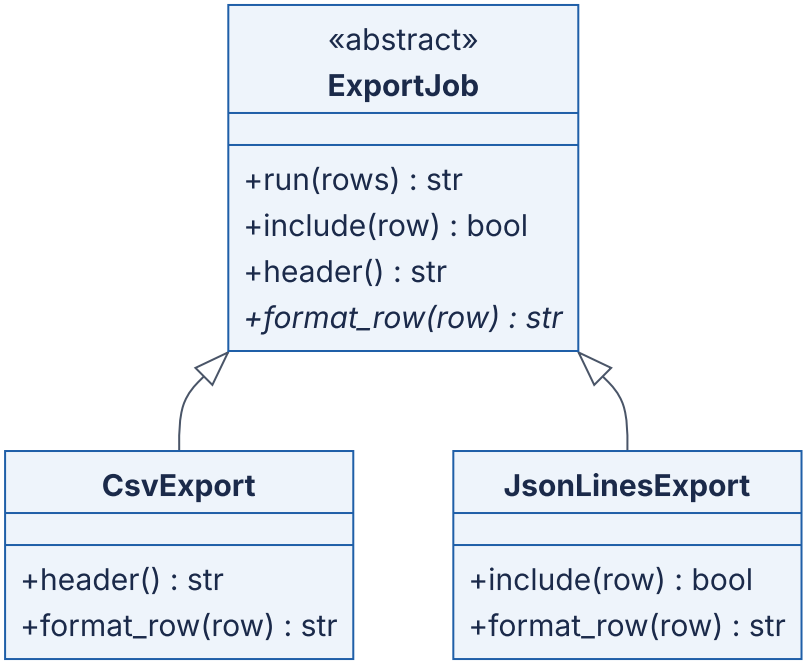
> DEFINITION
>
> **Definition — Template Method.** A method in a base class that defines the skeleton of an algorithm as a fixed sequence of steps, some of which are abstract **primitive operations** or overridable **hooks** that subclasses supply, so subclasses vary steps without changing the sequence [Gamma et al. 1994].

Problem it solves

The store exports orders nightly in CSV for finance and JSON Lines for the data lake. Both exports filter, format each row, add an optional header and write output, in that order; only the filter and formatting differ. Duplicating the loop means a fix to one (say, skipping test orders) is forgotten in the other. Frameworks use template methods everywhere: `unittest.TestCase.run()` calls your `setUp`, test method and `tearDown`; `threading.Thread.start()` arranges for your `run()` to execute; Java's `AbstractList` implements iteration on top of your `get()` and `size()`.

Structure

Figure 37.15 — Template Method for export jobs.



Reading the diagram. `run()` in the base class is the template method; it is not overridden. `format_row()` is abstract (a primitive operation every subclass must supply), while `include()` and `header()` are hooks with defaults that subclasses may override. Control flows from the base class down into subclass code, the “Hollywood principle” (“don’t call us, we’ll call you”). Everything is synchronous; output would cross a storage boundary when written to object storage. The fragility point is the inheritance coupling: a change to the skeleton affects every subclass.

Python implementation

```
        import json
        from abc import ABC, abstractmethod


        class ExportJob(ABC):
            def run(self, rows: list[dict]) -> str:          # the template method
                kept = [r for r in rows if self.include(r)]
                lines = [self.format_row(r) for r in kept]
                header = self.header()
                return "\n".join(([header] if header else []) + lines)

            def include(self, row) -> bool: return not row.get("test", False)   # hook
            def header(self) -> str | None: return None                          # hook
            @abstractmethod
            def format_row(self, row) -> str: ...                                # primi-
tive op


        class CsvExport(ExportJob):
            def header(self): return "order_id,total_cents"
            def format_row(self, r): return f"{r['order_id']},{r['total_cents']}"


        class JsonLinesExport(ExportJob):
            def include(self, r): return super().include(r) and r["status"] == "PAID"
            def format_row(self, r): return json.dumps(r, sort_keys=True)


        if __name__ == "__main__":
            rows = [{"order_id": "o-1", "total_cents": 4999, "status": "PAID"},
                    {"order_id": "o-2", "total_cents": 1999, "status": "PENDING"},
                    {"order_id": "o-3", "total_cents": 100, "status": "PAID", "test":
True}]
            print(CsvExport().run(rows))
            print(JsonLinesExport().run(rows))
```

Output:

```
order_id,total_cents
o-1,4999
o-2,1999
{"order_id": "o-1", "status": "PAID", "total_cents": 4999}
```

What to notice: the test-order filter lives once, in the base hook, and `JsonLinesExport` extends it with `super()`; neither subclass can reorder the steps.

When to use and when not

Use Template Method in frameworks and base classes where the sequence is invariant and must be enforced (test lifecycles, request handling, ETL jobs). Prefer Strategy (§37.7)

or passing functions when steps vary independently or must be combined at run time, since inheritance fixes the combination at class-definition time (composition over inheritance, §35.8). **Interview question:** *Template Method or Strategy for a pluggable ETL pipeline?* Template Method if the pipeline shape is fixed and you own the base class; Strategy (inject `extract`, `transform` and `load` objects) if teams must mix and match steps, which is the usual case at scale.

37.15.3 Chain of Responsibility

Intent

DEFINITION

Definition — Chain of Responsibility. A request is passed along a chain of handler objects; each handler either handles it, or does some work and passes it to the next, so the sender does not know which handler will act [Gamma et al. 1994].

Problem it solves

Every HTTP request to the order API must be logged, authenticated and rate-limited before reaching the application, and some requests must be stopped early (401, 429). Putting all of that at the top of every endpoint duplicates code and mixes concerns. A chain of middleware applies the concerns in a configured order, and any link can short-circuit. Java servlet filters, Express and ASGI middleware, gRPC interceptors and Envoy's HTTP filter chain all work this way.

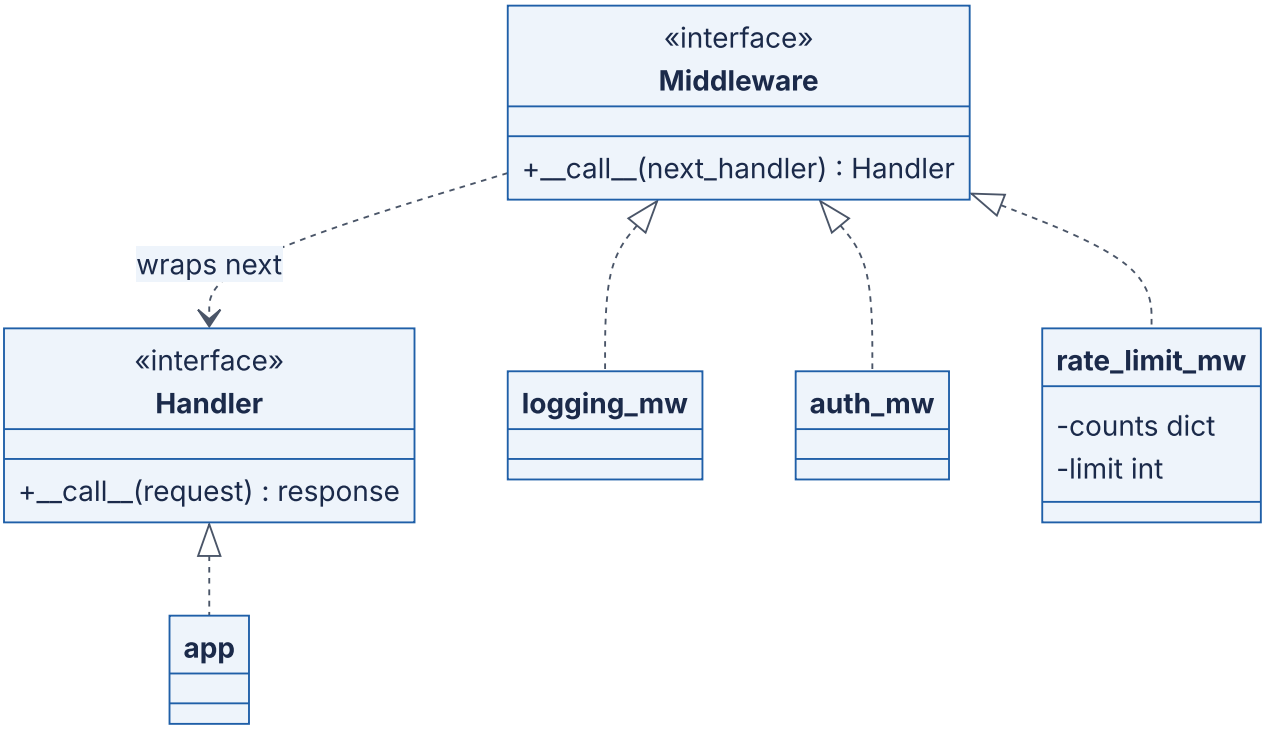
INDUSTRY EXAMPLE

Django middleware. Django builds its request pipeline from the `MIDDLEWARE` setting: each middleware is a factory that receives `get_response` (the next handler) and returns a callable. Requests pass through the list top to bottom and responses bottom to top, and a middleware can return a response without calling `get_response`, short-circuiting the chain. The documentation stresses that order matters, for example placing session middleware before authentication middleware, which reads the session. *Evidence: Primary* — [Django Docs]

The classic GoF form, in which exactly one handler takes the request (a support ticket escalating from bot to agent to engineer, or nested exception handlers), is the same structure with a different stopping rule.

Structure

Figure 37.16 — Middleware pipeline as a chain of responsibility.



Reading the diagram. A `Handler` turns a request into a response. A `Middleware` takes the next handler and returns a new handler that wraps it, so building the chain is function composition: `logging(auth(rate_limit(app)))`. A request flows inward through each link and the response flows back out, so the first middleware sees both the raw request and the final response (ideal for logging and timing). Links run synchronously in one request's call stack. `rate_limit_mw` holds in-memory per-client counters; across replicas this state must move to a shared store (§22.8). Any link can short-circuit, which is also the main failure mode: a misordered chain (rate limiting before authentication, so attackers exhaust real users' quotas).

Python implementation

```
        from typing import Callable

        Request = dict
        Response = tuple[int, str]
        Handler = Callable[[Request], Response]
        LOG: list[str] = []


        def logging_mw(next_h: Handler) -> Handler:
            def handle(req):
                status, body = next_h(req)
                LOG.append(f"{req['client']} {req['path']} -> {status}")
                return status, body
            return handle


        def auth_mw(next_h: Handler) -> Handler:
            def handle(req):
                if req.get("token") != "valid":
                    return 401, "unauthorized"                # short-circuit the chain
                return next_h(req)
            return handle


        def rate_limit_mw(limit: int):
            counts: dict[str, int] = {}
            def middleware(next_h: Handler) -> Handler:
                def handle(req):
                    counts[req["client"]] = counts.get(req["client"], 0) + 1
                    if counts[req["client"]] > limit:
                        return 429, "too many requests"
                    return next_h(req)
                return handle
            return middleware


        def app(req: Request) -> Response:
            return 200, f"orders for {req['client']}"


        def build(handler: Handler, middlewares) -> Handler:   # first listed = outer-
most
            for mw in reversed(middlewares):
                handler = mw(handler)
            return handler


        if __name__ == "__main__":
            pipeline = build(app, [logging_mw, auth_mw, rate_limit_mw(limit=2)])
            for client, token in [("ana", "valid")] * 3 + [("eve", "forged")]:
                pipeline({"client": client, "token": token, "path": "/orders"})
            print("\n".join(LOG))
```

Output:

```
ana /orders -> 200
ana /orders -> 200
ana /orders -> 429
eve /orders -> 401
```

What to notice: because authentication runs before rate limiting, the forged request never consumes a rate-limit slot; and because logging is outermost, it records short-circuited responses too. Reordering the list changes behavior without changing any handler.

When to use and when not

Use a chain when several independent handlers must process or filter a request in a configurable order: middleware, validation pipelines, approval workflows, event filters. Avoid long chains with no clear ordering contract; debugging "which link swallowed my request?" requires per-link logging or tracing. **Interview question:** *In what order would you put authentication, rate limiting, request logging, compression and caching middleware, and why?* Logging and tracing outermost to see everything; then a coarse per-IP rate limit to shed floods cheaply; then authentication; then per-user rate limiting; then the cache (keyed by user if responses are personalized); compression closest to the network on the way out.

37.16 Concurrency patterns

The patterns so far organize code within one thread of control. Concurrency patterns organize work across threads: who produces work, who executes it, how results come back, and how shared data is protected. They are cataloged in *Pattern-Oriented Software Architecture, Volume 2* [Schmidt et al. 2000], which also covers Reactor and Proactor (the event-loop designs of §6.13), Active Object, Monitor Object and Leader/Followers, and they are treated for the JVM in *Java Concurrency in Practice* [Goetz et al. 2006]. The primitives underneath (mutexes, condition variables, atomics) are in §6.10.

> NOTE
>
> **Python and the GIL.** In standard CPython builds the global interpreter lock lets only one thread execute Python bytecode at a time, so threads speed up I/O-bound work (the lock is released while waiting on sockets and disks) but not CPU-bound Python code; use processes (`ProcessPoolExecutor`) for that. Python 3.13 added an experimental free-threaded build without the GIL [PEP 703]. Correct code must not rely on the GIL for thread safety in either case.

37.16.1 Producer–consumer

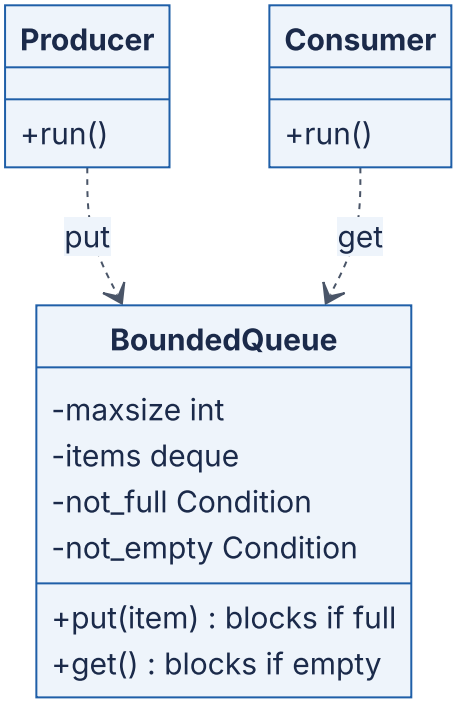
Intent and problem

DEFINITION

Definition — Producer-consumer. Producers put work items into a shared **bounded** buffer and consumers take them out, decoupling the rate at which work is created from the rate at which it is processed; when the buffer is full, producers block or are rejected, which is **backpressure** (§22.10).

The order service's request threads produce "send receipt" jobs faster at peak than the email provider accepts them. Calling the provider inline couples checkout latency to the provider; an unbounded in-memory list grows until the process runs out of memory. A bounded queue between request threads and a fixed set of sender threads absorbs bursts and pushes back when sustained load exceeds capacity. Message brokers (Chapter 30) are this pattern across machines.

Figure 37.17 — Producer-consumer with a bounded queue.



Reading the diagram. Producers and consumers never reference each other; both depend only on the queue. Inside the queue a lock and two condition variables (§6.10.3) implement blocking: `put` waits on "not full", `get` waits on "not empty". The hand-off is asynchronous from the producer's point of view. Items in the queue live only in process memory and are lost if the process crashes, which is the reason to use a durable broker when jobs must survive restarts. Failure points: a consumer that dies silently (work piles up), a

poison item that crashes every consumer, and shutdown (consumers must be told to stop). Scale by adding consumers until the downstream resource saturates.

```
        import queue
        import threading
        import time


        def producer(q: queue.Queue, pid: int, n: int, stats: dict):
            for i in range(n):
                t0 = time.perf_counter()
                q.put(f"receipt-{pid}-{i}")                    # blocks while the queue
is full
                stats["blocked_s"] += time.perf_counter() - t0


        def consumer(q: queue.Queue, done: list, lock: threading.Lock):
            while True:
                item = q.get()
                if item is None:                               # sentinel: shut down
                    break
                time.sleep(0.002)                              # simulated call to email
provider
                with lock:
                    done.append(item)


        if __name__ == "__main__":
            q: queue.Queue = queue.Queue(maxsize=10)           # bounded: backpressure
            done, lock, stats = [], threading.Lock(), {"blocked_s": 0.0}
            consumers = [threading.Thread(target=consumer, args=(q, done, lock)) for _
in range(4)]
            producers = [threading.Thread(target=producer, args=(q, p, 100, stats)) for
p in range(2)]
            for t in consumers + producers:
                t.start()
            for t in producers:
                t.join()
            for _ in consumers:
                q.put(None)                                    # one sentinel per con-
sumer
            for t in consumers:
                t.join()
            print(f"processed {len(done)} jobs; queue never exceeded {q.maxsize}; "
                  f"producers spent time blocked: {stats['blocked_s'] > 0.01}")
```

Output:

```
processed 200 jobs; queue never exceeded 10; producers spent time blocked: True
```

What to notice: producers generate 200 jobs instantly but consumers handle about 4 / 0.002 s = 2,000 jobs/s, so producers spend time blocked in `put`; that waiting is backpres-

sure doing its job. One sentinel per consumer gives a clean shutdown. In a web request path, prefer `put(timeout=...)` or `put_nowait` and return 503 rather than blocking a request thread indefinitely (load shedding, §22.9).

37.16.2 Thread pool

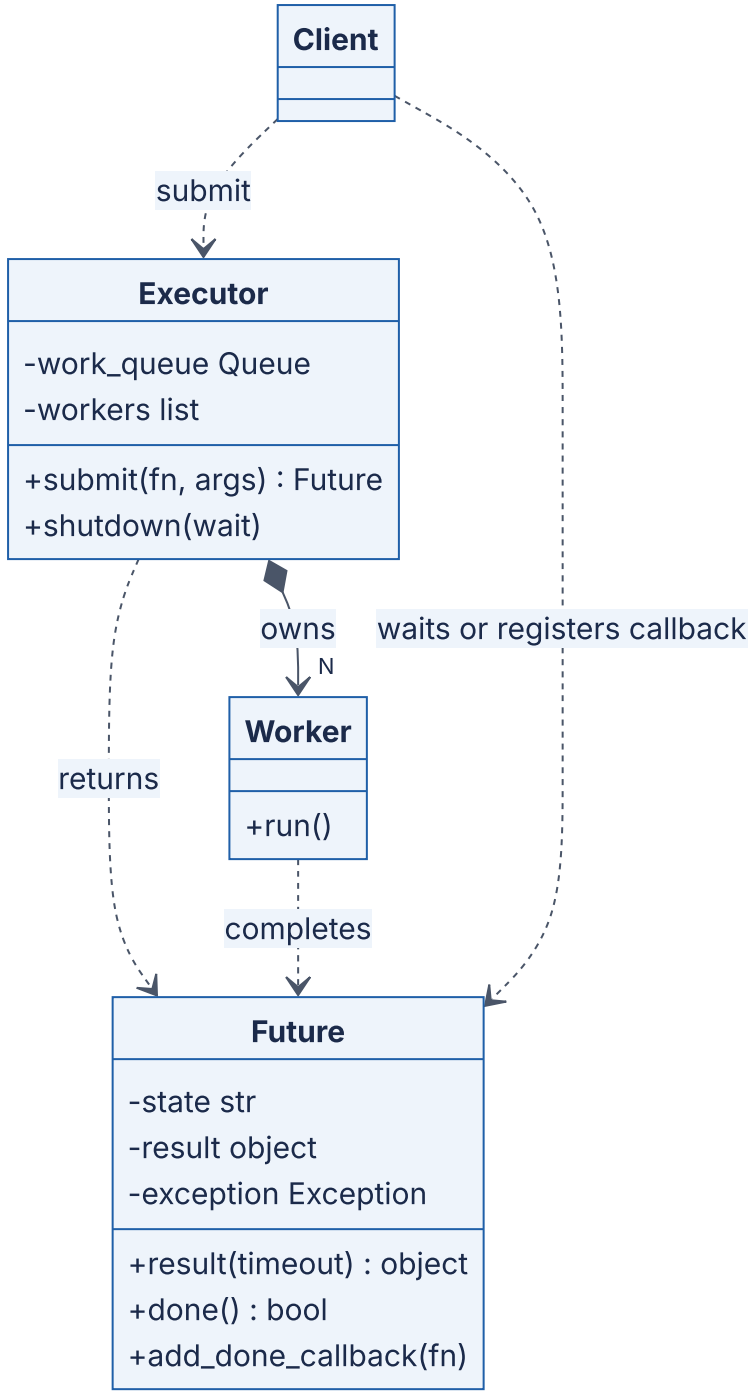
Intent and problem

DEFINITION

Definition — Thread pool. A fixed or bounded set of worker threads that take tasks from a queue and execute them, so the cost of creating threads is paid once and the number of concurrent tasks is capped [Goetz et al. 2006].

Creating a thread per request costs memory (often 1 MiB or more of reserved stack per thread, §6.3.5) and scheduling overhead, and under a spike it creates thousands of threads that thrash. A pool is producer–consumer with tasks as the items: callers submit tasks (Commands, §37.12) and receive futures. Java's `ThreadPoolExecutor` and Python's `concurrent.futures.ThreadPoolExecutor` are standard implementations. The key design decision is the pool size.

Figure 37.18 — Thread pool returning futures.



Reading the diagram. The client submits a callable and immediately receives a `Future`; submission is asynchronous. The executor owns N workers (composition, filled diamond), which pull tasks from the internal queue, run them, and complete the matching future with a result or an exception. The client later blocks on `result(timeout)` or registers a callback. State lives in memory: the task queue (which should be bounded) and the futures. Failure points: an unbounded queue that hides overload until memory runs out, tasks that block forever and starve the pool, and exceptions that vanish if no one calls `result()`. Scale by sizing N (below) and by separate pools per dependency (bulkheads, §22.14).

Mathematical foundations

FORMULA

Thread-pool sizing for mixed CPU and I/O work [Goetz et al. 2006]

$$N_{threads} = N_{cpu} \times U_{cpu} \times (1 + \frac{W}{C})$$

Symbol	Meaning	Units
$N_{threads}$	Number of worker threads that keeps the CPU at the target utilization	threads
N_{cpu}	Number of CPU cores available to the process	cores
U_{cpu}	Target CPU utilization, between 0 and 1	dimensionless
W	Time a task spends waiting (I/O, locks) per task	ms
C	CPU time a task spends computing per task	ms

WORKED EXAMPLE

Worked example — Sizing the receipt-rendering pool. Each task spends $C = 5$ ms of CPU rendering a receipt and $W = 95$ ms waiting on the template store and the email provider. The service has $N_{cpu} = 8$ cores and you target $U_{cpu} = 0.75$ to leave headroom.

1. Wait-to-compute ratio: $W/C = 95/5 = 19$.
2. Threads: $N_{threads} = 8 \times 0.75 \times (1 + 19) = 8 \times 0.75 \times 20 = 120$.
3. Throughput ceiling, by Little's Law (§4.11) with each task holding a thread for $W + C = 100$ ms: $\lambda_{max} = 120/0.1\ \text{s} = 1,200$ tasks/s.
4. CPU check: $1,200 \times 5$ ms $= 6$ core-seconds per second, which is $6/8 = 75\%$ of the cores, matching the target.

Interpretation. A pool of 8 (one per core) would cap throughput at $8/0.1 = 80$ tasks/s with the CPU 95% idle; a pool of 1,000 would not raise throughput above what the CPU and the downstream provider allow, and would add memory and contention.
Engineering implication. Measure W and C under realistic load, size the pool from the formula, and check the downstream limit separately: if the email provider allows only 500 requests/s, the provider, not the pool, is the bottleneck, and the pool should be sized for 500 tasks/s ($500 \times 0.1 = 50$ threads).

```
        import time
        from concurrent.futures import ThreadPoolExecutor, as_completed


        def fetch_rate(currency: str) -> tuple[str, float]:
            time.sleep(0.05)                                    # simulated 50 ms I/O
call
            if currency == "XXX":
                raise ValueError("unknown currency")
            return currency, {"USD": 1.0, "EUR": 1.08}.get(currency, 0.5)


        if __name__ == "__main__":
            currencies = ["USD", "EUR"] * 31 + ["GBP", "XXX"]  # 64 tasks
            t0 = time.perf_counter()
            ok, failed = 0, []
            with ThreadPoolExecutor(max_workers=16) as pool:
                futures = {pool.submit(fetch_rate, c): c for c in currencies}
                for fut in as_completed(futures):               # in completion order
                    try:
                        fut.result(timeout=1.0)
                        ok += 1
                    except ValueError as e:
                        failed.append(f"{futures[fut]}: {e}")
            elapsed = time.perf_counter() - t0
            print(f"{ok} ok, failed={failed}")
            print(f"sequential would take {len(currencies) * 0.05:.1f} s; pool took
{elapsed:.1f} s")
```

Output:

```
63 ok, failed=['XXX: unknown currency']
sequential would take 3.2 s; pool took 0.2 s
```

What to notice: 64 tasks of 50 ms on 16 workers take $\lceil 64/16 \rceil \times 0.05 = 0.2$ s. The exception raised inside a worker is re-raised by `result()` in the caller; if nobody calls `result()`, it is silently lost. The `with` block shuts the pool down and waits for running tasks.

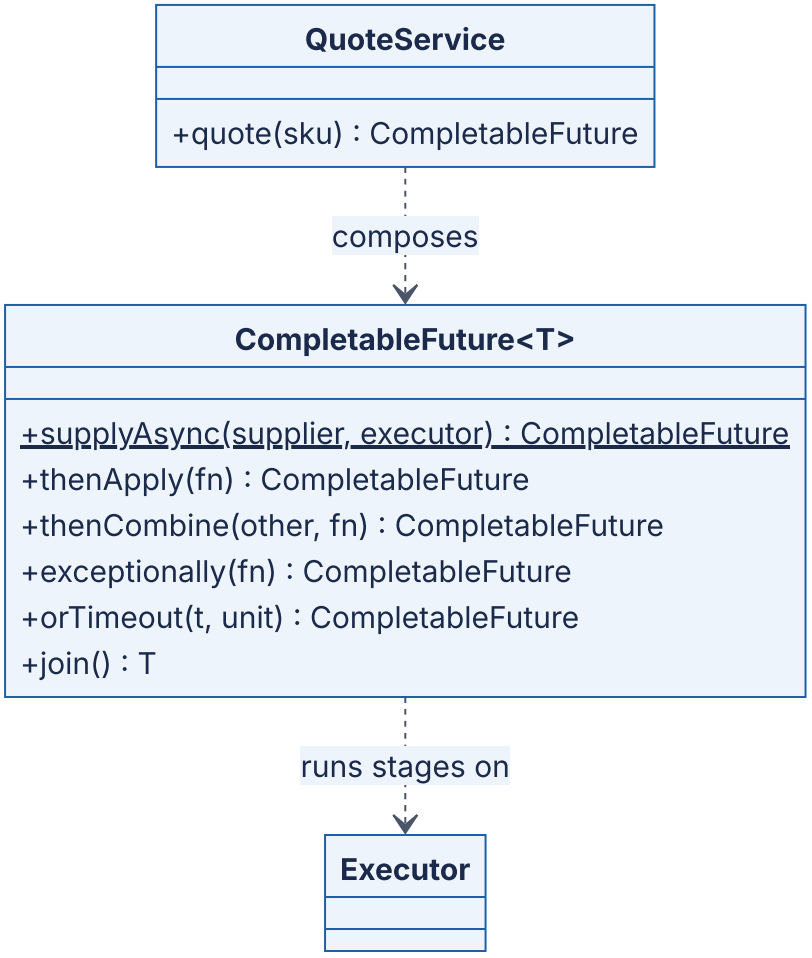
37.16.3 Future and promise

DEFINITION

Definition — Future and promise. A **future** is a read-only placeholder for a result that will be available later; a **promise** (Java's `CompletableFuture`, Python's `asyncio.Future`) is the writable side that the producer completes with a value or an error. Together they let a caller start work, continue, and later wait for or chain on the result.

Futures turn “call and wait” into “call, do other work, join”. Their real power is composition: start the price lookup and the stock check in parallel and combine them when both finish, so latency is the maximum of the two rather than the sum. Composition is where Java adds something language-specific: `CompletableFuture` chains asynchronous stages without blocking a thread.

Figure 37.19 — Composing futures.



Reading the diagram. `QuoteService` starts two asynchronous stages on an executor and combines their futures with `thenCombine`; each stage runs on a pool thread and completes its future, which triggers dependent stages. No thread blocks until the final `join()` (or never, if the result is returned as a future to an asynchronous web framework). Results live in memory. Failure points are a stage that never completes (add `orTimeout`) and errors, which propagate along the chain until handled by `exceptionally`. Scaling depends on the executor; using the default common pool for blocking I/O is a classic mistake.

```
        import java.util.concurrent.*;

        public class Quote {
            public static void main(String[] args) {
                ExecutorService io = Executors.newFixedThreadPool(8);   // dedicated
pool for I/O
                CompletableFuture<Integer> price = CompletableFuture.supplyAsync(() ->
sleepThen(80, 4999), io);
                CompletableFuture<Integer> stock = CompletableFuture.supplyAsync(() ->
sleepThen(60, 3), io);
                long t0 = System.nanoTime();
                String quote = price.thenCombine(stock, (p, s) -> s > 0 ? "price " + p
: "out of stock")
                                    .orTimeout(500, TimeUnit.MILLISECONDS)
                                    .exceptionally(e -> "fallback: cached price")
                                    .join();
                System.out.printf("%s in ~%d ms%n", quote, (System.nanoTime() - t0) /
1_000_000);
                io.shutdown();
            }
            static int sleepThen(int ms, int v) {
                try { Thread.sleep(ms); } catch (InterruptedException e) { throw new
RuntimeException(e); }
                return v;
            }
        }
```

The program prints `price 4999 in ~80 ms` (the time varies slightly): the two calls overlap, so latency is the longer one, not 140 ms. In Python the same composition is `asyncio.gather(price(), stock())` inside an event loop, or two `executor.submit` calls followed by two `result()` calls.

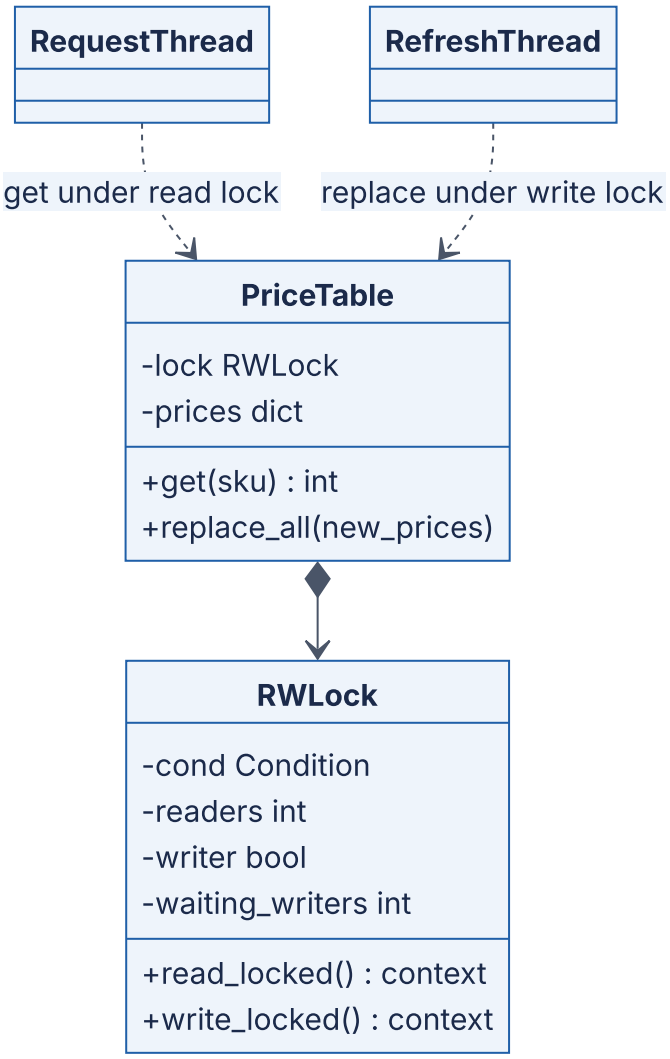
37.16.4 Read-write lock

DEFINITION

Definition — Read-write lock. A lock that allows any number of concurrent readers or exactly one writer, never both; it improves throughput when reads greatly outnumber writes and each read holds the lock long enough for contention to matter (§6.10.5).

A routing table or price list in memory is read by every request and replaced a few times per minute. A plain mutex serializes all readers; a read-write lock lets them proceed together. The design question is fairness: a reader-preferring lock can starve writers forever under constant reads, so most implementations prefer waiting writers. Python's standard library has no read-write lock; Java provides `ReentrantReadWriteLock` and `StampedLock` (which adds optimistic reads).

Figure 37.20 — A writer-preferring read-write lock.



Reading the diagram. `PriceTable` owns a `RWLock` built from one condition variable and three counters. Many request threads read at once; the refresh thread takes the write lock, which waits for active readers to drain and blocks new readers while it waits (writer preference). All access is synchronous and in-process; the protected data is in memory. Failure points: writer starvation (prevented here), upgrading a read lock to a write lock (deadlocks if two readers try), and long work done while holding the lock. An alternative that often scales better is copy-on-write: build a new immutable table and swap one reference, so readers take no lock at all.

```
        import threading
        import time
        from contextlib import contextmanager


        class RWLock:
            def __init__(self):
                self._cond = threading.Condition()
                self._readers, self._writer, self._waiting_writers = 0, False, 0

            @contextmanager
            def read_locked(self):
                with self._cond:
                    while self._writer or self._waiting_writers:   # writer preference
                        self._cond.wait()
                    self._readers += 1
                try:
                    yield
                finally:
                    with self._cond:
                        self._readers -= 1
                        if self._readers == 0:
                            self._cond.notify_all()

            @contextmanager
            def write_locked(self):
                with self._cond:
                    self._waiting_writers += 1
                    while self._writer or self._readers:
                        self._cond.wait()
                    self._waiting_writers -= 1
                    self._writer = True
                try:
                    yield
                finally:
                    with self._cond:
                        self._writer = False
                        self._cond.notify_all()


        if __name__ == "__main__":
            lock, accounts = RWLock(), {"a": 500, "b": 500}     # invariant: a + b ==
1000
            stats = {"violations": 0, "active": 0, "peak": 0}
            stats_lock = threading.Lock()

            def reader():
                for _ in range(300):
                    with lock.read_locked():
                        with stats_lock:
                            stats["active"] += 1
                            stats["peak"] = max(stats["peak"], stats["active"])
                        time.sleep(0.0002)                      # simulated work: read-
ers overlap
                        if accounts["a"] + accounts["b"] != 1000:
                            stats["violations"] += 1
```

```
                        with stats_lock:
                            stats["active"] -= 1

            def writer():
                for i in range(200):
                    with lock.write_locked():
                        accounts["a"] -= 1                     # invariant broken ...
                        time.sleep(0)                          # yield mid-update
                        accounts["b"] += 1                     # ... and restored
            threads = [threading.Thread(target=reader) for _ in range(6)] +
[threading.Thread(target=writer)]
            for t in threads: t.start()
            for t in threads: t.join()
            print(f"violations seen by readers: {stats['violations']}; "
                  f"readers overlapped: {stats['peak'] > 1}; final {accounts}")
```

Output:

```
violations seen by readers: 0; readers overlapped: True; final {'a': 300, 'b': 700}
```

What to notice: readers never observe the moment between the writer's two updates, yet several readers hold the lock simultaneously. Releasing in `finally` ensures an exception inside the critical section cannot leave the lock held forever.

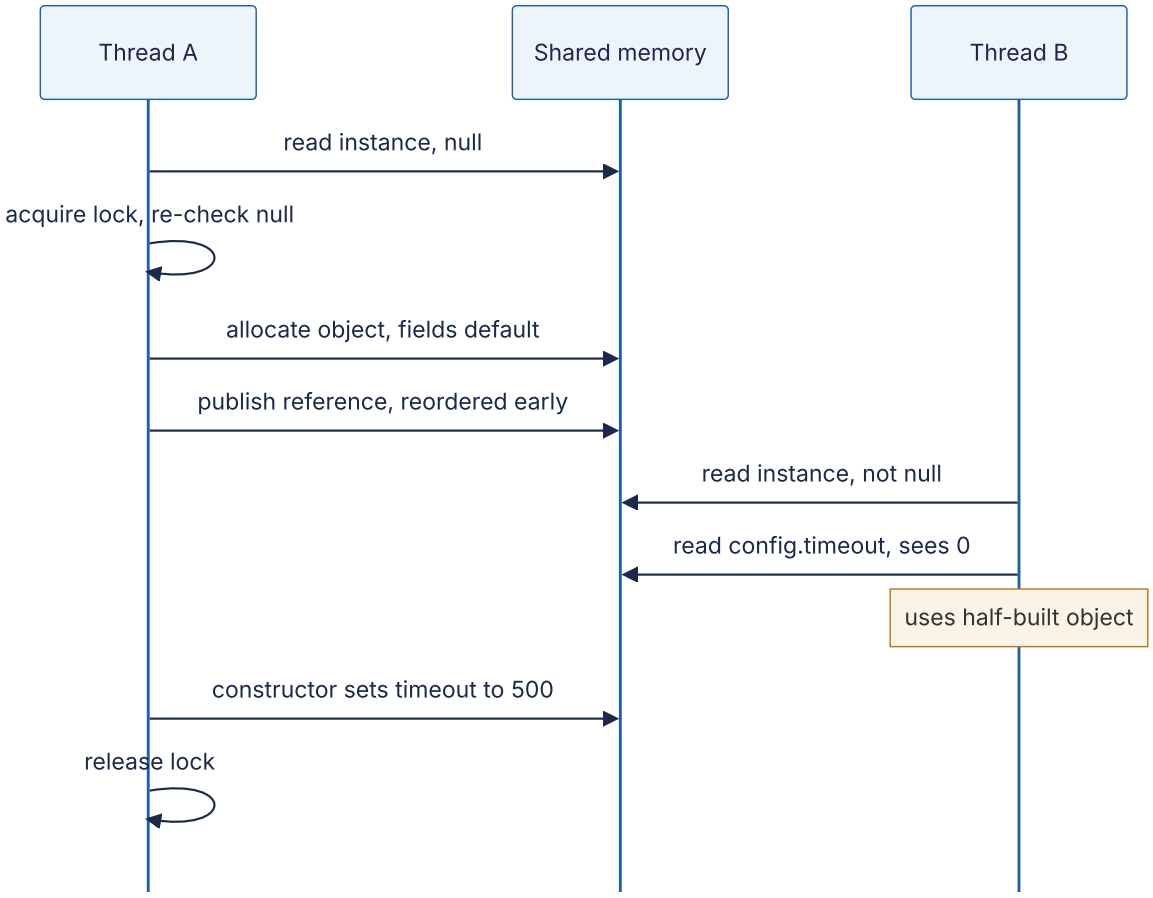
37.16.5 Double-checked locking and its pitfalls

DEFINITION

Definition — Double-checked locking. A lazy-initialization idiom that checks whether an object exists without a lock (fast path), and only if it does not, takes a lock, checks again, and creates it. It avoids locking on every access but is only correct if the language's memory model guarantees that a thread seeing the reference also sees the fully constructed object.

The Singleton in §37.6 used this idiom safely in Python. In Java before the Java 5 memory model (JSR-133, 2004), and in Java today without `volatile`, it is broken [Bacon et al.; Manson et al. 2005]. The compiler and CPU may reorder the steps of `instance = new Config()` (allocate memory, run the constructor, publish the reference) so that the reference is published *before* the constructor finishes. A second thread on the fast path sees a non-null reference, skips the lock, and reads fields that still hold default values.

Figure 37.21 — How double-checked locking fails without volatile.



Reading the diagram. Thread A takes the slow path and Thread B the fast path; both read and write shared memory. Without a happens-before edge, A's write of the reference may become visible to B before A's writes of the fields: B never takes the lock, so nothing orders its reads after A's constructor. The failure is silent and rare, appearing under load on weakly ordered hardware or after a JIT optimization. The fixes below create the missing ordering; none adds a lock to the fast path.

```
        public final class ConfigHolder {
            private static volatile Config instance;      // volatile: safe publication

            static Config get() {
                Config local = instance;                   // one volatile read on the
fast path
                if (local == null) {
                    synchronized (ConfigHolder.class) {
                        local = instance;
                        if (local == null) {
                            instance = local = new Config(500);   // fully built before
publication
                        }
                    }
                }
                return local;
            }

            static final class Config {
                final int timeoutMs;                       // final fields add a further
guarantee
                Config(int timeoutMs) { this.timeoutMs = timeoutMs; }
            }
        }
```

What to notice: `volatile` makes the write of `instance` a release and the read an acquire, so a thread that sees the reference also sees the constructor's writes. Simpler fixes usually win: the holder idiom or an enum (§37.6), eager initialization, or a plain `synchronized` method if the access is not hot. In Python, the risk is not reordering but the unlocked check-then-act race shown in §37.6.

Table 37.3 — Concurrency failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Unbounded queue exhausts memory	Producers faster than consumers, no backpressure	Queue depth and memory climb steadily	Shed load, restart	Bounded queues, reject or block when full
Pool starvation	Tasks block waiting on tasks in the same pool	All workers busy, throughput zero, no CPU	Restart, raise timeouts	Separate pools per dependency; no nested waits
Lost exceptions	Future never inspected	Missing results, no error logs	Re-run failed tasks	Always call `result()` or attach a callback that logs
Writer starvation	Reader-preferring lock under constant reads	Config refresh latency grows without bound	Pause readers	Writer-preferring lock or copy-on-write swap
Half-built object visible	Unsafe publication in double-checked locking	Rare `NullPointerException` or default values under load	Restart	`volatile`, holder idiom, immutable objects

37.17 Choosing a pattern

Patterns are answers; start from the question. Name the force that hurts (a growing `if` chain, an untestable class, a slow dependency on the request path), find the pattern that addresses that force, and state the trade-off you accept. If you cannot name the force, you do not need the pattern.

Table 37.4 — Pattern selection: problem → pattern → trade-off.

Problem you have	Pattern	Trade-off you accept
Concrete class chosen at run time from data	Simple factory (§37.3.1)	One more indirection; registry must be kept complete
Framework workflow needs a part supplied by extensions	Factory method (§37.3.2)	A subclass per variant
Several products must vary together per environment	Abstract factory (§37.4)	Adding a product type changes every factory
Object with many optional parts and cross-field rules	Builder (§37.5)	Extra class; mostly unnecessary with keyword arguments
Exactly one instance per process	Module or composition-root instance; Singleton (§37.6)	Global state, hidden dependencies, test leakage
Interchangeable algorithms chosen by config	Strategy (§37.7)	Clients must know which strategy to pick
Open-ended reactions to a change in one process	Observer (§37.8)	Synchronous fan-out latency; lapsed listeners
Third-party interface does not match yours	Adapter (§37.9)	Translation layer to maintain
Optional cross-cutting behavior to combine freely	Decorator (§37.10)	Many small objects; order-dependent stacks
Lazy creation, access control or remote object	Proxy (§37.11)	Hidden cost (N+1 queries, network calls)
Undo, queueing, logging or replaying requests	Command (§37.12)	A class per operation; undo must be exact
Domain logic tangled with storage code	Repository (§37.13)	Fake drifts unless contract-tested; poor for reporting
Classes create their own dependencies; hard to test	Dependency injection (§37.14)	Wiring code or a container to understand
Behavior and legal actions depend on mode	State (§37.15.1)	A class per state; transition logic spread across classes
Fixed algorithm skeleton with variable steps	Template Method (§37.15.2)	Inheritance coupling
Ordered, short-circuiting request processing	Chain of Responsibility (§37.15.3)	Order bugs; harder debugging
Bursty work must not overwhelm a slow consumer	Producer–consumer (§37.16.1)	Queueing delay; in-memory work lost on crash
Many short tasks, bounded concurrency	Thread pool + futures (§37.16.2–3)	Sizing, starvation and lost exceptions to manage
Read-mostly shared data	Read-write lock or copy-on-write (§37.16.4)	Writer starvation or copy cost
Hot lazy initialization	Holder idiom, `volatile` DCL (§37.16.5)	Subtle memory-model reasoning

RECOMMENDATION

Recommendation — Earn each pattern. Write the straightforward version first. Introduce a pattern when the second variant arrives (a second notifier, a second storage backend) or when a test cannot be written. In Python, try the lightest form first: a function argument before a Strategy hierarchy, a module before a Singleton, a closure before a Command class. Patterns applied speculatively add indirection that every future reader pays for.

37.18 Summary and revision notes

SUMMARY

- A **design pattern** is a named solution shape with known trade-offs. GoF groups patterns as creational, structural and behavioral; Repository and DI are enterprise patterns; producer–consumer, thread pools and futures are concurrency patterns.
- **Creational:** a simple factory centralizes a run-time class choice; a factory method lets subclasses choose the product; an abstract factory creates consistent families; a builder validates cross-field rules and emits an immutable product.
- **Singleton** means one instance *per process*; prefer one instance created in the composition root and injected. In Python, a module is the idiomatic singleton; in Java, the enum or holder idiom.
- **Strategy** swaps algorithms by composition; in Python a strategy is often a function. **State** swaps behavior as the object changes mode. **Template Method** fixes a skeleton and varies steps through inheritance.
- **Observer** is in-process, synchronous publish/subscribe; isolate failing observers and move slow ones behind a queue or broker (§30.5).
- **Adapter** changes an interface, **Decorator** keeps it and adds behavior, **Proxy** keeps it and controls access, **Facade** simplifies a subsystem. Python's `@decorator` syntax is a language feature, not the GoF pattern.
- **Command** turns requests into values for undo, queueing and replay. **Chain of Responsibility** underlies middleware; order matters.
- **Repository** hides storage behind a domain collection interface; contract-test every implementation. **Dependency injection** supplies dependencies from a composition root and realizes dependency inversion (§36.7).
- **Concurrency:** bound every queue; size pools with $N_{cpu} U_{cpu} (1 + W/C)$ and check downstream limits; always inspect futures; prefer writer-preferring read-write locks or copy-on-write for read-mostly data; double-checked locking in Java requires `volatile`.

37.19 Knowledge checkpoint

1. What problem does a simple factory solve that a constructor call does not?
2. When does an abstract factory become painful to extend?
3. Name two reasons to make a builder's product immutable.
4. A service with a "singleton" cache runs 30 pods with 2 processes each. How many caches exist?
5. Why does the naive singleton in §37.6 create several instances under concurrency?

6. How do Strategy and State differ, given that their class diagrams are almost identical?
7. List two differences between Observer and broker-based publish/subscribe.
8. Classify each as Adapter, Decorator or Proxy: a retry wrapper; a gRPC client stub; a wrapper converting a vendor SDK's dictionaries into domain objects.
9. Name two uses of Python's `@` decorator syntax that are not the GoF Decorator.
10. Why does running a new command clear the redo stack?
11. What keeps an in-memory fake repository honest?
12. What is the difference between dependency injection and a service locator?
13. Using the sizing formula, how many threads for 16 cores, 50% target utilization, $W = 40$ ms and $C = 10$ ms?
14. Why should authentication run before per-user rate limiting in a middleware chain?
15. What exactly goes wrong in Java double-checked locking without `volatile`?

Answers

1. It moves the "which concrete class?" decision, driven by run-time data, into one place, so callers depend only on the interface and new variants do not touch call sites.
2. When you add a new product type: every concrete factory must implement a new creation method.
3. Invariants checked in `build()` cannot later be broken; immutable objects are thread-safe to share and safe to use as cache keys or to retry.
4. $30 \times 2 = 60$ independent caches, each with its own contents.
5. Check-then-act is not atomic: all threads read `None` before any assigns, and each constructs an instance.
6. A strategy is chosen by the client from outside and usually stays fixed; a state object changes the context's state itself as transitions occur.
7. Observer is synchronous and in-process with no durability; pub/sub through a broker is asynchronous, durable, and decouples publisher and subscribers in time and deployment.
8. Retry wrapper: Decorator. gRPC stub: Proxy (remote). SDK translator: Adapter.
9. Registration (`@app.route`), `@property`, `@dataclass` and `@staticmethod` transform or register rather than wrap an object behind the same interface.
10. Redo replays undone commands on the state they were undone from; after a new command, that state no longer exists, so replaying them would corrupt the history.
11. A shared contract test suite run against both the fake and the real implementation.
12. With DI, dependencies are passed in and visible in the constructor; with a service locator, the object fetches them from a registry, hiding them and coupling it to the locator.
13. $16 \times 0.5 \times (1 + 40/10) = 16 \times 0.5 \times 5 = 40$ threads.
14. Unauthenticated (possibly forged) requests would otherwise consume a real user's quota, enabling a cheap denial of service against that user; also, user identity is not

known until authentication runs.
15. The write publishing the reference can become visible before the constructor's field writes; a thread on the unlocked fast path sees a non-null reference to a partially constructed object.

37.20 Interview questions

Basic

Q1. What is the Strategy pattern? Give a system-design example. Answer. It encapsulates interchangeable algorithms behind one interface, chosen at run time. Examples: load-balancing policies (round robin, least connections) in a proxy, cache eviction policies (LRU, LFU), and retry backoff policies. It keeps the context closed to modification and makes each algorithm testable alone.

Q2. Why is Singleton considered an anti-pattern? Answer. It is global mutable state: dependencies are hidden, tests leak state into one another, two configurations cannot coexist, and it invites contention. It also misleads in distributed systems, being one per process, not per system. Create one instance in the composition root and inject it instead.

Intermediate

Q3. Compare Adapter, Decorator and Proxy. Answer. All wrap another object. Adapter presents a *different* interface to make incompatible code fit; Decorator presents the *same* interface and adds behavior, and decorators stack; Proxy presents the same interface and controls access (lazy creation, permissions, remoteness). Infrastructure analogues: an anti-corruption layer (adapter), a caching or retrying client (decorator), an API gateway or sidecar (proxy).

Q4. How would you make an `OrderService` unit-testable if it currently constructs a database client, an HTTP payment client and calls `datetime.now()`? Answer. Define ports for the repository, payment gateway and clock; implement adapters for production; change `OrderService` to receive them via its constructor; wire them in a composition root; in tests pass an in-memory repository, a fake gateway and a fixed clock. Add contract tests so the fakes match the real adapters.

Advanced

Q5. Design the in-process notification pipeline for a service that sends 2,000 receipts per second at peak through a provider with 100 ms latency and a 1,500 requests/s limit. Answer. Request threads produce Commands into a bounded queue (producer–consumer); a sender pool consumes them. By Little's Law the provider limit al-

lows $1,500 \times 0.1 = 150$ requests in flight, so about 150 sender threads (fewer with an asynchronous client). Because 2,000/s exceeds 1,500/s, the queue grows at 500/s during the peak; bound it by memory (100,000 items ≈ 200 s of peak excess), and beyond that reject or spill to a durable broker. Wrap the client in decorators for retry with jitter and a circuit breaker, and make sends idempotent by order ID.

Q6. A Java service occasionally reads a configuration timeout of 0 under load. The config is lazily created with double-checked locking. Explain and fix. Answer. Without `volatile`, the reference to the config can be published before its constructor's writes are visible; a thread on the unlocked fast path sees a non-null, partially constructed object with default field values. Fix by declaring the field `volatile`, or better, use the initialization-on-demand holder idiom or eager initialization, and make the config's fields `final`.

37.21 Practical exercises

1. Add a `WhatsAppNotifier` to §37.3 by registering it from a separate module at import time. Which files changed?
2. Add a cloud family to the abstract factory in §37.4 with fake network latency, and write one contract test that runs against all three families.
3. Reorder the decorators in §37.10 to `RetryingPriceSource(CachingPriceSource(db))` and predict, then measure, the database call count.
4. Extend the Command example with an `ApplyCoupon` command whose undo restores the previous total, and persist the history to a JSON file.
5. Replace the read-write lock in §37.16.4 with copy-on-write (build a new dictionary, swap a reference) and compare reader throughput with 8 readers.
6. Vary `max_workers` in §37.16.2 from 1 to 64 and plot elapsed time; explain the curve with the sizing formula.

37.22 Design challenges

1. **Pluggable payments.** Design the payment module of a store that supports three processors, per-country routing, retries with idempotency keys, and a test mode. Identify where you would use Adapter, Strategy, Decorator, Abstract Factory and DI, and draw the class diagram.
2. **Collaborative editor backend.** Support undo/redo per user, an audit log, and asynchronous persistence of edits. Decide what the Command objects contain, where they are queued, and how undo interacts with other users' later edits.

37.23 References

1. Gamma, E., Helm, R., Johnson, R., Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
2. Freeman, E., Robson, E. *Head First Design Patterns*, 2nd ed. O'Reilly Media, 2020.
3. Fowler, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
4. Fowler, M. "Inversion of Control Containers and the Dependency Injection pattern." martinfowler.com, 2004. https://martinfowler.com/articles/injection.html
5. Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., Lea, D. *Java Concurrency in Practice*. Addison-Wesley, 2006.
6. Schmidt, D., Stal, M., Rohnert, H., Buschmann, F. *Pattern-Oriented Software Architecture, Volume 2: Patterns for Concurrent and Networked Objects*. Wiley, 2000.
7. Bloch, J. *Effective Java*, 3rd ed. Addison-Wesley, 2018.
8. Evans, E. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.
9. Bacon, D., Bloch, J., Bogda, J., et al. "The 'Double-Checked Locking is Broken' Declaration." https://www.cs.umd.edu/~pugh/java/memoryModel/DoubleCheckedLocking.html
10. Manson, J., Pugh, W., Adve, S. "The Java Memory Model." *ACM POPL*, 2005.
11. Norvig, P. "Design Patterns in Dynamic Programming." Presentation, Object World, 1996. https://norvig.com/design-patterns/
12. Brooker, M. "Exponential Backoff And Jitter." AWS Architecture Blog, 2015.
13. Smith, K. D., et al. "PEP 318 – Decorators for Functions and Methods." 2003. https://peps.python.org/pep-0318/
14. Gross, S. "PEP 703 – Making the Global Interpreter Lock Optional in CPython." 2023. https://peps.python.org/pep-0703/
15. Django Software Foundation. *Django documentation: Middleware*. https://docs.djangoproject.com/en/stable/topics/http/middleware/
16. Spring. *Spring Framework Reference: The IoC Container; Proxying Mechanisms*. https://docs.spring.io/spring-framework/reference/
17. Spring. *Spring Data JPA Reference: Repositories*. https://docs.spring.io/spring-data/jpa/reference/
18. Python Software Foundation. *Python 3 Standard Library:* `queue`, `concurrent.futures`, `threading`, `functools`, `logging`. https://docs.python.org/3/library/

37.24 Open verification items

- Norvig (1996): exact presentation title, venue and the "16 of 23 patterns" figure; URL of the slides.

- Bacon et al. declaration: full author list and date.
- PEP 318 author attribution (Kevin D. Smith, Jim Jewett, Skip Montanaro, Anthony Baxter).
- Spring documentation wording recommending constructor injection for mandatory dependencies (as of 2026).
- Spring proxy defaults: JDK dynamic proxy for interface-implementing beans in Spring Framework versus class-based (CGLIB) proxies by default in Spring Boot (as of 2026).
- PEP 703 free-threaded build status in current CPython releases (experimental in 3.13; status as of 2026).

Chapter 38 — UML and Low-Level Design

> **Level:** 2–3 · **Track:** A (Traditional) · **Estimated reading time:** 4 h

Chapters 35–37 gave you the vocabulary of object-oriented design: classes and interfaces, the SOLID principles, and the Gang of Four patterns. This chapter teaches you to *draw* a design so that other engineers can check it in minutes, and then to *produce* one under interview conditions. The first half covers the five Unified Modeling Language (UML) diagrams that working engineers actually use (class, sequence, component, state and activity) and the relationships between objects that trip people up most often. The second half is a method for low-level design (LLD) interviews and eight fully worked exercises, from the parking lot to a chess move validator, each with requirements, diagrams, runnable Python, the patterns used and the follow-up questions an interviewer is likely to ask.

Throughout, the diagrams are written in Mermaid, a text-based diagram language. Mermaid's class, sequence and state diagrams are close to UML notation; it has no component or activity diagram, so those two are drawn as flowcharts that approximate UML, and the text says so each time.

38.1 Learning objectives

After this chapter you should be able to:

- Read and draw UML class diagrams with correct visibility, multiplicity and the six relationship kinds (association, aggregation, composition, generalization, realization, dependency).
- Draw sequence diagrams with lifelines, activations, synchronous and asynchronous messages, and `alt` / `opt` / `loop` fragments for a real request flow.
- Draw component, state machine and activity diagrams, and choose the right diagram for a given question.
- Decide between association, aggregation and composition, and between modeling something as an attribute or as a relationship, and express each choice in code.
- Run a 45-minute LLD interview using a nine-step method, and self-assess against a rubric.
- Design, diagram and implement eight classic LLD problems with thread safety and extension points, and defend the patterns you chose.

38.2 Prerequisites

- Classes, encapsulation, inheritance, polymorphism and composition over inheritance — Chapter 35 (UML class diagrams are a notation for exactly these ideas).
- SOLID principles — Chapter 36 (every design review in §38.10 is argued in SOLID terms).
- Design patterns, especially Strategy (§37.7), Observer (§37.8), Decorator (§37.10), Factory (§37.3), State (§37.15) and the concurrency patterns of §37.16 — the exercises apply them.
- Mutexes, race conditions and lock striping — §6.9–§6.10 (thread safety in the exercises).
- Token bucket — §22.4 (the rate limiter exercise wraps it in a class design without re-deriving it).
- LRU and TTL as cache policies — §20.8–§20.9 (the LRU and key-value store exercises implement them).
- Optimistic concurrency — §11.15.3 (the key-value store's transactions use it).

38.3 Class diagrams

38.3.1 What UML is and why it exists

DEFINITION

Definition — Unified Modeling Language (UML). A general-purpose graphical language for specifying, visualizing and documenting software systems, standardized by the Object Management Group (OMG). The current version, UML 2.5.1 (December 2017), defines 14 diagram types divided into *structure* diagrams (what the system is made of) and *behavior* diagrams (what it does over time) [OMG 2017].

First-principles explanation

An architect does not hand a builder a written essay about a house; they hand over a floor plan, an elevation and a wiring diagram, each answering one question with conventions every builder already knows. A solid wall line, a door arc and a socket symbol mean the same thing on every plan. UML plays that role for object-oriented software: a hollow triangle always means "is a kind of", a filled diamond always means "owns and controls the lifetime of". Once both people know the conventions, a picture of 12 classes conveys in 30 seconds what 300 lines of code take 10 minutes to reveal.

Why does this exist?

By the early 1990s there were dozens of competing object-oriented notations. Grady Booch, James Rumbaugh and Ivar Jacobson merged their methods at Rational Software, and OMG adopted the result as UML 1.1 in 1997. UML 2.0 (2005) reworked sequence and activity diagrams substantially, and 2.5 (2015) simplified the specification without changing most notation [OMG 2017; Fowler 2003; Rumbaugh et al. 2004].

Fowler distinguishes three ways people use UML: as a **sketch** (informal, partial diagrams to communicate an idea), as a **blueprint** (complete diagrams that a programmer implements) and as a **programming language** (diagrams compiled to code) [Fowler 2003]. Almost all industry use, and all interview use, is sketching. The goal is not to satisfy the 790-page specification; it is to make the important decisions visible and unambiguous. This chapter follows the standard notation where it matters (relationships, multiplicity, message kinds) and relaxes it where it does not (exact fonts, every adornment).

Table 38.1 — The 14 UML 2.5.1 diagram types and how often you need them.

Family	Diagram	Answers the question	Use in practice
Structure	Class	What types exist and how are they related?	Daily; every LLD interview
Structure	Object	What does one snapshot of instances look like?	Occasionally, to explain a tricky class diagram
Structure	Package	How is the code grouped into modules?	Occasionally
Structure	Component	Which replaceable parts provide and require which interfaces?	Architecture reviews (§38.5)
Structure	Composite structure	What is inside one class at run time?	Rare
Structure	Deployment	Which artifacts run on which nodes?	Usually replaced by infrastructure diagrams
Structure	Profile	How is UML itself extended?	Tool builders only
Behavior	Use case	Which actors want which goals?	Requirements; often a bullet list instead
Behavior	Activity	What is the workflow, including parallel steps?	Business processes, pipelines (§38.7)
Behavior	State machine	How does one object react to events over its lifetime?	Lifecycles, protocols (§38.6)
Interaction	Sequence	Who calls whom, in what order?	Daily; every LLD interview (§38.4)
Interaction	Communication	Same as sequence, laid out by links	Rare
Interaction	Timing	How do states change against a time axis?	Hardware, real-time systems
Interaction	Interaction overview	How do several interactions combine?	Rare

The three interaction diagrams are a sub-family of behavior diagrams, so the split is 7 structure and 7 behavior diagrams. Five of them (class, sequence, component, state machine, activity) cover nearly everything you will draw.

38.3.2 Classes, visibility and multiplicity

DEFINITION

Definition — Class diagram. A structure diagram showing classifiers (classes, interfaces, enumerations), their **attributes** and **operations**, and the **relationships** among them. A class is drawn as a box with up to three compartments: name, attributes, operations.

How it works internally

Each attribute is written `visibility name : Type [multiplicity] = default`, and each operation `visibility name(parameter : Type) : ReturnType`. Mermaid uses the Java-like order `Type name` and `name() ReturnType`, which is also common in hand-drawn sketches; either is fine as long as it is consistent. Abstract classes and operations are written in *italics*; static members are underlined. Stereotypes in guillemets such as «interface», «abstract» or «enumeration» classify a box.

Multiplicity says how many instances may be at the far end of a relationship (or in a multi-valued attribute). It is written as `lower..upper`, where `*` means “unbounded”.

Table 38.2 — Class diagram notation: visibility, multiplicity and adornments.

Notation	Meaning	Python convention	Mermaid syntax
`+`	Public: visible to everyone	`name`	`+name`
`-`	Private: visible only inside the class	`_name` (or `__name`)	`-name`
`#`	Protected: visible to the class and subclasses	`_name` by convention	`#name`
`~`	Package: visible inside the same package or module	module-level, not exported	`~name`
italic	Abstract class or operation	`ABC`, `@abstractmethod`	`<<abstract>>`, `method()*`
underline	Static (class-level) member	class attribute, `@staticmethod`	`name$`
`1`	Exactly one	required reference	`"1"`
`0..1`	Zero or one (optional)	`Optional[X]`	`"0..1"`
`*` or `0..*`	Zero or more	`list[X]`, `dict[K, X]`	`"*"`
`1..*`	One or more	non-empty collection, checked in `__init__`	`"1..*"`
`2..4`	A bounded range	collection with a validated size	`"2..4"`

COMMON MISTAKE

Mistake — putting multiplicity at the wrong end. In `Order "1" --> "1..*" LineItem`, the `1..*` sits next to `LineItem` and is read from the *Order's* point of view: "an order has one or more line items". Students often read it the other way round. Read every association twice, once from each end: "each order has 1..* line items; each line item belongs to exactly 1 order."

38.3.3 The six relationships

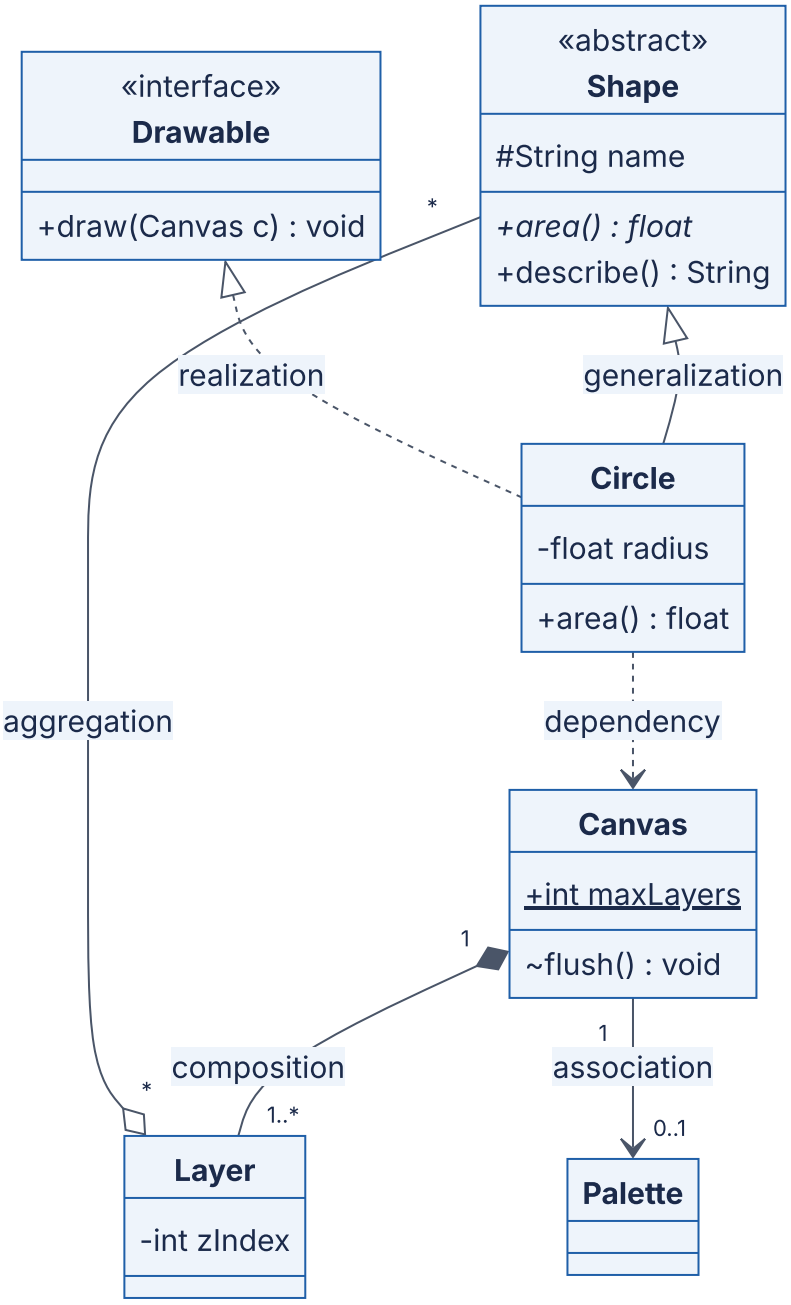
Relationships carry most of the meaning in a class diagram. UML defines more, but six cover practically every design.

Table 38.3 — The six class relationships.

Relationship	Line and end symbol	Meaning	Typical code	Mermaid
Association	Solid line; open arrowhead for navigability	A structural, lasting link: one object holds a reference to the other	Field `self.owner: Customer`	`A --> B`
Aggregation	Solid line, hollow diamond at the whole	Whole–part where the part can be shared and outlives the whole	Field holding objects created elsewhere	`Whole o-- Part`
Composition	Solid line, filled diamond at the whole	Whole–part with exclusive ownership; parts die with the whole	Whole creates parts privately	`Whole *-- Part`
Generalization (inheritance)	Solid line, hollow triangle at the parent	"Is a kind of"; subclass inherits and may override	`class Car(Vehicle)`	`Parent <|-- Child`
Realization	Dashed line, hollow triangle at the interface	A class implements an interface's contract	`class S3Store(BlobStore)` with ABC	`Interface <|.. Impl`
Dependency	Dashed line, open arrowhead	Transient use: a parameter, local variable or return type	`def render(self, c: Canvas)`	`A ..> B`

The relationships form a scale of coupling strength. From weakest to strongest: dependency (uses it for one call), association (remembers it), aggregation (contains it, shared), composition (owns it exclusively), and generalization (*is* it — the strongest coupling of all, which is why §35.8 recommends composition over inheritance). Realization couples only to a contract, so it is the relationship that the Dependency Inversion Principle (§36.7) wants you to draw most often [Martin 2002; Martin 2017].

Figure 38.1 — Class diagram notation: visibility, multiplicity and the six relationships.



Reading the diagram. `Shape` is an abstract class (stereotype and the `*` on `area()` mark it abstract) with a protected `name`; `Circle` inherits from it (solid line, hollow triangle at `Shape`) and also realizes the `Drawable` interface (dashed line, hollow triangle). A `Canvas` *composes* one or more `Layer`s (filled diamond at `Canvas`): layers are created by the canvas and deleted with it. Each `Layer` *aggregates* any number of shapes (hollow diamond), and one shape can appear on several layers, so it is shared. The canvas has a navigable *association* to an optional `Palette` (arrow toward `Palette`, multiplicity `0..1`). `Circle` merely *depends* on `Canvas` (dashed arrow): it receives one as a parameter in `draw` and does not keep it. The static `maxLayers` and package-visible `flush()` show the remaining

adornments. The diagram has no run-time direction or storage; it is a static map of compile-time coupling. If you change `Canvas`, every class with an arrow pointing at it (here `Circle`) may need to change, which is how a class diagram exposes change impact.

Common misconceptions

COMMON MISTAKE

Mistake — drawing every class and every getter. A class diagram that lists 40 classes with all their accessors hides the five decisions that matter. Show the classes that participate in the problem, the operations that define their responsibilities, and every relationship with multiplicity. Omit getters, setters and obvious constructors.

COMMON MISTAKE

Mistake — using inheritance for roles. `Librarian extends Member` looks natural until a librarian borrows a book under a different policy or a member becomes a librarian on Tuesday. Roles that an object can gain or lose are associations to a `Role` object (or a Strategy, §37.7), not subclasses. Generalization is for permanent "kind of" relationships that pass the Liskov test (§36.5).

INDUSTRY EXAMPLE

Diagrams as code on GitHub. Since February 2022 GitHub renders Mermaid code blocks in Markdown files, issues and pull requests, so diagrams can live next to the code and be reviewed as text diffs [GitHub 2022]. A diagram versioned with the code is far more likely to be kept current. *Evidence: Primary* — [GitHub 2022]

38.4 Sequence diagrams

38.4.1 Lifelines, messages and fragments

DEFINITION

Definition — Sequence diagram. An interaction diagram that shows objects or participants as vertical **lifelines** and the **messages** they exchange as horizontal arrows ordered top to bottom in time. **Execution specifications** (activation bars) show when a participant is busy, and **combined fragments** (`alt`, `opt`, `loop`, `par` and others) show conditional, optional, repeated and parallel behavior [OMG 2017].

First-principles explanation

A class diagram is a cast list; a sequence diagram is the script of one scene. It answers "when the user clicks *Buy*, who calls whom, in what order, and what comes back?" Time flows downward, so you can read the diagram like a story. It is the diagram that most often reveals a missing responsibility: if no lifeline can answer a message, a class is missing; if one lifeline receives every message, you have found a God object.

How it works internally

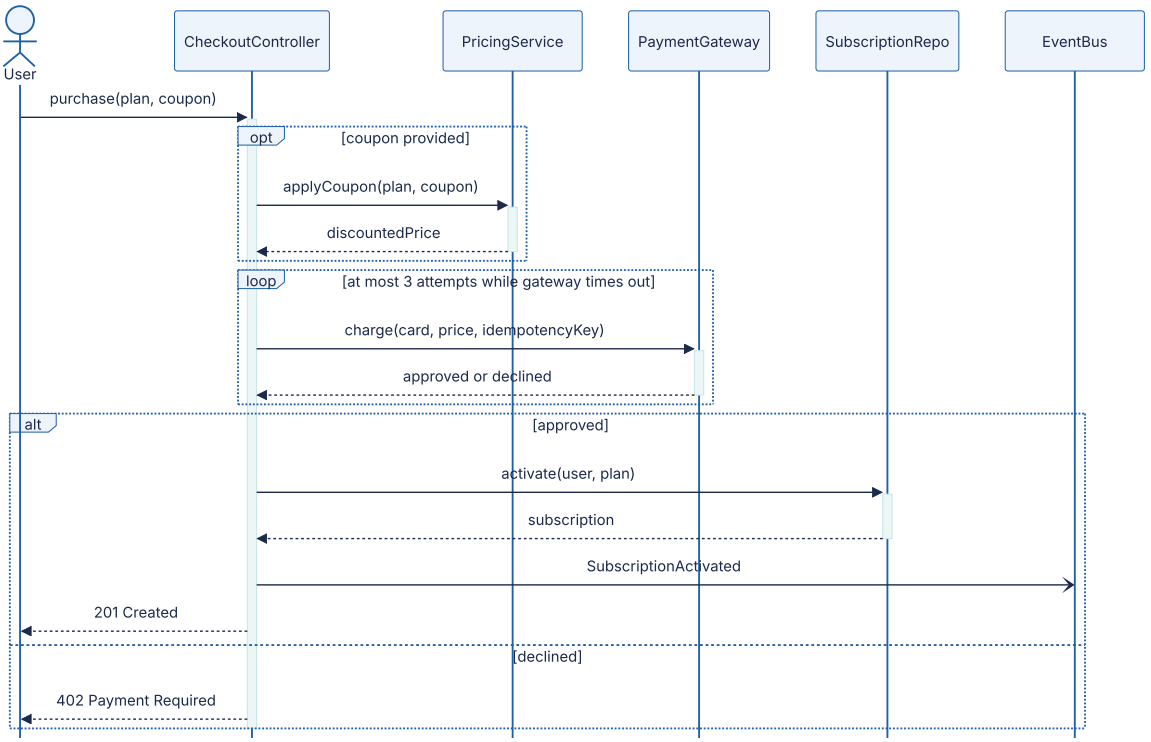
1. Place participants left to right in the order they first act; put the initiator (an actor) on the far left.
2. Draw each message as an arrow from caller to callee, labeled with the operation and key arguments.
3. A **synchronous** message (solid line, filled arrowhead) blocks the caller until the reply; a **reply** is a dashed line with an open arrowhead; an **asynchronous** message (solid line, open arrowhead) lets the caller continue immediately.
4. Activate the callee (a thin bar) while it is processing; nested bars show nested calls.
5. Wrap conditional or repeated sections in combined fragments, each labeled with its operator and a guard in square brackets.

Table 38.4 — Sequence diagram elements and their Mermaid syntax.

Element	UML notation	Meaning	Mermaid
Lifeline	Box with a dashed vertical line	One participant for the scene's duration	`participant X` / `actor U`
Execution (activation)	Thin bar on the lifeline	Participant is executing	`->>+` and `-->>-`, or `activate X`
Synchronous message	Solid line, filled arrowhead	Caller waits for the reply	`A->>B: op()`
Reply	Dashed line, open arrowhead	Return value	`B-->>A: result`
Asynchronous message	Solid line, open arrowhead	Fire and forget	`A-)B: event`
`alt` fragment	Box split by dashed lines, one guard per part	If/else: exactly one operand runs	`alt ... else ... end`
`opt` fragment	Box with one guard	If without else	`opt ... end`
`loop` fragment	Box with a guard or bounds	Repeat while the guard holds	`loop ... end`
`par` fragment	Box split into concurrent operands	Operands run in parallel	`par ... and ... end`
`break` / `critical`	Labeled box	Exit the enclosing interaction / atomic region	`break ... end`, `critical ... end`

UML 2.5.1 defines twelve interaction operators (also `seq`, `strict`, `neg`, `assert`, `ignore` and `consider`), but the six above appear in nearly every real diagram.

Figure 38.2 — Sequence diagram for a PhotoShare Pro purchase with opt, loop and alt fragments.



Reading the diagram. The *User* (an actor) synchronously calls `purchase` on the *CheckoutController*, which stays active for the whole scene. The `opt` fragment runs only when a coupon is supplied: the *PricingService* returns a discounted price. The `loop` fragment retries the *PaymentGateway* charge up to three times on timeout; the idempotency key makes those retries safe (§22.15), which is the most important detail an interviewer will probe. The `alt` fragment splits on the result: on approval the controller persists the subscription through the *SubscriptionRepo* (the storage boundary) and publishes `SubscriptionActivated` *asynchronously* to the *EventBus* (open arrowhead: it does not wait), then replies 201; on decline it replies 402. Failure points are visible as lifelines that can be slow: the external gateway and the repository. Scaling is not shown in a sequence diagram; it describes one request.

> INTERVIEW TIP
>
> **Tip — draw the unhappy path.** Interviewers rarely ask about the path where everything works. One `alt` for “declined” and one `loop` or `break` for “timeout” show that you think about failure, and they take ten seconds to draw.

COMMON MISTAKE

Mistake — one sequence diagram for the whole system. A sequence diagram should cover one use case, one scenario, ideally fewer than 10 lifelines and 25 messages. If it is larger, split it by use case or replace a group of lifelines with one participant (“PaymentSubsystem”) and draw that separately.

38.5 Component diagrams

38.5.1 Components and interfaces

DEFINITION

Definition — Component diagram. A structure diagram that shows **components** (modular, replaceable units that encapsulate their contents) and the **interfaces** they *provide* (offer to others, drawn as a “lollipop” circle) and *require* (need from others, drawn as a “socket” half-circle), connected by **assembly connectors** that wire a required interface to a matching provided one [OMG 2017].

First-principles explanation

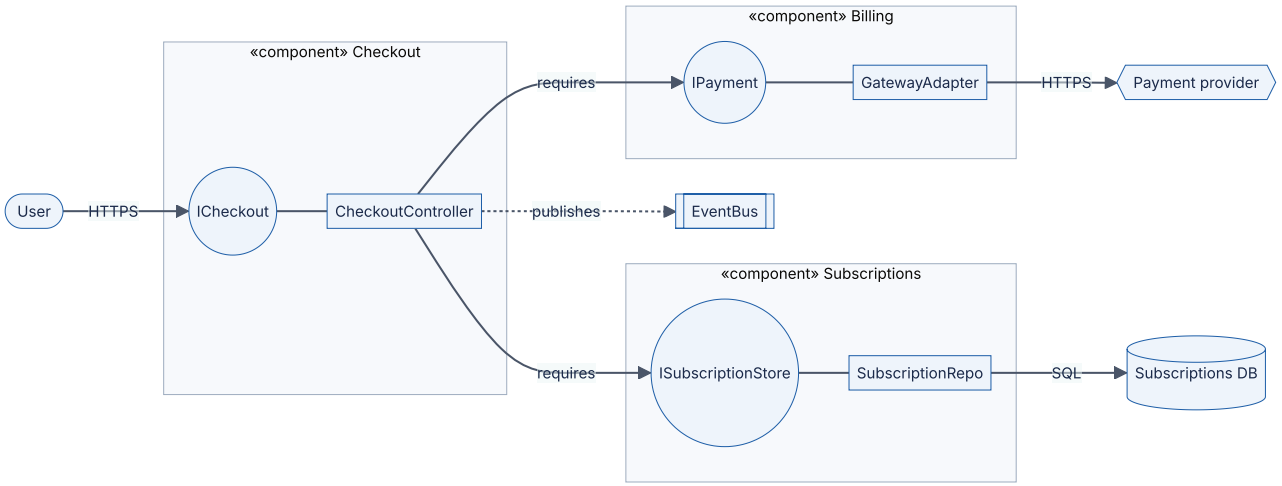
Think of household appliances and wall sockets. The kettle requires “230 V, type G plug”; the wall provides it. You can replace the kettle with any other appliance that requires the same socket, and you can rewire the house without touching the kettle. A component diagram draws software at that granularity: modules, libraries or services, and only the contracts between them, not their internal classes.

Why does this exist?

A class diagram of a 200-class system is unreadable, and a deployment diagram shows machines, not responsibilities. The component diagram sits between them: it shows which parts can be built, tested and replaced independently, which is exactly the question behind modular monoliths (§33.5), hexagonal architecture’s ports and adapters (§33.10) and the Dependency Inversion Principle at the architecture level (§36.8).

Mermaid has no component diagram, so Figure 38.3 approximates UML notation with a flowchart: each `subgraph` is a component, circle nodes are provided interfaces, and edges labeled “requires” are required interfaces wired by an assembly connector.

Figure 38.3 — Component view of the PhotoShare Pro checkout (flowchart approximation of UML component notation).



Reading the diagram. Three components appear as boxes. *Checkout* provides the `ICheckout` interface (circle) to the user over HTTPS and *requires* two interfaces: `IPayment`, provided by the *Billing* component, and `ISubscriptionStore`, provided by *Subscriptions*. The "requires" edges are the assembly connectors. Checkout depends only on the interface circles, never on `GatewayAdapter` or `SubscriptionRepo` directly, so either can be replaced (a different payment provider, a different database) without touching Checkout. Solid edges are synchronous calls; the dotted edge to the *EventBus* is asynchronous publication. Storage and external boundaries are the subscriptions database and the external payment provider, which are also the main failure points. Scaling: each component could be deployed as its own service later, and the interfaces become the service APIs.

RECOMMENDATION

Recommendation — name interfaces after the client's need. `IPayment` with `charge()` and `refund()` describes what Checkout needs; `IStripeClient` describes one vendor's implementation and leaks it into every client. Interfaces owned by the client side are the core of Dependency Inversion (§36.7).

38.6 State diagrams

38.6.1 States, transitions, guards and actions

DEFINITION

Definition — State machine diagram. A behavior diagram that shows the **states** one object (or protocol) can be in and the **transitions** between them. A transition is labeled `trigger [guard] / effect`: the **trigger** is the event, the **guard** a Boolean condition that must be true for the transition to fire, and the **effect** an action executed during the transition. A state may declare **entry**, **exit** and **do** behaviors, run on entering, on leaving, and while inside the state [OMG 2017].

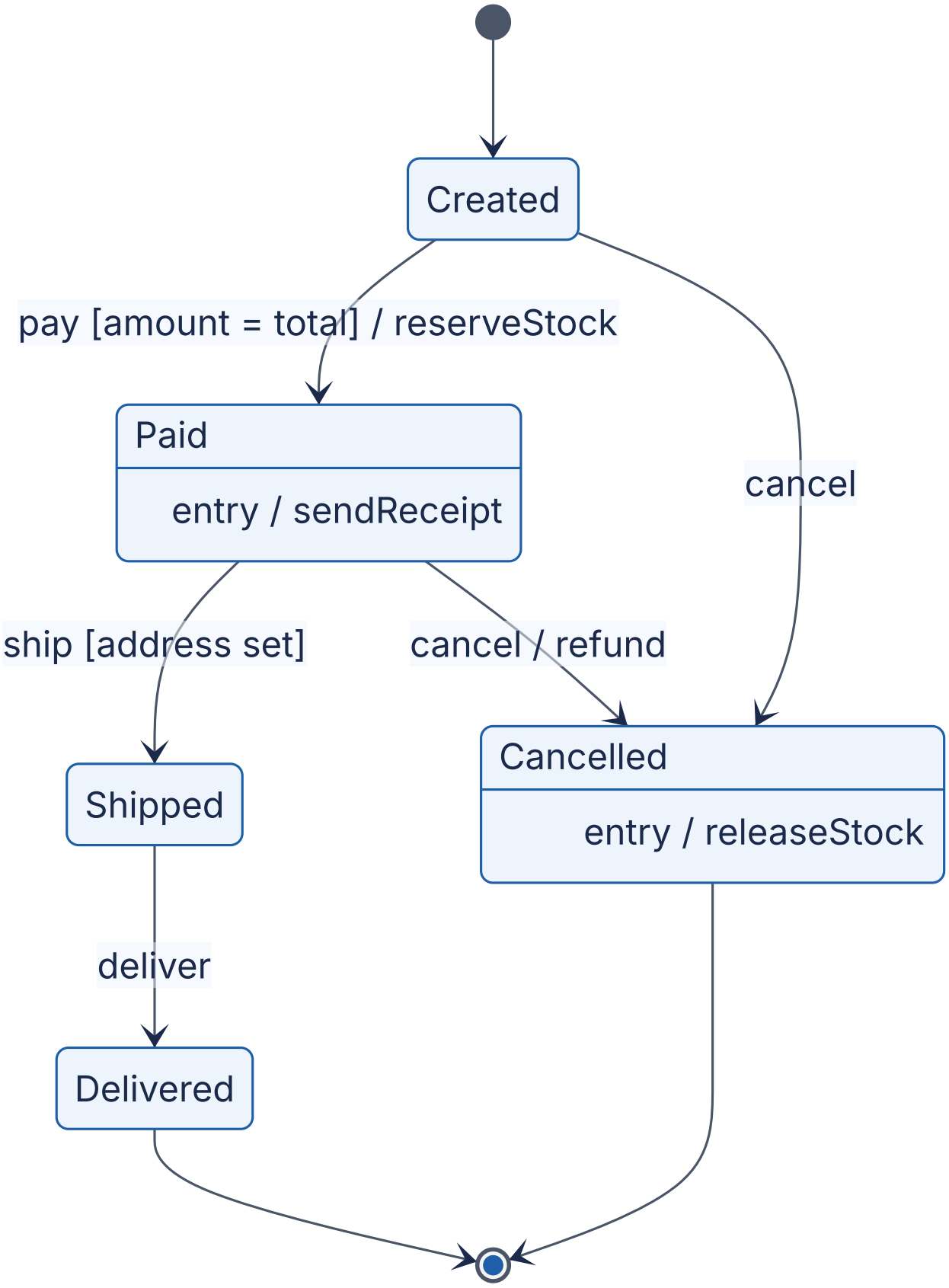
First-principles explanation

A traffic light is a state machine: three states, one event (a timer tick) and a fixed order. A pressed pedestrian button is a guard: "on tick, *if* someone is waiting, go to red". Business objects behave the same way. An order cannot be shipped before it is paid; a refund only makes sense for a paid order. Writing these rules as a diagram makes every illegal combination obvious: if there is no arrow labeled `ship` out of `Created`, shipping an unpaid order is impossible by construction.

How it works internally

1. A filled circle (the **initial pseudostate**) points to the first state; a bullseye is a **final state**.
2. When an event arrives, the machine looks for a transition out of the current state with that trigger.
3. If the guard is false (or no transition matches), the event is ignored or rejected, and the state does not change.
4. Otherwise it runs the current state's exit behavior, then the transition's effect, then the target state's entry behavior.
5. **Composite states** nest a sub-machine (for example, `Shipped` containing `InTransit` and `OutForDelivery`), and a **choice pseudostate** (a diamond) branches on guards after a single trigger.

Figure 38.4 — State machine for an order, with guards, effects and entry actions.



Reading the diagram. An order starts in *Created*. The `pay` event moves it to *Paid* only if the guard `amount = total` holds; the transition's effect reserves stock, and entering *Paid*

sends a receipt. From *Paid* it can be shipped (guarded by a known address) or canceled, and cancellation from *Paid* has a refund effect that cancellation from *Created* does not need. Entering *Cancelled* always releases stock. *Delivered* and *Cancelled* lead to the final state. Any event without an outgoing arrow is illegal in that state, so “cancel a delivered order” is rejected. There is no network or storage in this diagram; in a real system the state is persisted with the order, and every transition should be a single atomic update (for example `UPDATE ... SET state = 'PAID' WHERE id = ? AND state = 'CREATED'`) so that two concurrent events cannot both fire from the same state.

Code example

A transition table maps directly from the diagram to code, which keeps them in sync:

```
        from enum import Enum, auto


        class S(Enum):
            CREATED = auto()
            PAID = auto()
            SHIPPED = auto()
            DELIVERED = auto()
            CANCELLED = auto()


        class Order:
            # (state, event) -> (guard, next_state, effect); a guard of None means "al-
ways"
            TRANSITIONS = {
                (S.CREATED, "pay"):     (lambda o, amt: amt == o.total, S.PAID,
"reserve_stock"),
                (S.CREATED, "cancel"):  (None, S.CANCELLED, None),
                (S.PAID, "ship"):       (lambda o, _: o.address is not None, S.SHIPPED,
None),
                (S.PAID, "cancel"):     (None, S.CANCELLED, "refund"),
                (S.SHIPPED, "deliver"): (None, S.DELIVERED, None),
            }
            ENTRY = {S.PAID: "send_receipt", S.CANCELLED: "release_stock"}

            def __init__(self, total, address=None):
                self.state, self.total, self.address, self.log = S.CREATED, total, ad-
dress, []

            def fire(self, event, arg=None):
                rule = self.TRANSITIONS.get((self.state, event))
                if rule is None:
                    raise ValueError(f"event {event!r} not allowed in
{self.state.name}")
                guard, target, effect = rule
                if guard is not None and not guard(self, arg):
                    return False                        # guard false: no transition,
state unchanged
                if effect:
                    self.log.append(effect)             # transition effect runs first...
                self.state = target
                if target in self.ENTRY:
                    self.log.append(self.ENTRY[target])  # ...then the target's entry
action
                return True


        if __name__ == "__main__":
            o = Order(total=50, address="1 Main St")
            assert o.fire("pay", 40) is False and o.state is S.CREATED   # guard rejects
            assert o.fire("pay", 50) and o.state is S.PAID
            assert o.fire("ship") and o.fire("deliver")
            try:
                o.fire("cancel")
            except ValueError as e:
```

```
                print("rejected:", e)
            print(o.state.name, o.log)
```

Output: `rejected: event 'cancel' not allowed in DELIVERED`, then `DELIVERED ['reserve_stock', 'send_receipt']`.

What to notice:

- Each row of `TRANSITIONS` is one arrow of Figure 38.4, so reviewing the table against the diagram is mechanical.
- A false guard returns `False` without changing state; an event with no arrow raises. Distinguishing "not now" from "never in this state" helps callers respond correctly.
- The order of actions (exit, transition effect, entry) follows UML's run-to-completion semantics. For behavior that differs a lot per state, the State pattern (§37.15 and the vending machine in §38.10.4) replaces the table with one class per state.

38.7 Activity diagrams

38.7.1 Actions, forks, joins and swimlanes

DEFINITION

Definition — Activity diagram. A behavior diagram that models a workflow as **actions** connected by **control flows**, with **decision** and **merge** nodes for branching, **fork** and **join** bars for concurrency, and **activity partitions** (swimlanes) that assign actions to the participant responsible for them. Its semantics are based on tokens flowing through the graph, similar to a Petri net [OMG 2017].

First-principles explanation

A state diagram follows one object through its life; an activity diagram follows a piece of work through an organization. A recipe is an activity diagram: "boil water" and "chop vegetables" can happen in parallel (fork), both must finish before "add vegetables" (join), and "if the sauce is thick, add water" is a decision. Swimlanes add *who*: the chef chops, the kitchen porter boils.

How it works internally

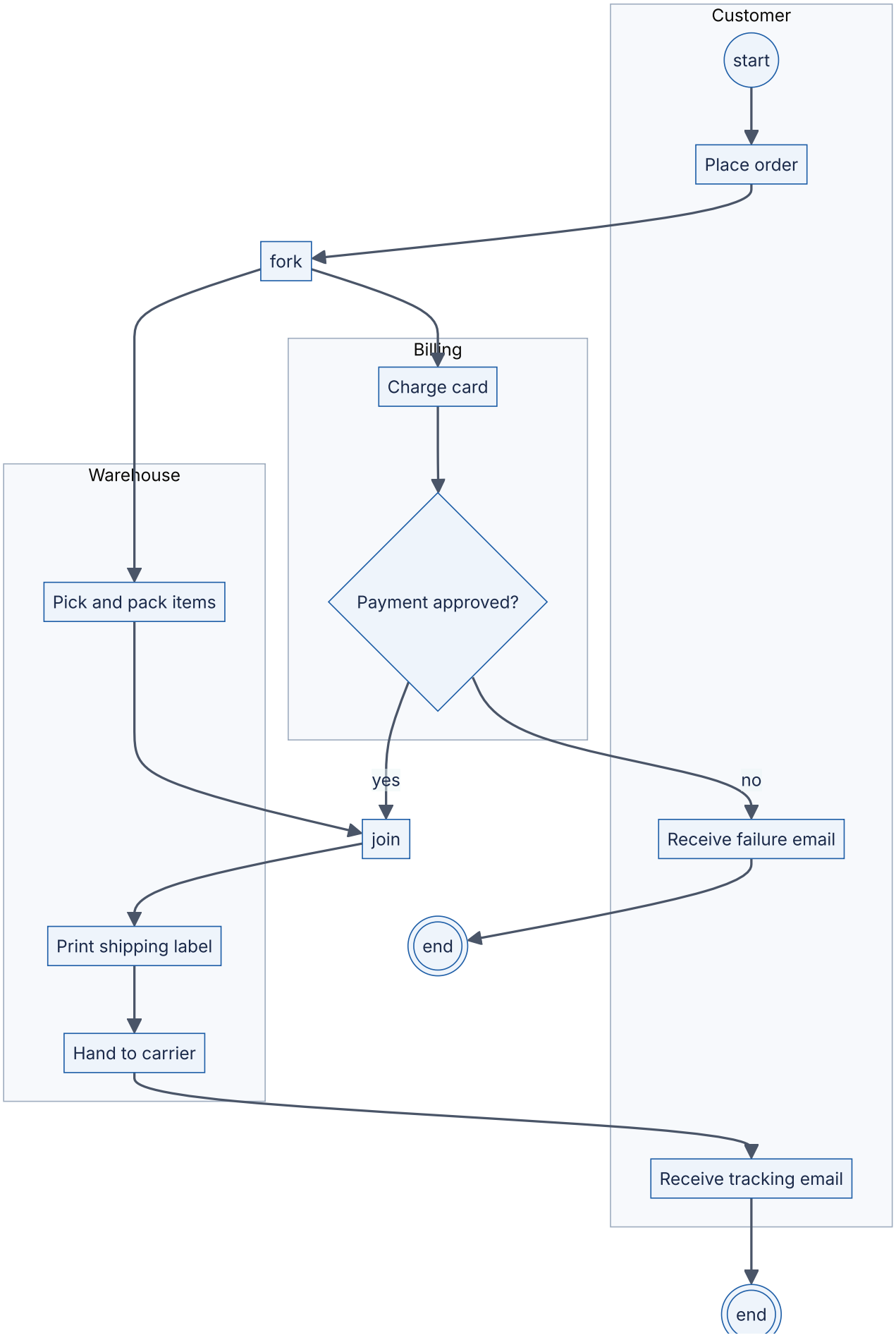
1. An initial node emits one token. Actions run when a token arrives and pass it on when they finish.
2. A **decision** sends the token down exactly one outgoing edge whose guard is true; a **merge** joins alternative paths without waiting.

3. A **fork** copies the token onto every outgoing edge, so those branches run concurrently. A **join** waits until a token has arrived on every incoming edge, then emits one.
4. An **activity final** node ends the whole activity; a **flow final** ends only the branch that reaches it.

Confusing a merge with a join is the classic error: a join after a decision waits forever for the branch that was never taken.

Mermaid has no activity diagram, so Figure 38.5 approximates UML notation with a flowchart: subgraphs are swimlanes, nodes labeled *fork* and *join* stand for the UML bars, and the circles are the initial and final nodes.

Figure 38.5 — Activity diagram for order fulfilment with swimlanes, fork and join (flowchart approximation of UML).


Customer
start
Place order
fork
Billing
Charge card
Warehouse
Pick and pack items
Payment approved?
yes
no
join
Receive failure email
Print shipping label
end
Hand to carrier
Receive tracking email
end



Reading the diagram. Three swimlanes assign responsibility: *Customer*, *Billing* and *Warehouse*. After the customer places the order, the fork starts two concurrent branches: billing charges the card while the warehouse picks and packs. The join waits for both: shipping must not start before payment is approved *and* the parcel is packed. If payment is declined, the decision routes to a failure email and an end node. A strict UML reading notes a subtle bug the diagram exposes: on the decline path the picked items are never returned to the shelf, and the join is never satisfied, so the warehouse branch is left dangling. A real design adds a compensating "restock items" action, which is the same reasoning as a saga's compensation (§32.7). Activity diagrams show no storage or protocols; they show ordering and concurrency, which is where workflow bugs hide.

Table 38.5 — Choosing a diagram for the question being asked.

Question	Diagram	Why not the others
What types exist and how are they coupled?	Class	Behavior diagrams hide structure
What happens, in what order, for one request?	Sequence	Class diagrams have no time axis
What are the replaceable parts and their contracts?	Component	Class diagrams are too fine-grained
Which events are legal for one object, and when?	State machine	Sequence diagrams show one path, not all legal ones
What is the workflow across roles, with parallel steps?	Activity	Sequence diagrams make parallelism and roles awkward
What does the data model look like in the database?	Entity-relationship (§11.17)	UML class diagrams can do it, but ER notation is the database convention

38.8 Object relationships in depth

38.8.1 Association, aggregation and composition

All three are structural links between objects that persist beyond a single method call; they differ in *ownership* and *lifetime*.

DEFINITION

Definition — Association, aggregation, composition. An **association** is a structural relationship in which instances of one class hold links to instances of another. **Aggregation** (UML "shared aggregation") is an association marked as whole–part, in which a part may be shared by several wholes and exists independently of them. **Composition** (UML "composite aggregation") is a whole–part association in which each part belongs to at most one whole at a time, and the whole is responsible for the part's existence: if the composite is deleted, its parts are deleted with it [OMG 2017].

First-principles explanation

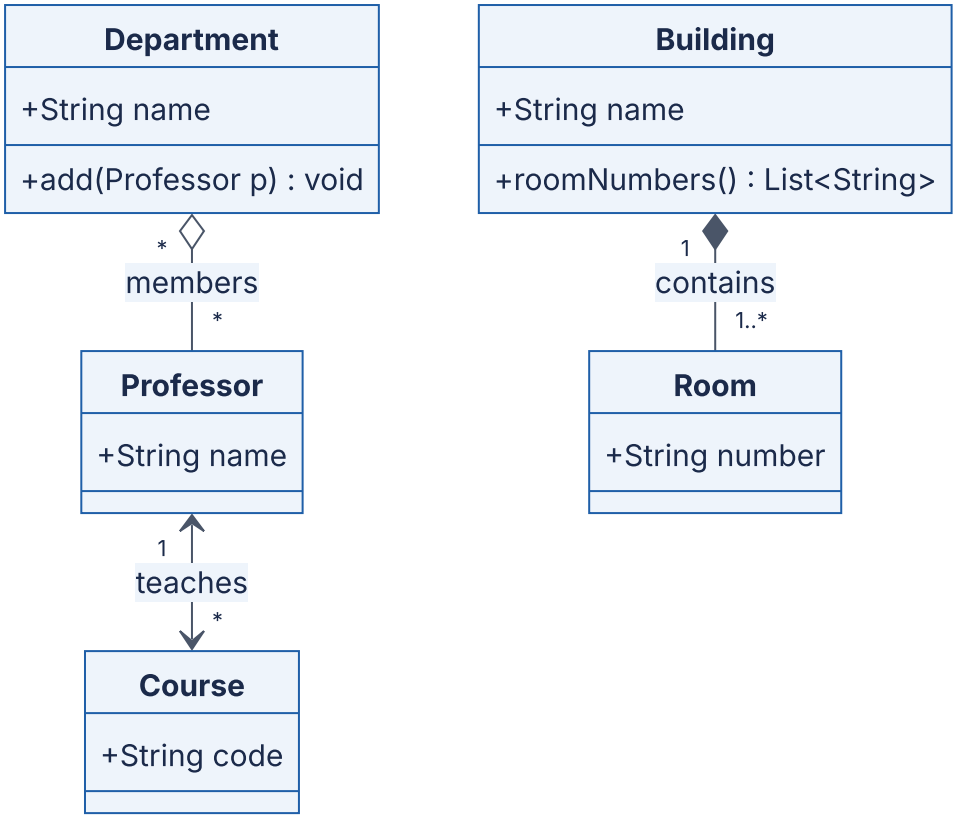
Three tests settle almost every case:

1. **Lifetime test.** If the whole is destroyed, must the part be destroyed too? A room cannot exist without its building: composition. A professor does not vanish when a department closes: not composition.
2. **Sharing test.** Can the part belong to two wholes at once? A professor can be in two departments: aggregation or plain association. A room is in exactly one building: composition.
3. **Creation test.** Who creates the part? If only the whole creates it and nobody outside holds a reference, it is composition. If the part is created elsewhere and handed in, it is aggregation or association.

Why does this matter?

The distinction decides concrete code questions: who constructs the object, whether it can be passed to the outside, whether deleting the parent must cascade (in the database, `ON DELETE CASCADE` for composition, a nullable foreign key or a join table for association), and whether the parent may expose its parts or only copies.

Figure 38.6 — Association, aggregation and composition in one model.



Reading the diagram. *Professor–Course* is a bidirectional **association** (arrowheads at both ends): each course has exactly one lecturer and a professor teaches any number of courses, and both sides can navigate to the other. *Department–Professor* is **aggregation** (hollow diamond at the department): professors are members of departments, may belong to several (multiplicity `*` at the department end), and outlive any department. *Building–Room* is **composition** (filled diamond): a room belongs to exactly one building and is created and destroyed with it. In storage terms the composition maps to a `rooms` table with a non-null `building_id` and `ON DELETE CASCADE`; the aggregation maps to a join table `department_members`. The failure point the diagram highlights is the bidirectional association: two references must be kept consistent, which is a frequent source of bugs.

Code example

```
        from __future__ import annotations


        class Professor:
            """Association: a Professor knows Courses and each Course knows its
Professor."""
            def __init__(self, name: str):
                self.name = name
                self.courses: list[Course] = []


        class Course:
            def __init__(self, code: str, lecturer: Professor):
                self.code = code
                self.lecturer = lecturer          # association end, multiplicity 1
                lecturer.courses.append(self)     # keep both ends consistent


        class Department:
            """Aggregation: professors are shared and outlive the department."""
            def __init__(self, name: str):
                self.name = name
                self.members: list[Professor] = []

            def add(self, p: Professor) -> None:
                self.members.append(p)            # the Professor was created elsewhere


        class Room:
            def __init__(self, number: str):
                self.number = number


        class Building:
            """Composition: the Building creates its Rooms; they never escape it."""
            def __init__(self, name: str, room_count: int):
                self.name = name
                self._rooms = [Room(f"{name}-{i}") for i in range(1, room_count + 1)]

            def room_numbers(self) -> list[str]:
                return [r.number for r in self._rooms]   # expose values, not the parts


        if __name__ == "__main__":
            ada = Professor("Ada")
            db = Course("CS145", ada)
            cs, math = Department("CS"), Department("Math")
            cs.add(ada); math.add(ada)                   # one part, two wholes: shared
            del cs                                       # the department goes away...
            print(ada.name, [c.code for c in ada.courses], len(math.members))  # ...Ada
does not
            hall = Building("Gates", 3)
            print(hall.room_numbers())
```

Output: `Ada ['CS145'] 1`, then `['Gates-1', 'Gates-2', 'Gates-3']`.

What to notice:

- Composition is expressed by *who creates* (`Building.__init__`) and *what escapes* (`room_numbers` returns strings, not `Room` objects). In a garbage-collected language nothing is "deleted"; composition means exclusive ownership and encapsulation.
- The bidirectional association needs code to keep both ends in sync (`lecturer.courses.append(self)`). Prefer one-directional associations unless both directions are genuinely navigated.
- Aggregation looks identical to association in code. The only difference is intent, documented by the diamond.

Table 38.6 — Association, aggregation and composition compared.

Aspect	Association	Aggregation	Composition
What it is	A lasting link between peers	Whole–part, shared part	Whole–part, exclusive part
Part lifetime	Independent	Independent	Bound to the whole
Part shared?	Yes	Yes	No (one whole at a time)
Who creates the part	Anyone	Usually outside the whole	The whole
UML symbol	Plain line or arrow	Hollow diamond at whole	Filled diamond at whole
Database mapping	Foreign key or join table	Join table, no cascade	Child table, `ON DELETE CASCADE`
Example	`Course` → `Professor`	`Playlist` ◇ `Song`	`Order` ◆ `LineItem`
Choose when	Objects collaborate as peers	You want to stress "collection of" without ownership	Part has no meaning outside the whole

NOTE

Note — aggregation is controversial. UML 2.5.1 itself says that the precise semantics of shared aggregation vary by application area and modeler [OMG 2017]. Fowler recommends ignoring aggregation altogether and quotes Jim Rumbaugh describing it as a "modeling placebo" [Fowler 2003]. In interviews, use composition when ownership matters and a plain association otherwise; if you draw a hollow diamond, be ready to say exactly what it adds.

38.8.2 Attribute or relationship?

UML treats an attribute and a navigable association as two notations for the same thing: a property of the class [OMG 2017; Fowler 2003]. `Order` with an attribute `customer :`

`Customer` means the same as a line from `Order` to `Customer`. The choice is about what you want the reader to notice.

1. **Value types are attributes.** Primitive values, enums, dates, and small immutable *value objects* such as `Money`, `Address` or `Coordinates` have no identity of their own (two `Money(5, "USD")` are interchangeable). Write them inside the box.
2. **Entities are relationships.** Classes with identity and their own lifecycle (`Customer`, `Product`, `ParkingSpot`) get a line, so multiplicity and ownership are visible.
3. **Relationships with data become classes.** If the link itself has properties, such as a student's grade in a course or a member's due date for a book, promote the link to an *association class* (`Enrollment`, `Loan`). In code it is always a class; in the database it is the join table.
4. **Collections of entities are always lines.** `items : List<LineItem>` hides the multiplicity and the composition that a filled diamond with `1..*` shows.

COMMON MISTAKE

Mistake — foreign keys in the class diagram. Writing `customerId : int` in `Order` copies the relational schema into the object model and hides the relationship. In a class diagram, draw the association; the identifier is a persistence detail that belongs in the entity-relationship diagram (§11.17).

38.9 The LLD interview method

38.9.1 What is being tested

DEFINITION

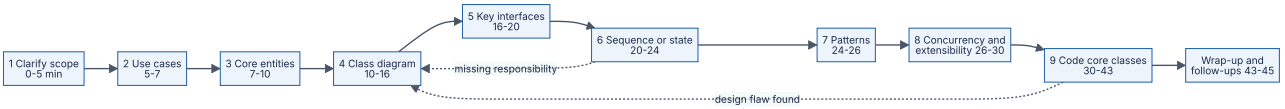
Definition — Low-level design (LLD) interview. A 45–60-minute interview in which you design the classes, interfaces, relationships and key algorithms of a single component or small application (a parking lot, an LRU cache, a rate limiter), usually ending with working or near-working code for the core classes. It is also called an *object-oriented design* or *machine coding* interview. It contrasts with the high-level design interview (§1.5, Chapter 87), which designs a distributed system out of services and data stores.

The interviewer is assessing whether you can turn a vague request into a clean object model, apply the principles of Chapters 35–37 without over-engineering, write correct code, and reason about concurrency and change. There is no single right answer; there are defensible and indefensible ones.

38.9.2 The nine steps

1. **Clarify requirements and scope (3–5 min).** Ask who the users are, which features are in scope, the scale (one lot or a chain of 500; one process or many), and what must *not* be built. Write the agreed functional requirements as a numbered list and two or three non-functional ones (thread safety, extensibility, latency).
2. **List use cases (2 min).** Actor plus verb: “driver parks vehicle”, “attendant views free spots”. Each later sequence diagram corresponds to one use case.
3. **Identify core entities (3 min).** Underline nouns in the requirements and keep those with behavior or identity. Discard synonyms and pure attributes (a “license plate” is an attribute of `Vehicle`, not a class).
4. **Draw relationships as a class diagram (5–7 min).** Add multiplicities and choose composition, association or inheritance deliberately (§38.8). Keep it to the 6–12 classes that matter.
5. **Define key interfaces (3–5 min).** Write the signatures of the public operations and the interfaces at variation points (allocation strategy, fee policy, storage). This is where Dependency Inversion (§36.7) shows.
6. **Sequence the main flows (3–5 min).** One sequence diagram (or a state diagram if the problem is lifecycle-heavy, such as a vending machine) for the most important use case. It validates the class diagram: every message must land on a class that owns that responsibility.
7. **Name the patterns you applied (2 min).** Strategy, State, Observer, Factory, Singleton: say *why* each earns its place and which change it absorbs [Gamma et al. 1994]. Do not add patterns to collect them.
8. **Address concurrency and extensibility (3–5 min).** Identify the shared mutable state, the critical sections and the lock granularity; then walk through two likely changes (a new vehicle type, a new pricing rule) and show which classes change.
9. **Code the core classes (12–15 min).** Write the domain model, the main operation end to end, and one test or driver that exercises it. Prefer correct and readable over complete.

Figure 38.7 — The LLD interview method with a 45-minute time budget.



Reading the diagram. The nine steps run left to right, each with a cumulative time window for a 45-minute slot. The solid arrows are the normal order. The two dotted arrows are feedback loops: drawing the sequence diagram often reveals a responsibility with no owner, and coding often reveals a flaw in the class diagram; in both cases you return briefly to step 4 and say so aloud, which interviewers read as a strength, not a weakness. The main failure points are the two ends: spending 15 minutes on requirements (no time

to code) or starting to code at minute 3 (no design to discuss). For a 60-minute slot, scale steps 4 and 9 most.

Table 38.7 — Time budget for a 45-minute LLD interview.

Minutes	Step	Output on the whiteboard or editor	Typical mistake
0–5	Clarify requirements	Numbered FR list, 2–3 NFRs, out-of-scope list	Assuming scope silently
5–7	Use cases	4–6 “actor verbs object” lines	Skipping, then forgetting a flow
7–10	Core entities	6–12 nouns with one-line responsibilities	Making every noun a class
10–16	Class diagram	Classes, relationships, multiplicities	Inheritance everywhere
16–20	Key interfaces	Signatures of public operations and strategies	Getters and setters only
20–24	Sequence or state diagram	One main flow including one failure branch	Drawing every flow
24–26	Patterns	2–4 named patterns with the reason for each	Pattern bingo
26–30	Concurrency and extensibility	Shared state, lock scope, two change scenarios	“Add a lock” with no scope
30–43	Code	Core classes, main operation, a driver or test	Boilerplate first, logic last
43–45	Wrap-up	Known limitations and next steps	Silence

38.9.3 How you are evaluated

Table 38.8 — LLD interview rubric.

Dimension	Below the bar	Meets the bar	Strong
Requirements	Starts designing immediately	Clarifies scope and lists FRs	Also surfaces NFRs, edge cases and explicit non-goals
Object model	God class or anemic data bags	Cohesive classes, sensible relationships	Clear ownership, value objects vs entities, justified multiplicities
Principles	Tight coupling to concrete classes	Interfaces at obvious variation points	Explains each choice in SOLID terms and avoids speculative generality
Patterns	None, or applied for their own sake	Appropriate pattern for the main variation	Names alternatives and why they lost
Code	Does not run or is mostly scaffolding	Core flow works, readable names	Runs, includes a test, handles failures and edge cases
Concurrency	Not considered	Identifies shared state and locks it	Discusses granularity, contention and atomic operations
Extensibility	Change requires editing many classes	One or two classes per change	Walks through concrete changes and their blast radius
Communication	Silent or unstructured	Explains while working	Drives the session, checks in, manages time

Calibrate by level: a junior candidate who meets the bar on the first five rows usually passes; a senior candidate is expected to be strong on concurrency and extensibility and to lead the discussion.

> INTERVIEW TIP
>
> **Tip — say the trade-off, then pick.** “I could model vehicle types with inheritance or with a `size` enum. Since the only difference is which spots fit, I’ll use the enum and avoid a class hierarchy with no behavior.” That sentence earns more credit than either choice alone.

Table 38.9 — Failure scenarios in an LLD interview.

What goes wrong	Why	Detection	Recovery	Prevention
No code at minute 40	Requirements and diagrams expanded to fill the time	Clock check at minute 25 shows step 6 unfinished	Code the single main operation; describe the rest	Announce the time plan at the start
Class explosion (20+ classes)	Every noun became a class; inheritance for each variant	Diagram no longer fits; many classes without behavior	Collapse variants into enums or strategies	Keep only nouns with behavior or identity
Interviewer asks for a change and many classes must change	Concrete dependencies, switch statements on type	Walking through the change touches 4+ classes	Introduce an interface at that variation point	Ask "what will change?" during step 5
Race condition pointed out	Shared mutable state with check-then-act	Two threads can both see a spot as free	Wrap check and act in one critical section	List shared state explicitly in step 8
Code does not run	Untested edge case or typo	Interviewer traces an example	Walk through a small example by hand	Write a 5-line driver early and grow it

38.10 LLD exercises

Each exercise follows the method of §38.9: requirements, a class diagram, one sequence or state diagram, runnable core code (tested with `python3`), the patterns used, and the extensions and follow-up questions interviewers ask. Exercises 1–4 are worked in full; 5–8 are more compact and focus on the one design decision that makes each problem interesting. All designs run in a single process; turning one into a distributed service is a high-level design question (Part X).

Table 38.10 — The eight exercises at a glance.

#	Problem	Core design decision	Main patterns	Concurrency concern
1	Parking lot	Spot allocation and pricing as pluggable policies	Strategy, Facade	Two cars claiming one spot
2	Elevator system	Dispatch policy separate from car state machine	Strategy, State machine	Hall calls from many panels
3	LRU cache	Hash map plus doubly linked list for O(1)	Interface + implementation	`get` mutates recency order
4	Vending machine	Behavior varies by state	State	Events during dispensing
5	Library system	Title versus physical copy; holds queue	Observer	Two members, one copy
6	Rate limiter	Algorithm behind an interface	Strategy, Decorator, Factory	Per-key atomic update
7	Key-value store	TTL expiry and nested transactions	Command-like write buffers	Commit atomicity, conflicts
8	Chess	Legal-move validation in layers	Polymorphism, Template-like hierarchy	Usually single-threaded

38.10.1 Exercise 1: Parking lot

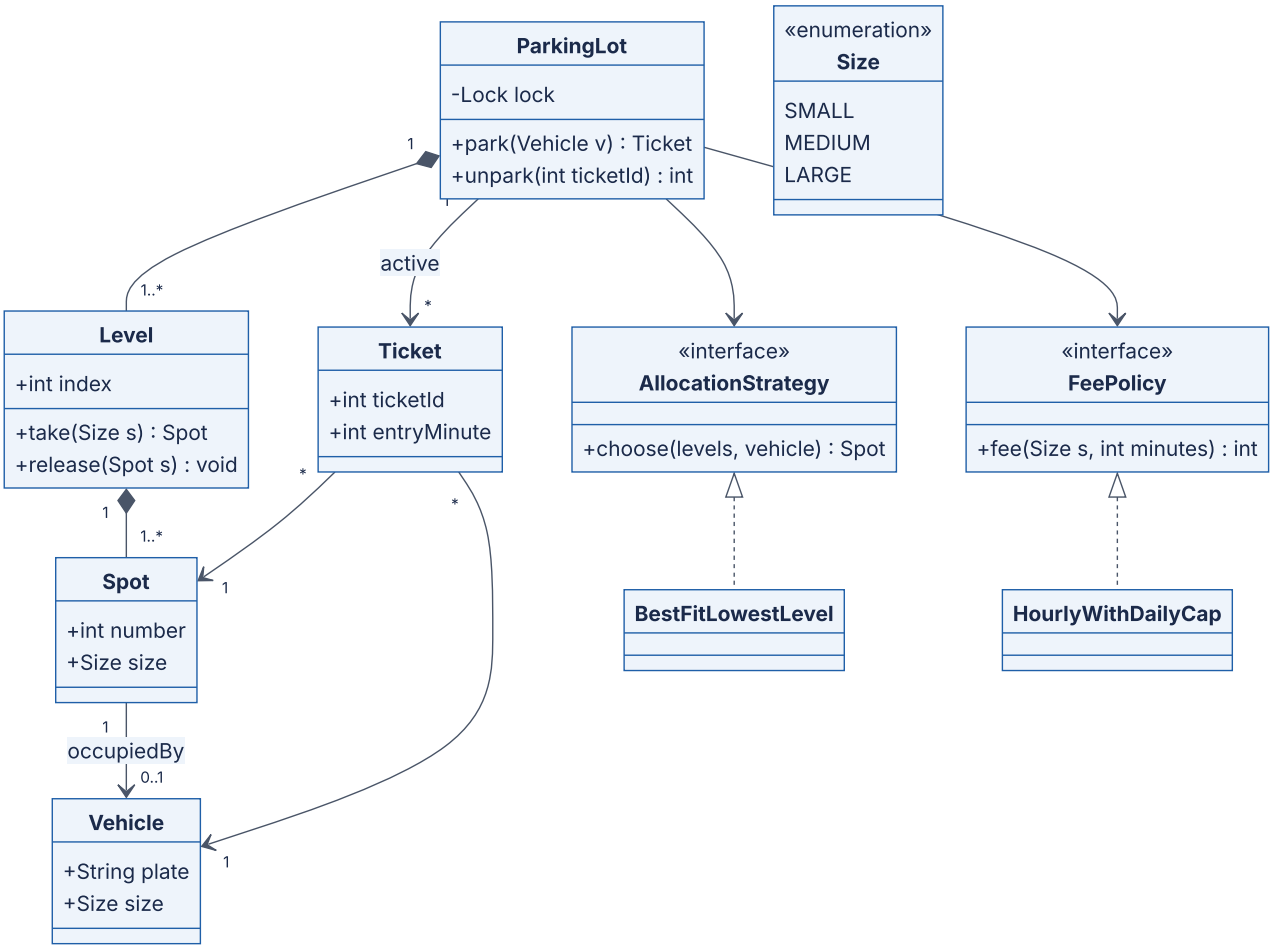
Requirements

Clarified with the interviewer:

1. A lot has several levels; each level has spots of three sizes: small (motorcycles), medium (cars) and large (vans and buses).
2. A vehicle may park in a spot of its size or larger. Choose the smallest fitting size first, then the lowest level, then the lowest spot number.
3. Entry issues a ticket; exit computes a fee from the vehicle size and the duration: an hourly rate with a daily cap.
4. Report “full” when no suitable spot exists. One vehicle (plate) cannot be parked twice.
5. Non-functional: thread safe (several gates operate concurrently); new allocation rules and pricing rules must not require changes to `ParkingLot`.

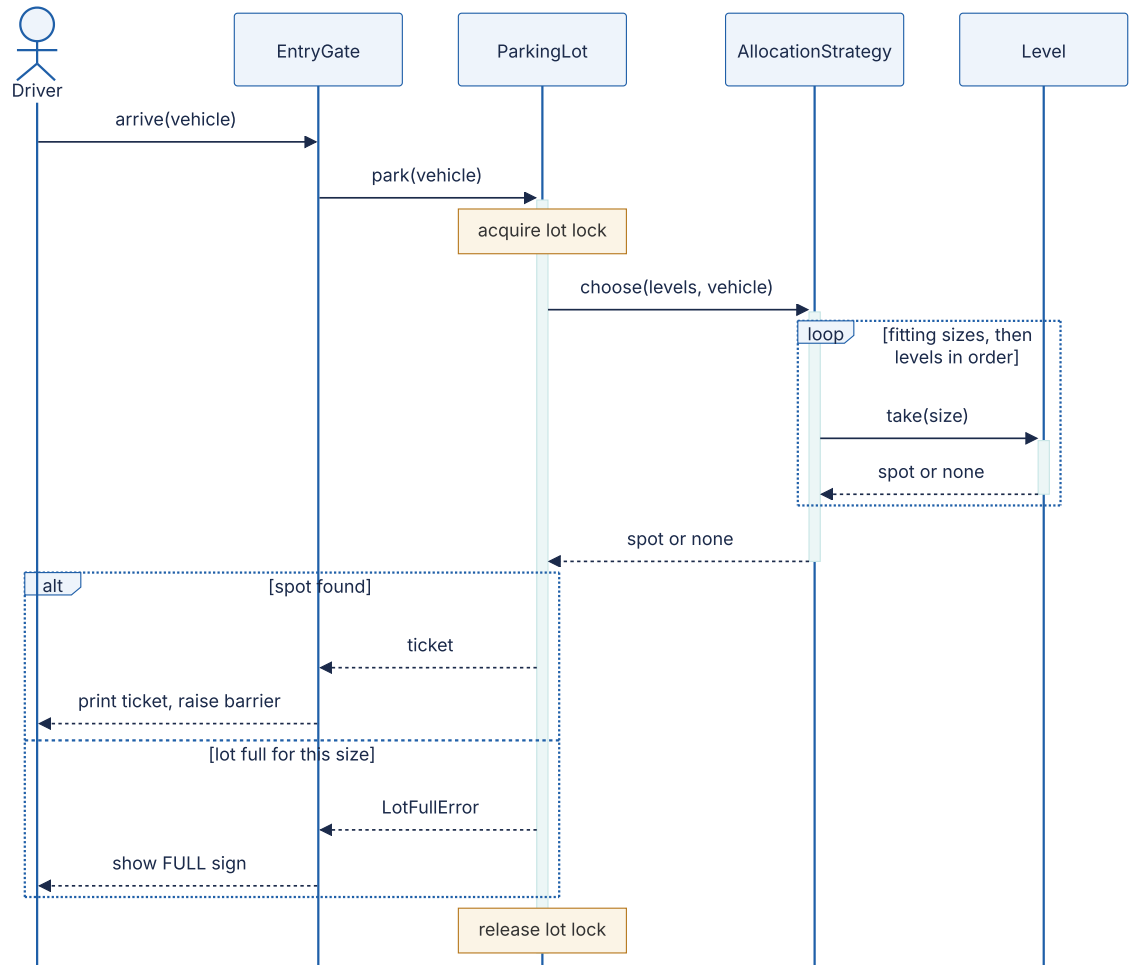
Out of scope: payment processing, reservations, physical sensors.

Figure 38.8 — Parking lot class diagram.



Reading the diagram. `ParkingLot` *composes* its levels and each `Level` composes its spots: they are created with the lot and meaningless without it. Vehicles and tickets are associations: a vehicle exists before and after parking. Each spot is occupied by at most one vehicle (`0..1`). The two interfaces are the variation points named in requirement 5: `AllocationStrategy` decides *which* spot, `FeePolicy` decides *how much*; the lot depends only on the interfaces (Dependency Inversion, §36.7). Vehicle kinds are a `Size` enumeration, not a subclass hierarchy, because the only behavior that differs is which spots fit. The single lock in `ParkingLot` is the concurrency boundary; there is no storage boundary in this in-memory design.

Figure 38.9 — Sequence diagram for parking a vehicle.



Reading the diagram. The entry gate calls `park` synchronously. The lot takes its lock *before* asking the strategy, so the search and the claim of the spot form one critical section: two gates cannot both receive the last medium spot. The strategy loops over fitting sizes and levels, asking each level to `take` a spot (a heap pop, which both finds and removes it). The `alt` fragment covers success (ticket printed) and the full case. The failure point is the lock: a strategy that does slow work (a database call, say) would serialize all gates, so strategies must be fast and in-memory. Scaling to a chain of lots means one `ParkingLot` per site, not one lock for all.

Mathematical foundations

FORMULA

Formula 38.1 — Parking fee with an hourly rate and a daily cap

$$F(t) = \left\lfloor \frac{t}{1440} \right\rfloor C + \min\left(C,\ r\left\lceil \frac{t \bmod 1440}{60} \right\rceil\right)$$

Symbol	Meaning	Units
$F(t)$	Fee charged at exit	dollars
t	Parked duration	minutes
r	Hourly rate for the vehicle size; each started hour is charged in full	dollars/hour
C	Daily cap for the vehicle size, charged for each full 24 hours	dollars/day
1440	Minutes in a day	minutes

WORKED EXAMPLE

Worked example — three exits. Rates: car $r = 4$, $C = 30$; van $r = 8$, $C = 60$.

1. Car, $t = 130$ min: full days $\lfloor 130/1440 \rfloor = 0$; started hours $\lceil 130/60 \rceil = 3$; $F = \min(30, 4 \times 3) = \$12$.
2. Car, $t = 540$ min: $\lceil 540/60 \rceil = 9$ hours; $4 \times 9 = 36 > 30$, so $F = \$30$.
3. Van, $t = 1,565$ min: one full day ($60) plus $1,565 - 1,440 = 125$ min; $\lceil 125/60 \rceil = 3$ hours; $\min(60, 8 \times 3) = 24$; $F = 60 + 24 = \$84$.

Interpretation. For cars the cap binds once $4h > 30$, that is from the 8th started hour (any stay over 420 minutes). **Engineering implication.** Pricing rules change often (weekend rates, grace periods, event pricing). Because they live behind `FeePolicy`, each new rule is a new class tested in isolation, and `ParkingLot` does not change (Open/Closed, §36.4). The code below reproduces all three numbers.

Code

```
        from __future__ import annotations

        import heapq
        import itertools
        import math
        import threading
        from abc import ABC, abstractmethod
        from dataclasses import dataclass
        from enum import IntEnum


        class Size(IntEnum):
            SMALL = 1    # motorcycle
            MEDIUM = 2   # car
            LARGE = 3    # van or bus


        @dataclass(frozen=True)
        class Vehicle:
            plate: str
            size: Size


        @dataclass
        class Spot:
            level: int
            number: int
            size: Size
            vehicle: Vehicle | None = None


        @dataclass(frozen=True)
        class Ticket:
            ticket_id: int
            vehicle: Vehicle
            spot: Spot
            entry_minute: int


        class Level:
            """Owns its spots (composition) and a min-heap of free spot numbers per
size."""
            def __init__(self, index: int, counts: dict[Size, int]):
                self.index, self.spots = index, []
                self._free: dict[Size, list[int]] = {s: [] for s in Size}
                for size, count in counts.items():
                    for _ in range(count):
                        n = len(self.spots)
                        self.spots.append(Spot(index, n, size))
                        heapq.heappush(self._free[size], n)

            def take(self, size: Size) -> Spot | None:
                return self.spots[heapq.heappop(self._free[size])] if self._free[size]
else None
```

```
            def release(self, spot: Spot) -> None:
                heapq.heappush(self._free[spot.size], spot.number)

            def free_count(self, size: Size) -> int:
                return len(self._free[size])


        class AllocationStrategy(ABC):                      # Strategy pattern
            @abstractmethod
            def choose(self, levels: list[Level], vehicle: Vehicle) -> Spot | None: ...


        class BestFitLowestLevel(AllocationStrategy):
            """Smallest spot size that fits, then lowest level, then lowest spot num-
ber."""
            def choose(self, levels, vehicle):
                for size in (s for s in Size if s >= vehicle.size):
                    for level in levels:
                        if (spot := level.take(size)) is not None:
                            return spot
                return None


        class FeePolicy(ABC):                               # Strategy pattern
            @abstractmethod
            def fee(self, size: Size, minutes: int) -> int: ...


        class HourlyWithDailyCap(FeePolicy):
            RATES = {Size.SMALL: (2, 15), Size.MEDIUM: (4, 30), Size.LARGE: (8, 60)}  #
$/h, $/day cap

            def fee(self, size, minutes):
                rate, cap = self.RATES[size]
                days, rest = divmod(minutes, 24 * 60)
                return days * cap + min(cap, rate * math.ceil(rest / 60))


        class LotFullError(Exception):
            pass


        class ParkingLot:                                    # Facade over levels,
strategy, fees
            def __init__(self, levels, strategy: AllocationStrategy, fees: FeePolicy,
clock):
                self.levels, self.strategy, self.fees, self.clock = levels, strategy,
fees, clock
                self._lock = threading.Lock()                # guards spots, tickets and
plates
                self._tickets: dict[int, Ticket] = {}
                self._plates: set[str] = set()
                self._ids = itertools.count(1)

            def park(self, vehicle: Vehicle) -> Ticket:
                with self._lock:
```

```
                if vehicle.plate in self._plates:
                    raise ValueError(f"{vehicle.plate} is already parked")
                spot = self.strategy.choose(self.levels, vehicle)
                if spot is None:
                    raise LotFullError(f"no spot for {vehicle.size.name}")
                spot.vehicle = vehicle
                ticket = Ticket(next(self._ids), vehicle, spot, self.clock())
                self._tickets[ticket.ticket_id] = ticket
                self._plates.add(vehicle.plate)
                return ticket

            def unpark(self, ticket_id: int) -> int:
                with self._lock:
                    ticket = self._tickets.pop(ticket_id)    # KeyError: unknown or al-
ready used
                    ticket.spot.vehicle = None
                    self.levels[ticket.spot.level].release(ticket.spot)
                    self._plates.discard(ticket.vehicle.plate)
                    minutes = self.clock() - ticket.entry_minute
                    return self.fees.fee(ticket.vehicle.size, minutes)


        if __name__ == "__main__":
            now = [0]                                       # fake clock in minutes
            lot = ParkingLot([Level(0, {Size.SMALL: 1, Size.MEDIUM: 2, Size.LARGE: 1}),
                              Level(1, {Size.MEDIUM: 2, Size.LARGE: 1})],
                             BestFitLowestLevel(), HourlyWithDailyCap(), lambda: now[0])
            moto = lot.park(Vehicle("M-1", Size.SMALL))
            cars = [lot.park(Vehicle(f"C-{i}", Size.MEDIUM)) for i in range(3)]
            print([(t.spot.level, t.spot.number, t.spot.size.name) for t in [moto,
*cars]])
            vans = [lot.park(Vehicle(f"V-{i}", Size.LARGE)) for i in range(2)]
            try:
                lot.park(Vehicle("V-9", Size.LARGE))
            except LotFullError as e:
                print("rejected:", e)
            now[0] = 130
            print("car, 130 min:", lot.unpark(cars[0].ticket_id))          #
ceil(130/60)=3 h -> $12
            now[0] = 540
            print("car, 540 min:", lot.unpark(cars[1].ticket_id))          # 9 h -> $36
-> cap $30
            now[0] = 1565
            print("van, 1565 min:", lot.unpark(vans[0].ticket_id))         # 1 day $60 +
3 h $24
            car = lot.park(Vehicle("C-7", Size.MEDIUM))                     # reuses
lowest free spot
            print("C-7 ->", car.spot.level, car.spot.number)

            # Concurrency check: 50 threads race for 10 medium spots; exactly 10 must
win.
            big = ParkingLot([Level(0, {Size.MEDIUM: 10})], BestFitLowestLevel(),
                             HourlyWithDailyCap(), lambda: 0)
            wins = []
            def attempt(i):
                try:
                    wins.append(big.park(Vehicle(f"T-{i}", Size.MEDIUM)).spot.number)
```

```
                except LotFullError:
                    pass
            threads = [threading.Thread(target=attempt, args=(i,)) for i in range(50)]
            for t in threads: t.start()
            for t in threads: t.join()
            assert sorted(wins) == list(range(10)), wins
            print("concurrent winners:", len(wins))
```

Output:

```
[(0, 0, 'SMALL'), (0, 1, 'MEDIUM'), (0, 2, 'MEDIUM'), (1, 0, 'MEDIUM')]
rejected: no spot for LARGE
car, 130 min: 12
car, 540 min: 30
van, 1565 min: 84
C-7 -> 0 1
concurrent winners: 10
```

What to notice:

- Each level keeps a min-heap of free spot numbers per size, so finding *and* claiming a spot is $O(\log n)$ per level instead of a linear scan of all spots; with S sizes and L levels the search is $O(SL \log n)$, which is constant for a fixed building.
- The check ("is there a free spot?") and the act ("mark it occupied") happen under one lock. Splitting them is the classic check-then-act race (§6.9); the 50-thread test would then occasionally hand one spot to two cars.
- Time comes from an injected `clock`, so fees are testable without waiting hours; tickets are single-use because `unpark` pops them.

Patterns used

Strategy for allocation and pricing (§37.7); **Facade**: `ParkingLot` is the only class the gates talk to; **composition** for lot–level–spot ownership; value objects (`Vehicle`, `Ticket`) are immutable dataclasses.

Extensions and follow-up questions

- *Electric-vehicle spots with chargers?* Add a `features` set to `Spot` and a strategy that prefers matching features; nothing else changes.
- *Reduce lock contention with 20 gates?* Lock per level and let the strategy try levels in a gate-specific order; or keep free counts per level in atomics and lock only the chosen level. Measure first: at a few cars per second, one lock is not a bottleneck.
- *Display free spots per level in real time?* Have `ParkingLot` publish `SpotTaken` / `SpotFreed` events to display boards (Observer, §37.8).
- *Lost ticket?* Look up the active ticket by plate (the `_plates` set becomes a plate→ticket map) and charge per policy.

- *Multiple lots across a city?* That becomes a high-level design: a reservation service with a database per region; the LLD here is the per-lot component.

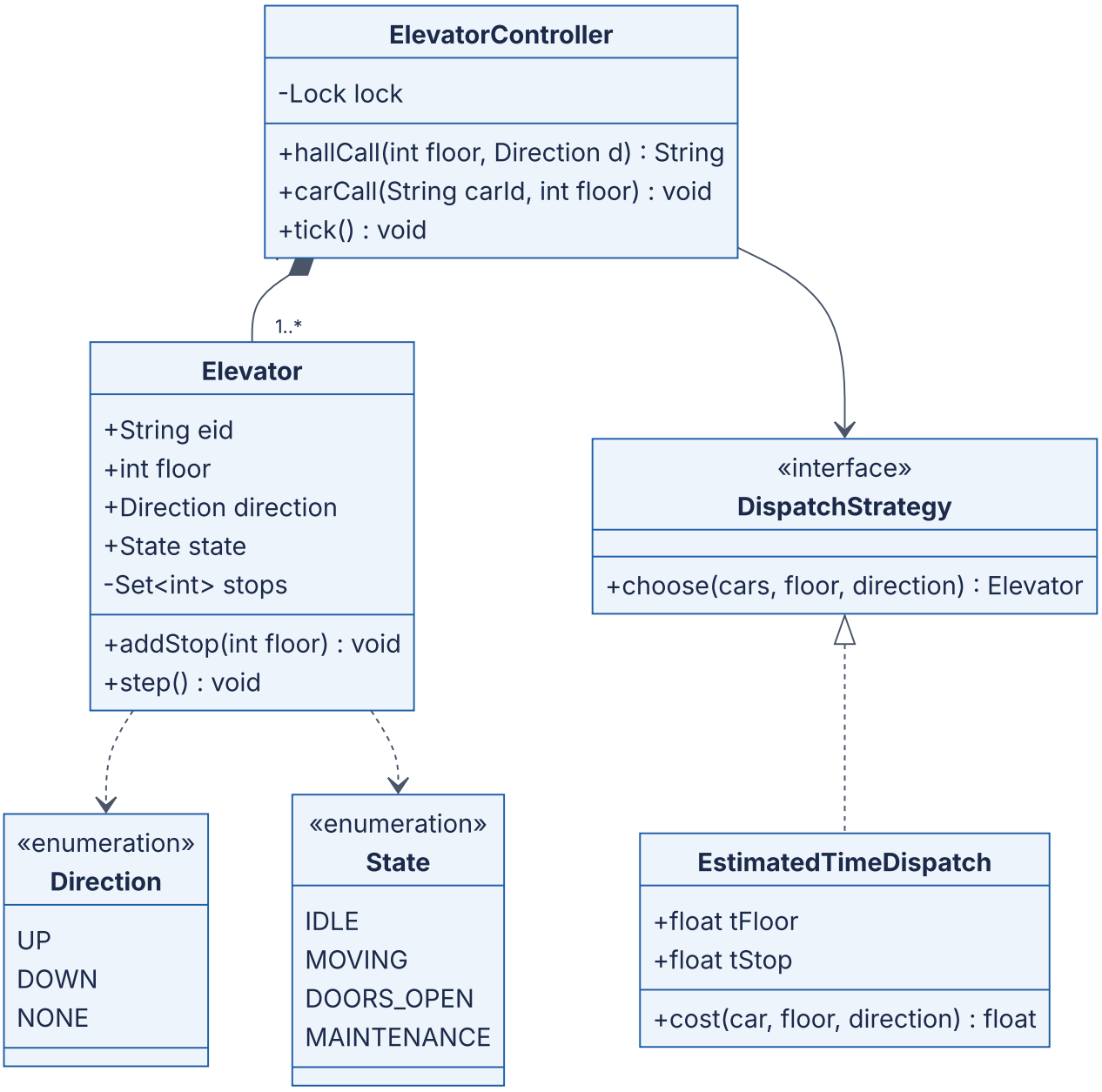
38.10.2 Exercise 2: Elevator system

Requirements

1. A building has *N* floors and *M* cars. Passengers press *hall calls* (floor plus direction) on each floor and *car calls* (destination) inside a car.
2. A dispatcher assigns each hall call to one car; the policy must be replaceable.
3. Each car serves its stops using the LOOK policy: keep moving in the current direction while stops remain ahead, then reverse; idle when there are no stops.
4. A car in maintenance is not dispatched.
5. Non-functional: hall calls arrive concurrently from many panels; the simulation must be deterministic for tests.

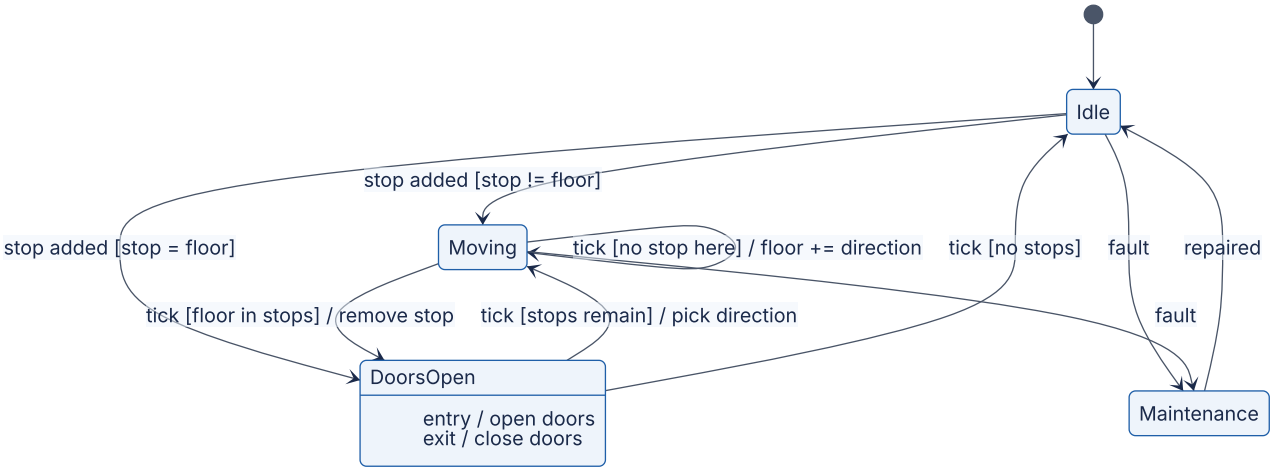
Out of scope: weight sensors, door obstruction, fire mode, energy optimization (all good follow-ups).

Figure 38.10 — Elevator system class diagram.



Reading the diagram. The `ElevatorController` composes the cars and owns the lock that serializes calls and ticks. *Which car* is decided by a `DispatchStrategy` (here `EstimatedTimeDispatch`); *how a car moves* is the car's own state machine (`step`). Keeping the two separate means you can swap in zoning or destination dispatch without touching car movement, and you can test each in isolation. A car's pending work is a set of stop floors. There is no storage boundary; the real-world failure points are a car that stops responding (mark it `MAINTENANCE` so the dispatcher skips it) and the controller itself, which in a real building has a hardware fallback.

Figure 38.11 — State machine of one elevator car.



Reading the diagram. A car starts *Idle*. When a stop is added it either opens its doors (the stop is the current floor) or starts *Moving*. Each tick while moving it either advances one floor or, if the current floor is a stop, removes the stop and enters *DoorsOpen*, whose entry action opens the doors and exit action closes them. Leaving *DoorsOpen* it re-applies LOOK: continue if stops remain, otherwise go *Idle*. A fault from *Idle* or *Moving* sends the car to *Maintenance* until repaired. There is deliberately no transition from *DoorsOpen* to *Moving* without passing the exit action, so a car can never move with open doors, the most important safety property in the whole design. In the code below, the guards are checked in `step()`, and the timed ticks are deterministic so tests do not depend on wall-clock time.

Mathematical foundations

FORMULA

Formula 38.2 — Estimated time for car e to reach a hall call r

$$T(e,r) = d \cdot t_{\text{floor}} + s \cdot t_{\text{stop}}, \qquad d = \begin{cases} |f_e - f_r| & \text{if } e \text{ is idle, or } f_r \text{ is ahead and } e \text{ moves in the call'} \\ |f_e - f_x| + |f_x - f_r| & \text{otherwise} \end{cases}$$

Symbol	Meaning	Units
$T(e,r)$	Estimated time until car e arrives at the caller	s
f_e, f_r	Current floor of the car; floor of the call	floor index
f_x	Farthest pending stop in the car's current direction (where it will turn around)	floor index
d	Floors the car travels before reaching the caller	floors
s	Pending stops on that path before the caller	stops
t_{floor}	Travel time per floor	s/floor
t_{stop}	Time lost per intermediate stop (decelerate, doors, accelerate)	s/stop

WORKED EXAMPLE

Worked example — hall call at floor 7 going up. $t_{\text{floor}} = 2$ s, $t_{\text{stop}} = 10$ s. Three cars:

1. Car A at floor 3, moving up, stops {5, 9}. The call is ahead and in A's direction: $d = |3 - 7| = 4$; stops before 7: {5}, $s = 1$. $T = 4 \times 2 + 1 \times 10 = 18$ s.
2. Car B at floor 10, moving down, stop {2}. B moves *toward* floor 7 but in the wrong direction, so it finishes its sweep first: $f_x = 2$, $d = |10 - 2| + |2 - 7| = 8 + 5 = 13$; $s = 1$ (floor 2). $T = 13 \times 2 + 10 = 36$ s.
3. Car C idle at floor 1: $d = 6$, $s = 0$, $T = 6 \times 2 = 12$ s.

The dispatcher picks C (12 s), even though A is closer (4 floors versus 6). **Interpretation.** Stops cost more than distance: one stop (10 s) equals five floors of travel. **Engineering implication.** "Nearest car" is the wrong heuristic; minimize estimated arrival time instead, and keep the policy behind an interface, because real buildings tune t_{stop} and add penalties (for example for a car that is nearly full).

Code

```
        from __future__ import annotations

        import threading
        from abc import ABC, abstractmethod
        from enum import Enum


        class Direction(Enum):
            UP = 1
            DOWN = -1
            NONE = 0


        class State(Enum):
            IDLE = "idle"
            MOVING = "moving"
            DOORS_OPEN = "doors_open"
            MAINTENANCE = "maintenance"


        class Elevator:
            """One car. step() advances one tick: move one floor, or open/close
doors."""
            def __init__(self, eid: str, floor: int = 0):
                self.eid, self.floor = eid, floor
                self.direction, self.state = Direction.NONE, State.IDLE
                self.stops: set[int] = set()
                self.served: list[int] = []

            def add_stop(self, floor: int) -> None:
                self.stops.add(floor)

            def stops_ahead(self) -> list[int]:
                d = self.direction.value
                return [s for s in self.stops if d and (s - self.floor) * d > 0]

            def _pick_direction(self) -> None:          # LOOK: keep going while work
lies ahead
                if self.stops_ahead():
                    return
                if not self.stops:
                    self.direction = Direction.NONE
                    return
                nearest = min(self.stops, key=lambda s: abs(s - self.floor))
                self.direction = Direction.UP if nearest > self.floor else
Direction.DOWN

            def step(self) -> None:
                if self.state is State.MAINTENANCE:
                    return
                if self.state is State.DOORS_OPEN:      # doors close after one tick of
dwell
                    self._pick_direction()
                    self.state = State.MOVING if self.stops else State.IDLE
```

```
                    return
                if self.floor in self.stops:            # arrived at a stop: open doors
                    self.stops.discard(self.floor)
                    self.served.append(self.floor)
                    self.state = State.DOORS_OPEN
                    return
                self._pick_direction()
                if self.direction is Direction.NONE:
                    self.state = State.IDLE
                    return
                self.floor += self.direction.value
                self.state = State.MOVING


        class DispatchStrategy(ABC):                      # Strategy pattern
            @abstractmethod
            def choose(self, cars: list[Elevator], floor: int, direction: Direction) ->
Elevator: ...


        class EstimatedTimeDispatch(DispatchStrategy):
            """Pick the car with the lowest estimated time to reach the caller (Formula
38.2)."""
            def __init__(self, t_floor: float = 2.0, t_stop: float = 10.0):
                self.t_floor, self.t_stop = t_floor, t_stop

            def cost(self, car: Elevator, floor: int, direction: Direction) -> float:
                d = car.direction.value
                between = lambda a, b: [s for s in car.stops if min(a, b) <= s <= max(a,
b) and s != floor]
                on_the_way = car.direction is Direction.NONE or (
                    (floor - car.floor) * d >= 0 and car.direction is direction)
                if on_the_way:
                    dist = abs(car.floor - floor)
                    stops = between(car.floor, floor)
                else:                                     # finish this sweep, then come
back
                    far = max(car.stops_ahead(), key=lambda s: abs(s - car.floor),
default=car.floor)
                    dist = abs(car.floor - far) + abs(far - floor)
                    stops = set(between(car.floor, far)) | set(between(far, floor))
                return dist * self.t_floor + len(stops) * self.t_stop

            def choose(self, cars, floor, direction):
                candidates = [c for c in cars if c.state is not State.MAINTENANCE]
                return min(candidates, key=lambda c: (self.cost(c, floor, direction),
c.eid))


        class ElevatorController:
            def __init__(self, cars: list[Elevator], strategy: DispatchStrategy):
                self.cars = {c.eid: c for c in cars}
                self.strategy = strategy
                self._lock = threading.Lock()             # hall calls arrive from many
panels

            def hall_call(self, floor: int, direction: Direction) -> str:
```

```
                with self._lock:
                    car = self.strategy.choose(list(self.cars.values()), floor, direc-
tion)
                    car.add_stop(floor)
                    return car.eid

            def car_call(self, eid: str, floor: int) -> None:
                with self._lock:
                    self.cars[eid].add_stop(floor)

            def tick(self) -> None:
                with self._lock:
                    for car in self.cars.values():
                        car.step()


        if __name__ == "__main__":
            a, b, c = Elevator("A", 3), Elevator("B", 10), Elevator("C", 1)
            a.direction, a.stops = Direction.UP, {5, 9}
            b.direction, b.stops = Direction.DOWN, {2}
            strat = EstimatedTimeDispatch()
            costs = {car.eid: strat.cost(car, 7, Direction.UP) for car in (a, b, c)}
            print("costs for hall call 7 UP:", costs)
            assert costs == {"A": 18.0, "B": 36.0, "C": 12.0}

            ctl = ElevatorController([a, b, c], strat)
            print("assigned to:", ctl.hall_call(7, Direction.UP))
            ctl.car_call("A", 2)                          # 2 is behind A: served on the
way back
            for _ in range(40):
                ctl.tick()
            print({car.eid: car.served for car in (a, b, c)})
            assert a.served == [5, 9, 2] and b.served == [2] and c.served == [7]
```

Output:

```
costs for hall call 7 UP: {'A': 18.0, 'B': 36.0, 'C': 12.0}
assigned to: C
{'A': [5, 9, 2], 'B': [2], 'C': [7]}
```

What to notice:

- Car A receives a car call for floor 2 while moving up; LOOK finishes the upward sweep (5, 9) before reversing, which is why it serves `[5, 9, 2]` rather than turning back immediately.
- `cost()` implements Formula 38.2 directly, so the worked example doubles as a unit test.
- The controller lock covers both dispatch and ticks, so a hall call can never be assigned on the basis of a car's half-updated state.

Patterns used

Strategy for dispatch; an explicit **state machine** for each car (an enum plus guarded transitions in `step`; with more states and per-state behavior, refactor to the State pattern of §38.10.4); **composition** of cars in the controller.

Extensions and follow-up questions

- *Hall calls should only be served by a car going the same way.* Store stops as `(floor, direction)` and let a car skip opposite-direction hall stops on the current sweep.
- *Peak-hour zoning.* A `ZonedDispatch` strategy assigns cars to floor bands at 08:30–09:30; select the strategy by time of day (a strategy that delegates to others).
- *Destination dispatch* (passengers enter their destination in the lobby): the dispatcher now sees both ends of each trip and can group passengers by destination; the car model is unchanged.
- *Starvation:* a call that is always outbid by newer calls. Add an age term to the cost or a maximum wait that forces assignment.
- *Real-time control:* replace the discrete tick with an event-driven controller (arrival sensors), keeping the same state machine.

38.10.3 Exercise 3: LRU cache

Requirements

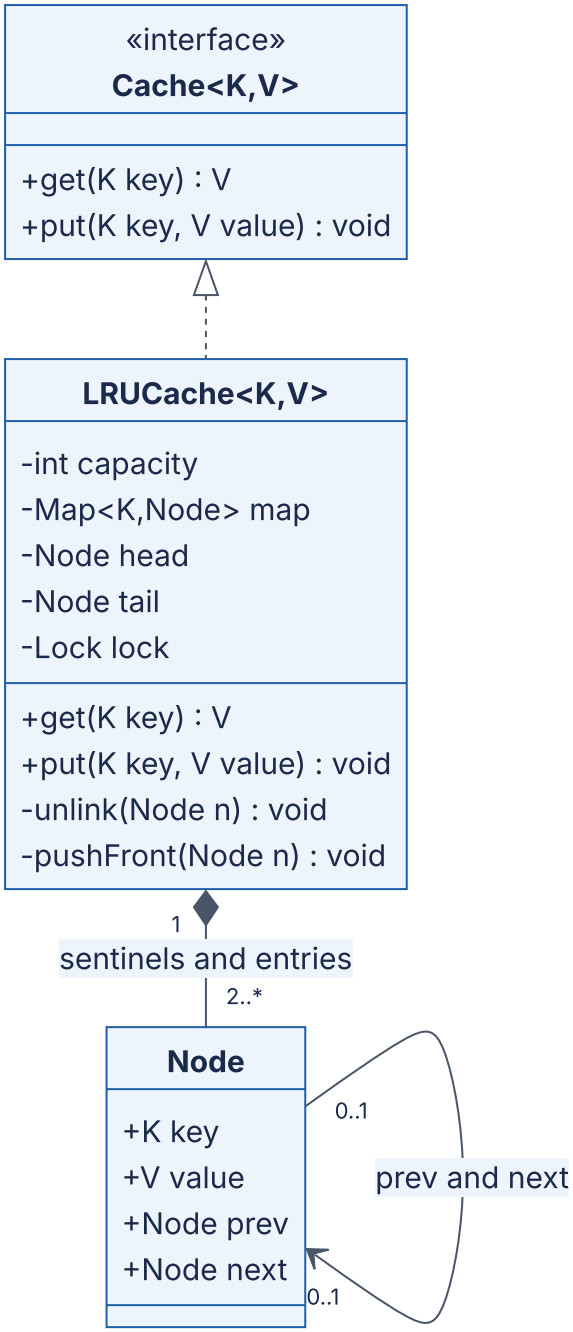
1. `get(key)` returns the value or a miss; `put(key, value)` inserts or updates.
2. Fixed capacity n; when full, `put` of a new key evicts the **least recently used** entry. Both reads and writes count as use.
3. Both operations in $O(1)$ time.
4. Thread safe; expose hit, miss and eviction counts.

The policy itself and its hit-ratio behavior are covered in §20.9; here the question is the data structure and the class design.

Why a hash map plus a doubly linked list

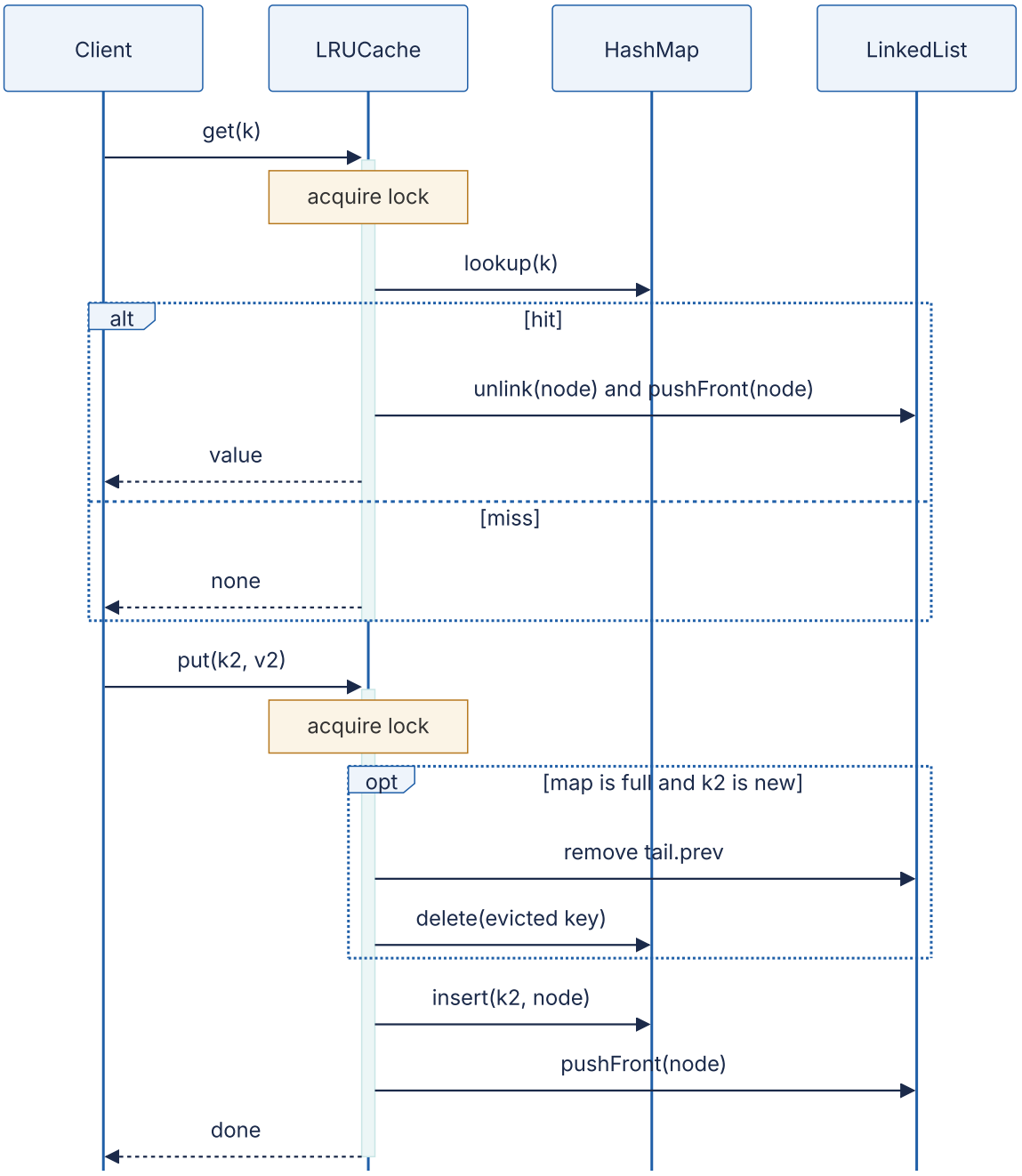
A hash map alone gives $O(1)$ lookup but no order. A list alone keeps order but finding a key is $O(n)$. Combined, the map stores `key → node` and the list keeps nodes in recency order: on each access you unlink the node (constant time, because a *doubly* linked node knows its predecessor) and move it to the front; to evict you remove the node before the tail. Two **sentinel** nodes at head and tail remove every empty-list special case.

Figure 38.12 — LRU cache class diagram.



Reading the diagram. Clients depend on the generic `Cache` interface, so an LFU or TinyLFU implementation (§20.9.3) can replace the LRU without changing them. `LRUCache` composes its nodes (at least the two sentinels, hence `2..*`); nodes are never handed out. Each node links to its neighbors in both directions, which is what makes unlinking $O(1)$. The private `unlink` and `pushFront` are the only list operations, and both assume the lock is held. The single lock is the concurrency boundary; there is no storage boundary, since the cache is in memory.

Figure 38.13 — Sequence diagram for get and put, including eviction.



Reading the diagram. `get` takes the lock, looks the key up in the map and, on a hit, moves the node to the front before returning; a miss changes nothing. `put` of a new key into a full cache first evicts the node before the tail sentinel and deletes its key from the map, then inserts and links the new node. All calls are synchronous and in memory. The failure point the diagram makes visible: `get` *writes* to the list, so it needs the same exclusive lock as `put`; a read-write lock that lets readers share would corrupt the list.

Code

```
        from __future__ import annotations

        import threading
        from abc import ABC, abstractmethod
        from typing import Generic, Hashable, Optional, TypeVar

        K = TypeVar("K", bound=Hashable)
        V = TypeVar("V")


        class Cache(ABC, Generic[K, V]):                    # the interface clients de-
pend on
            @abstractmethod
            def get(self, key: K) -> Optional[V]: ...
            @abstractmethod
            def put(self, key: K, value: V) -> None: ...


        class _Node:
            __slots__ = ("key", "value", "prev", "next")

            def __init__(self, key=None, value=None):
                self.key, self.value = key, value
                self.prev: _Node | None = None
                self.next: _Node | None = None


        class LRUCache(Cache[K, V]):
            """Hash map for O(1) lookup + doubly linked list for O(1) recency updates.
            Order: head.next is most recently used, tail.prev is least recently used."""

            def __init__(self, capacity: int):
                if capacity <= 0:
                    raise ValueError("capacity must be positive")
                self.capacity = capacity
                self._map: dict[K, _Node] = {}
                self._head, self._tail = _Node(), _Node()   # sentinels: no None checks
at the ends
                self._head.next, self._tail.prev = self._tail, self._head
                self._lock = threading.Lock()                # get() mutates order, so
it locks too
                self.hits = self.misses = self.evictions = 0

            # -- list primitives, each O(1); callers hold the lock --
            def _unlink(self, node: _Node) -> None:
                node.prev.next, node.next.prev = node.next, node.prev

            def _push_front(self, node: _Node) -> None:
                node.prev, node.next = self._head, self._head.next
                self._head.next.prev = node
                self._head.next = node

            def get(self, key):
                with self._lock:
```

```
                    node = self._map.get(key)
                    if node is None:
                        self.misses += 1
                        return None
                    self._unlink(node)
                    self._push_front(node)                  # mark as most recently
used
                    self.hits += 1
                    return node.value

            def put(self, key, value):
                with self._lock:
                    node = self._map.get(key)
                    if node is not None:                    # update in place, refresh
recency
                        node.value = value
                        self._unlink(node)
                        self._push_front(node)
                        return
                    if len(self._map) == self.capacity:     # evict the least recently
used
                        lru = self._tail.prev
                        self._unlink(lru)
                        del self._map[lru.key]
                        self.evictions += 1
                    node = _Node(key, value)
                    self._map[key] = node
                    self._push_front(node)

            def keys_mru_to_lru(self) -> list:
                with self._lock:
                    out, n = [], self._head.next
                    while n is not self._tail:
                        out.append(n.key)
                        n = n.next
                    return out


        if __name__ == "__main__":
            c = LRUCache[str, int](3)
            for k, v in [("a", 1), ("b", 2), ("c", 3)]:
                c.put(k, v)
            c.get("a")                                      # a becomes most recent
            c.put("d", 4)                                   # evicts b, the least re-
cent
            print(c.keys_mru_to_lru(), c.get("b"))
            assert c.keys_mru_to_lru() == ["d", "a", "c"] and c.get("b") is None

            # Stress: 8 threads x 20,000 mixed operations; invariants must hold after-
wards.
            import random
            big = LRUCache[int, int](100)
            def worker(seed):
                rng = random.Random(seed)
                for _ in range(20_000):
                    k = rng.randrange(300)
                    if rng.random() < 0.7:
```

```
                        big.get(k)
                    else:
                        big.put(k, k * k)
            ts = [threading.Thread(target=worker, args=(s,)) for s in range(8)]
            for t in ts: t.start()
            for t in ts: t.join()
            order = big.keys_mru_to_lru()
            assert len(order) == len(big._map) == 100 and set(order) == set(big._map)
            assert all(big._map[k].value == k * k for k in order)
            total = big.hits + big.misses
            print(f"ops={total} hit ratio={big.hits / total:.2f} evictions={big.evic-
tions}")
```

Output: `['d', 'a', 'c'] None`, then a line such as `ops=111952 hit ratio=0.33 evictions=32220` (`ops` counts only the `get` calls, about 70% of the 160,000 operations; the exact numbers vary with thread scheduling, while the invariants are asserted).

What to notice:

- Every operation does a constant number of map operations and pointer swaps: $O(1)$ time. Memory is one node per entry plus a map slot, roughly 100–200 bytes of overhead per entry in CPython, which matters when n reaches millions.
- The stress test checks *structural* invariants after 160,000 concurrent operations: the list and the map hold the same 100 keys and every value is intact. Without the lock, two threads can interleave the pointer assignments of `_unlink` and `_push_front` and lose or duplicate nodes; CPython's global interpreter lock makes this rare but does not prevent it, and free-threaded builds make it likely.
- The hit ratio of about 0.33 is what theory predicts for uniformly random keys: capacity 100 over 300 keys. Real workloads are skewed and do much better (§20.3.3).

Concurrency options

One lock serializes all operations, which is fine up to a few million operations per second in a compiled language and far beyond what one CPython process can issue. Beyond that, **lock striping** splits the cache into k independent LRU segments keyed by `hash(key) mod k`, each with its own lock (approximate global LRU, near-linear scaling). Libraries such as Caffeine avoid locking on reads entirely by recording accesses in buffers and replaying them under a lock in batches. Neither changes the class design: both are new implementations of `Cache`.

INDUSTRY EXAMPLE

Standard-library LRU implementations. CPython's `functools.lru_cache` (pure-Python fallback in `Lib/functools.py`) uses exactly this structure: a dictionary from key to link and a circular doubly linked list with a root sentinel, guarded by a lock. Java's `LinkedHashMap` constructed with `accessOrder = true` maintains entries in access order, and its documentation recommends overriding `removeEldestEntry` to build an LRU cache. *Evidence: Primary* — [Python Software Foundation: functools source; Oracle: LinkedHashMap]

Extensions and follow-up questions

- *Use `OrderedDict`?* Yes in production Python (`move_to_end`, `popitem(last=False)`); the interviewer usually wants the underlying structure first.
- *Add TTL.* Store an expiry in each node and treat expired entries as misses (lazy expiry); combine with Exercise 7's sweeper.
- *Evict by size in bytes rather than count.* Track a running byte total and evict from the tail until the new entry fits.
- *Prevent one scan from flushing the cache.* Segmented LRU or W-TinyLFU (§20.9.3), behind the same interface.

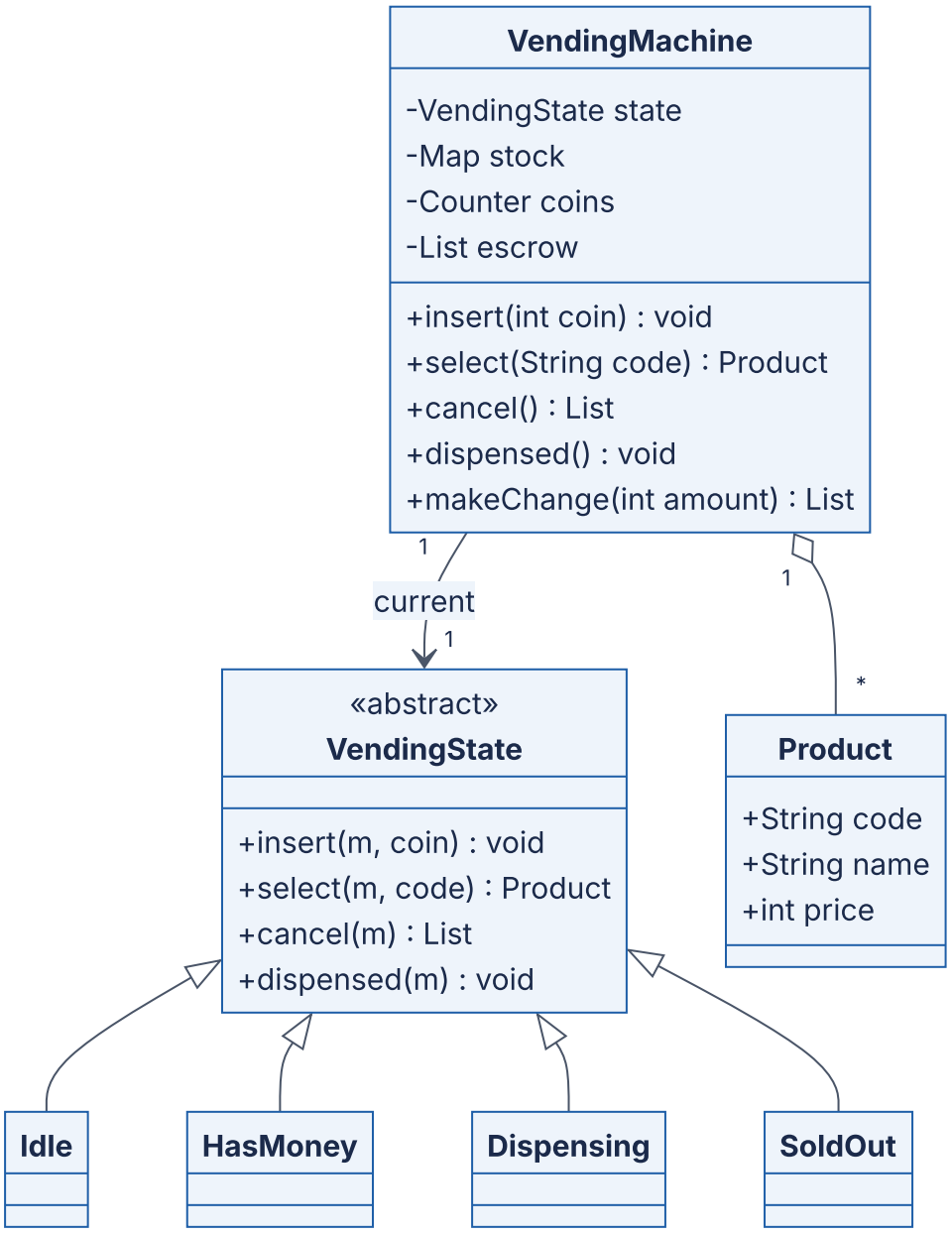
38.10.4 Exercise 4: Vending machine

Requirements

1. Customers insert coins (5, 10, 25 and 100 cents), select a product by slot code, receive the product and change, or cancel for a full refund.
2. A selection fails with a clear message if the slot is empty, the balance is short, or the machine cannot make change.
3. While the motor is dispensing, all input is rejected; when every slot is empty the machine is *sold out*.
4. Extensible: adding a state (for example *maintenance*) must not require editing every method of the machine.

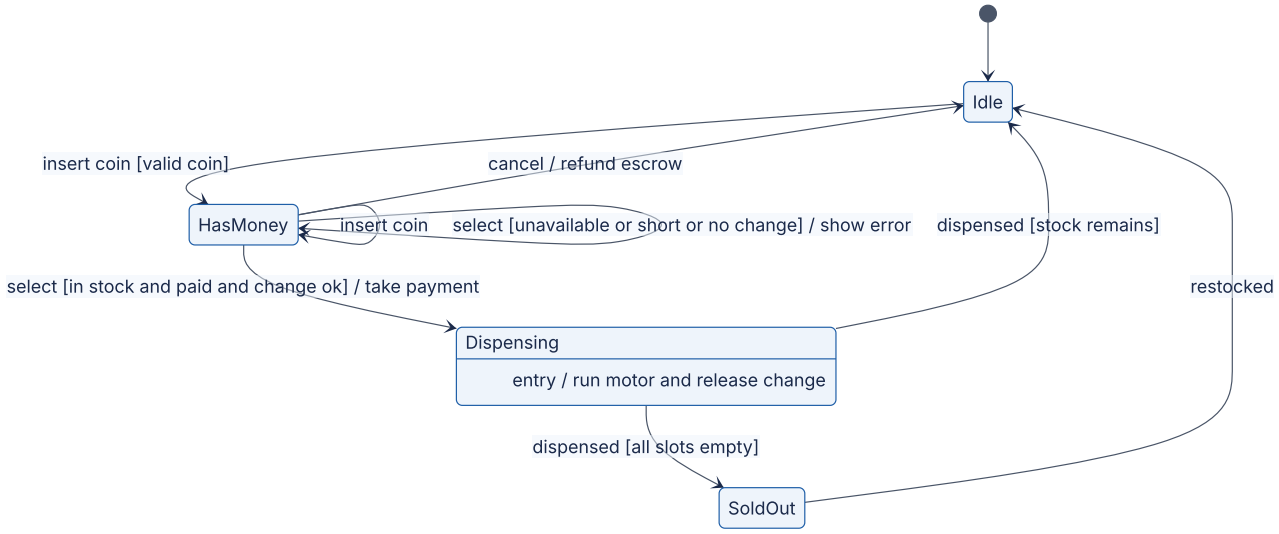
The fourth requirement is the reason for the **State pattern** (§37.15). Behavior depends heavily on state, and the naive implementation, one `if state == ...` chain in every method, grows with states × events and scatters each state's rules across the class.

Figure 38.14 — Vending machine class diagram (State pattern).



Reading the diagram. `VendingMachine` is the pattern's *Context*: its public methods forward every event to the current `VendingState` object. The abstract state rejects every event by default; each concrete state overrides only the events that are legal in it and decides the next state. Adding *Maintenance* means one new subclass and the transitions into it. Products are aggregated (the catalog is configured from outside). The coin box and escrow (coins inserted for the current purchase) are the machine's money state, and keeping escrow separate from the coin box is what makes `cancel` a simple refund.

Figure 38.15 — Vending machine state diagram.



Reading the diagram. From *Idle*, a valid coin moves the machine to *HasMoney*, where more coins accumulate. A selection that fails one of its guards leaves the state unchanged and shows an error; a successful one takes payment and enters *Dispensing*, whose entry action drives the motor and returns change. Only the `dispensed` event from the motor sensor leaves *Dispensing*, so coins or selections made mid-vend are rejected. When the last item is sold the machine goes to *SoldOut* until restocked. The failure point is the motor: if `dispensed` never arrives, the machine is stuck, so a real controller adds a timeout transition to a *Fault* state with a refund.

Code

```
        from __future__ import annotations

        from abc import ABC
        from collections import Counter
        from dataclasses import dataclass

        COINS = (100, 25, 10, 5)                                # cents, largest first


        class InvalidAction(Exception):
            pass


        @dataclass(frozen=True)
        class Product:
            code: str
            name: str
            price: int                                          # cents


        class VendingState(ABC):
            """Every event is rejected unless a concrete state overrides it."""
            name = "abstract"

            def insert(self, m: "VendingMachine", coin: int) -> None:
                raise InvalidAction(f"cannot insert coins while {self.name}")

            def select(self, m: "VendingMachine", code: str):
                raise InvalidAction(f"cannot select while {self.name}")

            def cancel(self, m: "VendingMachine") -> list[int]:
                raise InvalidAction(f"cannot cancel while {self.name}")

            def dispensed(self, m: "VendingMachine") -> None:
                raise InvalidAction(f"nothing is being dispensed while {self.name}")


        class Idle(VendingState):
            name = "idle"

            def insert(self, m, coin):
                m.accept(coin)
                m.state = HasMoney()

            def cancel(self, m):
                return []


        class HasMoney(VendingState):
            name = "has_money"

            def insert(self, m, coin):
                m.accept(coin)
```

```
            def select(self, m, code):
                product = m.products.get(code)
                if product is None or m.stock[code] == 0:
                    raise InvalidAction(f"{code} unavailable")
                if m.balance < product.price:
                    raise InvalidAction(f"insert {product.price - m.balance} more
cents")
                change = m.make_change(m.balance - product.price)
                if change is None:
                    raise InvalidAction("cannot make change; press cancel")
                m.complete_sale(code, change)
                m.state = Dispensing()
                return product, change

            def cancel(self, m):
                refund, m.escrow = m.escrow, []
                m.state = Idle()
                return refund


        class Dispensing(VendingState):
            name = "dispensing"

            def dispensed(self, m):                          # event from the motor sen-
sor
                m.state = SoldOut() if sum(m.stock.values()) == 0 else Idle()


        class SoldOut(VendingState):
            name = "sold_out"

            def cancel(self, m):
                return []


        class VendingMachine:                                # the Context of the State
pattern
            def __init__(self, products: list[Product], stock: dict[str, int], coins:
Counter):
                self.products = {p.code: p for p in products}
                self.stock, self.coins = dict(stock), Counter(coins)
                self.escrow: list[int] = []
                self.state: VendingState = Idle() if sum(self.stock.values()) else
SoldOut()

            # Public API: each call is delegated to the current state object.
            def insert(self, coin): self.state.insert(self, coin)
            def select(self, code): return self.state.select(self, code)
            def cancel(self): return self.state.cancel(self)
            def dispensed(self): self.state.dispensed(self)

            @property
            def balance(self) -> int:
                return sum(self.escrow)

            def accept(self, coin: int) -> None:
                if coin not in COINS:
```

```
                    raise InvalidAction(f"rejected coin {coin}")
                self.escrow.append(coin)

            def make_change(self, amount: int) -> list[int] | None:
                available = self.coins + Counter(self.escrow)    # inserted coins can be
reused
                out = []
                for coin in COINS:
                    n = min(amount // coin, available[coin])
                    out += [coin] * n
                    amount -= n * coin
                return out if amount == 0 else None

            def complete_sale(self, code: str, change: list[int]) -> None:
                self.coins += Counter(self.escrow)
                self.coins -= Counter(change)
                self.escrow = []
                self.stock[code] -= 1


        if __name__ == "__main__":
            vm = VendingMachine([Product("A1", "Water", 65), Product("B2", "Chips",
150)],
                                {"A1": 1, "B2": 0}, Counter({25: 2, 10: 2, 5: 2}))
            for action in (lambda: vm.select("A1"), lambda: vm.dispensed()):
                try:
                    action()
                except InvalidAction as e:
                    print("idle:", e)
            vm.insert(100)
            print("balance", vm.balance, "state", vm.state.name)
            try:
                vm.select("B2")
            except InvalidAction as e:
                print("has_money:", e)
            product, change = vm.select("A1")                # 100 - 65 = 35 -> 25 + 10
            print(product.name, "change", change, "state", vm.state.name)
            try:
                vm.insert(25)
            except InvalidAction as e:
                print("dispensing:", e)
            vm.dispensed()
            print("state", vm.state.name, "coin box", dict(vm.coins))
            assert change == [25, 10] and vm.state.name == "sold_out"
```

Output:

```
idle: cannot select while idle
idle: nothing is being dispensed while idle
balance 100 state has_money
has_money: B2 unavailable
Water change [25, 10] state dispensing
dispensing: cannot insert coins while dispensing
state sold_out coin box {25: 1, 10: 1, 5: 2, 100: 1}
```

What to notice:

- `VendingMachine` contains no conditionals on state; each rule lives in exactly one state class (Single Responsibility per state, Open/Closed for new states).
- `make_change` may use the coins just inserted, which matters when the coin box is nearly empty.
- Greedy change-making is optimal for unlimited US-style coins but can fail with *limited* coins: to return 30 cents from {one 25, three 10s}, greedy takes the 25 and then cannot make 5, although 10 + 10 + 10 works. A dynamic-programming search over available counts fixes it; the interface does not change.

Patterns used

State (§37.15) for behavior by state; the machine is the Context; states are stateless and could be shared singletons.

Extensions and follow-up questions

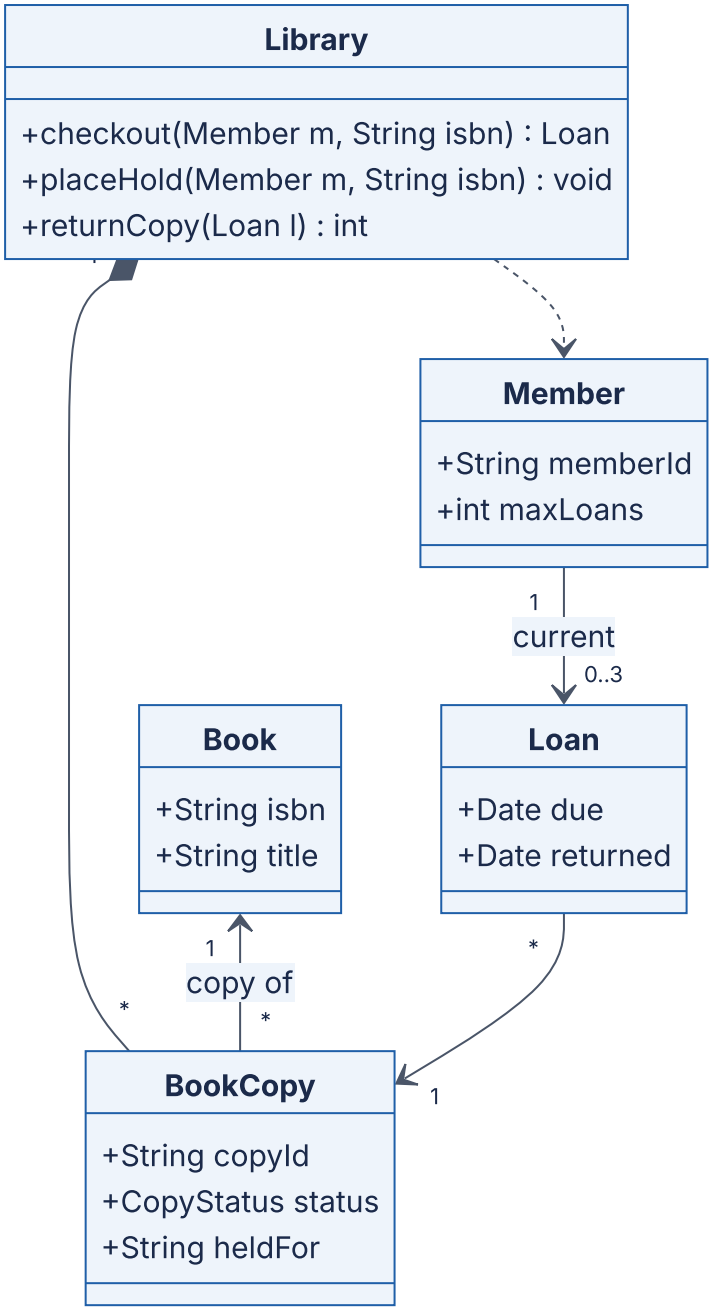
- *Card payments?* A `PaymentMethod` strategy; the *HasMoney* state becomes *Authorized* for cards.
- *Concurrency?* A physical machine has one user, but its controller receives events from buttons, the coin mechanism and the motor; serialize them through one event queue rather than locks in every state.
- *Remote telemetry and restocking alerts?* Observer on stock changes.
- *Why not the transition table of §38.6?* A table suits uniform behavior; the State pattern suits states with substantially different logic. Both are defensible; say which and why.

38.10.5 Exercise 5: Library management system

Requirements. Members borrow and return physical copies of books (21-day loans, at most 3 at a time); a late return is fined 25 cents per day, capped at $10; when no copy is available a member places a hold, and the next returned copy is set aside for the first member in the queue, who is notified.

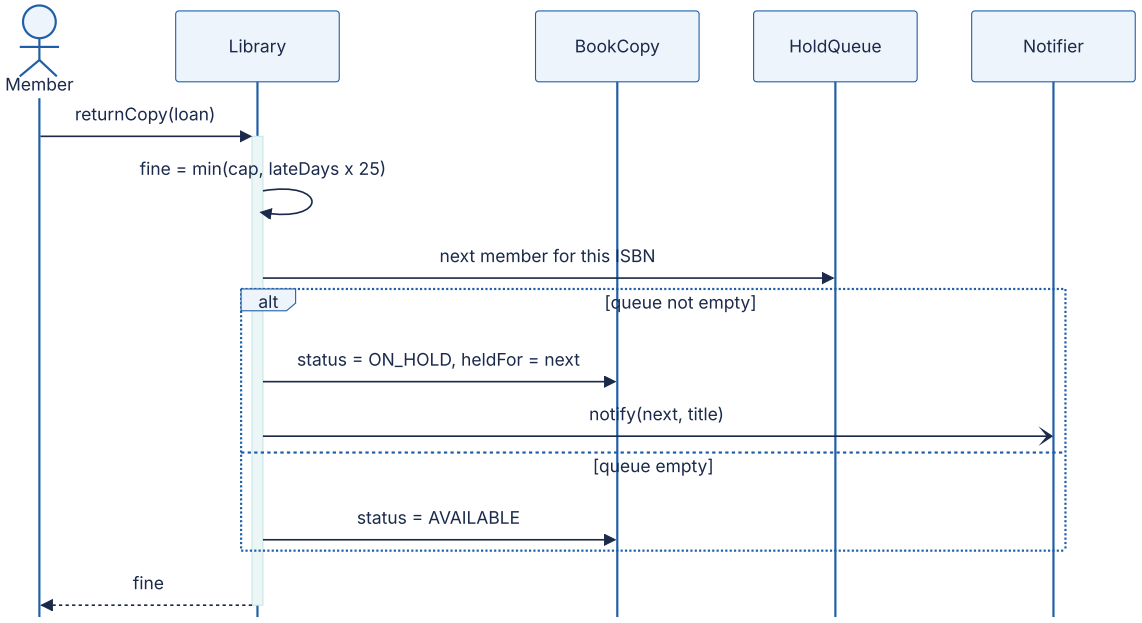
The key modeling decision is separating the *title* (`Book`, one per ISBN) from the *physical item* (`BookCopy`, many per title). Candidates who merge them cannot express "the library owns four copies of this book, two are on loan", and the hold queue belongs to the title, not to a copy. Librarians call the same distinction *manifestation* versus *item* [IFLA 1998]. `Loan` is an association class (§38.8.2) between member and copy that carries the due date.

Figure 38.16 — Library class diagram.



Reading the diagram. Each `BookCopy` refers to one `Book`; one book has many copies. A `Member` has at most three current `Loan`s, and each loan refers to exactly one copy. The library owns its copies and the hold queues (a FIFO of member IDs per ISBN, kept as an attribute in the code). In a database this becomes `books`, `copies`, `members`, `loans` and `holds` tables; the concurrency hazard is two checkouts of the last copy, solved with a conditional update on the copy's status.

Figure 38.17 — Sequence diagram for returning a copy that has a waiting hold.



Reading the diagram. Returning computes the fine, then checks the title's hold queue. If someone is waiting, the copy is set aside for them and a notification is sent *asynchronously* (an e-mail must not block the return desk); otherwise the copy becomes available. A later checkout by anyone other than the holder is refused.

```
        from __future__ import annotations

        from collections import defaultdict, deque
        from dataclasses import dataclass, field
        from datetime import date, timedelta
        from enum import Enum
        from typing import Callable


        class CopyStatus(Enum):
            AVAILABLE = "available"
            ON_LOAN = "on_loan"
            ON_HOLD = "on_hold"          # set aside for the member at the head of the
hold queue


        @dataclass(frozen=True)
        class Book:                      # the title: one per ISBN
            isbn: str
            title: str


        @dataclass
        class BookCopy:                  # the physical item: many per Book
            copy_id: str
            book: Book
            status: CopyStatus = CopyStatus.AVAILABLE
            held_for: str | None = None


        @dataclass
        class Member:
            member_id: str
            max_loans: int = 3
            loans: list["Loan"] = field(default_factory=list)


        @dataclass
        class Loan:
            copy: BookCopy
            member: Member
            due: date
            returned: date | None = None


        class Library:
            LOAN_DAYS, FINE_PER_DAY, FINE_CAP = 21, 25, 1_000      # cents

            def __init__(self, today: Callable[[], date]):
                self.today = today
                self.copies: dict[str, BookCopy] = {}
                self.by_isbn: dict[str, list[BookCopy]] = defaultdict(list)
                self.holds: dict[str, deque[str]] = defaultdict(deque)
                self.listeners: list[Callable[[str, str], None]] = []    # Observer

            def add_copy(self, copy: BookCopy) -> None:
```

```
                self.copies[copy.copy_id] = copy
                self.by_isbn[copy.book.isbn].append(copy)

            def checkout(self, member: Member, isbn: str) -> Loan:
                if len(member.loans) >= member.max_loans:
                    raise ValueError("loan limit reached")
                copy = next((c for c in self.by_isbn[isbn]
                             if c.status is CopyStatus.AVAILABLE
                             or (c.status is CopyStatus.ON_HOLD and c.held_for ==
member.member_id)), None)
                if copy is None:
                    raise LookupError("no copy available; place a hold")
                copy.status, copy.held_for = CopyStatus.ON_LOAN, None
                loan = Loan(copy, member, self.today() + timedelta(days=self.LOAN_DAYS))
                member.loans.append(loan)
                return loan

            def place_hold(self, member: Member, isbn: str) -> None:
                self.holds[isbn].append(member.member_id)

            def return_copy(self, loan: Loan) -> int:
                loan.returned = self.today()
                loan.member.loans.remove(loan)
                late_days = max(0, (loan.returned - loan.due).days)
                copy, queue = loan.copy, self.holds[loan.copy.book.isbn]
                if queue:                                       # hand the copy to the
next in line
                    copy.status, copy.held_for = CopyStatus.ON_HOLD, queue.popleft()
                    for notify in self.listeners:
                        notify(copy.held_for, copy.book.title)
                else:
                    copy.status = CopyStatus.AVAILABLE
                return min(self.FINE_CAP, late_days * self.FINE_PER_DAY)


        if __name__ == "__main__":
            day = [date(2026, 1, 5)]
            lib = Library(lambda: day[0])
            lib.listeners.append(lambda m, t: print(f"notify {m}: '{t}' is ready for
pickup"))
            sicp = Book("978-0262510875", "SICP")
            lib.add_copy(BookCopy("c1", sicp))
            ann, bob = Member("ann"), Member("bob")
            loan = lib.checkout(ann, sicp.isbn)
            try:
                lib.checkout(bob, sicp.isbn)
            except LookupError as e:
                print("bob:", e)
            lib.place_hold(bob, sicp.isbn)
            day[0] = loan.due + timedelta(days=6)               # six days late
            print("fine (cents):", lib.return_copy(loan))
            try:
                lib.checkout(Member("cat"), sicp.isbn)         # held for bob, not cat
            except LookupError as e:
                print("cat:", e)
            print("bob gets", lib.checkout(bob, sicp.isbn).copy.copy_id)
```

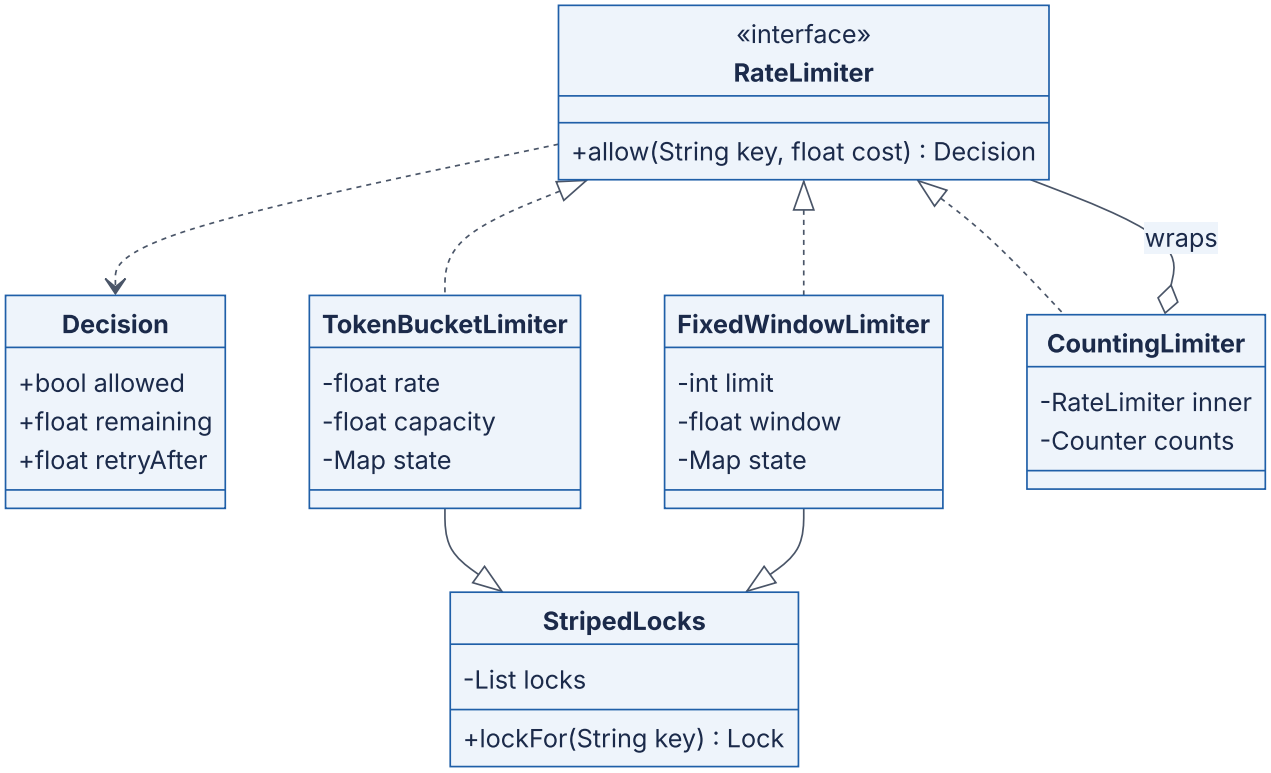
Output: `bob: no copy available; place a hold`, `notify bob: 'SICP' is ready for pickup`, `fine (cents): 150` (6 days × 25), `cat: no copy available; place a hold`, `bob gets c1`.

Patterns and follow-ups. Listeners implement **Observer** (§37.8) so e-mail, SMS and push channels can be added without touching `Library`. Typical follow-ups: hold expiry (a copy on hold for more than 3 days passes to the next member, needing a scheduled sweep like Exercise 7's); different loan policies per member type (a `LoanPolicy` strategy, not `StudentMember` subclasses); search by title or author (an index from words to ISBNs, or a search engine, §16); and concurrent checkouts across branches (a database transaction with `UPDATE copies SET status = 'ON_LOAN' WHERE copy_id = ? AND status = 'AVAILABLE'`, checking the affected row count).

38.10.6 Exercise 6: Rate limiter class design

Requirements. Offer `allow(key, cost)` returning a decision (allowed, remaining, retry-after); support at least two algorithms selected by configuration; be safe under concurrent calls; let operators add metrics without modifying algorithms. The token bucket algorithm itself, its formula and its parameter choices are in §22.4, and distributed enforcement is in §22.8; this exercise is about the *class design* around them.

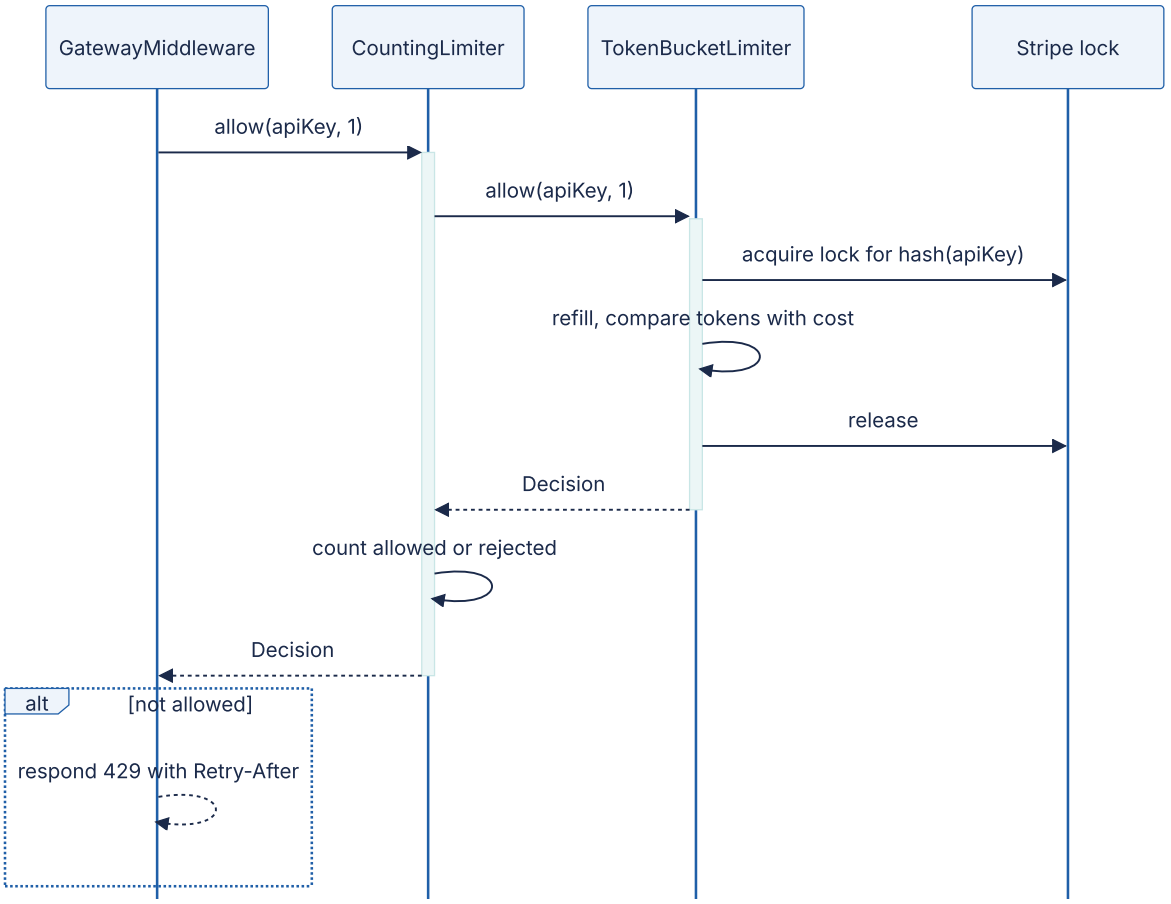
Figure 38.18 — Rate limiter class diagram.



Reading the diagram. Callers (gateway middleware) depend only on `RateLimiter` and the immutable `Decision` value object. Two algorithms realize the interface; `CountingLimiter` is a **Decorator** (§37.10) that also realizes it and wraps any other limiter

to add metrics. Both algorithms inherit lock striping, a fixed array of locks indexed by key hash, so unrelated keys rarely contend while operations on one key stay atomic. A factory (`build_limiter`) builds the right object from configuration.

Figure 38.19 — Sequence diagram for one admission decision.



Reading the diagram. The middleware calls the decorator, which delegates to the token bucket. The bucket takes only its key’s stripe lock, refills lazily and decides, so the read-modify-write is atomic per key. The decorator counts the outcome; the middleware turns a rejection into HTTP 429 with `Retry-After` from the decision (§22.3.4). Everything is in process; in a fleet, the same interface would be implemented by a Redis-backed class (§22.8.2) and callers would not change.

```
        from __future__ import annotations

        import threading
        import time
        from abc import ABC, abstractmethod
        from collections import Counter
        from dataclasses import dataclass


        @dataclass(frozen=True)
        class Decision:
            allowed: bool
            remaining: float
            retry_after: float          # seconds; 0 when allowed


        class RateLimiter(ABC):                             # what callers depend on
(DIP)
            @abstractmethod
            def allow(self, key: str, cost: float = 1.0) -> Decision: ...


        class _Striped:
            """Lock striping: N locks shared by all keys, so unrelated keys rarely con-
tend."""
            def __init__(self, stripes: int = 64):
                self._locks = [threading.Lock() for _ in range(stripes)]

            def lock_for(self, key: str) -> threading.Lock:
                return self._locks[hash(key) % len(self._locks)]


        class TokenBucketLimiter(RateLimiter, _Striped):
            """Per-key token bucket; the algorithm itself is explained in §22.4."""
            def __init__(self, rate: float, capacity: float, clock=time.monotonic):
                _Striped.__init__(self)
                self.rate, self.capacity, self.clock = rate, capacity, clock
                self._state: dict[str, tuple[float, float]] = {}       # key -> (tokens,
last)

            def allow(self, key, cost=1.0):
                with self.lock_for(key):
                    now = self.clock()
                    tokens, last = self._state.get(key, (self.capacity, now))
                    tokens = min(self.capacity, tokens + max(0.0, now - last) *
self.rate)
                    if tokens >= cost:
                        self._state[key] = (tokens - cost, now)
                        return Decision(True, tokens - cost, 0.0)
                    self._state[key] = (tokens, now)
                    return Decision(False, tokens, (cost - tokens) / self.rate)


        class FixedWindowLimiter(RateLimiter, _Striped):
            def __init__(self, limit: int, window: float, clock=time.monotonic):
                _Striped.__init__(self)
```

```
                self.limit, self.window, self.clock = limit, window, clock
                self._state: dict[str, tuple[int, int]] = {}           # key -> (window
id, count)

            def allow(self, key, cost=1.0):
                with self.lock_for(key):
                    now = self.clock()
                    wid = int(now // self.window)
                    cur, count = self._state.get(key, (wid, 0))
                    if cur != wid:
                        count = 0
                    if count + cost <= self.limit:
                        self._state[key] = (wid, count + cost)
                        return Decision(True, self.limit - count - cost, 0.0)
                    self._state[key] = (wid, count)
                    return Decision(False, self.limit - count, (wid + 1) * self.window
- now)


        class CountingLimiter(RateLimiter):                 # Decorator: adds metrics,
same interface
            def __init__(self, inner: RateLimiter):
                self.inner, self.counts = inner, Counter()

            def allow(self, key, cost=1.0):
                d = self.inner.allow(key, cost)
                self.counts["allowed" if d.allowed else "rejected"] += 1
                return d


        def build_limiter(config: dict, clock=time.monotonic) -> RateLimiter:   #
Factory
            kind = config["algorithm"]
            if kind == "token_bucket":
                inner = TokenBucketLimiter(config["rate"], config["burst"], clock)
            elif kind == "fixed_window":
                inner = FixedWindowLimiter(config["limit"], config["window_s"], clock)
            else:
                raise ValueError(f"unknown algorithm {kind}")
            return CountingLimiter(inner)


        if __name__ == "__main__":
            t = [0.0]
            rl = build_limiter({"algorithm": "token_bucket", "rate": 10, "burst": 20},
lambda: t[0])
            results = [rl.allow("key-42") for _ in range(25)]
            print(sum(d.allowed for d in results), "allowed; retry after",
                  round(results[-1].retry_after, 2), "s;", dict(rl.counts))
            t[0] = 1.0
            print("after 1 s:", sum(rl.allow("key-42").allowed for _ in range(25)), "al-
lowed")
            print("other key:", rl.allow("key-7").allowed)

            fw = build_limiter({"algorithm": "fixed_window", "limit": 5, "window_s":
60}, lambda: t[0])
            print("fixed window:", [fw.allow("u").allowed for _ in range(6)],
```

```
                  "retry", fw.allow("u").retry_after)

            # 16 threads x 1,000 calls on one key with the clock frozen: exactly 20 may
pass.
            tb = TokenBucketLimiter(rate=10, capacity=20, clock=lambda: 0.0)
            passed = []
            def hammer():
                passed.append(sum(tb.allow("hot").allowed for _ in range(1_000)))
            ts = [threading.Thread(target=hammer) for _ in range(16)]
            for th in ts: th.start()
            for th in ts: th.join()
            assert sum(passed) == 20, passed
            print("concurrent admits:", sum(passed))
```

Output:

```
20 allowed; retry after 0.1 s; {'allowed': 20, 'rejected': 5}
after 1 s: 10 allowed
other key: True
fixed window: [True, True, True, True, True, False] retry 59.0
concurrent admits: 20
```

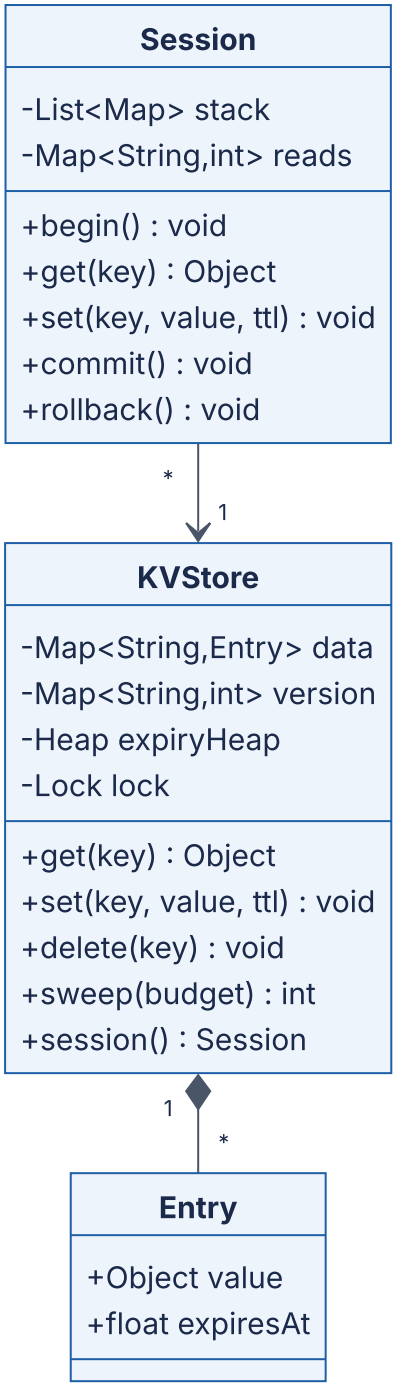
Patterns and follow-ups. Strategy (algorithms behind one interface), **Decorator** (metrics, logging, a "shadow mode" that logs but never rejects), **Factory** (configuration to object). The 16-thread test admits exactly 20 requests with the clock frozen, which proves per-key atomicity. Follow-ups: idle keys accumulate forever (evict with an LRU of keys, Exercise 3, or a TTL); hierarchical limits (a `CompositeLimiter` that must check all buckets *before* debiting any, or debit and refund on rejection); and fail-open versus fail-closed when a remote store is unreachable (§22.8.5), which is another decorator.

38.10.7 Exercise 7: In-memory key-value store with TTL and transactions

Requirements. `get`, `set(key, value, ttl)` and `delete`; keys with a time to live (TTL) disappear when it elapses; clients can `begin`, `commit` and `rollback` transactions, nested to any depth, and see their own uncommitted writes; a commit is all-or-nothing and fails if another client changed a key that this transaction read.

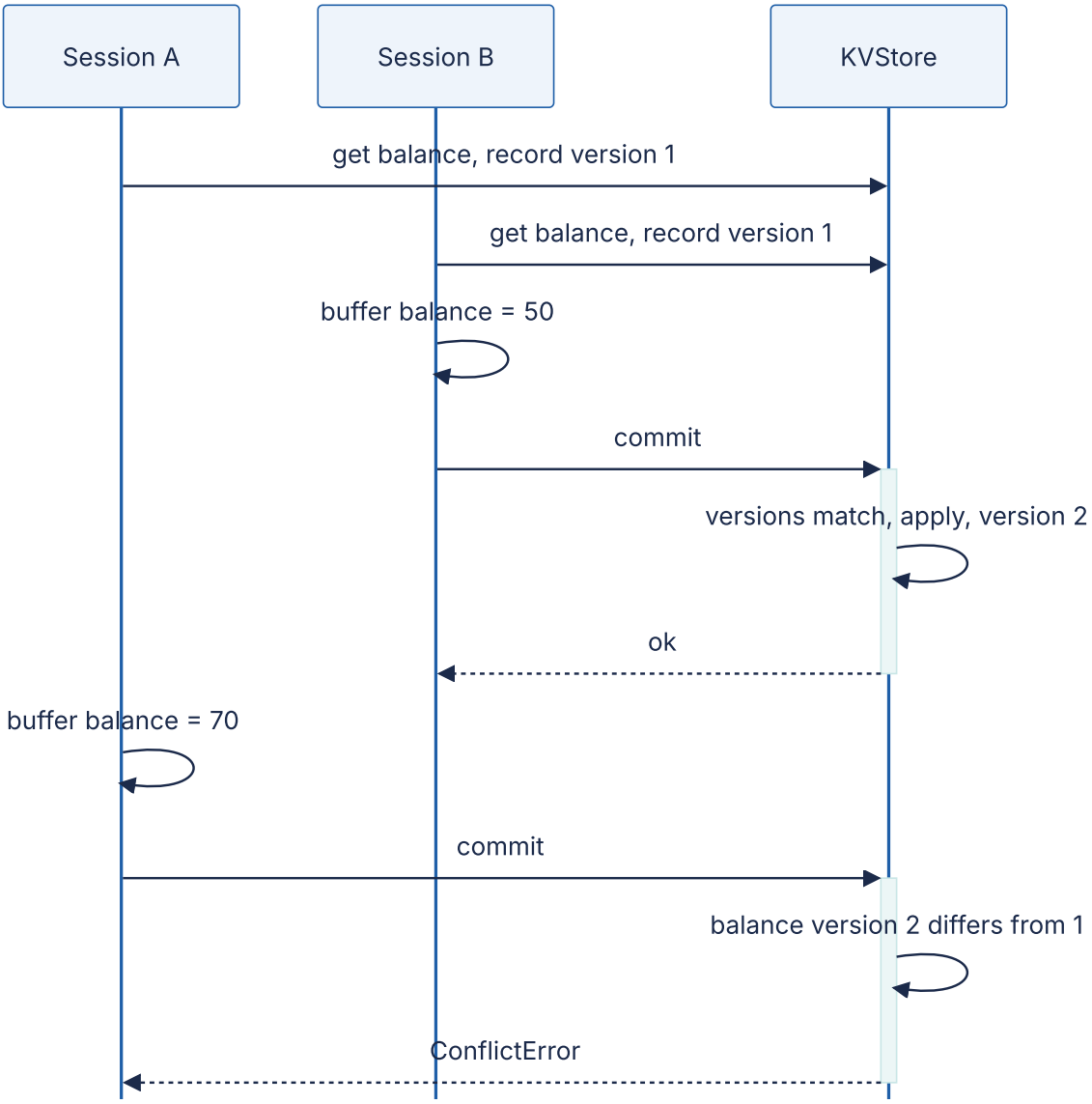
Design. Expiry uses two mechanisms, like Redis [Redis Docs: EXPIRE]: **lazy** expiry on every read (an expired key is deleted when touched) and an **active** sweep from a min-heap of `(expires_at, key)` with a work budget, so memory is reclaimed even for keys nobody reads. Transactions live in a per-client `Session` holding a stack of write buffers: `begin` pushes, `rollback` pops, an inner `commit` merges into the level below, and the outermost `commit` validates and applies the buffer under the store lock. Validation is **optimistic concurrency** (§11.15.3): the session records the version of each key it read, and the commit fails if any version changed.

Figure 38.20 — Key-value store class diagram.



Reading the diagram. The store composes its entries and keeps a version counter per key that increases on every write, delete or expiry. Many sessions share one store. Each session’s buffers and read-set are private, so uncommitted writes are invisible to other clients (no dirty reads). The store lock is the only synchronization point, and commits hold it for validation and application together.

Figure 38.21 — Sequence diagram for two conflicting transactions.



Reading the diagram. Both sessions read `balance` at version 1 and buffer their writes locally. B commits first; validation passes and the write bumps the version to 2. A's commit then finds a changed version and fails instead of silently overwriting B's debit (a lost update, §11.13.1). A retries the whole transaction with fresh reads.

```
        from __future__ import annotations

        import heapq
        import threading
        import time
        from collections import defaultdict
        from dataclasses import dataclass

        TOMBSTONE = object()                                  # "deleted inside this
transaction"


        class ConflictError(Exception):
            pass


        @dataclass
        class Entry:
            value: object
            expires_at: float | None


        class KVStore:
            def __init__(self, clock=time.monotonic):
                self.clock = clock
                self._data: dict[str, Entry] = {}
                self._version: dict[str, int] = defaultdict(int)     # bumps on every
change
                self._heap: list[tuple[float, str]] = []              # (expires_at,
key)
                self._lock = threading.Lock()

            def _live(self, key: str) -> Entry | None:               # lazy (passive)
expiry
                e = self._data.get(key)
                if e is not None and e.expires_at is not None and e.expires_at <=
self.clock():
                    del self._data[key]
                    self._version[key] += 1
                    return None
                return e

            def _apply(self, writes: dict) -> None:                   # caller holds the
lock
                now = self.clock()
                for key, (value, ttl) in writes.items():
                    if value is TOMBSTONE:
                        self._data.pop(key, None)
                    else:
                        exp = None if ttl is None else now + ttl
                        self._data[key] = Entry(value, exp)
                        if exp is not None:
                            heapq.heappush(self._heap, (exp, key))
                    self._version[key] += 1

            def get(self, key):
```

```
                with self._lock:
                    e = self._live(key)
                    return None if e is None else e.value

            def set(self, key, value, ttl: float | None = None) -> None:
                with self._lock:
                    self._apply({key: (value, ttl)})

            def delete(self, key) -> None:
                with self._lock:
                    self._apply({key: (TOMBSTONE, None)})

            def sweep(self, budget: int = 1_000) -> int:              # active expiry,
bounded work
                removed = 0
                with self._lock:
                    now = self.clock()
                    while self._heap and self._heap[0][0] <= now and budget > 0:
                        exp, key = heapq.heappop(self._heap)
                        budget -= 1
                        e = self._data.get(key)
                        if e is not None and e.expires_at == exp:     # skip stale heap
entries
                            del self._data[key]
                            self._version[key] += 1
                            removed += 1
                return removed

            def session(self) -> "Session":
                return Session(self)


        class Session:
            """Nested transactions with read-your-writes and optimistic conflict detec-
tion."""
            def __init__(self, store: KVStore):
                self.store = store
                self._stack: list[dict] = []                          # one write buffer
per level
                self._reads: dict[str, int] = {}                      # key -> version
first seen

            def begin(self) -> None:
                self._stack.append({})

            def get(self, key):
                for layer in reversed(self._stack):
                    if key in layer:
                        value = layer[key][0]
                        return None if value is TOMBSTONE else value
                with self.store._lock:
                    e = self.store._live(key)
                    if self._stack:
                        self._reads.setdefault(key, self.store._version[key])
                    return None if e is None else e.value

            def set(self, key, value, ttl=None) -> None:
```

```
                if self._stack:
                    self._stack[-1][key] = (value, ttl)               # TTL counts from
commit
                else:
                    self.store.set(key, value, ttl)

            def delete(self, key) -> None:
                self.set(key, TOMBSTONE)

            def rollback(self) -> None:
                if not self._stack:
                    raise RuntimeError("no transaction")
                self._stack.pop()
                if not self._stack:
                    self._reads.clear()

            def commit(self) -> None:
                if not self._stack:
                    raise RuntimeError("no transaction")
                top = self._stack.pop()
                if self._stack:                                       # inner commit:
merge down
                    self._stack[-1].update(top)
                    return
                reads, self._reads = self._reads, {}
                with self.store._lock:
                    for key, seen in reads.items():
                        self.store._live(key)
                        if self.store._version[key] != seen:
                            raise ConflictError(f"{key!r} changed since it was read")
                    self.store._apply(top)                            # all writes or
none


        if __name__ == "__main__":
            t = [0.0]
            kv = KVStore(clock=lambda: t[0])
            kv.set("session:9", "alice", ttl=10)
            t[0] = 9.9
            print("t=9.9:", kv.get("session:9"))
            t[0] = 10.0
            print("t=10.0:", kv.get("session:9"))

            s = kv.session()
            s.begin(); s.set("x", 1)
            s.begin(); s.set("x", 2); print("inner sees", s.get("x")); s.rollback()
            print("outer sees", s.get("x"), "store sees", kv.get("x"))
            s.commit(); print("after commit store sees", kv.get("x"))

            kv.set("balance", 100)
            a, b = kv.session(), kv.session()
            a.begin(); a.set("balance", a.get("balance") - 30)
            b.begin(); b.set("balance", b.get("balance") - 50); b.commit()
            try:
                a.commit()
            except ConflictError as e:
                print("a:", e, "| balance =", kv.get("balance"))
```

```
            for i in range(1_000):
                kv.set(f"tmp:{i}", i, ttl=5 if i % 2 else 50)
            t[0] += 6
            print("swept", kv.sweep(), "keys; remaining tmp keys:",
                  sum(kv.get(f"tmp:{i}") is not None for i in range(1_000)))
```

Output:

```
t=9.9: alice
t=10.0: None
inner sees 2
outer sees 1 store sees None
after commit store sees 1
a: 'balance' changed since it was read | balance = 50
swept 500 keys; remaining tmp keys: 500
```

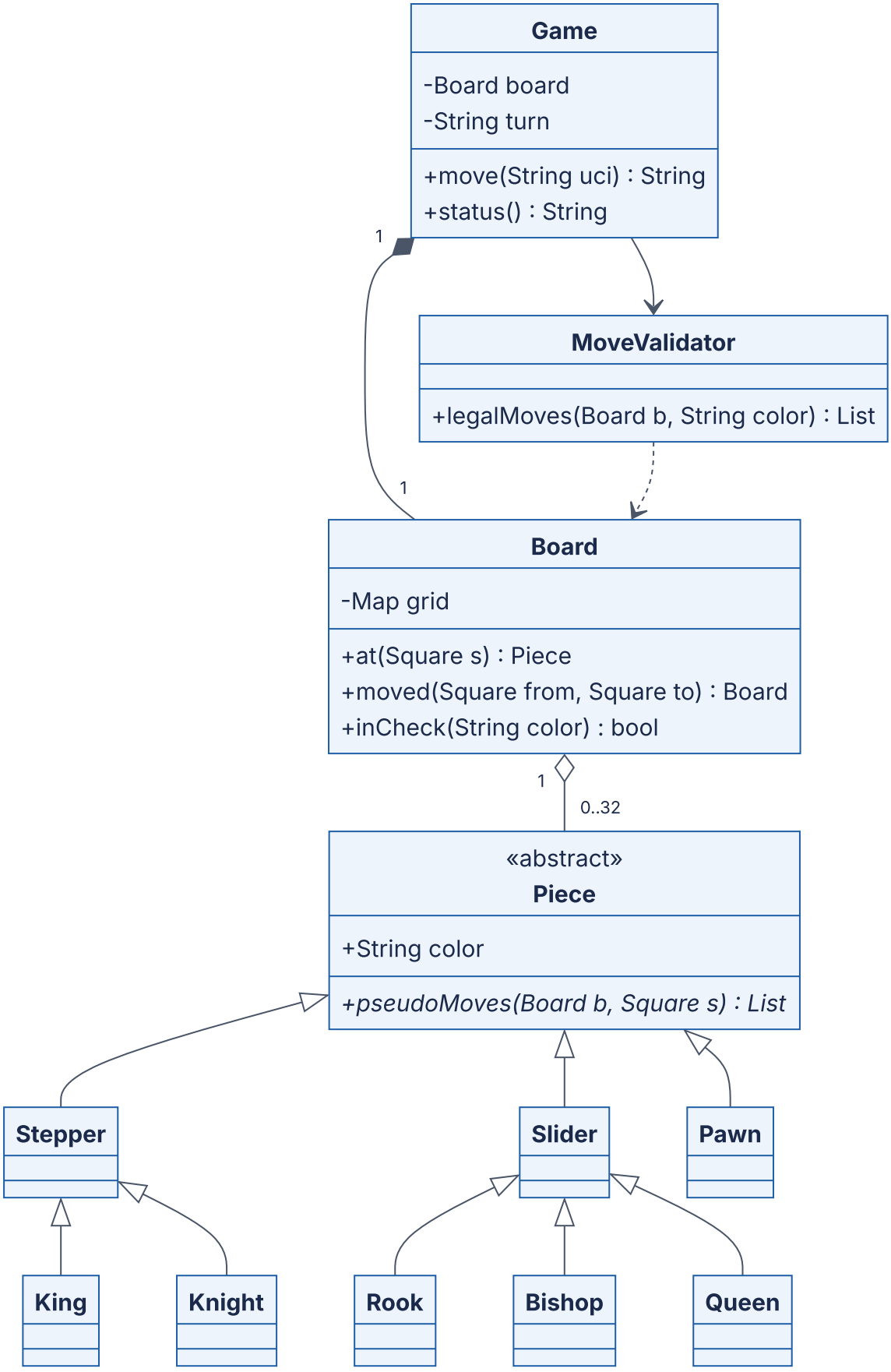
Patterns and follow-ups. Write buffers are a form of the **Command** pattern (recorded operations applied later, §37.12); the session-per-client design is a **Unit of Work**. A TTL inside a transaction counts from commit time, which you should state. Follow-ups: durability (an append-only log of committed buffers, fsync policy per §6.14); isolation level (every read is validated at commit under the store lock, so committed transactions are serializable in commit order, but a transaction can *observe* a mix of old and new values before it aborts, because reads do not come from a snapshot; MVCC snapshots, §11.14, fix that); memory limits (evict with LRU, Exercise 3); and why Redis transactions differ: `MULTI` / `EXEC` queue commands and run them without interleaving but do not roll back a failed command, while `WATCH` provides the same optimistic check-and-set as this design [Redis Docs: Transactions].

38.10.8 Exercise 8: Chess (move validation)

Requirements. Two players alternate moves on a standard board; the system rejects illegal moves, reports check, checkmate and stalemate. Out of scope for the first pass: castling, en passant, promotion, draw rules (all follow-ups).

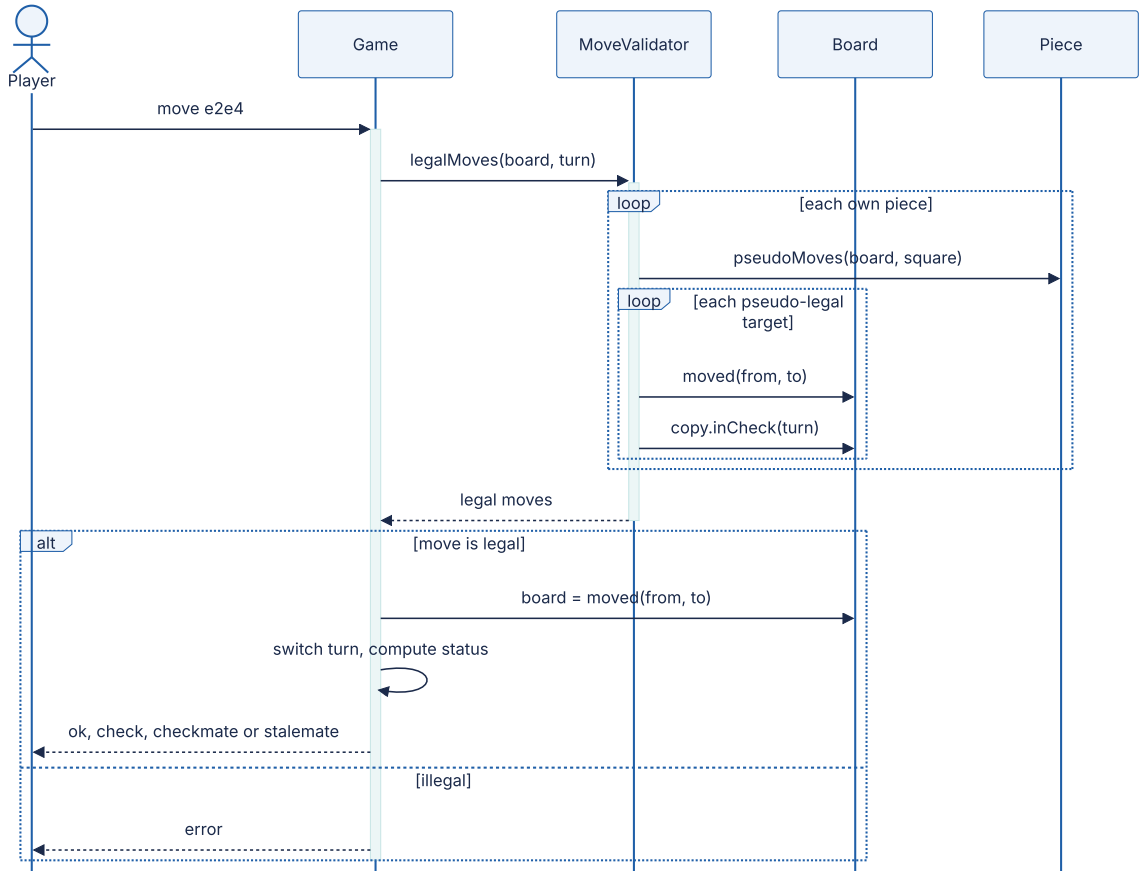
The key design decision is validating moves in two layers. Each piece knows only its own geometry and produces **pseudo-legal** moves (moves that follow the piece's movement rules and do not land on a friendly piece). A separate `MoveValidator` filters them to **legal** moves by applying each one to a copy of the board and rejecting it if the mover's king is attacked afterwards. This keeps check logic, which concerns the whole position, out of every piece class, and is the same split that the python-chess library exposes as `pseudo_legal_moves` and `legal_moves` [python-chess docs].

Figure 38.22 — Chess class diagram.


Game
-Board board
-String turn
+move(String uci) : String
+status() : String
1
MoveValidator
+legalMoves(Board b, String color) : List
1
Board
-Map grid
+at(Square s) : Piece
+moved(Square from, Square to) : Board
+inCheck(String color) : bool
1
0..32
«abstract»
Piece
+String color
+pseudoMoves(Board b, Square s) : List
Stepper
Slider
Pawn
King
Knight
Rook
Bishop
Queen

Reading the diagram. `Game` owns the board and the turn and delegates legality to `MoveValidator`. The piece hierarchy is a justified use of inheritance: behavior genuinely differs by kind and every subclass is substitutable for `Piece` (Liskov, §36.5). The two intermediate classes remove duplication: a `Stepper` hops by fixed offsets (king, knight); a `Slider` follows rays until blocked (rook, bishop, queen), so a queen is just "a slider with eight rays". Pieces are immutable and shared between board copies; `Board.moved` returns a new board, which removes all undo bugs from validation.

Figure 38.23 — Sequence diagram for validating and applying a move.



Reading the diagram. For every own piece, the validator asks for pseudo-legal targets, builds the resulting board, and keeps the move only if the mover's king is not attacked. `Game` applies a legal move, switches the turn and computes the status by generating the opponent's legal moves: none plus check is checkmate, none without check is stalemate. The cost is visible as two nested loops; about 20–40 moves per position, each needing an attack test, is trivial for a game server but too slow for an engine searching millions of positions, which uses bitboards and incremental updates instead.

```
        from __future__ import annotations

        from abc import ABC, abstractmethod

        Square = tuple[int, int]                                  # (file 0-7, rank 0-
7)
        ORTHO = [(1, 0), (-1, 0), (0, 1), (0, -1)]
        DIAG = [(1, 1), (1, -1), (-1, 1), (-1, -1)]


        def on_board(f: int, r: int) -> bool:
            return 0 <= f < 8 and 0 <= r < 8


        class Piece(ABC):
            symbol = "?"

            def __init__(self, color: str):
                self.color = color                                 # "w" or "b"

            @abstractmethod
            def pseudo_moves(self, board: "Board", sq: Square) -> list[Square]:
                """Moves that obey this piece's geometry, ignoring whether the own king
is left in check."""


        class Stepper(Piece):                                      # king, knight: one
hop
            offsets: list[Square] = []

            def pseudo_moves(self, board, sq):
                out = []
                for df, dr in self.offsets:
                    t = (sq[0] + df, sq[1] + dr)
                    if on_board(*t) and (board.at(t) is None or board.at(t).color !=
self.color):
                        out.append(t)
                return out


        class Slider(Piece):                                       # bishop, rook,
queen: rays
            rays: list[Square] = []

            def pseudo_moves(self, board, sq):
                out = []
                for df, dr in self.rays:
                    f, r = sq[0] + df, sq[1] + dr
                    while on_board(f, r):
                        occupant = board.at((f, r))
                        if occupant is None:
                            out.append((f, r))
                        else:
                            if occupant.color != self.color:
                                out.append((f, r))                 # capture, then the
ray stops
```

```
                            break
                    f, r = f + df, r + dr
            return out


    class King(Stepper):   symbol, offsets = "K", ORTHO + DIAG
    class Knight(Stepper): symbol, offsets = "N", [(1, 2), (2, 1), (-1, 2), (-2, 1),
                                                   (1, -2), (2, -1), (-1, -2), (-2,
-1)]
    class Bishop(Slider):  symbol, rays = "B", DIAG
    class Rook(Slider):    symbol, rays = "R", ORTHO
    class Queen(Slider):   symbol, rays = "Q", ORTHO + DIAG


    class Pawn(Piece):
        symbol = "P"

        def pseudo_moves(self, board, sq):
            d, start = (1, 1) if self.color == "w" else (-1, 6)
            f, r = sq
            out = []
            if on_board(f, r + d) and board.at((f, r + d)) is None:
                out.append((f, r + d))
                if r == start and board.at((f, r + 2 * d)) is None:
                    out.append((f, r + 2 * d))
            for df in (-1, 1):
                t = (f + df, r + d)
                if on_board(*t) and board.at(t) is not None and board.at(t).color
!= self.color:
                    out.append(t)
            return out


    class Board:
        def __init__(self):
            self.grid: dict[Square, Piece] = {}

        @classmethod
        def standard(cls) -> "Board":
            b, back = cls(), [Rook, Knight, Bishop, Queen, King, Bishop, Knight,
Rook]
            for f, kind in enumerate(back):
                b.grid[(f, 0)], b.grid[(f, 7)] = kind("w"), kind("b")
                b.grid[(f, 1)], b.grid[(f, 6)] = Pawn("w"), Pawn("b")
            return b

        def at(self, sq: Square) -> Piece | None:
            return self.grid.get(sq)

        def moved(self, src: Square, dst: Square) -> "Board":   # copy-on-move: no
undo bugs
            b = Board()
            b.grid = dict(self.grid)
            b.grid[dst] = b.grid.pop(src)
            return b

        def attacked(self, sq: Square, by: str) -> bool:
```

```
                return any(p.color == by and sq in p.pseudo_moves(self, s) for s, p in
self.grid.items())

            def in_check(self, color: str) -> bool:
                king = next(s for s, p in self.grid.items() if isinstance(p, King) and
p.color == color)
                return self.attacked(king, "b" if color == "w" else "w")


        class MoveValidator:
            """Legal = pseudo-legal for the piece AND does not leave the mover's king
attacked."""
            def legal_moves(self, board: Board, color: str) -> list[tuple[Square,
Square]]:
                return [(s, t) for s, p in list(board.grid.items()) if p.color == color
                        for t in p.pseudo_moves(board, s) if not board.moved(s,
t).in_check(color)]


        class Game:
            def __init__(self, validator: MoveValidator | None = None):
                self.board, self.turn = Board.standard(), "w"
                self.validator = validator or MoveValidator()

            @staticmethod
            def parse(sq: str) -> Square:
                return ord(sq[0]) - ord("a"), int(sq[1]) - 1

            def move(self, uci: str) -> str:
                src, dst = self.parse(uci[:2]), self.parse(uci[2:4])
                if (src, dst) not in self.validator.legal_moves(self.board, self.turn):
                    raise ValueError(f"illegal move {uci} for {self.turn}")
                self.board = self.board.moved(src, dst)
                self.turn = "b" if self.turn == "w" else "w"
                return self.status()

            def status(self) -> str:
                has_moves = bool(self.validator.legal_moves(self.board, self.turn))
                if self.board.in_check(self.turn):
                    return "check" if has_moves else "checkmate"
                return "ok" if has_moves else "stalemate"


        if __name__ == "__main__":
            g = Game()
            print("white opening moves:", len(g.validator.legal_moves(g.board, "w")))
# 20
            g.move("e2e4"); g.move("d7d6")
            print("after f1b5:", g.move("f1b5"))                # bishop checks the
king via d7
            try:
                g.move("a7a6")                                  # ignores the check:
illegal
            except ValueError as e:
                print(e)
            print("after c7c6:", g.move("c7c6"))                # blocks the check: le-
gal
```

```
fool = Game()
for uci in ["f2f3", "e7e5", "g2g4"]:
    fool.move(uci)
print("Fool's mate, after d8h4:", fool.move("d8h4"))
```

Output:

```
white opening moves: 20
after f1b5: check
illegal move a7a6 for b
after c7c6: ok
Fool's mate, after d8h4: checkmate
```

Patterns and follow-ups. Polymorphism over a small class hierarchy; validation as a separate policy object (it could become a chain of rules for variants such as Chess960). The 20 opening moves match the known count for the initial position, a quick correctness check. Follow-ups: castling and en passant need *history* (has the king moved? what was the last move?), so the board state grows into a `Position` with castling rights and an en-passant square; promotion turns a move into a value object with an optional piece; undo and replay come from storing moves as **Commands** (§37.12); draw by threefold repetition needs a position hash (Zobrist hashing); and online play turns `Game` into a server-side aggregate whose moves are validated on the server, never trusted from the client.

38.11 Summary and revision notes

SUMMARY

- **UML is a sketching language for design decisions.** UML 2.5.1 defines 14 diagram types; class, sequence, component, state machine and activity diagrams cover nearly all practical needs. Draw only what carries a decision.
- **Class diagrams:** visibility `+ - # ~`; multiplicity `1`, `0..1`, `*`, `1..*` read from the opposite end; six relationships from weakest to strongest coupling: dependency (dashed arrow), association (line), aggregation (hollow diamond), composition (filled diamond), and generalization (hollow triangle); realization (dashed line, hollow triangle) couples only to a contract.
- **Sequence diagrams** show one scenario over time: lifelines, activations, synchronous calls (filled arrowhead), replies (dashed), asynchronous messages (open arrowhead), and `alt` / `opt` / `loop` / `par` fragments. Always draw one unhappy path.
- **Component diagrams** show replaceable parts and their provided and required interfaces; **state machines** show legal events per state with `trigger [guard] / effect` and entry/exit actions; **activity diagrams** show workflows with decisions, forks, joins and swimlanes. Mermaid approximates component and activity notation with flowcharts.
- **Composition** means exclusive ownership and shared lifetime; **aggregation** is a shared whole–part link of debated value; **association** is a lasting peer link. Value types are attributes; entities are relationships; links with data become association classes.
- **LLD interview method:** clarify → use cases → entities → class diagram → interfaces → sequence or state diagram → patterns → concurrency and extensibility → code, in about 45 minutes with code starting by minute 30.
- **Exercise lessons:** put variation behind interfaces (allocation, pricing, dispatch, rate-limit algorithm); keep check-then-act inside one critical section; separate title from copy, pseudo-legal from legal, car movement from dispatch; use the State pattern when behavior differs per state; LRU is a hash map plus a doubly linked list with sentinels; TTL needs lazy plus active expiry; transactions need private write buffers and atomic, validated commits.

38.12 Knowledge checkpoint

1. What do `#` and `~` mean in front of a class member, and how is a static member shown?
2. In `Order "1" --> "1..*" LineItem`, how many line items can an order have, and how many orders can a line item belong to?

3. Give the line and end symbol for realization and for dependency.
4. What three tests distinguish composition from aggregation?
5. When should a relationship between two classes become a class of its own?
6. In a sequence diagram, how do synchronous and asynchronous messages differ in notation and meaning?
7. What is the difference between `alt` and `opt`?
8. In a state machine, what happens when an event arrives and the guard is false?
9. Why does a join after a decision node deadlock an activity?
10. In the LLD method, by what minute should you normally be coding in a 45-minute interview, and why?
11. Why does the LRU cache need a *doubly* linked list, and why must `get` take an exclusive lock?
12. Using Formula 38.2 with $t_{\text{floor}} = 2$ s and $t_{\text{stop}} = 10$ s, an idle car is 9 floors away and another car moving toward the caller in the right direction is 3 floors away with 2 stops in between. Which is dispatched?
13. Why is `Book` separate from `BookCopy` in the library design?
14. Why is the vending machine a better fit for the State pattern than for a transition table?
15. What are pseudo-legal moves, and why validate legality on a copy of the board?

Answers

1. `#` is protected (class and subclasses), `~` is package visibility. Static members are underlined (Mermaid: a trailing `$`).
2. One or more line items per order; exactly one order per line item. Multiplicity is written at the far end and read from the near class.
3. Realization: dashed line with a hollow triangle at the interface. Dependency: dashed line with an open arrowhead at the used class.
4. Lifetime (does the part die with the whole?), sharing (can the part belong to two wholes?), creation (does only the whole create it and keep it private?). Yes, no, yes means composition.
5. When the link has data of its own, such as a due date or a grade; it becomes an association class (`Loan`, `Enrollment`).
6. Synchronous: solid line, filled arrowhead; the caller waits for the reply (dashed line). Asynchronous: solid line, open arrowhead; the caller continues without waiting.
7. `alt` has two or more guarded operands, exactly one of which runs (if/else); `opt` has one operand that runs or not (if without else).
8. The transition does not fire; the object stays in its current state, and no exit, effect or entry behavior runs.
9. Only one branch receives a token after a decision, but a join waits for tokens on all incoming edges, so it never fires. Use a merge instead.

10. Around minute 30; the last 13–15 minutes are needed to write and exercise working code, which is part of the evaluation.
11. Unlinking a node in $O(1)$ needs its predecessor, which only a doubly linked node stores. `get` moves the node to the front, which mutates the list, so concurrent `get`s would race.
12. Idle car: $9 \times 2 = 18$ s. Moving car: $3 \times 2 + 2 \times 10 = 26$ s. The idle car (18 s) is dispatched.
13. A title has many physical copies with different statuses; loans attach to copies, holds attach to titles. Merging them cannot represent either correctly.
14. Its states differ substantially in behavior (accepting coins, computing change, ignoring input while dispensing); one class per state keeps each set of rules cohesive and makes adding a state a local change.
15. Moves that follow a piece's movement rules without considering whether the own king is left in check. Applying the move to a copy and testing for check handles pins, discovered checks and king moves uniformly without undo logic.

38.13 Interview questions

Basic

Q1. Explain the difference between aggregation and composition, with an example in code. Answer. Both are whole–part associations. In composition the whole exclusively owns its parts and controls their lifetime: an `Order` creates its `LineItem`s, never shares them, and deleting the order deletes them (filled diamond; `ON DELETE CASCADE`). In aggregation the parts exist independently and may be shared: a `Playlist` references `Song`s that exist in the catalog and in other playlists (hollow diamond). In code, composition shows as the whole constructing the parts privately and exposing values or copies; aggregation as references passed in from outside. Many practitioners model aggregation as a plain association because its semantics are loosely defined.

Q2. Which diagrams would you draw to explain a checkout flow to a new team member? Answer. A sequence diagram of the purchase scenario, including the payment-declined `alt` branch and the retry loop with an idempotency key; a state machine for the order lifecycle so they know which transitions are legal; and, if they are joining one module of several, a component diagram showing which interfaces their module provides and requires. A class diagram of the order aggregate helps if they will change the domain model.

Intermediate

Q3. Design a parking lot. How do you keep two entry gates from assigning the same spot? Answer. Follow §38.10.1: a `ParkingLot` facade composing levels and spots, `Size` enum for vehicles and spots, `AllocationStrategy` and `FeePolicy` interfaces, im-

mutable `Ticket`s. Finding a spot and marking it occupied must be one critical section; otherwise both gates can see the same spot as free (check-then-act). Use one lock for the lot, or a lock per level with the strategy claiming atomically within that level. In a database-backed design, claim with a conditional update (`WHERE occupied = false`) and check the row count.

Q4. Implement an LRU cache with O(1) operations. How would you scale it to many threads? Answer. Hash map from key to node plus a doubly linked list in recency order with head and tail sentinels; `get` moves the node to the front; `put` inserts at the front and evicts the node before the tail when full. A single lock is correct because `get` also mutates. To scale, stripe the cache into k independent segments by key hash (approximate global LRU), or use buffered access recording and batched reordering as high-performance libraries do; keep all of them behind one `Cache` interface.

Advanced

Q5. Design an elevator system for a 40-floor building with 6 cars. How do you choose which car answers a hall call, and how do you test it? Answer. Separate the dispatcher (a Strategy) from each car's state machine (Idle, Moving, DoorsOpen, Maintenance with guarded transitions; the doors can never be open while moving). Dispatch by minimum estimated arrival time (Formula 38.2), not nearest car, because stops cost more than travel; add an age term against starvation and a zoning strategy for the morning peak. Make the controller deterministic with a discrete tick and injected time, so the worked cost example and full sweeps (LOOK ordering) become unit tests; simulate traffic to compare average and p95 waiting time between strategies.

Q6. Design an in-memory key-value store with TTL and transactions. What isolation do you provide? Answer. Entries with an optional TTL; lazy expiry on read plus an active sweep driven by a min-heap with a per-run budget. Per-client sessions hold a stack of write buffers for nested transactions with read-your-writes; outermost commit validates the read-set versions and applies all writes atomically under the store lock (optimistic concurrency), raising a conflict otherwise. Committed transactions are serializable in commit order; reads inside a transaction are not from a snapshot, so add MVCC if transactions must observe consistent data before commit. Discuss durability (append-only log), memory limits (eviction), and why Redis `MULTI`/`EXEC` has no rollback but `WATCH` provides the same optimistic check.

38.14 Practical exercises

1. Draw the class diagram of a codebase you know (8–12 classes) and mark every relationship as dependency, association, aggregation, composition, generalization or realization. Find one inheritance that should be composition.

2. Add a `Maintenance` state to the vending machine that only an operator key can enter and leave; confirm that no existing state class changes except the transitions into it.
3. Add direction-aware hall stops to the elevator code (`(floor, direction)` pairs) and write a test where a car moving up passes a floor with a pending *down* call without stopping.
4. Replace the vending machine's greedy change-making with a dynamic-programming version that respects coin counts, and test the {one 25, three 10s} → 30 cents case.
5. Extend the chess code with castling: add castling rights to the board state and verify that castling through an attacked square is rejected.

38.15 Design challenges

1. **Food-delivery order lifecycle.** Design the classes, state machine and two sequence diagrams for an order that moves through placement, restaurant acceptance, courier assignment, pickup, delivery and refunds, with partial cancellations of individual items. Show where the state is persisted and how concurrent events (customer cancels while courier picks up) are resolved.
2. **Meeting-room booking system.** Rooms, recurring bookings, conflict detection, waitlists and calendar notifications for 2,000 rooms across 30 offices. Deliver the class diagram, the conflict-detection algorithm and its complexity, the concurrency strategy for two people booking the same slot, and three extension scenarios with their blast radius.

38.16 References

1. Object Management Group. *OMG Unified Modeling Language (OMG UML), Version 2.5.1* (formal/17-12-05). OMG, 2017. https://www.omg.org/spec/UML/2.5.1/
2. Fowler, M. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Addison-Wesley, 2003.
3. Rumbaugh, J., Jacobson, I., Booch, G. *The Unified Modeling Language Reference Manual*, 2nd ed. Addison-Wesley, 2004.
4. Gamma, E., Helm, R., Johnson, R., Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
5. Martin, R. C. *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall, 2002.
6. Martin, R. C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
7. GitHub. *Include diagrams in your Markdown files with Mermaid*. The GitHub Blog, February 2022. https://github.blog/2022-02-14-include-diagrams-markdown-files-mermaid/

8. Mermaid contributors. *Mermaid documentation* (class, sequence, state diagram and flowchart syntax), accessed 2026. https://mermaid.js.org/
9. Python Software Foundation. *functools — Higher-order functions and operations on callable objects* (documentation and `Lib/functools.py` source), accessed 2026. https://docs.python.org/3/library/functools.html
10. Oracle. *Class LinkedHashMap<K,V>*. Java SE API documentation, accessed 2026. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/LinkedHashMap.html
11. Redis. *EXPIRE* (command documentation, including how keys expire), accessed 2026. https://redis.io/docs/latest/commands/expire/
12. Redis. *Transactions* (MULTI, EXEC, DISCARD and WATCH), accessed 2026. https://redis.io/docs/latest/develop/interact/transactions/
13. Fiekas, N. and contributors. *python-chess documentation*, accessed 2026. https://python-chess.readthedocs.io/
14. IFLA Study Group on the Functional Requirements for Bibliographic Records. *Functional Requirements for Bibliographic Records: Final Report*. K. G. Saur, 1998.

38.17 Open verification items

- Exact wording and clause number in UML 2.5.1 stating that shared-aggregation semantics vary by application area and modeler; and the 790-page length of the specification PDF (approximate).
- Fowler’s quotation of Rumbaugh’s “modeling placebo” remark on aggregation (*UML Distilled*, 3rd ed.), page reference not checked.
- GitHub Mermaid announcement: exact publication date (believed 14 February 2022) and URL.
- Current implementation details of CPython’s `lru_cache` (C accelerator versus pure-Python fallback) and the per-entry memory overhead estimate of 100–200 bytes.
- Redis expiry and transaction behavior as documented in 2026.

Appendix E — Learning Roadmap and Proficiency Levels

> **Deliverable 3.** This appendix turns the 93 chapters into a schedule. It defines five proficiency levels, what you must know and be able to do at each, and week-by-week milestones for three tracks: the full 12-month program, a 16-week interview track, and a 24-week ML/AI infrastructure track for working engineers.

E.1 How to use this roadmap

1. Find your starting level with the self-assessment in §E.3. Do not skip a level because a topic "sounds familiar" — the readiness criteria test whether you can *explain and apply* a concept, not whether you recognize its name.
2. Pick a track (§E.4–E.6). Each week lists chapters, a hands-on task and a checkpoint. Budget the stated hours; if a week overruns, extend it rather than skipping the exercise — implementation is where understanding consolidates (Ch 3, §3.7).
3. At the end of every month, run the month's **milestone review**: answer the knowledge checkpoints of each chapter from memory, redraw the month's key architecture diagrams without notes, and solve one design challenge aloud in 45 minutes.
4. Record progress in a design journal (template in Ch 3, §3.10).

E.2 The five proficiency levels

Figure E.1 — Proficiency ladder.

WARNING / TRADE-OFF

Diagram failed to render. Source:

```
flowchart LR
  L1["Level 1<br/>Absolute beginner"] --> L2["Level 2<br/>Intermediate engineer"]
  L2 --> L3["Level 3<br/>Advanced engineer"]
  L3 --> L4["Level 4<br/>Senior / Staff"]
  L4 --> L5["Level 5<br/>Expert / Principal"]
```

Reading the diagram. The levels are cumulative: each assumes every competency of the previous one. The arrows are not time-boxed — a strong backend engineer may al-

ready sit at Level 3 for traditional systems but at Level 1 for ML infrastructure. Assess each track (A: traditional, B: ML/AI) separately.

Level 1 — Absolute beginner

Dimension	Definition
Required knowledge	Ch 1–8, 11, 35. What a client, server, API, database, process, thread, IP address, DNS lookup and HTTP request are; functional vs non-functional requirements; powers of ten and basic percentiles.
Expected competencies	Trace one HTTP request from browser to database and back, naming every hop. Write a CRUD REST API with a relational database. Model a simple domain in tables with primary/foreign keys. Do a back-of-the-envelope estimate of requests per second from daily active users.
Practical projects	P-01 REST API service; P-02 URL shortener (single node).
Architecture exercises	Draw a single-server web app; then add a load balancer and a second app server and explain what breaks (sessions).
Interview readiness	Not yet ready for system design rounds. Can answer “what happens when you type a URL?” thoroughly.
Explain independently	TCP vs UDP; what an index is and why it speeds reads but slows writes; what latency vs throughput mean; why HTTPS needs certificates.

Level 2 — Intermediate engineer

Dimension	Definition
Required knowledge	Ch 9–10, 12, 14, 17–22, 24, 30, 33, 36–37, 57. Replication, sharding, consistent hashing, caching patterns, CDNs, load-balancing algorithms, rate limiting, queues and pub/sub, monolith vs microservices, ML basics.
Expected competencies	Turn a vague prompt into numbered FRs/NFRs with targets. Estimate storage, bandwidth and RPS with peak factors. Choose SQL vs NoSQL with justification. Add a cache and explain invalidation and stampede protection. Design a rate limiter and pick an algorithm.
Practical projects	P-03 Redis cache; P-04 rate limiter; P-05 load-balancer simulation; P-06 message queue.
Architecture exercises	URL shortener, Pastebin, rate limiter and notification system (Ch 52) at 10^7 DAU.
Interview readiness	Can complete an entry-level (new-grad / mid-level) system design round: correct high-level architecture, reasonable estimates, identifies the main bottleneck.
Explain independently	Cache-aside vs write-through; why sharding needs a good key; at-least-once delivery and why consumers must be idempotent; Little’s Law applied to a thread pool.

Level 3 — Advanced engineer

Dimension	Definition
Required knowledge	Ch 13, 15–16, 23, 25–27, 29, 31–32, 34, 38–43, 58–62, 66. Storage engines, consistency models, Raft, quorums, Kafka internals, outbox/saga/CQRS, observability, Kubernetes, CI/CD, ML lifecycle, feature stores, model serving.
Expected competencies	Design a complete distributed application with explicit consistency choices, failure handling and observability. Explain a database's write path down to WAL and fsync. Design an ML system from problem formulation to online serving with a feature store.
Practical projects	P-07 distributed KV store; P-08 replicated DB simulation; P-09 notification service; P-10 real-time chat; P-11 distributed job scheduler.
Architecture exercises	All of Ch 52–53; ML case studies 81.5 (product recommendation) and 82.3 (spam detection).
Interview readiness	Passes a senior-level (L5/E5 equivalent) traditional round and a mid-level ML system design round: deep-dives on request, quantifies trade-offs, discusses failure modes unprompted.
Explain independently	Raft leader election and log matching; why `R + W > N` gives read-your-writes only under certain conditions; Kafka ISR and `acks=all`; point-in-time feature joins and training–serving skew.

Level 4 — Senior / Staff engineer

Dimension	Definition
Required knowledge	Ch 28, 44, 46–48, 50–51, 63–65, 67–80. Distributed transactions, multi-region design, cluster scheduling, multi-tenancy, security architecture, cost; GPU performance, distributed training, model optimization, recommendation architecture, MLOps, LLM inference, RAG, agents, AI platforms.
Expected competencies	Make and defend architecture decisions spanning teams: multi-region active–active, tenant isolation, SLOs and error budgets, cost models. Size a GPU fleet for training and for LLM serving from first principles. Design a RAG or agent platform with evaluation, guardrails and observability.
Practical projects	P-12 event-driven microservices; P-13 real-time analytics pipeline; P-14 recommendation engine; P-15 distributed ML inference; P-18 production RAG; P-19 scalable LLM inference.
Architecture exercises	All of Ch 54–56 and 81–86. Write a design document with ADRs for one case study and have it reviewed.
Interview readiness	Passes staff-level rounds: drives the conversation, scopes ambiguity, prioritizes the hardest sub-problem, connects technical choices to business, cost and organizational constraints.
Explain independently	Spanner commit-wait; why cells limit blast radius; ZeRO stage 3 memory math; continuous batching and PagedAttention; when RAG beats fine-tuning; how to evaluate an agent.

Level 5 — Expert / Principal architect

Dimension	Definition
Required knowledge	Ch 45, 49, 80 plus the primary papers cited across Parts V, IX, XIII and XVI. Theory (FLP, CAP proofs, BFT bounds, consistency hierarchies, scaling laws) and practice at 10^4–10^5 machine scale.
Expected competencies	Set multi-year technical direction; evaluate novel systems from papers; reason about organization-wide failure domains, reliability economics and sustainability; design platforms other teams build on.
Practical projects	P-16 distributed training platform; P-17 multi-tenant feature store; P-20 multi-agent orchestration platform.
Architecture exercises	Critique a published architecture (e.g., a Dynamo, Spanner or PagedAttention paper) and propose a design for a changed constraint. Lead a postmortem-driven redesign.
Interview readiness	Principal-level: articulates long-term trade-offs, migration strategy, organizational design (Conway's law), and the economics of the platform.
Explain independently	Why consensus is impossible in a fully asynchronous system with one crash failure and how real systems sidestep it; goodput at 16k-GPU scale; the cost/latency frontier of an LLM product.

E.3 Self-assessment

Score yourself 0–2 on each statement (0 = cannot, 1 = with notes, 2 = from memory, aloud, in under five minutes). Start at the first level where your total is below 14 of 20.

Table E.1 — Placement self-assessment.

Level	Ten statements to score
1	Explain DNS resolution · TCP handshake · HTTP request/response · process vs thread · what an index is · primary/foreign keys · REST verbs and status codes · latency vs throughput · p99 · RPS from DAU
2	Cache-aside and invalidation · consistent hashing · token bucket · L4 vs L7 load balancing · replication lag · sharding keys · at-least-once delivery · CDN pull vs push · SQL vs NoSQL choice · precision vs recall
3	Raft election · quorum math · Kafka partitions/ISR · outbox pattern · LSM vs B-tree · Kubernetes control plane · SLO/error budget · feature store PIT join · dynamic batching · canary analysis
4	2PC failure modes · multi-region active–active · tenant isolation options · cost model of a service · data/tensor/pipeline parallelism · KV-cache memory · RAG evaluation · agent guardrails · model routing · DRF scheduling
5	FLP intuition · linearizability vs serializability · PBFT bound · TrueTime · ZeRO-3 communication · MFU at scale · scaling laws · disaggregated serving · failure rates at 10k GPUs · multi-year platform migration

E.4 Track 1 — Full 12-month program (beginner to Level 4)

Budget: 10–12 hours per week · 52 weeks · Levels 1 → 4. Each week: read (≈ 60%), implement or exercise (≈ 30%), recall and explain (≈ 10%).

Figure E.2 — Twelve-month program at a glance.

WARNING / TRADE-OFF

Diagram failed to render. Source:

```
gantt
  title Twelve-month program
  dateFormat  YYYY-MM-DD
  axisFormat  M%m
  section Level 1
  Thinking, maths, CS foundations      :a1, 2027-01-04, 8w
  Web, data basics, OOD                :a2, after a1, 5w
  section Level 2
  Data systems, scale, caching         :b1, after a2, 7w
  Distributed basics, messaging        :b2, after b1, 5w
  section Level 3
  Distributed depth, Kafka, EDA        :c1, after b2, 5w
  Production infra, case studies I-II  :c2, after c1, 6w
  section Level 3-4 ML
  ML foundations, data, GPUs, serving  :d1, after c2, 8w
  section Level 4
  LLMs, RAG, agents, AI infra          :e1, after d1, 5w
  Expert topics, case studies, interviews :e2, after e1, 3w
```

Reading the diagram. Bars are sequential phases; the dates are an example start (any Monday works). Levels overlap at the boundaries because review weeks consolidate the previous level while starting the next. The ML phase begins only after production infrastructure because serving, MLOps and AI-infrastructure chapters assume Kubernetes, observability and deployment knowledge.

Table E.2 — Weekly milestones, 12-month program.

Wk	Chapters	Hands-on task	Checkpoint
1	1, 3	Set up design journal and flashcard deck	Explain FR vs NFR with three examples each
2	2	Write requirements + context diagram for a food-ordering app	Present it in 10 minutes
3	4 (§4.3–4.9)	Estimation drills: 20 powers-of-ten problems	Score ≥ 18/20
4	4 (§4.10–4.18)	Simulate an M/M/1 queue in Python; compare with formula	Explain why latency explodes near 100% utilization
5	5	Cache-stride benchmark	Draw the memory hierarchy with latencies from memory
6	6	Race condition + mutex fix; event-loop echo server	Explain epoll vs thread-per-connection
7	7	`dig`, `curl -w`, Wireshark on a TLS handshake	Narrate “type a URL” end to end
8	Review	Redo checkpoints Ch 1–7	**Month 2 milestone review**
9	8	P-01 REST API with pagination and JWT	Defend cursor vs offset pagination
10	11	Schema design + `EXPLAIN` on indexed vs unindexed queries	Explain B+ tree lookups in I/Os
11	35, 36	Refactor a small codebase with SOLID	Class diagram from memory
12	9, 10	SSE + WebSocket demo; password hashing	Choose a real-time technique for 3 scenarios
13	Review	P-02 URL shortener (single node)	**Level 1 exit**: self-assessment ≥ 14
14	12	Model one domain in a document DB and a wide-column DB	SQL vs NoSQL decision for 5 prompts
15	13	Toy LSM tree (memtable + SSTables + compaction)	Draw write path with WAL
16	14	Consistent-hashing ring with virtual nodes	Keys moved when adding a node
17	15, 16	Inverted index + BM25 in Python	Explain erasure coding overhead
18	17	Bloom filter, HyperLogLog, Snowflake IDs	False-positive-rate calculation

Wk	Chapters	Hands-on task	Checkpoint
19	18, 19	P-05 load-balancer simulation	Power-of-two-choices intuition
20	20, 21	P-03 Redis cache-aside with stampede protection	Five caching patterns from memory
21	22	P-04 distributed rate limiter	Token bucket parameters for a given SLA
22	23	Load test with `wrk` / `k6`; find the bottleneck	Explain coordinated omission
23	24	Lamport and vector clocks implementation	Two partial-failure stories
24	30	P-06 message queue with acks and DLQ	Delivery semantics table from memory
25	33, 37	Implement 6 design patterns	Monolith vs microservices debate
26	Review	Ch 52.1–52.4 case studies	**Level 2 exit + Month 6 milestone**
27	25	Jepsen-style anomaly hunt on a toy store	Consistency hierarchy diagram
28	26	Raft (election + log replication) — MIT 6.5840 style	Explain fencing tokens
29	27, 28	P-07 distributed KV store with quorums	2PC failure walkthrough
30	29, 31	Kafka locally: partitions, consumer groups, lag	Partition count sizing
31	32, 34	Outbox + saga demo	Choreography vs orchestration
32	38	Four LLD problems (parking lot, elevator, LRU, vending)	45-min LLD mock
33	39, 40	SLOs, burn-rate alerts, OpenTelemetry tracing	Error-budget policy
34	41, 42	Deploy P-01 to Kubernetes with HPA	Kubernetes control-plane walk-through
35	43	CI/CD with canary; Terraform a VPC	Rollback plan
36	52, 53	P-09 notification service; P-10 chat	Two 45-min mocks
37	54, 55	YouTube, Uber, payment system cases	Two 45-min mocks
38	Review	56 + P-11 job scheduler	**Level 3 exit (Track A)**

Wk	Chapters	Hands-on task	Checkpoint
39	57	Train/evaluate a classifier; compute all metrics	Precision/recall trade-off for fraud
40	58, 59	Small MLP in PyTorch; memory math	ML lifecycle diagram
41	60, 61	Batch + streaming feature pipeline	Lambda vs Kappa
42	62, 63	Feast feature store with PIT join	Training–serving skew causes
43	64	Roofline analysis of a matmul	Compute- vs memory-bound
44	65	DDP on 2 GPUs (or simulated); FLOPs estimate	6ND estimate for a model
45	66, 67	Serve a model with dynamic batching; quantize it	Batch size vs latency curve
46	68–71	P-14 two-tower recommender; drift monitor	Multi-stage funnel sizing
47	72	Implement attention + KV cache in NumPy	KV-cache size for a 70B model
48	73	Run vLLM; measure TTFT/TPOT vs concurrency	Continuous batching explanation
49	74, 75	P-18 production RAG with hybrid search + reranking	Retrieval metrics
50	76–78	Agent with tool calling, guardrails and eval set	Prompt-injection defenses
51	79, 80, 81–86 (2 cases)	Model gateway with token-based rate limits	Two ML/AI mocks
52	87–92	Full mock loop: 2 traditional + 2 ML/AI	**Level 4 readiness review**

Level 5 content (Ch 45, 49, 80 deep dives, primary-paper reading) is a continuing program after week 52: one paper and one design critique per week.

E.5 Track 2 — 16-week interview track (Level 2–3 engineer → senior interviews)

Budget: 12–15 hours/week. Assumes Level 2 fundamentals.

Table E.3 — Interview track.

Wk	Focus	Chapters	Practice
1	Framework and estimation	2, 4, 87, 88, 89	30 estimation drills
2	Data systems refresh	11, 12, 14	Q-T-01 … Q-T-04
3	Caching, CDN, LB	19, 20, 21	Q-T-05 … Q-T-08
4	Traffic management	22, 17	Rate limiter + URL shortener mocks
5	Distributed core	24, 25, 26	KV-store mock
6	Replication and transactions	27, 28	Payment-system mock
7	Messaging	30, 31, 32	Notification-system mock
8	Microservices and reliability	34, 39, 40	Mid-point mock with a peer
9	Social systems	53	News feed, chat, Twitter mocks
10	Media and files	54	YouTube, Dropbox mocks
11	Location and commerce	55	Uber, ticketing mocks
12	Data platforms	56, 16	Search, logging mocks
13	Senior depth	44, 47	Multi-region mock
14	Company preparation	90	Company-specific mocks
15	Practice library sweep	91	10 timed questions
16	Final review	Summaries of all above	3 full mocks; rubric self-score ≥ “Hire”

E.6 Track 3 — 24-week ML/AI infrastructure track (software engineer → ML/AI systems)

Budget: 10–12 hours/week. Assumes Level 3 on Track A.

Table E.4 — ML/AI track.

Wk	Chapters	Deliverable
1–2	4.9, 57	Metrics notebook; A/B test sample-size calculator
3	58	PyTorch MLP; parameter/activation memory worksheet
4	59, 60	Architecture diagram of an end-to-end ML system
5–6	61, 29	Lakehouse table + streaming feature job
7	62, 63	Feature store with PIT-correct training set
8–9	64	GPU profiling exercise; roofline worksheet
10–11	65	Multi-GPU training run; parallelism plan for a 70B model
12	66	Model server with dynamic batching and autoscaling
13	67	Quantization + ONNX/TensorRT comparison
14	68	Two-tower retrieval + ranker
15	69, 70	CI/CD/CT pipeline with drift-triggered retraining
16	71	Fairness audit and model card
17	72	Transformer from scratch (small)
18–19	73	LLM serving benchmark; capacity plan
20	74, 75	Production RAG with evaluation harness
21	76, 77	LoRA fine-tune; tool-using agent
22	78, 79	Eval pipeline; model gateway with routing and fallback
23	80, 81–85	Two case studies written as design docs
24	86, 92	Four ML/AI mock interviews

E.7 Monthly milestone review template

1. **Recall:** answer every knowledge-checkpoint question for the month's chapters without notes; mark misses for spaced repetition.

2. **Redraw:** reproduce the month’s three most important architecture diagrams from memory; compare with the book.
3. **Quantify:** solve three estimation problems from Ch 89 in under five minutes each.
4. **Design:** solve one design challenge aloud in 45 minutes; self-score with the Ch 88 rubric.
5. **Build:** demo the month’s project to someone else, including one injected failure.
6. **Reflect:** write half a page in the design journal — what is still unclear, which open question to research next.

Appendix A — Glossary

> Generated by `build/build.py` from the definition callouts in every chapter; the first (defining) occurrence of each term is used. Canonical usage rules are in `docs/project/terminology.md`.

A

Abstract factory — An interface for creating a *family* of related objects (for example a blob store and a work queue) without naming their concrete classes, so that a client receives products that are guaranteed to belong together [Gamma et al. 1994]. *(§37.4)*

Abstraction — Representing something by the operations that matter to its users while leaving out the details of how they are carried out. In code, an **abstract class** is a class that cannot be instantiated and may contain both implemented methods and **abstract methods** (declared, with no body) that subclasses must implement. An **interface** declares a set of methods a type promises to provide; in Java it has no instance state, and since Java 8 it may include `default` method bodies built only on its other methods. *(§35.7.1)*

Access-control model — A rule system that answers "may subject S perform action A on resource R (in context C)?" The three models in common use differ in what the rules are written in terms of: **roles** (RBAC), **attributes** (ABAC) or **relationships** (ReBAC). *(§8.12.1)*

ACID — The four guarantees of a database transaction, named by Härder and Reuter [Härder and Reuter 1983]: **Atomicity** — all of a transaction's writes take effect, or none do. **Consistency** — a transaction takes the database from one state satisfying all declared invariants (constraints) to another. **Isolation** — concurrently executing transactions do not see each other's partial effects; at the strongest level, the outcome equals some serial execution. **Durability** — once a commit is acknowledged, its effects survive crashes. *(§11.12)*

Active–passive and active–active — In an **active–passive** pair, one balancer carries all traffic while a standby monitors it and takes over the VIP on failure. In **active–active**, all balancers carry traffic at once (spread by DNS, ECMP or anycast), and survivors absorb a failed member's share. *(§19.7)*

Active recall — Deliberately retrieving information from memory without looking at the source, for example by answering a question, drawing a diagram or explaining aloud. Its

measured benefit for later retention is called the **testing effect**. *(§3.4.2)*

Activity diagram — A behavior diagram that models a workflow as **actions** connected by **control flows**, with **decision** and **merge** nodes for branching, **fork** and **join** bars for concurrency, and **activity partitions** (swimlanes) that assign actions to the participant responsible for them. Its semantics are based on tokens flowing through the graph, similar to a Petri net [OMG 2017]. *(§38.7.1)*

Adapter — A wrapper that converts the interface of an existing class into the interface its clients expect, so that classes with incompatible interfaces can work together [Gamma et al. 1994]. *(§37.9)*

Address space — The range of memory addresses a process can use. On Linux x86-64 with 4-level page tables, user space spans 2^{47} bytes = 128 TiB of *virtual* addresses; the kernel occupies the upper half and is not accessible from user mode. *(§6.3.2)*

Aggregate — A cluster of entities and value objects treated as one unit for data changes, with one **root** entity. Outside objects reference only the root, every change goes through it, and it enforces the aggregate's **invariants**. One transaction modifies one aggregate [Evans 2003; Vernon 2013]. *(§34.4.3)*

Analysis — The process that converts raw text into the terms stored in (or looked up from) an inverted index. An **analyzer** is a pipeline of zero or more **character filters** (rewrite the raw character stream), exactly one **tokenizer** (split the stream into tokens with positions and offsets) and zero or more **token filters** (modify, remove or add tokens). *(§16.4.1)*

Anticorruption layer (ACL) — A layer owned by a downstream context that translates between an upstream model and its own, so the upstream's concepts, codes and quirks never appear in the downstream model [Evans 2003]. *(§34.5.3)*

Anycast — An addressing scheme in which the same IP prefix is announced via BGP from many locations at once. Each client's packets are routed to whichever announcing location is "closest" according to BGP path selection. *(§7.5.3)*

Apache Flink — An open-source distributed engine that executes dataflow programs as a graph of long-running, pipelined operators, treating batch as a bounded special case of streaming. It provides event-time processing with watermarks, large keyed state, and exactly-once state consistency through asynchronous distributed snapshots [Carbone et al. 2015a]. *(§29.9.1)*

API gateway — A reverse proxy (§7.12) that is the single entry point for external clients into a set of services. It routes requests and applies **edge concerns** once: TLS termination, authentication, rate limiting, request validation, caching, protocol translation and sometimes aggregation [Richardson 2018]. *(§34.6.1)*

Architectural decision — A design choice that addresses an architecturally significant requirement — one that affects structure, quality attributes, interfaces or technology in a way that is costly to change later. Examples: choosing a relational database over a key-value store; storing files in object storage; making photo processing asynchronous. *(§1.8.1)*

Architecture Decision Record (ADR) — A short document that captures one architectural decision: its context, the decision itself, its status, and its consequences. ADRs are numbered, kept in sequence (often in the code repository), and never edited to reverse a decision — a new ADR *supersedes* the old one [Nygard 2011]. *(§1.8.1)*

Architecture documentation — The written and diagrammed record of a system's structure and of the decisions that shaped it. It is written so that people who were not in the room (new team members, reviewers, your future self) can understand *what* the system is and *why* it is that way. *(§2.12.1)*

Association, aggregation, composition — An **association** is a structural relationship in which instances of one class hold links to instances of another. **Aggregation** (UML "shared aggregation") is an association marked as whole–part, in which a part may be shared by several wholes and exists independently of them. **Composition** (UML "composite aggregation") is a whole–part association in which each part belongs to at most one whole at a time, and the whole is responsible for the part's existence: if the composite is deleted, its parts are deleted with it [OMG 2017]. *(§38.8.1)*

Asymmetric (public-key) cryptography — Cryptography using a mathematically linked **key pair**: a **public key** that can be shared with anyone and a **private key** kept secret by its owner. What one key does, only the other can undo or verify. Security rests on problems believed hard to invert: factoring large integers (RSA) or computing discrete logarithms on elliptic curves (ECC). *(§10.4.3)*

Asynchronous (message-based) communication — An interaction in which the sender hands a **message** to an intermediary — a **message broker** — and continues without waiting for the receiver to process it. The broker stores the message until a receiver takes it, so sender and receiver need not be available at the same time. *(§30.3.1)*

Atomic commitment — A protocol by which a set of **participants** (nodes that each executed part of a transaction) reach a single outcome, commit or abort, such that: (1) all participants that decide reach the same decision; (2) a participant cannot reverse a decision once made; (3) commit is decided only if every participant voted yes; (4) if there are no failures and every participant voted yes, the decision is commit; and (5) once failures are repaired and no new ones occur for long enough, every participant eventually decides [Bernstein et al. 1987]. *(§28.3.1)*

Audit log — An append-only record of security-relevant events — who (actor), did what (action), to which resource, when, from where, and with what result — retained so that

actions can be investigated and attributed. A log is **tamper-evident** if any modification, deletion or reordering of past entries can be detected. *(§10.12)*

Authenticated encryption with associated data (AEAD) — A symmetric scheme that provides confidentiality for the plaintext *and* integrity for both the ciphertext and optional unencrypted **associated data** (AAD), producing an authentication **tag** that decryption checks before releasing any plaintext. AES-GCM and ChaCha20-Poly1305 are AEAD schemes. *(§10.4.2)*

Authentication (AuthN) — Verifying *who* a client is. **Authorization (AuthZ)** is deciding *what* that verified identity may do (§8.12). AuthN failures return `401`; AuthZ failures return `403`. *(§8.11)*

Autocomplete (typeahead) — A service that, given the prefix a user has typed so far, returns a short ranked list (typically 5–10) of likely completions, usually popular past queries or entity names, on every keystroke or after a short pause. *(§16.9.1)*

Autoscaling — Automatically changing the number of instances of a service in response to a measured or forecast signal, within configured minimum and maximum bounds. **Reactive** autoscaling responds to current metrics; **scheduled** autoscaling follows a calendar; **predictive** autoscaling acts on a forecast. *(§18.8.2)*

Availability — The fraction of time (or of requests) for which a system serves correctly: A = uptime/(uptime + downtime), or, request-based, A = successful requests/valid requests. It is commonly quoted in “nines”: 99.9% is “three nines”. *(§4.14.1)*

B

Backend for frontend (BFF) — A server-side component dedicated to one kind of user interface (iOS, web, partner API), owned by or aligned with that interface’s team, which aggregates and reshapes downstream calls into exactly the API that interface needs [Newman 2015]. *(§34.6.2)*

Back-of-the-envelope estimation — A quick calculation, done with round numbers and stated assumptions, that gives the order of magnitude of a system’s load and resource needs (requests per second, storage, bandwidth, memory, number of nodes). The aim is to be within a factor of about 2–3 of the truth with a few minutes of arithmetic. This section introduces the method. Chapter 89 is the full handbook. *(§2.11.1)*

Backpressure — A flow-control mechanism in which a slower downstream component signals upstream components to reduce their sending rate, by blocking, by advertising available capacity, or by rejecting, so that work in progress stays bounded instead of piling up in unbounded buffers. *(§22.10)*

Backward and forward compatibility — **Backward compatibility**: newer code can read data written by older code. **Forward compatibility**: older code can read data written by newer code [Kleppmann 2017, ch. 4]. *(§8.9.3)*

Batch processing — Running a computation over a **bounded** dataset (one whose size is known when the job starts, such as "yesterday's logs") that reads immutable input files, writes new output files, and is judged mainly by **throughput** (bytes or records processed per second) and total completion time rather than by the latency of individual records. *(§29.3.1)*

BGP — A **path-vector** routing protocol in which neighboring routers (peers) exchange, over a TCP session on port 179, announcements of the form "prefix P is reachable via AS path [X, Y, Z]". Each AS chooses among the paths it hears according to its own **policy**, then re-announces its chosen path (with its own ASN prepended) to the peers its policy allows. *(§7.5.2)*

Bit and byte — A **bit** (symbol b) is a single binary digit, 0 or 1. A **byte** (symbol B) is 8 bits. Storage and memory are conventionally measured in bytes; network link speeds are conventionally measured in bits per second (bps). *(§4.5.1)*

Block, file and object storage — **Block storage** exposes a numbered array of fixed-size blocks (typically 512 B or 4 KiB) that a client reads and writes by offset; the client's own file system or database gives the blocks meaning. **File storage** exposes a hierarchical namespace of directories and byte-addressable files with POSIX-like operations (open, read at offset, write at offset, rename, lock), shared by many clients over a protocol such as NFS or SMB. **Object storage** exposes a flat namespace of immutable **objects**, each a byte string plus metadata addressed by a key within a **bucket**, accessed over HTTP with whole-object PUT, GET (optionally a byte range), DELETE and LIST. *(§15.3.1)*

Bloom filter — A probabilistic data structure for set membership, introduced by Burton Bloom in 1970, that stores a set of n items in an array of m bits using k hash functions [Bloom 1970]. A query answers either "definitely not in the set" or "probably in the set". It never gives a **false negative** (saying "no" for an item that was added), and gives a **false positive** (saying "yes" for an item that was not added) with a probability p that depends on m, n and k. *(§17.4.1)*

Bottleneck — The resource whose capacity limits the throughput of the whole system for a given workload. Throughput cannot rise above the bottleneck's capacity no matter how much the other resources are improved. Formally, it is the resource with the highest **service demand** per request (defined below). *(§2.6.1)*

Bounded context — An explicit boundary (in code, data and team ownership) within which one domain model and its ubiquitous language apply consistently. The same word may mean different things in different contexts [Evans 2003]. *(§34.5.1)*

Branch prediction — Hardware that guesses the outcome of a conditional jump (a **branch**, e.g. an `if` or a loop condition) before it is computed, so that the pipeline can keep fetching instructions. A wrong guess is a **misprediction** and forces a pipeline flush. *(§5.3.4)*

Broker and controller — A **broker** is a Kafka server process that stores partition replicas and serves produce and fetch requests. The **controller** is the component that owns cluster **metadata** (which brokers are alive, which topics and partitions exist, which replica leads each partition, and the in-sync replica sets) and makes decisions such as leader election when a broker fails. *(§31.4.1)*

B-tree and B+ tree — A **B-tree** is a balanced search tree whose nodes are disk pages holding many sorted keys; every leaf is at the same depth, and nodes split and merge to stay between half-full and full [Bayer and McCreight 1972]. A **B+ tree** is the variant used by almost every relational database: internal nodes hold only separator keys and child pointers, all keys and values (or row pointers) live in the leaves, and leaves are linked to their siblings so range scans can walk sideways [Comer 1979; Petrov 2019]. *(§11.8.1)*

B-tree storage engine — A storage engine that keeps each table or index as a B+ tree of fixed-size pages and **updates pages in place**: a change to a key rewrites the page that holds it. Internal pages hold separator keys and child page numbers; leaf pages hold the keys with either the rows themselves (a clustered index) or pointers to them. *(§13.6.1)*

Bucket, key, object and flat namespace — A **bucket** is a named container with its own access policy, region and configuration. An **object** is an immutable byte string (from zero bytes to tens of terabytes) plus system metadata (size, content type, checksum, version ID) and user metadata. A **key** is the object's full name within the bucket, for example `photos/2026/10/u42/abc.jpg`. The namespace is **flat**: the slashes are ordinary characters, and "folders" are an illusion produced by listing keys that share a **prefix** and grouping them at a **delimiter** (usually `/`). *(§15.4.1)*

Buffer pool — A region of memory, managed by the database itself, divided into page-sized **frames** that cache disk pages. A **page table** maps page IDs to frames; each frame carries a **pin count** (how many threads are using it right now), a **dirty flag** (whether it differs from the disk copy) and bookkeeping for the **replacement policy** that chooses which page to evict. *(§13.4.1)*

Builder — A separate object that assembles a complex product step by step, validates the combination of choices, and returns a finished (ideally immutable) product from a final `build()` call [Gamma et al. 1994; Bloch 2018]. *(§37.5)*

Bulkhead — Partitioning a service's limited resources (threads, connections, memory, instances) into isolated pools, one per dependency, tenant or workload class, so that exhaustion of one pool cannot starve the others. *(§22.14)*

C

Cache — A store that holds copies of data whose authoritative version lives elsewhere (the **backing store** or **source of truth**), placed so that reading the copy is faster or cheaper than reading the original. A lookup that finds the data is a **hit**; one that does not is a **miss**. The **cache hit ratio** h is hits divided by lookups; the **miss ratio** is $1 - h$. *(§20.3.1)*

Cache-aside — The application reads the cache first; on a miss it reads the database and populates the cache itself. On a write it updates the database and then **deletes** (invalidates) the cache entry. The cache never talks to the database. Also called **look-aside** or **demand-filled** caching. *(§20.7.1)*

Cache avalanche — A large fraction of cache entries becomes unavailable at once, because many keys expire together or because cache nodes fail or restart, so the database suddenly receives the full read load. *(§20.13)*

Cache coherence — The guarantee that all cores observe a single, consistent sequence of values for each memory location, even though each core keeps private copies in its own caches. *(§5.4.6)*

Cache key — The value a cache uses to decide whether two requests may receive the same stored response. By default it is the method plus the full URL (scheme, host, path and query string), extended by any request headers listed in the response's `Vary` header. *(§21.6.2)*

Cache line — The fixed-size unit in which data moves between memory and caches. On x86-64 and most ARM server cores a line is 64 bytes; some designs (for example Apple's M-series cores) use 128 bytes (as of 2024). A load of a single byte brings its whole line into the cache. *(§5.4.3)*

Cache penetration — Requests for keys that do not exist in the database. Because there is nothing to cache, every such request misses and reaches the database. A bug or an attacker iterating random IDs can push the database's full read load through the cache. *(§20.12)*

Cache stampede (thundering herd, dog-piling) — When a popular entry expires or is evicted, many concurrent requests miss at once and all recompute or reload it, multiplying load on the backend exactly when it is busiest. *(§20.11)*

Cache tiers — The **edge** tier is the set of PoPs that accept user connections. A **mid-tier** (also *parent*, *regional* or *upper-tier* cache) is a smaller set of larger caches that edges consult on a miss. An **origin shield** is a mid-tier designated as the single path to a given origin. The **origin** is the customer's own infrastructure that holds the authoritative content. *(§21.4.1)*

Capacity planning — The recurring process of forecasting demand, translating it into the resources needed to serve it within the service-level objective with stated headroom and redundancy, and acquiring those resources before they are needed, allowing for the lead time to obtain them. *(§18.11.1)*

CAP theorem (Gilbert and Lynch) — In an asynchronous network model, where messages may be delayed arbitrarily and there are no clocks, it is impossible to implement a read/write data object that guarantees both of the following in all fair executions, including those in which messages are lost: **Consistency (C):** the object is **atomic**, that is, linearizable (§25.3). **Availability (A):** every request received by a non-failing node must eventually result in a response. **Partition tolerance (P)** is not a property the system chooses; it is the assumption that the network may lose arbitrarily many messages between nodes. *(§25.8.1)*

Causal consistency — Writes that are **causally related** must be seen by every client in their causal order; writes that are **concurrent** (neither happens before the other) may be seen in different orders by different clients. Formally, each client's view must be a legal sequential history that respects the causal order induced by program order and reads-from [Ahamad et al. 1995]. *(§25.5.1)*

Cell-based architecture — A style in which a workload is deployed as multiple independent, complete copies called **cells**, each with its own compute and data and each serving a fixed subset of customers (or other partition key), placed behind a thin **cell router** that sends each request to its customer's cell. Cells share nothing at run time, so a failure, bad deployment or poison request affects only one cell. The fraction of customers affected by a failure is its **blast radius** [AWS 2023]. *(§33.12.1)*

Central processing unit (CPU) — The hardware that executes a program's **instructions**: simple operations such as "add these two numbers", "load a value from memory address X", or "if this value is zero, jump to instruction Y". A modern CPU package contains several independent **cores**, each of which can execute its own stream of instructions. *(§5.3.1)*

Certificate Transparency (CT) — A system of public, append-only, Merkle-tree-based logs into which CAs record every certificate they issue, so that mis-issued certificates become publicly detectable [RFC 6962]. *(§10.5)*

Chain of Responsibility — A request is passed along a chain of handler objects; each handler either handles it, or does some work and passes it to the next, so the sender does not know which handler will act [Gamma et al. 1994]. *(§37.15.3)*

Chain replication — Replicas are arranged in a linear **chain**. Every update enters at the **head** and is forwarded node by node to the **tail**; the tail replies to the client. Every query is answered by the tail. A separate, fault-tolerant **master** service detects failures and reconfigures the chain [van Renesse and Schneider 2004]. *(§27.5.1)*

Change data capture (CDC) — Extracting every committed insert, update and delete from a database, in commit order, as a stream of change events that other systems consume. **Log-based CDC** reads the database's replication log (PostgreSQL's write-ahead log via logical decoding, MySQL's binary log) rather than querying tables. *(§32.6.1)*

Choreography and orchestration — In **choreography**, each service reacts to other services' events and publishes its own; the saga's control flow is distributed across the participants and exists nowhere as one piece of code. In **orchestration**, a dedicated **orchestrator** sends commands to participants, receives their replies and decides the next step; the control flow lives in one place [Richardson 2018]. *(§32.8)*

Chubby — Google's lock service, providing coarse-grained advisory locks and small files in a file-system-like namespace, replicated across a five-node **cell** with Paxos, and used chiefly for leader election, naming and storing small amounts of configuration [Burrows 2006]. *(§26.14.1)*

CIDR block — An address range written as `address/prefix-length`, for example `192.168.10.64/26`. The prefix length n says that the first n bits identify the network; the remaining $32 - n$ bits (IPv4) or $128 - n$ bits (IPv6) identify hosts. The **subnet mask** is the same information as a dotted number with n one-bits followed by zeros (`/26` = `255.255.255.192`). *(§7.4.2)*

Circuit breaker — A client-side proxy for calls to a dependency that tracks recent failures and, when they cross a threshold, **opens**: it fails calls immediately without contacting the dependency for a cool-down period, then lets a few trial calls through (**half-open**) and **closes** again only if they succeed. *(§22.13)*

Class — A template that describes the attributes and methods every object of a kind will have. Creating an object from a class is called **instantiation**; the object is an **instance** of the class. *(§35.3.1)*

Class diagram — A structure diagram showing classifiers (classes, interfaces, enumerations), their **attributes** and **operations**, and the **relationships** among them. A class is drawn as a box with up to three compartments: name, attributes, operations. *(§38.3.2)*

Class invariant — A condition on an object's state that is true whenever no method of the object is running: after the constructor finishes and after every public method returns. *(§35.4.1)*

Clean architecture — Robert C. Martin's synthesis of hexagonal, onion and similar architectures, drawn as concentric circles: **entities** (enterprise business rules) at the center, then **use cases** (application business rules), then **interface adapters** (controllers, presenters, gateways), then **frameworks and drivers** (web, database, devices) at the edge, governed by one **dependency rule**: source-code dependencies point only inward [Martin 2017]. *(§33.11.1)*

Client–server architecture — A way of dividing a system into two roles: a **client**, which initiates requests on behalf of a user or another program, and a **server**, which waits for requests, performs work on shared resources (data, computation) and returns responses. The two communicate over a network using an agreed **protocol**. *(§8.3.1)*

Client-side load balancing — The calling process obtains the list of backend endpoints from service discovery (§34.7) and chooses a backend for each request itself, with no proxy in the path. A **lookaside** balancer is an external service that tells clients *which* backends to use (and with what weights) while the traffic still flows directly from client to backend. *(§19.10)*

Clock rate — The frequency at which a core's internal circuitry steps from one state to the next, measured in hertz (cycles per second). A 4 GHz core has a **cycle time** of $1 / (4 \times 10^9)$ s = 0.25 ns. *(§5.3.2)*

Clustered and secondary index — In a **clustered index** (an index-organized table), the leaf level of the index *is* the table: rows are stored in key order inside the index's leaf pages. A table can have at most one. A **secondary index** stores, for each key, a pointer to the row; the table can have many. *(§11.7.2)*

Cohesion and coupling — **Cohesion** is how strongly the elements inside one module (here, one class) belong together: whether they serve a single purpose. **Coupling** is how strongly one module depends on others: how much it must know about them, and how likely a change in one is to force a change in the other. Good designs have **high cohesion and low coupling**. Both terms come from structured design [Stevens, Myers & Constantine 1974]. *(§35.10.1)*

Cohesion and coupling (service level) — **Cohesion** is the degree to which code inside one service changes together for the same reasons. **Coupling** is the degree to which a change in one service forces a change, redeploy or coordinated release in another. Newman's shorthand: "the code that changes together, stays together" [Newman 2021]. *(§34.3.2)*

Columnar storage (column store) — A layout that stores each column of a table contiguously — all values of `price`, then all values of `country` — instead of storing each row's fields together. Column stores are optimized for analytical queries that read a few columns of many rows [Abadi et al. 2013]. *(§13.13.1)*

Command — An object that encapsulates a request (the operation and its arguments) so that it can be stored, passed around, queued, logged, replayed and undone [Gamma et al. 1994]. In this book's terminology a command is a request to do something, as opposed to an event, a fact that has happened. *(§37.12)*

Command Query Responsibility Segregation (CQRS) — An architecture that uses one model to process commands (writes that enforce invariants) and one or more sepa-

rate models to answer queries, each read model shaped for the queries it serves and typically updated asynchronously from the write side's events [Young 2010; Fowler 2011b]. *(§32.4.1)*

Compaction — The background merge of sorted runs in an LSM tree that removes overwritten values and tombstones and bounds the number of runs a read must consult. A **compaction strategy** decides which runs to merge and when; it sets the engine's read, write and space amplification. *(§13.9.1)*

Compare-and-swap (CAS) — An atomic CPU instruction that, in one indivisible step, compares a memory location with an expected value and, only if they are equal, replaces it with a new value, reporting whether it succeeded. On x86-64 it is `LOCK CMPXCHG`; ARMv8.1 and later provide `CAS`, and earlier ARM uses load-linked/store-conditional pairs. *(§6.10.4)*

Competing consumers — A pattern in which several consumer processes read from the same queue concurrently, each message going to whichever consumer takes it first, so that adding consumers raises throughput [Hohpe and Woolf 2003]. *(§30.4.1)*

Component — A unit of a system with a clear responsibility and an interface — for example a web service, a database, a cache, a message queue, a load balancer or a third-party payment provider. *(§1.3.1)*

Component diagram — A structure diagram that shows **components** (modular, replaceable units that encapsulate their contents) and the **interfaces** they *provide* (offer to others, drawn as a "lollipop" circle) and *require* (need from others, drawn as a "socket" half-circle), connected by **assembly connectors** that wire a required interface to a matching provided one [OMG 2017]. *(§38.5.1)*

Components of network delay — **Propagation delay**: time for a signal to travel the physical distance. **Transmission (serialization) delay**: time to push all of a packet's bits onto the link. **Queuing delay**: time spent waiting in buffers behind other packets. **Processing delay**: time for a device to examine headers, look up routes and copy the packet. *(§7.15.1)*

Composition and delegation — **Composition** builds an object out of other objects that it holds in its fields (a **has-a** relationship). **Delegation** is the object forwarding part of its work to one of those parts by calling the part's methods. "Favor object composition over class inheritance" is the second principle stated in the GoF introduction [Gamma et al. 1994, ch. 1]. *(§35.8.1)*

Concept-mastery loop — A fixed sequence of ten learning activities applied to one system-design concept. Each activity produces a concrete artifact (a definition, a diagram, a program, a design). The concept counts as mastered only when you can rebuild every artifact without notes. *(§3.3.1)*

Concurrency and parallelism — **Concurrency** is the *structuring* of a program as multiple tasks whose lifetimes overlap, so that progress on one does not wait for the others to finish. **Parallelism** is the *simultaneous execution* of multiple computations on multiple hardware units (cores, CPUs, GPUs). *(§6.8)*

Condition variable — A queue of threads waiting for some condition on shared state to become true, always used together with a mutex. `wait` atomically releases the mutex and sleeps; `notify` (`signal`) wakes one waiter and `notify_all` (`broadcast`) wakes all. *(§6.10.3)*

Conflict-free replicated data type (CRDT) — A data type whose replicas can be updated independently and concurrently without coordination and are guaranteed to converge to the same state once they have received the same updates, a property called **strong eventual consistency** [Shapiro et al. 2011a]. *(§27.14.3)*

Congestion window (cwnd) — A sender-side limit, in bytes or segments, on unacknowledged data in flight, based on the sender's estimate of what the network path can absorb. The sender may have at most $\min(\text{cwnd}, \text{rwnd})$ bytes outstanding. *(§7.7.7)*

Connection gateway — A dedicated, thin service tier whose only job is to terminate long-lived client connections (WebSocket, SSE, long polling), authenticate them, track which user and topics each connection belongs to, and move messages between those connections and back-end services through a messaging layer. It contains no business logic. *(§9.9.1)*

Connection pooler — A component that keeps a few open database connections and lends them to many clients per session, transaction or statement, either inside the application or as a shared proxy. *(§14.13.1)*

Consensus — A set of processes, each of which may **propose** a value, must **decide** on a single value such that: **agreement** — no two correct processes decide differently; **validity** (integrity) — the decided value was proposed by some process; **termination** — every correct process eventually decides. Once decided, a value never changes. *(§26.3.1)*

Consistent hashing — A way of assigning keys to nodes so that, when a node is added or removed, only about K/N of the K keys change owner, where N is the number of nodes. Both keys and nodes are hashed onto the same circular number space (the **ring**). Each key belongs to the first node found by moving clockwise from the key's position [Karger et al. 1997]. *(§3.3.2)*

Consistent hashing with bounded loads — A variant of consistent hashing in which every backend has a **capacity** of $\lceil (1+\varepsilon)\, m/n \rceil$ for m current items (requests, sessions) and n backends. An item goes to the first backend clockwise from its hash that is below capacity, so overflow from a full backend walks on to the next [Mirrokni, Thorup & Zadimoghaddam 2018]. *(§19.4.7)*

Consumer-driven contract (CDC) — A specification, generated by the consumer's tests, of the requests a consumer sends to a provider and the parts of the responses it relies on. The provider verifies every consumer's contract against its real implementation in its own build, so a breaking change fails before deployment [Robinson 2006; Pact Docs]. *(§34.8.5)*

Consumer group — A set of consumer instances sharing a `group.id` that divide a topic's partitions among themselves so that **each partition is consumed by exactly one member** at a time. Different groups consume the same topic independently, each with its own offsets. *(§31.7.2)*

Consumer lag — For one partition, lag = **log end offset** (or high watermark) minus the group's **committed offset**: the number of records written but not yet processed by the group. Group lag is the sum over partitions. *(§31.7.3)*

Content-defined chunking (CDC) — Splitting a byte stream into variable-size chunks whose boundaries are chosen by the content itself: a rolling hash is computed over a small sliding window, and a boundary is declared wherever the hash matches a pattern (for example, its low 13 bits equal a fixed value). Each chunk is then identified by a cryptographic hash of its bytes, so identical chunks are stored once (**deduplication**). *(§15.8.3)*

Content delivery network (CDN) — A distributed system of caching reverse proxies (§7.12) deployed in many network locations, which serves content on behalf of an **origin** (the authoritative server for the content) from the location nearest each user, fetching from the origin only on a cache miss. *(§21.3.1)*

Context diagram — A diagram that shows the whole system as a single box, surrounded by the people (actors) and external systems that interact with it, with each interaction labeled. Context diagrams come from structured analysis [DeMarco 1979] and are Level 1 of the C4 model [Brown C4]. *(§2.4.1)*

Context switch — The kernel operation that stops one thread running on a CPU core, saves its CPU state into its PCB, and loads the saved state of another thread so that it resumes where it left off. *(§6.4)*

Coordinated omission — A measurement error in which the load generator, by waiting for slow responses before sending more requests, fails to send (and therefore fails to measure) the requests that would have experienced the slowness. The tester and the system "coordinate" to omit the worst samples. The term and diagnosis are due to Gil Tene [Tene, How NOT to Measure Latency]. *(§23.5.3)*

Cost-based optimizer — The component that enumerates equivalent physical plans for a query (access paths, join orders, join algorithms), estimates each plan's cost from **statis-**

tics about the data using a **cost model**, and returns the plan with the lowest estimated cost. *(§11.10.1)*

Count-Min sketch — A $d \times w$ array of counters with one hash function per row that estimates the frequency of any item in a stream. Each update increments one counter per row; each query returns the minimum of the item's d counters. The estimate never underestimates and, with probability at least $1 - \delta$, overestimates by at most εN, where N is the total of all updates [Cormode & Muthukrishnan 2005]. *(§17.6.1)*

CPU-bound and I/O-bound — A workload is **CPU-bound** when its speed is limited by how fast the processor can execute instructions; giving it a faster CPU or more cores speeds it up. It is **I/O-bound** when it spends most of its time waiting for storage or the network; a faster CPU barely helps, but more concurrency or faster devices do. A related category, **memory-bound**, is limited by memory latency or bandwidth (§5.9) — the CPU looks "busy" but is mostly stalled. *(§5.8)*

CPU cache — A small, fast memory built from SRAM (static RAM) on the processor chip that holds copies of recently used main-memory data, so that most loads and stores do not have to wait for DRAM. *(§5.4.1)*

CPU scheduler — The part of the kernel that decides which ready thread runs next on each core and for how long. The rule it follows is the **scheduling policy**; the mechanism that performs the switch is the dispatcher (§6.4). *(§6.5)*

Critical section — A region of code that accesses shared state and must not be executed by more than one thread at a time. Guaranteeing this is **mutual exclusion**. *(§6.9)*

Cryptographic hash function — A deterministic function H that maps an input of any length to a fixed-length output (the **digest**), such that it is computationally infeasible to (1) find an input that hashes to a given digest (**preimage resistance**), (2) given one input, find a different input with the same digest (**second-preimage resistance**), and (3) find any two distinct inputs with the same digest (**collision resistance**). *(§10.4.1)*

Cuckoo filter — A hash table of b-slot buckets that stores a short **fingerprint** of each item (an f-bit hash) in one of two candidate buckets. Membership checks two buckets for a matching fingerprint; deletion removes one copy of the fingerprint. *(§17.4.7)*

D

Database-per-service — Each service's persistent data is private to that service and reachable by others only through its API or published events. The service may use its own database server, its own schema on a shared server, or private tables, provided no other service reads or writes them [Richardson 2018]. *(§34.11.1)*

DataFrame, Dataset and Catalyst — A **DataFrame** is a distributed table with a known schema (named, typed columns), manipulated with relational operators or SQL. A **Dataset** (Scala and Java) adds compile-time types to the same abstraction; in Python and R, every DataFrame is untyped at compile time. **Catalyst** is Spark SQL's extensible query optimizer, which turns DataFrame or SQL queries into an optimized physical plan using rule-based and cost-based transformations of trees [Armbrust et al. 2015]. *(§29.5.4)*

Data locality — Scheduling a task on or near the node that stores its input, so that the input is read from local disk or memory instead of over the network. Schedulers rank placements as **process-local** (data cached in the same executor), **node-local**, **rack-local** and **off-rack** (any). *(§29.7)*

Dead-letter queue (DLQ) — A separate queue or topic to which a broker or consumer moves messages that cannot be processed (delivery limit exceeded, rejected as invalid, expired), preserving them for inspection and later reprocessing instead of losing or endlessly retrying them [Hohpe and Woolf 2003, "Dead Letter Channel"]. *(§30.11.1)*

Deadline — An absolute point in time by which a client needs an RPC to complete. gRPC transmits it as a remaining timeout; when it passes, the client stops waiting with `DEADLINE_EXCEEDED` and the server can see that the call was canceled [gRPC Docs: Deadlines]. *(§8.8.4)*

Deadlock — A state in which each thread in a set is waiting for a resource held by another thread in the same set, so none can ever proceed. *(§6.11.1)*

Decomposition — Dividing a system into smaller parts (**components**) so that each part has one clear responsibility, a well-defined interface, and as few dependencies on other parts as possible. Each part can then be understood, built, scaled and changed mostly on its own. *(§2.3.2)*

Decorator (GoF) — An object that implements the same interface as the object it wraps, forwards calls to it, and adds behavior before or after them, so responsibilities can be added at run time and stacked in any order without subclassing [Gamma et al. 1994]. *(§37.10)*

Deliberate practice — Effortful practice designed to improve performance. It targets specific weaknesses, has clear goals, gives immediate feedback and is usually guided by a teacher or coach [Ericsson et al. 1993]. *(§3.4.6)*

Delivery guarantees — **At-most-once**: each message is processed zero or one times; failures lose messages but never duplicate them. **At-least-once**: each message is processed one or more times; failures cause duplicates but never loss. **Exactly-once** (more precisely **effectively-once**): the *observable effect* of processing is as if each message were processed exactly once, even though delivery may happen more than once. *(§30.8)*

Denormalization — Deliberately storing a fact more than once, or storing a derived value, to make important reads cheaper, while accepting the cost of keeping the copies consistent. *(§11.5.5)*

Dependency injection (DI) — A technique in which an object receives the objects it depends on (its dependencies) from outside, typically through its constructor, instead of creating or looking them up itself. The code that creates and connects all objects is the **composition root** [Fowler 2004]. *(§37.14)*

Dependency Inversion Principle (DIP) — (a) High-level modules should not depend on low-level modules; both should depend on abstractions. (b) Abstractions should not depend on details; details should depend on abstractions [Martin 2002]. **High-level** modules hold policy: the business rules that justify the system (checkout, invoicing). **Low-level** modules hold detail: mechanisms that could be swapped without changing the business (a payment vendor's SDK, SMTP, SQL). *(§36.7.1)*

Design assumption — A belief about the workload, the environment or the business that the design depends on but that has not yet been verified, for example "80% of requests are reads" or "the payment provider responds within 1 s at p99". An assumption is **stated** when it is written down with its value, its source, your confidence in it and how it will be checked. *(§2.7.1)*

Design by contract — A method of specifying software in which each routine states a **precondition** (what must be true when it is called: the caller's obligation), a **postcondition** (what it guarantees when it returns: the supplier's obligation), and each class states an **invariant** (what is true of every instance between calls). The approach, and the term, come from Bertrand Meyer and the Eiffel language [Meyer 1997]. *(§35.9.1)*

Design document (design doc) — A written proposal, produced before significant implementation, that states a problem, its goals and non-goals, a proposed design, the alternatives considered, and cross-cutting concerns (security, privacy, operations, cost), so that others can review it while changes are still cheap. *(§1.10.1)*

Designing for failure — Treating component failure as a *normal, expected event* rather than an exception. For every component and dependency, the design states what happens when it is slow, returns errors or disappears, and makes sure the **blast radius** (the set of users and features affected) is as small as possible. *(§2.9.1)*

Design pattern — A named description of a recurring design problem, the core of a solution (a set of collaborating classes or functions and their responsibilities), and the trade-offs of applying it. A pattern is a template you adapt, not a library you import [Gamma et al. 1994]. *(§37)*

Deterministic database — A database in which replicas and partitions first agree on a global order of transaction *inputs*, then execute transactions so that the outcome depends

only on that order and the initial state. Every node that executes the same log reaches the same state, so no node needs to ask another whether to commit [Thomson et al. 2012]. *(§28.9.1)*

Digital signature — A scheme with three algorithms: key generation, **sign** (using the private key, produces a signature over a message) and **verify** (using the public key, checks a signature against a message). A valid signature shows that the message was approved by the holder of the private key and has not changed since. Because verification needs only the public key, *anyone* can verify, which provides non-repudiation. *(§10.4.5)*

Direct memory access (DMA) — A mechanism that lets a device controller read from or write to main memory directly, without the CPU copying each word; the CPU only sets up the transfer and is told when it finishes. *(§5.7.3)*

Direct server return (DSR) — A forwarding mode in which the balancer delivers request packets to the backend but the backend sends responses **directly to the client**, using the VIP as its source address, bypassing the balancer on the return path. *(§19.3.1)*

Distributed denial of service (DDoS) — An attack that makes a service unavailable by overwhelming some finite resource — network bandwidth, packet-processing capacity, connection-state tables, or application CPU, memory and database capacity — with traffic from many sources at once, typically a **botnet** of compromised devices or abused open servers. *(§10.11)*

Distributed file system — A file system whose namespace and data are spread across many machines, so that files can be larger than any one disk and throughput can grow with the number of machines, while clients still use file-like operations (create, open, read, write or append, delete). *(§15.5.1)*

Distributed monolith — A system deployed as multiple services that cannot be changed, deployed or operated independently: services share databases, must be released together, or call each other synchronously so tightly that one failure stops all of them. It has the costs of distribution and the coupling of a monolith [Newman 2021]. *(§34.14.1)*

Distributed system — A set of independent computers (**nodes**) that have no shared memory and no shared clock, communicate only by exchanging messages over a network, and cooperate to provide a service. Van Steen and Tanenbaum describe it from the user's side as "a collection of autonomous computing elements that appears to its users as a single coherent system" [van Steen and Tanenbaum 2017]. *(§24.3.1)*

DNS time to live (TTL) — A per-record value, in seconds, that tells any cache how long it may reuse the record before asking again. **Negative caching** stores "this name does not exist" (NXDOMAIN) answers for a duration derived from the zone's SOA record [RFC 2308]. *(§7.6.4)*

Document database — A database that stores self-describing, hierarchical records — **documents**, usually JSON or a binary encoding of it such as BSON — grouped into **collections**, and that can index and query fields inside the documents, including nested fields and array elements. *(§12.5.1)*

Domain-driven design (DDD) — An approach that places a model of the business domain, built jointly by developers and domain experts, at the center of the design, expresses it directly in code, and divides large domains into explicitly bounded models [Evans 2003; Vernon 2013]. *(§34.4)*

Domain event — An immutable record of something domain experts care about that happened, named in the past tense (`OrderPlaced`, `CourierAssigned`), carrying the identifiers and facts its consumers need [Evans 2003; Vernon 2013]. *(§34.4.4)*

Domain Name System (DNS) — A hierarchical, distributed database that maps human-readable **domain names** (such as `www.example.com`) to data, most often IP addresses, and the protocol used to query it [RFC 1034; RFC 1035]. *(§7.6.1)*

Double-checked locking — A lazy-initialization idiom that checks whether an object exists without a lock (fast path), and only if it does not, takes a lock, checks again, and creates it. It avoids locking on every access but is only correct if the language's memory model guarantees that a thread seeing the reference also sees the fully constructed object. *(§37.16.5)*

DRAM (dynamic random-access memory) — The technology used for main memory. Each bit is stored as charge in a tiny capacitor that leaks and must be **refreshed** every few tens of milliseconds. DRAM is **volatile**: its contents are lost when power is removed. *(§5.5.1)*

Duck typing — A style of polymorphism in which an object is acceptable wherever it supports the operations actually used, regardless of its declared class ("if it walks like a duck and quacks like a duck"). Checking happens when the method is called. **Structural typing** is the static version: a type checker accepts an object if its shape matches a declared protocol. *(§35.6.2)*

Durability — The probability that data, once acknowledged as written, is not lost over a given period (usually a year). It is quoted in nines like availability: "eleven nines of durability" means an annual loss probability of 10^{-11} per object. *(§4.16.1)*

Durable execution — A programming model in which a function's progress is persisted by an engine as it runs, so that the function survives process crashes, deploys and arbitrarily long waits and continues exactly where it left off, as if the process had never died [Temporal Docs]. *(§32.11.1)*

Durable subscription — A subscription whose messages the broker retains while the subscriber is disconnected, so the subscriber receives everything published since it subscribed, within the retention period. An **ephemeral** (non-durable) subscription exists only while the subscriber is connected; messages published while it is away are never delivered to it. *(§30.5.2)*

Dynamic site acceleration (DSA) — Routing uncacheable requests through the CDN anyway, so that TCP and TLS terminate near the user and the edge forwards each request to the origin over pre-established, tuned, long-lived connections, sometimes along an overlay path chosen from measurements. *(§21.7.2)*

E

Elasticsearch cluster — A set of nodes that share one cluster state (index definitions, mappings and the location of every shard copy). An **index** is a logical collection of documents divided into **primary shards**; each primary has zero or more **replica shards**; every shard is one Lucene index. *(§16.8.1)*

Encapsulation — The process by which each layer of the stack prepends its own header (and sometimes appends a trailer) to the data unit handed down by the layer above, treating that data unit as an opaque payload. **Decapsulation** is the reverse at the receiver. *(§7.3.1)*

Entity and value object — An **entity** has a continuous identity that persists as its attributes change: two orders for the same pizza are two orders. A **value object** is defined entirely by its attributes, has no identity and is usually immutable: `Money(24.50, "USD")`, `Address` [Evans 2003]. *(§34.4.2)*

Envelope encryption — Encrypting data with a **data-encryption key (DEK)** generated for that data, then encrypting (“wrapping”) the DEK with a KEK held in a KMS, and storing the wrapped DEK alongside the ciphertext [AWS Docs: KMS concepts]. *(§10.9.2)*

Erasure code and RS(k, m) — An **erasure code** transforms k data fragments into $k + m$ fragments such that the original data can be rebuilt from some subset of them after the others are lost (“erased”). A **Reed–Solomon** code RS(k, m) is a **maximum distance separable (MDS)** code: *any* k of the $k + m$ fragments suffice, so it tolerates the loss of any m fragments. A code is **systematic** when the first k fragments are the unmodified data, so healthy reads need no decoding. *(§15.6.1)*

Event-driven architecture (EDA) — A style in which components communicate mainly by producing and consuming **events**, immutable records of facts that happened (“OrderPlaced”), through a broker or log, so that producers do not know which consumers exist or when they will run. *(§33.8)*

Event loop — A single thread that repeatedly waits for events (ready sockets, expired timers, completed tasks) using an I/O multiplexing call, and dispatches each event to a short, non-blocking handler. *(§6.13.4)*

Event sourcing — A persistence pattern in which the system of record for an entity is the append-only, ordered sequence of **domain events** that changed it (`AccountOpened`, `MoneyDeposited`, …). Current state is not stored as truth; it is derived by replaying the events, a **left fold** of a pure function `apply(state, event)` over the sequence [Fowler 2005]. *(§32.3.1)*

Event storming — A collaborative workshop technique created by Alberto Brandolini in which domain experts and developers place domain events (past tense, on sticky notes) along a timeline, then add commands, actors, policies ("whenever X, then Y"), external systems and hotspots (open questions), and group the result into aggregates and candidate bounded contexts [Brandolini: Introducing EventStorming]. *(§34.5.4)*

Eventual consistency — If no new updates are made to an object, eventually all reads return the last updated value [Vogels 2009]. Equivalently, replicas that have received the same set of updates **converge** to the same state. **Strong eventual consistency** strengthens this: replicas that have received the same updates are in the same state immediately, with no further coordination, which is what conflict-free replicated data types (CRDTs) guarantee (§27.14). *(§25.7.1)*

Expected value — The probability-weighted average outcome of a random quantity X: $E[X] = \sum_i x_i \, P(X = x_i)$. It is the long-run average over many repetitions. *(§4.6.3)*

Exponent — In b^n, the **base** b is multiplied by itself n times; n is the **exponent** (or power). By convention $b^0 = 1$ and $b^{-n} = 1/b^n$. *(§4.3.1)*

External consistency — If transaction T_1 commits before T_2 starts, in absolute (wall-clock) time, then T_1's commit timestamp is smaller than T_2's. Combined with serializability, this is **strict serializability**: the serial order respects real-time order, as linearizability does for single objects (§25.3). *(§28.8.3)*

F

Factory method — A method declared in a base class that returns a product through an interface, which subclasses override to decide which concrete product to create. The base class's other methods use the product without knowing its class [Gamma et al. 1994]. *(§37.3.2)*

Failover — The sequence that replaces a failed node in a role: **detection**, **election** of a replacement, **reconfiguration** (making the replacement consistent and fencing the old node) and **client redirection**. *(§27.11.1)*

Failure detector — A component that outputs, for each monitored process, whether it currently **suspects** that process has crashed. It is **complete** if every crashed process is eventually suspected and **accurate** if live processes are not suspected. In an asynchronous network no detector can be both perfectly complete and perfectly accurate, because a slow process is indistinguishable from a dead one [Chandra and Toueg 1996]. *(§27.10.1)*

False sharing — A performance problem in which threads on different cores write to *different* variables that happen to sit in the *same* cache line, so the coherence protocol bounces the line between cores as if they were sharing data. *(§5.4.7)*

Fencing token — A number, increasing with every grant of a lock or lease, that the holder attaches to every request to the protected resource. The resource remembers the highest token it has accepted and rejects any request carrying a lower one. *(§26.13)*

Firewall — A device or software function that permits or denies traffic according to rules. A **stateless** (packet-filter) firewall evaluates each packet independently against rules on addresses, ports and protocol. A **stateful** firewall tracks connections, so a reply to an allowed outbound connection is automatically permitted. **Application-layer firewalls**, including web application firewalls (WAFs), parse protocols such as HTTP to block malicious requests. *(§7.14)*

Fixed window counter — A limiter that divides time into aligned windows of length W (for example each clock minute), keeps one counter per key per window, and admits a request if the current window's counter is below the limit L. The counter resets at each window boundary. *(§22.6.1)*

FLP impossibility — In a fully asynchronous message-passing system with reliable links, no *deterministic* protocol can solve **consensus** (all correct processes decide the same value, the value was proposed by some process, and every correct process eventually decides) if even one process may crash-stop [Fischer et al. 1985]. *(§24.13.1)*

Forward secrecy — A property of a key-exchange protocol whereby compromise of a party's long-term private key does not reveal session keys from past sessions. It is achieved by using fresh **ephemeral** key pairs for every session (the "E" in ECDHE) and deleting them afterward. *(§10.4.4)*

Four golden signals — Google's site reliability engineering guidance says that if you can measure only four metrics of a user-facing system, measure **latency** (time to serve a request, tracking successful and failed requests separately), **traffic** (demand, such as requests per second), **errors** (rate of failed requests, explicit or implicit) and **saturation** (how "full" the service is, emphasizing its most constrained resource) [Beyer et al. 2016, ch. 6]. *(§23.3.3)*

Fragile base class problem — A change to a base class that is correct on its own breaks subclasses, because the subclasses depend on implementation details of the base (which

methods call which) that were never part of its documented interface. *(§35.5.2)*

Freshness and validation — A stored response is **fresh** while its age is below its **freshness lifetime** and may then be served without contacting the origin; once **stale**, it must normally be **revalidated**, by sending a conditional request (`If-None-Match` with the stored `ETag`, or `If-Modified-Since`), to which the origin answers `304 Not Modified` (keep using the copy) or `200 OK` with a new body [RFC 9111; RFC 9110]. *(§21.6.1)*

Functional dependency — In a relation R, the functional dependency $X \to Y$ holds if any two rows that agree on the columns X also agree on the columns Y. X is the **determinant**. Example: `product_id → product_name` because one product ID always has one name. *(§11.5.1)*

Functional requirement (FR) — A statement of a behavior the system must provide: an input, action or event, and the observable result. Functional requirements describe *what* the system does — for example, "a signed-in user can upload a JPEG or PNG photo up to 20 MB, and it appears in their followers' feeds". *(§1.6.1)*

Future and promise — A **future** is a read-only placeholder for a result that will be available later; a **promise** (Java's `CompletableFuture`, Python's `asyncio.Future`) is the writable side that the producer completes with a value or an error. Together they let a caller start work, continue, and later wait for or chain on the result. *(§37.16.3)*

G

Garbage collection (GC) — Automatic reclamation of heap memory that the program can no longer reach. A **tracing** collector finds live objects by following references from **roots** (stacks, globals, registers) and reclaims everything else. A **stop-the-world (STW) pause** is an interval in which all application threads (the **mutator**) are halted so the collector can work on a consistent heap [Jones et al. 2023]. *(§23.9.1)*

Geohash — A string encoding of a latitude/longitude cell, published by Gustavo Niemeyer in 2008, formed by alternately bisecting the longitude and latitude ranges, interleaving the resulting bits (longitude first) and writing every 5 bits as one base-32 character [Niemeyer 2008]. Each additional character subdivides the cell into 32 smaller cells, so a geohash that is a prefix of another names a cell containing it. *(§17.9.2)*

Gossip protocol — A protocol in which, every round, each node exchanges state with f randomly chosen peers (the **fanout**). In **push** gossip informed nodes send updates; in **pull** gossip nodes ask peers for news; **push–pull** does both. **Anti-entropy** gossip exchanges full state; **rumor mongering** spreads only recent updates and stops after a while [Demers et al. 1987]. *(§27.10.3)*

Graph database — A database whose data model is a graph of **vertices** (nodes) and **edges** (relationships), optimized for traversing edges. In the **property graph** model, nodes carry labels, edges carry a type and a direction, and both carry key-value properties. In the **RDF** model, all data is a set of subject–predicate–object **triples**. *(§12.7.1)*

GraphQL — A query language for APIs and a runtime for executing those queries against a strongly typed **schema**. The client sends a document describing exactly the fields it wants; the server returns JSON of the same shape [GraphQL Spec]. *(§8.7.1)*

Gray failure — A component failure that is *differentially observable*: some observers (often the system's own failure detector) consider the component healthy, while others (often the applications using it) experience it as failed or badly degraded. *(§24.8.2)*

gRPC — An open-source RPC framework, originally developed at Google and released in 2015, that uses Protocol Buffers as its interface definition language and message format and HTTP/2 as its transport [gRPC Docs]. *(§8.8.2)*

H

Happened-before (→) — The smallest relation on events such that: (1) if a and b occur in the same process and a comes first, $a \to b$; (2) if a is the sending of a message and b its receipt, $a \to b$; (3) if $a \to b$ and $b \to c$, then $a \to c$. Two distinct events are **concurrent** ($a \parallel b$) if neither $a \to b$ nor $b \to a$. *(§24.10.1)*

Hard disk drive (HDD) — A storage device that records data magnetically on spinning platters, read and written by heads mounted on a moving arm (the **actuator**). Data is organized in concentric **tracks** divided into **sectors** of 512 or 4096 bytes. *(§5.6.1)*

Hash function — A deterministic function h that maps an input of any length to a fixed-size integer (for example 32, 64 or 128 bits), called the **hash** or **digest**. A **non-cryptographic hash function** is designed for speed and statistical uniformity on ordinary inputs. A **cryptographic hash function** additionally resists an adversary who tries to find collisions or preimages (§10.4.1). *(§17.3.1)*

Hash slot (Redis Cluster) — One of 16,384 buckets into which Redis Cluster divides the keyspace: `slot = CRC16(key) mod 16384`. Each primary owns a set of slots. If a key contains `{...}`, only the substring inside the braces (a **hash tag**) is hashed, so related keys can be forced into one slot [Redis Docs: Cluster specification]. *(§20.6.4)*

Health check — A mechanism by which a balancer decides whether a backend should receive traffic. An **active** check sends synthetic probes (a TCP connect, an HTTP `GET /healthz`, a gRPC health RPC) on a timer. A **passive** check, also called **outlier detection**, watches real traffic for errors, timeouts or latency and **ejects** misbehaving backends for a period. *(§19.5.1)*

Hedged request — A request sent to one replica and, if no response arrives within a **hedge delay** d, sent again to a second replica; the client uses whichever response arrives first and cancels or ignores the other [Dean and Barroso 2013]. *(§23.7.1)*

Hexagonal architecture (ports and adapters) — A style, described by Alistair Cockburn, in which the application's core logic interacts with the outside world only through **ports**, interfaces defined by and in the language of the core, and each external technology (web framework, database, message broker, test harness) connects through an **adapter** that implements or calls a port [Cockburn 2005]. **Driving (primary) ports** are how the outside asks the application to do something; **driven (secondary) ports** are how the application asks the outside for something. *(§33.10.1)*

High-level design (HLD) — A description of a system as a set of major components (services, data stores, queues, caches, external dependencies), their responsibilities, the interfaces and protocols between them, data flows, and deployment topology — at a level of detail where each component is a box whose internals are not yet specified. *(§1.5.1)*

Horizontal scaling (scaling out) — Increasing capacity by adding more nodes that share the workload, for example by running 20 identical application servers behind a load balancer instead of one. Removing nodes when load falls is **scaling in**. *(§18.4.1)*

Hot key — A single key whose request rate exceeds what the node holding it can serve, so partitioning cannot help: all requests for one key go to one node. *(§20.15)*

Hot partition and hot key — A **hot partition** receives enough excess requests or data to saturate its node while peers idle. A **hot key** is a single key causing that excess. **Skew** is uneven distribution of data or load in general. *(§14.10.1)*

HTTP adaptive streaming — Video is encoded at several quality levels (a **bitrate ladder**), each cut into **segments** of a few seconds stored as ordinary files. A **manifest** lists the levels and segment URLs. The player downloads the manifest, then fetches segments one by one over HTTP, choosing a level for each segment. *(§21.8.1)*

HTTP cookie — A small name–value pair that a server asks the browser to store, via the `Set-Cookie` response header, and that the browser automatically returns in the `Cookie` request header on later requests to matching URLs [RFC 6265]. *(§8.10.1)*

Hybrid logical clock — A timestamp (l, c) where l is the largest physical clock value the node has seen (from its own clock or from any message) and c is a logical counter that orders events sharing the same l. Timestamps compare lexicographically: first by l, then by c. *(§24.12.1)*

HyperLogLog (HLL) — A probabilistic algorithm that estimates the number of distinct items (the **cardinality**) in a multiset using m small registers, with relative standard error about $1.04/\sqrt{m}$, independent of the cardinality itself [Flajolet et al. 2007]. *(§17.5.1)*

I

Idempotency key — A unique value, generated by the client once per *logical* operation and sent with every attempt of that operation (for example in an `Idempotency-Key` header), that the server uses to recognize retries: the first attempt executes and its result is stored under the key; later attempts with the same key return the stored result instead of executing again. *(§22.15.1)*

Idempotent consumer — A message consumer whose effect on state is the same whether a given message is processed once or many times, achieved either because the operation is naturally idempotent or because the consumer detects and skips messages it has already processed. *(§32.10)*

Idempotent method — A method for which the intended effect on the server of sending the same request $N > 0$ times is the same as sending it once [RFC 9110 §9.2.2]. The *response* may differ (a second `DELETE` may return 404); the *state* does not. *(§8.6.3)*

Identity and equality — **Identity** asks "are these the same object in memory?" (Python `is`, Java `==` on references). **Equality** asks "do these objects represent the same value by a rule the class defines?" (Python `==` via `__eq__`, Java `equals`). Unless a class overrides equality, both languages default to identity. *(§35.3.2)*

Index — A secondary data structure, kept in sync with a table by the database, that maps values of one or more **key columns** to the locations of the rows holding them, so that rows can be found without scanning the whole table. Most relational indexes are B+ trees (§11.8); others include hash, GiST, GIN and BRIN indexes in PostgreSQL and full-text and spatial indexes in MySQL. *(§11.7.1)*

In-doubt (blocked) transaction — A transaction at a participant that has voted yes but has not learned the outcome. It must hold its locks and cannot decide until the coordinator, or another participant that knows the outcome, answers. *(§28.4.3)*

Inheritance — A mechanism by which a class (the **subclass**, **child** or **derived class**) is defined as an extension of another (the **superclass**, **parent** or **base class**). The subclass automatically has the parent's attributes and methods, may add new ones, and may **override** inherited methods by providing its own version with the same name and parameters. *(§35.5.1)*

Instruction pipeline — A technique in which the core splits instruction execution into stages and works on different stages of several consecutive instructions at the same time, like an assembly line. *(§5.3.3)*

Interface Segregation Principle (ISP) — Clients should not be forced to depend on methods that they do not use [Martin 2002]. A **fat interface** serves many kinds of client;

a **role interface** describes what one kind of client needs from its collaborator [Fowler 2006]. *(§36.6.1)*

Interleaving — Mixing different kinds of problems or concepts within one practice session (A B C A C B), instead of finishing all practice on one kind before moving to the next (AAA BBB CCC, called **blocking**). *(§3.4.5)*

Inverted index — A mapping from each distinct **term** (a normalized word or token) to a **postings list**: the sorted list of identifiers of the documents that contain the term, optionally with the term's frequency and positions in each document. The set of all terms is the **dictionary** (or **vocabulary**, or **lexicon**). *(§16.3.1)*

I/O (input/output) — Any transfer of data between the CPU/memory and a device outside it: storage, network cards, GPUs, keyboards. A **device controller** on each device does the physical work; the CPU communicates with it through **device registers** and shared memory. *(§5.7)*

IOPS — Input/output operations per second: the number of independent read or write requests a device completes per second, quoted for a stated request size (usually 4 KiB) and **queue depth** (number of requests outstanding at once). *(§5.6.5)*

IP address — A numeric identifier assigned to a network interface that the Internet Protocol uses to deliver packets. An **IPv4** address is 32 bits, written as four decimal octets (`203.0.113.25`) [RFC 791]. An **IPv6** address is 128 bits, written as eight groups of four hexadecimal digits separated by colons (`2001:db8:0:1::25`) [RFC 8200]. *(§7.4.1)*

Isolation anomalies — **Dirty write:** a transaction overwrites another's uncommitted write. **Dirty read:** a transaction reads another's uncommitted write (which may later roll back). **Non-repeatable (fuzzy) read:** a transaction reads a row twice and sees different committed values because another transaction updated it in between. **Phantom:** a transaction runs the same *predicate* query twice (`WHERE status = 'pending'`) and gets a different set of rows because another transaction inserted or deleted matching rows. **Lost update:** two transactions read the same value, both compute a new value from it and write; the second write silently overwrites the first. **Read skew:** a transaction sees part of another transaction's effects (reads account A before a transfer commits and account B after it). **Write skew:** two transactions read an overlapping set of rows, each checks an invariant, then each writes a *different* row; each write is valid alone but together they violate the invariant. *(§11.13.1)*

ISR, LEO and high watermark — Each partition has a **replication factor** (RF) of replicas on different brokers; one is the **leader**, which handles all writes, and the others are **followers**, which fetch from the leader. The **in-sync replica set (ISR)** is the leader plus the followers that have fully caught up within `replica.lag.time.max.ms` (default 30 s). Each replica's **log end offset (LEO)** is the offset its next appended record will get. The **high watermark (HW)** is the offset below which every ISR member has the record;

records below the HW are **committed** and are the only ones consumers can read. *(§31.9.1)*

J

JSON Web Token (JWT) — A compact, URL-safe string representing a set of **claims** (statements about a subject) as a JSON object, digitally signed (JWS) or encrypted (JWE) so its integrity can be verified [RFC 7519]. This section covers the signed form, which is by far the most common. *(§8.13.1)*

K

Key management service (KMS) — A managed service, backed by hardware security modules (HSMs), that generates and stores **key-encryption keys (KEKs)** that never leave it in plaintext, performs cryptographic operations with them on request, enforces access policies on every use and logs every call. Examples: AWS KMS, Google Cloud KMS, Azure Key Vault, HashiCorp Vault's Transit engine. *(§10.9.2)*

Keys — A **superkey** is a set of columns whose values are unique across all rows. A **candidate key** is a minimal superkey: remove any column and it stops being unique. The **primary key** is the candidate key chosen as the row's main identifier; it must be unique and NOT NULL. Other candidate keys are enforced with `UNIQUE` constraints. A **foreign key** is a column (or set) in one table whose values must match a key of another table, a rule called **referential integrity**. *(§11.4.1)*

Key-value store — A database whose primary interface is `get(key)`, `put(key, value)` and `delete(key)`, where the value is opaque to the database or has a small set of built-in types. The key alone determines where the value lives. *(§12.4.1)*

L

Lambda and Kappa architectures — The **Lambda architecture** runs two parallel paths over the same immutable master dataset: a **batch layer** that periodically recomputes complete, accurate views from all history, and a **speed layer** that processes recent data incrementally with low latency; a **serving layer** merges the two at query time [Marz and Warren 2015]. The **Kappa architecture** uses a single stream-processing path over a replayable log; to change logic or fix a bug, it reprocesses history by replaying the log through a new version of the job [Kreps 2014]. *(§29.10)*

Lamport clock — Each process P_i keeps an integer counter C_i, initially 0. **Rule 1:** before each event (local, send or receive), $C_i \leftarrow C_i + 1$; a send attaches the new C_i to the message.

Rule 2: on receiving a message with timestamp t_m, set $C_i \leftarrow \max(C_i, t_m)$, then apply rule 1. The event's timestamp is the resulting C_i. *(§24.10.2)*

Last-writer-wins (LWW) — Each write carries a timestamp, and when replicas hold different values for an item, the one with the greatest timestamp is kept and the others are discarded. *(§27.14.1)*

Latency and bandwidth (hardware) — **Latency** is the time from issuing one request to receiving its first byte (for example, ~100 ns for a DRAM load). **Bandwidth** is the maximum rate at which data can be moved once transfers are flowing (for example, ~38 GB/s for one DDR5-4800 channel). Achieved bandwidth is **throughput**. *(§5.9.1)*

Law of Demeter (object form) — A method `M` of an object `O` should call methods only on: `O` itself; `M`'s parameters; objects `M` creates; and `O`'s own fields (its direct parts) [Lieberherr & Holland 1989]. Informally: "talk only to your immediate friends", or "use only one dot" (excluding fluent builders and value-object chains). *(§35.10.2)*

Layer 4 (L4) and layer 7 (L7) load balancing — An **L4 load balancer** makes its decision from transport-layer information (IP addresses, ports, protocol) and balances **connections or flows**: every packet of one TCP connection goes to the same backend. An **L7 load balancer** terminates the client connection, parses the application protocol (usually HTTP), and balances **requests**: two requests on one connection may go to different backends. *(§19.3.1)*

Layered architecture — A style that organizes code into horizontal **layers**, each with one kind of responsibility (presentation, business logic, persistence), where each layer may depend only on layers below it. A **tier** is a *physical* deployment boundary (browser, application server, database server); an **n-tier architecture** maps layers onto separately deployed tiers. *(§33.3.1)*

Leader election — A protocol by which the nodes of a group choose one node, the **leader**, to coordinate some activity (ordering writes, running a scheduler, owning a partition), and choose a replacement when it fails. The goal is *at most one* leader at a time (safety) and *eventually some* leader (liveness). *(§26.5.1)*

Leaderless replication — Any replica of a key may accept reads and writes; the client, or a **coordinator** node acting for it, sends each request to several replicas in parallel and treats it as successful once enough of them respond. There is no failover because there is no leader [DeCandia et al. 2007]. *(§27.7.1)*

Leaf–spine (folded Clos) network — A two-tier topology in which every **leaf** (top-of-rack) switch connects to every **spine** switch, and servers attach only to leaves. Any two servers on different leaves are exactly two switch hops apart (leaf → spine → leaf), and traffic is spread across all spines with ECMP. It is a form of the multistage **Clos network**,

first described by Charles Clos in 1953 for telephone switching. Larger fabrics add a third tier. *(§7.17.1)*

Leaky bucket — (a) **As a queue:** requests enter a FIFO buffer of capacity Q that is drained at a constant rate r; arrivals that find the buffer full are dropped. Output is perfectly smooth. (b) **As a meter:** a counter (the "water level") increases by one per admitted request and leaks at rate r; a request that would overflow capacity is rejected immediately. The meter form makes the same admit/reject decisions as a token bucket with the same rate and capacity. *(§22.5.1)*

Lease — A time-bounded grant of a right (to cache data, to act as leader, to hold a lock) from a grantor to a holder. Within the term the holder may act without consulting the grantor; after the term, the right lapses automatically, even if the holder has crashed or is unreachable [Gray and Cheriton 1989]. *(§26.11.1)*

Least connections — Send each new connection or request to the backend with the fewest **active** (in-flight) connections or requests, as counted by the balancer. *Weighted least connections* divides each count by the backend's weight. Envoy calls the request-counting variant **least request**; AWS ALB calls it **least outstanding requests**. *(§19.4.2)*

Least response time — Choose the backend with the lowest recent latency, usually an **exponentially weighted moving average (EWMA)** of observed response times, often multiplied by the in-flight count so that both queue length and speed count. *(§19.4.3)*

Linearizability — A history is **linearizable** if each operation can be assigned a single instant, its **linearization point**, somewhere between its invocation and its response, such that (1) the operations, ordered by these points, form a **legal sequential history** of the object (every read returns the value of the most recent preceding write), and (2) therefore whenever a precedes b in real time, a comes before b in that order. Pending operations may be either included (with a linearization point after their invocation) or dropped. An object is linearizable if all its histories are [Herlihy & Wing 1990]. *(§25.3.2)*

Liskov Substitution Principle (LSP) — Subtypes must be substitutable for their base types [Martin 2002]. More precisely, S is a **behavioral subtype** of T if every property you can prove about objects of type T, using T's specification, also holds for objects of type S [Liskov and Wing 1994]. *(§36.5.1)*

Little's Law — For any stable system observed over a long period, the average number of items in the system equals the average arrival rate multiplied by the average time each item spends in the system: $L = \lambda W$ [Little 1961]. *(§4.11.1)*

Load balancer — A component that receives traffic addressed to a single service endpoint, the **virtual IP address (VIP)** or service name, and distributes it across a **pool** of **backends** (also called *upstreams*, *targets* or *real servers*) according to a **balancing algorithm**, while excluding backends that are unhealthy. *(§19.3.1)*

Load shedding and throttling — **Load shedding** is a server rejecting some requests early and cheaply (usually with 503) because accepting them would push it past capacity, regardless of whether each client is within its quota. **Throttling** is slowing a client down, by delaying or rejecting its requests; rate limiting is the quota-based form. **Goodput** is the rate of requests completed *usefully*: correctly and before the client gave up. *(§22.9.1)*

Local and global secondary indexes — A **local** (document-partitioned) index lives in each partition and covers only its rows. A **global** (term-partitioned) index covers all rows and is partitioned by the *indexed value*, so all entries for one value live together [Kleppmann 2017]. *(§14.11.1)*

Log — An append-only, totally ordered sequence of records. Each record is assigned a monotonically increasing integer position called its **offset** when it is appended. Records are never modified in place; readers read sequentially from any offset they choose. *(§31.3.1)*

Logarithm — The logarithm base b of x, written $\log_b x$, is the exponent to which b must be raised to give x: if $b^y = x$ then $y = \log_b x$. *(§4.4.1)*

Log compaction — A retention policy (`cleanup.policy=compact`) that keeps **at least the latest record for each key** and discards older records with the same key, so the partition converges to a snapshot of the latest value per key while preserving the offsets and order of the records that remain. A record with a null value, a **tombstone**, marks a key as deleted and is itself removed after `delete.retention.ms` (default 24 hours). *(§31.10.2)*

Log matching property — If two logs contain an entry with the same index and term, then (1) the entries store the same command, and (2) the logs are identical in all preceding entries. *(§26.7.4)*

Log-structured merge tree (LSM tree) — A storage structure that buffers writes in a sorted in-memory table (the **memtable**), periodically writes it out as an immutable sorted file (a **sorted string table, SSTable**), and merges SSTables in the background (**compaction**) into progressively larger sorted runs organized in **levels**. Introduced by O'Neil, Cheng, Gawlick and O'Neil [O'Neil et al. 1996]; the memtable/SSTable design popularized by Google's Bigtable [Chang et al. 2006] and implemented in LevelDB, RocksDB, Cassandra, HBase, ScyllaDB and many others. *(§13.8.1)*

Log-structured storage — A storage design in which every write — insert, update or delete — is appended to the end of a file, and existing data is never modified in place. Old versions become garbage that a background **compaction** (or merge) process removes by copying live data into new files. *(§13.7.1)*

Long polling — A technique in which the client sends an HTTP request that the server deliberately does not answer until either new data is available or a server-side timeout

T_{hold} expires; on receiving any response, the client immediately sends the next request. *(§9.4.1)*

Low-level design (LLD) — A description of the internals of one component: its modules, classes or functions, data structures, algorithms, database schemas, API request and response formats, error handling and state transitions — at a level of detail from which an engineer can write code. *(§1.5.1)*

Low-level design (LLD) interview — A 45–60-minute interview in which you design the classes, interfaces, relationships and key algorithms of a single component or small application (a parking lot, an LRU cache, a rate limiter), usually ending with working or near-working code for the core classes. It is also called an *object-oriented design* or *machine coding* interview. It contrasts with the high-level design interview (§1.5, Chapter 87), which designs a distributed system out of services and data stores. *(§38.9.1)*

M

Maglev hashing — A consistent-hashing scheme that builds a **lookup table** of prime size M in which every entry names a backend. Each backend has its own pseudo-random **preference permutation** of table positions; backends take turns claiming their next preferred empty position until the table is full. A packet's backend is `table[hash(5-tuple) mod M]`: one array read [Eisenbud et al. 2016]. *(§19.4.8)*

MapReduce — A programming model and execution system in which the user writes two functions: **map**, which turns one input record into zero or more intermediate key–value pairs, and **reduce**, which receives one intermediate key with *all* values emitted for it and produces output. The system handles splitting, scheduling, regrouping by key, and failures [Dean and Ghemawat 2004]. *(§29.4.1)*

Matrix — A rectangular grid of numbers with m rows and n columns (an $m \times n$ matrix). A set of n embeddings of dimension d is naturally stored as an $n \times d$ matrix, one embedding per row. *(§4.9.3)*

Mean and median — The **mean** (arithmetic average) of n values is their sum divided by n. The **median** is the middle value after sorting (the average of the two middle values when n is even); half of the values are at or below it. *(§4.7.1)*

Memory hierarchy — The layered arrangement of storage technologies in a computer, from small, fast and expensive per byte (registers, caches) to large, slow and cheap per byte (disks, tape), in which each level acts as a cache for the level below it. *(§5.5.2)*

Merkle tree (hash tree) — A tree in which every leaf holds the hash of a data block and every interior node holds the hash of its children's hashes. The **root hash** summarizes the entire data set: changing any block changes the root [Merkle 1987]. *(§17.8.1)*

Message authentication code (MAC) — A short tag computed from a message and a secret key, $t = \mathrm{MAC}_K(m)$, such that without K it is infeasible to produce a valid tag for any new message. **HMAC** is the standard construction from a hash function: $\mathrm{HMAC}_K(m) = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel m))$ [Krawczyk et al. 1997]. *(§10.4.6)*

Message, command, event — A **message** is the transport unit a broker carries: headers plus a payload. A **command** is a message that asks one specific receiver to do something (“ResizePhoto”); it may be refused. An **event** is a message stating a fact that has already happened (“PhotoUploaded”); it cannot be refused, only reacted to, and its publisher does not know who reacts [Hohpe and Woolf 2003]. *(§30.6.1)*

Message ordering — **Global (total) ordering**: all consumers observe all messages in one sequence. **Per-key (partial) ordering**: messages that share a key — an account, an order, a device — are processed in send order, while messages with different keys may be processed in any relative order. *(§30.9)*

Message queue (point-to-point channel) — A broker-managed, durable buffer of messages in which each message is delivered to, and processed by, exactly one of the queue’s consumers; once a consumer acknowledges a message, the broker removes it [Hohpe and Woolf 2003]. *(§30.4.1)*

Metadata service — The component that maps names to data: directory entries, file or object attributes (size, owner, permissions, version, checksum) and the locations of each chunk or fragment. It is on the path of every open, create, list and delete, while the data path bypasses it. *(§15.7.1)*

Microservice — A service built around one business capability, owned by one team, deployable independently, and the sole owner of its data, which others reach only through its API or events. Lewis and Fowler describe the style as “an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms,” built around business capabilities and independently deployable [Lewis and Fowler 2014]. *(§34.3.1)*

Microservices — An architectural style that builds one application as a suite of small services, each running in its own process, owning its own data, communicating over lightweight network mechanisms, built around a business capability, and **independently deployable** by the team that owns it [Lewis and Fowler 2014]. *(§33.7.1)*

Mobile push notification — A message sent by an application’s server to a platform-operated push service — Apple Push Notification service (**APNs**) for Apple platforms, Firebase Cloud Messaging (**FCM**) for Android (and optionally iOS and web) — which delivers it, on a best-effort basis, over a single persistent connection that the operating system maintains to that service on behalf of all apps. *(§9.8.1)*

Modular monolith — A monolith whose code is divided into **modules** aligned with business domains, where each module exposes a small **public interface**, hides its internals and data, and may depend only on an explicitly declared, acyclic set of other modules. Boundaries are enforced by tooling, not just convention. *(§33.5.1)*

Monolith — A system whose functionality is built, tested and deployed as **one deployable unit**, typically one process image running as identical replicas and backed by one shared database. “Monolith” describes the *deployment* unit; it says nothing about whether the code inside is well structured. *(§33.4.1)*

Monotonic reads — A client never sees data move backwards: after reading some version, its later reads return that version or newer. *(§14.4.3)*

Multi-core processor — A single chip containing several complete cores, each with its own pipeline and private caches, sharing the last-level cache and memory. *(§5.3.6)*

Multipart upload — A protocol for uploading one large object as independently uploaded **parts**: the client *initiates* an upload and receives an upload ID, uploads numbered parts (in any order, in parallel, retrying any part), then *completes* the upload with the list of part numbers and their ETags, at which point the store assembles the object atomically. Until completion, or after an *abort*, the object is not visible. *(§15.8.1)*

Multi-primary replication — More than one node accepts writes for the same data and replicates to the others (also **multi-leader** or **active–active**). Concurrent uncoordinated changes to the same row create **write conflicts** that must be detected and resolved. *(§14.5.1)*

Mutex (mutual-exclusion lock) — An object with two operations, `lock` and `unlock`, guaranteeing that at most one thread holds it at a time. A thread that calls `lock` while another holds it waits. *(§6.10.1)*

MVCC — A concurrency-control technique in which a write creates a new **version** of a row instead of overwriting it in place, and each transaction reads the versions visible in its **snapshot**: the set of transactions that had committed when the snapshot was taken. Readers therefore never block writers and writers never block readers; only writers of the same row wait for each other. *(§11.14.1)*

N

Narrow and wide dependencies — A **narrow dependency** exists when each partition of the parent RDD is used by at most one partition of the child (`map`, `filter`, `union`, a join of co-partitioned inputs). A **wide (shuffle) dependency** exists when a child partition may depend on many or all parent partitions (`groupByKey`, `reduceByKey`, a join of unpartitioned inputs), which requires a shuffle. *(§29.5.2)*

Natural and surrogate keys — A **natural key** is a candidate key with business meaning (email address, ISBN, country code). A **surrogate key** is a meaningless value generated by the system (an auto-increment integer, a UUID) whose only job is identification. *(§11.4.2)*

Near-real-time (NRT) search — A search system in which newly written documents become searchable after a short, bounded delay (typically one second to a minute) rather than immediately, because making them visible requires creating a new segment and reopening the searcher. *(§16.10.1)*

Network Address Translation (NAT) — A function, usually in a router or gateway, that rewrites the source and/or destination IP addresses (and usually ports) of packets as they pass, keeping a **translation table** so that replies can be rewritten back. The most common form, **source NAT with port translation** (NAPT, often just "NAT"), lets many hosts with private addresses share one public address [RFC 3022]. *(§7.13)*

Network partition — A network fault after which some nodes that are running correctly cannot exchange messages with some other running nodes for a period of time. (This book reserves "partition" alone for a subset of data, §14.6; a network fault is always a **network partition**.) *(§24.7.1)*

NewSQL (distributed SQL) — Relational databases that keep SQL and ACID transactions with serializable or strong isolation while scaling horizontally across nodes and regions, by partitioning data into ranges replicated with a consensus protocol. The term was coined by analyst Matthew Aslett in 2011 [Pavlo & Aslett 2016]. *(§12.11)*

Non-functional requirement (NFR) — A requirement on *how well* the system performs its functions, or on constraints under which it operates, expressed as a measurable target — for example "p99 feed latency ≤ 300 ms at 2,000 RPS" or "monthly availability ≥ 99.9%". NFRs are also called **quality attributes** or, informally, the **"-ilities"** (scalability, availability, durability, …). *(§1.7.1)*

Normal forms (1NF to BCNF) — **1NF:** every column holds a single atomic value (no lists or repeating groups). **2NF:** 1NF, and no non-key column depends on only *part* of a composite candidate key. **3NF:** 2NF, and no non-key column depends on another non-key column (no transitive dependency); formally, for every non-trivial $X \to A$, either X is a superkey or A is part of some candidate key. **BCNF (Boyce–Codd normal form):** for every non-trivial $X \to A$, X is a superkey. BCNF removes 3NF's exception for key columns. *(§11.5.2)*

NoSQL database — A database that does not use the relational table-and-join model as its primary data model and typically trades some combination of cross-record transactions, ad-hoc querying and fixed schemas for horizontal scale-out, flexible schemas or availability during failures. The name is usually read as "not only SQL"; it describes a movement, not a specification. *(§12.3.1)*

NUMA (non-uniform memory access) — A memory architecture in which each processor socket (or each chiplet group within a socket) has its own directly attached memory. A core can access all memory in the machine, but accessing its **local** memory is faster than accessing **remote** memory attached to another socket. *(§5.10)*

O

OAuth 2.0 — An authorization framework that lets a third-party application obtain *limited* access to an HTTP service on behalf of a resource owner, by obtaining an **access token** from an authorization server instead of using the owner's credentials [RFC 6749]. *(§8.14.1)*

Object — A value in a running program that bundles **state** (data, stored in **attributes** or **fields**) with **behavior** (functions that operate on that data, called **methods**). Each object has its own copy of the state. *(§35.3.1)*

Observer — A one-to-many dependency in which a **subject** keeps a list of **observers** and notifies each of them when its state changes, so that the subject does not depend on what the observers do [Gamma et al. 1994]. *(§37.8)*

Offload ratio — The fraction of traffic a CDN serves without contacting the origin. **Request offload** counts requests; **byte offload** counts bytes. Both are a cache hit ratio (§20.3) measured at the boundary between the CDN and the origin. *(§21.5.1)*

Open and closed workload models — In an **open** model, new requests arrive at a rate that is independent of how fast the system answers: users on the internet do not wait for each other. In a **closed** model, a fixed population of N users (or load-generator threads) each sends a request, waits for the response, optionally pauses for a **think time** Z, and only then sends the next. A **partly open** model has sessions that arrive openly but issue several requests each, closed-loop, within the session [Schroeder et al. 2006]. *(§23.5.2)*

OpenAPI Specification (OAS) — A vendor-neutral, machine-readable format (YAML or JSON) for describing HTTP APIs: paths, operations, parameters, request and response bodies (as JSON Schema), status codes and security schemes [OpenAPI Specification]. It grew out of Swagger and is maintained by the OpenAPI Initiative; version 3.1 aligned its schemas with JSON Schema, and version 3.2.0 was released in September 2025. *(§8.16)*

Open/Closed Principle (OCP) — Software entities (classes, modules, functions) should be **open for extension** but **closed for modification**: you should be able to add new behavior by adding new code, without editing code that already works and is already depended upon [Martin 2002]. The principle was first stated by Bertrand Meyer [Meyer 1988]. *(§36.4.1)*

Operating system (OS) — The software layer that manages a computer's hardware (CPU, memory, storage, network) and provides running programs with safe, shared, abstract access to it. The privileged core of the OS that runs with full hardware access is the **kernel**. *(§6.3.1)*

Operation, history — An **operation** is one call by a client on a shared object, such as `write(x, 1)` or `read(x)`. It has an **invocation** event (the client sends the request, at real time t_{inv}) and, if the client hears back, a **response** event (at t_{resp}, carrying the return value). A **history** is the sequence of invocation and response events observed in one execution. An operation without a response is **pending**: it may or may not have taken effect. *(§25.3.1)*

Order of magnitude — A factor of ten. Two quantities differ by k orders of magnitude when their ratio is about 10^k. Reasoning in orders of magnitude means estimating only the *power of ten* of a quantity (is it 10, 1,000 or 1,000,000 requests per second?) because that, not the exact value, decides the architecture. *(§2.10.1)*

Origin shielding — Routing every cache miss for an origin through one designated cache tier (the **shield**), so that the origin sees at most one request per object per freshness period, regardless of how many edge PoPs missed. *(§21.5.3)*

Out-of-order (OoO) execution — A technique in which the core executes instructions as soon as their inputs are ready rather than strictly in program order, while still making the results *appear* in program order (it **retires** them in order). *(§5.3.5)*

OWASP Top 10 — A consensus ranking of the most critical web application security risk categories, published by the Open Worldwide Application Security Project from vulnerability data contributed by testing organizations plus a community survey. This section uses the **2021 edition** [OWASP 2021]. A 2025 edition has since been published; it keeps broken access control at #1, folds SSRF into it, and adds categories for software supply-chain failures and mishandling of exceptional conditions, so the architectural controls below remain applicable. *(§10.7)*

P

PACELC — If there is a **P**artition, how does the system trade off **A**vailability and **C**onsistency; **E**lse, when the system is running normally, how does it trade off **L**atency and **C**onsistency? A system is labeled by its two choices, for example PA/EL or PC/EC. *(§25.9.1)*

Page — The fixed-size unit (typically 4–16 KiB) in which a storage engine reads, writes, caches and logs data. A page on disk is sometimes called a **block**; the engine keeps a copy of a page in memory in a **frame** of its buffer pool. *(§13.3.1)*

Page cache — The kernel's in-RAM cache of file contents, organized in pages. Reads are served from it when possible; writes normally modify pages in it (marking them **dirty**) and return immediately, with the kernel writing them back to storage later. *(§6.6.6)*

Partial failure — A state in which some components of a system have failed while others continue to work, so that an operation may be applied in some places and not others, and the working components cannot reliably tell which components failed. *(§24.8.1)*

Partition key, sort key and item collection — In DynamoDB every **item** (a record of up to 400 KB, as of 2026) has a primary key made of a **partition key** and, optionally, a **sort key**. DynamoDB hashes the partition key to choose the storage partition. All items sharing a partition key form an **item collection**, stored together and ordered by sort key, so a single **Query** can return a contiguous, sorted range of them. *(§12.4.3)*

Partition (shard) — A subset of a dataset's rows, stored and served together, such that every row belongs to exactly one partition. **Partitioning** spreads partitions over nodes; **sharding** is a synonym used mostly for separate database servers. This book uses *partition* and *shard* interchangeably and reserves **network partition** for network splits (§24.7). The columns that choose a row's partition form the **partition key** (or **shard key**). *(§14.6.1)*

Partition skew and salting — **Skew** is an uneven distribution of records across partitions, usually caused by a few very frequent keys (a celebrity user, a null or default value, a bot). **Salting** appends a random suffix from a small range to hot keys so their records spread over several partitions, then removes the suffix in a second, much smaller aggregation (or, for joins, replicates the matching rows of the other side once per salt value). *(§29.6.3)*

Percentile — The q-th percentile of a set of measurements is the value at or below which q% of the measurements fall. Latency percentiles are written **p50** (the median), **p95**, **p99** and **p99.9** (also written p999): the 50th, 95th, 99th and 99.9th percentiles. *(§4.8.1)*

Percentile latency (p50, p95, p99, p99.9) — The latency value below which the given percentage of requests fall. If p99 latency is 300 ms, 99% of requests completed in 300 ms or less and 1% took longer. p50 is the median. *(§1.7.1)*

Percolator transaction — A snapshot-isolated, multi-row, multi-table transaction built entirely by a client library on top of a store that offers only single-row atomic updates and multiple timestamped versions per cell. Ordering comes from a central **timestamp oracle**; atomicity comes from making one row's lock, the **primary lock**, the commit point for the whole transaction [Peng and Dabek 2010]. *(§28.7.1)*

Performance engineering — The practice of measuring, explaining and improving the **latency**, **throughput** and **resource efficiency** of a system against explicit targets, using repeatable methods rather than intuition. Its outputs are diagnoses (why is it slow?),

changes (make it faster or cheaper) and predictions (how much capacity will we need?). *(§23.3.1)*

PKCE (Proof Key for Code Exchange, “pixy”) — An extension in which the client creates a random secret, the **code verifier**, sends only its SHA-256 hash, the **code challenge**, with the authorization request, and must present the verifier when redeeming the authorization code. An attacker who intercepts the code cannot redeem it without the verifier [RFC 7636]. *(§8.14.3)*

Point of presence (PoP) — A CDN site in one metropolitan area or interconnection facility, containing load balancers, caching proxy servers with RAM and SSD storage, and connections to local networks. Large CDNs operate hundreds of PoPs; some also place servers inside Internet service provider (ISP) networks. *(§21.3.2)*

Poison message — A message that fails processing every time it is delivered — because it is malformed, refers to data that does not exist, or triggers a bug — and that, if redelivered indefinitely, wastes consumer capacity and (in ordered lanes) blocks every message behind it. *(§30.10)*

Polyglot persistence — Using several different data-storage technologies within one system, each chosen for the access pattern it serves best, rather than forcing all data into one database [Fowler 2011; Sadalage & Fowler 2012]. *(§12.13.1)*

Polymorphism — The ability of one piece of code to work with values of different types. **Subtype polymorphism** means code written against a type also works with any of its subtypes, and each object responds to a method call with its own implementation. **Dynamic dispatch** is the run-time mechanism that picks which implementation runs, based on the object’s actual class rather than the declared type of the variable. *(§35.6.1)*

Power of two choices (P2C) — For each request, pick **two** backends uniformly at random and send the request to the less loaded of the two (by in-flight count, EWMA latency or another load signal). *(§19.4.4)*

Presumed abort (PA) — A 2PC variant in which a coordinator that has no information about a transaction answers inquiries with “abort”. Aborts therefore need no forced coordinator record, no forced participant record and no acknowledgments. *(§28.4.5)*

Primary–backup replication — One replica, the **primary**, receives all requests, executes them and propagates the result to one or more **backups**. If the primary fails, a backup is **promoted** to primary. It is also called **passive replication** [Alsberg and Day 1976; Budhiraja et al. 1993]. *(§27.4.1)*

Primary key in CQL — In Cassandra Query Language, `PRIMARY KEY ((a, b), c, d)` declares `(a, b)` as the **partition key** — hashed to choose the replicas and defining what is

stored together — and `c, d` as **clustering columns**, which order rows *within* the partition on disk. *(§12.6.2)*

Priority inversion — A situation in which a high-priority thread is blocked waiting for a lock held by a low-priority thread, while a medium-priority thread, which needs no lock, preempts the low-priority one, so the high-priority thread effectively runs at the lowest priority. *(§6.11.5)*

Probability — A number between 0 and 1 that measures how likely an event is: 0 means impossible, 1 means certain. If an experiment is repeated many times, the probability of event A, written $P(A)$, is the long-run fraction of repetitions in which A happens. *(§4.6.1)*

Process — A running instance of a program: its code, its private **virtual address space**, one or more threads of execution, and the OS resources it holds (open files, sockets, signal handlers, credentials). *(§6.3.2)*

Process control block (PCB) — The kernel data structure that records everything needed to stop a process and later resume it exactly where it left off. In Linux it is `struct task_struct`. *(§6.3.3)*

Process state — The scheduling status of a process (or thread): whether it is running on a CPU, ready and waiting for a CPU, or blocked waiting for an event. *(§6.3.4)*

Producer–consumer — Producers put work items into a shared **bounded** buffer and consumers take them out, decoupling the rate at which work is created from the rate at which it is processed; when the buffer is full, producers block or are rejected, which is **backpressure** (§22.10). *(§37.16.1)*

Profiler — A tool that attributes a resource (CPU time, wall-clock time, allocated bytes, lock wait time) to the code locations that consumed it. A **sampling profiler** interrupts the program at a fixed frequency and records the current call stack; a function's share of samples estimates its share of time. An **instrumenting** (deterministic, tracing) profiler inserts hooks at every function entry and exit and records exact call counts and durations. *(§23.4.1)*

Protocol — A set of rules two parties agree on for exchanging messages: message formats, the meaning of each field, and the actions each side takes on sending, receiving or timing out. A **protocol stack** is a set of protocols layered so that each uses the service provided by the layer below and offers a service to the layer above. *(§7.3.1)*

Proxy — An intermediary that terminates a client's connection and opens its own connection onward, relaying requests and responses. A **forward proxy** acts on behalf of **clients** and is chosen by them (or their network administrator). A **reverse proxy** acts on behalf of **servers** and is chosen by the site operator; clients usually do not know it exists. *(§7.12)*

Public key infrastructure (PKI) — The set of roles, policies and systems that bind public keys to identities. A **certificate** (X.509) is a document stating "this public key belongs to this name (for example `api.example.com`), valid from date A to date B", signed by a **certificate authority (CA)**. A client trusts a certificate if it can build a **chain** of valid signatures from it to a **root CA** certificate in its local **trust store**. *(§10.5)*

Publish/subscribe (pub/sub) — A messaging style in which a publisher sends a message to a named **topic** without knowing its receivers, and the broker delivers a copy to every **subscription** registered on that topic [Hohpe and Woolf 2003; Eugster et al. 2003]. *(§30.5.1)*

Q

Queueing system — A model in which **jobs** (requests, messages, packets) arrive at a **service center**, wait in a **queue** if all **servers** are busy, receive service, and depart. The **arrival rate** λ is the average number of jobs arriving per second; the **service rate** μ is the average number of jobs one server can complete per second when it is never idle, so the average **service time** is $S = 1/\mu$. *(§4.10.1)*

Quorum — A set of nodes whose agreement is sufficient for an operation to take effect. A **majority quorum** in a cluster of N voting nodes is any set of more than $N/2$ nodes. The essential **intersection property**: any two majority quorums share at least one node. *(§26.4.1)*

R

Race condition — A bug in which the result of a program depends on the relative timing or interleaving of concurrent operations on shared state. A **data race** is the specific case where two threads access the same memory location concurrently, at least one access is a write, and nothing orders them. *(§6.9)*

Raft term — A logical clock that divides time into consecutive, numbered periods. Each term begins with an election; if a candidate wins, it is the **leader** for the rest of the term, and there is at most one leader per term. Every RPC carries the sender's term; a node that sees a higher term immediately adopts it and becomes a follower; a request with a lower term is rejected. *(§26.7.1)*

Rate limiting — Enforcing an upper bound on how many requests (or how much work) a given **client identity** may submit to a resource over time. Requests beyond the bound are rejected (or, less often, delayed). The bound is usually stated as a **rate** (for example 100 requests per minute) plus a **burst** allowance. *(§22.3.1)*

Read repair and anti-entropy — **Read repair** fixes stale replicas discovered while serving a read: the coordinator writes the newest version back to replicas that returned older ones. **Anti-entropy** is a background process that compares whole replicas, typically with Merkle trees, and copies the differences [DeCandia et al. 2007; Demers et al. 1987]. *(§27.13.1)*

Read-through — The application asks only the cache; on a miss the *cache* loads the value from the database through a configured loader, stores it and returns it. *(§20.7.3)*

Read-write lock — A lock that allows any number of concurrent readers or exactly one writer, never both; it improves throughput when reads greatly outnumber writes and each read holds the lock long enough for contention to matter (§6.10.5). *(§37.16.4)*

Read-your-writes consistency — After a client writes a value, its own later reads return that value or a newer one. Other clients get no such promise. *(§14.4.3)*

Rebalancing — Moving partitions, or parts of them, between nodes so that data and load stay even as nodes join, leave or fail and as data grows, while staying available and moving no more data than necessary. *(§14.9.1)*

RED method — For every **service**, monitor **R**ate (requests per second), **E**rrors (failed requests per second) and **D**uration (the distribution of request latency, as a histogram from which percentiles are read). Tom Wilkie proposed it while at Weaveworks as a service-level complement to USE [Wilkie 2018]. *(§23.3.3)*

Reduce-side (sort-merge) join and broadcast (map-side) join — In a **reduce-side join**, both inputs are mapped to `(join key, tagged record)`, shuffled so that all records with the same key meet at one reducer, sorted, and combined there. In a **broadcast join**, the smaller input is copied in full to every map task, loaded into an in-memory hash table, and each record of the larger input is joined locally by lookup, with no shuffle of the larger input. *(§29.4.4)*

Register — A tiny, extremely fast storage slot inside a core, typically 64 bits wide for general-purpose registers. x86-64 exposes 16 general-purpose integer registers to programs; ARMv8-A (AArch64) exposes 31. Reading a register costs essentially nothing beyond the instruction that uses it. *(§5.3.2)*

Relation — A **relation** is a named set of **tuples** that share the same **attributes**. Each attribute has a name and a **domain** (the set of allowed values, such as 64-bit integers). In SQL a relation is a **table**, a tuple is a **row** and an attribute is a **column**. The relation's name and attributes together form its **schema**; the set of rows at a moment is its **instance**. *(§11.3.1)*

Relevance score — A number computed for each (query, document) pair that estimates how well the document satisfies the query. A **ranking function** computes it; the engine

returns the k highest-scoring documents. *(§16.5.1)*

Reliability, MTBF, MTTR and failure rate — **Reliability** $R(t)$ is the probability that a component operates without failure for a period of length t. The **mean time between failures (MTBF)** is the average operating time from the end of one repair to the next failure (for non-repairable parts the same quantity is called **MTTF**, mean time to failure). The **mean time to repair (MTTR)** is the average time from failure to restored service, including detection, diagnosis and repair. The **failure rate** λ is the average number of failures per unit of operating time. *(§4.15.1)*

Remote procedure call (RPC) — A mechanism that lets a program call a function in another process, usually on another machine, with syntax that resembles a local function call. Generated **stubs** serialize the arguments, send them over the network, and deserialize the result [Birrell and Nelson 1984]. *(§8.8.1)*

Replica recovery — Returning a replica to full membership after a crash or replacement, by **log catch-up** (replaying missed log entries), **snapshot plus log** (installing a copy of the state, then replaying from the snapshot's position) or **rebuild** (copying all data from peers). *(§27.12.1)*

Replicated state machine (RSM) — A service implemented as a deterministic state machine that is copied onto several nodes. Each node applies the same sequence of commands, taken from a replicated **log**, in the same order, so every node passes through the same sequence of states and produces the same outputs [Schneider 1990]. *(§26.3.2)*

Replication — Keeping a copy of the same data on more than one node. Each copy is a **replica**. In **primary–replica replication**, exactly one node, the **primary** (called the "master" in older documentation), accepts writes and streams its changes to the **replicas**, which apply them in the same order and serve reads. *(§14.3.1)*

Replication lag — The delay between a change committing on the primary and becoming visible on a replica, measured in seconds, bytes of log or transactions. *(§14.4.2)*

Replication log formats — What the log describes: the SQL statements executed (**statement-based**), the changed rows (**row-based**), the storage engine's page-level byte changes (**physical**, or WAL shipping), or decoded, engine-independent row changes per table (**logical**). *(§14.4.5)*

Replication modes — In **synchronous replication**, the primary reports a commit only after at least one designated replica has durably received (or applied) it. In **asynchronous replication**, it reports success after its local commit and ships the change afterwards. **Semi-synchronous replication**, MySQL's middle ground, waits until a replica has *received and logged* the change and falls back to asynchronous mode after a timeout. *(§14.4.1)*

Replication strategy — The rule that decides *which* replicas may accept a write, *in what order* the replicas apply writes, and *when* a write counts as done. The five families are **primary–backup**, **chain replication**, **multi-leader**, **leaderless** and **consensus-based** replication. *(§27.3.1)*

Replication topology — The graph along which leaders forward writes to each other: a **circular** ring, a **star** with a hub, or **all-to-all**. *(§27.6.1)*

Repository — An object that mediates between the domain model and data storage, presenting stored aggregates as an in-memory collection with methods such as `add`, `get` and domain-specific finders, so that domain code never sees SQL, ORM sessions or storage APIs [Fowler 2002; Evans 2003]. *(§37.13)*

Request–response lifecycle — The complete sequence of network and processing steps between a client issuing a request and receiving the full response, including name resolution, connection establishment, routing through intermediaries, application processing, data access and the return path. *(§8.5.1)*

Requirement — A statement of what the system must do (a **functional requirement**, e.g. “a customer can cancel an order before the restaurant accepts it”) or how well it must do it (a **non-functional requirement**, e.g. “99% of order placements complete within 800 ms”). A requirement is **testable** if you could write an automated check or a measurement that proves it is met. *(§2.5.1)*

Reservoir sampling — An algorithm that maintains a uniform random sample of k items from a stream whose length n is not known in advance, using $O(k)$ memory and a single pass: after any number of items, every item seen so far is in the sample with probability k/n [Vitter 1985]. *(§17.12.1)*

Resilient Distributed Dataset (RDD) — A read-only, partitioned collection of records that can be created only by deterministic operations (**transformations** such as `map`, `filter`, `join`) on stable storage or on other RDDs. Each RDD records its **lineage**, the graph of transformations that produced it, so a lost partition can be recomputed from its parents instead of being restored from a replica [Zaharia et al. 2012]. *(§29.5.1)*

REST (Representational State Transfer) — An architectural style for distributed hypermedia systems, defined by Roy Fielding in his 2000 doctoral dissertation as a set of constraints on how components interact [Fielding 2000]. In everyday usage, a “REST API” is an HTTP API that models its domain as **resources** identified by URIs and manipulated with the standard HTTP methods. *(§8.6.1)*

Round robin (RR) — Send the k-th request to backend $k \bmod N$. **Weighted round robin (WRR)** gives backend i an integer weight w_i and sends it a share $w_i / \sum_j w_j$ of requests in each cycle. *(§19.4.1)*

Routing and forwarding — **Forwarding** is the per-packet act of choosing an outgoing interface for a packet based on its destination address and the **forwarding table**. **Routing** is the control-plane process of building those tables by exchanging reachability information with other routers. *(§7.5.1)*

RUM conjecture — For any access method, bounding two of **R**ead overhead, **U**pdate overhead and **M**emory (space) overhead forces the third to be worse: you can optimize for at most two [Athanassoulis et al. 2016]. *(§13.11.1)*

S

Safe method — A method whose semantics are read-only: the client does not request, and does not expect, any state change on the server [RFC 9110 §9.2.1]. Logging or metrics side effects are allowed. *(§8.6.3)*

Saga — A long-lived transaction written as a sequence of local transactions $T_1, T_2, \ldots, T_n$, each committed independently, where each T_i has a **compensating transaction** C_i that semantically undoes it. Either $T_1 \ldots T_n$ all complete, or $T_1 \ldots T_j, C_j \ldots C_1$ run [Garcia-Molina and Salem 1987]. *(§28.6)*

Scalability — The ability of a system to handle a growing workload by adding resources, with throughput rising roughly in proportion to the resources added and latency staying within its objective. A system is *not* scalable for a workload if adding resources yields little or no extra throughput, or if the cost per unit of work rises sharply. *(§18.3.1)*

Scale cube — A model by Martin Abbott and Michael Fisher that classifies every scaling technique along three independent axes [Abbott and Fisher 2015]: the **X axis**, cloning identical copies of a service or its data behind a load balancer; the **Y axis**, splitting by function, service or resource so that each part handles different work; and the **Z axis**, splitting by customer, requester or other key so that each part handles the same work for a subset of the data or users. *(§18.10.1)*

Scaling bottleneck — The resource that limits throughput as load or node count grows: the resource with the largest service demand per unit of capacity (§2.6). For a horizontally scaled system, it is usually a resource that *all* nodes share and that cannot itself be scaled out by adding application nodes. *(§18.7.1)*

Search engine (as a data store) — A system such as Elasticsearch, OpenSearch or Apache Solr that stores JSON documents and builds an **inverted index** — a map from each term to the list of documents containing it — so it can answer full-text, relevance-ranked, faceted and fuzzy queries. *(§12.9)*

Secret — Any value that grants access and must be kept confidential: database passwords, API keys, OAuth client secrets, TLS and signing private keys, encryption keys,

webhook HMAC keys. **Secrets management** is the practice of storing, distributing, rotating, auditing and revoking secrets so that they are available to the workloads that need them and to nobody else. *(§10.8)*

Security property — A statement about what must remain true about a system even when an adversary is actively trying to violate it. The classic triad is **confidentiality** (only authorized parties can read data), **integrity** (data and behavior cannot be changed without detection) and **availability** (authorized parties can use the system when they need it), often abbreviated **CIA**. System designers add **authentication** (knowing who is acting), **authorization** (deciding what that actor may do) and **non-repudiation** (an actor cannot credibly deny an action they took). *(§10.3.1)*

Segment — A self-contained, immutable mini-index (its own dictionary, postings, stored fields, norms and doc values) covering a subset of documents. A Lucene index is a set of segments plus a **commit point** file (`segments_N`) listing which segments are live; a search runs over every live segment and combines the results. *(§16.6.1)*

Semaphore — An integer counter with two atomic operations introduced by Dijkstra: **wait** (P, or *acquire*) decrements it and blocks if the result would be negative; **signal** (V, or *release*) increments it and wakes a waiter [Dijkstra 1965]. *(§6.10.2)*

Sequence diagram — An interaction diagram that shows objects or participants as vertical **lifelines** and the **messages** they exchange as horizontal arrows ordered top to bottom in time. **Execution specifications** (activation bars) show when a participant is busy, and **combined fragments** (`alt`, `opt`, `loop`, `par` and others) show conditional, optional, repeated and parallel behavior [OMG 2017]. *(§38.4.1)*

Sequential consistency — A history is **sequentially consistent** if there is a single total order of all operations that is a legal sequential history of the objects and that respects each client's **program order** (the order in which that client issued its own operations). In Lamport's words, "the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program" [Lamport 1979]. *(§25.4.1)*

Serialization (encoding) — Translating an in-memory data structure into a sequence of bytes for storage or transmission; **deserialization (decoding)** is the reverse [Kleppmann 2017, ch. 4]. *(§8.9.1)*

Serverless — An architecture built from **functions as a service (FaaS)**, short-lived functions that the platform runs in response to events, combined with **managed services** (databases, queues, object storage, authentication) that the provider operates, such that the team provisions no servers, capacity scales automatically including **to zero**, and billing is per request and per unit of execution time rather than per running machine [Jonas et al. 2019]. *(§33.9.1)*

Server-Sent Events (SSE) — A W3C/WHATWG-standardized mechanism in which a client opens one long-lived HTTP request and the server streams a sequence of UTF-8 text events down the response body using the `text/event-stream` format; browsers expose it through the `EventSource` API, which parses events and reconnects automatically, resuming from the last received event ID [WHATWG HTML §9.2]. *(§9.5.1)*

Server-side rendering (SSR) — The server produces complete HTML for each request; the browser displays it immediately and then loads JavaScript to make it interactive (**hydration**). *(§8.4.2)*

Service discovery — The mechanism by which a client finds the current network locations of healthy instances of a service. A **service registry** stores instances, locations and health; a **lookup** path lets a client (client-side discovery) or a router (server-side discovery) query it [Richardson 2018]. *(§34.7)*

Service mesh — An infrastructure layer for service-to-service communication. Its **data plane** is a set of proxies through which all service traffic passes (a **sidecar** per workload instance, or shared per-node proxies); its **control plane** configures the proxies, issues certificates and collects state. It provides traffic management, security (mutual TLS, authorization) and telemetry without application changes [Istio Docs: Architecture; Linkerd Docs: Architecture]. *(§34.10.1)*

Service-oriented architecture (SOA) — An enterprise integration style, prominent from roughly 2000 to 2010, in which business capabilities are exposed as coarse-grained, reusable **services** with formal contracts, typically SOAP web services described in WSDL, connected through an **enterprise service bus (ESB)** that routes, transforms and orchestrates messages between them. *(§33.6.1)*

Session — Server-recognized state that links a series of requests to one authenticated user (or anonymous visitor) over time, typically identified by a cookie. *(§8.10.2)*

Session affinity (sticky sessions) — A balancer policy that sends all requests of one client session to the same backend, identified by a cookie, the client's IP address or a request key. *(§19.6)*

Session guarantees — **Read-your-writes (RYW):** if a session writes w and later reads, the read reflects w (or a later write). **Monotonic reads (MR):** if a session reads a value reflecting a set of writes, later reads reflect at least that set; the session never sees time go backwards. **Monotonic writes (MW):** a session's writes are applied at every replica in the order the session issued them. **Writes-follow-reads (WFR):** if a session reads a value produced by write w_1 and then writes w_2, every replica applies w_1 before w_2 (the new write is ordered after what the session saw). *(§25.6.1)*

Set-associative cache — A cache divided into S sets of W slots (**ways**). A line at a given address may be stored in any of the W ways of exactly one set. $W = 1$ is a **direct-mapped**

cache; a cache with a single set holding every line is **fully associative**. *(§5.4.4)*

Short polling — A technique in which the client sends an ordinary HTTP request at a fixed interval T to ask whether new data exists; the server answers immediately, either with the new data or with an empty response, and the connection's request ends. *(§9.3.1)*

Shuffle — The all-to-all redistribution of intermediate data between two stages, in which every task of the producing stage writes its output partitioned by key and every task of the consuming stage reads its partition from every producer. It costs serialization, local disk writes and reads, and network transfer, and it is a barrier between stages. *(§29.6.2)*

SIMD (single instruction, multiple data) — CPU instructions that apply the same operation to several data elements packed into one wide **vector register** at once — for example, adding eight pairs of 32-bit floats in a single instruction. *(§5.11.1)*

Simple factory — A function or static method that takes a description of what is wanted (a string, an enum, a config value) and returns an instance of the matching concrete class, behind a common interface. It is an idiom rather than a GoF pattern, but it is the most common factory in practice. *(§37.3.1)*

Simultaneous multithreading (SMT) — A technique in which one physical core runs two (occasionally more) instruction streams at once, interleaving them through the same execution units. Intel markets its implementation as **Hyper-Threading Technology**. Each stream appears to the operating system as a separate **logical CPU** (or **hardware thread**). *(§5.3.6)*

Single-decree (basic) Paxos — A consensus algorithm in which **proposers** suggest values, **acceptors** vote on them and **learners** find out the decided value. A value is **chosen** when a majority of acceptors has accepted it with the same proposal number. Paxos guarantees safety under any message delays, loss or reordering, and makes progress whenever one proposer can run both phases without interference from another. *(§26.6.1)*

Single-page application (SPA) / client-side rendering (CSR) — The server sends a nearly empty HTML shell plus a JavaScript bundle; the JavaScript calls JSON APIs and builds the page in the browser. Navigation afterwards happens without full page loads. *(§8.4.2)*

Single Responsibility Principle (SRP) — A module should be responsible to one, and only one, **actor**, where an actor is a group of people (a role, a team, a stakeholder) who request changes to the module for the same reasons [Martin 2017]. The earlier formulation is "a class should have only one reason to change" [Martin 2002]. *(§36.3.1)*

Singleton — A class that guarantees it has exactly one instance within a scope and provides a global access point to it [Gamma et al. 1994]. The scope is almost always one process, a fact that matters greatly in distributed systems. *(§37.6)*

Skip list — An ordered linked list augmented with extra levels of "express" pointers. Each node is promoted to the next level with probability p (typically 1/2 or 1/4), so level i contains about a fraction p^i of the nodes. Search, insert and delete take expected $O(\log n)$ time [Pugh 1990]. *(§17.7.1)*

Sliding window counter — A limiter that keeps fixed-window counters for the current and previous windows and estimates the count over the last W seconds by weighting the previous window's count by the fraction of it that still overlaps the sliding interval. *(§22.7.2)*

Sliding window log — A limiter that stores the timestamp of every admitted request per key and admits a new request only if fewer than L timestamps fall in the interval $(t - W, t]$. *(§22.7.1)*

Sloppy quorum and hinted handoff — In a **sloppy quorum**, a write that cannot reach W of a key's N home replicas is accepted by the next healthy nodes on the ring instead, and still counts toward W. Each stand-in stores the write with a **hint** naming the intended home replica and **hands it off** when that replica recovers [DeCandia et al. 2007]. *(§27.9.1)*

Slotted page — A page layout in which a **header** is followed by an array of **slots** (also called line pointers or item IDs) that grows from the front, while variable-length **records** (tuples) are packed from the back. Each slot stores the offset and length of one record; free space sits in the middle. *(§13.3.2)*

Software architecture — "The set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both" [Bass et al. 2021]. The international standard for architecture description defines it as the fundamental concepts or properties of a system in its environment, embodied in its elements, relationships, and in the principles of its design and evolution [ISO/IEC/IEEE 42010]. *(§1.4.1)*

Solid-state drive (SSD) — A storage device with no moving parts that stores data in **NAND flash** memory cells, managed by an on-board controller running a **flash translation layer** (FTL). *(§5.6.2)*

Spaced repetition — Reviewing material at increasing intervals, so that each review happens around the time you would otherwise begin to forget it. The underlying finding, that spacing study sessions apart beats massing them together, is the **spacing effect**. *(§3.4.3)*

Speedup — The ratio of the time a job takes on one processor (or node) to the time it takes on N: $S(N) = T(1)/T(N)$. Perfect, **linear** speedup is $S(N) = N$. *(§4.12.1)*

Split brain — A failure in which two or more parts of a cluster each believe they are the authoritative primary (or leader) and accept conflicting writes, so the system's state diverges. *(§26.12.1)*

SQL — The Structured Query Language, standardized by ANSI/ISO, with sublanguages for defining schema (DDL: `CREATE`, `ALTER`, `DROP`), manipulating data (DML: `SELECT`, `INSERT`, `UPDATE`, `DELETE`) and controlling transactions (`BEGIN`, `COMMIT`, `ROLLBACK`). Every major database implements the core standard plus its own dialect. *(§11.6.1)*

State — An object delegates its behavior to a separate state object and changes behavior by swapping that state object, so it appears to change its class; each state class handles the transitions that are legal from it [Gamma et al. 1994]. *(§37.15.1)*

Stateful service — A service whose instances hold data that must survive between requests and that clients or other instances depend on: a database, a cache node, a WebSocket server holding live connections, or an application server keeping sessions in memory. *(§18.5.1)*

Stateless service — A service that keeps no client-specific data in its own memory between requests. Every request carries, or lets the service look up from a shared store, everything needed to process it. *(§8.3.2)*

State machine diagram — A behavior diagram that shows the **states** one object (or protocol) can be in and the **transitions** between them. A transition is labeled `trigger [guard] / effect`: the **trigger** is the event, the **guard** a Boolean condition that must be true for the transition to fire, and the **effect** an action executed during the transition. A state may declare **entry**, **exit** and **do** behaviors, run on entering, on leaving, and while inside the state [OMG 2017]. *(§38.6.1)*

Static site generation (SSG) — HTML is produced at build time and served as static files, usually from a CDN (Chapter 21). Pages that change per user cannot be fully static. *(§8.4.2)*

Storage tier and lifecycle policy — A **storage tier** (storage class) is a price/performance point for the same data: **hot** (frequent access, highest storage price, no retrieval fee), **warm** or infrequent access (cheaper storage, per-GB retrieval fee, minimum storage duration), **cold** (cheaper again, slower or costlier retrieval) and **archive** (cheapest storage, retrieval in hours). A **lifecycle policy** is a set of rules, usually by prefix, tag or age, that automatically transitions objects between tiers and expires (deletes) them. *(§15.9)*

Strangler fig application — A migration strategy in which a new system is grown around the edges of an old one, taking over one capability at a time while the old system keeps running, until the old system can be retired. Fowler named it after strangler fig

vines, which grow around a host tree until they stand on their own [Fowler 2004]. *(§34.13.1)*

Strategy — A family of interchangeable algorithms, each behind a common interface, so that a client can be configured with one at run time without changing its code [Gamma et al. 1994]. *(§37.7)*

Stream processing, event time and processing time — **Stream processing** runs a long-lived computation over an **unbounded** dataset, one that grows forever, producing results incrementally. Each record has an **event time**, when the event actually happened (stamped by the device or service that produced it), and a **processing time**, when the system observes it. The difference is the record's **event-time skew**, which is variable and unbounded in the worst case. *(§29.8.1)*

STRIDE — A mnemonic for six threat categories, developed at Microsoft in 1999 by Loren Kohnfelder and Praerit Garg and popularized by Shostack [Shostack 2014]: **S**poofing, **T**ampering, **R**epudiation, **I**nformation disclosure, **D**enial of service and **E**levation of privilege. Each category is the violation of one security property. *(§10.3.4)*

Superscalar execution — A core that can issue and complete more than one instruction per cycle by having several execution units working in parallel. *(§5.3.5)*

Symmetric encryption — Encryption in which the same secret key is used to encrypt and decrypt. A **block cipher** such as AES (Advanced Encryption Standard) encrypts fixed 128-bit blocks under a 128-, 192- or 256-bit key; a **mode of operation** turns the block cipher into a scheme for messages of any length. *(§10.4.2)*

Synchronous and asynchronous inter-service communication — In **synchronous** communication the caller sends a request (REST, gRPC) and cannot complete its work until the response arrives, so caller and callee must be available at the same time. In **asynchronous** communication the sender publishes a message (event or command) to a broker or log and continues; the receiver processes it later (§30.3). *(§34.8.1)*

Synchronous communication — An interaction in which the sender waits for the receiver to process a request and return a response before it continues; both parties must be running and reachable at the same time. *(§30.3.1)*

System — A set of interacting **components** (programs, machines, data stores, networks and people) that together deliver a capability no single component delivers alone. Components interact only through **interfaces**: agreed formats and protocols for exchanging requests, responses and data. *(§1.3.1)*

System boundary — The line that separates what you are designing and are responsible for from everything outside it: users, other teams' systems and third-party services. You control what is inside. You can only *depend on* what is outside. *(§2.4.1)*

System call (syscall) — A controlled entry from user mode into the kernel to request a privileged service, such as reading a file, sending on a socket, allocating memory or creating a process. Linux x86-64 has a few hundred system calls, each identified by a number [Linux man-pages: syscalls(2)]. *(§6.7)*

System design — The process of deciding the components of a software system, their responsibilities, their interfaces, how data flows and is stored between them, and how they are deployed — so that the system meets its functional requirements *and* its non-functional requirements (performance, scalability, availability, durability, security, cost and others) under stated assumptions about load and failure. The output of the process is also called the *system design*: a set of decisions, diagrams and documents. *(§1.3.2)*

System design interview — A 45–60-minute interview in which a candidate designs a system from an open-ended prompt (for example, “design a photo-sharing service”) in conversation with an interviewer, who evaluates the candidate’s process, technical judgment and communication rather than a single correct answer. *(§1.11.1)*

System model — A precise statement of what a distributed algorithm may assume about (1) **timing**: bounds on message delay, processing speed and clock drift, and (2) **faults**: how nodes and links may misbehave. Proofs of correctness hold only within the stated model. *(§24.6.1)*

T

TCP — A connection-oriented transport protocol that provides a reliable, ordered, full-duplex **byte stream** between two endpoints identified by (IP address, port) pairs. It achieves reliability with sequence numbers, acknowledgments and retransmission; it protects a slow receiver with **flow control**; and it protects the network with **congestion control**. *(§7.7.1)*

Template Method — A method in a base class that defines the skeleton of an algorithm as a fixed sequence of steps, some of which are abstract **primitive operations** or overridable **hooks** that subclasses supply, so subclasses vary steps without changing the sequence [Gamma et al. 1994]. *(§37.15.2)*

Thread — An independent sequence of execution within a process, with its own program counter, registers and stack, sharing the process’s address space, heap, globals and open files with the process’s other threads. *(§6.3.5)*

Thread pool — A fixed or bounded set of worker threads that take tasks from a queue and execute them, so the cost of creating threads is paid once and the number of concurrent tasks is capped [Goetz et al. 2006]. *(§37.16.2)*

Threat model — A structured description of a system, the **assets** in it that are worth protecting, the **adversaries** who might attack it and their capabilities, the **threats** (ways a security property could be violated), and the **mitigations** chosen for each threat [Shostack 2014]. *(§10.3.2)*

Three-phase commit (3PC) — A commit protocol that inserts a **pre-commit** phase between voting and committing, so that no participant can commit until every operational participant knows that all voted yes. Under a synchronous system model with reliable failure detection and no network partitions, it lets the surviving participants always reach a decision without the failed coordinator [Skeen 1981]. *(§28.5.1)*

Time-of-day clock and monotonic clock — A **time-of-day** (wall-clock) clock returns the current date and time, for example as seconds since the Unix epoch (Linux `CLOCK_REALTIME`, Python `time.time()`). It is synchronized with other machines and *can jump* forward or backward when corrected. A **monotonic** clock (Linux `CLOCK_MONOTONIC`, Python `time.monotonic()`) only moves forward at a steady rate from an arbitrary starting point. It is meaningless across machines but correct for measuring durations on one machine. *(§24.9.1)*

Timeout and deadline — A **timeout** is a relative duration after which a caller stops waiting for one operation (for example 300 ms). A **deadline** is an absolute point in time by which the *whole* request must complete; it is passed along the call chain so that every downstream call knows how much time remains. *(§22.11)*

Time-series database (TSDB) — A database optimized for sequences of timestamped measurements. Data is organized into **series**, each identified by a metric name plus a set of **labels** (tags) such as `{host="web-3", region="eu"}`, and each series is an append-only list of **samples** `(timestamp, value)`. *(§12.8.1)*

Time to live (TTL) — The maximum time an entry may be served after it was stored. After the TTL the entry is treated as absent (or, with stale-while-revalidate, as stale). Redis removes expired keys lazily when they are accessed and actively by periodically sampling keys that have a TTL [Redis Docs: EXPIRE]. *(§20.8)*

TinyLFU — An **admission policy**: when the cache is full and a new candidate arrives, admit it only if its estimated access frequency exceeds that of the victim the eviction policy would remove. Frequencies are estimated by a small approximate sketch (a Count-Min sketch with 4-bit counters, §17.6) over a sample of recent requests, and **aged** by halving every counter after a fixed number of requests [Einziger, Friedman & Manes 2017]. *(§20.9.3)*

Token bucket — A rate limiter that keeps a counter of **tokens**, refilled continuously at **rate** r up to a maximum **capacity** (burst size) b. Each request consumes one token (or a cost c); a request that finds too few tokens is rejected. The long-run admitted rate is at most r, and up to b requests may be admitted at once. *(§22.4.1)*

Topic, partition and record — A **topic** is a named category of records (for example `orders`). A topic is split into **partitions**, each an independent, ordered, replicated log with its own offsets starting at 0. A **record** carries an optional **key**, a **value** (opaque bytes), a **timestamp** and optional **headers**. Kafka guarantees order *within* a partition, not across partitions of a topic. *(§31.5.1)*

Torn write (torn page) — A page write that is interrupted by a crash so that part of the page on disk is new and part is old. It arises because the database page (8 or 16 KiB) is larger than the unit the device or file system writes atomically (commonly 4 KiB or 512 bytes). *(§13.12.2)*

Trace, span and context propagation — A **trace** records one request's path through a distributed system under a unique **trace ID**. It is a tree of **spans**, each recording one unit of work (an inbound request, an outbound call, a query) with service, operation, start time, duration, status, its span ID and its **parent span ID**. **Context propagation** passes the trace ID and current span ID from caller to callee (in HTTP headers or message metadata) so the callee's spans join the trace [Sigelman et al. 2010; W3C Trace Context]. *(§34.9.1)*

Trade-off — A situation where improving one desired property of a system makes another worse. Examples: lower latency at higher cost, stronger consistency at lower availability during network partitions, faster delivery at higher operational risk. Every design decision is a trade-off. If you cannot name what an option makes worse, you have not understood it yet. *(§2.8.1)*

Transaction — A sequence of reads and writes that the database treats as one logical unit: either all of its effects become permanent together (**commit**) or none of them do (**abort** or **rollback**), and concurrently running transactions are isolated from each other's intermediate states. *(§11.11.1)*

Transactional outbox — A pattern in which a service writes the events it wants to publish into an **outbox table** in its own database, in the same local transaction as the state change, and a separate **message relay** later reads committed outbox rows and publishes them to the broker, retrying until the broker acknowledges [Richardson 2018]. *(§32.5.2)*

Transport Layer Security (TLS) — A protocol that gives two communicating applications **confidentiality** (eavesdroppers cannot read the data), **integrity** (tampering is detected) and **authentication** (at least the server proves its identity, usually with an X.509 certificate). **HTTPS** is HTTP carried over TLS, on TCP port 443 by default. TLS 1.3 is specified in RFC 8446 (2018). *(§7.10.1)*

Trie (prefix tree) — A tree in which each edge is labeled with a character and each node represents the string spelled by the path from the root; nodes that end a stored key are marked. Looking up a key or a prefix of length L takes $O(L)$ steps, independent of how many keys are stored [Fredkin 1960]. *(§17.11.1)*

TrueTime — Google's clock API that returns time as an interval with a guaranteed error bound instead of a single value. `TT.now()` returns $[earliest, latest]$, guaranteed to contain the absolute time at which it was called; `TT.after(t)` is true if t has definitely passed, and `TT.before(t)` is true if t has definitely not arrived. The half-width of the interval is the **uncertainty** ε [Corbett et al. 2012]. *(§28.8.1)*

Trust boundary — A line in a system where the level of trust changes: data or requests crossing it come from a party that the receiving side does not fully control or trust. Examples: the internet-to-load-balancer edge, the browser-to-server connection, a service calling a third-party API, a tenant's code running on shared infrastructure. *(§10.3.2)*

Two generals problem — Two armies camped on opposite hills must attack a city in the valley at the same time; either attacking alone is defeated. Their generals can communicate only by messengers who cross the valley and may be captured. The problem is to design a messaging protocol after which both generals are *certain* that they will attack together. No such protocol with a finite number of messages exists. *(§24.3.3)*

Two-phase commit (2PC) — An atomic commitment protocol in which one node, the **coordinator** (also called the transaction manager), first asks every participant (resource manager) to **prepare**, that is, to make its part of the transaction durable while keeping it invisible and to promise to commit on request; and then, only if every participant voted yes, durably records a **commit** decision and tells all participants to commit. Any no vote or timeout leads to abort [Gray 1978; Bernstein et al. 1987]. *(§28.4.1)*

Two-phase locking (2PL) — A protocol in which each transaction acquires locks (shared **S** for reading, exclusive **X** for writing) before accessing data and, once it releases any lock, acquires no new ones: a *growing* phase followed by a *shrinking* phase. **Strict 2PL** holds all X locks until commit or abort. Any schedule produced by 2PL is serializable [Eswaran et al. 1976]. *(§11.15.1)*

U

Ubiquitous language — A rigorous vocabulary built by developers and domain experts for one model and used consistently in conversation, tests and code, so that each term means exactly one thing within that model [Evans 2003]. *(§34.4.1)*

UDP — The User Datagram Protocol [RFC 768] is a connectionless transport protocol that adds only port numbers, a length and a checksum to IP. Its 8-byte header provides process-to-process delivery of individual **datagrams** with no handshake, no acknowledgments, no retransmission, no ordering, and no flow or congestion control. *(§7.8)*

Unified Modeling Language (UML) — A general-purpose graphical language for specifying, visualizing and documenting software systems, standardized by the Object Management Group (OMG). The current version, UML 2.5.1 (December 2017), defines 14

diagram types divided into *structure* diagrams (what the system is made of) and *behavior* diagrams (what it does over time) [OMG 2017]. *(§38.3.1)*

USE method — For every **resource**, check **U**tilization (the fraction of time the resource was busy, or the fraction of its capacity in use), **S**aturation (the amount of extra work queued because the resource could not service it) and **E**rrors (the count of error events) [Gregg USE]. A resource is any physical or software component with finite capacity: CPUs, memory, disks, network interfaces, controllers and interconnects, but also locks, thread pools, connection pools and file descriptors. *(§23.3.2)*

V

Varint — A variable-length encoding of an unsigned integer in base 128: each byte carries 7 bits of the number, least-significant group first, and the most significant bit (MSB) of each byte is a continuation flag — 1 means "another byte follows" [Protocol Buffers Docs: Encoding]. *(§8.9.2)*

Vector — An ordered list of d numbers, $\mathbf{a} = (a_1, a_2, \ldots, a_d)$, where d is the **dimension**. Geometrically, a vector is an arrow from the origin to a point in d-dimensional space; its **length** (or norm) is $\| \mathbf{a} \| = \sqrt{a_1^2 + \cdots + a_d^2}$. *(§4.9.1)*

Vector clock — With n processes, each process P_i keeps a vector $V_i[1..n]$, initially all zeros. Before each event, P_i increments $V_i[i]$; a send attaches a copy of V_i. On receiving a message carrying V_m, P_i sets $V_i[k] \leftarrow \max(V_i[k], V_m[k])$ for every k, then increments $V_i[i]$. *(§24.11.1)*

Vector database — A data store that indexes high-dimensional numeric vectors (**embeddings** produced by machine-learning models) and answers **approximate nearest neighbor (ANN) search** queries: "the k stored vectors most similar to this query vector", by cosine similarity, dot product or Euclidean distance. *(§12.10)*

Vertical scaling (scaling up) — Increasing the capacity of a single node by giving it more or faster CPU cores, memory, storage or network bandwidth, for example by moving a database from an 8-core to a 64-core machine. *(§18.3.1)*

Virtual memory — A mechanism in which every address a program uses is a **virtual address** that the hardware's **memory-management unit (MMU)** translates, using tables maintained by the kernel, into a **physical address** in RAM, or into a fault if no valid mapping exists. *(§6.6.1)*

Virtual node (vnode) — One of V ring positions belonging to the same physical node, typically obtained by hashing labels such as `db-3#vn17`; also called a **token**. *(§14.8.1)*

Volcano (iterator) model — An execution model in which a query plan is a tree of operators, each implementing the same interface — `open()`, `next()` and `close()` — and each `next()` call pulls one row from the operator's children on demand [Graefe 1994]. *(§11.9.2)*

W

Watermark — A watermark with value t is a statement flowing through the stream: "no more events with event time $\leq t$ are expected." It lets event-time operators decide that a window is complete. A **perfect** watermark is possible only with full knowledge of the sources; real systems use **heuristic** watermarks, such as the maximum event time seen minus a bound δ on out-of-orderness, which can be wrong. Events that arrive with event time behind the watermark are **late data**. *(§29.8.3)*

WebRTC — A set of IETF protocols (overview in RFC 8825 [Alvestrand 2021]) and a W3C browser API [W3C WebRTC] for real-time audio, video and arbitrary data directly between peers. Media travels over SRTP, data over SCTP data channels, both secured by DTLS, typically over UDP; peers discover a working network path using ICE with the help of STUN and TURN servers. *(§9.7.1)*

WebSocket — A protocol, standardized in RFC 6455 [Fette & Melnikov 2011], that starts as an HTTP/1.1 request, *upgrades* the underlying TCP connection, and thereafter carries a full-duplex sequence of framed messages in both directions with 2–14 bytes of framing overhead per frame. URLs use the schemes `ws://` (plain TCP, default port 80) and `wss://` (over TLS, default port 443). *(§9.6.1)*

Wide-column store — A database whose rows are identified by a key, sorted (globally or within a partition) by that key, and contain a sparse, variable set of columns grouped into **column families**; each row may have millions of columns and each cell may hold several timestamped versions. Bigtable, HBase, Cassandra and ScyllaDB belong to this family. *(§12.6.1)*

Write-ahead log (WAL) — An append-only file of **log records** describing every change to the database, written *before* the changed pages themselves reach disk. Each record is identified by a monotonically increasing **log sequence number (LSN)**, usually its byte offset in the log. Also called the redo log (InnoDB) or journal. *(§13.5.1)*

Write-around — Writes go directly to the database and bypass the cache; the cache is populated only by reads (usually cache-aside), and the written key is invalidated or left to expire. *(§20.7.6)*

Write-back — Writes update only the cache and are acknowledged immediately; the cache marks entries **dirty** and flushes them to the database asynchronously, in batches or after a delay. *(§20.7.5)*

Write-through — Every write goes to the cache, which synchronously writes it to the database before acknowledging; the cache is updated and the database is updated as one client-visible operation. *(§20.7.4)*

X

XA — The X/Open Distributed Transaction Processing standard interface between a **transaction manager** (TM, the coordinator) and **resource managers** (RMs: databases, message brokers), with an **application program** (AP) demarcating transactions. Its main calls are `xa_start` and `xa_end` (associate work with a global transaction ID, the XID), `xa_prepare`, `xa_commit`, `xa_rollback` and `xa_recover` (list in-doubt XIDs after a crash) [X/Open 1991]. *(§28.4.6)*

Z

Zero trust — A security model in which network location confers no trust: every request is authenticated, authorized and encrypted based on the identity of the user, the device and the workload, plus contextual signals, regardless of where it originates. NIST SP 800-207 describes it as moving defenses “from static, network-based perimeters to focus on users, assets, and resources” [NIST SP 800-207]. *(§10.10.1)*

Zipf (Zipf-like) distribution — A popularity law in which the item of popularity rank i is requested with probability proportional to $1/i^{s}$, where $s > 0$ is the **skew exponent**. Breslau et al. found that requests to web proxies follow a Zipf-like law with an exponent below 1 and that, as a consequence, hit ratio grows roughly logarithmically with cache size [Breslau et al. 1999]. *(§20.3.3)*

ZooKeeper — A replicated coordination service that exposes a small, file-system-like tree of data nodes (**znodes**) with linearizable writes, FIFO per-client ordering, ephemeral and sequential nodes, and change notifications (**watches**). Applications build locks, leader election, barriers and configuration management on top of these primitives [Hunt et al. 2010]. *(§26.8.1)*